

MT5HedgingEngine

Build a real Python trading platform from scratch — a step-by-step walkthrough of every file, every class, every function, with the actual source code inline.

This book teaches you to build the MT5HedgingEngine project end-to-end. We will create 72 Python files together — 50,959 lines of code, 59 classes (626 methods), and 468 module-level functions — in the dependency-correct order a working developer would build them.

Foreword — who this book is for

This book is for someone who knows a little Python — enough to open a REPL, write a function, and run a script — but has never built anything as large as a real trading platform. By the time you finish, you will have built one yourself, file by file, in the same order the original authors did, and you will understand why every line is the way it is.

The platform you will build does three things. First, it talks to MetaTrader 5 — a broker terminal — through MetaQuotes' official Python API: places orders, reads tick history, monitors open positions. Second, it scrapes prop-firm dashboards (Topstep, FundedNext, Tradovate, TradeOpss) using browser automation, because most prop firms have no public API. Third, it serves a Flask web dashboard where admins, managers and traders can see account state, payouts, and performance. A Tkinter desktop window ties the trader-side pieces together.

How the book is organised:

- **Chapter 1** walks you through getting a working Python development environment — installing Python, creating a virtual environment, the project folder layout, the requirements.txt file, the .env file, and the external pieces (MetaTrader 5 terminal, Chrome) you need on your machine.
- **Chapter 2** is a primer on the Python language features the rest of the book uses. Read it cover-to-cover or skim for unfamiliar names — you can come back to it whenever a later chapter mentions something you do not recognise.
- **Chapters 3 through 12** are the build itself, split into ten *phases*. Phase 1 is the boring foundation (settings, helpers); Phase 10 is the deployment scripts that ship the finished thing. Each phase opens with a one-page intro that tells you what you are about to build and why it comes next, then walks each file in order. Every class, every method, and every function gets four things: the source code itself, an explanation of which Python concepts it uses and why those were the right tools, a numbered step-by-step walkthrough of the body, and the function's own docstring where one exists.
- The **frontend chapter** picks up where the Python phases leave off and walks the user-facing layer: every Jinja template under dashboard/templates/, the static CSS under dashboard/static/, and the embedded JavaScript that gives each page its interactivity. Every template gets a stat block (LOC, expressions, forms, AJAX patterns), a one-line purpose, and the first ~30 lines of actual markup so you can read along.
- The **closing chapter** is a glossary plus a concept index — if you want to see real working examples of, say, decorators or generators or context managers, the index lists every module that uses each one.

How to use the book. Type the code in. Don't just read it. The whole point of building a project from scratch is that the muscle memory and the small surprises (a missing import, a typo in a config key, a mis-named column) are what teach you. The pages give you the destination; you take the steps. If something does not work, the right reaction is curiosity, not frustration — every error message is a piece of the language teaching you something specific.

Table of contents

Foreword — who this book is for

Chapter 1 — Project setup

Chapter 2 — Python concepts: a primer

- Modules and packages
- Classes and objects
- Inheritance
- Decorators
- Type hints
- Async / await
- Generators and yield
- Context managers (with-statements)
- Dataclasses
- Exceptions
- Comprehensions
- Properties
- f-strings
- Lambdas
- Dunder methods
- @classmethod and @staticmethod
- Module-level state

Chapter 3 — Phase 1 — Foundations: configuration and shared helpers

config/settings.py (32 loc, 0c, 0f)
config/production.py (145 loc, 4c, 1f)
config/hierarchy.py (452 loc, 0c, 24f)
utils/__init__.py (1 loc, 0c, 0f)
utils/data_processor.py (1349 loc, 0c, 14f)

Chapter 4 — Phase 2 — Persistence: the database layer

alembic/env.py (85 loc, 0c, 2f)
alembic/versions/44e368d8bfce_initial_schema.py (266 loc, 0c, 2f)
alembic/versions/5b29b54b57fa_add_firm_billing_column.py (31 loc, 0c, 2f)
dashboard/database.py (1940 loc, 2c, 90f)
dashboard/db.py (44 loc, 0c, 1f)
dashboard/models.py (330 loc, 18c, 0f)

Chapter 5 — Phase 3 — Talking to MetaTrader 5

connectors/mt5_connector.py (198 loc, 1c, 0f)
connectors/mt5_automator.py (2228 loc, 1c, 2f)

Chapter 6 — Phase 4 — Indicators and strategies

trader_companion/signals/__init__.py (2 loc, 0c, 0f)
trader_companion/signals/sma.py (27 loc, 0c, 1f)
trader_companion/signals/ema.py (27 loc, 0c, 1f)
trader_companion/signals/rsi.py (150 loc, 0c, 4f)
trader_companion/signals/macd.py (42 loc, 0c, 1f)
trader_companion/signals/bb.py (38 loc, 0c, 1f)
trader_companion/signals/atr.py (30 loc, 0c, 1f)
trader_companion/signals/adx.py (40 loc, 0c, 1f)
trader_companion/signals/dmi.py (42 loc, 0c, 1f)
trader_companion/signals/ccl.py (40 loc, 0c, 1f)
trader_companion/signals/momentum.py (33 loc, 0c, 1f)

trader_companion/signals/roc.py (33 loc, 0c, 1f)
trader_companion/signals/obv.py (35 loc, 0c, 1f)
trader_companion/signals/mfi.py (43 loc, 0c, 1f)
trader_companion/signals/stochastic.py (42 loc, 0c, 1f)
trader_companion/signals/supertrend.py (44 loc, 0c, 1f)
trader_companion/signals/sar.py (56 loc, 0c, 1f)
trader_companion/signals/tsi.py (39 loc, 0c, 1f)
trader_companion/signals/wr.py (38 loc, 0c, 1f)
trader_companion/signals/cmo.py (7 loc, 0c, 1f)
trader_companion/signals/coppock_curve.py (7 loc, 0c, 1f)
trader_companion/signals/donchian_channel.py (7 loc, 0c, 1f)
trader_companion/signals/elder_ray.py (7 loc, 0c, 1f)
trader_companion/signals/fractal.py (7 loc, 0c, 1f)
trader_companion/signals/gator_oscillator.py (7 loc, 0c, 1f)
trader_companion/signals/keltner_channel.py (7 loc, 0c, 1f)
trader_companion/signals/price_channel.py (7 loc, 0c, 1f)
trader_companion/signals/ultimate_oscillator.py (7 loc, 0c, 1f)
trader_companion/signals/vortex.py (7 loc, 0c, 1f)
trader_companion/strategies/__init__.py (2 loc, 0c, 0f)
trader_companion/strategies/rsi_overbought_oversold.py (22 loc, 0c, 1f)

Chapter 7 — Phase 5 — The trading engine

trader_companion/mt5_trading.py (2485 loc, 1c, 2f)
trader_companion/mt5_comment_parser.py (869 loc, 5c, 4f)
trader_companion/trade_limit_manager.py (388 loc, 2c, 4f)
trader_companion/hedge_protector.py (576 loc, 1c, 0f)

Chapter 8 — Phase 6 — Prop-firm dashboards

trader_companion/prop_firm_scrapers.py (1893 loc, 6c, 5f)
trader_companion/fundednext.py (1386 loc, 1c, 3f)
trader_companion/topstepx.py (4420 loc, 1c, 3f)
trader_companion/tradovate.py (3334 loc, 1c, 0f)
trader_companion/prop_firm_manager.py (2124 loc, 1c, 0f)

Chapter 9 — Phase 7 — Pushing data to the dashboard

trader_companion/push_data.py (194 loc, 0c, 4f)
trader_companion/mt5_dashboard_sync.py (840 loc, 1c, 1f)

Chapter 10 — Phase 8 — Desktop GUI

trader_companion/broker_selection.py (210 loc, 5c, 0f)
trader_companion/trader_app.py (8659 loc, 2c, 3f)

Chapter 11 — Phase 9 — Web dashboard

dashboard/email_service.py (260 loc, 0c, 4f)
dashboard/notes_service.py (76 loc, 0c, 3f)
dashboard/manage_api_keys.py (231 loc, 0c, 9f)
dashboard/api_client.py (201 loc, 1c, 0f)
dashboard/utils/sheet_helper.py (253 loc, 0c, 6f)
dashboard/utils/trade_matcher.py (361 loc, 1c, 0f)
dashboard/calc_like_sheet.py (111 loc, 0c, 3f)
dashboard/phase_manager.py (242 loc, 1c, 0f)
dashboard/watermark_service.py (688 loc, 0c, 22f)
dashboard/financial_overview.py (1718 loc, 1c, 24f)
dashboard/scheduler.py (605 loc, 0c, 12f)
dashboard/app.py (10470 loc, 2c, 172f)

Chapter 12 — Phase 10 — Deployment

wsgi.py (109 loc, 0c, 2f)

unicorn.conf.py (173 loc, 0c, 11f)
build.py (87 loc, 0c, 1f)

Chapter 13 — The frontend: templates, CSS, and the JavaScript glue

index.html (8,748 loc, 0 forms)
super_admin.html (2,721 loc, 0 forms)
quality_dashboard.html (1,731 loc, 0 forms)
financial_overview.html (1,177 loc, 1 form)
trader_dashboard.html (973 loc, 0 forms)
admin_dashboard.html (745 loc, 0 forms)
hierarchy.html (729 loc, 0 forms)
login.html (610 loc, 0 forms)
payout_history.html (554 loc, 1 form)
client_performance.html (551 loc, 0 forms)
trader_performance.html (418 loc, 0 forms)
change_password.html (397 loc, 0 forms)
client_management.html (348 loc, 0 forms)
maintenance.html (259 loc, 0 forms)
500.html (250 loc, 0 forms)

Chapter 14 — Glossary & concept index

Chapter 1 — Project setup

Before we write a single line of project code, we need a working development environment. This chapter is all setup — by the end of it you will have an empty project folder with Python, a virtual environment, the dependencies installed, and the external tools (MT5 terminal, Chrome) ready to go.

1.1 Install Python

Download Python 3.10 or newer from python.org/downloads. On Windows, tick the *Add Python to PATH* checkbox during install — without it, the `python` command will not be on your terminal's path. Verify the install by opening a fresh terminal and running:

```
python --version
```

You should see Python 3.10.x or newer. If you see 'python' is not recognised, the PATH did not get set — re-run the installer and tick the box, or add the Python folder to your PATH manually.

1.2 Create the project folder

Pick a place for the project and create the top-level folder. Everything we build lives inside this folder.

```
mkdir MT5HedgingEngine
cd MT5HedgingEngine
```

1.3 Create a virtual environment

A virtual environment is an isolated Python install that belongs to this project. It keeps the project's dependencies from clashing with anything else on your machine, and it is the single most useful habit a Python developer can have. Create and activate one:

```
python -m venv .venv
# Windows:
.venv\Scripts\activate
# macOS / Linux:
source .venv/bin/activate
```

After activation your terminal prompt should start with `(.venv)`. Every `pip` install from now on lands in this isolated environment, not in the system Python.

1.4 Create the folder layout

We will be building the following structure. Create the empty folders now — the files inside will be filled in chapter by chapter.

```
MT5HedgingEngine/
  config/
  utils/
  connectors/
  alembic/
  versions/
  trader_companion/
  signals/
  strategies/
```

```
utils/  
dashboard/  
utils/  
static/  
templates/
```

Python only treats a folder as an importable *package* when it contains an `__init__.py` file (it can be empty). For each of the folders above except `static/` and `templates/`, create an empty `__init__.py`:

```
# Windows PowerShell  
New-Item config/__init__.py, utils/__init__.py, connectors/__init__.py -ItemType File  
# macOS / Linux  
touch config/__init__.py utils/__init__.py connectors/__init__.py
```

1.5 Write requirements.txt

`requirements.txt` is the manifest of every third-party package the project depends on. Create it at the root with this content (the exact versions can be loosened later, but pinned versions reproduce the exact environment the original project ran on):

```
Flask  
Flask-Login  
Flask-Limiter  
SQLAlchemy  
alembic  
psycopg2-binary  
requests  
pandas  
numpy  
MetaTrader5  
selenium  
apscheduler  
python-dotenv  
werkzeug  
openpyxl  
pytz  
psutil  
websocket-client  
gunicorn  
reportlab
```

Install them all in one go (this will take a minute or two):

```
pip install -r requirements.txt
```

1.6 Create the .env file

Twelve-factor configuration says that anything that differs between dev and production should come from environment variables, not code. We use a `.env` file (loaded by `python-dotenv`) to keep those values out of git. Create `.env` in the project root with at least these keys:

```
FLASK_SECRET_KEY=change-me-to-a-long-random-string  
DATABASE_URL=sqlite:///dashboard.db  
MT5_LOGIN=  
MT5_PASSWORD=  
MT5_SERVER=  
GOOGLE_SHEETS_CREDS=  
SMTP_HOST=
```

```
SMTP_USER=  
SMTP_PASSWORD=
```

Add `.env` to your `.gitignore` **before** the first commit — it is going to hold real secrets later.

1.7 Install the MetaTrader 5 terminal

The MetaTrader5 Python package talks to a real terminal that has to be installed and logged in. Download it from your broker (or from *metatrader5.com*), install it, log in once with your account, and leave it running. The Python package on its own does nothing — it is a client to the terminal you just installed.

1.8 Install Chrome (for Selenium)

The prop-firm scrapers in Phase 6 drive a real Chrome browser. Install Chrome if you do not already have it. Modern Selenium ships with Selenium Manager, which downloads the matching chromedriver automatically — no extra install needed.

1.9 Verify the setup

Quick sanity check. From the project root, with the venv active, run:

```
python -c "import flask, sqlalchemy, pandas, MetaTrader5, selenium; print('OK')"
```

If you see OK, you are ready. If any of those imports fails, fix it before moving on — the rest of the book depends on them.

Chapter 2 — Python concepts: a primer

A short tour of the language features you will see across the rest of the book. Each section is brief by design — the goal is to give you a hook the real code can hang on. When a later chapter mentions a concept you do not remember, flip back here.

Modules and packages

Every `.py` file in Python is a *module*. A folder with an `__init__.py` file is a *package*. When you write from `trader_companion import mt5_trading`, Python finds the module file, executes it from top to bottom, and exposes the names defined inside (functions, classes, constants) under that module. Imports run only the first time — subsequent imports get the cached module object. This is why placing side-effect-heavy code at module scope is risky: it runs whenever the module is imported anywhere in the program.

Classes and objects

A class is a blueprint for objects. Calling the class — for example `Trade(symbol='EURUSD', volume=0.1)` — runs the special `__init__` method to construct an instance. Methods defined inside the class take `self` as their first parameter; Python passes the instance in automatically when you call `trade.close()`. Attributes set on `self` are per-instance; attributes set at class scope are shared across instances unless overridden.

Inheritance

A class can inherit from one (or many) base classes by listing them in parentheses: `class Topstep(PropFirm):`. The subclass gets every method and attribute from its parent for free, but can override any of them. `super().method()` calls the parent's version — useful when you want to extend behaviour rather than replace it. Inheritance is one form of code reuse; composition (holding another object as an attribute) is often simpler and more flexible.

Decorators

A decorator is a function that wraps another function (or class) to add behaviour. Writing `@app.route('/login')` above a function is shorthand for `login = app.route('/login')(login)`. Decorators are how Flask attaches URL handlers, how `@property` turns a method into an attribute lookup, and how `@staticmethod` tells Python that a method doesn't need `self`. Decorators stack: the one closest to the function runs first.

Type hints

Python is dynamically typed at runtime, but you can annotate parameters and returns: `def fetch(account: int) -> Trade:`. These annotations are not enforced — they exist for static checkers like `mypy/pyright`, IDE auto-complete, and human readers. The **typing** module supplies generics like `list[int]`, `Optional[str]`, and `dict[str, Any]`. From Python 3.10 onwards you can write `int | None` in place of `Optional[int]`.

Async / await

async def declares a coroutine — a function that returns an awaitable rather than a value. Inside it, **await** hands control back to the event loop while it waits for I/O. This lets a single thread juggle many slow operations (HTTP requests, websocket reads) concurrently without blocking. You launch coroutines with `asyncio.run(main())` or schedule them as tasks with `asyncio.create_task(coro)`.

Generators and yield

A function that uses **yield** instead of **return** becomes a generator: each call returns an iterator that produces values one at a time, on demand. This is how Python streams data without holding it all in memory. `for row in read_csv(path)` can iterate a million-row file lazily. `yield` from delegates to another iterator; combined with coroutines, generators were the foundation for `async/await`.

Context managers (with-statements)

with `open('file')` as `f`: guarantees that `f.close()` runs whether the block succeeds or raises. The object's `__enter__` runs at the start, `__exit__` at the end. Context managers express the acquire/release pattern: locks, database connections, transactions, temp files, MetaTrader-5 sessions. The `contextlib.contextmanager` decorator turns a generator into a context manager when you don't want to write the dunder methods by hand.

Dataclasses

A `@dataclass` auto-generates `__init__`, `__repr__`, and `__eq__` from the class's annotated fields. It removes the boilerplate from "plain data" objects. Use `field(default_factory=list)` for mutable defaults (never write `= []` at class scope — it leaks state across instances). `frozen=True` makes instances hashable and immutable.

Exceptions

Errors in Python travel up the call stack until something catches them. `try/except` catches; `raise` throws; `finally` runs cleanup regardless. Catch the narrowest exception you can — `except Exception` as the only handler will swallow `KeyboardInterrupt` on older code paths and hide real bugs. Define your own exception types by subclassing `Exception` when callers need to distinguish failure modes.

Comprehensions

`[x*2 for x in xs if x > 0]` builds a list in one expression. There are list, set, dict, and generator versions. Generator comprehensions use parentheses: `sum(x*2 for x in xs)` avoids materialising the intermediate list. Comprehensions are usually clearer than `map/filter` and faster than the equivalent `for-loop` because the list is sized once.

Properties

`@property` turns a method into an attribute lookup. `account.equity` can run a calculation each time it is accessed without exposing the call syntax. A matching `@equity.setter` lets you intercept assignment. Properties are how you migrate a public attribute to a computed value without breaking callers.

f-strings

`f'Account {a.id} balance ${a.balance:,.2f}'` is Python's modern interpolation syntax. The format spec after the colon (`,.2f`, `%`, `:<20`) controls width, precision, and alignment. f-strings are compiled into the bytecode at parse time, so they are also the fastest formatting option.

Lambdas

A **lambda** is a single-expression anonymous function. `sorted(trades, key=lambda t: t.profit)` avoids naming a one-line helper. Anything more than a single expression should be a regular `def` — lambdas lose statements, type hints, and a useful traceback name.

Dunder methods

Methods with double-underscore names (`__init__`, `__repr__`, `__eq__`, `__len__`) hook into Python's syntax. Defining `__len__` makes `len(obj)` work; `__iter__` makes it usable in `for`; `__getitem__` enables `obj[key]`. They are how user-defined classes behave like built-in types.

@classmethod and @staticmethod

`@classmethod` receives the class itself as its first argument (`cls` by convention). It is the standard way to define alternate constructors: `Trade.from_mt5_order(...)`. `@staticmethod` takes neither `self` nor `cls` — use it when a function logically belongs to the class's namespace but doesn't need access to any state.

Module-level state

Variables assigned at module top-level are global to that module. They are convenient for caches and singletons, but they can make testing harder because every import shares the same instance. Prefer passing values explicitly; reach for module-level state only when there is genuinely one of something (a database engine, a logger).

Chapter 3 — Phase 1 — Foundations: configuration and shared helpers

Every project starts with the boring-but-essential plumbing: a place for settings (the values that change between dev and production), and a small kit of helpers shared by everything else. We build these first because every later phase imports them. If you have not yet created a `config/` and `utils/` folder under your project root, do that now — Python only treats a folder as a package when it contains an `__init__.py` file, even an empty one.

Files we build in this phase, in order:

- `config/settings.py`
- `config/production.py`
- `config/hierarchy.py`
- `utils/__init__.py`
- `utils/data_processor.py`

Python concepts you'll meet in this phase

• f-strings (2) · Module-level state (2) · Context managers (with-statements) (2) · Comprehensions (2) · Exceptions (2) · Properties (1) · Classes and objects (1) · Decorators (1)

Each is explained in Chapter 2 (the primer). The number in parentheses is how many of this phase's files use that concept.

Step 3.1 — Build `config/settings.py`

What to do. Create the file `config/settings.py` in your project root and paste in the code shown in this step. The file is **32** lines long; we walk through it piece by piece below.

Depends on. This file imports from internal modules built in earlier steps: `config`. If any of those imports fail, jump back to the step that introduced them.

What this file is for. Centralised configuration. Reads from environment variables with sensible local-development defaults.

`config/settings.py`

32 loc · 0 classes · 0 functions · 2 imports

Centralised configuration. Reads from environment variables with sensible local-development defaults. across **32** lines.

Imports — what each one is for

```
import os
```

Why imported. Operating-system interface. Path manipulation, environment variables, working-directory operations, exit codes, and thin wrappers around POSIX/Windows syscalls.

Used in this file. `os.getenv`

Example. `os.path.join(root, 'subdir', 'file.txt')` # joins with the OS-correct separator

Other common scenarios.

- `os.environ.get('API_KEY')` to read environment variables
- `os.makedirs(path, exist_ok=True)` to create nested directories
- `os.listdir(path)` to list a directory's contents
- `os.remove(path)` / `os.rename(src, dst)` to delete or move a file
- `os.getcwd()` / `os.chdir(path)` to inspect or change the working dir

```
from config.hierarchy import get_client_profile
```

Why imported. Internal package — application settings and the firm hierarchy.

Used in this file. `get_client_profile`

Module constants

Each line below is followed by a plain-English note explaining what it does and why it is written that way.

```
CLIENT_NAME = os.getenv('CLIENT_NAME', 'Chris')
```

Reads `CLIENT_NAME` from the environment, defaulting to 'Chris' when not set — overridable per environment without a code change.

```
MT5_LOGIN = int(os.getenv('MT5_LOGIN', 3053752))
```

Reads `MT5_LOGIN` from the environment and converts it to int (env vars are always strings). Falls back to 3053752 when unset.

```
MT5_PASSWORD = os.getenv('MT5_PASSWORD', 'Blueedge1!')
```

Reads `MT5_PASSWORD` from the environment, defaulting to 'Blueedge1!' when not set — overridable per environment without a code change.

```
MT5_SERVER = os.getenv('MT5_SERVER', 'PlexyTrade-Server01')
```

Reads `MT5_SERVER` from the environment, defaulting to 'PlexyTrade-Server01' when not set — overridable per environment without a code change.

```
MT5_TERMINAL = os.getenv('MT5_TERMINAL', 'MetaTrader 5 (MetaTrader 5)')
```

Reads `MT5_TERMINAL` from the environment, defaulting to 'MetaTrader 5 (MetaTrader 5)' when not set — overridable per environment without a code change.

```
MT5_SYMBOL = os.getenv('MT5_SYMBOL', 'USTECH')
```

Reads MT5_SYMBOL from the environment, defaulting to 'USTECH' when not set — overridable per environment without a code change.

```
SHEET_URL = 'https://docs.google.com/spreadsheets/d/1vtuGcTe8ys44wHCJGJr6VoImeh8q0beaKkZMt0hd3VU/edit?usp=
```

URL constant pointing to an external resource. Hard-coded at load time; change here, not in callers.

```
ENABLE_GPT5_MINI = os.getenv('ENABLE_GPT5_MINI', 'true').lower() == 'true'
```

Boolean feature flag. Reads ENABLE_GPT5_MINI from the environment, lowercases it, and compares to 'true' — producing a real Python bool. Defaults to 'true'.

Step 3.2 — Build config/production.py

What to do. Create the file config/production.py in your project root and paste in the code shown in this step. The file is **145** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from requirements.txt.

What this file is for. Production-only overrides for settings.py.

config/production.py

145 loc · 4 classes · 1 functions · 2 imports

Production-only overrides for settings.py. It defines **4** classes and **1** module-level function, across **145** lines. Module docstring: *Production Configuration for MT5 Hedging Dashboard*

Module docstring

Production Configuration for MT5 Hedging Dashboard

===== This module provides
environment-specific configurations. Use environment variables to override settings in production.

Python concepts demonstrated in this module

• Classes and objects • Decorators • f-strings • Inheritance • Properties

Imports — what each one is for

```
import os
```

Why imported. Operating-system interface. Path manipulation, environment variables, working-directory operations, exit codes, and thin wrappers around POSIX/Windows syscalls.

Used in this file. os.getenv

Example. os.path.join(root, 'subdir', 'file.txt') # joins with the OS-correct separator

Other common scenarios.

- os.environ.get('API_KEY') to read environment variables

- `os.makedirs(path, exist_ok=True)` to create nested directories
- `os.listdir(path)` to list a directory's contents
- `os.remove(path)` / `os.rename(src, dst)` to delete or move a file
- `os.getcwd()` / `os.chdir(path)` to inspect or change the working dir

```
from datetime import timedelta
```

Why imported. Dates, times, durations. The fundamental temporal types: date, time, datetime, timedelta, timezone.

Used in this file. timedelta

Example. `datetime.now() - timedelta(days=7)` # one week ago

Other common scenarios.

- `datetime.strptime(s, '%Y-%m-%d')` parses a string
- `datetime.strftime(fmt)` formats one
- `datetime.fromtimestamp(n)` converts a Unix epoch
- `datetime.utcnow()` for UTC (better: `datetime.now(timezone.utc)`)

Classes

```
class Config
```

Base configuration class.

```
SECRET_KEY = os.getenv('FLASK_SECRET_KEY', 'change-this-in-production')
DEBUG = False
TESTING = False
APP_NAME = 'MT5 Hedging Dashboard'
APP_VERSION = '1.0.1'
SESSION_TYPE = 'filesystem'
SESSION_PERMANENT = True
PERMANENT_SESSION_LIFETIME = timedelta(hours=24)
SESSION_COOKIE_SECURE = True
SESSION_COOKIE_HTTPONLY = True
SESSION_COOKIE_SAMESITE = 'Lax'
DATABASE_TYPE = os.getenv('DATABASE_TYPE', 'sqlite')
SQLITE_PATH = os.getenv('SQLITE_PATH', 'dashboard/dashboard.db')
POSTGRES_HOST = os.getenv('POSTGRES_HOST', 'localhost')
POSTGRES_PORT = os.getenv('POSTGRES_PORT', '5432')
POSTGRES_DB = os.getenv('POSTGRES_DB', 'mt5_dashboard')
POSTGRES_USER = os.getenv('POSTGRES_USER', 'dashboard_user')
POSTGRES_PASSWORD = os.getenv('POSTGRES_PASSWORD', '')
REDIS_URL = os.getenv('REDIS_URL', 'redis://localhost:6379/0')
RATELIMIT_ENABLED = True
RATELIMIT_STORAGE_URL = os.getenv('RATELIMIT_STORAGE_URL', 'memory://')
RATELIMIT_DEFAULT = '10000 per day;2000 per hour'
RATELIMIT_HEADERS_ENABLED = True
CACHE_TYPE = os.getenv('CACHE_TYPE', 'simple')
CACHE_DEFAULT_TIMEOUT = 300
CACHE_REDIS_URL = os.getenv('CACHE_REDIS_URL', 'redis://localhost:6379/1')
SMTP_SERVER = os.getenv('SMTP_SERVER', 'smtp.gmail.com')
SMTP_PORT = int(os.getenv('SMTP_PORT', '587'))
SMTP_USERNAME = os.getenv('SMTP_USERNAME', '')
```

```

SMTP_PASSWORD = os.getenv('SMTP_PASSWORD', '')
SMTP_USE_TLS = True
EMAIL_ENABLED = os.getenv('EMAIL_ENABLED', 'false').lower() == 'true'
PASSWORD_HASH_ITERATIONS = 100000
API_KEY_LENGTH = 32
SESSION_TOKEN_LENGTH = 64
MAX_LOGIN_ATTEMPTS = 5
LOCKOUT_DURATION = timedelta(minutes=15)
LOG_LEVEL = os.getenv('LOG_LEVEL', 'INFO')
LOG_FORMAT = '%(asctime)s - %(name)s - %(levelname)s - %(message)s'
LOG_FILE = os.getenv('LOG_FILE', 'logs/dashboard.log')
CORS_ORIGINS = os.getenv('CORS_ORIGINS', '*').split(',')
MAX_CONTENT_LENGTH = 16 * 1024 * 1024
ENABLE_GPT5_MINI = os.getenv('ENABLE_GPT5_MINI', 'true').lower() == 'true'

```

Code (copy-paste safe)

```

class Config:
    """Base configuration class."""

    # Flask Core
    SECRET_KEY = os.getenv('FLASK_SECRET_KEY', 'change-this-in-production')
    DEBUG = False
    TESTING = False

    # Application
    APP_NAME = 'MT5 Hedging Dashboard'
    APP_VERSION = '1.0.1'

    # Session Configuration
    SESSION_TYPE = 'filesystem' # Options: 'filesystem', 'redis', 'sqlalchemy'
    SESSION_PERMANENT = True
    PERMANENT_SESSION_LIFETIME = timedelta(hours=24)
    SESSION_COOKIE_SECURE = True # HTTPS only
    SESSION_COOKIE_HTTPONLY = True
    SESSION_COOKIE_SAMESITE = 'Lax'

    # Database
    DATABASE_TYPE = os.getenv('DATABASE_TYPE', 'sqlite') # 'sqlite' or 'postgresql'

    # SQLite (Development/Small deployments)
    SQLITE_PATH = os.getenv('SQLITE_PATH', 'dashboard/dashboard.db')

    # PostgreSQL (Production)
    POSTGRES_HOST = os.getenv('POSTGRES_HOST', 'localhost')
    POSTGRES_PORT = os.getenv('POSTGRES_PORT', '5432')
    POSTGRES_DB = os.getenv('POSTGRES_DB', 'mt5_dashboard')
    POSTGRES_USER = os.getenv('POSTGRES_USER', 'dashboard_user')
    POSTGRES_PASSWORD = os.getenv('POSTGRES_PASSWORD', '')

    @property
    def DATABASE_URL(self):
        """Generate database URL based on configuration."""
        if self.DATABASE_TYPE == 'postgresql':
            return f"postgresql://{self.POSTGRES_USER}:{self.POSTGRES_PASSWORD}@{self.POSTGRES_HOST}:{self.POSTGRES_PORT}/{self.POSTGRES_DB}"
        return f"sqlite:/// {self.SQLITE_PATH}"

    # Redis Configuration (for caching and rate limiting)
    REDIS_URL = os.getenv('REDIS_URL', 'redis://localhost:6379/0')

    # Rate Limiting

```



```

RATELIMIT_ENABLED = True
RATELIMIT_STORAGE_URL = os.getenv('RATELIMIT_STORAGE_URL', 'memory://')
RATELIMIT_DEFAULT = "10000 per day;2000 per hour"
RATELIMIT_HEADERS_ENABLED = True

# Caching
CACHE_TYPE = os.getenv('CACHE_TYPE', 'simple') # 'simple', 'redis', 'filesystem'
CACHE_DEFAULT_TIMEOUT = 300 # 5 minutes
CACHE_REDIS_URL = os.getenv('CACHE_REDIS_URL', 'redis://localhost:6379/1')

# Email (SMTP)
SMTP_SERVER = os.getenv('SMTP_SERVER', 'smtp.gmail.com')
SMTP_PORT = int(os.getenv('SMTP_PORT', '587'))
SMTP_USERNAME = os.getenv('SMTP_USERNAME', '')
SMTP_PASSWORD = os.getenv('SMTP_PASSWORD', '')
SMTP_USE_TLS = True
EMAIL_ENABLED = os.getenv('EMAIL_ENABLED', 'false').lower() == 'true'

# Security
PASSWORD_HASH_ITERATIONS = 100000
API_KEY_LENGTH = 32
SESSION_TOKEN_LENGTH = 64
MAX_LOGIN_ATTEMPTS = 5
LOCKOUT_DURATION = timedelta(minutes=15)

# Logging
LOG_LEVEL = os.getenv('LOG_LEVEL', 'INFO')
LOG_FORMAT = '%(asctime)s - %(name)s - %(levelname)s - %(message)s'
LOG_FILE = os.getenv('LOG_FILE', 'logs/dashboard.log')

# CORS (Cross-Origin Resource Sharing)
CORS_ORIGINS = os.getenv('CORS_ORIGINS', '*').split(',')

# File Upload Limits
MAX_CONTENT_LENGTH = 16 * 1024 * 1024 # 16 MB

# Feature flags
# Toggle to enable GPT-5 mini for clients. Default true; can be overridden via ENV.
ENABLE_GPT5_MINI = os.getenv('ENABLE_GPT5_MINI', 'true').lower() == 'true'

```

Method: Config.DATABASE_URL

```

@property
def DATABASE_URL(self)

```

Generate database URL based on configuration.

Why these Python concepts were used

- **@property** — Wrapped in **@property** so callers access the value with attribute syntax (obj.x) instead of a method call. Lets the implementation compute or validate the value lazily without changing the call site.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **if self.DATABASE_TYPE == 'postgresql':** branches conditionally.

2. `return f'sqlite:///{self.SQLITE_PATH}'`.

Code (copy-paste safe)

```
def DATABASE_URL(self):
    """Generate database URL based on configuration."""
    if self.DATABASE_TYPE == 'postgresql':
        return f"postgresql://{self.POSTGRES_USER}:{self.POSTGRES_PASSWORD}@{self.POSTGRES_HOST}:{self.POSTGRES_PORT}"
    return f"sqlite:///self.SQLITE_PATH"
```

```
class DevelopmentConfig(Config)
```

Development configuration.

```
DEBUG = True
SESSION_COOKIE_SECURE = False
DATABASE_TYPE = 'sqlite'
RATELIMIT_STORAGE_URL = 'memory://'
CACHE_TYPE = 'simple'
LOG_LEVEL = 'DEBUG'
```

Code (copy-paste safe)

```
class DevelopmentConfig(Config):
    """Development configuration."""
    DEBUG = True
    SESSION_COOKIE_SECURE = False
    DATABASE_TYPE = 'sqlite'
    RATELIMIT_STORAGE_URL = 'memory://'
    CACHE_TYPE = 'simple'
    LOG_LEVEL = 'DEBUG'
```

```
class ProductionConfig(Config)
```

Production configuration.

```
DEBUG = False
SESSION_COOKIE_SECURE = True
DATABASE_TYPE = os.getenv('DATABASE_TYPE', 'postgresql')
RATELIMIT_STORAGE_URL = os.getenv('REDIS_URL', 'redis://localhost:6379/0')
CACHE_TYPE = 'redis'
LOG_LEVEL = 'WARNING'
SESSION_COOKIE_SECURE = True
SESSION_COOKIE_HTTPONLY = True
```

Code (copy-paste safe)

```
class ProductionConfig(Config):
    """Production configuration."""
    DEBUG = False
    SESSION_COOKIE_SECURE = True
    DATABASE_TYPE = os.getenv('DATABASE_TYPE', 'postgresql')
    RATELIMIT_STORAGE_URL = os.getenv('REDIS_URL', 'redis://localhost:6379/0')
    CACHE_TYPE = 'redis'
    LOG_LEVEL = 'WARNING'

    # Enhanced security for production
    SESSION_COOKIE_SECURE = True
    SESSION_COOKIE_HTTPONLY = True
```

```
class TestingConfig(Config)

    Testing configuration.

    TESTING = True
    DEBUG = True
    DATABASE_TYPE = 'sqlite'
    SQLITE_PATH = ':memory:'
    RATELIMIT_ENABLED = False
    EMAIL_ENABLED = False
```

Code (copy-paste safe)

```
class TestingConfig(Config):
    """Testing configuration."""
    TESTING = True
    DEBUG = True
    DATABASE_TYPE = 'sqlite'
    SQLITE_PATH = ':memory:'
    RATELIMIT_ENABLED = False
    EMAIL_ENABLED = False
```

Functions

```
def get_config(env=None)

    Get configuration based on environment.
```

Step by step (top-level body)

1. **if** env is None: branches conditionally.
2. **return** config.get(env, config['default'])().

Code (copy-paste safe)

```
def get_config(env=None):
    """Get configuration based on environment."""
    if env is None:
        env = os.getenv('FLASK_ENV', 'development')
    return config.get(env, config['default'])()
```

Step 3.3 — Build config/hierarchy.py

What to do. Create the file config/hierarchy.py in your project root and paste in the code shown in this step. The file is **452** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from requirements.txt.

What this file is for. Loads and validates the firm hierarchy (admin / manager / trader) tree out of hierarchy.json.

config/hierarchy.py

452 loc · 0 classes · 24 functions · 2 imports

Loads and validates the firm hierarchy (admin / manager / trader) tree out of hierarchy.json. It defines **24** module-level functions, across **452** lines.

Python concepts demonstrated in this module

• Comprehensions • Context managers (with-statements) • Exceptions • Module-level state

Imports — what each one is for

```
import json
```

Why imported. JSON encoding and decoding — the standard wire format for config files, API payloads, and inter-process messages.

Used in this file. json.dump, json.load

Example. json.loads(response.text) # parse a JSON string to dict

Other common scenarios.

- json.dumps(obj, indent=2) for pretty-printing
- json.dump(obj, file) / json.load(file) for file I/O
- default=str handles non-serialisable types like datetime
- object_hook= customises decoding (e.g., to dataclass instances)

```
import os
```

Why imported. Operating-system interface. Path manipulation, environment variables, working-directory operations, exit codes, and thin wrappers around POSIX/Windows syscalls.

Used in this file. os.makedirs, os.path, os.path.dirname, os.path.exists, os.path.join

Example. os.path.join(root, 'subdir', 'file.txt') # joins with the OS-correct separator

Other common scenarios.

- os.environ.get('API_KEY') to read environment variables
- os.makedirs(path, exist_ok=True) to create nested directories
- os.listdir(path) to list a directory's contents
- os.remove(path) / os.rename(src, dst) to delete or move a file
- os.getcwd() / os.chdir(path) to inspect or change the working dir

Module constants

Each line below is followed by a plain-English note explaining what it does and why it is written that way.

```
BASE_DIR = os.path.dirname(__file__)
```

Filesystem path resolved relative to the source file at import time — portable across machines.

```
RESTRUCTURED_FILE = os.path.join(BASE_DIR, 'hierarchy_restructured.json')
```

Module-level constant — bound once at import time and referenced from the functions and classes below.

```
LEGACY_FILE = os.path.join(BASE_DIR, 'hierarchy.json')
```

Module-level constant — bound once at import time and referenced from the functions and classes below.

```
HIERARCHY_FILE = RESTRUCTURED_FILE if os.path.exists(RESTRUCTURED_FILE) else LEGACY_FILE
```

Module-level constant — bound once at import time and referenced from the functions and classes below.

```
SYSTEM_HIERARCHY = load_hierarchy()
```

Module-level constant — bound once at import time and referenced from the functions and classes below.

Functions

```
def load_hierarchy():
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **dict comprehension** — Builds a dict in one expression (`{k: v for k, v in items}`) — same intent as a list comprehension but for mappings.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. `if os.path.exists(HIERARCHY_FILE):` branches conditionally.
2. `return {'admins': {}}.`

Code (copy-paste safe)

```
def load_hierarchy():
    if os.path.exists(HIERARCHY_FILE):
        with open(HIERARCHY_FILE, "r", encoding="utf-8") as f:
            data = json.load(f)

        # Compatibility: if the file is the "restructured" shape (admins -> clients + top-level traders)
        # convert it in-memory to the legacy nested shape {admins: {admin: {traders: {trader: {clients: ...}}}}
        try:
            if isinstance(data, dict) and 'admins' in data:
```

```

sample_admin = next(iter(data['admins'].values())) if data['admins'] else None
# Restructured format: admin entries contain 'clients' (list) and there is a top-level 't
if sample_admin is not None and 'clients' in sample_admin and 'traders' not in sample_admin:
    trader_registry = data.get('traders', {}) if isinstance(data.get('traders', {}),
    dict) else {}
    new_admins = {}
    for admin_name, admin_data in data.get('admins', {}).items():
        traders_map = {}
        for client in admin_data.get('clients', []):
            assigned = (client.get('assigned_trader') or '').strip()
            # Use assigned trader as key; if missing, group under empty-string key
            key = assigned
            if key not in traders_map:
                tr_email = trader_registry.get(key, {}).get('email', '') if key else ''
                traders_map[key] = {'email': tr_email, 'clients': []}
            # Keep assigned_trader on the client object for frontend/UI use
            client_copy = dict(client)
            traders_map[key]['clients'].append(client_copy)
        # Preserve all original admin fields (email, slack_user_id, etc.)
        preserved = {k: v for k, v in admin_data.items() if k != 'clients'}
        preserved['traders'] = traders_map
        new_admins[admin_name] = preserved
    data['admins'] = new_admins
except Exception:
    # If conversion fails for any reason, fall back to raw data
    pass

return data

return {"admins": {}}
```

```
def reload_hierarchy()
```

Reloads the hierarchy from disk to ensure freshness.

Why these Python concepts were used

- **global** — Rebinds a module-level name from inside the function. A smell in larger codebases — usually means a singleton or cache that could be passed in instead.

Step by step (top-level body)

1. Declares globals: SYSTEM_HIERARCHY.
2. Assigns new_data = load_hierarchy().
3. Calls SYSTEM_HIERARCHY.clear(...) for its side effect.
4. Calls SYSTEM_HIERARCHY.update(...) for its side effect.
5. **return** SYSTEM_HIERARCHY.

Code (copy-paste safe)

```

def reload_hierarchy():
    """Reloads the hierarchy from disk to ensure freshness."""
    global SYSTEM_HIERARCHY
    new_data = load_hierarchy()
    SYSTEM_HIERARCHY.clear()
    SYSTEM_HIERARCHY.update(new_data)
    return SYSTEM_HIERARCHY
```

```
def _to_flat_format(hierarchy_data)
```

Convert in-memory nested format back to flat format for disk persistence.

Why these Python concepts were used

- **dict comprehension** — Builds a dict in one expression (`{k: v for k, v in items}`) — same intent as a list comprehension but for mappings.

Step by step (top-level body)

1. Imports copy (lazy import inside the function).
2. Assigns `out = copy.deepcopy(hierarchy_data)`.
3. **for** `(admin_name, admin_data) in out.get('admins', {}).items():` iterates.
4. **return** out.

Code (copy-paste safe)

```
def _to_flat_format(hierarchy_data):
    """Convert in-memory nested format back to flat format for disk persistence."""
    import copy
    out = copy.deepcopy(hierarchy_data)
    for admin_name, admin_data in out.get('admins', {}).items():
        if 'traders' in admin_data and 'clients' not in admin_data:
            flat_clients = []
            for trader_name, trader_data in admin_data['traders'].items():
                for client in trader_data.get('clients', []):
                    c = dict(client)
                    c['assigned_trader'] = trader_name
                    flat_clients.append(c)
            # Preserve non-traders keys (email, slack_user_id, etc.)
            preserved = {k: v for k, v in admin_data.items() if k != 'traders'}
            out['admins'][admin_name] = {**preserved, 'clients': flat_clients}
    return out
```

```
def save_hierarchy(hierarchy_data)
```

Why these Python concepts were used

- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.

Step by step (top-level body)

1. Calls `os.makedirs(...)` for its side effect.
2. Assigns `flat = _to_flat_format(hierarchy_data)`.
3. **with** `open(HIERARCHY_FILE, 'w')`: enters a context manager.
4. **if** `hierarchy_data` is not `SYSTEM_HIERARCHY`: branches conditionally.

Code (copy-paste safe)

```
def save_hierarchy(hierarchy_data):
    # Ensure directory exists
    os.makedirs(os.path.dirname(HIERARCHY_FILE), exist_ok=True)
    # Always persist in flat format so the JSON structure stays consistent
```

```

flat = _to_flat_format(hierarchy_data)
with open(HIERARCHY_FILE, "w") as f:
    json.dump(flat, f, indent=4)

# Also update in-memory reference immediately
if hierarchy_data is not SYSTEM_HIERARCHY:
    SYSTEM_HIERARCHY.clear()
    SYSTEM_HIERARCHY.update(hierarchy_data)

def reassign_client_trader(client_name, admin_name, new_trader)

    Reassign a client to a different trader within the same admin. Auto-creates the new trader lane under the admin if it doesn't exist, pulling the email from the top-level traders registry. Returns True on success, False if the client or admin cannot be found.

```

Why these Python concepts were used

- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.

Step by step (top-level body)

1. Calls `reload_hierarchy(...)` for its side effect.
2. Assigns `admins = SYSTEM_HIERARCHY.get('admins', {})`.
3. **if** `admin_name` not in `admins`: branches conditionally.
4. Assigns `found_admin = None`.
5. Assigns `old_trader = None`.
6. **for** `search_admin` in `[admin_name] + [a for a in admins if a != admin_name]`: iterates.
7. **if** `old_trader` is `None` or `found_admin` is `None`: branches conditionally.
8. **if** `old_trader == new_trader` and `found_admin == admin_name`: branches conditionally.
9. Assigns `source_traders = admins[found_admin].get('traders', {})`.
10. Assigns `target_traders = admins[admin_name].get('traders', {})`.
11. **if** `new_trader` not in `target_traders`: branches conditionally.
12. Assigns `old_clients = source_traders[old_trader]['clients']`.
13. Assigns `client_obj = None`.
14. **for** `(i, c)` in `enumerate(old_clients)`: iterates.
15. ... and 6 more statement(s) in the body.

Code (copy-paste safe)

```

def reassign_client_trader(client_name, admin_name, new_trader):
    """
    Reassign a client to a different trader within the same admin.
    Auto-creates the new trader lane under the admin if it doesn't exist,
    pulling the email from the top-level traders registry.
    Returns True on success, False if the client or admin cannot be found.
    """
    reload_hierarchy()

```



```

admins = SYSTEM_HIERARCHY.get('admins', {})
if admin_name not in admins:
    return False

# Find current trader lane for this client – search specified admin first,
# then fall back to all admins (client may have been moved across admins)
found_admin = None
old_trader = None
for search_admin in ([admin_name] + [a for a in admins if a != admin_name]):
    traders = admins[search_admin].get('traders', {})
    for t_name, t_data in traders.items():
        for client in t_data.get('clients', []):
            if client.get('name') == client_name:
                found_admin = search_admin
                old_trader = t_name
                break
        if old_trader:
            break
    if old_trader:
        break

if old_trader is None or found_admin is None:
    return False

if old_trader == new_trader and found_admin == admin_name:
    return True # nothing to do

source_traders = admins[found_admin].get('traders', {})
target_traders = admins[admin_name].get('traders', {})

# Auto-create the new trader lane if missing under the target admin
if new_trader not in target_traders:
    trader_email = SYSTEM_HIERARCHY.get('traders', {}).get(new_trader, {}).get('email', '')
    target_traders[new_trader] = {'email': trader_email, 'clients': []}

# Remove client from old location
old_clients = source_traders[old_trader]['clients']
client_obj = None
for i, c in enumerate(old_clients):
    if c.get('name') == client_name:
        client_obj = old_clients.pop(i)
        break

if client_obj is None:
    return False

client_obj['assigned_trader'] = new_trader
target_traders[new_trader]['clients'].append(client_obj)

# Clean up empty trader lane
if not source_traders[old_trader]['clients']:
    del source_traders[old_trader]

save_hierarchy(SYSTEM_HIERARCHY)
return True

def add_admin(admin_name, email='')

```

Step by step (top-level body)

1. Calls `reload_hierarchy(...)` for its side effect.
2. **if** `admin_name` not in `SYSTEM_HIERARCHY['admins']`: branches conditionally.
3. **return** `False`.

Code (copy-paste safe)

```
def add_admin(admin_name, email=""):
    reload_hierarchy() # Ensure we have latest data
    if admin_name not in SYSTEM_HIERARCHY["admins"]:
        SYSTEM_HIERARCHY["admins"][admin_name] = {
            "email": email,
            "traders": {}
        }
    save_hierarchy(SYSTEM_HIERARCHY)
    return True
return False
```

```
def update_admin_details(admin_name, email, slack_user_id=None)
```

Step by step (top-level body)

1. Calls `reload_hierarchy(...)` for its side effect.
2. **if** `admin_name` in `SYSTEM_HIERARCHY['admins']`: branches conditionally.
3. **return** `False`.

Code (copy-paste safe)

```
def update_admin_details(admin_name, email, slack_user_id=None):
    reload_hierarchy()
    if admin_name in SYSTEM_HIERARCHY["admins"]:
        SYSTEM_HIERARCHY["admins"][admin_name]["email"] = email
        if slack_user_id is not None: # allow clearing by passing ''
            SYSTEM_HIERARCHY["admins"][admin_name]["slack_user_id"] = slack_user_id.strip()
        save_hierarchy(SYSTEM_HIERARCHY)
        return True
    return False
```

```
def update_trader_details(admin_name, trader_name, email)
```

Step by step (top-level body)

1. Calls `reload_hierarchy(...)` for its side effect.
2. **if** `admin_name` in `SYSTEM_HIERARCHY['admins']`: branches conditionally.
3. **return** `False`.

Code (copy-paste safe)

```
def update_trader_details(admin_name, trader_name, email):
    reload_hierarchy()
    if admin_name in SYSTEM_HIERARCHY["admins"]:
        traders = SYSTEM_HIERARCHY["admins"][admin_name]["traders"]
        if trader_name in traders:
            traders[trader_name]["email"] = email
        save_hierarchy(SYSTEM_HIERARCHY)
        return True
```

```
return False
```

```
def update_client_details(admin_name, trader_name, client_name, email)
```

Step by step (top-level body)

1. Calls `reload_hierarchy(...)` for its side effect.
2. **if** `admin_name` in `SYSTEM_HIERARCHY['admins']`: branches conditionally.
3. **return** `False`.

Code (copy-paste safe)

```
def update_client_details(admin_name, trader_name, client_name, email):
    reload_hierarchy()
    if admin_name in SYSTEM_HIERARCHY["admins"]:
        traders = SYSTEM_HIERARCHY["admins"][admin_name]["traders"]
        if trader_name in traders:
            clients = traders[trader_name]["clients"]
            for client in clients:
                if client["name"] == client_name:
                    client["email"] = email
                    save_hierarchy(SYSTEM_HIERARCHY)
                    return True
    return False
```

```
def update_client_category(admin_name, trader_name, client_name, category)
```

Step by step (top-level body)

1. Calls `reload_hierarchy(...)` for its side effect.
2. **if** `admin_name` in `SYSTEM_HIERARCHY['admins']`: branches conditionally.
3. **return** `False`.

Code (copy-paste safe)

```
def update_client_category(admin_name, trader_name, client_name, category):
    reload_hierarchy()
    if admin_name in SYSTEM_HIERARCHY["admins"]:
        traders = SYSTEM_HIERARCHY["admins"][admin_name]["traders"]
        if trader_name in traders:
            clients = traders[trader_name]["clients"]
            for client in clients:
                if client["name"] == client_name:
                    client["category"] = category
                    save_hierarchy(SYSTEM_HIERARCHY)
                    return True
    return False
```

```
def add_trader(admin_name, trader_name, email='')
```

Step by step (top-level body)

1. **if** `admin_name` in `SYSTEM_HIERARCHY['admins']`: branches conditionally.
2. **return** `False`.

Code (copy-paste safe)

```
def add_trader(admin_name, trader_name, email=""):
    if admin_name in SYSTEM_HIERARCHY["admins"]:
        if trader_name not in SYSTEM_HIERARCHY["admins"][admin_name]["traders"]:
            SYSTEM_HIERARCHY["admins"][admin_name]["traders"][trader_name] = {
                "email": email,
                "clients": []
            }
        save_hierarchy(SYSTEM_HIERARCHY)
        return True
    return False

def add_client(admin_name, trader_name, client_name, email='', category=''):
```

Why these Python concepts were used

- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with `append`, and clearer about intent: this is a transform, not a side-effecting loop.

Step by step (top-level body)

1. **if** `admin_name` in `SYSTEM_HIERARCHY['admins']`: branches conditionally.
2. **return** `False`.

Code (copy-paste safe)

```
def add_client(admin_name, trader_name, client_name, email="", category=""):
    if admin_name in SYSTEM_HIERARCHY["admins"]:
        traders = SYSTEM_HIERARCHY["admins"][admin_name]["traders"]
        if trader_name in traders:
            # Check if client exists
            existing_clients = [c["name"] for c in traders[trader_name]["clients"]]
            if client_name not in existing_clients:
                traders[trader_name]["clients"].append({
                    "name": client_name,
                    "email": email,
                    "category": category
                })
            save_hierarchy(SYSTEM_HIERARCHY)
            return True
    return False

def remove_admin(admin_name):
```

Step by step (top-level body)

1. **if** `admin_name` in `SYSTEM_HIERARCHY['admins']`: branches conditionally.
2. **return** `False`.

Code (copy-paste safe)

```
def remove_admin(admin_name):
    if admin_name in SYSTEM_HIERARCHY["admins"]:
        del SYSTEM_HIERARCHY["admins"][admin_name]
        save_hierarchy(SYSTEM_HIERARCHY)
        return True
```

```
return False
```

```
def remove_trader(admin_name, trader_name)
```

Step by step (top-level body)

1. **if** admin_name in SYSTEM_HIERARCHY['admins']: branches conditionally.
2. **return** False.

Code (copy-paste safe)

```
def remove_trader(admin_name, trader_name):
    if admin_name in SYSTEM_HIERARCHY["admins"]:
        traders = SYSTEM_HIERARCHY["admins"][admin_name]["traders"]
        if trader_name in traders:
            del traders[trader_name]
            save_hierarchy(SYSTEM_HIERARCHY)
            return True
    return False
```

```
def remove_client(admin_name, trader_name, client_name)
```

Step by step (top-level body)

1. **if** admin_name in SYSTEM_HIERARCHY['admins']: branches conditionally.
2. **return** False.

Code (copy-paste safe)

```
def remove_client(admin_name, trader_name, client_name):
    if admin_name in SYSTEM_HIERARCHY["admins"]:
        traders = SYSTEM_HIERARCHY["admins"][admin_name]["traders"]
        if trader_name in traders:
            clients = traders[trader_name]["clients"]
            for i, client in enumerate(clients):
                if client["name"] == client_name:
                    del clients[i]
                    save_hierarchy(SYSTEM_HIERARCHY)
                    return True
    return False
```

```
def rename_admin(old_name, new_name, new_email=None)
```

Rename an admin. Preserves all traders/clients underneath.

Step by step (top-level body)

1. Calls reload_hierarchy(...) for its side effect.
2. **if** old_name not in SYSTEM_HIERARCHY['admins']: branches conditionally.
3. **if** new_name != old_name and new_name in SYSTEM_HIERARCHY['admins']: branches conditionally.
4. Assigns admin_data = SYSTEM_HIERARCHY['admins'].pop(old_name).
5. **if** new_email is not None: branches conditionally.

6. Assigns `SYSTEM_HIERARCHY['admins'][new_name] = admin_data`.
7. Calls `save_hierarchy(...)` for its side effect.
8. **return** True.

Code (copy-paste safe)

```
def rename_admin(old_name, new_name, new_email=None):
    """Rename an admin. Preserves all traders/clients underneath."""
    reload_hierarchy()
    if old_name not in SYSTEM_HIERARCHY["admins"]:
        return False
    if new_name != old_name and new_name in SYSTEM_HIERARCHY["admins"]:
        return False # new name already taken
    admin_data = SYSTEM_HIERARCHY["admins"].pop(old_name)
    if new_email is not None:
        admin_data["email"] = new_email
    SYSTEM_HIERARCHY["admins"][new_name] = admin_data
    save_hierarchy(SYSTEM_HIERARCHY)
    return True
```

```
def rename_trader(admin_name, old_name, new_name, new_email=None)
```

Rename a trader under a given admin.

Step by step (top-level body)

1. Calls `reload_hierarchy(...)` for its side effect.
2. **if** `admin_name` not in `SYSTEM_HIERARCHY['admins']`: branches conditionally.
3. Assigns `traders = SYSTEM_HIERARCHY['admins'][admin_name]['traders']`.
4. **if** `old_name` not in `traders`: branches conditionally.
5. **if** `new_name != old_name` and `new_name` in `traders`: branches conditionally.
6. Assigns `trader_data = traders.pop(old_name)`.
7. **if** `new_email` is not None: branches conditionally.
8. Assigns `traders[new_name] = trader_data`.
9. Calls `save_hierarchy(...)` for its side effect.
10. **return** True.

Code (copy-paste safe)

```
def rename_trader(admin_name, old_name, new_name, new_email=None):
    """Rename a trader under a given admin."""
    reload_hierarchy()
    if admin_name not in SYSTEM_HIERARCHY["admins"]:
        return False
    traders = SYSTEM_HIERARCHY["admins"][admin_name]["traders"]
    if old_name not in traders:
        return False
    if new_name != old_name and new_name in traders:
        return False # new name already taken under this admin
    trader_data = traders.pop(old_name)
    if new_email is not None:
        trader_data["email"] = new_email
```

```
traders[new_name] = trader_data
save_hierarchy(SYSTEM_HIERARCHY)
return True
```

```
def rename_client(admin_name, trader_name, old_name, new_name, new_email=None, new_category=None)
    Rename a client under a given admin/trader.
```

Step by step (top-level body)

1. Calls `reload_hierarchy(...)` for its side effect.
2. **if** `admin_name` not in `SYSTEM_HIERARCHY['admins']`: branches conditionally.
3. Assigns `traders = SYSTEM_HIERARCHY['admins'][admin_name]['traders']`.
4. **if** `trader_name` not in `traders`: branches conditionally.
5. Assigns `clients = traders[trader_name]['clients']`.
6. **for** `client` in `clients`: iterates.
7. **return** `False`.

Code (copy-paste safe)

```
def rename_client(admin_name, trader_name, old_name, new_name, new_email=None, new_category=None):
    """Rename a client under a given admin/trader."""
    reload_hierarchy()
    if admin_name not in SYSTEM_HIERARCHY["admins"]:
        return False
    traders = SYSTEM_HIERARCHY["admins"][admin_name]["traders"]
    if trader_name not in traders:
        return False
    clients = traders[trader_name]["clients"]
    for client in clients:
        if client["name"] == old_name:
            client["name"] = new_name
            if new_email is not None:
                client["email"] = new_email
            if new_category is not None:
                client["category"] = new_category
            save_hierarchy(SYSTEM_HIERARCHY)
            return True
    return False
```

```
def move_client(client_name, old_admin, old_trader, new_admin, new_trader)
```

Why these Python concepts were used

- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to `sum`, `any`, `all`, or `max` where the intermediate list would be wasted.

Step by step (top-level body)

1. **if** `old_admin` not in `SYSTEM_HIERARCHY['admins']`: branches conditionally.
2. **if** `old_trader` not in `SYSTEM_HIERARCHY['admins'][old_admin]['traders']`: branches conditionally.
3. **if** `new_admin` not in `SYSTEM_HIERARCHY['admins']`: branches conditionally.

4. if new_trader not in SYSTEM_HIERARCHY['admins'][new_admin]['traders']: branches conditionally.
5. Assigns old_clients = SYSTEM_HIERARCHY['admins'][old_admin]['traders'][old_trad....
6. Assigns client_data = None.
7. Assigns client_index = -1.
8. for (i, c) in enumerate(old_clients): iterates.
9. if client_data is None: branches conditionally.
10. Assigns new_clients = SYSTEM_HIERARCHY['admins'][new_admin]['traders'][new_trad....
11. if any((c['name'] == client_name for c in new_clients)): branches conditionally.
12. Delete statement.
13. Calls new_clients.append(...) for its side effect.
14. Calls save_hierarchy(...) for its side effect.
15. ... and 1 more statement(s) in the body.

Code (copy-paste safe)

```
def move_client(client_name, old_admin, old_trader, new_admin, new_trader):
    # Verify existence of old location
    if old_admin not in SYSTEM_HIERARCHY["admins"]: return False
    if old_trader not in SYSTEM_HIERARCHY["admins"][old_admin]["traders"]: return False

    # Verify existence of new location
    if new_admin not in SYSTEM_HIERARCHY["admins"]: return False
    if new_trader not in SYSTEM_HIERARCHY["admins"][new_admin]["traders"]: return False

    # Find client
    old_clients = SYSTEM_HIERARCHY["admins"][old_admin]["traders"][old_trader]["clients"]
    client_data = None
    client_index = -1

    for i, c in enumerate(old_clients):
        if c["name"] == client_name:
            client_data = c
            client_index = i
            break

    if client_data is None: return False

    # Check if client already exists in new location (prevent duplicates)
    new_clients = SYSTEM_HIERARCHY["admins"][new_admin]["traders"][new_trader]["clients"]
    if any(c["name"] == client_name for c in new_clients):
        return False # Already exists there

    # Move
    del old_clients[client_index]
    new_clients.append(client_data)
    save_hierarchy(SYSTEM_HIERARCHY)
    return True

def move_trader(trader_name, old_admin, new_admin)
```

Step by step (top-level body)

1. **if** old_admin not in SYSTEM_HIERARCHY['admins']: branches conditionally.
2. **if** new_admin not in SYSTEM_HIERARCHY['admins']: branches conditionally.
3. Assigns old_traders = SYSTEM_HIERARCHY['admins'][old_admin]['traders'].
4. **if** trader_name not in old_traders: branches conditionally.
5. Assigns new_traders = SYSTEM_HIERARCHY['admins'][new_admin]['traders'].
6. **if** trader_name in new_traders: branches conditionally.
7. Assigns trader_data = old_traders[trader_name].
8. Delete statement.
9. Assigns new_traders[trader_name] = trader_data.
10. Calls save_hierarchy(...) for its side effect.
11. **return** True.

Code (copy-paste safe)

```
def move_trader(trader_name, old_admin, new_admin):
    # Verify existence
    if old_admin not in SYSTEM_HIERARCHY["admins"]: return False
    if new_admin not in SYSTEM_HIERARCHY["admins"]: return False

    old_traders = SYSTEM_HIERARCHY["admins"][old_admin]["traders"]
    if trader_name not in old_traders: return False

    new_traders = SYSTEM_HIERARCHY["admins"][new_admin]["traders"]
    if trader_name in new_traders: return False # Already exists

    # Move
    trader_data = old_traders[trader_name]
    del old_traders[trader_name]
    new_traders[trader_name] = trader_data
    save_hierarchy(SYSTEM_HIERARCHY)
    return True

def get_client_profile(client_name)
```

Finds the admin and trader for a given client name.

Step by step (top-level body)

1. **for** (admin, admin_data) in SYSTEM_HIERARCHY['admins'].items(): iterates.
2. **return** None.

Code (copy-paste safe)

```
def get_client_profile(client_name):
    """Finds the admin and trader for a given client name."""
    for admin, admin_data in SYSTEM_HIERARCHY["admins"].items():
        for trader, trader_data in admin_data["traders"].items():
            for client in trader_data["clients"]:
                if client["name"] == client_name:
                    return {"admin": admin, "trader": trader, "client": client_name, "email": client.get(
                        "email", "")}
    return None
```

```
def get_client_by_email(email)
```

Finds the client profile by email.

Step by step (top-level body)

1. **if** not email: branches conditionally.
2. Assigns email = email.lower().strip().
3. **for** (admin, admin_data) in SYSTEM_HIERARCHY['admins'].items(): iterates.
4. **return** None.

Code (copy-paste safe)

```
def get_client_by_email(email):
    """Finds the client profile by email."""
    if not email: return None
    email = email.lower().strip()
    for admin, admin_data in SYSTEM_HIERARCHY["admins"].items():
        for trader, trader_data in admin_data["traders"].items():
            for client in trader_data["clients"]:
                client_email = client.get("email", "").lower().strip()
                if client_email == email:
                    return {"admin": admin, "trader": trader, "client": client["name"], "email": client.get(
                        "email", "")}
    return None
```

```
def get_all_clients()
```

Returns a list of all client names.

Step by step (top-level body)

1. Assigns clients_list = [].
2. **for** admin_data in SYSTEM_HIERARCHY['admins'].values(): iterates.
3. **return** clients_list.

Code (copy-paste safe)

```
def get_all_clients():
    """Returns a list of all client names."""
    clients_list = []
    for admin_data in SYSTEM_HIERARCHY["admins"].values():
        for trader_data in admin_data["traders"].values():
            for client in trader_data["clients"]:
                clients_list.append(client["name"])
    return clients_list
```

```
def get_user_by_email(email)
```

Find specific user (Admin, Trader, or Client) by email. Returns: {'username': name, 'user_type': type, 'email': email, ...defaults}

Step by step (top-level body)

1. **if** not email: branches conditionally.

2. Assigns `email = email.lower().strip()`.
3. `for (admin_name, admin_data) in SYSTEM_HIERARCHY['admins'].items():` iterates.
4. `return None`.

Code (copy-paste safe)

```
def get_user_by_email(email):
    """
    Find specific user (Admin, Trader, or Client) by email.
    Returns: {'username': name, 'user_type': type, 'email': email, ...defaults}
    """
    if not email: return None
    email = email.lower().strip()

    for admin_name, admin_data in SYSTEM_HIERARCHY["admins"].items():
        # Check Admin
        if admin_data.get("email", "").lower().strip() == email:
            return {
                "username": admin_name,
                "user_type": "admin",
                "email": email,
                "parent_admin": None,
                "parent_trader": None,
                "must_change_password": 0,
                "is_active": 1
            }

    for trader_name, trader_data in admin_data["traders"].items():
        # Check Trader
        if trader_data.get("email", "").lower().strip() == email:
            return {
                "username": trader_name,
                "user_type": "trader",
                "email": email,
                "parent_admin": admin_name,
                "parent_trader": None,
                "must_change_password": 0,
                "is_active": 1
            }

    # Check Clients
    for client in trader_data["clients"]:
        client_email = client.get("email", "").lower().strip()
        if client_email == email:
            return {
                "username": client["name"],
                "user_type": "client",
                "email": email,
                "parent_admin": admin_name,
                "parent_trader": trader_name,
                "must_change_password": 0,
                "is_active": 1
            }

    return None
```

Step 3.4 — Build `utils/__init__.py`

What to do. Create the file `utils/__init__.py` in your project root and paste in the code shown in this step. The file is **1** line long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from `requirements.txt`.

utils/__init__.py

1 loc · 0 classes · 0 functions · 0 imports

Package marker. Importing this folder as a module runs the code here (often empty, sometimes used to re-export public names). across **1** lines.

Step 3.5 — Build `utils/data_processor.py`

What to do. Create the file `utils/data_processor.py` in your project root and paste in the code shown in this step. The file is **1349** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from `requirements.txt`.

What this file is for. Generic dataframe helpers — date normalisation, currency parsing, signed-number coercion.

utils/data_processor.py

1349 loc · 0 classes · 14 functions · 8 imports

Generic dataframe helpers — date normalisation, currency parsing, signed-number coercion. It defines **14** module-level functions, across **1,349** lines.

Python concepts demonstrated in this module

• Comprehensions • Context managers (with-statements) • Exceptions • f-strings •
Module-level state • Lambdas

Imports — what each one is for

```
from collections import defaultdict
```

Why imported. Specialised container datatypes that the dict / list / tuple / set primitives don't cover.

Used in this file. `defaultdict`

Example. `Counter(words)` # multiset that counts occurrences

Other common scenarios.

- `defaultdict(list)` auto-creates a default value on missing keys
- `deque(maxlen=N)` is a double-ended bounded queue

- OrderedDict (mostly redundant since 3.7 — dicts preserve order)
- namedtuple('Point', 'x y') for tiny immutable record types

```
from datetime import datetime
from datetime import date
from datetime import time
```

Why imported. Dates, times, durations. The fundamental temporal types: date, time, datetime, timedelta, timezone.

Used in this file. datetime

Example. datetime.now() - timedelta(days=7) # one week ago

Other common scenarios.

- datetime.strptime(s, '%Y-%m-%d') parses a string
- datetime.strftime(fmt) formats one
- datetime.fromtimestamp(n) converts a Unix epoch
- datetime.utcnow() for UTC (better: datetime.now(timezone.utc))

```
import pandas as pd
```

Why imported. DataFrame library — the workhorse for tabular data: loading CSV/Excel/SQL, joins, aggregations, time-series.

Used in this file. pd.isna, pd.read_csv, pd.to_datetime

Example. df = pd.read_csv(path); df.groupby('symbol')['pnl'].sum()

Other common scenarios.

- df.merge(other, on='key') joins like SQL
- df.resample('1D').agg(...) for time-series rebucketing
- df.to_sql / df.to_excel / df.to_parquet for output
- df.loc[mask, 'col'] for label-based selection/assignment

```
import requests
```

Why imported. The de-facto Python HTTP client. Synchronous; trades raw speed for an extremely friendly API.

Used in this file. requests.get

Example. requests.get(url, timeout=10).json()

Other common scenarios.

- Session() reuses TCP connections across calls
- params=, headers=, json=, data= for query/body shapes
- raise_for_status() turns 4xx/5xx into an exception
- stream=True for large downloads

```
from io import StringIO
```

Why imported. In-memory streams and the abstract base classes for file objects. Useful when an API expects a file but you only have bytes/text in memory.

Used in this file. StringIO

Example. io.BytesIO(b'data') # behaves like an open binary file

Other common scenarios.

- io.StringIO('text') for an in-memory text stream
- io.TextIOWrapper wraps a binary stream with an encoding

```
import io
```

Why imported. In-memory streams and the abstract base classes for file objects. Useful when an API expects a file but you only have bytes/text in memory.

Used in this file. io.BytesIO

Example. io.BytesIO(b'data') # behaves like an open binary file

Other common scenarios.

- io.StringIO('text') for an in-memory text stream
- io.TextIOWrapper wraps a binary stream with an encoding

```
import math
```

Why imported. Math functions — log, sqrt, trig, constants. Pure scalar operations (vectorised math lives in numpy).

Used in this file. math.isinf, math.isnan

Example. math.sqrt(x*x + y*y)

Other common scenarios.

- math.inf / math.nan / math.pi / math.e
- math.isnan(x), math.isinf(x) for predicates
- math.floor(x), math.ceil(x), math.trunc(x) for rounding

```
import re
```

Why imported. Regular expressions. Pattern matching, search, replace, and text extraction.

Used in this file. re.IGNORECASE, re.escape, re.findall, re.finditer, re.match, re.search, re.sub

Example. re.search(r'\d{3}-\d{4}', text) # returns a Match or None

Other common scenarios.

- re.findall(pattern, text) for every non-overlapping match
- re.sub(pattern, repl, text) to replace occurrences
- re.compile(...) to reuse a pattern (faster in a loop)
- Named groups: r'(?P<year>\d{4})' lets you do m.group('year')

Module constants

Each line below is followed by a plain-English note explaining what it does and why it is written that way.

```
_FIRM_MAP = {'mffu': 'My Funded Futures', 'mffuflex': 'My Funded Futures',  
            'myfundedfutures': 'My Funded Futures', 'myfundedfx': 'My Funded Futures', ...
```

Mapping or set literal — a lookup table referenced elsewhere in the module.

Functions

```
def clean_float(val)
```

Ensures float values are JSON compliant (no NaN or Inf). Returns 0.0 if invalid.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def clean_float(val):  
    """  
    Ensures float values are JSON compliant (no NaN or Inf).  
    Returns 0.0 if invalid.  
    """  
    try:  
        f_val = float(val)  
        if math.isnan(f_val) or math.isinf(f_val):  
            return 0.0  
        return f_val  
    except:  
        return 0.0
```

```
def normalize_prop_firm(name)
```

Normalize prop firm name to canonical form to prevent duplicates.

Step by step (top-level body)

1. **if** not name: branches conditionally.
2. Assigns original = str(name).strip().
3. Assigns key = original.lower().replace(' ', '').replace('_', '').
4. **if** key in _FIRM_MAP: branches conditionally.
5. **if** 'myfundedfutures' in key or 'myfundedfx' in key: branches conditionally.
6. **if** 'fundednext' in key: branches conditionally.
7. **if** 'topstep' in key: branches conditionally.
8. **if** 'fundingtick' in key: branches conditionally.
9. **return** original.

Code (copy-paste safe)

```
def normalize_prop_firm(name):
    """Normalize prop firm name to canonical form to prevent duplicates."""
    if not name:
        return "Unknown"
    original = str(name).strip()
    key = original.lower().replace(" ", "").replace("-", "")
    if key in _FIRM_MAP:
        return _FIRM_MAP[key]
    # Partial matches for common patterns
    if "myfundedfutures" in key or "myfundedfx" in key:
        return "My Funded Futures"
    if "fundednext" in key:
        return "FundedNext"
    if "topstep" in key:
        return "Topstep"
    if "fundingtick" in key:
        return "Funding Ticks"
    return original

def normalize_account_size(value)

    Normalize account size values to standard format: $X,XXX (v1.0.1) Handles: - "50k", "50K" → "$50,000"
    - "100000", "100,000" → "$100,000" - "$50,000" → "$50,000" (already correct) - "5000" → "$5,000"
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **if** not value: branches conditionally.
2. Assigns `val = str(value).strip().upper()`.
3. **if** `re.match('^\\$[\\d,]+$', val)`: branches conditionally.
4. Assigns `match = re.match('^\\$?(\\d+\\.?\\d*)K$', val, re.IGNORECASE)`.
5. **if** match: branches conditionally.
6. Assigns `val_clean = re.sub('[\\$,\\s]', '', val)`.
7. **try** block with 1 **except** clause.
8. **return** value.

Code (copy-paste safe)

```
def normalize_account_size(value):
    """
    Normalize account size values to standard format: $X,XXX (v1.0.1)

    Handles:
        - "50k", "50K" → "$50,000"
```



```

- "100000", "100,000" → "$100,000"
- "$50,000" → "$50,000" (already correct)
- "5000" → "$5,000"
"""
if not value:
    return value

val = str(value).strip().upper()

# Already in correct format
if re.match(r'^\${\d,}+$', val):
    return value

# Handle "50k" or "50K" format
match = re.match(r'^\${\d+\.?\d*}K$', val, re.IGNORECASE)
if match:
    num = float(match.group(1)) * 1000
    return f"${num:,.0f}"

# Handle plain numbers like "50000" or "50,000"
val_clean = re.sub(r'[\$,\\s]', '', val)
try:
    num = float(val_clean)
    return f"${num:,.0f}"
except:
    pass

# Return original if can't parse
return value

```

```
def clean_data_structure(data)
```

Recursively cleans a data structure (dict or list) to replace NaN/Inf with None. Also converts datetime objects to ISO format strings for JSON compatibility.

Why these Python concepts were used

- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **dict comprehension** — Builds a dict in one expression (`{k: v for k, v in items}`) — same intent as a list comprehension but for mappings.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.

Step by step (top-level body)

1. **if** `isinstance(data, dict)`: branches conditionally (with an **else/elif** arm).
2. **return** data.

Code (copy-paste safe)

```

def clean_data_structure(data):
    """
    Recursively cleans a data structure (dict or list) to replace NaN/Inf with None.
    Also converts datetime objects to ISO format strings for JSON compatibility.
    """
    if isinstance(data, dict):

```

```

        return {k: clean_data_structure(v) for k, v in data.items()}
    elif isinstance(data, list):
        return [clean_data_structure(v) for v in data]
    elif isinstance(data, float):
        if math.isnan(data) or math.isinf(data):
            return None
        return data
    elif hasattr(data, 'isoformat'):
        return data.isoformat()
    return data

```

```
def calculate_derived_metrics(df)
```

Applies Excel formulas to the DataFrame to ensure consistency with the Google Sheet.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.

Step by step (top-level body)

1. Defines a nested function `get_val(...)`.
2. Defines a nested function `is_blank(...)`.
3. **for** `col` in `['Hedge Net', 'Hedge Net.1']`: iterates.
4. **for** `(index, row)` in `df.iterrows()`: iterates.
5. **return** `df`.

Code (copy-paste safe)

```

def calculate_derived_metrics(df):
    """
    Applies Excel formulas to the DataFrame to ensure consistency with the Google Sheet.
    """
    # Helper to safely get float value
    def get_val(row, col):
        try:
            val = row.get(col)
            if pd.isna(val) or val == '':
                return 0.0
            # Handle string formatting if necessary
            if isinstance(val, str):
                s = val.replace('$', '').replace(' ', '').strip()
                if ',' in s:

```

```

        if '.' not in s:
            if re.search(r'\d{1,2}$', s):
                return 0.0
            s = s.replace(',', '')
        else:
            if re.search(r'\.', s):
                return 0.0
            s = s.replace(',', '')
        if s == '' or s == '-': return 0.0
        val = s
        return float(val)
    except:
        return 0.0

# Helper to check if blank
def is_blank(row, col):
    val = row.get(col)
    return pd.isna(val) or val == '' or str(val).strip() == ''

# Ensure target columns can hold any data type (prevent strict dtype errors)
for col in ['Hedge Net', 'Hedge Net.1']:
    if col not in df.columns:
        df[col] = ''
    df[col] = df[col].astype(object)

for index, row in df.iterrows():
    # --- Column N: Hedge Net ---
    # Formula: =IF(OR(ISBLANK(I3), G3<>"Fail"), "", -D3 + I3 + J3+K3+L3+M3)
    # I=Hedge Result 1, G=Status P1, D=Fee

    status_p1 = str(row.get('Status P1', ''))

    if is_blank(row, 'Hedge Result 1') or status_p1 != 'Fail':
        df.at[index, 'Hedge Net'] = ''
    else:
        fee = get_val(row, 'Fee')
        h1 = get_val(row, 'Hedge Result 1')
        h2 = get_val(row, 'Hedge Result 2')
        h3 = get_val(row, 'Hedge Result 3')
        h4 = get_val(row, 'Hedge Result 4')
        h5 = get_val(row, 'Hedge Result 5')

        # Formula: -Fee + Sum(Results)
        net = -fee + h1 + h2 + h3 + h4 + h5
        df.at[index, 'Hedge Net'] = net

    # --- Column AA: Hedge Net.1 ---
    # Formula:
    # IF S="Completed":
    #     SUM(Payouts) + SUM(Funded Hedge) + SUM(Phase 1 Hedge) - Fee - Activation Fee + SUM(Hedge Days
    # IF S="Fail":
    #     SUM(Funded Hedge) + SUM(Phase 1 Hedge) - Fee - Activation Fee
    # ELSE: ""

    # Support both 'Status' and 'Status Funded' column names
    status = str(row.get('Status') or row.get('Status Funded', ''))

    # Sums
    sum_payouts = sum([get_val(row, c) for c in ['Payout 1', 'Payout 2', 'Payout 3', 'Payout 4',
        'Payout 5', 'Payout 6']])

```

```

sum_funded_hedge = sum([get_val(row, c) for c in [
    'Hedge Result 1.1', 'Hedge Result 2.1', 'Hedge Result 3.1',
    'Hedge Result 4.1', 'Hedge Result 5.1', 'Hedge Result 6', 'Hedge Result 7'
]])

sum_phase1_hedge = sum([get_val(row, c) for c in [
    'Hedge Result 1', 'Hedge Result 2', 'Hedge Result 3',
    'Hedge Result 4', 'Hedge Result 5'
]])

fee = get_val(row, 'Fee')
activation_fee = get_val(row, 'Activation Fee')

sum_hedge_days = sum([get_val(row, f'Hedge Day {i}') for i in range(1, 51)])

if status == 'Completed':
    # SUM(AB,AD,AF,AH, T,U,V,W,X,Y,Z, I,J,K,L,M) - D - P + SUM(AL...BN)
    val = sum_payouts + sum_funded_hedge + sum_phase1_hedge - fee - activation_fee + sum_hedge_days
    df.at[index, 'Hedge Net.1'] = val

elif status == 'Fail':
    # T+U+V+W+X+Y+Z + I+J+K+L+M - D - P
    val = sum_funded_hedge + sum_phase1_hedge - fee - activation_fee
    df.at[index, 'Hedge Net.1'] = val

else:
    df.at[index, 'Hedge Net.1'] = ''

return df

```

```
def _fetch_gid_for_tab(sheet_key, tab_name_fragment)
```

Attempts to fetch the spreadsheet editor HTML to find the GID for a given tab name. Uses regex scanning around the tab name.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. try block with 1 **except** clause.

Code (copy-paste safe)

```

def _fetch_gid_for_tab(sheet_key, tab_name_fragment):
    """
    Attempts to fetch the spreadsheet editor HTML to find the GID for a given tab name.
    Uses regex scanning around the tab name.
    """
    try:

```

```

url = f"https://docs.google.com/spreadsheets/d/{sheet_key}/edit"
response = requests.get(url, timeout=10)
if response.status_code != 200:
    return None

content = response.text

# Regex to find the GID (quoted digits) shortly before the tab name
# We look for the tab name, then scan backwards.
# But regex can do this in one pass:
# Pattern: "GID" ... (not containing "GID") ... "Tab Name"

# We look for a pattern like: "12345", [...] "Profitability Waterlog"
# The intervening characters are usually JSON structure.

# Using a relatively safe window search
# Find all occurrences of the tab name
# For each occurrence, look at the preceding ~200 chars for a GID candidate
import re

# Escape the tab name for regex safety
safe_name = re.escape(tab_name_fragment)

matches = [m.start() for m in re.finditer(safe_name, content)]
for m in matches:
    # Look at a window before the match
    start = max(0, m - 300)
    window = content[start:m]

    # Find all quoted digit strings that are NOT followed by a colon (to avoid JSON keys)
    # Pattern: "(\d+)"(?!\:s*)
    # Note: In raw HTML of Google Sheets, the JSON is often stringified, so quotes are escaped as \"
    # We should look for both \"123\" and \"123\" just in case.

    # Debugging: Print window if not found
    # print(f"DEBUG WINDOW: {window}")

    candidates = []
    # specific pattern for stringified JSON GID: \"12345\"
    # structure: [integer, integer, "GID", ...

    # Simple digit extraction of reasonable length (8-10 digits usually)
    # Filter for numbers that look like GIDs (usually > 100000)

    raw_nums = re.findall(r'(\d{8,12})', window)

    # Also try the specific quoted format
    quoted_matches = re.findall(r'\\?"(\d+)\\?"(?!\:s*)', window)

    # Filter candidates
    valid_gids = []
    for num in quoted_matches:
        if len(num) > 5: # GIDs are usually long
            valid_gids.append(num)

    if valid_gids:
        return valid_gids[-1]

    return None
except Exception as e:

```

```
print(f"Error fetching GID for {tab_name_fragment}: {e}")
return None
```

```
def fetch_waterlog_history(sheet_url)
```

Fetches the 'Profitability Waterlog' tab data using dynamic GID discovery. Falls back to hardcoded GID if dynamic fetch fails. Assumes the main sheet URL is provided. Returns: list of dicts [{'date': 'YYYY-MM-DD', 'profit': 123.45}]

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with `append`, and clearer about intent: this is a transform, not a side-effecting loop.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Imports pandas (lazy import inside the function).
2. Imports urllib.parse (lazy import inside the function).
3. Lazy import from io.
4. Assigns `sheet_url = str(sheet_url).strip()`.
5. Assigns `match = re.search('/spreadsheets/d/([a-zA-Z0-9-_]+)', sheet_url)`.
6. **if** not `match`: branches conditionally.
7. Assigns `sheet_key = match.group(1)`.
8. Assigns `waterlog_gid = _fetch_gid_for_tab(sheet_key, 'Profitability Waterlog')`.
9. **if** not `waterlog_gid`: branches conditionally (with an **else/elif** arm).
10. Assigns `csv_url = f'https://docs.google.com/spreadsheets/d/{sheet_key}/expo...`
11. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def fetch_waterlog_history(sheet_url):
    """
    Fetches the 'Profitability Waterlog' tab data using dynamic GID discovery.
    Falls back to hardcoded GID if dynamic fetch fails.
    Assumes the main sheet URL is provided.
    Returns: list of dicts [{'date': 'YYYY-MM-DD', 'profit': 123.45}]
    """
    import pandas as pd
    import urllib.parse
    from io import StringIO

    # 1. Extract base URL / Sheet Key
    # sheet_url is e.g. https://docs.google.com/spreadsheets/d/KEY/edit...
    sheet_url = str(sheet_url).strip()
```

```

# Simple regex for KEY
match = re.search(r'/spreadsheets/d/([a-zA-Z0-9-_]+)', sheet_url)
if not match:
    # print("Invalid Sheet URL for waterlog fetch.")
    return []

sheet_key = match.group(1)

# Attempt to find dynamic GID
waterlog_gid = _fetch_gid_for_tab(sheet_key, "Profitability Waterlog")

if not waterlog_gid:
    print("Warning: Could not find dynamic GID for 'Profitability Waterlog', trying fallback.")
    waterlog_gid = "520289647" # Hardcoded GID fallback
else:
    # print(f"Found dynamic GID for Profitability Waterlog: {waterlog_gid}")
    pass

# Construct Export URL
csv_url = f"https://docs.google.com/spreadsheets/d/{sheet_key}/export?format=csv&gid={waterlog_gid}"

try:
    response = requests.get(csv_url, timeout=30)
    # response.raise_for_status() # Don't crash if waterlog missing
    if response.status_code != 200:
        return []

    if '<html' in response.text.lower():
        return []

    csv_io = StringIO(response.text)

    # Read CSV. User screenshot implies headers: Timestamp, Value
    # But maybe row 1 is header?
    try:
        df = pd.read_csv(csv_io)
    except:
        return []

    # Check columns
    if 'Timestamp' not in df.columns or 'Value' not in df.columns:
        # Maybe row 2 contains headers (skiprow=1)
        csv_io.seek(0)
        try:
            df = pd.read_csv(csv_io, skiprows=1)
        except:
            return []

    if 'Timestamp' not in df.columns or 'Value' not in df.columns:
        return []

    history = []
    for idx, row in df.iterrows():
        ts_raw = str(row.get('Timestamp', '')).strip()
        val_raw = str(row.get('Value', '')).strip()

        if not ts_raw or ts_raw.lower() == 'nan':
            continue

    try:

```

```

        # Parse date
        dt = pd.to_datetime(ts_raw, errors='coerce')
        if pd.isna(dt):
            continue
        date_str = dt.strftime('%Y-%m-%d')

        # Parse Value: "$123.45", "($123.45)" -> float
        val_clean = val_raw.replace('$', '').replace(',', '').replace(' ', '')
        if not val_clean or val_clean.lower() in ['- ', 'nan', 'null']:
            val = 0.0
        elif val_clean.startswith('(') and val_clean.endswith(')'):
            val = -float(val_clean[1:-1])
        else:
            val = float(val_clean)

        history.append({'date': date_str, 'profit': val})
    except:
        pass

    # Deduplicate same day - keep last entry
    # But if sorted by timestamp, last is latest.
    # Use dict to dedupe
    final = {}
    for item in history:
        final[item['date']] = item['profit']

    # Return list sorted by date
    sorted_dates = sorted(final.keys())
    return [{'date': d, 'profit': final[d]} for d in sorted_dates]

except Exception as e:
    print(f"Error fetching waterlog: {e}")
    return []

def fetch_evaluations(sheet_url)

```

Fetches evaluation data from a public Google Sheet CSV export. Finds the header row dynamically by looking for 'Prop Firm'. Also handles proper gid (Sheet ID) extraction from URL fragments. Fetches CSV (values) and XLSX (comments) in parallel for performance.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **dict comprehension** — Builds a dict in one expression (`{k: v for k, v in items}`) — same intent as a list comprehension but for mappings.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.
- **global** — Rebinds a module-level name from inside the function. A smell in larger codebases — usually means a singleton or cache that could be passed in instead.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Imports `urllib.parse` (lazy import inside the function).
2. Imports `concurrent.futures` (lazy import inside the function).
3. Assigns `sheet_url = sheet_url.strip()`.
4. **if** `'docs.google.com/spreadsheets'` not in `sheet_url`: branches conditionally.
5. Assigns `parsed = urllib.parse.urlparse(sheet_url)`.
6. **try** block with 1 **except** clause.
7. Assigns `gid = None`.
8. Assigns `query_params = urllib.parse.parse_qs(parsed.query)`.
9. **if** `'gid'` in `query_params`: branches conditionally.
10. **if** not `gid` and `parsed.fragment`: branches conditionally.
11. Defines a nested function `_fetch_xlsx_comments(...)`.
12. Defines a nested function `_fetch_csv_data(...)`.
13. **with** `concurrent.futures.ThreadPoolExecutor()`: enters a context manager.
14. **return** `(raw_data, xlsx_notes)`.

Code (copy-paste safe)

```
def fetch_evaluations(sheet_url):
    """
    Fetches evaluation data from a public Google Sheet CSV export.
    Finds the header row dynamically by looking for 'Prop Firm'.
    Also handles proper gid (Sheet ID) extraction from URL fragments.
    Fetches CSV (values) and XLSX (comments) in parallel for performance.
    """
    import urllib.parse
    import concurrent.futures

    # Clean the URL
    sheet_url = sheet_url.strip()

    # Check if it's a valid Google Sheets URL
```

```

if 'docs.google.com/spreadsheets' not in sheet_url:
    print("Invalid Google Sheets URL")
    return [], {}

# Parse URL
parsed = urllib.parse.urlparse(sheet_url)

# Extract Spreadsheet Key
try:
    path_parts = parsed.path.split('/')
    if 'd' in path_parts:
        key_idx = path_parts.index('d')
        sheet_key = path_parts[key_idx + 1]
    else:
        match = re.search(r'/spreadsheets/d/([a-zA-Z0-9-_]+)', sheet_url)
        if match:
            sheet_key = match.group(1)
        else:
            raise ValueError("Key not found")
except (ValueError, IndexError):
    print("Could not extract sheet key")
    # Try raw replace if parsing fails
    # Fallback to single threaded CSV-only fetch if key extraction fails
    return [], {}

# Extract GID (Sheet ID)
gid = None
query_params = urllib.parse.parse_qs(parsed.query)
if 'gid' in query_params:
    gid = query_params['gid'][0]

if not gid and parsed.fragment:
    try:
        frag_str = parsed.fragment
        if '?' in frag_str: frag_str = frag_str.split('?')[-1]
        frag_params = urllib.parse.parse_qs(frag_str)
        if 'gid' in frag_params: gid = frag_params['gid'][0]
    except:
        pass

# --- Helper Functions for Parallel Execution ---

def _fetch_xlsx_comments():
    notes = {}
    global OPENPYXL_AVAILABLE
    if not OPENPYXL_AVAILABLE:
        return notes

    try:
        base_url_xlsx = f"https://docs.google.com/spreadsheets/d/{sheet_key}/export"
        # Don't pass gid for XLSX - export full workbook to access Stats tab
        params_xlsx = {'format': 'xlsx'}

        xlsx_url = base_url_xlsx + "?" + urllib.parse.urlencode(params_xlsx)
        # print(f"DEBUG: Fetching XLSX for comments from: {xlsx_url}")

        resp = requests.get(xlsx_url, timeout=45)
        if resp.status_code == 200:
            wb = openpyxl.load_workbook(filename=io.BytesIO(resp.content), data_only=True)
            # Use Evaluations tab (first sheet) for comments

```

```

ws = wb.sheetnames[0] if wb.sheetnames else None
ws = wb[ws] if ws else wb.active

header_idx = -1
col_map = {}

# Scan first 20 rows
for r_idx, row in enumerate(ws.iter_rows(min_row=1, max_row=20, values_only=False)):
    row_vals = [str(c.value).strip() if c.value else '' for c in row]
    if any('Prop Firm' in str(v) for v in row_vals):
        header_idx = r_idx
        col_map = {idx: str(h).strip() for idx, h in enumerate(row_vals) if h}
        break
    elif any('Account Size' in str(v) for v in row_vals):
        header_idx = r_idx
        col_map = {idx: str(h).strip() for idx, h in enumerate(row_vals) if h}
        if 0 in col_map and not col_map[0]: col_map[0] = 'Prop Firm'
        break

if header_idx != -1:
    data_row_counter = 0
    for row_cells in ws.iter_rows(min_row=header_idx+2, values_only=False):
        # Determine if row is valid (Prop Firm check)
        is_valid = False
        for c_idx, cell in enumerate(row_cells):
            if c_idx in col_map and col_map[c_idx] == 'Prop Firm':
                if cell.value and str(cell.value).strip():
                    is_valid = True
                break

        if is_valid:
            for c_idx, cell in enumerate(row_cells):
                if c_idx in col_map and cell.comment:
                    c_name = col_map[c_idx]
                    if data_row_counter not in notes: notes[data_row_counter] = {}
                    notes[data_row_counter][c_name] = cell.comment.text.strip()
            data_row_counter += 1

# --- Extract Stats tab values (same XLSX workbook) ---
if 'Stats' in wb.sheetnames:
    try:
        stats_ws = wb['Stats']
        stats_tab = {}
        # Stats tab structure: column A = labels, column B = values
        # Parse both "Profitability - Completed" and "Current Cashflow - In Progress" sections
        current_section = None
        for row in stats_ws.iter_rows(min_row=1, max_row=30, max_col=2, values_only=True):
            label = str(row[0]).strip() if row[0] else ''
            val = row[1]
            if 'Profitability' in label and 'Completed' in label:
                current_section = 'profitability'
                continue
            elif 'Current Cashflow' in label:
                current_section = 'cashflow'
                continue
            elif 'Expected Value' in label:
                current_section = 'ev'
                continue
            elif 'Weekly' in label:
                current_section = None

```

```

        continue
    if current_section and label and val is not None and isinstance(val, (int,
float)):
        key = label.lower().replace(' ', '_')
        if current_section == 'cashflow':
            stats_tab[key] = round(float(val), 2)
        elif current_section == 'ev':
            stats_tab[key] = round(float(val), 2)
        else:
            stats_tab['prof_' + key] = round(float(val), 2)
    elif current_section and not label:
        current_section = None
    if stats_tab:
        notes['__stats_tab__'] = stats_tab
except Exception as e:
    print(f"Stats tab extraction failed (ignoring): {e}")

    return notes
except Exception as e:
    print(f"XLSX fetch failed (ignoring): {e}")
    return notes

def _fetch_csv_data():
    base_url = f"https://docs.google.com/spreadsheets/d/{sheet_key}/export"
    params = {'format': 'csv'}
    if gid: params['gid'] = gid
    csv_url = base_url + "?" + urllib.parse.urlencode(params)
    # print(f"DEBUG: Fetching CSV from: {csv_url}")

    try:
        response = requests.get(csv_url, timeout=30)
        if response.status_code != 200: return []
        if '<html' in response.text.lower(): return []

        df = pd.read_csv(StringIO(response.text), header=None)

        header_idx = -1
        found_via = None
        for i, row in df.head(10).iterrows():
            row_str = row.astype(str)
            if row_str.str.contains('Prop Firm', case=False, na=False).any():
                header_idx = i; found_via = 'Prop Firm'
                break
            elif row_str.str.contains('Account Size', case=False, na=False).any():
                header_idx = i; found_via = 'Account Size'
                break

        if header_idx != -1:
            df = pd.read_csv(StringIO(response.text), header=header_idx)
            cleaned_cols = [str(c).strip() for c in df.columns]

            if found_via == 'Account Size' and 'Prop Firm' not in cleaned_cols:
                if not cleaned_cols[0] or cleaned_cols[0].lower() == 'nan' or cleaned_cols[0].startswith(
'Unnamed:'):
                    cleaned_cols[0] = 'Prop Firm'
            df.columns = cleaned_cols

            # Normalize column name variants: some sheets use 'Status Funded' instead of 'Status'
            if 'Status Funded' in df.columns and 'Status' not in df.columns:
                df = df.rename(columns={'Status Funded': 'Status'})

```

```

allowed_columns = [
    'Prop Firm', 'Account Size', 'Date Purchased', 'Fee',
    'Date Started', 'Date Ended', 'Status P1', 'Account #',
    'Hedge Result 1', 'Hedge Result 2', 'Hedge Result 3', 'Hedge Result 4', 'Hedge Result 5',
    'Account #.1', 'Activation Fee', 'Date Started.1', 'Date Ended.1', 'Status', 'Status P1',
    'Hedge Result 1.1', 'Hedge Result 2.1', 'Hedge Result 3.1', 'Hedge Result 4.1', 'Hedge Result 5.1',
    'Hedge Result 6', 'Hedge Result 7', 'Hedge Net.1',
    'Payout 1', 'Date 1', 'Payout 2', 'Date 2', 'Payout 3', 'Date 3', 'Payout 4', 'Date 4',
    'Farming Net'
]
for i in range(1, 51):
    allowed_columns.append(f'Prop Day {i}')
    allowed_columns.append(f'Hedge Day {i}')

existing_columns = [c for c in df.columns if c in allowed_columns]
df = df[existing_columns]

if 'Prop Firm' in df.columns:
    df = df[df['Prop Firm'].notna()]
    df['Prop Firm'] = df['Prop Firm'].apply(normalize_prop_firm)
    df = calculate_derived_metrics(df)
    if 'Account Size' in df.columns:
        df['Account Size'] = df['Account Size'].apply(normalize_account_size)
    return clean_data_structure(df.to_dict(orient='records'))
return []
except Exception as e:
    print(f"CSV fetch failed: {e}")
    return []

# --- Execute Parallel Fetch ---
with concurrent.futures.ThreadPoolExecutor() as executor:
    future_xlsx = executor.submit(_fetch_xlsx_comments)
    future_csv = executor.submit(_fetch_csv_data)

    xlsx_notes = future_xlsx.result()
    raw_data = future_csv.result()

return raw_data, xlsx_notes

```

```
def parse_currency(val)
```

Parses a currency string (e.g., '\$100,000', '\$1,234.56') to a float. Returns 0.0 for values Google Sheets cannot parse as numbers, so that our SUM logic matches sheet SUM() behaviour exactly.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

Step by step (top-level body)

1. if val is None: branches conditionally.
2. try block with 1 except clause.

Code (copy-paste safe)

```
def parse_currency(val):
    """
```

```

Parses a currency string (e.g., '$100,000', '$1,234.56') to a float.
Returns 0.0 for values Google Sheets cannot parse as numbers, so that
our SUM logic matches sheet SUM() behaviour exactly.
"""
if val is None:
    return 0.0
try:
    s = str(val)
    # Normalize common currency/spacing variants seen in Sheets exports / HTML
    s = (
        s.replace('$', '')
        .replace('\u20ac', '')
        .replace('\u00a3', '')
        .replace('\u00a0', '') # NBSP
        .replace('\u202f', '') # NNBS (thin no-break space)
        .strip()
    )
    if not s or s.lower() == 'nan':
        return 0.0

    # Accounting negative: "(123.45)" → -123.45
    if len(s) >= 2 and s[0] == '(' and s[-1] == ')':
        s = '-' + s[1:-1].strip()
    if ',' in s:
        if '.' not in s:
            # No period: could be thousands ("1,234") or European decimal ("-573,79").
            # Google Sheets (US locale) cannot reliably parse these without a period,
            # so treat comma-decimal endings as unparseable (text → 0 in SUM).
            # Comma groups of exactly 3 digits are valid thousands separators.
            if re.search(r'\d{1,2}$', s): # ends in ,X or ,XX → ambiguous/euro → 0
                return 0.0
            s = s.replace(',', '') # thousands comma → strip safely
        else:
            # Has both comma and period.
            # Malformed if comma appears directly before period (e.g. "253,.96").
            if re.search(r'\. ', s):
                return 0.0
            s = s.replace(',', '') # valid US thousands separator
    return round(float(s), 2)
except:
    return 0.0

```

```

def calculate_statistics(evaluations, mt5_deals=None, mt5_account=None, xlsx_notes=None,
    historical_accounts=None)

```

Calculates detailed statistics matching the 'Stats' sheet structure. Uses exact SUMIF logic from Google Sheet formulas. If xlsx_notes contains Stats tab values (from XLSX export), uses those for the cashflow section to guarantee exact match with Google Sheets. historical_accounts: list of previous MT5 accounts to include in discrepancy calc.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **list comprehension** — Builds a list in a single expression ([f(x) for x in xs]). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.

- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns stats = {'profitability_completed': {'challenge_fees': 0.0, 'hedg....
2. if not evaluations and (not mt5_deals) and (not mt5_account): branches conditionally.
3. Assigns evaluations = evaluations or [].
4. Defines a nested function get_firm_stats(...).
5. Assigns TARGET_FIRMS = ['My Funded Futures', 'Funding Ticks', 'TradeDay', 'Funde....
6. for firm in TARGET_FIRMS: iterates.
7. Assigns sheet_hedge_total = 0.0.
8. Assigns P1_HEDGE_COLS = ['Hedge Result 1', 'Hedge Result 2', 'Hedge Result 3', 'H....
9. Assigns FUNDED_HEDGE_COLS = ['Hedge Result 1.1', 'Hedge Result 2.1', 'Hedge Result 3.....
10. Assigns HEDGE_DAY_COLS = [f'Hedge Day {i}' for i in range(1, 51)].
11. for ev in evaluations: iterates.
12. if stats['eval_totals']['total_failed'] > 0: branches conditionally.
13. Assigns total_eval_closed = stats['eval_totals']['total_passed'] + stats['eval_totals....
14. if total_eval_closed > 0: branches conditionally.
15. ... and 26 more statement(s) in the body.

Code (copy-paste safe)

```
def calculate_statistics(evaluations, mt5_deals=None, mt5_account=None, xlsx_notes=None,
    historical_accounts=None):
    """
    Calculates detailed statistics matching the 'Stats' sheet structure.
    Uses exact SUMIF logic from Google Sheet formulas.
    If xlsx_notes contains Stats tab values (from XLSX export), uses those
    for the cashflow section to guarantee exact match with Google Sheets.
    historical_accounts: list of previous MT5 accounts to include in discrepancy calc.
    """
    stats = {
        "profitability_completed": {
            "challenge_fees": 0.0, "hedging_results": 0.0, "farming_results": 0.0, "payouts": 0.0, "net_p
        },
        "cashflow_inprogress": {
```

```

        "challenge_fees": 0.0, "hedging_results": 0.0, "farming_results": 0.0, "payouts": 0.0, "net_
    },
    "evaluation_data": {}, # { "PropFirmName": { ...stats... } }
    "funded_data": {}, # { "PropFirmName": { ...stats... } }
    "eval_totals": {
        "total_running": 0, "not_started": 0, "ongoing": 0, "total_passed": 0, "total_failed": 0, "av
    },
    "funded_totals": {
        "not_started": 0, "ongoing": 0, "failed": 0, "completed": 0, "avg_net_failed": 0.0, "avg_net_
    },
    "expected_value": 0.0,
    "ev_tracking": {
        "total_net_ended": 0.0,
        "count_ended": 0
    },
    "hedging_review": {
        "total_deposits": 0.0,
        "total_withdrawals": 0.0,
        "current_balance": 0.0,
        "actual_hedging_results": 0.0,
        "sheet_hedging_results": 0.0,
        "sheet_hedging_component": 0.0,
        "sheet_farming_component": 0.0,
        "discrepancy": 0.0
    }
}

# If no evaluations and no MT5 data, return empty stats
if not evaluations and not mt5_deals and not mt5_account:
    return stats

evaluations = evaluations or []

# Helper to get or create firm stats
def get_firm_stats(firm_name, section):
    if firm_name not in stats[section]:
        stats[section][firm_name] = {
            "total_running": 0, "not_started": 0, "ongoing": 0, "total_passed": 0, "total_failed": 0,
            "failed": 0, "completed": 0, "net_completed_sum": 0.0, "total_funding": 0.0
        }
    return stats[section][firm_name]

# Pre-populate target firms to match sheet structure
TARGET_FIRMS = ["My Funded Futures", "Funding Ticks", "TradeDay", "FundedNext", "Topstep",
    "Top One Futures"]
for firm in TARGET_FIRMS:
    get_firm_stats(firm, "evaluation_data")
    get_firm_stats(firm, "funded_data")

sheet_hedge_total = 0.0

# Phase 1 hedge columns (J-N in sheet = Hedge Result 1-5)
P1_HEDGE_COLS = ['Hedge Result 1', 'Hedge Result 2', 'Hedge Result 3', 'Hedge Result 4', 'Hedge Result 5']
# Funded hedge columns – ALL of U:AA per the sheet formula
# =SUM(Evaluations!U:AA) spans Hedge Result 1.1 .. 5.1 + Hedge Result 6 .. 7.
# The sheet treats every value in U:AA as part of "Hedging Results" and NEVER
# as part of "Farming Results" (farming = Hedge Day columns only).
FUNDED_HEDGE_COLS = [
    'Hedge Result 1.1', 'Hedge Result 2.1', 'Hedge Result 3.1', 'Hedge Result 4.1', 'Hedge Result 5.1',
    'Hedge Result 6', 'Hedge Result 7',

```



```

]
# Hedge Day columns for farming (daily FA / consistency P&L).
# This matches the Google Sheets formula the Evaluations tab uses for Farming
# Results: =SUM(Evaluations!AM:AM, AO:AO, AQ:AQ, ... DA:DA) – i.e. every
# "Hedge Day N" column only (no Hedge Result 6/7, no Farming Net override).
HEDGE_DAY_COLS = [f'Hedge Day {i}' for i in range(1, 51)]

for ev in evaluations:
    try:
        firm = normalize_prop_firm(ev.get('Prop Firm', 'Unknown'))
        status_p1 = str(ev.get('Status P1', '')).strip()
        # Support both 'Status' and 'Status Funded' column names (sheet variant)
        status_funded = str(ev.get('Status') or ev.get('Status Funded', '')).strip()

        # Normalize for comparison (but keep original for exact match)
        status_p1_lower = status_p1.lower()
        status_funded_lower = status_funded.lower()

        # === CALCULATE VALUES USING EXACT SHEET SUMIF LOGIC ===

        # Get individual hedge results (NOT the calculated Hedge Net columns)
        # Round each row-level sum to 2dp to match Google Sheets cell precision.
        # funded_hedges = ALL of U:AA (HR 1.1..5.1 + HR 6..7) – the sheet's
        # Hedging Results formula sums this whole range.
        # hedge_days = Hedge Day 1..50 – the sheet's Farming Results formula
        # sums exactly these columns (AM, AO, AQ, ... DA = 34 cols today,
        # extended to 50 here for future-proofing).
        p1_hedges = round(sum(parse_currency(ev.get(col)) for col in P1_HEDGE_COLS), 2)
        funded_hedges = round(sum(parse_currency(ev.get(col)) for col in FUNDED_HEDGE_COLS), 2)
        hedge_days = round(sum(parse_currency(ev.get(col)) for col in HEDGE_DAY_COLS), 2)

        fee = parse_currency(ev.get('Fee'))
        activation_fee = parse_currency(ev.get('Activation Fee'))
        farming_net = parse_currency(ev.get('Farming Net'))
        payouts = round(sum(parse_currency(ev.get(f'Payout {i}')) for i in range(1, 7)), 2)

        # For sheet_hedge_total tracking
        hedge_net = parse_currency(ev.get('Hedge Net')) + parse_currency(ev.get('Hedge Net.1'))
        p1_hedge_net = parse_currency(ev.get('Hedge Net'))
        funded_hedge_net = parse_currency(ev.get('Hedge Net.1'))
        sheet_hedge_total += hedge_net

        # EV tracking: Sheet formula = SUM(0:0, AB:AB) / COUNT(0:0, AB:AB)
        # 0 = Hedge Net (P1), AB = Hedge Net.1 (Funded)
        # Count each non-empty cell independently (a row can contribute to both)
        raw_p1_hn = ev.get('Hedge Net')
        raw_fd_hn = ev.get('Hedge Net.1')
        if raw_p1_hn is not None and str(raw_p1_hn).strip() not in ('', '-'):
            stats["ev_tracking"]["total_net_ended"] += p1_hedge_net
            stats["ev_tracking"]["count_ended"] += 1
        if raw_fd_hn is not None and str(raw_fd_hn).strip() not in ('', '-'):
            stats["ev_tracking"]["total_net_ended"] += funded_hedge_net
            stats["ev_tracking"]["count_ended"] += 1

        # Normalize status lifecycle buckets (case-insensitive, tolerant of variants)
        is_deleted = ('deleted' in status_p1_lower) or ('deleted' in status_funded_lower)
        is_p1_fail = any(k in status_p1_lower for k in ('fail', 'breach', 'sl', 'closed'))
        is_funded_fail = any(k in status_funded_lower for k in ('fail', 'breach', 'sl', 'closed'))
        is_funded_completed = ('complete' in status_funded_lower)
        is_funded_ended = is_funded_fail or is_funded_completed

```

```

is_in_progress = (not is_deleted) and (not is_p1_fail) and (not is_funded_ended)

# === CASHFLOW - IN PROGRESS (stored key name; meaning = whole client / all accounts) ===
# Sum payouts, fees, hedging, and farming across every non-deleted row – not lifecycle-active
# Profitability – Completed remains the ended / inactive slice only.
#
# Sheet formulas being mirrored (Evaluations tab):
# Hedging Results = SUM(J:N) + SUM(U:AA)
#                 = P1 hedges (HR 1..5) + ALL funded hedges (HR 1.1..5.1 + HR 6..7)
# Farming Results = SUM(AM:AM, AO:AO, AQ:AQ, ... DA:DA)
#                 = Hedge Day 1..N only (no Hedge Result 6/7, no Farming Net override)
if not is_deleted:
    row_hedging = round(p1_hedges + funded_hedges, 2)
    row_farming = round(hedge_days, 2)

    stats["cashflow_inprogress"]["challenge_fees"] = round(stats["cashflow_inprogress"][
        "challenge_fees"] + fee + activation_fee, 2)
    stats["cashflow_inprogress"]["hedging_results"] = round(stats["cashflow_inprogress"][
        "hedging_results"] + row_hedging, 2)
    stats["cashflow_inprogress"]["farming_results"] = round(stats["cashflow_inprogress"][
        "farming_results"] + row_farming, 2)
    stats["cashflow_inprogress"]["payouts"] = round(stats["cashflow_inprogress"][
        "payouts"] + payouts, 2)
    stats["cashflow_inprogress"]["activation_fee"] = round(stats["cashflow_inprogress"][
        "activation_fee"] + activation_fee, 2)

# Skip deleted accounts for profitability and other sections
if is_deleted:
    continue

# === PROFITABILITY - COMPLETED (exact SUMIF logic from sheet) ===
# Challenge Fees formula: (SUMIF(Fee,P1="Fail") + SUMIF(Fee,Status="Completed") + SUMIF(Fee,S
# We store as positive, the display will show as negative

# Sheet formulas being mirrored (Evaluations tab):
# Hedging Results Completed
#     = SUMIF(J:N, H="Fail") # P1 hedges when P1 failed
#   + SUMIF(U:AA, T="Fail") + SUMIF(U:AA, T="Completed") # funded hedges when funded ended
#   + SUMIF(J:N, T="Fail") + SUMIF(J:N, T="Completed") # P1 hedges when funded ended
# Farming Results Completed
#     = SUMIFS(AM:DA, T:T, "Completed") # Hedge Days ONLY where Status=Completed

# Challenge Fees Completed: count ended rows exactly once
if is_p1_fail or is_funded_ended:
    stats["profitability_completed"]["challenge_fees"] = round(stats["profitability_completed
        "challenge_fees"] + fee + activation_fee, 2)

# Hedging Results Completed: avoid double-counting when both flags are set
if is_funded_ended:
    # ALL funded hedges (HR 1.1..5.1 + HR 6..7) + P1 hedges
    stats["profitability_completed"]["hedging_results"] = round(stats["profitability_completo
        "hedging_results"] + funded_hedges + p1_hedges, 2)
elif is_p1_fail:
    # Pure phase-1 failure (never reached funded end)
    stats["profitability_completed"]["hedging_results"] = round(stats["profitability_completo
        "hedging_results"] + p1_hedges, 2)

# Farming Results Completed: pure Hedge Day sum, only on Status="Completed"
# (matches =SUMIFS(AM:DA, T:T, "Completed") exactly).
if is_funded_completed:

```

```

        stats["profitability_completed"]["farming_results"] = round(stats["profitability_completed"]
            "farming_results"] + hedge_days, 2)

    # Payouts Completed: where Status=Completed ONLY (sheet formula: SUMIF Status="Completed")
    if is_funded_completed:
        stats["profitability_completed"]["payouts"] = round(stats["profitability_completed"]
            "payouts"] + payouts, 2)

    # Activation Fee for Completed (B25 in Net Profit formula)
    # Track activation fee for accounts that have ended (count once)
    if is_p1_fail or is_funded_ended:
        stats["profitability_completed"]["activation_fee"] = round(stats["profitability_completed"]
            "activation_fee"] + activation_fee, 2)
except Exception as e:
    print(f"Error processing evaluation row: {e}")
    continue

# Track "in progress" for evaluation data section (accounts not ended)
is_in_progress = (not is_deleted) and (not is_p1_fail) and (not is_funded_ended)

# --- 2. Evaluation Data ---
estats = get_firm_stats(firm, "evaluation_data")

# Calculate net_profit for this row
total_fee = fee + activation_fee
total_hedge = p1_hedges + funded_hedges
total_farm = farming_net + hedge_days
net_profit = payouts + total_hedge + total_farm - total_fee

# Eval Status Logic
# Active/Ongoing
if is_in_progress:
    estats["total_running"] += 1
    stats["eval_totals"]["total_running"] += 1

    if not ev.get('Date Started'):
        estats["not_started"] += 1
        stats["eval_totals"]["not_started"] += 1
    else:
        estats["ongoing"] += 1
        stats["eval_totals"]["ongoing"] += 1

# Passed (P1)
if status_p1 == 'Pass' or status_p1_lower == 'pass':
    estats["total_passed"] += 1
    stats["eval_totals"]["total_passed"] += 1

# Failed (P1)
if is_p1_fail:
    estats["total_failed"] += 1
    stats["eval_totals"]["total_failed"] += 1
    estats["net_failed_sum"] += net_profit
    stats["eval_totals"]["avg_net_failed"] += net_profit

    # EV tracking moved to cashflow section above (unconditional, matches Sheet SUM/COUNT)

# --- 3. Funded Data ---
fstats = get_firm_stats(firm, "funded_data")

# Only count if it reached funded stage (Status is present or P1 passed)

```

```

has_funded_status = status_funded and status_funded != '-'
p1_passed = status_p1 == 'Pass' or status_p1_lower == 'pass'

if has_funded_status or p1_passed:
    funding_amount = parse_currency(ev.get('Account Size'))

    # Total Funding: Sum of ALL accounts that reached funded stage
    fstats["total_funding"] += funding_amount
    stats["funded_totals"]["total_funding"] += funding_amount

    if is_in_progress:
        if not ev.get('Date Started.1'): # Funded Start Date
            fstats["not_started"] += 1
            stats["funded_totals"]["not_started"] += 1
        else:
            fstats["ongoing"] += 1
            stats["funded_totals"]["ongoing"] += 1

    elif is_funded_fail:
        fstats["failed"] += 1
        stats["funded_totals"]["failed"] += 1
        fstats["net_failed_sum"] += net_profit
        stats["funded_totals"]["avg_net_failed"] += net_profit

    # EV tracking moved to cashflow section above (unconditional, matches Sheet SUM/COUNT)

    elif is_funded_completed:
        fstats["completed"] += 1
        stats["funded_totals"]["completed"] += 1
        fstats["net_completed_sum"] += net_profit
        stats["funded_totals"]["avg_net_completed"] += net_profit

# --- Calculate Averages & Rates ---

# Eval Averages
if stats["eval_totals"]["total_failed"] > 0:
    stats["eval_totals"]["avg_net_failed"] /= stats["eval_totals"]["total_failed"]

total_eval_closed = stats["eval_totals"]["total_passed"] + stats["eval_totals"]["total_failed"]
if total_eval_closed > 0:
    stats["eval_totals"]["funded_rate"] = (stats["eval_totals"]["total_passed"] / total_eval_closed)

# Funded Averages
if stats["funded_totals"]["failed"] > 0:
    stats["funded_totals"]["avg_net_failed"] /= stats["funded_totals"]["failed"]

if stats["funded_totals"]["completed"] > 0:
    stats["funded_totals"]["avg_net_completed"] /= stats["funded_totals"]["completed"]

# Per-Firm Averages
for firm in stats["evaluation_data"]:
    d = stats["evaluation_data"][firm]
    if d["total_failed"] > 0:
        d["avg_net_failed"] = d["net_failed_sum"] / d["total_failed"]
    else:
        d["avg_net_failed"] = 0.0

    closed = d["total_passed"] + d["total_failed"]
    d["funded_rate"] = (d["total_passed"] / closed * 100) if closed > 0 else 0.0

for firm in stats["funded_data"]:
```

```

d = stats["funded_data"][firm]
if d["failed"] > 0:
    d["avg_net_failed"] = d["net_failed_sum"] / d["failed"]
else:
    d["avg_net_failed"] = 0.0

if d["completed"] > 0:
    d["avg_net_completed"] = d["net_completed_sum"] / d["completed"]
else:
    d["avg_net_completed"] = 0.0

# --- Expected Value ---
# Sheet formula: =IFERROR(SUM(Evaluations!0:0,Evaluations!AB:AB) / COUNT(Evaluations!0:0,Evaluations!
# 0 = Hedge Net (P1), AB = Hedge Net.1 (Funded) – average of non-empty hedge net cells
ev_data = stats["ev_tracking"]
if ev_data["count_ended"] > 0:
    stats["expected_value"] = round(ev_data["total_net_ended"] / ev_data["count_ended"], 2)
else:
    stats["expected_value"] = 0.0

# --- Hedging Review ---
# Sheet Hedging Results = Total Hedging Results + Total Farming Results (from ALL evaluations)
# cashflow_inprogress aggregates every non-deleted row (all accounts, active or ended).
# So we use those totals directly (not adding completed which would double-count)
total_hedging = stats["cashflow_inprogress"]["hedging_results"]
total_farming = stats["cashflow_inprogress"]["farming_results"]
stats["hedging_review"]["sheet_hedging_results"] = total_hedging + total_farming
stats["hedging_review"]["sheet_hedging_component"] = total_hedging
stats["hedging_review"]["sheet_farming_component"] = total_farming

# Debug logging (will be visible in server logs)
import sys
debug_log = []

if mt5_account:
    # Handle both dict (serialized) and object
    if isinstance(mt5_account, dict):
        balance = float(mt5_account.get('balance', 0.0) or 0.0)
        deposits = float(mt5_account.get('total_deposits', 0.0) or 0.0)
        withdrawals = float(mt5_account.get('total_withdrawals', 0.0) or 0.0)
    else:
        balance = float(getattr(mt5_account, 'balance', 0.0) or 0.0)
        deposits = float(getattr(mt5_account, 'total_deposits', 0.0) or 0.0)
        withdrawals = float(getattr(mt5_account, 'total_withdrawals', 0.0) or 0.0)

    stats["hedging_review"]["current_balance"] = balance
    stats["hedging_review"]["total_deposits"] = deposits
    stats["hedging_review"]["total_withdrawals"] = withdrawals

# Include historical MT5 accounts in the calculation
hist_dep = 0.0
hist_with = 0.0
hist_bal = 0.0
if historical_accounts:
    for acc in historical_accounts:
        hist_dep += float(acc.get('deposits', 0) or 0)
        hist_with += float(acc.get('withdrawals', 0) or 0)
        hist_bal += float(acc.get('final_balance', 0) or 0)

combined_deposits = deposits + hist_dep

```

```

combined_withdrawals = withdrawals + hist_with
combined_balance = balance + hist_bal

# Calculate Actual Hedging Results using the Google Sheet formula:
# =IF(AND(B20<>"", B22<>""), B22-(B20-B21), "")
# Which is: Combined Balance - (Combined Deposits - Combined Withdrawals)
# Note: withdrawals are already negative, so we add them
net_deposits = combined_deposits + combined_withdrawals
actual_hedging = combined_balance - net_deposits
stats["hedging_review"]["actual_hedging_results"] = actual_hedging

debug_log.append(
    f"MT5 Account: balance=${balance:.2f}, deposits=${deposits:.2f}, withdrawals=${withdrawals:.2f}"
)
if historical_accounts:
    debug_log.append(
        f"Historical: deposits=${hist_dep:.2f}, withdrawals=${hist_with:.2f}, balance=${hist_bal:.2f}"
    )
    debug_log.append(
        f"Combined: deposits=${combined_deposits:.2f}, withdrawals=${combined_withdrawals:.2f}, balance=${combined_balance:.2f}"
    )
    debug_log.append(
        f"Calculated: net_deposits=${net_deposits:.2f}, actual_hedging=${actual_hedging:.2f}"
    )
    has_mt5_data = True
else:
    debug_log.append("MT5 Account: NONE")
    has_mt5_data = False

# Process deals if available (for trade history tracking only, not hedging review)
if mt5_deals and len(mt5_deals) > 0:
    deal_types_seen = set()
    balance_count = 0
    trade_count = 0
    actual_profit = 0.0

    debug_log.append(f"MT5 Deals: {len(mt5_deals)} total")

    for deal in mt5_deals:
        # Handle both dict (serialized) and object
        if isinstance(deal, dict):
            d_type = deal.get('type')
            d_profit = float(deal.get('profit', 0.0) or 0.0)
            d_swap = float(deal.get('swap', 0.0) or 0.0)
            d_comm = float(deal.get('commission', 0.0) or 0.0)
        else:
            d_type = getattr(deal, 'type', None)
            d_profit = float(getattr(deal, 'profit', 0.0) or 0.0)
            d_swap = float(getattr(deal, 'swap', 0.0) or 0.0)
            d_comm = float(getattr(deal, 'commission', 0.0) or 0.0)

        deal_types_seen.add(str(d_type))

        # DEAL_TYPE_BALANCE = 2 or "BALANCE" (serialized from trader app)
        is_balance = d_type == 2 or str(d_type).upper() == "BALANCE"
        if is_balance:
            balance_count += 1
        else:
            trade_count += 1
            actual_profit += (d_profit + d_swap + d_comm)

    if not has_mt5_data:
        has_mt5_data = True

# Debug: store deal types seen for troubleshooting

```

```

debug_log.append(f" - Balance deals: {balance_count}, Trade deals: {trade_count}")
debug_log.append(f" - Trade profit from deals: ${actual_profit:.2f}")
debug_log.append(f" - Deal types seen: {list(deal_types_seen)}")

stats["hedging_review"]["_debug_deal_count"] = len(mt5_deals)
stats["hedging_review"]["_debug_deal_types"] = list(deal_types_seen)
stats["hedging_review"]["_debug_balance_count"] = balance_count
else:
    debug_log.append("MT5 Deals: NONE or empty")

# Print debug log to console/logs
print("\n[DEBUG] DATA_PROCESSOR DEBUG:")
for line in debug_log:
    print(f"    {line}")
print()

# --- Override cashflow with Stats tab values if available ---
# The XLSX Stats tab formula results are the authoritative values shown in Google Sheets.
# CSV export can produce slightly different values for cells containing formulas,
# so we override with the Stats tab values to guarantee an exact match.
stats_tab = (xlsx_notes or {}).get('__stats_tab__', {})
if stats_tab:
    cf = stats["cashflow_inprogress"]
    # Stats tab stores fees as negative; we store positive
    if 'challenge_fees' in stats_tab:
        cf['challenge_fees'] = abs(stats_tab['challenge_fees'])
    if 'hedging_results' in stats_tab:
        cf['hedging_results'] = stats_tab['hedging_results']
    if 'farming_results' in stats_tab:
        cf['farming_results'] = stats_tab['farming_results']
    if 'payouts' in stats_tab:
        cf['payouts'] = stats_tab['payouts']

    pc = stats["profitability_completed"]
    if 'prof_challenge_fees' in stats_tab:
        pc['challenge_fees'] = abs(stats_tab['prof_challenge_fees'])
    if 'prof_hedging_results' in stats_tab:
        pc['hedging_results'] = stats_tab['prof_hedging_results']
    if 'prof_farming_results' in stats_tab:
        pc['farming_results'] = stats_tab['prof_farming_results']
    if 'prof_payouts' in stats_tab:
        pc['payouts'] = stats_tab['prof_payouts']

    # Override EV with Stats tab value if available (but not if sheet returns 0 from IFERROR)
    if 'ev' in stats_tab and stats_tab['ev'] != 0:
        stats['expected_value'] = stats_tab['ev']

    # Recalculate Sheet Hedging Results from the (now-overridden) cashflow values
    # so it matches the Google Sheet's own formula (=Hedging Results + Farming Results)
    stats["hedging_review"]["sheet_hedging_results"] = (
        stats["cashflow_inprogress"]["hedging_results"] +
        stats["cashflow_inprogress"]["farming_results"]
    )
    stats["hedging_review"]["sheet_hedging_component"] = stats["cashflow_inprogress"]["hedging_results"]
    stats["hedging_review"]["sheet_farming_component"] = stats["cashflow_inprogress"]["farming_results"]

# --- Calculate Discrepancy ONCE from the final authoritative values ---
# This is the single source of truth displayed in the Hedging Review card.
# Discrepancy = Actual Hedging Results (from MT5) - Sheet Hedging Results (from Google Sheet)
if mt5_account:

```

```

stats["hedging_review"]["discrepancy"] = (
    stats["hedging_review"]["actual_hedging_results"] -
    stats["hedging_review"]["sheet_hedging_results"]
)

# --- Calculate Net Profit for each section ---
# Formula: Net Profit = Payouts + Hedging + Farming - Challenge Fees + Discrepancy
# Pick discrepancy directly from hedging_review (the value shown in the UI)
discrepancy = stats["hedging_review"]["discrepancy"]

for section in ["profitability_completed", "cashflow_inprogress"]:
    s = stats[section]
    # fees are stored positive but are expenses (subtract them)
    # Add discrepancy (B25) which adjusts for MT5 actual vs sheet recorded
    s["net_profit"] = s["payouts"] + s["hedging_results"] + s["farming_results"] - s[
        "challenge_fees"] + discrepancy

# Rounding
def round_dict(d):
    for k, v in d.items():
        if isinstance(v, float):
            d[k] = round(v, 2)
        elif isinstance(v, dict):
            round_dict(v)

round_dict(stats)
return stats

```

```
def extract_unique_values(data)
```

Extracts unique values for dropdown fields from the data. Merges with default options to ensure a good baseline.

Why these Python concepts were used

- **dict comprehension** — Builds a dict in one expression (`{k: v for k, v in items}`) — same intent as a list comprehension but for mappings.

Step by step (top-level body)

1. Assigns options = {'Prop Firm': {'My Funded Futures', 'FundedNext', 'Fundin....
2. **if** not data: branches conditionally.
3. **for** row in data: iterates.
4. **return** {k: sorted(list(v)) for k, v in options.items()}.

Code (copy-paste safe)

```

def extract_unique_values(data):
    """
    Extracts unique values for dropdown fields from the data.
    Merges with default options to ensure a good baseline.
    """
    # Default options (baseline)
    options = {
        'Prop Firm': {'My Funded Futures', 'FundedNext', 'Funding Ticks', 'Topstep', 'Lucid', 'TradeDay',
            'Alpha Futures', 'Tradeify', 'Top One Futures', 'Other'},
        'Account Size': {'$5,000', '$10,000', '$25,000', '$50,000', '$100,000', '$200,000'},
    }

```



```

        'Status': {'Active', 'Passed', 'Breached', 'Closed', 'Payout'}
    }

    if not data:
        return {k: sorted(list(v)) for k, v in options.items()}

    for row in data:
        # Prop Firm
        val = row.get('Prop Firm')
        if val: options['Prop Firm'].add(normalize_prop_firm(str(val).strip()))

        # Account Size - normalize to standard format
        val = row.get('Account Size')
        if val:
            normalized = normalize_account_size(val)
            options['Account Size'].add(normalized)

        # Status (P1)
        val = row.get('Status P1')
        if val: options['Status'].add(str(val).strip())

        # Status (Funded)
        val = row.get('Status')
        if val: options['Status'].add(str(val).strip())

    # Return sorted lists
    return {k: sorted(list(v)) for k, v in options.items()}

```

```
def group_deals_by_position(deals)
```

Groups a list of MT5 deals by position_id to form complete trades.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.
- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns positions = defaultdict(list).
2. for deal in deals: iterates.

3. Assigns trades = [].
4. for (pos_id, pos_deals) in positions.items(): iterates.
5. return trades.

Code (copy-paste safe)

```
def group_deals_by_position(deals):
    """
    Groups a list of MT5 deals by position_id to form complete trades.
    """
    positions = defaultdict(list)

    for deal in deals:
        # Access attributes safely (handle both object and dict)
        try:
            if isinstance(deal, dict):
                pos_id = deal.get('position_id')
            else:
                pos_id = getattr(deal, 'position_id', None)
        except:
            continue

        if pos_id is not None:
            positions[pos_id].append(deal)

    trades = []
    for pos_id, pos_deals in positions.items():
        if not pos_deals:
            continue

        # Sort by time to find entry and exit
        # Handle both object and dict access for 'time'
        pos_deals.sort(key=lambda x: x.get('time') if isinstance(x, dict) else getattr(x, 'time', 0))

        first_deal = pos_deals[0]
        last_deal = pos_deals[-1]

        # Helper to get value
        def get_val(obj, attr, default=0.0):
            if isinstance(obj, dict):
                return obj.get(attr, default)
            return getattr(obj, attr, default)

        # Aggregate values
        total_profit = sum(get_val(d, 'profit') for d in pos_deals)
        total_swap = sum(get_val(d, 'swap') for d in pos_deals)
        total_commission = sum(get_val(d, 'commission') for d in pos_deals)
        net_profit = total_profit + total_swap + total_commission

        # Basic info from first deal (Entry)
        symbol = get_val(first_deal, 'symbol', '')
        deal_type = get_val(first_deal, 'type', 0)
        volume = get_val(first_deal, 'volume', 0.0)
        open_time = get_val(first_deal, 'time', 0)
        open_price = get_val(first_deal, 'price', 0.0)

        # Exit info
        close_time = get_val(last_deal, 'time', 0)
        close_price = get_val(last_deal, 'price', 0.0)
```

```

trades.append({
    "position_id": pos_id,
    "symbol": symbol,
    "type": "BUY" if deal_type == 0 else "SELL",
    "volume": clean_float(volume),
    "open_time": datetime.fromtimestamp(open_time).strftime('%Y-%m-%d %H:%M'),
    "open_price": clean_float(open_price),
    "close_time": datetime.fromtimestamp(close_time).strftime('%Y-%m-%d %H:%M'),
    "close_price": clean_float(close_price),
    "net_profit": clean_float(round(net_profit, 2))
})

return trades

```

```
def serialize_positions(positions)
```

Converts MT5 position objects to dictionaries.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns data = [].
2. if positions is None: branches conditionally.
3. for pos in positions: iterates.
4. return data.

Code (copy-paste safe)

```

def serialize_positions(positions):
    """
    Converts MT5 position objects to dictionaries.
    """
    data = []
    if positions is None:
        return data

    for pos in positions:
        try:
            data.append({
                "ticket": getattr(pos, 'ticket', 0),
                "symbol": getattr(pos, 'symbol', ''),
                "type": "BUY" if getattr(pos, 'type', 0) == 0 else "SELL",
                "volume": clean_float(getattr(pos, 'volume', 0.0)),
                "open_price": clean_float(getattr(pos, 'price_open', 0.0)),
                "current_price": clean_float(getattr(pos, 'price_current', 0.0)),
                "sl": clean_float(getattr(pos, 'sl', 0.0)),
                "tp": clean_float(getattr(pos, 'tp', 0.0)),
                "swap": clean_float(getattr(pos, 'swap', 0.0)),
                "profit": clean_float(getattr(pos, 'profit', 0.0))
            })
        except:
            pass
    return data

```

```

    })
except:
    continue
return data

```

```
def serialize_account_info(account)
```

Converts MT5 account info object to dictionary.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

Step by step (top-level body)

1. **if** account is None: branches conditionally.
2. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def serialize_account_info(account):
    """
    Converts MT5 account info object to dictionary.
    """
    if account is None:
        return {}

    try:
        return {
            "login": getattr(account, 'login', 0),
            "balance": clean_float(getattr(account, 'balance', 0.0)),
            "equity": clean_float(getattr(account, 'equity', 0.0)),
            "profit": clean_float(getattr(account, 'profit', 0.0)),
            "margin": clean_float(getattr(account, 'margin', 0.0)),
            "margin_free": clean_float(getattr(account, 'margin_free', 0.0)),
            "margin_level": clean_float(getattr(account, 'margin_level', 0.0))
        }
    except:
        return {}

```

Checkpoint — what works now. You have a configured project skeleton. Nothing runs yet — the foundations layer is pure data and helpers — but every later phase imports from these modules. Import them in a Python REPL (`python -c "from config.settings import *; print('OK')"`) to confirm there are no syntax errors before moving on.

Chapter 4 — Phase 2 — Persistence: the database layer

Trading data has to live somewhere — accounts, payouts, watermarks, deals, notes. We use PostgreSQL in production and SQLite for development, talking to both through SQLAlchemy. Alembic owns the schema: each version file describes a forward step ("add this column") and a rollback step ("drop it"). In this phase we wire the engine, declare the ORM models that mirror the tables, and scaffold the first two Alembic revisions.

Files we build in this phase, in order:

- alembic/env.py
- alembic/versions/44e368d8bfce_initial_schema.py
- alembic/versions/5b29b54b57fa_add_firm_billing_column.py
- dashboard/database.py
- dashboard/db.py
- dashboard/models.py

Python concepts you'll meet in this phase

- Type hints (4) · Context managers (with-statements) (2) · Generators and yield (2) · Classes and objects (2) · f-strings (2) · Exceptions (2) · Dunder methods (1) · Properties (1)

Each is explained in Chapter 2 (the primer). The number in parentheses is how many of this phase's files use that concept.

Step 4.1 — Build alembic/env.py

What to do. Create the file alembic/env.py in your project root and paste in the code shown in this step. The file is **85** lines long; we walk through it piece by piece below.

Depends on. This file imports from internal modules built in earlier steps: alembic, dashboard. If any of those imports fail, jump back to the step that introduced them.

What this file is for. Alembic environment hook. Configures the migration runner.

alembic/env.py

85 loc · 0 classes · 2 functions · 7 imports

Alembic environment hook. Configures the migration runner. It defines **2** module-level functions, across **85** lines.

Python concepts demonstrated in this module

- Context managers (with-statements)
- Type hints

Imports — what each one is for

```
import os
```

Why imported. Operating-system interface. Path manipulation, environment variables, working-directory operations, exit codes, and thin wrappers around POSIX/Windows syscalls.

Used in this file. `os.environ`

Example. `os.path.join(root, 'subdir', 'file.txt')` # joins with the OS-correct separator

Other common scenarios.

- `os.environ.get('API_KEY')` to read environment variables
- `os.makedirs(path, exist_ok=True)` to create nested directories
- `os.listdir(path)` to list a directory's contents
- `os.remove(path)` / `os.rename(src, dst)` to delete or move a file
- `os.getcwd()` / `os.chdir(path)` to inspect or change the working dir

```
from logging.config import fileConfig
```

Why imported. Structured, levelled logging. Replaces `print()` with filterable, formattable output that can route to files, stderr, syslog, or HTTP endpoints.

Used in this file. `fileConfig`

Example. `logging.getLogger(__name__).info('connected to %s', host)`

Other common scenarios.

- `.debug()` / `.info()` / `.warning()` / `.error()` / `.exception()`
- `logger.exception()` captures the current traceback automatically
- `logging.basicConfig(level=logging.INFO)` for quick setup
- Handlers and Formatters route logs to files / streams / syslog

```
from dotenv import load_dotenv
```

Why imported. `python-dotenv` — load `.env` files into `os.environ` for twelve-factor configuration in development.

Used in this file. `load_dotenv`

Example. `load_dotenv()` # call once at process start

Other common scenarios.

- `dotenv_values('.env')` returns a dict without polluting `os.environ`
- Use a real secret store in production (vault, AWS SM, etc.)

```
from sqlalchemy import engine_from_config
```

```
from sqlalchemy import pool
```

Why imported. Python's most popular ORM and SQL toolkit. Models declare tables as classes; the engine handles connection pooling.

Used in this file. `engine_from_config`, `pool`

Example. `session.query(User).filter_by(email=e).first()`

Other common scenarios.

- Column / relationship / ForeignKey declare schema
- `session.add(obj)`; `session.commit()`
- with `engine.begin()` as `conn`: `conn.execute(text('SQL'))`
- 2.0-style: `select(User).where(User.id == 1)`

```
from alembic import context
```

Why imported. Migration framework that pairs with SQLAlchemy. Each version file describes a forward + rollback step.

Used in this file. `context`

Example. `alembic upgrade head` # apply all pending migrations

Other common scenarios.

- `alembic revision --autogenerate -m 'msg'` diffs models vs DB
- `op.add_column` / `op.drop_column` / `op.alter_column` inside a version

```
from dashboard.models import Base
```

Why imported. Internal package — the Flask dashboard. See chapter on dashboard/.

Used in this file. `Base`

Functions

```
def run_migrations_offline() -> None
```

Run migrations in 'offline' mode. This configures the context with just a URL and not an Engine, though an Engine is acceptable here as well. By skipping the Engine creation we don't even need a DBAPI to be available. Calls to `context.execute()` here emit the given string to the script output.

Why these Python concepts were used

- **return type annotation** — Annotated return type `-> None`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.

Step by step (top-level body)

1. Assigns `url = config.get_main_option('sqlalchemy.url')`.
2. Calls `context.configure(...)` for its side effect.
3. **with** `context.begin_transaction()`: enters a context manager.

Code (copy-paste safe)

```
def run_migrations_offline() -> None:
    """Run migrations in 'offline' mode.

    This configures the context with just a URL
    and not an Engine, though an Engine is acceptable
    here as well. By skipping the Engine creation
    we don't even need a DBAPI to be available.

    Calls to context.execute() here emit the given string to the
    script output.

    """
    url = config.get_main_option("sqlalchemy.url")
    context.configure(
        url=url,
        target_metadata=target_metadata,
        literal_binds=True,
        dialect_opts={"paramstyle": "named"},
    )

    with context.begin_transaction():
        context.run_migrations()

def run_migrations_online() -> None
```

Run migrations in 'online' mode. In this scenario we need to create an Engine and associate a connection with the context.

Why these Python concepts were used

- **return type annotation** — Annotated return type -> None. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.

Step by step (top-level body)

1. Assigns `connectable = engine_from_config(config.get_section(config.config_ini_s....`
2. **with** `connectable.connect()`: enters a context manager.

Code (copy-paste safe)

```
def run_migrations_online() -> None:
    """Run migrations in 'online' mode.

    In this scenario we need to create an Engine
    and associate a connection with the context.

    """
    connectable = engine_from_config(
        config.get_section(config.config_ini_section, {}),
        prefix="sqlalchemy.",
        poolclass=pool.NullPool,
    )

    with connectable.connect() as connection:
```



```

context.configure(
    connection=connection, target_metadata=target_metadata
)

with context.begin_transaction():
    context.run_migrations()

```

Step 4.2 — Build alembic/versions/44e368d8bfce_initial_schema.py

What to do. Create the file `alembic/versions/44e368d8bfce_initial_schema.py` in your project root and paste in the code shown in this step. The file is **266** lines long; we walk through it piece by piece below.

Depends on. This file imports from internal modules built in earlier steps: `alembic`. If any of those imports fail, jump back to the step that introduced them.

alembic/versions/44e368d8bfce_initial_schema.py

266 loc · 0 classes · 2 functions · 3 imports

It defines **2** module-level functions, across **266** lines. Module docstring: *initial_schema*

Module docstring

`initial_schema`

Revision ID: 44e368d8bfce Revises: Create Date: 2026-04-04 11:49:23.036333

Python concepts demonstrated in this module

- Type hints

Imports — what each one is for

```

from typing import Sequence
from typing import Union

```

Why imported. Type-hint primitives — generics, unions, `Optional`, `Callable`, `Protocol`, `TypeVar`, `TypedDict`.

Used in this file. `Sequence`, `Union`

Example. `def lookup(k: str) -> Optional[int]: ...`

Other common scenarios.

- `List[int]` / `Dict[str, Any]` / `Tuple[int, str]` (or bare `list[int]`+ on 3.9+)
- `Callable[[int, int], int]` for function signatures
- `Protocol` for structural typing (duck typing with checks)

- `TypeVar('T')` for generic functions and classes

```
from alembic import op
```

Why imported. Migration framework that pairs with SQLAlchemy. Each version file describes a forward + rollback step.

Used in this file. `op`

Example. `alembic upgrade head # apply all pending migrations`

Other common scenarios.

- `alembic revision --autogenerate -m 'msg'` diffs models vs DB
- `op.add_column` / `op.drop_column` / `op.alter_column` inside a version

```
import sqlalchemy as sa
```

Why imported. Python's most popular ORM and SQL toolkit. Models declare tables as classes; the engine handles connection pooling.

Used in this file. `sa.Column`, `sa.Float`, `sa.ForeignKeyConstraint`, `sa.Integer`, `sa.PrimaryKeyConstraint`, `sa.SmallInteger`, `sa.Text`, `sa.UniqueConstraint`

Example. `session.query(User).filter_by(email=e).first()`

Other common scenarios.

- `Column` / `relationship` / `ForeignKey` declare schema
- `session.add(obj); session.commit()`
- `with engine.begin() as conn: conn.execute(text('SQL'))`
- 2.0-style: `select(User).where(User.id == 1)`

Functions

```
def upgrade() -> None
```

Upgrade schema.

Why these Python concepts were used

- **return type annotation** — Annotated return type `-> None`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.

Step by step (top-level body)

1. Calls `op.create_table(...)` for its side effect.
2. Calls `op.create_table(...)` for its side effect.
3. Calls `op.create_table(...)` for its side effect.
4. Calls `op.create_table(...)` for its side effect.
5. Calls `op.create_table(...)` for its side effect.
6. Calls `op.create_table(...)` for its side effect.
7. Calls `op.create_index(...)` for its side effect.

8. Calls `op.create_index(...)` for its side effect.
9. Calls `op.create_table(...)` for its side effect.
10. Calls `op.create_table(...)` for its side effect.
11. Calls `op.create_index(...)` for its side effect.
12. Calls `op.create_table(...)` for its side effect.
13. Calls `op.create_table(...)` for its side effect.
14. Calls `op.create_index(...)` for its side effect.
15. ... and 9 more statement(s) in the body.

Code (copy-paste safe)

```
def upgrade() -> None:
    """Upgrade schema."""
    # ### commands auto generated by Alembic - please adjust! ###
    op.create_table('admin_passwords',
        sa.Column('id', sa.Integer(), autoincrement=True, nullable=False),
        sa.Column('username', sa.Text(), nullable=False),
        sa.Column('password_hash', sa.Text(), nullable=False),
        sa.Column('salt', sa.Text(), nullable=False),
        sa.Column('created_at', sa.Text(), nullable=False),
        sa.Column('updated_at', sa.Text(), nullable=True),
        sa.PrimaryKeyConstraint('id'),
        sa.UniqueConstraint('username')
    )
    op.create_table('api_keys',
        sa.Column('id', sa.Integer(), autoincrement=True, nullable=False),
        sa.Column('key_hash', sa.Text(), nullable=False),
        sa.Column('key_prefix', sa.Text(), nullable=False),
        sa.Column('admin', sa.Text(), nullable=False),
        sa.Column('trader', sa.Text(), nullable=False),
        sa.Column('client', sa.Text(), server_default=sa.text(''), nullable=True),
        sa.Column('scope', sa.Text(), server_default=sa.text('full'), nullable=True),
        sa.Column('created_at', sa.Text(), nullable=False),
        sa.Column('last_used', sa.Text(), nullable=True),
        sa.Column('is_active', sa.SmallInteger(), server_default=sa.text('1'), nullable=True),
        sa.PrimaryKeyConstraint('id'),
        sa.UniqueConstraint('key_hash')
    )
    op.create_table('audit_log',
        sa.Column('id', sa.Integer(), autoincrement=True, nullable=False),
        sa.Column('timestamp', sa.Text(), nullable=False),
        sa.Column('action', sa.Text(), nullable=False),
        sa.Column('user_type', sa.Text(), nullable=False),
        sa.Column('user_identifier', sa.Text(), nullable=False),
        sa.Column('ip_address', sa.Text(), nullable=True),
        sa.Column('details', sa.Text(), nullable=True),
        sa.Column('success', sa.SmallInteger(), server_default=sa.text('1'), nullable=True),
        sa.PrimaryKeyConstraint('id')
    )
    op.create_table('cell_notes',
        sa.Column('id', sa.Integer(), autoincrement=True, nullable=False),
        sa.Column('client_id', sa.Text(), nullable=False),
        sa.Column('row_index', sa.Integer(), nullable=False),
        sa.Column('column_key', sa.Text(), nullable=False),
        sa.Column('note_content', sa.Text(), nullable=True),
```

```

sa.Column('created_by', sa.Text(), nullable=True),
sa.Column('updated_at', sa.Text(), nullable=True),
sa.PrimaryKeyConstraint('id'),
sa.UniqueConstraint('client_id', 'row_index', 'column_key', name='uq_cell_notes')
)
op.create_table('clients_data',
sa.Column('id', sa.Integer(), autoincrement=True, nullable=False),
sa.Column('client_id', sa.Text(), nullable=False),
sa.Column('deals', sa.Text(), server_default=sa.text('[]'), nullable=True),
sa.Column('positions', sa.Text(), server_default=sa.text('[]'), nullable=True),
sa.Column('account', sa.Text(), server_default=sa.text('{}'), nullable=True),
sa.Column('evaluations', sa.Text(), server_default=sa.text('[]'), nullable=True),
sa.Column('statistics', sa.Text(), server_default=sa.text('{}'), nullable=True),
sa.Column('dropdown_options', sa.Text(), server_default=sa.text('{}'), nullable=True),
sa.Column('identity', sa.Text(), server_default=sa.text('{}'), nullable=True),
sa.Column('last_updated', sa.Text(), nullable=False),
sa.Column('hedge_accounts', sa.Text(), server_default=sa.text('[]'), nullable=True),
sa.Column('prop_accounts', sa.Text(), server_default=sa.text('[]'), nullable=True),
sa.Column('vps_accounts', sa.Text(), server_default=sa.text('[]'), nullable=True),
sa.Column('payment_info', sa.Text(), server_default=sa.text('[]'), nullable=True),
sa.Column('payment_address', sa.Text(), server_default=sa.text('{}'), nullable=True),
sa.PrimaryKeyConstraint('id'),
sa.UniqueConstraint('client_id')
)
op.create_table('daily_checklists',
sa.Column('id', sa.Integer(), autoincrement=True, nullable=False),
sa.Column('date', sa.Text(), nullable=False),
sa.Column('user_identfier', sa.Text(), nullable=False),
sa.Column('user_type', sa.Text(), nullable=False),
sa.Column('checklist_type', sa.Text(), nullable=False),
sa.Column('client_id', sa.Text(), server_default=sa.text(''), nullable=True),
sa.Column('items', sa.Text(), server_default=sa.text('[]'), nullable=True),
sa.Column('submitted_at', sa.Text(), nullable=False),
sa.Column('ip_address', sa.Text(), nullable=True),
sa.PrimaryKeyConstraint('id'),
sa.UniqueConstraint('date', 'user_identfier', 'checklist_type', 'client_id', name='uq_daily_checklists')
)
op.create_index('idx_checklist_client', 'daily_checklists', ['date', 'client_id'], unique=False)
op.create_index('idx_checklist_date', 'daily_checklists', ['date', 'user_identfier'], unique=False)
op.create_table('daily_watermarks',
sa.Column('client_id', sa.Text(), nullable=False),
sa.Column('date', sa.Text(), nullable=False),
sa.Column('net_profit_complete', sa.Float(), server_default=sa.text('0.0'), nullable=True),
sa.Column('source', sa.Text(), server_default=sa.text('auto'), nullable=True),
sa.Column('created_at', sa.Text(), server_default=sa.text('now()'), nullable=True),
sa.PrimaryKeyConstraint('client_id', 'date')
)
op.create_table('data_history',
sa.Column('id', sa.Integer(), autoincrement=True, nullable=False),
sa.Column('client_id', sa.Text(), nullable=False),
sa.Column('version', sa.Integer(), nullable=False),
sa.Column('action', sa.Text(), nullable=False),
sa.Column('changed_by', sa.Text(), nullable=True),
sa.Column('changed_by_type', sa.Text(), nullable=True),
sa.Column('ip_address', sa.Text(), nullable=True),
sa.Column('change_source', sa.Text(), nullable=True),
sa.Column('change_description', sa.Text(), nullable=True),
sa.Column('deals', sa.Text(), server_default=sa.text('[]'), nullable=True),
sa.Column('positions', sa.Text(), server_default=sa.text('[]'), nullable=True),
sa.Column('account', sa.Text(), server_default=sa.text('{}'), nullable=True),

```

```

sa.Column('evaluations', sa.Text(), server_default=sa.text('[]'), nullable=True),
sa.Column('statistics', sa.Text(), server_default=sa.text('{}'), nullable=True),
sa.Column('dropdown_options', sa.Text(), server_default=sa.text('{}'), nullable=True),
sa.Column('identity', sa.Text(), server_default=sa.text('{}'), nullable=True),
sa.Column('created_at', sa.Text(), nullable=False),
sa.PrimaryKeyConstraint('id'),
sa.UniqueConstraint('client_id', 'version', name='uq_data_history_client_version')
)
op.create_index('idx_data_history_client', 'data_history', ['client_id', 'version'], unique=False)
op.create_table('evaluations',
sa.Column('id', sa.Integer(), autoincrement=True, nullable=False),
sa.Column('account_signature', sa.Text(), nullable=False),
sa.Column('phase_number', sa.Integer(), nullable=False),
sa.Column('phase_type', sa.Text(), nullable=False),
sa.Column('status', sa.Text(), server_default=sa.text('pending'), nullable=True),
sa.Column('start_date', sa.Text(), nullable=True),
sa.Column('end_date', sa.Text(), nullable=True),
sa.Column('reset_id', sa.Text(), nullable=True),
sa.Column('parent_id', sa.Integer(), nullable=True),
sa.Column('meta_data', sa.Text(), server_default=sa.text('{}'), nullable=True),
sa.Column('created_at', sa.Text(), server_default=sa.text('now()'), nullable=True),
sa.ForeignKeyConstraint(['parent_id'], ['evaluations.id'], ),
sa.PrimaryKeyConstraint('id')
)
op.create_table('kyc_links',
sa.Column('id', sa.Integer(), autoincrement=True, nullable=False),
sa.Column('primary_client', sa.Text(), nullable=False),
sa.Column('linked_client', sa.Text(), nullable=False),
sa.Column('linked_by', sa.Text(), server_default=sa.text('super_admin'), nullable=True),
sa.Column('created_at', sa.Text(), server_default=sa.text('now()'), nullable=True),
sa.PrimaryKeyConstraint('id'),
sa.UniqueConstraint('primary_client', 'linked_client', name='uq_kyc_links')
)
op.create_index('idx_kyc_linked', 'kyc_links', ['linked_client'], unique=False)
op.create_index('idx_kyc_primary', 'kyc_links', ['primary_client'], unique=False)
op.create_table('login_attempts',
sa.Column('id', sa.Integer(), autoincrement=True, nullable=False),
sa.Column('username', sa.Text(), nullable=False),
sa.Column('user_type', sa.Text(), nullable=False),
sa.Column('ip_address', sa.Text(), nullable=True),
sa.Column('attempt_time', sa.Text(), nullable=False),
sa.Column('success', sa.SmallInteger(), server_default=sa.text('0'), nullable=True),
sa.PrimaryKeyConstraint('id')
)
op.create_table('phase_definitions',
sa.Column('id', sa.Integer(), autoincrement=True, nullable=False),
sa.Column('phase_name', sa.Text(), nullable=False),
sa.Column('phase_code', sa.Text(), nullable=False),
sa.Column('sequence_order', sa.Integer(), nullable=False),
sa.Column('ruleset', sa.Text(), server_default=sa.text('{}'), nullable=True),
sa.Column('next_phase_code', sa.Text(), nullable=True),
sa.PrimaryKeyConstraint('id'),
sa.UniqueConstraint('phase_code')
)
op.create_table('quality_scan_results',
sa.Column('id', sa.Integer(), autoincrement=True, nullable=False),
sa.Column('scan_date', sa.Text(), nullable=False),
sa.Column('client_id', sa.Text(), nullable=False),
sa.Column('trader', sa.Text(), nullable=True),
sa.Column('admin', sa.Text(), nullable=True),

```

```

sa.Column('total_issues', sa.Integer(), server_default=sa.text('0'), nullable=True),
sa.Column('issues', sa.Text(), server_default=sa.text('[]'), nullable=True),
sa.Column('health_score', sa.Float(), server_default=sa.text('100.0'), nullable=True),
sa.Column('created_at', sa.Text(), server_default=sa.text('now()'), nullable=True),
sa.PrimaryKeyConstraint('id')
)
op.create_index('idx_quality_scan_date', 'quality_scan_results', ['scan_date', 'client_id'], unique=True)
op.create_table('sessions',
sa.Column('id', sa.Integer(), autoincrement=True, nullable=False),
sa.Column('session_token', sa.Text(), nullable=False),
sa.Column('user_type', sa.Text(), nullable=False),
sa.Column('user_identifier', sa.Text(), nullable=False),
sa.Column('created_at', sa.Text(), nullable=False),
sa.Column('expires_at', sa.Text(), nullable=False),
sa.Column('ip_address', sa.Text(), nullable=True),
sa.PrimaryKeyConstraint('id'),
sa.UniqueConstraint('session_token')
)
op.create_table('system_settings',
sa.Column('key', sa.Text(), nullable=False),
sa.Column('value', sa.Text(), nullable=False),
sa.Column('updated_at', sa.Text(), nullable=False),
sa.Column('updated_by', sa.Text(), server_default=sa.text(''), nullable=True),
sa.PrimaryKeyConstraint('key')
)
op.create_table('user_credentials',
sa.Column('id', sa.Integer(), autoincrement=True, nullable=False),
sa.Column('username', sa.Text(), nullable=False),
sa.Column('email', sa.Text(), nullable=True),
sa.Column('password_hash', sa.Text(), nullable=False),
sa.Column('salt', sa.Text(), nullable=False),
sa.Column('user_type', sa.Text(), nullable=False),
sa.Column('parent_admin', sa.Text(), nullable=True),
sa.Column('parent_trader', sa.Text(), nullable=True),
sa.Column('is_active', sa.SmallInteger(), server_default=sa.text('1'), nullable=True),
sa.Column('must_change_password', sa.SmallInteger(), server_default=sa.text('1'), nullable=True),
sa.Column('last_login', sa.Text(), nullable=True),
sa.Column('created_at', sa.Text(), nullable=False),
sa.Column('updated_at', sa.Text(), nullable=True),
sa.PrimaryKeyConstraint('id'),
sa.UniqueConstraint('username', 'user_type', name='uq_user_credentials_username_type')
)
op.create_table('waterlog_periods',
sa.Column('client_id', sa.Text(), nullable=False),
sa.Column('from_date', sa.Text(), nullable=False),
sa.Column('to_date', sa.Text(), nullable=False),
sa.Column('period_low', sa.Float(), nullable=True),
sa.Column('period_high', sa.Float(), nullable=True),
sa.Column('split_pct', sa.Integer(), server_default=sa.text('50'), nullable=True),
sa.PrimaryKeyConstraint('client_id', 'from_date')
)

```

```
def downgrade() -> None
```

Downgrade schema.

Why these Python concepts were used

- **return type annotation** — Annotated return type -> None. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.

Step by step (top-level body)

1. Calls `op.drop_table(...)` for its side effect.
2. Calls `op.drop_table(...)` for its side effect.
3. Calls `op.drop_table(...)` for its side effect.
4. Calls `op.drop_table(...)` for its side effect.
5. Calls `op.drop_index(...)` for its side effect.
6. Calls `op.drop_table(...)` for its side effect.
7. Calls `op.drop_table(...)` for its side effect.
8. Calls `op.drop_table(...)` for its side effect.
9. Calls `op.drop_index(...)` for its side effect.
10. Calls `op.drop_index(...)` for its side effect.
11. Calls `op.drop_table(...)` for its side effect.
12. Calls `op.drop_table(...)` for its side effect.
13. Calls `op.drop_index(...)` for its side effect.
14. Calls `op.drop_table(...)` for its side effect.
15. ... and 9 more statement(s) in the body.

Code (copy-paste safe)

```
def downgrade() -> None:
    """Downgrade schema."""
    # ### commands auto generated by Alembic - please adjust! ###
    op.drop_table('waterlog_periods')
    op.drop_table('user_credentials')
    op.drop_table('system_settings')
    op.drop_table('sessions')
    op.drop_index('idx_quality_scan_date', table_name='quality_scan_results')
    op.drop_table('quality_scan_results')
    op.drop_table('phase_definitions')
    op.drop_table('login_attempts')
    op.drop_index('idx_kyc_primary', table_name='kyc_links')
    op.drop_index('idx_kyc_linked', table_name='kyc_links')
    op.drop_table('kyc_links')
    op.drop_table('evaluations')
    op.drop_index('idx_data_history_client', table_name='data_history')
    op.drop_table('data_history')
    op.drop_table('daily_watermarks')
    op.drop_index('idx_checklist_date', table_name='daily_checklists')
    op.drop_index('idx_checklist_client', table_name='daily_checklists')
    op.drop_table('daily_checklists')
    op.drop_table('clients_data')
    op.drop_table('cell_notes')
    op.drop_table('audit_log')
    op.drop_table('api_keys')
    op.drop_table('admin_passwords')
```

Step 4.3 — Build alembic/versions/5b29b54b57fa_add_firm_billing_column.py

What to do. Create the file `alembic/versions/5b29b54b57fa_add_firm_billing_column.py` in your project root and paste in the code shown in this step. The file is **31** lines long; we walk through it piece by piece below.

Depends on. This file imports from internal modules built in earlier steps: `alembic`. If any of those imports fail, jump back to the step that introduced them.

`alembic/versions/5b29b54b57fa_add_firm_billing_column.py`

31 loc · 0 classes · 2 functions · 3 imports

It defines **2** module-level functions, across **31** lines. Module docstring: `add_firm_billing_column`

Module docstring

`add_firm_billing_column`

Revision ID: 5b29b54b57fa Revises: 44e368d8bfce Create Date: 2026-04-14 17:23:49.376388

Python concepts demonstrated in this module

- Type hints

Imports — what each one is for

```
from typing import Sequence
from typing import Union
```

Why imported. Type-hint primitives — generics, unions, `Optional`, `Callable`, `Protocol`, `TypeVar`, `TypedDict`.

Used in this file. `Sequence`, `Union`

Example. `def lookup(k: str) -> Optional[int]: ...`

Other common scenarios.

- `List[int]` / `Dict[str, Any]` / `Tuple[int, str]` (or bare `list[int]`+ on 3.9+)
- `Callable[[int, int], int]` for function signatures
- `Protocol` for structural typing (duck typing with checks)
- `TypeVar('T')` for generic functions and classes

```
from alembic import op
```

Why imported. Migration framework that pairs with `SQLAlchemy`. Each version file describes a forward + rollback step.

Used in this file. `op`

Example. `alembic upgrade head # apply all pending migrations`

Other common scenarios.

- `alembic revision --autogenerate -m 'msg'` diffs models vs DB
- `op.add_column / op.drop_column / op.alter_column` inside a version

```
import sqlalchemy as sa
```

Why imported. Python's most popular ORM and SQL toolkit. Models declare tables as classes; the engine handles connection pooling.

Used in this file. `sa.Column`, `sa.Text`

Example. `session.query(User).filter_by(email=e).first()`

Other common scenarios.

- `Column / relationship / ForeignKey` declare schema
- `session.add(obj); session.commit()`
- with `engine.begin()` as `conn: conn.execute(text('SQL'))`
- 2.0-style: `select(User).where(User.id == 1)`

Functions

```
def upgrade() -> None
```

Upgrade schema.

Why these Python concepts were used

- **return type annotation** — Annotated return type `-> None`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.

Step by step (top-level body)

1. Calls `op.add_column(...)` for its side effect.

Code (copy-paste safe)

```
def upgrade() -> None:
    """Upgrade schema."""
    op.add_column('clients_data', sa.Column('firm_billing', sa.Text(), server_default='{}'))
```

```
def downgrade() -> None
```

Downgrade schema.

Why these Python concepts were used

- **return type annotation** — Annotated return type `-> None`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.

Step by step (top-level body)

1. Calls `op.drop_column(...)` for its side effect.

2. `pass` (placeholder).

Code (copy-paste safe)

```
def downgrade() -> None:
    """Downgrade schema."""
    op.drop_column('clients_data', 'firm_billing')
    pass
```

Step 4.4 — Build dashboard/database.py

What to do. Create the file `dashboard/database.py` in your project root and paste in the code shown in this step. The file is **1940** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from `requirements.txt`.

What this file is for. Database engine and session factory. The single source of connections used across the app.

dashboard/database.py

1940 loc · 2 classes · 90 functions · 11 imports

Database engine and session factory. The single source of connections used across the app. It defines **2** classes and **90** module-level functions, across **1,940** lines. Module docstring: *PostgreSQL Database Module for Trading Dashboard*

Module docstring

PostgreSQL Database Module for Trading Dashboard Provides secure storage with encrypted data and audit logging.

Migrated from SQLite — uses `psycopg2` with compatibility wrappers so all existing query code (? placeholders, `row['col']` access) keeps working.

Python concepts demonstrated in this module

- Classes and objects
- Comprehensions
- Context managers (with-statements)
- Decorators
- Dunder methods
- Exceptions
- f-strings
- Generators and yield
- Module-level state
- Properties
- Type hints

Imports — what each one is for

```
import json
```

Why imported. JSON encoding and decoding — the standard wire format for config files, API payloads, and inter-process messages.

Used in this file. `json.dumps`, `json.load`, `json.loads`

Example. `json.loads(response.text)` # parse a JSON string to dict

Other common scenarios.

- `json.dumps(obj, indent=2)` for pretty-printing
- `json.dump(obj, file)` / `json.load(file)` for file I/O
- `default=str` handles non-serialisable types like `datetime`
- `object_hook=` customises decoding (e.g., to `dataclass` instances)

`import os`

Why imported. Operating-system interface. Path manipulation, environment variables, working-directory operations, exit codes, and thin wrappers around POSIX/Windows syscalls.

Used in this file. `os.environ`, `os.environ.get`, `os.path`, `os.path.abspath`, `os.path.dirname`, `os.path.exists`, `os.path.join`

Example. `os.path.join(root, 'subdir', 'file.txt')` # joins with the OS-correct separator

Other common scenarios.

- `os.environ.get('API_KEY')` to read environment variables
- `os.makedirs(path, exist_ok=True)` to create nested directories
- `os.listdir(path)` to list a directory's contents
- `os.remove(path)` / `os.rename(src, dst)` to delete or move a file
- `os.getcwd()` / `os.chdir(path)` to inspect or change the working dir

`import hashlib`

Why imported. Cryptographic hash functions — SHA-256, MD5, BLAKE2.

Used in this file. `hashlib.pbkdf2_hmac`, `hashlib.sha256`

Example. `hashlib.sha256(b'hello').hexdigest()`

Other common scenarios.

- Pass data in chunks via `.update()` to hash large files
- Use `sha256` / `sha512` for integrity; never MD5 or SHA-1 for security
- For password hashing use `bcrypt/argon2`, NOT raw `hashlib`

`import secrets`

Why imported. Cryptographically strong randomness — tokens, session IDs, API keys. Never use `random` for anything security-sensitive.

Used in this file. `secrets.compare_digest`, `secrets.token_hex`, `secrets.token_urlsafe`

Example. `secrets.token_urlsafe(32)` # 32-byte url-safe random string

Other common scenarios.

- `secrets.token_hex(n)` for a hex string
- `secrets.choice(seq)` is the cryptographic equivalent of `random.choice`
- `secrets.compare_digest(a, b)` for timing-safe comparison

```
import psycopg2
```

Why imported. PostgreSQL driver. Used directly for raw SQL paths and as the dialect SQLAlchemy talks through.

Used in this file. *No direct attribute access detected — likely imported for side effects, re-export, or string references.*

Example. `psycopg2.connect(dsn)`

Other common scenarios.

- Always use parameterised queries: `cur.execute('... WHERE id=%s', (id,))`
- `psycopg2.extras.RealDictCursor` returns dict-like rows
- On 3+ the modern replacement is `psycopg` (`psycopg3`)

```
import psycopg2.extras
```

Why imported. PostgreSQL driver. Used directly for raw SQL paths and as the dialect SQLAlchemy talks through.

Used in this file. *No direct attribute access detected — likely imported for side effects, re-export, or string references.*

Example. `psycopg2.connect(dsn)`

Other common scenarios.

- Always use parameterised queries: `cur.execute('... WHERE id=%s', (id,))`
- `psycopg2.extras.RealDictCursor` returns dict-like rows
- On 3+ the modern replacement is `psycopg` (`psycopg3`)

```
import psycopg2.pool
```

Why imported. PostgreSQL driver. Used directly for raw SQL paths and as the dialect SQLAlchemy talks through.

Used in this file. `psycopg2.IntegrityError`, `psycopg2.OperationalError`, `psycopg2.extras`, `psycopg2.extras.RealDictCursor`, `psycopg2.pool`, `psycopg2.pool.SimpleConnectionPool`

Example. `psycopg2.connect(dsn)`

Other common scenarios.

- Always use parameterised queries: `cur.execute('... WHERE id=%s', (id,))`
- `psycopg2.extras.RealDictCursor` returns dict-like rows
- On 3+ the modern replacement is `psycopg` (`psycopg3`)

```
import logging
```

Why imported. Structured, levelled logging. Replaces `print()` with filterable, formattable output that can route to files, stderr, syslog, or HTTP endpoints.

Used in this file. `logging.getLogger`, `logging.info`

Example. `logging.getLogger(__name__).info('connected to %s', host)`

Other common scenarios.

- `.debug()` / `.info()` / `.warning()` / `.error()` / `.exception()`
- `logger.exception()` captures the current traceback automatically
- `logging.basicConfig(level=logging.INFO)` for quick setup
- Handlers and Formatters route logs to files / streams / syslog

```
from datetime import datetime
from datetime import timedelta
```

Why imported. Dates, times, durations. The fundamental temporal types: `date`, `time`, `datetime`, `timedelta`, `timezone`.

Used in this file. `datetime`, `timedelta`

Example. `datetime.now() - timedelta(days=7)` # one week ago

Other common scenarios.

- `datetime.strptime(s, '%Y-%m-%d')` parses a string
- `datetime.strftime(fmt)` formats one
- `datetime.fromtimestamp(n)` converts a Unix epoch
- `datetime.utcnow()` for UTC (better: `datetime.now(timezone.utc)`)

```
from contextlib import contextmanager
```

Why imported. Context-manager helpers. `@contextmanager` turns a generator into a with-block; `suppress()` swallows specific exceptions.

Used in this file. `contextmanager`

Example. `@contextmanager\ndef timer():\n t=time.time(); yield; print(time.time()-t)`

Other common scenarios.

- `ExitStack` stacks dynamic context managers (close many at once)
- `suppress(FileNotFoundError)` ignores a specific exception
- `closing(obj)` wraps any `close()`-able as a context manager

```
from dotenv import load_dotenv
```

Why imported. `python-dotenv` — load `.env` files into `os.environ` for twelve-factor configuration in development.

Used in this file. `load_dotenv`

Example. `load_dotenv()` # call once at process start

Other common scenarios.

- `dotenv_values('.env')` returns a dict without polluting `os.environ`
- Use a real secret store in production (vault, AWS SM, etc.)

Module constants

Each line below is followed by a plain-English note explaining what it does and why it is written that way.

```
DATABASE_URL = os.environ.get('DATABASE_URL', 'postgresql://postgres:postgres123@localhost:5432/tradeopss')
```

Equivalent to `os.getenv`: reads `DATABASE_URL` from the environment, default `'postgresql://postgres:postgres123@localhost:5432/tradeopss'`.

Classes

```
class _PgCursorWrapper
```

Wraps psycopg2 RealDictCursor; translates ? → %s.

Code (copy-paste safe)

```
class _PgCursorWrapper:
    """Wraps psycopg2 RealDictCursor; translates ? → %s."""

    def __init__(self, cursor):
        self._cursor = cursor

    def execute(self, sql, params=None):
        sql = sql.replace('?', '%s')
        self._cursor.execute(sql, params)
        return self

    def executemany(self, sql, params_list):
        sql = sql.replace('?', '%s')
        self._cursor.executemany(sql, params_list)
        return self

    def fetchone(self):
        return self._cursor.fetchone()

    def fetchall(self):
        return self._cursor.fetchall()

    @property
    def rowcount(self):
        return self._cursor.rowcount

    @property
    def lastrowid(self):
        return getattr(self._cursor, 'lastrowid', None)
```

Method: `_PgCursorWrapper.__init__`

```
def __init__(self, cursor)
```

Why these Python concepts were used

- `__init__` — The constructor. Runs after Python has allocated the instance; assigns starting attribute values to `self`.

Step by step (top-level body)

1. Assigns `self._cursor = cursor`.

Code (copy-paste safe)

```
def __init__(self, cursor):
```

```
self._cursor = cursor
```

Method: `_PgCursorWrapper.execute`

```
def execute(self, sql, params=None)
```

Step by step (top-level body)

1. Assigns `sql = sql.replace('?', '%s')`.
2. Calls `self._cursor.execute(...)` for its side effect.
3. **return** `self`.

Code (copy-paste safe)

```
def execute(self, sql, params=None):
    sql = sql.replace('?', '%s')
    self._cursor.execute(sql, params)
    return self
```

Method: `_PgCursorWrapper.executemany`

```
def executemany(self, sql, params_list)
```

Step by step (top-level body)

1. Assigns `sql = sql.replace('?', '%s')`.
2. Calls `self._cursor.executemany(...)` for its side effect.
3. **return** `self`.

Code (copy-paste safe)

```
def executemany(self, sql, params_list):
    sql = sql.replace('?', '%s')
    self._cursor.executemany(sql, params_list)
    return self
```

Method: `_PgCursorWrapper.fetchone`

```
def fetchone(self)
```

Step by step (top-level body)

1. **return** `self._cursor.fetchone()`.

Code (copy-paste safe)

```
def fetchone(self):
    return self._cursor.fetchone()
```

Method: `_PgCursorWrapper.fetchall`

```
def fetchall(self)
```

Step by step (top-level body)

1. **return** `self._cursor.fetchall()`.

Code (copy-paste safe)

```
def fetchall(self):
    return self._cursor.fetchall()
```

Method: `_PgCursorWrapper.rowcount`

```
@property
def rowcount(self)
```

Why these Python concepts were used

- **@property** — Wrapped in **@property** so callers access the value with attribute syntax (`obj.x`) instead of a method call. Lets the implementation compute or validate the value lazily without changing the call site.

Step by step (top-level body)

1. **return** `self._cursor.rowcount`.

Code (copy-paste safe)

```
def rowcount(self):
    return self._cursor.rowcount
```

Method: `_PgCursorWrapper.lastrowid`

```
@property
def lastrowid(self)
```

Why these Python concepts were used

- **@property** — Wrapped in **@property** so callers access the value with attribute syntax (`obj.x`) instead of a method call. Lets the implementation compute or validate the value lazily without changing the call site.

Step by step (top-level body)

1. **return** `getattr(self._cursor, 'lastrowid', None)`.

Code (copy-paste safe)

```
def lastrowid(self):
    return getattr(self._cursor, 'lastrowid', None)
```

```
class _PgConnWrapper
```

Wraps a psycopg2 connection to match the sqlite3 interface used throughout.

Code (copy-paste safe)

```
class _PgConnWrapper:
    """Wraps a psycopg2 connection to match the sqlite3 interface used throughout."""

    def __init__(self, raw_conn):
        self._conn = raw_conn

    def cursor(self):
```



```

        return _PgCursorWrapper(
            self._conn.cursor(cursor_factory=psycopg2.extras.RealDictCursor)
        )

    def execute(self, sql, params=None):
        cur = self.cursor()
        cur.execute(sql, params)
        return cur

    def commit(self):
        self._conn.commit()

    def rollback(self):
        self._conn.rollback()

    def close(self):
        self._conn.close()

```

Method: `_PgConnWrapper.__init__`

```
def __init__(self, raw_conn)
```

Why these Python concepts were used

- `__init__` — The constructor. Runs after Python has allocated the instance; assigns starting attribute values to self.

Step by step (top-level body)

1. Assigns `self._conn = raw_conn`.

Code (copy-paste safe)

```
def __init__(self, raw_conn):
    self._conn = raw_conn
```

Method: `_PgConnWrapper.cursor`

```
def cursor(self)
```

Step by step (top-level body)

1. `return _PgCursorWrapper(self._conn.cursor(cursor_factory=psycopg2.extras.R....`

Code (copy-paste safe)

```
def cursor(self):
    return _PgCursorWrapper(
        self._conn.cursor(cursor_factory=psycopg2.extras.RealDictCursor)
    )
```

Method: `_PgConnWrapper.execute`

```
def execute(self, sql, params=None)
```

Step by step (top-level body)

1. Assigns `cur = self.cursor()`.
2. Calls `cur.execute(...)` for its side effect.

3. return cur.

Code (copy-paste safe)

```
def execute(self, sql, params=None):
    cur = self.cursor()
    cur.execute(sql, params)
    return cur
```

Method: _PgConnWrapper.commit

```
def commit(self)
```

Step by step (top-level body)

1. Calls self._conn.commit(...) for its side effect.

Code (copy-paste safe)

```
def commit(self):
    self._conn.commit()
```

Method: _PgConnWrapper.rollback

```
def rollback(self)
```

Step by step (top-level body)

1. Calls self._conn.rollback(...) for its side effect.

Code (copy-paste safe)

```
def rollback(self):
    self._conn.rollback()
```

Method: _PgConnWrapper.close

```
def close(self)
```

Step by step (top-level body)

1. Calls self._conn.close(...) for its side effect.

Code (copy-paste safe)

```
def close(self):
    self._conn.close()
```

Functions

```
def _init_pool()
```

Initialize the connection pool on first use.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't

have to handle it.

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.

- **global** — Rebinds a module-level name from inside the function. A smell in larger codebases — usually means a singleton or cache that could be passed in instead.

Step by step (top-level body)

1. Declares globals: `_connection_pool`.
2. **if** `_connection_pool` is `None`: branches conditionally.

Code (copy-paste safe)

```
def _init_pool():
    """Initialize the connection pool on first use."""
    global _connection_pool
    if _connection_pool is None:
        try:
            _connection_pool = psycopg2.pool.SimpleConnectionPool(
                5, 15, # min=5, max=15 connections
                DATABASE_URL,
                connect_timeout=5 # 5-second timeout per connection
            )
            logger.info("[DB] Connection pool initialized (5-15 connections)")
        except Exception as e:
            logger.error(f"[DB] Failed to initialize connection pool: {e}")
            raise
```

```
def _get_pooled_connection()
```

Get a connection from the pool (creates pool on first call).

Step by step (top-level body)

1. **if** `_connection_pool` is `None`: branches conditionally.
2. **return** `_connection_pool.getconn()`.

Code (copy-paste safe)

```
def _get_pooled_connection():
    """Get a connection from the pool (creates pool on first call)."""
    if _connection_pool is None:
        _init_pool()
    return _connection_pool.getconn()
```

```
def _return_pooled_connection(conn)
```

Return a connection to the pool.

Step by step (top-level body)

1. **if** `_connection_pool` is not `None`: branches conditionally (with an **else/elif** arm).

Code (copy-paste safe)

```
def _return_pooled_connection(conn):
    """Return a connection to the pool."""
    if _connection_pool is not None:
        _connection_pool.putconn(conn)
    else:
        conn.close()
```

```
@contextmanager
def get_connection()
```

Context manager for database connections (PostgreSQL with pooling).

Why these Python concepts were used

- **@contextmanager** — Turns a generator (which **yields** exactly once) into a context manager — the code before the yield runs on enter, the code after runs on exit. A concise alternative to writing `__enter__` / `__exit__` by hand.
- **generator (yield)** — The body uses **yield**, so calling it returns an iterator instead of a value. Items are produced lazily — the function only runs as far as the next **yield** on each iteration. Right choice when the caller iterates results and the dataset is large or open-ended.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.

Step by step (top-level body)

1. Assigns `raw = None`.
2. **try** block with 2 **except** clauses, plus a **finally**.

Code (copy-paste safe)

```
def get_connection():
    """Context manager for database connections (PostgreSQL with pooling)."""
    raw = None
    try:
        # Get connection from pool (creates pool on first call)
        raw = _get_pooled_connection()

        # Reset connection state (auto-commit off, no pending transactions)
        raw.autocommit = False
        raw.rollback() # Clear any stale state

        conn = _PgConnWrapper(raw)
        try:
            yield conn
            # Auto-commit on successful exit (safety net)
            raw.commit()
        except Exception as e:
```

```

        # Log and rollback on error
        logger.warning(f"[DB] Transaction error, rolling back: {e}")
        raw.rollback()
        raise
    except psycopg2.OperationalError as e:
        # Connection pool exhausted or DB unreachable
        logger.error(f"[DB] Database connection error (pool issue?): {e}")
        raise
    except Exception as e:
        logger.error(f"[DB] Unexpected error in get_connection: {e}")
        raise
    finally:
        # Always return connection to pool
        if raw is not None:
            _return_pooled_connection(raw)

```

```
def get_db_path()
```

Legacy helper — returns DATABASE_URL for PostgreSQL.

Step by step (top-level body)

1. **return** DATABASE_URL.

Code (copy-paste safe)

```

def get_db_path():
    """Legacy helper — returns DATABASE_URL for PostgreSQL."""
    return DATABASE_URL

```

```
def check_and_repair_database()
```

Connectivity check (PostgreSQL doesn't need SQLite-style repair).

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def check_and_repair_database():
    """Connectivity check (PostgreSQL doesn't need SQLite-style repair)."""
    try:
        with get_connection() as conn:
            conn.execute('SELECT 1')
            return True, 'PostgreSQL connection OK'
    except Exception as e:

```

```
return False, f'PostgreSQL connection failed: {e}'
```

```
def init_database()
```

Schema is managed by Alembic migrations — verify connectivity and ensure columns exist.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def init_database():
    """Schema is managed by Alembic migrations – verify connectivity and ensure columns exist."""
    try:
        with get_connection() as conn:
            conn.execute('SELECT 1')
            conn.commit()
        print("Database connection verified (schema managed by Alembic)")
        # Defensive: ensure columns exist even if Alembic migration was stamped on legacy DB
        with get_connection() as conn:
            cursor = conn.cursor()
            for _col, _default in [('prop_accounts', "[]"), ('vps_accounts', "[]"),
                                   ('hedge_accounts', "[]"), ('mt5_credentials', "{}"),
                                   ('payment_info', "[]"), ('payment_address', "{}")]:
                try:
                    cursor.execute(
                        f"ALTER TABLE clients_data ADD COLUMN IF NOT EXISTS {_col} TEXT DEFAULT {_default}"
                    )
                except Exception:
                    pass
            conn.commit()
    except Exception as e:
        print(f"Database connection failed: {e}")
```

```
def hash_password(password: str, salt: str=None) -> tuple
```

Hash a password with salt using SHA-256 + PBKDF2.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., password: str, salt: str). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type -> tuple. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.

Step by step (top-level body)

1. **if** salt is None: branches conditionally.
2. Assigns password_hash = hashlib.pbkdf2_hmac('sha256', password.encode('utf-8'), s....
3. **return** (password_hash, salt).

Code (copy-paste safe)

```
def hash_password(password: str, salt: str = None) -> tuple:
    """Hash a password with salt using SHA-256 + PBKDF2."""
    if salt is None:
        salt = secrets.token_hex(32)

    # Use PBKDF2 with SHA-256
    password_hash = hashlib.pbkdf2_hmac(
        'sha256',
        password.encode('utf-8'),
        salt.encode('utf-8'),
        100000 # 100,000 iterations
    ).hex()

    return password_hash, salt
```

```
def verify_password(password: str, stored_hash: str, salt: str) -> bool
```

Verify a password against stored hash.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., password: str, stored_hash: str, salt: str). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type -> bool. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.

Step by step (top-level body)

1. Assigns (password_hash, _) = hash_password(password, salt).
2. **return** secrets.compare_digest(password_hash, stored_hash).

Code (copy-paste safe)

```
def verify_password(password: str, stored_hash: str, salt: str) -> bool:
    """Verify a password against stored hash."""
    password_hash, _ = hash_password(password, salt)
    return secrets.compare_digest(password_hash, stored_hash)

def set_admin_password(username: str, password: str) -> bool

    Set or update admin password.
```

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `username: str, password: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> bool`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `(password_hash, salt) = hash_password(password)`.
2. Assigns `now = datetime.now().isoformat()`.
3. **with** `get_connection()`: enters a context manager.

Code (copy-paste safe)

```
def set_admin_password(username: str, password: str) -> bool:
    """Set or update admin password."""
    password_hash, salt = hash_password(password)
    now = datetime.now().isoformat()

    with get_connection() as conn:
        cursor = conn.cursor()
        try:
            cursor.execute('''
                INSERT INTO admin_passwords (username, password_hash, salt, created_at)
                VALUES (?, ?, ?, ?)
                ON CONFLICT(username) DO UPDATE SET
                    password_hash = excluded.password_hash,
                    salt = excluded.salt,
                    updated_at = ?
            ''', (username, password_hash, salt, now, now))
            conn.commit()
            return True
        except Exception as e:
            print(f"Error setting admin password: {e}")
            return False
```

```
def admin_password_exists(username: str) -> bool
```

Return True if a password row already exists for the given admin username.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `username: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.

- **return type annotation** — Annotated return type -> bool. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.

Step by step (top-level body)

1. **with** `get_connection()`: enters a context manager.

Code (copy-paste safe)

```
def admin_password_exists(username: str) -> bool:
    """Return True if a password row already exists for the given admin username."""
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute(
            'SELECT 1 FROM admin_passwords WHERE username = ?',
            (username,)
        )
        return cursor.fetchone() is not None
```

```
def copy_admin_password_row(from_username: str, to_username: str) -> bool
```

If to_username has no row, copy password_hash/salt from from_username. Returns True if a row was copied.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `from_username: str`, `to_username: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type -> bool. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.

Step by step (top-level body)

1. **if** `admin_password_exists(to_username)`: branches conditionally.
2. **with** `get_connection()`: enters a context manager.

Code (copy-paste safe)

```
def copy_admin_password_row(from_username: str, to_username: str) -> bool:
    """
    If to_username has no row, copy password_hash/salt from from_username.
    Returns True if a row was copied.
    """
    if admin_password_exists(to_username):
        return False
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute(
            'SELECT password_hash, salt FROM admin_passwords WHERE username = ?',
            (from_username,)
        )
```

```

row = cursor.fetchone()
if not row:
    return False
now = datetime.now().isoformat()
cursor.execute(
    '''
        INSERT INTO admin_passwords (username, password_hash, salt, created_at)
        VALUES (?, ?, ?, ?)
    ''',
    (to_username, row['password_hash'], row['salt'], now),
)
conn.commit()
return True

```

```
def verify_admin_password(username: str, password: str) -> bool
```

Verify admin password.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `username: str`, `password: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> bool`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.

Step by step (top-level body)

1. **with** `get_connection()`: enters a context manager.

Code (copy-paste safe)

```

def verify_admin_password(username: str, password: str) -> bool:
    """Verify admin password."""
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute(
            'SELECT password_hash, salt FROM admin_passwords WHERE username = ?',
            (username,)
        )
        row = cursor.fetchone()

        if row is None:
            return False

        return verify_password(password, row['password_hash'], row['salt'])

```

```

def create_user(username: str, password: str, user_type: str, email: str=None, parent_admin: str=None,
parent_trader: str=None) -> bool

```

Create a new user with hashed password.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., username: str, password: str, user_type: str). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type -> bool. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns username = username.strip().
2. Assigns (password_hash, salt) = hash_password(password).
3. Assigns now = datetime.now().isoformat().
4. **with** get_connection(): enters a context manager.

Code (copy-paste safe)

```
def create_user(username: str, password: str, user_type: str,
               email: str = None, parent_admin: str = None,
               parent_trader: str = None) -> bool:
    """Create a new user with hashed password."""
    username = username.strip()
    password_hash, salt = hash_password(password)
    now = datetime.now().isoformat()

    with get_connection() as conn:
        cursor = conn.cursor()
        try:
            cursor.execute('''
                INSERT INTO user_credentials
                (username, email, password_hash, salt, user_type, parent_admin, parent_trader, created_at)
                VALUES (?, ?, ?, ?, ?, ?, ?, ?)
            ''', (username, email, password_hash, salt, user_type, parent_admin, parent_trader, now))
            conn.commit()
            return True
        except psycopg2.IntegrityError:
            # User already exists
            conn.rollback()
            return False
        except Exception as e:
            print(f"Error creating user: {e}")
            return False

def verify_user_password(username: str, user_type: str, password: str) -> dict:
    Verify user password and return user info if valid.
```

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `username: str`, `user_type: str`, `password: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> dict`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.

Step by step (top-level body)

1. **with** `get_connection()`: enters a context manager.

Code (copy-paste safe)

```
def verify_user_password(username: str, user_type: str, password: str) -> dict:
    """Verify user password and return user info if valid."""
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute('''
            SELECT id, username, email, password_hash, salt, user_type,
                   parent_admin, parent_trader, is_active, must_change_password
            FROM user_credentials
            WHERE username = ? AND user_type = ? AND is_active = 1
        ''', (username, user_type))
        row = cursor.fetchone()

        if row is None:
            return None

        if not verify_password(password, row['password_hash'], row['salt']):
            return None

        # Update last login
        cursor.execute(
            'UPDATE user_credentials SET last_login = ? WHERE id = ?',
            (datetime.now().isoformat(), row['id'])
        )
        conn.commit()

        return {
            'id': row['id'],
            'username': row['username'],
            'email': row['email'],
            'user_type': row['user_type'],
            'parent_admin': row['parent_admin'],
            'parent_trader': row['parent_trader'],
            'must_change_password': bool(row['must_change_password'])
        }

def verify_client_login(email: str, password: str) -> dict
```

Verify client login by email and password.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., email: str, password: str). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type -> dict. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.

Step by step (top-level body)

1. **with get_connection():** enters a context manager.

Code (copy-paste safe)

```
def verify_client_login(email: str, password: str) -> dict:
    """Verify client login by email and password."""
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute('''
            SELECT id, username, email, password_hash, salt, user_type,
                   parent_admin, parent_trader, is_active, must_change_password
            FROM user_credentials
            WHERE email = ? AND user_type = 'client' AND is_active = 1
        ''', (email,))
        row = cursor.fetchone()

        if row is None:
            return None

        if not verify_password(password, row['password_hash'], row['salt']):
            return None

        # Update last login
        cursor.execute(
            'UPDATE user_credentials SET last_login = ? WHERE id = ?',
            (datetime.now().isoformat(), row['id'])
        )
        conn.commit()

    return {
        'id': row['id'],
        'username': row['username'],
        'email': row['email'],
        'user_type': row['user_type'],
        'parent_admin': row['parent_admin'],
        'parent_trader': row['parent_trader'],
        'must_change_password': bool(row['must_change_password'])
    }
```

```
def update_user_password(username: str, user_type: str, new_password: str) -> bool
```

Update a user's password.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., username: str, user_type: str, new_password: str). Hints are not enforced at runtime — they document intent for static checkers

(mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.

- **return type annotation** — Annotated return type -> bool. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.

Step by step (top-level body)

1. Assigns (password_hash, salt) = hash_password(new_password).
2. Assigns now = datetime.now().isoformat().
3. **with** get_connection(): enters a context manager.

Code (copy-paste safe)

```
def update_user_password(username: str, user_type: str, new_password: str) -> bool:
    """Update a user's password."""
    password_hash, salt = hash_password(new_password)
    now = datetime.now().isoformat()

    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute('''
            UPDATE user_credentials
            SET password_hash = ?, salt = ?, must_change_password = 0, updated_at = ?
            WHERE username = ? AND user_type = ?
        ''', (password_hash, salt, now, username, user_type))
        conn.commit()
        return cursor.rowcount > 0
```

```
def get_user(username: str, user_type: str) -> dict
```

Get user info without password verification.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., username: str, user_type: str). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type -> dict. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **with** get_connection(): enters a context manager.

Code (copy-paste safe)

```
def get_user(username: str, user_type: str) -> dict:
```

```

"""Get user info without password verification."""
with get_connection() as conn:
    cursor = conn.cursor()
    cursor.execute('''
        SELECT id, username, email, user_type, parent_admin, parent_trader,
               is_active, must_change_password, last_login, created_at
        FROM user_credentials
        WHERE username = ? AND user_type = ?
    ''', (username, user_type))
    row = cursor.fetchone()
    return dict(row) if row else None

```

```
def list_users(user_type: str=None) -> list
```

List all users, optionally filtered by type.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `user_type: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> list`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with `append`, and clearer about intent: this is a transform, not a side-effecting loop.

Step by step (top-level body)

1. **with** `get_connection()`: enters a context manager.

Code (copy-paste safe)

```

def list_users(user_type: str = None) -> list:
    """List all users, optionally filtered by type."""
    with get_connection() as conn:
        cursor = conn.cursor()
        if user_type:
            cursor.execute('''
                SELECT id, username, email, user_type, parent_admin, parent_trader,
                       is_active, last_login, created_at
                FROM user_credentials WHERE user_type = ?
                ORDER BY created_at DESC
            ''', (user_type,))
        else:
            cursor.execute('''
                SELECT id, username, email, user_type, parent_admin, parent_trader,
                       is_active, last_login, created_at
                FROM user_credentials ORDER BY user_type, created_at DESC
            ''')
        return [dict(row) for row in cursor.fetchall()]

```

```
def deactivate_user(username: str, user_type: str) -> bool
```

Deactivate a user account.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `username: str, user_type: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> bool`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.

Step by step (top-level body)

1. **with** `get_connection()`: enters a context manager.

Code (copy-paste safe)

```
def deactivate_user(username: str, user_type: str) -> bool:
    """Deactivate a user account."""
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute(''
            UPDATE user_credentials SET is_active = 0, updated_at = ?
            WHERE username = ? AND user_type = ?
            ''', (datetime.now().isoformat(), username, user_type))
        conn.commit()
        return cursor.rowcount > 0
```

```
def activate_user(username: str, user_type: str) -> bool
```

Activate a user account.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `username: str, user_type: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> bool`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.

Step by step (top-level body)

1. **with** `get_connection()`: enters a context manager.

Code (copy-paste safe)

```
def activate_user(username: str, user_type: str) -> bool:
    """Activate a user account."""
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute(''
            UPDATE user_credentials SET is_active = 1, updated_at = ?
            WHERE username = ? AND user_type = ?
            ''', (datetime.now().isoformat(), username, user_type))
```



```

conn.commit()
return cursor.rowcount > 0

```

```

def reset_user_password(username: str, user_type: str, default_password: str='Test@123') -> str

```

Reset user password to the default password.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `username: str`, `user_type: str`, `default_password: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> str`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.

Step by step (top-level body)

1. Assigns `(password_hash, salt) = hash_password(default_password)`.
2. Assigns `now = datetime.now().isoformat()`.
3. **with** `get_connection()`: enters a context manager.

Code (copy-paste safe)

```

def reset_user_password(username: str, user_type: str, default_password: str = 'Test@123') -> str:
    """Reset user password to the default password."""
    password_hash, salt = hash_password(default_password)
    now = datetime.now().isoformat()

    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute('''
            UPDATE user_credentials
            SET password_hash = ?, salt = ?, must_change_password = 1, updated_at = ?
            WHERE username = ? AND user_type = ?
        ''', (password_hash, salt, now, username, user_type))
        conn.commit()

    if cursor.rowcount > 0:
        return default_password
    return None

```

```

def find_user_by_identifier(identifier: str) -> dict

```

Find a user by email or username across all user types. Returns user info including user_type if found. Also checks if identifier matches super_admin.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `identifier: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.

- **return type annotation** — Annotated return type -> dict. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **if** `identifier.lower()` in `['super_admin', 'superadmin', 'admin']`: branches conditionally.
2. **if** `identifier.lower()` in `['bef_admin', 'befadmin', 'bef']`: branches conditionally.
3. **if** `identifier.lower()` in `['kwok_admin', 'kwokadmin', 'kwok', 'showcase...']`: branches conditionally.
4. **with** `get_connection()`: enters a context manager.

Code (copy-paste safe)

```
def find_user_by_identifier(identifier: str) -> dict:
    """
    Find a user by email or username across all user types.
    Returns user info including user_type if found.
    Also checks if identifier matches super_admin.
    """
    # Check if it's super_admin
    if identifier.lower() in ['super_admin', 'superadmin', 'admin']:
        return {'user_type': 'super_admin', 'username': 'super_admin'}

    # Check if it's bef_admin
    if identifier.lower() in ['bef_admin', 'befadmin', 'bef']:
        return {'user_type': 'bef_admin', 'username': 'bef_admin'}

    # Kwok (investor-style) dashboard – full read access, no writes (enforced in app)
    if identifier.lower() in [
        'kwok_admin', 'kwokadmin', 'kwok',
        'showcase_admin', 'showcaseadmin', 'showcase', 'investor_demo',
        'investor', 'demo_investor',
    ]:
        return {'user_type': 'kwok_admin', 'username': 'kwok_admin'}

    with get_connection() as conn:
        cursor = conn.cursor()
        # Search by username or email across all user types
        cursor.execute('''
            SELECT id, username, email, user_type, parent_admin, parent_trader,
                   is_active, must_change_password, password_hash, salt, last_login
            FROM user_credentials
            WHERE (username = ? OR email = ?) AND is_active = 1
            ''', (identifier, identifier))
        row = cursor.fetchone()
        return dict(row) if row else None

def verify_user_by_identifier(identifier: str, password: str) -> dict:
    """
    Verify user credentials by email or username (auto-detect user type). Returns user info with user_type if
    successful, None otherwise.
    """
```

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., identifier: str, password: str). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type -> dict. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.

Step by step (top-level body)

1. Assigns user = find_user_by_identifier(identifier).
2. if not user: branches conditionally.
3. if user.get('user_type') in ('super_admin', 'bef_admin', 'kwok_admin'): branches conditionally.
4. Assigns stored_hash = user.get('password_hash').
5. Assigns salt = user.get('salt').
6. if not stored_hash or not salt: branches conditionally.
7. Assigns password_hash = hashlib.pbkdf2_hmac('sha256', password.encode(), salt.enc....
8. if password_hash == stored_hash: branches conditionally.
9. return None.

Code (copy-paste safe)

```
def verify_user_by_identifier(identifier: str, password: str) -> dict:
    """
    Verify user credentials by email or username (auto-detect user type).
    Returns user info with user_type if successful, None otherwise.
    """
    user = find_user_by_identifier(identifier)
    if not user:
        return None

    # Super admin and BEF admin have special handling
    if user.get('user_type') in ('super_admin', 'bef_admin', 'kwok_admin'):
        return user # Password check happens separately

    # Verify password for regular users
    stored_hash = user.get('password_hash')
    salt = user.get('salt')

    if not stored_hash or not salt:
        return None

    password_hash = hashlib.pbkdf2_hmac('sha256', password.encode(), salt.encode(), 100000).hex()

    if password_hash == stored_hash:
        # Update last login
        with get_connection() as conn:
            cursor = conn.cursor()
            cursor.execute(
                'UPDATE user_credentials SET last_login = ? WHERE id = ?',
```

```

        (datetime.now().isoformat(), user['id'])
    )
    conn.commit()

    # Remove sensitive data before returning
    user.pop('password_hash', None)
    user.pop('salt', None)
    return user

return None

def delete_user_credential(username: str, user_type: str) -> bool

```

Permanently delete a user credential.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `username: str, user_type: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> bool`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.

Step by step (top-level body)

1. **with** `get_connection()`: enters a context manager.

Code (copy-paste safe)

```

def delete_user_credential(username: str, user_type: str) -> bool:
    """Permanently delete a user credential."""
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute(''
            DELETE FROM user_credentials
            WHERE username = ? AND user_type = ?
            ''', (username, user_type))
        conn.commit()
        return cursor.rowcount > 0

def update_user_email(username: str, user_type: str, new_email: str) -> bool

```

Update a user's email address.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `username: str, user_type: str, new_email: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> bool`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.

- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.

Step by step (top-level body)

1. **with** `get_connection()`: enters a context manager.

Code (copy-paste safe)

```
def update_user_email(username: str, user_type: str, new_email: str) -> bool:
    """Update a user's email address."""
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute('''
            UPDATE user_credentials
            SET email = ?, updated_at = ?
            WHERE username = ? AND user_type = ?
        ''', (new_email, datetime.now().isoformat(), username, user_type))
        conn.commit()
        return cursor.rowcount > 0

def rename_user_credential(old_name: str, new_name: str, user_type: str) -> bool
    Rename a user in user_credentials table.
```

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `old_name: str`, `new_name: str`, `user_type: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> bool`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.

Step by step (top-level body)

1. **with** `get_connection()`: enters a context manager.

Code (copy-paste safe)

```
def rename_user_credential(old_name: str, new_name: str, user_type: str) -> bool:
    """Rename a user in user_credentials table."""
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute('''
            UPDATE user_credentials SET username = ?, updated_at = ?
            WHERE username = ? AND user_type = ?
        ''', (new_name, datetime.now().isoformat(), old_name, user_type))
        conn.commit()
        return cursor.rowcount > 0

def rename_client_in_db(old_name: str, new_name: str) -> bool
    Rename client_id across all client data tables.
```

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `old_name: str`, `new_name: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> bool`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **with** `get_connection()`: enters a context manager.

Code (copy-paste safe)

```
def rename_client_in_db(old_name: str, new_name: str) -> bool:
    """Rename client_id across all client data tables."""
    with get_connection() as conn:
        cursor = conn.cursor()
        for table in ['clients_data', 'data_history', 'cell_notes', 'daily_watermarks', 'waterlog_periods']:
            try:
                cursor.execute(f'UPDATE {table} SET client_id = ? WHERE client_id = ?', (new_name, old_name))
            except Exception:
                pass
        conn.commit()
    return True
```

```
def user_exists(username: str, user_type: str) -> bool
```

Check if a user already exists.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `username: str`, `user_type: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> bool`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.

Step by step (top-level body)

1. **with** `get_connection()`: enters a context manager.

Code (copy-paste safe)

```
def user_exists(username: str, user_type: str) -> bool:
    """Check if a user already exists."""
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute(
            'SELECT 1 FROM user_credentials WHERE username = ? AND user_type = ?',
            (username, user_type)
        )
        return cursor.fetchone() is not None

def record_login_attempt(username: str, user_type: str, ip_address: str, success: bool)
    Record a login attempt.
```

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `username: str`, `user_type: str`, `ip_address: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **with-statement** — Uses a `with` block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. `with get_connection():` enters a context manager.

Code (copy-paste safe)

```
def record_login_attempt(username: str, user_type: str, ip_address: str, success: bool):
    """Record a login attempt."""
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute('''
            INSERT INTO login_attempts (username, user_type, ip_address, attempt_time, success)
            VALUES (?, ?, ?, ?, ?)
            ''', (username, user_type, ip_address, datetime.now().isoformat(), 1 if success else 0))
        conn.commit()

def get_failed_login_count(username: str, user_type: str, minutes: int=15) -> int
    Get count of failed login attempts in the last X minutes.
```

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `username: str`, `user_type: str`, `minutes: int`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> int`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.

- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Lazy import from `datetime`.
2. Assigns `cutoff = (datetime.now() - timedelta(minutes=minutes)).isoformat()`.
3. **with** `get_connection()`: enters a context manager.

Code (copy-paste safe)

```
def get_failed_login_count(username: str, user_type: str, minutes: int = 15) -> int:
    """Get count of failed login attempts in the last X minutes."""
    from datetime import timedelta
    cutoff = (datetime.now() - timedelta(minutes=minutes)).isoformat()

    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute('''
            SELECT COUNT(*) as count FROM login_attempts
            WHERE username = ? AND user_type = ? AND success = 0 AND attempt_time > ?
        ''', (username, user_type, cutoff))
        row = cursor.fetchone()
        return row['count'] if row else 0

def is_account_locked(username: str, user_type: str, max_attempts: int=5) -> bool
    Check if account is locked due to too many failed attempts.
```

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `username: str`, `user_type: str`, `max_attempts: int`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> bool`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.

Step by step (top-level body)

1. **return** `get_failed_login_count(username, user_type) >= max_attempts`.

Code (copy-paste safe)

```
def is_account_locked(username: str, user_type: str, max_attempts: int = 5) -> bool:
    """Check if account is locked due to too many failed attempts."""
    return get_failed_login_count(username, user_type) >= max_attempts

def hash_api_key(api_key: str) -> str
    Hash an API key using SHA-256.
```

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `api_key: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.

- **return type annotation** — Annotated return type `-> str`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.

Step by step (top-level body)

1. `return hashlib.sha256(api_key.encode('utf-8')).hexdigest()`.

Code (copy-paste safe)

```
def hash_api_key(api_key: str) -> str:
    """Hash an API key using SHA-256."""
    return hashlib.sha256(api_key.encode('utf-8')).hexdigest()

def generate_api_key(admin: str, trader: str, client: str='', scope: str='full') -> str:
    Generate a new API key and store its hash. scope: 'full' for full access, 'readonly' for read-only endpoints only.
```

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `admin: str, trader: str, client: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.

- **return type annotation** — Annotated return type `-> str`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `api_key = 'tk_' + secrets.token_urlsafe(32)`.
2. Assigns `key_hash = hash_api_key(api_key)`.
3. Assigns `key_prefix = api_key[:12]`.
4. Assigns `now = datetime.now().isoformat()`.
5. **with** `get_connection()`: enters a context manager.

Code (copy-paste safe)

```
def generate_api_key(admin: str, trader: str, client: str = '', scope: str = 'full') -> str:
    """Generate a new API key and store its hash.

    scope: 'full' for full access, 'readonly' for read-only endpoints only.
    """
```

```

api_key = 'tk_' + secrets.token_urlsafe(32)
key_hash = hash_api_key(api_key)
key_prefix = api_key[:12] # Store prefix for identification
now = datetime.now().isoformat()

with get_connection() as conn:
    cursor = conn.cursor()
    try:
        cursor.execute('''
            INSERT INTO api_keys (key_hash, key_prefix, admin, trader, client, scope, created_at)
            VALUES (?, ?, ?, ?, ?, ?, ?)
        ''', (key_hash, key_prefix, admin, trader, client, scope, now))
        conn.commit()
        return api_key # Return the actual key (only time it's visible)
    except Exception as e:
        print(f"Error generating API key: {e}")
        return None

```

```
def validate_api_key(api_key: str) -> dict
```

Validate an API key and return user info if valid. Includes 'scope' in the result.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `api_key: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> dict`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.

Step by step (top-level body)

1. Assigns `key_hash = hash_api_key(api_key)`.
2. `with get_connection()`: enters a context manager.

Code (copy-paste safe)

```

def validate_api_key(api_key: str) -> dict:
    """Validate an API key and return user info if valid. Includes 'scope' in the result."""
    key_hash = hash_api_key(api_key)

    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute('''
            SELECT admin, trader, client, scope, created_at FROM api_keys
            WHERE key_hash = ? AND is_active = 1
        ''', (key_hash,))
        row = cursor.fetchone()

    if row:
        # Update last_used timestamp
        cursor.execute(
            'UPDATE api_keys SET last_used = ? WHERE key_hash = ?',
            (datetime.now().isoformat(), key_hash)
        )

```

```

        conn.commit()

    return {
        'admin': row['admin'],
        'trader': row['trader'],
        'client': row['client'],
        'scope': row['scope'] or 'full',
        'created': row['created_at']
    }

return None

```

```
def list_api_keys() -> list
```

List all API keys (showing only prefix).

Why these Python concepts were used

- **return type annotation** — Annotated return type `-> list`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.

Step by step (top-level body)

1. **with** `get_connection()`: enters a context manager.

Code (copy-paste safe)

```

def list_api_keys() -> list:
    """List all API keys (showing only prefix)."""
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute('''
            SELECT key_prefix, admin, trader, client, scope, created_at, last_used, is_active
            FROM api_keys ORDER BY created_at DESC
        ''')
    return [dict(row) for row in cursor.fetchall()]

```

```
def revoke_api_key(key_prefix: str) -> bool
```

Revoke an API key by its prefix.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `key_prefix: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> bool`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.

Step by step (top-level body)

1. **with** `get_connection()`: enters a context manager.

Code (copy-paste safe)

```
def revoke_api_key(key_prefix: str) -> bool:
    """Revoke an API key by its prefix."""
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute(
            'UPDATE api_keys SET is_active = 0 WHERE key_prefix = ?',
            (key_prefix,)
        )
        conn.commit()
        return cursor.rowcount > 0
```

```
def delete_api_key(key_prefix: str) -> bool
```

Permanently delete an API key by its prefix.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `key_prefix: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> bool`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.

Step by step (top-level body)

1. **with** `get_connection()`: enters a context manager.

Code (copy-paste safe)

```
def delete_api_key(key_prefix: str) -> bool:
    """Permanently delete an API key by its prefix."""
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute('DELETE FROM api_keys WHERE key_prefix = ?', (key_prefix,))
        conn.commit()
        return cursor.rowcount > 0
```

```
def add_kyc_link(primary_client: str, linked_client: str, linked_by: str='super_admin') -> bool
```

Link a secondary client account to a primary client as a KYC.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `primary_client: str`, `linked_client: str`, `linked_by: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.

- **return type annotation** — Annotated return type -> bool. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.

Step by step (top-level body)

1. **if** `primary_client == linked_client`: branches conditionally.
2. **with** `get_connection()`: enters a context manager.

Code (copy-paste safe)

```
def add_kyc_link(primary_client: str, linked_client: str, linked_by: str = 'super_admin') -> bool:
    """Link a secondary client account to a primary client as a KYC."""
    if primary_client == linked_client:
        return False
    with get_connection() as conn:
        cursor = conn.cursor()
        try:
            cursor.execute('''
                INSERT INTO kyc_links (primary_client, linked_client, linked_by, created_at)
                VALUES (?, ?, ?, ?)
            ''', (primary_client, linked_client, linked_by, datetime.now().isoformat()))
            conn.commit()
            return True
        except psycopg2.IntegrityError:
            conn.rollback()
            return False
```

```
def remove_kyc_link(primary_client: str, linked_client: str) -> bool
```

Remove a KYC link between two clients.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `primary_client: str`, `linked_client: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type -> bool. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.

Step by step (top-level body)

1. **with** `get_connection()`: enters a context manager.

Code (copy-paste safe)

```
def remove_kyc_link(primary_client: str, linked_client: str) -> bool:
    """Remove a KYC link between two clients."""
    with get_connection() as conn:
```

```

cursor = conn.cursor()
cursor.execute('DELETE FROM kyc_links WHERE primary_client = ? AND linked_client = ?',
               (primary_client, linked_client))
conn.commit()
return cursor.rowcount > 0

```

```
def get_kyc_linked_clients(primary_client: str) -> list
```

Get all linked KYC accounts for a primary client.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `primary_client: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> list`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.

Step by step (top-level body)

1. **with** `get_connection()`: enters a context manager.

Code (copy-paste safe)

```

def get_kyc_linked_clients(primary_client: str) -> list:
    """Get all linked KYC accounts for a primary client."""
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute('SELECT linked_client, linked_by, created_at FROM kyc_links WHERE primary_client = '
                       '(primary_client,))
        return [{'linked_client': r['linked_client'], 'linked_by': r['linked_by'],
                  'created_at': r['created_at']} for r in cursor.fetchall()]

```

```
def get_kyc_primary_for(linked_client: str) -> str
```

If this client is linked to someone, return the primary client name.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `linked_client: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> str`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **with** `get_connection()`: enters a context manager.

Code (copy-paste safe)

```
def get_kyc_primary_for(linked_client: str) -> str:
    """If this client is linked to someone, return the primary client name."""
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute('SELECT primary_client FROM kyc_links WHERE linked_client = ?',
                        (linked_client,))
        row = cursor.fetchone()
        return row['primary_client'] if row else None
```

```
def is_kyc_primary(client_name: str) -> bool
```

Check if this client is a primary KYC account (has linked accounts under it).

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `client_name: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> bool`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.

Step by step (top-level body)

1. **return** `len(get_kyc_linked_clients(client_name)) > 0`.

Code (copy-paste safe)

```
def is_kyc_primary(client_name: str) -> bool:
    """Check if this client is a primary KYC account (has linked accounts under it)."""
    return len(get_kyc_linked_clients(client_name)) > 0
```

```
def get_all_kyc_accounts(client_name: str) -> list
```

Get all KYC accounts for a client (including self). If client_name is primary → returns [self] + linked accounts. If client_name is linked → returns [primary] + all siblings + self.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `client_name: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> list`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with `append`, and clearer about intent: this is a transform, not a side-effecting loop.

Step by step (top-level body)

1. Assigns `linked = get_kyc_linked_clients(client_name)`.

2. **if** linked: branches conditionally.
3. Assigns `primary = get_kyc_primary_for(client_name)`.
4. **if** primary: branches conditionally.
5. **return** `[client_name]`.

Code (copy-paste safe)

```
def get_all_kyc_accounts(client_name: str) -> list:
    """Get all KYC accounts for a client (including self).
    If client_name is primary → returns [self] + linked accounts.
    If client_name is linked → returns [primary] + all siblings + self.
    """
    # Check if this client is a primary
    linked = get_kyc_linked_clients(client_name)
    if linked:
        return [client_name] + [l['linked_client'] for l in linked]
    # Check if this client is linked to a primary
    primary = get_kyc_primary_for(client_name)
    if primary:
        siblings = get_kyc_linked_clients(primary)
        return [primary] + [l['linked_client'] for l in siblings]
    return [client_name]
```

```
def get_all_kyc_links() -> list
```

Get all KYC links in the system (for admin view).

Why these Python concepts were used

- **return type annotation** — Annotated return type `-> list`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with `append`, and clearer about intent: this is a transform, not a side-effecting loop.

Step by step (top-level body)

1. **with** `get_connection()`: enters a context manager.

Code (copy-paste safe)

```
def get_all_kyc_links() -> list:
    """Get all KYC links in the system (for admin view)."""
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute(
            'SELECT primary_client, linked_client, linked_by, created_at FROM kyc_links ORDER BY primary'
        )
        return [dict(r) for r in cursor.fetchall()]
```

```
def save_client_data(client_id: str, data: dict, overwrite: bool=False) -> bool
```

Save client data to database. If `overwrite=True`, replaces all existing data with the provided data (used for sheet imports). If `overwrite=False` (default), merges: new values take precedence but missing keys fall

back to existing.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `client_id: str`, `data: dict`, `overwrite: bool`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> bool`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **dict comprehension** — Builds a dict in one expression (`{k: v for k, v in items}`) — same intent as a list comprehension but for mappings.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns `now = datetime.now().isoformat()`.
2. **with** `get_connection()`: enters a context manager.

Code (copy-paste safe)

```
def save_client_data(client_id: str, data: dict, overwrite: bool = False) -> bool:
    """Save client data to database.

    If overwrite=True, replaces all existing data with the provided data (used for sheet imports).
    If overwrite=False (default), merges: new values take precedence but missing keys fall back to existing.
    """
    now = datetime.now().isoformat()

    with get_connection() as conn:
        cursor = conn.cursor()
        try:
            if overwrite:
                # Full replacement - use provided data as-is, defaulting missing fields to empty
                merged_deals = data.get('deals', [])
                merged_positions = data.get('positions', [])
                merged_account = data.get('account', {})
                merged_evaluations = data.get('evaluations', [])
                merged_statistics = data.get('statistics', {})
                merged_dropdown_options = data.get('dropdown_options', {})
```

```

merged_identity = data.get('identity', {})
merged_hedge_accounts = data.get('hedge_accounts', [])
merged_prop_accounts = data.get('prop_accounts', [])
merged_vps_accounts = data.get('vps_accounts', [])
merged_payment_info = data.get('payment_info', [])
merged_payment_address = data.get('payment_address', {})
merged_mt5_credentials = data.get('mt5_credentials', {})
merged_firm_billing = data.get('firm_billing', {})
else:
    # Merge: get existing data so missing keys fall back gracefully
    cursor.execute('SELECT * FROM clients_data WHERE client_id = ?', (client_id,))
    row = cursor.fetchone()

    existing_data = {}
    if row:
        existing_data = {
            'deals': json.loads(row['deals']),
            'positions': json.loads(row['positions']),
            'account': json.loads(row['account']),
            'evaluations': json.loads(row['evaluations']),
            'statistics': json.loads(row['statistics']),
            'dropdown_options': json.loads(row['dropdown_options']),
            'identity': json.loads(row['identity']),
            'hedge_accounts': json.loads(row.get('hedge_accounts') or '[]'),
            'prop_accounts': json.loads(row.get('prop_accounts') or '[]'),
            'vps_accounts': json.loads(row.get('vps_accounts') or '[]'),
            'payment_info': json.loads(row.get('payment_info') or '[]'),
            'mt5_credentials': json.loads(row.get('mt5_credentials') or '{}'),
        }

    # Merge existing data with new data (new data takes precedence)
    merged_deals = data.get('deals', existing_data.get('deals', []))
    merged_positions = data.get('positions', existing_data.get('positions', []))
    merged_account = data.get('account', existing_data.get('account', {}))
    merged_evaluations = data.get('evaluations', existing_data.get('evaluations', []))
    merged_statistics = data.get('statistics', existing_data.get('statistics', {}))
    merged_dropdown_options = data.get('dropdown_options', existing_data.get('dropdown_options', {}))
    merged_identity = data.get('identity', existing_data.get('identity', {}))
    merged_hedge_accounts = data.get('hedge_accounts', existing_data.get('hedge_accounts', []))
    merged_prop_accounts = data.get('prop_accounts', existing_data.get('prop_accounts', []))
    merged_vps_accounts = data.get('vps_accounts', existing_data.get('vps_accounts', []))
    merged_payment_info = data.get('payment_info', existing_data.get('payment_info', {}))
    merged_payment_address = data.get('payment_address', existing_data.get('payment_address', {}))
    merged_mt5_credentials = data.get('mt5_credentials', existing_data.get('mt5_credentials', {}))
    merged_firm_billing = data.get('firm_billing', existing_data.get('firm_billing', {}))

    # Strip _notes from evaluations – notes are stored separately in cell_notes table
    clean_evaluations = [
        {k: v for k, v in ev.items() if k != '_notes'}
        if isinstance(ev, dict) else ev
        for ev in merged_evaluations
    ]

    # Normalize Prop Firm names before storing to prevent duplicates
    FIRM_NORMALIZE = {
        "mffu": "My Funded Futures", "mffuflex": "My Funded Futures",
        "myfundedfutures": "My Funded Futures", "myfundedfx": "My Funded Futures",
        "mff": "My Funded Futures",
        "topstep": "Topstep", "topstep": "Topstep",
    }

```

```

        "fundingticks": "Funding Ticks", "fundingtick": "Funding Ticks",
        "fundednext": "FundedNext",
        "tradeday": "TradeDay", "tradeify": "Tradeify",
        "alphafutures": "Alpha Futures",
        "toponefutures": "Top One Futures", "topone": "Top One Futures",
    }
    for ev in clean_evaluations:
        if isinstance(ev, dict) and ev.get('Prop Firm'):
            raw = ev['Prop Firm'].strip().lower().replace(" ", "").replace("_", "")
            if raw in FIRM_NORMALIZE:
                ev['Prop Firm'] = FIRM_NORMALIZE[raw]

    cursor.execute('''
        INSERT INTO clients_data (
            client_id, deals, positions, account, evaluations,
            statistics, dropdown_options, identity, last_updated,
            hedge_accounts, prop_accounts, vps_accounts, payment_info, payment_address,
            mt5_credentials, firm_billing
        ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
        ON CONFLICT(client_id) DO UPDATE SET
            deals = excluded.deals,
            positions = excluded.positions,
            account = excluded.account,
            evaluations = excluded.evaluations,
            statistics = excluded.statistics,
            dropdown_options = excluded.dropdown_options,
            identity = excluded.identity,
            last_updated = excluded.last_updated,
            hedge_accounts = excluded.hedge_accounts,
            prop_accounts = excluded.prop_accounts,
            vps_accounts = excluded.vps_accounts,
            payment_info = excluded.payment_info,
            payment_address = excluded.payment_address,
            mt5_credentials = excluded.mt5_credentials,
            firm_billing = excluded.firm_billing
    ''', (
        client_id,
        json.dumps(merged_deals),
        json.dumps(merged_positions),
        json.dumps(merged_account),
        json.dumps(clean_evaluations),
        json.dumps(merged_statistics),
        json.dumps(merged_dropdown_options),
        json.dumps(merged_identity),
        now,
        json.dumps(merged_hedge_accounts),
        json.dumps(merged_prop_accounts),
        json.dumps(merged_vps_accounts),
        json.dumps(merged_payment_info),
        json.dumps(merged_payment_address),
        json.dumps(merged_mt5_credentials),
        json.dumps(merged_firm_billing),
    ))
    conn.commit()
    return True
except Exception as e:
    print(f"Error saving client data: {e}")
    return False

```

```
def get_client_data(client_id: str) -> dict
```

Get client data from database.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `client_id: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> dict`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **with** `get_connection()`: enters a context manager.

Code (copy-paste safe)

```
def get_client_data(client_id: str) -> dict:
    """Get client data from database."""
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute('SELECT * FROM clients_data WHERE client_id = ?', (client_id,))
        row = cursor.fetchone()

    if row:
        try:
            identity = json.loads(row['identity'] or '{}') or {}
        except Exception:
            identity = {}
        return {
            'deals': json.loads(row['deals']),
            'positions': json.loads(row['positions']),
            'account': json.loads(row['account']),
            'evaluations': json.loads(row['evaluations']),
            'statistics': json.loads(row['statistics']),
            'dropdown_options': json.loads(row['dropdown_options']),
            'identity': identity,
            'sheet_url': identity.get('sheet_url') if isinstance(identity, dict) else None,
            'last_updated': row['last_updated'],
            'hedge_accounts': json.loads(row.get('hedge_accounts') or '[]'),
            'prop_accounts': json.loads(row.get('prop_accounts') or '[]'),
            'vps_accounts': json.loads(row.get('vps_accounts') or '[]'),
            'payment_info': json.loads(row.get('payment_info') or '[]'),
            'payment_address': json.loads(row.get('payment_address') or '{}'),
            'mt5_credentials': json.loads(row.get('mt5_credentials') or '{}'),
```

```

        'firm_billing': json.loads(row.get('firm_billing') or '{}'),
    }

    return None

```

```
def get_all_clients() -> dict
```

Get all client data in a single query (avoids N+1 pattern).

Why these Python concepts were used

- **return type annotation** — Annotated return type -> dict. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **with get_connection():** enters a context manager.

Code (copy-paste safe)

```

def get_all_clients() -> dict:
    """Get all client data in a single query (avoids N+1 pattern)."""
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute('SELECT * FROM clients_data')
        clients = {}
        for row in cursor.fetchall():
            client_id = row['client_id']
            try:
                identity = json.loads(row['identity'] or '{}') or {}
            except Exception:
                identity = {}
            clients[client_id] = {
                'deals': json.loads(row['deals']),
                'positions': json.loads(row['positions']),
                'account': json.loads(row['account']),
                'evaluations': json.loads(row['evaluations']),
                'statistics': json.loads(row['statistics']),
                'dropdown_options': json.loads(row['dropdown_options']),
                'identity': identity,
                'sheet_url': identity.get('sheet_url') if isinstance(identity, dict) else None,
                'last_updated': row['last_updated'],
                'hedge_accounts': json.loads(row.get('hedge_accounts') or '[]'),
                'prop_accounts': json.loads(row.get('prop_accounts') or '[]'),
                'vps_accounts': json.loads(row.get('vps_accounts') or '[]'),
                'payment_info': json.loads(row.get('payment_info') or '[]'),
                'payment_address': json.loads(row.get('payment_address') or '{}'),
            }

```

```

        'mt5_credentials': json.loads(row.get('mt5_credentials') or '{}'),
        'firm_billing': json.loads(row.get('firm_billing') or '{}'),
    }
    return clients

```

```
def get_all_client_identities() -> dict
```

Fetch only client_id + identity for all clients in one query. Used to avoid N+1 queries when only identity fields are needed.

Why these Python concepts were used

- **return type annotation** — Annotated return type -> dict. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.

Step by step (top-level body)

1. **with** `get_connection()`: enters a context manager.

Code (copy-paste safe)

```

def get_all_client_identities() -> dict:
    """Fetch only client_id + identity for all clients in one query.
    Used to avoid N+1 queries when only identity fields are needed."""
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute('SELECT client_id, identity FROM clients_data')
        result = {}
        for row in cursor.fetchall():
            try:
                identity = json.loads(row['identity'] or '{}') or {}
            except Exception:
                identity = {}
            result[row['client_id']] = identity
        return result

```

```
def delete_client_data(client_id: str) -> bool
```

Permanently delete all data for a client from the database.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `client_id: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type -> bool. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.

Step by step (top-level body)

1. **with** `get_connection()`: enters a context manager.

Code (copy-paste safe)

```
def delete_client_data(client_id: str) -> bool:
    """Permanently delete all data for a client from the database."""
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute('DELETE FROM clients_data WHERE client_id = ?', (client_id,))
        cursor.execute('DELETE FROM data_history WHERE client_id = ?', (client_id,))
        cursor.execute('DELETE FROM cell_notes WHERE client_id = ?', (client_id,))
        cursor.execute('DELETE FROM daily_watermarks WHERE client_id = ?', (client_id,))
        cursor.execute('DELETE FROM waterlog_periods WHERE client_id = ?', (client_id,))
        conn.commit()
        return cursor.rowcount >= 0

def get_clients_count() -> int
    Get count of clients in database.
```

Why these Python concepts were used

- **return type annotation** — Annotated return type `-> int`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **with** `get_connection()`: enters a context manager.

Code (copy-paste safe)

```
def get_clients_count() -> int:
    """Get count of clients in database."""
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute('SELECT COUNT(*) as count FROM clients_data')
        row = cursor.fetchone()
        return row['count'] if row else 0

def update_client_field(client_id: str, field: str, value) -> bool
    Update a specific field for a client.
```

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `client_id: str`, `field: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> bool`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.

- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `valid_fields = ['deals', 'positions', 'account', 'evaluations', 'statist...`
2. **if** `field not in valid_fields`: branches conditionally.
3. **with** `get_connection()`: enters a context manager.

Code (copy-paste safe)

```
def update_client_field(client_id: str, field: str, value) -> bool:
    """Update a specific field for a client."""
    valid_fields = ['deals', 'positions', 'account', 'evaluations', 'statistics', 'identity',
                    'dropdown_options',
                    'hedge_accounts', 'prop_accounts', 'vps_accounts', 'payment_info', 'payment_address']
    if field not in valid_fields:
        return False

    with get_connection() as conn:
        cursor = conn.cursor()

        # First ensure client exists
        cursor.execute('SELECT client_id FROM clients_data WHERE client_id = ?', (client_id,))
        if cursor.fetchone() is None:
            # Create new client record
            save_client_data(client_id, {field: value})
            return True

        # Update specific field
        cursor.execute(f'''
            UPDATE clients_data
            SET {field} = ?, last_updated = ?
            WHERE client_id = ?
        ''', (json.dumps(value), datetime.now().isoformat(), client_id))
        conn.commit()
        return True
```

```
def _repair_quality_table()
```

No-op — schema is managed by Alembic. PostgreSQL handles table integrity.

Step by step (top-level body)

1. Calls `print(...)` for its side effect.

Code (copy-paste safe)

```
def _repair_quality_table():
    """No-op — schema is managed by Alembic. PostgreSQL handles table integrity."""
    print("[DB] quality_scan_results table integrity is managed by PostgreSQL")
```

```
def save_quality_scan_results(scan_date: str, results: list)
```

Save quality scan results for all clients.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `scan_date: str`, `results: list`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def save_quality_scan_results(scan_date: str, results: list):
    """Save quality scan results for all clients."""
    try:
        with get_connection() as conn:
            cursor = conn.cursor()
            cursor.execute('DELETE FROM quality_scan_results WHERE scan_date = ?', (scan_date,))
            for r in results:
                cursor.execute('''
                    INSERT INTO quality_scan_results (scan_date, client_id, trader, admin, total_issues,
                        issues, health_score)
                    VALUES (?, ?, ?, ?, ?, ?, ?)
                ''', (scan_date, r['client_id'], r.get('trader'), r.get('admin'),
                    r['total_issues'], json.dumps(r['issues']), r['health_score']))
            conn.commit()
    except Exception as e:
        print(f"Error saving quality scan results: {e}")
        raise
```

```
def get_quality_scan_results(scan_date: str=None) -> list
```

Get quality scan results. If no date, returns latest scan.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `scan_date: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> list`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't

have to handle it.

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def get_quality_scan_results(scan_date: str = None) -> list:
    """Get quality scan results. If no date, returns latest scan."""
    try:
        return _get_quality_scan_results_inner(scan_date)
    except Exception as e:
        print(f"Error getting quality scan results: {e}")
        return []
```

```
def _get_quality_scan_results_inner(scan_date: str=None) -> list
```

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `scan_date: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> list`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **with** `get_connection()`: enters a context manager.

Code (copy-paste safe)

```
def _get_quality_scan_results_inner(scan_date: str = None) -> list:
    with get_connection() as conn:
        cursor = conn.cursor()
        if not scan_date:
            cursor.execute('SELECT MAX(scan_date) as d FROM quality_scan_results')
            row = cursor.fetchone()
            scan_date = row['d'] if row and row['d'] else None
        if not scan_date:
            return []
        cursor.execute('''
            SELECT * FROM quality_scan_results WHERE scan_date = ? ORDER BY health_score ASC
        ''', (scan_date,))
        results = []
        for row in cursor.fetchall():
            results.append({
                'client_id': row['client_id'],
                'trader': row['trader'],
                'admin': row['admin'],
```

```

        'total_issues': row['total_issues'],
        'issues': json.loads(row['issues']),
        'health_score': row['health_score'],
        'scan_date': row['scan_date'],
    })
    return results

```

```

def save_daily_checklist(date: str, user_identifier: str, user_type: str, checklist_type: str, items: list,
                        ip_address: str=None, client_id: str='')

```

Save a daily checklist submission (per client).

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `date: str`, `user_identifier: str`, `user_type: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.

- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.

Step by step (top-level body)

1. **with** `get_connection()`: enters a context manager.

Code (copy-paste safe)

```

def save_daily_checklist(date: str, user_identifier: str, user_type: str,
                        checklist_type: str, items: list, ip_address: str = None,
                        client_id: str = ''):
    """Save a daily checklist submission (per client)."""
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute('''
            INSERT INTO daily_checklists (date, user_identifier, user_type, checklist_type, client_id, ip_address,
            submitted_at, ip_address)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?)
            ON CONFLICT(date, user_identifier, checklist_type, client_id) DO UPDATE SET
                user_type = excluded.user_type,
                items = excluded.items,
                submitted_at = excluded.submitted_at,
                ip_address = excluded.ip_address
        ''', (date, user_identifier, user_type, checklist_type, client_id, json.dumps(items),
            datetime.now().isoformat(), ip_address))
        conn.commit()

```

```

def _ensure_checklist_client_column()

```

No-op — schema is managed by Alembic.

Step by step (top-level body)

1. **pass** (placeholder).

Code (copy-paste safe)

```

def _ensure_checklist_client_column():
    """No-op – schema is managed by Alembic."""

```

```
pass
```

```
def _ensure_settings_table()
```

No-op — schema is managed by Alembic.

Step by step (top-level body)

1. **pass** (placeholder).

Code (copy-paste safe)

```
def _ensure_settings_table():
    """No-op — schema is managed by Alembic."""
    pass
```

```
def get_setting(key: str) -> str
```

Get a system setting by key. Returns empty string if not found.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `key: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> str`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def get_setting(key: str) -> str:
    """Get a system setting by key. Returns empty string if not found."""
    try:
        with get_connection() as conn:
            cursor = conn.cursor()
            cursor.execute('SELECT value FROM system_settings WHERE key = ?', (key,))
            row = cursor.fetchone()
            return row['value'] if row else ''
    except Exception:
        _ensure_settings_table()
        return ''
```

```
def set_setting(key: str, value: str, updated_by: str='')
```

Set a system setting.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `key: str, value: str, updated_by: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.

Step by step (top-level body)

1. Calls `_ensure_settings_table(...)` for its side effect.
2. **with** `get_connection()`: enters a context manager.

Code (copy-paste safe)

```
def set_setting(key: str, value: str, updated_by: str = ''):
    """Set a system setting."""
    _ensure_settings_table()
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute('''
            INSERT INTO system_settings (key, value, updated_at, updated_by)
            VALUES (?, ?, ?, ?)
            ON CONFLICT(
                key) DO UPDATE SET value=excluded.value, updated_at=excluded.updated_at, updated_by=exc
            ''', (key, value, datetime.now().isoformat(), updated_by))
        conn.commit()
```

```
def get_daily_checklists(date: str, user_identifier: str=None) -> list
```

Get checklists for a date, optionally filtered by user.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `date: str, user_identifier: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> list`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **list comprehension** — Builds a list in a single expression `[f(x) for x in xs]`. Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **with** `get_connection()`: enters a context manager.

Code (copy-paste safe)

```
def get_daily_checklists(date: str, user_identifier: str = None) -> list:
    """Get checklists for a date, optionally filtered by user."""
    with get_connection() as conn:
        cursor = conn.cursor()
        if user_identifier:
            cursor.execute('SELECT * FROM daily_checklists WHERE date = ? AND user_identifier = ?',
                           (date, user_identifier))
        else:
            cursor.execute('SELECT * FROM daily_checklists WHERE date = ?', (date,))
    return [{
        'user_identifier': row['user_identifier'],
        'user_type': row['user_type'],
        'checklist_type': row['checklist_type'],
        'client_id': row['client_id'] if 'client_id' in row.keys() else '',
        'items': json.loads(row['items']),
        'submitted_at': row['submitted_at'],
    } for row in cursor.fetchall()]
```

```
def get_checklist_clients_for_date(date: str) -> set
```

Return set of client_ids that have a daily_summary checklist submitted for the given date.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `date: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> set`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **set comprehension** — Builds a set in one expression — usually to deduplicate the result of a transform.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def get_checklist_clients_for_date(date: str) -> set:
    """Return set of client_ids that have a daily_summary checklist submitted for the given date."""
    try:
        with get_connection() as conn:
            cursor = conn.cursor()
            cursor.execute(
                'SELECT DISTINCT client_id FROM daily_checklists WHERE date = ? AND checklist_type = 'da
                (date,)
            )
            return {row['client_id'] for row in cursor.fetchall()}
    except Exception:
        return set()
```

```
def get_summary_status_for_date(date: str) -> list
```

Return all daily_summary checklist submissions for the given date. Merges data from daily_checklists table AND audit_log. The 24-hour window runs from 23:05 UTC day-1 to 23:05 UTC day (= 2:05 AM Kenyan → 2:05 AM Kenyan). The `date` parameter is the UTC date (server runs UTC).

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., date: str). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type -> list. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.

Step by step (top-level body)

1. Assigns results = {}.
2. Lazy import from datetime.
3. **try** block with 1 **except** clause.
4. Assigns utc_start = (utc_date - _td(days=1)).strftime('%Y-%m-%d') + 'T23:05'.
5. Assigns utc_end = utc_date.strftime('%Y-%m-%d') + 'T23:05'.
6. **try** block with 1 **except** clause.
7. **return** list(results.values()).

Code (copy-paste safe)

```
def get_summary_status_for_date(date: str) -> list:
    """Return all daily_summary checklist submissions for the given date.
    Merges data from daily_checklists table AND audit_log.

    The 24-hour window runs from 23:05 UTC day-1 to 23:05 UTC day
    (= 2:05 AM Kenyan → 2:05 AM Kenyan). The `date` parameter is the
    UTC date (server runs UTC).
    """
    results = {} # client_id -> {client_id, submitted_by, submitted_at}

    from datetime import timedelta as _td
    try:
        utc_date = datetime.strptime(date, '%Y-%m-%d')
    except ValueError:
        return []

    # Window: 23:05 UTC previous day → 23:05 UTC this day
    # = 2:05 AM Kenyan day → 2:05 AM Kenyan day+1
    utc_start = (utc_date - _td(days=1)).strftime('%Y-%m-%d') + 'T23:05'
    utc_end = utc_date.strftime('%Y-%m-%d') + 'T23:05'
```

```

try:
    with get_connection() as conn:
        cursor = conn.cursor()
        # Source 1: daily_checklists – filter by submitted_at timestamp
        # so the window is exactly 23:05→23:05 UTC
        cursor.execute(
            "SELECT client_id, user_identifier, submitted_at FROM daily_checklists "
            "WHERE checklist_type = 'daily_summary' AND client_id != ' ' "
            "AND submitted_at >= ? AND submitted_at < ? "
            "ORDER BY submitted_at DESC",
            (utc_start, utc_end)
        )
        for row in cursor.fetchall():
            cid = row['client_id']
            if cid not in results:
                results[cid] = {'client_id': cid, 'submitted_by': row['user_identifier'],
                               'submitted_at': row['submitted_at']}

        # Source 2: audit_log – same 23:05→23:05 UTC window
        cursor.execute(
            "SELECT user_identifier, details, timestamp FROM audit_log "
            "WHERE action IN ('CHECKLIST_SUBMIT', 'SLACK_DAILY_SUMMARY') "
            "AND timestamp >= ? AND timestamp < ? AND success = 1 "
            "ORDER BY timestamp DESC",
            (utc_start, utc_end)
        )
        for row in cursor.fetchall():
            details = row['details'] or ''
            client_id = ''
            if ' for ' in details:
                part = details.split(' for ', 1)[1]
                import re
                part = re.sub(r':\s*\d+\s+sections?\s*$', '', part).strip()
                client_id = part
            if client_id and client_id not in results:
                results[client_id] = {'client_id': client_id, 'submitted_by': row['user_identifier'],
                                     'submitted_at': row['timestamp']}

except Exception:
    pass
return list(results.values())

```

```
def get_weekly_scan_results(end_date: str=None, days: int=7) -> list
```

Get quality scan results for a date range (default: last 7 days).

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `end_date: str`, `days: int`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> list`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def get_weekly_scan_results(end_date: str = None, days: int = 7) -> list:
    """Get quality scan results for a date range (default: last 7 days)."""
    try:
        return _get_weekly_scan_results_inner(end_date, days)
    except Exception as e:
        print(f"Error getting weekly scan results: {e}")
        return []
```

```
def _get_weekly_scan_results_inner(end_date: str=None, days: int=7) -> list
```

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `end_date: str`, `days: int`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> list`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.

Step by step (top-level body)

1. **with** `get_connection()`: enters a context manager.

Code (copy-paste safe)

```
def _get_weekly_scan_results_inner(end_date: str = None, days: int = 7) -> list:
    with get_connection() as conn:
        cursor = conn.cursor()
        if not end_date:
            end_date = datetime.now().strftime('%Y-%m-%d')
        start_date = (datetime.strptime(end_date, '%Y-%m-%d') - timedelta(days=days - 1)).strftime('%Y-%m-%d')
        cursor.execute('''
            SELECT * FROM quality_scan_results
            WHERE scan_date BETWEEN ? AND ?
            ORDER BY scan_date ASC, health_score ASC
        ''', (start_date, end_date))
        results = []
        for row in cursor.fetchall():
            results.append({
                'client_id': row['client_id'],
                'trader': row['trader'],
                'admin': row['admin'],
                'total_issues': row['total_issues'],
                'issues': json.loads(row['issues']),
                'health_score': row['health_score'],
                'scan_date': row['scan_date'],
            })
```

```
return results
```

```
def _ensure_qa_resolutions_table()
```

Ensure QA resolution table exists (small operational table).

Why these Python concepts were used

- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.

Step by step (top-level body)

1. **with** `get_connection()`: enters a context manager.

Code (copy-paste safe)

```
def _ensure_qa_resolutions_table():
    """Ensure QA resolution table exists (small operational table)."""
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute('''
            CREATE TABLE IF NOT EXISTS qa_resolutions (
                check_name TEXT NOT NULL,
                client_id TEXT NOT NULL,
                row_index INTEGER NOT NULL,
                resolved BOOLEAN NOT NULL DEFAULT TRUE,
                resolved_by TEXT NOT NULL DEFAULT '',
                resolved_at TEXT NOT NULL,
                notes TEXT NOT NULL DEFAULT '',
                PRIMARY KEY (check_name, client_id, row_index)
            )
        ''')
        conn.commit()
```

```
def get_qa_resolved_set(check_name: str) -> set
```

Return a set of (client_id, row_index) resolved for a given QA check.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `check_name: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> set`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.

Step by step (top-level body)

1. **if** `not check_name`: branches conditionally.

2. try block with 1 except clause.

Code (copy-paste safe)

```
def get_qa_resolved_set(check_name: str) -> set:
    """Return a set of (client_id, row_index) resolved for a given QA check."""
    if not check_name:
        return set()
    try:
        _ensure_qa_resolutions_table()
        with get_connection() as conn:
            cursor = conn.cursor()
            cursor.execute(
                'SELECT client_id, row_index FROM qa_resolutions WHERE check_name = ? AND resolved = TRUE'
                (check_name,)
            )
            rows = cursor.fetchall() or []
            out = set()
            for r in rows:
                try:
                    out.add((r['client_id'], int(r['row_index'])))
                except Exception:
                    continue
            return out
    except Exception:
        return set()
```

```
def is_qa_resolved(check_name: str, client_id: str, row_index: int) -> bool
```

Check whether a specific QA issue has been resolved.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `check_name: str`, `client_id: str`, `row_index: int`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> bool`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.

Step by step (top-level body)

1. **if not check_name or not client_id:** branches conditionally.
2. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def is_qa_resolved(check_name: str, client_id: str, row_index: int) -> bool:
    """Check whether a specific QA issue has been resolved."""
    if not check_name or not client_id:
        return False
```

```

try:
    _ensure_qa_resolutions_table()
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute(
            'SELECT resolved FROM qa_resolutions WHERE check_name = ? AND client_id = ? AND row_index = ?'
            (check_name, client_id, int(row_index))
        )
        row = cursor.fetchone()
        return bool(row and row.get('resolved'))
except Exception:
    return False

```

```
def mark_qa_resolved(check_name: str, client_id: str, row_index: int, resolved_by: str, notes: str=''):
```

Mark a QA issue resolved (idempotent).

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `check_name: str`, `client_id: str`, `row_index: int`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.

Step by step (top-level body)

1. **if** not `check_name` or not `client_id`: branches conditionally.
2. Calls `_ensure_qa_resolutions_table(...)` for its side effect.
3. **with** `get_connection()`: enters a context manager.

Code (copy-paste safe)

```

def mark_qa_resolved(check_name: str, client_id: str, row_index: int, resolved_by: str, notes: str = ''):
    """Mark a QA issue resolved (idempotent)."""
    if not check_name or not client_id:
        return
    _ensure_qa_resolutions_table()
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute('''
            INSERT INTO qa_resolutions (check_name, client_id, row_index, resolved, resolved_by, resolved_at,
            notes)
            VALUES (?, ?, ?, TRUE, ?, ?, ?)
            ON CONFLICT(check_name, client_id, row_index) DO UPDATE SET
                resolved = TRUE,
                resolved_by = excluded.resolved_by,
                resolved_at = excluded.resolved_at,
                notes = excluded.notes
        ''', (check_name, client_id, int(row_index), resolved_by or '', datetime.now().isoformat(), notes))
        conn.commit()

```

```
def get_client_activity(client_id: str) -> dict
```

Get last push time and last edit info for a client from data_history.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `client_id: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> dict`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **with** `get_connection()`: enters a context manager.

Code (copy-paste safe)

```
def get_client_activity(client_id: str) -> dict:
    """Get last push time and last edit info for a client from data_history."""
    with get_connection() as conn:
        cursor = conn.cursor()
        # Last push (from companion app / MT5)
        cursor.execute('''
            SELECT created_at, changed_by FROM data_history
            WHERE client_id = ? AND change_source = 'push'
            ORDER BY version DESC LIMIT 1
        ''', (client_id,))
        push_row = cursor.fetchone()

        # Last manual edit (from dashboard)
        cursor.execute('''
            SELECT created_at, changed_by, changed_by_type FROM data_history
            WHERE client_id = ? AND change_source IN ('dashboard_edit', 'dashboard_delete')
            ORDER BY version DESC LIMIT 1
        ''', (client_id,))
        edit_row = cursor.fetchone()

    return {
        'last_push_at': push_row['created_at'] if push_row else None,
        'last_push_by': push_row['changed_by'] if push_row else None,
        'last_edit_at': edit_row['created_at'] if edit_row else None,
        'last_edit_by': edit_row['changed_by'] if edit_row else None,
        'last_edit_by_type': edit_row['changed_by_type'] if edit_row else None,
    }
```

```
def get_next_version(client_id: str) -> int
```

Get the next version number for a client's data history.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `client_id: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.

- **return type annotation** — Annotated return type `-> int`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def get_next_version(client_id: str) -> int:
    """Get the next version number for a client's data history."""
    try:
        with get_connection() as conn:
            cursor = conn.cursor()
            cursor.execute(''
                SELECT MAX(version) as max_version FROM data_history WHERE client_id = ?
            '', (client_id,))
            row = cursor.fetchone()
            return (row['max_version'] or 0) + 1
    except Exception as e:
        print(f"Error getting next version: {e}")
        # Fallback to local timestamp-based ID or just start at 1 if DB read fails?
        # If DB read fails, snapshot will likely fail too, but at least we don't crash the app.
        return 1
```

```
def verify_data_saved(client_id: str, expected_evals_count: int=None, expected_stat_key: str=None) -> bool:
```

Verify that data was actually saved and committed to the database. This is a critical check to detect silent commit failures or connection issues. Pass `expected_evals_count` or `expected_stat_key` to verify specific fields were persisted. Returns: `True` if data is present and matches expected values, `False` otherwise.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `client_id: str`, `expected_evals_count: int`, `expected_stat_key: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> bool`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def verify_data_saved(client_id: str, expected_evals_count: int = None,
    expected_stat_key: str = None) -> bool:
    """
    Verify that data was actually saved and committed to the database.

    This is a critical check to detect silent commit failures or connection issues.
    Pass expected_evals_count or expected_stat_key to verify specific fields were persisted.

    Returns: True if data is present and matches expected values, False otherwise.
    """
    try:
        saved_data = get_client_data(client_id)
        if not saved_data:
            logger.warning(f"[DB VERIFY FAILED] No data found for {client_id} after save")
            return False

        if expected_evals_count is not None:
            actual_count = len(saved_data.get('evaluations', []))
            if actual_count != expected_evals_count:
                logger.warning(
                    f"[DB VERIFY FAILED] {client_id} evals count mismatch: "
                    f"expected {expected_evals_count}, got {actual_count}"
                )
                return False

        if expected_stat_key is not None:
            stats = saved_data.get('statistics', {})
            if expected_stat_key not in stats:
                logger.warning(
                    f"[DB VERIFY FAILED] {client_id} stats missing key: {expected_stat_key}"
                )
                return False

        logger.debug(f"[DB VERIFY OK] {client_id} data verified successfully")
        return True
    except Exception as e:
        logger.error(f"[DB VERIFY ERROR] Failed to verify {client_id}: {e}")
        return False
```

```
def save_data_snapshot(client_id: str, data: dict, action: str, changed_by: str=None,
    changed_by_type: str=None, ip_address: str=None, change_source: str=None, change_description: str=None)
```

Save a snapshot of client data to history for versioning/rollback. Version number is assigned atomically inside the transaction using an advisory lock to prevent duplicate-key races under concurrent writes.

Returns: The version number of the saved snapshot, or -1 on failure.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `client_id: str`, `data: dict`, `action: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE

auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.

- **return type annotation** — Annotated return type -> int. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def save_data_snapshot(client_id: str, data: dict, action: str,
                       changed_by: str = None, changed_by_type: str = None,
                       ip_address: str = None, change_source: str = None,
                       change_description: str = None) -> int:
    """
    Save a snapshot of client data to history for versioning/rollback.
    Version number is assigned atomically inside the transaction using
    an advisory lock to prevent duplicate-key races under concurrent writes.

    Returns:
        The version number of the saved snapshot, or -1 on failure.
    """
    try:
        now = datetime.now().isoformat()

        deals_json = json.dumps(data.get('deals', []))
        positions_json = json.dumps(data.get('positions', []))
        account_json = json.dumps(data.get('account', {}))
        evaluations_json = json.dumps(data.get('evaluations', []))
        statistics_json = json.dumps(data.get('statistics', {}))
        dropdown_options_json = json.dumps(data.get('dropdown_options', {}))
        identity_json = json.dumps(data.get('identity', {}))

        with get_connection() as conn:
            cursor = conn.cursor()

            # Advisory lock keyed on client_id hash — prevents concurrent
            # workers from reading the same MAX(version). Released on commit.
            cursor.execute(
                "SELECT pg_advisory_xact_lock(hashtext(%s))",
                (client_id,)
            )
            cursor.execute(
                'SELECT COALESCE(MAX(version), 0) AS max_ver FROM data_history '
                'WHERE client_id = %s',
                (client_id,)
```



```

    )
    row = cursor.fetchone()
    version = (row['max_ver'] if row else 0) + 1

    cursor.execute('''
        INSERT INTO data_history (
            client_id, version, action, changed_by, changed_by_type,
            ip_address, change_source, change_description,
            deals, positions, account, evaluations, statistics,
            dropdown_options, identity, created_at
        ) VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
    ''', (
        client_id, version, action, changed_by, changed_by_type,
        ip_address, change_source, change_description,
        deals_json, positions_json, account_json, evaluations_json, statistics_json,
        dropdown_options_json, identity_json, now
    ))
    conn.commit()
    return version
except Exception as e:
    print(f"Error saving data snapshot: {e}")
    return -1

```

```

def save_client_data_with_history(client_id: str, data: dict, action: str='UPDATE', changed_by: str=None,
    changed_by_type: str=None, ip_address: str=None, change_source: str=None, change_description: str=None)

```

Save client data AND create a history snapshot for versioning. Includes verification that data was actually committed to the database. Returns: Tuple of (success: bool, version: int)

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `client_id: str`, `data: dict`, `action: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type $\rightarrow$ tuple. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def save_client_data_with_history(client_id: str, data: dict,
    action: str = 'UPDATE',
    changed_by: str = None,
    changed_by_type: str = None,
    ip_address: str = None,
    change_source: str = None,
    change_description: str = None,
    overwrite: bool = False) -> tuple:

```

```

"""
Save client data AND create a history snapshot for versioning.

Includes verification that data was actually committed to the database.

Returns:
    Tuple of (success: bool, version: int)
"""
try:
    # First, save a snapshot to history
    version = save_data_snapshot(
        client_id, data, action, changed_by, changed_by_type,
        ip_address, change_source, change_description
    )

    if version <= 0:
        logger.error(f"[DB SAVE FAILED] Failed to create history snapshot for {client_id}")
        return (False, -1)

    # Then save the current data
    success = save_client_data(client_id, data, overwrite=overwrite)

    if not success:
        logger.error(f"[DB SAVE FAILED] Failed to save current data for {client_id} (v{version})")
        return (False, version)

    # CRITICAL: Verify data was actually committed (catch silent commit failures)
    evals_count = len(data.get('evaluations', []))
    if not verify_data_saved(client_id, expected_evals_count=evals_count):
        logger.error(
            f"[DB COMMIT VERIFICATION FAILED] {client_id} data not verified after save (v{version})."
            f"This indicates a potential database connection or commit issue."
        )
        # Don't fail here – data may be committed but verification query hit stale connection
        # Just log it so admin can investigate

    logger.info(f"[DB SAVE OK] {client_id} saved successfully (v{version}, {evals_count} evals)")
    return (success, version)

except Exception as e:
    logger.error(f"[DB SAVE ERROR] Failed to save {client_id}: {e}")
    import traceback
    logger.error(traceback.format_exc())
    return (False, -1)

```

```
def get_data_history(client_id: str, limit: int=50) -> list
```

Get the history of all data changes for a client. Returns list of history entries (newest first) with metadata.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `client_id: str`, `limit: int`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> list`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.

- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **list comprehension** — Builds a list in a single expression ([f(x) for x in xs]). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.

Step by step (top-level body)

1. **with** get_connection(): enters a context manager.

Code (copy-paste safe)

```
def get_data_history(client_id: str, limit: int = 50) -> list:
    """
    Get the history of all data changes for a client.

    Returns list of history entries (newest first) with metadata.
    """
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute('''
            SELECT id, client_id, version, action, changed_by, changed_by_type,
                   ip_address, change_source, change_description, created_at
            FROM data_history
            WHERE client_id = ?
            ORDER BY version DESC
            LIMIT ?
        ''', (client_id, limit))

    return [dict(row) for row in cursor.fetchall()]

def get_data_version(client_id: str, version: int) -> dict:
    Get a specific version of client data from history. Returns the full data dict for that version, or None if not found.
```

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., client_id: str, version: int). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type -> dict. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.

Step by step (top-level body)

1. **with** get_connection(): enters a context manager.

Code (copy-paste safe)

```
def get_data_version(client_id: str, version: int) -> dict:
    """
    Get a specific version of client data from history.

    Returns the full data dict for that version, or None if not found.
```

```

"""
with get_connection() as conn:
    cursor = conn.cursor()
    cursor.execute('''
        SELECT * FROM data_history WHERE client_id = ? AND version = ?
    ''', (client_id, version))
    row = cursor.fetchone()

    if row:
        return {
            'version': row['version'],
            'action': row['action'],
            'changed_by': row['changed_by'],
            'changed_by_type': row['changed_by_type'],
            'change_source': row['change_source'],
            'change_description': row['change_description'],
            'created_at': row['created_at'],
            'data': {
                'deals': json.loads(row['deals']),
                'positions': json.loads(row['positions']),
                'account': json.loads(row['account']),
                'evaluations': json.loads(row['evaluations']),
                'statistics': json.loads(row['statistics']),
                'dropdown_options': json.loads(row['dropdown_options']),
                'identity': json.loads(row['identity'])
            }
        }
    return None

```

```

def rollback_to_version(client_id: str, version: int, rolled_back_by: str=None, rolled_back_by_type: str=None,
    ip_address: str=None) -> tuple

```

Rollback client data to a specific historical version. Creates a new version entry marking this as a rollback. Returns: Tuple of (success: bool, new_version: int)

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `client_id: str`, `version: int`, `rolled_back_by: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> tuple`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `historical = get_data_version(client_id, version)`.
2. **if** not historical: branches conditionally.
3. **return** `save_client_data_with_history(client_id, historical['data'], action....`

Code (copy-paste safe)

```

def rollback_to_version(client_id: str, version: int,
    rolled_back_by: str = None,

```

```

        rolled_back_by_type: str = None,
        ip_address: str = None) -> tuple:
    """
    Rollback client data to a specific historical version.

    Creates a new version entry marking this as a rollback.

    Returns:
        Tuple of (success: bool, new_version: int)
    """
    # Get the historical version data
    historical = get_data_version(client_id, version)
    if not historical:
        return (False, -1)

    # Save as new current data with rollback action
    return save_client_data_with_history(
        client_id,
        historical['data'],
        action='ROLLBACK',
        changed_by=rolled_back_by,
        changed_by_type=rolled_back_by_type,
        ip_address=ip_address,
        change_source='rollback',
        change_description=f'Rolled back to version {version} from {historical["created_at"]}'
    )

```

```
def compare_versions(client_id: str, version1: int, version2: int) -> dict
```

Compare two versions of client data and return differences. Returns dict with changed fields and their old/new values.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `client_id: str`, `version1: int`, `version2: int`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> dict`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.

Step by step (top-level body)

1. Assigns `v1_data = get_data_version(client_id, version1)`.
2. Assigns `v2_data = get_data_version(client_id, version2)`.
3. `if not v1_data or not v2_data`: branches conditionally.
4. Assigns `differences = {'version1': version1, 'version2': version2, 'version1_da...`
5. `for field in ['deals', 'positions', 'account', 'evaluations', 'statist...]`: iterates.
6. `return differences`.

Code (copy-paste safe)

```
def compare_versions(client_id: str, version1: int, version2: int) -> dict:
```

```

"""
Compare two versions of client data and return differences.

Returns dict with changed fields and their old/new values.
"""
v1_data = get_data_version(client_id, version1)
v2_data = get_data_version(client_id, version2)

if not v1_data or not v2_data:
    return None

differences = {
    'version1': version1,
    'version2': version2,
    'version1_date': v1_data['created_at'],
    'version2_date': v2_data['created_at'],
    'changes': {}
}

# Compare each major field
for field in ['deals', 'positions', 'account', 'evaluations', 'statistics']:
    d1 = v1_data['data'].get(field)
    d2 = v2_data['data'].get(field)

    if d1 != d2:
        if isinstance(d1, list) and isinstance(d2, list):
            differences['changes'][field] = {
                'type': 'list',
                'v1_count': len(d1),
                'v2_count': len(d2),
                'changed': True
            }
        elif isinstance(d1, dict) and isinstance(d2, dict):
            # For dicts, find specific key changes
            changed_keys = []
            all_keys = set(d1.keys()) | set(d2.keys())
            for key in all_keys:
                if d1.get(key) != d2.get(key):
                    changed_keys.append(key)
            differences['changes'][field] = {
                'type': 'dict',
                'changed_keys': changed_keys
            }
        else:
            differences['changes'][field] = {
                'type': 'other',
                'changed': True
            }

    return differences

```

```
def get_latest_version(client_id: str) -> int
```

Get the latest version number for a client.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `client_id: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and

human readers. They are the fastest way to make a public function self-explanatory.

- **return type annotation** — Annotated return type `-> int`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.

Step by step (top-level body)

1. **with** `get_connection()`: enters a context manager.

Code (copy-paste safe)

```
def get_latest_version(client_id: str) -> int:
    """Get the latest version number for a client."""
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute('''
            SELECT MAX(version) as max_version FROM data_history WHERE client_id = ?
            ''', (client_id,))
        row = cursor.fetchone()
        return row['max_version'] or 0
```

```
def cleanup_old_history(client_id: str=None, keep_versions: int=10) -> int
```

Clean up old history entries, keeping only the latest N versions per client. Also deletes any history entries older than 30 days regardless of version count. Returns the number of deleted entries.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `client_id: str`, `keep_versions: int`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> int`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.

Step by step (top-level body)

1. Assigns `cutoff = (datetime.now() - timedelta(days=30)).isoformat()`.
2. **with** `get_connection()`: enters a context manager.

Code (copy-paste safe)

```
def cleanup_old_history(client_id: str = None, keep_versions: int = 10) -> int:
    """
    Clean up old history entries, keeping only the latest N versions per client.
    Also deletes any history entries older than 30 days regardless of version count.

    Returns the number of deleted entries.
    """
    cutoff = (datetime.now() - timedelta(days=30)).isoformat()
```

```

with get_connection() as conn:
    cursor = conn.cursor()
    total_deleted = 0

    if client_id:
        clients = [client_id]
    else:
        cursor.execute('SELECT DISTINCT client_id FROM data_history')
        clients = [row['client_id'] for row in cursor.fetchall()]

    for cid in clients:
        # Delete versions beyond keep_versions limit
        cursor.execute('''
            DELETE FROM data_history
            WHERE client_id = ? AND version NOT IN (
                SELECT version FROM data_history
                WHERE client_id = ?
                ORDER BY version DESC
                LIMIT ?
            )
        ''', (cid, cid, keep_versions))
        total_deleted += cursor.rowcount

        # Also delete anything older than 30 days
        cursor.execute('''
            DELETE FROM data_history
            WHERE client_id = ? AND created_at < ?
        ''', (cid, cutoff))
        total_deleted += cursor.rowcount

    conn.commit()
    return total_deleted

```

```
def cleanup_audit_log(keep_days: int=30) -> int
```

Delete audit log entries older than keep_days. Returns count deleted.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `keep_days: int`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> int`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.

Step by step (top-level body)

1. Assigns `cutoff = (datetime.now() - timedelta(days=keep_days)).isoformat()`.
2. **with** `get_connection()`: enters a context manager.

Code (copy-paste safe)

```

def cleanup_audit_log(keep_days: int = 30) -> int:
    """Delete audit log entries older than keep_days. Returns count deleted."""
    cutoff = (datetime.now() - timedelta(days=keep_days)).isoformat()

```



```

with get_connection() as conn:
    cursor = conn.cursor()
    cursor.execute('DELETE FROM audit_log WHERE timestamp < ?', (cutoff,))
    deleted = cursor.rowcount
    conn.commit()
    return deleted

```

```
def cleanup_database() -> dict
```

Master cleanup: prune data_history, audit_log, and expired sessions. Returns a summary dict of rows deleted per table.

Why these Python concepts were used

- **return type annotation** — Annotated return type `-> dict`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to `sum`, `any`, `all`, or `max` where the intermediate list would be wasted.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.

Step by step (top-level body)

1. Imports logging (lazy import inside the function).
2. Assigns `results = {}`.
3. Assigns `results['data_history'] = cleanup_old_history(keep_versions=10)`.
4. Assigns `results['audit_log'] = cleanup_audit_log(keep_days=30)`.
5. Calls `cleanup_expired_sessions(...)` for its side effect.
6. Assigns `results['sessions'] = 'cleaned'`.
7. Assigns `total = sum((v for v in results.values() if isinstance(v, int)))`.
8. Calls `logging.info(...)` for its side effect.
9. **return** results.

Code (copy-paste safe)

```

def cleanup_database() -> dict:
    """
    Master cleanup: prune data_history, audit_log, and expired sessions.
    Returns a summary dict of rows deleted per table.
    """
    import logging
    results = {}

    results['data_history'] = cleanup_old_history(keep_versions=10)
    results['audit_log'] = cleanup_audit_log(keep_days=30)
    cleanup_expired_sessions()
    results['sessions'] = 'cleaned'

```

```
total = sum(v for v in results.values() if isinstance(v, int))
logging.info(f"Database cleanup complete: {results} ({total} rows deleted)")
return results
```

```
def log_action(action: str, user_type: str, user_identifier: str, ip_address: str=None, details: str=None,
              success: bool=True)
```

Log an action to the audit log.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `action: str`, `user_type: str`, `user_identifier: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **with** `get_connection()`: enters a context manager.

Code (copy-paste safe)

```
def log_action(action: str, user_type: str, user_identifier: str,
              ip_address: str = None, details: str = None, success: bool = True):
    """Log an action to the audit log."""
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute('''
            INSERT INTO audit_log (timestamp, action, user_type, user_identifier, ip_address, details,
                                   success)
            VALUES (?, ?, ?, ?, ?, ?, ?)
        ''', (
            datetime.now().isoformat(),
            action,
            user_type,
            user_identifier,
            ip_address,
            details,
            1 if success else 0
        ))
        conn.commit()
```

```
def get_audit_log(limit: int=100, action_filter: str=None) -> list
```

Get recent audit log entries.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `limit: int`, `action_filter: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.

- **return type annotation** — Annotated return type -> list. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **list comprehension** — Builds a list in a single expression ([f(x) for x in xs]). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **with** get_connection(): enters a context manager.

Code (copy-paste safe)

```
def get_audit_log(limit: int = 100, action_filter: str = None) -> list:
    """Get recent audit log entries."""
    with get_connection() as conn:
        cursor = conn.cursor()

        if action_filter:
            cursor.execute('''
                SELECT * FROM audit_log
                WHERE action LIKE ?
                ORDER BY timestamp DESC LIMIT ?
            ''', (f'%{action_filter}%', limit))
        else:
            cursor.execute(
                'SELECT * FROM audit_log ORDER BY timestamp DESC LIMIT ?',
                (limit,)
            )

        return [dict(row) for row in cursor.fetchall()]
```

```
def create_session(user_type: str, user_identifier: str, ip_address: str=None, hours_valid: int=24) -> str:
    """Create a new session token.
```

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., user_type: str, user_identifier: str, ip_address: str). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type -> str. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.

Step by step (top-level body)

1. Assigns user_identifier = user_identifier.strip().
2. Assigns session_token = secrets.token_urlsafe(32).

3. Assigns `now = datetime.now()`.
4. Assigns `expires = now + timedelta(hours=hours_valid)`.
5. **with** `get_connection()`: enters a context manager.
6. **return** `session_token`.

Code (copy-paste safe)

```
def create_session(user_type: str, user_identifier: str, ip_address: str = None,
                  hours_valid: int = 24) -> str:
    """Create a new session token."""
    user_identifier = user_identifier.strip()
    session_token = secrets.token_urlsafe(32)
    now = datetime.now()
    expires = now + timedelta(hours=hours_valid)

    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute('''
            INSERT INTO sessions (session_token, user_type, user_identifier, created_at, expires_at,
                                ip_address)
            VALUES (?, ?, ?, ?, ?, ?)
        ''', (session_token, user_type, user_identifier, now.isoformat(), expires.isoformat(), ip_address))
        conn.commit()

    return session_token
```

```
def validate_session(session_token: str) -> dict
```

Validate a session token and return user info if valid.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `session_token: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> dict`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.

Step by step (top-level body)

1. **with** `get_connection()`: enters a context manager.

Code (copy-paste safe)

```
def validate_session(session_token: str) -> dict:
    """Validate a session token and return user info if valid."""
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute('''
            SELECT user_type, user_identifier, expires_at FROM sessions
            WHERE session_token = ?
        ''', (session_token,))
        row = cursor.fetchone()
```

```

if row:
    expires = datetime.fromisoformat(row['expires_at'])
    if datetime.now() < expires:
        return {
            'user_type': row['user_type'],
            'user_identifier': row['user_identifier']
        }
    else:
        # Session expired, delete it
        cursor.execute('DELETE FROM sessions WHERE session_token = ?', (session_token,))
        conn.commit()

return None

```

```
def delete_session(session_token: str)
```

Delete a session (logout).

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `session_token: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.

Step by step (top-level body)

1. **with** `get_connection()`: enters a context manager.

Code (copy-paste safe)

```

def delete_session(session_token: str):
    """Delete a session (logout)."""
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute('DELETE FROM sessions WHERE session_token = ?', (session_token,))
        conn.commit()

```

```
def cleanup_expired_sessions()
```

Delete all expired sessions.

Why these Python concepts were used

- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.

Step by step (top-level body)

1. **with** `get_connection()`: enters a context manager.

Code (copy-paste safe)

```

def cleanup_expired_sessions():
    """Delete all expired sessions."""
    with get_connection() as conn:
        cursor = conn.cursor()

```

```

cursor.execute(
    'DELETE FROM sessions WHERE expires_at < ?',
    (datetime.now().isoformat(),)
)
conn.commit()

```

```
def migrate_from_json(api_keys_file: str=None, data_file: str=None)
```

Migrate data from JSON files to SQLite database.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `api_keys_file: str`, `data_file: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `base_dir = os.path.dirname(os.path.abspath(__file__))`.
2. **if** `api_keys_file` is `None`: branches conditionally.
3. **if** `data_file` is `None`: branches conditionally.
4. Assigns `migrated = {'api_keys': 0, 'clients': 0}`.
5. **if** `os.path.exists(api_keys_file)`: branches conditionally.
6. **if** `os.path.exists(data_file)`: branches conditionally.
7. **return** `migrated`.

Code (copy-paste safe)

```

def migrate_from_json(api_keys_file: str = None, data_file: str = None):
    """Migrate data from JSON files to SQLite database."""
    base_dir = os.path.dirname(os.path.abspath(__file__))

    if api_keys_file is None:
        api_keys_file = os.path.join(base_dir, 'api_keys.json')
    if data_file is None:
        data_file = os.path.join(base_dir, 'dashboard_data.json')

    migrated = {'api_keys': 0, 'clients': 0}

    # Migrate API keys (note: we can't migrate the actual keys, only the metadata)
    if os.path.exists(api_keys_file):
        try:
            with open(api_keys_file, 'r') as f:
                old_keys = json.load(f)

```

```

        print(f"Found {len(old_keys)} API keys to migrate")
        print("NOTE: Existing API keys cannot be migrated (they were stored in plain text)")
        print("You will need to generate new API keys for each trader")
        migrated['api_keys'] = len(old_keys)
    except Exception as e:
        print(f"Error reading API keys file: {e}")

# Migrate client data
if os.path.exists(data_file):
    try:
        with open(data_file, 'r') as f:
            data = json.load(f)

        clients_db = data.get('clients_db', {})
        for client_id, client_data in clients_db.items():
            save_client_data(client_id, client_data)
            migrated['clients'] += 1

        print(f"Migrated {migrated['clients']} clients")
    except Exception as e:
        print(f"Error migrating client data: {e}")

return migrated

```

Step 4.5 — Build dashboard/db.py

What to do. Create the file `dashboard/db.py` in your project root and paste in the code shown in this step. The file is **44** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from `requirements.txt`.

What this file is for. Lower-level convenience wrappers around the SQLAlchemy session for places that don't want the ORM overhead.

dashboard/db.py

44 loc · 0 classes · 1 functions · 4 imports

Lower-level convenience wrappers around the SQLAlchemy session for places that don't want the ORM overhead. It defines **1** module-level function, across **44** lines. Module docstring: *SQLAlchemy engine + session factory*.

Module docstring

SQLAlchemy engine + session factory.

Reads `DATABASE_URL` from environment (or `.env` file). Falls back to SQLite for legacy compatibility.

Set in `.env`: `DATABASE_URL=postgresql://postgres:password@localhost:5432/tradeopss`

Python concepts demonstrated in this module

- Exceptions
- f-strings
- Generators and yield

Imports — what each one is for

```
import os
```

Why imported. Operating-system interface. Path manipulation, environment variables, working-directory operations, exit codes, and thin wrappers around POSIX/Windows syscalls.

Used in this file. `os.environ`, `os.environ.get`, `os.path`, `os.path.abspath`, `os.path.dirname`, `os.path.join`

Example. `os.path.join(root, 'subdir', 'file.txt')` # joins with the OS-correct separator

Other common scenarios.

- `os.environ.get('API_KEY')` to read environment variables
- `os.makedirs(path, exist_ok=True)` to create nested directories
- `os.listdir(path)` to list a directory's contents
- `os.remove(path)` / `os.rename(src, dst)` to delete or move a file
- `os.getcwd()` / `os.chdir(path)` to inspect or change the working dir

```
from sqlalchemy import create_engine
```

Why imported. Python's most popular ORM and SQL toolkit. Models declare tables as classes; the engine handles connection pooling.

Used in this file. `create_engine`

Example. `session.query(User).filter_by(email=e).first()`

Other common scenarios.

- Column / relationship / ForeignKey declare schema
- `session.add(obj)`; `session.commit()`
- with `engine.begin()` as `conn`: `conn.execute(text('SQL'))`
- 2.0-style: `select(User).where(User.id == 1)`

```
from sqlalchemy.orm import sessionmaker
```

Why imported. Python's most popular ORM and SQL toolkit. Models declare tables as classes; the engine handles connection pooling.

Used in this file. `sessionmaker`

Example. `session.query(User).filter_by(email=e).first()`

Other common scenarios.

- Column / relationship / ForeignKey declare schema
- `session.add(obj)`; `session.commit()`
- with `engine.begin()` as `conn`: `conn.execute(text('SQL'))`

- 2.0-style: `select(User).where(User.id == 1)`

`from dotenv import load_dotenv`

Why imported. python-dotenv — load .env files into `os.environ` for twelve-factor configuration in development.

Used in this file. `load_dotenv`

Example. `load_dotenv()` # call once at process start

Other common scenarios.

- `dotenv_values('.env')` returns a dict without polluting `os.environ`
- Use a real secret store in production (vault, AWS SM, etc.)

Module constants

Each line below is followed by a plain-English note explaining what it does and why it is written that way.

```
DATABASE_URL = os.environ.get('DATABASE_URL',
    f"sqlite:///{"os.path.join(os.path.dirname(os.path.abspath(__file__)), 'dashboard.db')}")
```

Equivalent to `os.getenv`: reads `DATABASE_URL` from the environment, default `f"sqlite:///{"os.path.join(os.path.dirname(os.path.abspath(__file__)), 'dashboard.db')}"`.

Functions

```
def get_session()
```

Dependency-style session: use as context manager.

Why these Python concepts were used

- **generator (yield)** — The body uses **yield**, so calling it returns an iterator instead of a value. Items are produced lazily — the function only runs as far as the next **yield** on each iteration. Right choice when the caller iterates results and the dataset is large or open-ended.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.

Step by step (top-level body)

1. Assigns `session = SessionLocal()`.
2. **try** block with 1 **except** clause, plus a **finally**.

Code (copy-paste safe)

```
def get_session():
    """Dependency-style session: use as context manager."""
    session = SessionLocal()
    try:
        yield session
        session.commit()
    except Exception:
```

```
        session.rollback()
        raise
    finally:
        session.close()
```

Step 4.6 — Build dashboard/models.py

What to do. Create the file `dashboard/models.py` in your project root and paste in the code shown in this step. The file is **330** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from `requirements.txt`.

What this file is for. SQLAlchemy ORM models. Each class corresponds to a table; fields use Column declarations; relationships are defined via `relationship()` and back-populated on the other side.

dashboard/models.py

330 loc · 18 classes · 0 functions · 3 imports

SQLAlchemy ORM models. Each class corresponds to a table; fields use Column declarations; relationships are defined via `relationship()` and back-populated on the other side. It defines **18** classes, across **330** lines. Module docstring: *SQLAlchemy ORM Models — mirrors the existing SQLite schema exactly*.

Module docstring

SQLAlchemy ORM Models — mirrors the existing SQLite schema exactly. Target: PostgreSQL (local dev first, then production).

Usage: `from dashboard.models import Base, ClientsData, UserCredentials, ... from dashboard.db import engine, SessionLocal`

Note: `server_default` is used everywhere so that raw SQL INSERTs (which bypass the ORM) still get correct column defaults.

Python concepts demonstrated in this module

- Classes and objects
- Inheritance

Imports — what each one is for

```
from datetime import datetime
```

Why imported. Dates, times, durations. The fundamental temporal types: `date`, `time`, `datetime`, `timedelta`, `timezone`.

Used in this file. *No direct attribute access detected — likely imported for side effects, re-export, or string references.*

Example. `datetime.now() - timedelta(days=7)` # one week ago

Other common scenarios.

- `datetime.strptime(s, '%Y-%m-%d')` parses a string
- `datetime.strftime(fmt)` formats one
- `datetime.fromtimestamp(n)` converts a Unix epoch
- `datetime.utcnow()` for UTC (better: `datetime.now(timezone.utc)`)

```
from sqlalchemy import BigInteger
from sqlalchemy import Column
from sqlalchemy import Float
from sqlalchemy import ForeignKey
from sqlalchemy import Index
from sqlalchemy import Integer
from sqlalchemy import SmallInteger
from sqlalchemy import String
from sqlalchemy import Text
from sqlalchemy import UniqueConstraint
from sqlalchemy import func
from sqlalchemy import text
```

Why imported. Python's most popular ORM and SQL toolkit. Models declare tables as classes; the engine handles connection pooling.

Used in this file. `Column`, `Float`, `ForeignKey`, `Index`, `Integer`, `SmallInteger`, `Text`, `UniqueConstraint`, `func`, `text`

Example. `session.query(User).filter_by(email=e).first()`

Other common scenarios.

- `Column` / `relationship` / `ForeignKey` declare schema
- `session.add(obj)`; `session.commit()`
- with `engine.begin()` as `conn`: `conn.execute(text('SQL'))`
- 2.0-style: `select(User).where(User.id == 1)`

```
from sqlalchemy.orm import DeclarativeBase
from sqlalchemy.orm import relationship
```

Why imported. Python's most popular ORM and SQL toolkit. Models declare tables as classes; the engine handles connection pooling.

Used in this file. `DeclarativeBase`, `relationship`

Example. `session.query(User).filter_by(email=e).first()`

Other common scenarios.

- `Column` / `relationship` / `ForeignKey` declare schema
- `session.add(obj)`; `session.commit()`
- with `engine.begin()` as `conn`: `conn.execute(text('SQL'))`
- 2.0-style: `select(User).where(User.id == 1)`

Classes

```
class Base(DeclarativeBase)
```

Code (copy-paste safe)

```
class Base(DeclarativeBase):
    pass

class ApiKey(Base)

    __tablename__ = 'api_keys'
    id = Column(Integer, primary_key=True, autoincrement=True)
    key_hash = Column(Text, unique=True, nullable=False)
    key_prefix = Column(Text, nullable=False)
    admin = Column(Text, nullable=False)
    trader = Column(Text, nullable=False)
    client = Column(Text, server_default=text(''))
    scope = Column(Text, server_default=text('full'))
    created_at = Column(Text, nullable=False)
    last_used = Column(Text)
    is_active = Column(SmallInteger, server_default=text('1'))
```

Code (copy-paste safe)

```
class ApiKey(Base):
    __tablename__ = 'api_keys'

    id = Column(Integer, primary_key=True, autoincrement=True)
    key_hash = Column(Text, unique=True, nullable=False)
    key_prefix = Column(Text, nullable=False)
    admin = Column(Text, nullable=False)
    trader = Column(Text, nullable=False)
    client = Column(Text, server_default=text(''))
    scope = Column(Text, server_default=text('full'))
    created_at = Column(Text, nullable=False)
    last_used = Column(Text)
    is_active = Column(SmallInteger, server_default=text('1'))
```

```
class AdminPassword(Base)

    __tablename__ = 'admin_passwords'
    id = Column(Integer, primary_key=True, autoincrement=True)
    username = Column(Text, unique=True, nullable=False)
    password_hash = Column(Text, nullable=False)
    salt = Column(Text, nullable=False)
    created_at = Column(Text, nullable=False)
    updated_at = Column(Text)
```

Code (copy-paste safe)

```
class AdminPassword(Base):
    __tablename__ = 'admin_passwords'

    id = Column(Integer, primary_key=True, autoincrement=True)
    username = Column(Text, unique=True, nullable=False)
    password_hash = Column(Text, nullable=False)
    salt = Column(Text, nullable=False)
    created_at = Column(Text, nullable=False)
    updated_at = Column(Text)
```

```

class UserCredential(Base)

__tablename__ = 'user_credentials'
__table_args__ = (UniqueConstraint('username', 'user_type', name='uq_user_credentials_username_type'),)
id = Column(Integer, primary_key=True, autoincrement=True)
username = Column(Text, nullable=False)
email = Column(Text)
password_hash = Column(Text, nullable=False)
salt = Column(Text, nullable=False)
user_type = Column(Text, nullable=False)
parent_admin = Column(Text)
parent_trader = Column(Text)
is_active = Column(SmallInteger, server_default=text('1'))
must_change_password = Column(SmallInteger, server_default=text('1'))
last_login = Column(Text)
created_at = Column(Text, nullable=False)
updated_at = Column(Text)

```

Code (copy-paste safe)

```

class UserCredential(Base):
    __tablename__ = 'user_credentials'
    __table_args__ = (
        UniqueConstraint('username', 'user_type', name='uq_user_credentials_username_type'),
    )

    id = Column(Integer, primary_key=True, autoincrement=True)
    username = Column(Text, nullable=False)
    email = Column(Text)
    password_hash = Column(Text, nullable=False)
    salt = Column(Text, nullable=False)
    user_type = Column(Text, nullable=False)
    parent_admin = Column(Text)
    parent_trader = Column(Text)
    is_active = Column(SmallInteger, server_default=text("1"))
    must_change_password = Column(SmallInteger, server_default=text("1"))
    last_login = Column(Text)
    created_at = Column(Text, nullable=False)
    updated_at = Column(Text)


class ClientsData(Base)

__tablename__ = 'clients_data'
id = Column(Integer, primary_key=True, autoincrement=True)
client_id = Column(Text, unique=True, nullable=False)
deals = Column(Text, server_default=text('[]'))
positions = Column(Text, server_default=text('[]'))
account = Column(Text, server_default=text('{}'))
evaluations = Column(Text, server_default=text('[]'))
statistics = Column(Text, server_default=text('{}'))
dropdown_options = Column(Text, server_default=text('{}'))
identity = Column(Text, server_default=text('{}'))
last_updated = Column(Text, nullable=False)
hedge_accounts = Column(Text, server_default=text('[]'))
prop_accounts = Column(Text, server_default=text('[]'))
vps_accounts = Column(Text, server_default=text('[]'))
payment_info = Column(Text, server_default=text('[]'))
payment_address = Column(Text, server_default=text('{}'))

```

Code (copy-paste safe)

```
class ClientsData(Base):
    __tablename__ = 'clients_data'

    id = Column(Integer, primary_key=True, autoincrement=True)
    client_id = Column(Text, unique=True, nullable=False)
    deals = Column(Text, server_default=text('[]'))
    positions = Column(Text, server_default=text('[]'))
    account = Column(Text, server_default=text('{}'))
    evaluations = Column(Text, server_default=text('[]'))
    statistics = Column(Text, server_default=text('{}'))
    dropdown_options = Column(Text, server_default=text('{}'))
    identity = Column(Text, server_default=text('{}'))
    last_updated = Column(Text, nullable=False)
    hedge_accounts = Column(Text, server_default=text('[]'))
    prop_accounts = Column(Text, server_default=text('[]'))
    vps_accounts = Column(Text, server_default=text('[]'))
    payment_info = Column(Text, server_default=text('[]'))
    payment_address = Column(Text, server_default=text('{}'))

class AuditLog(Base):
    __tablename__ = 'audit_log'
    id = Column(Integer, primary_key=True, autoincrement=True)
    timestamp = Column(Text, nullable=False)
    action = Column(Text, nullable=False)
    user_type = Column(Text, nullable=False)
    user_identifier = Column(Text, nullable=False)
    ip_address = Column(Text)
    details = Column(Text)
    success = Column(SmallInteger, server_default=text('1'))
```

Code (copy-paste safe)

```
class AuditLog(Base):
    __tablename__ = 'audit_log'

    id = Column(Integer, primary_key=True, autoincrement=True)
    timestamp = Column(Text, nullable=False)
    action = Column(Text, nullable=False)
    user_type = Column(Text, nullable=False)
    user_identifier = Column(Text, nullable=False)
    ip_address = Column(Text)
    details = Column(Text)
    success = Column(SmallInteger, server_default=text("1"))

class DataHistory(Base):
    __tablename__ = 'data_history'
    __table_args__ = (UniqueConstraint('client_id', 'version', name='uq_data_history_client_version'), Index(
        'idx_data...',
    ))
    id = Column(Integer, primary_key=True, autoincrement=True)
    client_id = Column(Text, nullable=False)
    version = Column(Integer, nullable=False)
    action = Column(Text, nullable=False)
    changed_by = Column(Text)
    changed_by_type = Column(Text)
    ip_address = Column(Text)
```

```

change_source = Column(Text)
change_description = Column(Text)
deals = Column(Text, server_default=text('[]'))
positions = Column(Text, server_default=text('[]'))
account = Column(Text, server_default=text('{}'))
evaluations = Column(Text, server_default=text('[]'))
statistics = Column(Text, server_default=text('{}'))
dropdown_options = Column(Text, server_default=text('{}'))
identity = Column(Text, server_default=text('{}'))
created_at = Column(Text, nullable=False)

```

Code (copy-paste safe)

```

class DataHistory(Base):
    __tablename__ = 'data_history'
    __table_args__ = (
        UniqueConstraint('client_id', 'version', name='uq_data_history_client_version'),
        Index('idx_data_history_client', 'client_id', 'version'),
    )

    id = Column(Integer, primary_key=True, autoincrement=True)
    client_id = Column(Text, nullable=False)
    version = Column(Integer, nullable=False)
    action = Column(Text, nullable=False)
    changed_by = Column(Text)
    changed_by_type = Column(Text)
    ip_address = Column(Text)
    change_source = Column(Text)
    change_description = Column(Text)
    deals = Column(Text, server_default=text('[]'))
    positions = Column(Text, server_default=text('[]'))
    account = Column(Text, server_default=text('{}'))
    evaluations = Column(Text, server_default=text('[]'))
    statistics = Column(Text, server_default=text('{}'))
    dropdown_options = Column(Text, server_default=text('{}'))
    identity = Column(Text, server_default=text('{}'))
    created_at = Column(Text, nullable=False)

```

```

class Session(Base):
    __tablename__ = 'sessions'
    id = Column(Integer, primary_key=True, autoincrement=True)
    session_token = Column(Text, unique=True, nullable=False)
    user_type = Column(Text, nullable=False)
    user_identifier = Column(Text, nullable=False)
    created_at = Column(Text, nullable=False)
    expires_at = Column(Text, nullable=False)
    ip_address = Column(Text)

```

Code (copy-paste safe)

```

class Session(Base):
    __tablename__ = 'sessions'

    id = Column(Integer, primary_key=True, autoincrement=True)
    session_token = Column(Text, unique=True, nullable=False)
    user_type = Column(Text, nullable=False)
    user_identifier = Column(Text, nullable=False)
    created_at = Column(Text, nullable=False)
    expires_at = Column(Text, nullable=False)

```

```
ip_address      = Column(Text)
```

```
class CellNote(Base)

__tablename__ = 'cell_notes'
__table_args__ = (UniqueConstraint('client_id', 'row_index', 'column_key', name='uq_cell_notes'),)
id = Column(Integer, primary_key=True, autoincrement=True)
client_id = Column(Text, nullable=False)
row_index = Column(Integer, nullable=False)
column_key = Column(Text, nullable=False)
note_content = Column(Text)
created_by = Column(Text)
updated_at = Column(Text)
```

Code (copy-paste safe)

```
class CellNote(Base):
    __tablename__ = 'cell_notes'
    __table_args__ = (
        UniqueConstraint('client_id', 'row_index', 'column_key', name='uq_cell_notes'),
    )

    id = Column(Integer, primary_key=True, autoincrement=True)
    client_id = Column(Text, nullable=False)
    row_index = Column(Integer, nullable=False)
    column_key = Column(Text, nullable=False)
    note_content = Column(Text)
    created_by = Column(Text)
    updated_at = Column(Text)
```

```
class DailyWatermark(Base)

__tablename__ = 'daily_watermarks'
client_id = Column(Text, primary_key=True, nullable=False)
date = Column(Text, primary_key=True, nullable=False)
net_profit_complete = Column(Float, server_default=text('0.0'))
source = Column(Text, server_default=text("'auto'"))
created_at = Column(Text, server_default=func.now())
```

Code (copy-paste safe)

```
class DailyWatermark(Base):
    __tablename__ = 'daily_watermarks'

    client_id = Column(Text, primary_key=True, nullable=False)
    date = Column(Text, primary_key=True, nullable=False)
    net_profit_complete = Column(Float, server_default=text("0.0"))
    source = Column(Text, server_default=text("'auto'"))
    created_at = Column(Text, server_default=func.now())
```

```
class WaterlogPeriod(Base)

__tablename__ = 'waterlog_periods'
client_id = Column(Text, primary_key=True, nullable=False)
from_date = Column(Text, primary_key=True, nullable=False)
to_date = Column(Text, nullable=False)
period_low = Column(Float)
period_high = Column(Float)
split_pct = Column(Integer, server_default=text('50'))
```


Code (copy-paste safe)

```

class WaterlogPeriod(Base):
    __tablename__ = 'waterlog_periods'

    client_id = Column(Text, primary_key=True, nullable=False)
    from_date = Column(Text, primary_key=True, nullable=False)
    to_date = Column(Text, nullable=False)
    period_low = Column(Float)
    period_high = Column(Float)
    split_pct = Column(Integer, server_default=text("50"))

class LoginAttempt(Base):
    __tablename__ = 'login_attempts'
    id = Column(Integer, primary_key=True, autoincrement=True)
    username = Column(Text, nullable=False)
    user_type = Column(Text, nullable=False)
    ip_address = Column(Text)
    attempt_time = Column(Text, nullable=False)
    success = Column(SmallInteger, server_default=text('0'))

```

Code (copy-paste safe)

```

class LoginAttempt(Base):
    __tablename__ = 'login_attempts'

    id = Column(Integer, primary_key=True, autoincrement=True)
    username = Column(Text, nullable=False)
    user_type = Column(Text, nullable=False)
    ip_address = Column(Text)
    attempt_time = Column(Text, nullable=False)
    success = Column(SmallInteger, server_default=text("0"))

class Evaluation(Base):
    __tablename__ = 'evaluations'
    id = Column(Integer, primary_key=True, autoincrement=True)
    account_signature = Column(Text, nullable=False)
    phase_number = Column(Integer, nullable=False)
    phase_type = Column(Text, nullable=False)
    status = Column(Text, server_default=text("'pending'"))
    start_date = Column(Text)
    end_date = Column(Text)
    reset_id = Column(Text)
    parent_id = Column(Integer, ForeignKey('evaluations.id'))
    meta_data = Column(Text, server_default=text("{}"))
    created_at = Column(Text, server_default=func.now())
    children = relationship('Evaluation', backref='parent', remote_side=[id])

```

Code (copy-paste safe)

```

class Evaluation(Base):
    __tablename__ = 'evaluations'

    id = Column(Integer, primary_key=True, autoincrement=True)
    account_signature = Column(Text, nullable=False)
    phase_number = Column(Integer, nullable=False)
    phase_type = Column(Text, nullable=False)

```

```

status          = Column(Text, server_default=text("'pending'"))
start_date      = Column(Text)
end_date        = Column(Text)
reset_id        = Column(Text)
parent_id       = Column(Integer, ForeignKey('evaluations.id'))
meta_data       = Column(Text, server_default=text("{}"))
created_at      = Column(Text, server_default=func.now())

children = relationship('Evaluation', backref='parent', remote_side=[id])

```

```

class PhaseDefinition(Base)

__tablename__ = 'phase_definitions'
id = Column(Integer, primary_key=True, autoincrement=True)
phase_name = Column(Text, nullable=False)
phase_code = Column(Text, unique=True, nullable=False)
sequence_order = Column(Integer, nullable=False)
ruleset = Column(Text, server_default=text("{}"))
next_phase_code = Column(Text)

```

Code (copy-paste safe)

```

class PhaseDefinition(Base):
    __tablename__ = 'phase_definitions'

    id = Column(Integer, primary_key=True, autoincrement=True)
    phase_name = Column(Text, nullable=False)
    phase_code = Column(Text, unique=True, nullable=False)
    sequence_order = Column(Integer, nullable=False)
    ruleset = Column(Text, server_default=text("{}"))
    next_phase_code = Column(Text)


class KycLink(Base)

__tablename__ = 'kyc_links'
__table_args__ = (UniqueConstraint('primary_client', 'linked_client', name='uq_kyc_links'), Index(
    'idx_kyc_primary...',
))
id = Column(Integer, primary_key=True, autoincrement=True)
primary_client = Column(Text, nullable=False)
linked_client = Column(Text, nullable=False)
linked_by = Column(Text, server_default=text("'super_admin'"))
created_at = Column(Text, server_default=func.now())

```

Code (copy-paste safe)

```

class KycLink(Base):
    __tablename__ = 'kyc_links'
    __table_args__ = (
        UniqueConstraint('primary_client', 'linked_client', name='uq_kyc_links'),
        Index('idx_kyc_primary', 'primary_client'),
        Index('idx_kyc_linked', 'linked_client'),
    )

    id = Column(Integer, primary_key=True, autoincrement=True)
    primary_client = Column(Text, nullable=False)
    linked_client = Column(Text, nullable=False)
    linked_by = Column(Text, server_default=text("'super_admin'"))
    created_at = Column(Text, server_default=func.now())

```

```

class QualityScanResult(Base)

__tablename__ = 'quality_scan_results'
__table_args__ = (Index('idx_quality_scan_date', 'scan_date', 'client_id'),)
id = Column(Integer, primary_key=True, autoincrement=True)
scan_date = Column(Text, nullable=False)
client_id = Column(Text, nullable=False)
trader = Column(Text)
admin = Column(Text)
total_issues = Column(Integer, server_default=text('0'))
issues = Column(Text, server_default=text('[]'))
health_score = Column(Float, server_default=text('100.0'))
created_at = Column(Text, server_default=func.now())

```

Code (copy-paste safe)

```

class QualityScanResult(Base):
    __tablename__ = 'quality_scan_results'
    __table_args__ = (
        Index('idx_quality_scan_date', 'scan_date', 'client_id'),
    )

    id = Column(Integer, primary_key=True, autoincrement=True)
    scan_date = Column(Text, nullable=False)
    client_id = Column(Text, nullable=False)
    trader = Column(Text)
    admin = Column(Text)
    total_issues = Column(Integer, server_default=text("0"))
    issues = Column(Text, server_default=text("[]"))
    health_score = Column(Float, server_default=text("100.0"))
    created_at = Column(Text, server_default=func.now())


class DailyChecklist(Base)

__tablename__ = 'daily_checklists'
__table_args__ = (UniqueConstraint('date', 'user_identifier', 'checklist_type', 'client_id',
    name='uq_daily_checkl...
id = Column(Integer, primary_key=True, autoincrement=True)
date = Column(Text, nullable=False)
user_identifier = Column(Text, nullable=False)
user_type = Column(Text, nullable=False)
checklist_type = Column(Text, nullable=False)
client_id = Column(Text, server_default=text(''))
items = Column(Text, server_default=text('[]'))
submitted_at = Column(Text, nullable=False)
ip_address = Column(Text)

```

Code (copy-paste safe)

```

class DailyChecklist(Base):
    __tablename__ = 'daily_checklists'
    __table_args__ = (
        UniqueConstraint('date', 'user_identifier', 'checklist_type', 'client_id',
            name='uq_daily_checklists'),
        Index('idx_checklist_date', 'date', 'user_identifier'),
        Index('idx_checklist_client', 'date', 'client_id'),
    )

    id = Column(Integer, primary_key=True, autoincrement=True)
    date = Column(Text, nullable=False)

```

```

user_identifier = Column(Text, nullable=False)
user_type      = Column(Text, nullable=False)
checklist_type = Column(Text, nullable=False)
client_id      = Column(Text, server_default=text(''))
items          = Column(Text, server_default=text('[]'))
submitted_at   = Column(Text, nullable=False)
ip_address     = Column(Text)

```

```

class SystemSetting(Base)

__tablename__ = 'system_settings'
key = Column(Text, primary_key=True)
value = Column(Text, nullable=False)
updated_at = Column(Text, nullable=False)
updated_by = Column(Text, server_default=text(''))

```

Code (copy-paste safe)

```

class SystemSetting(Base):
    __tablename__ = 'system_settings'

    key          = Column(Text, primary_key=True)
    value        = Column(Text, nullable=False)
    updated_at   = Column(Text, nullable=False)
    updated_by   = Column(Text, server_default=text(''))

```

Checkpoint — what works now. Your database schema exists. Run alembic upgrade head from the project root and the dashboard.db file should appear with all tables created. Open it with sqlite3 dashboard.db .schema to inspect the columns. You cannot yet write rows from application code — that comes in phase 9 — but the schema is in place.

Chapter 5 — Phase 3 — Talking to MetaTrader 5

The MT5 terminal exposes a Python API through the MetaTrader5 package. Before we can place a trade or read history, we need a thin wrapper that opens and closes the connection cleanly. That wrapper lives in `connectors/`; everything in later phases that talks to MT5 goes through it.

Files we build in this phase, in order:

- `connectors/mt5_connector.py`
- `connectors/mt5_automator.py`

Python concepts you'll meet in this phase

- Dunder methods (2) · f-strings (2) · Classes and objects (2) · Exceptions (2) · Comprehensions (1) · Lambdas (1) · Context managers (with-statements) (1)

Each is explained in Chapter 2 (the primer). The number in parentheses is how many of this phase's files use that concept.

Step 5.1 — Build `connectors/mt5_connector.py`

What to do. Create the file `connectors/mt5_connector.py` in your project root and paste in the code shown in this step. The file is **198** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from `requirements.txt`.

What this file is for. Manages the lifecycle of the MetaTrader 5 connection: initialize, login, shutdown, and reconnection on failure.

`connectors/mt5_connector.py`

198 loc · 1 classes · 0 functions · 2 imports

Manages the lifecycle of the MetaTrader 5 connection: initialize, login, shutdown, and reconnection on failure. It defines **1** class, across **198** lines.

Python concepts demonstrated in this module

- Classes and objects · Dunder methods · Exceptions · f-strings

Imports — what each one is for

```
import MetaTrader5 as mt5
```

Why imported. Official MetaQuotes Python API for MT5. Connects to a local terminal, places orders, reads tick/bar history.

Used in this file. `mt5.ORDER_FILLING_IOC`, `mt5.ORDER_TIME_GTC`, `mt5.ORDER_TYPE_BUY`, `mt5.ORDER_TYPE_SELL`, `mt5.TRADE_ACTION_DEAL`, `mt5.TRADE_RETCODE_DONE`, `mt5.account_info`, `mt5.history_deals_get`

Example. `mt5.initialize()`; `mt5.order_send(request)`

Other common scenarios.

- `mt5.symbols_get()` / `mt5.symbol_info(symbol)`
- `mt5.history_deals_get(from_, to_)` returns closed deals
- `mt5.account_info()` returns balance/equity/currency
- Always `mt5.shutdown()` in a finally to release the terminal

```
import pandas as pd
```

Why imported. DataFrame library — the workhorse for tabular data: loading CSV/Excel/SQL, joins, aggregations, time-series.

Used in this file. *No direct attribute access detected — likely imported for side effects, re-export, or string references.*

Example. `df = pd.read_csv(path)`; `df.groupby('symbol')['pnl'].sum()`

Other common scenarios.

- `df.merge(other, on='key')` joins like SQL
- `df.resample('1D').agg(...)` for time-series rebucketing
- `df.to_sql` / `df.to_excel` / `df.to_parquet` for output
- `df.loc[mask, 'col']` for label-based selection/assignment

Classes

```
class MT5Connector
```

Code (copy-paste safe)

```
class MT5Connector:
    def __init__(self, login, password, server, terminal_path=None):
        self.login = login
        self.password = password
        self.server = server
        self.terminal_path = terminal_path
        self.connected = False

    def connect(self):
        init_params = {}
        if self.terminal_path and "exe" in self.terminal_path:
            init_params["path"] = self.terminal_path

        if not mt5.initialize(**init_params):
            print("initialize() failed, error code =", mt5.last_error())
```

```

        return False

    if not self.login or not self.password or not self.server:
        print("MT5 Credentials missing. Skipping login.")
        return False

    authorized = mt5.login(self.login, password=self.password, server=self.server)
    if authorized:
        self.connected = True
        print(f"Connected to MT5 account #{self.login}")
    else:
        print(f"failed to connect at account #{self.login}, error code: {mt5.last_error()}")

    return self.connected

def shutdown(self):
    mt5.shutdown()
    self.connected = False

def get_deals(self, days=30, from_date=None):
    # Ensure MT5 is initialized and logged in for the current thread
    init_params = {}
    if self.terminal_path and "exe" in self.terminal_path:
        init_params["path"] = self.terminal_path

    if not mt5.initialize(**init_params):
        print(f"get_deals: mt5.initialize() failed with path: {self.terminal_path}")
        print("Error code:", mt5.last_error())
        return []

    if self.login and self.password and self.server:
        # Ensure login is int
        try:
            login_int = int(self.login)
        except:
            print(f"Invalid login format: {self.login}")
            return []

        if not mt5.login(login_int, password=self.password, server=self.server):
            print(f"get_deals: mt5.login failed for {self.login}")
            print("Error code:", mt5.last_error())
            return []

    import time
    to_timestamp = time.time() + 86400 # Add buffer (1 day)

    if from_date:
        from_timestamp = from_date.timestamp()
    elif days is None:
        from_timestamp = 0.0
    else:
        # Calculate from_timestamp relative to NOW, not the buffered to_timestamp
        from_timestamp = time.time() - (days * 24 * 3600)

    try:
        # Use timestamps to avoid datetime issues
        deals = mt5.history_deals_get(from_timestamp, to_timestamp)
    except Exception as e:
        print(f"get_deals: Exception calling history_deals_get: {e}")
        return []

```

```

        if deals is None:
            print("No deals found. Error code={}".format(mt5.last_error()))
            return []

        return deals

def get_deals_by_position(self, position_id):
    if not self.connected: return []
    try:
        deals = mt5.history_deals_get(position=position_id)
        if deals is None:
            return []
        return deals
    except Exception as e:
        print(f"get_deals_by_position error: {e}")
        return []

def get_account_info(self):
    if not self.connected: return None
    try:
        return mt5.account_info()
    except Exception as e:
        print(f"get_account_info error: {e}")
        return None

def get_positions(self):
    if not self.connected: return []
    try:
        positions = mt5.positions_get()
        if positions is None:
            return []
        return positions
    except Exception as e:
        print(f"get_positions error: {e}")
        return []
    except Exception as e:
        print(f"get_account_info error: {e}")
        return None

def get_positions(self):
    if not self.connected: return []
    try:
        positions = mt5.positions_get()
        if positions is None:
            return []
        return positions
    except Exception as e:
        print(f"get_positions error: {e}")
        return []

def place_order(self, symbol, order_type, volume, sl_points=None, tp_points=None, comment=""):
    """
    Places a market order.
    order_type: 'BUY' or 'SELL'
    sl_points: Stop Loss in points (optional)
    tp_points: Take Profit in points (optional)
    """
    if not self.connected:
        if not self.connect():
            return False

```



```

# Ensure symbol is selected
if not mt5.symbol_select(symbol, True):
    print(f"Failed to select symbol {symbol}")
    return False

symbol_info = mt5.symbol_info(symbol)
if symbol_info is None:
    print(f"{symbol} not found")
    return False

# Determine order type and price
if order_type.upper() == 'BUY':
    mt5_type = mt5.ORDER_TYPE_BUY
    price = mt5.symbol_info_tick(symbol).ask
elif order_type.upper() == 'SELL':
    mt5_type = mt5.ORDER_TYPE_SELL
    price = mt5.symbol_info_tick(symbol).bid
else:
    print(f"Invalid order type: {order_type}")
    return False

request = {
    "action": mt5.TRADE_ACTION_DEAL,
    "symbol": symbol,
    "volume": float(volume),
    "type": mt5_type,
    "price": price,
    "deviation": 20,
    "magic": 234000,
    "comment": comment,
    "type_time": mt5.ORDER_TIME_GTC,
    "type_filling": mt5.ORDER_FILLING_IOC,
}

# Calculate SL/TP prices if points provided
point = symbol_info.point
if sl_points:
    if mt5_type == mt5.ORDER_TYPE_BUY:
        request["sl"] = price - (sl_points * point)
    else:
        request["sl"] = price + (sl_points * point)

if tp_points:
    if mt5_type == mt5.ORDER_TYPE_BUY:
        request["tp"] = price + (tp_points * point)
    else:
        request["tp"] = price - (tp_points * point)

# Send order
result = mt5.order_send(request)

if result.retcode != mt5.TRADE_RETCODE_DONE:
    print(f"Order failed: {result.comment} ({result.retcode})")
    return False

print(f"Order placed successfully: {result.order}")
return True

```

Method: MT5Connector.__init__

```
def __init__(self, login, password, server, terminal_path=None)
```

Why these Python concepts were used

- **__init__** — The constructor. Runs after Python has allocated the instance; assigns starting attribute values to self.

Step by step (top-level body)

1. Assigns self.login = login.
2. Assigns self.password = password.
3. Assigns self.server = server.
4. Assigns self.terminal_path = terminal_path.
5. Assigns self.connected = False.

Code (copy-paste safe)

```
def __init__(self, login, password, server, terminal_path=None):
    self.login = login
    self.password = password
    self.server = server
    self.terminal_path = terminal_path
    self.connected = False
```

Method: MT5Connector.connect

```
def connect(self)
```

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns init_params = {}.
2. **if** self.terminal_path and 'exe' in self.terminal_path: branches conditionally.
3. **if** not mt5.initialize(**init_params): branches conditionally.
4. **if** not self.login or not self.password or (not self.server): branches conditionally.
5. Assigns authorized = mt5.login(self.login, password=self.password, server=self....
6. **if** authorized: branches conditionally (with an **else/elif** arm).
7. **return** self.connected.

Code (copy-paste safe)

```
def connect(self):
    init_params = {}
    if self.terminal_path and "exe" in self.terminal_path:
        init_params["path"] = self.terminal_path

    if not mt5.initialize(**init_params):
        print("initialize() failed, error code =", mt5.last_error())
        return False
```

```

    if not self.login or not self.password or not self.server:
        print("MT5 Credentials missing. Skipping login.")
        return False

    authorized = mt5.login(self.login, password=self.password, server=self.server)
    if authorized:
        self.connected = True
        print(f"Connected to MT5 account #{self.login}")
    else:
        print("failed to connect at account #{}, error code: {}".format(self.login, mt5.last_error()))

    return self.connected

```

Method: MT5Connector.shutdown

```
def shutdown(self)
```

Step by step (top-level body)

1. Calls `mt5.shutdown(...)` for its side effect.
2. Assigns `self.connected = False`.

Code (copy-paste safe)

```

def shutdown(self):
    mt5.shutdown()
    self.connected = False

```

Method: MT5Connector.get_deals

```
def get_deals(self, days=30, from_date=None)
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `init_params = {}`.
2. **if** `self.terminal_path` and `'exe'` in `self.terminal_path`: branches conditionally.
3. **if** `not mt5.initialize(**init_params)`: branches conditionally.
4. **if** `self.login` and `self.password` and `self.server`: branches conditionally.
5. Imports `time` (lazy import inside the function).
6. Assigns `to_timestamp = time.time() + 86400`.
7. **if** `from_date`: branches conditionally (with an **else/elif** arm).
8. **try** block with 1 **except** clause.
9. **if** `deals` is `None`: branches conditionally.

10. return deals.

Code (copy-paste safe)

```
def get_deals(self, days=30, from_date=None):
    # Ensure MT5 is initialized and logged in for the current thread
    init_params = {}
    if self.terminal_path and "exe" in self.terminal_path:
        init_params["path"] = self.terminal_path

    if not mt5.initialize(**init_params):
        print(f"get_deals: mt5.initialize() failed with path: {self.terminal_path}")
        print("Error code:", mt5.last_error())
        return []

    if self.login and self.password and self.server:
        # Ensure login is int
        try:
            login_int = int(self.login)
        except:
            print(f"Invalid login format: {self.login}")
            return []

        if not mt5.login(login_int, password=self.password, server=self.server):
            print(f"get_deals: mt5.login failed for {self.login}")
            print("Error code:", mt5.last_error())
            return []

    import time
    to_timestamp = time.time() + 86400 # Add buffer (1 day)

    if from_date:
        from_timestamp = from_date.timestamp()
    elif days is None:
        from_timestamp = 0.0
    else:
        # Calculate from_timestamp relative to NOW, not the buffered to_timestamp
        from_timestamp = time.time() - (days * 24 * 3600)

    try:
        # Use timestamps to avoid datetime issues
        deals = mt5.history_deals_get(from_timestamp, to_timestamp)
    except Exception as e:
        print(f"get_deals: Exception calling history_deals_get: {e}")
        return []

    if deals is None:
        print("No deals found. Error code={}".format(mt5.last_error()))
        return []

    return deals
```

Method: MT5Connector.get_deals_by_position

```
def get_deals_by_position(self, position_id)
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't

have to handle it.

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **if** not self.connected: branches conditionally.
2. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def get_deals_by_position(self, position_id):
    if not self.connected: return []
    try:
        deals = mt5.history_deals_get(position=position_id)
        if deals is None:
            return []
        return deals
    except Exception as e:
        print(f"get_deals_by_position error: {e}")
        return []
```

Method: MT5Connector.get_account_info

```
def get_account_info(self)
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **if** not self.connected: branches conditionally.
2. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def get_account_info(self):
    if not self.connected: return None
    try:
        return mt5.account_info()
    except Exception as e:
        print(f"get_account_info error: {e}")
        return None
```

Method: MT5Connector.get_positions

```
def get_positions(self)
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **if** not self.connected: branches conditionally.
2. **try** block with 2 **except** clauses.

Code (copy-paste safe)

```
def get_positions(self):
    if not self.connected: return []
    try:
        positions = mt5.positions_get()
        if positions is None:
            return []
        return positions
    except Exception as e:
        print(f"get_positions error: {e}")
        return []
    except Exception as e:
        print(f"get_account_info error: {e}")
        return None
```

Method: MT5Connector.get_positions

```
def get_positions(self)
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **if** not self.connected: branches conditionally.
2. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def get_positions(self):
    if not self.connected: return []
    try:
        positions = mt5.positions_get()
        if positions is None:
            return []
        return positions
    except Exception as e:
        print(f"get_positions error: {e}")
        return []
```

Method: MT5Connector.place_order

```
def place_order(self, symbol, order_type, volume, sl_points=None, tp_points=None, comment=''):
```

Places a market order. order_type: 'BUY' or 'SELL' sl_points: Stop Loss in points (optional) tp_points: Take Profit in points (optional)

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **if** not self.connected: branches conditionally.
2. **if** not mt5.symbol_select(symbol, True): branches conditionally.
3. Assigns symbol_info = mt5.symbol_info(symbol).
4. **if** symbol_info is None: branches conditionally.
5. **if** order_type.upper() == 'BUY': branches conditionally (with an **else/elif** arm).
6. Assigns request = {'action': mt5.TRADE_ACTION_DEAL, 'symbol': symbol, 'volu....
7. Assigns point = symbol_info.point.
8. **if** sl_points: branches conditionally.
9. **if** tp_points: branches conditionally.
10. Assigns result = mt5.order_send(request).
11. **if** result.retcode != mt5.TRADE_RETCODE_DONE: branches conditionally.
12. Calls print(...) for its side effect.
13. **return** True.

Code (copy-paste safe)

```
def place_order(self, symbol, order_type, volume, sl_points=None, tp_points=None, comment=""):
    """
    Places a market order.
    order_type: 'BUY' or 'SELL'
    sl_points: Stop Loss in points (optional)
    tp_points: Take Profit in points (optional)
    """
    if not self.connected:
        if not self.connect():
            return False

    # Ensure symbol is selected
    if not mt5.symbol_select(symbol, True):
        print(f"Failed to select symbol {symbol}")
        return False

    symbol_info = mt5.symbol_info(symbol)
    if symbol_info is None:
        print(f"{symbol} not found")
        return False
```

```

# Determine order type and price
if order_type.upper() == 'BUY':
    mt5_type = mt5.ORDER_TYPE_BUY
    price = mt5.symbol_info_tick(symbol).ask
elif order_type.upper() == 'SELL':
    mt5_type = mt5.ORDER_TYPE_SELL
    price = mt5.symbol_info_tick(symbol).bid
else:
    print(f"Invalid order type: {order_type}")
    return False

request = {
    "action": mt5.TRADE_ACTION_DEAL,
    "symbol": symbol,
    "volume": float(volume),
    "type": mt5_type,
    "price": price,
    "deviation": 20,
    "magic": 234000,
    "comment": comment,
    "type_time": mt5.ORDER_TIME_GTC,
    "type_filling": mt5.ORDER_FILLING_IOC,
}

# Calculate SL/TP prices if points provided
point = symbol_info.point
if sl_points:
    if mt5_type == mt5.ORDER_TYPE_BUY:
        request["sl"] = price - (sl_points * point)
    else:
        request["sl"] = price + (sl_points * point)

if tp_points:
    if mt5_type == mt5.ORDER_TYPE_BUY:
        request["tp"] = price + (tp_points * point)
    else:
        request["tp"] = price - (tp_points * point)

# Send order
result = mt5.order_send(request)

if result.retcode != mt5.TRADE_RETCODE_DONE:
    print(f"Order failed: {result.comment} ({result.retcode})")
    return False

print(f"Order placed successfully: {result.order}")
return True

```

Step 5.2 — Build connectors/mt5_automator.py

What to do. Create the file `connectors/mt5_automator.py` in your project root and paste in the code shown in this step. The file is **2228** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from `requirements.txt`.

What this file is for. Higher-level helper for issuing MT5 commands without callers having to handle the request/result tuples directly.

connectors/mt5_automator.py

2228 loc · 1 classes · 2 functions · 12 imports

Higher-level helper for issuing MT5 commands without callers having to handle the request/result tuples directly. It defines **1** class and **2** module-level functions, across **2,228** lines.

Python concepts demonstrated in this module

- Classes and objects
- Comprehensions
- Context managers (with-statements)
- Dunder methods
- Exceptions
- f-strings
- Lambdas

Imports — what each one is for

```
import sys
```

Why imported. Access to the running interpreter — `argv`, `stdin/stdout`, exit codes, the import search path, and the platform name.

Used in this file. `sys._MEIPASS`

Example. `sys.exit(1)` # terminate the process with a non-zero status

Other common scenarios.

- `sys.argv[1:]` reads the command-line arguments
- `sys.path.insert(0, 'extra')` prepends to the import search path
- `sys.stderr.write(msg)` writes to standard error
- `sys.platform` tells you 'win32', 'linux', or 'darwin'
- `sys.modules` is the global cache of already-imported modules

```
import os
```

Why imported. Operating-system interface. Path manipulation, environment variables, working-directory operations, exit codes, and thin wrappers around POSIX/Windows syscalls.

Used in this file. `os.add_dll_directory`, `os.environ`, `os.environ.get`, `os.getenv`, `os.path`, `os.path.abspath`, `os.path.basename`, `os.path.dirname`

Example. `os.path.join(root, 'subdir', 'file.txt')` # joins with the OS-correct separator

Other common scenarios.

- `os.environ.get('API_KEY')` to read environment variables
- `os.makedirs(path, exist_ok=True)` to create nested directories
- `os.listdir(path)` to list a directory's contents
- `os.remove(path)` / `os.rename(src, dst)` to delete or move a file
- `os.getcwd()` / `os.chdir(path)` to inspect or change the working dir

`import winreg`

Why imported. Read and write the Windows registry — used to discover installed MetaTrader terminals.

Used in this file. `winreg.EnumKey`, `winreg.HKEY_CURRENT_USER`, `winreg.HKEY_LOCAL_MACHINE`, `winreg.OpenKey`, `winreg.QueryValueEx`

Example. `winreg.OpenKey(HKEY_LOCAL_MACHINE, r'Software\X')`

Other common scenarios.

- `QueryValueEx` returns a (value, type) tuple
- `EnumKey` / `EnumValue` iterate sub-keys

`from pathlib import Path`

Why imported. Object-oriented filesystem paths. Replaces most uses of `os.path` with chainable methods and operator overloading.

Used in this file. *No direct attribute access detected — likely imported for side effects, re-export, or string references.*

Example. `Path('logs') / f'{name}.log' # /` is overloaded to join

Other common scenarios.

- `p.exists()`, `p.is_file()`, `p.is_dir()` for predicates
- `p.read_text()` / `p.write_text(content)` for one-shot file I/O
- `p.glob('**/*.py')` to walk a tree with a pattern
- `p.with_suffix('.bak')` to derive a sibling path

`from dotenv import load_dotenv`

Why imported. `python-dotenv` — load `.env` files into `os.environ` for twelve-factor configuration in development.

Used in this file. `load_dotenv`

Example. `load_dotenv()` # call once at process start

Other common scenarios.

- `dotenv_values('.env')` returns a dict without polluting `os.environ`
- Use a real secret store in production (vault, AWS SM, etc.)

`import logging`

Why imported. Structured, levelled logging. Replaces print() with filterable, formattable output that can route to files, stderr, syslog, or HTTP endpoints.

Used in this file. logging.INFO, logging.basicConfig, logging.debug, logging.error, logging.exception, logging.info, logging.warning

Example. logging.getLogger(__name__).info('connected to %s', host)

Other common scenarios.

- .debug() / .info() / .warning() / .error() / .exception()
- logger.exception() captures the current traceback automatically
- logging.basicConfig(level=logging.INFO) for quick setup
- Handlers and Formatters route logs to files / streams / syslog

import subprocess

Why imported. Run external programs. The standard way to spawn a child process, capture its output, and wait for its exit code.

Used in this file. subprocess.run

Example. subprocess.run(['git', 'rev-parse', 'HEAD'], capture_output=True, text=True, check=True)

Other common scenarios.

- Popen(...) for streaming I/O while the process is alive
- check_output(cmd) returns stdout as text and raises on failure
- DEVNULL / PIPE redirect stdin/stdout/stderr
- shell=True interprets the string with the system shell (security risk)

import psutil

Why imported. Cross-platform process and system information — find a running terminal, check CPU/memory, kill a stale process.

Used in this file. psutil.AccessDenied, psutil.NoSuchProcess, psutil.ZombieProcess, psutil.disk_partitions, psutil.process_iter

Example. psutil.process_iter(['name']) # iterate live processes

Other common scenarios.

- psutil.Process(pid).terminate() / .kill()
- psutil.cpu_percent(interval=1.0) measures CPU usage
- psutil.virtual_memory() returns RAM stats

import time

Why imported. Timestamps and sleep. Lower-level than datetime — works in Unix-epoch seconds.

Used in this file. time.sleep, time.time

Example. time.sleep(1.5) # pause this thread for 1.5 seconds

Other common scenarios.

- `time.time()` returns seconds since the epoch
- `time.monotonic()` for measuring elapsed time (never goes backward)
- `time.perf_counter()` for high-precision benchmarking
- `time.strftime` / `time.strptime` mirror the `datetime` versions

```
from time import sleep
```

Why imported. Timestamps and `sleep`. Lower-level than `datetime` — works in Unix-epoch seconds.

Used in this file. `sleep`

Example. `time.sleep(1.5)` # pause this thread for 1.5 seconds

Other common scenarios.

- `time.time()` returns seconds since the epoch
- `time.monotonic()` for measuring elapsed time (never goes backward)
- `time.perf_counter()` for high-precision benchmarking
- `time.strftime` / `time.strptime` mirror the `datetime` versions

```
import ctypes
```

Why imported. Call C functions from shared libraries (DLLs / `.so` / `.dylib`). Used here to reach Win32 APIs the `stdlib` doesn't expose.

Used in this file. *No direct attribute access detected — likely imported for side effects, re-export, or string references.*

Example. `ctypes.windll.user32.MessageBoxW(0, 'hi', 't', 0)`

Other common scenarios.

- `CDLL('libfoo.so')` loads a Unix shared object
- Define `argtypes`/`restype` to get type-checked calls
- `c_int` / `c_char_p` / Structure mirror C type layouts

```
from ctypes import wintypes
```

Why imported. Call C functions from shared libraries (DLLs / `.so` / `.dylib`). Used here to reach Win32 APIs the `stdlib` doesn't expose.

Used in this file. *No direct attribute access detected — likely imported for side effects, re-export, or string references.*

Example. `ctypes.windll.user32.MessageBoxW(0, 'hi', 't', 0)`

Other common scenarios.

- `CDLL('libfoo.so')` loads a Unix shared object
- Define `argtypes`/`restype` to get type-checked calls
- `c_int` / `c_char_p` / Structure mirror C type layouts

Classes

```

class MT5Automator
    _symbol_cache = {}
    _symbol_cache_timestamp = {}
    _cache_ttl = 300

```

Code (copy-paste safe)

```

class MT5Automator:
    # Class-level symbol cache to persist across instances
    _symbol_cache = {}
    _symbol_cache_timestamp = {}
    _cache_ttl = 300 # Cache for 5 minutes

    def __init__(self, login, password, server, symbol=None, terminal_path=None):
        # Safely convert login to integer
        try:
            self.login = int(str(login).strip()) if login else 0
        except (ValueError, TypeError):
            logging.error(f'Invalid login format: {login}')
            self.login = 0

        self.password = str(password) if password else ""
        self.server = str(server) if server else ""
        self.symbol = symbol
        self.terminal_path = terminal_path
        self.sl_points = float(os.getenv('MT5_SL_POINTS') or os.getenv('MT5_STOPLOSS_POINTS', '0'))
        self.tp_points = float(os.getenv('MT5_TP_POINTS') or os.getenv('MT5_TAKEPROFIT_POINTS', '0'))
        self.default_volume = float(os.getenv('MT5_VOLUME', '1'))

        # Rollover safety tracking - prevents multiple executions per day
        self.rollover_executed_today = {} # {prop_firm: date_string}

        # Store the actually connected symbol (will be set after successful connection)
        self.connected_symbol = None

        # Check if this is a PlexyTrade server (case-insensitive substring match)
        self.is_plexy_server = "plexy" in server.lower() if server else False
        if self.is_plexy_server:
            logging.info(f"PlexyTrade server detected: {server} - Lot sizes will be divided by 20")

        # Ensure all instance variables are properly initialized
        self.connected = False
        self.last_error = None

    def _get_cached_terminal_path(self):
        """Get previously successful terminal path for faster connection"""
        import tempfile
        cache_file = os.path.join(tempfile.gettempdir(), "mt5_terminal_cache.txt")
        try:
            if os.path.exists(cache_file):
                with open(cache_file, 'r') as f:
                    cached_path = f.read().strip()
                    if os.path.exists(cached_path):
                        return cached_path
        except Exception as e:
            logging.debug(f"Cache read failed: {e}")
        return None

    def _cache_successful_path(self, path):
        """Cache successful terminal path for future use"""

```

```

import tempfile
cache_file = os.path.join(tempfile.gettempdir(), "mt5_terminal_cache.txt")
try:
    with open(cache_file, 'w') as f:
        f.write(path)
    logging.info(f"[OK] Cached successful MT5 path: {path}")
except Exception as e:
    logging.debug(f"Cache write failed: {e}")

def connect(self):
    # SPEED OPTIMIZATION: Try cached terminal path first
    success = False
    cached_path = self._get_cached_terminal_path()
    if cached_path:
        if mt5.initialize(path=cached_path):
            logging.info(f'[FAST] MT5 initialized with cached path: {cached_path}')
            success = True
        else:
            logging.info(f'Cached path failed, trying alternatives: {cached_path}')

    # Try to initialize MT5 with the best available path
    if not success:
        # CRITICAL FIX: If user specifies a terminal path, ONLY use that terminal
        if self.terminal_path and self.terminal_path.strip():
            terminal_exe = os.path.join(self.terminal_path, "terminal64.exe")
            if os.path.exists(terminal_exe):
                if mt5.initialize(path=terminal_exe):
                    logging.info(f'MT5 initialized successfully with user-specified path: {terminal_exe}')
                    self._cache_successful_path(terminal_exe) # Cache success
                    success = True
                else:
                    logging.error(f'MT5 initialize failed for user-specified path: {terminal_exe}')
                    # Don't try other terminals when user specified one - respect their choice
                    error_code, error_msg = mt5.last_error()
                    self.last_error = f'MT5 initialize failed for selected terminal: {error_msg} (Code: {error_code})'
                    logging.error(self.last_error)
                    return False
            else:
                logging.error(f'User-specified terminal executable not found: {terminal_exe}')
                self.last_error = f'Terminal executable not found: {terminal_exe}'
                return False

    # If specific paths failed, try other available installations
    if not success:
        terminals = get_installed_mt5_terminals()
        for terminal in terminals:
            terminal_exe = os.path.join(terminal["path"], "terminal64.exe")
            if os.path.exists(terminal_exe):
                if mt5.initialize(path=terminal_exe):
                    logging.info(f'MT5 initialized successfully with detected path: {terminal_exe}')
                    self._cache_successful_path(terminal_exe) # Cache success
                    success = True
                    break
            else:
                logging.warning(f'MT5 initialize failed for detected path: {terminal_exe}')

    # Last resort: try default initialization
    if not success:
        if mt5.initialize():
            logging.info('MT5 initialized successfully with default path')

```

```

        success = True
    else:
        logging.error('MT5 initialize failed with default path')

if not success:
    error_code, error_msg = mt5.last_error()
    self.last_error = f'MT5 initialize failed: {error_msg} (Code: {error_code})'
    logging.error(self.last_error)
    return False

authorized = mt5.login(self.login, self.password, self.server)
if not authorized:
    error_msg = f'MT5 login failed for login={self.login}, server={self.server}'
    logging.error(error_msg)
    self.last_error = error_msg
    return False

# SPEED OPTIMIZATION: Fast symbol detection after successful connection
try:
    # Quick symbol detection - try user symbol first
    if self.symbol and mt5.symbol_select(self.symbol, True):
        self.connected_symbol = self.symbol
        logging.info(f"[FAST] Fast symbol detection: {self.connected_symbol}")
    else:
        # Quick fallback to first available symbol
        symbols = mt5.symbols_get()
        if symbols and len(symbols) > 0:
            self.connected_symbol = symbols[0].name
            if mt5.symbol_select(self.connected_symbol, True):
                logging.info(f"[FAST] Fast fallback symbol: {self.connected_symbol}")
            else:
                # Last resort: use EURUSD as default
                self.connected_symbol = "EURUSD"
                logging.info(f"[FAST] Default symbol: {self.connected_symbol}")
        else:
            self.connected_symbol = "EURUSD" # Safe default

except Exception as e:
    logging.warning(f"Fast symbol detection failed: {e}")
    self.connected_symbol = self.symbol if self.symbol else "EURUSD"

self.connected = True
return True

def monitor_connection(self):
    """
    Monitor MT5 connection status and attempt recovery if needed
    Call this periodically (e.g., every 30 seconds) during application operation
    """
    try:
        # Quick connection check
        terminal_info = mt5.terminal_info()
        if not terminal_info or not terminal_info.connected:
            logging.warning("■ MT5 connection monitor detected disconnection")
            return self.attempt_reconnection()

        # Check if trading is still allowed
        if not terminal_info.trade_allowed:
            logging.warning("■ MT5 connection monitor detected trading disabled")
            return False
    
```

```

        # Check account access
        account_info = mt5.account_info()
        if not account_info:
            logging.warning("■ MT5 connection monitor detected account access issues")
            return self.attempt_reconnection()

        return True

    except Exception as e:
        logging.error(f"Error in connection monitor: {e}")
        return False

def ensure_session_integrity(self):
    """
    Ensure MT5 session is properly initialized and maintained
    Call this at application startup and periodically during operation
    """
    try:
        logging.info("[SETUP] Ensuring MT5 session integrity...")

        # Check if MT5 is initialized
        if not mt5.initialize():
            logging.warning("MT5 not initialized, attempting to initialize...")
            if not mt5.initialize():
                logging.error("[ERROR] Failed to initialize MT5")
                return False

        # Check if we're logged in
        if not mt5 terminal_info():
            logging.warning("MT5 terminal info not available, attempting connection...")
            if not self.connect():
                logging.error("[ERROR] Failed to connect to MT5")
                return False

        # Perform comprehensive health check
        is_healthy, health_msg = self.check_connection_health()
        if not is_healthy:
            logging.warning(f"MT5 health check failed: {health_msg}, attempting recovery...")
            if not self.attempt_reconnection():
                logging.error("[ERROR] MT5 recovery failed")
                return False

        # Ensure symbol is properly selected
        if self.symbol:
            symbol_info = mt5.symbol_info(self.symbol)
            if symbol_info and not symbol_info.visible:
                logging.info(f"Ensuring symbol {self.symbol} is selected...")
                mt5.symbol_select(self.symbol, True)

        logging.info("[OK] MT5 session integrity confirmed")
        return True

    except Exception as e:
        logging.error(f"Error ensuring MT5 session: {e}")
        return False

def attempt_reconnection(self, max_retries=3):
    """
    Attempt to reconnect to MT5 if connection is lost
    """

```



```

for attempt in range(max_retries):
    try:
        logging.info(f"■ Attempting MT5 reconnection (attempt {attempt + 1}/{max_retries})...")

        # Shutdown current connection
        mt5.shutdown()

        # Wait a moment
        time.sleep(1)

        # Try to reconnect
        if self.connect():
            # Verify the reconnection worked
            is_healthy, health_msg = self.check_connection_health()
            if is_healthy:
                logging.info("[OK] MT5 reconnection successful")
                return True
            else:
                logging.warning(f"Reconnection completed but health check failed: {health_msg}")
        else:
            logging.warning(f"Reconnection attempt {attempt + 1} failed")

    except Exception as e:
        logging.error(f"Error during reconnection attempt {attempt + 1}: {e}")

logging.error(f"[ERROR] All {max_retries} reconnection attempts failed")
return False

def check_connection_health(self):
    """
    Comprehensive health check for MT5 connection and trading readiness
    Returns (is_healthy, error_message)
    """
    try:
        logging.info("■ Performing MT5 connection health check...")

        # 1. Check MT5 initialization
        if not mt5.initialize():
            return False, "MT5 not initialized"

        # 2. Check terminal connection
        terminal_info = mt5.terminal_info()
        if not terminal_info:
            return False, "Cannot get terminal info"

        if not terminal_info.connected:
            return False, "MT5 terminal not connected"

        if not terminal_info.trade_allowed:
            return False, "Trading not allowed in MT5 terminal"

        # 3. Check account access
        account_info = mt5.account_info()
        if not account_info:
            return False, "Cannot access account info"

        # 4. Check symbol availability (using configured symbol)
        if self.symbol:
            symbol_info = mt5.symbol_info(self.symbol)
            if not symbol_info:

```

```

        return False, f"Symbol {self.symbol} not found in MT5"

    if not symbol_info.visible:
        return False, f"Symbol {self.symbol} not visible in Market Watch"

    # 5. Check tick data availability
    tick = mt5.symbol_info_tick(self.symbol)
    if not tick or tick.bid <= 0 or tick.ask <= 0:
        return False, f"No live tick data for symbol {self.symbol}"

    logging.info("[OK] MT5 connection health check passed")
    return True, "All systems operational"

except Exception as e:
    error_msg = f"Health check failed: {e}"
    logging.error(f"[ERROR] {error_msg}")
    return False, error_msg

def verify_connection(self):
    """
    Verify MT5 connection is stable and ready for trading
    """
    try:
        # Check terminal info
        terminal_info = mt5.terminal_info()
        if not terminal_info:
            logging.error("MT5 terminal_info() returned None")
            return False

        if not terminal_info.connected:
            logging.error("MT5 terminal not connected")
            return False

        if not terminal_info.trade_allowed:
            logging.error("MT5 trading not allowed")
            return False

        # Check account info
        account_info = mt5.account_info()
        if not account_info:
            logging.error("MT5 account_info() returned None")
            return False

        logging.info(
            "[OK] MT5 connection verified - terminal connected, trading allowed, account accessible"
        )
        return True

    except Exception as e:
        logging.error(f"Error verifying MT5 connection: {e}")
        return False

def _verify_symbol_tick_data(self, symbol, max_retries=3):
    """
    Verify symbol has live tick data available
    """
    for attempt in range(max_retries):
        try:
            tick = mt5.symbol_info_tick(symbol)
            if tick and tick.bid > 0 and tick.ask > 0:
                logging.info(f"[OK] Tick data verified for {symbol}: bid={tick.bid}, ask={tick.ask}")

```

```

        return True

    if attempt < max_retries - 1:
        logging.warning(
            f"Tick data not available for {symbol} (attempt {attempt + 1}/{max_retries}), ret
            time.sleep(0.1) # Brief delay before retry

    except Exception as e:
        logging.error(f"Error getting tick data for {symbol}: {e}")
        if attempt < max_retries - 1:
            time.sleep(0.1)

    logging.error(f"[ERROR] No tick data available for {symbol} after {max_retries} attempts")
    return False

def is_autotrading_enabled(self):
    """
    Check if AutoTrading is enabled in MT5 using the existing connection
    Returns True if autotrading is enabled, False otherwise
    """
    try:
        # Don't initialize a new connection - use the existing one
        if not self.connected:
            print("[ERROR] MT5 not connected - cannot check AutoTrading status")
            return False

        # Use the already connected MT5 instance to check terminal info
        term_info = mt5.terminal_info()
        if not term_info:
            print("[ERROR] Could not get terminal info from existing MT5 connection")
            return False

        # Check AutoTrading status using the connected instance
        print("■ Checking AutoTrading status on existing MT5 connection...")

        # Basic status checks
        connected = getattr(term_info, 'connected', False)
        trade_allowed = getattr(term_info, 'trade_allowed', False)
        tradeapi_disabled = getattr(term_info, 'tradeapi_disabled', True)
        dlls_allowed = getattr(term_info, 'dlls_allowed', False)

        # Log the current status
        print(f"[CHECK] Connected: {connected}")
        print(f"[CHECK] Trade Allowed: {trade_allowed}")
        print(f"[CHECK] Trade API Disabled: {tradeapi_disabled}")
        print(f"[CHECK] DLLs Allowed: {dlls_allowed}")

        # AutoTrading is enabled if:
        # 1. MT5 is connected
        # 2. Trade is allowed (main AutoTrading setting)
        # 3. Trade API is not disabled
        autotrading_enabled = connected and trade_allowed and not tradeapi_disabled

        print(f"[RESULT] AutoTrading enabled: {autotrading_enabled}")
        return autotrading_enabled

    except Exception as e:
        print(f"[ERROR] AutoTrading status check failed: {e}")
        logging.error(f"Error checking auto trading status: {e}")
        return False

```

```

def ensure_symbol(self, symbol):
    """Ensure symbol is available for trading, with enhanced caching and fast paths"""
    try:
        # Validate input symbol
        if not symbol or symbol.strip() == "":
            logging.error(f"Invalid symbol provided: '{symbol}'")
            raise Exception(f"Invalid symbol provided: '{symbol}'")

        symbol = symbol.strip()

        # SPEED OPTIMIZATION: Check symbol cache first
        cache_key = f"{self.server}_{symbol}"
        current_time = time.time()

        if (cache_key in self._symbol_cache and
            cache_key in self._symbol_cache_timestamp and
            current_time - self._symbol_cache_timestamp[cache_key] < self._cache_ttl):

            cached_symbol = self._symbol_cache[cache_key]
            logging.info(f"[FAST] SPEED: Using cached symbol {symbol} → {cached_symbol}")
            return cached_symbol

        # First check if MT5 is connected
        if not mt5.terminal_info():
            logging.error("MT5 terminal not connected")
            # CRITICAL: Don't return symbol if MT5 is not connected - this causes trading failures
            logging.error("Cannot validate symbol - MT5 terminal not connected")
            return None

        # PRIORITY: Since users provide correct symbol names, try their symbol first
        logging.info(f"[TARGET] USER SYMBOL: Trying user-provided symbol '{symbol}' first")
        symbol_info = mt5.symbol_info(symbol)
        if symbol_info:
            tick = mt5.symbol_info_tick(symbol)
            if tick and (tick.bid > 0 or tick.ask > 0):
                logging.info(f"[OK] USER SYMBOL WORKS: '{symbol}' has active tick data")
                # Cache the successful result
                self._symbol_cache[cache_key] = symbol
                self._symbol_cache_timestamp[cache_key] = current_time
                return symbol
            else:
                # Symbol exists but no tick data - CRITICAL: Don't proceed with trading
                logging.error(f"[ERROR] Symbol {symbol} exists but no tick data available - cannot t")
                return None

        # Try to select the symbol if it wasn't found
        logging.info(f"[SIGNAL] SELECTING SYMBOL: Attempting to activate '{symbol}'")
        select_result = mt5.symbol_select(symbol, True)

        # Check again after selection
        symbol_info = mt5.symbol_info(symbol)
        if symbol_info:
            tick = mt5.symbol_info_tick(symbol)
            if tick and (tick.bid > 0 or tick.ask > 0):
                logging.info(f"[OK] SYMBOL ACTIVATED: '{symbol}' now has active tick data")
                self._symbol_cache[cache_key] = symbol
                self._symbol_cache_timestamp[cache_key] = current_time
                return symbol
            else:
                # Symbol selected but no tick - still return for trading attempt

```

```

        logging.info(f"[WARNING] SYMBOL SELECTED: '{symbol}' activated but no tick data yet")
        self._symbol_cache[cache_key] = symbol
        self._symbol_cache_timestamp[cache_key] = current_time
        return symbol

# SPEED OPTIMIZATION: For known symbols, try direct approach first
if symbol.upper() in ['USTECH', 'USTEC', 'XAUUSD', 'NAS100', 'NASDAQ']:
    # Try the symbol directly first (fastest path)
    symbol_info = mt5.symbol_info(symbol)
    if symbol_info:
        tick = mt5.symbol_info_tick(symbol)
        if tick and (tick.bid > 0 or tick.ask > 0):
            logging.info(f"[FAST] SPEED: Direct symbol access successful for {symbol}")
            # Cache the successful result
            self._symbol_cache[cache_key] = symbol
            self._symbol_cache_timestamp[cache_key] = current_time
            return symbol

# Get symbol variations to try (only if direct access failed)
symbol_variations = self._get_symbol_variations(symbol)
logging.info(f"[SEARCH] VARIATIONS: Trying {len(symbol_variations)} variations for '{symbol}'")

# SPEED OPTIMIZATION: Try most likely variations first
priority_variations = []
other_variations = []

for variation in symbol_variations:
    # Prioritize exact matches and simple variations
    if (variation == symbol or
        variation == symbol.upper() or
        variation in ['USTECH', 'USTEC', 'XAUUSD']):
        priority_variations.append(variation)
    else:
        other_variations.append(variation)

# Try priority variations first, then others
all_variations = priority_variations + other_variations[:20] # Limit to first 20 of others

for variation in all_variations:
    try:
        # First check if this variation has symbol info
        var_info = mt5.symbol_info(variation)
        if not var_info:
            logging.debug(f"Symbol variation {variation} not found")
            continue

        # Check if it already has tick data (means it's working)
        tick = mt5.symbol_info_tick(variation)
        if tick and (tick.bid > 0 or tick.ask > 0):
            logging.info(
                f"[FAST] SPEED: Symbol variation {variation} already has active tick data")
            self._symbol_cache[cache_key] = variation
            self._symbol_cache_timestamp[cache_key] = current_time
            return variation

        # Try to select the symbol (but don't fail if this returns False)
        # Some brokers return False even when symbol is already available
        select_result = mt5.symbol_select(variation, True)
        logging.debug(f"Symbol select result for {variation}: {select_result}")

        # After selection attempt, check again for tick data

```

```

        tick_after = mt5.symbol_info_tick(variation)
        if tick_after and (tick_after.bid > 0 or tick_after.ask > 0):
            logging.info(f"[OK] VARIATION SUCCESS: '{variation}' now has active tick data")
            self._symbol_cache[cache_key] = variation
            self._symbol_cache_timestamp[cache_key] = current_time
            return variation

        # If still no tick data, but symbol info exists, it might still work for some operations
        if var_info and var_info.visible:
            logging.info(
                f"[WARNING] VARIATION VISIBLE: '{variation}' is visible but no current tick data"
            )
            self._symbol_cache[cache_key] = variation
            self._symbol_cache_timestamp[cache_key] = current_time
            return variation

    except Exception as e:
        logging.debug(f"Error trying symbol variation {variation}: {e}")
        continue

    # CRITICAL: Don't return symbol as fallback if no valid symbol was found
    logging.error(f"[FAILED] No valid symbol found for {symbol} - cannot proceed with trading")
    return None

except Exception as e:
    logging.error(f"Error in ensure_symbol for '{symbol}': {e}")

    # CRITICAL: Always return user's symbol to prevent None errors
    # Users are expected to provide correct symbol names for their broker
    logging.warning(f"■ EMERGENCY FALLBACK: Returning user symbol '{symbol}' despite errors")
    return symbol

def _get_symbol_variations(self, symbol):
    """Get possible symbol variations for different MT5 brokers"""
    # Start with the original symbol
    variations = [symbol]

    # Add common variations based on symbol type
    symbol_upper = symbol.upper()

    # NASDAQ variations - comprehensive list based on actual MT5 charts
    if symbol_upper in ['USTEC', 'NASDAQ', 'NQ', 'NAS', 'USTECH100', 'USTECH', 'NAS100', 'NDX',
                       'NASDAQ100', 'TECH100', 'US100', 'SPX500']:
        variations.extend([
            # Primary NASDAQ symbols
            'USTEC', 'USTECH100', 'USTECH', 'NAS100', 'NASDAQ', 'NQ', 'NDX', 'NASDAQ100',
            'US100', 'TECH100', 'USTEC100', 'NASTECH', 'NASDAQTECH',

            # Suffixed variations (.m, m, -Z, etc.)
            'USTEC.m', 'USTECH100.m', 'USTECH.m', 'NAS100.m', 'NASDAQ.m', 'NQ.m', 'NDX.m',
            'USTECm', 'USTECH100m', 'USTECHm', 'NAS100m', 'NASDAQm', 'NQm', 'NDXm',
            'USTEC-Z', 'USTECH100-Z', 'USTECH-Z', 'NAS100-Z', 'NASDAQ-Z', 'NQ-Z', 'NDX-Z',

            # Broker-specific variations
            'USTECfx', 'USTECH100fx', 'USTECHfx', 'NAS100fx', 'NASDAQfx',
            'USTEC_c', 'USTECH_c', 'NAS100_c', 'NASDAQ_c',
            'USTEC.c', 'USTECH.c', 'NAS100.c', 'NASDAQ.c',

            # Alternative naming patterns
            'US_TECH', 'US-TECH', 'USTECH.', 'USTEC.', 'NAS100.',
            'USTECH100.', 'NASDAQ100.', 'TECH-100', 'TECH_100',

```

```

        # Contract-specific variations (futures style)
        'USTECH2024', 'USTEC2024', 'NAS2024', 'USTECH24', 'USTEC24', 'NAS24',
        'USTECHM24', 'USTECM24', 'NASM24', 'USTECHZ24', 'USTECZ24', 'NASZ24',

        # Additional broker variations
        'USTEC100', 'NASTECH100', 'USNASDAQ', 'NASDAQ_100', 'NASDAQ-100',
        'USTEC_100', 'USTEC-100', 'USTECH_100', 'USTECH-100',

        # Dot variations
        'USTEC.', 'USTECH.', 'NASDAQ.', 'NAS100.', 'NQ.',

        # Undercore variations
        'USTEC_', 'USTECH_', 'NASDAQ_', 'NAS100_', 'NQ_'
    ])

# Gold variations - comprehensive list for different brokers
elif symbol_upper in ['XAUUSD', 'GOLD', 'GLD', 'XAU']:
    variations.extend([
        'XAUUSD', 'GOLD', 'XAU', 'GOLDUSD', 'XAUUSD.',
        'XAUUSD.m', 'GOLD.m', 'XAUUSDm', 'GOLDm',
        'XAUUSD-Z', 'GOLD-Z', 'XAU/USD', 'GOLD/USD',
        'XAUUSDfx', 'GOLDfx', 'XAUUSD_MT5'
    ])

# Oil variations
elif symbol_upper in ['USOIL', 'OIL', 'CRUDE']:
    variations.extend(['USOIL', 'CRUDE', 'OIL', 'WTI', 'BRENT'])

# Forex pairs - try both with and without suffixes
elif len(symbol) == 6 and symbol_upper.endswith('USD'):
    base_pair = symbol_upper[:6]
    variations.extend([base_pair, base_pair + '.', base_pair + 'm', base_pair + 'c'])

# Remove duplicates while preserving order
seen = set()
result = []
for variant in variations:
    if variant not in seen:
        seen.add(variant)
        result.append(variant)

return result

def _log_available_symbols(self, failed_symbol):
    """Log some available symbols for debugging"""
    try:
        # Get all available symbols
        symbols = mt5.symbols_get()
        if symbols:
            # Log total count
            logging.info(f"Failed symbol: {failed_symbol}")
            logging.info(f"Total available symbols: {len(symbols)}")

            # Log first 20 symbols
            symbol_names = [s.name for s in symbols[:20]]
            logging.info(f"Available symbols (first 20): {symbol_names}")

            # Look for symbols containing parts of the failed symbol
            failed_upper = failed_symbol.upper()
            similar = []

```

```

        for s in symbols:
            symbol_name = s.name.upper()
            # Check for partial matches
            if (failed_upper[:3] in symbol_name or
                symbol_name[:3] in failed_upper or
                'XAU' in symbol_name or
                'GOLD' in symbol_name or
                'USTEC' in symbol_name or
                'NAS' in symbol_name):
                similar.append(s.name)

        if similar:
            logging.info(f"Similar/related symbols found: {similar[:10]}") # Show max 10

        # Check specifically for gold and nasdaq symbols
        gold_symbols = [s.name for s in symbols if any(x in s.name.upper() for x in ['XAU', 'GOLD'])]
        nasdaq_symbols = [s.name for s in symbols if any(x in s.name.upper() for x in ['USTEC', 'NASDAQ',
        'NDX'])]

        if gold_symbols:
            logging.info(f"Gold-related symbols: {gold_symbols}")
        if nasdaq_symbols:
            logging.info(f"NASDAQ-related symbols: {nasdaq_symbols}")

    else:
        logging.warning("No symbols available - check MT5 connection")
except Exception as e:
    logging.warning(f"Could not retrieve available symbols: {e}")

def get_connected_symbol(self):
    """Get the symbol that was detected during connection"""
    return self.connected_symbol

def get_safe_symbol(self, fallback_symbol=None):
    """Get a safe symbol to use - prefers fallback (configured), then connected symbol, then configured symbol
    # Priority 1: Use the explicitly requested/configured symbol if provided and available
    if fallback_symbol:
        # Try to select the requested symbol to ensure it's available
        if mt5.symbol_select(fallback_symbol, True):
            return fallback_symbol
        else:
            logging.warning(
                f"Requested symbol {fallback_symbol} not available, falling back to connected symbol")

    # Priority 2: Use connected symbol if no specific symbol requested or if requested symbol unavailable
    if self.connected_symbol:
        return self.connected_symbol
    elif self.symbol:
        return self.symbol
    else:
        # Ultimate fallback
        return "EURUSD"

def get_supported_filling_modes(self, symbol):
    info = mt5.symbol_info(symbol)
    if info is None:
        return [mt5.ORDER_FILLING_IOC]
    fillings = getattr(info, "trade_fillings", None)
    if not fillings or len(fillings) == 0:
        return [mt5.ORDER_FILLING_IOC, mt5.ORDER_FILLING_FOK, mt5.ORDER_FILLING_RETURN]

```



```

        return list(fillings)

def _calculate_sl_tp_price(self, symbol, order_type, price, sl_points, tp_points):
    """Calculate SL and TP prices from points - NASDAQ automation always uses 1.0 point value"""
    sl_price = None
    tp_price = None

    # Get symbol info for logging purposes
    symbol_info = mt5.symbol_info(symbol)
    if not symbol_info:
        logging.error(f"Could not get symbol info for {symbol}")
        return sl_price, tp_price

    # Get the minimum tick size for reference
    point = symbol_info.point

    # NASDAQ automation: ALWAYS use 1.0 point value regardless of symbol name or tick size
    point_value = 1.0
    logging.info(
        f"Symbol {symbol} - tick size: {point}, NASDAQ automation point value: {point_value}, price: {price}"
    )

    if sl_points and float(sl_points) > 0:
        sl_points_float = float(sl_points)
        price_difference = sl_points_float * point_value

        if order_type == "buy":
            sl_price = price - price_difference
        else: # sell
            sl_price = price + price_difference

        logging.info(
            f"SL calculation: {sl_points_float} points × {point_value} = {price_difference} price difference"
        )

    if tp_points and float(tp_points) > 0:
        tp_points_float = float(tp_points)
        price_difference = tp_points_float * point_value

        if order_type == "buy":
            tp_price = price + price_difference
        else: # sell
            tp_price = price - price_difference

        logging.info(
            f"TP calculation: {tp_points_float} points × {point_value} = {price_difference} price difference"
        )

    return sl_price, tp_price

def place_order(self, symbol, order_type, volume=None, sl=None, tp=None, comment=None):
    try:
        # PRE-TRADE HEALTH CHECK: Ensure MT5 is ready for trading
        is_healthy, health_error = self.check_connection_health()
        if not is_healthy:
            logging.error(f"[ERROR] Pre-trade health check failed: {health_error}")
            # Attempt reconnection
            logging.info("■ Attempting automatic reconnection...")
            if self.attempt_reconnection():
                # Re-check health after reconnection
                is_healthy, health_error = self.check_connection_health()
                if not is_healthy:
                    raise Exception(f"MT5 reconnection failed: {health_error}")
    
```

```

        else:
            raise Exception(f"MT5 health check failed and reconnection unsuccessful: {health_error}")

# Validate input parameters
if not symbol or symbol.strip() == "":
    logging.error(f"Invalid symbol provided to place_order: '{symbol}'")
    raise Exception(f"Invalid symbol provided: '{symbol}'")

symbol = symbol.strip()

# Ensure symbol is available and get the correct symbol name
corrected_symbol = self.ensure_symbol(symbol)

# CRITICAL: Validate that ensure_symbol didn't return None
if not corrected_symbol:
    logging.error(
        f"ensure_symbol returned None for '{symbol}' - MT5 connection or symbol validation failed")
    raise Exception(
        f"Symbol validation failed for '{symbol}' - check MT5 connection and symbol availability")

# Log order details with corrected symbol
logging.info(
    f"■ PLACING ORDER: {order_type.upper()} {corrected_symbol}, volume={volume}, sl={sl}, tp={tp}")

# Debug symbol info to understand MT5 properties (only log once per symbol)
if not hasattr(self, '_debugged_symbols'):
    self._debugged_symbols = set()
if corrected_symbol not in self._debugged_symbols:
    self.debug_symbol_info(corrected_symbol)
    self._debugged_symbols.add(corrected_symbol)

tick = mt5.symbol_info_tick(corrected_symbol)
print(f"[SEARCH] TICK RETRIEVAL DEBUG for {corrected_symbol}:")
print(f"    Raw tick result: {tick}")
if tick:
    print(f"    Tick ask: {tick.ask}, bid: {tick.bid}")
    print(f"    Tick time: {tick.time}")
else:
    print(f"    [ERROR] Tick is None - investigating...")

# Check MT5 initialization
if not mt5.initialize():
    print(f"    [ERROR] MT5 not initialized - attempting to initialize...")
    if mt5.initialize():
        print(f"    [OK] MT5 initialization successful")
        # Try getting tick again after initialization
        tick = mt5.symbol_info_tick(corrected_symbol)
        print(f"    Retry after init: {tick}")
    else:
        print(f"    [ERROR] MT5 initialization failed")

# Check terminal connection
terminal_info = mt5.terminal_info()
print(f"    Terminal info: {terminal_info}")
if terminal_info:
    print(f"    Terminal connected: {terminal_info.connected}")
    print(f"    Terminal trade allowed: {terminal_info.trade_allowed}")

# Check if symbol exists in symbol_info
symbol_info = mt5.symbol_info(corrected_symbol)

```

```

print(f"    Symbol info: {symbol_info}")
if symbol_info:
    print(f"    Symbol visible: {symbol_info.visible}")
    print(f"    Symbol selected: {symbol_info.select}")

    # Try to select the symbol explicitly
    if not symbol_info.visible:
        print(f"    Attempting to select symbol...")
        select_result = mt5.symbol_select(corrected_symbol, True)
        print(f"    Symbol select result: {select_result}")
        if select_result:
            # Try getting tick again after selecting
            tick = mt5.symbol_info_tick(corrected_symbol)
            print(f"    Tick after select: {tick}")

if tick is None:
    # Enhanced debugging for symbol issues
    logging.error(f"[ERROR] SYMBOL PRICE FETCH FAILED: {corrected_symbol}")

    # Check if symbol is selected in Market Watch
    selected = mt5.symbol_select(corrected_symbol, True)
    logging.error(f"[SEARCH] Symbol select result: {selected}")

    # Check symbol info
    symbol_info = mt5.symbol_info(corrected_symbol)
    if symbol_info:
        logging.error(
            f"[SEARCH] Symbol info exists: visible={symbol_info.visible}, tradeable={symbol_info.tradeable}")
    else:
        logging.error(f"[SEARCH] Symbol info is None - symbol may not exist")

    # Try to get symbols that match pattern
    matching_symbols = mt5.symbols_get(group=f"*{corrected_symbol}*")
    if matching_symbols:
        logging.error(f"[SEARCH] Found {len(matching_symbols)} matching symbols:")
        for sym in matching_symbols[:5]: # Show first 5 matches
            logging.error(f"    - {sym.name}")
    else:
        logging.error(f"[SEARCH] No symbols found matching pattern *{corrected_symbol}*")

    # Enhanced symbol debugging for "Could not get price" issues
    print(f"[SEARCH] SYMBOL DEBUG: No price data for {corrected_symbol}")
    print(f"    Checking symbol selection and market watch...")

    # Check if symbol is in Market Watch
    market_watch_symbols = mt5.symbols_get()
    if market_watch_symbols:
        symbol_names = [s.name for s in market_watch_symbols]
        if corrected_symbol not in symbol_names:
            print(f"[ERROR] SYMBOL ERROR: {corrected_symbol} not in Market Watch")
            print(f"    Available symbols: {symbol_names[:10]}") # Show first 10
        else:
            print(f"[OK] SYMBOL FOUND: {corrected_symbol} is in Market Watch")

    # Try exact symbol matching
    exact_match = mt5.symbol_info(corrected_symbol)
    if not exact_match:
        print(f"[ERROR] EXACT MATCH FAILED: {corrected_symbol} not found")
        # Try pattern matching for similar symbols
        if market_watch_symbols:

```

```

        similar_symbols = [s.name for s in market_watch_symbols
                           if corrected_symbol.lower() in s.name.lower() or s.name.lower()
                              in corrected_symbol.lower()]
        if similar_symbols:
            print(f"[SEARCH] SIMILAR SYMBOLS: {similar_symbols}")
        else:
            print(f"[OK] SYMBOL EXISTS: {corrected_symbol} found in MT5")

        raise Exception(
            f"Could not get price for {corrected_symbol} - MT5 connection issue or symbol not rec

# SUCCESS: We have a valid tick, now extract the price
price = tick.ask if order_type == "buy" else tick.bid
print(f"[OK] PRICE EXTRACTED: {order_type} price for {corrected_symbol} = {price}")
print(
    f"    Full tick data: ask={tick.ask}, bid={tick.bid}, spread={tick.ask - tick.bid if tick

# Validate price is reasonable
if price <= 0:
    print(f"[ERROR] INVALID PRICE: {price} <= 0")
    raise Exception(f"Invalid price {price} for {corrected_symbol}")

type_mt5 = mt5.ORDER_TYPE_BUY if order_type == "buy" else mt5.ORDER_TYPE_SELL
supported_fillings = self.get_supported_filling_modes(corrected_symbol)
last_error = None

if sl is None:
    sl = self.sl_points
if tp is None:
    tp = self.tp_points
if volume is None:
    volume = self.default_volume

# Apply PlexyTrade lot size adjustment - ONLY for USTECH (Nasdaq)
# XAUUSD (Gold) pip values are consistent across brokers, so no division needed
if self.is_plexy_server and volume > 0:
    # Only divide lot size for USTECH/Nasdaq symbols
    if any(x in corrected_symbol.upper() for x in ['USTECH', 'USTEC', 'NAS', 'NASDAQ', 'NDX',
        'NQ']):
        original_volume = volume
        volume = volume / 20.0
        logging.info(
            f"PlexyTrade adjustment for {corrected_symbol}: {original_volume} -> {volume} lot
    else:
        logging.info(
            f"PlexyTrade: No lot size adjustment for {corrected_symbol} (only divide USTECH,

# Ensure SL and TP are always set (never skip) - use defaults if 0
if sl is None or float(sl) <= 0:
    sl = self.sl_points if self.sl_points > 0 else 10 # Default 10 points if not set
    logging.info(f"Using default/minimum SL: {sl} points")
if tp is None or float(tp) <= 0:
    tp = self.tp_points if self.tp_points > 0 else 20 # Default 20 points if not set
    logging.info(f"Using default/minimum TP: {tp} points")

# Convert to float and ensure they're positive
sl = float(sl)
tp = float(tp)
volume = float(volume)

sl_price, tp_price = self._calculate_sl_tp_price(corrected_symbol, order_type, price, sl, tp)

```

```

logging.info(f"Order parameters: price={price}, sl_price={sl_price}, tp_price={tp_price}")

for filling_mode in supported_fillings:
    request = {
        "action": mt5.TRADE_ACTION_DEAL,
        "symbol": corrected_symbol,
        "volume": volume,
        "type": type_mt5,
        "price": price,
        "deviation": 20,
        "type_filling": filling_mode,
        "type_time": mt5.ORDER_TIME_GTC,
    }

    # Add comment if provided
    if comment:
        request["comment"] = comment

    # ALWAYS add SL and TP - never skip them
    if sl_price is not None:
        request["sl"] = sl_price
        logging.info(f"Setting SL price: {sl_price}")
    if tp_price is not None:
        request["tp"] = tp_price
        logging.info(f"Setting TP price: {tp_price}")

    logging.info(f"Sending order request: {request}")
    result = mt5.order_send(request)

    if result is not None:
        logging.info(f"Order result: retcode={result.retcode}, comment={result.comment}")
        if result.retcode == mt5.TRADE_RETCODE_DONE:
            logging.info(
                f"Order successful: {order_type} {corrected_symbol} {volume} at {price}, tick"
            )
            return result.order
        else:
            # Enhanced error handling for common MT5 trading issues
            error_msg = result.comment if result.comment else "Unknown error"

            # Check for automated trading permission issues
            if result.retcode == 10027: # TRADE_RETCODE_CLIENT_DISABLES_AT
                print(f"[ERROR] MT5 TRADING ERROR: Automated trading is disabled in MT5 terminal")
                print(f"[TOOL] SOLUTION: Enable automated trading in MT5:")
                print(f"    1. Go to Tools → Options → Expert Advisors")
                print(f"    2. Check 'Allow algorithmic trading'")
                print(f"    3. Check 'Allow DLL imports'")
                print(f"    4. Click OK and restart application")
                raise Exception(
                    "Automated trading disabled in MT5 - Enable in Tools → Options → Expert Advisors"
                )

            elif result.retcode == 10026: # TRADE_RETCODE_TRADE_DISABLED
                print(f"[ERROR] MT5 TRADING ERROR: Trading is disabled")
                print(f"[TOOL] SOLUTION: Check MT5 terminal settings and broker permissions")
                raise Exception("Trading disabled - Check MT5 settings and broker permissions")

            elif result.retcode == 10013: # TRADE_RETCODE_INVALID_REQUEST
                print(f"[ERROR] MT5 TRADING ERROR: Invalid trading request")
                print(f"[TOOL] SOLUTION: Check symbol, volume, and market hours")
                raise Exception(f"Invalid trading request: {error_msg}")

            elif result.retcode == 10004: # TRADE_RETCODE_REQUOTE

```

```

        print(f"[WARNING] MT5 TRADING: Price requote - retrying...")

    elif result.retcode == 10018: # TRADE_RETCODE_MARKET_CLOSED
        print(f"[ERROR] MT5 TRADING ERROR: Market is closed")
        print(f"[TOOL] SOLUTION: Wait for market opening hours")
        raise Exception("Market is closed - Wait for trading hours")

    elif result.retcode == 10019: # TRADE_RETCODE_NO_MONEY
        print(f"[ERROR] MT5 TRADING ERROR: Insufficient funds")
        print(f"[TOOL] SOLUTION: Check account balance and reduce position size")
        raise Exception("Insufficient funds - Check account balance")

    else:
        print(f"[ERROR] MT5 TRADING ERROR: {error_msg} (Code: {result.retcode})")
        print(f"[TOOL] SOLUTION: Check MT5 terminal for detailed error information")

        last_error = f"{error_msg} (Code: {result.retcode})"
    else:
        last_error = "No result returned"
        print(f"[ERROR] MT5 CRITICAL: No response from MT5 terminal")
        print(f"[TOOL] SOLUTION: Check MT5 terminal connection and restart if needed")

    logging.warning(
        f"Order attempt failed: {order_type} {corrected_symbol} {volume} at {price}, filling="

    logging.error(
        f"Order failed: {order_type} {corrected_symbol} {volume} at {price}, last_error={last_error}"
    )
    raise Exception(f"Order failed: {last_error}")

except Exception as e:
    logging.exception(f"Exception in place_order: {e}")
    raise

def buy_market(self, symbol, volume=None, sl=None, tp=None, comment=None):
    return self.place_order(symbol, "buy", volume=volume, sl=sl, tp=tp, comment=comment)

def sell_market(self, symbol, volume=None, sl=None, tp=None, comment=None):
    return self.place_order(symbol, "sell", volume=volume, sl=sl, tp=tp, comment=comment)

def is_connected(self):
    """Check if MT5 connection is still active"""
    try:
        # Try to get account info to test connection
        account_info = mt5.account_info()
        if account_info is None:
            return False
        return True
    except Exception:
        return False

def check_connection_and_disconnect_if_needed(self):
    """Check connection and disconnect if lost"""
    if not self.is_connected():
        logging.warning("MT5 connection lost, disconnecting...")
        self.disconnect()
        return False
    return True

def disconnect(self):
    """Properly disconnect from MT5 with enhanced cleanup and terminal closure"""

```

```

try:
    # First, perform standard MT5 API shutdown
    if mt5.terminal_info() is not None:
        mt5.shutdown()
        logging.info("MT5 API disconnected successfully")
    else:
        logging.info("MT5 API was already disconnected")

    # Force close MT5 terminal processes to ensure complete shutdown
    self._close_mt5_processes()

except Exception as e:
    logging.error(f"Error during MT5 disconnect: {e}")
    # Force shutdown and process termination even if there was an error
    try:
        mt5.shutdown()
    except:
        pass
    # Still attempt to close processes
    self._close_mt5_processes()

def _close_mt5_processes(self):
    """Force close all MT5 terminal processes"""
    try:
        closed_processes = []

        # Look for MT5 processes by name
        for proc in psutil.process_iter(['pid', 'name', 'exe']):
            try:
                proc_name = proc.info['name'].lower()
                proc_exe = proc.info['exe']

                # Check if this is an MT5 process
                if (proc_name in ['terminal64.exe', 'terminal.exe', 'metatrader5.exe'] or
                    (proc_exe and 'metatrader' in proc_exe.lower())):

                    proc.terminate() # Send termination signal
                    closed_processes.append(f"{proc_name} (PID: {proc.info['pid']})")

            except (psutil.NoSuchProcess, psutil.AccessDenied, psutil.ZombieProcess):
                # Process might have already closed or access denied
                continue
            except Exception as e:
                logging.warning(f"Error checking process: {e}")
                continue

        if closed_processes:
            logging.info(f"■ Closed MT5 processes: {'', '.join(closed_processes)}")

            # Wait a moment for graceful termination
            sleep(2)

            # Force kill any remaining MT5 processes
            for proc in psutil.process_iter(['pid', 'name', 'exe']):
                try:
                    proc_name = proc.info['name'].lower()
                    proc_exe = proc.info['exe']

                    if (proc_name in ['terminal64.exe', 'terminal.exe', 'metatrader5.exe'] or
                        (proc_exe and 'metatrader' in proc_exe.lower())):

```

```

        proc.kill() # Force kill if still running
        logging.info(
            f"■ Force killed stubborn MT5 process: {proc_name} (PID: {proc.info['pid']})"

        except (psutil.NoSuchProcess, psutil.AccessDenied, psutil.ZombieProcess):
            continue
        except Exception as e:
            logging.warning(f"Error force killing process: {e}")
            continue
    else:
        logging.info("■ No MT5 processes found to close")

except Exception as e:
    logging.error(f"Error closing MT5 processes: {e}")
    # Fallback: try using taskkill as last resort
    try:
        subprocess.run(['taskkill', '/f', '/im', 'terminal64.exe'],
                        capture_output=True, check=False)
        subprocess.run(['taskkill', '/f', '/im', 'terminal.exe'],
                        capture_output=True, check=False)
        logging.info("■ Used taskkill as fallback to close MT5 processes")
    except Exception as fallback_error:
        logging.error(f"Fallback taskkill also failed: {fallback_error}")

def get_account_info(self):
    info = mt5.account_info()
    if info is None:
        logging.error("No account info available")
        return {"balance": "", "profit": "", "drawdown": "", "open_trades": "", "Symbol": "",
                "Direction": ""}
    trades = mt5.positions_get()
    open_trades = len(trades) if trades else 0
    symbol = ""
    direction = ""
    if trades and open_trades > 0:
        pos = trades[0]
        symbol = getattr(pos, "symbol", "")
        if getattr(pos, "type", None) == mt5.POSITION_TYPE_BUY:
            direction = "Long"
        elif getattr(pos, "type", None) == mt5.POSITION_TYPE_SELL:
            direction = "Short"
        else:
            direction = ""
    return {
        "balance": str(round(info.balance, 2)),
        "profit": str(round(info.profit, 2)),
        "drawdown": "",
        "open_trades": str(open_trades),
        "Symbol": symbol,
        "Direction": direction
    }

def is_trade_open(self, ticket):
    positions = mt5.positions_get(ticket=ticket)
    return positions is not None and len(positions) > 0

def has_open_trade(self, symbol):
    """
    Returns True if there is any open position for the given symbol.
    """

```



```

positions = mt5.positions_get(symbol=symbol)
return positions is not None and len(positions) > 0

def get_trades_today_count(self, comment_filter=None):
    """Get the number of trades opened today

    Args:
        comment_filter: Optional string to filter trades by comment (e.g., "Combine1_")

    Returns:
        int: Number of trades opened today
    """
    try:
        from datetime import datetime, date
        import MetaTrader5 as mt5

        today = date.today()
        today_start = datetime.combine(today, datetime.min.time())
        today_end = datetime.combine(today, datetime.max.time())

        # Convert to timestamp
        from_date = int(today_start.timestamp())
        to_date = int(today_end.timestamp())

        # Get history deals (completed trades) for today
        deals = mt5.history_deals_get(from_date, to_date)

        # Also check current open positions opened today
        positions = mt5.positions_get()

        count = 0

        # Count completed deals (history)
        if deals:
            for deal in deals:
                # Only count entry deals (not exit deals)
                if deal.entry == mt5.DEAL_ENTRY_IN:
                    if comment_filter is None or (deal.comment and comment_filter in deal.comment):
                        count += 1

        # Count open positions opened today
        if positions:
            for pos in positions:
                pos_time = datetime.fromtimestamp(pos.time)
                if pos_time.date() == today:
                    if comment_filter is None or (pos.comment and comment_filter in pos.comment):
                        count += 1

        logging.info(f"Trades opened today: {count} (filter: {comment_filter})")
        return count

    except Exception as e:
        logging.error(f"Error getting today's trade count: {e}")
        return 0

def get_orphaned_mt5_positions_by_account(self, account_number):
    """Get MT5 positions for a specific Tradovate account

    Args:
        account_number: Tradovate account number to filter by

```

```

Returns:
    list: List of orphaned position tickets
    """
try:
    # Get all open MT5 positions
    positions = mt5.positions_get()
    if positions is None:
        return []

    orphaned_tickets = []
    for pos in positions:
        if pos.comment:
            # Check if position is from this account (comment is just the account number)
            if pos.comment.strip() == account_number:
                orphaned_tickets.append(pos.ticket)

    return orphaned_tickets
except Exception as e:
    logging.error(f"Error finding orphaned positions by account: {e}")
    return []

def close_orphaned_positions_by_account(self, account_number):
    """Close MT5 positions for a specific Tradovate account

    Args:
        account_number: Tradovate account number to filter by

    Returns:
        int: Number of positions closed
        """
    try:
        orphaned_tickets = self.get_orphaned_mt5_positions_by_account(account_number)
        closed_count = 0

        for ticket in orphaned_tickets:
            try:
                if self.close_trade(ticket):
                    closed_count += 1
                    logging.info(f"Closed orphaned MT5 position for account {account_number}: {ticket}")
            except Exception as e:
                logging.error(f"Error closing orphaned position {ticket}: {e}")

        return closed_count
    except Exception as e:
        logging.error(f"Error closing orphaned positions by account: {e}")
        return 0

def get_orphaned_mt5_positions(self, combine_comment_prefix):
    """Get MT5 positions that don't have corresponding Tradovate trades

    Args:
        combine_comment_prefix: Comment prefix to identify trades from this combine (e.g., "Combine1_")

    Returns:
        list: List of orphaned position tickets
        """
    try:
        import MetaTrader5 as mt5

        positions = mt5.positions_get()

```

```

        orphaned_tickets = []

        if positions:
            for pos in positions:
                # Check if this position belongs to our combine
                if pos.comment and combine_comment_prefix in pos.comment:
                    orphaned_tickets.append(pos.ticket)
                    logging.info(f"Found orphaned MT5 position: {pos.ticket} ({pos.comment})")

            return orphaned_tickets

    except Exception as e:
        logging.error(f"Error finding orphaned positions: {e}")
        return []

def close_orphaned_positions(self, combine_comment_prefix):
    """Close MT5 positions that don't have corresponding Tradovate trades

    Args:
        combine_comment_prefix: Comment prefix to identify trades from this combine (e.g., "Combine1

    Returns:
        int: Number of positions closed
    """
    try:
        orphaned_tickets = self.get_orphaned_mt5_positions(combine_comment_prefix)
        closed_count = 0

        for ticket in orphaned_tickets:
            try:
                if self.close_trade(ticket):
                    closed_count += 1
                    logging.info(f"Closed orphaned MT5 position: {ticket}")
            except Exception as e:
                logging.error(f"Error closing orphaned position {ticket}: {e}")

        return closed_count

    except Exception as e:
        logging.error(f"Error closing orphaned positions: {e}")
        return 0

def close_trade(self, ticket, retries=3, delay=2):
    for attempt in range(retries):
        positions = mt5.positions_get(ticket=ticket)
        if not positions:
            logging.info(f"Trade {ticket} already closed.")
            return True
        pos = positions[0]
        symbol = pos.symbol
        volume = pos.volume
        order_type = pos.type
        price = mt5.symbol_info_tick(
            symbol).bid if order_type == mt5.POSITION_TYPE_BUY else mt5.symbol_info_tick(symbol).ask
        close_type = mt5.ORDER_TYPE_SELL if order_type == mt5.POSITION_TYPE_BUY else mt5.ORDER_TYPE_BUY
        request = {
            "action": mt5.TRADE_ACTION_DEAL,
            "symbol": symbol,

```

```

        "volume": volume,
        "type": close_type,
        "position": ticket,
        "price": price,
        "deviation": 20,
        "type_filling": mt5.ORDER_FILLING_IOC,
        "type_time": mt5.ORDER_TIME_GTC,
    }
    result = mt5.order_send(request)
    if result is not None and result.retcode == mt5.TRADE_RETCODE_DONE:
        if not self.is_trade_open(ticket):
            logging.info(f"Trade {ticket} closed successfully.")
            return True
        sleep(delay)
    logging.error(f"Failed to close trade {ticket} after {retries} attempts.")
    return not self.is_trade_open(ticket)

def force_close_trade(self, ticket):
    positions = mt5.positions_get(ticket=ticket)
    if not positions:
        logging.info(f"Trade {ticket} already closed (force close).")
        return True
    pos = positions[0]
    symbol = pos.symbol
    volume = pos.volume
    order_type = pos.type
    price = mt5.symbol_info_tick(
        symbol).bid if order_type == mt5.POSITION_TYPE_BUY else mt5.symbol_info_tick(symbol).ask
    close_type = mt5.ORDER_TYPE_SELL if order_type == mt5.POSITION_TYPE_BUY else mt5.ORDER_TYPE_BUY
    for filling in [mt5.ORDER_FILLING_IOC, mt5.ORDER_FILLING_FOK, mt5.ORDER_FILLING_RETURN]:
        request = {
            "action": mt5.TRADE_ACTION_DEAL,
            "symbol": symbol,
            "volume": volume,
            "type": close_type,
            "position": ticket,
            "price": price,
            "deviation": 20,
            "type_filling": filling,
            "type_time": mt5.ORDER_TIME_GTC,
        }
        result = mt5.order_send(request)
        if result is not None and result.retcode == mt5.TRADE_RETCODE_DONE:
            if not self.is_trade_open(ticket):
                logging.info(f"Trade {ticket} force closed successfully.")
                return True
    logging.error(f"Failed to force close trade {ticket}.")
    return not self.is_trade_open(ticket)

def get_daily_trade_count(self, comment_filter=None):
    """Get count of trades placed today from both open positions and history

    Args:
        comment_filter: Can be either:
            - Tradovate account number (e.g., "MFFUEVSTP326057008") - preferred method
            - Old combine prefix (e.g., "Combine1_") - for backward compatibility
    """
    try:
        from datetime import datetime, date
        import tempfile

```

```

import os

today = date.today()

# Check if trades were reset for this filter today
temp_dir = tempfile.gettempdir()
reset_file = os.path.join(temp_dir, f"mt5_reset_{comment_filter}_{today.strftime('%Y%m%d')}.txt")
if os.path.exists(reset_file):
    # Return 0 if reset flag exists for today
    return 0

trade_count = 0

# Count from open positions
positions = mt5.positions_get()
if positions:
    for pos in positions:
        # Convert MT5 time to date
        pos_date = datetime.fromtimestamp(pos.time()).date()
        if pos_date == today:
            # If comment filter is provided, check if position comment matches
            if comment_filter:
                # For new format: exact match with account number
                # For old format: substring match with combine prefix
                if pos.comment and (pos.comment.strip()
                                     == comment_filter or comment_filter in str(pos.comment)):
                    trade_count += 1
            else:
                trade_count += 1

# Count from history deals (more reliable for completed trades)
# Get deals from start of today to now
today_start = datetime.combine(today, datetime.min.time())
today_end = datetime.now()

deals = mt5.history_deals_get(today_start, today_end)
if deals:
    # Only count entry deals (not exit deals to avoid double counting)
    for deal in deals:
        if deal.entry == mt5.DEAL_ENTRY_IN: # Entry deal only
            # If comment filter is provided, check if deal comment matches
            if comment_filter:
                # For new format: exact match with account number
                # For old format: substring match with combine prefix
                if deal.comment and (deal.comment.strip()
                                     == comment_filter or comment_filter in str(deal.comment)):
                    trade_count += 1
            else:
                trade_count += 1

logging.info(f"Daily trade count: {trade_count} (filter: {comment_filter})")
return trade_count

except Exception as e:
    logging.error(f"Error counting daily trades: {e}")
    return 0

def get_daily_trade_count_by_account(self, tradovate_account_number):
    """Get count of trades placed today for a specific Tradovate account

    Args:

```

```

        tradovate_account_number: Tradovate account number (e.g., "MFFUEVSTP326057008")

Returns:
    int: Number of trades opened today for this account
    """
    try:
        from datetime import datetime, date

        today = date.today()
        trade_count = 0

        # Count from open positions
        positions = mt5.positions_get()
        if positions:
            for pos in positions:
                # Convert MT5 time to date
                pos_date = datetime.fromtimestamp(pos.time()).date()
                if pos_date == today:
                    # Check if position comment matches the account number (comment is just the account number)
                    if pos.comment and pos.comment.strip() == tradovate_account_number:
                        trade_count += 1

        # Count from history deals
        today_start = datetime.combine(today, datetime.min.time())
        today_end = datetime.now()

        deals = mt5.history_deals_get(today_start, today_end)
        if deals:
            for deal in deals:
                if deal.entry == mt5.DEAL_ENTRY_IN: # Entry deal only
                    # Check if deal comment matches the account number (comment is just the account number)
                    if deal.comment and deal.comment.strip() == tradovate_account_number:
                        trade_count += 1

        logging.info(f"Daily trade count for account {tradovate_account_number}: {trade_count}")
        return trade_count

    except Exception as e:
        logging.error(f"Error counting daily trades for account {tradovate_account_number}: {e}")
        return 0

def reset_daily_trade_count(self, comment_filter=None):
    """Reset daily trade count for a specific filter by storing reset timestamp

    Args:
        comment_filter: Can be either:
            - Tradovate account number (e.g., "MFFUEVSTP326057008") - preferred method
            - Old combine prefix (e.g., "Combine1_") - for backward compatibility
    """
    try:
        from datetime import datetime
        import tempfile
        import os

        # Create a simple flag file to mark that trades were reset for this filter today
        temp_dir = tempfile.gettempdir()
        reset_file = os.path.join(temp_dir,
                                   f"mt5_reset_{comment_filter}_{datetime.now().strftime('%Y%m%d')}.flag")

        # Create the flag file
    
```

```

        with open(reset_file, 'w') as f:
            f.write(str(datetime.now().timestamp()))

        logging.info(f"Daily trade count reset for filter: {comment_filter}")
        return True

    except Exception as e:
        logging.error(f"Error resetting daily trade count: {e}")
        return False

def reset_daily_trade_count_by_account(self, tradovate_account_number):
    """Reset daily trade count for a specific Tradovate account

    Args:
        tradovate_account_number: Tradovate account number (e.g., "MFFUEVSTP326057008")
    """
    try:
        from datetime import datetime
        import tempfile
        import os

        # Create a simple flag file to mark that trades were reset for this account today
        temp_dir = tempfile.gettempdir()
        reset_file = os.path.join(temp_dir,
                                   f"mt5_reset_account_{tradovate_account_number}_{datetime.now().strftime('%Y%m%d')}.flag")

        # Create the flag file
        with open(reset_file, 'w') as f:
            f.write(str(datetime.now().timestamp()))

        logging.info(f"Daily trade count reset for Tradovate account: {tradovate_account_number}")
        return True

    except Exception as e:
        logging.error(f"Error resetting daily trade count for account {tradovate_account_number}: {e}")
        return False

def get_historical_profits_by_account(self, account_number):
    """Get total historical profits for trades from a specific Tradovate account"""
    try:
        from datetime import datetime, timedelta

        # Get deals from the last 30 days to get a good history
        end_time = datetime.now()
        start_time = end_time - timedelta(days=30)

        total_profit = 0.0

        # Get historical deals
        deals = mt5.history_deals_get(start_time, end_time)
        if deals:
            for deal in deals:
                # Check if deal comment matches the account number (comment is just the account number)
                if deal.comment and deal.comment.strip() == account_number:
                    # Only count exit deals for profit calculation (avoid double counting)
                    if deal.entry == mt5.DEAL_ENTRY_OUT:
                        total_profit += deal.profit

        logging.info(f"Historical profits for account {account_number}: ${total_profit:.2f}")
        return total_profit

```

```

except Exception as e:
    logging.error(f"Error getting historical profits by account: {e}")
    return 0.0

def get_historical_profits(self, comment_filter=None):
    """Get total historical profits for trades with specific comment filter"""
    try:
        from datetime import datetime, timedelta

        # Get deals from the last 30 days to get a good history
        end_time = datetime.now()
        start_time = end_time - timedelta(days=30)

        total_profit = 0.0

        # Get historical deals
        deals = mt5.history_deals_get(start_time, end_time)
        if deals:
            for deal in deals:
                # Check if deal comment matches the filter (for specific combine)
                if comment_filter and comment_filter not in str(deal.comment):
                    continue

                # Only count exit deals for profit calculation (avoid double counting)
                if deal.entry == mt5.DEAL_ENTRY_OUT:
                    total_profit += deal.profit

        logging.info(f"Historical profits for {comment_filter}: ${total_profit:.2f}")
        return total_profit

    except Exception as e:
        logging.error(f"Error calculating historical profits: {e}")
        return 0.0

def close_orphaned_trades(self, expected_tradovate_trades):
    """Close MT5 trades that don't have corresponding Tradovate trades

    Args:
        expected_tradovate_trades: List of Tradovate trade symbols/IDs that should have MT5 counterparts

    Returns:
        List of closed MT5 trade tickets
    """
    try:
        closed_trades = []
        positions = mt5.positions_get()

        if not positions:
            return closed_trades

        for pos in positions:
            should_close = False

            # If no Tradovate trades expected, close all MT5 trades
            if not expected_tradovate_trades:
                should_close = True
                reason = "no corresponding Tradovate trades"
            else:
                # Check if this MT5 trade has a corresponding Tradovate trade
                # This is a simplified check - in practice you might need more sophisticated matching
    
```



```

        mt5_symbol = pos.symbol
        has_counterpart = False

        for tradovate_trade in expected_tradovate_trades:
            # Simple symbol matching - you may want to enhance this logic
            if str(tradovate_trade).upper() in mt5_symbol.upper():
                has_counterpart = True
                break

        if not has_counterpart:
            should_close = True
            reason = f"no matching Tradovate trade found"

        if should_close:
            logging.info(f"Closing orphaned MT5 trade {pos.ticket}: {pos.symbol} ({reason})")
            if self.close_trade(pos.ticket):
                closed_trades.append(pos.ticket)
                logging.info(f"[OK] Closed orphaned trade {pos.ticket}")
            else:
                logging.error(f"[ERROR] Failed to close orphaned trade {pos.ticket}")

        if closed_trades:
            logging.info(f"Closed {len(closed_trades)} orphaned MT5 trades: {closed_trades}")

        return closed_trades

    except Exception as e:
        logging.error(f"Error closing orphaned trades: {e}")
        return []

def extract_tradovate_account_from_comment(self, comment):
    """Extract Tradovate account number from MT5 comment

    Args:
        comment: MT5 order comment (e.g., "MFFUEVSTP326057008")

    Returns:
        str: Tradovate account number or "Unknown" if not found
    """
    try:
        if comment and comment.strip():
            # For new format: comment is just the account number
            account_number = comment.strip()
            return account_number if account_number else "Unknown"
        return "Unknown"
    except Exception as e:
        logging.error(f"Error extracting account from comment '{comment}': {e}")
        return "Unknown"

def get_trades_by_tradovate_account(self, tradovate_account_number=None):
    """Get all MT5 trades associated with a specific Tradovate account

    Args:
        tradovate_account_number: Tradovate account number to filter by

    Returns:
        dict: Dictionary with 'open_positions' and 'history_deals' lists
    """
    try:
        from datetime import datetime, date

```

```

result = {
    'open_positions': [],
    'history_deals': []
}

# Get open positions
positions = mt5.positions_get()
if positions:
    for pos in positions:
        if pos.comment:
            account_from_comment = self.extract_tradovate_account_from_comment(pos.comment)
            if tradovate_account_number is None or account_from_comment == tradovate_account_number:
                result['open_positions'].append({
                    'ticket': pos.ticket,
                    'symbol': pos.symbol,
                    'volume': pos.volume,
                    'type': 'BUY' if pos.type == mt5.POSITION_TYPE_BUY else 'SELL',
                    'open_time': datetime.fromtimestamp(pos.time),
                    'comment': pos.comment,
                    'tradovate_account': account_from_comment
                })

# Get today's history deals
today = date.today()
today_start = datetime.combine(today, datetime.min.time())
today_end = datetime.combine(today, datetime.max.time())
from_date = int(today_start.timestamp())
to_date = int(today_end.timestamp())

deals = mt5.history_deals_get(from_date, to_date)
if deals:
    for deal in deals:
        if deal.comment:
            account_from_comment = self.extract_tradovate_account_from_comment(deal.comment)
            if tradovate_account_number is None or account_from_comment == tradovate_account_number:
                result['history_deals'].append({
                    'ticket': deal.ticket,
                    'symbol': deal.symbol,
                    'volume': deal.volume,
                    'type': 'BUY' if deal.type == mt5.DEAL_TYPE_BUY else 'SELL',
                    'time': datetime.fromtimestamp(deal.time),
                    'comment': deal.comment,
                    'tradovate_account': account_from_comment,
                    'entry': deal.entry
                })

return result

except Exception as e:
    logging.error(f"Error getting trades by Tradovate account: {e}")
    return {'open_positions': [], 'history_deals': []}

def get_symbol_info(self, symbol):
    """Get symbol information"""
    try:
        return mt5.symbol_info(symbol)
    except Exception as e:
        logging.error(f"Error getting symbol info for {symbol}: {e}")
        return None

def debug_symbol_info(self, symbol):

```

```

"""Debug method to print all available symbol information"""
try:
    info = mt5.symbol_info(symbol)
    if info:
        logging.info(f"=== Symbol Info for {symbol} ===")
        for attr in dir(info):
            if not attr.startswith('_'):
                try:
                    value = getattr(info, attr)
                    logging.info(f"  {attr}: {value}")
                except:
                    pass
        logging.info("=== End Symbol Info ===")
        return info
    else:
        logging.error(f"No symbol info available for {symbol}")
        return None
except Exception as e:
    logging.error(f"Error debugging symbol info: {e}")
    return None

def get_tick_data(self, symbol):
    """Get current tick data for symbol"""
    try:
        return mt5.symbol_info_tick(symbol)
    except Exception as e:
        logging.error(f"Error getting tick data for {symbol}: {e}")
        return None

def should_close_trades_for_rollover(self, prop_firm_name):
    """
    Check if trades should be closed based on prop firm rollover schedules.

    Closing Times (Eastern Time):
    - Trade Day: 5:00 PM ET
    - Funding Ticks: 5:00 PM ET
    - Tradeify: 4:59 PM ET
    - MFFU: 4:10 PM ET
    - Alpha Futures: 4:20 PM ET

    Args:
        prop_firm_name: Name of the prop firm

    Returns:
        bool: True if trades should be closed now, False otherwise
    """
    import datetime
    import pytz

    try:
        # Get current time in Eastern timezone
        eastern = pytz.timezone('US/Eastern')
        current_time = datetime.datetime.now(eastern)

        # Define closing times for each prop firm (24-hour format)
        closing_schedules = {
            "TradeDay": (17, 0), # 5:00 PM Eastern Time
            "Funding Ticks": (17, 0), # 5:00 PM Eastern Time
            "Tradeify": (16, 59), # 4:59 PM Eastern Time
            "MFFU": (16, 10), # 4:10 PM Eastern Standard Time
        }
    
```

```

        "Alpha Futures": (16, 20), # 4:20 PM Eastern Time
    }

    # Get closing time for this prop firm
    closing_time_tuple = closing_schedules.get(prop_firm_name)

    if closing_time_tuple is None:
        # This prop firm doesn't require trade closing
        return False

    # Convert closing time to datetime object for proper comparison
    closing_hour, closing_minute = closing_time_tuple
    closing_time = current_time.replace(hour=closing_hour, minute=closing_minute, second=0,
                                         microsecond=0)

    # Get current date string for tracking
    current_date = current_time.strftime("%Y-%m-%d")

    # Safety check: Have we already executed rollover for this prop firm today?
    if self.rollover_executed_today.get(prop_firm_name) == current_date:
        return False # Already executed today, skip to prevent duplicates

    # Check if current time is at or past closing time (with safety buffer)
    # This ensures we don't miss rollover due to system delays
    if current_time >= closing_time:
        # Also check if we're on a weekday (Monday = 0, Sunday = 6)
        if current_time.weekday() < 5: # Monday through Friday
            current_time_str = current_time.strftime("%H:%M")
            closing_time_str = closing_time.strftime("%H:%M")
            logging.info(
                f"■ Market rollover time reached for {prop_firm_name} at {current_time_str} ET (
            return True

    return False

except Exception as e:
    logging.error(f"Error checking rollover schedule for {prop_firm_name}: {e}")
    return False

def close_trades_for_rollover(self, prop_firm_name, account_comment_prefix=None):
    """
    Close all MT5 trades for market rollover based on prop firm schedule.

    Args:
        prop_firm_name: Name of the prop firm
        account_comment_prefix: Account number to filter trades (e.g., "MFFU123456")

    Returns:
        list: List of closed trade tickets
    """
    if not self.should_close_trades_for_rollover(prop_firm_name):
        return []

    try:
        # Get all open positions
        positions = mt5.positions_get()
        if positions is None:
            logging.warning("No positions found or error getting positions")
            return []

        closed_tickets = []

```

```

for position in positions:
    # Filter by account comment prefix if provided
    if account_comment_prefix and not position.comment.startswith(account_comment_prefix):
        continue

    # Close the position
    if self.close_trade(position.ticket):
        closed_tickets.append(position.ticket)
        logging.info(
            f"■ ROLLOVER: Closed trade {position.ticket} for {prop_firm_name} market rollover"
        )
    else:
        logging.error(f"[ERROR] Failed to close trade {position.ticket} for rollover")

if closed_tickets:
    logging.info(
        f"■ ROLLOVER COMPLETE: Closed {len(closed_tickets)} trades for {prop_firm_name} at m

    # Mark rollover as executed today to prevent duplicate execution
    import datetime
    import pytz
    eastern = pytz.timezone('US/Eastern')
    current_date = datetime.datetime.now(eastern).strftime("%Y-%m-%d")
    self.rollover_executed_today[prop_firm_name] = current_date

return closed_tickets

except Exception as e:
    logging.error(f"Error closing trades for rollover ({prop_firm_name}): {e}")
    return []

```

Method: MT5Automator.__init__

```
def __init__(self, login, password, server, symbol=None, terminal_path=None)
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.
- **__init__** — The constructor. Runs after Python has allocated the instance; assigns starting attribute values to self.

Step by step (top-level body)

1. **try** block with 1 **except** clause.
2. Assigns `self.password = str(password) if password else "`.
3. Assigns `self.server = str(server) if server else "`.
4. Assigns `self.symbol = symbol`.
5. Assigns `self.terminal_path = terminal_path`.

6. Assigns `self.sl_points = float(os.getenv('MT5_SL_POINTS') or os.getenv('MT5_STOPLO....`
7. Assigns `self.tp_points = float(os.getenv('MT5_TP_POINTS') or os.getenv('MT5_TAKEPR....`
8. Assigns `self.default_volume = float(os.getenv('MT5_VOLUME', '1'))`.
9. Assigns `self.rollover_executed_today = {}`.
10. Assigns `self.connected_symbol = None`.
11. Assigns `self.is_plexy_server = 'plexy' in server.lower()` if server else False.
12. **if** `self.is_plexy_server`: branches conditionally.
13. Assigns `self.connected = False`.
14. Assigns `self.last_error = None`.

Code (copy-paste safe)

```
def __init__(self, login, password, server, symbol=None, terminal_path=None):
    # Safely convert login to integer
    try:
        self.login = int(str(login).strip()) if login else 0
    except (ValueError, TypeError):
        logging.error(f'Invalid login format: {login}')
        self.login = 0

    self.password = str(password) if password else ""
    self.server = str(server) if server else ""
    self.symbol = symbol
    self.terminal_path = terminal_path
    self.sl_points = float(os.getenv('MT5_SL_POINTS') or os.getenv('MT5_STOPLOSS_POINTS', '0'))
    self.tp_points = float(os.getenv('MT5_TP_POINTS') or os.getenv('MT5_TAKEPROFIT_POINTS', '0'))
    self.default_volume = float(os.getenv('MT5_VOLUME', '1'))

    # Rollover safety tracking - prevents multiple executions per day
    self.rollover_executed_today = {} # {prop_firm: date_string}

    # Store the actually connected symbol (will be set after successful connection)
    self.connected_symbol = None

    # Check if this is a PlexyTrade server (case-insensitive substring match)
    self.is_plexy_server = "plexy" in server.lower() if server else False
    if self.is_plexy_server:
        logging.info(f"PlexyTrade server detected: {server} - Lot sizes will be divided by 20")

    # Ensure all instance variables are properly initialized
    self.connected = False
    self.last_error = None
```

Method: MT5Automator._get_cached_terminal_path

```
def _get_cached_terminal_path(self)
    Get previously successful terminal path for faster connection
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Imports tempfile (lazy import inside the function).
2. Assigns `cache_file = os.path.join(tempfile.gettempdir(), 'mt5_terminal_cache.t....`
3. **try** block with 1 **except** clause.
4. **return** None.

Code (copy-paste safe)

```
def _get_cached_terminal_path(self):
    """Get previously successful terminal path for faster connection"""
    import tempfile
    cache_file = os.path.join(tempfile.gettempdir(), "mt5_terminal_cache.txt")
    try:
        if os.path.exists(cache_file):
            with open(cache_file, 'r') as f:
                cached_path = f.read().strip()
                if os.path.exists(cached_path):
                    return cached_path
    except Exception as e:
        logging.debug(f"Cache read failed: {e}")
    return None
```

Method: MT5Automator._cache_successful_path

```
def _cache_successful_path(self, path)

    Cache successful terminal path for future use
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Imports tempfile (lazy import inside the function).
2. Assigns `cache_file = os.path.join(tempfile.gettempdir(), 'mt5_terminal_cache.t....`
3. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def _cache_successful_path(self, path):
```

```

"""Cache successful terminal path for future use"""
import tempfile
cache_file = os.path.join(tempfile.gettempdir(), "mt5_terminal_cache.txt")
try:
    with open(cache_file, 'w') as f:
        f.write(path)
    logging.info(f"[OK] Cached successful MT5 path: {path}")
except Exception as e:
    logging.debug(f"Cache write failed: {e}")

```

Method: MT5Automator.connect

```
def connect(self)
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns success = False.
2. Assigns cached_path = self._get_cached_terminal_path().
3. **if** cached_path: branches conditionally.
4. **if** not success: branches conditionally.
5. **if** not success: branches conditionally.
6. **if** not success: branches conditionally.
7. **if** not success: branches conditionally.
8. Assigns authorized = mt5.login(self.login, self.password, self.server).
9. **if** not authorized: branches conditionally.
10. **try** block with 1 **except** clause.
11. Assigns self.connected = True.
12. **return** True.

Code (copy-paste safe)

```

def connect(self):
    # SPEED OPTIMIZATION: Try cached terminal path first
    success = False
    cached_path = self._get_cached_terminal_path()
    if cached_path:
        if mt5.initialize(path=cached_path):
            logging.info(f'[FAST] MT5 initialized with cached path: {cached_path}')
            success = True
        else:

```



```

        logging.info(f'Cached path failed, trying alternatives: {cached_path}')

# Try to initialize MT5 with the best available path
if not success:
    # CRITICAL FIX: If user specifies a terminal path, ONLY use that terminal
    if self.terminal_path and self.terminal_path.strip():
        terminal_exe = os.path.join(self.terminal_path, "terminal64.exe")
        if os.path.exists(terminal_exe):
            if mt5.initialize(path=terminal_exe):
                logging.info(f'MT5 initialized successfully with user-specified path: {terminal_exe}')
                self._cache_successful_path(terminal_exe) # Cache success
                success = True
            else:
                logging.error(f'MT5 initialize failed for user-specified path: {terminal_exe}')
                # Don't try other terminals when user specified one - respect their choice
                error_code, error_msg = mt5.last_error()
                self.last_error = f'MT5 initialize failed for selected terminal: {error_msg} (Code: {error_code})'
                logging.error(self.last_error)
                return False
        else:
            logging.error(f'User-specified terminal executable not found: {terminal_exe}')
            self.last_error = f'Terminal executable not found: {terminal_exe}'
            return False

# If specific paths failed, try other available installations
if not success:
    terminals = get_installed_mt5_terminals()
    for terminal in terminals:
        terminal_exe = os.path.join(terminal["path"], "terminal64.exe")
        if os.path.exists(terminal_exe):
            if mt5.initialize(path=terminal_exe):
                logging.info(f'MT5 initialized successfully with detected path: {terminal_exe}')
                self._cache_successful_path(terminal_exe) # Cache success
                success = True
                break
            else:
                logging.warning(f'MT5 initialize failed for detected path: {terminal_exe}')

# Last resort: try default initialization
if not success:
    if mt5.initialize():
        logging.info('MT5 initialized successfully with default path')
        success = True
    else:
        logging.error('MT5 initialize failed with default path')

if not success:
    error_code, error_msg = mt5.last_error()
    self.last_error = f'MT5 initialize failed: {error_msg} (Code: {error_code})'
    logging.error(self.last_error)
    return False

authorized = mt5.login(self.login, self.password, self.server)
if not authorized:
    error_msg = f'MT5 login failed for login={self.login}, server={self.server}'
    logging.error(error_msg)
    self.last_error = error_msg
    return False

# SPEED OPTIMIZATION: Fast symbol detection after successful connection

```

```

try:
    # Quick symbol detection - try user symbol first
    if self.symbol and mt5.symbol_select(self.symbol, True):
        self.connected_symbol = self.symbol
        logging.info(f"[FAST] Fast symbol detection: {self.connected_symbol}")
    else:
        # Quick fallback to first available symbol
        symbols = mt5.symbols_get()
        if symbols and len(symbols) > 0:
            self.connected_symbol = symbols[0].name
            if mt5.symbol_select(self.connected_symbol, True):
                logging.info(f"[FAST] Fast fallback symbol: {self.connected_symbol}")
            else:
                # Last resort: use EURUSD as default
                self.connected_symbol = "EURUSD"
                logging.info(f"[FAST] Default symbol: {self.connected_symbol}")
        else:
            self.connected_symbol = "EURUSD" # Safe default

except Exception as e:
    logging.warning(f"Fast symbol detection failed: {e}")
    self.connected_symbol = self.symbol if self.symbol else "EURUSD"

self.connected = True
return True

```

Method: MT5Automator.monitor_connection

```
def monitor_connection(self)
```

Monitor MT5 connection status and attempt recovery if needed Call this periodically (e.g., every 30 seconds) during application operation

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def monitor_connection(self):
    """
    Monitor MT5 connection status and attempt recovery if needed
    Call this periodically (e.g., every 30 seconds) during application operation
    """
    try:
        # Quick connection check
        terminal_info = mt5.terminal_info()
        if not terminal_info or not terminal_info.connected:
            logging.warning("■ MT5 connection monitor detected disconnection")
            return self.attempt_reconnection()
    
```

```

        # Check if trading is still allowed
        if not terminal_info.trade_allowed:
            logging.warning("■ MT5 connection monitor detected trading disabled")
            return False

        # Check account access
        account_info = mt5.account_info()
        if not account_info:
            logging.warning("■ MT5 connection monitor detected account access issues")
            return self.attempt_reconnection()

        return True

    except Exception as e:
        logging.error(f"Error in connection monitor: {e}")
        return False

```

Method: MT5Automator.ensure_session_integrity

```
def ensure_session_integrity(self)
```

Ensure MT5 session is properly initialized and maintained Call this at application startup and periodically during operation

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def ensure_session_integrity(self):
    """
    Ensure MT5 session is properly initialized and maintained
    Call this at application startup and periodically during operation
    """
    try:
        logging.info("[SETUP] Ensuring MT5 session integrity...")

        # Check if MT5 is initialized
        if not mt5.initialize():
            logging.warning("MT5 not initialized, attempting to initialize...")
            if not mt5.initialize():
                logging.error("[ERROR] Failed to initialize MT5")
                return False

        # Check if we're logged in
        if not mt5.terminal_info():
            logging.warning("MT5 terminal info not available, attempting connection...")
            if not self.connect():
                logging.error("[ERROR] Failed to connect to MT5")
                return False

```

```

# Perform comprehensive health check
is_healthy, health_msg = self.check_connection_health()
if not is_healthy:
    logging.warning(f"MT5 health check failed: {health_msg}, attempting recovery...")
    if not self.attempt_reconnection():
        logging.error("[ERROR] MT5 recovery failed")
        return False

# Ensure symbol is properly selected
if self.symbol:
    symbol_info = mt5.symbol_info(self.symbol)
    if symbol_info and not symbol_info.visible:
        logging.info(f"Ensuring symbol {self.symbol} is selected...")
        mt5.symbol_select(self.symbol, True)

logging.info("[OK] MT5 session integrity confirmed")
return True

except Exception as e:
    logging.error(f"Error ensuring MT5 session: {e}")
    return False

```

Method: MT5Automator.attempt_reconnection

```
def attempt_reconnection(self, max_retries=3)
```

Attempt to reconnect to MT5 if connection is lost

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **for** attempt in range(max_retries): iterates.
2. Calls logging.error(...) for its side effect.
3. **return** False.

Code (copy-paste safe)

```

def attempt_reconnection(self, max_retries=3):
    """
    Attempt to reconnect to MT5 if connection is lost
    """
    for attempt in range(max_retries):
        try:
            logging.info(f"■ Attempting MT5 reconnection (attempt {attempt + 1}/{max_retries})...")

            # Shutdown current connection
            mt5.shutdown()

            # Wait a moment
            time.sleep(1)

```

```

        # Try to reconnect
        if self.connect():
            # Verify the reconnection worked
            is_healthy, health_msg = self.check_connection_health()
            if is_healthy:
                logging.info("[OK] MT5 reconnection successful")
                return True
            else:
                logging.warning(f"Reconnection completed but health check failed: {health_msg}")
        else:
            logging.warning(f"Reconnection attempt {attempt + 1} failed")

    except Exception as e:
        logging.error(f"Error during reconnection attempt {attempt + 1}: {e}")

    logging.error(f"[ERROR] All {max_retries} reconnection attempts failed")
    return False

```

Method: MT5Automator.check_connection_health

```
def check_connection_health(self)
```

Comprehensive health check for MT5 connection and trading readiness Returns (is_healthy, error_message)

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def check_connection_health(self):
    """
    Comprehensive health check for MT5 connection and trading readiness
    Returns (is_healthy, error_message)
    """
    try:
        logging.info("■ Performing MT5 connection health check...")

        # 1. Check MT5 initialization
        if not mt5.initialize():
            return False, "MT5 not initialized"

        # 2. Check terminal connection
        terminal_info = mt5.terminal_info()
        if not terminal_info:
            return False, "Cannot get terminal info"

        if not terminal_info.connected:
            return False, "MT5 terminal not connected"

```

```

        if not terminal_info.trade_allowed:
            return False, "Trading not allowed in MT5 terminal"

        # 3. Check account access
        account_info = mt5.account_info()
        if not account_info:
            return False, "Cannot access account info"

        # 4. Check symbol availability (using configured symbol)
        if self.symbol:
            symbol_info = mt5.symbol_info(self.symbol)
            if not symbol_info:
                return False, f"Symbol {self.symbol} not found in MT5"

            if not symbol_info.visible:
                return False, f"Symbol {self.symbol} not visible in Market Watch"

        # 5. Check tick data availability
        tick = mt5.symbol_info_tick(self.symbol)
        if not tick or tick.bid <= 0 or tick.ask <= 0:
            return False, f"No live tick data for symbol {self.symbol}"

        logging.info("[OK] MT5 connection health check passed")
        return True, "All systems operational"

    except Exception as e:
        error_msg = f"Health check failed: {e}"
        logging.error(f"[ERROR] {error_msg}")
        return False, error_msg

```

Method: MT5Automator.verify_connection

```
def verify_connection(self)

    Verify MT5 connection is stable and ready for trading
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def verify_connection(self):
    """
    Verify MT5 connection is stable and ready for trading
    """
    try:
        # Check terminal info
        terminal_info = mt5.terminal_info()
        if not terminal_info:
            logging.error("MT5 terminal_info() returned None")

```

```

        return False

    if not terminal_info.connected:
        logging.error("MT5 terminal not connected")
        return False

    if not terminal_info.trade_allowed:
        logging.error("MT5 trading not allowed")
        return False

    # Check account info
    account_info = mt5.account_info()
    if not account_info:
        logging.error("MT5 account_info() returned None")
        return False

    logging.info(
        "[OK] MT5 connection verified - terminal connected, trading allowed, account accessible")
    return True

except Exception as e:
    logging.error(f"Error verifying MT5 connection: {e}")
    return False

```

Method: MT5Automator._verify_symbol_tick_data

```
def _verify_symbol_tick_data(self, symbol, max_retries=3)
```

Verify symbol has live tick data available

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **for** attempt in range(max_retries): iterates.
2. Calls logging.error(...) for its side effect.
3. **return** False.

Code (copy-paste safe)

```

def _verify_symbol_tick_data(self, symbol, max_retries=3):
    """
    Verify symbol has live tick data available
    """
    for attempt in range(max_retries):
        try:
            tick = mt5.symbol_info_tick(symbol)
            if tick and tick.bid > 0 and tick.ask > 0:
                logging.info(f"[OK] Tick data verified for {symbol}: bid={tick.bid}, ask={tick.ask}")
                return True

            if attempt < max_retries - 1:

```

```

        logging.warning(
            f"Tick data not available for {symbol} (attempt {attempt + 1}/{max_retries}), ret
        time.sleep(0.1) # Brief delay before retry

    except Exception as e:
        logging.error(f"Error getting tick data for {symbol}: {e}")
        if attempt < max_retries - 1:
            time.sleep(0.1)

    logging.error(f"[ERROR] No tick data available for {symbol} after {max_retries} attempts")
    return False

```

Method: MT5Automator.is_autotrading_enabled

```
def is_autotrading_enabled(self)
```

Check if AutoTrading is enabled in MT5 using the existing connection Returns True if autotrading is enabled, False otherwise

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def is_autotrading_enabled(self):
    """
    Check if AutoTrading is enabled in MT5 using the existing connection
    Returns True if autotrading is enabled, False otherwise
    """
    try:
        # Don't initialize a new connection - use the existing one
        if not self.connected:
            print("[ERROR] MT5 not connected - cannot check AutoTrading status")
            return False

        # Use the already connected MT5 instance to check terminal info
        term_info = mt5.terminal_info()
        if not term_info:
            print("[ERROR] Could not get terminal info from existing MT5 connection")
            return False

        # Check AutoTrading status using the connected instance
        print("■ Checking AutoTrading status on existing MT5 connection...")

        # Basic status checks
        connected = getattr(term_info, 'connected', False)
        trade_allowed = getattr(term_info, 'trade_allowed', False)
        tradeapi_disabled = getattr(term_info, 'tradeapi_disabled', True)
        dlls_allowed = getattr(term_info, 'dlls_allowed', False)

```



```

# Log the current status
print(f"[CHECK] Connected: {connected}")
print(f"[CHECK] Trade Allowed: {trade_allowed}")
print(f"[CHECK] Trade API Disabled: {tradeapi_disabled}")
print(f"[CHECK] DLLs Allowed: {dlls_allowed}")

# AutoTrading is enabled if:
# 1. MT5 is connected
# 2. Trade is allowed (main AutoTrading setting)
# 3. Trade API is not disabled
autotrading_enabled = connected and trade_allowed and not tradeapi_disabled

print(f"[RESULT] AutoTrading enabled: {autotrading_enabled}")
return autotrading_enabled

except Exception as e:
    print(f"[ERROR] AutoTrading status check failed: {e}")
    logging.error(f"Error checking auto trading status: {e}")
    return False

```

Method: MT5Automator.ensure_symbol

```
def ensure_symbol(self, symbol)
```

Ensure symbol is available for trading, with enhanced caching and fast paths

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def ensure_symbol(self, symbol):
    """Ensure symbol is available for trading, with enhanced caching and fast paths"""
    try:
        # Validate input symbol
        if not symbol or symbol.strip() == "":
            logging.error(f"Invalid symbol provided: '{symbol}'")
            raise Exception(f"Invalid symbol provided: '{symbol}'")

        symbol = symbol.strip()

        # SPEED OPTIMIZATION: Check symbol cache first
        cache_key = f"{self.server}_{symbol}"
        current_time = time.time()

        if (cache_key in self._symbol_cache and
            cache_key in self._symbol_cache_timestamp and

```

```

        current_time - self._symbol_cache_timestamp[cache_key] < self._cache_ttl):

        cached_symbol = self._symbol_cache[cache_key]
        logging.info(f"[FAST] SPEED: Using cached symbol {symbol} → {cached_symbol}")
        return cached_symbol

# First check if MT5 is connected
if not mt5.terminal_info():
    logging.error("MT5 terminal not connected")
    # CRITICAL: Don't return symbol if MT5 is not connected - this causes trading failures
    logging.error("Cannot validate symbol - MT5 terminal not connected")
    return None

# PRIORITY: Since users provide correct symbol names, try their symbol first
logging.info(f"[TARGET] USER SYMBOL: Trying user-provided symbol '{symbol}' first")
symbol_info = mt5.symbol_info(symbol)
if symbol_info:
    tick = mt5.symbol_info_tick(symbol)
    if tick and (tick.bid > 0 or tick.ask > 0):
        logging.info(f"[OK] USER SYMBOL WORKS: '{symbol}' has active tick data")
        # Cache the successful result
        self._symbol_cache[cache_key] = symbol
        self._symbol_cache_timestamp[cache_key] = current_time
        return symbol
    else:
        # Symbol exists but no tick data - CRITICAL: Don't proceed with trading
        logging.error(f"[ERROR] Symbol {symbol} exists but no tick data available - cannot tr
        return None

# Try to select the symbol if it wasn't found
logging.info(f"[SIGNAL] SELECTING SYMBOL: Attempting to activate '{symbol}'")
select_result = mt5.symbol_select(symbol, True)

# Check again after selection
symbol_info = mt5.symbol_info(symbol)
if symbol_info:
    tick = mt5.symbol_info_tick(symbol)
    if tick and (tick.bid > 0 or tick.ask > 0):
        logging.info(f"[OK] SYMBOL ACTIVATED: '{symbol}' now has active tick data")
        self._symbol_cache[cache_key] = symbol
        self._symbol_cache_timestamp[cache_key] = current_time
        return symbol
    else:
        # Symbol selected but no tick - still return for trading attempt
        logging.info(f"[WARNING] SYMBOL SELECTED: '{symbol}' activated but no tick data yet")
        self._symbol_cache[cache_key] = symbol
        self._symbol_cache_timestamp[cache_key] = current_time
        return symbol

# SPEED OPTIMIZATION: For known symbols, try direct approach first
if symbol.upper() in ['USTECH', 'USTEC', 'XAUUSD', 'NAS100', 'NASDAQ']:
    # Try the symbol directly first (fastest path)
    symbol_info = mt5.symbol_info(symbol)
    if symbol_info:
        tick = mt5.symbol_info_tick(symbol)
        if tick and (tick.bid > 0 or tick.ask > 0):
            logging.info(f"[FAST] SPEED: Direct symbol access successful for {symbol}")
            # Cache the successful result
            self._symbol_cache[cache_key] = symbol
            self._symbol_cache_timestamp[cache_key] = current_time

```

```

        return symbol

    # Get symbol variations to try (only if direct access failed)
    symbol_variations = self._get_symbol_variations(symbol)
    logging.info(f"[SEARCH] VARIATIONS: Trying {len(symbol_variations)} variations for '{symbol}'")

    # SPEED OPTIMIZATION: Try most likely variations first
    priority_variations = []
    other_variations = []

    for variation in symbol_variations:
        # Prioritize exact matches and simple variations
        if (variation == symbol or
            variation == symbol.upper() or
            variation in ['USTECH', 'USTEC', 'XAUUSD']):
            priority_variations.append(variation)
        else:
            other_variations.append(variation)

    # Try priority variations first, then others
    all_variations = priority_variations + other_variations[:20] # Limit to first 20 of others

    for variation in all_variations:
        try:
            # First check if this variation has symbol info
            var_info = mt5.symbol_info(variation)
            if not var_info:
                logging.debug(f"Symbol variation {variation} not found")
                continue

            # Check if it already has tick data (means it's working)
            tick = mt5.symbol_info_tick(variation)
            if tick and (tick.bid > 0 or tick.ask > 0):
                logging.info(
                    f"[FAST] SPEED: Symbol variation {variation} already has active tick data")
                self._symbol_cache[cache_key] = variation
                self._symbol_cache_timestamp[cache_key] = current_time
                return variation

            # Try to select the symbol (but don't fail if this returns False)
            # Some brokers return False even when symbol is already available
            select_result = mt5.symbol_select(variation, True)
            logging.debug(f"Symbol select result for {variation}: {select_result}")

            # After selection attempt, check again for tick data
            tick_after = mt5.symbol_info_tick(variation)
            if tick_after and (tick_after.bid > 0 or tick_after.ask > 0):
                logging.info(f"[OK] VARIATION SUCCESS: '{variation}' now has active tick data")
                self._symbol_cache[cache_key] = variation
                self._symbol_cache_timestamp[cache_key] = current_time
                return variation

            # If still no tick data, but symbol info exists, it might still work for some operations
            if var_info and var_info.visible:
                logging.info(
                    f"[WARNING] VARIATION VISIBLE: '{variation}' is visible but no current tick data")
                self._symbol_cache[cache_key] = variation
                self._symbol_cache_timestamp[cache_key] = current_time
                return variation

        except Exception as e:

```

```

        logging.debug(f"Error trying symbol variation {variation}: {e}")
        continue

    # CRITICAL: Don't return symbol as fallback if no valid symbol was found
    logging.error(f"[FAILED] No valid symbol found for {symbol} - cannot proceed with trading")
    return None

except Exception as e:
    logging.error(f"Error in ensure_symbol for '{symbol}': {e}")

    # CRITICAL: Always return user's symbol to prevent None errors
    # Users are expected to provide correct symbol names for their broker
    logging.warning(f"■ EMERGENCY FALLBACK: Returning user symbol '{symbol}' despite errors")
    return symbol

```

Method: MT5Automator._get_symbol_variations

```
def _get_symbol_variations(self, symbol)

    Get possible symbol variations for different MT5 brokers
```

Step by step (top-level body)

1. Assigns variations = [symbol].
2. Assigns symbol_upper = symbol.upper().
3. if symbol_upper in ['USTEC', 'NASDAQ', 'NQ', 'NAS', 'USTECH100', 'USTE...: branches conditionally (with an else/elif arm).
4. Assigns seen = set().
5. Assigns result = [].
6. for variant in variations: iterates.
7. return result.

Code (copy-paste safe)

```

def _get_symbol_variations(self, symbol):
    """Get possible symbol variations for different MT5 brokers"""
    # Start with the original symbol
    variations = [symbol]

    # Add common variations based on symbol type
    symbol_upper = symbol.upper()

    # NASDAQ variations - comprehensive list based on actual MT5 charts
    if symbol_upper in ['USTEC', 'NASDAQ', 'NQ', 'NAS', 'USTECH100', 'USTECH', 'NAS100', 'NDX',
        'NASDAQ100', 'TECH100', 'US100', 'SPX500']:
        variations.extend([
            # Primary NASDAQ symbols
            'USTEC', 'USTECH100', 'USTECH', 'NAS100', 'NASDAQ', 'NQ', 'NDX', 'NASDAQ100',
            'US100', 'TECH100', 'USTEC100', 'NASTECH', 'NASDAQTECH',

            # Suffixed variations (.m, m, -Z, etc.)
            'USTEC.m', 'USTECH100.m', 'USTECH.m', 'NAS100.m', 'NASDAQ.m', 'NQ.m', 'NDX.m',
            'USTECm', 'USTECH100m', 'USTECHm', 'NAS100m', 'NASDAQm', 'NQm', 'NDXm',
            'USTEC-Z', 'USTECH100-Z', 'USTECH-Z', 'NAS100-Z', 'NASDAQ-Z', 'NQ-Z', 'NDX-Z',

            # Broker-specific variations

```

```

        'USTECfx', 'USTECH100fx', 'USTECHfx', 'NAS100fx', 'NASDAQfx',
        'USTEC_c', 'USTECH_c', 'NAS100_c', 'NASDAQ_c',
        'USTEC.c', 'USTECH.c', 'NAS100.c', 'NASDAQ.c',

        # Alternative naming patterns
        'US_TECH', 'US-TECH', 'USTECH.', 'USTEC.', 'NAS100.',
        'USTECH100.', 'NASDAQ100.', 'TECH-100', 'TECH_100',

        # Contract-specific variations (futures style)
        'USTECH2024', 'USTEC2024', 'NAS2024', 'USTECH24', 'USTEC24', 'NAS24',
        'USTECHM24', 'USTECM24', 'NASM24', 'USTECHZ24', 'USTECZ24', 'NASZ24',

        # Additional broker variations
        'USTEC100', 'NASTECH100', 'USNASDAQ', 'NASDAQ_100', 'NASDAQ-100',
        'USTEC_100', 'USTEC-100', 'USTECH_100', 'USTECH-100',

        # Dot variations
        'USTEC.', 'USTECH.', 'NASDAQ.', 'NAS100.', 'NQ.',

        # Undercore variations
        'USTEC_', 'USTECH_', 'NASDAQ_', 'NAS100_', 'NQ_'
    ])

# Gold variations - comprehensive list for different brokers
elif symbol_upper in ['XAUUSD', 'GOLD', 'GLD', 'XAU']:
    variations.extend([
        'XAUUSD', 'GOLD', 'XAU', 'GOLDUSD', 'XAUUSD.',
        'XAUUSD.m', 'GOLD.m', 'XAUUSDm', 'GOLDm',
        'XAUUSD-Z', 'GOLD-Z', 'XAU/USD', 'GOLD/USD',
        'XAUUSDfx', 'GOLDfx', 'XAUUSD_MT5'
    ])

# Oil variations
elif symbol_upper in ['USOIL', 'OIL', 'CRUDE']:
    variations.extend(['USOIL', 'CRUDE', 'OIL', 'WTI', 'BRENT'])

# Forex pairs - try both with and without suffixes
elif len(symbol) == 6 and symbol_upper.endswith('USD'):
    base_pair = symbol_upper[:6]
    variations.extend([base_pair, base_pair + '.', base_pair + 'm', base_pair + 'c'])

# Remove duplicates while preserving order
seen = set()
result = []
for variant in variations:
    if variant not in seen:
        seen.add(variant)
        result.append(variant)

return result

```

Method: MT5Automator._log_available_symbols

```
def _log_available_symbols(self, failed_symbol)
```

Log some available symbols for debugging

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with `append`, and clearer about intent: this is a transform, not a side-effecting loop.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to `sum`, `any`, `all`, or `max` where the intermediate list would be wasted.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def _log_available_symbols(self, failed_symbol):
    """Log some available symbols for debugging"""
    try:
        # Get all available symbols
        symbols = mt5.symbols_get()
        if symbols:
            # Log total count
            logging.info(f"Failed symbol: {failed_symbol}")
            logging.info(f"Total available symbols: {len(symbols)}")

            # Log first 20 symbols
            symbol_names = [s.name for s in symbols[:20]]
            logging.info(f"Available symbols (first 20): {symbol_names}")

            # Look for symbols containing parts of the failed symbol
            failed_upper = failed_symbol.upper()
            similar = []
            for s in symbols:
                symbol_name = s.name.upper()
                # Check for partial matches
                if (failed_upper[:3] in symbol_name or
                    symbol_name[:3] in failed_upper or
                    'XAU' in symbol_name or
                    'GOLD' in symbol_name or
                    'USTEC' in symbol_name or
                    'NAS' in symbol_name):
                    similar.append(s.name)

            if similar:
                logging.info(f"Similar/related symbols found: {similar[:10]}") # Show max 10

            # Check specifically for gold and nasdaq symbols
            gold_symbols = [s.name for s in symbols if any(x in s.name.upper() for x in ['XAU', 'GOLD', 'NDX'])]
            nasdaq_symbols = [s.name for s in symbols if any(x in s.name.upper() for x in ['USTEC', 'USTEC', 'NDX'])]

            if gold_symbols:
                logging.info(f"Gold-related symbols: {gold_symbols}")
            if nasdaq_symbols:
                logging.info(f"NASDAQ-related symbols: {nasdaq_symbols}")
```

```

        else:
            logging.warning("No symbols available - check MT5 connection")
    except Exception as e:
        logging.warning(f"Could not retrieve available symbols: {e}")

```

Method: MT5Automator.get_connected_symbol

```
def get_connected_symbol(self)
```

Get the symbol that was detected during connection

Step by step (top-level body)

1. **return** self.connected_symbol.

Code (copy-paste safe)

```

def get_connected_symbol(self):
    """Get the symbol that was detected during connection"""
    return self.connected_symbol

```

Method: MT5Automator.get_safe_symbol

```
def get_safe_symbol(self, fallback_symbol=None)
```

Get a safe symbol to use - prefers fallback (configured), then connected symbol, then configured symbol

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **if** fallback_symbol: branches conditionally.
2. **if** self.connected_symbol: branches conditionally (with an **else/elif** arm).

Code (copy-paste safe)

```

def get_safe_symbol(self, fallback_symbol=None):
    """Get a safe symbol to use - prefers fallback (configured), then connected symbol, then configured symbol
    # Priority 1: Use the explicitly requested/configured symbol if provided and available
    if fallback_symbol:
        # Try to select the requested symbol to ensure it's available
        if mt5.symbol_select(fallback_symbol, True):
            return fallback_symbol
        else:
            logging.warning(
                f"Requested symbol {fallback_symbol} not available, falling back to connected symbol"
            )
    # Priority 2: Use connected symbol if no specific symbol requested or if requested symbol unavailable
    if self.connected_symbol:
        return self.connected_symbol
    elif self.symbol:
        return self.symbol
    else:
        # Ultimate fallback
        return "EURUSD"

```

Method: MT5Automator.get_supported_filling_modes

```
def get_supported_filling_modes(self, symbol)
```

Step by step (top-level body)

1. Assigns `info = mt5.symbol_info(symbol)`.
2. **if** `info` is `None`: branches conditionally.
3. Assigns `fillings = getattr(info, 'trade_fillings', None)`.
4. **if** not `fillings` or `len(fillings) == 0`: branches conditionally.
5. **return** `list(fillings)`.

Code (copy-paste safe)

```
def get_supported_filling_modes(self, symbol):
    info = mt5.symbol_info(symbol)
    if info is None:
        return [mt5.ORDER_FILLING_IOC]
    fillings = getattr(info, "trade_fillings", None)
    if not fillings or len(fillings) == 0:
        return [mt5.ORDER_FILLING_IOC, mt5.ORDER_FILLING_FOK, mt5.ORDER_FILLING_RETURN]
    return list(fillings)
```

Method: MT5Automator._calculate_sl_tp_price

```
def _calculate_sl_tp_price(self, symbol, order_type, price, sl_points, tp_points)
```

Calculate SL and TP prices from points - NASDAQ automation always uses 1.0 point value

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `sl_price = None`.
2. Assigns `tp_price = None`.
3. Assigns `symbol_info = mt5.symbol_info(symbol)`.
4. **if** not `symbol_info`: branches conditionally.
5. Assigns `point = symbol_info.point`.
6. Assigns `point_value = 1.0`.
7. Calls `logging.info(...)` for its side effect.
8. **if** `sl_points` and `float(sl_points) > 0`: branches conditionally.
9. **if** `tp_points` and `float(tp_points) > 0`: branches conditionally.
10. **return** `(sl_price, tp_price)`.

Code (copy-paste safe)

```
def _calculate_sl_tp_price(self, symbol, order_type, price, sl_points, tp_points):
    """Calculate SL and TP prices from points - NASDAQ automation always uses 1.0 point value"""
```



```

sl_price = None
tp_price = None

# Get symbol info for logging purposes
symbol_info = mt5.symbol_info(symbol)
if not symbol_info:
    logging.error(f"Could not get symbol info for {symbol}")
    return sl_price, tp_price

# Get the minimum tick size for reference
point = symbol_info.point

# NASDAQ automation: ALWAYS use 1.0 point value regardless of symbol name or tick size
point_value = 1.0
logging.info(
    f"Symbol {symbol} - tick size: {point}, NASDAQ automation point value: {point_value}, price: {price}"

if sl_points and float(sl_points) > 0:
    sl_points_float = float(sl_points)
    price_difference = sl_points_float * point_value

    if order_type == "buy":
        sl_price = price - price_difference
    else: # sell
        sl_price = price + price_difference

    logging.info(
        f"SL calculation: {sl_points_float} points × {point_value} = {price_difference} price difference"

if tp_points and float(tp_points) > 0:
    tp_points_float = float(tp_points)
    price_difference = tp_points_float * point_value

    if order_type == "buy":
        tp_price = price + price_difference
    else: # sell
        tp_price = price - price_difference

    logging.info(
        f"TP calculation: {tp_points_float} points × {point_value} = {price_difference} price difference"

return sl_price, tp_price

```

Method: MT5Automator.place_order

```
def place_order(self, symbol, order_type, volume=None, sl=None, tp=None, comment=None)
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **list comprehension** — Builds a list in a single expression ([f(x) for x in xs]). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. try block with 1 **except** clause.

Code (copy-paste safe)

```
def place_order(self, symbol, order_type, volume=None, sl=None, tp=None, comment=None):
    try:
        # PRE-TRADE HEALTH CHECK: Ensure MT5 is ready for trading
        is_healthy, health_error = self.check_connection_health()
        if not is_healthy:
            logging.error(f"[ERROR] Pre-trade health check failed: {health_error}")
            # Attempt reconnection
            logging.info("■ Attempting automatic reconnection...")
            if self.attempt_reconnection():
                # Re-check health after reconnection
                is_healthy, health_error = self.check_connection_health()
                if not is_healthy:
                    raise Exception(f"MT5 reconnection failed: {health_error}")
            else:
                raise Exception(f"MT5 health check failed and reconnection unsuccessful: {health_error}")

        # Validate input parameters
        if not symbol or symbol.strip() == "":
            logging.error(f"Invalid symbol provided to place_order: '{symbol}'")
            raise Exception(f"Invalid symbol provided: '{symbol}'")

        symbol = symbol.strip()

        # Ensure symbol is available and get the correct symbol name
        corrected_symbol = self.ensure_symbol(symbol)

        # CRITICAL: Validate that ensure_symbol didn't return None
        if not corrected_symbol:
            logging.error(
                f"ensure_symbol returned None for '{symbol}' - MT5 connection or symbol validation failed"
            )
            raise Exception(
                f"Symbol validation failed for '{symbol}' - check MT5 connection and symbol availability"
            )

        # Log order details with corrected symbol
        logging.info(
            f"■ PLACING ORDER: {order_type.upper()} {corrected_symbol}, volume={volume}, sl={sl}, tp={tp}"
        )

        # Debug symbol info to understand MT5 properties (only log once per symbol)
        if not hasattr(self, '_debugged_symbols'):
            self._debugged_symbols = set()
        if corrected_symbol not in self._debugged_symbols:
            self.debug_symbol_info(corrected_symbol)
            self._debugged_symbols.add(corrected_symbol)

        tick = mt5.symbol_info_tick(corrected_symbol)
```

```

print(f"[SEARCH] TICK RETRIEVAL DEBUG for {corrected_symbol}:")
print(f"    Raw tick result: {tick}")
if tick:
    print(f"    Tick ask: {tick.ask}, bid: {tick.bid}")
    print(f"    Tick time: {tick.time}")
else:
    print(f"    [ERROR] Tick is None - investigating...")

    # Check MT5 initialization
    if not mt5.initialize():
        print(f"    [ERROR] MT5 not initialized - attempting to initialize...")
        if mt5.initialize():
            print(f"    [OK] MT5 initialization successful")
            # Try getting tick again after initialization
            tick = mt5.symbol_info_tick(corrected_symbol)
            print(f"    Retry after init: {tick}")
        else:
            print(f"    [ERROR] MT5 initialization failed")

    # Check terminal connection
    terminal_info = mt5.terminal_info()
    print(f"    Terminal info: {terminal_info}")
    if terminal_info:
        print(f"    Terminal connected: {terminal_info.connected}")
        print(f"    Terminal trade allowed: {terminal_info.trade_allowed}")

    # Check if symbol exists in symbol_info
    symbol_info = mt5.symbol_info(corrected_symbol)
    print(f"    Symbol info: {symbol_info}")
    if symbol_info:
        print(f"    Symbol visible: {symbol_info.visible}")
        print(f"    Symbol selected: {symbol_info.select}")

        # Try to select the symbol explicitly
        if not symbol_info.visible:
            print(f"    Attempting to select symbol...")
            select_result = mt5.symbol_select(corrected_symbol, True)
            print(f"    Symbol select result: {select_result}")
            if select_result:
                # Try getting tick again after selecting
                tick = mt5.symbol_info_tick(corrected_symbol)
                print(f"    Tick after select: {tick}")

if tick is None:
    # Enhanced debugging for symbol issues
    logging.error(f"[ERROR] SYMBOL PRICE FETCH FAILED: {corrected_symbol}")

    # Check if symbol is selected in Market Watch
    selected = mt5.symbol_select(corrected_symbol, True)
    logging.error(f"[SEARCH] Symbol select result: {selected}")

    # Check symbol info
    symbol_info = mt5.symbol_info(corrected_symbol)
    if symbol_info:
        logging.error(
            f"[SEARCH] Symbol info exists: visible={symbol_info.visible}, tradeable={symbol_info.tradeable}"
        )
    else:
        logging.error(f"[SEARCH] Symbol info is None - symbol may not exist")

    # Try to get symbols that match pattern

```

```

matching_symbols = mt5.symbols_get(group=f"*{corrected_symbol}*")
if matching_symbols:
    logging.error(f"[SEARCH] Found {len(matching_symbols)} matching symbols:")
    for sym in matching_symbols[:5]: # Show first 5 matches
        logging.error(f"    - {sym.name}")
else:
    logging.error(f"[SEARCH] No symbols found matching pattern *{corrected_symbol}*")

# Enhanced symbol debugging for "Could not get price" issues
print(f"[SEARCH] SYMBOL DEBUG: No price data for {corrected_symbol}")
print(f"    Checking symbol selection and market watch...")

# Check if symbol is in Market Watch
market_watch_symbols = mt5.symbols_get()
if market_watch_symbols:
    symbol_names = [s.name for s in market_watch_symbols]
    if corrected_symbol not in symbol_names:
        print(f"[ERROR] SYMBOL ERROR: {corrected_symbol} not in Market Watch")
        print(f"    Available symbols: {symbol_names[:10]}") # Show first 10
    else:
        print(f"[OK] SYMBOL FOUND: {corrected_symbol} is in Market Watch")

# Try exact symbol matching
exact_match = mt5.symbol_info(corrected_symbol)
if not exact_match:
    print(f"[ERROR] EXACT MATCH FAILED: {corrected_symbol} not found")
    # Try pattern matching for similar symbols
    if market_watch_symbols:
        similar_symbols = [s.name for s in market_watch_symbols
                           if corrected_symbol.lower() in s.name.lower() or s.name.lower()
                              in corrected_symbol.lower()]
        if similar_symbols:
            print(f"[SEARCH] SIMILAR SYMBOLS: {similar_symbols}")
    else:
        print(f"[OK] SYMBOL EXISTS: {corrected_symbol} found in MT5")

    raise Exception(
        f"Could not get price for {corrected_symbol} - MT5 connection issue or symbol not rec

# SUCCESS: We have a valid tick, now extract the price
price = tick.ask if order_type == "buy" else tick.bid
print(f"[OK] PRICE EXTRACTED: {order_type} price for {corrected_symbol} = {price}")
print(
    f"    Full tick data: ask={tick.ask}, bid={tick.bid}, spread={tick.ask - tick.bid if tick

# Validate price is reasonable
if price <= 0:
    print(f"[ERROR] INVALID PRICE: {price} <= 0")
    raise Exception(f"Invalid price {price} for {corrected_symbol}")

type_mt5 = mt5.ORDER_TYPE_BUY if order_type == "buy" else mt5.ORDER_TYPE_SELL
supported_fillings = self.get_supported_filling_modes(corrected_symbol)
last_error = None

if sl is None:
    sl = self.sl_points
if tp is None:
    tp = self.tp_points
if volume is None:
    volume = self.default_volume

```

```

# Apply PlexyTrade lot size adjustment - ONLY for USTECH (Nasdaq)
# XAUUSD (Gold) pip values are consistent across brokers, so no division needed
if self.is_plexy_server and volume > 0:
    # Only divide lot size for USTECH/Nasdaq symbols
    if any(x in corrected_symbol.upper() for x in ['USTECH', 'USTEC', 'NAS', 'NASDAQ', 'NDX',
        'NQ']):
        original_volume = volume
        volume = volume / 20.0
        logging.info(
            f"PlexyTrade adjustment for {corrected_symbol}: {original_volume} -> {volume} lot
    else:
        logging.info(
            f"PlexyTrade: No lot size adjustment for {corrected_symbol} (only divide USTECH,

# Ensure SL and TP are always set (never skip) - use defaults if 0
if sl is None or float(sl) <= 0:
    sl = self.sl_points if self.sl_points > 0 else 10 # Default 10 points if not set
    logging.info(f"Using default/minimum SL: {sl} points")
if tp is None or float(tp) <= 0:
    tp = self.tp_points if self.tp_points > 0 else 20 # Default 20 points if not set
    logging.info(f"Using default/minimum TP: {tp} points")

# Convert to float and ensure they're positive
sl = float(sl)
tp = float(tp)
volume = float(volume)

sl_price, tp_price = self._calculate_sl_tp_price(corrected_symbol, order_type, price, sl, tp)

logging.info(f"Order parameters: price={price}, sl_price={sl_price}, tp_price={tp_price}")

for filling_mode in supported_fillings:
    request = {
        "action": mt5.TRADE_ACTION_DEAL,
        "symbol": corrected_symbol,
        "volume": volume,
        "type": type_mt5,
        "price": price,
        "deviation": 20,
        "type_filling": filling_mode,
        "type_time": mt5.ORDER_TIME_GTC,
    }

    # Add comment if provided
    if comment:
        request["comment"] = comment

    # ALWAYS add SL and TP - never skip them
    if sl_price is not None:
        request["sl"] = sl_price
        logging.info(f"Setting SL price: {sl_price}")
    if tp_price is not None:
        request["tp"] = tp_price
        logging.info(f"Setting TP price: {tp_price}")

    logging.info(f"Sending order request: {request}")
    result = mt5.order_send(request)

    if result is not None:
        logging.info(f"Order result: retcode={result.retcode}, comment={result.comment}")

```

```

if result.retcode == mt5.TRADE_RETCODE_DONE:
    logging.info(
        f"Order successful: {order_type} {corrected_symbol} {volume} at {price}, tick
    )
    return result.order
else:
    # Enhanced error handling for common MT5 trading issues
    error_msg = result.comment if result.comment else "Unknown error"

    # Check for automated trading permission issues
    if result.retcode == 10027: # TRADE_RETCODE_CLIENT_DISABLES_AT
        print(f"[ERROR] MT5 TRADING ERROR: Automated trading is disabled in MT5 terminal")
        print(f"[TOOL] SOLUTION: Enable automated trading in MT5:")
        print(f"    1. Go to Tools → Options → Expert Advisors")
        print(f"    2. Check 'Allow algorithmic trading'")
        print(f"    3. Check 'Allow DLL imports'")
        print(f"    4. Click OK and restart application")
        raise Exception(
            "Automated trading disabled in MT5 - Enable in Tools → Options → Expert Advisors")

    elif result.retcode == 10026: # TRADE_RETCODE_TRADE_DISABLED
        print(f"[ERROR] MT5 TRADING ERROR: Trading is disabled")
        print(f"[TOOL] SOLUTION: Check MT5 terminal settings and broker permissions")
        raise Exception("Trading disabled - Check MT5 settings and broker permissions")

    elif result.retcode == 10013: # TRADE_RETCODE_INVALID_REQUEST
        print(f"[ERROR] MT5 TRADING ERROR: Invalid trading request")
        print(f"[TOOL] SOLUTION: Check symbol, volume, and market hours")
        raise Exception(f"Invalid trading request: {error_msg}")

    elif result.retcode == 10004: # TRADE_RETCODE_REQUOTE
        print(f"[WARNING] MT5 TRADING: Price requote - retrying...")

    elif result.retcode == 10018: # TRADE_RETCODE_MARKET_CLOSED
        print(f"[ERROR] MT5 TRADING ERROR: Market is closed")
        print(f"[TOOL] SOLUTION: Wait for market opening hours")
        raise Exception("Market is closed - Wait for trading hours")

    elif result.retcode == 10019: # TRADE_RETCODE_NO_MONEY
        print(f"[ERROR] MT5 TRADING ERROR: Insufficient funds")
        print(f"[TOOL] SOLUTION: Check account balance and reduce position size")
        raise Exception("Insufficient funds - Check account balance")

    else:
        print(f"[ERROR] MT5 TRADING ERROR: {error_msg} (Code: {result.retcode})")
        print(f"[TOOL] SOLUTION: Check MT5 terminal for detailed error information")

        last_error = f"{error_msg} (Code: {result.retcode})"
else:
    last_error = "No result returned"
    print(f"[ERROR] MT5 CRITICAL: No response from MT5 terminal")
    print(f"[TOOL] SOLUTION: Check MT5 terminal connection and restart if needed")

logging.warning(
    f"Order attempt failed: {order_type} {corrected_symbol} {volume} at {price}, filling=
)

logging.error(
    f"Order failed: {order_type} {corrected_symbol} {volume} at {price}, last_error={last_error}
)
raise Exception(f"Order failed: {last_error}")

except Exception as e:

```

```
logging.exception(f"Exception in place_order: {e}")
raise
```

Method: MT5Automator.buy_market

```
def buy_market(self, symbol, volume=None, sl=None, tp=None, comment=None)
```

Step by step (top-level body)

1. **return** self.place_order(symbol, 'buy', volume=volume, sl=sl, tp=tp, commen....

Code (copy-paste safe)

```
def buy_market(self, symbol, volume=None, sl=None, tp=None, comment=None):
    return self.place_order(symbol, "buy", volume=volume, sl=sl, tp=tp, comment=comment)
```

Method: MT5Automator.sell_market

```
def sell_market(self, symbol, volume=None, sl=None, tp=None, comment=None)
```

Step by step (top-level body)

1. **return** self.place_order(symbol, 'sell', volume=volume, sl=sl, tp=tp, comme....

Code (copy-paste safe)

```
def sell_market(self, symbol, volume=None, sl=None, tp=None, comment=None):
    return self.place_order(symbol, "sell", volume=volume, sl=sl, tp=tp, comment=comment)
```

Method: MT5Automator.is_connected

```
def is_connected(self)
```

Check if MT5 connection is still active

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def is_connected(self):
    """Check if MT5 connection is still active"""
    try:
        # Try to get account info to test connection
        account_info = mt5.account_info()
        if account_info is None:
            return False
        return True
    except Exception:
        return False
```

Method: MT5Automator.check_connection_and_disconnect_if_needed

```
def check_connection_and_disconnect_if_needed(self)
```

Check connection and disconnect if lost

Step by step (top-level body)

1. `if not self.is_connected()`: branches conditionally.
2. `return True`.

Code (copy-paste safe)

```
def check_connection_and_disconnect_if_needed(self):
    """Check connection and disconnect if lost"""
    if not self.is_connected():
        logging.warning("MT5 connection lost, disconnecting...")
        self.disconnect()
        return False
    return True
```

Method: MT5Automator.disconnect

```
def disconnect(self)
```

Properly disconnect from MT5 with enhanced cleanup and terminal closure

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def disconnect(self):
    """Properly disconnect from MT5 with enhanced cleanup and terminal closure"""
    try:
        # First, perform standard MT5 API shutdown
        if mt5.terminal_info() is not None:
            mt5.shutdown()
            logging.info("MT5 API disconnected successfully")
        else:
            logging.info("MT5 API was already disconnected")

        # Force close MT5 terminal processes to ensure complete shutdown
        self._close_mt5_processes()

    except Exception as e:
        logging.error(f"Error during MT5 disconnect: {e}")
        # Force shutdown and process termination even if there was an error
        try:
            mt5.shutdown()
        except:
            pass
```



```
# Still attempt to close processes
self._close_mt5_processes()
```

Method: MT5Automator._close_mt5_processes

```
def _close_mt5_processes(self)
```

Force close all MT5 terminal processes

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def _close_mt5_processes(self):
    """Force close all MT5 terminal processes"""
    try:
        closed_processes = []

        # Look for MT5 processes by name
        for proc in psutil.process_iter(['pid', 'name', 'exe']):
            try:
                proc_name = proc.info['name'].lower()
                proc_exe = proc.info['exe']

                # Check if this is an MT5 process
                if (proc_name in ['terminal64.exe', 'terminal.exe', 'metatrader5.exe'] or
                    (proc_exe and 'metatrader' in proc_exe.lower())):

                    proc.terminate() # Send termination signal
                    closed_processes.append(f"{proc_name} (PID: {proc.info['pid']})")

            except (psutil.NoSuchProcess, psutil.AccessDenied, psutil.ZombieProcess):
                # Process might have already closed or access denied
                continue
            except Exception as e:
                logging.warning(f"Error checking process: {e}")
                continue

        if closed_processes:
            logging.info(f"■ Closed MT5 processes: {'', '.join(closed_processes)}")

            # Wait a moment for graceful termination
            sleep(2)

            # Force kill any remaining MT5 processes
            for proc in psutil.process_iter(['pid', 'name', 'exe']):
                try:
                    proc_name = proc.info['name'].lower()
                    proc_exe = proc.info['exe']
```

```

        if (proc_name in ['terminal64.exe', 'terminal.exe', 'metatrader5.exe'] or
            (proc_exe and 'metatrader' in proc_exe.lower())):

            proc.kill() # Force kill if still running
            logging.info(
                f"■ Force killed stubborn MT5 process: {proc_name} (PID: {proc.info['pid']})"

            )

        except (psutil.NoSuchProcess, psutil.AccessDenied, psutil.ZombieProcess):
            continue
        except Exception as e:
            logging.warning(f"Error force killing process: {e}")
            continue
    else:
        logging.info("■ No MT5 processes found to close")

except Exception as e:
    logging.error(f"Error closing MT5 processes: {e}")
# Fallback: try using taskkill as last resort
try:
    subprocess.run(['taskkill', '/f', '/im', 'terminal64.exe'],
                    capture_output=True, check=False)
    subprocess.run(['taskkill', '/f', '/im', 'terminal.exe'],
                    capture_output=True, check=False)
    logging.info("■ Used taskkill as fallback to close MT5 processes")
except Exception as fallback_error:
    logging.error(f"Fallback taskkill also failed: {fallback_error}")

```

Method: MT5Automator.get_account_info

```
def get_account_info(self)
```

Why these Python concepts were used

- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns `info = mt5.account_info()`.
2. **if** `info` is `None`: branches conditionally.
3. Assigns `trades = mt5.positions_get()`.
4. Assigns `open_trades = len(trades)` if `trades` else 0.
5. Assigns `symbol = ""`.
6. Assigns `direction = ""`.
7. **if** `trades` and `open_trades > 0`: branches conditionally.
8. **return** `{'balance': str(round(info.balance, 2)), 'profit': str(round(info.p...`

Code (copy-paste safe)

```

def get_account_info(self):
    info = mt5.account_info()
    if info is None:
        logging.error("No account info available")
        return {"balance": "", "profit": "", "drawdown": "", "open_trades": "", "Symbol": "",
                "Direction": ""}

```

```

trades = mt5.positions_get()
open_trades = len(trades) if trades else 0
symbol = ""
direction = ""
if trades and open_trades > 0:
    pos = trades[0]
    symbol = getattr(pos, "symbol", "")
    if getattr(pos, "type", None) == mt5.POSITION_TYPE_BUY:
        direction = "Long"
    elif getattr(pos, "type", None) == mt5.POSITION_TYPE_SELL:
        direction = "Short"
    else:
        direction = ""
return {
    "balance": str(round(info.balance, 2)),
    "profit": str(round(info.profit, 2)),
    "drawdown": "",
    "open_trades": str(open_trades),
    "Symbol": symbol,
    "Direction": direction
}

```

Method: MT5Automator.is_trade_open

```
def is_trade_open(self, ticket)
```

Step by step (top-level body)

1. Assigns positions = mt5.positions_get(ticket=ticket).
2. **return** positions is not None and len(positions) > 0.

Code (copy-paste safe)

```

def is_trade_open(self, ticket):
    positions = mt5.positions_get(ticket=ticket)
    return positions is not None and len(positions) > 0

```

Method: MT5Automator.has_open_trade

```
def has_open_trade(self, symbol)
```

Returns True if there is any open position for the given symbol.

Step by step (top-level body)

1. Assigns positions = mt5.positions_get(symbol=symbol).
2. **return** positions is not None and len(positions) > 0.

Code (copy-paste safe)

```

def has_open_trade(self, symbol):
    """
    Returns True if there is any open position for the given symbol.
    """
    positions = mt5.positions_get(symbol=symbol)
    return positions is not None and len(positions) > 0

```

Method: MT5Automator.get_trades_today_count

```
def get_trades_today_count(self, comment_filter=None)
```

Get the number of trades opened today Args: comment_filter: Optional string to filter trades by comment (e.g., "Combine1_") Returns: int: Number of trades opened today

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def get_trades_today_count(self, comment_filter=None):
    """Get the number of trades opened today

    Args:
        comment_filter: Optional string to filter trades by comment (e.g., "Combine1_")

    Returns:
        int: Number of trades opened today
    """
    try:
        from datetime import datetime, date
        import MetaTrader5 as mt5

        today = date.today()
        today_start = datetime.combine(today, datetime.min.time())
        today_end = datetime.combine(today, datetime.max.time())

        # Convert to timestamp
        from_date = int(today_start.timestamp())
        to_date = int(today_end.timestamp())

        # Get history deals (completed trades) for today
        deals = mt5.history_deals_get(from_date, to_date)

        # Also check current open positions opened today
        positions = mt5.positions_get()

        count = 0

        # Count completed deals (history)
        if deals:
            for deal in deals:
                # Only count entry deals (not exit deals)
                if deal.entry == mt5.DEAL_ENTRY_IN:
                    if comment_filter is None or (deal.comment and comment_filter in deal.comment):
                        count += 1

        # Count open positions opened today
        if positions:
            for pos in positions:
```

```

        pos_time = datetime.fromtimestamp(pos.time)
        if pos_time.date() == today:
            if comment_filter is None or (pos.comment and comment_filter in pos.comment):
                count += 1

    logging.info(f"Trades opened today: {count} (filter: {comment_filter})")
    return count

except Exception as e:
    logging.error(f"Error getting today's trade count: {e}")
    return 0

```

Method: MT5Automator.get_orphaned_mt5_positions_by_account

```
def get_orphaned_mt5_positions_by_account(self, account_number)
```

Get MT5 positions for a specific Tradovate account Args: account_number: Tradovate account number to filter by Returns: list: List of orphaned position tickets

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def get_orphaned_mt5_positions_by_account(self, account_number):
    """Get MT5 positions for a specific Tradovate account

    Args:
        account_number: Tradovate account number to filter by

    Returns:
        list: List of orphaned position tickets
    """
    try:
        # Get all open MT5 positions
        positions = mt5.positions_get()
        if positions is None:
            return []

        orphaned_tickets = []
        for pos in positions:
            if pos.comment:
                # Check if position is from this account (comment is just the account number)
                if pos.comment.strip() == account_number:
                    orphaned_tickets.append(pos.ticket)

        return orphaned_tickets
    except Exception as e:
        logging.error(f"Error finding orphaned positions by account: {e}")
        return []

```

Method: MT5Automator.close_orphaned_positions_by_account

```
def close_orphaned_positions_by_account(self, account_number)
```

Close MT5 positions for a specific Tradovate account Args: *account_number*: Tradovate account number to filter by Returns: *int*: Number of positions closed

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def close_orphaned_positions_by_account(self, account_number):
    """Close MT5 positions for a specific Tradovate account

    Args:
        account_number: Tradovate account number to filter by

    Returns:
        int: Number of positions closed
    """
    try:
        orphaned_tickets = self.get_orphaned_mt5_positions_by_account(account_number)
        closed_count = 0

        for ticket in orphaned_tickets:
            try:
                if self.close_trade(ticket):
                    closed_count += 1
                    logging.info(f"Closed orphaned MT5 position for account {account_number}: {ticket}")
            except Exception as e:
                logging.error(f"Error closing orphaned position {ticket}: {e}")

        return closed_count
    except Exception as e:
        logging.error(f"Error closing orphaned positions by account: {e}")
        return 0
```

Method: MT5Automator.get_orphaned_mt5_positions

```
def get_orphaned_mt5_positions(self, combine_comment_prefix)
```

Get MT5 positions that don't have corresponding Tradovate trades Args: *combine_comment_prefix*: Comment prefix to identify trades from this combine (e.g., "Combine1_") Returns: *list*: List of orphaned position tickets

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def get_orphaned_mt5_positions(self, combine_comment_prefix):
    """Get MT5 positions that don't have corresponding Tradovate trades

    Args:
        combine_comment_prefix: Comment prefix to identify trades from this combine (e.g., "Combine1_")

    Returns:
        list: List of orphaned position tickets
    """
    try:
        import MetaTrader5 as mt5

        positions = mt5.positions_get()
        orphaned_tickets = []

        if positions:
            for pos in positions:
                # Check if this position belongs to our combine
                if pos.comment and combine_comment_prefix in pos.comment:
                    orphaned_tickets.append(pos.ticket)
                    logging.info(f"Found orphaned MT5 position: {pos.ticket} ({pos.comment})")

        return orphaned_tickets

    except Exception as e:
        logging.error(f"Error finding orphaned positions: {e}")
        return []
```

Method: MT5Automator.close_orphaned_positions

```
def close_orphaned_positions(self, combine_comment_prefix)

    Close MT5 positions that don't have corresponding Tradovate trades
    Args: combine_comment_prefix:
    Comment prefix to identify trades from this combine (e.g., "Combine1_")
    Returns: int: Number of
    positions closed
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. try block with 1 except clause.

Code (copy-paste safe)

```
def close_orphaned_positions(self, combine_comment_prefix):
    """Close MT5 positions that don't have corresponding Tradovate trades

    Args:
        combine_comment_prefix: Comment prefix to identify trades from this combine (e.g., "Combine1

    Returns:
        int: Number of positions closed
    """
    try:
        orphaned_tickets = self.get_orphaned_mt5_positions(combine_comment_prefix)
        closed_count = 0

        for ticket in orphaned_tickets:
            try:
                if self.close_trade(ticket):
                    closed_count += 1
                    logging.info(f"Closed orphaned MT5 position: {ticket}")
            else:
                logging.warning(f"Failed to close orphaned MT5 position: {ticket}")
        except Exception as e:
            logging.error(f"Error closing orphaned position {ticket}: {e}")

        return closed_count

    except Exception as e:
        logging.error(f"Error closing orphaned positions: {e}")
        return 0
```

Method: MT5Automator.close_trade

```
def close_trade(self, ticket, retries=3, delay=2)
```

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **for** attempt in range(retries): iterates.
2. Calls logging.error(...) for its side effect.
3. **return** not self.is_trade_open(ticket).

Code (copy-paste safe)

```
def close_trade(self, ticket, retries=3, delay=2):
    for attempt in range(retries):
        positions = mt5.positions_get(ticket=ticket)
        if not positions:
            logging.info(f"Trade {ticket} already closed.")
            return True
```



```

pos = positions[0]
symbol = pos.symbol
volume = pos.volume
order_type = pos.type
price = mt5.symbol_info_tick(
    symbol).bid if order_type == mt5.POSITION_TYPE_BUY else mt5.symbol_info_tick(symbol).ask
close_type = mt5.ORDER_TYPE_SELL if order_type == mt5.POSITION_TYPE_BUY else mt5.ORDER_TYPE_
request = {
    "action": mt5.TRADE_ACTION_DEAL,
    "symbol": symbol,
    "volume": volume,
    "type": close_type,
    "position": ticket,
    "price": price,
    "deviation": 20,
    "type_filling": mt5.ORDER_FILLING_IOC,
    "type_time": mt5.ORDER_TIME_GTC,
}
result = mt5.order_send(request)
if result is not None and result.retcode == mt5.TRADE_RETCODE_DONE:
    if not self.is_trade_open(ticket):
        logging.info(f"Trade {ticket} closed successfully.")
        return True
    sleep(delay)
logging.error(f"Failed to close trade {ticket} after {retries} attempts.")
return not self.is_trade_open(ticket)

```

Method: MT5Automator.force_close_trade

```
def force_close_trade(self, ticket)
```

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns positions = mt5.positions_get(ticket=ticket).
2. **if** not positions: branches conditionally.
3. Assigns pos = positions[0].
4. Assigns symbol = pos.symbol.
5. Assigns volume = pos.volume.
6. Assigns order_type = pos.type.
7. Assigns price = mt5.symbol_info_tick(symbol).bid if order_type == mt5.POS....
8. Assigns close_type = mt5.ORDER_TYPE_SELL if order_type == mt5.POSITION_TYPE_BU....
9. **for** filling in [mt5.ORDER_FILLING_IOC, mt5.ORDER_FILLING_FOK, mt5.ORDER_...: iterates.
10. Calls logging.error(...) for its side effect.
11. **return** not self.is_trade_open(ticket).

Code (copy-paste safe)

```
def force_close_trade(self, ticket):
    positions = mt5.positions_get(ticket=ticket)
    if not positions:
        logging.info(f"Trade {ticket} already closed (force close).")
        return True
    pos = positions[0]
    symbol = pos.symbol
    volume = pos.volume
    order_type = pos.type
    price = mt5.symbol_info_tick(
        symbol).bid if order_type == mt5.POSITION_TYPE_BUY else mt5.symbol_info_tick(symbol).ask
    close_type = mt5.ORDER_TYPE_SELL if order_type == mt5.POSITION_TYPE_BUY else mt5.ORDER_TYPE_BUY
    for filling in [mt5.ORDER_FILLING_IOC, mt5.ORDER_FILLING_FOK, mt5.ORDER_FILLING_RETURN]:
        request = {
            "action": mt5.TRADE_ACTION_DEAL,
            "symbol": symbol,
            "volume": volume,
            "type": close_type,
            "position": ticket,
            "price": price,
            "deviation": 20,
            "type_filling": filling,
            "type_time": mt5.ORDER_TIME_GTC,
        }
        result = mt5.order_send(request)
        if result is not None and result.retcode == mt5.TRADE_RETCODE_DONE:
            if not self.is_trade_open(ticket):
                logging.info(f"Trade {ticket} force closed successfully.")
                return True
        logging.error(f"Failed to force close trade {ticket}.")
    return not self.is_trade_open(ticket)
```

Method: MT5Automator.get_daily_trade_count

```
def get_daily_trade_count(self, comment_filter=None)
```

Get count of trades placed today from both open positions and history Args: comment_filter: Can be either: - Tradovate account number (e.g., "MFFUEVSTP326057008") - preferred method - Old combine prefix (e.g., "Combine1_") - for backward compatibility

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def get_daily_trade_count(self, comment_filter=None):
    """Get count of trades placed today from both open positions and history
```

```

Args:
    comment_filter: Can be either:
        - Tradovate account number (e.g., "MFFUEVSTP326057008") - preferred method
        - Old combine prefix (e.g., "Combine1_") - for backward compatibility
"""
try:
    from datetime import datetime, date
    import tempfile
    import os

    today = date.today()

    # Check if trades were reset for this filter today
    temp_dir = tempfile.gettempdir()
    reset_file = os.path.join(temp_dir, f"mt5_reset_{comment_filter}_{today.strftime('%Y%m%d')}.txt")
    if os.path.exists(reset_file):
        # Return 0 if reset flag exists for today
        return 0

    trade_count = 0

    # Count from open positions
    positions = mt5.positions_get()
    if positions:
        for pos in positions:
            # Convert MT5 time to date
            pos_date = datetime.fromtimestamp(pos.time).date()
            if pos_date == today:
                # If comment filter is provided, check if position comment matches
                if comment_filter:
                    # For new format: exact match with account number
                    # For old format: substring match with combine prefix
                    if pos.comment and (pos.comment.strip()
                                         == comment_filter or comment_filter in str(pos.comment)):
                        trade_count += 1
                else:
                    trade_count += 1

    # Count from history deals (more reliable for completed trades)
    # Get deals from start of today to now
    today_start = datetime.combine(today, datetime.min.time())
    today_end = datetime.now()

    deals = mt5.history_deals_get(today_start, today_end)
    if deals:
        # Only count entry deals (not exit deals to avoid double counting)
        for deal in deals:
            if deal.entry == mt5.DEAL_ENTRY_IN: # Entry deal only
                # If comment filter is provided, check if deal comment matches
                if comment_filter:
                    # For new format: exact match with account number
                    # For old format: substring match with combine prefix
                    if deal.comment and (deal.comment.strip()
                                         == comment_filter or comment_filter in str(deal.comment)):
                        trade_count += 1
                else:
                    trade_count += 1

    logging.info(f"Daily trade count: {trade_count} (filter: {comment_filter})")
    return trade_count

```

```

except Exception as e:
    logging.error(f"Error counting daily trades: {e}")
    return 0

```

Method: MT5Automator.get_daily_trade_count_by_account

```
def get_daily_trade_count_by_account(self, tradovate_account_number)
```

Get count of trades placed today for a specific Tradovate account Args: tradovate_account_number: Tradovate account number (e.g., "MFFUEVSTP326057008") Returns: int: Number of trades opened today for this account

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def get_daily_trade_count_by_account(self, tradovate_account_number):
    """Get count of trades placed today for a specific Tradovate account

    Args:
        tradovate_account_number: Tradovate account number (e.g., "MFFUEVSTP326057008")

    Returns:
        int: Number of trades opened today for this account
    """
    try:
        from datetime import datetime, date

        today = date.today()
        trade_count = 0

        # Count from open positions
        positions = mt5.positions_get()
        if positions:
            for pos in positions:
                # Convert MT5 time to date
                pos_date = datetime.fromtimestamp(pos.time).date()
                if pos_date == today:
                    # Check if position comment matches the account number (comment is just the account number)
                    if pos.comment and pos.comment.strip() == tradovate_account_number:
                        trade_count += 1

        # Count from history deals
        today_start = datetime.combine(today, datetime.min.time())
        today_end = datetime.now()

        deals = mt5.history_deals_get(today_start, today_end)
        if deals:

```

```

        for deal in deals:
            if deal.entry == mt5.DEAL_ENTRY_IN: # Entry deal only
                # Check if deal comment matches the account number (comment is just the account number)
                if deal.comment and deal.comment.strip() == tradovate_account_number:
                    trade_count += 1

    logging.info(f"Daily trade count for account {tradovate_account_number}: {trade_count}")
    return trade_count

except Exception as e:
    logging.error(f"Error counting daily trades for account {tradovate_account_number}: {e}")
    return 0

```

Method: MT5Automator.reset_daily_trade_count

```
def reset_daily_trade_count(self, comment_filter=None)
```

Reset daily trade count for a specific filter by storing reset timestamp Args: *comment_filter*: Can be either:
 - Tradovate account number (e.g., "MFFUEVSTP326057008") - preferred method - Old combine prefix (e.g., "Combine1_") - for backward compatibility

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def reset_daily_trade_count(self, comment_filter=None):
    """Reset daily trade count for a specific filter by storing reset timestamp

    Args:
        comment_filter: Can be either:
            - Tradovate account number (e.g., "MFFUEVSTP326057008") - preferred method
            - Old combine prefix (e.g., "Combine1_") - for backward compatibility
    """
    try:
        from datetime import datetime
        import tempfile
        import os

        # Create a simple flag file to mark that trades were reset for this filter today
        temp_dir = tempfile.gettempdir()
        reset_file = os.path.join(temp_dir,
                                   f"mt5_reset_{comment_filter}_{datetime.now().strftime('%Y%m%d')}.flag")

        # Create the flag file
        with open(reset_file, 'w') as f:

```

```

        f.write(str(datetime.now().timestamp()))

    logging.info(f"Daily trade count reset for filter: {comment_filter}")
    return True

except Exception as e:
    logging.error(f"Error resetting daily trade count: {e}")
    return False

```

Method: MT5Automator.reset_daily_trade_count_by_account

```
def reset_daily_trade_count_by_account(self, tradovate_account_number)
```

Reset daily trade count for a specific Tradovate account Args: tradovate_account_number: Tradovate account number (e.g., "MFFUEVSTP326057008")

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def reset_daily_trade_count_by_account(self, tradovate_account_number):
    """Reset daily trade count for a specific Tradovate account

    Args:
        tradovate_account_number: Tradovate account number (e.g., "MFFUEVSTP326057008")
    """
    try:
        from datetime import datetime
        import tempfile
        import os

        # Create a simple flag file to mark that trades were reset for this account today
        temp_dir = tempfile.gettempdir()
        reset_file = os.path.join(temp_dir,
                                   f"mt5_reset_account_{tradovate_account_number}_{datetime.now().strftime('%Y%m%d')}.flag")

        # Create the flag file
        with open(reset_file, 'w') as f:
            f.write(str(datetime.now().timestamp()))

        logging.info(f"Daily trade count reset for Tradovate account: {tradovate_account_number}")
        return True

    except Exception as e:
        logging.error(f"Error resetting daily trade count for account {tradovate_account_number}: {e}")
        return False

```

Method: MT5Automator.get_historical_profits_by_account

```
def get_historical_profits_by_account(self, account_number)
```

Get total historical profits for trades from a specific Tradovate account

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def get_historical_profits_by_account(self, account_number):
    """Get total historical profits for trades from a specific Tradovate account"""
    try:
        from datetime import datetime, timedelta

        # Get deals from the last 30 days to get a good history
        end_time = datetime.now()
        start_time = end_time - timedelta(days=30)

        total_profit = 0.0

        # Get historical deals
        deals = mt5.history_deals_get(start_time, end_time)
        if deals:
            for deal in deals:
                # Check if deal comment matches the account number (comment is just the account number)
                if deal.comment and deal.comment.strip() == account_number:
                    # Only count exit deals for profit calculation (avoid double counting)
                    if deal.entry == mt5.DEAL_ENTRY_OUT:
                        total_profit += deal.profit

        logging.info(f"Historical profits for account {account_number}: ${total_profit:.2f}")
        return total_profit

    except Exception as e:
        logging.error(f"Error getting historical profits by account: {e}")
        return 0.0
```

Method: MT5Automator.get_historical_profits

```
def get_historical_profits(self, comment_filter=None)
```

Get total historical profits for trades with specific comment filter

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't

have to handle it.

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def get_historical_profits(self, comment_filter=None):
    """Get total historical profits for trades with specific comment filter"""
    try:
        from datetime import datetime, timedelta

        # Get deals from the last 30 days to get a good history
        end_time = datetime.now()
        start_time = end_time - timedelta(days=30)

        total_profit = 0.0

        # Get historical deals
        deals = mt5.history_deals_get(start_time, end_time)
        if deals:
            for deal in deals:
                # Check if deal comment matches the filter (for specific combine)
                if comment_filter and comment_filter not in str(deal.comment):
                    continue

                # Only count exit deals for profit calculation (avoid double counting)
                if deal.entry == mt5.DEAL_ENTRY_OUT:
                    total_profit += deal.profit

            logging.info(f"Historical profits for {comment_filter}: ${total_profit:.2f}")
            return total_profit

    except Exception as e:
        logging.error(f"Error calculating historical profits: {e}")
        return 0.0
```

Method: MT5Automator.close_orphaned_trades

```
def close_orphaned_trades(self, expected_tradovate_trades)
```

Close MT5 trades that don't have corresponding Tradovate trades Args: *expected_tradovate_trades: List of Tradovate trade symbols/IDs that should have MT5 counterparts* Returns: *List of closed MT5 trade tickets*

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. try block with 1 except clause.

Code (copy-paste safe)

```
def close_orphaned_trades(self, expected_tradovate_trades):
    """Close MT5 trades that don't have corresponding Tradovate trades

    Args:
        expected_tradovate_trades: List of Tradovate trade symbols/IDs that should have MT5 counterpart

    Returns:
        List of closed MT5 trade tickets
    """
    try:
        closed_trades = []
        positions = mt5.positions_get()

        if not positions:
            return closed_trades

        for pos in positions:
            should_close = False

            # If no Tradovate trades expected, close all MT5 trades
            if not expected_tradovate_trades:
                should_close = True
                reason = "no corresponding Tradovate trades"
            else:
                # Check if this MT5 trade has a corresponding Tradovate trade
                # This is a simplified check - in practice you might need more sophisticated matching
                mt5_symbol = pos.symbol
                has_counterpart = False

                for tradovate_trade in expected_tradovate_trades:
                    # Simple symbol matching - you may want to enhance this logic
                    if str(tradovate_trade).upper() in mt5_symbol.upper():
                        has_counterpart = True
                        break

                if not has_counterpart:
                    should_close = True
                    reason = f"no matching Tradovate trade found"

            if should_close:
                logging.info(f"Closing orphaned MT5 trade {pos.ticket}: {pos.symbol} ({reason})")
                if self.close_trade(pos.ticket):
                    closed_trades.append(pos.ticket)
                    logging.info(f"[OK] Closed orphaned trade {pos.ticket}")
                else:
                    logging.error(f"[ERROR] Failed to close orphaned trade {pos.ticket}")

        if closed_trades:
            logging.info(f"Closed {len(closed_trades)} orphaned MT5 trades: {closed_trades}")

        return closed_trades

    except Exception as e:
        logging.error(f"Error closing orphaned trades: {e}")
        return []
```

Method: MT5Automator.extract_tradovate_account_from_comment

```
def extract_tradovate_account_from_comment(self, comment)
```

Extract Tradovate account number from MT5 comment Args: comment: MT5 order comment (e.g., "MFFUEVSTP326057008") Returns: str: Tradovate account number or "Unknown" if not found

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def extract_tradovate_account_from_comment(self, comment):
    """Extract Tradovate account number from MT5 comment

    Args:
        comment: MT5 order comment (e.g., "MFFUEVSTP326057008")

    Returns:
        str: Tradovate account number or "Unknown" if not found
    """
    try:
        if comment and comment.strip():
            # For new format: comment is just the account number
            account_number = comment.strip()
            return account_number if account_number else "Unknown"
        return "Unknown"
    except Exception as e:
        logging.error(f"Error extracting account from comment '{comment}': {e}")
        return "Unknown"
```

Method: MT5Automator.get_trades_by_tradovate_account

```
def get_trades_by_tradovate_account(self, tradovate_account_number=None)
```

Get all MT5 trades associated with a specific Tradovate account Args: tradovate_account_number: Tradovate account number to filter by Returns: dict: Dictionary with 'open_positions' and 'history_deals' lists

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. try block with 1 **except** clause.

Code (copy-paste safe)

```
def get_trades_by_tradovate_account(self, tradovate_account_number=None):
    """Get all MT5 trades associated with a specific Tradovate account

    Args:
        tradovate_account_number: Tradovate account number to filter by

    Returns:
        dict: Dictionary with 'open_positions' and 'history_deals' lists
    """
    try:
        from datetime import datetime, date

        result = {
            'open_positions': [],
            'history_deals': []
        }

        # Get open positions
        positions = mt5.positions_get()
        if positions:
            for pos in positions:
                if pos.comment:
                    account_from_comment = self.extract_tradovate_account_from_comment(pos.comment)
                    if tradovate_account_number is None or account_from_comment == tradovate_account_number:
                        result['open_positions'].append({
                            'ticket': pos.ticket,
                            'symbol': pos.symbol,
                            'volume': pos.volume,
                            'type': 'BUY' if pos.type == mt5.POSITION_TYPE_BUY else 'SELL',
                            'open_time': datetime.fromtimestamp(pos.time),
                            'comment': pos.comment,
                            'tradovate_account': account_from_comment
                        })

        # Get today's history deals
        today = date.today()
        today_start = datetime.combine(today, datetime.min.time())
        today_end = datetime.combine(today, datetime.max.time())
        from_date = int(today_start.timestamp())
        to_date = int(today_end.timestamp())

        deals = mt5.history_deals_get(from_date, to_date)
        if deals:
            for deal in deals:
                if deal.comment:
                    account_from_comment = self.extract_tradovate_account_from_comment(deal.comment)
                    if tradovate_account_number is None or account_from_comment == tradovate_account_number:
                        result['history_deals'].append({
```

```

        'ticket': deal.ticket,
        'symbol': deal.symbol,
        'volume': deal.volume,
        'type': 'BUY' if deal.type == mt5.DEAL_TYPE_BUY else 'SELL',
        'time': datetime.fromtimestamp(deal.time),
        'comment': deal.comment,
        'tradovate_account': account_from_comment,
        'entry': deal.entry
    })

    return result

except Exception as e:
    logging.error(f"Error getting trades by Tradovate account: {e}")
    return {'open_positions': [], 'history_deals': []}

```

Method: MT5Automator.get_symbol_info

```
def get_symbol_info(self, symbol)
```

Get symbol information

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def get_symbol_info(self, symbol):
    """Get symbol information"""
    try:
        return mt5.symbol_info(symbol)
    except Exception as e:
        logging.error(f"Error getting symbol info for {symbol}: {e}")
        return None

```

Method: MT5Automator.debug_symbol_info

```
def debug_symbol_info(self, symbol)
```

Debug method to print all available symbol information

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def debug_symbol_info(self, symbol):
    """Debug method to print all available symbol information"""
    try:
        info = mt5.symbol_info(symbol)
        if info:
            logging.info(f"=== Symbol Info for {symbol} ===")
            for attr in dir(info):
                if not attr.startswith('_'):
                    try:
                        value = getattr(info, attr)
                        logging.info(f"  {attr}: {value}")
                    except:
                        pass
            logging.info("=== End Symbol Info ===")
            return info
        else:
            logging.error(f"No symbol info available for {symbol}")
            return None
    except Exception as e:
        logging.error(f"Error debugging symbol info: {e}")
        return None
```

Method: MT5Automator.get_tick_data

```
def get_tick_data(self, symbol)
```

Get current tick data for symbol

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def get_tick_data(self, symbol):
    """Get current tick data for symbol"""
    try:
        return mt5.symbol_info_tick(symbol)
    except Exception as e:
        logging.error(f"Error getting tick data for {symbol}: {e}")
        return None
```

Method: MT5Automator.should_close_trades_for_rollover

```
def should_close_trades_for_rollover(self, prop_firm_name)
```

Check if trades should be closed based on prop firm rollover schedules. Closing Times (Eastern Time): - Trade Day: 5:00 PM ET - Funding Ticks: 5:00 PM ET - Tradeify: 4:59 PM ET - MFFU: 4:10 PM ET - Alpha Futures: 4:20 PM ET Args: prop_firm_name: Name of the prop firm Returns: bool: True if trades should be closed now, False otherwise

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Imports datetime (lazy import inside the function).
2. Imports pytz (lazy import inside the function).
3. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def should_close_trades_for_rollover(self, prop_firm_name):
    """
    Check if trades should be closed based on prop firm rollover schedules.

    Closing Times (Eastern Time):
    - Trade Day: 5:00 PM ET
    - Funding Ticks: 5:00 PM ET
    - Tradeify: 4:59 PM ET
    - MFFU: 4:10 PM ET
    - Alpha Futures: 4:20 PM ET

    Args:
        prop_firm_name: Name of the prop firm

    Returns:
        bool: True if trades should be closed now, False otherwise
    """
    import datetime
    import pytz

    try:
        # Get current time in Eastern timezone
        eastern = pytz.timezone('US/Eastern')
        current_time = datetime.datetime.now(eastern)

        # Define closing times for each prop firm (24-hour format)
        closing_schedules = {
            "TradeDay": (17, 0), # 5:00 PM Eastern Time
            "Funding Ticks": (17, 0), # 5:00 PM Eastern Time
            "Tradeify": (16, 59), # 4:59 PM Eastern Time
            "MFFU": (16, 10), # 4:10 PM Eastern Standard Time
            "Alpha Futures": (16, 20), # 4:20 PM Eastern Time
        }

        # Get closing time for this prop firm
        closing_time_tuple = closing_schedules.get(prop_firm_name)
```

```

if closing_time_tuple is None:
    # This prop firm doesn't require trade closing
    return False

# Convert closing time to datetime object for proper comparison
closing_hour, closing_minute = closing_time_tuple
closing_time = current_time.replace(hour=closing_hour, minute=closing_minute, second=0,
    microsecond=0)

# Get current date string for tracking
current_date = current_time.strftime("%Y-%m-%d")

# Safety check: Have we already executed rollover for this prop firm today?
if self.rollover_executed_today.get(prop_firm_name) == current_date:
    return False # Already executed today, skip to prevent duplicates

# Check if current time is at or past closing time (with safety buffer)
# This ensures we don't miss rollover due to system delays
if current_time >= closing_time:
    # Also check if we're on a weekday (Monday = 0, Sunday = 6)
    if current_time.weekday() < 5: # Monday through Friday
        current_time_str = current_time.strftime("%H:%M")
        closing_time_str = closing_time.strftime("%H:%M")
        logging.info(
            f"■ Market rollover time reached for {prop_firm_name} at {current_time_str} ET (
        return True

return False

except Exception as e:
    logging.error(f"Error checking rollover schedule for {prop_firm_name}: {e}")
    return False

```

Method: MT5Automator.close_trades_for_rollover

```
def close_trades_for_rollover(self, prop_firm_name, account_comment_prefix=None)
```

Close all MT5 trades for market rollover based on prop firm schedule. Args: prop_firm_name: Name of the prop firm account_comment_prefix: Account number to filter trades (e.g., "MFFU123456") Returns: list: List of closed trade tickets

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **if** not self.should_close_trades_for_rollover(prop_firm_name): branches conditionally.
2. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def close_trades_for_rollover(self, prop_firm_name, account_comment_prefix=None):
    """
```

Close all MT5 trades for market rollover based on prop firm schedule.

Args:

prop_firm_name: Name of the prop firm
account_comment_prefix: Account number to filter trades (e.g., "MFFU123456")

Returns:

list: List of closed trade tickets

"""

```
if not self.should_close_trades_for_rollover(prop_firm_name):  
    return []
```

try:

```
    # Get all open positions  
    positions = mt5.positions_get()  
    if positions is None:  
        logging.warning("No positions found or error getting positions")  
        return []
```

```
    closed_tickets = []
```

```
    for position in positions:  
        # Filter by account comment prefix if provided  
        if account_comment_prefix and not position.comment.startswith(account_comment_prefix):  
            continue
```

```
        # Close the position
```

```
        if self.close_trade(position.ticket):  
            closed_tickets.append(position.ticket)  
            logging.info(
```

```
                f"■ ROLLOVER: Closed trade {position.ticket} for {prop_firm_name} market rollover")
```

```
        else:
```

```
            logging.error(f"[ERROR] Failed to close trade {position.ticket} for rollover")
```

```
    if closed_tickets:
```

```
        logging.info(  
            f"■ ROLLOVER COMPLETE: Closed {len(closed_tickets)} trades for {prop_firm_name} at m
```

```
        # Mark rollover as executed today to prevent duplicate execution
```

```
        import datetime
```

```
        import pytz
```

```
        eastern = pytz.timezone('US/Eastern')
```

```
        current_date = datetime.datetime.now(eastern).strftime("%Y-%m-%d")
```

```
        self.rollover_executed_today[prop_firm_name] = current_date
```

```
    return closed_tickets
```

```
except Exception as e:
```

```
    logging.error(f"Error closing trades for rollover ({prop_firm_name}): {e}")
```

```
    return []
```

Functions

```
def setup_pyinstaller_mt5_environment()
```

Enhanced MT5 environment setup for PyInstaller builds

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.
2. **return** False.

Code (copy-paste safe)

```
def setup_pyinstaller_mt5_environment():
    """Enhanced MT5 environment setup for PyInstaller builds"""
    try:
        # Detect if running in PyInstaller bundle
        if hasattr(sys, '_MEIPASS'):
            print("[SETUP] Detected PyInstaller environment - applying MT5 fixes...")

            # Set up DLL search paths for MT5
            if hasattr(os, 'add_dll_directory'):
                bundle_dir = sys._MEIPASS
                try:
                    os.add_dll_directory(bundle_dir)
                    print(f"    Added bundle directory to DLL path: {bundle_dir}")
                except Exception as e:
                    print(f"    [WARNING] Warning: Could not add bundle directory: {e}")

            # Add common MT5 paths to DLL search
            mt5_paths = [
                r"C:\Program Files\MetaTrader 5",
                r"C:\Program Files (x86)\MetaTrader 5",
                r"C:\Program Files\MetaTrader 5 Terminal"
            ]

            for path in mt5_paths:
                if os.path.exists(path):
                    try:
                        if hasattr(os, 'add_dll_directory'):
                            os.add_dll_directory(path)
                        # Also add to PATH
                        current_path = os.environ.get('PATH', '')
                        if path not in current_path:
                            os.environ['PATH'] = f"{path};{current_path}"
                            print(f"    Added MT5 path: {path}")
                    except Exception as e:
                        print(f"    [WARNING] Warning: Could not add MT5 path {path}: {e}")

            # Set MT5 environment variables
            os.environ['MT5_TERMINAL_PATH'] = ''
            os.environ['PYINSTALLER_MT5_FIX'] = '1'

            print("    [OK] PyInstaller MT5 environment setup completed")
            return True

    except Exception as e:
        print(f"    [WARNING] PyInstaller MT5 setup warning: {e}")
```

```
return False
```

```
def get_installed_mt5_terminals()
```

Detect installed MetaTrader 5 terminals on Windows Returns a list of dictionaries with terminal info

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to `sum`, `any`, `all`, or `max` where the intermediate list would be wasted.
- **lambda** — Uses a single-expression anonymous function — typically as the `key=` for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns `terminals = []`.
2. Calls `logging.info(...)` for its side effect.
3. Assigns `username = os.getenv('USERNAME', '')`.
4. Assigns `common_paths = ['C:\\Program Files\\MetaTrader 5', 'C:\\Program Files (x...`
5. Assigns `registry_paths = [(winreg.HKEY_LOCAL_MACHINE, 'SOFTWARE\\MetaQuotes\\Termi...`
6. **for** `(hkey, reg_path) in registry_paths`: iterates.
7. **for** `path in common_paths`: iterates.
8. **try** block with 1 **except** clause.
9. Assigns `current_dir = os.path.dirname(os.path.abspath(__file__))`.
10. Assigns `parent_dir = os.path.dirname(current_dir)`.
11. Assigns `portable_paths = [os.path.join(parent_dir, 'MT5'), os.path.join(parent_dir...`
12. **for** `path in portable_paths`: iterates.
13. **try** block with 1 **except** clause.
14. Assigns `unique_terminals = []`.
15. ... and 13 more statement(s) in the body.

Code (copy-paste safe)

```
def get_installed_mt5_terminals():
```

```

"""
Detect installed MetaTrader 5 terminals on Windows
Returns a list of dictionaries with terminal info
"""

# Only detect paths - do not initialize anything
terminals = []

# Log detection start without initialization
logging.info("[OK] Starting MT5 path detection (without initialization)")

# Extended common installation paths - including the known working path
username = os.getenv('USERNAME', '')
common_paths = [
    r"C:\Program Files\MetaTrader 5",
    r"C:\Program Files (x86)\MetaTrader 5",
    r"C:\Program Files\MetaTrader 5 Terminal", # Known working path
    rf"C:\Users\{username}\AppData\Roaming\MetaQuotes\Terminal",
    rf"C:\Users\{username}\AppData\Local\Programs\MetaTrader 5",
    rf"C:\Users\{username}\Documents\MetaTrader 5",
    rf"C:\Users\{username}\Desktop\MetaTrader 5",
    r"D:\Program Files\MetaTrader 5",
    r"D:\Program Files (x86)\MetaTrader 5",
    r"E:\Program Files\MetaTrader 5",
    r"E:\Program Files (x86)\MetaTrader 5",
]

# Check multiple registry locations for MT5 installations
registry_paths = [
    (winreg.HKEY_LOCAL_MACHINE, r"SOFTWARE\MetaQuotes\Terminal"),
    (winreg.HKEY_LOCAL_MACHINE, r"SOFTWARE\Wow6432Node\MetaQuotes\Terminal"),
    (winreg.HKEY_CURRENT_USER, r"SOFTWARE\MetaQuotes\Terminal"),
    (winreg.HKEY_LOCAL_MACHINE, r"SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall"),
]

for hkey, reg_path in registry_paths:
    try:
        with winreg.OpenKey(hkey, reg_path) as key:
            i = 0
            while True:
                try:
                    subkey_name = winreg.EnumKey(key, i)
                    try:
                        with winreg.OpenKey(key, subkey_name) as subkey:
                            # Try different value names for path
                            path_values = ["Path", "InstallLocation", "UninstallString", "DisplayIcon"]
                            for value_name in path_values:
                                try:
                                    value = winreg.QueryValueEx(subkey, value_name)[0]
                                    if value:
                                        # Extract directory from various formats
                                        if value_name == "UninstallString":
                                            path = os.path.dirname(value)
                                        elif value_name == "DisplayIcon":
                                            path = os.path.dirname(value)
                                        else:
                                            path = value

                                        # Check if this is a MetaTrader directory
                                        if os.path.exists(path):
                                            mt5_exe = os.path.join(path, "terminal64.exe")

```

```

        if os.path.exists(mt5_exe):
            # Avoid duplicates
            if not any(t["path"] == path for t in terminals):
                terminals.append({
                    "name": f"MetaTrader 5 ({subkey_name})",
                    "path": path,
                    "source": f"registry_{hkey}_{reg_path}"
                })
            break
        except (FileNotFoundError, OSError):
            continue
    except (FileNotFoundError, OSError):
        pass
    i += 1
except OSError:
    break
except (FileNotFoundError, OSError):
    continue

# Check common installation directories
for path in common_paths:
    if os.path.exists(path):
        mt5_exe = os.path.join(path, "terminal64.exe")
        if os.path.exists(mt5_exe):
            # Avoid duplicates
            if not any(t["path"] == path for t in terminals):
                terminals.append({
                    "name": f"MetaTrader 5 ({os.path.basename(path})",
                    "path": path,
                    "source": "common_path"
                })

# Search for MT5 installations in all drives
try:
    import psutil
    for disk in psutil.disk_partitions():
        drive = disk.mountpoint
        search_paths = [
            os.path.join(drive, "Program Files", "MetaTrader 5"),
            os.path.join(drive, "Program Files (x86)", "MetaTrader 5"),
            os.path.join(drive, "MT5"),
            os.path.join(drive, "MetaTrader5"),
            os.path.join(drive, "MetaTrader 5"),
        ]
        for path in search_paths:
            if os.path.exists(path):
                mt5_exe = os.path.join(path, "terminal64.exe")
                if os.path.exists(mt5_exe):
                    if not any(t["path"] == path for t in terminals):
                        terminals.append({
                            "name": f"MetaTrader 5 ({drive}{os.path.basename(path})",
                            "path": path,
                            "source": "drive_search"
                        })
except ImportError:
    # If psutil is not available, skip drive search
    pass

# Check for portable installations in current directory and subdirectories
current_dir = os.path.dirname(os.path.abspath(__file__))

```

```

parent_dir = os.path.dirname(current_dir)
portable_paths = [
    os.path.join(parent_dir, "MT5"),
    os.path.join(parent_dir, "MetaTrader5"),
    os.path.join(parent_dir, "MetaTrader 5"),
    os.path.join(current_dir, "MT5"),
    os.path.join(current_dir, "MetaTrader5"),
    os.path.join(current_dir, "MetaTrader 5"),
]

for path in portable_paths:
    if os.path.exists(path):
        mt5_exe = os.path.join(path, "terminal64.exe")
        if os.path.exists(mt5_exe):
            if not any(t["path"] == path for t in terminals):
                terminals.append({
                    "name": f"MetaTrader 5 (Portable - {os.path.basename(path)}",
                    "path": path,
                    "source": "portable"
                })

# Search in START MENU shortcuts
try:
    start_menu_paths = [
        rf"C:\Users\{username}\AppData\Roaming\Microsoft\Windows\Start Menu\Programs",
        r"C:\ProgramData\Microsoft\Windows\Start Menu\Programs",
    ]

    for start_path in start_menu_paths:
        if os.path.exists(start_path):
            for root, dirs, files in os.walk(start_path):
                for file in files:
                    if "metatrader" in file.lower() and file.endswith(".lnk"):
                        try:
                            import win32com.client
                            shell = win32com.client.Dispatch("WScript.Shell")
                            shortcut = shell.CreateShortCut(os.path.join(root, file))
                            target_path = os.path.dirname(shortcut.Targetpath)
                            if os.path.exists(target_path):
                                mt5_exe = os.path.join(target_path, "terminal64.exe")
                                if os.path.exists(mt5_exe):
                                    if not any(t["path"] == target_path for t in terminals):
                                        terminals.append({
                                            "name": f"MetaTrader 5 (Shortcut - {os.path.basename(file)}",
                                            "path": target_path,
                                            "source": "start_menu"
                                        })
                        except ImportError:
                            # If win32com is not available, skip shortcut search
                            break
except Exception:
    # If there's any error in shortcut search, continue without it
    pass

# Remove duplicates and sort by name
unique_terminals = []
seen_paths = set()
for terminal in terminals:
    if terminal["path"] not in seen_paths:
        unique_terminals.append(terminal)

```

```

        seen_paths.add(terminal["path"])

# Test each terminal to identify which ones work
working_terminals = []
non_working_terminals = []
working_path = r"C:\Program Files\MetaTrader 5 Terminal"

for terminal in unique_terminals:
    # Mark the known working terminal
    if terminal["path"] == working_path:
        terminal["name"] = f"[OK] {terminal['name']} (Recommended)"
        terminal["is_working"] = True
        working_terminals.append(terminal)
    else:
        terminal["is_working"] = False
        non_working_terminals.append(terminal)

# Sort: working terminals first (recommended), then others alphabetically
working_terminals.sort(key=lambda x: x["name"])
non_working_terminals.sort(key=lambda x: x["name"])

# Combine with working terminals first
final_terminals = working_terminals + non_working_terminals

# If no terminals found, add default entry
if not final_terminals:
    final_terminals.append({
        "name": "MetaTrader 5 (Default)",
        "path": "",
        "source": "default",
        "is_working": False
    })

# Log found terminals for debugging
logging.info(f"Found {len(final_terminals)} MT5 terminals:")
for terminal in final_terminals:
    status = "[OK] RECOMMENDED" if terminal.get("is_working") else "[WARNING] Unknown"
    logging.info(f"  {status} {terminal['name']} - {terminal['path']} (source: {terminal['source']})")

return final_terminals

```

Checkpoint — what works now. You can connect to the MT5 terminal from Python. Open the terminal, log in, then in a REPL: from connectors.mt5_connector import MT5Connector; MT5Connector().connect(). A successful return means the rest of the trading engine has something to talk to.

Chapter 6 — Phase 4 — Indicators and strategies

A strategy is a recipe that decides when to buy or sell. Most strategies are built out of *technical indicators* — RSI, moving averages, Bollinger Bands. Each indicator is a tiny pure function: it takes a pandas DataFrame of OHLC bars and returns a Series of values. We build the helper module first, then a single indicator template, then the rest, and finally the strategy that combines them.

Files we build in this phase, in order:

- trader_companion/signals/__init__.py
- trader_companion/signals/sma.py
- trader_companion/signals/ema.py
- trader_companion/signals/rsi.py
- trader_companion/signals/macd.py
- trader_companion/signals/bb.py
- trader_companion/signals/atr.py
- trader_companion/signals/adx.py
- trader_companion/signals/dmi.py
- trader_companion/signals/cci.py
- trader_companion/signals/momentum.py
- trader_companion/signals/roc.py
- trader_companion/signals/obv.py
- trader_companion/signals/mfi.py
- trader_companion/signals/stochastic.py
- trader_companion/signals/supertrend.py
- trader_companion/signals/sar.py
- trader_companion/signals/tsi.py
- trader_companion/signals/wr.py
- trader_companion/signals/cmo.py
- trader_companion/signals/coppock_curve.py
- trader_companion/signals/donchian_channel.py
- trader_companion/signals/elder_ray.py
- trader_companion/signals/fractal.py
- trader_companion/signals/gator_oscillator.py
- trader_companion/signals/keltner_channel.py

- `trader_companion/signals/price_channel.py`
- `trader_companion/signals/ultimate_oscillator.py`
- `trader_companion/signals/vortex.py`
- `trader_companion/strategies/__init__.py`
- `trader_companion/strategies/rsi_overbought_oversold.py`

Python concepts you'll meet in this phase

- Comprehensions (18) · Exceptions (18) · Type hints (11) · f-strings (1)

Each is explained in Chapter 2 (the primer). The number in parentheses is how many of this phase's files use that concept.

Step 6.1 — Build `trader_companion/signals/__init__.py`

What to do. Create the file `trader_companion/signals/__init__.py` in your project root and paste in the code shown in this step. The file is **2** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from `requirements.txt`.

`trader_companion/signals/__init__.py`

2 loc · 0 classes · 0 functions · 0 imports

About the signals folder

Each file in `trader_companion/signals/` implements a single technical indicator. They follow the same shape: a function that takes a pandas DataFrame of OHLC bars and returns a Series of the indicator's values. Many are placeholders today — the signals that are wired into strategies are **rsi**, **ema**, **sma**, **macd**, **bb**, **atr**, and **supertrend**.

Package marker. Importing this folder as a module runs the code here (often empty, sometimes used to re-export public names). across **2** lines.

Step 6.2 — Build `trader_companion/signals/sma.py`

What to do. Create the file `trader_companion/signals/sma.py` in your project root and paste in the code shown in this step. The file is **27** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from `requirements.txt`.

trader_companion/signals/sma.py

27 loc · 0 classes · 1 functions · 3 imports

Single-indicator module — see the signals folder note above. Most of the work is a pandas computation feeding off OHLC bars. It defines **1** module-level function, across **27** lines.

Python concepts demonstrated in this module

- Comprehensions
- Exceptions

Imports — what each one is for

```
import MetaTrader5 as mt5
```

Why imported. Official MetaQuotes Python API for MT5. Connects to a local terminal, places orders, reads tick/bar history.

Used in this file. `mt5.copy_rates_from_pos`

Example. `mt5.initialize(); mt5.order_send(request)`

Other common scenarios.

- `mt5.symbols_get()` / `mt5.symbol_info(symbol)`
- `mt5.history_deals_get(from_, to_)` returns closed deals
- `mt5.account_info()` returns balance/equity/currency
- Always `mt5.shutdown()` in a finally to release the terminal

```
import pandas as pd
```

Why imported. DataFrame library — the workhorse for tabular data: loading CSV/Excel/SQL, joins, aggregations, time-series.

Used in this file. `pd.Series`

Example. `df = pd.read_csv(path); df.groupby('symbol')['pnl'].sum()`

Other common scenarios.

- `df.merge(other, on='key')` joins like SQL
- `df.resample('1D').agg(...)` for time-series rebucketing
- `df.to_sql` / `df.to_excel` / `df.to_parquet` for output
- `df.loc[mask, 'col']` for label-based selection/assignment

```
import numpy as np
```

Why imported. N-dimensional arrays — vectorised numerical computation. Backs pandas; used directly for indicator math.

Used in this file. `np.array`

Example. `np.where(close > open, 1, -1)` # vectorised conditional

Other common scenarios.

- Broadcasting: `arr * scalar`, `arr + arr2` work element-wise
- `np.nan / np.isnan(x)` for missing values
- `np.array(list)` creates a typed array; choose dtype carefully

Functions

```
def get_sma_signal(symbol, timeframe, period=21, return_value=False)
```

Get SMA value for a given symbol and timeframe. Args: symbol: Trading symbol timeframe: MT5 timeframe period: SMA period return_value: If True, returns the SMA value Returns: SMA value or None

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with `append`, and clearer about intent: this is a transform, not a side-effecting loop.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def get_sma_signal(symbol, timeframe, period=21, return_value=False):
    """
    Get SMA value for a given symbol and timeframe.
    Args:
        symbol: Trading symbol
        timeframe: MT5 timeframe
        period: SMA period
        return_value: If True, returns the SMA value
    Returns:
        SMA value or None
    """
    try:
        rates = mt5.copy_rates_from_pos(symbol, timeframe, 0, period + 10)
        if rates is None or len(rates) < period:
            return None
        closes = np.array([rate[4] for rate in rates])
        sma = pd.Series(closes).rolling(window=period).mean().iloc[-1]
        if return_value:
            return sma
        return sma
    except Exception:
        return None
```

Step 6.3 — Build `trader_companion/signals/ema.py`

What to do. Create the file `trader_companion/signals/ema.py` in your project root and paste in the code shown in this step. The file is **27** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from `requirements.txt`.

trader_companion/signals/ema.py

27 loc · 0 classes · 1 functions · 3 imports

Single-indicator module — see the signals folder note above. Most of the work is a pandas computation feeding off OHLC bars. It defines **1** module-level function, across **27** lines.

Python concepts demonstrated in this module

- Comprehensions
- Exceptions

Imports — what each one is for

```
import MetaTrader5 as mt5
```

Why imported. Official MetaQuotes Python API for MT5. Connects to a local terminal, places orders, reads tick/bar history.

Used in this file. `mt5.copy_rates_from_pos`

Example. `mt5.initialize(); mt5.order_send(request)`

Other common scenarios.

- `mt5.symbols_get()` / `mt5.symbol_info(symbol)`
- `mt5.history_deals_get(from_, to_)` returns closed deals
- `mt5.account_info()` returns balance/equity/currency
- Always `mt5.shutdown()` in a finally to release the terminal

```
import pandas as pd
```

Why imported. DataFrame library — the workhorse for tabular data: loading CSV/Excel/SQL, joins, aggregations, time-series.

Used in this file. `pd.Series`

Example. `df = pd.read_csv(path); df.groupby('symbol')['pnl'].sum()`

Other common scenarios.

- `df.merge(other, on='key')` joins like SQL
- `df.resample('1D').agg(...)` for time-series rebucketing
- `df.to_sql` / `df.to_excel` / `df.to_parquet` for output
- `df.loc[mask, 'col']` for label-based selection/assignment

```
import numpy as np
```

Why imported. N-dimensional arrays — vectorised numerical computation. Backs pandas; used directly for indicator math.

Used in this file. `np.array`

Example. `np.where(close > open, 1, -1)` # vectorised conditional

Other common scenarios.

- Broadcasting: `arr * scalar`, `arr + arr2` work element-wise
- `np.nan / np.isnan(x)` for missing values
- `np.array(list)` creates a typed array; choose dtype carefully

Functions

```
def get_ema_signal(symbol, timeframe, period=21, return_value=False)
```

Get EMA value for a given symbol and timeframe. Args: symbol: Trading symbol timeframe: MT5 timeframe period: EMA period return_value: If True, returns the EMA value Returns: EMA value or None

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def get_ema_signal(symbol, timeframe, period=21, return_value=False):
    """
    Get EMA value for a given symbol and timeframe.
    Args:
        symbol: Trading symbol
        timeframe: MT5 timeframe
        period: EMA period
        return_value: If True, returns the EMA value
    Returns:
        EMA value or None
    """
    try:
        rates = mt5.copy_rates_from_pos(symbol, timeframe, 0, period + 10)
        if rates is None or len(rates) < period:
            return None
        closes = np.array([rate[4] for rate in rates])
        ema = pd.Series(closes).ewm(span=period, adjust=False).mean().iloc[-1]
        if return_value:
            return ema
        return ema
    except Exception:
        return None
```

Step 6.4 — Build trader_companion/signals/rsi.py

What to do. Create the file `trader_companion/signals/rsi.py` in your project root and paste in the code shown in this step. The file is **150** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from `requirements.txt`.

What this file is for. Relative Strength Index. The most-developed signal in the tree — illustrates how the rest of the signals are organised.

trader_companion/signals/rsi.py

150 loc · 0 classes · 4 functions · 4 imports

Relative Strength Index. The most-developed signal in the tree — illustrates how the rest of the signals are organised. It defines **4** module-level functions, across **150** lines. Module docstring: *RSI Signal Generator*

Module docstring

RSI Signal Generator Provides RSI-based trading signals for automated trading

Python concepts demonstrated in this module

• Comprehensions • Exceptions • f-strings • Type hints

Imports — what each one is for

```
import logging
```

Why imported. Structured, levelled logging. Replaces `print()` with filterable, formattable output that can route to files, stderr, syslog, or HTTP endpoints.

Used in this file. `logging.debug`, `logging.error`, `logging.warning`

Example. `logging.getLogger(__name__).info('connected to %s', host)`

Other common scenarios.

- `.debug()` / `.info()` / `.warning()` / `.error()` / `.exception()`
- `logger.exception()` captures the current traceback automatically
- `logging.basicConfig(level=logging.INFO)` for quick setup
- Handlers and Formatters route logs to files / streams / syslog

```
import MetaTrader5 as mt5
```

Why imported. Official MetaQuotes Python API for MT5. Connects to a local terminal, places orders, reads tick/bar history.

Used in this file. `mt5.TIMEFRAME_M5`, `mt5.copy_rates_from_pos`

Example. `mt5.initialize(); mt5.order_send(request)`

Other common scenarios.

- `mt5.symbols_get()` / `mt5.symbol_info(symbol)`
- `mt5.history_deals_get(from_, to_)` returns closed deals
- `mt5.account_info()` returns balance/equity/currency
- Always `mt5.shutdown()` in a finally to release the terminal

```
import numpy as np
```

Why imported. N-dimensional arrays — vectorised numerical computation. Backs pandas; used directly for indicator math.

Used in this file. *No direct attribute access detected — likely imported for side effects, re-export, or string references.*

Example. `np.where(close > open, 1, -1)` # vectorised conditional

Other common scenarios.

- Broadcasting: `arr * scalar`, `arr + arr2` work element-wise
- `np.nan` / `np.isnan(x)` for missing values
- `np.array(list)` creates a typed array; choose dtype carefully

```
from typing import Union
from typing import Tuple
from typing import Optional
```

Why imported. Type-hint primitives — generics, unions, Optional, Callable, Protocol, TypeVar, TypedDict.

Used in this file. Optional, Tuple, Union

Example. `def lookup(k: str) -> Optional[int]: ...`

Other common scenarios.

- `List[int]` / `Dict[str, Any]` / `Tuple[int, str]` (or bare `list[int]`+ on 3.9+)
- `Callable[[int, int], int]` for function signatures
- Protocol for structural typing (duck typing with checks)
- `TypeVar('T')` for generic functions and classes

Functions

```
def calculate_rsi(prices: list, period: int=14) -> float
```

Calculate RSI (Relative Strength Index) Args: prices: List of closing prices period: RSI calculation period Returns: RSI value (0-100)

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `prices: list, period: int`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> float`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with `append`, and clearer about intent: this is a transform, not a side-effecting loop.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **if** `len(prices) < period + 1`: branches conditionally.
2. Assigns `deltas = [prices[i] - prices[i - 1] for i in range(1, len(prices))]`.
3. Assigns `gains = [max(0, delta) for delta in deltas]`.
4. Assigns `losses = [max(0, -delta) for delta in deltas]`.
5. **if** `len(gains) >= period`: branches conditionally (with an **else/elif** arm).
6. **if** `avg_loss == 0`: branches conditionally.
7. Assigns `rs = avg_gain / avg_loss`.
8. Assigns `rsi = 100 - 100 / (1 + rs)`.
9. **return** `rsi`.

Code (copy-paste safe)

```
def calculate_rsi(prices: list, period: int = 14) -> float:
    """
    Calculate RSI (Relative Strength Index)

    Args:
        prices: List of closing prices
        period: RSI calculation period

    Returns:
        RSI value (0-100)
    """
    if len(prices) < period + 1:
        return 50.0 # Neutral RSI if not enough data

    # Calculate price changes
    deltas = [prices[i] - prices[i-1] for i in range(1, len(prices))]

    # Separate gains and losses
    gains = [max(0, delta) for delta in deltas]
    losses = [max(0, -delta) for delta in deltas]

    # Calculate average gains and losses
    if len(gains) >= period:
        avg_gain = sum(gains[-period:]) / period
        avg_loss = sum(losses[-period:]) / period
    else:
        avg_gain = sum(gains) / len(gains) if gains else 0
        avg_loss = sum(losses) / len(losses) if losses else 0

    rs = avg_gain / avg_loss
    rsi = 100 - 100 / (1 + rs)
    return rsi
```

```

    avg_loss = sum(losses) / len(losses) if losses else 0

    # Calculate RSI
    if avg_loss == 0:
        return 100.0

    rs = avg_gain / avg_loss
    rsi = 100 - (100 / (1 + rs))

    return rsi

```

```
def get_price_data(symbol: str, timeframe: int, count: int=100) -> Optional[list]
```

Get price data from MT5 Args: symbol: Trading symbol timeframe: MT5 timeframe constant count: Number of bars to retrieve Returns: List of closing prices or None if error

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., symbol: str, timeframe: int, count: int). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type -> Optional[list]. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **list comprehension** — Builds a list in a single expression ([f(x) for x in xs]). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def get_price_data(symbol: str, timeframe: int, count: int = 100) -> Optional[list]:
    """
    Get price data from MT5

    Args:
        symbol: Trading symbol
        timeframe: MT5 timeframe constant
        count: Number of bars to retrieve

    Returns:
        List of closing prices or None if error
    """
    try:
        # Get rates from MT5
        rates = mt5.copy_rates_from_pos(symbol, timeframe, 0, count)

        if rates is None or len(rates) == 0:
            logging.warning(f"No price data available for {symbol}")

```



```

        return None

    # Extract closing prices
    closes = [rate[4] for rate in rates] # rate[4] is close price

    return closes

except Exception as e:
    logging.error(f"Error getting price data for {symbol}: {e}")
    return None

def get_rsi_signal(symbol: str, timeframe: int=mt5.TIMEFRAME_M5, period: int=14, overbought: float=70,
    oversold: float=30, return_value: bool=False) -> Union[str, Tuple[str, float]]

```

Get RSI trading signal Args: symbol: Trading symbol timeframe: MT5 timeframe constant period: RSI calculation period overbought: Overbought threshold oversold: Oversold threshold return_value: Whether to return RSI value along with signal Returns: Trading signal ('buy', 'sell', 'hold') or tuple of (signal, rsi_value)

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., symbol: str, timeframe: int, period: int). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type -> Union[str, Tuple[str, float]]. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. try block with 1 **except** clause.

Code (copy-paste safe)

```

def get_rsi_signal(symbol: str, timeframe: int = mt5.TIMEFRAME_M5,
    period: int = 14, overbought: float = 70, oversold: float = 30,
    return_value: bool = False) -> Union[str, Tuple[str, float]]:
    """
    Get RSI trading signal

    Args:
        symbol: Trading symbol
        timeframe: MT5 timeframe constant
        period: RSI calculation period
        overbought: Overbought threshold
        oversold: Oversold threshold
        return_value: Whether to return RSI value along with signal

    Returns:
        Trading signal ('buy', 'sell', 'hold') or tuple of (signal, rsi_value)
    """
    try:

```

```

# Get price data
prices = get_price_data(symbol, timeframe, period + 50) # Get extra data for accuracy

if prices is None or len(prices) < period + 1:
    logging.warning(f"Insufficient price data for RSI calculation on {symbol}")
    if return_value:
        return 'hold', 50.0
    return 'hold'

# Calculate RSI
rsi_value = calculate_rsi(prices, period)

# Generate signal
if rsi_value <= oversold:
    signal = 'buy'
elif rsi_value >= overbought:
    signal = 'sell'
else:
    signal = 'hold'

logging.debug(f"RSI Signal for {symbol}: RSI={rsi_value:.2f}, Signal={signal}")

if return_value:
    return signal, rsi_value
return signal

except Exception as e:
    logging.error(f"Error calculating RSI signal for {symbol}: {e}")
    if return_value:
        return 'hold', 50.0
    return 'hold'

def get_rsi_value(symbol: str, timeframe: int=mt5.TIMEFRAME_M5, period: int=14) -> float

```

Get current RSI value for symbol Args: symbol: Trading symbol timeframe: MT5 timeframe constant period: RSI calculation period Returns: Current RSI value

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., symbol: str, timeframe: int, period: int). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type -> float. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def get_rsi_value(symbol: str, timeframe: int = mt5.TIMEFRAME_M5, period: int = 14) -> float:
    """
    Get current RSI value for symbol

    Args:
        symbol: Trading symbol
        timeframe: MT5 timeframe constant
        period: RSI calculation period

    Returns:
        Current RSI value
    """
    try:
        prices = get_price_data(symbol, timeframe, period + 50)

        if prices is None or len(prices) < period + 1:
            return 50.0

        return calculate_rsi(prices, period)

    except Exception as e:
        logging.error(f"Error getting RSI value for {symbol}: {e}")
        return 50.0
```

Step 6.5 — Build trader_companion/signals/macd.py

What to do. Create the file trader_companion/signals/macd.py in your project root and paste in the code shown in this step. The file is **42** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from requirements.txt.

trader_companion/signals/macd.py

42 loc · 0 classes · 1 functions · 3 imports

Single-indicator module — see the signals folder note above. Most of the work is a pandas computation feeding off OHLC bars. It defines **1** module-level function, across **42** lines.

Python concepts demonstrated in this module

- Comprehensions
- Exceptions

Imports — what each one is for

import MetaTrader5 as mt5

Why imported. Official MetaQuotes Python API for MT5. Connects to a local terminal, places orders, reads tick/bar history.

Used in this file. mt5.copy_rates_from_pos

Example. `mt5.initialize(); mt5.order_send(request)`

Other common scenarios.

- `mt5.symbols_get()` / `mt5.symbol_info(symbol)`
- `mt5.history_deals_get(from_, to_)` returns closed deals
- `mt5.account_info()` returns balance/equity/currency
- Always `mt5.shutdown()` in a finally to release the terminal

```
import pandas as pd
```

Why imported. DataFrame library — the workhorse for tabular data: loading CSV/Excel/SQL, joins, aggregations, time-series.

Used in this file. `pd.Series`

Example. `df = pd.read_csv(path); df.groupby('symbol')['pnl'].sum()`

Other common scenarios.

- `df.merge(other, on='key')` joins like SQL
- `df.resample('1D').agg(...)` for time-series rebucketing
- `df.to_sql` / `df.to_excel` / `df.to_parquet` for output
- `df.loc[mask, 'col']` for label-based selection/assignment

```
import numpy as np
```

Why imported. N-dimensional arrays — vectorised numerical computation. Backs pandas; used directly for indicator math.

Used in this file. `np.array`

Example. `np.where(close > open, 1, -1)` # vectorised conditional

Other common scenarios.

- Broadcasting: `arr * scalar`, `arr + arr2` work element-wise
- `np.nan` / `np.isnan(x)` for missing values
- `np.array(list)` creates a typed array; choose dtype carefully

Functions

```
def get_macd_signal(symbol, timeframe, fast_period=12, slow_period=26, signal_period=9, return_value=False):
```

Get MACD signal for a given symbol and timeframe. Args: symbol: Trading symbol timeframe: MT5 timeframe fast_period: Fast EMA period slow_period: Slow EMA period signal_period: Signal line EMA period return_value: If True, returns (signal, macd, signal_line) tuple, otherwise just signal Returns: If return_value=False: signal string ("buy", "sell", or None) If return_value=True: tuple (signal, macd, signal_line)

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't

have to handle it.

- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def get_macd_signal(symbol, timeframe, fast_period=12, slow_period=26, signal_period=9, return_value=False):
    """
    Get MACD signal for a given symbol and timeframe.
    Args:
        symbol: Trading symbol
        timeframe: MT5 timeframe
        fast_period: Fast EMA period
        slow_period: Slow EMA period
        signal_period: Signal line EMA period
        return_value: If True, returns (signal, macd, signal_line) tuple, otherwise just signal
    Returns:
        If return_value=False: signal string ("buy", "sell", or None)
        If return_value=True: tuple (signal, macd, signal_line)
    """
    try:
        rates = mt5.copy_rates_from_pos(symbol, timeframe, 0, slow_period + signal_period + 10)
        if rates is None or len(rates) < slow_period + signal_period:
            return None
        closes = np.array([rate[4] for rate in rates])
        fast_ema = pd.Series(closes).ewm(span=fast_period, adjust=False).mean()
        slow_ema = pd.Series(closes).ewm(span=slow_period, adjust=False).mean()
        macd = fast_ema - slow_ema
        signal_line = macd.ewm(span=signal_period, adjust=False).mean()
        # Use the last value for signal
        macd_val = macd.iloc[-1]
        signal_val = signal_line.iloc[-1]
        if macd_val > signal_val:
            signal = "buy"
        elif macd_val < signal_val:
            signal = "sell"
        else:
            signal = None
        if return_value:
            return signal, macd_val, signal_val
        return signal
    except Exception:
        return None
```

Step 6.6 — Build trader_companion/signals/bb.py

What to do. Create the file `trader_companion/signals/bb.py` in your project root and paste in the code shown in this step. The file is **38** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from `requirements.txt`.

trader_companion/signals/bb.py

38 loc · 0 classes · 1 functions · 3 imports

Single-indicator module — see the signals folder note above. Most of the work is a pandas computation feeding off OHLC bars. It defines **1** module-level function, across **38** lines.

Python concepts demonstrated in this module

- Comprehensions
- Exceptions

Imports — what each one is for

```
import MetaTrader5 as mt5
```

Why imported. Official MetaQuotes Python API for MT5. Connects to a local terminal, places orders, reads tick/bar history.

Used in this file. `mt5.copy_rates_from_pos`

Example. `mt5.initialize(); mt5.order_send(request)`

Other common scenarios.

- `mt5.symbols_get()` / `mt5.symbol_info(symbol)`
- `mt5.history_deals_get(from_, to_)` returns closed deals
- `mt5.account_info()` returns balance/equity/currency
- Always `mt5.shutdown()` in a finally to release the terminal

```
import pandas as pd
```

Why imported. DataFrame library — the workhorse for tabular data: loading CSV/Excel/SQL, joins, aggregations, time-series.

Used in this file. `pd.Series`

Example. `df = pd.read_csv(path); df.groupby('symbol')['pnl'].sum()`

Other common scenarios.

- `df.merge(other, on='key')` joins like SQL
- `df.resample('1D').agg(...)` for time-series rebucketing
- `df.to_sql` / `df.to_excel` / `df.to_parquet` for output
- `df.loc[mask, 'col']` for label-based selection/assignment

```
import numpy as np
```

Why imported. N-dimensional arrays — vectorised numerical computation. Backs pandas; used directly for indicator math.

Used in this file. np.array

Example. np.where(close > open, 1, -1) # vectorised conditional

Other common scenarios.

- Broadcasting: arr * scalar, arr + arr2 work element-wise
- np.nan / np.isnan(x) for missing values
- np.array(list) creates a typed array; choose dtype carefully

Functions

```
def get_bb_signal(symbol, timeframe, period=20, deviation=2.0, return_value=False)
```

Get Bollinger Bands signal for a given symbol and timeframe. Args: symbol: Trading symbol timeframe: MT5 timeframe period: BB period deviation: Standard deviation multiplier return_value: If True, returns (signal, upper, lower, close), else just signal Returns: Signal ("upper", "lower", or None) or (signal, upper, lower, close)

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **list comprehension** — Builds a list in a single expression ([f(x) for x in xs]). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.

Step by step (top-level body)

1. try block with 1 except clause.

Code (copy-paste safe)

```
def get_bb_signal(symbol, timeframe, period=20, deviation=2.0, return_value=False):
    """
    Get Bollinger Bands signal for a given symbol and timeframe.
    Args:
        symbol: Trading symbol
        timeframe: MT5 timeframe
        period: BB period
        deviation: Standard deviation multiplier
        return_value: If True, returns (signal, upper, lower, close), else just signal
    Returns:
        Signal ("upper", "lower", or None) or (signal, upper, lower, close)
    """
    try:
        rates = mt5.copy_rates_from_pos(symbol, timeframe, 0, period + 10)
        if rates is None or len(rates) < period:
            return None
        closes = np.array([rate[4] for rate in rates])
        series = pd.Series(closes)
        sma = series.rolling(window=period).mean().iloc[-1]
        std = series.rolling(window=period).std().iloc[-1]
```

```

upper = sma + deviation * std
lower = sma - deviation * std
close = closes[-1]
signal = None
if close > upper:
    signal = "upper"
elif close < lower:
    signal = "lower"
if return_value:
    return signal, upper, lower, close
return signal
except Exception:
    return None

```

Step 6.7 — Build trader_companion/signals/atr.py

What to do. Create the file trader_companion/signals/atr.py in your project root and paste in the code shown in this step. The file is **30** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from requirements.txt.

trader_companion/signals/atr.py

30 loc · 0 classes · 1 functions · 3 imports

Single-indicator module — see the signals folder note above. Most of the work is a pandas computation feeding off OHLC bars. It defines **1** module-level function, across **30** lines.

Python concepts demonstrated in this module

- Comprehensions
- Exceptions

Imports — what each one is for

```
import MetaTrader5 as mt5
```

Why imported. Official MetaQuotes Python API for MT5. Connects to a local terminal, places orders, reads tick/bar history.

Used in this file. mt5.copy_rates_from_pos

Example. mt5.initialize(); mt5.order_send(request)

Other common scenarios.

- mt5.symbols_get() / mt5.symbol_info(symbol)
- mt5.history_deals_get(from_, to_) returns closed deals
- mt5.account_info() returns balance/equity/currency
- Always mt5.shutdown() in a finally to release the terminal


```
import pandas as pd
```

Why imported. DataFrame library — the workhorse for tabular data: loading CSV/Excel/SQL, joins, aggregations, time-series.

Used in this file. `pd.Series`

Example. `df = pd.read_csv(path); df.groupby('symbol')['pnl'].sum()`

Other common scenarios.

- `df.merge(other, on='key')` joins like SQL
- `df.resample('1D').agg(...)` for time-series rebucketing
- `df.to_sql` / `df.to_excel` / `df.to_parquet` for output
- `df.loc[mask, 'col']` for label-based selection/assignment

```
import numpy as np
```

Why imported. N-dimensional arrays — vectorised numerical computation. Backs pandas; used directly for indicator math.

Used in this file. `np.abs`, `np.array`, `np.maximum`

Example. `np.where(close > open, 1, -1)` # vectorised conditional

Other common scenarios.

- Broadcasting: `arr * scalar`, `arr + arr2` work element-wise
- `np.nan` / `np.isnan(x)` for missing values
- `np.array(list)` creates a typed array; choose dtype carefully

Functions

```
def get_atr_signal(symbol, timeframe, period=14, return_value=False)
```

*Get Average True Range (ATR) value for a given symbol and timeframe. Args: symbol: Trading symbol
timeframe: MT5 timeframe period: ATR period return_value: If True, returns the ATR value Returns: ATR value or None*

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def get_atr_signal(symbol, timeframe, period=14, return_value=False):  
    """  
    Get Average True Range (ATR) value for a given symbol and timeframe.
```

```

Args:
    symbol: Trading symbol
    timeframe: MT5 timeframe
    period: ATR period
    return_value: If True, returns the ATR value
Returns:
    ATR value or None
"""
try:
    rates = mt5.copy_rates_from_pos(symbol, timeframe, 0, period + 2)
    if rates is None or len(rates) < period + 1:
        return None
    highs = np.array([rate[2] for rate in rates])
    lows = np.array([rate[3] for rate in rates])
    closes = np.array([rate[4] for rate in rates])
    trs = np.maximum(highs[1:] - lows[1:], np.abs(highs[1:] - closes[:-1]), np.abs(lows[1:] - closes[:-1]))
    atr = pd.Series(trs).rolling(window=period).mean().iloc[-1]
    if return_value:
        return atr
    return atr
except Exception:
    return None

```

Step 6.8 — Build `trader_companion/signals/adx.py`

What to do. Create the file `trader_companion/signals/adx.py` in your project root and paste in the code shown in this step. The file is **40** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from `requirements.txt`.

`trader_companion/signals/adx.py`

40 loc · 0 classes · 1 functions · 3 imports

Single-indicator module — see the signals folder note above. Most of the work is a pandas computation feeding off OHLC bars. It defines **1** module-level function, across **40** lines.

Python concepts demonstrated in this module

- Comprehensions
- Exceptions

Imports — *what each one is for*

```
import MetaTrader5 as mt5
```

Why imported. Official MetaQuotes Python API for MT5. Connects to a local terminal, places orders, reads tick/bar history.

Used in this file. `mt5.copy_rates_from_pos`

Example. `mt5.initialize(); mt5.order_send(request)`

Other common scenarios.

- `mt5.symbols_get()` / `mt5.symbol_info(symbol)`
- `mt5.history_deals_get(from_, to_)` returns closed deals
- `mt5.account_info()` returns balance/equity/currency
- Always `mt5.shutdown()` in a finally to release the terminal

```
import pandas as pd
```

Why imported. DataFrame library — the workhorse for tabular data: loading CSV/Excel/SQL, joins, aggregations, time-series.

Used in this file. `pd.Series`

Example. `df = pd.read_csv(path); df.groupby('symbol')['pnl'].sum()`

Other common scenarios.

- `df.merge(other, on='key')` joins like SQL
- `df.resample('1D').agg(...)` for time-series rebucketing
- `df.to_sql` / `df.to_excel` / `df.to_parquet` for output
- `df.loc[mask, 'col']` for label-based selection/assignment

```
import numpy as np
```

Why imported. N-dimensional arrays — vectorised numerical computation. Backs pandas; used directly for indicator math.

Used in this file. `np.abs`, `np.array`, `np.maximum`, `np.where`

Example. `np.where(close > open, 1, -1)` # vectorised conditional

Other common scenarios.

- Broadcasting: `arr * scalar`, `arr + arr2` work element-wise
- `np.nan` / `np.isnan(x)` for missing values
- `np.array(list)` creates a typed array; choose dtype carefully

Functions

```
def get_adx_signal(symbol, timeframe, period=14, threshold=25, return_value=False)
```

Get ADX value for a given symbol and timeframe. Args: symbol: Trading symbol timeframe: MT5 timeframe period: ADX period threshold: ADX threshold for trend strength return_value: If True, returns (signal, adx_value), else just signal Returns: Signal ("trend" or None) or (signal, adx_value)

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with `append`, and clearer about intent: this is a transform, not a side-effecting loop.
- **ternary expression** — Uses `x` if `cond` else `y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def get_adx_signal(symbol, timeframe, period=14, threshold=25, return_value=False):
    """
    Get ADX value for a given symbol and timeframe.
    Args:
        symbol: Trading symbol
        timeframe: MT5 timeframe
        period: ADX period
        threshold: ADX threshold for trend strength
        return_value: If True, returns (signal, adx_value), else just signal
    Returns:
        Signal ("trend" or None) or (signal, adx_value)
    """
    try:
        rates = mt5.copy_rates_from_pos(symbol, timeframe, 0, period + 50)
        if rates is None or len(rates) < period + 1:
            return None
        highs = np.array([rate[2] for rate in rates])
        lows = np.array([rate[3] for rate in rates])
        closes = np.array([rate[4] for rate in rates])
        plus_dm = highs[1:] - highs[:-1]
        minus_dm = lows[:-1] - lows[1:]
        plus_dm = np.where((plus_dm > minus_dm) & (plus_dm > 0), plus_dm, 0)
        minus_dm = np.where((minus_dm > plus_dm) & (minus_dm > 0), minus_dm, 0)
        tr = np.maximum(highs[1:] - lows[1:], np.abs(highs[1:] - closes[:-1]), np.abs(lows[1:] - closes[:-1]))
        atr = pd.Series(tr).rolling(window=period).mean()
        plus_di = 100 * pd.Series(plus_dm).rolling(window=period).mean() / atr
        minus_di = 100 * pd.Series(minus_dm).rolling(window=period).mean() / atr
        dx = 100 * np.abs(plus_di - minus_di) / (plus_di + minus_di)
        adx = pd.Series(dx).rolling(window=period).mean().iloc[-1]
        signal = "trend" if adx > threshold else None
        if return_value:
            return signal, adx
        return signal
    except Exception:
        return None
```

Step 6.9 — Build `trader_companion/signals/dmi.py`

What to do. Create the file `trader_companion/signals/dmi.py` in your project root and paste in the code shown in this step. The file is **42** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from `requirements.txt`.

trader_companion/signals/dmi.py

42 loc · 0 classes · 1 functions · 3 imports

Single-indicator module — see the signals folder note above. Most of the work is a pandas computation feeding off OHLC bars. It defines **1** module-level function, across **42** lines.

Python concepts demonstrated in this module

- Comprehensions
- Exceptions

Imports — *what each one is for*

```
import MetaTrader5 as mt5
```

Why imported. Official MetaQuotes Python API for MT5. Connects to a local terminal, places orders, reads tick/bar history.

Used in this file. `mt5.copy_rates_from_pos`

Example. `mt5.initialize(); mt5.order_send(request)`

Other common scenarios.

- `mt5.symbols_get()` / `mt5.symbol_info(symbol)`
- `mt5.history_deals_get(from_, to_)` returns closed deals
- `mt5.account_info()` returns balance/equity/currency
- Always `mt5.shutdown()` in a finally to release the terminal

```
import pandas as pd
```

Why imported. DataFrame library — the workhorse for tabular data: loading CSV/Excel/SQL, joins, aggregations, time-series.

Used in this file. `pd.Series`

Example. `df = pd.read_csv(path); df.groupby('symbol')['pnl'].sum()`

Other common scenarios.

- `df.merge(other, on='key')` joins like SQL
- `df.resample('1D').agg(...)` for time-series rebucketing
- `df.to_sql` / `df.to_excel` / `df.to_parquet` for output
- `df.loc[mask, 'col']` for label-based selection/assignment

```
import numpy as np
```

Why imported. N-dimensional arrays — vectorised numerical computation. Backs pandas; used directly for indicator math.

Used in this file. `np.abs`, `np.array`, `np.maximum`, `np.where`

Example. `np.where(close > open, 1, -1)` # vectorised conditional

Other common scenarios.

- Broadcasting: `arr * scalar`, `arr + arr2` work element-wise
- `np.nan` / `np.isnan(x)` for missing values
- `np.array(list)` creates a typed array; choose `dtype` carefully

Functions

```
def get_dmi_signal(symbol, timeframe, period=14, threshold=25, return_value=False)
```

Get Directional Movement Index (DMI) signal for a given symbol and timeframe. Args: symbol: Trading symbol timeframe: MT5 timeframe period: DMI period threshold: DMI threshold return_value: If True, returns (signal, plus_di, minus_di), else just signal Returns: Signal ("bullish", "bearish", or None) or (signal, plus_di, minus_di)

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with `append`, and clearer about intent: this is a transform, not a side-effecting loop.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def get_dmi_signal(symbol, timeframe, period=14, threshold=25, return_value=False):
    """
    Get Directional Movement Index (DMI) signal for a given symbol and timeframe.
    Args:
        symbol: Trading symbol
        timeframe: MT5 timeframe
        period: DMI period
        threshold: DMI threshold
        return_value: If True, returns (signal, plus_di, minus_di), else just signal
    Returns:
        Signal ("bullish", "bearish", or None) or (signal, plus_di, minus_di)
    """
    try:
        rates = mt5.copy_rates_from_pos(symbol, timeframe, 0, period + 50)
        if rates is None or len(rates) < period + 1:
            return None
        highs = np.array([rate[2] for rate in rates])
        lows = np.array([rate[3] for rate in rates])
        closes = np.array([rate[4] for rate in rates])
        plus_dm = highs[1:] - highs[:-1]
        minus_dm = lows[:-1] - lows[1:]
        plus_dm = np.where((plus_dm > minus_dm) & (plus_dm > 0), plus_dm, 0)
        minus_dm = np.where((minus_dm > plus_dm) & (minus_dm > 0), minus_dm, 0)
        tr = np.maximum(highs[1:] - lows[1:], np.abs(highs[1:] - closes[:-1]), np.abs(lows[1:] - closes[:-1]))
        atr = pd.Series(tr).rolling(window=period).mean()
        plus_di = 100 * pd.Series(plus_dm).rolling(window=period).mean() / atr
        minus_di = 100 * pd.Series(minus_dm).rolling(window=period).mean() / atr
        signal = None
```

```

    if plus_di.iloc[-1] > minus_di.iloc[-1] and plus_di.iloc[-1] > threshold:
        signal = "bullish"
    elif minus_di.iloc[-1] > plus_di.iloc[-1] and minus_di.iloc[-1] > threshold:
        signal = "bearish"
    if return_value:
        return signal, plus_di.iloc[-1], minus_di.iloc[-1]
    return signal
except Exception:
    return None

```

Step 6.10 — Build trader_companion/signals/cci.py

What to do. Create the file `trader_companion/signals/cci.py` in your project root and paste in the code shown in this step. The file is **40** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from `requirements.txt`.

trader_companion/signals/cci.py

40 loc · 0 classes · 1 functions · 3 imports

Single-indicator module — see the signals folder note above. Most of the work is a pandas computation feeding off OHLC bars. It defines **1** module-level function, across **40** lines.

Python concepts demonstrated in this module

- Comprehensions
- Exceptions

Imports — what each one is for

```
import MetaTrader5 as mt5
```

Why imported. Official MetaQuotes Python API for MT5. Connects to a local terminal, places orders, reads tick/bar history.

Used in this file. `mt5.copy_rates_from_pos`

Example. `mt5.initialize(); mt5.order_send(request)`

Other common scenarios.

- `mt5.symbols_get()` / `mt5.symbol_info(symbol)`
- `mt5.history_deals_get(from_, to_)` returns closed deals
- `mt5.account_info()` returns balance/equity/currency
- Always `mt5.shutdown()` in a finally to release the terminal

```
import pandas as pd
```

Why imported. DataFrame library — the workhorse for tabular data: loading CSV/Excel/SQL, joins, aggregations, time-series.

Used in this file. `pd.Series`

Example. `df = pd.read_csv(path); df.groupby('symbol')['pnl'].sum()`

Other common scenarios.

- `df.merge(other, on='key')` joins like SQL
- `df.resample('1D').agg(...)` for time-series rebucketing
- `df.to_sql` / `df.to_excel` / `df.to_parquet` for output
- `df.loc[mask, 'col']` for label-based selection/assignment

`import numpy as np`

Why imported. N-dimensional arrays — vectorised numerical computation. Backs pandas; used directly for indicator math.

Used in this file. `np.abs`, `np.array`, `np.mean`

Example. `np.where(close > open, 1, -1)` # vectorised conditional

Other common scenarios.

- Broadcasting: `arr * scalar`, `arr + arr2` work element-wise
- `np.nan` / `np.isnan(x)` for missing values
- `np.array(list)` creates a typed array; choose dtype carefully

Functions

```
def get_cci_signal(symbol, timeframe, period=20, overbought=100, oversold=-100, return_value=False)
```

Get Commodity Channel Index (CCI) signal for a given symbol and timeframe. Args: symbol: Trading symbol timeframe: MT5 timeframe period: CCI period overbought: Overbought threshold oversold: Oversold threshold return_value: If True, returns (signal, cci_value), else just signal Returns: Signal ("buy", "sell", or None) or (signal, cci_value)

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def get_cci_signal(symbol, timeframe, period=20, overbought=100, oversold=-100, return_value=False):
```



```

"""
Get Commodity Channel Index (CCI) signal for a given symbol and timeframe.
Args:
    symbol: Trading symbol
    timeframe: MT5 timeframe
    period: CCI period
    overbought: Overbought threshold
    oversold: Oversold threshold
    return_value: If True, returns (signal, cci_value), else just signal
Returns:
    Signal ("buy", "sell", or None) or (signal, cci_value)
"""
try:
    rates = mt5.copy_rates_from_pos(symbol, timeframe, 0, period + 10)
    if rates is None or len(rates) < period:
        return None
    highs = np.array([rate[2] for rate in rates])
    lows = np.array([rate[3] for rate in rates])
    closes = np.array([rate[4] for rate in rates])
    typical_price = (highs + lows + closes) / 3
    tp_series = pd.Series(typical_price)
    sma = tp_series.rolling(window=period).mean().iloc[-1]
    mean_dev = np.mean(np.abs(tp_series[-period:] - sma))
    cci = (tp_series.iloc[-1] - sma) / (0.015 * mean_dev) if mean_dev != 0 else 0
    signal = None
    if cci > overbought:
        signal = "sell"
    elif cci < oversold:
        signal = "buy"
    if return_value:
        return signal, cci
    return signal
except Exception:
    return None

```

Step 6.11 — Build trader_companion/signals/momentum.py

What to do. Create the file `trader_companion/signals/momentum.py` in your project root and paste in the code shown in this step. The file is **33** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from `requirements.txt`.

trader_companion/signals/momentum.py

33 loc · 0 classes · 1 functions · 3 imports

Single-indicator module — see the signals folder note above. Most of the work is a pandas computation feeding off OHLC bars. It defines **1** module-level function, across **33** lines.

Python concepts demonstrated in this module

- Comprehensions
- Exceptions

Imports — what each one is for

```
import MetaTrader5 as mt5
```

Why imported. Official MetaQuotes Python API for MT5. Connects to a local terminal, places orders, reads tick/bar history.

Used in this file. `mt5.copy_rates_from_pos`

Example. `mt5.initialize(); mt5.order_send(request)`

Other common scenarios.

- `mt5.symbols_get()` / `mt5.symbol_info(symbol)`
- `mt5.history_deals_get(from_, to_)` returns closed deals
- `mt5.account_info()` returns balance/equity/currency
- Always `mt5.shutdown()` in a finally to release the terminal

```
import pandas as pd
```

Why imported. DataFrame library — the workhorse for tabular data: loading CSV/Excel/SQL, joins, aggregations, time-series.

Used in this file. *No direct attribute access detected — likely imported for side effects, re-export, or string references.*

Example. `df = pd.read_csv(path); df.groupby('symbol')['pnl'].sum()`

Other common scenarios.

- `df.merge(other, on='key')` joins like SQL
- `df.resample('1D').agg(...)` for time-series rebucketing
- `df.to_sql` / `df.to_excel` / `df.to_parquet` for output
- `df.loc[mask, 'col']` for label-based selection/assignment

```
import numpy as np
```

Why imported. N-dimensional arrays — vectorised numerical computation. Backs pandas; used directly for indicator math.

Used in this file. `np.array`

Example. `np.where(close > open, 1, -1)` # vectorised conditional

Other common scenarios.

- Broadcasting: `arr * scalar`, `arr + arr2` work element-wise
- `np.nan` / `np.isnan(x)` for missing values
- `np.array(list)` creates a typed array; choose dtype carefully

Functions

```
def get_momentum_signal(symbol, timeframe, period=10, threshold=0.3, return_value=False)
```

Get Momentum value for a given symbol and timeframe. Args: symbol: Trading symbol timeframe: MT5 timeframe period: Momentum period threshold: Momentum threshold return_value: If True, returns (signal, momentum_value), else just signal Returns: Signal ("bullish", "bearish", or None) or (signal, momentum_value)

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def get_momentum_signal(symbol, timeframe, period=10, threshold=0.3, return_value=False):
    """
    Get Momentum value for a given symbol and timeframe.
    Args:
        symbol: Trading symbol
        timeframe: MT5 timeframe
        period: Momentum period
        threshold: Momentum threshold
        return_value: If True, returns (signal, momentum_value), else just signal
    Returns:
        Signal ("bullish", "bearish", or None) or (signal, momentum_value)
    """
    try:
        rates = mt5.copy_rates_from_pos(symbol, timeframe, 0, period + 10)
        if rates is None or len(rates) < period + 1:
            return None
        closes = np.array([rate[4] for rate in rates])
        momentum = closes[-1] - closes[-period-1]
        signal = None
        if momentum > threshold:
            signal = "bullish"
        elif momentum < -threshold:
            signal = "bearish"
        if return_value:
            return signal, momentum
        return signal
    except Exception:
        return None
```

Step 6.12 — Build trader_companion/signals/roc.py

What to do. Create the file `trader_companion/signals/roc.py` in your project root and paste in the code shown in this step. The file is **33** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from `requirements.txt`.

trader_companion/signals/roc.py

33 loc · 0 classes · 1 functions · 3 imports

Single-indicator module — see the signals folder note above. Most of the work is a pandas computation feeding off OHLC bars. It defines **1** module-level function, across **33** lines.

Python concepts demonstrated in this module

- Comprehensions
- Exceptions

Imports — what each one is for

```
import MetaTrader5 as mt5
```

Why imported. Official MetaQuotes Python API for MT5. Connects to a local terminal, places orders, reads tick/bar history.

Used in this file. `mt5.copy_rates_from_pos`

Example. `mt5.initialize(); mt5.order_send(request)`

Other common scenarios.

- `mt5.symbols_get()` / `mt5.symbol_info(symbol)`
- `mt5.history_deals_get(from_, to_)` returns closed deals
- `mt5.account_info()` returns balance/equity/currency
- Always `mt5.shutdown()` in a finally to release the terminal

```
import pandas as pd
```

Why imported. DataFrame library — the workhorse for tabular data: loading CSV/Excel/SQL, joins, aggregations, time-series.

Used in this file. *No direct attribute access detected — likely imported for side effects, re-export, or string references.*

Example. `df = pd.read_csv(path); df.groupby('symbol')['pnl'].sum()`

Other common scenarios.

- `df.merge(other, on='key')` joins like SQL
- `df.resample('1D').agg(...)` for time-series rebucketing
- `df.to_sql` / `df.to_excel` / `df.to_parquet` for output
- `df.loc[mask, 'col']` for label-based selection/assignment

```
import numpy as np
```

Why imported. N-dimensional arrays — vectorised numerical computation. Backs pandas; used directly for indicator math.

Used in this file. `np.array`

Example. `np.where(close > open, 1, -1)` # vectorised conditional

Other common scenarios.

- Broadcasting: `arr * scalar`, `arr + arr2` work element-wise
- `np.nan` / `np.isnan(x)` for missing values
- `np.array(list)` creates a typed array; choose dtype carefully

Functions

```
def get_roc_signal(symbol, timeframe, period=12, threshold=0, return_value=False)
```

*Get Rate of Change (ROC) signal for a given symbol and timeframe. Args: symbol: Trading symbol
timeframe: MT5 timeframe period: ROC period threshold: ROC threshold return_value: If True, returns
(signal, roc_value), else just signal Returns: Signal ("bullish", "bearish", or None) or (signal, roc_value)*

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def get_roc_signal(symbol, timeframe, period=12, threshold=0, return_value=False):
    """
    Get Rate of Change (ROC) signal for a given symbol and timeframe.
    Args:
        symbol: Trading symbol
        timeframe: MT5 timeframe
        period: ROC period
        threshold: ROC threshold
        return_value: If True, returns (signal, roc_value), else just signal
    Returns:
        Signal ("bullish", "bearish", or None) or (signal, roc_value)
    """
    try:
        rates = mt5.copy_rates_from_pos(symbol, timeframe, 0, period + 10)
        if rates is None or len(rates) < period + 1:
            return None
        closes = np.array([rate[4] for rate in rates])
        roc = ((closes[-1] - closes[-period-1]) / closes[-period-1]) * 100
        signal = None
        if roc > threshold:
```

```

        signal = "bullish"
    elif roc < -threshold:
        signal = "bearish"
    if return_value:
        return signal, roc
    return signal
except Exception:
    return None

```

Step 6.13 — Build trader_companion/signals/obv.py

What to do. Create the file `trader_companion/signals/obv.py` in your project root and paste in the code shown in this step. The file is **35** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from `requirements.txt`.

trader_companion/signals/obv.py

35 loc · 0 classes · 1 functions · 3 imports

Single-indicator module — see the signals folder note above. Most of the work is a pandas computation feeding off OHLC bars. It defines **1** module-level function, across **35** lines.

Python concepts demonstrated in this module

- Comprehensions
- Exceptions

Imports — what each one is for

```
import MetaTrader5 as mt5
```

Why imported. Official MetaQuotes Python API for MT5. Connects to a local terminal, places orders, reads tick/bar history.

Used in this file. `mt5.copy_rates_from_pos`

Example. `mt5.initialize(); mt5.order_send(request)`

Other common scenarios.

- `mt5.symbols_get()` / `mt5.symbol_info(symbol)`
- `mt5.history_deals_get(from_, to_)` returns closed deals
- `mt5.account_info()` returns balance/equity/currency
- Always `mt5.shutdown()` in a finally to release the terminal

```
import pandas as pd
```

Why imported. DataFrame library — the workhorse for tabular data: loading CSV/Excel/SQL, joins, aggregations, time-series.

Used in this file. *No direct attribute access detected — likely imported for side effects, re-export, or string references.*

Example. `df = pd.read_csv(path); df.groupby('symbol')['pnl'].sum()`

Other common scenarios.

- `df.merge(other, on='key')` joins like SQL
- `df.resample('1D').agg(...)` for time-series rebucketing
- `df.to_sql` / `df.to_excel` / `df.to_parquet` for output
- `df.loc[mask, 'col']` for label-based selection/assignment

```
import numpy as np
```

Why imported. N-dimensional arrays — vectorised numerical computation. Backs pandas; used directly for indicator math.

Used in this file. `np.array`

Example. `np.where(close > open, 1, -1)` # vectorised conditional

Other common scenarios.

- Broadcasting: `arr * scalar`, `arr + arr2` work element-wise
- `np.nan` / `np.isnan(x)` for missing values
- `np.array(list)` creates a typed array; choose dtype carefully

Functions

```
def get_obv_signal(symbol, timeframe, return_value=False)
```

*Get On-Balance Volume (OBV) signal for a given symbol and timeframe. Args: symbol: Trading symbol
timeframe: MT5 timeframe return_value: If True, returns the OBV value Returns: OBV value or None*

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def get_obv_signal(symbol, timeframe, return_value=False):
    """
    Get On-Balance Volume (OBV) signal for a given symbol and timeframe.
    Args:
        symbol: Trading symbol
```

```

    timeframe: MT5 timeframe
    return_value: If True, returns the OBV value
Returns:
    OBV value or None
"""
try:
    rates = mt5.copy_rates_from_pos(symbol, timeframe, 0, 100)
    if rates is None or len(rates) < 2:
        return None
    closes = np.array([rate[4] for rate in rates])
    volumes = np.array([rate[5] for rate in rates])
    obv = [volumes[0]]
    for i in range(1, len(closes)):
        if closes[i] > closes[i-1]:
            obv.append(obv[-1] + volumes[i])
        elif closes[i] < closes[i-1]:
            obv.append(obv[-1] - volumes[i])
        else:
            obv.append(obv[-1])
    obv_value = obv[-1]
    if return_value:
        return obv_value
    return obv_value
except Exception:
    return None

```

Step 6.14 — Build trader_companion/signals/mfi.py

What to do. Create the file `trader_companion/signals/mfi.py` in your project root and paste in the code shown in this step. The file is **43** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from `requirements.txt`.

trader_companion/signals/mfi.py

43 loc · 0 classes · 1 functions · 3 imports

Single-indicator module — see the signals folder note above. Most of the work is a pandas computation feeding off OHLC bars. It defines **1** module-level function, across **43** lines.

Python concepts demonstrated in this module

- Comprehensions
- Exceptions

Imports — what each one is for

```
import MetaTrader5 as mt5
```

Why imported. Official MetaQuotes Python API for MT5. Connects to a local terminal, places orders, reads tick/bar history.

Used in this file. `mt5.copy_rates_from_pos`

Example. `mt5.initialize(); mt5.order_send(request)`

Other common scenarios.

- `mt5.symbols_get()` / `mt5.symbol_info(symbol)`
- `mt5.history_deals_get(from_, to_)` returns closed deals
- `mt5.account_info()` returns balance/equity/currency
- Always `mt5.shutdown()` in a finally to release the terminal

```
import pandas as pd
```

Why imported. DataFrame library — the workhorse for tabular data: loading CSV/Excel/SQL, joins, aggregations, time-series.

Used in this file. `pd.Series`

Example. `df = pd.read_csv(path); df.groupby('symbol')['pnl'].sum()`

Other common scenarios.

- `df.merge(other, on='key')` joins like SQL
- `df.resample('1D').agg(...)` for time-series rebucketing
- `df.to_sql` / `df.to_excel` / `df.to_parquet` for output
- `df.loc[mask, 'col']` for label-based selection/assignment

```
import numpy as np
```

Why imported. N-dimensional arrays — vectorised numerical computation. Backs pandas; used directly for indicator math.

Used in this file. `np.array`, `np.where`

Example. `np.where(close > open, 1, -1)` # vectorised conditional

Other common scenarios.

- Broadcasting: `arr * scalar`, `arr + arr2` work element-wise
- `np.nan` / `np.isnan(x)` for missing values
- `np.array(list)` creates a typed array; choose dtype carefully

Functions

```
def get_mfi_signal(symbol, timeframe, period=14, overbought=80, oversold=20, return_value=False)
```

*Get Money Flow Index (MFI) signal for a given symbol and timeframe. Args: symbol: Trading symbol
timeframe: MT5 timeframe period: MFI period overbought: Overbought threshold oversold: Oversold
threshold return_value: If True, returns (signal, mfi_value), else just signal Returns: Signal ("buy", "sell",
or None) or (signal, mfi_value)*

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't

have to handle it.

- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **ternary expression** — Uses `x` if `cond` else `y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. try block with 1 **except** clause.

Code (copy-paste safe)

```
def get_mfi_signal(symbol, timeframe, period=14, overbought=80, oversold=20, return_value=False):
    """
    Get Money Flow Index (MFI) signal for a given symbol and timeframe.
    Args:
        symbol: Trading symbol
        timeframe: MT5 timeframe
        period: MFI period
        overbought: Overbought threshold
        oversold: Oversold threshold
        return_value: If True, returns (signal, mfi_value), else just signal
    Returns:
        Signal ("buy", "sell", or None) or (signal, mfi_value)
    """
    try:
        rates = mt5.copy_rates_from_pos(symbol, timeframe, 0, period + 10)
        if rates is None or len(rates) < period + 1:
            return None
        highs = np.array([rate[2] for rate in rates])
        lows = np.array([rate[3] for rate in rates])
        closes = np.array([rate[4] for rate in rates])
        volumes = np.array([rate[5] for rate in rates])
        typical_price = (highs + lows + closes) / 3
        money_flow = typical_price * volumes
        positive_flow = np.where(typical_price[1:] > typical_price[:-1], money_flow[1:], 0)
        negative_flow = np.where(typical_price[1:] < typical_price[:-1], money_flow[1:], 0)
        pos_mf = pd.Series(positive_flow).rolling(window=period).sum().iloc[-1]
        neg_mf = pd.Series(negative_flow).rolling(window=period).sum().iloc[-1]
        mfi = 100 * pos_mf / (pos_mf + neg_mf) if (pos_mf + neg_mf) != 0 else 50
        signal = None
        if mfi > overbought:
            signal = "sell"
        elif mfi < oversold:
            signal = "buy"
        if return_value:
            return signal, mfi
        return signal
    except Exception:
        return None
```

Step 6.15 — Build trader_companion/signals/stochastic.py

What to do. Create the file `trader_companion/signals/stochastic.py` in your project root and paste in the code shown in this step. The file is **42** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from `requirements.txt`.

trader_companion/signals/stochastic.py

42 loc · 0 classes · 1 functions · 3 imports

Single-indicator module — see the signals folder note above. Most of the work is a pandas computation feeding off OHLC bars. It defines **1** module-level function, across **42** lines.

Python concepts demonstrated in this module

- Comprehensions
- Exceptions

Imports — what each one is for

```
import MetaTrader5 as mt5
```

Why imported. Official MetaQuotes Python API for MT5. Connects to a local terminal, places orders, reads tick/bar history.

Used in this file. `mt5.copy_rates_from_pos`

Example. `mt5.initialize(); mt5.order_send(request)`

Other common scenarios.

- `mt5.symbols_get()` / `mt5.symbol_info(symbol)`
- `mt5.history_deals_get(from_, to_)` returns closed deals
- `mt5.account_info()` returns balance/equity/currency
- Always `mt5.shutdown()` in a finally to release the terminal

```
import pandas as pd
```

Why imported. DataFrame library — the workhorse for tabular data: loading CSV/Excel/SQL, joins, aggregations, time-series.

Used in this file. `pd.Series`

Example. `df = pd.read_csv(path); df.groupby('symbol')['pnl'].sum()`

Other common scenarios.

- `df.merge(other, on='key')` joins like SQL
- `df.resample('1D').agg(...)` for time-series rebucketing
- `df.to_sql` / `df.to_excel` / `df.to_parquet` for output
- `df.loc[mask, 'col']` for label-based selection/assignment

```
import numpy as np
```

Why imported. N-dimensional arrays — vectorised numerical computation. Backs pandas; used directly for indicator math.

Used in this file. np.array

Example. np.where(close > open, 1, -1) # vectorised conditional

Other common scenarios.

- Broadcasting: arr * scalar, arr + arr2 work element-wise
- np.nan / np.isnan(x) for missing values
- np.array(list) creates a typed array; choose dtype carefully

Functions

```
def get_stochastic_signal(symbol, timeframe, k_period=14, d_period=3, overbought=80, oversold=20,
    return_value=False)
```

*Get Stochastic Oscillator signal for a given symbol and timeframe. Args: symbol: Trading symbol
timeframe: MT5 timeframe k_period: %K period d_period: %D period overbought: Overbought threshold
oversold: Oversold threshold return_value: If True, returns (signal, k_value, d_value), else just signal
Returns: Signal ("buy", "sell", or None) or (signal, k_value, d_value)*

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **list comprehension** — Builds a list in a single expression ([f(x) for x in xs]). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.

Step by step (top-level body)

1. try block with 1 except clause.

Code (copy-paste safe)

```
def get_stochastic_signal(symbol, timeframe, k_period=14, d_period=3, overbought=80, oversold=20,
    return_value=False):
    """
    Get Stochastic Oscillator signal for a given symbol and timeframe.
    Args:
        symbol: Trading symbol
        timeframe: MT5 timeframe
        k_period: %K period
        d_period: %D period
        overbought: Overbought threshold
        oversold: Oversold threshold
        return_value: If True, returns (signal, k_value, d_value), else just signal
    Returns:
        Signal ("buy", "sell", or None) or (signal, k_value, d_value)
    """
    try:
        rates = mt5.copy_rates_from_pos(symbol, timeframe, 0, k_period + d_period + 10)
        if rates is None or len(rates) < k_period + d_period:
            return None
```

```

highs = np.array([rate[2] for rate in rates])
lows = np.array([rate[3] for rate in rates])
closes = np.array([rate[4] for rate in rates])
lowest_low = pd.Series(lows).rolling(window=k_period).min()
highest_high = pd.Series(highs).rolling(window=k_period).max()
k = 100 * (closes - lowest_low) / (highest_high - lowest_low)
d = k.rolling(window=d_period).mean()
k_value = k.iloc[-1]
d_value = d.iloc[-1]
signal = None
if k_value > overbought:
    signal = "sell"
elif k_value < oversold:
    signal = "buy"
if return_value:
    return signal, k_value, d_value
return signal
except Exception:
    return None

```

Step 6.16 — Build trader_companion/signals/supertrend.py

What to do. Create the file `trader_companion/signals/supertrend.py` in your project root and paste in the code shown in this step. The file is **44** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from `requirements.txt`.

trader_companion/signals/supertrend.py

44 loc · 0 classes · 1 functions · 3 imports

Single-indicator module — see the signals folder note above. Most of the work is a pandas computation feeding off OHLC bars. It defines **1** module-level function, across **44** lines.

Python concepts demonstrated in this module

- Comprehensions
- Exceptions

Imports — what each one is for

```
import MetaTrader5 as mt5
```

Why imported. Official MetaQuotes Python API for MT5. Connects to a local terminal, places orders, reads tick/bar history.

Used in this file. `mt5.copy_rates_from_pos`

Example. `mt5.initialize(); mt5.order_send(request)`

Other common scenarios.

- `mt5.symbols_get()` / `mt5.symbol_info(symbol)`
- `mt5.history_deals_get(from_, to_)` returns closed deals
- `mt5.account_info()` returns balance/equity/currency
- Always `mt5.shutdown()` in a finally to release the terminal

```
import pandas as pd
```

Why imported. DataFrame library — the workhorse for tabular data: loading CSV/Excel/SQL, joins, aggregations, time-series.

Used in this file. `pd.Series`

Example. `df = pd.read_csv(path); df.groupby('symbol')['pnl'].sum()`

Other common scenarios.

- `df.merge(other, on='key')` joins like SQL
- `df.resample('1D').agg(...)` for time-series rebucketing
- `df.to_sql` / `df.to_excel` / `df.to_parquet` for output
- `df.loc[mask, 'col']` for label-based selection/assignment

```
import numpy as np
```

Why imported. N-dimensional arrays — vectorised numerical computation. Backs pandas; used directly for indicator math.

Used in this file. `np.abs`, `np.array`, `np.maximum`, `np.ones_like`, `np.zeros_like`

Example. `np.where(close > open, 1, -1)` # vectorised conditional

Other common scenarios.

- Broadcasting: `arr * scalar`, `arr + arr2` work element-wise
- `np.nan` / `np.isnan(x)` for missing values
- `np.array(list)` creates a typed array; choose dtype carefully

Functions

```
def get_supertrend_signal(symbol, timeframe, period=10, multiplier=3.0, return_value=False)
```

Get Supertrend signal for a given symbol and timeframe. Args: symbol: Trading symbol timeframe: MT5 timeframe period: ATR period for Supertrend multiplier: Multiplier for ATR return_value: If True, returns (signal, supertrend), else just signal Returns: Signal ("bullish", "bearish", or None) or (signal, supertrend)

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.

• **ternary expression** — Uses `x` if `cond` else `y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. `try` block with 1 `except` clause.

Code (copy-paste safe)

```
def get_supertrend_signal(symbol, timeframe, period=10, multiplier=3.0, return_value=False):
    """
    Get Supertrend signal for a given symbol and timeframe.
    Args:
        symbol: Trading symbol
        timeframe: MT5 timeframe
        period: ATR period for Supertrend
        multiplier: Multiplier for ATR
        return_value: If True, returns (signal, supertrend), else just signal
    Returns:
        Signal ("bullish", "bearish", or None) or (signal, supertrend)
    """
    try:
        rates = mt5.copy_rates_from_pos(symbol, timeframe, 0, period + 10)
        if rates is None or len(rates) < period + 1:
            return None
        highs = np.array([rate[2] for rate in rates])
        lows = np.array([rate[3] for rate in rates])
        closes = np.array([rate[4] for rate in rates])
        atr = pd.Series(np.maximum(highs[1:] - lows[1:], np.abs(highs[1:] - closes[:-1]), np.abs(lows[1:] - closes[:-1]))).rolling(window=period).mean()
        hl2 = (highs + lows) / 2
        upperband = hl2 - (multiplier * atr)
        lowerband = hl2 + (multiplier * atr)
        supertrend = np.zeros_like(closes)
        direction = np.ones_like(closes)
        for i in range(1, len(closes)):
            if closes[i] > upperband[i]:
                direction[i] = 1
            elif closes[i] < lowerband[i]:
                direction[i] = -1
            else:
                direction[i] = direction[i-1]
            supertrend[i] = lowerband[i] if direction[i] == 1 else upperband[i]
        signal = "bullish" if direction[-1] == 1 else "bearish"
        if return_value:
            return signal, supertrend[-1]
        return signal
    except Exception:
        return None
```

Step 6.17 — Build `trader_companion/signals/sar.py`

What to do. Create the file `trader_companion/signals/sar.py` in your project root and paste in the code shown in this step. The file is **56** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from `requirements.txt`.

trader_companion/signals/sar.py

56 loc · 0 classes · 1 functions · 3 imports

Single-indicator module — see the signals folder note above. Most of the work is a pandas computation feeding off OHLC bars. It defines **1** module-level function, across **56** lines.

Python concepts demonstrated in this module

- Comprehensions
- Exceptions

Imports — what each one is for

```
import MetaTrader5 as mt5
```

Why imported. Official MetaQuotes Python API for MT5. Connects to a local terminal, places orders, reads tick/bar history.

Used in this file. `mt5.copy_rates_from_pos`

Example. `mt5.initialize(); mt5.order_send(request)`

Other common scenarios.

- `mt5.symbols_get()` / `mt5.symbol_info(symbol)`
- `mt5.history_deals_get(from_, to_)` returns closed deals
- `mt5.account_info()` returns balance/equity/currency
- Always `mt5.shutdown()` in a finally to release the terminal

```
import pandas as pd
```

Why imported. DataFrame library — the workhorse for tabular data: loading CSV/Excel/SQL, joins, aggregations, time-series.

Used in this file. *No direct attribute access detected — likely imported for side effects, re-export, or string references.*

Example. `df = pd.read_csv(path); df.groupby('symbol')['pnl'].sum()`

Other common scenarios.

- `df.merge(other, on='key')` joins like SQL
- `df.resample('1D').agg(...)` for time-series rebucketing
- `df.to_sql` / `df.to_excel` / `df.to_parquet` for output
- `df.loc[mask, 'col']` for label-based selection/assignment


```
import numpy as np
```

Why imported. N-dimensional arrays — vectorised numerical computation. Backs pandas; used directly for indicator math.

Used in this file. `np.array`, `np.zeros_like`

Example. `np.where(close > open, 1, -1)` # vectorised conditional

Other common scenarios.

- Broadcasting: `arr * scalar`, `arr + arr2` work element-wise
- `np.nan` / `np.isnan(x)` for missing values
- `np.array(list)` creates a typed array; choose dtype carefully

Functions

```
def get_sar_signal(symbol, timeframe, step=0.02, max_step=0.2, return_value=False)
```

Get Parabolic SAR signal for a given symbol and timeframe. Args: symbol: Trading symbol timeframe: MT5 timeframe step: SAR step max_step: SAR max step return_value: If True, returns (signal, sar_value), else just signal Returns: Signal ("buy", "sell", or None) or (signal, sar_value)

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def get_sar_signal(symbol, timeframe, step=0.02, max_step=0.2, return_value=False):
    """
    Get Parabolic SAR signal for a given symbol and timeframe.
    Args:
        symbol: Trading symbol
        timeframe: MT5 timeframe
        step: SAR step
        max_step: SAR max step
        return_value: If True, returns (signal, sar_value), else just signal
    Returns:
        Signal ("buy", "sell", or None) or (signal, sar_value)
    """
    try:
        rates = mt5.copy_rates_from_pos(symbol, timeframe, 0, 100)
        if rates is None or len(rates) < 2:
            return None
        highs = np.array([rate[2] for rate in rates])
```

```

    lows = np.array([rate[3] for rate in rates])
    closes = np.array([rate[4] for rate in rates])
    sar = np.zeros_like(closes)
    trend = 1 # 1 for up, -1 for down
    af = step
    ep = highs[0] if trend == 1 else lows[0]
    sar[0] = lows[0] if trend == 1 else highs[0]
    for i in range(1, len(closes)):
        sar[i] = sar[i-1] + af * (ep - sar[i-1])
        if trend == 1:
            if highs[i] > ep:
                ep = highs[i]
                af = min(af + step, max_step)
            if lows[i] < sar[i]:
                trend = -1
                sar[i] = ep
                ep = lows[i]
                af = step
        else:
            if lows[i] < ep:
                ep = lows[i]
                af = min(af + step, max_step)
            if highs[i] > sar[i]:
                trend = 1
                sar[i] = ep
                ep = highs[i]
                af = step
    sar_value = sar[-1]
    signal = "buy" if closes[-1] > sar_value else "sell"
    if return_value:
        return signal, sar_value
    return signal
except Exception:
    return None

```

Step 6.18 — Build trader_companion/signals/tsi.py

What to do. Create the file `trader_companion/signals/tsi.py` in your project root and paste in the code shown in this step. The file is **39** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from `requirements.txt`.

trader_companion/signals/tsi.py

39 loc · 0 classes · 1 functions · 3 imports

Single-indicator module — see the signals folder note above. Most of the work is a pandas computation feeding off OHLC bars. It defines **1** module-level function, across **39** lines.

Python concepts demonstrated in this module

- Comprehensions
- Exceptions

Imports — what each one is for

```
import MetaTrader5 as mt5
```

Why imported. Official MetaQuotes Python API for MT5. Connects to a local terminal, places orders, reads tick/bar history.

Used in this file. `mt5.copy_rates_from_pos`

Example. `mt5.initialize(); mt5.order_send(request)`

Other common scenarios.

- `mt5.symbols_get()` / `mt5.symbol_info(symbol)`
- `mt5.history_deals_get(from_, to_)` returns closed deals
- `mt5.account_info()` returns balance/equity/currency
- Always `mt5.shutdown()` in a finally to release the terminal

```
import pandas as pd
```

Why imported. DataFrame library — the workhorse for tabular data: loading CSV/Excel/SQL, joins, aggregations, time-series.

Used in this file. `pd.Series`

Example. `df = pd.read_csv(path); df.groupby('symbol')['pnl'].sum()`

Other common scenarios.

- `df.merge(other, on='key')` joins like SQL
- `df.resample('1D').agg(...)` for time-series rebucketing
- `df.to_sql` / `df.to_excel` / `df.to_parquet` for output
- `df.loc[mask, 'col']` for label-based selection/assignment

```
import numpy as np
```

Why imported. N-dimensional arrays — vectorised numerical computation. Backs pandas; used directly for indicator math.

Used in this file. `np.abs`, `np.array`, `np.diff`

Example. `np.where(close > open, 1, -1)` # vectorised conditional

Other common scenarios.

- Broadcasting: `arr * scalar`, `arr + arr2` work element-wise
- `np.nan` / `np.isnan(x)` for missing values
- `np.array(list)` creates a typed array; choose dtype carefully

Functions

```
def get_tsi_signal(symbol, timeframe, r=25, s=13, return_value=False)
```

*Get True Strength Index (TSI) signal for a given symbol and timeframe. Args: symbol: Trading symbol
timeframe: MT5 timeframe r: Long EMA period s: Short EMA period return_value: If True, returns (signal, tsi_value), else just signal Returns: Signal ("bullish", "bearish", or None) or (signal, tsi_value)*

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **list comprehension** — Builds a list in a single expression ([f(x) for x in xs]). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def get_tsi_signal(symbol, timeframe, r=25, s=13, return_value=False):
    """
    Get True Strength Index (TSI) signal for a given symbol and timeframe.
    Args:
        symbol: Trading symbol
        timeframe: MT5 timeframe
        r: Long EMA period
        s: Short EMA period
        return_value: If True, returns (signal, tsi_value), else just signal
    Returns:
        Signal ("bullish", "bearish", or None) or (signal, tsi_value)
    """
    try:
        rates = mt5.copy_rates_from_pos(symbol, timeframe, 0, r + s + 10)
        if rates is None or len(rates) < r + s:
            return None
        closes = np.array([rate[4] for rate in rates])
        diff = np.diff(closes)
        abs_diff = np.abs(diff)
        ema1 = pd.Series(diff).ewm(span=s, adjust=False).mean()
        ema2 = ema1.ewm(span=r, adjust=False).mean()
        abs_ema1 = pd.Series(abs_diff).ewm(span=s, adjust=False).mean()
        abs_ema2 = abs_ema1.ewm(span=r, adjust=False).mean()
        tsi = 100 * (ema2.iloc[-1] / abs_ema2.iloc[-1]) if abs_ema2.iloc[-1] != 0 else 0
        signal = None
        if tsi > 25:
            signal = "bullish"
        elif tsi < -25:
            signal = "bearish"
        if return_value:
            return signal, tsi
        return signal
    except Exception:
        return None
```

Step 6.19 — Build trader_companion/signals/wr.py

What to do. Create the file `trader_companion/signals/wr.py` in your project root and paste in the code shown in this step. The file is **38** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from `requirements.txt`.

trader_companion/signals/wr.py

38 loc · 0 classes · 1 functions · 3 imports

Single-indicator module — see the signals folder note above. Most of the work is a pandas computation feeding off OHLC bars. It defines **1** module-level function, across **38** lines.

Python concepts demonstrated in this module

- Comprehensions
- Exceptions

Imports — what each one is for

```
import MetaTrader5 as mt5
```

Why imported. Official MetaQuotes Python API for MT5. Connects to a local terminal, places orders, reads tick/bar history.

Used in this file. `mt5.copy_rates_from_pos`

Example. `mt5.initialize(); mt5.order_send(request)`

Other common scenarios.

- `mt5.symbols_get()` / `mt5.symbol_info(symbol)`
- `mt5.history_deals_get(from_, to_)` returns closed deals
- `mt5.account_info()` returns balance/equity/currency
- Always `mt5.shutdown()` in a finally to release the terminal

```
import pandas as pd
```

Why imported. DataFrame library — the workhorse for tabular data: loading CSV/Excel/SQL, joins, aggregations, time-series.

Used in this file. *No direct attribute access detected — likely imported for side effects, re-export, or string references.*

Example. `df = pd.read_csv(path); df.groupby('symbol')['pnl'].sum()`

Other common scenarios.

- `df.merge(other, on='key')` joins like SQL

- `df.resample('1D').agg(...)` for time-series rebucketing
- `df.to_sql` / `df.to_excel` / `df.to_parquet` for output
- `df.loc[mask, 'col']` for label-based selection/assignment

`import numpy as np`

Why imported. N-dimensional arrays — vectorised numerical computation. Backs pandas; used directly for indicator math.

Used in this file. `np.array`, `np.max`, `np.min`

Example. `np.where(close > open, 1, -1)` # vectorised conditional

Other common scenarios.

- Broadcasting: `arr * scalar`, `arr + arr2` work element-wise
- `np.nan` / `np.isnan(x)` for missing values
- `np.array(list)` creates a typed array; choose `dtype` carefully

Functions

```
def get_wr_signal(symbol, timeframe, period=14, overbought=-20, oversold=-80, return_value=False)
```

Get Williams %R signal for a given symbol and timeframe. Args: symbol: Trading symbol timeframe: MT5 timeframe period: WR period overbought: Overbought threshold oversold: Oversold threshold return_value: If True, returns (signal, wr_value), else just signal Returns: Signal ("buy", "sell", or None) or (signal, wr_value)

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with `append`, and clearer about intent: this is a transform, not a side-effecting loop.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def get_wr_signal(symbol, timeframe, period=14, overbought=-20, oversold=-80, return_value=False):
    """
    Get Williams %R signal for a given symbol and timeframe.
    Args:
        symbol: Trading symbol
        timeframe: MT5 timeframe
        period: WR period
        overbought: Overbought threshold
        oversold: Oversold threshold
        return_value: If True, returns (signal, wr_value), else just signal
    Returns:
        Signal ("buy", "sell", or None) or (signal, wr_value)
    """
    try:
```

```

rates = mt5.copy_rates_from_pos(symbol, timeframe, 0, period + 10)
if rates is None or len(rates) < period:
    return None
highs = np.array([rate[2] for rate in rates])
lows = np.array([rate[3] for rate in rates])
closes = np.array([rate[4] for rate in rates])
highest_high = np.max(highs[-period:])
lowest_low = np.min(lows[-period:])
wr = -100 * (highest_high - closes[-1]) / (highest_high - lowest_low)
signal = None
if wr > overbought:
    signal = "sell"
elif wr < oversold:
    signal = "buy"
if return_value:
    return signal, wr
return signal
except Exception:
    return None

```

Step 6.20 — Build trader_companion/signals/cmo.py

What to do. Create the file `trader_companion/signals/cmo.py` in your project root and paste in the code shown in this step. The file is **7** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from `requirements.txt`.

trader_companion/signals/cmo.py

7 loc · 0 classes · 1 functions · 1 imports

Single-indicator module — see the signals folder note above. Most of the work is a pandas computation feeding off OHLC bars. It defines **1** module-level function, across **7** lines.

Python concepts demonstrated in this module

- Type hints

Imports — what each one is for

```
import pandas as pd
```

Why imported. DataFrame library — the workhorse for tabular data: loading CSV/Excel/SQL, joins, aggregations, time-series.

Used in this file. `pd.DataFrame`

Example. `df = pd.read_csv(path); df.groupby('symbol')['pnl'].sum()`

Other common scenarios.

- `df.merge(other, on='key')` joins like SQL
- `df.resample('1D').agg(...)` for time-series rebucketing
- `df.to_sql` / `df.to_excel` / `df.to_parquet` for output
- `df.loc[mask, 'col']` for label-based selection/assignment

Functions

```
def get_cmo_signal(df: pd.DataFrame, period: int=14)
```

Why these Python concepts were used

• **type-annotated parameters** — Parameters carry type hints (e.g., `df: pd.DataFrame, period: int`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.

Step by step (top-level body)

1. `return 0`.

Code (copy-paste safe)

```
def get_cmo_signal(df: pd.DataFrame, period: int = 14):
    # Placeholder for Chande Momentum Oscillator (CMO) signal logic
    # Return 1 for buy, -1 for sell, 0 for neutral
    return 0
```

Step 6.21 — Build trader_companion/signals/coppock_curve.py

What to do. Create the file `trader_companion/signals/coppock_curve.py` in your project root and paste in the code shown in this step. The file is **7** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from `requirements.txt`.

trader_companion/signals/coppock_curve.py

7 loc · 0 classes · 1 functions · 1 imports

Single-indicator module — see the signals folder note above. Most of the work is a pandas computation feeding off OHLC bars. It defines **1** module-level function, across **7** lines.

Python concepts demonstrated in this module

- Type hints

Imports — what each one is for

```
import pandas as pd
```

Why imported. DataFrame library — the workhorse for tabular data: loading CSV/Excel/SQL, joins, aggregations, time-series.

Used in this file. `pd.DataFrame`

Example. `df = pd.read_csv(path); df.groupby('symbol')['pnl'].sum()`

Other common scenarios.

- `df.merge(other, on='key')` joins like SQL
- `df.resample('1D').agg(...)` for time-series rebucketing
- `df.to_sql` / `df.to_excel` / `df.to_parquet` for output
- `df.loc[mask, 'col']` for label-based selection/assignment

Functions

```
def get_coppock_curve_signal(df: pd.DataFrame)
```

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `df: pd.DataFrame`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.

Step by step (top-level body)

1. `return 0`.

Code (copy-paste safe)

```
def get_coppock_curve_signal(df: pd.DataFrame):
    # Placeholder for Coppock Curve signal logic
    # Return 1 for buy, -1 for sell, 0 for neutral
    return 0
```

Step 6.22 — Build trader_companion/signals/donchian_channel.py

What to do. Create the file `trader_companion/signals/donchian_channel.py` in your project root and paste in the code shown in this step. The file is 7 lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from `requirements.txt`.

trader_companion/signals/donchian_channel.py

7 loc · 0 classes · 1 functions · 1 imports

Single-indicator module — see the signals folder note above. Most of the work is a pandas computation feeding off OHLC bars. It defines **1** module-level function, across **7** lines.

Python concepts demonstrated in this module

- Type hints

Imports — what each one is for

```
import pandas as pd
```

Why imported. DataFrame library — the workhorse for tabular data: loading CSV/Excel/SQL, joins, aggregations, time-series.

Used in this file. `pd.DataFrame`

Example. `df = pd.read_csv(path); df.groupby('symbol')['pnl'].sum()`

Other common scenarios.

- `df.merge(other, on='key')` joins like SQL
- `df.resample('1D').agg(...)` for time-series rebucketing
- `df.to_sql` / `df.to_excel` / `df.to_parquet` for output
- `df.loc[mask, 'col']` for label-based selection/assignment

Functions

```
def get_donchian_channel_signal(df: pd.DataFrame, period: int=20)
```

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `df: pd.DataFrame, period: int`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.

Step by step (top-level body)

1. `return 0`.

Code (copy-paste safe)

```
def get_donchian_channel_signal(df: pd.DataFrame, period: int = 20):  
    # Placeholder for Donchian Channel signal logic  
    # Return 1 for buy, -1 for sell, 0 for neutral  
    return 0
```

Step 6.23 — Build trader_companion/signals/elder_ray.py

What to do. Create the file `trader_companion/signals/elder_ray.py` in your project root and paste in the code shown in this step. The file is **7** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from `requirements.txt`.

trader_companion/signals/elder_ray.py

7 loc · 0 classes · 1 functions · 1 imports

Single-indicator module — see the signals folder note above. Most of the work is a pandas computation feeding off OHLC bars. It defines **1** module-level function, across **7** lines.

Python concepts demonstrated in this module

- Type hints

Imports — what each one is for

```
import pandas as pd
```

Why imported. DataFrame library — the workhorse for tabular data: loading CSV/Excel/SQL, joins, aggregations, time-series.

Used in this file. `pd.DataFrame`

Example. `df = pd.read_csv(path); df.groupby('symbol')['pnl'].sum()`

Other common scenarios.

- `df.merge(other, on='key')` joins like SQL
- `df.resample('1D').agg(...)` for time-series rebucketing
- `df.to_sql` / `df.to_excel` / `df.to_parquet` for output
- `df.loc[mask, 'col']` for label-based selection/assignment

Functions

```
def get_elder_ray_signal(df: pd.DataFrame, period: int=13)
```

Why these Python concepts were used

• **type-annotated parameters** — Parameters carry type hints (e.g., `df: pd.DataFrame, period: int`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.

Step by step (top-level body)

1. `return 0.`

Code (copy-paste safe)

```
def get_elder_ray_signal(df: pd.DataFrame, period: int = 13):
```

```
# Placeholder for Elder Ray Index signal logic
# Return 1 for buy, -1 for sell, 0 for neutral
return 0
```

Step 6.24 — Build `trader_companion/signals/fractal.py`

What to do. Create the file `trader_companion/signals/fractal.py` in your project root and paste in the code shown in this step. The file is **7** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from `requirements.txt`.

`trader_companion/signals/fractal.py`

7 loc · 0 classes · 1 functions · 1 imports

Single-indicator module — see the signals folder note above. Most of the work is a pandas computation feeding off OHLC bars. It defines **1** module-level function, across **7** lines.

Python concepts demonstrated in this module

- Type hints

Imports — what each one is for

```
import pandas as pd
```

Why imported. DataFrame library — the workhorse for tabular data: loading CSV/Excel/SQL, joins, aggregations, time-series.

Used in this file. `pd.DataFrame`

Example. `df = pd.read_csv(path); df.groupby('symbol')['pnl'].sum()`

Other common scenarios.

- `df.merge(other, on='key')` joins like SQL
- `df.resample('1D').agg(...)` for time-series rebucketing
- `df.to_sql` / `df.to_excel` / `df.to_parquet` for output
- `df.loc[mask, 'col']` for label-based selection/assignment

Functions

```
def get_fractal_signal(df: pd.DataFrame)
```

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `df: pd.DataFrame`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and

human readers. They are the fastest way to make a public function self-explanatory.

Step by step (top-level body)

1. `return 0`.

Code (copy-paste safe)

```
def get_fractal_signal(df: pd.DataFrame):
    # Placeholder for Fractal Indicator signal logic
    # Return 1 for buy, -1 for sell, 0 for neutral
    return 0
```

Step 6.25 — Build trader_companion/signals/gator_oscillator.py

What to do. Create the file `trader_companion/signals/gator_oscillator.py` in your project root and paste in the code shown in this step. The file is **7** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from `requirements.txt`.

trader_companion/signals/gator_oscillator.py

7 loc · 0 classes · 1 functions · 1 imports

Single-indicator module — see the signals folder note above. Most of the work is a pandas computation feeding off OHLC bars. It defines **1** module-level function, across **7** lines.

Python concepts demonstrated in this module

- Type hints

Imports — what each one is for

```
import pandas as pd
```

Why imported. DataFrame library — the workhorse for tabular data: loading CSV/Excel/SQL, joins, aggregations, time-series.

Used in this file. `pd.DataFrame`

Example. `df = pd.read_csv(path); df.groupby('symbol')['pnl'].sum()`

Other common scenarios.

- `df.merge(other, on='key')` joins like SQL
- `df.resample('1D').agg(...)` for time-series rebucketing
- `df.to_sql` / `df.to_excel` / `df.to_parquet` for output

- `df.loc[mask, 'col']` for label-based selection/assignment

Functions

```
def get_gator_oscillator_signal(df: pd.DataFrame)
```

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `df: pd.DataFrame`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.

Step by step (top-level body)

1. `return 0`.

Code (copy-paste safe)

```
def get_gator_oscillator_signal(df: pd.DataFrame):  
    # Placeholder for Gator Oscillator signal logic  
    # Return 1 for buy, -1 for sell, 0 for neutral  
    return 0
```

Step 6.26 — Build trader_companion/signals/keltner_channel.py

What to do. Create the file `trader_companion/signals/keltner_channel.py` in your project root and paste in the code shown in this step. The file is **7** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from `requirements.txt`.

trader_companion/signals/keltner_channel.py

7 loc · 0 classes · 1 functions · 1 imports

Single-indicator module — see the signals folder note above. Most of the work is a pandas computation feeding off OHLC bars. It defines **1** module-level function, across **7** lines.

Python concepts demonstrated in this module

- Type hints

Imports — what each one is for

```
import pandas as pd
```

Why imported. DataFrame library — the workhorse for tabular data: loading CSV/Excel/SQL, joins, aggregations, time-series.

Used in this file. `pd.DataFrame`

Example. `df = pd.read_csv(path); df.groupby('symbol')['pnl'].sum()`

Other common scenarios.

- `df.merge(other, on='key')` joins like SQL
- `df.resample('1D').agg(...)` for time-series rebucketing
- `df.to_sql` / `df.to_excel` / `df.to_parquet` for output
- `df.loc[mask, 'col']` for label-based selection/assignment

Functions

```
def get_keltner_channel_signal(df: pd.DataFrame, period: int=20, atr_mult: float=2.0)
```

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `df: pd.DataFrame`, `period: int`, `atr_mult: float`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.

Step by step (top-level body)

1. `return 0`.

Code (copy-paste safe)

```
def get_keltner_channel_signal(df: pd.DataFrame, period: int = 20, atr_mult: float = 2.0):
    # Placeholder for Keltner Channel signal logic
    # Return 1 for buy, -1 for sell, 0 for neutral
    return 0
```

Step 6.27 — Build trader_companion/signals/price_channel.py

What to do. Create the file `trader_companion/signals/price_channel.py` in your project root and paste in the code shown in this step. The file is 7 lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from `requirements.txt`.

trader_companion/signals/price_channel.py

7 loc · 0 classes · 1 functions · 1 imports

Single-indicator module — see the signals folder note above. Most of the work is a pandas computation feeding off OHLC bars. It defines **1** module-level function, across **7** lines.

Python concepts demonstrated in this module

- Type hints

Imports — what each one is for

```
import pandas as pd
```

Why imported. DataFrame library — the workhorse for tabular data: loading CSV/Excel/SQL, joins, aggregations, time-series.

Used in this file. `pd.DataFrame`

Example. `df = pd.read_csv(path); df.groupby('symbol')['pnl'].sum()`

Other common scenarios.

- `df.merge(other, on='key')` joins like SQL
- `df.resample('1D').agg(...)` for time-series rebucketing
- `df.to_sql` / `df.to_excel` / `df.to_parquet` for output
- `df.loc[mask, 'col']` for label-based selection/assignment

Functions

```
def get_price_channel_signal(df: pd.DataFrame, period: int=20)
```

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `df: pd.DataFrame`, `period: int`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.

Step by step (top-level body)

1. `return 0`.

Code (copy-paste safe)

```
def get_price_channel_signal(df: pd.DataFrame, period: int = 20):
    # Placeholder for Price Channel signal logic
    # Return 1 for buy, -1 for sell, 0 for neutral
    return 0
```

Step 6.28 — Build trader_companion/signals/ultimate_oscillator.py

What to do. Create the file `trader_companion/signals/ultimate_oscillator.py` in your project root and paste in the code shown in this step. The file is **7** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from `requirements.txt`.

trader_companion/signals/ultimate_oscillator.py

7 loc · 0 classes · 1 functions · 1 imports

Single-indicator module — see the signals folder note above. Most of the work is a pandas computation feeding off OHLC bars. It defines **1** module-level function, across **7** lines.

Python concepts demonstrated in this module

- Type hints

Imports — what each one is for

```
import pandas as pd
```

Why imported. DataFrame library — the workhorse for tabular data: loading CSV/Excel/SQL, joins, aggregations, time-series.

Used in this file. `pd.DataFrame`

Example. `df = pd.read_csv(path); df.groupby('symbol')['pnl'].sum()`

Other common scenarios.

- `df.merge(other, on='key')` joins like SQL
- `df.resample('1D').agg(...)` for time-series rebucketing
- `df.to_sql` / `df.to_excel` / `df.to_parquet` for output
- `df.loc[mask, 'col']` for label-based selection/assignment

Functions

```
def get_ultimate_oscillator_signal(df: pd.DataFrame)
```

Why these Python concepts were used

• **type-annotated parameters** — Parameters carry type hints (e.g., `df: pd.DataFrame`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.

Step by step (top-level body)

1. `return 0.`

Code (copy-paste safe)

```
def get_ultimate_oscillator_signal(df: pd.DataFrame):
    # Placeholder for Ultimate Oscillator signal logic
    # Return 1 for buy, -1 for sell, 0 for neutral
    return 0
```

Step 6.29 — Build trader_companion/signals/vortex.py

What to do. Create the file `trader_companion/signals/vortex.py` in your project root and paste in the code shown in this step. The file is **7** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from `requirements.txt`.

trader_companion/signals/vortex.py

7 loc · 0 classes · 1 functions · 1 imports

Single-indicator module — see the `signals` folder note above. Most of the work is a pandas computation feeding off OHLC bars. It defines **1** module-level function, across **7** lines.

Python concepts demonstrated in this module

- Type hints

Imports — what each one is for

```
import pandas as pd
```

Why imported. DataFrame library — the workhorse for tabular data: loading CSV/Excel/SQL, joins, aggregations, time-series.

Used in this file. `pd.DataFrame`

Example. `df = pd.read_csv(path); df.groupby('symbol')['pnl'].sum()`

Other common scenarios.

- `df.merge(other, on='key')` joins like SQL
- `df.resample('1D').agg(...)` for time-series rebucketing
- `df.to_sql` / `df.to_excel` / `df.to_parquet` for output
- `df.loc[mask, 'col']` for label-based selection/assignment

Functions

```
def get_vortex_signal(df: pd.DataFrame, period: int=14)
```

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `df: pd.DataFrame, period: int`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.

Step by step (top-level body)

1. `return 0`.

Code (copy-paste safe)

```
def get_vortex_signal(df: pd.DataFrame, period: int = 14):
    # Placeholder for Vortex Indicator signal logic
    # Return 1 for buy, -1 for sell, 0 for neutral
    return 0
```

Step 6.30 — Build `trader_companion/strategies/__init__.py`

What to do. Create the file `trader_companion/strategies/__init__.py` in your project root and paste in the code shown in this step. The file is **2** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from `requirements.txt`.

`trader_companion/strategies/__init__.py`

2 loc · 0 classes · 0 functions · 0 imports

Package marker. Importing this folder as a module runs the code here (often empty, sometimes used to re-export public names). across **2** lines.

Step 6.31 — Build `trader_companion/strategies/rsi_overbought_oversold.py`

What to do. Create the file `trader_companion/strategies/rsi_overbought_oversold.py` in your project root and paste in the code shown in this step. The file is **22** lines long; we walk through it piece by piece below.

Depends on. This file imports from internal modules built in earlier steps: `signals`. If any of those imports fail, jump back to the step that introduced them.

`trader_companion/strategies/rsi_overbought_oversold.py`

22 loc · 0 classes · 1 functions · 1 imports

It defines **1** module-level function, across **22** lines.

Imports — what each one is for

```
from signals.rsi import get_rsi_signal
```

Why imported. Sub-package of `trader_companion` — one technical indicator per file.

Used in this file. `get_rsi_signal`

Functions

```
def rsi_overbought_oversold_strategy(symbol, timeframe, rsi_period=14, rsi_overbought=70, rsi_oversold=30,
    risk_percent=2.0, return_value=False)
```

RSI Overbought/Oversold strategy logic. User-editable parameters: `rsi_period`: int - RSI calculation period `rsi_overbought`: int/float - Overbought threshold `rsi_oversold`: int/float - Oversold threshold `risk_percent`: float - Risk per trade (not used in signal, but available for position sizing) Returns: 'buy', 'sell', or None (or tuple with RSI value if `return_value=True`)

Step by step (top-level body)

1. `return get_rsi_signal(symbol=symbol, timeframe=timeframe, period=rsi_perio....`

Code (copy-paste safe)

```
def rsi_overbought_oversold_strategy(symbol, timeframe, rsi_period=14, rsi_overbought=70, rsi_oversold=30,
    risk_percent=2.0, return_value=False):
    """
    RSI Overbought/Oversold strategy logic.
    User-editable parameters:
        rsi_period: int - RSI calculation period
        rsi_overbought: int/float - Overbought threshold
        rsi_oversold: int/float - Oversold threshold
        risk_percent: float - Risk per trade (not used in signal, but available for position sizing)
    Returns:
        'buy', 'sell', or None (or tuple with RSI value if return_value=True)
    """
    return get_rsi_signal(
        symbol=symbol,
        timeframe=timeframe,
        period=rsi_period,
        overbought=rsi_overbought,
        oversold=rsi_oversold,
        return_value=return_value
    )
```

Checkpoint — what works now. Indicator math works in isolation. Feed any indicator function a small DataFrame of OHLC bars and inspect the output Series. The strategy in `strategies/rsi_overbought_oversold.py` can now decide buy/sell on synthetic data — but it has nothing to send the decision to yet. That is the next phase.

Chapter 7 — Phase 5 — The trading engine

Now we wire the signals and the MT5 connector together. `mt5_trading.py` sends orders and reads positions; `mt5_comment_parser.py` tags those orders with structured strategy metadata; `trade_limit_manager.py` enforces the daily loss/position-size guardrails; and `hedge_protector.py` opens offsetting positions on a hedge account so that drawdown on the funded leg is bounded. This is the heart of the system.

Files we build in this phase, in order:

- `trader_companion/mt5_trading.py`
- `trader_companion/mt5_comment_parser.py`
- `trader_companion/trade_limit_manager.py`
- `trader_companion/hedge_protector.py`

Python concepts you'll meet in this phase

- Dunder methods (4) · Classes and objects (4) · f-strings (4) · Exceptions (4) · Lambdas (3)
- Comprehensions (2) · Context managers (with-statements) (2) · Properties (2)

Each is explained in Chapter 2 (the primer). The number in parentheses is how many of this phase's files use that concept.

Step 7.1 — Build `trader_companion/mt5_trading.py`

What to do. Create the file `trader_companion/mt5_trading.py` in your project root and paste in the code shown in this step. The file is **2485** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from `requirements.txt`.

What this file is for. All MetaTrader 5 trading helpers: order submission, position lookup, fill reconciliation. Wraps the official MetaTrader5 Python package.

`trader_companion/mt5_trading.py`

2485 loc · 1 classes · 2 functions · 12 imports

All MetaTrader 5 trading helpers: order submission, position lookup, fill reconciliation. Wraps the official MetaTrader5 Python package. It defines **1** class and **2** module-level functions, across **2,485** lines.

Python concepts demonstrated in this module

• Classes and objects • Comprehensions • Context managers (with-statements) • Dunder methods • Exceptions • f-strings • Lambdas

Imports — what each one is for

```
import sys
```

Why imported. Access to the running interpreter — argv, stdin/stdout, exit codes, the import search path, and the platform name.

Used in this file. sys._MEIPASS

Example. sys.exit(1) # terminate the process with a non-zero status

Other common scenarios.

- sys.argv[1:] reads the command-line arguments
- sys.path.insert(0, 'extra') prepends to the import search path
- sys.stderr.write(msg) writes to standard error
- sys.platform tells you 'win32', 'linux', or 'darwin'
- sys.modules is the global cache of already-imported modules

```
import os
```

Why imported. Operating-system interface. Path manipulation, environment variables, working-directory operations, exit codes, and thin wrappers around POSIX/Windows syscalls.

Used in this file. os.add_dll_directory, os.environ, os.environ.get, os.getenv, os.path, os.path.abspath, os.path.basename, os.path.dirname

Example. os.path.join(root, 'subdir', 'file.txt') # joins with the OS-correct separator

Other common scenarios.

- os.environ.get('API_KEY') to read environment variables
- os.makedirs(path, exist_ok=True) to create nested directories
- os.listdir(path) to list a directory's contents
- os.remove(path) / os.rename(src, dst) to delete or move a file
- os.getcwd() / os.chdir(path) to inspect or change the working dir

```
import winreg
```

Why imported. Read and write the Windows registry — used to discover installed MetaTrader terminals.

Used in this file. winreg.EnumKey, winreg.HKEY_CURRENT_USER, winreg.HKEY_LOCAL_MACHINE, winreg.OpenKey, winreg.QueryValueEx

Example. winreg.OpenKey(HKEY_LOCAL_MACHINE, r'Software\\X')

Other common scenarios.

- QueryValueEx returns a (value, type) tuple
- EnumKey / EnumValue iterate sub-keys

```
from pathlib import Path
```

Why imported. Object-oriented filesystem paths. Replaces most uses of os.path with chainable methods and operator overloading.

Used in this file. *No direct attribute access detected — likely imported for side effects, re-export, or string references.*

Example. Path('logs') / f'{name}.log' # / is overloaded to join

Other common scenarios.

- p.exists(), p.is_file(), p.is_dir() for predicates
- p.read_text() / p.write_text(content) for one-shot file I/O
- p.glob('**/*.py') to walk a tree with a pattern
- p.with_suffix('.bak') to derive a sibling path

```
from dotenv import load_dotenv
```

Why imported. python-dotenv — load .env files into os.environ for twelve-factor configuration in development.

Used in this file. load_dotenv

Example. load_dotenv() # call once at process start

Other common scenarios.

- dotenv_values('.env') returns a dict without polluting os.environ
- Use a real secret store in production (vault, AWS SM, etc.)

```
import logging
```

Why imported. Structured, levelled logging. Replaces print() with filterable, formattable output that can route to files, stderr, syslog, or HTTP endpoints.

Used in this file. logging.INFO, logging.basicConfig, logging.debug, logging.error, logging.exception, logging.info, logging.warning

Example. logging.getLogger(__name__).info('connected to %s', host)

Other common scenarios.

- .debug() / .info() / .warning() / .error() / .exception()
- logger.exception() captures the current traceback automatically
- logging.basicConfig(level=logging.INFO) for quick setup
- Handlers and Formatters route logs to files / streams / syslog

```
import subprocess
```

Why imported. Run external programs. The standard way to spawn a child process, capture its output, and wait for its exit code.

Used in this file. subprocess.run

Example. subprocess.run(['git', 'rev-parse', 'HEAD'], capture_output=True, text=True, check=True)

Other common scenarios.

- Popen(...) for streaming I/O while the process is alive
- check_output(cmd) returns stdout as text and raises on failure
- DEVNULL / PIPE redirect stdin/stdout/stderr
- shell=True interprets the string with the system shell (security risk)

import psutil

Why imported. Cross-platform process and system information — find a running terminal, check CPU/memory, kill a stale process.

Used in this file. psutil.AccessDenied, psutil.NoSuchProcess, psutil.ZombieProcess, psutil.disk_partitions, psutil.process_iter

Example. psutil.process_iter(['name']) # iterate live processes

Other common scenarios.

- psutil.Process(pid).terminate() / .kill()
- psutil.cpu_percent(interval=1.0) measures CPU usage
- psutil.virtual_memory() returns RAM stats

import time

Why imported. Timestamps and sleep. Lower-level than datetime — works in Unix-epoch seconds.

Used in this file. time.sleep, time.time

Example. time.sleep(1.5) # pause this thread for 1.5 seconds

Other common scenarios.

- time.time() returns seconds since the epoch
- time.monotonic() for measuring elapsed time (never goes backward)
- time.perf_counter() for high-precision benchmarking
- time.strptime / time.strftime mirror the datetime versions

from time import sleep

Why imported. Timestamps and sleep. Lower-level than datetime — works in Unix-epoch seconds.

Used in this file. sleep

Example. time.sleep(1.5) # pause this thread for 1.5 seconds

Other common scenarios.

- time.time() returns seconds since the epoch
- time.monotonic() for measuring elapsed time (never goes backward)
- time.perf_counter() for high-precision benchmarking

- `time.strptime / time.strptime` mirror the `datetime` versions

```
import ctypes
```

Why imported. Call C functions from shared libraries (DLLs / `.so` / `.dylib`). Used here to reach Win32 APIs the `stdlib` doesn't expose.

Used in this file. *No direct attribute access detected — likely imported for side effects, re-export, or string references.*

Example. `ctypes.windll.user32.MessageBoxW(0, 'hi', 't', 0)`

Other common scenarios.

- `CDLL('libfoo.so')` loads a Unix shared object
- Define `argtypes/restype` to get type-checked calls
- `c_int / c_char_p` / Structure mirror C type layouts

```
from ctypes import wintypes
```

Why imported. Call C functions from shared libraries (DLLs / `.so` / `.dylib`). Used here to reach Win32 APIs the `stdlib` doesn't expose.

Used in this file. *No direct attribute access detected — likely imported for side effects, re-export, or string references.*

Example. `ctypes.windll.user32.MessageBoxW(0, 'hi', 't', 0)`

Other common scenarios.

- `CDLL('libfoo.so')` loads a Unix shared object
- Define `argtypes/restype` to get type-checked calls
- `c_int / c_char_p` / Structure mirror C type layouts

Classes

```
class MT5API
    _symbol_cache = {}
    _symbol_cache_timestamp = {}
    _cache_ttl = 300
```

Code (copy-paste safe)

```
class MT5API:
    # Class-level symbol cache to persist across instances
    _symbol_cache = {}
    _symbol_cache_timestamp = {}
    _cache_ttl = 300 # Cache for 5 minutes

    def __init__(self, login, password, server, symbol=None, terminal_path=None):
        # Safely convert login to integer
        try:
            self.login = int(str(login).strip()) if login else 0
        except (ValueError, TypeError):
            logging.error(f'Invalid login format: {login}')
            self.login = 0
```

```

self.password = str(password) if password else ""
self.server = str(server) if server else ""
self.symbol = symbol
self.terminal_path = terminal_path
self.sl_points = float(os.getenv('MT5_SL_POINTS') or os.getenv('MT5_STOPLOSS_POINTS', '0'))
self.tp_points = float(os.getenv('MT5_TP_POINTS') or os.getenv('MT5_TAKEPROFIT_POINTS', '0'))
self.default_volume = float(os.getenv('MT5_VOLUME', '1'))

# Rollover safety tracking - prevents multiple executions per day
self.rollover_executed_today = {} # {prop_firm: date_string}

# Store the actually connected symbol (will be set after successful connection)
self.connected_symbol = None

# Check if this is a PlexyTrade server (case-insensitive substring match)
self.is_plexy_server = "plexy" in server.lower() if server else False
if self.is_plexy_server:
    logging.info(f"PlexyTrade server detected: {server} - Lot sizes will be divided by 20")

# Ensure all instance variables are properly initialized
self.connected = False
self.last_error = None

def _get_cached_terminal_path(self):
    """Get previously successful terminal path for faster connection"""
    import tempfile
    cache_file = os.path.join(tempfile.gettempdir(), "mt5_terminal_cache.txt")
    try:
        if os.path.exists(cache_file):
            with open(cache_file, 'r') as f:
                cached_path = f.read().strip()
                if os.path.exists(cached_path):
                    return cached_path
    except Exception as e:
        logging.debug(f"Cache read failed: {e}")
    return None

def _cache_successful_path(self, path):
    """Cache successful terminal path for future use"""
    import tempfile
    cache_file = os.path.join(tempfile.gettempdir(), "mt5_terminal_cache.txt")
    try:
        with open(cache_file, 'w') as f:
            f.write(path)
        logging.info(f"[OK] Cached successful MT5 path: {path}")
    except Exception as e:
        logging.debug(f"Cache write failed: {e}")

def connect(self):
    # SPEED OPTIMIZATION: Try cached terminal path first
    success = False
    cached_path = self._get_cached_terminal_path()
    if cached_path:
        if mt5.initialize(path=cached_path):
            logging.info(f'[FAST] MT5 initialized with cached path: {cached_path}')
            success = True
        else:
            logging.info(f'Cached path failed, trying alternatives: {cached_path}')

    # Try to initialize MT5 with the best available path

```

```

if not success:
    # CRITICAL FIX: If user specifies a terminal path, ONLY use that terminal
    if self.terminal_path and self.terminal_path.strip():
        terminal_exe = os.path.join(self.terminal_path, "terminal64.exe")
        if os.path.exists(terminal_exe):
            if mt5.initialize(path=terminal_exe):
                logging.info(f'MT5 initialized successfully with user-specified path: {terminal_exe}')
                self._cache_successful_path(terminal_exe) # Cache success
                success = True
            else:
                logging.error(f'MT5 initialize failed for user-specified path: {terminal_exe}')
                # Don't try other terminals when user specified one - respect their choice
                error_code, error_msg = mt5.last_error()
                self.last_error = f'MT5 initialize failed for selected terminal: {error_msg} (Code: {error_code})'
                logging.error(self.last_error)
                return False
        else:
            logging.error(f'User-specified terminal executable not found: {terminal_exe}')
            self.last_error = f'Terminal executable not found: {terminal_exe}'
            return False

# If specific paths failed, try other available installations
if not success:
    terminals = get_installed_mt5_terminals()
    for terminal in terminals:
        terminal_exe = os.path.join(terminal["path"], "terminal64.exe")
        if os.path.exists(terminal_exe):
            if mt5.initialize(path=terminal_exe):
                logging.info(f'MT5 initialized successfully with detected path: {terminal_exe}')
                self._cache_successful_path(terminal_exe) # Cache success
                success = True
                break
            else:
                logging.warning(f'MT5 initialize failed for detected path: {terminal_exe}')

# Last resort: try default initialization
if not success:
    if mt5.initialize():
        logging.info('MT5 initialized successfully with default path')
        success = True
    else:
        logging.error('MT5 initialize failed with default path')

if not success:
    error_code, error_msg = mt5.last_error()
    self.last_error = f'MT5 initialize failed: {error_msg} (Code: {error_code})'
    logging.error(self.last_error)
    return False

authorized = mt5.login(self.login, self.password, self.server)
if not authorized:
    error_msg = f'MT5 login failed for login={self.login}, server={self.server}'
    logging.error(error_msg)
    self.last_error = error_msg
    return False

# SPEED OPTIMIZATION: Fast symbol detection after successful connection
try:
    # Quick symbol detection - try user symbol first
    if self.symbol and mt5.symbol_select(self.symbol, True):

```

```

        self.connected_symbol = self.symbol
        logging.info(f"[FAST] Fast symbol detection: {self.connected_symbol}")
    else:
        # Quick fallback to first available symbol
        symbols = mt5.symbols_get()
        if symbols and len(symbols) > 0:
            self.connected_symbol = symbols[0].name
            if mt5.symbol_select(self.connected_symbol, True):
                logging.info(f"[FAST] Fast fallback symbol: {self.connected_symbol}")
            else:
                # Last resort: use EURUSD as default
                self.connected_symbol = "EURUSD"
                logging.info(f"[FAST] Default symbol: {self.connected_symbol}")
        else:
            self.connected_symbol = "EURUSD" # Safe default

except Exception as e:
    logging.warning(f"Fast symbol detection failed: {e}")
    self.connected_symbol = self.symbol if self.symbol else "EURUSD"

self.connected = True
return True

# After successful connection, detect and store the connected symbol
try:
    # Try to get the current symbol from the market watch or from a symbol list
    # First try to get symbols from the market watch
    symbols = mt5.symbols_get()
    if symbols and len(symbols) > 0:
        # Use the first available symbol from market watch as the connected symbol
        self.connected_symbol = symbols[0].name
        logging.info(f"Connected symbol detected: {self.connected_symbol}")

        # Try to select this symbol to ensure it's available
        if not mt5.symbol_select(self.connected_symbol, True):
            logging.warning(f"Could not select detected symbol: {self.connected_symbol}")
            # Try to find an alternative symbol
            for symbol in symbols[:5]: # Try first 5 symbols
                if mt5.symbol_select(symbol.name, True):
                    self.connected_symbol = symbol.name
                    logging.info(f"Alternative connected symbol selected: {self.connected_symbol}")
                    break
    else:
        # Fallback: try common symbol names if no symbols available
        common_symbols = ["EURUSD", "GBPUSD", "USDJPY", "USDCHF", "AUDUSD", "NZDUSD", "USDCAD"]
        for symbol in common_symbols:
            if mt5.symbol_select(symbol, True):
                self.connected_symbol = symbol
                logging.info(f"Fallback connected symbol selected: {self.connected_symbol}")
                break

except Exception as e:
    logging.warning(f"Could not detect connected symbol: {e}")
    # If detection fails, use the configured symbol if available
    if self.symbol:
        self.connected_symbol = self.symbol

return authorized

def monitor_connection(self):

```

```

"""
Monitor MT5 connection status and attempt recovery if needed
Call this periodically (e.g., every 30 seconds) during application operation
"""
try:
    # Quick connection check
    terminal_info = mt5.terminal_info()
    if not terminal_info or not terminal_info.connected:
        logging.warning("■ MT5 connection monitor detected disconnection")
        return self.attempt_reconnection()

    # Check if trading is still allowed
    if not terminal_info.trade_allowed:
        logging.warning("■ MT5 connection monitor detected trading disabled")
        return False

    # Check account access
    account_info = mt5.account_info()
    if not account_info:
        logging.warning("■ MT5 connection monitor detected account access issues")
        return self.attempt_reconnection()

    return True

except Exception as e:
    logging.error(f"Error in connection monitor: {e}")
    return False
"""

Ensure MT5 session is properly initialized and maintained
Call this at application startup and periodically during operation
"""
try:
    logging.info("[SETUP] Ensuring MT5 session integrity...")

    # Check if MT5 is initialized
    if not mt5.initialize():
        logging.warning("MT5 not initialized, attempting to initialize...")
        if not mt5.initialize():
            logging.error("[ERROR] Failed to initialize MT5")
            return False

    # Check if we're logged in
    if not mt5.terminal_info():
        logging.warning("MT5 terminal info not available, attempting connection...")
        if not self.connect():
            logging.error("[ERROR] Failed to connect to MT5")
            return False

    # Perform comprehensive health check
    is_healthy, health_msg = self.check_connection_health()
    if not is_healthy:
        logging.warning(f"MT5 health check failed: {health_msg}, attempting recovery...")
        if not self.attempt_reconnection():
            logging.error("[ERROR] MT5 recovery failed")
            return False

    # Ensure symbol is properly selected
    if self.symbol:
        symbol_info = mt5.symbol_info(self.symbol)
        if symbol_info and not symbol_info.visible:

```

```

        logging.info(f"Ensuring symbol {self.symbol} is selected...")
        mt5.symbol_select(self.symbol, True)

    logging.info("[OK] MT5 session integrity confirmed")
    return True

except Exception as e:
    logging.error(f"Error ensuring MT5 session: {e}")
    return False
"""
Attempt to reconnect to MT5 if connection is lost
"""
for attempt in range(max_retries):
    try:
        logging.info(f"■ Attempting MT5 reconnection (attempt {attempt + 1}/{max_retries})...")

        # Shutdown current connection
        mt5.shutdown()

        # Wait a moment
        time.sleep(1)

        # Try to reconnect
        if self.connect():
            # Verify the reconnection worked
            is_healthy, health_msg = self.check_connection_health()
            if is_healthy:
                logging.info("[OK] MT5 reconnection successful")
                return True
            else:
                logging.warning(f"Reconnection completed but health check failed: {health_msg}")
        else:
            logging.warning(f"Reconnection attempt {attempt + 1} failed")

    except Exception as e:
        logging.error(f"Error during reconnection attempt {attempt + 1}: {e}")

logging.error(f"[ERROR] All {max_retries} reconnection attempts failed")
return False
"""
Comprehensive health check for MT5 connection and trading readiness
Returns (is_healthy, error_message)
"""
try:
    logging.info(f"■ Performing MT5 connection health check...")

    # 1. Check MT5 initialization
    if not mt5.initialize():
        return False, "MT5 not initialized"

    # 2. Check terminal connection
    terminal_info = mt5.terminal_info()
    if not terminal_info:
        return False, "Cannot get terminal info"

    if not terminal_info.connected:
        return False, "MT5 terminal not connected"

    if not terminal_info.trade_allowed:
        return False, "Trading not allowed in MT5 terminal"

```

```

# 3. Check account access
account_info = mt5.account_info()
if not account_info:
    return False, "Cannot access account info"

# 4. Check symbol availability (using configured symbol)
if self.symbol:
    symbol_info = mt5.symbol_info(self.symbol)
    if not symbol_info:
        return False, f"Symbol {self.symbol} not found in MT5"

    if not symbol_info.visible:
        return False, f"Symbol {self.symbol} not visible in Market Watch"

# 5. Check tick data availability
tick = mt5.symbol_info_tick(self.symbol)
if not tick or tick.bid <= 0 or tick.ask <= 0:
    return False, f"No live tick data for symbol {self.symbol}"

logging.info("[OK] MT5 connection health check passed")
return True, "All systems operational"

except Exception as e:
    error_msg = f"Health check failed: {e}"
    logging.error(f"[ERROR] {error_msg}")
    return False, error_msg
"""
Verify MT5 connection is stable and ready for trading
"""
try:
    # Check terminal info
    terminal_info = mt5.terminal_info()
    if not terminal_info:
        logging.error("MT5 terminal_info() returned None")
        return False

    if not terminal_info.connected:
        logging.error("MT5 terminal not connected")
        return False

    if not terminal_info.trade_allowed:
        logging.error("MT5 trading not allowed")
        return False

    # Check account info
    account_info = mt5.account_info()
    if not account_info:
        logging.error("MT5 account_info() returned None")
        return False

    logging.info(
        "[OK] MT5 connection verified - terminal connected, trading allowed, account accessible"
    )
    return True

except Exception as e:
    logging.error(f"Error verifying MT5 connection: {e}")
    return False

def _verify_symbol_tick_data(self, symbol, max_retries=3):
    """

```

```

Verify symbol has live tick data available
"""
for attempt in range(max_retries):
    try:
        tick = mt5.symbol_info_tick(symbol)
        if tick and tick.bid > 0 and tick.ask > 0:
            logging.info(f"[OK] Tick data verified for {symbol}: bid={tick.bid}, ask={tick.ask}")
            return True

        if attempt < max_retries - 1:
            logging.warning(
                f"Tick data not available for {symbol} (attempt {attempt + 1}/{max_retries}), retrying"
            )
            time.sleep(0.1) # Brief delay before retry

    except Exception as e:
        logging.error(f"Error getting tick data for {symbol}: {e}")
        if attempt < max_retries - 1:
            time.sleep(0.1)

logging.error(f"[ERROR] No tick data available for {symbol} after {max_retries} attempts")
return False

def is_autotrading_enabled(self):
    """
    Check if AutoTrading is enabled in MT5 using the existing connection
    Returns True if autotrading is enabled, False otherwise
    """
    try:
        # Don't initialize a new connection - use the existing one
        if not self.connected:
            print("[ERROR] MT5 not connected - cannot check AutoTrading status")
            return False

        # Use the already connected MT5 instance to check terminal info
        term_info = mt5.terminal_info()
        if not term_info:
            print("[ERROR] Could not get terminal info from existing MT5 connection")
            return False

        # Check AutoTrading status using the connected instance
        print("■ Checking AutoTrading status on existing MT5 connection...")

        # Basic status checks
        connected = getattr(term_info, 'connected', False)
        trade_allowed = getattr(term_info, 'trade_allowed', False)
        tradeapi_disabled = getattr(term_info, 'tradeapi_disabled', True)
        dlls_allowed = getattr(term_info, 'dlls_allowed', False)

        # Log the current status
        print(f"[CHECK] Connected: {connected}")
        print(f"[CHECK] Trade Allowed: {trade_allowed}")
        print(f"[CHECK] Trade API Disabled: {tradeapi_disabled}")
        print(f"[CHECK] DLLs Allowed: {dlls_allowed}")

        # AutoTrading is enabled if:
        # 1. MT5 is connected
        # 2. Trade is allowed (main AutoTrading setting)
        # 3. Trade API is not disabled
        autotrading_enabled = connected and trade_allowed and not tradeapi_disabled

        print(f"[RESULT] AutoTrading enabled: {autotrading_enabled}")

```



```

        return autotrading_enabled

    except Exception as e:
        print(f"[ERROR] AutoTrading status check failed: {e}")
        logging.error(f"Error checking auto trading status: {e}")
        return False

def ensure_symbol(self, symbol):
    """Ensure symbol is available for trading, with enhanced caching and fast paths"""
    try:
        # Validate input symbol
        if not symbol or symbol.strip() == "":
            logging.error(f"Invalid symbol provided: '{symbol}'")
            raise Exception(f"Invalid symbol provided: '{symbol}'")

        symbol = symbol.strip()

        # SPEED OPTIMIZATION: Check symbol cache first
        cache_key = f"{self.server}_{symbol}"
        current_time = time.time()

        if (cache_key in self._symbol_cache and
            cache_key in self._symbol_cache_timestamp and
            current_time - self._symbol_cache_timestamp[cache_key] < self._cache_ttl):

            cached_symbol = self._symbol_cache[cache_key]
            logging.info(f"[FAST] SPEED: Using cached symbol {symbol} → {cached_symbol}")
            return cached_symbol

        # First check if MT5 is connected
        if not mt5.terminal_info():
            logging.error("MT5 terminal not connected")
            # CRITICAL: Don't return symbol if MT5 is not connected - this causes trading failures
            logging.error("Cannot validate symbol - MT5 terminal not connected")
            return None

        # PRIORITY: Since users provide correct symbol names, try their symbol first
        logging.info(f"[TARGET] USER SYMBOL: Trying user-provided symbol '{symbol}' first")
        symbol_info = mt5.symbol_info(symbol)
        if symbol_info:
            tick = mt5.symbol_info_tick(symbol)
            if tick and (tick.bid > 0 or tick.ask > 0):
                logging.info(f"[OK] USER SYMBOL WORKS: '{symbol}' has active tick data")
                # Cache the successful result
                self._symbol_cache[cache_key] = symbol
                self._symbol_cache_timestamp[cache_key] = current_time
                return symbol
            else:
                # Symbol exists but no tick data - CRITICAL: Don't proceed with trading
                logging.error(f"[ERROR] Symbol {symbol} exists but no tick data available - cannot tr
                return None

        # Try to select the symbol if it wasn't found
        logging.info(f"[SIGNAL] SELECTING SYMBOL: Attempting to activate '{symbol}'")
        select_result = mt5.symbol_select(symbol, True)

        # Check again after selection
        symbol_info = mt5.symbol_info(symbol)
        if symbol_info:
            tick = mt5.symbol_info_tick(symbol)
            if tick and (tick.bid > 0 or tick.ask > 0):

```

```

        logging.info(f"[OK] SYMBOL ACTIVATED: '{symbol}' now has active tick data")
        self._symbol_cache[cache_key] = symbol
        self._symbol_cache_timestamp[cache_key] = current_time
        return symbol
    else:
        # Symbol selected but no tick - still return for trading attempt
        logging.info(f"[WARNING] SYMBOL SELECTED: '{symbol}' activated but no tick data yet")
        self._symbol_cache[cache_key] = symbol
        self._symbol_cache_timestamp[cache_key] = current_time
        return symbol

# SPEED OPTIMIZATION: For known symbols, try direct approach first
if symbol.upper() in ['USTECH', 'USTEC', 'XAUUSD', 'NAS100', 'NASDAQ']:
    # Try the symbol directly first (fastest path)
    symbol_info = mt5.symbol_info(symbol)
    if symbol_info:
        tick = mt5.symbol_info_tick(symbol)
        if tick and (tick.bid > 0 or tick.ask > 0):
            logging.info(f"[FAST] SPEED: Direct symbol access successful for {symbol}")
            # Cache the successful result
            self._symbol_cache[cache_key] = symbol
            self._symbol_cache_timestamp[cache_key] = current_time
            return symbol

# Get symbol variations to try (only if direct access failed)
symbol_variations = self._get_symbol_variations(symbol)
logging.info(f"[SEARCH] VARIATIONS: Trying {len(symbol_variations)} variations for '{symbol}'")

# SPEED OPTIMIZATION: Try most likely variations first
priority_variations = []
other_variations = []

for variation in symbol_variations:
    # Prioritize exact matches and simple variations
    if (variation == symbol or
        variation == symbol.upper() or
        variation in ['USTECH', 'USTEC', 'XAUUSD']):
        priority_variations.append(variation)
    else:
        other_variations.append(variation)

# Try priority variations first, then others
all_variations = priority_variations + other_variations[:20] # Limit to first 20 of others

for variation in all_variations:
    try:
        # First check if this variation has symbol info
        var_info = mt5.symbol_info(variation)
        if not var_info:
            logging.debug(f"Symbol variation {variation} not found")
            continue

        # Check if it already has tick data (means it's working)
        tick = mt5.symbol_info_tick(variation)
        if tick and (tick.bid > 0 or tick.ask > 0):
            logging.info(
                f"[FAST] SPEED: Symbol variation {variation} already has active tick data")
            self._symbol_cache[cache_key] = variation
            self._symbol_cache_timestamp[cache_key] = current_time
            return variation

```

```

        # Try to select the symbol (but don't fail if this returns False)
        # Some brokers return False even when symbol is already available
        select_result = mt5.symbol_select(variation, True)
        logging.debug(f"Symbol select result for {variation}: {select_result}")

        # After selection attempt, check again for tick data
        tick_after = mt5.symbol_info_tick(variation)
        if tick_after and (tick_after.bid > 0 or tick_after.ask > 0):
            logging.info(f"[OK] VARIATION SUCCESS: '{variation}' now has active tick data")
            self._symbol_cache[cache_key] = variation
            self._symbol_cache_timestamp[cache_key] = current_time
            return variation

        # If still no tick data, but symbol info exists, it might still work for some operations
        if var_info and var_info.visible:
            logging.info(
                f"[WARNING] VARIATION VISIBLE: '{variation}' is visible but no current tick data"
            )
            self._symbol_cache[cache_key] = variation
            self._symbol_cache_timestamp[cache_key] = current_time
            return variation

    except Exception as e:
        logging.debug(f"Error trying symbol variation {variation}: {e}")
        continue

    # CRITICAL: Don't return symbol as fallback if no valid symbol was found
    logging.error(f"[FAILED] No valid symbol found for {symbol} - cannot proceed with trading")
    return None

except Exception as e:
    logging.error(f"Error in ensure_symbol for '{symbol}': {e}")

    # CRITICAL: Always return user's symbol to prevent None errors
    # Users are expected to provide correct symbol names for their broker
    logging.warning(f"■ EMERGENCY FALLBACK: Returning user symbol '{symbol}' despite errors")
    return symbol

def _get_symbol_variations(self, symbol):
    """Get possible symbol variations for different MT5 brokers"""
    # Start with the original symbol
    variations = [symbol]

    # Add common variations based on symbol type
    symbol_upper = symbol.upper()

    # NASDAQ variations - comprehensive list based on actual MT5 charts
    if symbol_upper in ['USTEC', 'NASDAQ', 'NQ', 'NAS', 'USTECH100', 'USTECH', 'NAS100', 'NDX',
                        'NASDAQ100', 'TECH100', 'US100', 'SPX500']:
        variations.extend([
            # Primary NASDAQ symbols
            'USTEC', 'USTECH100', 'USTECH', 'NAS100', 'NASDAQ', 'NQ', 'NDX', 'NASDAQ100',
            'US100', 'TECH100', 'USTEC100', 'NASTECH', 'NASDAQTECH',

            # Suffixed variations (.m, m, -Z, etc.)
            'USTEC.m', 'USTECH100.m', 'USTECH.m', 'NAS100.m', 'NASDAQ.m', 'NQ.m', 'NDX.m',
            'USTECm', 'USTECH100m', 'USTECHm', 'NAS100m', 'NASDAQm', 'NQm', 'NDXm',
            'USTEC-Z', 'USTECH100-Z', 'USTECH-Z', 'NAS100-Z', 'NASDAQ-Z', 'NQ-Z', 'NDX-Z',

            # Broker-specific variations
            'USTECfx', 'USTECH100fx', 'USTECHfx', 'NAS100fx', 'NASDAQfx',

```

```

        'USTEC_c', 'USTECH_c', 'NAS100_c', 'NASDAQ_c',
        'USTEC.c', 'USTECH.c', 'NAS100.c', 'NASDAQ.c',

        # Alternative naming patterns
        'US_TECH', 'US-TECH', 'USTECH.', 'USTEC.', 'NAS100.',
        'USTECH100.', 'NASDAQ100.', 'TECH-100', 'TECH_100',

        # Contract-specific variations (futures style)
        'USTECH2024', 'USTEC2024', 'NAS2024', 'USTECH24', 'USTEC24', 'NAS24',
        'USTECHM24', 'USTECM24', 'NASM24', 'USTECHZ24', 'USTECZ24', 'NASZ24',

        # Additional broker variations
        'USTEC100', 'NASTECH100', 'USNASDAQ', 'NASDAQ_100', 'NASDAQ-100',
        'USTEC_100', 'USTEC-100', 'USTECH_100', 'USTECH-100',

        # Dot variations
        'USTEC.', 'USTECH.', 'NASDAQ.', 'NAS100.', 'NQ.',

        # Undercore variations
        'USTEC_', 'USTECH_', 'NASDAQ_', 'NAS100_', 'NQ_'
    ])

# Gold variations - comprehensive list for different brokers
elif symbol_upper in ['XAUUSD', 'GOLD', 'GLD', 'XAU']:
    variations.extend([
        'XAUUSD', 'GOLD', 'XAU', 'GOLDUSD', 'XAUUSD.',
        'XAUUSD.m', 'GOLD.m', 'XAUUSDm', 'GOLDm',
        'XAUUSD-Z', 'GOLD-Z', 'XAU/USD', 'GOLD/USD',
        'XAUUSDfx', 'GOLDfx', 'XAUUSD_MT5'
    ])

# Oil variations
elif symbol_upper in ['USOIL', 'OIL', 'CRUDE']:
    variations.extend(['USOIL', 'CRUDE', 'OIL', 'WTI', 'BRENT'])

# Forex pairs - try both with and without suffixes
elif len(symbol) == 6 and symbol_upper.endswith('USD'):
    base_pair = symbol_upper[:6]
    variations.extend([base_pair, base_pair + '.', base_pair + 'm', base_pair + 'c'])

# Remove duplicates while preserving order
seen = set()
result = []
for variant in variations:
    if variant not in seen:
        seen.add(variant)
        result.append(variant)

return result

def _log_available_symbols(self, failed_symbol):
    """Log some available symbols for debugging"""
    try:
        # Get all available symbols
        symbols = mt5.symbols_get()
        if symbols:
            # Log total count
            logging.info(f"Failed symbol: {failed_symbol}")
            logging.info(f"Total available symbols: {len(symbols)}")

            # Log first 20 symbols

```

```

symbol_names = [s.name for s in symbols[:20]]
logging.info(f"Available symbols (first 20): {symbol_names}")

# Look for symbols containing parts of the failed symbol
failed_upper = failed_symbol.upper()
similar = []
for s in symbols:
    symbol_name = s.name.upper()
    # Check for partial matches
    if (failed_upper[:3] in symbol_name or
        symbol_name[:3] in failed_upper or
        'XAU' in symbol_name or
        'GOLD' in symbol_name or
        'USTEC' in symbol_name or
        'NAS' in symbol_name):
        similar.append(s.name)

if similar:
    logging.info(f"Similar/related symbols found: {similar[:10]}") # Show max 10

# Check specifically for gold and nasdaq symbols
gold_symbols = [s.name for s in symbols if any(x in s.name.upper() for x in ['XAU', 'GOLD'])]
nasdaq_symbols = [s.name for s in symbols if any(x in s.name.upper() for x in ['USTEC', 'NASDAQ', 'NDX'])]

if gold_symbols:
    logging.info(f"Gold-related symbols: {gold_symbols}")
if nasdaq_symbols:
    logging.info(f"NASDAQ-related symbols: {nasdaq_symbols}")

else:
    logging.warning("No symbols available - check MT5 connection")
except Exception as e:
    logging.warning(f"Could not retrieve available symbols: {e}")

def get_connected_symbol(self):
    """Get the symbol that was detected during connection"""
    return self.connected_symbol

def get_safe_symbol(self, fallback_symbol=None):
    """Get a safe symbol to use - prefers fallback (configured), then connected symbol, then configured symbol
    # Priority 1: Use the explicitly requested/configured symbol if provided and available
    if fallback_symbol:
        # Try to select the requested symbol to ensure it's available
        if mt5.symbol_select(fallback_symbol, True):
            return fallback_symbol
        else:
            logging.warning(
                f"Requested symbol {fallback_symbol} not available, falling back to connected symbol")
    # Priority 2: Use connected symbol if no specific symbol requested or if requested symbol unavailable
    if self.connected_symbol:
        return self.connected_symbol
    elif self.symbol:
        return self.symbol
    else:
        # Ultimate fallback
        return "EURUSD"

def get_supported_filling_modes(self, symbol):

```

```

info = mt5.symbol_info(symbol)
if info is None:
    return [mt5.ORDER_FILLING_IOC]
fillings = getattr(info, "trade_fillings", None)
if not fillings or len(fillings) == 0:
    return [mt5.ORDER_FILLING_IOC, mt5.ORDER_FILLING_FOK, mt5.ORDER_FILLING_RETURN]
return list(fillings)

def check_connection_health(self):
    """
    Comprehensive health check for MT5 connection and trading readiness
    Returns (is_healthy, error_message)
    """
    try:
        logging.info("■ Performing MT5 connection health check...")

        # 1. Check MT5 initialization
        if not mt5.initialize():
            return False, "MT5 not initialized"

        # 2. Check terminal connection
        terminal_info = mt5.terminal_info()
        if not terminal_info:
            return False, "Cannot get terminal info"

        if not terminal_info.connected:
            return False, "MT5 terminal not connected"

        if not terminal_info.trade_allowed:
            return False, "Trading not allowed in MT5 terminal"

        # 3. Check account access
        account_info = mt5.account_info()
        if not account_info:
            return False, "Cannot access account info"

        # 4. Check symbol availability (using configured symbol)
        if self.symbol:
            symbol_info = mt5.symbol_info(self.symbol)
            if not symbol_info:
                return False, f"Symbol {self.symbol} not found in MT5"

            if not symbol_info.visible:
                return False, f"Symbol {self.symbol} not visible in Market Watch"

        # 5. Check tick data availability
        tick = mt5.symbol_info_tick(self.symbol)
        if not tick or tick.bid <= 0 or tick.ask <= 0:
            return False, f"No live tick data for symbol {self.symbol}"

        logging.info("[OK] MT5 connection health check passed")
        return True, "All systems operational"

    except Exception as e:
        error_msg = f"Health check failed: {e}"
        logging.error(f"[ERROR] {error_msg}")
        return False, error_msg

def attempt_reconnection(self, max_retries=3):
    """

```

```

Attempt to reconnect to MT5 if connection is lost
"""
for attempt in range(max_retries):
    try:
        logging.info(f"■ Attempting MT5 reconnection (attempt {attempt + 1}/{max_retries})...")

        # Shutdown current connection
        mt5.shutdown()

        # Wait a moment
        time.sleep(1)

        # Try to reconnect
        if self.connect():
            # Verify the reconnection worked
            is_healthy, health_msg = self.check_connection_health()
            if is_healthy:
                logging.info("[OK] MT5 reconnection successful")
                return True
            else:
                logging.warning(f"Reconnection completed but health check failed: {health_msg}")
        else:
            logging.warning(f"Reconnection attempt {attempt + 1} failed")

    except Exception as e:
        logging.error(f"Error during reconnection attempt {attempt + 1}: {e}")

logging.error(f"[ERROR] All {max_retries} reconnection attempts failed")
return False

def _calculate_sl_tp_price(self, symbol, order_type, price, sl_points, tp_points):
    """Calculate SL and TP prices from points - NASDAQ automation always uses 1.0 point value"""
    sl_price = None
    tp_price = None

    # Get symbol info for logging purposes
    symbol_info = mt5.symbol_info(symbol)
    if not symbol_info:
        logging.error(f"Could not get symbol info for {symbol}")
        return sl_price, tp_price

    # Get the minimum tick size for reference
    point = symbol_info.point

    # NASDAQ automation: ALWAYS use 1.0 point value regardless of symbol name or tick size
    # EXCEPTION: For XAUUSD (Gold), use the actual point value (usually 0.01) as the blueprint points
    # are calibrated for standard ticks (e.g. 1710 points = $17.10 price movement)
    if any(x in symbol.upper() for x in ['XAU', 'GOLD', 'GC']):
        point_value = point
        logging.info(f"Gold symbol detected ({symbol}), using actual point value: {point_value}")
    else:
        point_value = 1.0
        logging.info(
            f"Symbol {symbol} - tick size: {point}, NASDAQ automation point value: {point_value}, pri

    if sl_points and float(sl_points) > 0:
        sl_points_float = float(sl_points)
        price_difference = sl_points_float * point_value

        if order_type == "buy":

```

```

        sl_price = price - price_difference
    else: # sell
        sl_price = price + price_difference

    logging.info(
        f"SL calculation: {sl_points_float} points × {point_value} = {price_difference} price dif

if tp_points and float(tp_points) > 0:
    tp_points_float = float(tp_points)
    price_difference = tp_points_float * point_value

    if order_type == "buy":
        tp_price = price + price_difference
    else: # sell
        tp_price = price - price_difference

    logging.info(
        f"TP calculation: {tp_points_float} points × {point_value} = {price_difference} price dif

return sl_price, tp_price

def place_order(self, symbol, order_type, volume=None, sl=None, tp=None, comment=None):
    try:
        # PRE-TRADE HEALTH CHECK: Ensure MT5 is ready for trading
        is_healthy, health_error = self.check_connection_health()
        if not is_healthy:
            logging.error(f"[ERROR] Pre-trade health check failed: {health_error}")
            # Attempt reconnection
            logging.info("■ Attempting automatic reconnection...")
            if self.attempt_reconnection():
                # Re-check health after reconnection
                is_healthy, health_error = self.check_connection_health()
                if not is_healthy:
                    raise Exception(f"MT5 reconnection failed: {health_error}")
            else:
                raise Exception(f"MT5 health check failed and reconnection unsuccessful: {health_error}")

        # Validate input parameters
        if not symbol or symbol.strip() == "":
            logging.error(f"Invalid symbol provided to place_order: '{symbol}'")
            raise Exception(f"Invalid symbol provided: '{symbol}'")

        symbol = symbol.strip()

        # Ensure symbol is available and get the correct symbol name
        corrected_symbol = self.ensure_symbol(symbol)

        # CRITICAL: Validate that ensure_symbol didn't return None
        if not corrected_symbol:
            logging.error(
                f"ensure_symbol returned None for '{symbol}' - MT5 connection or symbol validation fa
            raise Exception(
                f"Symbol validation failed for '{symbol}' - check MT5 connection and symbol availabi

        # Log order details with corrected symbol
        logging.info(
            f"■ PLACING ORDER: {order_type.upper()} {corrected_symbol}, volume={volume}, sl={sl}, tp={tp

        # Debug symbol info to understand MT5 properties (only log once per symbol)
        if not hasattr(self, '_debugged_symbols'):

```



```

        self._debugged_symbols = set()
    if corrected_symbol not in self._debugged_symbols:
        self.debug_symbol_info(corrected_symbol)
        self._debugged_symbols.add(corrected_symbol)

    tick = mt5.symbol_info_tick(corrected_symbol)
    print(f"[SEARCH] TICK RETRIEVAL DEBUG for {corrected_symbol}:")
    print(f"    Raw tick result: {tick}")
    if tick:
        print(f"    Tick ask: {tick.ask}, bid: {tick.bid}")
        print(f"    Tick time: {tick.time}")
    else:
        print(f"    [ERROR] Tick is None - investigating...")

    # Check MT5 initialization
    if not mt5.initialize():
        print(f"    [ERROR] MT5 not initialized - attempting to initialize...")
        if mt5.initialize():
            print(f"    [OK] MT5 initialization successful")
            # Try getting tick again after initialization
            tick = mt5.symbol_info_tick(corrected_symbol)
            print(f"    Retry after init: {tick}")
        else:
            print(f"    [ERROR] MT5 initialization failed")

    # Check terminal connection
    terminal_info = mt5.terminal_info()
    print(f"    Terminal info: {terminal_info}")
    if terminal_info:
        print(f"    Terminal connected: {terminal_info.connected}")
        print(f"    Terminal trade allowed: {terminal_info.trade_allowed}")

    # Check if symbol exists in symbol_info
    symbol_info = mt5.symbol_info(corrected_symbol)
    print(f"    Symbol info: {symbol_info}")
    if symbol_info:
        print(f"    Symbol visible: {symbol_info.visible}")
        print(f"    Symbol selected: {symbol_info.select}")

    # Try to select the symbol explicitly
    if not symbol_info.visible:
        print(f"    Attempting to select symbol...")
        select_result = mt5.symbol_select(corrected_symbol, True)
        print(f"    Symbol select result: {select_result}")
        if select_result:
            # Try getting tick again after selecting
            tick = mt5.symbol_info_tick(corrected_symbol)
            print(f"    Tick after select: {tick}")

    if tick is None:
        # Enhanced debugging for symbol issues
        logging.error(f"[ERROR] SYMBOL PRICE FETCH FAILED: {corrected_symbol}")

    # Check if symbol is selected in Market Watch
    selected = mt5.symbol_select(corrected_symbol, True)
    logging.error(f"[SEARCH] Symbol select result: {selected}")

    # Check symbol info
    symbol_info = mt5.symbol_info(corrected_symbol)
    if symbol_info:

```

```

        logging.error(
            f"[SEARCH] Symbol info exists: visible={symbol_info.visible}, tradeable={symbol_info.tradeable}"
        )
    else:
        logging.error(f"[SEARCH] Symbol info is None - symbol may not exist")

    # Try to get symbols that match pattern
    matching_symbols = mt5.symbols_get(group=f"*{corrected_symbol}*")
    if matching_symbols:
        logging.error(f"[SEARCH] Found {len(matching_symbols)} matching symbols:")
        for sym in matching_symbols[:5]: # Show first 5 matches
            logging.error(f"    - {sym.name}")
    else:
        logging.error(f"[SEARCH] No symbols found matching pattern *{corrected_symbol}*")

    # Enhanced symbol debugging for "Could not get price" issues
    print(f"[SEARCH] SYMBOL DEBUG: No price data for {corrected_symbol}")
    print(f"    Checking symbol selection and market watch...")

    # Check if symbol is in Market Watch
    market_watch_symbols = mt5.symbols_get()
    if market_watch_symbols:
        symbol_names = [s.name for s in market_watch_symbols]
        if corrected_symbol not in symbol_names:
            print(f"[ERROR] SYMBOL ERROR: {corrected_symbol} not in Market Watch")
            print(f"    Available symbols: {symbol_names[:10]}") # Show first 10
        else:
            print(f"[OK] SYMBOL FOUND: {corrected_symbol} is in Market Watch")

    # Try exact symbol matching
    exact_match = mt5.symbol_info(corrected_symbol)
    if not exact_match:
        print(f"[ERROR] EXACT MATCH FAILED: {corrected_symbol} not found")
        # Try pattern matching for similar symbols
        if market_watch_symbols:
            similar_symbols = [s.name for s in market_watch_symbols
                               if corrected_symbol.lower() in s.name.lower() or s.name.lower()
                               in corrected_symbol.lower()]
            if similar_symbols:
                print(f"[SEARCH] SIMILAR SYMBOLS: {similar_symbols}")
        else:
            print(f"[OK] SYMBOL EXISTS: {corrected_symbol} found in MT5")

    raise Exception(
        f"Could not get price for {corrected_symbol} - MT5 connection issue or symbol not recognized"
    )

    # SUCCESS: We have a valid tick, now extract the price
    price = tick.ask if order_type == "buy" else tick.bid
    print(f"[OK] PRICE EXTRACTED: {order_type} price for {corrected_symbol} = {price}")
    print(
        f"    Full tick data: ask={tick.ask}, bid={tick.bid}, spread={tick.ask - tick.bid if tick.ask > tick.bid else 0}"
    )

    # Validate price is reasonable
    if price <= 0:
        print(f"[ERROR] INVALID PRICE: {price} <= 0")
        raise Exception(f"Invalid price {price} for {corrected_symbol}")

    type_mt5 = mt5.ORDER_TYPE_BUY if order_type == "buy" else mt5.ORDER_TYPE_SELL
    supported_fillings = self.get_supported_filling_modes(corrected_symbol)
    last_error = None

    if sl is None:

```

```

        sl = self.sl_points
    if tp is None:
        tp = self.tp_points
    if volume is None:
        volume = self.default_volume

    # Apply PlexyTrade lot size adjustment - ONLY for USTECH (Nasdaq)
    # XAUUSD (Gold) pip values are consistent across brokers, so no division needed
    if self.is_plexy_server and volume > 0:
        # Only divide lot size for USTECH/Nasdaq symbols
        if any(x in corrected_symbol.upper() for x in ['USTECH', 'USTEC', 'NAS', 'NASDAQ', 'NDX',
            'NQ']):
            original_volume = volume
            volume = volume / 20.0
            logging.info(
                f"PlexyTrade adjustment for {corrected_symbol}: {original_volume} -> {volume} lot
        else:
            logging.info(
                f"PlexyTrade: No lot size adjustment for {corrected_symbol} (only divide USTECH,

    # Ensure SL and TP are always set (never skip) - use defaults if 0
    if sl is None or float(sl) <= 0:
        sl = self.sl_points if self.sl_points > 0 else 10 # Default 10 points if not set
        logging.info(f"Using default/minimum SL: {sl} points")
    if tp is None or float(tp) <= 0:
        tp = self.tp_points if self.tp_points > 0 else 20 # Default 20 points if not set
        logging.info(f"Using default/minimum TP: {tp} points")

    # Convert to float and ensure they're positive
    sl = float(sl)
    tp = float(tp)
    volume = float(volume)

    # Normalize volume to meet symbol requirements (step size, min, max)
    sym_info = mt5.symbol_info(corrected_symbol)
    if sym_info:
        step_vol = sym_info.volume_step
        min_vol = sym_info.volume_min
        max_vol = sym_info.volume_max

        if step_vol > 0:
            # Round to nearest multiple of step_vol
            volume = round(volume / step_vol) * step_vol

            # Fix floating point precision
            try:
                step_str = f"{step_vol:.10f}".rstrip('0').rstrip('.')
                decimals = 0
                if "." in step_str:
                    decimals = len(step_str.split(".")[1])
                volume = round(volume, decimals)
            except Exception:
                volume = round(volume, 2)

        if min_vol > 0 and volume < min_vol:
            logging.warning(f"Volume {volume} < min {min_vol}, clamped to min")
            volume = min_vol
        elif max_vol > 0 and volume > max_vol:
            logging.warning(f"Volume {volume} > max {max_vol}, clamped to max")
            volume = max_vol

```

```

        logging.info(f"Normalized volume: {volume} (Step: {step_vol}, Min: {min_vol})")

    sl_price, tp_price = self._calculate_sl_tp_price(corrected_symbol, order_type, price, sl, tp)

    logging.info(f"Order parameters: price={price}, sl_price={sl_price}, tp_price={tp_price}")

    for filling_mode in supported_fillings:
        request = {
            "action": mt5.TRADE_ACTION_DEAL,
            "symbol": corrected_symbol,
            "volume": volume,
            "type": type_mt5,
            "price": price,
            "deviation": 20,
            "type_filling": filling_mode,
            "type_time": mt5.ORDER_TIME_GTC,
        }

        # Add comment if provided
        if comment:
            request["comment"] = comment

        # ALWAYS add SL and TP - never skip them
        if sl_price is not None:
            request["sl"] = sl_price
            logging.info(f"Setting SL price: {sl_price}")
        if tp_price is not None:
            request["tp"] = tp_price
            logging.info(f"Setting TP price: {tp_price}")

        logging.info(f"Sending order request: {request}")
        result = mt5.order_send(request)

        if result is not None:
            logging.info(f"Order result: retcode={result.retcode}, comment={result.comment}")
            if result.retcode == mt5.TRADE_RETCODE_DONE:
                logging.info(
                    f"Order successful: {order_type} {corrected_symbol} {volume} at {price}, tick"
                )
                return result.order
            else:
                # Enhanced error handling for common MT5 trading issues
                error_msg = result.comment if result.comment else "Unknown error"

                # Check for automated trading permission issues
                if result.retcode == 10027: # TRADE_RETCODE_CLIENT_DISABLES_AT
                    print(f"[ERROR] MT5 TRADING ERROR: Automated trading is disabled in MT5 terminal")
                    print(f"[TOOL] SOLUTION: Enable automated trading in MT5:")
                    print(f"  1. Go to Tools → Options → Expert Advisors")
                    print(f"  2. Check 'Allow algorithmic trading'")
                    print(f"  3. Check 'Allow DLL imports'")
                    print(f"  4. Click OK and restart application")
                    raise Exception(
                        "Automated trading disabled in MT5 - Enable in Tools → Options → Expert Advisors"
                    )

                elif result.retcode == 10026: # TRADE_RETCODE_TRADE_DISABLED
                    print(f"[ERROR] MT5 TRADING ERROR: Trading is disabled")
                    print(f"[TOOL] SOLUTION: Check MT5 terminal settings and broker permissions")
                    raise Exception("Trading disabled - Check MT5 settings and broker permissions")

        # SUCCESS: We have a valid tick, now extract the price

```

```

price = tick.ask if order_type == "buy" else tick.bid
print(f"[OK] PRICE EXTRACTED: {order_type} price for {corrected_symbol} = {price}")
print(
    f"    Full tick data: ask={tick.ask}, bid={tick.bid}, spread={tick.ask - tick.bid if tick.ask > tick.bid else 0}"
)

# Validate price is reasonable
if price <= 0:
    print(f"[ERROR] INVALID PRICE: {price} <= 0")
    raise Exception(f"Invalid price {price} for {corrected_symbol}")

type_mt5 = mt5.ORDER_TYPE_BUY if order_type == "buy" else mt5.ORDER_TYPE_SELL
supported_fillings = self.get_supported_filling_modes(corrected_symbol)
last_error = None

if sl is None:
    sl = self.sl_points
if tp is None:
    tp = self.tp_points
if volume is None:
    volume = self.default_volume

# Apply PlexyTrade lot size adjustment - ONLY for USTECH (Nasdaq)
# XAUUSD (Gold) pip values are consistent across brokers, so no division needed
if self.is_plexy_server and volume > 0:
    # Only divide lot size for USTECH/Nasdaq symbols
    if any(x in corrected_symbol.upper() for x in ['USTECH', 'USTEC', 'NAS', 'NASDAQ', 'NDX', 'NQ']):
        original_volume = volume
        volume = volume / 20.0
        logging.info(
            f"PlexyTrade adjustment for {corrected_symbol}: {original_volume} -> {volume} lots"
        )
    else:
        logging.info(
            f"PlexyTrade: No lot size adjustment for {corrected_symbol} (only divide USTECH, USTEC, NAS, NASDAQ, NDX, NQ)"
        )

# Ensure SL and TP are always set (never skip) - use defaults if 0
if sl is None or float(sl) <= 0:
    sl = self.sl_points if self.sl_points > 0 else 10 # Default 10 points if not set
    logging.info(f"Using default/minimum SL: {sl} points")
if tp is None or float(tp) <= 0:
    tp = self.tp_points if self.tp_points > 0 else 20 # Default 20 points if not set
    logging.info(f"Using default/minimum TP: {tp} points")

# Convert to float and ensure they're positive
sl = float(sl)
tp = float(tp)
volume = float(volume)

sl_price, tp_price = self._calculate_sl_tp_price(corrected_symbol, order_type, price, sl, tp)

logging.info(f"Order parameters: price={price}, sl_price={sl_price}, tp_price={tp_price}")

for filling_mode in supported_fillings:
    request = {
        "action": mt5.TRADE_ACTION_DEAL,
        "symbol": corrected_symbol,
        "volume": volume,
        "type": type_mt5,
        "price": price,
        "deviation": 20,
    }

```

```

        "type_filling": filling_mode,
        "type_time": mt5.ORDER_TIME_GTC,
    }

    # Add comment if provided
    if comment:
        request["comment"] = comment

    # ALWAYS add SL and TP - never skip them
    if sl_price is not None:
        request["sl"] = sl_price
        logging.info(f"Setting SL price: {sl_price}")
    if tp_price is not None:
        request["tp"] = tp_price
        logging.info(f"Setting TP price: {tp_price}")

    logging.info(f"Sending order request: {request}")
    result = mt5.order_send(request)

    if result is not None:
        logging.info(f"Order result: retcode={result.retcode}, comment={result.comment}")
        if result.retcode == mt5.TRADE_RETCODE_DONE:
            logging.info(
                f"Order successful: {order_type} {corrected_symbol} {volume} at {price}, tick"
            )
            return result.order
        else:
            # Enhanced error handling for common MT5 trading issues
            error_msg = result.comment if result.comment else "Unknown error"

            # Check for automated trading permission issues
            if result.retcode == 10027: # TRADE_RETCODE_CLIENT_DISABLES_AT
                print(f"[ERROR] MT5 TRADING ERROR: Automated trading is disabled in MT5 terminal")
                print(f"[TOOL] SOLUTION: Enable automated trading in MT5:")
                print(f"    1. Go to Tools → Options → Expert Advisors")
                print(f"    2. Check 'Allow algorithmic trading'")
                print(f"    3. Check 'Allow DLL imports'")
                print(f"    4. Click OK and restart application")
                raise Exception(
                    "Automated trading disabled in MT5 - Enable in Tools → Options → Expert"
                )

            elif result.retcode == 10026: # TRADE_RETCODE_TRADE_DISABLED
                print(f"[ERROR] MT5 TRADING ERROR: Trading is disabled")
                print(f"[TOOL] SOLUTION: Check MT5 terminal settings and broker permissions")
                raise Exception("Trading disabled - Check MT5 settings and broker permissions")

            elif result.retcode == 10013: # TRADE_RETCODE_INVALID_REQUEST
                print(f"[ERROR] MT5 TRADING ERROR: Invalid trading request")
                print(f"[TOOL] SOLUTION: Check symbol, volume, and market hours")
                raise Exception(f"Invalid trading request: {error_msg}")

            elif result.retcode == 10004: # TRADE_RETCODE_REQUOTE
                print(f"[WARNING] MT5 TRADING: Price requote - retrying...")

            elif result.retcode == 10018: # TRADE_RETCODE_MARKET_CLOSED
                print(f"[ERROR] MT5 TRADING ERROR: Market is closed")
                print(f"[TOOL] SOLUTION: Wait for market opening hours")
                raise Exception("Market is closed - Wait for trading hours")

            elif result.retcode == 10019: # TRADE_RETCODE_NO_MONEY
                print(f"[ERROR] MT5 TRADING ERROR: Insufficient funds")

```

```

        print(f"[TOOL] SOLUTION: Check account balance and reduce position size")
        raise Exception("Insufficient funds - Check account balance")

    else:
        print(f"[ERROR] MT5 TRADING ERROR: {error_msg} (Code: {result.retcode})")
        print(f"[TOOL] SOLUTION: Check MT5 terminal for detailed error information")

        last_error = f"{error_msg} (Code: {result.retcode})"
    else:
        last_error = "No result returned"
        print(f"[ERROR] MT5 CRITICAL: No response from MT5 terminal")
        print(f"[TOOL] SOLUTION: Check MT5 terminal connection and restart if needed")

    logging.warning(
        f"Order attempt failed: {order_type} {corrected_symbol} {volume} at {price}, filling: {filling}"
    )

    logging.error(
        f"Order failed: {order_type} {corrected_symbol} {volume} at {price}, last_error={last_error}"
    )
    raise Exception(f"Order failed: {last_error}")

except Exception as e:
    logging.exception(f"Exception in place_order: {e}")
    raise

def buy_market(self, symbol, volume=None, sl=None, tp=None, comment=None):
    return self.place_order(symbol, "buy", volume=volume, sl=sl, tp=tp, comment=comment)

def sell_market(self, symbol, volume=None, sl=None, tp=None, comment=None):
    return self.place_order(symbol, "sell", volume=volume, sl=sl, tp=tp, comment=comment)

def is_connected(self):
    """Check if MT5 connection is still active"""
    try:
        # Try to get account info to test connection
        account_info = mt5.account_info()
        if account_info is None:
            return False
        return True
    except Exception:
        return False

def check_connection_and_disconnect_if_needed(self):
    """Check connection and disconnect if lost"""
    if not self.is_connected():
        logging.warning("MT5 connection lost, disconnecting...")
        self.disconnect()
        return False
    return True

def disconnect(self):
    """Properly disconnect from MT5 with enhanced cleanup and terminal closure"""
    try:
        # First, perform standard MT5 API shutdown
        if mt5.terminal_info() is not None:
            mt5.shutdown()
            logging.info("MT5 API disconnected successfully")
        else:
            logging.info("MT5 API was already disconnected")

        # Force close MT5 terminal processes to ensure complete shutdown

```

```

        self._close_mt5_processes()

    except Exception as e:
        logging.error(f"Error during MT5 disconnect: {e}")
        # Force shutdown and process termination even if there was an error
        try:
            mt5.shutdown()
        except:
            pass
        # Still attempt to close processes
        self._close_mt5_processes()

def _close_mt5_processes(self):
    """Force close all MT5 terminal processes"""
    try:
        closed_processes = []

        # Look for MT5 processes by name
        for proc in psutil.process_iter(['pid', 'name', 'exe']):
            try:
                proc_name = proc.info['name'].lower()
                proc_exe = proc.info['exe']

                # Check if this is an MT5 process
                if (proc_name in ['terminal64.exe', 'terminal.exe', 'metatrader5.exe'] or
                    (proc_exe and 'metatrader' in proc_exe.lower())):

                    proc.terminate() # Send termination signal
                    closed_processes.append(f"{proc_name} (PID: {proc.info['pid']})")

            except (psutil.NoSuchProcess, psutil.AccessDenied, psutil.ZombieProcess):
                # Process might have already closed or access denied
                continue
            except Exception as e:
                logging.warning(f"Error checking process: {e}")
                continue

        if closed_processes:
            logging.info(f"■ Closed MT5 processes: {' '.join(closed_processes)}")

            # Wait a moment for graceful termination
            sleep(2)

            # Force kill any remaining MT5 processes
            for proc in psutil.process_iter(['pid', 'name', 'exe']):
                try:
                    proc_name = proc.info['name'].lower()
                    proc_exe = proc.info['exe']

                    if (proc_name in ['terminal64.exe', 'terminal.exe', 'metatrader5.exe'] or
                        (proc_exe and 'metatrader' in proc_exe.lower())):

                        proc.kill() # Force kill if still running
                        logging.info(
                            f"■ Force killed stubborn MT5 process: {proc_name} (PID: {proc.info['pid']})")

                except (psutil.NoSuchProcess, psutil.AccessDenied, psutil.ZombieProcess):
                    continue
                except Exception as e:
                    logging.warning(f"Error force killing process: {e}")

```



```

        continue
    else:
        logging.info("■ No MT5 processes found to close")

except Exception as e:
    logging.error(f"Error closing MT5 processes: {e}")
    # Fallback: try using taskkill as last resort
    try:
        subprocess.run(['taskkill', '/f', '/im', 'terminal64.exe'],
                        capture_output=True, check=False)
        subprocess.run(['taskkill', '/f', '/im', 'terminal.exe'],
                        capture_output=True, check=False)
        logging.info("■ Used taskkill as fallback to close MT5 processes")
    except Exception as fallback_error:
        logging.error(f"Fallback taskkill also failed: {fallback_error}")

def get_account_info(self):
    info = mt5.account_info()
    if info is None:
        logging.error("No account info available")
        return {"balance": "", "profit": "", "drawdown": "", "open_trades": "", "Symbol": "",
                "Direction": ""}
    trades = mt5.positions_get()
    open_trades = len(trades) if trades else 0
    symbol = ""
    direction = ""
    if trades and open_trades > 0:
        pos = trades[0]
        symbol = getattr(pos, "symbol", "")
        if getattr(pos, "type", None) == mt5.POSITION_TYPE_BUY:
            direction = "Long"
        elif getattr(pos, "type", None) == mt5.POSITION_TYPE_SELL:
            direction = "Short"
        else:
            direction = ""
    return {
        "balance": str(round(info.balance, 2)),
        "profit": str(round(info.profit, 2)),
        "drawdown": "",
        "open_trades": str(open_trades),
        "Symbol": symbol,
        "Direction": direction
    }

def is_trade_open(self, ticket):
    positions = mt5.positions_get(ticket=ticket)
    return positions is not None and len(positions) > 0

def has_open_trade(self, symbol):
    """
    Returns True if there is any open position for the given symbol.
    """
    positions = mt5.positions_get(symbol=symbol)
    return positions is not None and len(positions) > 0

def get_trades_today_count(self, comment_filter=None):
    """Get the number of trades opened today

    Args:
        comment_filter: Optional string to filter trades by comment (e.g., "Combine1_")
    """

```

```

Returns:
    int: Number of trades opened today
    """
try:
    from datetime import datetime, date
    import MetaTrader5 as mt5

    today = date.today()
    today_start = datetime.combine(today, datetime.min.time())
    today_end = datetime.combine(today, datetime.max.time())

    # Convert to timestamp
    from_date = int(today_start.timestamp())
    to_date = int(today_end.timestamp())

    # Get history deals (completed trades) for today
    deals = mt5.history_deals_get(from_date, to_date)

    # Also check current open positions opened today
    positions = mt5.positions_get()

    count = 0

    # Count completed deals (history)
    if deals:
        for deal in deals:
            # Only count entry deals (not exit deals)
            if deal.entry == mt5.DEAL_ENTRY_IN:
                if comment_filter is None or (deal.comment and comment_filter in deal.comment):
                    count += 1

    # Count open positions opened today
    if positions:
        for pos in positions:
            pos_time = datetime.fromtimestamp(pos.time)
            if pos_time.date() == today:
                if comment_filter is None or (pos.comment and comment_filter in pos.comment):
                    count += 1

    logging.info(f"Trades opened today: {count} (filter: {comment_filter})")
    return count

except Exception as e:
    logging.error(f"Error getting today's trade count: {e}")
    return 0

def get_orphaned_mt5_positions_by_account(self, account_number):
    """Get MT5 positions for a specific Tradovate account

    Args:
        account_number: Tradovate account number to filter by

    Returns:
        list: List of orphaned position tickets
    """
    try:
        # Get all open MT5 positions
        positions = mt5.positions_get()
        if positions is None:
            return []

```

```

        orphaned_tickets = []
        for pos in positions:
            if pos.comment:
                # Check if position is from this account (comment starts with account number, may have
                # trailing spaces)
                if pos.comment.strip().startswith(account_number):
                    orphaned_tickets.append(pos.ticket)

        return orphaned_tickets
    except Exception as e:
        logging.error(f"Error finding orphaned positions by account: {e}")
        return []

def close_orphaned_positions_by_account(self, account_number):
    """Close MT5 positions for a specific Tradovate account

    Args:
        account_number: Tradovate account number to filter by

    Returns:
        int: Number of positions closed
    """
    try:
        orphaned_tickets = self.get_orphaned_mt5_positions_by_account(account_number)
        closed_count = 0

        for ticket in orphaned_tickets:
            try:
                if self.close_trade(ticket):
                    closed_count += 1
                    logging.info(f"Closed orphaned MT5 position for account {account_number}: {ticket}")
            except Exception as e:
                logging.error(f"Error closing orphaned position {ticket}: {e}")

        return closed_count
    except Exception as e:
        logging.error(f"Error closing orphaned positions by account: {e}")
        return 0

def get_orphaned_mt5_positions(self, combine_comment_prefix):
    """Get MT5 positions that don't have corresponding Tradovate trades

    Args:
        combine_comment_prefix: Comment prefix to identify trades from this combine (e.g., "Combine1")

    Returns:
        list: List of orphaned position tickets
    """
    try:
        import MetaTrader5 as mt5

        positions = mt5.positions_get()
        orphaned_tickets = []

        if positions:
            for pos in positions:
                # Check if this position belongs to our combine
                if pos.comment and combine_comment_prefix in pos.comment:
                    orphaned_tickets.append(pos.ticket)
                    logging.info(f"Found orphaned MT5 position: {pos.ticket} ({pos.comment})")

        return orphaned_tickets
    
```

```

except Exception as e:
    logging.error(f"Error finding orphaned positions: {e}")
    return []

def close_orphaned_positions(self, combine_comment_prefix):
    """Close MT5 positions that don't have corresponding Tradovate trades

    Args:
        combine_comment_prefix: Comment prefix to identify trades from this combine (e.g., "Combine1

    Returns:
        int: Number of positions closed
    """
    try:
        orphaned_tickets = self.get_orphaned_mt5_positions(combine_comment_prefix)
        closed_count = 0

        for ticket in orphaned_tickets:
            try:
                if self.close_trade(ticket):
                    closed_count += 1
                    logging.info(f"Closed orphaned MT5 position: {ticket}")
            else:
                logging.warning(f"Failed to close orphaned MT5 position: {ticket}")
        except Exception as e:
            logging.error(f"Error closing orphaned position {ticket}: {e}")

        return closed_count

    except Exception as e:
        logging.error(f"Error closing orphaned positions: {e}")
        return 0

def close_trade(self, ticket, retries=3, delay=2):
    for attempt in range(retries):
        positions = mt5.positions_get(ticket=ticket)
        if not positions:
            logging.info(f"Trade {ticket} already closed.")
            return True
        pos = positions[0]
        symbol = pos.symbol
        volume = pos.volume
        order_type = pos.type
        price = mt5.symbol_info_tick(
            symbol).bid if order_type == mt5.POSITION_TYPE_BUY else mt5.symbol_info_tick(symbol).ask
        close_type = mt5.ORDER_TYPE_SELL if order_type == mt5.POSITION_TYPE_BUY else mt5.ORDER_TYPE_B
        request = {
            "action": mt5.TRADE_ACTION_DEAL,
            "symbol": symbol,
            "volume": volume,
            "type": close_type,
            "position": ticket,
            "price": price,
            "deviation": 20,
            "type_filling": mt5.ORDER_FILLING_IOC,
            "type_time": mt5.ORDER_TIME_GTC,
        }
        result = mt5.order_send(request)
        if result is not None and result.retcode == mt5.TRADE_RETCODE_DONE:
            if not self.is_trade_open(ticket):

```

```

        logging.info(f"Trade {ticket} closed successfully.")
        return True
    sleep(delay)
    logging.error(f"Failed to close trade {ticket} after {retries} attempts.")
    return not self.is_trade_open(ticket)

def force_close_trade(self, ticket):
    positions = mt5.positions_get(ticket=ticket)
    if not positions:
        logging.info(f"Trade {ticket} already closed (force close).")
        return True
    pos = positions[0]
    symbol = pos.symbol
    volume = pos.volume
    order_type = pos.type
    price = mt5.symbol_info_tick(
        symbol).bid if order_type == mt5.POSITION_TYPE_BUY else mt5.symbol_info_tick(symbol).ask
    close_type = mt5.ORDER_TYPE_SELL if order_type == mt5.POSITION_TYPE_BUY else mt5.ORDER_TYPE_BUY
    for filling in [mt5.ORDER_FILLING_IOC, mt5.ORDER_FILLING_FOK, mt5.ORDER_FILLING_RETURN]:
        request = {
            "action": mt5.TRADE_ACTION_DEAL,
            "symbol": symbol,
            "volume": volume,
            "type": close_type,
            "position": ticket,
            "price": price,
            "deviation": 20,
            "type_filling": filling,
            "type_time": mt5.ORDER_TIME_GTC,
        }
        result = mt5.order_send(request)
        if result is not None and result.retcode == mt5.TRADE_RETCODE_DONE:
            if not self.is_trade_open(ticket):
                logging.info(f"Trade {ticket} force closed successfully.")
                return True
    logging.error(f"Failed to force close trade {ticket}.")
    return not self.is_trade_open(ticket)

def get_daily_trade_count(self, comment_filter=None):
    """Get count of trades placed today from both open positions and history

    Args:
        comment_filter: Can be either:
            - Tradovate account number (e.g., "MFFUEVSTP326057008") - preferred method
            - Old combine prefix (e.g., "Combine1_") - for backward compatibility
    """
    try:
        from datetime import datetime, date
        import tempfile
        import os

        today = date.today()

        # Check if trades were reset for this filter today
        temp_dir = tempfile.gettempdir()
        reset_file = os.path.join(temp_dir, f"mt5_reset_{comment_filter}_{today.strftime('%Y%m%d')}.t")
        if os.path.exists(reset_file):
            # Return 0 if reset flag exists for today
            return 0

        trade_count = 0
    
```

```

# Count from open positions
positions = mt5.positions_get()
if positions:
    for pos in positions:
        # Convert MT5 time to date
        pos_date = datetime.fromtimestamp(pos.time).date()
        if pos_date == today:
            # If comment filter is provided, check if position comment matches
            if comment_filter:
                # For new format: exact match with account number
                # For old format: substring match with combine prefix
                if pos.comment and (pos.comment.strip(
                    ) == comment_filter or comment_filter in str(pos.comment)):
                    trade_count += 1
            else:
                trade_count += 1

# Count from history deals (more reliable for completed trades)
# Get deals from start of today to now
today_start = datetime.combine(today, datetime.min.time())
today_end = datetime.now()

deals = mt5.history_deals_get(today_start, today_end)
if deals:
    # Only count entry deals (not exit deals to avoid double counting)
    for deal in deals:
        if deal.entry == mt5.DEAL_ENTRY_IN: # Entry deal only
            # If comment filter is provided, check if deal comment matches
            if comment_filter:
                # For new format: exact match with account number
                # For old format: substring match with combine prefix
                if deal.comment and (deal.comment.strip(
                    ) == comment_filter or comment_filter in str(deal.comment)):
                    trade_count += 1
            else:
                trade_count += 1

    logging.info(f"Daily trade count: {trade_count} (filter: {comment_filter})")
    return trade_count

except Exception as e:
    logging.error(f"Error counting daily trades: {e}")
    return 0
    return 0

def get_daily_trade_count_by_account(self, tradovate_account_number):
    """Get count of trades placed today for a specific Tradovate account

    Args:
        tradovate_account_number: Tradovate account number (e.g., "MFFUEVSTP326057008")

    Returns:
        int: Number of trades opened today for this account
    """
    try:
        from datetime import datetime, date

        today = date.today()
        trade_count = 0

        # Count from open positions

```

```

positions = mt5.positions_get()
if positions:
    for pos in positions:
        # Convert MT5 time to date
        pos_date = datetime.fromtimestamp(pos.time()).date()
        if pos_date == today:
            # Check if position comment matches the account number (comment starts with account number)
            if pos.comment and pos.comment.strip().startswith(tradovate_account_number):
                trade_count += 1

# Count from history deals
today_start = datetime.combine(today, datetime.min.time())
today_end = datetime.now()

deals = mt5.history_deals_get(today_start, today_end)
if deals:
    for deal in deals:
        if deal.entry == mt5.DEAL_ENTRY_IN: # Entry deal only
            # Check if deal comment matches the account number (comment starts with account number)
            if deal.comment and deal.comment.strip().startswith(tradovate_account_number):
                trade_count += 1

logging.info(f"Daily trade count for account {tradovate_account_number}: {trade_count}")
return trade_count

except Exception as e:
    logging.error(f"Error counting daily trades for account {tradovate_account_number}: {e}")
    return 0

def reset_daily_trade_count(self, comment_filter=None):
    """Reset daily trade count for a specific filter by storing reset timestamp

    Args:
        comment_filter: Can be either:
            - Tradovate account number (e.g., "MFFUEVSTP326057008") - preferred method
            - Old combine prefix (e.g., "Combine1") - for backward compatibility
    """
    try:
        from datetime import datetime
        import tempfile
        import os

        # Create a simple flag file to mark that trades were reset for this filter today
        temp_dir = tempfile.gettempdir()
        reset_file = os.path.join(temp_dir,
                                   f"mt5_reset_{comment_filter}_{datetime.now().strftime('%Y%m%d')}.flag")

        # Create the flag file
        with open(reset_file, 'w') as f:
            f.write(str(datetime.now().timestamp()))

        logging.info(f"Daily trade count reset for filter: {comment_filter}")
        return True

    except Exception as e:
        logging.error(f"Error resetting daily trade count: {e}")
        return False

def reset_daily_trade_count_by_account(self, tradovate_account_number):
    """Reset daily trade count for a specific Tradovate account

```

```

Args:
    tradovate_account_number: Tradovate account number (e.g., "MFFUEVSTP326057008")
"""
try:
    from datetime import datetime
    import tempfile
    import os

    # Create a simple flag file to mark that trades were reset for this account today
    temp_dir = tempfile.gettempdir()
    reset_file = os.path.join(temp_dir,
                              f"mt5_reset_account_{tradovate_account_number}_{datetime.now().strftime('%Y%m%d')}.flag")

    # Create the flag file
    with open(reset_file, 'w') as f:
        f.write(str(datetime.now().timestamp()))

    logging.info(f"Daily trade count reset for Tradovate account: {tradovate_account_number}")
    return True

except Exception as e:
    logging.error(f"Error resetting daily trade count for account {tradovate_account_number}: {e}")
    return False

def get_historical_profits_by_account(self, account_number):
    """Get total historical profits for trades from a specific Tradovate account"""
    try:
        from datetime import datetime, timedelta

        # Get deals from the last 30 days to get a good history
        end_time = datetime.now()
        start_time = end_time - timedelta(days=30)

        total_profit = 0.0

        # Get historical deals
        deals = mt5.history_deals_get(start_time, end_time)
        if deals:
            for deal in deals:
                # Check if deal comment matches the account number (comment starts with account number)
                if deal.comment and deal.comment.strip().startswith(account_number):
                    # Only count exit deals for profit calculation (avoid double counting)
                    if deal.entry == mt5.DEAL_ENTRY_OUT:
                        total_profit += deal.profit

        logging.info(f"Historical profits for account {account_number}: ${total_profit:.2f}")
        return total_profit

    except Exception as e:
        logging.error(f"Error getting historical profits by account: {e}")
        return 0.0

def get_historical_profits(self, comment_filter=None):
    """Get total historical profits for trades with specific comment filter"""
    try:
        from datetime import datetime, timedelta

        # Get deals from the last 30 days to get a good history
        end_time = datetime.now()
        start_time = end_time - timedelta(days=30)

```



```

total_profit = 0.0

# Get historical deals
deals = mt5.history_deals_get(start_time, end_time)
if deals:
    for deal in deals:
        # Check if deal comment matches the filter (for specific combine)
        if comment_filter and comment_filter not in str(deal.comment):
            continue

        # Only count exit deals for profit calculation (avoid double counting)
        if deal.entry == mt5.DEAL_ENTRY_OUT:
            total_profit += deal.profit

    logging.info(f"Historical profits for {comment_filter}: ${total_profit:.2f}")
    return total_profit

except Exception as e:
    logging.error(f"Error calculating historical profits: {e}")
    return 0.0

def close_orphaned_trades(self, expected_tradovate_trades):
    """Close MT5 trades that don't have corresponding Tradovate trades

    Args:
        expected_tradovate_trades: List of Tradovate trade symbols/IDs that should have MT5 counterparts

    Returns:
        List of closed MT5 trade tickets
    """
    try:
        closed_trades = []
        positions = mt5.positions_get()

        if not positions:
            return closed_trades

        for pos in positions:
            should_close = False

            # If no Tradovate trades expected, close all MT5 trades
            if not expected_tradovate_trades:
                should_close = True
                reason = "no corresponding Tradovate trades"
            else:
                # Check if this MT5 trade has a corresponding Tradovate trade
                # This is a simplified check - in practice you might need more sophisticated matching
                mt5_symbol = pos.symbol
                has_counterpart = False

                for tradovate_trade in expected_tradovate_trades:
                    # Simple symbol matching - you may want to enhance this logic
                    if str(tradovate_trade).upper() in mt5_symbol.upper():
                        has_counterpart = True
                        break

                if not has_counterpart:
                    should_close = True
                    reason = f"no matching Tradovate trade found"

            if should_close:

```

```

        logging.info(f"Closing orphaned MT5 trade {pos.ticket}: {pos.symbol} ({reason})")
        if self.close_trade(pos.ticket):
            closed_trades.append(pos.ticket)
            logging.info(f"[OK] Closed orphaned trade {pos.ticket}")
        else:
            logging.error(f"[ERROR] Failed to close orphaned trade {pos.ticket}")

    if closed_trades:
        logging.info(f"Closed {len(closed_trades)} orphaned MT5 trades: {closed_trades}")

    return closed_trades

except Exception as e:
    logging.error(f"Error closing orphaned trades: {e}")
    return []

def extract_tradovate_account_from_comment(self, comment):
    """Extract Tradovate account number from MT5 comment

    Args:
        comment: MT5 order comment (e.g., "MFFUEVSTP326057008_CH1" or "ACCOUNT_FA")

    Returns:
        str: Tradovate account number or "Unknown" if not found
    """
    try:
        if comment and comment.strip():
            import re
            # For new format: comment is account number + optional phase suffix
            # Strip the phase suffix if present
            account_number = comment.strip()
            # Remove phase abbreviation suffix if present
            if '_' in account_number:
                # Check for numbered formats (_CH1-4, _FD1-4, _DD1-4)
                if re.search(r'_(CH|FD|DD)\d+$', account_number):
                    account_number = re.sub(r'_(CH|FD|DD)\d+$', '', account_number)
                # Check for simple farming format: _FA
                elif account_number.endswith('_FA'):
                    account_number = account_number[:-3]
                # Check for unknown phase marker
                elif account_number.endswith('_UNK'):
                    account_number = account_number[:-4]
            return account_number if account_number else "Unknown"
        return "Unknown"
    except Exception as e:
        logging.error(f"Error extracting account from comment '{comment}': {e}")
        return "Unknown"

def get_trades_by_tradovate_account(self, tradovate_account_number=None):
    """Get all MT5 trades associated with a specific Tradovate account

    Args:
        tradovate_account_number: Tradovate account number to filter by

    Returns:
        dict: Dictionary with 'open_positions' and 'history_deals' lists
    """
    try:
        from datetime import datetime, date

        result = {

```

```

        'open_positions': [],
        'history_deals': []
    }

    # Get open positions
    positions = mt5.positions_get()
    if positions:
        for pos in positions:
            if pos.comment:
                account_from_comment = self.extract_tradovate_account_from_comment(pos.comment)
                if tradovate_account_number is None or account_from_comment == tradovate_account_number:
                    result['open_positions'].append({
                        'ticket': pos.ticket,
                        'symbol': pos.symbol,
                        'volume': pos.volume,
                        'type': 'BUY' if pos.type == mt5.POSITION_TYPE_BUY else 'SELL',
                        'open_time': datetime.fromtimestamp(pos.time),
                        'comment': pos.comment,
                        'tradovate_account': account_from_comment
                    })

    # Get today's history deals
    today = date.today()
    today_start = datetime.combine(today, datetime.min.time())
    today_end = datetime.combine(today, datetime.max.time())
    from_date = int(today_start.timestamp())
    to_date = int(today_end.timestamp())

    deals = mt5.history_deals_get(from_date, to_date)
    if deals:
        for deal in deals:
            if deal.comment:
                account_from_comment = self.extract_tradovate_account_from_comment(deal.comment)
                if tradovate_account_number is None or account_from_comment == tradovate_account_number:
                    result['history_deals'].append({
                        'ticket': deal.ticket,
                        'symbol': deal.symbol,
                        'volume': deal.volume,
                        'type': 'BUY' if deal.type == mt5.DEAL_TYPE_BUY else 'SELL',
                        'time': datetime.fromtimestamp(deal.time),
                        'comment': deal.comment,
                        'tradovate_account': account_from_comment,
                        'entry': deal.entry
                    })

    return result

except Exception as e:
    logging.error(f"Error getting trades by Tradovate account: {e}")
    return {'open_positions': [], 'history_deals': []}

def get_symbol_info(self, symbol):
    """Get symbol information"""
    try:
        return mt5.symbol_info(symbol)
    except Exception as e:
        logging.error(f"Error getting symbol info for {symbol}: {e}")
        return None

def debug_symbol_info(self, symbol):

```

```

"""Debug method to print all available symbol information"""
try:
    info = mt5.symbol_info(symbol)
    if info:
        logging.info(f"=== Symbol Info for {symbol} ===")
        for attr in dir(info):
            if not attr.startswith('_'):
                try:
                    value = getattr(info, attr)
                    logging.info(f"  {attr}: {value}")
                except:
                    pass
        logging.info("=== End Symbol Info ===")
        return info
    else:
        logging.error(f"No symbol info available for {symbol}")
        return None
except Exception as e:
    logging.error(f"Error debugging symbol info: {e}")
    return None

def get_tick_data(self, symbol):
    """Get current tick data for symbol"""
    try:
        return mt5.symbol_info_tick(symbol)
    except Exception as e:
        logging.error(f"Error getting tick data for {symbol}: {e}")
        return None

def should_close_trades_for_rollover(self, prop_firm_name):
    """
    Check if trades should be closed based on prop firm rollover schedules.

    Closing Times (Eastern Time):
    - Trade Day: 5:00 PM ET
    - Funding Ticks: 5:00 PM ET
    - Tradeify: 4:59 PM ET
    - MFFU: 4:10 PM ET
    - Alpha Futures: 4:20 PM ET

    Args:
        prop_firm_name: Name of the prop firm

    Returns:
        bool: True if trades should be closed now, False otherwise
    """
    import datetime
    import pytz

    try:
        # Get current time in Eastern timezone
        eastern = pytz.timezone('US/Eastern')
        current_time = datetime.datetime.now(eastern)

        # Define closing times for each prop firm (24-hour format)
        closing_schedules = {
            "Funding Ticks": (17, 0), # 5:00 PM Eastern Time
            "Tradeify": (16, 59), # 4:59 PM Eastern Time
            "MFFU": (16, 10), # 4:10 PM Eastern Standard Time
            "Alpha Futures": (16, 20), # 4:20 PM Eastern Time
        }
    
```

```

    }

    # Get closing time for this prop firm
    closing_time_tuple = closing_schedules.get(prop_firm_name)

    if closing_time_tuple is None:
        # This prop firm doesn't require trade closing
        return False

    # Convert closing time to datetime object for proper comparison
    closing_hour, closing_minute = closing_time_tuple
    closing_time = current_time.replace(hour=closing_hour, minute=closing_minute, second=0,
                                         microsecond=0)

    # Get current date string for tracking
    current_date = current_time.strftime("%Y-%m-%d")

    # Safety check: Have we already executed rollover for this prop firm today?
    if self.rollover_executed_today.get(prop_firm_name) == current_date:
        return False # Already executed today, skip to prevent duplicates

    # Check if current time is at or past closing time (with safety buffer)
    # This ensures we don't miss rollover due to system delays
    if current_time >= closing_time:
        # Also check if we're on a weekday (Monday = 0, Sunday = 6)
        if current_time.weekday() < 5: # Monday through Friday
            current_time_str = current_time.strftime("%H:%M")
            closing_time_str = closing_time.strftime("%H:%M")
            logging.info(
                f"■ Market rollover time reached for {prop_firm_name} at {current_time_str} ET (
            )
            return True

    return False

except Exception as e:
    logging.error(f"Error checking rollover schedule for {prop_firm_name}: {e}")
    return False

def close_trades_for_rollover(self, prop_firm_name, account_comment_prefix=None):
    """
    Close all MT5 trades for market rollover based on prop firm schedule.

    Args:
        prop_firm_name: Name of the prop firm
        account_comment_prefix: Account number to filter trades (e.g., "MFFU123456")

    Returns:
        list: List of closed trade tickets
    """
    if not self.should_close_trades_for_rollover(prop_firm_name):
        return []

    try:
        # Get all open positions
        positions = mt5.positions_get()
        if positions is None:
            logging.warning("No positions found or error getting positions")
            return []

        closed_tickets = []

```

```

for position in positions:
    # Filter by account comment prefix if provided
    if account_comment_prefix and not position.comment.startswith(account_comment_prefix):
        continue

    # Close the position
    if self.close_trade(position.ticket):
        closed_tickets.append(position.ticket)
        logging.info(
            f"■ ROLLOVER: Closed trade {position.ticket} for {prop_firm_name} market rollover"
        )
    else:
        logging.error(f"[ERROR] Failed to close trade {position.ticket} for rollover")

if closed_tickets:
    logging.info(
        f"■ ROLLOVER COMPLETE: Closed {len(closed_tickets)} trades for {prop_firm_name} at m

    # Mark rollover as executed today to prevent duplicate execution
    import datetime
    import pytz
    eastern = pytz.timezone('US/Eastern')
    current_date = datetime.datetime.now(eastern).strftime("%Y-%m-%d")
    self.rollover_executed_today[prop_firm_name] = current_date

return closed_tickets

except Exception as e:
    logging.error(f"Error closing trades for rollover ({prop_firm_name}): {e}")
    return []

```

Method: MT5API.__init__

```
def __init__(self, login, password, server, symbol=None, terminal_path=None)
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.
- **__init__** — The constructor. Runs after Python has allocated the instance; assigns starting attribute values to self.

Step by step (top-level body)

1. **try** block with 1 **except** clause.
2. Assigns `self.password = str(password) if password else "`.
3. Assigns `self.server = str(server) if server else "`.
4. Assigns `self.symbol = symbol`.
5. Assigns `self.terminal_path = terminal_path`.

6. Assigns `self.sl_points = float(os.getenv('MT5_SL_POINTS') or os.getenv('MT5_STOPLO....`
7. Assigns `self.tp_points = float(os.getenv('MT5_TP_POINTS') or os.getenv('MT5_TAKEPR....`
8. Assigns `self.default_volume = float(os.getenv('MT5_VOLUME', '1'))`.
9. Assigns `self.rollover_executed_today = {}`.
10. Assigns `self.connected_symbol = None`.
11. Assigns `self.is_plexy_server = 'plexy' in server.lower()` if server else False.
12. **if** `self.is_plexy_server`: branches conditionally.
13. Assigns `self.connected = False`.
14. Assigns `self.last_error = None`.

Code (copy-paste safe)

```
def __init__(self, login, password, server, symbol=None, terminal_path=None):
    # Safely convert login to integer
    try:
        self.login = int(str(login).strip()) if login else 0
    except (ValueError, TypeError):
        logging.error(f'Invalid login format: {login}')
        self.login = 0

    self.password = str(password) if password else ""
    self.server = str(server) if server else ""
    self.symbol = symbol
    self.terminal_path = terminal_path
    self.sl_points = float(os.getenv('MT5_SL_POINTS') or os.getenv('MT5_STOPLOSS_POINTS', '0'))
    self.tp_points = float(os.getenv('MT5_TP_POINTS') or os.getenv('MT5_TAKEPROFIT_POINTS', '0'))
    self.default_volume = float(os.getenv('MT5_VOLUME', '1'))

    # Rollover safety tracking - prevents multiple executions per day
    self.rollover_executed_today = {} # {prop_firm: date_string}

    # Store the actually connected symbol (will be set after successful connection)
    self.connected_symbol = None

    # Check if this is a PlexyTrade server (case-insensitive substring match)
    self.is_plexy_server = "plexy" in server.lower() if server else False
    if self.is_plexy_server:
        logging.info(f"PlexyTrade server detected: {server} - Lot sizes will be divided by 20")

    # Ensure all instance variables are properly initialized
    self.connected = False
    self.last_error = None
```

Method: MT5API._get_cached_terminal_path

```
def _get_cached_terminal_path(self)
    Get previously successful terminal path for faster connection
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Imports tempfile (lazy import inside the function).
2. Assigns `cache_file = os.path.join(tempfile.gettempdir(), 'mt5_terminal_cache.t....`
3. **try** block with 1 **except** clause.
4. **return** None.

Code (copy-paste safe)

```
def _get_cached_terminal_path(self):
    """Get previously successful terminal path for faster connection"""
    import tempfile
    cache_file = os.path.join(tempfile.gettempdir(), "mt5_terminal_cache.txt")
    try:
        if os.path.exists(cache_file):
            with open(cache_file, 'r') as f:
                cached_path = f.read().strip()
                if os.path.exists(cached_path):
                    return cached_path
    except Exception as e:
        logging.debug(f"Cache read failed: {e}")
    return None
```

Method: MT5API._cache_successful_path

```
def _cache_successful_path(self, path)

    Cache successful terminal path for future use
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Imports tempfile (lazy import inside the function).
2. Assigns `cache_file = os.path.join(tempfile.gettempdir(), 'mt5_terminal_cache.t....`
3. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def _cache_successful_path(self, path):
```



```

"""Cache successful terminal path for future use"""
import tempfile
cache_file = os.path.join(tempfile.gettempdir(), "mt5_terminal_cache.txt")
try:
    with open(cache_file, 'w') as f:
        f.write(path)
    logging.info(f"[OK] Cached successful MT5 path: {path}")
except Exception as e:
    logging.debug(f"Cache write failed: {e}")

```

Method: MT5API.connect

```
def connect(self)
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns success = False.
2. Assigns cached_path = self._get_cached_terminal_path().
3. **if** cached_path: branches conditionally.
4. **if** not success: branches conditionally.
5. **if** not success: branches conditionally.
6. **if** not success: branches conditionally.
7. **if** not success: branches conditionally.
8. Assigns authorized = mt5.login(self.login, self.password, self.server).
9. **if** not authorized: branches conditionally.
10. **try** block with 1 **except** clause.
11. Assigns self.connected = True.
12. **return** True.
13. **try** block with 1 **except** clause.
14. **return** authorized.

Code (copy-paste safe)

```

def connect(self):
    # SPEED OPTIMIZATION: Try cached terminal path first
    success = False
    cached_path = self._get_cached_terminal_path()
    if cached_path:

```

```

        if mt5.initialize(path=cached_path):
            logging.info(f'[FAST] MT5 initialized with cached path: {cached_path}')
            success = True
        else:
            logging.info(f'Cached path failed, trying alternatives: {cached_path}')

# Try to initialize MT5 with the best available path
if not success:
    # CRITICAL FIX: If user specifies a terminal path, ONLY use that terminal
    if self.terminal_path and self.terminal_path.strip():
        terminal_exe = os.path.join(self.terminal_path, "terminal64.exe")
        if os.path.exists(terminal_exe):
            if mt5.initialize(path=terminal_exe):
                logging.info(f'MT5 initialized successfully with user-specified path: {terminal_exe}')
                self._cache_successful_path(terminal_exe) # Cache success
                success = True
            else:
                logging.error(f'MT5 initialize failed for user-specified path: {terminal_exe}')
                # Don't try other terminals when user specified one - respect their choice
                error_code, error_msg = mt5.last_error()
                self.last_error = f'MT5 initialize failed for selected terminal: {error_msg} (Code: {error_code})'
                logging.error(self.last_error)
                return False
        else:
            logging.error(f'User-specified terminal executable not found: {terminal_exe}')
            self.last_error = f'Terminal executable not found: {terminal_exe}'
            return False

# If specific paths failed, try other available installations
if not success:
    terminals = get_installed_mt5_terminals()
    for terminal in terminals:
        terminal_exe = os.path.join(terminal["path"], "terminal64.exe")
        if os.path.exists(terminal_exe):
            if mt5.initialize(path=terminal_exe):
                logging.info(f'MT5 initialized successfully with detected path: {terminal_exe}')
                self._cache_successful_path(terminal_exe) # Cache success
                success = True
                break
            else:
                logging.warning(f'MT5 initialize failed for detected path: {terminal_exe}')

# Last resort: try default initialization
if not success:
    if mt5.initialize():
        logging.info('MT5 initialized successfully with default path')
        success = True
    else:
        logging.error('MT5 initialize failed with default path')

if not success:
    error_code, error_msg = mt5.last_error()
    self.last_error = f'MT5 initialize failed: {error_msg} (Code: {error_code})'
    logging.error(self.last_error)
    return False

authorized = mt5.login(self.login, self.password, self.server)
if not authorized:
    error_msg = f'MT5 login failed for login={self.login}, server={self.server}'
    logging.error(error_msg)

```

```

        self.last_error = error_msg
        return False

# SPEED OPTIMIZATION: Fast symbol detection after successful connection
try:
    # Quick symbol detection - try user symbol first
    if self.symbol and mt5.symbol_select(self.symbol, True):
        self.connected_symbol = self.symbol
        logging.info(f"[FAST] Fast symbol detection: {self.connected_symbol}")
    else:
        # Quick fallback to first available symbol
        symbols = mt5.symbols_get()
        if symbols and len(symbols) > 0:
            self.connected_symbol = symbols[0].name
            if mt5.symbol_select(self.connected_symbol, True):
                logging.info(f"[FAST] Fast fallback symbol: {self.connected_symbol}")
            else:
                # Last resort: use EURUSD as default
                self.connected_symbol = "EURUSD"
                logging.info(f"[FAST] Default symbol: {self.connected_symbol}")
        else:
            self.connected_symbol = "EURUSD" # Safe default

except Exception as e:
    logging.warning(f"Fast symbol detection failed: {e}")
    self.connected_symbol = self.symbol if self.symbol else "EURUSD"

self.connected = True
return True

# After successful connection, detect and store the connected symbol
try:
    # Try to get the current symbol from the market watch or from a symbol list
    # First try to get symbols from the market watch
    symbols = mt5.symbols_get()
    if symbols and len(symbols) > 0:
        # Use the first available symbol from market watch as the connected symbol
        self.connected_symbol = symbols[0].name
        logging.info(f"Connected symbol detected: {self.connected_symbol}")

        # Try to select this symbol to ensure it's available
        if not mt5.symbol_select(self.connected_symbol, True):
            logging.warning(f"Could not select detected symbol: {self.connected_symbol}")
            # Try to find an alternative symbol
            for symbol in symbols[:5]: # Try first 5 symbols
                if mt5.symbol_select(symbol.name, True):
                    self.connected_symbol = symbol.name
                    logging.info(f"Alternative connected symbol selected: {self.connected_symbol}")
                    break
    else:
        # Fallback: try common symbol names if no symbols available
        common_symbols = ["EURUSD", "GBPUSD", "USDJPY", "USDCHF", "AUDUSD", "NZDUSD", "USDCAD"]
        for symbol in common_symbols:
            if mt5.symbol_select(symbol, True):
                self.connected_symbol = symbol
                logging.info(f"Fallback connected symbol selected: {self.connected_symbol}")
                break

except Exception as e:
    logging.warning(f"Could not detect connected symbol: {e}")

```

```

        # If detection fails, use the configured symbol if available
        if self.symbol:
            self.connected_symbol = self.symbol

    return authorized

```

Method: MT5API.monitor_connection

```
def monitor_connection(self)
```

Monitor MT5 connection status and attempt recovery if needed Call this periodically (e.g., every 30 seconds) during application operation

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.
2. Module/function docstring.
3. **try** block with 1 **except** clause.
4. Module/function docstring.
5. **for** attempt in range(max_retries): iterates.
6. Calls logging.error(...) for its side effect.
7. **return** False.
8. Module/function docstring.
9. **try** block with 1 **except** clause.
10. Module/function docstring.
11. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def monitor_connection(self):
    """
    Monitor MT5 connection status and attempt recovery if needed
    Call this periodically (e.g., every 30 seconds) during application operation
    """
    try:
        # Quick connection check
        terminal_info = mt5.terminal_info()
        if not terminal_info or not terminal_info.connected:
            logging.warning("■ MT5 connection monitor detected disconnection")
            return self.attempt_reconnection()

        # Check if trading is still allowed
        if not terminal_info.trade_allowed:

```

```

        logging.warning("■ MT5 connection monitor detected trading disabled")
        return False

    # Check account access
    account_info = mt5.account_info()
    if not account_info:
        logging.warning("■ MT5 connection monitor detected account access issues")
        return self.attempt_reconnection()

    return True

except Exception as e:
    logging.error(f"Error in connection monitor: {e}")
    return False
"""
Ensure MT5 session is properly initialized and maintained
Call this at application startup and periodically during operation
"""
try:
    logging.info("[SETUP] Ensuring MT5 session integrity...")

    # Check if MT5 is initialized
    if not mt5.initialize():
        logging.warning("MT5 not initialized, attempting to initialize...")
        if not mt5.initialize():
            logging.error("[ERROR] Failed to initialize MT5")
            return False

    # Check if we're logged in
    if not mt5.terminal_info():
        logging.warning("MT5 terminal info not available, attempting connection...")
        if not self.connect():
            logging.error("[ERROR] Failed to connect to MT5")
            return False

    # Perform comprehensive health check
    is_healthy, health_msg = self.check_connection_health()
    if not is_healthy:
        logging.warning(f"MT5 health check failed: {health_msg}, attempting recovery...")
        if not self.attempt_reconnection():
            logging.error("[ERROR] MT5 recovery failed")
            return False

    # Ensure symbol is properly selected
    if self.symbol:
        symbol_info = mt5.symbol_info(self.symbol)
        if symbol_info and not symbol_info.visible:
            logging.info(f"Ensuring symbol {self.symbol} is selected...")
            mt5.symbol_select(self.symbol, True)

    logging.info("[OK] MT5 session integrity confirmed")
    return True

except Exception as e:
    logging.error(f"Error ensuring MT5 session: {e}")
    return False
"""
Attempt to reconnect to MT5 if connection is lost
"""
for attempt in range(max_retries):

```

```

try:
    logging.info(f"■ Attempting MT5 reconnection (attempt {attempt + 1}/{max_retries})...")

    # Shutdown current connection
    mt5.shutdown()

    # Wait a moment
    time.sleep(1)

    # Try to reconnect
    if self.connect():
        # Verify the reconnection worked
        is_healthy, health_msg = self.check_connection_health()
        if is_healthy:
            logging.info("[OK] MT5 reconnection successful")
            return True
        else:
            logging.warning(f"Reconnection completed but health check failed: {health_msg}")
    else:
        logging.warning(f"Reconnection attempt {attempt + 1} failed")

except Exception as e:
    logging.error(f"Error during reconnection attempt {attempt + 1}: {e}")

logging.error(f"[ERROR] All {max_retries} reconnection attempts failed")
return False
"""

Comprehensive health check for MT5 connection and trading readiness
Returns (is_healthy, error_message)
"""
try:
    logging.info("■ Performing MT5 connection health check...")

    # 1. Check MT5 initialization
    if not mt5.initialize():
        return False, "MT5 not initialized"

    # 2. Check terminal connection
    terminal_info = mt5.terminal_info()
    if not terminal_info:
        return False, "Cannot get terminal info"

    if not terminal_info.connected:
        return False, "MT5 terminal not connected"

    if not terminal_info.trade_allowed:
        return False, "Trading not allowed in MT5 terminal"

    # 3. Check account access
    account_info = mt5.account_info()
    if not account_info:
        return False, "Cannot access account info"

    # 4. Check symbol availability (using configured symbol)
    if self.symbol:
        symbol_info = mt5.symbol_info(self.symbol)
        if not symbol_info:
            return False, f"Symbol {self.symbol} not found in MT5"

        if not symbol_info.visible:

```

```

        return False, f"Symbol {self.symbol} not visible in Market Watch"

    # 5. Check tick data availability
    tick = mt5.symbol_info_tick(self.symbol)
    if not tick or tick.bid <= 0 or tick.ask <= 0:
        return False, f"No live tick data for symbol {self.symbol}"

    logging.info("[OK] MT5 connection health check passed")
    return True, "All systems operational"

except Exception as e:
    error_msg = f"Health check failed: {e}"
    logging.error(f"[ERROR] {error_msg}")
    return False, error_msg
"""
Verify MT5 connection is stable and ready for trading
"""
try:
    # Check terminal info
    terminal_info = mt5.terminal_info()
    if not terminal_info:
        logging.error("MT5 terminal_info() returned None")
        return False

    if not terminal_info.connected:
        logging.error("MT5 terminal not connected")
        return False

    if not terminal_info.trade_allowed:
        logging.error("MT5 trading not allowed")
        return False

    # Check account info
    account_info = mt5.account_info()
    if not account_info:
        logging.error("MT5 account_info() returned None")
        return False

    logging.info(
        "[OK] MT5 connection verified - terminal connected, trading allowed, account accessible"
    )
    return True

except Exception as e:
    logging.error(f"Error verifying MT5 connection: {e}")
    return False

```

Method: MT5API._verify_symbol_tick_data

```
def _verify_symbol_tick_data(self, symbol, max_retries=3)
```

Verify symbol has live tick data available

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **for** attempt in range(max_retries): iterates.
2. Calls logging.error(...) for its side effect.
3. **return** False.

Code (copy-paste safe)

```
def _verify_symbol_tick_data(self, symbol, max_retries=3):
    """
    Verify symbol has live tick data available
    """
    for attempt in range(max_retries):
        try:
            tick = mt5.symbol_info_tick(symbol)
            if tick and tick.bid > 0 and tick.ask > 0:
                logging.info(f"[OK] Tick data verified for {symbol}: bid={tick.bid}, ask={tick.ask}")
                return True

            if attempt < max_retries - 1:
                logging.warning(
                    f"Tick data not available for {symbol} (attempt {attempt + 1}/{max_retries}), ret
                    time.sleep(0.1) # Brief delay before retry

        except Exception as e:
            logging.error(f"Error getting tick data for {symbol}: {e}")
            if attempt < max_retries - 1:
                time.sleep(0.1)

    logging.error(f"[ERROR] No tick data available for {symbol} after {max_retries} attempts")
    return False
```

Method: MT5API.is_autotrading_enabled

```
def is_autotrading_enabled(self)
```

Check if AutoTrading is enabled in MT5 using the existing connection Returns True if autotrading is enabled, False otherwise

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def is_autotrading_enabled(self):
```



```

"""
Check if AutoTrading is enabled in MT5 using the existing connection
Returns True if autotrading is enabled, False otherwise
"""
try:
    # Don't initialize a new connection - use the existing one
    if not self.connected:
        print("[ERROR] MT5 not connected - cannot check AutoTrading status")
        return False

    # Use the already connected MT5 instance to check terminal info
    term_info = mt5.terminal_info()
    if not term_info:
        print("[ERROR] Could not get terminal info from existing MT5 connection")
        return False

    # Check AutoTrading status using the connected instance
    print("■ Checking AutoTrading status on existing MT5 connection...")

    # Basic status checks
    connected = getattr(term_info, 'connected', False)
    trade_allowed = getattr(term_info, 'trade_allowed', False)
    tradeapi_disabled = getattr(term_info, 'tradeapi_disabled', True)
    dlls_allowed = getattr(term_info, 'dlls_allowed', False)

    # Log the current status
    print(f"[CHECK] Connected: {connected}")
    print(f"[CHECK] Trade Allowed: {trade_allowed}")
    print(f"[CHECK] Trade API Disabled: {tradeapi_disabled}")
    print(f"[CHECK] DLLs Allowed: {dlls_allowed}")

    # AutoTrading is enabled if:
    # 1. MT5 is connected
    # 2. Trade is allowed (main AutoTrading setting)
    # 3. Trade API is not disabled
    autotrading_enabled = connected and trade_allowed and not tradeapi_disabled

    print(f"[RESULT] AutoTrading enabled: {autotrading_enabled}")
    return autotrading_enabled

except Exception as e:
    print(f"[ERROR] AutoTrading status check failed: {e}")
    logging.error(f"Error checking auto trading status: {e}")
    return False

```

Method: MT5API.ensure_symbol

```
def ensure_symbol(self, symbol)
```

Ensure symbol is available for trading, with enhanced caching and fast paths

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def ensure_symbol(self, symbol):
    """Ensure symbol is available for trading, with enhanced caching and fast paths"""
    try:
        # Validate input symbol
        if not symbol or symbol.strip() == "":
            logging.error(f"Invalid symbol provided: '{symbol}'")
            raise Exception(f"Invalid symbol provided: '{symbol}'")

        symbol = symbol.strip()

        # SPEED OPTIMIZATION: Check symbol cache first
        cache_key = f"{self.server}_{symbol}"
        current_time = time.time()

        if (cache_key in self._symbol_cache and
            cache_key in self._symbol_cache_timestamp and
            current_time - self._symbol_cache_timestamp[cache_key] < self._cache_ttl):

            cached_symbol = self._symbol_cache[cache_key]
            logging.info(f"[FAST] SPEED: Using cached symbol {symbol} → {cached_symbol}")
            return cached_symbol

        # First check if MT5 is connected
        if not mt5.terminal_info():
            logging.error("MT5 terminal not connected")
            # CRITICAL: Don't return symbol if MT5 is not connected - this causes trading failures
            logging.error("Cannot validate symbol - MT5 terminal not connected")
            return None

        # PRIORITY: Since users provide correct symbol names, try their symbol first
        logging.info(f"[TARGET] USER SYMBOL: Trying user-provided symbol '{symbol}' first")
        symbol_info = mt5.symbol_info(symbol)
        if symbol_info:
            tick = mt5.symbol_info_tick(symbol)
            if tick and (tick.bid > 0 or tick.ask > 0):
                logging.info(f"[OK] USER SYMBOL WORKS: '{symbol}' has active tick data")
                # Cache the successful result
                self._symbol_cache[cache_key] = symbol
                self._symbol_cache_timestamp[cache_key] = current_time
                return symbol
            else:
                # Symbol exists but no tick data - CRITICAL: Don't proceed with trading
                logging.error(f"[ERROR] Symbol {symbol} exists but no tick data available - cannot tr
                return None

        # Try to select the symbol if it wasn't found
        logging.info(f"[SIGNAL] SELECTING SYMBOL: Attempting to activate '{symbol}'")
        select_result = mt5.symbol_select(symbol, True)

        # Check again after selection
        symbol_info = mt5.symbol_info(symbol)
```

```

if symbol_info:
    tick = mt5.symbol_info_tick(symbol)
    if tick and (tick.bid > 0 or tick.ask > 0):
        logging.info(f"[OK] SYMBOL ACTIVATED: '{symbol}' now has active tick data")
        self._symbol_cache[cache_key] = symbol
        self._symbol_cache_timestamp[cache_key] = current_time
        return symbol
    else:
        # Symbol selected but no tick - still return for trading attempt
        logging.info(f"[WARNING] SYMBOL SELECTED: '{symbol}' activated but no tick data yet")
        self._symbol_cache[cache_key] = symbol
        self._symbol_cache_timestamp[cache_key] = current_time
        return symbol

# SPEED OPTIMIZATION: For known symbols, try direct approach first
if symbol.upper() in ['USTECH', 'USTEC', 'XAUUSD', 'NAS100', 'NASDAQ']:
    # Try the symbol directly first (fastest path)
    symbol_info = mt5.symbol_info(symbol)
    if symbol_info:
        tick = mt5.symbol_info_tick(symbol)
        if tick and (tick.bid > 0 or tick.ask > 0):
            logging.info(f"[FAST] SPEED: Direct symbol access successful for {symbol}")
            # Cache the successful result
            self._symbol_cache[cache_key] = symbol
            self._symbol_cache_timestamp[cache_key] = current_time
            return symbol

# Get symbol variations to try (only if direct access failed)
symbol_variations = self._get_symbol_variations(symbol)
logging.info(f"[SEARCH] VARIATIONS: Trying {len(symbol_variations)} variations for '{symbol}'")

# SPEED OPTIMIZATION: Try most likely variations first
priority_variations = []
other_variations = []

for variation in symbol_variations:
    # Prioritize exact matches and simple variations
    if (variation == symbol or
        variation == symbol.upper() or
        variation in ['USTECH', 'USTEC', 'XAUUSD']):
        priority_variations.append(variation)
    else:
        other_variations.append(variation)

# Try priority variations first, then others
all_variations = priority_variations + other_variations[:20] # Limit to first 20 of others

for variation in all_variations:
    try:
        # First check if this variation has symbol info
        var_info = mt5.symbol_info(variation)
        if not var_info:
            logging.debug(f"Symbol variation {variation} not found")
            continue

        # Check if it already has tick data (means it's working)
        tick = mt5.symbol_info_tick(variation)
        if tick and (tick.bid > 0 or tick.ask > 0):
            logging.info(
                f"[FAST] SPEED: Symbol variation {variation} already has active tick data")

```

```

        self._symbol_cache[cache_key] = variation
        self._symbol_cache_timestamp[cache_key] = current_time
        return variation

    # Try to select the symbol (but don't fail if this returns False)
    # Some brokers return False even when symbol is already available
    select_result = mt5.symbol_select(variation, True)
    logging.debug(f"Symbol select result for {variation}: {select_result}")

    # After selection attempt, check again for tick data
    tick_after = mt5.symbol_info_tick(variation)
    if tick_after and (tick_after.bid > 0 or tick_after.ask > 0):
        logging.info(f"[OK] VARIATION SUCCESS: '{variation}' now has active tick data")
        self._symbol_cache[cache_key] = variation
        self._symbol_cache_timestamp[cache_key] = current_time
        return variation

    # If still no tick data, but symbol info exists, it might still work for some operations
    if var_info and var_info.visible:
        logging.info(
            f"[WARNING] VARIATION VISIBLE: '{variation}' is visible but no current tick data"
        )
        self._symbol_cache[cache_key] = variation
        self._symbol_cache_timestamp[cache_key] = current_time
        return variation

    except Exception as e:
        logging.debug(f"Error trying symbol variation {variation}: {e}")
        continue

    # CRITICAL: Don't return symbol as fallback if no valid symbol was found
    logging.error(f"[FAILED] No valid symbol found for {symbol} - cannot proceed with trading")
    return None

    except Exception as e:
        logging.error(f"Error in ensure_symbol for '{symbol}': {e}")

    # CRITICAL: Always return user's symbol to prevent None errors
    # Users are expected to provide correct symbol names for their broker
    logging.warning(f"■ EMERGENCY FALLBACK: Returning user symbol '{symbol}' despite errors")
    return symbol

```

Method: MT5API._get_symbol_variations

```
def _get_symbol_variations(self, symbol)
```

Get possible symbol variations for different MT5 brokers

Step by step (top-level body)

1. Assigns variations = [symbol].
2. Assigns symbol_upper = symbol.upper().
3. if symbol_upper in ['USTEC', 'NASDAQ', 'NQ', 'NAS', 'USTECH100', 'USTE...: branches conditionally (with an **else/elif** arm).
4. Assigns seen = set().
5. Assigns result = [].
6. for variant in variations: iterates.

7. return result.

Code (copy-paste safe)

```
def _get_symbol_variations(self, symbol):
    """Get possible symbol variations for different MT5 brokers"""
    # Start with the original symbol
    variations = [symbol]

    # Add common variations based on symbol type
    symbol_upper = symbol.upper()

    # NASDAQ variations - comprehensive list based on actual MT5 charts
    if symbol_upper in ['USTEC', 'NASDAQ', 'NQ', 'NAS', 'USTECH100', 'USTECH', 'NAS100', 'NDX',
                       'NASDAQ100', 'TECH100', 'US100', 'SPX500']:
        variations.extend([
            # Primary NASDAQ symbols
            'USTEC', 'USTECH100', 'USTECH', 'NAS100', 'NASDAQ', 'NQ', 'NDX', 'NASDAQ100',
            'US100', 'TECH100', 'USTEC100', 'NASTECH', 'NASDAQTECH',

            # Suffixed variations (.m, m, -Z, etc.)
            'USTEC.m', 'USTECH100.m', 'USTECH.m', 'NAS100.m', 'NASDAQ.m', 'NQ.m', 'NDX.m',
            'USTECm', 'USTECH100m', 'USTECHm', 'NAS100m', 'NASDAQm', 'NQm', 'NDXm',
            'USTEC-Z', 'USTECH100-Z', 'USTECH-Z', 'NAS100-Z', 'NASDAQ-Z', 'NQ-Z', 'NDX-Z',

            # Broker-specific variations
            'USTECfx', 'USTECH100fx', 'USTECHfx', 'NAS100fx', 'NASDAQfx',
            'USTEC_c', 'USTECH_c', 'NAS100_c', 'NASDAQ_c',
            'USTEC.c', 'USTECH.c', 'NAS100.c', 'NASDAQ.c',

            # Alternative naming patterns
            'US_TECH', 'US-TECH', 'USTECH.', 'USTEC.', 'NAS100.',
            'USTECH100.', 'NASDAQ100.', 'TECH-100', 'TECH_100',

            # Contract-specific variations (futures style)
            'USTECH2024', 'USTEC2024', 'NAS2024', 'USTECH24', 'USTEC24', 'NAS24',
            'USTECHM24', 'USTECM24', 'NASM24', 'USTECHZ24', 'USTECZ24', 'NASZ24',

            # Additional broker variations
            'USTEC100', 'NASTECH100', 'USNASDAQ', 'NASDAQ_100', 'NASDAQ-100',
            'USTEC_100', 'USTEC-100', 'USTECH_100', 'USTECH-100',

            # Dot variations
            'USTEC.', 'USTECH.', 'NASDAQ.', 'NAS100.', 'NQ.',

            # Undercore variations
            'USTEC_', 'USTECH_', 'NASDAQ_', 'NAS100_', 'NQ_'
        ])

    # Gold variations - comprehensive list for different brokers
    elif symbol_upper in ['XAUUSD', 'GOLD', 'GLD', 'XAU']:
        variations.extend([
            'XAUUSD', 'GOLD', 'XAU', 'GOLDUSD', 'XAUUSD.',
            'XAUUSD.m', 'GOLD.m', 'XAUUSDm', 'GOLDm',
            'XAUUSD-Z', 'GOLD-Z', 'XAU/USD', 'GOLD/USD',
            'XAUUSDfx', 'GOLDfx', 'XAUUSD_MT5'
        ])

    # Oil variations
    elif symbol_upper in ['USOIL', 'OIL', 'CRUDE']:
```

```

        variations.extend(['USOIL', 'CRUDE', 'OIL', 'WTI', 'BRENT'])

# Forex pairs - try both with and without suffixes
elif len(symbol) == 6 and symbol_upper.endswith('USD'):
    base_pair = symbol_upper[:6]
    variations.extend([base_pair, base_pair + '.', base_pair + 'm', base_pair + 'c'])

# Remove duplicates while preserving order
seen = set()
result = []
for variant in variations:
    if variant not in seen:
        seen.add(variant)
        result.append(variant)

return result

```

Method: MT5API._log_available_symbols

```
def _log_available_symbols(self, failed_symbol)
```

Log some available symbols for debugging

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with `append`, and clearer about intent: this is a transform, not a side-effecting loop.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to `sum`, `any`, `all`, or `max` where the intermediate list would be wasted.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def _log_available_symbols(self, failed_symbol):
    """Log some available symbols for debugging"""
    try:
        # Get all available symbols
        symbols = mt5.symbols_get()
        if symbols:
            # Log total count
            logging.info(f"Failed symbol: {failed_symbol}")
            logging.info(f"Total available symbols: {len(symbols)}")

            # Log first 20 symbols
            symbol_names = [s.name for s in symbols[:20]]
            logging.info(f"Available symbols (first 20): {symbol_names}")

            # Look for symbols containing parts of the failed symbol

```

```

        failed_upper = failed_symbol.upper()
        similar = []
        for s in symbols:
            symbol_name = s.name.upper()
            # Check for partial matches
            if (failed_upper[:3] in symbol_name or
                symbol_name[:3] in failed_upper or
                'XAU' in symbol_name or
                'GOLD' in symbol_name or
                'USTEC' in symbol_name or
                'NAS' in symbol_name):
                similar.append(s.name)

        if similar:
            logging.info(f"Similar/related symbols found: {similar[:10]}") # Show max 10

            # Check specifically for gold and nasdaq symbols
            gold_symbols = [s.name for s in symbols if any(x in s.name.upper() for x in ['XAU', 'GOLD'])]
            nasdaq_symbols = [s.name for s in symbols if any(x in s.name.upper() for x in ['USTEC', 'NASDAQ',
                                                                                          'NDX'])]

            if gold_symbols:
                logging.info(f"Gold-related symbols: {gold_symbols}")
            if nasdaq_symbols:
                logging.info(f"NASDAQ-related symbols: {nasdaq_symbols}")

        else:
            logging.warning("No symbols available - check MT5 connection")
    except Exception as e:
        logging.warning(f"Could not retrieve available symbols: {e}")

```

Method: MT5API.get_connected_symbol

```
def get_connected_symbol(self)
```

Get the symbol that was detected during connection

Step by step (top-level body)

1. return self.connected_symbol.

Code (copy-paste safe)

```

def get_connected_symbol(self):
    """Get the symbol that was detected during connection"""
    return self.connected_symbol

```

Method: MT5API.get_safe_symbol

```
def get_safe_symbol(self, fallback_symbol=None)
```

Get a safe symbol to use - prefers fallback (configured), then connected symbol, then configured symbol

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **if** `fallback_symbol`: branches conditionally.
2. **if** `self.connected_symbol`: branches conditionally (with an **else/elif** arm).

Code (copy-paste safe)

```
def get_safe_symbol(self, fallback_symbol=None):
    """Get a safe symbol to use - prefers fallback (configured), then connected symbol, then configured symbol
    # Priority 1: Use the explicitly requested/configured symbol if provided and available
    if fallback_symbol:
        # Try to select the requested symbol to ensure it's available
        if mt5.symbol_select(fallback_symbol, True):
            return fallback_symbol
        else:
            logging.warning(
                f'Requested symbol {fallback_symbol} not available, falling back to connected symbol'
            )
    # Priority 2: Use connected symbol if no specific symbol requested or if requested symbol unavailable
    if self.connected_symbol:
        return self.connected_symbol
    elif self.symbol:
        return self.symbol
    else:
        # Ultimate fallback
        return "EURUSD"
```

Method: `MT5API.get_supported_filling_modes`

```
def get_supported_filling_modes(self, symbol)
```

Step by step (top-level body)

1. Assigns `info = mt5.symbol_info(symbol)`.
2. **if** `info` is `None`: branches conditionally.
3. Assigns `fillings = getattr(info, 'trade_fillings', None)`.
4. **if** not `fillings` or `len(fillings) == 0`: branches conditionally.
5. **return** `list(fillings)`.

Code (copy-paste safe)

```
def get_supported_filling_modes(self, symbol):
    info = mt5.symbol_info(symbol)
    if info is None:
        return [mt5.ORDER_FILLING_IOC]
    fillings = getattr(info, "trade_fillings", None)
    if not fillings or len(fillings) == 0:
        return [mt5.ORDER_FILLING_IOC, mt5.ORDER_FILLING_FOK, mt5.ORDER_FILLING_RETURN]
    return list(fillings)
```

Method: `MT5API.check_connection_health`

```
def check_connection_health(self)
```

Comprehensive health check for MT5 connection and trading readiness Returns (is_healthy, error_message)

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def check_connection_health(self):
    """
    Comprehensive health check for MT5 connection and trading readiness
    Returns (is_healthy, error_message)
    """
    try:
        logging.info("■ Performing MT5 connection health check...")

        # 1. Check MT5 initialization
        if not mt5.initialize():
            return False, "MT5 not initialized"

        # 2. Check terminal connection
        terminal_info = mt5.terminal_info()
        if not terminal_info:
            return False, "Cannot get terminal info"

        if not terminal_info.connected:
            return False, "MT5 terminal not connected"

        if not terminal_info.trade_allowed:
            return False, "Trading not allowed in MT5 terminal"

        # 3. Check account access
        account_info = mt5.account_info()
        if not account_info:
            return False, "Cannot access account info"

        # 4. Check symbol availability (using configured symbol)
        if self.symbol:
            symbol_info = mt5.symbol_info(self.symbol)
            if not symbol_info:
                return False, f"Symbol {self.symbol} not found in MT5"

            if not symbol_info.visible:
                return False, f"Symbol {self.symbol} not visible in Market Watch"

        # 5. Check tick data availability
        tick = mt5.symbol_info_tick(self.symbol)
        if not tick or tick.bid <= 0 or tick.ask <= 0:
            return False, f"No live tick data for symbol {self.symbol}"

        logging.info("[OK] MT5 connection health check passed")
        return True, "All systems operational"

    except Exception as e:
        error_msg = f"Health check failed: {e}"
```

```
logging.error(f"[ERROR] {error_msg}")
return False, error_msg
```

Method: MT5API.attempt_reconnection

```
def attempt_reconnection(self, max_retries=3)
```

Attempt to reconnect to MT5 if connection is lost

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **for** attempt in range(max_retries): iterates.
2. Calls logging.error(...) for its side effect.
3. **return** False.

Code (copy-paste safe)

```
def attempt_reconnection(self, max_retries=3):
    """
    Attempt to reconnect to MT5 if connection is lost
    """
    for attempt in range(max_retries):
        try:
            logging.info(f"■ Attempting MT5 reconnection (attempt {attempt + 1}/{max_retries})...")

            # Shutdown current connection
            mt5.shutdown()

            # Wait a moment
            time.sleep(1)

            # Try to reconnect
            if self.connect():
                # Verify the reconnection worked
                is_healthy, health_msg = self.check_connection_health()
                if is_healthy:
                    logging.info("[OK] MT5 reconnection successful")
                    return True
                else:
                    logging.warning(f"Reconnection completed but health check failed: {health_msg}")
            else:
                logging.warning(f"Reconnection attempt {attempt + 1} failed")

        except Exception as e:
            logging.error(f"Error during reconnection attempt {attempt + 1}: {e}")

    logging.error(f"[ERROR] All {max_retries} reconnection attempts failed")
    return False
```

Method: MT5API._calculate_sl_tp_price

```
def _calculate_sl_tp_price(self, symbol, order_type, price, sl_points, tp_points)
```

Calculate SL and TP prices from points - NASDAQ automation always uses 1.0 point value

Why these Python concepts were used

- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `sl_price = None`.
2. Assigns `tp_price = None`.
3. Assigns `symbol_info = mt5.symbol_info(symbol)`.
4. **if not** `symbol_info`: branches conditionally.
5. Assigns `point = symbol_info.point`.
6. **if any**((`x in symbol.upper()` for `x in ['XAU', 'GOLD', 'GC']`)): branches conditionally (with an **else/elif** arm).
7. **if** `sl_points` and `float(sl_points) > 0`: branches conditionally.
8. **if** `tp_points` and `float(tp_points) > 0`: branches conditionally.
9. **return** (`sl_price`, `tp_price`).

Code (copy-paste safe)

```
def _calculate_sl_tp_price(self, symbol, order_type, price, sl_points, tp_points):
    """Calculate SL and TP prices from points - NASDAQ automation always uses 1.0 point value"""
    sl_price = None
    tp_price = None

    # Get symbol info for logging purposes
    symbol_info = mt5.symbol_info(symbol)
    if not symbol_info:
        logging.error(f"Could not get symbol info for {symbol}")
        return sl_price, tp_price

    # Get the minimum tick size for reference
    point = symbol_info.point

    # NASDAQ automation: ALWAYS use 1.0 point value regardless of symbol name or tick size
    # EXCEPTION: For XAUUSD (Gold), use the actual point value (usually 0.01) as the blueprint points
    # are calibrated for standard ticks (e.g. 1710 points = $17.10 price movement)
    if any(x in symbol.upper() for x in ['XAU', 'GOLD', 'GC']):
        point_value = point
        logging.info(f"Gold symbol detected ({symbol}), using actual point value: {point_value}")
    else:
        point_value = 1.0
        logging.info(
            f"Symbol {symbol} - tick size: {point}, NASDAQ automation point value: {point_value}, pri
```

```

if sl_points and float(sl_points) > 0:
    sl_points_float = float(sl_points)
    price_difference = sl_points_float * point_value

    if order_type == "buy":
        sl_price = price - price_difference
    else: # sell
        sl_price = price + price_difference

    logging.info(
        f"SL calculation: {sl_points_float} points × {point_value} = {price_difference} price dif

if tp_points and float(tp_points) > 0:
    tp_points_float = float(tp_points)
    price_difference = tp_points_float * point_value

    if order_type == "buy":
        tp_price = price + price_difference
    else: # sell
        tp_price = price - price_difference

    logging.info(
        f"TP calculation: {tp_points_float} points × {point_value} = {price_difference} price dif

return sl_price, tp_price

```

Method: MT5API.place_order

```
def place_order(self, symbol, order_type, volume=None, sl=None, tp=None, comment=None)
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **list comprehension** — Builds a list in a single expression ([f(x) for x in xs]). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. try block with 1 **except** clause.

Code (copy-paste safe)

```
def place_order(self, symbol, order_type, volume=None, sl=None, tp=None, comment=None):
```

```

try:
    # PRE-TRADE HEALTH CHECK: Ensure MT5 is ready for trading
    is_healthy, health_error = self.check_connection_health()
    if not is_healthy:
        logging.error(f"[ERROR] Pre-trade health check failed: {health_error}")
        # Attempt reconnection
        logging.info("■ Attempting automatic reconnection...")
        if self.attempt_reconnection():
            # Re-check health after reconnection
            is_healthy, health_error = self.check_connection_health()
            if not is_healthy:
                raise Exception(f"MT5 reconnection failed: {health_error}")
        else:
            raise Exception(f"MT5 health check failed and reconnection unsuccessful: {health_error}")

    # Validate input parameters
    if not symbol or symbol.strip() == "":
        logging.error(f"Invalid symbol provided to place_order: '{symbol}'")
        raise Exception(f"Invalid symbol provided: '{symbol}'")

    symbol = symbol.strip()

    # Ensure symbol is available and get the correct symbol name
    corrected_symbol = self.ensure_symbol(symbol)

    # CRITICAL: Validate that ensure_symbol didn't return None
    if not corrected_symbol:
        logging.error(
            f"ensure_symbol returned None for '{symbol}' - MT5 connection or symbol validation failed")
        raise Exception(
            f"Symbol validation failed for '{symbol}' - check MT5 connection and symbol availability")

    # Log order details with corrected symbol
    logging.info(
        f"■ PLACING ORDER: {order_type.upper()} {corrected_symbol}, volume={volume}, sl={sl}, tp={tp}")

    # Debug symbol info to understand MT5 properties (only log once per symbol)
    if not hasattr(self, '_debugged_symbols'):
        self._debugged_symbols = set()
    if corrected_symbol not in self._debugged_symbols:
        self.debug_symbol_info(corrected_symbol)
        self._debugged_symbols.add(corrected_symbol)

    tick = mt5.symbol_info_tick(corrected_symbol)
    print(f"[SEARCH] TICK RETRIEVAL DEBUG for {corrected_symbol}:")
    print(f"    Raw tick result: {tick}")
    if tick:
        print(f"    Tick ask: {tick.ask}, bid: {tick.bid}")
        print(f"    Tick time: {tick.time}")
    else:
        print(f"    [ERROR] Tick is None - investigating...")

    # Check MT5 initialization
    if not mt5.initialize():
        print(f"    [ERROR] MT5 not initialized - attempting to initialize...")
    if mt5.initialize():
        print(f"    [OK] MT5 initialization successful")
        # Try getting tick again after initialization
        tick = mt5.symbol_info_tick(corrected_symbol)
        print(f"    Retry after init: {tick}")

```

```

        else:
            print(f"    [ERROR] MT5 initialization failed")

# Check terminal connection
terminal_info = mt5.terminal_info()
print(f"    Terminal info: {terminal_info}")
if terminal_info:
    print(f"    Terminal connected: {terminal_info.connected}")
    print(f"    Terminal trade allowed: {terminal_info.trade_allowed}")

# Check if symbol exists in symbol_info
symbol_info = mt5.symbol_info(corrected_symbol)
print(f"    Symbol info: {symbol_info}")
if symbol_info:
    print(f"    Symbol visible: {symbol_info.visible}")
    print(f"    Symbol selected: {symbol_info.select}")

# Try to select the symbol explicitly
if not symbol_info.visible:
    print(f"    Attempting to select symbol...")
    select_result = mt5.symbol_select(corrected_symbol, True)
    print(f"    Symbol select result: {select_result}")
    if select_result:
        # Try getting tick again after selecting
        tick = mt5.symbol_info_tick(corrected_symbol)
        print(f"    Tick after select: {tick}")

if tick is None:
    # Enhanced debugging for symbol issues
    logging.error(f"[ERROR] SYMBOL PRICE FETCH FAILED: {corrected_symbol}")

# Check if symbol is selected in Market Watch
selected = mt5.symbol_select(corrected_symbol, True)
logging.error(f"[SEARCH] Symbol select result: {selected}")

# Check symbol info
symbol_info = mt5.symbol_info(corrected_symbol)
if symbol_info:
    logging.error(
        f"[SEARCH] Symbol info exists: visible={symbol_info.visible}, tradeable={symbol_info.tradeable}")
else:
    logging.error(f"[SEARCH] Symbol info is None - symbol may not exist")

# Try to get symbols that match pattern
matching_symbols = mt5.symbols_get(group=f"*{corrected_symbol}*")
if matching_symbols:
    logging.error(f"[SEARCH] Found {len(matching_symbols)} matching symbols:")
    for sym in matching_symbols[:5]: # Show first 5 matches
        logging.error(f"    - {sym.name}")
else:
    logging.error(f"[SEARCH] No symbols found matching pattern *{corrected_symbol}*")

# Enhanced symbol debugging for "Could not get price" issues
print(f"[SEARCH] SYMBOL DEBUG: No price data for {corrected_symbol}")
print(f"    Checking symbol selection and market watch...")

# Check if symbol is in Market Watch
market_watch_symbols = mt5.symbols_get()
if market_watch_symbols:
    symbol_names = [s.name for s in market_watch_symbols]

```

```

        if corrected_symbol not in symbol_names:
            print(f"[ERROR] SYMBOL ERROR: {corrected_symbol} not in Market Watch")
            print(f"    Available symbols: {symbol_names[:10]}") # Show first 10
        else:
            print(f"[OK] SYMBOL FOUND: {corrected_symbol} is in Market Watch")

# Try exact symbol matching
exact_match = mt5.symbol_info(corrected_symbol)
if not exact_match:
    print(f"[ERROR] EXACT MATCH FAILED: {corrected_symbol} not found")
    # Try pattern matching for similar symbols
    if market_watch_symbols:
        similar_symbols = [s.name for s in market_watch_symbols
                           if corrected_symbol.lower() in s.name.lower() or s.name.lower()
                              in corrected_symbol.lower()]

        if similar_symbols:
            print(f"[SEARCH] SIMILAR SYMBOLS: {similar_symbols}")
    else:
        print(f"[OK] SYMBOL EXISTS: {corrected_symbol} found in MT5")

    raise Exception(
        f"Could not get price for {corrected_symbol} - MT5 connection issue or symbol not rec

# SUCCESS: We have a valid tick, now extract the price
price = tick.ask if order_type == "buy" else tick.bid
print(f"[OK] PRICE EXTRACTED: {order_type} price for {corrected_symbol} = {price}")
print(
    f"    Full tick data: ask={tick.ask}, bid={tick.bid}, spread={tick.ask - tick.bid if tick

# Validate price is reasonable
if price <= 0:
    print(f"[ERROR] INVALID PRICE: {price} <= 0")
    raise Exception(f"Invalid price {price} for {corrected_symbol}")

type_mt5 = mt5.ORDER_TYPE_BUY if order_type == "buy" else mt5.ORDER_TYPE_SELL
supported_fillings = self.get_supported_filling_modes(corrected_symbol)
last_error = None

if sl is None:
    sl = self.sl_points
if tp is None:
    tp = self.tp_points
if volume is None:
    volume = self.default_volume

# Apply PlexyTrade lot size adjustment - ONLY for USTECH (Nasdaq)
# XAUUSD (Gold) pip values are consistent across brokers, so no division needed
if self.is_plexy_server and volume > 0:
    # Only divide lot size for USTECH/Nasdaq symbols
    if any(x in corrected_symbol.upper() for x in ['USTECH', 'USTEC', 'NAS', 'NASDAQ', 'NDX',
        'NQ']):
        original_volume = volume
        volume = volume / 20.0
        logging.info(
            f"PlexyTrade adjustment for {corrected_symbol}: {original_volume} -> {volume} lot
    else:
        logging.info(
            f"PlexyTrade: No lot size adjustment for {corrected_symbol} (only divide USTECH,

# Ensure SL and TP are always set (never skip) - use defaults if 0

```

```

if sl is None or float(sl) <= 0:
    sl = self.sl_points if self.sl_points > 0 else 10 # Default 10 points if not set
    logging.info(f"Using default/minimum SL: {sl} points")
if tp is None or float(tp) <= 0:
    tp = self.tp_points if self.tp_points > 0 else 20 # Default 20 points if not set
    logging.info(f"Using default/minimum TP: {tp} points")

# Convert to float and ensure they're positive
sl = float(sl)
tp = float(tp)
volume = float(volume)

# Normalize volume to meet symbol requirements (step size, min, max)
sym_info = mt5.symbol_info(corrected_symbol)
if sym_info:
    step_vol = sym_info.volume_step
    min_vol = sym_info.volume_min
    max_vol = sym_info.volume_max

    if step_vol > 0:
        # Round to nearest multiple of step_vol
        volume = round(volume / step_vol) * step_vol

        # Fix floating point precision
        try:
            step_str = f"{step_vol:.10f}".rstrip('0').rstrip('.')
            decimals = 0
            if "." in step_str:
                decimals = len(step_str.split(".")[1])
            volume = round(volume, decimals)
        except Exception:
            volume = round(volume, 2)

    if min_vol > 0 and volume < min_vol:
        logging.warning(f"Volume {volume} < min {min_vol}, clamped to min")
        volume = min_vol
    elif max_vol > 0 and volume > max_vol:
        logging.warning(f"Volume {volume} > max {max_vol}, clamped to max")
        volume = max_vol

    logging.info(f"Normalized volume: {volume} (Step: {step_vol}, Min: {min_vol})")

sl_price, tp_price = self._calculate_sl_tp_price(corrected_symbol, order_type, price, sl, tp)

logging.info(f"Order parameters: price={price}, sl_price={sl_price}, tp_price={tp_price}")

for filling_mode in supported_fillings:
    request = {
        "action": mt5.TRADE_ACTION_DEAL,
        "symbol": corrected_symbol,
        "volume": volume,
        "type": type_mt5,
        "price": price,
        "deviation": 20,
        "type_filling": filling_mode,
        "type_time": mt5.ORDER_TIME_GTC,
    }

    # Add comment if provided
    if comment:

```



```

        request["comment"] = comment

    # ALWAYS add SL and TP - never skip them
    if sl_price is not None:
        request["sl"] = sl_price
        logging.info(f"Setting SL price: {sl_price}")
    if tp_price is not None:
        request["tp"] = tp_price
        logging.info(f"Setting TP price: {tp_price}")

    logging.info(f"Sending order request: {request}")
    result = mt5.order_send(request)

    if result is not None:
        logging.info(f"Order result: retcode={result.retcode}, comment={result.comment}")
        if result.retcode == mt5.TRADE_RETCODE_DONE:
            logging.info(
                f"Order successful: {order_type} {corrected_symbol} {volume} at {price}, tick={tick}"
            )
            return result.order
        else:
            # Enhanced error handling for common MT5 trading issues
            error_msg = result.comment if result.comment else "Unknown error"

            # Check for automated trading permission issues
            if result.retcode == 10027: # TRADE_RETCODE_CLIENT_DISABLES_AT
                print(f"[ERROR] MT5 TRADING ERROR: Automated trading is disabled in MT5 terminal")
                print(f"[TOOL] SOLUTION: Enable automated trading in MT5:")
                print(f"    1. Go to Tools → Options → Expert Advisors")
                print(f"    2. Check 'Allow algorithmic trading'")
                print(f"    3. Check 'Allow DLL imports'")
                print(f"    4. Click OK and restart application")
                raise Exception(
                    "Automated trading disabled in MT5 - Enable in Tools → Options → Expert Advisors"
                )

            elif result.retcode == 10026: # TRADE_RETCODE_TRADE_DISABLED
                print(f"[ERROR] MT5 TRADING ERROR: Trading is disabled")
                print(f"[TOOL] SOLUTION: Check MT5 terminal settings and broker permissions")
                raise Exception("Trading disabled - Check MT5 settings and broker permissions")

    # SUCCESS: We have a valid tick, now extract the price
    price = tick.ask if order_type == "buy" else tick.bid
    print(f"[OK] PRICE EXTRACTED: {order_type} price for {corrected_symbol} = {price}")
    print(
        f"    Full tick data: ask={tick.ask}, bid={tick.bid}, spread={tick.ask - tick.bid if tick.ask > tick.bid else 0}"
    )

    # Validate price is reasonable
    if price <= 0:
        print(f"[ERROR] INVALID PRICE: {price} <= 0")
        raise Exception(f"Invalid price {price} for {corrected_symbol}")

    type_mt5 = mt5.ORDER_TYPE_BUY if order_type == "buy" else mt5.ORDER_TYPE_SELL
    supported_fillings = self.get_supported_filling_modes(corrected_symbol)
    last_error = None

    if sl is None:
        sl = self.sl_points
    if tp is None:
        tp = self.tp_points
    if volume is None:
        volume = self.default_volume

```

```

# Apply PlexyTrade lot size adjustment - ONLY for USTECH (Nasdaq)
# XAUUSD (Gold) pip values are consistent across brokers, so no division needed
if self.is_plexy_server and volume > 0:
    # Only divide lot size for USTECH/Nasdaq symbols
    if any(x in corrected_symbol.upper() for x in ['USTECH', 'USTEC', 'NAS', 'NASDAQ', 'NDX',
        'NQ']):
        original_volume = volume
        volume = volume / 20.0
        logging.info(
            f"PlexyTrade adjustment for {corrected_symbol}: {original_volume} -> {volume} lot
    else:
        logging.info(
            f"PlexyTrade: No lot size adjustment for {corrected_symbol} (only divide USTECH,

# Ensure SL and TP are always set (never skip) - use defaults if 0
if sl is None or float(sl) <= 0:
    sl = self.sl_points if self.sl_points > 0 else 10 # Default 10 points if not set
    logging.info(f"Using default/minimum SL: {sl} points")
if tp is None or float(tp) <= 0:
    tp = self.tp_points if self.tp_points > 0 else 20 # Default 20 points if not set
    logging.info(f"Using default/minimum TP: {tp} points")

# Convert to float and ensure they're positive
sl = float(sl)
tp = float(tp)
volume = float(volume)

sl_price, tp_price = self._calculate_sl_tp_price(corrected_symbol, order_type, price, sl, tp)

logging.info(f"Order parameters: price={price}, sl_price={sl_price}, tp_price={tp_price}")

for filling_mode in supported_fillings:
    request = {
        "action": mt5.TRADE_ACTION_DEAL,
        "symbol": corrected_symbol,
        "volume": volume,
        "type": type_mt5,
        "price": price,
        "deviation": 20,
        "type_filling": filling_mode,
        "type_time": mt5.ORDER_TIME_GTC,
    }

    # Add comment if provided
    if comment:
        request["comment"] = comment

    # ALWAYS add SL and TP - never skip them
    if sl_price is not None:
        request["sl"] = sl_price
        logging.info(f"Setting SL price: {sl_price}")
    if tp_price is not None:
        request["tp"] = tp_price
        logging.info(f"Setting TP price: {tp_price}")

    logging.info(f"Sending order request: {request}")
    result = mt5.order_send(request)

    if result is not None:
        logging.info(f"Order result: retcode={result.retcode}, comment={result.comment}")

```

```

if result.retcode == mt5.TRADE_RETCODE_DONE:
    logging.info(
        f"Order successful: {order_type} {corrected_symbol} {volume} at {price}, tick
    )
    return result.order
else:
    # Enhanced error handling for common MT5 trading issues
    error_msg = result.comment if result.comment else "Unknown error"

    # Check for automated trading permission issues
    if result.retcode == 10027: # TRADE_RETCODE_CLIENT_DISABLES_AT
        print(f"[ERROR] MT5 TRADING ERROR: Automated trading is disabled in MT5 terminal")
        print(f"[TOOL] SOLUTION: Enable automated trading in MT5:")
        print(f"    1. Go to Tools → Options → Expert Advisors")
        print(f"    2. Check 'Allow algorithmic trading'")
        print(f"    3. Check 'Allow DLL imports'")
        print(f"    4. Click OK and restart application")
        raise Exception(
            "Automated trading disabled in MT5 - Enable in Tools → Options → Expert Advisors"
        )

    elif result.retcode == 10026: # TRADE_RETCODE_TRADE_DISABLED
        print(f"[ERROR] MT5 TRADING ERROR: Trading is disabled")
        print(f"[TOOL] SOLUTION: Check MT5 terminal settings and broker permissions")
        raise Exception("Trading disabled - Check MT5 settings and broker permissions")

    elif result.retcode == 10013: # TRADE_RETCODE_INVALID_REQUEST
        print(f"[ERROR] MT5 TRADING ERROR: Invalid trading request")
        print(f"[TOOL] SOLUTION: Check symbol, volume, and market hours")
        raise Exception(f"Invalid trading request: {error_msg}")

    elif result.retcode == 10004: # TRADE_RETCODE_REQUOTE
        print(f"[WARNING] MT5 TRADING: Price requote - retrying...")

    elif result.retcode == 10018: # TRADE_RETCODE_MARKET_CLOSED
        print(f"[ERROR] MT5 TRADING ERROR: Market is closed")
        print(f"[TOOL] SOLUTION: Wait for market opening hours")
        raise Exception("Market is closed - Wait for trading hours")

    elif result.retcode == 10019: # TRADE_RETCODE_NO_MONEY
        print(f"[ERROR] MT5 TRADING ERROR: Insufficient funds")
        print(f"[TOOL] SOLUTION: Check account balance and reduce position size")
        raise Exception("Insufficient funds - Check account balance")

    else:
        print(f"[ERROR] MT5 TRADING ERROR: {error_msg} (Code: {result.retcode})")
        print(f"[TOOL] SOLUTION: Check MT5 terminal for detailed error information")

        last_error = f"{error_msg} (Code: {result.retcode})"
else:
    last_error = "No result returned"
    print(f"[ERROR] MT5 CRITICAL: No response from MT5 terminal")
    print(f"[TOOL] SOLUTION: Check MT5 terminal connection and restart if needed")

logging.warning(
    f"Order attempt failed: {order_type} {corrected_symbol} {volume} at {price}, filling=
)

logging.error(
    f"Order failed: {order_type} {corrected_symbol} {volume} at {price}, last_error={last_error}
)
raise Exception(f"Order failed: {last_error}")

except Exception as e:

```

```
logging.exception(f"Exception in place_order: {e}")
raise
```

Method: MT5API.buy_market

```
def buy_market(self, symbol, volume=None, sl=None, tp=None, comment=None)
```

Step by step (top-level body)

1. **return** self.place_order(symbol, 'buy', volume=volume, sl=sl, tp=tp, commen....

Code (copy-paste safe)

```
def buy_market(self, symbol, volume=None, sl=None, tp=None, comment=None):
    return self.place_order(symbol, "buy", volume=volume, sl=sl, tp=tp, comment=comment)
```

Method: MT5API.sell_market

```
def sell_market(self, symbol, volume=None, sl=None, tp=None, comment=None)
```

Step by step (top-level body)

1. **return** self.place_order(symbol, 'sell', volume=volume, sl=sl, tp=tp, comme....

Code (copy-paste safe)

```
def sell_market(self, symbol, volume=None, sl=None, tp=None, comment=None):
    return self.place_order(symbol, "sell", volume=volume, sl=sl, tp=tp, comment=comment)
```

Method: MT5API.is_connected

```
def is_connected(self)
```

Check if MT5 connection is still active

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def is_connected(self):
    """Check if MT5 connection is still active"""
    try:
        # Try to get account info to test connection
        account_info = mt5.account_info()
        if account_info is None:
            return False
        return True
    except Exception:
        return False
```

Method: MT5API.check_connection_and_disconnect_if_needed

```
def check_connection_and_disconnect_if_needed(self)
```

Check connection and disconnect if lost

Step by step (top-level body)

1. `if not self.is_connected()`: branches conditionally.
2. `return True`.

Code (copy-paste safe)

```
def check_connection_and_disconnect_if_needed(self):
    """Check connection and disconnect if lost"""
    if not self.is_connected():
        logging.warning("MT5 connection lost, disconnecting...")
        self.disconnect()
        return False
    return True
```

Method: MT5API.disconnect

```
def disconnect(self)
```

Properly disconnect from MT5 with enhanced cleanup and terminal closure

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def disconnect(self):
    """Properly disconnect from MT5 with enhanced cleanup and terminal closure"""
    try:
        # First, perform standard MT5 API shutdown
        if mt5.terminal_info() is not None:
            mt5.shutdown()
            logging.info("MT5 API disconnected successfully")
        else:
            logging.info("MT5 API was already disconnected")

        # Force close MT5 terminal processes to ensure complete shutdown
        self._close_mt5_processes()

    except Exception as e:
        logging.error(f"Error during MT5 disconnect: {e}")
        # Force shutdown and process termination even if there was an error
        try:
            mt5.shutdown()
        except:
            pass
```

```
# Still attempt to close processes
self._close_mt5_processes()
```

Method: MT5API._close_mt5_processes

```
def _close_mt5_processes(self)
```

Force close all MT5 terminal processes

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def _close_mt5_processes(self):
    """Force close all MT5 terminal processes"""
    try:
        closed_processes = []

        # Look for MT5 processes by name
        for proc in psutil.process_iter(['pid', 'name', 'exe']):
            try:
                proc_name = proc.info['name'].lower()
                proc_exe = proc.info['exe']

                # Check if this is an MT5 process
                if (proc_name in ['terminal64.exe', 'terminal.exe', 'metatrader5.exe'] or
                    (proc_exe and 'metatrader' in proc_exe.lower())):

                    proc.terminate() # Send termination signal
                    closed_processes.append(f"{proc_name} (PID: {proc.info['pid']})")

            except (psutil.NoSuchProcess, psutil.AccessDenied, psutil.ZombieProcess):
                # Process might have already closed or access denied
                continue
            except Exception as e:
                logging.warning(f"Error checking process: {e}")
                continue

        if closed_processes:
            logging.info(f"■ Closed MT5 processes: {' '.join(closed_processes)}")

            # Wait a moment for graceful termination
            sleep(2)

            # Force kill any remaining MT5 processes
            for proc in psutil.process_iter(['pid', 'name', 'exe']):
                try:
                    proc_name = proc.info['name'].lower()
                    proc_exe = proc.info['exe']
```

```

        if (proc_name in ['terminal64.exe', 'terminal.exe', 'metatrader5.exe'] or
            (proc_exe and 'metatrader' in proc_exe.lower())):

            proc.kill() # Force kill if still running
            logging.info(
                f"■ Force killed stubborn MT5 process: {proc_name} (PID: {proc.info['pid']})")

        except (psutil.NoSuchProcess, psutil.AccessDenied, psutil.ZombieProcess):
            continue
        except Exception as e:
            logging.warning(f"Error force killing process: {e}")
            continue
    else:
        logging.info("■ No MT5 processes found to close")

except Exception as e:
    logging.error(f"Error closing MT5 processes: {e}")
# Fallback: try using taskkill as last resort
try:
    subprocess.run(['taskkill', '/f', '/im', 'terminal64.exe'],
                    capture_output=True, check=False)
    subprocess.run(['taskkill', '/f', '/im', 'terminal.exe'],
                    capture_output=True, check=False)
    logging.info("■ Used taskkill as fallback to close MT5 processes")
except Exception as fallback_error:
    logging.error(f"Fallback taskkill also failed: {fallback_error}")

```

Method: MT5API.get_account_info

```
def get_account_info(self)
```

Why these Python concepts were used

- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns `info = mt5.account_info()`.
2. **if** `info` is `None`: branches conditionally.
3. Assigns `trades = mt5.positions_get()`.
4. Assigns `open_trades = len(trades)` if `trades` else 0.
5. Assigns `symbol = ""`.
6. Assigns `direction = ""`.
7. **if** `trades` and `open_trades > 0`: branches conditionally.
8. **return** `{'balance': str(round(info.balance, 2)), 'profit': str(round(info.p...`

Code (copy-paste safe)

```

def get_account_info(self):
    info = mt5.account_info()
    if info is None:
        logging.error("No account info available")
        return {"balance": "", "profit": "", "drawdown": "", "open_trades": "", "Symbol": "",
                "Direction": ""}

```

```

trades = mt5.positions_get()
open_trades = len(trades) if trades else 0
symbol = ""
direction = ""
if trades and open_trades > 0:
    pos = trades[0]
    symbol = getattr(pos, "symbol", "")
    if getattr(pos, "type", None) == mt5.POSITION_TYPE_BUY:
        direction = "Long"
    elif getattr(pos, "type", None) == mt5.POSITION_TYPE_SELL:
        direction = "Short"
    else:
        direction = ""
return {
    "balance": str(round(info.balance, 2)),
    "profit": str(round(info.profit, 2)),
    "drawdown": "",
    "open_trades": str(open_trades),
    "Symbol": symbol,
    "Direction": direction
}

```

Method: MT5API.is_trade_open

```
def is_trade_open(self, ticket)
```

Step by step (top-level body)

1. Assigns `positions = mt5.positions_get(ticket=ticket)`.
2. **return** `positions` is not `None` and `len(positions) > 0`.

Code (copy-paste safe)

```

def is_trade_open(self, ticket):
    positions = mt5.positions_get(ticket=ticket)
    return positions is not None and len(positions) > 0

```

Method: MT5API.has_open_trade

```
def has_open_trade(self, symbol)
```

Returns True if there is any open position for the given symbol.

Step by step (top-level body)

1. Assigns `positions = mt5.positions_get(symbol=symbol)`.
2. **return** `positions` is not `None` and `len(positions) > 0`.

Code (copy-paste safe)

```

def has_open_trade(self, symbol):
    """
    Returns True if there is any open position for the given symbol.
    """
    positions = mt5.positions_get(symbol=symbol)
    return positions is not None and len(positions) > 0

```

Method: MT5API.get_trades_today_count


```
def get_trades_today_count(self, comment_filter=None)
```

Get the number of trades opened today Args: comment_filter: Optional string to filter trades by comment (e.g., "Combine1_") Returns: int: Number of trades opened today

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def get_trades_today_count(self, comment_filter=None):
    """Get the number of trades opened today

    Args:
        comment_filter: Optional string to filter trades by comment (e.g., "Combine1_")

    Returns:
        int: Number of trades opened today
    """
    try:
        from datetime import datetime, date
        import MetaTrader5 as mt5

        today = date.today()
        today_start = datetime.combine(today, datetime.min.time())
        today_end = datetime.combine(today, datetime.max.time())

        # Convert to timestamp
        from_date = int(today_start.timestamp())
        to_date = int(today_end.timestamp())

        # Get history deals (completed trades) for today
        deals = mt5.history_deals_get(from_date, to_date)

        # Also check current open positions opened today
        positions = mt5.positions_get()

        count = 0

        # Count completed deals (history)
        if deals:
            for deal in deals:
                # Only count entry deals (not exit deals)
                if deal.entry == mt5.DEAL_ENTRY_IN:
                    if comment_filter is None or (deal.comment and comment_filter in deal.comment):
                        count += 1

        # Count open positions opened today
        if positions:
            for pos in positions:
```

```

        pos_time = datetime.fromtimestamp(pos.time)
        if pos_time.date() == today:
            if comment_filter is None or (pos.comment and comment_filter in pos.comment):
                count += 1

    logging.info(f"Trades opened today: {count} (filter: {comment_filter})")
    return count

except Exception as e:
    logging.error(f"Error getting today's trade count: {e}")
    return 0

```

Method: MT5API.get_orphaned_mt5_positions_by_account

```
def get_orphaned_mt5_positions_by_account(self, account_number)
```

Get MT5 positions for a specific Tradovate account Args: *account_number*: Tradovate account number to filter by Returns: *list*: List of orphaned position tickets

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def get_orphaned_mt5_positions_by_account(self, account_number):
    """Get MT5 positions for a specific Tradovate account

    Args:
        account_number: Tradovate account number to filter by

    Returns:
        list: List of orphaned position tickets
    """
    try:
        # Get all open MT5 positions
        positions = mt5.positions_get()
        if positions is None:
            return []

        orphaned_tickets = []
        for pos in positions:
            if pos.comment:
                # Check if position is from this account (comment starts with account number, may have
                # trailing spaces)
                if pos.comment.strip().startswith(account_number):
                    orphaned_tickets.append(pos.ticket)

        return orphaned_tickets
    except Exception as e:
        logging.error(f"Error finding orphaned positions by account: {e}")
        return []

```

Method: MT5API.close_orphaned_positions_by_account

```
def close_orphaned_positions_by_account(self, account_number)
```

Close MT5 positions for a specific Tradovate account Args: account_number: Tradovate account number to filter by Returns: int: Number of positions closed

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def close_orphaned_positions_by_account(self, account_number):
    """Close MT5 positions for a specific Tradovate account

    Args:
        account_number: Tradovate account number to filter by

    Returns:
        int: Number of positions closed
    """
    try:
        orphaned_tickets = self.get_orphaned_mt5_positions_by_account(account_number)
        closed_count = 0

        for ticket in orphaned_tickets:
            try:
                if self.close_trade(ticket):
                    closed_count += 1
                    logging.info(f"Closed orphaned MT5 position for account {account_number}: {ticket}")
            except Exception as e:
                logging.error(f"Error closing orphaned position {ticket}: {e}")

        return closed_count
    except Exception as e:
        logging.error(f"Error closing orphaned positions by account: {e}")
        return 0
```

Method: MT5API.get_orphaned_mt5_positions

```
def get_orphaned_mt5_positions(self, combine_comment_prefix)
```

Get MT5 positions that don't have corresponding Tradovate trades Args: combine_comment_prefix: Comment prefix to identify trades from this combine (e.g., "Combine1_") Returns: list: List of orphaned position tickets

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def get_orphaned_mt5_positions(self, combine_comment_prefix):
    """Get MT5 positions that don't have corresponding Tradovate trades

    Args:
        combine_comment_prefix: Comment prefix to identify trades from this combine (e.g., "Combine1_")

    Returns:
        list: List of orphaned position tickets
    """
    try:
        import MetaTrader5 as mt5

        positions = mt5.positions_get()
        orphaned_tickets = []

        if positions:
            for pos in positions:
                # Check if this position belongs to our combine
                if pos.comment and combine_comment_prefix in pos.comment:
                    orphaned_tickets.append(pos.ticket)
                    logging.info(f"Found orphaned MT5 position: {pos.ticket} ({pos.comment})")

        return orphaned_tickets

    except Exception as e:
        logging.error(f"Error finding orphaned positions: {e}")
        return []
```

Method: MT5API.close_orphaned_positions

```
def close_orphaned_positions(self, combine_comment_prefix)

    Close MT5 positions that don't have corresponding Tradovate trades
    Args: combine_comment_prefix:
    Comment prefix to identify trades from this combine (e.g., "Combine1_")
    Returns: int: Number of
    positions closed
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. try block with 1 except clause.

Code (copy-paste safe)

```
def close_orphaned_positions(self, combine_comment_prefix):
    """Close MT5 positions that don't have corresponding Tradovate trades

    Args:
        combine_comment_prefix: Comment prefix to identify trades from this combine (e.g., "Combine1

    Returns:
        int: Number of positions closed
    """
    try:
        orphaned_tickets = self.get_orphaned_mt5_positions(combine_comment_prefix)
        closed_count = 0

        for ticket in orphaned_tickets:
            try:
                if self.close_trade(ticket):
                    closed_count += 1
                    logging.info(f"Closed orphaned MT5 position: {ticket}")
            else:
                logging.warning(f"Failed to close orphaned MT5 position: {ticket}")
        except Exception as e:
            logging.error(f"Error closing orphaned position {ticket}: {e}")

        return closed_count

    except Exception as e:
        logging.error(f"Error closing orphaned positions: {e}")
        return 0
```

Method: MT5API.close_trade

```
def close_trade(self, ticket, retries=3, delay=2)
```

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **for** attempt in range(retries): iterates.
2. Calls logging.error(...) for its side effect.
3. **return** not self.is_trade_open(ticket).

Code (copy-paste safe)

```
def close_trade(self, ticket, retries=3, delay=2):
    for attempt in range(retries):
        positions = mt5.positions_get(ticket=ticket)
        if not positions:
            logging.info(f"Trade {ticket} already closed.")
            return True
```

```

pos = positions[0]
symbol = pos.symbol
volume = pos.volume
order_type = pos.type
price = mt5.symbol_info_tick(
    symbol).bid if order_type == mt5.POSITION_TYPE_BUY else mt5.symbol_info_tick(symbol).ask
close_type = mt5.ORDER_TYPE_SELL if order_type == mt5.POSITION_TYPE_BUY else mt5.ORDER_TYPE_BUY
request = {
    "action": mt5.TRADE_ACTION_DEAL,
    "symbol": symbol,
    "volume": volume,
    "type": close_type,
    "position": ticket,
    "price": price,
    "deviation": 20,
    "type_filling": mt5.ORDER_FILLING_IOC,
    "type_time": mt5.ORDER_TIME_GTC,
}
result = mt5.order_send(request)
if result is not None and result.retcode == mt5.TRADE_RETCODE_DONE:
    if not self.is_trade_open(ticket):
        logging.info(f"Trade {ticket} closed successfully.")
        return True
    sleep(delay)
logging.error(f"Failed to close trade {ticket} after {retries} attempts.")
return not self.is_trade_open(ticket)

```

Method: MT5API.force_close_trade

```
def force_close_trade(self, ticket)
```

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns `positions = mt5.positions_get(ticket=ticket)`.
2. **if** not positions: branches conditionally.
3. Assigns `pos = positions[0]`.
4. Assigns `symbol = pos.symbol`.
5. Assigns `volume = pos.volume`.
6. Assigns `order_type = pos.type`.
7. Assigns `price = mt5.symbol_info_tick(symbol).bid` if `order_type == mt5.POS...`
8. Assigns `close_type = mt5.ORDER_TYPE_SELL` if `order_type == mt5.POSITION_TYPE_BU...`
9. **for** filling in `[mt5.ORDER_FILLING_IOC, mt5.ORDER_FILLING_FOK, mt5.ORDER_...]`: iterates.
10. Calls `logging.error(...)` for its side effect.
11. **return** not `self.is_trade_open(ticket)`.

Code (copy-paste safe)

```
def force_close_trade(self, ticket):
    positions = mt5.positions_get(ticket=ticket)
    if not positions:
        logging.info(f"Trade {ticket} already closed (force close).")
        return True
    pos = positions[0]
    symbol = pos.symbol
    volume = pos.volume
    order_type = pos.type
    price = mt5.symbol_info_tick(
        symbol).bid if order_type == mt5.POSITION_TYPE_BUY else mt5.symbol_info_tick(symbol).ask
    close_type = mt5.ORDER_TYPE_SELL if order_type == mt5.POSITION_TYPE_BUY else mt5.ORDER_TYPE_BUY
    for filling in [mt5.ORDER_FILLING_IOC, mt5.ORDER_FILLING_FOK, mt5.ORDER_FILLING_RETURN]:
        request = {
            "action": mt5.TRADE_ACTION_DEAL,
            "symbol": symbol,
            "volume": volume,
            "type": close_type,
            "position": ticket,
            "price": price,
            "deviation": 20,
            "type_filling": filling,
            "type_time": mt5.ORDER_TIME_GTC,
        }
        result = mt5.order_send(request)
        if result is not None and result.retcode == mt5.TRADE_RETCODE_DONE:
            if not self.is_trade_open(ticket):
                logging.info(f"Trade {ticket} force closed successfully.")
                return True
        logging.error(f"Failed to force close trade {ticket}.")
    return not self.is_trade_open(ticket)
```

Method: MT5API.get_daily_trade_count

```
def get_daily_trade_count(self, comment_filter=None)
```

Get count of trades placed today from both open positions and history Args: comment_filter: Can be either: - Tradovate account number (e.g., "MFFUEVSTP326057008") - preferred method - Old combine prefix (e.g., "Combine1_") - for backward compatibility

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def get_daily_trade_count(self, comment_filter=None):
    """Get count of trades placed today from both open positions and history
```

```

Args:
    comment_filter: Can be either:
        - Tradovate account number (e.g., "MFFUEVSTP326057008") - preferred method
        - Old combine prefix (e.g., "Combine1_") - for backward compatibility
"""
try:
    from datetime import datetime, date
    import tempfile
    import os

    today = date.today()

    # Check if trades were reset for this filter today
    temp_dir = tempfile.gettempdir()
    reset_file = os.path.join(temp_dir, f"mt5_reset_{comment_filter}_{today.strftime('%Y%m%d')}.txt")
    if os.path.exists(reset_file):
        # Return 0 if reset flag exists for today
        return 0

    trade_count = 0

    # Count from open positions
    positions = mt5.positions_get()
    if positions:
        for pos in positions:
            # Convert MT5 time to date
            pos_date = datetime.fromtimestamp(pos.time).date()
            if pos_date == today:
                # If comment filter is provided, check if position comment matches
                if comment_filter:
                    # For new format: exact match with account number
                    # For old format: substring match with combine prefix
                    if pos.comment and (pos.comment.strip()
                                         == comment_filter or comment_filter in str(pos.comment)):
                        trade_count += 1
                else:
                    trade_count += 1

    # Count from history deals (more reliable for completed trades)
    # Get deals from start of today to now
    today_start = datetime.combine(today, datetime.min.time())
    today_end = datetime.now()

    deals = mt5.history_deals_get(today_start, today_end)
    if deals:
        # Only count entry deals (not exit deals to avoid double counting)
        for deal in deals:
            if deal.entry == mt5.DEAL_ENTRY_IN: # Entry deal only
                # If comment filter is provided, check if deal comment matches
                if comment_filter:
                    # For new format: exact match with account number
                    # For old format: substring match with combine prefix
                    if deal.comment and (deal.comment.strip()
                                         == comment_filter or comment_filter in str(deal.comment)):
                        trade_count += 1
                else:
                    trade_count += 1

    logging.info(f"Daily trade count: {trade_count} (filter: {comment_filter})")
    return trade_count

```



```

except Exception as e:
    logging.error(f"Error counting daily trades: {e}")
    return 0
    return 0

```

Method: MT5API.get_daily_trade_count_by_account

```
def get_daily_trade_count_by_account(self, tradovate_account_number)
```

Get count of trades placed today for a specific Tradovate account Args: tradovate_account_number: Tradovate account number (e.g., "MFFUEVSTP326057008") Returns: int: Number of trades opened today for this account

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def get_daily_trade_count_by_account(self, tradovate_account_number):
    """Get count of trades placed today for a specific Tradovate account

    Args:
        tradovate_account_number: Tradovate account number (e.g., "MFFUEVSTP326057008")

    Returns:
        int: Number of trades opened today for this account
    """
    try:
        from datetime import datetime, date

        today = date.today()
        trade_count = 0

        # Count from open positions
        positions = mt5.positions_get()
        if positions:
            for pos in positions:
                # Convert MT5 time to date
                pos_date = datetime.fromtimestamp(pos.time).date()
                if pos_date == today:
                    # Check if position comment matches the account number (comment starts with account number)
                    if pos.comment and pos.comment.strip().startswith(tradovate_account_number):
                        trade_count += 1

        # Count from history deals
        today_start = datetime.combine(today, datetime.min.time())
        today_end = datetime.now()

        deals = mt5.history_deals_get(today_start, today_end)

```

```

    if deals:
        for deal in deals:
            if deal.entry == mt5.DEAL_ENTRY_IN: # Entry deal only
                # Check if deal comment matches the account number (comment starts with account number)
                if deal.comment and deal.comment.strip().startswith(tradovate_account_number):
                    trade_count += 1

        logging.info(f"Daily trade count for account {tradovate_account_number}: {trade_count}")
        return trade_count

except Exception as e:
    logging.error(f"Error counting daily trades for account {tradovate_account_number}: {e}")
    return 0

```

Method: MT5API.reset_daily_trade_count

```
def reset_daily_trade_count(self, comment_filter=None)
```

Reset daily trade count for a specific filter by storing reset timestamp Args: *comment_filter*: Can be either:

- Tradovate account number (e.g., "MFFUEVSTP326057008") - preferred method - Old combine prefix (e.g., "Combine1_") - for backward compatibility

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def reset_daily_trade_count(self, comment_filter=None):
    """Reset daily trade count for a specific filter by storing reset timestamp

    Args:
        comment_filter: Can be either:
            - Tradovate account number (e.g., "MFFUEVSTP326057008") - preferred method
            - Old combine prefix (e.g., "Combine1_") - for backward compatibility
    """
    try:
        from datetime import datetime
        import tempfile
        import os

        # Create a simple flag file to mark that trades were reset for this filter today
        temp_dir = tempfile.gettempdir()
        reset_file = os.path.join(temp_dir,
                                   f"mt5_reset_{comment_filter}_{datetime.now().strftime('%Y%m%d')}.flag")

        # Create the flag file
    
```

```

with open(reset_file, 'w') as f:
    f.write(str(datetime.now().timestamp()))

logging.info(f"Daily trade count reset for filter: {comment_filter}")
return True

except Exception as e:
    logging.error(f"Error resetting daily trade count: {e}")
    return False

```

Method: MT5API.reset_daily_trade_count_by_account

```
def reset_daily_trade_count_by_account(self, tradovate_account_number)
```

Reset daily trade count for a specific Tradovate account Args: tradovate_account_number: Tradovate account number (e.g., "MFFUEVSTP326057008")

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def reset_daily_trade_count_by_account(self, tradovate_account_number):
    """Reset daily trade count for a specific Tradovate account

    Args:
        tradovate_account_number: Tradovate account number (e.g., "MFFUEVSTP326057008")
    """
    try:
        from datetime import datetime
        import tempfile
        import os

        # Create a simple flag file to mark that trades were reset for this account today
        temp_dir = tempfile.gettempdir()
        reset_file = os.path.join(temp_dir,
                                   f"mt5_reset_account_{tradovate_account_number}_{datetime.now().strftime('%Y%m%d')}.flag")

        # Create the flag file
        with open(reset_file, 'w') as f:
            f.write(str(datetime.now().timestamp()))

        logging.info(f"Daily trade count reset for Tradovate account: {tradovate_account_number}")
        return True

    except Exception as e:
        logging.error(f"Error resetting daily trade count for account {tradovate_account_number}: {e}")

```

```
return False
```

Method: MT5API.get_historical_profits_by_account

```
def get_historical_profits_by_account(self, account_number)

    Get total historical profits for trades from a specific Tradovate account
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def get_historical_profits_by_account(self, account_number):
    """Get total historical profits for trades from a specific Tradovate account"""
    try:
        from datetime import datetime, timedelta

        # Get deals from the last 30 days to get a good history
        end_time = datetime.now()
        start_time = end_time - timedelta(days=30)

        total_profit = 0.0

        # Get historical deals
        deals = mt5.history_deals_get(start_time, end_time)
        if deals:
            for deal in deals:
                # Check if deal comment matches the account number (comment starts with account number)
                if deal.comment and deal.comment.strip().startswith(account_number):
                    # Only count exit deals for profit calculation (avoid double counting)
                    if deal.entry == mt5.DEAL_ENTRY_OUT:
                        total_profit += deal.profit

        logging.info(f"Historical profits for account {account_number}: ${total_profit:.2f}")
        return total_profit

    except Exception as e:
        logging.error(f"Error getting historical profits by account: {e}")
        return 0.0
```

Method: MT5API.get_historical_profits

```
def get_historical_profits(self, comment_filter=None)

    Get total historical profits for trades with specific comment filter
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def get_historical_profits(self, comment_filter=None):
    """Get total historical profits for trades with specific comment filter"""
    try:
        from datetime import datetime, timedelta

        # Get deals from the last 30 days to get a good history
        end_time = datetime.now()
        start_time = end_time - timedelta(days=30)

        total_profit = 0.0

        # Get historical deals
        deals = mt5.history_deals_get(start_time, end_time)
        if deals:
            for deal in deals:
                # Check if deal comment matches the filter (for specific combine)
                if comment_filter and comment_filter not in str(deal.comment):
                    continue

                # Only count exit deals for profit calculation (avoid double counting)
                if deal.entry == mt5.DEAL_ENTRY_OUT:
                    total_profit += deal.profit

        logging.info(f"Historical profits for {comment_filter}: ${total_profit:.2f}")
        return total_profit

    except Exception as e:
        logging.error(f"Error calculating historical profits: {e}")
        return 0.0
```

Method: MT5API.close_orphaned_trades

```
def close_orphaned_trades(self, expected_tradovate_trades)
```

Close MT5 trades that don't have corresponding Tradovate trades Args: *expected_tradovate_trades*: List of Tradovate trade symbols/IDs that should have MT5 counterparts Returns: List of closed MT5 trade tickets

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. try block with 1 except clause.

Code (copy-paste safe)

```
def close_orphaned_trades(self, expected_tradovate_trades):
    """Close MT5 trades that don't have corresponding Tradovate trades

    Args:
        expected_tradovate_trades: List of Tradovate trade symbols/IDs that should have MT5 counterpart

    Returns:
        List of closed MT5 trade tickets
    """
    try:
        closed_trades = []
        positions = mt5.positions_get()

        if not positions:
            return closed_trades

        for pos in positions:
            should_close = False

            # If no Tradovate trades expected, close all MT5 trades
            if not expected_tradovate_trades:
                should_close = True
                reason = "no corresponding Tradovate trades"
            else:
                # Check if this MT5 trade has a corresponding Tradovate trade
                # This is a simplified check - in practice you might need more sophisticated matching
                mt5_symbol = pos.symbol
                has_counterpart = False

                for tradovate_trade in expected_tradovate_trades:
                    # Simple symbol matching - you may want to enhance this logic
                    if str(tradovate_trade).upper() in mt5_symbol.upper():
                        has_counterpart = True
                        break

                if not has_counterpart:
                    should_close = True
                    reason = f"no matching Tradovate trade found"

            if should_close:
                logging.info(f"Closing orphaned MT5 trade {pos.ticket}: {pos.symbol} ({reason})")
                if self.close_trade(pos.ticket):
                    closed_trades.append(pos.ticket)
                    logging.info(f"[OK] Closed orphaned trade {pos.ticket}")
                else:
                    logging.error(f"[ERROR] Failed to close orphaned trade {pos.ticket}")

        if closed_trades:
            logging.info(f"Closed {len(closed_trades)} orphaned MT5 trades: {closed_trades}")

        return closed_trades

    except Exception as e:
        logging.error(f"Error closing orphaned trades: {e}")
```

```
return []
```

Method: MT5API.extract_tradovate_account_from_comment

```
def extract_tradovate_account_from_comment(self, comment)
```

Extract Tradovate account number from MT5 comment Args: comment: MT5 order comment (e.g., "MFFUEVSTP326057008_CH1" or "ACCOUNT_FA") Returns: str: Tradovate account number or "Unknown" if not found

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def extract_tradovate_account_from_comment(self, comment):
    """Extract Tradovate account number from MT5 comment

    Args:
        comment: MT5 order comment (e.g., "MFFUEVSTP326057008_CH1" or "ACCOUNT_FA")

    Returns:
        str: Tradovate account number or "Unknown" if not found
    """
    try:
        if comment and comment.strip():
            import re
            # For new format: comment is account number + optional phase suffix
            # Strip the phase suffix if present
            account_number = comment.strip()
            # Remove phase abbreviation suffix if present
            if '_' in account_number:
                # Check for numbered formats (_CH1-4, _FD1-4, _DD1-4)
                if re.search(r'_(CH|FD|DD)\d+$', account_number):
                    account_number = re.sub(r'_(CH|FD|DD)\d+$', '', account_number)
                # Check for simple farming format: _FA
                elif account_number.endswith('_FA'):
                    account_number = account_number[:-3]
                # Check for unknown phase marker
                elif account_number.endswith('_UNK'):
                    account_number = account_number[:-4]
            return account_number if account_number else "Unknown"
        return "Unknown"
    except Exception as e:
        logging.error(f"Error extracting account from comment '{comment}': {e}")
        return "Unknown"
```

Method: MT5API.get_trades_by_tradovate_account

```
def get_trades_by_tradovate_account(self, tradovate_account_number=None)
```

Get all MT5 trades associated with a specific Tradovate account Args: *tradovate_account_number*:
Tradovate account number to filter by Returns: dict: Dictionary with 'open_positions' and 'history_deals'
lists

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def get_trades_by_tradovate_account(self, tradovate_account_number=None):
    """Get all MT5 trades associated with a specific Tradovate account

    Args:
        tradovate_account_number: Tradovate account number to filter by

    Returns:
        dict: Dictionary with 'open_positions' and 'history_deals' lists
    """
    try:
        from datetime import datetime, date

        result = {
            'open_positions': [],
            'history_deals': []
        }

        # Get open positions
        positions = mt5.positions_get()
        if positions:
            for pos in positions:
                if pos.comment:
                    account_from_comment = self.extract_tradovate_account_from_comment(pos.comment)
                    if tradovate_account_number is None or account_from_comment == tradovate_account_number:
                        result['open_positions'].append({
                            'ticket': pos.ticket,
                            'symbol': pos.symbol,
                            'volume': pos.volume,
                            'type': 'BUY' if pos.type == mt5.POSITION_TYPE_BUY else 'SELL',
                            'open_time': datetime.fromtimestamp(pos.time),
                            'comment': pos.comment,
                            'tradovate_account': account_from_comment
                        })
```



```

# Get today's history deals
today = date.today()
today_start = datetime.combine(today, datetime.min.time())
today_end = datetime.combine(today, datetime.max.time())
from_date = int(today_start.timestamp())
to_date = int(today_end.timestamp())

deals = mt5.history_deals_get(from_date, to_date)
if deals:
    for deal in deals:
        if deal.comment:
            account_from_comment = self.extract_tradovate_account_from_comment(deal.comment)
            if tradovate_account_number is None or account_from_comment == tradovate_account_number:
                result['history_deals'].append({
                    'ticket': deal.ticket,
                    'symbol': deal.symbol,
                    'volume': deal.volume,
                    'type': 'BUY' if deal.type == mt5.DEAL_TYPE_BUY else 'SELL',
                    'time': datetime.fromtimestamp(deal.time),
                    'comment': deal.comment,
                    'tradovate_account': account_from_comment,
                    'entry': deal.entry
                })

    return result

except Exception as e:
    logging.error(f"Error getting trades by Tradovate account: {e}")
    return {'open_positions': [], 'history_deals': []}

```

Method: MT5API.get_symbol_info

```
def get_symbol_info(self, symbol)

    Get symbol information
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def get_symbol_info(self, symbol):
    """Get symbol information"""
    try:
        return mt5.symbol_info(symbol)
    except Exception as e:
        logging.error(f"Error getting symbol info for {symbol}: {e}")
        return None

```

Method: MT5API.debug_symbol_info

```
def debug_symbol_info(self, symbol)
```

Debug method to print all available symbol information

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def debug_symbol_info(self, symbol):
    """Debug method to print all available symbol information"""
    try:
        info = mt5.symbol_info(symbol)
        if info:
            logging.info(f"=== Symbol Info for {symbol} ===")
            for attr in dir(info):
                if not attr.startswith('_'):
                    try:
                        value = getattr(info, attr)
                        logging.info(f"  {attr}: {value}")
                    except:
                        pass
            logging.info("=== End Symbol Info ===")
            return info
        else:
            logging.error(f"No symbol info available for {symbol}")
            return None
    except Exception as e:
        logging.error(f"Error debugging symbol info: {e}")
        return None
```

Method: MT5API.get_tick_data

```
def get_tick_data(self, symbol)
```

Get current tick data for symbol

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. try block with 1 except clause.

Code (copy-paste safe)

```
def get_tick_data(self, symbol):
    """Get current tick data for symbol"""
    try:
        return mt5.symbol_info_tick(symbol)
    except Exception as e:
        logging.error(f"Error getting tick data for {symbol}: {e}")
        return None
```

Method: MT5API.should_close_trades_for_rollover

```
def should_close_trades_for_rollover(self, prop_firm_name)
```

Check if trades should be closed based on prop firm rollover schedules. Closing Times (Eastern Time): - Trade Day: 5:00 PM ET - Funding Ticks: 5:00 PM ET - Tradeify: 4:59 PM ET - MFFU: 4:10 PM ET - Alpha Futures: 4:20 PM ET Args: prop_firm_name: Name of the prop firm Returns: bool: True if trades should be closed now, False otherwise

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Imports datetime (lazy import inside the function).
2. Imports pytz (lazy import inside the function).
3. try block with 1 except clause.

Code (copy-paste safe)

```
def should_close_trades_for_rollover(self, prop_firm_name):
    """
    Check if trades should be closed based on prop firm rollover schedules.

    Closing Times (Eastern Time):
    - Trade Day: 5:00 PM ET
    - Funding Ticks: 5:00 PM ET
    - Tradeify: 4:59 PM ET
    - MFFU: 4:10 PM ET
    - Alpha Futures: 4:20 PM ET

    Args:
        prop_firm_name: Name of the prop firm

    Returns:
        bool: True if trades should be closed now, False otherwise
    """
    import datetime
    import pytz
```

```

try:
    # Get current time in Eastern timezone
    eastern = pytz.timezone('US/Eastern')
    current_time = datetime.datetime.now(eastern)

    # Define closing times for each prop firm (24-hour format)
    closing_schedules = {
        "Funding Ticks": (17, 0), # 5:00 PM Eastern Time
        "Tradeify": (16, 59), # 4:59 PM Eastern Time
        "MFFU": (16, 10), # 4:10 PM Eastern Standard Time
        "Alpha Futures": (16, 20), # 4:20 PM Eastern Time
    }

    # Get closing time for this prop firm
    closing_time_tuple = closing_schedules.get(prop_firm_name)

    if closing_time_tuple is None:
        # This prop firm doesn't require trade closing
        return False

    # Convert closing time to datetime object for proper comparison
    closing_hour, closing_minute = closing_time_tuple
    closing_time = current_time.replace(hour=closing_hour, minute=closing_minute, second=0,
        microsecond=0)

    # Get current date string for tracking
    current_date = current_time.strftime("%Y-%m-%d")

    # Safety check: Have we already executed rollover for this prop firm today?
    if self.rollover_executed_today.get(prop_firm_name) == current_date:
        return False # Already executed today, skip to prevent duplicates

    # Check if current time is at or past closing time (with safety buffer)
    # This ensures we don't miss rollover due to system delays
    if current_time >= closing_time:
        # Also check if we're on a weekday (Monday = 0, Sunday = 6)
        if current_time.weekday() < 5: # Monday through Friday
            current_time_str = current_time.strftime("%H:%M")
            closing_time_str = closing_time.strftime("%H:%M")
            logging.info(
                f"■ Market rollover time reached for {prop_firm_name} at {current_time_str} ET (
            return True

    return False

except Exception as e:
    logging.error(f"Error checking rollover schedule for {prop_firm_name}: {e}")
    return False

```

Method: MT5API.close_trades_for_rollover

```
def close_trades_for_rollover(self, prop_firm_name, account_comment_prefix=None)
```

Close all MT5 trades for market rollover based on prop firm schedule. Args: prop_firm_name: Name of the prop firm account_comment_prefix: Account number to filter trades (e.g., "MFFU123456") Returns: list: List of closed trade tickets

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **if** not `self.should_close_trades_for_rollover(prop_firm_name)`: branches conditionally.
2. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def close_trades_for_rollover(self, prop_firm_name, account_comment_prefix=None):
    """
    Close all MT5 trades for market rollover based on prop firm schedule.

    Args:
        prop_firm_name: Name of the prop firm
        account_comment_prefix: Account number to filter trades (e.g., "MFFU123456")

    Returns:
        list: List of closed trade tickets
    """
    if not self.should_close_trades_for_rollover(prop_firm_name):
        return []

    try:
        # Get all open positions
        positions = mt5.positions_get()
        if positions is None:
            logging.warning("No positions found or error getting positions")
            return []

        closed_tickets = []

        for position in positions:
            # Filter by account comment prefix if provided
            if account_comment_prefix and not position.comment.startswith(account_comment_prefix):
                continue

            # Close the position
            if self.close_trade(position.ticket):
                closed_tickets.append(position.ticket)
                logging.info(
                    f"■ ROLLOVER: Closed trade {position.ticket} for {prop_firm_name} market rollover"
                )
            else:
                logging.error(f"[ERROR] Failed to close trade {position.ticket} for rollover")

        if closed_tickets:
            logging.info(
                f"■ ROLLOVER COMPLETE: Closed {len(closed_tickets)} trades for {prop_firm_name} at m"
            )

            # Mark rollover as executed today to prevent duplicate execution
            import datetime
            import pytz
            eastern = pytz.timezone('US/Eastern')
            current_date = datetime.datetime.now(eastern).strftime("%Y-%m-%d")
```

```

        self.rollover_executed_today[prop_firm_name] = current_date

    return closed_tickets

except Exception as e:
    logging.error(f"Error closing trades for rollover ({prop_firm_name}): {e}")
    return []

```

Functions

```
def setup_pyinstaller_mt5_environment()
```

Enhanced MT5 environment setup for PyInstaller builds

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.
2. **return** False.

Code (copy-paste safe)

```

def setup_pyinstaller_mt5_environment():
    """Enhanced MT5 environment setup for PyInstaller builds"""
    try:
        # Detect if running in PyInstaller bundle
        if hasattr(sys, '_MEIPASS'):
            print("[SETUP] Detected PyInstaller environment - applying MT5 fixes...")

            # Set up DLL search paths for MT5
            if hasattr(os, 'add_dll_directory'):
                bundle_dir = sys._MEIPASS
                try:
                    os.add_dll_directory(bundle_dir)
                    print(f"    Added bundle directory to DLL path: {bundle_dir}")
                except Exception as e:
                    print(f"    [WARNING] Warning: Could not add bundle directory: {e}")

            # Add common MT5 paths to DLL search
            mt5_paths = [
                r"C:\Program Files\MetaTrader 5",
                r"C:\Program Files (x86)\MetaTrader 5",
                r"C:\Program Files\MetaTrader 5 Terminal"
            ]

            for path in mt5_paths:
                if os.path.exists(path):
                    try:
                        if hasattr(os, 'add_dll_directory'):
                            os.add_dll_directory(path)

```

```

        # Also add to PATH
        current_path = os.environ.get('PATH', '')
        if path not in current_path:
            os.environ['PATH'] = f"{path};{current_path}"
        print(f"    Added MT5 path: {path}")
    except Exception as e:
        print(f"    [WARNING] Warning: Could not add MT5 path {path}: {e}")

    # Set MT5 environment variables
    os.environ['MT5_TERMINAL_PATH'] = ''
    os.environ['PYINSTALLER_MT5_FIX'] = '1'

    print("    [OK] PyInstaller MT5 environment setup completed")
    return True

except Exception as e:
    print(f"    [WARNING] PyInstaller MT5 setup warning: {e}")
return False

```

```
def get_installed_mt5_terminals()
```

Detect installed MetaTrader 5 terminals on Windows Returns a list of dictionaries with terminal info

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.
- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns terminals = [].
2. Calls logging.info(...) for its side effect.
3. Assigns username = os.getenv('USERNAME', "").
4. Assigns common_paths = ['C:\\Program Files\\MetaTrader 5', 'C:\\Program Files (x...
5. Assigns registry_paths = [(winreg.HKEY_LOCAL_MACHINE, 'SOFTWARE\\MetaQuotes\\Termi...
6. **for** (hkey, reg_path) in registry_paths: iterates.
7. **for** path in common_paths: iterates.

8. **try** block with 1 **except** clause.
9. Assigns `current_dir = os.path.dirname(os.path.abspath(__file__))`.
10. Assigns `parent_dir = os.path.dirname(current_dir)`.
11. Assigns `portable_paths = [os.path.join(parent_dir, 'MT5'), os.path.join(parent_dir....`
12. **for** `path in portable_paths`: iterates.
13. **try** block with 1 **except** clause.
14. Assigns `unique_terminals = []`.
15. ... and 13 more statement(s) in the body.

Code (copy-paste safe)

```
def get_installed_mt5_terminals():
    """
    Detect installed MetaTrader 5 terminals on Windows
    Returns a list of dictionaries with terminal info
    """
    # Only detect paths - do not initialize anything
    terminals = []

    # Log detection start without initialization
    logging.info("[OK] Starting MT5 path detection (without initialization)")

    # Extended common installation paths - including the known working path
    username = os.getenv('USERNAME', '')
    common_paths = [
        r"C:\Program Files\MetaTrader 5",
        r"C:\Program Files (x86)\MetaTrader 5",
        r"C:\Program Files\MetaTrader 5 Terminal", # Known working path
        rf"C:\Users\{username}\AppData\Roaming\MetaQuotes\Terminal",
        rf"C:\Users\{username}\AppData\Local\Programs\MetaTrader 5",
        rf"C:\Users\{username}\Documents\MetaTrader 5",
        rf"C:\Users\{username}\Desktop\MetaTrader 5",
        r"D:\Program Files\MetaTrader 5",
        r"D:\Program Files (x86)\MetaTrader 5",
        r"E:\Program Files\MetaTrader 5",
        r"E:\Program Files (x86)\MetaTrader 5",
    ]

    # Check multiple registry locations for MT5 installations
    registry_paths = [
        (winreg.HKEY_LOCAL_MACHINE, r"SOFTWARE\MetaQuotes\Terminal"),
        (winreg.HKEY_LOCAL_MACHINE, r"SOFTWARE\Wow6432Node\MetaQuotes\Terminal"),
        (winreg.HKEY_CURRENT_USER, r"SOFTWARE\MetaQuotes\Terminal"),
        (winreg.HKEY_LOCAL_MACHINE, r"SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall"),
    ]

    for hkey, reg_path in registry_paths:
        try:
            with winreg.OpenKey(hkey, reg_path) as key:
                i = 0
                while True:
                    try:
                        subkey_name = winreg.EnumKey(key, i)
                    except:
                        with winreg.OpenKey(key, subkey_name) as subkey:
```



```

# Try different value names for path
path_values = ["Path", "InstallLocation", "UninstallString", "DisplayIcon"]
for value_name in path_values:
    try:
        value = winreg.QueryValueEx(subkey, value_name)[0]
        if value:
            # Extract directory from various formats
            if value_name == "UninstallString":
                path = os.path.dirname(value)
            elif value_name == "DisplayIcon":
                path = os.path.dirname(value)
            else:
                path = value

            # Check if this is a MetaTrader directory
            if os.path.exists(path):
                mt5_exe = os.path.join(path, "terminal64.exe")
                if os.path.exists(mt5_exe):
                    # Avoid duplicates
                    if not any(t["path"] == path for t in terminals):
                        terminals.append({
                            "name": f"MetaTrader 5 ({subkey_name})",
                            "path": path,
                            "source": f"registry_{hkey}_{reg_path}"
                        })
                    break
            except (FileNotFoundError, OSError):
                continue
        except (FileNotFoundError, OSError):
            pass
        i += 1
    except OSError:
        break
except (FileNotFoundError, OSError):
    continue

# Check common installation directories
for path in common_paths:
    if os.path.exists(path):
        mt5_exe = os.path.join(path, "terminal64.exe")
        if os.path.exists(mt5_exe):
            # Avoid duplicates
            if not any(t["path"] == path for t in terminals):
                terminals.append({
                    "name": f"MetaTrader 5 ({os.path.basename(path)}",
                    "path": path,
                    "source": "common_path"
                })

# Search for MT5 installations in all drives
try:
    import psutil
    for disk in psutil.disk_partitions():
        drive = disk.mountpoint
        search_paths = [
            os.path.join(drive, "Program Files", "MetaTrader 5"),
            os.path.join(drive, "Program Files (x86)", "MetaTrader 5"),
            os.path.join(drive, "MT5"),
            os.path.join(drive, "MetaTrader5"),
            os.path.join(drive, "MetaTrader 5"),

```

```

    ]
    for path in search_paths:
        if os.path.exists(path):
            mt5_exe = os.path.join(path, "terminal64.exe")
            if os.path.exists(mt5_exe):
                if not any(t["path"] == path for t in terminals):
                    terminals.append({
                        "name": f"MetaTrader 5 ({os.path.basename(path)})",
                        "path": path,
                        "source": "drive_search"
                    })
except ImportError:
    # If psutil is not available, skip drive search
    pass

# Check for portable installations in current directory and subdirectories
current_dir = os.path.dirname(os.path.abspath(__file__))
parent_dir = os.path.dirname(current_dir)
portable_paths = [
    os.path.join(parent_dir, "MT5"),
    os.path.join(parent_dir, "MetaTrader5"),
    os.path.join(parent_dir, "MetaTrader 5"),
    os.path.join(current_dir, "MT5"),
    os.path.join(current_dir, "MetaTrader5"),
    os.path.join(current_dir, "MetaTrader 5"),
]

for path in portable_paths:
    if os.path.exists(path):
        mt5_exe = os.path.join(path, "terminal64.exe")
        if os.path.exists(mt5_exe):
            if not any(t["path"] == path for t in terminals):
                terminals.append({
                    "name": f"MetaTrader 5 (Portable - {os.path.basename(path)})",
                    "path": path,
                    "source": "portable"
                })

# Search in START MENU shortcuts
try:
    start_menu_paths = [
        rf"C:\Users\{username}\AppData\Roaming\Microsoft\Windows\Start Menu\Programs",
        r"C:\ProgramData\Microsoft\Windows\Start Menu\Programs",
    ]

    for start_path in start_menu_paths:
        if os.path.exists(start_path):
            for root, dirs, files in os.walk(start_path):
                for file in files:
                    if "metatrader" in file.lower() and file.endswith(".lnk"):
                        try:
                            import win32com.client
                            shell = win32com.client.Dispatch("WScript.Shell")
                            shortcut = shell.CreateShortCut(os.path.join(root, file))
                            target_path = os.path.dirname(shortcut.Targetpath)
                            if os.path.exists(target_path):
                                mt5_exe = os.path.join(target_path, "terminal64.exe")
                                if os.path.exists(mt5_exe):
                                    if not any(t["path"] == target_path for t in terminals):
                                        terminals.append({

```

```

        "name": f"MetaTrader 5 (Shortcut - {os.path.basename(file)})",
        "path": target_path,
        "source": "start_menu"
    })
except ImportError:
    # If win32com is not available, skip shortcut search
    break
except Exception:
    # If there's any error in shortcut search, continue without it
    pass

# Remove duplicates and sort by name
unique_terminals = []
seen_paths = set()
for terminal in terminals:
    if terminal["path"] not in seen_paths:
        unique_terminals.append(terminal)
        seen_paths.add(terminal["path"])

# Test each terminal to identify which ones work
working_terminals = []
non_working_terminals = []
working_path = r"C:\Program Files\MetaTrader 5 Terminal"

for terminal in unique_terminals:
    # Mark the known working terminal
    if terminal["path"] == working_path:
        terminal["name"] = f"[OK] {terminal['name']} (Recommended)"
        terminal["is_working"] = True
        working_terminals.append(terminal)
    else:
        terminal["is_working"] = False
        non_working_terminals.append(terminal)

# Sort: working terminals first (recommended), then others alphabetically
working_terminals.sort(key=lambda x: x["name"])
non_working_terminals.sort(key=lambda x: x["name"])

# Combine with working terminals first
final_terminals = working_terminals + non_working_terminals

# If no terminals found, add default entry
if not final_terminals:
    final_terminals.append({
        "name": "MetaTrader 5 (Default)",
        "path": "",
        "source": "default",
        "is_working": False
    })

# Log found terminals for debugging
logging.info(f"Found {len(final_terminals)} MT5 terminals:")
for terminal in final_terminals:
    status = "[OK] RECOMMENDED" if terminal.get("is_working") else "[WARNING] Unknown"
    logging.info(f" {status} {terminal['name']} - {terminal['path']} (source: {terminal['source']})")

return final_terminals

```

Step 7.2 — Build trader_companion/mt5_comment_parser.py

What to do. Create the file `trader_companion/mt5_comment_parser.py` in your project root and paste in the code shown in this step. The file is **869** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from `requirements.txt`.

What this file is for. Parses the structured comment field on MT5 orders so that trades can be linked back to the strategy that placed them.

trader_companion/mt5_comment_parser.py

869 loc · 5 classes · 4 functions · 5 imports

Parses the structured comment field on MT5 orders so that trades can be linked back to the strategy that placed them. It defines **5** classes and **4** module-level functions, across **869** lines. Module docstring: *MT5 Comment Parser Module*

Module docstring

MT5 Comment Parser Module Parses MT5 trade comments from TradeAccountConnector to extract account info and phase data.

Comment Format: {TradovateAccountNumber}{PhaseSuffix} Example: MFFUEVSTP326057008_CH1

Phase Suffix Reference: - _CH1-4: Challenge Trade 1-4 - _FD0: Funded Base (MFFU style) - _FD1-4: Funded/Payout 1-4 - _DD1-4: Double Dip 1-4 - _FA: Farming/Consistency - _FA_DDMMYY: Farming with date (e.g., _FA_210126 = Jan 21, 2026) - _UNK: Unknown phase

Python concepts demonstrated in this module

- Classes and objects
- Dataclasses
- Decorators
- Dunder methods
- Enums
- Exceptions
- f-strings
- Inheritance
- Properties
- Type hints

Imports — what each one is for

```
import re
```

Why imported. Regular expressions. Pattern matching, search, replace, and text extraction.

Used in this file. `re.IGNORECASE`, `re.Match`, `re.match`

Example. `re.search(r'\d{3}-\d{4}', text)` # returns a Match or None

Other common scenarios.

- `re.findall(pattern, text)` for every non-overlapping match
- `re.sub(pattern, repl, text)` to replace occurrences

- `re.compile(...)` to reuse a pattern (faster in a loop)
- Named groups: `r'(?P<year>\d{4})'` lets you do `m.group('year')`

```
from datetime import datetime
```

Why imported. Dates, times, durations. The fundamental temporal types: `date`, `time`, `datetime`, `timedelta`, `timezone`.

Used in this file. `datetime`

Example. `datetime.now() - timedelta(days=7)` # one week ago

Other common scenarios.

- `datetime.strptime(s, '%Y-%m-%d')` parses a string
- `datetime.strftime(fmt)` formats one
- `datetime.fromtimestamp(n)` converts a Unix epoch
- `datetime.utcnow()` for UTC (better: `datetime.now(timezone.utc)`)

```
from typing import Dict
from typing import List
from typing import Optional
from typing import Tuple
from typing import Any
```

Why imported. Type-hint primitives — generics, unions, `Optional`, `Callable`, `Protocol`, `TypeVar`, `TypedDict`.

Used in this file. `Any`, `Dict`, `List`, `Optional`, `Tuple`

Example. `def lookup(k: str) -> Optional[int]: ...`

Other common scenarios.

- `List[int]` / `Dict[str, Any]` / `Tuple[int, str]` (or bare `list[int]` on 3.9+)
- `Callable[[int, int], int]` for function signatures
- `Protocol` for structural typing (duck typing with checks)
- `TypeVar('T')` for generic functions and classes

```
from dataclasses import dataclass
from dataclasses import field
```

Why imported. `@dataclass` auto-generates `__init__` / `__repr__` / `__eq__` from annotated fields — boilerplate-free record types.

Used in this file. `dataclass`, `field`

Example. `@dataclass\nclass Trade:\n symbol: str\n volume: float = 0.1`

Other common scenarios.

- `field(default_factory=list)` for mutable defaults (NEVER use `= []`)
- `frozen=True` makes instances hashable and immutable
- `asdict(obj)` / `astuple(obj)` convert to dict or tuple
- `@dataclass(slots=True)` saves memory (3.10+)

```
from enum import Enum
```

Why imported. Enumerated constants. Replaces magic strings/numbers with a type that has both a name and a value.

Used in this file. Enum

Example. `class Phase(Enum):\n CHALLENGE = 'challenge'\n FUNDED = 'funded'`

Other common scenarios.

- `IntEnum` for an enum that compares equal to its int value
- `auto()` lets you skip explicit values
- `Phase['CHALLENGE']` looks up by name; `Phase('challenge')` by value

Classes

```
class Phase(Enum)
```

Trading phase types.

```
CHALLENGE = 'CH'\nFUNDED = 'FD'\nDOUBLE_DIP = 'DD'\nFARMING = 'FA'\nUNKNOWN = 'UNK'\nLEGACY = 'LEGACY'
```

Code (copy-paste safe)

```
class Phase(Enum):\n    """Trading phase types."""\n    CHALLENGE = "CH"          # Challenge trades\n    FUNDED = "FD"             # Funded/Payout trades\n    DOUBLE_DIP = "DD"         # Double Dip trades\n    FARMING = "FA"            # Farming/Consistency phase\n    UNKNOWN = "UNK"           # Unknown phase\n    LEGACY = "LEGACY"
```

```
@dataclass
```

```
class ParsedComment
```

Parsed MT5 comment data.

```
account_number: Optional[str] = None\nphase: Optional[Phase] = None\nphase_code: Optional[str] = None\ntrade_number: Optional[int] = None\nfarming_date: Optional[datetime] = None\nraw_comment: str = ''\nis_valid: bool = False
```

Code (copy-paste safe)

```
class ParsedComment:\n    """Parsed MT5 comment data."""\n    account_number: Optional[str] = None\n    phase: Optional[Phase] = None
```

```

phase_code: Optional[str] = None # Raw phase code (CH, FD, DD, FA, UNK)
trade_number: Optional[int] = None
farming_date: Optional[datetime] = None
raw_comment: str = ""
is_valid: bool = False

def __str__(self):
    if not self.is_valid:
        return f"Invalid: {self.raw_comment}"

    date_str = f" ({self.farming_date.strftime('%d/%m/%y')})" if self.farming_date else ""
    trade_str = f" Trade #{self.trade_number}" if self.trade_number else ""
    return f"{self.account_number} | {self.phase.name}{trade_str}{date_str}"

def to_dict(self) -> Dict:
    """Convert to dictionary."""
    return {
        "account_number": self.account_number,
        "phase": self.phase.value if self.phase else None,
        "phase_name": self.phase.name if self.phase else None,
        "phase_code": self.phase_code,
        "trade_number": self.trade_number,
        "farming_date": self.farming_date.isoformat() if self.farming_date else None,
        "raw_comment": self.raw_comment,
        "is_valid": self.is_valid
    }

```

Method: ParsedComment.__str__

```
def __str__(self)
```

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.
- **__str__** — Defines **__str__** so the instance has a useful string representation in logs, the REPL, and error messages.

Step by step (top-level body)

1. **if** not self.is_valid: branches conditionally.
2. Assigns date_str = f" ({self.farming_date.strftime('%d/%m/%y')})" if self.fa....
3. Assigns trade_str = f" Trade #{self.trade_number}" if self.trade_number else "".
4. **return** f'{self.account_number} | {self.phase.name}{trade_str}{date_str}'.

Code (copy-paste safe)

```

def __str__(self):
    if not self.is_valid:
        return f"Invalid: {self.raw_comment}"

    date_str = f" ({self.farming_date.strftime('%d/%m/%y')})" if self.farming_date else ""
    trade_str = f" Trade #{self.trade_number}" if self.trade_number else ""
    return f"{self.account_number} | {self.phase.name}{trade_str}{date_str}"

```

Method: ParsedComment.to_dict

```
def to_dict(self) -> Dict
```

Convert to dictionary.

Why these Python concepts were used

- **return type annotation** — Annotated return type -> Dict. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **return** {'account_number': self.account_number, 'phase': self.phase.value i....

Code (copy-paste safe)

```
def to_dict(self) -> Dict:
    """Convert to dictionary."""
    return {
        "account_number": self.account_number,
        "phase": self.phase.value if self.phase else None,
        "phase_name": self.phase.name if self.phase else None,
        "phase_code": self.phase_code,
        "trade_number": self.trade_number,
        "farming_date": self.farming_date.isoformat() if self.farming_date else None,
        "raw_comment": self.raw_comment,
        "is_valid": self.is_valid
    }
```

@dataclass

class AggregatedTrade

Aggregated trade data for a specific account/phase combination.

```
account_number: str
phase: Phase
phase_code: str
trade_number: Optional[int] = None
farming_date: Optional[datetime] = None
total_profit: float = 0.0
total_commission: float = 0.0
total_swap: float = 0.0
total_fee: float = 0.0
deal_count: int = 0
deals: List[Dict] = field(default_factory=list)
earliest_deal_time: Optional[str] = None
latest_deal_time: Optional[str] = None
```

Code (copy-paste safe)

```
class AggregatedTrade:
    """Aggregated trade data for a specific account/phase combination."""
    account_number: str
    phase: Phase
    phase_code: str
```



```

trade_number: Optional[int] = None
farming_date: Optional[datetime] = None
total_profit: float = 0.0
total_commission: float = 0.0
total_swap: float = 0.0
total_fee: float = 0.0
deal_count: int = 0
deals: List[Dict] = field(default_factory=list)
earliest_deal_time: Optional[str] = None # ISO string of first deal time
latest_deal_time: Optional[str] = None   # ISO string of last deal time

@property
def net_profit(self) -> float:
    """Net profit including all costs."""
    return self.total_profit + self.total_commission + self.total_swap + self.total_fee

@property
def account_signature(self) -> str:
    """Get first4+last4 account signature for matching."""
    return get_account_signature(self.account_number)

def get_key(self) -> str:
    """Get unique key for this aggregation."""
    date_suffix = f"_{self.farming_date.strftime('%d%m%y')}" if self.farming_date else ""
    trade_suffix = str(self.trade_number) if self.trade_number else ""
    return f"{self.account_number}_{self.phase_code}{trade_suffix}{date_suffix}"

def to_dict(self) -> Dict:
    """Convert to dictionary."""
    return {
        "account_number": self.account_number,
        "phase": self.phase.value,
        "phase_name": self.phase.name,
        "phase_code": self.phase_code,
        "trade_number": self.trade_number,
        "farming_date": self.farming_date.isoformat() if self.farming_date else None,
        "timestamp": self.farming_date.timestamp(
            ) if self.farming_date else None, # Add timestamp for server filtering
        "total_profit": round(self.total_profit, 2),
        "total_commission": round(self.total_commission, 2),
        "total_swap": round(self.total_swap, 2),
        "total_fee": round(self.total_fee, 2),
        "net_profit": round(self.net_profit, 2),
        "deal_count": self.deal_count,
        "key": self.get_key(),
        "account_signature": self.account_signature,
        "open_time": self.earliest_deal_time,
        "close_time": self.latest_deal_time
    }

```

Method: AggregatedTrade.net_profit

```

@property
def net_profit(self) -> float
    Net profit including all costs.

```

Why these Python concepts were used

- **@property** — Wrapped in **@property** so callers access the value with attribute syntax (`obj.x`) instead of a method call. Lets the implementation compute or validate the value lazily without changing the call site.
- **return type annotation** — Annotated return type `-> float`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.

Step by step (top-level body)

1. `return self.total_profit + self.total_commission + self.total_swap + self.....`

Code (copy-paste safe)

```
def net_profit(self) -> float:
    """Net profit including all costs."""
    return self.total_profit + self.total_commission + self.total_swap + self.total_fee
```

Method: `AggregatedTrade.account_signature`

```
@property
def account_signature(self) -> str

    Get first4+last4 account signature for matching.
```

Why these Python concepts were used

- **@property** — Wrapped in **@property** so callers access the value with attribute syntax (`obj.x`) instead of a method call. Lets the implementation compute or validate the value lazily without changing the call site.
- **return type annotation** — Annotated return type `-> str`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.

Step by step (top-level body)

1. `return get_account_signature(self.account_number).`

Code (copy-paste safe)

```
def account_signature(self) -> str:
    """Get first4+last4 account signature for matching."""
    return get_account_signature(self.account_number)
```

Method: `AggregatedTrade.get_key`

```
def get_key(self) -> str

    Get unique key for this aggregation.
```

Why these Python concepts were used

- **return type annotation** — Annotated return type `-> str`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns `date_suffix = f"_{self.farming_date.strftime('%d%m%y')}"` if `self.farmin...`
2. Assigns `trade_suffix = str(self.trade_number)` if `self.trade_number` else `"`.
3. **return** `f'{self.account_number}_{self.phase_code}{trade_suffix}{date_suffix}'`.

Code (copy-paste safe)

```
def get_key(self) -> str:
    """Get unique key for this aggregation."""
    date_suffix = f"_{self.farming_date.strftime('%d%m%y')}" if self.farming_date else ""
    trade_suffix = str(self.trade_number) if self.trade_number else ""
    return f'{self.account_number}_{self.phase_code}{trade_suffix}{date_suffix}'
```

Method: AggregatedTrade.to_dict

```
def to_dict(self) -> Dict
```

Convert to dictionary.

Why these Python concepts were used

- **return type annotation** — Annotated return type -> Dict. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **ternary expression** — Uses `x` if `cond` else `y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **return** `{'account_number': self.account_number, 'phase': self.phase.value, ...`

Code (copy-paste safe)

```
def to_dict(self) -> Dict:
    """Convert to dictionary."""
    return {
        "account_number": self.account_number,
        "phase": self.phase.value,
        "phase_name": self.phase.name,
        "phase_code": self.phase_code,
        "trade_number": self.trade_number,
        "farming_date": self.farming_date.isoformat() if self.farming_date else None,
        "timestamp": self.farming_date.timestamp(
            ) if self.farming_date else None, # Add timestamp for server filtering
        "total_profit": round(self.total_profit, 2),
        "total_commission": round(self.total_commission, 2),
        "total_swap": round(self.total_swap, 2),
        "total_fee": round(self.total_fee, 2),
        "net_profit": round(self.net_profit, 2),
        "deal_count": self.deal_count,
        "key": self.get_key(),
        "account_signature": self.account_signature,
        "open_time": self.earliest_deal_time,
        "close_time": self.latest_deal_time
    }
```

```
class MT5CommentParser
```

Parser for MT5 trade comments following TradeAccountConnector format. Comment Format: {TradovateAccountNumber}{PhaseSuffix} Examples: MFFUEVSTP326057008_CH1 -> Account MFFUEVSTP326057008, Challenge Trade 1 MFFUEVSTP326057008_FD2 -> Account MFFUEVSTP326057008, Funded/Payout 2 MFFUEVSTP326057008_FA -> Account MFFUEVSTP326057008, Farming phase MFFUEVSTP326057008_FA_210126 -> Farming on Jan 21, 2026 MFFUEVSTP326057008_DD1 -> Double Dip Trade 1

```
PHASE_MEANINGS = {'MFFU': {'CH': 'Challenge trades', 'FD0': 'Base funded trade', 'FD': 'Payout',
    'DD': 'Double Dip...'}}
```

Code (copy-paste safe)

```
class MT5CommentParser:
    """
    Parser for MT5 trade comments following TradeAccountConnector format.

    Comment Format: {TradovateAccountNumber}{PhaseSuffix}

    Examples:
    MFFUEVSTP326057008_CH1 -> Account MFFUEVSTP326057008, Challenge Trade 1
    MFFUEVSTP326057008_FD2 -> Account MFFUEVSTP326057008, Funded/Payout 2
    MFFUEVSTP326057008_FA -> Account MFFUEVSTP326057008, Farming phase
    MFFUEVSTP326057008_FA_210126 -> Farming on Jan 21, 2026
    MFFUEVSTP326057008_DD1 -> Double Dip Trade 1
    """

    # Phase mappings by prop firm
    PHASE_MEANINGS = {
        "MFFU": {
            "CH": "Challenge trades",
            "FD0": "Base funded trade",
            "FD": "Payout",
            "DD": "Double Dip",
            "FA": "Farming/Consistency (5 days)"
        },
        "Tradeify": {
            "CH": "Challenge trades",
            "FD": "Payout",
            "DD": "Double Dip",
            "FA": "Consistency"
        },
        "Funding Ticks": {
            "CH": "Challenge trades",
            "FD": "Payout",
            "DD": "Double Dip",
            "FA": "Farming (6 days)"
        },
        "Alpha Futures": {
            "CH": "Challenge trades",
            "FD": "Payout",
            "DD": "Double Dip",
            "FA": "Farming"
        }
    }

    def __init__(self):
        """Initialize the parser with regex patterns."""
        # Patterns ordered by specificity (most specific first)
        self.patterns = [
```

```

        # Numbered phases: _CH1, _FD2, _DD3
        (r'^(.+?)_(CH|FD|DD)(\d+)$', self._parse_numbered_phase),
        # Farming with date: _FA_DDMMYY
        (r'^(.+?)_FA_(\d{6})$', self._parse_farming_with_date),
        # Simple farming: _FA
        (r'^(.+?)_FA$', self._parse_simple_farming),
        # Unknown phase: _UNK
        (r'^(.+?)_UNK$', self._parse_unknown_phase),
        # Legacy Combine format
        (r'^Combine(\d+)(.*)$', self._parse_legacy_combine),
    ]

def parse(self, comment: str) -> ParsedComment:
    """
    Parse an MT5 trade comment.

    Args:
        comment: The MT5 order comment string

    Returns:
        ParsedComment object with extracted data
    """
    result = ParsedComment(raw_comment=comment or "")

    if not comment:
        return result

    comment = comment.strip()

    # Try each pattern
    for pattern, handler in self.patterns:
        match = re.match(pattern, comment, re.IGNORECASE)
        if match:
            return handler(match, comment)

    # No pattern matched - check if it's just an account number
    # (comment without phase suffix)
    if comment and not comment.startswith("Combine"):
        result.account_number = comment
        result.is_valid = False # Mark as not fully valid without phase

    return result

def _parse_numbered_phase(self, match: re.Match, comment: str) -> ParsedComment:
    """Parse numbered phases: CH, FD, DD with trade number."""
    account = match.group(1)
    phase_code = match.group(2).upper()
    trade_num = int(match.group(3))

    phase_map = {
        "CH": Phase.CHALLENGE,
        "FD": Phase.FUNDED,
        "DD": Phase.DOUBLE_DIP
    }

    return ParsedComment(
        account_number=account,
        phase=phase_map.get(phase_code, Phase.UNKNOWN),
        phase_code=phase_code,
        trade_number=trade_num,
    )

```

```

        raw_comment=comment,
        is_valid=True
    )

def _parse_farming_with_date(self, match: re.Match, comment: str) -> ParsedComment:
    """Parse farming phase with date: _FA_DDMMYY."""
    account = match.group(1)
    date_str = match.group(2) # DDMMYY format

    farming_date = None
    try:
        farming_date = datetime.strptime(date_str, "%d%m%y")
    except ValueError:
        pass

    return ParsedComment(
        account_number=account,
        phase=Phase.FARMING,
        phase_code="FA",
        farming_date=farming_date,
        raw_comment=comment,
        is_valid=True
    )

def _parse_simple_farming(self, match: re.Match, comment: str) -> ParsedComment:
    """Parse simple farming phase: _FA."""
    return ParsedComment(
        account_number=match.group(1),
        phase=Phase.FARMING,
        phase_code="FA",
        raw_comment=comment,
        is_valid=True
    )

def _parse_unknown_phase(self, match: re.Match, comment: str) -> ParsedComment:
    """Parse unknown phase: _UNK."""
    return ParsedComment(
        account_number=match.group(1),
        phase=Phase.UNKNOWN,
        phase_code="UNK",
        raw_comment=comment,
        is_valid=True
    )

def _parse_legacy_combine(self, match: re.Match, comment: str) -> ParsedComment:
    """Parse legacy Combine format: Combine{N}_. """
    combinenum = match.group(1)
    rest = match.group(2)
    # Usually Combine1_Account
    return ParsedComment(
        account_number=rest,
        phase=Phase.CHALLENGE,
        phase_code="CH",
        trade_number=int(combinenum),
        raw_comment=comment,
        is_valid=True
    )

def get_phase_meaning(self, phase_code: str, prop_firm: str = "MFFU") -> str:
    """

```

Get human-readable meaning for a phase code.

Args:

phase_code: Phase code (CH, FD, DD, FA)
prop_firm: Prop firm name for context-specific meaning

Returns:

Description of the phase
""
firm_meanings = self.PHASE_MEANINGS.get(prop_firm, self.PHASE_MEANINGS["MFFU"])
return firm_meanings.get(phase_code, f"Unknown phase: {phase_code}")

Method: MT5CommentParser.__init__

```
def __init__(self)
```

Initialize the parser with regex patterns.

Why these Python concepts were used

- **__init__** — The constructor. Runs after Python has allocated the instance; assigns starting attribute values to self.

Step by step (top-level body)

1. Assigns self.patterns = [(r'^(.+?)_(CH|FD|DD)(\d+)\$', self._parse_numbered_phase)]...

Code (copy-paste safe)

```
def __init__(self):
    """Initialize the parser with regex patterns."""
    # Patterns ordered by specificity (most specific first)
    self.patterns = [
        # Numbered phases: _CH1, _FD2, _DD3
        (r'^(.+?)_(CH|FD|DD)(\d+)$', self._parse_numbered_phase),
        # Farming with date: _FA_DDMMYY
        (r'^(.+?)_FA_(\d{6})$', self._parse_farming_with_date),
        # Simple farming: _FA
        (r'^(.+?)_FA$', self._parse_simple_farming),
        # Unknown phase: _UNK
        (r'^(.+?)_UNK$', self._parse_unknown_phase),
        # Legacy Combine format
        (r'^Combine(\d+)_(.*)$', self._parse_legacy_combine),
    ]
```

Method: MT5CommentParser.parse

```
def parse(self, comment: str) -> ParsedComment
```

Parse an MT5 trade comment. Args: comment: The MT5 order comment string Returns: ParsedComment object with extracted data

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., comment: str). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.

- **return type annotation** — Annotated return type -> `ParsedComment`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.

Step by step (top-level body)

1. Assigns `result = ParsedComment(raw_comment=comment or "")`.
2. **if** not comment: branches conditionally.
3. Assigns `comment = comment.strip()`.
4. **for** (pattern, handler) in `self.patterns`: iterates.
5. **if** comment and (not `comment.startswith('Combine')`): branches conditionally.
6. **return** result.

Code (copy-paste safe)

```
def parse(self, comment: str) -> ParsedComment:
    """
    Parse an MT5 trade comment.

    Args:
        comment: The MT5 order comment string

    Returns:
        ParsedComment object with extracted data
    """
    result = ParsedComment(raw_comment=comment or "")

    if not comment:
        return result

    comment = comment.strip()

    # Try each pattern
    for pattern, handler in self.patterns:
        match = re.match(pattern, comment, re.IGNORECASE)
        if match:
            return handler(match, comment)

    # No pattern matched - check if it's just an account number
    # (comment without phase suffix)
    if comment and not comment.startswith("Combine"):
        result.account_number = comment
        result.is_valid = False # Mark as not fully valid without phase

    return result
```

Method: `MT5CommentParser._parse_numbered_phase`

```
def _parse_numbered_phase(self, match: re.Match, comment: str) -> ParsedComment
    Parse numbered phases: CH, FD, DD with trade number.
```

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `match: re.Match`, `comment: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.

- **return type annotation** — Annotated return type -> `ParsedComment`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.

Step by step (top-level body)

1. Assigns `account = match.group(1)`.
2. Assigns `phase_code = match.group(2).upper()`.
3. Assigns `trade_num = int(match.group(3))`.
4. Assigns `phase_map = {'CH': Phase.CHALLENGE, 'FD': Phase.FUNDED, 'DD': Phase.D...`
5. **return** `ParsedComment(account_number=account, phase=phase_map.get(phase_cod....`

Code (copy-paste safe)

```
def _parse_numbered_phase(self, match: re.Match, comment: str) -> ParsedComment:
    """Parse numbered phases: CH, FD, DD with trade number."""
    account = match.group(1)
    phase_code = match.group(2).upper()
    trade_num = int(match.group(3))

    phase_map = {
        "CH": Phase.CHALLENGE,
        "FD": Phase.FUNDED,
        "DD": Phase.DOUBLE_DIP
    }

    return ParsedComment(
        account_number=account,
        phase=phase_map.get(phase_code, Phase.UNKNOWN),
        phase_code=phase_code,
        trade_number=trade_num,
        raw_comment=comment,
        is_valid=True
    )
```

Method: `MT5CommentParser._parse_farming_with_date`

```
def _parse_farming_with_date(self, match: re.Match, comment: str) -> ParsedComment
    Parse farming phase with date: _FA_DDMMYY.
```

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `match: re.Match, comment: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type -> `ParsedComment`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

Step by step (top-level body)

1. Assigns `account = match.group(1)`.

2. Assigns `date_str = match.group(2)`.
3. Assigns `farming_date = None`.
4. **try** block with 1 **except** clause.
5. **return** `ParsedComment(account_number=account, phase=Phase.FARMING, phase_co....`

Code (copy-paste safe)

```
def _parse_farming_with_date(self, match: re.Match, comment: str) -> ParsedComment:
    """Parse farming phase with date: _FA_DDMMYY."""
    account = match.group(1)
    date_str = match.group(2) # DDMMYY format

    farming_date = None
    try:
        farming_date = datetime.strptime(date_str, "%d%m%y")
    except ValueError:
        pass

    return ParsedComment(
        account_number=account,
        phase=Phase.FARMING,
        phase_code="FA",
        farming_date=farming_date,
        raw_comment=comment,
        is_valid=True
    )
```

Method: `MT5CommentParser._parse_simple_farming`

```
def _parse_simple_farming(self, match: re.Match, comment: str) -> ParsedComment
    Parse simple farming phase: _FA.
```

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `match: re.Match, comment: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> ParsedComment`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.

Step by step (top-level body)

1. **return** `ParsedComment(account_number=match.group(1), phase=Phase.FARMING, p....`

Code (copy-paste safe)

```
def _parse_simple_farming(self, match: re.Match, comment: str) -> ParsedComment:
    """Parse simple farming phase: _FA."""
    return ParsedComment(
        account_number=match.group(1),
        phase=Phase.FARMING,
        phase_code="FA",
        raw_comment=comment,
        is_valid=True
    )
```

Method: MT5CommentParser._parse_unknown_phase

```
def _parse_unknown_phase(self, match: re.Match, comment: str) -> ParsedComment
    Parse unknown phase: _UNK.
```

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., match: re.Match, comment: str). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type -> ParsedComment. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.

Step by step (top-level body)

1. **return** ParsedComment(account_number=match.group(1), phase=Phase.UNKNOWN, p....

Code (copy-paste safe)

```
def _parse_unknown_phase(self, match: re.Match, comment: str) -> ParsedComment:
    """Parse unknown phase: _UNK."""
    return ParsedComment(
        account_number=match.group(1),
        phase=Phase.UNKNOWN,
        phase_code="UNK",
        raw_comment=comment,
        is_valid=True
    )
```

Method: MT5CommentParser._parse_legacy_combine

```
def _parse_legacy_combine(self, match: re.Match, comment: str) -> ParsedComment
    Parse legacy Combine format: Combine{N}_.
```

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., match: re.Match, comment: str). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type -> ParsedComment. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.

Step by step (top-level body)

1. Assigns combinenum = match.group(1).
2. Assigns rest = match.group(2).
3. **return** ParsedComment(account_number=rest, phase=Phase.CHALLENGE, phase_cod....

Code (copy-paste safe)

```
def _parse_legacy_combine(self, match: re.Match, comment: str) -> ParsedComment:
    """Parse legacy Combine format: Combine{N}_."""
    combinenum = match.group(1)
    rest = match.group(2)
    # Usually Combine1_Account
```

```

    return ParsedComment(
        account_number=rest,
        phase=Phase.CHALLENGE,
        phase_code="CH",
        trade_number=int(combinenum),
        raw_comment=comment,
        is_valid=True
    )

```

Method: MT5CommentParser.get_phase_meaning

```
def get_phase_meaning(self, phase_code: str, prop_firm: str='MFFU') -> str
```

*Get human-readable meaning for a phase code. Args: phase_code: Phase code (CH, FD, DD, FA)
prop_firm: Prop firm name for context-specific meaning Returns: Description of the phase*

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `phase_code: str`, `prop_firm: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> str`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `firm_meanings = self.PHASE_MEANINGS.get(prop_firm, self.PHASE_MEANINGS['M....`
2. **return** `firm_meanings.get(phase_code, f'Unknown phase: {phase_code}')`.

Code (copy-paste safe)

```

def get_phase_meaning(self, phase_code: str, prop_firm: str = "MFFU") -> str:
    """
    Get human-readable meaning for a phase code.

    Args:
        phase_code: Phase code (CH, FD, DD, FA)
        prop_firm: Prop firm name for context-specific meaning

    Returns:
        Description of the phase
    """
    firm_meanings = self.PHASE_MEANINGS.get(prop_firm, self.PHASE_MEANINGS["MFFU"])
    return firm_meanings.get(phase_code, f"Unknown phase: {phase_code}")

```

```
class MT5DealAggregator
```

Aggregates MT5 deals by account and phase based on comment parsing.

Code (copy-paste safe)

```

class MT5DealAggregator:
    """
    Aggregates MT5 deals by account and phase based on comment parsing.

```

```

"""

def __init__(self):
    """Initialize the aggregator."""
    self.parser = MT5CommentParser()
    self.aggregations: Dict[str, AggregatedTrade] = {}
    self.unmatched_deals: List[Dict] = []
    self.parse_log: List[str] = []

def reset(self):
    """Reset aggregation state."""
    self.aggregations = {}
    self.unmatched_deals = []
    self.parse_log = []

def add_deal(self, deal: Dict) -> Optional[str]:
    """
    Add a single deal to the aggregation.

    Args:
        deal: MT5 deal dictionary with fields like:
            - comment: Order comment
            - profit: Deal profit
            - commission: Commission
            - swap: Swap charges
            - fee: Additional fees
            - type: Deal type (BUY, SELL, BALANCE, etc.)
            - entry: Entry type (IN, OUT, INOUT)

    Returns:
        Aggregation key if matched, None if unmatched
    """
    # Skip balance/credit operations
    deal_type = str(deal.get('type', '')).upper()
    if deal_type in ['BALANCE', 'CREDIT', '2', '3', 'CHARGE', 'CORRECTION', 'BONUS']:
        return None

    # Parse the comment
    comment = deal.get('comment', '')
    parsed = self.parser.parse(comment)

    if not parsed.is_valid or not parsed.account_number:
        self.unmatched_deals.append(deal)
        self.parse_log.append(f"■ Unmatched: {comment or '(empty)'}")
        return None

    # Get deal date for clustering
    deal_time_str = deal.get('time', '')
    deal_date = None
    try:
        if deal_time_str:
            # Handle ISO format string (YYYY-MM-DDTHH:MM:SS)
            if isinstance(deal_time_str, str):
                if 'T' in deal_time_str:
                    deal_date = datetime.fromisoformat(deal_time_str).date()
                else:
                    # Fallback parsing if just YYYY-MM-DD
                    deal_date = datetime.strptime(deal_time_str.split()[0], '%Y-%m-%d').date()
            # Handle timestamp
            elif isinstance(deal_time_str, (int, float)):

```

```

        deal_date = datetime.fromtimestamp(deal_time_str).date()
except:
    pass # Keep deal_date as None if parsing fails

# For Farming (FA), if no date in comment, use deal date
# For Reset Accounts (CH/FD), we also want to separate by date to distinguish resets
# So we update parsed.farming_date if it's currently None, using the deal date

# Override/Set farming_date for grouping if not present in comment
if not parsed.farming_date and deal_date:
    # Always populate farming_date (used as grouping date for all phases now)
    parsed.farming_date = datetime(deal_date.year, deal_date.month, deal_date.day)

# Build aggregation key
key = self._build_key(parsed)

# Create or update aggregation
if key not in self.aggregations:
    self.aggregations[key] = AggregatedTrade(
        account_number=parsed.account_number,
        phase=parsed.phase,
        phase_code=parsed.phase_code,
        trade_number=parsed.trade_number,
        farming_date=parsed.farming_date
    )

agg = self.aggregations[key]
agg.total_profit += deal.get('profit', 0) or 0
agg.total_commission += deal.get('commission', 0) or 0
agg.total_swap += deal.get('swap', 0) or 0
agg.total_fee += deal.get('fee', 0) or 0
agg.deal_count += 1
agg.deals.append(deal)

# Track earliest/latest deal times for timestamp notes
deal_time = deal.get('time')
if deal_time:
    if agg.earliest_deal_time is None or str(deal_time) < agg.earliest_deal_time:
        agg.earliest_deal_time = str(deal_time)
    if agg.latest_deal_time is None or str(deal_time) > agg.latest_deal_time:
        agg.latest_deal_time = str(deal_time)

return key

def _build_key(self, parsed: ParsedComment) -> str:
    """
    Build a unique key for an aggregation.
    Now includes DATE for ALL phases to support FundedNext resets.
    """
    parts = [parsed.account_number, parsed.phase_code]

    if parsed.trade_number is not None:
        parts.append(str(parsed.trade_number))

    # Always append date to separate trades by day
    # This allows the server to match Jan trades to Account A and Feb trades to Account B (Reset)
    date_to_use = parsed.farming_date or datetime.now()
    parts.append(date_to_use.strftime('%d%m%y'))

    return "_".join(parts)

```

```

def process_deals(self, deals: List[Dict]) -> Tuple[Dict[str, AggregatedTrade], List[Dict]]:
    """
    Process a list of deals and aggregate by account/phase.

    Args:
        deals: List of MT5 deal dictionaries

    Returns:
        Tuple of (aggregations dict, unmatched deals list)
    """
    self.reset()

    for deal in deals:
        self.add_deal(deal)

    return self.aggregations, self.unmatched_deals

def get_summary(self) -> Dict:
    """Get a summary of the aggregation."""
    return {
        "total_aggregations": len(self.aggregations),
        "total_unmatched": len(self.unmatched_deals),
        "by_phase": self._summarize_by_phase(),
        "by_account": self._summarize_by_account(),
        "parse_log": self.parse_log
    }

def _summarize_by_phase(self) -> Dict[str, Dict]:
    """Summarize aggregations by phase."""
    summary = {}
    for key, agg in self.aggregations.items():
        phase_name = agg.phase.name
        if phase_name not in summary:
            summary[phase_name] = {"count": 0, "total_net_profit": 0.0}
        summary[phase_name]["count"] += 1
        summary[phase_name]["total_net_profit"] += agg.net_profit
    return summary

def _summarize_by_account(self) -> Dict[str, Dict]:
    """Summarize aggregations by account."""
    summary = {}
    for key, agg in self.aggregations.items():
        account = agg.account_number
        if account not in summary:
            summary[account] = {"phases": {}, "total_net_profit": 0.0}

        phase_key = f"{agg.phase_code}{agg.trade_number or ''}"
        summary[account]["phases"][phase_key] = agg.net_profit
        summary[account]["total_net_profit"] += agg.net_profit

    return summary

def to_dashboard_format(self) -> List[Dict]:
    """
    Convert aggregations to dashboard-compatible format.

    Returns list of dicts with:
    - account_number: Account identifier
    - phase: Phase code (CH, FD, DD, FA)
    - trade_number: Trade number (1-4) or None
    """

```

```

- farming_date: Date if farming phase
- net_profit: Total net profit for this combination
- deal_count: Number of deals
"""
result = []
for key, agg in self.aggregations.items():
    result.append(agg.to_dict())
return result

```

Method: MT5DealAggregator.__init__

```
def __init__(self)

    Initialize the aggregator.
```

Why these Python concepts were used

- **__init__** — The constructor. Runs after Python has allocated the instance; assigns starting attribute values to self.

Step by step (top-level body)

1. Assigns self.parser = MT5CommentParser().
2. Declares self.aggregations: Dict[str, AggregatedTrade] = {}.
3. Declares self.unmatched_deals: List[Dict] = [].
4. Declares self.parse_log: List[str] = [].

Code (copy-paste safe)

```
def __init__(self):
    """Initialize the aggregator."""
    self.parser = MT5CommentParser()
    self.aggregations: Dict[str, AggregatedTrade] = {}
    self.unmatched_deals: List[Dict] = []
    self.parse_log: List[str] = []

```

Method: MT5DealAggregator.reset

```
def reset(self)

    Reset aggregation state.
```

Step by step (top-level body)

1. Assigns self.aggregations = {}.
2. Assigns self.unmatched_deals = [].
3. Assigns self.parse_log = [].

Code (copy-paste safe)

```
def reset(self):
    """Reset aggregation state."""
    self.aggregations = {}
    self.unmatched_deals = []
    self.parse_log = []

```


Method: MT5DealAggregator.add_deal

```
def add_deal(self, deal: Dict) -> Optional[str]
```

Add a single deal to the aggregation. Args: deal: MT5 deal dictionary with fields like: - comment: Order comment - profit: Deal profit - commission: Commission - swap: Swap charges - fee: Additional fees - type: Deal type (BUY, SELL, BALANCE, etc.) - entry: Entry type (IN, OUT, INOUT) Returns: Aggregation key if matched, None if unmatched

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `deal: Dict`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> Optional[str]`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.

Step by step (top-level body)

1. Assigns `deal_type = str(deal.get('type', '')).upper()`.
2. **if** `deal_type` in `['BALANCE', 'CREDIT', '2', '3', 'CHARGE', 'CORRECTION']`: branches conditionally.
3. Assigns `comment = deal.get('comment', '')`.
4. Assigns `parsed = self.parser.parse(comment)`.
5. **if** `not parsed.is_valid` or `not parsed.account_number`: branches conditionally.
6. Assigns `deal_time_str = deal.get('time', '')`.
7. Assigns `deal_date = None`.
8. **try** block with 1 **except** clause.
9. **if** `not parsed.farming_date` and `deal_date`: branches conditionally.
10. Assigns `key = self._build_key(parsed)`.
11. **if** `key` not in `self.aggregations`: branches conditionally.
12. Assigns `agg = self.aggregations[key]`.
13. Updates `agg.total_profit` in place (Add).
14. Updates `agg.total_commission` in place (Add).
15. ... and 7 more statement(s) in the body.

Code (copy-paste safe)

```
def add_deal(self, deal: Dict) -> Optional[str]:
```

```

"""
Add a single deal to the aggregation.

Args:
    deal: MT5 deal dictionary with fields like:
        - comment: Order comment
        - profit: Deal profit
        - commission: Commission
        - swap: Swap charges
        - fee: Additional fees
        - type: Deal type (BUY, SELL, BALANCE, etc.)
        - entry: Entry type (IN, OUT, INOUT)

Returns:
    Aggregation key if matched, None if unmatched
"""
# Skip balance/credit operations
deal_type = str(deal.get('type', '')).upper()
if deal_type in ['BALANCE', 'CREDIT', '2', '3', 'CHARGE', 'CORRECTION', 'BONUS']:
    return None

# Parse the comment
comment = deal.get('comment', '')
parsed = self.parser.parse(comment)

if not parsed.is_valid or not parsed.account_number:
    self.unmatched_deals.append(deal)
    self.parse_log.append(f"■■■ Unmatched: {comment or '(empty)'}")
    return None

# Get deal date for clustering
deal_time_str = deal.get('time', '')
deal_date = None
try:
    if deal_time_str:
        # Handle ISO format string (YYYY-MM-DDTHH:MM:SS)
        if isinstance(deal_time_str, str):
            if 'T' in deal_time_str:
                deal_date = datetime.fromisoformat(deal_time_str).date()
            else:
                # Fallback parsing if just YYYY-MM-DD
                deal_date = datetime.strptime(deal_time_str.split()[0], '%Y-%m-%d').date()
        # Handle timestamp
        elif isinstance(deal_time_str, (int, float)):
            deal_date = datetime.fromtimestamp(deal_time_str).date()
except:
    pass # Keep deal_date as None if parsing fails

# For Farming (FA), if no date in comment, use deal date
# For Reset Accounts (CH/FD), we also want to separate by date to distinguish resets
# So we update parsed.farming_date if it's currently None, using the deal date

# Override/Set farming_date for grouping if not present in comment
if not parsed.farming_date and deal_date:
    # Always populate farming_date (used as grouping date for all phases now)
    parsed.farming_date = datetime(deal_date.year, deal_date.month, deal_date.day)

# Build aggregation key
key = self._build_key(parsed)

# Create or update aggregation

```

```

if key not in self.aggregations:
    self.aggregations[key] = AggregatedTrade(
        account_number=parsed.account_number,
        phase=parsed.phase,
        phase_code=parsed.phase_code,
        trade_number=parsed.trade_number,
        farming_date=parsed.farming_date
    )

    agg = self.aggregations[key]
    agg.total_profit += deal.get('profit', 0) or 0
    agg.total_commission += deal.get('commission', 0) or 0
    agg.total_swap += deal.get('swap', 0) or 0
    agg.total_fee += deal.get('fee', 0) or 0
    agg.deal_count += 1
    agg.deals.append(deal)

    # Track earliest/latest deal times for timestamp notes
    deal_time = deal.get('time')
    if deal_time:
        if agg.earliest_deal_time is None or str(deal_time) < agg.earliest_deal_time:
            agg.earliest_deal_time = str(deal_time)
        if agg.latest_deal_time is None or str(deal_time) > agg.latest_deal_time:
            agg.latest_deal_time = str(deal_time)

return key

```

Method: MT5DealAggregator._build_key

```
def _build_key(self, parsed: ParsedComment) -> str
```

Build a unique key for an aggregation. Now includes DATE for ALL phases to support FundedNext resets.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `parsed: ParsedComment`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> str`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.

Step by step (top-level body)

1. Assigns `parts = [parsed.account_number, parsed.phase_code]`.
2. **if** `parsed.trade_number` is not `None`: branches conditionally.
3. Assigns `date_to_use = parsed.farming_date` or `datetime.now()`.
4. Calls `parts.append(...)` for its side effect.
5. **return** `'_'.join(parts)`.

Code (copy-paste safe)

```

def _build_key(self, parsed: ParsedComment) -> str:
    """
    Build a unique key for an aggregation.
    Now includes DATE for ALL phases to support FundedNext resets.
    """

```

```

"""
parts = [parsed.account_number, parsed.phase_code]

if parsed.trade_number is not None:
    parts.append(str(parsed.trade_number))

# Always append date to separate trades by day
# This allows the server to match Jan trades to Account A and Feb trades to Account B (Reset)
date_to_use = parsed.farming_date or datetime.now()
parts.append(date_to_use.strftime('%d%m%y'))

return "_".join(parts)

```

Method: MT5DealAggregator.process_deals

```
def process_deals(self, deals: List[Dict]) -> Tuple[Dict[str, AggregatedTrade], List[Dict]]
```

Process a list of deals and aggregate by account/phase. Args: deals: List of MT5 deal dictionaries

Returns: Tuple of (aggregations dict, unmatched deals list)

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `deals: List[Dict]`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> Tuple[Dict[str, AggregatedTrade], List[Dict]]`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.

Step by step (top-level body)

1. Calls `self.reset(...)` for its side effect.
2. `for deal in deals:` iterates.
3. `return` (`self.aggregations`, `self.unmatched_deals`).

Code (copy-paste safe)

```

def process_deals(self, deals: List[Dict]) -> Tuple[Dict[str, AggregatedTrade], List[Dict]]:
    """
    Process a list of deals and aggregate by account/phase.

    Args:
        deals: List of MT5 deal dictionaries

    Returns:
        Tuple of (aggregations dict, unmatched deals list)
    """
    self.reset()

    for deal in deals:
        self.add_deal(deal)

    return self.aggregations, self.unmatched_deals

```

Method: MT5DealAggregator.get_summary

```
def get_summary(self) -> Dict
```

Get a summary of the aggregation.

Why these Python concepts were used

- **return type annotation** — Annotated return type -> Dict. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.

Step by step (top-level body)

1. **return** {'total_aggregations': len(self.aggregations), 'total_unmatched': l....

Code (copy-paste safe)

```
def get_summary(self) -> Dict:
    """Get a summary of the aggregation."""
    return {
        "total_aggregations": len(self.aggregations),
        "total_unmatched": len(self.unmatched_deals),
        "by_phase": self._summarize_by_phase(),
        "by_account": self._summarize_by_account(),
        "parse_log": self.parse_log
    }
```

Method: MT5DealAggregator._summarize_by_phase

```
def _summarize_by_phase(self) -> Dict[str, Dict]
```

Summarize aggregations by phase.

Why these Python concepts were used

- **return type annotation** — Annotated return type -> Dict[str, Dict]. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.

Step by step (top-level body)

1. Assigns summary = {}.
2. **for** (key, agg) in self.aggregations.items(): iterates.
3. **return** summary.

Code (copy-paste safe)

```
def _summarize_by_phase(self) -> Dict[str, Dict]:
    """Summarize aggregations by phase."""
    summary = {}
    for key, agg in self.aggregations.items():
        phase_name = agg.phase.name
        if phase_name not in summary:
            summary[phase_name] = {"count": 0, "total_net_profit": 0.0}
        summary[phase_name]["count"] += 1
        summary[phase_name]["total_net_profit"] += agg.net_profit
    return summary
```

Method: MT5DealAggregator._summarize_by_account

```
def _summarize_by_account(self) -> Dict[str, Dict]
```

Summarize aggregations by account.

Why these Python concepts were used

- **return type annotation** — Annotated return type -> Dict[str, Dict]. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns summary = {}.
2. for (key, agg) in self.aggregations.items(): iterates.
3. return summary.

Code (copy-paste safe)

```
def _summarize_by_account(self) -> Dict[str, Dict]:
    """Summarize aggregations by account."""
    summary = {}
    for key, agg in self.aggregations.items():
        account = agg.account_number
        if account not in summary:
            summary[account] = {"phases": {}, "total_net_profit": 0.0}

        phase_key = f"{agg.phase_code}{agg.trade_number or ''}"
        summary[account]["phases"][phase_key] = agg.net_profit
        summary[account]["total_net_profit"] += agg.net_profit

    return summary
```

Method: MT5DealAggregator.to_dashboard_format

```
def to_dashboard_format(self) -> List[Dict]
```

Convert aggregations to dashboard-compatible format. Returns list of dicts with:

- *account_number*: Account identifier
- *phase*: Phase code (CH, FD, DD, FA)
- *trade_number*: Trade number (1-4) or None
- *farming_date*: Date if farming phase
- *net_profit*: Total net profit for this combination
- *deal_count*: Number of deals

Why these Python concepts were used

- **return type annotation** — Annotated return type -> List[Dict]. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.

Step by step (top-level body)

1. Assigns result = [].
2. for (key, agg) in self.aggregations.items(): iterates.
3. return result.

Code (copy-paste safe)

```
def to_dashboard_format(self) -> List[Dict]:
    """
    Convert aggregations to dashboard-compatible format.

    Returns list of dicts with:
```

```

- account_number: Account identifier
- phase: Phase code (CH, FD, DD, FA)
- trade_number: Trade number (1-4) or None
- farming_date: Date if farming phase
- net_profit: Total net profit for this combination
- deal_count: Number of deals
"""
result = []
for key, agg in self.aggregations.items():
    result.append(agg.to_dict())
return result

```

Functions

```
def get_account_signature(account: str) -> str
```

Generate account signature from first 4 + last 4/5 characters. Used for matching truncated account numbers. Handles truncated format with '...' like: FNFT...59574 Examples: MFFUEVSTP326057008 -> mffu7008 (full account) FNFT...59574 -> fnft59574 (truncated - use last 5) MFFU7008 -> mffu7008 (short account)

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `account: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> str`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **if** not `account`: branches conditionally.
2. Assigns `account = account.strip()`.
3. **if** `'...'` in `account`: branches conditionally.
4. **if** `len(account) <= 8`: branches conditionally.
5. **return** `(account[:4] + account[-4:]).lower()`.

Code (copy-paste safe)

```

def get_account_signature(account: str) -> str:
    """
    Generate account signature from first 4 + last 4/5 characters.
    Used for matching truncated account numbers.

    Handles truncated format with '...' like: FNFT...59574

    Examples:
        MFFUEVSTP326057008 -> mffu7008 (full account)
        FNFT...59574 -> fnft59574 (truncated - use last 5)
        MFFU7008 -> mffu7008 (short account)
    """

```

```

if not account:
    return ""
account = account.strip()

# Handle truncated format: PREFIX...SUFFIX
if '...' in account:
    parts = account.split('...')
    if len(parts) == 2:
        prefix = parts[0][:4] if len(parts[0]) >= 4 else parts[0]
        suffix = parts[1] # Keep full suffix (usually 5 digits)
        return (prefix + suffix).lower()

# Standard format: first 4 + last 4
if len(account) <= 8:
    return account.lower()
return (account[:4] + account[-4:]).lower()

```

```
def parse_mt5_comment(comment: str) -> Dict
```

*Convenience function to parse a single MT5 comment. Args: comment: MT5 order comment string
Returns: Dictionary with parsed data: - account_number: Tradovate account number - phase: Phase code (CH, FD, DD, FA) - trade_number: Trade number (1-4) or None for farming - farming_date: Date if farming phase with date suffix - is_valid: Whether the comment was successfully parsed*

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., comment: str). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type -> Dict. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.

Step by step (top-level body)

1. Assigns parser = MT5CommentParser().
2. Assigns result = parser.parse(comment).
3. **return** result.to_dict().

Code (copy-paste safe)

```

def parse_mt5_comment(comment: str) -> Dict:
    """
    Convenience function to parse a single MT5 comment.

    Args:
        comment: MT5 order comment string

    Returns:
        Dictionary with parsed data:
        - account_number: Tradovate account number
        - phase: Phase code (CH, FD, DD, FA)
        - trade_number: Trade number (1-4) or None for farming
        - farming_date: Date if farming phase with date suffix
        - is_valid: Whether the comment was successfully parsed
    """
    parser = MT5CommentParser()

```



```

result = parser.parse(comment)
return result.to_dict()

```

```

def aggregate_deals_by_comment(deals: List[Dict]) -> Tuple[List[Dict], List[Dict], List[str]]

```

Convenience function to aggregate deals by parsed comments. Args: deals: List of MT5 deal dictionaries Returns: Tuple of (aggregated_data, unmatched_deals, log_messages)

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., deals: List[Dict]). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type -> Tuple[List[Dict], List[Dict], List[str]]. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.

Step by step (top-level body)

1. Assigns aggregator = MT5DealAggregator().
2. Calls aggregator.process_deals(...) for its side effect.
3. **return** (aggregator.to_dashboard_format(), aggregator.unmatched_deals, aggr....

Code (copy-paste safe)

```

def aggregate_deals_by_comment(deals: List[Dict]) -> Tuple[List[Dict], List[Dict], List[str]]:
    """
    Convenience function to aggregate deals by parsed comments.

    Args:
        deals: List of MT5 deal dictionaries

    Returns:
        Tuple of (aggregated_data, unmatched_deals, log_messages)
    """
    aggregator = MT5DealAggregator()
    aggregator.process_deals(deals)

    return (
        aggregator.to_dashboard_format(),
        aggregator.unmatched_deals,
        aggregator.parse_log
    )

```

```

def aggregate_deals_by_position(deals: List[Any]) -> Tuple[List[Dict], List[Dict], List[str]]

```

Aggregate deals by position_id first, then by comment/phase. Also handles "Farming" cluster logic: - If Phase is FARMING (FA) and no date in comment, uses Trade Date (Time). Args: deals: List of MT5 deal objects or dicts Returns: Tuple of (aggregated_data, unmatched_positions, log_messages)

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., deals: List[Any]). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.

- **return type annotation** — Annotated return type -> Tuple[List[Dict], List[Dict], List[str]]. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns parser = MT5CommentParser().
2. Assigns log_messages = [].
3. Assigns positions = {}.
4. **for** deal in deals: iterates.
5. Calls log_messages.append(...) for its side effect.
6. Assigns position_data = [].
7. Assigns unmatched = [].
8. **for** (pid, deal_list) in positions.items(): iterates.
9. Calls log_messages.append(...) for its side effect.
10. Assigns aggregated = {}.
11. **for** pos in position_data: iterates.
12. **for** (key, agg) in aggregated.items(): iterates.
13. **return** (list(aggregated.values()), unmatched, log_messages).

Code (copy-paste safe)

```
def aggregate_deals_by_position(deals: List[Any]) -> Tuple[List[Dict], List[Dict], List[str]]:
    """
    Aggregate deals by position_id first, then by comment/phase.

    Also handles "Farming" cluster logic:
    - If Phase is FARMING (FA) and no date in comment, uses Trade Date (Time).

    Args:
        deals: List of MT5 deal objects or dicts

    Returns:
        Tuple of (aggregated_data, unmatched_positions, log_messages)
    """
    parser = MT5CommentParser()
    log_messages = []

    # Step 1: Group deals by position_id
```

```

positions = {}
for deal in deals:
    # Handle both MT5 deal objects and dicts
    if hasattr(deal, '_asdict'):
        d = deal._asdict()
    elif hasattr(deal, 'position_id'):
        d = {
            'position_id': deal.position_id,
            'ticket': deal.ticket,
            'comment': deal.comment,
            'profit': deal.profit,
            'commission': getattr(deal, 'commission', 0),
            'swap': getattr(deal, 'swap', 0),
            'fee': getattr(deal, 'fee', 0),
            'entry': deal.entry,
            'type': deal.type,
            'volume': getattr(deal, 'volume', 0),
            'symbol': getattr(deal, 'symbol', ''),
            'time': getattr(deal, 'time', 0),
        }
    else:
        d = deal

    # Skip balance / credit / internal-transfer operations regardless of
    # whether the broker assigned a position_id to them. Some brokers tag
    # "internal transfer" balance ops with a real position_id which would
    # otherwise leak their amount into a hedge aggregate.
    d_type_str = str(d.get('type', '')).upper()
    if d_type_str in ('BALANCE', 'CREDIT', '2', '3', 'CHARGE', 'CORRECTION', 'BONUS'):
        continue
    d_comment_str = str(d.get('comment', '') or '').strip().lower()
    if 'internal transfer' in d_comment_str:
        continue

    pid = d.get('position_id', 0)
    if pid == 0:
        continue # Skip balance/credit operations

    if pid not in positions:
        positions[pid] = []
    positions[pid].append(d)

log_messages.append(f"Found {len(positions)} positions from {len(deals)} deals")

# Step 2: For each position, find entry deal with comment and sum profits
position_data = []
unmatched = []

for pid, deal_list in positions.items():
    # Find entry deal (entry=0/IN) with valid comment, and detect exit deals.
    # 'entry' can be integer (0=IN, 1=OUT, 2=INOUT, 3=OUT_BY) or string ("IN"/"OUT"/"INOUT"/"OUT_BY")
    entry_deal = None
    has_exit_deal = False
    exit_time = 0

    total_profit = 0.0
    total_commission = 0.0
    total_swap = 0.0
    total_fee = 0.0

    for d in deal_list:

```

```

entry_val = d.get('entry', '')
# Normalise to string for comparison
entry_str = str(entry_val).upper() if entry_val != '' else ''
is_entry = entry_val == 0 or entry_str == 'IN'
is_exit = entry_val in (1, 2, 3) or entry_str in ('OUT', 'INOUT', 'OUT_BY')

if is_entry and entry_deal is None:
    entry_deal = d
if is_exit:
    has_exit_deal = True

# Track latest time (exit time)
t = d.get('time_raw', d.get('time', 0))
if isinstance(t, str):
    try:
        t = datetime.fromisoformat(t).timestamp()
    except (ValueError, AttributeError):
        t = 0

if isinstance(t, (int, float)) and t > exit_time:
    exit_time = t

total_profit += d.get('profit', 0) or 0
total_commission += d.get('commission', 0) or 0
total_swap += d.get('swap', 0) or 0
total_fee += d.get('fee', 0) or 0

# Keep open positions in aggregation. Caller-side push logic decides whether
# zero-value FA rows should be sent (e.g., only when active positions exist).
if not has_exit_deal:
    log_messages.append(f"■■ Open position {pid} (no exit deal yet)")

# If no entry deal found, try to find any deal with a valid comment
if not entry_deal:
    for d in deal_list:
        comment = d.get('comment', '')
        if comment and ('CH' in comment or 'FD' in comment or 'FA' in comment or 'DD' in comment):
            entry_deal = d
            break

if not entry_deal:
    entry_deal = deal_list[0] # Fallback to first deal

comment = entry_deal.get('comment', '')
parsed = parser.parse(comment)

if not parsed.is_valid or not parsed.account_number:
    unmatched.append({
        'position_id': pid,
        'comment': comment,
        'total_profit': total_profit,
        'deal_count': len(deal_list)
    })
    continue

# --- FARMING DATE INFERENCE ---
# If Phase is FA but no farming_date, use the exit time date
if parsed.phase_code == 'FA' and not parsed.farming_date:
    if exit_time > 0:
        parsed.farming_date = datetime.fromtimestamp(exit_time)

```

```

# -----

position_data.append({
    'position_id': pid,
    'account_number': parsed.account_number,
    'phase': parsed.phase.value if parsed.phase else None,
    'phase_name': parsed.phase.name if parsed.phase else None,
    'phase_code': parsed.phase_code,
    'trade_number': parsed.trade_number,
    'farming_date': parsed.farming_date.isoformat() if parsed.farming_date else None,
    'timestamp': exit_time, # Store timestamp for sorting
    'total_profit': round(total_profit, 2),
    'total_commission': round(total_commission, 2),
    'total_swap': round(total_swap, 2),
    'total_fee': round(total_fee, 2),
    'net_profit': round(total_profit + total_commission + total_swap + total_fee, 2),
    'deal_count': len(deal_list),
    'has_open_position': not has_exit_deal,
    'account_signature': get_account_signature(parsed.account_number),
    'raw_comment': comment
})

log_messages.append(f"Matched {len(position_data)} positions, {len(unmatched)} unmatched")

# Step 3: Aggregate by account + phase (in case multiple positions have same account/phase)
# ALSO group by DATE for Farming if not already specific
aggregated = {}

for pos in position_data:
    key = f"{pos['account_number']}_{pos['phase_code']}{pos['trade_number'] or ''}"

    # For Farming, always append Date to key to separate days
    if pos['phase_code'] == 'FA' and pos.get('farming_date'):
        # Convert ISO string back to date for key or use string slice
        # ISO format: YYYY-MM-DDTHH:MM:SS
        date_str = pos['farming_date'][:10] # YYYY-MM-DD
        key += f"_{date_str}"
    elif pos.get('farming_date'):
        # Some other phases might have dates too (rare)
        date_str = pos['farming_date'][:10].replace('-', '')
        key += f"_{date_str}"

    if key not in aggregated:
        aggregated[key] = {
            'account_number': pos['account_number'],
            'phase': pos['phase'],
            'phase_name': pos['phase_name'],
            'phase_code': pos['phase_code'],
            'trade_number': pos['trade_number'],
            'farming_date': pos['farming_date'],
            'total_profit': 0.0,
            'total_commission': 0.0,
            'total_swap': 0.0,
            'total_fee': 0.0,
            'net_profit': 0.0,
            'deal_count': 0,
            'position_count': 0,
            'has_open_position': False,
            'account_signature': pos['account_signature'],
            'key': key,

```

```

        'timestamp': pos['timestamp'] # Keep one timestamp
    }

    agg = aggregated[key]
    agg['total_profit'] += pos['total_profit']
    agg['total_commission'] += pos['total_commission']
    agg['total_swap'] += pos['total_swap']
    agg['total_fee'] += pos['total_fee']
    agg['net_profit'] += pos['net_profit']
    agg['deal_count'] += pos['deal_count']
    agg['position_count'] += 1
    agg['has_open_position'] = agg['has_open_position'] or bool(pos.get('has_open_position'))

    # Update timestamp to latest in group (usually irrelevant for same day)
    if pos['timestamp'] > agg['timestamp']:
        agg['timestamp'] = pos['timestamp']

# Round final values
for key, agg in aggregated.items():
    agg['total_profit'] = round(agg['total_profit'], 2)
    agg['total_commission'] = round(agg['total_commission'], 2)
    agg['total_swap'] = round(agg['total_swap'], 2)
    agg['total_fee'] = round(agg['total_fee'], 2)
    agg['net_profit'] = round(agg['net_profit'], 2)

return (
    list(aggregated.values()),
    unmatched,
    log_messages
)

```

Step 7.3 — Build

trader_companion/trade_limit_manager.py

What to do. Create the file `trader_companion/trade_limit_manager.py` in your project root and paste in the code shown in this step. The file is **388** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from `requirements.txt`.

What this file is for. Enforces per-account daily loss and position-size limits. The guardrail that prevents one bad day from killing a funded account.

trader_companion/trade_limit_manager.py

388 loc · 2 classes · 4 functions · 5 imports

Enforces per-account daily loss and position-size limits. The guardrail that prevents one bad day from killing a funded account. It defines **2** classes and **4** module-level functions, across **388** lines. Module docstring: *Trade Limit Manager for Combine-Based Trading*

Module docstring

Trade Limit Manager for Combine-Based Trading Manages trade limits based on the number of combines purchased and active MT5 trades

Python concepts demonstrated in this module

• Classes and objects • Comprehensions • Context managers (with-statements) • Dunder methods • Exceptions • f-strings • Lambdas • Type hints

Imports — what each one is for

```
import sys
```

Why imported. Access to the running interpreter — argv, stdin/stdout, exit codes, the import search path, and the platform name.

Used in this file. sys.path, sys.path.append

Example. sys.exit(1) # terminate the process with a non-zero status

Other common scenarios.

- sys.argv[1:] reads the command-line arguments
- sys.path.insert(0, 'extra') prepends to the import search path
- sys.stderr.write(msg) writes to standard error
- sys.platform tells you 'win32', 'linux', or 'darwin'
- sys.modules is the global cache of already-imported modules

```
import os
```

Why imported. Operating-system interface. Path manipulation, environment variables, working-directory operations, exit codes, and thin wrappers around POSIX/Windows syscalls.

Used in this file. os.path, os.path.dirname, os.path.exists

Example. os.path.join(root, 'subdir', 'file.txt') # joins with the OS-correct separator

Other common scenarios.

- os.environ.get('API_KEY') to read environment variables
- os.makedirs(path, exist_ok=True) to create nested directories
- os.listdir(path) to list a directory's contents
- os.remove(path) / os.rename(src, dst) to delete or move a file
- os.getcwd() / os.chdir(path) to inspect or change the working dir

```
import logging
```

Why imported. Structured, levelled logging. Replaces print() with filterable, formattable output that can route to files, stderr, syslog, or HTTP endpoints.

Used in this file. logging.error, logging.info, logging.warning

Example. `logging.getLogger(__name__).info('connected to %s', host)`

Other common scenarios.

- `.debug()` / `.info()` / `.warning()` / `.error()` / `.exception()`
- `logger.exception()` captures the current traceback automatically
- `logging.basicConfig(level=logging.INFO)` for quick setup
- Handlers and Formatters route logs to files / streams / syslog

```
import time
```

Why imported. Timestamps and sleep. Lower-level than `datetime` — works in Unix-epoch seconds.

Used in this file. `time.time`

Example. `time.sleep(1.5)` # pause this thread for 1.5 seconds

Other common scenarios.

- `time.time()` returns seconds since the epoch
- `time.monotonic()` for measuring elapsed time (never goes backward)
- `time.perf_counter()` for high-precision benchmarking
- `time.strftime` / `time.strptime` mirror the `datetime` versions

```
from typing import Dict
from typing import List
from typing import Optional
from typing import Tuple
```

Why imported. Type-hint primitives — generics, unions, `Optional`, `Callable`, `Protocol`, `TypeVar`, `TypedDict`.

Used in this file. `Dict`, `Tuple`

Example. `def lookup(k: str) -> Optional[int]: ...`

Other common scenarios.

- `List[int]` / `Dict[str, Any]` / `Tuple[int, str]` (or bare `list[int]`+ on 3.9+)
- `Callable[[int, int], int]` for function signatures
- `Protocol` for structural typing (duck typing with checks)
- `TypeVar('T')` for generic functions and classes

Classes

```
class TradeLimit
```

Represents trade limit information for a combine

Code (copy-paste safe)

```
class TradeLimit:
    """Represents trade limit information for a combine"""
    def __init__(self, combine_id: str, max_trades: int, account_name: str = ""):
        self.combine_id = combine_id
        self.max_trades = max_trades
```



```

        self.account_name = account_name
        self.active_trades = 0
        self.last_check = 0

    def can_place_trade(self) -> bool:
        """Check if a new trade can be placed"""
        return self.active_trades < self.max_trades

    def remaining_trades(self) -> int:
        """Get number of remaining trade slots"""
        return max(0, self.max_trades - self.active_trades)

```

Method: TradeLimit.__init__

```
def __init__(self, combine_id: str, max_trades: int, account_name: str='')
```

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `combine_id: str`, `max_trades: int`, `account_name: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **__init__** — The constructor. Runs after Python has allocated the instance; assigns starting attribute values to `self`.

Step by step (top-level body)

1. Assigns `self.combine_id = combine_id`.
2. Assigns `self.max_trades = max_trades`.
3. Assigns `self.account_name = account_name`.
4. Assigns `self.active_trades = 0`.
5. Assigns `self.last_check = 0`.

Code (copy-paste safe)

```

def __init__(self, combine_id: str, max_trades: int, account_name: str = ""):
    self.combine_id = combine_id
    self.max_trades = max_trades
    self.account_name = account_name
    self.active_trades = 0
    self.last_check = 0

```

Method: TradeLimit.can_place_trade

```
def can_place_trade(self) -> bool
```

Check if a new trade can be placed

Why these Python concepts were used

- **return type annotation** — Annotated return type `-> bool`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.

Step by step (top-level body)

1. **return** self.active_trades < self.max_trades.

Code (copy-paste safe)

```
def can_place_trade(self) -> bool:
    """Check if a new trade can be placed"""
    return self.active_trades < self.max_trades
```

Method: TradeLimit.remaining_trades

```
def remaining_trades(self) -> int
```

Get number of remaining trade slots

Why these Python concepts were used

- **return type annotation** — Annotated return type -> int. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.

Step by step (top-level body)

1. **return** max(0, self.max_trades - self.active_trades).

Code (copy-paste safe)

```
def remaining_trades(self) -> int:
    """Get number of remaining trade slots"""
    return max(0, self.max_trades - self.active_trades)
```

```
class TradeLimitManager
```

Manages trade limits across multiple combines

Code (copy-paste safe)

```
class TradeLimitManager:
    """Manages trade limits across multiple combines"""

    def __init__(self, max_trades: int = 5):
        self.combines: Dict[str, TradeLimit] = {}
        self.mt5 = None
        self.last_mt5_check = 0
        self.check_interval = 5 # Check MT5 every 5 seconds
        self.max_trades = max_trades # Store the max trades limit

        # Default combine configurations
        self.default_combines = {
            "5_combine": 5,
            "10_combine": 10,
            "15_combine": 15,
            "20_combine": 20
        }

        # Add a default combine with the specified max trades
        self.add_combine("default", max_trades, "Default Account")

        self._initialize_mt5()

    def _initialize_mt5(self):
        """Initialize MT5 connection - MT5 API will be set by the main application"""
```

```

# MT5 API instance will be set by the application through set_mt5_api() method
self.mt5 = None

def add_combine(self, combine_id: str, max_trades: int, account_name: str = "") -> bool:
    """Add a new combine with specified trade limit"""
    try:
        self.combines[combine_id] = TradeLimit(combine_id, max_trades, account_name)
        logging.info(f"■ Added combine {combine_id} with {max_trades} trade limit")
        return True
    except Exception as e:
        logging.error(f"■ Failed to add combine {combine_id}: {e}")
        return False

def remove_combine(self, combine_id: str) -> bool:
    """Remove a combine"""
    try:
        if combine_id in self.combines:
            del self.combines[combine_id]
            logging.info(f"■ Removed combine {combine_id}")
            return True
        else:
            logging.warning(f"■■ Combine {combine_id} not found")
            return False
    except Exception as e:
        logging.error(f"■ Failed to remove combine {combine_id}: {e}")
        return False

def set_mt5_api(self, mt5_api):
    """Set the MT5 API instance from the main application"""
    self.mt5 = mt5_api
    if mt5_api:
        logging.info("■ MT5 API set for trade limit checking")

def get_active_trades_from_mt5(self) -> int:
    """Get number of active trades from MT5 terminal"""
    if not self.mt5:
        return 0

    try:
        # Initialize MT5 if not already done
        if not self.mt5.initialize():
            logging.error("■ Failed to initialize MT5 for trade checking")
            return 0

        # Get all open positions
        positions = self.mt5.positions_get()
        if positions is None:
            positions = []

        active_count = len(positions)
        logging.info(f"■ MT5 active trades: {active_count}")

        # Get detailed position info
        if active_count > 0:
            logging.info("■ Active positions:")
            for i, pos in enumerate(positions):
                symbol = pos.symbol
                volume = pos.volume
                position_type = "BUY" if pos.type == 0 else "SELL"
                profit = pos.profit
    
```

```

        logging.info(f"    {i+1}. {symbol} {position_type} {volume} lots (P&L: {profit:.2f})")

    # Shutdown MT5 connection
    self.mt5.shutdown()

    return active_count

except Exception as e:
    logging.error(f"■ Error checking MT5 active trades: {e}")
    try:
        self.mt5.shutdown()
    except:
        pass
    return 0

def update_active_trades_all_combines(self) -> bool:
    """Update active trade counts for all combines from MT5"""
    current_time = time.time()

    # Only check if enough time has passed
    if current_time - self.last_mt5_check < self.check_interval:
        return True

    try:
        total_active = self.get_active_trades_from_mt5()

        # Update all combines with current active trade count
        for combine_id, combine in self.combines.items():
            combine.active_trades = total_active
            combine.last_check = current_time
            logging.info(f"■ {combine_id}: {combine.active_trades}/{combine.max_trades} trades active")

        self.last_mt5_check = current_time
        return True

    except Exception as e:
        logging.error(f"■ Failed to update active trades: {e}")
        return False

def can_place_trade(self, combine_id: str = None) -> Tuple[bool, str]:
    """
    Check if a new trade can be placed
    Returns: (can_place, reason)
    """
    # Update active trades from MT5
    self.update_active_trades_all_combines()

    # Use default combine if none specified
    if not combine_id:
        combine_id = "default"

    if combine_id in self.combines:
        # Check specific combine
        combine = self.combines[combine_id]
        if combine.can_place_trade():
            remaining = combine.remaining_trades()
            return True, f"■ Can place trade. {remaining} slots remaining for {combine_id}"
        else:
            error_message = f"■ Trade limit reached for {combine_id} ({combine.active_trades}/{combine.max_trades})"
            # Show popup alert for trade limit exceeded

```

```

        popup_message = (f"■ TRADE LIMIT EXCEEDED ■\n\n"
                          f"Account: {combine_id}\n"
                          f"Active Trades: {combine.active_trades}\n"
                          f"Maximum Allowed: {combine.max_trades}\n\n"
                          f"You have reached your combine purchase limit!\n"
                          f"Close existing trades or purchase more combines to continue trading.")

        show_trade_limit_alert(popup_message, "Trade Limit Exceeded")

        return False, error_message

    elif self.combines:
        # Check if any combine allows trading
        available_combines = []
        for cid, combine in self.combines.items():
            if combine.can_place_trade():
                available_combines.append((cid, combine.remaining_trades()))

        if available_combines:
            total_remaining = sum(remaining for _, remaining in available_combines)
            combine_names = ", ".join(f"{cid}({remaining})" for cid, remaining in available_combines)
            return True, f"■ Can place trade. Available combines: {combine_names}"
        else:
            active_info = ", ".join(f"{cid}({c.active_trades}/{c.max_trades})"
                                     for cid, c in self.combines.items())
            error_message = f"■ All trade limits reached. Active: {active_info}"

            # Show popup alert for all limits exceeded
            popup_message = (f"■ ALL TRADE LIMITS EXCEEDED ■\n\n"
                              f"All your combines have reached their maximum trades:\n"
                              f"{active_info}\n\n"
                              f"Please close existing trades or purchase more combines to continue trading.")

            show_trade_limit_alert(popup_message, "All Trade Limits Exceeded")

            return False, error_message

    else:
        error_message = "■ No combines configured"
        show_trade_limit_alert("No trading combines configured! Please set up your combine limits.",
                               "Configuration Error")
        return False, error_message

def get_current_trade_count(self) -> int:
    """Get the current number of active trades from MT5"""
    try:
        return self.get_active_trades_from_mt5()
    except Exception as e:
        logging.error(f"Failed to get current trade count: {e}")
        return 0

def get_trade_summary(self) -> Dict:
    """Get comprehensive trade limit summary"""
    self.update_active_trades_all_combines()

    summary = {
        "total_combines": len(self.combines),
        "total_max_trades": sum(c.max_trades for c in self.combines.values()),
        "total_active_trades": sum(c.active_trades for c in self.combines.values()),
        "total_remaining": sum(c.remaining_trades() for c in self.combines.values()),
    }

```

```

        "mt5_active_trades": self.get_active_trades_from_mt5() if self.mt5 else 0,
        "combines": {}
    }

    for combine_id, combine in self.combines.items():
        summary["combines"][combine_id] = {
            "max_trades": combine.max_trades,
            "active_trades": combine.active_trades,
            "remaining": combine.remaining_trades(),
            "can_trade": combine.can_place_trade(),
            "account_name": combine.account_name
        }

    return summary

def print_trade_status(self):
    """Print detailed trade status"""
    summary = self.get_trade_summary()

    print("\n" + "="*60)
    print("■ TRADE LIMIT STATUS")
    print("="*60)

    print(f"■ Total Combines: {summary['total_combines']}")
    print(f"■ Total Trade Capacity: {summary['total_max_trades']}")
    print(f"■ Active Trades (MT5): {summary['mt5_active_trades']}")
    print(f"■ Available Slots: {summary['total_remaining']}")

    if summary['combines']:
        print(f"\n■ COMBINE DETAILS:")
        for combine_id, info in summary['combines'].items():
            status = "■ Available" if info['can_trade'] else "■ Full"
            account_info = f"({info['account_name']})" if info['account_name'] else ""
            print(f"    {combine_id}{account_info}: {info['active_trades']}/{info['max_trades']} {status}")

    can_trade, reason = self.can_place_trade()
    print(f"\n■ TRADE STATUS: {reason}")
    print("="*60)

def load_combines_from_config(self, config_file: str = "combine_config.txt") -> bool:
    """Load combine configuration from file"""
    try:
        if not os.path.exists(config_file):
            # Create default config file
            self.create_default_config(config_file)

        with open(config_file, 'r') as f:
            for line in f:
                line = line.strip()
                if line and not line.startswith('#'):
                    parts = line.split(',')
                    if len(parts) >= 2:
                        combine_id = parts[0].strip()
                        max_trades = int(parts[1].strip())
                        account_name = parts[2].strip() if len(parts) > 2 else ""
                        self.add_combine(combine_id, max_trades, account_name)

        logging.info(f"■ Loaded combines from {config_file}")
        return True

    except Exception as e:

```

```

        logging.error(f"■ Failed to load combines from {config_file}: {e}")
        return False

def create_default_config(self, config_file: str = "combine_config.txt"):
    """Create default combine configuration file"""
    try:
        with open(config_file, 'w') as f:
            f.write("# Combine Configuration File\n")
            f.write("# Format: combine_id, max_trades, account_name (optional)\n")
            f.write("# Example: 5_combine, 5, Main Account\n\n")

            f.write("# Default configurations (modify as needed)\n")
            f.write("# 5_combine, 5, Primary\n")
            f.write("# 10_combine, 10, Secondary\n")
            f.write("# 15_combine, 15, Advanced\n")
            f.write("# 20_combine, 20, Professional\n")

        logging.info(f"■ Created default config file: {config_file}")

    except Exception as e:
        logging.error(f"■ Failed to create default config: {e}")

```

Method: TradeLimitManager.__init__

```
def __init__(self, max_trades: int=5)
```

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `max_trades: int`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **__init__** — The constructor. Runs after Python has allocated the instance; assigns starting attribute values to self.

Step by step (top-level body)

1. Declares `self.combines: Dict[str, TradeLimit] = {}`.
2. Assigns `self.mt5 = None`.
3. Assigns `self.last_mt5_check = 0`.
4. Assigns `self.check_interval = 5`.
5. Assigns `self.max_trades = max_trades`.
6. Assigns `self.default_combines = {'5_combine': 5, '10_combine': 10, '15_combine': 15, '20_....`
7. Calls `self.add_combine(...)` for its side effect.
8. Calls `self._initialize_mt5(...)` for its side effect.

Code (copy-paste safe)

```

def __init__(self, max_trades: int = 5):
    self.combines: Dict[str, TradeLimit] = {}
    self.mt5 = None
    self.last_mt5_check = 0
    self.check_interval = 5 # Check MT5 every 5 seconds
    self.max_trades = max_trades # Store the max trades limit

```

```

# Default combine configurations
self.default_combines = {
    "5_combine": 5,
    "10_combine": 10,
    "15_combine": 15,
    "20_combine": 20
}

# Add a default combine with the specified max trades
self.add_combine("default", max_trades, "Default Account")

self._initialize_mt5()

```

Method: TradeLimitManager._initialize_mt5

```
def _initialize_mt5(self)
```

Initialize MT5 connection - MT5 API will be set by the main application

Step by step (top-level body)

1. Assigns self.mt5 = None.

Code (copy-paste safe)

```

def _initialize_mt5(self):
    """Initialize MT5 connection - MT5 API will be set by the main application"""
    # MT5 API instance will be set by the application through set_mt5_api() method
    self.mt5 = None

```

Method: TradeLimitManager.add_combine

```
def add_combine(self, combine_id: str, max_trades: int, account_name: str='') -> bool
```

Add a new combine with specified trade limit

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., combine_id: str, max_trades: int, account_name: str). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type -> bool. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. try block with 1 **except** clause.

Code (copy-paste safe)

```
def add_combine(self, combine_id: str, max_trades: int, account_name: str = "") -> bool:
```



```

        """Add a new combine with specified trade limit"""
        try:
            self.combines[combine_id] = TradeLimit(combine_id, max_trades, account_name)
            logging.info(f"■ Added combine {combine_id} with {max_trades} trade limit")
            return True
        except Exception as e:
            logging.error(f"■ Failed to add combine {combine_id}: {e}")
            return False

```

Method: TradeLimitManager.remove_combine

```
def remove_combine(self, combine_id: str) -> bool
```

Remove a combine

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `combine_id: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> bool`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def remove_combine(self, combine_id: str) -> bool:
    """Remove a combine"""
    try:
        if combine_id in self.combines:
            del self.combines[combine_id]
            logging.info(f"■ Removed combine {combine_id}")
            return True
        else:
            logging.warning(f"■ Combine {combine_id} not found")
            return False
    except Exception as e:
        logging.error(f"■ Failed to remove combine {combine_id}: {e}")
        return False

```

Method: TradeLimitManager.set_mt5_api

```
def set_mt5_api(self, mt5_api)
```

Set the MT5 API instance from the main application

Step by step (top-level body)

1. Assigns `self.mt5 = mt5_api`.
2. `if mt5_api`: branches conditionally.

Code (copy-paste safe)

```
def set_mt5_api(self, mt5_api):
    """Set the MT5 API instance from the main application"""
    self.mt5 = mt5_api
    if mt5_api:
        logging.info("■ MT5 API set for trade limit checking")
```

Method: `TradeLimitManager.get_active_trades_from_mt5`

```
def get_active_trades_from_mt5(self) -> int

    Get number of active trades from MT5 terminal
```

Why these Python concepts were used

- **return type annotation** — Annotated return type `-> int`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **try/except** — Wraps part of the body in `try/except`. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. `if not self.mt5`: branches conditionally.
2. `try` block with 1 `except` clause.

Code (copy-paste safe)

```
def get_active_trades_from_mt5(self) -> int:
    """Get number of active trades from MT5 terminal"""
    if not self.mt5:
        return 0

    try:
        # Initialize MT5 if not already done
        if not self.mt5.initialize():
            logging.error("■ Failed to initialize MT5 for trade checking")
            return 0

        # Get all open positions
        positions = self.mt5.positions_get()
        if positions is None:
            positions = []

        active_count = len(positions)
        logging.info(f"■ MT5 active trades: {active_count}")

        # Get detailed position info
```

```

        if active_count > 0:
            logging.info("■ Active positions:")
            for i, pos in enumerate(positions):
                symbol = pos.symbol
                volume = pos.volume
                position_type = "BUY" if pos.type == 0 else "SELL"
                profit = pos.profit
                logging.info(f"    {i+1}. {symbol} {position_type} {volume} lots (P&L: {profit:.2f})")

        # Shutdown MT5 connection
        self.mt5.shutdown()

        return active_count

    except Exception as e:
        logging.error(f"■ Error checking MT5 active trades: {e}")
        try:
            self.mt5.shutdown()
        except:
            pass
        return 0

```

Method: TradeLimitManager.update_active_trades_all_combines

```
def update_active_trades_all_combines(self) -> bool
```

Update active trade counts for all combines from MT5

Why these Python concepts were used

- **return type annotation** — Annotated return type -> bool. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `current_time = time.time()`.
2. `if current_time - self.last_mt5_check < self.check_interval`: branches conditionally.
3. `try` block with 1 `except` clause.

Code (copy-paste safe)

```

def update_active_trades_all_combines(self) -> bool:
    """Update active trade counts for all combines from MT5"""
    current_time = time.time()

    # Only check if enough time has passed
    if current_time - self.last_mt5_check < self.check_interval:
        return True

    try:
        total_active = self.get_active_trades_from_mt5()

```

```

        # Update all combines with current active trade count
        for combine_id, combine in self.combines.items():
            combine.active_trades = total_active
            combine.last_check = current_time
            logging.info(f"■ {combine_id}: {combine.active_trades}/{combine.max_trades} trades active")

        self.last_mt5_check = current_time
        return True

    except Exception as e:
        logging.error(f"■ Failed to update active trades: {e}")
        return False

```

Method: TradeLimitManager.can_place_trade

```
def can_place_trade(self, combine_id: str=None) -> Tuple[bool, str]
```

Check if a new trade can be placed Returns: (can_place, reason)

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `combine_id: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> Tuple[bool, str]`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to `sum`, `any`, `all`, or `max` where the intermediate list would be wasted.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Calls `self.update_active_trades_all_combines(...)` for its side effect.
2. `if not combine_id:` branches conditionally.
3. `if combine_id in self.combines:` branches conditionally (with an **else/elif** arm).

Code (copy-paste safe)

```

def can_place_trade(self, combine_id: str = None) -> Tuple[bool, str]:
    """
    Check if a new trade can be placed
    Returns: (can_place, reason)
    """
    # Update active trades from MT5
    self.update_active_trades_all_combines()

    # Use default combine if none specified
    if not combine_id:
        combine_id = "default"

    if combine_id in self.combines:
        # Check specific combine
        combine = self.combines[combine_id]

```

```

if combine.can_place_trade():
    remaining = combine.remaining_trades()
    return True, f"■ Can place trade. {remaining} slots remaining for {combine_id}"
else:
    error_message = f"■ Trade limit reached for {combine_id} ({combine.active_trades}/{combine.max_trades})"

    # Show popup alert for trade limit exceeded
    popup_message = (f"■ TRADE LIMIT EXCEEDED ■\n\n"
                    f"Account: {combine_id}\n"
                    f"Active Trades: {combine.active_trades}\n"
                    f"Maximum Allowed: {combine.max_trades}\n\n"
                    f"You have reached your combine purchase limit!\n"
                    f"Close existing trades or purchase more combines to continue trading.")

    show_trade_limit_alert(popup_message, "Trade Limit Exceeded")

    return False, error_message

elif self.combines:
    # Check if any combine allows trading
    available_combines = []
    for cid, combine in self.combines.items():
        if combine.can_place_trade():
            available_combines.append((cid, combine.remaining_trades()))

    if available_combines:
        total_remaining = sum(remaining for _, remaining in available_combines)
        combine_names = ", ".join(f"{cid}({remaining})" for cid, remaining in available_combines)
        return True, f"■ Can place trade. Available combines: {combine_names}"
    else:
        active_info = ", ".join(f"{cid}({c.active_trades}/{c.max_trades})"
                                for cid, c in self.combines.items())
        error_message = f"■ All trade limits reached. Active: {active_info}"

        # Show popup alert for all limits exceeded
        popup_message = (f"■ ALL TRADE LIMITS EXCEEDED ■\n\n"
                        f"All your combines have reached their maximum trades:\n"
                        f"{active_info}\n\n"
                        f>Please close existing trades or purchase more combines to continue trading.")

        show_trade_limit_alert(popup_message, "All Trade Limits Exceeded")

        return False, error_message

else:
    error_message = "■ No combines configured"
    show_trade_limit_alert("No trading combines configured! Please set up your combine limits.",
                          "Configuration Error")
    return False, error_message

```

Method: TradeLimitManager.get_current_trade_count

```
def get_current_trade_count(self) -> int
```

Get the current number of active trades from MT5

Why these Python concepts were used

- **return type annotation** — Annotated return type -> int. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def get_current_trade_count(self) -> int:
    """Get the current number of active trades from MT5"""
    try:
        return self.get_active_trades_from_mt5()
    except Exception as e:
        logging.error(f"Failed to get current trade count: {e}")
        return 0
```

Method: TradeLimitManager.get_trade_summary

```
def get_trade_summary(self) -> Dict
    Get comprehensive trade limit summary
```

Why these Python concepts were used

- **return type annotation** — Annotated return type -> Dict. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Calls `self.update_active_trades_all_combines(...)` for its side effect.
2. Assigns `summary = {'total_combines': len(self.combines), 'total_max_trades'...`
3. **for** `(combine_id, combine)` in `self.combines.items()`: iterates.
4. **return** summary.

Code (copy-paste safe)

```
def get_trade_summary(self) -> Dict:
    """Get comprehensive trade limit summary"""
    self.update_active_trades_all_combines()

    summary = {
        "total_combines": len(self.combines),
        "total_max_trades": sum(c.max_trades for c in self.combines.values()),
        "total_active_trades": sum(c.active_trades for c in self.combines.values()),
        "total_remaining": sum(c.remaining_trades() for c in self.combines.values()),
        "mt5_active_trades": self.get_active_trades_from_mt5() if self.mt5 else 0,
```

```

        "combines": {}
    }

    for combine_id, combine in self.combines.items():
        summary["combines"][combine_id] = {
            "max_trades": combine.max_trades,
            "active_trades": combine.active_trades,
            "remaining": combine.remaining_trades(),
            "can_trade": combine.can_place_trade(),
            "account_name": combine.account_name
        }

    return summary

```

Method: TradeLimitManager.print_trade_status

```
def print_trade_status(self)
    Print detailed trade status
```

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns summary = self.get_trade_summary().
2. Calls print(...) for its side effect.
3. Calls print(...) for its side effect.
4. Calls print(...) for its side effect.
5. Calls print(...) for its side effect.
6. Calls print(...) for its side effect.
7. Calls print(...) for its side effect.
8. Calls print(...) for its side effect.
9. if summary['combines']: branches conditionally.
10. Assigns (can_trade, reason) = self.can_place_trade().
11. Calls print(...) for its side effect.
12. Calls print(...) for its side effect.

Code (copy-paste safe)

```
def print_trade_status(self):
    """Print detailed trade status"""
    summary = self.get_trade_summary()

    print("\n" + "="*60)
    print("■ TRADE LIMIT STATUS")
    print("="*60)

```

```

print(f"■ Total Combines: {summary['total_combines']}")
print(f"■ Total Trade Capacity: {summary['total_max_trades']}")
print(f"■ Active Trades (MT5): {summary['mt5_active_trades']}")
print(f"■ Available Slots: {summary['total_remaining']}")

if summary['combines']:
    print(f"\n■ COMBINE DETAILS:")
    for combine_id, info in summary['combines'].items():
        status = "■ Available" if info['can_trade'] else "■ Full"
        account_info = f"({info['account_name']})" if info['account_name'] else ""
        print(f"    {combine_id}{account_info}: {info['active_trades']}/{info['max_trades']} {status}")

    can_trade, reason = self.can_place_trade()
    print(f"\n■ TRADE STATUS: {reason}")
    print("="*60)

```

Method: TradeLimitManager.load_combines_from_config

```
def load_combines_from_config(self, config_file: str='combine_config.txt') -> bool
```

Load combine configuration from file

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `config_file: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> bool`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def load_combines_from_config(self, config_file: str = "combine_config.txt") -> bool:
    """Load combine configuration from file"""
    try:
        if not os.path.exists(config_file):
            # Create default config file
            self.create_default_config(config_file)

        with open(config_file, 'r') as f:
            for line in f:
                line = line.strip()

```



```

        if line and not line.startswith('#'):
            parts = line.split(',')
            if len(parts) >= 2:
                combine_id = parts[0].strip()
                max_trades = int(parts[1].strip())
                account_name = parts[2].strip() if len(parts) > 2 else ""
                self.add_combine(combine_id, max_trades, account_name)

        logging.info(f"■ Loaded combines from {config_file}")
        return True

    except Exception as e:
        logging.error(f"■ Failed to load combines from {config_file}: {e}")
        return False

```

Method: TradeLimitManager.create_default_config

```
def create_default_config(self, config_file: str='combine_config.txt')
```

Create default combine configuration file

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `config_file: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def create_default_config(self, config_file: str = "combine_config.txt"):
    """Create default combine configuration file"""
    try:
        with open(config_file, 'w') as f:
            f.write("# Combine Configuration File\n")
            f.write("# Format: combine_id, max_trades, account_name (optional)\n")
            f.write("# Example: 5_combine, 5, Main Account\n\n")

            f.write("# Default configurations (modify as needed)\n")
            f.write("# 5_combine, 5, Primary\n")
            f.write("# 10_combine, 10, Secondary\n")
            f.write("# 15_combine, 15, Advanced\n")
            f.write("# 20_combine, 20, Professional\n")

        logging.info(f"■ Created default config file: {config_file}")

    except Exception as e:

```

```
logging.error(f"■ Failed to create default config: {e}")
```

Functions

```
def show_trade_limit_alert(message: str, title: str='Trade Limit Exceeded')
```

Show a popup alert when trade limits are exceeded

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `message: str, title: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **if POPUP_AVAILABLE**: branches conditionally (with an **else/elif** arm).

Code (copy-paste safe)

```
def show_trade_limit_alert(message: str, title: str = "Trade Limit Exceeded"):
    """
    Show a popup alert when trade limits are exceeded
    """
    if POPUP_AVAILABLE:
        try:
            # Create a temporary root window (hidden)
            root = tk.Tk()
            root.withdraw() # Hide the main window

            # Show the error message
            messagebox.showerror(title, message)

            # Destroy the root window
            root.destroy()

        except Exception as e:
            logging.error(f"Failed to show popup alert: {e}")
            print(f"■■ Popup alert failed: {message}")
    else:
        # Fallback to console output
        print(f"■ {title}: {message}")
```

```
def check_trade_limit(combine_id: str=None) -> Tuple[bool, str]
```

Quick function to check if a trade can be placed Returns: (can_place, reason)

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `combine_id: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and

human readers. They are the fastest way to make a public function self-explanatory.

- **return type annotation** — Annotated return type -> Tuple[bool, str]. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.

Step by step (top-level body)

1. **return** trade_limit_manager.can_place_trade(combine_id).

Code (copy-paste safe)

```
def check_trade_limit(combine_id: str = None) -> Tuple[bool, str]:
    """
    Quick function to check if a trade can be placed
    Returns: (can_place, reason)
    """
    return trade_limit_manager.can_place_trade(combine_id)
```

```
def get_trade_summary() -> Dict
```

Get current trade limit summary

Why these Python concepts were used

- **return type annotation** — Annotated return type -> Dict. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.

Step by step (top-level body)

1. **return** trade_limit_manager.get_trade_summary().

Code (copy-paste safe)

```
def get_trade_summary() -> Dict:
    """Get current trade limit summary"""
    return trade_limit_manager.get_trade_summary()
```

```
def initialize_trade_limits(config_file: str='combine_config.txt') -> bool
```

Initialize trade limits from configuration

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., config_file: str). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.

- **return type annotation** — Annotated return type -> bool. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.

Step by step (top-level body)

1. **return** trade_limit_manager.load_combines_from_config(config_file).

Code (copy-paste safe)

```
def initialize_trade_limits(config_file: str = "combine_config.txt") -> bool:
    """Initialize trade limits from configuration"""
    return trade_limit_manager.load_combines_from_config(config_file)
```

Step 7.4 — Build trader_companion/hedge_protector.py

What to do. Create the file trader_companion/hedge_protector.py in your project root and paste in the code shown in this step. The file is **576** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from requirements.txt.

What this file is for. The hedging engine. Watches a primary account for net exposure and opens offsetting positions on a hedge account so that drawdown on the funded leg is bounded.

trader_companion/hedge_protector.py

576 loc · 1 classes · 0 functions · 6 imports

The hedging engine. Watches a primary account for net exposure and opens offsetting positions on a hedge account so that drawdown on the funded leg is bounded. It defines **1** class, across **576** lines.

Module docstring: *Hedge Protector — Double-SL Prevention Engine*

Module docstring

Hedge Protector — Double-SL Prevention Engine

Monitors both MT5 and Tradovate positions in real-time. When a trade closes via SL on one side, immediately closes the hedge on the other side. When a trade closes via TP, leaves the hedge running (chance for double TP).

Matching: MT5 comment format = "{TradovateAccountNumber}_{PhaseCode}" e.g., "FNFTCHHARRISONOUKA85625_CH1", "MFFU_EV_ST_P326057008_FD1"

Architecture: - MT5: Polls positions every 500ms, detects removals - Tradovate: REST polling every 5s for position change detection - Engine: Matches pairs, determines SL vs TP, triggers close

Python concepts demonstrated in this module

• Classes and objects • Decorators • Dunder methods • Exceptions • f-strings • Lambdas • Properties

Imports — what each one is for

import time

Why imported. Timestamps and sleep. Lower-level than datetime — works in Unix-epoch seconds.

Used in this file. time.sleep

Example. time.sleep(1.5) # pause this thread for 1.5 seconds

Other common scenarios.

- time.time() returns seconds since the epoch

- `time.monotonic()` for measuring elapsed time (never goes backward)
- `time.perf_counter()` for high-precision benchmarking
- `time.strptime` / `time.strftime` mirror the `datetime` versions

`import json`

Why imported. JSON encoding and decoding — the standard wire format for config files, API payloads, and inter-process messages.

Used in this file. *No direct attribute access detected — likely imported for side effects, re-export, or string references.*

Example. `json.loads(response.text)` # parse a JSON string to dict

Other common scenarios.

- `json.dumps(obj, indent=2)` for pretty-printing
- `json.dump(obj, file)` / `json.load(file)` for file I/O
- `default=str` handles non-serialisable types like `datetime`
- `object_hook=` customises decoding (e.g., to `dataclass` instances)

`import threading`

Why imported. Threads and synchronisation primitives. Used when work is I/O-bound and the GIL is not a bottleneck (the GIL prevents speed-up on CPU-bound work).

Used in this file. `threading.Thread`

Example. `Thread(target=worker, args=(q,), daemon=True).start()`

Other common scenarios.

- `Lock` / `RLock` to guard shared state
- `Event` for one-shot signals between threads
- `Queue` (from `queue`) for producer/consumer patterns
- `threading.local()` for thread-local storage

`import logging`

Why imported. Structured, levelled logging. Replaces `print()` with filterable, formattable output that can route to files, `stderr`, `syslog`, or HTTP endpoints.

Used in this file. *No direct attribute access detected — likely imported for side effects, re-export, or string references.*

Example. `logging.getLogger(__name__).info('connected to %s', host)`

Other common scenarios.

- `.debug()` / `.info()` / `.warning()` / `.error()` / `.exception()`
- `logger.exception()` captures the current traceback automatically
- `logging.basicConfig(level=logging.INFO)` for quick setup
- `Handlers` and `Formatters` route logs to files / streams / `syslog`

```
from datetime import datetime
from datetime import timezone
```

Why imported. Dates, times, durations. The fundamental temporal types: date, time, datetime, timedelta, timezone.

Used in this file. *No direct attribute access detected — likely imported for side effects, re-export, or string references.*

Example. `datetime.now() - timedelta(days=7)` # one week ago

Other common scenarios.

- `datetime.strptime(s, '%Y-%m-%d')` parses a string
- `datetime.strftime(fmt)` formats one
- `datetime.fromtimestamp(n)` converts a Unix epoch
- `datetime.utcnow()` for UTC (better: `datetime.now(timezone.utc)`)

```
from collections import defaultdict
```

Why imported. Specialised container datatypes that the dict / list / tuple / set primitives don't cover.

Used in this file. *No direct attribute access detected — likely imported for side effects, re-export, or string references.*

Example. `Counter(words)` # multiset that counts occurrences

Other common scenarios.

- `defaultdict(list)` auto-creates a default value on missing keys
- `deque(maxlen=N)` is a double-ended bounded queue
- `OrderedDict` (mostly redundant since 3.7 — dicts preserve order)
- `namedtuple('Point', 'x y')` for tiny immutable record types

Classes

```
class HedgeProtector
```

Real-time hedge protection engine. Monitors MT5 and Tradovate positions. When a position closes: - If SL hit → close the matching hedge on the other platform - If TP hit → do nothing (let hedge run for potential double TP) Usage: protector = HedgeProtector(mt5_api=mt5_api_instance, tradovate_accounts={firm_name: tradovate_account_instance, ...}, on_event=callback_fn, # (event_type, message) for UI logging) protector.start() ... protector.stop()

Code (copy-paste safe)

```
class HedgeProtector:
    """
    Real-time hedge protection engine.

    Monitors MT5 and Tradovate positions. When a position closes:
    - If SL hit → close the matching hedge on the other platform
    - If TP hit → do nothing (let hedge run for potential double TP)

    Usage:
    protector = HedgeProtector(
```


[illegible]


```

        for ticket, prev_pos in self._mt5_positions.items():
            if ticket not in current:
                self._on_mt5_position_closed(prev_pos)

        self._mt5_positions = current

def _on_mt5_position_closed(self, pos):
    """Called when an MT5 position disappears (closed)."""
    ticket = pos['ticket']
    comment = pos.get('comment', '')
    symbol = pos['symbol']
    profit = pos['profit']

    # Skip if no comment (not a hedge trade)
    if not comment or '_' not in comment:
        return

    # Parse the Tradovate account from comment
    # Format: "{TradovateAccountNumber}_{PhaseCode}"
    # e.g., "FNFTCHHARRISONOUKA85625_CH1"
    parts = comment.rsplit('_', 1)
    if len(parts) < 2:
        return

    # The account number is everything before the last underscore
    # But phase codes can have underscores too (e.g., FA_210126)
    # So we need to find the tradovate account by trying different splits
    tv_account_number = self._extract_tradovate_account(comment)
    if not tv_account_number:
        return

    phase_code = comment[len(tv_account_number) + 1:] if len(comment) > len(tv_account_number) else ''

    # Check for duplicate
    hedge_key = f"mt5_{ticket}"
    if hedge_key in self._closed_hedges:
        return
    self._closed_hedges.add(hedge_key)

    # Determine if SL or TP hit
    close_reason = self._determine_mt5_close_reason(pos)

    self.on_event(close_reason,
                  f"MT5 #{ticket} {symbol} closed – {close_reason.upper()} – "
                  f"P&L: ${profit:+.2f} – Comment: {comment}")

    if close_reason == "sl_detected":
        # SL HIT → Close the Tradovate hedge immediately
        self.stats["sl_protected"] += 1
        self._close_tradovate_hedge(tv_account_number, symbol, pos)
    elif close_reason == "tp_detected":
        # TP HIT → Let the hedge run
        self.stats["tp_passed"] += 1
        self.on_event("tp_detected",
                      f"TP hit on MT5 #{ticket} – letting Tradovate hedge run for double TP potential")
        # Cancel remaining Tradovate bracket orders (stop/limit)
        self._cancel_tradovate_orders(tv_account_number)

    # Fire status change callback with account + phase_key from comment
    if self.on_status_change and phase_code:

```

```

        try:
            self.on_status_change(tv_account_number, close_reason, phase_code)
        except Exception as _sc_e:
            self.on_event("error", f"Status change callback failed: {_sc_e}")

def _determine_mt5_close_reason(self, pos):
    """Determine if an MT5 position closed via SL or TP.

    Logic:
    - If profit < 0 → SL (or manual loss close)
    - If profit > 0 AND tp was set AND price reached tp → TP
    - If profit > 0 but no tp set → manual close (treat as neutral, don't close hedge)

    For hedging safety, we treat negative profit as SL.
    """
    profit = pos.get('profit', 0)
    sl = pos.get('sl', 0)
    tp = pos.get('tp', 0)
    price_current = pos.get('price_current', 0)
    price_open = pos.get('price_open', 0)
    pos_type = pos.get('type', 0)

    if profit < 0:
        return "sl_detected"

    if profit >= 0 and tp > 0:
        # Check if price was near TP
        if pos_type == 0: # Buy – TP is above
            if price_current >= tp * 0.999:
                return "tp_detected"
        else: # Sell – TP is below
            if price_current <= tp * 1.001:
                return "tp_detected"

    if profit >= 0:
        return "tp_detected"

    return "sl_detected"

def _extract_tradovate_account(self, comment):
    """Extract Tradovate account number from MT5 comment.

    Tries to match against known connected Tradovate accounts.
    Comment format: "{AccountNumber}_{PhaseCode}"
    """
    # Get all known Tradovate account names
    known_accounts = set()
    for firm_name, info in self._tv_account_info.items():
        acct_name = info.get('name', '')
        if acct_name:
            known_accounts.add(acct_name)

    # Try exact prefix match against known accounts
    for acct_name in known_accounts:
        if comment.startswith(acct_name):
            return acct_name

    # Fallback: split on known phase code patterns
    import re
    # Match everything before _CH\d, _FD\d, _DD\d, _FA, _EV, etc.

```

```

m = re.match(r'^^(.+?)_(CH\d|FD\d|DD\d|FA|EV)', comment)
if m:
    return m.group(1)

# Last resort: everything before the last _
parts = comment.rsplit('_', 1)
return parts[0] if len(parts) == 2 else None

def _close_tradovate_hedge(self, tv_account_number, mt5_symbol, mt5_pos):
    """Close the matching Tradovate position when MT5 SL is hit."""
    self.on_event("close_sent",
                  f"■■■ Closing Tradovate hedge for account {tv_account_number} (MT5 SL hit)")

    # Find which Tradovate connection has this account
    for firm_name, tv_account in self.tradovate_accounts.items():
        try:
            # Check if this connection owns the account
            acct_info = self._tv_account_info.get(firm_name, {})
            acct_name = acct_info.get('name', '')
            acct_id = acct_info.get('id')

            if tv_account_number in acct_name or acct_name in tv_account_number:
                # Found the matching Tradovate connection → liquidate
                self.on_event("close_sent",
                              f"■■■ Liquidating {firm_name} account {acct_name} via API")
                result = tv_account.liquidate_position_api(account_id=acct_id)
                if result:
                    self.on_event("close_sent",
                                  f"■ Tradovate hedge closed for {acct_name}")
                else:
                    self.on_event("error",
                                  f"■ Failed to liquidate Tradovate position on {acct_name}")
                # Cancel remaining bracket orders
                self._cancel_tradovate_orders_by_firm(firm_name)
                return

            # Also check all accounts under this connection
            accounts = tv_account._api_fetch("/account/list")
            if accounts:
                for acct in accounts:
                    name = acct.get('name', '')
                    if tv_account_number in name or name in tv_account_number:
                        aid = acct['id']
                        self.on_event("close_sent",
                                      f"■■■ Liquidating {firm_name} account {name} (id={aid}) via API")
                        result = tv_account.liquidate_position_api(account_id=aid)
                        if result:
                            self.on_event("close_sent",
                                          f"■ Tradovate hedge closed for {name}")
                        else:
                            self.on_event("error",
                                          f"■ Failed to liquidate Tradovate position on {name}")
                        # Cancel remaining bracket orders
                        self._cancel_tradovate_orders_by_firm(firm_name)
                        return

        except Exception as e:
            self.on_event("error", f"Error closing Tradovate hedge on {firm_name}: {e}")

    self.on_event("warn", f"■ No matching Tradovate account found for: {tv_account_number}")

```



```

        self._close_mt5_hedge(acct_name, contract_id)
    else:
        self.stats["tp_passed"] += 1
        self.on_event("tp_detected",
                      f"TP hit on Tradovate {acct_name} – letting MT5 hedge run for double TP potential")
    # Cancel remaining Tradovate bracket orders (stop/limit) after any close
    self._cancel_tradovate_orders_by_firm(firm_name)

    # Fire status change callback – extract phase_key from matching MT5 comment
    if self.on_status_change:
        phase_key = self._get_phase_key_from_mt5(acct_name)
        if phase_key:
            try:
                self.on_status_change(acct_name, close_reason, phase_key)
            except Exception as _sc_e:
                self.on_event("error", f"Status change callback failed: {_sc_e}")

def _close_mt5_hedge(self, tv_account_name, tv_contract_id):
    """Close the matching MT5 position when Tradovate SL is hit."""
    if not self.mt5_api or not mt5:
        self.on_event("error", "Cannot close MT5 hedge – MT5 not connected")
        return

    self.on_event("close_sent",
                  f"■■■ Closing MT5 hedge for Tradovate account {tv_account_name} (Tradovate SL hit)")

    # Find MT5 positions whose comment contains this Tradovate account
    positions = mt5.positions_get()
    if not positions:
        self.on_event("warn", "No open MT5 positions found")
        return

    closed_count = 0
    for pos in positions:
        comment = getattr(pos, 'comment', '')
        if not comment:
            continue
        # Check if comment references this Tradovate account
        if tv_account_name in comment:
            self.on_event("close_sent",
                          f"■■■ Closing MT5 #{pos.ticket} {pos.symbol} (comment: {comment})")
            try:
                success = self.mt5_api.close_trade(pos.ticket)
                if success:
                    self.on_event("close_sent",
                                  f"■ MT5 #{pos.ticket} closed successfully")
                    closed_count += 1
                else:
                    self.on_event("error",
                                  f"■ Failed to close MT5 #{pos.ticket}")
            except Exception as e:
                self.on_event("error", f"Error closing MT5 #{pos.ticket}: {e}")

    if closed_count == 0:
        self.on_event("warn",
                      f"■ No matching MT5 positions found for Tradovate account {tv_account_name}")
    else:
        self.on_event("close_sent",
                      f"■■■ Closed {closed_count} MT5 position(s) for {tv_account_name}")

def _get_phase_key_from_mt5(self, tv_account_name):

```

```

"""Extract phase_key from the MT5 position comment that matches this Tradovate account.

Checks both open positions and the last snapshot for recently closed ones.
Comment format: "{AccountNumber}_{phase_key}"
"""
if not mt5:
    return None
# Check open positions first
positions = mt5.positions_get()
if positions:
    for pos in positions:
        comment = getattr(pos, 'comment', '')
        if comment and tv_account_name in comment:
            suffix = comment[len(tv_account_name):]
            if suffix.startswith('_'):
                return suffix[1:] # strip leading underscore
# Check cached snapshot (position may have just closed)
for ticket, pos_data in self._mt5_positions.items():
    comment = pos_data.get('comment', '')
    if comment and tv_account_name in comment:
        suffix = comment[len(tv_account_name):]
        if suffix.startswith('_'):
            return suffix[1:]
return None

def _cancel_tradovate_orders(self, tv_account_number):
    """Cancel all pending Tradovate orders for the given account number."""
    for firm_name, tv_account in self.tradovate_accounts.items():
        try:
            acct_info = self._tv_account_info.get(firm_name, {})
            acct_name = acct_info.get('name', '')
            acct_id = acct_info.get('id')
            if tv_account_number in acct_name or acct_name in tv_account_number:
                if hasattr(tv_account, 'cancel_all_orders_api'):
                    cancelled = tv_account.cancel_all_orders_api(account_id=acct_id)
                    if cancelled > 0:
                        self.on_event("info",
                                      f"■ Cancelled {cancelled} Tradovate order(s) for {acct_name}")
                return
            # Check all accounts under this connection
            accounts = tv_account._api_fetch("/account/list")
            if accounts:
                for acct in accounts:
                    name = acct.get('name', '')
                    if tv_account_number in name or name in tv_account_number:
                        if hasattr(tv_account, 'cancel_all_orders_api'):
                            cancelled = tv_account.cancel_all_orders_api(account_id=acct['id'])
                            if cancelled > 0:
                                self.on_event("info",
                                              f"■ Cancelled {cancelled} Tradovate order(s) for {name}")
                return
        except Exception as e:
            self.on_event("error", f"Failed to cancel Tradovate orders for {firm_name}: {e}")

def _cancel_tradovate_orders_by_firm(self, firm_name):
    """Cancel all pending Tradovate orders for a specific firm connection."""
    tv_account = self.tradovate_accounts.get(firm_name)
    if not tv_account or not hasattr(tv_account, 'cancel_all_orders_api'):
        return
    try:

```

[illegible]

```
def __init__(self, mt5_api=None, tradovate_accounts=None, on_event=None, on_status_change=None)
```

Args: mt5_api: MT5API instance (from mt5_trading.py) — must be connected tradovate_accounts: dict of {firm_name: TradovateAccount} — connected instances on_event: callback(event_type: str, message: str) for UI notifications event_type: "info", "warn", "sl_detected", "tp_detected", "close_sent", "error" on_status_change: callback(acct_name: str, close_reason: str, phase_key: str) Called immediately when TP/SL detected so dashboard status can be updated. close_reason: "tp_detected" or "sl_detected" phase_key: e.g. "challenge_trade2", "funded_trade1" (from MT5 comment)

Why these Python concepts were used

- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **__init__** — The constructor. Runs after Python has allocated the instance; assigns starting attribute values to self.

Step by step (top-level body)

1. Assigns self.mt5_api = mt5_api.
2. Assigns self.tradovate_accounts = tradovate_accounts or {}.
3. Assigns self.on_event = on_event or (lambda t, m: print(f'[HEDGE {t.upper()}] {m}')).
4. Assigns self.on_status_change = on_status_change.
5. Assigns self._running = False.
6. Assigns self._mt5_thread = None.
7. Assigns self._tv_threads = {}.
8. Assigns self._mt5_positions = {}.
9. Assigns self._tv_positions = {}.
10. Assigns self._tv_account_info = {}.
11. Assigns self._closed_hedges = set().
12. Assigns self.stats = {'sl_protected': 0, 'tp_passed': 0, 'errors': 0, 'mt5_pol....

Code (copy-paste safe)

```
def __init__(self, mt5_api=None, tradovate_accounts=None, on_event=None, on_status_change=None):
    """
    Args:
        mt5_api: MT5API instance (from mt5_trading.py) — must be connected
        tradovate_accounts: dict of {firm_name: TradovateAccount} — connected instances
        on_event: callback(event_type: str, message: str) for UI notifications
            event_type: "info", "warn", "sl_detected", "tp_detected", "close_sent", "error"
        on_status_change: callback(acct_name: str, close_reason: str, phase_key: str)
            Called immediately when TP/SL detected so dashboard status can be updated.
            close_reason: "tp_detected" or "sl_detected"
            phase_key: e.g. "challenge_trade2", "funded_trade1" (from MT5 comment)
    """
    self.mt5_api = mt5_api
    self.tradovate_accounts = tradovate_accounts or {}
```



```

self.on_event = on_event or (lambda t, m: print(f"[HEDGE {t.upper()}] {m}"))
self.on_status_change = on_status_change

self._running = False
self._mt5_thread = None
self._tv_threads = {}          # {firm_name: polling_thread}

# Position snapshots for change detection
self._mt5_positions = {}      # {ticket: {symbol, type, volume, comment, sl, tp, profit, price_open}}
self._tv_positions = {}      # {firm_name: {contract_id: {netPos, netPrice, ...}}}

# Tradovate account info cache
self._tv_account_info = {}    # {firm_name: {id, name, env, token}}

# Track which hedges we've already closed (prevent re-triggers)
self._closed_hedges = set()   # set of "mt5_{ticket}" or "tv_{firm}_{acct_id}"

# Stats
self.stats = {
    "sl_protected": 0,
    "tp_passed": 0,
    "errors": 0,
    "mt5_polls": 0,
    "tv_events": 0,
}

```

Method: HedgeProtector.start

```
def start(self)

    Start monitoring both MT5 and Tradovate positions.
```

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **if** self._running: branches conditionally.
2. Assigns self._running = True.
3. Calls self.on_event(...) for its side effect.
4. **if** self.mt5_api: branches conditionally.
5. **for** (firm_name, tv_account) in self.tradovate_accounts.items(): iterates.
6. Calls self.on_event(...) for its side effect.

Code (copy-paste safe)

```

def start(self):
    """Start monitoring both MT5 and Tradovate positions."""
    if self._running:
        return
    self._running = True
    self.on_event("info", "Hedge Protector starting...")

    # Start MT5 position poller
    if self.mt5_api:

```

```

        self._mt5_thread = threading.Thread(target=self._mt5_poll_loop, daemon=True,
                                             name="HedgeProtector-MT5")
        self._mt5_thread.start()
        self.on_event("info", "MT5 position monitor started (500ms poll)")

    # Start Tradovate REST position pollers
    for firm_name, tv_account in self.tradovate_accounts.items():
        self._start_tradovate_rest(firm_name, tv_account)

    self.on_event("info",
                  f"Hedge Protector active – monitoring {len(self.tradovate_accounts)} Tradovate account(s) (RE

```

Method: HedgeProtector.stop

```
def stop(self)

    Stop all monitoring.
```

Step by step (top-level body)

1. Assigns `self._running = False`.
2. Calls `self.on_event(...)` for its side effect.
3. Calls `self._tv_threads.clear(...)` for its side effect.
4. Calls `self.on_event(...)` for its side effect.

Code (copy-paste safe)

```
def stop(self):
    """Stop all monitoring."""
    self._running = False
    self.on_event("info", "Hedge Protector stopping...")

    # Stop all Tradovate polling threads
    self._tv_threads.clear()
    self.on_event("info", "Hedge Protector stopped")
```

Method: HedgeProtector.is_running

```
@property
def is_running(self)
```

Why these Python concepts were used

- **@property** — Wrapped in **@property** so callers access the value with attribute syntax (`obj.x`) instead of a method call. Lets the implementation compute or validate the value lazily without changing the call site.

Step by step (top-level body)

1. **return** `self._running`.

Code (copy-paste safe)

```
def is_running(self):
    return self._running
```

Method: HedgeProtector._mt5_poll_loop

```
def _mt5_poll_loop(self)
```

Poll MT5 positions every 500ms, detect closures.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **while** self._running: loops until the condition is false.

Code (copy-paste safe)

```
def _mt5_poll_loop(self):
    """Poll MT5 positions every 500ms, detect closures."""
    while self._running:
        try:
            self._mt5_check_positions()
            self.stats["mt5_polls"] += 1
        except Exception as e:
            self.on_event("error", f"MT5 poll error: {e}")
            self.stats["errors"] += 1
        time.sleep(0.5)
```

Method: HedgeProtector._mt5_check_positions

```
def _mt5_check_positions(self)
```

Snapshot MT5 positions and detect any that disappeared (closed).

Step by step (top-level body)

1. **if** not mt5: branches conditionally.
2. Assigns positions = mt5.positions_get().
3. Assigns current = {}.
4. **if** positions: branches conditionally.
5. **for** (ticket, prev_pos) in self._mt5_positions.items(): iterates.
6. Assigns self._mt5_positions = current.

Code (copy-paste safe)

```
def _mt5_check_positions(self):
    """Snapshot MT5 positions and detect any that disappeared (closed)."""
    if not mt5:
        return

    positions = mt5.positions_get()
    current = {}
    if positions:
        for pos in positions:
            current[pos.ticket] = {
```

```

        "ticket": pos.ticket,
        "symbol": pos.symbol,
        "type": pos.type,          # 0=Buy, 1=Sell
        "volume": pos.volume,
        "comment": getattr(pos, 'comment', ''),
        "sl": pos.sl,
        "tp": pos.tp,
        "profit": pos.profit,
        "price_open": pos.price_open,
        "price_current": pos.price_current,
    }

    # Detect closed positions (were in previous snapshot, gone now)
    for ticket, prev_pos in self._mt5_positions.items():
        if ticket not in current:
            self._on_mt5_position_closed(prev_pos)

    self._mt5_positions = current

```

Method: HedgeProtector._on_mt5_position_closed

```
def _on_mt5_position_closed(self, pos)
```

Called when an MT5 position disappears (closed).

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns ticket = pos['ticket'].
2. Assigns comment = pos.get('comment', "").
3. Assigns symbol = pos['symbol'].
4. Assigns profit = pos['profit'].
5. **if** not comment or '_' not in comment: branches conditionally.
6. Assigns parts = comment.rsplit('_', 1).
7. **if** len(parts) < 2: branches conditionally.
8. Assigns tv_account_number = self._extract_tradovate_account(comment).
9. **if** not tv_account_number: branches conditionally.
10. Assigns phase_code = comment[len(tv_account_number) + 1:] if len(comment) > le....
11. Assigns hedge_key = f'mt5_{ticket}'.
12. **if** hedge_key in self._closed_hedges: branches conditionally.

13. Calls `self._closed_hedges.add(...)` for its side effect.
14. Assigns `close_reason = self._determine_mt5_close_reason(pos)`.
15. ... and 3 more statement(s) in the body.

Code (copy-paste safe)

```
def _on_mt5_position_closed(self, pos):
    """Called when an MT5 position disappears (closed)."""
    ticket = pos['ticket']
    comment = pos.get('comment', '')
    symbol = pos['symbol']
    profit = pos['profit']

    # Skip if no comment (not a hedge trade)
    if not comment or '_' not in comment:
        return

    # Parse the Tradovate account from comment
    # Format: "{TradovateAccountNumber}_{PhaseCode}"
    # e.g., "FNFTCHHARRISONOUKA85625_CH1"
    parts = comment.rsplit('_', 1)
    if len(parts) < 2:
        return

    # The account number is everything before the last underscore
    # But phase codes can have underscores too (e.g., FA_210126)
    # So we need to find the tradovate account by trying different splits
    tv_account_number = self._extract_tradovate_account(comment)
    if not tv_account_number:
        return

    phase_code = comment[len(tv_account_number) + 1:] if len(comment) > len(tv_account_number) else ''

    # Check for duplicate
    hedge_key = f"mt5_{ticket}"
    if hedge_key in self._closed_hedges:
        return
    self._closed_hedges.add(hedge_key)

    # Determine if SL or TP hit
    close_reason = self._determine_mt5_close_reason(pos)

    self.on_event(close_reason,
                  f"MT5 #{ticket} {symbol} closed - {close_reason.upper()} - "
                  f"P&L: ${profit:+.2f} - Comment: {comment}")

    if close_reason == "sl_detected":
        # SL HIT → Close the Tradovate hedge immediately
        self.stats["sl_protected"] += 1
        self._close_tradovate_hedge(tv_account_number, symbol, pos)
    elif close_reason == "tp_detected":
        # TP HIT → Let the hedge run
        self.stats["tp_passed"] += 1
        self.on_event("tp_detected",
                      f"TP hit on MT5 #{ticket} - letting Tradovate hedge run for double TP potential")
        # Cancel remaining Tradovate bracket orders (stop/limit)
        self._cancel_tradovate_orders(tv_account_number)

    # Fire status change callback with account + phase_key from comment
```

```

if self.on_status_change and phase_code:
    try:
        self.on_status_change(tv_account_number, close_reason, phase_code)
    except Exception as _sc_e:
        self.on_event("error", f"Status change callback failed: {_sc_e}")

```

Method: HedgeProtector._determine_mt5_close_reason

```
def _determine_mt5_close_reason(self, pos)
```

Determine if an MT5 position closed via SL or TP. Logic: - If profit < 0 → SL (or manual loss close) - If profit > 0 AND tp was set AND price reached tp → TP - If profit > 0 but no tp set → manual close (treat as neutral, don't close hedge) For hedging safety, we treat negative profit as SL.

Step by step (top-level body)

1. Assigns profit = pos.get('profit', 0).
2. Assigns sl = pos.get('sl', 0).
3. Assigns tp = pos.get('tp', 0).
4. Assigns price_current = pos.get('price_current', 0).
5. Assigns price_open = pos.get('price_open', 0).
6. Assigns pos_type = pos.get('type', 0).
7. if profit < 0: branches conditionally.
8. if profit >= 0 and tp > 0: branches conditionally.
9. if profit >= 0: branches conditionally.
10. return 'sl_detected'.

Code (copy-paste safe)

```

def _determine_mt5_close_reason(self, pos):
    """Determine if an MT5 position closed via SL or TP.

    Logic:
    - If profit < 0 → SL (or manual loss close)
    - If profit > 0 AND tp was set AND price reached tp → TP
    - If profit > 0 but no tp set → manual close (treat as neutral, don't close hedge)

    For hedging safety, we treat negative profit as SL.
    """
    profit = pos.get('profit', 0)
    sl = pos.get('sl', 0)
    tp = pos.get('tp', 0)
    price_current = pos.get('price_current', 0)
    price_open = pos.get('price_open', 0)
    pos_type = pos.get('type', 0)

    if profit < 0:
        return "sl_detected"

    if profit >= 0 and tp > 0:
        # Check if price was near TP
        if pos_type == 0: # Buy – TP is above
            if price_current >= tp * 0.999:

```

```

        return "tp_detected"
    else: # Sell — TP is below
        if price_current <= tp * 1.001:
            return "tp_detected"

    if profit >= 0:
        return "tp_detected"

    return "sl_detected"

```

Method: HedgeProtector._extract_tradovate_account

```
def _extract_tradovate_account(self, comment)
```

Extract Tradovate account number from MT5 comment. Tries to match against known connected Tradovate accounts. Comment format: "{AccountNumber}_{PhaseCode}"

Why these Python concepts were used

- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns known_accounts = set().
2. for (firm_name, info) in self._tv_account_info.items(): iterates.
3. for acct_name in known_accounts: iterates.
4. Imports re (lazy import inside the function).
5. Assigns m = re.match('^(.+?)_(CH\d|FD\d|DD\d|FA|EV)', comment).
6. if m: branches conditionally.
7. Assigns parts = comment.rsplit('_', 1).
8. return parts[0] if len(parts) == 2 else None.

Code (copy-paste safe)

```

def _extract_tradovate_account(self, comment):
    """Extract Tradovate account number from MT5 comment.

    Tries to match against known connected Tradovate accounts.
    Comment format: "{AccountNumber}_{PhaseCode}"
    """
    # Get all known Tradovate account names
    known_accounts = set()
    for firm_name, info in self._tv_account_info.items():
        acct_name = info.get('name', '')
        if acct_name:
            known_accounts.add(acct_name)

    # Try exact prefix match against known accounts
    for acct_name in known_accounts:
        if comment.startswith(acct_name):
            return acct_name

    # Fallback: split on known phase code patterns
    import re

```

```

# Match everything before _CH\d, _FD\d, _DD\d, _FA, _EV, etc.
m = re.match(r'^(.+?)_(CH\d|FD\d|DD\d|FA|EV)', comment)
if m:
    return m.group(1)

# Last resort: everything before the last _
parts = comment.rsplit('_', 1)
return parts[0] if len(parts) == 2 else None

```

Method: HedgeProtector._close_tradovate_hedge

```
def _close_tradovate_hedge(self, tv_account_number, mt5_symbol, mt5_pos)
```

Close the matching Tradovate position when MT5 SL is hit.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Calls `self.on_event(...)` for its side effect.
2. `for (firm_name, tv_account) in self.tradovate_accounts.items():` iterates.
3. Calls `self.on_event(...)` for its side effect.

Code (copy-paste safe)

```

def _close_tradovate_hedge(self, tv_account_number, mt5_symbol, mt5_pos):
    """Close the matching Tradovate position when MT5 SL is hit."""
    self.on_event("close_sent",
                  f"■■■ Closing Tradovate hedge for account {tv_account_number} (MT5 SL hit)")

    # Find which Tradovate connection has this account
    for firm_name, tv_account in self.tradovate_accounts.items():
        try:
            # Check if this connection owns the account
            acct_info = self._tv_account_info.get(firm_name, {})
            acct_name = acct_info.get('name', '')
            acct_id = acct_info.get('id')

            if tv_account_number in acct_name or acct_name in tv_account_number:
                # Found the matching Tradovate connection → liquidate
                self.on_event("close_sent",
                              f"■■■ Liquidating {firm_name} account {acct_name} via API")
                result = tv_account.liquidate_position_api(account_id=acct_id)
                if result:
                    self.on_event("close_sent",
                                  f"■ Tradovate hedge closed for {acct_name}")
            else:
                self.on_event("error",
                              f"■ Failed to liquidate Tradovate position on {acct_name}")
            # Cancel remaining bracket orders
            self._cancel_tradovate_orders_by_firm(firm_name)
        return

```



```

        # Also check all accounts under this connection
        accounts = tv_account._api_fetch("/account/list")
        if accounts:
            for acct in accounts:
                name = acct.get('name', '')
                if tv_account_number in name or name in tv_account_number:
                    aid = acct['id']
                    self.on_event("close_sent",
                                f"■■ Liquidating {firm_name} account {name} (id={aid}) via API")
                    result = tv_account.liquidate_position_api(account_id=aid)
                    if result:
                        self.on_event("close_sent",
                                    f"■■ Tradovate hedge closed for {name}")
                    else:
                        self.on_event("error",
                                    f"■■ Failed to liquidate Tradovate position on {name}")
                # Cancel remaining bracket orders
                self._cancel_tradovate_orders_by_firm(firm_name)
            return

    except Exception as e:
        self.on_event("error", f"Error closing Tradovate hedge on {firm_name}: {e}")

    self.on_event("warn", f"■■ No matching Tradovate account found for: {tv_account_number}")

```

Method: HedgeProtector._start_tradovate_rest

```
def _start_tradovate_rest(self, firm_name, tv_account)
```

Start REST position polling for a Tradovate account.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.
2. Assigns `t = threading.Thread(target=self._tv_rest_poll_loop, args=(fi....`
3. Calls `t.start(...)` for its side effect.
4. Assigns `self._tv_threads[firm_name] = t`.
5. Calls `self.on_event(...)` for its side effect.

Code (copy-paste safe)

```

def _start_tradovate_rest(self, firm_name, tv_account):
    """Start REST position polling for a Tradovate account."""
    # Cache account info on startup
    try:
        accounts = tv_account._api_fetch("/account/list")
        if accounts and len(accounts) > 0:
            acct = accounts[0]

```

```

        self._tv_account_info[firm_name] = {
            'id': acct['id'],
            'name': acct.get('name', '?'),
        }
        self.on_event("info",
            f"Tradovate {firm_name}: account {acct.get('name', '?')} (id={acct['id']})")
        # Cache all accounts for multi-account logins
        for a in accounts:
            key = f"{firm_name}_{a['id']}"
            self._tv_account_info[key] = {
                'id': a['id'],
                'name': a.get('name', '?'),
            }
    except Exception as e:
        self.on_event("warn", f"Could not fetch account list for {firm_name}: {e}")

    t = threading.Thread(target=self._tv_rest_poll_loop, args=(firm_name, tv_account),
        daemon=True, name=f"HedgeProtector-TV-REST-{firm_name}")
    t.start()
    self._tv_threads[firm_name] = t
    self.on_event("info", f"Tradovate REST poller started for {firm_name} (5s interval)")

```

Method: HedgeProtector._on_tradovate_position_closed

```
def _on_tradovate_position_closed(self, firm_name, tv_account, position_entity, prev_net_pos)
```

Called when a Tradovate position goes to netPos=0 (closed).

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns `acct_info = self._tv_account_info.get(firm_name, {})`.
2. Assigns `acct_name = acct_info.get('name', '?')`.
3. Assigns `acct_id = acct_info.get('id', 0)`.
4. Assigns `contract_id = position_entity.get('contractId', 0)`.
5. Assigns `hedge_key = f'tv_{firm_name}_{contract_id}'`.
6. **if** `hedge_key` in `self._closed_hedges`: branches conditionally.
7. Calls `self._closed_hedges.add(...)` for its side effect.
8. Assigns `bought_val = position_entity.get('boughtValue', 0)`.
9. Assigns `sold_val = position_entity.get('soldValue', 0)`.
10. Assigns `realized_pnl = sold_val - bought_val`.

11. Assigns `close_reason = 'sl_detected'` if `realized_pnl < 0` else `'tp_detected'`.
12. Calls `self.on_event(...)` for its side effect.
13. **if** `close_reason == 'sl_detected'`: branches conditionally (with an **else/elif** arm).
14. Calls `self._cancel_tradovate_orders_by_firm(...)` for its side effect.
15. ... and 1 more statement(s) in the body.

Code (copy-paste safe)

```
def _on_tradovate_position_closed(self, firm_name, tv_account, position_entity, prev_net_pos):
    """Called when a Tradovate position goes to netPos=0 (closed)."""
    acct_info = self._tv_account_info.get(firm_name, {})
    acct_name = acct_info.get('name', '?')
    acct_id = acct_info.get('id', 0)
    contract_id = position_entity.get('contractId', 0)

    # Check for duplicate
    hedge_key = f"tv_{firm_name}_{contract_id}"
    if hedge_key in self._closed_hedges:
        return
    self._closed_hedges.add(hedge_key)

    # Get P&L info from the position entity
    bought_val = position_entity.get('boughtValue', 0)
    sold_val = position_entity.get('soldValue', 0)
    realized_pnl = sold_val - bought_val # Rough P&L estimate

    # Determine SL or TP: if prev position was closed at a loss → SL
    # For hedging: Tradovate hedge is OPPOSITE of MT5
    # If Tradovate lost money → that's the Tradovate SL → close the MT5 side
    close_reason = "sl_detected" if realized_pnl < 0 else "tp_detected"

    self.on_event(close_reason,
                  f"Tradovate {firm_name} ({acct_name}) position closed – {close_reason.upper()} – "
                  f"Contract: {contract_id}, prev_netPos: {prev_net_pos}")

    if close_reason == "sl_detected":
        self.stats["sl_protected"] += 1
        self._close_mt5_hedge(acct_name, contract_id)
    else:
        self.stats["tp_passed"] += 1
        self.on_event("tp_detected",
                      f"TP hit on Tradovate {acct_name} – letting MT5 hedge run for double TP potential")
    # Cancel remaining Tradovate bracket orders (stop/limit) after any close
    self._cancel_tradovate_orders_by_firm(firm_name)

    # Fire status change callback – extract phase_key from matching MT5 comment
    if self.on_status_change:
        phase_key = self._get_phase_key_from_mt5(acct_name)
        if phase_key:
            try:
                self.on_status_change(acct_name, close_reason, phase_key)
            except Exception as _sc_e:
                self.on_event("error", f"Status change callback failed: {_sc_e}")
```

Method: HedgeProtector._close_mt5_hedge

```
def _close_mt5_hedge(self, tv_account_name, tv_contract_id)
```

Close the matching MT5 position when Tradovate SL is hit.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **if** not self.mt5_api or not mt5: branches conditionally.
2. Calls self.on_event(...) for its side effect.
3. Assigns positions = mt5.positions_get().
4. **if** not positions: branches conditionally.
5. Assigns closed_count = 0.
6. **for** pos in positions: iterates.
7. **if** closed_count == 0: branches conditionally (with an **else/elif** arm).

Code (copy-paste safe)

```
def _close_mt5_hedge(self, tv_account_name, tv_contract_id):
    """Close the matching MT5 position when Tradovate SL is hit."""
    if not self.mt5_api or not mt5:
        self.on_event("error", "Cannot close MT5 hedge – MT5 not connected")
        return

    self.on_event("close_sent",
                  f"■■■ Closing MT5 hedge for Tradovate account {tv_account_name} (Tradovate SL hit)")

    # Find MT5 positions whose comment contains this Tradovate account
    positions = mt5.positions_get()
    if not positions:
        self.on_event("warn", "No open MT5 positions found")
        return

    closed_count = 0
    for pos in positions:
        comment = getattr(pos, 'comment', '')
        if not comment:
            continue
        # Check if comment references this Tradovate account
        if tv_account_name in comment:
            self.on_event("close_sent",
                          f"■■■ Closing MT5 #{pos.ticket} {pos.symbol} (comment: {comment})")
            try:
                success = self.mt5_api.close_trade(pos.ticket)
                if success:
                    self.on_event("close_sent",
                                  f"■ MT5 #{pos.ticket} closed successfully")
                    closed_count += 1
                else:
                    self.on_event("error",
                                  f"■ Failed to close MT5 #{pos.ticket}")
```

```

        except Exception as e:
            self.on_event("error", f"Error closing MT5 #{pos.ticket}: {e}")

    if closed_count == 0:
        self.on_event("warn",
            f"■ No matching MT5 positions found for Tradovate account {tv_account_name}")
    else:
        self.on_event("close_sent",
            f"■■ Closed {closed_count} MT5 position(s) for {tv_account_name}")

```

Method: HedgeProtector._get_phase_key_from_mt5

```
def _get_phase_key_from_mt5(self, tv_account_name)
```

Extract phase_key from the MT5 position comment that matches this Tradovate account. Checks both open positions and the last snapshot for recently closed ones. Comment format: "{AccountNumber}_{phase_key}"

Step by step (top-level body)

1. if not mt5: branches conditionally.
2. Assigns positions = mt5.positions_get().
3. if positions: branches conditionally.
4. for (ticket, pos_data) in self._mt5_positions.items(): iterates.
5. return None.

Code (copy-paste safe)

```

def _get_phase_key_from_mt5(self, tv_account_name):
    """Extract phase_key from the MT5 position comment that matches this Tradovate account.

    Checks both open positions and the last snapshot for recently closed ones.
    Comment format: "{AccountNumber}_{phase_key}"
    """
    if not mt5:
        return None
    # Check open positions first
    positions = mt5.positions_get()
    if positions:
        for pos in positions:
            comment = getattr(pos, 'comment', '')
            if comment and tv_account_name in comment:
                suffix = comment[len(tv_account_name):]
                if suffix.startswith('_'):
                    return suffix[1:] # strip leading underscore
    # Check cached snapshot (position may have just closed)
    for ticket, pos_data in self._mt5_positions.items():
        comment = pos_data.get('comment', '')
        if comment and tv_account_name in comment:
            suffix = comment[len(tv_account_name):]
            if suffix.startswith('_'):
                return suffix[1:]
    return None

```

Method: HedgeProtector._cancel_tradovate_orders

```
def _cancel_tradovate_orders(self, tv_account_number)
```

Cancel all pending Tradovate orders for the given account number.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **for** (firm_name, tv_account) in self.tradovate_accounts.items(): iterates.

Code (copy-paste safe)

```
def _cancel_tradovate_orders(self, tv_account_number):
    """Cancel all pending Tradovate orders for the given account number."""
    for firm_name, tv_account in self.tradovate_accounts.items():
        try:
            acct_info = self._tv_account_info.get(firm_name, {})
            acct_name = acct_info.get('name', '')
            acct_id = acct_info.get('id')
            if tv_account_number in acct_name or acct_name in tv_account_number:
                if hasattr(tv_account, 'cancel_all_orders_api'):
                    cancelled = tv_account.cancel_all_orders_api(account_id=acct_id)
                    if cancelled > 0:
                        self.on_event("info",
                                      f"■ Cancelled {cancelled} Tradovate order(s) for {acct_name}")
                return
            # Check all accounts under this connection
            accounts = tv_account._api_fetch("/account/list")
            if accounts:
                for acct in accounts:
                    name = acct.get('name', '')
                    if tv_account_number in name or name in tv_account_number:
                        if hasattr(tv_account, 'cancel_all_orders_api'):
                            cancelled = tv_account.cancel_all_orders_api(account_id=acct['id'])
                            if cancelled > 0:
                                self.on_event("info",
                                              f"■ Cancelled {cancelled} Tradovate order(s) for {name}")
                return
        except Exception as e:
            self.on_event("error", f"Failed to cancel Tradovate orders for {firm_name}: {e}")
```

Method: HedgeProtector._cancel_tradovate_orders_by_firm

```
def _cancel_tradovate_orders_by_firm(self, firm_name)
```

Cancel all pending Tradovate orders for a specific firm connection.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `tv_account = self.tradovate_accounts.get(firm_name)`.
2. **if** not `tv_account` or not `hasattr(tv_account, 'cancel_all_orders_api')`: branches conditionally.
3. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def _cancel_tradovate_orders_by_firm(self, firm_name):
    """Cancel all pending Tradovate orders for a specific firm connection."""
    tv_account = self.tradovate_accounts.get(firm_name)
    if not tv_account or not hasattr(tv_account, 'cancel_all_orders_api'):
        return
    try:
        acct_info = self._tv_account_info.get(firm_name, {})
        acct_id = acct_info.get('id')
        cancelled = tv_account.cancel_all_orders_api(account_id=acct_id)
        if cancelled > 0:
            self.on_event("info", f"■ Cancelled {cancelled} Tradovate order(s) for {firm_name}")
    except Exception as e:
        self.on_event("error", f"Failed to cancel Tradovate orders for {firm_name}: {e}")
```

Method: HedgeProtector._tv_rest_poll_loop

```
def _tv_rest_poll_loop(self, firm_name, tv_account)

    Poll Tradovate positions via REST API every 5 seconds.
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `prev_positions = {}`.
2. **while** `self._running`: loops until the condition is false.

Code (copy-paste safe)

```
def _tv_rest_poll_loop(self, firm_name, tv_account):
    """Poll Tradovate positions via REST API every 5 seconds."""
    prev_positions = {}

    while self._running:
        try:
            positions = tv_account.get_positions_api()
            current = {}
            for p in positions:
                key = (p.get('accountId', 0), p.get('contractId', 0))
                current[key] = p.get('netPos', 0)
```

```

        # Detect positions that went to 0
        for key, prev_net in prev_positions.items():
            cur_net = current.get(key, 0)
            if prev_net != 0 and cur_net == 0:
                # Position closed
                entity = {'accountId': key[0], 'contractId': key[1], 'netPos': 0,
                          'boughtValue': 0, 'soldValue': 0}
                self._on_tradovate_position_closed(firm_name, tv_account, entity, prev_net)

        prev_positions = current
        self.stats["tv_events"] += 1

    except Exception as e:
        self.on_event("error", f"Tradovate REST poll error ({firm_name}): {e}")
        self.stats["errors"] += 1

    time.sleep(5)

```

Method: HedgeProtector.get_status

```
def get_status(self)
```

Return current status for UI display.

Step by step (top-level body)

1. Assigns `mt5_count = len(self._mt5_positions)`.
2. Assigns `tv_firms = len(self._tv_threads)`.
3. **return** `{'running': self._running, 'mt5_positions': mt5_count, 'tv_connecti....`

Code (copy-paste safe)

```

def get_status(self):
    """Return current status for UI display."""
    mt5_count = len(self._mt5_positions)
    tv_firms = len(self._tv_threads)
    return {
        "running": self._running,
        "mt5_positions": mt5_count,
        "tv_connections": tv_firms,
        "sl_protected": self.stats["sl_protected"],
        "tp_passed": self.stats["tp_passed"],
        "errors": self.stats["errors"],
    }

```

Method: HedgeProtector.clear_closed_cache

```
def clear_closed_cache(self)
```

Clear the closed hedge cache (for re-monitoring after restart).

Step by step (top-level body)

1. Calls `self._closed_hedges.clear(...)` for its side effect.

Code (copy-paste safe)

```

def clear_closed_cache(self):
    """Clear the closed hedge cache (for re-monitoring after restart)."""

```



```
self._closed_hedges.clear()
```

Checkpoint — what works now. The trading engine is complete. With MT5 running and a logged-in account, `mt5_trading.MT5Trader().place_order(...)` sends real orders; `trade_limit_manager` blocks ones that would breach daily limits; `hedge_protector` can open offsetting positions on a hedge account. Test on a demo account before pointing it at anything live.

Chapter 8 — Phase 6 — Prop-firm dashboards

Funded accounts live behind prop-firm dashboards (Topstep, FundedNext, Tradovate, TradeOpss) that have no official API. We use Selenium and the Chrome DevTools Protocol to log in, read account state, and post it to our own database. We build a base scraper class first, then the firm-specific subclasses, then a manager that coordinates them.

Files we build in this phase, in order:

- `trader_companion/prop_firm_scrapers.py`
- `trader_companion/fundednext.py`
- `trader_companion/topstepx.py`
- `trader_companion/tradovate.py`
- `trader_companion/prop_firm_manager.py`

Python concepts you'll meet in this phase

- Dunder methods (5) · Classes and objects (5) · f-strings (5) · Comprehensions (5) · Exceptions (4) · Context managers (with-statements) (4) · Decorators (2) · Module-level state (1)

Each is explained in Chapter 2 (the primer). The number in parentheses is how many of this phase's files use that concept.

Step 8.1 — Build `trader_companion/prop_firm_scrapers.py`

What to do. Create the file `trader_companion/prop_firm_scrapers.py` in your project root and paste in the code shown in this step. The file is **1893** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from `requirements.txt`.

What this file is for. Browser-automation scrapers (Selenium / Chrome DevTools Protocol) that pull account state out of prop-firm dashboards that have no official API.

`trader_companion/prop_firm_scrapers.py`

1893 loc · 6 classes · 5 functions · 10 imports

Browser-automation scrapers (Selenium / Chrome DevTools Protocol) that pull account state out of prop-firm dashboards that have no official API. It defines **6** classes and **5** module-level functions, across **1,893** lines. Module docstring: *Prop Firm Dashboard Scrapers — CDP-based (no Selenium required)*

Module docstring

Prop Firm Dashboard Scrapers — CDP-based (no Selenium required) Scrapes account data from Tradeify, Lucid Trading, TopStep, and MFFU dashboards via Chrome DevTools Protocol connecting to an already-open Chrome debug port.

Each class follows the standard interface: - login() → attach to existing Chrome tab, verify logged in - get_account_stats() → basic account info for GUI display - get_billing_history() → subscription/purchase history - get_payouts() → payout history - get_all_accounts() → all accounts - is_connected() → bool - close() / disconnect()

Requires Chrome running with: --remote-debugging-port=9222 --remote-allow-origins=*
--user-data-dir="<path>"

Python concepts demonstrated in this module

- Classes and objects
- Comprehensions
- Context managers (with-statements)
- Dunder methods
- Exceptions
- f-strings
- Module-level state
- Inheritance

Imports — what each one is for

```
import json
```

Why imported. JSON encoding and decoding — the standard wire format for config files, API payloads, and inter-process messages.

Used in this file. json.JSONDecodeError, json.dumps, json.loads

Example. json.loads(response.text) # parse a JSON string to dict

Other common scenarios.

- json.dumps(obj, indent=2) for pretty-printing
- json.dump(obj, file) / json.load(file) for file I/O
- default=str handles non-serialisable types like datetime
- object_hook= customises decoding (e.g., to dataclass instances)

```
import logging
```

Why imported. Structured, levelled logging. Replaces print() with filterable, formattable output that can route to files, stderr, syslog, or HTTP endpoints.

Used in this file. logging.INFO, logging.basicConfig, logging.getLogger

Example. logging.getLogger(__name__).info('connected to %s', host)

Other common scenarios.

- .debug() / .info() / .warning() / .error() / .exception()

- `logger.exception()` captures the current traceback automatically
- `logging.basicConfig(level=logging.INFO)` for quick setup
- Handlers and Formatters route logs to files / streams / syslog

`import os`

Why imported. Operating-system interface. Path manipulation, environment variables, working-directory operations, exit codes, and thin wrappers around POSIX/Windows syscalls.

Used in this file. `os.environ`, `os.environ.get`, `os.path`, `os.path.isfile`, `os.path.join`

Example. `os.path.join(root, 'subdir', 'file.txt')` # joins with the OS-correct separator

Other common scenarios.

- `os.environ.get('API_KEY')` to read environment variables
- `os.makedirs(path, exist_ok=True)` to create nested directories
- `os.listdir(path)` to list a directory's contents
- `os.remove(path)` / `os.rename(src, dst)` to delete or move a file
- `os.getcwd()` / `os.chdir(path)` to inspect or change the working dir

`import re`

Why imported. Regular expressions. Pattern matching, search, replace, and text extraction.

Used in this file. *No direct attribute access detected — likely imported for side effects, re-export, or string references.*

Example. `re.search(r'\d{3}-\d{4}', text)` # returns a Match or None

Other common scenarios.

- `re.findall(pattern, text)` for every non-overlapping match
- `re.sub(pattern, repl, text)` to replace occurrences
- `re.compile(...)` to reuse a pattern (faster in a loop)
- Named groups: `r'(?P<year>\d{4})'` lets you do `m.group('year')`

`import shutil`

Why imported. High-level filesystem operations: copying trees, moving files across filesystems, deleting directories, locating executables on PATH.

Used in this file. `shutil.which`

Example. `shutil.copytree(src, dst)` # recursively copies a tree

Other common scenarios.

- `shutil.move(src, dst)` moves or renames
- `shutil.rmtree(path)` deletes a directory and all its contents
- `shutil.which('git')` finds an executable on PATH
- `shutil.disk_usage(path)` returns total / used / free bytes

```
import subprocess
```

Why imported. Run external programs. The standard way to spawn a child process, capture its output, and wait for its exit code.

Used in this file. `subprocess.DEVNULL`, `subprocess.Popen`

Example. `subprocess.run(['git', 'rev-parse', 'HEAD'], capture_output=True, text=True, check=True)`

Other common scenarios.

- `Popen(...)` for streaming I/O while the process is alive
- `check_output(cmd)` returns stdout as text and raises on failure
- `DEVNULL` / PIPE redirect stdin/stdout/stderr
- `shell=True` interprets the string with the system shell (security risk)

```
import threading
```

Why imported. Threads and synchronisation primitives. Used when work is I/O-bound and the GIL is not a bottleneck (the GIL prevents speed-up on CPU-bound work).

Used in this file. `threading.Lock`, `threading.RLock`

Example. `Thread(target=worker, args=(q,), daemon=True).start()`

Other common scenarios.

- `Lock` / `RLock` to guard shared state
- `Event` for one-shot signals between threads
- `Queue` (from `queue`) for producer/consumer patterns
- `threading.local()` for thread-local storage

```
import time
```

Why imported. Timestamps and sleep. Lower-level than `datetime` — works in Unix-epoch seconds.

Used in this file. `time.sleep`, `time.time`

Example. `time.sleep(1.5)` # pause this thread for 1.5 seconds

Other common scenarios.

- `time.time()` returns seconds since the epoch
- `time.monotonic()` for measuring elapsed time (never goes backward)
- `time.perf_counter()` for high-precision benchmarking
- `time.strftime` / `time.strptime` mirror the `datetime` versions

```
import urllib.request
```

Why imported. Stdlib URL utilities — parsing, encoding, and a basic HTTP client. Mostly useful for the parsing helpers; for outbound HTTP, requests is more pleasant.

Used in this file. *No direct attribute access detected — likely imported for side effects, re-export, or string references.*

Example. `urllib.parse.urlencode({'q': 'hello world'})`

Other common scenarios.

- `urllib.parse.urlparse(url)` splits scheme/host/path/query
- `urllib.parse.quote(s)` percent-encodes a path segment
- `urllib.request.urlopen(url)` is the basic HTTP client

```
import urllib.parse
```

Why imported. Stdlib URL utilities — parsing, encoding, and a basic HTTP client. Mostly useful for the parsing helpers; for outbound HTTP, requests is more pleasant.

Used in this file. `urllib.parse`, `urllib.parse.quote`, `urllib.request`, `urllib.request.Request`, `urllib.request.urlopen`

Example. `urllib.parse.urlencode({'q': 'hello world'})`

Other common scenarios.

- `urllib.parse.urlparse(url)` splits scheme/host/path/query
- `urllib.parse.quote(s)` percent-encodes a path segment
- `urllib.request.urlopen(url)` is the basic HTTP client

Module constants

Each line below is followed by a plain-English note explaining what it does and why it is written that way.

```
CDP_DEBUG_PORT = 9222
```

Numeric literal — a tunable parameter compiled into the source.

```
CDP_USER_DATA_DIR = os.path.join(os.environ.get('LOCALAPPDATA', ''), 'Google', 'Chrome', 'User Data CDP')
```

Module-level constant — bound once at import time and referenced from the functions and classes below.

Classes

```
class CDPBase
```

Base class for CDP-based prop firm scrapers. Connects to a Chrome tab via WebSocket and executes JS fetch() calls from the browser context to use the site's session cookies/tokens.

```
DOMAIN = ''
FIRM_NAME = ''
_msg_id = 0
```

Code (copy-paste safe)

```
class CDPBase:
    """
    Base class for CDP-based prop firm scrapers.
    Connects to a Chrome tab via WebSocket and executes JS fetch() calls
    from the browser context to use the site's session cookies/tokens.
    """

    DOMAIN = ""          # Override in subclasses: e.g. "app-f.tradeify.co"
    FIRM_NAME = ""        # Override: e.g. "Tradeify"
    _msg_id = 0
```

```

def __init__(self, debug_port=9222, pair_id="default"):
    self.debug_port = debug_port
    self.pair_id = pair_id
    self.ws = None
    self.tab_url = None
    self.logged_in = False
    self._login_timestamp = None
    self._cached_stats = None
    self._stats_last_fetch = 0
    self.lock = threading.RLock()
    self.logger = logging.getLogger(f"{self.FIRM_NAME}_{pair_id}")

# ■■ CDP Connection ■■

def _find_tab(self):
    """Find the browser tab matching this firm's domain."""
    try:
        data = urllib.request.urlopen(
            f'http://127.0.0.1:{self.debug_port}/json', timeout=5
        ).read()
        tabs = json.loads(data)
        for t in tabs:
            if t.get('type') == 'page' and self.DOMAIN in t.get('url', ''):
                return t
    except Exception as e:
        self.logger.error(f"[CDP] Failed to list tabs: {e}")
    return None

def _connect_ws(self, tab):
    """Connect WebSocket to a specific tab."""
    ws_url = tab.get('webSocketDebuggerUrl')
    if not ws_url:
        raise ConnectionError(f"No webSocketDebuggerUrl for tab: {tab.get('url')}")
    # Activate the target first – Chrome requires this for CDP to respond
    target_id = tab.get('id', '')
    if target_id:
        try:
            urllib.request.urlopen(
                f'http://127.0.0.1:{self.debug_port}/json/activate/{target_id}',
                timeout=5
            )
        except Exception:
            pass # Non-critical, continue anyway
    self.ws = websocket.create_connection(ws_url, timeout=30)
    self.tab_url = tab.get('url')
    self.logger.info(f"[CDP] Connected to {self.tab_url}")

def _send(self, method, params=None, timeout=30):
    """Send a CDP command and return the result."""
    CDPBase._msg_id += 1
    msg_id = CDPBase._msg_id
    msg = {'id': msg_id, 'method': method}
    if params:
        msg['params'] = params
    self.ws.send(json.dumps(msg))
    # Read responses until we get our ID back, skipping CDP events
    deadline = time.time() + timeout
    old_timeout = self.ws.gettimeout()
    try:
        while time.time() < deadline:

```

```

        self.ws.setTimeout(max(0.1, deadline - time.time()))
        try:
            resp = json.loads(self.ws.recv())
        except websocket.WebSocketTimeoutException:
            continue
        if resp.get('id') == msg_id:
            return resp
        # Skip CDP events (no 'id' field)
    finally:
        self.ws.setTimeout(old_timeout)
    raise TimeoutError(f"CDP response timeout for {method}")

def _js(self, expression, await_promise=False, timeout=None):
    """Execute JavaScript in the page and return the result value."""
    params = {'expression': expression, 'returnByValue': True}
    if await_promise:
        params['awaitPromise'] = True
    resp = self._send('Runtime.evaluate', params, timeout=timeout or (30 if await_promise else 10))
    result = resp.get('result', {}).get('result', {})
    if result.get('subtype') == 'error':
        desc = result.get('description', 'JS error')
        raise RuntimeError(f"JS error: {desc}")
    return result.get('value')

def _fetch_json(self, url, method='GET', body=None, extra_headers=None):
    """Execute a fetch() call from the browser context and parse JSON response."""
    headers = {'Content-Type': 'application/json'}
    if extra_headers:
        headers.update(extra_headers)

    if body and method == 'POST':
        js = f"""
            (async () => {{
                const r = await fetch({json.dumps(url)}, {{
                    method: 'POST',
                    credentials: 'include',
                    headers: {json.dumps(headers)},
                    body: JSON.stringify({json.dumps(body)})
                }});
                const text = await r.text();
                return JSON.stringify({{status: r.status, body: text}});
            }})()
        """
    else:
        fetch_opts = f"{{credentials: 'include', headers: {json.dumps(headers)}}}"
        js = f"""
            (async () => {{
                const r = await fetch({json.dumps(url)}, {fetch_opts});
                const text = await r.text();
                return JSON.stringify({{status: r.status, body: text}});
            }})()
        """

    raw = self._js(js, await_promise=True)
    if not raw:
        return None
    result = json.loads(raw)
    status = result.get('status', 0)
    if status != 200:
        self.logger.warning(f"[FETCH] {method} {url} → HTTP {status}")

```



```

        return None
    try:
        return json.loads(result['body'])
    except (json.JSONDecodeError, KeyError):
        return result.get('body')

def _fetch_json_bearer(self, url, token, method='GET', body=None):
    """Fetch with Bearer token auth."""
    return self._fetch_json(url, method=method, body=body,
                            extra_headers={'Authorization': f'Bearer {token}'})

# ■■■ Standard Interface ■■■

def login(self, open_url=None):
    """
    Attach to an existing Chrome tab for this firm.
    If open_url is provided and no tab is found, launch/reuse Chrome debug
    and open the URL, then wait for the tab to appear.
    """
    if not WS_AVAILABLE:
        raise ImportError("websocket-client package required: pip install websocket-client")

    tab = self._find_tab()

    # If no tab found and we have a URL, launch Chrome and open the tab
    if not tab and open_url:
        ensure_chrome_debug(url=open_url, port=self.debug_port)
        # Wait for the tab to appear (user may need to finish loading)
        for _ in range(30):
            time.sleep(1)
            tab = self._find_tab()
            if tab:
                break

    if not tab:
        raise ConnectionError(
            f"No {self.FIRM_NAME} tab found on port {self.debug_port}. "
            f"Open {self.DOMAIN} in Chrome with --remote-debugging-port={self.debug_port}")
    self._connect_ws(tab)
    self.logged_in = True
    self._login_timestamp = time.time()
    self.logger.info(f"[LOGIN] Attached to {self.FIRM_NAME} tab")
    return True

def is_connected(self):
    """Check if the WebSocket connection is alive."""
    try:
        if not self.ws or not self.ws.connected:
            return False
        # Quick ping with short timeout
        self._send('Runtime.evaluate',
                   {'expression': '1+1', 'returnByValue': True},
                   timeout=5)
        return True
    except Exception:
        self.logged_in = False
        return False

def disconnect(self):
    """Close the WebSocket (does NOT close the Chrome tab)."""

```

```

        self.logged_in = False
    if self.ws:
        try:
            self.ws.close()
        except Exception:
            pass
        self.ws = None

    def close(self):
        """Alias for disconnect."""
        self.disconnect()

    def get_account_stats(self):
        """Override in subclasses."""
        return {}

    def get_billing_history(self):
        """Override in subclasses."""
        return []

    def get_all_accounts(self):
        """Override in subclasses."""
        return []

    def get_payouts(self):
        """Override in subclasses."""
        return []

```

Method: CDPBase.__init__

```
def __init__(self, debug_port=9222, pair_id='default')
```

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **__init__** — The constructor. Runs after Python has allocated the instance; assigns starting attribute values to self.

Step by step (top-level body)

1. Assigns `self.debug_port = debug_port`.
2. Assigns `self.pair_id = pair_id`.
3. Assigns `self.ws = None`.
4. Assigns `self.tab_url = None`.
5. Assigns `self.logged_in = False`.
6. Assigns `self._login_timestamp = None`.
7. Assigns `self._cached_stats = None`.
8. Assigns `self._stats_last_fetch = 0`.
9. Assigns `self.lock = threading.RLock()`.
10. Assigns `self.logger = logging.getLogger(f'{self.FIRM_NAME}_{pair_id}')`.

Code (copy-paste safe)

```
def __init__(self, debug_port=9222, pair_id="default"):
    self.debug_port = debug_port
    self.pair_id = pair_id
    self.ws = None
    self.tab_url = None
    self.logged_in = False
    self._login_timestamp = None
    self._cached_stats = None
    self._stats_last_fetch = 0
    self.lock = threading.RLock()
    self.logger = logging.getLogger(f"{self.FIRM_NAME}_{pair_id}")
```

Method: CDPBase._find_tab

```
def _find_tab(self)
```

Find the browser tab matching this firm's domain.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.
2. **return None**.

Code (copy-paste safe)

```
def _find_tab(self):
    """Find the browser tab matching this firm's domain."""
    try:
        data = urllib.request.urlopen(
            f'http://127.0.0.1:{self.debug_port}/json', timeout=5
        ).read()
        tabs = json.loads(data)
        for t in tabs:
            if t.get('type') == 'page' and self.DOMAIN in t.get('url', ''):
                return t
    except Exception as e:
        self.logger.error(f"[CDP] Failed to list tabs: {e}")
    return None
```

Method: CDPBase._connect_ws

```
def _connect_ws(self, tab)
```

Connect WebSocket to a specific tab.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't

have to handle it.

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.

Step by step (top-level body)

1. Assigns `ws_url = tab.get('webSocketDebuggerUrl')`.
2. `if not ws_url`: branches conditionally.
3. Assigns `target_id = tab.get('id', '')`.
4. `if target_id`: branches conditionally.
5. Assigns `self.ws = websocket.create_connection(ws_url, timeout=30)`.
6. Assigns `self.tab_url = tab.get('url')`.
7. Calls `self.logger.info(...)` for its side effect.

Code (copy-paste safe)

```
def _connect_ws(self, tab):
    """Connect WebSocket to a specific tab."""
    ws_url = tab.get('webSocketDebuggerUrl')
    if not ws_url:
        raise ConnectionError(f"No webSocketDebuggerUrl for tab: {tab.get('url')}")
    # Activate the target first – Chrome requires this for CDP to respond
    target_id = tab.get('id', '')
    if target_id:
        try:
            urllib.request.urlopen(
                f'http://127.0.0.1:{self.debug_port}/json/activate/{target_id}',
                timeout=5
            )
        except Exception:
            pass # Non-critical, continue anyway
    self.ws = websocket.create_connection(ws_url, timeout=30)
    self.tab_url = tab.get('url')
    self.logger.info(f"[CDP] Connected to {self.tab_url}")
```

Method: CDPBase._send

```
def _send(self, method, params=None, timeout=30)
```

Send a CDP command and return the result.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.

Step by step (top-level body)

1. Updates `CDPBase._msg_id` in place (Add).
2. Assigns `msg_id = CDPBase._msg_id`.
3. Assigns `msg = {'id': msg_id, 'method': method}`.
4. **if** `params`: branches conditionally.
5. Calls `self.ws.send(...)` for its side effect.
6. Assigns `deadline = time.time() + timeout`.
7. Assigns `old_timeout = self.ws.gettimeout()`.
8. **try** block with 0 **except** clauses, plus a **finally**.
9. **raise** `TimeoutError(f'CDP response timeout for {method}')`.

Code (copy-paste safe)

```
def _send(self, method, params=None, timeout=30):
    """Send a CDP command and return the result."""
    CDPBase._msg_id += 1
    msg_id = CDPBase._msg_id
    msg = {'id': msg_id, 'method': method}
    if params:
        msg['params'] = params
    self.ws.send(json.dumps(msg))
    # Read responses until we get our ID back, skipping CDP events
    deadline = time.time() + timeout
    old_timeout = self.ws.gettimeout()
    try:
        while time.time() < deadline:
            self.ws.settimeout(max(0.1, deadline - time.time()))
            try:
                resp = json.loads(self.ws.recv())
            except websocket.WebSocketTimeoutException:
                continue
            if resp.get('id') == msg_id:
                return resp
            # Skip CDP events (no 'id' field)
    finally:
        self.ws.settimeout(old_timeout)
    raise TimeoutError(f"CDP response timeout for {method}")
```

Method: `CDPBase._js`

```
def _js(self, expression, await_promise=False, timeout=None)
```

Execute JavaScript in the page and return the result value.

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns `params = {'expression': expression, 'returnByValue': True}`.
2. `if await_promise:` branches conditionally.
3. Assigns `resp = self._send('Runtime.evaluate', params, timeout=timeout or....)`
4. Assigns `result = resp.get('result', {}).get('result', {})`.
5. `if result.get('subtype') == 'error':` branches conditionally.
6. `return result.get('value')`.

Code (copy-paste safe)

```
def _js(self, expression, await_promise=False, timeout=None):
    """Execute JavaScript in the page and return the result value."""
    params = {'expression': expression, 'returnByValue': True}
    if await_promise:
        params['awaitPromise'] = True
    resp = self._send('Runtime.evaluate', params, timeout=timeout or (30 if await_promise else 10))
    result = resp.get('result', {}).get('result', {})
    if result.get('subtype') == 'error':
        desc = result.get('description', 'JS error')
        raise RuntimeError(f"JS error: {desc}")
    return result.get('value')
```

Method: CDPBase._fetch_json

```
def _fetch_json(self, url, method='GET', body=None, extra_headers=None)
```

Execute a fetch() call from the browser context and parse JSON response.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `headers = {'Content-Type': 'application/json'}`.
2. `if extra_headers:` branches conditionally.
3. `if body and method == 'POST':` branches conditionally (with an **else/elif** arm).
4. Assigns `raw = self._js(js, await_promise=True)`.
5. `if not raw:` branches conditionally.
6. Assigns `result = json.loads(raw)`.

7. Assigns `status = result.get('status', 0)`.
8. `if status != 200`: branches conditionally.
9. `try` block with 1 **except** clause.

Code (copy-paste safe)

```
def _fetch_json(self, url, method='GET', body=None, extra_headers=None):
    """Execute a fetch() call from the browser context and parse JSON response."""
    headers = {'Content-Type': 'application/json'}
    if extra_headers:
        headers.update(extra_headers)

    if body and method == 'POST':
        js = f"""
            (async () => {{
                const r = await fetch({json.dumps(url)}, {{
                    method: 'POST',
                    credentials: 'include',
                    headers: {json.dumps(headers)},
                    body: JSON.stringify({json.dumps(body)})
                }});
                const text = await r.text();
                return JSON.stringify({{status: r.status, body: text}});
            }})()
        """
    else:
        fetch_opts = f"{{credentials: 'include', headers: {json.dumps(headers)}}}"
        js = f"""
            (async () => {{
                const r = await fetch({json.dumps(url)}, {fetch_opts});
                const text = await r.text();
                return JSON.stringify({{status: r.status, body: text}});
            }})()
        """

    raw = self._js(js, await_promise=True)
    if not raw:
        return None
    result = json.loads(raw)
    status = result.get('status', 0)
    if status != 200:
        self.logger.warning(f"[FETCH] {method} {url} → HTTP {status}")
        return None
    try:
        return json.loads(result['body'])
    except (json.JSONDecodeError, KeyError):
        return result.get('body')
```

Method: `CDPBase._fetch_json_bearer`

```
def _fetch_json_bearer(self, url, token, method='GET', body=None)

    Fetch with Bearer token auth.
```

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **return** self._fetch_json(url, method=method, body=body, extra_headers={'Aut...

Code (copy-paste safe)

```
def _fetch_json_bearer(self, url, token, method='GET', body=None):
    """Fetch with Bearer token auth."""
    return self._fetch_json(url, method=method, body=body,
                            extra_headers={'Authorization': f'Bearer {token}'})
```

Method: CDPBase.login

```
def login(self, open_url=None)
```

Attach to an existing Chrome tab for this firm. If open_url is provided and no tab is found, launch/reuse Chrome debug and open the URL, then wait for the tab to appear.

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.

Step by step (top-level body)

1. **if not** WS_AVAILABLE: branches conditionally.
2. Assigns tab = self._find_tab().
3. **if not** tab and open_url: branches conditionally.
4. **if not** tab: branches conditionally.
5. Calls self._connect_ws(...) for its side effect.
6. Assigns self.logged_in = True.
7. Assigns self._login_timestamp = time.time().
8. Calls self.logger.info(...) for its side effect.
9. **return** True.

Code (copy-paste safe)

```
def login(self, open_url=None):
    """
    Attach to an existing Chrome tab for this firm.
    If open_url is provided and no tab is found, launch/reuse Chrome debug
    and open the URL, then wait for the tab to appear.
    """
    if not WS_AVAILABLE:
        raise ImportError("websocket-client package required: pip install websocket-client")

    tab = self._find_tab()

    # If no tab found and we have a URL, launch Chrome and open the tab
    if not tab and open_url:
        ensure_chrome_debug(url=open_url, port=self.debug_port)
```



```

        # Wait for the tab to appear (user may need to finish loading)
        for _ in range(30):
            time.sleep(1)
            tab = self._find_tab()
            if tab:
                break

        if not tab:
            raise ConnectionError(
                f"No {self.FIRM_NAME} tab found on port {self.debug_port}. "
                f"Open {self.DOMAIN} in Chrome with --remote-debugging-port={self.debug_port}")
        self._connect_ws(tab)
        self.logged_in = True
        self._login_timestamp = time.time()
        self.logger.info(f"[LOGIN] Attached to {self.FIRM_NAME} tab")
        return True

```

Method: CDPBase.is_connected

```
def is_connected(self)
```

Check if the WebSocket connection is alive.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def is_connected(self):
    """Check if the WebSocket connection is alive."""
    try:
        if not self.ws or not self.ws.connected:
            return False
        # Quick ping with short timeout
        self._send('Runtime.evaluate',
                    {'expression': '1+1', 'returnByValue': True},
                    timeout=5)
        return True
    except Exception:
        self.logged_in = False
        return False

```

Method: CDPBase.disconnect

```
def disconnect(self)
```

Close the WebSocket (does NOT close the Chrome tab).

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't

have to handle it.

Step by step (top-level body)

1. Assigns `self.logged_in = False`.
2. `if self.ws:` branches conditionally.

Code (copy-paste safe)

```
def disconnect(self):
    """Close the WebSocket (does NOT close the Chrome tab)."""
    self.logged_in = False
    if self.ws:
        try:
            self.ws.close()
        except Exception:
            pass
    self.ws = None
```

Method: CDPBase.close

```
def close(self)
```

Alias for disconnect.

Step by step (top-level body)

1. Calls `self.disconnect(...)` for its side effect.

Code (copy-paste safe)

```
def close(self):
    """Alias for disconnect."""
    self.disconnect()
```

Method: CDPBase.get_account_stats

```
def get_account_stats(self)
```

Override in subclasses.

Step by step (top-level body)

1. `return {}`.

Code (copy-paste safe)

```
def get_account_stats(self):
    """Override in subclasses."""
    return {}
```

Method: CDPBase.get_billing_history

```
def get_billing_history(self)
```

Override in subclasses.

Step by step (top-level body)

1. `return []`.

Code (copy-paste safe)

```
def get_billing_history(self):
    """Override in subclasses."""
    return []
```

Method: CDPBase.get_all_accounts

```
def get_all_accounts(self)

    Override in subclasses.
```

Step by step (top-level body)

1. return [].

Code (copy-paste safe)

```
def get_all_accounts(self):
    """Override in subclasses."""
    return []
```

Method: CDPBase.get_payouts

```
def get_payouts(self)

    Override in subclasses.
```

Step by step (top-level body)

1. return [].

Code (copy-paste safe)

```
def get_payouts(self):
    """Override in subclasses."""
    return []
```

```
class TradeifyAccount(CDPBase)
```

Tradeify dashboard scraper — MUI React app at app-f.tradeify.co Auth: session cookies (credentials: include)

```
DOMAIN = 'tradeify.co'
FIRM_NAME = 'Tradeify'
BASE = 'https://app-f.tradeify.co'
```

Code (copy-paste safe)

```
class TradeifyAccount(CDPBase):
    """
    Tradeify dashboard scraper — MUI React app at app-f.tradeify.co
    Auth: session cookies (credentials: include)
    """
    DOMAIN = "tradeify.co"
    FIRM_NAME = "Tradeify"
    BASE = "https://app-f.tradeify.co"

    def _get_profile(self):
```

```

        return self._fetch_json(f"{self.BASE}/api/auth/profile/")

def _get_broker_credentials(self):
    return self._fetch_json(f"{self.BASE}/api/dashboard/broker-credentials")

def _get_account_overview(self):
    return self._fetch_json(
        f"{self.BASE}/api/dashboard/account-overview?hide_blow_account=true&page=1&page_size=10")

def get_account_stats(self):
    """Return account stats from Tradeify account-overview + broker credentials."""
    with self.lock:
        now = time.time()
        if now - self._stats_last_fetch < 2.0 and self._cached_stats:
            return self._cached_stats

        stats = {
            "Account Number": "Unknown",
            "Balance": "N/A",
            "Equity": "N/A",
            "Profit/Loss": "N/A",
            "Open Trades": "0",
            "Symbol": "",
            "Direction": "",
            "Drawdown": "N/A",
            "Profit Target": "N/A",
            "Phase": "N/A",
            "Status": "N/A",
        }

    try:
        # account-overview has balance, status, type – nested {success, data: {success, ..., data
        overview = self._get_account_overview()
        if overview and overview.get('success'):
            outer = overview.get('data', {})
            items = outer.get('data', []) if isinstance(outer, dict) else []
            if isinstance(items, list) and items:
                first = items[0]
                stats["Account Number"] = first.get('broker_account_id', 'Unknown')
                bal = first.get('account_balance')
                if bal is not None:
                    try:
                        stats["Balance"] = f"${float(bal):,.2f}"
                    except (ValueError, TypeError):
                        pass
                init_bal = first.get('initial_balance')
                daily_loss = first.get('daily_loss_limit')
                if daily_loss is not None:
                    try:
                        stats["Drawdown"] = f"${float(daily_loss):,.2f}"
                    except (ValueError, TypeError):
                        pass
                profit_target = first.get('profit_target')
                if profit_target is not None:
                    try:
                        stats["Profit Target"] = f"${float(profit_target):,.2f}"
                    except (ValueError, TypeError):
                        pass
                stats["Phase"] = first.get('funded_status', first.get('account_type', 'N/A'))
                stats["Status"] = first.get('account_status', 'N/A')

```

```

        # Fallback to broker creds for account name if overview didn't have it
        if stats["Account Number"] == "Unknown":
            creds = self._get_broker_credentials()
            if creds and isinstance(creds, dict):
                outer = creds.get('data', creds)
                items = outer.get('data', outer) if isinstance(outer, dict) else outer
                if isinstance(items, list) and items:
                    stats["Account Number"] = items[0].get('name', 'Unknown')
                    if stats["Status"] == "N/A":
                        stats["Status"] = "Active"

    except Exception as e:
        self.logger.warning(f"[STATS] Error: {e}")

    self._cached_stats = stats
    self._stats_last_fetch = now
    return stats

def get_billing_history(self):
    """Get order history as billing records with Tradovate account linking."""
    billing = []
    page = 1
    while True:
        result = self._fetch_json(
            f"{self.BASE}/api/dashboard/get-order-list?page={page}&page_size=100")
        if not result or not isinstance(result, dict):
            break
        items = result.get('data', [])
        if not isinstance(items, list) or not items:
            break
        for item in items:
            plan = item.get('plan', {}) or {}
            payment = item.get('payment', {}) or {}
            broker_acct = item.get('broker_account', {}) or {}
            status_raw = str(item.get('status', '')).lower()
            price = float(item.get('amount', payment.get('amount_paid', plan.get('price', 0))) or 0)
            acct_no = broker_acct.get('broker_account_id', broker_acct.get('account_id', ''))
            billing.append({
                "sn": str(item.get("id", "")),
                "account_no": acct_no or str(item.get("id", "")),
                "login": acct_no,
                "status": "APPROVED" if status_raw in ("completed", "active", "paid",
                    "approved") else str(item.get("status", "")),
                "date": str(item.get("created_at", ""))[:10],
                "paid_amount": f"${price:.2f}",
                "paid_amount_numeric": price,
                "funding_package": f"{plan.get('plan_type', '')} {plan.get('account_type', '')}".strip(),
                "order_type": item.get("order_type", ""),
                "coupon_code": (item.get('coupon_data') or {}).get('code', ''),
                "payment_method": payment.get("payment_method", ""),
                "transaction_id": payment.get("transaction_id", ""),
            })
        meta = result.get('meta', {})
        if page >= meta.get('total_pages', 1):
            break
        page += 1
    return billing

def get_account_mapping(self):
    """Map Tradeify broker_account_id -> Tradovate account info.

```

```

    Uses get-order-list (broker_account has tradovate account_id) and
    account-overview (has broker_account_id + account_id) to build
    a mapping keyed by broker_account_id.
    """
    mapping = {}
    # account-overview gives broker_account_id -> account_id
    overview = self._fetch_json(
        f"{self.BASE}/api/dashboard/account-overview?hide_blow_account=false&page=1&page_size=100")
    if overview and overview.get('success'):
        outer = overview.get('data', {})
        items = outer.get('data', []) if isinstance(outer, dict) else []
        for item in (items if isinstance(items, list) else []):
            bid = item.get('broker_account_id', '')
            if bid:
                balance = item.get('account_balance')
                initial = item.get('initial_balance')
                profit_target = item.get('profit_target')
                daily_loss = item.get('daily_loss_limit')
                # min_equity = initial_balance - daily_loss_limit
                min_equity = None
                if initial is not None and daily_loss is not None:
                    try:
                        min_equity = float(initial) - float(daily_loss)
                    except (ValueError, TypeError):
                        pass
                acct_status = item.get('account_status', '')
                mapping[bid] = {
                    "tradovate_account_name": bid,
                    "tradovate_account_id": item.get('account_id'),
                    "account_type": item.get('account_type'),
                    "funded_status": item.get('funded_status'),
                    "account_status": acct_status,
                    "initial_balance": initial,
                    "balance": balance,
                    "starting_balance": initial,
                    "profit_target": profit_target,
                    "min_equity": min_equity,
                    "breached": str(acct_status).lower() in ("breached", "failed", "blown"),
                }
                self.logger.info(
                    f"[MAPPING] broker={bid} account_id={item.get('account_id')} "
                    f"type={item.get('account_type')} target={profit_target} min_eq={min_equity}")
            self.logger.info(f"[MAPPING] Built mapping for {len(mapping)} account(s)")
    return mapping

def get_all_accounts(self):
    """Get all accounts from account-overview endpoint."""
    overview = self._get_account_overview()
    if not overview or not overview.get('success'):
        return []
    outer = overview.get('data', {})
    items = outer.get('data', []) if isinstance(outer, dict) else []
    return items if isinstance(items, list) else []

def get_payouts(self):
    """Get payout tracking data."""
    result = self._fetch_json(
        f"{self.BASE}/api/payouts/payout-tracking?page=1&page_size=100"
        f"&start_date=2020-01-01&end_date=2030-12-31"
    )

```

```

    if not result or not result.get('success'):
        return []
    # {success, data: {success, ..., data: [...]}}
    outer = result.get('data', {})
    items = outer.get('data', []) if isinstance(outer, dict) else []
    return items if isinstance(items, list) else []

```

Method: TradeifyAccount._get_profile

```
def _get_profile(self)
```

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **return** self._fetch_json(f'{self.BASE}/api/auth/profile/').

Code (copy-paste safe)

```
def _get_profile(self):
    return self._fetch_json(f'{self.BASE}/api/auth/profile/')
```

Method: TradeifyAccount._get_broker_credentials

```
def _get_broker_credentials(self)
```

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **return** self._fetch_json(f'{self.BASE}/api/dashboard/broker-credentials').

Code (copy-paste safe)

```
def _get_broker_credentials(self):
    return self._fetch_json(f'{self.BASE}/api/dashboard/broker-credentials')
```

Method: TradeifyAccount._get_account_overview

```
def _get_account_overview(self)
```

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **return** self._fetch_json(f'{self.BASE}/api/dashboard/account-overview?hide_....

Code (copy-paste safe)

```
def _get_account_overview(self):
    return self._fetch_json(
```

```
f"{self.BASE}/api/dashboard/account-overview?hide_blow_account=true&page=1&page_size=10")
```

Method: TradeifyAccount.get_account_stats

```
def get_account_stats(self)
```

Return account stats from Tradeify account-overview + broker credentials.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **with self.lock**: enters a context manager.

Code (copy-paste safe)

```
def get_account_stats(self):
    """Return account stats from Tradeify account-overview + broker credentials."""
    with self.lock:
        now = time.time()
        if now - self._stats_last_fetch < 2.0 and self._cached_stats:
            return self._cached_stats

        stats = {
            "Account Number": "Unknown",
            "Balance": "N/A",
            "Equity": "N/A",
            "Profit/Loss": "N/A",
            "Open Trades": "0",
            "Symbol": "",
            "Direction": "",
            "Drawdown": "N/A",
            "Profit Target": "N/A",
            "Phase": "N/A",
            "Status": "N/A",
        }

        try:
            # account-overview has balance, status, type – nested {success, data: {success, ..., data: {}}}
            overview = self._get_account_overview()
            if overview and overview.get('success'):
                outer = overview.get('data', {})
                items = outer.get('data', []) if isinstance(outer, dict) else []
                if isinstance(items, list) and items:
```



```

first = items[0]
stats["Account Number"] = first.get('broker_account_id', 'Unknown')
bal = first.get('account_balance')
if bal is not None:
    try:
        stats["Balance"] = f"${float(bal):,.2f}"
    except (ValueError, TypeError):
        pass
init_bal = first.get('initial_balance')
daily_loss = first.get('daily_loss_limit')
if daily_loss is not None:
    try:
        stats["Drawdown"] = f"${float(daily_loss):,.2f}"
    except (ValueError, TypeError):
        pass
profit_target = first.get('profit_target')
if profit_target is not None:
    try:
        stats["Profit Target"] = f"${float(profit_target):,.2f}"
    except (ValueError, TypeError):
        pass
stats["Phase"] = first.get('funded_status', first.get('account_type', 'N/A'))
stats["Status"] = first.get('account_status', 'N/A')

# Fallback to broker creds for account name if overview didn't have it
if stats["Account Number"] == "Unknown":
    creds = self._get_broker_credentials()
    if creds and isinstance(creds, dict):
        outer = creds.get('data', creds)
        items = outer.get('data', outer) if isinstance(outer, dict) else outer
        if isinstance(items, list) and items:
            stats["Account Number"] = items[0].get('name', 'Unknown')
            if stats["Status"] == "N/A":
                stats["Status"] = "Active"

except Exception as e:
    self.logger.warning(f"[STATS] Error: {e}")

self._cached_stats = stats
self._stats_last_fetch = now
return stats

```

Method: TradeifyAccount.get_billing_history

```
def get_billing_history(self)
```

Get order history as billing records with Tradovate account linking.

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns `billing = []`.
2. Assigns `page = 1`.
3. **while** `True`: loops until the condition is false.
4. **return** `billing`.

Code (copy-paste safe)

```
def get_billing_history(self):
    """Get order history as billing records with Tradovate account linking."""
    billing = []
    page = 1
    while True:
        result = self._fetch_json(
            f"{self.BASE}/api/dashboard/get-order-list?page={page}&page_size=100")
        if not result or not isinstance(result, dict):
            break
        items = result.get('data', [])
        if not isinstance(items, list) or not items:
            break
        for item in items:
            plan = item.get('plan', {}) or {}
            payment = item.get('payment', {}) or {}
            broker_acct = item.get('broker_account', {}) or {}
            status_raw = str(item.get('status', '')).lower()
            price = float(item.get('amount', payment.get('amount_paid', plan.get('price', 0))) or 0)
            acct_no = broker_acct.get('broker_account_id', broker_acct.get('account_id', ''))
            billing.append({
                "sn": str(item.get("id", "")),
                "account_no": acct_no or str(item.get("id", "")),
                "login": acct_no,
                "status": "APPROVED" if status_raw in ("completed", "active", "paid",
                    "approved") else str(item.get("status", "")),
                "date": str(item.get("created_at", ""))[:10],
                "paid_amount": f"${price:.2f}",
                "paid_amount_numeric": price,
                "funding_package": f"{plan.get('plan_type', '')} {plan.get('account_type', '')}".strip(),
                "order_type": item.get("order_type", ""),
                "coupon_code": (item.get('coupon_data') or {}).get('code', ''),
                "payment_method": payment.get("payment_method", ""),
                "transaction_id": payment.get("transaction_id", ""),
            })
        meta = result.get('meta', {})
        if page >= meta.get('total_pages', 1):
            break
        page += 1
    return billing
```

Method: TradeifyAccount.get_account_mapping

```
def get_account_mapping(self)
```

Map Tradeify broker_account_id -> Tradovate account info. Uses get-order-list (broker_account has tradovate account_id) and account-overview (has broker_account_id + account_id) to build a mapping keyed by broker_account_id.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns mapping = {}.
2. Assigns overview = self._fetch_json(f'{self.BASE}/api/dashboard/account-over....
3. **if** overview and overview.get('success'): branches conditionally.
4. Calls self.logger.info(...) for its side effect.
5. **return** mapping.

Code (copy-paste safe)

```
def get_account_mapping(self):
    """Map Tradeify broker_account_id -> Tradovate account info.

    Uses get-order-list (broker_account has tradovate account_id) and
    account-overview (has broker_account_id + account_id) to build
    a mapping keyed by broker_account_id.
    """
    mapping = {}
    # account-overview gives broker_account_id -> account_id
    overview = self._fetch_json(
        f"{self.BASE}/api/dashboard/account-overview?hide_blown_account=false&page=1&page_size=100")
    if overview and overview.get('success'):
        outer = overview.get('data', {})
        items = outer.get('data', []) if isinstance(outer, dict) else []
        for item in (items if isinstance(items, list) else []):
            bid = item.get('broker_account_id', '')
            if bid:
                balance = item.get('account_balance')
                initial = item.get('initial_balance')
                profit_target = item.get('profit_target')
                daily_loss = item.get('daily_loss_limit')
                # min_equity = initial_balance - daily_loss_limit
                min_equity = None
                if initial is not None and daily_loss is not None:
                    try:
                        min_equity = float(initial) - float(daily_loss)
                    except (ValueError, TypeError):
                        pass
                acct_status = item.get('account_status', '')
                mapping[bid] = {
                    "tradovate_account_name": bid,
                    "tradovate_account_id": item.get('account_id'),
                    "account_type": item.get('account_type'),
                    "funded_status": item.get('funded_status'),
```

```

        "account_status": acct_status,
        "initial_balance": initial,
        "balance": balance,
        "starting_balance": initial,
        "profit_target": profit_target,
        "min_equity": min_equity,
        "breached": str(acct_status).lower() in ("breached", "failed", "blown"),
    }
    self.logger.info(
        f"[MAPPING] broker={bid} account_id={item.get('account_id')} "
        f"type={item.get('account_type')} target={profit_target} min_eq={min_equity}")
    self.logger.info(f"[MAPPING] Built mapping for {len(mapping)} account(s)")
    return mapping

```

Method: TradeifyAccount.get_all_accounts

```
def get_all_accounts(self)
```

Get all accounts from account-overview endpoint.

Why these Python concepts were used

- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns `overview = self._get_account_overview()`.
2. `if not overview or not overview.get('success')`: branches conditionally.
3. Assigns `outer = overview.get('data', {})`.
4. Assigns `items = outer.get('data', [])` if `isinstance(outer, dict)` else `[]`.
5. `return items` if `isinstance(items, list)` else `[]`.

Code (copy-paste safe)

```

def get_all_accounts(self):
    """Get all accounts from account-overview endpoint."""
    overview = self._get_account_overview()
    if not overview or not overview.get('success'):
        return []
    outer = overview.get('data', {})
    items = outer.get('data', []) if isinstance(outer, dict) else []
    return items if isinstance(items, list) else []

```

Method: TradeifyAccount.get_payouts

```
def get_payouts(self)
```

Get payout tracking data.

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns `result = self._fetch_json(f'{self.BASE}/api/payouts/payout-trackin...')`
2. `if not result or not result.get('success')`: branches conditionally.
3. Assigns `outer = result.get('data', {})`.
4. Assigns `items = outer.get('data', []) if isinstance(outer, dict) else []`.
5. **return** `items if isinstance(items, list) else []`.

Code (copy-paste safe)

```
def get_payouts(self):
    """Get payout tracking data."""
    result = self._fetch_json(
        f"{self.BASE}/api/payouts/payout-tracking?page=1&page_size=100"
        f"&start_date=2020-01-01&end_date=2030-12-31"
    )
    if not result or not result.get('success'):
        return []
    # {success, data: {success, ..., data: [...]}
    outer = result.get('data', {})
    items = outer.get('data', []) if isinstance(outer, dict) else []
    return items if isinstance(items, list) else []
```

```
class LucidTradingAccount(CDPBase)
```

Lucid Trading dashboard scraper — Angular 19 app at dash.lucidtrading.com Auth: Bearer JWT from localStorage('auth_token') + cookies. API base: https://dash.lucidtrading.com/api Key: localStorage('userKey')

```
DOMAIN = 'lucidtrading.com'
FIRM_NAME = 'Lucid Trading'
BASE = 'https://dash.lucidtrading.com'
```

Code (copy-paste safe)

```
class LucidTradingAccount(CDPBase):
    """
    Lucid Trading dashboard scraper — Angular 19 app at dash.lucidtrading.com
    Auth: Bearer JWT from localStorage('auth_token') + cookies.
    API base: https://dash.lucidtrading.com/api
    Key: localStorage('userKey')
    """
    DOMAIN = "lucidtrading.com"
    FIRM_NAME = "Lucid Trading"
    BASE = "https://dash.lucidtrading.com"

    def _get_token(self):
        """Get JWT from localStorage."""
        return self._js("localStorage.getItem('auth_token')")

    def _get_user_key(self):
```

```

        """Get userKey from localStorage."""
        return self._js("localStorage.getItem('userKey')")

def _fetch_lucid(self, url):
    """Fetch from Lucid API with Bearer token."""
    token = self._get_token()
    if token:
        return self._fetch_json_bearer(url, token)
    return self._fetch_json(url)

def get_account_stats(self):
    """Get account stats from Lucid Trading."""
    with self.lock:
        now = time.time()
        if now - self._stats_last_fetch < 2.0 and self._cached_stats:
            return self._cached_stats

    stats = {
        "Account Number": "Unknown",
        "Balance": "N/A",
        "Equity": "N/A",
        "Profit/Loss": "N/A",
        "Open Trades": "0",
        "Symbol": "",
        "Direction": "",
        "Drawdown": "N/A",
        "Profit Target": "N/A",
        "Phase": "N/A",
        "Status": "N/A",
    }

    try:
        user_key = self._get_user_key()
        if not user_key:
            return stats

        # /api/users/summary/{userKey} returns array of accounts
        summary = self._fetch_lucid(
            f"{self.BASE}/api/users/summary/{user_key}")
        if isinstance(summary, list) and summary:
            first = summary[0]
            stats["Account Number"] = first.get('accountName', 'Unknown')
            stats["Status"] = first.get('status', 'N/A')
            stats["Phase"] = first.get('planCode', first.get('accountType', 'N/A'))
            bal = first.get('accountBalance')
            if isinstance(bal, (int, float)):
                stats["Balance"] = f"${bal:,.2f}"
            mll = first.get('minAccountBalance')
            if isinstance(mll, (int, float)):
                stats["Drawdown"] = f"${mll:,.2f}"
            target = first.get('profitTarget')
            if isinstance(target, (int, float)):
                stats["Profit Target"] = f"${target:,.2f}"
            pnl = first.get('totalPnlPeriod')
            if isinstance(pnl, (int, float)):
                stats["Profit/Loss"] = f"${pnl:,.2f}"

        # Get profile info
        profile = self._fetch_lucid(
            f"{self.BASE}/api/users/wp-profile?userKey={user_key}")

```

```

        if isinstance(profile, dict):
            name = f"{profile.get('firstName', '')} {profile.get('lastName', '')}".strip()
            if name:
                stats["Account Number"] = f"{stats['Account Number']} ({name})"

    except Exception as e:
        self.logger.warning(f"[STATS] Error: {e}")

    self._cached_stats = stats
    self._stats_last_fetch = now
    return stats

def get_billing_history(self):
    """Get order history from Lucid's profile/order-history API with account linking."""
    user_key = self._get_user_key()
    if not user_key:
        return []
    orders = self._fetch_lucid(
        f"{self.BASE}/api/users/order-history?userKey={user_key}&limit=50&offset=0")
    if not isinstance(orders, list):
        return []

    # Build plan label -> accountName mapping for linking orders to accounts
    acct_by_plan = {}
    summary = self._fetch_lucid(f"{self.BASE}/api/users/summary/{user_key}")
    if isinstance(summary, list):
        for s in summary:
            label = (s.get('planLabel') or '').strip()
            name = s.get('accountName', '')
            if label and name:
                acct_by_plan[label] = name

    billing = []
    for item in orders:
        amount = float(item.get("totalAmount", 0) or 0)
        status_raw = str(item.get("status", "")).lower()
        product = item.get("productNames", "")
        # Resolve account name: match productNames against planLabel
        # e.g. "LucidFlex 50K NT_TDV" matches planLabel "LucidFlex 50K"
        matched_acct = ""
        for label, acct_name in acct_by_plan.items():
            if label in product or product in label:
                matched_acct = acct_name
                break
        billing.append({
            "sn": str(item.get("orderId", "")),
            "account_no": matched_acct or str(item.get("orderId", "")),
            "login": matched_acct,
            "status": "APPROVED" if status_raw in ("completed", "active", "paid", "approved") else status_raw,
            "date": str(item.get("dateCreated", ""))[:10],
            "paid_amount": f"${amount:.2f}",
            "paid_amount_numeric": amount,
            "funding_package": product,
            "payment_method": item.get("paymentMethodTitle", ""),
            "transaction_id": item.get("transactionId", "")
        })
    return billing

def get_account_mapping(self):

```

```

"""Map Lucid account names to summary info for billing linking."""
user_key = self._get_user_key()
if not user_key:
    return {}
summary = self._fetch_lucid(f"{self.BASE}/api/users/summary/{user_key}")
if not isinstance(summary, list):
    return {}
mapping = {}
for s in summary:
    name = s.get('accountName', '')
    if name:
        balance = s.get('accountBalance')
        min_equity = s.get('minAccountBalance') # minimum account balance before breach
        profit_target = s.get('profitTarget') # target to pass
        status_raw = s.get('status', '')
        mapping[name] = {
            "tradovate_account_name": name,
            "plan_title": s.get('planLabel', s.get('planCode', '')),
            "balance": balance,
            "starting_balance": None,
            "status": status_raw,
            "profit_target": profit_target,
            "min_equity": min_equity,
            "breached": str(status_raw).lower() in ("breached", "failed", "blown"),
        }
        self.logger.info(f"[MAPPING] account={name} plan={s.get('planLabel')} "
                        f"target={profit_target} min_eq={min_equity}")
self.logger.info(f"[MAPPING] Built mapping for {len(mapping)} account(s)")
return mapping

def get_all_accounts(self):
    """Get all accounts from summary endpoint."""
    user_key = self._get_user_key()
    if not user_key:
        return []
    summary = self._fetch_lucid(
        f"{self.BASE}/api/users/summary/{user_key}")
    return summary if isinstance(summary, list) else []

def get_payouts(self):
    """Get payout history."""
    user_key = self._get_user_key()
    if not user_key:
        return []
    result = self._fetch_lucid(
        f"{self.BASE}/api/payout/payout-history?userKey={user_key}")
    if not result:
        return []
    return result if isinstance(result, list) else result.get('data', [])

```

Method: LucidTradingAccount._get_token

```
def _get_token(self)
    Get JWT from localStorage.
```

Step by step (top-level body)

1. **return** self._js("localStorage.getItem('auth_token')").

Code (copy-paste safe)

```
def _get_token(self):
    """Get JWT from localStorage."""
    return self._js("localStorage.getItem('auth_token')")
```

Method: LucidTradingAccount._get_user_key

```
def _get_user_key(self)
    Get userKey from localStorage.
```

Step by step (top-level body)

1. **return** self._js("localStorage.getItem('userKey')").

Code (copy-paste safe)

```
def _get_user_key(self):
    """Get userKey from localStorage."""
    return self._js("localStorage.getItem('userKey')")
```

Method: LucidTradingAccount._fetch_lucid

```
def _fetch_lucid(self, url)
    Fetch from Lucid API with Bearer token.
```

Step by step (top-level body)

1. Assigns token = self._get_token().
2. **if** token: branches conditionally.
3. **return** self._fetch_json(url).

Code (copy-paste safe)

```
def _fetch_lucid(self, url):
    """Fetch from Lucid API with Bearer token."""
    token = self._get_token()
    if token:
        return self._fetch_json_bearer(url, token)
    return self._fetch_json(url)
```

Method: LucidTradingAccount.get_account_stats

```
def get_account_stats(self)
    Get account stats from Lucid Trading.
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.

Step by step (top-level body)

1. **with self.lock**: enters a context manager.

Code (copy-paste safe)

```
def get_account_stats(self):
    """Get account stats from Lucid Trading."""
    with self.lock:
        now = time.time()
        if now - self._stats_last_fetch < 2.0 and self._cached_stats:
            return self._cached_stats

    stats = {
        "Account Number": "Unknown",
        "Balance": "N/A",
        "Equity": "N/A",
        "Profit/Loss": "N/A",
        "Open Trades": "0",
        "Symbol": "",
        "Direction": "",
        "Drawdown": "N/A",
        "Profit Target": "N/A",
        "Phase": "N/A",
        "Status": "N/A",
    }

    try:
        user_key = self._get_user_key()
        if not user_key:
            return stats

        # /api/users/summary/{userKey} returns array of accounts
        summary = self._fetch_lucid(
            f"{self.BASE}/api/users/summary/{user_key}")
        if isinstance(summary, list) and summary:
            first = summary[0]
            stats["Account Number"] = first.get('accountName', 'Unknown')
            stats["Status"] = first.get('status', 'N/A')
            stats["Phase"] = first.get('planCode', first.get('accountType', 'N/A'))
            bal = first.get('accountBalance')
            if isinstance(bal, (int, float)):
                stats["Balance"] = f"${bal:,.2f}"
            mll = first.get('minAccountBalance')
            if isinstance(mll, (int, float)):
                stats["Drawdown"] = f"${mll:,.2f}"
            target = first.get('profitTarget')
            if isinstance(target, (int, float)):
                stats["Profit Target"] = f"${target:,.2f}"
            pnl = first.get('totalPnlPeriod')
            if isinstance(pnl, (int, float)):
                stats["Profit/Loss"] = f"${pnl:,.2f}"

        # Get profile info
```

```

        profile = self._fetch_lucid(
            f"{self.BASE}/api/users/wp-profile?userKey={user_key}")
        if isinstance(profile, dict):
            name = f"{profile.get('firstName', '')} {profile.get('lastName', '')}".strip()
            if name:
                stats["Account Number"] = f"{stats['Account Number']} ({name})"

    except Exception as e:
        self.logger.warning(f"[STATS] Error: {e}")

    self._cached_stats = stats
    self._stats_last_fetch = now
    return stats

```

Method: LucidTradingAccount.get_billing_history

```
def get_billing_history(self)
```

Get order history from Lucid's profile/order-history API with account linking.

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns `user_key = self._get_user_key()`.
2. `if not user_key`: branches conditionally.
3. Assigns `orders = self._fetch_lucid(f"{self.BASE}/api/users/order-history?u....`
4. `if not isinstance(orders, list)`: branches conditionally.
5. Assigns `acct_by_plan = {}`.
6. Assigns `summary = self._fetch_lucid(f"{self.BASE}/api/users/summary/{user_k....`
7. `if isinstance(summary, list)`: branches conditionally.
8. Assigns `billing = []`.
9. `for item in orders`: iterates.
10. `return billing`.

Code (copy-paste safe)

```

def get_billing_history(self):
    """Get order history from Lucid's profile/order-history API with account linking."""
    user_key = self._get_user_key()
    if not user_key:
        return []
    orders = self._fetch_lucid(
        f"{self.BASE}/api/users/order-history?userKey={user_key}&limit=50&offset=0")
    if not isinstance(orders, list):

```

```

        return []

# Build plan label -> accountName mapping for linking orders to accounts
acct_by_plan = {}
summary = self._fetch_lucid(f"{self.BASE}/api/users/summary/{user_key}")
if isinstance(summary, list):
    for s in summary:
        label = (s.get('planLabel') or '').strip()
        name = s.get('accountName', '')
        if label and name:
            acct_by_plan[label] = name

billing = []
for item in orders:
    amount = float(item.get("totalAmount", 0) or 0)
    status_raw = str(item.get("status", "")).lower()
    product = item.get("productNames", "")
    # Resolve account name: match productNames against planLabel
    # e.g. "LucidFlex 50K NT_TDV" matches planLabel "LucidFlex 50K"
    matched_acct = ""
    for label, acct_name in acct_by_plan.items():
        if label in product or product in label:
            matched_acct = acct_name
            break
    billing.append({
        "sn": str(item.get("orderId", "")),
        "account_no": matched_acct or str(item.get("orderId", "")),
        "login": matched_acct,
        "status": "APPROVED" if status_raw in ("completed", "active", "paid", "approved") else status_raw,
        "date": str(item.get("dateCreated", ""))[:10],
        "paid_amount": f"${amount:.2f}",
        "paid_amount_numeric": amount,
        "funding_package": product,
        "payment_method": item.get("paymentMethodTitle", ""),
        "transaction_id": item.get("transactionId", ""),
    })
return billing

```

Method: LucidTradingAccount.get_account_mapping

```
def get_account_mapping(self)
```

Map Lucid account names to summary info for billing linking.

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.

Step by step (top-level body)

1. Assigns `user_key = self._get_user_key()`.
2. `if not user_key`: branches conditionally.
3. Assigns `summary = self._fetch_lucid(f"{self.BASE}/api/users/summary/{user_k...`

4. `if not isinstance(summary, list):` branches conditionally.
5. Assigns `mapping = {}`.
6. `for s in summary:` iterates.
7. Calls `self.logger.info(...)` for its side effect.
8. `return mapping`.

Code (copy-paste safe)

```
def get_account_mapping(self):
    """Map Lucid account names to summary info for billing linking."""
    user_key = self._get_user_key()
    if not user_key:
        return {}
    summary = self._fetch_lucid(f"{self.BASE}/api/users/summary/{user_key}")
    if not isinstance(summary, list):
        return {}
    mapping = {}
    for s in summary:
        name = s.get('accountName', '')
        if name:
            balance = s.get('accountBalance')
            min_equity = s.get('minAccountBalance') # minimum account balance before breach
            profit_target = s.get('profitTarget')    # target to pass
            status_raw = s.get('status', '')
            mapping[name] = {
                "tradovate_account_name": name,
                "plan_title": s.get('planLabel', s.get('planCode', '')),
                "balance": balance,
                "starting_balance": None,
                "status": status_raw,
                "profit_target": profit_target,
                "min_equity": min_equity,
                "breached": str(status_raw).lower() in ("breached", "failed", "blown"),
            }
            self.logger.info(f"[MAPPING] account={name} plan={s.get('planLabel')} "
                           f"target={profit_target} min_eq={min_equity}")
    self.logger.info(f"[MAPPING] Built mapping for {len(mapping)} account(s)")
    return mapping
```

Method: `LucidTradingAccount.get_all_accounts`

```
def get_all_accounts(self)
```

Get all accounts from summary endpoint.

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns `user_key = self._get_user_key()`.
2. `if not user_key`: branches conditionally.
3. Assigns `summary = self._fetch_lucid(f'{self.BASE}/api/users/summary/{user_k...`
4. `return summary` if `isinstance(summary, list)` else `[]`.

Code (copy-paste safe)

```
def get_all_accounts(self):
    """Get all accounts from summary endpoint."""
    user_key = self._get_user_key()
    if not user_key:
        return []
    summary = self._fetch_lucid(
        f'{self.BASE}/api/users/summary/{user_key}')
    return summary if isinstance(summary, list) else []
```

Method: `LucidTradingAccount.get_payouts`

```
def get_payouts(self)
```

Get payout history.

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns `user_key = self._get_user_key()`.
2. `if not user_key`: branches conditionally.
3. Assigns `result = self._fetch_lucid(f'{self.BASE}/api/payout/payout-history....`
4. `if not result`: branches conditionally.
5. `return result` if `isinstance(result, list)` else `result.get('data', [])`.

Code (copy-paste safe)

```
def get_payouts(self):
    """Get payout history."""
    user_key = self._get_user_key()
    if not user_key:
        return []
    result = self._fetch_lucid(
        f'{self.BASE}/api/payout/payout-history?userKey={user_key}')
    if not result:
        return []
    return result if isinstance(result, list) else result.get('data', [])
```

```
class TopStepAccount(CDPBase)
```

TopStep dashboard scraper — React + Apollo GraphQL at dashboard.topstep.com Auth: httpOnly session cookies (credentials: include) REST base: https://api.topstep.com GraphQL: https://crystal.topstep.com/graphql/q

```
DOMAIN = 'topstep.com'
FIRM_NAME = 'TopStep'
REST_BASE = 'https://api.topstep.com'
GQL_URL = 'https://crystal.topstep.com/graphql/q'
```

Code (copy-paste safe)

```
class TopStepAccount(CDPBase):
    """
    TopStep dashboard scraper — React + Apollo GraphQL at dashboard.topstep.com
    Auth: httpOnly session cookies (credentials: include)
    REST base: https://api.topstep.com
    GraphQL: https://crystal.topstep.com/graphql/q
    """
    DOMAIN = "topstep.com"
    FIRM_NAME = "TopStep"
    REST_BASE = "https://api.topstep.com"
    GQL_URL = "https://crystal.topstep.com/graphql/q"

    def _fetch_ts(self, path, params=None):
        """Fetch from TopStep REST API with cookies."""
        url = f"{self.REST_BASE}{path}"
        if params:
            qs = '&'.join(f"{k}={v}" for k, v in params.items())
            url += f"?{qs}"
        return self._fetch_json(url)

    def _graphql(self, query, variables=None):
        """Execute a GraphQL query against TopStep's Crystal endpoint."""
        body = {'query': query}
        if variables:
            body['variables'] = variables
        headers = {'Content-Type': 'application/json'}

        js = f"""
        (async () => {{
            const r = await fetch({json.dumps(self.GQL_URL)}, {{
                method: 'POST',
                credentials: 'include',
                headers: {json.dumps(headers)},
                body: JSON.stringify({json.dumps(body)})
            }});
            const text = await r.text();
            return JSON.stringify({{status: r.status, body: text}});
        }})()
        """
        raw = self._js(js, await_promise=True)
        if not raw:
            return None
        result = json.loads(raw)
        if result.get('status') != 200:
            return None
        try:
            return json.loads(result['body'])
        except:
```

```

        except (json.JSONDecodeError, KeyError):
            return None

def _get_profile(self):
    return self._fetch_ts('/me/profile/')

def _get_user_id(self):
    """Extract user ID from profile. Profile: {user: {user: {...}, id: N}}."""
    profile = self._get_profile()
    if not profile:
        return None
    # Try nested user.user structure
    user = profile.get('user', profile)
    if isinstance(user, dict):
        uid = user.get('id', user.get('userId', user.get('legacyAppId')))
        if uid:
            return uid
        inner = user.get('user', {})
        if isinstance(inner, dict):
            return inner.get('legacyAppId', inner.get('id'))
    return None

def _get_email(self):
    """Extract email from profile."""
    profile = self._get_profile()
    if not profile:
        return None
    user = profile.get('user', profile)
    if isinstance(user, dict):
        email = user.get('email')
        if email:
            return email
        inner = user.get('user', {})
        if isinstance(inner, dict):
            return inner.get('email')
    return None

def get_account_stats(self):
    """Get account stats from TopStep."""
    with self.lock:
        now = time.time()
        if now - self._stats_last_fetch < 2.0 and self._cached_stats:
            return self._cached_stats

        stats = {
            "Account Number": "Unknown",
            "Balance": "N/A",
            "Equity": "N/A",
            "Profit/Loss": "N/A",
            "Open Trades": "0",
            "Symbol": "",
            "Direction": "",
            "Drawdown": "N/A",
            "Profit Target": "N/A",
            "Phase": "N/A",
            "Status": "N/A",
        }

    try:
        profile = self._get_profile()

```



```

        if profile:
            # Profile: {user: {user: {email, ...}, id: N}}
            user = profile.get('user', profile)
            inner = user.get('user', user) if isinstance(user, dict) else user
            if isinstance(inner, dict):
                stats["Account Number"] = inner.get('email', 'Unknown')

        accounts = self._fetch_ts('/me/accounts/basic', {
            'offset': '0', 'limit': '15',
            'sortBy': 'createdAt', 'sortOrder': 'desc'
        })
        if accounts:
            # Response: {accounts: [...], hasMore, total}
            items = accounts.get('accounts', accounts.get('data', []))
            if isinstance(items, list) and items:
                first = items[0]
                stats["Account Number"] = first.get('accountNumber', first.get('name', stats[
                    "Account Number"]))
                bal = first.get('balance', first.get('accountBalance'))
                if isinstance(bal, (int, float)):
                    stats["Balance"] = f"${bal:,.2f}"
                stats["Status"] = first.get('status', 'N/A')
                stats["Phase"] = first.get('type', first.get('phase', 'N/A'))

    except Exception as e:
        self.logger.warning(f"[STATS] Error: {e}")

    self._cached_stats = stats
    self._stats_last_fetch = now
    return stats

def get_billing_history(self):
    """Get purchase history via GraphQL."""
    user_id = self._get_user_id()
    if not user_id:
        # Fallback to REST
        result = self._fetch_ts('/me/purchases/')
        if result:
            items = result.get('data', result) if isinstance(result, dict) else result
            return items if isinstance(items, list) else []
        return []

    gql = f"""
    {{
        normalizedPurchasesByUser(userid: {user_id}, first: 100) {{
            nodes {{
                id
                source
                type
                subtotal
                discount
                tax
                total
                method
                createdAt
            }}
        }}
    }}
    """
    result = self._graphql(gql)

```

```

    if not result:
        return []
    nodes = (result.get('data', {}))
        .get('normalizedPurchasesByUser', {})
        .get('nodes', []))
    billing = []
    for item in nodes:
        billing.append({
            "sn": str(item.get("id", "")),
            "account_no": "",
            "status": "APPROVED",
            "date": str(item.get("createdAt", ""))[:10],
            "paid_amount": f"${item.get('total', 0):.2f}",
            "paid_amount_numeric": float(item.get("total", 0) or 0),
            "funding_package": item.get("type", item.get("source", "")),
            "payment_method": item.get("method", ""),
        })
    return billing

def get_subscriptions(self):
    """Get subscription history via GraphQL."""
    user_id = self._get_user_id()
    if not user_id:
        return []
    gql = f"""
    {{
        normalizedSubsByUser(userid: {user_id}, first: 50) {{
            nodes {{
                id
                source
                productName
                amount
                total
                status
                createdAt
            }}
        }}
    }}
    """
    result = self._graphql(gql)
    if not result:
        return []
    return (result.get('data', {}))
        .get('normalizedSubsByUser', {})
        .get('nodes', []))

def get_all_accounts(self):
    """Get all accounts from REST."""
    accounts = self._fetch_ts('/me/accounts/basic', {
        'offset': '0', 'limit': '100',
        'sortBy': 'createdAt', 'sortOrder': 'desc'
    })
    if accounts:
        # Response: {accounts: [...], hasMore, total}
        items = accounts.get('accounts', accounts.get('data', []))
        if isinstance(items, list):
            return items
    return []

def get_payouts(self):

```

```

"""Get payouts from REST."""
result = self._fetch_ts('/me/payouts/')
if not result:
    return []
items = result.get('data', result) if isinstance(result, dict) else result
return items if isinstance(items, list) else []

```

Method: TopStepAccount._fetch_ts

```
def _fetch_ts(self, path, params=None)
```

Fetch from TopStep REST API with cookies.

Why these Python concepts were used

- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns url = f'{self.REST_BASE}{path}'.
2. if params: branches conditionally.
3. return self._fetch_json(url).

Code (copy-paste safe)

```

def _fetch_ts(self, path, params=None):
    """Fetch from TopStep REST API with cookies."""
    url = f"{self.REST_BASE}{path}"
    if params:
        qs = '&'.join(f"{k}={v}" for k, v in params.items())
        url += f"?{qs}"
    return self._fetch_json(url)

```

Method: TopStepAccount._graphql

```
def _graphql(self, query, variables=None)
```

Execute a GraphQL query against TopStep's Crystal endpoint.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns body = {'query': query}.
2. if variables: branches conditionally.

3. Assigns headers = {'Content-Type': 'application/json'}.
4. Assigns js = f""" (async () => {{\n const r
5. Assigns raw = self._js(js, await_promise=True).
6. if not raw: branches conditionally.
7. Assigns result = json.loads(raw).
8. if result.get('status') != 200: branches conditionally.
9. try block with 1 except clause.

Code (copy-paste safe)

```
def _graphql(self, query, variables=None):
    """Execute a GraphQL query against TopStep's Crystal endpoint."""
    body = {'query': query}
    if variables:
        body['variables'] = variables
    headers = {'Content-Type': 'application/json'}

    js = f"""
    (async () => {{
        const r = await fetch({json.dumps(self.GQL_URL)}, {{
            method: 'POST',
            credentials: 'include',
            headers: {json.dumps(headers)},
            body: JSON.stringify({json.dumps(body)})
        }});
        const text = await r.text();
        return JSON.stringify({{status: r.status, body: text}});
    }}())
    """
    raw = self._js(js, await_promise=True)
    if not raw:
        return None
    result = json.loads(raw)
    if result.get('status') != 200:
        return None
    try:
        return json.loads(result['body'])
    except (json.JSONDecodeError, KeyError):
        return None
```

Method: TopStepAccount._get_profile

```
def _get_profile(self)
```

Step by step (top-level body)

1. return self._fetch_ts('/me/profile/').

Code (copy-paste safe)

```
def _get_profile(self):
    return self._fetch_ts('/me/profile/')
```

Method: TopStepAccount._get_user_id

```
def _get_user_id(self)
```

Extract user ID from profile. Profile: {user: {user: {...}, id: N}}.

Why these Python concepts were used

- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.

Step by step (top-level body)

1. Assigns `profile = self._get_profile()`.
2. **if not profile:** branches conditionally.
3. Assigns `user = profile.get('user', profile)`.
4. **if isinstance(user, dict):** branches conditionally.
5. **return None.**

Code (copy-paste safe)

```
def _get_user_id(self):
    """Extract user ID from profile. Profile: {user: {user: {...}, id: N}}."""
    profile = self._get_profile()
    if not profile:
        return None
    # Try nested user.user structure
    user = profile.get('user', profile)
    if isinstance(user, dict):
        uid = user.get('id', user.get('userId', user.get('legacyAppId')))
        if uid:
            return uid
        inner = user.get('user', {})
        if isinstance(inner, dict):
            return inner.get('legacyAppId', inner.get('id'))
    return None
```

Method: TopStepAccount._get_email

```
def _get_email(self)
```

Extract email from profile.

Why these Python concepts were used

- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.

Step by step (top-level body)

1. Assigns `profile = self._get_profile()`.
2. **if not profile:** branches conditionally.
3. Assigns `user = profile.get('user', profile)`.
4. **if isinstance(user, dict):** branches conditionally.
5. **return None.**

Code (copy-paste safe)

```
def _get_email(self):
    """Extract email from profile."""
    profile = self._get_profile()
    if not profile:
        return None
    user = profile.get('user', profile)
    if isinstance(user, dict):
        email = user.get('email')
        if email:
            return email
        inner = user.get('user', {})
        if isinstance(inner, dict):
            return inner.get('email')
    return None
```

Method: TopStepAccount.get_account_stats

```
def get_account_stats(self)
    Get account stats from TopStep.
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **with self.lock**: enters a context manager.

Code (copy-paste safe)

```
def get_account_stats(self):
    """Get account stats from TopStep."""
    with self.lock:
        now = time.time()
        if now - self._stats_last_fetch < 2.0 and self._cached_stats:
            return self._cached_stats

    stats = {
        "Account Number": "Unknown",
        "Balance": "N/A",
        "Equity": "N/A",
        "Profit/Loss": "N/A",
        "Open Trades": "0",
        "Symbol": "",
        "Direction": "",
```

```

        "Drawdown": "N/A",
        "Profit Target": "N/A",
        "Phase": "N/A",
        "Status": "N/A",
    }

    try:
        profile = self._get_profile()
        if profile:
            # Profile: {user: {user: {email, ...}, id: N}}
            user = profile.get('user', profile)
            inner = user.get('user', user) if isinstance(user, dict) else user
            if isinstance(inner, dict):
                stats["Account Number"] = inner.get('email', 'Unknown')

        accounts = self._fetch_ts('/me/accounts/basic', {
            'offset': '0', 'limit': '15',
            'sortBy': 'createdAt', 'sortOrder': 'desc'
        })
        if accounts:
            # Response: {accounts: [...], hasMore, total}
            items = accounts.get('accounts', accounts.get('data', []))
            if isinstance(items, list) and items:
                first = items[0]
                stats["Account Number"] = first.get('accountNumber', first.get('name', stats[
                    "Account Number"]))
                bal = first.get('balance', first.get('accountBalance'))
                if isinstance(bal, (int, float)):
                    stats["Balance"] = f"${bal:,.2f}"
                stats["Status"] = first.get('status', 'N/A')
                stats["Phase"] = first.get('type', first.get('phase', 'N/A'))

    except Exception as e:
        self.logger.warning(f"[STATS] Error: {e}")

    self._cached_stats = stats
    self._stats_last_fetch = now
    return stats

```

Method: TopStepAccount.get_billing_history

```
def get_billing_history(self)

    Get purchase history via GraphQL.
```

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns `user_id = self._get_user_id()`.

2. **if** not `user_id`: branches conditionally.
3. Assigns `gql = f'\n {{\n normalizedPurchasesByUser(use....`
4. Assigns `result = self._graphql(gql)`.
5. **if** not `result`: branches conditionally.
6. Assigns `nodes = result.get('data', {}).get('normalizedPurchasesByUser', {...`
7. Assigns `billing = []`.
8. **for** `item` in `nodes`: iterates.
9. **return** `billing`.

Code (copy-paste safe)

```
def get_billing_history(self):
    """Get purchase history via GraphQL."""
    user_id = self._get_user_id()
    if not user_id:
        # Fallback to REST
        result = self._fetch_ts('/me/purchases/')
        if result:
            items = result.get('data', result) if isinstance(result, dict) else result
            return items if isinstance(items, list) else []
        return []

    gql = f"""
    {{
      normalizedPurchasesByUser(userid: {user_id}, first: 100) {{
        nodes {{
          id
          source
          type
          subtotal
          discount
          tax
          total
          method
          createdAt
        }}
      }}
    }}
    """
    result = self._graphql(gql)
    if not result:
        return []
    nodes = (result.get('data', {}).
              .get('normalizedPurchasesByUser', {}).
              .get('nodes', []))
    billing = []
    for item in nodes:
        billing.append({
            "sn": str(item.get("id", "")),
            "account_no": "",
            "status": "APPROVED",
            "date": str(item.get("createdAt", ""))[:10],
            "paid_amount": f"${item.get('total', 0):.2f}",
            "paid_amount_numeric": float(item.get("total", 0) or 0),
            "funding_package": item.get("type", item.get("source", "")),
        })
```



```

        "payment_method": item.get("method", ""),
    })
    return billing

```

Method: TopStepAccount.get_subscriptions

```
def get_subscriptions(self)
    Get subscription history via GraphQL.
```

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `user_id = self._get_user_id()`.
2. `if not user_id`: branches conditionally.
3. Assigns `gql = f'\n {{\n normalizedSubsByUser(userid: ....`
4. Assigns `result = self._graphql(gql)`.
5. `if not result`: branches conditionally.
6. `return result.get('data', {}).get('normalizedSubsByUser', {}).get('nodes', ....`

Code (copy-paste safe)

```
def get_subscriptions(self):
    """Get subscription history via GraphQL."""
    user_id = self._get_user_id()
    if not user_id:
        return []
    gql = f"""
    {{
      normalizedSubsByUser(userid: {user_id}, first: 50) {{
        nodes {{
          id
          source
          productName
          amount
          total
          status
          createdAt
        }}
      }}
    }}
    """
    result = self._graphql(gql)
    if not result:
        return []
    return (result.get('data', {}).
            .get('normalizedSubsByUser', {}).
            .get('nodes', []))

```

Method: TopStepAccount.get_all_accounts

```
def get_all_accounts(self)
```

Get all accounts from REST.

Why these Python concepts were used

- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.

Step by step (top-level body)

1. Assigns `accounts = self._fetch_ts('/me/accounts/basic', {'offset': '0', 'lim....`
2. `if accounts:` branches conditionally.
3. `return []`.

Code (copy-paste safe)

```
def get_all_accounts(self):
    """Get all accounts from REST."""
    accounts = self._fetch_ts('/me/accounts/basic', {
        'offset': '0', 'limit': '100',
        'sortBy': 'createdAt', 'sortOrder': 'desc'
    })
    if accounts:
        # Response: {accounts: [...], hasMore, total}
        items = accounts.get('accounts', accounts.get('data', []))
        if isinstance(items, list):
            return items
    return []
```

Method: TopStepAccount.get_payouts

```
def get_payouts(self)
```

Get payouts from REST.

Why these Python concepts were used

- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns `result = self._fetch_ts('/me/payouts/')`.
2. `if not result:` branches conditionally.
3. Assigns `items = result.get('data', result)` if `isinstance(result, dict)` el...
4. `return items` if `isinstance(items, list)` else `[]`.

Code (copy-paste safe)

```
def get_payouts(self):
    """Get payouts from REST."""
    result = self._fetch_ts('/me/payouts/')
    if not result:
        return []
    items = result.get('data', result) if isinstance(result, dict) else result
```

```
return items if isinstance(items, list) else []
```

```
class MFFUAccount(CDPBase)
```

MFFU dashboard scraper — Next.js app at myfundedfutures.com Auth: session cookies (credentials: include) API base: <https://api.myfundedfutures.com/api>

```
DOMAIN = 'myfundedfutures.com'
```

```
FIRM_NAME = 'MFFU'
```

```
API_BASE = 'https://api.myfundedfutures.com/api'
```

Code (copy-paste safe)

```
class MFFUAccount(CDPBase):
```

```
    """
```

```
    MFFU dashboard scraper — Next.js app at myfundedfutures.com
```

```
    Auth: session cookies (credentials: include)
```

```
    API base: https://api.myfundedfutures.com/api
```

```
    """
```

```
    DOMAIN = "myfundedfutures.com"
```

```
    FIRM_NAME = "MFFU"
```

```
    API_BASE = "https://api.myfundedfutures.com/api"
```

```
    def _get_profile(self):
```

```
        return self._fetch_json(f"{self.API_BASE}/getProfile/")
```

```
    def get_account_stats(self):
```

```
        """Get account stats from MFFU profile."""
```

```
        with self.lock:
```

```
            now = time.time()
```

```
            if now - self._stats_last_fetch < 2.0 and self._cached_stats:
```

```
                return self._cached_stats
```

```
        stats = {
```

```
            "Account Number": "Unknown",
```

```
            "Balance": "N/A",
```

```
            "Equity": "N/A",
```

```
            "Profit/Loss": "N/A",
```

```
            "Open Trades": "0",
```

```
            "Symbol": "",
```

```
            "Direction": "",
```

```
            "Drawdown": "N/A",
```

```
            "Profit Target": "N/A",
```

```
            "Phase": "N/A",
```

```
            "Status": "N/A",
```

```
        }
```

```
    try:
```

```
        profile = self._get_profile()
```

```
        if profile:
```

```
            # Profile: {business_id, id, username, email, defaultFuturesAccount: {account_name, b
```

```
            stats["Account Number"] = profile.get('username', profile.get('email', 'Unknown'))
```

```
            acct = profile.get('defaultFuturesAccount', profile.get('default_account', {}))
```

```
            if isinstance(acct, dict):
```

```
                stats["Account Number"] = acct.get('account_name', stats["Account Number"])
```

```
                bal = acct.get('balance')
```

```
                if isinstance(bal, (int, float)):
```

```
                    stats["Balance"] = f"${bal:,.2f}"
```

```
                drawdown = acct.get('max_drawdown_amount', acct.get('drawdown'))
```

```

        if isinstance(drawdown, (int, float)):
            stats["Drawdown"] = f"${drawdown:,.2f}"
        pnl = acct.get('pnl', acct.get('total_pnl'))
        if isinstance(pnl, (int, float)):
            stats["Profit/Loss"] = f"${pnl:,.2f}"
        starting = acct.get('starting_balance')
        target = acct.get('profit_target')
        if isinstance(target, (int, float)) and target:
            stats["Profit Target"] = f"${target:,.2f}"
        stats["Status"] = acct.get('status', 'N/A')
        stats["Phase"] = acct.get('stage', 'N/A')

    except Exception as e:
        self.logger.warning(f"[STATS] Error: {e}")

    self._cached_stats = stats
    self._stats_last_fetch = now
    return stats

def _fetch_post_empty(self, url):
    """POST with no body and no Content-Type (required by some MFFU endpoints)."""
    js = f"""
    (async () => {{
        const r = await fetch({json.dumps(url)}, {{
            method: 'POST',
            credentials: 'include'
        }});
        const text = await r.text();
        return JSON.stringify({{status: r.status, body: text}});
    }})()
    """
    raw = self._js(js, await_promise=True)
    if not raw:
        return None
    result = json.loads(raw)
    if result.get('status') != 200:
        self.logger.warning(f"[FETCH] POST {url} \u2192 HTTP {result.get('status')}")
        return None
    try:
        return json.loads(result['body'])
    except (json.JSONDecodeError, KeyError):
        return result.get('body')

def get_billing_history(self):
    """Get billing history: try DOM scrape first (has account numbers), then API fallback."""
    # DOM scrape is preferred because it includes MFFU account numbers
    # (e.g. MFFUEVSCL223761104) that the API endpoints do not return.
    dom_billing = self._scrape_billing_dom()
    if dom_billing:
        return dom_billing

    # Fallback: API endpoints (subscriptions + receipts)
    billing = []
    _APPROVED = ("active", "paid", "approved", "processed", "completed", "expired")

    subs = self._fetch_post_empty(f"{self.API_BASE}/getSubscriptions/")
    if subs:
        ok = subs.get('ok', subs) if isinstance(subs, dict) else subs
        items = ok.get('subscriptions', ok) if isinstance(ok, dict) else ok
        if isinstance(items, list):

```

```

        for item in items:
            billing.append({
                "sn": str(item.get("id", "")),
                "account_no": str(item.get("account_name", item.get("id", ""))),
                "status": "APPROVED" if str(item.get("status", "")).lower() in _APPROVED else str(
                    item.get("status", "")),
                "date": str(item.get("created_at", item.get("date", "")))[:10],
                "paid_amount": f"${item.get('price', item.get('amount', 0))}",
                "paid_amount_numeric": float(item.get("price", item.get("amount", 0)) or 0),
                "funding_package": item.get("plan_name", item.get("plan", item.get("name", ""))),
                "payment_method": item.get("payment_method", ""),
                "coupon": item.get("coupon_code", ""),
            })

receipts = self._fetch_post_empty(f"{self.API_BASE}/getReceipts/")
if receipts:
    items = receipts.get('ok', receipts) if isinstance(receipts, dict) else receipts
    if isinstance(items, list):
        for item in items:
            billing.append({
                "sn": str(item.get("order_number", item.get("id", ""))),
                "account_no": str(item.get("account_name", "")),
                "status": "APPROVED" if str(item.get("status", "")).lower() in _APPROVED else str(
                    item.get("status", "")),
                "date": str(item.get("created_at", item.get("date", "")))[:10],
                "paid_amount": f"${item.get('price_paid', item.get('amount', 0))}",
                "paid_amount_numeric": float(item.get("price_paid", item.get("amount", 0)) or 0),
                "funding_package": item.get("plan_name", item.get("plan", "")),
            })

return billing

def _scrape_billing_dom(self):
    """Scrape billing from the Prop Account Subscriptions table at /billing."""
    # Navigate to billing page
    current = self._js("window.location.href") or ""
    if "/billing" not in current:
        self._js("window.location.href = 'https://myfundedfutures.com/billing'")
        time.sleep(4)

    # Extract all rows from the billing table.
    # Columns: STARTED | ACCOUNT | RENEWS/EXPIRES | PRICE | COUPON | METHOD | STATUS | ACTIONS
    raw = self._js("""
        (function() {
            var rows = document.querySelectorAll('table tbody tr');
            if (rows.length === 0) {
                // Try alternate selectors for Next.js / Tailwind tables
                rows = document.querySelectorAll('[class*="billing"] tr, [class*="subscription"] tr');
            }
            var results = [];
            for (var i = 0; i < rows.length; i++) {
                var cells = rows[i].querySelectorAll('td');
                if (cells.length < 7) continue;
                results.push({
                    started: (cells[0] || {}).innerText || '',
                    account: (cells[1] || {}).innerText || '',
                    renews: (cells[2] || {}).innerText || '',
                    price: (cells[3] || {}).innerText || '',
                    coupon: (cells[4] || {}).innerText || '',
                    method: (cells[5] || {}).innerText || '',
                })
            }
        })
    """)

```

```

        status: (cells[6] || {}).innerText || ''
    });
}
return JSON.stringify(results);
})();
"""

if not raw:
    self.logger.warning("[BILLING] DOM scrape returned nothing")
    return []

try:
    items = json.loads(raw)
except (json.JSONDecodeError, TypeError):
    self.logger.warning("[BILLING] DOM scrape JSON parse failed")
    return []

billing = []
for item in items:
    # Parse account: may contain badge text + account number, e.g. "Scale50K\nMFFUEVSCL223761104"
    acct_text = item.get("account", "").strip()
    acct_lines = [l.strip() for l in acct_text.split("\n") if l.strip()]
    # Account number is typically the longest line starting with MFFU
    account_no = ""
    funding_package = ""
    for line in acct_lines:
        if line.upper().startswith("MFFU"):
            account_no = line
        elif not funding_package:
            funding_package = line # e.g. "Scale50K"

    # Parse price
    price_str = item.get("price", "").strip()
    price_numeric = 0.0
    try:
        price_numeric = float(price_str.replace("$", "").replace(",", "").strip())
    except (ValueError, AttributeError):
        # "Will not renew" or empty
        pass

    # Parse status – map to APPROVED for active/expired paid subscriptions
    status_raw = item.get("status", "").strip().lower()
    if status_raw in ("active", "expired") and price_numeric > 0:
        status = "APPROVED"
    elif status_raw == "cancelled":
        status = "CANCELLED"
    else:
        status = status_raw.upper()

    # Parse date: "Sep 29 2025" → "2025-09-29"
    date_str = item.get("started", "").strip()
    try:
        from datetime import datetime as _dt
        parsed = _dt.strptime(date_str, "%b %d %Y")
        date_str = parsed.strftime("%Y-%m-%d")
    except (ValueError, TypeError):
        pass # keep as-is

    if account_no and price_numeric > 0:
        billing.append({

```

```

        "sn": account_no,
        "account_no": account_no,
        "status": status,
        "date": date_str,
        "paid_amount": f"${price_numeric:.2f}",
        "paid_amount_numeric": price_numeric,
        "funding_package": funding_package,
        "payment_method": item.get("method", "").strip(),
        "coupon": item.get("coupon", "").strip(),
    })

    self.logger.info(f"[BILLING] DOM scrape got {len(billing)} record(s)")
    return billing

def get_account_mapping(self):
    """Build login_id → account info mapping from MFFU prop accounts.
    Returns dict of account_name → {tradovate_account_name, balance, starting_balance, breached, ...}
    """
    mapping = {}
    accounts = self.get_all_accounts()
    for acct in accounts:
        acct_name = acct.get("account_name", "")
        if not acct_name:
            continue
        balance = acct.get("balance", acct.get("current_balance"))
        starting = acct.get("starting_balance", acct.get("initial_balance"))
        status = acct.get("status", "")
        breached = status.lower() in ("breached", "failed", "blown") if status else False
        # Extract profit target and drawdown limit from the account object
        profit_target = acct.get("profit_target")
        max_dd = acct.get("max_drawdown_amount", acct.get("max_drawdown"))
        min_equity = None
        if starting is not None and max_dd is not None:
            try:
                min_equity = float(starting) - float(max_dd)
            except (ValueError, TypeError):
                pass
        mapping[acct_name] = {
            "tradovate_account_name": acct_name,
            "balance": balance,
            "starting_balance": starting,
            "breached": breached,
            "breachedby": acct.get("breached_by", acct.get("breach_reason", "")),
            "status": status,
            "account_id": acct.get("id", ""),
            "profit_target": profit_target,
            "min_equity": min_equity,
        }
    self.logger.info(f"[MAPPING] Got {len(mapping)} MFFU account(s)")
    return mapping

def get_all_accounts(self):
    """Get all prop accounts (paginated)."""
    all_accounts = []
    page = 1
    while True:
        result = self._fetch_json(
            f"{self.API_BASE}/user-prop-accounts/?page={page}&page_size=100")
        if not result:
            break

```

```

        # Response: {ok: {total: N, accounts: [...]}}
        ok = result.get('ok', result) if isinstance(result, dict) else result
        items = ok.get('accounts', ok.get('results', [])) if isinstance(ok, dict) else ok
        if isinstance(items, list):
            all_accounts.extend(items)
        # Check for next page
        total = ok.get('total', 0) if isinstance(ok, dict) else 0
        if len(all_accounts) >= total or not items:
            break
        page += 1
        if page > 10:
            break
    return all_accounts

def get_payouts(self):
    """Get past payouts (paginated)."""
    all_payouts = []
    page = 0
    while True:
        result = self._fetch_json(
            f"{self.API_BASE}/getPastPayouts/?page={page}&page_size=50")
        if not result:
            break
        ok = result.get('ok', result)
        if isinstance(ok, dict):
            data = ok.get('data', {})
            items = data.get('past_payouts', []) if isinstance(data, dict) else []
            all_payouts.extend(items)
            total = ok.get('total', 0)
            if len(all_payouts) >= total:
                break
        elif isinstance(ok, list):
            all_payouts.extend(ok)
            break
        else:
            break
        page += 1
        if page > 10: # Safety limit
            break
    return all_payouts

def get_available_payouts(self):
    """Get current available and upcoming payouts."""
    result = self._fetch_json(f"{self.API_BASE}/getUserPayoutsPage/")
    if not result:
        return {}
    return result.get('ok', result)

def get_account_categories(self):
    """Get account category metadata."""
    return self._fetch_json(f"{self.API_BASE}/user-prop-account-categories/")

```

Method: MFFUAccount._get_profile

```
def _get_profile(self)
```

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **return** self._fetch_json(f'{self.API_BASE}/getProfile/').

Code (copy-paste safe)

```
def _get_profile(self):
    return self._fetch_json(f'{self.API_BASE}/getProfile/')
```

Method: MFFUAccount.get_account_stats

```
def get_account_stats(self)
```

Get account stats from MFFU profile.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.

Step by step (top-level body)

1. **with** self.lock: enters a context manager.

Code (copy-paste safe)

```
def get_account_stats(self):
    """Get account stats from MFFU profile."""
    with self.lock:
        now = time.time()
        if now - self._stats_last_fetch < 2.0 and self._cached_stats:
            return self._cached_stats

    stats = {
        "Account Number": "Unknown",
        "Balance": "N/A",
        "Equity": "N/A",
        "Profit/Loss": "N/A",
        "Open Trades": "0",
        "Symbol": "",
        "Direction": "",
        "Drawdown": "N/A",
        "Profit Target": "N/A",
        "Phase": "N/A",
        "Status": "N/A",
    }

    try:
        profile = self._get_profile()
        if profile:
```

```

# Profile: {business_id, id, username, email, defaultFuturesAccount: {account_name, b
stats["Account Number"] = profile.get('username', profile.get('email', 'Unknown'))
acct = profile.get('defaultFuturesAccount', profile.get('default_account', {}))
if isinstance(acct, dict):
    stats["Account Number"] = acct.get('account_name', stats["Account Number"])
    bal = acct.get('balance')
    if isinstance(bal, (int, float)):
        stats["Balance"] = f"${bal:,.2f}"
    drawdown = acct.get('max_drawdown_amount', acct.get('drawdown'))
    if isinstance(drawdown, (int, float)):
        stats["Drawdown"] = f"${drawdown:,.2f}"
    pnl = acct.get('pnl', acct.get('total_pnl'))
    if isinstance(pnl, (int, float)):
        stats["Profit/Loss"] = f"${pnl:,.2f}"
    starting = acct.get('starting_balance')
    target = acct.get('profit_target')
    if isinstance(target, (int, float)) and target:
        stats["Profit Target"] = f"${target:,.2f}"
    stats["Status"] = acct.get('status', 'N/A')
    stats["Phase"] = acct.get('stage', 'N/A')

except Exception as e:
    self.logger.warning(f"[STATS] Error: {e}")

self._cached_stats = stats
self._stats_last_fetch = now
return stats

```

Method: MFFUAccount._fetch_post_empty

```
def _fetch_post_empty(self, url)
```

POST with no body and no Content-Type (required by some MFFU endpoints).

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `js = f"\n (async () => {{\n const r ....`
2. Assigns `raw = self._js(js, await_promise=True)`.
3. **if not raw**: branches conditionally.
4. Assigns `result = json.loads(raw)`.
5. **if result.get('status') != 200**: branches conditionally.
6. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def _fetch_post_empty(self, url):
    """POST with no body and no Content-Type (required by some MFFU endpoints)."""

```

```

js = f"""
    (async () => {{
        const r = await fetch({json.dumps(url)}, {{
            method: 'POST',
            credentials: 'include'
        }});
        const text = await r.text();
        return JSON.stringify({{status: r.status, body: text}});
    }}())
"""
raw = self._js(js, await_promise=True)
if not raw:
    return None
result = json.loads(raw)
if result.get('status') != 200:
    self.logger.warning(f"[FETCH] POST {url} \u2192 HTTP {result.get('status')}")
    return None
try:
    return json.loads(result['body'])
except (json.JSONDecodeError, KeyError):
    return result.get('body')

```

Method: MFFUAccount.get_billing_history

```
def get_billing_history(self)
```

Get billing history: try DOM scrape first (has account numbers), then API fallback.

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns `dom_billing = self._scrape_billing_dom()`.
2. **if** `dom_billing`: branches conditionally.
3. Assigns `billing = []`.
4. Assigns `_APPROVED = ('active', 'paid', 'approved', 'processed', 'completed', ....`
5. Assigns `subs = self._fetch_post_empty(f'{self.API_BASE}/getSubscriptions/')`.
6. **if** `subs`: branches conditionally.
7. Assigns `receipts = self._fetch_post_empty(f'{self.API_BASE}/getReceipts/')`.
8. **if** `receipts`: branches conditionally.
9. **return** `billing`.

Code (copy-paste safe)

```
def get_billing_history(self):
```

```

"""Get billing history: try DOM scrape first (has account numbers), then API fallback."""
# DOM scrape is preferred because it includes MFFU account numbers
# (e.g. MFFUEVSL223761104) that the API endpoints do not return.
dom_billing = self._scrape_billing_dom()
if dom_billing:
    return dom_billing

# Fallback: API endpoints (subscriptions + receipts)
billing = []
_APPROVED = ("active", "paid", "approved", "processed", "completed", "expired")

subs = self._fetch_post_empty(f"{self.API_BASE}/getSubscriptions/")
if subs:
    ok = subs.get('ok', subs) if isinstance(subs, dict) else subs
    items = ok.get('subscriptions', ok) if isinstance(ok, dict) else ok
    if isinstance(items, list):
        for item in items:
            billing.append({
                "sn": str(item.get("id", "")),
                "account_no": str(item.get("account_name", item.get("id", ""))),
                "status": "APPROVED" if str(item.get("status", "")).lower() in _APPROVED else str(
                    item.get("status", "")),
                "date": str(item.get("created_at", item.get("date", "")))[:10],
                "paid_amount": f"${item.get('price', item.get('amount', 0))}",
                "paid_amount_numeric": float(item.get("price", item.get("amount", 0)) or 0),
                "funding_package": item.get("plan_name", item.get("plan", item.get("name", ""))),
                "payment_method": item.get("payment_method", ""),
                "coupon": item.get("coupon_code", ""),
            })

receipts = self._fetch_post_empty(f"{self.API_BASE}/getReceipts/")
if receipts:
    items = receipts.get('ok', receipts) if isinstance(receipts, dict) else receipts
    if isinstance(items, list):
        for item in items:
            billing.append({
                "sn": str(item.get("order_number", item.get("id", ""))),
                "account_no": str(item.get("account_name", "")),
                "status": "APPROVED" if str(item.get("status", "")).lower() in _APPROVED else str(
                    item.get("status", "")),
                "date": str(item.get("created_at", item.get("date", "")))[:10],
                "paid_amount": f"${item.get('price_paid', item.get('amount', 0))}",
                "paid_amount_numeric": float(item.get("price_paid", item.get("amount", 0)) or 0),
                "funding_package": item.get("plan_name", item.get("plan", "")),
            })

return billing

```

Method: MFFUAccount._scrape_billing_dom

```
def _scrape_billing_dom(self)
```

Scrape billing from the Prop Account Subscriptions table at /billing.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with `append`, and clearer about intent: this is a transform, not a side-effecting loop.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `current = self._js('window.location.href')` or `""`.
2. `if '/billing'` not in `current`: branches conditionally.
3. Assigns `raw = self._js("\n (function() {\n va....`
4. `if` not `raw`: branches conditionally.
5. `try` block with 1 **except** clause.
6. Assigns `billing = []`.
7. `for` `item` in `items`: iterates.
8. Calls `self.logger.info(...)` for its side effect.
9. `return` `billing`.

Code (copy-paste safe)

```
def _scrape_billing_dom(self):
    """Scrape billing from the Prop Account Subscriptions table at /billing."""
    # Navigate to billing page
    current = self._js("window.location.href") or ""
    if "/billing" not in current:
        self._js("window.location.href = 'https://myfundedfutures.com/billing'")
        time.sleep(4)

    # Extract all rows from the billing table.
    # Columns: STARTED | ACCOUNT | RENEWS/EXPIRES | PRICE | COUPON | METHOD | STATUS | ACTIONS
    raw = self._js("""
    (function() {
        var rows = document.querySelectorAll('table tbody tr');
        if (rows.length === 0) {
            // Try alternate selectors for Next.js / Tailwind tables
            rows = document.querySelectorAll('[class*="billing"] tr, [class*="subscription"] tr');
        }
        var results = [];
        for (var i = 0; i < rows.length; i++) {
            var cells = rows[i].querySelectorAll('td');
            if (cells.length < 7) continue;
            results.push({
                started: (cells[0] || {}).innerText || '',
                account: (cells[1] || {}).innerText || '',
                renews: (cells[2] || {}).innerText || '',
                price: (cells[3] || {}).innerText || '',
                coupon: (cells[4] || {}).innerText || '',
                method: (cells[5] || {}).innerText || '',
                status: (cells[6] || {}).innerText || ''
            });
        }
        return JSON.stringify(results);
    })()
    """)
```

```

if not raw:
    self.logger.warning("[BILLING] DOM scrape returned nothing")
    return []

try:
    items = json.loads(raw)
except (json.JSONDecodeError, TypeError):
    self.logger.warning("[BILLING] DOM scrape JSON parse failed")
    return []

billing = []
for item in items:
    # Parse account: may contain badge text + account number, e.g. "Scale50K\nMFFUEVSC223761104"
    acct_text = item.get("account", "").strip()
    acct_lines = [l.strip() for l in acct_text.split("\n") if l.strip()]
    # Account number is typically the longest line starting with MFFU
    account_no = ""
    funding_package = ""
    for line in acct_lines:
        if line.upper().startswith("MFFU"):
            account_no = line
        elif not funding_package:
            funding_package = line # e.g. "Scale50K"

    # Parse price
    price_str = item.get("price", "").strip()
    price_numeric = 0.0
    try:
        price_numeric = float(price_str.replace("$", "").replace(",", "").strip())
    except (ValueError, AttributeError):
        # "Will not renew" or empty
        pass

    # Parse status – map to APPROVED for active/expired paid subscriptions
    status_raw = item.get("status", "").strip().lower()
    if status_raw in ("active", "expired") and price_numeric > 0:
        status = "APPROVED"
    elif status_raw == "cancelled":
        status = "CANCELLED"
    else:
        status = status_raw.upper()

    # Parse date: "Sep 29 2025" → "2025-09-29"
    date_str = item.get("started", "").strip()
    try:
        from datetime import datetime as _dt
        parsed = _dt.strptime(date_str, "%b %d %Y")
        date_str = parsed.strftime("%Y-%m-%d")
    except (ValueError, TypeError):
        pass # keep as-is

    if account_no and price_numeric > 0:
        billing.append({
            "sn": account_no,
            "account_no": account_no,
            "status": status,
            "date": date_str,
            "paid_amount": f"${price_numeric:.2f}",
            "paid_amount_numeric": price_numeric,
            "funding_package": funding_package,

```

```

        "payment_method": item.get("method", "").strip(),
        "coupon": item.get("coupon", "").strip(),
    })

    self.logger.info(f"[BILLING] DOM scrape got {len(billing)} record(s)")
    return billing

```

Method: MFFUAccount.get_account_mapping

```
def get_account_mapping(self)
```

Build login_id → account info mapping from MFFU prop accounts. Returns dict of account_name → {tradovate_account_name, balance, starting_balance, breached, ...}

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns mapping = {}.
2. Assigns accounts = self.get_all_accounts().
3. **for** acct in accounts: iterates.
4. Calls self.logger.info(...) for its side effect.
5. **return** mapping.

Code (copy-paste safe)

```

def get_account_mapping(self):
    """Build login_id → account info mapping from MFFU prop accounts.
    Returns dict of account_name → {tradovate_account_name, balance, starting_balance, breached, ...}
    """
    mapping = {}
    accounts = self.get_all_accounts()
    for acct in accounts:
        acct_name = acct.get("account_name", "")
        if not acct_name:
            continue
        balance = acct.get("balance", acct.get("current_balance"))
        starting = acct.get("starting_balance", acct.get("initial_balance"))
        status = acct.get("status", "")
        breached = status.lower() in ("breached", "failed", "blown") if status else False
        # Extract profit target and drawdown limit from the account object
        profit_target = acct.get("profit_target")
        max_dd = acct.get("max_drawdown_amount", acct.get("max_drawdown"))
        min_equity = None
        if starting is not None and max_dd is not None:
            try:
                min_equity = float(starting) - float(max_dd)

```

```

        except (ValueError, TypeError):
            pass
    mapping[acct_name] = {
        "tradovate_account_name": acct_name,
        "balance": balance,
        "starting_balance": starting,
        "breached": breached,
        "breachedby": acct.get("breached_by", acct.get("breach_reason", "")),
        "status": status,
        "account_id": acct.get("id", ""),
        "profit_target": profit_target,
        "min_equity": min_equity,
    }
    self.logger.info(f"[MAPPING] Got {len(mapping)} MFFU account(s)")
    return mapping

```

Method: MFFUAccount.get_all_accounts

```
def get_all_accounts(self)
    Get all prop accounts (paginated).
```

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns all_accounts = [].
2. Assigns page = 1.
3. **while** True: loops until the condition is false.
4. **return** all_accounts.

Code (copy-paste safe)

```

def get_all_accounts(self):
    """Get all prop accounts (paginated)."""
    all_accounts = []
    page = 1
    while True:
        result = self._fetch_json(
            f"{self.API_BASE}/user-prop-accounts/?page={page}&page_size=100")
        if not result:
            break
        # Response: {ok: {total: N, accounts: [...]}}
        ok = result.get('ok', result) if isinstance(result, dict) else result
        items = ok.get('accounts', ok.get('results', [])) if isinstance(ok, dict) else ok
        if isinstance(items, list):
            all_accounts.extend(items)
        # Check for next page
        total = ok.get('total', 0) if isinstance(ok, dict) else 0

```



```

        if len(all_accounts) >= total or not items:
            break
        page += 1
        if page > 10:
            break
    return all_accounts

```

Method: MFFUAccount.get_payouts

```

def get_payouts(self)
    Get past payouts (paginated).

```

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns `all_payouts = []`.
2. Assigns `page = 0`.
3. **while** `True`: loops until the condition is false.
4. **return** `all_payouts`.

Code (copy-paste safe)

```

def get_payouts(self):
    """Get past payouts (paginated)."""
    all_payouts = []
    page = 0
    while True:
        result = self._fetch_json(
            f"{self.API_BASE}/getPastPayouts/?page={page}&page_size=50")
        if not result:
            break
        ok = result.get('ok', result)
        if isinstance(ok, dict):
            data = ok.get('data', {})
            items = data.get('past_payouts', []) if isinstance(data, dict) else []
            all_payouts.extend(items)
            total = ok.get('total', 0)
            if len(all_payouts) >= total:
                break
        elif isinstance(ok, list):
            all_payouts.extend(ok)
            break
        else:
            break
        page += 1
        if page > 10: # Safety limit
            break

```

```
return all_payouts
```

Method: MFFUAccount.get_available_payouts

```
def get_available_payouts(self)
```

Get current available and upcoming payouts.

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `result = self._fetch_json(f'{self.API_BASE}/getUserPayoutsPage/')`.
2. `if not result:` branches conditionally.
3. `return result.get('ok', result)`.

Code (copy-paste safe)

```
def get_available_payouts(self):
    """Get current available and upcoming payouts."""
    result = self._fetch_json(f'{self.API_BASE}/getUserPayoutsPage/')
    if not result:
        return {}
    return result.get('ok', result)
```

Method: MFFUAccount.get_account_categories

```
def get_account_categories(self)
```

Get account category metadata.

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. `return self._fetch_json(f'{self.API_BASE}/user-prop-account-categories/')`.

Code (copy-paste safe)

```
def get_account_categories(self):
    """Get account category metadata."""
    return self._fetch_json(f'{self.API_BASE}/user-prop-account-categories/')
```

```
class FundedNextCDPAccount(CDPBase)
```

FundedNext dashboard scraper — CDP-based (no Selenium). Connects to <https://app.fundednext.com> via Chrome DevTools Protocol. Auth: tokenV1 cookie → Bearer token for API calls.

```
DOMAIN = 'fundednext.com'
FIRM_NAME = 'FundedNext'
API_BASE = 'https://api.fundednext.com/api/v1'
```

Code (copy-paste safe)

```
class FundedNextCDPAccount(CDPBase):
    """
    FundedNext dashboard scraper – CDP-based (no Selenium).
    Connects to https://app.fundednext.com via Chrome DevTools Protocol.
    Auth: tokenV1 cookie → Bearer token for API calls.
    """
    DOMAIN = "fundednext.com"
    FIRM_NAME = "FundedNext"
    API_BASE = "https://api.fundednext.com/api/v1"

    def __init__(self, username=None, debug_port=9222, pair_id="default"):
        super().__init__(debug_port=debug_port, pair_id=pair_id)
        self.username = username or ""

    # ■■ Auth helpers ■■

    def _get_token(self):
        """Extract tokenV1 from browser cookies."""
        return self._js("""
            (function() {
                var c = document.cookie.split(';').find(function(c) {
                    return c.trim().indexOf('tokenV1=') === 0;
                });
                return c ? decodeURIComponent(c.split('=')[1]) : null;
            })()
        """)

    def _get_email(self):
        """Get user email from localStorage or fallback to self.username."""
        if self.username:
            return self.username
        email = self._js("""
            (function() {
                try {
                    var u = JSON.parse(localStorage.getItem('user') || '{}');
                    return u.email || '';
                } catch(e) { return ''; }
            })()
        """)
        return email or ""

    # ■■ Navigation helpers ■■

    def _navigate_to(self, path):
        """Navigate to a FundedNext page by path."""
        current = self._js("window.location.href") or ""
        if path in current:
            return True
        self._js(f"window.location.href = 'https://app.fundednext.com{path}'")
        time.sleep(3)
        return True

    def _switch_type_tab(self, tab_name="Futures"):
        """Click a type tab (Futures / CFDs) via ant-tabs."""
        result = self._js(f"""
            (function() {{
                var tabs = document.querySelectorAll('.ant-tabs-tab-btn');
                for (var i = 0; i < tabs.length; i++) {{

```

```

        if (tabs[i].textContent.trim() === '{tab_name}') {{
            tabs[i].click();
            return 'clicked';
        }}
    }}
    return 'not_found';
}})()
"""
)
if result == "clicked":
    time.sleep(3)
    return True
return False

def _switch_status_tab(self, status="Active"):
    """Click a status tab (Active/Inactive/Breached)."""
    result = self._js(f"""
        (function() {{
            var btns = document.querySelectorAll('.account-wrapper__create-account button');
            for (var i = 0; i < btns.length; i++) {{
                if (btns[i].textContent.trim() === '{status}') {{
                    btns[i].click();
                    return 'clicked';
                }}
            }}
            return 'not_found';
        }})()
    """)
    if result == "clicked":
        time.sleep(2)
        return True
    return False

def _has_accounts(self):
    """Check if current tab view has accounts."""
    return not self._js("""
        (function() {
            var el = document.querySelector('.no-account-wrapper');
            return el && el.offsetParent !== null;
        })()
    """)

# ■■■ Standard Interface ■■■

def get_account_stats(self):
    """Get FundedNext account stats from the dashboard DOM."""
    with self.lock:
        now = time.time()
        if (now - self._stats_last_fetch) < 2.0 and self._cached_stats:
            return self._cached_stats

        if not self.is_connected():
            return self._default_stats("Not Connected")

        self._navigate_to("/accounts")

        stats = self._default_stats("Unknown")

        # Extract all card data in one JS call
        card_data = self._js("""
            (function() {

```

```

        var result = {account: '', balance: '', equity: '', serverType: '',
                      accountType: '', challenge: '', size: ''};
        // Account number from .dashboard-card h3
        var h3s = document.querySelectorAll('.dashboard-card h3');
        for (var i = 0; i < h3s.length; i++) {
            var text = h3s[i].textContent;
            var m = text.match(/FNFT\\w+/);
            if (m) {
                result.account = m[0];
                var parts = text.match(/^(.+?)\\s*\\|\\s*(\\w+)\\s*\\|\\/);
                if (parts) { result.challenge = parts[1].trim(); result.size = parts[2].trim();
                    break;
                }
            }
        }
        // Balance, Equity, Server Type, Account Type from .active-account-card p
        var ps = document.querySelectorAll('.active-account-card p');
        for (var j = 0; j < ps.length; j++) {
            var t = ps[j].textContent.trim();
            if (t.indexOf('Balance:') === 0) result.balance = t.split(':')[1].trim();
            else if (t.indexOf('Equity:') === 0) result.equity = t.split(':')[1].trim();
            else if (t.indexOf('Server Type:') === 0) result.serverType = t.split(':')[1].trim();
            else if (t.indexOf('Account Type:') === 0) result.accountType = t.split(':')[1].trim();
        }
        return JSON.stringify(result);
    })()
    """)

    if card_data:
        try:
            d = json.loads(card_data)
            if d.get("account"):
                stats["Account Number"] = d["account"]
            if d.get("balance"):
                stats["Balance"] = d["balance"]
            if d.get("equity"):
                stats["Equity"] = d["equity"]
            if d.get("challenge"):
                stats["Phase"] = d["challenge"]
            if d.get("size"):
                stats["Size"] = d["size"]
            if d.get("accountType"):
                stats["Status"] = d["accountType"]
        except (json.JSONDecodeError, TypeError):
            pass

    self._cached_stats = stats
    self._stats_last_fetch = now
    return stats

def _default_stats(self, account_text="N/A"):
    return {
        "Account Number": account_text,
        "Balance": "N/A",
        "Equity": "N/A",
        "Profit/Loss": "N/A",
        "Open Trades": "0",
        "Symbol": "",
        "Direction": "",
        "Drawdown": "N/A",
    }

```

```

        "Profit Target": "N/A",
        "Phase": "N/A",
        "Status": "N/A",
    }

def get_all_accounts(self):
    """Get all accounts from dashboard cards (Futures + CFDs, Active)."""
    if not self.is_connected():
        return []

    with self.lock:
        self._navigate_to("/accounts")
        time.sleep(2)
        accounts = []

        for type_tab in ["Futures", "CFDs"]:
            self._switch_type_tab(type_tab)
            self._switch_status_tab("Active")
            time.sleep(1)

            if not self._has_accounts():
                continue

        # Parse all .dashboard-card elements in a single JS call
        raw = self._js("""
            (function() {
                var cards = document.querySelectorAll('.dashboard-card');
                var results = [];
                for (var i = 0; i < cards.length; i++) {
                    var text = cards[i].textContent;
                    if (!text.trim()) continue;
                    var acct = {account: '', balance: '', equity: '',
                                serverType: '', accountType: '', challenge: '', size: ''};
                    var m = text.match(/FNFT\\w+/);
                    if (m) acct.account = m[0];
                    var parts = text.match(/^(.+?)\\s*\\|\\s*(\\w+)\\s*\\|\\/);
                    if (parts) { acct.challenge = parts[1].trim(); acct.size = parts[2].trim(); }
                    var lines = text.split('\\n');
                    for (var j = 0; j < lines.length; j++) {
                        var l = lines[j].trim();
                        if (l.indexOf('Balance:') === 0) acct.balance = l.split(':')[1].trim();
                        else if (l.indexOf('Equity:') === 0) acct.equity = l.split(':')[1].trim();
                        else if (l.indexOf('Server Type:') === 0) acct.serverType = l.split(':')[1].trim();
                        else if (l.indexOf('Account Type:') === 0) acct.accountType = l.split(':')[1].trim();
                    }
                    results.push(acct);
                }
                return JSON.stringify(results);
            })()
        """)

        if raw:
            try:
                parsed = json.loads(raw)
                for item in parsed:
                    stats = self._default_stats(item.get("account", "Unknown"))
                    stats["Balance"] = item.get("balance", "N/A")
                    stats["Equity"] = item.get("equity", "N/A")

```

```

        stats["Phase"] = item.get("challenge", "N/A")
        stats["Size"] = item.get("size", "N/A")
        stats["Status"] = item.get("accountType", "N/A")
        stats["Server Type"] = item.get("serverType", "N/A")
        stats["Type"] = type_tab
        accounts.append(stats)
    except (json.JSONDecodeError, TypeError):
        pass

    self.logger.info(f"[ACCOUNTS] Extracted {len(accounts)} accounts")
    return accounts

def get_billing_history(self):
    """Fetch billing via FundedNext REST API, with DOM scrape fallback."""
    token = self._get_token()
    if not token:
        self.logger.warning("[BILLING] No tokenV1 cookie found")
        return self._scrape_billing_dom()

    email = self._get_email()
    if not email:
        self.logger.warning("[BILLING] No email available")
        return self._scrape_billing_dom()

    url = f"{self.API_BASE}/pending-payment-history?email={email}&type=1&account_id=&page=1&limit=20"
    data = self._fetch_json_bearer(url, token)
    if not data:
        self.logger.info("[BILLING] API returned empty – falling back to DOM scrape")
        return self._scrape_billing_dom()

    items = data.get("data", {}).get("data", []) if isinstance(data.get("data"), dict) else data.get("data", [])
    if not items:
        self.logger.info("[BILLING] API data empty – falling back to DOM scrape")
        return self._scrape_billing_dom()

    billing = []
    for item in items:
        billing.append({
            "sn": str(item.get("id", "")),
            "account_no": str(item.get("login", "")),
            "payment_method": item.get("payment_method", ""),
            "invoice": item.get("invoice_path", ""),
            "status": "APPROVED" if item.get("status") == 1 else str(item.get("status", "")),
            "date": (item.get("created_at") or "")[:10],
            "transaction_id": item.get("transaction_id", ""),
            "transition_type": item.get("payments_for", ""),
            "paid_amount": f"${item.get('paid_amount', 0):.2f}",
            "funding_package": item.get("funding_package", ""),
            "paid_amount_numeric": float(item.get("paid_amount", 0) or 0),
            "login": item.get("login"),
        })

    self.logger.info(f"[BILLING] Got {len(billing)} records via API")
    return billing

def _scrape_billing_dom(self):
    """Scrape billing history from the DOM table on /billing/billing-history."""
    self._navigate_to("/billing/billing-history")
    time.sleep(3)

```

```

raw = self._js("""
(function() {
    var rows = document.querySelectorAll('.ant-table-wrapper table tbody tr.ant-table-row');
    if (rows.length === 0) {
        rows = document.querySelectorAll('table tbody tr');
    }
    var results = [];
    for (var i = 0; i < rows.length; i++) {
        var cells = rows[i].querySelectorAll('td');
        if (cells.length < 9) continue;
        results.push({
            sn: cells[0].innerText.trim(),
            account_no: cells[1].innerText.trim(),
            payment_method: cells[2].innerText.trim(),
            status: cells[4].innerText.trim(),
            date: cells[5].innerText.trim(),
            transaction_id: cells[6].innerText.trim(),
            transition_type: cells[7].innerText.trim(),
            paid_amount: cells[8].innerText.trim(),
            funding_package: cells[9] ? cells[9].innerText.trim() : ''
        });
    }
    return JSON.stringify(results);
})();
""")

if not raw:
    return []

try:
    items = json.loads(raw)
except (json.JSONDecodeError, TypeError):
    return []

billing = []
for item in items:
    amount_str = item.get("paid_amount", "0")
    amount_num = 0.0
    try:
        amount_num = float(amount_str.replace("$", "").replace(",", "").strip())
    except (ValueError, AttributeError):
        pass

    billing.append({
        "sn": item.get("sn", ""),
        "account_no": item.get("account_no", ""),
        "payment_method": item.get("payment_method", ""),
        "invoice": "",
        "status": item.get("status", "").upper(),
        "date": item.get("date", ""),
        "transaction_id": item.get("transaction_id", ""),
        "transition_type": item.get("transition_type", ""),
        "paid_amount": f"${amount_num:.2f}" if amount_num else amount_str,
        "funding_package": item.get("funding_package", ""),
        "paid_amount_numeric": amount_num,
        "login": item.get("account_no", "")
    })

self.logger.info(f"[BILLING] Got {len(billing)} records via DOM scrape")
return billing

```



```

def get_payouts(self):
    """FundedNext doesn't have a separate payouts endpoint – included in billing."""
    return []

def get_account_mapping(self):
    """
    Get mapping from FundedNext login ID to Tradovate account name
    by extracting React fiber props from .dashboard-card elements.
    """
    self._navigate_to("/accounts")
    time.sleep(2)
    self._switch_type_tab("Futures")
    time.sleep(2)

    # Check for page error
    body_text = self._js("document.body.innerText.substring(0, 500)") or ""
    if "Something Went Wrong" in body_text:
        self.logger.warning("[MAPPING] Futures tab returned error page")
        return {}

    raw = self._js("""
    (function() {
        var cards = document.querySelectorAll('.dashboard-card');
        var results = [];
        for (var i = 0; i < cards.length; i++) {
            var keys = Object.keys(cards[i]);
            for (var j = 0; j < keys.length; j++) {
                if (keys[j].indexOf('__reactFiber') !== -1) {
                    var node = cards[i][keys[j]];
                    for (var k = 0; k < 30 && node; k++) {
                        var p = node.memoizedProps;
                        if (p && p.account && p.account.login) {
                            var acct = p.account;
                            results.push({
                                login: acct.login,
                                account_id: acct.id,
                                tradovate_name: (acct.tradovate_account_name || {
                                    }).tradovate_account_name || null,
                                plan_title: (acct.plan || {}).title || null,
                                balance: acct.balance,
                                starting_balance: acct.starting_balance,
                                server_type: (acct.server || {}).server_type || null,
                                breached: acct.breached,
                                breachedby: acct.breachedby || null,
                                profit_target: acct.profit_target || acct.target || null,
                                drawdown: acct.drawdown || acct.max_drawdown || null
                            });
                        }
                        break;
                    }
                    node = node.return;
                }
            }
            break;
        }
    })()
    """)

    mapping = {}

```

```

if raw:
    try:
        accounts = json.loads(raw)
        for acct in accounts:
            login_id = acct.get("login")
            if login_id:
                starting = acct.get("starting_balance")
                dd = acct.get("drawdown")
                min_equity = None
                if starting is not None and dd is not None:
                    try:
                        min_equity = float(starting) - float(dd)
                    except (ValueError, TypeError):
                        pass
                mapping[str(login_id)] = {
                    "tradovate_account_name": acct.get("tradovate_name"),
                    "account_id": acct.get("account_id"),
                    "plan_title": acct.get("plan_title"),
                    "balance": acct.get("balance"),
                    "starting_balance": starting,
                    "server_type": acct.get("server_type"),
                    "breached": acct.get("breached"),
                    "breachedby": acct.get("breachedby"),
                    "profit_target": acct.get("profit_target"),
                    "min_equity": min_equity,
                }
                self.logger.info(
                    f"[MAPPING] login={login_id} -> tradovate={acct.get('tradovate_name')} "
                    f"({acct.get('plan_title')}) target={acct.get('profit_target')} min_eq={min_e
            except (json.JSONDecodeError, TypeError):
                pass

        self.logger.info(f"[MAPPING] Built mapping for {len(mapping)} account(s)")
        return mapping

def get_account_overview(self, account_id):
    """Get detailed account overview via the FundedNext API."""
    token = self._get_token()
    if not token:
        self.logger.warning("[OVERVIEW] No tokenV1 cookie found")
        return None

    data = self._fetch_json_bearer(
        f"{self.API_BASE}/account-overview?account_id={account_id}", token)
    if not data:
        return None

    overview = data.get("data", {})
    if not overview:
        self.logger.warning(f"[OVERVIEW] Empty data for account_id={account_id}")
        return None

    acct_details = overview.get("account_details", {})
    self.logger.info(
        f"[OVERVIEW] account_id={account_id} | "
        f"name={acct_details.get('account_name')} | "
        f"breached={acct_details.get('breached')}")
    return overview

```

Method: FundedNextCDPAccount.__init__

```
def __init__(self, username=None, debug_port=9222, pair_id='default')
```

Why these Python concepts were used

- **__init__** — The constructor. Runs after Python has allocated the instance; assigns starting attribute values to self.

Step by step (top-level body)

1. Calls `super().__init__(...)` for its side effect.
2. Assigns `self.username = username or ""`.

Code (copy-paste safe)

```
def __init__(self, username=None, debug_port=9222, pair_id="default"):
    super().__init__(debug_port=debug_port, pair_id=pair_id)
    self.username = username or ""
```

Method: FundedNextCDPAccount._get_token

```
def _get_token(self)

    Extract tokenV1 from browser cookies.
```

Step by step (top-level body)

1. **return** `self._js("\n (function() {\n var c = docu....`

Code (copy-paste safe)

```
def _get_token(self):
    """Extract tokenV1 from browser cookies."""
    return self._js("""
        (function() {
            var c = document.cookie.split(';').find(function(c) {
                return c.trim().indexOf('tokenV1=') === 0;
            });
            return c ? decodeURIComponent(c.split('=')[1]) : null;
        })()
    """)
```

Method: FundedNextCDPAccount._get_email

```
def _get_email(self)

    Get user email from localStorage or fallback to self.username.
```

Step by step (top-level body)

1. **if** `self.username`: branches conditionally.
2. Assigns `email = self._js("\n (function() {\n tr....`
3. **return** `email or ""`.

Code (copy-paste safe)

```
def _get_email(self):
    """Get user email from localStorage or fallback to self.username."""
    if self.username:
        return self.username
```

```

email = self._js("""
    (function() {
        try {
            var u = JSON.parse(localStorage.getItem('user') || '{}');
            return u.email || '';
        } catch(e) { return ''; }
    })()
    """)
return email or ""

```

Method: FundedNextCDPAccount._navigate_to

```
def _navigate_to(self, path)
```

Navigate to a FundedNext page by path.

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `current = self._js('window.location.href')` or `''`.
2. **if** `path` in `current`: branches conditionally.
3. Calls `self._js(...)` for its side effect.
4. Calls `time.sleep(...)` for its side effect.
5. **return** `True`.

Code (copy-paste safe)

```

def _navigate_to(self, path):
    """Navigate to a FundedNext page by path."""
    current = self._js("window.location.href") or ""
    if path in current:
        return True
    self._js(f"window.location.href = 'https://app.fundednext.com{path}'")
    time.sleep(3)
    return True

```

Method: FundedNextCDPAccount._switch_type_tab

```
def _switch_type_tab(self, tab_name='Futures')
```

Click a type tab (Futures / CFDs) via ant-tabs.

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `result = self._js(f"\n (function() {{\n ....`
2. **if** `result == 'clicked'`: branches conditionally.

3. **return** False.

Code (copy-paste safe)

```
def _switch_type_tab(self, tab_name="Futures"):
    """Click a type tab (Futures / CFDs) via ant-tabs."""
    result = self._js(f"""
        (function() {{
            var tabs = document.querySelectorAll('.ant-tabs-tab-btn');
            for (var i = 0; i < tabs.length; i++) {{
                if (tabs[i].textContent.trim() === '{tab_name}') {{
                    tabs[i].click();
                    return 'clicked';
                }}
            }}
            return 'not_found';
        }})()
    """)
    if result == "clicked":
        time.sleep(3)
        return True
    return False
```

Method: FundedNextCDPAccount._switch_status_tab

```
def _switch_status_tab(self, status='Active')
```

Click a status tab (Active/Inactive/Breached).

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `result = self._js(f"\n (function() {{\n ....`
2. `if result == 'clicked':` branches conditionally.
3. **return** False.

Code (copy-paste safe)

```
def _switch_status_tab(self, status="Active"):
    """Click a status tab (Active/Inactive/Breached)."""
    result = self._js(f"""
        (function() {{
            var btns = document.querySelectorAll('.account-wrapper__create-account button');
            for (var i = 0; i < btns.length; i++) {{
                if (btns[i].textContent.trim() === '{status}') {{
                    btns[i].click();
                    return 'clicked';
                }}
            }}
            return 'not_found';
        }})()
    """)
    if result == "clicked":
        time.sleep(2)
        return True
```

```
return False
```

Method: FundedNextCDPAccount._has_accounts

```
def _has_accounts(self)
```

Check if current tab view has accounts.

Step by step (top-level body)

1. **return** not self._js("\n (function() {\n var el =...

Code (copy-paste safe)

```
def _has_accounts(self):
    """Check if current tab view has accounts."""
    return not self._js("""
        (function() {
            var el = document.querySelector('.no-account-wrapper');
            return el && el.offsetParent !== null;
        })()
    """)
```

Method: FundedNextCDPAccount.get_account_stats

```
def get_account_stats(self)
```

Get FundedNext account stats from the dashboard DOM.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.

Step by step (top-level body)

1. **with** self.lock: enters a context manager.

Code (copy-paste safe)

```
def get_account_stats(self):
    """Get FundedNext account stats from the dashboard DOM."""
    with self.lock:
        now = time.time()
        if (now - self._stats_last_fetch) < 2.0 and self._cached_stats:
            return self._cached_stats

        if not self.is_connected():
            return self._default_stats("Not Connected")

        self._navigate_to("/accounts")

        stats = self._default_stats("Unknown")

        # Extract all card data in one JS call
        card_data = self._js("""
```

```

(function() {
    var result = {account: '', balance: '', equity: '', serverType: '',
                  accountType: '', challenge: '', size: ''};
    // Account number from .dashboard-card h3
    var h3s = document.querySelectorAll('.dashboard-card h3');
    for (var i = 0; i < h3s.length; i++) {
        var text = h3s[i].textContent;
        var m = text.match(/FNFT\\w+/);
        if (m) {
            result.account = m[0];
            var parts = text.match(/^(.+?)\\s*\\|\\s*(\\w+)\\s*\\|/);
            if (parts) { result.challenge = parts[1].trim(); result.size = parts[2].trim();
                        break;
            }
        }
    }
    // Balance, Equity, Server Type, Account Type from .active-account-card p
    var ps = document.querySelectorAll('.active-account-card p');
    for (var j = 0; j < ps.length; j++) {
        var t = ps[j].textContent.trim();
        if (t.indexOf('Balance:') === 0) result.balance = t.split(':')[1].trim();
        else if (t.indexOf('Equity:') === 0) result.equity = t.split(':')[1].trim();
        else if (t.indexOf('Server Type:') === 0) result.serverType = t.split(':')[1].trim();
        else if (t.indexOf('Account Type:') === 0) result.accountType = t.split(':')[1].trim();
    }
    return JSON.stringify(result);
})();

"""

if card_data:
    try:
        d = json.loads(card_data)
        if d.get("account"):
            stats["Account Number"] = d["account"]
        if d.get("balance"):
            stats["Balance"] = d["balance"]
        if d.get("equity"):
            stats["Equity"] = d["equity"]
        if d.get("challenge"):
            stats["Phase"] = d["challenge"]
        if d.get("size"):
            stats["Size"] = d["size"]
        if d.get("accountType"):
            stats["Status"] = d["accountType"]
    except (json.JSONDecodeError, TypeError):
        pass

self._cached_stats = stats
self._stats_last_fetch = now
return stats

```

Method: FundedNextCDPAccount._default_stats

```
def _default_stats(self, account_text='N/A')
```

Step by step (top-level body)

1. return {'Account Number': account_text, 'Balance': 'N/A', 'Equity': 'N/A',....

Code (copy-paste safe)

```
def _default_stats(self, account_text="N/A"):
    return {
        "Account Number": account_text,
        "Balance": "N/A",
        "Equity": "N/A",
        "Profit/Loss": "N/A",
        "Open Trades": "0",
        "Symbol": "",
        "Direction": "",
        "Drawdown": "N/A",
        "Profit Target": "N/A",
        "Phase": "N/A",
        "Status": "N/A",
    }
```

Method: `FundedNextCDPAccount.get_all_accounts`

```
def get_all_accounts(self)
```

Get all accounts from dashboard cards (Futures + CFDs, Active).

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **if** `not self.is_connected()`: branches conditionally.
2. **with** `self.lock`: enters a context manager.

Code (copy-paste safe)

```
def get_all_accounts(self):
    """Get all accounts from dashboard cards (Futures + CFDs, Active)."""
    if not self.is_connected():
        return []

    with self.lock:
        self._navigate_to("/accounts")
        time.sleep(2)
        accounts = []

        for type_tab in ["Futures", "CFDs"]:
            self._switch_type_tab(type_tab)
            self._switch_status_tab("Active")
            time.sleep(1)

            if not self._has_accounts():
                continue

        # Parse all .dashboard-card elements in a single JS call
```



```

raw = self._js("""
(function() {
    var cards = document.querySelectorAll('.dashboard-card');
    var results = [];
    for (var i = 0; i < cards.length; i++) {
        var text = cards[i].textContent;
        if (!text.trim()) continue;
        var acct = {account: '', balance: '', equity: '',
                    serverType: '', accountType: '', challenge: '', size: ''};
        var m = text.match(/FNFT\\w+/);
        if (m) acct.account = m[0];
        var parts = text.match(/^(.+?)\\s*\\|\\s*(\\w+)\\s*\\|/);
        if (parts) { acct.challenge = parts[1].trim(); acct.size = parts[2].trim(); }
        var lines = text.split('\\n');
        for (var j = 0; j < lines.length; j++) {
            var l = lines[j].trim();
            if (l.indexOf('Balance:') === 0) acct.balance = l.split(':')[1].trim();
            else if (l.indexOf('Equity:') === 0) acct.equity = l.split(':')[1].trim();
            else if (l.indexOf('Server Type:') === 0) acct.serverType = l.split(':')[1].trim();
            else if (l.indexOf('Account Type:') === 0) acct.accountType = l.split(':')[1].trim();
        }
        results.push(acct);
    }
    return JSON.stringify(results);
})();
""")

if raw:
    try:
        parsed = json.loads(raw)
        for item in parsed:
            stats = self._default_stats(item.get("account", "Unknown"))
            stats["Balance"] = item.get("balance", "N/A")
            stats["Equity"] = item.get("equity", "N/A")
            stats["Phase"] = item.get("challenge", "N/A")
            stats["Size"] = item.get("size", "N/A")
            stats["Status"] = item.get("accountType", "N/A")
            stats["Server Type"] = item.get("serverType", "N/A")
            stats["Type"] = type_tab
            accounts.append(stats)
        except (json.JSONDecodeError, TypeError):
            pass

    self.logger.info(f"[ACCOUNTS] Extracted {len(accounts)} accounts")
    return accounts

```

Method: FundedNextCDPAccount.get_billing_history

```
def get_billing_history(self)
```

Fetch billing via FundedNext REST API, with DOM scrape fallback.

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns token = self._get_token().
2. if not token: branches conditionally.
3. Assigns email = self._get_email().
4. if not email: branches conditionally.
5. Assigns url = f'{self.API_BASE}/pending-payment-history?email={email}&t...
6. Assigns data = self._fetch_json_bearer(url, token).
7. if not data: branches conditionally.
8. Assigns items = data.get('data', {}).get('data', []) if isinstance(data.g....
9. if not items: branches conditionally.
10. Assigns billing = [].
11. for item in items: iterates.
12. Calls self.logger.info(...) for its side effect.
13. return billing.

Code (copy-paste safe)

```
def get_billing_history(self):
    """Fetch billing via FundedNext REST API, with DOM scrape fallback."""
    token = self._get_token()
    if not token:
        self.logger.warning("[BILLING] No tokenV1 cookie found")
        return self._scrape_billing_dom()

    email = self._get_email()
    if not email:
        self.logger.warning("[BILLING] No email available")
        return self._scrape_billing_dom()

    url = f"{self.API_BASE}/pending-payment-history?email={email}&type=1&account_id=&page=1&limit=20"
    data = self._fetch_json_bearer(url, token)
    if not data:
        self.logger.info("[BILLING] API returned empty – falling back to DOM scrape")
        return self._scrape_billing_dom()

    items = data.get("data", {}).get("data", []) if isinstance(data.get("data"), dict) else data.get(
        "data", [])
    if not items:
        self.logger.info("[BILLING] API data empty – falling back to DOM scrape")
        return self._scrape_billing_dom()

    billing = []
    for item in items:
        billing.append({
            "sn": str(item.get("id", "")),
```

```

        "account_no": str(item.get("login", "")),
        "payment_method": item.get("payment_method", ""),
        "invoice": item.get("invoice_path", ""),
        "status": "APPROVED" if item.get("status") == 1 else str(item.get("status", "")),
        "date": (item.get("created_at") or "")[:10],
        "transaction_id": item.get("transaction_id", ""),
        "transition_type": item.get("payments_for", ""),
        "paid_amount": f"${item.get('paid_amount', 0):.2f}",
        "funding_package": item.get("funding_package", ""),
        "paid_amount_numeric": float(item.get("paid_amount", 0) or 0),
        "login": item.get("login"),
    })

self.logger.info(f"[BILLING] Got {len(billing)} records via API")
return billing

```

Method: FundedNextCDPAccount._scrape_billing_dom

```
def _scrape_billing_dom(self)
```

Scrape billing history from the DOM table on /billing/billing-history.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Calls `self._navigate_to(...)` for its side effect.
2. Calls `time.sleep(...)` for its side effect.
3. Assigns `raw = self._js("\n (function() {\n va....`
4. **if** not `raw`: branches conditionally.
5. **try** block with 1 **except** clause.
6. Assigns `billing = []`.
7. **for** `item in items`: iterates.
8. Calls `self.logger.info(...)` for its side effect.
9. **return** `billing`.

Code (copy-paste safe)

```

def _scrape_billing_dom(self):
    """Scrape billing history from the DOM table on /billing/billing-history."""
    self._navigate_to("/billing/billing-history")
    time.sleep(3)

    raw = self._js("""

```

```

(function() {
    var rows = document.querySelectorAll('.ant-table-wrapper table tbody tr.ant-table-row');
    if (rows.length === 0) {
        rows = document.querySelectorAll('table tbody tr');
    }
    var results = [];
    for (var i = 0; i < rows.length; i++) {
        var cells = rows[i].querySelectorAll('td');
        if (cells.length < 9) continue;
        results.push({
            sn: cells[0].innerText.trim(),
            account_no: cells[1].innerText.trim(),
            payment_method: cells[2].innerText.trim(),
            status: cells[4].innerText.trim(),
            date: cells[5].innerText.trim(),
            transaction_id: cells[6].innerText.trim(),
            transition_type: cells[7].innerText.trim(),
            paid_amount: cells[8].innerText.trim(),
            funding_package: cells[9] ? cells[9].innerText.trim() : ''
        });
    }
    return JSON.stringify(results);
})();
""")

if not raw:
    return []

try:
    items = json.loads(raw)
except (json.JSONDecodeError, TypeError):
    return []

billing = []
for item in items:
    amount_str = item.get("paid_amount", "0")
    amount_num = 0.0
    try:
        amount_num = float(amount_str.replace("$", "").replace(",", "").strip())
    except (ValueError, AttributeError):
        pass

    billing.append({
        "sn": item.get("sn", ""),
        "account_no": item.get("account_no", ""),
        "payment_method": item.get("payment_method", ""),
        "invoice": "",
        "status": item.get("status", "").upper(),
        "date": item.get("date", ""),
        "transaction_id": item.get("transaction_id", ""),
        "transition_type": item.get("transition_type", ""),
        "paid_amount": f"${amount_num:.2f}" if amount_num else amount_str,
        "funding_package": item.get("funding_package", ""),
        "paid_amount_numeric": amount_num,
        "login": item.get("account_no", "")
    })

self.logger.info(f"[BILLING] Got {len(billing)} records via DOM scrape")
return billing

```

Method: FundedNextCDPAccount.get_payouts

```
def get_payouts(self)
```

FundedNext doesn't have a separate payouts endpoint — included in billing.

Step by step (top-level body)

1. **return []**.

Code (copy-paste safe)

```
def get_payouts(self):
    """FundedNext doesn't have a separate payouts endpoint – included in billing."""
    return []
```

Method: FundedNextCDPAccount.get_account_mapping

```
def get_account_mapping(self)
```

Get mapping from FundedNext login ID to Tradovate account name by extracting React fiber props from .dashboard-card elements.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Calls `self._navigate_to(...)` for its side effect.
2. Calls `time.sleep(...)` for its side effect.
3. Calls `self._switch_type_tab(...)` for its side effect.
4. Calls `time.sleep(...)` for its side effect.
5. Assigns `body_text = self._js('document.body.innerText.substring(0, 500)')` or `''`.
6. **if** 'Something Went Wrong' in `body_text`: branches conditionally.
7. Assigns `raw = self._js("\n (function() {\n va....`
8. Assigns `mapping = {}`.
9. **if** `raw`: branches conditionally.
10. Calls `self.logger.info(...)` for its side effect.
11. **return** `mapping`.

Code (copy-paste safe)

```
def get_account_mapping(self):
    """
    Get mapping from FundedNext login ID to Tradovate account name
    by extracting React fiber props from .dashboard-card elements.
    """
```

```

self._navigate_to("/accounts")
time.sleep(2)
self._switch_type_tab("Futures")
time.sleep(2)

# Check for page error
body_text = self._js("document.body.innerText.substring(0, 500)") or ""
if "Something Went Wrong" in body_text:
    self.logger.warning("[MAPPING] Futures tab returned error page")
    return {}

raw = self._js("""
(function() {
    var cards = document.querySelectorAll('.dashboard-card');
    var results = [];
    for (var i = 0; i < cards.length; i++) {
        var keys = Object.keys(cards[i]);
        for (var j = 0; j < keys.length; j++) {
            if (keys[j].indexOf('__reactFiber') !== -1) {
                var node = cards[i][keys[j]];
                for (var k = 0; k < 30 && node; k++) {
                    var p = node.memoizedProps;
                    if (p && p.account && p.account.login) {
                        var acct = p.account;
                        results.push({
                            login: acct.login,
                            account_id: acct.id,
                            tradovate_name: (acct.tradovate_account_name || {
                                }).tradovate_account_name || null,
                            plan_title: (acct.plan || {}).title || null,
                            balance: acct.balance,
                            starting_balance: acct.starting_balance,
                            server_type: (acct.server || {}).server_type || null,
                            breached: acct.breached,
                            breachedby: acct.breachedby || null,
                            profit_target: acct.profit_target || acct.target || null,
                            drawdown: acct.drawdown || acct.max_drawdown || null
                        });
                        break;
                    }
                }
                node = node.return;
            }
        }
        break;
    }
    return JSON.stringify(results);
})();
""")

mapping = {}
if raw:
    try:
        accounts = json.loads(raw)
        for acct in accounts:
            login_id = acct.get("login")
            if login_id:
                starting = acct.get("starting_balance")
                dd = acct.get("drawdown")
                min_equity = None

```

```

        if starting is not None and dd is not None:
            try:
                min_equity = float(starting) - float(dd)
            except (ValueError, TypeError):
                pass
        mapping[str(login_id)] = {
            "tradovate_account_name": acct.get("tradovate_name"),
            "account_id": acct.get("account_id"),
            "plan_title": acct.get("plan_title"),
            "balance": acct.get("balance"),
            "starting_balance": starting,
            "server_type": acct.get("server_type"),
            "breached": acct.get("breached"),
            "breachedby": acct.get("breachedby"),
            "profit_target": acct.get("profit_target"),
            "min_equity": min_equity,
        }
        self.logger.info(
            f"[MAPPING] login={login_id} -> tradovate={acct.get('tradovate_name')} "
            f"({acct.get('plan_title')}) target={acct.get('profit_target')} min_eq={min_e"
        except (json.JSONDecodeError, TypeError):
            pass

    self.logger.info(f"[MAPPING] Built mapping for {len(mapping)} account(s)")
    return mapping

```

Method: FundedNextCDPAccount.get_account_overview

```
def get_account_overview(self, account_id)
```

Get detailed account overview via the FundedNext API.

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns token = self._get_token().
2. if not token: branches conditionally.
3. Assigns data = self._fetch_json_bearer(f'{self.API_BASE}/account-overvie....
4. if not data: branches conditionally.
5. Assigns overview = data.get('data', {}).
6. if not overview: branches conditionally.
7. Assigns acct_details = overview.get('account_details', {}).
8. Calls self.logger.info(...) for its side effect.
9. return overview.

Code (copy-paste safe)

```

def get_account_overview(self, account_id):
    """Get detailed account overview via the FundedNext API."""
    token = self._get_token()

```

```

if not token:
    self.logger.warning("[OVERVIEW] No tokenV1 cookie found")
    return None

data = self._fetch_json_bearer(
    f"{self.API_BASE}/account-overview?account_id={account_id}", token)
if not data:
    return None

overview = data.get("data", {})
if not overview:
    self.logger.warning(f"[OVERVIEW] Empty data for account_id={account_id}")
    return None

acct_details = overview.get("account_details", {})
self.logger.info(
    f"[OVERVIEW] account_id={account_id} | "
    f"name={acct_details.get('account_name')} | "
    f"breached={acct_details.get('breached')}")
return overview

```

Functions

```
def _find_chrome_exe()
```

Locate Chrome executable on Windows.

Step by step (top-level body)

1. Assigns candidates = [].
2. **for** env in ('PROGRAMFILES', 'PROGRAMFILES(X86)', 'LOCALAPPDATA'): iterates.
3. **for** p in candidates: iterates.
4. Assigns found = shutil.which('chrome') or shutil.which('google-chrome').
5. **return** found.

Code (copy-paste safe)

```

def _find_chrome_exe():
    """Locate Chrome executable on Windows."""
    candidates = []
    for env in ("PROGRAMFILES", "PROGRAMFILES(X86)", "LOCALAPPDATA"):
        base = os.environ.get(env, "")
        if base:
            candidates.append(os.path.join(base, "Google", "Chrome", "Application", "chrome.exe"))
    for p in candidates:
        if os.path.isfile(p):
            return p
    # Fallback: check PATH
    found = shutil.which("chrome") or shutil.which("google-chrome")
    return found

```

```
def _is_chrome_debug_running(port=CDP_DEBUG_PORT)
```

Check if Chrome debug port is already responding.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def _is_chrome_debug_running(port=CDP_DEBUG_PORT):
    """Check if Chrome debug port is already responding."""
    try:
        urllib.request.urlopen(f'http://127.0.0.1:{port}/json/version', timeout=2)
        return True
    except Exception:
        return False
```

```
def ensure_chrome_debug(url=None, port=CDP_DEBUG_PORT)
```

Ensure a Chrome instance with remote debugging is running. If one is already running on the port, just open the URL as a new tab. If not, launch a new Chrome process with the debug port. Returns True if Chrome is available on the port.

Why these Python concepts were used

- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.
- **global** — Rebinds a module-level name from inside the function. A smell in larger codebases — usually means a singleton or cache that could be passed in instead.

Step by step (top-level body)

1. Declares globals: `_chrome_process`.
2. **with** `_chrome_lock`: enters a context manager.

Code (copy-paste safe)

```
def ensure_chrome_debug(url=None, port=CDP_DEBUG_PORT):
    """
    Ensure a Chrome instance with remote debugging is running.
    If one is already running on the port, just open the URL as a new tab.
    If not, launch a new Chrome process with the debug port.
    Returns True if Chrome is available on the port.
    """
    global _chrome_process

    with _chrome_lock:
```

```

# Already running?
if _is_chrome_debug_running(port):
    if url:
        _open_tab_in_debug_chrome(url, port)
    return True

# Launch new Chrome
chrome_exe = _find_chrome_exe()
if not chrome_exe:
    raise FileNotFoundError(
        "Chrome not found. Install Google Chrome or set it on PATH.")

args = [
    chrome_exe,
    f"--remote-debugging-port={port}",
    "--remote-allow-origins=*",
    f"--user-data-dir={CDP_USER_DATA_DIR}",
]
if url:
    args.append(url)

_chrome_process = subprocess.Popen(
    args,
    stdout=subprocess.DEVNULL,
    stderr=subprocess.DEVNULL,
)

# Wait for debug port to be ready
for _ in range(30):
    time.sleep(0.5)
    if _is_chrome_debug_running(port):
        # Chrome first-run may hijack the initial URL – open it as a tab too
        if url:
            time.sleep(1)
            _open_tab_in_debug_chrome(url, port)
        return True

raise TimeoutError(
    f"Chrome launched but debug port {port} not responding after 15s")

```

```
def _open_tab_in_debug_chrome(url, port=CDP_DEBUG_PORT)
```

Open a new tab in the already-running debug Chrome via the /json/new endpoint. Skips if a tab with the same domain is already open.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.
2. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def _open_tab_in_debug_chrome(url, port=CDP_DEBUG_PORT):
    """Open a new tab in the already-running debug Chrome via the /json/new endpoint.
    Skips if a tab with the same domain is already open."""
    try:
        # Check if a tab with this domain is already open
        from urllib.parse import urlparse
        target_domain = urlparse(url).netloc
        data = urllib.request.urlopen(
            f'http://127.0.0.1:{port}/json', timeout=5).read()
        tabs = json.loads(data)
        for t in tabs:
            if t.get('type') == 'page':
                tab_domain = urlparse(t.get('url', '')).netloc
                if tab_domain and target_domain and tab_domain == target_domain:
                    return # Already open
    except Exception:
        pass

    try:
        encoded = urllib.parse.quote(url, safe=':/')
        req = urllib.request.Request(
            f'http://127.0.0.1:{port}/json/new?{encoded}',
            method='PUT'
        )
        urllib.request.urlopen(req, timeout=5)
    except Exception:
        try:
            urllib.request.urlopen(
                f'http://127.0.0.1:{port}/json/new?url={url}', timeout=5)
        except Exception:
            pass

def quick_test(debug_port=9222)
```

Test all available prop firm scrapers against open Chrome tabs.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns scrapers = [('Tradeify', TradeifyAccount), ('Lucid Trading', LucidTr....
2. **for** (name, cls) in scrapers: iterates.

Code (copy-paste safe)

```
def quick_test(debug_port=9222):
    """Test all available prop firm scrapers against open Chrome tabs."""
    scrapers = [
        ("Tradeify", TradeifyAccount),
        ("Lucid Trading", LucidTradingAccount),
```

```

        ("TopStep", TopStepAccount),
        ("MFFU", MFFUAccount),
        ("FundedNext", FundedNextCDPAccount),
    ]

    for name, cls in scrapers:
        print(f"\n{'='*60}")
        print(f"  {name}")
        print(f"{'='*60}")
        scraper = cls(debug_port=debug_port)
        try:
            scraper.login()
            print(f"  Connected: {scraper.is_connected()}")

            stats = scraper.get_account_stats()
            print(f"\n  Account Stats:")
            for k, v in stats.items():
                print(f"    {k}: {v}")

            accounts = scraper.get_all_accounts()
            print(f"\n  Accounts: {len(accounts)}")
            for i, acct in enumerate(accounts[:3]):
                acct_name = acct.get('accountName', acct.get('account_number', acct.get('name', f'#{i+1}'))
                print(f"    {acct_name}")

            payouts = scraper.get_payouts()
            print(f"  Payouts: {len(payouts)}")

            billing = scraper.get_billing_history()
            print(f"  Billing records: {len(billing)}")

            scraper.close()
        except ConnectionError as e:
            print(f"  Tab not found — skipping ({e})")
        except Exception as e:
            print(f"  Error: {e}")

```

Step 8.2 — Build trader_companion/fundednext.py

What to do. Create the file trader_companion/fundednext.py in your project root and paste in the code shown in this step. The file is **1386** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from requirements.txt.

What this file is for. FundedNext-specific scraper and account adapter.

trader_companion/fundednext.py

1386 loc · 1 classes · 3 functions · 18 imports

FundedNext-specific scraper and account adapter. It defines **1** class and **3** module-level functions, across **1,386** lines. Module docstring: *FundedNext Dashboard Integration - Selenium Browser*

Module *docstring*

FundedNext Dashboard Integration - Selenium Browser Automation Scrapes account statistics from <https://app.fundednext.com/accounts> Follows the same pattern as TradovateAccount and TopStepXAccount.

Python concepts demonstrated in this module

- Classes and objects
- Comprehensions
- Context managers (with-statements)
- Decorators
- Dunder methods
- Exceptions
- f-strings

Imports — *what each one is for*

```
import os
```

Why imported. Operating-system interface. Path manipulation, environment variables, working-directory operations, exit codes, and thin wrappers around POSIX/Windows syscalls.

Used in this file. `os.environ`, `os.environ.get`, `os.makedirs`, `os.path`, `os.path.exists`, `os.path.join`

Example. `os.path.join(root, 'subdir', 'file.txt')` # joins with the OS-correct separator

Other common scenarios.

- `os.environ.get('API_KEY')` to read environment variables
- `os.makedirs(path, exist_ok=True)` to create nested directories
- `os.listdir(path)` to list a directory's contents
- `os.remove(path)` / `os.rename(src, dst)` to delete or move a file
- `os.getcwd()` / `os.chdir(path)` to inspect or change the working dir

```
import sys
```

Why imported. Access to the running interpreter — `argv`, `stdin/stdout`, exit codes, the import search path, and the platform name.

Used in this file. *No direct attribute access detected — likely imported for side effects, re-export, or string references.*

Example. `sys.exit(1)` # terminate the process with a non-zero status

Other common scenarios.

- `sys.argv[1:]` reads the command-line arguments
- `sys.path.insert(0, 'extra')` prepends to the import search path
- `sys.stderr.write(msg)` writes to standard error
- `sys.platform` tells you 'win32', 'linux', or 'darwin'
- `sys.modules` is the global cache of already-imported modules

```
import re
```

Why imported. Regular expressions. Pattern matching, search, replace, and text extraction.

Used in this file. `re.findall`, `re.match`, `re.search`, `re.sub`

Example. `re.search(r'\d{3}-\d{4}', text)` # returns a Match or None

Other common scenarios.

- `re.findall(pattern, text)` for every non-overlapping match
- `re.sub(pattern, repl, text)` to replace occurrences
- `re.compile(...)` to reuse a pattern (faster in a loop)
- Named groups: `r'(?P<year>\d{4})'` lets you do `m.group('year')`

```
import time
```

Why imported. Timestamps and sleep. Lower-level than `datetime` — works in Unix-epoch seconds.

Used in this file. `time.sleep`, `time.time`

Example. `time.sleep(1.5)` # pause this thread for 1.5 seconds

Other common scenarios.

- `time.time()` returns seconds since the epoch
- `time.monotonic()` for measuring elapsed time (never goes backward)
- `time.perf_counter()` for high-precision benchmarking
- `time.strptime` / `time.strftime` mirror the `datetime` versions

```
import threading
```

Why imported. Threads and synchronisation primitives. Used when work is I/O-bound and the GIL is not a bottleneck (the GIL prevents speed-up on CPU-bound work).

Used in this file. `threading.RLock`

Example. `Thread(target=worker, args=(q,), daemon=True).start()`

Other common scenarios.

- Lock / RLock to guard shared state
- Event for one-shot signals between threads
- Queue (from `queue`) for producer/consumer patterns
- `threading.local()` for thread-local storage

```
import logging
```

Why imported. Structured, levelled logging. Replaces `print()` with filterable, formattable output that can route to files, `stderr`, `syslog`, or HTTP endpoints.

Used in this file. `logging.INFO`, `logging.basicConfig`, `logging.getLogger`

Example. `logging.getLogger(__name__).info('connected to %s', host)`

Other common scenarios.

- `.debug()` / `.info()` / `.warning()` / `.error()` / `.exception()`

- `logger.exception()` captures the current traceback automatically
- `logging.basicConfig(level=logging.INFO)` for quick setup
- Handlers and Formatters route logs to files / streams / syslog

`import hashlib`

Why imported. Cryptographic hash functions — SHA-256, MD5, BLAKE2.

Used in this file. `hashlib.md5`

Example. `hashlib.sha256(b'hello').hexdigest()`

Other common scenarios.

- Pass data in chunks via `.update()` to hash large files
- Use `sha256` / `sha512` for integrity; never MD5 or SHA-1 for security
- For password hashing use `bcrypt/argon2`, NOT raw `hashlib`

`import tempfile`

Why imported. Temporary files and directories that clean themselves up.

Used in this file. `tempfile.gettempdir`

Example. with `tempfile.TemporaryDirectory()` as `d`: ...

Other common scenarios.

- `tempfile.NamedTemporaryFile(delete=False)` for a real path on disk
- `tempfile.mkstemp()` returns a low-level (fd, path) pair

`from selenium import webdriver`

Why imported. Browser automation. Drives a real Chrome instance — the fallback for scraping prop-firm dashboards that have no API.

Used in this file. `webdriver`

Example. `driver.get(url); driver.find_element(By.ID, 'login')`

Other common scenarios.

- `WebDriverWait` + `expected_conditions` for reliable waits
- `ChromeOptions().add_argument('--headless=new')`
- `execute_script(js, *args)` reaches into the page for things the DOM API can't express

`from selenium.webdriver.common.by import By`

Why imported. Browser automation. Drives a real Chrome instance — the fallback for scraping prop-firm dashboards that have no API.

Used in this file. `By`

Example. `driver.get(url); driver.find_element(By.ID, 'login')`

Other common scenarios.

- `WebDriverWait` + `expected_conditions` for reliable waits

- `ChromeOptions().add_argument('--headless=new')`
- `execute_script(js, *args)` reaches into the page for things the DOM API can't express

```
from selenium.webdriver.common.keys import Keys
```

Why imported. Browser automation. Drives a real Chrome instance — the fallback for scraping prop-firm dashboards that have no API.

Used in this file. Keys

Example. `driver.get(url); driver.find_element(By.ID, 'login')`

Other common scenarios.

- `WebDriverWait + expected_conditions` for reliable waits
- `ChromeOptions().add_argument('--headless=new')`
- `execute_script(js, *args)` reaches into the page for things the DOM API can't express

```
from selenium.webdriver.support.ui import WebDriverWait
```

Why imported. Browser automation. Drives a real Chrome instance — the fallback for scraping prop-firm dashboards that have no API.

Used in this file. WebDriverWait

Example. `driver.get(url); driver.find_element(By.ID, 'login')`

Other common scenarios.

- `WebDriverWait + expected_conditions` for reliable waits
- `ChromeOptions().add_argument('--headless=new')`
- `execute_script(js, *args)` reaches into the page for things the DOM API can't express

```
from selenium.webdriver.support import expected_conditions as EC
```

Why imported. Browser automation. Drives a real Chrome instance — the fallback for scraping prop-firm dashboards that have no API.

Used in this file. EC

Example. `driver.get(url); driver.find_element(By.ID, 'login')`

Other common scenarios.

- `WebDriverWait + expected_conditions` for reliable waits
- `ChromeOptions().add_argument('--headless=new')`
- `execute_script(js, *args)` reaches into the page for things the DOM API can't express

```
from selenium.webdriver.chrome.options import Options
```

Why imported. Browser automation. Drives a real Chrome instance — the fallback for scraping prop-firm dashboards that have no API.

Used in this file. Options

Example. `driver.get(url); driver.find_element(By.ID, 'login')`

Other common scenarios.

- `WebDriverWait + expected_conditions` for reliable waits
- `ChromeOptions().add_argument('--headless=new')`
- `execute_script(js, *args)` reaches into the page for things the DOM API can't express

```
from selenium.webdriver.chrome.service import Service
```

Why imported. Browser automation. Drives a real Chrome instance — the fallback for scraping prop-firm dashboards that have no API.

Used in this file. *No direct attribute access detected — likely imported for side effects, re-export, or string references.*

Example. `driver.get(url); driver.find_element(By.ID, 'login')`

Other common scenarios.

- `WebDriverWait + expected_conditions` for reliable waits
- `ChromeOptions().add_argument('--headless=new')`
- `execute_script(js, *args)` reaches into the page for things the DOM API can't express

```
from selenium.common.exceptions import TimeoutException
from selenium.common.exceptions import WebDriverException
from selenium.common.exceptions import NoSuchElementException
from selenium.common.exceptions import StaleElementReferenceException
```

Why imported. Browser automation. Drives a real Chrome instance — the fallback for scraping prop-firm dashboards that have no API.

Used in this file. `NoSuchElementException`, `StaleElementReferenceException`, `TimeoutException`

Example. `driver.get(url); driver.find_element(By.ID, 'login')`

Other common scenarios.

- `WebDriverWait + expected_conditions` for reliable waits
- `ChromeOptions().add_argument('--headless=new')`
- `execute_script(js, *args)` reaches into the page for things the DOM API can't express

```
from functools import wraps
```

Why imported. Higher-order helpers — caching, partial application, decorator authoring tools, reducers.

Used in this file. `wraps`

Example. `@lru_cache(maxsize=128) # memoise this pure function`

Other common scenarios.

- `partial(f, x=1)` pre-binds arguments
- `reduce(f, iterable)` folds an iterable to a single value
- `@wraps(fn)` preserves `__name__`/`__doc__` when writing decorators
- `@cached_property` memoises a property per-instance

```
from dotenv import load_dotenv
```

Why imported. python-dotenv — load .env files into os.environ for twelve-factor configuration in development.

Used in this file. load_dotenv

Example. load_dotenv() # call once at process start

Other common scenarios.

- dotenv_values('.env') returns a dict without polluting os.environ
- Use a real secret store in production (vault, AWS SM, etc.)

Classes

```
class FundedNextAccount
```

FundedNext dashboard automation class with Selenium-based web scraping. Reads account statistics from <https://app.fundednext.com/accounts>. Supports two connection modes: 1. attach_to_existing=True -> Connects to an already-open Chrome via remote debugging 2. attach_to_existing=False -> Launches a new Chrome and logs in with credentials Follows the same interface as TradovateAccount and TopStepXAccount: - login() / connect() - get_account_stats() - is_connected() - close() / disconnect()

```
_chrome_instances = {}
```

Code (copy-paste safe)

```
class FundedNextAccount:
    """
    FundedNext dashboard automation class with Selenium-based web scraping.
    Reads account statistics from https://app.fundednext.com/accounts.

    Supports two connection modes:
    1. attach_to_existing=True -> Connects to an already-open Chrome via remote debugging
    2. attach_to_existing=False -> Launches a new Chrome and logs in with credentials

    Follows the same interface as TradovateAccount and TopStepXAccount:
    - login() / connect()
    - get_account_stats()
    - is_connected()
    - close() / disconnect()
    """

    _chrome_instances = {}

    def __init__(self, username=None, password=None, pair_id=None,
                  attach_to_existing=True, debug_port=9444,
                  use_real_profile=False):
        self.username = username or ""
        self.password = password or ""
        self.pair_id = pair_id or "default"
        self.attach_to_existing = attach_to_existing
        self.debug_port = debug_port
        self.use_real_profile = use_real_profile
        self.logged_in = False
        self.driver = None
        self._placing_order = False
        self._login_timestamp = None
        self._first_stats_fetch = True
```

```

self.base_url = "https://app.fundednext.com"
self.accounts_url = "https://app.fundednext.com/accounts"
self.login_url = "https://app.fundednext.com"
self.lock = threading.RLock()

self.logger = logging.getLogger(f"FundedNext_{self.username}_{self.pair_id}")
self.logger.info(
    f"[INIT] FundedNextAccount initializing for user={username}, pair_id={pair_id}, attach={attach}"

instance_key = f"fundednext_{username}_{self.pair_id}"

# Reuse existing Chrome instance if available
if instance_key in self._chrome_instances:
    self.logger.info(f"[INIT] Reusing existing Chrome instance for FundedNext: {username}")
    existing_driver = self._chrome_instances[instance_key]
    try:
        existing_driver.current_url
        self.driver = existing_driver
        self.logger.info("[INIT] Successfully connected to existing Chrome instance")
        return
    except Exception as e:
        self.logger.info(f"[INIT] Existing Chrome instance dead ({e}), creating new one")
        del self._chrome_instances[instance_key]

try:
    self.driver = self._initialize_driver()
    self._chrome_instances[instance_key] = self.driver
    self.logger.info(f"[INIT] Chrome instance registered for FundedNext: {username}")
except Exception as e:
    self.logger.error(f"[INIT ERROR] Failed to initialize WebDriver: {e}")
    raise Exception(f"Failed to initialize WebDriver: {e}")

def _get_chromedriver_path(self):
    """Let Selenium Manager handle ChromeDriver automatically"""
    return None

def _copy_chrome_cookies(self, chrome_profile_src, dest_user_data_dir):
    """Copy Chrome login cookies from user's profile to temp profile"""
    import shutil
    try:
        src_default = os.path.join(chrome_profile_src, "Default")
        dst_default = os.path.join(dest_user_data_dir, "Default")
        os.makedirs(dst_default, exist_ok=True)

        # Copy cookie-related files
        cookie_files = ["Cookies", "Cookies-journal", "Login Data", "Login Data-journal",
                        "Web Data", "Web Data-journal", "Preferences", "Secure Preferences"]
        for fname in cookie_files:
            src = os.path.join(src_default, fname)
            dst = os.path.join(dst_default, fname)
            if os.path.exists(src) and not os.path.exists(dst):
                try:
                    shutil.copy2(src, dst)
                    self.logger.info(f"[PROFILE] Copied {fname}")
                except Exception as e:
                    self.logger.debug(f"[PROFILE] Could not copy {fname}: {e}")

        # Copy Local State for encryption keys
        src_local = os.path.join(chrome_profile_src, "Local State")
        dst_local = os.path.join(dest_user_data_dir, "Local State")

```

```

        if os.path.exists(src_local) and not os.path.exists(dst_local):
            shutil.copy2(src_local, dst_local)
            self.logger.info("[PROFILE] Copied Local State (encryption keys)")

    except Exception as e:
        self.logger.warning(f"[PROFILE] Cookie copy failed: {e}")

def _initialize_driver(self):
    """Initialize Chrome WebDriver - attach to existing or launch new with profile cookies"""
    chrome_options = Options()

    if self.attach_to_existing:
        # === ATTACH MODE: Connect to already-running Chrome ===
        self.logger.info(f"[DRIVER] Attaching to existing Chrome on port {self.debug_port}...")
        chrome_options.add_experimental_option("debuggerAddress", f"127.0.0.1:{self.debug_port}")

        try:
            driver = webdriver.Chrome(options=chrome_options)
            self.logger.info(f"[DRIVER] Attached to Chrome. Current URL: {driver.current_url}")

            if "fundednext.com" in driver.current_url:
                self.logged_in = True
                self._login_timestamp = time.time()
                self.logger.info("[DRIVER] Already on FundedNext - session active")

            return driver

        except Exception as e:
            self.logger.warning(f"[DRIVER] Failed to attach ({e}). Falling back to profile mode...")
            self.attach_to_existing = False
            chrome_options = Options()

    # === PROFILE MODE: Launch Chrome with COPY of user's profile for cookies ===
    # This reuses login cookies from the user's normal Chrome session
    chrome_profile_src = os.path.join(os.environ.get('LOCALAPPDATA', ''),
                                      'Google', 'Chrome', 'User Data')

    if self.use_real_profile:
        # USE REAL PROFILE: requires closing all other Chrome instances first
        self.logger.info("[DRIVER] Using real Chrome profile (requires no other Chrome running)")
        user_data_dir = chrome_profile_src
    else:
        unique_id = f"fundednext_{self.username}_{self.pair_id}"
        unique_hash = hashlib.md5(unique_id.encode()).hexdigest()[:8]
        user_data_dir = os.path.join(tempfile.gettempdir(), f"chrome_fundednext_{unique_hash}")

        # Copy the Default profile's Cookies file if our temp dir is empty
        profile_default = os.path.join(user_data_dir, "Default")
        if not os.path.exists(os.path.join(profile_default, "Cookies")):
            self._copy_chrome_cookies(chrome_profile_src, user_data_dir)

    chrome_options.add_argument(f"--user-data-dir={user_data_dir}")

    if not self.use_real_profile:
        debug_port = 9500 + (int(hashlib.md5(f"fundednext_{self.username}_{self.pair_id}".encode(
            )).hexdigest()[:4], 16) % 50)
        chrome_options.add_argument(f"--remote-debugging-port={debug_port}")

    # Standard Chrome options
    chrome_options.add_argument("--no-sandbox")

```

```

chrome_options.add_argument("--disable-dev-shm-usage")
chrome_options.add_argument("--disable-gpu")
chrome_options.add_argument("--disable-extensions")
chrome_options.add_argument("--no-first-run")
chrome_options.add_argument("--no-default-browser-check")
chrome_options.add_argument("--disable-features=TranslateUI")
chrome_options.add_argument("--window-size=1200,800")

# Anti-detection
chrome_options.add_experimental_option("excludeSwitches", ["enable-automation", "enable-logging"])
chrome_options.add_experimental_option('useAutomationExtension', False)
chrome_options.add_argument("--disable-blink-features=AutomationControlled")

prefs = {
    "profile.default_content_setting_values": {
        "popups": 2,
        "notifications": 2,
    }
}
chrome_options.add_experimental_option("prefs", prefs)

try:
    self.logger.info("[CHROME] Launching new Chrome instance for FundedNext...")
    start_time = time.time()
    driver = webdriver.Chrome(options=chrome_options)
    elapsed = time.time() - start_time
    self.logger.info(f"[CHROME] Chrome launched in {elapsed:.1f}s")

    driver.set_page_load_timeout(30)
    driver.implicitly_wait(2)
    driver.execute_script("Object.defineProperty(navigator, 'webdriver', {get: () => undefined})")

    return driver

except Exception as e:
    self.logger.error(f"[DRIVER ERROR] Failed to initialize Chrome: {e}")
    raise

def login(self, max_retries=3):
    """Login to FundedNext platform"""
    with self.lock:
        # If attached to existing session, just verify we're on FundedNext
        if self.attach_to_existing and self.driver:
            try:
                current_url = self.driver.current_url
                if "fundednext.com" in current_url:
                    self.logged_in = True
                    self._login_timestamp = time.time()
                    self.logger.info(f"[LOGIN] Already logged in via existing Chrome: {current_url}")
                    return True
            except Exception:
                pass

        if not self.username or not self.password:
            self.logger.error("Username and password required for FundedNext login")
            return False

        for attempt in range(max_retries):
            try:
                self.logger.info(f"[LOGIN] FundedNext login attempt {attempt + 1}/{max_retries}")

```

```

        if not self.driver:
            self.driver = self._initialize_driver()

        self.driver.get(self.login_url)
        time.sleep(3)

        wait = WebDriverWait(self.driver, 15)

        # FundedNext login form - email and password
        email_field = wait.until(
            EC.presence_of_element_located((By.CSS_SELECTOR,
                "input[type='email'], input[name='email'], input[placeholder*='mail']"))
        )
        email_field.send_keys(Keys.CONTROL + "a")
        time.sleep(0.2)
        email_field.send_keys(self.username)

        password_field = self.driver.find_element(By.CSS_SELECTOR,
            "input[type='password'], input[name='password']")
        password_field.send_keys(Keys.CONTROL + "a")
        time.sleep(0.2)
        password_field.send_keys(self.password)

        # Click login button
        login_button = self.driver.find_element(By.XPATH,
            "//button[@type='submit'] | //button[contains(text(), 'Login')] | //button[contains(
        login_button.click()

        # Wait for redirect
        time.sleep(5)

        max_wait = 20
        start_time = time.time()
        while time.time() - start_time < max_wait:
            current_url = self.driver.current_url.lower()

            if any(kw in current_url for kw in ["accounts", "dashboard", "overview"]):
                self.logged_in = True
                self._login_timestamp = time.time()
                self.logger.info(f"[LOGIN] FundedNext login successful: {current_url}")
                return True

            # Check for error messages
            try:
                error_el = self.driver.find_element(By.CSS_SELECTOR,
                    "[class*='error'], [class*='alert-danger'], .text-danger")
                if error_el.is_displayed():
                    self.logger.warning(f"[LOGIN] Error: {error_el.text}")
                    break
            except NoSuchElementException:
                pass

            time.sleep(1)

        self.logger.warning(f"[LOGIN] Attempt {attempt + 1} timed out")

    except Exception as e:
        self.logger.error(f"[LOGIN] Attempt {attempt + 1} failed: {e}")
        time.sleep(3)

    return False

```

```

def connect(self):
    """Connect to FundedNext - alias for login()"""
    return self.login()

def navigate_to_accounts(self):
    """Navigate to the accounts page if not already there"""
    try:
        current_url = self.driver.current_url
        if "/accounts" in current_url:
            return True

        self.logger.info("[NAV] Navigating to accounts page...")
        self.driver.get(self.accounts_url)
        time.sleep(3)

        # Wait for the account-wrapper to load (confirmed selector from real DOM)
        WebDriverWait(self.driver, 15).until(
            EC.presence_of_element_located((By.CSS_SELECTOR, ".account-wrapper"))
        )
        return True

    except Exception as e:
        self.logger.error(f"[NAV] Failed to navigate to accounts: {e}")
        return False

def switch_type_tab(self, tab_name="CFDs"):
    """
    Switch between CFDs and Futures tabs.
    FundedNext uses ant-tabs: .ant-tabs-tab > .ant-tabs-tab-btn
    Uses CDP Runtime.evaluate to avoid Selenium page-load hangs on tab switch.
    """
    try:
        # Use CDP to click tab – avoids Selenium hanging when the tab
        # triggers a React Server Component transition
        result = self.driver.execute_cdp_cmd("Runtime.evaluate", {
            "expression": f"""
                (function() {{
                    var tabs = document.querySelectorAll('.ant-tabs-tab-btn');
                    for (var i = 0; i < tabs.length; i++) {{
                        if (tabs[i].textContent.trim() === '{tab_name}') {{
                            tabs[i].click();
                            return 'clicked';
                        }}
                    }}
                    return 'not_found';
                }})()
            """,
            "returnByValue": True,
            "timeout": 5000
        })
        status = result.get("result", {}).get("value", "")
        if status == "clicked":
            time.sleep(3)
            self.logger.info(f"[NAV] Switched to type tab: {tab_name}")
            return True
        else:
            self.logger.warning(f"[NAV] Type tab '{tab_name}' not found via CDP")
            return False
    except Exception as e:
        self.logger.warning(f"[NAV] CDP tab switch failed ({e}), trying Selenium fallback")

```

```

    try:
        tabs = self.driver.find_elements(By.CSS_SELECTOR, ".ant-tabs-tab")
        for tab in tabs:
            if tab.text.strip() == tab_name:
                tab.click()
                time.sleep(2)
                self.logger.info(f"[NAV] Switched to type tab via Selenium: {tab_name}")
                return True
    except Exception as e2:
        self.logger.error(f"[NAV] Selenium fallback also failed: {e2}")
    return False

def switch_status_tab(self, status="Active"):
    """
    Switch between Active/Inactive/Breached status tabs.
    These are buttons inside .account-wrapper__create-account with Tailwind classes.
    """
    try:
        buttons = self.driver.find_elements(
            By.CSS_SELECTOR, ".account-wrapper__create-account button")
        for btn in buttons:
            if btn.text.strip() == status:
                btn.click()
                time.sleep(2)
                self.logger.info(f"[NAV] Switched to status tab: {status}")
                return True
        self.logger.warning(f"[NAV] Status tab '{status}' not found")
        return False
    except Exception as e:
        self.logger.error(f"[NAV] Failed to switch status tab: {e}")
        return False

def has_accounts(self):
    """
    Check if the current tab view has any accounts.
    FundedNext shows .no-account-wrapper when empty.
    """
    try:
        no_acct = self.driver.find_elements(By.CSS_SELECTOR, ".no-account-wrapper")
        if no_acct and no_acct[0].is_displayed():
            return False
        return True
    except Exception:
        return False

def is_connected(self):
    """Check if connected to FundedNext"""
    try:
        if not self.driver:
            return False
        current_url = self.driver.current_url
        return "fundednext.com" in current_url
    except Exception:
        return False

@retry_on_stale_element(max_retries=3, delay=1)
def get_account_stats(self):
    """
    Get FundedNext account statistics from the dashboard.
    Returns dict with same keys as Tradovate/TopStepX for GUI compatibility:

```



```

    Account Number, Balance, Profit/Loss, Open Trades, Symbol, Direction
    Plus FundedNext-specific fields:
    Equity, Drawdown, Profit Target, Phase, Status
    """
    if not self.lock.acquire(blocking=False):
        return getattr(self, '_cached_stats', self._default_stats("Trading..."))

    try:
        if getattr(self, '_placing_order', False):
            return getattr(self, '_cached_stats', self._default_stats("Trading..."))

        # Cache TTL
        cache_ttl = 2.0
        current_time = time.time()
        last_fetch = getattr(self, '_stats_last_fetch_time', 0)
        if (current_time - last_fetch) < cache_ttl and hasattr(self, '_cached_stats'):
            return self._cached_stats

        if not self.is_connected():
            return self._default_stats("Not Connected")

        # Ensure we're on the accounts page
        self.navigate_to_accounts()

        stats = {
            "Account Number": "Unknown",
            "Balance": "N/A",
            "Equity": "N/A",
            "Profit/Loss": "N/A",
            "Open Trades": "0",
            "Symbol": "",
            "Direction": "",
            "Drawdown": "N/A",
            "Profit Target": "N/A",
            "Phase": "N/A",
            "Status": "N/A",
        }

        try:
            self._extract_account_info(stats)
        except Exception as e:
            self.logger.warning(f"Could not extract account info: {e}")

        try:
            self._extract_balance_equity(stats)
        except Exception as e:
            self.logger.warning(f"Could not extract balance/equity: {e}")

        try:
            self._extract_drawdown_target(stats)
        except Exception as e:
            self.logger.warning(f"Could not extract drawdown/target: {e}")

        try:
            self._extract_phase_status(stats)
        except Exception as e:
            self.logger.warning(f"Could not extract phase/status: {e}")

        self._cached_stats = stats
        self._stats_last_fetch_time = time.time()

```

```

        return stats

    except Exception as e:
        self.logger.error(f"Failed to get FundedNext stats: {e}")
        return self._default_stats("Error")
    finally:
        self.lock.release()

def get_all_accounts(self):
    """
    Get statistics for ALL accounts listed on the FundedNext accounts page.
    Checks both Futures and CFDs tabs (Futures first), Active status.
    Returns a list of dicts, one per account.
    """
    if not self.is_connected():
        return []

    with self.lock:
        try:
            self.navigate_to_accounts()
            time.sleep(2)

            accounts = []

            # Check both type tabs – Futures first (primary use case)
            for type_tab in ["Futures", "CFDs"]:
                self.switch_type_tab(type_tab)
                self.switch_status_tab("Active")
                time.sleep(1)

                if not self.has_accounts():
                    self.logger.info(f"[ACCOUNTS] No active accounts under {type_tab}")
                    continue

                acct_elements = self._find_account_elements()
                for i, element in enumerate(acct_elements):
                    try:
                        acct_stats = self._parse_account_element(element, i)
                        if acct_stats:
                            acct_stats["Type"] = type_tab
                            accounts.append(acct_stats)
                    except Exception as e:
                        self.logger.warning(f"[ACCOUNTS] Failed to parse account {i}: {e}")

                self.logger.info(f"[ACCOUNTS] Extracted {len(accounts)} accounts total")
            return accounts

        except Exception as e:
            self.logger.error(f"[ACCOUNTS] Failed to get all accounts: {e}")
            return []

def _find_account_elements(self):
    """Find account card elements in the current tab view.

    Real DOM structure (confirmed):
    .account-wrapper__content
    .tw-w-full
    .dashboard-card          <-- THE CARD
    h3 (title: "Futures Legacy Challenge | 50K | Account: FNFT...")
    .active-account-card

```

```

        .border-right
        p "Server Type: TRADOVATE"
        p "Balance: $49407.6"
    div
        p "Equity: $49,407.60"
        p "Account Type: Challenge Account"
    button.activeBusinessType "Dashboard"
"""
try:
    cards = self.driver.find_elements(By.CSS_SELECTOR, ".dashboard-card")
    if cards:
        self.logger.info(f"[ACCOUNTS] Found {len(cards)} .dashboard-card elements")
        return cards
except Exception:
    pass

# Fallback: any element inside account-wrapper with dollar values
try:
    cards = self.driver.find_elements(
        By.XPATH, "//*[contains(@class, 'account-wrapper__content')]/div[.//p[contains(text(),
    if cards:
        self.logger.info(f"[ACCOUNTS] Found {len(cards)} cards via Balance fallback")
        return cards
except Exception:
    pass

self.logger.info("[ACCOUNTS] No account elements found")
return []

def _parse_account_element(self, element, index):
    """Parse a .dashboard-card element into a stats dict.

    Card text format (confirmed):
    Line 0: "Futures Legacy Challenge | 50K | Account: FNFTCHHARRISONOUKA85625"
    Line 1: "Server Type: TRADOVATE"
    Line 2: "Balance: $49407.6"
    Line 3: "Equity: $49,407.60"
    Line 4: "Account Type: Challenge Account"
    """
    try:
        text = element.text
        if not text.strip():
            return None

        stats = {
            "Account Number": "Unknown",
            "Balance": "N/A",
            "Equity": "N/A",
            "Profit/Loss": "N/A",
            "Drawdown": "N/A",
            "Profit Target": "N/A",
            "Phase": "N/A",
            "Status": "N/A",
            "Server Type": "N/A",
            "Account Type": "N/A",
            "Challenge": "N/A",
            "Size": "N/A",
        }

        lines = [l.strip() for l in text.split('\n') if l.strip()]

```

```

for line in lines:
    # Title line: "Futures Legacy Challenge | 50K | Account: FNFT..."
    acct_id = re.search(r'FNFT\w+', line)
    if acct_id:
        stats["Account Number"] = acct_id.group()
        # Parse challenge info from title
        title_match = re.match(r'(.+?)\s*\|\s*(\w+)\s*\|\s*Account:', line)
        if title_match:
            stats["Challenge"] = title_match.group(1).strip()
            stats["Size"] = title_match.group(2).strip()
            continue

    # Labeled values: "Balance: $49407.6", "Equity: $49,407.60" etc.
    labeled = re.match(r'^([A-Za-z ]+):\s*(.+)$', line)
    if labeled:
        label = labeled.group(1).strip().lower()
        value = labeled.group(2).strip()

        if label == 'balance':
            stats["Balance"] = value
        elif label == 'equity':
            stats["Equity"] = value
        elif label == 'server type':
            stats["Server Type"] = value
        elif label == 'account type':
            stats["Account Type"] = value
            # Derive status from account type
            if 'challenge' in value.lower():
                stats["Status"] = "Challenge"
            elif 'funded' in value.lower():
                stats["Status"] = "Funded"
        elif 'drawdown' in label or 'max loss' in label:
            stats["Drawdown"] = value
        elif 'target' in label or 'profit target' in label:
            stats["Profit Target"] = value

    # Set Phase from Challenge name if available
    if stats["Phase"] == "N/A" and stats["Challenge"] != "N/A":
        stats["Phase"] = stats["Challenge"]

    return stats

except Exception as e:
    self.logger.warning(f"[PARSE] Failed to parse element {index}: {e}")
    return None

def _extract_account_info(self, stats):
    """Extract account number from dashboard.
    Real DOM: FNFT ID is in a colored <span> inside h3 within .dashboard-card
    """
    # Primary: span with FNFT ID inside .dashboard-card h3
    try:
        fnft_spans = self.driver.find_elements(
            By.CSS_SELECTOR, ".dashboard-card h3 span span")
        for span in fnft_spans:
            text = span.text.strip()
            if text.startswith("FNFT"):
                stats["Account Number"] = text
                self.logger.info(f"[ACCOUNT] Found account: {text}")
        return
    
```

```

except Exception:
    pass

# Fallback: search for FNFT pattern anywhere
try:
    elements = self.driver.find_elements(By.XPATH, "//span[contains(text(), 'FNFT')]")
    for el in elements:
        text = el.text.strip()
        match = re.search(r'FNFT\\w+', text)
        if match:
            stats["Account Number"] = match.group()
            self.logger.info(f"[ACCOUNT] Found account (fallback): {match.group()}")
            return
except Exception:
    pass

def _extract_balance_equity(self, stats):
    """Extract balance and equity from dashboard.
    Real DOM: <p> tags inside .active-account-card with format "Balance: $49407.6"
    """
    # Primary: p tags inside .active-account-card
    try:
        p_tags = self.driver.find_elements(By.CSS_SELECTOR, ".active-account-card p")
        for p in p_tags:
            text = p.text.strip()
            labeled = re.match(r'^([A-Za-z /&]+):\\s*(\\.+)$', text)
            if not labeled:
                continue
            label = labeled.group(1).strip().lower()
            value = labeled.group(2).strip()

            if label == 'balance':
                stats["Balance"] = value
            elif label == 'equity':
                stats["Equity"] = value
            elif label in ('server type',):
                stats.setdefault("Server Type", value)
            elif label in ('account type',):
                stats.setdefault("Account Type", value)
    except Exception:
        pass

    # Fallback: regex scan body text
    if stats["Balance"] == "N/A":
        try:
            body_text = self.driver.find_element(By.TAG_NAME, "body").text
            for line in body_text.split('\\n'):
                match = re.match(r'Balance:\\s*(\\$[\\d,]+\\.?\\d*)', line.strip())
                if match:
                    stats["Balance"] = match.group(1)
                match = re.match(r'Equity:\\s*(\\$[\\d,]+\\.?\\d*)', line.strip())
                if match:
                    stats["Equity"] = match.group(1)
        except Exception:
            pass

def _extract_drawdown_target(self, stats):
    """Extract drawdown and profit target info"""
    patterns = [
        ("Drawdown", ["drawdown", "max loss", "daily loss", "max drawdown"]),

```

```

        ("Profit Target", ["profit target", "target", "goal"]),
    ]

    for stat_key, keywords in patterns:
        for keyword in keywords:
            try:
                xpath = f"//*[contains(translate(text(), 'ABCDEFGHIJKLMNOPQRSTUVWXYZ', 'abcdefghijklmnopqrstuvwxyz'), '{keyword}')]"/>
                elements = self.driver.find_elements(By.XPATH, xpath)
                for el in elements:
                    # Check the element itself and its siblings/children for values
                    parent = el.find_element(By.XPATH, "..")
                    parent_text = parent.text

                    # Look for dollar amounts or percentages
                    money = re.search(r'-?\$[\d,]+\.\d*', parent_text)
                    pct = re.search(r'\d.\d+%', parent_text)

                    if money:
                        stats[stat_key] = money.group()
                        break
                    elif pct:
                        stats[stat_key] = pct.group()
                        break

                if stats[stat_key] != "N/A":
                    break
            except Exception:
                continue

    def _extract_phase_status(self, stats):
        """Extract account phase and status from the dashboard-card h3 title.
        Real DOM h3 text: "Futures Legacy Challenge | 50K | Account: FNFT..."
        Account Type p tag: "Account Type: Challenge Account"
        """
        try:
            h3s = self.driver.find_elements(By.CSS_SELECTOR, ".dashboard-card h3")
            for h3 in h3s:
                text = h3.text.strip()
                # Parse: "Futures Legacy Challenge | 50K | Account: FNFT..."
                title_match = re.match(r'(.+)\s*\|\s*(\w+)\s*\|', text)
                if title_match:
                    stats["Phase"] = title_match.group(1).strip()
                    stats["Size"] = title_match.group(2).strip()
                    break
        except Exception:
            pass

        # Status from Account Type <p> tag
        try:
            p_tags = self.driver.find_elements(By.CSS_SELECTOR, ".active-account-card p")
            for p in p_tags:
                text = p.text.strip()
                if text.startswith("Account Type:"):
                    acct_type = text.split(":", 1)[1].strip()
                    stats["Status"] = acct_type
                    break
        except Exception:
            pass

        # Section title (e.g. "General Account")

```

```

try:
    section = self.driver.find_elements(By.CSS_SELECTOR, ".account-list-title")
    if section:
        stats.setdefault("Section", section[0].text.strip())
except Exception:
    pass

def get_page_snapshot(self):
    """
    Debug helper: Get a text snapshot of the current page content.
    Useful for discovering DOM structure and available selectors.
    """
    try:
        if not self.driver:
            return "No driver available"

        snapshot = {
            "url": self.driver.current_url,
            "title": self.driver.title,
            "body_text": "",
            "tables": [],
            "cards": [],
            "dollar_values": [],
        }

        # Get full page text
        body = self.driver.find_element(By.TAG_NAME, "body")
        snapshot["body_text"] = body.text[:5000]

        # Find all dollar amounts
        dollar_matches = re.findall(r'-?\$[\d,]+\.\d*', body.text)
        snapshot["dollar_values"] = dollar_matches[:20]

        # Find tables
        tables = self.driver.find_elements(By.TAG_NAME, "table")
        for i, table in enumerate(tables[:5]):
            rows = table.find_elements(By.TAG_NAME, "tr")
            table_data = []
            for row in rows[:10]:
                cells = row.find_elements(By.CSS_SELECTOR, "td, th")
                table_data.append([c.text.strip() for c in cells])
            snapshot["tables"].append(table_data)

        # Find card-like elements
        card_selectors = ["[class*='card']", "[class*='Card']", "[class*='account']"]
        for sel in card_selectors:
            try:
                cards = self.driver.find_elements(By.CSS_SELECTOR, sel)
                for card in cards[:10]:
                    text = card.text.strip()
                    if text and len(text) > 10:
                        snapshot["cards"].append(text[:500])
            except Exception:
                continue

        return snapshot

    except Exception as e:
        return f"Snapshot failed: {e}"

def _default_stats(self, account_text="N/A"):

```

```

"""Return default stats dict"""
return {
    "Account Number": account_text,
    "Balance": "N/A",
    "Equity": "N/A",
    "Profit/Loss": "N/A",
    "Open Trades": "0",
    "Symbol": "",
    "Direction": "",
    "Drawdown": "N/A",
    "Profit Target": "N/A",
    "Phase": "N/A",
    "Status": "N/A",
}

def get_billing_history(self):
    """
    Scrape billing history from https://app.fundednext.com/billing/billing-history.
    Returns list of dicts with keys:
        sn, account_no, payment_method, invoice, status, date,
        transaction_id, transition_type, paid_amount, funding_package, payment_proof

    Uses the Ant Design table: .ant-table-wrapper.billing-page-table
    """
    billing_url = f"{self.base_url}/billing/billing-history"
    try:
        self.logger.info(f"[BILLING] Navigating to {billing_url}")
        self.driver.get(billing_url)
        time.sleep(3)

        # Wait for the billing table to render
        try:
            WebDriverWait(self.driver, 10).until(
                EC.presence_of_element_located((By.CSS_SELECTOR, ".ant-table-wrapper table"))
            )
        except TimeoutException:
            self.logger.warning("[BILLING] Table did not load in time")
            return []

        rows = self.driver.find_elements(By.CSS_SELECTOR,
            ".ant-table-wrapper table tbody tr.ant-table-row")
        if not rows:
            self.logger.info("[BILLING] No billing rows found")
            return []

        # Column order (confirmed from DOM exploration):
        # SN | Account No | Payment Method | Invoice | Status | Date |
        # Transaction ID | Transition Type | Paid Amount | Funding Package | Payment Proof
        col_keys = [
            "sn", "account_no", "payment_method", "invoice", "status",
            "date", "transaction_id", "transition_type", "paid_amount",
            "funding_package", "payment_proof"
        ]

        billing = []
        for row in rows:
            cells = row.find_elements(By.TAG_NAME, "td")
            entry = {}
            for i, key in enumerate(col_keys):
                if i < len(cells):

```



```

        entry[key] = cells[i].text.strip()
    else:
        entry[key] = ""
        # Clean paid_amount to numeric
        raw_amt = entry.get("paid_amount", "")
        cleaned = re.sub(r'^\d.', '', raw_amt)
        entry["paid_amount_numeric"] = float(cleaned) if cleaned else 0.0
        billing.append(entry)

    self.logger.info(f"[BILLING] Extracted {len(billing)} billing records")
    return billing

except Exception as e:
    self.logger.error(f"[BILLING] Failed to scrape billing: {e}")
    return []

def get_billing_via_api(self):
    """
    Fetch billing history via the FundedNext REST API instead of DOM scraping.
    Returns the same format as get_billing_history() but with richer data
    including the 'login' field (FundedNext internal account ID).

    API: GET /api/v1/pending-payment-history?email=...&type=1&page=1&limit=20
    Requires Bearer token from the tokenV1 cookie.
    """
    import json as _json
    try:
        # Get auth token from cookie
        token = self.driver.execute_script("""
            var c = document.cookie.split(';').find(function(c) {
                return c.trim().indexOf('tokenV1=') === 0;
            });
            return c ? decodeURIComponent(c.split('=')[1]) : null;
        """)
        if not token:
            self.logger.warning("[BILLING-API] No tokenV1 cookie found, falling back to DOM scrape")
            return self.get_billing_history()

        # Get user email from profile or cookie
        email = self.username
        if not email:
            profile = self.driver.execute_script("""
                try {
                    var u = JSON.parse(localStorage.getItem('user') || '{}');
                    return u.email || '';
                } catch(e) { return ''; }
            """)
            email = profile

        if not email:
            self.logger.warning("[BILLING-API] No email available, falling back to DOM scrape")
            return self.get_billing_history()

        # Call the billing API via the browser's fetch (avoids CORS issues)
        api_url = f"https://api.fundednext.com/api/v1/pending-payment-history?email={email}&type=1&page=1&limit=20"
        result = self.driver.execute_script("""
            var url = arguments[0];
            var token = arguments[1];
            try {
                var resp = await fetch(url, {

```

```

        headers: {
            'Authorization': 'Bearer ' + token,
            'Accept': 'application/json'
        }
    });
    var text = await resp.text();
    return {status: resp.status, body: text};
} catch(e) {
    return {error: e.toString()};
}
""" , api_url, token)

if not result or result.get('error'):
    self.logger.warning(f"[BILLING-API] Fetch failed: {result}")
    return self.get_billing_history()

if result.get('status', 0) != 200:
    self.logger.warning(f"[BILLING-API] HTTP {result.get('status')}, falling back to DOM")
    return self.get_billing_history()

data = _json.loads(result['body'])
items = data.get('data', {}).get('data', []) if isinstance(data.get('data'), dict) else data.get(
    'data', [])
if not items:
    self.logger.info("[BILLING-API] No billing records from API")
    return []

# Convert API format to the same format as get_billing_history()
billing = []
for item in items:
    entry = {
        "sn": str(item.get("id", "")),
        "account_no": str(item.get("login", "")),
        "payment_method": item.get("payment_method", ""),
        "invoice": item.get("invoice_path", ""),
        "status": "APPROVED" if item.get("status") == 1 else str(item.get("status", "")),
        "date": (item.get("created_at") or "")[:10], # "2026-03-30T..." -> "2026-03-30"
        "transaction_id": item.get("transaction_id", ""),
        "transition_type": item.get("payments_for", ""),
        "paid_amount": f"${item.get('paid_amount', 0):.2f}",
        "funding_package": item.get("funding_package", ""),
        "payment_proof": item.get("payment_proof", ""),
        "paid_amount_numeric": float(item.get("paid_amount", 0)),
        # Extra API fields not in DOM scrape
        "login": item.get("login"),
    }
    billing.append(entry)

self.logger.info(f"[BILLING-API] Got {len(billing)} billing records via API")
return billing

except Exception as e:
    self.logger.warning(f"[BILLING-API] API billing failed ({e}), falling back to DOM scrape")
    return self.get_billing_history()

def get_account_mapping(self):
    """
    Get the mapping from FundedNext billing login ID to Tradovate account name.

    Navigates to accounts page, clicks Futures tab, and extracts the

```

account data from React fiber props on dashboard cards.

Returns dict: {login_id: {"tradovate_account_name": "FNFT...", "plan_title": "...", ...}}

The React fiber on each .dashboard-card contains:

```
account.login -> 945576089
account.tradovate_account_name.tradovate_account_name -> "FNFTCHHARRISONOUKA85625"
account.plan.title -> "Futures Legacy Challenge | 50K"
account.balance, account.starting_balance, etc.
"""
mapping = {}
try:
    self.navigate_to_accounts()
    time.sleep(2)
    self.switch_type_tab("Futures")
    time.sleep(2)

    # Check for page error
    body_check = self.driver.execute_cdp_cmd("Runtime.evaluate", {
        "expression": "document.body.innerText.substring(0, 500)",
        "returnByValue": True,
        "timeout": 5000
    })
    body_text = body_check.get("result", {}).get("value", "")
    if "Something Went Wrong" in body_text:
        self.logger.warning("[MAPPING] Futures tab returned error page")
        return mapping

    # Extract React fiber data from all dashboard cards
    fiber_result = self.driver.execute_cdp_cmd("Runtime.evaluate", {
        "expression": """
        (function() {
            var cards = document.querySelectorAll('.dashboard-card');
            var results = [];
            for (var i = 0; i < cards.length; i++) {
                var keys = Object.keys(cards[i]);
                for (var j = 0; j < keys.length; j++) {
                    if (keys[j].indexOf('__reactFiber') !== -1) {
                        var node = cards[i][keys[j]];
                        for (var k = 0; k < 30 && node; k++) {
                            var p = node.memoizedProps;
                            if (p && p.account && p.account.login) {
                                var acct = p.account;
                                results.push({
                                    login: acct.login,
                                    account_id: acct.id,
                                    tradovate_name: (acct.tradovate_account_name || {
                                        }).tradovate_account_name || null,
                                    plan_title: (acct.plan || {}).title || null,
                                    balance: acct.balance,
                                    starting_balance: acct.starting_balance,
                                    server_type: (acct.server || {}).server_type || null,
                                    breached: acct.breached,
                                    breachedby: acct.breachedby || null
                                });
                                break;
                            }
                        }
                        node = node.return;
                    }
                }
            }
            return results;
        })
        """
    })
    break;
```

```

        }
    }
    }
    return JSON.stringify(results);
}())
"""
"returnByValue": True,
"timeout": 10000
})

import json as _json
raw = fiber_result.get("result", {}).get("value", "[[]]")
accounts = _json.loads(raw)

for acct in accounts:
    login_id = acct.get("login")
    tv_name = acct.get("tradovate_name")
    if login_id:
        mapping[str(login_id)] = {
            "tradovate_account_name": tv_name,
            "account_id": acct.get("account_id"),
            "plan_title": acct.get("plan_title"),
            "balance": acct.get("balance"),
            "starting_balance": acct.get("starting_balance"),
            "server_type": acct.get("server_type"),
            "breached": acct.get("breached"),
            "breachedby": acct.get("breachedby"),
        }
        self.logger.info(
            f"[MAPPING] login={login_id} -> tradovate={tv_name} "
            f"({acct.get('plan_title')})")

    self.logger.info(f"[MAPPING] Built mapping for {len(mapping)} account(s)")
    return mapping

except Exception as e:
    self.logger.error(f"[MAPPING] Failed to get account mapping: {e}")
    return mapping

def get_account_overview(self, account_id):
    """
    Get detailed account overview via the FundedNext API.

    API: GET /api/v1/account-overview?account_id={account_id}

    Returns dict with account_details (breached, breached_by, login, account_name, type, etc.),
    stats (balance, profit, win_rate), objectives, and breach_event_details.
    Returns None on failure.
    """
    import json as _json
    try:
        token = self.driver.execute_script("""
            var c = document.cookie.split(';').find(function(c) {
                return c.trim().indexOf('tokenV1=') === 0;
            });
            return c ? decodeURIComponent(c.split('=')[1]) : null;
        """)
        if not token:
            self.logger.warning("[OVERVIEW] No tokenV1 cookie found")
            return None
    
```

```

result = self.driver.execute_cdp_cmd("Runtime.evaluate", {
    "expression": f"""
        (async function() {{
            try {{
                var resp = await fetch(
                    'https://api.fundednext.com/api/v1/account-overview?account_id={account_id}',
                    {{headers: {{'Authorization': 'Bearer {token}',
                        'Accept': 'application/json'}}}}
                );
                return await resp.text();
            }} catch(e) {{
                return JSON.stringify({{error: e.toString()}});
            }}
        }})()
    """,
    "returnByValue": True,
    "awaitPromise": True,
    "timeout": 15000
})

raw = result.get("result", {}).get("value", "{}")
data = _json.loads(raw)

if data.get("error"):
    self.logger.warning(f"[OVERVIEW] API error for account_id={account_id}: {data['error']}")
    return None

overview = data.get("data", {})
if not overview:
    self.logger.warning(f"[OVERVIEW] Empty data for account_id={account_id}")
    return None

acct_details = overview.get("account_details", {})
self.logger.info(
    f"[OVERVIEW] account_id={account_id} | "
    f"name={acct_details.get('account_name')} | "
    f"breached={acct_details.get('breached')} | "
    f"breached_by={acct_details.get('breached_by')}")

return overview

except Exception as e:
    self.logger.error(f"[OVERVIEW] Failed for account_id={account_id}: {e}")
    return None

def disconnect(self):
    """Disconnect from FundedNext"""
    self.logged_in = False
    if self.driver:
        try:
            # If attached to existing Chrome, don't quit - just detach
            if self.attach_to_existing:
                self.logger.info("[DISCONNECT] Detaching from existing Chrome (not closing)")
                self.driver = None
            else:
                self.logger.info("[DISCONNECT] Closing Chrome instance")
                self.driver.quit()
                self.driver = None
        except Exception as e:
            self.logger.warning(f"[DISCONNECT] Error: {e}")

```

```

        self.driver = None

    # Remove from registry
    instance_key = f"fundednext_{self.username}_{self.pair_id}"
    self._chrome_instances.pop(instance_key, None)

    def close(self):
        """Close connection - alias for disconnect()"""
        self.disconnect()

```

Method: FundedNextAccount.__init__

```

def __init__(self, username=None, password=None, pair_id=None, attach_to_existing=True, debug_port=9444,
             use_real_profile=False)

```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.
- **__init__** — The constructor. Runs after Python has allocated the instance; assigns starting attribute values to self.

Step by step (top-level body)

1. Assigns `self.username = username` or `"`.
2. Assigns `self.password = password` or `"`.
3. Assigns `self.pair_id = pair_id` or `'default'`.
4. Assigns `self.attach_to_existing = attach_to_existing`.
5. Assigns `self.debug_port = debug_port`.
6. Assigns `self.use_real_profile = use_real_profile`.
7. Assigns `self.logged_in = False`.
8. Assigns `self.driver = None`.
9. Assigns `self._placing_order = False`.
10. Assigns `self._login_timestamp = None`.
11. Assigns `self._first_stats_fetch = True`.
12. Assigns `self.base_url = 'https://app.fundednext.com'`.
13. Assigns `self.accounts_url = 'https://app.fundednext.com/accounts'`.
14. Assigns `self.login_url = 'https://app.fundednext.com'`.
15. ... and 6 more statement(s) in the body.

Code (copy-paste safe)

```

def __init__(self, username=None, password=None, pair_id=None,
              attach_to_existing=True, debug_port=9444,
              use_real_profile=False):
    self.username = username or ""
    self.password = password or ""
    self.pair_id = pair_id or "default"
    self.attach_to_existing = attach_to_existing
    self.debug_port = debug_port
    self.use_real_profile = use_real_profile
    self.logged_in = False
    self.driver = None
    self._placing_order = False
    self._login_timestamp = None
    self._first_stats_fetch = True
    self.base_url = "https://app.fundednext.com"
    self.accounts_url = "https://app.fundednext.com/accounts"
    self.login_url = "https://app.fundednext.com"
    self.lock = threading.RLock()

    self.logger = logging.getLogger(f"FundedNext_{self.username}_{self.pair_id}")
    self.logger.info(
        f"[INIT] FundedNextAccount initializing for user={username}, pair_id={pair_id}, attach={attach_to_existing}"
    )

    instance_key = f"fundednext_{username}_{self.pair_id}"

    # Reuse existing Chrome instance if available
    if instance_key in self._chrome_instances:
        self.logger.info(f"[INIT] Reusing existing Chrome instance for FundedNext: {username}")
        existing_driver = self._chrome_instances[instance_key]
        try:
            existing_driver.current_url
            self.driver = existing_driver
            self.logger.info("[INIT] Successfully connected to existing Chrome instance")
            return
        except Exception as e:
            self.logger.info(f"[INIT] Existing Chrome instance dead ({e}), creating new one")
            del self._chrome_instances[instance_key]

    try:
        self.driver = self._initialize_driver()
        self._chrome_instances[instance_key] = self.driver
        self.logger.info(f"[INIT] Chrome instance registered for FundedNext: {username}")
    except Exception as e:
        self.logger.error(f"[INIT ERROR] Failed to initialize WebDriver: {e}")
        raise Exception(f"Failed to initialize WebDriver: {e}")

```

Method: FundedNextAccount._get_chromedriver_path

```
def _get_chromedriver_path(self)
```

Let Selenium Manager handle ChromeDriver automatically

Step by step (top-level body)

1. return None.

Code (copy-paste safe)

```

def _get_chromedriver_path(self):
    """Let Selenium Manager handle ChromeDriver automatically"""

```

```
return None
```

Method: FundedNextAccount._copy_chrome_cookies

```
def _copy_chrome_cookies(self, chrome_profile_src, dest_user_data_dir)
```

Copy Chrome login cookies from user's profile to temp profile

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Imports `shutil` (lazy import inside the function).
2. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def _copy_chrome_cookies(self, chrome_profile_src, dest_user_data_dir):
    """Copy Chrome login cookies from user's profile to temp profile"""
    import shutil
    try:
        src_default = os.path.join(chrome_profile_src, "Default")
        dst_default = os.path.join(dest_user_data_dir, "Default")
        os.makedirs(dst_default, exist_ok=True)

        # Copy cookie-related files
        cookie_files = ["Cookies", "Cookies-journal", "Login Data", "Login Data-journal",
                        "Web Data", "Web Data-journal", "Preferences", "Secure Preferences"]
        for fname in cookie_files:
            src = os.path.join(src_default, fname)
            dst = os.path.join(dst_default, fname)
            if os.path.exists(src) and not os.path.exists(dst):
                try:
                    shutil.copy2(src, dst)
                    self.logger.info(f"[PROFILE] Copied {fname}")
                except Exception as e:
                    self.logger.debug(f"[PROFILE] Could not copy {fname}: {e}")

        # Copy Local State for encryption keys
        src_local = os.path.join(chrome_profile_src, "Local State")
        dst_local = os.path.join(dest_user_data_dir, "Local State")
        if os.path.exists(src_local) and not os.path.exists(dst_local):
            shutil.copy2(src_local, dst_local)
            self.logger.info("[PROFILE] Copied Local State (encryption keys)")

    except Exception as e:
        self.logger.warning(f"[PROFILE] Cookie copy failed: {e}")
```

Method: FundedNextAccount._initialize_driver

```
def _initialize_driver(self)
```


Initialize Chrome WebDriver - attach to existing or launch new with profile cookies

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.

Step by step (top-level body)

1. Assigns `chrome_options = Options()`.
2. `if self.attach_to_existing`: branches conditionally.
3. Assigns `chrome_profile_src = os.path.join(os.environ.get('LOCALAPPDATA', ''), 'Google'....`
4. `if self.use_real_profile`: branches conditionally (with an **else/elif** arm).
5. Calls `chrome_options.add_argument(...)` for its side effect.
6. `if not self.use_real_profile`: branches conditionally.
7. Calls `chrome_options.add_argument(...)` for its side effect.
8. Calls `chrome_options.add_argument(...)` for its side effect.
9. Calls `chrome_options.add_argument(...)` for its side effect.
10. Calls `chrome_options.add_argument(...)` for its side effect.
11. Calls `chrome_options.add_argument(...)` for its side effect.
12. Calls `chrome_options.add_argument(...)` for its side effect.
13. Calls `chrome_options.add_argument(...)` for its side effect.
14. Calls `chrome_options.add_argument(...)` for its side effect.
15. ... and 6 more statement(s) in the body.

Code (copy-paste safe)

```
def _initialize_driver(self):
    """Initialize Chrome WebDriver - attach to existing or launch new with profile cookies"""
    chrome_options = Options()

    if self.attach_to_existing:
        # === ATTACH MODE: Connect to already-running Chrome ===
        self.logger.info(f"[DRIVER] Attaching to existing Chrome on port {self.debug_port}...")
        chrome_options.add_experimental_option("debuggerAddress", f"127.0.0.1:{self.debug_port}")

    try:
        driver = webdriver.Chrome(options=chrome_options)
        self.logger.info(f"[DRIVER] Attached to Chrome. Current URL: {driver.current_url}")

        if "fundednext.com" in driver.current_url:
            self.logged_in = True
            self._login_timestamp = time.time()
```

```

        self.logger.info("[DRIVER] Already on FundedNext - session active")

    return driver

except Exception as e:
    self.logger.warning(f"[DRIVER] Failed to attach ({e}). Falling back to profile mode...")
    self.attach_to_existing = False
    chrome_options = Options()

# === PROFILE MODE: Launch Chrome with COPY of user's profile for cookies ===
# This reuses login cookies from the user's normal Chrome session
chrome_profile_src = os.path.join(os.environ.get('LOCALAPPDATA', ''),
                                  'Google', 'Chrome', 'User Data')

if self.use_real_profile:
    # USE REAL PROFILE: requires closing all other Chrome instances first
    self.logger.info("[DRIVER] Using real Chrome profile (requires no other Chrome running)")
    user_data_dir = chrome_profile_src
else:
    unique_id = f"fundednext_{self.username}_{self.pair_id}"
    unique_hash = hashlib.md5(unique_id.encode()).hexdigest()[:8]
    user_data_dir = os.path.join(tempfile.gettempdir(), f"chrome_fundednext_{unique_hash}")

    # Copy the Default profile's Cookies file if our temp dir is empty
    profile_default = os.path.join(user_data_dir, "Default")
    if not os.path.exists(os.path.join(profile_default, "Cookies")):
        self._copy_chrome_cookies(chrome_profile_src, user_data_dir)

chrome_options.add_argument(f"--user-data-dir={user_data_dir}")

if not self.use_real_profile:
    debug_port = 9500 + (int(hashlib.md5(f"fundednext_{self.username}_{self.pair_id}".encode(
    )).hexdigest()[:4], 16) % 50)
    chrome_options.add_argument(f"--remote-debugging-port={debug_port}")

# Standard Chrome options
chrome_options.add_argument("--no-sandbox")
chrome_options.add_argument("--disable-dev-shm-usage")
chrome_options.add_argument("--disable-gpu")
chrome_options.add_argument("--disable-extensions")
chrome_options.add_argument("--no-first-run")
chrome_options.add_argument("--no-default-browser-check")
chrome_options.add_argument("--disable-features=TranslateUI")
chrome_options.add_argument("--window-size=1200,800")

# Anti-detection
chrome_options.add_experimental_option("excludeSwitches", ["enable-automation", "enable-logging"])
chrome_options.add_experimental_option('useAutomationExtension', False)
chrome_options.add_argument("--disable-blink-features=AutomationControlled")

prefs = {
    "profile.default_content_setting_values": {
        "popups": 2,
        "notifications": 2,
    }
}
chrome_options.add_experimental_option("prefs", prefs)

try:
    self.logger.info("[CHROME] Launching new Chrome instance for FundedNext...")

```

```

start_time = time.time()
driver = webdriver.Chrome(options=chrome_options)
elapsed = time.time() - start_time
self.logger.info(f"[CHROME] Chrome launched in {elapsed:.1f}s")

driver.set_page_load_timeout(30)
driver.implicitly_wait(2)
driver.execute_script("Object.defineProperty(navigator, 'webdriver', {get: () => undefined})")

return driver

except Exception as e:
    self.logger.error(f"[DRIVER ERROR] Failed to initialize Chrome: {e}")
    raise

```

Method: FundedNextAccount.login

```
def login(self, max_retries=3)
```

Login to FundedNext platform

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to `sum`, `any`, `all`, or `max` where the intermediate list would be wasted.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **with self.lock**: enters a context manager.

Code (copy-paste safe)

```

def login(self, max_retries=3):
    """Login to FundedNext platform"""
    with self.lock:
        # If attached to existing session, just verify we're on FundedNext
        if self.attach_to_existing and self.driver:
            try:
                current_url = self.driver.current_url
                if "fundednext.com" in current_url:
                    self.logged_in = True
                    self._login_timestamp = time.time()
                    self.logger.info(f"[LOGIN] Already logged in via existing Chrome: {current_url}")
                    return True
            except Exception:
                pass

        if not self.username or not self.password:
            self.logger.error("Username and password required for FundedNext login")

```

```

        return False

    for attempt in range(max_retries):
        try:
            self.logger.info(f"[LOGIN] FundedNext login attempt {attempt + 1}/{max_retries}")

            if not self.driver:
                self.driver = self._initialize_driver()

            self.driver.get(self.login_url)
            time.sleep(3)

            wait = WebDriverWait(self.driver, 15)

            # FundedNext login form - email and password
            email_field = wait.until(
                EC.presence_of_element_located((By.CSS_SELECTOR,
                    "input[type='email'], input[name='email'], input[placeholder*='mail']"))
            )
            email_field.send_keys(Keys.CONTROL + "a")
            time.sleep(0.2)
            email_field.send_keys(self.username)

            password_field = self.driver.find_element(By.CSS_SELECTOR,
                "input[type='password'], input[name='password']")
            password_field.send_keys(Keys.CONTROL + "a")
            time.sleep(0.2)
            password_field.send_keys(self.password)

            # Click login button
            login_button = self.driver.find_element(By.XPATH,
                "//button[@type='submit'] | //button[contains(text(), 'Login')] | //button[contains(
            login_button.click()

            # Wait for redirect
            time.sleep(5)

            max_wait = 20
            start_time = time.time()
            while time.time() - start_time < max_wait:
                current_url = self.driver.current_url.lower()

                if any(kw in current_url for kw in ["accounts", "dashboard", "overview"]):
                    self.logged_in = True
                    self._login_timestamp = time.time()
                    self.logger.info(f"[LOGIN] FundedNext login successful: {current_url}")
                    return True

            # Check for error messages
            try:
                error_el = self.driver.find_element(By.CSS_SELECTOR,
                    "[class*='error'], [class*='alert-danger'], .text-danger")
                if error_el.is_displayed():
                    self.logger.warning(f"[LOGIN] Error: {error_el.text}")
                    break
            except NoSuchElementException:
                pass

            time.sleep(1)

        self.logger.warning(f"[LOGIN] Attempt {attempt + 1} timed out")

```

```

        except Exception as e:
            self.logger.error(f"[LOGIN] Attempt {attempt + 1} failed: {e}")
            time.sleep(3)

    return False

```

Method: FundedNextAccount.connect

```

def connect(self)
    Connect to FundedNext - alias for login()

```

Step by step (top-level body)

1. **return** self.login().

Code (copy-paste safe)

```

def connect(self):
    """Connect to FundedNext - alias for login()"""
    return self.login()

```

Method: FundedNextAccount.navigate_to_accounts

```

def navigate_to_accounts(self)
    Navigate to the accounts page if not already there

```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def navigate_to_accounts(self):
    """Navigate to the accounts page if not already there"""
    try:
        current_url = self.driver.current_url
        if "/accounts" in current_url:
            return True

        self.logger.info("[NAV] Navigating to accounts page...")
        self.driver.get(self.accounts_url)
        time.sleep(3)

        # Wait for the account-wrapper to load (confirmed selector from real DOM)
        WebDriverWait(self.driver, 15).until(
            EC.presence_of_element_located((By.CSS_SELECTOR, ".account-wrapper"))
        )
        return True

```

```

except Exception as e:
    self.logger.error(f"[NAV] Failed to navigate to accounts: {e}")
    return False

```

Method: FundedNextAccount.switch_type_tab

```
def switch_type_tab(self, tab_name='CFDs')
```

*Switch between CFDs and Futures tabs. FundedNext uses ant-tabs: .ant-tabs-tab > .ant-tabs-tab-btn
Uses CDP Runtime.evaluate to avoid Selenium page-load hangs on tab switch.*

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def switch_type_tab(self, tab_name="CFDs"):
    """
    Switch between CFDs and Futures tabs.
    FundedNext uses ant-tabs: .ant-tabs-tab > .ant-tabs-tab-btn
    Uses CDP Runtime.evaluate to avoid Selenium page-load hangs on tab switch.
    """
    try:
        # Use CDP to click tab – avoids Selenium hanging when the tab
        # triggers a React Server Component transition
        result = self.driver.execute_cdp_cmd("Runtime.evaluate", {
            "expression": f"""
                (function() {{
                    var tabs = document.querySelectorAll('.ant-tabs-tab-btn');
                    for (var i = 0; i < tabs.length; i++) {{
                        if (tabs[i].textContent.trim() === '{tab_name}') {{
                            tabs[i].click();
                            return 'clicked';
                        }}
                    }}
                    return 'not_found';
                }})()
            """,
            "returnByValue": True,
            "timeout": 5000
        })
        status = result.get("result", {}).get("value", "")
        if status == "clicked":
            time.sleep(3)
            self.logger.info(f"[NAV] Switched to type tab: {tab_name}")
            return True
        else:
            self.logger.warning(f"[NAV] Type tab '{tab_name}' not found via CDP")
            return False
    except Exception as e:

```

```

self.logger.warning(f"[NAV] CDP tab switch failed ({e}), trying Selenium fallback")
try:
    tabs = self.driver.find_elements(By.CSS_SELECTOR, ".ant-tabs-tab")
    for tab in tabs:
        if tab.text.strip() == tab_name:
            tab.click()
            time.sleep(2)
            self.logger.info(f"[NAV] Switched to type tab via Selenium: {tab_name}")
            return True
except Exception as e2:
    self.logger.error(f"[NAV] Selenium fallback also failed: {e2}")
return False

```

Method: FundedNextAccount.switch_status_tab

```
def switch_status_tab(self, status='Active')
```

Switch between Active/Inactive/Breached status tabs. These are buttons inside .account-wrapper__create-account with Tailwind classes.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def switch_status_tab(self, status="Active"):
    """
    Switch between Active/Inactive/Breached status tabs.
    These are buttons inside .account-wrapper__create-account with Tailwind classes.
    """
    try:
        buttons = self.driver.find_elements(
            By.CSS_SELECTOR, ".account-wrapper__create-account button")
        for btn in buttons:
            if btn.text.strip() == status:
                btn.click()
                time.sleep(2)
                self.logger.info(f"[NAV] Switched to status tab: {status}")
                return True
        self.logger.warning(f"[NAV] Status tab '{status}' not found")
        return False
    except Exception as e:
        self.logger.error(f"[NAV] Failed to switch status tab: {e}")
        return False

```

Method: FundedNextAccount.has_accounts

```
def has_accounts(self)
```

Check if the current tab view has any accounts. FundedNext shows .no-account-wrapper when empty.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def has_accounts(self):
    """
    Check if the current tab view has any accounts.
    FundedNext shows .no-account-wrapper when empty.
    """
    try:
        no_acct = self.driver.find_elements(By.CSS_SELECTOR, ".no-account-wrapper")
        if no_acct and no_acct[0].is_displayed():
            return False
        return True
    except Exception:
        return False
```

Method: FundedNextAccount.is_connected

```
def is_connected(self)

    Check if connected to FundedNext
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def is_connected(self):
    """Check if connected to FundedNext"""
    try:
        if not self.driver:
            return False
        current_url = self.driver.current_url
        return "fundednext.com" in current_url
    except Exception:
        return False
```

Method: FundedNextAccount.get_account_stats

```
@retry_on_stale_element(max_retries=3, delay=1)
def get_account_stats(self)
```


Get FundedNext account statistics from the dashboard. Returns dict with same keys as Tradovate/TopStepX for GUI compatibility: Account Number, Balance, Profit/Loss, Open Trades, Symbol, Direction Plus FundedNext-specific fields: Equity, Drawdown, Profit Target, Phase, Status

Why these Python concepts were used

- **@retry_on_stale_element** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **if** not self.lock.acquire(blocking=False): branches conditionally.
2. **try** block with 1 **except** clause, plus a **finally**.

Code (copy-paste safe)

```
def get_account_stats(self):
    """
    Get FundedNext account statistics from the dashboard.
    Returns dict with same keys as Tradovate/TopStepX for GUI compatibility:
    Account Number, Balance, Profit/Loss, Open Trades, Symbol, Direction
    Plus FundedNext-specific fields:
    Equity, Drawdown, Profit Target, Phase, Status
    """
    if not self.lock.acquire(blocking=False):
        return getattr(self, '_cached_stats', self._default_stats("Trading..."))

    try:
        if getattr(self, '_placing_order', False):
            return getattr(self, '_cached_stats', self._default_stats("Trading..."))

        # Cache TTL
        cache_ttl = 2.0
        current_time = time.time()
        last_fetch = getattr(self, '_stats_last_fetch_time', 0)
        if (current_time - last_fetch) < cache_ttl and hasattr(self, '_cached_stats'):
            return self._cached_stats

        if not self.is_connected():
            return self._default_stats("Not Connected")

        # Ensure we're on the accounts page
        self.navigate_to_accounts()

        stats = {
            "Account Number": "Unknown",
            "Balance": "N/A",
            "Equity": "N/A",
            "Profit/Loss": "N/A",
            "Open Trades": "0",
            "Symbol": "",
            "Direction": "",
```

```

        "Drawdown": "N/A",
        "Profit Target": "N/A",
        "Phase": "N/A",
        "Status": "N/A",
    }

    try:
        self._extract_account_info(stats)
    except Exception as e:
        self.logger.warning(f"Could not extract account info: {e}")

    try:
        self._extract_balance_equity(stats)
    except Exception as e:
        self.logger.warning(f"Could not extract balance/equity: {e}")

    try:
        self._extract_drawdown_target(stats)
    except Exception as e:
        self.logger.warning(f"Could not extract drawdown/target: {e}")

    try:
        self._extract_phase_status(stats)
    except Exception as e:
        self.logger.warning(f"Could not extract phase/status: {e}")

    self._cached_stats = stats
    self._stats_last_fetch_time = time.time()
    return stats

except Exception as e:
    self.logger.error(f"Failed to get FundedNext stats: {e}")
    return self._default_stats("Error")
finally:
    self.lock.release()

```

Method: FundedNextAccount.get_all_accounts

```
def get_all_accounts(self)
```

Get statistics for ALL accounts listed on the FundedNext accounts page. Checks both Futures and CFDs tabs (Futures first), Active status. Returns a list of dicts, one per account.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **if** not self.is_connected(): branches conditionally.
2. **with** self.lock: enters a context manager.

Code (copy-paste safe)

```
def get_all_accounts(self):
    """
    Get statistics for ALL accounts listed on the FundedNext accounts page.
    Checks both Futures and CFDs tabs (Futures first), Active status.
    Returns a list of dicts, one per account.
    """
    if not self.is_connected():
        return []

    with self.lock:
        try:
            self.navigate_to_accounts()
            time.sleep(2)

            accounts = []

            # Check both type tabs – Futures first (primary use case)
            for type_tab in ["Futures", "CFDs"]:
                self.switch_type_tab(type_tab)
                self.switch_status_tab("Active")
                time.sleep(1)

                if not self.has_accounts():
                    self.logger.info(f"[ACCOUNTS] No active accounts under {type_tab}")
                    continue

                acct_elements = self._find_account_elements()
                for i, element in enumerate(acct_elements):
                    try:
                        acct_stats = self._parse_account_element(element, i)
                        if acct_stats:
                            acct_stats["Type"] = type_tab
                            accounts.append(acct_stats)
                    except Exception as e:
                        self.logger.warning(f"[ACCOUNTS] Failed to parse account {i}: {e}")

                self.logger.info(f"[ACCOUNTS] Extracted {len(accounts)} accounts total")
                return accounts

        except Exception as e:
            self.logger.error(f"[ACCOUNTS] Failed to get all accounts: {e}")
            return []
```

Method: FundedNextAccount._find_account_elements

```
def _find_account_elements(self)
```

Find account card elements in the current tab view. Real DOM structure (confirmed):

```
.account-wrapper__content .tw-w-full .dashboard-card <-- THE CARD h3 (title: "Futures Legacy  
Challenge | 50K | Account: FNFT...") .active-account-card .border-right p "Server Type: TRADOVATE" p  
"Balance: $49407.6" div p "Equity: $49,407.60" p "Account Type: Challenge Account"  
button.activeBusinessType "Dashboard"
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.
2. **try** block with 1 **except** clause.
3. Calls `self.logger.info(...)` for its side effect.
4. **return []**.

Code (copy-paste safe)

```
def _find_account_elements(self):
    """Find account card elements in the current tab view.

    Real DOM structure (confirmed):
    .account-wrapper__content
    .tw-w-full
    .dashboard-card          <-- THE CARD
    h3 (title: "Futures Legacy Challenge | 50K | Account: FNFT...")
    .active-account-card
    .border-right
    p "Server Type: TRADOVATE"
    p "Balance: $49407.6"
    div
    p "Equity: $49,407.60"
    p "Account Type: Challenge Account"
    button.activeBusinessType "Dashboard"
    """
    try:
        cards = self.driver.find_elements(By.CSS_SELECTOR, ".dashboard-card")
        if cards:
            self.logger.info(f"[ACCOUNTS] Found {len(cards)} .dashboard-card elements")
            return cards
    except Exception:
        pass

    # Fallback: any element inside account-wrapper with dollar values
    try:
        cards = self.driver.find_elements(
            By.XPATH, "//*[@class='account-wrapper__content']//div[./p[contains(text(), '$')]]")
        if cards:
            self.logger.info(f"[ACCOUNTS] Found {len(cards)} cards via Balance fallback")
            return cards
    except Exception:
        pass

    self.logger.info("[ACCOUNTS] No account elements found")
    return []
```

Method: FundedNextAccount._parse_account_element

```
def _parse_account_element(self, element, index)
```

Parse a .dashboard-card element into a stats dict. Card text format (confirmed): Line 0: "Futures Legacy Challenge | 50K | Account: FNFTCHHARRISONOUKA85625" Line 1: "Server Type: TRADOVATE" Line 2: "Balance: \$49407.6" Line 3: "Equity: \$49,407.60" Line 4: "Account Type: Challenge Account"

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **list comprehension** — Builds a list in a single expression ([f(x) for x in xs]). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. try block with 1 **except** clause.

Code (copy-paste safe)

```
def _parse_account_element(self, element, index):
    """Parse a .dashboard-card element into a stats dict.

    Card text format (confirmed):
    Line 0: "Futures Legacy Challenge | 50K | Account: FNFTCHHARRISONOUKA85625"
    Line 1: "Server Type: TRADOVATE"
    Line 2: "Balance: $49407.6"
    Line 3: "Equity: $49,407.60"
    Line 4: "Account Type: Challenge Account"
    """
    try:
        text = element.text
        if not text.strip():
            return None

        stats = {
            "Account Number": "Unknown",
            "Balance": "N/A",
            "Equity": "N/A",
            "Profit/Loss": "N/A",
            "Drawdown": "N/A",
            "Profit Target": "N/A",
            "Phase": "N/A",
            "Status": "N/A",
            "Server Type": "N/A",
            "Account Type": "N/A",
            "Challenge": "N/A",
            "Size": "N/A",
        }

        lines = [l.strip() for l in text.split('\n') if l.strip()]

        for line in lines:
            # Title line: "Futures Legacy Challenge | 50K | Account: FNFT..."
            acct_id = re.search(r'FNFT\w+', line)
            if acct_id:
                stats["Account Number"] = acct_id.group()
                # Parse challenge info from title
```

```

        title_match = re.match(r'(.+?)\s*\|\s*(\w+)\s*\|\s*Account:', line)
        if title_match:
            stats["Challenge"] = title_match.group(1).strip()
            stats["Size"] = title_match.group(2).strip()
            continue

    # Labeled values: "Balance: $49407.6", "Equity: $49,407.60" etc.
    labeled = re.match(r'^([A-Za-z ]+)\s*(.+)$', line)
    if labeled:
        label = labeled.group(1).strip().lower()
        value = labeled.group(2).strip()

        if label == 'balance':
            stats["Balance"] = value
        elif label == 'equity':
            stats["Equity"] = value
        elif label == 'server type':
            stats["Server Type"] = value
        elif label == 'account type':
            stats["Account Type"] = value
            # Derive status from account type
            if 'challenge' in value.lower():
                stats["Status"] = "Challenge"
            elif 'funded' in value.lower():
                stats["Status"] = "Funded"
        elif 'drawdown' in label or 'max loss' in label:
            stats["Drawdown"] = value
        elif 'target' in label or 'profit target' in label:
            stats["Profit Target"] = value

    # Set Phase from Challenge name if available
    if stats["Phase"] == "N/A" and stats["Challenge"] != "N/A":
        stats["Phase"] = stats["Challenge"]

    return stats

except Exception as e:
    self.logger.warning(f"[PARSE] Failed to parse element {index}: {e}")
    return None

```

Method: FundedNextAccount._extract_account_info

```
def _extract_account_info(self, stats)
```

Extract account number from dashboard. Real DOM: FNFT ID is in a colored inside h3 within .dashboard-card

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

2. try block with 1 except clause.

Code (copy-paste safe)

```
def _extract_account_info(self, stats):
    """Extract account number from dashboard.
    Real DOM: FNFT ID is in a colored <span> inside h3 within .dashboard-card
    """
    # Primary: span with FNFT ID inside .dashboard-card h3
    try:
        fnft_spans = self.driver.find_elements(
            By.CSS_SELECTOR, ".dashboard-card h3 span span")
        for span in fnft_spans:
            text = span.text.strip()
            if text.startswith("FNFT"):
                stats["Account Number"] = text
                self.logger.info(f"[ACCOUNT] Found account: {text}")
                return
    except Exception:
        pass

    # Fallback: search for FNFT pattern anywhere
    try:
        elements = self.driver.find_elements(By.XPATH, "//span[contains(text(), 'FNFT')]")
        for el in elements:
            text = el.text.strip()
            match = re.search(r'FNFT\\w+', text)
            if match:
                stats["Account Number"] = match.group()
                self.logger.info(f"[ACCOUNT] Found account (fallback): {match.group()}")
                return
    except Exception:
        pass
```

Method: FundedNextAccount._extract_balance_equity

```
def _extract_balance_equity(self, stats)
```

Extract balance and equity from dashboard. Real DOM: <p> tags inside .active-account-card with format "Balance: \$49407.6"

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

Step by step (top-level body)

1. try block with 1 except clause.
2. if stats['Balance'] == 'N/A': branches conditionally.

Code (copy-paste safe)

```
def _extract_balance_equity(self, stats):
    """Extract balance and equity from dashboard.
    Real DOM: <p> tags inside .active-account-card with format "Balance: $49407.6"
    """
    # Primary: p tags inside .active-account-card
```

```

try:
    p_tags = self.driver.find_elements(By.CSS_SELECTOR, ".active-account-card p")
    for p in p_tags:
        text = p.text.strip()
        labeled = re.match(r'^([A-Za-z /&]+):\s*(.+)$', text)
        if not labeled:
            continue
        label = labeled.group(1).strip().lower()
        value = labeled.group(2).strip()

        if label == 'balance':
            stats["Balance"] = value
        elif label == 'equity':
            stats["Equity"] = value
        elif label in ('server type',):
            stats.setdefault("Server Type", value)
        elif label in ('account type',):
            stats.setdefault("Account Type", value)
except Exception:
    pass

# Fallback: regex scan body text
if stats["Balance"] == "N/A":
    try:
        body_text = self.driver.find_element(By.TAG_NAME, "body").text
        for line in body_text.split('\n'):
            match = re.match(r'Balance:\s*(\${\d,}+\.\d*)', line.strip())
            if match:
                stats["Balance"] = match.group(1)
            match = re.match(r'Equity:\s*(\${\d,}+\.\d*)', line.strip())
            if match:
                stats["Equity"] = match.group(1)
    except Exception:
        pass

```

Method: FundedNextAccount._extract_drawdown_target

```
def _extract_drawdown_target(self, stats)
```

Extract drawdown and profit target info

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns patterns = [('Drawdown', ['drawdown', 'max loss', 'daily loss', 'max....
2. **for** (stat_key, keywords) in patterns: iterates.

Code (copy-paste safe)

```

def _extract_drawdown_target(self, stats):
    """Extract drawdown and profit target info"""

```



```

patterns = [
    ("Drawdown", ["drawdown", "max loss", "daily loss", "max drawdown"]),
    ("Profit Target", ["profit target", "target", "goal"]),
]

for stat_key, keywords in patterns:
    for keyword in keywords:
        try:
            xpath = f"//*[contains(translate(text(), 'ABCDEFGHIJKLMNOPQRSTUVWXYZ', 'abcdefghijklmnopqrstuvwxyz'), '{keyword}')]"/>
            elements = self.driver.find_elements(By.XPATH, xpath)
            for el in elements:
                # Check the element itself and its siblings/children for values
                parent = el.find_element(By.XPATH, "..")
                parent_text = parent.text

                # Look for dollar amounts or percentages
                money = re.search(r'-?\$[\d,]+\.\d*', parent_text)
                pct = re.search(r'\d.\d+%', parent_text)

                if money:
                    stats[stat_key] = money.group()
                    break
                elif pct:
                    stats[stat_key] = pct.group()
                    break

            if stats[stat_key] != "N/A":
                break
        except Exception:
            continue

```

Method: FundedNextAccount._extract_phase_status

```
def _extract_phase_status(self, stats)
```

Extract account phase and status from the dashboard-card h3 title. Real DOM h3 text: "Futures Legacy Challenge | 50K | Account: FNFT..." Account Type p tag: "Account Type: Challenge Account"

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

Step by step (top-level body)

1. **try** block with 1 **except** clause.
2. **try** block with 1 **except** clause.
3. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def _extract_phase_status(self, stats):
    """Extract account phase and status from the dashboard-card h3 title.
    Real DOM h3 text: "Futures Legacy Challenge | 50K | Account: FNFT..."
    Account Type p tag: "Account Type: Challenge Account"
    """
    try:

```

```

h3s = self.driver.find_elements(By.CSS_SELECTOR, ".dashboard-card h3")
for h3 in h3s:
    text = h3.text.strip()
    # Parse: "Futures Legacy Challenge | 50K | Account: FNFT..."
    title_match = re.match(r'(.+?)\s*\|\s*(\w+)\s*\|', text)
    if title_match:
        stats["Phase"] = title_match.group(1).strip()
        stats["Size"] = title_match.group(2).strip()
        break
except Exception:
    pass

# Status from Account Type <p> tag
try:
    p_tags = self.driver.find_elements(By.CSS_SELECTOR, ".active-account-card p")
    for p in p_tags:
        text = p.text.strip()
        if text.startswith("Account Type:"):
            acct_type = text.split(":", 1)[1].strip()
            stats["Status"] = acct_type
            break
except Exception:
    pass

# Section title (e.g. "General Account")
try:
    section = self.driver.find_elements(By.CSS_SELECTOR, ".account-list-title")
    if section:
        stats.setdefault("Section", section[0].text.strip())
except Exception:
    pass

```

Method: FundedNextAccount.get_page_snapshot

```
def get_page_snapshot(self)
```

Debug helper: Get a text snapshot of the current page content. Useful for discovering DOM structure and available selectors.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **list comprehension** — Builds a list in a single expression ([f(x) for x in xs]). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def get_page_snapshot(self):
    """
    Debug helper: Get a text snapshot of the current page content.

```

```

Useful for discovering DOM structure and available selectors.
"""
try:
    if not self.driver:
        return "No driver available"

    snapshot = {
        "url": self.driver.current_url,
        "title": self.driver.title,
        "body_text": "",
        "tables": [],
        "cards": [],
        "dollar_values": [],
    }

    # Get full page text
    body = self.driver.find_element(By.TAG_NAME, "body")
    snapshot["body_text"] = body.text[:5000]

    # Find all dollar amounts
    dollar_matches = re.findall(r'-?\$[\d,]+\.\d*', body.text)
    snapshot["dollar_values"] = dollar_matches[:20]

    # Find tables
    tables = self.driver.find_elements(By.TAG_NAME, "table")
    for i, table in enumerate(tables[:5]):
        rows = table.find_elements(By.TAG_NAME, "tr")
        table_data = []
        for row in rows[:10]:
            cells = row.find_elements(By.CSS_SELECTOR, "td, th")
            table_data.append([c.text.strip() for c in cells])
        snapshot["tables"].append(table_data)

    # Find card-like elements
    card_selectors = ["[class*='card']", "[class*='Card']", "[class*='account']"]
    for sel in card_selectors:
        try:
            cards = self.driver.find_elements(By.CSS_SELECTOR, sel)
            for card in cards[:10]:
                text = card.text.strip()
                if text and len(text) > 10:
                    snapshot["cards"].append(text[:500])
        except Exception:
            continue

    return snapshot

except Exception as e:
    return f"Snapshot failed: {e}"

```

Method: FundedNextAccount._default_stats

```
def _default_stats(self, account_text='N/A')
```

Return default stats dict

Step by step (top-level body)

1. return {'Account Number': account_text, 'Balance': 'N/A', 'Equity': 'N/A',....

Code (copy-paste safe)

```
def _default_stats(self, account_text="N/A"):
    """Return default stats dict"""
    return {
        "Account Number": account_text,
        "Balance": "N/A",
        "Equity": "N/A",
        "Profit/Loss": "N/A",
        "Open Trades": "0",
        "Symbol": "",
        "Direction": "",
        "Drawdown": "N/A",
        "Profit Target": "N/A",
        "Phase": "N/A",
        "Status": "N/A",
    }
```

Method: FundedNextAccount.get_billing_history

```
def get_billing_history(self)
```

Scrape billing history from <https://app.fundednext.com/billing/billing-history>. Returns list of dicts with keys: sn, account_no, payment_method, invoice, status, date, transaction_id, transition_type, paid_amount, funding_package, payment_proof Uses the Ant Design table: .ant-table-wrapper.billing-page-table

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns `billing_url = f'{self.base_url}/billing/billing-history'`.
2. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def get_billing_history(self):
    """
    Scrape billing history from https://app.fundednext.com/billing/billing-history.
    Returns list of dicts with keys:
        sn, account_no, payment_method, invoice, status, date,
        transaction_id, transition_type, paid_amount, funding_package, payment_proof

    Uses the Ant Design table: .ant-table-wrapper.billing-page-table
    """
    billing_url = f"{self.base_url}/billing/billing-history"
    try:
        self.logger.info(f"[BILLING] Navigating to {billing_url}")
        self.driver.get(billing_url)
        time.sleep(3)
```

```

# Wait for the billing table to render
try:
    WebDriverWait(self.driver, 10).until(
        EC.presence_of_element_located((By.CSS_SELECTOR, ".ant-table-wrapper table"))
    )
except TimeoutException:
    self.logger.warning("[BILLING] Table did not load in time")
    return []

rows = self.driver.find_elements(By.CSS_SELECTOR,
    ".ant-table-wrapper table tbody tr.ant-table-row")
if not rows:
    self.logger.info("[BILLING] No billing rows found")
    return []

# Column order (confirmed from DOM exploration):
# SN | Account No | Payment Method | Invoice | Status | Date |
# Transaction ID | Transition Type | Paid Amount | Funding Package | Payment Proof
col_keys = [
    "sn", "account_no", "payment_method", "invoice", "status",
    "date", "transaction_id", "transition_type", "paid_amount",
    "funding_package", "payment_proof"
]

billing = []
for row in rows:
    cells = row.find_elements(By.TAG_NAME, "td")
    entry = {}
    for i, key in enumerate(col_keys):
        if i < len(cells):
            entry[key] = cells[i].text.strip()
        else:
            entry[key] = ""
    # Clean paid_amount to numeric
    raw_amt = entry.get("paid_amount", "")
    cleaned = re.sub(r'^\d.', '', raw_amt)
    entry["paid_amount_numeric"] = float(cleaned) if cleaned else 0.0
    billing.append(entry)

self.logger.info(f"[BILLING] Extracted {len(billing)} billing records")
return billing

except Exception as e:
    self.logger.error(f"[BILLING] Failed to scrape billing: {e}")
    return []

```

Method: FundedNextAccount.get_billing_via_api

```
def get_billing_via_api(self)
```

Fetch billing history via the FundedNext REST API instead of DOM scraping. Returns the same format as `get_billing_history()` but with richer data including the 'login' field (FundedNext internal account ID). API: `GET /api/v1/pending-payment-history?email=...&type=1&page=1&limit=20` Requires Bearer token from the `tokenV1` cookie.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't

have to handle it.

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Imports json (lazy import inside the function).
2. try block with 1 **except** clause.

Code (copy-paste safe)

```
def get_billing_via_api(self):
    """
    Fetch billing history via the FundedNext REST API instead of DOM scraping.
    Returns the same format as get_billing_history() but with richer data
    including the 'login' field (FundedNext internal account ID).

    API: GET /api/v1/pending-payment-history?email=...&type=1&page=1&limit=20
    Requires Bearer token from the tokenV1 cookie.
    """
    import json as _json
    try:
        # Get auth token from cookie
        token = self.driver.execute_script("""
            var c = document.cookie.split(';').find(function(c) {
                return c.trim().indexOf('tokenV1=') === 0;
            });
            return c ? decodeURIComponent(c.split('=')[1]) : null;
        """)
        if not token:
            self.logger.warning("[BILLING-API] No tokenV1 cookie found, falling back to DOM scrape")
            return self.get_billing_history()

        # Get user email from profile or cookie
        email = self.username
        if not email:
            profile = self.driver.execute_script("""
                try {
                    var u = JSON.parse(localStorage.getItem('user') || '{}');
                    return u.email || '';
                } catch(e) { return ''; }
            """)
            email = profile

        if not email:
            self.logger.warning("[BILLING-API] No email available, falling back to DOM scrape")
            return self.get_billing_history()

        # Call the billing API via the browser's fetch (avoids CORS issues)
        api_url = f"https://api.fundednext.com/api/v1/pending-payment-history?email={email}&type=1&page=1&limit=20"
        result = self.driver.execute_script("""
            var url = arguments[0];
            var token = arguments[1];
            fetch(url, {
                method: 'GET',
                headers: {
                    'Authorization': 'Bearer ' + token
                }
            })
            .then(function(response) {
                return response.json();
            })
            .catch(function(error) {
                console.error('Error fetching billing history: ', error);
                return null;
            })
        """, api_url, token)
        return result
```

```

        try {
            var resp = await fetch(url, {
                headers: {
                    'Authorization': 'Bearer ' + token,
                    'Accept': 'application/json'
                }
            });
            var text = await resp.text();
            return {status: resp.status, body: text};
        } catch(e) {
            return {error: e.toString()};
        }
    """, api_url, token)

    if not result or result.get('error'):
        self.logger.warning(f"[BILLING-API] Fetch failed: {result}")
        return self.get_billing_history()

    if result.get('status', 0) != 200:
        self.logger.warning(f"[BILLING-API] HTTP {result.get('status')}, falling back to DOM")
        return self.get_billing_history()

    data = _json.loads(result['body'])
    items = data.get('data', {}).get('data', []) if isinstance(data.get('data'), dict) else data.get('data', [])
    if not items:
        self.logger.info("[BILLING-API] No billing records from API")
        return []

    # Convert API format to the same format as get_billing_history()
    billing = []
    for item in items:
        entry = {
            "sn": str(item.get("id", "")),
            "account_no": str(item.get("login", "")),
            "payment_method": item.get("payment_method", ""),
            "invoice": item.get("invoice_path", ""),
            "status": "APPROVED" if item.get("status") == 1 else str(item.get("status", "")),
            "date": (item.get("created_at") or "")[:10], # "2026-03-30T..." -> "2026-03-30"
            "transaction_id": item.get("transaction_id", ""),
            "transition_type": item.get("payments_for", ""),
            "paid_amount": f"${item.get('paid_amount', 0):.2f}",
            "funding_package": item.get("funding_package", ""),
            "payment_proof": item.get("payment_proof", ""),
            "paid_amount_numeric": float(item.get("paid_amount", 0)),
            # Extra API fields not in DOM scrape
            "login": item.get("login"),
        }
        billing.append(entry)

    self.logger.info(f"[BILLING-API] Got {len(billing)} billing records via API")
    return billing

except Exception as e:
    self.logger.warning(f"[BILLING-API] API billing failed ({e}), falling back to DOM scrape")
    return self.get_billing_history()

```

Method: FundedNextAccount.get_account_mapping

```
def get_account_mapping(self)
```

Get the mapping from FundedNext billing login ID to Tradovate account name. Navigates to accounts page, clicks Futures tab, and extracts the account data from React fiber props on dashboard cards. Returns dict: {login_id: {"tradovate_account_name": "FNFT...", "plan_title": "...", ...}} The React fiber on each .dashboard-card contains: account.login -> 945576089
account.tradovate_account_name.tradovate_account_name -> "FNFTCHHARRISONOUKA85625"
account.plan.title -> "Futures Legacy Challenge | 50K" account.balance, account.starting_balance, etc.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns mapping = {}.
2. try block with 1 except clause.

Code (copy-paste safe)

```
def get_account_mapping(self):
    """
    Get the mapping from FundedNext billing login ID to Tradovate account name.

    Navigates to accounts page, clicks Futures tab, and extracts the
    account data from React fiber props on dashboard cards.

    Returns dict: {login_id: {"tradovate_account_name": "FNFT...", "plan_title": "...", ...}}

    The React fiber on each .dashboard-card contains:
    account.login -> 945576089
    account.tradovate_account_name.tradovate_account_name -> "FNFTCHHARRISONOUKA85625"
    account.plan.title -> "Futures Legacy Challenge | 50K"
    account.balance, account.starting_balance, etc.
    """
    mapping = {}
    try:
        self.navigate_to_accounts()
        time.sleep(2)
        self.switch_type_tab("Futures")
        time.sleep(2)

        # Check for page error
        body_check = self.driver.execute_cdp_cmd("Runtime.evaluate", {
            "expression": "document.body.innerText.substring(0, 500)",
            "returnByValue": True,
            "timeout": 5000
        })
        body_text = body_check.get("result", {}).get("value", "")
        if "Something Went Wrong" in body_text:
            self.logger.warning("[MAPPING] Futures tab returned error page")
            return mapping

        # Extract React fiber data from all dashboard cards
        fiber_result = self.driver.execute_cdp_cmd("Runtime.evaluate", {
```



```

"expression": """
    (function() {
        var cards = document.querySelectorAll('.dashboard-card');
        var results = [];
        for (var i = 0; i < cards.length; i++) {
            var keys = Object.keys(cards[i]);
            for (var j = 0; j < keys.length; j++) {
                if (keys[j].indexOf('__reactFiber') !== -1) {
                    var node = cards[i][keys[j]];
                    for (var k = 0; k < 30 && node; k++) {
                        var p = node.memoizedProps;
                        if (p && p.account && p.account.login) {
                            var acct = p.account;
                            results.push({
                                login: acct.login,
                                account_id: acct.id,
                                tradovate_name: (acct.tradovate_account_name || {
                                    tradovate_account_name || null,
                                }).title || null,
                                balance: acct.balance,
                                starting_balance: acct.starting_balance,
                                server_type: (acct.server || {}).server_type || null,
                                breached: acct.breached,
                                breachedby: acct.breachedby || null
                            });
                            break;
                        }
                        node = node.return;
                    }
                    break;
                }
            }
        }
        return JSON.stringify(results);
    })()
    "",
    "returnByValue": True,
    "timeout": 10000
})

import json as _json
raw = fiber_result.get("result", {}).get("value", "[[]]")
accounts = _json.loads(raw)

for acct in accounts:
    login_id = acct.get("login")
    tv_name = acct.get("tradovate_name")
    if login_id:
        mapping[str(login_id)] = {
            "tradovate_account_name": tv_name,
            "account_id": acct.get("account_id"),
            "plan_title": acct.get("plan_title"),
            "balance": acct.get("balance"),
            "starting_balance": acct.get("starting_balance"),
            "server_type": acct.get("server_type"),
            "breached": acct.get("breached"),
            "breachedby": acct.get("breachedby"),
        }
    self.logger.info(
        f"[MAPPING] login={login_id} -> tradovate={tv_name} "

```

```

        f"({acct.get('plan_title')})")

    self.logger.info(f"[MAPPING] Built mapping for {len(mapping)} account(s)")
    return mapping

except Exception as e:
    self.logger.error(f"[MAPPING] Failed to get account mapping: {e}")
    return mapping

```

Method: FundedNextAccount.get_account_overview

```
def get_account_overview(self, account_id)
```

Get detailed account overview via the FundedNext API. API: GET /api/v1/account-overview?account_id={account_id} Returns dict with account_details (breached, breached_by, login, account_name, type, etc.), stats (balance, profit, win_rate), objectives, and breach_event_details. Returns None on failure.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Imports json (lazy import inside the function).
2. try block with 1 **except** clause.

Code (copy-paste safe)

```

def get_account_overview(self, account_id):
    """
    Get detailed account overview via the FundedNext API.

    API: GET /api/v1/account-overview?account_id={account_id}

    Returns dict with account_details (breached, breached_by, login, account_name, type, etc.),
    stats (balance, profit, win_rate), objectives, and breach_event_details.
    Returns None on failure.
    """
    import json as _json
    try:
        token = self.driver.execute_script("""
            var c = document.cookie.split(';').find(function(c) {
                return c.trim().indexOf('tokenV1=') === 0;
            });
            return c ? decodeURIComponent(c.split('=')[1]) : null;
        """)
        if not token:
            self.logger.warning("[OVERVIEW] No tokenV1 cookie found")
            return None

        result = self.driver.execute_cdp_cmd("Runtime.evaluate", {
            "expression": f"""

```

```

        (async function() {{
            try {{
                var resp = await fetch(
                    'https://api.fundednext.com/api/v1/account-overview?account_id={account_id}',
                    {{headers: {{'Authorization': 'Bearer {token}',
                        'Accept': 'application/json'}}}}
                );
                return await resp.text();
            }} catch(e) {{
                return JSON.stringify({{error: e.toString()}});
            }}
        }})()
        """
        "returnByValue": True,
        "awaitPromise": True,
        "timeout": 15000
    })

    raw = result.get("result", {}).get("value", "{}")
    data = _json.loads(raw)

    if data.get("error"):
        self.logger.warning(f"[OVERVIEW] API error for account_id={account_id}: {data['error']}")
        return None

    overview = data.get("data", {})
    if not overview:
        self.logger.warning(f"[OVERVIEW] Empty data for account_id={account_id}")
        return None

    acct_details = overview.get("account_details", {})
    self.logger.info(
        f"[OVERVIEW] account_id={account_id} | "
        f"name={acct_details.get('account_name')} | "
        f"breached={acct_details.get('breached')} | "
        f"breached_by={acct_details.get('breached_by')}")

    return overview

except Exception as e:
    self.logger.error(f"[OVERVIEW] Failed for account_id={account_id}: {e}")
    return None

```

Method: FundedNextAccount.disconnect

```

def disconnect(self)
    Disconnect from FundedNext

```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `self.logged_in = False`.
2. `if self.driver:` branches conditionally.
3. Assigns `instance_key = f'fundednext_{self.username}_{self.pair_id}'`.
4. Calls `self._chrome_instances.pop(...)` for its side effect.

Code (copy-paste safe)

```
def disconnect(self):
    """Disconnect from FundedNext"""
    self.logged_in = False
    if self.driver:
        try:
            # If attached to existing Chrome, don't quit - just detach
            if self.attach_to_existing:
                self.logger.info("[DISCONNECT] Detaching from existing Chrome (not closing)")
                self.driver = None
            else:
                self.logger.info("[DISCONNECT] Closing Chrome instance")
                self.driver.quit()
                self.driver = None
        except Exception as e:
            self.logger.warning(f"[DISCONNECT] Error: {e}")
            self.driver = None

    # Remove from registry
    instance_key = f"fundednext_{self.username}_{self.pair_id}"
    self._chrome_instances.pop(instance_key, None)
```

Method: `FundedNextAccount.close`

```
def close(self)
    Close connection - alias for disconnect()
```

Step by step (top-level body)

1. Calls `self.disconnect(...)` for its side effect.

Code (copy-paste safe)

```
def close(self):
    """Close connection - alias for disconnect()"""
    self.disconnect()
```

Functions

```
def retry_on_stale_element(max_retries=3, delay=1)
```

Decorator to retry web operations that may fail due to stale elements

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.

Step by step (top-level body)

1. Defines a nested function decorator(...).
2. **return** decorator.

Code (copy-paste safe)

```
def retry_on_stale_element(max_retries=3, delay=1):
    """Decorator to retry web operations that may fail due to stale elements"""
    def decorator(func):
        @wraps(func)
        def wrapper(self, *args, **kwargs):
            for attempt in range(max_retries):
                try:
                    return func(self, *args, **kwargs)
                except (StaleElementReferenceException, NoSuchElementException) as e:
                    if attempt == max_retries - 1:
                        self.logger.error(f"Max retries reached for {func.__name__}: {e}")
                        raise e
                    self.logger.warning(
                        f"Retrying {func.__name__} due to stale element (attempt {attempt + 1})")
                    time.sleep(delay)
            return None
        return wrapper
    return decorator
```

```
def cdp_scrape(debug_port=9222)
```

Scrape FundedNext data via Chrome DevTools Protocol directly. Requires Chrome running with --remote-debugging-port=<port>. Falls back to Selenium if CDP connection fails.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Imports urllib.request (lazy import inside the function).
2. Imports json (lazy import inside the function).

3. try block with 1 except clause.

Code (copy-paste safe)

```
def cdp_scrape(debug_port=9222):
    """
    Scrape FundedNext data via Chrome DevTools Protocol directly.
    Requires Chrome running with --remote-debugging-port=<port>.
    Falls back to Selenium if CDP connection fails.
    """
    import urllib.request
    import json

    try:
        data = urllib.request.urlopen(f'http://127.0.0.1:{debug_port}/json', timeout=5).read()
        tabs = json.loads(data)
        fn_tab = next((t for t in tabs if 'fundednext' in t.get('url', '')), None)

        if not fn_tab:
            print("FundedNext tab not found. Tabs available:")
            for t in tabs:
                if t.get('type') == 'page':
                    print(f" {t['title']}: {t['url'][:80]}")
            return None

        import websocket
        ws = websocket.create_connection(fn_tab['webSocketDebuggerUrl'], timeout=10)

        # Get full page text
        ws.send(json.dumps({'id': 1, 'method': 'Runtime.evaluate',
                           'params': {'expression': 'document.body.innerText', 'returnByValue': True}}))
        body_text = json.loads(ws.recv())['result']['result']['value']

        # Get dollar values
        ws.send(json.dumps({'id': 2, 'method': 'Runtime.evaluate',
                           'params': {
                               'expression': '(document.body.innerText.match(/-?\\$[\\d,]+\\.?\\$\\s?/))',
                               'returnByValue': True}}))
        dollars = json.loads(ws.recv())['result']['result']['value']

        # Get card elements
        js_cards = '''(function() => {
            const cards = document.querySelectorAll(
                '[class*="card"]', [class*="Card"], [class*="account"], [class*="Account"]);
            const r = []; cards.forEach((c, i) => {
                if(c.innerText.trim().length > 20) r.push({i: i, cls: c.className.substring(0,200),
                    text: c.innerText.substring(0,600)});
            }); return JSON.stringify(r.slice(0, 15));
        })()'''
        ws.send(json.dumps({'id': 3, 'method': 'Runtime.evaluate',
                           'params': {'expression': js_cards, 'returnByValue': True}}))
        cards = json.loads(json.loads(ws.recv())['result']['result']['value'])

        ws.close()

        return {'url': fn_tab['url'], 'title': fn_tab['title'],
                'body_text': body_text, 'dollars': dollars, 'cards': cards}
    except Exception as e:
        print(f"CDP scrape failed: {e}")
        return None
```

```
def quick_test(debug_port=9222, mode='auto')
```

Quick test: Try CDP-direct first, then fall back to Selenium with fresh Chrome. mode: "cdp" = CDP only, "selenium" = Selenium only, "auto" = try both

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.

Step by step (top-level body)

1. Imports time (lazy import inside the function).
2. **if** mode in ('auto', 'cdp'): branches conditionally.
3. Calls print(...) for its side effect.
4. Calls print(...) for its side effect.
5. Assigns fn = FundedNextAccount(attach_to_existing=False, use_real_prof....
6. Calls fn.driver.get(...) for its side effect.
7. Calls time_mod.sleep(...) for its side effect.
8. Assigns current_url = fn.driver.current_url.
9. Calls print(...) for its side effect.
10. **if** 'fundednext.com/accounts' in current_url: branches conditionally (with an **else/elif** arm).
11. Calls print(...) for its side effect.
12. Calls print(...) for its side effect.
13. Assigns snapshot = fn.get_page_snapshot().
14. **if** isinstance(snapshot, dict): branches conditionally (with an **else/elif** arm).
15. ... and 7 more statement(s) in the body.

Code (copy-paste safe)

```
def quick_test(debug_port=9222, mode="auto"):
    """
    Quick test: Try CDP-direct first, then fall back to Selenium with fresh Chrome.
    mode: "cdp" = CDP only, "selenium" = Selenium only, "auto" = try both
    """
    import time as time_mod

    if mode in ("auto", "cdp"):
        print("=== Trying CDP-direct mode ===")
        cdp_data = cdp_scrape(debug_port)
        if cdp_data:
            print(f"\nURL: {cdp_data['url']}")
            print(f"Title: {cdp_data['title']}")
            print(f"\nDollar values:\n{cdp_data['dollars']}")
            print(f"\nCards ({len(cdp_data['cards'])}):")
            for c in cdp_data['cards'][:10]:
```

```

        print(f"\n--- Card {c['i']} (class: {c['cls'][:60]}) ---")
        print(c['text'][:400])
    print(f"\nBody text:\n{cdp_data['body_text'][:5000]}")
    return cdp_data
if mode == "cdp":
    print("CDP mode failed - Chrome debug port not available")
    return None

print("\n=== Selenium mode: launching fresh Chrome ===")
print("A new Chrome window will open. Please log in to FundedNext when prompted.")

fn = FundedNextAccount(attach_to_existing=False, use_real_profile=False)

# Navigate to FundedNext login
fn.driver.get("https://app.fundednext.com/accounts")
time_mod.sleep(3)

current_url = fn.driver.current_url
print(f"Current URL: {current_url}")

if "fundednext.com/accounts" in current_url:
    fn.logged_in = True
    print("Already logged in!")
else:
    print("\nPlease log in to FundedNext in the Chrome window that just opened.")
    print("After logging in and reaching the Accounts page, press Enter here...")
    input(">>> Press Enter when you're on the Accounts page: ")
    time_mod.sleep(2)

print(f"\nNow on: {fn.driver.current_url}")

print("\n=== Page Snapshot ===")
snapshot = fn.get_page_snapshot()
if isinstance(snapshot, dict):
    print(f"URL: {snapshot['url']}")
    print(f"Title: {snapshot['title']}")
    print(f"\nDollar values found: {snapshot['dollar_values']}")
    print(f"\nCards found: {len(snapshot['cards'])}")
    for i, card in enumerate(snapshot['cards'][:10]):
        print(f"\n--- Card {i} ---")
        print(card[:400])
    print(f"\nBody text (first 3000 chars):\n{snapshot['body_text'][:3000]}")
else:
    print(snapshot)

print("\n=== Account Stats ===")
stats = fn.get_account_stats()
for k, v in stats.items():
    print(f"  {k}: {v}")

print("\n=== All Accounts ===")
all_accts = fn.get_all_accounts()
for i, acct in enumerate(all_accts):
    print(f"\n--- Account {i+1} ---")
    for k, v in acct.items():
        print(f"  {k}: {v}")

return fn

```


Step 8.3 — Build trader_companion/topstepx.py

What to do. Create the file trader_companion/topstepx.py in your project root and paste in the code shown in this step. The file is **4420** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from requirements.txt.

What this file is for. TopstepX-specific scraper and account adapter.

trader_companion/topstepx.py

4420 loc · 1 classes · 3 functions · 24 imports

TopstepX-specific scraper and account adapter. It defines **1** class and **3** module-level functions, across **4,420** lines. Module docstring: *TopStepX Trading Platform Integration - MFFU Blueprint Compatible*

Module docstring

TopStepX Trading Platform Integration - MFFU Blueprint Compatible Handles connection and trading operations with TopStepX platform using MFFU blueprint structure

Python concepts demonstrated in this module

• Classes and objects • Comprehensions • Context managers (with-statements) • Decorators • Dunder methods • Exceptions • f-strings • Properties

Imports — what each one is for

```
import os
```

Why imported. Operating-system interface. Path manipulation, environment variables, working-directory operations, exit codes, and thin wrappers around POSIX/Windows syscalls.

Used in this file. os.getenv, os.path, os.path.dirname, os.path.join

Example. os.path.join(root, 'subdir', 'file.txt') # joins with the OS-correct separator

Other common scenarios.

- os.environ.get('API_KEY') to read environment variables
- os.makedirs(path, exist_ok=True) to create nested directories
- os.listdir(path) to list a directory's contents
- os.remove(path) / os.rename(src, dst) to delete or move a file
- os.getcwd() / os.chdir(path) to inspect or change the working dir

```
import sys
```

Why imported. Access to the running interpreter — argv, stdin/stdout, exit codes, the import search path, and the platform name.

Used in this file. `sys.path`, `sys.path.insert`

Example. `sys.exit(1)` # terminate the process with a non-zero status

Other common scenarios.

- `sys.argv[1:]` reads the command-line arguments
- `sys.path.insert(0, 'extra')` prepends to the import search path
- `sys.stderr.write(msg)` writes to standard error
- `sys.platform` tells you 'win32', 'linux', or 'darwin'
- `sys.modules` is the global cache of already-imported modules

`import time`

Why imported. Timestamps and sleep. Lower-level than `datetime` — works in Unix-epoch seconds.

Used in this file. `time.sleep`, `time.time`

Example. `time.sleep(1.5)` # pause this thread for 1.5 seconds

Other common scenarios.

- `time.time()` returns seconds since the epoch
- `time.monotonic()` for measuring elapsed time (never goes backward)
- `time.perf_counter()` for high-precision benchmarking
- `time.strftime` / `time.strptime` mirror the `datetime` versions

`import threading`

Why imported. Threads and synchronisation primitives. Used when work is I/O-bound and the GIL is not a bottleneck (the GIL prevents speed-up on CPU-bound work).

Used in this file. `threading.RLock`, `threading.Thread`

Example. `Thread(target=worker, args=(q,), daemon=True).start()`

Other common scenarios.

- Lock / RLock to guard shared state
- Event for one-shot signals between threads
- Queue (from `queue`) for producer/consumer patterns
- `threading.local()` for thread-local storage

`import logging`

Why imported. Structured, levelled logging. Replaces `print()` with filterable, formattable output that can route to files, `stderr`, `syslog`, or HTTP endpoints.

Used in this file. `logging.INFO`, `logging.basicConfig`, `logging.error`, `logging.getLogger`, `logging.info`, `logging.warning`

Example. `logging.getLogger(__name__).info('connected to %s', host)`

Other common scenarios.

- `.debug()` / `.info()` / `.warning()` / `.error()` / `.exception()`

- `logger.exception()` captures the current traceback automatically
- `logging.basicConfig(level=logging.INFO)` for quick setup
- Handlers and Formatters route logs to files / streams / syslog

`import random`

Why imported. Pseudo-random numbers — fine for jitter, sampling, shuffling. NEVER for security; use secrets for that.

Used in this file. *No direct attribute access detected — likely imported for side effects, re-export, or string references.*

Example. `random.uniform(0.5, 1.5)` # backoff jitter

Other common scenarios.

- `random.choice(seq)` picks one item
- `random.shuffle(list)` rearranges in place
- `random.seed(n)` makes the sequence reproducible
- `random.sample(seq, k)` draws k unique items

`import hashlib`

Why imported. Cryptographic hash functions — SHA-256, MD5, BLAKE2.

Used in this file. `hashlib.md5`

Example. `hashlib.sha256(b'hello').hexdigest()`

Other common scenarios.

- Pass data in chunks via `.update()` to hash large files
- Use sha256 / sha512 for integrity; never MD5 or SHA-1 for security
- For password hashing use `bcrypt/argon2`, NOT raw `hashlib`

`import tempfile`

Why imported. Temporary files and directories that clean themselves up.

Used in this file. `tempfile.gettempdir`

Example. `with tempfile.TemporaryDirectory() as d: ...`

Other common scenarios.

- `tempfile.NamedTemporaryFile(delete=False)` for a real path on disk
- `tempfile.mkstemp()` returns a low-level (fd, path) pair

`from selenium import webdriver`

Why imported. Browser automation. Drives a real Chrome instance — the fallback for scraping prop-firm dashboards that have no API.

Used in this file. `webdriver`

Example. `driver.get(url); driver.find_element(By.ID, 'login')`

Other common scenarios.

- `WebDriverWait + expected_conditions` for reliable waits
- `ChromeOptions().add_argument('--headless=new')`
- `execute_script(js, *args)` reaches into the page for things the DOM API can't express

```
from selenium.webdriver.common.by import By
```

Why imported. Browser automation. Drives a real Chrome instance — the fallback for scraping prop-firm dashboards that have no API.

Used in this file. By

Example. `driver.get(url); driver.find_element(By.ID, 'login')`

Other common scenarios.

- `WebDriverWait + expected_conditions` for reliable waits
- `ChromeOptions().add_argument('--headless=new')`
- `execute_script(js, *args)` reaches into the page for things the DOM API can't express

```
from selenium.webdriver.common.keys import Keys
```

Why imported. Browser automation. Drives a real Chrome instance — the fallback for scraping prop-firm dashboards that have no API.

Used in this file. Keys

Example. `driver.get(url); driver.find_element(By.ID, 'login')`

Other common scenarios.

- `WebDriverWait + expected_conditions` for reliable waits
- `ChromeOptions().add_argument('--headless=new')`
- `execute_script(js, *args)` reaches into the page for things the DOM API can't express

```
from selenium.webdriver.common.action_chains import ActionChains
```

Why imported. Browser automation. Drives a real Chrome instance — the fallback for scraping prop-firm dashboards that have no API.

Used in this file. ActionChains

Example. `driver.get(url); driver.find_element(By.ID, 'login')`

Other common scenarios.

- `WebDriverWait + expected_conditions` for reliable waits
- `ChromeOptions().add_argument('--headless=new')`
- `execute_script(js, *args)` reaches into the page for things the DOM API can't express

```
from selenium.webdriver.support.ui import WebDriverWait
```

Why imported. Browser automation. Drives a real Chrome instance — the fallback for scraping prop-firm dashboards that have no API.

Used in this file. WebDriverWait

Example. driver.get(url); driver.find_element(By.ID, 'login')

Other common scenarios.

- WebDriverWait + expected_conditions for reliable waits
- ChromeOptions().add_argument('--headless=new')
- execute_script(js, *args) reaches into the page for things the DOM API can't express

```
from selenium.webdriver.support import expected_conditions as EC
```

Why imported. Browser automation. Drives a real Chrome instance — the fallback for scraping prop-firm dashboards that have no API.

Used in this file. EC

Example. driver.get(url); driver.find_element(By.ID, 'login')

Other common scenarios.

- WebDriverWait + expected_conditions for reliable waits
- ChromeOptions().add_argument('--headless=new')
- execute_script(js, *args) reaches into the page for things the DOM API can't express

```
from selenium.webdriver.common.action_chains import ActionChains
```

Why imported. Browser automation. Drives a real Chrome instance — the fallback for scraping prop-firm dashboards that have no API.

Used in this file. ActionChains

Example. driver.get(url); driver.find_element(By.ID, 'login')

Other common scenarios.

- WebDriverWait + expected_conditions for reliable waits
- ChromeOptions().add_argument('--headless=new')
- execute_script(js, *args) reaches into the page for things the DOM API can't express

```
from selenium.webdriver.chrome.options import Options
```

Why imported. Browser automation. Drives a real Chrome instance — the fallback for scraping prop-firm dashboards that have no API.

Used in this file. Options

Example. driver.get(url); driver.find_element(By.ID, 'login')

Other common scenarios.

- WebDriverWait + expected_conditions for reliable waits
- ChromeOptions().add_argument('--headless=new')
- execute_script(js, *args) reaches into the page for things the DOM API can't express

```
from selenium.webdriver.chrome.service import Service
```

Why imported. Browser automation. Drives a real Chrome instance — the fallback for scraping prop-firm dashboards that have no API.

Used in this file. *No direct attribute access detected — likely imported for side effects, re-export, or string references.*

Example. `driver.get(url); driver.find_element(By.ID, 'login')`

Other common scenarios.

- `WebDriverWait + expected_conditions` for reliable waits
- `ChromeOptions().add_argument('--headless=new')`
- `execute_script(js, *args)` reaches into the page for things the DOM API can't express

```
from selenium.common.exceptions import TimeoutException
from selenium.common.exceptions import WebDriverException
from selenium.common.exceptions import NoSuchElementException
from selenium.common.exceptions import StaleElementReferenceException
```

Why imported. Browser automation. Drives a real Chrome instance — the fallback for scraping prop-firm dashboards that have no API.

Used in this file. `NoSuchElementException`, `StaleElementReferenceException`, `TimeoutException`

Example. `driver.get(url); driver.find_element(By.ID, 'login')`

Other common scenarios.

- `WebDriverWait + expected_conditions` for reliable waits
- `ChromeOptions().add_argument('--headless=new')`
- `execute_script(js, *args)` reaches into the page for things the DOM API can't express

```
from datetime import datetime
```

Why imported. Dates, times, durations. The fundamental temporal types: `date`, `time`, `datetime`, `timedelta`, `timezone`.

Used in this file. `datetime`

Example. `datetime.now() - timedelta(days=7) # one week ago`

Other common scenarios.

- `datetime.strptime(s, '%Y-%m-%d')` parses a string
- `datetime.strftime(fmt)` formats one
- `datetime.fromtimestamp(n)` converts a Unix epoch
- `datetime.utcnow()` for UTC (better: `datetime.now(timezone.utc)`)

```
from functools import wraps
```

Why imported. Higher-order helpers — caching, partial application, decorator authoring tools, reducers.

Used in this file. `wraps`

Example. `@lru_cache(maxsize=128) # memoise this pure function`

Other common scenarios.

- `partial(f, x=1)` pre-binds arguments
- `reduce(f, iterable)` folds an iterable to a single value
- `@wraps(fn)` preserves `__name__`/`__doc__` when writing decorators
- `@cached_property` memoises a property per-instance

```
from dotenv import load_dotenv
```

Why imported. `python-dotenv` — load `.env` files into `os.environ` for twelve-factor configuration in development.

Used in this file. `load_dotenv`

Example. `load_dotenv()` # call once at process start

Other common scenarios.

- `dotenv_values('.env')` returns a dict without polluting `os.environ`
- Use a real secret store in production (vault, AWS SM, etc.)

```
import requests
```

Why imported. The de-facto Python HTTP client. Synchronous; trades raw speed for an extremely friendly API.

Used in this file. `requests.Session`

Example. `requests.get(url, timeout=10).json()`

Other common scenarios.

- `Session()` reuses TCP connections across calls
- `params=`, `headers=`, `json=`, `data=` for query/body shapes
- `raise_for_status()` turns 4xx/5xx into an exception
- `stream=True` for large downloads

```
import base64
```

Why imported. `base64` — module specific to this codebase. See the chapter that covers its source for the full definition.

Used in this file. `base64.urlsafe_b64decode`

```
import json
```

Why imported. JSON encoding and decoding — the standard wire format for config files, API payloads, and inter-process messages.

Used in this file. `json.loads`

Example. `json.loads(response.text)` # parse a JSON string to dict

Other common scenarios.

- `json.dumps(obj, indent=2)` for pretty-printing

- `json.dump(obj, file)` / `json.load(file)` for file I/O
- `default=str` handles non-serialisable types like `datetime`
- `object_hook=` customises decoding (e.g., to `dataclass` instances)

Module constants

Each line below is followed by a plain-English note explaining what it does and why it is written that way.

```
DEFAULT_SYMBOL = os.getenv('TOPSTEPX_SYMBOL') or 'MNQM25'
```

Module-level constant — bound once at import time and referenced from the functions and classes below.

```
DEFAULT_TP = os.getenv('TOPSTEPX_TAKEPROFIT_TICKS')
```

Reads `TOPSTEPX_TAKEPROFIT_TICKS` from the environment. Returns `None` if unset.

```
DEFAULT_SL = os.getenv('TOPSTEPX_STOPLOSS_TICKS')
```

Reads `TOPSTEPX_STOPLOSS_TICKS` from the environment. Returns `None` if unset.

Classes

```
class TopStepXAccount
```

TopStepX trading automation class with robust Selenium-based web automation. Handles login, symbol selection, order placement, and account statistics. Ensures single Chrome instance per account to prevent resource conflicts. Follows MFFU blueprint pattern for consistency.

```
_chrome_instances = {}
```

Code (copy-paste safe)

```
class TopStepXAccount:
    """
    TopStepX trading automation class with robust Selenium-based web automation.
    Handles login, symbol selection, order placement, and account statistics.
    Ensures single Chrome instance per account to prevent resource conflicts.
    Follows MFFU blueprint pattern for consistency.
    """

    # Class-level registry to track Chrome instances per account
    _chrome_instances = {}

    def __init__(self, username=None, password=None, pair_id=None):
        self.username = username or ""
        self.password = password or ""
        self.pair_id = pair_id or "default"
        self.logged_in = False
        self.driver = None
        self.first_trade_attempted = False
        self._first_stats_fetch = True
        self._placing_order = False # Flag to block stats fetching during order placement
        self._login_timestamp = None # Track when we logged in for debugging
        self.base_url = "https://www.topstepx.com"
        self.login_url = "https://www.topstepx.com/login"
        self.user_api_url = "https://userapi.topstepx.com"
        self.chart_api_url = "https://chartapi.topstepx.com"
```



```

self._api_token = None
self._api_session = None
self._api_user_id = None
self._api_account_id = None
self._api_token_expiry = 0
self._delay_snapshots_enabled = True # Enable automatic snapshot capture on delays
self.lock = threading.RLock() # Thread safety for Selenium operations

# Initialize logger - MFFU blueprint standard
self.logger = logging.getLogger(f"TopStepX_{self.username}_{self.pair_id}")
self.logger.info(f"[INIT] TopStepXAccount initializing for user={self.username}, pair_id={self.pair_id}")

# Create unique instance key using both username and pair_id
instance_key = f"{self.username}_{self.pair_id}"
self.logger.info(f"[INIT] Instance key: {instance_key}")

# Check if Chrome instance already exists for this specific account+pair combination
if instance_key in self._chrome_instances:
    self.logger.info(
        f"[INIT] Reusing existing Chrome instance for TopStepX account: {self.username} (Pair: {self.pair_id})"
    )
    existing_driver = self._chrome_instances[instance_key]
    # Test if existing driver is still alive
    try:
        existing_driver.current_url
        self.driver = existing_driver
        self.logger.info(f"[INIT] Successfully connected to existing Chrome instance")
        return
    except Exception as e:
        self.logger.info(f"[INIT] Existing Chrome instance is dead ({e}), creating new one")
        # Remove dead instance from registry
        del self._chrome_instances[instance_key]

# Initialize driver with error handling
self.logger.info(f"[INIT] No existing Chrome instance found - creating new WebDriver")
try:
    self.logger.info(f"[INIT] Calling _initialize_driver()...")
    self.driver = self._initialize_driver()
    self.logger.info(f"[INIT] WebDriver created successfully: {self.driver}")
    # Register the Chrome instance for this specific account+pair combination
    self._chrome_instances[instance_key] = self.driver
    self.logger.info(
        f"[INIT] Chrome instance registered for TopStepX: {self.username} (Pair: {self.pair_id})"
    )
except Exception as e:
    self.logger.error(f"[INIT ERROR] Failed to initialize WebDriver: {str(e)}")
    import traceback
    self.logger.error(f"[INIT ERROR] Full traceback:\n{traceback.format_exc()}")
    raise Exception(
        f"Failed to initialize WebDriver: {str(e)}. This may indicate VPS Chrome setup issues."
    )

def _get_chromedriver_path(self):
    """Get the path to ChromeDriver executable with auto-compatibility check"""
    # SELENIUM 4.15+ AUTO-MANAGEMENT: Let Selenium Manager handle ChromeDriver automatically
    # This ensures ChromeDriver always matches the installed Chrome version
    logging.info(f"[AUTO] Using Selenium Manager for automatic ChromeDriver version matching")
    return None # Return None to let Selenium Manager handle it

def _ensure_chrome_compatibility(self):
    """Ensure Chrome-ChromeDriver version compatibility"""
    try:
        # Import the compatibility manager

```

```

sys.path.insert(0, os.path.dirname(os.path.dirname(__file__)))
from chrome_auto_compatibility import ChromeVersionManager # type: ignore

manager = ChromeVersionManager()
success = manager.ensure_compatibility()

if success:
    logging.info("[CHECK] Chrome-ChromeDriver compatibility verified")
else:
    logging.warning("[WARNING] Chrome-ChromeDriver compatibility could not be ensured")

except Exception as e:
    logging.warning(f"Chrome compatibility check failed: {e}")
    # Continue anyway - don't break existing functionality

def _initialize_driver(self):
    """Initialize Chrome WebDriver with crash-resistant options"""
    self.logger.info("[DRIVER] Starting Chrome WebDriver initialization...")
    chrome_options = Options()

    # Create persistent user data directory per account+pair to ensure separate Chrome instances
    # Use username+pair_id hash to create unique directory per account+pair combination
    unique_id = f"{self.username}_{self.pair_id}"
    unique_hash = hashlib.md5(unique_id.encode()).hexdigest()[:8]
    user_data_dir = os.path.join(tempfile.gettempdir(), f"chrome_topstepx_{unique_hash}")
    self.logger.info(f"[DRIVER] User data dir: {user_data_dir}")
    chrome_options.add_argument(f"--user-data-dir={user_data_dir}")

    # Use unique debugging port per account+pair
    port_base = 9322 # Different from Tradovate (9222) to avoid conflicts
    port_offset = int(unique_hash[:4], 16) % 100 # 0-99 offset based on unique_id
    debug_port = port_base + port_offset
    self.logger.info(f"[DRIVER] Remote debugging port: {debug_port}")
    chrome_options.add_argument(f"--remote-debugging-port={debug_port}")

    # Position windows differently for each pair to avoid overlapping
    pair_num = 0
    try:
        # Handle various pair_id formats: "pair_0", "topstepx_pair_0", "default", etc.
        if "pair_" in self.pair_id:
            # Extract number from formats like "pair_0" or "topstepx_pair_0"
            pair_part = self.pair_id.split("pair_-1"] # Get the part after last "pair_"
            pair_num = int(pair_part) if pair_part.isdigit() else 0
        elif self.pair_id.isdigit():
            pair_num = int(self.pair_id)
    except (ValueError, IndexError):
        pair_num = 0 # Fallback to 0 if parsing fails

    window_x = 150 + (pair_num * 50) # Offset from Tradovate windows
    window_y = 100 + (pair_num * 50)
    self.logger.info(f"[DRIVER] Window position: {window_x}, {window_y}")
    chrome_options.add_argument(f"--window-position={window_x},{window_y}")

    # SPEED OPTIMIZATION: Essential options only for fastest startup
    self.logger.info("[DRIVER] Adding Chrome options...")
    chrome_options.add_argument("--no-sandbox")
    chrome_options.add_argument("--disable-dev-shm-usage")
    chrome_options.add_argument("--disable-gpu")
    chrome_options.add_argument("--disable-extensions")
    chrome_options.add_argument("--no-first-run")

```

```

chrome_options.add_argument("--no-default-browser-check")
chrome_options.add_argument("--disable-features=TranslateUI")

# SPEED OPTIMIZATION: Smaller window for faster rendering
chrome_options.add_argument("--window-size=1200,800") # Reduced from 1400,900

# Anti-detection settings (keep minimal)
chrome_options.add_experimental_option("excludeSwitches", ["enable-automation", "enable-logging"])
chrome_options.add_experimental_option('useAutomationExtension', False)
chrome_options.add_argument("--disable-blink-features=AutomationControlled")

# Minimal preferences to prevent crashes
prefs = {
    "profile.default_content_setting_values": {
        "popups": 2, # Block popups only
        "notifications": 2, # Block notifications only
    }
}
chrome_options.add_experimental_option("prefs", prefs)

try:
    # Get ChromeDriver path (returns None for auto-management in Selenium 4.15+)
    chromedriver_path = self._get_chromedriver_path()

    # Create the WebDriver instance - let Selenium Manager handle ChromeDriver automatically
    self.logger.info("[CHROME] Initializing Chrome browser...")
    self.logger.info("[CHROME] Using Selenium Manager for automatic ChromeDriver version matching")
    self.logger.info(
        "[CHROME] Selenium Manager will download ChromeDriver if needed (happens once per Chrome"
    )
    self.logger.info("[CHROME] If already cached, Chrome will launch immediately...")

    import time
    start_time = time.time()
    driver = webdriver.Chrome(options=chrome_options)
    elapsed = time.time() - start_time

    if elapsed > 5:
        self.logger.info(
            f"[CHROME] ✓ Chrome launched in {elapsed:.1f}s (ChromeDriver was downloaded/updated)"
        )
    else:
        self.logger.info(f"[CHROME] ✓ Chrome launched in {elapsed:.1f}s (using cached ChromeDriver)")

    self.logger.info("[DRIVER] Chrome instance created, setting timeouts...")
    # SPEED OPTIMIZATION: Aggressive timeouts for faster performance
    driver.set_page_load_timeout(15) # Reduced from 30
    driver.implicitly_wait(2) # Reduced from 3

    # Remove automation detection
    self.logger.info("[DRIVER] Executing anti-detection script...")
    driver.execute_script("Object.defineProperty(navigator, 'webdriver', {get: () => undefined})")

    self.logger.info(
        f"[DRIVER] Chrome WebDriver initialized successfully for TopStepX {self.username}"
    )
    return driver

except Exception as e:
    self.logger.error(f"[DRIVER ERROR] Failed to initialize Chrome WebDriver: {e}")
    import traceback
    self.logger.error(f"[DRIVER ERROR] Full traceback:\n{traceback.format_exc()}")
    raise Exception(f"ChromeDriver initialization failed: {e}")

```

```

def login(self, max_retries=3):
    """
    Login to TopStepX platform
    """
    # Acquire lock to prevent stats fetching during login
    with self.lock:
        if not self.username or not self.password:
            self.logger.error("Username and password are required for TopStepX login")
            return False

        for attempt in range(max_retries):
            try:
                self.logger.info(f"TopStepX login attempt {attempt + 1}/{max_retries}")

                if not self.driver:
                    if not self.initialize_driver():
                        raise Exception("Failed to initialize WebDriver")

                # Navigate to login page
                self.logger.info(f"Navigating to TopStepX login: {self.login_url}")
                self.driver.get(self.login_url)
                time.sleep(3)

                # Log current page info
                self.logger.info(f"Current URL: {self.driver.current_url}")
                self.logger.info(f"Page title: {self.driver.title}")

                # Wait for login form
                wait = WebDriverWait(self.driver, 15)

                # Find and fill username field (TopStepX uses 'userName' not 'email')
                username_field = wait.until(
                    EC.presence_of_element_located((By.NAME, "userName")))
                )
                # Highlight existing value and type over it (no deletion needed)
                username_field.send_keys(Keys.CONTROL + "a") # Select all existing text
                time.sleep(0.2) # Allow selection to complete
                username_field.send_keys(self.username) # Type over selected text
                self.logger.info(f"Username field filled: {self.username}")

                # Find and fill password field
                password_field = self.driver.find_element(By.NAME, "password")
                # Highlight existing value and type over it (no deletion needed)
                password_field.send_keys(Keys.CONTROL + "a") # Select all existing text
                time.sleep(0.2) # Allow selection to complete
                password_field.send_keys(self.password) # Type over selected text
                self.logger.info("Password field filled")

                # Click login button (Sign In button)
                login_button = self.driver.find_element(By.XPATH, "//button[@type='submit']")
                self.logger.info("Clicking Sign In button")
                login_button.click()

                # Wait for page to load after login
                time.sleep(3)

                # Wait for either success or error indicators
                success_indicators = [
                    "dashboard",
                    "trading",

```

```

        "account",
        "logout" # logout button indicates successful login
    ]

    # Check if login was successful
    max_wait_time = 15
    start_time = time.time()

    while time.time() - start_time < max_wait_time:
        current_url = self.driver.current_url.lower()
        page_source = self.driver.page_source.lower()

        # Check for success indicators
        if any(indicator in current_url for indicator in success_indicators):
            self.logged_in = True
            self._login_timestamp = time.time() # Track login time
            self.logger.info(f"TopStepX login successful - redirected to: {current_url}")
            self._first_stats_fetch = True
            self._extract_api_token() # Extract JWT for REST API calls
            return True

        # Check if we're no longer on the login page (indicates success)
        if "/login" not in current_url and "topstepx.com" in current_url:
            self.logged_in = True
            self._login_timestamp = time.time() # Track login time
            self.logger.info(f"TopStepX login successful - left login page: {current_url}")
            self._first_stats_fetch = True
            self._extract_api_token() # Extract JWT for REST API calls
            return True

        # Check for error messages or still on login page
        if "sign in" in page_source and "/login" in current_url:
            # Still on login page, check for errors
            error_elements = self.driver.find_elements(By.XPATH,
                "//*[contains(@class, 'error') or contains(@class, 'alert') or contains(t

            if error_elements:
                error_text = error_elements[0].text
                self.logger.error(f"TopStepX login failed: {error_text}")
                break

        time.sleep(1)

    # If we get here, login likely failed
    self.logger.error("TopStepX login failed: Timeout or unknown error")

except TimeoutException:
    self.logger.error(f"TopStepX login attempt {attempt + 1} timed out")
except Exception as e:
    self.logger.error(f"TopStepX login attempt {attempt + 1} failed: {e}")

if attempt < max_retries - 1:
    time.sleep(5) # Wait before retry

return False

def is_connected(self):
    """Check if we're connected to TopStepX"""
    try:
        if not self.driver or not self.logged_in:
            return False

```



```

except Exception as e:
    self.logger.error(f"[API] Failed to extract token: {e}")
    return False

def _refresh_api_token(self):
    """Re-extract token from browser if expired or missing"""
    if self._api_token and time.time() < self._api_token_expiry - 60:
        return True # Token still valid
    self.logger.info("[API] Token expired or missing, re-extracting from browser")
    return self._extract_api_token()

def _api_get(self, path, base_url=None):
    """Make authenticated GET request to TopStepX API. Returns parsed JSON or None."""
    if not self._api_session:
        if not self._extract_api_token():
            return None
    self._refresh_api_token()
    url = f"{base_url or self.user_api_url}{path}"
    try:
        resp = self._api_session.get(url, timeout=10)
        if resp.status_code == 200:
            ct = resp.headers.get("content-type", "")
            if "json" in ct or resp.text.strip().startswith(("{", "[")):
                return resp.json()
            return resp.text
        elif resp.status_code == 204:
            return [] # No content
        else:
            self.logger.warning(f"[API] GET {path} returned {resp.status_code}")
            return None
    except Exception as e:
        self.logger.error(f"[API] GET {path} failed: {e}")
        return None

def _api_post(self, path, data=None, base_url=None):
    """Make authenticated POST request to TopStepX API. Returns parsed JSON or None."""
    if not self._api_session:
        if not self._extract_api_token():
            return None
    self._refresh_api_token()
    url = f"{base_url or self.user_api_url}{path}"
    try:
        resp = self._api_session.post(url, json=data, timeout=10)
        if resp.status_code in (200, 201):
            ct = resp.headers.get("content-type", "")
            if "json" in ct or resp.text.strip().startswith(("{", "[")):
                return resp.json()
            return resp.text
        elif resp.status_code == 204:
            return []
        else:
            self.logger.warning(f"[API] POST {path} returned {resp.status_code}")
            return None
    except Exception as e:
        self.logger.error(f"[API] POST {path} failed: {e}")
        return None

@property
def api_available(self):
    """Check if REST API is available (token extracted and session ready)"""

```

```

return self._api_session is not None and self._api_token is not None

# =====
# REST API DATA METHODS
# =====

def api_get_user(self):
    """Get user profile via REST API.
    Returns dict with: userName, email, userId, firstName, lastName, has2FA, etc."""
    return self._api_get("/User")

def api_get_trading_accounts(self):
    """Get all trading accounts via REST API.
    Returns list of account dicts with: accountId, accountName, balance,
    startingBalance, profitAndLoss, winRate, totalTrades, status,
    realizedDayPnl, openPnl, highestBalance, maximumLoss, etc."""
    return self._api_get("/TradingAccount")

def api_get_positions(self, user_id=None):
    """Get all open positions via REST API.
    Returns list of position dicts."""
    uid = user_id or self._api_user_id
    if not uid:
        self.logger.warning("[API] No user_id available for positions query")
        return []
    return self._api_get(f"/Position/all/user/{uid}") or []

def api_get_orders(self, account_id=None):
    """Get pending orders via REST API.
    Returns list of order dicts."""
    acct_id = account_id or self._api_account_id
    if not acct_id:
        # Try to get from trading accounts
        accounts = self.api_get_trading_accounts()
        if accounts and isinstance(accounts, list) and len(accounts) > 0:
            acct_id = accounts[0].get("accountId")
            self._api_account_id = acct_id
    if not acct_id:
        return []
    return self._api_get(f"/Order?accountId={acct_id}") or []

def api_get_trades(self, account_id=None):
    """Get trade history via REST API.
    Returns list of trade dicts with: id, symbolId, contractId, accountId,
    entryPrice, exitPrice, fees, pnl, positionSize, tradeDuration, etc."""
    acct_id = account_id or self._api_account_id
    if not acct_id:
        accounts = self.api_get_trading_accounts()
        if accounts and isinstance(accounts, list) and len(accounts) > 0:
            acct_id = accounts[0].get("accountId")
            self._api_account_id = acct_id
    if not acct_id:
        return []
    return self._api_get(f"/Trade/id/{acct_id}") or []

def api_get_contracts(self):
    """Get all active tradeable contracts via REST API.
    Returns list with: productId, productName, contractId, contractName,
    tickValue, tickSize, pointValue, fees, exchange, expiration, etc."""
    return self._api_get("/UserContract/active/nonprofesional") or []

```



```

def api_get_metadata(self):
    """Get symbol metadata via REST API.
    Returns list with: symbol, friendlyName, fullName, tickSize, tickValue,
    contractCost, fees, decimalPlaces, exchange, etc."""
    return self._api_get("/Metadata") or []

def api_get_market_status(self):
    """Get market open/close status for all symbols via REST API.
    Returns list with: symbolId, contractId, status, isOpen, openedAt, etc."""
    return self._api_get("/MarketStatus/markets") or []

def api_get_trading_rules(self):
    """Get trading rules (DAILY_LOSS, MAXIMUM_LOSS, MARGIN_SCALE) via REST API."""
    return self._api_get("/TradingRule") or []

def api_get_template_rules(self, template_id=1):
    """Get account template rules (loss limits, etc.) via REST API."""
    return self._api_get(f"/AccountTemplateRule/rules/{template_id}/all") or []

def api_get_violations(self, account_id=None):
    """Get active rule violations via REST API."""
    acct_id = account_id or self._api_account_id
    if not acct_id:
        accounts = self.api_get_trading_accounts()
        if accounts and isinstance(accounts, list) and len(accounts) > 0:
            acct_id = accounts[0].get("accountId")
            self._api_account_id = acct_id
    if not acct_id:
        return []
    return self._api_get(f"/Violations/active/{acct_id}") or []

def api_get_linked_orders(self, account_id=None):
    """Get linked/bracket orders via REST API."""
    acct_id = account_id or self._api_account_id
    if not acct_id:
        accounts = self.api_get_trading_accounts()
        if accounts and isinstance(accounts, list) and len(accounts) > 0:
            acct_id = accounts[0].get("accountId")
            self._api_account_id = acct_id
    if not acct_id:
        return {"linkedOrders": []}
    return self._api_get(f"/LinkedOrder?tradingAccountId={acct_id}")

def api_get_user_settings(self):
    """Get user settings (risk, toMake, autoCenter, etc.) via REST API."""
    return self._api_get("/UserSettings")

def api_get_notifications(self):
    """Get user notifications via REST API."""
    return self._api_get("/UserNotification") or []

def api_get_chart_config(self):
    """Get TradingView chart config (supported resolutions, etc.) via REST API."""
    return self._api_get("/Config", base_url=self.chart_api_url)

def get_blueprint_config(self, phase_key="challenge_trade1", size_key="50k"):
    """
    Get TopStepX blueprint configuration from PropFirmManager
    Returns config for the specified phase and account size
    """

```

```
if not PROP_FIRM_MANAGER_AVAILABLE:
    logging.warning("PropFirmManager not available - using defaults")
    return {
        'topstepx_symbol': DEFAULT_SYMBOL,
        'topstepx_qty': 1,
        'topstepx_tp_ticks': DEFAULT_TP,
        'topstepx_sl_ticks': DEFAULT_SL
    }

try:
    manager = PropFirmManager()

    # Force TopStep firm code since this is TopStepX integration
    # We don't need to detect from username because we are in the TopStepX module
    firm_code = "TopStep"

    # Get strategy config for TopStepX
    manager.set_prop_firm(firm_code)
    config = manager.get_prop_firm_strategy_config(phase_key, size_key)

    if config and 'topstepx_symbol' in config:
        logging.info(f"TopStepX blueprint loaded: {phase_key}/{size_key} -> {config}")
        return config
    else:
        logging.warning("No TopStepX-specific config found, using defaults")
        return {
            'topstepx_symbol': DEFAULT_SYMBOL,
            'topstepx_qty': 1,
            'topstepx_tp_ticks': DEFAULT_TP,
            'topstepx_sl_ticks': DEFAULT_SL
        }

except Exception as e:
    logging.error(f"Failed to get TopStepX blueprint config: {e}")
    return {
        'topstepx_symbol': DEFAULT_SYMBOL,
        'topstepx_qty': 1,
        'topstepx_tp_ticks': DEFAULT_TP,
        'topstepx_sl_ticks': DEFAULT_SL
    }

# ████████████████████████████████████████████████████████████████████████████████
# ACCOUNT SWITCHING
# ████████████████████████████████████████████████████████████████████████████████

def list_accounts(self):
    """List all trading accounts available for this user.
    Returns list of dicts with: accountId, accountName, balance, status, ineligible, etc.
    Uses REST API if available, otherwise falls back to reading the UI dropdown."""
    if self.api_available:
        accounts = self.api_get_trading_accounts()
        if accounts:
            return accounts

    # Fallback: read the MuiSelect dropdown options from UI
    try:
        items = self.driver.execute_script("""
            const select = document.querySelector('[role="combobox"]');
            if (!select) return null;
            select.click(); // open dropdown
```

```

        await new Promise(r => setTimeout(r, 500));
        const options = document.querySelectorAll('[role="option"], .MuiMenuItem-root');
        const result = Array.from(options).map(o => ({text: o.textContent.trim()}));
        // close by pressing Escape
        document.dispatchEvent(new KeyboardEvent('keydown', {key: 'Escape'}));
        return result;
    """)
    return items or []
except Exception as e:
    self.logger.warning(f"[SWITCH] Could not list accounts from UI: {e}")
    return []

def switch_account(self, account_id=None, account_name_contains=None):
    """Switch to a sub-account via the MuiSelect dropdown.

    Minimal/fast path:
    1. Per-session cache hit → instant return.
    2. Read selector text → if already shows target, cache + return.
    3. Click selector → click matching <li> → done.

    No API calls, no _ensure_on_trading_page, no extra DOM scans.
    """
    with self.lock:
        try:
            search = (account_name_contains or "").strip()
            cache_key = str(account_id) if account_id else search.lower()
            if not search and not account_id:
                return False

            # 1) Cache hit
            if cache_key and getattr(self, "_last_switched_key", None) == cache_key:
                return True

            # 2) Locate the selector and check if already active
            select_el = self.driver.find_element(By.XPATH,
                "//div[contains(@class, 'MuiSelect-select') and contains(@class, 'MuiInputBase-input')]")
            try:
                current = (select_el.text or "").strip()
            except Exception:
                current = ""
            if search and current and search.lower() in current.lower():
                self._last_switched_key = cache_key
                return True

            # 3) Open dropdown and click the matching option
            # Dismiss any backdrop first (a leftover from a previous interaction
            # can intercept the click and silently swallow it).
            try:
                self.driver.execute_script(
                    "document.querySelectorAll('.MuiBackdrop-root').forEach(el => el.click());"
                )
            except Exception:
                pass

            # MUI Select needs a real pointer event to open. JS-click sometimes works,
            # but a native Selenium .click() is much more reliable. Try native first,
            # fall back to JS if intercepted.
            opened = False
            try:

```

```

        select_el.click()
        opened = True
    except Exception:
        try:
            self.driver.execute_script("arguments[0].click()", select_el)
            opened = True
        except Exception as click_e:
            self.logger.error(f"[SWITCH] Could not click selector: {click_e}")
            return False

# Poll for menu items (up to 3s – first dropdown open after page load can be slow)
items = []
deadline = time.time() + 3.0
option_xpath = (
    "//li[contains(@class, 'MuiMenuItem-root')] | "
    "//*[ @role='option' ] | "
    "//div[contains(@class, 'MuiPopover-root')]/li"
)
while time.time() < deadline:
    items = self.driver.find_elements(By.XPATH, option_xpath)
    if items:
        break
    time.sleep(0.05)

if not items:
    # Diagnostic dump so we can see what actually rendered
    try:
        popover_html = self.driver.execute_script("""
            const pop = document.querySelector(
                '.MuiPopover-root, .MuiMenu-root, [role="presentation"]');
            return pop ? pop.outerHTML.substring(0, 2000) : '(no popover element found)';
        """)
        self.logger.error(
            f"[SWITCH] Dropdown opened but no options found. HTML: {popover_html}")
    except Exception:
        pass

clicked = False
needle = search.lower() if search else None
for item in items:
    try:
        text = (item.text or "").strip()
    except Exception:
        continue
    if not text:
        continue
    if needle and needle in text.lower():
        # Native click first, then JS fallback
        try:
            item.click()
        except Exception:
            try:
                self.driver.execute_script("arguments[0].click()", item)
            except Exception:
                continue
        clicked = True
        self.logger.info(f"[SWITCH] Clicked: {text[:80]}")
        break

if not clicked:

```

```

        self.logger.error(f"[SWITCH] '{search}' not found among {len(items)} dropdown options")
    try:
        ActionChains(self.driver).send_keys(Keys.ESCAPE).perform()
    except Exception:
        pass
    return False

    # Invalidate cached stats so next poll fetches fresh data
    self._cached_stats = None
    self._stats_last_fetch_time = 0
    self._first_stats_fetch = True
    self._last_switched_key = cache_key

    # Brief settle so the selector text + downstream UI catch up
    time.sleep(0.4)
    return True

except Exception as e:
    self.logger.error(f"[SWITCH] Account switch failed: {e}")
    return False

def cleanup_chrome_instance(self):
    """
    Clean up Chrome instance for this specific account+pair combination
    Part of MFFU blueprint pattern for proper resource management
    """
    instance_key = f"{self.username}_{self.pair_id}"

    try:
        if instance_key in self._chrome_instances:
            driver = self._chrome_instances[instance_key]
            try:
                driver.quit()
                logging.info(
                    f"Chrome instance cleaned up for TopStepX: {self.username} (Pair: {self.pair_id})"
                )
            except Exception as e:
                logging.warning(f"Error closing Chrome instance for TopStepX {self.username}: {e}")

            # Remove from registry
            del self._chrome_instances[instance_key]

    except Exception as e:
        logging.error(f"Error during TopStepX Chrome cleanup: {e}")

def disconnect(self):
    """
    Disconnect from TopStepX and clean up resources
    MFFU blueprint standard method
    """
    try:
        self.logged_in = False

        if self.driver:
            try:
                # Try to logout gracefully first
                if "topstepx.com" in self.driver.current_url:
                    logout_selectors = [
                        "//button[contains(text(), 'Logout')]",
                        "//a[contains(text(), 'Logout')]",
                        "//button[contains(@class, 'logout')]",
                    ]

```

```

        "//a[contains(@class, 'logout')]"
    ]

    for selector in logout_selectors:
        try:
            logout_btn = self.driver.find_element(By.XPATH, selector)
            logout_btn.click()
            time.sleep(1)
            logging.info("TopStepX logout successful")
            break
        except:
            continue

    except Exception as e:
        logging.warning(f"Graceful TopStepX logout failed: {e}")

    # Clean up Chrome instance
    self.cleanup_chrome_instance()
    self.driver = None

    logging.info(f"TopStepX disconnection completed for {self.username}")

except Exception as e:
    logging.error(f"Error during TopStepX disconnection: {e}")

def __del__(self):
    """
    Destructor to ensure cleanup on object deletion
    MFFU blueprint standard pattern
    """
    try:
        if hasattr(self, 'driver') and self.driver:
            self.disconnect()
    except:
        pass # Ignore errors during cleanup

def _capture_delay_snapshot(self, context_name, delay_ms, threshold_ms=200):
    """
    Capture DOM snapshot when a delay exceeds threshold.

    Args:
        context_name: Description of what operation was delayed (e.g., "contract_to_quantity")
        delay_ms: Actual delay in milliseconds
        threshold_ms: Threshold in milliseconds (default 200ms)
    """
    if not self._delay_snapshots_enabled or delay_ms < threshold_ms:
        return

    try:
        from datetime import datetime
        timestamp = datetime.now().strftime("%Y%m%d_%H%M%S_%f")[:-3] # Include milliseconds

        # Create filename with delay info
        filename = f"delay_snapshot_{context_name}_{int(delay_ms)}ms_{timestamp}.html"

        # Capture full page source
        page_source = self.driver.page_source

        # Capture focused element info
        focused_element_info = "Unknown"

```

```

try:
    focused = self.driver.execute_script("return document.activeElement.outerHTML;")
    focused_element_info = focused[:500] if focused else "None"
except:
    pass

# Capture contract field state
contract_field_info = "Unknown"
try:
    contract_field = self.driver.find_element(By.XPATH,
        "//input[contains(@class, 'MuiAutocomplete-input')]")
    contract_field_info = f"Value: '{contract_field.get_attribute('value')}', Enabled: {contract_field.is_enabled()}"
except:
    pass

# Capture quantity field state
quantity_field_info = "Unknown"
try:
    quantity_field = self.driver.find_element(By.XPATH, "//input[@type='number'][@min='1']")
    quantity_field_info = f"Value: '{quantity_field.get_attribute('value')}', Enabled: {quantity_field.is_enabled()}"
except:
    pass

# Build comprehensive HTML with metadata
html_content = f"""<!-- DELAY SNAPSHOT -->
<!-- Context: {context_name} -->
<!-- Delay: {delay_ms}ms (threshold: {threshold_ms}ms) -->
<!-- Timestamp: {timestamp} -->
<!-- Current URL: {self.driver.current_url} -->
<!-- Focused Element: {focused_element_info} -->
<!-- Contract Field: {contract_field_info} -->
<!-- Quantity Field: {quantity_field_info} -->

{page_source}
"""

# Write to file
with open(filename, 'w', encoding='utf-8') as f:
    f.write(html_content)

self.logger.warning(
    f"■■■ DELAY DETECTED: {context_name} took {delay_ms}ms (>{threshold_ms}ms threshold)")
self.logger.warning(f"■ Snapshot saved: {filename}")

except Exception as e:
    self.logger.debug(f"Could not capture delay snapshot: {e}")

def get_account_info(self):
    """Get TopStepX account information"""
    try:
        if not self.is_connected():
            raise Exception("Not connected to TopStepX")

        # Only navigate to trading page if we're clearly not on a trading-related page
        current_url = self.driver.current_url
        if "/trade" not in current_url and "/dashboard" not in current_url and "/account" not in current_url:
            self.logger.info("Not on trading/account page, navigating to get account info")
            self._ensure_on_trading_page()
        else:
            self.logger.debug("Already on trading/account page, skipping navigation for account info")

```

```

account_info = {
    "platform": "TopStepX",
    "status": "Connected",
    "balance": "N/A",
    "equity": "N/A",
    "margin": "N/A",
    "account_type": "N/A"
}

# Extract account balance information from the interface
try:
    # Look for balance displays - based on HTML analysis
    balance_selectors = [
        "//span[contains(text(), 'BAL:')]//following-sibling::span",
        "//div[contains(@aria-label, 'Current Account Balance')]//span[contains(text(), '$')]"
        "//span[contains(text(), '$') and contains(@class, 'balance')]"
    ]

    for selector in balance_selectors:
        try:
            balance_element = self.driver.find_element(By.XPATH, selector)
            balance_text = balance_element.text.strip()
            if '$' in balance_text:
                account_info["balance"] = balance_text
                account_info["equity"] = balance_text # Often the same in trading accounts
                break
        except:
            continue

    # Look for account type information
    account_type_selectors = [
        "//span[contains(text(), 'Trading Combine') or contains(text(), 'Funded') or contains"
        "//div[contains(@class, 'account')]//span[contains(text(), 'K')]"
    ]

    for selector in account_type_selectors:
        try:
            account_element = self.driver.find_element(By.XPATH, selector)
            account_info["account_type"] = account_element.text.strip()
            break
        except:
            continue

    # Look for P&L information
    pnl_selectors = [
        "//span[contains(text(), 'RP&L:')]//following-sibling::span",
        "//span[contains(text(), 'UP&L:')]//following-sibling::span"
    ]

    realized_pnl = unrealized_pnl = "N/A"
    try:
        realized_element = self.driver.find_element(By.XPATH, pnl_selectors[0])
        realized_pnl = realized_element.text.strip()
    except:
        pass

    try:
        unrealized_element = self.driver.find_element(By.XPATH, pnl_selectors[1])
        unrealized_pnl = unrealized_element.text.strip()
    except:

```



```

        pass

        account_info["realized_pnl"] = realized_pnl
        account_info["unrealized_pnl"] = unrealized_pnl

    except Exception as e:
        self.logger.warning(f"Could not extract all account info: {e}")

    self.logger.info(f"TopStepX account info retrieved: {account_info}")
    return account_info

except Exception as e:
    self.logger.error(f"Failed to get TopStepX account info: {e}")
    return None

def get_account_stats(self):
    """Get TopStepX account statistics from positions tab - required by GUI"""
    # Use lock to prevent conflict with order placement
    if not self.lock.acquire(blocking=False):
        # If locked (e.g. placing order), return cached stats immediately
        return getattr(self, '_cached_stats', {
            "Account Number": "Trading...",
            "Balance": "N/A",
            "Profit/Loss": "N/A",
            "Open Trades": "N/A",
            "Symbol": "",
            "Direction": ""
        })
    try:
        if getattr(self, '_placing_order', False):
            return getattr(self, '_cached_stats', {
                "Account Number": "Trading...",
                "Balance": "N/A",
                "Profit/Loss": "N/A",
                "Open Trades": "N/A",
                "Symbol": "",
                "Direction": ""
            })

        # PERFORMANCE OPTIMIZATION: Return cached stats if fetched within the last 2 seconds
        # This prevents excessive DOM queries when GUI polls every 1 second
        cache_ttl = 2.0 # Cache time-to-live in seconds
        current_time = time.time()
        last_fetch_time = getattr(self, '_stats_last_fetch_time', 0)
        time_since_fetch = current_time - last_fetch_time

        if time_since_fetch < cache_ttl and hasattr(self, '_cached_stats'):
            # Return cached stats (still fresh)
            return self._cached_stats

    try:
        if not self.is_connected():
            return {
                "Account Number": "Not Connected",
                "Balance": "N/A",
                "Profit/Loss": "N/A",
                "Open Trades": "N/A",
                "Symbol": "",
                "Direction": ""
            }

```

```

    }

# Initialize default stats
stats = {
    "Account Number": "Unknown",
    "Balance": "N/A",
    "Profit/Loss": "N/A",
    "Open Trades": "0",
    "Symbol": "",
    "Direction": ""
}

# FAST PATH: Use REST API if available (much faster than Selenium DOM scraping)
if self.api_available:
    try:
        api_stats = self._get_stats_via_api()
        if api_stats:
            self._cached_stats = api_stats
            self._stats_last_fetch_time = time.time()
            return api_stats
    except Exception as e:
        self.logger.warning(f"[API] Stats via API failed, falling back to Selenium: {e}")

    try:
        # Check if positions data is already visible before clicking tabs
        positions_visible = False
        try:
            # Look for positions grid or position data
            self.driver.find_element(By.XPATH,
                "//*[div[@role='grid']] | //div[contains(@class, 'MuiDataGrid')] | //*[contains(
            positions_visible = True
            self.logger.debug("Positions data already visible, skipping tab navigation")
        except:
            pass

        # Navigate to positions tab to extract stats
        if not positions_visible:
            if not self.switch_to_positions_tab():
                self.logger.warning("Could not switch to Positions tab")

        # Extract stats from positions DataGrid
        self._extract_positions_stats(stats)

        # Try to get account balance from top bar or other locations
        self._extract_account_balance(stats)

    except Exception as e:
        self.logger.warning(f"Could not extract all stats from positions: {e}")

# Cache the stats WITH TIMESTAMP for performance optimization
self._cached_stats = stats
self._stats_last_fetch_time = time.time()
return stats

except Exception as e:
    self.logger.error(f"Failed to get TopStepX account stats: {e}")
    return {
        "Account Number": "Error",
        "Balance": "Error",
        "Profit/Loss": "Error",

```

```

        "Open Trades": "Error",
        "Symbol": "",
        "Direction": ""
    }
    finally:
        self.lock.release()

def _extract_positions_stats(self, stats):
    """Extract statistics from the positions DataGrid"""
    try:
        # Count open positions from the DataGrid rows
        position_rows = self.driver.find_elements(By.XPATH, "//*[div[@role='row']][@data-id]")
        open_trades_count = len(position_rows)
        stats["Open Trades"] = str(open_trades_count)

        if open_trades_count > 0:
            # Get symbol and position info from first position
            try:
                # Extract symbol from first position row
                symbol_cell = self.driver.find_element(By.XPATH,
                    "//*[div[@role='row']][@data-id][1]//div[@data-field='symbolName']")
                symbol_text = symbol_cell.text.strip()
                if symbol_text:
                    stats["Symbol"] = symbol_text

                # Extract position size to determine direction
                position_cell = self.driver.find_element(By.XPATH,
                    "//*[div[@role='row']][@data-id][1]//div[@data-field='positionSize']")
                position_size = position_cell.text.strip()
                if position_size:
                    try:
                        size_num = int(position_size)
                        stats["Direction"] = "Long" if size_num > 0 else "Short" if size_num < 0 else "Flat"
                    except:
                        stats["Direction"] = "Long" # Default assumption

                # Extract P&L from positions
                pnl_cells = self.driver.find_elements(By.XPATH,
                    "//*[div[@data-field='profitAndLoss']]//span")
                total_pnl = 0.0
                for cell in pnl_cells:
                    pnl_text = cell.text.strip()
                    if '$' in pnl_text:
                        try:
                            # Extract numeric value from $-1.00 format
                            pnl_value = float(pnl_text.replace('$', '').replace(',', ''))
                            total_pnl += pnl_value
                        except:
                            continue

                stats["Profit/Loss"] = f"${total_pnl:.2f}"

            except Exception as e:
                self.logger.warning(f"Could not extract position details: {e}")

        except Exception as e:
            self.logger.warning(f"Could not extract positions stats: {e}")

def _extract_account_balance(self, stats):
    """Extract account balance from various UI locations"""

```

```

try:
    # Look for balance in common locations
    balance_selectors = [
        "//span[contains(text(), 'BAL:')]//following-sibling::span",
        "//div[contains(@class, 'balance')]//span[contains(text(), '$')]",
        "//span[contains(text(), '$') and contains(@class, 'balance')]",
        "//div[contains(@aria-label, 'balance')]//span",
        "//span[text()][contains(., 'Balance')]//following-sibling::span"
    ]

    for selector in balance_selectors:
        try:
            balance_element = self.driver.find_element(By.XPATH, selector)
            balance_text = balance_element.text.strip()
            if '$' in balance_text and any(char.isdigit() for char in balance_text):
                stats["Balance"] = balance_text
                break
        except:
            continue

    # Try to get account number from HTML element (most reliable)
    try:
        self.logger.info(f"[ACCOUNT] Looking for account number in HTML elements...")

        # Target the specific span element containing the account number
        # <div class="MuiBox-root css-8uhtka"><span>...</span><span>|</span><span>50KTC-V2-342449</span></div>
        account_selectors = [
            "//div[contains(@class, 'MuiBox-root')]//span[contains(text(), 'V2-')]", # Span with V2-
            "//span[contains(text(), '50KTC-V2-')]", # Direct span with account pattern
            "//span[contains(text(), 'V2-') and contains(text(), '-')] ", # Any span with V2 pattern
            "//div[contains(@class, 'css-8uhtka')]//span[last()]", # Last span in the specific div
            "//div[contains(@class, 'css-8uhtka')]//span[3]" # Third span in the specific div
        ]

        account_found = False
        for i, selector in enumerate(account_selectors):
            try:
                account_element = self.driver.find_element(By.XPATH, selector)
                account_text = account_element.text.strip()
                self.logger.info(f"[ACCOUNT] Selector {i} found: '{account_text}'")

                if account_text and "V2-" in account_text and "-" in account_text:
                    # Format: "50KTC-V2-342449-32181797" -> "V2-...1797"
                    account_parts = account_text.split("-")
                    self.logger.info(f"[ACCOUNT] Account parts: {account_parts}")
                    if len(account_parts) >= 4: # ["50KTC", "V2", "342449", "32181797"]
                        last_part = account_parts[-1] # "32181797"
                        if len(last_part) >= 4:
                            formatted_account = f"V2-...{last_part[-4:]}" # "V2-...1797"
                            stats["Account Number"] = formatted_account
                            self.logger.info(
                                f"■ [ACCOUNT] Formatted TopStepX account number: {formatted_account}"
                            )
                            account_found = True
                            break
            except Exception as selector_error:
                self.logger.debug(f"[ACCOUNT] Selector {i} failed: {selector_error}")
                continue

        if not account_found:
            self.logger.info(f"[ACCOUNT] No account number found in HTML elements")

```

```

except Exception as e:
    self.logger.warning(f"Could not extract account from HTML: {e}")

# If title extraction didn't work, try page elements
if stats["Account Number"] == "Unknown":
    self.logger.info("[ACCOUNT] Title extraction failed, trying page elements...")
    account_selectors = [
        "//span[contains(text(), 'Account')]/following-sibling::span",
        "//div[contains(@class, 'account')]/span",
        "//span[contains(@class, 'account-number')]",
        "//span[contains(text(), 'Trading Combine')]", # This might be picking up the wrong
        "//span[contains(text(), '50K')]"
    ]

    for i, selector in enumerate(account_selectors):
        try:
            account_element = self.driver.find_element(By.XPATH, selector)
            account_text = account_element.text.strip()
            self.logger.info(f"[ACCOUNT] Selector {i} found: '{account_text}'")

            if account_text and len(account_text) > 3:
                # Skip if this is just descriptive text we don't want
                if account_text in ["$50K TRADING COMBINE", "Trading Combine", "50K"]:
                    self.logger.info(f"[ACCOUNT] Skipping descriptive text: '{account_text}'")
                    continue

                # Format TopStepX account number like other prop firms
                # From "50KTC-V2-342449-32181797" to "V2-...1797"
                if "V2-" in account_text and "-" in account_text:
                    parts = account_text.split("-")
                    # Find V2 part and get last 4 digits of final part
                    v2_index = -1
                    for j, part in enumerate(parts):
                        if part == "V2":
                            v2_index = j
                            break

                    if v2_index >= 0 and len(parts) > v2_index + 1:
                        last_part = parts[-1] # Get the last part (32181797)
                        if len(last_part) >= 4:
                            formatted_account = f"V2-...{last_part[-4:]}" # V2-...1797
                            stats["Account Number"] = formatted_account
                            self.logger.info(
                                f"■ [ACCOUNT] Element-based formatted account: {formatted_account}"
                            )
                            break

                # Only use as fallback if it contains useful info (not descriptive text)
                if not any(bad_text in account_text for bad_text in ["TRADING", "COMBINE", "$"]):
                    stats["Account Number"] = account_text
                    self.logger.info(f"[ACCOUNT] Using fallback account text: '{account_text}'")
                    break
        except Exception as e:
            self.logger.debug(f"Selector {i} failed: {e}")
            continue

# Final check - ensure we have a proper account number
final_account = stats["Account Number"]
if final_account == "Unknown":
    stats["Account Number"] = "TopStepX"

```

```

        self.logger.info("[ACCOUNT] Using final fallback: TopStepX")

        self.logger.info(f"■ [ACCOUNT] Final account number: '{stats['Account Number']}'")

    except Exception as e:
        self.logger.warning(f"Could not extract account balance: {e}")

def _get_stats_via_api(self):
    """Get account stats via REST API (fast path, no Selenium needed).
    Returns stats dict compatible with get_account_stats() format, or None on failure."""
    accounts = self.api_get_trading_accounts()
    if not accounts or not isinstance(accounts, list) or len(accounts) == 0:
        return None
    acct = accounts[0]
    self._api_account_id = acct.get("accountId")

    # Format account number: "50KTC-V2-342449-32181797" -> "V2-...1797"
    acct_name = acct.get("accountName", "")
    account_number = "TopStepX"
    if "V2-" in acct_name:
        parts = acct_name.split("-")
        if len(parts) >= 4 and len(parts[-1]) >= 4:
            account_number = f"V2-...{parts[-1][-4:]}"

    balance = acct.get("balance", 0)
    realized_pnl = acct.get("realizedDayPnl", 0)
    open_pnl = acct.get("openPnl", 0)
    total_pnl = realized_pnl + open_pnl

    # Get positions for open trades count
    positions = self.api_get_positions()
    open_count = len(positions) if positions else 0

    symbol = ""
    direction = ""
    if open_count > 0 and isinstance(positions, list):
        pos = positions[0]
        symbol = pos.get("contractDisplayName", pos.get("symbolId", ""))
        size = pos.get("positionSize", 0)
        direction = "Long" if size > 0 else "Short" if size < 0 else ""

    return {
        "Account Number": account_number,
        "Balance": f"${balance:,.2f}",
        "Profit/Loss": f"${total_pnl:,.2f}",
        "Open Trades": str(open_count),
        "Symbol": symbol,
        "Direction": direction,
    }

def place_order(self, symbol, quantity, side, order_type="market", price=None, tp_dollars=None,
    sl_dollars=None):
    """Generic method to place orders on TopStepX"""
    try:
        side = side.upper()
        if side == "BUY":
            return self.place_buy_order(symbol, quantity, order_type, tp_dollars, sl_dollars)
        elif side == "SELL":
            return self.place_sell_order(symbol, quantity, order_type, tp_dollars, sl_dollars)
        else:

```

```

        raise ValueError(f"Invalid order side: {side}. Must be 'BUY' or 'SELL'")

    except Exception as e:
        self.logger.error(f"Failed to place {side} order: {e}")
        return {
            "success": False,
            "message": f"Failed to place {side} order: {e}",
            "platform": "TopStepX"
        }

def prepare_buy_order(self, symbol, quantity):
    """
    THREADING OPTIMIZATION: Prepare BUY order (symbol search, quantity setup) WITHOUT clicking button
    Use this before threading to minimize time inside parallel execution.

    Returns: True if preparation successful, False otherwise
    """
    with self.lock:
        try:
            if not self.is_connected():
                raise Exception("Not connected to TopStepX")

            self.logger.info(f"■ PRE-THREAD] Preparing TopStepX BUY order: {symbol} x{quantity}")

            # Ensure we're on the Order tab
            if not self.switch_to_order_tab():
                self.logger.warning("Could not switch to Order tab")
                return False

            # Step 1: Set contract symbol (SLOW - do this before threading)
            self.logger.info(f"■ PRE-THREAD] Setting symbol: {symbol}")
            if not self._set_contract_symbol(symbol):
                self.logger.error(f"Failed to set contract symbol: {symbol}")
                return False

            # Step 2: Set quantity (SLOW - do this before threading)
            self.logger.info(f"■ PRE-THREAD] Setting quantity: {quantity}")
            if not self._set_quantity(quantity):
                self.logger.error(f"Failed to set quantity: {quantity}")
                return False

            self.logger.info(f"■ [■ PRE-THREAD] Order prepared - ready for button click")
            return True

        except Exception as e:
            self.logger.error(f"■ PRE-THREAD] Preparation failed: {e}")
            return False

def execute_prepared_buy_order(self):
    """
    THREADING OPTIMIZATION: Execute already-prepared BUY order (just click button).
    Call prepare_buy_order() first, then use this inside threading for instant execution.

    Returns: Order result dict
    """
    with self.lock:
        try:
            self.logger.info(f"■ THREAD-EXEC] Clicking BUY button...")

            # Just click the button - everything else is already prepared

```

```

        if not self._click_buy_button(skip_post_trade_setup=True):
            raise Exception("Failed to click BUY button")

        self.logger.info(f"■ [■ THREAD-EXEC] BUY button clicked")

        order_id = f"TSX_BUY_{int(time.time())}"
        return {
            "success": True,
            "message": "TopStepX BUY order executed",
            "order_id": order_id,
            "platform": "TopStepX",
            "side": "BUY"
        }

    except Exception as e:
        self.logger.error(f"[■ THREAD-EXEC] Execution failed: {e}")
        return {
            "success": False,
            "message": f"TopStepX BUY execution failed: {e}",
            "platform": "TopStepX"
        }

def prepare_sell_order(self, symbol, quantity):
    """
    THREADING OPTIMIZATION: Prepare SELL order (symbol search, quantity setup) WITHOUT clicking button
    Use this before threading to minimize time inside parallel execution.

    Returns: True if preparation successful, False otherwise
    """
    with self.lock:
        try:
            if not self.is_connected():
                raise Exception("Not connected to TopStepX")

            self.logger.info(f"[■ PRE-THREAD] Preparing TopStepX SELL order: {symbol} x{quantity}")

            # Ensure we're on the Order tab
            if not self.switch_to_order_tab():
                self.logger.warning("Could not switch to Order tab")
                return False

            # Step 1: Set contract symbol (SLOW - do this before threading)
            self.logger.info(f"[■ PRE-THREAD] Setting symbol: {symbol}")
            if not self._set_contract_symbol(symbol):
                self.logger.error(f"Failed to set contract symbol: {symbol}")
                return False

            # Step 2: Set quantity (SLOW - do this before threading)
            self.logger.info(f"[■ PRE-THREAD] Setting quantity: {quantity}")
            if not self._set_quantity(quantity):
                self.logger.error(f"Failed to set quantity: {quantity}")
                return False

            self.logger.info(f"■ [■ PRE-THREAD] Order prepared - ready for button click")
            return True

        except Exception as e:
            self.logger.error(f"[■ PRE-THREAD] Preparation failed: {e}")
            return False

def execute_prepared_sell_order(self):

```



```

"""
THREADING OPTIMIZATION: Execute already-prepared SELL order (just click button).
Call prepare_sell_order() first, then use this inside threading for instant execution.

Returns: Order result dict
"""
with self.lock:
    try:
        self.logger.info(f"[■ THREAD-EXEC] Clicking SELL button...")

        # Just click the button - everything else is already prepared
        if not self._click_sell_button(skip_post_trade_setup=True):
            raise Exception("Failed to click SELL button")

        self.logger.info(f"[■ THREAD-EXEC] SELL button clicked")

        order_id = f"TSX_SELL_{int(time.time())}"
        return {
            "success": True,
            "message": "TopStepX SELL order executed",
            "order_id": order_id,
            "platform": "TopStepX",
            "side": "SELL"
        }

    except Exception as e:
        self.logger.error(f"[■ THREAD-EXEC] Execution failed: {e}")
        return {
            "success": False,
            "message": f"TopStepX SELL execution failed: {e}",
            "platform": "TopStepX"
        }

def place_buy_order(self, symbol, quantity, order_type="market", tp_dollars=None, sl_dollars=None,
    skip_post_trade_setup=False):
    """
    Place a BUY order on TopStepX with optional post-trade TP/SL setup

    Args:
        symbol: Trading symbol (e.g., 'NQM25')
        quantity: Number of contracts
        order_type: Order type (default: 'market')
        tp_dollars: Take profit in dollars
        sl_dollars: Stop loss in dollars
        skip_post_trade_setup: If True, skip TP/SL editing in Positions tab.
                               Caller should manually call setup_post_trade_tp_sl_positions() later.
                               Use this in hedging mode to place MT5 trade first for speed.
    """
    # Acquire lock to prevent stats fetching during order placement
    with self.lock:
        try:
            # TIME CHECKPOINT: Start overall timing
            order_start_time = time.time()

            # Dismiss any MUI backdrop overlay before interacting with UI
            self._dismiss_backdrop()

            if not self.is_connected():
                raise Exception("Not connected to TopStepX")

            if skip_post_trade_setup:

```

```

        self.logger.info(
            f"■ [FAST MODE] Placing TopStepX BUY order: {symbol} x{quantity} (TP/SL setup de
else:
    self.logger.info(f"Placing TopStepX BUY order: {symbol} x{quantity}")

# Ensure we're on the Order tab for trade entry
if not self.switch_to_order_tab():
    self.logger.warning("Could not switch to Order tab, attempting trade anyway")

# MANDATORY: Setup TP/SL values are REQUIRED
if tp_dollars is None and sl_dollars is None:
    raise Exception("■ TRADE BLOCKED: TP and SL values are REQUIRED before placing any t

if tp_dollars is None or sl_dollars is None:
    self.logger.warning(f"■ Missing TP or SL: TP=${tp_dollars}, SL=${sl_dollars}")
    # Set default values if only one is missing
    if tp_dollars is None:
        tp_dollars = 500 # Default TP: $500
        self.logger.warning(f"■ Using default TP: ${tp_dollars}")
    if sl_dollars is None:
        sl_dollars = 1000 # Default SL: $1000
        self.logger.warning(f"■ Using default SL: ${sl_dollars}")

# Enhanced execution with detailed logging
self.logger.info(f"[TRADE] Starting TopStepX BUY order execution sequence")
self.logger.info(
    f"[PARAMS] Symbol: {symbol}, Quantity: {quantity}, TP: ${tp_dollars}, SL: ${sl_dollar

# Set contract symbol with detailed logging
self.logger.info(f"[STEP 1] Setting contract symbol: {symbol}")
symbol_result = self._set_contract_symbol(symbol)
if not symbol_result:
    error_msg = f"Failed to set contract symbol: {symbol}"
    self.logger.error(f"■ [STEP 1] {error_msg}")
    raise Exception(error_msg)
else:
    self.logger.info(f"■ [STEP 1] Contract symbol set successfully: {symbol}")

# Set quantity with detailed logging
self.logger.info(f"[STEP 2] Setting quantity: {quantity}")
quantity_result = self._set_quantity(quantity)
if not quantity_result:
    error_msg = f"Failed to set quantity: {quantity}"
    self.logger.error(f"■ [STEP 2] {error_msg}")
    raise Exception(error_msg)
else:
    self.logger.info(f"■ [STEP 2] Quantity set successfully: {quantity}")

# OPTIMIZED: In FAST mode, skip everything and go straight to BUY button
if skip_post_trade_setup:
    # FAST MODE: Go directly from quantity → BUY button
    self.logger.info(f"■ [FAST MODE] Skipping order type, TP/SL - going straight to BUY"
else:
    # NORMAL MODE: Set order type if not market
    if order_type.lower() != "market":
        self.logger.info(f"[STEP 3] Setting order type: {order_type}")
        if not self._set_order_type(order_type):
            self.logger.warning(
                f"■ [STEP 3] Failed to set order type to {order_type}, using market ord
    else:

```

```

        self.logger.info(f"■ [STEP 3] Order type set: {order_type}")
    else:
        self.logger.info(f"[STEP 3] Using default market order type")

# OPTIMIZED: In FAST mode, TP/SL already skipped above
if not skip_post_trade_setup:
    # NORMAL MODE: Set TP/SL on order form before placing trade
    _orderform_tp_ok = False
    _orderform_sl_ok = False
    if tp_dollars and tp_dollars > 0:
        self.logger.info(f"[STEP 4] Setting Take Profit: ${tp_dollars}")
        if not self._set_take_profit_price(tp_dollars):
            self.logger.warning(f"■■ [STEP 4] Failed to set Take Profit on order form")
        else:
            self.logger.info(f"■ [STEP 4] Take Profit set: ${tp_dollars}")
            _orderform_tp_ok = True
    else:
        self.logger.info(f"[STEP 4] Skipping Take Profit (tp_dollars={tp_dollars})")
        _orderform_tp_ok = True # Nothing to do counts as success

    if sl_dollars and sl_dollars > 0:
        self.logger.info(f"[STEP 5] Setting Stop Loss: ${sl_dollars}")
        if not self._set_stop_loss_price(sl_dollars):
            self.logger.warning(f"■■ [STEP 5] Failed to set Stop Loss on order form")
        else:
            self.logger.info(f"■ [STEP 5] Stop Loss set: ${sl_dollars}")
            _orderform_sl_ok = True
    else:
        self.logger.info(f"[STEP 5] Skipping Stop Loss (sl_dollars={sl_dollars})")
        _orderform_sl_ok = True

# Click BUY button with detailed logging
self.logger.info(f"[STEP 6] Clicking BUY button")
buy_result = self._click_buy_button(skip_post_trade_setup=skip_post_trade_setup)
if not buy_result:
    error_msg = "Failed to click BUY button"
    self.logger.error(f"■ [STEP 6] {error_msg}")
    raise Exception(error_msg)
else:
    self.logger.info(f"■ [STEP 6] BUY button clicked successfully")

# OPTIMIZED: Minimal wait reduced from 0.3s to 0.1s
time.sleep(0.1)

# CONDITIONAL: Skip post-trade setup if requested (for hedging mode speed)
if skip_post_trade_setup:
    # TIME CHECKPOINT: End timing for fast mode
    order_end_time = time.time()
    total_order_time_ms = (order_end_time - order_start_time) * 1000

    self.logger.info("■ FAST MODE: Skipping post-trade TP/SL setup for hedging mode")
    self.logger.info(
        "■ Caller should call setup_post_trade_tp_sl_positions() after MT5 hedge placement"
    )
    self.logger.info(f"■■ Total order placement time: {total_order_time_ms:.1f}ms")

    # Capture snapshot if order placement took too long
    self._capture_delay_snapshot("fast_order_placement", total_order_time_ms,
                                threshold_ms=1000)

    order_id = f"TSX_BUY_{int(time.time())}"

```

```

self.logger.info(f"■ TopStepX BUY order submitted (fast mode): {symbol} x{quantity}")

return {
    "success": True,
    "message": f"TopStepX BUY order placed (fast mode): {symbol} x{quantity}",
    "order_id": order_id,
    "platform": "TopStepX",
    "symbol": symbol,
    "quantity": quantity,
    "side": "BUY",
    "tp_dollars": tp_dollars, # Return these for later setup
    "sl_dollars": sl_dollars,
    "placement_time_ms": total_order_time_ms
}

# NORMAL MODE: Continue with TP/SL setup as before
self.logger.info(f"[POST-TRADE] Verifying TP/SL setup")

# SPEED: If both TP and SL were already set on the order form, the broker
# attached them to the working order – skip the redundant Positions-tab pass
# entirely (saves ~15-25s of grid wait + field edits per trade).
_need_positions_setup = (
    ((tp_dollars and tp_dollars > 0) and not _orderform_tp_ok) or
    ((sl_dollars and sl_dollars > 0) and not _orderform_sl_ok)
)

if _need_positions_setup:
    # Switch to positions tab to verify trade and setup TP/SL
    if self.switch_to_positions_tab():
        self.logger.info("■ Switched to Positions tab for TP/SL fallback")
    else:
        self.logger.warning("■ Could not switch to Positions tab")
    setup_result = self.setup_post_trade_tp_sl_positions(tp_dollars, sl_dollars)
    if not setup_result:
        self.logger.warning(
            "■■ TP/SL setup in positions failed - trade placed but risk not managed")
    else:
        self.logger.info("■ TP/SL setup completed in positions")
else:
    self.logger.info("■ Skipping Positions-tab TP/SL setup (already set on order form)")

order_id = f"TSX_BUY_{int(time.time())}"
self.logger.info(f"TopStepX BUY order submitted: {symbol} x{quantity}")

# POST-TRADE VERIFICATION: Check what was actually executed (DELAYED)
# Delay this check significantly to avoid interrupting trade processing
actual_executed_symbol = None
try:
    self.logger.info("[POST-TRADE] Scheduling delayed verification in 3 seconds...")
    # Use a separate thread or delayed execution to avoid blocking
    import threading

    def delayed_verification():
        try:
            time.sleep(3) # Wait longer for trade to fully process
            # Check positions table for the actual executed symbol
            stats = self.get_account_stats()
            if stats and stats.get("Symbol"):
                executed_symbol = stats["Symbol"]
                self.logger.info(f"[POST-TRADE] Actual executed symbol: '{executed_symbol}'")

```

```

        # Critical verification
        if symbol.upper() in ['NQM25', 'NQM26'] and executed_symbol:
            if 'MNQ' in executed_symbol.upper() and symbol.upper(
                ) not in executed_symbol.upper():
                self.logger.error(f"■ CRITICAL MISMATCH DETECTED:")
                self.logger.error(f"    Requested: {symbol} (full contract)")
                self.logger.error(f"    Executed: {executed_symbol} (micro contract)")
                self.logger.error(
                    f"    Result: 5x smaller position size than intended!")
            elif executed_symbol.upper() == symbol.upper():
                self.logger.info(
                    f"■ VERIFIED: Correct contract executed - {executed_symbol}")
            else:
                self.logger.warning(
                    f"■■ Unexpected executed symbol: {executed_symbol}")
        except Exception as verify_error:
            self.logger.warning(f"Delayed post-trade verification failed: {verify_error}")

        # Start verification in background thread to avoid blocking
        verification_thread = threading.Thread(target=delayed_verification, daemon=True)
        verification_thread.start()

    except Exception as verify_error:
        self.logger.warning(f"Could not start delayed verification: {verify_error}")

    result_dict = {
        "success": True,
        "message": f"TopStepX BUY order placed: {symbol} x{quantity}",
        "order_id": order_id,
        "platform": "TopStepX",
        "symbol": symbol,
        "executed_symbol": actual_executed_symbol or symbol,
        "quantity": quantity,
        "side": "BUY"
    }
    return result_dict

    except Exception as e:
        self.logger.error(f"TopStepX BUY order failed: {e}")
        return {
            "success": False,
            "message": f"TopStepX BUY order failed: {e}",
            "platform": "TopStepX"
        }

def place_sell_order(self, symbol, quantity, order_type="market", tp_dollars=None, sl_dollars=None,
    skip_post_trade_setup=False):
    """
    Place a SELL order on TopStepX with optional post-trade TP/SL setup

    Args:
        symbol: Trading symbol (e.g., 'NQM25')
        quantity: Number of contracts
        order_type: Order type (default: 'market')
        tp_dollars: Take profit in dollars
        sl_dollars: Stop loss in dollars
        skip_post_trade_setup: If True, skip TP/SL editing in Positions tab.
            Caller should manually call setup_post_trade_tp_sl_positions() later.
            Use this in hedging mode to place MT5 trade first for speed.
    """

```

```

# Acquire lock to prevent stats fetching during order placement
with self.lock:
    try:
        # TIME CHECKPOINT: Start overall timing
        order_start_time = time.time()

        # Dismiss any MUI backdrop overlay before interacting with UI
        self._dismiss_backdrop()

        if not self.is_connected():
            raise Exception("Not connected to TopStepX")

        if skip_post_trade_setup:
            self.logger.info(
                f"■ [FAST MODE] Placing TopStepX SELL order: {symbol} x{quantity} (TP/SL setup d
            else:
                self.logger.info(f"Placing TopStepX SELL order: {symbol} x{quantity}")

        # Ensure we're on the Order tab for trade entry
        if not self.switch_to_order_tab():
            self.logger.warning("Could not switch to Order tab, attempting trade anyway")

        # MANDATORY: Setup TP/SL values are REQUIRED
        if tp_dollars is None and sl_dollars is None:
            raise Exception("■ TRADE BLOCKED: TP and SL values are REQUIRED before placing any t

        if tp_dollars is None or sl_dollars is None:
            self.logger.warning(f"■ Missing TP or SL: TP=${tp_dollars}, SL=${sl_dollars}")
            # Set default values if only one is missing
            if tp_dollars is None:
                tp_dollars = 500 # Default TP: $500
                self.logger.warning(f"■ Using default TP: ${tp_dollars}")
            if sl_dollars is None:
                sl_dollars = 1000 # Default SL: $1000
                self.logger.warning(f"■ Using default SL: ${sl_dollars}")

        # Set contract symbol - re-enabled for exact test replication
        self.logger.info(f"■ [EXACT-TEST-SELL] Setting contract symbol: {symbol}")
        if not self._set_contract_symbol(symbol):
            raise Exception(f"Failed to set contract symbol: {symbol}")

        # Set quantity
        self.logger.info(f"[STEP 2] Setting quantity: {quantity}")
        if not self._set_quantity(quantity):
            raise Exception(f"Failed to set quantity: {quantity}")
        else:
            self.logger.info(f"■ [STEP 2] Quantity set successfully: {quantity}")

        # OPTIMIZED: In FAST mode, skip everything and go straight to SELL button
        if skip_post_trade_setup:
            # FAST MODE: Go directly from quantity → SELL button
            self.logger.info(f"■ [FAST MODE] Skipping order type, TP/SL - going straight to SELL
        else:
            # NORMAL MODE: Set order type if not market
            if order_type.lower() != "market":
                if not self._set_order_type(order_type):
                    self.logger.warning(
                        f"Failed to set order type to {order_type}, using market order")

        # OPTIMIZED: In FAST mode, TP/SL already skipped above

```

```

_orderform_tp_ok = False
_orderform_sl_ok = False
if not skip_post_trade_setup:
    # NORMAL MODE: Set TP/SL on order form before placing trade
    if tp_dollars and tp_dollars > 0:
        self.logger.info(f"Setting Take Profit: ${tp_dollars}")
        if not self._set_take_profit_price(tp_dollars):
            self.logger.warning(f"Failed to set Take Profit on order form")
        else:
            self.logger.info(f"Take Profit set: ${tp_dollars}")
            _orderform_tp_ok = True
    else:
        _orderform_tp_ok = True

    if sl_dollars and sl_dollars > 0:
        self.logger.info(f"Setting Stop Loss: ${sl_dollars}")
        if not self._set_stop_loss_price(sl_dollars):
            self.logger.warning(f"Failed to set Stop Loss on order form")
        else:
            self.logger.info(f"Stop Loss set: ${sl_dollars}")
            _orderform_sl_ok = True
    else:
        _orderform_sl_ok = True

# Click SELL button
if not self._click_sell_button(skip_post_trade_setup=skip_post_trade_setup):
    raise Exception("Failed to click SELL button")

# Minimal wait for order processing
time.sleep(0.1)

# CONDITIONAL: Skip post-trade setup if requested (for hedging mode speed)
if skip_post_trade_setup:
    # TIME CHECKPOINT: End timing for fast mode
    order_end_time = time.time()
    total_order_time_ms = (order_end_time - order_start_time) * 1000

    self.logger.info("■ FAST MODE: Skipping post-trade TP/SL setup for hedging mode")
    self.logger.info(
        "■ Caller should call setup_post_trade_tp_sl_positions() after MT5 hedge placement"
    )
    self.logger.info(f"■■ Total order placement time: {total_order_time_ms:.1f}ms")

    # Capture snapshot if order placement took too long
    self._capture_delay_snapshot("fast_sell_order_placement", total_order_time_ms,
                                threshold_ms=1000)

    order_id = f"TSX_SELL_{int(time.time())}"
    self.logger.info(f"■ TopStepX SELL order submitted (fast mode): {symbol} x{quantity}")

    return {
        "success": True,
        "message": f"TopStepX SELL order placed (fast mode): {symbol} x{quantity}",
        "order_id": order_id,
        "platform": "TopStepX",
        "symbol": symbol,
        "quantity": quantity,
        "side": "SELL",
        "tp_dollars": tp_dollars, # Return these for later setup
        "sl_dollars": sl_dollars,
        "placement_time_ms": total_order_time_ms
    }

```

```

    }

    # NORMAL MODE: Continue with TP/SL setup as before
    self.logger.info(f"[POST-TRADE] Verifying TP/SL setup")

    # SPEED: skip the redundant Positions-tab pass when order form already attached TP/SL
    _need_positions_setup = (
        ((tp_dollars and tp_dollars > 0) and not _orderform_tp_ok) or
        ((sl_dollars and sl_dollars > 0) and not _orderform_sl_ok)
    )

    if _need_positions_setup:
        if self.switch_to_positions_tab():
            self.logger.info("■ Switched to Positions tab for TP/SL fallback")
        else:
            self.logger.warning("■ Could not switch to Positions tab")
            setup_result = self.setup_post_trade_tp_sl_positions(tp_dollars, sl_dollars)
            if not setup_result:
                self.logger.warning(
                    "TP/SL setup in positions failed - trade placed but risk not managed")
            else:
                self.logger.info("TP/SL setup completed in positions")
        else:
            self.logger.info("■ Skipping Positions-tab TP/SL setup (already set on order form)")

    order_id = f"TSX_SELL_{int(time.time())}"
    self.logger.info(f"TopStepX SELL order submitted: {symbol} x{quantity}")

    return {
        "success": True,
        "message": f"TopStepX SELL order placed: {symbol} x{quantity}",
        "order_id": order_id,
        "platform": "TopStepX",
        "symbol": symbol,
        "quantity": quantity,
        "side": "SELL"
    }

except Exception as e:
    self.logger.error(f"TopStepX SELL order failed: {e}")
    return {
        "success": False,
        "message": f"TopStepX SELL order failed: {e}",
        "platform": "TopStepX"
    }

def close_all_positions(self):
    """Close all open positions on TopStepX"""
    try:
        if not self.is_connected():
            raise Exception("Not connected to TopStepX")

        self.logger.info("Closing all TopStepX positions")

        # Navigate to trading page if not already there
        self._ensure_on_trading_page()

        # Click "Flatten All" button to close all positions
        if not self._click_flatten_all_button():
            self.logger.warning("Failed to click Flatten All button, trying alternative methods")

```



```

        # Alternative: Check for individual position close buttons
        if not self._close_individual_positions():
            raise Exception("Failed to close positions using any method")

    # Wait for positions to close
    time.sleep(2)

    self.logger.info("All TopStepX positions closed successfully")
    return {
        "success": True,
        "message": "All TopStepX positions closed",
        "platform": "TopStepX"
    }

except Exception as e:
    self.logger.error(f"Failed to close TopStepX positions: {e}")
    return {
        "success": False,
        "message": f"Failed to close TopStepX positions: {e}",
        "platform": "TopStepX"
    }

def get_positions(self):
    """Get current positions from TopStepX"""
    try:
        if not self.is_connected():
            raise Exception("Not connected to TopStepX")

        self._ensure_on_trading_page()

        # Navigate to positions tab if needed
        positions_tab_selectors = [
            "//div[contains(text(), 'Positions')][@role='tab']",
            "//button[contains(text(), 'Positions')]",
            "//tab[contains(text(), 'Position')]"
        ]

        for selector in positions_tab_selectors:
            try:
                positions_tab = self.driver.find_element(By.XPATH, selector)
                positions_tab.click()
                time.sleep(1)
                break
            except:
                continue

        positions = []

        # Look for position data in the grid
        try:
            # Check if there are any positions
            no_positions_elements = self.driver.find_elements(By.XPATH,
                "//*[contains(text(), 'No Positions') or contains(text(), 'No Active Position')]")

            if no_positions_elements:
                self.logger.info("No positions found on TopStepX")
                return positions

            # Try to extract position data from the grid
            position_rows = self.driver.find_elements(By.XPATH,

```

```

        "//div[@role='grid']//div[@role='row'][position()>1]") # Skip header row

    for row in position_rows:
        try:
            cells = row.find_elements(By.XPATH, ".//div[@role='gridcell']")
            if len(cells) >= 4: # Ensure we have enough data
                position = {
                    "symbol": cells[1].text.strip() if len(cells) > 1 else "N/A",
                    "quantity": cells[2].text.strip() if len(cells) > 2 else "0",
                    "side": "LONG" if "+" in cells[2].text else "SHORT" if "-" in cells[
                        2].text else "N/A",
                    "entry_price": cells[3].text.strip() if len(cells) > 3 else "N/A",
                    "pnl": cells[6].text.strip() if len(cells) > 6 else "N/A",
                    "platform": "TopStepX"
                }
                positions.append(position)
            except Exception as e:
                self.logger.warning(f"Failed to parse position row: {e}")
                continue

    except Exception as e:
        self.logger.warning(f"Failed to extract position data: {e}")

    self.logger.info(f"Retrieved {len(positions)} positions from TopStepX")
    return positions

except Exception as e:
    self.logger.error(f"Failed to get TopStepX positions: {e}")
    return []

def get_orders(self):
    """Get current pending orders from TopStepX"""
    try:
        if not self.is_connected():
            raise Exception("Not connected to TopStepX")

        self._ensure_on_trading_page()

        # Navigate to orders tab
        orders_tab_selectors = [
            "//div[contains(text(), 'Orders')][@role='tab']",
            "//button[contains(text(), 'Orders')]",
            "//tab[contains(text(), 'Order')]"
        ]

        for selector in orders_tab_selectors:
            try:
                orders_tab = self.driver.find_element(By.XPATH, selector)
                orders_tab.click()
                time.sleep(1)
                break
            except:
                continue

        orders = []

        # Look for order data in the grid
        try:
            # Check if there are any orders
            no_orders_elements = self.driver.find_elements(By.XPATH,

```

```

        "//*[contains(text(), 'No Orders') or contains(text(), 'No Pending Orders')]")

    if no_orders_elements:
        self.logger.info("No pending orders found on TopStepX")
        return orders

    # Try to extract order data from the grid
    order_rows = self.driver.find_elements(By.XPATH,
        "//*[div[@role='grid']]//div[@role='row'][position()>1]") # Skip header row

    for row in order_rows:
        try:
            cells = row.find_elements(By.XPATH, ".//div[@role='gridcell']")
            if len(cells) >= 4:
                order = {
                    "symbol": cells[1].text.strip() if len(cells) > 1 else "N/A",
                    "quantity": cells[2].text.strip() if len(cells) > 2 else "0",
                    "side": "BUY" if "Buy" in cells[3].text else "SELL" if "Sell" in cells[
                        3].text else "N/A",
                    "order_type": cells[4].text.strip() if len(cells) > 4 else "N/A",
                    "price": cells[5].text.strip() if len(cells) > 5 else "N/A",
                    "status": cells[6].text.strip() if len(cells) > 6 else "PENDING",
                    "platform": "TopStepX"
                }
                orders.append(order)
            except Exception as e:
                self.logger.warning(f"Failed to parse order row: {e}")
                continue

    except Exception as e:
        self.logger.warning(f"Failed to extract order data: {e}")

    self.logger.info(f"Retrieved {len(orders)} orders from TopStepX")
    return orders

except Exception as e:
    self.logger.error(f"Failed to get TopStepX orders: {e}")
    return []

def cancel_all_orders(self):
    """Cancel all pending orders on TopStepX"""
    try:
        if not self.is_connected():
            raise Exception("Not connected to TopStepX")

        self.logger.info("Cancelling all TopStepX orders")

        # Navigate to trading page
        self._ensure_on_trading_page()

        # Click "Cancel All" button
        cancel_all_selectors = [
            "//*[button[contains(text(), 'Cancel All')]]",
            "//*[button[contains(text(), 'CANCEL ALL')]]"
        ]

        cancel_button = None
        for selector in cancel_all_selectors:
            try:
                cancel_button = WebDriverWait(self.driver, 5).until(

```

```

        EC.element_to_be_clickable((By.XPATH, selector))
    )
    break
except TimeoutException:
    continue

if cancel_button:
    cancel_button.click()
    self.logger.info("Cancel All button clicked")

    # Handle confirmation if needed
    time.sleep(1)
    self._handle_order_confirmation()

    return {
        "success": True,
        "message": "All TopStepX orders cancelled",
        "platform": "TopStepX"
    }
else:
    self.logger.warning("Cancel All button not found or not enabled")
    return {
        "success": False,
        "message": "Cancel All button not available",
        "platform": "TopStepX"
    }

except Exception as e:
    self.logger.error(f"Failed to cancel TopStepX orders: {e}")
    return {
        "success": False,
        "message": f"Failed to cancel TopStepX orders: {e}",
        "platform": "TopStepX"
    }

def edit_position_tp_sl(self, symbol=None, tp_dollars=None, sl_dollars=None):
    """
    Edit Take Profit and Stop Loss for existing positions on TopStepX
    Args:
        symbol: Symbol to edit (optional, if None edits first position)
        tp_dollars: Take Profit amount in dollars
        sl_dollars: Stop Loss amount in dollars
    """
    try:
        if not self.is_connected():
            raise Exception("Not connected to TopStepX")

        self.logger.info(f"Editing TP/SL for TopStepX position: TP=${tp_dollars}, SL=${sl_dollars}")

        # Navigate to trading page
        self._ensure_on_trading_page()

        # Navigate to Positions tab
        if not self._navigate_to_positions_tab():
            raise Exception("Failed to navigate to Positions tab")

        # Find the position row to edit
        position_row = self._find_position_row(symbol)
        if not position_row:
            raise Exception(f"Position not found for symbol: {symbol or 'any'}")
    
```

```

# Edit Take Profit if specified
if tp_dollars is not None:
    if self._edit_position_tp(position_row, tp_dollars):
        self.logger.info(f"Successfully set TP to ${tp_dollars}")
    else:
        self.logger.warning(f"Failed to set TP to ${tp_dollars}")

# Edit Stop Loss if specified
if sl_dollars is not None:
    if self._edit_position_sl(position_row, sl_dollars):
        self.logger.info(f"Successfully set SL to ${sl_dollars}")
    else:
        self.logger.warning(f"Failed to set SL to ${sl_dollars}")

return {
    "success": True,
    "message": f"TopStepX TP/SL updated: TP=${tp_dollars}, SL=${sl_dollars}",
    "platform": "TopStepX"
}

except Exception as e:
    self.logger.error(f"Failed to edit TopStepX TP/SL: {e}")
    return {
        "success": False,
        "message": f"Failed to edit TopStepX TP/SL: {e}",
        "platform": "TopStepX"
    }

def _navigate_to_positions_tab(self):
    """Navigate to the Positions tab using TopStepX-specific selectors"""
    try:
        self._dismiss_backdrop()
        # Look for Positions tab using the exact HTML structure provided
        positions_tab_selectors = [
            # Primary selector based on provided HTML - targets the clickable tab button
            "//*[role='tab' and contains(@id, 'positionTab')]",
            "//*[class='dock-tab-btn' and contains(@aria-controls, 'positionTab')]",
            # Secondary selector - targets the drag initiator with Positions text
            "//*[class='drag-initiator' and @role='tab' and contains(text(), 'Positions')]",
            # Fallback selectors for different UI variations
            "//*[contains(@class, 'dock-tab') and .//text()[contains(., 'Positions')]]",
            "//*[contains(text(), 'Positions')][@role='tab']",
            "//*[button[contains(text(), 'Positions')]",
            # Generic fallback
            "//*[contains(@class, 'tab') and contains(text(), 'Positions')]"
        ]

        self.logger.info("Attempting to navigate to Positions tab...")

        for i, selector in enumerate(positions_tab_selectors, 1):
            try:
                self.logger.debug(f"Trying selector {i}: {selector}")
                positions_tab = WebDriverWait(self.driver, 5).until(
                    EC.presence_of_element_located((By.XPATH, selector))
                )

                # JS click to bypass any backdrop overlay
                self.driver.execute_script("arguments[0].click();", positions_tab)
                time.sleep(1.5) # Allow tab content to load

                # Verify we're on the positions tab by checking for positions content

```

```

        try:
            # Look for positions grid or "No Positions" message
            WebDriverWait(self.driver, 3).until(
                EC.any_of(
                    EC.presence_of_element_located((By.XPATH, "//div[@role='grid']")),
                    EC.presence_of_element_located((By.XPATH,
                                                    "//*[contains(text(), 'No Positions') or contains(text(), 'No Active
                                                    )
                )
            )
            self.logger.info(f"✓ Successfully navigated to Positions tab using selector {i}")
            return True
        except TimeoutException:
            self.logger.warning(f"Tab clicked but positions content not found with selector {i}")
            continue

    except TimeoutException:
        self.logger.debug(f"Selector {i} not found: {selector}")
        continue
    except Exception as e:
        self.logger.warning(f"Error with selector {i}: {e}")
        continue

    self.logger.error("Could not find or click Positions tab with any selector")
    return False

except Exception as e:
    self.logger.error(f"Failed to navigate to Positions tab: {e}")
    return False

def _dismiss_backdrop(self):
    """Dismiss any MUI backdrop/modal overlay that blocks clicks on the trading page."""
    try:
        self.driver.execute_script("""
            // Click all backdrop overlays to dismiss them
            document.querySelectorAll('.MuiBackdrop-root').forEach(el => el.click());
            // Also try pressing Escape to close any open popover/modal
            document.dispatchEvent(new KeyboardEvent('keydown', {key: 'Escape', bubbles: true}));
        """)
        time.sleep(0.3)
    except Exception:
        pass

def _navigate_to_trading_tab(self):
    """Navigate to the Trading tab using TopStepX-specific selectors"""
    try:
        self._dismiss_backdrop()
        # Look for Trading tab using similar structure to positions tab
        trading_tab_selectors = [
            # Primary selector - targets the trading tab button
            "//div[@role='tab' and contains(@id, 'trading')]",
            "//div[@class='dock-tab-btn' and contains(@aria-controls, 'trading')]",
            # Secondary selector - targets tab with Trading text
            "//div[@class='drag-initiator' and @role='tab' and contains(text(), 'Trading')]",
            # Fallback selectors for different UI variations
            "//div[contains(@class, 'dock-tab') and .//text()[contains(., 'Trading')]]",
            "//div[contains(text(), 'Trading')][@role='tab']",
            "//button[contains(text(), 'Trading')]",
            # Generic fallback
            "//div[contains(@class, 'tab') and contains(text(), 'Trading')]"
        ]
    ]

```

```

self.logger.info("Attempting to navigate to Trading tab...")

for i, selector in enumerate(trading_tab_selectors, 1):
    try:
        self.logger.debug(f"Trying selector {i}: {selector}")
        trading_tab = WebDriverWait(self.driver, 5).until(
            EC.presence_of_element_located((By.XPATH, selector))
        )

        # JS click to bypass any backdrop overlay
        self.driver.execute_script("arguments[0].click();", trading_tab)
        time.sleep(1.5) # Allow tab content to load

        # Verify we're on the trading tab by checking for trading interface elements
        try:
            # Look for Buy/Sell buttons or order form
            WebDriverWait(self.driver, 3).until(
                EC.any_of(
                    EC.presence_of_element_located((By.XPATH,
                        "//button[contains(text(), 'Buy') or contains(text(), 'BUY')]")),
                    EC.presence_of_element_located((By.XPATH,
                        "//input[@placeholder='Quantity' or contains(@aria-label, 'Quantity')]"))
                )
            )
            self.logger.info(f"✓ Successfully navigated to Trading tab using selector {i}")
            return True
        except TimeoutException:
            self.logger.warning(f"Tab clicked but trading content not found with selector {i}")
            continue

    except TimeoutException:
        self.logger.debug(f"Selector {i} not found: {selector}")
        continue
    except Exception as e:
        self.logger.warning(f"Error with selector {i}: {e}")
        continue

self.logger.error("Could not find or click Trading tab with any selector")
return False

except Exception as e:
    self.logger.error(f"Failed to navigate to Trading tab: {e}")
    return False

def _click_order_tab(self):
    """Click the Order tab to access order editing fields after placing an order"""
    try:
        self.logger.info("■ Attempting to click Order tab...")

        # Simplified selectors based on actual HTML structure
        order_tab_selectors = [
            # Most specific - targets the exact button with orderCardTab ID
            "//div[@role='tab' and contains(@id, 'orderCardTab')]",

            # Targets dock-tab-btn with Order text inside
            "//div[@class='dock-tab-btn' and .//span[text()='Order']]",

            # Any tab role containing Order text
            "//div[@role='tab' and contains(., 'Order') and not(contains(., 'Positions'))]",

            # Broadest fallback

```

```

        "//*[contains(@class, 'dock-tab') and .//text()='Order']"
    ]

    for i, selector in enumerate(order_tab_selectors, 1):
        try:
            self.logger.info(f"[ATTEMPT {i}] Trying selector: {selector}")

            # Try to find the element
            order_tab = WebDriverWait(self.driver, 3).until(
                EC.presence_of_element_located((By.XPATH, selector))
            )

            self.logger.info(f"✓ Element found with selector {i}")

            # Try multiple click methods
            click_success = False

            # Method 1: Standard click
            try:
                order_tab.click()
                self.logger.info(f"✓ Standard click succeeded")
                click_success = True
            except Exception as e:
                self.logger.warning(f"Standard click failed: {e}")

            # Method 2: JavaScript click (more reliable)
            if not click_success:
                try:
                    self.driver.execute_script("arguments[0].click();", order_tab)
                    self.logger.info(f"✓ JavaScript click succeeded")
                    click_success = True
                except Exception as e:
                    self.logger.warning(f"JavaScript click failed: {e}")

            # Method 3: Action chains click
            if not click_success:
                try:
                    from selenium.webdriver.common.action_chains import ActionChains
                    ActionChains(self.driver).move_to_element(order_tab).click().perform()
                    self.logger.info(f"✓ ActionChains click succeeded")
                    click_success = True
                except Exception as e:
                    self.logger.warning(f"ActionChains click failed: {e}")

            if click_success:
                self.logger.info(f"■ Order tab clicked successfully with selector {i}")
                time.sleep(1.5) # Allow tab to activate
                return True
            else:
                self.logger.warning(f"All click methods failed for selector {i}")
                continue

        except TimeoutException:
            self.logger.debug(f"Selector {i} not found within timeout")
            continue
        except Exception as e:
            self.logger.warning(f"Error with selector {i}: {e}")
            continue

    # If all selectors failed, log available tabs for debugging

```



```

self.logger.error("■ Could not click Order tab with any selector")
try:
    all_tabs = self.driver.find_elements(By.XPATH, "//div[@role='tab']")
    self.logger.info(f"Available tabs ({len(all_tabs)}):")
    for idx, tab in enumerate(all_tabs):
        tab_text = tab.text.strip() or tab.get_attribute('id') or 'Unknown'
        self.logger.info(f"  Tab {idx+1}: {tab_text}")
except:
    pass

return False

except Exception as e:
    self.logger.error(f"Failed to click Order tab: {e}")
    import traceback
    self.logger.error(traceback.format_exc())
    return False

def switch_tab(self, tab_name):
    """
    Generic method to switch between tabs in the TopStepX interface
    OPTIMIZED: Fast tab switching with reduced timeouts

    Args:
        tab_name (str): Name of the tab to switch to. Options:
            - 'Positions' - View open positions and P&L
            - 'Order' - Access order entry form
            - 'Trades' - View trade history
            - 'Quotes' - View market quotes
            - 'Accounts' - View account information
            - 'Chart' - View charts

    Returns:
        bool: True if tab switch was successful, False otherwise
    """
    try:
        self._dismiss_backdrop()
        # Build selectors based on tab name (prioritize fastest/most reliable)
        selectors = [
            # Strategy 1: By exact text match (fastest)
            f"//div[@role='tab']//div[text()='{{tab_name}}']",
            # Strategy 2: By tab ID containing the tab name
            f"//div[@role='tab' and contains(@id, '{{tab_name.lower()}}')]",
            # Strategy 3: By partial text match
            f"//div[@role='tab' and contains(., '{{tab_name}}')]"
        ]

        for selector in selectors:
            try:
                # FAST: Reduced timeout from 3s to 0.5s per selector
                tab_element = WebDriverWait(self.driver, 0.5).until(
                    EC.element_to_be_clickable((By.XPATH, selector))
                )

                # FAST: Use only JS click for speed
                self.driver.execute_script("arguments[0].click();", tab_element)
                time.sleep(0.1) # Minimal wait - reduced from 0.5s
                return True

            except TimeoutException:

```

```

        continue
    except Exception:
        continue

    return False

except Exception as e:
    self.logger.error(f"Failed to switch to '{tab_name}' tab: {e}")
    return False
def switch_to_positions_tab(self):
    """
    Switch to the Positions tab to view open positions
    OPTIMIZED: Fast switch with minimal verification

    Returns:
        bool: True if successful, False otherwise
    """
    try:
        # Switch to Positions tab
        if not self.switch_tab("Positions"):
            return False

        # FAST: Minimal wait - reduced from 3s to 0.5s
        try:
            WebDriverWait(self.driver, 0.5).until(
                EC.presence_of_element_located((By.XPATH,
                    "//div[@role='grid' or contains(@class, 'MuiDataGrid')]"))
            )
        except TimeoutException:
            pass # Continue anyway

        return True

    except Exception as e:
        self.logger.error(f"Failed to switch to Positions tab: {e}")
        return False

def switch_to_order_tab(self):
    """
    Switch to the Order tab to access order entry form

    Returns:
        bool: True if successful, False otherwise
    """
    try:
        # Switch to Order tab
        if not self.switch_tab("Order"):
            self.logger.warning("Could not switch to Order tab")
            return False

        # Verify Buy button or quantity input is visible after switch
        try:
            WebDriverWait(self.driver, 3).until(
                EC.any_of(
                    EC.presence_of_element_located((By.XPATH,
                        "//button[contains(text(), 'Buy') or contains(text(), 'BUY')]")),
                    EC.presence_of_element_located((By.XPATH,
                        "//input[@placeholder='Quantity' or contains(@aria-label, 'Quantity')]"))
                )
            )
        except TimeoutException:
            pass # Continue anyway

        return True

    except Exception as e:
        self.logger.error(f"Failed to switch to Order tab: {e}")
        return False

```

```

        self.logger.debug("Verified order form is visible")
        return True
    except TimeoutException:
        self.logger.warning("Order tab switched but form not visible")
        return True # Still return True as tab switch succeeded

except Exception as e:
    self.logger.error(f"Failed to switch to Order tab: {e}")
    return False

def get_active_tab(self):
    """
    Detect which tab is currently active

    Returns:
        str: Name of active tab, or None if cannot determine

    Example:
        current_tab = self.get_active_tab()
        if current_tab == "Positions":
            # Already on positions tab
            pass
    """
    try:
        # Find element with class 'dock-tab-active'
        active_tab = self.driver.find_element(
            By.XPATH,
            "//div[contains(@class, 'dock-tab-active')]"
        )

        # Extract tab name from text content
        tab_text = active_tab.text.strip()

        # Normalize tab name (remove extra characters, emojis, etc.)
        # The Order tab may have emoji and "H" for hotkey
        if "Order" in tab_text:
            return "Order"
        elif "Position" in tab_text:
            return "Positions"
        elif "Trade" in tab_text and "Trader" not in tab_text:
            return "Trades"
        elif "Quote" in tab_text:
            return "Quotes"
        elif "Account" in tab_text:
            return "Accounts"
        elif "Chart" in tab_text:
            return "Chart"
        else:
            # Return the cleaned text
            return tab_text

    except Exception as e:
        self.logger.debug(f"Could not detect active tab: {e}")
        return None

def _find_position_row(self, symbol=None):
    """Find the position row in the positions grid"""
    try:
        # Wait for positions grid to load after tab switch
        self.logger.info("Waiting for positions grid to load...")

```

```

time.sleep(2)

# Check if there are no positions first
no_positions_elements = self.driver.find_elements(By.XPATH,
    "//*[contains(text(), 'No Positions') or contains(text(), 'No Active Position') or contains(text(), 'No Positions')]")
if no_positions_elements:
    self.logger.warning("No positions found on TopStepX - positions grid is empty")
    return None

# Look for position rows in the MUI DataGrid with multiple selector patterns
position_row_selectors = [
    # MUI DataGrid specific selectors (most likely)
    "//*[contains(@class, 'MuiDataGrid-main')]/div[@role='row'][position()>1]", # MUI DataGrid row
    "//*[contains(@role='grid')]/div[@role='row'][position()>1]", # Standard grid rows (skip header)
    "//*[contains(@class, 'MuiDataGrid-row')]", # MUI DataGrid row class
    "//*[contains(@class, 'grid')]/div[contains(@class, 'row')][position()>1]", # Alternative grid rows
    "//*[table//tr[position()>1]]", # Table-based positions
    "//*[div[@role='rowgroup']]/div[@role='row']" # Row group structure
]

position_rows = []
for selector in position_row_selectors:
    try:
        rows = self.driver.find_elements(By.XPATH, selector)
        if rows:
            position_rows = rows
            self.logger.info(f"Found {len(rows)} position rows using selector: {selector}")
            break
    except Exception as e:
        self.logger.debug(f"Selector failed: {selector} - {e}")
        continue

if not position_rows:
    self.logger.warning("No position rows found in grid")
    return None

# If no specific symbol, return first position
if symbol is None:
    self.logger.info(f"Using first available position (total: {len(position_rows)})")
    return position_rows[0]

# Look for position with matching symbol
self.logger.info(f"Searching for position with symbol: {symbol}")
for i, row in enumerate(position_rows):
    try:
        # Try different cell selection methods
        cell_selectors = [
            "//*[div[@role='gridcell']]",
            "//*[div[contains(@class, 'cell')]]",
            "//*[td]"
        ]

        cells = []
        for cell_selector in cell_selectors:
            cells = row.find_elements(By.XPATH, cell_selector)
            if cells:
                break

        if len(cells) > 1:
            # Check multiple potential symbol columns (usually column 1 or 2)

```

```

        for col_idx in [1, 2, 0]: # Try column 1, then 2, then 0
            if col_idx < len(cells):
                row_symbol = cells[col_idx].text.strip()
                if row_symbol and symbol.upper() in row_symbol.upper():
                    self.logger.info(
                        f"✓ Found position row {i+1} for symbol: {row_symbol} (column {col_idx})"
                    )
                    return row

        # Log what we found for debugging
        cell_contents = [cell.text.strip() for cell in cells[:5]] # First 5 columns
        self.logger.debug(f"Row {i+1} contents: {cell_contents}")

    except Exception as e:
        self.logger.warning(f"Failed to check row {i+1} symbol: {e}")
        continue

    self.logger.warning(f"Position not found for symbol: {symbol}")
    # Log available positions for debugging
    try:
        self.logger.info("Available positions:")
        for i, row in enumerate(position_rows[:3]): # Show first 3 rows
            cells = row.find_elements(By.XPATH,
                "//*[div[@role='gridcell'] | .//div[contains(@class, 'cell')]] | .//td")
            if cells:
                cell_contents = [cell.text.strip() for cell in cells[:3]]
                self.logger.info(f"  Row {i+1}: {cell_contents}")
    except:
        pass

    return None

except Exception as e:
    self.logger.error(f"Failed to find position row: {e}")
    return None

def _edit_position_tp(self, position_row, tp_dollars):
    """Edit Take Profit by clicking the 'To Make' column (data-field='toMake')"""
    try:
        # Find the 'To Make' cell using the exact HTML structure provided
        # Look for MUI DataGrid cell with data-field="toMake"
        to_make_cell_selectors = [
            # Primary selector: exact data-field match for "toMake"
            "//*[div[@role='cell' and @data-field='toMake']]",
            # Secondary: MUI DataGrid cell with toMake field
            "//*[div[contains(@class, 'MuiDataGrid-cell') and @data-field='toMake']]",
            # Fallback: cell containing the pencil icon and $ value in toMake context
            "//*[div[@role='cell' and contains(@data-field, 'toMake')]]",
            # Generic fallback: look for editable cell with $ and pencil icon
            "//*[div[@role='cell' and contains(@class, 'editable') and .//span[contains(text(), '$)]]]"
        ]

        to_make_cell = None
        for i, selector in enumerate(to_make_cell_selectors, 1):
            try:
                to_make_cell = position_row.find_element(By.XPATH, selector)
                self.logger.info(f"✓ Found 'To Make' cell using selector {i}: {selector}")
                break
            except:
                self.logger.debug(f"Selector {i} not found: {selector}")
                continue

```

```

if not to_make_cell:
    self.logger.error("Could not find 'To Make' cell with any selector")
    return False

# Click the pencil icon or the cell itself to activate editing
pencil_icon_selectors = [
    # Click the pencil icon specifically
    ".*//svg[@data-icon='pencil']",
    # Click the span containing the $ value
    ".*//span[contains(@class, 'css-') and contains(text(), '$')]",
    # Click the cell itself if no pencil found
    ".*"
]

clicked = False
for selector in pencil_icon_selectors:
    try:
        click_target = to_make_cell.find_element(By.XPATH, selector)
        self.logger.info(f"Clicking TP edit target: {selector}")
        ActionChains(self.driver).click(click_target).perform()
        clicked = True
        break
    except:
        continue

if not clicked:
    # Fallback: double-click the cell directly
    ActionChains(self.driver).double_click(to_make_cell).perform()

time.sleep(0.5) # Reduced from 1.5s

# Look for input field that appears after clicking
input_field = None

# First try to get the currently focused element (most reliable)
try:
    active_element = self.driver.switch_to.active_element
    if active_element and active_element.tag_name == 'input':
        current_value = active_element.get_attribute("value")
        if not (current_value and ("1000" in str(current_value) or len(str(current_value).replace(
            ".", "")) > 6)):
            input_field = active_element
except:
    pass

# If no focused input found, try our selectors
if not input_field:
    input_selectors = [
        # First try to find input within the clicked cell
        ".*//input[@type='text']",
        ".*//input[@type='number']",
        ".*//input[contains(@class, 'MuiInput')]",
        ".*//input",
        # Then try to find input in the modal/dialog that might appear
        ".*//div[contains(@class, 'MuiDialog') or contains(@class, 'modal')].*//input[@type='text']",
        ".*//div[contains(@class, 'MuiDialog') or contains(@class, 'modal')].*//input[@type='number']",
        # Try any visible input field (but be careful this might find wrong field)
        ".*//input[@type='text' and not(@disabled)]",
        ".*//input[@type='number' and not(@disabled)]"
    ]

```

```

        for selector in input_selectors:
            try:
                if selector.startswith("./"):
                    # Search within the To Make cell
                    input_field = to_make_cell.find_element(By.XPATH, selector)
                else:
                    # Search globally
                    input_field = WebDriverWait(self.driver, 3).until(
                        EC.element_to_be_clickable((By.XPATH, selector))
                    )
                self.logger.info(f"Found TP input field: {selector}")
                break
            except (TimeoutException, Exception):
                continue

        if input_field:
            # Clear and enter the TP value (FAST - edit only ONCE)
            input_field.click()
            input_field.send_keys(Keys.CONTROL + "a")
            input_field.send_keys(Keys.DELETE)
            input_field.send_keys(str(tp_dollars))
            input_field.send_keys(Keys.ENTER)
            time.sleep(0.3) # Reduced from 0.5s
            return True
        else:
            self.logger.error("Could not find input field for TP edit")
            return False

    except Exception as e:
        self.logger.error(f"Failed to edit position TP: {e}")
        return False

def _edit_position_sl(self, position_row, sl_dollars):
    """Edit Stop Loss by clicking the 'Risk' column (data-field='risk')"""
    try:
        # Find the 'Risk' cell using the exact HTML structure provided
        # Look for MUI DataGrid cell with data-field="risk"
        risk_cell_selectors = [
            # Primary selector: exact data-field match for "risk"
            ".*div[@role='cell' and @data-field='risk']",
            # Secondary: MUI DataGrid cell with risk field
            ".*div[contains(@class, 'MuiDataGrid-cell') and @data-field='risk']",
            # Fallback: cell containing the pencil icon and $ value in risk context
            ".*div[@role='cell' and contains(@data-field, 'risk')]",
            # Generic fallback: look for editable cell with $ and pencil icon (not toMake)
            ".*div[@role='cell' and contains(@class, 'editable') and ./span[contains(text(), '$')]]"
        ]

        risk_cell = None
        for i, selector in enumerate(risk_cell_selectors, 1):
            try:
                risk_cell = position_row.find_element(By.XPATH, selector)
                self.logger.info(f"✓ Found 'Risk' cell using selector {i}: {selector}")
                break
            except:
                self.logger.debug(f"Selector {i} not found: {selector}")
                continue

        if not risk_cell:
            self.logger.error("Could not find 'Risk' cell with any selector")

```

```

        return False

# Click the pencil icon or the cell itself to activate editing
pencil_icon_selectors = [
    # Click the pencil icon specifically
    ".*//svg[@data-icon='pencil']",
    # Click the span containing the $ value
    ".*//span[contains(@class, 'css-') and contains(text(), '$')]",
    # Click the cell itself if no pencil found
    "."
]

clicked = False
for selector in pencil_icon_selectors:
    try:
        click_target = risk_cell.find_element(By.XPATH, selector)
        self.logger.info(f"Clicking SL edit target: {selector}")
        ActionChains(self.driver).click(click_target).perform()
        clicked = True
        break
    except:
        continue

if not clicked:
    # Fallback: double-click the cell directly
    self.logger.info("Fallback: Double-clicking 'Risk' cell directly")
    ActionChains(self.driver).double_click(risk_cell).perform()

time.sleep(1.5)

# Look for input field that appears after clicking
# Priority: Find the currently focused/active input field first
input_field = None

# First try to get the currently focused element (most reliable)
try:
    active_element = self.driver.switch_to.active_element
    if active_element and active_element.tag_name == 'input':
        # Validate this is not a contract quantity field
        current_value = active_element.get_attribute("value")
        if not (current_value and ("1000" in str(current_value) or len(str(current_value).replace(".", "")) > 6)):
            input_field = active_element
            self.logger.info("✓ Using focused input field as SL target")
except Exception as e:
    self.logger.debug(f"Could not get active element: {e}")

# If no focused input found, try our selectors
if not input_field:
    input_selectors = [
        # First try to find input within the clicked cell
        ".*//input[@type='text']",
        ".*//input[@type='number']",
        ".*//input[contains(@class, 'MuiInput')]",
        ".*//input",
        # Then try to find input in the modal/dialog that might appear
        ".*//div[contains(@class, 'MuiDialog') or contains(@class, 'modal')]/input[@type='text']",
        ".*//div[contains(@class, 'MuiDialog') or contains(@class, 'modal')]/input[@type='number']",
        # Try any visible input field (but be careful this might find wrong field)
        ".*//input[@type='text' and not(@disabled)]",
    ]

```



```

        "//input[@type='number' and not(@disabled)]"
    ]

    for selector in input_selectors:
        try:
            if selector.startswith("./"):
                # Search within the Risk cell
                input_field = risk_cell.find_element(By.XPATH, selector)
            else:
                # Search globally
                input_field = WebDriverWait(self.driver, 3).until(
                    EC.element_to_be_clickable((By.XPATH, selector))
                )
            self.logger.info(f"Found SL input field: {selector}")
            break
        except (TimeoutException, Exception):
            continue

    if input_field:
        # Clear and enter the SL value (FAST - edit only ONCE)
        input_field.click()
        input_field.send_keys(Keys.CONTROL + "a")
        input_field.send_keys(Keys.DELETE)
        input_field.send_keys(str(sl_dollars))
        input_field.send_keys(Keys.ENTER)
        time.sleep(0.3) # Reduced from 0.5s
        return True
    else:
        self.logger.error("Could not find input field for SL edit")
        return False

except Exception as e:
    self.logger.error(f"Failed to edit position SL: {e}")
    return False

def setup_post_trade_tp_sl_positions(self, tp_dollars, sl_dollars):
    """
    Setup TP/SL in the positions section AFTER trade is placed
    OPTIMIZED: Edits each field exactly once with no retries for speed
    """
    try:
        self.logger.info(f"[TP/SL-SETUP] Starting - TP: ${tp_dollars}, SL: ${sl_dollars}")

        # CRITICAL: Switch to Positions tab first
        self.logger.info("[TP/SL-SETUP] Switching to Positions tab...")
        if not self.switch_to_positions_tab():
            self.logger.error("[TP/SL-SETUP] ■ Failed to switch to Positions tab")
            return False

        self.logger.info("[TP/SL-SETUP] ■ On Positions tab")

        # Wait for positions grid to load - Increased wait for reliability
        time.sleep(1.5) # Increased from 0.3s to 1.5s to ensure grid is fully interactive
        self.logger.info("[TP/SL-SETUP] Grid load wait completed")

        # Track if we've successfully edited each field
        sl_edited = False
        tp_edited = False

        # Edit Risk field (SL) - SINGLE EDIT, NO RETRIES

```

```

if sl_dollars and sl_dollars > 0:
    self.logger.info(f"[TP/SL-SETUP] Editing SL (Risk) field: ${sl_dollars}")
    sl_edited = self._edit_sl_field_fast(sl_dollars)
    if sl_edited:
        self.logger.info("[TP/SL-SETUP] ■ SL edited successfully")
    else:
        self.logger.error("[TP/SL-SETUP] ■ SL edit failed")

# Edit To Make field (TP) - SINGLE EDIT, NO RETRIES
if tp_dollars and tp_dollars > 0:
    self.logger.info(f"[TP/SL-SETUP] Editing TP (To Make) field: ${tp_dollars}")
    tp_edited = self._edit_tp_field_fast(tp_dollars)
    if tp_edited:
        self.logger.info("[TP/SL-SETUP] ■ TP edited successfully")
    else:
        self.logger.error("[TP/SL-SETUP] ■ TP edit failed")

# Determine overall success
if tp_dollars and sl_dollars:
    success = tp_edited and sl_edited
    self.logger.info(
        f"[TP/SL-SETUP] Final result - TP: {tp_edited}, SL: {sl_edited}, Overall: {success}")
elif tp_dollars:
    success = tp_edited
    self.logger.info(f"[TP/SL-SETUP] Final result - TP only: {tp_edited}")
elif sl_dollars:
    success = sl_edited
    self.logger.info(f"[TP/SL-SETUP] Final result - SL only: {sl_edited}")
else:
    success = True # Nothing to edit
    self.logger.info("[TP/SL-SETUP] No TP/SL values to edit")

return success

except Exception as e:
    self.logger.error(f"[TP/SL-SETUP] ■ Exception during TP/SL setup: {e}")
    import traceback
    self.logger.error(traceback.format_exc())
    return False

def _edit_sl_field_fast(self, sl_dollars):
    """FAST: Edit SL (Risk) field directly without searching for position row"""
    max_retries = 3
    for attempt in range(max_retries):
        try:
            self.logger.info(
                f"[FAST-SL] Starting SL edit (Attempt {attempt+1}/{max_retries}): ${sl_dollars}")

            # CRITICAL: We need to click on the actual dollar VALUE, not the pencil icon
            # The pencil is just an indicator - clicking the $ value activates editing
            value_selectors = [
                # Click the span with the dollar value inside the Risk cell
                "//div[@data-field='risk']//span[contains(text(), '$')]",
                # Or click anywhere in the Risk cell that's editable
                "//div[@data-field='risk' and contains(@class, 'MuiDataGrid-cell--editable')]",
                # Or just the Risk cell itself
                "//div[@data-field='risk' and @role='cell']",
            ]

            clicked = False

```

```

for i, selector in enumerate(value_selectors, 1):
    try:
        value_element = WebDriverWait(self.driver, 1.0).until(
            EC.element_to_be_clickable((By.XPATH, selector))
        )
        self.logger.info(f"[FAST-SL] Found Risk value element with selector {i}")

        # Double-click to activate editing (common pattern for editable grids)
        from selenium.webdriver.common.action_chains import ActionChains
        ActionChains(self.driver).double_click(value_element).perform()
        self.logger.info("[FAST-SL] Double-clicked Risk value")
        clicked = True
        break
    except Exception as e:
        self.logger.debug(f"[FAST-SL] Selector {i} failed: {e}")
        continue

if not clicked:
    self.logger.warning(f"[FAST-SL] Attempt {attempt+1}: Could not find or click Risk value")
    time.sleep(0.5)
    continue

time.sleep(0.5) # Wait for input to appear

# Use active element (most reliable)
input_field = self.driver.switch_to.active_element
self.logger.info(
    f"[FAST-SL] Active element after double-click: {input_field.tag_name if input_field.tag_name != 'html' else ''}"
)

if input_field and input_field.tag_name == 'input':
    self.logger.info(f"[FAST-SL] Found input field, typing ${sl_dollars}")
    # Clear field and type new value
    input_field.send_keys(Keys.CONTROL + "a")
    input_field.send_keys(Keys.DELETE)
    input_field.send_keys(str(sl_dollars))
    input_field.send_keys(Keys.ENTER)
    time.sleep(0.2) # Minimal wait
    self.logger.info("[FAST-SL] ■ SL edit completed")
    return True
else:
    self.logger.warning(
        f"[FAST-SL] Attempt {attempt+1}: Active element is not an input: {input_field.tag_name if input_field.tag_name != 'html' else ''}"
    )
    # Try to find input field manually
    try:
        input_field = WebDriverWait(self.driver, 1.0).until(
            EC.presence_of_element_located((By.XPATH, "//div[@data-field='risk']/input"))
        )
        self.logger.info("[FAST-SL] Found input field manually")
        input_field.send_keys(Keys.CONTROL + "a")
        input_field.send_keys(Keys.DELETE)
        input_field.send_keys(str(sl_dollars))
        input_field.send_keys(Keys.ENTER)
        time.sleep(0.2)
        self.logger.info("[FAST-SL] ■ SL edit completed (manual input find)")
        return True
    except Exception as e:
        self.logger.error(f"[FAST-SL] Could not find input manually: {e}")

# If we got here, something failed but didn't raise exception
time.sleep(0.5)

```

```

        except Exception as e:
            self.logger.error(f"[FAST-SL] ■ Attempt {attempt+1} Exception: {e}")
            time.sleep(0.5)

    self.logger.error(f"[FAST-SL] Failed to edit SL after {max_retries} attempts")
    return False

def _edit_tp_field_fast(self, tp_dollars):
    """FAST: Edit TP (To Make) field directly without searching for position row"""
    max_retries = 3
    for attempt in range(max_retries):
        try:
            self.logger.info(
                f"[FAST-TP] Starting TP edit (Attempt {attempt+1}/{max_retries}): ${tp_dollars}")

            # CRITICAL: We need to click on the actual dollar VALUE, not the pencil icon
            # The pencil is just an indicator - clicking the $ value activates editing
            value_selectors = [
                # Click the span with the dollar value inside the To Make cell
                "//div[@data-field='toMake']//span[contains(text(), '$')]",
                # Or click anywhere in the To Make cell that's editable
                "//div[@data-field='toMake' and contains(@class, 'MuiDataGrid-cell--editable')]",
                # Or just the To Make cell itself
                "//div[@data-field='toMake' and @role='cell']",
            ]

            clicked = False
            for i, selector in enumerate(value_selectors, 1):
                try:
                    value_element = WebDriverWait(self.driver, 1.0).until(
                        EC.element_to_be_clickable((By.XPATH, selector))
                    )
                    self.logger.info(f"[FAST-TP] Found To Make value element with selector {i}")

                    # Double-click to activate editing (common pattern for editable grids)
                    from selenium.webdriver.common.action_chains import ActionChains
                    ActionChains(self.driver).double_click(value_element).perform()
                    self.logger.info("[FAST-TP] Double-clicked To Make value")
                    clicked = True
                    break
                except Exception as e:
                    self.logger.debug(f"[FAST-TP] Selector {i} failed: {e}")
                    continue

            if not clicked:
                self.logger.warning(
                    f"[FAST-TP] Attempt {attempt+1}: Could not find or click To Make value")
                time.sleep(0.5)
                continue

            time.sleep(0.5) # Wait for input to appear

            # Use active element (most reliable)
            input_field = self.driver.switch_to.active_element
            self.logger.info(
                f"[FAST-TP] Active element after double-click: {input_field.tag_name if input_field}")

            if input_field and input_field.tag_name == 'input':
                self.logger.info(f"[FAST-TP] Found input field, typing ${tp_dollars}")
                # Clear field and type new value

```

```

        input_field.send_keys(Keys.CONTROL + "a")
        input_field.send_keys(Keys.DELETE)
        input_field.send_keys(str(tp_dollars))
        input_field.send_keys(Keys.ENTER)
        time.sleep(0.2) # Minimal wait
        self.logger.info("[FAST-TP] ■ TP edit completed")
        return True
    else:
        self.logger.warning(
            f"[FAST-TP] Attempt {attempt+1}: Active element is not an input: {input_field.tag}")
        # Try to find input field manually
        try:
            input_field = WebDriverWait(self.driver, 1.0).until(
                EC.presence_of_element_located((By.XPATH, "//div[@data-field='toMake']//input")
            )
            self.logger.info("[FAST-TP] Found input field manually")
            input_field.send_keys(Keys.CONTROL + "a")
            input_field.send_keys(Keys.DELETE)
            input_field.send_keys(str(tp_dollars))
            input_field.send_keys(Keys.ENTER)
            time.sleep(0.2)
            self.logger.info("[FAST-TP] ■ TP edit completed (manual input find)")
            return True
        except Exception as e:
            self.logger.error(f"[FAST-TP] Could not find input manually: {e}")

    # If we got here, something failed but didn't raise exception
    time.sleep(0.5)

except Exception as e:
    self.logger.error(f"[FAST-TP] ■ Attempt {attempt+1} Exception: {e}")
    time.sleep(0.5)

self.logger.error(f"[FAST-TP] Failed to edit TP after {max_retries} attempts")
return False

def setup_inline_tp_sl(self, tp_dollars, sl_dollars):
    """
    Set TP/SL using inline editable fields in the data grid (NEW METHOD)
    This replaces the old gear button modal approach
    OPTIMIZED: First checks if values are already correct before attempting edits
    """
    try:
        self.logger.info(f"■ Setting up inline TP/SL - SL: ${sl_dollars}, TP: ${tp_dollars}")

        # Set Risk (SL) field if sl_dollars > 0
        if sl_dollars > 0:
            # FIRST: Check if Risk field is already set to the correct value
            self.logger.info(f"[CHECK] Checking if Risk field is already set to ${sl_dollars}")
            if self._validate_risk_field_value(sl_dollars):
                self.logger.info(f"■ Risk field already correctly set to ${sl_dollars} - skipping edit")
            else:
                if not self._edit_risk_field(sl_dollars):
                    self.logger.error("Failed to set Risk (SL) field")
                    return False
        else:
            self.logger.info("Skipping Risk field (SL=0)")

        # Set To Make (TP) field if tp_dollars > 0

```

```

if tp_dollars > 0:
    # FIRST: Check if To Make field is already set to the correct value
    self.logger.info(f"[CHECK] Checking if To Make field is already set to ${tp_dollars}")
    if self._validate_to_make_field_value(tp_dollars):
        self.logger.info(
            f"■ To Make field already correctly set to ${tp_dollars} - skipping edit")
    else:
        if not self._edit_to_make_field(tp_dollars):
            self.logger.error("Failed to set To Make (TP) field")
            return False
else:
    self.logger.info("Skipping To Make field (TP=0)")

self.logger.info("■ Inline TP/SL setup completed successfully")
return True

except Exception as e:
    self.logger.error(f"Failed to setup inline TP/SL: {e}")
    return False

def _edit_risk_field(self, sl_dollars):
    """Edit the Risk field in the data grid to set SL"""
    try:
        self.logger.info(f"■ Editing Risk field to ${sl_dollars}")

        # Find the Risk field using the data attributes from the HTML
        risk_selectors = [
            "//div[@data-field='risk']//svg[contains(@class, 'fa-pencil')]",
            "//div[@role='cell'][@data-field='risk']//svg[@data-icon='pencil']",
            "//div[contains(@class, 'MuiDataGrid-cell') and @data-field='risk']//svg",
            ".MuiDataGrid-cell[data-field='risk'] svg[data-icon='pencil']"
        ]

        # Find and click the pencil icon to start editing
        pencil_icon = None
        for selector in risk_selectors:
            try:
                if selector.startswith("//"):
                    pencil_icon = self.driver.find_element(By.XPATH, selector)
                else:
                    pencil_icon = self.driver.find_element(By.CSS_SELECTOR, selector)

                if pencil_icon.is_displayed():
                    self.logger.info(f"Found Risk pencil icon with selector: {selector}")
                    break
            except:
                continue

        if not pencil_icon:
            self.logger.error("Could not find Risk field pencil icon")
            return False

        # Click the pencil icon to start editing
        pencil_icon.click()
        time.sleep(0.5)

        # Look for the input field that appears
        input_selectors = [
            "//div[@data-field='risk']//input",
            "//input[contains(@class, 'MuiInputBase-input')]",

```

```

        "//div[contains(@class, 'MuiDataGrid-cell') and @data-field='risk']//input"
    ]

    input_field = None
    for selector in input_selectors:
        try:
            input_field = WebDriverWait(self.driver, 3).until(
                EC.presence_of_element_located((By.XPATH, selector))
            )
            if input_field.is_displayed():
                break
        except:
            continue

    if not input_field:
        self.logger.error("Could not find Risk input field after clicking pencil")
        return False

    # Clear and enter the value
    input_field.click()
    time.sleep(0.2)
    input_field.send_keys(Keys.CONTROL + "a") # Select all
    time.sleep(0.2)
    input_field.send_keys(str(sl_dollars)) # Type new value
    time.sleep(0.3)
    input_field.send_keys(Keys.ENTER) # Confirm
    time.sleep(0.5)

    self.logger.info(f"■ Risk field set to ${sl_dollars}")
    return True

except Exception as e:
    self.logger.error(f"Failed to edit Risk field: {e}")
    return False

def _edit_to_make_field(self, tp_dollars):
    """Edit the To Make field in the data grid to set TP"""
    try:
        self.logger.info(f"■ Editing To Make field to ${tp_dollars}")

        # Find the To Make field using the data attributes from the HTML
        to_make_selectors = [
            "//div[@data-field='toMake']//svg[contains(@class, 'fa-pencil')]",
            "//div[@role='cell'][@data-field='toMake']//svg[@data-icon='pencil']",
            "//div[contains(@class, 'MuiDataGrid-cell') and @data-field='toMake']//svg",
            ".MuiDataGrid-cell[data-field='toMake'] svg[data-icon='pencil']"
        ]

        # Find and click the pencil icon to start editing
        pencil_icon = None
        for selector in to_make_selectors:
            try:
                if selector.startswith("//"):
                    pencil_icon = self.driver.find_element(By.XPATH, selector)
                else:
                    pencil_icon = self.driver.find_element(By.CSS_SELECTOR, selector)

                if pencil_icon.is_displayed():
                    self.logger.info(f"Found To Make pencil icon with selector: {selector}")
                    break
            except:
                continue
    
```

```

        except:
            continue

    if not pencil_icon:
        self.logger.error("Could not find To Make field pencil icon")
        return False

    # Click the pencil icon to start editing
    pencil_icon.click()
    time.sleep(0.5)

    # Look for the input field that appears
    input_selectors = [
        "//div[@data-field='toMake']//input",
        "//input[contains(@class, 'MuiInputBase-input')]",
        "//div[contains(@class, 'MuiDataGrid-cell') and @data-field='toMake']//input"
    ]

    input_field = None
    for selector in input_selectors:
        try:
            input_field = WebDriverWait(self.driver, 3).until(
                EC.presence_of_element_located((By.XPATH, selector))
            )
            if input_field.is_displayed():
                break
        except:
            continue

    if not input_field:
        self.logger.error("Could not find To Make input field after clicking pencil")
        return False

    # Clear and enter the value
    input_field.click()
    time.sleep(0.2)
    input_field.send_keys(Keys.CONTROL + "a") # Select all
    time.sleep(0.2)
    input_field.send_keys(str(tp_dollars)) # Type new value
    time.sleep(0.3)
    input_field.send_keys(Keys.ENTER) # Confirm
    time.sleep(0.5)

    self.logger.info(f"■ To Make field set to ${tp_dollars}")
    return True

except Exception as e:
    self.logger.error(f"Failed to edit To Make field: {e}")
    return False

def _edit_positions_risk_field(self, sl_dollars):
    """
    Edit the Risk field (SL) in the positions section by clicking the cell to make it editable
    """
    try:
        self.logger.info(f"■ Clicking Risk cell to make it editable for SL=${sl_dollars}")

        # First ensure we find the positions table to avoid targeting order form
        positions_table = None
        try:

```



```

        positions_table = self.driver.find_element(By.XPATH,
            "//div[contains(@class, 'MuiDataGrid-root')]")
        self.logger.info("[CONTEXT] Found positions DataGrid table")
    except:
        self.logger.warning("Could not find positions table context")

# Find and double-click the Risk cell directly - matching exact HTML structure
risk_cell_selectors = [
    "//div[@class='MuiDataGrid-cell--withRenderer MuiDataGrid-cell MuiDataGrid-cell--textCent",
    "//td[@data-field='risk']", # Simple Risk cell
    "//div[@data-field='risk']", # Risk div
    "//td[contains(@class, 'MuiDataGrid-cell--editable') and @data-field='risk']", # Editabl
    "//div[contains(@class, 'MuiDataGrid-cell--editable') and @data-field='risk']", # Editab
]

risk_cell = None
for selector in risk_cell_selectors:
    try:
        risk_cell = self.driver.find_element(By.XPATH, selector)
        if risk_cell.is_displayed():
            break
    except:
        continue

if not risk_cell:
    self.logger.error("Could not find Risk cell in positions section")
    return False

# Double-click the cell to make it editable and enter value directly
self.logger.info(f"[DOUBLE-CLICK] Double-clicking Risk cell to edit value")

# Use ActionChains for reliable double-click
from selenium.webdriver.common.action_chains import ActionChains
actions = ActionChains(self.driver)
actions.double_click(risk_cell).perform()
time.sleep(0.15) # Reduced: Wait for cell to become editable

# Type the value directly (cell should now be in edit mode)
self.logger.info(f"[TYPE] Typing ${sl_dollars} into Risk field")

# SAFEGUARD: Don't proceed if sl_dollars is 0 or invalid
if not sl_dollars or sl_dollars <= 0:
    self.logger.error(f"■ BLOCKED: Will not set Risk field to invalid value: {sl_dollars}")
    return False

actions.send_keys(Keys.CONTROL + "a").perform() # Select all
time.sleep(0.05) # Reduced: Brief pause after select all

# SAFEGUARD: Double-check we're about to type a valid value
value_to_type = str(sl_dollars)
if not value_to_type or value_to_type in ['0', '0.0', '']:
    self.logger.error(f"■ BLOCKED: Will not type invalid value: '{value_to_type}'")
    return False

actions.send_keys(value_to_type).perform() # Type new value
actions.send_keys(Keys.ENTER).perform() # Confirm
time.sleep(0.1) # Reduced: Brief pause after confirmation

self.logger.info(f"Risk field set to ${sl_dollars}")
return True

```

```

except Exception as e:
    self.logger.error(f"Failed to edit positions Risk field: {e}")
    return False

def _edit_positions_to_make_field(self, tp_dollars):
    """
    Edit the To Make field (TP) in the positions section by clicking the cell to make it editable
    """
    try:
        self.logger.info(f"■ Clicking To Make cell to make it editable for TP=${tp_dollars}")

        # First ensure we find the positions table to avoid targeting order form
        positions_table = None
        try:
            positions_table = self.driver.find_element(By.XPATH,
                "//div[contains(@class, 'MuiDataGrid-root')]")
            self.logger.info("[CONTEXT] Found positions DataGrid table")
        except:
            self.logger.warning("Could not find positions table context")

        # Find and double-click the To Make cell directly - matching exact HTML structure
        to_make_cell_selectors = [
            "//div[@class='MuiDataGrid-cell--withRenderer MuiDataGrid-cell MuiDataGrid-cell--textCent",
            "//td[@data-field='toMake']", # Simple To Make cell
            "//div[@data-field='toMake']", # To Make div
            "//td[contains(@class, 'MuiDataGrid-cell--editable') and @data-field='toMake']", # Editable
            "//div[contains(@class, 'MuiDataGrid-cell--editable') and @data-field='toMake']", # Editable
        ]

        to_make_cell = None
        for selector in to_make_cell_selectors:
            try:
                to_make_cell = self.driver.find_element(By.XPATH, selector)
                if to_make_cell.is_displayed():
                    break
            except:
                continue

        if not to_make_cell:
            self.logger.error("Could not find To Make cell in positions section")
            return False

        # Double-click the cell to make it editable and enter value directly
        self.logger.info(f"[DOUBLE-CLICK] Double-clicking To Make cell to edit value")

        # Use ActionChains for reliable double-click
        from selenium.webdriver.common.action_chains import ActionChains
        actions = ActionChains(self.driver)
        actions.double_click(to_make_cell).perform()
        time.sleep(0.15) # Reduced: Wait for cell to become editable

        # Type the value directly (cell should now be in edit mode)
        self.logger.info(f"[TYPE] Typing ${tp_dollars} into To Make field")

        # SAFEGUARD: Don't proceed if tp_dollars is 0 or invalid
        if not tp_dollars or tp_dollars <= 0:
            self.logger.error(f"■ BLOCKED: Will not set To Make field to invalid value: {tp_dollars}")
            return False

        actions.send_keys(Keys.CONTROL + "a").perform() # Select all

```

```

time.sleep(0.05) # Reduced: Brief pause after select all

# SAFEGUARD: Double-check we're about to type a valid value
value_to_type = str(tp_dollars)
if not value_to_type or value_to_type in ['0', '0.0', '']:
    self.logger.error(f"■ BLOCKED: Will not type invalid value: '{value_to_type}'")
    return False

actions.send_keys(value_to_type).perform() # Type new value
actions.send_keys(Keys.ENTER).perform() # Confirm
time.sleep(0.1) # Reduced: Brief pause after confirmation

self.logger.info(f"To Make field set to ${tp_dollars}")
return True

except Exception as e:
    self.logger.error(f"Failed to edit positions To Make field: {e}")
    return False

def _validate_risk_field_value(self, expected_sl_dollars):
    """
    Validate that the Risk field was set to the correct SL value
    Returns True if the value matches, False otherwise
    Uses the same selectors as the editing function for consistency
    """
    try:
        self.logger.info(f"■ Validating Risk field value (expected: ${expected_sl_dollars})")

        # Use the same selectors as _edit_position_sl for consistency
        risk_cell_selectors = [
            # Primary selector: exact data-field match for "risk"
            ".*//div[@role='cell' and @data-field='risk']",
            # Secondary: MUI DataGrid cell with risk field
            ".*//div[contains(@class, 'MuiDataGrid-cell') and @data-field='risk']",
            # Fallback: cell containing the pencil icon and $ value in risk context
            ".*//div[@role='cell' and contains(@data-field, 'risk')]",
        ]

        current_value = None
        for selector in risk_cell_selectors:
            try:
                # Search from the body element to ensure we find the right cell
                body_element = self.driver.find_element(By.TAG_NAME, "body")
                risk_cell = body_element.find_element(By.XPATH, selector)
                text_content = risk_cell.text.strip()

                # Look for dollar amounts in the text
                if '$' in text_content or text_content.replace('.', '').replace(',', '').isdigit():
                    current_value = text_content
                    self.logger.debug(
                        f"Found Risk field text: '{text_content}' using selector: {selector}")
                    break
            except:
                continue

        if current_value:
            # Extract numeric value from the text (remove $ and other chars)
            import re
            # More robust regex to handle various formats: $1,510, $1510, 1510, $1,510.00, etc.
            numeric_match = re.search(r'[\d,]+(?:\.\d+)?', current_value.replace('$', ''))

```

```

        if numeric_match:
            # Remove commas and convert to float
            clean_value = numeric_match.group().replace(',', '')
            current_numeric = float(clean_value)
            expected_numeric = float(expected_sl_dollars)

            # Check if values match (with small tolerance for float comparison)
            if abs(current_numeric - expected_numeric) < 0.01:
                self.logger.info(
                    f"■ Risk field validation passed: '{current_value}' matches expected ${expected_sl_dollars}"
                )
                return True
            else:
                self.logger.warning(
                    f"■ Risk field validation failed: '{current_value}' != ${expected_sl_dollars}"
                )
                return False
        else:
            self.logger.warning(f"Could not extract numeric value from Risk field: '{current_value}'")
            return False
    else:
        self.logger.warning("Could not find Risk field value for validation")
        return False

except Exception as e:
    self.logger.error(f"Error validating Risk field: {e}")
    return False

def _validate_to_make_field_value(self, expected_tp_dollars):
    """
    Validate that the To Make field was set to the correct TP value
    Returns True if the value matches, False otherwise
    Uses the same selectors as the editing function for consistency
    """
    try:
        self.logger.info(f"■ Validating To Make field value (expected: ${expected_tp_dollars})")

        # Use the same selectors as _edit_position_tp for consistency
        to_make_cell_selectors = [
            # Primary selector: exact data-field match for "toMake"
            ".*div[@role='cell' and @data-field='toMake']",
            # Secondary: MUI DataGrid cell with toMake field
            ".*div[contains(@class, 'MuiDataGrid-cell') and @data-field='toMake']",
            # Fallback: cell containing the pencil icon and $ value in toMake context
            ".*div[@role='cell' and contains(@data-field, 'toMake')]",
        ]

        current_value = None
        for selector in to_make_cell_selectors:
            try:
                # Search from the body element to ensure we find the right cell
                body_element = self.driver.find_element(By.TAG_NAME, "body")
                to_make_cell = body_element.find_element(By.XPATH, selector)
                text_content = to_make_cell.text.strip()

                # Look for dollar amounts in the text
                if '$' in text_content or text_content.replace('.', '').replace(',', '').isdigit():
                    current_value = text_content
                    self.logger.debug(
                        f"Found To Make field text: '{text_content}' using selector: {selector}"
                    )
                    break
            except:
                pass
    except:
        pass

```

```

        continue

    if current_value:
        # Extract numeric value from the text (remove $ and other chars)
        import re
        # More robust regex to handle various formats: $1,510, $1510, 1510, $1,510.00, etc.
        numeric_match = re.search(r'[\d,]+(?:\.\d+)?', current_value.replace('$', ''))
        if numeric_match:
            # Remove commas and convert to float
            clean_value = numeric_match.group().replace(',', '')
            current_numeric = float(clean_value)
            expected_numeric = float(expected_tp_dollars)

            # Check if values match (with small tolerance for float comparison)
            if abs(current_numeric - expected_numeric) < 0.01:
                self.logger.info(
                    f"■ To Make field validation passed: '{current_value}' matches expected ${expected_tp_dollars}"
                )
                return True
            else:
                self.logger.warning(
                    f"■ To Make field validation failed: '{current_value}' != ${expected_tp_dollars}"
                )
                return False
        else:
            self.logger.warning(
                f"Could not extract numeric value from To Make field: '{current_value}'"
            )
            return False
    else:
        self.logger.warning("Could not find To Make field value for validation")
        return False

except Exception as e:
    self.logger.error(f"Error validating To Make field: {e}")
    return False

def _ensure_on_trading_page(self):
    """Ensure we're on the main trading page and on the Trading tab"""
    try:
        self._dismiss_backdrop()
        # Check if we're already on the trading page
        current_url = self.driver.current_url
        if "topstepx.com" in current_url and "/login" not in current_url:
            # If we're already on /trade, be more patient waiting for the Buy button
            if "/trade" in current_url:
                self.logger.info("Already on trading page, waiting for interface to load...")
                wait = WebDriverWait(self.driver, 20) # Wait longer if already on /trade
                try:
                    wait.until(EC.presence_of_element_located((By.XPATH,
                                                                "//button[contains(text(), 'Buy') or contains(text(), 'BUY')]")))

                    # Also ensure we're on the Trading tab
                    self.logger.info("Ensuring we're on the Trading tab...")
                    if not self._navigate_to_trading_tab():
                        self.logger.warning("Could not navigate to Trading tab, but Buy button found!")

                return True
            except TimeoutException:
                self.logger.warning("Buy button not found on /trade page after 20 seconds")
                pass
        else:
            # Not on /trade, check if Buy button exists on current page

```

```

        wait = WebDriverWait(self.driver, 5) # Shorter wait for non-trading pages
        try:
            wait.until(EC.presence_of_element_located((By.XPATH,
                "//button[contains(text(), 'Buy') or contains(text(), 'BUY')]")))
            return True
        except TimeoutException:
            pass

    # Only navigate if we're not already on the trading page
    if "/trade" not in current_url:
        trading_url = f"{self.base_url}/trade"
        self.logger.info(f"Navigating to trading page: {trading_url}")
        self.driver.get(trading_url)
        time.sleep(3)

        # Wait for trading interface to load
        wait = WebDriverWait(self.driver, 15)
        wait.until(EC.presence_of_element_located((By.XPATH,
            "//button[contains(text(), 'Buy') or contains(text(), 'BUY')]")))
    else:
        self.logger.warning(
            "Already on /trade but Buy button not found - page may not be fully loaded")
        return False

    # Ensure we're on the Trading tab
    self.logger.info("Ensuring we're on the Trading tab...")
    if not self._navigate_to_trading_tab():
        self.logger.warning("Could not navigate to Trading tab after navigating to trading page")

    return True

except Exception as e:
    self.logger.error(f"Failed to navigate to trading page: {e}")
    return False

@retry_on_stale_element(max_retries=2, delay=0.5)
def _set_contract_symbol(self, symbol):
    """
    ULTRA-FAST: Direct symbol entry with minimal waits
    """
    try:
        start_time = time.time()

        # Find the contract symbol field (no logging for speed)
        # Try multiple selectors for the symbol field
        symbol_selectors = [
            "//input[contains(@class, 'MuiAutocomplete-input') and @role='combobox']",
            "//input[contains(@class, 'MuiAutocomplete-input')]",
            "//input[@placeholder='Symbol']",
            "//input[@placeholder='Search Symbol']",
            "//input[@aria-label='Symbol']",
            "//input[@type='text' and contains(@class, 'MuiInputBase-input')]"
        ]

        symbol_field = None
        for selector in symbol_selectors:
            try:
                symbol_field = WebDriverWait(self.driver, 1).until(
                    EC.presence_of_element_located((By.XPATH, selector)))
                if symbol_field:

```

```

        break
    except:
        continue

if not symbol_field:
    raise Exception("Could not find symbol input field with any selector")

# SPEED: short-circuit if the field already shows the requested symbol
try:
    existing_value = (symbol_field.get_attribute('value') or '').strip().upper()
except Exception:
    existing_value = ''
if existing_value and existing_value == symbol.upper():
    elapsed_ms = (time.time() - start_time) * 1000
    self.logger.info(f"■ Symbol already set: {existing_value} ({elapsed_ms:.0f}ms, no-op)")
    return True

# Dismiss any overlays, then JS-focus the field (bypasses click interception)
self._dismiss_backdrop()
self.driver.execute_script("arguments[0].focus(); arguments[0].click();", symbol_field)
symbol_field.send_keys(Keys.CONTROL + "a")
symbol_field.send_keys(Keys.DELETE)

# JS clear to ensure it's 100% empty
self.driver.execute_script("""
    var field = arguments[0];
    field.value = '';
    field.dispatchEvent(new Event('input', { bubbles: true }));
    field.dispatchEvent(new Event('change', { bubbles: true }));
""", symbol_field)

# Type first letter to trigger dropdown
first_char = symbol[0] if symbol else 'N'
symbol_field.send_keys(first_char)
time.sleep(0.15) # Reduced wait

# FAST DROPDOWN: Reduced timeout and no debug logging
dropdown_clicked = False
try:
    dropdown_options = WebDriverWait(self.driver, 2).until(
        EC.presence_of_all_elements_located((By.CSS_SELECTOR, "li.MuiAutocomplete-option"))
    )

    # Fast loop - find and click match immediately (no logging)
    for option in dropdown_options:
        try:
            spans = option.find_elements(By.TAG_NAME, "span")
            if len(spans) < 2:
                continue

            opt_symbol = spans[0].text.strip().upper()
            opt_desc = spans[1].text.strip().upper()

            # Match logic (streamlined)
            matched = False
            if symbol.upper() in ['NQM6', 'NQM25', 'NQM26']:
                matched = (
                    opt_symbol == 'NQM25' or opt_symbol == 'NQM26') and 'MICRO' not in opt_desc
            elif symbol.upper() in ['MNQM6', 'MNQM25', 'MNQM26']:
                matched = (

```

```

        opt_symbol == 'MNQM25' or opt_symbol == 'MNQM26') and 'MICRO' in opt_des
    elif symbol.upper() in ['NQH6', 'NQH25', 'NQH26']:
        matched = (
            opt_symbol == 'NQH25' or opt_symbol == 'NQH26') and 'MICRO' not in opt_de
    elif symbol.upper() in ['MNQH6', 'MNQH25', 'MNQH26']:
        matched = (
            opt_symbol == 'MNQH25' or opt_symbol == 'MNQH26') and 'MICRO' in opt_des
    elif opt_symbol == symbol.upper():
        matched = True

    if matched:
        # Fast click - JS only
        self.driver.execute_script("arguments[0].click();", option)
        dropdown_clicked = True
        time.sleep(0.1) # Minimal wait
        break
    except:
        continue

except:
    pass # Silent fail, will use fallback

# FAST VERIFY: Quick check and fallback if needed
time.sleep(0.15) # Reduced wait

# Re-find the field for verification using the same robust selectors
found_field = None
for selector in symbol_selectors:
    try:
        found_field = self.driver.find_element(By.XPATH, selector)
        if found_field and found_field.is_displayed():
            symbol_field = found_field
            break
    except:
        continue

final_value = symbol_field.get_attribute('value') or '' if symbol_field else ''

# Fast fallback: If dropdown didn't work, type directly
if not dropdown_clicked or final_value.strip().upper() != symbol.upper():
    if symbol_field:
        # Clear and type full symbol (JS click to bypass overlays)
        self.driver.execute_script("arguments[0].focus(); arguments[0].click();", symbol_fie
        symbol_field.send_keys(Keys.CONTROL + "a")
        symbol_field.send_keys(Keys.DELETE)
        symbol_field.send_keys(symbol.upper())
        symbol_field.send_keys(Keys.ENTER)
        time.sleep(0.15) # Reduced wait
        final_value = symbol_field.get_attribute('value') or ''
    else:
        self.logger.warning("Could not find symbol field for fallback entry")

# Final verification (streamlined)
success = False
if final_value:
    if symbol.upper() in ['NQM6', 'NQM25', 'NQM26']:
        success = 'NQM25' in final_value.upper() or 'NQM26' in final_value.upper()
    elif symbol.upper() in ['MNQM6', 'MNQM25', 'MNQM26']:
        success = 'MNQM25' in final_value.upper() or 'MNQM26' in final_value.upper()
    elif symbol.upper() in ['NQH6', 'NQH25', 'NQH26']:

```



```

        success = 'NQH25' in final_value.upper() or 'NQH26' in final_value.upper()
    elif symbol.upper() in ['MNQH6', 'MNQH25', 'MNQH26']:
        success = 'MNQH25' in final_value.upper() or 'MNQH26' in final_value.upper()
    else:
        success = symbol.upper() in final_value.upper()

    elapsed_ms = (time.time() - start_time) * 1000

    if success:
        self.logger.info(f"■ Symbol set: {final_value} ({elapsed_ms:.0f}ms)")
        return True
    else:
        self.logger.error(f"■ Symbol failed: expected '{symbol}', got '{final_value}'")
        return False

except Exception as e:
    self.logger.error(f"■ Symbol selection error: {e}")
    return False

def _get_current_contract_symbol(self):
    """Get the currently selected contract symbol from the field"""
    try:
        # Find the contract symbol input field
        symbol_selectors = [
            "input[placeholder*='Search']",
            "input[aria-label*='symbol']",
            "input[aria-label*='contract']",
            ".MuiAutocomplete-input",
            "input[data-testid*='symbol']"
        ]

        for selector in symbol_selectors:
            try:
                elements = self.driver.find_elements(By.CSS_SELECTOR, selector)
                for element in elements:
                    if element.is_displayed():
                        current_value = element.get_attribute('value')
                        if current_value and ('NQ' in current_value or 'MNQ' in current_value):
                            self.logger.info(f"■ Found current symbol: {current_value}")
                            return current_value
            except:
                continue

        self.logger.warning("■■ Could not find current symbol in any field")
        return None

    except Exception as e:
        self.logger.error(f"■ Error getting current symbol: {e}")
        return None

@retry_on_stale_element(max_retries=3, delay=1)
def _set_quantity(self, quantity):
    """FAST: Set quantity field with minimal waits"""
    try:
        # Fast selector - most specific first
        quantity_field = WebDriverWait(self.driver, 2).until(
            EC.presence_of_element_located((By.XPATH, "//input[@type='number'][@min='1']"))
        )

        # JS focus+click to bypass any overlay, then keyboard interaction

```

```

        self.driver.execute_script("arguments[0].focus(); arguments[0].click();", quantity_field)
        quantity_field.send_keys(Keys.CONTROL + "a")
        quantity_field.send_keys(str(quantity))
        quantity_field.send_keys(Keys.TAB)
        time.sleep(0.05) # Minimal wait

    return True

except Exception as e:
    self.logger.error(f"■ Qty failed: {e}")
    return False

def _set_order_type(self, order_type):
    """Set the order type (Market, Limit, etc.)"""
    try:
        wait = WebDriverWait(self.driver, 10)

        # Find order type dropdown
        order_type_selectors = [
            "//div[contains(@aria-label, 'Order Type')]",
            "//select[contains(@id, 'orderType')]",
            "//div[contains(text(), 'Market')]" # Current order type display
        ]

        order_type_field = None
        for selector in order_type_selectors:
            try:
                order_type_field = wait.until(EC.element_to_be_clickable((By.XPATH, selector)))
                break
            except TimeoutException:
                continue

        if not order_type_field:
            self.logger.warning("Could not find order type field")
            return False

        # Click on the dropdown
        order_type_field.click()
        time.sleep(1)

        # Select the desired order type
        option_xpath = f"//li[contains(text(), '{order_type.title()}')]"
        try:
            option = self.driver.find_element(By.XPATH, option_xpath)
            option.click()
            time.sleep(0.5)

            self.logger.info(f"Order type set to: {order_type}")
            return True
        except:
            self.logger.warning(f"Could not find order type option: {order_type}")
            return False

    except Exception as e:
        self.logger.error(f"Failed to set order type: {e}")
        return False

@retry_on_stale_element(max_retries=3, delay=1)
def _click_buy_button(self, skip_post_trade_setup=False):
    """ULTRA-FAST: Click BUY immediately"""

```

```

try:
    # Fast find and click
    buy_button = WebDriverWait(self.driver, 2).until(
        EC.element_to_be_clickable(By.XPATH, "//button[contains(text(), 'Buy')]"))
    )
    self.driver.execute_script("arguments[0].click();", buy_button)

    # In fast mode, minimal wait
    if skip_post_trade_setup:
        time.sleep(0.3)
        return True

    # Normal mode: handle confirmation
    time.sleep(0.5)
    self._handle_order_confirmation()
    return True

except Exception as e:
    self.logger.error(f"■ BUY failed: {e}")
    return False

@retry_on_stale_element(max_retries=3, delay=1)
def _click_sell_button(self, skip_post_trade_setup=False):
    """ULTRA-FAST: Click SELL immediately"""
    try:
        # Fast find and click
        sell_button = WebDriverWait(self.driver, 2).until(
            EC.element_to_be_clickable(By.XPATH, "//button[contains(text(), 'Sell')]"))
        )
        self.driver.execute_script("arguments[0].click();", sell_button)

        # In fast mode, minimal wait
        if skip_post_trade_setup:
            time.sleep(0.3)
            return True

        # Normal mode: handle confirmation
        time.sleep(0.5)
        self._handle_order_confirmation()
        return True

    except Exception as e:
        self.logger.error(f"■ SELL failed: {e}")
        return False
        if not self._click_order_tab():
            self.logger.warning("Failed to click Order tab after SELL button")

        # Wait for any confirmation dialogs and handle them
        time.sleep(1)
        self._handle_order_confirmation()

        return True

    except Exception as e:
        self.logger.error(f"Failed to click SELL button: {e}")
        return False

def _click_flatten_all_button(self):
    """Click the Flatten All button to close all positions"""
    try:

```

```

wait = WebDriverWait(self.driver, 10)

# Find Flatten All button - based on HTML analysis
flatten_selectors = [
    "//button[contains(text(), 'Flatten All')]",
    "//button[contains(text(), 'Close All')]",
    "//button[contains(text(), 'FLATTEN ALL')]"
]

flatten_button = None
for selector in flatten_selectors:
    try:
        flatten_button = wait.until(EC.element_to_be_clickable((By.XPATH, selector)))
        break
    except TimeoutException:
        continue

if not flatten_button:
    self.logger.warning("Could not find Flatten All button")
    return False

# Click the Flatten All button
flatten_button.click()
self.logger.info("Flatten All button clicked")

# Handle any confirmation dialogs
time.sleep(1)
self._handle_order_confirmation()

return True

except Exception as e:
    self.logger.error(f"Failed to click Flatten All button: {e}")
    return False

def _close_individual_positions(self):
    """Try to close individual positions if Flatten All is not available"""
    try:
        # Look for individual position close buttons
        close_buttons = self.driver.find_elements(By.XPATH,
            "//button[contains(text(), 'Close Position')]")

        if not close_buttons:
            self.logger.warning("No individual position close buttons found")
            return False

        for button in close_buttons:
            if button.is_enabled():
                button.click()
                time.sleep(1)
                self._handle_order_confirmation()

        self.logger.info("Individual positions closed")
        return True

    except Exception as e:
        self.logger.error(f"Failed to close individual positions: {e}")
        return False

def _handle_order_confirmation(self):

```

```

"""Handle any order confirmation dialogs that may appear"""
try:
    # Look for confirmation dialogs and click confirm if present
    confirmation_selectors = [
        "//button[contains(text(), 'Confirm')]",
        "//button[contains(text(), 'YES')]",
        "//button[contains(text(), 'OK')]",
        "//button[contains(text(), 'Submit')]"
    ]

    for selector in confirmation_selectors:
        try:
            confirm_button = WebDriverWait(self.driver, 2).until(
                EC.element_to_be_clickable((By.XPATH, selector))
            )
            confirm_button.click()
            self.logger.info("Order confirmation handled")
            time.sleep(0.5)
            break
        except TimeoutException:
            continue

except Exception as e:
    # No confirmation dialog is fine
    pass

def disconnect(self):
    """Disconnect from TopStepX and clean up"""
    try:
        if self.driver:
            self.driver.quit()
            self.driver = None

        self.logged_in = False
        self.logger.info("Disconnected from TopStepX")

    except Exception as e:
        self.logger.error(f"Error during TopStepX disconnect: {e}")

def close(self):
    """
    Close the TopStepX connection and quit Chrome driver
    Alias for disconnect() to match Tradovate interface
    Ensures Chrome browser always closes on disconnect
    """
    self.disconnect()

def prepare_field_for_input(self, field_element, field_name="Field"):
    """
    Prepare field for new input by selecting all existing content
    Uses highlight-and-type-over approach instead of deletion
    """
    try:
        self.logger.info(f"[PREPARE] Preparing {field_name} for new input")

        # Select all existing content (no deletion needed)
        field_element.send_keys(Keys.CONTROL + "a")
        time.sleep(0.2) # Allow selection to complete

        # Verify field is ready for input

```

```

        current_value = field_element.get_attribute("value") or ""
        self.logger.info(f"[PREPARE] {field_name} current value: '{current_value}' (will be replaced)

    return True

except Exception as e:
    self.logger.error(f"[CLEAR] Error clearing {field_name}: {e}")
    return False

def _set_field_with_clearing(self, selectors, value, field_name, input_type="text"):
    """
    OPTIMIZED: Generic method to set any field with fast clearing
    FAST-FAIL: Reduced timeout to 0.5s for quick failure on non-existent fields
    """
    try:
        # CRITICAL: Reduced from 3s to 0.5s for fast-fail on missing fields
        wait = WebDriverWait(self.driver, 0.5)

        # Find the field using provided selectors
        field_element = None
        for selector in selectors:
            try:
                field_element = wait.until(EC.element_to_be_clickable((By.XPATH, selector)))
                break
            except TimeoutException:
                continue

        if not field_element:
            self.logger.debug(f"Could not find {field_name} field")
            return False

        # OPTIMIZED: Fast JavaScript clear and set (no delays)
        self.driver.execute_script("""
            var field = arguments[0];
            var value = arguments[1];
            field.focus();
            field.value = value;
            field.dispatchEvent(new Event('input', { bubbles: true }));
            field.dispatchEvent(new Event('change', { bubbles: true }));
            """, field_element, str(value))

        self.logger.info(f"■ {field_name}: {value}")
        return True

    except Exception as e:
        self.logger.debug(f"Skip {field_name}: {e}")
        return False

def _set_stop_loss_price(self, price):
    """Set stop loss price if such field exists"""
    stop_loss_selectors = [
        "input[contains(@placeholder, 'Stop Loss') or contains(@aria-label, 'Stop Loss')]",
        "input[contains(@id, 'stopLoss') or contains(@id, 'stop-loss')]",
        "input[contains(@class, 'stop-loss') or contains(@class, 'stopLoss')]",
        "input[@type='number'][contains(../label/text(), 'Stop')]"
    ]
    return self._set_field_with_clearing(stop_loss_selectors, price, "Stop Loss Price", "number")

def _set_take_profit_price(self, price):
    """Set take profit price if such field exists"""

```

```

take_profit_selectors = [
    "//input[contains(@placeholder, 'Take Profit') or contains(@aria-label, 'Take Profit')]",
    "//input[contains(@id, 'takeProfit') or contains(@id, 'take-profit')]",
    "//input[contains(@class, 'take-profit') or contains(@class, 'takeProfit')]",
    "//input[@type='number'][contains(../label/text(), 'Target')]"
]
return self._set_field_with_clearing(take_profit_selectors, price, "Take Profit Price", "number")

def _set_limit_price(self, price):
    """Set limit price if such field exists"""
    limit_price_selectors = [
        "//input[contains(@placeholder, 'Limit Price') or contains(@aria-label, 'Limit Price')]",
        "//input[contains(@id, 'limitPrice') or contains(@id, 'limit-price')]",
        "//input[contains(@class, 'limit-price') or contains(@class, 'limitPrice')]",
        "//input[@type='number'][contains(../label/text(), 'Limit')]"
    ]
    return self._set_field_with_clearing(limit_price_selectors, price, "Limit Price", "number")

def clear_all_visible_inputs(self):
    """
    Optimized method to quickly clear main trading input fields only
    Focuses on contract and quantity fields for speed
    """
    try:
        # Only clear the essential trading fields for speed
        essential_selectors = [
            "//input[contains(@class, 'MuiAutocomplete-input')]", # Contract field
            "//input[@type='number'][@min='1']", # Quantity field
        ]

        cleared_count = 0
        for selector in essential_selectors:
            try:
                input_elem = self.driver.find_element(By.XPATH, selector)
                if input_elem.is_displayed() and input_elem.is_enabled():
                    field_value = input_elem.get_attribute("value") or ""

                    if field_value.strip(): # Only clear if it has content
                        # Fast JavaScript clear - no delays
                        self.driver.execute_script("""
                            arguments[0].focus();
                            arguments[0].select();
                            arguments[0].value = '';
                            arguments[0].dispatchEvent(new Event('input', { bubbles: true }));
                        """, input_elem)
                        cleared_count += 1

            except Exception:
                continue # Skip if field not found

        return cleared_count > 0

    except Exception as e:
        self.logger.debug(f"Field clearing skipped: {e}")
        return False

def __del__(self):
    """Cleanup when object is destroyed"""
    self.disconnect()

```

Method: TopStepXAccount.__init__

```
def __init__(self, username=None, password=None, pair_id=None)
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.
- **__init__** — The constructor. Runs after Python has allocated the instance; assigns starting attribute values to self.

Step by step (top-level body)

1. Assigns self.username = username or "".
2. Assigns self.password = password or "".
3. Assigns self.pair_id = pair_id or 'default'.
4. Assigns self.logged_in = False.
5. Assigns self.driver = None.
6. Assigns self.first_trade_attempted = False.
7. Assigns self._first_stats_fetch = True.
8. Assigns self._placing_order = False.
9. Assigns self._login_timestamp = None.
10. Assigns self.base_url = 'https://www.topstepx.com'.
11. Assigns self.login_url = 'https://www.topstepx.com/login'.
12. Assigns self.user_api_url = 'https://userapi.topstepx.com'.
13. Assigns self.chart_api_url = 'https://chartapi.topstepx.com'.
14. Assigns self._api_token = None.
15. ... and 13 more statement(s) in the body.

Code (copy-paste safe)

```
def __init__(self, username=None, password=None, pair_id=None):
    self.username = username or ""
    self.password = password or ""
    self.pair_id = pair_id or "default"
    self.logged_in = False
    self.driver = None
    self.first_trade_attempted = False
    self._first_stats_fetch = True
    self._placing_order = False # Flag to block stats fetching during order placement
    self._login_timestamp = None # Track when we logged in for debugging
    self.base_url = "https://www.topstepx.com"
```



```

self.login_url = "https://www.topstepx.com/login"
self.user_api_url = "https://userapi.topstepx.com"
self.chart_api_url = "https://chartapi.topstepx.com"
self._api_token = None
self._api_session = None
self._api_user_id = None
self._api_account_id = None
self._api_token_expiry = 0
self._delay_snapshots_enabled = True # Enable automatic snapshot capture on delays
self.lock = threading.RLock() # Thread safety for Selenium operations

# Initialize logger - MFFU blueprint standard
self.logger = logging.getLogger(f"TopStepX_{self.username}_{self.pair_id}")
self.logger.info(f"[INIT] TopStepXAccount initializing for user={username}, pair_id={pair_id}")

# Create unique instance key using both username and pair_id
instance_key = f"{username}_{self.pair_id}"
self.logger.info(f"[INIT] Instance key: {instance_key}")

# Check if Chrome instance already exists for this specific account+pair combination
if instance_key in self._chrome_instances:
    self.logger.info(
        f"[INIT] Reusing existing Chrome instance for TopStepX account: {username} (Pair: {self.pair_id})"
    )
    existing_driver = self._chrome_instances[instance_key]
    # Test if existing driver is still alive
    try:
        existing_driver.current_url
        self.driver = existing_driver
        self.logger.info("[INIT] Successfully connected to existing Chrome instance")
        return
    except Exception as e:
        self.logger.info(f"[INIT] Existing Chrome instance is dead ({e}), creating new one")
        # Remove dead instance from registry
        del self._chrome_instances[instance_key]

# Initialize driver with error handling
self.logger.info("[INIT] No existing Chrome instance found - creating new WebDriver")
try:
    self.logger.info("[INIT] Calling _initialize_driver()...")
    self.driver = self._initialize_driver()
    self.logger.info(f"[INIT] WebDriver created successfully: {self.driver}")
    # Register the Chrome instance for this specific account+pair combination
    self._chrome_instances[instance_key] = self.driver
    self.logger.info(
        f"[INIT] Chrome instance registered for TopStepX: {username} (Pair: {self.pair_id})"
    )
except Exception as e:
    self.logger.error(f"[INIT ERROR] Failed to initialize WebDriver: {str(e)}")
    import traceback
    self.logger.error(f"[INIT ERROR] Full traceback:\n{traceback.format_exc()}")
    raise Exception(
        f"Failed to initialize WebDriver: {str(e)}. This may indicate VPS Chrome setup issues."
    )

```

Method: TopStepXAccount._get_chromedriver_path

```
def _get_chromedriver_path(self)
```

Get the path to ChromeDriver executable with auto-compatibility check

Step by step (top-level body)

1. Calls `logging.info(...)` for its side effect.

2. `return None`.

Code (copy-paste safe)

```
def _get_chromedriver_path(self):
    """Get the path to ChromeDriver executable with auto-compatibility check"""
    # SELENIUM 4.15+ AUTO-MANAGEMENT: Let Selenium Manager handle ChromeDriver automatically
    # This ensures ChromeDriver always matches the installed Chrome version
    logging.info("[AUTO] Using Selenium Manager for automatic ChromeDriver version matching")
    return None
```

Method: `TopStepXAccount._ensure_chrome_compatibility`

```
def _ensure_chrome_compatibility(self)
    Ensure Chrome-ChromeDriver version compatibility
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. `try` block with 1 `except` clause.

Code (copy-paste safe)

```
def _ensure_chrome_compatibility(self):
    """Ensure Chrome-ChromeDriver version compatibility"""
    try:
        # Import the compatibility manager
        sys.path.insert(0, os.path.dirname(os.path.dirname(__file__)))
        from chrome_auto_compatibility import ChromeVersionManager # type: ignore

        manager = ChromeVersionManager()
        success = manager.ensure_compatibility()

        if success:
            logging.info("[CHECK] Chrome-ChromeDriver compatibility verified")
        else:
            logging.warning("[WARNING] Chrome-ChromeDriver compatibility could not be ensured")

    except Exception as e:
        logging.warning(f"Chrome compatibility check failed: {e}")
```

Method: `TopStepXAccount._initialize_driver`

```
def _initialize_driver(self)
    Initialize Chrome WebDriver with crash-resistant options
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Calls `self.logger.info(...)` for its side effect.
2. Assigns `chrome_options = Options()`.
3. Assigns `unique_id = f'{self.username}_{self.pair_id}'`.
4. Assigns `unique_hash = hashlib.md5(unique_id.encode()).hexdigest()[:8]`.
5. Assigns `user_data_dir = os.path.join(tempfile.gettempdir(), f'chrome_topstepx_{un....`
6. Calls `self.logger.info(...)` for its side effect.
7. Calls `chrome_options.add_argument(...)` for its side effect.
8. Assigns `port_base = 9322`.
9. Assigns `port_offset = int(unique_hash[:4], 16) % 100`.
10. Assigns `debug_port = port_base + port_offset`.
11. Calls `self.logger.info(...)` for its side effect.
12. Calls `chrome_options.add_argument(...)` for its side effect.
13. Assigns `pair_num = 0`.
14. **try** block with 1 **except** clause.
15. ... and 19 more statement(s) in the body.

Code (copy-paste safe)

```
def _initialize_driver(self):
    """Initialize Chrome WebDriver with crash-resistant options"""
    self.logger.info("[DRIVER] Starting Chrome WebDriver initialization...")
    chrome_options = Options()

    # Create persistent user data directory per account+pair to ensure separate Chrome instances
    # Use username+pair_id hash to create unique directory per account+pair combination
    unique_id = f'{self.username}_{self.pair_id}'
    unique_hash = hashlib.md5(unique_id.encode()).hexdigest()[:8]
    user_data_dir = os.path.join(tempfile.gettempdir(), f'chrome_topstepx_{unique_hash}')
    self.logger.info(f"[DRIVER] User data dir: {user_data_dir}")
    chrome_options.add_argument(f"--user-data-dir={user_data_dir}")

    # Use unique debugging port per account+pair
    port_base = 9322 # Different from Tradovate (9222) to avoid conflicts
    port_offset = int(unique_hash[:4], 16) % 100 # 0-99 offset based on unique_id
    debug_port = port_base + port_offset
```

```

self.logger.info(f"[DRIVER] Remote debugging port: {debug_port}")
chrome_options.add_argument(f"--remote-debugging-port={debug_port}")

# Position windows differently for each pair to avoid overlapping
pair_num = 0
try:
    # Handle various pair_id formats: "pair_0", "topstepx_pair_0", "default", etc.
    if "pair_" in self.pair_id:
        # Extract number from formats like "pair_0" or "topstepx_pair_0"
        pair_part = self.pair_id.split("pair_-")[-1] # Get the part after last "pair_"
        pair_num = int(pair_part) if pair_part.isdigit() else 0
    elif self.pair_id.isdigit():
        pair_num = int(self.pair_id)
except (ValueError, IndexError):
    pair_num = 0 # Fallback to 0 if parsing fails

window_x = 150 + (pair_num * 50) # Offset from Tradovate windows
window_y = 100 + (pair_num * 50)
self.logger.info(f"[DRIVER] Window position: {window_x}, {window_y}")
chrome_options.add_argument(f"--window-position={window_x},{window_y}")

# SPEED OPTIMIZATION: Essential options only for fastest startup
self.logger.info("[DRIVER] Adding Chrome options...")
chrome_options.add_argument("--no-sandbox")
chrome_options.add_argument("--disable-dev-shm-usage")
chrome_options.add_argument("--disable-gpu")
chrome_options.add_argument("--disable-extensions")
chrome_options.add_argument("--no-first-run")
chrome_options.add_argument("--no-default-browser-check")
chrome_options.add_argument("--disable-features=TranslateUI")

# SPEED OPTIMIZATION: Smaller window for faster rendering
chrome_options.add_argument("--window-size=1200,800") # Reduced from 1400,900

# Anti-detection settings (keep minimal)
chrome_options.add_experimental_option("excludeSwitches", ["enable-automation", "enable-logging"])
chrome_options.add_experimental_option('useAutomationExtension', False)
chrome_options.add_argument("--disable-blink-features=AutomationControlled")

# Minimal preferences to prevent crashes
prefs = {
    "profile.default_content_setting_values": {
        "popups": 2, # Block popups only
        "notifications": 2, # Block notifications only
    }
}
chrome_options.add_experimental_option("prefs", prefs)

try:
    # Get ChromeDriver path (returns None for auto-management in Selenium 4.15+)
    chromedriver_path = self._get_chromedriver_path()

    # Create the WebDriver instance - let Selenium Manager handle ChromeDriver automatically
    self.logger.info("[CHROME] Initializing Chrome browser...")
    self.logger.info("[CHROME] Using Selenium Manager for automatic ChromeDriver version matching")
    self.logger.info(
        "[CHROME] Selenium Manager will download ChromeDriver if needed (happens once per Chrome"
    )
    self.logger.info("[CHROME] If already cached, Chrome will launch immediately...")

    import time

```

```

start_time = time.time()
driver = webdriver.Chrome(options=chrome_options)
elapsed = time.time() - start_time

if elapsed > 5:
    self.logger.info(
        f"[CHROME] ✓ Chrome launched in {elapsed:.1f}s (ChromeDriver was downloaded/updated)"
    )
else:
    self.logger.info(f"[CHROME] ✓ Chrome launched in {elapsed:.1f}s (using cached ChromeDriver)")

self.logger.info("[DRIVER] Chrome instance created, setting timeouts...")
# SPEED OPTIMIZATION: Aggressive timeouts for faster performance
driver.set_page_load_timeout(15) # Reduced from 30
driver.implicitly_wait(2) # Reduced from 3

# Remove automation detection
self.logger.info("[DRIVER] Executing anti-detection script...")
driver.execute_script("Object.defineProperty(navigator, 'webdriver', {get: () => undefined})")

self.logger.info(
    f"[DRIVER] Chrome WebDriver initialized successfully for TopStepX {self.username}"
)
return driver

except Exception as e:
    self.logger.error(f"[DRIVER ERROR] Failed to initialize Chrome WebDriver: {e}")
    import traceback
    self.logger.error(f"[DRIVER ERROR] Full traceback:\n{traceback.format_exc()}")
    raise Exception(f"ChromeDriver initialization failed: {e}")

```

Method: TopStepXAccount.login

```
def login(self, max_retries=3)
```

Login to TopStepX platform

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.

Step by step (top-level body)

1. **with self.lock**: enters a context manager.

Code (copy-paste safe)

```

def login(self, max_retries=3):
    """
    Login to TopStepX platform
    """
    # Acquire lock to prevent stats fetching during login
    with self.lock:
        if not self.username or not self.password:
            self.logger.error("Username and password are required for TopStepX login")
            return False

        for attempt in range(max_retries):
            try:
                self.logger.info(f"TopStepX login attempt {attempt + 1}/{max_retries}")

                if not self.driver:
                    if not self.initialize_driver():
                        raise Exception("Failed to initialize WebDriver")

                # Navigate to login page
                self.logger.info(f"Navigating to TopStepX login: {self.login_url}")
                self.driver.get(self.login_url)
                time.sleep(3)

                # Log current page info
                self.logger.info(f"Current URL: {self.driver.current_url}")
                self.logger.info(f"Page title: {self.driver.title}")

                # Wait for login form
                wait = WebDriverWait(self.driver, 15)

                # Find and fill username field (TopStepX uses 'userName' not 'email')
                username_field = wait.until(
                    EC.presence_of_element_located((By.NAME, "userName")))
                )
                # Highlight existing value and type over it (no deletion needed)
                username_field.send_keys(Keys.CONTROL + "a") # Select all existing text
                time.sleep(0.2) # Allow selection to complete
                username_field.send_keys(self.username) # Type over selected text
                self.logger.info(f"Username field filled: {self.username}")

                # Find and fill password field
                password_field = self.driver.find_element(By.NAME, "password")
                # Highlight existing value and type over it (no deletion needed)
                password_field.send_keys(Keys.CONTROL + "a") # Select all existing text
                time.sleep(0.2) # Allow selection to complete
                password_field.send_keys(self.password) # Type over selected text
                self.logger.info("Password field filled")

                # Click login button (Sign In button)
                login_button = self.driver.find_element(By.XPATH, "//button[@type='submit']")
                self.logger.info("Clicking Sign In button")
                login_button.click()

                # Wait for page to load after login
                time.sleep(3)

                # Wait for either success or error indicators
                success_indicators = [
                    "dashboard",
                    "trading",

```

```

        "account",
        "logout" # logout button indicates successful login
    ]

    # Check if login was successful
    max_wait_time = 15
    start_time = time.time()

    while time.time() - start_time < max_wait_time:
        current_url = self.driver.current_url.lower()
        page_source = self.driver.page_source.lower()

        # Check for success indicators
        if any(indicator in current_url for indicator in success_indicators):
            self.logged_in = True
            self._login_timestamp = time.time() # Track login time
            self.logger.info(f"TopStepX login successful - redirected to: {current_url}")
            self._first_stats_fetch = True
            self._extract_api_token() # Extract JWT for REST API calls
            return True

        # Check if we're no longer on the login page (indicates success)
        if "/login" not in current_url and "topstepx.com" in current_url:
            self.logged_in = True
            self._login_timestamp = time.time() # Track login time
            self.logger.info(f"TopStepX login successful - left login page: {current_url}")
            self._first_stats_fetch = True
            self._extract_api_token() # Extract JWT for REST API calls
            return True

        # Check for error messages or still on login page
        if "sign in" in page_source and "/login" in current_url:
            # Still on login page, check for errors
            error_elements = self.driver.find_elements(By.XPATH,
                "//*[contains(@class, 'error') or contains(@class, 'alert') or contains(t
            if error_elements:
                error_text = error_elements[0].text
                self.logger.error(f"TopStepX login failed: {error_text}")
                break

        time.sleep(1)

    # If we get here, login likely failed
    self.logger.error("TopStepX login failed: Timeout or unknown error")

except TimeoutException:
    self.logger.error(f"TopStepX login attempt {attempt + 1} timed out")
except Exception as e:
    self.logger.error(f"TopStepX login attempt {attempt + 1} failed: {e}")

if attempt < max_retries - 1:
    time.sleep(5) # Wait before retry

return False

```

Method: TopStepXAccount.is_connected

```
def is_connected(self)
```

Check if we're connected to TopStepX

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def is_connected(self):
    """Check if we're connected to TopStepX"""
    try:
        if not self.driver or not self.logged_in:
            return False

        # Check if we're still on a valid TopStepX page
        current_url = self.driver.current_url
        return "topstepx.com" in current_url and self.logged_in

    except Exception:
        return False
```

Method: TopStepXAccount.validate_credentials

```
def validate_credentials(self)

    Validate if credentials are set and ready for login Returns (is_valid, message)
```

Step by step (top-level body)

1. **if** not self.username: branches conditionally.
2. **if** not self.password: branches conditionally.
3. **return** (True, 'Credentials are valid').

Code (copy-paste safe)

```
def validate_credentials(self):
    """
    Validate if credentials are set and ready for login
    Returns (is_valid, message)
    """
    if not self.username:
        return False, "Username is required"
    if not self.password:
        return False, "Password is required"
    return True, "Credentials are valid"
```

Method: TopStepXAccount._extract_api_token

```
def _extract_api_token(self)

    Extract JWT token from browser localStorage after login
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code [\(copy-paste safe\)](#)

```
def _extract_api_token(self):
    """Extract JWT token from browser localStorage after login"""
    try:
        if not self.driver:
            return False
        token = self.driver.execute_script("return localStorage.getItem('token')")
        if not token or len(token) < 50:
            self.logger.warning("[API] No JWT token found in localStorage")
            return False
        self._api_token = token
        # Decode JWT claims to get userId and expiry
        parts = token.split(".")
        if len(parts) >= 2:
            payload = parts[1] + "=" * (4 - len(parts[1]) % 4)
            try:
                claims = json.loads(base64.urlsafe_b64decode(payload))
                self._api_user_id = claims.get(
                    "http://schemas.xmlsoap.org/ws/2005/05/identity/claims/nameidentifier"
                )
                self._api_token_expiry = claims.get("exp", 0)
                self.logger.info(f"[API] JWT extracted - userId={self._api_user_id}, "
                                f"expires={datetime.fromtimestamp(self._api_token_expiry).isoformat()}")
            except Exception as e:
                self.logger.warning(f"[API] Could not decode JWT claims: {e}")
        # Build requests session
        self._api_session = requests.Session()
        self._api_session.headers.update({
            "Authorization": f"Bearer {self._api_token}",
            "Content-Type": "application/json",
            "Accept": "application/json",
            "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36",
            "Origin": "https://www.topstepx.com",
            "Referer": "https://www.topstepx.com/",
        })
        self.logger.info("[API] REST API session initialized")
        return True
    except Exception as e:
        self.logger.error(f"[API] Failed to extract token: {e}")
        return False
```

Method: TopStepXAccount._refresh_api_token

```
def _refresh_api_token(self)
```

Re-extract token from browser if expired or missing

Step by step (top-level body)

1. **if** `self._api_token` and `time.time() < self._api_token_expiry - 60`: branches conditionally.
2. Calls `self.logger.info(...)` for its side effect.
3. **return** `self._extract_api_token()`.

Code (copy-paste safe)

```
def _refresh_api_token(self):
    """Re-extract token from browser if expired or missing"""
    if self._api_token and time.time() < self._api_token_expiry - 60:
        return True # Token still valid
    self.logger.info("[API] Token expired or missing, re-extracting from browser")
    return self._extract_api_token()
```

Method: `TopStepXAccount._api_get`

```
def _api_get(self, path, base_url=None)
```

Make authenticated GET request to TopStepX API. Returns parsed JSON or None.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **if** `not self._api_session`: branches conditionally.
2. Calls `self._refresh_api_token(...)` for its side effect.
3. Assigns `url = f'{base_url or self.user_api_url}{path}'`.
4. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def _api_get(self, path, base_url=None):
    """Make authenticated GET request to TopStepX API. Returns parsed JSON or None."""
    if not self._api_session:
        if not self._extract_api_token():
            return None
    self._refresh_api_token()
    url = f'{base_url or self.user_api_url}{path}'
    try:
        resp = self._api_session.get(url, timeout=10)
        if resp.status_code == 200:
            ct = resp.headers.get("content-type", "")
            if "json" in ct or resp.text.strip().startswith(("{", "[")):
                return resp.json()
            return resp.text
        elif resp.status_code == 204:
            return [] # No content
        else:
            self.logger.warning(f"[API] GET {path} returned {resp.status_code}")
            return None
    except Exception as e:
```

```
self.logger.error(f"[API] GET {path} failed: {e}")
return None
```

Method: TopStepXAccount._api_post

```
def _api_post(self, path, data=None, base_url=None)
```

Make authenticated POST request to TopStepX API. Returns parsed JSON or None.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **if** not self._api_session: branches conditionally.
2. Calls self._refresh_api_token(...) for its side effect.
3. Assigns url = f'{base_url or self.user_api_url}{path}'.
4. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def _api_post(self, path, data=None, base_url=None):
    """Make authenticated POST request to TopStepX API. Returns parsed JSON or None."""
    if not self._api_session:
        if not self._extract_api_token():
            return None
    self._refresh_api_token()
    url = f'{base_url or self.user_api_url}{path}'
    try:
        resp = self._api_session.post(url, json=data, timeout=10)
        if resp.status_code in (200, 201):
            ct = resp.headers.get("content-type", "")
            if "json" in ct or resp.text.strip().startswith(("{", "[")):
                return resp.json()
            return resp.text
        elif resp.status_code == 204:
            return []
        else:
            self.logger.warning(f"[API] POST {path} returned {resp.status_code}")
            return None
    except Exception as e:
        self.logger.error(f"[API] POST {path} failed: {e}")
        return None
```

Method: TopStepXAccount.api_available

```
@property
def api_available(self)
```

Check if REST API is available (token extracted and session ready)

Why these Python concepts were used

- **@property** — Wrapped in **@property** so callers access the value with attribute syntax (`obj.x`) instead of a method call. Lets the implementation compute or validate the value lazily without changing the call site.

Step by step (top-level body)

1. **return** `self._api_session` is not `None` and `self._api_token` is not `None`.

Code (copy-paste safe)

```
def api_available(self):
    """Check if REST API is available (token extracted and session ready)"""
    return self._api_session is not None and self._api_token is not None
```

Method: `TopStepXAccount.api_get_user`

```
def api_get_user(self)
```

Get user profile via REST API. Returns dict with: `userName`, `email`, `userId`, `firstName`, `lastName`, `has2FA`, etc.

Step by step (top-level body)

1. **return** `self._api_get('/User')`.

Code (copy-paste safe)

```
def api_get_user(self):
    """Get user profile via REST API.
    Returns dict with: userName, email, userId, firstName, lastName, has2FA, etc."""
    return self._api_get("/User")
```

Method: `TopStepXAccount.api_get_trading_accounts`

```
def api_get_trading_accounts(self)
```

Get all trading accounts via REST API. Returns list of account dicts with: `accountId`, `accountName`, `balance`, `startingBalance`, `profitAndLoss`, `winRate`, `totalTrades`, `status`, `realizedDayPnl`, `openPnl`, `highestBalance`, `maximumLoss`, etc.

Step by step (top-level body)

1. **return** `self._api_get('/TradingAccount')`.

Code (copy-paste safe)

```
def api_get_trading_accounts(self):
    """Get all trading accounts via REST API.
    Returns list of account dicts with: accountId, accountName, balance,
    startingBalance, profitAndLoss, winRate, totalTrades, status,
    realizedDayPnl, openPnl, highestBalance, maximumLoss, etc."""
    return self._api_get("/TradingAccount")
```

Method: `TopStepXAccount.api_get_positions`

```
def api_get_positions(self, user_id=None)
```

Get all open positions via REST API. Returns list of position dicts.

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `uid = user_id` or `self._api_user_id`.
2. `if not uid`: branches conditionally.
3. `return self._api_get(f'/Position/all/user/{uid}')` or `[]`.

Code (copy-paste safe)

```
def api_get_positions(self, user_id=None):
    """Get all open positions via REST API.
    Returns list of position dicts."""
    uid = user_id or self._api_user_id
    if not uid:
        self.logger.warning("[API] No user_id available for positions query")
        return []
    return self._api_get(f"/Position/all/user/{uid}") or []
```

Method: TopStepXAccount.api_get_orders

```
def api_get_orders(self, account_id=None)
```

Get pending orders via REST API. Returns list of order dicts.

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.

Step by step (top-level body)

1. Assigns `acct_id = account_id` or `self._api_account_id`.
2. `if not acct_id`: branches conditionally.
3. `if not acct_id`: branches conditionally.
4. `return self._api_get(f'/Order?accountId={acct_id}')` or `[]`.

Code (copy-paste safe)

```
def api_get_orders(self, account_id=None):
    """Get pending orders via REST API.
    Returns list of order dicts."""
    acct_id = account_id or self._api_account_id
    if not acct_id:
        # Try to get from trading accounts
        accounts = self.api_get_trading_accounts()
        if accounts and isinstance(accounts, list) and len(accounts) > 0:
            acct_id = accounts[0].get("accountId")
            self._api_account_id = acct_id
```

```

    if not acct_id:
        return []
    return self._api_get(f"/Order?accountId={acct_id}") or []

```

Method: TopStepXAccount.api_get_trades

```
def api_get_trades(self, account_id=None)
```

Get trade history via REST API. Returns list of trade dicts with: id, symbolId, contractId, accountId, entryPrice, exitPrice, fees, pnl, positionSize, tradeDuration, etc.

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.

Step by step (top-level body)

1. Assigns `acct_id = account_id` or `self._api_account_id`.
2. `if not acct_id`: branches conditionally.
3. `if not acct_id`: branches conditionally.
4. `return self._api_get(f"/Trade/id/{acct_id}")` or `[]`.

Code (copy-paste safe)

```

def api_get_trades(self, account_id=None):
    """Get trade history via REST API.
    Returns list of trade dicts with: id, symbolId, contractId, accountId,
    entryPrice, exitPrice, fees, pnl, positionSize, tradeDuration, etc."""
    acct_id = account_id or self._api_account_id
    if not acct_id:
        accounts = self.api_get_trading_accounts()
        if accounts and isinstance(accounts, list) and len(accounts) > 0:
            acct_id = accounts[0].get("accountId")
            self._api_account_id = acct_id
    if not acct_id:
        return []
    return self._api_get(f"/Trade/id/{acct_id}") or []

```

Method: TopStepXAccount.api_get_contracts

```
def api_get_contracts(self)
```

Get all active tradeable contracts via REST API. Returns list with: productId, productName, contractId, contractName, tickValue, tickSize, pointValue, fees, exchange, expiration, etc.

Step by step (top-level body)

1. `return self._api_get('/UserContract/active/nonprofesional')` or `[]`.

Code (copy-paste safe)

```

def api_get_contracts(self):
    """Get all active tradeable contracts via REST API.
    Returns list with: productId, productName, contractId, contractName,

```

```
tickValue, tickSize, pointValue, fees, exchange, expiration, etc."""
return self._api_get("/UserContract/active/nonprofesional") or []
```

Method: TopStepXAccount.api_get_metadata

```
def api_get_metadata(self)
```

Get symbol metadata via REST API. Returns list with: symbol, friendlyName, fullName, tickSize, tickValue, contractCost, fees, decimalPlaces, exchange, etc.

Step by step (top-level body)

1. **return** self._api_get('/Metadata') or [].

Code (copy-paste safe)

```
def api_get_metadata(self):
    """Get symbol metadata via REST API.
    Returns list with: symbol, friendlyName, fullName, tickSize, tickValue,
    contractCost, fees, decimalPlaces, exchange, etc."""
    return self._api_get("/Metadata") or []
```

Method: TopStepXAccount.api_get_market_status

```
def api_get_market_status(self)
```

Get market open/close status for all symbols via REST API. Returns list with: symbolId, contractId, status, isOpen, openedAt, etc.

Step by step (top-level body)

1. **return** self._api_get('/MarketStatus/markets') or [].

Code (copy-paste safe)

```
def api_get_market_status(self):
    """Get market open/close status for all symbols via REST API.
    Returns list with: symbolId, contractId, status, isOpen, openedAt, etc."""
    return self._api_get("/MarketStatus/markets") or []
```

Method: TopStepXAccount.api_get_trading_rules

```
def api_get_trading_rules(self)
```

Get trading rules (DAILY_LOSS, MAXIMUM_LOSS, MARGIN_SCALE) via REST API.

Step by step (top-level body)

1. **return** self._api_get('/TradingRule') or [].

Code (copy-paste safe)

```
def api_get_trading_rules(self):
    """Get trading rules (DAILY_LOSS, MAXIMUM_LOSS, MARGIN_SCALE) via REST API."""
    return self._api_get("/TradingRule") or []
```

Method: TopStepXAccount.api_get_template_rules

```
def api_get_template_rules(self, template_id=1)
```

Get account template rules (loss limits, etc.) via REST API.

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **return** `self._api_get(f'/AccountTemplateRule/rules/{template_id}/all')` or `[]`.

Code (copy-paste safe)

```
def api_get_template_rules(self, template_id=1):
    """Get account template rules (loss limits, etc.) via REST API."""
    return self._api_get(f"/AccountTemplateRule/rules/{template_id}/all") or []
```

Method: `TopStepXAccount.api_get_violations`

```
def api_get_violations(self, account_id=None)
```

Get active rule violations via REST API.

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.

Step by step (top-level body)

1. Assigns `acct_id = account_id` or `self._api_account_id`.
2. **if not** `acct_id`: branches conditionally.
3. **if not** `acct_id`: branches conditionally.
4. **return** `self._api_get(f'/Violations/active/{acct_id}')` or `[]`.

Code (copy-paste safe)

```
def api_get_violations(self, account_id=None):
    """Get active rule violations via REST API."""
    acct_id = account_id or self._api_account_id
    if not acct_id:
        accounts = self.api_get_trading_accounts()
        if accounts and isinstance(accounts, list) and len(accounts) > 0:
            acct_id = accounts[0].get("accountId")
            self._api_account_id = acct_id
    if not acct_id:
        return []
    return self._api_get(f"/Violations/active/{acct_id}") or []
```

Method: `TopStepXAccount.api_get_linked_orders`

```
def api_get_linked_orders(self, account_id=None)
```

Get linked/bracket orders via REST API.

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.

Step by step (top-level body)

1. Assigns `acct_id = account_id` or `self._api_account_id`.
2. `if not acct_id`: branches conditionally.
3. `if not acct_id`: branches conditionally.
4. `return self._api_get(f'/LinkedOrder?tradingAccountId={acct_id}')`.

Code (copy-paste safe)

```
def api_get_linked_orders(self, account_id=None):
    """Get linked/bracket orders via REST API."""
    acct_id = account_id or self._api_account_id
    if not acct_id:
        accounts = self.api_get_trading_accounts()
        if accounts and isinstance(accounts, list) and len(accounts) > 0:
            acct_id = accounts[0].get("accountId")
            self._api_account_id = acct_id
    if not acct_id:
        return {"linkedOrders": []}
    return self._api_get(f"/LinkedOrder?tradingAccountId={acct_id}")
```

Method: TopStepXAccount.api_get_user_settings

```
def api_get_user_settings(self)
    Get user settings (risk, toMake, autoCenter, etc.) via REST API.
```

Step by step (top-level body)

1. `return self._api_get('/UserSettings')`.

Code (copy-paste safe)

```
def api_get_user_settings(self):
    """Get user settings (risk, toMake, autoCenter, etc.) via REST API."""
    return self._api_get("/UserSettings")
```

Method: TopStepXAccount.api_get_notifications

```
def api_get_notifications(self)
    Get user notifications via REST API.
```

Step by step (top-level body)

1. `return self._api_get('/UserNotification') or []`.

Code (copy-paste safe)

```
def api_get_notifications(self):
```

```

        """Get user notifications via REST API."""
        return self._api_get("/UserNotification") or []

```

Method: TopStepXAccount.api_get_chart_config

```
def api_get_chart_config(self)
```

Get TradingView chart config (supported resolutions, etc.) via REST API.

Step by step (top-level body)

1. **return** self._api_get('/Config', base_url=self.chart_api_url).

Code (copy-paste safe)

```

def api_get_chart_config(self):
    """Get TradingView chart config (supported resolutions, etc.) via REST API."""
    return self._api_get("/Config", base_url=self.chart_api_url)

```

Method: TopStepXAccount.get_blueprint_config

```
def get_blueprint_config(self, phase_key='challenge_trade1', size_key='50k')
```

Get TopStepX blueprint configuration from PropFirmManager Returns config for the specified phase and account size

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **if not** PROP_FIRM_MANAGER_AVAILABLE: branches conditionally.
2. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def get_blueprint_config(self, phase_key="challenge_trade1", size_key="50k"):
    """
    Get TopStepX blueprint configuration from PropFirmManager
    Returns config for the specified phase and account size
    """
    if not PROP_FIRM_MANAGER_AVAILABLE:
        logging.warning("PropFirmManager not available - using defaults")
        return {
            'topstepx_symbol': DEFAULT_SYMBOL,
            'topstepx_qty': 1,
            'topstepx_tp_ticks': DEFAULT_TP,
            'topstepx_sl_ticks': DEFAULT_SL
        }

    try:
        manager = PropFirmManager()

```

```

# Force TopStep firm code since this is TopStepX integration
# We don't need to detect from username because we are in the TopStepX module
firm_code = "TopStep"

# Get strategy config for TopStepX
manager.set_prop_firm(firm_code)
config = manager.get_prop_firm_strategy_config(phase_key, size_key)

if config and 'topstepx_symbol' in config:
    logging.info(f"TopStepX blueprint loaded: {phase_key}/{size_key} -> {config}")
    return config
else:
    logging.warning("No TopStepX-specific config found, using defaults")
    return {
        'topstepx_symbol': DEFAULT_SYMBOL,
        'topstepx_qty': 1,
        'topstepx_tp_ticks': DEFAULT_TP,
        'topstepx_sl_ticks': DEFAULT_SL
    }

except Exception as e:
    logging.error(f"Failed to get TopStepX blueprint config: {e}")
    return {
        'topstepx_symbol': DEFAULT_SYMBOL,
        'topstepx_qty': 1,
        'topstepx_tp_ticks': DEFAULT_TP,
        'topstepx_sl_ticks': DEFAULT_SL
    }

```

Method: TopStepXAccount.list_accounts

```
def list_accounts(self)
```

List all trading accounts available for this user. Returns list of dicts with: accountId, accountName, balance, status, ineligible, etc. Uses REST API if available, otherwise falls back to reading the UI dropdown.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **if** self.api_available: branches conditionally.
2. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def list_accounts(self):
    """List all trading accounts available for this user.
    Returns list of dicts with: accountId, accountName, balance, status, ineligible, etc.
    Uses REST API if available, otherwise falls back to reading the UI dropdown."""
    if self.api_available:
        accounts = self.api_get_trading_accounts()

```

```

        if accounts:
            return accounts

# Fallback: read the MuiSelect dropdown options from UI
try:
    items = self.driver.execute_script("""
        const select = document.querySelector('[role="combobox"]');
        if (!select) return null;
        select.click(); // open dropdown
        await new Promise(r => setTimeout(r, 500));
        const options = document.querySelectorAll('[role="option"], .MuiMenuItem-root');
        const result = Array.from(options).map(o => ({text: o.textContent.trim()}));
        // close by pressing Escape
        document.dispatchEvent(new KeyboardEvent('keydown', {key: 'Escape'}));
        return result;
    """)
    return items or []
except Exception as e:
    self.logger.warning(f"[SWITCH] Could not list accounts from UI: {e}")
    return []

```

Method: TopStepXAccount.switch_account

```
def switch_account(self, account_id=None, account_name_contains=None)
```

Switch to a sub-account via the MuiSelect dropdown. Minimal/fast path: 1. Per-session cache hit → instant return. 2. Read selector text → if already shows target, cache + return. 3. Click selector → click matching → done. No API calls, no _ensure_on_trading_page, no extra DOM scans.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **with self.lock**: enters a context manager.

Code (copy-paste safe)

```

def switch_account(self, account_id=None, account_name_contains=None):
    """Switch to a sub-account via the MuiSelect dropdown.

    Minimal/fast path:
    1. Per-session cache hit → instant return.
    2. Read selector text → if already shows target, cache + return.
    3. Click selector → click matching <li> → done.

    No API calls, no _ensure_on_trading_page, no extra DOM scans.

```

```

"""
with self.lock:
    try:
        search = (account_name_contains or "").strip()
        cache_key = str(account_id) if account_id else search.lower()
        if not search and not account_id:
            return False

        # 1) Cache hit
        if cache_key and getattr(self, "_last_switched_key", None) == cache_key:
            return True

        # 2) Locate the selector and check if already active
        select_el = self.driver.find_element(By.XPATH,
            "//div[contains(@class, 'MuiSelect-select') and contains(@class, 'MuiInputBase-input
        )
        try:
            current = (select_el.text or "").strip()
        except Exception:
            current = ""
        if search and current and search.lower() in current.lower():
            self._last_switched_key = cache_key
            return True

        # 3) Open dropdown and click the matching option
        # Dismiss any backdrop first (a leftover from a previous interaction
        # can intercept the click and silently swallow it).
        try:
            self.driver.execute_script(
                "document.querySelectorAll('.MuiBackdrop-root').forEach(el => el.click());"
            )
        except Exception:
            pass

        # MUI Select needs a real pointer event to open. JS-click sometimes works,
        # but a native Selenium .click() is much more reliable. Try native first,
        # fall back to JS if intercepted.
        opened = False
        try:
            select_el.click()
            opened = True
        except Exception:
            try:
                self.driver.execute_script("arguments[0].click()", select_el)
                opened = True
            except Exception as click_e:
                self.logger.error(f"[SWITCH] Could not click selector: {click_e}")
                return False

        # Poll for menu items (up to 3s – first dropdown open after page load can be slow)
        items = []
        deadline = time.time() + 3.0
        option_xpath = (
            "//li[contains(@class, 'MuiMenuItem-root')] | "
            "//*[ @role='option'] | "
            "//div[contains(@class, 'MuiPopover-root')]//li"
        )
        while time.time() < deadline:
            items = self.driver.find_elements(By.XPATH, option_xpath)
            if items:

```

```

        break
    time.sleep(0.05)

if not items:
    # Diagnostic dump so we can see what actually rendered
    try:
        popover_html = self.driver.execute_script("""
            const pop = document.querySelector(
                '.MuiPopover-root, .MuiMenu-root, [role="presentation"]');
            return pop ? pop.outerHTML.substring(0, 2000) : '(no popover element found)';
        """)
        self.logger.error(
            f"[SWITCH] Dropdown opened but no options found. HTML: {popover_html}")
    except Exception:
        pass

clicked = False
needle = search.lower() if search else None
for item in items:
    try:
        text = (item.text or "").strip()
    except Exception:
        continue
    if not text:
        continue
    if needle and needle in text.lower():
        # Native click first, then JS fallback
        try:
            item.click()
        except Exception:
            try:
                self.driver.execute_script("arguments[0].click()", item)
            except Exception:
                continue
        clicked = True
        self.logger.info(f"[SWITCH] Clicked: {text[:80]}")
        break

if not clicked:
    self.logger.error(f"[SWITCH] '{search}' not found among {len(items)} dropdown options")
    try:
        ActionChains(self.driver).send_keys(Keys.ESCAPE).perform()
    except Exception:
        pass
    return False

# Invalidate cached stats so next poll fetches fresh data
self._cached_stats = None
self._stats_last_fetch_time = 0
self._first_stats_fetch = True
self._last_switched_key = cache_key

# Brief settle so the selector text + downstream UI catch up
time.sleep(0.4)
return True

except Exception as e:
    self.logger.error(f"[SWITCH] Account switch failed: {e}")
    return False

```

Method: TopStepXAccount.cleanup_chrome_instance

```
def cleanup_chrome_instance(self)
```

Clean up Chrome instance for this specific account+pair combination Part of MFFU blueprint pattern for proper resource management

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `instance_key = f'{self.username}_{self.pair_id}'`.
2. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def cleanup_chrome_instance(self):
    """
    Clean up Chrome instance for this specific account+pair combination
    Part of MFFU blueprint pattern for proper resource management
    """
    instance_key = f"{self.username}_{self.pair_id}"

    try:
        if instance_key in self._chrome_instances:
            driver = self._chrome_instances[instance_key]
            try:
                driver.quit()
                logging.info(
                    f"Chrome instance cleaned up for TopStepX: {self.username} (Pair: {self.pair_id})"
                )
            except Exception as e:
                logging.warning(f"Error closing Chrome instance for TopStepX {self.username}: {e}")

            # Remove from registry
            del self._chrome_instances[instance_key]

    except Exception as e:
        logging.error(f"Error during TopStepX Chrome cleanup: {e}")
```

Method: TopStepXAccount.disconnect

```
def disconnect(self)
```

Disconnect from TopStepX and clean up resources MFFU blueprint standard method

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def disconnect(self):
    """
    Disconnect from TopStepX and clean up resources
    MFFU blueprint standard method
    """
    try:
        self.logged_in = False

        if self.driver:
            try:
                # Try to logout gracefully first
                if "topstepx.com" in self.driver.current_url:
                    logout_selectors = [
                        "//button[contains(text(), 'Logout')]",
                        "//a[contains(text(), 'Logout')]",
                        "//button[contains(@class, 'logout')]",
                        "//a[contains(@class, 'logout')]"
                    ]

                    for selector in logout_selectors:
                        try:
                            logout_btn = self.driver.find_element(By.XPATH, selector)
                            logout_btn.click()
                            time.sleep(1)
                            logging.info("TopStepX logout successful")
                            break
                        except:
                            continue

                    except Exception as e:
                        logging.warning(f"Graceful TopStepX logout failed: {e}")

                # Clean up Chrome instance
                self.cleanup_chrome_instance()
                self.driver = None

            logging.info(f"TopStepX disconnection completed for {self.username}")

        except Exception as e:
            logging.error(f"Error during TopStepX disconnection: {e}")
```

Method: TopStepXAccount.__del__

```
def __del__(self)
```

Destructor to ensure cleanup on object deletion MFFU blueprint standard pattern

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't

have to handle it.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def __del__(self):
    """
    Destructor to ensure cleanup on object deletion
    MFFU blueprint standard pattern
    """
    try:
        if hasattr(self, 'driver') and self.driver:
            self.disconnect()
    except:
        pass
```

Method: TopStepXAccount._capture_delay_snapshot

```
def _capture_delay_snapshot(self, context_name, delay_ms, threshold_ms=200)
```

Capture DOM snapshot when a delay exceeds threshold. Args: context_name: Description of what operation was delayed (e.g., "contract_to_quantity") delay_ms: Actual delay in milliseconds threshold_ms: Threshold in milliseconds (default 200ms)

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **if not self._delay_snapshots_enabled or delay_ms < threshold_ms**: branches conditionally.
2. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def _capture_delay_snapshot(self, context_name, delay_ms, threshold_ms=200):
    """
    Capture DOM snapshot when a delay exceeds threshold.

    Args:
        context_name: Description of what operation was delayed (e.g., "contract_to_quantity")
        delay_ms: Actual delay in milliseconds
        threshold_ms: Threshold in milliseconds (default 200ms)
    """
    if not self._delay_snapshots_enabled or delay_ms < threshold_ms:
```

```

        return

    try:
        from datetime import datetime
        timestamp = datetime.now().strftime("%Y%m%d_%H%M%S%f")[:-3] # Include milliseconds

        # Create filename with delay info
        filename = f"delay_snapshot_{context_name}_{int(delay_ms)}ms_{timestamp}.html"

        # Capture full page source
        page_source = self.driver.page_source

        # Capture focused element info
        focused_element_info = "Unknown"
        try:
            focused = self.driver.execute_script("return document.activeElement.outerHTML;")
            focused_element_info = focused[:500] if focused else "None"
        except:
            pass

        # Capture contract field state
        contract_field_info = "Unknown"
        try:
            contract_field = self.driver.find_element(By.XPATH,
                "//input[contains(@class, 'MuiAutocomplete-input')]")
            contract_field_info = f"Value: '{contract_field.get_attribute('value')}', Enabled: {contract_field.is_enabled()}"
        except:
            pass

        # Capture quantity field state
        quantity_field_info = "Unknown"
        try:
            quantity_field = self.driver.find_element(By.XPATH, "//input[@type='number'][@min='1']")
            quantity_field_info = f"Value: '{quantity_field.get_attribute('value')}', Enabled: {quantity_field.is_enabled()}"
        except:
            pass

        # Build comprehensive HTML with metadata
        html_content = f"""<!-- DELAY SNAPSHOT -->
<!-- Context: {context_name} -->
<!-- Delay: {delay_ms}ms (threshold: {threshold_ms}ms) -->
<!-- Timestamp: {timestamp} -->
<!-- Current URL: {self.driver.current_url} -->
<!-- Focused Element: {focused_element_info} -->
<!-- Contract Field: {contract_field_info} -->
<!-- Quantity Field: {quantity_field_info} -->

{page_source}
"""

        # Write to file
        with open(filename, 'w', encoding='utf-8') as f:
            f.write(html_content)

        self.logger.warning(
            f"■■■ DELAY DETECTED: {context_name} took {delay_ms}ms (>{threshold_ms}ms threshold)")
        self.logger.warning(f"■ Snapshot saved: {filename}")

    except Exception as e:
        self.logger.debug(f"Could not capture delay snapshot: {e}")

```

Method: TopStepXAccount.get_account_info

```
def get_account_info(self)
```

Get TopStepX account information

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def get_account_info(self):
    """Get TopStepX account information"""
    try:
        if not self.is_connected():
            raise Exception("Not connected to TopStepX")

        # Only navigate to trading page if we're clearly not on a trading-related page
        current_url = self.driver.current_url
        if "/trade" not in current_url and "/dashboard" not in current_url and "/account" not in current_url:
            self.logger.info("Not on trading/account page, navigating to get account info")
            self._ensure_on_trading_page()
        else:
            self.logger.debug("Already on trading/account page, skipping navigation for account info")

        account_info = {
            "platform": "TopStepX",
            "status": "Connected",
            "balance": "N/A",
            "equity": "N/A",
            "margin": "N/A",
            "account_type": "N/A"
        }

        # Extract account balance information from the interface
        try:
            # Look for balance displays - based on HTML analysis
            balance_selectors = [
                "//span[contains(text(), 'BAL:')]//following-sibling::span",
                "//div[contains(@aria-label, 'Current Account Balance')]//span[contains(text(), '$')]",
                "//span[contains(text(), '$') and contains(@class, 'balance')]"
            ]

            for selector in balance_selectors:
                try:
                    balance_element = self.driver.find_element(By.XPATH, selector)
                    balance_text = balance_element.text.strip()
```

```

        if '$' in balance_text:
            account_info["balance"] = balance_text
            account_info["equity"] = balance_text # Often the same in trading accounts
            break
    except:
        continue

    # Look for account type information
    account_type_selectors = [
        "//span[contains(text(), 'Trading Combine') or contains(text(), 'Funded') or contains(
        text(), 'Futures')]//div[contains(@class, 'account')]/span[contains(text(), 'K')]"
    ]

    for selector in account_type_selectors:
        try:
            account_element = self.driver.find_element(By.XPATH, selector)
            account_info["account_type"] = account_element.text.strip()
            break
        except:
            continue

    # Look for P&L information
    pnl_selectors = [
        "//span[contains(text(), 'RP&L:')]//following-sibling::span",
        "//span[contains(text(), 'UP&L:')]//following-sibling::span"
    ]

    realized_pnl = unrealized_pnl = "N/A"
    try:
        realized_element = self.driver.find_element(By.XPATH, pnl_selectors[0])
        realized_pnl = realized_element.text.strip()
    except:
        pass

    try:
        unrealized_element = self.driver.find_element(By.XPATH, pnl_selectors[1])
        unrealized_pnl = unrealized_element.text.strip()
    except:
        pass

    account_info["realized_pnl"] = realized_pnl
    account_info["unrealized_pnl"] = unrealized_pnl

except Exception as e:
    self.logger.warning(f"Could not extract all account info: {e}")

self.logger.info(f"TopStepX account info retrieved: {account_info}")
return account_info

except Exception as e:
    self.logger.error(f"Failed to get TopStepX account info: {e}")
    return None

```

Method: TopStepXAccount.get_account_stats

```
def get_account_stats(self)
```

Get TopStepX account statistics from positions tab - required by GUI

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **if** not self.lock.acquire(blocking=False): branches conditionally.
2. **try** block with 0 **except** clauses, plus a **finally**.

Code (copy-paste safe)

```
def get_account_stats(self):
    """Get TopStepX account statistics from positions tab - required by GUI"""
    # Use lock to prevent conflict with order placement
    if not self.lock.acquire(blocking=False):
        # If locked (e.g. placing order), return cached stats immediately
        return getattr(self, '_cached_stats', {
            "Account Number": "Trading...",
            "Balance": "N/A",
            "Profit/Loss": "N/A",
            "Open Trades": "N/A",
            "Symbol": "",
            "Direction": ""
        })

    try:
        if getattr(self, '_placing_order', False):
            return getattr(self, '_cached_stats', {
                "Account Number": "Trading...",
                "Balance": "N/A",
                "Profit/Loss": "N/A",
                "Open Trades": "N/A",
                "Symbol": "",
                "Direction": ""
            })

        # PERFORMANCE OPTIMIZATION: Return cached stats if fetched within the last 2 seconds
        # This prevents excessive DOM queries when GUI polls every 1 second
        cache_ttl = 2.0 # Cache time-to-live in seconds
        current_time = time.time()
        last_fetch_time = getattr(self, '_stats_last_fetch_time', 0)
        time_since_fetch = current_time - last_fetch_time

        if time_since_fetch < cache_ttl and hasattr(self, '_cached_stats'):
            # Return cached stats (still fresh)
            return self._cached_stats

    try:
        if not self.is_connected():
            return {
                "Account Number": "Not Connected",
                "Balance": "N/A",
                "Profit/Loss": "N/A",
                "Open Trades": "N/A",
                "Symbol": "",
                "Direction": ""
            }
```

```

    }

# Initialize default stats
stats = {
    "Account Number": "Unknown",
    "Balance": "N/A",
    "Profit/Loss": "N/A",
    "Open Trades": "0",
    "Symbol": "",
    "Direction": ""
}

# FAST PATH: Use REST API if available (much faster than Selenium DOM scraping)
if self.api_available:
    try:
        api_stats = self._get_stats_via_api()
        if api_stats:
            self._cached_stats = api_stats
            self._stats_last_fetch_time = time.time()
            return api_stats
    except Exception as e:
        self.logger.warning(f"[API] Stats via API failed, falling back to Selenium: {e}")

try:
    # Check if positions data is already visible before clicking tabs
    positions_visible = False
    try:
        # Look for positions grid or position data
        self.driver.find_element(By.XPATH,
            "//*[div[@role='grid']] | //div[contains(@class, 'MuiDataGrid')] | //*[contains(
        positions_visible = True
        self.logger.debug("Positions data already visible, skipping tab navigation")
    except:
        pass

    # Navigate to positions tab to extract stats
    if not positions_visible:
        if not self.switch_to_positions_tab():
            self.logger.warning("Could not switch to Positions tab")

    # Extract stats from positions DataGrid
    self._extract_positions_stats(stats)

    # Try to get account balance from top bar or other locations
    self._extract_account_balance(stats)

except Exception as e:
    self.logger.warning(f"Could not extract all stats from positions: {e}")

# Cache the stats WITH TIMESTAMP for performance optimization
self._cached_stats = stats
self._stats_last_fetch_time = time.time()
return stats

except Exception as e:
    self.logger.error(f"Failed to get TopStepX account stats: {e}")
    return {
        "Account Number": "Error",
        "Balance": "Error",
        "Profit/Loss": "Error",

```

```

        "Open Trades": "Error",
        "Symbol": "",
        "Direction": ""
    }
    finally:
        self.lock.release()

```

Method: TopStepXAccount._extract_positions_stats

```
def _extract_positions_stats(self, stats)
```

Extract statistics from the positions DataGrid

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def _extract_positions_stats(self, stats):
    """Extract statistics from the positions DataGrid"""
    try:
        # Count open positions from the DataGrid rows
        position_rows = self.driver.find_elements(By.XPATH, "//div[@role='row'][@data-id]")
        open_trades_count = len(position_rows)
        stats["Open Trades"] = str(open_trades_count)

        if open_trades_count > 0:
            # Get symbol and position info from first position
            try:
                # Extract symbol from first position row
                symbol_cell = self.driver.find_element(By.XPATH,
                    "//div[@role='row'][@data-id][1]//div[@data-field='symbolName']")
                symbol_text = symbol_cell.text.strip()
                if symbol_text:
                    stats["Symbol"] = symbol_text

                # Extract position size to determine direction
                position_cell = self.driver.find_element(By.XPATH,
                    "//div[@role='row'][@data-id][1]//div[@data-field='positionSize']")
                position_size = position_cell.text.strip()
                if position_size:
                    try:
                        size_num = int(position_size)
                        stats["Direction"] = "Long" if size_num > 0 else "Short" if size_num < 0 else ""
                    except:
                        stats["Direction"] = "Long" # Default assumption
            except:
                stats["Direction"] = "Long" # Default assumption
    except:
        stats["Open Trades"] = "Error"
        stats["Symbol"] = ""
        stats["Direction"] = ""

```

```

        # Extract P&L from positions
        pnl_cells = self.driver.find_elements(By.XPATH,
            "//div[@data-field='profitAndLoss']//span")
        total_pnl = 0.0
        for cell in pnl_cells:
            pnl_text = cell.text.strip()
            if '$' in pnl_text:
                try:
                    # Extract numeric value from $-1.00 format
                    pnl_value = float(pnl_text.replace('$', '').replace(',', ''))
                    total_pnl += pnl_value
                except:
                    continue

        stats["Profit/Loss"] = f"${total_pnl:.2f}"

    except Exception as e:
        self.logger.warning(f"Could not extract position details: {e}")

except Exception as e:
    self.logger.warning(f"Could not extract positions stats: {e}")

```

Method: TopStepXAccount._extract_account_balance

```

def _extract_account_balance(self, stats)

    Extract account balance from various UI locations

```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def _extract_account_balance(self, stats):
    """Extract account balance from various UI locations"""
    try:
        # Look for balance in common locations
        balance_selectors = [
            "//span[contains(text(), 'BAL:')]//following-sibling::span",
            "//div[contains(@class, 'balance')]//span[contains(text(), '$')]",
            "//span[contains(text(), '$') and contains(@class, 'balance')]",
            "//div[contains(@aria-label, 'balance')]//span",
            "//span[text()[contains(., 'Balance')]]//following-sibling::span"
        ]

        for selector in balance_selectors:

```



```

try:
    balance_element = self.driver.find_element(By.XPATH, selector)
    balance_text = balance_element.text.strip()
    if '$' in balance_text and any(char.isdigit() for char in balance_text):
        stats["Balance"] = balance_text
        break
except:
    continue

# Try to get account number from HTML element (most reliable)
try:
    self.logger.info(f"[ACCOUNT] Looking for account number in HTML elements...")

    # Target the specific span element containing the account number
    # <div class="MuiBox-root css-8uhtka"><span>...</span><span>|</span><span>50KTC-V2-342449</span></div>
    account_selectors = [
        "//div[contains(@class, 'MuiBox-root')]//span[contains(text(), 'V2-')]", # Span with V2-
        "//span[contains(text(), '50KTC-V2-')]", # Direct span with account pattern
        "//span[contains(text(), 'V2-') and contains(text(), '-')] ", # Any span with V2 pattern
        "//div[contains(@class, 'css-8uhtka')]//span[last()]", # Last span in the specific div
        "//div[contains(@class, 'css-8uhtka')]//span[3]" # Third span in the specific div
    ]

    account_found = False
    for i, selector in enumerate(account_selectors):
        try:
            account_element = self.driver.find_element(By.XPATH, selector)
            account_text = account_element.text.strip()
            self.logger.info(f"[ACCOUNT] Selector {i} found: '{account_text}'")

            if account_text and "V2-" in account_text and "-" in account_text:
                # Format: "50KTC-V2-342449-32181797" -> "V2-...1797"
                account_parts = account_text.split("-")
                self.logger.info(f"[ACCOUNT] Account parts: {account_parts}")
                if len(account_parts) >= 4: # ["50KTC", "V2", "342449", "32181797"]
                    last_part = account_parts[-1] # "32181797"
                    if len(last_part) >= 4:
                        formatted_account = f"V2-...{last_part[-4:]}" # "V2-...1797"
                        stats["Account Number"] = formatted_account
                        self.logger.info(
                            f"■ [ACCOUNT] Formatted TopStepX account number: {formatted_account}"
                        )
                        account_found = True
                        break
        except Exception as selector_error:
            self.logger.debug(f"[ACCOUNT] Selector {i} failed: {selector_error}")
            continue

    if not account_found:
        self.logger.info(f"[ACCOUNT] No account number found in HTML elements")

except Exception as e:
    self.logger.warning(f"Could not extract account from HTML: {e}")

# If title extraction didn't work, try page elements
if stats["Account Number"] == "Unknown":
    self.logger.info("[ACCOUNT] Title extraction failed, trying page elements...")
    account_selectors = [
        "//span[contains(text(), 'Account')]/following-sibling::span",
        "//div[contains(@class, 'account')]/span",
        "//span[contains(@class, 'account-number')]",
    ]

```

```

        "//span[contains(text(), 'Trading Combine')]", # This might be picking up the wrong
        "//span[contains(text(), '50K')]"
    ]

    for i, selector in enumerate(account_selectors):
        try:
            account_element = self.driver.find_element(By.XPATH, selector)
            account_text = account_element.text.strip()
            self.logger.info(f"[ACCOUNT] Selector {i} found: '{account_text}'")

            if account_text and len(account_text) > 3:
                # Skip if this is just descriptive text we don't want
                if account_text in ["$50K TRADING COMBINE", "Trading Combine", "50K"]:
                    self.logger.info(f"[ACCOUNT] Skipping descriptive text: '{account_text}'")
                    continue

                # Format TopStepX account number like other prop firms
                # From "50KTC-V2-342449-32181797" to "V2-...1797"
                if "V2-" in account_text and "-" in account_text:
                    parts = account_text.split("-")
                    # Find V2 part and get last 4 digits of final part
                    v2_index = -1
                    for j, part in enumerate(parts):
                        if part == "V2":
                            v2_index = j
                            break

                    if v2_index >= 0 and len(parts) > v2_index + 1:
                        last_part = parts[-1] # Get the last part (32181797)
                        if len(last_part) >= 4:
                            formatted_account = f"V2-...{last_part[-4:]}" # V2-...1797
                            stats["Account Number"] = formatted_account
                            self.logger.info(
                                f"■ [ACCOUNT] Element-based formatted account: {formatted_account}"
                            )
                            break

                    # Only use as fallback if it contains useful info (not descriptive text)
                    if not any(bad_text in account_text for bad_text in ["TRADING", "COMBINE", "$"]):
                        stats["Account Number"] = account_text
                        self.logger.info(f"[ACCOUNT] Using fallback account text: '{account_text}'")
                        break
        except Exception as e:
            self.logger.debug(f"Selector {i} failed: {e}")
            continue

    # Final check - ensure we have a proper account number
    final_account = stats["Account Number"]
    if final_account == "Unknown":
        stats["Account Number"] = "TopStepX"
        self.logger.info("[ACCOUNT] Using final fallback: TopStepX")

    self.logger.info(f"■ [ACCOUNT] Final account number: '{stats['Account Number']}'")

    except Exception as e:
        self.logger.warning(f"Could not extract account balance: {e}")

```

Method: TopStepXAccount._get_stats_via_api

```
def _get_stats_via_api(self)
```

Get account stats via REST API (fast path, no Selenium needed). Returns stats dict compatible with `get_account_stats()` format, or None on failure.

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns `accounts = self.api_get_trading_accounts()`.
2. `if not accounts or not isinstance(accounts, list) or len(accounts) == 0:` branches conditionally.
3. Assigns `acct = accounts[0]`.
4. Assigns `self._api_account_id = acct.get('accountId')`.
5. Assigns `acct_name = acct.get('accountName', '')`.
6. Assigns `account_number = 'TopStepX'`.
7. `if 'V2-' in acct_name:` branches conditionally.
8. Assigns `balance = acct.get('balance', 0)`.
9. Assigns `realized_pnl = acct.get('realizedDayPnl', 0)`.
10. Assigns `open_pnl = acct.get('openPnl', 0)`.
11. Assigns `total_pnl = realized_pnl + open_pnl`.
12. Assigns `positions = self.api_get_positions()`.
13. Assigns `open_count = len(positions) if positions else 0`.
14. Assigns `symbol = ''`.
15. ... and 3 more statement(s) in the body.

Code (copy-paste safe)

```
def _get_stats_via_api(self):
    """Get account stats via REST API (fast path, no Selenium needed).
    Returns stats dict compatible with get_account_stats() format, or None on failure."""
    accounts = self.api_get_trading_accounts()
    if not accounts or not isinstance(accounts, list) or len(accounts) == 0:
        return None
    acct = accounts[0]
    self._api_account_id = acct.get("accountId")

    # Format account number: "50KTC-V2-342449-32181797" -> "V2-...1797"
    acct_name = acct.get("accountName", "")
    account_number = "TopStepX"
    if "V2-" in acct_name:
        parts = acct_name.split("-")
        if len(parts) >= 4 and len(parts[-1]) >= 4:
            account_number = f"V2-...{parts[-1][-4:]}"
```

```

balance = acct.get("balance", 0)
realized_pnl = acct.get("realizedDayPnl", 0)
open_pnl = acct.get("openPnl", 0)
total_pnl = realized_pnl + open_pnl

# Get positions for open trades count
positions = self.api_get_positions()
open_count = len(positions) if positions else 0

symbol = ""
direction = ""
if open_count > 0 and isinstance(positions, list):
    pos = positions[0]
    symbol = pos.get("contractDisplayName", pos.get("symbolId", ""))
    size = pos.get("positionSize", 0)
    direction = "Long" if size > 0 else "Short" if size < 0 else ""

return {
    "Account Number": account_number,
    "Balance": f"${balance:,.2f}",
    "Profit/Loss": f"${total_pnl:,.2f}",
    "Open Trades": str(open_count),
    "Symbol": symbol,
    "Direction": direction,
}

```

Method: TopStepXAccount.place_order

```

def place_order(self, symbol, quantity, side, order_type='market', price=None, tp_dollars=None,
    sl_dollars=None)

```

Generic method to place orders on TopStepX

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def place_order(self, symbol, quantity, side, order_type="market", price=None, tp_dollars=None,
    sl_dollars=None):
    """Generic method to place orders on TopStepX"""
    try:
        side = side.upper()
        if side == "BUY":
            return self.place_buy_order(symbol, quantity, order_type, tp_dollars, sl_dollars)
        elif side == "SELL":
            return self.place_sell_order(symbol, quantity, order_type, tp_dollars, sl_dollars)
    
```

```

    else:
        raise ValueError(f"Invalid order side: {side}. Must be 'BUY' or 'SELL'")

except Exception as e:
    self.logger.error(f"Failed to place {side} order: {e}")
    return {
        "success": False,
        "message": f"Failed to place {side} order: {e}",
        "platform": "TopStepX"
    }

```

Method: TopStepXAccount.prepare_buy_order

```
def prepare_buy_order(self, symbol, quantity)
```

THREADING OPTIMIZATION: Prepare BUY order (symbol search, quantity setup) WITHOUT clicking button. Use this before threading to minimize time inside parallel execution. Returns: True if preparation successful, False otherwise

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.

Step by step (top-level body)

1. **with self.lock:** enters a context manager.

Code (copy-paste safe)

```

def prepare_buy_order(self, symbol, quantity):
    """
    THREADING OPTIMIZATION: Prepare BUY order (symbol search, quantity setup) WITHOUT clicking button
    Use this before threading to minimize time inside parallel execution.

    Returns: True if preparation successful, False otherwise
    """
    with self.lock:
        try:
            if not self.is_connected():
                raise Exception("Not connected to TopStepX")

            self.logger.info(f"[■ PRE-THREAD] Preparing TopStepX BUY order: {symbol} x{quantity}")

            # Ensure we're on the Order tab
            if not self.switch_to_order_tab():
                self.logger.warning("Could not switch to Order tab")
                return False

            # Step 1: Set contract symbol (SLOW - do this before threading)

```

```

self.logger.info(f"■ PRE-THREAD] Setting symbol: {symbol}")
if not self._set_contract_symbol(symbol):
    self.logger.error(f"Failed to set contract symbol: {symbol}")
    return False

# Step 2: Set quantity (SLOW - do this before threading)
self.logger.info(f"■ PRE-THREAD] Setting quantity: {quantity}")
if not self._set_quantity(quantity):
    self.logger.error(f"Failed to set quantity: {quantity}")
    return False

self.logger.info(f"■ [■ PRE-THREAD] Order prepared - ready for button click")
return True

except Exception as e:
    self.logger.error(f"■ PRE-THREAD] Preparation failed: {e}")
    return False

```

Method: TopStepXAccount.execute_prepared_buy_order

```
def execute_prepared_buy_order(self)
```

THREADING OPTIMIZATION: Execute already-prepared BUY order (just click button). Call prepare_buy_order() first, then use this inside threading for instant execution. Returns: Order result dict

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.

Step by step (top-level body)

1. **with self.lock:** enters a context manager.

Code (copy-paste safe)

```

def execute_prepared_buy_order(self):
    """
    THREADING OPTIMIZATION: Execute already-prepared BUY order (just click button).
    Call prepare_buy_order() first, then use this inside threading for instant execution.

    Returns: Order result dict
    """
    with self.lock:
        try:
            self.logger.info(f"■ THREAD-EXEC] Clicking BUY button...")

            # Just click the button - everything else is already prepared
            if not self._click_buy_button(skip_post_trade_setup=True):
                raise Exception("Failed to click BUY button")

```

```

self.logger.info(f"■ [■ THREAD-EXEC] BUY button clicked")

order_id = f"TSX_BUY_{int(time.time())}"
return {
    "success": True,
    "message": "TopStepX BUY order executed",
    "order_id": order_id,
    "platform": "TopStepX",
    "side": "BUY"
}

except Exception as e:
    self.logger.error(f"■ [■ THREAD-EXEC] Execution failed: {e}")
    return {
        "success": False,
        "message": f"TopStepX BUY execution failed: {e}",
        "platform": "TopStepX"
    }

```

Method: TopStepXAccount.prepare_sell_order

```
def prepare_sell_order(self, symbol, quantity)
```

THREADING OPTIMIZATION: Prepare SELL order (symbol search, quantity setup) WITHOUT clicking button. Use this before threading to minimize time inside parallel execution. Returns: True if preparation successful, False otherwise

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.

Step by step (top-level body)

1. **with self.lock:** enters a context manager.

Code (copy-paste safe)

```

def prepare_sell_order(self, symbol, quantity):
    """
    THREADING OPTIMIZATION: Prepare SELL order (symbol search, quantity setup) WITHOUT clicking button
    Use this before threading to minimize time inside parallel execution.

    Returns: True if preparation successful, False otherwise
    """
    with self.lock:
        try:
            if not self.is_connected():
                raise Exception("Not connected to TopStepX")

```

```

self.logger.info(f"■ PRE-THREAD] Preparing TopStepX SELL order: {symbol} x{quantity}")

# Ensure we're on the Order tab
if not self.switch_to_order_tab():
    self.logger.warning("Could not switch to Order tab")
    return False

# Step 1: Set contract symbol (SLOW - do this before threading)
self.logger.info(f"■ PRE-THREAD] Setting symbol: {symbol}")
if not self._set_contract_symbol(symbol):
    self.logger.error(f"Failed to set contract symbol: {symbol}")
    return False

# Step 2: Set quantity (SLOW - do this before threading)
self.logger.info(f"■ PRE-THREAD] Setting quantity: {quantity}")
if not self._set_quantity(quantity):
    self.logger.error(f"Failed to set quantity: {quantity}")
    return False

self.logger.info(f"■ [■ PRE-THREAD] Order prepared - ready for button click")
return True

except Exception as e:
    self.logger.error(f"■ PRE-THREAD] Preparation failed: {e}")
    return False

```

Method: TopStepXAccount.execute_prepared_sell_order

```
def execute_prepared_sell_order(self)
```

THREADING OPTIMIZATION: Execute already-prepared SELL order (just click button). Call prepare_sell_order() first, then use this inside threading for instant execution. Returns: Order result dict

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.

Step by step (top-level body)

1. **with self.lock:** enters a context manager.

Code (copy-paste safe)

```

def execute_prepared_sell_order(self):
    """
    THREADING OPTIMIZATION: Execute already-prepared SELL order (just click button).
    Call prepare_sell_order() first, then use this inside threading for instant execution.

    Returns: Order result dict

```



```

"""
with self.lock:
    try:
        self.logger.info(f"[■ THREAD-EXEC] Clicking SELL button...")

        # Just click the button - everything else is already prepared
        if not self._click_sell_button(skip_post_trade_setup=True):
            raise Exception("Failed to click SELL button")

        self.logger.info(f"[■ THREAD-EXEC] SELL button clicked")

        order_id = f"TSX_SELL_{int(time.time())}"
        return {
            "success": True,
            "message": "TopStepX SELL order executed",
            "order_id": order_id,
            "platform": "TopStepX",
            "side": "SELL"
        }

    except Exception as e:
        self.logger.error(f"[■ THREAD-EXEC] Execution failed: {e}")
        return {
            "success": False,
            "message": f"TopStepX SELL execution failed: {e}",
            "platform": "TopStepX"
        }

```

Method: TopStepXAccount.place_buy_order

```

def place_buy_order(self, symbol, quantity, order_type='market', tp_dollars=None, sl_dollars=None,
    skip_post_trade_setup=False)

```

Place a BUY order on TopStepX with optional post-trade TP/SL setup Args: symbol: Trading symbol (e.g., 'NQM25') quantity: Number of contracts order_type: Order type (default: 'market') tp_dollars: Take profit in dollars sl_dollars: Stop loss in dollars skip_post_trade_setup: If True, skip TP/SL editing in Positions tab. Caller should manually call setup_post_trade_tp_sl_positions() later. Use this in hedging mode to place MT5 trade first for speed.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.

Step by step (top-level body)

1. with self.lock: enters a context manager.

Code (copy-paste safe)

```
def place_buy_order(self, symbol, quantity, order_type="market", tp_dollars=None, sl_dollars=None,
                    skip_post_trade_setup=False):
    """
    Place a BUY order on TopStepX with optional post-trade TP/SL setup

    Args:
        symbol: Trading symbol (e.g., 'NQM25')
        quantity: Number of contracts
        order_type: Order type (default: 'market')
        tp_dollars: Take profit in dollars
        sl_dollars: Stop loss in dollars
        skip_post_trade_setup: If True, skip TP/SL editing in Positions tab.
                               Caller should manually call setup_post_trade_tp_sl_positions() later.
                               Use this in hedging mode to place MT5 trade first for speed.
    """
    # Acquire lock to prevent stats fetching during order placement
    with self.lock:
        try:
            # TIME CHECKPOINT: Start overall timing
            order_start_time = time.time()

            # Dismiss any MUI backdrop overlay before interacting with UI
            self._dismiss_backdrop()

            if not self.is_connected():
                raise Exception("Not connected to TopStepX")

            if skip_post_trade_setup:
                self.logger.info(
                    f"■ [FAST MODE] Placing TopStepX BUY order: {symbol} x{quantity} (TP/SL setup de
            else:
                self.logger.info(f"Placing TopStepX BUY order: {symbol} x{quantity}")

            # Ensure we're on the Order tab for trade entry
            if not self.switch_to_order_tab():
                self.logger.warning("Could not switch to Order tab, attempting trade anyway")

            # MANDATORY: Setup TP/SL values are REQUIRED
            if tp_dollars is None and sl_dollars is None:
                raise Exception("■ TRADE BLOCKED: TP and SL values are REQUIRED before placing any t

            if tp_dollars is None or sl_dollars is None:
                self.logger.warning(f"■ Missing TP or SL: TP=${tp_dollars}, SL=${sl_dollars}")
                # Set default values if only one is missing
                if tp_dollars is None:
                    tp_dollars = 500 # Default TP: $500
                    self.logger.warning(f"■ Using default TP: ${tp_dollars}")
                if sl_dollars is None:
                    sl_dollars = 1000 # Default SL: $1000
                    self.logger.warning(f"■ Using default SL: ${sl_dollars}")

            # Enhanced execution with detailed logging
            self.logger.info(f"[TRADE] Starting TopStepX BUY order execution sequence")
            self.logger.info(
                f"[PARAMS] Symbol: {symbol}, Quantity: {quantity}, TP: ${tp_dollars}, SL: ${sl_dollar

            # Set contract symbol with detailed logging
```

```

self.logger.info(f"[STEP 1] Setting contract symbol: {symbol}")
symbol_result = self._set_contract_symbol(symbol)
if not symbol_result:
    error_msg = f"Failed to set contract symbol: {symbol}"
    self.logger.error(f"■ [STEP 1] {error_msg}")
    raise Exception(error_msg)
else:
    self.logger.info(f"■ [STEP 1] Contract symbol set successfully: {symbol}")

# Set quantity with detailed logging
self.logger.info(f"[STEP 2] Setting quantity: {quantity}")
quantity_result = self._set_quantity(quantity)
if not quantity_result:
    error_msg = f"Failed to set quantity: {quantity}"
    self.logger.error(f"■ [STEP 2] {error_msg}")
    raise Exception(error_msg)
else:
    self.logger.info(f"■ [STEP 2] Quantity set successfully: {quantity}")

# OPTIMIZED: In FAST mode, skip everything and go straight to BUY button
if skip_post_trade_setup:
    # FAST MODE: Go directly from quantity → BUY button
    self.logger.info(f"■ [FAST MODE] Skipping order type, TP/SL - going straight to BUY")
else:
    # NORMAL MODE: Set order type if not market
    if order_type.lower() != "market":
        self.logger.info(f"[STEP 3] Setting order type: {order_type}")
        if not self._set_order_type(order_type):
            self.logger.warning(
                f"■ [STEP 3] Failed to set order type to {order_type}, using market order"
            )
        else:
            self.logger.info(f"■ [STEP 3] Order type set: {order_type}")
    else:
        self.logger.info(f"[STEP 3] Using default market order type")

# OPTIMIZED: In FAST mode, TP/SL already skipped above
if not skip_post_trade_setup:
    # NORMAL MODE: Set TP/SL on order form before placing trade
    _orderform_tp_ok = False
    _orderform_sl_ok = False
    if tp_dollars and tp_dollars > 0:
        self.logger.info(f"[STEP 4] Setting Take Profit: ${tp_dollars}")
        if not self._set_take_profit_price(tp_dollars):
            self.logger.warning(f"■ [STEP 4] Failed to set Take Profit on order form")
        else:
            self.logger.info(f"■ [STEP 4] Take Profit set: ${tp_dollars}")
            _orderform_tp_ok = True
    else:
        self.logger.info(f"[STEP 4] Skipping Take Profit (tp_dollars={tp_dollars})")
        _orderform_tp_ok = True # Nothing to do counts as success

    if sl_dollars and sl_dollars > 0:
        self.logger.info(f"[STEP 5] Setting Stop Loss: ${sl_dollars}")
        if not self._set_stop_loss_price(sl_dollars):
            self.logger.warning(f"■ [STEP 5] Failed to set Stop Loss on order form")
        else:
            self.logger.info(f"■ [STEP 5] Stop Loss set: ${sl_dollars}")
            _orderform_sl_ok = True
    else:
        self.logger.info(f"[STEP 5] Skipping Stop Loss (sl_dollars={sl_dollars})")

```

```

        _orderform_sl_ok = True

# Click BUY button with detailed logging
self.logger.info(f"[STEP 6] Clicking BUY button")
buy_result = self._click_buy_button(skip_post_trade_setup=skip_post_trade_setup)
if not buy_result:
    error_msg = "Failed to click BUY button"
    self.logger.error(f"■ [STEP 6] {error_msg}")
    raise Exception(error_msg)
else:
    self.logger.info(f"■ [STEP 6] BUY button clicked successfully")

# OPTIMIZED: Minimal wait reduced from 0.3s to 0.1s
time.sleep(0.1)

# CONDITIONAL: Skip post-trade setup if requested (for hedging mode speed)
if skip_post_trade_setup:
    # TIME CHECKPOINT: End timing for fast mode
    order_end_time = time.time()
    total_order_time_ms = (order_end_time - order_start_time) * 1000

    self.logger.info("■ FAST MODE: Skipping post-trade TP/SL setup for hedging mode")
    self.logger.info(
        "■ Caller should call setup_post_trade_tp_sl_positions() after MT5 hedge placement"
    )
    self.logger.info(f"■■ Total order placement time: {total_order_time_ms:.1f}ms")

    # Capture snapshot if order placement took too long
    self._capture_delay_snapshot("fast_order_placement", total_order_time_ms,
                                threshold_ms=1000)

    order_id = f"TSX_BUY_{int(time.time())}"
    self.logger.info(f"■ TopStepX BUY order submitted (fast mode): {symbol} x{quantity}")

    return {
        "success": True,
        "message": f"TopStepX BUY order placed (fast mode): {symbol} x{quantity}",
        "order_id": order_id,
        "platform": "TopStepX",
        "symbol": symbol,
        "quantity": quantity,
        "side": "BUY",
        "tp_dollars": tp_dollars, # Return these for later setup
        "sl_dollars": sl_dollars,
        "placement_time_ms": total_order_time_ms
    }

# NORMAL MODE: Continue with TP/SL setup as before
self.logger.info(f"[POST-TRADE] Verifying TP/SL setup")

# SPEED: If both TP and SL were already set on the order form, the broker
# attached them to the working order – skip the redundant Positions-tab pass
# entirely (saves ~15-25s of grid wait + field edits per trade).
_need_positions_setup = (
    ((tp_dollars and tp_dollars > 0) and not _orderform_tp_ok) or
    ((sl_dollars and sl_dollars > 0) and not _orderform_sl_ok)
)

if _need_positions_setup:
    # Switch to positions tab to verify trade and setup TP/SL
    if self.switch_to_positions_tab():

```

```

        self.logger.info("■ Switched to Positions tab for TP/SL fallback")
    else:
        self.logger.warning("■ Could not switch to Positions tab")
        setup_result = self.setup_post_trade_tp_sl_positions(tp_dollars, sl_dollars)
        if not setup_result:
            self.logger.warning(
                "■■ TP/SL setup in positions failed - trade placed but risk not managed")
        else:
            self.logger.info("■ TP/SL setup completed in positions")
    else:
        self.logger.info("■ Skipping Positions-tab TP/SL setup (already set on order form)")

order_id = f"TSX_BUY_{int(time.time())}"
self.logger.info(f"TopStepX BUY order submitted: {symbol} x{quantity}")

# POST-TRADE VERIFICATION: Check what was actually executed (DELAYED)
# Delay this check significantly to avoid interrupting trade processing
actual_executed_symbol = None
try:
    self.logger.info("[POST-TRADE] Scheduling delayed verification in 3 seconds...")
    # Use a separate thread or delayed execution to avoid blocking
    import threading

    def delayed_verification():
        try:
            time.sleep(3) # Wait longer for trade to fully process
            # Check positions table for the actual executed symbol
            stats = self.get_account_stats()
            if stats and stats.get("Symbol"):
                executed_symbol = stats["Symbol"]
                self.logger.info(f"[POST-TRADE] Actual executed symbol: '{executed_symbol}'")

            # Critical verification
            if symbol.upper() in ['NQM25', 'NQM26'] and executed_symbol:
                if 'MNQ' in executed_symbol.upper() and symbol.upper(
                    ) not in executed_symbol.upper():
                    self.logger.error(f"■ CRITICAL MISMATCH DETECTED:")
                    self.logger.error(f"    Requested: {symbol} (full contract)")
                    self.logger.error(f"    Executed: {executed_symbol} (micro contract)")
                    self.logger.error(
                        f"    Result: 5x smaller position size than intended!")
                elif executed_symbol.upper() == symbol.upper():
                    self.logger.info(
                        f"■ VERIFIED: Correct contract executed - {executed_symbol}")
                else:
                    self.logger.warning(
                        f"■■ Unexpected executed symbol: {executed_symbol}")
        except Exception as verify_error:
            self.logger.warning(f"Delayed post-trade verification failed: {verify_error}")

    # Start verification in background thread to avoid blocking
    verification_thread = threading.Thread(target=delayed_verification, daemon=True)
    verification_thread.start()

except Exception as verify_error:
    self.logger.warning(f"Could not start delayed verification: {verify_error}")

result_dict = {
    "success": True,
    "message": f"TopStepX BUY order placed: {symbol} x{quantity}",

```

```

        "order_id": order_id,
        "platform": "TopStepX",
        "symbol": symbol,
        "executed_symbol": actual_executed_symbol or symbol,
        "quantity": quantity,
        "side": "BUY"
    }
    return result_dict

except Exception as e:
    self.logger.error(f"TopStepX BUY order failed: {e}")
    return {
        "success": False,
        "message": f"TopStepX BUY order failed: {e}",
        "platform": "TopStepX"
    }

```

Method: TopStepXAccount.place_sell_order

```
def place_sell_order(self, symbol, quantity, order_type='market', tp_dollars=None, sl_dollars=None,
    skip_post_trade_setup=False)
```

Place a SELL order on TopStepX with optional post-trade TP/SL setup Args: *symbol: Trading symbol (e.g., 'NQM25') quantity: Number of contracts order_type: Order type (default: 'market') tp_dollars: Take profit in dollars sl_dollars: Stop loss in dollars skip_post_trade_setup: If True, skip TP/SL editing in Positions tab. Caller should manually call setup_post_trade_tp_sl_positions() later. Use this in hedging mode to place MT5 trade first for speed.*

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.

Step by step (top-level body)

1. **with self.lock:** enters a context manager.

Code (copy-paste safe)

```
def place_sell_order(self, symbol, quantity, order_type="market", tp_dollars=None, sl_dollars=None,
    skip_post_trade_setup=False):
    """
    Place a SELL order on TopStepX with optional post-trade TP/SL setup

    Args:
        symbol: Trading symbol (e.g., 'NQM25')
        quantity: Number of contracts
        order_type: Order type (default: 'market')
        tp_dollars: Take profit in dollars

```

```

sl_dollars: Stop loss in dollars
skip_post_trade_setup: If True, skip TP/SL editing in Positions tab.
                        Caller should manually call setup_post_trade_tp_sl_positions() later.
                        Use this in hedging mode to place MT5 trade first for speed.
"""
# Acquire lock to prevent stats fetching during order placement
with self.lock:
    try:
        # TIME CHECKPOINT: Start overall timing
        order_start_time = time.time()

        # Dismiss any MUI backdrop overlay before interacting with UI
        self._dismiss_backdrop()

        if not self.is_connected():
            raise Exception("Not connected to TopStepX")

        if skip_post_trade_setup:
            self.logger.info(
                f"■ [FAST MODE] Placing TopStepX SELL order: {symbol} x{quantity} (TP/SL setup d
            else:
                self.logger.info(f"Placing TopStepX SELL order: {symbol} x{quantity}")

        # Ensure we're on the Order tab for trade entry
        if not self.switch_to_order_tab():
            self.logger.warning("Could not switch to Order tab, attempting trade anyway")

        # MANDATORY: Setup TP/SL values are REQUIRED
        if tp_dollars is None and sl_dollars is None:
            raise Exception("■ TRADE BLOCKED: TP and SL values are REQUIRED before placing any t

        if tp_dollars is None or sl_dollars is None:
            self.logger.warning(f"■ Missing TP or SL: TP=${tp_dollars}, SL=${sl_dollars}")
            # Set default values if only one is missing
            if tp_dollars is None:
                tp_dollars = 500 # Default TP: $500
                self.logger.warning(f"■ Using default TP: ${tp_dollars}")
            if sl_dollars is None:
                sl_dollars = 1000 # Default SL: $1000
                self.logger.warning(f"■ Using default SL: ${sl_dollars}")

        # Set contract symbol - re-enabled for exact test replication
        self.logger.info(f"■ [EXACT-TEST-SELL] Setting contract symbol: {symbol}")
        if not self._set_contract_symbol(symbol):
            raise Exception(f"Failed to set contract symbol: {symbol}")

        # Set quantity
        self.logger.info(f"[STEP 2] Setting quantity: {quantity}")
        if not self._set_quantity(quantity):
            raise Exception(f"Failed to set quantity: {quantity}")
        else:
            self.logger.info(f"■ [STEP 2] Quantity set successfully: {quantity}")

        # OPTIMIZED: In FAST mode, skip everything and go straight to SELL button
        if skip_post_trade_setup:
            # FAST MODE: Go directly from quantity → SELL button
            self.logger.info(f"■ [FAST MODE] Skipping order type, TP/SL - going straight to SELL
        else:
            # NORMAL MODE: Set order type if not market
            if order_type.lower() != "market":

```

```

        if not self._set_order_type(order_type):
            self.logger.warning(
                f"Failed to set order type to {order_type}, using market order")

# OPTIMIZED: In FAST mode, TP/SL already skipped above
_orderform_tp_ok = False
_orderform_sl_ok = False
if not skip_post_trade_setup:
    # NORMAL MODE: Set TP/SL on order form before placing trade
    if tp_dollars and tp_dollars > 0:
        self.logger.info(f"Setting Take Profit: ${tp_dollars}")
        if not self._set_take_profit_price(tp_dollars):
            self.logger.warning(f"Failed to set Take Profit on order form")
        else:
            self.logger.info(f"Take Profit set: ${tp_dollars}")
            _orderform_tp_ok = True
    else:
        _orderform_tp_ok = True

    if sl_dollars and sl_dollars > 0:
        self.logger.info(f"Setting Stop Loss: ${sl_dollars}")
        if not self._set_stop_loss_price(sl_dollars):
            self.logger.warning(f"Failed to set Stop Loss on order form")
        else:
            self.logger.info(f"Stop Loss set: ${sl_dollars}")
            _orderform_sl_ok = True
    else:
        _orderform_sl_ok = True

# Click SELL button
if not self._click_sell_button(skip_post_trade_setup=skip_post_trade_setup):
    raise Exception("Failed to click SELL button")

# Minimal wait for order processing
time.sleep(0.1)

# CONDITIONAL: Skip post-trade setup if requested (for hedging mode speed)
if skip_post_trade_setup:
    # TIME CHECKPOINT: End timing for fast mode
    order_end_time = time.time()
    total_order_time_ms = (order_end_time - order_start_time) * 1000

    self.logger.info("■ FAST MODE: Skipping post-trade TP/SL setup for hedging mode")
    self.logger.info(
        "■ Caller should call setup_post_trade_tp_sl_positions() after MT5 hedge placement")
    self.logger.info(f"■■ Total order placement time: {total_order_time_ms:.1f}ms")

# Capture snapshot if order placement took too long
self._capture_delay_snapshot("fast_sell_order_placement", total_order_time_ms,
                             threshold_ms=1000)

order_id = f"TSX_SELL_{int(time.time())}"
self.logger.info(f"■ TopStepX SELL order submitted (fast mode): {symbol} x{quantity}")

return {
    "success": True,
    "message": f"TopStepX SELL order placed (fast mode): {symbol} x{quantity}",
    "order_id": order_id,
    "platform": "TopStepX",
    "symbol": symbol,

```



```

        "quantity": quantity,
        "side": "SELL",
        "tp_dollars": tp_dollars, # Return these for later setup
        "sl_dollars": sl_dollars,
        "placement_time_ms": total_order_time_ms
    }

# NORMAL MODE: Continue with TP/SL setup as before
self.logger.info(f"[POST-TRADE] Verifying TP/SL setup")

# SPEED: skip the redundant Positions-tab pass when order form already attached TP/SL
_need_positions_setup = (
    ((tp_dollars and tp_dollars > 0) and not _orderform_tp_ok) or
    ((sl_dollars and sl_dollars > 0) and not _orderform_sl_ok)
)

if _need_positions_setup:
    if self.switch_to_positions_tab():
        self.logger.info("■ Switched to Positions tab for TP/SL fallback")
    else:
        self.logger.warning("■ Could not switch to Positions tab")
    setup_result = self.setup_post_trade_tp_sl_positions(tp_dollars, sl_dollars)
    if not setup_result:
        self.logger.warning(
            "TP/SL setup in positions failed - trade placed but risk not managed")
    else:
        self.logger.info("TP/SL setup completed in positions")
else:
    self.logger.info("■ Skipping Positions-tab TP/SL setup (already set on order form)")

order_id = f"TSX_SELL_{int(time.time())}"
self.logger.info(f"TopStepX SELL order submitted: {symbol} x{quantity}")

return {
    "success": True,
    "message": f"TopStepX SELL order placed: {symbol} x{quantity}",
    "order_id": order_id,
    "platform": "TopStepX",
    "symbol": symbol,
    "quantity": quantity,
    "side": "SELL"
}

except Exception as e:
    self.logger.error(f"TopStepX SELL order failed: {e}")
    return {
        "success": False,
        "message": f"TopStepX SELL order failed: {e}",
        "platform": "TopStepX"
    }
}

```

Method: TopStepXAccount.close_all_positions

```
def close_all_positions(self)
```

Close all open positions on TopStepX

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def close_all_positions(self):
    """Close all open positions on TopStepX"""
    try:
        if not self.is_connected():
            raise Exception("Not connected to TopStepX")

        self.logger.info("Closing all TopStepX positions")

        # Navigate to trading page if not already there
        self._ensure_on_trading_page()

        # Click "Flatten All" button to close all positions
        if not self._click_flatten_all_button():
            self.logger.warning("Failed to click Flatten All button, trying alternative methods")

            # Alternative: Check for individual position close buttons
            if not self._close_individual_positions():
                raise Exception("Failed to close positions using any method")

        # Wait for positions to close
        time.sleep(2)

        self.logger.info("All TopStepX positions closed successfully")
        return {
            "success": True,
            "message": "All TopStepX positions closed",
            "platform": "TopStepX"
        }

    except Exception as e:
        self.logger.error(f"Failed to close TopStepX positions: {e}")
        return {
            "success": False,
            "message": f"Failed to close TopStepX positions: {e}",
            "platform": "TopStepX"
        }
```

Method: TopStepXAccount.get_positions

```
def get_positions(self)
```

Get current positions from TopStepX

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def get_positions(self):
    """Get current positions from TopStepX"""
    try:
        if not self.is_connected():
            raise Exception("Not connected to TopStepX")

        self._ensure_on_trading_page()

        # Navigate to positions tab if needed
        positions_tab_selectors = [
            "//div[contains(text(), 'Positions')][@role='tab']",
            "//button[contains(text(), 'Positions')]",
            "//tab[contains(text(), 'Position')]"
        ]

        for selector in positions_tab_selectors:
            try:
                positions_tab = self.driver.find_element(By.XPATH, selector)
                positions_tab.click()
                time.sleep(1)
                break
            except:
                continue

        positions = []

        # Look for position data in the grid
        try:
            # Check if there are any positions
            no_positions_elements = self.driver.find_elements(By.XPATH,
                "//*[contains(text(), 'No Positions') or contains(text(), 'No Active Position')]")

            if no_positions_elements:
                self.logger.info("No positions found on TopStepX")
                return positions

            # Try to extract position data from the grid
            position_rows = self.driver.find_elements(By.XPATH,
                "//div[@role='grid']/div[@role='row'][position()>1]") # Skip header row

            for row in position_rows:
```

```

try:
    cells = row.find_elements(By.XPATH, ".//div[@role='gridcell']")
    if len(cells) >= 4: # Ensure we have enough data
        position = {
            "symbol": cells[1].text.strip() if len(cells) > 1 else "N/A",
            "quantity": cells[2].text.strip() if len(cells) > 2 else "0",
            "side": "LONG" if "+" in cells[2].text else "SHORT" if "-" in cells[
                2].text else "N/A",
            "entry_price": cells[3].text.strip() if len(cells) > 3 else "N/A",
            "pnl": cells[6].text.strip() if len(cells) > 6 else "N/A",
            "platform": "TopStepX"
        }
        positions.append(position)
    except Exception as e:
        self.logger.warning(f"Failed to parse position row: {e}")
        continue

except Exception as e:
    self.logger.warning(f"Failed to extract position data: {e}")

self.logger.info(f"Retrieved {len(positions)} positions from TopStepX")
return positions

except Exception as e:
    self.logger.error(f"Failed to get TopStepX positions: {e}")
    return []

```

Method: TopStepXAccount.get_orders

```
def get_orders(self)
```

Get current pending orders from TopStepX

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def get_orders(self):
    """Get current pending orders from TopStepX"""
    try:
        if not self.is_connected():
            raise Exception("Not connected to TopStepX")

```

```

self._ensure_on_trading_page()

# Navigate to orders tab
orders_tab_selectors = [
    "//div[contains(text(), 'Orders')][@role='tab']",
    "//button[contains(text(), 'Orders')]",
    "//tab[contains(text(), 'Order')]"
]

for selector in orders_tab_selectors:
    try:
        orders_tab = self.driver.find_element(By.XPATH, selector)
        orders_tab.click()
        time.sleep(1)
        break
    except:
        continue

orders = []

# Look for order data in the grid
try:
    # Check if there are any orders
    no_orders_elements = self.driver.find_elements(By.XPATH,
        "//*[contains(text(), 'No Orders') or contains(text(), 'No Pending Orders')]")

    if no_orders_elements:
        self.logger.info("No pending orders found on TopStepX")
        return orders

    # Try to extract order data from the grid
    order_rows = self.driver.find_elements(By.XPATH,
        "//div[@role='grid']/div[@role='row'][position()>1]") # Skip header row

    for row in order_rows:
        try:
            cells = row.find_elements(By.XPATH, ".//div[@role='gridcell']")
            if len(cells) >= 4:
                order = {
                    "symbol": cells[1].text.strip() if len(cells) > 1 else "N/A",
                    "quantity": cells[2].text.strip() if len(cells) > 2 else "0",
                    "side": "BUY" if "Buy" in cells[3].text else "SELL" if "Sell" in cells[
                        3].text else "N/A",
                    "order_type": cells[4].text.strip() if len(cells) > 4 else "N/A",
                    "price": cells[5].text.strip() if len(cells) > 5 else "N/A",
                    "status": cells[6].text.strip() if len(cells) > 6 else "PENDING",
                    "platform": "TopStepX"
                }
                orders.append(order)
            except Exception as e:
                self.logger.warning(f"Failed to parse order row: {e}")
                continue

        except Exception as e:
            self.logger.warning(f"Failed to extract order data: {e}")

    self.logger.info(f"Retrieved {len(orders)} orders from TopStepX")
    return orders

except Exception as e:

```

```
self.logger.error(f"Failed to get TopStepX orders: {e}")
return []
```

Method: TopStepXAccount.cancel_all_orders

```
def cancel_all_orders(self)
```

Cancel all pending orders on TopStepX

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def cancel_all_orders(self):
    """Cancel all pending orders on TopStepX"""
    try:
        if not self.is_connected():
            raise Exception("Not connected to TopStepX")

        self.logger.info("Cancelling all TopStepX orders")

        # Navigate to trading page
        self._ensure_on_trading_page()

        # Click "Cancel All" button
        cancel_all_selectors = [
            "//button[contains(text(), 'Cancel All')]",
            "//button[contains(text(), 'CANCEL ALL')]"
        ]

        cancel_button = None
        for selector in cancel_all_selectors:
            try:
                cancel_button = WebDriverWait(self.driver, 5).until(
                    EC.element_to_be_clickable((By.XPATH, selector))
                )
                break
            except TimeoutException:
                continue

        if cancel_button:
            cancel_button.click()
            self.logger.info("Cancel All button clicked")

            # Handle confirmation if needed
            time.sleep(1)
```

```

        self._handle_order_confirmation()

        return {
            "success": True,
            "message": "All TopStepX orders cancelled",
            "platform": "TopStepX"
        }
    else:
        self.logger.warning("Cancel All button not found or not enabled")
        return {
            "success": False,
            "message": "Cancel All button not available",
            "platform": "TopStepX"
        }

except Exception as e:
    self.logger.error(f"Failed to cancel TopStepX orders: {e}")
    return {
        "success": False,
        "message": f"Failed to cancel TopStepX orders: {e}",
        "platform": "TopStepX"
    }

```

Method: TopStepXAccount.edit_position_tp_sl

```
def edit_position_tp_sl(self, symbol=None, tp_dollars=None, sl_dollars=None)
```

Edit Take Profit and Stop Loss for existing positions on TopStepX Args: *symbol*: Symbol to edit (optional, if None edits first position) *tp_dollars*: Take Profit amount in dollars *sl_dollars*: Stop Loss amount in dollars

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def edit_position_tp_sl(self, symbol=None, tp_dollars=None, sl_dollars=None):
    """
    Edit Take Profit and Stop Loss for existing positions on TopStepX
    Args:
        symbol: Symbol to edit (optional, if None edits first position)
        tp_dollars: Take Profit amount in dollars
        sl_dollars: Stop Loss amount in dollars
    """
    try:
        if not self.is_connected():

```

```

        raise Exception("Not connected to TopStepX")

    self.logger.info(f"Editing TP/SL for TopStepX position: TP=${tp_dollars}, SL=${sl_dollars}")

    # Navigate to trading page
    self._ensure_on_trading_page()

    # Navigate to Positions tab
    if not self._navigate_to_positions_tab():
        raise Exception("Failed to navigate to Positions tab")

    # Find the position row to edit
    position_row = self._find_position_row(symbol)
    if not position_row:
        raise Exception(f"Position not found for symbol: {symbol or 'any'}")

    # Edit Take Profit if specified
    if tp_dollars is not None:
        if self._edit_position_tp(position_row, tp_dollars):
            self.logger.info(f"Successfully set TP to ${tp_dollars}")
        else:
            self.logger.warning(f"Failed to set TP to ${tp_dollars}")

    # Edit Stop Loss if specified
    if sl_dollars is not None:
        if self._edit_position_sl(position_row, sl_dollars):
            self.logger.info(f"Successfully set SL to ${sl_dollars}")
        else:
            self.logger.warning(f"Failed to set SL to ${sl_dollars}")

    return {
        "success": True,
        "message": f"TopStepX TP/SL updated: TP=${tp_dollars}, SL=${sl_dollars}",
        "platform": "TopStepX"
    }

except Exception as e:
    self.logger.error(f"Failed to edit TopStepX TP/SL: {e}")
    return {
        "success": False,
        "message": f"Failed to edit TopStepX TP/SL: {e}",
        "platform": "TopStepX"
    }

```

Method: TopStepXAccount._navigate_to_positions_tab

```
def _navigate_to_positions_tab(self)
```

Navigate to the Positions tab using TopStepX-specific selectors

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. try block with 1 except clause.

Code (copy-paste safe)

```
def _navigate_to_positions_tab(self):
    """Navigate to the Positions tab using TopStepX-specific selectors"""
    try:
        self._dismiss_backdrop()
        # Look for Positions tab using the exact HTML structure provided
        positions_tab_selectors = [
            # Primary selector based on provided HTML - targets the clickable tab button
            "//div[@role='tab' and contains(@id, 'positionTab')]",
            "//div[@class='dock-tab-btn' and contains(@aria-controls, 'positionTab')]",
            # Secondary selector - targets the drag initiator with Positions text
            "//div[@class='drag-initiator' and @role='tab' and contains(text(), 'Positions')]",
            # Fallback selectors for different UI variations
            "//div[contains(@class, 'dock-tab') and .//text()[contains(., 'Positions')]]",
            "//div[contains(text(), 'Positions')][@role='tab']",
            "//button[contains(text(), 'Positions')]",
            # Generic fallback
            "//div[contains(@class, 'tab') and contains(text(), 'Positions')]"
        ]

        self.logger.info("Attempting to navigate to Positions tab...")

        for i, selector in enumerate(positions_tab_selectors, 1):
            try:
                self.logger.debug(f"Trying selector {i}: {selector}")
                positions_tab = WebDriverWait(self.driver, 5).until(
                    EC.presence_of_element_located((By.XPATH, selector))
                )

                # JS click to bypass any backdrop overlay
                self.driver.execute_script("arguments[0].click();", positions_tab)
                time.sleep(1.5) # Allow tab content to load

                # Verify we're on the positions tab by checking for positions content
                try:
                    # Look for positions grid or "No Positions" message
                    WebDriverWait(self.driver, 3).until(
                        EC.any_of(
                            EC.presence_of_element_located((By.XPATH, "//div[@role='grid']")),
                            EC.presence_of_element_located((By.XPATH,
                                "//*[contains(text(), 'No Positions') or contains(text(), 'No Active
                                "
                            ))
                        )
                    )
                    self.logger.info(f"✓ Successfully navigated to Positions tab using selector {i}")
                    return True
                except TimeoutException:
                    self.logger.warning(f"Tab clicked but positions content not found with selector {i}")
                    continue

            except TimeoutException:
                self.logger.debug(f"Selector {i} not found: {selector}")
                continue
            except Exception as e:
                self.logger.warning(f"Error with selector {i}: {e}")
                continue

        self.logger.error("Could not find or click Positions tab with any selector")
```

```

        return False

    except Exception as e:
        self.logger.error(f"Failed to navigate to Positions tab: {e}")
        return False

```

Method: TopStepXAccount._dismiss_backdrop

```
def _dismiss_backdrop(self)
```

Dismiss any MUI backdrop/modal overlay that blocks clicks on the trading page.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def _dismiss_backdrop(self):
    """Dismiss any MUI backdrop/modal overlay that blocks clicks on the trading page."""
    try:
        self.driver.execute_script("""
            // Click all backdrop overlays to dismiss them
            document.querySelectorAll('.MuiBackdrop-root').forEach(el => el.click());
            // Also try pressing Escape to close any open popover/modal
            document.dispatchEvent(new KeyboardEvent('keydown', {key: 'Escape', bubbles: true}));
        """)
        time.sleep(0.3)
    except Exception:
        pass

```

Method: TopStepXAccount._navigate_to_trading_tab

```
def _navigate_to_trading_tab(self)
```

Navigate to the Trading tab using TopStepX-specific selectors

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def _navigate_to_trading_tab(self):
    """Navigate to the Trading tab using TopStepX-specific selectors"""

```

```

try:
    self._dismiss_backdrop()
    # Look for Trading tab using similar structure to positions tab
    trading_tab_selectors = [
        # Primary selector - targets the trading tab button
        "//*[role='tab' and contains(@id, 'trading')]",
        "//*[class='dock-tab-btn' and contains(@aria-controls, 'trading')]",
        # Secondary selector - targets tab with Trading text
        "//*[class='drag-initiator' and @role='tab' and contains(text(), 'Trading')]",
        # Fallback selectors for different UI variations
        "//*[contains(@class, 'dock-tab') and .//text()[contains(., 'Trading')]]",
        "//*[contains(text(), 'Trading')][@role='tab']",
        "//*[button[contains(text(), 'Trading')]]",
        # Generic fallback
        "//*[contains(@class, 'tab') and contains(text(), 'Trading')]"
    ]

    self.logger.info("Attempting to navigate to Trading tab...")

    for i, selector in enumerate(trading_tab_selectors, 1):
        try:
            self.logger.debug(f"Trying selector {i}: {selector}")
            trading_tab = WebDriverWait(self.driver, 5).until(
                EC.presence_of_element_located((By.XPATH, selector))
            )

            # JS click to bypass any backdrop overlay
            self.driver.execute_script("arguments[0].click();", trading_tab)
            time.sleep(1.5) # Allow tab content to load

            # Verify we're on the trading tab by checking for trading interface elements
            try:
                # Look for Buy/Sell buttons or order form
                WebDriverWait(self.driver, 3).until(
                    EC.any_of(
                        EC.presence_of_element_located((By.XPATH,
                            "//*[button[contains(text(), 'Buy') or contains(text(), 'BUY')]]")),
                        EC.presence_of_element_located((By.XPATH,
                            "//*[input[@placeholder='Quantity' or contains(@aria-label, 'Quantity')]]"))
                    )
                )
                self.logger.info(f"✓ Successfully navigated to Trading tab using selector {i}")
                return True
            except TimeoutException:
                self.logger.warning(f"Tab clicked but trading content not found with selector {i}")
                continue

        except TimeoutException:
            self.logger.debug(f"Selector {i} not found: {selector}")
            continue
        except Exception as e:
            self.logger.warning(f"Error with selector {i}: {e}")
            continue

    self.logger.error("Could not find or click Trading tab with any selector")
    return False

except Exception as e:
    self.logger.error(f"Failed to navigate to Trading tab: {e}")
    return False

```

Method: TopStepXAccount._click_order_tab

```
def _click_order_tab(self)
```

Click the Order tab to access order editing fields after placing an order

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def _click_order_tab(self):
    """Click the Order tab to access order editing fields after placing an order"""
    try:
        self.logger.info("■ Attempting to click Order tab...")

        # Simplified selectors based on actual HTML structure
        order_tab_selectors = [
            # Most specific - targets the exact button with orderCardTab ID
            "//*[@role='tab' and contains(@id, 'orderCardTab')]",

            # Targets dock-tab-btn with Order text inside
            "//*[@class='dock-tab-btn' and .//span[text()='Order']]",

            # Any tab role containing Order text
            "//*[@role='tab' and contains(., 'Order') and not(contains(., 'Positions'))]",

            # Broadest fallback
            "//*[contains(@class, 'dock-tab') and .//text()='Order']"
        ]

        for i, selector in enumerate(order_tab_selectors, 1):
            try:
                self.logger.info(f"[ATTEMPT {i}] Trying selector: {selector}")

                # Try to find the element
                order_tab = WebDriverWait(self.driver, 3).until(
                    EC.presence_of_element_located((By.XPATH, selector))
                )

                self.logger.info(f"✓ Element found with selector {i}")

                # Try multiple click methods
                click_success = False

                # Method 1: Standard click
                try:
                    order_tab.click()
                    self.logger.info(f"✓ Standard click succeeded")
                    click_success = True
```

```

except Exception as e:
    self.logger.warning(f"Standard click failed: {e}")

# Method 2: JavaScript click (more reliable)
if not click_success:
    try:
        self.driver.execute_script("arguments[0].click();", order_tab)
        self.logger.info(f"✓ JavaScript click succeeded")
        click_success = True
    except Exception as e:
        self.logger.warning(f"JavaScript click failed: {e}")

# Method 3: Action chains click
if not click_success:
    try:
        from selenium.webdriver.common.action_chains import ActionChains
        ActionChains(self.driver).move_to_element(order_tab).click().perform()
        self.logger.info(f"✓ ActionChains click succeeded")
        click_success = True
    except Exception as e:
        self.logger.warning(f"ActionChains click failed: {e}")

if click_success:
    self.logger.info(f"■ Order tab clicked successfully with selector {i}")
    time.sleep(1.5) # Allow tab to activate
    return True
else:
    self.logger.warning(f"All click methods failed for selector {i}")
    continue

except TimeoutException:
    self.logger.debug(f"Selector {i} not found within timeout")
    continue
except Exception as e:
    self.logger.warning(f"Error with selector {i}: {e}")
    continue

# If all selectors failed, log available tabs for debugging
self.logger.error(f"■ Could not click Order tab with any selector")
try:
    all_tabs = self.driver.find_elements(By.XPATH, "//div[@role='tab']")
    self.logger.info(f"Available tabs ({len(all_tabs)}):")
    for idx, tab in enumerate(all_tabs):
        tab_text = tab.text.strip() or tab.get_attribute('id') or 'Unknown'
        self.logger.info(f"  Tab {idx+1}: {tab_text}")
except:
    pass

return False

except Exception as e:
    self.logger.error(f"Failed to click Order tab: {e}")
    import traceback
    self.logger.error(traceback.format_exc())
    return False

```

Method: TopStepXAccount.switch_tab

```
def switch_tab(self, tab_name)
```

Generic method to switch between tabs in the TopStepX interface OPTIMIZED: Fast tab switching with reduced timeouts Args: *tab_name (str): Name of the tab to switch to. Options: - 'Positions' - View open positions and P&L - 'Order' - Access order entry form - 'Trades' - View trade history - 'Quotes' - View market quotes - 'Accounts' - View account information - 'Chart' - View charts* Returns: *bool: True if tab switch was successful, False otherwise*

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def switch_tab(self, tab_name):
    """
    Generic method to switch between tabs in the TopStepX interface
    OPTIMIZED: Fast tab switching with reduced timeouts

    Args:
        tab_name (str): Name of the tab to switch to. Options:
            - 'Positions' - View open positions and P&L
            - 'Order' - Access order entry form
            - 'Trades' - View trade history
            - 'Quotes' - View market quotes
            - 'Accounts' - View account information
            - 'Chart' - View charts

    Returns:
        bool: True if tab switch was successful, False otherwise
    """
    try:
        self._dismiss_backdrop()
        # Build selectors based on tab name (prioritize fastest/most reliable)
        selectors = [
            # Strategy 1: By exact text match (fastest)
            f"//div[@role='tab']//div[text()='{{tab_name}}']",
            # Strategy 2: By tab ID containing the tab name
            f"//div[@role='tab' and contains(@id, '{{tab_name.lower()}}')]",
            # Strategy 3: By partial text match
            f"//div[@role='tab' and contains(., '{{tab_name}}')]"
        ]

        for selector in selectors:
            try:
                # FAST: Reduced timeout from 3s to 0.5s per selector
                tab_element = WebDriverWait(self.driver, 0.5).until(
                    EC.element_to_be_clickable((By.XPATH, selector))
                )

                # FAST: Use only JS click for speed
                self.driver.execute_script("arguments[0].click();", tab_element)
```

```

        time.sleep(0.1) # Minimal wait - reduced from 0.5s
        return True

    except TimeoutException:
        continue
    except Exception:
        continue

    return False

except Exception as e:
    self.logger.error(f"Failed to switch to '{tab_name}' tab: {e}")
    return False

```

Method: TopStepXAccount.switch_to_positions_tab

```
def switch_to_positions_tab(self)
```

Switch to the Positions tab to view open positions OPTIMIZED: Fast switch with minimal verification

Returns: bool: True if successful, False otherwise

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def switch_to_positions_tab(self):
    """
    Switch to the Positions tab to view open positions
    OPTIMIZED: Fast switch with minimal verification

    Returns:
        bool: True if successful, False otherwise
    """
    try:
        # Switch to Positions tab
        if not self.switch_tab("Positions"):
            return False

        # FAST: Minimal wait - reduced from 3s to 0.5s
        try:
            WebDriverWait(self.driver, 0.5).until(
                EC.presence_of_element_located((By.XPATH,
                    "//div[@role='grid' or contains(@class, 'MuiDataGrid')]"))
            )
        except TimeoutException:
            pass # Continue anyway

    return True

```

```

except Exception as e:
    self.logger.error(f"Failed to switch to Positions tab: {e}")
    return False

```

Method: TopStepXAccount.switch_to_order_tab

```
def switch_to_order_tab(self)
```

Switch to the Order tab to access order entry form Returns: bool: True if successful, False otherwise

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def switch_to_order_tab(self):
    """
    Switch to the Order tab to access order entry form

    Returns:
        bool: True if successful, False otherwise
    """
    try:
        # Switch to Order tab
        if not self.switch_tab("Order"):
            self.logger.warning("Could not switch to Order tab")
            return False

        # Verify Buy button or quantity input is visible after switch
        try:
            WebDriverWait(self.driver, 3).until(
                EC.any_of(
                    EC.presence_of_element_located((By.XPATH,
                                                    "//button[contains(text(), 'Buy') or contains(text(), 'BUY')]")),
                    EC.presence_of_element_located((By.XPATH,
                                                    "//input[@placeholder='Quantity' or contains(@aria-label, 'Quantity')]"))
                )
            )
            self.logger.debug("Verified order form is visible")
            return True
        except TimeoutException:
            self.logger.warning("Order tab switched but form not visible")
            return True # Still return True as tab switch succeeded

    except Exception as e:
        self.logger.error(f"Failed to switch to Order tab: {e}")
        return False

```

Method: TopStepXAccount.get_active_tab


```
def get_active_tab(self)
```

Detect which tab is currently active Returns: str: Name of active tab, or None if cannot determine

Example: current_tab = self.get_active_tab() if current_tab == "Positions": # Already on positions tab
pass

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def get_active_tab(self):
    """
    Detect which tab is currently active

    Returns:
        str: Name of active tab, or None if cannot determine

    Example:
        current_tab = self.get_active_tab()
        if current_tab == "Positions":
            # Already on positions tab
            pass
    """
    try:
        # Find element with class 'dock-tab-active'
        active_tab = self.driver.find_element(
            By.XPATH,
            "//*[@div[contains(@class, 'dock-tab-active')]]"
        )

        # Extract tab name from text content
        tab_text = active_tab.text.strip()

        # Normalize tab name (remove extra characters, emojis, etc.)
        # The Order tab may have emoji and "H" for hotkey
        if "Order" in tab_text:
            return "Order"
        elif "Position" in tab_text:
            return "Positions"
        elif "Trade" in tab_text and "Trader" not in tab_text:
            return "Trades"
        elif "Quote" in tab_text:
            return "Quotes"
        elif "Account" in tab_text:
            return "Accounts"
        elif "Chart" in tab_text:
            return "Chart"
        else:
            # Return the cleaned text
```

```

        return tab_text

    except Exception as e:
        self.logger.debug(f"Could not detect active tab: {e}")
        return None

```

Method: TopStepXAccount._find_position_row

```
def _find_position_row(self, symbol=None)
```

Find the position row in the positions grid

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **list comprehension** — Builds a list in a single expression ([f(x) for x in xs]). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def _find_position_row(self, symbol=None):
    """Find the position row in the positions grid"""
    try:
        # Wait for positions grid to load after tab switch
        self.logger.info("Waiting for positions grid to load...")
        time.sleep(2)

        # Check if there are no positions first
        no_positions_elements = self.driver.find_elements(By.XPATH,
            '//*[contains(text(), 'No Positions') or contains(text(), 'No Active Position') or contains(text(), 'No Positions Found')]')
        if no_positions_elements:
            self.logger.warning("No positions found on TopStepX - positions grid is empty")
            return None

        # Look for position rows in the MUI DataGrid with multiple selector patterns
        position_row_selectors = [
            # MUI DataGrid specific selectors (most likely)
            "//*[contains(@class, 'MuiDataGrid-main')]/div[@role='row'][position()>1]", # MUI DataGrid
            "//*[contains(@class, 'MuiDataGrid-main')]/div[@role='grid']/div[@role='row'][position()>1]", # Standard grid rows (skip header)
            "//*[contains(@class, 'MuiDataGrid-row')]", # MUI DataGrid row class
            "//*[contains(@class, 'grid')]/div[contains(@class, 'row')][position()>1]", # Alternative grid rows
            "//*[table/tr[position()>1]]", # Table-based positions
            "//*[div[@role='rowgroup']]/div[@role='row']" # Row group structure
        ]

        position_rows = []
        for selector in position_row_selectors:
            try:
                rows = self.driver.find_elements(By.XPATH, selector)
                if rows:
                    return rows[0].text
            except:
                pass
    except:
        return None

```

```

        position_rows = rows
        self.logger.info(f"Found {len(rows)} position rows using selector: {selector}")
        break
    except Exception as e:
        self.logger.debug(f"Selector failed: {selector} - {e}")
        continue

if not position_rows:
    self.logger.warning("No position rows found in grid")
    return None

# If no specific symbol, return first position
if symbol is None:
    self.logger.info(f"Using first available position (total: {len(position_rows)})")
    return position_rows[0]

# Look for position with matching symbol
self.logger.info(f"Searching for position with symbol: {symbol}")
for i, row in enumerate(position_rows):
    try:
        # Try different cell selection methods
        cell_selectors = [
            ".*//div[@role='gridcell']",
            ".*//div[contains(@class, 'cell')]",
            ".*//td"
        ]

        cells = []
        for cell_selector in cell_selectors:
            cells = row.find_elements(By.XPATH, cell_selector)
            if cells:
                break

        if len(cells) > 1:
            # Check multiple potential symbol columns (usually column 1 or 2)
            for col_idx in [1, 2, 0]: # Try column 1, then 2, then 0
                if col_idx < len(cells):
                    row_symbol = cells[col_idx].text.strip()
                    if row_symbol and symbol.upper() in row_symbol.upper():
                        self.logger.info(
                            f"✓ Found position row {i+1} for symbol: {row_symbol} (column {col_idx})"
                        )
                        return row

            # Log what we found for debugging
            cell_contents = [cell.text.strip() for cell in cells[:5]] # First 5 columns
            self.logger.debug(f"Row {i+1} contents: {cell_contents}")

    except Exception as e:
        self.logger.warning(f"Failed to check row {i+1} symbol: {e}")
        continue

self.logger.warning(f"Position not found for symbol: {symbol}")
# Log available positions for debugging
try:
    self.logger.info("Available positions:")
    for i, row in enumerate(position_rows[:3]): # Show first 3 rows
        cells = row.find_elements(By.XPATH,
            ".*//div[@role='gridcell'] | .*//div[contains(@class, 'cell')] | .*//td")
        if cells:
            cell_contents = [cell.text.strip() for cell in cells[:3]]

```

```

        self.logger.info(f" Row {i+1}: {cell_contents}")
    except:
        pass

    return None

except Exception as e:
    self.logger.error(f"Failed to find position row: {e}")
    return None

```

Method: TopStepXAccount._edit_position_tp

```
def _edit_position_tp(self, position_row, tp_dollars)

    Edit Take Profit by clicking the 'To Make' column (data-field='toMake')
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def _edit_position_tp(self, position_row, tp_dollars):
    """Edit Take Profit by clicking the 'To Make' column (data-field='toMake')"""
    try:
        # Find the 'To Make' cell using the exact HTML structure provided
        # Look for MUI DataGrid cell with data-field="toMake"
        to_make_cell_selectors = [
            # Primary selector: exact data-field match for "toMake"
            ".*div[@role='cell' and @data-field='toMake']",
            # Secondary: MUI DataGrid cell with toMake field
            ".*div[contains(@class, 'MuiDataGrid-cell') and @data-field='toMake']",
            # Fallback: cell containing the pencil icon and $ value in toMake context
            ".*div[@role='cell' and contains(@data-field, 'toMake')]",
            # Generic fallback: look for editable cell with $ and pencil icon
            ".*div[@role='cell' and contains(@class, 'editable') and .//span[contains(text(), '$')]]"
        ]

        to_make_cell = None
        for i, selector in enumerate(to_make_cell_selectors, 1):
            try:
                to_make_cell = position_row.find_element(By.XPATH, selector)
                self.logger.info(f"✓ Found 'To Make' cell using selector {i}: {selector}")
                break
            except:
                self.logger.debug(f"Selector {i} not found: {selector}")
                continue

        if not to_make_cell:
            self.logger.error("Could not find 'To Make' cell with any selector")
            return False
    
```

```

# Click the pencil icon or the cell itself to activate editing
pencil_icon_selectors = [
    # Click the pencil icon specifically
    ".*//svg[@data-icon='pencil']",
    # Click the span containing the $ value
    ".*//span[contains(@class, 'css-') and contains(text(), '$')]",
    # Click the cell itself if no pencil found
    "."
]

clicked = False
for selector in pencil_icon_selectors:
    try:
        click_target = to_make_cell.find_element(By.XPATH, selector)
        self.logger.info(f"Clicking TP edit target: {selector}")
        ActionChains(self.driver).click(click_target).perform()
        clicked = True
        break
    except:
        continue

if not clicked:
    # Fallback: double-click the cell directly
    ActionChains(self.driver).double_click(to_make_cell).perform()

time.sleep(0.5) # Reduced from 1.5s

# Look for input field that appears after clicking
input_field = None

# First try to get the currently focused element (most reliable)
try:
    active_element = self.driver.switch_to.active_element
    if active_element and active_element.tag_name == 'input':
        current_value = active_element.get_attribute("value")
        if not (current_value and ("1000" in str(current_value) or len(str(current_value).replace(".", "")) > 6)):
            input_field = active_element
except:
    pass

# If no focused input found, try our selectors
if not input_field:
    input_selectors = [
        # First try to find input within the clicked cell
        ".*//input[@type='text']",
        ".*//input[@type='number']",
        ".*//input[contains(@class, 'MuiInput')]",
        ".*//input",
        # Then try to find input in the modal/dialog that might appear
        ".*//div[contains(@class, 'MuiDialog') or contains(@class, 'modal')].*//input[@type='text']",
        ".*//div[contains(@class, 'MuiDialog') or contains(@class, 'modal')].*//input[@type='number']",
        # Try any visible input field (but be careful this might find wrong field)
        ".*//input[@type='text' and not(@disabled)]",
        ".*//input[@type='number' and not(@disabled)]"
    ]

    for selector in input_selectors:
        try:
            if selector.startswith(".*//"):

```

```

        # Search within the To Make cell
        input_field = to_make_cell.find_element(By.XPATH, selector)
    else:
        # Search globally
        input_field = WebDriverWait(self.driver, 3).until(
            EC.element_to_be_clickable((By.XPATH, selector))
        )
        self.logger.info(f"Found TP input field: {selector}")
        break
    except (TimeoutException, Exception):
        continue

if input_field:
    # Clear and enter the TP value (FAST - edit only ONCE)
    input_field.click()
    input_field.send_keys(Keys.CONTROL + "a")
    input_field.send_keys(Keys.DELETE)
    input_field.send_keys(str(tp_dollars))
    input_field.send_keys(Keys.ENTER)
    time.sleep(0.3) # Reduced from 0.5s
    return True
else:
    self.logger.error("Could not find input field for TP edit")
    return False

except Exception as e:
    self.logger.error(f"Failed to edit position TP: {e}")
    return False

```

Method: TopStepXAccount._edit_position_sl

```

def _edit_position_sl(self, position_row, sl_dollars)
    Edit Stop Loss by clicking the 'Risk' column (data-field='risk')

```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def _edit_position_sl(self, position_row, sl_dollars):
    """Edit Stop Loss by clicking the 'Risk' column (data-field='risk')"""
    try:
        # Find the 'Risk' cell using the exact HTML structure provided
        # Look for MUI DataGrid cell with data-field="risk"
        risk_cell_selectors = [
            # Primary selector: exact data-field match for "risk"
            "._div[@role='cell' and @data-field='risk']",
            # Secondary: MUI DataGrid cell with risk field
            "._div[contains(@class, 'MuiDataGrid-cell') and @data-field='risk']",

```

```

        # Fallback: cell containing the pencil icon and $ value in risk context
        ".//div[@role='cell' and contains(@data-field, 'risk')]",
        # Generic fallback: look for editable cell with $ and pencil icon (not toMake)
        ".//div[@role='cell' and contains(@class, 'editable') and .//span[contains(text(), '$')]]"
    ]

    risk_cell = None
    for i, selector in enumerate(risk_cell_selectors, 1):
        try:
            risk_cell = position_row.find_element(By.XPATH, selector)
            self.logger.info(f"✓ Found 'Risk' cell using selector {i}: {selector}")
            break
        except:
            self.logger.debug(f"Selector {i} not found: {selector}")
            continue

    if not risk_cell:
        self.logger.error("Could not find 'Risk' cell with any selector")
        return False

    # Click the pencil icon or the cell itself to activate editing
    pencil_icon_selectors = [
        # Click the pencil icon specifically
        ".//svg[@data-icon='pencil']",
        # Click the span containing the $ value
        ".//span[contains(@class, 'css-') and contains(text(), '$')]",
        # Click the cell itself if no pencil found
        "."
    ]

    clicked = False
    for selector in pencil_icon_selectors:
        try:
            click_target = risk_cell.find_element(By.XPATH, selector)
            self.logger.info(f"Clicking SL edit target: {selector}")
            ActionChains(self.driver).click(click_target).perform()
            clicked = True
            break
        except:
            continue

    if not clicked:
        # Fallback: double-click the cell directly
        self.logger.info("Fallback: Double-clicking 'Risk' cell directly")
        ActionChains(self.driver).double_click(risk_cell).perform()

    time.sleep(1.5)

    # Look for input field that appears after clicking
    # Priority: Find the currently focused/active input field first
    input_field = None

    # First try to get the currently focused element (most reliable)
    try:
        active_element = self.driver.switch_to.active_element
        if active_element and active_element.tag_name == 'input':
            # Validate this is not a contract quantity field
            current_value = active_element.get_attribute("value")
            if not (current_value and ("1000" in str(current_value) or len(str(current_value).replace(".", "")) > 6)):

```

```

        input_field = active_element
        self.logger.info("✓ Using focused input field as SL target")
    except Exception as e:
        self.logger.debug(f"Could not get active element: {e}")

    # If no focused input found, try our selectors
    if not input_field:
        input_selectors = [
            # First try to find input within the clicked cell
            "//*[@type='text']",
            "//*[@type='number']",
            "//*[@contains(@class, 'MuiInput')]",
            "//*[@input",
            # Then try to find input in the modal/dialog that might appear
            "//div[contains(@class, 'MuiDialog') or contains(@class, 'modal')]/input[@type='text'",
            "//div[contains(@class, 'MuiDialog') or contains(@class, 'modal')]/input[@type='numb",
            # Try any visible input field (but be careful this might find wrong field)
            "//input[@type='text' and not(@disabled)]",
            "//input[@type='number' and not(@disabled)]"
        ]

        for selector in input_selectors:
            try:
                if selector.startswith("//*[@"):
                    # Search within the Risk cell
                    input_field = risk_cell.find_element(By.XPATH, selector)
                else:
                    # Search globally
                    input_field = WebDriverWait(self.driver, 3).until(
                        EC.element_to_be_clickable((By.XPATH, selector))
                    )
                self.logger.info(f"Found SL input field: {selector}")
                break
            except (TimeoutException, Exception):
                continue

        if input_field:
            # Clear and enter the SL value (FAST - edit only ONCE)
            input_field.click()
            input_field.send_keys(Keys.CONTROL + "a")
            input_field.send_keys(Keys.DELETE)
            input_field.send_keys(str(sl_dollars))
            input_field.send_keys(Keys.ENTER)
            time.sleep(0.3) # Reduced from 0.5s
            return True
        else:
            self.logger.error("Could not find input field for SL edit")
            return False

    except Exception as e:
        self.logger.error(f"Failed to edit position SL: {e}")
        return False

```

Method: TopStepXAccount.setup_post_trade_tp_sl_positions

```
def setup_post_trade_tp_sl_positions(self, tp_dollars, sl_dollars)
```

Setup TP/SL in the positions section AFTER trade is placed OPTIMIZED: Edits each field exactly once with no retries for speed

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def setup_post_trade_tp_sl_positions(self, tp_dollars, sl_dollars):
    """
    Setup TP/SL in the positions section AFTER trade is placed
    OPTIMIZED: Edits each field exactly once with no retries for speed
    """
    try:
        self.logger.info(f"[TP/SL-SETUP] Starting - TP: ${tp_dollars}, SL: ${sl_dollars}")

        # CRITICAL: Switch to Positions tab first
        self.logger.info("[TP/SL-SETUP] Switching to Positions tab...")
        if not self.switch_to_positions_tab():
            self.logger.error("[TP/SL-SETUP] ■ Failed to switch to Positions tab")
            return False

        self.logger.info("[TP/SL-SETUP] ■ On Positions tab")

        # Wait for positions grid to load - Increased wait for reliability
        time.sleep(1.5) # Increased from 0.3s to 1.5s to ensure grid is fully interactive
        self.logger.info("[TP/SL-SETUP] Grid load wait completed")

        # Track if we've successfully edited each field
        sl_edited = False
        tp_edited = False

        # Edit Risk field (SL) - SINGLE EDIT, NO RETRIES
        if sl_dollars and sl_dollars > 0:
            self.logger.info(f"[TP/SL-SETUP] Editing SL (Risk) field: ${sl_dollars}")
            sl_edited = self._edit_sl_field_fast(sl_dollars)
            if sl_edited:
                self.logger.info("[TP/SL-SETUP] ■ SL edited successfully")
            else:
                self.logger.error("[TP/SL-SETUP] ■ SL edit failed")

        # Edit To Make field (TP) - SINGLE EDIT, NO RETRIES
        if tp_dollars and tp_dollars > 0:
            self.logger.info(f"[TP/SL-SETUP] Editing TP (To Make) field: ${tp_dollars}")
            tp_edited = self._edit_tp_field_fast(tp_dollars)
            if tp_edited:
                self.logger.info("[TP/SL-SETUP] ■ TP edited successfully")
            else:
                self.logger.error("[TP/SL-SETUP] ■ TP edit failed")

        # Determine overall success
        if tp_dollars and sl_dollars:
            success = tp_edited and sl_edited
```

```

        self.logger.info(
            f"[TP/SL-SETUP] Final result - TP: {tp_edited}, SL: {sl_edited}, Overall: {success}")
    elif tp_dollars:
        success = tp_edited
        self.logger.info(f"[TP/SL-SETUP] Final result - TP only: {tp_edited}")
    elif sl_dollars:
        success = sl_edited
        self.logger.info(f"[TP/SL-SETUP] Final result - SL only: {sl_edited}")
    else:
        success = True # Nothing to edit
        self.logger.info("[TP/SL-SETUP] No TP/SL values to edit")

    return success

except Exception as e:
    self.logger.error(f"[TP/SL-SETUP] ■ Exception during TP/SL setup: {e}")
    import traceback
    self.logger.error(traceback.format_exc())
    return False

```

Method: TopStepXAccount._edit_sl_field_fast

```
def _edit_sl_field_fast(self, sl_dollars)
```

FAST: Edit SL (Risk) field directly without searching for position row

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns `max_retries = 3`.
2. **for** `attempt in range(max_retries)`: iterates.
3. Calls `self.logger.error(...)` for its side effect.
4. **return** `False`.

Code (copy-paste safe)

```

def _edit_sl_field_fast(self, sl_dollars):
    """FAST: Edit SL (Risk) field directly without searching for position row"""
    max_retries = 3
    for attempt in range(max_retries):
        try:
            self.logger.info(
                f"[FAST-SL] Starting SL edit (Attempt {attempt+1}/{max_retries}): ${sl_dollars}")

            # CRITICAL: We need to click on the actual dollar VALUE, not the pencil icon
            # The pencil is just an indicator - clicking the $ value activates editing
            value_selectors = [

```

```

        # Click the span with the dollar value inside the Risk cell
        "//div[@data-field='risk']//span[contains(text(), '$')]",
        # Or click anywhere in the Risk cell that's editable
        "//div[@data-field='risk' and contains(@class, 'MuiDataGrid-cell--editable')]",
        # Or just the Risk cell itself
        "//div[@data-field='risk' and @role='cell']",
    ]

    clicked = False
    for i, selector in enumerate(value_selectors, 1):
        try:
            value_element = WebDriverWait(self.driver, 1.0).until(
                EC.element_to_be_clickable((By.XPATH, selector))
            )
            self.logger.info(f"[FAST-SL] Found Risk value element with selector {i}")

            # Double-click to activate editing (common pattern for editable grids)
            from selenium.webdriver.common.action_chains import ActionChains
            ActionChains(self.driver).double_click(value_element).perform()
            self.logger.info("[FAST-SL] Double-clicked Risk value")
            clicked = True
            break
        except Exception as e:
            self.logger.debug(f"[FAST-SL] Selector {i} failed: {e}")
            continue

    if not clicked:
        self.logger.warning(f"[FAST-SL] Attempt {attempt+1}: Could not find or click Risk value")
        time.sleep(0.5)
        continue

    time.sleep(0.5) # Wait for input to appear

    # Use active element (most reliable)
    input_field = self.driver.switch_to.active_element
    self.logger.info(
        f"[FAST-SL] Active element after double-click: {input_field.tag_name if input_field else None}"
    )

    if input_field and input_field.tag_name == 'input':
        self.logger.info(f"[FAST-SL] Found input field, typing ${sl_dollars}")
        # Clear field and type new value
        input_field.send_keys(Keys.CONTROL + "a")
        input_field.send_keys(Keys.DELETE)
        input_field.send_keys(str(sl_dollars))
        input_field.send_keys(Keys.ENTER)
        time.sleep(0.2) # Minimal wait
        self.logger.info("[FAST-SL] ■ SL edit completed")
        return True
    else:
        self.logger.warning(
            f"[FAST-SL] Attempt {attempt+1}: Active element is not an input: {input_field.tag_name if input_field else None}"
        )
        # Try to find input field manually
        try:
            input_field = WebDriverWait(self.driver, 1.0).until(
                EC.presence_of_element_located((By.XPATH, "//div[@data-field='risk']/input"))
            )
            self.logger.info("[FAST-SL] Found input field manually")
            input_field.send_keys(Keys.CONTROL + "a")
            input_field.send_keys(Keys.DELETE)
            input_field.send_keys(str(sl_dollars))
        except:
            self.logger.warning(f"[FAST-SL] Attempt {attempt+1}: Could not find input field manually")
            continue

```

```

        input_field.send_keys(Keys.ENTER)
        time.sleep(0.2)
        self.logger.info("[FAST-SL] ■ SL edit completed (manual input find)")
        return True
    except Exception as e:
        self.logger.error(f"[FAST-SL] Could not find input manually: {e}")

    # If we got here, something failed but didn't raise exception
    time.sleep(0.5)

except Exception as e:
    self.logger.error(f"[FAST-SL] ■ Attempt {attempt+1} Exception: {e}")
    time.sleep(0.5)

self.logger.error(f"[FAST-SL] Failed to edit SL after {max_retries} attempts")
return False

```

Method: TopStepXAccount._edit_tp_field_fast

```
def _edit_tp_field_fast(self, tp_dollars)
```

FAST: Edit TP (To Make) field directly without searching for position row

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns `max_retries = 3`.
2. `for attempt in range(max_retries):` iterates.
3. Calls `self.logger.error(...)` for its side effect.
4. `return False`.

Code (copy-paste safe)

```

def _edit_tp_field_fast(self, tp_dollars):
    """FAST: Edit TP (To Make) field directly without searching for position row"""
    max_retries = 3
    for attempt in range(max_retries):
        try:
            self.logger.info(
                f"[FAST-TP] Starting TP edit (Attempt {attempt+1}/{max_retries}): ${tp_dollars}"
            )

            # CRITICAL: We need to click on the actual dollar VALUE, not the pencil icon
            # The pencil is just an indicator - clicking the $ value activates editing
            value_selectors = [
                # Click the span with the dollar value inside the To Make cell
                "//div[@data-field='toMake']//span[contains(text(), '$')]",
                # Or click anywhere in the To Make cell that's editable
            ]

```

```

        "//div[@data-field='toMake' and contains(@class, 'MuiDataGrid-cell--editable')]",
        # Or just the To Make cell itself
        "//div[@data-field='toMake' and @role='cell']",
    ]

    clicked = False
    for i, selector in enumerate(value_selectors, 1):
        try:
            value_element = WebDriverWait(self.driver, 1.0).until(
                EC.element_to_be_clickable((By.XPATH, selector))
            )
            self.logger.info(f"[FAST-TP] Found To Make value element with selector {i}")

            # Double-click to activate editing (common pattern for editable grids)
            from selenium.webdriver.common.action_chains import ActionChains
            ActionChains(self.driver).double_click(value_element).perform()
            self.logger.info("[FAST-TP] Double-clicked To Make value")
            clicked = True
            break
        except Exception as e:
            self.logger.debug(f"[FAST-TP] Selector {i} failed: {e}")
            continue

    if not clicked:
        self.logger.warning(
            f"[FAST-TP] Attempt {attempt+1}: Could not find or click To Make value"
        )
        time.sleep(0.5)
        continue

    time.sleep(0.5) # Wait for input to appear

    # Use active element (most reliable)
    input_field = self.driver.switch_to.active_element
    self.logger.info(
        f"[FAST-TP] Active element after double-click: {input_field.tag_name if input_field}"
    )

    if input_field and input_field.tag_name == 'input':
        self.logger.info(f"[FAST-TP] Found input field, typing ${tp_dollars}")
        # Clear field and type new value
        input_field.send_keys(Keys.CONTROL + "a")
        input_field.send_keys(Keys.DELETE)
        input_field.send_keys(str(tp_dollars))
        input_field.send_keys(Keys.ENTER)
        time.sleep(0.2) # Minimal wait
        self.logger.info("[FAST-TP] ■ TP edit completed")
        return True
    else:
        self.logger.warning(
            f"[FAST-TP] Attempt {attempt+1}: Active element is not an input: {input_field.tag_name}"
        )
        # Try to find input field manually
        try:
            input_field = WebDriverWait(self.driver, 1.0).until(
                EC.presence_of_element_located((By.XPATH, "//div[@data-field='toMake']//input"))
            )
            self.logger.info("[FAST-TP] Found input field manually")
            input_field.send_keys(Keys.CONTROL + "a")
            input_field.send_keys(Keys.DELETE)
            input_field.send_keys(str(tp_dollars))
            input_field.send_keys(Keys.ENTER)
            time.sleep(0.2)
        except:
            self.logger.warning(f"[FAST-TP] Manual find failed")

```

```

        self.logger.info("[FAST-TP] ■ TP edit completed (manual input find)")
        return True
    except Exception as e:
        self.logger.error(f"[FAST-TP] Could not find input manually: {e}")

    # If we got here, something failed but didn't raise exception
    time.sleep(0.5)

    except Exception as e:
        self.logger.error(f"[FAST-TP] ■ Attempt {attempt+1} Exception: {e}")
        time.sleep(0.5)

    self.logger.error(f"[FAST-TP] Failed to edit TP after {max_retries} attempts")
    return False

```

Method: TopStepXAccount.setup_inline_tp_sl

```
def setup_inline_tp_sl(self, tp_dollars, sl_dollars)
```

Set TP/SL using inline editable fields in the data grid (NEW METHOD) This replaces the old gear button modal approach OPTIMIZED: First checks if values are already correct before attempting edits

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def setup_inline_tp_sl(self, tp_dollars, sl_dollars):
    """
    Set TP/SL using inline editable fields in the data grid (NEW METHOD)
    This replaces the old gear button modal approach
    OPTIMIZED: First checks if values are already correct before attempting edits
    """
    try:
        self.logger.info(f"■ Setting up inline TP/SL - SL: ${sl_dollars}, TP: ${tp_dollars}")

        # Set Risk (SL) field if sl_dollars > 0
        if sl_dollars > 0:
            # FIRST: Check if Risk field is already set to the correct value
            self.logger.info(f"[CHECK] Checking if Risk field is already set to ${sl_dollars}")
            if self._validate_risk_field_value(sl_dollars):
                self.logger.info(f"■ Risk field already correctly set to ${sl_dollars} - skipping edit")
            else:
                if not self._edit_risk_field(sl_dollars):
                    self.logger.error("Failed to set Risk (SL) field")
                    return False
        else:
            self.logger.info("Skipping Risk field (SL=0)")

        # Set To Make (TP) field if tp_dollars > 0

```

```

if tp_dollars > 0:
    # FIRST: Check if To Make field is already set to the correct value
    self.logger.info(f"[CHECK] Checking if To Make field is already set to ${tp_dollars}")
    if self._validate_to_make_field_value(tp_dollars):
        self.logger.info(
            f"■ To Make field already correctly set to ${tp_dollars} - skipping edit")
    else:
        if not self._edit_to_make_field(tp_dollars):
            self.logger.error("Failed to set To Make (TP) field")
            return False
else:
    self.logger.info("Skipping To Make field (TP=0)")

self.logger.info("■ Inline TP/SL setup completed successfully")
return True

except Exception as e:
    self.logger.error(f"Failed to setup inline TP/SL: {e}")
    return False

```

Method: TopStepXAccount._edit_risk_field

```
def _edit_risk_field(self, sl_dollars)
```

Edit the Risk field in the data grid to set SL

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def _edit_risk_field(self, sl_dollars):
    """Edit the Risk field in the data grid to set SL"""
    try:
        self.logger.info(f"■ Editing Risk field to ${sl_dollars}")

        # Find the Risk field using the data attributes from the HTML
        risk_selectors = [
            "//div[@data-field='risk']//svg[contains(@class, 'fa-pencil')]",
            "//div[@role='cell'][@data-field='risk']//svg[@data-icon='pencil']",
            "//div[contains(@class, 'MuiDataGrid-cell') and @data-field='risk']//svg",
            ".MuiDataGrid-cell[data-field='risk'] svg[data-icon='pencil']"
        ]

        # Find and click the pencil icon to start editing
        pencil_icon = None
        for selector in risk_selectors:
            try:
                if selector.startswith("//"):
                    pencil_icon = self.driver.find_element(By.XPATH, selector)

```

```

        else:
            pencil_icon = self.driver.find_element(By.CSS_SELECTOR, selector)

            if pencil_icon.is_displayed():
                self.logger.info(f"Found Risk pencil icon with selector: {selector}")
                break
        except:
            continue

    if not pencil_icon:
        self.logger.error("Could not find Risk field pencil icon")
        return False

    # Click the pencil icon to start editing
    pencil_icon.click()
    time.sleep(0.5)

    # Look for the input field that appears
    input_selectors = [
        "//*[@data-field='risk']/input",
        "//*[@input[contains(@class, 'MuiInputBase-input')]]",
        "//*[@div[contains(@class, 'MuiDataGrid-cell') and @data-field='risk']/input"
    ]

    input_field = None
    for selector in input_selectors:
        try:
            input_field = WebDriverWait(self.driver, 3).until(
                EC.presence_of_element_located((By.XPATH, selector))
            )
            if input_field.is_displayed():
                break
        except:
            continue

    if not input_field:
        self.logger.error("Could not find Risk input field after clicking pencil")
        return False

    # Clear and enter the value
    input_field.click()
    time.sleep(0.2)
    input_field.send_keys(Keys.CONTROL + "a") # Select all
    time.sleep(0.2)
    input_field.send_keys(str(sl_dollars)) # Type new value
    time.sleep(0.3)
    input_field.send_keys(Keys.ENTER) # Confirm
    time.sleep(0.5)

    self.logger.info(f"■ Risk field set to ${sl_dollars}")
    return True

except Exception as e:
    self.logger.error(f"Failed to edit Risk field: {e}")
    return False

```

Method: TopStepXAccount._edit_to_make_field

```
def _edit_to_make_field(self, tp_dollars)
```


Edit the To Make field in the data grid to set TP

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def _edit_to_make_field(self, tp_dollars):
    """Edit the To Make field in the data grid to set TP"""
    try:
        self.logger.info(f"■ Editing To Make field to ${tp_dollars}")

        # Find the To Make field using the data attributes from the HTML
        to_make_selectors = [
            "//div[@data-field='toMake']//svg[contains(@class, 'fa-pencil')]",
            "//div[@role='cell'][@data-field='toMake']//svg[@data-icon='pencil']",
            "//div[contains(@class, 'MuiDataGrid-cell') and @data-field='toMake']//svg",
            ".MuiDataGrid-cell[data-field='toMake'] svg[data-icon='pencil']"
        ]

        # Find and click the pencil icon to start editing
        pencil_icon = None
        for selector in to_make_selectors:
            try:
                if selector.startswith("//"):
                    pencil_icon = self.driver.find_element(By.XPATH, selector)
                else:
                    pencil_icon = self.driver.find_element(By.CSS_SELECTOR, selector)

                if pencil_icon.is_displayed():
                    self.logger.info(f"Found To Make pencil icon with selector: {selector}")
                    break
            except:
                continue

        if not pencil_icon:
            self.logger.error("Could not find To Make field pencil icon")
            return False

        # Click the pencil icon to start editing
        pencil_icon.click()
        time.sleep(0.5)

        # Look for the input field that appears
        input_selectors = [
            "//div[@data-field='toMake']//input",
            "//input[contains(@class, 'MuiInputBase-input')]",
            "//div[contains(@class, 'MuiDataGrid-cell') and @data-field='toMake']//input"
        ]

        input_field = None
```

```

for selector in input_selectors:
    try:
        input_field = WebDriverWait(self.driver, 3).until(
            EC.presence_of_element_located((By.XPATH, selector))
        )
        if input_field.is_displayed():
            break
    except:
        continue

if not input_field:
    self.logger.error("Could not find To Make input field after clicking pencil")
    return False

# Clear and enter the value
input_field.click()
time.sleep(0.2)
input_field.send_keys(Keys.CONTROL + "a") # Select all
time.sleep(0.2)
input_field.send_keys(str(tp_dollars)) # Type new value
time.sleep(0.3)
input_field.send_keys(Keys.ENTER) # Confirm
time.sleep(0.5)

self.logger.info(f"■ To Make field set to ${tp_dollars}")
return True

except Exception as e:
    self.logger.error(f"Failed to edit To Make field: {e}")
    return False

```

Method: TopStepXAccount._edit_positions_risk_field

```
def _edit_positions_risk_field(self, sl_dollars)
```

Edit the Risk field (SL) in the positions section by clicking the cell to make it editable

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def _edit_positions_risk_field(self, sl_dollars):
    """
    Edit the Risk field (SL) in the positions section by clicking the cell to make it editable
    """
    try:
        self.logger.info(f"■ Clicking Risk cell to make it editable for SL=${sl_dollars}")

        # First ensure we find the positions table to avoid targeting order form

```

```

positions_table = None
try:
    positions_table = self.driver.find_element(By.XPATH,
        "//div[contains(@class, 'MuiDataGrid-root')]")
    self.logger.info("[CONTEXT] Found positions DataGrid table")
except:
    self.logger.warning("Could not find positions table context")

# Find and double-click the Risk cell directly - matching exact HTML structure
risk_cell_selectors = [
    "//div[@class='MuiDataGrid-cell--withRenderer MuiDataGrid-cell MuiDataGrid-cell--textCent",
    "//td[@data-field='risk']", # Simple Risk cell
    "//div[@data-field='risk']", # Risk div
    "//td[contains(@class, 'MuiDataGrid-cell--editable') and @data-field='risk']", # Editable
    "//div[contains(@class, 'MuiDataGrid-cell--editable') and @data-field='risk']", # Editable
]

risk_cell = None
for selector in risk_cell_selectors:
    try:
        risk_cell = self.driver.find_element(By.XPATH, selector)
        if risk_cell.is_displayed():
            break
    except:
        continue

if not risk_cell:
    self.logger.error("Could not find Risk cell in positions section")
    return False

# Double-click the cell to make it editable and enter value directly
self.logger.info(f"[DOUBLE-CLICK] Double-clicking Risk cell to edit value")

# Use ActionChains for reliable double-click
from selenium.webdriver.common.action_chains import ActionChains
actions = ActionChains(self.driver)
actions.double_click(risk_cell).perform()
time.sleep(0.15) # Reduced: Wait for cell to become editable

# Type the value directly (cell should now be in edit mode)
self.logger.info(f"[TYPE] Typing ${sl_dollars} into Risk field")

# SAFEGUARD: Don't proceed if sl_dollars is 0 or invalid
if not sl_dollars or sl_dollars <= 0:
    self.logger.error(f"■ BLOCKED: Will not set Risk field to invalid value: {sl_dollars}")
    return False

actions.send_keys(Keys.CONTROL + "a").perform() # Select all
time.sleep(0.05) # Reduced: Brief pause after select all

# SAFEGUARD: Double-check we're about to type a valid value
value_to_type = str(sl_dollars)
if not value_to_type or value_to_type in ['0', '0.0', '']:
    self.logger.error(f"■ BLOCKED: Will not type invalid value: '{value_to_type}'")
    return False

actions.send_keys(value_to_type).perform() # Type new value
actions.send_keys(Keys.ENTER).perform() # Confirm
time.sleep(0.1) # Reduced: Brief pause after confirmation

self.logger.info(f"Risk field set to ${sl_dollars}")

```

```

        return True

    except Exception as e:
        self.logger.error(f"Failed to edit positions Risk field: {e}")
        return False

```

Method: TopStepXAccount._edit_positions_to_make_field

```
def _edit_positions_to_make_field(self, tp_dollars)
```

Edit the To Make field (TP) in the positions section by clicking the cell to make it editable

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def _edit_positions_to_make_field(self, tp_dollars):
    """
    Edit the To Make field (TP) in the positions section by clicking the cell to make it editable
    """
    try:
        self.logger.info(f"■ Clicking To Make cell to make it editable for TP=${tp_dollars}")

        # First ensure we find the positions table to avoid targeting order form
        positions_table = None
        try:
            positions_table = self.driver.find_element(By.XPATH,
                "//div[contains(@class, 'MuiDataGrid-root')]")
            self.logger.info("[CONTEXT] Found positions DataGrid table")
        except:
            self.logger.warning("Could not find positions table context")

        # Find and double-click the To Make cell directly - matching exact HTML structure
        to_make_cell_selectors = [
            "//div[@class='MuiDataGrid-cell--withRenderer MuiDataGrid-cell MuiDataGrid-cell--textCent",
            "//td[@data-field='toMake']", # Simple To Make cell
            "//div[@data-field='toMake']", # To Make div
            "//td[contains(@class, 'MuiDataGrid-cell--editable') and @data-field='toMake']", # Editable
            "//div[contains(@class, 'MuiDataGrid-cell--editable') and @data-field='toMake']", # Editable
        ]

        to_make_cell = None
        for selector in to_make_cell_selectors:
            try:
                to_make_cell = self.driver.find_element(By.XPATH, selector)
                if to_make_cell.is_displayed():
                    break
            except:
                continue

```

```

if not to_make_cell:
    self.logger.error("Could not find To Make cell in positions section")
    return False

# Double-click the cell to make it editable and enter value directly
self.logger.info(f"[DOUBLE-CLICK] Double-clicking To Make cell to edit value")

# Use ActionChains for reliable double-click
from selenium.webdriver.common.action_chains import ActionChains
actions = ActionChains(self.driver)
actions.double_click(to_make_cell).perform()
time.sleep(0.15) # Reduced: Wait for cell to become editable

# Type the value directly (cell should now be in edit mode)
self.logger.info(f"[TYPE] Typing ${tp_dollars} into To Make field")

# SAFEGUARD: Don't proceed if tp_dollars is 0 or invalid
if not tp_dollars or tp_dollars <= 0:
    self.logger.error(f"■ BLOCKED: Will not set To Make field to invalid value: {tp_dollars}")
    return False

actions.send_keys(Keys.CONTROL + "a").perform() # Select all
time.sleep(0.05) # Reduced: Brief pause after select all

# SAFEGUARD: Double-check we're about to type a valid value
value_to_type = str(tp_dollars)
if not value_to_type or value_to_type in ['0', '0.0', '']:
    self.logger.error(f"■ BLOCKED: Will not type invalid value: '{value_to_type}'")
    return False

actions.send_keys(value_to_type).perform() # Type new value
actions.send_keys(Keys.ENTER).perform() # Confirm
time.sleep(0.1) # Reduced: Brief pause after confirmation

self.logger.info(f"To Make field set to ${tp_dollars}")
return True

except Exception as e:
    self.logger.error(f"Failed to edit positions To Make field: {e}")
    return False

```

Method: TopStepXAccount._validate_risk_field_value

```
def _validate_risk_field_value(self, expected_sl_dollars)
```

Validate that the Risk field was set to the correct SL value Returns True if the value matches, False otherwise Uses the same selectors as the editing function for consistency

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. try block with 1 except clause.

Code (copy-paste safe)

```
def _validate_risk_field_value(self, expected_sl_dollars):
    """
    Validate that the Risk field was set to the correct SL value
    Returns True if the value matches, False otherwise
    Uses the same selectors as the editing function for consistency
    """
    try:
        self.logger.info(f"■ Validating Risk field value (expected: ${expected_sl_dollars})")

        # Use the same selectors as _edit_position_sl for consistency
        risk_cell_selectors = [
            # Primary selector: exact data-field match for "risk"
            ".*//div[@role='cell' and @data-field='risk']",
            # Secondary: MUI DataGrid cell with risk field
            ".*//div[contains(@class, 'MuiDataGrid-cell') and @data-field='risk']",
            # Fallback: cell containing the pencil icon and $ value in risk context
            ".*//div[@role='cell' and contains(@data-field, 'risk')]",
        ]

        current_value = None
        for selector in risk_cell_selectors:
            try:
                # Search from the body element to ensure we find the right cell
                body_element = self.driver.find_element(By.TAG_NAME, "body")
                risk_cell = body_element.find_element(By.XPATH, selector)
                text_content = risk_cell.text.strip()

                # Look for dollar amounts in the text
                if '$' in text_content or text_content.replace('.', '').replace(',', '').isdigit():
                    current_value = text_content
                    self.logger.debug(
                        f"Found Risk field text: '{text_content}' using selector: {selector}")
                    break
            except:
                continue

        if current_value:
            # Extract numeric value from the text (remove $ and other chars)
            import re
            # More robust regex to handle various formats: $1,510, $1510, 1510, $1,510.00, etc.
            numeric_match = re.search(r'[\d,]+(?:\.\d+)?', current_value.replace('$', ''))
            if numeric_match:
                # Remove commas and convert to float
                clean_value = numeric_match.group().replace(',', '')
                current_numeric = float(clean_value)
                expected_numeric = float(expected_sl_dollars)

                # Check if values match (with small tolerance for float comparison)
                if abs(current_numeric - expected_numeric) < 0.01:
                    self.logger.info(
                        f"■ Risk field validation passed: '{current_value}' matches expected ${expected_sl_dollars}")
                    return True
                else:
                    self.logger.warning(
                        f"■ Risk field validation failed: '{current_value}' != ${expected_sl_dollars}")
                    return False
```

```

        else:
            self.logger.warning(f"Could not extract numeric value from Risk field: '{current_valu
            return False
    else:
        self.logger.warning("Could not find Risk field value for validation")
        return False

except Exception as e:
    self.logger.error(f"Error validating Risk field: {e}")
    return False

```

Method: TopStepXAccount._validate_to_make_field_value

```
def _validate_to_make_field_value(self, expected_tp_dollars)
```

Validate that the To Make field was set to the correct TP value Returns True if the value matches, False otherwise Uses the same selectors as the editing function for consistency

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def _validate_to_make_field_value(self, expected_tp_dollars):
    """
    Validate that the To Make field was set to the correct TP value
    Returns True if the value matches, False otherwise
    Uses the same selectors as the editing function for consistency
    """
    try:
        self.logger.info(f"■ Validating To Make field value (expected: ${expected_tp_dollars})")

        # Use the same selectors as _edit_position_tp for consistency
        to_make_cell_selectors = [
            # Primary selector: exact data-field match for "toMake"
            ".*div[@role='cell' and @data-field='toMake']",
            # Secondary: MUI DataGrid cell with toMake field
            ".*div[contains(@class, 'MuiDataGrid-cell') and @data-field='toMake']",
            # Fallback: cell containing the pencil icon and $ value in toMake context
            ".*div[@role='cell' and contains(@data-field, 'toMake')]",
        ]

        current_value = None
        for selector in to_make_cell_selectors:
            try:
                # Search from the body element to ensure we find the right cell
                body_element = self.driver.find_element(By.TAG_NAME, "body")
                to_make_cell = body_element.find_element(By.XPATH, selector)
                text_content = to_make_cell.text.strip()
            
```

```

        # Look for dollar amounts in the text
        if '$' in text_content or text_content.replace('.', '').replace(',', '').isdigit():
            current_value = text_content
            self.logger.debug(
                f"Found To Make field text: '{text_content}' using selector: {selector}")
            break
    except:
        continue

    if current_value:
        # Extract numeric value from the text (remove $ and other chars)
        import re
        # More robust regex to handle various formats: $1,510, $1510, 1510, $1,510.00, etc.
        numeric_match = re.search(r'[\d,]+(?:\.\d+)?', current_value.replace('$', ''))
        if numeric_match:
            # Remove commas and convert to float
            clean_value = numeric_match.group().replace(',', '')
            current_numeric = float(clean_value)
            expected_numeric = float(expected_tp_dollars)

            # Check if values match (with small tolerance for float comparison)
            if abs(current_numeric - expected_numeric) < 0.01:
                self.logger.info(
                    f"■ To Make field validation passed: '{current_value}' matches expected ${expected_tp_dollars}")
                return True
            else:
                self.logger.warning(
                    f"■ To Make field validation failed: '{current_value}' != ${expected_tp_dollars}")
                return False
        else:
            self.logger.warning(
                f"Could not extract numeric value from To Make field: '{current_value}'")
            return False
    else:
        self.logger.warning("Could not find To Make field value for validation")
        return False

except Exception as e:
    self.logger.error(f"Error validating To Make field: {e}")
    return False

```

Method: TopStepXAccount._ensure_on_trading_page

```
def _ensure_on_trading_page(self)
    Ensure we're on the main trading page and on the Trading tab
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def _ensure_on_trading_page(self):
    """Ensure we're on the main trading page and on the Trading tab"""
    try:
        self._dismiss_backdrop()
        # Check if we're already on the trading page
        current_url = self.driver.current_url
        if "topstepx.com" in current_url and "/login" not in current_url:
            # If we're already on /trade, be more patient waiting for the Buy button
            if "/trade" in current_url:
                self.logger.info("Already on trading page, waiting for interface to load...")
                wait = WebDriverWait(self.driver, 20) # Wait longer if already on /trade
                try:
                    wait.until(EC.presence_of_element_located((By.XPATH,
                                                                "//button[contains(text(), 'Buy') or contains(text(), 'BUY')]")))

                    # Also ensure we're on the Trading tab
                    self.logger.info("Ensuring we're on the Trading tab...")
                    if not self._navigate_to_trading_tab():
                        self.logger.warning("Could not navigate to Trading tab, but Buy button found")

                return True
            except TimeoutException:
                self.logger.warning("Buy button not found on /trade page after 20 seconds")
                pass
        else:
            # Not on /trade, check if Buy button exists on current page
            wait = WebDriverWait(self.driver, 5) # Shorter wait for non-trading pages
            try:
                wait.until(EC.presence_of_element_located((By.XPATH,
                                                            "//button[contains(text(), 'Buy') or contains(text(), 'BUY')]")))
                return True
            except TimeoutException:
                pass

        # Only navigate if we're not already on the trading page
        if "/trade" not in current_url:
            trading_url = f"{self.base_url}/trade"
            self.logger.info(f"Navigating to trading page: {trading_url}")
            self.driver.get(trading_url)
            time.sleep(3)

            # Wait for trading interface to load
            wait = WebDriverWait(self.driver, 15)
            wait.until(EC.presence_of_element_located((By.XPATH,
                                                        "//button[contains(text(), 'Buy') or contains(text(), 'BUY')]")))
        else:
            self.logger.warning(
                "Already on /trade but Buy button not found - page may not be fully loaded")
            return False

        # Ensure we're on the Trading tab
        self.logger.info("Ensuring we're on the Trading tab...")
        if not self._navigate_to_trading_tab():
            self.logger.warning("Could not navigate to Trading tab after navigating to trading page")

        return True

    except Exception as e:
```

```
self.logger.error(f"Failed to navigate to trading page: {e}")
return False
```

Method: TopStepXAccount._set_contract_symbol

```
@retry_on_stale_element(max_retries=2, delay=0.5)
def _set_contract_symbol(self, symbol)
```

ULTRA-FAST: Direct symbol entry with minimal waits

Why these Python concepts were used

- **@retry_on_stale_element** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def _set_contract_symbol(self, symbol):
    """
    ULTRA-FAST: Direct symbol entry with minimal waits
    """
    try:
        start_time = time.time()

        # Find the contract symbol field (no logging for speed)
        # Try multiple selectors for the symbol field
        symbol_selectors = [
            "//*[@input[contains(@class, 'MuiAutocomplete-input') and @role='combobox']]",
            "//*[@input[contains(@class, 'MuiAutocomplete-input')]]",
            "//*[@input[@placeholder='Symbol']]",
            "//*[@input[@placeholder='Search Symbol']]",
            "//*[@input[@aria-label='Symbol']]",
            "//*[@input[@type='text' and contains(@class, 'MuiInputBase-input')]]"
        ]

        symbol_field = None
        for selector in symbol_selectors:
            try:
                symbol_field = WebDriverWait(self.driver, 1).until(
                    EC.presence_of_element_located((By.XPATH, selector)))
            if symbol_field:
                break
        except:
```

```

        continue

    if not symbol_field:
        raise Exception("Could not find symbol input field with any selector")

    # SPEED: short-circuit if the field already shows the requested symbol
    try:
        existing_value = (symbol_field.get_attribute('value') or '').strip().upper()
    except Exception:
        existing_value = ''
    if existing_value and existing_value == symbol.upper():
        elapsed_ms = (time.time() - start_time) * 1000
        self.logger.info(f"■ Symbol already set: {existing_value} ({elapsed_ms:.0f}ms, no-op)")
        return True

    # Dismiss any overlays, then JS-focus the field (bypasses click interception)
    self._dismiss_backdrop()
    self.driver.execute_script("arguments[0].focus(); arguments[0].click();", symbol_field)
    symbol_field.send_keys(Keys.CONTROL + "a")
    symbol_field.send_keys(Keys.DELETE)

    # JS clear to ensure it's 100% empty
    self.driver.execute_script("""
        var field = arguments[0];
        field.value = '';
        field.dispatchEvent(new Event('input', { bubbles: true }));
        field.dispatchEvent(new Event('change', { bubbles: true }));
    """, symbol_field)

    # Type first letter to trigger dropdown
    first_char = symbol[0] if symbol else 'N'
    symbol_field.send_keys(first_char)
    time.sleep(0.15) # Reduced wait

    # FAST DROPDOWN: Reduced timeout and no debug logging
    dropdown_clicked = False
    try:
        dropdown_options = WebDriverWait(self.driver, 2).until(
            EC.presence_of_all_elements_located((By.CSS_SELECTOR, "li.MuiAutocomplete-option"))
        )

        # Fast loop - find and click match immediately (no logging)
        for option in dropdown_options:
            try:
                spans = option.find_elements(By.TAG_NAME, "span")
                if len(spans) < 2:
                    continue

                opt_symbol = spans[0].text.strip().upper()
                opt_desc = spans[1].text.strip().upper()

                # Match logic (streamlined)
                matched = False
                if symbol.upper() in ['NQM6', 'NQM25', 'NQM26']:
                    matched = (
                        opt_symbol == 'NQM25' or opt_symbol == 'NQM26') and 'MICRO' not in opt_desc
                elif symbol.upper() in ['MNQM6', 'MNQM25', 'MNQM26']:
                    matched = (
                        opt_symbol == 'MNQM25' or opt_symbol == 'MNQM26') and 'MICRO' in opt_desc
                elif symbol.upper() in ['NQH6', 'NQH25', 'NQH26']:

```

```

        matched = (
            opt_symbol == 'NQH25' or opt_symbol == 'NQH26') and 'MICRO' not in opt_desc
    elif symbol.upper() in ['MNQH6', 'MNQH25', 'MNQH26']:
        matched = (
            opt_symbol == 'MNQH25' or opt_symbol == 'MNQH26') and 'MICRO' in opt_desc
    elif opt_symbol == symbol.upper():
        matched = True

    if matched:
        # Fast click - JS only
        self.driver.execute_script("arguments[0].click();", option)
        dropdown_clicked = True
        time.sleep(0.1) # Minimal wait
        break
    except:
        continue

except:
    pass # Silent fail, will use fallback

# FAST VERIFY: Quick check and fallback if needed
time.sleep(0.15) # Reduced wait

# Re-find the field for verification using the same robust selectors
found_field = None
for selector in symbol_selectors:
    try:
        found_field = self.driver.find_element(By.XPATH, selector)
        if found_field and found_field.is_displayed():
            symbol_field = found_field
            break
    except:
        continue

final_value = symbol_field.get_attribute('value') or '' if symbol_field else ''

# Fast fallback: If dropdown didn't work, type directly
if not dropdown_clicked or final_value.strip().upper() != symbol.upper():
    if symbol_field:
        # Clear and type full symbol (JS click to bypass overlays)
        self.driver.execute_script("arguments[0].focus(); arguments[0].click();", symbol_field)
        symbol_field.send_keys(Keys.CONTROL + "a")
        symbol_field.send_keys(Keys.DELETE)
        symbol_field.send_keys(symbol.upper())
        symbol_field.send_keys(Keys.ENTER)
        time.sleep(0.15) # Reduced wait
        final_value = symbol_field.get_attribute('value') or ''
    else:
        self.logger.warning("Could not find symbol field for fallback entry")

# Final verification (streamlined)
success = False
if final_value:
    if symbol.upper() in ['NQM6', 'NQM25', 'NQM26']:
        success = 'NQM25' in final_value.upper() or 'NQM26' in final_value.upper()
    elif symbol.upper() in ['MNQM6', 'MNQM25', 'MNQM26']:
        success = 'MNQM25' in final_value.upper() or 'MNQM26' in final_value.upper()
    elif symbol.upper() in ['NQH6', 'NQH25', 'NQH26']:
        success = 'NQH25' in final_value.upper() or 'NQH26' in final_value.upper()
    elif symbol.upper() in ['MNQH6', 'MNQH25', 'MNQH26']:

```

```

        success = 'MNQH25' in final_value.upper() or 'MNQH26' in final_value.upper()
    else:
        success = symbol.upper() in final_value.upper()

    elapsed_ms = (time.time() - start_time) * 1000

    if success:
        self.logger.info(f"■ Symbol set: {final_value} ({elapsed_ms:.0f}ms)")
        return True
    else:
        self.logger.error(f"■ Symbol failed: expected '{symbol}', got '{final_value}'")
        return False

except Exception as e:
    self.logger.error(f"■ Symbol selection error: {e}")
    return False

```

Method: TopStepXAccount._get_current_contract_symbol

```
def _get_current_contract_symbol(self)

    Get the currently selected contract symbol from the field
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def _get_current_contract_symbol(self):
    """Get the currently selected contract symbol from the field"""
    try:
        # Find the contract symbol input field
        symbol_selectors = [
            "input[placeholder*='Search']",
            "input[aria-label*='symbol']",
            "input[aria-label*='contract']",
            ".MuiAutocomplete-input",
            "input[data-testid*='symbol']"
        ]

        for selector in symbol_selectors:
            try:
                elements = self.driver.find_elements(By.CSS_SELECTOR, selector)
                for element in elements:
                    if element.is_displayed():
                        current_value = element.get_attribute('value')
                        if current_value and ('NQ' in current_value or 'MNQ' in current_value):
                            self.logger.info(f"■ Found current symbol: {current_value}")
                            return current_value
            except:

```

```

        continue

    self.logger.warning("■■■ Could not find current symbol in any field")
    return None

except Exception as e:
    self.logger.error(f"■■ Error getting current symbol: {e}")
    return None

```

Method: TopStepXAccount._set_quantity

```

@retry_on_stale_element(max_retries=3, delay=1)
def _set_quantity(self, quantity)

```

FAST: Set quantity field with minimal waits

Why these Python concepts were used

- **@retry_on_stale_element** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def _set_quantity(self, quantity):
    """FAST: Set quantity field with minimal waits"""
    try:
        # Fast selector - most specific first
        quantity_field = WebDriverWait(self.driver, 2).until(
            EC.presence_of_element_located((By.XPATH, "//input[@type='number'][@min='1']"))
        )

        # JS focus+click to bypass any overlay, then keyboard interaction
        self.driver.execute_script("arguments[0].focus(); arguments[0].click();", quantity_field)
        quantity_field.send_keys(Keys.CONTROL + "a")
        quantity_field.send_keys(str(quantity))
        quantity_field.send_keys(Keys.TAB)
        time.sleep(0.05) # Minimal wait

        return True

    except Exception as e:
        self.logger.error(f"■■ Qty failed: {e}")
        return False

```

Method: TopStepXAccount._set_order_type

```

def _set_order_type(self, order_type)

```

Set the order type (Market, Limit, etc.)

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def _set_order_type(self, order_type):
    """Set the order type (Market, Limit, etc.)"""
    try:
        wait = WebDriverWait(self.driver, 10)

        # Find order type dropdown
        order_type_selectors = [
            "//div[contains(@aria-label, 'Order Type')]",
            "//select[contains(@id, 'orderType')]",
            "//div[contains(text(), 'Market')]" # Current order type display
        ]

        order_type_field = None
        for selector in order_type_selectors:
            try:
                order_type_field = wait.until(EC.element_to_be_clickable((By.XPATH, selector)))
                break
            except TimeoutException:
                continue

        if not order_type_field:
            self.logger.warning("Could not find order type field")
            return False

        # Click on the dropdown
        order_type_field.click()
        time.sleep(1)

        # Select the desired order type
        option_xpath = f"//li[contains(text(), '{order_type.title()}')]"
        try:
            option = self.driver.find_element(By.XPATH, option_xpath)
            option.click()
            time.sleep(0.5)

            self.logger.info(f"Order type set to: {order_type}")
            return True
        except:
            self.logger.warning(f"Could not find order type option: {order_type}")
            return False

    except Exception as e:
        self.logger.error(f"Failed to set order type: {e}")
```

```
return False
```

Method: TopStepXAccount._click_buy_button

```
@retry_on_stale_element(max_retries=3, delay=1)
def _click_buy_button(self, skip_post_trade_setup=False)
```

ULTRA-FAST: Click BUY immediately

Why these Python concepts were used

- **@retry_on_stale_element** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def _click_buy_button(self, skip_post_trade_setup=False):
    """ULTRA-FAST: Click BUY immediately"""
    try:
        # Fast find and click
        buy_button = WebDriverWait(self.driver, 2).until(
            EC.element_to_be_clickable((By.XPATH, "//button[contains(text(), 'Buy')]"))
        )
        self.driver.execute_script("arguments[0].click();", buy_button)

        # In fast mode, minimal wait
        if skip_post_trade_setup:
            time.sleep(0.3)
            return True

        # Normal mode: handle confirmation
        time.sleep(0.5)
        self._handle_order_confirmation()
        return True

    except Exception as e:
        self.logger.error(f"■ BUY failed: {e}")
        return False
```

Method: TopStepXAccount._click_sell_button

```
@retry_on_stale_element(max_retries=3, delay=1)
def _click_sell_button(self, skip_post_trade_setup=False)
```

ULTRA-FAST: Click SELL immediately

Why these Python concepts were used

- **@retry_on_stale_element** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 2 **except** clauses.

Code (copy-paste safe)

```
def _click_sell_button(self, skip_post_trade_setup=False):
    """ULTRA-FAST: Click SELL immediately"""
    try:
        # Fast find and click
        sell_button = WebDriverWait(self.driver, 2).until(
            EC.element_to_be_clickable((By.XPATH, "//button[contains(text(), 'Sell')]"))
        )
        self.driver.execute_script("arguments[0].click();", sell_button)

        # In fast mode, minimal wait
        if skip_post_trade_setup:
            time.sleep(0.3)
            return True

        # Normal mode: handle confirmation
        time.sleep(0.5)
        self._handle_order_confirmation()
        return True

    except Exception as e:
        self.logger.error(f"■ SELL failed: {e}")
        return False
        if not self._click_order_tab():
            self.logger.warning("Failed to click Order tab after SELL button")

        # Wait for any confirmation dialogs and handle them
        time.sleep(1)
        self._handle_order_confirmation()

        return True

    except Exception as e:
        self.logger.error(f"Failed to click SELL button: {e}")
        return False
```

Method: TopStepXAccount._click_flatten_all_button

```
def _click_flatten_all_button(self)

    Click the Flatten All button to close all positions
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def _click_flatten_all_button(self):
    """Click the Flatten All button to close all positions"""
    try:
        wait = WebDriverWait(self.driver, 10)

        # Find Flatten All button - based on HTML analysis
        flatten_selectors = [
            "//button[contains(text(), 'Flatten All')]",
            "//button[contains(text(), 'Close All')]",
            "//button[contains(text(), 'FLATTEN ALL')]"
        ]

        flatten_button = None
        for selector in flatten_selectors:
            try:
                flatten_button = wait.until(EC.element_to_be_clickable((By.XPATH, selector)))
                break
            except TimeoutException:
                continue

        if not flatten_button:
            self.logger.warning("Could not find Flatten All button")
            return False

        # Click the Flatten All button
        flatten_button.click()
        self.logger.info("Flatten All button clicked")

        # Handle any confirmation dialogs
        time.sleep(1)
        self._handle_order_confirmation()

        return True

    except Exception as e:
        self.logger.error(f"Failed to click Flatten All button: {e}")
        return False
```

Method: TopStepXAccount._close_individual_positions

```
def _close_individual_positions(self)

    Try to close individual positions if Flatten All is not available
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def _close_individual_positions(self):
    """Try to close individual positions if Flatten All is not available"""
    try:
        # Look for individual position close buttons
        close_buttons = self.driver.find_elements(By.XPATH,
            "//button[contains(text(), 'Close Position')]")

        if not close_buttons:
            self.logger.warning("No individual position close buttons found")
            return False

        for button in close_buttons:
            if button.is_enabled():
                button.click()
                time.sleep(1)
                self._handle_order_confirmation()

        self.logger.info("Individual positions closed")
        return True

    except Exception as e:
        self.logger.error(f"Failed to close individual positions: {e}")
        return False
```

Method: TopStepXAccount._handle_order_confirmation

```
def _handle_order_confirmation(self)

    Handle any order confirmation dialogs that may appear
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def _handle_order_confirmation(self):
    """Handle any order confirmation dialogs that may appear"""
    try:
        # Look for confirmation dialogs and click confirm if present
        confirmation_selectors = [
```

```

        "//button[contains(text(), 'Confirm')]",
        "//button[contains(text(), 'YES')]",
        "//button[contains(text(), 'OK')]",
        "//button[contains(text(), 'Submit')]"
    ]

    for selector in confirmation_selectors:
        try:
            confirm_button = WebDriverWait(self.driver, 2).until(
                EC.element_to_be_clickable((By.XPATH, selector))
            )
            confirm_button.click()
            self.logger.info("Order confirmation handled")
            time.sleep(0.5)
            break
        except TimeoutException:
            continue

    except Exception as e:
        # No confirmation dialog is fine
        pass

```

Method: TopStepXAccount.disconnect

```
def disconnect(self)

    Disconnect from TopStepX and clean up
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def disconnect(self):
    """Disconnect from TopStepX and clean up"""
    try:
        if self.driver:
            self.driver.quit()
            self.driver = None

        self.logged_in = False
        self.logger.info("Disconnected from TopStepX")

    except Exception as e:
        self.logger.error(f"Error during TopStepX disconnect: {e}")

```

Method: TopStepXAccount.close

```
def close(self)
```

Close the TopStepX connection and quit Chrome driver Alias for disconnect() to match Tradovate interface Ensures Chrome browser always closes on disconnect

Step by step (top-level body)

1. Calls `self.disconnect(...)` for its side effect.

Code (copy-paste safe)

```
def close(self):
    """
    Close the TopStepX connection and quit Chrome driver
    Alias for disconnect() to match Tradovate interface
    Ensures Chrome browser always closes on disconnect
    """
    self.disconnect()
```

Method: TopStepXAccount.prepare_field_for_input

```
def prepare_field_for_input(self, field_element, field_name='Field')
```

Prepare field for new input by selecting all existing content Uses highlight-and-type-over approach instead of deletion

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def prepare_field_for_input(self, field_element, field_name="Field"):
    """
    Prepare field for new input by selecting all existing content
    Uses highlight-and-type-over approach instead of deletion
    """
    try:
        self.logger.info(f"[PREPARE] Preparing {field_name} for new input")

        # Select all existing content (no deletion needed)
        field_element.send_keys(Keys.CONTROL + "a")
        time.sleep(0.2) # Allow selection to complete

        # Verify field is ready for input
        current_value = field_element.get_attribute("value") or ""
        self.logger.info(f"[PREPARE] {field_name} current value: '{current_value}' (will be replaced)")

        return True

    except Exception as e:
        self.logger.error(f"[CLEAR] Error clearing {field_name}: {e}")
        return False
```

Method: TopStepXAccount._set_field_with_clearing

```
def _set_field_with_clearing(self, selectors, value, field_name, input_type='text')
```

OPTIMIZED: Generic method to set any field with fast clearing FAST-FAIL: Reduced timeout to 0.5s for quick failure on non-existent fields

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def _set_field_with_clearing(self, selectors, value, field_name, input_type="text"):
    """
    OPTIMIZED: Generic method to set any field with fast clearing
    FAST-FAIL: Reduced timeout to 0.5s for quick failure on non-existent fields
    """
    try:
        # CRITICAL: Reduced from 3s to 0.5s for fast-fail on missing fields
        wait = WebDriverWait(self.driver, 0.5)

        # Find the field using provided selectors
        field_element = None
        for selector in selectors:
            try:
                field_element = wait.until(EC.element_to_be_clickable((By.XPATH, selector)))
                break
            except TimeoutException:
                continue

        if not field_element:
            self.logger.debug(f"Could not find {field_name} field")
            return False

        # OPTIMIZED: Fast JavaScript clear and set (no delays)
        self.driver.execute_script("""
            var field = arguments[0];
            var value = arguments[1];
            field.focus();
            field.value = value;
            field.dispatchEvent(new Event('input', { bubbles: true }));
            field.dispatchEvent(new Event('change', { bubbles: true }));
            """, field_element, str(value))

        self.logger.info(f"■ {field_name}: {value}")
        return True

    except Exception as e:
        self.logger.debug(f"Skip {field_name}: {e}")
```

```
return False
```

Method: TopStepXAccount._set_stop_loss_price

```
def _set_stop_loss_price(self, price)
```

Set stop loss price if such field exists

Step by step (top-level body)

1. Assigns stop_loss_selectors = ["//input[contains(@placeholder, 'Stop Loss') or contains....
2. **return** self._set_field_with_clearing(stop_loss_selectors, price, 'Stop Los....

Code (copy-paste safe)

```
def _set_stop_loss_price(self, price):
    """Set stop loss price if such field exists"""
    stop_loss_selectors = [
        "//input[contains(@placeholder, 'Stop Loss') or contains(@aria-label, 'Stop Loss')]",
        "//input[contains(@id, 'stopLoss') or contains(@id, 'stop-loss')]",
        "//input[contains(@class, 'stop-loss') or contains(@class, 'stopLoss')]",
        "//input[@type='number'][contains(../label/text(), 'Stop')]"
    ]
    return self._set_field_with_clearing(stop_loss_selectors, price, "Stop Loss Price", "number")
```

Method: TopStepXAccount._set_take_profit_price

```
def _set_take_profit_price(self, price)
```

Set take profit price if such field exists

Step by step (top-level body)

1. Assigns take_profit_selectors = ["//input[contains(@placeholder, 'Take Profit') or contai....
2. **return** self._set_field_with_clearing(take_profit_selectors, price, 'Take P....

Code (copy-paste safe)

```
def _set_take_profit_price(self, price):
    """Set take profit price if such field exists"""
    take_profit_selectors = [
        "//input[contains(@placeholder, 'Take Profit') or contains(@aria-label, 'Take Profit')]",
        "//input[contains(@id, 'takeProfit') or contains(@id, 'take-profit')]",
        "//input[contains(@class, 'take-profit') or contains(@class, 'takeProfit')]",
        "//input[@type='number'][contains(../label/text(), 'Target')]"
    ]
    return self._set_field_with_clearing(take_profit_selectors, price, "Take Profit Price", "number")
```

Method: TopStepXAccount._set_limit_price

```
def _set_limit_price(self, price)
```

Set limit price if such field exists

Step by step (top-level body)

1. Assigns limit_price_selectors = ["//input[contains(@placeholder, 'Limit Price') or contai....
2. **return** self._set_field_with_clearing(limit_price_selectors, price, 'Limit

Code (copy-paste safe)

```
def _set_limit_price(self, price):
    """Set limit price if such field exists"""
    limit_price_selectors = [
        "//input[contains(@placeholder, 'Limit Price') or contains(@aria-label, 'Limit Price')]",
        "//input[contains(@id, 'limitPrice') or contains(@id, 'limit-price')]",
        "//input[contains(@class, 'limit-price') or contains(@class, 'limitPrice')]",
        "//input[@type='number'][contains(../label/text(), 'Limit')]"
    ]
    return self._set_field_with_clearing(limit_price_selectors, price, "Limit Price", "number")
```

Method: TopStepXAccount.clear_all_visible_inputs

```
def clear_all_visible_inputs(self)
```

Optimized method to quickly clear main trading input fields only Focuses on contract and quantity fields for speed

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def clear_all_visible_inputs(self):
    """
    Optimized method to quickly clear main trading input fields only
    Focuses on contract and quantity fields for speed
    """
    try:
        # Only clear the essential trading fields for speed
        essential_selectors = [
            "//input[contains(@class, 'MuiAutocomplete-input')]", # Contract field
            "//input[@type='number'][@min='1']", # Quantity field
        ]

        cleared_count = 0
        for selector in essential_selectors:
            try:
                input_elem = self.driver.find_element(By.XPATH, selector)
                if input_elem.is_displayed() and input_elem.is_enabled():
                    field_value = input_elem.get_attribute("value") or ""

                    if field_value.strip(): # Only clear if it has content
                        # Fast JavaScript clear - no delays
                        self.driver.execute_script("""
                            arguments[0].focus();
                            arguments[0].select();
                            arguments[0].value = '';
                            arguments[0].dispatchEvent(new Event('input', { bubbles: true }));
                        """, input_elem)
```



```

        """", input_elem)
        cleared_count += 1

    except Exception:
        continue # Skip if field not found

    return cleared_count > 0

except Exception as e:
    self.logger.debug(f"Field clearing skipped: {e}")
    return False

```

Method: TopStepXAccount.__del__

```
def __del__(self)
    Cleanup when object is destroyed
```

Step by step (top-level body)

1. Calls self.disconnect(...) for its side effect.

Code (copy-paste safe)

```
def __del__(self):
    """Cleanup when object is destroyed"""
    self.disconnect()
```

Functions

```
def retry_on_stale_element(max_retries=3, delay=1)
    Decorator to retry web operations that may fail due to stale elements
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.

Step by step (top-level body)

1. Defines a nested function decorator(...).
2. **return** decorator.

Code (copy-paste safe)

```
def retry_on_stale_element(max_retries=3, delay=1):
    """Decorator to retry web operations that may fail due to stale elements"""
```

```

def decorator(func):
    @wraps(func)
    def wrapper(self, *args, **kwargs):
        for attempt in range(max_retries):
            try:
                return func(self, *args, **kwargs)
            except (StaleElementReferenceException, NoSuchElementException) as e:
                if attempt == max_retries - 1:
                    self.logger.error(f"Max retries reached for {func.__name__}: {e}")
                    raise e
                self.logger.warning(
                    f"Retrying {func.__name__} due to stale element (attempt {attempt + 1})")
                time.sleep(delay)
            except Exception as e:
                # Don't retry for other exceptions
                raise e
        return None
    return wrapper
return decorator

```

```
def test_topstepx_connection()
```

Test TopStepX connection with sample credentials

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Calls `print(...)` for its side effect.
2. Calls `print(...)` for its side effect.
3. Assigns `test_username = input("Enter TopStepX username (or 'skip' to skip test): ")`.
4. `if test_username.lower() == 'skip':` branches conditionally.
5. Assigns `test_password = input('Enter TopStepX password: ')`.
6. Assigns `account = TopStepXAccount(username=test_username, password=test_pas....`
7. **try** block with 1 **except** clause, plus a **finally**.

Code (copy-paste safe)

```

def test_topstepx_connection():
    """Test TopStepX connection with sample credentials"""
    print("■ Testing TopStepX Trading Integration")
    print("=" * 50)

    # These would need to be real credentials for actual testing
    test_username = input("Enter TopStepX username (or 'skip' to skip test): ")
    if test_username.lower() == 'skip':
        print("■■ TopStepX test skipped")
    return

```

```

test_password = input("Enter TopStepX password: ")

account = TopStepXAccount(username=test_username, password=test_password)

try:
    print("\n■ Attempting TopStepX login...")
    if account.login():
        print("■ TopStepX login successful!")

        # Test account info
        print("\n■ Getting account information...")
        account_info = account.get_account_info()
        if account_info:
            print(f"Account Info: {account_info}")

        # Test getting positions
        print("\n■ Getting current positions...")
        positions = account.get_positions()
        print(f"Positions: {positions}")

        # Test getting orders
        print("\n■ Getting current orders...")
        orders = account.get_orders()
        print(f"Orders: {orders}")

        # Ask user if they want to test actual trading
        test_trading = input("\n■ Do you want to test actual order placement? (yes/no): ").lower()
        if test_trading == 'yes':
            print("\n■ WARNING: This will place real orders!")
            confirm = input("Type 'CONFIRM' to proceed with live trading test: ")

            if confirm == 'CONFIRM':
                print("\n■ Testing order placement...")

                # Test small market orders
                symbol = input("Enter symbol to trade (e.g., MNQM25): ") or "MNQM25"
                quantity = int(input("Enter quantity (default 1): ") or "1")

                print(f"\n■ Placing BUY order: {symbol} x{quantity}")
                buy_result = account.place_buy_order(symbol, quantity)
                print(f"BUY Result: {buy_result}")

                if buy_result.get("success"):
                    # Wait a moment then close the position
                    time.sleep(2)
                    print(f"\n■ Placing SELL order to close: {symbol} x{quantity}")
                    sell_result = account.place_sell_order(symbol, quantity)
                    print(f"SELL Result: {sell_result}")

                # Test close all positions
                print("\n■ Testing close all positions...")
                close_result = account.close_all_positions()
                print(f"Close All Result: {close_result}")

            else:
                print("■ Live trading test cancelled")
        else:
            print("■ Order placement test skipped")

        # Test order methods without actual execution (dry run)

```

```

        print("\n■ Testing order methods (dry run)...")
        print("Note: These would place real orders in a live environment")

    else:
        print("■ TopStepX login failed")

except Exception as e:
    print(f"■ TopStepX test error: {e}")
    import traceback
    traceback.print_exc()

finally:
    # Cleanup - disconnect
    try:
        account.disconnect()
        print("■ Disconnected from TopStepX")
    except:
        pass

def test_topstepx_connection()

```

Test the TopStepX connection and basic functionality

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Lazy import from dotenv.
2. Calls `load_dotenv(...)` for its side effect.
3. Assigns `account = TopStepXAccount()`.
4. **try** block with 1 **except** clause, plus a **finally**.

Code (copy-paste safe)

```

def test_topstepx_connection():
    """Test the TopStepX connection and basic functionality"""
    from dotenv import load_dotenv
    load_dotenv()

    account = TopStepXAccount()

    try:
        print("■ Connecting to TopStepX...")
        if account.connect():
            print("■ Connected successfully!")

        # Test navigation
        print("■ Testing navigation...")
        account._ensure_on_trading_page()

        print("■ Basic functionality test passed")
    except:
        pass

```

```

else:
    print("■ Connection failed")
except Exception as e:
    print(f"■ Test failed: {e}")
    import traceback
    traceback.print_exc()
finally:
    print("\n■ Disconnecting from TopStepX...")
    account.disconnect()
    print("■ TopStepX test completed")

```

Step 8.4 — Build trader_companion/tradovate.py

What to do. Create the file trader_companion/tradovate.py in your project root and paste in the code shown in this step. The file is **3334** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from requirements.txt.

What this file is for. Tradovate-specific scraper and account adapter.

trader_companion/tradovate.py

3334 loc · 1 classes · 0 functions · 17 imports

Tradovate-specific scraper and account adapter. It defines **1** class, across **3,334** lines.

Python concepts demonstrated in this module

• Classes and objects • Comprehensions • Context managers (with-statements) • Dunder methods • Exceptions • f-strings • Lambdas

Imports — what each one is for

```
import os
```

Why imported. Operating-system interface. Path manipulation, environment variables, working-directory operations, exit codes, and thin wrappers around POSIX/Windows syscalls.

Used in this file. os.environ, os.environ.pop, os.getenv, os.path, os.path.dirname, os.path.join

Example. os.path.join(root, 'subdir', 'file.txt') # joins with the OS-correct separator

Other common scenarios.

- os.environ.get('API_KEY') to read environment variables
- os.makedirs(path, exist_ok=True) to create nested directories
- os.listdir(path) to list a directory's contents
- os.remove(path) / os.rename(src, dst) to delete or move a file

- `os.getcwd()` / `os.chdir(path)` to inspect or change the working dir

`import sys`

Why imported. Access to the running interpreter — `argv`, `stdin/stdout`, exit codes, the import search path, and the platform name.

Used in this file. `sys.path`, `sys.path.insert`

Example. `sys.exit(1)` # terminate the process with a non-zero status

Other common scenarios.

- `sys.argv[1:]` reads the command-line arguments
- `sys.path.insert(0, 'extra')` prepends to the import search path
- `sys.stderr.write(msg)` writes to standard error
- `sys.platform` tells you 'win32', 'linux', or 'darwin'
- `sys.modules` is the global cache of already-imported modules

`import time`

Why imported. Timestamps and sleep. Lower-level than `datetime` — works in Unix-epoch seconds.

Used in this file. `time.sleep`, `time.strftime`, `time.time`

Example. `time.sleep(1.5)` # pause this thread for 1.5 seconds

Other common scenarios.

- `time.time()` returns seconds since the epoch
- `time.monotonic()` for measuring elapsed time (never goes backward)
- `time.perf_counter()` for high-precision benchmarking
- `time.strftime` / `time.strptime` mirror the `datetime` versions

`import json`

Why imported. JSON encoding and decoding — the standard wire format for config files, API payloads, and inter-process messages.

Used in this file. `json.dumps`

Example. `json.loads(response.text)` # parse a JSON string to dict

Other common scenarios.

- `json.dumps(obj, indent=2)` for pretty-printing
- `json.dump(obj, file)` / `json.load(file)` for file I/O
- `default=str` handles non-serialisable types like `datetime`
- `object_hook=` customises decoding (e.g., to `dataclass` instances)

`import threading`

Why imported. Threads and synchronisation primitives. Used when work is I/O-bound and the GIL is not a bottleneck (the GIL prevents speed-up on CPU-bound work).

Used in this file. threading.RLock

Example. Thread(target=worker, args=(q,), daemon=True).start()

Other common scenarios.

- Lock / RLock to guard shared state
- Event for one-shot signals between threads
- Queue (from queue) for producer/consumer patterns
- threading.local() for thread-local storage

import logging

Why imported. Structured, levelled logging. Replaces print() with filterable, formattable output that can route to files, stderr, syslog, or HTTP endpoints.

Used in this file. logging.INFO, logging.basicConfig, logging.debug, logging.error, logging.info, logging.warning

Example. logging.getLogger(__name__).info('connected to %s', host)

Other common scenarios.

- .debug() / .info() / .warning() / .error() / .exception()
- logger.exception() captures the current traceback automatically
- logging.basicConfig(level=logging.INFO) for quick setup
- Handlers and Formatters route logs to files / streams / syslog

import random

Why imported. Pseudo-random numbers — fine for jitter, sampling, shuffling. NEVER for security; use secrets for that.

Used in this file. *No direct attribute access detected — likely imported for side effects, re-export, or string references.*

Example. random.uniform(0.5, 1.5) # backoff jitter

Other common scenarios.

- random.choice(seq) picks one item
- random.shuffle(list) rearranges in place
- random.seed(n) makes the sequence reproducible
- random.sample(seq, k) draws k unique items

import hashlib

Why imported. Cryptographic hash functions — SHA-256, MD5, BLAKE2.

Used in this file. *No direct attribute access detected — likely imported for side effects, re-export, or string references.*

Example. hashlib.sha256(b'hello').hexdigest()

Other common scenarios.

- Pass data in chunks via `.update()` to hash large files
- Use sha256 / sha512 for integrity; never MD5 or SHA-1 for security
- For password hashing use `bcrypt/argon2`, NOT raw `hashlib`

`import tempfile`

Why imported. Temporary files and directories that clean themselves up.

Used in this file. `tempfile.gettempdir`

Example. with `tempfile.TemporaryDirectory()` as `d`: ...

Other common scenarios.

- `tempfile.NamedTemporaryFile(delete=False)` for a real path on disk
- `tempfile.mkstemp()` returns a low-level (fd, path) pair

`from selenium import webdriver`

Why imported. Browser automation. Drives a real Chrome instance — the fallback for scraping prop-firm dashboards that have no API.

Used in this file. `webdriver`

Example. `driver.get(url); driver.find_element(By.ID, 'login')`

Other common scenarios.

- `WebDriverWait + expected_conditions` for reliable waits
- `ChromeOptions().add_argument('--headless=new')`
- `execute_script(js, *args)` reaches into the page for things the DOM API can't express

`from selenium.webdriver.common.by import By`

Why imported. Browser automation. Drives a real Chrome instance — the fallback for scraping prop-firm dashboards that have no API.

Used in this file. `By`

Example. `driver.get(url); driver.find_element(By.ID, 'login')`

Other common scenarios.

- `WebDriverWait + expected_conditions` for reliable waits
- `ChromeOptions().add_argument('--headless=new')`
- `execute_script(js, *args)` reaches into the page for things the DOM API can't express

`from selenium.webdriver.common.keys import Keys`

Why imported. Browser automation. Drives a real Chrome instance — the fallback for scraping prop-firm dashboards that have no API.

Used in this file. `Keys`

Example. `driver.get(url); driver.find_element(By.ID, 'login')`

Other common scenarios.

- WebDriverWait + expected_conditions for reliable waits
- ChromeOptions().add_argument('--headless=new')
- execute_script(js, *args) reaches into the page for things the DOM API can't express

```
from selenium.webdriver.support.ui import WebDriverWait
```

Why imported. Browser automation. Drives a real Chrome instance — the fallback for scraping prop-firm dashboards that have no API.

Used in this file. WebDriverWait

Example. driver.get(url); driver.find_element(By.ID, 'login')

Other common scenarios.

- WebDriverWait + expected_conditions for reliable waits
- ChromeOptions().add_argument('--headless=new')
- execute_script(js, *args) reaches into the page for things the DOM API can't express

```
from selenium.webdriver.support import expected_conditions as EC
```

Why imported. Browser automation. Drives a real Chrome instance — the fallback for scraping prop-firm dashboards that have no API.

Used in this file. EC

Example. driver.get(url); driver.find_element(By.ID, 'login')

Other common scenarios.

- WebDriverWait + expected_conditions for reliable waits
- ChromeOptions().add_argument('--headless=new')
- execute_script(js, *args) reaches into the page for things the DOM API can't express

```
from selenium.webdriver.common.action_chains import ActionChains
```

Why imported. Browser automation. Drives a real Chrome instance — the fallback for scraping prop-firm dashboards that have no API.

Used in this file. ActionChains

Example. driver.get(url); driver.find_element(By.ID, 'login')

Other common scenarios.

- WebDriverWait + expected_conditions for reliable waits
- ChromeOptions().add_argument('--headless=new')
- execute_script(js, *args) reaches into the page for things the DOM API can't express

```
from selenium.webdriver.chrome.options import Options
```

Why imported. Browser automation. Drives a real Chrome instance — the fallback for scraping prop-firm dashboards that have no API.

Used in this file. Options

Example. `driver.get(url); driver.find_element(By.ID, 'login')`

Other common scenarios.

- `WebDriverWait + expected_conditions` for reliable waits
- `ChromeOptions().add_argument('--headless=new')`
- `execute_script(js, *args)` reaches into the page for things the DOM API can't express

```
from dotenv import load_dotenv
```

Why imported. `python-dotenv` — load `.env` files into `os.environ` for twelve-factor configuration in development.

Used in this file. `load_dotenv`

Example. `load_dotenv()` # call once at process start

Other common scenarios.

- `dotenv_values('.env')` returns a dict without polluting `os.environ`
- Use a real secret store in production (vault, AWS SM, etc.)

Module constants

Each line below is followed by a plain-English note explaining what it does and why it is written that way.

```
DEFAULT_SYMBOL = os.getenv('TRADOVATE_SYMBOL') or 'MNQM6'
```

Module-level constant — bound once at import time and referenced from the functions and classes below.

```
DEFAULT_TP = os.getenv('TRADOVATE_TAKEPROFIT_TICKS')
```

Reads `TRADOVATE_TAKEPROFIT_TICKS` from the environment. Returns `None` if unset.

```
DEFAULT_SL = os.getenv('TRADOVATE_STOPLOSS_TICKS')
```

Reads `TRADOVATE_STOPLOSS_TICKS` from the environment. Returns `None` if unset.

Classes

```
class TradovateAccount
```

Tradovate trading automation class with robust Selenium-based web automation. Handles login, symbol selection, order placement, ATM configuration, and account statistics. Ensures single Chrome instance per account to prevent resource conflicts.

```
_chrome_instances = {}
```

Code (copy-paste safe)

```
class TradovateAccount:
    """
    Tradovate trading automation class with robust Selenium-based web automation.
    Handles login, symbol selection, order placement, ATM configuration, and account statistics.
    Ensures single Chrome instance per account to prevent resource conflicts.
    """

    # Class-level registry to track Chrome instances per account
```

```

_chrome_instances = {}

def __init__(self, username, password, pair_id=None, trading_mode="Simulation"):
    self.username = username
    self.password = password
    self.pair_id = pair_id or "default"
    self.trading_mode = trading_mode # "Simulation" or "Live Trading"
    self.logged_in = False
    self.driver = None
    self.first_trade_attempted = False
    self._first_stats_fetch = True
    self._placing_order = False # Flag to block stats fetching during order placement
    self._login_timestamp = None # Track when we logged in for debugging
    self.lock = threading.RLock() # Thread safety for Selenium operations

    # Create unique instance key using both username and pair_id
    instance_key = f"{username}_{self.pair_id}"

    # Check if Chrome instance already exists for this specific account+pair combination
    if instance_key in self._chrome_instances:
        logging.info(f"Reusing existing Chrome instance for account: {username} (Pair: {self.pair_id})")
        existing_driver = self._chrome_instances[instance_key]
        # Test if existing driver is still alive
        try:
            existing_driver.current_url
            self.driver = existing_driver
            logging.info("Successfully connected to existing Chrome instance")
            return
        except Exception:
            logging.info("Existing Chrome instance is dead, creating new one")
            # Remove dead instance from registry
            del self._chrome_instances[instance_key]

    # Initialize driver with error handling
    try:
        self.driver = self._initialize_driver()
    except Exception as e:
        raise Exception(
            f"Failed to initialize WebDriver: {str(e)}. This may indicate VPS Chrome setup issues.")

def _get_chromedriver_path(self):
    """Get the path to ChromeDriver executable - FAST VERSION (no compatibility check)"""
    # SELENIUM 4.15+ AUTO-MANAGEMENT: Let Selenium Manager handle ChromeDriver automatically
    # This ensures ChromeDriver always matches the installed Chrome version
    logging.info("[AUTO] Using Selenium Manager for automatic ChromeDriver version matching")
    return None # Return None to let Selenium Manager handle it

def _ensure_chrome_compatibility(self):
    """Ensure Chrome-ChromeDriver version compatibility"""
    try:
        # Import the compatibility manager
        sys.path.insert(0, os.path.dirname(os.path.dirname(__file__)))
        from chrome_auto_compatibility import ChromeVersionManager # type: ignore

        manager = ChromeVersionManager()
        success = manager.ensure_compatibility()

        if success:
            logging.info("[CHECK] Chrome-ChromeDriver compatibility verified")
        else:

```

```

        logging.warning("[WARNING] Chrome-ChromeDriver compatibility could not be ensured")

except Exception as e:
    logging.warning(f"Chrome compatibility check failed: {e}")
    # Continue anyway - don't break existing functionality

def _initialize_driver(self):
    """Initialize Chrome WebDriver with crash-resistant options"""
    chrome_options = Options()

    # PERFORMANCE: Removed user-data-dir to speed up Chrome launch
    # Incognito mode is faster than loading persistent profiles
    # Each Chrome instance will be fresh and fast

    # PERFORMANCE: Removed window positioning - let Chrome decide (faster)
    # PERFORMANCE: Removed remote debugging port - not needed for basic automation

    # MAXIMUM SPEED: Only critical options for fastest Chrome launch
    chrome_options.add_argument("--no-sandbox") # Required for some systems
    chrome_options.add_argument("--disable-dev-shm-usage") # Required for VPS/Docker

    # CRASH FIX: Improve stability and prevent crashes
    chrome_options.add_argument("--disable-crash-reporter") # Disable crash reporting
    chrome_options.add_argument("--disable-in-process-stack-traces") # Reduce memory overhead
    chrome_options.add_argument("--disable-logging") # Reduce disk I/O
    chrome_options.add_argument("--log-level=3") # Suppress console messages (3=FATAL only)
    chrome_options.add_argument("--silent") # Suppress console output

    # MEMORY MANAGEMENT: Prevent memory leaks and crashes
    chrome_options.add_argument("--disable-features=TranslateUI") # Disable translate
    chrome_options.add_argument("--disable-features=MediaRouter") # Disable media router
    chrome_options.add_argument("--disable-component-update") # Disable component updates
    chrome_options.add_argument("--disable-background-timer-throttling") # Better tab management
    chrome_options.add_argument("--disable-backgrounding-occluded-windows") # Keep windows active
    chrome_options.add_argument("--disable-renderer-backgrounding") # Prevent renderer from sleeping

    # RENDERING FIX: Enable GPU acceleration for proper page rendering
    # Removed --disable-gpu to fix white screen issues
    chrome_options.add_argument("--enable-features=NetworkService,NetworkServiceInProcess")

    # PERFORMANCE: Faster page loading optimizations
    chrome_options.add_argument("--disable-extensions") # No extensions = faster
    chrome_options.add_argument("--disable-plugins") # No plugins = faster
    # Images enabled – needed for CAPTCHA solving and normal page rendering
    chrome_options.add_argument("--disable-background-networking") # No background requests
    chrome_options.add_argument("--disable-default-apps") # No default apps
    chrome_options.add_argument("--disable-sync") # No sync

    # SPEED OPTIMIZATION: Smaller window for faster rendering
    chrome_options.add_argument("--window-size=1200,800") # Reduced from 1400,900

    # FIX: Disable hardware acceleration fallback that can cause white screens
    chrome_options.add_argument("--disable-software-rasterizer")

    # Anti-detection settings (keep minimal)
    chrome_options.add_experimental_option("excludeSwitches", ["enable-automation", "enable-logging"])
    chrome_options.add_experimental_option('useAutomationExtension', False)
    chrome_options.add_argument("--disable-blink-features=AutomationControlled")

    # Minimal preferences to prevent crashes

```

```

prefs = {
    "profile.default_content_setting_values": {
        "popups": 2, # Block popups only
        "notifications": 2, # Block notifications only
    }
}
chrome_options.add_experimental_option("prefs", prefs)

try:
    # Ensure Chrome-ChromeDriver compatibility before initialization
    logging.info("[COMPAT] Checking Chrome-ChromeDriver compatibility...")
    self._ensure_chrome_compatibility()

    # Get ChromeDriver path (returns None for auto-management in Selenium 4.15+)
    chromedriver_path = self._get_chromedriver_path()

    # Create the WebDriver instance - let Selenium Manager handle ChromeDriver automatically
    logging.info("[CHROME] Initializing Chrome browser...")
    logging.info("[CHROME] Using Selenium Manager for automatic ChromeDriver version matching")
    logging.info(
        "[CHROME] Selenium Manager will download ChromeDriver if needed (happens once per Chrome"
    )
    logging.info("[CHROME] If already cached, Chrome will launch immediately...")

    import time
    start_time = time.time()
    driver = webdriver.Chrome(options=chrome_options)
    elapsed = time.time() - start_time

    if elapsed > 5:
        logging.info(
            f"[CHROME] ✓ Chrome launched in {elapsed:.1f}s (ChromeDriver was downloaded/updated)"
        )
    else:
        logging.info(f"[CHROME] ✓ Chrome launched in {elapsed:.1f}s (using cached ChromeDriver)")

    # MAXIMUM SPEED: Ultra-aggressive timeouts
    driver.set_page_load_timeout(30) # Increased to 30 seconds to fix white screen issues
    driver.implicitly_wait(2) # Increased to 2 seconds for better element detection

    # CRASH RECOVERY: Add page crash detection callback
    try:
        # Enable Chrome DevTools Protocol for crash detection
        driver.execute_cdp_cmd('Page.enable', {})
        logging.info("[STABILITY] Chrome crash detection enabled")
    except Exception as e:
        logging.warning(f"Could not enable crash detection: {e}")

    logging.info(f"Chrome WebDriver initialized successfully for {self.username}")
    return driver

except Exception as e:
    logging.error(f"Failed to initialize Chrome WebDriver: {e}")
    raise Exception(f"ChromeDriver initialization failed: {e}")

def _validate_blueprint_parameters(self, symbol, qty, tp=None, sl=None, prop_firm=None, phase=None,
    account_size=None, strict_mode=True):
    """
    Validates trading parameters against blueprint configuration to prevent incorrect trades.

    Args:
        symbol (str): Trading symbol being used

```

```

    qty (int): Quantity being traded
    tp (int, optional): Take profit ticks
    sl (int, optional): Stop loss ticks
    prop_firm (str, optional): Prop firm name for blueprint lookup
    phase (str, optional): Trading phase (challenge, funded, farming)
    account_size (str, optional): Account size for blueprint lookup
    strict_mode (bool): If True, blocks trades on validation failure. If False, only logs warning

Returns:
    tuple: (is_valid, validation_message, blueprint_config)
"""
try:
    # Import blueprint manager here to avoid circular imports
    from prop_firm_manager import PropFirmManager

    # If no blueprint context provided, try to get from environment
    if not prop_firm:
        prop_firm = os.getenv("SELECTED_PROP_FIRM", "MFFU")
    if not phase:
        phase = os.getenv("TRADING_PHASE", "challenge_trade1")
    if not account_size:
        account_size = os.getenv("ACCOUNT_SIZE", "50k")

    # Get blueprint configuration
    prop_firm_manager = PropFirmManager()
    prop_firm_manager.set_prop_firm(prop_firm)
    blueprint_config = prop_firm_manager.get_prop_firm_strategy_config(phase, account_size)

    if not blueprint_config:
        error_msg = f"[X] BLUEPRINT VALIDATION FAILED: No blueprint config found for {prop_firm}"
        logging.error(error_msg)
        if strict_mode:
            raise Exception(error_msg)
        return False, error_msg, None

    # Validate each parameter against blueprint
    validation_errors = []
    validation_warnings = []

    # Symbol validation
    expected_symbol = blueprint_config.get('tradovate_symbol')
    if expected_symbol and symbol != expected_symbol:
        error_msg = f"[X] SYMBOL MISMATCH: Expected {expected_symbol}, got {symbol}"
        validation_errors.append(error_msg)

    # Quantity validation
    expected_qty = blueprint_config.get('tradovate_qty')
    if expected_qty and qty != expected_qty:
        error_msg = f"[X] QUANTITY MISMATCH: Expected {expected_qty}, got {qty}"
        validation_errors.append(error_msg)

    # Take profit validation (if provided)
    if tp is not None:
        expected_tp = blueprint_config.get('tradovate_tp_ticks')
        if expected_tp and tp != expected_tp:
            error_msg = f"[X] TP MISMATCH: Expected {expected_tp}, got {tp}"
            validation_errors.append(error_msg)

    # Stop loss validation (if provided)
    if sl is not None:

```

```

        expected_sl = blueprint_config.get('tradovate_sl_ticks')
        if expected_sl and sl != expected_sl:
            error_msg = f"[X] SL MISMATCH: Expected {expected_sl}, got {sl}"
            validation_errors.append(error_msg)

    # Log validation results
    if validation_errors:
        full_error_msg = f"[ALARM] BLUEPRINT VALIDATION FAILED for {prop_firm}/{phase}/{account_size}"
        logging.error(full_error_msg)
        if strict_mode:
            raise Exception(full_error_msg)
        return False, full_error_msg, blueprint_config

    if validation_warnings:
        warning_msg = f"[WARNING] BLUEPRINT WARNINGS for {prop_firm}/{phase}/{account_size}: {';'
        logging.warning(warning_msg)

    success_msg = f"[CHECK] BLUEPRINT VALIDATION PASSED for {prop_firm}/{phase}/{account_size}: {';'
    logging.info(success_msg)

    return True, success_msg, blueprint_config

except ImportError:
    warning_msg = "[WARNING] BLUEPRINT VALIDATION SKIPPED: Could not import PropFirmManager"
    logging.warning(warning_msg)
    return True, warning_msg, None
except Exception as e:
    error_msg = f"[X] BLUEPRINT VALIDATION ERROR: {str(e)}"
    logging.error(error_msg)
    if strict_mode:
        raise Exception(error_msg)
    return False, error_msg, None

def _force_trade_override(self, symbol, qty, side, tp=None, sl=None,
    override_reason="Emergency override"):
    """
    Emergency method to force a trade execution bypassing blueprint validation.
    Use only in critical situations where manual override is absolutely necessary.

    Args:
        symbol (str): Trading symbol
        qty (int): Quantity
        side (str): "buy" or "sell"
        tp (int, optional): Take profit ticks
        sl (int, optional): Stop loss ticks
        override_reason (str): Reason for override (logged for audit)

    Returns:
        bool: Success status
    """
    logging.warning(f"[ALARM] EMERGENCY TRADE OVERRIDE: {override_reason}")
    logging.warning(f"[ALARM] OVERRIDE DETAILS: {side.upper()} {symbol} x{qty}, TP={tp}, SL={sl}")

    try:
        # Call the original order placement without validation
        return self._place_order_side_unvalidated(symbol, qty, side, tp, sl)
    except Exception as e:
        logging.error(f"[ALARM] EMERGENCY OVERRIDE FAILED: {e}")
        return False

def _place_order_side_unvalidated(self, symbol, qty, side, tp=None, sl=None, on_click=None,
```

```

skip_positions_refresh=False):
    """
    Original order placement method without blueprint validation.
    Used internally for emergency overrides.
    """
    # This would be the original _place_order_side logic without validation
    # For now, we'll just call the existing method but skip the validation part
    # In a full implementation, this would duplicate the order placement logic

    # Temporarily disable validation by setting environment variable
    original_strict_mode = os.getenv("BLUEPRINT_STRICT_MODE")
    os.environ["BLUEPRINT_STRICT_MODE"] = "false"

    try:
        result = self._place_order_side(symbol, qty, side, tp, sl, on_click, skip_positions_refresh)
        return True
    except Exception as e:
        logging.error(f"[ALARM] UNVALIDATED TRADE FAILED: {e}")
        return False
    finally:
        # Restore original setting
        if original_strict_mode is not None:
            os.environ["BLUEPRINT_STRICT_MODE"] = original_strict_mode
        else:
            os.environ.pop("BLUEPRINT_STRICT_MODE", None)

def get_validation_status(self):
    """
    Get current blueprint validation configuration status.

    Returns:
        dict: Validation configuration and status
    """
    return {
        "strict_mode": os.getenv("BLUEPRINT_STRICT_MODE", "true").lower() == "true",
        "selected_prop_firm": os.getenv("SELECTED_PROP_FIRM", "MFFU"),
        "trading_phase": os.getenv("TRADING_PHASE", "challenge_trade1"),
        "account_size": os.getenv("ACCOUNT_SIZE", "50k"),
        "validation_enabled": True
    }

def login(self):
    """
    Perform full login sequence with robust error handling and retries.
    """
    # Acquire lock to prevent stats fetching during login
    with self.lock:
        if self.logged_in:
            return True

        # --- For testing: use hardcoded credentials if not provided ---
        if not self.username or not self.password:
            self.username = "TYLERTURNER63"
            self.password = "J5140A9013A9553tv="

        try:
            logging.info(f"Starting Tradovate login for user: {self.username}")
            # Always open Tradovate in the first tab
            self.driver.get("https://trader.tradovate.com/welcome")
            self.driver.switch_to.window(self.driver.window_handles[0])

```



```

logging.info("Navigated to Tradovate welcome page")

# Wait for login form or trading mode selection with optimized timeout
try:
    WebDriverWait(self.driver, 10).until( # MAXIMUM SPEED: Reduced from 15 to 10
        EC.any_of(
            EC.visibility_of_element_located((By.ID, "name-input")),
            EC.visibility_of_element_located((By.XPATH,
                "//h1[contains(text(), 'Select a Trading Mode')]"))
        )
    )
    logging.info("Login form or trading mode selection loaded successfully")
except Exception as e:
    logging.error(f"Login form or trading mode selection not found: {e}")
    # Take screenshot for debugging
    try:
        screenshot_path = os.path.join(tempfile.gettempdir(),
            f"tradovate_login_error_{int(time.time())}.png")
        self.driver.save_screenshot(screenshot_path)
        logging.info(f"Error screenshot saved to: {screenshot_path}")
    except:
        pass
    raise Exception(
        "Login form or trading mode selection not accessible. Please check internet connection")

# Check if already at trading mode selection
if self.driver.find_elements(By.XPATH, "//h1[contains(text(), 'Select a Trading Mode')]"):
    logging.info("Already at trading mode selection page, skipping login form.")
else:
    # Fill login form
    try:
        username_field = WebDriverWait(self.driver, 15).until( # Increased timeout
            EC.element_to_be_clickable((By.ID, "name-input"))
        )
        password_field = WebDriverWait(self.driver, 15).until(
            EC.element_to_be_clickable((By.ID, "password-input"))
        )
        logging.info("Login fields found, filling credentials...")

        # Clear and fill username
        username_field.click()
        time.sleep(0.2)
        username_field.clear()
        username_field.send_keys(self.username)
        logging.info("Username filled successfully")

        time.sleep(0.3)

        # Clear and fill password
        password_field.click()
        time.sleep(0.2)
        password_field.clear()
        password_field.send_keys(self.password)
        logging.info("Password filled successfully")

        time.sleep(0.5)

        # Click login button
        login_button = WebDriverWait(self.driver, 15).until(
            EC.element_to_be_clickable((By.XPATH, "//button[.//span[text()='Login']]"))
    )

```

```

    )
    self.driver.execute_script("arguments[0].click();", login_button)
    logging.info("Login button clicked successfully")

    # SPEED OPTIMIZATION: Wait for trading mode selection with optimized timeout
    WebDriverWait(self.driver, 25).until( # Reduced from 45 to 25
        EC.visibility_of_element_located((By.XPATH,
            "//h1[contains(text(), 'Select a Trading Mode')]"))
    )
    logging.info("Successfully reached 'Select a Trading Mode' page")

except Exception as e:
    logging.error(f"Failed to reach trading mode selection page after login: {e}")
    # Check for error messages
    try:
        error_elements = self.driver.find_elements(By.CSS_SELECTOR,
            ".error, .alert, .warning, [class*='error'], [class*='alert']")
        if error_elements:
            error_text = error_elements[0].text
            raise Exception(f"Login failed: {error_text}")
    except:
        pass

    # Take screenshot for debugging
    try:
        screenshot_path = os.path.join(tempfile.gettempdir(),
            f"tradovate_login_failed_{int(time.time())}.png")
        self.driver.save_screenshot(screenshot_path)
        logging.info(f"Login failure screenshot saved to: {screenshot_path}")
    except:
        pass

    raise Exception("Login may have failed - could not reach trading mode selection")

# Dismiss any interstitial pages that may appear before mode selection
self._dismiss_interstitial_pages(timeout=3)

# SPEED OPTIMIZATION: Try trading access with prioritized methods based on mode
connected = False

if self.trading_mode == "Live Trading":
    logging.info("■ STARTING LIVE TRADING MODE")
    access_attempts = [
        self._launch_live_trading_tab
    ]
else:
    logging.info("■ STARTING SIMULATION MODE")
    access_attempts = [
        self._launch_simulation_tab, # Primary method - most reliable
        self._launch_simulation_tab_fallback1, # Most successful fallback
        self._launch_simulation_tab_fallback2, # Secondary fallback
        self._launch_simulation_tab_fallback3, # Less common scenarios
        self._launch_simulation_tab_fallback_different_broker # Last resort
    ]

for attempt_func in access_attempts:
    try:
        logging.info(f"Trying access method: {attempt_func.__name__}")
        attempt_func()

        # Wait for trading interface to load with optimized timeout

```

```

        try:
            WebDriverWait(self.driver, 15).until( # SPEED OPTIMIZATION: Reduced from 30
                EC.visibility_of_element_located((By.XPATH,
                    "//li[contains(@class, 'lm_tab')]//span[text()='Order Ticket']"))
            )
            time.sleep(1) # SPEED OPTIMIZATION: Reduced from 2 to 1
            self._wait_for_no_overlay(timeout=5) # SPEED OPTIMIZATION: Reduced from 10 t

            # Verify connection with account stats
            stats = self.get_account_stats()
            if (stats.get("Account Number") and
                stats.get("Account Number") != "Not Connected" and
                stats.get("Balance") not in ("N/A", "Error", None, "")):

                self.logged_in = True
                logging.info(
                    f"[CHECK] Trading tab launched and account connected - login complete")
                connected = True
                break
            else:
                logging.info(
                    f"Trading interface loaded but account not connected (Account: {stats})")
        except Exception as e:
            logging.error(f"Trading interface failed to load: {e}")
            self.logged_in = False

    except Exception as e:
        logging.warning(f"Access method failed: {e}")
        self.logged_in = False

    if not connected:
        logging.error(f"[X] Could not connect to {self.trading_mode} after all methods.")
        self.logged_in = False
        return False

    return True

except Exception as e:
    logging.error(f"Login process failed: {e}")
    self.logged_in = False
    return False

def _launch_simulation_tab_fallback1(self):
    """Fallback 1: Try clicking the first visible 'Start Simulated Trading', 'Launch' or 'Access Simu
    try:
        # First try Start Simulated Trading button
        try:
            button = WebDriverWait(self.driver, 3).until(
                EC.element_to_be_clickable((By.XPATH, "//button[contains(., 'Start Simulated Trading'
            )
            self.driver.execute_script(
                "arguments[0].scrollIntoView({behavior: 'smooth', block: 'center'});", button)
            time.sleep(0.2)
            button.click()
            logging.info("Fallback1: Clicked 'Start Simulated Trading' button (standard click)")
            return
        except:
            pass

        # Then try Launch button

```

```

try:
    button = WebDriverWait(self.driver, 3).until(
        EC.element_to_be_clickable((By.XPATH, "//button[.//span[text()='Launch']]"))
    )
    self.driver.execute_script(
        "arguments[0].scrollIntoView({behavior: 'smooth', block: 'center'});", button)
    time.sleep(0.2)
    button.click()
    logging.info("Fallback1: Clicked 'Launch' button (standard click)")
    return
except:
    pass

# Then try Access Simulation button
button = WebDriverWait(self.driver, 2).until(
    EC.element_to_be_clickable((By.XPATH, "//button[.//span[text()='Access Simulation']]"))
)
self.driver.execute_script("arguments[0].scrollIntoView({behavior: 'smooth', block: 'center'});", button)
time.sleep(0.2)
button.click()
logging.info("Fallback1: Clicked 'Access Simulation' button (standard click)")
except Exception as e:
    logging.warning(f"Fallback1 failed: {e}")
    raise

def _launch_simulation_tab_fallback2(self):
    """Fallback 2: Try clicking any button with 'Start Simulated Trading', 'Launch' or 'Simulation'
    try:
        # First try Start Simulated Trading
        try:
            button = WebDriverWait(self.driver, 3).until(
                EC.element_to_be_clickable((By.XPATH, "//button[contains(., 'Simulated Trading')]"))
            )
            self.driver.execute_script(
                "arguments[0].scrollIntoView({behavior: 'smooth', block: 'center'});", button)
            time.sleep(0.2)
            button.click()
            logging.info("Fallback2: Clicked button with 'Simulated Trading' text (standard click)")
            return
        except:
            pass

        # Then try Launch
        try:
            button = WebDriverWait(self.driver, 3).until(
                EC.element_to_be_clickable((By.XPATH, "//button[.//span[contains(text(), 'Launch')]]"))
            )
            self.driver.execute_script(
                "arguments[0].scrollIntoView({behavior: 'smooth', block: 'center'});", button)
            time.sleep(0.2)
            button.click()
            logging.info("Fallback2: Clicked button with 'Launch' in span (standard click)")
            return
        except:
            pass

        # Then try Simulation
        button = WebDriverWait(self.driver, 2).until(
            EC.element_to_be_clickable((By.XPATH, "//button[.//span[contains(text(), 'Simulation')]]"))

```

```

    )
    self.driver.execute_script("arguments[0].scrollIntoView({behavior: 'smooth', block: 'center'})"
                                button)
    time.sleep(0.2)
    button.click()
    logging.info("Fallback2: Clicked button with 'Simulation' in span (standard click)")
except Exception as e:
    logging.warning(f"Fallback2 failed: {e}")
    raise

def _launch_simulation_tab_fallback3(self):
    """Fallback 3: Try clicking any button with 'Launch' or 'Simulation' in its text."""
    try:
        # First try Launch
        try:
            button = WebDriverWait(self.driver, 3).until(
                EC.element_to_be_clickable((By.XPATH, "//button[contains(text(), 'Launch')]"))
            )
            self.driver.execute_script(
                "arguments[0].scrollIntoView({behavior: 'smooth', block: 'center'});", button)
            time.sleep(0.2)
            button.click()
            logging.info("Fallback3: Clicked button with 'Launch' in text (standard click)")
            return
        except:
            pass

        # Then try Simulation
        button = WebDriverWait(self.driver, 2).until(
            EC.element_to_be_clickable((By.XPATH, "//button[contains(text(), 'Simulation')]"))
        )
        self.driver.execute_script("arguments[0].scrollIntoView({behavior: 'smooth', block: 'center'})"
                                    button)
        time.sleep(0.2)
        button.click()
        logging.info("Fallback3: Clicked button with 'Simulation' in text (standard click)")
    except Exception as e:
        logging.warning(f"Fallback3 failed: {e}")
        raise

def _fast_input_with_validation(self, element, value, field_name="field"):
    """Enhanced input method using proven Selenium approach that preserves text after typing"""
    try:
        max_attempts = 3
        for attempt in range(max_attempts):
            # Brief delay to let user see the field before we start typing
            time.sleep(0.5 if attempt == 0 else 0.3)
            logging.debug(f"Starting input for {field_name}: {value}")

            # Use proven Selenium approach - click, select all, delete, type character by character
            element.click()
            time.sleep(0.1)
            element.send_keys(Keys.CONTROL + "a")
            element.send_keys(Keys.DELETE)
            time.sleep(0.1)

            # Type character by character
            for char in str(value):
                element.send_keys(char)
                time.sleep(0.05) # Small delay between characters
    
```

```

        time.sleep(0.2) # Brief pause after typing is complete

        # Verify the value is there
        current_value = element.get_attribute("value")
        if current_value == str(value):
            logging.debug(f"[CHECK] {field_name} input successful: {value}")
            return True
        else:
            logging.warning(
                f"[WARNING] {field_name} input attempt {attempt + 1}: expected {value}, got {current_value}"
            )
            if attempt < max_attempts - 1:
                time.sleep(0.5)
                continue

        logging.warning(f"[X] Input for {field_name} failed after {max_attempts} attempts")
        return False

    except Exception as e:
        logging.warning(f"Input with validation failed for {field_name}: {e}")
        return False

def _complete_login_process(self):
    """Complete the login process - verify account access with enhanced stability"""
    try:
        logging.info("Verifying login completion...")

        # SPEED OPTIMIZATION: Fast trading interface verification
        try:
            WebDriverWait(self.driver, 10).until( # Reduced from 20 to 10
                EC.visibility_of_element_located((By.XPATH,
                    "//li[contains(@class, 'lm_tab')]//span[text()='Order Ticket']"))
            )
            logging.info("[CHECK] Trading interface detected and loaded")
        except Exception as e:
            logging.warning(f"Order Ticket tab not found, trying alternative verification: {e}")
            # Alternative verification - look for any tab
            try:
                WebDriverWait(self.driver, 5).until( # Reduced from 10 to 5
                    EC.presence_of_element_located((By.XPATH, "//li[contains(@class, 'lm_tab')]"))
                )
                logging.info("[CHECK] Trading interface tabs detected")
            except Exception as e2:
                logging.warning(f"No tabs found, assuming minimal interface: {e2}")

        # SPEED OPTIMIZATION: Reduced interface stabilization time
        time.sleep(1.5) # Reduced from 3 to 1.5

        # Final verification - try to access account stats with enhanced error handling
        try:
            # Check if driver is still alive before attempting verification
            current_url = self.driver.current_url
            if "tradovate.com" not in current_url:
                raise Exception(f"Browser navigated away from Tradovate: {current_url}")

            # Try to get basic stats but don't fail if it doesn't work
            stats = self.get_account_stats()
            if stats:
                logging.info("[CHECK] Account stats accessible - full verification successful")
            else:
                logging.info("[CHECK] Basic login verified, stats not immediately available")
        
```

```

except Exception as e:
    logging.warning(f"Account stats verification failed but login appears successful: {e}")
    # Don't fail completely - the login might still be successful

    # Mark as logged in if we got this far
    self.logged_in = True
    self._login_timestamp = time.time() # Track when we logged in
    logging.info("[CHECK] Tradovate login process completed successfully")
    print(f"[UNLOCK] Login state set to True at {time.strftime('%H:%M:%S')}")

except Exception as e:
    logging.error(f"Login verification failed: {e}")
    self.logged_in = False
    raise Exception(f"Login verification failed: {str(e)}")

def _launch_live_trading_tab(self):
    """Launch the LIVE TRADING tab after successful authentication"""
    try:
        logging.info("Looking for LIVE TRADING button on 'Select a Trading Mode' page...")

        self._wait_for_no_overlay(timeout=2)
        time.sleep(1)

        # --- Selectors for LIVE TRADING button ---
        live_selectors = [
            # Primary selector provided by user
            "//*[@data-testid='live-trading-button']",

            # Fallbacks
            "//*[@button[contains(., 'Start Trading')]]",
            "//*[@button[./span[text()='Start Trading']]]",
            "//*[@button[contains(text(), 'Start Trading')]]",
            "//*[@button[contains(@class, 'MuiButton-contained') and contains(., 'Start Trading')]]"
        ]

        # Try each selector
        live_button = None
        for selector in live_selectors:
            try:
                buttons = self.driver.find_elements(By.XPATH, selector)
                for button in buttons:
                    if button.is_displayed() and button.is_enabled():
                        live_button = button
                        logging.info(f"Found LIVE TRADING button with selector: {selector}")
                        break
            if live_button:
                break
        except Exception as e:
            logging.debug(f"Selector failed: {selector} ({e})")
            continue

        if not live_button:
            raise Exception("Could not find LIVE TRADING button")

        # Click the button
        logging.info("Clicking LIVE TRADING button...")
        self.driver.execute_script("arguments[0].click();", live_button)

        # Dismiss any interstitial pages (e.g. 'Welcome to Tradovate Prop')
        time.sleep(1)

```

```

self._dismiss_interstitial_pages(timeout=5)

# Wait for interface to load
logging.info("Waiting for LIVE TRADING interface to load...")
WebDriverWait(self.driver, 60).until(
    EC.any_of(
        EC.visibility_of_element_located((By.XPATH,
            "//li[contains(@class, 'lm_tab')]/span[text()='Order Ticket']")),
        EC.visibility_of_element_located((By.XPATH, "//div[contains(@class, 'order-ticket')]"))
    )
)
logging.info("✓ LIVE TRADING interface loaded successfully")
time.sleep(1)
self._wait_for_no_overlay(timeout=10)

except Exception as e:
    logging.error(f"Failed to launch LIVE TRADING tab: {e}")
    raise

def _launch_simulation_tab(self):
    """Launch the simulation tab after successful authentication - enhanced for reliability"""
    try:
        logging.info("Looking for simulation access button on 'Select a Trading Mode' page...")

        self._wait_for_no_overlay(timeout=2) # Reduced from 5 to 2

        # Wait for the page to be fully loaded
        time.sleep(1) # Reduced from 2 to 1

        # --- More comprehensive selectors for simulation button (FASTER ORDER) ---
        simulation_selectors = [
            # NEW: Primary selector provided by user
            "//button[@data-testid='simulation-button']",

            # NEW: Handle 'Start Simulated Trading' button (primary)
            "//button[contains(@class, 'MuiButton-contained') and contains(., 'Start Simulated Trading')]",
            "//button[./span[text()='Start Simulated Trading']]",
            "//button[contains(text(), 'Start Simulated Trading')]",

            # Handle the 'Launch' button text (most common)
            "//button[contains(@class, 'MuiButton-contained') and ./span[text()='Launch']]",
            "//button[contains(@class, 'fat-button') and ./span[text()='Launch']]",
            "//button[./span[text()='Launch']]",
            "//button[contains(text(), 'Launch')]",

            # Handle the different broker scenario (Access Simulation)
            "//button[contains(@class, 'MuiButton-contained') and ./span[text()='Access Simulation']]",
            "//button[contains(@class, 'fat-button') and ./span[text()='Access Simulation']]",

            # Original selectors (keep for compatibility)
            "//button[contains(@class, 'MuiButton-root') and ./span[text()='Access Simulation']]",
            "//button[./span[text()='Access Simulation']]",
            "//button[contains(text(), 'Access Simulation')]",

            # Broader selectors (last resort)
            "//button[./span[contains(text(), 'Access Simulation')]]",
            "//button[contains(@class, 'MuiButton-root') and ./span[contains(text(), 'Simulation')]]",
            "//button[./span[contains(text(), 'Simulated Trading')]]",
            "//button[contains(translate(text(), 'ABCDEFGHIJKLMNOPQRSTUVWXYZ', 'abcdefghijklmnopqrstuv'), 'ABCDEFGHIJKLMNOPQRSTUVWXYZ', 'abcdefghijklmnopqrstuv')]",
            "//button[contains(translate(text(), 'ABCDEFGHIJKLMNOPQRSTUVWXYZ', 'abcdefghijklmnopqrstuv'), 'ABCDEFGHIJKLMNOPQRSTUVWXYZ', 'abcdefghijklmnopqrstuv')]"
        ]
    
```



```

]

# Try each selector (FASTER - reduced timeout)
simulation_button = None
for selector in simulation_selectors:
    try:
        buttons = self.driver.find_elements(By.XPATH, selector)
        for button in buttons:
            if button.is_displayed() and button.is_enabled():
                simulation_button = button
                logging.info(f"Found simulation button with selector: {selector}")
                break
        if simulation_button:
            break
    except Exception as e:
        logging.debug(f"Selector failed: {selector} ({e})")
        continue

if not simulation_button:
    # Try fallbacks faster - skip debugging for speed
    logging.warning("Direct selectors failed, trying fallbacks quickly")
    self._try_all_simulation_fallbacks_fast()
    return

# Try click methods (FASTER - fewer attempts)
click_success = False
for attempt in range(3): # Reduced from 5 to 3
    if click_success:
        break

    try:
        logging.info(f"Attempt {attempt+1} to click simulation button")

        if attempt == 0:
            # First try: JavaScript click (most reliable)
            self.driver.execute_script("arguments[0].click();", simulation_button)
            click_success = True
            logging.info("Simulation button clicked with JavaScript click")
        elif attempt == 1:
            # Second try: standard click
            simulation_button.click()
            click_success = True
            logging.info("Simulation button clicked with standard click")
        else:
            # Third try: ActionChains
            ActionChains(self.driver).move_to_element(simulation_button).click().perform()
            click_success = True
            logging.info("Simulation button clicked with ActionChains")
    except Exception as e:
        logging.warning(f"Click attempt {attempt+1} failed: {e}")
        time.sleep(0.5) # Reduced from 1 to 0.5

if not click_success:
    # Try fallbacks quickly
    logging.warning("All direct click methods failed, trying fallbacks")
    self._try_all_simulation_fallbacks_fast()
    return

# Dismiss any interstitial pages (e.g. 'Welcome to Tradovate Prop')
time.sleep(1)

```

```

self._dismiss_interstitial_pages(timeout=5)

# Wait for simulation interface to load - REDUCED TIMEOUT
# Give 1 minute for the page to load after clicking (reduced from 5 minutes)
logging.info("Waiting for simulation interface to load (up to 60 seconds)...")
try:
    WebDriverWait(self.driver, 60).until( # Reduced from 300 to 60 seconds
        EC.any_of(
            EC.visibility_of_element_located((By.XPATH,
                "//li[contains(@class, 'lm_tab')]/span[text()='Order Ticket']")),
            EC.visibility_of_element_located((By.XPATH,
                "//div[contains(@class, 'order-ticket')]")),
            EC.visibility_of_element_located((By.XPATH, "//span[text()='Order Ticket']"))
        )
    )
    logging.info("✓ Simulation trading interface loaded successfully")
    time.sleep(1) # Reduced from 2 to 1 second
    self._wait_for_no_overlay(timeout=10)
except Exception as e:
    logging.warning(f"Trading interface loading verification timed out after 60 seconds: {e}")
    logging.info("Interface may still be loading, but simulation access was clicked")

except Exception as e:
    logging.error(f"Failed to launch simulation tab: {e}")
    self._try_all_simulation_fallbacks_fast()

def _try_all_simulation_fallbacks_fast(self):
    """Try all simulation fallbacks in sequence - FASTER version"""
    fallbacks = [
        self._launch_simulation_tab_fallback_different_broker, # NEW: Add the different broker fallback
        self._launch_simulation_tab_fallback1,
        self._launch_simulation_tab_fallback2,
        self._launch_simulation_tab_fallback3
    ]

    for i, fallback in enumerate(fallbacks):
        try:
            logging.info(f"Trying simulation fallback method {i+1}")
            fallback()
            # If we get here without exception, assume success
            logging.info(f"Simulation fallback method {i+1} succeeded")
            return
        except Exception as e:
            logging.warning(f"Simulation fallback method {i+1} failed: {e}")

    # If all fallbacks fail, raise exception
    raise Exception(
        "All simulation access methods failed. Please check the trading platform UI or try manual log"
    )

def _launch_simulation_tab_fallback_different_broker(self):
    """NEW: Fallback for different broker with specific button structure - supports both Launch and A
    try:
        # First try Launch button (most common)
        try:
            button = WebDriverWait(self.driver, 3).until(
                EC.element_to_be_clickable((
                    By.XPATH,
                    "//button[contains(@class, 'MuiButton-contained')] and contains(@class, 'fat-butto"
                ))
            )

```

```

        self.driver.execute_script("arguments[0].click();", button)
        logging.info("Different broker: Clicked 'Launch' button with JavaScript")
        time.sleep(2)
        return
    except:
        pass

    # Then try Access Simulation button structure
    try:
        button = WebDriverWait(self.driver, 2).until(
            EC.element_to_be_clickable((
                By.XPATH,
                "//button[contains(@class, 'MuiButton-contained') and contains(@class, 'fat-button')]
            ))
        )
        self.driver.execute_script("arguments[0].click();", button)
        logging.info("Different broker: Clicked 'Access Simulation' button with JavaScript")
        time.sleep(2)
        return
    except:
        pass

    # Try alternative selectors for Launch
    try:
        button = WebDriverWait(self.driver, 2).until(
            EC.element_to_be_clickable((
                By.XPATH,
                "//button[contains(@class, 'fat-button') and contains(@class, 'full-width-button')]
            ))
        )
        self.driver.execute_script("arguments[0].click();", button)
        logging.info("Different broker: Clicked Launch with alternative selector")
        time.sleep(2)
        return
    except:
        pass

    # Try alternative selectors for Access Simulation
    try:
        button = WebDriverWait(self.driver, 2).until(
            EC.element_to_be_clickable((
                By.XPATH,
                "//button[contains(@class, 'fat-button') and contains(@class, 'full-width-button')]
            ))
        )
        self.driver.execute_script("arguments[0].click();", button)
        logging.info("Different broker: Clicked Access Simulation with alternative selector")
        time.sleep(2)
        return
    except:
        pass

    raise Exception("No Launch or Access Simulation buttons found")

except Exception as e:
    logging.warning(f"Different broker fallback failed: {e}")
    raise Exception(f"Different broker simulation access failed: {e}")

def _launch_simulation_tab_fallback_new(self):
    """New fallback: Try tabbing to the button and using Enter key."""

```

```

try:
    # First try to find any button that might be the simulation button
    buttons = self.driver.find_elements(By.TAG_NAME, "button")
    simulation_button = None

    for button in buttons:
        try:
            if button.is_displayed() and button.is_enabled():
                text = button.text.lower()
                if 'simulation' in text or 'access' in text or 'launch' in text:
                    simulation_button = button
                    break
        except:
            continue

    if simulation_button:
        # Try to focus and press Enter
        self.driver.execute_script("arguments[0].focus();", simulation_button)
        time.sleep(1)
        ActionChains(self.driver).send_keys(Keys.RETURN).perform()
        logging.info("Pressed Enter on focused simulation/launch button")
        time.sleep(3)
        return

    # If no button found, try tabbing and pressing enter
    body = self.driver.find_element(By.TAG_NAME, "body")
    body.click() # Focus on body

    # Press tab multiple times to try to reach the button
    action = ActionChains(self.driver)
    for _ in range(10): # Try up to 10 tabs
        action.send_keys(Keys.TAB).perform()
        time.sleep(0.5)

    # Try pressing Enter after each tab
    action.send_keys(Keys.RETURN).perform()
    time.sleep(1)

    # Check if we've reached a new page
    try:
        if WebDriverWait(self.driver, 2).until(
            EC.presence_of_element_located((By.XPATH,
                "//div[contains(@class, 'trading-interface') or contains(@class, 'chart')]"))
        ):
            logging.info("New page detected after tab+enter, likely successful")
            return
    except:
        continue

    raise Exception("Tab navigation failed to find simulation button")
except Exception as e:
    logging.warning(f"New fallback failed: {e}")
    raise

def _debug_page_elements(self):
    """Enhanced debugging to identify available page elements"""
    try:
        logging.info("=== ENHANCED DEBUGGING: Page Elements Analysis ===")
        try:
            current_url = self.driver.current_url

```

```

        page_title = self.driver.title
        logging.info(f"Current URL: {current_url}")
        logging.info(f"Page Title: {page_title}")

        # Take screenshot for debugging
        screenshot_path = os.path.join(tempfile.gettempdir(),
                                       f"tradovate_debug_{int(time.time())}.png")
        self.driver.save_screenshot(screenshot_path)
        logging.info(f"Debug screenshot saved to: {screenshot_path}")
    except:
        pass

    # Capture and log all buttons
    try:
        buttons = self.driver.find_elements(By.TAG_NAME, "button")
        logging.info(f"=== All Buttons Found ({len(buttons)} total) ===")
        for i, button in enumerate(buttons[:20]): # Log first 20 buttons
            try:
                text = button.text.strip()
                classes = button.get_attribute("class") or ""
                visible = button.is_displayed()
                enabled = button.is_enabled()
                status = f"'✓' if visible else 'X'visible, {'✓' if enabled else 'X'}enabled"
                if text:
                    logging.info(
                        f"Button {i+1}: '{text}' | Classes: {classes[:50]}... | Status: {status}"
                    )
                elif classes:
                    logging.info(
                        f"Button {i+1}: [no text] | Classes: {classes[:50]}... | Status: {status}"
                    )
            except Exception as e:
                logging.debug(f"Button {i+1}: Error reading - {e}")

        # Find the most likely simulation button
        for button in buttons:
            try:
                if button.is_displayed() and button.is_enabled():
                    text = button.text.lower()
                    if 'simulation' in text or 'access simulation' in text:
                        rect = button.rect
                        logging.info(f"LIKELY TARGET: '{button.text}' at position {rect}")
            except:
                continue
    except:
        pass

    except Exception as debug_e:
        logging.error(f"Debug logging failed: {debug_e}")

    def _wait_for_no_overlay(self, timeout=3):
        try:
            WebDriverWait(self.driver, timeout).until(
                EC.invisibility_of_element_located((By.CSS_SELECTOR, ".spinner, .loading, .overlay"))
            )
        except Exception:
            pass

    def _dismiss_interstitial_pages(self, timeout=5):
        """Dismiss intermittent interstitial pages that appear before the trading interface.

        Handles:

```

```

- 'Welcome to Tradovate Prop' full-screen modal with Continue button
- Cookie consent banners with Accept Cookies button
- Any other MUI dialog with a Continue/OK/Accept button
"""
deadline = time.time() + timeout
dismissed_any = False

while time.time() < deadline:
    found = False

    # 1) Cookie consent banner
    try:
        cookie_btns = self.driver.find_elements(By.XPATH,
            "//button[contains(translate(., 'ABCDEFGHIJKLMNOPQRSTUVWXYZ', 'abcdefghijklmnopqrstuvwxyz'))"
        )
        for btn in cookie_btns:
            if btn.is_displayed() and btn.is_enabled():
                self.driver.execute_script("arguments[0].click();", btn)
                logging.info("Dismissed cookie consent banner")
                found = True
                dismissed_any = True
                break
    except Exception:
        pass

    # 2) Full-screen 'Welcome to Tradovate Prop' modal – Continue button
    try:
        continue_btns = self.driver.find_elements(By.XPATH,
            "//div[contains(@class, 'MuiDialog-paperFullScreen')]"
            "//button[contains(translate(., 'ABCDEFGHIJKLMNOPQRSTUVWXYZ', 'abcdefghijklmnopqrstuvwxyz'))"
        )
        for btn in continue_btns:
            if btn.is_displayed() and btn.is_enabled():
                self.driver.execute_script("arguments[0].click();", btn)
                logging.info("Dismissed 'Welcome to Tradovate Prop' interstitial (Continue)")
                found = True
                dismissed_any = True
                time.sleep(1)
                break
    except Exception:
        pass

    # 3) Generic MUI dialog with Continue / OK / Accept / Got it / Close
    if not found:
        try:
            generic_btns = self.driver.find_elements(By.XPATH,
                "//div[contains(@class, 'MuiDialog-root') or contains(@class, 'MuiModal-root')]"
                "//button[contains(translate(., 'ABCDEFGHIJKLMNOPQRSTUVWXYZ', 'abcdefghijklmnopqrstuvwxyz'))"
                " or contains(translate(., 'ABCDEFGHIJKLMNOPQRSTUVWXYZ', 'abcdefghijklmnopqrstuvwxyz'))"
                " or contains(translate(., 'ABCDEFGHIJKLMNOPQRSTUVWXYZ', 'abcdefghijklmnopqrstuvwxyz'))"
                " or contains(translate(., 'ABCDEFGHIJKLMNOPQRSTUVWXYZ', 'abcdefghijklmnopqrstuvwxyz'))"
                " or contains(translate(., 'ABCDEFGHIJKLMNOPQRSTUVWXYZ', 'abcdefghijklmnopqrstuvwxyz'))"
            )
            for btn in generic_btns:
                if btn.is_displayed() and btn.is_enabled():
                    self.driver.execute_script("arguments[0].click();", btn)
                    logging.info(f"Dismissed generic MUI dialog (button text: {btn.text.strip()})")
                    found = True
                    dismissed_any = True
                    time.sleep(0.5)

```

```

        break
    except Exception:
        pass

    if not found:
        break # No interstitial detected, stop polling
    time.sleep(0.5)

    if dismissed_any:
        logging.info("Interstitial page(s) dismissed successfully")
    return dismissed_any

def close(self):
    """Close the browser and cleanup"""
    try:
        if self.driver:
            self.driver.quit()
            logging.info("Tradovate browser closed")
    except Exception as e:
        logging.error(f"Error closing browser: {e}")
    finally:
        if self.logged_in:
            print(f"[LOCK] Login state changed to False (browser closed) at {time.strftime('%H:%M:%S')}")
        self.logged_in = False
        self._login_timestamp = None
        self.driver = None

def get_account_stats(self):
    """Optimized stats - return cached during order placement OR if recently fetched"""
    # Use lock to prevent conflict with order placement
    if not self.lock.acquire(blocking=False):
        # If locked (e.g. placing order), return cached stats immediately
        return getattr(self, '_cached_stats', {
            "Account Number": "Trading...",
            "Balance": "N/A",
            "Profit/Loss": "N/A",
            "Open Trades": "N/A",
            "Symbol": "",
            "Direction": ""
        })

    try:
        if getattr(self, '_placing_order', False):
            return getattr(self, '_cached_stats', {
                "Account Number": "Trading...",
                "Balance": "N/A",
                "Profit/Loss": "N/A",
                "Open Trades": "N/A",
                "Symbol": "",
                "Direction": ""
            })

        # PERFORMANCE OPTIMIZATION: Return cached stats if fetched within the last 2 seconds
        # This prevents excessive DOM queries when GUI polls every 1 second
        cache_ttl = 2.0 # Cache time-to-live in seconds
        current_time = time.time()
        last_fetch_time = getattr(self, '_stats_last_fetch_time', 0)
        time_since_fetch = current_time - last_fetch_time

        if time_since_fetch < cache_ttl and hasattr(self, '_cached_stats'):

```

```

        # Return cached stats (still fresh)
        return self._cached_stats

# Always try to fetch stats from the live page, even if not self.logged_in
try:
    if not self.driver:
        return {
            "Account Number": "Not Connected",
            "Balance": "N/A",
            "Profit/Loss": "N/A",
            "Open Trades": "N/A",
            "Symbol": "",
            "Direction": ""
        }

    stats = {
        "Account Number": "Unknown",
        "Balance": "N/A",
        "Profit/Loss": "N/A",
        "Open Trades": "0",
        "Symbol": "",
        "Direction": ""
    }

    # Check for Order Ticket tab quickly
    try:
        order_ticket_tabs = self.driver.find_elements(
            By.XPATH, "//li[contains(@class, 'lm_tab')]/span[text()='Order Ticket']"
        )
        if not order_ticket_tabs:
            stats["Account Number"] = "Not Connected"
            return stats
    except Exception:
        stats["Account Number"] = "Not Connected"
        return stats

    # --- Try to extract account number from visible elements ---
    try:
        account_elements = self.driver.find_elements(By.CSS_SELECTOR,
            "[class*='account'], [data-testid*='account']")
        for el in account_elements:
            text = el.text.strip()
            if text and any(char.isdigit() for char in text):
                lines = text.splitlines()
                for line in lines:
                    if line.strip() and any(char.isdigit() for char in line):
                        acc = line.strip()
                        if len(acc) >= 9:
                            stats["Account Number"] = acc[:4] + "..." + acc[-5:]
                        else:
                            stats["Account Number"] = acc
                        break
                if stats["Account Number"] != "Unknown":
                    break
    except Exception:
        pass

    # --- Try to extract balance ---
    try:
        balance_elements = self.driver.find_elements(By.CSS_SELECTOR,

```



```

        "[class*='balance'], [class*='equity']")
    for el in balance_elements:
        balance_text = el.text
        import re
        balance_match = re.search(r'[\$]?([0-9,]+\.[0-9]*)', balance_text)
        if balance_match:
            stats["Balance"] = f"${balance_match.group(1)}"
            break
    except Exception:
        pass

    # --- Try to extract open trades ---
    try:
        open_trades_elements = self.driver.find_elements(By.XPATH,
            "//span[contains(text(),'Open') and contains(text(),'Trade')]")
        for el in open_trades_elements:
            text = el.text
            import re
            match = re.search(r'(\d+)', text)
            if match:
                stats["Open Trades"] = match.group(1)
                break
    except Exception:
        pass

    # Cache the stats WITH TIMESTAMP for performance optimization
    self._cached_stats = stats
    self._stats_last_fetch_time = time.time()
    return stats

except Exception as e:
    logging.warning(f"Error fetching stats: {e}")
    return {
        "Account Number": "Error",
        "Balance": "N/A",
        "Profit/Loss": "N/A",
        "Open Trades": "N/A",
        "Symbol": "",
        "Direction": ""
    }
finally:
    self.lock.release()

def refresh_positions_tab(self):
    """Refresh the Tradovate Positions tab to update open trades info."""
    try:
        driver = self.driver
        positions_tab = WebDriverWait(driver, 2).until(
            EC.element_to_be_clickable((By.XPATH,
                "//li[contains(@class, 'lm_tab')]//span[text()='Positions']"))
        )
        positions_tab.click()
        self._wait_for_no_overlay(timeout=0.5)
    except Exception:
        pass

def prepare_buy_order(self, symbol, qty, tp=None, sl=None):
    """
    THREADING OPTIMIZATION: Prepare BUY order (symbol search, quantity, ATM) WITHOUT clicking button.
    Use this before threading to minimize time inside parallel execution.

```

```

Returns: True if preparation successful, False otherwise
"""
with self.lock:
    try:
        print(f"■ PRE-THREAD] Preparing Tradovate BUY order: {symbol} x{qty}")

        # Quick interface check
        current_url = self.driver.current_url
        if not current_url or "tradovate" not in current_url:
            print(f"■ PRE-THREAD] Not on Tradovate page, logging in...")
            self.login()

        # Prepare order (slow operations done before threading)
        self._wait_for_no_overlay(timeout=0.2)
        self._select_symbol(symbol)
        self._set_quantity(qty)
        self._setup_atm(tp, sl)

        print(f"■ [■ PRE-THREAD] Order prepared - ready for button click")
        return True

    except Exception as e:
        print(f"■ PRE-THREAD] Preparation failed: {e}")
        return False

def execute_prepared_buy_order(self, skip_positions_refresh=False):
    """
    THREADING OPTIMIZATION: Execute already-prepared BUY order (just click button).
    Call prepare_buy_order() first, then use this inside threading for instant execution.

    Returns: None (raises exception on failure)
    """
    with self.lock:
        try:
            print(f"■ THREAD-EXEC] Clicking BUY button...")
            self._do_place_order("buy", on_click=None, skip_positions_refresh=skip_positions_refresh)
            print(f"■ [■ THREAD-EXEC] BUY button clicked")
        except Exception as e:
            print(f"■ THREAD-EXEC] Execution failed: {e}")
            raise

def prepare_sell_order(self, symbol, qty, tp=None, sl=None):
    """
    THREADING OPTIMIZATION: Prepare SELL order (symbol search, quantity, ATM) WITHOUT clicking button
    Use this before threading to minimize time inside parallel execution.

    Returns: True if preparation successful, False otherwise
    """
    with self.lock:
        try:
            print(f"■ PRE-THREAD] Preparing Tradovate SELL order: {symbol} x{qty}")

            # Quick interface check
            current_url = self.driver.current_url
            if not current_url or "tradovate" not in current_url:
                print(f"■ PRE-THREAD] Not on Tradovate page, logging in...")
                self.login()

            # Prepare order (slow operations done before threading)
            self._wait_for_no_overlay(timeout=0.2)

```

```

        self._select_symbol(symbol)
        self._set_quantity(qty)
        self._setup_atm(tp, sl)

        print(f"■ [■ PRE-THREAD] Order prepared - ready for button click")
        return True

    except Exception as e:
        print(f"■ [■ PRE-THREAD] Preparation failed: {e}")
        return False

def execute_prepared_sell_order(self, skip_positions_refresh=False):
    """
    THREADING OPTIMIZATION: Execute already-prepared SELL order (just click button).
    Call prepare_sell_order() first, then use this inside threading for instant execution.

    Returns: None (raises exception on failure)
    """
    with self.lock:
        try:
            print(f"■ [■ THREAD-EXEC] Clicking SELL button...")
            self._do_place_order("sell", on_click=None, skip_positions_refresh=skip_positions_refresh)
            print(f"■ [■ THREAD-EXEC] SELL button clicked")
        except Exception as e:
            print(f"■ [■ THREAD-EXEC] Execution failed: {e}")
            raise

def get_active_account(self):
    """Get the full (unmasked) account number currently active in the Tradovate UI.

    DOM structure:
    div.account-selector-wrapper
    div.account-selector.dropdown
    div[data-toggle="dropdown"]
    div.account > div.name > div ← contains account number
    ul.dropdown-menu
    li.selected > a.account > div.name > div.main ← selected account

    Returns:
    str: The account number string (e.g. "FTDFYSLX50349776193"), or None.
    """
    if not self.driver:
        return None
    try:
        # Primary: Read from the header account-selector name element
        name_els = self.driver.find_elements(
            By.CSS_SELECTOR, "div.account-selector div.name div")
        for el in name_els:
            try:
                text = el.text.strip()
                if text and any(c.isdigit() for c in text) and len(text) >= 6:
                    return text
            except Exception:
                continue

        # Fallback: Read from the selected dropdown item
        selected_els = self.driver.find_elements(
            By.CSS_SELECTOR, "ul.dropdown-menu li.selected div.main")
        for el in selected_els:
            try:

```

```

        text = el.text.strip()
        if text and any(c.isdigit() for c in text):
            return text
    except Exception:
        continue

    # Last resort: Any element with class containing 'account' that has digits
    account_elements = self.driver.find_elements(
        By.CSS_SELECTOR, "[class*='account']")
    for el in account_elements:
        text = el.text.strip()
        if text and any(c.isdigit() for c in text):
            for line in text.splitlines():
                line = line.strip()
                if line and any(c.isdigit() for c in line) and len(line) >= 6:
                    return line

    except Exception as e:
        logging.warning(f"get_active_account failed: {e}")
    return None

def get_all_accounts(self):
    """Open the account dropdown and return a list of all account names.

    Returns:
        list[str]: Account name strings from the dropdown, or empty list on failure.
    """
    if not self.driver:
        return []

    try:
        # Open the dropdown
        triggers = self.driver.find_elements(
            By.CSS_SELECTOR, "div.account-selector [data-toggle='dropdown']")
        opened = False
        for trigger in triggers:
            try:
                if trigger.is_displayed():
                    self.driver.execute_script("arguments[0].click();", trigger)
                    time.sleep(1.0)
                    opened = True
                    break
            except Exception:
                continue

        if not opened:
            # Fallback: click account-selector itself
            selectors = self.driver.find_elements(By.CSS_SELECTOR, "div.account-selector")
            for sel in selectors:
                try:
                    if sel.is_displayed():
                        self.driver.execute_script("arguments[0].click();", sel)
                        time.sleep(1.0)
                        opened = True
                        break
                except Exception:
                    continue

        if not opened:
            print("[GET_ALL_ACCOUNTS] Could not open dropdown")

```

```

        return []

    # Read all account entries
    account_links = self.driver.find_elements(
        By.CSS_SELECTOR, "div.account-selector ul.dropdown-menu li a.account:not(.logout)")

    names = []
    for link in account_links:
        try:
            main_divs = link.find_elements(By.CSS_SELECTOR, "div.name div.main")
            if main_divs:
                name = main_divs[0].text.strip()
            else:
                name = link.text.strip().split('\n')[0].strip()
            if name:
                names.append(name)
        except Exception:
            continue

    # Close dropdown safely with Escape key (never click body – could hit Logout)
    try:
        from selenium.webdriver.common.keys import Keys
        self.driver.find_element(By.TAG_NAME, "body").send_keys(Keys.ESCAPE)
        time.sleep(0.3)
    except Exception:
        # Fallback: re-click the trigger to toggle dropdown closed
        try:
            for trigger in triggers:
                if trigger.is_displayed():
                    self.driver.execute_script("arguments[0].click();", trigger)
                    time.sleep(0.3)
                    break
        except Exception:
            pass

    print(f"[GET_ALL_ACCOUNTS] Found {len(names)} accounts: {names}")
    return names

except Exception as e:
    logging.warning(f"get_all_accounts failed: {e}")
    return []

def switch_account(self, target_account):
    """Switch to a specific sub-account via Tradovate's account dropdown.

    DOM structure (from live Tradovate page):
    div.account-selector-wrapper
    div.account-selector.dropdown
    div[data-toggle="dropdown"]      ← CLICK to open dropdown
    ul.dropdown-menu                ← dropdown list
    li > a.account                  ← each account entry (skip a.logout)
    div.name > div.main              ← account number text

    Args:
        target_account: Account number string (or unique substring) to select.

    Returns:
        bool: True if account was switched (or already active), False on failure.
    """
    if not self.driver or not target_account:

```

```

        return False

import re as _re
target_account = str(target_account).strip()
target_lower = target_account.lower()
digit_match = _re.search(r'(\d{5,})$', target_account)
digit_suffix = digit_match.group(1) if digit_match else None
print(f"[ACCOUNT SWITCH] Target: {target_account} (suffix: {digit_suffix})")

try:
    # ■■ Check if already on the correct account ■■
    current = self.get_active_account()
    if current:
        cur_lower = current.lower()
        if target_lower in cur_lower or cur_lower in target_lower:
            print(f"[ACCOUNT SWITCH] Already on correct account: {current}")
            return True
        if digit_suffix and current.endswith(digit_suffix):
            print(f"[ACCOUNT SWITCH] Already correct (suffix match): {current}")
            return True

    print(f"[ACCOUNT SWITCH] Current: {current} → Need: {target_account}")

    max_attempts = 3
    for attempt in range(1, max_attempts + 1):
        if attempt > 1:
            print(f"[ACCOUNT SWITCH] ■■ Retry {attempt}/{max_attempts} ■■")
            time.sleep(1)

        # ■■ STEP 1: Click div[data-toggle="dropdown"] inside div.account-selector to open ■■
        dropdown_opened = False
        triggers = self.driver.find_elements(
            By.CSS_SELECTOR, "div.account-selector [data-toggle='dropdown']")
        for trigger in triggers:
            try:
                if trigger.is_displayed():
                    print(f"[ACCOUNT SWITCH] STEP 1: Clicking div[data-toggle=dropdown]")
                    self.driver.execute_script("arguments[0].click();", trigger)
                    time.sleep(1.5)
                    dropdown_opened = True
                    break
            except Exception:
                continue

        # Fallback: click the account-selector div itself
        if not dropdown_opened:
            selectors = self.driver.find_elements(
                By.CSS_SELECTOR, "div.account-selector")
            for sel in selectors:
                try:
                    if sel.is_displayed():
                        print(f"[ACCOUNT SWITCH] STEP 1 fallback: Clicking div.account-selector")
                        self.driver.execute_script("arguments[0].click();", sel)
                        time.sleep(1.5)
                        dropdown_opened = True
                        break
                except Exception:
                    continue

        if not dropdown_opened:

```

```

        print(f"[ACCOUNT SWITCH] Could not open dropdown (attempt {attempt})")
        continue

# ■■■ STEP 2: Read all accounts from ul.dropdown-menu ■■■
# Each account: li > a.account (not .logout) > div.name > div.main
account_links = self.driver.find_elements(
    By.CSS_SELECTOR, "div.account-selector ul.dropdown-menu li a.account:not(.logout)")

print(f"[ACCOUNT SWITCH] STEP 2: Found {len(account_links)} account entries in dropdown")

matched_link = None
for link in account_links:
    try:
        # Get the account name from div.main inside this <a>
        main_divs = link.find_elements(By.CSS_SELECTOR, "div.name div.main")
        if main_divs:
            acct_name = main_divs[0].text.strip()
        else:
            acct_name = link.text.strip().split('\n')[0].strip()

        is_selected = False
        try:
            parent_li = link.find_element(By.XPATH, "..")
            is_selected = "selected" in (parent_li.get_attribute("class") or "")
        except Exception:
            pass

        marker = " ← CURRENT" if is_selected else ""
        print(f" - {acct_name}{marker}")

        # Match: exact substring or suffix
        if target_lower in acct_name.lower() or acct_name.lower() in target_lower:
            if not is_selected:
                matched_link = link
                print(f"[ACCOUNT SWITCH] MATCH (exact): {acct_name}")
                break
            elif digit_suffix and acct_name.endswith(digit_suffix):
                if not is_selected:
                    matched_link = link
                    print(f"[ACCOUNT SWITCH] MATCH (suffix {digit_suffix}): {acct_name}")
                    break
    except Exception as item_err:
        logging.debug(f"[ACCOUNT SWITCH] Error reading dropdown item: {item_err}")
        continue

if not matched_link:
    print(f"[ACCOUNT SWITCH] '{target_account}' not found in dropdown (attempt {attempt})")
    # Close dropdown before retry
    try:
        from selenium.webdriver.common.keys import Keys as _Keys
        self.driver.find_element(By.TAG_NAME, 'body').send_keys(_Keys.ESCAPE)
        time.sleep(0.3)
    except Exception:
        pass
    continue

# ■■■ STEP 3: Click the matched account link ■■■
print(f"[ACCOUNT SWITCH] STEP 3: Clicking matched account (attempt {attempt})")

# Try multiple click methods – JS click often doesn't work on <a> elements

```

```

click_succeeded = False

# Method 1: Selenium native .click() (most reliable for <a> links)
try:
    matched_link.click()
    click_succeeded = True
    print(f"[ACCOUNT SWITCH] Used native .click()")
except Exception as e1:
    print(f"[ACCOUNT SWITCH] Native click failed: {e1}")

# Method 2: ActionChains click
if not click_succeeded:
    try:
        ActionChains(self.driver).move_to_element(matched_link).click().perform()
        click_succeeded = True
        print(f"[ACCOUNT SWITCH] Used ActionChains click")
    except Exception as e2:
        print(f"[ACCOUNT SWITCH] ActionChains click failed: {e2}")

# Method 3: JS click on the <a> element
if not click_succeeded:
    try:
        self.driver.execute_script("arguments[0].click();", matched_link)
        click_succeeded = True
        print(f"[ACCOUNT SWITCH] Used JS click")
    except Exception as e3:
        print(f"[ACCOUNT SWITCH] JS click failed: {e3}")

# Method 4: JS click on div.main inside the link
if not click_succeeded:
    try:
        inner = matched_link.find_elements(By.CSS_SELECTOR, "div.name div.main")
        if inner:
            self.driver.execute_script("arguments[0].click();", inner[0])
            click_succeeded = True
            print(f"[ACCOUNT SWITCH] Used JS click on inner div.main")
    except Exception as e4:
        print(f"[ACCOUNT SWITCH] Inner click failed: {e4}")

if not click_succeeded:
    print(f"[ACCOUNT SWITCH] ALL click methods failed (attempt {attempt})")
    continue

time.sleep(2) # wait for account switch

# ■■■ STEP 4: VERIFY the account actually changed ■■■
new_account = self.get_active_account()
if new_account:
    new_lower = new_account.lower()
    if target_lower in new_lower or new_lower in target_lower:
        print(f"[ACCOUNT SWITCH] ■ VERIFIED – now on: {new_account}")
        self._stats_last_fetch_time = 0
        return True
    elif digit_suffix and new_account.endswith(digit_suffix):
        print(f"[ACCOUNT SWITCH] ■ VERIFIED (suffix) – now on: {new_account}")
        self._stats_last_fetch_time = 0
        return True
    else:
        print(
            f"[ACCOUNT SWITCH] ■ Account did NOT change (attempt {attempt}). Still on: {"

```



```

        self._stats_last_fetch_time = 0
        # Will retry on next iteration
    else:
        print(f"[ACCOUNT SWITCH] ■ Could not read account after click (attempt {attempt})")
        self._stats_last_fetch_time = 0
        # Will retry on next iteration

    # All attempts exhausted
    print(f"[ACCOUNT SWITCH] ■ FAILED after {max_attempts} attempts for '{target_account}'")
    return False

except Exception as e:
    logging.error(f"[ACCOUNT SWITCH] Exception: {e}")
    try:
        self.driver.find_element(By.TAG_NAME, 'body').send_keys(Keys.ESCAPE)
    except Exception:
        pass
    return False

def verify_active_account(self, expected_account):
    """Verify that the currently active account matches the expected one.

    Args:
        expected_account: Expected account number (or substring).

    Returns:
        bool: True if the active account matches, False otherwise.
    """
    if not expected_account:
        return True # No expectation set, skip check

    current = self.get_active_account()
    if not current:
        logging.warning(f"[ACCOUNT] Cannot verify – unable to read active account")
        return False

    expected_lower = str(expected_account).strip().lower()
    if expected_lower in current.lower():
        logging.info(f"[ACCOUNT] Verified: active={current}, expected={expected_account}")
        return True

    logging.warning(f"[ACCOUNT] MISMATCH: active={current}, expected={expected_account}")
    return False

def buy_market(self, symbol=None, qty=1, tp=None, sl=None, on_click=None, skip_positions_refresh=False,
               prop_firm=None, phase=None, account_size=None, expected_account=None):
    self._place_order_side(symbol, qty, "buy", tp, sl, on_click=on_click,
                           skip_positions_refresh=skip_positions_refresh, prop_firm=prop_firm, phase=phase, account_size=account_size)

def sell_market(self, symbol=None, qty=1, tp=None, sl=None, on_click=None, skip_positions_refresh=False,
                prop_firm=None, phase=None, account_size=None, expected_account=None):
    self._place_order_side(symbol, qty, "sell", tp, sl, on_click=on_click,
                           skip_positions_refresh=skip_positions_refresh, prop_firm=prop_firm, phase=phase, account_size=account_size)

def _set_quantity(self, qty):
    print(f"[ZAP] Setting quantity: {qty}")
    try:
        qty_input = WebDriverWait(self.driver, 3).until(
            EC.element_to_be_clickable((By.XPATH,
                                         "//div[contains(@class, 'trading-ticket-main-entry')]/div[contains(@class, 'qty')]"))
    
```

```

    )

    # SPEED OPTIMIZATION: Check if quantity is already correct first
    current_qty = qty_input.get_attribute("value") or ""
    if current_qty.strip() == str(qty).strip():
        print(f"[ZAP] Quantity {qty} already set - skipping update")
        return

    # Update quantity with proper input clearing delays
    qty_input.click()
    time.sleep(0.1) # Increased delay for reliable clearing
    qty_input.send_keys(Keys.CONTROL + "a")
    qty_input.send_keys(Keys.DELETE)
    time.sleep(0.1) # Increased delay before typing new value
    qty_input.send_keys(str(qty))
    time.sleep(0.05) # Brief delay after typing
    print(f"[ZAP] Quantity updated to {qty}")

except Exception as e:
    print(f"[WARNING] Quantity setting failed: {e}")
    raise

def _setup_atm(self, tp=None, sl=None):
    """Optimized ATM setup with smart input checking"""
    tp = tp if tp is not None else (os.getenv("TRADOVATE_TAKEPROFIT_TICKS") or DEFAULT_TP)
    sl = sl if sl is not None else (os.getenv("TRADOVATE_STOPLOSS_TICKS") or DEFAULT_SL)
    print(f"[ZAP] Setting ATM - TP: {tp}, SL: {sl}")

    try:
        inputs = self.driver.find_elements(By.CSS_SELECTOR, "div.numeric-input input.form-control")
        if len(inputs) < 2:
            print(f"[WARNING] Only found {len(inputs)} ATM inputs, skipping ATM setup")
            return

        tp_input = inputs[0]
        sl_input = inputs[1]

        # SPEED OPTIMIZATION: Check both inputs first, only update if needed
        current_tp = tp_input.get_attribute("value") or ""
        current_sl = sl_input.get_attribute("value") or ""

        tp_needs_update = current_tp.strip() != str(tp).strip()
        sl_needs_update = current_sl.strip() != str(sl).strip()

        if not tp_needs_update and not sl_needs_update:
            print(f"[ZAP] ATM already configured: TP={tp}, SL={sl} - skipping update")
            return

        # Only update TP if needed
        if tp_needs_update:
            tp_input.click()
            time.sleep(0.15) # Increased delay for reliable clearing
            tp_input.send_keys(Keys.CONTROL + "a")
            tp_input.send_keys(Keys.DELETE)
            time.sleep(0.1) # Increased delay before typing
            tp_input.send_keys(str(tp))
            time.sleep(0.15) # Increased delay after typing TP
            # Tab to next field to confirm TP entry
            tp_input.send_keys(Keys.TAB)
            time.sleep(0.2) # Wait for tab to process

```

```

        print(f"[ZAP] TP updated to {tp} and tabbed to SL field")
    else:
        print(f"[ZAP] TP {tp} already correct")

    # Only update SL if needed
    if sl_needs_update:
        sl_input.click()
        time.sleep(0.15) # Increased delay for reliable clearing
        sl_input.send_keys(Keys.CONTROL + "a")
        sl_input.send_keys(Keys.DELETE)
        time.sleep(0.1) # Increased delay before typing
        sl_input.send_keys(str(sl))
        time.sleep(0.15) # Increased delay after typing
        # Tab out to confirm SL entry
        sl_input.send_keys(Keys.TAB)
        time.sleep(0.2) # Wait for tab to process
        print(f"[ZAP] SL updated to {sl} and tabbed out")
    else:
        print(f"[ZAP] SL {sl} already correct")

    print(f"[ZAP] ATM setup complete: TP={tp}, SL={sl}")

except Exception as e:
    print(f"[WARNING] ATM setup failed: {e}")
    import traceback
    traceback.print_exc()
    # Don't raise - continue without ATM if it fails

def _detect_account_size(self, stats):
    """
    Heuristic detection of account size based on balance.
    Returns normalized string like "50k", "100k", or None.
    """
    try:
        balance_str = stats.get("Balance", "N/A")
        if not balance_str or balance_str == "N/A":
            return None

        # Clean string: "$50,230.50" -> 50230.50
        clean_bal = balance_str.replace("$", "").replace(",", "").strip()
        bal = float(clean_bal)

        # Define ranges (balance +/- variance) with strict boundaries
        # 25k TIER: 20k-38k
        if 20000 <= bal <= 38000: return "25k"
        # 50k TIER: 40k-65k
        if 40000 <= bal <= 65000: return "50k"
        # 100k TIER: 90k-115k
        if 90000 <= bal <= 115000: return "100k"
        # 150k TIER: 140k-165k
        if 140000 <= bal <= 165000: return "150k"
        # 200k TIER: 190k-215k
        if 190000 <= bal <= 215000: return "200k"
        # 250k TIER: 240k-265k
        if 240000 <= bal <= 265000: return "250k"
        # 300k TIER: 290k-315k
        if 290000 <= bal <= 315000: return "300k"

        return None
    except Exception:

```

```

        return None

def _place_order_side(self, symbol, qty, side, tp=None, sl=None, on_click=None,
    skip_positions_refresh=False, prop_firm=None, phase=None, account_size=None, expected_account=None)
    """
    Place a market order with enhanced crash detection and recovery
    """
    # Acquire lock to prevent stats fetching during order placement
    with self.lock:
        # --- ACCOUNT NUMBER VERIFICATION ---
        if expected_account:
            expected_account = str(expected_account).strip()
            current_account = self.get_active_account()
            if current_account:
                if expected_account.lower() not in current_account.lower() and current_account.lower()
                    ) not in expected_account.lower():
                    # Also check numeric suffix match
                    import re as _re
                    expected_digits = _re.search(r'(\d{5,})$', expected_account)
                    suffix_match = expected_digits and current_account.endswith(expected_digits.group(0))
                    if not suffix_match:
                        print(
                            f"[ACCOUNT] Active '{current_account}' ≠ expected '{expected_account}'. S
                        )
                        switched = self.switch_account(expected_account)
                        if switched:
                            # Confirm the switch actually worked
                            confirmed = self.get_active_account()
                            print(f"[ACCOUNT] ■ Switched – confirmed active: {confirmed}")
                        else:
                            error_msg = f"[BLOCK] Account switch to '{expected_account}' failed – st
                            print(error_msg)
                            raise Exception(error_msg)
                    else:
                        print(
                            f"[ACCOUNT] ■ Suffix match: active '{current_account}' matches expected
                        )
                else:
                    print(f"[ACCOUNT] ■ Verified: '{current_account}' matches '{expected_account}'")
            else:
                print(
                    f"[ACCOUNT] ■ Could not read active account. Attempting switch to '{expected_acc
                )
                switched = self.switch_account(expected_account)
                if not switched:
                    print(f"[ACCOUNT] ■ Switch attempt failed. Proceeding anyway.")
        # --- ACCOUNT SIZE PROTECTION ---
        # Try to populate account_size from env if missing, for safety
        check_size = account_size or os.getenv("ACCOUNT_SIZE", "50k")

        if check_size:
            try:
                # Get fresh stats (this is safe because _block_stats_fetching hasn't been called yet)
                current_stats = self.get_account_stats()
                detected_size = self._detect_account_size(current_stats)

                if detected_size:
                    # Normalize blueprint size (e.g. "50k" -> "50k", "$50,000" -> "50k")
                    bp_size_norm = str(check_size).lower().replace("$", "").replace(",", "")

                    # Handle numerical inputs like "50000"
                    if bp_size_norm.isdigit():
                        val = int(bp_size_norm)

```

```

        elif 20000 <= val <= 38000: bp_size_norm = "25k"
        elif 45000 <= val <= 55000: bp_size_norm = "50k"
        elif 95000 <= val <= 105000: bp_size_norm = "100k"
        elif 145000 <= val <= 155000: bp_size_norm = "150k"
        elif 195000 <= val <= 205000: bp_size_norm = "200k"
        elif 245000 <= val <= 255000: bp_size_norm = "250k"
        elif 295000 <= val <= 305000: bp_size_norm = "300k"

    # Only compare if we successfully normalized to a "k" string or we are comparing
    # If bp_size_norm is still "60000" (weird size), detected "50k" won't match.

    # Compare
    if detected_size != bp_size_norm:
        error_msg = f"[BLOCK] ACCOUNT SIZE MISMATCH! Blueprint requires {bp_size_norm}"
        print(error_msg)
        raise Exception(error_msg)
    else:
        print(
            f"[CHECK] Account size OK: Blueprint {bp_size_norm} matches Account {detected_size}"
        )
    else:
        print(
            f"[WARNING] Could not detect account size from balance ({current_stats.get('balance')})"
        )
    except Exception as size_check_err:
        if "ACCOUNT SIZE MISMATCH" in str(size_check_err):
            raise # Re-raise blocking error
        print(f"[WARNING] Account size check failed: {size_check_err}")

# CRITICAL DEBUG: Log the symbol BEFORE any fallback logic
print(
    f"[SYMBOL DEBUG] _place_order_side RECEIVED symbol parameter: '{symbol}' (type: {type(symbol)})"
)
print(f"[SYMBOL DEBUG] DEFAULT_SYMBOL = '{DEFAULT_SYMBOL}'")
print(f"[SYMBOL DEBUG] prop_firm={prop_firm}, phase={phase}, account_size={account_size}")

original_symbol = symbol
symbol = symbol or DEFAULT_SYMBOL

if original_symbol != symbol:
    print(
        f"[SYMBOL WARNING] Symbol was None/empty, fell back to DEFAULT_SYMBOL: '{original_symbol}'"
    )
else:
    print(f"[SYMBOL OK] Using provided symbol: '{symbol}'")

print(f"[SEARCH] _place_order_side DEBUG: Placing {side.upper()} order: {symbol} x{qty}")
print(f"[SEARCH] _place_order_side DEBUG: Current logged_in state: {self.logged_in}")

# ■■■ API-FIRST: Try placing via REST API (fast, no UI interaction) ■■■
try:
    print(f"[API-FIRST] Attempting REST API order: {side.upper()} {qty}x {symbol}")

    # Resolve account_id from expected_account name
    api_account_id = None
    if expected_account:
        accounts = self._api_fetch("/account/list")
        if accounts and isinstance(accounts, list):
            expected_lower = str(expected_account).lower()
            for acct in accounts:
                acct_name = acct.get('name', '').lower()
                if expected_lower in acct_name or acct_name in expected_lower:
                    api_account_id = acct['id']
            print(

```

```

        f"[API-FIRST] Matched account '{acct.get('name')}' (id={api_account_id})"
        break
    # Also try numeric suffix match
    import re as _re
    expected_digits = _re.search(r'(\d{5,})$', str(expected_account))
    if expected_digits and acct_name.endswith(expected_digits.group(1).lower()):
        api_account_id = acct['id']
        print(
            f"[API-FIRST] Suffix-matched account '{acct.get('name')}' (id={api_account_id})"
        )
        break
    if not api_account_id:
        print(f"[API-FIRST] ■ No account match for '{expected_account}' – using default")

    api_result = self.place_order_api(
        symbol=symbol, side=side, qty=qty,
        order_type="Market", tp=tp, sl=sl,
        account_id=api_account_id,
    )

    if api_result:
        order_id = api_result.get('orderId', api_result.get('id', '?'))
        status = api_result.get('ordStatus', '?')
        print(
            f"[API-FIRST] ■ API order SUCCESS: id={order_id} status={status} – Selenium skip"
        )
        if on_click:
            try:
                on_click()
            except Exception:
                pass
        return # API succeeded – done!
    else:
        print(f"[API-FIRST] ■ API returned None – falling back to Selenium")
    except Exception as api_err:
        print(f"[API-FIRST] ■ API attempt failed: {api_err} – falling back to Selenium")

# ■■■ SELENIUM FALLBACK: Place via UI clicking ■■■
print(f"[SELENIUM FALLBACK] Proceeding with Selenium-based order placement")

# [ALARM] BLUEPRINT VALIDATION DISABLED: Using input timing instead
print(f"[CHECK] TRADE APPROVED: Blueprint validation disabled - using input timing")

# Add input clearing delays for reliable value entry
import time
time.sleep(0.2) # Brief delay to ensure inputs are ready

# Add browser health check before starting
try:
    current_url = self.driver.current_url
    if "data:" in current_url or not current_url or "tradovate" not in current_url:
        raise Exception("Browser is in invalid state before order placement")
except Exception as health_error:
    print(f"Browser health check failed: {health_error}")
    raise Exception("Browser crashed or disconnected")

# Block other actions - important to prevent UI interference during order placement
self._block_stats_fetching(True)
try:
    # SPEED OPTIMIZATION: Enhanced login check for manual trading
    need_login = False
    login_reason = ""

```

```

# Debug current login state
login_age = "never" if not self._login_timestamp else f"{int(time.time() - self._login_timestamp)}s"
print(f"[SEARCH] LOGIN STATE DEBUG: logged_in={self.logged_in}, last_login={login_age}")

if not self.logged_in:
    # SMART LOGIN CHECK: Verify if interface is actually unavailable before forcing login
    print("[SEARCH] SMART CHECK: logged_in=False but verifying interface availability...")
    try:
        current_url = self.driver.current_url
        if "trader.tradovate.com" in current_url:
            # Check if trading interface is actually accessible
            order_ticket_available = self.driver.find_elements(
                By.XPATH, "//li[contains(@class, 'lm_tab')]/span[text()='Order Ticket']"
            )
            if order_ticket_available:
                print(
                    "[SETUP] SMART CHECK SUCCESS: Interface available despite logged_in=False"
                )
                self.logged_in = True
                self._login_timestamp = time.time()
            else:
                need_login = True
                login_reason = "logged_in flag is False and Order Ticket not accessible"
        else:
            need_login = True
            login_reason = f"logged_in flag is False and not on trading page (URL: {current_url})"
    except Exception as check_error:
        need_login = True
        login_reason = f"logged_in flag is False and interface check failed: {check_error}"
    else:
        # For manual trades, do a quick interface check instead of full login
        print("[ZAP] FAST MANUAL: Checking trading interface availability...")
        try:
            # Quick check: Can we access the trading interface directly?
            current_url = self.driver.current_url
            if not current_url or "tradovate" not in current_url:
                need_login = True
                login_reason = f"not on Tradovate page (URL: {current_url})"
            else:
                # Check if Order Ticket tab is accessible
                order_ticket_visible = self.driver.find_elements(
                    By.XPATH, "//li[contains(@class, 'lm_tab')]/span[text()='Order Ticket']"
                )
                if order_ticket_visible:
                    print(
                        "[ZAP] FAST MANUAL: Order Ticket tab accessible - proceeding directly"
                    )
                else:
                    print(
                        "[ZAP] FAST MANUAL: Order Ticket not visible - may need interface refresh"
                    )
                    # Try a quick navigation to trading page instead of full login
                    if "trader" not in current_url:
                        print("[ZAP] FAST MANUAL: Navigating to trading interface...")
                        self.driver.get("https://trader.tradovate.com/")
                        time.sleep(2) # Brief wait for page load
        except Exception as interface_error:
            need_login = True
            login_reason = f"interface check failed: {interface_error}"

# Only perform login if actually needed
if need_login:
    print(f"[WARNING] Login required: {login_reason}")

```

```

        # Proceed directly to full login without page refresh
        print("[REFRESH] Performing login...")
        self.login()
    else:
        print("[ZAP] FAST MANUAL: Login check passed - proceeding with trade execution")

        self._wait_for_no_overlay(timeout=0.2)
        self._select_symbol(symbol)
        self._set_quantity(qty)
        self._setup_atm(tp, sl)
        self._do_place_order(side, on_click=on_click, skip_positions_refresh=skip_positions_refre

except Exception as e:
    # Check if it's a browser crash
    try:
        current_url = self.driver.current_url
        if "data:" in current_url or not current_url:
            print(f"Browser crashed during {side} order placement")
            raise Exception(f"Browser crashed during {side} order - cannot continue")
    except:
        print(f"Browser completely unresponsive during {side} order")
        raise Exception(f"Browser crashed during {side} order - cannot continue")
    raise
finally:
    # Ensure stats fetching is unblocked even if an exception occurs
    self._block_stats_fetching(False)

def _block_stats_fetching(self, block=True):
    """Set a flag to block stats fetching during order placement."""
    self._placing_order = block

def _select_symbol(self, symbol):
    print(f"Selecting symbol: {symbol}")
    self._wait_for_no_overlay()

# SPEED OPTIMIZATION: Try direct symbol selection first (for fast manual trading)
try:
    print(f"[ZAP] FAST PATH: Attempting direct symbol selection...")
    # Check if symbol input is already accessible without tab switching
    symbol_input = WebDriverWait(self.driver, 2).until(
        EC.element_to_be_clickable((By.XPATH,
            "//div[contains(@class, 'trading-ticket-main-entry')]/input[contains(@class, 'search
        )

    # Check if symbol is already correctly set
    current_symbol = symbol_input.get_attribute("value") or ""
    if current_symbol.strip().upper() == symbol.upper():
        print(f"[ZAP] FAST PATH SUCCESS: Symbol {symbol} already selected, skipping")
        return

    # Symbol input is accessible, proceed with direct selection
    symbol_input.click()
    time.sleep(0.1) # Reduced delay for speed
    symbol_input.send_keys(Keys.CONTROL + "a")
    symbol_input.send_keys(Keys.DELETE)
    time.sleep(0.1)

    # Type symbol with moderate delays to ensure dropdown appears
    for char in symbol:
        symbol_input.send_keys(char)

```



```

        time.sleep(0.1) # Moderate typing speed for dropdown stability

# Wait for dropdown and click option directly - EXACT MATCH to avoid selecting MNQM6 when loc
# Try exact match first (most reliable)
try:
    dropdown_option = WebDriverWait(self.driver, 3).until(
        EC.visibility_of_element_located((
            By.XPATH,
            f"//ul[contains(@class, 'dropdown-menu')]//a[starts-with(normalize-space(.), '{symbol}')"
        ))
    )
    self.driver.execute_script("arguments[0].click();", dropdown_option)
    print(f"[ZAP] FAST PATH SUCCESS: Symbol {symbol} selected directly (exact match)")
    self._wait_for_no_overlay()
    return
except:
    # Fallback: If exact match fails, try contains but click the first matching option
    print(f"[ZAP] Exact match failed, trying contains match...")
    dropdown_option = WebDriverWait(self.driver, 2).until(
        EC.visibility_of_element_located((
            By.XPATH,
            f"//ul[contains(@class, 'dropdown-menu')]//a[contains(text(), '{symbol}')] or .//a"
        ))
    )
    self.driver.execute_script("arguments[0].click();", dropdown_option)
    print(f"[ZAP] FAST PATH SUCCESS: Symbol {symbol} selected directly (contains match)")
    self._wait_for_no_overlay()
    return

except Exception as fast_error:
    print(f"[ZAP] Fast path failed: {fast_error}, falling back to tab navigation...")

# ORIGINAL PATH: Full tab navigation for when fast path fails
max_attempts = 5
for attempt in range(max_attempts):
    try:
        # Step 1: Click Positions tab
        positions_tab = WebDriverWait(self.driver, 10).until(
            EC.element_to_be_clickable((By.XPATH,
                f"//li[contains(@class, 'lm_tab')]//span[text()='Positions']"))
        )
        positions_tab.click()
        time.sleep(0.3)
    except Exception as e:
        print(f"Warning: Positions tab interaction failed: {e}")

    try:
        # Step 2: Click Order Ticket tab
        order_ticket_tab = WebDriverWait(self.driver, 10).until(
            EC.element_to_be_clickable((By.XPATH,
                f"//li[contains(@class, 'lm_tab')]//span[text()='Order Ticket']"))
        )
        order_ticket_tab.click()
        time.sleep(0.3)
    except Exception as e:
        print(f"Error: Order Ticket tab not accessible: {e}")
        raise

    try:
        # Step 3: Find and interact with symbol input

```

```

symbol_input = WebDriverWait(self.driver, 10).until(
    EC.element_to_be_clickable((By.XPATH,
        "//div[contains(@class, 'trading-ticket-main-entry')]//input[contains(@class, 'se
    )
symbol_input.click()
time.sleep(0.3)
symbol_input.send_keys(Keys.CONTROL + "a")
symbol_input.send_keys(Keys.DELETE)
time.sleep(0.3)

# Type symbol character by character with slower speed for dropdown stability
for char in symbol:
    symbol_input.send_keys(char)
    time.sleep(0.15) # Slower typing to ensure dropdown appears

# Step 4: Wait for and click dropdown option - EXACT MATCH to avoid selecting MNQM6 when
try:
    # Try exact match first (more reliable)
    dropdown_option = WebDriverWait(self.driver, 5).until(
        EC.visibility_of_element_located((
            By.XPATH,
            f"//ul[contains(@class, 'dropdown-menu')]//a[starts-with(normalize-space(.),
        ))
    )
    self.driver.execute_script("arguments[0].click();", dropdown_option)
    print(f"Symbol {symbol} selected from dropdown (exact match)")
    break # Success!
except:
    # Fallback: try contains match
    print(f"Exact match failed, trying contains match for {symbol}...")
    dropdown_option = WebDriverWait(self.driver, 5).until(
        EC.visibility_of_element_located((
            By.XPATH,
            f"//ul[contains(@class, 'dropdown-menu')]//a[contains(text(), '{symbol}')] or
        ))
    )
    self.driver.execute_script("arguments[0].click();", dropdown_option)
    print(f"Symbol {symbol} selected from dropdown (contains match)")
    break # Success!

except Exception as e:
    print(f"Attempt {attempt+1} to select symbol failed: {e}")
    if attempt == max_attempts - 1:
        raise Exception(
            "Symbol input not found or could not select symbol after several attempts")
    time.sleep(1)

self._wait_for_no_overlay()

def _recover_from_symbol_failure(self):
    """Recovery method: Click Positions tab, then Order Ticket tab, then clear symbol field"""
    try:
        print("[REFRESH] Starting symbol selection recovery...")

        # Step 1: Click Positions tab
        try:
            positions_tab = WebDriverWait(self.driver, 3).until(
                EC.element_to_be_clickable((By.XPATH,
                    "//li[contains(@class, 'lm_tab')]//span[text()='Positions']"))
            )

```

```

        positions_tab.click()
        print("[CHECK] Clicked Positions tab")
        time.sleep(0.5)
    except Exception as e:
        print(f"[WARNING] Could not click Positions tab: {e}")

    # Step 2: Click Order Ticket tab
    try:
        order_ticket_tab = WebDriverWait(self.driver, 3).until(
            EC.element_to_be_clickable((By.XPATH,
                "//li[contains(@class, 'lm_tab')]//span[text()='Order Ticket']"))
        )
        order_ticket_tab.click()
        print("[CHECK] Clicked Order Ticket tab")
        time.sleep(0.5)
    except Exception as e:
        print(f"[WARNING] Could not click Order Ticket tab: {e}")

    # Step 3: Clear symbol field
    try:
        symbol_input = WebDriverWait(self.driver, 3).until(
            EC.visibility_of_element_located((By.XPATH,
                "//div[contains(@class, 'trading-ticket-main-entry')]//input[contains(@class, 'se')]"))
        )
        symbol_input.click()
        time.sleep(0.2)
        symbol_input.send_keys(Keys.CONTROL + "a")
        symbol_input.send_keys(Keys.DELETE)
        print("[CHECK] Cleared symbol field")
        time.sleep(0.3)
    except Exception as e:
        print(f"[WARNING] Could not clear symbol field: {e}")

    print("[REFRESH] Recovery completed")

except Exception as recovery_error:
    print(f"[X] Recovery failed: {recovery_error}")

def _do_place_order(self, side, on_click=None, skip_positions_refresh=False):
    """Optimized order placement with better error handling"""
    self._placing_order = True

    try:
        # Find buy/sell button with better error handling
        try:
            side_button = WebDriverWait(self.driver, 5).until(
                EC.element_to_be_clickable((By.XPATH,
                    f"//label[contains(translate(text(), 'ABCDEFGHIJKLMNOPQRSTUVWXYZ', 'abcdefghijklmnopqrstuvwxyz'), '{side.lower()}')]"))
            )
            current_class = side_button.get_attribute("class") or ""
            if "active" not in current_class:
                self.driver.execute_script("arguments[0].click();", side_button)
                print(f"{side.upper()} button selected")
            else:
                print(f"{side.upper()} button already active")
        except Exception as e:
            print(f"Failed to select {side} button: {e}")
            raise

        # Click Send button with retry logic

```

```

try:
    send_button = WebDriverWait(self.driver, 5).until(
        EC.element_to_be_clickable((By.XPATH,
            "//button[contains(@class, 'btn-primary') and text()='Send']"))
    )
    self.driver.execute_script("arguments[0].click();", send_button)
    print("Send button clicked")
except Exception as e:
    print(f"Failed to click Send button: {e}")
    raise

# Brief wait for any confirmation dialog to render (appears intermittently)
time.sleep(0.5)

# Detect confirmation dialog by container classes OR "DO NOT SHOW" marker text
confirm_containers = self.driver.find_elements(By.XPATH,
    "//div[contains(@class, 'popover') or contains(@class, 'modal') or contains(@class, 'confirm"
    " or contains(@class, 'dialog') or contains(@class, 'overlay'))]"
)
do_not_show_markers = self.driver.find_elements(By.XPATH,
    "//*[contains(translate(text(), 'ABCDEFGHIJKLMNOPQRSTUVWXYZ', 'abcdefghijklmnopqrstuvwxyz'),"
)
dialog_detected = bool(confirm_containers or do_not_show_markers)

if dialog_detected:
    print("Confirmation dialog detected")

    # Check "Do not show again" checkbox if present
    try:
        do_not_show = self.driver.find_element(By.XPATH,
            "//input[@type='checkbox' and (following-sibling::*[contains(translate(text(), "
            "'ABCDEFGHIJKLMNOPQRSTUVWXYZ', 'abcdefghijklmnopqrstuvwxyz'), 'do not show'])"
            " or ancestor::label[contains(translate(text(), "
            "'ABCDEFGHIJKLMNOPQRSTUVWXYZ', 'abcdefghijklmnopqrstuvwxyz'), 'do not show'])]]"
        )
        if not do_not_show.is_selected():
            self.driver.execute_script("arguments[0].click();", do_not_show)
            print("'Do not show again' checkbox checked")
    except Exception:
        # Try broader search for the checkbox near "do not show" text
        try:
            do_not_show = self.driver.find_element(By.XPATH,
                "//*[contains(translate(text(), 'ABCDEFGHIJKLMNOPQRSTUVWXYZ', 'abcdefghijklmnopqrstuvwxyz'),"
                "'do not show')]/preceding-sibling::input[@type='checkbox']"
                " | //*[contains(translate(text(), 'ABCDEFGHIJKLMNOPQRSTUVWXYZ', 'abcdefghijklmnopqrstuvwxyz'),"
                "'do not show')]/following-sibling::input[@type='checkbox']"
                " | //*[contains(translate(text(), 'ABCDEFGHIJKLMNOPQRSTUVWXYZ', 'abcdefghijklmnopqrstuvwxyz'),"
                "'do not show')]/ancestor::label//input[@type='checkbox']"
                " | //*[contains(translate(text(), 'ABCDEFGHIJKLMNOPQRSTUVWXYZ', 'abcdefghijklmnopqrstuvwxyz'),"
                "'do not show')]/parent::*//input[@type='checkbox']]"
            )
            if not do_not_show.is_selected():
                self.driver.execute_script("arguments[0].click();", do_not_show)
                print("'Do not show again' checkbox checked (alt)")
        except Exception:
            print("No 'Do not show again' checkbox found")

    # Click the confirmation Buy/Sell button (matches current trade side)
    confirmed = False

    # Try side-specific button inside known container first

```

```

try:
    side_confirm = self.driver.find_element(By.XPATH,
        f"//div[contains(@class,'popover') or contains(@class,'modal') or contains(@class,"
        f" or contains(@class,'dialog') or contains(@class,'overlay'))]"
        f"//div[contains(@class,'btn') and contains(translate(text(),' "
        f"'ABCDEFGHIJKLMNOPQRSTUVWXYZ','abcdefghijklmnopqrstuvwxyz'),'{side.lower()}')]"
        f" | //div[contains(@class,'popover') or contains(@class,'modal') or contains(@c"
        f" or contains(@class,'dialog') or contains(@class,'overlay'))]"
        f"//button[contains(translate(text(),' "
        f"'ABCDEFGHIJKLMNOPQRSTUVWXYZ','abcdefghijklmnopqrstuvwxyz'),'{side.lower()}')]"
    )
    self.driver.execute_script("arguments[0].click();", side_confirm)
    print(f"Confirmation {side.upper()} button clicked (in container)")
    confirmed = True
except Exception:
    pass

# Fallback: side-specific button near "Cancel" (the order ticket has no Cancel)
if not confirmed:
    try:
        side_confirm = self.driver.find_element(By.XPATH,
            f"//button[contains(translate(text(),'ABCDEFGHIJKLMNOPQRSTUVWXYZ','abcdefghijklmnopqrstuvwxyz"
            f" and following-sibling::button[contains(translate(text(),'ABCDEFGHIJKLMNOPQRSTUVWXYZ"
            f" | //button[contains(translate(text(),'ABCDEFGHIJKLMNOPQRSTUVWXYZ','abcdefghijklmnopqrstuvwxyz"
            f" and preceding-sibling::button[contains(translate(text(),'ABCDEFGHIJKLMNOPQRSTUVWXYZ"
            f" | //div[contains(@class,'btn') and contains(translate(text(),'ABCDEFGHIJKLMNOPQRSTUVWXYZ"
            f" and (following-sibling::*[contains(translate(text(),'ABCDEFGHIJKLMNOPQRSTUVWXYZ"
            f" or preceding-sibling::*[contains(translate(text(),'ABCDEFGHIJKLMNOPQRSTUVWXYZ"
        )
        self.driver.execute_script("arguments[0].click();", side_confirm)
        print(f"Confirmation {side.upper()} button clicked (near Cancel)")
        confirmed = True
    except Exception:
        pass

# Fallback: OK / Confirm / Continue / Yes buttons
if not confirmed:
    try:
        fallback_btn = self.driver.find_element(By.XPATH,
            "//button[contains(translate(text(),'ABCDEFGHIJKLMNOPQRSTUVWXYZ','abcdefghijklmnopqrstuvwxyz"
            " or contains(translate(text(),'ABCDEFGHIJKLMNOPQRSTUVWXYZ','abcdefghijklmnopqrstuvwxyz"
            " or contains(translate(text(),'ABCDEFGHIJKLMNOPQRSTUVWXYZ','abcdefghijklmnopqrstuvwxyz"
            " or contains(translate(text(),'ABCDEFGHIJKLMNOPQRSTUVWXYZ','abcdefghijklmnopqrstuvwxyz"
            " | //div[contains(@class,'btn-success') or contains(@class,'btn-primary'))]"
        )
        self.driver.execute_script("arguments[0].click();", fallback_btn)
        print("Confirmation dialog handled (fallback)")
        confirmed = True
    except Exception:
        pass

if not confirmed:
    print("Confirmation dialog found but no button matched")

# Dismiss any post-order notification/toast that may block the UI
try:
    dismiss_btn = WebDriverWait(self.driver, 1).until(
        EC.element_to_be_clickable((By.XPATH,
            "//button[contains(@class,'close') or @aria-label='Close' or @aria-label='Dismiss"
            " | //div[contains(@class,'toast')]//button"
        ))
    )

```



```

        }} catch(e) {{
            cb({{ok: false, error: e.toString()}});
        }}
    }})();
    """
    try:
        result = self.driver.execute_async_script(js)
        if result and result.get('ok'):
            return result.get('data')
        # Log the actual error instead of silently returning None
        status = result.get('status', '?') if result else '?'
        err = result.get('error', '') if result else 'no result'
        data = result.get('data', '') if result else ''
        print(f"[API] fetch {endpoint} FAILED: status={status} error={err} data={data}")
        return None
    except Exception as e:
        print(f"[API] fetch {endpoint} exception: {e}")
        return None

def _get_account_for_api(self):
    """Get the first account id and name for API order placement."""
    accounts = self._api_fetch("/account/list")
    if not accounts or not isinstance(accounts, list) or len(accounts) == 0:
        return None, None
    acct = accounts[0]
    return acct['id'], acct.get('name', '?')

def get_min_equity(self, account_id=None):
    """Get the minimum equity (drawdown limit) for an account via the API.

    Uses /cashBalance/getCashBalanceSnapshot for current net liq,
    /userAccountAutoLiq/deps for trailing drawdown limits, and
    /accountRiskStatus/list for the max net liq (trailing reference).

    Returns:
    dict with 'net_liq', 'min_equity', 'drawdown_remaining' or None on failure.
    """
    try:
        if not account_id:
            account_id, _ = self._get_account_for_api()
        if not account_id:
            return None

        snapshot = self._api_fetch("/cashBalance/getCashBalanceSnapshot", "POST", {
            "accountId": account_id}) or {}
        net_liq = snapshot.get('netLiq', 0)
        net_liq_sod = snapshot.get('netLiqSOD', 0) # balance at midnight

        # Use deps endpoint – returns the autoLiq entry directly for this account
        al_raw = self._api_fetch(f"/userAccountAutoLiq/deps?masterid={account_id}") or {}
        # deps can return a single object or a list with one entry
        if isinstance(al_raw, list):
            al = al_raw[0] if al_raw else {}
        else:
            al = al_raw

        trailing_max_drawdown = al.get('trailingMaxDrawdown', 0) or 0
        trailing_max_drawdown_limit = al.get('trailingMaxDrawdownLimit', 0) or 0
        trailing_mode = al.get('trailingMaxDrawdownMode', '')

        print(f"[MIN EQUITY] autoLiq raw for {account_id}: TMD={trailing_max_drawdown}, "

```

```

        f"TMDL={trailing_max_drawdown_limit}, mode={trailing_mode}, keys={list(al.keys())}")

# Get max net liq from accountRiskStatus (the trailing reference point)
max_net_liq = 0
risk_raw = self._api_fetch(f"/accountRiskStatus/deps?masterid={account_id}") or {}
if isinstance(risk_raw, list):
    rs = risk_raw[0] if risk_raw else {}
else:
    rs = risk_raw
max_net_liq = rs.get('maxNetLiq', 0) or 0

# EOD trailing drawdown:
# SL floor from SOD: midnight_balance - trailing_max_drawdown
# Absolute floor:    trailingMaxDrawdownLimit - trailing_max_drawdown
# min_equity = max(sod_floor, absolute_floor)
if trailing_max_drawdown > 0:
    absolute_floor = (
        trailing_max_drawdown_limit - trailing_max_drawdown) if trailing_max_drawdown_limit > 0
    if net_liq_sod > 0:
        sod_floor = net_liq_sod - trailing_max_drawdown
        min_equity = max(sod_floor, absolute_floor)
    else:
        min_equity = absolute_floor
else:
    min_equity = 0

drawdown_remaining = net_liq - min_equity if min_equity else net_liq

result = {
    'net_liq': net_liq,
    'net_liq_sod': net_liq_sod,
    'min_equity': min_equity,
    'trailing_max_drawdown': trailing_max_drawdown,
    'trailing_max_drawdown_limit': trailing_max_drawdown_limit,
    'trailing_mode': trailing_mode,
    'max_net_liq': max_net_liq,
    'drawdown_remaining': drawdown_remaining,
}
print(f"[MIN EQUITY] account={account_id}: netLiq=${net_liq:,.2f}, "
      f"SOD=${net_liq_sod:,.2f}, minEquity=${min_equity:,.2f}, "
      f"remaining=${drawdown_remaining:,.2f}, mode={trailing_mode}")
return result
except Exception as e:
    import traceback
    print(f"[MIN EQUITY] Failed to get min equity: {e}")
    traceback.print_exc()
    return None

def place_order_api(self, symbol, side, qty, order_type="Market", price=None,
                    stop_price=None, tp=None, sl=None, account_id=None):
    """Place an order via the Tradovate REST API (no Selenium UI clicking).

    Args:
        symbol:      Contract symbol e.g. "NQM6", "ESM6"
        side:        "Buy" or "Sell"
        qty:         Number of contracts (int)
        order_type:  "Market", "Limit", "Stop", "StopLimit", "MIT"
        price:       Limit price (required for Limit/StopLimit)
        stop_price:  Stop price (required for Stop/StopLimit)
        tp:          Take profit – in ticks (int) or absolute price (float > tick_size * 200)

```



```

        sl:          Stop loss – in ticks (int) or absolute price (float > tick_size * 200)
        account_id: Specific account ID (default: first account)

Returns:
    dict with order result on success, None on failure

Tick reference:
    NQ/MNQ = 0.25/tick ($5.00/tick NQ, $0.50/tick MNQ)
    ES/MES = 0.25/tick ($12.50/tick ES, $1.25/tick MES)
    YM/MYM = 1.00/tick ($5.00/tick YM, $0.50/tick MYM)
    CL/MCL = 0.01/tick ($10.00/tick CL, $1.00/tick MCL)
"""
# Normalize side
side = side.capitalize()
if side not in ("Buy", "Sell"):
    print(f"[API ORDER] Invalid side: {side}")
    return None

# Get account
if not account_id:
    account_id, account_name = self._get_account_for_api()
    if not account_id:
        print("[API ORDER] No account found")
        return None
else:
    # Look up account name – accountSpec needs the name string, not numeric ID
    account_name = str(account_id)
    accounts = self._api_fetch("/account/list")
    if accounts and isinstance(accounts, list):
        for acct in accounts:
            if acct.get('id') == account_id:
                account_name = acct.get('name', str(account_id))
                break

# Find contract by symbol name
contract = self._api_fetch(f"/contract/find?name={symbol}")
if not contract or not contract.get('id'):
    print(f"[API ORDER] Contract not found: {symbol}")
    return None
contract_id = contract['id']
contract_name = contract.get('name', symbol)

# Get tick size from contract maturity (for tick-based TP/SL)
tick_size = None
if tp is not None or sl is not None:
    mat_id = contract.get('contractMaturityId')
    if mat_id:
        mat = self._api_fetch(f"/contractMaturity/item?id={mat_id}")
        if mat:
            prod_id = mat.get('productId')
            if prod_id:
                product = self._api_fetch(f"/product/item?id={prod_id}")
                if product:
                    tick_size = product.get('tickSize', 0.25)
    if not tick_size:
        # Fallback: common tick sizes by symbol prefix
        sym_upper = symbol.upper()
        if any(sym_upper.startswith(p) for p in ("NQ", "MNQ", "ES", "MES")):
            tick_size = 0.25
        elif any(sym_upper.startswith(p) for p in ("YM", "MYM")):

```

```

        tick_size = 1.0
    elif any(sym_upper.startswith(p) for p in ("CL", "MCL")):
        tick_size = 0.01
    else:
        tick_size = 0.25
    print(f"[API ORDER] Using fallback tick size: {tick_size}")

# ■■ ref_price is determined AFTER the market fill (see below) ■■
# Pre-order quotes from /md/getQuote are unreliable on prop firm accounts
# and have caused catastrophic bracket pricing (e.g. 7089 instead of 26474).
# We now ALWAYS place the market order first, then use the fill price.

print(f"[API ORDER] {side} {qty}x {contract_name} (id={contract_id}) on {account_name} – {order_type}
      + (f" tick={tick_size}" if tick_size else "")")

# Build order payload
payload = {
    "accountSpec": account_name,
    "accountId": account_id,
    "action": side,
    "symbol": contract_name,
    "orderQty": qty,
    "orderType": order_type,
    "isAutomated": False,
}
if price is not None and order_type in ("Limit", "StopLimit"):
    payload["price"] = price
if stop_price is not None and order_type in ("Stop", "StopLimit", "MIT"):
    payload["stopPrice"] = stop_price

# Place market order FIRST, then add brackets using fill price
self._placing_order = True
try:
    print(f"[API ORDER] Placing market order via /order/placeOrder")
    result = self._api_fetch("/order/placeOrder", "POST", payload)

    if not result:
        print(f"[API ORDER] ■ Order rejected or failed")
        return None

    # Parse result – may be nested or flat
    if isinstance(result, dict):
        order_id = result.get('orderId') or result.get('id') or '?'
        status = result.get('ordStatus', '?')
        fill_price = result.get('avgPx') or result.get('price')
        print(f"[API ORDER] ■ Order placed: id={order_id} status={status} fillPrice={fill_price}")
        print(f"[API ORDER] Full result: {result}")
    else:
        print(f"[API ORDER] ■ Order placed (non-dict response): {result}")
        order_id = '?'
        fill_price = None

    # Now add TP/SL brackets using the actual fill price
    if tp is not None or sl is not None:
        import time as _time

        # Retry loop – market orders may take a moment to fill
        if not fill_price and order_id != '?':
            for attempt in range(8):
                _time.sleep(0.5)

```

```

        try:
            order_detail = self._api_fetch(f"/order/item?id={order_id}")
            if order_detail and isinstance(order_detail, dict):
                fill_price = order_detail.get('avgPx') or order_detail.get('price')
                detail_status = order_detail.get('ordStatus', '?')
                print(
                    f"[API ORDER] Fill check #{attempt+1}: status={detail_status} avgPx={fill_price}"
                )
                if fill_price:
                    break
        except Exception as ex:
            print(f"[API ORDER] Fill check #{attempt+1} error: {ex}")

    # Fallback: check position for price
    if not fill_price:
        try:
            positions = self._api_fetch("/position/list")
            if positions and isinstance(positions, list):
                for pos in positions:
                    if pos.get('contractId') == contract_id and pos.get(
                        'accountId') == account_id:
                        fill_price = pos.get('price') or pos.get('netPrice')
                        print(f"[API ORDER] Got price from position: {fill_price}")
                        break
        except Exception as ex:
            print(f"[API ORDER] Position lookup error: {ex}")

    if fill_price:
        # Sanity check: NQ-like symbols should have prices > 1000
        sym_upper = symbol.upper()
        if any(sym_upper.startswith(p) for p in ("NQ", "MNQ")) and fill_price < 1000:
            print(
                f"[API ORDER] ■ SANITY FAIL: fill_price={fill_price} is too low for {symbol}"
            )
        else:
            print(f"[API ORDER] Placing brackets at fill_price={fill_price}")
            self._place_brackets_post_fill(
                account_name, account_id, contract_name, qty,
                side, fill_price, tp, sl, tick_size
            )
        else:
            print(f"[API ORDER] ■ Could not determine fill price – brackets NOT placed!")

    return result

except Exception as e:
    import traceback
    print(f"[API ORDER] ■ Exception: {e}")
    traceback.print_exc()
    return None
finally:
    self._placing_order = False

def _place_brackets_post_fill(self, account_name, account_id, contract_name,
                             qty, side, fill_price, tp, sl, tick_size):
    """Place TP and SL bracket orders after a market order has filled.

    For BUY main order: TP = Sell Limit (above fill), SL = Sell Stop (below fill)
    For SELL main order: TP = Buy Limit (below fill), SL = Buy Stop (above fill)
    """
    tp_price = self._resolve_bracket_price(tp, tick_size, fill_price, side, is_tp=True) if tp else None
    sl_price = self._resolve_bracket_price(sl, tick_size, fill_price, side, is_tp=False) if sl else None

```

```

if not tp_price and not sl_price:
    print(f"[BRACKET] No valid bracket prices computed – skipping")
    return

# Determine bracket order sides and types
if side == "Buy":
    # Close a long: Sell Limit for TP, Sell Stop for SL
    tp_action, tp_type = "Sell", "Limit"
    sl_action, sl_type = "Sell", "Stop"
else:
    # Close a short: Buy Limit for TP, Buy Stop for SL
    tp_action, tp_type = "Buy", "Limit"
    sl_action, sl_type = "Buy", "Stop"

print(f"[BRACKET] Main={side} fill={fill_price} | "
      f"TP={tp_action} {tp_type} @ {tp_price} | SL={sl_action} {sl_type} @ {sl_price}")

# Place TP order
if tp_price:
    tp_payload = {
        "accountSpec": account_name,
        "accountId": account_id,
        "action": tp_action,
        "symbol": contract_name,
        "orderQty": qty,
        "orderType": tp_type,
        "price": tp_price,
        "isAutomated": False,
    }
    print(f"[BRACKET] Placing TP: {tp_action} {qty}x {contract_name} {tp_type} @ {tp_price}")
    try:
        tp_result = self._api_fetch("/order/placeOrder", "POST", tp_payload)
        if tp_result:
            print(f"[BRACKET] ■ TP placed: {tp_result}")
        else:
            print(f"[BRACKET] ■ TP failed – no response")
    except Exception as e:
        print(f"[BRACKET] ■ TP exception: {e}")

# Place SL order
if sl_price:
    sl_payload = {
        "accountSpec": account_name,
        "accountId": account_id,
        "action": sl_action,
        "symbol": contract_name,
        "orderQty": qty,
        "orderType": sl_type,
        "stopPrice": sl_price,
        "isAutomated": False,
    }
    print(f"[BRACKET] Placing SL: {sl_action} {qty}x {contract_name} {sl_type} @ {sl_price}")
    try:
        sl_result = self._api_fetch("/order/placeOrder", "POST", sl_payload)
        if sl_result:
            print(f"[BRACKET] ■ SL placed: {sl_result}")
        else:
            print(f"[BRACKET] ■ SL failed – no response")
    except Exception as e:
        print(f"[BRACKET] ■ SL exception: {e}")

```

```

def _resolve_bracket_price(self, value, tick_size, ref_price, side, is_tp):
    """Convert a TP/SL tick count to an absolute price.

    Values are Tradovate tick counts (e.g. 151 ticks for NQ).
    Price offset = ticks × tick_size (151 × 0.25 = 37.75 pts).

    For Buy orders: TP is above ref_price, SL is below
    For Sell orders: TP is below ref_price, SL is above
    """
    if value is None or tick_size is None:
        return None

    if ref_price is None:
        print(f"[API ORDER] Cannot calculate bracket: no reference price available")
        return None

    # Convert tick count → price offset
    offset = value * tick_size
    if side == "Buy":
        result = round(ref_price + offset, 10) if is_tp else round(ref_price - offset, 10)
    else:
        result = round(ref_price - offset, 10) if is_tp else round(ref_price + offset, 10)

    print(f"[API ORDER] Bracket calc: {value} ticks × {tick_size} = {offset} pts, "
          f"ref={ref_price} → {'TP' if is_tp else 'SL'}={result}")
    return result

def cancel_order_api(self, order_id):
    """Cancel an order via the REST API."""
    result = self._api_fetch("/order/cancelOrder", "POST", {"orderId": order_id})
    if result:
        print(f"[API] Order {order_id} cancelled")
    return result

def liquidate_position_api(self, account_id=None):
    """Liquidate (flatten) all positions on the account via the REST API.

    Strategy:
    1. Check for open positions first
    2. Try /order/liquidatePosition endpoint
    3. If that fails, close each position individually with opposite market orders

    Returns: dict with results, or None if no positions / all failed
    """
    if not account_id:
        account_id, account_name = self._get_account_for_api()
        if not account_id:
            print("[API] No account found for liquidation")
            return None
    else:
        account_name = str(account_id)

    # Step 1: Check if there are actually open positions
    positions = self.get_positions_api(account_id)
    open_positions = [p for p in positions if p.get('netPos', 0) != 0]
    if not open_positions:
        print(f"[API] No open positions on {account_name} – already flat")
        return None

    print(f"[API] Found {len(open_positions)} open position(s) on {account_name} – liquidating...")

```

```

# Step 2: Try the liquidatePosition endpoint
result = self._api_fetch("/order/liquidatePosition", "POST", {"accountId": account_id})
if result:
    print(f"[API] ■ Positions liquidated via /order/liquidatePosition")
    return result

# Step 3: Fallback – close each position with opposite market orders
print(f"[API] liquidatePosition failed – falling back to individual close orders")
closed = 0
for pos in open_positions:
    net_pos = pos.get('netPos', 0)
    contract_id = pos.get('contractId')
    if net_pos == 0 or not contract_id:
        continue

    # Get contract name for the order
    contract = self._api_fetch(f"/contract/item?id={contract_id}")
    contract_name = contract.get('name', '') if contract else ''
    if not contract_name:
        print(f"[API] Could not resolve contract {contract_id} – skipping")
        continue

    close_side = "Sell" if net_pos > 0 else "Buy"
    close_qty = abs(net_pos)
    payload = {
        "accountSpec": account_name,
        "accountId": account_id,
        "action": close_side,
        "symbol": contract_name,
        "orderQty": close_qty,
        "orderType": "Market",
        "isAutomated": False,
    }
    print(f"[API] Closing: {close_side} {close_qty}x {contract_name}")
    close_result = self._api_fetch("/order/placeOrder", "POST", payload)
    if close_result:
        print(f"[API] ■ Closed {contract_name}")
        closed += 1
    else:
        print(f"[API] ■ Failed to close {contract_name}")

return {"closed": closed, "total": len(open_positions)} if closed > 0 else None

def get_positions_api(self, account_id=None):
    """Get current open positions via the REST API."""
    positions = self._api_fetch("/position/list") or []
    if account_id:
        positions = [p for p in positions if p.get('accountId') == account_id]
    return positions

def cancel_all_orders_api(self, account_id=None):
    """Cancel all working (pending) orders on the account via the REST API.

    Only cancels if the account has no open positions (i.e. is flat).
    This cleans up orphaned bracket orders (stop/limit) after a position
    is closed by TP, SL, or manual liquidation.
    """
    # Only cancel if no active positions
    positions = self.get_positions_api(account_id)
    open_positions = [p for p in positions if p.get('netPos', 0) != 0]

```

```

if open_positions:
    return 0

orders = self.get_orders_api()
if not orders:
    return 0

cancelled = 0
for order in orders:
    status = order.get('ordStatus', '')
    if status not in ('Working', 'Accepted', 'PendingNew'):
        continue
    if account_id and order.get('accountId') != account_id:
        continue
    oid = order.get('id')
    if oid:
        try:
            self.cancel_order_api(oid)
            cancelled += 1
        except Exception as e:
            print(f"[API] Failed to cancel order {oid}: {e}")
if cancelled > 0:
    print(f"[API] ■ Cancelled {cancelled} pending order(s) (account is flat)")
return cancelled

def get_orders_api(self):
    """Get current working orders via the REST API."""
    return self._api_fetch("/order/list") or []

def get_trade_history(self):
    """Get full trade history for ALL accounts under this login.

    Returns list of dicts, one per account:
        account_name, account_id, environment, balance, balance_sod,
        realized_pnl, open_pnl, trailing_max_drawdown, trailing_max_drawdown_limit,
        drawdown_mode, daily_pnl, fills
    """
    try:
        # Get environment info
        env_info = self.driver.execute_script("""
            try {
                var auth = JSON.parse(sessionStorage.getItem('api_authenticator_state')) || '{}';
                return {env: auth.environment || 'demo', username: auth.username || ''};
            } catch(e) { return {env: 'demo'}; }
        """)
        environment = env_info.get('env', 'demo') if env_info else 'demo'

        # Get ALL accounts under this login
        accounts = self._api_fetch("/account/list")
        if not accounts or not isinstance(accounts, list) or len(accounts) == 0:
            return None

        # Shared data – fetch once for all accounts
        all_fills = self._api_fetch("/fill/list") or []
        all_autoliq = self._api_fetch("/userAccountAutoLiq/list") or []

        # Resolve contract names for fills
        contract_cache = {}
        for f in all_fills:
            cid = f.get('contractId')

```

```

        if cid and cid not in contract_cache:
            c = self._api_fetch(f"/contract/item?id={cid}")
            contract_cache[cid] = c.get('name', str(cid)) if c else str(cid)

# Index auto-liq by account ID
autoliq_by_acct = {}
for al in all_autoliq:
    autoliq_by_acct[al.get('accountId', al.get('account'))] = al

from collections import defaultdict
results = []

for acct in accounts:
    aid = acct['id']
    aname = acct.get('name', '?')

    # Per-account data
    balance_logs = self._api_fetch(f"/cashBalanceLog/ldeps?masterids={aid}") or []
    snapshot = self._api_fetch("/cashBalance/getCashBalanceSnapshot", "POST", {
        "accountId": aid}) or {}

    # Filter fills for this account
    acct_fills = [f for f in all_fills if f.get('accountId') == aid]

    # Build daily P&L from cashBalanceLog
    daily_raw = defaultdict(lambda: {"trades": 0, "gross_pnl": 0.0, "fees": 0.0,
        "balance_eod": 0.0})
    for entry in sorted(balance_logs, key=lambda x: x.get('id', 0)):
        td = entry.get('tradeDate', {})
        date_str = f"{td.get('year', 0)}-{td.get('month', 1):02d}-{td.get('day', 1):02d}"
        ctype = entry.get('cashChangeType', '')
        delta = entry.get('delta', 0)
        daily_raw[date_str]["balance_eod"] = entry.get('amount', 0)
        if ctype == "TradePaired":
            daily_raw[date_str]["gross_pnl"] += delta
            daily_raw[date_str]["trades"] += 1
        elif ctype in ("Commission", "ExchangeFee", "ClearingFee", "NfaFee"):
            daily_raw[date_str]["fees"] += delta

    daily_pnl = []
    for date_str in sorted(daily_raw.keys()):
        d = daily_raw[date_str]
        net = d["gross_pnl"] + d["fees"]
        daily_pnl.append({
            "date": date_str,
            "trades": d["trades"],
            "gross_pnl": round(d["gross_pnl"], 2),
            "fees": round(d["fees"], 2),
            "net_pnl": round(net, 2),
            "balance_eod": round(d["balance_eod"], 2),
        })

    # Build fills list for this account
    fill_list = []
    for f in sorted(acct_fills, key=lambda x: x.get('timestamp', '')):
        td = f.get('tradeDate', {})
        date_str = f"{td.get('year', 0)}-{td.get('month', 1):02d}-{td.get('day', 1):02d}"
        ts = f.get('timestamp', '')
        time_str = ts[11:19] if len(ts) > 19 else ts
        fill_list.append({

```



```

        "date": date_str,
        "time": time_str,
        "action": f.get('action', '?'),
        "qty": f.get('qty', 0),
        "contract": contract_cache.get(f.get('contractId'), '?'),
        "price": f.get('price', 0),
        "order_id": f.get('orderId', 0),
    })

    al = autoliq_by_acct.get(aid, {})

    results.append({
        "account_name": aname,
        "account_id": aid,
        "environment": environment,
        "balance": snapshot.get('netLiq', 0),
        "balance_sod": snapshot.get('netLiqSOD', 0),
        "realized_pnl": snapshot.get('realizedPnL', 0),
        "open_pnl": snapshot.get('openPnL', 0),
        "trailing_max_drawdown": al.get('trailingMaxDrawdown', 0),
        "trailing_max_drawdown_limit": al.get('trailingMaxDrawdownLimit', 0),
        "drawdown_mode": al.get('trailingMaxDrawdownMode', '?'),
        "daily_pnl": daily_pnl,
        "fills": fill_list,
    })

    return results
except Exception as e:
    print(f"[HISTORY] get_trade_history failed: {e}")
    import traceback
    traceback.print_exc()
    return None

def get_mnq_daily_pnl(self):
    """Get daily P&L for days with MNQ trading activity (farming days).

    Identifies farming days by checking for MNQ contract fills, then
    pulls the daily P&L from cashBalanceLog for those dates.

    Returns list of dicts, one per account that has MNQ activity:
    [{"account_name": str, "account_id": int,
      "mnq_daily_pnl": [{"date": "YYYY-MM-DD", "net_pnl": float}, ...]}]
    """
    try:
        accounts = self._api_fetch("/account/list")
        if not accounts or not isinstance(accounts, list):
            return []

        all_fills = self._api_fetch("/fill/list") or []

        # Resolve unique contract IDs to names
        contract_cache = {}
        for f in all_fills:
            cid = f.get('contractId')
            if cid and cid not in contract_cache:
                c = self._api_fetch(f"/contract/item?id={cid}")
                contract_cache[cid] = c.get('name', str(cid)) if c else str(cid)

        # Identify MNQ contract IDs
        mnq_contract_ids = {cid for cid, name in contract_cache.items()}

```

```

        if str(name).upper().startswith('MNQ')}}

if not mnq_contract_ids:
    return []

from collections import defaultdict
results = []

for acct in accounts:
    aid = acct['id']
    aname = acct.get('name', '?')

    # Find dates where this account had MNQ fills
    mnq_dates = set()
    for f in all_fills:
        if f.get('accountId') == aid and f.get('contractId') in mnq_contract_ids:
            td = f.get('tradeDate', {})
            date_str = f"{td.get('year', 0)}-{td.get('month', 1):02d}-{td.get('day', 1):02d}"
            mnq_dates.add(date_str)

    if not mnq_dates:
        continue

    # Get daily P&L from cashBalanceLog for MNQ dates only
    balance_logs = self._api_fetch(f"/cashBalanceLog/ldeps?masterids={aid}") or []

    daily_raw = defaultdict(lambda: {"gross_pnl": 0.0, "fees": 0.0})
    for entry in balance_logs:
        td = entry.get('tradeDate', {})
        date_str = f"{td.get('year', 0)}-{td.get('month', 1):02d}-{td.get('day', 1):02d}"
        if date_str not in mnq_dates:
            continue
        ctype = entry.get('cashChangeType', '')
        delta = entry.get('delta', 0)
        if ctype == "TradePaired":
            daily_raw[date_str]["gross_pnl"] += delta
        elif ctype in ("Commission", "ExchangeFee", "ClearingFee", "NfaFee"):
            daily_raw[date_str]["fees"] += delta

    mnq_daily = []
    for date_str in sorted(daily_raw.keys()):
        d = daily_raw[date_str]
        net = round(d["gross_pnl"] + d["fees"], 2)
        mnq_daily.append({"date": date_str, "net_pnl": net})

    if mnq_daily:
        results.append({
            "account_name": aname,
            "account_id": aid,
            "mnq_daily_pnl": mnq_daily,
        })

    return results
except Exception as e:
    print(f"[FARMING] get_mnq_daily_pnl failed: {e}")
    import traceback
    traceback.print_exc()
    return []

```

Method: TradovateAccount.__init__

```
def __init__(self, username, password, pair_id=None, trading_mode='Simulation')
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.
- **__init__** — The constructor. Runs after Python has allocated the instance; assigns starting attribute values to self.

Step by step (top-level body)

1. Assigns `self.username = username`.
2. Assigns `self.password = password`.
3. Assigns `self.pair_id = pair_id` or 'default'.
4. Assigns `self.trading_mode = trading_mode`.
5. Assigns `self.logged_in = False`.
6. Assigns `self.driver = None`.
7. Assigns `self.first_trade_attempted = False`.
8. Assigns `self._first_stats_fetch = True`.
9. Assigns `self._placing_order = False`.
10. Assigns `self._login_timestamp = None`.
11. Assigns `self.lock = threading.RLock()`.
12. Assigns `instance_key = f'{username}_{self.pair_id}'`.
13. **if** `instance_key` in `self._chrome_instances`: branches conditionally.
14. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def __init__(self, username, password, pair_id=None, trading_mode="Simulation"):
    self.username = username
    self.password = password
    self.pair_id = pair_id or "default"
    self.trading_mode = trading_mode # "Simulation" or "Live Trading"
    self.logged_in = False
    self.driver = None
    self.first_trade_attempted = False
    self._first_stats_fetch = True
    self._placing_order = False # Flag to block stats fetching during order placement
    self._login_timestamp = None # Track when we logged in for debugging
    self.lock = threading.RLock() # Thread safety for Selenium operations

    # Create unique instance key using both username and pair_id
    instance_key = f"{username}_{self.pair_id}"
```

```

# Check if Chrome instance already exists for this specific account+pair combination
if instance_key in self._chrome_instances:
    logging.info(f"Reusing existing Chrome instance for account: {username} (Pair: {self.pair_id})")
    existing_driver = self._chrome_instances[instance_key]
    # Test if existing driver is still alive
    try:
        existing_driver.current_url
        self.driver = existing_driver
        logging.info("Successfully connected to existing Chrome instance")
        return
    except Exception:
        logging.info("Existing Chrome instance is dead, creating new one")
        # Remove dead instance from registry
        del self._chrome_instances[instance_key]

# Initialize driver with error handling
try:
    self.driver = self._initialize_driver()
except Exception as e:
    raise Exception(
        f"Failed to initialize WebDriver: {str(e)}. This may indicate VPS Chrome setup issues."
    )

```

Method: TradovateAccount._get_chromedriver_path

```
def _get_chromedriver_path(self)
```

Get the path to ChromeDriver executable - FAST VERSION (no compatibility check)

Step by step (top-level body)

1. Calls logging.info(...) for its side effect.
2. return None.

Code (copy-paste safe)

```

def _get_chromedriver_path(self):
    """Get the path to ChromeDriver executable - FAST VERSION (no compatibility check)"""
    # SELENIUM 4.15+ AUTO-MANAGEMENT: Let Selenium Manager handle ChromeDriver automatically
    # This ensures ChromeDriver always matches the installed Chrome version
    logging.info("[AUTO] Using Selenium Manager for automatic ChromeDriver version matching")
    return None

```

Method: TradovateAccount._ensure_chrome_compatibility

```
def _ensure_chrome_compatibility(self)
```

Ensure Chrome-ChromeDriver version compatibility

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. try block with 1 except clause.

Code (copy-paste safe)

```
def _ensure_chrome_compatibility(self):
    """Ensure Chrome-ChromeDriver version compatibility"""
    try:
        # Import the compatibility manager
        sys.path.insert(0, os.path.dirname(os.path.dirname(__file__)))
        from chrome_auto_compatibility import ChromeVersionManager # type: ignore

        manager = ChromeVersionManager()
        success = manager.ensure_compatibility()

        if success:
            logging.info("[CHECK] Chrome-ChromeDriver compatibility verified")
        else:
            logging.warning("[WARNING] Chrome-ChromeDriver compatibility could not be ensured")

    except Exception as e:
        logging.warning(f"Chrome compatibility check failed: {e}")
```

Method: TradovateAccount._initialize_driver

```
def _initialize_driver(self)
    Initialize Chrome WebDriver with crash-resistant options
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.

Step by step (top-level body)

1. Assigns `chrome_options = Options()`.
2. Calls `chrome_options.add_argument(...)` for its side effect.
3. Calls `chrome_options.add_argument(...)` for its side effect.
4. Calls `chrome_options.add_argument(...)` for its side effect.
5. Calls `chrome_options.add_argument(...)` for its side effect.
6. Calls `chrome_options.add_argument(...)` for its side effect.
7. Calls `chrome_options.add_argument(...)` for its side effect.
8. Calls `chrome_options.add_argument(...)` for its side effect.
9. Calls `chrome_options.add_argument(...)` for its side effect.
10. Calls `chrome_options.add_argument(...)` for its side effect.

11. Calls `chrome_options.add_argument(...)` for its side effect.
12. Calls `chrome_options.add_argument(...)` for its side effect.
13. Calls `chrome_options.add_argument(...)` for its side effect.
14. Calls `chrome_options.add_argument(...)` for its side effect.
15. ... and 14 more statement(s) in the body.

Code (copy-paste safe)

```
def _initialize_driver(self):
    """Initialize Chrome WebDriver with crash-resistant options"""
    chrome_options = Options()

    # PERFORMANCE: Removed user-data-dir to speed up Chrome launch
    # Incognito mode is faster than loading persistent profiles
    # Each Chrome instance will be fresh and fast

    # PERFORMANCE: Removed window positioning - let Chrome decide (faster)
    # PERFORMANCE: Removed remote debugging port - not needed for basic automation

    # MAXIMUM SPEED: Only critical options for fastest Chrome launch
    chrome_options.add_argument("--no-sandbox") # Required for some systems
    chrome_options.add_argument("--disable-dev-shm-usage") # Required for VPS/Docker

    # CRASH FIX: Improve stability and prevent crashes
    chrome_options.add_argument("--disable-crash-reporter") # Disable crash reporting
    chrome_options.add_argument("--disable-in-process-stack-traces") # Reduce memory overhead
    chrome_options.add_argument("--disable-logging") # Reduce disk I/O
    chrome_options.add_argument("--log-level=3") # Suppress console messages (3=FATAL only)
    chrome_options.add_argument("--silent") # Suppress console output

    # MEMORY MANAGEMENT: Prevent memory leaks and crashes
    chrome_options.add_argument("--disable-features=TranslateUI") # Disable translate
    chrome_options.add_argument("--disable-features=MediaRouter") # Disable media router
    chrome_options.add_argument("--disable-component-update") # Disable component updates
    chrome_options.add_argument("--disable-background-timer-throttling") # Better tab management
    chrome_options.add_argument("--disable-backgrounding-occluded-windows") # Keep windows active
    chrome_options.add_argument("--disable-renderer-backgrounding") # Prevent renderer from sleeping

    # RENDERING FIX: Enable GPU acceleration for proper page rendering
    # Removed --disable-gpu to fix white screen issues
    chrome_options.add_argument("--enable-features=NetworkService,NetworkServiceInProcess")

    # PERFORMANCE: Faster page loading optimizations
    chrome_options.add_argument("--disable-extensions") # No extensions = faster
    chrome_options.add_argument("--disable-plugins") # No plugins = faster
    # Images enabled - needed for CAPTCHA solving and normal page rendering
    chrome_options.add_argument("--disable-background-networking") # No background requests
    chrome_options.add_argument("--disable-default-apps") # No default apps
    chrome_options.add_argument("--disable-sync") # No sync

    # SPEED OPTIMIZATION: Smaller window for faster rendering
    chrome_options.add_argument("--window-size=1200,800") # Reduced from 1400,900

    # FIX: Disable hardware acceleration fallback that can cause white screens
    chrome_options.add_argument("--disable-software-rasterizer")

    # Anti-detection settings (keep minimal)
    chrome_options.add_experimental_option("excludeSwitches", ["enable-automation", "enable-logging"])
```

```

chrome_options.add_experimental_option('useAutomationExtension', False)
chrome_options.add_argument("--disable-blink-features=AutomationControlled")

# Minimal preferences to prevent crashes
prefs = {
    "profile.default_content_setting_values": {
        "popups": 2, # Block popups only
        "notifications": 2, # Block notifications only
    }
}
chrome_options.add_experimental_option("prefs", prefs)

try:
    # Ensure Chrome-ChromeDriver compatibility before initialization
    logging.info("[COMPAT] Checking Chrome-ChromeDriver compatibility...")
    self._ensure_chrome_compatibility()

    # Get ChromeDriver path (returns None for auto-management in Selenium 4.15+)
    chromedriver_path = self._get_chromedriver_path()

    # Create the WebDriver instance - let Selenium Manager handle ChromeDriver automatically
    logging.info("[CHROME] Initializing Chrome browser...")
    logging.info("[CHROME] Using Selenium Manager for automatic ChromeDriver version matching")
    logging.info(
        "[CHROME] Selenium Manager will download ChromeDriver if needed (happens once per Chrome"
    )
    logging.info("[CHROME] If already cached, Chrome will launch immediately...")

    import time
    start_time = time.time()
    driver = webdriver.Chrome(options=chrome_options)
    elapsed = time.time() - start_time

    if elapsed > 5:
        logging.info(
            f"[CHROME] ✓ Chrome launched in {elapsed:.1f}s (ChromeDriver was downloaded/updated)"
        )
    else:
        logging.info(f"[CHROME] ✓ Chrome launched in {elapsed:.1f}s (using cached ChromeDriver)")

    # MAXIMUM SPEED: Ultra-aggressive timeouts
    driver.set_page_load_timeout(30) # Increased to 30 seconds to fix white screen issues
    driver.implicitly_wait(2) # Increased to 2 seconds for better element detection

    # CRASH RECOVERY: Add page crash detection callback
    try:
        # Enable Chrome DevTools Protocol for crash detection
        driver.execute_cdp_cmd('Page.enable', {})
        logging.info("[STABILITY] Chrome crash detection enabled")
    except Exception as e:
        logging.warning(f"Could not enable crash detection: {e}")

    logging.info(f"Chrome WebDriver initialized successfully for {self.username}")
    return driver

except Exception as e:
    logging.error(f"Failed to initialize Chrome WebDriver: {e}")
    raise Exception(f"ChromeDriver initialization failed: {e}")

```

Method: TradovateAccount._validate_blueprint_parameters

```

def _validate_blueprint_parameters(self, symbol, qty, tp=None, sl=None, prop_firm=None, phase=None,

```

```
account_size=None, strict_mode=True)
```

Validates trading parameters against blueprint configuration to prevent incorrect trades. Args: symbol (str): Trading symbol being used qty (int): Quantity being traded tp (int, optional): Take profit ticks sl (int, optional): Stop loss ticks prop_firm (str, optional): Prop firm name for blueprint lookup phase (str, optional): Trading phase (challenge, funded, farming) account_size (str, optional): Account size for blueprint lookup strict_mode (bool): If True, blocks trades on validation failure. If False, only logs warnings. Returns: tuple: (is_valid, validation_message, blueprint_config)

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.

Step by step (top-level body)

1. **try** block with 2 **except** clauses.

Code (copy-paste safe)

```
def _validate_blueprint_parameters(self, symbol, qty, tp=None, sl=None, prop_firm=None, phase=None,
    account_size=None, strict_mode=True):
    """
    Validates trading parameters against blueprint configuration to prevent incorrect trades.

    Args:
        symbol (str): Trading symbol being used
        qty (int): Quantity being traded
        tp (int, optional): Take profit ticks
        sl (int, optional): Stop loss ticks
        prop_firm (str, optional): Prop firm name for blueprint lookup
        phase (str, optional): Trading phase (challenge, funded, farming)
        account_size (str, optional): Account size for blueprint lookup
        strict_mode (bool): If True, blocks trades on validation failure. If False, only logs warning

    Returns:
        tuple: (is_valid, validation_message, blueprint_config)
    """
    try:
        # Import blueprint manager here to avoid circular imports
        from prop_firm_manager import PropFirmManager

        # If no blueprint context provided, try to get from environment
        if not prop_firm:
            prop_firm = os.getenv("SELECTED_PROP_FIRM", "MFFU")
        if not phase:
            phase = os.getenv("TRADING_PHASE", "challenge_trade1")
        if not account_size:
            account_size = os.getenv("ACCOUNT_SIZE", "50k")

        # Get blueprint configuration
        prop_firm_manager = PropFirmManager()
```



```

prop_firm_manager.set_prop_firm(prop_firm)
blueprint_config = prop_firm_manager.get_prop_firm_strategy_config(phase, account_size)

if not blueprint_config:
    error_msg = f"[X] BLUEPRINT VALIDATION FAILED: No blueprint config found for {prop_firm}"
    logging.error(error_msg)
    if strict_mode:
        raise Exception(error_msg)
    return False, error_msg, None

# Validate each parameter against blueprint
validation_errors = []
validation_warnings = []

# Symbol validation
expected_symbol = blueprint_config.get('tradovate_symbol')
if expected_symbol and symbol != expected_symbol:
    error_msg = f"[X] SYMBOL MISMATCH: Expected {expected_symbol}, got {symbol}"
    validation_errors.append(error_msg)

# Quantity validation
expected_qty = blueprint_config.get('tradovate_qty')
if expected_qty and qty != expected_qty:
    error_msg = f"[X] QUANTITY MISMATCH: Expected {expected_qty}, got {qty}"
    validation_errors.append(error_msg)

# Take profit validation (if provided)
if tp is not None:
    expected_tp = blueprint_config.get('tradovate_tp_ticks')
    if expected_tp and tp != expected_tp:
        error_msg = f"[X] TP MISMATCH: Expected {expected_tp}, got {tp}"
        validation_errors.append(error_msg)

# Stop loss validation (if provided)
if sl is not None:
    expected_sl = blueprint_config.get('tradovate_sl_ticks')
    if expected_sl and sl != expected_sl:
        error_msg = f"[X] SL MISMATCH: Expected {expected_sl}, got {sl}"
        validation_errors.append(error_msg)

# Log validation results
if validation_errors:
    full_error_msg = f"[ALARM] BLUEPRINT VALIDATION FAILED for {prop_firm}/{phase}/{account_size}"
    logging.error(full_error_msg)
    if strict_mode:
        raise Exception(full_error_msg)
    return False, full_error_msg, blueprint_config

if validation_warnings:
    warning_msg = f"[WARNING] BLUEPRINT WARNINGS for {prop_firm}/{phase}/{account_size}: {'; '.join(warnings)}"
    logging.warning(warning_msg)

success_msg = f"[CHECK] BLUEPRINT VALIDATION PASSED for {prop_firm}/{phase}/{account_size}: {'; '.join(successes)}"
logging.info(success_msg)

return True, success_msg, blueprint_config

except ImportError:
    warning_msg = "[WARNING] BLUEPRINT VALIDATION SKIPPED: Could not import PropFirmManager"
    logging.warning(warning_msg)

```

```

        return True, warning_msg, None
    except Exception as e:
        error_msg = f"[X] BLUEPRINT VALIDATION ERROR: {str(e)}"
        logging.error(error_msg)
        if strict_mode:
            raise Exception(error_msg)
        return False, error_msg, None

```

Method: TradovateAccount._force_trade_override

```
def _force_trade_override(self, symbol, qty, side, tp=None, sl=None, override_reason='Emergency override')
```

Emergency method to force a trade execution bypassing blueprint validation. Use only in critical situations where manual override is absolutely necessary. Args: symbol (str): Trading symbol qty (int): Quantity side (str): "buy" or "sell" tp (int, optional): Take profit ticks sl (int, optional): Stop loss ticks override_reason (str): Reason for override (logged for audit) Returns: bool: Success status

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Calls `logging.warning(...)` for its side effect.
2. Calls `logging.warning(...)` for its side effect.
3. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def _force_trade_override(self, symbol, qty, side, tp=None, sl=None, override_reason="Emergency override"):
    """
    Emergency method to force a trade execution bypassing blueprint validation.
    Use only in critical situations where manual override is absolutely necessary.

    Args:
        symbol (str): Trading symbol
        qty (int): Quantity
        side (str): "buy" or "sell"
        tp (int, optional): Take profit ticks
        sl (int, optional): Stop loss ticks
        override_reason (str): Reason for override (logged for audit)

    Returns:
        bool: Success status
    """
    logging.warning(f"[ALARM] EMERGENCY TRADE OVERRIDE: {override_reason}")
    logging.warning(f"[ALARM] OVERRIDE DETAILS: {side.upper()} {symbol} x{qty}, TP={tp}, SL={sl}")

    try:
        # Call the original order placement without validation
        return self._place_order_side_unvalidated(symbol, qty, side, tp, sl)
    except Exception as e:
        logging.error(f"[ALARM] EMERGENCY OVERRIDE FAILED: {e}")

```

```
return False
```

Method: TradovateAccount._place_order_side_unvalidated

```
def _place_order_side_unvalidated(self, symbol, qty, side, tp=None, sl=None, on_click=None,
    skip_positions_refresh=False)
```

Original order placement method without blueprint validation. Used internally for emergency overrides.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `original_strict_mode = os.getenv('BLUEPRINT_STRICT_MODE')`.
2. Assigns `os.environ['BLUEPRINT_STRICT_MODE'] = 'false'`.
3. **try** block with 1 **except** clause, plus a **finally**.

Code (copy-paste safe)

```
def _place_order_side_unvalidated(self, symbol, qty, side, tp=None, sl=None, on_click=None,
    skip_positions_refresh=False):
    """
    Original order placement method without blueprint validation.
    Used internally for emergency overrides.
    """
    # This would be the original _place_order_side logic without validation
    # For now, we'll just call the existing method but skip the validation part
    # In a full implementation, this would duplicate the order placement logic

    # Temporarily disable validation by setting environment variable
    original_strict_mode = os.getenv("BLUEPRINT_STRICT_MODE")
    os.environ["BLUEPRINT_STRICT_MODE"] = "false"

    try:
        result = self._place_order_side(symbol, qty, side, tp, sl, on_click, skip_positions_refresh)
        return True
    except Exception as e:
        logging.error(f"[ALARM] UNVALIDATED TRADE FAILED: {e}")
        return False
    finally:
        # Restore original setting
        if original_strict_mode is not None:
            os.environ["BLUEPRINT_STRICT_MODE"] = original_strict_mode
        else:
            os.environ.pop("BLUEPRINT_STRICT_MODE", None)
```

Method: TradovateAccount.get_validation_status

```
def get_validation_status(self)
```

Get current blueprint validation configuration status. Returns: dict: Validation configuration and status

Step by step (top-level body)

1. **return** {'strict_mode': os.getenv('BLUEPRINT_STRICT_MODE', 'true').lower()

Code (copy-paste safe)

```
def get_validation_status(self):
    """
    Get current blueprint validation configuration status.

    Returns:
        dict: Validation configuration and status
    """
    return {
        "strict_mode": os.getenv("BLUEPRINT_STRICT_MODE", "true").lower() == "true",
        "selected_prop_firm": os.getenv("SELECTED_PROP_FIRM", "MFFU"),
        "trading_phase": os.getenv("TRADING_PHASE", "challenge_trade1"),
        "account_size": os.getenv("ACCOUNT_SIZE", "50k"),
        "validation_enabled": True
    }
```

Method: TradovateAccount.login

```
def login(self)
    Perform full login sequence with robust error handling and retries.
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.

Step by step (top-level body)

1. **with self.lock**: enters a context manager.

Code (copy-paste safe)

```
def login(self):
    """
    Perform full login sequence with robust error handling and retries.
    """
    # Acquire lock to prevent stats fetching during login
    with self.lock:
        if self.logged_in:
            return True

        # --- For testing: use hardcoded credentials if not provided ---
        if not self.username or not self.password:
            self.username = "TYLERTURNER63"
```

```

self.password = "J5140A9013A9553tv="

try:
    logging.info(f"Starting Tradovate login for user: {self.username}")
    # Always open Tradovate in the first tab
    self.driver.get("https://trader.tradovate.com/welcome")
    self.driver.switch_to.window(self.driver.window_handles[0])
    logging.info("Navigated to Tradovate welcome page")

    # Wait for login form or trading mode selection with optimized timeout
    try:
        WebDriverWait(self.driver, 10).until( # MAXIMUM SPEED: Reduced from 15 to 10
            EC.any_of(
                EC.visibility_of_element_located((By.ID, "name-input")),
                EC.visibility_of_element_located((By.XPATH,
                    "//h1[contains(text(), 'Select a Trading Mode')]"))
            )
        )
        logging.info("Login form or trading mode selection loaded successfully")
    except Exception as e:
        logging.error(f"Login form or trading mode selection not found: {e}")
        # Take screenshot for debugging
        try:
            screenshot_path = os.path.join(tempfile.gettempdir(),
                f"tradovate_login_error_{int(time.time())}.png")
            self.driver.save_screenshot(screenshot_path)
            logging.info(f"Error screenshot saved to: {screenshot_path}")
        except:
            pass
        raise Exception(
            "Login form or trading mode selection not accessible. Please check internet connection")

    # Check if already at trading mode selection
    if self.driver.find_elements(By.XPATH, "//h1[contains(text(), 'Select a Trading Mode')]"):
        logging.info("Already at trading mode selection page, skipping login form.")
    else:
        # Fill login form
        try:
            username_field = WebDriverWait(self.driver, 15).until( # Increased timeout
                EC.element_to_be_clickable((By.ID, "name-input"))
            )
            password_field = WebDriverWait(self.driver, 15).until(
                EC.element_to_be_clickable((By.ID, "password-input"))
            )
            logging.info("Login fields found, filling credentials...")

            # Clear and fill username
            username_field.click()
            time.sleep(0.2)
            username_field.clear()
            username_field.send_keys(self.username)
            logging.info("Username filled successfully")

            time.sleep(0.3)

            # Clear and fill password
            password_field.click()
            time.sleep(0.2)
            password_field.clear()
            password_field.send_keys(self.password)

```

```

        logging.info("Password filled successfully")

        time.sleep(0.5)

        # Click login button
        login_button = WebDriverWait(self.driver, 15).until(
            EC.element_to_be_clickable((By.XPATH, "//button[.//span[text()='Login']]"))
        )
        self.driver.execute_script("arguments[0].click();", login_button)
        logging.info("Login button clicked successfully")

        # SPEED OPTIMIZATION: Wait for trading mode selection with optimized timeout
        WebDriverWait(self.driver, 25).until( # Reduced from 45 to 25
            EC.visibility_of_element_located((By.XPATH,
                "//h1[contains(text(), 'Select a Trading Mode')]"))
        )
        logging.info("Successfully reached 'Select a Trading Mode' page")

    except Exception as e:
        logging.error(f"Failed to reach trading mode selection page after login: {e}")
        # Check for error messages
        try:
            error_elements = self.driver.find_elements(By.CSS_SELECTOR,
                ".error, .alert, .warning, [class*='error'], [class*='alert']")
            if error_elements:
                error_text = error_elements[0].text
                raise Exception(f"Login failed: {error_text}")
        except:
            pass

        # Take screenshot for debugging
        try:
            screenshot_path = os.path.join(tempfile.gettempdir(),
                f"tradovate_login_failed_{int(time.time())}.png")
            self.driver.save_screenshot(screenshot_path)
            logging.info(f"Login failure screenshot saved to: {screenshot_path}")
        except:
            pass

        raise Exception("Login may have failed - could not reach trading mode selection")

    # Dismiss any interstitial pages that may appear before mode selection
    self._dismiss_interstitial_pages(timeout=3)

    # SPEED OPTIMIZATION: Try trading access with prioritized methods based on mode
    connected = False

    if self.trading_mode == "Live Trading":
        logging.info("■ STARTING LIVE TRADING MODE")
        access_attempts = [
            self._launch_live_trading_tab
        ]
    else:
        logging.info("■ STARTING SIMULATION MODE")
        access_attempts = [
            self._launch_simulation_tab, # Primary method - most reliable
            self._launch_simulation_tab_fallback1, # Most successful fallback
            self._launch_simulation_tab_fallback2, # Secondary fallback
            self._launch_simulation_tab_fallback3, # Less common scenarios
            self._launch_simulation_tab_fallback_different_broker # Last resort

```

```

    ]

    for attempt_func in access_attempts:
        try:
            logging.info(f"Trying access method: {attempt_func.__name__}")
            attempt_func()

            # Wait for trading interface to load with optimized timeout
            try:
                WebDriverWait(self.driver, 15).until( # SPEED OPTIMIZATION: Reduced from 30
                    EC.visibility_of_element_located((By.XPATH,
                        "//li[contains(@class, 'lm_tab')]//span[text()='Order Ticket']"))
                )
                time.sleep(1) # SPEED OPTIMIZATION: Reduced from 2 to 1
                self._wait_for_no_overlay(timeout=5) # SPEED OPTIMIZATION: Reduced from 10 t

            # Verify connection with account stats
            stats = self.get_account_stats()
            if (stats.get("Account Number") and
                stats.get("Account Number") != "Not Connected" and
                stats.get("Balance") not in ("N/A", "Error", None, "")):

                self.logged_in = True
                logging.info(
                    f"[CHECK] Trading tab launched and account connected - login complete
                connected = True
                break
            else:
                logging.info(
                    f"Trading interface loaded but account not connected (Account: {stats
            except Exception as e:
                logging.error(f"Trading interface failed to load: {e}")
                self.logged_in = False

        except Exception as e:
            logging.warning(f"Access method failed: {e}")
            self.logged_in = False

    if not connected:
        logging.error(f"[X] Could not connect to {self.trading_mode} after all methods.")
        self.logged_in = False
        return False

    return True

except Exception as e:
    logging.error(f"Login process failed: {e}")
    self.logged_in = False
    return False

```

Method: TradovateAccount._launch_simulation_tab_fallback1

```
def _launch_simulation_tab_fallback1(self)
```

Fallback 1: Try clicking the first visible 'Start Simulated Trading', 'Launch' or 'Access Simulation' button.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't

have to handle it.

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def _launch_simulation_tab_fallback1(self):
    """Fallback 1: Try clicking the first visible 'Start Simulated Trading', 'Launch' or 'Access Simulation' button"""
    try:
        # First try Start Simulated Trading button
        button = WebDriverWait(self.driver, 3).until(
            EC.element_to_be_clickable((By.XPATH, "//button[contains(., 'Start Simulated Trading')]"))
        )
        self.driver.execute_script(
            "arguments[0].scrollIntoView({behavior: 'smooth', block: 'center'});", button)
        time.sleep(0.2)
        button.click()
        logging.info("Fallback1: Clicked 'Start Simulated Trading' button (standard click)")
        return
    except:
        pass

    # Then try Launch button
    try:
        button = WebDriverWait(self.driver, 3).until(
            EC.element_to_be_clickable((By.XPATH, "//button[.//span[text()='Launch']]"))
        )
        self.driver.execute_script(
            "arguments[0].scrollIntoView({behavior: 'smooth', block: 'center'});", button)
        time.sleep(0.2)
        button.click()
        logging.info("Fallback1: Clicked 'Launch' button (standard click)")
        return
    except:
        pass

    # Then try Access Simulation button
    button = WebDriverWait(self.driver, 2).until(
        EC.element_to_be_clickable((By.XPATH, "//button[.//span[text()='Access Simulation']]"))
    )
    self.driver.execute_script("arguments[0].scrollIntoView({behavior: 'smooth', block: 'center'});", button)
    time.sleep(0.2)
    button.click()
    logging.info("Fallback1: Clicked 'Access Simulation' button (standard click)")
except Exception as e:
    logging.warning(f"Fallback1 failed: {e}")
    raise
```

Method: TradovateAccount._launch_simulation_tab_fallback2


```
def _launch_simulation_tab_fallback2(self)
```

Fallback 2: Try clicking any button with 'Start Simulated Trading', 'Launch' or 'Simulation' in span text.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def _launch_simulation_tab_fallback2(self):
    """Fallback 2: Try clicking any button with 'Start Simulated Trading', 'Launch' or 'Simulation' in span text"""
    try:
        # First try Start Simulated Trading
        try:
            button = WebDriverWait(self.driver, 3).until(
                EC.element_to_be_clickable((By.XPATH, "//button[contains(., 'Simulated Trading')]"))
            )
            self.driver.execute_script(
                "arguments[0].scrollIntoView({behavior: 'smooth', block: 'center'});", button)
            time.sleep(0.2)
            button.click()
            logging.info("Fallback2: Clicked button with 'Simulated Trading' text (standard click)")
            return
        except:
            pass

        # Then try Launch
        try:
            button = WebDriverWait(self.driver, 3).until(
                EC.element_to_be_clickable((By.XPATH, "//button[//span[contains(text(), 'Launch')]]"))
            )
            self.driver.execute_script(
                "arguments[0].scrollIntoView({behavior: 'smooth', block: 'center'});", button)
            time.sleep(0.2)
            button.click()
            logging.info("Fallback2: Clicked button with 'Launch' in span (standard click)")
            return
        except:
            pass

        # Then try Simulation
        button = WebDriverWait(self.driver, 2).until(
            EC.element_to_be_clickable((By.XPATH, "//button[//span[contains(text(), 'Simulation')]]"))
        )
        self.driver.execute_script("arguments[0].scrollIntoView({behavior: 'smooth', block: 'center'});", button)
        time.sleep(0.2)
        button.click()
```

```

        logging.info("Fallback2: Clicked button with 'Simulation' in span (standard click)")
    except Exception as e:
        logging.warning(f"Fallback2 failed: {e}")
        raise

```

Method: TradovateAccount._launch_simulation_tab_fallback3

```
def _launch_simulation_tab_fallback3(self)
```

Fallback 3: Try clicking any button with 'Launch' or 'Simulation' in its text.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def _launch_simulation_tab_fallback3(self):
    """Fallback 3: Try clicking any button with 'Launch' or 'Simulation' in its text."""
    try:
        # First try Launch
        try:
            button = WebDriverWait(self.driver, 3).until(
                EC.element_to_be_clickable((By.XPATH, "//button[contains(text(), 'Launch')]"))
            )
            self.driver.execute_script(
                "arguments[0].scrollIntoView({behavior: 'smooth', block: 'center'});", button)
            time.sleep(0.2)
            button.click()
            logging.info("Fallback3: Clicked button with 'Launch' in text (standard click)")
            return
        except:
            pass

        # Then try Simulation
        button = WebDriverWait(self.driver, 2).until(
            EC.element_to_be_clickable((By.XPATH, "//button[contains(text(), 'Simulation')]"))
        )
        self.driver.execute_script("arguments[0].scrollIntoView({behavior: 'smooth', block: 'center'});",
            button)
        time.sleep(0.2)
        button.click()
        logging.info("Fallback3: Clicked button with 'Simulation' in text (standard click)")
    except Exception as e:
        logging.warning(f"Fallback3 failed: {e}")
        raise

```

Method: TradovateAccount._fast_input_with_validation

```
def _fast_input_with_validation(self, element, value, field_name='field')
```

Enhanced input method using proven Selenium approach that preserves text after typing

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def _fast_input_with_validation(self, element, value, field_name="field"):
    """Enhanced input method using proven Selenium approach that preserves text after typing"""
    try:
        max_attempts = 3
        for attempt in range(max_attempts):
            # Brief delay to let user see the field before we start typing
            time.sleep(0.5 if attempt == 0 else 0.3)
            logging.debug(f"Starting input for {field_name}: {value}")

            # Use proven Selenium approach - click, select all, delete, type character by character
            element.click()
            time.sleep(0.1)
            element.send_keys(Keys.CONTROL + "a")
            element.send_keys(Keys.DELETE)
            time.sleep(0.1)

            # Type character by character
            for char in str(value):
                element.send_keys(char)
                time.sleep(0.05) # Small delay between characters

            time.sleep(0.2) # Brief pause after typing is complete

            # Verify the value is there
            current_value = element.get_attribute("value")
            if current_value == str(value):
                logging.debug(f"[CHECK] {field_name} input successful: {value}")
                return True
            else:
                logging.warning(
                    f"[WARNING] {field_name} input attempt {attempt + 1}: expected {value}, got {current_value}"
                )
                if attempt < max_attempts - 1:
                    time.sleep(0.5)
                    continue

        logging.warning(f"[X] Input for {field_name} failed after {max_attempts} attempts")
```

```

        return False

    except Exception as e:
        logging.warning(f"Input with validation failed for {field_name}: {e}")
        return False

```

Method: TradovateAccount._complete_login_process

```
def _complete_login_process(self)
```

Complete the login process - verify account access with enhanced stability

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def _complete_login_process(self):
    """Complete the login process - verify account access with enhanced stability"""
    try:
        logging.info("Verifying login completion...")

        # SPEED OPTIMIZATION: Fast trading interface verification
        try:
            WebDriverWait(self.driver, 10).until( # Reduced from 20 to 10
                EC.visibility_of_element_located((By.XPATH,
                    "//li[contains(@class, 'lm_tab')]/span[text()='Order Ticket']"))
            )
            logging.info("[CHECK] Trading interface detected and loaded")
        except Exception as e:
            logging.warning(f"Order Ticket tab not found, trying alternative verification: {e}")
            # Alternative verification - look for any tab
            try:
                WebDriverWait(self.driver, 5).until( # Reduced from 10 to 5
                    EC.presence_of_element_located((By.XPATH, "//li[contains(@class, 'lm_tab')]"))
                )
                logging.info("[CHECK] Trading interface tabs detected")
            except Exception as e2:
                logging.warning(f"No tabs found, assuming minimal interface: {e2}")

        # SPEED OPTIMIZATION: Reduced interface stabilization time
        time.sleep(1.5) # Reduced from 3 to 1.5

        # Final verification - try to access account stats with enhanced error handling
        try:
            # Check if driver is still alive before attempting verification
            current_url = self.driver.current_url

```

```

        if "tradovate.com" not in current_url:
            raise Exception(f"Browser navigated away from Tradovate: {current_url}")

        # Try to get basic stats but don't fail if it doesn't work
        stats = self.get_account_stats()
        if stats:
            logging.info("[CHECK] Account stats accessible - full verification successful")
        else:
            logging.info("[CHECK] Basic login verified, stats not immediately available")

    except Exception as e:
        logging.warning(f"Account stats verification failed but login appears successful: {e}")
        # Don't fail completely - the login might still be successful

    # Mark as logged in if we got this far
    self.logged_in = True
    self._login_timestamp = time.time() # Track when we logged in
    logging.info("[CHECK] Tradovate login process completed successfully")
    print(f"[UNLOCK] Login state set to True at {time.strftime('%H:%M:%S')}")

    except Exception as e:
        logging.error(f"Login verification failed: {e}")
        self.logged_in = False
        raise Exception(f"Login verification failed: {str(e)}")

```

Method: TradovateAccount._launch_live_trading_tab

```
def _launch_live_trading_tab(self)
```

Launch the LIVE TRADING tab after successful authentication

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def _launch_live_trading_tab(self):
    """Launch the LIVE TRADING tab after successful authentication"""
    try:
        logging.info("Looking for LIVE TRADING button on 'Select a Trading Mode' page...")

        self._wait_for_no_overlay(timeout=2)
        time.sleep(1)

        # --- Selectors for LIVE TRADING button ---
        live_selectors = [
            # Primary selector provided by user

```

```

        "//button[@data-testid='live-trading-button']",

        # Fallbacks
        "//button[contains(., 'Start Trading')]",
        "//button[.//span[text()='Start Trading']]",
        "//button[contains(text(), 'Start Trading')]",
        "//button[contains(@class, 'MuiButton-contained') and contains(., 'Start Trading')]"
    ]

    # Try each selector
    live_button = None
    for selector in live_selectors:
        try:
            buttons = self.driver.find_elements(By.XPATH, selector)
            for button in buttons:
                if button.is_displayed() and button.is_enabled():
                    live_button = button
                    logging.info(f"Found LIVE TRADING button with selector: {selector}")
                    break
            if live_button:
                break
        except Exception as e:
            logging.debug(f"Selector failed: {selector} ({e})")
            continue

    if not live_button:
        raise Exception("Could not find LIVE TRADING button")

    # Click the button
    logging.info("Clicking LIVE TRADING button...")
    self.driver.execute_script("arguments[0].click();", live_button)

    # Dismiss any interstitial pages (e.g. 'Welcome to Tradovate Prop')
    time.sleep(1)
    self._dismiss_interstitial_pages(timeout=5)

    # Wait for interface to load
    logging.info("Waiting for LIVE TRADING interface to load...")
    WebDriverWait(self.driver, 60).until(
        EC.any_of(
            EC.visibility_of_element_located((By.XPATH,
                "//li[contains(@class, 'lm_tab')]/span[text()='Order Ticket']")),
            EC.visibility_of_element_located((By.XPATH, "//div[contains(@class, 'order-ticket')]"))
        )
    )
    logging.info("✓ LIVE TRADING interface loaded successfully")
    time.sleep(1)
    self._wait_for_no_overlay(timeout=10)

except Exception as e:
    logging.error(f"Failed to launch LIVE TRADING tab: {e}")
    raise

```

Method: TradovateAccount._launch_simulation_tab

```
def _launch_simulation_tab(self)
```

Launch the simulation tab after successful authentication - enhanced for reliability

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def _launch_simulation_tab(self):
    """Launch the simulation tab after successful authentication - enhanced for reliability"""
    try:
        logging.info("Looking for simulation access button on 'Select a Trading Mode' page...")

        self._wait_for_no_overlay(timeout=2) # Reduced from 5 to 2

        # Wait for the page to be fully loaded
        time.sleep(1) # Reduced from 2 to 1

        # --- More comprehensive selectors for simulation button (FASTER ORDER) ---
        simulation_selectors = [
            # NEW: Primary selector provided by user
            "//*[@data-testid='simulation-button']",

            # NEW: Handle 'Start Simulated Trading' button (primary)
            "//*[contains(@class, 'MuiButton-contained') and contains(., 'Start Simulated Trading')]",
            "//*[contains(text(), 'Start Simulated Trading')]",

            # Handle the 'Launch' button text (most common)
            "//*[contains(@class, 'MuiButton-contained') and contains(., 'Launch')]",
            "//*[contains(@class, 'fat-button') and contains(., 'Launch')]",

            # Handle the different broker scenario (Access Simulation)
            "//*[contains(@class, 'MuiButton-contained') and contains(., 'Access Simulation')]",
            "//*[contains(@class, 'fat-button') and contains(., 'Access Simulation')]",

            # Original selectors (keep for compatibility)
            "//*[contains(@class, 'MuiButton-root') and contains(., 'Access Simulation')]",
            "//*[contains(text(), 'Access Simulation')]",

            # Broader selectors (last resort)
            "//*[contains(text(), 'Access Simulation')]",
            "//*[contains(@class, 'MuiButton-root') and contains(., 'Simulation')]",
            "//*[contains(text(), 'Simulated Trading')]",
            "//*[contains(translate(text(), 'ABCDEFGHIJKLMNOPQRSTUVWXYZ', 'abcdefghijklmnopqrstuvwxyz'), 'ABCDEFGHIJKLMNOPQRSTUVWXYZ')]",
            "//*[contains(translate(text(), 'ABCDEFGHIJKLMNOPQRSTUVWXYZ', 'abcdefghijklmnopqrstuvwxyz'), 'abcdefghijklmnopqrstuvwxyz')]"
        ]

        # Try each selector (FASTER - reduced timeout)
        simulation_button = None
        for selector in simulation_selectors:
            try:
```

```

        buttons = self.driver.find_elements(By.XPATH, selector)
        for button in buttons:
            if button.is_displayed() and button.is_enabled():
                simulation_button = button
                logging.info(f"Found simulation button with selector: {selector}")
                break
        if simulation_button:
            break
    except Exception as e:
        logging.debug(f"Selector failed: {selector} ({e})")
        continue

if not simulation_button:
    # Try fallbacks faster - skip debugging for speed
    logging.warning("Direct selectors failed, trying fallbacks quickly")
    self._try_all_simulation_fallbacks_fast()
    return

# Try click methods (FASTER - fewer attempts)
click_success = False
for attempt in range(3): # Reduced from 5 to 3
    if click_success:
        break

    try:
        logging.info(f"Attempt {attempt+1} to click simulation button")

        if attempt == 0:
            # First try: JavaScript click (most reliable)
            self.driver.execute_script("arguments[0].click();", simulation_button)
            click_success = True
            logging.info("Simulation button clicked with JavaScript click")
        elif attempt == 1:
            # Second try: standard click
            simulation_button.click()
            click_success = True
            logging.info("Simulation button clicked with standard click")
        else:
            # Third try: ActionChains
            ActionChains(self.driver).move_to_element(simulation_button).click().perform()
            click_success = True
            logging.info("Simulation button clicked with ActionChains")
    except Exception as e:
        logging.warning(f"Click attempt {attempt+1} failed: {e}")
        time.sleep(0.5) # Reduced from 1 to 0.5

if not click_success:
    # Try fallbacks quickly
    logging.warning("All direct click methods failed, trying fallbacks")
    self._try_all_simulation_fallbacks_fast()
    return

# Dismiss any interstitial pages (e.g. 'Welcome to Tradovate Prop')
time.sleep(1)
self._dismiss_interstitial_pages(timeout=5)

# Wait for simulation interface to load - REDUCED TIMEOUT
# Give 1 minute for the page to load after clicking (reduced from 5 minutes)
logging.info("Waiting for simulation interface to load (up to 60 seconds)...")
try:

```



```

WebDriverWait(self.driver, 60).until( # Reduced from 300 to 60 seconds
    EC.any_of(
        EC.visibility_of_element_located((By.XPATH,
            "//li[contains(@class, 'lm_tab')]/span[text()='Order Ticket']")),
        EC.visibility_of_element_located((By.XPATH,
            "//div[contains(@class, 'order-ticket')]")),
        EC.visibility_of_element_located((By.XPATH, "//span[text()='Order Ticket']"))
    )
)
logging.info("✓ Simulation trading interface loaded successfully")
time.sleep(1) # Reduced from 2 to 1 second
self._wait_for_no_overlay(timeout=10)
except Exception as e:
    logging.warning(f"Trading interface loading verification timed out after 60 seconds: {e}")
    logging.info("Interface may still be loading, but simulation access was clicked")

except Exception as e:
    logging.error(f"Failed to launch simulation tab: {e}")
    self._try_all_simulation_fallbacks_fast()

```

Method: TradovateAccount._try_all_simulation_fallbacks_fast

```
def _try_all_simulation_fallbacks_fast(self)

    Try all simulation fallbacks in sequence - FASTER version
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.

Step by step (top-level body)

1. Assigns fallbacks = [self._launch_simulation_tab_fallback_different_broker, s....
2. **for** (i, fallback) in enumerate(fallbacks): iterates.
3. **raise** Exception('All simulation access methods failed. Please check the t...

Code (copy-paste safe)

```

def _try_all_simulation_fallbacks_fast(self):
    """Try all simulation fallbacks in sequence - FASTER version"""
    fallbacks = [
        self._launch_simulation_tab_fallback_different_broker, # NEW: Add the different broker fallback
        self._launch_simulation_tab_fallback1,
        self._launch_simulation_tab_fallback2,
        self._launch_simulation_tab_fallback3
    ]

    for i, fallback in enumerate(fallbacks):
        try:
            logging.info(f"Trying simulation fallback method {i+1}")
            fallback()

```

```

        # If we get here without exception, assume success
        logging.info(f"Simulation fallback method {i+1} succeeded")
        return
    except Exception as e:
        logging.warning(f"Simulation fallback method {i+1} failed: {e}")

# If all fallbacks fail, raise exception
raise Exception(
    "All simulation access methods failed. Please check the trading platform UI or try manual log

```

Method: TradovateAccount._launch_simulation_tab_fallback_different_broker

```
def _launch_simulation_tab_fallback_different_broker(self)
```

NEW: Fallback for different broker with specific button structure - supports both Launch and Access Simulation

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def _launch_simulation_tab_fallback_different_broker(self):
    """NEW: Fallback for different broker with specific button structure - supports both Launch and Access Simulation"""
    try:
        # First try Launch button (most common)
        try:
            button = WebDriverWait(self.driver, 3).until(
                EC.element_to_be_clickable((
                    By.XPATH,
                    "//button[contains(@class, 'MuiButton-contained') and contains(@class, 'fat-button')]
                ))
            )
            self.driver.execute_script("arguments[0].click();", button)
            logging.info("Different broker: Clicked 'Launch' button with JavaScript")
            time.sleep(2)
            return
        except:
            pass

        # Then try Access Simulation button structure
        try:
            button = WebDriverWait(self.driver, 2).until(
                EC.element_to_be_clickable((
                    By.XPATH,
                    "//button[contains(@class, 'MuiButton-contained') and contains(@class, 'fat-button')]
                ))
            )

```

```

        )
        self.driver.execute_script("arguments[0].click();", button)
        logging.info("Different broker: Clicked 'Access Simulation' button with JavaScript")
        time.sleep(2)
        return
    except:
        pass

    # Try alternative selectors for Launch
    try:
        button = WebDriverWait(self.driver, 2).until(
            EC.element_to_be_clickable((
                By.XPATH,
                "//button[contains(@class, 'fat-button') and contains(@class, 'full-width-button')
            ))
        )
        self.driver.execute_script("arguments[0].click();", button)
        logging.info("Different broker: Clicked Launch with alternative selector")
        time.sleep(2)
        return
    except:
        pass

    # Try alternative selectors for Access Simulation
    try:
        button = WebDriverWait(self.driver, 2).until(
            EC.element_to_be_clickable((
                By.XPATH,
                "//button[contains(@class, 'fat-button') and contains(@class, 'full-width-button')
            ))
        )
        self.driver.execute_script("arguments[0].click();", button)
        logging.info("Different broker: Clicked Access Simulation with alternative selector")
        time.sleep(2)
        return
    except:
        pass

    raise Exception("No Launch or Access Simulation buttons found")

except Exception as e:
    logging.warning(f"Different broker fallback failed: {e}")
    raise Exception(f"Different broker simulation access failed: {e}")

```

Method: TradovateAccount._launch_simulation_tab_fallback_new

```
def _launch_simulation_tab_fallback_new(self)
```

New fallback: Try tabbing to the button and using Enter key.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def _launch_simulation_tab_fallback_new(self):
    """New fallback: Try tabbing to the button and using Enter key."""
    try:
        # First try to find any button that might be the simulation button
        buttons = self.driver.find_elements(By.TAG_NAME, "button")
        simulation_button = None

        for button in buttons:
            try:
                if button.is_displayed() and button.is_enabled():
                    text = button.text.lower()
                    if 'simulation' in text or 'access' in text or 'launch' in text:
                        simulation_button = button
                        break
            except:
                continue

        if simulation_button:
            # Try to focus and press Enter
            self.driver.execute_script("arguments[0].focus();", simulation_button)
            time.sleep(1)
            ActionChains(self.driver).send_keys(Keys.RETURN).perform()
            logging.info("Pressed Enter on focused simulation/launch button")
            time.sleep(3)
            return

        # If no button found, try tabbing and pressing enter
        body = self.driver.find_element(By.TAG_NAME, "body")
        body.click() # Focus on body

        # Press tab multiple times to try to reach the button
        action = ActionChains(self.driver)
        for _ in range(10): # Try up to 10 tabs
            action.send_keys(Keys.TAB).perform()
            time.sleep(0.5)

        # Try pressing Enter after each tab
        action.send_keys(Keys.RETURN).perform()
        time.sleep(1)

        # Check if we've reached a new page
        try:
            if WebDriverWait(self.driver, 2).until(
                EC.presence_of_element_located((By.XPATH,
                    "//div[contains(@class, 'trading-interface') or contains(@class, 'chart')]"))
            ):
                logging.info("New page detected after tab+enter, likely successful")
                return
        except:
            continue

        raise Exception("Tab navigation failed to find simulation button")
```

```

except Exception as e:
    logging.warning(f"New fallback failed: {e}")
    raise

```

Method: TradovateAccount._debug_page_elements

```
def _debug_page_elements(self)
```

Enhanced debugging to identify available page elements

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def _debug_page_elements(self):
    """Enhanced debugging to identify available page elements"""
    try:
        logging.info("=== ENHANCED DEBUGGING: Page Elements Analysis ===")
        try:
            current_url = self.driver.current_url
            page_title = self.driver.title
            logging.info(f"Current URL: {current_url}")
            logging.info(f"Page Title: {page_title}")

            # Take screenshot for debugging
            screenshot_path = os.path.join(tempfile.gettempdir(),
                                           f"tradovate_debug_{int(time.time())}.png")
            self.driver.save_screenshot(screenshot_path)
            logging.info(f"Debug screenshot saved to: {screenshot_path}")
        except:
            pass

        # Capture and log all buttons
        try:
            buttons = self.driver.find_elements(By.TAG_NAME, "button")
            logging.info(f"=== All Buttons Found ({len(buttons)} total) ===")
            for i, button in enumerate(buttons[:20]): # Log first 20 buttons
                try:
                    text = button.text.strip()
                    classes = button.get_attribute("class") or ""
                    visible = button.is_displayed()
                    enabled = button.is_enabled()
                    status = f"'✓' if visible else 'x'}visible, {'✓' if enabled else 'x'}enabled"
                    if text:
                        logging.info(
                            f"Button {i+1}: '{text}' | Classes: {classes[:50]}... | Status: {status}"

```

```

        elif classes:
            logging.info(
                f"Button {i+1}: [no text] | Classes: {classes[:50]}... | Status: {status}"
            )
        except Exception as e:
            logging.debug(f"Button {i+1}: Error reading - {e}")

    # Find the most likely simulation button
    for button in buttons:
        try:
            if button.is_displayed() and button.is_enabled():
                text = button.text.lower()
                if 'simulation' in text or 'access simulation' in text:
                    rect = button.rect
                    logging.info(f"LIKELY TARGET: '{button.text}' at position {rect}")
        except:
            continue

    except:
        pass

except Exception as debug_e:
    logging.error(f"Debug logging failed: {debug_e}")

```

Method: TradovateAccount._wait_for_no_overlay

```
def _wait_for_no_overlay(self, timeout=3)
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def _wait_for_no_overlay(self, timeout=3):
    try:
        WebDriverWait(self.driver, timeout).until(
            EC.invisibility_of_element_located((By.CSS_SELECTOR, ".spinner, .loading, .overlay"))
        )
    except Exception:
        pass

```

Method: TradovateAccount._dismiss_interstitial_pages

```
def _dismiss_interstitial_pages(self, timeout=5)
```

Dismiss intermittent interstitial pages that appear before the trading interface. Handles: - 'Welcome to Tradovate Prop' full-screen modal with Continue button - Cookie consent banners with Accept Cookies button - Any other MUI dialog with a Continue/OK/Accept button

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't

have to handle it.

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `deadline = time.time() + timeout`.
2. Assigns `dismissed_any = False`.
3. **while** `time.time() < deadline`: loops until the condition is false.
4. **if** `dismissed_any`: branches conditionally.
5. **return** `dismissed_any`.

Code (copy-paste safe)

```
def _dismiss_interstitial_pages(self, timeout=5):
    """Dismiss intermittent interstitial pages that appear before the trading interface.

    Handles:
    - 'Welcome to Tradovate Prop' full-screen modal with Continue button
    - Cookie consent banners with Accept Cookies button
    - Any other MUI dialog with a Continue/OK/Accept button
    """
    deadline = time.time() + timeout
    dismissed_any = False

    while time.time() < deadline:
        found = False

        # 1) Cookie consent banner
        try:
            cookie_btns = self.driver.find_elements(By.XPATH,
                "//button[contains(translate(., 'ABCDEFGHIJKLMNOPQRSTUVWXYZ', 'abcdefghijklmnopqrstuvwxyz'))"
            )
            for btn in cookie_btns:
                if btn.is_displayed() and btn.is_enabled():
                    self.driver.execute_script("arguments[0].click();", btn)
                    logging.info("Dismissed cookie consent banner")
                    found = True
                    dismissed_any = True
                    break
        except Exception:
            pass

        # 2) Full-screen 'Welcome to Tradovate Prop' modal – Continue button
        try:
            continue_btns = self.driver.find_elements(By.XPATH,
                "//div[contains(@class, 'MuiDialog-paperFullScreen')]"
                "//button[contains(translate(., 'ABCDEFGHIJKLMNOPQRSTUVWXYZ', 'abcdefghijklmnopqrstuvwxyz'))"
            )
            for btn in continue_btns:
                if btn.is_displayed() and btn.is_enabled():
                    self.driver.execute_script("arguments[0].click();", btn)
                    logging.info("Dismissed 'Welcome to Tradovate Prop' interstitial (Continue)")
                    found = True
                    dismissed_any = True
                    time.sleep(1)
                    break
```



```

except Exception as e:
    logging.error(f"Error closing browser: {e}")
finally:
    if self.logged_in:
        print(f"[LOCK] Login state changed to False (browser closed) at {time.strftime('%H:%M:%S')}")
    self.logged_in = False
    self._login_timestamp = None
    self.driver = None

```

Method: TradovateAccount.get_account_stats

```
def get_account_stats(self)
```

Optimized stats - return cached during order placement OR if recently fetched

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **if** not self.lock.acquire(blocking=False): branches conditionally.
2. **try** block with 0 **except** clauses, plus a **finally**.

Code (copy-paste safe)

```

def get_account_stats(self):
    """Optimized stats - return cached during order placement OR if recently fetched"""
    # Use lock to prevent conflict with order placement
    if not self.lock.acquire(blocking=False):
        # If locked (e.g. placing order), return cached stats immediately
        return getattr(self, '_cached_stats', {
            "Account Number": "Trading...",
            "Balance": "N/A",
            "Profit/Loss": "N/A",
            "Open Trades": "N/A",
            "Symbol": "",
            "Direction": ""
        })

    try:
        if getattr(self, '_placing_order', False):
            return getattr(self, '_cached_stats', {
                "Account Number": "Trading...",
                "Balance": "N/A",
                "Profit/Loss": "N/A",
                "Open Trades": "N/A",
                "Symbol": "",
                "Direction": ""
            })
    
```

```

# PERFORMANCE OPTIMIZATION: Return cached stats if fetched within the last 2 seconds
# This prevents excessive DOM queries when GUI polls every 1 second
cache_ttl = 2.0 # Cache time-to-live in seconds
current_time = time.time()
last_fetch_time = getattr(self, '_stats_last_fetch_time', 0)
time_since_fetch = current_time - last_fetch_time

if time_since_fetch < cache_ttl and hasattr(self, '_cached_stats'):
    # Return cached stats (still fresh)
    return self._cached_stats

# Always try to fetch stats from the live page, even if not self.logged_in
try:
    if not self.driver:
        return {
            "Account Number": "Not Connected",
            "Balance": "N/A",
            "Profit/Loss": "N/A",
            "Open Trades": "N/A",
            "Symbol": "",
            "Direction": ""
        }

    stats = {
        "Account Number": "Unknown",
        "Balance": "N/A",
        "Profit/Loss": "N/A",
        "Open Trades": "0",
        "Symbol": "",
        "Direction": ""
    }

    # Check for Order Ticket tab quickly
    try:
        order_ticket_tabs = self.driver.find_elements(
            By.XPATH, "//li[contains(@class, 'lm_tab')]/span[text()='Order Ticket']"
        )
        if not order_ticket_tabs:
            stats["Account Number"] = "Not Connected"
            return stats
    except Exception:
        stats["Account Number"] = "Not Connected"
        return stats

    # --- Try to extract account number from visible elements ---
    try:
        account_elements = self.driver.find_elements(By.CSS_SELECTOR,
            "[class*='account'], [data-testid*='account']")
        for el in account_elements:
            text = el.text.strip()
            if text and any(char.isdigit() for char in text):
                lines = text.splitlines()
                for line in lines:
                    if line.strip() and any(char.isdigit() for char in line):
                        acc = line.strip()
                        if len(acc) >= 9:
                            stats["Account Number"] = acc[:4] + "..." + acc[-5:]
                        else:
                            stats["Account Number"] = acc
                        break

```

```

        if stats["Account Number"] != "Unknown":
            break
    except Exception:
        pass

    # --- Try to extract balance ---
    try:
        balance_elements = self.driver.find_elements(By.CSS_SELECTOR,
            "[class*='balance'], [class*='equity']")
        for el in balance_elements:
            balance_text = el.text
            import re
            balance_match = re.search(r'[\$]?([0-9,]+\.[0-9]*)', balance_text)
            if balance_match:
                stats["Balance"] = f"${balance_match.group(1)}"
                break
    except Exception:
        pass

    # --- Try to extract open trades ---
    try:
        open_trades_elements = self.driver.find_elements(By.XPATH,
            "//span[contains(text(),'Open') and contains(text(),'Trade')]")
        for el in open_trades_elements:
            text = el.text
            import re
            match = re.search(r'(\d+)', text)
            if match:
                stats["Open Trades"] = match.group(1)
                break
    except Exception:
        pass

    # Cache the stats WITH TIMESTAMP for performance optimization
    self._cached_stats = stats
    self._stats_last_fetch_time = time.time()
    return stats

except Exception as e:
    logging.warning(f"Error fetching stats: {e}")
    return {
        "Account Number": "Error",
        "Balance": "N/A",
        "Profit/Loss": "N/A",
        "Open Trades": "N/A",
        "Symbol": "",
        "Direction": ""
    }
finally:
    self.lock.release()

```

Method: TradovateAccount.refresh_positions_tab

```
def refresh_positions_tab(self)
```

Refresh the Tradovate Positions tab to update open trades info.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def refresh_positions_tab(self):
    """Refresh the Tradovate Positions tab to update open trades info."""
    try:
        driver = self.driver
        positions_tab = WebDriverWait(driver, 2).until(
            EC.element_to_be_clickable((By.XPATH,
                "//li[contains(@class, 'lm_tab')]//span[text()='Positions']"))
        )
        positions_tab.click()
        self._wait_for_no_overlay(timeout=0.5)
    except Exception:
        pass
```

Method: TradovateAccount.prepare_buy_order

```
def prepare_buy_order(self, symbol, qty, tp=None, sl=None)
```

THREADING OPTIMIZATION: Prepare BUY order (symbol search, quantity, ATM) WITHOUT clicking button. Use this before threading to minimize time inside parallel execution. Returns: True if preparation successful, False otherwise

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **with self.lock**: enters a context manager.

Code (copy-paste safe)

```
def prepare_buy_order(self, symbol, qty, tp=None, sl=None):
    """
    THREADING OPTIMIZATION: Prepare BUY order (symbol search, quantity, ATM) WITHOUT clicking button.
    Use this before threading to minimize time inside parallel execution.

    Returns: True if preparation successful, False otherwise
    """
    with self.lock:
        try:
            print(f"■ PRE-THREAD] Preparing Tradovate BUY order: {symbol} x{qty}")
```

```

# Quick interface check
current_url = self.driver.current_url
if not current_url or "tradovate" not in current_url:
    print(f"■ PRE-THREAD] Not on Tradovate page, logging in...")
    self.login()

# Prepare order (slow operations done before threading)
self._wait_for_no_overlay(timeout=0.2)
self._select_symbol(symbol)
self._set_quantity(qty)
self._setup_atm(tp, sl)

print(f"■ [■ PRE-THREAD] Order prepared - ready for button click")
return True

except Exception as e:
    print(f"■ PRE-THREAD] Preparation failed: {e}")
    return False

```

Method: TradovateAccount.execute_prepared_buy_order

```
def execute_prepared_buy_order(self, skip_positions_refresh=False)
```

THREADING OPTIMIZATION: Execute already-prepared BUY order (just click button). Call prepare_buy_order() first, then use this inside threading for instant execution. Returns: None (raises exception on failure)

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.

Step by step (top-level body)

1. **with self.lock:** enters a context manager.

Code (copy-paste safe)

```

def execute_prepared_buy_order(self, skip_positions_refresh=False):
    """
    THREADING OPTIMIZATION: Execute already-prepared BUY order (just click button).
    Call prepare_buy_order() first, then use this inside threading for instant execution.

    Returns: None (raises exception on failure)
    """
    with self.lock:
        try:
            print(f"■ THREAD-EXEC] Clicking BUY button...")
            self._do_place_order("buy", on_click=None, skip_positions_refresh=skip_positions_refresh)

```

```

        print(f"■ [■ THREAD-EXEC] BUY button clicked")
    except Exception as e:
        print(f"■ [■ THREAD-EXEC] Execution failed: {e}")
        raise

```

Method: TradovateAccount.prepare_sell_order

```
def prepare_sell_order(self, symbol, qty, tp=None, sl=None)
```

THREADING OPTIMIZATION: Prepare SELL order (symbol search, quantity, ATM) WITHOUT clicking button. Use this before threading to minimize time inside parallel execution. Returns: True if preparation successful, False otherwise

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **with self.lock:** enters a context manager.

Code (copy-paste safe)

```

def prepare_sell_order(self, symbol, qty, tp=None, sl=None):
    """
    THREADING OPTIMIZATION: Prepare SELL order (symbol search, quantity, ATM) WITHOUT clicking button
    Use this before threading to minimize time inside parallel execution.

    Returns: True if preparation successful, False otherwise
    """
    with self.lock:
        try:
            print(f"■ [■ PRE-THREAD] Preparing Tradovate SELL order: {symbol} x{qty}")

            # Quick interface check
            current_url = self.driver.current_url
            if not current_url or "tradovate" not in current_url:
                print(f"■ [■ PRE-THREAD] Not on Tradovate page, logging in...")
                self.login()

            # Prepare order (slow operations done before threading)
            self._wait_for_no_overlay(timeout=0.2)
            self._select_symbol(symbol)
            self._set_quantity(qty)
            self._setup_atm(tp, sl)

            print(f"■ [■ PRE-THREAD] Order prepared - ready for button click")
            return True

        except Exception as e:
            print(f"■ [■ PRE-THREAD] Preparation failed: {e}")

```

```
return False
```

Method: TradovateAccount.execute_prepared_sell_order

```
def execute_prepared_sell_order(self, skip_positions_refresh=False)
```

THREADING OPTIMIZATION: Execute already-prepared SELL order (just click button). Call prepare_sell_order() first, then use this inside threading for instant execution. Returns: None (raises exception on failure)

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.

Step by step (top-level body)

1. **with self.lock:** enters a context manager.

Code (copy-paste safe)

```
def execute_prepared_sell_order(self, skip_positions_refresh=False):
    """
    THREADING OPTIMIZATION: Execute already-prepared SELL order (just click button).
    Call prepare_sell_order() first, then use this inside threading for instant execution.

    Returns: None (raises exception on failure)
    """
    with self.lock:
        try:
            print(f"■ THREAD-EXEC Clicking SELL button...")
            self._do_place_order("sell", on_click=None, skip_positions_refresh=skip_positions_refresh)
            print(f"■ [■ THREAD-EXEC] SELL button clicked")
        except Exception as e:
            print(f"■ THREAD-EXEC Execution failed: {e}")
            raise
```

Method: TradovateAccount.get_active_account

```
def get_active_account(self)
```

Get the full (unmasked) account number currently active in the Tradovate UI. DOM structure: div.account-selector-wrapper div.account-selector.dropdown div[data-toggle="dropdown"] div.account > div.name > div ← contains account number ul.dropdown-menu li.selected > a.account > div.name > div.main ← selected account Returns: str: The account number string (e.g. "FTDFYSLX50349776193"), or None.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **if not self.driver:** branches conditionally.
2. **try** block with 1 **except** clause.
3. **return None**.

Code (copy-paste safe)

```
def get_active_account(self):
    """Get the full (unmasked) account number currently active in the Tradovate UI.

    DOM structure:
    div.account-selector-wrapper
    div.account-selector.dropdown
    div[data-toggle="dropdown"]
    div.account > div.name > div ← contains account number
    ul.dropdown-menu
    li.selected > a.account > div.name > div.main ← selected account

    Returns:
    str: The account number string (e.g. "FTDFYSLX50349776193"), or None.
    """
    if not self.driver:
        return None
    try:
        # Primary: Read from the header account-selector name element
        name_els = self.driver.find_elements(
            By.CSS_SELECTOR, "div.account-selector div.name div")
        for el in name_els:
            try:
                text = el.text.strip()
                if text and any(c.isdigit() for c in text) and len(text) >= 6:
                    return text
            except Exception:
                continue

        # Fallback: Read from the selected dropdown item
        selected_els = self.driver.find_elements(
            By.CSS_SELECTOR, "ul.dropdown-menu li.selected div.main")
        for el in selected_els:
            try:
                text = el.text.strip()
                if text and any(c.isdigit() for c in text):
                    return text
            except Exception:
                continue

        # Last resort: Any element with class containing 'account' that has digits
```



```

        account_elements = self.driver.find_elements(
            By.CSS_SELECTOR, "[class*='account']")
        for el in account_elements:
            text = el.text.strip()
            if text and any(c.isdigit() for c in text):
                for line in text.splitlines():
                    line = line.strip()
                    if line and any(c.isdigit() for c in line) and len(line) >= 6:
                        return line

    except Exception as e:
        logging.warning(f"get_active_account failed: {e}")
    return None

```

Method: TradovateAccount.get_all_accounts

```
def get_all_accounts(self)
```

Open the account dropdown and return a list of all account names. Returns: list[str]: Account name strings from the dropdown, or empty list on failure.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **if not self.driver:** branches conditionally.
2. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def get_all_accounts(self):
    """Open the account dropdown and return a list of all account names.

    Returns:
        list[str]: Account name strings from the dropdown, or empty list on failure.
    """
    if not self.driver:
        return []

    try:
        # Open the dropdown
        triggers = self.driver.find_elements(
            By.CSS_SELECTOR, "div.account-selector [data-toggle='dropdown']")
        opened = False
        for trigger in triggers:
            try:
                if trigger.is_displayed():
                    self.driver.execute_script("arguments[0].click();", trigger)
                    time.sleep(1.0)
                    opened = True
                    break
            except Exception:

```

```

        continue

    if not opened:
        # Fallback: click account-selector itself
        selectors = self.driver.find_elements(By.CSS_SELECTOR, "div.account-selector")
        for sel in selectors:
            try:
                if sel.is_displayed():
                    self.driver.execute_script("arguments[0].click();", sel)
                    time.sleep(1.0)
                    opened = True
                    break
            except Exception:
                continue

    if not opened:
        print("[GET_ALL_ACCOUNTS] Could not open dropdown")
        return []

    # Read all account entries
    account_links = self.driver.find_elements(
        By.CSS_SELECTOR, "div.account-selector ul.dropdown-menu li a.account:not(.logout)")

    names = []
    for link in account_links:
        try:
            main_divs = link.find_elements(By.CSS_SELECTOR, "div.name div.main")
            if main_divs:
                name = main_divs[0].text.strip()
            else:
                name = link.text.strip().split('\n')[0].strip()
            if name:
                names.append(name)
        except Exception:
            continue

    # Close dropdown safely with Escape key (never click body – could hit Logout)
    try:
        from selenium.webdriver.common.keys import Keys
        self.driver.find_element(By.TAG_NAME, "body").send_keys(Keys.ESCAPE)
        time.sleep(0.3)
    except Exception:
        # Fallback: re-click the trigger to toggle dropdown closed
        try:
            for trigger in triggers:
                if trigger.is_displayed():
                    self.driver.execute_script("arguments[0].click();", trigger)
                    time.sleep(0.3)
                    break
        except Exception:
            pass

    print(f"[GET_ALL_ACCOUNTS] Found {len(names)} accounts: {names}")
    return names

except Exception as e:
    logging.warning(f"get_all_accounts failed: {e}")
    return []

```

Method: TradovateAccount.switch_account

```
def switch_account(self, target_account)
```

Switch to a specific sub-account via Tradovate's account dropdown. DOM structure (from live Tradovate page): `div.account-selector-wrapper` `div.account-selector.dropdown` `div[data-toggle="dropdown"]` ← *CLICK to open dropdown* `ul.dropdown-menu` ← *dropdown list* `li > a.account` ← *each account entry (skip a.logout)* `div.name > div.main` ← *account number text* Args: `target_account`: Account number string (or unique substring) to select. Returns: `bool`: True if account was switched (or already active), False on failure.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. `if not self.driver or not target_account`: branches conditionally.
2. Imports `re` (lazy import inside the function).
3. Assigns `target_account = str(target_account).strip()`.
4. Assigns `target_lower = target_account.lower()`.
5. Assigns `digit_match = _re.search('(\d{5,})$', target_account)`.
6. Assigns `digit_suffix = digit_match.group(1)` if `digit_match` else `None`.
7. Calls `print(...)` for its side effect.
8. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def switch_account(self, target_account):
    """Switch to a specific sub-account via Tradovate's account dropdown.

    DOM structure (from live Tradovate page):
    div.account-selector-wrapper
    div.account-selector.dropdown
    div[data-toggle="dropdown"]    ← CLICK to open dropdown
    ul.dropdown-menu              ← dropdown list
    li > a.account                ← each account entry (skip a.logout)
    div.name > div.main           ← account number text

    Args:
        target_account: Account number string (or unique substring) to select.

    Returns:
        bool: True if account was switched (or already active), False on failure.
    """
    if not self.driver or not target_account:
```

```

        return False

import re as _re
target_account = str(target_account).strip()
target_lower = target_account.lower()
digit_match = _re.search(r'(\d{5,})$', target_account)
digit_suffix = digit_match.group(1) if digit_match else None
print(f"[ACCOUNT SWITCH] Target: {target_account} (suffix: {digit_suffix})")

try:
    # ■■ Check if already on the correct account ■■
    current = self.get_active_account()
    if current:
        cur_lower = current.lower()
        if target_lower in cur_lower or cur_lower in target_lower:
            print(f"[ACCOUNT SWITCH] Already on correct account: {current}")
            return True
        if digit_suffix and current.endswith(digit_suffix):
            print(f"[ACCOUNT SWITCH] Already correct (suffix match): {current}")
            return True

    print(f"[ACCOUNT SWITCH] Current: {current} → Need: {target_account}")

    max_attempts = 3
    for attempt in range(1, max_attempts + 1):
        if attempt > 1:
            print(f"[ACCOUNT SWITCH] ■■ Retry {attempt}/{max_attempts} ■■")
            time.sleep(1)

        # ■■ STEP 1: Click div[data-toggle="dropdown"] inside div.account-selector to open ■■
        dropdown_opened = False
        triggers = self.driver.find_elements(
            By.CSS_SELECTOR, "div.account-selector [data-toggle='dropdown']")
        for trigger in triggers:
            try:
                if trigger.is_displayed():
                    print(f"[ACCOUNT SWITCH] STEP 1: Clicking div[data-toggle=dropdown]")
                    self.driver.execute_script("arguments[0].click();", trigger)
                    time.sleep(1.5)
                    dropdown_opened = True
                    break
            except Exception:
                continue

        # Fallback: click the account-selector div itself
        if not dropdown_opened:
            selectors = self.driver.find_elements(
                By.CSS_SELECTOR, "div.account-selector")
            for sel in selectors:
                try:
                    if sel.is_displayed():
                        print(f"[ACCOUNT SWITCH] STEP 1 fallback: Clicking div.account-selector")
                        self.driver.execute_script("arguments[0].click();", sel)
                        time.sleep(1.5)
                        dropdown_opened = True
                        break
                except Exception:
                    continue

        if not dropdown_opened:

```

```

        print(f"[ACCOUNT SWITCH] Could not open dropdown (attempt {attempt})")
        continue

# ■■■ STEP 2: Read all accounts from ul.dropdown-menu ■■■
# Each account: li > a.account (not .logout) > div.name > div.main
account_links = self.driver.find_elements(
    By.CSS_SELECTOR, "div.account-selector ul.dropdown-menu li a.account:not(.logout)")

print(f"[ACCOUNT SWITCH] STEP 2: Found {len(account_links)} account entries in dropdown")

matched_link = None
for link in account_links:
    try:
        # Get the account name from div.main inside this <a>
        main_divs = link.find_elements(By.CSS_SELECTOR, "div.name div.main")
        if main_divs:
            acct_name = main_divs[0].text.strip()
        else:
            acct_name = link.text.strip().split('\n')[0].strip()

        is_selected = False
        try:
            parent_li = link.find_element(By.XPATH, "..")
            is_selected = "selected" in (parent_li.get_attribute("class") or "")
        except Exception:
            pass

        marker = " ← CURRENT" if is_selected else ""
        print(f" - {acct_name}{marker}")

        # Match: exact substring or suffix
        if target_lower in acct_name.lower() or acct_name.lower() in target_lower:
            if not is_selected:
                matched_link = link
                print(f"[ACCOUNT SWITCH] MATCH (exact): {acct_name}")
                break
            elif digit_suffix and acct_name.endswith(digit_suffix):
                if not is_selected:
                    matched_link = link
                    print(f"[ACCOUNT SWITCH] MATCH (suffix {digit_suffix}): {acct_name}")
                    break
    except Exception as item_err:
        logging.debug(f"[ACCOUNT SWITCH] Error reading dropdown item: {item_err}")
        continue

if not matched_link:
    print(f"[ACCOUNT SWITCH] '{target_account}' not found in dropdown (attempt {attempt})")
    # Close dropdown before retry
    try:
        from selenium.webdriver.common.keys import Keys as _Keys
        self.driver.find_element(By.TAG_NAME, 'body').send_keys(_Keys.ESCAPE)
        time.sleep(0.3)
    except Exception:
        pass
    continue

# ■■■ STEP 3: Click the matched account link ■■■
print(f"[ACCOUNT SWITCH] STEP 3: Clicking matched account (attempt {attempt})")

# Try multiple click methods – JS click often doesn't work on <a> elements

```

```

click_succeeded = False

# Method 1: Selenium native .click() (most reliable for <a> links)
try:
    matched_link.click()
    click_succeeded = True
    print(f"[ACCOUNT SWITCH] Used native .click()")
except Exception as e1:
    print(f"[ACCOUNT SWITCH] Native click failed: {e1}")

# Method 2: ActionChains click
if not click_succeeded:
    try:
        ActionChains(self.driver).move_to_element(matched_link).click().perform()
        click_succeeded = True
        print(f"[ACCOUNT SWITCH] Used ActionChains click")
    except Exception as e2:
        print(f"[ACCOUNT SWITCH] ActionChains click failed: {e2}")

# Method 3: JS click on the <a> element
if not click_succeeded:
    try:
        self.driver.execute_script("arguments[0].click();", matched_link)
        click_succeeded = True
        print(f"[ACCOUNT SWITCH] Used JS click")
    except Exception as e3:
        print(f"[ACCOUNT SWITCH] JS click failed: {e3}")

# Method 4: JS click on div.main inside the link
if not click_succeeded:
    try:
        inner = matched_link.find_elements(By.CSS_SELECTOR, "div.name div.main")
        if inner:
            self.driver.execute_script("arguments[0].click();", inner[0])
            click_succeeded = True
            print(f"[ACCOUNT SWITCH] Used JS click on inner div.main")
    except Exception as e4:
        print(f"[ACCOUNT SWITCH] Inner click failed: {e4}")

if not click_succeeded:
    print(f"[ACCOUNT SWITCH] ALL click methods failed (attempt {attempt})")
    continue

time.sleep(2) # wait for account switch

# ■■■ STEP 4: VERIFY the account actually changed ■■■
new_account = self.get_active_account()
if new_account:
    new_lower = new_account.lower()
    if target_lower in new_lower or new_lower in target_lower:
        print(f"[ACCOUNT SWITCH] ■ VERIFIED – now on: {new_account}")
        self._stats_last_fetch_time = 0
        return True
    elif digit_suffix and new_account.endswith(digit_suffix):
        print(f"[ACCOUNT SWITCH] ■ VERIFIED (suffix) – now on: {new_account}")
        self._stats_last_fetch_time = 0
        return True
    else:
        print(
            f"[ACCOUNT SWITCH] ■ Account did NOT change (attempt {attempt}). Still on: {"

```

```

        self._stats_last_fetch_time = 0
        # Will retry on next iteration
    else:
        print(f"[ACCOUNT SWITCH] ■ Could not read account after click (attempt {attempt})")
        self._stats_last_fetch_time = 0
        # Will retry on next iteration

    # All attempts exhausted
    print(f"[ACCOUNT SWITCH] ■ FAILED after {max_attempts} attempts for '{target_account}'")
    return False

except Exception as e:
    logging.error(f"[ACCOUNT SWITCH] Exception: {e}")
    try:
        self.driver.find_element(By.TAG_NAME, 'body').send_keys(Keys.ESCAPE)
    except Exception:
        pass
    return False

```

Method: TradovateAccount.verify_active_account

```
def verify_active_account(self, expected_account)
```

Verify that the currently active account matches the expected one. Args: expected_account: Expected account number (or substring). Returns: bool: True if the active account matches, False otherwise.

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **if** not expected_account: branches conditionally.
2. Assigns current = self.get_active_account().
3. **if** not current: branches conditionally.
4. Assigns expected_lower = str(expected_account).strip().lower().
5. **if** expected_lower in current.lower(): branches conditionally.
6. Calls logging.warning(...) for its side effect.
7. **return** False.

Code (copy-paste safe)

```

def verify_active_account(self, expected_account):
    """Verify that the currently active account matches the expected one.

    Args:
        expected_account: Expected account number (or substring).

    Returns:
        bool: True if the active account matches, False otherwise.
    """
    if not expected_account:
        return True # No expectation set, skip check

    current = self.get_active_account()

```

```

if not current:
    logging.warning(f"[ACCOUNT] Cannot verify – unable to read active account")
    return False

expected_lower = str(expected_account).strip().lower()
if expected_lower in current.lower():
    logging.info(f"[ACCOUNT] Verified: active={current}, expected={expected_account}")
    return True

logging.warning(f"[ACCOUNT] MISMATCH: active={current}, expected={expected_account}")
return False

```

Method: TradovateAccount.buy_market

```

def buy_market(self, symbol=None, qty=1, tp=None, sl=None, on_click=None, skip_positions_refresh=False,
    prop_firm=None, phase=None, account_size=None, expected_account=None)

```

Step by step (top-level body)

1. Calls self._place_order_side(...) for its side effect.

Code (copy-paste safe)

```

def buy_market(self, symbol=None, qty=1, tp=None, sl=None, on_click=None, skip_positions_refresh=False,
    prop_firm=None, phase=None, account_size=None, expected_account=None):
    self._place_order_side(symbol, qty, "buy", tp, sl, on_click=on_click,
        skip_positions_refresh=skip_positions_refresh, prop_firm=prop_firm, phase=phase, account_size=account_size)

```

Method: TradovateAccount.sell_market

```

def sell_market(self, symbol=None, qty=1, tp=None, sl=None, on_click=None, skip_positions_refresh=False,
    prop_firm=None, phase=None, account_size=None, expected_account=None)

```

Step by step (top-level body)

1. Calls self._place_order_side(...) for its side effect.

Code (copy-paste safe)

```

def sell_market(self, symbol=None, qty=1, tp=None, sl=None, on_click=None, skip_positions_refresh=False,
    prop_firm=None, phase=None, account_size=None, expected_account=None):
    self._place_order_side(symbol, qty, "sell", tp, sl, on_click=on_click,
        skip_positions_refresh=skip_positions_refresh, prop_firm=prop_firm, phase=phase, account_size=account_size)

```

Method: TradovateAccount._set_quantity

```

def _set_quantity(self, qty)

```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.

Step by step (top-level body)

1. Calls `print(...)` for its side effect.
2. `try` block with 1 `except` clause.

Code (copy-paste safe)

```
def _set_quantity(self, qty):
    print(f"[ZAP] Setting quantity: {qty}")
    try:
        qty_input = WebDriverWait(self.driver, 3).until(
            EC.element_to_be_clickable((By.XPATH,
                "//div[contains(@class, 'trading-ticket-main-entry')]/div[contains(@class, 'qty')]/"
            ))

        # SPEED OPTIMIZATION: Check if quantity is already correct first
        current_qty = qty_input.get_attribute("value") or ""
        if current_qty.strip() == str(qty).strip():
            print(f"[ZAP] Quantity {qty} already set - skipping update")
            return

        # Update quantity with proper input clearing delays
        qty_input.click()
        time.sleep(0.1) # Increased delay for reliable clearing
        qty_input.send_keys(Keys.CONTROL + "a")
        qty_input.send_keys(Keys.DELETE)
        time.sleep(0.1) # Increased delay before typing new value
        qty_input.send_keys(str(qty))
        time.sleep(0.05) # Brief delay after typing
        print(f"[ZAP] Quantity updated to {qty}")

    except Exception as e:
        print(f"[WARNING] Quantity setting failed: {e}")
        raise
```

Method: TradovateAccount._setup_atm

```
def _setup_atm(self, tp=None, sl=None)
```

Optimized ATM setup with smart input checking

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns `tp = tp` if `tp` is not `None` else `os.getenv('TRADOVATE_TAKEPROFIT....')`
2. Assigns `sl = sl` if `sl` is not `None` else `os.getenv('TRADOVATE_STOPLOSS_T....')`

3. Calls `print(...)` for its side effect.

4. `try` block with 1 `except` clause.

Code (copy-paste safe)

```
def _setup_atm(self, tp=None, sl=None):
    """Optimized ATM setup with smart input checking"""
    tp = tp if tp is not None else (os.getenv("TRADOVATE_TAKEPROFIT_TICKS") or DEFAULT_TP)
    sl = sl if sl is not None else (os.getenv("TRADOVATE_STOPLOSS_TICKS") or DEFAULT_SL)
    print(f"[ZAP] Setting ATM - TP: {tp}, SL: {sl}")

    try:
        inputs = self.driver.find_elements(By.CSS_SELECTOR, "div.numeric-input input.form-control")
        if len(inputs) < 2:
            print(f"[WARNING] Only found {len(inputs)} ATM inputs, skipping ATM setup")
            return

        tp_input = inputs[0]
        sl_input = inputs[1]

        # SPEED OPTIMIZATION: Check both inputs first, only update if needed
        current_tp = tp_input.get_attribute("value") or ""
        current_sl = sl_input.get_attribute("value") or ""

        tp_needs_update = current_tp.strip() != str(tp).strip()
        sl_needs_update = current_sl.strip() != str(sl).strip()

        if not tp_needs_update and not sl_needs_update:
            print(f"[ZAP] ATM already configured: TP={tp}, SL={sl} - skipping update")
            return

        # Only update TP if needed
        if tp_needs_update:
            tp_input.click()
            time.sleep(0.15) # Increased delay for reliable clearing
            tp_input.send_keys(Keys.CONTROL + "a")
            tp_input.send_keys(Keys.DELETE)
            time.sleep(0.1) # Increased delay before typing
            tp_input.send_keys(str(tp))
            time.sleep(0.15) # Increased delay after typing TP
            # Tab to next field to confirm TP entry
            tp_input.send_keys(Keys.TAB)
            time.sleep(0.2) # Wait for tab to process
            print(f"[ZAP] TP updated to {tp} and tabbed to SL field")
        else:
            print(f"[ZAP] TP {tp} already correct")

        # Only update SL if needed
        if sl_needs_update:
            sl_input.click()
            time.sleep(0.15) # Increased delay for reliable clearing
            sl_input.send_keys(Keys.CONTROL + "a")
            sl_input.send_keys(Keys.DELETE)
            time.sleep(0.1) # Increased delay before typing
            sl_input.send_keys(str(sl))
            time.sleep(0.15) # Increased delay after typing
            # Tab out to confirm SL entry
            sl_input.send_keys(Keys.TAB)
            time.sleep(0.2) # Wait for tab to process
            print(f"[ZAP] SL updated to {sl} and tabbed out")
```

```

else:
    print(f"[ZAP] SL {sl} already correct")

print(f"[ZAP] ATM setup complete: TP={tp}, SL={sl}")

except Exception as e:
    print(f"[WARNING] ATM setup failed: {e}")
    import traceback
    traceback.print_exc()

```

Method: TradovateAccount._detect_account_size

```
def _detect_account_size(self, stats)
```

Heuristic detection of account size based on balance. Returns normalized string like "50k", "100k", or None.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def _detect_account_size(self, stats):
    """
    Heuristic detection of account size based on balance.
    Returns normalized string like "50k", "100k", or None.
    """
    try:
        balance_str = stats.get("Balance", "N/A")
        if not balance_str or balance_str == "N/A":
            return None

        # Clean string: "$50,230.50" -> 50230.50
        clean_bal = balance_str.replace("$", "").replace(",", "").strip()
        bal = float(clean_bal)

        # Define ranges (balance +/- variance) with strict boundaries
        # 25k TIER: 20k-38k
        if 20000 <= bal <= 38000: return "25k"
        # 50k TIER: 40k-65k
        if 40000 <= bal <= 65000: return "50k"
        # 100k TIER: 90k-115k
        if 90000 <= bal <= 115000: return "100k"
        # 150k TIER: 140k-165k
        if 140000 <= bal <= 165000: return "150k"
        # 200k TIER: 190k-215k
        if 190000 <= bal <= 215000: return "200k"
        # 250k TIER: 240k-265k
        if 240000 <= bal <= 265000: return "250k"
        # 300k TIER: 290k-315k
        if 290000 <= bal <= 315000: return "300k"

    return None

```

```
except Exception:
    return None
```

Method: TradovateAccount._place_order_side

```
def _place_order_side(self, symbol, qty, side, tp=None, sl=None, on_click=None, skip_positions_refresh=False,
    prop_firm=None, phase=None, account_size=None, expected_account=None)
```

Place a market order with enhanced crash detection and recovery

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **with self.lock:** enters a context manager.

Code (copy-paste safe)

```
def _place_order_side(self, symbol, qty, side, tp=None, sl=None, on_click=None, skip_positions_refresh=False,
    prop_firm=None, phase=None, account_size=None, expected_account=None):
    """
    Place a market order with enhanced crash detection and recovery
    """
    # Acquire lock to prevent stats fetching during order placement
    with self.lock:
        # --- ACCOUNT NUMBER VERIFICATION ---
        if expected_account:
            expected_account = str(expected_account).strip()
            current_account = self.get_active_account()
            if current_account:
                if expected_account.lower() not in current_account.lower() and current_account.lower()
                    not in expected_account.lower():
                    # Also check numeric suffix match
                    import re as _re
                    expected_digits = _re.search(r'(\d{5,})$', expected_account)
                    suffix_match = expected_digits and current_account.endswith(expected_digits.group(0))
                    if not suffix_match:
                        print(
                            f"[ACCOUNT] Active '{current_account}' ≠ expected '{expected_account}'. S
                        )
                        switched = self.switch_account(expected_account)
                        if switched:
```

```

        # Confirm the switch actually worked
        confirmed = self.get_active_account()
        print(f"[ACCOUNT] ■ Switched – confirmed active: {confirmed}")
    else:
        error_msg = f"[BLOCK] Account switch to '{expected_account}' failed – st
        print(error_msg)
        raise Exception(error_msg)
    else:
        print(
            f"[ACCOUNT] ■ Suffix match: active '{current_account}' matches expected
    else:
        print(f"[ACCOUNT] ■ Verified: '{current_account}' matches '{expected_account}'")
else:
    print(
        f"[ACCOUNT] ■ Could not read active account. Attempting switch to '{expected_acc
    switched = self.switch_account(expected_account)
    if not switched:
        print(f"[ACCOUNT] ■ Switch attempt failed. Proceeding anyway.")
# --- ACCOUNT SIZE PROTECTION ---
# Try to populate account_size from env if missing, for safety
check_size = account_size or os.getenv("ACCOUNT_SIZE", "50k")

if check_size:
    try:
        # Get fresh stats (this is safe because _block_stats_fetching hasn't been called yet)
        current_stats = self.get_account_stats()
        detected_size = self._detect_account_size(current_stats)

        if detected_size:
            # Normalize blueprint size (e.g. "50k" -> "50k", "$50,000" -> "50k")
            bp_size_norm = str(check_size).lower().replace("$", "").replace(",", "")

            # Handle numerical inputs like "50000"
            if bp_size_norm.isdigit():
                val = int(bp_size_norm)
                if 20000 <= val <= 38000: bp_size_norm = "25k"
                elif 45000 <= val <= 55000: bp_size_norm = "50k"
                elif 95000 <= val <= 105000: bp_size_norm = "100k"
                elif 145000 <= val <= 155000: bp_size_norm = "150k"
                elif 195000 <= val <= 205000: bp_size_norm = "200k"
                elif 245000 <= val <= 255000: bp_size_norm = "250k"
                elif 295000 <= val <= 305000: bp_size_norm = "300k"

            # Only compare if we successfully normalized to a "k" string or we are comparing
            # If bp_size_norm is still "60000" (weird size), detected "50k" won't match.

            # Compare
            if detected_size != bp_size_norm:
                error_msg = f"[BLOCK] ACCOUNT SIZE MISMATCH! Blueprint requires {bp_size_norm
                print(error_msg)
                raise Exception(error_msg)
            else:
                print(
                    f"[CHECK] Account size OK: Blueprint {bp_size_norm} matches Account {dete
    else:
        print(
            f"[WARNING] Could not detect account size from balance ({current_stats.get('B
except Exception as size_check_err:
    if "ACCOUNT SIZE MISMATCH" in str(size_check_err):
        raise # Re-raise blocking error

```

```

        print(f"[WARNING] Account size check failed: {size_check_err}")

# CRITICAL DEBUG: Log the symbol BEFORE any fallback logic
print(
    f"[SYMBOL DEBUG] _place_order_side RECEIVED symbol parameter: '{symbol}' (type: {type(symbol)})"
)
print(f"[SYMBOL DEBUG] DEFAULT_SYMBOL = '{DEFAULT_SYMBOL}'")
print(f"[SYMBOL DEBUG] prop_firm={prop_firm}, phase={phase}, account_size={account_size}")

original_symbol = symbol
symbol = symbol or DEFAULT_SYMBOL

if original_symbol != symbol:
    print(
        f"[SYMBOL WARNING] Symbol was None/empty, fell back to DEFAULT_SYMBOL: '{original_symbol}'"
    )
else:
    print(f"[SYMBOL OK] Using provided symbol: '{symbol}'")

print(f"[SEARCH] _place_order_side DEBUG: Placing {side.upper()} order: {symbol} x{qty}")
print(f"[SEARCH] _place_order_side DEBUG: Current logged_in state: {self.logged_in}")

# ■■ API-FIRST: Try placing via REST API (fast, no UI interaction) ■■
try:
    print(f"[API-FIRST] Attempting REST API order: {side.upper()} {qty}x {symbol}")

    # Resolve account_id from expected_account name
    api_account_id = None
    if expected_account:
        accounts = self._api_fetch("/account/list")
        if accounts and isinstance(accounts, list):
            expected_lower = str(expected_account).lower()
            for acct in accounts:
                acct_name = acct.get('name', '').lower()
                if expected_lower in acct_name or acct_name in expected_lower:
                    api_account_id = acct['id']
                    print(
                        f"[API-FIRST] Matched account '{acct.get('name')}' (id={api_account_id})"
                    )
                    break
            # Also try numeric suffix match
            import re as _re
            expected_digits = _re.search(r'(\d{5,})$', str(expected_account))
            if expected_digits and acct_name.endswith(expected_digits.group(1).lower()):
                api_account_id = acct['id']
                print(
                    f"[API-FIRST] Suffix-matched account '{acct.get('name')}' (id={api_account_id})"
                )
                break
        if not api_account_id:
            print(f"[API-FIRST] ■ No account match for '{expected_account}' – using default")

    api_result = self.place_order_api(
        symbol=symbol, side=side, qty=qty,
        order_type="Market", tp=tp, sl=sl,
        account_id=api_account_id,
    )

    if api_result:
        order_id = api_result.get('orderId', api_result.get('id', ''))
        status = api_result.get('ordStatus', '')
        print(
            f"[API-FIRST] ■ API order SUCCESS: id={order_id} status={status} – Selenium skip"
        )
        if on_click:

```

```

        try:
            on_click()
        except Exception:
            pass
        return # API succeeded – done!
    else:
        print(f"[API-FIRST] ■ API returned None – falling back to Selenium")
except Exception as api_err:
    print(f"[API-FIRST] ■ API attempt failed: {api_err} – falling back to Selenium")

# ■■■ SELENIUM FALLBACK: Place via UI clicking ■■■
print(f"[SELENIUM FALLBACK] Proceeding with Selenium-based order placement")

# [ALARM] BLUEPRINT VALIDATION DISABLED: Using input timing instead
print(f"[CHECK] TRADE APPROVED: Blueprint validation disabled - using input timing")

# Add input clearing delays for reliable value entry
import time
time.sleep(0.2) # Brief delay to ensure inputs are ready

# Add browser health check before starting
try:
    current_url = self.driver.current_url
    if "data:" in current_url or not current_url or "tradovate" not in current_url:
        raise Exception("Browser is in invalid state before order placement")
except Exception as health_error:
    print(f"Browser health check failed: {health_error}")
    raise Exception("Browser crashed or disconnected")

# Block other actions - important to prevent UI interference during order placement
self._block_stats_fetching(True)
try:
    # SPEED OPTIMIZATION: Enhanced login check for manual trading
    need_login = False
    login_reason = ""

    # Debug current login state
    login_age = "never" if not self._login_timestamp else f"{int(time.time() - self._login_timestamp)}s"
    print(f"[SEARCH] LOGIN STATE DEBUG: logged_in={self.logged_in}, last_login={login_age}")

    if not self.logged_in:
        # SMART LOGIN CHECK: Verify if interface is actually unavailable before forcing login
        print("[SEARCH] SMART CHECK: logged_in=False but verifying interface availability...")
        try:
            current_url = self.driver.current_url
            if "trader.tradovate.com" in current_url:
                # Check if trading interface is actually accessible
                order_ticket_available = self.driver.find_elements(
                    By.XPATH, "//li[contains(@class, 'lm_tab')]/span[text()='Order Ticket']"
                )
                if order_ticket_available:
                    print(
                        "[SETUP] SMART CHECK SUCCESS: Interface available despite logged_in=False"
                    )
                    self.logged_in = True
                    self._login_timestamp = time.time()
                else:
                    need_login = True
                    login_reason = "logged_in flag is False and Order Ticket not accessible"
            else:
                need_login = True
        except:
            need_login = True

```

```

        login_reason = f"logged_in flag is False and not on trading page (URL: {current_url})"
    except Exception as check_error:
        need_login = True
        login_reason = f"logged_in flag is False and interface check failed: {check_error}"
    else:
        # For manual trades, do a quick interface check instead of full login
        print("[ZAP] FAST MANUAL: Checking trading interface availability...")
        try:
            # Quick check: Can we access the trading interface directly?
            current_url = self.driver.current_url
            if not current_url or "tradovate" not in current_url:
                need_login = True
                login_reason = f"not on Tradovate page (URL: {current_url})"
            else:
                # Check if Order Ticket tab is accessible
                order_ticket_visible = self.driver.find_elements(
                    By.XPATH, "//li[contains(@class, 'lm_tab')]/span[text()='Order Ticket']"
                )
                if order_ticket_visible:
                    print(
                        "[ZAP] FAST MANUAL: Order Ticket tab accessible - proceeding directly"
                    )
                else:
                    print(
                        "[ZAP] FAST MANUAL: Order Ticket not visible - may need interface refresh"
                    )
                    # Try a quick navigation to trading page instead of full login
                    if "trader" not in current_url:
                        print("[ZAP] FAST MANUAL: Navigating to trading interface...")
                        self.driver.get("https://trader.tradovate.com/")
                        time.sleep(2) # Brief wait for page load
        except Exception as interface_error:
            need_login = True
            login_reason = f"interface check failed: {interface_error}"

    # Only perform login if actually needed
    if need_login:
        print(f"[WARNING] Login required: {login_reason}")

        # Proceed directly to full login without page refresh
        print("[REFRESH] Performing login...")
        self.login()
    else:
        print("[ZAP] FAST MANUAL: Login check passed - proceeding with trade execution")

    self._wait_for_no_overlay(timeout=0.2)
    self._select_symbol(symbol)
    self._set_quantity(qty)
    self._setup_atm(tp, sl)
    self._do_place_order(side, on_click=on_click, skip_positions_refresh=skip_positions_refresh)

except Exception as e:
    # Check if it's a browser crash
    try:
        current_url = self.driver.current_url
        if "data:" in current_url or not current_url:
            print(f"Browser crashed during {side} order placement")
            raise Exception(f"Browser crashed during {side} order - cannot continue")
    except:
        print(f"Browser completely unresponsive during {side} order")
        raise Exception(f"Browser crashed during {side} order - cannot continue")
    raise

```



```

finally:
    # Ensure stats fetching is unblocked even if an exception occurs
    self._block_stats_fetching(False)

```

Method: TradovateAccount._block_stats_fetching

```

def _block_stats_fetching(self, block=True)

    Set a flag to block stats fetching during order placement.

```

Step by step (top-level body)

1. Assigns self._placing_order = block.

Code (copy-paste safe)

```

def _block_stats_fetching(self, block=True):
    """Set a flag to block stats fetching during order placement."""
    self._placing_order = block

```

Method: TradovateAccount._select_symbol

```

def _select_symbol(self, symbol)

```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.

Step by step (top-level body)

1. Calls print(...) for its side effect.
2. Calls self._wait_for_no_overlay(...) for its side effect.
3. **try** block with 1 **except** clause.
4. Assigns max_attempts = 5.
5. **for** attempt in range(max_attempts): iterates.
6. Calls self._wait_for_no_overlay(...) for its side effect.

Code (copy-paste safe)

```

def _select_symbol(self, symbol):
    print(f"Selecting symbol: {symbol}")
    self._wait_for_no_overlay()

    # SPEED OPTIMIZATION: Try direct symbol selection first (for fast manual trading)
    try:
        print(f"[ZAP] FAST PATH: Attempting direct symbol selection...")
        # Check if symbol input is already accessible without tab switching
        symbol_input = WebDriverWait(self.driver, 2).until(

```

```

        EC.element_to_be_clickable((By.XPATH,
            "//div[contains(@class, 'trading-ticket-main-entry')]//input[contains(@class, 'search')]
        ))

    # Check if symbol is already correctly set
    current_symbol = symbol_input.get_attribute("value") or ""
    if current_symbol.strip().upper() == symbol.upper():
        print(f"[ZAP] FAST PATH SUCCESS: Symbol {symbol} already selected, skipping")
        return

    # Symbol input is accessible, proceed with direct selection
    symbol_input.click()
    time.sleep(0.1) # Reduced delay for speed
    symbol_input.send_keys(Keys.CONTROL + "a")
    symbol_input.send_keys(Keys.DELETE)
    time.sleep(0.1)

    # Type symbol with moderate delays to ensure dropdown appears
    for char in symbol:
        symbol_input.send_keys(char)
        time.sleep(0.1) # Moderate typing speed for dropdown stability

    # Wait for dropdown and click option directly - EXACT MATCH to avoid selecting MNQM6 when loc
    # Try exact match first (most reliable)
    try:
        dropdown_option = WebDriverWait(self.driver, 3).until(
            EC.visibility_of_element_located((
                By.XPATH,
                f"//ul[contains(@class, 'dropdown-menu')]//a[starts-with(normalize-space(.), '{symbol}')]
            ))
        )
        self.driver.execute_script("arguments[0].click();", dropdown_option)
        print(f"[ZAP] FAST PATH SUCCESS: Symbol {symbol} selected directly (exact match)")
        self._wait_for_no_overlay()
        return
    except:
        # Fallback: If exact match fails, try contains but click the first matching option
        print(f"[ZAP] Exact match failed, trying contains match...")
        dropdown_option = WebDriverWait(self.driver, 2).until(
            EC.visibility_of_element_located((
                By.XPATH,
                f"//ul[contains(@class, 'dropdown-menu')]//a[contains(text(), '{symbol}')] or .//a[contains(text(), '{symbol}')]
            ))
        )
        self.driver.execute_script("arguments[0].click();", dropdown_option)
        print(f"[ZAP] FAST PATH SUCCESS: Symbol {symbol} selected directly (contains match)")
        self._wait_for_no_overlay()
        return

    except Exception as fast_error:
        print(f"[ZAP] Fast path failed: {fast_error}, falling back to tab navigation...")

    # ORIGINAL PATH: Full tab navigation for when fast path fails
    max_attempts = 5
    for attempt in range(max_attempts):
        try:
            # Step 1: Click Positions tab
            positions_tab = WebDriverWait(self.driver, 10).until(
                EC.element_to_be_clickable((By.XPATH,
                    "//li[contains(@class, 'lm_tab')]//span[text()='Positions']"))
            )

```

```

    )
    positions_tab.click()
    time.sleep(0.3)
except Exception as e:
    print(f"Warning: Positions tab interaction failed: {e}")

try:
    # Step 2: Click Order Ticket tab
    order_ticket_tab = WebDriverWait(self.driver, 10).until(
        EC.element_to_be_clickable((By.XPATH,
            "//li[contains(@class, 'lm_tab')]//span[text()='Order Ticket']"))
    )
    order_ticket_tab.click()
    time.sleep(0.3)
except Exception as e:
    print(f"Error: Order Ticket tab not accessible: {e}")
    raise

try:
    # Step 3: Find and interact with symbol input
    symbol_input = WebDriverWait(self.driver, 10).until(
        EC.element_to_be_clickable((By.XPATH,
            "//div[contains(@class, 'trading-ticket-main-entry')]//input[contains(@class, 'se')]"))
    )
    symbol_input.click()
    time.sleep(0.3)
    symbol_input.send_keys(Keys.CONTROL + "a")
    symbol_input.send_keys(Keys.DELETE)
    time.sleep(0.3)

    # Type symbol character by character with slower speed for dropdown stability
    for char in symbol:
        symbol_input.send_keys(char)
        time.sleep(0.15) # Slower typing to ensure dropdown appears

    # Step 4: Wait for and click dropdown option - EXACT MATCH to avoid selecting MNQM6 when
    try:
        # Try exact match first (more reliable)
        dropdown_option = WebDriverWait(self.driver, 5).until(
            EC.visibility_of_element_located((
                By.XPATH,
                f"//ul[contains(@class, 'dropdown-menu')]//a[starts-with(normalize-space(.), '{symbol}')]"))
        )
        self.driver.execute_script("arguments[0].click();", dropdown_option)
        print(f"Symbol {symbol} selected from dropdown (exact match)")
        break # Success!
    except:
        # Fallback: try contains match
        print(f"Exact match failed, trying contains match for {symbol}...")
        dropdown_option = WebDriverWait(self.driver, 5).until(
            EC.visibility_of_element_located((
                By.XPATH,
                f"//ul[contains(@class, 'dropdown-menu')]//a[contains(text(), '{symbol}')]"))
        )
        self.driver.execute_script("arguments[0].click();", dropdown_option)
        print(f"Symbol {symbol} selected from dropdown (contains match)")
        break # Success!

except Exception as e:

```

```

        print(f"Attempt {attempt+1} to select symbol failed: {e}")
        if attempt == max_attempts - 1:
            raise Exception(
                "Symbol input not found or could not select symbol after several attempts")
        time.sleep(1)

    self._wait_for_no_overlay()

```

Method: TradovateAccount._recover_from_symbol_failure

```
def _recover_from_symbol_failure(self)
```

Recovery method: Click Positions tab, then Order Ticket tab, then clear symbol field

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def _recover_from_symbol_failure(self):
    """Recovery method: Click Positions tab, then Order Ticket tab, then clear symbol field"""
    try:
        print("[REFRESH] Starting symbol selection recovery...")

        # Step 1: Click Positions tab
        try:
            positions_tab = WebDriverWait(self.driver, 3).until(
                EC.element_to_be_clickable((By.XPATH,
                    "//li[contains(@class, 'lm_tab')]/span[text()='Positions']"))
            )
            positions_tab.click()
            print("[CHECK] Clicked Positions tab")
            time.sleep(0.5)
        except Exception as e:
            print(f"[WARNING] Could not click Positions tab: {e}")

        # Step 2: Click Order Ticket tab
        try:
            order_ticket_tab = WebDriverWait(self.driver, 3).until(
                EC.element_to_be_clickable((By.XPATH,
                    "//li[contains(@class, 'lm_tab')]/span[text()='Order Ticket']"))
            )
            order_ticket_tab.click()
            print("[CHECK] Clicked Order Ticket tab")
            time.sleep(0.5)
        except Exception as e:
            print(f"[WARNING] Could not click Order Ticket tab: {e}")

        # Step 3: Clear symbol field
        try:

```

```

symbol_input = WebDriverWait(self.driver, 3).until(
    EC.visibility_of_element_located((By.XPATH,
        "//div[contains(@class, 'trading-ticket-main-entry')]//input[contains(@class, 'se
    )
symbol_input.click()
time.sleep(0.2)
symbol_input.send_keys(Keys.CONTROL + "a")
symbol_input.send_keys(Keys.DELETE)
print("[CHECK] Cleared symbol field")
time.sleep(0.3)
except Exception as e:
    print(f"[WARNING] Could not clear symbol field: {e}")

print("[REFRESH] Recovery completed")

except Exception as recovery_error:
    print(f"[X] Recovery failed: {recovery_error}")

```

Method: TradovateAccount._do_place_order

```
def _do_place_order(self, side, on_click=None, skip_positions_refresh=False)
```

Optimized order placement with better error handling

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.

Step by step (top-level body)

1. Assigns `self._placing_order = True`.
2. **try** block with 1 **except** clause, plus a **finally**.

Code (copy-paste safe)

```

def _do_place_order(self, side, on_click=None, skip_positions_refresh=False):
    """Optimized order placement with better error handling"""
    self._placing_order = True

    try:
        # Find buy/sell button with better error handling
        try:
            side_button = WebDriverWait(self.driver, 5).until(
                EC.element_to_be_clickable((By.XPATH,
                    f"//label[contains(translate(text(), 'ABCDEFGHIJKLMNOPQRSTUVWXYZ', 'abcdefghijklmnopqrstuvwxyz'),
                )
            current_class = side_button.get_attribute("class") or ""
            if "active" not in current_class:
                self.driver.execute_script("arguments[0].click();", side_button)
                print(f"{side.upper()} button selected")
            else:

```

```

        print(f"{side.upper()} button already active")
except Exception as e:
    print(f"Failed to select {side} button: {e}")
    raise

# Click Send button with retry logic
try:
    send_button = WebDriverWait(self.driver, 5).until(
        EC.element_to_be_clickable((By.XPATH,
            "//button[contains(@class, 'btn-primary') and text()='Send']"))
    )
    self.driver.execute_script("arguments[0].click();", send_button)
    print("Send button clicked")
except Exception as e:
    print(f"Failed to click Send button: {e}")
    raise

# Brief wait for any confirmation dialog to render (appears intermittently)
time.sleep(0.5)

# Detect confirmation dialog by container classes OR "DO NOT SHOW" marker text
confirm_containers = self.driver.find_elements(By.XPATH,
    "//div[contains(@class, 'popover') or contains(@class, 'modal') or contains(@class, 'confirm"
    " or contains(@class, 'dialog') or contains(@class, 'overlay')]]")
)
do_not_show_markers = self.driver.find_elements(By.XPATH,
    "//*[contains(translate(text(), 'ABCDEFGHIJKLMNOPQRSTUVWXYZ', 'abcdefghijklmnopqrstuvwxyz'), 'do not show')]")
)
dialog_detected = bool(confirm_containers or do_not_show_markers)

if dialog_detected:
    print("Confirmation dialog detected")

    # Check "Do not show again" checkbox if present
    try:
        do_not_show = self.driver.find_element(By.XPATH,
            "//input[@type='checkbox' and (following-sibling::*[contains(translate(text(), "
            "'ABCDEFGHIJKLMNOPQRSTUVWXYZ', 'abcdefghijklmnopqrstuvwxyz'), 'do not show']]"
            " or ancestor::label[contains(translate(text(), "
            "'ABCDEFGHIJKLMNOPQRSTUVWXYZ', 'abcdefghijklmnopqrstuvwxyz'), 'do not show'])]]"
        )
        if not do_not_show.is_selected():
            self.driver.execute_script("arguments[0].click();", do_not_show)
            print("'Do not show again' checkbox checked")
    except Exception:
        # Try broader search for the checkbox near "do not show" text
        try:
            do_not_show = self.driver.find_element(By.XPATH,
                "//*[contains(translate(text(), 'ABCDEFGHIJKLMNOPQRSTUVWXYZ', 'abcdefghijklmnopqrstuvwxyz'), 'do not show')]/preceding-sibling::input[@type='checkbox']"
                " | //*[contains(translate(text(), 'ABCDEFGHIJKLMNOPQRSTUVWXYZ', 'abcdefghijklmnopqrstuvwxyz'), 'do not show')]/following-sibling::input[@type='checkbox']"
                " | //*[contains(translate(text(), 'ABCDEFGHIJKLMNOPQRSTUVWXYZ', 'abcdefghijklmnopqrstuvwxyz'), 'do not show')]/ancestor::label//input[@type='checkbox']"
                " | //*[contains(translate(text(), 'ABCDEFGHIJKLMNOPQRSTUVWXYZ', 'abcdefghijklmnopqrstuvwxyz'), 'do not show')]/parent::*//input[@type='checkbox']"
            )
            if not do_not_show.is_selected():
                self.driver.execute_script("arguments[0].click();", do_not_show)
                print("'Do not show again' checkbox checked (alt)")

```

```

        except Exception:
            print("No 'Do not show again' checkbox found")

# Click the confirmation Buy/Sell button (matches current trade side)
confirmed = False

# Try side-specific button inside known container first
try:
    side_confirm = self.driver.find_element(By.XPATH,
        f"//div[contains(@class,'popover') or contains(@class,'modal') or contains(@class,"
        f" or contains(@class,'dialog') or contains(@class,'overlay'))]"
        f"//div[contains(@class,'btn') and contains(translate(text(),'ABCDEFGHIJKLMNOPQRSTUVWXYZ',"
        f"'ABCDEFGHIJKLMNOPQRSTUVWXYZ','abcdefghijklmnopqrstuvwxyz'),'{side.lower()}')]"
        f" | //div[contains(@class,'popover') or contains(@class,'modal') or contains(@class,"
        f" or contains(@class,'dialog') or contains(@class,'overlay'))]"
        f"//button[contains(translate(text(),'ABCDEFGHIJKLMNOPQRSTUVWXYZ',"
        f"'ABCDEFGHIJKLMNOPQRSTUVWXYZ','abcdefghijklmnopqrstuvwxyz'),'{side.lower()}')]"
    )
    self.driver.execute_script("arguments[0].click();", side_confirm)
    print(f"Confirmation {side.upper()} button clicked (in container)")
    confirmed = True
except Exception:
    pass

# Fallback: side-specific button near "Cancel" (the order ticket has no Cancel)
if not confirmed:
    try:
        side_confirm = self.driver.find_element(By.XPATH,
            f"//button[contains(translate(text(),'ABCDEFGHIJKLMNOPQRSTUVWXYZ','abcdefghijklmnopqrstuvwxyz',"
            f" and following-sibling::button[contains(translate(text(),'ABCDEFGHIJKLMNOPQRSTUVWXYZ',"
            f" | //button[contains(translate(text(),'ABCDEFGHIJKLMNOPQRSTUVWXYZ','abcdefghijklmnopqrstuvwxyz',"
            f" and preceding-sibling::button[contains(translate(text(),'ABCDEFGHIJKLMNOPQRSTUVWXYZ',"
            f" | //div[contains(@class,'btn') and contains(translate(text(),'ABCDEFGHIJKLMNOPQRSTUVWXYZ',"
            f" and (following-sibling::*[contains(translate(text(),'ABCDEFGHIJKLMNOPQRSTUVWXYZ',"
            f" or preceding-sibling::*[contains(translate(text(),'ABCDEFGHIJKLMNOPQRSTUVWXYZ',"
        )
        self.driver.execute_script("arguments[0].click();", side_confirm)
        print(f"Confirmation {side.upper()} button clicked (near Cancel)")
        confirmed = True
    except Exception:
        pass

# Fallback: OK / Confirm / Continue / Yes buttons
if not confirmed:
    try:
        fallback_btn = self.driver.find_element(By.XPATH,
            f"//button[contains(translate(text(),'ABCDEFGHIJKLMNOPQRSTUVWXYZ','abcdefghijklmnopqrstuvwxyz',"
            f" or contains(translate(text(),'ABCDEFGHIJKLMNOPQRSTUVWXYZ','abcdefghijklmnopqrstuvwxyz',"
            f" or contains(translate(text(),'ABCDEFGHIJKLMNOPQRSTUVWXYZ','abcdefghijklmnopqrstuvwxyz',"
            f" or contains(translate(text(),'ABCDEFGHIJKLMNOPQRSTUVWXYZ','abcdefghijklmnopqrstuvwxyz',"
            f" | //div[contains(@class,'btn-success') or contains(@class,'btn-primary'))]"
        )
        self.driver.execute_script("arguments[0].click();", fallback_btn)
        print("Confirmation dialog handled (fallback)")
        confirmed = True
    except Exception:
        pass

if not confirmed:
    print("Confirmation dialog found but no button matched")

```

```

# Dismiss any post-order notification/toast that may block the UI
try:
    dismiss_btn = WebDriverWait(self.driver, 1).until(
        EC.element_to_be_clickable((By.XPATH,
            "//button[contains(@class, 'close') or @aria-label='Close' or @aria-label='Dismiss']"
            " | //div[contains(@class, 'toast')]/button"
            " | //div[contains(@class, 'notification')]/button[contains(@class, 'close')]"
        )))
    self.driver.execute_script("arguments[0].click();", dismiss_btn)
    print("Post-order notification dismissed")
except Exception:
    pass

print(f"Order placed: {side.upper()}")

# Click positions tab after order placement to refresh data (unless skipped for manual trading)
if not skip_positions_refresh:
    try:
        time.sleep(0.5)
        positions_tab = WebDriverWait(self.driver, 3).until(
            EC.element_to_be_clickable((By.XPATH,
                "//li[contains(@class, 'lm_tab')]/span[text()='Positions']"
            )))
        positions_tab.click()
        print("Positions tab clicked after order placement")
        time.sleep(0.3)
    except Exception as e:
        print(f"Failed to click Positions tab after order: {e}")
    else:
        print("Skipping positions tab refresh for manual trading speed")

except Exception as e:
    print(f"Order placement failed: {e}")
    raise
finally:
    self._placing_order = False

```

Method: TradovateAccount._api_fetch

```
def _api_fetch(self, endpoint, method='GET', body=None)
```

Execute a fetch() call in the browser against the Tradovate REST API. Returns parsed JSON data or None on failure.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns `body_js = f'opts.body = JSON.stringify({json.dumps(body)});' if bod....`
2. Assigns `js = f"\n var cb = arguments[arguments.length - 1];\n ....`
3. `try` block with 1 **except** clause.

Code (copy-paste safe)

```
def _api_fetch(self, endpoint, method="GET", body=None):
    """Execute a fetch() call in the browser against the Tradovate REST API.
    Returns parsed JSON data or None on failure."""
    body_js = f'opts.body = JSON.stringify({json.dumps(body)});' if body else ""
    js = f"""
    var cb = arguments[arguments.length - 1];
    (async function() {{
        try {{
            var auth = JSON.parse(sessionStorage.getItem('api_authenticator_state') || '{}');
            var token = auth.token || '';
            var env = auth.environment || 'demo';
            var base = 'https://' + env + '.tradovateapi.com/v1';
            var opts = {{
                method: '{method}',
                headers: {{
                    'Authorization': 'Bearer ' + token,
                    'Content-Type': 'application/json',
                    'Accept': 'application/json'
                }}
            }};
            {{body_js}}
            var r = await fetch(base + '{endpoint}', opts);
            var txt = await r.text();
            var d = null;
            try {{ d = JSON.parse(txt); }} catch(e) {{}}
            cb({{ok: r.ok, status: r.status, data: d}});
        }} catch(e) {{
            cb({{ok: false, error: e.toString()}});
        }}
    }})();
    """
    try:
        result = self.driver.execute_async_script(js)
        if result and result.get('ok'):
            return result.get('data')
        # Log the actual error instead of silently returning None
        status = result.get('status', '?') if result else '?'
        err = result.get('error', '') if result else 'no result'
        data = result.get('data', '') if result else ''
        print(f"[API] fetch {endpoint} FAILED: status={status} error={err} data={data}")
        return None
    except Exception as e:
        print(f"[API] fetch {endpoint} exception: {e}")
        return None
```

Method: `TradovateAccount._get_account_for_api`

```
def _get_account_for_api(self)
```

Get the first account id and name for API order placement.

Why these Python concepts were used

- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.

Step by step (top-level body)

1. Assigns `accounts = self._api_fetch('/account/list')`.
2. **if** not `accounts` or not `isinstance(accounts, list)` or `len(accounts) == 0`: branches conditionally.
3. Assigns `acct = accounts[0]`.
4. **return** (`acct['id'], acct.get('name', '?')`).

Code (copy-paste safe)

```
def _get_account_for_api(self):
    """Get the first account id and name for API order placement."""
    accounts = self._api_fetch("/account/list")
    if not accounts or not isinstance(accounts, list) or len(accounts) == 0:
        return None, None
    acct = accounts[0]
    return acct['id'], acct.get('name', '?')
```

Method: TradovateAccount.get_min_equity

```
def get_min_equity(self, account_id=None)
```

Get the minimum equity (drawdown limit) for an account via the API. Uses /cashBalance/getCashBalanceSnapshot for current net liq, /userAccountAutoLiq/deps for trailing drawdown limits, and /accountRiskStatus/list for the max net liq (trailing reference). Returns: dict with 'net_liq', 'min_equity', 'drawdown_remaining' or None on failure.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def get_min_equity(self, account_id=None):
    """Get the minimum equity (drawdown limit) for an account via the API.

    Uses /cashBalance/getCashBalanceSnapshot for current net liq,
    /userAccountAutoLiq/deps for trailing drawdown limits, and
    /accountRiskStatus/list for the max net liq (trailing reference).
```

```

Returns:
    dict with 'net_liq', 'min_equity', 'drawdown_remaining' or None on failure.
"""
try:
    if not account_id:
        account_id, _ = self._get_account_for_api()
    if not account_id:
        return None

    snapshot = self._api_fetch("/cashBalance/getCashBalanceSnapshot", "POST", {
        "accountId": account_id}) or {}
    net_liq = snapshot.get('netLiq', 0)
    net_liq_sod = snapshot.get('netLiqSOD', 0) # balance at midnight

    # Use deps endpoint – returns the autoLiq entry directly for this account
    al_raw = self._api_fetch(f"/userAccountAutoLiq/deps?masterid={account_id}") or {}
    # deps can return a single object or a list with one entry
    if isinstance(al_raw, list):
        al = al_raw[0] if al_raw else {}
    else:
        al = al_raw

    trailing_max_drawdown = al.get('trailingMaxDrawdown', 0) or 0
    trailing_max_drawdown_limit = al.get('trailingMaxDrawdownLimit', 0) or 0
    trailing_mode = al.get('trailingMaxDrawdownMode', '')

    print(f"[MIN EQUITY] autoLiq raw for {account_id}: TMD={trailing_max_drawdown}, "
          f"TMDL={trailing_max_drawdown_limit}, mode={trailing_mode}, keys={list(al.keys())}")

    # Get max net liq from accountRiskStatus (the trailing reference point)
    max_net_liq = 0
    risk_raw = self._api_fetch(f"/accountRiskStatus/deps?masterid={account_id}") or {}
    if isinstance(risk_raw, list):
        rs = risk_raw[0] if risk_raw else {}
    else:
        rs = risk_raw
    max_net_liq = rs.get('maxNetLiq', 0) or 0

    # EOD trailing drawdown:
    # SL floor from SOD: midnight_balance - trailing_max_drawdown
    # Absolute floor:    trailingMaxDrawdownLimit - trailing_max_drawdown
    # min_equity = max(sod_floor, absolute_floor)
    if trailing_max_drawdown > 0:
        absolute_floor = (
            trailing_max_drawdown_limit - trailing_max_drawdown) if trailing_max_drawdown_limit > 0 else 0
        if net_liq_sod > 0:
            sod_floor = net_liq_sod - trailing_max_drawdown
            min_equity = max(sod_floor, absolute_floor)
        else:
            min_equity = absolute_floor
    else:
        min_equity = 0

    drawdown_remaining = net_liq - min_equity if min_equity else net_liq

    result = {
        'net_liq': net_liq,
        'net_liq_sod': net_liq_sod,
        'min_equity': min_equity,
        'trailing_max_drawdown': trailing_max_drawdown,

```

```

        'trailing_max_drawdown_limit': trailing_max_drawdown_limit,
        'trailing_mode': trailing_mode,
        'max_net_liq': max_net_liq,
        'drawdown_remaining': drawdown_remaining,
    }
    print(f"[MIN EQUITY] account={account_id}: netLiq=${net_liq:,.2f}, "
          f"SOD=${net_liq_sod:,.2f}, minEquity=${min_equity:,.2f}, "
          f"remaining=${drawdown_remaining:,.2f}, mode={trailing_mode}")
    return result
except Exception as e:
    import traceback
    print(f"[MIN EQUITY] Failed to get min equity: {e}")
    traceback.print_exc()
    return None

```

Method: TradovateAccount.place_order_api

```

def place_order_api(self, symbol, side, qty, order_type='Market', price=None, stop_price=None, tp=None,
                    sl=None, account_id=None)

```

*Place an order via the Tradovate REST API (no Selenium UI clicking). Args: symbol: Contract symbol e.g. "NQM6", "ESM6" side: "Buy" or "Sell" qty: Number of contracts (int) order_type: "Market", "Limit", "Stop", "StopLimit", "MIT" price: Limit price (required for Limit/StopLimit) stop_price: Stop price (required for Stop/StopLimit) tp: Take profit — in ticks (int) or absolute price (float > tick_size * 200) sl: Stop loss — in ticks (int) or absolute price (float > tick_size * 200) account_id: Specific account ID (default: first account) Returns: dict with order result on success, None on failure Tick reference: NQ/MNQ = 0.25/tick (\$5.00/tick NQ, \$0.50/tick MNQ) ES/MES = 0.25/tick (\$12.50/tick ES, \$1.25/tick MES) YM/MYM = 1.00/tick (\$5.00/tick YM, \$0.50/tick MYM) CL/MCL = 0.01/tick (\$10.00/tick CL, \$1.00/tick MCL)*

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns side = side.capitalize().
2. if side not in ('Buy', 'Sell'): branches conditionally.
3. if not account_id: branches conditionally (with an **else/elif** arm).
4. Assigns contract = self._api_fetch(f'/contract/find?name={symbol}').

5. if not contract or not contract.get('id'): branches conditionally.
6. Assigns contract_id = contract['id'].
7. Assigns contract_name = contract.get('name', symbol).
8. Assigns tick_size = None.
9. if tp is not None or sl is not None: branches conditionally.
10. Calls print(...) for its side effect.
11. Assigns payload = {'accountSpec': account_name, 'accountId': account_id, 'a....
12. if price is not None and order_type in ('Limit', 'StopLimit'): branches conditionally.
13. if stop_price is not None and order_type in ('Stop', 'StopLimit', 'MIT'): branches conditionally.
14. Assigns self._placing_order = True.
15. ... and 1 more statement(s) in the body.

Code (copy-paste safe)

```
def place_order_api(self, symbol, side, qty, order_type="Market", price=None,
                    stop_price=None, tp=None, sl=None, account_id=None):
    """Place an order via the Tradovate REST API (no Selenium UI clicking).

    Args:
        symbol:      Contract symbol e.g. "NQM6", "ESM6"
        side:        "Buy" or "Sell"
        qty:         Number of contracts (int)
        order_type:  "Market", "Limit", "Stop", "StopLimit", "MIT"
        price:       Limit price (required for Limit/StopLimit)
        stop_price:  Stop price (required for Stop/StopLimit)
        tp:          Take profit – in ticks (int) or absolute price (float > tick_size * 200)
        sl:          Stop loss – in ticks (int) or absolute price (float > tick_size * 200)
        account_id:  Specific account ID (default: first account)

    Returns:
        dict with order result on success, None on failure

    Tick reference:
        NQ/MNQ = 0.25/tick ($5.00/tick NQ, $0.50/tick MNQ)
        ES/MES = 0.25/tick ($12.50/tick ES, $1.25/tick MES)
        YM/MYM = 1.00/tick ($5.00/tick YM, $0.50/tick MYM)
        CL/MCL = 0.01/tick ($10.00/tick CL, $1.00/tick MCL)
    """
    # Normalize side
    side = side.capitalize()
    if side not in ("Buy", "Sell"):
        print(f"[API ORDER] Invalid side: {side}")
        return None

    # Get account
    if not account_id:
        account_id, account_name = self._get_account_for_api()
        if not account_id:
            print("[API ORDER] No account found")
            return None
    else:
        # Look up account name – accountSpec needs the name string, not numeric ID
        account_name = str(account_id)
```

```

accounts = self._api_fetch("/account/list")
if accounts and isinstance(accounts, list):
    for acct in accounts:
        if acct.get('id') == account_id:
            account_name = acct.get('name', str(account_id))
            break

# Find contract by symbol name
contract = self._api_fetch(f"/contract/find?name={symbol}")
if not contract or not contract.get('id'):
    print(f"[API ORDER] Contract not found: {symbol}")
    return None
contract_id = contract['id']
contract_name = contract.get('name', symbol)

# Get tick size from contract maturity (for tick-based TP/SL)
tick_size = None
if tp is not None or sl is not None:
    mat_id = contract.get('contractMaturityId')
    if mat_id:
        mat = self._api_fetch(f"/contractMaturity/item?id={mat_id}")
        if mat:
            prod_id = mat.get('productId')
            if prod_id:
                product = self._api_fetch(f"/product/item?id={prod_id}")
                if product:
                    tick_size = product.get('tickSize', 0.25)
    if not tick_size:
        # Fallback: common tick sizes by symbol prefix
        sym_upper = symbol.upper()
        if any(sym_upper.startswith(p) for p in ("NQ", "MNQ", "ES", "MES")):
            tick_size = 0.25
        elif any(sym_upper.startswith(p) for p in ("YM", "MYM")):
            tick_size = 1.0
        elif any(sym_upper.startswith(p) for p in ("CL", "MCL")):
            tick_size = 0.01
        else:
            tick_size = 0.25
    print(f"[API ORDER] Using fallback tick size: {tick_size}")

# ■■ ref_price is determined AFTER the market fill (see below) ■■
# Pre-order quotes from /md/getQuote are unreliable on prop firm accounts
# and have caused catastrophic bracket pricing (e.g. 7089 instead of 26474).
# We now ALWAYS place the market order first, then use the fill price.

print(f"[API ORDER] {side} {qty}x {contract_name} (id={contract_id}) on {account_name} – {order_t}
    + (f" tick={tick_size}" if tick_size else "")

# Build order payload
payload = {
    "accountSpec": account_name,
    "accountId": account_id,
    "action": side,
    "symbol": contract_name,
    "orderQty": qty,
    "orderType": order_type,
    "isAutomated": False,
}
if price is not None and order_type in ("Limit", "StopLimit"):
    payload["price"] = price

```

```

if stop_price is not None and order_type in ("Stop", "StopLimit", "MIT"):
    payload["stopPrice"] = stop_price

# Place market order FIRST, then add brackets using fill price
self._placing_order = True
try:
    print(f"[API ORDER] Placing market order via /order/placeOrder")
    result = self._api_fetch("/order/placeOrder", "POST", payload)

    if not result:
        print(f"[API ORDER] ■ Order rejected or failed")
        return None

    # Parse result – may be nested or flat
    if isinstance(result, dict):
        order_id = result.get('orderId') or result.get('id') or '?'
        status = result.get('ordStatus', '?')
        fill_price = result.get('avgPx') or result.get('price')
        print(f"[API ORDER] ■ Order placed: id={order_id} status={status} fillPrice={fill_price}")
        print(f"[API ORDER] Full result: {result}")
    else:
        print(f"[API ORDER] ■ Order placed (non-dict response): {result}")
        order_id = '?'
        fill_price = None

    # Now add TP/SL brackets using the actual fill price
    if tp is not None or sl is not None:
        import time as _time

        # Retry loop – market orders may take a moment to fill
        if not fill_price and order_id != '?':
            for attempt in range(8):
                _time.sleep(0.5)
                try:
                    order_detail = self._api_fetch(f"/order/item?id={order_id}")
                    if order_detail and isinstance(order_detail, dict):
                        fill_price = order_detail.get('avgPx') or order_detail.get('price')
                        detail_status = order_detail.get('ordStatus', '?')
                        print(
                            f"[API ORDER] Fill check #{attempt+1}: status={detail_status} avgPx={fill_price}"
                        )
                        if fill_price:
                            break
                except Exception as ex:
                    print(f"[API ORDER] Fill check #{attempt+1} error: {ex}")

        # Fallback: check position for price
        if not fill_price:
            try:
                positions = self._api_fetch("/position/list")
                if positions and isinstance(positions, list):
                    for pos in positions:
                        if pos.get('contractId') == contract_id and pos.get(
                            'accountId') == account_id:
                            fill_price = pos.get('price') or pos.get('netPrice')
                            print(f"[API ORDER] Got price from position: {fill_price}")
                            break
            except Exception as ex:
                print(f"[API ORDER] Position lookup error: {ex}")

        if fill_price:

```

```

        # Sanity check: NQ-like symbols should have prices > 1000
        sym_upper = symbol.upper()
        if any(sym_upper.startswith(p) for p in ("NQ", "MNQ")) and fill_price < 1000:
            print(
                f"[API ORDER] ■ SANITY FAIL: fill_price={fill_price} is too low for {symbol}"
            )
        else:
            print(f"[API ORDER] Placing brackets at fill_price={fill_price}")
            self._place_brackets_post_fill(
                account_name, account_id, contract_name, qty,
                side, fill_price, tp, sl, tick_size
            )
        else:
            print(f"[API ORDER] ■ Could not determine fill price – brackets NOT placed!")

    return result

except Exception as e:
    import traceback
    print(f"[API ORDER] ■ Exception: {e}")
    traceback.print_exc()
    return None
finally:
    self._placing_order = False

```

Method: TradovateAccount._place_brackets_post_fill

```

def _place_brackets_post_fill(self, account_name, account_id, contract_name, qty, side, fill_price, tp, sl, tick_size):

```

Place TP and SL bracket orders after a market order has filled. For BUY main order: TP = Sell Limit (above fill), SL = Sell Stop (below fill) For SELL main order: TP = Buy Limit (below fill), SL = Buy Stop (above fill)

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns `tp_price = self._resolve_bracket_price(tp, tick_size, fill_price, si....`
2. Assigns `sl_price = self._resolve_bracket_price(sl, tick_size, fill_price, si....`
3. **if** `not tp_price` and `(not sl_price)`: branches conditionally.
4. **if** `side == 'Buy'`: branches conditionally (with an **else/elif** arm).
5. Calls `print(...)` for its side effect.
6. **if** `tp_price`: branches conditionally.
7. **if** `sl_price`: branches conditionally.

Code (copy-paste safe)

```
def _place_brackets_post_fill(self, account_name, account_id, contract_name,
                              qty, side, fill_price, tp, sl, tick_size):
    """Place TP and SL bracket orders after a market order has filled.

    For BUY main order: TP = Sell Limit (above fill), SL = Sell Stop (below fill)
    For SELL main order: TP = Buy Limit (below fill), SL = Buy Stop (above fill)
    """
    tp_price = self._resolve_bracket_price(tp, tick_size, fill_price, side, is_tp=True) if tp else None
    sl_price = self._resolve_bracket_price(sl, tick_size, fill_price, side, is_tp=False) if sl else None

    if not tp_price and not sl_price:
        print(f"[BRACKET] No valid bracket prices computed – skipping")
        return

    # Determine bracket order sides and types
    if side == "Buy":
        # Close a long: Sell Limit for TP, Sell Stop for SL
        tp_action, tp_type = "Sell", "Limit"
        sl_action, sl_type = "Sell", "Stop"
    else:
        # Close a short: Buy Limit for TP, Buy Stop for SL
        tp_action, tp_type = "Buy", "Limit"
        sl_action, sl_type = "Buy", "Stop"

    print(f"[BRACKET] Main={side} fill={fill_price} | "
          f"TP={tp_action} {tp_type} @ {tp_price} | SL={sl_action} {sl_type} @ {sl_price}")

    # Place TP order
    if tp_price:
        tp_payload = {
            "accountSpec": account_name,
            "accountId": account_id,
            "action": tp_action,
            "symbol": contract_name,
            "orderQty": qty,
            "orderType": tp_type,
            "price": tp_price,
            "isAutomated": False,
        }
        print(f"[BRACKET] Placing TP: {tp_action} {qty}x {contract_name} {tp_type} @ {tp_price}")
        try:
            tp_result = self._api_fetch("/order/placeOrder", "POST", tp_payload)
            if tp_result:
                print(f"[BRACKET] ■ TP placed: {tp_result}")
            else:
                print(f"[BRACKET] ■ TP failed – no response")
        except Exception as e:
            print(f"[BRACKET] ■ TP exception: {e}")

    # Place SL order
    if sl_price:
        sl_payload = {
            "accountSpec": account_name,
            "accountId": account_id,
            "action": sl_action,
            "symbol": contract_name,
            "orderQty": qty,
            "orderType": sl_type,
```

```

        "stopPrice": sl_price,
        "isAutomated": False,
    }
    print(f"[BRACKET] Placing SL: {sl_action} {qty}x {contract_name} {sl_type} @ {sl_price}")
    try:
        sl_result = self._api_fetch("/order/placeOrder", "POST", sl_payload)
        if sl_result:
            print(f"[BRACKET] ■ SL placed: {sl_result}")
        else:
            print(f"[BRACKET] ■ SL failed – no response")
    except Exception as e:
        print(f"[BRACKET] ■ SL exception: {e}")

```

Method: TradovateAccount._resolve_bracket_price

```
def _resolve_bracket_price(self, value, tick_size, ref_price, side, is_tp)
```

Convert a TP/SL tick count to an absolute price. Values are Tradovate tick counts (e.g. 151 ticks for NQ). Price offset = ticks × tick_size (151 × 0.25 = 37.75 pts). For Buy orders: TP is above ref_price, SL is below For Sell orders: TP is below ref_price, SL is above

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. if value is None or tick_size is None: branches conditionally.
2. if ref_price is None: branches conditionally.
3. Assigns offset = value * tick_size.
4. if side == 'Buy': branches conditionally (with an **else/elif** arm).
5. Calls print(...) for its side effect.
6. **return** result.

Code (copy-paste safe)

```

def _resolve_bracket_price(self, value, tick_size, ref_price, side, is_tp):
    """Convert a TP/SL tick count to an absolute price.

    Values are Tradovate tick counts (e.g. 151 ticks for NQ).
    Price offset = ticks × tick_size (151 × 0.25 = 37.75 pts).

    For Buy orders: TP is above ref_price, SL is below
    For Sell orders: TP is below ref_price, SL is above
    """
    if value is None or tick_size is None:
        return None

    if ref_price is None:
        print(f"[API ORDER] Cannot calculate bracket: no reference price available")
        return None

```

```
# Convert tick count → price offset
offset = value * tick_size
if side == "Buy":
    result = round(ref_price + offset, 10) if is_tp else round(ref_price - offset, 10)
else:
    result = round(ref_price - offset, 10) if is_tp else round(ref_price + offset, 10)

print(f"[API ORDER] Bracket calc: {value} ticks × {tick_size} = {offset} pts, "
      f"ref={ref_price} → {'TP' if is_tp else 'SL'}={result}")
return result
```

Method: TradovateAccount.cancel_order_api

```
def cancel_order_api(self, order_id)
```

Cancel an order via the REST API.

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `result = self._api_fetch('/order/cancelOrder', 'POST', {'orderId':....`
2. **if** `result`: branches conditionally.
3. **return** `result`.

Code (copy-paste safe)

```
def cancel_order_api(self, order_id):
    """Cancel an order via the REST API."""
    result = self._api_fetch("/order/cancelOrder", "POST", {"orderId": order_id})
    if result:
        print(f"[API] Order {order_id} cancelled")
    return result
```

Method: TradovateAccount.liquidate_position_api

```
def liquidate_position_api(self, account_id=None)
```

Liquidate (flatten) all positions on the account via the REST API. Strategy: 1. Check for open positions first 2. Try /order/liquidatePosition endpoint 3. If that fails, close each position individually with opposite market orders Returns: dict with results, or None if no positions / all failed

Why these Python concepts were used

- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with `append`, and clearer about intent: this is a transform, not a side-effecting loop.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **if** not `account_id`: branches conditionally (with an **else/elif** arm).
2. Assigns `positions = self.get_positions_api(account_id)`.
3. Assigns `open_positions = [p for p in positions if p.get('netPos', 0) != 0]`.
4. **if** not `open_positions`: branches conditionally.
5. Calls `print(...)` for its side effect.
6. Assigns `result = self._api_fetch('/order/liquidatePosition', 'POST', {'acc....`
7. **if** `result`: branches conditionally.
8. Calls `print(...)` for its side effect.
9. Assigns `closed = 0`.
10. **for** `pos` in `open_positions`: iterates.
11. **return** `{'closed': closed, 'total': len(open_positions)}` if `closed > 0` else....

Code (copy-paste safe)

```
def liquidate_position_api(self, account_id=None):
    """Liquidate (flatten) all positions on the account via the REST API.

    Strategy:
    1. Check for open positions first
    2. Try /order/liquidatePosition endpoint
    3. If that fails, close each position individually with opposite market orders

    Returns: dict with results, or None if no positions / all failed
    """
    if not account_id:
        account_id, account_name = self._get_account_for_api()
        if not account_id:
            print("[API] No account found for liquidation")
            return None
    else:
        account_name = str(account_id)

    # Step 1: Check if there are actually open positions
    positions = self.get_positions_api(account_id)
    open_positions = [p for p in positions if p.get('netPos', 0) != 0]
    if not open_positions:
        print(f"[API] No open positions on {account_name} – already flat")
        return None

    print(f"[API] Found {len(open_positions)} open position(s) on {account_name} – liquidating...")

    # Step 2: Try the liquidatePosition endpoint
    result = self._api_fetch("/order/liquidatePosition", "POST", {"accountId": account_id})
    if result:
        print(f"[API] ■ Positions liquidated via /order/liquidatePosition")
        return result

    # Step 3: Fallback – close each position with opposite market orders
    print(f"[API] liquidatePosition failed – falling back to individual close orders")
    closed = 0
    for pos in open_positions:
        net_pos = pos.get('netPos', 0)
        contract_id = pos.get('contractId')
```

```

if net_pos == 0 or not contract_id:
    continue

# Get contract name for the order
contract = self._api_fetch(f"/contract/item?id={contract_id}")
contract_name = contract.get('name', '') if contract else ''
if not contract_name:
    print(f"[API] Could not resolve contract {contract_id} – skipping")
    continue

close_side = "Sell" if net_pos > 0 else "Buy"
close_qty = abs(net_pos)
payload = {
    "accountSpec": account_name,
    "accountId": account_id,
    "action": close_side,
    "symbol": contract_name,
    "orderQty": close_qty,
    "orderType": "Market",
    "isAutomated": False,
}
print(f"[API] Closing: {close_side} {close_qty}x {contract_name}")
close_result = self._api_fetch("/order/placeOrder", "POST", payload)
if close_result:
    print(f"[API] ■ Closed {contract_name}")
    closed += 1
else:
    print(f"[API] ■ Failed to close {contract_name}")

return {"closed": closed, "total": len(open_positions)} if closed > 0 else None

```

Method: TradovateAccount.get_positions_api

```
def get_positions_api(self, account_id=None)
```

Get current open positions via the REST API.

Why these Python concepts were used

- **list comprehension** — Builds a list in a single expression ([f(x) for x in xs]). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.

Step by step (top-level body)

1. Assigns positions = self._api_fetch('/position/list') or [].
2. if account_id: branches conditionally.
3. return positions.

Code (copy-paste safe)

```

def get_positions_api(self, account_id=None):
    """Get current open positions via the REST API."""
    positions = self._api_fetch("/position/list") or []
    if account_id:
        positions = [p for p in positions if p.get('accountId') == account_id]
    return positions

```

Method: TradovateAccount.cancel_all_orders_api

```
def cancel_all_orders_api(self, account_id=None)
```

Cancel all working (pending) orders on the account via the REST API. Only cancels if the account has no open positions (i.e. is flat). This cleans up orphaned bracket orders (stop/limit) after a position is closed by TP, SL, or manual liquidation.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **list comprehension** — Builds a list in a single expression ([f(x) for x in xs]). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `positions = self.get_positions_api(account_id)`.
2. Assigns `open_positions = [p for p in positions if p.get('netPos', 0) != 0]`.
3. **if** `open_positions`: branches conditionally.
4. Assigns `orders = self.get_orders_api()`.
5. **if** not `orders`: branches conditionally.
6. Assigns `cancelled = 0`.
7. **for** `order in orders`: iterates.
8. **if** `cancelled > 0`: branches conditionally.
9. **return** `cancelled`.

Code (copy-paste safe)

```
def cancel_all_orders_api(self, account_id=None):
    """Cancel all working (pending) orders on the account via the REST API.

    Only cancels if the account has no open positions (i.e. is flat).
    This cleans up orphaned bracket orders (stop/limit) after a position
    is closed by TP, SL, or manual liquidation.
    """
    # Only cancel if no active positions
    positions = self.get_positions_api(account_id)
    open_positions = [p for p in positions if p.get('netPos', 0) != 0]
    if open_positions:
        return 0

    orders = self.get_orders_api()
    if not orders:
        return 0

    cancelled = 0
    for order in orders:
        status = order.get('ordStatus', '')
        if status not in ('Working', 'Accepted', 'PendingNew'):
            continue
```

```

        if account_id and order.get('accountId') != account_id:
            continue
        oid = order.get('id')
        if oid:
            try:
                self.cancel_order_api(oid)
                cancelled += 1
            except Exception as e:
                print(f"[API] Failed to cancel order {oid}: {e}")
    if cancelled > 0:
        print(f"[API] ■ Cancelled {cancelled} pending order(s) (account is flat)")
    return cancelled

```

Method: TradovateAccount.get_orders_api

```
def get_orders_api(self)
```

Get current working orders via the REST API.

Step by step (top-level body)

1. **return** self._api_fetch('/order/list') or [].

Code (copy-paste safe)

```

def get_orders_api(self):
    """Get current working orders via the REST API."""
    return self._api_fetch("/order/list") or []

```

Method: TradovateAccount.get_trade_history

```
def get_trade_history(self)
```

*Get full trade history for ALL accounts under this login. Returns list of dicts, one per account:
 account_name, account_id, environment, balance, balance_sod, realized_pnl, open_pnl,
 trailing_max_drawdown, trailing_max_drawdown_limit, drawdown_mode, daily_pnl, fills*

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **list comprehension** — Builds a list in a single expression ([f(x) for x in xs]). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. try block with 1 except clause.

Code (copy-paste safe)

```
def get_trade_history(self):
    """Get full trade history for ALL accounts under this login.

    Returns list of dicts, one per account:
        account_name, account_id, environment, balance, balance_sod,
        realized_pnl, open_pnl, trailing_max_drawdown, trailing_max_drawdown_limit,
        drawdown_mode, daily_pnl, fills
    """
    try:
        # Get environment info
        env_info = self.driver.execute_script("""
            try {
                var auth = JSON.parse(sessionStorage.getItem('api_authenticator_state') || '{}');
                return {env: auth.environment || 'demo', username: auth.username || ''};
            } catch(e) { return {env: 'demo'}; }
            """)
        environment = env_info.get('env', 'demo') if env_info else 'demo'

        # Get ALL accounts under this login
        accounts = self._api_fetch("/account/list")
        if not accounts or not isinstance(accounts, list) or len(accounts) == 0:
            return None

        # Shared data – fetch once for all accounts
        all_fills = self._api_fetch("/fill/list") or []
        all_autoliq = self._api_fetch("/userAccountAutoLiq/list") or []

        # Resolve contract names for fills
        contract_cache = {}
        for f in all_fills:
            cid = f.get('contractId')
            if cid and cid not in contract_cache:
                c = self._api_fetch(f"/contract/item?id={cid}")
                contract_cache[cid] = c.get('name', str(cid)) if c else str(cid)

        # Index auto-liq by account ID
        autoliq_by_acct = {}
        for al in all_autoliq:
            autoliq_by_acct[al.get('accountId', al.get('account'))] = al

        from collections import defaultdict
        results = []

        for acct in accounts:
            aid = acct['id']
            aname = acct.get('name', '?')

            # Per-account data
            balance_logs = self._api_fetch(f"/cashBalanceLog/ldeps?masterids={aid}") or []
            snapshot = self._api_fetch("/cashBalance/getCashBalanceSnapshot", "POST", {
                "accountId": aid}) or {}

            # Filter fills for this account
            acct_fills = [f for f in all_fills if f.get('accountId') == aid]

            # Build daily P&L from cashBalanceLog
```



```

daily_raw = defaultdict(lambda: {"trades": 0, "gross_pnl": 0.0, "fees": 0.0,
    "balance_eod": 0.0})
for entry in sorted(balance_logs, key=lambda x: x.get('id', 0)):
    td = entry.get('tradeDate', {})
    date_str = f"{td.get('year', 0)}-{td.get('month', 1):02d}-{td.get('day', 1):02d}"
    ctype = entry.get('cashChangeType', '')
    delta = entry.get('delta', 0)
    daily_raw[date_str]["balance_eod"] = entry.get('amount', 0)
    if ctype == "TradePaired":
        daily_raw[date_str]["gross_pnl"] += delta
        daily_raw[date_str]["trades"] += 1
    elif ctype in ("Commission", "ExchangeFee", "ClearingFee", "NfaFee"):
        daily_raw[date_str]["fees"] += delta

daily_pnl = []
for date_str in sorted(daily_raw.keys()):
    d = daily_raw[date_str]
    net = d["gross_pnl"] + d["fees"]
    daily_pnl.append({
        "date": date_str,
        "trades": d["trades"],
        "gross_pnl": round(d["gross_pnl"], 2),
        "fees": round(d["fees"], 2),
        "net_pnl": round(net, 2),
        "balance_eod": round(d["balance_eod"], 2),
    })

# Build fills list for this account
fill_list = []
for f in sorted(acct_fills, key=lambda x: x.get('timestamp', '')):
    td = f.get('tradeDate', {})
    date_str = f"{td.get('year', 0)}-{td.get('month', 1):02d}-{td.get('day', 1):02d}"
    ts = f.get('timestamp', '')
    time_str = ts[11:19] if len(ts) > 19 else ts
    fill_list.append({
        "date": date_str,
        "time": time_str,
        "action": f.get('action', '?'),
        "qty": f.get('qty', 0),
        "contract": contract_cache.get(f.get('contractId', '?'), ''),
        "price": f.get('price', 0),
        "order_id": f.get('orderId', 0),
    })

al = autoliq_by_acct.get(aid, {})

results.append({
    "account_name": aname,
    "account_id": aid,
    "environment": environment,
    "balance": snapshot.get('netLiq', 0),
    "balance_sod": snapshot.get('netLiqSOD', 0),
    "realized_pnl": snapshot.get('realizedPnL', 0),
    "open_pnl": snapshot.get('openPnL', 0),
    "trailing_max_drawdown": al.get('trailingMaxDrawdown', 0),
    "trailing_max_drawdown_limit": al.get('trailingMaxDrawdownLimit', 0),
    "drawdown_mode": al.get('trailingMaxDrawdownMode', '?'),
    "daily_pnl": daily_pnl,
    "fills": fill_list,
})

```

```

        return results
    except Exception as e:
        print(f"[HISTORY] get_trade_history failed: {e}")
        import traceback
        traceback.print_exc()
        return None

```

Method: TradovateAccount.get_mnq_daily_pnl

```
def get_mnq_daily_pnl(self)
```

Get daily P&L for days with MNQ trading activity (farming days). Identifies farming days by checking for MNQ contract fills, then pulls the daily P&L from cashBalanceLog for those dates. Returns list of dicts, one per account that has MNQ activity: [{"account_name": str, "account_id": int, "mnq_daily_pnl": [{"date": "YYYY-MM-DD", "net_pnl": float}, ...]}

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **set comprehension** — Builds a set in one expression — usually to deduplicate the result of a transform.
- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def get_mnq_daily_pnl(self):
    """Get daily P&L for days with MNQ trading activity (farming days).

    Identifies farming days by checking for MNQ contract fills, then
    pulls the daily P&L from cashBalanceLog for those dates.

    Returns list of dicts, one per account that has MNQ activity:
        [{"account_name": str, "account_id": int,
         "mnq_daily_pnl": [{"date": "YYYY-MM-DD", "net_pnl": float}, ...]}]
    """
    try:
        accounts = self._api_fetch("/account/list")
        if not accounts or not isinstance(accounts, list):
            return []

        all_fills = self._api_fetch("/fill/list") or []

```

```

# Resolve unique contract IDs to names
contract_cache = {}
for f in all_fills:
    cid = f.get('contractId')
    if cid and cid not in contract_cache:
        c = self._api_fetch(f"/contract/item?id={cid}")
        contract_cache[cid] = c.get('name', str(cid)) if c else str(cid)

# Identify MNQ contract IDs
mnq_contract_ids = {cid for cid, name in contract_cache.items()
                     if str(name).upper().startswith('MNQ')}

if not mnq_contract_ids:
    return []

from collections import defaultdict
results = []

for acct in accounts:
    aid = acct['id']
    aname = acct.get('name', '?')

    # Find dates where this account had MNQ fills
    mnq_dates = set()
    for f in all_fills:
        if f.get('accountId') == aid and f.get('contractId') in mnq_contract_ids:
            td = f.get('tradeDate', {})
            date_str = f"{td.get('year', 0)}-{td.get('month', 1):02d}-{td.get('day', 1):02d}"
            mnq_dates.add(date_str)

    if not mnq_dates:
        continue

    # Get daily P&L from cashBalanceLog for MNQ dates only
    balance_logs = self._api_fetch(f"/cashBalanceLog/ldeps?masterids={aid}") or []

    daily_raw = defaultdict(lambda: {"gross_pnl": 0.0, "fees": 0.0})
    for entry in balance_logs:
        td = entry.get('tradeDate', {})
        date_str = f"{td.get('year', 0)}-{td.get('month', 1):02d}-{td.get('day', 1):02d}"
        if date_str not in mnq_dates:
            continue
        ctype = entry.get('cashChangeType', '')
        delta = entry.get('delta', 0)
        if ctype == "TradePaired":
            daily_raw[date_str]["gross_pnl"] += delta
        elif ctype in ("Commission", "ExchangeFee", "ClearingFee", "NfaFee"):
            daily_raw[date_str]["fees"] += delta

    mnq_daily = []
    for date_str in sorted(daily_raw.keys()):
        d = daily_raw[date_str]
        net = round(d["gross_pnl"] + d["fees"], 2)
        mnq_daily.append({"date": date_str, "net_pnl": net})

    if mnq_daily:
        results.append({
            "account_name": aname,
            "account_id": aid,
            "mnq_daily_pnl": mnq_daily,

```

```

    })

    return results
except Exception as e:
    print(f"[FARMING] get_mnq_daily_pnl failed: {e}")
    import traceback
    traceback.print_exc()
    return []

```

Step 8.5 — Build trader_companion/prop_firm_manager.py

What to do. Create the file `trader_companion/prop_firm_manager.py` in your project root and paste in the code shown in this step. The file is **2124** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from `requirements.txt`.

What this file is for. Coordinates the per-firm scraper instances, normalises their outputs, and pushes account state to the dashboard.

trader_companion/prop_firm_manager.py

2124 loc · 1 classes · 0 functions · 3 imports

Coordinates the per-firm scraper instances, normalises their outputs, and pushes account state to the dashboard. It defines **1** class, across **2,124** lines.

Python concepts demonstrated in this module

- Classes and objects
- Comprehensions
- Dunder methods
- f-strings
- Type hints

Imports — what each one is for

`import logging`

Why imported. Structured, levelled logging. Replaces `print()` with filterable, formattable output that can route to files, stderr, syslog, or HTTP endpoints.

Used in this file. `logging.getLogger`

Example. `logging.getLogger(__name__).info('connected to %s', host)`

Other common scenarios.

- `.debug()` / `.info()` / `.warning()` / `.error()` / `.exception()`
- `logger.exception()` captures the current traceback automatically
- `logging.basicConfig(level=logging.INFO)` for quick setup

- Handlers and Formatters route logs to files / streams / syslog

```
import datetime
```

Why imported. Dates, times, durations. The fundamental temporal types: date, time, datetime, timedelta, timezone.

Used in this file. datetime.date, datetime.date.today

Example. datetime.now() - timedelta(days=7) # one week ago

Other common scenarios.

- datetime.strptime(s, '%Y-%m-%d') parses a string
- datetime.strftime(fmt) formats one
- datetime.fromtimestamp(n) converts a Unix epoch
- datetime.utcnow() for UTC (better: datetime.now(timezone.utc))

```
from typing import Dict
from typing import Optional
from typing import Tuple
```

Why imported. Type-hint primitives — generics, unions, Optional, Callable, Protocol, TypeVar, TypedDict.

Used in this file. Dict, Optional, Tuple

Example. def lookup(k: str) -> Optional[int]: ...

Other common scenarios.

- List[int] / Dict[str, Any] / Tuple[int, str] (or bare list[int]+ on 3.9+)
- Callable[[int, int], int] for function signatures
- Protocol for structural typing (duck typing with checks)
- TypeVar('T') for generic functions and classes

Classes

```
class PropFirmManager
```

Manages prop firm-specific configurations and blueprints. Supported Prop Firms: - MFFU: My Funded Futures - Funded Next: Funded Next - FundingTicks: Funding Ticks - TopStep: TopStep - Trade Day: Trade Day (EOD Account Type) - Tradeify: Tradeify (Growth Account) - Top One Futures: Top One Futures All configurations are for \$50,000 accounts and manual trading only.

```
DETECTED_TO_BLUEPRINT = {'MFFU': 'MFFU_Flex', 'MFFU_Flex': 'MFFU_Flex', 'Funded Next': 'Funded Next',
    'FundedNext': 'Fund...
_PHASE_TRADE_ORDER = {'MFFU_Flex': {'Challenge': ['challenge_trade1', 'challenge_trade2'], 'Funded': [
    'funded_trade1',...
_HARD_STOP_THRESHOLDS: Dict[str, float] = {'TopStep': 0.0, 'Funded Next': 50000.0, 'FundingTicks': 50000.0,
    'TradeDay': 50000.0, 'Tradeify': 50000.0, 'AlphaFutures': 50000.0, 'Apex': 50000.0, 'Lucid': 50000.0,
_PROFIT_TARGETS: Dict[str, Dict[str, float]] = {'MFFU': {'Challenge': 3020, 'Funded': 1020}, 'MFFU_Flex':
    'Challenge': 3020, 'Funded': 5400}, 'Funded Next': {'Challenge': 3050, 'Funded': 5200}, 'FundingTicks
_NQ_TICK_TO_POINT_RATIO = 4
_FARMING_BUFFER_POINTS = 4
```

Code (copy-paste safe)

```
class PropFirmManager:
    """
    Manages prop firm-specific configurations and blueprints.

    Supported Prop Firms:
    - MFFU: My Funded Futures
    - Funded Next: Funded Next
    - FundingTicks: Funding Ticks
    - TopStep: TopStep
    - Trade Day: Trade Day (EOD Account Type)
    - Tradeify: Tradeify (Growth Account)
    - Top One Futures: Top One Futures

    All configurations are for $50,000 accounts and manual trading only.
    """

    # Mapping from detected firm code -> UI dropdown blueprint name
    DETECTED_TO_BLUEPRINT = {
        "MFFU": "MFFU_Flex",
        "MFFU_Flex": "MFFU_Flex",
        "Funded Next": "Funded Next",
        "FundedNext": "Funded Next",
        "FundingTicks": "FundingTicks",
        "Trade Day": "TradeDay",
        "TopStep": "TopStep",
        "Apex": "Apex",
        "Tradeify": "Tradeify",
        "Lucid": "Lucid",
        "AlphaFutures": "Alpha Futures",
        "AlphaFutures GC": "AlphaFutures GC",
        "Top One Futures": "Top One Futures",
    }

    def __init__(self):
        self.logger = logging.getLogger(__name__)
        self.current_firm_code = "MFFU_Flex" # Default prop firm

        # Per-account daily direction locks: {account_number: {"date": date, "direction": "BUY"/"SELL"}}
        self._account_direction_locks: Dict[str, Dict] = {}

        # Prop firm blueprints - $50k account configurations only core challenge with the flex addon
        self.firm_blueprints = {
            "MFFU_Flex": {
                "name": "MFFU_Flex",
                "account_sizes": ["$50,000"],
                "trading_phases": ["Challenge Phase", "Funded Phase", "Double Dip Phase", "Farming Phase"],
                "strategy_configs": {
                    "challenge_trade1": {
                        "50k": {
                            "tradovate_symbol": "NQM6",
                            "tradovate_qty": 2,
                            "tradovate_tp_ticks": 151,
                            "tradovate_sl_ticks": 201,
                            "mt5_volume": 4.6,
                            "mt5_tp_points": 46,
                            "mt5_sl_points": 42
                        }
                    }
                }
            },
        },
```

```

"challenge_trade2": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 151,
        "tradovate_sl_ticks": 201,
        "mt5_volume": 8.4,
        "mt5_tp_points": 46,
        "mt5_sl_points": 42
    }
},
"funded_trade1": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 540,
        "tradovate_sl_ticks": 201,
        "mt5_volume": 16,
        "mt5_tp_points": 46,
        "mt5_sl_points": 139
    }
},
"funded_trade2": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 240,
        "tradovate_sl_ticks": 261,
        "mt5_volume": 20,
        "mt5_tp_points": 61,
        "mt5_sl_points": 64
    }
},
"funded_trade3": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 240,
        "tradovate_sl_ticks": 245,
        "mt5_volume": 18.0,
        "mt5_tp_points": 57,
        "mt5_sl_points": 64
    }
},
"funded_trade4": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 240,
        "tradovate_sl_ticks": 238,
        "mt5_volume": 13.6,
        "mt5_tp_points": 55,
        "mt5_sl_points": 64
    }
},
"funded_trade_doubledip_1": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 540,
        "tradovate_sl_ticks": 201,

```

```

        "mt5_volume": 13,
        "mt5_tp_points": 46,
        "mt5_sl_points": 139
    }
}, "funded_trade_doubledip_2": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 240,
        "tradovate_sl_ticks": 261,
        "mt5_volume": 14,
        "mt5_tp_points": 61,
        "mt5_sl_points": 64
    }
}, "funded_trade_doubledip_3": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 240,
        "tradovate_sl_ticks": 245,
        "mt5_volume": 9,
        "mt5_tp_points": 57,
        "mt5_sl_points": 64
    }
}, "funded_trade_doubledip_4": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 240,
        "tradovate_sl_ticks": 238,
        "mt5_volume": 20,
        "mt5_tp_points": 55,
        "mt5_sl_points": 64
    }
},
"farming": {
    "50k": {
        "tradovate_symbol": "MNQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 154,
        "tradovate_sl_ticks": 600,
        "mt5_volume": 3.2,
        "mt5_tp_points": 146,
        "mt5_sl_points": 43
    }
}
}
},
"Funded Next": {
    "name": "Funded Next",
    "account_sizes": ["$50,000"],
    "trading_phases": ["Challenge Phase", "Payout 1", "Payout 2", "Payout 3", "Payout 4",
        "Farming Phase"],
    "strategy_configs": {
        "challenge_trade1": {
            "50k": {
                "tradovate_symbol": "NQM6",
                "tradovate_qty": 2,
                "tradovate_tp_ticks": 102,
                "tradovate_sl_ticks": 200,

```



```

        "mt5_volume": 3.6,
        "mt5_tp_points": 46,
        "mt5_sl_points": 30
    }
},
"challenge_trade2": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 102,
        "tradovate_sl_ticks": 200,
        "mt5_volume": 6.2,
        "mt5_tp_points": 46,
        "mt5_sl_points": 30
    }
},
"challenge_trade3": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 101,
        "tradovate_sl_ticks": 200,
        "mt5_volume": 10.4,
        "mt5_tp_points": 46,
        "mt5_sl_points": 30
    }
},
"funded_trade1": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 520,
        "tradovate_sl_ticks": 200,
        "mt5_volume": 18.4,
        "mt5_tp_points": 46,
        "mt5_sl_points": 134
    }
},
"funded_trade2": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 220,
        "tradovate_sl_ticks": 260,
        "mt5_volume": 17,
        "mt5_tp_points": 61,
        "mt5_sl_points": 59
    }
},
"funded_trade3": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 220,
        "tradovate_sl_ticks": 245,
        "mt5_volume": 3,
        "mt5_tp_points": 57,
        "mt5_sl_points": 59
    }
},

```

```

    "funded_trade4": {
        "50k": {
            "tradovate_symbol": "NQM6",
            "tradovate_qty": 2,
            "tradovate_tp_ticks": 220,
            "tradovate_sl_ticks": 238,
            "mt5_volume": 0,
            "mt5_tp_points": 55,
            "mt5_sl_points": 59
        }
    },
    "farming": {
        "50k": {
            "tradovate_symbol": "MNQM6",
            "tradovate_qty": 2,
            "tradovate_tp_ticks": 204,
            "tradovate_sl_ticks": 600,
            "mt5_volume": 3.2,
            "mt5_tp_points": 146,
            "mt5_sl_points": 55
        }
    }
},
"FundingTicks": {
    "name": "FundingTicks",
    "account_sizes": ["$50,000"],
    "trading_phases": ["Challenge Phase", "Funded Phase", "Farming Phase"],
    "strategy_configs": {
        "challenge_trade1": {
            "50k": {
                "tradovate_symbol": "NQM6",
                "tradovate_qty": 2,
                "tradovate_tp_ticks": 127,
                "tradovate_sl_ticks": 250,
                "mt5_volume": 5.2,
                "mt5_tp_points": 58,
                "mt5_sl_points": 36
            }
        },
        "challenge_trade2": {
            "50k": {
                "tradovate_symbol": "NQM6",
                "tradovate_qty": 2,
                "tradovate_tp_ticks": 127,
                "tradovate_sl_ticks": 250,
                "mt5_volume": 7.8,
                "mt5_tp_points": 58,
                "mt5_sl_points": 36
            }
        },
        "funded_trade1": {
            "50k": {
                "tradovate_symbol": "NQM6",
                "tradovate_qty": 2,
                "tradovate_tp_ticks": 500,
                "tradovate_sl_ticks": 250,
                "mt5_volume": 13.8,
                "mt5_tp_points": 58,
                "mt5_sl_points": 129
            }
        }
    }
}

```

```

    }
  },
  "funded_trade2": {
    "50k": {
      "tradovate_symbol": "NQM6",
      "tradovate_qty": 2,
      "tradovate_tp_ticks": 150,
      "tradovate_sl_ticks": 250,
      "mt5_volume": 18,
      "mt5_tp_points": 58,
      "mt5_sl_points": 42
    }
  },
  "funded_trade3": {
    "50k": {
      "tradovate_symbol": "NQM6",
      "tradovate_qty": 2,
      "tradovate_tp_ticks": 150,
      "tradovate_sl_ticks": 250,
      "mt5_volume": 8.6,
      "mt5_tp_points": 58,
      "mt5_sl_points": 42
    }
  },
  "funded_trade4": {
    "50k": {
      "tradovate_symbol": "NQM6",
      "tradovate_qty": 2,
      "tradovate_tp_ticks": 150,
      "tradovate_sl_ticks": 250,
      "mt5_volume": 0,
      "mt5_tp_points": 58,
      "mt5_sl_points": 42
    }
  },
  "farming": {
    "50k": {
      "tradovate_symbol": "MNQM6",
      "tradovate_qty": 2,
      "tradovate_tp_ticks": 204,
      "tradovate_sl_ticks": 600,
      "mt5_volume": 1.6,
      "mt5_tp_points": 146,
      "mt5_sl_points": 55
    }
  }
},
"TopStep": {
  "name": "TopStep",
  "account_sizes": ["$50,000"],
  "trading_phases": ["Challenge Phase", "Funded Phase", "Double Dip Phase", "Farming Phase"],
  "strategy_configs": {
    "challenge_trade1": {
      "50k": {
        "topstepx_symbol": "NQM26",
        "topstepx_qty": 2,
        "topstepx_tp_ticks": 151,
        "topstepx_sl_ticks": 201,
        "mt5_volume": 4.6,

```

```

        "mt5_tp_points": 46,
        "mt5_sl_points": 42
    }
},
"challenge_trade2": {
    "50k": {
        "topstepx_symbol": "NQM26",
        "topstepx_qty": 2,
        "topstepx_tp_ticks": 151,
        "topstepx_sl_ticks": 201,
        "mt5_volume": 8.4,
        "mt5_tp_points": 46,
        "mt5_sl_points": 42
    }
},
"funded_trade1": {
    "50k": {
        "topstepx_symbol": "NQM26",
        "topstepx_qty": 2,
        "topstepx_tp_ticks": 340,
        "topstepx_sl_ticks": 201,
        "mt5_volume": 16,
        "mt5_tp_points": 46,
        "mt5_sl_points": 89
    }
},
"funded_trade2": {
    "50k": {
        "topstepx_symbol": "NQM26",
        "topstepx_qty": 2,
        "topstepx_tp_ticks": 140,
        "topstepx_sl_ticks": 161,
        "mt5_volume": 20,
        "mt5_tp_points": 36,
        "mt5_sl_points": 39
    }
},
"funded_trade3": {
    "50k": {
        "topstepx_symbol": "NQM26",
        "topstepx_qty": 2,
        "topstepx_tp_ticks": 140,
        "topstepx_sl_ticks": 146,
        "mt5_volume": 18.0,
        "mt5_tp_points": 32,
        "mt5_sl_points": 39
    }
},
"funded_trade4": {
    "50k": {
        "topstepx_symbol": "NQM26",
        "topstepx_qty": 2,
        "topstepx_tp_ticks": 140,
        "topstepx_sl_ticks": 139,
        "mt5_volume": 13.6,
        "mt5_tp_points": 30,
        "mt5_sl_points": 39
    }
},
"funded_trade_doubledip_1": {
    "50k": {

```

```

        "topstepx_symbol": "NQM26",
        "topstepx_qty": 2,
        "topstepx_tp_ticks": 340,
        "topstepx_sl_ticks": 201,
        "mt5_volume": 16,
        "mt5_tp_points": 46,
        "mt5_sl_points": 89
    }
}, "funded_trade_doubledip_2": {
    "50k": {
        "topstepx_symbol": "NQM26",
        "topstepx_qty": 2,
        "topstepx_tp_ticks": 140,
        "topstepx_sl_ticks": 161,
        "mt5_volume": 20,
        "mt5_tp_points": 36,
        "mt5_sl_points": 39
    }
}, "funded_trade_doubledip_3": {
    "50k": {
        "topstepx_symbol": "NQM26",
        "topstepx_qty": 2,
        "topstepx_tp_ticks": 140,
        "topstepx_sl_ticks": 146,
        "mt5_volume": 9,
        "mt5_tp_points": 32,
        "mt5_sl_points": 39
    }
}, "funded_trade_doubledip_4": {
    "50k": {
        "topstepx_symbol": "NQM26",
        "topstepx_qty": 2,
        "topstepx_tp_ticks": 140,
        "topstepx_sl_ticks": 139,
        "mt5_volume": 20,
        "mt5_tp_points": 30,
        "mt5_sl_points": 39
    }
},
    "farming": {
        "50k": {
            "topstepx_symbol": "MNQM26",
            "topstepx_qty": 2,
            "topstepx_tp_ticks": 154,
            "topstepx_sl_ticks": 600,
            "mt5_volume": 3.6,
            "mt5_tp_points": 146,
            "mt5_sl_points": 43
        }
    }
},
    "Lucid": {
        "name": "Lucid",
        "account_sizes": ["$50,000"],
        "trading_phases": ["Challenge Phase", "Funded Phase", "Double Dip Phase", "Farming Phase"],
        "strategy_configs": {
            "challenge_trade1": {
                "50k": {
                    "tradovate_symbol": "NQM6",

```

```

        "tradovate_qty": 2,
        "tradovate_tp_ticks": 151,
        "tradovate_sl_ticks": 200,
        "mt5_volume": 5,
        "mt5_tp_points": 46,
        "mt5_sl_points": 42
    }
},
"challenge_trade2": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 151,
        "tradovate_sl_ticks": 200,
        "mt5_volume": 8,
        "mt5_tp_points": 46,
        "mt5_sl_points": 42
    }
},
"funded_trade1": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 340,
        "tradovate_sl_ticks": 200,
        "mt5_volume": 15,
        "mt5_tp_points": 46,
        "mt5_sl_points": 89
    }
},
"funded_trade2": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 140,
        "tradovate_sl_ticks": 170,
        "mt5_volume": 18,
        "mt5_tp_points": 38,
        "mt5_sl_points": 39
    }
},
"funded_trade3": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 140,
        "tradovate_sl_ticks": 160,
        "mt5_volume": 18.0,
        "mt5_tp_points": 36,
        "mt5_sl_points": 39
    }
},
"funded_trade4": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 140,
        "tradovate_sl_ticks": 155,
        "mt5_volume": 0,
        "mt5_tp_points": 34,

```

```

        "mt5_sl_points": 39
    }
},
"farming": {
    "50k": {
        "tradovate_symbol": "MNQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 154,
        "tradovate_sl_ticks": 600,
        "mt5_volume": 3.6,
        "mt5_tp_points": 146,
        "mt5_sl_points": 43
    }
}
},
"TradeDay": {
    "name": "TradeDay",
    "account_sizes": ["$50,000"],
    "trading_phases": ["Challenge Phase", "Funded Phase", "Double Dip Phase", "Farming Phase"],
    "strategy_configs": {
        "challenge_trade1": {
            "50k": {
                "tradovate_symbol": "NQM6",
                "tradovate_qty": 2,
                "tradovate_tp_ticks": 62,
                "tradovate_sl_ticks": 200,
                "mt5_volume": 3.6,
                "mt5_tp_points": 46,
                "mt5_sl_points": 20
            }
        },
        "challenge_trade2": {
            "50k": {
                "tradovate_symbol": "NQM6",
                "tradovate_qty": 2,
                "tradovate_tp_ticks": 62,
                "tradovate_sl_ticks": 200,
                "mt5_volume": 5.2,
                "mt5_tp_points": 46,
                "mt5_sl_points": 20
            }
        },
        "challenge_trade3": {
            "50k": {
                "tradovate_symbol": "NQM6",
                "tradovate_qty": 2,
                "tradovate_tp_ticks": 62,
                "tradovate_sl_ticks": 200,
                "mt5_volume": 7.4,
                "mt5_tp_points": 46,
                "mt5_sl_points": 20
            }
        },
        "challenge_trade4": {
            "50k": {
                "tradovate_symbol": "NQM6",
                "tradovate_qty": 2,
                "tradovate_tp_ticks": 62,
                "tradovate_sl_ticks": 200,

```

```

        "mt5_volume": 10.2,
        "mt5_tp_points": 46,
        "mt5_sl_points": 20
    }
},
"challenge_trade5": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 62,
        "tradovate_sl_ticks": 200,
        "mt5_volume": 14.2,
        "mt5_tp_points": 46,
        "mt5_sl_points": 20
    }
},
"funded_trade1": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 500,
        "tradovate_sl_ticks": 200,
        "mt5_volume": 20,
        "mt5_tp_points": 46,
        "mt5_sl_points": 129
    }
},
"funded_trade2": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 500,
        "tradovate_sl_ticks": 200,
        "mt5_volume": 28,
        "mt5_tp_points": 46,
        "mt5_sl_points": 129
    }
},
"funded_trade3": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 500,
        "tradovate_sl_ticks": 200,
        "mt5_volume": 20,
        "mt5_tp_points": 46,
        "mt5_sl_points": 129
    }
}
},
"AlphaFutures": {
    "name": "AlphaFutures",
    "account_sizes": ["$50,000", "$100,000", "$150,000"],
    "trading_phases": ["Challenge Phase", "Funded Phase", "Farming Phase"],
    "strategy_configs": {
        "challenge_trade1": {
            "50k": {
                "tradovate_symbol": "NQM6",
                "tradovate_qty": 2,

```



```

        "tradovate_tp_ticks": 202,
        "tradovate_sl_ticks": 175,
        "mt5_volume": 3,
        "mt5_tp_points": 39,
        "mt5_sl_points": 55
    },
    "100k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 4,
        "tradovate_tp_ticks": 202,
        "tradovate_sl_ticks": 175,
        "mt5_volume": 6,
        "mt5_tp_points": 39,
        "mt5_sl_points": 55
    },
    "150k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 6,
        "tradovate_tp_ticks": 202,
        "tradovate_sl_ticks": 175,
        "mt5_volume": 9,
        "mt5_tp_points": 39,
        "mt5_sl_points": 55
    }
},
"challenge_trade2": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 202,
        "tradovate_sl_ticks": 175,
        "mt5_volume": 7.4,
        "mt5_tp_points": 39,
        "mt5_sl_points": 55
    },
    "100k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 4,
        "tradovate_tp_ticks": 202,
        "tradovate_sl_ticks": 175,
        "mt5_volume": 14.8,
        "mt5_tp_points": 39,
        "mt5_sl_points": 55
    },
    "150k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 6,
        "tradovate_tp_ticks": 202,
        "tradovate_sl_ticks": 175,
        "mt5_volume": 22.2,
        "mt5_tp_points": 39,
        "mt5_sl_points": 55
    }
},
"funded_trade1": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 600,
        "tradovate_sl_ticks": 175,

```

```

        "mt5_volume": 22,
        "mt5_tp_points": 39,
        "mt5_sl_points": 154
    },
    "100k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 4,
        "tradovate_tp_ticks": 600,
        "tradovate_sl_ticks": 175,
        "mt5_volume": 44,
        "mt5_tp_points": 39,
        "mt5_sl_points": 154
    },
    "150k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 6,
        "tradovate_tp_ticks": 600,
        "tradovate_sl_ticks": 175,
        "mt5_volume": 66,
        "mt5_tp_points": 39,
        "mt5_sl_points": 154
    }
},
"funded_trade2": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 100,
        "tradovate_sl_ticks": 175,
        "mt5_volume": 40,
        "mt5_tp_points": 39,
        "mt5_sl_points": 29
    },
    "100k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 4,
        "tradovate_tp_ticks": 100,
        "tradovate_sl_ticks": 175,
        "mt5_volume": 80,
        "mt5_tp_points": 39,
        "mt5_sl_points": 29
    },
    "150k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 6,
        "tradovate_tp_ticks": 100,
        "tradovate_sl_ticks": 175,
        "mt5_volume": 120,
        "mt5_tp_points": 39,
        "mt5_sl_points": 29
    }
},
"funded_trade3": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 200,
        "tradovate_sl_ticks": 175,
        "mt5_volume": 25,
        "mt5_tp_points": 39,

```

```

        "mt5_sl_points": 54
    },
    "100k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 4,
        "tradovate_tp_ticks": 200,
        "tradovate_sl_ticks": 175,
        "mt5_volume": 50,
        "mt5_tp_points": 39,
        "mt5_sl_points": 54
    },
    "150k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 6,
        "tradovate_tp_ticks": 200,
        "tradovate_sl_ticks": 175,
        "mt5_volume": 75,
        "mt5_tp_points": 39,
        "mt5_sl_points": 54
    }
},
"funded_trade4": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 200,
        "tradovate_sl_ticks": 175,
        "mt5_volume": 15,
        "mt5_tp_points": 39,
        "mt5_sl_points": 54
    },
    "100k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 4,
        "tradovate_tp_ticks": 200,
        "tradovate_sl_ticks": 175,
        "mt5_volume": 30,
        "mt5_tp_points": 39,
        "mt5_sl_points": 54
    },
    "150k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 6,
        "tradovate_tp_ticks": 200,
        "tradovate_sl_ticks": 175,
        "mt5_volume": 45,
        "mt5_tp_points": 39,
        "mt5_sl_points": 54
    }
},
"farming": {
    "50k": {
        "tradovate_symbol": "MNQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 204,
        "tradovate_sl_ticks": 600,
        "mt5_volume": 3.6,
        "mt5_tp_points": 146,
        "mt5_sl_points": 55
    }
}

```

```

    }
  },
  "AlphaFutures GC": {
    "name": "AlphaFutures GC",
    "account_sizes": ["$50,000"],
    "trading_phases": ["Challenge Phase", "Funded Phase", "Farming Phase"],
    "strategy_configs": {
      "challenge_trade1": {
        "50k": {
          "tradovate_symbol": "GCM6",
          "tradovate_qty": 1,
          "tradovate_tp_ticks": 201,
          "tradovate_sl_ticks": 175,
          "mt5_symbol": "XAUUSD",
          "mt5_volume": 0.09,
          "mt5_tp_points": 1710,
          "mt5_sl_points": 2050
        }
      },
      "challenge_trade2": {
        "50k": {
          "tradovate_symbol": "GCM6",
          "tradovate_qty": 1,
          "tradovate_tp_ticks": 201,
          "tradovate_sl_ticks": 175,
          "mt5_symbol": "XAUUSD",
          "mt5_volume": 0.2,
          "mt5_tp_points": 1710,
          "mt5_sl_points": 2050
        }
      },
      "funded_trade1": {
        "50k": {
          "tradovate_symbol": "GCM6",
          "tradovate_qty": 2,
          "tradovate_tp_ticks": 300,
          "tradovate_sl_ticks": 88,
          "mt5_symbol": "XAUUSD",
          "mt5_volume": 0.49,
          "mt5_tp_points": 1710,
          "mt5_sl_points": 6040
        }
      },
      "funded_trade2": {
        "50k": {
          "tradovate_symbol": "GCM6",
          "tradovate_qty": 1,
          "tradovate_tp_ticks": 100,
          "tradovate_sl_ticks": 175,
          "mt5_symbol": "XAUUSD",
          "mt5_volume": 0.75,
          "mt5_tp_points": 1710,
          "mt5_sl_points": 1040
        }
      },
      "funded_trade3": {
        "50k": {
          "tradovate_symbol": "GCM6",
          "tradovate_qty": 1,

```

```

        "tradovate_tp_ticks": 200,
        "tradovate_sl_ticks": 175,
        "mt5_symbol": "XAUUSD",
        "mt5_volume": 0.24,
        "mt5_tp_points": 1710,
        "mt5_sl_points": 2040
    }
},
"funded_trade4": {
    "50k": {
        "tradovate_symbol": "GCM6",
        "tradovate_qty": 1,
        "tradovate_tp_ticks": 200,
        "tradovate_sl_ticks": 175,
        "mt5_symbol": "XAUUSD",
        "mt5_volume": 0,
        "mt5_tp_points": 1710,
        "mt5_sl_points": 2040
    }
},
"farming": {
    "50k": {
        "tradovate_symbol": "MGCM6",
        "tradovate_qty": 1,
        "tradovate_tp_ticks": 204,
        "tradovate_sl_ticks": 600,
        "mt5_symbol": "XAUUSD",
        "mt5_volume": 0.9,
        "mt5_tp_points": 596,
        "mt5_sl_points": 208
    }
}
},
"Tradeify": {
    "name": "Tradeify",
    "account_sizes": ["$50,000"],
    "trading_phases": ["Challenge Phase", "Payout 1", "Payout 2", "Payout 3", "Payout 4",
        "Farming (Consistency)"],
    "strategy_configs": {
        "challenge_trade1": {
            "50k": {
                "tradovate_symbol": "NQM6",
                "tradovate_qty": 2,
                "tradovate_tp_ticks": 102,
                "tradovate_sl_ticks": 200,
                "mt5_volume": 3,
                "mt5_tp_points": 46,
                "mt5_sl_points": 30
            }
        },
        "challenge_trade2": {
            "50k": {
                "tradovate_symbol": "NQM6",
                "tradovate_qty": 2,
                "tradovate_tp_ticks": 102,
                "tradovate_sl_ticks": 200,
                "mt5_volume": 5.2,
                "mt5_tp_points": 46,
                "mt5_sl_points": 30
            }
        }
    }
}
}

```

```

    }
}, "challenge_trade3": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 102,
        "tradovate_sl_ticks": 200,
        "mt5_volume": 8.8,
        "mt5_tp_points": 46,
        "mt5_sl_points": 30
    }
},
"funded_trade1": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 540,
        "tradovate_sl_ticks": 200,
        "mt5_volume": 15,
        "mt5_tp_points": 46,
        "mt5_sl_points": 139
    }
},
"funded_trade2": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 240,
        "tradovate_sl_ticks": 260,
        "mt5_volume": 18,
        "mt5_tp_points": 61,
        "mt5_sl_points": 64
    }
},
"funded_trade3": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 240,
        "tradovate_sl_ticks": 245,
        "mt5_volume": 3,
        "mt5_tp_points": 57,
        "mt5_sl_points": 64
    }
},
"funded_trade4": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 240,
        "tradovate_sl_ticks": 238,
        "mt5_volume": 0,
        "mt5_tp_points": 55,
        "mt5_sl_points": 64
    }
},
"farming": {
    "50k": {
        "tradovate_symbol": "MNQM6",
        "tradovate_qty": 2,

```

```

        "tradovate_tp_ticks": 154,
        "tradovate_sl_ticks": 600,
        "mt5_volume": 3.6,
        "mt5_tp_points": 146,
        "mt5_sl_points": 43
    }
}
},
"Apex": {
    "name": "Apex",
    "account_sizes": ["$50,000"],
    "trading_phases": ["Challenge Phase", "Payout 1", "Payout 2", "Payout 3", "Payout 4",
        "Farming (Consistency)"],
    "strategy_configs": {
        "challenge_trade1": {
            "50k": {
                "tradovate_symbol": "NQM6",
                "tradovate_qty": 2,
                "tradovate_tp_ticks": 310,
                "tradovate_sl_ticks": 100,
                "mt5_volume": 0,
                "mt5_tp_points": 21,
                "mt5_sl_points": 82
            }
        },
        "challenge_trade2": {
            "50k": {
                "tradovate_symbol": "NQM6",
                "tradovate_qty": 2,
                "tradovate_tp_ticks": 410,
                "tradovate_sl_ticks": 100,
                "mt5_volume": 0,
                "mt5_tp_points": 21,
                "mt5_sl_points": 107
            }
        },
        "funded_trade1": {
            "50k": {
                "tradovate_symbol": "NQM6",
                "tradovate_qty": 2,
                "tradovate_tp_ticks": 200,
                "tradovate_sl_ticks": 100,
                "mt5_volume": 0,
                "mt5_tp_points": 21,
                "mt5_sl_points": 54
            }
        },
        "funded_trade2": {
            "50k": {
                "tradovate_symbol": "NQM6",
                "tradovate_qty": 2,
                "tradovate_tp_ticks": 100,
                "tradovate_sl_ticks": 100,
                "mt5_volume": 0,
                "mt5_tp_points": 21,
                "mt5_sl_points": 29
            }
        },
        "funded_trade3": {

```

```

        "50k": {
            "tradovate_symbol": "NQM6",
            "tradovate_qty": 2,
            "tradovate_tp_ticks": 150,
            "tradovate_sl_ticks": 100,
            "mt5_volume": 0,
            "mt5_tp_points": 21,
            "mt5_sl_points": 42
        }
    },
    "funded_trade4": {
        "50k": {
            "tradovate_symbol": "NQM6",
            "tradovate_qty": 2,
            "tradovate_tp_ticks": 150,
            "tradovate_sl_ticks": 100,
            "mt5_volume": 0,
            "mt5_tp_points": 21,
            "mt5_sl_points": 42
        }
    },
    "farming": {
        "50k": {
            "tradovate_symbol": "MNQM6",
            "tradovate_qty": 3,
            "tradovate_tp_ticks": 173,
            "tradovate_sl_ticks": 500,
            "mt5_volume": 0,
            "mt5_tp_points": 121,
            "mt5_sl_points": 48
        }
    }
},
"Top One Futures": {
    "name": "Top One Futures Elite Access",
    "account_sizes": ["$50,000"],
    "trading_phases": ["Challenge Phase", "Payout 1", "Payout 2", "Double Dip Payout 1",
        "Double Dip Payout 2", "Farming Phase"],
    "strategy_configs": {
        "challenge_trade1": {
            "50k": {
                "tradovate_symbol": "NQM6",
                "tradovate_qty": 2,
                "tradovate_tp_ticks": 301,
                "tradovate_sl_ticks": 200,
                "mt5_volume": 0,
                "mt5_tp_points": 0,
                "mt5_sl_points": 0
            }
        },
        "funded_trade1": {
            "50k": {
                "tradovate_symbol": "NQM6",
                "tradovate_qty": 1,
                "tradovate_tp_ticks": 240,
                "tradovate_sl_ticks": 200,
                "mt5_volume": 0,
                "mt5_tp_points": 0,
                "mt5_sl_points": 0,

```



```

        "profit_target": 3500
    },
    "funded_trade2": {
        "50k": {
            "tradovate_symbol": "NQM6",
            "tradovate_qty": 1,
            "tradovate_tp_ticks": 102,
            "tradovate_sl_ticks": 200,
            "mt5_volume": 0,
            "mt5_tp_points": 0,
            "mt5_sl_points": 0,
            "profit_target": 1000
        }
    },
    "funded_trade_doubledip_1": {
        "50k": {
            "tradovate_symbol": "NQM6",
            "tradovate_qty": 1,
            "tradovate_tp_ticks": 240,
            "tradovate_sl_ticks": 200,
            "mt5_volume": 0,
            "mt5_tp_points": 0,
            "mt5_sl_points": 0,
            "profit_target": 3500
        }
    },
    "funded_trade_doubledip_2": {
        "50k": {
            "tradovate_symbol": "NQM6",
            "tradovate_qty": 1,
            "tradovate_tp_ticks": 102,
            "tradovate_sl_ticks": 200,
            "mt5_volume": 0,
            "mt5_tp_points": 0,
            "mt5_sl_points": 0,
            "profit_target": 1000
        }
    },
    "farming": {
        "50k": {
            "tradovate_symbol": "MNQM6",
            "tradovate_qty": 2,
            "tradovate_tp_ticks": 254,
            "tradovate_sl_ticks": 600,
            "mt5_volume": 0,
            "mt5_tp_points": 0,
            "mt5_sl_points": 0
        }
    }
}

```

```

def detect_prop_firm(self, username: str) -> Optional[str]:
    """Detect prop firm based on username prefix. Returns None if unrecognized."""
    if not username or (isinstance(username, str) and len(username) < 4):
        self.logger.warning(f"Username too short: '{username}', cannot detect prop firm")
        return None

    prefix = username[:4].upper()

```

```

    if prefix in self.firm_blueprints:
        self.logger.info(f"Detected prop firm '{prefix}' from username: '{username}'")
        return prefix

    if any(prefix.startswith(var) for var in ["APEX"]):
        return "Apex"
    elif any(prefix.startswith(var) for var in ["FTPR"]):
        return "FundingTicks"
    elif any(prefix.startswith(var) for var in ["ELTD", "TDFU"]):
        return "Trade Day"
    elif any(prefix.startswith(var) for var in ["FNFT", "FTFN"]):
        return "Funded Next"
    elif any(prefix.startswith(var) for var in ["MFFU"]):
        return "MFFU"
    elif any(prefix.startswith(var) for var in ["V2-"]):
        return "TopStep"
    elif any(prefix.startswith(var) for var in ["TDFY", "FTDF"]):
        return "Tradeify"
    elif any(prefix.startswith(var) for var in ["LFE0", "LFF0"]):
        return "Lucid"
    elif any(prefix.startswith(var) for var in ["AFAD"]):
        return "AlphaFutures"

    self.logger.warning(f"Unknown prefix '{prefix}' from account '{username}' – prop firm not recognized")
    return None

def get_firm_info(self, firm_code: str) -> Dict:
    """Get complete prop firm information."""
    # Normalize firm_code to handle variations
    normalized_code = firm_code
    if firm_code == "Alpha Futures":
        normalized_code = "AlphaFutures"
    elif firm_code == "FundedNext":
        normalized_code = "Funded Next"
    elif firm_code == "TopOneFutures":
        normalized_code = "Top One Futures"
    elif firm_code in ("MFFU", "My Funded Futures"):
        # Legacy alias – MFFU is no longer a separate blueprint
        normalized_code = "MFFU_Flex"

    return self.firm_blueprints.get(normalized_code, self.firm_blueprints["MFFU_Flex"])

def get_account_sizes(self, firm_code: str) -> list:
    """Get account sizes for specific prop firm."""
    firm_info = self.get_firm_info(firm_code)
    return firm_info.get("account_sizes", ["$50,000"])

def get_trading_phases(self, firm_code: str) -> list:
    """Get trading phases for specific prop firm."""
    firm_info = self.get_firm_info(firm_code)
    return firm_info.get("trading_phases", ["Challenge Phase", "Funded Phase", "Farming Phase"])

def convert_account_size_to_key(self, account_size: str, firm_code: str = None) -> str:
    """Convert account size string to size key (e.g., '$50,000' -> '50k')"""
    if not account_size:
        return "50k"
    s = account_size.lower().replace(",", "").replace("$", "").strip()
    if "150" in s or s == "150k":
        return "150k"
    if "100" in s or s == "100k":
        return "100k"

```

```

        return "100k"
    return "50k"

def get_strategy_config(self, firm_code: str, phase_key: str, size_key: str = "50k") -> Dict:
    """Get strategy configuration for specific prop firm and phase."""
    # Normalize size_key: "$50,000" -> "50k", "$100,000" -> "100k", etc.
    size_key = self.convert_account_size_to_key(size_key, firm_code)
    self.logger.info(
        f"[DEBUG get_strategy_config] firm_code='{firm_code}', phase_key='{phase_key}', size_key='{size_key}'")

    firm_info = self.get_firm_info(firm_code)
    strategy_configs = firm_info.get("strategy_configs", {})

    phase_config = strategy_configs.get(phase_key, {})
    if not phase_config:
        self.logger.warning(f"Phase '{phase_key}' not found for '{firm_code}', using MFFU_Flex default")
        mffu_configs = self.firm_blueprints["MFFU_Flex"]["strategy_configs"]
        phase_config = mffu_configs.get(phase_key, {})

    self.logger.info(
        f"[DEBUG get_strategy_config] phase_config keys: {list(phase_config.keys())} if phase_config empty")

    config = phase_config.get(size_key)
    if not config:
        # For farming phase, try to fallback to 50k first before using MFFU_Flex fallback
        if phase_key == "farming" and size_key != "50k" and "50k" in phase_config:
            self.logger.info(
                f"Farming config not found for '{size_key}', using 50k farming config instead")
            config = phase_config["50k"]
        else:
            self.logger.warning(
                f"Config not found for '{firm_code}/{phase_key}/{size_key}', using MFFU_Flex fallback")
            config = self.firm_blueprints["MFFU_Flex"]["strategy_configs"]["challenge_trade1"]["50k"]
    else:
        self.logger.info(
            f"[DEBUG get_strategy_config] Found config: qty={config.get('tradovate_qty', 'N/A')}, vol={config.get('tradovate_vol', 'N/A')}")

    if config:
        config = config.copy()
        if 'mt5_volume' in config:
            config['mt5_volume'] = round(config['mt5_volume'], 2)

    return config

def validate_firm_setup(self, username: str) -> Tuple[Optional[str], Dict]:
    """Validate and get complete prop firm setup for a username."""
    firm_code = self.detect_prop_firm(username)
    if firm_code is None:
        self.logger.warning(f"Could not detect prop firm for '{username}'")
        return None, {}
    firm_info = self.get_firm_info(firm_code)

    self.logger.info(f"Prop firm setup for '{username}': {firm_info['name']} ({firm_code})")
    return firm_code, firm_info

def set_prop_firm(self, firm_name, broker=None, blueprint_firm=None):
    """Set the current prop firm

    Args:
        firm_name: The prop firm name (MFFU, TopStep, Other, etc.)
    """

```

```

        broker: The broker platform for 'Other' mode (TopStep or Tradovate)
        blueprint_firm: The specific blueprint to use when in Other mode (e.g. MFFU_Flex)
"""
self.logger.info(
    f"Setting prop firm: '{firm_name}', broker: '{broker}', blueprint: '{blueprint_firm}'")

firm_mapping = {
    "MFFU": "MFFU_Flex", # Legacy alias
    "My Funded Futures": "MFFU_Flex", # Legacy alias
    "MFFU_Flex": "MFFU_Flex",
    "Funded Next": "Funded Next",
    "FundedNext": "Funded Next",
    "FundingTicks": "FundingTicks",
    "TopStep": "TopStep",
    "Tradeify": "Tradeify",
    "Apex": "Apex",
    "Alpha Futures": "AlphaFutures",
    "AlphaFutures": "AlphaFutures",
    "AlphaFutures GC": "AlphaFutures GC",
    "Top One Futures": "Top One Futures",
    "Other": "MFFU_Flex" # Default fallback
}

# For "Other" prop firm, we prefer the specific blueprint if provided
if firm_name == "Other":
    if blueprint_firm and blueprint_firm in self.firm_blueprints:
        self.current_firm_code = blueprint_firm
        self.logger.info(f"Other mode: Using specific blueprint {self.current_firm_code}")
    elif broker and broker == "TopStep":
        self.current_firm_code = "TopStep"
    else: # Tradovate or any other
        self.current_firm_code = "MFFU_Flex"

    if not blueprint_firm:
        self.logger.info(f"Other mode: Mapped to {self.current_firm_code} based on broker '{broker}'")
else:
    self.current_firm_code = firm_mapping.get(firm_name, firm_name)

self.logger.info(f"Set prop firm to: {self.current_firm_code}")

def get_prop_firm_strategy_config(self, trading_phase, account_size="$50,000", balance_performance=0):
    """Get strategy config for current prop firm (manual trading only)"""
    self.logger.info(
        f"Looking up: '{self.current_firm_code}', phase='{trading_phase}', size='{account_size}'")

    firm_info = self.firm_blueprints.get(self.current_firm_code, self.firm_blueprints["MFFU_Flex"])

    if self.current_firm_code not in self.firm_blueprints:
        self.logger.warning(f"Blueprint '{self.current_firm_code}' not found! Using MFFU_Flex fallback")

    # Convert account size to size key (e.g., "$50,000" -> "50k", "$100,000" -> "100k", "$150,000" -> "150k")
    size_key = "50k" # Default
    if "$100,000" in account_size or "100k" in account_size.lower():
        size_key = "100k"
    elif "$150,000" in account_size or "150k" in account_size.lower():
        size_key = "150k"
    elif "$50,000" in account_size or "50k" in account_size.lower():
        size_key = "50k"

    self.logger.info(f"[DEBUG] Converted account_size '{account_size}' to size_key '{size_key}'")

```

```

# Default phase key
phase_key = "challenge_trade1"

if trading_phase == "Challenge Phase":
    if self.current_firm_code == "Top One Futures":
        if balance_performance >= 5.0:
            phase_key = "challenge_trade3"
        elif balance_performance >= 2.5:
            phase_key = "challenge_trade2"
        else:
            phase_key = "challenge_trade1"
    else:
        phase_key = "challenge_trade2" if balance_performance >= 2.5 else "challenge_trade1"

self.logger.info(f"[DEBUG] Using phase_key '{phase_key}' for phase '{trading_phase}'")

if trading_phase == "Funded Phase":
    if self.current_firm_code in ("MFFU", "MFFU_Flex"):
        # MFFU / MFFU_Flex Funded Phase Logic (Condensed Payouts)
        if balance_performance < 2.0:
            phase_key = "funded_trade1"
        elif balance_performance < 4.0:
            phase_key = "funded_trade2"
        elif balance_performance < 6.0:
            phase_key = "funded_trade3"
        else:
            phase_key = "funded_trade4"
    elif self.current_firm_code == "Trade Day":
        phase_key = "funded"
    elif self.current_firm_code == "TopStep":
        # TopStep Funded Phase Logic (Condensed Payouts)
        if balance_performance < 2.0:
            phase_key = "funded_trade1"
        elif balance_performance < 4.0:
            phase_key = "funded_trade2"
        elif balance_performance < 6.0:
            phase_key = "funded_trade3"
        else:
            phase_key = "funded_trade4"
    elif self.current_firm_code == "Top One Futures":
        # Top One Futures uses Payout 1 / Payout 2 selection instead of
        # a single "Funded Phase". This branch only runs as a safety
        # fallback if "Funded Phase" is somehow still selected; default
        # to funded_trade1 so behavior matches Payout 1.
        phase_key = "funded_trade1"
    elif self.current_firm_code == "FundingTicks":
        phase_key = "funded_trade1" # Default to trade 1
    elif self.current_firm_code == "Tradeify":
        phase_key = "funded_trade1"
    else:
        # Default for others
        phase_key = "funded_trade1"

if trading_phase == "Double Dip Phase":
    if self.current_firm_code in ("TopStep", "MFFU", "MFFU_Flex"):
        # Double Dip Phase Logic (shared by TopStep / MFFU / MFFU_Flex)
        if balance_performance < 2.0:
            phase_key = "funded_trade_doubledip_1"
        elif balance_performance < 4.0:
            phase_key = "funded_trade_doubledip_2"

```

```

        elif balance_performance < 6.0:
            phase_key = "funded_trade_doubledip_3"
        else:
            phase_key = "funded_trade_doubledip_4"
    elif self.current_firm_code == "Top One Futures":
        # Top One Futures Double Dip Phase Logic (same structure as funded)
        if balance_performance < 2.0:
            phase_key = "funded_trade_doubledip_1"
        elif balance_performance < 4.0:
            phase_key = "funded_trade_doubledip_2"
        elif balance_performance < 6.0:
            phase_key = "funded_trade_doubledip_3"
        else:
            phase_key = "funded_trade_doubledip_4"
    else:
        phase_key = "funded" # Fallback

elif trading_phase == "Double Dip Payout 1":
    # Top One Futures Double Dip uses the same structure as Payout 1
    phase_key = "funded_trade_doubledip_1"

elif trading_phase == "Double Dip Payout 2":
    # Top One Futures Double Dip uses the same structure as Payout 2
    phase_key = "funded_trade_doubledip_2"

elif trading_phase == "Payout 1":
    phase_key = "funded_trade1"

elif trading_phase == "Payout 2":
    phase_key = "funded_trade2"

elif trading_phase == "Payout 3":
    phase_key = "funded_trade3"

elif trading_phase == "Payout 4":
    phase_key = "funded_trade4"

elif trading_phase == "Farming Phase":
    phase_key = "farming"

elif trading_phase == "Farming (Consistency)":
    phase_key = "farming"

strategy_configs = firm_info.get("strategy_configs", {})
phase_configs = strategy_configs.get(phase_key, {})
config = phase_configs.get(size_key, {})

self.logger.info(
    f"[DEBUG] Retrieved config for {self.current_firm_code}/{phase_key}/{size_key}: {bool(config)}"
)
if config:
    self.logger.info(
        f"[DEBUG] Config qty={config.get('tradovate_qty', 'N/A')}, volume={config.get('mt5_volume', 'N/A')}"
    )

if phase_key == "farming" and self.current_firm_code == "Funded Next":
    # Strict logic: Funded Next farming MUST use 50k blueprint regardless of size
    if "50k" in phase_configs:
        self.logger.info(f"Enforcing Funded Next 50k farming blueprint for '{size_key}'")
        config = phase_configs["50k"]

if not config:

```

```

# For farming phase, try to fallback to 50k first before using MFFU fallback
if phase_key == "farming" and size_key != "50k" and "50k" in phase_configs:
    self.logger.info(
        f"Farming config not found for '{size_key}', using 50k farming config instead")
    config = phase_configs["50k"]

# Try to find a fallback within the same firm if possible
if trading_phase == "Funded Phase" and not config:
    # Try alternative funded keys
    for alt_key in ["funded", "funded_trade1"]:
        if alt_key in strategy_configs:
            config = strategy_configs[alt_key].get(size_key, {})
            if config:
                phase_key = alt_key
                break

if not config:
    mffu_configs = self.firm_blueprints["MFFU_Flex"]["strategy_configs"]
    # Map current phase_key to MFFU_Flex equivalent if possible
    mffu_phase_key = phase_key
    if "funded" in phase_key: mffu_phase_key = "funded_trade1"
    elif "farming" in phase_key: mffu_phase_key = "farming"
    else: mffu_phase_key = "challenge_trade1"

    fallback = mffu_configs.get(mffu_phase_key, {}).get(size_key, {})

    if fallback:
        self.logger.warning(
            f"No config for {self.current_firm_code}/{phase_key}, using MFFU_Flex {mffu_phase_key}")
        return fallback
    else:
        ultimate_fallback = {
            "tradovate_symbol": "MNQM6",
            "tradovate_qty": 2,
            "tradovate_tp_ticks": 154,
            "tradovate_sl_ticks": 400,
            "mt5_volume": 4.0,
            "mt5_tp_points": 98,
            "mt5_sl_points": 41
        }
        self.logger.error(f"No valid config, using ultimate fallback")
        return ultimate_fallback

return config

def get_prop_firm_account_sizes(self, firm_code=None):
    """Get account sizes for detected or specified prop firm"""
    if firm_code is None:
        firm_code = self.current_firm_code
    return self.get_account_sizes(firm_code)

def get_prop_firm_trading_phases(self, firm_code=None):
    """Get trading phases for detected or specified prop firm"""
    if firm_code is None:
        firm_code = self.current_firm_code
    return self.get_trading_phases(firm_code)

def format_volume_for_display(self, volume):
    """Format MT5 volume for display (used by GUI)"""
    if volume == 0:

```



```

        "challenge_trade4", "challenge_trade5"],
        "Funded": ["funded_trade1", "funded_trade2", "funded_trade3"],
    },
    "AlphaFutures": {
        "Challenge": ["challenge_trade1", "challenge_trade2"],
        "Funded": ["funded_trade1", "funded_trade2", "funded_trade3", "funded_trade4"],
        "Farming": ["farming"],
    },
    "AlphaFutures GC": {
        "Challenge": ["challenge_trade1", "challenge_trade2"],
        "Funded": ["funded_trade1", "funded_trade2", "funded_trade3", "funded_trade4"],
        "Farming": ["farming"],
    },
    "Tradeify": {
        "Challenge": ["challenge_trade1", "challenge_trade2", "challenge_trade3"],
        "Funded": ["funded_trade1", "funded_trade2", "funded_trade3", "funded_trade4"],
        "Farming": ["farming"],
    },
    "Apex": {
        "Challenge": ["challenge_trade1", "challenge_trade2"],
        "Funded": ["funded_trade1", "funded_trade2", "funded_trade3", "funded_trade4"],
        "Farming": ["farming"],
    },
    "Top One Futures": {
        "Challenge": ["challenge_trade1", "challenge_trade2", "challenge_trade3"],
        "Funded": ["funded_trade1", "funded_trade1_2", "funded_trade2", "funded_trade2_2",
                    "funded_trade3", "funded_trade3_2", "funded_trade4", "funded_trade4_2"],
        "Double Dip": ["funded_trade_doubledip_1", "funded_trade_doubledip_1_2",
                        "funded_trade_doubledip_2", "funded_trade_doubledip_2_2",
                        "funded_trade_doubledip_3", "funded_trade_doubledip_3_2",
                        "funded_trade_doubledip_4", "funded_trade_doubledip_4_2"],
    },
}

```

```

def predict_next_trade(self, firm_code: str, current_phase: str,
                       current_profit: float, size_key: str = "50k") -> Dict:
    """Predict current stage and next trade based on balance.

    Walks the ordered trade progression for the firm/phase, computing
    cumulative profit at each stage boundary. Returns a dict with:
        current_stage - human label like "Challenge Trade 2"
        next_stage    - human label for next trade, or "Phase Complete"
        next_config   - the blueprint config dict for the next trade (or None)
        next_phase_key - the blueprint key for the next trade
        balance_target - the cumulative $ target at end of current stage
        stages        - list of (label, phase_key, cumul_target) for all stages
    """
    # Normalize phase display to match _PHASE_TRADE_ORDER keys
    phase_map = {"Challenge": "Challenge", "Funded": "Funded",
                  "Farming": "Farming", "Double Dip": "Double Dip",
                  "Payout 1": "Funded", "Payout 2": "Funded",
                  "Payout 3": "Funded", "Payout 4": "Funded"}
    phase_key_group = phase_map.get(current_phase, current_phase)

    firm_orders = self._PHASE_TRADE_ORDER.get(firm_code, {})
    trade_keys = firm_orders.get(phase_key_group, [])

    if not trade_keys:
        return {"current_stage": current_phase, "next_stage": "-",
                "next_config": None, "next_phase_key": None,

```

```

        "balance_target": 0, "stages": []}

# Build cumulative profit milestones for each stage
stages = []
cumulative = 0.0
for key in trade_keys:
    cfg = self.get_strategy_config(firm_code, key, size_key)
    if not cfg:
        continue
    sym = cfg.get("tradovate_symbol", "") or cfg.get("topstepx_symbol", "")
    qty = int(cfg.get("tradovate_qty", 0) or cfg.get("topstepx_qty", 0))
    tp = int(cfg.get("tradovate_tp_ticks", 0) or cfg.get("topstepx_tp_ticks", 0))
    tick_val = self.get_tick_value(sym)
    stage_profit = qty * tp * tick_val
    cumulative += stage_profit
    label = key.replace("_", " ").replace("trade", "Trade ").title()
    # Clean up label
    label = label.replace("Challenge Trade ", "Challenge #") \
        .replace("Funded Trade ", "Funded #") \
        .replace("Funded ", "Funded #") if "trade" in key.lower() else label
    label = key.replace("_", " ").title()
    stages.append((label, key, round(cumulative, 2), round(stage_profit, 2)))

# Find which stage we're currently in based on profit
current_idx = 0
for i, (_, _, cumul_target, _) in enumerate(stages):
    if current_profit < cumul_target - 0.01:
        current_idx = i
        break
else:
    # Profit exceeds all stages – we're past the last one
    current_idx = len(stages) - 1

current_label, current_key, current_target, current_stage_profit = stages[current_idx]

# Next trade
if current_idx + 1 < len(stages):
    next_label, next_key, next_target, next_profit = stages[current_idx + 1]
    next_cfg = self.get_strategy_config(firm_code, next_key, size_key)
else:
    next_label = "Phase Complete"
    next_key = None
    next_cfg = None
    next_target = current_target

return {
    "current_stage": current_label,
    "current_phase_key": current_key,
    "current_target": current_target,
    "current_stage_profit": current_stage_profit,
    "next_stage": next_label,
    "next_config": next_cfg,
    "next_phase_key": next_key,
    "next_target": next_target,
    "balance_target": current_target,
    "stages": stages,
}

def get_stage_start_target(self, firm_code: str, current_phase: str,
                           phase_key: str, size_key: str = "50k") -> float:

```

```

"""Return the cumulative $ profit expected BEFORE this stage begins.

E.g. if the phase order is [trade1, trade2, trade3] and each has a
$1000 TP target, then:
    trade1 start = $0
    trade2 start = $1000
    trade3 start = $2000

This is the profit the account should already have when entering
this stage. Used to adjust TP so the trade doesn't overshoot or
undershoot the stage target.
"""
phase_map = {"Challenge": "Challenge", "Funded": "Funded",
             "Farming": "Farming", "Double Dip": "Double Dip",
             "Payout 1": "Funded", "Payout 2": "Funded",
             "Payout 3": "Funded", "Payout 4": "Funded"}
phase_group = phase_map.get(current_phase, current_phase)

firm_orders = self._PHASE_TRADE_ORDER.get(firm_code, {})
trade_keys = firm_orders.get(phase_group, [])

cumulative = 0.0
for key in trade_keys:
    if key == phase_key:
        return round(cumulative, 2)
    cfg = self.get_strategy_config(firm_code, key, size_key)
    if not cfg:
        continue
    sym = cfg.get("tradovate_symbol", "") or cfg.get("topstepx_symbol", "")
    qty = int(cfg.get("tradovate_qty", 0) or cfg.get("topstepx_qty", 0))
    tp = int(cfg.get("tradovate_tp_ticks", 0) or cfg.get("topstepx_tp_ticks", 0))
    tick_val = self.get_tick_value(sym)
    cumulative += qty * tp * tick_val
return round(cumulative, 2)

def adjust_tp_sl_for_balance(self, config: Dict, current_profit: float) -> Dict:
    """Adjust TP and SL ticks/points based on current account profit.

    If the account already has profit, TP is reduced so the trade doesn't
    overshoot the blueprint target. If the account has a loss, TP is
    increased to recover back to the target. SL is adjusted inversely.

    Returns a NEW config dict with adjusted values (original is not mutated).
    """
    config = config.copy()
    symbol = config.get("tradovate_symbol", "") or config.get("topstepx_symbol", "")
    qty = int(config.get("tradovate_qty", 0) or config.get("topstepx_qty", 0))
    orig_tp = int(config.get("tradovate_tp_ticks", 0) or config.get("topstepx_tp_ticks", 0))
    orig_sl = int(config.get("tradovate_sl_ticks", 0) or config.get("topstepx_sl_ticks", 0))
    mt5_tp = int(config.get("mt5_tp_points", 0))
    mt5_sl = int(config.get("mt5_sl_points", 0))

    if qty <= 0 or orig_tp <= 0:
        return config

    tick_val = self.get_tick_value(symbol)
    # Blueprint dollar target for the Tradovate trade
    target_profit = qty * orig_tp * tick_val
    # Blueprint dollar risk for the Tradovate trade
    target_loss = qty * orig_sl * tick_val

```



```

        hard_stop = 0.0
    else:
        hard_stop = 50100.0
    else:
        hard_stop = self._HARD_STOP_THRESHOLDS.get(firm_code, 50000.0)

    if current_balance <= hard_stop:
        return "Fail"

    return "In Progress"

# NQ tick-to-MT5-point conversion ratio (1 MT5 point = 4 Tradovate ticks)
_NQ_TICK_TO_POINT_RATIO = 4
# Safety buffer subtracted from safe distance (MT5 points)
_FARMING_BUFFER_POINTS = 4

def adjust_farming_tp_sl(self, config: Dict, current_balance: float,
                        firm_code: str) -> Dict:
    """Adjust MT5 TP for farming trades based on hard-stop proximity.

    Farming trades use micro contracts (MNQM6). If the MT5 TP is so
    large that the prop account would breach the hard-stop threshold
    before MT5 closes, we cap MT5 TP to a safe distance.

    Only MT5 TP is adjusted – Tradovate TP/SL and MT5 SL stay unchanged.

    Returns a NEW config dict (original is not mutated).
    """
    config = config.copy()
    symbol = config.get("tradovate_symbol", "") or config.get("topstepx_symbol", "")

    # Only applies to micro contracts (farming)
    if "MNQ" not in symbol.upper():
        return config

    mt5_tp = float(config.get("mt5_tp_points", 0))
    if mt5_tp <= 0:
        return config

    # Determine hard-stop threshold
    if firm_code in ("MFFU", "MFFU_Flex", "My Funded Futures"):
        # MFFU zero-start vs traditional detection:
        # If balance < $50,100 it can't be a traditional $50k account
        # (would already be breached), so it must be zero-start.
        if current_balance < 50100.0:
            hard_stop = 0.0
        else:
            hard_stop = 50100.0
    else:
        hard_stop = self._HARD_STOP_THRESHOLDS.get(firm_code, 50000.0)

    distance_to_max_loss = current_balance - hard_stop
    if distance_to_max_loss <= 0:
        self.logger.warning(
            f"[Farming TP] Balance ${current_balance:,.2f} already at/below "
            f"hard stop ${hard_stop:,.2f} for {firm_code}")
        return config

    tick_val = self.get_tick_value(symbol)
    trado_qty = int(config.get("tradovate_qty", 1) or config.get("topstepx_qty", 1))

```


[illegible]


```

str]]:
    """Check whether the selected blueprint matches the connected account.

    Args:
        account_number: Account identifier from Tradovate/TopStep.
        selected_blueprint: The blueprint currently selected in the UI.

    Returns:
        (is_match, detected_blueprint)
        - is_match is True when they are compatible.
        - detected_blueprint is the blueprint we think the account belongs to.
    """
    detected = self.detect_firm_from_account(account_number)
    if detected is None:
        # Detection inconclusive – allow but warn
        return True, None

    # Treat MFFU and MFFU_Flex as compatible
    compatible_groups = [
        {"MFFU", "MFFU_Flex"},
        {"Alpha Futures", "AlphaFutures GC"},
    ]
    for group in compatible_groups:
        if detected in group and selected_blueprint in group:
            return True, detected

    is_match = (detected == selected_blueprint)
    if not is_match:
        self.logger.warning(
            f"Blueprint MISMATCH: account '{account_number}' belongs to '{detected}' "
            f"but '{selected_blueprint}' is selected"
        )
    return is_match, detected

```

Method: PropFirmManager.__init__

```
def __init__(self)
```

Why these Python concepts were used

- `__init__` — The constructor. Runs after Python has allocated the instance; assigns starting attribute values to self.

Step by step (top-level body)

1. Assigns `self.logger = logging.getLogger(__name__)`.
2. Assigns `self.current_firm_code = 'MFFU_Flex'`.
3. Declares `self._account_direction_locks: Dict[str, Dict] = {}`.
4. Assigns `self.firm_blueprints = {'MFFU_Flex': {'name': 'MFFU_Flex', 'account_sizes': ['$5....`

Code (copy-paste safe)

```

def __init__(self):
    self.logger = logging.getLogger(__name__)
    self.current_firm_code = "MFFU_Flex" # Default prop firm

    # Per-account daily direction locks: {account_number: {"date": date, "direction": "BUY"/"SELL"}}
    self._account_direction_locks: Dict[str, Dict] = {}

```

```

# Prop firm blueprints - $50k account configurations only core challenge with the flex addon
self.firm_blueprints = {
    "MFFU_Flex": {
        "name": "MFFU_Flex",
        "account_sizes": ["$50,000"],
        "trading_phases": ["Challenge Phase", "Funded Phase", "Double Dip Phase", "Farming Phase"],
        "strategy_configs": {
            "challenge_trade1": {
                "50k": {
                    "tradovate_symbol": "NQM6",
                    "tradovate_qty": 2,
                    "tradovate_tp_ticks": 151,
                    "tradovate_sl_ticks": 201,
                    "mt5_volume": 4.6,
                    "mt5_tp_points": 46,
                    "mt5_sl_points": 42
                }
            },
            "challenge_trade2": {
                "50k": {
                    "tradovate_symbol": "NQM6",
                    "tradovate_qty": 2,
                    "tradovate_tp_ticks": 151,
                    "tradovate_sl_ticks": 201,
                    "mt5_volume": 8.4,
                    "mt5_tp_points": 46,
                    "mt5_sl_points": 42
                }
            },
            "funded_trade1": {
                "50k": {
                    "tradovate_symbol": "NQM6",
                    "tradovate_qty": 2,
                    "tradovate_tp_ticks": 540,
                    "tradovate_sl_ticks": 201,
                    "mt5_volume": 16,
                    "mt5_tp_points": 46,
                    "mt5_sl_points": 139
                }
            },
            "funded_trade2": {
                "50k": {
                    "tradovate_symbol": "NQM6",
                    "tradovate_qty": 2,
                    "tradovate_tp_ticks": 240,
                    "tradovate_sl_ticks": 261,
                    "mt5_volume": 20,
                    "mt5_tp_points": 61,
                    "mt5_sl_points": 64
                }
            },
            "funded_trade3": {
                "50k": {
                    "tradovate_symbol": "NQM6",
                    "tradovate_qty": 2,
                    "tradovate_tp_ticks": 240,
                    "tradovate_sl_ticks": 245,
                    "mt5_volume": 18.0,
                    "mt5_tp_points": 57,
                    "mt5_sl_points": 64
                }
            }
        }
    }
}

```

```

    }
  },
  "funded_trade4": {
    "50k": {
      "tradovate_symbol": "NQM6",
      "tradovate_qty": 2,
      "tradovate_tp_ticks": 240,
      "tradovate_sl_ticks": 238,
      "mt5_volume": 13.6,
      "mt5_tp_points": 55,
      "mt5_sl_points": 64
    }
  },
  "funded_trade_doubledip_1": {
    "50k": {
      "tradovate_symbol": "NQM6",
      "tradovate_qty": 2,
      "tradovate_tp_ticks": 540,
      "tradovate_sl_ticks": 201,
      "mt5_volume": 13,
      "mt5_tp_points": 46,
      "mt5_sl_points": 139
    }
  },
  "funded_trade_doubledip_2": {
    "50k": {
      "tradovate_symbol": "NQM6",
      "tradovate_qty": 2,
      "tradovate_tp_ticks": 240,
      "tradovate_sl_ticks": 261,
      "mt5_volume": 14,
      "mt5_tp_points": 61,
      "mt5_sl_points": 64
    }
  },
  "funded_trade_doubledip_3": {
    "50k": {
      "tradovate_symbol": "NQM6",
      "tradovate_qty": 2,
      "tradovate_tp_ticks": 240,
      "tradovate_sl_ticks": 245,
      "mt5_volume": 9,
      "mt5_tp_points": 57,
      "mt5_sl_points": 64
    }
  },
  "funded_trade_doubledip_4": {
    "50k": {
      "tradovate_symbol": "NQM6",
      "tradovate_qty": 2,
      "tradovate_tp_ticks": 240,
      "tradovate_sl_ticks": 238,
      "mt5_volume": 20,
      "mt5_tp_points": 55,
      "mt5_sl_points": 64
    }
  },
  "farming": {
    "50k": {
      "tradovate_symbol": "MNQM6",
      "tradovate_qty": 2,
      "tradovate_tp_ticks": 154,
      "tradovate_sl_ticks": 600,
      "mt5_volume": 3.2,

```

```

        "mt5_tp_points": 146,
        "mt5_sl_points": 43
    }
}
},
"Funded Next": {
    "name": "Funded Next",
    "account_sizes": ["$50,000"],
    "trading_phases": ["Challenge Phase", "Payout 1", "Payout 2", "Payout 3", "Payout 4",
        "Farming Phase"],
    "strategy_configs": {
        "challenge_trade1": {
            "50k": {
                "tradovate_symbol": "NQM6",
                "tradovate_qty": 2,
                "tradovate_tp_ticks": 102,
                "tradovate_sl_ticks": 200,
                "mt5_volume": 3.6,
                "mt5_tp_points": 46,
                "mt5_sl_points": 30
            }
        },
        "challenge_trade2": {
            "50k": {
                "tradovate_symbol": "NQM6",
                "tradovate_qty": 2,
                "tradovate_tp_ticks": 102,
                "tradovate_sl_ticks": 200,
                "mt5_volume": 6.2,
                "mt5_tp_points": 46,
                "mt5_sl_points": 30
            }
        },
        "challenge_trade3": {
            "50k": {
                "tradovate_symbol": "NQM6",
                "tradovate_qty": 2,
                "tradovate_tp_ticks": 101,
                "tradovate_sl_ticks": 200,
                "mt5_volume": 10.4,
                "mt5_tp_points": 46,
                "mt5_sl_points": 30
            }
        },
        "funded_trade1": {
            "50k": {
                "tradovate_symbol": "NQM6",
                "tradovate_qty": 2,
                "tradovate_tp_ticks": 520,
                "tradovate_sl_ticks": 200,
                "mt5_volume": 18.4,
                "mt5_tp_points": 46,
                "mt5_sl_points": 134
            }
        },
        "funded_trade2": {
            "50k": {
                "tradovate_symbol": "NQM6",
                "tradovate_qty": 2,

```

```

        "tradovate_tp_ticks": 220,
        "tradovate_sl_ticks": 260,
        "mt5_volume": 17,
        "mt5_tp_points": 61,
        "mt5_sl_points": 59
    }
},
"funded_trade3": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 220,
        "tradovate_sl_ticks": 245,
        "mt5_volume": 3,
        "mt5_tp_points": 57,
        "mt5_sl_points": 59
    }
},
"funded_trade4": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 220,
        "tradovate_sl_ticks": 238,
        "mt5_volume": 0,
        "mt5_tp_points": 55,
        "mt5_sl_points": 59
    }
},
"farming": {
    "50k": {
        "tradovate_symbol": "MNQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 204,
        "tradovate_sl_ticks": 600,
        "mt5_volume": 3.2,
        "mt5_tp_points": 146,
        "mt5_sl_points": 55
    }
}
},
"FundingTicks": {
    "name": "FundingTicks",
    "account_sizes": ["$50,000"],
    "trading_phases": ["Challenge Phase", "Funded Phase", "Farming Phase"],
    "strategy_configs": {
        "challenge_trade1": {
            "50k": {
                "tradovate_symbol": "NQM6",
                "tradovate_qty": 2,
                "tradovate_tp_ticks": 127,
                "tradovate_sl_ticks": 250,
                "mt5_volume": 5.2,
                "mt5_tp_points": 58,
                "mt5_sl_points": 36
            }
        },
        "challenge_trade2": {
            "50k": {

```

```

        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 127,
        "tradovate_sl_ticks": 250,
        "mt5_volume": 7.8,
        "mt5_tp_points": 58,
        "mt5_sl_points": 36
    }
},
"funded_trade1": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 500,
        "tradovate_sl_ticks": 250,
        "mt5_volume": 13.8,
        "mt5_tp_points": 58,
        "mt5_sl_points": 129
    }
},
"funded_trade2": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 150,
        "tradovate_sl_ticks": 250,
        "mt5_volume": 18,
        "mt5_tp_points": 58,
        "mt5_sl_points": 42
    }
},
"funded_trade3": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 150,
        "tradovate_sl_ticks": 250,
        "mt5_volume": 8.6,
        "mt5_tp_points": 58,
        "mt5_sl_points": 42
    }
},
"funded_trade4": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 150,
        "tradovate_sl_ticks": 250,
        "mt5_volume": 0,
        "mt5_tp_points": 58,
        "mt5_sl_points": 42
    }
},
"farming": {
    "50k": {
        "tradovate_symbol": "MNQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 204,
        "tradovate_sl_ticks": 600,
        "mt5_volume": 1.6,

```

```

        "mt5_tp_points": 146,
        "mt5_sl_points": 55
    }
}
},
"TopStep": {
    "name": "TopStep",
    "account_sizes": ["$50,000"],
    "trading_phases": ["Challenge Phase", "Funded Phase", "Double Dip Phase", "Farming Phase"],
    "strategy_configs": {
        "challenge_trade1": {
            "50k": {
                "topstepx_symbol": "NQM26",
                "topstepx_qty": 2,
                "topstepx_tp_ticks": 151,
                "topstepx_sl_ticks": 201,
                "mt5_volume": 4.6,
                "mt5_tp_points": 46,
                "mt5_sl_points": 42
            }
        },
        "challenge_trade2": {
            "50k": {
                "topstepx_symbol": "NQM26",
                "topstepx_qty": 2,
                "topstepx_tp_ticks": 151,
                "topstepx_sl_ticks": 201,
                "mt5_volume": 8.4,
                "mt5_tp_points": 46,
                "mt5_sl_points": 42
            }
        },
        "funded_trade1": {
            "50k": {
                "topstepx_symbol": "NQM26",
                "topstepx_qty": 2,
                "topstepx_tp_ticks": 340,
                "topstepx_sl_ticks": 201,
                "mt5_volume": 16,
                "mt5_tp_points": 46,
                "mt5_sl_points": 89
            }
        },
        "funded_trade2": {
            "50k": {
                "topstepx_symbol": "NQM26",
                "topstepx_qty": 2,
                "topstepx_tp_ticks": 140,
                "topstepx_sl_ticks": 161,
                "mt5_volume": 20,
                "mt5_tp_points": 36,
                "mt5_sl_points": 39
            }
        },
        "funded_trade3": {
            "50k": {
                "topstepx_symbol": "NQM26",
                "topstepx_qty": 2,
                "topstepx_tp_ticks": 140,

```

```

        "topstepx_sl_ticks": 146,
        "mt5_volume": 18.0,
        "mt5_tp_points": 32,
        "mt5_sl_points": 39
    }
},
"funded_trade4": {
    "50k": {
        "topstepx_symbol": "NQM26",
        "topstepx_qty": 2,
        "topstepx_tp_ticks": 140,
        "topstepx_sl_ticks": 139,
        "mt5_volume": 13.6,
        "mt5_tp_points": 30,
        "mt5_sl_points": 39
    }
}, "funded_trade_doubledip_1": {
    "50k": {
        "topstepx_symbol": "NQM26",
        "topstepx_qty": 2,
        "topstepx_tp_ticks": 340,
        "topstepx_sl_ticks": 201,
        "mt5_volume": 16,
        "mt5_tp_points": 46,
        "mt5_sl_points": 89
    }
}, "funded_trade_doubledip_2": {
    "50k": {
        "topstepx_symbol": "NQM26",
        "topstepx_qty": 2,
        "topstepx_tp_ticks": 140,
        "topstepx_sl_ticks": 161,
        "mt5_volume": 20,
        "mt5_tp_points": 36,
        "mt5_sl_points": 39
    }
}, "funded_trade_doubledip_3": {
    "50k": {
        "topstepx_symbol": "NQM26",
        "topstepx_qty": 2,
        "topstepx_tp_ticks": 140,
        "topstepx_sl_ticks": 146,
        "mt5_volume": 9,
        "mt5_tp_points": 32,
        "mt5_sl_points": 39
    }
}, "funded_trade_doubledip_4": {
    "50k": {
        "topstepx_symbol": "NQM26",
        "topstepx_qty": 2,
        "topstepx_tp_ticks": 140,
        "topstepx_sl_ticks": 139,
        "mt5_volume": 20,
        "mt5_tp_points": 30,
        "mt5_sl_points": 39
    }
},
"farming": {
    "50k": {
        "topstepx_symbol": "MNQM26",

```



```

        "topstepx_qty": 2,
        "topstepx_tp_ticks": 154,
        "topstepx_sl_ticks": 600,
        "mt5_volume": 3.6,
        "mt5_tp_points": 146,
        "mt5_sl_points": 43
    }
}
},
"Lucid": {
    "name": "Lucid",
    "account_sizes": ["$50,000"],
    "trading_phases": ["Challenge Phase", "Funded Phase", "Double Dip Phase", "Farming Phase"],
    "strategy_configs": {
        "challenge_trade1": {
            "50k": {
                "tradovate_symbol": "NQM6",
                "tradovate_qty": 2,
                "tradovate_tp_ticks": 151,
                "tradovate_sl_ticks": 200,
                "mt5_volume": 5,
                "mt5_tp_points": 46,
                "mt5_sl_points": 42
            }
        },
        "challenge_trade2": {
            "50k": {
                "tradovate_symbol": "NQM6",
                "tradovate_qty": 2,
                "tradovate_tp_ticks": 151,
                "tradovate_sl_ticks": 200,
                "mt5_volume": 8,
                "mt5_tp_points": 46,
                "mt5_sl_points": 42
            }
        },
        "funded_trade1": {
            "50k": {
                "tradovate_symbol": "NQM6",
                "tradovate_qty": 2,
                "tradovate_tp_ticks": 340,
                "tradovate_sl_ticks": 200,
                "mt5_volume": 15,
                "mt5_tp_points": 46,
                "mt5_sl_points": 89
            }
        },
        "funded_trade2": {
            "50k": {
                "tradovate_symbol": "NQM6",
                "tradovate_qty": 2,
                "tradovate_tp_ticks": 140,
                "tradovate_sl_ticks": 170,
                "mt5_volume": 18,
                "mt5_tp_points": 38,
                "mt5_sl_points": 39
            }
        },
        "funded_trade3": {

```

```

        "50k": {
            "tradovate_symbol": "NQM6",
            "tradovate_qty": 2,
            "tradovate_tp_ticks": 140,
            "tradovate_sl_ticks": 160,
            "mt5_volume": 18.0,
            "mt5_tp_points": 36,
            "mt5_sl_points": 39
        }
    },
    "funded_trade4": {
        "50k": {
            "tradovate_symbol": "NQM6",
            "tradovate_qty": 2,
            "tradovate_tp_ticks": 140,
            "tradovate_sl_ticks": 155,
            "mt5_volume": 0,
            "mt5_tp_points": 34,
            "mt5_sl_points": 39
        }
    },
    "farming": {
        "50k": {
            "tradovate_symbol": "MNQM6",
            "tradovate_qty": 2,
            "tradovate_tp_ticks": 154,
            "tradovate_sl_ticks": 600,
            "mt5_volume": 3.6,
            "mt5_tp_points": 146,
            "mt5_sl_points": 43
        }
    }
},
"TradeDay": {
    "name": "TradeDay",
    "account_sizes": ["$50,000"],
    "trading_phases": ["Challenge Phase", "Funded Phase", "Double Dip Phase", "Farming Phase"],
    "strategy_configs": {
        "challenge_trade1": {
            "50k": {
                "tradovate_symbol": "NQM6",
                "tradovate_qty": 2,
                "tradovate_tp_ticks": 62,
                "tradovate_sl_ticks": 200,
                "mt5_volume": 3.6,
                "mt5_tp_points": 46,
                "mt5_sl_points": 20
            }
        },
        "challenge_trade2": {
            "50k": {
                "tradovate_symbol": "NQM6",
                "tradovate_qty": 2,
                "tradovate_tp_ticks": 62,
                "tradovate_sl_ticks": 200,
                "mt5_volume": 5.2,
                "mt5_tp_points": 46,
                "mt5_sl_points": 20
            }
        }
    }
}

```

```

},
"challenge_trade3": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 62,
        "tradovate_sl_ticks": 200,
        "mt5_volume": 7.4,
        "mt5_tp_points": 46,
        "mt5_sl_points": 20
    }
},
"challenge_trade4": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 62,
        "tradovate_sl_ticks": 200,
        "mt5_volume": 10.2,
        "mt5_tp_points": 46,
        "mt5_sl_points": 20
    }
},
"challenge_trade5": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 62,
        "tradovate_sl_ticks": 200,
        "mt5_volume": 14.2,
        "mt5_tp_points": 46,
        "mt5_sl_points": 20
    }
},
"funded_trade1": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 500,
        "tradovate_sl_ticks": 200,
        "mt5_volume": 20,
        "mt5_tp_points": 46,
        "mt5_sl_points": 129
    }
},
"funded_trade2": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 500,
        "tradovate_sl_ticks": 200,
        "mt5_volume": 28,
        "mt5_tp_points": 46,
        "mt5_sl_points": 129
    }
},
"funded_trade3": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,

```

```

        "tradovate_tp_ticks": 500,
        "tradovate_sl_ticks": 200,
        "mt5_volume": 20,
        "mt5_tp_points": 46,
        "mt5_sl_points": 129
    }
}
},
"AlphaFutures": {
    "name": "AlphaFutures",
    "account_sizes": ["$50,000", "$100,000", "$150,000"],
    "trading_phases": ["Challenge Phase", "Funded Phase", "Farming Phase"],
    "strategy_configs": {
        "challenge_trade1": {
            "50k": {
                "tradovate_symbol": "NQM6",
                "tradovate_qty": 2,
                "tradovate_tp_ticks": 202,
                "tradovate_sl_ticks": 175,
                "mt5_volume": 3,
                "mt5_tp_points": 39,
                "mt5_sl_points": 55
            },
            "100k": {
                "tradovate_symbol": "NQM6",
                "tradovate_qty": 4,
                "tradovate_tp_ticks": 202,
                "tradovate_sl_ticks": 175,
                "mt5_volume": 6,
                "mt5_tp_points": 39,
                "mt5_sl_points": 55
            },
            "150k": {
                "tradovate_symbol": "NQM6",
                "tradovate_qty": 6,
                "tradovate_tp_ticks": 202,
                "tradovate_sl_ticks": 175,
                "mt5_volume": 9,
                "mt5_tp_points": 39,
                "mt5_sl_points": 55
            }
        },
        "challenge_trade2": {
            "50k": {
                "tradovate_symbol": "NQM6",
                "tradovate_qty": 2,
                "tradovate_tp_ticks": 202,
                "tradovate_sl_ticks": 175,
                "mt5_volume": 7.4,
                "mt5_tp_points": 39,
                "mt5_sl_points": 55
            },
            "100k": {
                "tradovate_symbol": "NQM6",
                "tradovate_qty": 4,
                "tradovate_tp_ticks": 202,
                "tradovate_sl_ticks": 175,
                "mt5_volume": 14.8,
                "mt5_tp_points": 39,
            }
        }
    }
}

```

```

        "mt5_sl_points": 55
    },
    "150k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 6,
        "tradovate_tp_ticks": 202,
        "tradovate_sl_ticks": 175,
        "mt5_volume": 22.2,
        "mt5_tp_points": 39,
        "mt5_sl_points": 55
    },
    },
    "funded_trade1": {
        "50k": {
            "tradovate_symbol": "NQM6",
            "tradovate_qty": 2,
            "tradovate_tp_ticks": 600,
            "tradovate_sl_ticks": 175,
            "mt5_volume": 22,
            "mt5_tp_points": 39,
            "mt5_sl_points": 154
        },
        "100k": {
            "tradovate_symbol": "NQM6",
            "tradovate_qty": 4,
            "tradovate_tp_ticks": 600,
            "tradovate_sl_ticks": 175,
            "mt5_volume": 44,
            "mt5_tp_points": 39,
            "mt5_sl_points": 154
        },
        "150k": {
            "tradovate_symbol": "NQM6",
            "tradovate_qty": 6,
            "tradovate_tp_ticks": 600,
            "tradovate_sl_ticks": 175,
            "mt5_volume": 66,
            "mt5_tp_points": 39,
            "mt5_sl_points": 154
        }
    },
    "funded_trade2": {
        "50k": {
            "tradovate_symbol": "NQM6",
            "tradovate_qty": 2,
            "tradovate_tp_ticks": 100,
            "tradovate_sl_ticks": 175,
            "mt5_volume": 40,
            "mt5_tp_points": 39,
            "mt5_sl_points": 29
        },
        "100k": {
            "tradovate_symbol": "NQM6",
            "tradovate_qty": 4,
            "tradovate_tp_ticks": 100,
            "tradovate_sl_ticks": 175,
            "mt5_volume": 80,
            "mt5_tp_points": 39,
            "mt5_sl_points": 29
        }
    },
    },

```

```

        "150k": {
            "tradovate_symbol": "NQM6",
            "tradovate_qty": 6,
            "tradovate_tp_ticks": 100,
            "tradovate_sl_ticks": 175,
            "mt5_volume": 120,
            "mt5_tp_points": 39,
            "mt5_sl_points": 29
        }
    },
    "funded_trade3": {
        "50k": {
            "tradovate_symbol": "NQM6",
            "tradovate_qty": 2,
            "tradovate_tp_ticks": 200,
            "tradovate_sl_ticks": 175,
            "mt5_volume": 25,
            "mt5_tp_points": 39,
            "mt5_sl_points": 54
        },
        "100k": {
            "tradovate_symbol": "NQM6",
            "tradovate_qty": 4,
            "tradovate_tp_ticks": 200,
            "tradovate_sl_ticks": 175,
            "mt5_volume": 50,
            "mt5_tp_points": 39,
            "mt5_sl_points": 54
        },
        "150k": {
            "tradovate_symbol": "NQM6",
            "tradovate_qty": 6,
            "tradovate_tp_ticks": 200,
            "tradovate_sl_ticks": 175,
            "mt5_volume": 75,
            "mt5_tp_points": 39,
            "mt5_sl_points": 54
        }
    },
    "funded_trade4": {
        "50k": {
            "tradovate_symbol": "NQM6",
            "tradovate_qty": 2,
            "tradovate_tp_ticks": 200,
            "tradovate_sl_ticks": 175,
            "mt5_volume": 15,
            "mt5_tp_points": 39,
            "mt5_sl_points": 54
        },
        "100k": {
            "tradovate_symbol": "NQM6",
            "tradovate_qty": 4,
            "tradovate_tp_ticks": 200,
            "tradovate_sl_ticks": 175,
            "mt5_volume": 30,
            "mt5_tp_points": 39,
            "mt5_sl_points": 54
        },
        "150k": {
            "tradovate_symbol": "NQM6",

```

```

        "tradovate_qty": 6,
        "tradovate_tp_ticks": 200,
        "tradovate_sl_ticks": 175,
        "mt5_volume": 45,
        "mt5_tp_points": 39,
        "mt5_sl_points": 54
    }
},
"farming": {
    "50k": {
        "tradovate_symbol": "MNQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 204,
        "tradovate_sl_ticks": 600,
        "mt5_volume": 3.6,
        "mt5_tp_points": 146,
        "mt5_sl_points": 55
    }
}
},
"AlphaFutures GC": {
    "name": "AlphaFutures GC",
    "account_sizes": ["$50,000"],
    "trading_phases": ["Challenge Phase", "Funded Phase", "Farming Phase"],
    "strategy_configs": {
        "challenge_trade1": {
            "50k": {
                "tradovate_symbol": "GCM6",
                "tradovate_qty": 1,
                "tradovate_tp_ticks": 201,
                "tradovate_sl_ticks": 175,
                "mt5_symbol": "XAUUSD",
                "mt5_volume": 0.09,
                "mt5_tp_points": 1710,
                "mt5_sl_points": 2050
            }
        },
        "challenge_trade2": {
            "50k": {
                "tradovate_symbol": "GCM6",
                "tradovate_qty": 1,
                "tradovate_tp_ticks": 201,
                "tradovate_sl_ticks": 175,
                "mt5_symbol": "XAUUSD",
                "mt5_volume": 0.2,
                "mt5_tp_points": 1710,
                "mt5_sl_points": 2050
            }
        },
        "funded_trade1": {
            "50k": {
                "tradovate_symbol": "GCM6",
                "tradovate_qty": 2,
                "tradovate_tp_ticks": 300,
                "tradovate_sl_ticks": 88,
                "mt5_symbol": "XAUUSD",
                "mt5_volume": 0.49,
                "mt5_tp_points": 1710,
                "mt5_sl_points": 6040
            }
        }
    }
}

```

```

    }
},
"funded_trade2": {
    "50k": {
        "tradovate_symbol": "GCM6",
        "tradovate_qty": 1,
        "tradovate_tp_ticks": 100,
        "tradovate_sl_ticks": 175,
        "mt5_symbol": "XAUUSD",
        "mt5_volume": 0.75,
        "mt5_tp_points": 1710,
        "mt5_sl_points": 1040
    }
},
"funded_trade3": {
    "50k": {
        "tradovate_symbol": "GCM6",
        "tradovate_qty": 1,
        "tradovate_tp_ticks": 200,
        "tradovate_sl_ticks": 175,
        "mt5_symbol": "XAUUSD",
        "mt5_volume": 0.24,
        "mt5_tp_points": 1710,
        "mt5_sl_points": 2040
    }
},
"funded_trade4": {
    "50k": {
        "tradovate_symbol": "GCM6",
        "tradovate_qty": 1,
        "tradovate_tp_ticks": 200,
        "tradovate_sl_ticks": 175,
        "mt5_symbol": "XAUUSD",
        "mt5_volume": 0,
        "mt5_tp_points": 1710,
        "mt5_sl_points": 2040
    }
},
"farming": {
    "50k": {
        "tradovate_symbol": "MGCM6",
        "tradovate_qty": 1,
        "tradovate_tp_ticks": 204,
        "tradovate_sl_ticks": 600,
        "mt5_symbol": "XAUUSD",
        "mt5_volume": 0.9,
        "mt5_tp_points": 596,
        "mt5_sl_points": 208
    }
}
},
"Tradeify": {
    "name": "Tradeify",
    "account_sizes": ["$50,000"],
    "trading_phases": ["Challenge Phase", "Payout 1", "Payout 2", "Payout 3", "Payout 4",
        "Farming (Consistency)"],
    "strategy_configs": {
        "challenge_trade1": {
            "50k": {

```



```

        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 102,
        "tradovate_sl_ticks": 200,
        "mt5_volume": 3,
        "mt5_tp_points": 46,
        "mt5_sl_points": 30
    }
},
"challenge_trade2": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 102,
        "tradovate_sl_ticks": 200,
        "mt5_volume": 5.2,
        "mt5_tp_points": 46,
        "mt5_sl_points": 30
    }
}, "challenge_trade3": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 102,
        "tradovate_sl_ticks": 200,
        "mt5_volume": 8.8,
        "mt5_tp_points": 46,
        "mt5_sl_points": 30
    }
},
"funded_trade1": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 540,
        "tradovate_sl_ticks": 200,
        "mt5_volume": 15,
        "mt5_tp_points": 46,
        "mt5_sl_points": 139
    }
},
"funded_trade2": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 240,
        "tradovate_sl_ticks": 260,
        "mt5_volume": 18,
        "mt5_tp_points": 61,
        "mt5_sl_points": 64
    }
},
"funded_trade3": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 240,
        "tradovate_sl_ticks": 245,
        "mt5_volume": 3,
        "mt5_tp_points": 57,

```

```

        "mt5_sl_points": 64
    }
},
"funded_trade4": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 240,
        "tradovate_sl_ticks": 238,
        "mt5_volume": 0,
        "mt5_tp_points": 55,
        "mt5_sl_points": 64
    }
},
"farming": {
    "50k": {
        "tradovate_symbol": "MNQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 154,
        "tradovate_sl_ticks": 600,
        "mt5_volume": 3.6,
        "mt5_tp_points": 146,
        "mt5_sl_points": 43
    }
}
},
"Apex": {
    "name": "Apex",
    "account_sizes": ["$50,000"],
    "trading_phases": ["Challenge Phase", "Payout 1", "Payout 2", "Payout 3", "Payout 4",
        "Farming (Consistency)"],
    "strategy_configs": {
        "challenge_trade1": {
            "50k": {
                "tradovate_symbol": "NQM6",
                "tradovate_qty": 2,
                "tradovate_tp_ticks": 310,
                "tradovate_sl_ticks": 100,
                "mt5_volume": 0,
                "mt5_tp_points": 21,
                "mt5_sl_points": 82
            }
        },
        "challenge_trade2": {
            "50k": {
                "tradovate_symbol": "NQM6",
                "tradovate_qty": 2,
                "tradovate_tp_ticks": 410,
                "tradovate_sl_ticks": 100,
                "mt5_volume": 0,
                "mt5_tp_points": 21,
                "mt5_sl_points": 107
            }
        },
        "funded_trade1": {
            "50k": {
                "tradovate_symbol": "NQM6",
                "tradovate_qty": 2,
                "tradovate_tp_ticks": 200,

```

```

        "tradovate_sl_ticks": 100,
        "mt5_volume": 0,
        "mt5_tp_points": 21,
        "mt5_sl_points": 54
    }
},
"funded_trade2": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 100,
        "tradovate_sl_ticks": 100,
        "mt5_volume": 0,
        "mt5_tp_points": 21,
        "mt5_sl_points": 29
    }
},
"funded_trade3": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 150,
        "tradovate_sl_ticks": 100,
        "mt5_volume": 0,
        "mt5_tp_points": 21,
        "mt5_sl_points": 42
    }
},
"funded_trade4": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 150,
        "tradovate_sl_ticks": 100,
        "mt5_volume": 0,
        "mt5_tp_points": 21,
        "mt5_sl_points": 42
    }
},
"farming": {
    "50k": {
        "tradovate_symbol": "MNQM6",
        "tradovate_qty": 3,
        "tradovate_tp_ticks": 173,
        "tradovate_sl_ticks": 500,
        "mt5_volume": 0,
        "mt5_tp_points": 121,
        "mt5_sl_points": 48
    }
}
},
"Top One Futures": {
    "name": "Top One Futures Elite Access",
    "account_sizes": ["$50,000"],
    "trading_phases": ["Challenge Phase", "Payout 1", "Payout 2", "Double Dip Payout 1",
        "Double Dip Payout 2", "Farming Phase"],
    "strategy_configs": {
        "challenge_trade1": {
            "50k": {

```

```

        "tradovate_symbol": "NQM6",
        "tradovate_qty": 2,
        "tradovate_tp_ticks": 301,
        "tradovate_sl_ticks": 200,
        "mt5_volume": 0,
        "mt5_tp_points": 0,
        "mt5_sl_points": 0
    }
},
"funded_trade1": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 1,
        "tradovate_tp_ticks": 240,
        "tradovate_sl_ticks": 200,
        "mt5_volume": 0,
        "mt5_tp_points": 0,
        "mt5_sl_points": 0,
        "profit_target": 3500
    }
},
"funded_trade2": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 1,
        "tradovate_tp_ticks": 102,
        "tradovate_sl_ticks": 200,
        "mt5_volume": 0,
        "mt5_tp_points": 0,
        "mt5_sl_points": 0,
        "profit_target": 1000
    }
},
"funded_trade_doubledip_1": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 1,
        "tradovate_tp_ticks": 240,
        "tradovate_sl_ticks": 200,
        "mt5_volume": 0,
        "mt5_tp_points": 0,
        "mt5_sl_points": 0,
        "profit_target": 3500
    }
},
"funded_trade_doubledip_2": {
    "50k": {
        "tradovate_symbol": "NQM6",
        "tradovate_qty": 1,
        "tradovate_tp_ticks": 102,
        "tradovate_sl_ticks": 200,
        "mt5_volume": 0,
        "mt5_tp_points": 0,
        "mt5_sl_points": 0,
        "profit_target": 1000
    }
},
"farming": {
    "50k": {
        "tradovate_symbol": "MNQM6",

```

```

        "tradovate_qty": 2,
        "tradovate_tp_ticks": 254,
        "tradovate_sl_ticks": 600,
        "mt5_volume": 0,
        "mt5_tp_points": 0,
        "mt5_sl_points": 0
    }
}
}
}
}

```

Method: PropFirmManager.detect_prop_firm

```
def detect_prop_firm(self, username: str) -> Optional[str]
```

Detect prop firm based on username prefix. Returns None if unrecognized.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., username: str). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type -> Optional[str]. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.

Step by step (top-level body)

1. **if** not username or (isinstance(username, str) and len(username) < 4): branches conditionally.
2. Assigns prefix = username[:4].upper().
3. **if** prefix in self.firm_blueprints: branches conditionally.
4. **if** any((prefix.startswith(var) for var in ['APEX'])): branches conditionally (with an **else/elif** arm).
5. Calls self.logger.warning(...) for its side effect.
6. **return** None.

Code (copy-paste safe)

```

def detect_prop_firm(self, username: str) -> Optional[str]:
    """Detect prop firm based on username prefix. Returns None if unrecognized."""
    if not username or (isinstance(username, str) and len(username) < 4):
        self.logger.warning(f"Username too short: '{username}', cannot detect prop firm")
        return None

    prefix = username[:4].upper()

```

```

if prefix in self.firm_blueprints:
    self.logger.info(f"Detected prop firm '{prefix}' from username: '{username}'")
    return prefix

if any(prefix.startswith(var) for var in ["APEX"]):
    return "Apex"
elif any(prefix.startswith(var) for var in ["FTPR"]):
    return "FundingTicks"
elif any(prefix.startswith(var) for var in ["ELTD", "TDFU"]):
    return "Trade Day"
elif any(prefix.startswith(var) for var in ["FNFT", "FTFN"]):
    return "Funded Next"
elif any(prefix.startswith(var) for var in ["MFFU"]):
    return "MFFU"
elif any(prefix.startswith(var) for var in ["V2-"]):
    return "TopStep"
elif any(prefix.startswith(var) for var in ["TDFY", "FTDF"]):
    return "Tradeify"
elif any(prefix.startswith(var) for var in ["LFE0", "LFF0"]):
    return "Lucid"
elif any(prefix.startswith(var) for var in ["AFAD"]):
    return "AlphaFutures"

self.logger.warning(f"Unknown prefix '{prefix}' from account '{username}' – prop firm not recognized")
return None

```

Method: PropFirmManager.get_firm_info

```
def get_firm_info(self, firm_code: str) -> Dict
```

Get complete prop firm information.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `firm_code: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> Dict`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.

Step by step (top-level body)

1. Assigns `normalized_code = firm_code`.
2. **if** `firm_code == 'Alpha Futures'`: branches conditionally (with an **else/elif** arm).
3. **return** `self.firm_blueprints.get(normalized_code, self.firm_blueprints['MFF....`

Code (copy-paste safe)

```

def get_firm_info(self, firm_code: str) -> Dict:
    """Get complete prop firm information."""
    # Normalize firm_code to handle variations
    normalized_code = firm_code
    if firm_code == "Alpha Futures":
        normalized_code = "AlphaFutures"
    elif firm_code == "FundedNext":
        normalized_code = "Funded Next"
    elif firm_code == "TopOneFutures":
        normalized_code = "Top One Futures"

```

```

elif firm_code in ("MFFU", "My Funded Futures"):
    # Legacy alias – MFFU is no longer a separate blueprint
    normalized_code = "MFFU_Flex"

return self.firm_blueprints.get(normalized_code, self.firm_blueprints["MFFU_Flex"])

```

Method: PropFirmManager.get_account_sizes

```
def get_account_sizes(self, firm_code: str) -> list
```

Get account sizes for specific prop firm.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `firm_code: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> list`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.

Step by step (top-level body)

1. Assigns `firm_info = self.get_firm_info(firm_code)`.
2. **return** `firm_info.get('account_sizes', ['$50,000'])`.

Code (copy-paste safe)

```

def get_account_sizes(self, firm_code: str) -> list:
    """Get account sizes for specific prop firm."""
    firm_info = self.get_firm_info(firm_code)
    return firm_info.get("account_sizes", ["$50,000"])

```

Method: PropFirmManager.get_trading_phases

```
def get_trading_phases(self, firm_code: str) -> list
```

Get trading phases for specific prop firm.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `firm_code: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> list`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.

Step by step (top-level body)

1. Assigns `firm_info = self.get_firm_info(firm_code)`.
2. **return** `firm_info.get('trading_phases', ['Challenge Phase', 'Funded Phase'], ...)`

Code (copy-paste safe)

```

def get_trading_phases(self, firm_code: str) -> list:
    """Get trading phases for specific prop firm."""
    firm_info = self.get_firm_info(firm_code)

```

```
return firm_info.get("trading_phases", ["Challenge Phase", "Funded Phase", "Farming Phase"])
```

Method: PropFirmManager.convert_account_size_to_key

```
def convert_account_size_to_key(self, account_size: str, firm_code: str=None) -> str
```

Convert account size string to size key (e.g., '\$50,000' -> '50k')

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `account_size: str`, `firm_code: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> str`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.

Step by step (top-level body)

1. **if not** `account_size`: branches conditionally.
2. Assigns `s = account_size.lower().replace(',', '').replace('$', '').strip()`.
3. **if** `'150' in s` or `s == '150k'`: branches conditionally.
4. **if** `'100' in s` or `s == '100k'`: branches conditionally.
5. **return** `'50k'`.

Code (copy-paste safe)

```
def convert_account_size_to_key(self, account_size: str, firm_code: str = None) -> str:
    """Convert account size string to size key (e.g., '$50,000' -> '50k')"""
    if not account_size:
        return "50k"
    s = account_size.lower().replace(",", "").replace("$", "").strip()
    if "150" in s or s == "150k":
        return "150k"
    if "100" in s or s == "100k":
        return "100k"
    return "50k"
```

Method: PropFirmManager.get_strategy_config

```
def get_strategy_config(self, firm_code: str, phase_key: str, size_key: str='50k') -> Dict
```

Get strategy configuration for specific prop firm and phase.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `firm_code: str`, `phase_key: str`, `size_key: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> Dict`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns `size_key = self.convert_account_size_to_key(size_key, firm_code)`.
2. Calls `self.logger.info(...)` for its side effect.
3. Assigns `firm_info = self.get_firm_info(firm_code)`.
4. Assigns `strategy_configs = firm_info.get('strategy_configs', {})`.
5. Assigns `phase_config = strategy_configs.get(phase_key, {})`.
6. `if not phase_config:` branches conditionally.
7. Calls `self.logger.info(...)` for its side effect.
8. Assigns `config = phase_config.get(size_key)`.
9. `if not config:` branches conditionally (with an **else/elif** arm).
10. `if config:` branches conditionally.
11. **return** config.

Code (copy-paste safe)

```
def get_strategy_config(self, firm_code: str, phase_key: str, size_key: str = "50k") -> Dict:
    """Get strategy configuration for specific prop firm and phase."""
    # Normalize size_key: "$50,000" -> "50k", "$100,000" -> "100k", etc.
    size_key = self.convert_account_size_to_key(size_key, firm_code)
    self.logger.info(
        f"[DEBUG get_strategy_config] firm_code='{firm_code}', phase_key='{phase_key}', size_key='{size_key}'")

    firm_info = self.get_firm_info(firm_code)
    strategy_configs = firm_info.get("strategy_configs", {})

    phase_config = strategy_configs.get(phase_key, {})
    if not phase_config:
        self.logger.warning(f"Phase '{phase_key}' not found for '{firm_code}', using MFFU_Flex default")
        mffu_configs = self.firm_blueprints["MFFU_Flex"]["strategy_configs"]
        phase_config = mffu_configs.get(phase_key, {})

    self.logger.info(
        f"[DEBUG get_strategy_config] phase_config keys: {list(phase_config.keys())} if phase_config")

    config = phase_config.get(size_key)
    if not config:
        # For farming phase, try to fallback to 50k first before using MFFU_Flex fallback
        if phase_key == "farming" and size_key != "50k" and "50k" in phase_config:
            self.logger.info(
                f"Farming config not found for '{size_key}', using 50k farming config instead")
            config = phase_config["50k"]
        else:
            self.logger.warning(
                f"Config not found for '{firm_code}/{phase_key}/{size_key}', using MFFU_Flex fallback")
            config = self.firm_blueprints["MFFU_Flex"]["strategy_configs"]["challenge_trade1"]["50k"]
    else:
        self.logger.info(
            f"[DEBUG get_strategy_config] Found config: qty={config.get('tradovate_qty', 'N/A')}, vol={config.get('tradovate_vol', 'N/A')}")
```

```

if config:
    config = config.copy()
    if 'mt5_volume' in config:
        config['mt5_volume'] = round(config['mt5_volume'], 2)

return config

```

Method: PropFirmManager.validate_firm_setup

```
def validate_firm_setup(self, username: str) -> Tuple[Optional[str], Dict]
```

Validate and get complete prop firm setup for a username.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `username: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> Tuple[Optional[str], Dict]`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `firm_code = self.detect_prop_firm(username)`.
2. **if** `firm_code` is `None`: branches conditionally.
3. Assigns `firm_info = self.get_firm_info(firm_code)`.
4. Calls `self.logger.info(...)` for its side effect.
5. **return** (`firm_code`, `firm_info`).

Code (copy-paste safe)

```

def validate_firm_setup(self, username: str) -> Tuple[Optional[str], Dict]:
    """Validate and get complete prop firm setup for a username."""
    firm_code = self.detect_prop_firm(username)
    if firm_code is None:
        self.logger.warning(f"Could not detect prop firm for '{username}'")
        return None, {}
    firm_info = self.get_firm_info(firm_code)

    self.logger.info(f"Prop firm setup for '{username}': {firm_info['name']} ({firm_code})")
    return firm_code, firm_info

```

Method: PropFirmManager.set_prop_firm

```
def set_prop_firm(self, firm_name, broker=None, blueprint_firm=None)
```

Set the current prop firm Args: `firm_name`: The prop firm name (MFFU, TopStep, Other, etc.) `broker`: The broker platform for 'Other' mode (TopStep or Tradovate) `blueprint_firm`: The specific blueprint to use when in Other mode (e.g. MFFU_Flex)

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Calls `self.logger.info(...)` for its side effect.
2. Assigns `firm_mapping = {'MFFU': 'MFFU_Flex', 'My Funded Futures': 'MFFU_Flex', '...`
3. `if firm_name == 'Other':` branches conditionally (with an **else/elif** arm).
4. Calls `self.logger.info(...)` for its side effect.

Code (copy-paste safe)

```
def set_prop_firm(self, firm_name, broker=None, blueprint_firm=None):
    """Set the current prop firm

    Args:
        firm_name: The prop firm name (MFFU, TopStep, Other, etc.)
        broker: The broker platform for 'Other' mode (TopStep or Tradovate)
        blueprint_firm: The specific blueprint to use when in Other mode (e.g. MFFU_Flex)
    """
    self.logger.info(
        f"Setting prop firm: '{firm_name}', broker: '{broker}', blueprint: '{blueprint_firm}'")

    firm_mapping = {
        "MFFU": "MFFU_Flex", # Legacy alias
        "My Funded Futures": "MFFU_Flex", # Legacy alias
        "MFFU_Flex": "MFFU_Flex",
        "Funded Next": "Funded Next",
        "FundedNext": "Funded Next",
        "FundingTicks": "FundingTicks",
        "TopStep": "TopStep",
        "Tradeify": "Tradeify",
        "Apex": "Apex",
        "Alpha Futures": "AlphaFutures",
        "AlphaFutures": "AlphaFutures",
        "AlphaFutures GC": "AlphaFutures GC",
        "Top One Futures": "Top One Futures",
        "Other": "MFFU_Flex" # Default fallback
    }

    # For "Other" prop firm, we prefer the specific blueprint if provided
    if firm_name == "Other":
        if blueprint_firm and blueprint_firm in self.firm_blueprints:
            self.current_firm_code = blueprint_firm
            self.logger.info(f"Other mode: Using specific blueprint {self.current_firm_code}")
        elif broker and broker == "TopStep":
            self.current_firm_code = "TopStep"
        else: # Tradovate or any other
            self.current_firm_code = "MFFU_Flex"

        if not blueprint_firm:
            self.logger.info(f"Other mode: Mapped to {self.current_firm_code} based on broker '{broker}'")
    else:
        self.current_firm_code = firm_mapping.get(firm_name, firm_name)

    self.logger.info(f"Set prop firm to: {self.current_firm_code}")
```

Method: PropFirmManager.get_prop_firm_strategy_config

```
def get_prop_firm_strategy_config(self, trading_phase, account_size='$50,000', balance_performance=0.0):  
    Get strategy config for current prop firm (manual trading only)
```

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Calls `self.logger.info(...)` for its side effect.
2. Assigns `firm_info = self.firm_blueprints.get(self.current_firm_code, self.fir....`
3. **if** `self.current_firm_code not in self.firm_blueprints`: branches conditionally.
4. Assigns `size_key = '50k'`.
5. **if** `'$100,000' in account_size` or `'100k' in account_size.lower()`: branches conditionally (with an **else/elif** arm).
6. Calls `self.logger.info(...)` for its side effect.
7. Assigns `phase_key = 'challenge_trade1'`.
8. **if** `trading_phase == 'Challenge Phase'`: branches conditionally.
9. Calls `self.logger.info(...)` for its side effect.
10. **if** `trading_phase == 'Funded Phase'`: branches conditionally.
11. **if** `trading_phase == 'Double Dip Phase'`: branches conditionally (with an **else/elif** arm).
12. Assigns `strategy_configs = firm_info.get('strategy_configs', {})`.
13. Assigns `phase_configs = strategy_configs.get(phase_key, {})`.
14. Assigns `config = phase_configs.get(size_key, {})`.
15. ... and 5 more statement(s) in the body.

Code (copy-paste safe)

```
def get_prop_firm_strategy_config(self, trading_phase, account_size="$50,000", balance_performance=0.0):  
    """Get strategy config for current prop firm (manual trading only)"""  
    self.logger.info(  
        f"Looking up: '{self.current_firm_code}', phase='{trading_phase}', size='{account_size}'")  
  
    firm_info = self.firm_blueprints.get(self.current_firm_code, self.firm_blueprints["MFFU_Flex"])  
  
    if self.current_firm_code not in self.firm_blueprints:  
        self.logger.warning(f"Blueprint '{self.current_firm_code}' not found! Using MFFU_Flex fallback")  
  
    # Convert account size to size key (e.g., "$50,000" -> "50k", "$100,000" -> "100k", "$150,000" -> "150k")  
    size_key = "50k" # Default  
    if "$100,000" in account_size or "100k" in account_size.lower():  
        size_key = "100k"  
    elif "$150,000" in account_size or "150k" in account_size.lower():
```

```

        size_key = "150k"
    elif "$50,000" in account_size or "50k" in account_size.lower():
        size_key = "50k"

    self.logger.info(f"[DEBUG] Converted account_size '{account_size}' to size_key '{size_key}'")

    # Default phase key
    phase_key = "challenge_trade1"

    if trading_phase == "Challenge Phase":
        if self.current_firm_code == "Top One Futures":
            if balance_performance >= 5.0:
                phase_key = "challenge_trade3"
            elif balance_performance >= 2.5:
                phase_key = "challenge_trade2"
            else:
                phase_key = "challenge_trade1"
        else:
            phase_key = "challenge_trade2" if balance_performance >= 2.5 else "challenge_trade1"

    self.logger.info(f"[DEBUG] Using phase_key '{phase_key}' for phase '{trading_phase}'")

    if trading_phase == "Funded Phase":
        if self.current_firm_code in ("MFFU", "MFFU_Flex"):
            # MFFU / MFFU_Flex Funded Phase Logic (Condensed Payouts)
            if balance_performance < 2.0:
                phase_key = "funded_trade1"
            elif balance_performance < 4.0:
                phase_key = "funded_trade2"
            elif balance_performance < 6.0:
                phase_key = "funded_trade3"
            else:
                phase_key = "funded_trade4"
        elif self.current_firm_code == "Trade Day":
            phase_key = "funded"
        elif self.current_firm_code == "TopStep":
            # TopStep Funded Phase Logic (Condensed Payouts)
            if balance_performance < 2.0:
                phase_key = "funded_trade1"
            elif balance_performance < 4.0:
                phase_key = "funded_trade2"
            elif balance_performance < 6.0:
                phase_key = "funded_trade3"
            else:
                phase_key = "funded_trade4"
        elif self.current_firm_code == "Top One Futures":
            # Top One Futures uses Payout 1 / Payout 2 selection instead of
            # a single "Funded Phase". This branch only runs as a safety
            # fallback if "Funded Phase" is somehow still selected; default
            # to funded_trade1 so behavior matches Payout 1.
            phase_key = "funded_trade1"
        elif self.current_firm_code == "FundingTicks":
            phase_key = "funded_trade1" # Default to trade 1
        elif self.current_firm_code == "Tradeify":
            phase_key = "funded_trade1"
        else:
            # Default for others
            phase_key = "funded_trade1"

    if trading_phase == "Double Dip Phase":

```

```

if self.current_firm_code in ("TopStep", "MFFU", "MFFU_Flex"):
    # Double Dip Phase Logic (shared by TopStep / MFFU / MFFU_Flex)
    if balance_performance < 2.0:
        phase_key = "funded_trade_doubledip_1"
    elif balance_performance < 4.0:
        phase_key = "funded_trade_doubledip_2"
    elif balance_performance < 6.0:
        phase_key = "funded_trade_doubledip_3"
    else:
        phase_key = "funded_trade_doubledip_4"
elif self.current_firm_code == "Top One Futures":
    # Top One Futures Double Dip Phase Logic (same structure as funded)
    if balance_performance < 2.0:
        phase_key = "funded_trade_doubledip_1"
    elif balance_performance < 4.0:
        phase_key = "funded_trade_doubledip_2"
    elif balance_performance < 6.0:
        phase_key = "funded_trade_doubledip_3"
    else:
        phase_key = "funded_trade_doubledip_4"
else:
    phase_key = "funded" # Fallback

elif trading_phase == "Double Dip Payout 1":
    # Top One Futures Double Dip uses the same structure as Payout 1
    phase_key = "funded_trade_doubledip_1"

elif trading_phase == "Double Dip Payout 2":
    # Top One Futures Double Dip uses the same structure as Payout 2
    phase_key = "funded_trade_doubledip_2"

elif trading_phase == "Payout 1":
    phase_key = "funded_trade1"

elif trading_phase == "Payout 2":
    phase_key = "funded_trade2"

elif trading_phase == "Payout 3":
    phase_key = "funded_trade3"

elif trading_phase == "Payout 4":
    phase_key = "funded_trade4"

elif trading_phase == "Farming Phase":
    phase_key = "farming"

elif trading_phase == "Farming (Consistency)":
    phase_key = "farming"

strategy_configs = firm_info.get("strategy_configs", {})
phase_configs = strategy_configs.get(phase_key, {})
config = phase_configs.get(size_key, {})

self.logger.info(
    f"[DEBUG] Retrieved config for {self.current_firm_code}/{phase_key}/{size_key}: {bool(config)}")
if config:
    self.logger.info(
        f"[DEBUG] Config qty={config.get('tradovate_qty', 'N/A')}, volume={config.get('mt5_volume')}")

if phase_key == "farming" and self.current_firm_code == "Funded Next":

```

```

# Strict logic: Funded Next farming MUST use 50k blueprint regardless of size
if "50k" in phase_configs:
    self.logger.info(f"Enforcing Funded Next 50k farming blueprint for '{size_key}'")
    config = phase_configs["50k"]

if not config:
    # For farming phase, try to fallback to 50k first before using MFFU fallback
    if phase_key == "farming" and size_key != "50k" and "50k" in phase_configs:
        self.logger.info(
            f"Farming config not found for '{size_key}', using 50k farming config instead")
        config = phase_configs["50k"]

# Try to find a fallback within the same firm if possible
if trading_phase == "Funded Phase" and not config:
    # Try alternative funded keys
    for alt_key in ["funded", "funded_trade1"]:
        if alt_key in strategy_configs:
            config = strategy_configs[alt_key].get(size_key, {})
            if config:
                phase_key = alt_key
                break

if not config:
    mffu_configs = self.firm_blueprints["MFFU_Flex"]["strategy_configs"]
    # Map current phase_key to MFFU_Flex equivalent if possible
    mffu_phase_key = phase_key
    if "funded" in phase_key: mffu_phase_key = "funded_trade1"
    elif "farming" in phase_key: mffu_phase_key = "farming"
    else: mffu_phase_key = "challenge_trade1"

    fallback = mffu_configs.get(mffu_phase_key, {}).get(size_key, {})

    if fallback:
        self.logger.warning(
            f"No config for {self.current_firm_code}/{phase_key}, using MFFU_Flex {mffu_phase_key}")
        return fallback
    else:
        ultimate_fallback = {
            "tradovate_symbol": "MNQM6",
            "tradovate_qty": 2,
            "tradovate_tp_ticks": 154,
            "tradovate_sl_ticks": 400,
            "mt5_volume": 4.0,
            "mt5_tp_points": 98,
            "mt5_sl_points": 41
        }
        self.logger.error(f"No valid config, using ultimate fallback")
        return ultimate_fallback

return config

```

Method: PropFirmManager.get_prop_firm_account_sizes

```
def get_prop_firm_account_sizes(self, firm_code=None)
```

Get account sizes for detected or specified prop firm

Step by step (top-level body)

1. if `firm_code` is `None`: branches conditionally.

2. **return** self.get_account_sizes(firm_code).

Code (copy-paste safe)

```
def get_prop_firm_account_sizes(self, firm_code=None):
    """Get account sizes for detected or specified prop firm"""
    if firm_code is None:
        firm_code = self.current_firm_code
    return self.get_account_sizes(firm_code)
```

Method: PropFirmManager.get_prop_firm_trading_phases

```
def get_prop_firm_trading_phases(self, firm_code=None)
```

Get trading phases for detected or specified prop firm

Step by step (top-level body)

1. **if** firm_code is None: branches conditionally.
2. **return** self.get_trading_phases(firm_code).

Code (copy-paste safe)

```
def get_prop_firm_trading_phases(self, firm_code=None):
    """Get trading phases for detected or specified prop firm"""
    if firm_code is None:
        firm_code = self.current_firm_code
    return self.get_trading_phases(firm_code)
```

Method: PropFirmManager.format_volume_for_display

```
def format_volume_for_display(self, volume)
```

Format MT5 volume for display (used by GUI)

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **if** volume == 0: branches conditionally.
2. **return** f'{volume:.1f}'.

Code (copy-paste safe)

```
def format_volume_for_display(self, volume):
    """Format MT5 volume for display (used by GUI)"""
    if volume == 0:
        return "0.0"
    return f"{volume:.1f}"
```

Method: PropFirmManager.get_tick_value

```
def get_tick_value(self, symbol: str) -> float
```


Get tick value for a given trading symbol. Returns the dollar value per tick for the given contract symbol. This is the single source of truth for tick values across the app.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `symbol: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> float`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns `symbol_upper = symbol.upper()` if `symbol` else `""`.
2. **if** `'MGC'` in `symbol_upper`: branches conditionally.
3. **if** `'GC'` in `symbol_upper`: branches conditionally.
4. **if** `'MNQ'` in `symbol_upper`: branches conditionally.
5. **if** `'NQ'` in `symbol_upper`: branches conditionally.
6. **return** `0.5`.

Code (copy-paste safe)

```
def get_tick_value(self, symbol: str) -> float:
    """Get tick value for a given trading symbol.

    Returns the dollar value per tick for the given contract symbol.
    This is the single source of truth for tick values across the app.
    """
    symbol_upper = symbol.upper() if symbol else ""
    # Micro Gold contracts
    if "MGC" in symbol_upper:
        return 1.0
    # Standard Gold contracts
    if "GC" in symbol_upper:
        return 10.0
    # Micro NASDAQ contracts (check before standard NQ)
    if "MNQ" in symbol_upper:
        return 0.5
    # Standard NASDAQ contracts
    if "NQ" in symbol_upper:
        return 5.0
    # Default to micro contract value
    return 0.5
```

Method: `PropFirmManager.predict_next_trade`

```
def predict_next_trade(self, firm_code: str, current_phase: str, current_profit: float,
    size_key: str='50k') -> Dict
```

Predict current stage and next trade based on balance. Walks the ordered trade progression for the firm/phase, computing cumulative profit at each stage boundary. Returns a dict with: `current_stage` -

human label like "Challenge Trade 2" next_stage - human label for next trade, or "Phase Complete"
next_config - the blueprint config dict for the next trade (or None) next_phase_key - the blueprint key for
the next trade balance_target - the cumulative \$ target at end of current stage stages - list of (label,
phase_key, cumul_target) for all stages

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `firm_code: str`, `current_phase: str`, `current_profit: float`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> Dict`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns `phase_map = {'Challenge': 'Challenge', 'Funded': 'Funded', 'Farming':....`
2. Assigns `phase_key_group = phase_map.get(current_phase, current_phase)`.
3. Assigns `firm_orders = self._PHASE_TRADE_ORDER.get(firm_code, {})`.
4. Assigns `trade_keys = firm_orders.get(phase_key_group, [])`.
5. `if not trade_keys:` branches conditionally.
6. Assigns `stages = []`.
7. Assigns `cumulative = 0.0`.
8. `for key in trade_keys:` iterates.
9. Assigns `current_idx = 0`.
10. `for (i, (_, _, cumul_target, _)) in enumerate(stages):` iterates.
11. Assigns `(current_label, current_key, current_target, current_stage_profit) = stages[current_idx]`.
12. `if current_idx + 1 < len(stages):` branches conditionally (with an **else/elif** arm).
13. `return {'current_stage': current_label, 'current_phase_key': current_key, ...`

Code (copy-paste safe)

```
def predict_next_trade(self, firm_code: str, current_phase: str,
                      current_profit: float, size_key: str = "50k") -> Dict:
    """Predict current stage and next trade based on balance.

    Walks the ordered trade progression for the firm/phase, computing
    cumulative profit at each stage boundary. Returns a dict with:
        current_stage - human label like "Challenge Trade 2"
        next_stage    - human label for next trade, or "Phase Complete"
        next_config   - the blueprint config dict for the next trade (or None)
        next_phase_key - the blueprint key for the next trade
        balance_target - the cumulative $ target at end of current stage
        stages        - list of (label, phase_key, cumul_target) for all stages
    """
    # Normalize phase display to match _PHASE_TRADE_ORDER keys
```

```

phase_map = {"Challenge": "Challenge", "Funded": "Funded",
             "Farming": "Farming", "Double Dip": "Double Dip",
             "Payout 1": "Funded", "Payout 2": "Funded",
             "Payout 3": "Funded", "Payout 4": "Funded"}
phase_key_group = phase_map.get(current_phase, current_phase)

firm_orders = self._PHASE_TRADE_ORDER.get(firm_code, {})
trade_keys = firm_orders.get(phase_key_group, [])

if not trade_keys:
    return {"current_stage": current_phase, "next_stage": "-",
            "next_config": None, "next_phase_key": None,
            "balance_target": 0, "stages": []}

# Build cumulative profit milestones for each stage
stages = []
cumulative = 0.0
for key in trade_keys:
    cfg = self.get_strategy_config(firm_code, key, size_key)
    if not cfg:
        continue
    sym = cfg.get("tradovate_symbol", "") or cfg.get("topstepx_symbol", "")
    qty = int(cfg.get("tradovate_qty", 0) or cfg.get("topstepx_qty", 0))
    tp = int(cfg.get("tradovate_tp_ticks", 0) or cfg.get("topstepx_tp_ticks", 0))
    tick_val = self.get_tick_value(sym)
    stage_profit = qty * tp * tick_val
    cumulative += stage_profit
    label = key.replace("_", " ").replace("trade", "Trade ").title()
    # Clean up label
    label = label.replace("Challenge Trade ", "Challenge #") \
               .replace("Funded Trade ", "Funded #") \
               .replace("Funded ", "Funded #") if "trade" in key.lower() else label
    label = key.replace("_", " ").title()
    stages.append((label, key, round(cumulative, 2), round(stage_profit, 2)))

# Find which stage we're currently in based on profit
current_idx = 0
for i, (_, _, cumul_target, _) in enumerate(stages):
    if current_profit < cumul_target - 0.01:
        current_idx = i
        break
else:
    # Profit exceeds all stages – we're past the last one
    current_idx = len(stages) - 1

current_label, current_key, current_target, current_stage_profit = stages[current_idx]

# Next trade
if current_idx + 1 < len(stages):
    next_label, next_key, next_target, next_profit = stages[current_idx + 1]
    next_cfg = self.get_strategy_config(firm_code, next_key, size_key)
else:
    next_label = "Phase Complete"
    next_key = None
    next_cfg = None
    next_target = current_target

return {
    "current_stage": current_label,
    "current_phase_key": current_key,

```

```

        "current_target": current_target,
        "current_stage_profit": current_stage_profit,
        "next_stage": next_label,
        "next_config": next_cfg,
        "next_phase_key": next_key,
        "next_target": next_target,
        "balance_target": current_target,
        "stages": stages,
    }

```

Method: PropFirmManager.get_stage_start_target

```

def get_stage_start_target(self, firm_code: str, current_phase: str, phase_key: str,
    size_key: str='50k') -> float

```

Return the cumulative \$ profit expected BEFORE this stage begins. E.g. if the phase order is [trade1, trade2, trade3] and each has a \$1000 TP target, then: trade1 start = \$0 trade2 start = \$1000 trade3 start = \$2000 This is the profit the account should already have when entering this stage. Used to adjust TP so the trade doesn't overshoot or undershoot the stage target.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., firm_code: str, current_phase: str, phase_key: str). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type -> float. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.

Step by step (top-level body)

1. Assigns phase_map = {'Challenge': 'Challenge', 'Funded': 'Funded', 'Farming':.....
2. Assigns phase_group = phase_map.get(current_phase, current_phase).
3. Assigns firm_orders = self._PHASE_TRADE_ORDER.get(firm_code, {}).
4. Assigns trade_keys = firm_orders.get(phase_group, []).
5. Assigns cumulative = 0.0.
6. **for** key in trade_keys: iterates.
7. **return** round(cumulative, 2).

Code (copy-paste safe)

```

def get_stage_start_target(self, firm_code: str, current_phase: str,
    phase_key: str, size_key: str = "50k") -> float:
    """Return the cumulative $ profit expected BEFORE this stage begins.

    E.g. if the phase order is [trade1, trade2, trade3] and each has a
    $1000 TP target, then:
        trade1 start = $0
        trade2 start = $1000
        trade3 start = $2000

    This is the profit the account should already have when entering
    this stage. Used to adjust TP so the trade doesn't overshoot or
    undershoot the stage target.

```

```

"""
phase_map = {"Challenge": "Challenge", "Funded": "Funded",
             "Farming": "Farming", "Double Dip": "Double Dip",
             "Payout 1": "Funded", "Payout 2": "Funded",
             "Payout 3": "Funded", "Payout 4": "Funded"}
phase_group = phase_map.get(current_phase, current_phase)

firm_orders = self._PHASE_TRADE_ORDER.get(firm_code, {})
trade_keys = firm_orders.get(phase_group, [])

cumulative = 0.0
for key in trade_keys:
    if key == phase_key:
        return round(cumulative, 2)
    cfg = self.get_strategy_config(firm_code, key, size_key)
    if not cfg:
        continue
    sym = cfg.get("tradovate_symbol", "") or cfg.get("topstepx_symbol", "")
    qty = int(cfg.get("tradovate_qty", 0) or cfg.get("topstepx_qty", 0))
    tp = int(cfg.get("tradovate_tp_ticks", 0) or cfg.get("topstepx_tp_ticks", 0))
    tick_val = self.get_tick_value(sym)
    cumulative += qty * tp * tick_val
return round(cumulative, 2)

```

Method: PropFirmManager.adjust_tp_sl_for_balance

```
def adjust_tp_sl_for_balance(self, config: Dict, current_profit: float) -> Dict
```

Adjust TP and SL ticks/points based on current account profit. If the account already has profit, TP is reduced so the trade doesn't overshoot the blueprint target. If the account has a loss, TP is increased to recover back to the target. SL is adjusted inversely. Returns a NEW config dict with adjusted values (original is not mutated).

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., config: Dict, current_profit: float). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type -> Dict. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns config = config.copy().
2. Assigns symbol = config.get('tradovate_symbol', '') or config.get('topstep....
3. Assigns qty = int(config.get('tradovate_qty', 0) or config.get('topstep....
4. Assigns orig_tp = int(config.get('tradovate_tp_ticks', 0) or config.get('to....
5. Assigns orig_sl = int(config.get('tradovate_sl_ticks', 0) or config.get('to....

6. Assigns `mt5_tp = int(config.get('mt5_tp_points', 0))`.
7. Assigns `mt5_sl = int(config.get('mt5_sl_points', 0))`.
8. **if** `qty <= 0` or `orig_tp <= 0`: branches conditionally.
9. Assigns `tick_val = self.get_tick_value(symbol)`.
10. Assigns `target_profit = qty * orig_tp * tick_val`.
11. Assigns `target_loss = qty * orig_sl * tick_val`.
12. **if** `target_profit == 0`: branches conditionally.
13. Assigns `remaining_profit = target_profit - current_profit`.
14. Assigns `min_tp_dollars = target_profit * 0.1`.
15. ... and 14 more statement(s) in the body.

Code (copy-paste safe)

```
def adjust_tp_sl_for_balance(self, config: Dict, current_profit: float) -> Dict:
    """Adjust TP and SL ticks/points based on current account profit.

    If the account already has profit, TP is reduced so the trade doesn't
    overshoot the blueprint target. If the account has a loss, TP is
    increased to recover back to the target. SL is adjusted inversely.

    Returns a NEW config dict with adjusted values (original is not mutated).
    """
    config = config.copy()
    symbol = config.get("tradovate_symbol", "") or config.get("topstepx_symbol", "")
    qty = int(config.get("tradovate_qty", 0) or config.get("topstepx_qty", 0))
    orig_tp = int(config.get("tradovate_tp_ticks", 0) or config.get("topstepx_tp_ticks", 0))
    orig_sl = int(config.get("tradovate_sl_ticks", 0) or config.get("topstepx_sl_ticks", 0))
    mt5_tp = int(config.get("mt5_tp_points", 0))
    mt5_sl = int(config.get("mt5_sl_points", 0))

    if qty <= 0 or orig_tp <= 0:
        return config

    tick_val = self.get_tick_value(symbol)
    # Blueprint dollar target for the Tradovate trade
    target_profit = qty * orig_tp * tick_val
    # Blueprint dollar risk for the Tradovate trade
    target_loss = qty * orig_sl * tick_val

    if target_profit == 0:
        return config

    remaining_profit = target_profit - current_profit
    # Floor: at least 10% of original TP to avoid zero/negative TP
    min_tp_dollars = target_profit * 0.10
    remaining_profit = max(remaining_profit, min_tp_dollars)

    # Adjustment ratio
    tp_ratio = remaining_profit / target_profit

    # Adjusted Tradovate TP ticks (round to nearest int, minimum 5 ticks)
    adjusted_tp = max(5, round(orig_tp * tp_ratio))

    # SL adjustment: if in profit, we can afford a wider SL (profit cushion).
```

```

# If in loss, keep blueprint SL – don't tighten, let the prop account
# reach its actual drawdown limit before the hedge closes.
if current_profit > 0:
    # Profit acts as a buffer we can afford to lose → widen SL
    adjusted_sl_dollars = target_loss + current_profit
else:
    # In loss – keep blueprint SL so hedge stays open until drawdown limit
    adjusted_sl_dollars = target_loss
sl_ratio = adjusted_sl_dollars / target_loss if target_loss > 0 else 1.0
adjusted_sl = max(10, round(orig_sl * sl_ratio))

# Apply Tradovate adjustments
tp_key = "tradovate_tp_ticks" if "tradovate_tp_ticks" in config else "topstepx_tp_ticks"
sl_key = "tradovate_sl_ticks" if "tradovate_sl_ticks" in config else "topstepx_sl_ticks"
config[tp_key] = adjusted_tp
config[sl_key] = adjusted_sl

# Apply SWAPPED ratios to MT5 TP/SL points – hedging means
# Tradovate TP ↔ MT5 SL (hedge loses when main wins)
# Tradovate SL ↔ MT5 TP (hedge wins when main loses)
if mt5_tp > 0:
    config["mt5_tp_points"] = max(5, round(mt5_tp * sl_ratio))
if mt5_sl > 0:
    config["mt5_sl_points"] = max(5, round(mt5_sl * tp_ratio))

self.logger.info(
    f"[TP/SL Adjust] profit=${current_profit:.2f}, "
    f"target=${target_profit:.2f}, remaining=${remaining_profit:.2f}, "
    f"TP: {orig_tp}→{adjusted_tp} ticks, SL: {orig_sl}→{adjusted_sl} ticks, "
    f"MT5 TP(sl_ratio): {mt5_tp}→{config.get('mt5_tp_points')}, "
    f"MT5 SL(tp_ratio): {mt5_sl}→{config.get('mt5_sl_points')}")

return config

```

Method: PropFirmManager.compute_account_status

```

def compute_account_status(self, firm_code: str, phase: str, current_balance: float,
    starting_balance: float=50000.0, breached: bool=False) -> str

```

Compute the account status based on balance vs profit targets. Returns one of: 'Pass', 'Fail', 'In Progress', or None if unable to determine. Parameters: firm_code: Canonical firm identifier (e.g. 'MFFU', 'TopStep') phase: Current phase display name ('Challenge' or 'Funded') current_balance: Live broker balance starting_balance: Account starting balance (default \$50,000) breached: Whether the prop firm reports the account as breached

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., firm_code: str, phase: str, current_balance: float). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type -> str. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **if** breached: branches conditionally.
2. Assigns targets = self._PROFIT_TARGETS.get(firm_code, {}).
3. Assigns phase_key = 'Challenge' if phase == 'Challenge' else 'Funded'.
4. Assigns target = targets.get(phase_key).
5. **if** target is None: branches conditionally.
6. Assigns profit = current_balance - starting_balance.
7. **if** profit >= target: branches conditionally.
8. **if** firm_code in ('MFFU', 'MFFU_Flex', 'My Funded Futures'): branches conditionally (with an **else/elif** arm).
9. **if** current_balance <= hard_stop: branches conditionally.
10. **return** 'In Progress'.

Code (copy-paste safe)

```
def compute_account_status(self, firm_code: str, phase: str,
                           current_balance: float,
                           starting_balance: float = 50000.0,
                           breached: bool = False) -> str:
    """Compute the account status based on balance vs profit targets.

    Returns one of: 'Pass', 'Fail', 'In Progress', or None if unable to determine.

    Parameters:
        firm_code: Canonical firm identifier (e.g. 'MFFU', 'TopStep')
        phase: Current phase display name ('Challenge' or 'Funded')
        current_balance: Live broker balance
        starting_balance: Account starting balance (default $50,000)
        breached: Whether the prop firm reports the account as breached
    """
    if breached:
        return "Fail"

    targets = self._PROFIT_TARGETS.get(firm_code, {})
    phase_key = "Challenge" if phase == "Challenge" else "Funded"
    target = targets.get(phase_key)
    if target is None:
        return None # unknown firm/phase, can't determine

    profit = current_balance - starting_balance

    if profit >= target:
        return "Pass"

    # Check if balance is at or below breach threshold
    if firm_code in ("MFFU", "MFFU_Flex", "My Funded Futures"):
        if current_balance < 50100.0:
            hard_stop = 0.0
        else:
            hard_stop = 50100.0
    else:
        hard_stop = self._HARD_STOP_THRESHOLDS.get(firm_code, 50000.0)
```



```

        if current_balance <= hard_stop:
            return "Fail"

    return "In Progress"

```

Method: PropFirmManager.adjust_farming_tp_sl

```
def adjust_farming_tp_sl(self, config: Dict, current_balance: float, firm_code: str) -> Dict
```

Adjust MT5 TP for farming trades based on hard-stop proximity. Farming trades use micro contracts (MNQM6). If the MT5 TP is so large that the prop account would breach the hard-stop threshold before MT5 closes, we cap MT5 TP to a safe distance. Only MT5 TP is adjusted — Tradovate TP/SL and MT5 SL stay unchanged. Returns a NEW config dict (original is not mutated).

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., config: Dict, current_balance: float, firm_code: str). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type -> Dict. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns config = config.copy().
2. Assigns symbol = config.get('tradovate_symbol', '') or config.get('topstep....
3. **if** 'MNQ' not in symbol.upper(): branches conditionally.
4. Assigns mt5_tp = float(config.get('mt5_tp_points', 0)).
5. **if** mt5_tp <= 0: branches conditionally.
6. **if** firm_code in ('MFFU', 'MFFU_Flex', 'My Funded Futures'): branches conditionally (with an **else/elif** arm).
7. Assigns distance_to_max_loss = current_balance - hard_stop.
8. **if** distance_to_max_loss <= 0: branches conditionally.
9. Assigns tick_val = self.get_tick_value(symbol).
10. Assigns trado_qty = int(config.get('tradovate_qty', 1) or config.get('topstep....
11. **if** tick_val <= 0 or trado_qty <= 0: branches conditionally.
12. Assigns distance_ticks = distance_to_max_loss / (tick_val * trado_qty).
13. Assigns distance_mt5_pts = distance_ticks / self._NQ_TICK_TO_POINT_RATIO.
14. Assigns safe_distance = distance_mt5_pts - self._FARMING_BUFFER_POINTS.
15. ... and 4 more statement(s) in the body.

Code (copy-paste safe)

```

def adjust_farming_tp_sl(self, config: Dict, current_balance: float,
                        firm_code: str) -> Dict:
    """Adjust MT5 TP for farming trades based on hard-stop proximity.

    Farming trades use micro contracts (MNQM6). If the MT5 TP is so
    large that the prop account would breach the hard-stop threshold
    before MT5 closes, we cap MT5 TP to a safe distance.

    Only MT5 TP is adjusted – Tradovate TP/SL and MT5 SL stay unchanged.

    Returns a NEW config dict (original is not mutated).
    """
    config = config.copy()
    symbol = config.get("tradovate_symbol", "") or config.get("topstepx_symbol", "")

    # Only applies to micro contracts (farming)
    if "MNQ" not in symbol.upper():
        return config

    mt5_tp = float(config.get("mt5_tp_points", 0))
    if mt5_tp <= 0:
        return config

    # Determine hard-stop threshold
    if firm_code in ("MFFU", "MFFU_Flex", "My Funded Futures"):
        # MFFU zero-start vs traditional detection:
        # If balance < $50,100 it can't be a traditional $50k account
        # (would already be breached), so it must be zero-start.
        if current_balance < 50100.0:
            hard_stop = 0.0
        else:
            hard_stop = 50100.0
    else:
        hard_stop = self._HARD_STOP_THRESHOLDS.get(firm_code, 50000.0)

    distance_to_max_loss = current_balance - hard_stop
    if distance_to_max_loss <= 0:
        self.logger.warning(
            f"[Farming TP] Balance ${current_balance:,.2f} already at/below "
            f"hard stop ${hard_stop:,.2f} for {firm_code}")
        return config

    tick_val = self.get_tick_value(symbol)
    trado_qty = int(config.get("tradovate_qty", 1) or config.get("topstepx_qty", 1))
    if tick_val <= 0 or trado_qty <= 0:
        return config

    # Distance in Tradovate ticks, then convert to MT5 points
    distance_ticks = distance_to_max_loss / (tick_val * trado_qty)
    distance_mt5_pts = distance_ticks / self._NQ_TICK_TO_POINT_RATIO
    safe_distance = distance_mt5_pts - self._FARMING_BUFFER_POINTS

    # Only cap if MT5 TP exceeds safe distance AND balance is close
    max_loss_dollars = mt5_tp * tick_val * trado_qty
    needs_adjustment = (mt5_tp > safe_distance
                        and distance_to_max_loss < max_loss_dollars)

    if needs_adjustment and safe_distance > 0:
        recommended = int(safe_distance)
        self.logger.info(

```

```

        f"[Farming TP Cap] {firm_code}: balance=${current_balance:.2f}, "
        f"hard_stop=${hard_stop:.2f}, distance=${distance_to_max_loss:.2f}, "
        f"safe={safe_distance:.1f}pts → MT5 TP {mt5_tp}→{recommended}")
    config["mt5_tp_points"] = max(5, recommended)
else:
    self.logger.info(
        f"[Farming TP OK] {firm_code}: MT5 TP {mt5_tp} pts <= "
        f"safe distance {safe_distance:.1f} pts – no change needed")

    return config

```

Method: PropFirmManager.get_default_config

```
def get_default_config(self) -> Dict
```

Get the default strategy config (MFFU_Flex challenge_trade1 50k). Used as the ultimate fallback when no specific config is available. Always pulls from the actual MFFU_Flex blueprint, never hardcoded values.

Why these Python concepts were used

- **return type annotation** — Annotated return type -> Dict. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.

Step by step (top-level body)

1. `return self.get_strategy_config('MFFU_Flex', 'challenge_trade1', '50k').`

Code (copy-paste safe)

```

def get_default_config(self) -> Dict:
    """Get the default strategy config (MFFU_Flex challenge_trade1 50k).

    Used as the ultimate fallback when no specific config is available.
    Always pulls from the actual MFFU_Flex blueprint, never hardcoded values.
    """
    return self.get_strategy_config("MFFU_Flex", "challenge_trade1", "50k")

```

Method: PropFirmManager.get_default_symbol

```
def get_default_symbol(self, broker: str='tradovate') -> str
```

Get the default trading symbol for a broker platform. Pulls from the MFFU_Flex challenge_trade1 blueprint.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `broker: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type -> str. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.

Step by step (top-level body)

1. `Assigns config = self.get_default_config().`
2. `if broker.lower() == 'topstepx' or broker.lower() == 'topstep':` branches conditionally.

3. **return** config.get('tradovate_symbol', '').

Code (copy-paste safe)

```
def get_default_symbol(self, broker: str = "tradovate") -> str:
    """Get the default trading symbol for a broker platform.

    Pulls from the MFFU_Flex challenge_trade1 blueprint.
    """
    config = self.get_default_config()
    if broker.lower() == "topstepx" or broker.lower() == "topstep":
        return config.get("topstepx_symbol", config.get("tradovate_symbol", ""))
    return config.get("tradovate_symbol", "")
```

Method: PropFirmManager.check_and_lock_direction

```
def check_and_lock_direction(self, account_number: str, side: str) -> bool
```

*Enforce one-direction-per-day rule *per account*. Args: account_number: The Tradovate/TopStep account identifier. side: 'BUY' or 'SELL'. Returns: True if the trade is allowed, False if blocked.*

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., account_number: str, side: str). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type -> bool. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **if** not account_number or account_number in ('Unknown', 'Not Connected'): branches conditionally.
2. Assigns today = datetime.date.today().
3. Assigns side_upper = side.upper().
4. Assigns lock = self._account_direction_locks.get(account_number).
5. **if** lock is None or lock['date'] != today: branches conditionally.
6. **if** lock['direction'] == side_upper: branches conditionally.
7. Calls self.logger.warning(...) for its side effect.
8. **return** False.

Code (copy-paste safe)

```
def check_and_lock_direction(self, account_number: str, side: str) -> bool:
    """Enforce one-direction-per-day rule *per account*.

    Args:
        account_number: The Tradovate/TopStep account identifier.
        side: 'BUY' or 'SELL'.

    Returns:
        True if the trade is allowed, False if blocked.
```

```

"""
if not account_number or account_number in ("Unknown", "Not Connected"):
    # Can't enforce without a valid account – allow but warn
    self.logger.warning("Direction lock skipped: no valid account number")
    return True

today = datetime.date.today()
side_upper = side.upper()
lock = self._account_direction_locks.get(account_number)

# New day → reset
if lock is None or lock["date"] != today:
    self._account_direction_locks[account_number] = {"date": today, "direction": side_upper}
    self.logger.info(
        f"[RULE] Daily direction LOCKED to {side_upper} for account {account_number} ({today})")
    return True

# Same day – check direction
if lock["direction"] == side_upper:
    return True

self.logger.warning(
    f"[BLOCK] Direction violation for {account_number}: "
    f"locked={lock['direction']}, attempted={side_upper}"
)
return False

```

Method: PropFirmManager.get_locked_direction

```
def get_locked_direction(self, account_number: str) -> Optional[str]
```

Return the locked direction for an account today, or None.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `account_number: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> Optional[str]`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.

Step by step (top-level body)

1. **if not account_number:** branches conditionally.
2. Assigns `today = datetime.date.today()`.
3. Assigns `lock = self._account_direction_locks.get(account_number)`.
4. **if lock and lock['date'] == today:** branches conditionally.
5. **return None.**

Code (copy-paste safe)

```

def get_locked_direction(self, account_number: str) -> Optional[str]:
    """Return the locked direction for an account today, or None."""
    if not account_number:
        return None
    today = datetime.date.today()

```

```

lock = self._account_direction_locks.get(account_number)
if lock and lock["date"] == today:
    return lock["direction"]
return None

```

Method: PropFirmManager.reset_direction_lock

```
def reset_direction_lock(self, account_number: str) -> None
```

Manually reset the direction lock for a specific account.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `account_number: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> None`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Calls `self._account_direction_locks.pop(...)` for its side effect.
2. Calls `self.logger.info(...)` for its side effect.

Code (copy-paste safe)

```

def reset_direction_lock(self, account_number: str) -> None:
    """Manually reset the direction lock for a specific account."""
    self._account_direction_locks.pop(account_number, None)
    self.logger.info(f"[RULE] Direction lock reset for account {account_number}")

```

Method: PropFirmManager.detect_firm_from_account

```
def detect_firm_from_account(self, account_number: str) -> Optional[str]
```

Detect the prop firm from a Tradovate/TopStep account number. Returns the UI-facing blueprint name (e.g. 'MFFU_Flex', 'Funded Next') or None if detection fails.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `account_number: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> Optional[str]`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `detected_code = self.detect_prop_firm(account_number)`.
2. **if** `detected_code` is `None`: branches conditionally.

3. Assigns `blueprint = self.DETECTED_TO_BLUEPRINT.get(detected_code)`.

4. `if blueprint`: branches conditionally (with an `else/elif` arm).

5. `return blueprint`.

Code (copy-paste safe)

```
def detect_firm_from_account(self, account_number: str) -> Optional[str]:
    """Detect the prop firm from a Tradovate/TopStep account number.

    Returns the UI-facing blueprint name (e.g. 'MFFU_Flex', 'Funded Next')
    or None if detection fails.
    """
    detected_code = self.detect_prop_firm(account_number)
    if detected_code is None:
        self.logger.warning(f"Account '{account_number}' – prop firm could not be identified")
        return None
    blueprint = self.DETECTED_TO_BLUEPRINT.get(detected_code)
    if blueprint:
        self.logger.info(
            f"Account '{account_number}' detected as '{detected_code}' → blueprint '{blueprint}'")
    else:
        self.logger.warning(
            f"Account '{account_number}' detected as '{detected_code}' but no blueprint mapping exists")
    return blueprint
```

Method: `PropFirmManager.validate_blueprint_match`

```
def validate_blueprint_match(self, account_number: str, selected_blueprint: str) -> Tuple[bool, Optional[str]]:
```

Check whether the selected blueprint matches the connected account. Args: `account_number`: Account identifier from Tradovate/TopStep. `selected_blueprint`: The blueprint currently selected in the UI. Returns: (`is_match`, `detected_blueprint`) - `is_match` is True when they are compatible. - `detected_blueprint` is the blueprint we think the account belongs to.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `account_number: str`, `selected_blueprint: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> Tuple[bool, Optional[str]]`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `detected = self.detect_firm_from_account(account_number)`.

2. `if detected is None`: branches conditionally.

3. Assigns `compatible_groups = [{'MFFU', 'MFFU_Flex'}, {'Alpha Futures', 'AlphaFutures G...}]`.

4. `for group in compatible_groups`: iterates.

5. Assigns `is_match = detected == selected_blueprint`.

6. `if not is_match`: branches conditionally.

7. `return (is_match, detected)`.

Code (copy-paste safe)

```
def validate_blueprint_match(self, account_number: str, selected_blueprint: str) -> Tuple[bool, Optional[str]]:
    """Check whether the selected blueprint matches the connected account.

    Args:
        account_number: Account identifier from Tradovate/TopStep.
        selected_blueprint: The blueprint currently selected in the UI.

    Returns:
        (is_match, detected_blueprint)
        - is_match is True when they are compatible.
        - detected_blueprint is the blueprint we think the account belongs to.
    """
    detected = self.detect_firm_from_account(account_number)
    if detected is None:
        # Detection inconclusive – allow but warn
        return True, None

    # Treat MFFU and MFFU_Flex as compatible
    compatible_groups = [
        {"MFFU", "MFFU_Flex"},
        {"Alpha Futures", "AlphaFutures GC"},
    ]
    for group in compatible_groups:
        if detected in group and selected_blueprint in group:
            return True, detected

    is_match = (detected == selected_blueprint)
    if not is_match:
        self.logger.warning(
            f"Blueprint MISMATCH: account '{account_number}' belongs to '{detected}' "
            f"but '{selected_blueprint}' is selected"
        )
    return is_match, detected
```

Checkpoint — what works now. The scrapers can log into each prop firm and read account state. Run any of them in isolation and watch a Chrome window open, log in, and navigate to the dashboard page. The data they extract is still in-memory only — it will be pushed to the server in the next phase.

Chapter 9 — Phase 7 — Pushing data to the dashboard

The desktop companion runs on the trader's machine. The web dashboard runs on a server. They communicate through an HTTP ingestion API. In this phase we build the outbound HTTP client and the periodic sync that pushes MT5 deal history to the server.

Files we build in this phase, in order:

- `trader_companion/push_data.py`
- `trader_companion/mt5_dashboard_sync.py`

Python concepts you'll meet in this phase

- f-strings (2) · Exceptions (2) · Dunder methods (1) · Classes and objects (1) · Module-level state (1) · Comprehensions (1)

Each is explained in Chapter 2 (the primer). The number in parentheses is how many of this phase's files use that concept.

Step 9.1 — Build `trader_companion/push_data.py`

What to do. Create the file `trader_companion/push_data.py` in your project root and paste in the code shown in this step. The file is **194** lines long; we walk through it piece by piece below.

Depends on. This file imports from internal modules built in earlier steps: `trader_companion`. If any of those imports fail, jump back to the step that introduced them.

What this file is for. Outbound HTTP client for the dashboard's ingestion API.

`trader_companion/push_data.py`

194 loc · 0 classes · 4 functions · 7 imports

Outbound HTTP client for the dashboard's ingestion API. It defines **4** module-level functions, across **194** lines. Module docstring: *MT5 Data Pusher - Command Line Interface*

Module docstring

MT5 Data Pusher - Command Line Interface Simple script for traders to push data to the dashboard from command line. NO API KEY REQUIRED - just your email address!

Python concepts demonstrated in this module

- Exceptions · f-strings

Imports — what each one is for

```
import sys
```

Why imported. Access to the running interpreter — argv, stdin/stdout, exit codes, the import search path, and the platform name.

Used in this file. sys.exit, sys.path, sys.path.insert

Example. sys.exit(1) # terminate the process with a non-zero status

Other common scenarios.

- sys.argv[1:] reads the command-line arguments
- sys.path.insert(0, 'extra') prepends to the import search path
- sys.stderr.write(msg) writes to standard error
- sys.platform tells you 'win32', 'linux', or 'darwin'
- sys.modules is the global cache of already-imported modules

```
import os
```

Why imported. Operating-system interface. Path manipulation, environment variables, working-directory operations, exit codes, and thin wrappers around POSIX/Windows syscalls.

Used in this file. os.path, os.path.abspath, os.path.dirname

Example. os.path.join(root, 'subdir', 'file.txt') # joins with the OS-correct separator

Other common scenarios.

- os.environ.get('API_KEY') to read environment variables
- os.makedirs(path, exist_ok=True) to create nested directories
- os.listdir(path) to list a directory's contents
- os.remove(path) / os.rename(src, dst) to delete or move a file
- os.getcwd() / os.chdir(path) to inspect or change the working dir

```
import json
```

Why imported. JSON encoding and decoding — the standard wire format for config files, API payloads, and inter-process messages.

Used in this file. json.dumps

Example. json.loads(response.text) # parse a JSON string to dict

Other common scenarios.

- json.dumps(obj, indent=2) for pretty-printing
- json.dump(obj, file) / json.load(file) for file I/O
- default=str handles non-serialisable types like datetime
- object_hook= customises decoding (e.g., to dataclass instances)

```
import gzip as _gzip
```

Why imported. Read/write gzip-compressed streams.

Used in this file. `_gzip.compress`

Example. with `gzip.open('logs.gz', 'rt')` as `f`: ...

Other common scenarios.

- `gzip.compress(b)` / `gzip.decompress(b)` for in-memory blobs
- Pair with `shutil.copyfileobj` for streaming compression

```
import argparse
```

Why imported. Command-line argument parsing. Generates `--help`, type-coerces values, and dispatches sub-commands.

Used in this file. `argparse.ArgumentParser`

Example. `parser.add_argument('--verbose', action='store_true')`

Other common scenarios.

- `type=int` / `type=Path` coerces the value
- subparsers for git-style sub-commands
- `ArgumentDefaultsHelpFormatter` shows defaults in `--help`

```
import requests
```

Why imported. The de-facto Python HTTP client. Synchronous; trades raw speed for an extremely friendly API.

Used in this file. `requests.post`

Example. `requests.get(url, timeout=10).json()`

Other common scenarios.

- `Session()` reuses TCP connections across calls
- `params=`, `headers=`, `json=`, `data=` for query/body shapes
- `raise_for_status()` turns 4xx/5xx into an exception
- `stream=True` for large downloads

```
from trader_companion.trader_app import MT5DataPusher
```

Why imported. Internal package — the desktop trader companion. See chapter on `trader_companion/`.

Used in this file. `MT5DataPusher`

Functions

```
def lookup_client(url, email)
```

Lookup client hierarchy from email - NO API KEY.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't

have to handle it.

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def lookup_client(url, email):
    """Lookup client hierarchy from email - NO API KEY."""
    try:
        response = requests.post(
            f"{url.rstrip('/')}/api/client/auth",
            json={"email": email},
            headers={"Content-Type": "application/json"},
            timeout=15
        )

        if response.status_code == 200:
            data = response.json()
            if data.get("status") == "success":
                return data.get("identity", {}), None
            else:
                return None, data.get("message", "Client not found")
        else:
            return None, f"API Error: {response.status_code}"
    except Exception as e:
        return None, str(e)
```

```
def push_data(url, email, account, positions, deals, statistics)
```

Push data to dashboard - NO API KEY. Sends gzip-compressed JSON.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def push_data(url, email, account, positions, deals, statistics):
    """Push data to dashboard - NO API KEY. Sends gzip-compressed JSON."""
    try:
        payload = {
            "email": email,
            "account": account,
            "positions": positions,
            "deals": deals,
            "statistics": statistics,
```

```

        "evaluations": [],
        "dropdown_options": {}
    }
    raw = json.dumps(payload).encode('utf-8')
    compressed = _gzip.compress(raw, compresslevel=6)
    response = requests.post(
        f"{url.rstrip('/')}/api/client/push",
        data=compressed,
        headers={"Content-Type": "application/json", "Content-Encoding": "gzip"},
        timeout=30
    )

    if response.status_code == 200:
        data = response.json()
        if data.get("status") == "success":
            return True, data.get("message", "Data pushed successfully")
        else:
            return False, data.get("message", "Push failed")
    else:
        return False, f"HTTP {response.status_code}"
except Exception as e:
    return False, str(e)

```

```
def migrate_sheet(url, email, sheet_url)
```

Migrate data from Google Sheets.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def migrate_sheet(url, email, sheet_url):
    """Migrate data from Google Sheets."""
    try:
        response = requests.post(
            f"{url.rstrip('/')}/api/client/migrate_sheet",
            json={"email": email, "sheet_url": sheet_url},
            headers={"Content-Type": "application/json"},
            timeout=60
        )

        if response.status_code == 200:
            data = response.json()
            if data.get("status") == "success":
                return True, f"Imported {data.get('records_imported', 0)} records"
            else:
                return False, data.get("message", "Migration failed")
        else:
            return False, f"HTTP {response.status_code}"
    
```

```
except Exception as e:
    return False, str(e)
```

```
def main()
```

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `parser = argparse.ArgumentParser(description='Push MT5 data to Tra....`
2. Calls `parser.add_argument(...)` for its side effect.
3. Calls `parser.add_argument(...)` for its side effect.
4. Calls `parser.add_argument(...)` for its side effect.
5. Calls `parser.add_argument(...)` for its side effect.
6. Calls `parser.add_argument(...)` for its side effect.
7. Calls `parser.add_argument(...)` for its side effect.
8. Calls `parser.add_argument(...)` for its side effect.
9. Assigns `args = parser.parse_args()`.
10. Calls `print(...)` for its side effect.
11. Calls `print(...)` for its side effect.
12. Calls `print(...)` for its side effect.
13. Calls `print(...)` for its side effect.
14. Assigns `(client_info, error) = lookup_client(args.url, args.email)`.
15. ... and 22 more statement(s) in the body.

Code (copy-paste safe)

```
def main():
    parser = argparse.ArgumentParser(description='Push MT5 data to Trading Dashboard (No API Key Required)')
    parser.add_argument('--url', default='https://www.tradeopss.com', help='Dashboard URL')
    parser.add_argument('--email', required=True, help='Your registered client email')
    parser.add_argument('--sheet', help='Google Sheet URL to migrate data from')
    parser.add_argument('--mt5-login', help='MT5 account login')
    parser.add_argument('--mt5-password', help='MT5 account password')
    parser.add_argument('--mt5-server', help='MT5 server name')
    parser.add_argument('--days', type=int, default=30, help='Days of history to fetch')

    args = parser.parse_args()

    print("=" * 50)
    print("MT5 Data Pusher (No API Key Required!)")
    print("=" * 50)

    # Lookup client by email first
    print(f"\n[*] Looking up client: {args.email}")
    client_info, error = lookup_client(args.url, args.email)
```

```

if error:
    print(f"\n[X] Lookup failed: {error}")
    print("    Make sure your email is registered in the system.")
    sys.exit(1)

client_name = client_info.get('client', '')
trader_name = client_info.get('trader', '')
admin_name = client_info.get('admin', '')
category = client_info.get('category', '')

print(f"    ✓ Client: {client_name}")
print(f"    ✓ Trader: {trader_name}")
print(f"    ✓ Admin: {admin_name}")
print(f"    ✓ Category: {category}")

# If sheet URL provided, migrate from sheets
if args.sheet:
    print(f"\n[*] Migrating data from Google Sheets...")
    success, msg = migrate_sheet(args.url, args.email, args.sheet)
    if success:
        print(f"\n[✓] {msg}")
    else:
        print(f"\n[X] {msg}")
        sys.exit(1)
    print("\n[*] Done!")
    return

# Otherwise, push MT5 data
pusher = MT5DataPusher(args.url)

# Connect to MT5 if credentials provided
if args.mt5_login and args.mt5_password and args.mt5_server:
    print(f"\n[*] Connecting to MT5 account {args.mt5_login}...")
    success, msg = pusher.connect_mt5(args.mt5_login, args.mt5_password, args.mt5_server)
    print(f"    {msg}")

    if not success:
        print("\n[!] Failed to connect to MT5. Exiting.")
        sys.exit(1)
else:
    # Try to connect to already running MT5
    print("\n[*] Connecting to running MT5 terminal...")
    success, msg = pusher.connect_mt5()
    print(f"    {msg}")

    if not success:
        print("\n[!] No MT5 connection. Provide --mt5-login, --mt5-password, --mt5-server")
        print("    Or use --sheet to migrate from Google Sheets instead.")
        sys.exit(1)

# Get data
account = pusher.get_account_info() or {}
if account:
    print(f"\n[+] Account: #{account.get('login', 'N/A')} @ {account.get('server', 'N/A')}")
    print(f"    Balance: {account.get('balance', 0)} {account.get('currency', '')}")
    print(f"    Equity: {account.get('equity', 0)} {account.get('currency', '')}")

positions = pusher.get_positions()
deals = pusher.get_deals(days=args.days)
statistics = pusher.calculate_statistics(deals)

```

```

# Push data
print(f"\n[*] Pushing data for client: {client_name}")
success, msg = push_data(args.url, args.email, account, positions, deals, statistics)

if success:
    print(f"\n[✓] {msg}")
else:
    print(f"\n[X] {msg}")
    sys.exit(1)

# Disconnect
pusher.disconnect_mt5()
print("\n[*] Done!")

```

Step 9.2 — Build trader_companion/mt5_dashboard_sync.py

What to do. Create the file `trader_companion/mt5_dashboard_sync.py` in your project root and paste in the code shown in this step. The file is **840** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from `requirements.txt`.

What this file is for. Periodically pushes the local MT5 deal history to the dashboard's database.

trader_companion/mt5_dashboard_sync.py

840 loc · 1 classes · 1 functions · 5 imports

Periodically pushes the local MT5 deal history to the dashboard's database. It defines **1** class and **1** module-level function, across **840** lines. Module docstring: *MT5 Dashboard Sync Service*

Module docstring

MT5 Dashboard Sync Service Syncs MT5 trading history to the TradeConnector Dashboard

Python concepts demonstrated in this module

- Classes and objects
- Comprehensions
- Dunder methods
- Exceptions
- f-strings

Module-level state

Imports — what each one is for

```
import MetaTrader5 as mt5
```

Why imported. Official MetaQuotes Python API for MT5. Connects to a local terminal, places orders, reads tick/bar history.

Used in this file. mt5.DEAL_ENTRY_IN, mt5.DEAL_ENTRY_OUT, mt5.DEAL_TYPE_BUY, mt5.DEAL_TYPE_SELL, mt5.account_info, mt5.history_deals_get, mt5.initialize, mt5.last_error

Example. mt5.initialize(); mt5.order_send(request)

Other common scenarios.

- mt5.symbols_get() / mt5.symbol_info(symbol)
- mt5.history_deals_get(from_, to_) returns closed deals
- mt5.account_info() returns balance/equity/currency
- Always mt5.shutdown() in a finally to release the terminal

```
import requests
```

Why imported. The de-facto Python HTTP client. Synchronous; trades raw speed for an extremely friendly API.

Used in this file. requests.exceptions, requests.exceptions.ConnectionError, requests.exceptions.Timeout, requests.get, requests.post

Example. requests.get(url, timeout=10).json()

Other common scenarios.

- Session() reuses TCP connections across calls
- params=, headers=, json=, data= for query/body shapes
- raise_for_status() turns 4xx/5xx into an exception
- stream=True for large downloads

```
import logging
```

Why imported. Structured, levelled logging. Replaces print() with filterable, formattable output that can route to files, stderr, syslog, or HTTP endpoints.

Used in this file. logging.error, logging.info, logging.warning

Example. logging.getLogger(__name__).info('connected to %s', host)

Other common scenarios.

- .debug() / .info() / .warning() / .error() / .exception()
- logger.exception() captures the current traceback automatically
- logging.basicConfig(level=logging.INFO) for quick setup
- Handlers and Formatters route logs to files / streams / syslog

```
from datetime import datetime
from datetime import timedelta
```

Why imported. Dates, times, durations. The fundamental temporal types: date, time, datetime, timedelta, timezone.

Used in this file. datetime, timedelta

Example. datetime.now() - timedelta(days=7) # one week ago

Other common scenarios.

- `datetime.strptime(s, '%Y-%m-%d')` parses a string
- `datetime.strftime(fmt)` formats one
- `datetime.fromtimestamp(n)` converts a Unix epoch
- `datetime.utcnow()` for UTC (better: `datetime.now(timezone.utc)`)

`import json`

Why imported. JSON encoding and decoding — the standard wire format for config files, API payloads, and inter-process messages.

Used in this file. *No direct attribute access detected — likely imported for side effects, re-export, or string references.*

Example. `json.loads(response.text)` # parse a JSON string to dict

Other common scenarios.

- `json.dumps(obj, indent=2)` for pretty-printing
- `json.dump(obj, file)` / `json.load(file)` for file I/O
- `default=str` handles non-serialisable types like `datetime`
- `object_hook=` customises decoding (e.g., to `dataclass` instances)

Classes

```
class MT5DashboardSync
```

Code (copy-paste safe)

```
class MT5DashboardSync:
    def __init__(self, dashboard_url="http://localhost:5000/api", user_email=None):
        """
        Initialize MT5 Dashboard Sync

        Args:
            dashboard_url: Base URL for dashboard API
            user_email: User email for authentication
        """
        self.dashboard_url = dashboard_url.rstrip('/')
        self.user_email = user_email
        self.auth_token = None
        self._is_connected = False
        logging.info(f"[MT5 SYNC] Initialized for email: {user_email}")

    def set_auth_token(self, token):
        """Set authentication token from dashboard login"""
        self.auth_token = token
        logging.info("[MT5 SYNC] Authentication token set")

    def check_dashboard_health(self):
        """
        Check if dashboard backend is running and healthy

        Returns:
            dict: {'connected': bool, 'status': str, 'message': str}
```

```

"""
try:
    # Check backend health endpoint
    backend_url = self.dashboard_url.replace('/api', '')
    health_url = f"{backend_url}/health"

    response = requests.get(health_url, timeout=3)

    if response.status_code == 200:
        data = response.json()
        if data.get('status') == 'healthy':
            self._is_connected = True
            return {
                'connected': True,
                'status': 'healthy',
                'message': 'Dashboard connected',
                'version': data.get('version', 'unknown')
            }

        self._is_connected = False
        return {
            'connected': False,
            'status': 'unhealthy',
            'message': f'Dashboard responded with status {response.status_code}'
        }

except requests.exceptions.Timeout:
    self._is_connected = False
    return {
        'connected': False,
        'status': 'timeout',
        'message': 'Dashboard connection timeout (3s)'
    }
except requests.exceptions.ConnectionError:
    self._is_connected = False
    return {
        'connected': False,
        'status': 'error',
        'message': 'Cannot connect to dashboard. Is it running?'
    }
except Exception as e:
    self._is_connected = False
    return {
        'connected': False,
        'status': 'error',
        'message': f'Health check failed: {str(e)}'
    }

def is_connected(self):
    """Check if dashboard is connected"""
    return self._is_connected

def register_mt5_account(self, mt5_login, mt5_name, mt5_server, broker=None):
    """
    Log MT5 connection - MT5 is only used to extract Tradovate/TopStep data

    MT5 account is NOT registered in dashboard. MT5 is simply a data source
    for extracting prop firm account numbers and hedge amounts.

    Args:

```

```

        mt5_login: MT5 account login number
        mt5_name: MT5 account name
        mt5_server: MT5 server name
        broker: Broker name (optional)

Returns:
    dict: Always returns success
"""
client_name = f"{mt5_login}: {mt5_name}"
logging.info(f"[MT5 DATA SOURCE] MT5 connected: {client_name}")
logging.info(f"[MT5 DATA SOURCE] MT5 will be used to extract Tradovate/TopStep account data")
return {'success': True, 'message': 'MT5 connected as data source'}

def login(self, email, password):
    """
    Login to dashboard and get authentication token

    Args:
        email: User email
        password: User password

    Returns:
        dict: {'success': bool, 'token': str, 'error': str}
    """
    try:
        # Demo mode credentials (matches dashboard frontend)
        demo_credentials = {
            'harryodhiambo17@gmail.com': 'demo123',
            'admin@tradeconnector.com': 'admin123',
            'test@example.com': 'test123'
        }

        # Check if using demo credentials
        if email in demo_credentials and demo_credentials[email] == password:
            # Generate demo token
            demo_token = f'demo-token-{int(datetime.now().timestamp())}'
            self.auth_token = demo_token
            self.user_email = email

            logging.info(f"[MT5 SYNC] Demo login successful for {email}")
            return {
                'success': True,
                'token': demo_token,
                'user': {
                    'email': email,
                    'firstName': 'Harry' if email == 'harryodhiambo17@gmail.com' else 'Demo',
                    'lastName': 'Odhiambo' if email == 'harryodhiambo17@gmail.com' else 'User'
                }
            }

        # Try real API call if not demo credentials
        url = f"{self.dashboard_url}/auth/login"
        payload = {
            'email': email,
            'password': password
        }

        logging.info(f"[MT5 SYNC] Attempting login to {url}...")
        response = requests.post(url, json=payload, timeout=30)

        if response.status_code == 200:

```

```

        data = response.json()
        token = data.get('token')

        if token:
            self.auth_token = token
            self.user_email = email
            logging.info(f"[MT5 SYNC] Login successful for {email}")
            return {
                'success': True,
                'token': token,
                'user': data.get('user', {})
            }
        else:
            return {
                'success': False,
                'error': 'No token received from server'
            }
    else:
        error_data = response.json() if response.content else {}
        error_msg = error_data.get('error', f'Login failed with status {response.status_code}')

        # Add helpful message about demo credentials
        if response.status_code == 401:
            error_msg += '\n\nTry demo credentials:\nEmail: harryodhiambo17@gmail.com\nPassw

        logging.error(f"[MT5 SYNC] Login failed: {error_msg}")
        return {
            'success': False,
            'error': error_msg
        }

except requests.exceptions.Timeout:
    logging.error("[MT5 SYNC] Login request timeout")
    return {
        'success': False,
        'error': 'Login request timeout (30s). Dashboard may be slow or not responding.'
    }
except requests.exceptions.ConnectionError as e:
    logging.error(f"[MT5 SYNC] Connection error: {e}")
    # If connection fails, suggest demo credentials as fallback
    return {
        'success': False,
        'error': f'Cannot connect to dashboard. Try demo credentials instead:\nEmail: harryodhia

    }
except Exception as e:
    logging.error(f"[MT5 SYNC] Login error: {e}")
    return {
        'success': False,
        'error': f'Login error: {str(e)}'
    }
}

def get_mt5_history_by_account(self, account_number=None, days_back=7):
    """
    Fetch MT5 trading history filtered by account number (from comment)

    Args:
        account_number: Prop firm account number to filter by (optional)
        days_back: Number of days to look back for history (default: 7)

    Returns:

```

```

List of trade dictionaries filtered by account number
"""
try:
    logging.info(f"[MT5 SYNC] Fetching history for last {days_back} days...")

    # Check MT5 initialization status FIRST
    terminal_info = mt5.terminal_info()
    if terminal_info is None:
        logging.error("[MT5 SYNC] MT5 terminal_info() returned None - MT5 not initialized")
        if not mt5.initialize():
            logging.error("[MT5 SYNC] Failed to initialize MT5")
            return []
        else:
            logging.info("[MT5 SYNC] MT5 initialized successfully")
    else:
        logging.info(f"[MT5 SYNC] MT5 is initialized: {terminal_info.connected}")

    # Calculate date range
    end_date = datetime.now()
    start_date = end_date - timedelta(days=days_back)

    # Get deals (closed trades)
    logging.info(
        f"[MT5 SYNC] Requesting deals from {start_date.strftime('%Y-%m-%d %H:%M:%S')} to {end_date.strftime('%Y-%m-%d %H:%M:%S')}"
    )
    deals = mt5.history_deals_get(start_date, end_date)

    if deals is None:
        logging.warning("[MT5 SYNC] MT5 returned None for deals - check MT5 connection")
        logging.warning(f"[MT5 SYNC] MT5 last error: {mt5.last_error()}")
        return []

    if len(deals) == 0:
        logging.warning("[MT5 SYNC] No deals found in specified period")
        logging.warning(
            f"[MT5 SYNC] Checked period: {start_date.strftime('%Y-%m-%d')} to {end_date.strftime('%Y-%m-%d %H:%M:%S')}"
        )
        logging.warning(f"[MT5 SYNC] Try checking MT5 'Account History' tab to verify trades exist")
        return []

    logging.info(f"[MT5 SYNC] Found {len(deals)} deals, processing into trades...")
    logging.info(
        f"[MT5 SYNC] First deal sample: ticket={deals[0].ticket if deals else 'N/A'}, time={deals[0].time if deals else 'N/A'}"
    )

    # Process deals into trade format
    trades = []
    deal_pairs = {} # Group entry and exit deals

    for deal in deals:
        # Skip balance operations
        if deal.type not in [mt5.DEAL_TYPE_BUY, mt5.DEAL_TYPE_SELL]:
            continue

        # Filter by account number if specified (comment contains account number)
        if account_number:
            comment = deal.comment or ''
            if account_number not in comment:
                continue

        position_id = deal.position_id

        if position_id not in deal_pairs:

```

```

        deal_pairs[position_id] = {'entry': None, 'exit': None}

    # Determine if this is entry or exit
    if deal.entry == mt5.DEAL_ENTRY_IN:
        deal_pairs[position_id]['entry'] = deal
    elif deal.entry == mt5.DEAL_ENTRY_OUT:
        deal_pairs[position_id]['exit'] = deal

    # Combine entry/exit pairs into complete trades
    for position_id, pair in deal_pairs.items():
        if pair['entry'] and pair['exit']:
            entry = pair['entry']
            exit_deal = pair['exit']

            # Extract account info from comment
            comment = entry.comment or ''
            account_info = self._extract_account_from_comment(comment)

            # Debug log first few comments to understand format
            if len(trades) < 5:
                logging.info(
                    f"[MT5 SYNC DEBUG] Trade {position_id} comment: '{comment}' -> Account: '{account_info['accountNumber']}'"
                )

            trade = {
                'ticket': position_id,
                'symbol': entry.symbol,
                'type': 'buy' if entry.type == mt5.DEAL_TYPE_BUY else 'sell',
                'volume': entry.volume,
                'openPrice': entry.price,
                'closePrice': exit_deal.price,
                'openTime': datetime.fromtimestamp(entry.time).isoformat(),
                'closeTime': datetime.fromtimestamp(exit_deal.time).isoformat(),
                'profit': exit_deal.profit,
                'commission': entry.commission + exit_deal.commission,
                'swap': entry.swap + exit_deal.swap,
                'comment': comment,
                'accountNumber': account_info['accountNumber'],
                'propFirm': account_info['propFirm']
            }

            trades.append(trade)

    logging.info(f"[MT5 SYNC] Fetched {len(trades)} completed trades")
    if account_number:
        logging.info(f"[MT5 SYNC] Filtered for account: {account_number}")
    return trades

except Exception as e:
    logging.error(f"[MT5 SYNC] Error fetching MT5 history: {e}")
    return []

def _extract_account_from_comment(self, comment):
    """
    Extract account number and prop firm from MT5 comment

    Supported formats:
    - TopStep: 'PA-TS123456', 'TS123456', 'PA123456', or variations
    - Tradovate: 'TRDV123456', 'TV123456', or variations
    - Generic: Treats any alphanumeric string as account number
    - Empty: Returns empty account number (trade will be skipped)
    """

```

```

- With Phase: 'ACCOUNT_CH1', 'ACCOUNT_FD1', etc. (phase suffix is stripped)

Returns:
    dict: {'accountNumber': str, 'propFirm': str}
"""
if not comment or not comment.strip():
    return {'accountNumber': '', 'propFirm': 'Unknown'}

comment = comment.strip()

# Strip phase abbreviation suffix if present (e.g., _CH1, _FD2, _DD3, _FA)
# This must be done BEFORE other processing
import re
if '_' in comment:
    # Check for numbered formats (_CH1-4, _FD1-4, _DD1-4)
    if re.search(r'_(CH|FD|DD)\d+$', comment):
        comment = re.sub(r'_(CH|FD|DD)\d+$', '', comment)
    # Check for simple farming format: _FA
    elif comment.endswith('_FA'):
        comment = comment[:-3]
    # Check for unknown phase marker
    elif comment.endswith('_UNK'):
        comment = comment[:-4]

# Remove common noise characters
clean_comment = comment.replace('#', '').replace('/', '').strip()

# Quick heuristic: detect Other-mode combined comments that start
# with an account identifier then an underscore and a prop-firm token
# Example: '123456_PropFirm_Challenge_T1' -> account=123456, propFirm=PropFirm
try:
    parts = clean_comment.split('_')
    if len(parts) >= 2:
        first = parts[0].strip()
        second = parts[1].strip()
        # First part looks like a numeric account or contains known prefixes
        if re.match(r'^[0-9]{3,}$', first) or any(k in first.upper() for k in ['TRDV', 'TV', 'TS', 'PA']):
            return {'accountNumber': first, 'propFirm': second}
except Exception:
    # If anything goes wrong with this heuristic, continue with normal parsing
    pass

# Check for TopStep patterns (PA-, TS, TopStep keywords)
if any(keyword in clean_comment.upper() for keyword in ['TS', 'TOPSTEP', 'PA-', 'PA ']):
    # Extract just the numeric part or full account number
    import re
    # Try to find PA-XXXXX or TS-XXXXX pattern
    match = re.search(r'(?:(PA-?|TS-?)([A-Z0-9]+)', clean_comment.upper())
    if match:
        account_num = match.group(1)
    else:
        # Remove all prefix keywords and get what's left
        account_num = clean_comment.upper()
        for prefix in ['PA-', 'PA ', 'TS-', 'TS ', 'TOPSTEP-', 'TOPSTEP ']:
            account_num = account_num.replace(prefix, '')
        account_num = account_num.strip()

    return {'accountNumber': account_num, 'propFirm': 'TopStep'}

# Check for Tradovate patterns (TRDV, TV, Tradovate keywords)

```



```

if any(keyword in clean_comment.upper() for keyword in ['TRDV', 'TV', 'TRADOVATE']):
    import re
    # Try to find TRDV-XXXXX or TV-XXXXX pattern
    match = re.search(r'(?:(TRDV-?|TV-?)([A-Z0-9]+)', clean_comment.upper())
    if match:
        account_num = match.group(1)
    else:
        # Remove all prefix keywords
        account_num = clean_comment.upper()
        for prefix in ['TRDV-', 'TRDV ', 'TV-', 'TV ', 'TRADOVATE-', 'TRADOVATE ']:
            account_num = account_num.replace(prefix, '')
        account_num = account_num.strip()

    return {'accountNumber': account_num, 'propFirm': 'Tradovate'}

# If no specific pattern but comment exists, use it as account number
# This handles cases where account number is just digits or simple format
if clean_comment and len(clean_comment) >= 3: # Minimum reasonable account number length
    return {'accountNumber': clean_comment, 'propFirm': 'Auto-detected'}

# Comment too short or invalid
return {'accountNumber': '', 'propFirm': 'Unknown'}

def get_mt5_client_info(self):
    """
    Get MT5 account info including client name

    Returns:
        dict: {'login': int, 'name': str, 'clientName': str}
              clientName format: "login: name" (e.g., "3053752: Tyler Turner")
    """
    try:
        if not mt5.initialize():
            logging.error("[MT5 SYNC] Failed to initialize MT5")
            return None

        account_info = mt5.account_info()
        if not account_info:
            logging.error("[MT5 SYNC] Failed to get account info")
            return None

        client_info = {
            'login': account_info.login,
            'name': account_info.name,
            'clientName': f"{account_info.login}: {account_info.name}"
        }

        logging.info(f"[MT5 SYNC] Client: {client_info['clientName']}")
        return client_info

    except Exception as e:
        logging.error(f"[MT5 SYNC] Error getting client info: {e}")
        return None

def _determine_phase_from_lotsize(self, lot_size):
    """
    Determine trading phase based on lot size

    Different phases use different lot sizes:
    - Evaluation: Smaller lots (e.g., 1.0, 2.0)

```

```

- Funded: Medium lots (e.g., 3.0, 4.0, 5.0)
- Farming: Larger lots (e.g., 6.0+)

Args:
    lot_size: MT5 lot size

Returns:
    Tuple of (phase, hedge_column_number)
    """
# These thresholds can be customized based on your strategy
if lot_size <= 2.0:
    # Evaluation phase - lot sizes 1.0, 2.0
    hedge_num = int(lot_size) # 1.0 -> hedge 1, 2.0 -> hedge 2, etc.
    return 'evaluation', hedge_num
elif lot_size <= 5.0:
    # Funded phase - lot sizes 3.0, 4.0, 5.0
    hedge_num = int(lot_size - 2.0) # 3.0 -> hedge 1, 4.0 -> hedge 2, 5.0 -> hedge 3
    return 'funded', hedge_num
else:
    # Farming phase - lot sizes 6.0+
    hedge_num = int(lot_size - 5.0) # 6.0 -> hedge 1, 7.0 -> hedge 2, etc.
    return 'farming', hedge_num

def get_mt5_history(self, days_back=30):
    """
    Fetch all MT5 trading history (backward compatibility)

    Args:
        days_back: Number of days to look back for history

    Returns:
        List of trade dictionaries
    """
    return self.get_mt5_history_by_account(account_number=None, days_back=days_back)

# MT5 account matching removed - MT5 is only used to extract Tradovate/TopStep account numbers from t

# Account lookup removed - MT5 only extracts trade data, dashboard auto-creates accounts

def auto_discover_and_sync(self):
    """
    Automatically discover all prop firm accounts from MT5 and sync to dashboard.

    Process:
    1. Fetch all MT5 trades
    2. Extract unique account numbers from comments
    3. Group trades by account and lot size to determine phase
    4. Auto-create/update accounts in dashboard
    5. Sync all hedge results

    Returns:
        dict: Summary of discovery and sync results
    """
    try:
        if not self.user_email or not self.auth_token:
            return {
                'success': False,
                'error': 'Not authenticated. Please login first.'
            }

        logging.info("[MT5 AUTO-DISCOVER] Starting automatic account discovery...")

```

```

# Get MT5 account info
client_info = self.get_mt5_client_info()
if not client_info:
    return {
        'success': False,
        'error': 'Failed to get MT5 account info'
    }

mt5_account = f"{client_info['login']}: {client_info['name']}"
logging.info(f"[MT5 AUTO-DISCOVER] MT5 Hedge Account: {mt5_account}")

# Fetch all MT5 trades (last 7 days for faster processing)
logging.info("[MT5 AUTO-DISCOVER] Fetching MT5 trade history...")
logging.info(
    f"[MT5 AUTO-DISCOVER] Calling get_mt5_history_by_account(account_number=None, days_back=7)"
)
all_trades = self.get_mt5_history_by_account(account_number=None, days_back=7)
logging.info(
    f"[MT5 AUTO-DISCOVER] get_mt5_history_by_account() returned {len(all_trades)} if all_trades"
)

if not all_trades:
    logging.warning(
        "[MT5 AUTO-DISCOVER] No trades found in MT5 history - check MT5 connection and trade
    )
    return {
        'success': False,
        'error': 'No trades found in MT5 history (last 7 days)'
    }

logging.info(f"[MT5 AUTO-DISCOVER] Found {len(all_trades)} total trades")

# Group trades by account number and analyze lot sizes
logging.info("[MT5 AUTO-DISCOVER] Grouping trades by account...")
accounts_data = {}
skipped_trades = 0

for trade in all_trades:
    account_num = trade.get('accountNumber', '').strip()
    if not account_num:
        skipped_trades += 1
        if skipped_trades <= 3: # Log first 3 skipped trades
            logging.warning(
                f"[MT5 AUTO-DISCOVER] Skipping trade {trade.get('ticket')} - no account number
            )
            continue

    if account_num not in accounts_data:
        accounts_data[account_num] = {
            'trades': [],
            'lot_sizes': set(),
            'propFirm': trade.get('propFirm', 'Auto-detected'),
            'phases': {} # phase -> {trades, total_pnl, hedge_columns}
        }

    accounts_data[account_num]['trades'].append(trade)
    accounts_data[account_num]['lot_sizes'].add(trade['volume'])

    # Update prop firm if we detect a more specific one
    if trade.get('propFirm') and trade['propFirm'] != 'Unknown' and trade[
        'propFirm'] != 'Auto-detected':
        accounts_data[account_num]['propFirm'] = trade['propFirm']

# Log summary of skipped trades

```

```

if skipped_trades > 0:
    logging.warning(
        f"[MT5 AUTO-DISCOVER] ■■■ Skipped {skipped_trades} of {len(all_trades)} trades (no a
    logging.warning(
        f"[MT5 AUTO-DISCOVER] Successfully extracted {len(accounts_data)} unique accounts fr
else:
    logging.info(
        f"[MT5 AUTO-DISCOVER] ✓ Successfully extracted all {len(all_trades)} trades into {len

# Determine phase from lot size
phase, hedge_num = self._determine_phase_from_lotsize(trade['volume'])

if phase not in accounts_data[account_num]['phases']:
    accounts_data[account_num]['phases'][phase] = {
        'trades': [],
        'hedge_columns': {},
        'total_pnl': 0
    }

# Add to hedge column
if hedge_num not in accounts_data[account_num]['phases'][phase]['hedge_columns']:
    accounts_data[account_num]['phases'][phase]['hedge_columns'][hedge_num] = {
        'trades': [],
        'pnl': 0
    }

accounts_data[account_num]['phases'][phase]['hedge_columns'][hedge_num]['trades'].append(
    trade)
accounts_data[account_num]['phases'][phase]['hedge_columns'][hedge_num]['pnl'] += trade[
    'profit']
accounts_data[account_num]['phases'][phase]['total_pnl'] += trade['profit']

if skipped_trades > 0:
    logging.warning(f"[MT5 AUTO-DISCOVER] Skipped {skipped_trades} trades with no account num

logging.info(f"[MT5 AUTO-DISCOVER] Discovered {len(accounts_data)} unique accounts:")

# Count TopStep vs Tradovate
topstep_count = sum(1 for d in accounts_data.values() if d['propFirm'] == 'TopStep')
tradovate_count = sum(1 for d in accounts_data.values() if d['propFirm'] == 'Tradovate')

logging.info(f" TopStep Accounts: {topstep_count}")
logging.info(f" Tradovate Accounts: {tradovate_count}")
logging.info(f" Other Accounts: {len(accounts_data) - topstep_count - tradovate_count}")

results = {
    'success': True,
    'mt5Account': mt5_account,
    'totalTrades': len(all_trades),
    'accountsDiscovered': len(accounts_data),
    'topStepCount': topstep_count,
    'tradovateCount': tradovate_count,
    'accounts': []
}

# Process each account
for account_num, data in accounts_data.items():
    prop_firm = data['propFirm']
    logging.info(f"\n[MT5 AUTO-DISCOVER] Account: {account_num} ({prop_firm})")
    logging.info(f" Total Trades: {len(data['trades'])}")

```

```

logging.info(f" Lot Sizes: {sorted(data['lot_sizes'])}")
logging.info(f" Phases Detected: {list(data['phases'].keys())}")

account_result = {
    'accountNumber': account_num,
    'propFirm': prop_firm,
    'totalTrades': len(data['trades']),
    'phases': {}
}

# Sync each phase for this account
for phase, phase_data in data['phases'].items():
    logging.info(f"\n Phase: {phase}")
    logging.info(f" Trades: {len(phase_data['trades'])}")
    logging.info(f" Hedge Columns: {sorted(phase_data['hedge_columns'].keys())}")
    logging.info(f" Total P&L: ${phase_data['total_pnl']:.2f}")

    # Prepare hedge results for this phase
    hedge_results = {}
    for hedge_num, hedge_data in phase_data['hedge_columns'].items():
        hedge_amount = hedge_data['pnl']
        logging.info(
            f" Hedge {hedge_num}: {len(hedge_data['trades'])} trades, ${hedge_amount:.2f}"

        if phase == 'farming':
            # Farming uses propDay and hedgeDay – store clean numeric, no $ prefix
            hedge_results[f'hedgeDay{hedge_num}'] = f"{hedge_amount:.2f}"
            logging.info(
                f" → Dashboard field: hedgeDay{hedge_num} = ${hedge_amount:.2f}")
        else:
            # Evaluation and Funded use hedgeResult – store clean numeric, no $ prefix
            hedge_results[f'hedgeResult{hedge_num}'] = f"{hedge_amount:.2f}"
            logging.info(
                f" → Dashboard field: hedgeResult{hedge_num} = ${hedge_amount:.2f}")

    # Add net total – store clean numeric, no $ prefix
    if phase == 'farming':
        hedge_results['farmingNet'] = f"{phase_data['total_pnl']:.2f}"
    else:
        hedge_results['hedgeNet'] = f"{phase_data['total_pnl']:.2f}"

    account_result['phases'][phase] = {
        'trades': len(phase_data['trades']),
        'hedgeColumns': sorted(phase_data['hedge_columns'].keys()),
        'totalPnL': phase_data['total_pnl'],
        'hedgeResults': hedge_results
    }

results['accounts'].append(account_result)

# Send all data to dashboard for auto-creation/update
logging.info(f"\n[MT5 AUTO-DISCOVER] Sending discovery data to dashboard...")

headers = {
    'Authorization': f'Bearer {self.auth_token}',
    'Content-Type': 'application/json'
}

payload = {
    'email': self.user_email,

```

```

        'mt5Account': mt5_account,
        'accountsData': results['accounts'],
        'allTrades': all_trades
    }

    url = f"{self.dashboard_url}/mt5-sync/auto-discover"

    response = requests.post(url, json=payload, headers=headers, timeout=30)

    if response.status_code == 200:
        api_result = response.json()
        results['syncResult'] = api_result
        logging.info(f"[MT5 AUTO-DISCOVER] Success! {api_result.get('message', 'Accounts synced')}")
        return results
    else:
        error_msg = f"Auto-discover sync failed: {response.status_code}"
        try:
            error_data = response.json()
            error_msg = error_data.get('error', error_msg)
        except:
            pass

        logging.error(f"[MT5 AUTO-DISCOVER] {error_msg}")
        results['success'] = False
        results['error'] = error_msg
        return results

except Exception as e:
    logging.error(f"[MT5 AUTO-DISCOVER] Error: {e}")
    import traceback
    traceback.print_exc()
    return {
        'success': False,
        'error': str(e)
    }
}

def sync_to_dashboard(self, account_number=None, phase=None, prop_firm=None):
    """
    Extract Tradovate/TopStep account numbers and hedge amounts from MT5 trades

    MT5 is ONLY used to extract:
    1. Prop firm account numbers from trade comments (Tradovate, TopStep)
    2. Hedge amounts based on lot sizes
    3. Trade P&L per hedge column

    MT5 account is NOT matched - just extract data and send to auto-discover.
    Use auto_discover_and_sync() instead for full automation.

    Args:
        account_number: Prop firm account number (optional)
        phase: Trading phase (optional)
        prop_firm: Prop firm name (optional)

    Returns:
        dict: Sync result (redirects to auto-discover)
    """
    logging.info("[MT5 SYNC] Redirecting to auto-discover for full account extraction...")
    return self.auto_discover_and_sync()

def get_sync_status(self, account_number):

```

```

"""
Get sync status for an account

Args:
    account_number: Prop firm account number

Returns:
    dict: Sync status
"""
try:
    if not self.auth_token:
        return {
            'success': False,
            'error': 'Not authenticated'
        }

    headers = {
        'Authorization': f'Bearer {self.auth_token}'
    }

    url = f"{self.dashboard_url}/mt5-sync/account/{account_number}"

    response = requests.get(url, headers=headers, timeout=10)

    if response.status_code == 200:
        return response.json()
    else:
        return {
            'success': False,
            'error': f'Status check failed: {response.status_code}'
        }

except Exception as e:
    logging.error(f"[MT5 SYNC] Error checking sync status: {e}")
    return {
        'success': False,
        'error': str(e)
    }

def auto_sync_all_accounts(self, accounts_config):
    """
    Automatically sync all configured accounts

    Args:
        accounts_config: List of account dictionaries with:
            - accountNumber
            - phase
            - propFirm

    Returns:
        dict: Summary of sync results
    """
    results = {
        'total': len(accounts_config),
        'success': 0,
        'failed': 0,
        'details': []
    }

    for account in accounts_config:

```

```

        account_number = account.get('accountNumber')
        phase = account.get('phase')
        prop_firm = account.get('propFirm')

        logging.info(f"[MT5 SYNC] Auto-syncing account {account_number} ({phase})")

        result = self.sync_to_dashboard(account_number, phase, prop_firm)

        if result.get('success'):
            results['success'] += 1
        else:
            results['failed'] += 1

        results['details'].append({
            'accountNumber': account_number,
            'phase': phase,
            'result': result
        })

    logging.info(f"[MT5 SYNC] Auto-sync complete: {results['success']}/{results['total']} successful")

    return results

```

Method: MT5DashboardSync.__init__

```
def __init__(self, dashboard_url='http://localhost:5000/api', user_email=None)
```

Initialize MT5 Dashboard Sync Args: dashboard_url: Base URL for dashboard API user_email: User email for authentication

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **__init__** — The constructor. Runs after Python has allocated the instance; assigns starting attribute values to self.

Step by step (top-level body)

1. Assigns `self.dashboard_url = dashboard_url.rstrip('/')`.
2. Assigns `self.user_email = user_email`.
3. Assigns `self.auth_token = None`.
4. Assigns `self._is_connected = False`.
5. Calls `logging.info(...)` for its side effect.

Code (copy-paste safe)

```

def __init__(self, dashboard_url="http://localhost:5000/api", user_email=None):
    """
    Initialize MT5 Dashboard Sync

    Args:
        dashboard_url: Base URL for dashboard API
        user_email: User email for authentication
    """
    self.dashboard_url = dashboard_url.rstrip('/')

```



```

self.user_email = user_email
self.auth_token = None
self._is_connected = False
logging.info(f"[MT5 SYNC] Initialized for email: {user_email}")

```

Method: MT5DashboardSync.set_auth_token

```

def set_auth_token(self, token)

    Set authentication token from dashboard login

```

Step by step (top-level body)

1. Assigns `self.auth_token = token`.
2. Calls `logging.info(...)` for its side effect.

Code (copy-paste safe)

```

def set_auth_token(self, token):
    """Set authentication token from dashboard login"""
    self.auth_token = token
    logging.info("[MT5 SYNC] Authentication token set")

```

Method: MT5DashboardSync.check_dashboard_health

```

def check_dashboard_health(self)

    Check if dashboard backend is running and healthy Returns: dict: {'connected': bool, 'status': str, 'message': str}

```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 3 **except** clauses.

Code (copy-paste safe)

```

def check_dashboard_health(self):
    """
    Check if dashboard backend is running and healthy

    Returns:
        dict: {'connected': bool, 'status': str, 'message': str}
    """
    try:
        # Check backend health endpoint
        backend_url = self.dashboard_url.replace('/api', '')
        health_url = f"{backend_url}/health"

        response = requests.get(health_url, timeout=3)

```

```

        if response.status_code == 200:
            data = response.json()
            if data.get('status') == 'healthy':
                self._is_connected = True
                return {
                    'connected': True,
                    'status': 'healthy',
                    'message': 'Dashboard connected',
                    'version': data.get('version', 'unknown')
                }

            self._is_connected = False
            return {
                'connected': False,
                'status': 'unhealthy',
                'message': f'Dashboard responded with status {response.status_code}'
            }

    except requests.exceptions.Timeout:
        self._is_connected = False
        return {
            'connected': False,
            'status': 'timeout',
            'message': 'Dashboard connection timeout (3s)'
        }

    except requests.exceptions.ConnectionError:
        self._is_connected = False
        return {
            'connected': False,
            'status': 'error',
            'message': 'Cannot connect to dashboard. Is it running?'
        }

    except Exception as e:
        self._is_connected = False
        return {
            'connected': False,
            'status': 'error',
            'message': f'Health check failed: {str(e)}'
        }
}

```

Method: MT5DashboardSync.is_connected

```
def is_connected(self)
```

Check if dashboard is connected

Step by step (top-level body)

1. return self._is_connected.

Code (copy-paste safe)

```

def is_connected(self):
    """Check if dashboard is connected"""
    return self._is_connected

```

Method: MT5DashboardSync.register_mt5_account

```
def register_mt5_account(self, mt5_login, mt5_name, mt5_server, broker=None)
```

Log MT5 connection - MT5 is only used to extract Tradovate/TopStep data MT5 account is NOT registered in dashboard. MT5 is simply a data source for extracting prop firm account numbers and hedge amounts. Args: mt5_login: MT5 account login number mt5_name: MT5 account name mt5_server: MT5 server name broker: Broker name (optional) Returns: dict: Always returns success

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `client_name = f'{mt5_login}: {mt5_name}'`.
2. Calls `logging.info(...)` for its side effect.
3. Calls `logging.info(...)` for its side effect.
4. **return** `{'success': True, 'message': 'MT5 connected as data source'}`.

Code (copy-paste safe)

```
def register_mt5_account(self, mt5_login, mt5_name, mt5_server, broker=None):
    """
    Log MT5 connection - MT5 is only used to extract Tradovate/TopStep data

    MT5 account is NOT registered in dashboard. MT5 is simply a data source
    for extracting prop firm account numbers and hedge amounts.

    Args:
        mt5_login: MT5 account login number
        mt5_name: MT5 account name
        mt5_server: MT5 server name
        broker: Broker name (optional)

    Returns:
        dict: Always returns success
    """
    client_name = f'{mt5_login}: {mt5_name}'
    logging.info(f"[MT5 DATA SOURCE] MT5 connected: {client_name}")
    logging.info(f"[MT5 DATA SOURCE] MT5 will be used to extract Tradovate/TopStep account data")
    return {'success': True, 'message': 'MT5 connected as data source'}
```

Method: MT5DashboardSync.login

```
def login(self, email, password)
```

Login to dashboard and get authentication token Args: email: User email password: User password

Returns: dict: {'success': bool, 'token': str, 'error': str}

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **try** block with 3 **except** clauses.

Code (copy-paste safe)

```
def login(self, email, password):
    """
    Login to dashboard and get authentication token

    Args:
        email: User email
        password: User password

    Returns:
        dict: {'success': bool, 'token': str, 'error': str}
    """
    try:
        # Demo mode credentials (matches dashboard frontend)
        demo_credentials = {
            'harryodhiambo17@gmail.com': 'demo123',
            'admin@tradeconnector.com': 'admin123',
            'test@example.com': 'test123'
        }

        # Check if using demo credentials
        if email in demo_credentials and demo_credentials[email] == password:
            # Generate demo token
            demo_token = f'demo-token-{int(datetime.now().timestamp())}'
            self.auth_token = demo_token
            self.user_email = email

            logging.info(f"[MT5 SYNC] Demo login successful for {email}")
            return {
                'success': True,
                'token': demo_token,
                'user': {
                    'email': email,
                    'firstName': 'Harry' if email == 'harryodhiambo17@gmail.com' else 'Demo',
                    'lastName': 'Odhiambo' if email == 'harryodhiambo17@gmail.com' else 'User'
                }
            }

        # Try real API call if not demo credentials
        url = f"{self.dashboard_url}/auth/login"
        payload = {
            'email': email,
            'password': password
        }

        logging.info(f"[MT5 SYNC] Attempting login to {url}...")
        response = requests.post(url, json=payload, timeout=30)

        if response.status_code == 200:
            data = response.json()
            token = data.get('token')

            if token:
```

```

        self.auth_token = token
        self.user_email = email
        logging.info(f"[MT5 SYNC] Login successful for {email}")
        return {
            'success': True,
            'token': token,
            'user': data.get('user', {})
        }
    else:
        return {
            'success': False,
            'error': 'No token received from server'
        }
    else:
        error_data = response.json() if response.content else {}
        error_msg = error_data.get('error', f'Login failed with status {response.status_code}')

        # Add helpful message about demo credentials
        if response.status_code == 401:
            error_msg += '\n\nTry demo credentials:\nEmail: harryodhiambo17@gmail.com\nPassw

        logging.error(f"[MT5 SYNC] Login failed: {error_msg}")
        return {
            'success': False,
            'error': error_msg
        }

except requests.exceptions.Timeout:
    logging.error("[MT5 SYNC] Login request timeout")
    return {
        'success': False,
        'error': 'Login request timeout (30s). Dashboard may be slow or not responding.'
    }
except requests.exceptions.ConnectionError as e:
    logging.error(f"[MT5 SYNC] Connection error: {e}")
    # If connection fails, suggest demo credentials as fallback
    return {
        'success': False,
        'error': f'Cannot connect to dashboard. Try demo credentials instead:\nEmail: harryodhiambo17@gmail.com\nPassw
    }
except Exception as e:
    logging.error(f"[MT5 SYNC] Login error: {e}")
    return {
        'success': False,
        'error': f'Login error: {str(e)}'
    }
}

```

Method: MT5DashboardSync.get_mt5_history_by_account

```
def get_mt5_history_by_account(self, account_number=None, days_back=7)
```

Fetch MT5 trading history filtered by account number (from comment) Args: account_number: Prop firm account number to filter by (optional) days_back: Number of days to look back for history (default: 7) Returns: List of trade dictionaries filtered by account number

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't

have to handle it.

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses `x` if `cond` else `y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def get_mt5_history_by_account(self, account_number=None, days_back=7):
    """
    Fetch MT5 trading history filtered by account number (from comment)

    Args:
        account_number: Prop firm account number to filter by (optional)
        days_back: Number of days to look back for history (default: 7)

    Returns:
        List of trade dictionaries filtered by account number
    """
    try:
        logging.info(f"[MT5 SYNC] Fetching history for last {days_back} days...")

        # Check MT5 initialization status FIRST
        terminal_info = mt5.terminal_info()
        if terminal_info is None:
            logging.error("[MT5 SYNC] MT5 terminal_info() returned None - MT5 not initialized")
            if not mt5.initialize():
                logging.error("[MT5 SYNC] Failed to initialize MT5")
                return []
            else:
                logging.info("[MT5 SYNC] MT5 initialized successfully")
        else:
            logging.info(f"[MT5 SYNC] MT5 is initialized: {terminal_info.connected}")

        # Calculate date range
        end_date = datetime.now()
        start_date = end_date - timedelta(days=days_back)

        # Get deals (closed trades)
        logging.info(
            f"[MT5 SYNC] Requesting deals from {start_date.strftime('%Y-%m-%d %H:%M:%S')} to {end_date.strftime('%Y-%m-%d %H:%M:%S')}"
        )
        deals = mt5.history_deals_get(start_date, end_date)

        if deals is None:
            logging.warning("[MT5 SYNC] MT5 returned None for deals - check MT5 connection")
            logging.warning(f"[MT5 SYNC] MT5 last error: {mt5.last_error()}")
            return []

        if len(deals) == 0:
            logging.warning("[MT5 SYNC] No deals found in specified period")
            logging.warning(
                f"[MT5 SYNC] Checked period: {start_date.strftime('%Y-%m-%d')} to {end_date.strftime('%Y-%m-%d')}"
            )
            logging.warning(f"[MT5 SYNC] Try checking MT5 'Account History' tab to verify trades exist")
            return []
```

```

logging.info(f"[MT5 SYNC] Found {len(deals)} deals, processing into trades...")
logging.info(
    f"[MT5 SYNC] First deal sample: ticket={deals[0].ticket if deals else 'N/A'}, time={deals[0].time if deals else 'N/A'}"
)

# Process deals into trade format
trades = []
deal_pairs = {} # Group entry and exit deals

for deal in deals:
    # Skip balance operations
    if deal.type not in [mt5.DEAL_TYPE_BUY, mt5.DEAL_TYPE_SELL]:
        continue

    # Filter by account number if specified (comment contains account number)
    if account_number:
        comment = deal.comment or ''
        if account_number not in comment:
            continue

    position_id = deal.position_id

    if position_id not in deal_pairs:
        deal_pairs[position_id] = {'entry': None, 'exit': None}

    # Determine if this is entry or exit
    if deal.entry == mt5.DEAL_ENTRY_IN:
        deal_pairs[position_id]['entry'] = deal
    elif deal.entry == mt5.DEAL_ENTRY_OUT:
        deal_pairs[position_id]['exit'] = deal

# Combine entry/exit pairs into complete trades
for position_id, pair in deal_pairs.items():
    if pair['entry'] and pair['exit']:
        entry = pair['entry']
        exit_deal = pair['exit']

        # Extract account info from comment
        comment = entry.comment or ''
        account_info = self._extract_account_from_comment(comment)

        # Debug log first few comments to understand format
        if len(trades) < 5:
            logging.info(
                f"[MT5 SYNC DEBUG] Trade {position_id} comment: '{comment}' -> Account: '{account_info}'"
            )

        trade = {
            'ticket': position_id,
            'symbol': entry.symbol,
            'type': 'buy' if entry.type == mt5.DEAL_TYPE_BUY else 'sell',
            'volume': entry.volume,
            'openPrice': entry.price,
            'closePrice': exit_deal.price,
            'openTime': datetime.fromtimestamp(entry.time).isoformat(),
            'closeTime': datetime.fromtimestamp(exit_deal.time).isoformat(),
            'profit': exit_deal.profit,
            'commission': entry.commission + exit_deal.commission,
            'swap': entry.swap + exit_deal.swap,
            'comment': comment,
            'accountNumber': account_info['accountNumber'],
        }

```

```

        'propFirm': account_info['propFirm']
    }

    trades.append(trade)

    logging.info(f"[MT5 SYNC] Fetched {len(trades)} completed trades")
    if account_number:
        logging.info(f"[MT5 SYNC] Filtered for account: {account_number}")
    return trades

except Exception as e:
    logging.error(f"[MT5 SYNC] Error fetching MT5 history: {e}")
    return []

```

Method: MT5DashboardSync._extract_account_from_comment

```
def _extract_account_from_comment(self, comment)
```

Extract account number and prop firm from MT5 comment Supported formats: - TopStep: 'PA-TS123456', 'TS123456', 'PA123456', or variations - Tradovate: 'TRDV123456', 'TV123456', or variations - Generic: Treats any alphanumeric string as account number - Empty: Returns empty account number (trade will be skipped) - With Phase: 'ACCOUNT_CH1', 'ACCOUNT_FD1', etc. (phase suffix is stripped) Returns: dict: {'accountNumber': str, 'propFirm': str}

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.

Step by step (top-level body)

1. **if** not comment or not comment.strip(): branches conditionally.
2. Assigns comment = comment.strip().
3. Imports re (lazy import inside the function).
4. **if** '_' in comment: branches conditionally.
5. Assigns clean_comment = comment.replace('#', "").replace('/', "").strip().
6. **try** block with 1 **except** clause.
7. **if** any((keyword in clean_comment.upper() for keyword in ['TS', 'TOPSTE...: branches conditionally.
8. **if** any((keyword in clean_comment.upper() for keyword in ['TRDV', 'TV',...: branches conditionally.
9. **if** clean_comment and len(clean_comment) >= 3: branches conditionally.
10. **return** {'accountNumber': "", 'propFirm': 'Unknown'}.

Code (copy-paste safe)

```

def _extract_account_from_comment(self, comment):
    """
    Extract account number and prop firm from MT5 comment

```


Supported formats:

- TopStep: 'PA-TS123456', 'TS123456', 'PA123456', or variations
- Tradovate: 'TRDV123456', 'TV123456', or variations
- Generic: Treats any alphanumeric string as account number
- Empty: Returns empty account number (trade will be skipped)
- With Phase: 'ACCOUNT_CH1', 'ACCOUNT_FD1', etc. (phase suffix is stripped)

Returns:

```
dict: {'accountNumber': str, 'propFirm': str}
```

```
"""
```

```
if not comment or not comment.strip():
```

```
    return {'accountNumber': '', 'propFirm': 'Unknown'}
```

```
comment = comment.strip()
```

```
# Strip phase abbreviation suffix if present (e.g., _CH1, _FD2, _DD3, _FA)
```

```
# This must be done BEFORE other processing
```

```
import re
```

```
if '_' in comment:
```

```
    # Check for numbered formats (_CH1-4, _FD1-4, _DD1-4)
```

```
    if re.search(r'_(CH|FD|DD)\d+$', comment):
```

```
        comment = re.sub(r'_(CH|FD|DD)\d+$', '', comment)
```

```
    # Check for simple farming format: _FA
```

```
    elif comment.endswith('_FA'):
```

```
        comment = comment[:-3]
```

```
    # Check for unknown phase marker
```

```
    elif comment.endswith('_UNK'):
```

```
        comment = comment[:-4]
```

```
# Remove common noise characters
```

```
clean_comment = comment.replace('#', '').replace('/', '').strip()
```

```
# Quick heuristic: detect Other-mode combined comments that start
```

```
# with an account identifier then an underscore and a prop-firm token
```

```
# Example: '123456_PropFirm_Challenge_T1' -> account=123456, propFirm=PropFirm
```

```
try:
```

```
    parts = clean_comment.split('_')
```

```
    if len(parts) >= 2:
```

```
        first = parts[0].strip()
```

```
        second = parts[1].strip()
```

```
        # First part looks like a numeric account or contains known prefixes
```

```
        if re.match(r'^[0-9]{3,}$', first) or any(k in first.upper() for k in ['TRDV', 'TV', 'TS
```

```
        'PA']):
```

```
            return {'accountNumber': first, 'propFirm': second}
```

```
except Exception:
```

```
    # If anything goes wrong with this heuristic, continue with normal parsing
```

```
    pass
```

```
# Check for TopStep patterns (PA-, TS, TopStep keywords)
```

```
if any(keyword in clean_comment.upper() for keyword in ['TS', 'TOPSTEP', 'PA-', 'PA ']):
```

```
    # Extract just the numeric part or full account number
```

```
    import re
```

```
    # Try to find PA-XXXXX or TS-XXXXX pattern
```

```
    match = re.search(r'(?PA-?|TS-?)([A-Z0-9]+)', clean_comment.upper())
```

```
    if match:
```

```
        account_num = match.group(1)
```

```
    else:
```

```
        # Remove all prefix keywords and get what's left
```

```
        account_num = clean_comment.upper()
```

```
        for prefix in ['PA-', 'PA ', 'TS-', 'TS ', 'TOPSTEP-', 'TOPSTEP ']:
```

```

        account_num = account_num.replace(prefix, '')
        account_num = account_num.strip()

    return {'accountNumber': account_num, 'propFirm': 'TopStep'}

# Check for Tradovate patterns (TRDV, TV, Tradovate keywords)
if any(keyword in clean_comment.upper() for keyword in ['TRDV', 'TV', 'TRADOVATE']):
    import re
    # Try to find TRDV-XXXXX or TV-XXXXX pattern
    match = re.search(r'(?TRDV-?|TV-?)([A-Z0-9]+)', clean_comment.upper())
    if match:
        account_num = match.group(1)
    else:
        # Remove all prefix keywords
        account_num = clean_comment.upper()
        for prefix in ['TRDV-', 'TRDV ', 'TV-', 'TV ', 'TRADOVATE-', 'TRADOVATE ']:
            account_num = account_num.replace(prefix, '')
        account_num = account_num.strip()

    return {'accountNumber': account_num, 'propFirm': 'Tradovate'}

# If no specific pattern but comment exists, use it as account number
# This handles cases where account number is just digits or simple format
if clean_comment and len(clean_comment) >= 3: # Minimum reasonable account number length
    return {'accountNumber': clean_comment, 'propFirm': 'Auto-detected'}

# Comment too short or invalid
return {'accountNumber': '', 'propFirm': 'Unknown'}

```

Method: MT5DashboardSync.get_mt5_client_info

```
def get_mt5_client_info(self)
```

*Get MT5 account info including client name Returns: dict: {'login': int, 'name': str, 'clientName': str}
clientName format: "login: name" (e.g., "3053752: Tyler Turner")*

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def get_mt5_client_info(self):
    """
    Get MT5 account info including client name

    Returns:
        dict: {'login': int, 'name': str, 'clientName': str}
            clientName format: "login: name" (e.g., "3053752: Tyler Turner")
    """
    try:

```

```

if not mt5.initialize():
    logging.error("[MT5 SYNC] Failed to initialize MT5")
    return None

account_info = mt5.account_info()
if not account_info:
    logging.error("[MT5 SYNC] Failed to get account info")
    return None

client_info = {
    'login': account_info.login,
    'name': account_info.name,
    'clientName': f"{account_info.login}: {account_info.name}"
}

logging.info(f"[MT5 SYNC] Client: {client_info['clientName']}")
return client_info

except Exception as e:
    logging.error(f"[MT5 SYNC] Error getting client info: {e}")
    return None

```

Method: MT5DashboardSync._determine_phase_from_lotsize

```
def _determine_phase_from_lotsize(self, lot_size)
```

Determine trading phase based on lot size Different phases use different lot sizes: - Evaluation: Smaller lots (e.g., 1.0, 2.0) - Funded: Medium lots (e.g., 3.0, 4.0, 5.0) - Farming: Larger lots (e.g., 6.0+) Args: lot_size: MT5 lot size Returns: Tuple of (phase, hedge_column_number)

Step by step (top-level body)

1. if lot_size <= 2.0: branches conditionally (with an **else/elif** arm).

Code (copy-paste safe)

```

def _determine_phase_from_lotsize(self, lot_size):
    """
    Determine trading phase based on lot size

    Different phases use different lot sizes:
    - Evaluation: Smaller lots (e.g., 1.0, 2.0)
    - Funded: Medium lots (e.g., 3.0, 4.0, 5.0)
    - Farming: Larger lots (e.g., 6.0+)

    Args:
        lot_size: MT5 lot size

    Returns:
        Tuple of (phase, hedge_column_number)
    """
    # These thresholds can be customized based on your strategy
    if lot_size <= 2.0:
        # Evaluation phase - lot sizes 1.0, 2.0
        hedge_num = int(lot_size) # 1.0 -> hedge 1, 2.0 -> hedge 2, etc.
        return 'evaluation', hedge_num
    elif lot_size <= 5.0:
        # Funded phase - lot sizes 3.0, 4.0, 5.0
        hedge_num = int(lot_size - 2.0) # 3.0 -> hedge 1, 4.0 -> hedge 2, 5.0 -> hedge 3
        return 'funded', hedge_num

```

```

else:
    # Farming phase - lot sizes 6.0+
    hedge_num = int(lot_size - 5.0) # 6.0 -> hedge 1, 7.0 -> hedge 2, etc.
    return 'farming', hedge_num

```

Method: MT5DashboardSync.get_mt5_history

```
def get_mt5_history(self, days_back=30)
```

Fetch all MT5 trading history (backward compatibility) Args: days_back: Number of days to look back for history Returns: List of trade dictionaries

Step by step (top-level body)

1. **return** self.get_mt5_history_by_account(account_number=None, days_back=days....

Code (copy-paste safe)

```

def get_mt5_history(self, days_back=30):
    """
    Fetch all MT5 trading history (backward compatibility)

    Args:
        days_back: Number of days to look back for history

    Returns:
        List of trade dictionaries
    """
    return self.get_mt5_history_by_account(account_number=None, days_back=days_back)

```

Method: MT5DashboardSync.auto_discover_and_sync

```
def auto_discover_and_sync(self)
```

Automatically discover all prop firm accounts from MT5 and sync to dashboard. Process: 1. Fetch all MT5 trades 2. Extract unique account numbers from comments 3. Group trades by account and lot size to determine phase 4. Auto-create/update accounts in dashboard 5. Sync all hedge results Returns: dict: Summary of discovery and sync results

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def auto_discover_and_sync(self):
    """
    Automatically discover all prop firm accounts from MT5 and sync to dashboard.

    Process:
    1. Fetch all MT5 trades
    2. Extract unique account numbers from comments
    3. Group trades by account and lot size to determine phase
    4. Auto-create/update accounts in dashboard
    5. Sync all hedge results

    Returns:
    dict: Summary of discovery and sync results
    """
    try:
        if not self.user_email or not self.auth_token:
            return {
                'success': False,
                'error': 'Not authenticated. Please login first.'
            }

        logging.info("[MT5 AUTO-DISCOVER] Starting automatic account discovery...")

        # Get MT5 account info
        client_info = self.get_mt5_client_info()
        if not client_info:
            return {
                'success': False,
                'error': 'Failed to get MT5 account info'
            }

        mt5_account = f"{client_info['login']}: {client_info['name']}"
        logging.info(f"[MT5 AUTO-DISCOVER] MT5 Hedge Account: {mt5_account}")

        # Fetch all MT5 trades (last 7 days for faster processing)
        logging.info("[MT5 AUTO-DISCOVER] Fetching MT5 trade history...")
        logging.info(
            f"[MT5 AUTO-DISCOVER] Calling get_mt5_history_by_account(account_number=None, days_back=7)"
        )
        all_trades = self.get_mt5_history_by_account(account_number=None, days_back=7)
        logging.info(
            f"[MT5 AUTO-DISCOVER] get_mt5_history_by_account() returned {len(all_trades)} if all_trades"
        )

        if not all_trades:
            logging.warning(
                "[MT5 AUTO-DISCOVER] No trades found in MT5 history - check MT5 connection and trade"
            )
            return {
                'success': False,
                'error': 'No trades found in MT5 history (last 7 days)'
            }

        logging.info(f"[MT5 AUTO-DISCOVER] Found {len(all_trades)} total trades")

        # Group trades by account number and analyze lot sizes
        logging.info("[MT5 AUTO-DISCOVER] Grouping trades by account...")
        accounts_data = {}
        skipped_trades = 0

        for trade in all_trades:
```

```

account_num = trade.get('accountNumber', '').strip()
if not account_num:
    skipped_trades += 1
    if skipped_trades <= 3: # Log first 3 skipped trades
        logging.warning(
            f"[MT5 AUTO-DISCOVER] Skipping trade {trade.get('ticket')} - no account number"
        )
    continue

if account_num not in accounts_data:
    accounts_data[account_num] = {
        'trades': [],
        'lot_sizes': set(),
        'propFirm': trade.get('propFirm', 'Auto-detected'),
        'phases': {} # phase -> {trades, total_pnl, hedge_columns}
    }

accounts_data[account_num]['trades'].append(trade)
accounts_data[account_num]['lot_sizes'].add(trade['volume'])

# Update prop firm if we detect a more specific one
if trade.get('propFirm') and trade['propFirm'] != 'Unknown' and trade[
    'propFirm'] != 'Auto-detected':
    accounts_data[account_num]['propFirm'] = trade['propFirm']

# Log summary of skipped trades
if skipped_trades > 0:
    logging.warning(
        f"[MT5 AUTO-DISCOVER] ■■ Skipped {skipped_trades} of {len(all_trades)} trades (no a
    )
    logging.warning(
        f"[MT5 AUTO-DISCOVER] Successfully extracted {len(accounts_data)} unique accounts fro
else:
    logging.info(
        f"[MT5 AUTO-DISCOVER] ✓ Successfully extracted all {len(all_trades)} trades into {len
    )

# Determine phase from lot size
phase, hedge_num = self._determine_phase_from_lotsize(trade['volume'])

if phase not in accounts_data[account_num]['phases']:
    accounts_data[account_num]['phases'][phase] = {
        'trades': [],
        'hedge_columns': {},
        'total_pnl': 0
    }

# Add to hedge column
if hedge_num not in accounts_data[account_num]['phases'][phase]['hedge_columns']:
    accounts_data[account_num]['phases'][phase]['hedge_columns'][hedge_num] = {
        'trades': [],
        'pnl': 0
    }

accounts_data[account_num]['phases'][phase]['hedge_columns'][hedge_num]['trades'].append(
    trade)
accounts_data[account_num]['phases'][phase]['hedge_columns'][hedge_num]['pnl'] += trade[
    'profit']
accounts_data[account_num]['phases'][phase]['total_pnl'] += trade['profit']

if skipped_trades > 0:
    logging.warning(f"[MT5 AUTO-DISCOVER] Skipped {skipped_trades} trades with no account num
)
logging.info(f"[MT5 AUTO-DISCOVER] Discovered {len(accounts_data)} unique accounts:")

```

```

# Count TopStep vs Tradovate
topstep_count = sum(1 for d in accounts_data.values() if d['propFirm'] == 'TopStep')
tradovate_count = sum(1 for d in accounts_data.values() if d['propFirm'] == 'Tradovate')

logging.info(f" TopStep Accounts: {topstep_count}")
logging.info(f" Tradovate Accounts: {tradovate_count}")
logging.info(f" Other Accounts: {len(accounts_data) - topstep_count - tradovate_count}")

results = {
    'success': True,
    'mt5Account': mt5_account,
    'totalTrades': len(all_trades),
    'accountsDiscovered': len(accounts_data),
    'topStepCount': topstep_count,
    'tradovateCount': tradovate_count,
    'accounts': []
}

# Process each account
for account_num, data in accounts_data.items():
    prop_firm = data['propFirm']
    logging.info(f"\n[MT5 AUTO-DISCOVER] Account: {account_num} ({prop_firm})")
    logging.info(f" Total Trades: {len(data['trades'])}")
    logging.info(f" Lot Sizes: {sorted(data['lot_sizes'])}")
    logging.info(f" Phases Detected: {list(data['phases'].keys())}")

    account_result = {
        'accountNumber': account_num,
        'propFirm': prop_firm,
        'totalTrades': len(data['trades']),
        'phases': {}
    }

    # Sync each phase for this account
    for phase, phase_data in data['phases'].items():
        logging.info(f"\n Phase: {phase}")
        logging.info(f" Trades: {len(phase_data['trades'])}")
        logging.info(f" Hedge Columns: {sorted(phase_data['hedge_columns'].keys())}")
        logging.info(f" Total P&L: ${phase_data['total_pnl']:.2f}")

        # Prepare hedge results for this phase
        hedge_results = {}
        for hedge_num, hedge_data in phase_data['hedge_columns'].items():
            hedge_amount = hedge_data['pnl']
            logging.info(
                f" Hedge {hedge_num}: {len(hedge_data['trades'])} trades, ${hedge_amount:.2f}"
            )

            if phase == 'farming':
                # Farming uses propDay and hedgeDay – store clean numeric, no $ prefix
                hedge_results[f'hedgeDay{hedge_num}'] = f"{hedge_amount:.2f}"
                logging.info(
                    f" → Dashboard field: hedgeDay{hedge_num} = ${hedge_amount:.2f}"
                )
            else:
                # Evaluation and Funded use hedgeResult – store clean numeric, no $ prefix
                hedge_results[f'hedgeResult{hedge_num}'] = f"{hedge_amount:.2f}"
                logging.info(
                    f" → Dashboard field: hedgeResult{hedge_num} = ${hedge_amount:.2f}"
                )

        # Add net total – store clean numeric, no $ prefix
        if phase == 'farming':

```

```

        hedge_results['farmingNet'] = f"{phase_data['total_pnl']:.2f}"
    else:
        hedge_results['hedgeNet'] = f"{phase_data['total_pnl']:.2f}"

    account_result['phases'][phase] = {
        'trades': len(phase_data['trades']),
        'hedgeColumns': sorted(phase_data['hedge_columns'].keys()),
        'totalPnL': phase_data['total_pnl'],
        'hedgeResults': hedge_results
    }

    results['accounts'].append(account_result)

# Send all data to dashboard for auto-creation/update
logging.info(f"\n[MT5 AUTO-DISCOVER] Sending discovery data to dashboard...")

headers = {
    'Authorization': f'Bearer {self.auth_token}',
    'Content-Type': 'application/json'
}

payload = {
    'email': self.user_email,
    'mt5Account': mt5_account,
    'accountsData': results['accounts'],
    'allTrades': all_trades
}

url = f"{self.dashboard_url}/mt5-sync/auto-discover"

response = requests.post(url, json=payload, headers=headers, timeout=30)

if response.status_code == 200:
    api_result = response.json()
    results['syncResult'] = api_result
    logging.info(f"[MT5 AUTO-DISCOVER] Success! {api_result.get('message', 'Accounts synced')}")
    return results
else:
    error_msg = f"Auto-discover sync failed: {response.status_code}"
    try:
        error_data = response.json()
        error_msg = error_data.get('error', error_msg)
    except:
        pass

    logging.error(f"[MT5 AUTO-DISCOVER] {error_msg}")
    results['success'] = False
    results['error'] = error_msg
    return results

except Exception as e:
    logging.error(f"[MT5 AUTO-DISCOVER] Error: {e}")
    import traceback
    traceback.print_exc()
    return {
        'success': False,
        'error': str(e)
    }
}

```


Method: MT5DashboardSync.sync_to_dashboard

```
def sync_to_dashboard(self, account_number=None, phase=None, prop_firm=None)
```

Extract Tradovate/TopStep account numbers and hedge amounts from MT5 trades MT5 is ONLY used to extract: 1. Prop firm account numbers from trade comments (Tradovate, TopStep) 2. Hedge amounts based on lot sizes 3. Trade P&L per hedge column MT5 account is NOT matched - just extract data and send to auto-discover. Use auto_discover_and_sync() instead for full automation. Args: account_number: Prop firm account number (optional) phase: Trading phase (optional) prop_firm: Prop firm name (optional) Returns: dict: Sync result (redirects to auto-discover)

Step by step (top-level body)

1. Calls logging.info(...) for its side effect.
2. return self.auto_discover_and_sync().

Code (copy-paste safe)

```
def sync_to_dashboard(self, account_number=None, phase=None, prop_firm=None):
    """
    Extract Tradovate/TopStep account numbers and hedge amounts from MT5 trades

    MT5 is ONLY used to extract:
    1. Prop firm account numbers from trade comments (Tradovate, TopStep)
    2. Hedge amounts based on lot sizes
    3. Trade P&L per hedge column

    MT5 account is NOT matched - just extract data and send to auto-discover.
    Use auto_discover_and_sync() instead for full automation.

    Args:
        account_number: Prop firm account number (optional)
        phase: Trading phase (optional)
        prop_firm: Prop firm name (optional)

    Returns:
        dict: Sync result (redirects to auto-discover)
    """
    logging.info("[MT5 SYNC] Redirecting to auto-discover for full account extraction...")
    return self.auto_discover_and_sync()
```

Method: MT5DashboardSync.get_sync_status

```
def get_sync_status(self, account_number)
```

Get sync status for an account Args: account_number: Prop firm account number Returns: dict: Sync status

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. try block with 1 except clause.

Code (copy-paste safe)

```
def get_sync_status(self, account_number):
    """
    Get sync status for an account

    Args:
        account_number: Prop firm account number

    Returns:
        dict: Sync status
    """
    try:
        if not self.auth_token:
            return {
                'success': False,
                'error': 'Not authenticated'
            }

        headers = {
            'Authorization': f'Bearer {self.auth_token}'
        }

        url = f"{self.dashboard_url}/mt5-sync/account/{account_number}"

        response = requests.get(url, headers=headers, timeout=10)

        if response.status_code == 200:
            return response.json()
        else:
            return {
                'success': False,
                'error': f'Status check failed: {response.status_code}'
            }

    except Exception as e:
        logging.error(f"[MT5 SYNC] Error checking sync status: {e}")
        return {
            'success': False,
            'error': str(e)
        }
```

Method: MT5DashboardSync.auto_sync_all_accounts

```
def auto_sync_all_accounts(self, accounts_config)
```

Automatically sync all configured accounts Args: accounts_config: List of account dictionaries with: - accountNumber - phase - propFirm Returns: dict: Summary of sync results

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns results = {'total': len(accounts_config), 'success': 0, 'failed': 0....

2. **for** account in accounts_config: iterates.

3. Calls logging.info(...) for its side effect.

4. **return** results.

Code (copy-paste safe)

```
def auto_sync_all_accounts(self, accounts_config):
    """
    Automatically sync all configured accounts

    Args:
        accounts_config: List of account dictionaries with:
            - accountNumber
            - phase
            - propFirm

    Returns:
        dict: Summary of sync results
    """
    results = {
        'total': len(accounts_config),
        'success': 0,
        'failed': 0,
        'details': []
    }

    for account in accounts_config:
        account_number = account.get('accountNumber')
        phase = account.get('phase')
        prop_firm = account.get('propFirm')

        logging.info(f"[MT5 SYNC] Auto-syncing account {account_number} ({phase})")

        result = self.sync_to_dashboard(account_number, phase, prop_firm)

        if result.get('success'):
            results['success'] += 1
        else:
            results['failed'] += 1

        results['details'].append({
            'accountNumber': account_number,
            'phase': phase,
            'result': result
        })

    logging.info(f"[MT5 SYNC] Auto-sync complete: {results['success']}/{results['total']} successful")

    return results
```

Functions

```
def get_mt5_sync(dashboard_url='http://localhost:5000/api', user_email=None)
```

Get or create global MT5 sync instance

Why these Python concepts were used

- **global** — Rebinds a module-level name from inside the function. A smell in larger codebases — usually means a singleton or cache that could be passed in instead.

Step by step (top-level body)

1. Declares globals: `_mt5_sync_instance`.
2. `if _mt5_sync_instance is None or _mt5_sync_instance.user_email != user...:` branches conditionally.
3. `return _mt5_sync_instance`.

Code (copy-paste safe)

```
def get_mt5_sync(dashboard_url="http://localhost:5000/api", user_email=None):
    """Get or create global MT5 sync instance"""
    global _mt5_sync_instance

    if _mt5_sync_instance is None or _mt5_sync_instance.user_email != user_email:
        _mt5_sync_instance = MT5DashboardSync(dashboard_url, user_email)

    return _mt5_sync_instance
```

Checkpoint — what works now. The desktop side now talks to the dashboard. Run `push_data.py` with test data and check the rows arrive in the database. `mt5_dashboard_sync` will keep that flow running on a schedule.

Chapter 10 — Phase 8 — Desktop GUI

Tkinter ships with Python — no extra install needed. We build a small broker-picker window first to learn the patterns (a window, a frame, a button, a callback), then the full trader-companion app that ties everything in phases 3–7 together under a single tabbed interface.

Files we build in this phase, in order:

- trader_companion/broker_selection.py
- trader_companion/trader_app.py

Python concepts you'll meet in this phase

- Dunder methods (2) · Classes and objects (2) · Decorators (2) · Comprehensions (2) · @classmethod and @staticmethod (2) · Type hints (2) · f-strings (1) · Exceptions (1)

Each is explained in Chapter 2 (the primer). The number in parentheses is how many of this phase's files use that concept.

Step 10.1 — Build trader_companion/broker_selection.py

What to do. Create the file trader_companion/broker_selection.py in your project root and paste in the code shown in this step. The file is **210** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from requirements.txt.

What this file is for. Model + helpers for choosing which broker terminal to connect to when multiple MT5 installations are present.

trader_companion/broker_selection.py

210 loc · 5 classes · 0 functions · 6 imports

Model + helpers for choosing which broker terminal to connect to when multiple MT5 installations are present. It defines **5** classes, across **210** lines. Module docstring: ■ *Broker Selection System - Tradovate Only*

Module docstring

■ Broker Selection System - Tradovate Only Trade Account Connector - Simplified for Tradovate integration

Python concepts demonstrated in this module

• Classes and objects • @classmethod and @staticmethod • Comprehensions • Decorators • Dunder methods • Type hints

Imports — what each one is for

```
import tkinter as tk
```

Why imported. The standard-library GUI toolkit. Powers the desktop trader companion's window.

Used in this file. tk.Toplevel

Example. root = tk.Tk(); tk.Button(root, text='Go').pack()

Other common scenarios.

- ttk.* for the modern themed widgets (Treeview, Notebook, ...)
- after(ms, fn) schedules a callback on the GUI thread
- StringVar / IntVar bind variables to widgets

```
from tkinter import ttk
from tkinter import messagebox
from tkinter import StringVar
from tkinter import BooleanVar
```

Why imported. The standard-library GUI toolkit. Powers the desktop trader companion's window.

Used in this file. StringVar, ttk

Example. root = tk.Tk(); tk.Button(root, text='Go').pack()

Other common scenarios.

- ttk.* for the modern themed widgets (Treeview, Notebook, ...)
- after(ms, fn) schedules a callback on the GUI thread
- StringVar / IntVar bind variables to widgets

```
import logging
```

Why imported. Structured, levelled logging. Replaces print() with filterable, formattable output that can route to files, stderr, syslog, or HTTP endpoints.

Used in this file. logging.getLogger

Example. logging.getLogger(__name__).info('connected to %s', host)

Other common scenarios.

- .debug() / .info() / .warning() / .error() / .exception()
- logger.exception() captures the current traceback automatically
- logging.basicConfig(level=logging.INFO) for quick setup
- Handlers and Formatters route logs to files / streams / syslog

```
from typing import Dict
```

```
from typing import Callable
from typing import Optional
```

Why imported. Type-hint primitives — generics, unions, Optional, Callable, Protocol, TypeVar, TypedDict.

Used in this file. Callable, Dict, Optional

Example. `def lookup(k: str) -> Optional[int]: ...`

Other common scenarios.

- `List[int]` / `Dict[str, Any]` / `Tuple[int, str]` (or bare `list[int]`+ on 3.9+)
- `Callable[[int, int], int]` for function signatures
- Protocol for structural typing (duck typing with checks)
- `TypeVar('T')` for generic functions and classes

```
import os
```

Why imported. Operating-system interface. Path manipulation, environment variables, working-directory operations, exit codes, and thin wrappers around POSIX/Windows syscalls.

Used in this file. *No direct attribute access detected — likely imported for side effects, re-export, or string references.*

Example. `os.path.join(root, 'subdir', 'file.txt')` # joins with the OS-correct separator

Other common scenarios.

- `os.environ.get('API_KEY')` to read environment variables
- `os.makedirs(path, exist_ok=True)` to create nested directories
- `os.listdir(path)` to list a directory's contents
- `os.remove(path)` / `os.rename(src, dst)` to delete or move a file
- `os.getcwd()` / `os.chdir(path)` to inspect or change the working dir

```
from datetime import datetime
```

Why imported. Dates, times, durations. The fundamental temporal types: date, time, datetime, timedelta, timezone.

Used in this file. *No direct attribute access detected — likely imported for side effects, re-export, or string references.*

Example. `datetime.now() - timedelta(days=7)` # one week ago

Other common scenarios.

- `datetime.strptime(s, '%Y-%m-%d')` parses a string
- `datetime.strftime(fmt)` formats one
- `datetime.fromtimestamp(n)` converts a Unix epoch
- `datetime.utcnow()` for UTC (better: `datetime.now(timezone.utc)`)

Classes

```
class BrokerConfig
```

Configuration management for Tradovate broker

```
BROKERS = {'tradovate': {'name': 'Tradovate', 'display_name': 'Tradovate (Web-based)', 'type': 'web', 'co
```

Code (copy-paste safe)

```
class BrokerConfig:
    """
    Configuration management for Tradovate broker
    """

    BROKERS = {
        'tradovate': {
            'name': 'Tradovate',
            'display_name': 'Tradovate (Web-based)',
            'type': 'web',
            'connection_class': 'TradovateAccount',
            'description': 'Web-based futures trading platform',
            'features': [
                '■ Web-based platform (no installation)',
                '■ No additional software required',
                '■ $0.85 per contract commission',
                '■ Free platform and data feeds',
                '■ Easy setup and familiar interface',
                '■ Current stable integration'
            ],
            'requirements': [
                'Tradovate account credentials',
                'Internet connection',
                'Chrome browser'
            ],
            'costs': {
                'platform': 'FREE',
                'data': 'FREE',
                'commission': '$0.85/contract',
                'monthly': '$0'
            },
            'setup_complexity': 'Easy',
            'recommended_for': 'All traders',
            'credentials': ['username', 'password']
        }
    }

    @classmethod
    def get_broker_config(cls, broker_type: str) -> Dict:
        """Get configuration for specific broker"""
        return cls.BROKERS.get(broker_type, {})

    @classmethod
    def list_brokers(cls) -> Dict[str, str]:
        """List all available brokers"""
        return {k: v['display_name'] for k, v in cls.BROKERS.items()}
```

Method: BrokerConfig.get_broker_config

```
@classmethod
def get_broker_config(cls, broker_type: str) -> Dict
```

Get configuration for specific broker

Why these Python concepts were used

- **@classmethod** — Marked **@classmethod** — receives the class itself as the first argument. The standard pattern for alternate constructors that build an instance from a different input shape (e.g., from a dict, a CSV row, or an MT5 order tuple).
- **type-annotated parameters** — Parameters carry type hints (e.g., `broker_type: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> Dict`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.

Step by step (top-level body)

1. **return** `cls.BROKERS.get(broker_type, {})`.

Code (copy-paste safe)

```
def get_broker_config(cls, broker_type: str) -> Dict:
    """Get configuration for specific broker"""
    return cls.BROKERS.get(broker_type, {})
```

Method: BrokerConfig.list_brokers

```
@classmethod
def list_brokers(cls) -> Dict[str, str]
```

List all available brokers

Why these Python concepts were used

- **@classmethod** — Marked **@classmethod** — receives the class itself as the first argument. The standard pattern for alternate constructors that build an instance from a different input shape (e.g., from a dict, a CSV row, or an MT5 order tuple).
- **return type annotation** — Annotated return type `-> Dict[str, str]`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **dict comprehension** — Builds a dict in one expression `{k: v for k, v in items}` — same intent as a list comprehension but for mappings.

Step by step (top-level body)

1. **return** `{k: v['display_name'] for k, v in cls.BROKERS.items()}`.

Code (copy-paste safe)

```
def list_brokers(cls) -> Dict[str, str]:
    """List all available brokers"""
    return {k: v['display_name'] for k, v in cls.BROKERS.items()}
```

```
class BrokerSelectionSection
```

Simplified broker selection - Tradovate only

Code (copy-paste safe)

```
class BrokerSelectionSection:
```

```

"""
Simplified broker selection - Tradovate only
"""

def __init__(self, parent, on_selection_change: Optional[Callable] = None):
    self.parent = parent
    self.on_selection_change = on_selection_change
    self.confirmed_broker = 'tradovate' # Default to Tradovate

    # Create section frame
    self.section_frame = ttk.LabelFrame(parent, text="Broker Platform", padding=(10, 10))

    # Simple label since only Tradovate is supported
    self.broker_label = ttk.Label(
        self.section_frame,
        text="Using Tradovate (Web-based futures trading)",
        font=("Segoe UI", 10)
    )
    self.broker_label.pack(anchor="w", pady=(0, 5))

    # Features display
    features_text = "■ No installation required • ■ $0.85/contract • ■ Free platform & data"
    self.features_label = ttk.Label(
        self.section_frame,
        text=features_text,
        font=("Segoe UI", 8),
        foreground="#666666"
    )
    self.features_label.pack(anchor="w")

def pack(self, **kwargs):
    """Pack the section frame"""
    self.section_frame.pack(**kwargs)

def get_selected_broker(self) -> str:
    """Get selected broker"""
    return 'tradovate'

```

Method: BrokerSelectionSection.__init__

```
def __init__(self, parent, on_selection_change: Optional[Callable]=None)
```

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `on_selection_change: Optional[Callable]`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **__init__** — The constructor. Runs after Python has allocated the instance; assigns starting attribute values to self.

Step by step (top-level body)

1. Assigns `self.parent = parent`.
2. Assigns `self.on_selection_change = on_selection_change`.
3. Assigns `self.confirmed_broker = 'tradovate'`.

4. Assigns `self.section_frame = ttk.LabelFrame(parent, text='Broker Platform', padding=(1....`
5. Assigns `self.broker_label = ttk.Label(self.section_frame, text='Using Tradovate (Web-....`
6. Calls `self.broker_label.pack(...)` for its side effect.
7. Assigns `features_text = '■ No installation required • ■ $0.85/contract • ■ Free p....`
8. Assigns `self.features_label = ttk.Label(self.section_frame, text=features_text, font=('...`
9. Calls `self.features_label.pack(...)` for its side effect.

Code (copy-paste safe)

```
def __init__(self, parent, on_selection_change: Optional[Callable] = None):
    self.parent = parent
    self.on_selection_change = on_selection_change
    self.confirmed_broker = 'tradovate' # Default to Tradovate

    # Create section frame
    self.section_frame = ttk.LabelFrame(parent, text="Broker Platform", padding=(10, 10))

    # Simple label since only Tradovate is supported
    self.broker_label = ttk.Label(
        self.section_frame,
        text="Using Tradovate (Web-based futures trading)",
        font=("Segoe UI", 10)
    )
    self.broker_label.pack(anchor="w", pady=(0, 5))

    # Features display
    features_text = "■ No installation required • ■ $0.85/contract • ■ Free platform & data"
    self.features_label = ttk.Label(
        self.section_frame,
        text=features_text,
        font=("Segoe UI", 8),
        foreground="#666666"
    )
    self.features_label.pack(anchor="w")
```

Method: BrokerSelectionSection.pack

```
def pack(self, **kwargs)
```

Pack the section frame

Why these Python concepts were used

- ****kwargs** — Accepts arbitrary keyword arguments. Usually for forwarding to another function (a thin wrapper) or to support optional named parameters without listing them all in the signature.

Step by step (top-level body)

1. Calls `self.section_frame.pack(...)` for its side effect.

Code (copy-paste safe)

```
def pack(self, **kwargs):
    """Pack the section frame"""
    self.section_frame.pack(**kwargs)
```

Method: BrokerSelectionSection.get_selected_broker

```
def get_selected_broker(self) -> str

Get selected broker
```

Why these Python concepts were used

- **return type annotation** — Annotated return type -> str. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.

Step by step (top-level body)

1. **return** 'tradovate'.

Code (copy-paste safe)

```
def get_selected_broker(self) -> str:
    """Get selected broker"""
    return 'tradovate'
```

```
class BrokerCredentialsDialog
```

Credentials dialog - simplified for Tradovate only

Code (copy-paste safe)

```
class BrokerCredentialsDialog:
    """
    Credentials dialog - simplified for Tradovate only
    """

    def __init__(self, parent, broker_id: str = "tradovate"):
        self.parent = parent
        self.broker_id = broker_id
        self.result = None
        self.credentials = {}

    def show(self):
        """Show credentials dialog - for Tradovate, return empty (credentials handled by combine cards)"""
        # For Tradovate, credentials are handled in the combine cards
        return "ok", {}
```

Method: BrokerCredentialsDialog.__init__

```
def __init__(self, parent, broker_id: str='tradovate')
```

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., broker_id: str). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **__init__** — The constructor. Runs after Python has allocated the instance; assigns starting attribute values to self.

Step by step (top-level body)

1. Assigns self.parent = parent.

2. Assigns `self.broker_id = broker_id`.
3. Assigns `self.result = None`.
4. Assigns `self.credentials = {}`.

Code (copy-paste safe)

```
def __init__(self, parent, broker_id: str = "tradovate"):
    self.parent = parent
    self.broker_id = broker_id
    self.result = None
    self.credentials = {}
```

Method: `BrokerCredentialsDialog.show`

```
def show(self)
```

Show credentials dialog - for Tradovate, return empty (credentials handled by combine cards)

Step by step (top-level body)

1. `return ('ok', {})`.

Code (copy-paste safe)

```
def show(self):
    """Show credentials dialog - for Tradovate, return empty (credentials handled by combine cards)"""
    # For Tradovate, credentials are handled in the combine cards
    return "ok", {}
```

```
class BrokerSelectionView
```

Complete broker selection view - simplified for Tradovate

Code (copy-paste safe)

```
class BrokerSelectionView:
    """
    Complete broker selection view - simplified for Tradovate
    """

    def __init__(self, master, on_confirm: Optional[Callable] = None):
        self.master = master
        self.on_confirm = on_confirm
        self.selected_broker = StringVar(value="tradovate")

        # Create main window
        self.root = tk.Toplevel(master)
        self.root.title("Broker Selection")
        self.root.geometry("500x300")
        self.root.resizable(False, False)

        # Center window
        self.root.transient(master)
        self.root.grab_set()

        self._create_widgets()

    def _create_widgets(self):
```

```

"""Create dialog widgets"""
# Header
header_frame = ttk.Frame(self.root, padding=(20, 20, 20, 10))
header_frame.pack(fill="x")

header_label = ttk.Label(
    header_frame,
    text="Trading Platform Selection",
    font=("Segoe UI", 14, "bold")
)
header_label.pack()

# Content
content_frame = ttk.Frame(self.root, padding=(20, 0, 20, 20))
content_frame.pack(fill="both", expand=True)

# Tradovate info
info_text = """■ Tradovate Platform

■ Web-based futures trading platform
■ No installation required
■ $0.85 per contract commission
■ Free platform and data feeds
■ Stable, proven integration

This app is currently optimized for Tradovate trading.
Enter your Tradovate credentials in the combine sections to begin trading."""

info_label = ttk.Label(
    content_frame,
    text=info_text,
    font=("Segoe UI", 10),
    justify="left"
)
info_label.pack(pady=(0, 20))

# Buttons
button_frame = ttk.Frame(content_frame)
button_frame.pack(fill="x")

ttk.Button(
    button_frame,
    text="Continue with Tradovate",
    command=self._confirm_selection
).pack(side="right", padx=(10, 0))

ttk.Button(
    button_frame,
    text="Cancel",
    command=self._cancel
).pack(side="right")

def _confirm_selection(self):
    """Confirm broker selection"""
    if self.on_confirm:
        self.on_confirm("tradovate")
    self.root.destroy()

def _cancel(self):
    """Cancel selection"""

```

```
self.root.destroy()
```

Method: BrokerSelectionView.__init__

```
def __init__(self, master, on_confirm: Optional[Callable]=None)
```

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `on_confirm: Optional[Callable]`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **__init__** — The constructor. Runs after Python has allocated the instance; assigns starting attribute values to self.

Step by step (top-level body)

1. Assigns `self.master = master`.
2. Assigns `self.on_confirm = on_confirm`.
3. Assigns `self.selected_broker = StringVar(value='tradovate')`.
4. Assigns `self.root = tk.Toplevel(master)`.
5. Calls `self.root.title(...)` for its side effect.
6. Calls `self.root.geometry(...)` for its side effect.
7. Calls `self.root.resizable(...)` for its side effect.
8. Calls `self.root.transient(...)` for its side effect.
9. Calls `self.root.grab_set(...)` for its side effect.
10. Calls `self._create_widgets(...)` for its side effect.

Code (copy-paste safe)

```
def __init__(self, master, on_confirm: Optional[Callable] = None):
    self.master = master
    self.on_confirm = on_confirm
    self.selected_broker = StringVar(value="tradovate")

    # Create main window
    self.root = tk.Toplevel(master)
    self.root.title("Broker Selection")
    self.root.geometry("500x300")
    self.root.resizable(False, False)

    # Center window
    self.root.transient(master)
    self.root.grab_set()

    self._create_widgets()
```

Method: BrokerSelectionView._create_widgets

```
def _create_widgets(self)
```

Create dialog widgets

Step by step (top-level body)

1. Assigns `header_frame = ttk.Frame(self.root, padding=(20, 20, 20, 10))`.
2. Calls `header_frame.pack(...)` for its side effect.
3. Assigns `header_label = ttk.Label(header_frame, text='Trading Platform Selection'....`
4. Calls `header_label.pack(...)` for its side effect.
5. Assigns `content_frame = ttk.Frame(self.root, padding=(20, 0, 20, 20))`.
6. Calls `content_frame.pack(...)` for its side effect.
7. Assigns `info_text = '■ Tradovate Platform\n\n■ Web-based futures trading plat....`
8. Assigns `info_label = ttk.Label(content_frame, text=info_text, font=('Segoe UI'....`
9. Calls `info_label.pack(...)` for its side effect.
10. Assigns `button_frame = ttk.Frame(content_frame)`.
11. Calls `button_frame.pack(...)` for its side effect.
12. Calls `tk.Button(button_frame, text='Continue with Tradovate',
command=self._confirm_selection).pack(...)` for its side effect.
13. Calls `tk.Button(button_frame, text='Cancel', command=self._cancel).pack(...)` for its side effect.

Code (copy-paste safe)

```
def _create_widgets(self):
    """Create dialog widgets"""
    # Header
    header_frame = ttk.Frame(self.root, padding=(20, 20, 20, 10))
    header_frame.pack(fill="x")

    header_label = ttk.Label(
        header_frame,
        text="Trading Platform Selection",
        font=("Segoe UI", 14, "bold")
    )
    header_label.pack()

    # Content
    content_frame = ttk.Frame(self.root, padding=(20, 0, 20, 20))
    content_frame.pack(fill="both", expand=True)

    # Tradovate info
    info_text = """■ Tradovate Platform

■ Web-based futures trading platform
■ No installation required
■ $0.85 per contract commission
■ Free platform and data feeds
■ Stable, proven integration

This app is currently optimized for Tradovate trading.
Enter your Tradovate credentials in the combine sections to begin trading."""

    info_label = ttk.Label(
        content_frame,
        text=info_text,
```



```

        font=("Segoe UI", 10),
        justify="left"
    )
    info_label.pack(pady=(0, 20))

    # Buttons
    button_frame = ttk.Frame(content_frame)
    button_frame.pack(fill="x")

    ttk.Button(
        button_frame,
        text="Continue with Tradovate",
        command=self._confirm_selection
    ).pack(side="right", padx=(10, 0))

    ttk.Button(
        button_frame,
        text="Cancel",
        command=self._cancel
    ).pack(side="right")

```

Method: BrokerSelectionView._confirm_selection

```
def _confirm_selection(self)
```

Confirm broker selection

Step by step (top-level body)

1. if self.on_confirm: branches conditionally.
2. Calls self.root.destroy(...) for its side effect.

Code (copy-paste safe)

```

def _confirm_selection(self):
    """Confirm broker selection"""
    if self.on_confirm:
        self.on_confirm("tradovate")
    self.root.destroy()

```

Method: BrokerSelectionView._cancel

```
def _cancel(self)
```

Cancel selection

Step by step (top-level body)

1. Calls self.root.destroy(...) for its side effect.

Code (copy-paste safe)

```

def _cancel(self):
    """Cancel selection"""
    self.root.destroy()

```

```
class BrokerCredentialsDialog
```

Code (copy-paste safe)

```
class BrokerCredentialsDialog:
    def __init__(self, parent, broker_id="tradovate"):
        pass

    def show(self):
        return "ok", {}
```

Method: BrokerCredentialsDialog.__init__

```
def __init__(self, parent, broker_id='tradovate')
```

Why these Python concepts were used

- **__init__** — The constructor. Runs after Python has allocated the instance; assigns starting attribute values to self.

Step by step (top-level body)

1. **pass** (placeholder).

Code (copy-paste safe)

```
def __init__(self, parent, broker_id="tradovate"):
    pass
```

Method: BrokerCredentialsDialog.show

```
def show(self)
```

Step by step (top-level body)

1. **return** ('ok', {}).

Code (copy-paste safe)

```
def show(self):
    return "ok", {}
```

Step 10.2 — Build trader_companion/trader_app.py

What to do. Create the file trader_companion/trader_app.py in your project root and paste in the code shown in this step. The file is **8659** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from requirements.txt.

What this file is for. The Tkinter main window. Wires together the MT5 connection, the prop-firm scrapers, the trade-limit manager, and the hedge protector under a tabbed UI. The largest single file in the codebase.

trader_companion/trader_app.py

8659 loc · 2 classes · 3 functions · 15 imports

The Tkinter main window. Wires together the MT5 connection, the prop-firm scrapers, the trade-limit manager, and the hedge protector under a tabbed UI. The largest single file in the codebase. It defines **2** classes and **3** module-level functions, across **8,659** lines.

Python concepts demonstrated in this module

• Classes and objects • @classmethod and @staticmethod • Comprehensions • Context managers (with-statements) • Decorators • Dunder methods • Exceptions • f-strings • Lambdas • Type hints • Walrus operator (:=)

Imports — what each one is for

```
import sys
```

Why imported. Access to the running interpreter — argv, stdin/stdout, exit codes, the import search path, and the platform name.

Used in this file. sys._MEIPASS, sys.path, sys.path.insert

Example. sys.exit(1) # terminate the process with a non-zero status

Other common scenarios.

- sys.argv[1:] reads the command-line arguments
- sys.path.insert(0, 'extra') prepends to the import search path
- sys.stderr.write(msg) writes to standard error
- sys.platform tells you 'win32', 'linux', or 'darwin'
- sys.modules is the global cache of already-imported modules

```
import os
```

Why imported. Operating-system interface. Path manipulation, environment variables, working-directory operations, exit codes, and thin wrappers around POSIX/Windows syscalls.

Used in this file. os.add_dll_directory, os.environ, os.environ.get, os.path, os.path.abspath, os.path.basename, os.path.dirname, os.path.exists

Example. os.path.join(root, 'subdir', 'file.txt') # joins with the OS-correct separator

Other common scenarios.

- os.environ.get('API_KEY') to read environment variables
- os.makedirs(path, exist_ok=True) to create nested directories
- os.listdir(path) to list a directory's contents
- os.remove(path) / os.rename(src, dst) to delete or move a file
- os.getcwd() / os.chdir(path) to inspect or change the working dir

```
import sys
```

Why imported. Access to the running interpreter — argv, stdin/stdout, exit codes, the import search path, and the platform name.

Used in this file. sys._MEIPASS, sys.path, sys.path.insert

Example. sys.exit(1) # terminate the process with a non-zero status

Other common scenarios.

- sys.argv[1:] reads the command-line arguments
- sys.path.insert(0, 'extra') prepends to the import search path
- sys.stderr.write(msg) writes to standard error
- sys.platform tells you 'win32', 'linux', or 'darwin'
- sys.modules is the global cache of already-imported modules

```
import os
```

Why imported. Operating-system interface. Path manipulation, environment variables, working-directory operations, exit codes, and thin wrappers around POSIX/Windows syscalls.

Used in this file. os.add_dll_directory, os.environ, os.environ.get, os.path, os.path.abspath, os.path.basename, os.path.dirname, os.path.exists

Example. os.path.join(root, 'subdir', 'file.txt') # joins with the OS-correct separator

Other common scenarios.

- os.environ.get('API_KEY') to read environment variables
- os.makedirs(path, exist_ok=True) to create nested directories
- os.listdir(path) to list a directory's contents
- os.remove(path) / os.rename(src, dst) to delete or move a file
- os.getcwd() / os.chdir(path) to inspect or change the working dir

```
import json
```

Why imported. JSON encoding and decoding — the standard wire format for config files, API payloads, and inter-process messages.

Used in this file. json.dump, json.dumps, json.load

Example. json.loads(response.text) # parse a JSON string to dict

Other common scenarios.

- json.dumps(obj, indent=2) for pretty-printing
- json.dump(obj, file) / json.load(file) for file I/O
- default=str handles non-serialisable types like datetime
- object_hook= customises decoding (e.g., to dataclass instances)

```
import gzip as _gzip
```

Why imported. Read/write gzip-compressed streams.

Used in this file. _gzip.compress

Example. with gzip.open('logs.gz', 'rt') as f: ...

Other common scenarios.

- gzip.compress(b) / gzip.decompress(b) for in-memory blobs
- Pair with shutil.copyfileobj for streaming compression

```
import requests
```

Why imported. The de-facto Python HTTP client. Synchronous; trades raw speed for an extremely friendly API.

Used in this file. requests.exceptions, requests.exceptions.ConnectionError, requests.exceptions.Timeout, requests.get, requests.post

Example. requests.get(url, timeout=10).json()

Other common scenarios.

- Session() reuses TCP connections across calls
- params=, headers=, json=, data= for query/body shapes
- raise_for_status() turns 4xx/5xx into an exception
- stream=True for large downloads

```
import time
```

Why imported. Timestamps and sleep. Lower-level than datetime — works in Unix-epoch seconds.

Used in this file. time.sleep, time.time

Example. time.sleep(1.5) # pause this thread for 1.5 seconds

Other common scenarios.

- time.time() returns seconds since the epoch
- time.monotonic() for measuring elapsed time (never goes backward)
- time.perf_counter() for high-precision benchmarking
- time.strftime / time.strptime mirror the datetime versions

```
from datetime import datetime
```

Why imported. Dates, times, durations. The fundamental temporal types: date, time, datetime, timedelta, timezone.

Used in this file. datetime

Example. datetime.now() - timedelta(days=7) # one week ago

Other common scenarios.

- datetime.strptime(s, '%Y-%m-%d') parses a string
- datetime.strftime(fmt) formats one
- datetime.fromtimestamp(n) converts a Unix epoch
- datetime.utcnow() for UTC (better: datetime.now(timezone.utc))

```
import threading
```

Why imported. Threads and synchronisation primitives. Used when work is I/O-bound and the GIL is not a bottleneck (the GIL prevents speed-up on CPU-bound work).

Used in this file. `threading.Event`, `threading.Lock`, `threading.Thread`

Example. `Thread(target=worker, args=(q,), daemon=True).start()`

Other common scenarios.

- `Lock` / `RLock` to guard shared state
- `Event` for one-shot signals between threads
- `Queue` (from `queue`) for producer/consumer patterns
- `threading.local()` for thread-local storage

```
from concurrent.futures import ThreadPoolExecutor
from concurrent.futures import as_completed
```

Why imported. `concurrent.futures` — high-level pools of threads or processes. Submit work, get back a `Future` you can await.

Used in this file. `ThreadPoolExecutor`, `as_completed`

Example. `with ThreadPoolExecutor(8) as ex: results = list(ex.map(fetch, urls))`

Other common scenarios.

- `ProcessPoolExecutor` for CPU-bound work (sidesteps the GIL)
- `ex.submit(fn, *args)` returns a `Future`
- `as_completed(futures)` yields them in completion order

```
from collections import defaultdict
```

Why imported. Specialised container datatypes that the `dict` / `list` / `tuple` / `set` primitives don't cover.

Used in this file. `defaultdict`

Example. `Counter(words)` # multiset that counts occurrences

Other common scenarios.

- `defaultdict(list)` auto-creates a default value on missing keys
- `deque(maxlen=N)` is a double-ended bounded queue
- `OrderedDict` (mostly redundant since 3.7 — dicts preserve order)
- `namedtuple('Point', 'x y')` for tiny immutable record types

```
from typing import Dict
```

Why imported. Type-hint primitives — generics, unions, `Optional`, `Callable`, `Protocol`, `TypeVar`, `TypedDict`.

Used in this file. `Dict`

Example. `def lookup(k: str) -> Optional[int]: ...`

Other common scenarios.

- `List[int]` / `Dict[str, Any]` / `Tuple[int, str]` (or bare `list[int]`+ on 3.9+)
- `Callable[[int, int], int]` for function signatures
- Protocol for structural typing (duck typing with checks)
- `TypeVar('T')` for generic functions and classes

`import re`

Why imported. Regular expressions. Pattern matching, search, replace, and text extraction.

Used in this file. `re.IGNORECASE`, `re.findall`, `re.match`, `re.search`, `re.split`

Example. `re.search(r'\d{3}-\d{4}', text)` # returns a `Match` or `None`

Other common scenarios.

- `re.findall(pattern, text)` for every non-overlapping match
- `re.sub(pattern, repl, text)` to replace occurrences
- `re.compile(...)` to reuse a pattern (faster in a loop)
- Named groups: `r'(?P<year>\d{4})'` lets you do `m.group('year')`

`import random`

Why imported. Pseudo-random numbers — fine for jitter, sampling, shuffling. NEVER for security; use secrets for that.

Used in this file. `random.choice`, `random.randint`, `random.sample`

Example. `random.uniform(0.5, 1.5)` # backoff jitter

Other common scenarios.

- `random.choice(seq)` picks one item
- `random.shuffle(list)` rearranges in place
- `random.seed(n)` makes the sequence reproducible
- `random.sample(seq, k)` draws `k` unique items

Module constants

Each line below is followed by a plain-English note explaining what it does and why it is written that way.

`APP_VERSION = '1.5.6'`

String literal — fixed at code-load time.

`RELEASE_DISABLE_HEDGE_GUARD = True`

Module-level constant — bound once at import time and referenced from the functions and classes below.

`RELEASE_DISABLE_STATUS_POLL = True`

Module-level constant — bound once at import time and referenced from the functions and classes below.

`RELEASE_DISABLE_AUTO_STATUS_UPDATES = True`

Module-level constant — bound once at import time and referenced from the functions and classes below.

```
RELEASE_DISABLE_PROP_DASHBOARD_ACCESS = True
```

Module-level constant — bound once at import time and referenced from the functions and classes below.

```
RELEASE_DISABLE_PUSH_BILLING = True
```

Module-level constant — bound once at import time and referenced from the functions and classes below.

```
_TRADOVATE_IMPORT_ERROR = str(_trado_err) if not TRADOVATE_AVAILABLE and '_trado_err' in dir(  
    ) and _trado_err else None
```

Module-level constant — bound once at import time and referenced from the functions and classes below.

```
_TOPSTEPX_IMPORT_ERROR = str(_tsx_err) if not TOPSTEPX_AVAILABLE and '_tsx_err' in dir(  
    ) and _tsx_err else None
```

Module-level constant — bound once at import time and referenced from the functions and classes below.

```
_FUNDEDNEXT_IMPORT_ERROR = str(_fn_err) if not FUNDEDNEXT_AVAILABLE and '_fn_err' in dir(  
    ) and _fn_err else None
```

Module-level constant — bound once at import time and referenced from the functions and classes below.

Classes

```
class MT5DataPusher
```

Handles MT5 data extraction and API pushing.

Code (copy-paste safe)

```
class MT5DataPusher:  
    """Handles MT5 data extraction and API pushing."""  
  
    def __init__(self, dashboard_url="http://127.0.0.1:5001", api_key=None):  
        self.dashboard_url = dashboard_url.rstrip('/')  
        self.api_key = api_key  
        self.connected = False  
        self.login = None  
        self.server = None  
  
    def connect_mt5(self, login=None, password=None, server=None, terminal_path=None):  
        """Connect to MT5 terminal."""  
        if not MT5_AVAILABLE:  
            err_detail = globals().get('_mt5_err', 'unknown')  
            return False, f"MetaTrader5 module not installed.\n{err_detail}"  
  
        init_params = {}  
        if terminal_path:  
            init_params['path'] = terminal_path
```



```

if not mt5.initialize(**init_params):
    error = mt5.last_error()
    return False, f"MT5 initialization failed: {error}"

if login and password and server:
    try:
        login_int = int(login)
    except ValueError:
        return False, "Login must be a number"

    if not mt5.login(login_int, password=password, server=server):
        error = mt5.last_error()
        return False, f"MT5 login failed: {error}"

    self.login = login_int
    self.server = server

self.connected = True
account = mt5.account_info()
if account:
    # Update server info from actual account connection if not manually provided
    if not self.server:
        self.server = account.server
    # Also store company for FNFT detection
    self.company = account.company
    return True, f"Connected to account #{account.login} ({account.server})"
return True, "Connected to MT5 (no account logged in)"

def disconnect_mt5(self):
    """Disconnect from MT5."""
    if MT5_AVAILABLE:
        mt5.shutdown()
    self.connected = False
    return True, "Disconnected from MT5"

def get_account_info(self, include_balance_history=False):
    """Get account information, optionally including full-history deposits/withdrawals."""
    if not self.connected:
        return None

    account = mt5.account_info()
    if not account:
        return None

    total_deposits = 0.0
    total_withdrawals = 0.0
    if include_balance_history:
        try:
            from_timestamp = 0 # From the beginning
            to_timestamp = time.time() + 86400
            deals = mt5.history_deals_get(from_timestamp, to_timestamp)
            if deals:
                for deal in deals:
                    if deal.type == 2: # DEAL_TYPE_BALANCE
                        if deal.profit > 0:
                            total_deposits += deal.profit
                        else:
                            total_withdrawals += deal.profit
        except Exception as e:
            print(f"Error calculating deposits/withdrawals: {e}")

```

```

return {
    "login": account.login,
    "server": account.server,
    "balance": account.balance,
    "equity": account.equity,
    "profit": account.profit,
    "margin": account.margin,
    "margin_free": account.margin_free,
    "margin_level": account.margin_level if account.margin > 0 else 0,
    "leverage": account.leverage,
    "currency": account.currency,
    "name": account.name,
    "company": account.company,
    "credit": getattr(account, 'credit', 0.0),
    "total_deposits": total_deposits,
    "total_withdrawals": total_withdrawals
}

def get_positions(self):
    """Get open positions."""
    if not self.connected:
        return []

    positions = mt5.positions_get()
    if positions is None:
        return []

    result = []
    for pos in positions:
        result.append({
            "ticket": pos.ticket,
            "symbol": pos.symbol,
            "type": "BUY" if pos.type == 0 else "SELL",
            "volume": pos.volume,
            "price_open": pos.price_open,
            "price_current": pos.price_current,
            "sl": pos.sl,
            "tp": pos.tp,
            "profit": pos.profit,
            "swap": pos.swap,
            "time": datetime.fromtimestamp(pos.time).isoformat(),
            "magic": pos.magic,
            "comment": pos.comment
        })
    return result

def get_deals(self, days=30):
    """Get deal history."""
    if not self.connected:
        return []

    from_timestamp = time.time() - (days * 24 * 3600)
    to_timestamp = time.time() + 86400

    deals = mt5.history_deals_get(from_timestamp, to_timestamp)
    if deals is None:
        return []

    result = []
    for deal in deals:

```

```

        result.append({
            "ticket": deal.ticket,
            "order": deal.order,
            "position_id": deal.position_id,
            "symbol": deal.symbol,
            "type": self._deal_type_to_string(deal.type),
            "entry": self._entry_to_string(deal.entry),
            "volume": deal.volume,
            "price": deal.price,
            "profit": deal.profit,
            "commission": deal.commission,
            "swap": deal.swap,
            "fee": deal.fee,
            "time": datetime.fromtimestamp(deal.time).isoformat(),
            "time_raw": deal.time,
            "magic": deal.magic,
            "comment": deal.comment
        })
    return result

def _deal_type_to_string(self, deal_type):
    types = {0: "BUY", 1: "SELL", 2: "BALANCE", 3: "CREDIT",
             4: "CHARGE", 5: "CORRECTION", 6: "BONUS"}
    return types.get(deal_type, str(deal_type))

def _entry_to_string(self, entry):
    entries = {0: "IN", 1: "OUT", 2: "INOUT", 3: "OUT_BY"}
    return entries.get(entry, str(entry))

def calculate_statistics(self, deals):
    """Calculate trading statistics from deals."""
    if not deals:
        return {}

    # Filter actual trades (not balance operations)
    trades = [d for d in deals if d.get('type') in ['BUY', 'SELL'] and d.get('entry') == 'OUT']

    if not trades:
        return {"total_trades": 0}

    profits = [t['profit'] for t in trades]
    winning = [p for p in profits if p > 0]
    losing = [p for p in profits if p < 0]

    return {
        "total_trades": len(trades),
        "winning_trades": len(winning),
        "losing_trades": len(losing),
        "win_rate": round(len(winning) / len(trades) * 100, 2) if trades else 0,
        "total_profit": round(sum(profits), 2),
        "average_win": round(sum(winning) / len(winning), 2) if winning else 0,
        "average_loss": round(sum(losing) / len(losing), 2) if losing else 0,
        "profit_factor": round(abs(sum(winning) / sum(losing)), 2) if losing and sum(losing) != 0 else 0,
        "largest_win": round(max(winning), 2) if winning else 0,
        "largest_loss": round(min(losing), 2) if losing else 0
    }

def parse_deal_comment_v2(self, comment):
    """
    Parse MT5 deal comment using the new TradeAccountConnector format.

```

Comment Format: {TradovateAccountNumber}{PhaseSuffix}

Phase Suffixes:

- _CH1-4: Challenge Trade 1-4
- _FD0: Funded Base (MFFU style)
- _FD1-4: Funded/Payout 1-4
- _DD1-4: Double Dip 1-4
- _FA: Farming/Consistency
- _FA_DDMMYY: Farming with date (e.g., _FA_210126 = Jan 21, 2026)
- _UNK: Unknown phase

Returns:

dict with parsed data or None if cannot parse

"""

if COMMENT_PARSER_AVAILABLE:

return parse_mt5_comment(comment)

else:

Fallback to basic parsing

return self.parse_deal_comment(comment)

def aggregate_deals_by_comment_v2(self, deals):

"""

Aggregate deals by account and phase using the new comment parser.

Returns:

Tuple of (aggregated_data, unmatched_deals, log_messages)

"""

if COMMENT_PARSER_AVAILABLE:

return aggregate_deals_by_comment(deals)

else:

Fallback to basic aggregation

aggregated, unmatched = self.aggregate_deals_by_account(deals)

return [], unmatched, ["Comment parser not available, using basic aggregation"]

def get_deals_grouped_by_phase(self, days=365):

"""

Get deals grouped by account and phase based on comments.

Uses position-based aggregation which:

- Groups deals by position_id (each closed position has entry + exit deals)
- Gets comment from entry deal (e.g., FNFT...59574_CH1)
- Sums profit from all deals in that position (entry has profit=0, exit has actual profit)

Returns:

dict with structure:

```
{
    'aggregated': [list of aggregated trade data],
    'unmatched': [list of positions without matching comments],
    'summary': {summary statistics},
    'log': [parsing log messages]
}
```

"""

deals = self.get_deals(days=days)

if not deals:

return {'aggregated': [], 'unmatched': [], 'summary': {}, 'log': ['No deals found']}

if COMMENT_PARSER_AVAILABLE:

Use position-based aggregation for correct profit calculation

Entry deal has comment but profit=0, exit deal has profit but different comment

aggregated, unmatched, log = aggregate_deals_by_position(deals)

```

    # Build summary
    by_phase = {}
    for agg in aggregated:
        phase_name = agg.get('phase_name', 'UNKNOWN')
        if phase_name not in by_phase:
            by_phase[phase_name] = {'count': 0, 'total_net_profit': 0.0}
        by_phase[phase_name]['count'] += 1
        by_phase[phase_name]['total_net_profit'] += agg.get('net_profit', 0)

    return {
        'aggregated': aggregated,
        'unmatched': unmatched,
        'summary': {'by_phase': by_phase},
        'log': log
    }
else:
    aggregated, unmatched = self.aggregate_deals_by_account(deals)
    return {
        'aggregated': [],
        'unmatched': unmatched,
        'summary': {},
        'log': ['Comment parser module not available']
    }

def parse_deal_comment(self, comment):
    """
    Parse deal comment to extract account number and stage.

    Comment formats (MFFU example - middle parts abbreviated):
    - MFFU...81001 contains account ending in 81001
    - Stage is identified by _CH{n}, _FU{n}, _FA{n}_ patterns

    Returns:
    dict with 'account_suffix' (last 5 digits), 'stage' (CH/FU/FA), 'stage_num'
    or None if cannot parse
    """
    import re

    if not comment:
        return None

    result = {
        'account_suffix': None,
        'stage': None,
        'stage_num': None,
        'farming_date': None,
        'raw_comment': comment
    }

    # Extract account number - look for 5+ digit sequences
    # The account number appears at the end or within the comment
    account_matches = re.findall(r'(\d{5,})', comment)
    if account_matches:
        # Use the last match, take last 5 digits as identifier
        result['account_suffix'] = account_matches[-1][-5:]

    # Extract stage from comment
    # Challenge: _CH1, _CH2, etc. or CH1, CH2
    ch_match = re.search(r'_?CH(\d+)', comment, re.IGNORECASE)
    if ch_match:

```

```

        result['stage'] = 'CH'
        result['stage_num'] = int(ch_match.group(1))
        return result

# Funded: _FU1, _FU2, etc. or FU1, FU2
fu_match = re.search(r'_?FU(\d+)', comment, re.IGNORECASE)
if fu_match:
    result['stage'] = 'FU'
    result['stage_num'] = int(fu_match.group(1))
    return result

# Farming: _FA1_DD/MM or FA1_DD/MM
fa_match = re.search(r'_?FA(\d+)_?(\d{1,2}[/\-\]\d{1,2})?', comment, re.IGNORECASE)
if fa_match:
    result['stage'] = 'FA'
    result['stage_num'] = int(fa_match.group(1))
    if fa_match.group(2):
        result['farming_date'] = fa_match.group(2)
    return result

# If we found an account but no stage, return partial result
if result['account_suffix']:
    return result

return None

def aggregate_deals_by_account(self, deals):
    """
    Aggregate deals by account number and stage.

    Returns dict: {
        'account_suffix': {
            'CH': {1: total_profit, 2: total_profit, ...},
            'FU': {1: total_profit, 2: total_profit, ...},
            'FA': {1: {'profit': total_profit, 'date': 'DD/MM'}, ...}
        }
    }
    """
    aggregated = {}
    unmatched = []

    for deal in deals:
        # Skip balance operations
        if deal.get('type') in ['BALANCE', 'CREDIT', '2', '3']:
            continue

        comment = deal.get('comment', '')
        parsed = self.parse_deal_comment(comment)

        if not parsed or not parsed['account_suffix']:
            unmatched.append(deal)
            continue

        account = parsed['account_suffix']
        stage = parsed.get('stage')
        stage_num = parsed.get('stage_num')

        if account not in aggregated:
            aggregated[account] = {'CH': {}, 'FU': {}, 'FA': {}}

        # Calculate deal P/L (profit + swap + commission)

```

```

        profit = (deal.get('profit', 0) or 0) + (deal.get('swap', 0) or 0) + (deal.get('commission',
        0) or 0)

    if stage and stage_num:
        if stage == 'FA':
            if stage_num not in aggregated[account]['FA']:
                aggregated[account]['FA'][stage_num] = {'profit': 0, 'date': parsed.get(
                    'farming_date')}
            aggregated[account]['FA'][stage_num]['profit'] += profit
        else:
            if stage_num not in aggregated[account][stage]:
                aggregated[account][stage][stage_num] = 0
            aggregated[account][stage][stage_num] += profit
    else:
        # Has account but no stage - could be a general trade
        unmatched.append(deal)

    return aggregated, unmatched

def push_to_dashboard(self, client_name, admin_name="", trader_name=""):
    """Push all data to the dashboard."""
    if not self.api_key:
        return False, "API key not set"

    print(f"--- Client {client_name} Info Start ---")

    account = self.get_account_info()
    positions = self.get_positions()

    # Calculate days from 23rd of last month
    from datetime import datetime as _dt
    _now = _dt.now()
    if _now.month == 1:
        _from_date = _dt(_now.year - 1, 12, 23)
    else:
        _from_date = _dt(_now.year, _now.month - 1, 23)
    _days_since_23rd = (_now - _from_date).days + 1
    deals = self.get_deals(days=_days_since_23rd)

    # Merge in deep-history farming deals for correct hedge day calculation.
    # 365 days can undercount long-lived farming accounts and push to wrong Hedge Day slot.
    all_deals_full = self.get_deals(days=365) or []
    if deals is None:
        deals = []
    existing_ids = {d.get('ticket') or d.get('order') for d in deals}
    for d in all_deals_full:
        if '_FA' in str(d.get('comment', '')).upper():
            deal_id = d.get('ticket') or d.get('order')
            if deal_id not in existing_ids:
                deals.append(d)
                existing_ids.add(deal_id)

    statistics = self.calculate_statistics(deals)

    payload = {
        "identity": {
            "admin": admin_name or "Admin",
            "trader": trader_name or "Trader",
            "client": client_name
        },
    },

```

```

        "account": account or {},
        "positions": positions,
        "deals": deals,
        "statistics": statistics,
        "evaluations": [],
        "dropdown_options": {}
    }

    headers = {
        "Content-Type": "application/json",
        "X-API-Key": self.api_key
    }

    try:
        response = requests.post(
            f"{self.dashboard_url}/api/update_data",
            json=payload,
            headers=headers,
            timeout=120
        )

        if response.status_code == 200:
            data = response.json()
            print(f"--- Client {client_name} Info End ---")
            if data.get('status') == 'success':
                return True, f"Data pushed successfully for {client_name}"
            return False, data.get('message', 'Unknown error')
        else:
            print(f"--- Client {client_name} Info End (Failed) ---")
            return False, f"HTTP {response.status_code}: {response.text}"

    except requests.exceptions.ConnectionError:
        print(f"--- Client {client_name} Info End (Connection Error) ---")
        return False, f"Cannot connect to dashboard at {self.dashboard_url}"
    except requests.exceptions.Timeout:
        print(f"--- Client {client_name} Info End (Timeout) ---")
        return False, "Request timed out"
    except Exception as e:
        print(f"--- Client {client_name} Info End (Error) ---")
        return False, str(e)

def parse_deal_comment(self, comment):
    """
    Parse MT5 deal comment to extract account number and stage info.

    Comment formats:
    - Challenge: {account}_CH{n} (e.g., "12345_CH1", "67890_CH2")
    - Funded: {account}_FU{n} (e.g., "12345_FU1", "12345_FU2")
    - Farming: {account}_FA{n}_{DD/MM} (e.g., "12345_FA1_15/01")

    Returns dict with:
    - account: The account number (with middle part extracted if needed)
    - stage: 'challenge', 'funded', or 'farming'
    - stage_num: The number (1, 2, 3, etc.)
    - date: Optional date for farming (DD/MM format)
    """
    import re

    if not comment:
        return None

```



```

comment = comment.strip()

# Pattern for Challenge: {account}_CH{n}
ch_match = re.match(r'^(.+?)_CH(\d+)$', comment, re.IGNORECASE)
if ch_match:
    return {
        'account': ch_match.group(1),
        'stage': 'challenge',
        'stage_num': int(ch_match.group(2)),
        'date': None
    }

# Pattern for Funded: {account}_FU{n}
fu_match = re.match(r'^(.+?)_FU(\d+)$', comment, re.IGNORECASE)
if fu_match:
    return {
        'account': fu_match.group(1),
        'stage': 'funded',
        'stage_num': int(fu_match.group(2)),
        'date': None
    }

# Pattern for Double Dip: {account}_DD{n}
dd_match = re.match(r'^(.+?)_DD(\d+)$', comment, re.IGNORECASE)
if dd_match:
    return {
        'account': dd_match.group(1),
        'stage': 'doubledip',
        'stage_num': int(dd_match.group(2)),
        'date': None
    }

# Pattern for Farming: {account}_FA{n}_{DD/MM}
fa_match = re.match(r'^(.+?)_FA(\d+)_(\d{1,2}/\d{1,2})$', comment, re.IGNORECASE)
if fa_match:
    return {
        'account': fa_match.group(1),
        'stage': 'farming',
        'stage_num': int(fa_match.group(2)),
        'date': fa_match.group(3)
    }

return None

def extract_account_core(self, account_num):
    """
    Extract the core/middle part of an account number for matching.
    This handles cases where the full account number might have prefixes/suffixes.

    For example:
    - "HFM-123456-USD" -> "123456"
    - "123456" -> "123456"
    - "ACC123456END" -> "123456" (extracts numeric middle)
    """
    import re

    if not account_num:
        return None

    account_str = str(account_num).strip()

```

```

# First try: extract all digits as a group
digits = re.findall(r'\d+', account_str)
if digits:
    # Return the longest group of digits (likely the account number)
    return max(digits, key=len)

return account_str

def process_deals_for_evaluations(self, deals, evaluations):
    """
    Process deals and match them to evaluations based on comments.
    Uses the new TradeAccountConnector comment format.

    Comment Format: {TradovateAccountNumber}_{Phase}{Number}
    - CH1-4: Challenge Hedge Results 1-5
    - FD0-4: Funded Hedge Results (FD0=base, FD1-4=payouts)
    - DD1-4: Double Dip (treated like funded)
    - FA or FA_DDMMYY: Farming days

    Args:
        deals: List of MT5 deals with 'comment' field
        evaluations: List of evaluation records

    Returns:
        Tuple of (updated_evaluations, match_log)
    """
    if not deals or not evaluations:
        return evaluations, ["No deals or evaluations to process"]

    match_log = []

    # Use the new parser if available
    if COMMENT_PARSER_AVAILABLE:
        return self._process_deals_with_new_parser(deals, evaluations)

    match_log.append("■■■ Using legacy parser - install mt5_comment_parser for full support")
    return self._process_deals_legacy(deals, evaluations)

def _process_deals_with_new_parser(self, deals, evaluations):
    """
    Process deals using the new MT5 comment parser.
    Matches account numbers and updates the correct hedge result fields.
    """
    match_log = []

    # Step 1: Aggregate deals by comment using the new parser
    aggregator = MT5DealAggregator()
    aggregator.process_deals(deals)

    aggregated = aggregator.to_dashboard_format()
    match_log.append(f"■■■ Aggregated {len(aggregated)} trade groups from {len(deals)} deals")
    match_log.append(f"    Unmatched deals: {len(aggregator.unmatched_deals)}")

    if not aggregated:
        match_log.append("■■■ No valid trade groups found in deals")
        return evaluations, match_log

    # Step 2: Build account lookup from evaluations
    # We need to match full account numbers, not just suffixes
    eval_lookup = {} # Maps account_number -> list of (eval_index, account_type)

```

```

for idx, ev in enumerate(evaluations):
    # Challenge account (Account #) - used for CH phase
    ch_account = str(ev.get('Account #', '')).strip()
    if ch_account:
        if ch_account not in eval_lookup:
            eval_lookup[ch_account] = []
        eval_lookup[ch_account].append((idx, 'challenge'))

    # Also add partial matches (last 8-10 chars for flexibility)
    for suffix_len in [8, 10, 12]:
        if len(ch_account) >= suffix_len:
            suffix = ch_account[-suffix_len:]
            if suffix not in eval_lookup:
                eval_lookup[suffix] = []
            eval_lookup[suffix].append((idx, 'challenge'))

    # Funded account (Account #.1) - used for FD, DD, FA phases
    fu_account = str(ev.get('Account #.1', '')).strip()
    if fu_account:
        if fu_account not in eval_lookup:
            eval_lookup[fu_account] = []
        eval_lookup[fu_account].append((idx, 'funded'))

    # Also add partial matches
    for suffix_len in [8, 10, 12]:
        if len(fu_account) >= suffix_len:
            suffix = fu_account[-suffix_len:]
            if suffix not in eval_lookup:
                eval_lookup[suffix] = []
            eval_lookup[suffix].append((idx, 'funded'))

match_log.append(f"■ Built account lookup with {len(eval_lookup)} entries")

# Sample accounts for debug
sample_accounts = list(eval_lookup.keys())[:5]
match_log.append(f"    Sample accounts: {sample_accounts}")

# Step 3: Process each aggregated trade group
updates_made = 0

# Sort aggregated groups by date to ensure chronological order for farming days
aggregated.sort(key=lambda x: str(x.get('farming_date') or ''))

# --- Same-day FA aggregation + hedge-day slot calculation ---
# Step 1: Merge trades on the same date into one entry per (account, date).
#         sum net_profit/deal_count, earliest open_time, latest close_time.
# Step 2: Count ALL distinct FA trading dates per account from the full MT5 history window
#         (acts as our "3-month history scan"). That count IS the hedge day number
#         for the latest trade – no sheet-slot scanning needed.
# Step 3: Only push the LATEST date per account; earlier dates are already in the sheet.
def _normalize_fa_day(_agg):
    """Return canonical YYYY-MM-DD farming day key using timestamps first, then fallback date text
    for _k in ("close_time", "open_time"):
        _tv = _agg.get(_k)
        if _tv:
            _s = str(_tv).replace('T', ' ')
            if len(_s) >= 10:
                return _s[:10]
    _fd = str(_agg.get('farming_date') or '').strip()
    if not _fd:

```

```

        return ''
    # Supports DD/MM from comments, and ISO-like values from parser output.
    try:
        if '/' in _fd:
            _d, _m = _fd.split('/')[2]
            _d = int(_d)
            _m = int(_m)
            _y = datetime.now().year
            return f"{_y:04d}-{_m:02d}-{_d:02d}"
        if '-' in _fd and len(_fd) >= 10:
            return _fd[:10]
    except Exception:
        pass
    return _fd

_fa_by_key = {}    # (account_number, normalized_day_key) -> merged dict
_non_fa = []
for _agg in aggregated:
    if _agg.get('phase_code') == 'FA':
        _date_key = _normalize_fa_day(_agg)
        _key = (_agg.get('account_number', ''), _date_key)
        if _key not in _fa_by_key:
            _fa_by_key[_key] = dict(_agg)
        else:
            _m = _fa_by_key[_key]
            _m['net_profit'] = _m.get('net_profit', 0) + _agg.get('net_profit', 0)
            _m['deal_count'] = _m.get('deal_count', 0) + _agg.get('deal_count', 0)
            # Earliest open_time
            if _agg.get('open_time') and (
                not _m.get('open_time') or str(_agg['open_time']) < str(_m['open_time'])):
                _m['open_time'] = _agg['open_time']
            # Latest close_time (deduplication key)
            if _agg.get('close_time') and (
                not _m.get('close_time') or str(_agg['close_time']) > str(_m['close_time'])):
                _m['close_time'] = _agg['close_time']
        else:
            _non_fa.append(_agg)

# Group merged FA entries by account, sort chronologically.
# The hedge day number for the latest trade = total distinct trading days in history.
# Only keep the latest entry per account for the actual push.
_fa_per_account = {}    # account_number -> sorted list of (normalized_day_key, entry)
for (acct, day_key), entry in _fa_by_key.items():
    _fa_per_account.setdefault(acct, []).append((day_key, entry))

_fa_to_push = []    # only latest per account, tagged with _fa_slot
for acct, date_entries in _fa_per_account.items():
    date_entries.sort(key=lambda x: x[0])    # chronological order by normalized day
    total_days = len(date_entries)    # count = hedge day slot
    latest_day_key, latest_entry = date_entries[-1]
    tagged = dict(latest_entry)
    tagged['_fa_slot'] = total_days    # pre-computed slot number
    _fa_to_push.append(tagged)
    match_log.append(
        f"    ■ {acct}: {total_days} FA day(s) in MT5 history "
        f"→ will push as Hedge Day {total_days} ({latest_day_key})"
    )

# Rebuild aggregated: non-FA entries + one FA entry per account (latest day only)
aggregated = _non_fa + sorted(_fa_to_push, key=lambda x: str(x.get('farming_date') or ''))

```

```

match_log.append(f"    After FA processing: {len(_fa_to_push)} FA account(s) queued for push")

# (legacy fallback: still used for edge-cases without a farming_date)
account_farming_slots = {}

# Pre-build per-account set of close_times already stored in Hedge Day Notes.
# This is the deduplication guard: if a close_time is already recorded in
# any "Hedge Day N Note" field, that trade has already been pushed and we skip it.
# Maps account_number -> set of close_time signature strings
account_pushed_close_times = {}

def _get_pushed_close_times(account_number, phase_code):
    if account_number in account_pushed_close_times:
        return account_pushed_close_times[account_number]
    pushed = set()
    fa_matches = self._find_evaluation_match(account_number, phase_code, eval_lookup)
    for fa_idx, fa_type in (fa_matches or []):
        if fa_type != 'funded':
            continue
        ev = evaluations[fa_idx]
        for day_num in range(1, 51):
            note = ev.get(f'Hedge Day {day_num} Note', '')
            if note and 'Close:' in str(note):
                # Extract close time signature: "Open: ... | Close: 2026-01-21 16:00"
                close_part = str(note).split('Close:')[1].strip()
                if close_part:
                    pushed.add(close_part)
        break # Use first funded eval match
    account_pushed_close_times[account_number] = pushed
    return pushed

for agg in aggregated:
    account_number = agg.get('account_number', '')
    phase_code = agg.get('phase_code', '')
    trade_number = agg.get('trade_number')
    farming_date = agg.get('farming_date')
    net_profit = agg.get('net_profit', 0)
    deal_count = agg.get('deal_count', 0)

    # Find matching evaluation
    eval_matches = self._find_evaluation_match(account_number, phase_code, eval_lookup)

    if not eval_matches:
        match_log.append(
            f"■■■ No match: {account_number}_{phase_code}{trade_number or ''} = ${net_profit:.2f}"
        )
        continue

    # Special handling for Farming phase to ensure sequential day filling
    forced_day_num = None
    skip_farming = False
    if phase_code == 'FA':
        close_time = agg.get('close_time')
        def _fmt_time_fa(t):
            try:
                return str(t)[:16].replace('T', ' ')
            except:
                return str(t)
        close_sig = _fmt_time_fa(close_time) if close_time else None

    # Deduplication: check if this close_time was already pushed

```

```

if close_sig:
    pushed_times = _get_pushed_close_times(account_number, phase_code)
    if close_sig in pushed_times:
        match_log.append(f"■■■ SKIP (already pushed) {account_number}_FA close={close_sig}")
        skip_farming = True

if not skip_farming:
    # Slot number is pre-computed from MT5 history count (distinct FA trading days).
    # Re-pushes on the same day should keep writing to the same Hedge Day slot.
    fa_slot = agg.get('_fa_slot')
    if fa_slot:
        forced_day_num = fa_slot
    else:
        # Fallback for entries without a farming_date / legacy comments
        if trade_number and trade_number > 0:
            forced_day_num = trade_number
        else:
            if account_number not in account_farming_slots:
                account_farming_slots[account_number] = {'__next_slot': 1}
            slot = account_farming_slots[account_number]['__next_slot']
            account_farming_slots[account_number]['__next_slot'] += 1
            forced_day_num = slot

if skip_farming:
    continue

# Determine which field to update based on phase
# Use first match to determine field name logic
field_name = self._get_field_name_for_phase(
    phase_code, trade_number, farming_date, evaluations, eval_matches[0][0],
    forced_day_num=forced_day_num
)

if not field_name:
    match_log.append(f"■■■ Unknown field for {phase_code}{trade_number or ''}")
    continue

# Update ALL matching evaluations
for eval_idx, account_type in eval_matches:
    # Verify this is the right type of match
    if phase_code == 'CH' and account_type != 'challenge':
        continue
    if phase_code in ['FD', 'DD', 'FA'] and account_type != 'funded':
        continue

    # Update the field
    evaluations[eval_idx][field_name] = net_profit

# Store open/close timestamps as a companion note
open_time = agg.get('open_time')
close_time = agg.get('close_time')
def _fmt_time(t):
    try:
        return str(t)[:16].replace('T', ' ')
    except:
        return str(t)
if open_time or close_time:
    note = f"Open: {_fmt_time(open_time)} | Close: {_fmt_time(close_time)}"
    evaluations[eval_idx][f'{field_name} Note'] = note
    # Register this close_time so re-entrant pushes in the same run are also blocked

```

```

        if phase_code == 'FA' and close_time:
            close_sig_written = _fmt_time(close_time)
            account_pushed_close_times.setdefault(account_number, set()).add(close_sig_written)

        updates_made += 1

        eval_account = evaluations[eval_idx].get(
            'Account #' if account_type == 'challenge' else 'Account #.1', 'N/A')
        match_log.append(
            f"■ {account_number}_{phase_code}{trade_number or ''} → [{field_name}] = ${net_profit}")
        match_log.append(f"    Matched to eval row: {eval_account} (Row {eval_idx})")
        if open_time or close_time:
            match_log.append(f"    ■ Open: {_fmt_time(open_time)} | Close: {_fmt_time(close_time)}")

        match_log.append(f"\n■ Total updates made: {updates_made}")
        return evaluations, match_log

def _find_evaluation_match(self, account_number, phase_code, eval_lookup):
    """
    Find matching evaluation(s) for an account number.
    Tries exact match first, then partial matches.
    """
    # Try exact match first
    if account_number in eval_lookup:
        return eval_lookup[account_number]

    # Try matching by suffix (last N characters)
    for suffix_len in [12, 10, 8]:
        if len(account_number) >= suffix_len:
            suffix = account_number[-suffix_len:]
            if suffix in eval_lookup:
                return eval_lookup[suffix]

    # Try finding accounts that contain this number as substring
    for key, matches in eval_lookup.items():
        if account_number in key or key in account_number:
            return matches

    return []

def _get_field_name_for_phase(self, phase_code, trade_number, farming_date, evaluations, eval_idx,
    forced_day_num=None):
    """
    Determine the correct field name to update based on phase.

    Phase mappings:
    - CH1-5: Hedge Result 1-5 (Challenge)
    - FD0: Hedge Result 1.1 (Funded base)
    - FD1-4: Hedge Result 2.1-5.1 (Funded payouts)
    - DD1-4: Hedge Result 6-7 or similar
    - FA: Hedge Day N (based on date ordering)
    """
    if phase_code == 'CH':
        # Challenge: CH1 → Hedge Result 1, CH2 → Hedge Result 2, etc.
        if trade_number and 1 <= trade_number <= 5:
            return f"Hedge Result {trade_number}"

    elif phase_code == 'FD':
        # Funded: FD0 → Hedge Result 1.1, FD1 → Hedge Result 2.1, etc.
        if trade_number is not None:

```

```

        if trade_number == 0:
            return "Hedge Result 1.1"
        elif 1 <= trade_number <= 4:
            return f"Hedge Result {trade_number + 1}.1"
        elif trade_number == 5:
            return "Hedge Result 6"
        elif trade_number == 6:
            return "Hedge Result 7"

    elif phase_code == 'DD':
        # Double Dip: DD1 -> Hedge Result 1.1, DD2 -> Hedge Result 2.1
        if trade_number:
            return f"Hedge Result {trade_number}.1"

    elif phase_code == 'FA':
        # Farming: Use date to determine day number (using forced sequence if available)
        if forced_day_num:
            return f"Hedge Day {forced_day_num}"

        if farming_date:
            # Calculate which farming day this is based on the date
            # Legacy fallback if no forced sequence
            day_number = self._calculate_farming_day(farming_date, evaluations, eval_idx)
            if day_number and 1 <= day_number <= 34:
                return f"Hedge Day {day_number}"
        elif trade_number:
            # If no date but has trade number, use that
            if 1 <= trade_number <= 34:
                return f"Hedge Day {trade_number}"

    return None

def _calculate_farming_day(self, farming_date_str, evaluations, eval_idx):
    """
    Calculate which farming day number to use based on the date.

    Farming dates in comments are DDMMYY format (e.g., 210126 = Jan 21, 2026).
    We need to figure out which day number (1-34) this corresponds to.

    Strategy: Look at existing farming dates in the evaluation to determine sequence,
    or use the first farming date as day 1 and count from there.
    """
    from datetime import datetime

    # Parse the farming date
    if isinstance(farming_date_str, str):
        try:
            farming_date = datetime.fromisoformat(farming_date_str)
        except:
            return None
    else:
        farming_date = farming_date_str

    if not farming_date:
        return None

    # Get the evaluation record to check for existing farming dates
    ev = evaluations[eval_idx] if eval_idx < len(evaluations) else {}

    # Find the first empty farming day slot

```



```

for day_num in range(1, 51):
    field_name = f"Hedge Day {day_num}"
    existing_value = ev.get(field_name)

    # Check if this slot is empty or has no value
    if existing_value is None or existing_value == '' or existing_value == 0:
        return day_num

# All slots full, return the last one
return 50

def _process_deals_legacy(self, deals, evaluations):
    """Legacy deal processing for backward compatibility."""
    match_log = []
    deal_groups = {}

    for deal in deals:
        comment = deal.get('comment', '')
        parsed = self.parse_deal_comment(comment)

        if not parsed or not parsed.get('account_suffix'):
            continue

        # Skip balance operations
        d_type = str(deal.get('type', '')).upper()
        if d_type in ['BALANCE', 'CREDIT', '2', '3']:
            continue

        # Only process closed trades (OUT)
        if deal.get('entry') != 'OUT':
            continue

        account_suffix = parsed['account_suffix']
        stage = parsed.get('stage')
        stage_num = parsed.get('stage_num')

        if not stage or not stage_num:
            continue

        key = (account_suffix, stage, stage_num)

        if key not in deal_groups:
            deal_groups[key] = []
        deal_groups[key].append(deal)

    match_log.append(f"Found {len(deal_groups)} unique account/stage combinations in deals")

    # Build account lookup from evaluations
    # Maps account_suffix (last 5 digits) -> evaluation index
    eval_lookup_ch = {} # Challenge accounts (Account #)
    eval_lookup_fu = {} # Funded accounts (Account #.1)

    for idx, ev in enumerate(evaluations):
        # Challenge account (Account #) - extract last 5 digits for matching
        ch_account = ev.get('Account #', '')
        if ch_account:
            ch_suffix = str(ch_account).strip()[-5:] if len(str(ch_account).strip()) >= 5 else str(
                ch_account).strip()
            if ch_suffix:
                eval_lookup_ch[ch_suffix] = idx

```

```

# Funded account (Account #.1) - extract last 5 digits for matching
fu_account = ev.get('Account #.1', '')
if fu_account:
    fu_suffix = str(fu_account).strip()[-5:] if len(str(fu_account).strip()) >= 5 else str(
        fu_account).strip()
    if fu_suffix:
        eval_lookup_fu[fu_suffix] = idx

match_log.append(
    f"Built lookup: {len(eval_lookup_ch)} challenge accounts, {len(eval_lookup_fu)} funded accounts"

# Debug: Show some of the lookup keys
if eval_lookup_ch:
    sample_ch = list(eval_lookup_ch.keys())[:3]
    match_log.append(f"    Sample CH accounts: {sample_ch}")
if eval_lookup_fu:
    sample_fu = list(eval_lookup_fu.keys())[:3]
    match_log.append(f"    Sample FU accounts: {sample_fu}")

# Process each deal group and update evaluations
for (account_suffix, stage, stage_num), group_deals in deal_groups.items():
    # Calculate total profit for this group
    total_profit = sum(
        (d.get('profit', 0) or 0) + (d.get('swap', 0) or 0) + (d.get('commission', 0) or 0)
        for d in group_deals
    )

    # Find matching evaluation based on stage
    # CH = Challenge (Account #), FU = Funded (Account #.1), FA = Farming (Account #.1)
    eval_idx = None
    if stage == 'CH':
        eval_idx = eval_lookup_ch.get(account_suffix)
    elif stage in ['FU', 'FA']:
        eval_idx = eval_lookup_fu.get(account_suffix)

    if eval_idx is None:
        match_log.append(
            f"■■ No match for {account_suffix}_{stage}{stage_num}: ${total_profit:.2f} ({len(group_deals)} deals)"
        )
        continue

    # Determine field name to update
    if stage == 'CH':
        # Challenge uses: Hedge Result 1, Hedge Result 2, etc.
        field_name = f"Hedge Result {stage_num}"
    elif stage == 'FU':
        # Funded uses: Hedge Result 1.1, Hedge Result 2.1, etc. (up to 7)
        field_name = f"Hedge Result {stage_num}.1"
    elif stage == 'FA':
        # Farming uses: Hedge Day {n}
        field_name = f"Hedge Day {stage_num}"
    else:
        match_log.append(f"■■ Unknown stage {stage} for {account_suffix}")
        continue

    # Update the evaluation – store clean numeric, no $ prefix
    evaluations[eval_idx][field_name] = f"{total_profit:.2f}"
    match_log.append(
        f"✓ {account_suffix}_{stage}{stage_num} -> [{field_name}] = ${total_profit:.2f} ({len(group_deals)} deals)"
    )

return evaluations, match_log

```

Method: MT5DataPusher.__init__

```
def __init__(self, dashboard_url='http://127.0.0.1:5001', api_key=None)
```

Why these Python concepts were used

- **__init__** — The constructor. Runs after Python has allocated the instance; assigns starting attribute values to self.

Step by step (top-level body)

1. Assigns `self.dashboard_url = dashboard_url.rstrip('/')`.
2. Assigns `self.api_key = api_key`.
3. Assigns `self.connected = False`.
4. Assigns `self.login = None`.
5. Assigns `self.server = None`.

Code (copy-paste safe)

```
def __init__(self, dashboard_url="http://127.0.0.1:5001", api_key=None):
    self.dashboard_url = dashboard_url.rstrip('/')
    self.api_key = api_key
    self.connected = False
    self.login = None
    self.server = None
```

Method: MT5DataPusher.connect_mt5

```
def connect_mt5(self, login=None, password=None, server=None, terminal_path=None)
```

Connect to MT5 terminal.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **if not MT5_AVAILABLE**: branches conditionally.
2. Assigns `init_params = {}`.
3. **if terminal_path**: branches conditionally.
4. **if not mt5.initialize(**init_params)**: branches conditionally.
5. **if login and password and server**: branches conditionally.
6. Assigns `self.connected = True`.
7. Assigns `account = mt5.account_info()`.
8. **if account**: branches conditionally.

9. **return** (True, 'Connected to MT5 (no account logged in)').

Code (copy-paste safe)

```
def connect_mt5(self, login=None, password=None, server=None, terminal_path=None):
    """Connect to MT5 terminal."""
    if not MT5_AVAILABLE:
        err_detail = globals().get('_mt5_err', 'unknown')
        return False, f"MetaTrader5 module not installed.\n{err_detail}"

    init_params = {}
    if terminal_path:
        init_params['path'] = terminal_path

    if not mt5.initialize(**init_params):
        error = mt5.last_error()
        return False, f"MT5 initialization failed: {error}"

    if login and password and server:
        try:
            login_int = int(login)
        except ValueError:
            return False, "Login must be a number"

        if not mt5.login(login_int, password=password, server=server):
            error = mt5.last_error()
            return False, f"MT5 login failed: {error}"

        self.login = login_int
        self.server = server

    self.connected = True
    account = mt5.account_info()
    if account:
        # Update server info from actual account connection if not manually provided
        if not self.server:
            self.server = account.server
        # Also store company for FNFT detection
        self.company = account.company
        return True, f"Connected to account #{account.login} ({account.server})"
    return True, "Connected to MT5 (no account logged in)"
```

Method: MT5DataPusher.disconnect_mt5

```
def disconnect_mt5(self)
```

Disconnect from MT5.

Step by step (top-level body)

1. **if** MT5_AVAILABLE: branches conditionally.
2. Assigns self.connected = False.
3. **return** (True, 'Disconnected from MT5').

Code (copy-paste safe)

```
def disconnect_mt5(self):
    """Disconnect from MT5."""
    if MT5_AVAILABLE:
```

```

        mt5.shutdown()
    self.connected = False
    return True, "Disconnected from MT5"

```

Method: MT5DataPusher.get_account_info

```
def get_account_info(self, include_balance_history=False)
```

Get account information, optionally including full-history deposits/withdrawals.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **if** not self.connected: branches conditionally.
2. Assigns account = mt5.account_info().
3. **if** not account: branches conditionally.
4. Assigns total_deposits = 0.0.
5. Assigns total_withdrawals = 0.0.
6. **if** include_balance_history: branches conditionally.
7. **return** {'login': account.login, 'server': account.server, 'balance': accou....

Code (copy-paste safe)

```

def get_account_info(self, include_balance_history=False):
    """Get account information, optionally including full-history deposits/withdrawals."""
    if not self.connected:
        return None

    account = mt5.account_info()
    if not account:
        return None

    total_deposits = 0.0
    total_withdrawals = 0.0
    if include_balance_history:
        try:
            from_timestamp = 0 # From the beginning
            to_timestamp = time.time() + 86400
            deals = mt5.history_deals_get(from_timestamp, to_timestamp)
            if deals:
                for deal in deals:
                    if deal.type == 2: # DEAL_TYPE_BALANCE
                        if deal.profit > 0:
                            total_deposits += deal.profit
                        else:

```

```

        total_withdrawals += deal.profit
    except Exception as e:
        print(f"Error calculating deposits/withdrawals: {e}")

    return {
        "login": account.login,
        "server": account.server,
        "balance": account.balance,
        "equity": account.equity,
        "profit": account.profit,
        "margin": account.margin,
        "margin_free": account.margin_free,
        "margin_level": account.margin_level if account.margin > 0 else 0,
        "leverage": account.leverage,
        "currency": account.currency,
        "name": account.name,
        "company": account.company,
        "credit": getattr(account, 'credit', 0.0),
        "total_deposits": total_deposits,
        "total_withdrawals": total_withdrawals
    }

```

Method: MT5DataPusher.get_positions

```
def get_positions(self)
```

Get open positions.

Why these Python concepts were used

- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **if** not self.connected: branches conditionally.
2. Assigns positions = mt5.positions_get().
3. **if** positions is None: branches conditionally.
4. Assigns result = [].
5. **for** pos in positions: iterates.
6. **return** result.

Code (copy-paste safe)

```

def get_positions(self):
    """Get open positions."""
    if not self.connected:
        return []

    positions = mt5.positions_get()
    if positions is None:
        return []

    result = []
    for pos in positions:
        result.append({
            "ticket": pos.ticket,

```

```

        "symbol": pos.symbol,
        "type": "BUY" if pos.type == 0 else "SELL",
        "volume": pos.volume,
        "price_open": pos.price_open,
        "price_current": pos.price_current,
        "sl": pos.sl,
        "tp": pos.tp,
        "profit": pos.profit,
        "swap": pos.swap,
        "time": datetime.fromtimestamp(pos.time).isoformat(),
        "magic": pos.magic,
        "comment": pos.comment
    })
    return result

```

Method: MT5DataPusher.get_deals

```
def get_deals(self, days=30)
```

Get deal history.

Step by step (top-level body)

1. **if** not self.connected: branches conditionally.
2. Assigns from_timestamp = time.time() - days * 24 * 3600.
3. Assigns to_timestamp = time.time() + 86400.
4. Assigns deals = mt5.history_deals_get(from_timestamp, to_timestamp).
5. **if** deals is None: branches conditionally.
6. Assigns result = [].
7. **for** deal in deals: iterates.
8. **return** result.

Code (copy-paste safe)

```

def get_deals(self, days=30):
    """Get deal history."""
    if not self.connected:
        return []

    from_timestamp = time.time() - (days * 24 * 3600)
    to_timestamp = time.time() + 86400

    deals = mt5.history_deals_get(from_timestamp, to_timestamp)
    if deals is None:
        return []

    result = []
    for deal in deals:
        result.append({
            "ticket": deal.ticket,
            "order": deal.order,
            "position_id": deal.position_id,
            "symbol": deal.symbol,
            "type": self._deal_type_to_string(deal.type),
            "entry": self._entry_to_string(deal.entry),

```

```

        "volume": deal.volume,
        "price": deal.price,
        "profit": deal.profit,
        "commission": deal.commission,
        "swap": deal.swap,
        "fee": deal.fee,
        "time": datetime.fromtimestamp(deal.time).isoformat(),
        "time_raw": deal.time,
        "magic": deal.magic,
        "comment": deal.comment
    })
    return result

```

Method: MT5DataPusher._deal_type_to_string

```
def _deal_type_to_string(self, deal_type)
```

Step by step (top-level body)

1. Assigns types = {0: 'BUY', 1: 'SELL', 2: 'BALANCE', 3: 'CREDIT', 4: 'CHAR....
2. **return** types.get(deal_type, str(deal_type)).

Code (copy-paste safe)

```

def _deal_type_to_string(self, deal_type):
    types = {0: "BUY", 1: "SELL", 2: "BALANCE", 3: "CREDIT",
             4: "CHARGE", 5: "CORRECTION", 6: "BONUS"}
    return types.get(deal_type, str(deal_type))

```

Method: MT5DataPusher._entry_to_string

```
def _entry_to_string(self, entry)
```

Step by step (top-level body)

1. Assigns entries = {0: 'IN', 1: 'OUT', 2: 'INOUT', 3: 'OUT_BY'}.
2. **return** entries.get(entry, str(entry)).

Code (copy-paste safe)

```

def _entry_to_string(self, entry):
    entries = {0: "IN", 1: "OUT", 2: "INOUT", 3: "OUT_BY"}
    return entries.get(entry, str(entry))

```

Method: MT5DataPusher.calculate_statistics

```
def calculate_statistics(self, deals)
```

Calculate trading statistics from deals.

Why these Python concepts were used

- **list comprehension** — Builds a list in a single expression ([f(x) for x in xs]). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. if not deals: branches conditionally.
2. Assigns trades = [d for d in deals if d.get('type') in ['BUY', 'SELL'] and....
3. if not trades: branches conditionally.
4. Assigns profits = [t['profit'] for t in trades].
5. Assigns winning = [p for p in profits if p > 0].
6. Assigns losing = [p for p in profits if p < 0].
7. return {'total_trades': len(trades), 'winning_trades': len(winning), 'losi....

Code (copy-paste safe)

```
def calculate_statistics(self, deals):
    """Calculate trading statistics from deals."""
    if not deals:
        return {}

    # Filter actual trades (not balance operations)
    trades = [d for d in deals if d.get('type') in ['BUY', 'SELL'] and d.get('entry') == 'OUT']

    if not trades:
        return {"total_trades": 0}

    profits = [t['profit'] for t in trades]
    winning = [p for p in profits if p > 0]
    losing = [p for p in profits if p < 0]

    return {
        "total_trades": len(trades),
        "winning_trades": len(winning),
        "losing_trades": len(losing),
        "win_rate": round(len(winning) / len(trades) * 100, 2) if trades else 0,
        "total_profit": round(sum(profits), 2),
        "average_win": round(sum(winning) / len(winning), 2) if winning else 0,
        "average_loss": round(sum(losing) / len(losing), 2) if losing else 0,
        "profit_factor": round(abs(sum(winning) / sum(losing)), 2) if losing and sum(losing) != 0 else 0,
        "largest_win": round(max(winning), 2) if winning else 0,
        "largest_loss": round(min(losing), 2) if losing else 0
    }
```

Method: MT5DataPusher.parse_deal_comment_v2

```
def parse_deal_comment_v2(self, comment)
```

Parse MT5 deal comment using the new TradeAccountConnector format. Comment Format:

{TradovateAccountNumber}{PhaseSuffix} Phase Suffixes: -_CH1-4: Challenge Trade 1-4 -_FD0:

Funded Base (MFFU style) -_FD1-4: Funded/Payout 1-4 -_DD1-4: Double Dip 1-4 -_FA:

Farming/Consistency -_FA_DDMMYY: Farming with date (e.g., _FA_210126 = Jan 21, 2026) -_UNK:

Unknown phase Returns: dict with parsed data or None if cannot parse

Step by step (top-level body)

1. if COMMENT_PARSER_AVAILABLE: branches conditionally (with an **else/elif** arm).

Code (copy-paste safe)

```
def parse_deal_comment_v2(self, comment):
    """
    Parse MT5 deal comment using the new TradeAccountConnector format.

    Comment Format: {TradovateAccountNumber}{PhaseSuffix}

    Phase Suffixes:
    - _CH1-4: Challenge Trade 1-4
    - _FD0: Funded Base (MFFU style)
    - _FD1-4: Funded/Payout 1-4
    - _DD1-4: Double Dip 1-4
    - _FA: Farming/Consistency
    - _FA_DDMMYY: Farming with date (e.g., _FA_210126 = Jan 21, 2026)
    - _UNK: Unknown phase

    Returns:
    dict with parsed data or None if cannot parse
    """
    if COMMENT_PARSER_AVAILABLE:
        return parse_mt5_comment(comment)
    else:
        # Fallback to basic parsing
        return self.parse_deal_comment(comment)
```

Method: MT5DataPusher.aggregate_deals_by_comment_v2

```
def aggregate_deals_by_comment_v2(self, deals)

Aggregate deals by account and phase using the new comment parser. Returns: Tuple of
(aggregated_data, unmatched_deals, log_messages)
```

Step by step (top-level body)

1. if COMMENT_PARSER_AVAILABLE: branches conditionally (with an **else/elif** arm).

Code (copy-paste safe)

```
def aggregate_deals_by_comment_v2(self, deals):
    """
    Aggregate deals by account and phase using the new comment parser.

    Returns:
    Tuple of (aggregated_data, unmatched_deals, log_messages)
    """
    if COMMENT_PARSER_AVAILABLE:
        return aggregate_deals_by_comment(deals)
    else:
        # Fallback to basic aggregation
        aggregated, unmatched = self.aggregate_deals_by_account(deals)
        return [], unmatched, ["Comment parser not available, using basic aggregation"]
```

Method: MT5DataPusher.get_deals_grouped_by_phase

```
def get_deals_grouped_by_phase(self, days=365)

Get deals grouped by account and phase based on comments. Uses position-based aggregation which: -
Groups deals by position_id (each closed position has entry + exit deals) - Gets comment from entry deal
(e.g., FNFT...59574_CH1) - Sums profit from all deals in that position (entry has profit=0, exit has actual
profit) Returns: dict with structure: { 'aggregated': [list of aggregated trade data], 'unmatched': [list of
```

positions without matching comments], 'summary': {summary statistics}, 'log': [parsing log messages]}

Step by step (top-level body)

1. Assigns deals = self.get_deals(days=days).
2. if not deals: branches conditionally.
3. if COMMENT_PARSER_AVAILABLE: branches conditionally (with an **else/elif** arm).

Code (copy-paste safe)

```
def get_deals_grouped_by_phase(self, days=365):
    """
    Get deals grouped by account and phase based on comments.

    Uses position-based aggregation which:
    - Groups deals by position_id (each closed position has entry + exit deals)
    - Gets comment from entry deal (e.g., FNFT...59574_CH1)
    - Sums profit from all deals in that position (entry has profit=0, exit has actual profit)

    Returns:
    dict with structure:
    {
        'aggregated': [list of aggregated trade data],
        'unmatched': [list of positions without matching comments],
        'summary': {summary statistics},
        'log': [parsing log messages]
    }
    """
    deals = self.get_deals(days=days)
    if not deals:
        return {'aggregated': [], 'unmatched': [], 'summary': {}, 'log': ['No deals found']}

    if COMMENT_PARSER_AVAILABLE:
        # Use position-based aggregation for correct profit calculation
        # Entry deal has comment but profit=0, exit deal has profit but different comment
        aggregated, unmatched, log = aggregate_deals_by_position(deals)

        # Build summary
        by_phase = {}
        for agg in aggregated:
            phase_name = agg.get('phase_name', 'UNKNOWN')
            if phase_name not in by_phase:
                by_phase[phase_name] = {'count': 0, 'total_net_profit': 0.0}
            by_phase[phase_name]['count'] += 1
            by_phase[phase_name]['total_net_profit'] += agg.get('net_profit', 0)

        return {
            'aggregated': aggregated,
            'unmatched': unmatched,
            'summary': {'by_phase': by_phase},
            'log': log
        }
    else:
        aggregated, unmatched = self.aggregate_deals_by_account(deals)
        return {
            'aggregated': [],
            'unmatched': unmatched,
            'summary': {},
            'log': ['Comment parser module not available']
        }
```

}

Method: MT5DataPusher.parse_deal_comment

```
def parse_deal_comment(self, comment)
```

Parse deal comment to extract account number and stage. Comment formats (MFFU example - middle parts abbreviated): - MFFU...81001 contains account ending in 81001 - Stage is identified by _CH{n}, _FU{n}, _FA{n}_ patterns Returns: dict with 'account_suffix' (last 5 digits), 'stage' (CH/FU/FA), 'stage_num' or None if cannot parse

Step by step (top-level body)

1. Imports re (lazy import inside the function).
2. if not comment: branches conditionally.
3. Assigns result = {'account_suffix': None, 'stage': None, 'stage_num': None....
4. Assigns account_matches = re.findall('\d{5,}', comment).
5. if account_matches: branches conditionally.
6. Assigns ch_match = re.search('_?CH(\d+)', comment, re.IGNORECASE).
7. if ch_match: branches conditionally.
8. Assigns fu_match = re.search('_?FU(\d+)', comment, re.IGNORECASE).
9. if fu_match: branches conditionally.
10. Assigns fa_match = re.search('_?FA(\d+)_?(\d{1,2}[/\-\]\d{1,2})?', comment....
11. if fa_match: branches conditionally.
12. if result['account_suffix']: branches conditionally.
13. return None.

Code (copy-paste safe)

```
def parse_deal_comment(self, comment):
    """
    Parse deal comment to extract account number and stage.

    Comment formats (MFFU example - middle parts abbreviated):
    - MFFU...81001 contains account ending in 81001
    - Stage is identified by _CH{n}, _FU{n}, _FA{n}_ patterns

    Returns:
        dict with 'account_suffix' (last 5 digits), 'stage' (CH/FU/FA), 'stage_num'
        or None if cannot parse
    """
    import re

    if not comment:
        return None

    result = {
        'account_suffix': None,
        'stage': None,
        'stage_num': None,
        'farming_date': None,
```

```

        'raw_comment': comment
    }

    # Extract account number - look for 5+ digit sequences
    # The account number appears at the end or within the comment
    account_matches = re.findall(r'(\d{5,})', comment)
    if account_matches:
        # Use the last match, take last 5 digits as identifier
        result['account_suffix'] = account_matches[-1][-5:]

    # Extract stage from comment
    # Challenge: _CH1, _CH2, etc. or CH1, CH2
    ch_match = re.search(r'_?CH(\d+)', comment, re.IGNORECASE)
    if ch_match:
        result['stage'] = 'CH'
        result['stage_num'] = int(ch_match.group(1))
        return result

    # Funded: _FU1, _FU2, etc. or FU1, FU2
    fu_match = re.search(r'_?FU(\d+)', comment, re.IGNORECASE)
    if fu_match:
        result['stage'] = 'FU'
        result['stage_num'] = int(fu_match.group(1))
        return result

    # Farming: _FA1_DD/MM or FA1_DD/MM
    fa_match = re.search(r'_?FA(\d+)_?(\d{1,2}[/\-]\d{1,2})?', comment, re.IGNORECASE)
    if fa_match:
        result['stage'] = 'FA'
        result['stage_num'] = int(fa_match.group(1))
        if fa_match.group(2):
            result['farming_date'] = fa_match.group(2)
        return result

    # If we found an account but no stage, return partial result
    if result['account_suffix']:
        return result

    return None

```

Method: MT5DataPusher.aggregate_deals_by_account

```
def aggregate_deals_by_account(self, deals)
```

Aggregate deals by account number and stage. Returns dict: { 'account_suffix': { 'CH': {1: total_profit, 2: total_profit, ...}, 'FU': {1: total_profit, 2: total_profit, ...}, 'FA': {1: {'profit': total_profit, 'date': 'DD/MM'}, ...} }

Step by step (top-level body)

1. Assigns aggregated = {}.
2. Assigns unmatched = [].
3. **for** deal in deals: iterates.
4. **return** (aggregated, unmatched).

Code (copy-paste safe)

```
def aggregate_deals_by_account(self, deals):
    """
```

Aggregate deals by account number and stage.

```
Returns dict: {
    'account_suffix': {
        'CH': {1: total_profit, 2: total_profit, ...},
        'FU': {1: total_profit, 2: total_profit, ...},
        'FA': {1: {'profit': total_profit, 'date': 'DD/MM'}, ...}
    }
}
"""
aggregated = {}
unmatched = []

for deal in deals:
    # Skip balance operations
    if deal.get('type') in ['BALANCE', 'CREDIT', '2', '3']:
        continue

    comment = deal.get('comment', '')
    parsed = self.parse_deal_comment(comment)

    if not parsed or not parsed['account_suffix']:
        unmatched.append(deal)
        continue

    account = parsed['account_suffix']
    stage = parsed.get('stage')
    stage_num = parsed.get('stage_num')

    if account not in aggregated:
        aggregated[account] = {'CH': {}, 'FU': {}, 'FA': {}}

    # Calculate deal P/L (profit + swap + commission)
    profit = (deal.get('profit', 0) or 0) + (deal.get('swap', 0) or 0) + (deal.get('commission',
    0) or 0)

    if stage and stage_num:
        if stage == 'FA':
            if stage_num not in aggregated[account]['FA']:
                aggregated[account]['FA'][stage_num] = {'profit': 0, 'date': parsed.get(
                    'farming_date')}
            aggregated[account]['FA'][stage_num]['profit'] += profit
        else:
            if stage_num not in aggregated[account][stage]:
                aggregated[account][stage][stage_num] = 0
            aggregated[account][stage][stage_num] += profit
    else:
        # Has account but no stage - could be a general trade
        unmatched.append(deal)

return aggregated, unmatched
```

Method: MT5DataPusher.push_to_dashboard

```
def push_to_dashboard(self, client_name, admin_name='', trader_name='')
```

Push all data to the dashboard.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **set comprehension** — Builds a set in one expression — usually to deduplicate the result of a transform.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. `if not self.api_key`: branches conditionally.
2. Calls `print(...)` for its side effect.
3. Assigns `account = self.get_account_info()`.
4. Assigns `positions = self.get_positions()`.
5. Lazy import from `datetime`.
6. Assigns `_now = _dt.now()`.
7. `if _now.month == 1`: branches conditionally (with an **else/elif** arm).
8. Assigns `_days_since_23rd = (_now - _from_date).days + 1`.
9. Assigns `deals = self.get_deals(days=_days_since_23rd)`.
10. Assigns `all_deals_full = self.get_deals(days=365)` or `[]`.
11. `if deals is None`: branches conditionally.
12. Assigns `existing_ids = {d.get('ticket') or d.get('order') for d in deals}`.
13. **for** `d in all_deals_full`: iterates.
14. Assigns `statistics = self.calculate_statistics(deals)`.
15. ... and 3 more statement(s) in the body.

Code (copy-paste safe)

```
def push_to_dashboard(self, client_name, admin_name="", trader_name=""):
    """Push all data to the dashboard."""
    if not self.api_key:
        return False, "API key not set"

    print(f"--- Client {client_name} Info Start ---")

    account = self.get_account_info()
    positions = self.get_positions()

    # Calculate days from 23rd of last month
    from datetime import datetime as _dt
    _now = _dt.now()
    if _now.month == 1:
        _from_date = _dt(_now.year - 1, 12, 23)
    else:
        _from_date = _dt(_now.year, _now.month - 1, 23)
    _days_since_23rd = (_now - _from_date).days + 1
    deals = self.get_deals(days=_days_since_23rd)
```

```

# Merge in deep-history farming deals for correct hedge day calculation.
# 365 days can undercount long-lived farming accounts and push to wrong Hedge Day slot.
all_deals_full = self.get_deals(days=365) or []
if deals is None:
    deals = []
existing_ids = {d.get('ticket') or d.get('order') for d in deals}
for d in all_deals_full:
    if '_FA' in str(d.get('comment', '')).upper():
        deal_id = d.get('ticket') or d.get('order')
        if deal_id not in existing_ids:
            deals.append(d)
            existing_ids.add(deal_id)

statistics = self.calculate_statistics(deals)

payload = {
    "identity": {
        "admin": admin_name or "Admin",
        "trader": trader_name or "Trader",
        "client": client_name
    },
    "account": account or {},
    "positions": positions,
    "deals": deals,
    "statistics": statistics,
    "evaluations": [],
    "dropdown_options": {}
}

headers = {
    "Content-Type": "application/json",
    "X-API-Key": self.api_key
}

try:
    response = requests.post(
        f"{self.dashboard_url}/api/update_data",
        json=payload,
        headers=headers,
        timeout=120
    )

    if response.status_code == 200:
        data = response.json()
        print(f"--- Client {client_name} Info End ---")
        if data.get('status') == 'success':
            return True, f>Data pushed successfully for {client_name}"
        return False, data.get('message', 'Unknown error')
    else:
        print(f"--- Client {client_name} Info End (Failed) ---")
        return False, f"HTTP {response.status_code}: {response.text}"

except requests.exceptions.ConnectionError:
    print(f"--- Client {client_name} Info End (Connection Error) ---")
    return False, f"Cannot connect to dashboard at {self.dashboard_url}"
except requests.exceptions.Timeout:
    print(f"--- Client {client_name} Info End (Timeout) ---")
    return False, "Request timed out"
except Exception as e:
    print(f"--- Client {client_name} Info End (Error) ---")

```



```
return False, str(e)
```

Method: MT5DataPusher.parse_deal_comment

```
def parse_deal_comment(self, comment)
```

Parse MT5 deal comment to extract account number and stage info. Comment formats: - Challenge: {account}_CH{n} (e.g., "12345_CH1", "67890_CH2") - Funded: {account}_FU{n} (e.g., "12345_FU1", "12345_FU2") - Farming: {account}_FA{n}_{DD/MM} (e.g., "12345_FA1_15/01") Returns dict with: - account: The account number (with middle part extracted if needed) - stage: 'challenge', 'funded', or 'farming' - stage_num: The number (1, 2, 3, etc.) - date: Optional date for farming (DD/MM format)

Step by step (top-level body)

1. Imports re (lazy import inside the function).
2. if not comment: branches conditionally.
3. Assigns comment = comment.strip().
4. Assigns ch_match = re.match('^(.+?)_CH(\d+)\$', comment, re.IGNORECASE).
5. if ch_match: branches conditionally.
6. Assigns fu_match = re.match('^(.+?)_FU(\d+)\$', comment, re.IGNORECASE).
7. if fu_match: branches conditionally.
8. Assigns dd_match = re.match('^(.+?)_DD(\d+)\$', comment, re.IGNORECASE).
9. if dd_match: branches conditionally.
10. Assigns fa_match = re.match('^(.+?)_FA(\d+)_\d{1,2}/\d{1,2}\$', comment,....
11. if fa_match: branches conditionally.
12. return None.

Code (copy-paste safe)

```
def parse_deal_comment(self, comment):
    """
    Parse MT5 deal comment to extract account number and stage info.

    Comment formats:
    - Challenge: {account}_CH{n} (e.g., "12345_CH1", "67890_CH2")
    - Funded: {account}_FU{n} (e.g., "12345_FU1", "12345_FU2")
    - Farming: {account}_FA{n}_{DD/MM} (e.g., "12345_FA1_15/01")

    Returns dict with:
    - account: The account number (with middle part extracted if needed)
    - stage: 'challenge', 'funded', or 'farming'
    - stage_num: The number (1, 2, 3, etc.)
    - date: Optional date for farming (DD/MM format)
    """
    import re

    if not comment:
        return None

    comment = comment.strip()

    # Pattern for Challenge: {account}_CH{n}
```

```

ch_match = re.match(r'^(.+?)_CH(\d+)$', comment, re.IGNORECASE)
if ch_match:
    return {
        'account': ch_match.group(1),
        'stage': 'challenge',
        'stage_num': int(ch_match.group(2)),
        'date': None
    }

# Pattern for Funded: {account}_FU{n}
fu_match = re.match(r'^(.+?)_FU(\d+)$', comment, re.IGNORECASE)
if fu_match:
    return {
        'account': fu_match.group(1),
        'stage': 'funded',
        'stage_num': int(fu_match.group(2)),
        'date': None
    }

# Pattern for Double Dip: {account}_DD{n}
dd_match = re.match(r'^(.+?)_DD(\d+)$', comment, re.IGNORECASE)
if dd_match:
    return {
        'account': dd_match.group(1),
        'stage': 'doubledip',
        'stage_num': int(dd_match.group(2)),
        'date': None
    }

# Pattern for Farming: {account}_FA{n}_{DD/MM}
fa_match = re.match(r'^(.+?)_FA(\d+)_(\d{1,2}/\d{1,2})$', comment, re.IGNORECASE)
if fa_match:
    return {
        'account': fa_match.group(1),
        'stage': 'farming',
        'stage_num': int(fa_match.group(2)),
        'date': fa_match.group(3)
    }

return None

```

Method: MT5DataPusher.extract_account_core

```
def extract_account_core(self, account_num)
```

Extract the core/middle part of an account number for matching. This handles cases where the full account number might have prefixes/suffixes. For example: - "HFM-123456-USD" -> "123456" - "123456" -> "123456" - "ACC123456END" -> "123456" (extracts numeric middle)

Step by step (top-level body)

1. Imports re (lazy import inside the function).
2. if not account_num: branches conditionally.
3. Assigns account_str = str(account_num).strip().
4. Assigns digits = re.findall('\d+', account_str).
5. if digits: branches conditionally.

6. **return** account_str.

Code (copy-paste safe)

```
def extract_account_core(self, account_num):
    """
    Extract the core/middle part of an account number for matching.
    This handles cases where the full account number might have prefixes/suffixes.

    For example:
    - "HFM-123456-USD" -> "123456"
    - "123456" -> "123456"
    - "ACC123456END" -> "123456" (extracts numeric middle)
    """
    import re

    if not account_num:
        return None

    account_str = str(account_num).strip()

    # First try: extract all digits as a group
    digits = re.findall(r'\d+', account_str)
    if digits:
        # Return the longest group of digits (likely the account number)
        return max(digits, key=len)

    return account_str
```

Method: MT5DataPusher.process_deals_for_evaluations

```
def process_deals_for_evaluations(self, deals, evaluations)

    Process deals and match them to evaluations based on comments. Uses the new
    TradeAccountConnector comment format. Comment Format:
    {TradovateAccountNumber}_{Phase}{Number} - CH1-4: Challenge Hedge Results 1-5 - FD0-4: Funded
    Hedge Results (FD0=base, FD1-4=payouts) - DD1-4: Double Dip (treated like funded) - FA or
    FA_DDMMYY: Farming days Args: deals: List of MT5 deals with 'comment' field evaluations: List of
    evaluation records Returns: Tuple of (updated_evaluations, match_log)
```

Step by step (top-level body)

1. **if** not deals or not evaluations: branches conditionally.
2. Assigns match_log = [].
3. **if** COMMENT_PARSER_AVAILABLE: branches conditionally.
4. Calls match_log.append(...) for its side effect.
5. **return** self._process_deals_legacy(deals, evaluations).

Code (copy-paste safe)

```
def process_deals_for_evaluations(self, deals, evaluations):
    """
    Process deals and match them to evaluations based on comments.
    Uses the new TradeAccountConnector comment format.

    Comment Format: {TradovateAccountNumber}_{Phase}{Number}
```

- CH1-4: Challenge Hedge Results 1-5
- FD0-4: Funded Hedge Results (FD0=base, FD1-4=payouts)
- DD1-4: Double Dip (treated like funded)
- FA or FA_DDMYY: Farming days

Args:

deals: List of MT5 deals with 'comment' field
evaluations: List of evaluation records

Returns:

Tuple of (updated_evaluations, match_log)

"""

if not deals or not evaluations:

return evaluations, ["No deals or evaluations to process"]

match_log = []

Use the new parser if available

if COMMENT_PARSER_AVAILABLE:

return self._process_deals_with_new_parser(deals, evaluations)

match_log.append("■■■ Using legacy parser - install mt5_comment_parser for full support")

return self._process_deals_legacy(deals, evaluations)

Method: MT5DataPusher._process_deals_with_new_parser

```
def _process_deals_with_new_parser(self, deals, evaluations)
```

Process deals using the new MT5 comment parser. Matches account numbers and updates the correct hedge result fields.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns match_log = [].
2. Assigns aggregator = MT5DealAggregator().
3. Calls aggregator.process_deals(...) for its side effect.
4. Assigns aggregated = aggregator.to_dashboard_format().
5. Calls match_log.append(...) for its side effect.

6. Calls `match_log.append(...)` for its side effect.
7. `if` not aggregated: branches conditionally.
8. Assigns `eval_lookup = {}`.
9. `for (idx, ev) in enumerate(evaluations)`: iterates.
10. Calls `match_log.append(...)` for its side effect.
11. Assigns `sample_accounts = list(eval_lookup.keys())[:5]`.
12. Calls `match_log.append(...)` for its side effect.
13. Assigns `updates_made = 0`.
14. Calls `aggregated.sort(...)` for its side effect.
15. ... and 16 more statement(s) in the body.

Code (copy-paste safe)

```
def _process_deals_with_new_parser(self, deals, evaluations):
    """
    Process deals using the new MT5 comment parser.
    Matches account numbers and updates the correct hedge result fields.
    """
    match_log = []

    # Step 1: Aggregate deals by comment using the new parser
    aggregator = MT5DealAggregator()
    aggregator.process_deals(deals)

    aggregated = aggregator.to_dashboard_format()
    match_log.append(f"■ Aggregated {len(aggregated)} trade groups from {len(deals)} deals")
    match_log.append(f"    Unmatched deals: {len(aggregator.unmatched_deals)}")

    if not aggregated:
        match_log.append("■■ No valid trade groups found in deals")
        return evaluations, match_log

    # Step 2: Build account lookup from evaluations
    # We need to match full account numbers, not just suffixes
    eval_lookup = {} # Maps account_number -> list of (eval_index, account_type)

    for idx, ev in enumerate(evaluations):
        # Challenge account (Account #) - used for CH phase
        ch_account = str(ev.get('Account #', '')).strip()
        if ch_account:
            if ch_account not in eval_lookup:
                eval_lookup[ch_account] = []
            eval_lookup[ch_account].append((idx, 'challenge'))

            # Also add partial matches (last 8-10 chars for flexibility)
            for suffix_len in [8, 10, 12]:
                if len(ch_account) >= suffix_len:
                    suffix = ch_account[-suffix_len:]
                    if suffix not in eval_lookup:
                        eval_lookup[suffix] = []
                    eval_lookup[suffix].append((idx, 'challenge'))

        # Funded account (Account #.1) - used for FD, DD, FA phases
        fu_account = str(ev.get('Account #.1', '')).strip()
```

```

if fu_account:
    if fu_account not in eval_lookup:
        eval_lookup[fu_account] = []
    eval_lookup[fu_account].append((idx, 'funded'))

    # Also add partial matches
    for suffix_len in [8, 10, 12]:
        if len(fu_account) >= suffix_len:
            suffix = fu_account[-suffix_len:]
            if suffix not in eval_lookup:
                eval_lookup[suffix] = []
            eval_lookup[suffix].append((idx, 'funded'))

match_log.append(f"■ Built account lookup with {len(eval_lookup)} entries")

# Sample accounts for debug
sample_accounts = list(eval_lookup.keys())[:5]
match_log.append(f"    Sample accounts: {sample_accounts}")

# Step 3: Process each aggregated trade group
updates_made = 0

# Sort aggregated groups by date to ensure chronological order for farming days
aggregated.sort(key=lambda x: str(x.get('farming_date') or ''))

# --- Same-day FA aggregation + hedge-day slot calculation ---
# Step 1: Merge trades on the same date into one entry per (account, date).
#         sum net_profit/deal_count, earliest open_time, latest close_time.
# Step 2: Count ALL distinct FA trading dates per account from the full MT5 history window
#         (acts as our "3-month history scan"). That count IS the hedge day number
#         for the latest trade – no sheet-slot scanning needed.
# Step 3: Only push the LATEST date per account; earlier dates are already in the sheet.
def _normalize_fa_day(_agg):
    """Return canonical YYYY-MM-DD farming day key using timestamps first, then fallback date text"""
    for _k in ("close_time", "open_time"):
        _tv = _agg.get(_k)
        if _tv:
            _s = str(_tv).replace('T', ' ')
            if len(_s) >= 10:
                return _s[:10]
    _fd = str(_agg.get('farming_date') or '').strip()
    if not _fd:
        return ''
    # Supports DD/MM from comments, and ISO-like values from parser output.
    try:
        if '/' in _fd:
            _d, _m = _fd.split('/')[0:2]
            _d = int(_d)
            _m = int(_m)
            _y = datetime.now().year
            return f"{_y:04d}-{_m:02d}-{_d:02d}"
        if '-' in _fd and len(_fd) >= 10:
            return _fd[:10]
    except Exception:
        pass
    return _fd

_fa_by_key = {} # (account_number, normalized_day_key) -> merged dict
_non_fa = []
for _agg in aggregated:

```

```

if _agg.get('phase_code') == 'FA':
    _date_key = _normalize_fa_day(_agg)
    _key = (_agg.get('account_number', ''), _date_key)
    if _key not in _fa_by_key:
        _fa_by_key[_key] = dict(_agg)
    else:
        _m = _fa_by_key[_key]
        _m['net_profit'] = _m.get('net_profit', 0) + _agg.get('net_profit', 0)
        _m['deal_count'] = _m.get('deal_count', 0) + _agg.get('deal_count', 0)
        # Earliest open_time
        if _agg.get('open_time') and (
            not _m.get('open_time') or str(_agg['open_time']) < str(_m['open_time'])):
            _m['open_time'] = _agg['open_time']
        # Latest close_time (deduplication key)
        if _agg.get('close_time') and (
            not _m.get('close_time') or str(_agg['close_time']) > str(_m['close_time'])):
            _m['close_time'] = _agg['close_time']
else:
    _non_fa.append(_agg)

# Group merged FA entries by account, sort chronologically.
# The hedge day number for the latest trade = total distinct trading days in history.
# Only keep the latest entry per account for the actual push.
_fa_per_account = {} # account_number -> sorted list of (normalized_day_key, entry)
for (acct, day_key), entry in _fa_by_key.items():
    _fa_per_account.setdefault(acct, []).append((day_key, entry))

_fa_to_push = [] # only latest per account, tagged with _fa_slot
for acct, date_entries in _fa_per_account.items():
    date_entries.sort(key=lambda x: x[0]) # chronological order by normalized day
    total_days = len(date_entries) # count = hedge day slot
    latest_day_key, latest_entry = date_entries[-1]
    tagged = dict(latest_entry)
    tagged['_fa_slot'] = total_days # pre-computed slot number
    _fa_to_push.append(tagged)
match_log.append(
    f" ■ {acct}: {total_days} FA day(s) in MT5 history "
    f"→ will push as Hedge Day {total_days} ({latest_day_key})"
)

# Rebuild aggregated: non-FA entries + one FA entry per account (latest day only)
aggregated = _non_fa + sorted(_fa_to_push, key=lambda x: str(x.get('farming_date') or ''))
match_log.append(f" After FA processing: {len(_fa_to_push)} FA account(s) queued for push")

# (legacy fallback: still used for edge-cases without a farming_date)
account_farming_slots = {}

# Pre-build per-account set of close_times already stored in Hedge Day Notes.
# This is the deduplication guard: if a close_time is already recorded in
# any "Hedge Day N Note" field, that trade has already been pushed and we skip it.
# Maps account_number -> set of close_time signature strings
account_pushed_close_times = {}

def _get_pushed_close_times(account_number, phase_code):
    if account_number in account_pushed_close_times:
        return account_pushed_close_times[account_number]
    pushed = set()
    fa_matches = self._find_evaluation_match(account_number, phase_code, eval_lookup)
    for fa_idx, fa_type in (fa_matches or []):
        if fa_type != 'funded':

```

```

        continue
    ev = evaluations[fa_idx]
    for day_num in range(1, 51):
        note = ev.get(f'Hedge Day {day_num} Note', '')
        if note and 'Close:' in str(note):
            # Extract close time signature: "Open: ... | Close: 2026-01-21 16:00"
            close_part = str(note).split('Close:')[1].strip()
            if close_part:
                pushed.add(close_part)
        break # Use first funded eval match
    account_pushed_close_times[account_number] = pushed
    return pushed

for agg in aggregated:
    account_number = agg.get('account_number', '')
    phase_code = agg.get('phase_code', '')
    trade_number = agg.get('trade_number')
    farming_date = agg.get('farming_date')
    net_profit = agg.get('net_profit', 0)
    deal_count = agg.get('deal_count', 0)

    # Find matching evaluation
    eval_matches = self._find_evaluation_match(account_number, phase_code, eval_lookup)

    if not eval_matches:
        match_log.append(
            f"■■ No match: {account_number}_{phase_code}{trade_number or ''} = ${net_profit:.2f}"
        )
        continue

    # Special handling for Farming phase to ensure sequential day filling
    forced_day_num = None
    skip_farming = False
    if phase_code == 'FA':
        close_time = agg.get('close_time')
        def _fmt_time_fa(t):
            try:
                return str(t)[:16].replace('T', ' ')
            except:
                return str(t)
        close_sig = _fmt_time_fa(close_time) if close_time else None

    # Deduplication: check if this close_time was already pushed
    if close_sig:
        pushed_times = _get_pushed_close_times(account_number, phase_code)
        if close_sig in pushed_times:
            match_log.append(f"■■ SKIP (already pushed) {account_number}_FA close={close_sig}")
            skip_farming = True

    if not skip_farming:
        # Slot number is pre-computed from MT5 history count (distinct FA trading days).
        # Re-pushes on the same day should keep writing to the same Hedge Day slot.
        fa_slot = agg.get('_fa_slot')
        if fa_slot:
            forced_day_num = fa_slot
        else:
            # Fallback for entries without a farming_date / legacy comments
            if trade_number and trade_number > 0:
                forced_day_num = trade_number
            else:
                if account_number not in account_farming_slots:

```



```

        account_farming_slots[account_number] = {'__next_slot': 1}
        slot = account_farming_slots[account_number]['__next_slot']
        account_farming_slots[account_number]['__next_slot'] += 1
        forced_day_num = slot

    if skip_farming:
        continue

    # Determine which field to update based on phase
    # Use first match to determine field name logic
    field_name = self._get_field_name_for_phase(
        phase_code, trade_number, farming_date, evaluations, eval_matches[0][0],
        forced_day_num=forced_day_num
    )

    if not field_name:
        match_log.append(f"■ Unknown field for {phase_code}{trade_number or ''}")
        continue

    # Update ALL matching evaluations
    for eval_idx, account_type in eval_matches:
        # Verify this is the right type of match
        if phase_code == 'CH' and account_type != 'challenge':
            continue
        if phase_code in ['FD', 'DD', 'FA'] and account_type != 'funded':
            continue

        # Update the field
        evaluations[eval_idx][field_name] = net_profit

        # Store open/close timestamps as a companion note
        open_time = agg.get('open_time')
        close_time = agg.get('close_time')
        def _fmt_time(t):
            try:
                return str(t)[:16].replace('T', ' ')
            except:
                return str(t)
        if open_time or close_time:
            note = f"Open: {_fmt_time(open_time)} | Close: {_fmt_time(close_time)}"
            evaluations[eval_idx][f'{field_name} Note'] = note
            # Register this close_time so re-entrant pushes in the same run are also blocked
            if phase_code == 'FA' and close_time:
                close_sig_written = _fmt_time(close_time)
                account_pushed_close_times.setdefault(account_number, set()).add(close_sig_written)

        updates_made += 1

    eval_account = evaluations[eval_idx].get(
        'Account #' if account_type == 'challenge' else 'Account #.1', 'N/A')
    match_log.append(
        f"■ {account_number}-{phase_code}{trade_number or ''} → [{field_name}] = ${net_profit}"
    )
    match_log.append(f"    Matched to eval row: {eval_account} (Row {eval_idx})")
    if open_time or close_time:
        match_log.append(f"    ■ Open: {_fmt_time(open_time)} | Close: {_fmt_time(close_time)}")

    match_log.append(f"\n■ Total updates made: {updates_made}")
    return evaluations, match_log

```

Method: MT5DataPusher._find_evaluation_match

```
def _find_evaluation_match(self, account_number, phase_code, eval_lookup)
```

Find matching evaluation(s) for an account number. Tries exact match first, then partial matches.

Step by step (top-level body)

1. **if** `account_number` in `eval_lookup`: branches conditionally.
2. **for** `suffix_len` in `[12, 10, 8]`: iterates.
3. **for** (`key`, `matches`) in `eval_lookup.items()`: iterates.
4. **return** `[]`.

Code (copy-paste safe)

```
def _find_evaluation_match(self, account_number, phase_code, eval_lookup):
    """
    Find matching evaluation(s) for an account number.
    Tries exact match first, then partial matches.
    """
    # Try exact match first
    if account_number in eval_lookup:
        return eval_lookup[account_number]

    # Try matching by suffix (last N characters)
    for suffix_len in [12, 10, 8]:
        if len(account_number) >= suffix_len:
            suffix = account_number[-suffix_len:]
            if suffix in eval_lookup:
                return eval_lookup[suffix]

    # Try finding accounts that contain this number as substring
    for key, matches in eval_lookup.items():
        if account_number in key or key in account_number:
            return matches

    return []
```

Method: MT5DataPusher._get_field_name_for_phase

```
def _get_field_name_for_phase(self, phase_code, trade_number, farming_date, evaluations, eval_idx,
    forced_day_num=None)
```

Determine the correct field name to update based on phase. Phase mappings: - CH1-5: Hedge Result 1-5 (Challenge) - FD0: Hedge Result 1.1 (Funded base) - FD1-4: Hedge Result 2.1-5.1 (Funded payouts) - DD1-4: Hedge Result 6-7 or similar - FA: Hedge Day N (based on date ordering)

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **if** `phase_code == 'CH'`: branches conditionally (with an **else/elif** arm).
2. **return** `None`.

Code (copy-paste safe)

```
def _get_field_name_for_phase(self, phase_code, trade_number, farming_date, evaluations, eval_idx,
                              forced_day_num=None):
    """
    Determine the correct field name to update based on phase.

    Phase mappings:
    - CH1-5: Hedge Result 1-5 (Challenge)
    - FD0: Hedge Result 1.1 (Funded base)
    - FD1-4: Hedge Result 2.1-5.1 (Funded payouts)
    - DD1-4: Hedge Result 6-7 or similar
    - FA: Hedge Day N (based on date ordering)
    """
    if phase_code == 'CH':
        # Challenge: CH1 → Hedge Result 1, CH2 → Hedge Result 2, etc.
        if trade_number and 1 <= trade_number <= 5:
            return f"Hedge Result {trade_number}"

    elif phase_code == 'FD':
        # Funded: FD0 → Hedge Result 1.1, FD1 → Hedge Result 2.1, etc.
        if trade_number is not None:
            if trade_number == 0:
                return "Hedge Result 1.1"
            elif 1 <= trade_number <= 4:
                return f"Hedge Result {trade_number + 1}.1"
            elif trade_number == 5:
                return "Hedge Result 6"
            elif trade_number == 6:
                return "Hedge Result 7"

    elif phase_code == 'DD':
        # Double Dip: DD1 -> Hedge Result 1.1, DD2 -> Hedge Result 2.1
        if trade_number:
            return f"Hedge Result {trade_number}.1"

    elif phase_code == 'FA':
        # Farming: Use date to determine day number (using forced sequence if available)
        if forced_day_num:
            return f"Hedge Day {forced_day_num}"

        if farming_date:
            # Calculate which farming day this is based on the date
            # Legacy fallback if no forced sequence
            day_number = self._calculate_farming_day(farming_date, evaluations, eval_idx)
            if day_number and 1 <= day_number <= 34:
                return f"Hedge Day {day_number}"
        elif trade_number:
            # If no date but has trade number, use that
            if 1 <= trade_number <= 34:
                return f"Hedge Day {trade_number}"

    return None
```

Method: MT5DataPusher._calculate_farming_day

```
def _calculate_farming_day(self, farming_date_str, evaluations, eval_idx)
```

Calculate which farming day number to use based on the date. Farming dates in comments are DDMMYY format (e.g., 210126 = Jan 21, 2026). We need to figure out which day number (1-34) this corresponds to. Strategy: Look at existing farming dates in the evaluation to determine sequence, or use the first farming date as day 1 and count from there.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Lazy import from datetime.
2. `if isinstance(farming_date_str, str):` branches conditionally (with an **else/elif** arm).
3. `if not farming_date:` branches conditionally.
4. Assigns `ev = evaluations[eval_idx]` if `eval_idx < len(evaluations)` else `{}`.
5. `for day_num in range(1, 51):` iterates.
6. `return 50`.

Code (copy-paste safe)

```
def _calculate_farming_day(self, farming_date_str, evaluations, eval_idx):
    """
    Calculate which farming day number to use based on the date.

    Farming dates in comments are DDMMYY format (e.g., 210126 = Jan 21, 2026).
    We need to figure out which day number (1-34) this corresponds to.

    Strategy: Look at existing farming dates in the evaluation to determine sequence,
    or use the first farming date as day 1 and count from there.
    """
    from datetime import datetime

    # Parse the farming date
    if isinstance(farming_date_str, str):
        try:
            farming_date = datetime.fromisoformat(farming_date_str)
        except:
            return None
    else:
        farming_date = farming_date_str

    if not farming_date:
        return None

    # Get the evaluation record to check for existing farming dates
```

```

ev = evaluations[eval_idx] if eval_idx < len(evaluations) else {}

# Find the first empty farming day slot
for day_num in range(1, 51):
    field_name = f"Hedge Day {day_num}"
    existing_value = ev.get(field_name)

    # Check if this slot is empty or has no value
    if existing_value is None or existing_value == '' or existing_value == 0:
        return day_num

# All slots full, return the last one
return 50

```

Method: MT5DataPusher._process_deals_legacy

```
def _process_deals_legacy(self, deals, evaluations)
```

Legacy deal processing for backward compatibility.

Why these Python concepts were used

- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns match_log = [].
2. Assigns deal_groups = {}.
3. **for** deal in deals: iterates.
4. Calls match_log.append(...) for its side effect.
5. Assigns eval_lookup_ch = {}.
6. Assigns eval_lookup_fu = {}.
7. **for** (idx, ev) in enumerate(evaluations): iterates.
8. Calls match_log.append(...) for its side effect.
9. **if** eval_lookup_ch: branches conditionally.
10. **if** eval_lookup_fu: branches conditionally.
11. **for** ((account_suffix, stage, stage_num), group_deals) in deal_groups.items(): iterates.
12. **return** (evaluations, match_log).

Code (copy-paste safe)

```

def _process_deals_legacy(self, deals, evaluations):
    """Legacy deal processing for backward compatibility."""
    match_log = []

```

```

deal_groups = {}

for deal in deals:
    comment = deal.get('comment', '')
    parsed = self.parse_deal_comment(comment)

    if not parsed or not parsed.get('account_suffix'):
        continue

    # Skip balance operations
    d_type = str(deal.get('type', '')).upper()
    if d_type in ['BALANCE', 'CREDIT', '2', '3']:
        continue

    # Only process closed trades (OUT)
    if deal.get('entry') != 'OUT':
        continue

    account_suffix = parsed['account_suffix']
    stage = parsed.get('stage')
    stage_num = parsed.get('stage_num')

    if not stage or not stage_num:
        continue

    key = (account_suffix, stage, stage_num)

    if key not in deal_groups:
        deal_groups[key] = []
    deal_groups[key].append(deal)

match_log.append(f"Found {len(deal_groups)} unique account/stage combinations in deals")

# Build account lookup from evaluations
# Maps account_suffix (last 5 digits) -> evaluation index
eval_lookup_ch = {} # Challenge accounts (Account #)
eval_lookup_fu = {} # Funded accounts (Account #.1)

for idx, ev in enumerate(evaluations):
    # Challenge account (Account #) - extract last 5 digits for matching
    ch_account = ev.get('Account #', '')
    if ch_account:
        ch_suffix = str(ch_account).strip()[-5:] if len(str(ch_account).strip()) >= 5 else str(
            ch_account).strip()
        if ch_suffix:
            eval_lookup_ch[ch_suffix] = idx

    # Funded account (Account #.1) - extract last 5 digits for matching
    fu_account = ev.get('Account #.1', '')
    if fu_account:
        fu_suffix = str(fu_account).strip()[-5:] if len(str(fu_account).strip()) >= 5 else str(
            fu_account).strip()
        if fu_suffix:
            eval_lookup_fu[fu_suffix] = idx

match_log.append(
    f"Built lookup: {len(eval_lookup_ch)} challenge accounts, {len(eval_lookup_fu)} funded accounts")

# Debug: Show some of the lookup keys
if eval_lookup_ch:

```

```

        sample_ch = list(eval_lookup_ch.keys())[3]
        match_log.append(f"    Sample CH accounts: {sample_ch}")
    if eval_lookup_fu:
        sample_fu = list(eval_lookup_fu.keys())[3]
        match_log.append(f"    Sample FU accounts: {sample_fu}")

    # Process each deal group and update evaluations
    for (account_suffix, stage, stage_num), group_deals in deal_groups.items():
        # Calculate total profit for this group
        total_profit = sum(
            (d.get('profit', 0) or 0) + (d.get('swap', 0) or 0) + (d.get('commission', 0) or 0)
            for d in group_deals
        )

        # Find matching evaluation based on stage
        # CH = Challenge (Account #), FU = Funded (Account #.1), FA = Farming (Account #.1)
        eval_idx = None
        if stage == 'CH':
            eval_idx = eval_lookup_ch.get(account_suffix)
        elif stage in ['FU', 'FA']:
            eval_idx = eval_lookup_fu.get(account_suffix)

        if eval_idx is None:
            match_log.append(
                f"■■ No match for {account_suffix}_{stage}{stage_num}: ${total_profit:.2f} ({len(group_deals)} deals)"
            )
            continue

        # Determine field name to update
        if stage == 'CH':
            # Challenge uses: Hedge Result 1, Hedge Result 2, etc.
            field_name = f"Hedge Result {stage_num}"
        elif stage == 'FU':
            # Funded uses: Hedge Result 1.1, Hedge Result 2.1, etc. (up to 7)
            field_name = f"Hedge Result {stage_num}.1"
        elif stage == 'FA':
            # Farming uses: Hedge Day {n}
            field_name = f"Hedge Day {stage_num}"
        else:
            match_log.append(f"■■ Unknown stage {stage} for {account_suffix}")
            continue

        # Update the evaluation – store clean numeric, no $ prefix
        evaluations[eval_idx][field_name] = f"{total_profit:.2f}"
        match_log.append(
            f"✓ {account_suffix}_{stage}{stage_num} -> [{field_name}] = ${total_profit:.2f} ({len(group_deals)} deals)"
        )

    return evaluations, match_log

```

```
class TradeOpssAIApp
```

GUI Application for Tradeopss AI.

```

C_BG = '#0D1117'
C_BG_SEC = '#161B22'
C_BG_THIRD = '#1C2333'
C_BORDER = '#30363D'
C_ACCENT = '#0969DA'
C_ACCENT_HV = '#218BFF'
C_GOLD = '#F59E0B'

```

```

C_SUCCESS = '#1A7F37'
C_ERROR = '#CF222E'
C_TEXT = '#E6EDF3'
C_TEXT_DIM = '#8B949E'
C_TEXT_DARK = '#24292F'
PROP_FIRM_COLORS = {'My Funded Futures': '#3B8ED0', 'MFFU': '#3B8ED0', 'TopStep': '#DA3633',
    'Apex': '#E67E22', 'Fun...
PHASE_BADGE = {'Challenge': ('#FEF3C7', '#92400E'), 'Funded': ('#D1FAE5', '#065F46'), 'Farming': ('#DBEAF
    '#...
_FA_HISTORY_CACHE_TTL = 300
_TRADOVATE_FARMING_CACHE_TTL = 300
_FIRM_MAP = {'My Funded Futures': 'MFFU_Flex', 'MFFU': 'MFFU', 'Funding Ticks': 'FundingTicks',
    'Funded Next'...
_FAILED_STATUSES = {'fail', 'failed', 'breach', 'delete', 'deleted', 'closed', 'sl', 'ended', 'lost'}
_INACTIVE_KEYWORDS = ('fail', 'breach', 'delete', 'closed', 'sl', 'ended', 'lost')
_FUNDED_START_BALANCE_MODE: Dict[str, str] = {'MFFU': 'ZERO', 'MFFU_Flex': 'ZERO', 'TopStep': 'ZERO',
    'Funded Next': 'ACCOUNT_SIZE', 'FundingTicks': 'ACCOUNT_SIZE', 'TradeDay': 'ACCOUNT_SIZE', 'Tradeify
_DAY_ABBREVS = {'mon': 0, 'monday': 0, 'tue': 1, 'tues': 1, 'tuesday': 1, 'wed': 2, 'weds': 2, 'wednesday
    '...
_ALL_PHASE_FIELD_SETS = [('Challenge', [f'Hedge Result {i}' for i in range(1, 6)]), ('Funded', [
    f'Hedge Result {i}.1' for...
_PHASE_GLOW = {'Challenge': ('#F59E0B', '#1A1000', '#FBBF24'), 'Funded': ('#22C55E', '#001A0A', '#4ADE80
    'Fa...
_BROWSER_MONITORED_FIRMS = {'Funded Next': {'class_available': 'CDP_SCRAPERS_AVAILABLE',
    'account_class': 'FundedNextCDPAcco...
_SIGNAL_INDICATORS = None

```

Code (copy-paste safe)

```

class TradeOpssAIApp:
    """GUI Application for Tradeopss AI."""

    # ■■■ Design System (FuturesEngine-inspired Dark Theme) ■■■
    C_BG = "#0D1117" # GitHub dark background
    C_BG_SEC = "#161B22" # Card / secondary surface
    C_BG_THIRD = "#1C2333" # Tertiary / input fields
    C_BORDER = "#30363D" # Subtle borders
    C_ACCENT = "#0969DA" # Blue accent
    C_ACCENT_HV = "#218BFF" # Blue hover
    C_GOLD = "#F59E0B" # Amber gold (brand)
    C_SUCCESS = "#1A7F37" # Green
    C_ERROR = "#CF222E" # Red
    C_TEXT = "#E6EDF3" # Primary text
    C_TEXT_DIM = "#8B949E" # Muted text
    C_TEXT_DARK = "#24292F" # Dark text (for light pill backgrounds)

    PROP_FIRM_COLORS = {
        "My Funded Futures": "#3B8ED0",
        "MFFU": "#3B8ED0",
        "TopStep": "#DA3633",
        "Apex": "#E67E22",
        "Funded Next": "#E91E63",
        "FundingTicks": "#F1C40F",
        "TradeDay": "#9B59B6",
        "Tradeify": "#1ABC9C",
        "Alpha Futures": "#2980B9",
        "Top One Futures": "#0D9488",
    }

    PHASE_BADGE = {

```



```

"Challenge": ("FEF3C7", "92400E"), # warm-yellow bg, brown text
"Funded":   ("D1FAE5", "065F46"), # green bg, dark-green text
"Farming":  ("DBEAFE", "1E40AF"),  # blue bg, dark-blue text
}

def __init__(self):
    # ■■ CTK root window ■■
    if CTK_AVAILABLE:
        ctk.set_appearance_mode("Dark")
        ctk.set_default_color_theme("blue")
        self.root = ctk.CTk()
    else:
        self.root = tk.Tk()
    self.root.title(f"Tradeopss AI v{APP_VERSION}")
    self.root.geometry("680x612")
    self.root.minsize(595, 527)
    if CTK_AVAILABLE:
        self.root.configure(fg_color=self.C_BG)
    else:
        self.root.configure(bg=self.C_BG)
    self.root.resizable(True, True)
    self.root.bind("<Control-m>", self._test_min_equity)

    # Set Window Icon + section logo
    self._section_logo = None # small logo for section headers
    try:
        if hasattr(sys, '_MEIPASS'):
            _base = sys._MEIPASS
        else:
            _base = os.path.dirname(os.path.abspath(__file__))
        from PIL import Image as PILImage, ImageTk
        png_path = os.path.join(_base, 'logo.png')
        if os.path.exists(png_path):
            _pil_icon = PILImage.open(png_path).convert('RGBA')
            bbox = _pil_icon.getbbox()
            if bbox:
                _pil_icon = _pil_icon.crop(bbox)
            w, h = _pil_icon.size
            s = max(w, h)
            # Dark background so logo is visible in taskbar
            _sq = PILImage.new('RGBA', (s, s), (6, 14, 26, 255))
            _sq.paste(_pil_icon, ((s - w) // 2, (s - h) // 2), _pil_icon)
            # Taskbar icon (64x64)
            _taskbar = _sq.resize((64, 64), PILImage.LANCZOS)
            self._app_icon = ImageTk.PhotoImage(_taskbar)
            self.root.iconphoto(True, self._app_icon)
            self.root.after(200, lambda: self.root.iconphoto(True, self._app_icon))
            # Small logo for section headers (18x18)
            _small = _sq.resize((18, 18), PILImage.LANCZOS)
            self._section_logo = ImageTk.PhotoImage(_small)
        else:
            ico_path = os.path.join(_base, 'logo.ico')
            if os.path.exists(ico_path):
                self.root.iconbitmap(ico_path)
                self.root.after(200, lambda: self.root.iconbitmap(ico_path))
    except Exception:
        pass

    self.pusher = MT5DataPusher()
    self.auto_push_enabled = False

```

```

self.auto_push_thread = None
self._push_lock = threading.Lock()
self._push_in_progress = False
self._push_pending = False
self.client_info = None

# Auto-trade scheduler state
self.auto_trade_enabled = False
self.auto_trade_thread = None
self._auto_trade_stop = threading.Event()
self._auto_trade_scheduled_dt = None

# Trading engine state
self.trading_api = None
self.tradovate_account = None
self.topstepx_account = None
self._broker_connections = {
    } # {firm_name: {user_entry, pass_entry, status_var, connect_btn, account, row_frame}}
self._propfirm_browsers = {} # {firm_name: FundedNextAccount/etc} for dashboard scraping
self.prop_firm_mgr = PropFirmManager() if PROP_FIRM_AVAILABLE else None
self._auto_trading_stop = threading.Event()
self._auto_trading_thread = None
self._direction_locks = {}
self._active_trade_rows = []
self._firm_billing_summary = {} # {firm_name: {total_fees, total_payouts, records: [...]}}
self.push_billing_btn = None
self._hedge_protector = None
self._status_poll_active = False # real-time status polling flag
self._last_known_statuses = {} # {acct_display: last_computed_status} for change detection
self._cached_acct_mappings = {} # {firm_name: {acct_key: info}} cached on connect

self._show_login_screen()

# ■■ Login Screen ■■
def _show_login_screen(self):
    """Show a full-screen login/email verification screen – always required."""
    # Try loading saved email to pre-fill
    saved_email = ""
    config_path = os.path.join(os.path.dirname(__file__), "trader_config.json")
    if os.path.exists(config_path):
        try:
            with open(config_path, 'r') as f:
                cfg = json.load(f)
                saved_email = cfg.get('client_email', '').strip()
        except Exception:
            pass

    self._login_frame = ctk.CTkFrame(self.root, fg_color=self.C_BG) if CTK_AVAILABLE else \
        tk.Frame(self.root, bg=self.C_BG)
    self._login_frame.pack(fill="both", expand=True)

    # Center box
    center = ctk.CTkFrame(self._login_frame, fg_color=self.C_BG_SEC, corner_radius=12,
        border_width=1, border_color=self.C_BORDER) if CTK_AVAILABLE else \
        tk.Frame(self._login_frame, bg="#161B22")
    center.place(relx=0.5, rely=0.45, anchor="center")

    if CTK_AVAILABLE:
        ctk.CTkLabel(center, text="Client Verification",
            font=("Segoe UI", 13),

```

```

        text_color=self.C_TEXT).pack(pady=(30, 16))

self._login_email = ctk.CTkEntry(center, placeholder_text="Enter Registered Email",
                                width=300, height=40,
                                font=("Segoe UI", 12),
                                fg_color=self.C_BG_THIRD,
                                border_color=self.C_BORDER,
                                text_color=self.C_TEXT)
self._login_email.pack(pady=(0, 16), padx=30)
self._login_email.bind("<Return>", lambda e: self._verify_login())

# Pre-fill saved email
if saved_email:
    self._login_email.insert(0, saved_email)

self._login_btn = ctk.CTkButton(center, text="VERIFY ACCESS",
                                width=300, height=40,
                                command=self._verify_login,
                                fg_color=self.C_ACCENT,
                                hover_color=self.C_ACCENT_HV,
                                font=("Segoe UI", 12, "bold"))
self._login_btn.pack(pady=(0, 12), padx=30)

self._login_status = ctk.CTkLabel(center, text="",
                                   font=("Segoe UI", 11),
                                   text_color=self.C_ERROR)
self._login_status.pack(pady=(0, 24))

def _verify_login(self):
    """Verify the email against the dashboard API."""
    email = self._login_email.get().strip()
    if not email:
        self._login_status.configure(text="Please enter an email address.")
        return

    self._login_btn.configure(state="disabled", text="VERIFYING...")
    self._login_status.configure(text="", text_color=self.C_TEXT_DIM)
    self.root.update_idletasks()

    def _check():
        try:
            response = requests.post(
                "https://www.tradeopss.com/api/client/auth",
                json={"email": email},
                headers={"Content-Type": "application/json"},
                timeout=30
            )
            if response.status_code == 200:
                data = response.json()
                if data.get("status") == "success":
                    self.root.after(0, lambda: self._finish_login(email))
                    return
                else:
                    msg = data.get("message", "Email not found")
                    self.root.after(0, lambda: self._login_fail(f"Access Denied: {msg}"))
                    return
            else:
                self.root.after(0, lambda: self._login_fail(f"Server error ({response.status_code})"))
                return
        except requests.exceptions.ConnectionError:

```

```

        self.root.after(0, lambda: self._login_fail("Cannot connect to server"))
    except Exception as e:
        self.root.after(0, lambda: self._login_fail(str(e)))

    threading.Thread(target=_check, daemon=True).start()

def _login_fail(self, msg):
    """Show login failure message."""
    self._login_status.configure(text=msg, text_color=self.C_ERROR)
    self._login_btn.configure(state="normal", text="VERIFY ACCESS")

def _finish_login(self, email):
    """Tear down login screen, build main UI, and auto-lookup."""
    if hasattr(self, '_login_frame'):
        self._login_frame.destroy()

    self.setup_ui()
    self.load_config()

    # Set the email and trigger lookup
    self.client_email_entry.delete(0, tk.END)
    self.client_email_entry.insert(0, email)
    self.root.after(200, self.lookup_client)

# ■■ Helper: create a section card ■■
def _section_card(self, parent, title="", icon="", **kw):
    """Create a styled card frame with optional title strip."""
    card = ctk.CTkFrame(parent, fg_color=self.C_BG_SEC, corner_radius=8,
                        border_width=1, border_color=self.C_BORDER) if CTK_AVAILABLE else \
        tk.Frame(parent, bg='#161B22')
    if title and CTK_AVAILABLE:
        hdr = ctk.CTkFrame(card, fg_color="transparent", height=26)
        hdr.pack(fill="x", padx=10, pady=(6, 0))
        hdr.pack_propagate(False)
        if self._section_logo:
            lbl = ctk.CTkLabel(hdr, text="", image=self._section_logo,
                              width=18, height=18)
            lbl.pack(side="left", padx=(0, 6))
        elif icon:
            ctk.CTkLabel(hdr, text=icon, font=("Segoe UI", 12)).pack(side="left", padx=(0, 6))
        ctk.CTkLabel(hdr, text=title, font=("Segoe UI", 10, "bold"),
                    text_color=self.C_GOLD).pack(side="left")
    return card

# ■■ Helper: styled Ctk entry ■■
def _ctk_entry(self, parent, width=200, show=None, placeholder=None):
    if CTK_AVAILABLE:
        kw = dict(width=width, height=32, fg_color=self.C_BG_THIRD,
                  border_color=self.C_BORDER, text_color=self.C_TEXT,
                  font=("Segoe UI", 11))
        if show:
            kw["show"] = show
        if placeholder:
            kw["placeholder_text"] = placeholder
        return ctk.CTkEntry(parent, **kw)
    else:
        e = ttk.Entry(parent, width=width // 8)
        if show:
            e.configure(show=show)
        return e

```

```
# ■■■ Helper: styled CTK button ■■■
def _ctk_button(self, parent, text="", command=None, fg=None, hover=None, width=140, **kw):
    fg = fg or self.C_ACCENT
    hover = hover or self.C_ACCENT_HV
    if CTk_AVAILABLE:
        return ctk.CTkButton(parent, text=text, command=command,
                              fg_color=fg, hover_color=hover,
                              font=("Segoe UI", 11, "bold"),
                              corner_radius=6, height=34, width=width, **kw)
    else:
        return ttk.Button(parent, text=text, command=command)

# ■■■ Helper: status pill ■■■
def _status_pill(self, parent, text, bg_color, text_color):
    if CTk_AVAILABLE:
        pill = ctk.CTkFrame(parent, fg_color=bg_color, corner_radius=8, height=22)
        ctk.CTkLabel(pill, text=text, font=("Segoe UI", 9, "bold"),
                      text_color=text_color).pack(padx=8, pady=2)
        return pill
    else:
        lbl = tk.Label(parent, text=text, bg=bg_color, fg=text_color,
                       font=("Segoe UI", 9, "bold"), padx=6, pady=1)
        return lbl

def setup_ui(self):
    """Setup the modern CTK user interface – two-column single-screen layout."""
    if not CTk_AVAILABLE:
        # ■■■ Fallback: simple ttk layout ■■■
        self.main_canvas = tk.Canvas(self.root, bg=self.C_BG, highlightthickness=0)
        sb = ttk.Scrollbar(self.root, orient="vertical", command=self.main_canvas.yview)
        self.scrollable_frame = ttk.Frame(self.main_canvas)
        self.scrollable_frame.bind("<Configure>",
                                    lambda e: self.main_canvas.configure(scrollregion=self.main_canvas.bbox("all")))
        self.main_canvas.create_window((0, 0), window=self.scrollable_frame, anchor="nw")
        self.main_canvas.configure(yscrollcommand=sb.set)
        self.main_canvas.pack(side="left", fill="both", expand=True)
        sb.pack(side="right", fill="y")
        main = self.scrollable_frame
        style = ttk.Style(); style.theme_use('clam')
        self.notebook = ttk.Notebook(main)
        self.notebook.pack(fill="both", expand=True, padx=8, pady=4)
        tab_dash = ttk.Frame(self.notebook); self.notebook.add(tab_dash, text="Settings")
        self._build_dashboard_tab(tab_dash)
        self._build_trading_engine_ui(tab_dash)
        log_frame = ttk.LabelFrame(main, text="Status Log", padding=4)
        log_frame.pack(fill="both", expand=True, padx=8, pady=4)
        self.log_text = scrolledtext.ScrolledText(log_frame, height=6, bg='#0a0e1a',
                                                    fg='#22c55e', font=('Consolas', 9),
                                                    insertbackground='white', relief='flat')
        self.log_text.pack(fill="both", expand=True)
        self.status_var = tk.StringVar(value="Ready")
        self.status_label = ttk.Label(main, textvariable=self.status_var)
        self.status_label.pack(fill="x", padx=8, pady=(0, 4))
        self.last_deal_ticket = 0
        self.last_deal_count = 0
        self.auto_push_thread = None
        # Activity feed list (fallback)
        self._activity_items = []
        return
```

[illegible]

```

        command=self._toggle_auto_trade,
        fg=self.C_ACCENT, hover=self.C_ACCENT_HV, width=110)
self.auto_trade_btn.pack(side="left", padx=(8, 4), pady=5)

self.auto_trade_immediate_var = tk.BooleanVar(value=False)
self.auto_trade_signal_var = tk.BooleanVar(value=False)
if CTK_AVAILABLE:
    ctk.CTkCheckBox(toolbar, text="Now", variable=self.auto_trade_immediate_var,
                    font=("Segoe UI", 9), text_color=self.C_TEXT_DIM,
                    fg_color=self.C_ACCENT, border_color=self.C_BORDER,
                    hover_color=self.C_ACCENT_HV, width=40,
                    checkbox_width=16, checkbox_height=16).pack(side="left", padx=(0, 6), pady=5)
    ctk.CTkCheckBox(toolbar, text="Actual Signal", variable=self.auto_trade_signal_var,
                    font=("Segoe UI", 9), text_color=self.C_TEXT_DIM,
                    fg_color="#f59e0b", border_color=self.C_BORDER,
                    hover_color="#d97706", width=90,
                    checkbox_width=16, checkbox_height=16).pack(side="left", padx=(0, 6), pady=5)

self.auto_trade_status_var = tk.StringVar(value="Off")
ctk.CTkLabel(toolbar, textvariable=self.auto_trade_status_var,
             font=("Segoe UI", 9), text_color=self.C_TEXT_DIM).pack(side="left", padx=(0, 6))

self.auto_trade_countdown_var = tk.StringVar(value="")
ctk.CTkLabel(toolbar, textvariable=self.auto_trade_countdown_var,
             font=("Consolas", 9), text_color=self.C_GOLD).pack(side="left")

self.auto_trade_firms_var = tk.StringVar(value="")

if not RELEASE_DISABLE_PUSH BILLING:
    # Separator before Push Billing
    ctk.CTkFrame(toolbar, width=1, fg_color=self.C_BORDER).pack(side="left", fill="y", pady=6)

    self.push_billing_btn = self._ctk_button(toolbar, text="■ Push Billing",
                                             command=self._push_billing_data,
                                             fg="#f59e0b", hover="#d97706", width=110)
    self.push_billing_btn.pack(side="left", padx=(8, 4), pady=5)

self._ctk_button(toolbar, text="Save Config", command=self.save_config,
                 fg=self.C_BG_THIRD, hover=self.C_BORDER, width=90).pack(side="right", padx=(0,
3), pady=5)

self._ctk_button(toolbar, text="X Close All", command=self._close_all_trades,
                 fg="#DC2626", hover="#B91C1C", width=90).pack(side="right", padx=(0, 4), pady=5)

# ■■■ Row 1: Active Trades (main area – futuristic terminal) ■■■
trades_card = ctk.CTkFrame(self._live_view, fg_color="#000000", corner_radius=10,
                           border_width=1, border_color="#0F4C75")
trades_card.grid(row=1, column=0, sticky="nsew", pady=(0, 4))

# Terminal-style top bezel
trades_bezel = ctk.CTkFrame(trades_card, fg_color="#030D1B", height=36, corner_radius=0)
trades_bezel.pack(fill="x", padx=3, pady=(3, 0))
trades_bezel.pack_propagate(False)

# Traffic-light dots
dot_bar = ctk.CTkFrame(trades_bezel, fg_color="transparent")
dot_bar.pack(side="left", padx=10)
for c in ["#EF4444", "#F59E0B", "#22C55E"]:
    ctk.CTkFrame(dot_bar, width=8, height=8, fg_color=c,
                 corner_radius=4).pack(side="left", padx=2, pady=8)

```

```

    ctk.CTkLabel(trades_bezel, text="■ ACTIVE TRADES",
                font=("Consolas", 11, "bold"),
                text_color="#00D4FF").pack(side="left", padx=(10, 0))

    self.trades_count_var = tk.StringVar(value="[ - ]")
    ctk.CTkLabel(trades_bezel, textvariable=self.trades_count_var,
                font=("Consolas", 9), text_color="#3B6978").pack(side="left", padx=14)

    self.load_trades_btn = ctk.CTkButton(trades_bezel, text="■ SCAN", width=70, height=24,
                                        command=self._load_active_trades,
                                        fg_color="#0A2647", hover_color="#144272",
                                        border_width=1, border_color="#205295",
                                        font=("Consolas", 9, "bold"),
                                        text_color="#00D4FF", corner_radius=4)
    self.load_trades_btn.pack(side="right", padx=10)

    # Column headers – grid-aligned with row content
    hdr = ctk.CTkFrame(trades_card, fg_color="#060E1A", corner_radius=0, height=26)
    hdr.pack(fill="x", padx=3, pady=(1, 0))
    hdr.pack_propagate(False)
    # 3px accent spacer to match row left bar
    ctk.CTkFrame(hdr, width=3, fg_color="transparent").pack(side="left")
    for label, w in [("PROP FIRM", 110), ("ACCOUNT", 88), ("SIZE", 68),
                    ("PHASE", 88), ("NEXT", 100)]:
        ctk.CTkLabel(hdr, text=label, width=w,
                    font=("Consolas", 8, "bold"),
                    text_color="#3B6978", anchor="w").pack(side="left", padx=(8, 0))
    ctk.CTkLabel(hdr, text="ACTION",
                font=("Consolas", 8, "bold"),
                text_color="#3B6978").pack(side="right", padx=(0, 24))

    # Scrollable trade rows (fills remaining space)
    self._trades_scroll = ctk.CTkScrollableFrame(trades_card, fg_color="#020A14")
    self._trades_scroll.pack(fill="both", expand=True, padx=3, pady=(0, 3))
    self._trades_inner = self._trades_scroll

    # ■■■ Row 2: Live Activity (compact bottom) ■■■
    display_frame = ctk.CTkFrame(self._live_view, fg_color="#000000", corner_radius=10,
                                border_width=1, border_color="#1E293B")
    display_frame.grid(row=2, column=0, sticky="nsew")

    # Screen header – monitor bezel
    screen_top = ctk.CTkFrame(display_frame, fg_color="#0F172A", height=32, corner_radius=0)
    screen_top.pack(fill="x", padx=3, pady=(3, 0))
    screen_top.pack_propagate(False)
    dot_row = ctk.CTkFrame(screen_top, fg_color="transparent")
    dot_row.pack(side="left", padx=10)
    for c in ["#EF4444", "#F59E0B", "#22C55E"]:
        ctk.CTkFrame(dot_row, width=8, height=8, fg_color=c,
                    corner_radius=4).pack(side="left", padx=2, pady=8)
    ctk.CTkLabel(screen_top, text="LIVE ACTIVITY", font=("Consolas", 10, "bold"),
                text_color="#64748B").pack(side="left", padx=(10, 0))
    self._live_clock_var = tk.StringVar(value="")
    ctk.CTkLabel(screen_top, textvariable=self._live_clock_var,
                font=("Consolas", 9), text_color="#475569").pack(side="right", padx=10)
    self._tick_live_clock()

    # Activity feed
    self._activity_scroll = ctk.CTkScrollableFrame(display_frame, fg_color="#020617",
                                                corner_radius=0)

```



```

self._activity_scroll.pack(fill="both", expand=True, padx=3)
self._activity_items = []

# Stats strip
stats_strip = ctk.CTkFrame(display_frame, fg_color="#0F172A", height=28, corner_radius=0)
stats_strip.pack(fill="x", padx=3, pady=(0, 3))
stats_strip.pack_propagate(False)
self._stat_trades_var = tk.StringVar(value="Trades: 0")
self._stat_queue_var = tk.StringVar(value="Queue: 0")
self._stat_push_var = tk.StringVar(value="Push: idle")
for var in [self._stat_trades_var, self._stat_queue_var, self._stat_push_var]:
    ctk.CTkLabel(stats_strip, textvariable=var, font=("Consolas", 9),
                 text_color="#475569").pack(side="left", padx=(12, 16), pady=4)

# Hidden log widget (still needed by self.log() method)
self.log_text = tk.Text(self._live_view, height=0)

# ■■■ VIEW 2: Controls (hidden, full-screen takeover) ■■■
self._controls_visible = False
self._controls_view = ctk.CTkFrame(body, fg_color="transparent")
# Don't pack yet – toggled on click

self.notebook = ctk.CTkTabview(self._controls_view, fg_color=self.C_BG,
                               segmented_button_fg_color=self.C_BG_SEC,
                               segmented_button_selected_color=self.C_ACCENT,
                               segmented_button_unselected_color=self.C_BG_THIRD,
                               text_color=self.C_TEXT, corner_radius=8)
self.notebook.pack(fill="both", expand=True)
tab_settings = self.notebook.add(" Settings ")

self._build_combined_settings_tab(tab_settings)

# ■■■ Bottom status bar ■■■
status_bar = ctk.CTkFrame(outer, fg_color=self.C_BG_SEC, height=24, corner_radius=0)
status_bar.pack(fill="x", side="bottom")
status_bar.pack_propagate(False)
self.status_var = tk.StringVar(value="Ready – enter your email to get started")
self.status_label = ctk.CTkLabel(status_bar, textvariable=self.status_var,
                                 font=("Segoe UI", 9), text_color=self.C_TEXT_DIM)
self.status_label.pack(side="left", padx=12, pady=2)

# State for smart auto-push
self.last_deal_ticket = 0
self.last_deal_count = 0
self.auto_push_thread = None

def _tick_live_clock(self):
    """Update the live display clock every second."""
    try:
        self._live_clock_var.set(datetime.now().strftime("%H:%M:%S"))
        self.root.after(1000, self._tick_live_clock)
    except Exception:
        pass

def _add_activity(self, text, kind="info"):
    """Add an entry to the Live Activity display panel.
    kind: 'info', 'trade', 'push', 'error', 'queue', 'success'
    """
    COLORS = {
        "info": "#64748B",

```

```

        "trade":    "#F59E0B",
        "push":    "#3B82F6",
        "error":   "#EF4444",
        "queue":   "#A78BFA",
        "success": "#22C55E",
    }
    ICONS = {
        "info":    "■",
        "trade":   "■",
        "push":    "■",
        "error":   "✖",
        "queue":   "■",
        "success": "✓",
    }
    if not CTK_AVAILABLE:
        return

    color = COLORS.get(kind, COLORS["info"])
    icon = ICONS.get(kind, ".")
    ts = datetime.now().strftime("%H:%M:%S")

    row = ctk.CTkFrame(self._activity_scroll, fg_color="transparent", height=22)
    row.pack(fill="x", padx=4, pady=1)
    row.pack_propagate(False)
    ctk.CTkLabel(row, text=f"{icon}", font=("Segoe UI", 10),
                 text_color=color, width=16).pack(side="left", padx=(4, 4))
    ctk.CTkLabel(row, text=ts, font=("Consolas", 9),
                 text_color="#334155").pack(side="left", padx=(0, 6))
    ctk.CTkLabel(row, text=text, font=("Segoe UI", 9),
                 text_color=color, anchor="w").pack(side="left", fill="x", expand=True)

    self._activity_items.append({"frame": row, "kind": kind})

    # Keep max 80 items
    while len(self._activity_items) > 80:
        old = self._activity_items.pop(0)
        try:
            old["frame"].destroy()
        except Exception:
            pass

    # Auto-scroll to bottom
    try:
        self._activity_scroll._parent_canvas.yview_moveto(1.0)
    except Exception:
        pass

def _toggle_controls_panel(self):
    """Swap between Live Display and Controls views (full-screen takeover)."""
    if self._controls_visible:
        # Hide controls, show live display
        self._controls_view.pack_forget()
        self._live_view.pack(fill="both", expand=True)
        self._controls_visible = False
        self._panel_btn.configure(text="■ Controls")
    else:
        # Hide live display, show controls
        self._live_view.pack_forget()
        self._controls_view.pack(fill="both", expand=True)
        self._controls_visible = True

```

```

        self._panel_btn.configure(text="■ Back to Live")

# ■■ Build Dashboard Tab ■■
def _build_combined_settings_tab(self, parent):
    """Build the combined Settings tab – Dashboard + Trading Engine in one page."""
    if CTK_AVAILABLE:
        scroll = ctk.CTkScrollableFrame(parent, fg_color="transparent")
        scroll.pack(fill="both", expand=True)
        parent = scroll
    self._build_dashboard_tab(parent)
    self._build_trading_engine_ui(parent)

def _build_dashboard_tab(self, parent):
    """Build the Dashboard section – compact layout."""

    # ■■ Connection Target ■■
    settings = parent
    conn_card = self._section_card(settings, "CONNECTION TARGET", "■")
    conn_card.pack(fill="x", padx=4, pady=(4, 2))

    conn_inner = ctk.CTkFrame(conn_card, fg_color="transparent") if CTK_AVAILABLE else \
        tk.Frame(conn_card, bg="#161B22")
    conn_inner.pack(fill="x", padx=10, pady=(2, 6))

    if CTK_AVAILABLE:
        ctk.CTkLabel(conn_inner, text="Target:", font=("Segoe UI", 11),
            text_color=self.C_TEXT_DIM).pack(side="left", padx=(0, 8))

    self.target_var = tk.StringVar()
    self.url_keys = ["TradeOpps (Production)", "Localhost (Development)"]
    self.url_values = {
        "TradeOpps (Production)": "https://www.tradeopss.com",
        "Localhost (Development)": "http://127.0.0.1:5001"
    }

    if CTK_AVAILABLE:
        self.url_selector = ctk.CTkComboBox(conn_inner, variable=self.target_var,
            values=self.url_keys, state="readonly",
            width=280, height=32,
            fg_color=self.C_BG_THIRD,
            border_color=self.C_BORDER,
            button_color=self.C_ACCENT,
            dropdown_fg_color=self.C_BG_SEC,
            dropdown_hover_color=self.C_BG_THIRD,
            text_color=self.C_TEXT,
            font=("Segoe UI", 11))
        self.url_selector.pack(side="left", padx=(0, 8))
        self.url_selector.set(self.url_keys[0])
    else:
        self.url_selector = ttk.Combobox(conn_inner, textvariable=self.target_var,
            state="readonly", width=32)
        self.url_selector['values'] = self.url_keys
        self.url_selector.pack(side="left", fill="x", expand=True)
        self.url_selector.current(0)

    # Hidden entry for backward-compatible URL storage
    self.url_entry = ttk.Entry(settings)
    self.url_entry.insert(0, self.url_values["TradeOpps (Production)"])

```

```

def on_target_change(event=None):
    selection = self.target_var.get()
    if "Localhost" in selection:
        password = simpledialog.askstring("Developer Access",
            "Enter password for local development:", show='*')
        if password == "tradeopss@123":
            self.url_entry.delete(0, tk.END)
            self.url_entry.insert(0, self.url_values[selection])
            self.log("Switched to Localhost")
            self.status_var.set("Target: Localhost (Dev)")
        else:
            messagebox.showerror("Access Denied", "Incorrect password.")
            if CTK_AVAILABLE:
                self.url_selector.set(self.url_keys[0])
            else:
                self.url_selector.current(0)
            self.url_entry.delete(0, tk.END)
            self.url_entry.insert(0, self.url_values["TradeOpps (Production)"])
    else:
        self.url_entry.delete(0, tk.END)
        self.url_entry.insert(0, self.url_values[selection])
        self.log("Switched to Production")
        self.status_var.set("Target: Production")

if CTK_AVAILABLE:
    self.url_selector.configure(command=lambda _: on_target_change())
else:
    self.url_selector.bind("<<ComboboxSelected>>", on_target_change)

# ■■ Client Identification (hidden entry for compat) ■■
self.client_email_entry = ctk.CTkEntry(settings, width=0, height=0) if CTK_AVAILABLE else tk.Entry(
    settings)
# Don't pack – hidden, used only as data holder

self.hierarchy_var = tk.StringVar(value="")
self.hierarchy_label = ctk.CTkLabel(settings, textvariable=self.hierarchy_var,
    font=("Segoe UI", 10, "italic"),
    text_color=self.C_TEXT_DIM) if CTK_AVAILABLE else \
    ttk.Label(settings, textvariable=self.hierarchy_var)
# Don't pack – hidden

# ■■ Import Data ■■
imp_card = self._section_card(settings, "IMPORT DATA", "■")
imp_card.pack(fill="x", padx=4, pady=2)

imp_inner = ctk.CTkFrame(imp_card, fg_color="transparent") if CTK_AVAILABLE else \
    tk.Frame(imp_card, bg="#161B22")
imp_inner.pack(fill="x", padx=12, pady=(4, 4))

self.import_source = tk.StringVar(value="sheet")
if CTK_AVAILABLE:
    ctk.CTkRadioButton(imp_inner, text="Google Sheets", variable=self.import_source,
        value="sheet", command=self._toggle_import_source,
        font=("Segoe UI", 11), text_color=self.C_TEXT,
        fg_color=self.C_ACCENT, border_color=self.C_BORDER).pack(side="left", padx=
            14))
    ctk.CTkRadioButton(imp_inner, text="CSV File", variable=self.import_source,
        value="csv", command=self._toggle_import_source,
        font=("Segoe UI", 11), text_color=self.C_TEXT,
        fg_color=self.C_ACCENT, border_color=self.C_BORDER).pack(side="left")

```

```

else:
    ttk.Radiobutton(imp_inner, text="Google Sheets", variable=self.import_source,
                    value="sheet", command=self._toggle_import_source).pack(side="left", padx=(0,
    ttk.Radiobutton(imp_inner, text="CSV File", variable=self.import_source,
                    value="csv", command=self._toggle_import_source).pack(side="left")

# Sheet URL row
self.sheet_input_frame = ctk.CTkFrame(imp_card, fg_color="transparent") if CTk_AVAILABLE else \
    tk.Frame(imp_card, bg="#161B22")
self.sheet_input_frame.pack(fill="x", padx=12, pady=4)
if CTk_AVAILABLE:
    ctk.CTkLabel(self.sheet_input_frame, text="URL:", font=("Segoe UI", 11),
                  text_color=self.C_TEXT_DIM).pack(side="left", padx=(0, 6))
self.sheet_url_entry = self._ctk_entry(self.sheet_input_frame, width=380,
                                       placeholder="Google Sheet URL...")
self.sheet_url_entry.pack(side="left", fill="x", expand=True)

# CSV row (hidden by default)
self.csv_input_frame = ctk.CTkFrame(imp_card, fg_color="transparent") if CTk_AVAILABLE else \
    tk.Frame(imp_card, bg="#161B22")
self.csv_path_var = tk.StringVar()
if CTk_AVAILABLE:
    ctk.CTkLabel(self.csv_input_frame, text="File:", font=("Segoe UI", 11),
                  text_color=self.C_TEXT_DIM).pack(side="left", padx=(0, 6))
    self.csv_path_entry = ctk.CTkEntry(self.csv_input_frame, textvariable=self.csv_path_var,
                                       width=280, height=32, state="disabled",
                                       fg_color=self.C_BG_THIRD, border_color=self.C_BORDER,
                                       text_color=self.C_TEXT)
else:
    self.csv_path_entry = ttk.Entry(self.csv_input_frame, textvariable=self.csv_path_var,
                                    width=36, state='readonly')
self.csv_path_entry.pack(side="left", padx=(0, 6))
self._ctk_button(self.csv_input_frame, text="Browse...", command=self._browse_csv,
                  fg=self.C_BG_THIRD, hover=self.C_BORDER, width=90).pack(side="left")

imp_btn_row = ctk.CTkFrame(imp_card, fg_color="transparent") if CTk_AVAILABLE else \
    tk.Frame(imp_card, bg="#161B22")
imp_btn_row.pack(fill="x", padx=12, pady=(4, 8))
self.import_btn = self._ctk_button(imp_btn_row, text="Import Sheet Data", command=self._do_import,
                                   fg="#6366F1", hover="#4F46E5", width=180)
self.import_btn.pack(side="left")
self.import_hint_text = "Sheet must be publicly shared"
if CTk_AVAILABLE:
    self.import_hint = ctk.CTkLabel(imp_btn_row, text=self.import_hint_text,
                                    font=("Segoe UI", 9, "italic"),
                                    text_color=self.C_TEXT_DIM)
else:
    self.import_hint = ttk.Label(imp_btn_row, text=self.import_hint_text)
self.import_hint.pack(side="left", padx=(12, 0))

def log(self, message, level="INFO"):
    """Add a message to the log and the live activity display."""
    timestamp = datetime.now().strftime("%H:%M:%S")
    self.log_text.insert(tk.END, f"[{timestamp}] {message}\n")
    self.log_text.see(tk.END)
    # Feed into live display
    kind = "info"
    msg_lower = message.lower()
    if level == "ERROR" or "■" in message or "failed" in msg_lower:
        kind = "error"

```

```

elif "■" in message or "success" in msg_lower:
    kind = "success"
elif "trade" in msg_lower or "buy" in msg_lower or "sell" in msg_lower or "■" in message:
    kind = "trade"
elif "push" in msg_lower or "■" in message or "syncd" in msg_lower:
    kind = "push"
elif "queue" in msg_lower or "scheduled" in msg_lower or "■" in message:
    kind = "queue"
try:
    self._add_activity(message, kind)
except Exception:
    pass
try:
    self.root.update_idletasks()
except Exception:
    pass

def lookup_client(self):
    """Lookup client hierarchy from email - NO API KEY REQUIRED."""
    email = self.client_email_entry.get().strip()
    dashboard_url = self.url_entry.get().strip().rstrip('/')

    if not email:
        messagebox.showerror("Error", "Please enter the client email")
        return

    self.log(f"Looking up client: {email}")
    self.hierarchy_var.set("Looking up...")
    self.root.update_idletasks()

def _do_lookup():
    try:
        # Use public endpoint - no API key needed
        response = requests.post(
            f"{dashboard_url}/api/client/auth",
            json={"email": email},
            headers={"Content-Type": "application/json"},
            timeout=30
        )

        if response.status_code == 200:
            data = response.json()
            if data.get("status") == "success":
                def _on_success(data=data):
                    self.client_info = data.get("identity", {})
                    client = self.client_info.get("client", "Unknown")
                    trader = self.client_info.get("trader", "Unknown")
                    admin = self.client_info.get("admin", "Unknown")
                    category = self.client_info.get("category", "Unknown")

                    self.hierarchy_var.set(
                        f"■ {client} → Trader: {trader} → Admin: {admin} | Category: {category}"
                    )
                if CTK_AVAILABLE:
                    self.hierarchy_label.configure(text_color='#16a34a')
                else:
                    self.hierarchy_label.configure(foreground='#16a34a')
                self.log(f"■ Client found: {client} → {trader} → {admin}")

            # Auto-fetch hedge accounts to populate MT5 credentials
            def _fetch_hedge_creds(cl=client, url=dashboard_url):

```

```

try:
    r2 = requests.get(
        f"{url}/api/data?client_id={cl}",
        timeout=15
    )
    if r2.status_code == 200:
        d2 = r2.json()
        hedge_accounts = d2.get('hedge_accounts') or []
        if hedge_accounts:
            ha = hedge_accounts[0]
            ha_login = str(ha.get('login', '')).strip()
            ha_pass = str(ha.get('password', '')).strip()
            ha_server = str(ha.get('server', '')).strip()
            def _fill_creds(l=ha_login, p=ha_pass, s=ha_server):
                if l:
                    self.mt5_login.delete(0, 'end')
                    self.mt5_login.insert(0, l)
                if p:
                    self.mt5_password.delete(0, 'end')
                    self.mt5_password.insert(0, p)
                if s:
                    self.mt5_server.delete(0, 'end')
                    self.mt5_server.insert(0, s)
                if l or s:
                    self.log(
                        f"■ MT5 credentials auto-filled from hedge account"
                    )
            self.root.after(0, _fill_creds)
        except Exception:
            pass
        threading.Thread(target=_fetch_hedge_creds, daemon=True).start()
        self.root.after(0, _on_success)
    else:
        error_msg = data.get("message", "Client not found")
        def _on_not_found(msg=error_msg):
            self.hierarchy_var.set(f"■ {msg}")
            if CTK_AVAILABLE:
                self.hierarchy_label.configure(text_color='#dc2626')
            else:
                self.hierarchy_label.configure(foreground='#dc2626')
            self.client_info = None
            self.log(f"■ Lookup failed: {msg}", "ERROR")
            self.root.after(0, _on_not_found)
        else:
            error_msg = f"API Error: {response.status_code}"
            try:
                error_data = response.json()
                error_msg = error_data.get("message", error_msg)
            except:
                pass
            def _on_error(msg=error_msg):
                self.hierarchy_var.set(f"■ {msg}")
                if CTK_AVAILABLE:
                    self.hierarchy_label.configure(text_color='#dc2626')
                else:
                    self.hierarchy_label.configure(foreground='#dc2626')
                self.client_info = None
                self.log(f"■ Lookup failed: {msg}", "ERROR")
                self.root.after(0, _on_error)

except requests.exceptions.Timeout:

```

```

def _on_timeout():
    self.hierarchy_var.set("■ Connection timeout")
    if CTK_AVAILABLE:
        self.hierarchy_label.configure(text_color='#dc2626')
    else:
        self.hierarchy_label.configure(foreground='#dc2626')
    self.log("■ Connection timeout", "ERROR")
    self.root.after(0, _on_timeout)
except requests.exceptions.ConnectionError:
    def _on_conn_err():
        self.hierarchy_var.set("■ Cannot connect to server")
        if CTK_AVAILABLE:
            self.hierarchy_label.configure(text_color='#dc2626')
        else:
            self.hierarchy_label.configure(foreground='#dc2626')
        self.log("■ Cannot connect to server", "ERROR")
        self.root.after(0, _on_conn_err)
except Exception as e:
    def _on_exc(err=str(e)):
        self.hierarchy_var.set(f"■ Error: {err}")
        if CTK_AVAILABLE:
            self.hierarchy_label.configure(text_color='#dc2626')
        else:
            self.hierarchy_label.configure(foreground='#dc2626')
        self.log(f"■ Error: {err}", "ERROR")
        self.root.after(0, _on_exc)

threading.Thread(target=_do_lookup, daemon=True).start()

def toggle_mt5_connection(self):
    """Connect or disconnect from MT5."""
    if self.pusher.connected:
        success, msg = self.pusher.disconnect_mt5()
        self.mt5_btn.configure(text="Connect to MT5")
        self.log(msg)
    else:
        login = self.mt5_login.get().strip()
        password = self.mt5_password.get()
        server = self.mt5_server.get().strip()

        success, msg = self.pusher.connect_mt5(login, password, server)
        if success:
            self.mt5_btn.configure(text="Disconnect MT5")
            self.log(msg, "INFO" if success else "ERROR")

def _push_billing_data(self):
    """Push only firm billing (actual fees & payouts) to the dashboard."""
    if RELEASE_DISABLE_PUSH_BILLING:
        self.log("■ Push Billing is disabled in this release", "WARN")
        return

    if not self._firm_billing_summary:
        self.log("■ No billing data collected yet – connect to prop firm dashboards first", "WARN")
        return

    dashboard_url = self.url_entry.get().strip().rstrip('/')
    email = self.client_email_entry.get().strip()

    if not self.client_info:
        messagebox.showerror("Error", "Please lookup the client first")

```



```

return

def _do_push():
    try:
        if self.push_billing_btn:
            self.root.after(0, lambda: self.push_billing_btn.configure(state="disabled"))
            self.log("■ Pushing billing data to dashboard...")

            # Match billing records to individual evaluations so
            # per-account Fee, Date Purchased, and Date Started get pushed
            import re as _re
            all_evals = [rd.get("eval") for rd in self._active_trade_rows if rd.get("eval")]
            filled_count = 0

            # Build a combined lookup from all firms' billing records
            billing_by_acct = {}
            for firm_name, summary in self._firm_billing_summary.items():
                for rec in summary.get("records", []):
                    acct_no = (rec.get("account_no") or "").strip()
                    amount = rec.get("amount", 0)
                    bill_date = rec.get("date", "")
                    if acct_no and amount > 0:
                        billing_by_acct[acct_no] = {
                            "amount": amount,
                            "date": bill_date,
                            "firm": firm_name,
                        }

            if all_evals and billing_by_acct:
                for ev in all_evals:
                    acct_challenge = (ev.get("Account #") or "").strip()
                    acct_funded = (ev.get("Account #.1") or "").strip()
                    existing_fee = (ev.get("Fee") or "").strip()
                    fee_filled = False
                    if existing_fee:
                        try:
                            fee_filled = float(existing_fee.replace("$", "").replace(",", "")) > 0
                        except ValueError:
                            fee_filled = existing_fee not in ("", "$0", "$0.00", "0")

                    matched = None
                    for acct_key in [acct_challenge, acct_funded]:
                        if not acct_key:
                            continue
                        if acct_key in billing_by_acct:
                            matched = billing_by_acct[acct_key]
                            break
                    for bill_acct, bill_info in billing_by_acct.items():
                        if bill_acct in acct_key or acct_key in bill_acct:
                            matched = bill_info
                            break
                    if matched:
                        break

                    if matched:
                        billing_fee = f"${matched['amount']:.2f}"
                        ev["Fee"] = billing_fee
                        if not (ev.get("Date Purchased") or "").strip() and matched["date"]:
                            ev["Date Purchased"] = matched["date"]
                        if not (ev.get("Date Started") or "").strip() and matched["date"]:

```

```

        ev["Date Started"] = matched["date"]
        filled_count += 1

    self.root.after(0, lambda c=filled_count:
        self.log(f"■ Matched billing to {c} evaluation(s)"))

    payload = {
        "email": email,
        "firm_billing": self._firm_billing_summary,
    }
    if all_evals and filled_count > 0:
        payload["evaluations"] = all_evals

    response = requests.post(
        f"{dashboard_url}/api/client/push",
        json=payload,
        headers={"Content-Type": "application/json"},
        timeout=30
    )

    if response.status_code == 200:
        data = response.json()
        if data.get("status") == "success":
            firms = list(self._firm_billing_summary.keys())
            total_fees = sum(f["total_fees"] for f in self._firm_billing_summary.values())
            total_payouts = sum(f["total_payouts"] for f in self._firm_billing_summary.values())
            self.root.after(0, lambda: self.log(
                f"■ Billing pushed – {len(firms)} firm(s): "
                f"Fees ${total_fees:,.2f} | Payouts ${total_payouts:,.2f}"))
        else:
            self.root.after(0, lambda: self.log(
                f"■ Billing push failed: {data.get('message', 'Unknown error')}", "ERROR"))
    else:
        self.root.after(0, lambda: self.log(
            f"■ Billing push HTTP {response.status_code}", "ERROR"))
    except Exception as e:
        self.root.after(0, lambda err=str(e): self.log(f"■ Billing push error: {err}", "ERROR"))
    finally:
        if self.push_billing_btn:
            self.root.after(0, lambda: self.push_billing_btn.configure(state="normal"))

    threading.Thread(target=_do_push, daemon=True).start()

def push_data(self):
    """Push data to dashboard - NO API KEY REQUIRED."""
    dashboard_url = self.url_entry.get().strip().rstrip('/')
    email = self.client_email_entry.get().strip()

    # Use looked-up hierarchy info
    if not self.client_info:
        messagebox.showerror("Error",
            "Please lookup the client first by entering email and clicking 'Lookup'")
        return

    client_name = self.client_info.get('client', '')

    if not client_name:
        messagebox.showerror("Error", "Client lookup failed - no client name found")
        return

    # Prevent overlapping push workers. If a push is already running,

```

```

# queue exactly one follow-up run so the latest state still gets sent.
with self._push_lock:
    if self._push_in_progress:
        if not self._push_pending:
            self._push_pending = True
            self.log("■ Push already running – queued one follow-up push")
        return
    self._push_in_progress = True

# Guard checks passed – disable the button and run everything in a background thread
# so the UI stays responsive during MT5 calls and the HTTP round-trip.
if hasattr(self, 'push_btn_live') and self.push_btn_live:
    try:
        self.push_btn_live.configure(state="disabled")
    except Exception:
        pass
self.log(f"■ Pushing {client_name}...")
self.status_var.set("Pushing data...")

def _do_push():
    should_run_follow_up = False
    try:
        self._push_data_worker(self.dashboard_url, self.email, self.client_name)
    finally:
        with self._push_lock:
            self._push_in_progress = False
            should_run_follow_up = self._push_pending
            self._push_pending = False

    if should_run_follow_up:
        try:
            self.root.after(0, lambda: self.log("■ Running queued push"))
            self.root.after(0, self.push_data)
        except Exception:
            pass
    elif hasattr(self, 'push_btn_live') and self.push_btn_live:
        try:
            self.root.after(0, lambda: self.push_btn_live.configure(state="normal"))
        except Exception:
            pass

threading.Thread(target=_do_push, daemon=True).start()

# Per-login cache for the 365-day farming history:
# { login_key: (fetched_at_epoch, [deals]) }
# TTL is 5 minutes – auto-push fires frequently so we avoid re-scanning MT5 history
# on every tick. Only the last-24h slice is always fetched fresh.
_FA_HISTORY_CACHE_TTL = 300 # seconds

# Tradovate MNQ farming history cache: { firm_name: (fetched_at_epoch, mnq_data) }
# Fetching fills + balance logs is slow; reuse within the same TTL window.
_TRADOVATE_FARMING_CACHE_TTL = 300 # seconds

def _push_data_worker(self, dashboard_url, email, client_name):
    """Heavy push work – runs on a background thread."""
    def _log(msg, level="INFO"):
        self.root.after(0, lambda m=msg, lv=level: self.log(m, lv))
    def _status(msg):
        self.root.after(0, lambda m=msg: self.status_var.set(m))

    account = self.pusher.get_account_info(include_balance_history=False)

```

```

if not account:
    _log("■■ MT5 account info returned empty – pushing with no account data", "ERROR")
    account = {}

# Log detailed MT5 account information
if account:
    login = account.get('login', 'N/A')
    company = account.get('company', 'Unknown')
    server = account.get('server', 'Unknown')
    balance = account.get('balance', 0)
    equity = account.get('equity', 0)
    deposits = account.get('total_deposits', 0)
    withdrawals = account.get('total_withdrawals', 0)
    _log(f"■■ MT5 ACCOUNT INFO:")
    _log(f"    Login: {login} | Company: {company} | Server: {server}")
    _log(f"    Balance: ${balance:,.2f} | Equity: ${equity:,.2f}")
    _log(f"    Deposits: ${deposits:,.2f} | Withdrawals: ${withdrawals:,.2f}")

positions = self.pusher.get_positions()
if positions is None:
    _log("■■ MT5 positions returned None – sending empty list")
    positions = []

# Always fetch the last 24 h fresh (fast – small result set).
_today_cutoff = time.time() - 86400
raw_deals = self.pusher.get_deals(days=1) or []

# For farming aggregation we need long-range history for accurate Hedge Day slots.
# Cache it per MT5 login with a 5-minute TTL so repeated auto-push calls are fast.
_fa_history_days = 365
_login_key = str(account.get('login', 'unknown'))
if not hasattr(self, '_fa_history_cache'):
    self._fa_history_cache = {}

_cached = self._fa_history_cache.get(_login_key)
_now = time.time()

if _cached and (_now - _cached[0]) < self._FA_HISTORY_CACHE_TTL:
    _log(f"■■ Using cached FA history ({int(_now - _cached[0]))s old)")
    _cached_deals = _cached[1]
else:
    _log(f"■■ Fetching {_fa_history_days}-day FA history (cache miss or expired)")
    _full = self.pusher.get_deals(days=_fa_history_days) or []
    # Store only the deals older than today's cutoff to keep cache lean.
    # Strip internal transfers up-front so they can NEVER resurface from
    # the cache – guarantees the latest live MT5 state always wins and
    # no stale balance op leaks into hedge aggregates on subsequent pushes.
    _cached_deals = []
    for _d in _full:
        if _d.get('time_raw', 0) >= _today_cutoff:
            continue
        _dt = str(_d.get('type', '')).upper()
        if _dt in ('BALANCE', 'CREDIT', '2', '3', 'CHARGE', 'CORRECTION', 'BONUS'):
            _cl = str(_d.get('comment', '') or '').strip().lower()
            if ('internal transfer' in _cl) or (not _cl):
                continue
            _cached_deals.append(_d)
    self._fa_history_cache[_login_key] = (_now, _cached_deals)

# Merge: fresh last-24h + cached history gives the full aggregation input.

```

```

_raw_deal_ids = {d.get('ticket') for d in raw_deals}
all_history_deals = list(raw_deals) + [d for d in _cached_deals if d.get(
    'ticket') not in _raw_deal_ids]

# Derive last-trading-day deals – already fetched above as raw_deals.
# Fall back to the most-recent day if there were no deals in the last 24 h.
if not raw_deals and all_history_deals:
    _max_ts = max(d.get('time_raw', 0) for d in all_history_deals)
    _day_start = _max_ts - 86400
    raw_deals = [d for d in all_history_deals if d.get('time_raw', 0) >= _day_start]

# Mark history-only FA deals so the filter can skip re-parsing them.
# Non-FA deals (CH, FD, DD) should only use last-trading-day data.
# FA deals need the full 90-day history to correctly compute hedge day slots.
_last_trading_day_ids = {d.get('ticket') for d in raw_deals}
aggregation_raw_deals = []
for _d in all_history_deals:
    c_up = str(_d.get('comment', '')).upper()
    is_fa = '_FA' in c_up
    is_history_only = _d.get('ticket') not in _raw_deal_ids
    d_type = str(_d.get('type', '')).upper()
    is_balance_op = d_type in ['BALANCE', 'CREDIT', '2', '3', 'CHARGE', 'CORRECTION', 'BONUS']
    _comment_lower = str(_d.get('comment', '') or '').strip().lower()
    is_internal_transfer = (
        'internal transfer' in _comment_lower
        or (is_balance_op and not _comment_lower)
    )

    if is_internal_transfer:
        # Internal transfers / unattributed balance ops never contribute to
        # hedge results; drop them so they cannot inflate eval values.
        continue

    if is_balance_op:
        # Keep other balance ops (deposits with comments, charges, bonuses, etc.)
        aggregation_raw_deals.append(_d)
    elif is_fa:
        # FA: only push last-trading-day deals (what to push).
        # Full history is used separately for day COUNT only (fa_day_keys_by_account below).
        if _d.get('ticket') in _last_trading_day_ids:
            aggregation_raw_deals.append(_d)
        else:
            # CH / FD / DD: only last trading day – avoids stale/replaced deals from old resets
            if _d.get('ticket') in _last_trading_day_ids:
                aggregation_raw_deals.append(_d)

_fa_count = sum(1 for d in aggregation_raw_deals if '_FA' in str(d.get('comment', '')).upper())
_other_count = len(aggregation_raw_deals) - _fa_count
_log(
    f"■ Deal scope: {_other_count} CH/FD/DD (last trading day) + {_fa_count} FA (last trading day)"

# Pre-compute full FA trading-day counts per account from full history,
# then use this count as the authoritative Hedge Day slot index.
fa_day_keys_by_account = {}
for _d in all_history_deals:
    _comment = str(_d.get('comment', '') or '')
    if '_FA' not in _comment.upper():
        continue

    _parsed = self.pusher.parse_deal_comment_v2(_comment)

```

```

        if not _parsed:
            continue
        # parse_deal_comment_v2 may return either a ParsedComment-like object
        # or a plain dict (parse_mt5_comment convenience path).
        _parsed_is_dict = isinstance(_parsed, dict)
        _is_valid = (_parsed.get('is_valid') if _parsed_is_dict else getattr(_parsed, 'is_valid', True))
        if _is_valid is False:
            continue

        _account_key = str(
            (_parsed.get('account_number') if _parsed_is_dict else getattr(_parsed, 'account_number', '')) or ''
        ).strip()
        if not _account_key:
            continue

        _day_key = ''
        _parsed_date = (_parsed.get('farming_date') if _parsed_is_dict else getattr(_parsed, 'farming_date', None))
        if _parsed_date:
            try:
                if hasattr(_parsed_date, 'strftime'):
                    _day_key = _parsed_date.strftime('%Y-%m-%d')
                else:
                    _day_key = str(_parsed_date)[:10]
            except Exception:
                _day_key = str(_parsed_date)[:10]

        if not _day_key:
            _ts = _d.get('time_raw', 0)
            if isinstance(_ts, (int, float)) and _ts > 0:
                try:
                    _day_key = datetime.fromtimestamp(_ts).strftime('%Y-%m-%d')
                except Exception:
                    _day_key = ''

        if _day_key:
            fa_day_keys_by_account.setdefault(_account_key, set()).add(_day_key)

fa_day_count_by_account = {
    acc: len(days)
    for acc, days in fa_day_keys_by_account.items()
    if days
}

# FNFT challenge filter: challenge accounts reuse the same account numbers
# across resets, so only keep last 24h of deals with _CH in the comment.
# Funded (_FU, _FD, _DD, _FA) deals keep the full date range.
is_fnft = False
try:
    srv = str(self.pusher.server or '').upper()
    cmp = str(getattr(self.pusher, 'company', '') or '').upper()
    if 'FUNDEDNEXT' in srv or 'FNFT' in srv or 'FUNDEDNEXT' in cmp or 'FNFT' in cmp:
        is_fnft = True
except Exception:
    pass

if is_fnft:
    _24h_ago = time.time() - 86400
    if raw_deals:

```

```

before_count = len(raw_deals)
raw_deals = [
    d for d in raw_deals
    if '_CH' not in str(d.get('comment', '')).upper()
    or d.get('time_raw', 0) >= _24h_ago
]
dropped = before_count - len(raw_deals)
if dropped:
    _log(f"■ Filtered {dropped} old FNFT challenge deal(s) (>24h)")
if aggregation_raw_deals:
    aggregation_raw_deals = [
        d for d in aggregation_raw_deals
        if '_CH' not in str(d.get('comment', '')).upper()
        or d.get('time_raw', 0) >= _24h_ago
    ]

# Filter deals: keep balance ops and trades with valid comments.
# FA-history-only deals are pre-validated (added because comment has _FA) - skip re-parse.
def _filter_valid_push_deals(source_deals, skip_fa_reparse=False):
    filtered = []
    for deal in source_deals or []:
        d_type = str(deal.get('type', '')).upper()
        if d_type in ['BALANCE', 'CREDIT', '2', '3', 'CHARGE', 'CORRECTION', 'BONUS']:
            # Drop internal transfers – they are NOT real P&L and must
            # never reach statistics or the dashboard. Catches both:
            # - explicit "internal transfer" comment (any sign)
            # - empty-comment balance ops (legacy untagged transfers)
            _bal_comment_l = str(deal.get('comment', '') or '').strip().lower()
            if ('internal transfer' in _bal_comment_l or (not _bal_comment_l)):
                continue
            filtered.append(deal)
            continue

        if skip_fa_reparse and deal.get('_fa_history_only'):
            filtered.append(deal)
            continue

    # Also skip re-parsing for any FA-tagged deal when skip_fa_reparse=True.
    # All FA deals were pre-validated by the _FA comment check in the build loop.
    if skip_fa_reparse and '_FA' in str(deal.get('comment', '')).upper():
        filtered.append(deal)
        continue

    comment = deal.get('comment', '')
    parsed = self.pusher.parse_deal_comment_v2(comment)

    is_valid = False
    if parsed:
        if hasattr(parsed, 'is_valid'):
            is_valid = parsed.is_valid
        else:
            is_valid = True

    if is_valid:
        filtered.append(deal)
    return filtered
deals = _filter_valid_push_deals(raw_deals)
aggregation_deals = _filter_valid_push_deals(aggregation_raw_deals, skip_fa_reparse=True)

if len(deals) < len(raw_deals):

```

```

        _log(f"Filtered {len(raw_deals) - len(deals)} deals with invalid comments")

statistics = self.pusher.calculate_statistics(deals)

# Aggregate hedge results locally, including farming history for correct FA slotting.
aggregated_by_comment = []
comment_summary = {}
latest_fa_aggregates = []
if COMMENT_PARSER_AVAILABLE and aggregation_deals:
    aggregated_by_comment, _unmatched, _agg_log = aggregate_deals_by_position(aggregation_deals)

    # Keep open-position aggregates strictly on the current trading day.
    # This prevents stale multi-day open entries from re-triggering FA pushes.
    _today_local = datetime.now().date()

    def _is_current_trading_day_open(agg):
        ts = agg.get('timestamp')
        if isinstance(ts, (int, float)) and ts > 0:
            try:
                return datetime.fromtimestamp(ts).date() == _today_local
            except Exception:
                pass

        farming_date = str(agg.get('farming_date') or '').strip()
        if len(farming_date) >= 10:
            try:
                return datetime.fromisoformat(farming_date.replace('Z', '+00:00')).date(
                ) == _today_local
            except Exception:
                try:
                    return datetime.strptime(farming_date[:10], '%Y-%m-%d').date() == _today_local
                except Exception:
                    pass
        return False

    if aggregated_by_comment:
        _fresh_open_aggregates = []
        _dropped_stale_open = 0
        for agg in aggregated_by_comment:
            if bool(agg.get('has_open_position')) and not _is_current_trading_day_open(agg):
                _dropped_stale_open += 1
                continue
            _fresh_open_aggregates.append(agg)

        if _dropped_stale_open:
            _log(
                f"■ Suppressed {_dropped_stale_open} stale open trade group(s) from prior trading"
            )
        aggregated_by_comment = _fresh_open_aggregates

    def _normalize_fa_day(agg):
        timestamp = agg.get('timestamp')
        if isinstance(timestamp, (int, float)) and timestamp > 0:
            try:
                return datetime.fromtimestamp(timestamp).strftime('%Y-%m-%d')
            except Exception:
                pass
        farming_date = str(agg.get('farming_date') or '').strip()
        if len(farming_date) >= 10:
            return farming_date[:10]
        return farming_date

```



```

fa_by_account_day = {}
non_fa_aggregates = []
for agg in aggregated_by_comment:
    if agg.get('phase_code') != 'FA':
        non_fa_aggregates.append(agg)
        continue

    day_key = _normalize_fa_day(agg)
    account_key = str(agg.get('account_number') or '')
    merge_key = (account_key, day_key)
    existing = fa_by_account_day.get(merge_key)
    if not existing:
        fa_by_account_day[merge_key] = dict(agg)
        continue

    existing['net_profit'] = round(existing.get('net_profit', 0) + agg.get('net_profit', 0), 0),
    existing['deal_count'] = existing.get('deal_count', 0) + agg.get('deal_count', 0)
    if agg.get('timestamp', 0) > existing.get('timestamp', 0):
        existing['timestamp'] = agg.get('timestamp', 0)
        existing['farming_date'] = agg.get('farming_date')

latest_fa_aggregates = []
fa_accounts = {}
for (account_key, day_key), agg in fa_by_account_day.items():
    fa_accounts.setdefault(account_key, []).append((day_key, agg))

for account_key, entries in fa_accounts.items():
    entries.sort(key=lambda item: item[0])
    total_slots = fa_day_count_by_account.get(account_key, len(entries))
    # Push only the LATEST farming day, labelled as Hedge Day <total_slots>.
    # total_slots is derived from the full 90-day history so the slot number
    # is always correct even though we only send one entry per account.
    latest_day_key, latest_agg = entries[-1]
    tagged = dict(latest_agg)
    tagged['trade_number'] = total_slots
    tagged['_fa_slot'] = total_slots
    tagged['field_name'] = f"Hedge Day {total_slots}"
    latest_fa_aggregates.append(tagged)
    _log(f"■ {account_key}: {total_slots} FA day(s) → pushing as Hedge Day {total_slots}")

aggregated_by_comment = non_fa_aggregates + latest_fa_aggregates

# Guard against false FA zero pushes when there are no active positions.
# We still keep zero FA rows when positions are active (user-visible signal that a trade is r
has_active_positions = bool(positions)
if not has_active_positions and aggregated_by_comment:
    filtered_aggregates = []
    dropped_fa_zeros = 0
    for agg in aggregated_by_comment:
        is_fa = agg.get('phase_code') == 'FA'
        net_profit = float(agg.get('net_profit', 0) or 0)
        deal_count = int(agg.get('deal_count', 0) or 0)
        has_open_position = bool(agg.get('has_open_position'))
        if is_fa and abs(net_profit) < 1e-9 and deal_count <= 1 and not has_open_position:
            dropped_fa_zeros += 1
            continue
        filtered_aggregates.append(agg)
    if dropped_fa_zeros:
        _log(f"■ Suppressed {dropped_fa_zeros} stale FA zero row(s) (no open FA trade)")
    aggregated_by_comment = filtered_aggregates

```

```

by_phase = {}
for agg in aggregated_by_comment:
    phase_name = agg.get('phase_name', 'UNKNOWN')
    if phase_name not in by_phase:
        by_phase[phase_name] = {'count': 0, 'total_net_profit': 0.0}
    by_phase[phase_name]['count'] += 1
    by_phase[phase_name]['total_net_profit'] += agg.get('net_profit', 0)
comment_summary = {'by_phase': by_phase}

# Collect Tradovate MNQ daily P&L for Prop Day values.
# ONLY runs if at least one Tradovate account is actively connected.
# Cache-miss fetches are done in a background thread so they never add
# to push wall-time – the result is available for the NEXT push.
tradovate_farming_days = []
if not hasattr(self, '_tradovate_farming_cache'):
    self._tradovate_farming_cache = {}
_tv_now = time.time()

# Find all connected Tradovate accounts (account object must be present).
_connected_tv = [
    (firm_name, conn.get("account"))
    for firm_name, conn in self._broker_connections.items()
    if conn.get("account") and hasattr(conn.get("account"), 'get_mnq_daily_pnl')
]

if not _connected_tv:
    if latest_fa_aggregates:
        _log("■ Tradovate not connected – Prop Day values will not be updated", "WARN")
    else:
        for firm_name, tv_account in _connected_tv:
            _tv_cached = self._tradovate_farming_cache.get(firm_name)
            if _tv_cached and (_tv_now - _tv_cached[0]) < self._TRADOVATE_FARMING_CACHE_TTL:
                # Cache hit – instant, no API call.
                mnq_data = _tv_cached[1]
                if mnq_data:
                    total_days = sum(len(a.get('mnq_daily_pnl', [])) for a in mnq_data)
                    _log(f"■ {firm_name}: {total_days} Prop Day(s) ready (cached)")
                    tradovate_farming_days.extend(mnq_data)
                else:
                    # Cache miss – use whatever we have now (empty on first push),
                    # and refresh asynchronously so the NEXT push benefits.
                    existing = (_tv_cached[1] if _tv_cached else []) or []
                    if existing:
                        total_days = sum(len(a.get('mnq_daily_pnl', [])) for a in existing)
                        _log(
                            f"■ {firm_name}: {total_days} Prop Day(s) from stale cache (refreshing in ba
                            tradovate_farming_days.extend(existing)
                    else:
                        _log(f"■ {firm_name}: fetching Tradovate history in background (available next p

def _refresh_tv_cache(fn=firm_name, acc=tv_account):
    try:
        data = acc.get_mnq_daily_pnl() or []
        self._tradovate_farming_cache[fn] = (time.time(), data)
    except Exception as _e:
        pass # silently swallow; no-op on next push miss
    threading.Thread(target=_refresh_tv_cache, daemon=True).start()

# Cross-check: log Prop Day coverage vs Hedge Day coverage.
if latest_fa_aggregates and tradovate_farming_days:

```

```

for fa_agg in latest_fa_aggregates:
    acc_key = str(fa_agg.get('account_number') or '')
    hedge_slot = fa_agg.get('_fa_slot', '?')
    acc_digits = [d for d in re.findall(r'\d+', acc_key.upper()) if len(d) >= 4]
    for tv_data in tradovate_farming_days:
        tv_name = (tv_data.get('account_name') or '').upper()
        if acc_key.upper() in tv_name or any(d in tv_name for d in acc_digits):
            prop_count = len(tv_data.get('mnq_daily_pnl', []))
            _log(f"■ {acc_key}: Hedge Day {hedge_slot} ↔ {prop_count} Prop Day(s)")
            break

# Pre-push diagnostic: log what we're about to send
pos_count = len(positions) if positions else 0
deal_count = len(deals) if deals else 0
agg_count = len(aggregated_by_comment) if aggregated_by_comment else 0
bal = account.get('balance', 0)
_log(f"■ Payload: Bal=${bal:,.0f} | {deal_count} deals | {pos_count} pos | {agg_count} hedge gro

# Detailed per-row logging of what's being pushed
if aggregated_by_comment:
    _log(f"\n■ INDIVIDUAL ROWS BEING PUSHED:")
    _log(f"{'='*80}")

# Group by account for summary
by_account = {}
by_phase = {}

for i, agg in enumerate(aggregated_by_comment):
    account_num = agg.get('account_number', 'Unknown')
    phase_code = agg.get('phase_code', 'UNKNOWN')
    phase_name = agg.get('phase_name', 'Unknown')
    trade_num = agg.get('trade_number', 0)
    net_profit = agg.get('net_profit', 0)
    deal_cnt = agg.get('deal_count', 1)
    field_name = agg.get('field_name', '?')

    # Track per account
    if account_num not in by_account:
        by_account[account_num] = {'rows': [], 'total_profit': 0, 'total_deals': 0}
    by_account[account_num]['rows'].append({
        'phase': f"{phase_code}{trade_num}",
        'profit': net_profit,
        'deals': deal_cnt,
        'field': field_name
    })
    by_account[account_num]['total_profit'] += net_profit
    by_account[account_num]['total_deals'] += deal_cnt

    # Track per phase
    if phase_code not in by_phase:
        by_phase[phase_code] = {'count': 0, 'total_profit': 0}
    by_phase[phase_code]['count'] += 1
    by_phase[phase_code]['total_profit'] += net_profit

# Log individual row
farming_date = agg.get('farming_date', '')
date_str = f"({farming_date})" if farming_date else ""
_log(
    f"    [{i+1:2d}] {account_num}_{phase_code}{trade_num} → {field_name:20s} = ${net_pro

_log(f"{'='*80}")

```

```

_log(f"\n■ SUMMARY BY ACCOUNT:")
for account_num in sorted(by_account.keys()):
    data = by_account[account_num]
    row_count = len(data['rows'])
    total_p = data['total_profit']
    total_d = data['total_deals']
    phases = ', '.join(r['phase'] for r in data['rows'])
    _log(
        f"    {account_num:20s} | {row_count:2d} row(s) | Phases: {phases:30s} | Total: ${total_p:10.2f} | {total_d:10.2f} deals"

_log(f"\n■ SUMMARY BY PHASE:")
for phase_code in sorted(by_phase.keys()):
    data = by_phase[phase_code]
    phase_names = {'CH': 'Challenge', 'FD': 'Funded', 'DD': 'DoubleDip', 'FA': 'Farming'}
    phase_name = phase_names.get(phase_code, phase_code)
    _log(
        f"    {phase_name:15s} ({phase_code}) | {data['count']:2d} group(s) | Total: ${data['total_profit']:10.2f} | {data['total_deals']:10.2f} deals"

payload = {
    "email": email,
    "account": account,
    "positions": positions,
    "deals": deals,
    "statistics": statistics,
    "evaluations": [],
    "aggregated_by_comment": aggregated_by_comment,
    "prefer_client_aggregation": True,
    "comment_summary": comment_summary,
    "dropdown_options": {},
    "firm_billing": self._firm_billing_summary if self._firm_billing_summary else None,
}

# Only include Tradovate prop day data if we actually have it.
# Omitting the key entirely means the server will never touch existing Prop Day values.
if tradovate_farming_days:
    payload["tradovate_farming_days"] = tradovate_farming_days

try:
    # Use public endpoint - no API key needed
    response = _gzip_post(
        f"{dashboard_url}/api/client/push",
        payload,
        timeout=120
    )

    if response.status_code == 200:
        try:
            data = response.json()
        except ValueError:
            _log("■ Server returned invalid JSON (not JSON response)", "ERROR")
            _status("Push failed - bad response")
            return
        if data.get("status") == "success":
            bal = account.get('balance', 0)
            dep = account.get('total_deposits', 0)
            hedge_log = data.get("hedge_match_log", [])
            hedge_updates = data.get("hedge_updates", 0)
            _log(
                f"\n■ PUSH SUCCESSFUL → Bal: ${bal:,.0f} | Dep: ${dep:,.0f} | {len(deals)} deals"

            # Log detailed response from server showing what was processed

```

```

        _log(f"\n■ SERVER PROCESSING LOG ({len(hedge_log)} entries):")
        _log(f"{' '*80}")
        for i, entry in enumerate(hedge_log, 1):
            # Highlight different types of entries
            if entry.startswith("■"):
                _log(f"    {entry}", "INFO")
            elif entry.startswith("■■") or entry.startswith("■"):
                _log(f"    {entry}", "WARN")
            elif entry.startswith("■") or entry.startswith("■") or entry.startswith("■"):
                _log(f"    {entry}", "INFO")
            else:
                _log(f"    {entry}", "INFO")
        _log(f"{' '*80}")

        if hedge_updates:
            _log(f"\n■ CONFIRMED: {hedge_updates} hedge cell(s) updated and saved on dashboa
        elif agg_count:
            _log(
                "\n■■ Server acknowledged push but returned 0 hedge updates. Check account-n
        else:
            _log("\n■ No hedge aggregates in this push (MT5 data only)")

        _status("Ready - Data pushed!")
        try:
            self.root.after(0, lambda hu=hedge_updates: self._stat_push_var.set(f"Push: ✓ {h
        except Exception:
            pass
        else:
            _log(f"■ {data.get('message', 'Push failed')}", "ERROR")
            _status("Push failed")
    else:
        error_msg = f"HTTP {response.status_code}"
        try:
            error_msg = response.json().get("message", error_msg)
        except Exception:
            pass
        _log(f"■ Push failed: {error_msg}", "ERROR")
        _status("Push failed")

except requests.exceptions.Timeout:
    _log("■ Push timeout – server did not respond within 120s", "ERROR")
    _status("Push failed - timeout")
except requests.exceptions.ConnectionError:
    _log("■ Push failed – cannot connect to server", "ERROR")
    _status("Push failed - no connection")
except Exception as e:
    _log(f"■ Push error: {e}", "ERROR")
    _status("Push failed")

# Run hedging review in the background – uses account data already in hand, no second MT5 call.
try:
    threading.Thread(
        target=self._push_hedging_review_worker,
        args=(dashboard_url, email, account),
        daemon=True
    ).start()
except Exception as hr_err:
    _log(f"■■ Hedging review thread failed to start: {hr_err}", "ERROR")

def push_hedging_review(self):

```

```

"""Push hedging review data – called from toolbar button (fetches own account info)."""
dashboard_url = self.url_entry.get().strip().rstrip('/')
email = self.client_email_entry.get().strip()

if not self.client_info:
    messagebox.showerror("Error",
        "Please lookup the client first by entering email and clicking 'Lookup'")
    return

if not self.pusher.connected:
    messagebox.showerror("Error", "Please connect to MT5 first")
    return

self.status_var.set("Pushing hedging review...")
account = self.pusher.get_account_info(include_balance_history=True)
if not account:
    self.log("■ Hedging review skipped – MT5 account info is empty", "ERROR")
    self.status_var.set("Hedging review skipped")
    return

threading.Thread(
    target=self._push_hedging_review_worker,
    args=(dashboard_url, email, account),
    daemon=True
).start()

def _push_hedging_review_worker(self, dashboard_url, email, account):
    """Send hedging review payload – refreshes account totals in the background if needed."""
    def _log(msg, level="INFO"):
        self.root.after(0, lambda m=msg, lv=level: self.log(m, lv))
    def _status(msg):
        self.root.after(0, lambda m=msg: self.status_var.set(m))

    if self.pusher.connected:
        try:
            account = self.pusher.get_account_info(include_balance_history=True) or account
        except Exception:
            pass

    deposits = float(account.get('total_deposits', 0) or 0)
    withdrawals = float(account.get('total_withdrawals', 0) or 0)
    balance = float(account.get('balance', 0) or 0)

    _log(f"■ HR payload: Dep=${deposits:,.0f} | Wth=${withdrawals:,.0f} | Bal=${balance:,.0f}")

    payload = {
        "email": email,
        "total_deposits": deposits,
        "total_withdrawals": withdrawals,
        "current_balance": balance
    }

    try:
        response = requests.post(
            f"{dashboard_url}/api/client/push_hedging_review",
            json=payload,
            headers={"Content-Type": "application/json"},
            timeout=30
        )

        if response.status_code == 200:

```

```

        try:
            data = response.json()
        except ValueError:
            _log("■ Hedging review: server returned invalid JSON", "ERROR")
            _status("HR failed - bad response")
            return
        if data.get("status") == "success":
            hr = data.get("hedging_review") or {}
            actual = hr.get('actual_hedging_results', 0)
            disc = hr.get('discrepancy', 0)
            _log(
                f"■ Hedging Review → Dep: ${deposits:,.0f} | Wth: ${withdrawals:,.0f} | Actual:
            _status("Ready - Hedging review pushed!")
        else:
            _log(f"■ {data.get('message', 'Push failed')}", "ERROR")
            _status("Push failed")
    else:
        error_msg = f"HTTP {response.status_code}"
        try:
            error_msg = response.json().get("message", error_msg)
        except Exception:
            pass
        _log(f"■ Push failed: {error_msg}", "ERROR")
        _status("Push failed")

except requests.exceptions.Timeout:
    _log("■ Hedging review timeout", "ERROR")
    _status("HR timeout")
except requests.exceptions.ConnectionError:
    _log("■ Hedging review – cannot connect", "ERROR")
    _status("HR connection failed")
except Exception as e:
    _log(f"■ Hedging review error: {e}", "ERROR")
    _status("Push failed")

def push_mt5_only(self):
    """Push ONLY MT5 data (deals, positions, account) to recalculate hedging review."""
    dashboard_url = self.url_entry.get().strip().rstrip('/')
    email = self.client_email_entry.get().strip()

    if not self.client_info:
        messagebox.showerror("Error",
            "Please lookup the client first by entering email and clicking 'Lookup'")
        return

    if not self.pusher.connected:
        messagebox.showerror("Error", "Please connect to MT5 first to push MT5 data")
        return

    client_name = self.client_info.get('client', '')

    self.log(f"■ MT5 rebalance push for {client_name}...")
    self.status_var.set("Pushing MT5 data...")

    # Get MT5 data
    account = self.pusher.get_account_info() or {}
    deals = self.pusher.get_deals(days=365) # Get 1 year of deals for hedging calculations

    if not account:
        self.log("■ No account info available", "ERROR")

```

```

        messagebox.showerror("Error", "Could not retrieve account information from MT5.")
        return

# Extract rebalance data directly from account info (MT5 provides cumulative totals)
balance = account.get('balance', 0)
deposits = account.get('total_deposits', 0)
withdrawals = account.get('total_withdrawals', 0)

# Calculate actual hedging results from deals (non-BALANCE trades)
actual_hedging = 0.0
trade_count = 0
for deal in (deals or []):
    d_type = str(deal.get('type', '')).upper()
    if d_type not in ['BALANCE', 'CREDIT', '2', '3']: # Skip balance and credit operations
        profit = deal.get('profit', 0) or 0
        swap = deal.get('swap', 0) or 0
        commission = deal.get('commission', 0) or 0
        actual_hedging += (profit + swap + commission)
        if deal.get('entry') == 'OUT': # Count closed trades
            trade_count += 1

self.log(
    f"■ Bal: ${balance:,.0f} | Dep: ${deposits:,.0f} | Wth: ${withdrawals:,.0f} | Hedge: ${actual_hedging:,.0f}"
)

payload = {
    "email": email,
    "account": account,
    "positions": [],
    "deals": deals or [], # Include deals for actual hedging calculation
    "statistics": {}, # Let server recalculate with MT5 data
    # NOTE: Do NOT include "evaluations" key - server will preserve existing data
    "dropdown_options": {}
}

try:
    response = _gzip_post(
        f"{dashboard_url}/api/client/push",
        payload,
        timeout=120
    )

    if response.status_code == 200:
        data = response.json()

        if data.get("status") == "success":
            self.log(f"■ Rebalance pushed – Bal: ${balance:,.0f} | Dep: ${deposits:,.0f}")
            self.status_var.set("Rebalance data pushed!")
        else:
            self.log(f"■ Push failed: {data.get('message', 'Unknown error')}", "ERROR")
            self.status_var.set("Push failed")
    else:
        error_msg = f"HTTP {response.status_code}"
        try:
            error_msg = response.json().get("message", error_msg)
        except:
            pass
        self.log(f"■ Rebalance push failed: {error_msg}", "ERROR")
        self.status_var.set("Push failed")

except Exception as e:

```



```

        self.log(f"■ Push error: {e}", "ERROR")
        self.status_var.set("Push failed")

def show_deal_comments(self):
    """Debug function to show all deal comments from MT5."""
    if not self.pusher.connected:
        messagebox.showerror("Error", "Please connect to MT5 first")
        return

    self.log("=*60)
    self.log("■ MT5 DEAL COMMENTS DEBUG")
    self.log("=*60)

    deals = self.pusher.get_deals(days=365)

    if not deals:
        self.log("No deals found")
        return

    self.log(f"Total deals: {len(deals)}\n")

    # Group by unique comments
    comment_counts = {}
    for deal in deals:
        comment = deal.get('comment', '') or '(empty)'
        d_type = deal.get('type', '')
        if d_type not in ['BALANCE', 'CREDIT', '2', '3']: # Skip balance ops
            if comment not in comment_counts:
                comment_counts[comment] = {'count': 0, 'total_profit': 0, 'sample_deal': deal}
            comment_counts[comment]['count'] += 1
            comment_counts[comment]['total_profit'] += (deal.get('profit', 0) or 0)

    self.log(f"Unique comments: {len(comment_counts)}\n")
    self.log("-=*60)

    for comment, info in sorted(comment_counts.items()):
        parsed = self.pusher.parse_deal_comment(comment)

        self.log(f"\n■ Comment: '{comment}'")
        self.log(f"    Deals: {info['count']}, Total P/L: ${info['total_profit']:.2f}")

        if parsed:
            self.log(f"    ✓ Parsed -> Account: {parsed['account_suffix']}")
            if parsed.get('stage'):
                self.log(f"    ✓ Stage: {parsed['stage']}{parsed['stage_num']}")
            else:
                self.log(f"    ■■ No stage pattern found (CH/FU/FA)")
        else:
            self.log(f"    ■ Could not parse comment")

    self.log("\n" + "=*60)
    self.log("■ Comment format expected:")
    self.log("    Challenge: ..._CH{n} or ...CH{n}")
    self.log("    Funded:    ..._FU{n} or ...FU{n}")
    self.log("    Farming:   ..._FA{n}_DD/MM or ...FA{n}")
    self.log("=*60)

def sync_hedge_results(self):
    """
    Sync hedge results from MT5 deal comments to evaluation records.

```

```

Parses deal comments to extract account number and stage:
- Challenge: {account}_CH{n}
- Funded: {account}_FU{n}
- Farming: {account}_FA{n}_{DD/MM}

Then updates the appropriate Hedge Result fields in evaluations.
"""
dashboard_url = self.url_entry.get().strip().rstrip('/')
email = self.client_email_entry.get().strip()

if not self.client_info:
    messagebox.showerror("Error",
        "Please lookup the client first by entering email and clicking 'Lookup'")
    return

if not self.pusher.connected:
    messagebox.showerror("Error", "Please connect to MT5 first")
    return

client_name = self.client_info.get('client', '')

self.log("=*60)
self.log("■ SYNC HEDGE RESULTS FROM MT5 COMMENTS")
self.log("=*60)
self.status_var.set("Syncing hedge results...")

# Step 1: Get current evaluations from dashboard
self.log("\n■ Step 1: Fetching current evaluations from dashboard...")
try:
    response = requests.get(
        f"{dashboard_url}/api/data?client_id={client_name}",
        cookies=self.session_cookies if hasattr(self, 'session_cookies') else {},
        timeout=60
    )

    if response.status_code != 200:
        self.log(f"■ Failed to fetch data: HTTP {response.status_code}", "ERROR")
        messagebox.showerror("Error",
            "Could not fetch current data from dashboard. Try logging in via browser first.")
        return

    data = response.json()
    evaluations = data.get('evaluations', [])

    if not evaluations:
        self.log("■■ No evaluations found in dashboard", "WARNING")
        messagebox.showwarning("Warning",
            "No evaluations found. Please import from Google Sheets first.")
        return

    self.log(f" ✓ Found {len(evaluations)} evaluation records")

except Exception as e:
    self.log(f"■ Error fetching data: {e}", "ERROR")
    messagebox.showerror("Error", f"Failed to fetch data: {e}")
    return

# Step 2: Get deals from MT5
self.log("\n■ Step 2: Fetching deals from MT5...")
deals = self.pusher.get_deals(days=365) # Get 1 year of deals

```

```

if not deals:
    self.log("■■■ No deals found in MT5", "WARNING")
    messagebox.showwarning("Warning", "No deals found in MT5 history.")
    return

self.log(f"    ✓ Found {len(deals)} deals")

# Show sample comments for debugging
comments_with_data = [d.get('comment', '') for d in deals if d.get('comment')]
unique_comments = list(set(comments_with_data))[:10]
self.log(f"\n■ Sample deal comments found:")
for c in unique_comments:
    parsed = self.pusher.parse_deal_comment(c)
    if parsed and parsed.get('stage'):
        self.log(
            f"    ✓ '{c}' -> {parsed['stage']}{parsed['stage_num']}, account: {parsed['account_suffi"
        elif parsed and parsed.get('account_suffix'):
            self.log(f"    . '{c}' -> account: {parsed['account_suffix']} (no stage found)")
        else:
            self.log(f"    . '{c}' (not matching pattern)")

# Step 3: Process deals and update evaluations
self.log("\n■ Step 3: Processing deals and matching to evaluations...")
updated_evals, match_log = self.pusher.process_deals_for_evaluations(deals, evaluations)

for log_line in match_log:
    self.log(f"    {log_line}")

# Step 4: Push updated evaluations back to dashboard
self.log("\n■ Step 4: Pushing updated evaluations to dashboard...")

payload = {
    "email": email,
    "evaluations": updated_evals,
    "statistics": {}, # Let server recalculate
    "dropdown_options": {}
}

try:
    response = _gzip_post(
        f"{dashboard_url}/api/client/push",
        payload,
        timeout=120
    )

    if response.status_code == 200:
        data = response.json()
        if data.get("status") == "success":
            self.log(f"\n■ HEDGE RESULTS SYNCED SUCCESSFULLY!")
            self.log(f"    Updated {len(updated_evals)} evaluation records")
            self.log(f"="*60)
            self.status_var.set("Hedge results synced!")
            messagebox.showinfo("Success", "Hedge results synced from MT5 comments!")
        else:
            self.log(f"■ Sync failed: {data.get('message', 'Unknown error')}", "ERROR")
            self.status_var.set("Sync failed")
    else:
        self.log(f"■ HTTP {response.status_code}", "ERROR")
        self.status_var.set("Sync failed")

except Exception as e:

```

```

        self.log(f"■ Sync error: {e}", "ERROR")
        self.status_var.set("Sync failed")

def analyze_comments_v2(self):
    """
    Analyze MT5 deal comments using the new TradeAccountConnector format.

    Shows detailed breakdown of:
    - Comment format: {TradovateAccountNumber}{PhaseSuffix}
    - Phases: CH (Challenge), FD (Funded), DD (DoubleDip), FA (Farming)
    """
    if not self.pusher.connected:
        messagebox.showerror("Error", "Please connect to MT5 first")
        return

    self.log("=*70)
    self.log("■ MT5 COMMENT ANALYSIS (TradeAccountConnector Format)")
    self.log("=*70)

    if not COMMENT_PARSER_AVAILABLE:
        self.log("■■■ Comment Parser module not available!", "ERROR")
        self.log("    Please ensure mt5_comment_parser.py is in the TradeOpssAI folder")
        return

    deals = self.pusher.get_deals(days=365)

    if not deals:
        self.log("No deals found")
        return

    self.log(f"Total deals fetched: {len(deals)}\n")

    # Use the new parser
    parser = MT5CommentParser()

    # Analyze all unique comments
    comment_analysis = {}
    for deal in deals:
        comment = deal.get('comment', '') or ''
        d_type = str(deal.get('type', '')).upper()

        if d_type in ['BALANCE', 'CREDIT', '2', '3']: # Skip balance and credit ops
            continue

        if comment not in comment_analysis:
            parsed = parser.parse(comment)
            comment_analysis[comment] = {
                'parsed': parsed,
                'count': 0,
                'total_profit': 0,
                'total_commission': 0,
                'total_swap': 0
            }

        comment_analysis[comment]['count'] += 1
        comment_analysis[comment]['total_profit'] += deal.get('profit', 0) or 0
        comment_analysis[comment]['total_commission'] += deal.get('commission', 0) or 0
        comment_analysis[comment]['total_swap'] += deal.get('swap', 0) or 0

    self.log(f"Unique comments found: {len(comment_analysis)}\n")

```

```

self.log("-"*70)

# Group by validity
valid_comments = []
invalid_comments = []

for comment, data in comment_analysis.items():
    parsed = data['parsed']
    if parsed.is_valid:
        valid_comments.append((comment, data))
    else:
        invalid_comments.append((comment, data))

# Show valid comments first
self.log(f"\n■ VALID COMMENTS ({len(valid_comments)}):\n")

for comment, data in sorted(valid_comments, key=lambda x: x[0]):
    parsed = data['parsed']
    net_profit = data['total_profit'] + data['total_commission'] + data['total_swap']

    self.log(f"■ '{comment}'")
    self.log(f"    Account: {parsed.account_number}")
    self.log(f"    Phase: {parsed.phase.name} ({parsed.phase_code})")
    if parsed.trade_number:
        self.log(f"    Trade #: {parsed.trade_number}")
    if parsed.farming_date:
        self.log(f"    Farming Date: {parsed.farming_date.strftime('%Y-%m-%d')}")
    self.log(f"    Deals: {data['count']}, Net P/L: ${net_profit:.2f}")
    self.log("")

# Show invalid comments
if invalid_comments:
    self.log(f"\n■■ UNRECOGNIZED COMMENTS ({len(invalid_comments)}):\n")

    for comment, data in sorted(invalid_comments, key=lambda x: x[0]):
        net_profit = data['total_profit'] + data['total_commission'] + data['total_swap']
        self.log(f"■ '{comment or '(empty)'}'")
        self.log(f"    Deals: {data['count']}, Net P/L: ${net_profit:.2f}")
        self.log("")

self.log("-"*70)
self.log("\n■ EXPECTED COMMENT FORMATS:")
self.log("    Challenge:    {Account}_CH{1-4}      (e.g., MFFUEVSTP326057008_CH1)")
self.log("    Funded:       {Account}_FD{0-4}      (e.g., MFFUEVSTP326057008_FD2)")
self.log("    Double Dip:   {Account}_DD{1-4}      (e.g., MFFUEVSTP326057008_DD1)")
self.log("    Farming:      {Account}_FA           (e.g., MFFUEVSTP326057008_FA)")
self.log("    Farming+Date: {Account}_FA_DDMMYY   (e.g., MFFUEVSTP326057008_FA_210126)")
self.log("="*70)

def show_aggregated_data(self):
    """
    Show aggregated deal data by account and phase.
    Uses the new comment parser to group deals.
    """
    if not self.pusher.connected:
        messagebox.showerror("Error", "Please connect to MT5 first")
        return

    self.log("="*70)
    self.log("■ AGGREGATED DEAL DATA BY ACCOUNT/PHASE")

```

```

self.log("=*70)

result = self.pusher.get_deals_grouped_by_phase(days=365)

aggregated = result.get('aggregated', [])
unmatched = result.get('unmatched', [])
summary = result.get('summary', {})

self.log(f"\nTotal aggregation groups: {len(aggregated)}")
self.log(f"Unmatched deals: {len(unmatched)}\n")

if not aggregated:
    self.log("■■ No aggregated data available")
    self.log("    Make sure your deals have comments in the correct format")
    return

# Group by account for display
by_account = {}
for agg in aggregated:
    account = agg.get('account_number', 'Unknown')
    if account not in by_account:
        by_account[account] = []
    by_account[account].append(agg)

self.log("-=*70)

for account, trades in sorted(by_account.items()):
    account_total = sum(t.get('net_profit', 0) for t in trades)
    self.log(f"\n■ ACCOUNT: {account}")
    self.log(f"    Total Net P/L: ${account_total:.2f}")
    self.log("")

    for trade in sorted(trades, key=lambda x: (x.get('phase_code', ''), x.get('trade_number',
0) or 0)):
        phase_code = trade.get('phase_code', '?')
        phase_name = trade.get('phase_name', 'Unknown')
        trade_num = trade.get('trade_number', '')
        net_profit = trade.get('net_profit', 0)
        deal_count = trade.get('deal_count', 0)
        farming_date = trade.get('farming_date', '')

        label = f"{phase_code}{trade_num or ''}"
        if farming_date:
            label += f" ({farming_date})"

        self.log(f"    [{label}] {phase_name}: ${net_profit:.2f} ({deal_count} deals)")

# Show summary
self.log("\n" + "-=*70)
self.log("\n■ SUMMARY BY PHASE:")
by_phase = summary.get('by_phase', {})
for phase, data in sorted(by_phase.items()):
    self.log(
        f"    {phase}: {data.get('count', 0)} groups, Total: ${data.get('total_net_profit', 0):.2f}"

def push_by_comment(self):
    """
    Push hedge results to dashboard by matching MT5 order comments to evaluation accounts.

    This uses the TradeAccountConnector comment format:

```

- {TradovateAccountNumber}_CH{1-4}: Challenge Hedge Results 1-5
- {TradovateAccountNumber}_FD{0-4}: Funded Hedge Results
- {TradovateAccountNumber}_DD{1-4}: Double Dip (funded hedge results)
- {TradovateAccountNumber}_FA or _FA_DDMMYY: Farming Hedge Days

The function:

1. Aggregates MT5 deals by account/phase from comments
2. Sends aggregated data to dashboard
3. Dashboard matches account numbers and updates hedge results

```

"""
dashboard_url = self.url_entry.get().strip().rstrip('/')
email = self.client_email_entry.get().strip()

if not self.client_info:
    messagebox.showerror("Error",
        "Please lookup the client first by entering email and clicking 'Lookup'")
    return

if not self.pusher.connected:
    messagebox.showerror("Error", "Please connect to MT5 first")
    return

if not COMMENT_PARSER_AVAILABLE:
    messagebox.showerror("Error", "Comment Parser module not available!")
    return

client_name = self.client_info.get('client', '')

self.log("=*70)
self.log("■ PUSH HEDGE RESULTS BY COMMENT")
self.log("=*70)
self.log("Comment Format: {Account}_{Phase}{Number}")
self.log(" CH1-5 → Hedge Result 1-5 (Challenge)")
self.log(" FD0-4 → Hedge Result 1.1-5.1 (Funded)")
self.log(" DD1-4 → Additional Funded Hedge Results")
self.log(" FA/FA_DDMMYY → Hedge Day 1-50 (Farming)")
self.log("Account Matching: First 4 + Last 4 characters")
self.log("=*70)
self.status_var.set("Processing MT5 deals...")

# Step 1: Get and aggregate deals from MT5
self.log("\n■ Step 1: Aggregating deals from MT5 by comment...")

result = self.pusher.get_deals_grouped_by_phase(days=365)

aggregated = result.get('aggregated', [])
unmatched = result.get('unmatched', [])
summary = result.get('summary', {})

if not aggregated:
    self.log("■■ No deals with valid comments found", "WARNING")
    messagebox.showwarning("Warning",
        "No deals found with valid comment format.\nExpected: {Account}_CH{1-5}, {Account}_FD{0-4}")
    return

self.log(f" ✓ Found {len(aggregated)} trade groups")
self.log(f" ■■ {len(unmatched)} deals without valid comments")

# Show sample groups
for agg in aggregated[:5]:

```

```

        account = agg.get('account_number', '')
        phase = agg.get('phase_code', '')
        trade_num = agg.get('trade_number', '')
        profit = agg.get('net_profit', 0)
        sig = f"{account[:4]}...{account[-4:]}" if len(account) >= 8 else account
        self.log(f"    • {sig}_{phase}{trade_num or ''}: ${profit:.2f}")

if len(aggregated) > 5:
    self.log(f"    ... and {len(aggregated) - 5} more groups")

# Step 2: Get account info
self.log("\n■ Step 2: Getting MT5 account info...")
account = self.pusher.get_account_info() or {}
deals = self.pusher.get_deals(days=365)

if account:
    self.log(f"    Balance: ${account.get('balance', 0):.2f}")
    self.log(f"    Deposits: ${account.get('total_deposits', 0):.2f}")
    self.log(f"    Withdrawals: ${account.get('total_withdrawals', 0):.2f}")

# Step 3: Send to dashboard (dashboard will do the matching)
self.log("\n■ Step 3: Sending to dashboard for matching...")
self.status_var.set("Pushing to dashboard...")

# Prepare aggregated data for dashboard
trade_data = []
for agg in aggregated:
    trade_data.append({
        "account_number": agg.get('account_number'),
        "phase_code": agg.get('phase_code'),
        "trade_number": agg.get('trade_number'),
        "farming_date": agg.get('farming_date'),
        "net_profit": agg.get('net_profit'),
        "deal_count": agg.get('deal_count')
    })

payload = {
    "email": email,
    "account": account,
    "positions": self.pusher.get_positions(),
    "deals": deals,
    "aggregated_by_comment": trade_data, # Dashboard will match and update
    "comment_summary": {
        "total_groups": len(aggregated),
        "unmatched_deals": len(unmatched),
        "by_phase": summary.get('by_phase', {})
    },
    "statistics": {}, # Let server recalculate
    "dropdown_options": {}
}

try:
    response = _gzip_post(
        f"{dashboard_url}/api/client/push",
        payload,
        timeout=120
    )

    if response.status_code == 200:
        data = response.json()

```



```

        if data.get("status") == "success":
            # Show match log from dashboard
            hedge_log = data.get("hedge_match_log", [])
            hedge_updates = data.get("hedge_updates", 0)

            self.log(f"\n■ DASHBOARD MATCHING RESULTS:")
            for log_line in hedge_log:
                self.log(f"    {log_line}")

            self.log(f"\n■ HEDGE RESULTS PUSHED SUCCESSFULLY!")
            self.log(f"    {hedge_updates} hedge results updated")
            self.log(f"    {hedge_updates} hedge results updated")
            self.log(f"    {hedge_updates} hedge results updated")
            self.status_var.set(f"Pushed! {hedge_updates} updates")
            messagebox.showinfo("Success",
                                f"Pushed {len(trade_data)} trade groups.\n{hedge_updates} hedge results updated")
        else:
            self.log(f"■ Push failed: {data.get('message', 'Unknown error')}", "ERROR")
            self.status_var.set("Push failed")
    else:
        error_msg = f"HTTP {response.status_code}"
        try:
            error_msg = response.json().get("message", error_msg)
        except:
            pass
        self.log(f"■ Push failed: {error_msg}", "ERROR")
        self.status_var.set("Push failed")

except Exception as e:
    self.log(f"■ Push error: {e}", "ERROR")
    self.status_var.set("Push failed")

# ■■ Import source toggle helpers ■■

def _toggle_import_source(self):
    """Show/hide the correct input row based on selected import source."""
    if self.import_source.get() == 'sheet':
        self.csv_input_frame.pack_forget()
        self.sheet_input_frame.pack(fill="x", pady=2)
        self.import_btn.configure(text="Import Sheet Data")
        self.import_hint.configure(text="Sheet must be publicly shared")
    else:
        self.sheet_input_frame.pack_forget()
        self.csv_input_frame.pack(fill="x", pady=2)
        self.import_btn.configure(text="Import CSV File")
        self.import_hint.configure(text="Use a CSV exported from the dashboard")

def _browse_csv(self):
    """Open file dialog to pick a CSV file."""
    path = filedialog.askopenfilename(
        title="Select CSV File",
        filetypes=[("CSV files", "*.csv"), ("All files", "*.*")]
    )
    if path:
        self.csv_path_var.set(path)

def _do_import(self):
    """Route to the correct import method."""
    if self.import_source.get() == 'sheet':
        self.migrate_from_sheet()
    else:

```

```

        self.import_from_csv()

def import_from_csv(self):
    """Import evaluation data from a CSV file previously exported from the dashboard."""
    email = self.client_email_entry.get().strip()
    csv_path = self.csv_path_var.get().strip()
    dashboard_url = self.url_entry.get().strip().rstrip('/')

    if not email:
        messagebox.showerror("Error", "Please enter your client email first")
        return

    if not csv_path:
        messagebox.showerror("Error", "Please select a CSV file first")
        return

    if not os.path.isfile(csv_path):
        messagebox.showerror("Error", f"File not found:\n{csv_path}")
        return

    if not self.client_info:
        messagebox.showerror("Error",
            "Please lookup the client first by entering email and clicking 'Lookup'")
        return

    client_name = self.client_info.get('client', '')
    if not client_name:
        messagebox.showerror("Error", "Client lookup failed - no client name found")
        return

    # Confirm before proceeding
    if not messagebox.askyesno("Confirm CSV Import",
        f"This will import CSV data into {client_name}'s dashboard.\n\n"
        f"File: {os.path.basename(csv_path)}\n\n"
        "Existing rows matched by Account # will be updated.\n\n"
        "New rows will be appended.\n\nContinue?"):
        return

    self.log(f"■ Importing CSV for {client_name}...")
    self.status_var.set("Importing CSV...")
    self.import_btn.configure(state="disabled")
    self.root.update_idletasks()

def _do_import():
    try:
        with open(csv_path, 'rb') as f:
            response = requests.post(
                f"{dashboard_url}/api/client/import_csv_companion",
                data={"email": email},
                files={"file": (os.path.basename(csv_path), f, "text/csv")},
                timeout=120
            )

        if response.status_code != 200:
            error_msg = f"HTTP {response.status_code}"
            try:
                error_msg = response.json().get("message", error_msg)
            except:
                pass
            def _on_http_err(msg=error_msg):

```

```

        self.log(f"■ CSV import failed: {msg}", "ERROR")
        self.status_var.set("Import failed")
        self.import_btn.configure(state="normal")
        messagebox.showerror("Error", msg)
        self.root.after(0, _on_http_err)
        return

data = response.json()
if data.get("status") != "success":
    error_msg = data.get("message", "Import failed")
    def _on_api_err(msg=error_msg):
        self.log(f"■ {msg}", "ERROR")
        self.status_var.set("Import failed")
        self.import_btn.configure(state="normal")
        messagebox.showerror("Error", msg)
    self.root.after(0, _on_api_err)
    return

updated = data.get('updated', 0)
added = data.get('added', 0)
total = data.get('total_rows', 0)
def _on_success():
    self.log(f"    ■ CSV import complete!")
    self.log(f"    {updated} rows updated, {added} rows added ({total} total evaluations)")
    self.status_var.set(f"Imported {updated + added} rows from CSV")
    self.import_btn.configure(state="normal")
    messagebox.showinfo("Success",
        f"CSV import complete!\n\n"
        f"• {updated} rows updated\n"
        f"• {added} rows added\n"
        f"• {total} total evaluations")
    self.lookup_client()
    self.root.after(0, _on_success)

except requests.exceptions.Timeout:
    def _on_timeout():
        self.log("■ Connection timeout during CSV import", "ERROR")
        self.status_var.set("Timeout")
        self.import_btn.configure(state="normal")
        messagebox.showerror("Timeout", "Connection timed out. Please try again.")
    self.root.after(0, _on_timeout)
except requests.exceptions.ConnectionError:
    def _on_conn_err():
        self.log("■ Could not connect to dashboard server", "ERROR")
        self.status_var.set("Connection failed")
        self.import_btn.configure(state="normal")
        messagebox.showerror("Error",
            "Could not connect to dashboard server. Check the URL and try again.")
    self.root.after(0, _on_conn_err)
except Exception as e:
    def _on_exc(err=str(e)):
        self.log(f"■ CSV import error: {err}", "ERROR")
        self.status_var.set("Import failed")
        self.import_btn.configure(state="normal")
        messagebox.showerror("Error", err)
    self.root.after(0, _on_exc)

threading.Thread(target=_do_import, daemon=True).start()

def migrate_from_sheet(self):

```

```

"""Migrate data from Google Sheets to the dashboard with verification."""
email = self.client_email_entry.get().strip()
sheet_url = self.sheet_url_entry.get().strip()
dashboard_url = self.url_entry.get().strip().rstrip('/')

if not email:
    messagebox.showerror("Error", "Please enter your client email first")
    return

if not sheet_url:
    messagebox.showerror("Error", "Please enter the Google Sheet URL")
    return

if 'docs.google.com/spreadsheets' not in sheet_url:
    messagebox.showerror("Error", "Please enter a valid Google Sheets URL")
    return

self.log(f"Migrating sheet data to dashboard...")
self.status_var.set("Migrating sheet data...")
self.root.update_idletasks()

def _do_migrate():
    try:
        response = requests.post(
            f"{dashboard_url}/api/client/migrate_sheet",
            json={"email": email, "sheet_url": sheet_url},
            headers={"Content-Type": "application/json"},
            timeout=180
        )

        if response.status_code != 200:
            error_msg = f"HTTP {response.status_code}"
            try:
                error_msg = response.json().get("message", error_msg)
            except:
                pass
            def _on_err(msg=error_msg):
                self.log(f"■ Migration failed: {msg}", "ERROR")
                self.status_var.set("Migration failed")
                messagebox.showerror("Error", msg)
            self.root.after(0, _on_err)
            return

        data = response.json()
        if data.get("status") != "success":
            error_msg = data.get("message", "Migration failed")
            def _on_api_err(msg=error_msg):
                self.log(f"■ {msg}", "ERROR")
                self.status_var.set("Migration failed")
                messagebox.showerror("Error", msg)
            self.root.after(0, _on_api_err)
            return

        records = data.get("records_imported", 0)
        def _on_success():
            self.log(f" ■ Successfully imported {records} records")
            self.log(f" Dashboard data fully replaced.")
            self.status_var.set(f"Imported {records} records")
            messagebox.showinfo("Success",
                f"Successfully imported {records} records.\nDashboard data has been updated.")

```

```

        self.lookup_client()
        self.root.after(0, _on_success)

except requests.exceptions.Timeout:
    def _on_timeout():
        self.log("■ Connection timeout - server is still processing the sheet", "ERROR")
        self.status_var.set("Timeout")
        messagebox.showerror("Timeout",
            "Connection timed out. The sheet may be too large or the server is busy. Please t
        self.root.after(0, _on_timeout)
except requests.exceptions.ConnectionError:
    def _on_conn_err():
        self.log("■ Could not connect to dashboard server", "ERROR")
        self.status_var.set("Connection failed")
        messagebox.showerror("Error",
            "Could not connect to dashboard server. Check the URL and try again.")
        self.root.after(0, _on_conn_err)
except Exception as e:
    def _on_exc(err=str(e)):
        self.log(f"■ Migration error: {err}", "ERROR")
        self.status_var.set("Migration failed")
        messagebox.showerror("Error", err)
        self.root.after(0, _on_exc)

threading.Thread(target=_do_migrate, daemon=True).start()

def verify_stats(self, local_stats, dashboard_stats):
    """Compare local stats with dashboard stats and return list of discrepancies."""
    discrepancies = []
    tolerance = 0.01 # Allow $0.01 difference for rounding

    # Helper to compare values
    def compare(name, local_val, dash_val):
        if isinstance(local_val, (int, float)) and isinstance(dash_val, (int, float)):
            if abs(local_val - dash_val) > tolerance:
                discrepancies.append(f"{name}: Local=${local_val:,.2f} vs Dashboard=${dash_val:,.2f}")
        elif local_val != dash_val:
            discrepancies.append(f"{name}: Local={local_val} vs Dashboard={dash_val}")

    # Compare profitability_completed
    local_prof = local_stats.get('profitability_completed', {})
    dash_prof = dashboard_stats.get('profitability_completed', {})
    compare("Prof.Challenge Fees", local_prof.get('challenge_fees', 0), dash_prof.get('challenge_fees', 0))
    compare("Prof.Hedging Results", local_prof.get('hedging_results', 0), dash_prof.get('hedging_results', 0))
    compare("Prof.Farming Results", local_prof.get('farming_results', 0), dash_prof.get('farming_results', 0))
    compare("Prof.Payouts", local_prof.get('payouts', 0), dash_prof.get('payouts', 0))
    compare("Prof.Net Profit", local_prof.get('net_profit', 0), dash_prof.get('net_profit', 0))

    # Compare cashflow_inprogress
    local_cash = local_stats.get('cashflow_inprogress', {})
    dash_cash = dashboard_stats.get('cashflow_inprogress', {})
    compare("Cash.Challenge Fees", local_cash.get('challenge_fees', 0), dash_cash.get('challenge_fees', 0))
    compare("Cash.Hedging Results", local_cash.get('hedging_results', 0), dash_cash.get('hedging_results', 0))
    compare("Cash.Farming Results", local_cash.get('farming_results', 0), dash_cash.get('farming_results', 0))

```

```

compare("Cash.Payouts", local_cash.get('payouts', 0), dash_cash.get('payouts', 0))
compare("Cash.Net Profit", local_cash.get('net_profit', 0), dash_cash.get('net_profit', 0))

# Compare eval_totals
local_et = local_stats.get('eval_totals', {})
dash_et = dashboard_stats.get('eval_totals', {})
compare("Eval.Total Running", local_et.get('total_running', 0), dash_et.get('total_running', 0))
compare("Eval.Total Passed", local_et.get('total_passed', 0), dash_et.get('total_passed', 0))
compare("Eval.Total Failed", local_et.get('total_failed', 0), dash_et.get('total_failed', 0))

# Compare funded_totals
local_ft = local_stats.get('funded_totals', {})
dash_ft = dashboard_stats.get('funded_totals', {})
compare("Funded.Not Started", local_ft.get('not_started', 0), dash_ft.get('not_started', 0))
compare("Funded.Ongoing", local_ft.get('ongoing', 0), dash_ft.get('ongoing', 0))
compare("Funded.Failed", local_ft.get('failed', 0), dash_ft.get('failed', 0))
compare("Funded.Completed", local_ft.get('completed', 0), dash_ft.get('completed', 0))

return discrepancies

def toggle_auto_push(self):
    """Toggle automatic data pushing."""
    if self.auto_push_enabled:
        self.auto_push_enabled = False
        try:
            self.auto_btn.configure(text="Auto-Push")
        except Exception:
            pass
        try:
            self.auto_btn_live.configure(text="Auto-Push")
        except Exception:
            pass
        self.log("Auto-push stopped")
    else:
        if not self.client_info:
            messagebox.showerror("Error", "Please lookup the client first")
            return

        # Initialize state
        self.last_deal_count = 0
        self.last_deal_ticket = 0

        self.auto_push_enabled = True
        try:
            self.auto_btn.configure(text="Stop Auto-Push")
        except Exception:
            pass
        try:
            self.auto_btn_live.configure(text="Stop Auto-Push")
        except Exception:
            pass

        self.log("Smart Auto-push started (Checking for new trades...)")
        self.auto_push_thread = threading.Thread(target=self.auto_push_loop, daemon=True)
        self.auto_push_thread.start()

def check_and_push_update(self):
    """Check if new trades exist and push update if so."""
    if not self.auto_push_enabled: return

    try:

```

```

if not self.pusher.connected:
    self.log("■■ Auto-push: MT5 not connected – skipping check", "ERROR")
    return

deals = self.pusher.get_deals(days=30)

if not deals:
    self.log("■■ Auto-push: MT5 returned 0 deals – connection issue?")
    return

current_count = len(deals)
last_deal = deals[-1]
current_ticket = last_deal.get('ticket')

if current_ticket is None:
    self.log("■■ Auto-push: last deal has no ticket ID")

# First check – initialize state and push once
if self.last_deal_count == 0:
    self.last_deal_count = current_count
    self.last_deal_ticket = current_ticket
    self.log(f"■ Auto-push scan: {current_count} deals, last ticket: {current_ticket}")
    self.push_data()
    return

if current_count > self.last_deal_count or current_ticket != self.last_deal_ticket:
    self.log(
        f"■ New trade! Deals: {self.last_deal_count}→{current_count} | Ticket: {current_ticket}"

    self.last_deal_count = current_count
    self.last_deal_ticket = current_ticket

    self.push_data()

except Exception as e:
    self.log(f"■ Auto-push check error: {e}", "ERROR")

def auto_push_loop(self):
    """Background loop for smart auto-pushing."""
    cycle = 0
    while self.auto_push_enabled:
        try:
            self.root.after(0, self.check_and_push_update)
        except Exception as e:
            # Thread-safe: can't call self.log from background thread directly
            try:
                self.root.after(0, lambda err=str(e): self.log(f"■ Auto-push loop error: {err}", "ERROR"))
            except Exception:
                pass

        # Check frequently (every 10s)
        for _ in range(10):
            if not self.auto_push_enabled:
                break
            time.sleep(1)

        cycle += 1
        # Heartbeat every ~60s (6 cycles of 10s)
        if cycle % 6 == 0 and self.auto_push_enabled:
            try:

```

```

        self.root.after(0, lambda c=cycle: self.log(
            f"■ Auto-push heartbeat (cycle {c}) – watching for new trades"))
    except Exception:
        pass

# ===== Trading Engine =====

def _build_trading_engine_ui(self, parent):
    """Build the Trading Engine section with CTK styled cards."""

    # ■■ MT5 Connection Card ■■
    mt5_card = self._section_card(parent, "MT5 CONNECTION", "■")
    mt5_card.pack(fill="x", padx=4, pady=(4, 2))

    mt5_row = ctk.CTkFrame(mt5_card, fg_color="transparent") if CTK_AVAILABLE else \
        tk.Frame(mt5_card, bg="#161B22")
    mt5_row.pack(fill="x", padx=10, pady=(2, 2))

    if CTK_AVAILABLE:
        ctk.CTkLabel(mt5_row, text="Login:", font=("Segoe UI", 11),
            text_color=self.C_TEXT_DIM).pack(side="left", padx=(0, 4))
        self.mt5_login = self._ctk_entry(mt5_row, width=120)
        self.mt5_login.pack(side="left", padx=(0, 8))

    if CTK_AVAILABLE:
        ctk.CTkLabel(mt5_row, text="Pass:", font=("Segoe UI", 11),
            text_color=self.C_TEXT_DIM).pack(side="left", padx=(0, 4))
        self.mt5_password = self._ctk_entry(mt5_row, width=120, show="*")
        self.mt5_password.pack(side="left", padx=(0, 8))

    if CTK_AVAILABLE:
        ctk.CTkLabel(mt5_row, text="Server:", font=("Segoe UI", 11),
            text_color=self.C_TEXT_DIM).pack(side="left", padx=(0, 4))
        self.mt5_server = self._ctk_entry(mt5_row, width=160)
        self.mt5_server.pack(side="left", padx=(0, 8))

    mt5_btn_row = ctk.CTkFrame(mt5_card, fg_color="transparent") if CTK_AVAILABLE else \
        tk.Frame(mt5_card, bg="#161B22")
    mt5_btn_row.pack(fill="x", padx=10, pady=(0, 4))
    self.mt5_btn = self._ctk_button(mt5_btn_row, text="Connect MT5",
        command=self.toggle_mt5_connection,
        fg="#24292F", hover="#000000", width=140)
    self.mt5_btn.pack(side="left")

    if not TRADING_ENGINE_AVAILABLE:
        if CTK_AVAILABLE:
            ctk.CTkLabel(mt5_card, text="Trading engine modules not loaded – broker trading unavailable",
                font=("Segoe UI", 9, "italic"), text_color=self.C_GOLD,
                wraplength=500).pack(padx=12, pady=(0, 8))
        return

    # ■■ Broker Connections Card ■■
    broker_card = self._section_card(parent, "BROKER CONNECTIONS", "■")
    broker_card.pack(fill="x", padx=4, pady=2)

    # Global settings row: Platform + Mode + Connect All
    bk_global = ctk.CTkFrame(broker_card, fg_color="transparent") if CTK_AVAILABLE else \
        tk.Frame(broker_card, bg="#161B22")
    bk_global.pack(fill="x", padx=10, pady=(2, 2))

    if CTK_AVAILABLE:

```



```

        ctk.CTkLabel(bk_global, text="Platform:", font=("Segoe UI", 11),
                    text_color=self.C_TEXT_DIM).pack(side="left", padx=(0, 4))
self.broker_var = tk.StringVar(value="Tradovate")
platforms = ["Tradovate", "TopStepX"]
if CTK_AVAILABLE:
    ctk.CTkComboBox(bk_global, variable=self.broker_var, values=platforms,
                    state="readonly", width=120, height=30,
                    fg_color=self.C_BG_THIRD, border_color=self.C_BORDER,
                    button_color=self.C_ACCENT, text_color=self.C_TEXT,
                    dropdown_fg_color=self.C_BG_SEC).pack(side="left", padx=(0, 12))
else:
    ttk.Combobox(bk_global, textvariable=self.broker_var, values=platforms,
                 state='readonly', width=14).pack(side="left", padx=(0, 8))

if CTK_AVAILABLE:
    ctk.CTkLabel(bk_global, text="Mode:", font=("Segoe UI", 11),
                text_color=self.C_TEXT_DIM).pack(side="left", padx=(0, 4))
self.trading_mode_var = tk.StringVar(value="Simulation")
if CTK_AVAILABLE:
    ctk.CTkComboBox(bk_global, variable=self.trading_mode_var,
                    values=["Simulation", "Live"], state="readonly",
                    width=120, height=30, fg_color=self.C_BG_THIRD,
                    border_color=self.C_BORDER, button_color=self.C_ACCENT,
                    text_color=self.C_TEXT,
                    dropdown_fg_color=self.C_BG_SEC).pack(side="left", padx=(0, 12))
else:
    ttk.Combobox(bk_global, textvariable=self.trading_mode_var,
                 values=["Simulation", "Live"], state='readonly', width=14).pack(side="left", padx=
                 10))

self.broker_connect_all_btn = self._ctk_button(bk_global, text="Connect All",
                                                command=self._connect_all_brokers,
                                                fg="#24292F", hover="#000000", width=100)
self.broker_connect_all_btn.pack(side="left", padx=(0, 6))

self.broker_status_var = tk.StringVar(value="")
if CTK_AVAILABLE:
    ctk.CTkLabel(bk_global, textvariable=self.broker_status_var,
                font=("Segoe UI", 9), text_color=self.C_TEXT_DIM).pack(side="left")
else:
    ttk.Label(bk_global, textvariable=self.broker_status_var).pack(side="left")

# Dynamic broker rows container (populated when trades load)
self._broker_rows_frame = ctk.CTkFrame(broker_card, fg_color="transparent") if CTK_AVAILABLE else
    tk.Frame(broker_card, bg="#161B22")
self._broker_rows_frame.pack(fill="x", padx=6, pady=(0, 4))

if CTK_AVAILABLE:
    ctk.CTkLabel(self._broker_rows_frame,
                text="Load trades to populate prop firm connections",
                font=("Segoe UI", 9, "italic"), text_color="#4A5568").pack(pady=4)

# ■■ Hedge Mode / Direction (inline) ■■
opts_row = ctk.CTkFrame(parent, fg_color="transparent") if CTK_AVAILABLE else \
    tk.Frame(parent)
opts_row.pack(fill="x", padx=10, pady=(2, 2))

self.hedge_mode_var = tk.StringVar(value="Hedging")
if CTK_AVAILABLE:
    ctk.CTkRadioButton(opts_row, text="Hedging (Broker+MT5)", variable=self.hedge_mode_var,

```

```

        value="Hedging", font=("Segoe UI", 11), text_color=self.C_TEXT,
        fg_color=self.C_ACCENT, border_color=self.C_BORDER).pack(side="left", padx=
        12))
    ctk.CTkRadioButton(opts_row, text="Broker Only", variable=self.hedge_mode_var,
        value="BrokerOnly", font=("Segoe UI", 11), text_color=self.C_TEXT,
        fg_color=self.C_ACCENT, border_color=self.C_BORDER).pack(side="left", padx=
        20))
else:
    ttk.Radiobutton(opts_row, text="Hedging (Broker+MT5)", variable=self.hedge_mode_var,
        value="Hedging").pack(side="left", padx=(0, 8))
    ttk.Radiobutton(opts_row, text="Broker Only", variable=self.hedge_mode_var,
        value="BrokerOnly").pack(side="left", padx=(0, 16))

self.direction_var = tk.StringVar(value="All Trades")
if CTK_AVAILABLE:
    ctk.CTkLabel(opts_row, text="Direction:", font=("Segoe UI", 11),
        text_color=self.C_TEXT_DIM).pack(side="left", padx=(0, 4))
    ctk.CTkComboBox(opts_row, variable=self.direction_var,
        values=["All Trades", "Buy Only", "Sell Only"],
        state="readonly", width=130, height=32,
        fg_color=self.C_BG_THIRD, border_color=self.C_BORDER,
        button_color=self.C_ACCENT, text_color=self.C_TEXT,
        dropdown_fg_color=self.C_BG_SEC).pack(side="left")
else:
    ttk.Label(opts_row, text="Direction:").pack(side="left", padx=(0, 4))
    ttk.Combobox(opts_row, textvariable=self.direction_var,
        values=["All Trades", "Buy Only", "Sell Only"],
        state='readonly', width=12).pack(side="left")

# ■■■ Hidden vars for backward compat with save/load config ■■■
self.prop_firm_var = tk.StringVar(value="MFFU_Flex")
self.phase_var = tk.StringVar(value="challenge_trade1")
self.acct_size_var = tk.StringVar(value="$50,000")
self.strategy_var = tk.StringVar(value="Random Entries")

# Dummy combos (hidden, for _on_prop_firm_change / _update_account_sizes compat)
self.prop_firm_combo = ttk.Combobox(parent)
self.phase_combo = ttk.Combobox(parent)
self.acct_size_combo = ttk.Combobox(parent)

# ■■■ Phase detection helpers ■■■

_FIRM_MAP = {
    "My Funded Futures": "MFFU_Flex",
    "MFFU": "MFFU",
    "Funding Ticks": "FundingTicks",
    "Funded Next": "Funded Next",
    "FundedNext": "Funded Next",
    "TopStep": "TopStep",
    "TradeDay": "TradeDay",
    "Tradeify": "Tradeify",
    "Alpha Futures": "AlphaFutures",
    "Apex": "Apex",
    "Top One Futures": "Top One Futures",
}

_FAILED_STATUSES = {"fail", "failed", "breach", "delete", "deleted", "closed", "sl", "ended", "lost"}

# Keywords for substring matching (catches "Fail", "Failed", "Breached", etc.)
_INACTIVE_KEYWORDS = ("fail", "breach", "delete", "closed", "sl", "ended", "lost")

```

```

# ■■ Funded-phase starting balance map ■■■
# Some prop firms reset the funded balance to a value that differs from
# the challenge "Account Size". Without this, current-profit math is wrong
# by tens of thousands of dollars and TP/SL adjustment goes haywire
# (e.g. MFFU funded balance starts at $0 – using $50K as start makes the
# adjuster think we are -$50K down and inflates TP ~15x to "recover").
#
# Convention:
# - "ZERO" → funded balance literally starts at $0
# - "ACCOUNT_SIZE" (default) → starts at the challenge size (e.g. $50K)
_FUNDED_START_BALANCE_MODE: Dict[str, str] = {
    "MFFU": "ZERO",
    "MFFU_Flex": "ZERO",
    "TopStep": "ZERO", # TopStep funded accounts also start at $0
    "Funded Next": "ACCOUNT_SIZE",
    "FundingTicks": "ACCOUNT_SIZE",
    "TradeDay": "ACCOUNT_SIZE",
    "Tradeify": "ACCOUNT_SIZE",
    "AlphaFutures": "ACCOUNT_SIZE",
    "Apex": "ACCOUNT_SIZE",
    "Lucid": "ACCOUNT_SIZE",
    "Top One Futures": "ACCOUNT_SIZE",
}

def _resolve_starting_balance(self, ev, current_phase, acct_size):
    """Return the firm-correct starting balance for profit math.

    For Funded / Payout phases on firms whose funded balance starts at $0
    (MFFU family), this returns 0 instead of the challenge size. All other
    firms / phases fall back to the challenge size parsed from acct_size.
    """
    # Parse acct_size as the default
    start_str = str(acct_size or '').replace("$", "").replace(",", "").strip().lower()
    if start_str.endswith("k"):
        try:
            default_start = float(start_str[:-1]) * 1000
        except ValueError:
            default_start = 50000.0
    elif start_str.replace(".", "").isdigit():
        default_start = float(start_str)
    else:
        default_start = 50000.0

    # Only Funded / Payout / Double Dip phases differ from challenge size
    funded_phases = ("Funded", "Payout 1", "Payout 2", "Payout 3", "Payout 4", "Double Dip")
    if current_phase not in funded_phases:
        return default_start

    firm_raw = (ev or {}).get("Prop Firm", "") if isinstance(ev, dict) else ""
    canonical = self._FIRM_MAP.get(firm_raw, firm_raw)
    mode = self._FUNDED_START_BALANCE_MODE.get(canonical, "ACCOUNT_SIZE")
    if mode == "ZERO":
        return 0.0
    return default_start

def _detect_eval_phase(self, ev):
    """Determine current phase display name and blueprint key for an evaluation."""
    challenge_status = (ev.get("Status P1", "") or "").strip().lower()
    funded_status = (ev.get("Status", "") or "").strip().lower()
    has_funded_acct = bool((ev.get("Account #.1", "") or "").strip())

```

```

# Check if farming data exists – must have BOTH the farming marker
# AND actual Hedge Day cell data for THIS account (not just sheet columns)
has_farming_marker = bool((ev.get("Prop Day 1", "") or "").strip())
has_hedge_day_data = False
if has_farming_marker:
    for i in range(1, 35):
        val = (ev.get(f"Hedge Day {i}", "") or "").strip()
        if val and val not in ("-", "-"):
            has_hedge_day_data = True
            break

if has_farming_marker and has_hedge_day_data:
    return "Farming", "farming"
elif has_funded_acct and funded_status not in self._FAILED_STATUSES:
    return "Funded", "funded_trade1"
else:
    return "Challenge", "challenge_trade1"

def _resolve_phase_key_from_day(self, ev, firm_code, current_phase):
    """Use the day placeholder cell index to determine the correct blueprint key.

    The day placeholder position (0-based) maps directly to the trade order
    in _PHASE_TRADE_ORDER. E.g. cell index 2 → third trade in the sequence.

    Scans all field sets if the detected phase has no day placeholder,
    and corrects the phase accordingly.

    Returns (resolved_phase_key, day_index, day_name) or (None, None, None)
    if no day placeholder is found.
    """
    day_idx, day_name, is_today, matched_phase = self._find_tradeable_day_cell(ev, current_phase)
    if day_idx is None:
        return None, None, None

    # If day was found in a different phase's fields, correct the phase
    effective_phase = matched_phase if matched_phase else current_phase
    if matched_phase and matched_phase != current_phase:
        self.log(f"■ Phase correction: detected '{current_phase}' but day "
                f"placeholder found in '{matched_phase}' fields")

    if not self.prop_firm_mgr:
        return None, day_idx, day_name

    # Normalize phase for _PHASE_TRADE_ORDER lookup
    phase_map = {"Challenge": "Challenge", "Funded": "Funded",
                 "Farming": "Farming", "Double Dip": "Double Dip",
                 "Payout 1": "Funded", "Payout 2": "Funded",
                 "Payout 3": "Funded", "Payout 4": "Funded"}
    phase_group = phase_map.get(effective_phase, effective_phase)

    firm_orders = self.prop_firm_mgr._PHASE_TRADE_ORDER.get(firm_code, {})
    trade_keys = firm_orders.get(phase_group, [])

    if not trade_keys:
        return None, day_idx, day_name

    # Clamp day index to available trade keys
    key_idx = min(day_idx, len(trade_keys) - 1)
    resolved_key = trade_keys[key_idx]
    return resolved_key, day_idx, day_name

```

```

# Day-name abbreviations that traders use as placeholders
_DAY_ABBREVS = {
    "mon": 0, "monday": 0,
    "tue": 1, "tues": 1, "tuesday": 1,
    "wed": 2, "weds": 2, "wednesday": 2,
    "thu": 3, "thur": 3, "thurs": 3, "thursday": 3,
    "fri": 4, "friday": 4,
    "sat": 5, "saturday": 5,
    "sun": 6, "sunday": 6,
}

@classmethod
def _parse_day_token(cls, value):
    """Extract a weekday number (0=Mon..6=Sun) from a cell value.

    Lenient parsing that handles surrounding punctuation, extra text,
    and embedded day names (e.g. ``"MONDAY ✓"`, ``"Mon-04/27"`,
    ``"Mon - waiting"``). Returns the weekday number or ``None``.
    """
    if value is None:
        return None
    try:
        s = str(value).strip().lower()
    except Exception:
        return None
    if not s:
        return None
    # Whole-string match first
    if s in cls._DAY_ABBREVS:
        return cls._DAY_ABBREVS[s]
    # Tokenise on common separators (whitespace, hyphen, slash, comma, colon)
    import re as _re
    for tok in _re.split(r"[\s\-/,:;\.]+", s):
        tok = tok.strip()
        if tok in cls._DAY_ABBREVS:
            return cls._DAY_ABBREVS[tok]
    return None

def _get_phase_fields(self, current_phase):
    """Return the ordered list of eval field names for a phase."""
    if current_phase == "Challenge":
        return [f"Hedge Result {i}" for i in range(1, 6)]
    elif current_phase in ("Funded", "Payout 1", "Payout 2", "Payout 3", "Payout 4"):
        return [f"Hedge Result {i}.1" for i in range(1, 8)]
    elif current_phase == "Double Dip":
        return [f"Hedge Result {i}.1" for i in range(1, 8)]
    elif current_phase == "Farming":
        return [f"Hedge Day {i}" for i in range(1, 35)]
    return []

# All possible field sets for day placeholder scanning (phase → fields)
_ALL_PHASE_FIELD_SETS = [
    ("Challenge", [f"Hedge Result {i}" for i in range(1, 6)]),
    ("Funded", [f"Hedge Result {i}.1" for i in range(1, 8)]),
    ("Farming", [f"Hedge Day {i}" for i in range(1, 35)]),
]

def _count_completed_trades(self, ev, current_phase):
    """Count how many trading day cells are filled for the current phase.

    A filled cell = a trade already taken. Empty/blank = not yet traded.

```

```

Day-name placeholders (MON, TUE, etc.) are NOT counted as completed.
Returns (completed_count, total_fields, next_empty_index).
next_empty_index is 0-based: the position of the first empty cell.
"""
fields = self._get_phase_fields(current_phase)

completed = 0
next_empty = None
for i, f in enumerate(fields):
    val = ev.get(f, None)
    val_str = str(val).strip() if val is not None else ""
    if val_str in ("", "-", "."):
        # Empty cell
        if next_empty is None:
            next_empty = i
    elif self._parse_day_token(val_str) is not None:
        # Day placeholder – not yet traded
        if next_empty is None:
            next_empty = i
    else:
        # Has a real value (dollar amount, etc.) – completed trade
        completed += 1

if next_empty is None:
    next_empty = len(fields) # All filled

return completed, len(fields), next_empty

def _find_tradeable_day_cell(self, ev, current_phase):
    """Find the cell that should be traded today based on day placeholders.

    A cell is tradeable today if it contains today's day name OR a
    previous weekday that hasn't been filled with a result yet (i.e. the
    trader missed entering a day and it's still pending).

    A cell whose day is AFTER today = already been prepared for the next
    trading day and should NOT be traded now.

    Scans the detected phase's fields first. If nothing found, falls back
    to scanning ALL other phase field sets so a misdetected phase doesn't
    block trading.

    Returns (stage_index, day_name, is_today, matched_phase) or
    (None, None, False, None).
    stage_index is 0-based position in the field list.
    matched_phase is the phase name whose fields contained the match.
    """
    import datetime
    today_weekday = datetime.date.today().weekday() # 0=Mon .. 4=Fri

    # Build ordered list: detected phase first, then all others as fallback
    primary_fields = self._get_phase_fields(current_phase)
    search_order = [(current_phase, primary_fields)]
    for phase_name, field_list in self._ALL_PHASE_FIELD_SETS:
        if field_list != primary_fields:
            search_order.append((phase_name, field_list))

    for phase_name, fields in search_order:
        best_idx = None
        best_day_name = None

```

```

best_day_num = -1
best_is_today = False

for i, f in enumerate(fields):
    val = ev.get(f, None)
    if val is None:
        continue
    day_num = self._parse_day_token(val)
    if day_num is None:
        continue # Not a day placeholder (result value or empty)

    if day_num == today_weekday:
        # Exact match – this cell is for today
        return i, str(val).strip().upper(), True, phase_name
    elif day_num < today_weekday:
        # Previous day still has a day placeholder (not yet traded).
        # Pick the most-recent missed day.
        if best_idx is None or day_num > best_day_num:
            best_idx = i
            best_day_name = str(val).strip().upper()
            best_day_num = day_num
            best_is_today = False
        # day_num > today_weekday → future day, skip

if best_idx is not None:
    return best_idx, best_day_name, best_is_today, phase_name

return None, None, False, None

def _validate_stage_consistency(self, prediction, ev, current_phase, acct_num):
    """Validate whether a trade should proceed using day placeholders as
    the primary gate, with balance and cell count as advisory warnings.

    Day placeholder rule (GATE – controls trade/no-trade):
    - Cell with today's day (or a missed previous day) → TRADE
    - No matching day placeholder found → NO TRADE
    - Only future day placeholders → NO TRADE (already prepared)

    Balance + cell count (ADVISORY – warnings only, never block):
    - If balance stage or cell count disagree, log warnings
    - But trade still proceeds if day placeholder confirms

    Returns (should_trade: bool, message: str).
    """
    import datetime
    fields = self._get_phase_fields(current_phase)
    stages = prediction.get("stages", []) if prediction else []

    # ■■ Primary gate: day placeholder ■■
    day_idx, day_name, is_today, matched_phase = self._find_tradeable_day_cell(ev, current_phase)

    if day_idx is None:
        # No day placeholder for today or any missed day → don't trade
        # Check if there's a future day placeholder across ALL field sets
        future_days = []
        today_wd = datetime.date.today().weekday()
        all_fields = []
        for _, flist in self._ALL_PHASE_FIELD_SETS:
            all_fields.extend(flist)
        for i, f in enumerate(all_fields):

```

```

        val = ev.get(f, None)
        if val is None:
            continue
        dn = self._parse_day_token(val)
        if dn is not None and dn > today_wd:
            future_days.append((i, str(val).strip().upper()))

    if future_days:
        next_day = future_days[0]
        msg = (f"■■ {acct_num}: No trade today – "
              f"next day placeholder is {next_day[1]} in cell {next_day[0] + 1}. "
              f"Already prepared for next trading day.")
    else:
        msg = (f"■ {acct_num}: No day placeholder found for today – "
              f"trader needs to enter a day name (MON/TUE/etc.) "
              f"in the next cell to enable trading.")
    return False, msg

# Day placeholder found – trade is confirmed
day_stage_idx = min(day_idx, len(stages) - 1) if stages else day_idx
day_label = "today" if is_today else f"missed {day_name}"

msgs = []
msgs.append(
    f"■ {acct_num}: Trade confirmed via {day_name} placeholder "
    f"({'today' if is_today else 'pending from earlier'}) "
    f"in cell {day_idx + 1}")

# ■■ Advisory: balance check ■■
if prediction and stages:
    balance_stage_idx = 0
    current_key = prediction.get("current_phase_key", "")
    for i, (_, key, _, _) in enumerate(stages):
        if key == current_key:
            balance_stage_idx = i
            break

    if balance_stage_idx != day_stage_idx:
        msgs.append(
            f"■■ Balance advisory: balance suggests stage "
            f"{balance_stage_idx + 1} ({prediction['current_stage']}), "
            f"but day placeholder is in cell {day_idx + 1} (stage {day_stage_idx + 1})")

# ■■ Advisory: cell count check ■■
completed, total, _ = self._count_completed_trades(ev, current_phase)
if stages:
    expected_completed = day_idx # Cells before the day placeholder should be filled
    if completed != expected_completed:
        msgs.append(
            f"■■ Cell count advisory: {completed} cells filled, "
            f"expected {expected_completed} before cell {day_idx + 1}")

return True, "\n".join(msgs)

def _get_current_phase_profit(self, ev, current_phase, broker_account=None, acct_size=None):
    """Get the current P/L for the active phase.

    Priority:
    1. Live broker equity (equity - starting balance) – most accurate
    2. Eval hedge result fields – fallback when broker not available

```



```

"""
# ■■ 1. Try live broker equity ■■
if broker_account and acct_size:
    try:
        stats = broker_account.get_account_stats()
        balance_str = stats.get("Balance", "N/A")
        if balance_str and balance_str != "N/A":
            cleaned = balance_str.replace("$", "").replace(",", "").strip()
            equity = float(cleaned)
            # Firm-aware starting balance. Critical for MFFU funded
            # accounts which start at $0 – using $50K here would make
            # the TP adjuster think we are -$50K down and inflate TP.
            starting = self._resolve_starting_balance(ev, current_phase, acct_size)
            live_profit = equity - starting
            if abs(live_profit) > 0.01:
                return live_profit
    except Exception:
        pass

# ■■ 2. Fallback: sum eval hedge result fields ■■
total = 0.0
fields = []
if current_phase == "Challenge":
    fields = [f"Hedge Result {i}" for i in range(1, 6)]
elif current_phase in ("Funded", "Payout 1", "Payout 2", "Payout 3", "Payout 4"):
    fields = [f"Hedge Result {i}.1" for i in range(1, 8)]
elif current_phase == "Farming":
    fields = [f"Hedge Day {i}" for i in range(1, 35)]
elif current_phase == "Double Dip":
    fields = [f"Hedge Result {i}.1" for i in range(1, 8)]

for f in fields:
    val = ev.get(f, None)
    if val is None or val == "" or val == "-":
        continue
    try:
        cleaned = str(val).replace("$", "").replace(",", "").strip()
        total += float(cleaned)
    except (ValueError, TypeError):
        continue
return total

def _get_next_phase(self, firm_code, current_display):
    """Get the next phase display name from trading_phases progression."""
    if not self.prop_firm_mgr:
        return "-"
    phases = self.prop_firm_mgr.get_prop_firm_trading_phases(firm_code)
    if not phases:
        return "-"

    # Match current_display to a phase in the list
    current_idx = -1
    for i, ph in enumerate(phases):
        if current_display.lower() in ph.lower():
            current_idx = i
            break

    if current_idx < 0:
        # Not found – guess by keyword
        for i, ph in enumerate(phases):

```

```

        if "challenge" in ph.lower() and "challenge" in current_display.lower():
            current_idx = i
            break
        if "fund" in ph.lower() and "fund" in current_display.lower():
            current_idx = i
            break
        if "farm" in ph.lower() and "farm" in current_display.lower():
            current_idx = i
            break

    if current_idx < 0 or current_idx >= len(phases) - 1:
        return "Complete ✓"

    return phases[current_idx + 1]

def _is_eval_active(self, ev):
    """Check if an evaluation is active (not failed/ended/deleted).

    Uses substring matching so 'Fail', 'Failed', 'Breached' etc. all caught.
    """
    p1 = (ev.get("Status P1", "") or "").strip().lower()
    funded = (ev.get("Status", "") or "").strip().lower()
    has_funded_acct = bool((ev.get("Account #.1", "") or "").strip())
    has_challenge_acct = bool((ev.get("Account #", "") or "").strip())

    p1_inactive = any(kw in p1 for kw in self._INACTIVE_KEYWORDS) if p1 else False
    funded_inactive = any(kw in funded for kw in (*self._INACTIVE_KEYWORDS,
        "complete")) if funded else False

    # If challenge failed (regardless of whether funded acct number exists)
    if p1_inactive and not has_funded_acct:
        return False
    # If funded status is failed/completed
    if funded_inactive:
        return False
    # If challenge failed AND funded also failed – both phases dead
    if p1_inactive and has_funded_acct and funded_inactive:
        return False
    # Must have at least one account number
    if not has_challenge_acct and not has_funded_acct:
        return False
    return True

def _refresh_eval_for_account(self, acct_num):
    """Fetch fresh eval data from dashboard for a specific account.

    Returns the updated eval dict, or None if fetch fails.
    When duplicate rows exist for the same account, prefers the active one.
    """
    try:
        email = self.client_email_entry.get().strip()
        dashboard_url = self.url_entry.get().strip().rstrip('/')
        if not email or not dashboard_url:
            return None
        r = requests.post(
            f"{dashboard_url}/api/client/data",
            json={"email": email},
            headers={"Content-Type": "application/json"},
            timeout=10
        )
    
```

```

        if r.status_code != 200:
            return None
        data = r.json()
        best = None
        for ev in data.get("evaluations", []):
            if ev.get("_deleted"):
                continue
            acct1 = (ev.get("Account #.1", "") or ev.get("Account #", "") or "").strip()
            acct0 = (ev.get("Account #", "") or "").strip()
            if acct_num in (acct1, acct0):
                is_active = ev.get("_is_active", self._is_eval_active(ev))
                if is_active:
                    return ev # Active match – return immediately
                if best is None:
                    best = ev # Keep first inactive as fallback
        return best
    except Exception:
        pass
    return None

def _load_active_trades(self):
    """Fetch evaluations from dashboard and populate the active trades list."""
    email = self.client_email_entry.get().strip()
    dashboard_url = self.url_entry.get().strip().rstrip('/')

    if not email:
        messagebox.showerror("Error", "Go to Dashboard tab, enter client email and click Lookup first")
        return

    self.log("Loading active trades from dashboard...")
    self.load_trades_btn.configure(state='disabled')
    self.trades_count_var.set("Loading...")

    def _do_load():
        try:
            r = requests.post(
                f"{dashboard_url}/api/client/data",
                json={"email": email},
                headers={"Content-Type": "application/json"},
                timeout=15
            )
            if r.status_code != 200:
                msg = "Unknown error"
                try:
                    msg = r.json().get("message", msg)
                except Exception:
                    pass
                self.root.after(0, lambda m=msg: self.log(f"Failed to load trades: {m}", "ERROR"))
                self.root.after(0, lambda: self.trades_count_var.set("Load failed"))
                return
            data = r.json()
            evaluations = data.get("evaluations", [])

            # Filter using dashboard's _is_active flag (source of truth)
            # Falls back to local _is_eval_active() if flag missing
            active_evals = []
            skipped_count = 0
            for ev in evaluations:
                if ev.get("_deleted"):
                    skipped_count += 1

```

```

        continue

    # Use dashboard-computed flag if available, else local fallback
    if "_is_active" in ev:
        is_active = ev["_is_active"]
    else:
        is_active = self._is_eval_active(ev)

    if not is_active:
        skipped_count += 1
        continue

    # Must have at least one account number
    if not (ev.get("Account #", "") or "").strip() and not (ev.get("Account #.1",
        "") or "").strip():
        skipped_count += 1
        continue

    active_evals.append(ev)

self.root.after(0, lambda t=len(evaluations), a=len(active_evals), s=skipped_count:
    self.log(
        f"■ {t} total evaluations → {a} active, {s} filtered out (failed/completed/deleted)")

self.root.after(0, lambda ae=active_evals: self._populate_trade_rows(ae))

# Populate broker connection rows per prop firm
prop_accounts = data.get("prop_accounts", [])
self.root.after(0, lambda ae=active_evals, pa=prop_accounts: self._populate_broker_rows(ae,
    pa))

# Auto-launch browsers for prop firms that need dashboard monitoring
active_firms = list(dict.fromkeys(
    ev.get("Prop Firm", "") for ev in active_evals if ev.get("Prop Firm")))
if not RELEASE_DISABLE_PROP_DASHBOARD_ACCESS:
    self.root.after(2000, lambda af=active_firms: self._auto_launch_propfirm_browsers(af))

# Trigger a status poll immediately after scan
if not RELEASE_DISABLE_STATUS_POLL:
    try:
        self._poll_tradovate_balances()
    except Exception:
        pass

# Auto-fill MT5 credentials from TradeOps dashboard.
# Prefer Hedge Accounts Configuration rows (what users actually edit in the UI),
# then fall back to legacy mt5_credentials if present.
mt5_login = ""
mt5_pass = ""
mt5_server = ""

hedge_accounts = data.get("hedge_accounts") or []
client_name = ((data.get("identity") or {}).get("client") or "").strip().lower()
chosen_hedge = None
for hedge in hedge_accounts:
    if str(hedge.get("platform") or "").strip().upper() != "MT5":
        continue
    if not ((hedge.get("login") or "").strip() and (hedge.get("password") or "").strip())
        hedge.get("server") or "").strip():
        continue

```

```

        hedge_name = str(hedge.get("name") or "").strip().lower()
        if client_name and hedge_name == client_name:
            chosen_hedge = hedge
            break
        if chosen_hedge is None:
            chosen_hedge = hedge

    if chosen_hedge:
        mt5_login = (chosen_hedge.get("login") or "").strip()
        mt5_pass = (chosen_hedge.get("password") or "").strip()
        mt5_server = (chosen_hedge.get("server") or "").strip()
    else:
        mt5_creds = data.get("mt5_credentials") or {}
        mt5_login = (mt5_creds.get("login") or "").strip()
        mt5_pass = (mt5_creds.get("password") or "").strip()
        mt5_server = (mt5_creds.get("server") or "").strip()

    if mt5_login and mt5_pass and mt5_server:
        def _fill_mt5(login=mt5_login, pwd=mt5_pass, srv=mt5_server):
            # Only fill if fields are currently empty
            if not self.mt5_login.get().strip():
                self.mt5_login.delete(0, tk.END)
                self.mt5_login.insert(0, login)
            if not self.mt5_password.get().strip():
                self.mt5_password.delete(0, tk.END)
                self.mt5_password.insert(0, pwd)
            if not self.mt5_server.get().strip():
                self.mt5_server.delete(0, tk.END)
                self.mt5_server.insert(0, srv)
            self.log("■ MT5 credentials auto-filled from TradeOps dashboard")
        self.root.after(0, _fill_mt5)

    except Exception as e:
        self.root.after(0, lambda: self.log(f"Load trades failed: {e}", "ERROR"))
        self.root.after(0, lambda: self.trades_count_var.set("Load failed"))
    finally:
        self.root.after(0, lambda: self.load_trades_btn.configure(state='normal'))

    threading.Thread(target=_do_load, daemon=True).start()

# Futuristic phase badge palette (border_color, bg, fg)
_PHASE_GLOW = {
    "Challenge": ("F59E0B", "1A1000", "FBBF24"),
    "Funded":    ("22C55E", "001A0A", "4ADE80"),
    "Farming":   ("3B82F6", "001030", "60A5FA"),
}

def _populate_trade_rows(self, evaluations):
    """Clear and rebuild the active trade rows – futuristic terminal style."""
    for child in self._trades_inner.wininfo_children():
        child.destroy()
    self._active_trade_rows.clear()

    if not evaluations:
        if CTK_AVAILABLE:
            empty = ctk.CTkFrame(self._trades_inner, fg_color="transparent")
            empty.pack(fill="both", expand=True, pady=40)
            ctk.CTkLabel(empty, text="■", font=("Consolas", 28),
                        text_color="#0F4C75").pack()
            ctk.CTkLabel(empty, text="NO ACTIVE TRADES DETECTED",

```

```

        font=("Consolas", 11, "bold"),
        text_color="#1B4965").pack(pady=(4, 2))
    ctk.CTkLabel(empty, text="Connect and verify access to populate",
        font=("Consolas", 9),
        text_color="#0F3460").pack()
else:
    tk.Label(self._trades_inner, text="No active trades found",
        fg='#94a3b8', bg='#020A14', font=('Segoe UI', 10, 'italic')).pack(pady=20)
self.trades_count_var.set("[ 0 ]")
return

# Daily bias per prop firm (persisted, resets each day)
firms_seen = set()
for ev in evaluations:
    firms_seen.add(ev.get("Prop Firm", "Unknown"))
    firm_bias = self._get_daily_bias(firms_seen)
# Store for auto-trade compatibility
self._auto_trade_firm_sides = firm_bias

# Log bias
bias_parts = []
for f, s in sorted(firm_bias.items()):
    arrow = "▲" if s == "buy" else "▼"
    bias_parts.append(f"{arrow} {f}: {s.upper()}")
self.log(f"Direction bias: {' '.join(bias_parts)}")

for idx, ev in enumerate(evaluations):
    prop_firm_name = ev.get("Prop Firm", "Unknown")
    firm_code = self._FIRM_MAP.get(prop_firm_name, "MFFU_Flex")
    acct_num = (ev.get("Account #.1", "") or ev.get("Account #", "") or "-").strip()
    acct_size = ev.get("Account Size", "-") or "-"
    current_display, phase_key = self._detect_eval_phase(ev)

    # Resolve phase_key from day placeholder (primary source of truth)
    resolved_key, _di, _dn = self._resolve_phase_key_from_day(ev, firm_code, current_display)
    if resolved_key:
        phase_key = resolved_key

    next_display = self._get_next_phase(firm_code, current_display)

    strip_color = self.PROP_FIRM_COLORS.get(prop_firm_name, "#95A5A6")
    glow_border, glow_bg, glow_fg = self._PHASE_GLOW.get(
        current_display, ("#475569", "#0A0F1A", "#94A3B8"))

    bias = firm_bias.get(prop_firm_name, "buy")

    if CTK_AVAILABLE:
        row_bg = "#050D18" if idx % 2 == 0 else "#071020"
        row_frame = ctk.CTkFrame(self._trades_inner, fg_color=row_bg,
            corner_radius=4, height=44,
            border_width=1, border_color="#0A1628")
        row_frame.pack(fill="x", pady=1, padx=1)
        row_frame.pack_propagate(False)

        # Left accent bar
        ctk.CTkFrame(row_frame, width=3, fg_color=strip_color,
            corner_radius=0).pack(side="left", fill="y")

        # Prop firm name – aligned with header col
        ctk.CTkLabel(row_frame, text=prop_firm_name[:18], width=110,

```

```

        font=("Consolas", 10, "bold"), text_color=strip_color,
        anchor="w").pack(side="left", padx=(8, 0))

# Account number
acct_display = acct_num[-8:] if len(acct_num) > 8 else acct_num
ctk.CTkLabel(row_frame, text=acct_display,
            width=88, font=("Consolas", 10),
            text_color="#6B8DAD", anchor="w").pack(side="left", padx=(8, 0))

# Size
ctk.CTkLabel(row_frame, text=acct_size[:10], width=68,
            font=("Consolas", 10), text_color="#4A7C8F",
            anchor="w").pack(side="left", padx=(8, 0))

# Phase badge – neon pill with border glow
phase_holder = ctk.CTkFrame(row_frame, fg_color="transparent", width=88)
phase_holder.pack(side="left", padx=(8, 0))
phase_holder.pack_propagate(False)
phase_pill = ctk.CTkFrame(phase_holder, fg_color=glow_bg,
                        corner_radius=8, border_width=1,
                        border_color=glow_border)
phase_pill.pack(side="left", pady=6)
ctk.CTkLabel(phase_pill, text=current_display.upper(),
            font=("Consolas", 8, "bold"),
            text_color=glow_fg).pack(padx=8, pady=1)

# Next phase with arrow
ctk.CTkLabel(row_frame, text=f"→ {next_display}", width=100,
            font=("Consolas", 9), text_color="#00D4FF",
            anchor="w").pack(side="left", padx=(8, 0))

# BUY / SELL action buttons – bias-aware
btn_frame = ctk.CTkFrame(row_frame, fg_color="transparent")
btn_frame.pack(side="right", padx=8)

row_data = {
    "frame": row_frame, "eval": ev, "firm_code": firm_code,
    "phase_key": phase_key, "acct_size": acct_size,
    "acct_num": acct_num, "current_phase": current_display,
}

# Active button gets neon glow, opposite gets muted/grayed
if bias == "buy":
    buy_fg, buy_brd, buy_txt = "#052E16", "#16A34A", "#4ADE80"
    sell_fg, sell_brd, sell_txt = "#0A0F1A", "#1A1A2E", "#2A3040"
else:
    buy_fg, buy_brd, buy_txt = "#0A0F1A", "#1A1A2E", "#2A3040"
    sell_fg, sell_brd, sell_txt = "#2D0A0A", "#DC2626", "#F87171"

buy_btn = ctk.CTkButton(btn_frame, text="▲ BUY", width=58, height=26,
                        fg_color=buy_fg, hover_color="#14532D" if bias == "buy" else "#0A0F1A",
                        border_width=1, border_color=buy_brd,
                        font=("Consolas", 9, "bold"),
                        text_color=buy_txt, corner_radius=4,
                        command=lambda rd=row_data: self._execute_row_trade("buy", rd))
buy_btn.pack(side="left", padx=(0, 3))

sell_btn = ctk.CTkButton(btn_frame, text="▼ SELL", width=58, height=26,
                        fg_color=sell_fg, hover_color="#450A0A" if bias == "sell" else "#2D0A0A",
                        border_width=1, border_color=sell_brd,

```

```

        font=("Consolas", 9, "bold"),
        text_color=sell_txt, corner_radius=4,
        command=lambda rd=row_data: self._execute_row_trade("sell", rd))
    sell_btn.pack(side="left")
else:
    # Fallback plain tk
    row_bg = '#050D18' if idx % 2 == 0 else '#071020'
    row_frame = tk.Frame(self._trades_inner, bg=row_bg)
    row_frame.pack(fill="x", pady=1)

    tk.Label(row_frame, text=prop_firm_name[:16], width=14, anchor='w',
             bg=row_bg, fg='#e2e8f0', font=('Consolas', 9)).pack(side="left", padx=2)
    tk.Label(row_frame, text=acct_num[:12], width=10, anchor='w',
             bg=row_bg, fg='#6B8DAD', font=('Consolas', 9)).pack(side="left", padx=2)
    tk.Label(row_frame, text=acct_size[:10], width=8, anchor='w',
             bg=row_bg, fg='#4A7C8F', font=('Consolas', 9)).pack(side="left", padx=2)
    tk.Label(row_frame, text=current_display, width=14, anchor='w',
             bg=row_bg, fg='#fbbf24', font=('Consolas', 9, 'bold')).pack(side="left", padx=2)
    tk.Label(row_frame, text=f"→ {next_display}", width=14, anchor='w',
             bg=row_bg, fg='#00D4FF', font=('Consolas', 9)).pack(side="left", padx=2)

    btn_frame = tk.Frame(row_frame, bg=row_bg)
    btn_frame.pack(side="left", padx=4)

    row_data = {
        "frame": row_frame, "eval": ev, "firm_code": firm_code,
        "phase_key": phase_key, "acct_size": acct_size,
        "acct_num": acct_num, "current_phase": current_display,
    }

    buy_btn = tk.Button(btn_frame, text="▲ BUY",
                       bg='#052E16' if bias == 'buy' else '#0A0F1A',
                       fg='#4ADE80' if bias == 'buy' else '#2A3040',
                       font=('Consolas', 8, 'bold'), relief='flat', padx=6, pady=1,
                       command=lambda rd=row_data: self._execute_row_trade("buy", rd))
    buy_btn.pack(side="left", padx=(0, 4))

    sell_btn = tk.Button(btn_frame, text="▼ SELL",
                       bg='#2D0A0A' if bias == 'sell' else '#0A0F1A',
                       fg='#F87171' if bias == 'sell' else '#2A3040',
                       font=('Consolas', 8, 'bold'), relief='flat', padx=6, pady=1,
                       command=lambda rd=row_data: self._execute_row_trade("sell", rd))
    sell_btn.pack(side="left")

    row_data["buy_btn"] = buy_btn
    row_data["sell_btn"] = sell_btn
    self._active_trade_rows.append(row_data)

count = len(self._active_trade_rows)
self.trades_count_var.set(f"[ {count} ]")
# Update stats strip
try:
    self._stat_queue_var.set(f"Queue: {count}")
except Exception:
    pass
self.log(f"Loaded {count} active trades from dashboard")

def _eval_has_payout(self, ev):
    """Check if any hedge result field contains 'payout' text."""
    if not ev:

```



```

        return False
    for key, val in ev.items():
        if "hedge result" in key.lower() and isinstance(val, str) and "payout" in val.lower():
            return True
    return False

def _execute_row_trade(self, side, row_data):
    """Execute a trade for a specific row, then remove the row."""
    # Check for payout – skip account if any hedge result has payout text
    ev = row_data.get("eval", {})
    if self._eval_has_payout(ev):
        acct_num = row_data.get("acct_num", "?")
        messagebox.showwarning("Payout Pending",
                               f"Account {acct_num} has a PAYOUT pending.\n"
                               f"Request payout first before continuing.")
        return

    # Direction filter
    direction = self.direction_var.get()
    if direction == "Buy Only" and side == "sell":
        messagebox.showwarning("Direction Lock", "Direction is set to Buy Only")
        return
    if direction == "Sell Only" and side == "buy":
        messagebox.showwarning("Direction Lock", "Direction is set to Sell Only")
        return

    firm_code = row_data["firm_code"]
    phase_key = row_data["phase_key"]
    acct_size = row_data["acct_size"]
    acct_num = row_data["acct_num"]

    # ■■■ Resolve phase_key from day placeholder (primary source of truth) ■■■
    fresh_ev = self._refresh_eval_for_account(acct_num)
    if fresh_ev:
        ev = fresh_ev
        row_data["eval"] = fresh_ev
    resolved_key, day_idx, day_name = self._resolve_phase_key_from_day(
        ev, firm_code, row_data["current_phase"])
    if resolved_key is None:
        messagebox.showwarning("No Day Placeholder",
                               f"Account {acct_num}: No day placeholder found.\n"
                               f"Enter a day name (MON/TUE/etc.) in the next cell to enable trading.")
        return
    if resolved_key != phase_key:
        self.log(f"■ {acct_num}: Day cell {day_idx + 1} ({day_name}) → "
                f"blueprint {resolved_key} (was {phase_key})")
        phase_key = resolved_key

    # Get trade config from blueprint
    config = None
    if self.prop_firm_mgr:
        config = self.prop_firm_mgr.get_strategy_config(firm_code, phase_key, acct_size)
    if not config:
        messagebox.showerror("Error", f"No blueprint config for {firm_code} / {phase_key} / {acct_size}")
        return

    hedging = self.hedge_mode_var.get() == "Hedging"
    prop_firm_name = row_data["eval"].get("Prop Firm", firm_code) if row_data.get("eval") else firm_code
    # Auto-detect platform: TopStep firms always use TopStepX (Selenium)
    if "topstep" in prop_firm_name.lower():

```

```

        platform = "TopStepX"
    else:
        platform = self.broker_var.get()
    broker_account = self._get_broker_for_firm(prop_firm_name)

    if not broker_account:
        messagebox.showerror("Error", f"Connect broker for {prop_firm_name} first")
        return

    mt5_api = None
    if hedging:
        mt5_api = self._get_mt5_trading_api()
        if not mt5_api:
            messagebox.showerror("Error", "Connect MT5 for hedging mode")
            return

    trado_sym = config.get("tradovate_symbol", "") or config.get("topstepx_symbol", "")
    trado_qty = int(config.get("tradovate_qty", 2) or config.get("topstepx_qty", 2))

    # ■■ Farming: cap MT5 TP based on hard-stop proximity ■■
    ev = row_data.get("eval", {})
    _is_farming_sym = "MNQ" in (config.get("tradovate_symbol", "") or config.get("topstepx_symbol",
        "")).upper()
    if _is_farming_sym and self.prop_firm_mgr:
        try:
            _bal = None
            if broker_account:
                _stats = broker_account.get_account_stats()
                _bal_str = _stats.get("Balance", "")
                if _bal_str and _bal_str not in ("N/A", "Error", ""):
                    _bal = float(_bal_str.replace("$", "").replace(",", ""))
            if _bal is not None:
                orig_mt5_tp = int(config.get("mt5_tp_points", 0))
                config = self.prop_firm_mgr.adjust_farming_tp_sl(config, _bal, firm_code)
                new_mt5_tp = int(config.get("mt5_tp_points", 0))
                if new_mt5_tp != orig_mt5_tp:
                    self.log(
                        f"■ Farming TP cap {acct_num}: balance=${_bal:,.2f} → MT5 TP {orig_mt5_tp}→
                    else:
                        self.log(f"■ Farming TP OK {acct_num}: MT5 TP {orig_mt5_tp} pts within safe rang
                else:
                    self.log(f"■ Farming {acct_num}: could not read balance – using blueprint TP/SL")
            except Exception as _fe:
                self.log(f"■ Farming TP check failed for {acct_num}: {_fe}")
        # TP/SL comes directly from the stage blueprint (selected by day placeholder)

    trado_tp = int(config.get("tradovate_tp_ticks", 151) or config.get("topstepx_tp_ticks", 151))
    trado_sl = int(config.get("tradovate_sl_ticks", 200) or config.get("topstepx_sl_ticks", 200))
    blueprint_tp_orig = trado_tp # save originals for confirm dialog
    blueprint_sl_orig = trado_sl
    _adj_reasons = [] # collect adjustment explanations for confirm dialog
    mt5_sym = config.get("mt5_symbol", "NAS100")
    mt5_vol = float(config.get("mt5_volume", 2.8))
    mt5_tp = int(config.get("mt5_tp_points", 46))
    mt5_sl = int(config.get("mt5_sl_points", 42))

    # ■■ Adjust TP based on stage progress ■■
    # If we already have profit in this stage, shrink TP so we don't overshoot.
    # If we're short of the expected start balance, grow TP to catch up.
    if self.prop_firm_mgr and broker_account and not _is_farming_sym:

```

```

try:
    current_profit = self._get_current_phase_profit(
        ev, row_data["current_phase"], broker_account=broker_account, acct_size=acct_size)
    size_key = self.prop_firm_mgr.convert_account_size_to_key(acct_size)
    stage_start = self.prop_firm_mgr.get_stage_start_target(
        firm_code, row_data["current_phase"], phase_key, size_key)
    stage_profit_so_far = current_profit - stage_start
    trado_sym_for_calc = config.get("tradovate_symbol", "") or config.get("topstepx_symbol",
    tick_val = self.prop_firm_mgr.get_tick_value(
        trado_sym_for_calc) if self.prop_firm_mgr else 5.0
    trado_qty_for_calc = int(config.get("tradovate_qty", 1) or config.get("topstepx_qty", 1))
    if tick_val > 0 and trado_qty_for_calc > 0:
        orig_tp = trado_tp
        orig_mt5_sl = mt5_sl
        # Convert stage profit to ticks and subtract from TP
        profit_ticks = stage_profit_so_far / (tick_val * trado_qty_for_calc)
        adjusted_tp = max(5, round(trado_tp - profit_ticks))
        tp_ratio = adjusted_tp / trado_tp if trado_tp > 0 else 1.0
        adjusted_mt5_sl = max(5, round(mt5_sl * tp_ratio))
        if adjusted_tp != trado_tp:
            trado_tp = adjusted_tp
            mt5_sl = adjusted_mt5_sl
            _adj_reasons.append(
                f"TP {blueprint_tp_orig}→{trado_tp}t: stage P/L ${stage_profit_so_far:+,.0f}
                f"(already earned in this stage)")
            self.log(f"■ TP adjust {acct_num}: stage_start=${stage_start:+,.0f}, "
                f"current P/L=${current_profit:+,.2f}, stage P/L=${stage_profit_so_far:+,
                f"TP {orig_tp}→{trado_tp}t, MT5 SL {orig_mt5_sl}→{mt5_sl}pts")
except Exception as _te:
    self.log(f"■ TP adjust failed for {acct_num}: {_te}")

# ■ Adjust SL based on midnight balance + drawdown protection ■
if broker_account and platform == "Tradovate" and hasattr(broker_account, 'get_min_equity'):
    try:
        min_eq_data = broker_account.get_min_equity()
        if min_eq_data:
            live_net_liq = min_eq_data['net_liq']
            net_liq_sod = min_eq_data.get('net_liq_sod', 0)
            live_min_equity = min_eq_data.get('min_equity', 0)
            tmdl = min_eq_data.get('trailing_max_drawdown_limit', 50000)
            trado_qty_for_calc = int(config.get("tradovate_qty", 1) or config.get("topstepx_qty",
            trado_sym_for_calc = config.get("tradovate_symbol", "") or config.get("topstepx_symbol",
            tick_val = self.prop_firm_mgr.get_tick_value(
                trado_sym_for_calc) if self.prop_firm_mgr else 5.0

        # Step 1: Midnight balance SL – floor = SOD - blueprint_sl_dollars
        if net_liq_sod > 0 and tick_val > 0 and trado_qty_for_calc > 0:
            blueprint_sl_dollars = trado_sl * tick_val * trado_qty_for_calc
            sl_floor = net_liq_sod - blueprint_sl_dollars
            available = live_net_liq - sl_floor
            if available > 0:
                midnight_sl = max(10, int(available / (tick_val * trado_qty_for_calc)))
                if midnight_sl != trado_sl:
                    orig_sl = trado_sl
                    orig_mt5_tp = mt5_tp
                    trado_sl = midnight_sl
                    mt5_tp = max(5, int(trado_sl / 4) - 1)
                    daily_pnl = live_net_liq - net_liq_sod
                    _adj_reasons.append(

```

```

        f"SL {blueprint_sl_orig}→{trado_sl}t: midnight bal ${net_liq_sod:,.0f}"
        f"daily P/L ${daily_pnl:+,.0f}")
    self.log(f"■ Midnight SL {acct_num}: SOD=${net_liq_sod:,.2f}, "
            f"live=${live_net_liq:,.2f}, daily P/L=${daily_pnl:+,.2f} → "
            f"SL {orig_sl}→{trado_sl}t, MT5 TP {orig_mt5_tp}→{mt5_tp}pts")
    else:
        self.log(
            f"■ Midnight SL OK {acct_num}: SOD=${net_liq_sod:,.2f}, SL={trado_sl}")
    else:
        self.log(f"■ Midnight SL floor breached {acct_num}: "
                f"live=${live_net_liq:,.2f} < floor=${sl_floor:,.2f} – using min SL")
        _adj_reasons.append(
            f"SL → 10t: balance below midnight SL floor ${sl_floor:,.0f}")
        trado_sl = 10
        mt5_tp = max(5, int(trado_sl / 4) - 1)

    # Step 2: TMDL drawdown cap – further tighten if near breach
    if live_min_equity > 0 and live_net_liq < tmdl:
        drawdown_remaining = live_net_liq - live_min_equity
        current_sl_risk = trado_sl * tick_val * trado_qty_for_calc
        if drawdown_remaining > 0 and current_sl_risk > drawdown_remaining:
            orig_sl = trado_sl
            orig_mt5_tp = mt5_tp
            trado_sl = max(10, int(drawdown_remaining / (tick_val * trado_qty_for_calc)))
            mt5_tp = max(5, int(trado_sl / 4) - 1)
            _adj_reasons.append(
                f"SL →{trado_sl}t: near drawdown limit, only ${drawdown_remaining:,.0f}")
            self.log(f"■ TMDL SL cap {acct_num}: remaining=${drawdown_remaining:,.2f} → "
                    f"SL {orig_sl}→{trado_sl}t, MT5 TP {orig_mt5_tp}→{mt5_tp}pts")
        elif drawdown_remaining <= 0:
            self.log(f"■ No drawdown room for {acct_num} (${drawdown_remaining:,.2f})")
        elif live_min_equity > 0:
            self.log(f"■ TMDL OK {acct_num}: ${live_net_liq:,.2f} ≥ TMDL=${tmdl:,.0f}")
    except Exception:
        pass

    hedge_text = f" + MT5 {'SELL' if side == 'buy' else 'BUY'}) {mt5_vol} {mt5_sym}" if hedging else ""

    # Stage info from day placeholder
    stage_text = (f"\n\n■ Stage Info ■"
                 f"\nDay Cell: {day_idx + 1} ({day_name}) → Blueprint: {phase_key}")

    # Adjustment explanations
    if _adj_reasons:
        adj_text = "\n\n■ Adjustments ■\n" + "\n".join(f"• {r}" for r in _adj_reasons)
        adj_text += f"\n(Blueprint was TP {blueprint_tp_orig}t / SL {blueprint_sl_orig}t)"
    else:
        adj_text = ""

    confirm = messagebox.askyesno("Confirm Trade",
        f"{side.upper()} {trado_qty} {trado_sym} on {platform}\n"
        f"{hedge_text}\n\n"
        f"Account: {acct_num} | {firm_code}\n"
        f"Phase: {row_data['current_phase']} | Size: {acct_size}\n"
        f"TP: {trado_tp} ticks | SL: {trado_sl} ticks"
        f"{stage_text}{adj_text}\n\nProceed?")
    if not confirm:
        return

    # Disable buttons immediately

```

```

row_data["buy_btn"].configure(state='disabled', text="...")
row_data["sell_btn"].configure(state='disabled', text="...")

prop_name = ev.get("Prop Firm", firm_code) if (ev := row_data.get("eval")) else firm_code
hedge_tag = f" ↔ MT5 {'SELL' if side == 'buy' else 'BUY'} {mt5_vol} {mt5_sym}" if hedging else ""
self.log(
    f"■ {side.upper()} {trado_qty} {trado_sym} → {prop_name} {acct_num} [{row_data['current_phase']}]")

def _do_trade():
    try:
        # 1. Broker order
        if platform == "Tradovate":
            if side == "buy":
                order_result = broker_account.buy_market(trado_sym, trado_qty, tp=trado_tp,
                    sl=trado_sl, expected_account=acct_num)
            else:
                order_result = broker_account.sell_market(trado_sym, trado_qty, tp=trado_tp,
                    sl=trado_sl, expected_account=acct_num)
        elif platform == "TopStepX":
            # Switch to the correct account if multiple accounts under same login
            if hasattr(broker_account, 'switch_account') and acct_num:
                switched = broker_account.switch_account(account_name_contains=acct_num)
                if not switched:
                    raise Exception(f"TopStepX could not switch to account {acct_num}")
            # TopStepX uses two-digit year futures codes (NQM26, MNQM26)
            _tsx_sym = _to_topstepx_symbol(trado_sym)
            # Convert ticks to dollars for TopStepX: dollars = ticks * tick_value * quantity
            _tsx_tick_val = self.prop_firm_mgr.get_tick_value(_tsx_sym) if self.prop_firm_mgr else 1
            _tsx_tp_dollars = trado_tp * _tsx_tick_val * trado_qty
            _tsx_sl_dollars = trado_sl * _tsx_tick_val * trado_qty
            if side == "buy":
                order_result = broker_account.place_buy_order(_tsx_sym, trado_qty,
                    tp_dollars=_tsx_tp_dollars, sl_dollars=_tsx_sl_dollars)
            else:
                order_result = broker_account.place_sell_order(_tsx_sym, trado_qty,
                    tp_dollars=_tsx_tp_dollars, sl_dollars=_tsx_sl_dollars)

        # Some broker implementations return a status dict instead of raising.
        # Normalize failures so UI/logs don't report false fills.
        if isinstance(order_result, dict) and order_result.get("success") is False:
            raise Exception(order_result.get("message") or "Broker reported unsuccessful order")

        self.log(
            f"■ {platform} filled {side.upper()} {trado_qty} {trado_sym} | TP:{trado_tp} SL:{trado_sl}")

        # 2. MT5 hedge (opposite direction)
        if hedging and mt5_api:
            hedge_side = "sell" if side == "buy" else "buy"
            comment = f"{acct_num}_{phase_key}"
            if hedge_side == "buy":
                mt5_api.buy_market(mt5_sym, mt5_vol, sl=mt5_sl, tp=mt5_tp, comment=comment)
            else:
                mt5_api.sell_market(mt5_sym, mt5_vol, sl=mt5_sl, tp=mt5_tp, comment=comment)
            self.log(
                f"■ MT5 hedge {hedge_side.upper()} {mt5_vol} {mt5_sym} TP:{mt5_tp} SL:{mt5_sl} c")

        # ■■ Auto-status: set "In Progress" when trade goes out ■■
        _ev = row_data.get("eval")
        if _ev:
            _has_funded = bool((_ev.get("Account #.1") or "").strip())

```

```

        _sf = "Status" if _has_funded else "Status P1"
        _cur = (_ev.get(_sf) or "").strip().lower()
        if not _cur or _cur in ("not started", "in progress", ""):
            _ev[_sf] = "In Progress"
            self.log(f"■ Auto-status: {acct_num} → {_sf}='In Progress'")

# Remove row from list
def _remove():
    row_data["frame"].destroy()
    if row_data in self._active_trade_rows:
        self._active_trade_rows.remove(row_data)
    remaining = len(self._active_trade_rows)
    self.trades_count_var.set(
        f"{remaining} active trade{'s' if remaining != 1 else ''}"
        if remaining > 0 else "All trades complete ✓")
    try:
        self._stat_queue_var.set(f"Queue: {remaining}")
        # Increment trade count
        cur = self._stat_trades_var.get()
        n = int(cur.split(":")[1].strip()) + 1 if ":" in cur else 1
        self._stat_trades_var.set(f"Trades: {n}")
    except Exception:
        pass

self.root.after(0, _remove)

except Exception as e:
    self.log(f"■ Trade failed for {acct_num}: {e}", "ERROR")
    self.root.after(0, lambda: messagebox.showerror("Trade Error", str(e)))
    # Re-enable buttons on failure
    self.root.after(0, lambda: row_data["buy_btn"].configure(state='normal', text="▲ BUY"))
    self.root.after(0, lambda: row_data["sell_btn"].configure(state='normal', text="▼ SELL"))

threading.Thread(target=_do_trade, daemon=True).start()

# ■■ Auto-Trade Scheduler Logic ■■

def _toggle_auto_trade(self):
    """Toggle the auto-trade scheduler on/off."""
    if self.auto_trade_enabled:
        self._stop_auto_trade()
    else:
        self._start_auto_trade()

# ■■ Hedge Protector ████████████████████████████████████████████████████████████

def _test_min_equity(self, event=None):
    """Test: dump min equity data from cached prop firm mappings and Tradovate API."""
    self.log(f"■ Testing min equity sources...")
    # 1. Cached prop firm mappings
    if self._cached_acct_mappings:
        for firm_name, mapping in self._cached_acct_mappings.items():
            self.log(f" --- {firm_name} (cached mapping) ---")
            for key, info in mapping.items():
                me = info.get("min_equity")
                pt = info.get("profit_target")
                bal = info.get("balance")
                start = info.get("starting_balance")
                self.log(f"      {key}: bal=${bal}, start=${start}, min_eq=${me}, target=${pt}")
    else:
```

```

        self.log(" ■ No cached account mappings (connect prop firm dashboards first)")
# 2. Tradovate API fallback
found = False
for firm_name, conn in self._broker_connections.items():
    acct = conn.get("account")
    if not acct or not hasattr(acct, 'get_min_equity'):
        continue
    found = True
    self.log(f" --- {firm_name} (Tradovate API) ---")
    try:
        result = acct.get_min_equity()
        if result:
            self.log(f"      Net Liq:           ${result['net_liq']:, .2f}")
            self.log(f"      Min Equity:        ${result['min_equity']:, .2f}")
            self.log(f"      Drawdown Remaining: ${result['drawdown_remaining']:, .2f}")
            self.log(f"      Max Net Liq:       ${result.get('max_net_liq', 0):, .2f}")
            self.log(f"      Trailing Max DD:   ${result['trailing_max_drawdown']:, .2f}")
            self.log(f"      Trailing DD Limit: ${result['trailing_max_drawdown_limit']:, .2f}")
            self.log(f"      Mode:              {result.get('trailing_mode', '?')}")
        else:
            self.log(f"      ■ get_min_equity returned None")
    except Exception as e:
        self.log(f"      ■ Error: {e}", "ERROR")
if not found:
    self.log(" ■ No connected Tradovate brokers")
self.log("■ Min equity test complete.")

def _start_hedge_protector(self):
    """Start (or restart) the Hedge Protector engine. Called automatically on broker connect."""
    if RELEASE_DISABLE_HEDGE_GUARD:
        return

    if not HEDGE_PROTECTOR_AVAILABLE:
        return

    # If already running, stop first so we can pick up newly connected accounts
    if self._hedge_protector and self._hedge_protector.is_running:
        try:
            self._hedge_protector.stop()
        except Exception:
            pass
        self._hedge_protector = None

    # Gather every connected Tradovate account
    connected_tv = {}
    for firm_name, conn in self._broker_connections.items():
        acct = conn.get("account")
        if acct and hasattr(acct, '_api_fetch'):
            connected_tv[firm_name] = acct

    if not connected_tv:
        return # nothing to guard yet

    # Get MT5 API
    mt5_api = self._get_mt5_trading_api() if hasattr(self, '_get_mt5_trading_api') else None

    def on_event(event_type, message):
        """Route HedgeProtector events to the UI."""
        try:
            kind = {

```

```

        "info": "info", "warn": "queue", "error": "error",
        "sl_detected": "error", "tp_detected": "success",
        "close_sent": "trade",
    }.get(event_type, "info")
    self.root.after(0, lambda: self._add_activity(f"■■ {message}", kind))
    self.root.after(0, lambda: self.log(f"■■ [{event_type.upper()}] {message}"))
except Exception:
    pass

def on_status_change(acct_name, close_reason, phase_key):
    """Immediately update dashboard status when TP/SL detected."""
    self.root.after(0, lambda: self._handle_hedge_status_change(acct_name, close_reason, phase_key))

self._hedge_protector = HedgeProtector(
    mt5_api=mt5_api,
    tradovate_accounts=connected_tv,
    on_event=on_event,
    on_status_change=on_status_change,
)
self._hedge_protector.start()

self.log(f"■■ Hedge Guard ACTIVE – monitoring {len(connected_tv)} Tradovate account(s)")

def _stop_hedge_protector(self):
    """Stop the Hedge Protector engine."""
    if self._hedge_protector:
        stats = self._hedge_protector.get_status()
        self._hedge_protector.stop()
        self._hedge_protector = None
        self.log(
            f"■■ Hedge Guard stopped – SL protected: {stats.get('sl_protected', 0)}, TP passed: {sta

def _handle_hedge_status_change(self, acct_name, close_reason, phase_key):
    """Update dashboard status immediately when TP/SL detected by hedge protector.

    Args:
        acct_name: Tradovate account name (e.g. "FNFTCHHARRISON0UKA85625")
        close_reason: "tp_detected" or "sl_detected"
        phase_key: Blueprint stage key from MT5 comment (e.g. "challenge_trade2", "funded_trade1")
    """
    if RELEASE_DISABLE_AUTO_STATUS_UPDATES:
        return

import re

# 1. Find the matching eval from active trade rows
matched_ev = None
matched_rd = None
acct_lower = acct_name.lower()
for rd in self._active_trade_rows:
    ev = rd.get("eval")
    if not ev:
        continue
    acct_ch = (ev.get("Account #") or "").strip().lower()
    acct_fd = (ev.get("Account #.1") or "").strip().lower()
    if acct_lower in acct_ch or acct_ch in acct_lower or \
        acct_lower in acct_fd or acct_fd in acct_lower:
        matched_ev = ev
        matched_rd = rd
        break

```



```

if not matched_ev:
    self.log(f"■ Hedge status: no eval found for {acct_name} – skipping status update")
    return

# 2. Determine which status field to update
has_funded = bool((matched_ev.get("Account #.1") or "").strip())
status_field = "Status" if has_funded else "Status P1"

# 3. Extract trade number from phase_key → status value
# phase_key examples: "challenge_trade1", "funded_trade2", "funded_trade_doubledip_3"
# Dashboard TP statuses: "Hit TP1", "Hit TP2", "Hit TP3", "Hit TP4"
# Dashboard SL statuses: "Fail"
if close_reason == "sl_detected":
    new_status = "Fail"
else:
    # Extract the trade number from the phase_key
    m = re.search(r'(\d+)\$', phase_key)
    trade_num = int(m.group(1)) if m else 1
    new_status = f"Hit TP{trade_num}"

current_status = (matched_ev.get(status_field) or "").strip()
if current_status.lower() == new_status.lower():
    self.log(f"■ Hedge status: {acct_name} already {new_status}")
    return

# 4. Update the eval
matched_ev[status_field] = new_status
self.log(f"■ Hedge status: {acct_name} [{phase_key}] → {status_field}='{new_status}' "
        f"({'SL hit' if close_reason == 'sl_detected' else 'TP hit'})")
self._add_activity(
    f"■ {acct_name}: {new_status} ({phase_key})",
    "error" if close_reason == "sl_detected" else "success")

# 5. Push to dashboard immediately
email = self.client_email_entry.get().strip()
dashboard_url = self.url_entry.get().strip().rstrip('/')
if not email or not dashboard_url:
    self.log(f"■ Hedge status: no email/dashboard URL – skipping push")
    return
all_evals = [rd.get("eval") for rd in self._active_trade_rows if rd.get("eval")]
if not all_evals:
    return

def _push():
    try:
        import requests
        payload = {
            "email": email,
            "evaluations": all_evals,
            "statistics": {},
            "dropdown_options": {},
            "force_fields": [status_field],
        }
        resp = requests.post(
            f"{dashboard_url}/api/client/push",
            json=payload,
            headers={"Content-Type": "application/json"},
            timeout=15)
        if resp.status_code == 200 and resp.json().get("status") == "success":
            self.root.after(0, lambda:

```



```

        cancelled = acct.cancel_all_orders_api(account_id=aid)
        if cancelled > 0:
            self.root.after(0, lambda fn=firm_name, n=aname, c=cancelled:
                self.log(f" ■ {fn} – {n} cancelled {c} orphaned order(s)"))
        except Exception:
            pass
        continue
    self.root.after(0, lambda fn=firm_name, n=aname, cnt=len(open_pos):
        self.log(f" ■ {fn} – {n} closing {cnt} position(s)..."))
    result = acct.liquidate_position_api(account_id=aid)
    if result:
        closed_tv += 1
        self.root.after(0, lambda fn=firm_name, n=aname:
            self.log(f" ■ {fn} – {n} liquidated"))
    else:
        errors.append(f"{firm_name}/{aname}: liquidate returned None")
        self.root.after(0, lambda fn=firm_name, n=aname:
            self.log(f" ■ {fn} – {n} liquidation FAILED", "ERROR"))
    # Cancel remaining bracket orders (stop/limit)
    try:
        cancelled = acct.cancel_all_orders_api(account_id=aid)
        if cancelled > 0:
            self.root.after(0, lambda fn=firm_name, n=aname, c=cancelled:
                self.log(f" ■ {fn} – {n} cancelled {c} pending order(s)"))
    except Exception:
        pass
    else:
        self.root.after(0, lambda fn=firm_name:
            self.log(f" ■ {fn} – could not fetch account list", "ERROR"))
        errors.append(f"{firm_name}: account list fetch failed")
    elif hasattr(acct, 'close_all_positions'):
        # TopStepX
        acct.close_all_positions()
        closed_tv += 1
        self.root.after(0, lambda fn=firm_name:
            self.log(f" ■ {fn} positions closed"))
    except Exception as e:
        errors.append(f"{firm_name}: {e}")
        self.root.after(0, lambda fn=firm_name, err=str(e):
            self.log(f" ■ {fn} close failed: {err}", "ERROR"))

# 2. Close all MT5 positions
try:
    if not MT5_AVAILABLE:
        raise RuntimeError("MetaTrader5 module is not available")
    positions = mt5.positions_get()
    if positions:
        mt5_api = self._get_mt5_trading_api() if hasattr(self, '_get_mt5_trading_api') else None
        for pos in positions:
            try:
                if mt5_api:
                    mt5_api.close_trade(pos.ticket)
            else:
                # Direct close via MT5 API
                request = {
                    "action": mt5.TRADE_ACTION_DEAL,
                    "position": pos.ticket,
                    "symbol": pos.symbol,
                    "volume": pos.volume,
                    "type": mt5.ORDER_TYPE_SELL if pos.type == 0 else mt5.ORDER_TYPE_BUY,

```

```

        "type_filling": mt5.ORDER_FILLING_IOC,
    }
    mt5.order_send(request)
    closed_mt5 += 1
    self.root.after(0, lambda t=pos.ticket, s=pos.symbol:
        self.log(f" ■ MT5 #{t} {s} closed"))
except Exception as e:
    errors.append(f"MT5 #{pos.ticket}: {e}")
    self.root.after(0, lambda t=pos.ticket, err=str(e):
        self.log(f" ■ MT5 #{t} close failed: {err}", "ERROR"))
except Exception as e:
    errors.append(f"MT5: {e}")
    self.root.after(0, lambda err=str(e):
        self.log(f" ■ MT5 close skipped: {err}"))

# Summary
summary = f"■ Close All done – Brokers: {closed_tv}, MT5: {closed_mt5}"
if errors:
    summary += f", Errors: {len(errors)}"
self.root.after(0, lambda s=summary: self.log(s))
self.root.after(0, lambda s=summary: self._add_activity(s, "error"))

threading.Thread(target=_do_close_all, daemon=True, name="CloseAll").start()

def _refresh_hedge_status(self):
    """Periodically check Hedge Protector health and log stats."""
    if not self._hedge_protector or not self._hedge_protector.is_running:
        return
    # Schedule next check
    self.root.after(5000, self._refresh_hedge_status)

def _start_auto_trade(self):
    """Activate auto-trade: compute randomized start time, begin countdown."""
    from datetime import datetime, timedelta, timezone

    # Validation: need trades loaded
    if not self._active_trade_rows:
        self.log("■ Load trades first before enabling auto-trade", "WARN")
        return

    # Validation: need at least one broker connected
    connected_firms = [f for f, c in self._broker_connections.items() if c.get("account")]
    if not connected_firms:
        self.log("■ Connect at least one broker before enabling auto-trade", "WARN")
        return

    # Validation: hedging mode needs MT5
    if self.hedge_mode_var.get() == "Hedging":
        mt5_api = self._get_mt5_trading_api()
        if not mt5_api:
            self.log("■ Connect MT5 first for hedging mode auto-trade", "WARN")
            return

    # Validation: signal mode needs MT5 for price data
    if self.auto_trade_signal_var.get():
        if not SIGNALS_AVAILABLE:
            self.log("■ Signal indicators not available – install required packages", "WARN")
            return
        if not self._ensure_mt5_for_signals():
            self.log("■ Connect MT5 first for Actual Signal mode (indicators need price data)", "WARN")

```

```

        return

EAT = timezone(timedelta(hours=3)) # East Africa Time (UTC+3)
now_eat = datetime.now(EAT)

immediate = self.auto_trade_immediate_var.get()

if immediate:
    # Execute immediately (5-second grace period)
    scheduled_eat = now_eat + timedelta(seconds=5)
    offset_minutes = 0
else:
    # Base time: 2:05 AM EAT today (or tomorrow if already past ~5:05 AM)
    base = now_eat.replace(hour=2, minute=5, second=0, microsecond=0)

    # Random offset: 0 to 180 minutes (3 hours)
    offset_minutes = random.randint(0, 180)
    scheduled_eat = base + timedelta(minutes=offset_minutes)

    # If the scheduled time already passed today, schedule for tomorrow
    if scheduled_eat <= now_eat:
        scheduled_eat += timedelta(days=1)

self._auto_trade_scheduled_dt = scheduled_eat
self.auto_trade_enabled = True
self._auto_trade_stop.clear()

use_signal = self.auto_trade_signal_var.get() and SIGNALS_AVAILABLE
self._auto_trade_use_signal = use_signal

if use_signal:
    # Direction will be determined by indicator signals at execution time
    self._auto_trade_firm_sides = {} # filled at execution
    self.auto_trade_firms_var.set(" ■ Directions from indicator signals")
else:
    # Daily bias per prop firm (persisted, resets each day)
    firms_in_rows = set()
    for rd in self._active_trade_rows:
        firm_name = rd["eval"].get("Prop Firm", rd["firm_code"])
        firms_in_rows.add(firm_name)
    self._auto_trade_firm_sides = self._get_daily_bias(firms_in_rows)

    # Build display string
    dir_lines = []
    for firm, s in self._auto_trade_firm_sides.items():
        arrow = "▲" if s == "buy" else "▼"
        dir_lines.append(f" {arrow} {s.upper():4s} {firm}")
    self.auto_trade_firms_var.set("\n".join(dir_lines))

mode_label = "indicator signals" if use_signal else "random dirs per firm"
time_str = scheduled_eat.strftime("%I:%M %p EAT")
self.auto_trade_btn.configure(text="■ Stop Auto-Trade")
if CTK_AVAILABLE:
    self.auto_trade_btn.configure(fg_color='#dc2626', hover_color='#b91c1c')
if immediate:
    self.auto_trade_status_var.set(f"Executing in 5s – {mode_label}")
    self.log(f"■ Auto-trade starting immediately – {mode_label}")
else:
    self.auto_trade_status_var.set(f"Scheduled at {time_str} – {mode_label}")
    self.log(f"■ Auto-trade scheduled at {time_str} (+{offset_minutes}min random offset)")

```

```

if not use_signal:
    for firm, s in self._auto_trade_firm_sides.items():
        self.log(f"    {'▲' if s == 'buy' else '▼'} {firm} → {s.upper()}")
else:
    self.log("    ■ Directions will be generated from indicators at execution time")

# Start background countdown / executor thread
self.auto_trade_thread = threading.Thread(
    target=self._auto_trade_loop, daemon=True)
self.auto_trade_thread.start()

# Start UI countdown ticker
self._tick_auto_trade_countdown()

def _stop_auto_trade(self):
    """Cancel auto-trade scheduler."""
    self.auto_trade_enabled = False
    self._auto_trade_stop.set()
    self._auto_trade_scheduled_dt = None
    self.auto_trade_btn.configure(text="■ Start Auto-Trade")
    if CTK_AVAILABLE:
        self.auto_trade_btn.configure(fg_color=self.C_ACCENT, hover_color=self.C_ACCENT_HV)
    self.auto_trade_status_var.set("Auto-trade off")
    self.auto_trade_countdown_var.set("")
    self.auto_trade_firms_var.set("")
    self._auto_trade_firm_sides = {}
    self.log("■ Auto-trade cancelled")

def _tick_auto_trade_countdown(self):
    """Update the countdown label every second."""
    if not self.auto_trade_enabled or not self._auto_trade_scheduled_dt:
        return
    from datetime import datetime, timedelta, timezone
    EAT = timezone(timedelta(hours=3))
    now = datetime.now(EAT)
    remaining = self._auto_trade_scheduled_dt - now
    if remaining.total_seconds() <= 0:
        self.auto_trade_countdown_var.set("Executing now...")
        return
    hours, rem = divmod(int(remaining.total_seconds()), 3600)
    minutes, seconds = divmod(rem, 60)
    self.auto_trade_countdown_var.set(f"Starts in {hours}h {minutes}m {seconds}s")
    self.root.after(1000, self._tick_auto_trade_countdown)

def _auto_trade_loop(self):
    """Background thread: wait until scheduled time, then execute all trades."""
    from datetime import datetime, timedelta, timezone
    EAT = timezone(timedelta(hours=3))

    while self.auto_trade_enabled and not self._auto_trade_stop.is_set():
        now = datetime.now(EAT)
        if now >= self._auto_trade_scheduled_dt:
            # Time to execute
            self.root.after(0, lambda: self.auto_trade_countdown_var.set("Executing now..."))
            self.root.after(0, self._auto_execute_all_trades)
            return
        # Sleep 1 second between checks
        self._auto_trade_stop.wait(timeout=1)

def _auto_execute_all_trades(self):

```

```

"""Execute trades for ALL loaded rows, parallel across prop firms.

Each prop firm has its own Chrome instance opened during initialization.
Trades for different firms run in parallel threads (one thread per firm),
while trades for the same firm run sequentially within that thread.
"""
firm_sides = getattr(self, '_auto_trade_firm_sides', {})
use_signal = getattr(self, '_auto_trade_use_signal', False)
rows = list(self._active_trade_rows) # snapshot

if not rows:
    self.log("■ No trades to execute – list is empty")
    self._stop_auto_trade()
    return

self.log(f"■ Auto-executing {len(rows)} accounts (parallel per firm)...")

hedging = self.hedge_mode_var.get() == "Hedging"
default_platform = self.broker_var.get()
mt5_api = self._get_mt5_trading_api() if hedging else None
if hedging and not mt5_api:
    messagebox.showerror(
        "MT5 Not Connected",
        "Connect MT5 first, or switch to 'Broker Only' mode to skip MT5 hedging."
    )
    self.log("■ Auto-trade aborted – MT5 not connected (Hedging mode requires MT5)")
    self._stop_auto_trade()
    return

# Group rows by firm so each firm's Chrome runs in its own thread
rows_by_firm = defaultdict(list)
for row_data in rows:
    firm_name = row_data["eval"].get("Prop Firm", row_data["firm_code"])
    rows_by_firm[firm_name].append(row_data)

# ■■ PAYOUT CHECK: skip accounts with payout pending ■■
payout_skipped = []
for firm_name, firm_rows in list(rows_by_firm.items()):
    clean = []
    for rd in firm_rows:
        if self._eval_has_payout(rd.get("eval", {})):
            payout_skipped.append(rd)
            self.log(
                f"    ■ {rd['acct_num']} ({firm_name} / {rd['current_phase']}) – PAYOUT pending,
            else:
                clean.append(rd)
    rows_by_firm[firm_name] = clean
if payout_skipped:
    self.log(f"■ {len(payout_skipped)} account(s) skipped – payout pending (request payout first)

# ■■ PRE-VALIDATE: check which accounts actually exist on each Tradovate ■■
skipped_rows = []
not_connected_firms = []
if default_platform == "Tradovate":
    self.log("■ Pre-validating accounts against Tradovate dropdowns...")
    validated_by_firm = {}
    for firm_name, firm_rows in list(rows_by_firm.items()):
        broker_account = self._get_broker_for_firm(firm_name)
        if not broker_account:
            # Firm is NOT connected – skip ALL its accounts immediately

```

```

        not_connected_firms.append(firm_name)
        self.log(f"■ {firm_name} is NOT connected – skipping {len(firm_rows)} account(s)")
        for rd in firm_rows:
            skipped_rows.append(rd)
            self.log(
                f"    ■ {rd['acct_num']} ({firm_name} / {rd['current_phase']}) – firm not connected"
            )
        rows_by_firm[firm_name] = [] # clear all rows for this firm
        continue

# Read all accounts from this firm's Tradovate dropdown
import re as _re
try:
    trado_accounts = broker_account.get_all_accounts()
except Exception as e:
    self.log(f"■ Could not read accounts for {firm_name}: {e}")
    trado_accounts = []

if not trado_accounts:
    self.log(f"■ No accounts listed for {firm_name} – skipping pre-filter")
    continue

trado_lower = [a.lower() for a in trado_accounts]

valid_rows = []
for rd in firm_rows:
    acct = str(rd["acct_num"]).strip()
    acct_lower = acct.lower()
    digit_match = _re.search(r'(\d{5,})$', acct)
    digit_suffix = digit_match.group(1) if digit_match else None

    found = False
    for ta in trado_accounts:
        ta_lower = ta.lower()
        if acct_lower in ta_lower or ta_lower in acct_lower:
            found = True
            break
        if digit_suffix and ta.endswith(digit_suffix):
            found = True
            break

    if found:
        valid_rows.append(rd)
    else:
        skipped_rows.append(rd)

rows_by_firm[firm_name] = valid_rows
validated_by_firm[firm_name] = len(valid_rows)

# Log skipped accounts (those not found in Tradovate dropdown)
missing_only = [rd for rd in skipped_rows if rd["eval"].get("Prop Firm", rd["firm_code"]) not in not_connected_firms]
if missing_only:
    self.log(f"■ {len(missing_only)} account(s) NOT found on Tradovate – skipped:")
    for rd in missing_only:
        fn = rd["eval"].get("Prop Firm", rd["firm_code"])
        self.log(f"    ■ {rd['acct_num']} ({fn} / {rd['current_phase']})")

# Mark ALL skipped rows visually (unconnected + not found)
for rd in skipped_rows:
    def _mark_skipped(rd=rd):

```



```

        try:
            rd["buy_btn"].configure(state='disabled', text="N/A")
            rd["sell_btn"].configure(state='disabled', text="N/A")
        except Exception:
            pass
        self.root.after(0, _mark_skipped)

# Remove empty firms
rows_by_firm = {f: r for f, r in rows_by_firm.items() if r}

total_valid = sum(len(r) for r in rows_by_firm.values())
self.log(f"■ {total_valid} account(s) validated, {len(skipped_rows)} skipped")

if not rows_by_firm:
    self.log("■ No valid accounts remain – aborting auto-trade")
    self._stop_auto_trade()
    return

total_success = threading.Lock()
counters = {"success": 0, "fail": 0, "skipped": len(skipped_rows)}

def _execute_firm_trades(firm_name, firm_rows):
    """Execute all trades for one firm sequentially on its own Chrome."""
    # Auto-detect platform: TopStep firms always use TopStepX (Selenium)
    if "topstep" in firm_name.lower():
        platform = "TopStepX"
    else:
        platform = default_platform
    broker_account = self._get_broker_for_firm(firm_name)
    if not broker_account:
        for rd in firm_rows:
            self.root.after(0, lambda fn=firm_name, an=rd["acct_num"]: self.log(
                f"■ No broker connected for {fn} – {an} skipped", "ERROR"))
            with total_success:
                counters["fail"] += 1
        return

    for row_data in firm_rows:
        if self._auto_trade_stop.is_set():
            break

        firm_code = row_data["firm_code"]
        phase_key = row_data["phase_key"]
        acct_size = row_data["acct_size"]
        acct_num = row_data["acct_num"]

        # ■■■ Resolve phase_key from day placeholder (primary source of truth) ■■■
        auto_ev = row_data.get("eval", {})
        fresh_auto_ev = self._refresh_eval_for_account(acct_num)
        if fresh_auto_ev:
            auto_ev = fresh_auto_ev
            row_data["eval"] = fresh_auto_ev
        resolved_key, day_idx, day_name = self._resolve_phase_key_from_day(
            auto_ev, firm_code, row_data.get("current_phase", ""))
        if resolved_key is None:
            # Build a diagnostic message: list any day-like values found,
            # so the user can see whether the cell really has a placeholder
            # or only a future-day prep.
            import datetime as _dtmod
            today_wd = _dtmod.date.today().weekday()

```

```

found_days = []
future_days = []
for _ph, _flist in self._ALL_PHASE_FIELD_SETS:
    for _i, _f in enumerate(_flist):
        _v = auto_ev.get(_f, None)
        _dn2 = self._parse_day_token(_v)
        if _dn2 is None:
            continue
        label = f"_{f}={str(_v).strip()}"
        if _dn2 > today_wd:
            future_days.append(label)
        else:
            found_days.append(label)
if future_days and not found_days:
    diag = f"only future placeholder(s) [{', '.join(future_days[:3])}]"
elif found_days:
    diag = (f"placeholder(s) found [{', '.join(found_days[:3])}] "
           f"but firm '{firm_code}' has no trade order for that phase")
else:
    diag = "no day name in any Hedge Result/Hedge Day cell"
_an, _diag = acct_num, diag
self.root.after(0, lambda an=_an, d=_diag:
    self.log(f"■ {an}: No tradeable day placeholder – {d}", "ERROR"))
with total_success:
    counters["fail"] += 1
    continue
if resolved_key != phase_key:
    _an, _di, _dn, _rk, _pk = acct_num, day_idx, day_name, resolved_key, phase_key
    self.root.after(0, lambda an=_an, di=_di, dn=_dn, rk=_rk, pk=_pk:
        self.log(f"■ {an}: Day cell {di + 1} ({dn}) → blueprint {rk} (was {pk})"))
    phase_key = resolved_key

# Determine direction: signal-based or random bias
if use_signal:
    # Lazy-compute signal once per firm
    if firm_name not in firm_sides:
        config_tmp = None
        if self.prop_firm_mgr:
            config_tmp = self.prop_firm_mgr.get_strategy_config(
                firm_code, phase_key, acct_size)
        mt5_sym = (config_tmp or {}).get("mt5_symbol", "NAS100")
        sig = self._get_signal_direction(mt5_sym)
        firm_sides[firm_name] = sig
        self.root.after(0, lambda fn=firm_name, s=sig, sym=mt5_sym:
            self.log(f" ■ {fn} ({sym}) → signal: {s.upper()}"))
    side = firm_sides.get(firm_name, random.choice(["buy", "sell"]))

config = None
if self.prop_firm_mgr:
    config = self.prop_firm_mgr.get_strategy_config(
        firm_code, phase_key, acct_size)
if not config:
    self.root.after(0, lambda an=acct_num, fc=firm_code, pk=phase_key, sz=acct_size: self
        f"■ No blueprint: {an} ({fc}/{pk}/{sz}) – skipped", "ERROR"))
    with total_success:
        counters["fail"] += 1
        continue

trado_sym = config.get("tradovate_symbol", "") or config.get("topstepx_symbol", "")
trado_qty = int(config.get("tradovate_qty", 2) or config.get("topstepx_qty", 2))

```

```

# Log resolved blueprint up-front so the user can verify the
# correct trade is being placed (catches farming/challenge mix-ups).
_bp_tp = config.get("tradovate_tp_ticks", config.get("topstepx_tp_ticks", "?"))
_bp_sl = config.get("tradovate_sl_ticks", config.get("topstepx_sl_ticks", "?"))
_an, _fc, _pk, _sz = acct_num, firm_code, phase_key, acct_size
_bs, _bq, _bt, _bsl = trado_sym, trado_qty, _bp_tp, _bp_sl
self.root.after(0, lambda an=_an, fc=_fc, pk=_pk, sz=_sz, bs=_bs, bq=_bq, bt=_bt, bsl=_bsl:
    self.log(f"■ {an}: blueprint {fc}/{pk}/{sz} → "
            f"{bs} qty={bq} TP={bt}t SL={bsl}t"))

# ■■ Farming: cap MT5 TP based on hard-stop proximity ■■
_is_farming_sym_auto = "MNQ" in (config.get("tradovate_symbol", "") or config.get(
    "topstepx_symbol", "")).upper()
if _is_farming_sym_auto and self.prop_firm_mgr:
    # Farming: cap MT5 TP based on hard-stop proximity
    try:
        _bal_auto = None
        if broker_account:
            _stats_auto = broker_account.get_account_stats()
            _bal_str_auto = _stats_auto.get("Balance", "")
            if _bal_str_auto and _bal_str_auto not in ("N/A", "Error", ""):
                _bal_auto = float(_bal_str_auto.replace("$", "").replace(",", ""))
        if _bal_auto is not None:
            orig_mt5_tp_a = int(config.get("mt5_tp_points", 0))
            config = self.prop_firm_mgr.adjust_farming_tp_sl(config, _bal_auto, firm_code)
            new_mt5_tp_a = int(config.get("mt5_tp_points", 0))
            if new_mt5_tp_a != orig_mt5_tp_a:
                _an, _bal_v, _omt, _nmt = acct_num, _bal_auto, orig_mt5_tp_a, new_mt5_tp_a
                self.root.after(0, lambda an=_an, bv=_bal_v, omt=_omt, nmt=_nmt:
                    self.log(
                        f"■ Farming TP cap {an}: balance=${bv:.2f} → MT5 TP {omt}→{nmt}"))
            else:
                _an = acct_num
                self.root.after(0, lambda an=_an, omt=orig_mt5_tp_a:
                    self.log(f"■ Farming TP OK {an}: MT5 TP {omt} pts within safe range"))
        else:
            _an = acct_num
            self.root.after(0, lambda an=_an:
                self.log(f"■ Farming {an}: could not read balance – using blueprint TP/SL"))
    except Exception as _fe:
        _an, _err = acct_num, str(_fe)
        self.root.after(0, lambda an=_an, err=_err:
            self.log(f"■ Farming TP check failed for {an}: {err}"))
# TP/SL comes directly from the stage blueprint (selected by day placeholder)

trado_tp = int(config.get("tradovate_tp_ticks", 151) or config.get("topstepx_tp_ticks", 151))
trado_sl = int(config.get("tradovate_sl_ticks", 200) or config.get("topstepx_sl_ticks", 200))
mt5_sym = config.get("mt5_symbol", "NAS100")
mt5_vol = float(config.get("mt5_volume", 2.8))
mt5_tp = int(config.get("mt5_tp_points", 46))
mt5_sl = int(config.get("mt5_sl_points", 42))

# ■■ Adjust TP based on stage progress ■■
if self.prop_firm_mgr and broker_account and not _is_farming_sym_auto:
    try:
        auto_profit = self._get_current_phase_profit(
            auto_ev, row_data.get("current_phase", ""),
            broker_account=broker_account, acct_size=acct_size)
        size_key_a = self.prop_firm_mgr.convert_account_size_to_key(acct_size)
        stage_start = self.prop_firm_mgr.get_stage_start_target(

```

```

        firm_code, row_data.get("current_phase", ""), phase_key, size_key_a)
    stage_profit_so_far = auto_profit - stage_start
    trado_sym_for_calc = config.get("tradovate_symbol", "") or config.get(
        "topstepx_symbol", "")
    tick_val = self.prop_firm_mgr.get_tick_value(
        trado_sym_for_calc) if self.prop_firm_mgr else 5.0
    trado_qty_for_calc = int(config.get("tradovate_qty", 1) or config.get("topstepx_
        1))
    if tick_val > 0 and trado_qty_for_calc > 0:
        orig_tp_a = trado_tp
        orig_mt5_sl_a = mt5_sl
        profit_ticks = stage_profit_so_far / (tick_val * trado_qty_for_calc)
        adjusted_tp = max(5, round(trado_tp - profit_ticks))
        tp_ratio = adjusted_tp / trado_tp if trado_tp > 0 else 1.0
        adjusted_mt5_sl = max(5, round(mt5_sl * tp_ratio))
        if adjusted_tp != trado_tp:
            trado_tp = adjusted_tp
            mt5_sl = adjusted_mt5_sl
        _an, _ss, _ap, _sp = acct_num, stage_start, auto_profit, stage_profit_so_
        _otp, _ntp, _oms, _nms = orig_tp_a, trado_tp, orig_mt5_sl_a, mt5_sl
        self.root.after(0, lambda an=_an, ss=_ss, ap=_ap, sp=_sp, otp=_otp, ntp=
            oms=_oms, nms=_nms:
            self.log(f"■ TP adjust {an}: stage_start=${ss:,.0f}, "
                f"P/L=${ap:,.2f}, stage P/L=${sp:+,.2f} → "
                f"TP {otp}→{ntp}t, MT5 SL {oms}→{nms}pts"))
    except Exception as _te:
        _an, _err = acct_num, str(_te)
        self.root.after(0, lambda an=_an, err=_err:
            self.log(f"■ TP adjust failed for {an}: {err}"))

# ■■ Adjust SL based on midnight balance + drawdown protection ■■
if broker_account and platform == "Tradovate" and hasattr(broker_account, 'get_min_equity'):
    try:
        min_eq_data = broker_account.get_min_equity()
        if min_eq_data:
            live_net_liq = min_eq_data['net_liq']
            net_liq_sod = min_eq_data.get('net_liq_sod', 0)
            live_min_equity = min_eq_data.get('min_equity', 0)
            tmdl = min_eq_data.get('trailing_max_drawdown_limit', 50000)
            trado_qty_for_calc = int(config.get("tradovate_qty", 1) or config.get(
                "topstepx_qty", 1))
            trado_sym_for_calc = config.get("tradovate_symbol", "") or config.get(
                "topstepx_symbol", "")
            tick_val = self.prop_firm_mgr.get_tick_value(
                trado_sym_for_calc) if self.prop_firm_mgr else 5.0

            # Step 1: Midnight balance SL – floor = SOD - blueprint_sl_dollars
            if net_liq_sod > 0 and tick_val > 0 and trado_qty_for_calc > 0:
                blueprint_sl_dollars = trado_sl * tick_val * trado_qty_for_calc
                sl_floor = net_liq_sod - blueprint_sl_dollars
                available = live_net_liq - sl_floor
                if available > 0:
                    midnight_sl = max(10, int(available / (tick_val * trado_qty_for_calc)))
                    if midnight_sl != trado_sl:
                        orig_sl = trado_sl
                        orig_mt5_tp = mt5_tp
                        trado_sl = midnight_sl
                        mt5_tp = max(5, int(trado_sl / 4) - 1)
                        _an = acct_num
                        _sod, _nl, _dpnl = net_liq_sod, live_net_liq, live_net_liq - net_

```

```

        _osl, _nsl, _omt, _nmt = orig_sl, trado_sl, orig_mt5_tp, mt5_tp
        self.root.after(0, lambda an=_an, sod=_sod, nl=_nl, dp=_dpnl,
                        osl=_osl, nsl=_nsl, omt=_omt, nmt=_nmt:
                        self.log(f"■ Midnight SL {an}: SOD=${sod:,.2f}, "
                                f"live=${nl:,.2f}, daily P/L=${dp:+,.2f} → "
                                f"SL {osl}→{nsl}t, MT5 TP {omt}→{nmt}pts"))
    else:
        _an, _sod = acct_num, net_liq_sod
        self.root.after(0, lambda an=_an, sod=_sod, sl=trado_sl:
                        self.log(
                            f"■ Midnight SL OK {an}: SOD=${sod:,.2f}, SL={sl}t uncha
else:
    _an, _nl, _fl = acct_num, live_net_liq, sl_floor
    self.root.after(0, lambda an=_an, nl=_nl, fl=_fl:
                    self.log(f"■ Midnight SL floor breached {an}: "
                            f"live=${nl:,.2f} < floor=${fl:,.2f} – using min SL"))
    trado_sl = 10
    mt5_tp = max(5, int(trado_sl / 4) - 1)

# Step 2: TMDL drawdown cap – further tighten if near breach
if live_min_equity > 0 and live_net_liq < tmdl:
    drawdown_remaining = live_net_liq - live_min_equity
    current_sl_risk = trado_sl * tick_val * trado_qty_for_calc
    if drawdown_remaining > 0 and current_sl_risk > drawdown_remaining:
        orig_sl = trado_sl
        orig_mt5_tp = mt5_tp
        trado_sl = max(10, int(drawdown_remaining / (
            tick_val * trado_qty_for_calc)))
        mt5_tp = max(5, int(trado_sl / 4) - 1)
        _an, _dr = acct_num, drawdown_remaining
        _osl, _nsl, _omt, _nmt = orig_sl, trado_sl, orig_mt5_tp, mt5_tp
        self.root.after(0, lambda an=_an, dr=_dr, osl=_osl, nsl=_nsl, omt=_omt,
                            nmt=_nmt:
                            self.log(f"■ TMDL SL cap {an}: remaining=${dr:,.2f} → "
                                    f"SL {osl}→{nsl}t, MT5 TP {omt}→{nmt}pts"))
    elif drawdown_remaining <= 0:
        _an, _dr = acct_num, drawdown_remaining
        self.root.after(0, lambda an=_an, dr=_dr:
                        self.log(f"■ No drawdown room for {an} (${dr:,.2f})"))
elif live_min_equity > 0:
    _an, _nl, _tmdl = acct_num, live_net_liq, tmdl
    self.root.after(0, lambda an=_an, nl=_nl, t=_tmdl:
                    self.log(f"■ TMDL OK {an}: ${nl:,.2f} ≥ TMDL=${t:,.0f}"))
except Exception:
    pass

try:
    # 1. Broker order – uses this firm's own Chrome instance
    if platform == "Tradovate":
        if side == "buy":
            order_result = broker_account.buy_market(trado_sym, trado_qty, tp=trado_tp,
                                                    sl=trado_sl, expected_account=acct_num)
        else:
            order_result = broker_account.sell_market(trado_sym, trado_qty, tp=trado_tp,
                                                    sl=trado_sl, expected_account=acct_num)
    elif platform == "TopStepX":
        # Ensure we are on the intended sub-account before placing the order.
        if hasattr(broker_account, 'switch_account') and acct_num:
            switched = broker_account.switch_account(account_name_contains=acct_num)
            if not switched:

```

```

        raise Exception(f"TopStepX could not switch to account {acct_num}")

# TopStepX uses two-digit year futures codes (NQ26, MNQM26)
_tsx_sym = _to_topstepx_symbol(trado_sym)
_tsx_tick_val = self.prop_firm_mgr.get_tick_value(
    _tsx_sym) if self.prop_firm_mgr else 0.5
_tsx_tp_dollars = trado_tp * _tsx_tick_val * trado_qty
_tsx_sl_dollars = trado_sl * _tsx_tick_val * trado_qty
if side == "buy":
    order_result = broker_account.place_buy_order(
        _tsx_sym,
        trado_qty,
        tp_dollars=_tsx_tp_dollars,
        sl_dollars=_tsx_sl_dollars,
    )
else:
    order_result = broker_account.place_sell_order(
        _tsx_sym,
        trado_qty,
        tp_dollars=_tsx_tp_dollars,
        sl_dollars=_tsx_sl_dollars,
    )

# TopStepX returns status dicts on both success and failure.
# Convert broker-declared failures to exceptions so counters/logs are accurate.
if isinstance(order_result, dict) and order_result.get("success") is False:
    raise Exception(order_result.get("message") or "Broker reported unsuccessful order")

self.root.after(0, lambda an=acct_num, fc=firm_code, sd=side, sym=trado_sym,
    qty=trado_qty:
    self.log(f"■ {platform} {sd.upper()} {qty} {sym} → {an} ({fc})"))

# 2. MT5 hedge (opposite direction)
if hedging and mt5_api:
    hedge_side = "sell" if side == "buy" else "buy"
    comment = f"{acct_num}_{phase_key}"
    if hedge_side == "buy":
        mt5_api.buy_market(mt5_sym, mt5_vol, sl=mt5_sl, tp=mt5_tp, comment=comment)
    else:
        mt5_api.sell_market(mt5_sym, mt5_vol, sl=mt5_sl, tp=mt5_tp, comment=comment)
    self.root.after(0, lambda an=acct_num, hs=hedge_side, vol=mt5_vol, sym=mt5_sym,
        cmt=comment:
        self.log(f"■ MT5 hedge {hs.upper()} {vol} {sym} comment:{cmt} → {an}"))

with total_success:
    counters["success"] += 1

# ■■ Auto-status: set "In Progress" when trades go out ■■
if not RELEASE_DISABLE_AUTO_STATUS_UPDATES:
    _ev = row_data.get("eval")
    if _ev:
        _has_funded = bool((row_data.get("eval", {}).get("Account #.1") or "").strip())
        _sf = "Status" if _has_funded else "Status P1"
        _cur = (_ev.get(_sf) or "").strip().lower()
        # Only set In Progress if status is empty, Not Started, or already In Progress
        if not _cur or _cur in ("not started", "in progress", ""):
            _ev[_sf] = "In Progress"
            self.root.after(0, lambda an=acct_num, sf=_sf:
                self.log(f"■ Auto-status: {an} → {_sf}='In Progress'"))

# Remove row from UI

```

```

def _remove(rd=row_data):
    rd["frame"].destroy()
    if rd in self._active_trade_rows:
        self._active_trade_rows.remove(rd)
    remaining = len(self._active_trade_rows)
    self.trades_count_var.set(
        f"{remaining} active trade{'s' if remaining != 1 else ''}"
        if remaining > 0 else "All trades complete ✓")
    self.root.after(0, _remove)

# Small delay between trades on the same account
time.sleep(2)

except Exception as e:
    with total_success:
        counters["fail"] += 1
    self.root.after(0, lambda an=acct_num, err=str(e):
        self.log(f"■ Auto-trade failed for {an}: {err}", "ERROR"))

def _dispatch_parallel():
    num_firms = len(rows_by_firm)
    self.root.after(0, lambda n=num_firms: self.log(
        f"■ Dispatching trades across {n} firm(s) in parallel..."))
    with ThreadPoolExecutor(max_workers=num_firms) as executor:
        futures = {
            executor.submit(_execute_firm_trades, firm, firm_rows): firm
            for firm, firm_rows in rows_by_firm.items()
        }
        for future in as_completed(futures):
            firm = futures[future]
            try:
                future.result()
            except Exception as e:
                self.root.after(0, lambda fn=firm, err=str(e):
                    self.log(f"■ Firm thread {fn} crashed: {err}", "ERROR"))

# Final summary
self.root.after(0, lambda s=counters["success"], f=counters["fail"], sk=counters["skipped"]:
    self.log(
        f"■ Auto-trade complete: {s} succeeded, {f} failed, {sk} skipped (not on Tradovate)"
    self.root.after(0, self._stop_auto_trade)

threading.Thread(target=_dispatch_parallel, daemon=True).start()

def _on_prop_firm_change(self, event=None):
    """Update phase and size options when prop firm changes (compat stub)."""
    pass

def _update_account_sizes(self):
    """Update account size (compat stub)."""
    pass

def _populate_broker_rows(self, evaluations, prop_accounts):
    """Build one connection row per unique prop firm from active evaluations."""
    for child in self._broker_rows_frame.winfo_children():
        child.destroy()
    # Preserve any already-connected accounts
    old_connections = dict(self._broker_connections)
    self._broker_connections.clear()

    # Get unique prop firms in order

```

```

firms = []
seen = set()
for ev in evaluations:
    firm = ev.get("Prop Firm", "Unknown")
    if firm not in seen:
        seen.add(firm)
        firms.append(firm)

if not firms:
    if CTK_AVAILABLE:
        ctk.CTkLabel(self._broker_rows_frame,
                      text="No active prop firms – load trades first",
                      font=("Segoe UI", 9, "italic"), text_color="#4A5568").pack(pady=4)
    return

# Header row
if CTK_AVAILABLE:
    hdr = ctk.CTkFrame(self._broker_rows_frame, fg_color="transparent")
    hdr.pack(fill="x", padx=4, pady=(2, 0))
    ctk.CTkLabel(hdr, text="PROP FIRM", width=110, font=("Consolas", 8, "bold"),
                  text_color="#4A5568", anchor="w").pack(side="left", padx=(12, 0))
    ctk.CTkLabel(hdr, text="USERNAME", width=140, font=("Consolas", 8, "bold"),
                  text_color="#4A5568", anchor="w").pack(side="left", padx=(8, 0))
    ctk.CTkLabel(hdr, text="PASSWORD", width=110, font=("Consolas", 8, "bold"),
                  text_color="#4A5568", anchor="w").pack(side="left", padx=(8, 0))

# Build lookup from prop_accounts by prop_firm name
pa_lookup = {}
pa_unmatched = [] # accounts with creds but no firm match yet
for pa in (prop_accounts or []):
    pf = (pa.get("prop_firm") or "").strip()
    has_creds = bool((pa.get("tradovate_username") or "").strip() and
                     (pa.get("tradovate_password") or "").strip()))
    if pf and pf not in pa_lookup:
        pa_lookup[pf] = pa
    elif has_creds and not pf:
        pa_unmatched.append(pa)

# Log what we received for debugging
if prop_accounts:
    self.log(f"Dashboard prop_accounts: {len(prop_accounts)} entries, "
            f"matched firms: {list(pa_lookup.keys())}")

auto_count = 0
for firm in firms:
    strip_color = self.PROP_FIRM_COLORS.get(firm, "#95A5A6")
    # Try exact match first, then case-insensitive, then alias match, then unmatched pool
    pa = pa_lookup.get(firm, {})
    if not pa:
        for pf_key, pf_val in pa_lookup.items():
            if pf_key.lower() == firm.lower():
                pa = pf_val
                break
    if not pa:
        # Alias matching: firm name variants that refer to the same firm
        _FIRM_ALIASES = {
            "funded next": ["fundednext"],
            "fundednext": ["funded next"],
            "my funded futures": ["mffu"],
            "mffu": ["my funded futures"],

```



```

        "lucid": ["lucid trading"],
        "lucid trading": ["lucid"],
    }
    firm_lower = firm.lower()
    aliases = _FIRM_ALIASES.get(firm_lower, [])
    for pf_key, pf_val in pa_lookup.items():
        if pf_key.lower() in aliases:
            pa = pf_val
            break
    if not pa and pa_unmatched:
        pa = pa_unmatched.pop(0)
    pre_user = (pa.get("tradovate_username") or "").strip()
    pre_pass = (pa.get("tradovate_password") or "").strip()

    # Carry over existing connected account if available
    old_conn = old_connections.get(firm, {})
    existing_account = old_conn.get("account")

    if CTK_AVAILABLE:
        row = ctk.CTkFrame(self._broker_rows_frame, fg_color="#0A1220",
                           corner_radius=4, height=36,
                           border_width=1, border_color="#0F1A2A")
        row.pack(fill="x", padx=4, pady=1)
        row.pack_propagate(False)

        # Accent bar
        ctk.CTkFrame(row, width=3, fg_color=strip_color,
                     corner_radius=0).pack(side="left", fill="y")

        # Firm name
        ctk.CTkLabel(row, text=firm[:16], width=110,
                     font=("Consolas", 10, "bold"), text_color=strip_color,
                     anchor="w").pack(side="left", padx=(8, 0))

        # User entry
        user_entry = ctk.CTkEntry(row, width=140, height=26,
                                  fg_color=self.C_BG_THIRD, border_color=self.C_BORDER,
                                  text_color=self.C_TEXT, font=("Consolas", 10))
        user_entry.pack(side="left", padx=(8, 0))
        if pre_user:
            user_entry.insert(0, pre_user)

        # Pass entry
        pass_entry = ctk.CTkEntry(row, width=110, height=26, show="*",
                                  fg_color=self.C_BG_THIRD, border_color=self.C_BORDER,
                                  text_color=self.C_TEXT, font=("Consolas", 10))
        pass_entry.pack(side="left", padx=(8, 0))
        if pre_pass:
            pass_entry.insert(0, pre_pass)

        # Status indicator
        status_var = tk.StringVar(value="■" if existing_account else "■")
        status_lbl = ctk.CTkLabel(row, textvariable=status_var,
                                  font=("Segoe UI", 11), width=24,
                                  text_color="#22C55E" if existing_account else "#4A5568")
        status_lbl.pack(side="right", padx=(0, 8))

        # Connect button
        btn_text = "Connected" if existing_account else "Connect"
        conn_btn = ctk.CTkButton(row, text=btn_text, width=72, height=24,

```

```

        fg_color="#14532D" if existing_account else "#1A2332",
        hover_color="#24292F",
        border_width=1, border_color=self.C_BORDER,
        font=("Consolas", 9), text_color=self.C_TEXT,
        corner_radius=4,
        command=lambda f=firm: self._connect_broker_firm(f))
conn_btn.pack(side="right", padx=(8, 4))

# History button – shows full Tradovate trade history for this account
hist_btn = ctk.CTkButton(row, text="■ History", width=78, height=24,
        fg_color="#1A2332", hover_color="#2A3342",
        border_width=1, border_color="#3B4B5E",
        font=("Consolas", 9), text_color="#60A5FA",
        corner_radius=4,
        command=lambda f=firm: self._show_trade_history(f))
hist_btn.pack(side="right", padx=(4, 0))

# Dashboard button – show for all firms (use _BROWSER_MONITORED_FIRMS with case-insensitive)
_dash_cfg = self._BROWSER_MONITORED_FIRMS.get(firm)
if not _dash_cfg:
    # Case-insensitive lookup
    for _bk, _bv in self._BROWSER_MONITORED_FIRMS.items():
        if _bk.lower() == firm.lower():
            _dash_cfg = _bv
            break
if _dash_cfg and not RELEASE_DISABLE_PROP_DASHBOARD_ACCESS:
    dash_btn = ctk.CTkButton(row, text="■ Dashboard", width=90, height=24,
        fg_color="#1A1A3E", hover_color="#2A2A5E",
        border_width=1, border_color="#3B3B6E",
        font=("Consolas", 9), text_color="#A78BFA",
        corner_radius=4,
        command=lambda f=firm: self._launch_propfirm_dashboard(f))
    dash_btn.pack(side="right", padx=(4, 0))
else:
    row = tk.Frame(self._broker_rows_frame, bg="#0A1220")
    row.pack(fill="x", padx=4, pady=1)
    tk.Label(row, text=firm[:16], width=14, anchor='w',
        bg="#0A1220", fg=strip_color, font=('Consolas', 9)).pack(side="left", padx=2)
    user_entry = ttk.Entry(row, width=16)
    user_entry.pack(side="left", padx=2)
    if pre_user:
        user_entry.insert(0, pre_user)
    pass_entry = ttk.Entry(row, width=12, show="*")
    pass_entry.pack(side="left", padx=2)
    if pre_pass:
        pass_entry.insert(0, pre_pass)
    conn_btn = ttk.Button(row, text="Connect",
        command=lambda f=firm: self._connect_broker_firm(f))
    conn_btn.pack(side="left", padx=2)
    status_var = tk.StringVar(value="■" if existing_account else "■")
    status_lbl = ttk.Label(row, textvariable=status_var)
    status_lbl.pack(side="left", padx=2)

if pre_user and pre_pass:
    auto_count += 1

self._broker_connections[firm] = {
    "user_entry": user_entry,
    "pass_entry": pass_entry,
    "connect_btn": conn_btn,

```

```

        "status_var": status_var,
        "status_lbl": status_lbl,
        "row_frame": row,
        "account": existing_account,
    }

    if auto_count:
        self.log(f"Broker credentials found for {auto_count} prop firm(s) – auto-connecting...")
        # Auto-connect all firms that have credentials from dashboard
        self.root.after(500, self._auto_connect_populated_brokers)

def _connect_broker_firm(self, firm_name):
    """Connect a single prop firm's broker account."""
    conn = self._broker_connections.get(firm_name)
    if not conn:
        return

    # Auto-detect platform: TopStep firms always use TopStepX (Selenium)
    firm_lower = firm_name.lower()
    if "topstep" in firm_lower:
        platform = "TopStepX"
    else:
        platform = self.broker_var.get()

    user = conn["user_entry"].get().strip()
    pwd = conn["pass_entry"].get().strip()
    mode = self.trading_mode_var.get()

    if not user or not pwd:
        messagebox.showerror("Error", f"Enter username and password for {firm_name}")
        return

    self.log(f"Connecting {firm_name} to {platform}...")
    conn["status_var"].set("■")
    conn["connect_btn"].configure(text="...")

def _do_connect():
    try:
        if platform == "Tradovate":
            if not TRADOVATE_AVAILABLE:
                err = _TRADOVATE_IMPORT_ERROR or 'unknown reason'
                self.root.after(0, lambda: conn["status_var"].set("■"))
                self.log(f"Tradovate import failed: {err}", "ERROR")
                self.root.after(0, lambda: conn["connect_btn"].configure(text="Connect"))
                return
            account = TradovateAccount(user, pwd, trading_mode=mode)
            account.login()
        elif platform == "TopStepX":
            if not TOPSTEPX_AVAILABLE:
                err = _TOPSTEPX_IMPORT_ERROR or 'unknown reason'
                self.root.after(0, lambda: conn["status_var"].set("■"))
                self.log(f"TopStepX import failed: {err}", "ERROR")
                self.root.after(0, lambda: conn["connect_btn"].configure(text="Connect"))
                return
            account = TopStepXAccount(user, pwd)
            account.login()
        else:
            self.root.after(0, lambda: conn["status_var"].set("■"))
            self.log(f"Unknown platform: {platform}", "ERROR")
            self.root.after(0, lambda: conn["connect_btn"].configure(text="Connect"))

```

```

        return

    conn["account"] = account
    # Also keep legacy references for backward compatibility
    if platform == "Tradovate":
        self.tradovate_account = account
    else:
        self.topstepx_account = account

    def _update_ui():
        conn["status_var"].set("■")
        conn["connect_btn"].configure(text="Connected", fg_color="#14532D")
        if hasattr(conn.get("status_lbl"), "configure"):
            conn["status_lbl"].configure(text_color="#22C55E")
        # Update global status
        connected = sum(1 for c in self._broker_connections.values() if c.get("account"))
        total = len(self._broker_connections)
        self.broker_status_var.set(f"{connected}/{total} connected")
        self.root.after(0, _update_ui)
        self.log(f"■ {firm_name} connected to {platform} ({mode})")
        # Release build: no status polling / hedge guard side-effects
        if not RELEASE_DISABLE_STATUS_POLL:
            self.root.after(0, self._start_status_polling)
        if not RELEASE_DISABLE_HEDGE_GUARD:
            self.root.after(500, self._start_hedge_protector)

    except Exception as e:
        def _fail():
            conn["status_var"].set("■")
            conn["connect_btn"].configure(text="Retry", fg_color="#450A0A")
            self.root.after(0, _fail)
            self.log(f"■ {firm_name} connection failed: {e}", "ERROR")

    threading.Thread(target=_do_connect, daemon=True).start()

def _connect_all_brokers(self):
    """Connect all prop firms that have credentials filled in."""
    if not self._broker_connections:
        self.log("■ Load trades first to see prop firm connections", "WARN")
        return

    to_connect = []
    for firm, conn in self._broker_connections.items():
        user = conn["user_entry"].get().strip()
        pwd = conn["pass_entry"].get().strip()
        if user and pwd and not conn.get("account"):
            to_connect.append(firm)

    if not to_connect:
        self.log("All populated firms are already connected")
        return

    self.log(f"Connecting {len(to_connect)} broker(s)...")

    def _do_connect_all():
        for firm in to_connect:
            self.root.after(0, lambda f=firm: self._connect_broker_firm(f))
            time.sleep(3) # stagger connections

    threading.Thread(target=_do_connect_all, daemon=True).start()

```

```

def _auto_connect_populated_brokers(self):
    """Auto-connect all broker rows that have dashboard credentials pre-filled."""
    to_connect = []
    for firm, conn in self._broker_connections.items():
        user = conn["user_entry"].get().strip()
        pwd = conn["pass_entry"].get().strip()
        if user and pwd and not conn.get("account"):
            to_connect.append(firm)

    if not to_connect:
        return

    self.log(f"■ Auto-connecting {len(to_connect)} broker(s) from dashboard credentials...")

    def _do_auto():
        for firm in to_connect:
            self.root.after(0, lambda f=firm: self._connect_broker_firm(f))
            time.sleep(3) # stagger to avoid overwhelming

    threading.Thread(target=_do_auto, daemon=True).start()

def _show_trade_history(self, firm_name):
    """Show a popup window with full Tradovate trade history for the given firm."""
    conn = self._broker_connections.get(firm_name)
    if not conn or not conn.get("account"):
        self.log(f"■ {firm_name} is not connected – connect first", "WARN")
        return

    account = conn["account"]
    if not hasattr(account, 'get_trade_history'):
        self.log(f"■ {firm_name} account does not support trade history", "WARN")
        return

    self.log(f"■ Loading trade history for {firm_name}...")

    def _fetch_and_show():
        try:
            data = account.get_trade_history()
            if not data:
                self.root.after(0, lambda: self.log(f"■ No history data for {firm_name}", "WARN"))
                return
            self.root.after(0, lambda d=data: self._build_history_window(firm_name, d))
        except Exception as e:
            self.root.after(0, lambda: self.log(f"■ History fetch failed: {e}", "ERROR"))

    threading.Thread(target=_fetch_and_show, daemon=True).start()

def _build_history_window(self, firm_name, data):
    """Build and display the trade history popup window.
    data is a list of account dicts (one per account under this login)."""
    if isinstance(data, dict):
        data = [data] # backwards compat – single account

    win = tk.Toplevel(self.root)
    acct_count = len(data)
    win.title(f"■ Trade History – {firm_name} ({acct_count} account{'s' if acct_count > 1 else ''})")
    win.geometry("800x660")
    win.configure(bg="#0A0E17")
    win.attributes("-topmost", True)
    win.after(500, lambda: win.attributes("-topmost", False))

```



```

            ("Qty", 35), ("Contract", 70), ("Price", 80)]:
tk.Label(fhdr, text=col_name, width=col_w // 7, bg="#151D2B", fg="#64748B",
        font=("Consolas", 9, "bold"), anchor="center").pack(side="left", padx=1)

for i, fill in enumerate(fills):
    bg = "#0D1320" if i % 2 == 0 else "#0A0E17"
    fr = tk.Frame(f_inner, bg=bg)
    fr.pack(fill="x")
    action_color = "#22C55E" if fill['action'] == 'Buy' else "#EF4444"
    tk.Label(fr, text=fill['date'], width=11, bg=bg, fg="#CBD5E1",
            font=("Consolas", 9), anchor="center").pack(side="left", padx=1)
    tk.Label(fr, text=fill['time'], width=9, bg=bg, fg="#94A3B8",
            font=("Consolas", 9), anchor="center").pack(side="left", padx=1)
    tk.Label(fr, text=fill['action'], width=5, bg=bg, fg=action_color,
            font=("Consolas", 9, "bold"), anchor="center").pack(side="left", padx=1)
    tk.Label(fr, text=str(fill['qty']), width=5, bg=bg, fg="#CBD5E1",
            font=("Consolas", 9), anchor="center").pack(side="left", padx=1)
    tk.Label(fr, text=fill['contract'], width=10, bg=bg, fg="#60A5FA",
            font=("Consolas", 9), anchor="center").pack(side="left", padx=1)
    tk.Label(fr, text=f"{fill['price']:.2f}", width=11, bg=bg, fg="#E2E8F0",
            font=("Consolas", 9), anchor="center").pack(side="left", padx=1)

_BROWSER_MONITORED_FIRMS = {
    "Funded Next": {"class_available": "CDP_SCRAPERS_AVAILABLE",
        "account_class": "FundedNextCDPAccount",
        "login_url": "https://app.fundednext.com", "accounts_url": "https://app.fur
        "cdp": True},
    "FundedNext": {"class_available": "CDP_SCRAPERS_AVAILABLE",
        "account_class": "FundedNextCDPAccount",
        "login_url": "https://app.fundednext.com", "accounts_url": "https://app.fur
        "cdp": True},
    # CDP-based scrapers – attach to existing Chrome tab (no Selenium needed)
    "Tradeify": {"class_available": "CDP_SCRAPERS_AVAILABLE", "account_class": "TradeifyAcco
        "login_url": "https://app-f.tradeify.co", "accounts_url": "https://app-f.tr
        "cdp": True},
    "Lucid Trading": {"class_available": "CDP_SCRAPERS_AVAILABLE",
        "account_class": "LucidTradingAccount",
        "login_url": "https://dash.lucidtrading.com", "accounts_url": "https://dash
        "cdp": True},
    "Lucid": {"class_available": "CDP_SCRAPERS_AVAILABLE",
        "account_class": "LucidTradingAccount",
        "login_url": "https://dash.lucidtrading.com", "accounts_url": "https://dash
        "cdp": True},
    "TopStep": {"class_available": "CDP_SCRAPERS_AVAILABLE", "account_class": "TopStepAccou
        "login_url": "https://dashboard.topstep.com", "accounts_url": "https://dash
        "cdp": True},
    "MFFU": {"class_available": "CDP_SCRAPERS_AVAILABLE", "account_class": "MFFUAccount"
        "login_url": "https://myfundedfutures.com", "accounts_url": "https://myfunde
        "cdp": True},
    "My Funded Futures": {"class_available": "CDP_SCRAPERS_AVAILABLE", "account_class": "MFFUAccount"
        "login_url": "https://myfundedfutures.com", "accounts_url": "https://myfunde
        "cdp": True},
}

def _auto_launch_propfirm_browsers(self, active_firms):
    """
    Detect which active prop firms have browser-based dashboards and store
    them so the UI dashboard buttons know what to launch. Does NOT auto-open
    browsers – the user clicks the "Dashboard" button in the broker row.
    Tradovate/TopStepX auto-connect with credentials as before.

```

```

"""
if RELEASE_DISABLE_PROP_DASHBOARD_ACCESS:
    return

for firm in active_firms:
    cfg = self._BROWSER_MONITORED_FIRMS.get(firm)
    if not cfg:
        continue
    class_avail = globals().get(cfg["class_available"], False)
    if class_avail:
        self.log(f"■ {firm} has a dashboard – click the Dashboard button to connect")

def _launch_propfirm_dashboard(self, firm_name):
    """
    Launch a Chrome browser for the prop firm's dashboard, show login dialog.
    Called when user clicks the 'Dashboard' button in a broker row.
    For CDP-based scrapers, attaches to an existing Chrome tab instead of launching new Chrome.
    """
    if RELEASE_DISABLE_PROP_DASHBOARD_ACCESS:
        self.log("■ Prop firm dashboard access is disabled in this release", "WARN")
        return

    cfg = self._BROWSER_MONITORED_FIRMS.get(firm_name)
    if not cfg:
        # Case-insensitive fallback
        for _bk, _bv in self._BROWSER_MONITORED_FIRMS.items():
            if _bk.lower() == firm_name.lower():
                cfg = _bv
                break

    if not cfg:
        self.log(f"■ No dashboard config for {firm_name}", "WARN")
        return

    # Skip if already connected
    if firm_name in self._propfirm_browsers and self._propfirm_browsers[firm_name]:
        try:
            if self._propfirm_browsers[firm_name].is_connected():
                self.log(f"■ {firm_name} dashboard already open")
                return
        except Exception:
            pass

    class_avail = globals().get(cfg["class_available"], False)
    if not class_avail:
        self.log(f"■ {firm_name} browser module not available", "WARN")
        return

    # CDP-based scrapers: launch Chrome (or reuse) → open tab → show login dialog → attach
    if cfg.get("cdp"):
        self.log(f"■ Attaching to {firm_name} dashboard tab (CDP)...")
        def _do_cdp_launch():
            try:
                account_cls = globals().get(cfg["account_class"])
                if not account_cls:
                    self.root.after(0, lambda: self.log(f"■ {firm_name}: scraper class not found",
                                                         "ERROR"))
                return

                login_url = cfg.get("login_url", "")

                # Ensure Chrome debug is running and the tab is open

```

```

try:
    ensure_chrome_debug(url=login_url, port=9222)
except FileNotFoundError as e:
    self.root.after(0, lambda err=str(e):
        self.log(f"■ {err}", "ERROR"))
    return

acct = account_cls(debug_port=9222)

# Try to attach immediately (tab may already be logged in)
try:
    acct.login(open_url=login_url)
    self._propfirm_browsers[firm_name] = acct
    self.root.after(0, lambda:
        self.log(f"■ {firm_name} dashboard connected via CDP"))
    acct_mapping = self._autofill_challenge_fees(firm_name, acct)
    self._cached_acct_mappings[firm_name] = acct_mapping or {}
    self._auto_detect_breached_accounts(firm_name, acct, acct_mapping=acct_mapping)
    return
except ConnectionError:
    # Tab not ready yet – show login dialog
    pass

# Show dialog asking user to log in, then retry
self.root.after(0, lambda: self._show_cdp_login_dialog(
    firm_name, acct, cfg))

except Exception as e:
    self.root.after(0, lambda err=str(e):
        self.log(f"■ {firm_name} CDP launch failed: {err}", "ERROR"))
    threading.Thread(target=_do_cdp_launch, daemon=True).start()
    return

def _show_propfirm_login_dialog(self, firm_name, account, cfg):
    """Show a dialog prompting the user to log in to the prop firm dashboard."""
    if CTK_AVAILABLE:
        dialog = ctk.CTkToplevel(self.root)
    else:
        dialog = tk.Toplevel(self.root)
    dialog.title(f"{firm_name} Dashboard Login")
    dialog.geometry("380x140")
    dialog.resizable(False, False)
    dialog.attributes("-topmost", True)

    status_var = tk.StringVar(
        value=f"Please log in to {firm_name} in the browser window,\nthen click the button below.")
    if CTK_AVAILABLE:
        status_lbl = ctk.CTkLabel(dialog, textvariable=status_var, font=("Consolas", 11),
                                wraplength=340, justify="center")
        status_lbl.pack(pady=(16, 8), padx=16)
        confirm_btn = ctk.CTkButton(dialog, text="■ I've logged in", width=160, height=30,
                                    fg_color="#14532D", hover_color="#166534",
                                    font=("Consolas", 11), text_color="#D1FAE5",
                                    command=lambda: self._on_login_confirmed(firm_name, account, cfg,
                                                                                dialog, status_var, confirm_btn))
        confirm_btn.pack(pady=(4, 16))
    else:
        status_lbl = tk.Label(dialog, textvariable=status_var, font=("Consolas", 10),
                              wraplength=340, justify="center")
        status_lbl.pack(pady=(16, 8), padx=16)

```

```

        confirm_btn = tk.Button(dialog, text="■ I've logged in", width=20,
                                command=lambda: self._on_login_confirmed(firm_name, account, cfg, dialog, status_var, confirm_btn))
        confirm_btn.pack(pady=(4, 16))

def _on_login_confirmed(self, firm_name, account, cfg, dialog, status_var, confirm_btn):
    """Called when user clicks 'I've logged in' – disable button and verify in background."""
    confirm_btn.configure(state="disabled")
    status_var.set("■ Verifying login...")
    threading.Thread(target=self._verify_propfirm_browser,
                    args=(firm_name, account, cfg, dialog, status_var),
                    daemon=True).start()

def _show_cdp_login_dialog(self, firm_name, acct, cfg):
    """Show dialog for CDP-based scrapers asking user to log in, then attach."""
    if CTK_AVAILABLE:
        dialog = ctk.CTkToplevel(self.root)
    else:
        dialog = tk.Toplevel(self.root)
    dialog.title(f"{firm_name} Dashboard Login")
    dialog.geometry("400x150")
    dialog.resizable(False, False)
    dialog.attributes("-topmost", True)

    status_var = tk.StringVar(
        value=f"Please log in to {firm_name} in the Chrome window,\nthen click the button below.")
    if CTK_AVAILABLE:
        ctk.CTkLabel(dialog, textvariable=status_var, font=("Consolas", 11),
                    wraplength=360, justify="center").pack(pady=(16, 8), padx=16)
        confirm_btn = ctk.CTkButton(
            dialog, text="■ I've logged in", width=160, height=30,
            fg_color="#14532D", hover_color="#166634",
            font=("Consolas", 11), text_color="#D1FAE5",
            command=lambda: self._on_cdp_login_confirmed(
                firm_name, acct, cfg, dialog, status_var, confirm_btn))
        confirm_btn.pack(pady=(4, 16))
    else:
        tk.Label(dialog, textvariable=status_var, font=("Consolas", 10),
                wraplength=360, justify="center").pack(pady=(16, 8), padx=16)
        confirm_btn = tk.Button(
            dialog, text="■ I've logged in", width=20,
            command=lambda: self._on_cdp_login_confirmed(
                firm_name, acct, cfg, dialog, status_var, confirm_btn))
        confirm_btn.pack(pady=(4, 16))

def _on_cdp_login_confirmed(self, firm_name, acct, cfg, dialog, status_var, confirm_btn):
    """Attach CDP after user confirms they've logged in."""
    confirm_btn.configure(state="disabled")
    status_var.set("■ Attaching to dashboard...")

def _attach():
    try:
        login_url = cfg.get("login_url", "")
        acct.login(open_url=login_url)
        self._propfirm_browsers[firm_name] = acct
        self.root.after(0, lambda: self.log(
            f"■ {firm_name} dashboard connected via CDP"))
        acct_mapping = self._autofill_challenge_fees(firm_name, acct)
        self._cached_acct_mappings[firm_name] = acct_mapping or {}
        self._auto_detect_breached_accounts(firm_name, acct, acct_mapping=acct_mapping)

```

```

        self.root.after(0, dialog.destroy)
    except ConnectionError:
        self.root.after(0, lambda: status_var.set(
            f"■ Could not find {firm_name} tab.\n"
            f"Make sure {cfg.get('login_url', '')} is open and loaded."))
        self.root.after(0, lambda: confirm_btn.configure(state="normal"))
    except Exception as e:
        self.root.after(0, lambda err=str(e): status_var.set(f"■ {err}"))
        self.root.after(0, lambda: confirm_btn.configure(state="normal"))

threading.Thread(target=_attach, daemon=True).start()

def _verify_propfirm_browser(self, firm_name, account, cfg, dialog=None, status_var=None):
    """Navigate to accounts page, verify login, auto-fill fees, close dialog."""
    def _update_status(msg):
        if status_var:
            self.root.after(0, lambda m=msg: status_var.set(m))

    def _close_dialog():
        if dialog:
            self.root.after(0, lambda: dialog.destroy())

    try:
        _update_status("■ Navigating to accounts page...")
        account.driver.get(cfg["accounts_url"])
        time.sleep(3)

        current_url = account.driver.current_url
        if "accounts" in current_url or account.is_connected():
            account.logged_in = True
            account._login_timestamp = time.time()
            self.root.after(0, lambda:
                self.log(f"■ {firm_name} browser connected – monitoring active"))

            # Auto-fill challenge fees from billing history
            _update_status("■ Fetching billing history...")
            acct_mapping = self._autofill_challenge_fees(firm_name, account)

            # Auto-detect breached accounts and mark as failed
            _update_status("■ Checking breach status...")
            self._cached_acct_mappings[firm_name] = acct_mapping or {}
            self._auto_detect_breached_accounts(firm_name, account, acct_mapping=acct_mapping)

            _update_status("■ Connected!")
            time.sleep(1)
            _close_dialog()
        else:
            self.root.after(0, lambda:
                self.log(f"■ {firm_name} may not be logged in (URL: {current_url}). "
                    f"You can reconnect later.", "WARN"))
            _update_status("■ Login not detected – try again")
            # Re-enable the button so user can retry
            if dialog:
                self.root.after(0, lambda: [
                    w.configure(state="normal")
                    for w in dialog.wininfo_children()
                    if hasattr(w, 'configure') and isinstance(w, (
                        ctk.CTkButton if CTK_AVAILABLE else tk.Button,))
                ])
    except Exception as e:

```

```

        self.root.after(0, lambda err=str(e):
            self.log(f"■ {firm_name} browser verification failed: {err}", "ERROR"))
        _update_status(f"■ Error: {str(e)[:50]}")

def _autofill_challenge_fees(self, firm_name, account):
    """
    Scrape billing history from the prop firm dashboard and auto-fill
    the 'Fee', 'Date Purchased', and 'Account #' fields of active evaluations.
    Then push the updated evaluations to the dashboard.

    For FundedNext Futures accounts, also resolves the billing login ID
    to the Tradovate account name (e.g. 945576089 -> FNFTCHHARRISONOUKA85625).

    Returns the acct_mapping dict (or {}) so callers can reuse it for breach detection.
    """
    acct_mapping = {}
    try:
        if not hasattr(account, 'get_billing_history'):
            self.root.after(0, lambda:
                self.log(f"■ {firm_name}: No get_billing_history method – skipping"))
            return acct_mapping

        self.root.after(0, lambda:
            self.log(f"■ {firm_name}: Scraping billing history..."))

        # Prefer API-based billing if available (richer data with login field)
        if hasattr(account, 'get_billing_via_api'):
            billing = account.get_billing_via_api()
        else:
            billing = account.get_billing_history()
        if not billing:
            self.root.after(0, lambda:
                self.log(f"■ {firm_name}: No billing records found"))
            return acct_mapping

        self.root.after(0, lambda b=len(billing):
            self.log(f"■ {firm_name}: Found {b} billing record(s)"))

        # Get account mapping (billing login -> Tradovate account name)
        if hasattr(account, 'get_account_mapping'):
            try:
                self.root.after(0, lambda:
                    self.log(f"■ {firm_name}: Fetching account mapping..."))
                acct_mapping = account.get_account_mapping()
                if acct_mapping:
                    self.root.after(0, lambda n=len(acct_mapping):
                        self.log(f"■ {firm_name}: Got {n} login→account mapping(s)"))
            except Exception as e:
                self.root.after(0, lambda err=str(e):
                    self.log(f"■ {firm_name}: Account mapping failed: {err}", "WARN"))

        # Log all billing entries for visibility
        for i, entry in enumerate(billing):
            acct = entry.get("account_no", "?")
            amt = entry.get("paid_amount", "?")
            status = entry.get("status", "?")
            date = entry.get("date", "?")
            pkg = entry.get("funding_package", "?")
            self.root.after(0, lambda idx=i, a=acct, m=amt, s=status, d=date, p=pkg:
                self.log(

```

```

        f"    ■ Billing #{idx+1}: Acct={a} | Amount={m} | Status={s} | Date={d} | Pkg={p}"

# Build lookup: account_no -> {amount, date, package}
# First entry wins for billing_by_acct (preserves original fee under login key).
# Last entry wins for billing_by_login (latest fee maps to current Tradovate account).
# This handles resets: e.g. FundedNext login 946645337 has $139.04 (new) + $142.13 (reset).
# The $139.04 stays under "946645337", while $142.13 maps to the active Tradovate account.
billing_by_acct = {}
billing_by_size = {}
billing_by_login = {}
for entry in billing:
    acct_no = (entry.get("account_no") or "").strip()
    status = (entry.get("status") or "").strip().upper()
    amount = entry.get("paid_amount_numeric", 0.0)
    bill_date = (entry.get("date") or "").strip()
    pkg = (entry.get("funding_package") or "").strip()
    login_id = str(entry.get("login") or acct_no).strip()
    if acct_no and amount > 0 and status == "APPROVED":
        info = {"amount": amount, "date": bill_date, "package": pkg, "account_no": acct_no,
                "login": login_id}
        # First entry wins – keeps oldest fee under the raw account_no key
        if acct_no not in billing_by_acct:
            billing_by_acct[acct_no] = info
        # Extract numeric size from package string (e.g. "50000" from "Futures Legacy Challenge")
        import re as _re
        size_match = _re.search(r'(\d{4,})', pkg.replace(",", ""))
        if size_match:
            size_key = int(size_match.group(1))
            if size_key not in billing_by_size:
                billing_by_size[size_key] = dict(info)
        # Last entry wins – latest fee maps to current active Tradovate account
        billing_by_login[login_id] = dict(info)

# Resolve billing login IDs to Tradovate account names via account mapping
# This enriches billing_by_acct so matching by FNFT/TDFY/LFE account name works.
# billing_by_login has the LATEST entry per login → maps to current active Tradovate account
for login_id, bill_info in billing_by_login.items():
    if login_id in acct_mapping:
        tv_name = acct_mapping[login_id].get("tradovate_account_name")
        if tv_name and tv_name not in billing_by_acct:
            enriched = dict(bill_info)
            enriched["tradovate_account_name"] = tv_name
            billing_by_acct[tv_name] = enriched
        if tv_name:
            self.root.after(0, lambda lid=login_id, tv=tv_name, amt=bill_info["amount"]:
                self.log(f"    ■ Mapped billing login {lid} → {tv} (${amt:.2f})"))

# Log challenge fees per account
for acct_key, info in billing_by_acct.items():
    self.root.after(0, lambda a=acct_key, t=info["amount"]:
        self.log(f"    ■ {a}: ${t:.2f}"))

if not billing_by_acct and not billing_by_size:
    self.root.after(0, lambda:
        self.log(f"    ■ {firm_name}: No APPROVED billing entries with amount > 0"))
    return acct_mapping

self.root.after(0, lambda n=len(billing_by_acct), s=len(billing_by_size),
    ak=list(billing_by_acct.keys()), sk=list(billing_by_size.keys()):
    self.log(f"    ■ {firm_name}: {n} by-acct ({ak}), {s} by-size ({sk})"))

```

```

# Log active trade rows for debugging
row_count = len(self._active_trade_rows)
self.root.after(0, lambda c=row_count:
    self.log(f"■ {firm_name}: Checking {c} active trade row(s) for missing Fee/Date"))

# Canonical firm name for comparison
canonical_firm = self._FIRM_MAP.get(firm_name, firm_name)

# Match billing to active evaluations missing Fee or Date Purchased
updated_evals = []
filled_count = 0
for row_data in self._active_trade_rows:
    ev = row_data.get("eval")
    if not ev:
        continue

    # Check if eval belongs to this prop firm (flexible matching)
    ev_firm = ev.get("Prop Firm", "")
    ev_canonical = self._FIRM_MAP.get(ev_firm, ev_firm)
    if ev_canonical != canonical_firm and ev_firm != firm_name:
        continue

    acct_challenge = (ev.get("Account #") or "").strip()
    acct_funded = (ev.get("Account #.1") or "").strip()
    existing_fee = (ev.get("Fee") or "").strip()
    existing_date = (ev.get("Date Purchased") or "").strip()

    fee_filled = False
    if existing_fee:
        try:
            existing_val = float(existing_fee.replace("$", "").replace(",", ""))
            fee_filled = existing_val > 0
        except ValueError:
            fee_filled = existing_fee not in ("", "$0", "$0.00", "0")
    date_filled = bool(existing_date)

    # Log each eval's current state
    self.root.after(0, lambda f=ev_firm, ac=acct_challenge, af=acct_funded,
        ef=existing_fee, ed=existing_date, ff=fee_filled, df=date_filled:
        self.log(f" ■ Eval: Firm={f} | Acct#={ac} | Acct#.1={af} | "
            f"Fee='{ef}' ({'✓' if ff else 'X'}) | Date='{ed}' ({'✓' if df else 'X'})"))

# Note: we no longer skip early – fee is always updated if it differs from billing

# Try matching by Account # or Account #.1
matched = None
matched_via = ""
for acct_key, label in [(acct_challenge, "Account #"), (acct_funded, "Account #.1")]:
    if not acct_key:
        continue
    # Direct match
    if acct_key in billing_by_acct:
        matched = billing_by_acct[acct_key]
        matched_via = f"{label} direct: {acct_key}"
        break
    # Partial match: billing account_no may be a substring
    for bill_acct, bill_info in billing_by_acct.items():
        if bill_acct in acct_key or acct_key in bill_acct:
            matched = bill_info
            matched_via = f"{label} partial: eval={acct_key} ~ bill={bill_acct}"

```



```

        break
    if matched:
        break

# Fallback: match by Account Size when Account # is empty
if not matched and billing_by_size:
    ev_size_str = (ev.get("Account Size") or "").strip()
    import re as _re
    size_match = _re.search(r'(\d[\d,]*)', ev_size_str.replace(",", ""))
    if size_match:
        ev_size = int(size_match.group(1))
        if ev_size in billing_by_size:
            matched = billing_by_size[ev_size]
            matched_via = f"Account Size fallback: ${ev_size:,} → bill acct {matched.get('acct_challenge')}"

if matched:
    billing_fee = f"${matched['amount']:.2f}"
    changes = []
    # Always overwrite fee with billing value
    ev["Fee"] = billing_fee
    changes.append(f"Fee={billing_fee}")
    if not date_filled and matched["date"]:
        ev["Date Purchased"] = matched["date"]
        changes.append(f"DatePurchased={matched['date']}")
    # Also set Date Started from billing date when empty
    existing_start = (ev.get("Date Started") or "").strip()
    if not existing_start and matched["date"]:
        ev["Date Started"] = matched["date"]
        changes.append(f"DateStarted={matched['date']}")
    # Auto-fill Account # from account mapping when empty
    tv_name = matched.get("tradovate_account_name")
    if not acct_challenge and tv_name:
        ev["Account #"] = tv_name
        changes.append(f"Account#={tv_name}")
    elif not acct_challenge and matched.get("login") and str(matched["login"]) in acct_mapping:
        tv_name = acct_mapping[str(matched["login"])]["tradovate_account_name"]
        if tv_name:
            ev["Account #"] = tv_name
            changes.append(f"Account#={tv_name}")
    filled_count += 1
    updated_evals.append(ev)
    self.root.after(0, lambda v=matched_via, c=", ".join(changes):
        self.log(f"    ■ Matched ({v}): {c}"))
else:
    acct_display = acct_challenge or acct_funded or "no-acct"
    bill_keys = list(billing_by_acct.keys())
    self.root.after(0, lambda a=acct_display, bk=bill_keys:
        self.log(f"    ■ No billing match for {a} (billing accts: {bk})"))

if not filled_count:
    self.root.after(0, lambda:
        self.log(f"    ■ {firm_name}: No accounts needed Fee/Date updates"))
# Still compute per-firm summary even when no evals updated
total_fees, total_payouts, records = self._compute_firm_billing(
    firm_name, account, billing)
self._firm_billing_summary[firm_name] = {
    "total_fees": total_fees, "total_payouts": total_payouts, "records": records}
self.root.after(0, lambda f=firm_name, tf=total_fees, tp=total_payouts:
    self.log(f"    ■ {f} Summary – Total Fees: ${tf:.2f} | Total Payouts: ${tp:.2f}"))

```

```

        return acct_mapping

self.root.after(0, lambda c=filled_count:
    self.log(f"■ {firm_name}: Pushing {c} updated eval(s) to dashboard..."))

# Push updated evaluations to dashboard
email = self.client_email_entry.get().strip()
dashboard_url = self.url_entry.get().strip().rstrip('/')
if not email or not dashboard_url:
    self.root.after(0, lambda:
        self.log(f"■ Cannot push fee updates – no email/dashboard URL", "WARN"))
    return acct_mapping

# Collect ALL active evals (server expects the full set)
all_evals = [rd.get("eval") for rd in self._active_trade_rows if rd.get("eval")]

# Debug: log the Fee value we're about to push
for ev_debug in all_evals:
    ev_fee = ev_debug.get("Fee", "N/A")
    ev_acct = ev_debug.get("Account #", "?")
    self.root.after(0, lambda f=ev_fee, a=ev_acct:
        self.log(f"    ■ Pushing eval Acct={a} Fee={f}"))

self.root.after(0, lambda n=len(all_evals):
    self.log(f"■ {firm_name}: Sending {n} total eval(s) in push payload"))

payload = {
    "email": email,
    "evaluations": all_evals,
    "statistics": {},
    "dropdown_options": {},
    "firm_billing": self._firm_billing_summary,
    "force_fields": ["Fee", "Date Purchased", "Date Started"],
}

try:
    response = _gzip_post(
        f"{dashboard_url}/api/client/push",
        payload,
        timeout=30
    )
    if response.status_code == 200:
        data = response.json()
        if data.get("status") == "success":
            self.root.after(0, lambda c=filled_count:
                self.log(
                    f"■ Auto-filled Fee & Date Purchased for {c} account(s) → synced to dash
                ))
        else:
            self.root.after(0, lambda m=data.get('message', 'Unknown'):
                self.log(f"■ Fee sync response: {m}", "WARN"))
    else:
        self.root.after(0, lambda s=response.status_code:
            self.log(f"■ Fee sync failed: HTTP {s}", "WARN"))
except Exception as e:
    self.root.after(0, lambda err=str(e):
        self.log(f"■ Fee sync error: {err}", "WARN"))

# ■■ Per-firm totals: aggregate challenge fees + payouts ■■
total_fees, total_payouts, records = self._compute_firm_billing(
    firm_name, account, billing)

```

[illegible]

```

self.root.after(0, lambda: self.log("■ Status polling started (every 5 min)"))
# Run FIRST poll immediately, then schedule repeating every 5 min
def _first_poll():
    try:
        self._poll_tradovate_balances()
    except Exception as e:
        self.root.after(0, lambda err=str(e):
            self.log(f"■ First status poll error: {err}", "WARN"))
threading.Thread(target=_first_poll, daemon=True).start()
self.root.after(300_000, self._poll_account_status)

def _poll_account_status(self):
    """Self-rescheduling timer – fetches Tradovate balances and updates P1 status."""
    if RELEASE_DISABLE_STATUS_POLL:
        return

    if not self._status_poll_active:
        return

    def _poll_all():
        try:
            self._poll_tradovate_balances()
        except Exception as e:
            self.root.after(0, lambda err=str(e):
                self.log(f"■ Status poll error: {err}", "WARN"))

    threading.Thread(target=_poll_all, daemon=True).start()
    self.root.after(300_000, self._poll_account_status)

def _poll_tradovate_balances(self):
    """Fetch live balances from Tradovate broker API and update P1/Status on evaluations."""
    if RELEASE_DISABLE_STATUS_POLL:
        return

    # ■■ 1. Gather all Tradovate account balances from broker connections ■■
    tv_balances = {} # {account_name: netLiq}
    for firm_name, conn in list(self._broker_connections.items()):
        tv_account = conn.get("account")
        if not tv_account:
            continue
        # Support both TradovateAccount (_api_fetch) and DOM-based stats
        if not hasattr(tv_account, '_api_fetch'):
            continue
        try:
            # First check if browser/driver is alive
            try:
                _ = tv_account.driver.title
            except Exception:
                self.root.after(0, lambda fn=firm_name:
                    self.log(f"■ Status poll: {fn} browser not accessible", "WARN"))
                continue

            raw = tv_account._api_fetch("/account/list")
            if not raw or not isinstance(raw, list):
                continue
            for acct in raw:
                aid = acct.get('id')
                aname = acct.get('name', '')
                if not aid or not aname:
                    continue

```

```

        snapshot = tv_account._api_fetch(
            "/cashBalance/getCashBalanceSnapshot", "POST",
            {"accountId": aid})
        if snapshot and isinstance(snapshot, dict):
            net_liq = snapshot.get('netLiq')
            if net_liq is not None:
                tv_balances[aname] = float(net_liq)
    except Exception as e:
        self.root.after(0, lambda fn=firm_name, err=str(e):
            self.log(f"■ Status poll: API error for {fn}: {err}", "WARN"))

if not tv_balances:
    self.root.after(0, lambda: self.log("■ Status poll: no Tradovate balances fetched", "WARN"))
    return

# Log Tradovate accounts on first poll only
if not hasattr(self, '_poll_logged_tv_accounts'):
    self._poll_logged_tv_accounts = True
    self.root.after(0, lambda n=len(tv_balances), names=list(tv_balances.keys()):
        self.log(f"■ Status poll: {n} Tradovate account(s): {names}"))

# Build case-insensitive lookup
tv_lower = {k.lower(): (k, v) for k, v in tv_balances.items()}

# ■■ 2. Fetch evaluations from dashboard ■■
email = self.client_email_entry.get().strip()
dashboard_url = self.url_entry.get().strip().rstrip('/')
if not email or not dashboard_url:
    return

try:
    r = requests.post(
        f"{dashboard_url}/api/client/data",
        json={"email": email},
        headers={"Content-Type": "application/json"},
        timeout=15)
    if r.status_code != 200:
        self.root.after(0, lambda s=r.status_code:
            self.log(f"■ Status poll: dashboard returned HTTP {s}", "WARN"))
        return
    all_evals = r.json().get("evaluations", [])
except Exception as e:
    # Only log dashboard connection errors once to avoid spam
    if not getattr(self, '_poll_dashboard_err_logged', False):
        self._poll_dashboard_err_logged = True
        self.root.after(0, lambda err=str(e):
            self.log(f"■ Status poll: dashboard not reachable (will retry silently)", "WARN"))
    return

# Dashboard is reachable – reset error flag
self._poll_dashboard_err_logged = False

if not all_evals:
    return

# ■■ 3. Match evaluations to Tradovate balances and compute status ■■
any_changed = False
for ev in all_evals:
    if not ev or ev.get("_deleted"):
        continue

```

```

acct_challenge = (ev.get("Account #") or "").strip()
acct_funded = (ev.get("Account #.1") or "").strip()
current_p1 = (ev.get("Status P1") or "").strip().lower()
current_status = (ev.get("Status") or "").strip().lower()

# Determine which account and field to check
has_funded = bool(acct_funded)
if has_funded:
    acct_key = acct_funded
    status_field = "Status"
    current_check = current_status
else:
    acct_key = acct_challenge
    status_field = "Status P1"
    current_check = current_p1

if not acct_key:
    continue

# Skip accounts already passed or failed
if "pass" in current_check or any(kw in current_check for kw in ("fail", "breach", "delete",
    "closed", "ended", "lost")):
    continue

# ■■ Find matching Tradovate balance ■■
acct_key_lower = acct_key.lower()
match = tv_lower.get(acct_key_lower)
if not match:
    # Try partial match (account name might be substring)
    for tv_name_lower, tv_pair in tv_lower.items():
        if tv_name_lower in acct_key_lower or acct_key_lower in tv_name_lower:
            match = tv_pair
            break
if not match:
    # Account not found on any connected Tradovate – mark as Failed
    miss_key = f"_miss_logged_{acct_key}"
    if not self._last_known_statuses.get(miss_key):
        self._last_known_statuses[miss_key] = True
        self.root.after(0, lambda a=acct_key, tvk=list(tv_balances.keys()):
            self.log(f"    ■ No match for '{a}' in Tradovate accounts {tvk} → marking Fail"))
    if current_check not in ("fail", "failed", "breach", "delete", "deleted", "closed", "ende
        "lost"):
        ev[status_field] = "Fail"
        any_changed = True
        self.root.after(0, lambda a=acct_key, sf=status_field:
            self.log(f"    ■ {a}: {sf}='Fail' – not found on Tradovate"))
    continue

tv_name_orig, bal = match

# ■■ Get starting balance from Account Size ■■
size_str = (ev.get("Account Size") or "").replace("$", "").replace(",", "").strip().lower()
if size_str.endswith("k"):
    start = float(size_str[:-1]) * 1000
elif size_str.replace(".", "").isdigit():
    start = float(size_str)
else:
    start = 50000.0

# ■■ Get live targets from cached mapping if available ■■

```

```

canonical_firm = self._FIRM_MAP.get(ev.get("Prop Firm", ""), ev.get("Prop Firm", ""))
phase = "Funded" if has_funded else "Challenge"
live_target = None
live_min_eq = None
for _fn, mapping in self._cached_acct_mappings.items():
    for _mk, info in mapping.items():
        tv_name = info.get("tradovate_account_name", "")
        if tv_name and (tv_name.lower() in acct_key_lower or acct_key_lower in tv_name.lower()):
            live_target = info.get("profit_target")
            live_min_eq = info.get("min_equity")
            if info.get("starting_balance"):
                try:
                    start = float(info["starting_balance"])
                except (ValueError, TypeError):
                    pass
            break
    if live_target is not None:
        break

# ■■ Compute status ■■
# Use profit targets and min equity from _PROFIT_TARGETS table.
# Only mark Failed if balance <= minimum equity limit.
# Only mark Pass if balance >= start + profit target.
# Otherwise In Progress (if balance moved) or skip.
computed = None
try:
    # Get profit target and min equity for this firm/phase
    pt_target = None
    pt_min_eq = None

    # First try live targets from cached prop firm mapping
    if live_target is not None:
        pt_target = float(live_target)
    if live_min_eq is not None:
        pt_min_eq = float(live_min_eq)

    # Fallback to hardcoded profit targets table
    if pt_target is None and self.prop_firm_mgr:
        targets = self.prop_firm_mgr._PROFIT_TARGETS.get(canonical_firm, {})
        phase_key = "Challenge" if phase == "Challenge" else "Funded"
        t = targets.get(phase_key)
        if t is not None:
            pt_target = float(t)

    # Determine status
    if pt_min_eq is not None and bal <= pt_min_eq:
        computed = "Fail"
    elif pt_target is not None and bal >= (start + pt_target):
        computed = "Pass"
    elif abs(bal - start) > 0.50:
        computed = "In Progress"
    # else: balance hasn't moved, skip
except (ValueError, TypeError):
    continue

if not computed:
    continue

# Set the status on the eval
if computed.lower() != current_check:

```

```

        ev[status_field] = computed
        any_changed = True
        self.root.after(0, lambda a=acct_key, c=computed, b=bal, s=start:
            self.log(f"    ■ {a}: {c} (bal=${b:,.2f}, start=${s:,.0f}))")

# ■■ 4. Always push all evals so dashboard stays in sync ■■
if not any_changed:
    return

payload = {
    "email": email,
    "evaluations": all_evals,
    "statistics": {},
    "dropdown_options": {},
    "force_fields": ["Status P1", "Status"],
}
try:
    resp = requests.post(
        f"{dashboard_url}/api/client/push",
        json=payload,
        headers={"Content-Type": "application/json"},
        timeout=30)
    if resp.status_code == 200:
        rj = resp.json()
        if rj.get("status") == "success":
            self.root.after(0, lambda: self.log(f"■ Status poll: synced to dashboard"))
        else:
            self.root.after(0, lambda m=rj.get('message', '?'):
                self.log(f"■ Status poll push rejected: {m}", "WARN"))
    else:
        self.root.after(0, lambda s=resp.status_code, t=resp.text[:200]:
            self.log(f"■ Status poll push HTTP {s}: {t}", "WARN"))
except Exception as e:
    self.root.after(0, lambda err=str(e):
        self.log(f"■ Status poll push error: {err}", "WARN"))

def _auto_detect_breached_accounts(self, firm_name, account, acct_mapping=None):
    """
    Check if any evaluations have been breached on FundedNext
    and auto-set their status to 'Failed' with the breach reason.

    Uses the account mapping (from get_account_mapping) which includes
    breach status from the React fiber, plus optionally the account-overview
    API for detailed breach info (breach reason, reset price etc).

    Fetches ALL evaluations from the dashboard (not just _active_trade_rows)
    since breached accounts may already be filtered out of the active rows.
    """
    try:
        if not acct_mapping:
            if hasattr(account, 'get_account_mapping'):
                acct_mapping = account.get_account_mapping()
            if not acct_mapping:
                return

        # Build reverse lookup: tradovate_account_name -> mapping info
        tv_to_info = {}
        for login_id, info in acct_mapping.items():
            tv_name = info.get("tradovate_account_name")
            if tv_name:

```



```

        tv_to_info[tv_name] = info
        tv_to_info[login_id] = info # also index by login

self.root.after(0, lambda keys=list(tv_to_info.keys()):
    self.log(f"■ Breach check: mapping keys = {keys}"))

# Canonical firm name for comparison
canonical_firm = self._FIRM_MAP.get(firm_name, firm_name)

# Fetch ALL evaluations from dashboard (not just _active_trade_rows,
# since breached accounts may have been filtered out as inactive)
email = self.client_email_entry.get().strip()
dashboard_url = self.url_entry.get().strip().rstrip('/')
all_evals = []
if email and dashboard_url:
    try:
        r = requests.get(
            f"{dashboard_url}/api/client/data",
            params={"email": email},
            timeout=15
        )
        if r.status_code == 200:
            all_evals = r.json().get("evaluations", [])
    except Exception as e:
        self.root.after(0, lambda err=str(e):
            self.log(f"■ Breach check: couldn't fetch evals: {err}", "WARN"))

if not all_evals:
    # Fallback to active trade rows
    all_evals = [rd.get("eval") for rd in self._active_trade_rows if rd.get("eval")]

breached_count = 0
updated_evals = []

self.root.after(0, lambda n=len(all_evals), fn=firm_name, cf=canonical_firm:
    self.log(f"■ Breach check: {n} eval(s), firm_name='{fn}', canonical='{cf}'))

for ev in all_evals:
    if not ev:
        continue

    # Skip deleted
    if ev.get("_deleted"):
        continue

    # Check if eval belongs to this prop firm (with alias support)
    ev_firm = ev.get("Prop Firm", "")
    ev_canonical = self._FIRM_MAP.get(ev_firm, ev_firm)
    firm_match = (ev_canonical == canonical_firm or ev_firm == firm_name)
    if not firm_match:
        # Check aliases: firm name variants that refer to the same firm
        _FIRM_ALIASES = {
            "funded next": ["fundednext"],
            "fundednext": ["funded next"],
            "my funded futures": ["mffu"],
            "mffu": ["my funded futures"],
        }
        ev_aliases = _FIRM_ALIASES.get(ev_firm.lower(), [])
        firm_match = firm_name.lower() in ev_aliases or canonical_firm.lower() in ev_aliases
    if not firm_match:

```

```

        continue

    acct_challenge = (ev.get("Account #") or "").strip()
    acct_funded = (ev.get("Account #.1") or "").strip()
    current_status_p1 = (ev.get("Status P1") or "").strip().lower()
    current_status = (ev.get("Status") or "").strip().lower()

    self.root.after(0, lambda ac=acct_challenge, af=acct_funded, sp=current_status_p1,
                    s=current_status:
        self.log(f"■ Breach eval: Acct#='{ac}' Acct#.1='{af}' P1='{sp}' Status='{s}'"))

    # Skip if already marked as failed/breached
    if any(kw in current_status_p1 for kw in self._INACTIVE_KEYWORDS):
        if not acct_funded or any(kw in current_status for kw in self._INACTIVE_KEYWORDS):
            self.root.after(0, lambda: self.log(f"■ Breach check: already inactive, skipping"))
            continue

    # Find matching account in mapping
    matched_info = None
    for acct_key in [acct_challenge, acct_funded]:
        if not acct_key:
            continue
        if acct_key in tv_to_info:
            matched_info = tv_to_info[acct_key]
            break
    # Partial match
    for tv_name, info in tv_to_info.items():
        if tv_name in acct_key or acct_key in tv_name:
            matched_info = info
            break
    if matched_info:
        break

    if not matched_info:
        # Fallback: match by Account Size to any mapping entry
        ev_size_str = (ev.get("Account Size") or "").strip()
        import re as _re
        size_match = _re.search(r'(\d[\d,]*)', ev_size_str.replace(",", ""))
        if size_match:
            ev_size = int(size_match.group(1))
            for _lid, info in acct_mapping.items():
                sb = info.get("starting_balance")
                if sb and int(sb) == ev_size:
                    matched_info = info
                    self.root.after(0, lambda sz=ev_size:
                        self.log(f"■ Breach: matched by Account Size ${sz:,}"))
                    break

    if not matched_info:
        self.root.after(0, lambda ac=acct_challenge, af=acct_funded:
            self.log(f"■ Breach check: no mapping match for Acct#='{ac}' Acct#.1='{af}'"))
        continue

    # ■■ Determine account status: Fail / Pass / In Progress ■■
    breached = matched_info.get("breached")
    acct_display = acct_challenge or acct_funded
    changes = []

    # Detect current phase for this eval
    has_funded_acct = bool(acct_funded)

```

```

if has_funded_acct:
    phase_for_status = "Funded"
    status_field = "Status"
    current_status_check = current_status
else:
    phase_for_status = "Challenge"
    status_field = "Status P1"
    current_status_check = current_status_p1

# Skip if already marked with a terminal status
if any(kw in current_status_check for kw in self._INACTIVE_KEYWORDS):
    self.root.after(0, lambda: self.log(f"■ Status check: already inactive, skipping"))
    continue

if breached and breached != 0:
    # Account is breached – get detailed info from API if available
    breach_reason = matched_info.get("breachedby") or "Breached"
    account_id = matched_info.get("account_id")
    if account_id and hasattr(account, 'get_account_overview'):
        try:
            overview = account.get_account_overview(account_id)
            if overview:
                details = overview.get("account_details", {})
                breach_reason = details.get("breached_by") or breach_reason
        except Exception:
            pass

    ev[status_field] = "Fail"
    changes.append(f"{status_field}='Fail'")
    breached_count += 1
    self.root.after(0, lambda a=acct_display, c=" ".join(changes):
        self.log(f" ■ BREACHED: {a} → {c}"))
elif self.prop_firm_mgr:
    # ■■ Balance-based auto-status: Pass / In Progress / Fail ■■
    # Prefer live profit_target / min_equity from dashboard API
    acct_balance = matched_info.get("balance")
    acct_starting = matched_info.get("starting_balance") or matched_info.get(
        "initial_balance")
    live_target = matched_info.get("profit_target")
    live_min_eq = matched_info.get("min_equity")
    if acct_balance is not None:
        try:
            bal = float(acct_balance)
            start = float(acct_starting) if acct_starting else 50000.0

            if live_min_eq is not None or live_target is not None:
                # ■■ Use live values from prop firm dashboard ■■
                min_eq = float(live_min_eq) if live_min_eq is not None else None
                target = float(live_target) if live_target is not None else None

                if min_eq is not None and bal < min_eq:
                    computed = "Fail"
                elif target is not None and bal >= (start + target):
                    computed = "Pass"
                elif abs(bal - start) > 0.50:
                    # Balance differs from starting → a trade was placed
                    computed = "In Progress"
                elif min_eq is not None and target is not None:
                    computed = "In Progress"
                else:

```

```

        computed = self.prop_firm_mgr.compute_account_status(
            canonical_firm, phase_for_status, bal, start,
            breached=False)
    else:
        # ■■ No live data – fall back to hardcoded targets ■■
        computed = self.prop_firm_mgr.compute_account_status(
            canonical_firm, phase_for_status, bal, start,
            breached=False)

    # If balance is exactly the starting balance, keep "Not Started"
    if computed == "In Progress" and abs(bal - start) < 0.50:
        if current_status_check in ("not started", ""):
            computed = None # no change – no trade placed yet

    # Only update if status actually changed
    last_key = f"{acct_display}_{status_field}"
    last_known = self._last_known_statuses.get(last_key)
    if computed and computed.lower(
        ) != current_status_check and computed != last_known:
        self._last_known_statuses[last_key] = computed
        if computed == "Pass":
            ev[status_field] = "Pass"
            changes.append(f"{status_field}='Pass'")
            self.root.after(0, lambda a=acct_display, b=bal, s=start:
                self.log(f" ■ PASS: {a} – balance=${b:,.2f} (start=${s:,.0f})")
        elif computed == "In Progress":
            ev[status_field] = "In Progress"
            changes.append(f"{status_field}='In Progress'")
            self.root.after(0, lambda a=acct_display, b=bal, s=start:
                self.log(
                    f" ■ IN PROGRESS: {a} – balance=${b:,.2f} (start=${s:,.0f})")
        elif computed == "Fail":
            ev[status_field] = "Fail"
            changes.append(f"{status_field}='Fail'")
            breached_count += 1
            self.root.after(0, lambda a=acct_display, b=bal, s=start:
                self.log(f" ■ FAIL: {a} – balance=${b:,.2f} (start=${s:,.0f})")
    except (ValueError, TypeError) as _e:
        self.root.after(0, lambda a=acct_display, b=acct_balance, err=str(_e):
            self.log(f"■ {a}: couldn't parse balance '{b}': {err}", "WARN"))

    if changes:
        updated_evals.append(ev)

    status_updates = len(updated_evals)
    if not status_updates:
        self.root.after(0, lambda:
            self.log(f"■ {firm_name}: No status changes detected"))
    return

self.root.after(0, lambda c=status_updates, b=breached_count:
    self.log(f"■ {firm_name}: {c} status update(s) ({b} breached) – pushing to dashboard...")

# Push to dashboard
email = self.client_email_entry.get().strip()
dashboard_url = self.url_entry.get().strip().rstrip('/')
if not email or not dashboard_url:
    return

payload = {

```

```

        "email": email,
        "evaluations": all_evals,
        "statistics": {},
        "dropdown_options": {},
        "force_fields": ["Status P1", "Status"],
    }

    try:
        response = _gzip_post(
            f"{dashboard_url}/api/client/push",
            payload,
            timeout=30
        )
        if response.status_code == 200:
            data = response.json()
            if data.get("status") == "success":
                self.root.after(0, lambda c=status_updates, b=breached_count:
                    self.log(f"■ Auto-status: {c} update(s) ({b} failed) → synced to dashboard"))
            else:
                self.root.after(0, lambda m=data.get('message', 'Unknown'):
                    self.log(f"■ Breach sync response: {m}", "WARN"))
        else:
            self.root.after(0, lambda s=response.status_code:
                self.log(f"■ Breach sync failed: HTTP {s}", "WARN"))
    except Exception as e:
        self.root.after(0, lambda err=str(e):
            self.log(f"■ Breach sync error: {err}", "WARN"))

except Exception as e:
    self.root.after(0, lambda err=str(e):
        self.log(f"■ {firm_name} breach detection failed: {err}", "WARN"))

def _get_broker_for_firm(self, firm_name):
    """Get the connected broker account for a specific prop firm."""
    conn = self._broker_connections.get(firm_name)
    if conn and conn.get("account"):
        return conn["account"]
    # If the firm has a row in broker connections but isn't connected, do NOT
    # fall back – it means the user chose not to connect this firm.
    if conn is not None:
        return None
    # Legacy fallback: only for setups without multi-firm broker panel
    # Auto-detect TopStep firms to use TopStepX
    if "topstep" in firm_name.lower():
        if self.topstepx_account:
            return self.topstepx_account
        return None
    platform = self.broker_var.get()
    if platform == "Tradovate" and self.tradovate_account:
        return self.tradovate_account
    if platform == "TopStepX" and self.topstepx_account:
        return self.topstepx_account
    return None

def _get_trade_config(self):
    """Get current trade configuration from blueprint."""
    if not self.prop_firm_mgr:
        return None
    firm = self.prop_firm_var.get()
    phase = self.phase_var.get()

```

```

size = self.acct_size_var.get()
config = self.prop_firm_mgr.get_strategy_config(firm, phase, size)
return config

def _get_mt5_trading_api(self):
    """Get or create the MT5 trading API from companion's existing MT5 connection."""
    if self.trading_api and hasattr(self.trading_api, 'is_connected') and self.trading_api.is_connected:
        return self.trading_api
    # Try to create from companion's MT5 credentials
    login = self.mt5_login.get().strip()
    pwd = self.mt5_password.get().strip()
    server = self.mt5_server.get().strip()
    if login and pwd and server:
        try:
            self.trading_api = MT5API(login, pwd, server)
            if self.trading_api.connect():
                return self.trading_api
        except Exception as e:
            self.log(f"MT5 trading API connection failed: {e}", "ERROR")
    return None

def _ensure_mt5_for_signals(self):
    """Ensure MT5 is initialized so indicators can fetch price data.

    Works for both hedging and non-hedging clients:
    - If pusher already connected MT5, it's already initialized → returns True
    - Otherwise, tries to initialize from saved credentials

    Returns:
    bool: True if MT5 is ready for price data queries.
    """
    if not MT5_AVAILABLE:
        return False
    # Already connected via pusher?
    if self.pusher.connected:
        return True
    # Already connected via trading API?
    if mt5_terminal_info():
        return True
    # Try to connect using saved credentials
    login = self.mt5_login.get().strip()
    pwd = self.mt5_password.get().strip()
    server = self.mt5_server.get().strip()
    if login and pwd and server:
        success, msg = self.pusher.connect_mt5(login, pwd, server)
        if success:
            self.log("■ MT5 connected for signal data")
            return True
        self.log(f"■ MT5 auto-connect failed: {msg}", "WARN")
    return False

# ===== Daily Bias Persistence =====

def _get_daily_bias(self, firms):
    """Get or create today's direction bias per prop firm.
    Persisted to trader_bias.json so it survives app restarts.
    Resets automatically on a new calendar day (EAT)."""
    from datetime import datetime, timedelta, timezone
    EAT = timezone(timedelta(hours=3))
    today_str = datetime.now(EAT).strftime("%Y-%m-%d")

```

```

bias_path = os.path.join(os.path.dirname(__file__), "trader_bias.json")
saved = {}
if os.path.exists(bias_path):
    try:
        with open(bias_path, 'r') as f:
            saved = json.load(f)
    except Exception:
        saved = {}

# Reset if date changed
if saved.get("date") != today_str:
    saved = {"date": today_str, "firms": {}}

firm_bias = saved.get("firms", {})
changed = False
# Assign in a deterministic order so balancing is reproducible
for firm in sorted(firms):
    if firm not in firm_bias:
        # Diversify: pick the direction held by FEWER existing firms.
        # This prevents every firm from coincidentally landing on BUY.
        buys = sum(1 for d in firm_bias.values() if d == "buy")
        sells = sum(1 for d in firm_bias.values() if d == "sell")
        if buys > sells:
            firm_bias[firm] = "sell"
        elif sells > buys:
            firm_bias[firm] = "buy"
        else:
            # Tie (or first firm) – random
            firm_bias[firm] = random.choice(["buy", "sell"])
        changed = True

if changed or saved.get("date") != today_str:
    saved["date"] = today_str
    saved["firms"] = firm_bias
    try:
        with open(bias_path, 'w') as f:
            json.dump(saved, f, indent=2)
    except Exception:
        pass

    return {f: firm_bias[f] for f in firms}

# ===== Indicator-Based Signal =====

# Signal functions mapped by name → (callable, buy_values, sell_values)
# buy_values/sell_values are the return strings that map to buy/sell
_SIGNAL_INDICATORS = None # populated lazily

@classmethod
def _get_indicator_map(cls):
    """Build indicator map lazily (needs imports to be resolved)."""
    if cls._SIGNAL_INDICATORS is not None:
        return cls._SIGNAL_INDICATORS
    indicators = {}
    try:
        indicators["RSI"] = (get_rsi_signal, {"buy"}, {"sell"})
    except Exception:
        pass
    try:
        indicators["MACD"] = (get_macd_signal, {"buy"}, {"sell"})

```

```

except Exception:
    pass
try:
    indicators["Stochastic"] = (get_stochastic_signal, {"buy"}, {"sell"})
except Exception:
    pass
try:
    indicators["CCI"] = (get_cci_signal, {"buy"}, {"sell"})
except Exception:
    pass
try:
    indicators["Supertrend"] = (get_supertrend_signal, {"bullish"}, {"bearish"})
except Exception:
    pass
try:
    indicators["Momentum"] = (get_momentum_signal, {"bullish"}, {"bearish"})
except Exception:
    pass
try:
    indicators["BollingerBands"] = (get_bb_signal, {"lower"}, {"upper"})
except Exception:
    pass
cls._SIGNAL_INDICATORS = indicators
return indicators

def _get_signal_direction(self, mt5_symbol, timeframe=None, num_indicators=3):
    """Generate a trade direction by polling a random subset of indicators.

    Picks `num_indicators` random indicators, queries each on the given
    MT5 symbol, and uses majority vote to decide buy vs sell.
    Falls back to random if no indicators produce a signal.
    """
    if not MT5_AVAILABLE:
        self.root.after(0, lambda: self.log("■ MetaTrader5 import unavailable – using random direction", "WARN"))
        return random.choice(["buy", "sell"])
    mt5_mod = mt5
    if timeframe is None:
        timeframe = mt5_mod.TIMEFRAME_M5

    # Ensure MT5 is connected (non-hedging clients may not have it open)
    if not mt5_mod.terminal_info():
        self._ensure_mt5_for_signals()
    if not mt5_mod.terminal_info():
        self.root.after(0, lambda: self.log("■ MT5 not connected – using random direction", "WARN"))
        return random.choice(["buy", "sell"])

    indicators = self._get_indicator_map()
    if not indicators:
        self.root.after(0, lambda: self.log("■ No signal indicators available – using random", "WARN"))
        return random.choice(["buy", "sell"])

    # Pick a random subset
    available = list(indicators.keys())
    pick_count = min(num_indicators, len(available))
    chosen = random.sample(available, pick_count)

    buy_votes = 0
    sell_votes = 0
    details = []

```



```

for name in chosen:
    func, buy_vals, sell_vals = indicators[name]
    try:
        result = func(mt5_symbol, timeframe)
        if result is None:
            details.append(f"{name}=neutral")
            continue
        sig = result.lower() if isinstance(result, str) else str(result).lower()
        if sig in buy_vals:
            buy_votes += 1
            details.append(f"{name}=BUY")
        elif sig in sell_vals:
            sell_votes += 1
            details.append(f"{name}=SELL")
        else:
            details.append(f"{name}={sig}")
    except Exception as e:
        details.append(f"{name}=err")

detail_str = ", ".join(details)
if buy_votes > sell_votes:
    direction = "buy"
elif sell_votes > buy_votes:
    direction = "sell"
else:
    direction = random.choice(["buy", "sell"])
detail_str += " (tie→random)"

self.root.after(0, lambda d=detail_str, dir=direction:
    self.log(f"    ■ Signal vote: {d} → {dir.upper()}"))
return direction

# ===== Version History & Rollback =====

def save_config(self):
    """Save configuration to file."""
    config = {
        "dashboard_url": self.url_entry.get(), # Save dynamic URL
        "client_email": self.client_email_entry.get(),
        "sheet_url": self.sheet_url_entry.get(),
        "mt5_login": self.mt5_login.get(),
        "mt5_server": self.mt5_server.get()
    }
    # Trading engine settings
    if hasattr(self, 'broker_var'):
        config["broker_platform"] = self.broker_var.get()
        config["trading_mode"] = self.trading_mode_var.get()
        config["prop_firm"] = self.prop_firm_var.get()
        config["phase"] = self.phase_var.get()
        config["account_size"] = self.acct_size_var.get()
        config["hedge_mode"] = self.hedge_mode_var.get()
        config["direction"] = self.direction_var.get()
        config["strategy"] = self.strategy_var.get()

    config_path = os.path.join(os.path.dirname(__file__), "trader_config.json")
    with open(config_path, 'w') as f:
        json.dump(config, f, indent=2)

    self.log("Configuration saved")
    messagebox.showinfo("Saved", "Configuration saved successfully")

```

```

def load_config(self):
    """Load configuration from file."""
    config_path = os.path.join(os.path.dirname(__file__), "trader_config.json")
    if os.path.exists(config_path):
        try:
            with open(config_path, 'r') as f:
                config = json.load(f)

            # Do NOT load dashboard_url - keep hardcoded TradeOpps
            # saved_url = config.get('dashboard_url')
            # if saved_url:
            #     self.url_entry.delete(0, tk.END)
            #     self.url_entry.insert(0, saved_url)

            if config.get('client_email'):
                self.client_email_entry.delete(0, tk.END)
                self.client_email_entry.insert(0, config.get('client_email', ''))

            if config.get('sheet_url'):
                self.sheet_url_entry.delete(0, tk.END)
                self.sheet_url_entry.insert(0, config.get('sheet_url', ''))

            if config.get('mt5_login'):
                self.mt5_login.delete(0, tk.END)
                self.mt5_login.insert(0, config.get('mt5_login', ''))

            if config.get('mt5_server'):
                self.mt5_server.delete(0, tk.END)
                self.mt5_server.insert(0, config.get('mt5_server', ''))

            # Trading engine settings
            if hasattr(self, 'broker_var'):
                if config.get('broker_platform'):
                    self.broker_var.set(config['broker_platform'])
                if config.get('trading_mode'):
                    self.trading_mode_var.set(config['trading_mode'])
                if config.get('prop_firm'):
                    self.prop_firm_var.set(config['prop_firm'])
                    self._on_prop_firm_change()
                if config.get('phase'):
                    self.phase_var.set(config['phase'])
                if config.get('account_size'):
                    self.acct_size_var.set(config['account_size'])
                if config.get('hedge_mode'):
                    self.hedge_mode_var.set(config['hedge_mode'])
                if config.get('direction'):
                    self.direction_var.set(config['direction'])
                if config.get('strategy'):
                    self.strategy_var.set(config['strategy'])

            self.log("Configuration loaded")
        except Exception as e:
            self.log(f"Failed to load config: {e}", "ERROR")

def run(self):
    """Run the application."""
    self.root.mainloop()

```

Method: TradeOpssAIApp.__init__

```
def __init__(self)
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **lambda** — Uses a single-expression anonymous function — typically as the `key=` for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.
- **__init__** — The constructor. Runs after Python has allocated the instance; assigns starting attribute values to `self`.

Step by step (top-level body)

1. **if** `CTK_AVAILABLE`: branches conditionally (with an **else/elif** arm).
2. Calls `self.root.title(...)` for its side effect.
3. Calls `self.root.geometry(...)` for its side effect.
4. Calls `self.root.minsize(...)` for its side effect.
5. **if** `CTK_AVAILABLE`: branches conditionally (with an **else/elif** arm).
6. Calls `self.root.resizable(...)` for its side effect.
7. Calls `self.root.bind(...)` for its side effect.
8. Assigns `self._section_logo = None`.
9. **try** block with 1 **except** clause.
10. Assigns `self.pusher = MT5DataPusher()`.
11. Assigns `self.auto_push_enabled = False`.
12. Assigns `self.auto_push_thread = None`.
13. Assigns `self._push_lock = threading.Lock()`.
14. Assigns `self._push_in_progress = False`.
15. ... and 23 more statement(s) in the body.

Code (copy-paste safe)

```
def __init__(self):
    # ■■ CTK root window ■■
    if CTK_AVAILABLE:
        ctk.set_appearance_mode("Dark")
        ctk.set_default_color_theme("blue")
        self.root = ctk.CTk()
    else:
        self.root = tk.Tk()
    self.root.title(f"Tradeopss AI v{APP_VERSION}")
    self.root.geometry("680x612")
```

```

self.root.minsize(595, 527)
if CTK_AVAILABLE:
    self.root.configure(fg_color=self.C_BG)
else:
    self.root.configure(bg=self.C_BG)
self.root.resizable(True, True)
self.root.bind("<Control-m>", self._test_min_equity)

# Set Window Icon + section logo
self._section_logo = None # small logo for section headers
try:
    if hasattr(sys, '_MEIPASS'):
        _base = sys._MEIPASS
    else:
        _base = os.path.dirname(os.path.abspath(__file__))
    from PIL import Image as PILImage, ImageTk
    png_path = os.path.join(_base, 'logo.png')
    if os.path.exists(png_path):
        _pil_icon = PILImage.open(png_path).convert('RGBA')
        bbox = _pil_icon.getbbox()
        if bbox:
            _pil_icon = _pil_icon.crop(bbox)
        w, h = _pil_icon.size
        s = max(w, h)
        # Dark background so logo is visible in taskbar
        _sq = PILImage.new('RGBA', (s, s), (6, 14, 26, 255))
        _sq.paste(_pil_icon, ((s - w) // 2, (s - h) // 2), _pil_icon)
        # Taskbar icon (64x64)
        _taskbar = _sq.resize((64, 64), PILImage.LANCZOS)
        self._app_icon = ImageTk.PhotoImage(_taskbar)
        self.root.iconphoto(True, self._app_icon)
        self.root.after(200, lambda: self.root.iconphoto(True, self._app_icon))
        # Small logo for section headers (18x18)
        _small = _sq.resize((18, 18), PILImage.LANCZOS)
        self._section_logo = ImageTk.PhotoImage(_small)
    else:
        ico_path = os.path.join(_base, 'logo.ico')
        if os.path.exists(ico_path):
            self.root.iconbitmap(ico_path)
            self.root.after(200, lambda: self.root.iconbitmap(ico_path))
except Exception:
    pass

self.pusher = MT5DataPusher()
self.auto_push_enabled = False
self.auto_push_thread = None
self._push_lock = threading.Lock()
self._push_in_progress = False
self._push_pending = False
self.client_info = None

# Auto-trade scheduler state
self.auto_trade_enabled = False
self.auto_trade_thread = None
self._auto_trade_stop = threading.Event()
self._auto_trade_scheduled_dt = None

# Trading engine state
self.trading_api = None
self.tradovate_account = None

```

```

self.topstepx_account = None
self._broker_connections = {
    } # {firm_name: {user_entry, pass_entry, status_var, connect_btn, account, row_frame}}
self._propfirm_browsers = {} # {firm_name: FundedNextAccount/etc} for dashboard scraping
self.prop_firm_mgr = PropFirmManager() if PROP_FIRM_AVAILABLE else None
self._auto_trading_stop = threading.Event()
self._auto_trading_thread = None
self._direction_locks = {}
self._active_trade_rows = []
self._firm_billing_summary = {} # {firm_name: {total_fees, total_payouts, records: [...]}}
self.push_billing_btn = None
self._hedge_protector = None
self._status_poll_active = False # real-time status polling flag
self._last_known_statuses = {} # {acct_display: last_computed_status} for change detection
self._cached_acct_mappings = {} # {firm_name: {acct_key: info}} cached on connect

self._show_login_screen()

```

Method: TradeOpssAIApp._show_login_screen

```
def _show_login_screen(self)
```

Show a full-screen login/email verification screen — always required.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns saved_email = "".
2. Assigns config_path = os.path.join(os.path.dirname(__file__), 'trader_config.js....
3. **if** os.path.exists(config_path): branches conditionally.
4. Assigns self._login_frame = ctk.CTkFrame(self.root, fg_color=self.C_BG) if CTK_AVAILA....
5. Calls self._login_frame.pack(...) for its side effect.
6. Assigns center = ctk.CTkFrame(self._login_frame, fg_color=self.C_BG_SEC, c....
7. Calls center.place(...) for its side effect.
8. **if** CTK_AVAILABLE: branches conditionally.

Code (copy-paste safe)

```

def _show_login_screen(self):
    """Show a full-screen login/email verification screen — always required."""
    # Try loading saved email to pre-fill

```

```

saved_email = ""
config_path = os.path.join(os.path.dirname(__file__), "trader_config.json")
if os.path.exists(config_path):
    try:
        with open(config_path, 'r') as f:
            cfg = json.load(f)
            saved_email = cfg.get('client_email', '').strip()
    except Exception:
        pass

self._login_frame = ctk.CTkFrame(self.root, fg_color=self.C_BG) if CTK_AVAILABLE else \
    tk.Frame(self.root, bg=self.C_BG)
self._login_frame.pack(fill="both", expand=True)

# Center box
center = ctk.CTkFrame(self._login_frame, fg_color=self.C_BG_SEC, corner_radius=12,
                      border_width=1, border_color=self.C_BORDER) if CTK_AVAILABLE else \
    tk.Frame(self._login_frame, bg="#161B22")
center.place(relx=0.5, rely=0.45, anchor="center")

if CTK_AVAILABLE:
    ctk.CTkLabel(center, text="Client Verification",
                 font=("Segoe UI", 13),
                 text_color=self.C_TEXT).pack(pady=(30, 16))

    self._login_email = ctk.CTkEntry(center, placeholder_text="Enter Registered Email",
                                     width=300, height=40,
                                     font=("Segoe UI", 12),
                                     fg_color=self.C_BG_THIRD,
                                     border_color=self.C_BORDER,
                                     text_color=self.C_TEXT)
    self._login_email.pack(pady=(0, 16), padx=30)
    self._login_email.bind("<Return>", lambda e: self._verify_login())

    # Pre-fill saved email
    if saved_email:
        self._login_email.insert(0, saved_email)

    self._login_btn = ctk.CTkButton(center, text="VERIFY ACCESS",
                                    width=300, height=40,
                                    command=self._verify_login,
                                    fg_color=self.C_ACCENT,
                                    hover_color=self.C_ACCENT_HV,
                                    font=("Segoe UI", 12, "bold"))
    self._login_btn.pack(pady=(0, 12), padx=30)

    self._login_status = ctk.CTkLabel(center, text="",
                                      font=("Segoe UI", 11),
                                      text_color=self.C_ERROR)
    self._login_status.pack(pady=(0, 24))

```

Method: TradeOpssAIApp._verify_login

```
def _verify_login(self)
```

Verify the email against the dashboard API.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.

Step by step (top-level body)

1. Assigns email = self._login_email.get().strip().
2. if not email: branches conditionally.
3. Calls self._login_btn.configure(...) for its side effect.
4. Calls self._login_status.configure(...) for its side effect.
5. Calls self.root.update_idletasks(...) for its side effect.
6. Defines a nested function _check(...).
7. Calls threading.Thread(target=_check, daemon=True).start(...) for its side effect.

Code (copy-paste safe)

```
def _verify_login(self):
    """Verify the email against the dashboard API."""
    email = self._login_email.get().strip()
    if not email:
        self._login_status.configure(text="Please enter an email address.")
        return

    self._login_btn.configure(state="disabled", text="VERIFYING...")
    self._login_status.configure(text="", text_color=self.C_TEXT_DIM)
    self.root.update_idletasks()

    def _check():
        try:
            response = requests.post(
                "https://www.tradeopss.com/api/client/auth",
                json={"email": email},
                headers={"Content-Type": "application/json"},
                timeout=30
            )
            if response.status_code == 200:
                data = response.json()
                if data.get("status") == "success":
                    self.root.after(0, lambda: self._finish_login(email))
                    return
                else:
                    msg = data.get("message", "Email not found")
                    self.root.after(0, lambda: self._login_fail(f"Access Denied: {msg}"))
                    return
            else:
                self.root.after(0, lambda: self._login_fail(f"Server error ({response.status_code})"))
                return
```

```

        except requests.exceptions.ConnectionError:
            self.root.after(0, lambda: self._login_fail("Cannot connect to server"))
        except Exception as e:
            self.root.after(0, lambda: self._login_fail(str(e)))

    threading.Thread(target=_check, daemon=True).start()

```

Method: TradeOpssAIApp._login_fail

```
def _login_fail(self, msg)

    Show login failure message.
```

Step by step (top-level body)

1. Calls self._login_status.configure(...) for its side effect.
2. Calls self._login_btn.configure(...) for its side effect.

Code (copy-paste safe)

```
def _login_fail(self, msg):
    """Show login failure message."""
    self._login_status.configure(text=msg, text_color=self.C_ERROR)
    self._login_btn.configure(state="normal", text="VERIFY ACCESS")

```

Method: TradeOpssAIApp._finish_login

```
def _finish_login(self, email)

    Tear down login screen, build main UI, and auto-lookup.
```

Step by step (top-level body)

1. if hasattr(self, '_login_frame'): branches conditionally.
2. Calls self.setup_ui(...) for its side effect.
3. Calls self.load_config(...) for its side effect.
4. Calls self.client_email_entry.delete(...) for its side effect.
5. Calls self.client_email_entry.insert(...) for its side effect.
6. Calls self.root.after(...) for its side effect.

Code (copy-paste safe)

```
def _finish_login(self, email):
    """Tear down login screen, build main UI, and auto-lookup."""
    if hasattr(self, '_login_frame'):
        self._login_frame.destroy()

    self.setup_ui()
    self.load_config()

    # Set the email and trigger lookup
    self.client_email_entry.delete(0, tk.END)
    self.client_email_entry.insert(0, email)
    self.root.after(200, self.lookup_client)

```


Method: TradeOpssAIApp._section_card

```
def _section_card(self, parent, title='', icon='', **kw)
```

Create a styled card frame with optional title strip.

Why these Python concepts were used

- ****kw** — Accepts arbitrary keyword arguments. Usually for forwarding to another function (a thin wrapper) or to support optional named parameters without listing them all in the signature.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns card = ctk.CTkFrame(parent, fg_color=self.C_BG_SEC, corner_radiu....
2. if title and CTK_AVAILABLE: branches conditionally.
3. return card.

Code (copy-paste safe)

```
def _section_card(self, parent, title="", icon="", **kw):
    """Create a styled card frame with optional title strip."""
    card = ctk.CTkFrame(parent, fg_color=self.C_BG_SEC, corner_radius=8,
                        border_width=1, border_color=self.C_BORDER) if CTK_AVAILABLE else \
        tk.Frame(parent, bg='#161B22')
    if title and CTK_AVAILABLE:
        hdr = ctk.CTkFrame(card, fg_color="transparent", height=26)
        hdr.pack(fill="x", padx=10, pady=(6, 0))
        hdr.pack_propagate(False)
        if self._section_logo:
            lbl = ctk.CTkLabel(hdr, text="", image=self._section_logo,
                              width=18, height=18)
            lbl.pack(side="left", padx=(0, 6))
        elif icon:
            ctk.CTkLabel(hdr, text=icon, font=("Segoe UI", 12)).pack(side="left", padx=(0, 6))
        ctk.CTkLabel(hdr, text=title, font=("Segoe UI", 10, "bold"),
                    text_color=self.C_GOLD).pack(side="left")
    return card
```

Method: TradeOpssAIApp._ctk_entry

```
def _ctk_entry(self, parent, width=200, show=None, placeholder=None)
```

Step by step (top-level body)

1. if CTK_AVAILABLE: branches conditionally (with an **else/elif** arm).

Code (copy-paste safe)

```
def _ctk_entry(self, parent, width=200, show=None, placeholder=None):
    if CTK_AVAILABLE:
        kw = dict(width=width, height=32, fg_color=self.C_BG_THIRD,
                  border_color=self.C_BORDER, text_color=self.C_TEXT,
                  font=("Segoe UI", 11))
        if show:
            kw["show"] = show
        if placeholder:
```

```

        kw["placeholder_text"] = placeholder
    return ctk.CTkEntry(parent, **kw)
else:
    e = ttk.Entry(parent, width=width // 8)
    if show:
        e.configure(show=show)
    return e

```

Method: TradeOpssAIApp._ctk_button

```
def _ctk_button(self, parent, text='', command=None, fg=None, hover=None, width=140, **kw)
```

Why these Python concepts were used

- ****kw** — Accepts arbitrary keyword arguments. Usually for forwarding to another function (a thin wrapper) or to support optional named parameters without listing them all in the signature.

Step by step (top-level body)

1. Assigns `fg = fg or self.C_ACCENT`.
2. Assigns `hover = hover or self.C_ACCENT_HV`.
3. `if CTK_AVAILABLE`: branches conditionally (with an **else/elif** arm).

Code (copy-paste safe)

```

def _ctk_button(self, parent, text="", command=None, fg=None, hover=None, width=140, **kw):
    fg = fg or self.C_ACCENT
    hover = hover or self.C_ACCENT_HV
    if CTK_AVAILABLE:
        return ctk.CTkButton(parent, text=text, command=command,
                              fg_color=fg, hover_color=hover,
                              font=("Segoe UI", 11, "bold"),
                              corner_radius=6, height=34, width=width, **kw)
    else:
        return ttk.Button(parent, text=text, command=command)

```

Method: TradeOpssAIApp._status_pill

```
def _status_pill(self, parent, text, bg_color, text_color)
```

Step by step (top-level body)

1. `if CTK_AVAILABLE`: branches conditionally (with an **else/elif** arm).

Code (copy-paste safe)

```

def _status_pill(self, parent, text, bg_color, text_color):
    if CTK_AVAILABLE:
        pill = ctk.CTkFrame(parent, fg_color=bg_color, corner_radius=8, height=22)
        ctk.CTkLabel(pill, text=text, font=("Segoe UI", 9, "bold"),
                      text_color=text_color).pack(padx=8, pady=2)
        return pill
    else:
        lbl = tk.Label(parent, text=text, bg=bg_color, fg=text_color,
                       font=("Segoe UI", 9, "bold"), padx=6, pady=1)
        return lbl

```

Method: TradeOpssAIApp.setup_ui

```
def setup_ui(self)
```

Setup the modern CTK user interface — two-column single-screen layout.

Why these Python concepts were used

- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.

Step by step (top-level body)

1. `if not CTk_AVAILABLE:` branches conditionally.
2. Assigns `outer = ctk.CTkFrame(self.root, fg_color=self.C_BG)`.
3. Calls `outer.pack(...)` for its side effect.
4. Assigns `top_bar = ctk.CTkFrame(outer, fg_color='#1A2332', height=38, corner....`
5. Calls `top_bar.pack(...)` for its side effect.
6. Calls `top_bar.pack_propagate(...)` for its side effect.
7. Calls `ctk.CTkFrame(top_bar, height=3, fg_color=self.C_GOLD, corner_radius=0).pack(...)` for its side effect.
8. Assigns `bar_inner = ctk.CTkFrame(top_bar, fg_color='transparent')`.
9. Calls `bar_inner.pack(...)` for its side effect.
10. Assigns `self._conn_dot = ctk.CTkFrame(bar_inner, width=8, height=8, fg_color='#EF4....`
11. Calls `self._conn_dot.pack(...)` for its side effect.
12. Calls `ctk.CTkLabel(bar_inner, text='MT5', font=('Segoe UI', 9), text_color=self.C_TEXT_DIM).pack(...)` for its side effect.
13. Assigns `self._panel_btn = ctk.CTkButton(bar_inner, text='■ Controls', width=110, h....`
14. Calls `self._panel_btn.pack(...)` for its side effect.
15. ... and 90 more statement(s) in the body.

Code (copy-paste safe)

```
def setup_ui(self):
    """Setup the modern CTK user interface – two-column single-screen layout."""
    if not CTk_AVAILABLE:
        # ■■ Fallback: simple ttk layout ■■
        self.main_canvas = tk.Canvas(self.root, bg=self.C_BG, highlightthickness=0)
        sb = ttk.Scrollbar(self.root, orient="vertical", command=self.main_canvas.yview)
        self.scrollable_frame = ttk.Frame(self.main_canvas)
        self.scrollable_frame.bind("<Configure>",
            lambda e: self.main_canvas.configure(scrollregion=self.main_canvas.bbox("all")))
        self.main_canvas.create_window((0, 0), window=self.scrollable_frame, anchor="nw")
        self.main_canvas.configure(yscrollcommand=sb.set)
        self.main_canvas.pack(side="left", fill="both", expand=True)
        sb.pack(side="right", fill="y")
        main = self.scrollable_frame
        style = ttk.Style(); style.theme_use('clam')
        self.notebook = ttk.Notebook(main)
        self.notebook.pack(fill="both", expand=True, padx=8, pady=4)
        tab_dash = ttk.Frame(self.notebook); self.notebook.add(tab_dash, text="Settings")
        self._build_dashboard_tab(tab_dash)
```



```

# ■■■ Row 0: Compact toolbar (Push + Auto-Trade in one strip) ■■■
toolbar = ctk.CTkFrame(self._live_view, fg_color=self.C_BG_SEC, corner_radius=8,
                        border_width=1, border_color=self.C_BORDER, height=38)
toolbar.grid(row=0, column=0, sticky="ew", pady=(0, 4))
toolbar.pack_propagate(False)

self.push_btn_live = self._ctk_button(toolbar, text="Push Data", command=self.push_data,
                                       fg=self.C_SUCCESS, hover="#16a34a", width=90)
self.push_btn_live.pack(side="left", padx=(8, 4), pady=5)
self.auto_btn_live = self._ctk_button(toolbar, text="Auto-Push", command=self.toggle_auto_push,
                                       fg=self.C_ACCENT, hover=self.C_ACCENT_HV, width=90)
self.auto_btn_live.pack(side="left", padx=(0, 8), pady=5)

# Separator
ctk.CTkFrame(toolbar, width=1, fg_color=self.C_BORDER).pack(side="left", fill="y", pady=6)

self.auto_trade_btn = self._ctk_button(toolbar, text="■ Auto-Trade",
                                       command=self._toggle_auto_trade,
                                       fg=self.C_ACCENT, hover=self.C_ACCENT_HV, width=110)
self.auto_trade_btn.pack(side="left", padx=(8, 4), pady=5)

self.auto_trade_immediate_var = tk.BooleanVar(value=False)
self.auto_trade_signal_var = tk.BooleanVar(value=False)
if CTK_AVAILABLE:
    ctk.CTkCheckBox(toolbar, text="Now", variable=self.auto_trade_immediate_var,
                    font=("Segoe UI", 9), text_color=self.C_TEXT_DIM,
                    fg_color=self.C_ACCENT, border_color=self.C_BORDER,
                    hover_color=self.C_ACCENT_HV, width=40,
                    checkbox_width=16, checkbox_height=16).pack(side="left", padx=(0, 6), pady=5)
    ctk.CTkCheckBox(toolbar, text="Actual Signal", variable=self.auto_trade_signal_var,
                    font=("Segoe UI", 9), text_color=self.C_TEXT_DIM,
                    fg_color="#f59e0b", border_color=self.C_BORDER,
                    hover_color="#d97706", width=90,
                    checkbox_width=16, checkbox_height=16).pack(side="left", padx=(0, 6), pady=5)

self.auto_trade_status_var = tk.StringVar(value="Off")
ctk.CTkLabel(toolbar, textvariable=self.auto_trade_status_var,
              font=("Segoe UI", 9), text_color=self.C_TEXT_DIM).pack(side="left", padx=(0, 6))

self.auto_trade_countdown_var = tk.StringVar(value="")
ctk.CTkLabel(toolbar, textvariable=self.auto_trade_countdown_var,
              font=("Consolas", 9), text_color=self.C_GOLD).pack(side="left")

self.auto_trade_firms_var = tk.StringVar(value="")

if not RELEASE_DISABLE_PUSH_BILLING:
    # Separator before Push Billing
    ctk.CTkFrame(toolbar, width=1, fg_color=self.C_BORDER).pack(side="left", fill="y", pady=6)

    self.push_billing_btn = self._ctk_button(toolbar, text="■ Push Billing",
                                             command=self._push_billing_data,
                                             fg="#f59e0b", hover="#d97706", width=110)
    self.push_billing_btn.pack(side="left", padx=(8, 4), pady=5)

    self._ctk_button(toolbar, text="Save Config", command=self.save_config,
                     fg=self.C_BG_THIRD, hover=self.C_BORDER, width=90).pack(side="right", padx=(0,
                                                                 8), pady=5)

    self._ctk_button(toolbar, text="X Close All", command=self._close_all_trades,
                     fg="#DC2626", hover="#B91C1C", width=90).pack(side="right", padx=(0, 4), pady=5)

```

```

# ■■ Row 1: Active Trades (main area – futuristic terminal) ■■
trades_card = ctk.CTkFrame(self._live_view, fg_color="#000000", corner_radius=10,
                           border_width=1, border_color="#0F4C75")
trades_card.grid(row=1, column=0, sticky="nsew", pady=(0, 4))

# Terminal-style top bezel
trades_bezel = ctk.CTkFrame(trades_card, fg_color="#030D1B", height=36, corner_radius=0)
trades_bezel.pack(fill="x", padx=3, pady=(3, 0))
trades_bezel.pack_propagate(False)

# Traffic-light dots
dot_bar = ctk.CTkFrame(trades_bezel, fg_color="transparent")
dot_bar.pack(side="left", padx=10)
for c in ["#EF4444", "#F59E0B", "#22C55E"]:
    ctk.CTkFrame(dot_bar, width=8, height=8, fg_color=c,
                 corner_radius=4).pack(side="left", padx=2, pady=8)

ctk.CTkLabel(trades_bezel, text="■ ACTIVE TRADES",
             font=("Consolas", 11, "bold"),
             text_color="#00D4FF").pack(side="left", padx=(10, 0))

self.trades_count_var = tk.StringVar(value="[ - ]")
ctk.CTkLabel(trades_bezel, textvariable=self.trades_count_var,
             font=("Consolas", 9), text_color="#3B6978").pack(side="left", padx=14)

self.load_trades_btn = ctk.CTkButton(trades_bezel, text="■ SCAN", width=70, height=24,
                                     command=self._load_active_trades,
                                     fg_color="#0A2647", hover_color="#144272",
                                     border_width=1, border_color="#205295",
                                     font=("Consolas", 9, "bold"),
                                     text_color="#00D4FF", corner_radius=4)
self.load_trades_btn.pack(side="right", padx=10)

# Column headers – grid-aligned with row content
hdr = ctk.CTkFrame(trades_card, fg_color="#060E1A", corner_radius=0, height=26)
hdr.pack(fill="x", padx=3, pady=(1, 0))
hdr.pack_propagate(False)
# 3px accent spacer to match row left bar
ctk.CTkFrame(hdr, width=3, fg_color="transparent").pack(side="left")
for label, w in [("PROP FIRM", 110), ("ACCOUNT", 88), ("SIZE", 68),
                 ("PHASE", 88), ("NEXT", 100)]:
    ctk.CTkLabel(hdr, text=label, width=w,
                 font=("Consolas", 8, "bold"),
                 text_color="#3B6978", anchor="w").pack(side="left", padx=(8, 0))
ctk.CTkLabel(hdr, text="ACTION",
             font=("Consolas", 8, "bold"),
             text_color="#3B6978").pack(side="right", padx=(0, 24))

# Scrollable trade rows (fills remaining space)
self._trades_scroll = ctk.CTkScrollableFrame(trades_card, fg_color="#020A14")
self._trades_scroll.pack(fill="both", expand=True, padx=3, pady=(0, 3))
self._trades_inner = self._trades_scroll

# ■■ Row 2: Live Activity (compact bottom) ■■
display_frame = ctk.CTkFrame(self._live_view, fg_color="#000000", corner_radius=10,
                             border_width=1, border_color="#1E293B")
display_frame.grid(row=2, column=0, sticky="nsew")

# Screen header – monitor bezel
screen_top = ctk.CTkFrame(display_frame, fg_color="#0F172A", height=32, corner_radius=0)

```

```

screen_top.pack(fill="x", padx=3, pady=(3, 0))
screen_top.pack_propagate(False)
dot_row = ctk.CTkFrame(screen_top, fg_color="transparent")
dot_row.pack(side="left", padx=10)
for c in ["#EF4444", "#F59E0B", "#22C55E"]:
    ctk.CTkFrame(dot_row, width=8, height=8, fg_color=c,
                 corner_radius=4).pack(side="left", padx=2, pady=8)
ctk.CTkLabel(screen_top, text="LIVE ACTIVITY", font=("Consolas", 10, "bold"),
             text_color="#64748B").pack(side="left", padx=(10, 0))
self._live_clock_var = tk.StringVar(value="")
ctk.CTkLabel(screen_top, textvariable=self._live_clock_var,
             font=("Consolas", 9), text_color="#475569").pack(side="right", padx=10)
self._tick_live_clock()

# Activity feed
self._activity_scroll = ctk.CTkScrollableFrame(display_frame, fg_color="#020617",
                                              corner_radius=0)
self._activity_scroll.pack(fill="both", expand=True, padx=3)
self._activity_items = []

# Stats strip
stats_strip = ctk.CTkFrame(display_frame, fg_color="#0F172A", height=28, corner_radius=0)
stats_strip.pack(fill="x", padx=3, pady=(0, 3))
stats_strip.pack_propagate(False)
self._stat_trades_var = tk.StringVar(value="Trades: 0")
self._stat_queue_var = tk.StringVar(value="Queue: 0")
self._stat_push_var = tk.StringVar(value="Push: idle")
for var in [self._stat_trades_var, self._stat_queue_var, self._stat_push_var]:
    ctk.CTkLabel(stats_strip, textvariable=var, font=("Consolas", 9),
                 text_color="#475569").pack(side="left", padx=(12, 16), pady=4)

# Hidden log widget (still needed by self.log() method)
self.log_text = tk.Text(self._live_view, height=0)

# ■■■ VIEW 2: Controls (hidden, full-screen takeover) ■■■
self._controls_visible = False
self._controls_view = ctk.CTkFrame(body, fg_color="transparent")
# Don't pack yet – toggled on click

self.notebook = ctk.CTkTabview(self._controls_view, fg_color=self.C_BG,
                              segmented_button_fg_color=self.C_BG_SEC,
                              segmented_button_selected_color=self.C_ACCENT,
                              segmented_button_unselected_color=self.C_BG_THIRD,
                              text_color=self.C_TEXT, corner_radius=8)
self.notebook.pack(fill="both", expand=True)
tab_settings = self.notebook.add(" Settings ")

self._build_combined_settings_tab(tab_settings)

# ■■■ Bottom status bar ■■■
status_bar = ctk.CTkFrame(outer, fg_color=self.C_BG_SEC, height=24, corner_radius=0)
status_bar.pack(fill="x", side="bottom")
status_bar.pack_propagate(False)
self.status_var = tk.StringVar(value="Ready – enter your email to get started")
self.status_label = ctk.CTkLabel(status_bar, textvariable=self.status_var,
                                font=("Segoe UI", 9), text_color=self.C_TEXT_DIM)
self.status_label.pack(side="left", padx=12, pady=2)

# State for smart auto-push
self.last_deal_ticket = 0

```

```
self.last_deal_count = 0
self.auto_push_thread = None
```

Method: TradeOpssAIApp._tick_live_clock

```
def _tick_live_clock(self)
```

Update the live display clock every second.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def _tick_live_clock(self):
    """Update the live display clock every second."""
    try:
        self._live_clock_var.set(datetime.now().strftime("%H:%M:%S"))
        self.root.after(1000, self._tick_live_clock)
    except Exception:
        pass
```

Method: TradeOpssAIApp._add_activity

```
def _add_activity(self, text, kind='info')
```

Add an entry to the Live Activity display panel. kind: 'info', 'trade', 'push', 'error', 'queue', 'success'

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `COLORS = {'info': '#64748B', 'trade': '#F59E0B', 'push': '#3B82F6'....`
2. Assigns `ICONS = {'info': '■', 'trade': '■', 'push': '■', 'error': '✕', 'q....`
3. **if not CTK_AVAILABLE**: branches conditionally.
4. Assigns `color = COLORS.get(kind, COLORS['info'])`.
5. Assigns `icon = ICONS.get(kind, '•')`.
6. Assigns `ts = datetime.now().strftime('%H:%M:%S')`.
7. Assigns `row = ctk.CTkFrame(self._activity_scroll, fg_color='transparent....`
8. Calls `row.pack(...)` for its side effect.

9. Calls `row.pack_propagate(...)` for its side effect.
10. Calls `ctk.CTkLabel(row, text=f'{icon}', font=('Segoe UI', 10), text_color=color, width=16).pack(...)` for its side effect.
11. Calls `ctk.CTkLabel(row, text=ts, font=('Consolas', 9), text_color='#334155').pack(...)` for its side effect.
12. Calls `ctk.CTkLabel(row, text=text, font=('Segoe UI', 9), text_color=color, anchor='w').pack(...)` for its side effect.
13. Calls `self._activity_items.append(...)` for its side effect.
14. **while** `len(self._activity_items) > 80`: loops until the condition is false.
15. ... and 1 more statement(s) in the body.

Code (copy-paste safe)

```
def _add_activity(self, text, kind="info"):
    """Add an entry to the Live Activity display panel.
    kind: 'info', 'trade', 'push', 'error', 'queue', 'success'
    """
    COLORS = {
        "info": "#64748B",
        "trade": "#F59E0B",
        "push": "#3B82F6",
        "error": "#EF4444",
        "queue": "#A78BFA",
        "success": "#22C55E",
    }
    ICONS = {
        "info": "■",
        "trade": "■",
        "push": "■",
        "error": "✖",
        "queue": "■",
        "success": "✓",
    }
    if not CTk_AVAILABLE:
        return

    color = COLORS.get(kind, COLORS["info"])
    icon = ICONS.get(kind, "•")
    ts = datetime.now().strftime("%H:%M:%S")

    row = ctk.CTkFrame(self._activity_scroll, fg_color="transparent", height=22)
    row.pack(fill="x", padx=4, pady=1)
    row.pack_propagate(False)
    ctk.CTkLabel(row, text=f"{icon}", font=("Segoe UI", 10),
                  text_color=color, width=16).pack(side="left", padx=(4, 4))
    ctk.CTkLabel(row, text=ts, font=("Consolas", 9),
                  text_color="#334155").pack(side="left", padx=(0, 6))
    ctk.CTkLabel(row, text=text, font=("Segoe UI", 9),
                  text_color=color, anchor="w").pack(side="left", fill="x", expand=True)

    self._activity_items.append({"frame": row, "kind": kind})

    # Keep max 80 items
    while len(self._activity_items) > 80:
        old = self._activity_items.pop(0)
        try:
            old["frame"].destroy()
```

```

        except Exception:
            pass

    # Auto-scroll to bottom
    try:
        self._activity_scroll._parent_canvas.yview_moveto(1.0)
    except Exception:
        pass

```

Method: TradeOpssAIApp._toggle_controls_panel

```
def _toggle_controls_panel(self)
```

Swap between Live Display and Controls views (full-screen takeover).

Step by step (top-level body)

1. if self._controls_visible: branches conditionally (with an **else/elif** arm).

Code (copy-paste safe)

```

def _toggle_controls_panel(self):
    """Swap between Live Display and Controls views (full-screen takeover)."""
    if self._controls_visible:
        # Hide controls, show live display
        self._controls_view.pack_forget()
        self._live_view.pack(fill="both", expand=True)
        self._controls_visible = False
        self._panel_btn.configure(text="■ Controls")
    else:
        # Hide live display, show controls
        self._live_view.pack_forget()
        self._controls_view.pack(fill="both", expand=True)
        self._controls_visible = True
        self._panel_btn.configure(text="■ Back to Live")

```

Method: TradeOpssAIApp._build_combined_settings_tab

```
def _build_combined_settings_tab(self, parent)
```

Build the combined Settings tab — Dashboard + Trading Engine in one page.

Step by step (top-level body)

1. if CTK_AVAILABLE: branches conditionally.
2. Calls self._build_dashboard_tab(...) for its side effect.
3. Calls self._build_trading_engine_ui(...) for its side effect.

Code (copy-paste safe)

```

def _build_combined_settings_tab(self, parent):
    """Build the combined Settings tab – Dashboard + Trading Engine in one page."""
    if CTK_AVAILABLE:
        scroll = ctk.CTkScrollableFrame(parent, fg_color="transparent")
        scroll.pack(fill="both", expand=True)
        parent = scroll
    self._build_dashboard_tab(parent)
    self._build_trading_engine_ui(parent)

```

Method: TradeOpssAIApp._build_dashboard_tab

```
def _build_dashboard_tab(self, parent)
```

Build the Dashboard section — compact layout.

Why these Python concepts were used

- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns settings = parent.
2. Assigns conn_card = self._section_card(settings, 'CONNECTION TARGET', '■').
3. Calls conn_card.pack(...) for its side effect.
4. Assigns conn_inner = ctk.CTkFrame(conn_card, fg_color='transparent') if CTK_AV....
5. Calls conn_inner.pack(...) for its side effect.
6. **if** CTK_AVAILABLE: branches conditionally.
7. Assigns self.target_var = tk.StringVar().
8. Assigns self.url_keys = ['TradeOps (Production)', 'Localhost (Development)'].
9. Assigns self.url_values = {'TradeOps (Production)': 'https://www.tradeopss.com', '....
10. **if** CTK_AVAILABLE: branches conditionally (with an **else/elif** arm).
11. Assigns self.url_entry = ttk.Entry(settings).
12. Calls self.url_entry.insert(...) for its side effect.
13. Defines a nested function on_target_change(...).
14. **if** CTK_AVAILABLE: branches conditionally (with an **else/elif** arm).
15. ... and 26 more statement(s) in the body.

Code (copy-paste safe)

```
def _build_dashboard_tab(self, parent):
    """Build the Dashboard section — compact layout."""

    # ■■ Connection Target ■■
    settings = parent
    conn_card = self._section_card(settings, "CONNECTION TARGET", "■")
    conn_card.pack(fill="x", padx=4, pady=(4, 2))

    conn_inner = ctk.CTkFrame(conn_card, fg_color="transparent") if CTK_AVAILABLE else \
        tk.Frame(conn_card, bg="#161B22")
    conn_inner.pack(fill="x", padx=10, pady=(2, 6))

    if CTK_AVAILABLE:
        ctk.CTkLabel(conn_inner, text="Target:", font=("Segoe UI", 11),
```

```

        text_color=self.C_TEXT_DIM).pack(side="left", padx=(0, 8))

self.target_var = tk.StringVar()
self.url_keys = ["TradeOpps (Production)", "Localhost (Development)"]
self.url_values = {
    "TradeOpps (Production)": "https://www.tradeopss.com",
    "Localhost (Development)": "http://127.0.0.1:5001"
}

if CTK_AVAILABLE:
    self.url_selector = ctk.CTkComboBox(conn_inner, variable=self.target_var,
                                       values=self.url_keys, state="readonly",
                                       width=280, height=32,
                                       fg_color=self.C_BG_THIRD,
                                       border_color=self.C_BORDER,
                                       button_color=self.C_ACCENT,
                                       dropdown_fg_color=self.C_BG_SEC,
                                       dropdown_hover_color=self.C_BG_THIRD,
                                       text_color=self.C_TEXT,
                                       font=("Segoe UI", 11))
    self.url_selector.pack(side="left", padx=(0, 8))
    self.url_selector.set(self.url_keys[0])
else:
    self.url_selector = ttk.Combobox(conn_inner, textvariable=self.target_var,
                                     state="readonly", width=32)
    self.url_selector['values'] = self.url_keys
    self.url_selector.pack(side="left", fill="x", expand=True)
    self.url_selector.current(0)

# Hidden entry for backward-compat URL storage
self.url_entry = ttk.Entry(settings)
self.url_entry.insert(0, self.url_values["TradeOpps (Production)"])

def on_target_change(event=None):
    selection = self.target_var.get()
    if "Localhost" in selection:
        password = simpledialog.askstring("Developer Access",
                                          "Enter password for local development:", show='*')
        if password == "tradeopss@123":
            self.url_entry.delete(0, tk.END)
            self.url_entry.insert(0, self.url_values[selection])
            self.log("Switched to Localhost")
            self.status_var.set("Target: Localhost (Dev)")
        else:
            messagebox.showerror("Access Denied", "Incorrect password.")
            if CTK_AVAILABLE:
                self.url_selector.set(self.url_keys[0])
            else:
                self.url_selector.current(0)
            self.url_entry.delete(0, tk.END)
            self.url_entry.insert(0, self.url_values["TradeOpps (Production)"])
    else:
        self.url_entry.delete(0, tk.END)
        self.url_entry.insert(0, self.url_values[selection])
        self.log("Switched to Production")
        self.status_var.set("Target: Production")

if CTK_AVAILABLE:
    self.url_selector.configure(command=lambda _: on_target_change())
else:

```

```

        self.url_selector.bind("<<ComboboxSelected>>", on_target_change)

# ■■■ Client Identification (hidden entry for compat) ■■■
self.client_email_entry = ctk.CTkEntry(settings, width=0, height=0) if CTK_AVAILABLE else tk.Entry(
    settings)
# Don't pack – hidden, used only as data holder

self.hierarchy_var = tk.StringVar(value="")
self.hierarchy_label = ctk.CTkLabel(settings, textvariable=self.hierarchy_var,
                                   font=("Segoe UI", 10, "italic"),
                                   text_color=self.C_TEXT_DIM) if CTK_AVAILABLE else \
    ttk.Label(settings, textvariable=self.hierarchy_var)
# Don't pack – hidden

# ■■■ Import Data ■■■
imp_card = self._section_card(settings, "IMPORT DATA", "■")
imp_card.pack(fill="x", padx=4, pady=2)

imp_inner = ctk.CTkFrame(imp_card, fg_color="transparent") if CTK_AVAILABLE else \
    tk.Frame(imp_card, bg="#161B22")
imp_inner.pack(fill="x", padx=12, pady=(4, 4))

self.import_source = tk.StringVar(value="sheet")
if CTK_AVAILABLE:
    ctk.CTkRadioButton(imp_inner, text="Google Sheets", variable=self.import_source,
                       value="sheet", command=self._toggle_import_source,
                       font=("Segoe UI", 11), text_color=self.C_TEXT,
                       fg_color=self.C_ACCENT, border_color=self.C_BORDER).pack(side="left", padx=
14))
    ctk.CTkRadioButton(imp_inner, text="CSV File", variable=self.import_source,
                       value="csv", command=self._toggle_import_source,
                       font=("Segoe UI", 11), text_color=self.C_TEXT,
                       fg_color=self.C_ACCENT, border_color=self.C_BORDER).pack(side="left")
else:
    ttk.Radiobutton(imp_inner, text="Google Sheets", variable=self.import_source,
                   value="sheet", command=self._toggle_import_source).pack(side="left", padx=(0,
    ttk.Radiobutton(imp_inner, text="CSV File", variable=self.import_source,
                   value="csv", command=self._toggle_import_source).pack(side="left")

# Sheet URL row
self.sheet_input_frame = ctk.CTkFrame(imp_card, fg_color="transparent") if CTK_AVAILABLE else \
    tk.Frame(imp_card, bg="#161B22")
self.sheet_input_frame.pack(fill="x", padx=12, pady=4)
if CTK_AVAILABLE:
    ctk.CTkLabel(self.sheet_input_frame, text="URL:", font=("Segoe UI", 11),
                 text_color=self.C_TEXT_DIM).pack(side="left", padx=(0, 6))
self.sheet_url_entry = self._ctk_entry(self.sheet_input_frame, width=380,
                                       placeholder="Google Sheet URL...")
self.sheet_url_entry.pack(side="left", fill="x", expand=True)

# CSV row (hidden by default)
self.csv_input_frame = ctk.CTkFrame(imp_card, fg_color="transparent") if CTK_AVAILABLE else \
    tk.Frame(imp_card, bg="#161B22")
self.csv_path_var = tk.StringVar()
if CTK_AVAILABLE:
    ctk.CTkLabel(self.csv_input_frame, text="File:", font=("Segoe UI", 11),
                 text_color=self.C_TEXT_DIM).pack(side="left", padx=(0, 6))
    self.csv_path_entry = ctk.CTkEntry(self.csv_input_frame, textvariable=self.csv_path_var,
                                       width=280, height=32, state="disabled",
                                       fg_color=self.C_BG_THIRD, border_color=self.C_BORDER,

```

```

text_color=self.C_TEXT)
else:
    self.csv_path_entry = ttk.Entry(self.csv_input_frame, textvariable=self.csv_path_var,
                                    width=36, state='readonly')
    self.csv_path_entry.pack(side="left", padx=(0, 6))
    self._ctk_button(self.csv_input_frame, text="Browse...", command=self._browse_csv,
                    fg=self.C_BG_THIRD, hover=self.C_BORDER, width=90).pack(side="left")

    imp_btn_row = ctk.CTkFrame(imp_card, fg_color="transparent") if CTK_AVAILABLE else \
        tk.Frame(imp_card, bg="#161B22")
    imp_btn_row.pack(fill="x", padx=12, pady=(4, 8))
    self.import_btn = self._ctk_button(imp_btn_row, text="Import Sheet Data", command=self._do_import,
                                      fg="#6366F1", hover="#4F46E5", width=180)
    self.import_btn.pack(side="left")
    self.import_hint_text = "Sheet must be publicly shared"
    if CTK_AVAILABLE:
        self.import_hint = ctk.CTkLabel(imp_btn_row, text=self.import_hint_text,
                                       font=("Segoe UI", 9, "italic"),
                                       text_color=self.C_TEXT_DIM)
    else:
        self.import_hint = ttk.Label(imp_btn_row, text=self.import_hint_text)
    self.import_hint.pack(side="left", padx=(12, 0))

```

Method: TradeOpssAIApp.log

```
def log(self, message, level='INFO')
```

Add a message to the log and the live activity display.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns timestamp = datetime.now().strftime('%H:%M:%S').
2. Calls self.log_text.insert(...) for its side effect.
3. Calls self.log_text.see(...) for its side effect.
4. Assigns kind = 'info'.
5. Assigns msg_lower = message.lower().
6. if level == 'ERROR' or '■' in message or 'failed' in msg_lower: branches conditionally (with an **else/elif** arm).
7. **try** block with 1 **except** clause.
8. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def log(self, message, level="INFO"):
    """Add a message to the log and the live activity display."""
    timestamp = datetime.now().strftime("%H:%M:%S")

```

```

self.log_text.insert(tk.END, f"[{timestamp}] {message}\n")
self.log_text.see(tk.END)
# Feed into live display
kind = "info"
msg_lower = message.lower()
if level == "ERROR" or "■" in message or "failed" in msg_lower:
    kind = "error"
elif "■" in message or "success" in msg_lower:
    kind = "success"
elif "trade" in msg_lower or "buy" in msg_lower or "sell" in msg_lower or "■" in message:
    kind = "trade"
elif "push" in msg_lower or "■" in message or "syncd" in msg_lower:
    kind = "push"
elif "queue" in msg_lower or "scheduled" in msg_lower or "■" in message:
    kind = "queue"
try:
    self._add_activity(message, kind)
except Exception:
    pass
try:
    self.root.update_idletasks()
except Exception:
    pass

```

Method: TradeOpssAIApp.lookup_client

```
def lookup_client(self)
```

Lookup client hierarchy from email - NO API KEY REQUIRED.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.

Step by step (top-level body)

1. Assigns email = self.client_email_entry.get().strip().
2. Assigns dashboard_url = self.url_entry.get().strip().rstrip('/').
3. **if not email:** branches conditionally.
4. Calls self.log(...) for its side effect.
5. Calls self.hierarchy_var.set(...) for its side effect.
6. Calls self.root.update_idletasks(...) for its side effect.
7. Defines a nested function _do_lookup(...).
8. Calls threading.Thread(target=_do_lookup, daemon=True).start(...) for its side effect.

Code (copy-paste safe)

```

def lookup_client(self):
    """Lookup client hierarchy from email - NO API KEY REQUIRED."""
    email = self.client_email_entry.get().strip()
    dashboard_url = self.url_entry.get().strip().rstrip('/')

    if not email:
        messagebox.showerror("Error", "Please enter the client email")
        return

    self.log(f"Looking up client: {email}")
    self.hierarchy_var.set("Looking up...")
    self.root.update_idletasks()

    def _do_lookup():
        try:
            # Use public endpoint - no API key needed
            response = requests.post(
                f"{dashboard_url}/api/client/auth",
                json={"email": email},
                headers={"Content-Type": "application/json"},
                timeout=30
            )

            if response.status_code == 200:
                data = response.json()
                if data.get("status") == "success":
                    def _on_success(data=data):
                        self.client_info = data.get("identity", {})
                        client = self.client_info.get("client", "Unknown")
                        trader = self.client_info.get("trader", "Unknown")
                        admin = self.client_info.get("admin", "Unknown")
                        category = self.client_info.get("category", "Unknown")

                        self.hierarchy_var.set(
                            f"■ {client} → Trader: {trader} → Admin: {admin} | Category: {category}"
                        )
                        if CTK_AVAILABLE:
                            self.hierarchy_label.configure(text_color='#16a34a')
                        else:
                            self.hierarchy_label.configure(foreground='#16a34a')
                        self.log(f"■ Client found: {client} → {trader} → {admin}")

                    # Auto-fetch hedge accounts to populate MT5 credentials
                    def _fetch_hedge_creds(cl=client, url=dashboard_url):
                        try:
                            r2 = requests.get(
                                f"{url}/api/data?client_id={cl}",
                                timeout=15
                            )
                        except:
                            pass
                        if r2.status_code == 200:
                            d2 = r2.json()
                            hedge_accounts = d2.get('hedge_accounts') or []
                            if hedge_accounts:
                                ha = hedge_accounts[0]
                                ha_login = str(ha.get('login', '')).strip()
                                ha_pass = str(ha.get('password', '')).strip()
                                ha_server = str(ha.get('server', '')).strip()
                                def _fill_creds(l=ha_login, p=ha_pass, s=ha_server):
                                    if l:
                                        self.mt5_login.delete(0, 'end')
                                        self.mt5_login.insert(0, l)

```



```

        if p:
            self.mt5_password.delete(0, 'end')
            self.mt5_password.insert(0, p)
        if s:
            self.mt5_server.delete(0, 'end')
            self.mt5_server.insert(0, s)
        if l or s:
            self.log(
                f"■ MT5 credentials auto-filled from hedge account"
            )
        self.root.after(0, _fill_creds)
    except Exception:
        pass
    threading.Thread(target=_fetch_hedge_creds, daemon=True).start()
    self.root.after(0, _on_success)
else:
    error_msg = data.get("message", "Client not found")
    def _on_not_found(msg=error_msg):
        self.hierarchy_var.set(f"■ {msg}")
        if CTK_AVAILABLE:
            self.hierarchy_label.configure(text_color='#dc2626')
        else:
            self.hierarchy_label.configure(foreground='#dc2626')
        self.client_info = None
        self.log(f"■ Lookup failed: {msg}", "ERROR")
    self.root.after(0, _on_not_found)
else:
    error_msg = f"API Error: {response.status_code}"
    try:
        error_data = response.json()
        error_msg = error_data.get("message", error_msg)
    except:
        pass
    def _on_error(msg=error_msg):
        self.hierarchy_var.set(f"■ {msg}")
        if CTK_AVAILABLE:
            self.hierarchy_label.configure(text_color='#dc2626')
        else:
            self.hierarchy_label.configure(foreground='#dc2626')
        self.client_info = None
        self.log(f"■ Lookup failed: {msg}", "ERROR")
    self.root.after(0, _on_error)

except requests.exceptions.Timeout:
    def _on_timeout():
        self.hierarchy_var.set("■ Connection timeout")
        if CTK_AVAILABLE:
            self.hierarchy_label.configure(text_color='#dc2626')
        else:
            self.hierarchy_label.configure(foreground='#dc2626')
        self.log("■ Connection timeout", "ERROR")
    self.root.after(0, _on_timeout)
except requests.exceptions.ConnectionError:
    def _on_conn_err():
        self.hierarchy_var.set("■ Cannot connect to server")
        if CTK_AVAILABLE:
            self.hierarchy_label.configure(text_color='#dc2626')
        else:
            self.hierarchy_label.configure(foreground='#dc2626')
        self.log("■ Cannot connect to server", "ERROR")
    self.root.after(0, _on_conn_err)

```

```

except Exception as e:
    def _on_exc(err=str(e)):
        self.hierarchy_var.set(f"■ Error: {err}")
        if CTK_AVAILABLE:
            self.hierarchy_label.configure(text_color='#dc2626')
        else:
            self.hierarchy_label.configure(foreground='#dc2626')
        self.log(f"■ Error: {err}", "ERROR")
    self.root.after(0, _on_exc)

threading.Thread(target=_do_lookup, daemon=True).start()

```

Method: TradeOpssAIApp.toggle_mt5_connection

```
def toggle_mt5_connection(self)
```

Connect or disconnect from MT5.

Why these Python concepts were used

- **ternary expression** — Uses `x` if `cond` else `y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **if** `self.pusher.connected`: branches conditionally (with an **else/elif** arm).

Code (copy-paste safe)

```

def toggle_mt5_connection(self):
    """Connect or disconnect from MT5."""
    if self.pusher.connected:
        success, msg = self.pusher.disconnect_mt5()
        self.mt5_btn.configure(text="Connect to MT5")
        self.log(msg)
    else:
        login = self.mt5_login.get().strip()
        password = self.mt5_password.get()
        server = self.mt5_server.get().strip()

        success, msg = self.pusher.connect_mt5(login, password, server)
        if success:
            self.mt5_btn.configure(text="Disconnect MT5")
            self.log(msg, "INFO" if success else "ERROR")

```

Method: TradeOpssAIApp._push_billing_data

```
def _push_billing_data(self)
```

Push only firm billing (actual fees & payouts) to the dashboard.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with `append`, and clearer about intent: this is a transform, not a side-effecting loop.

- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.
- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.

Step by step (top-level body)

1. **if** RELEASE_DISABLE_PUSH_BILLING: branches conditionally.
2. **if** not self._firm_billing_summary: branches conditionally.
3. Assigns dashboard_url = self.url_entry.get().strip().rstrip('/').
4. Assigns email = self.client_email_entry.get().strip().
5. **if** not self.client_info: branches conditionally.
6. Defines a nested function _do_push(...).
7. Calls threading.Thread(target=_do_push, daemon=True).start(...) for its side effect.

Code (copy-paste safe)

```
def _push_billing_data(self):
    """Push only firm billing (actual fees & payouts) to the dashboard."""
    if RELEASE_DISABLE_PUSH_BILLING:
        self.log("■ Push Billing is disabled in this release", "WARN")
        return

    if not self._firm_billing_summary:
        self.log("■ No billing data collected yet – connect to prop firm dashboards first", "WARN")
        return

    dashboard_url = self.url_entry.get().strip().rstrip('/')
    email = self.client_email_entry.get().strip()

    if not self.client_info:
        messagebox.showerror("Error", "Please lookup the client first")
        return

    def _do_push():
        try:
            if self.push_billing_btn:
                self.root.after(0, lambda: self.push_billing_btn.configure(state="disabled"))
                self.log("■ Pushing billing data to dashboard...")

                # Match billing records to individual evaluations so
                # per-account Fee, Date Purchased, and Date Started get pushed
                import re as _re
                all_evals = [rd.get("eval") for rd in self._active_trade_rows if rd.get("eval")]
                filled_count = 0

                # Build a combined lookup from all firms' billing records
                billing_by_acct = {}
```

```

for firm_name, summary in self._firm_billing_summary.items():
    for rec in summary.get("records", []):
        acct_no = (rec.get("account_no") or "").strip()
        amount = rec.get("amount", 0)
        bill_date = rec.get("date", "")
        if acct_no and amount > 0:
            billing_by_acct[acct_no] = {
                "amount": amount,
                "date": bill_date,
                "firm": firm_name,
            }

if all_evals and billing_by_acct:
    for ev in all_evals:
        acct_challenge = (ev.get("Account #") or "").strip()
        acct_funded = (ev.get("Account #.1") or "").strip()
        existing_fee = (ev.get("Fee") or "").strip()
        fee_filled = False
        if existing_fee:
            try:
                fee_filled = float(existing_fee.replace("$", "").replace(", ", "")) > 0
            except ValueError:
                fee_filled = existing_fee not in ("", "$0", "$0.00", "0")

        matched = None
        for acct_key in [acct_challenge, acct_funded]:
            if not acct_key:
                continue
            if acct_key in billing_by_acct:
                matched = billing_by_acct[acct_key]
                break
            for bill_acct, bill_info in billing_by_acct.items():
                if bill_acct in acct_key or acct_key in bill_acct:
                    matched = bill_info
                    break
            if matched:
                break

        if matched:
            billing_fee = f"${matched['amount']:.2f}"
            ev["Fee"] = billing_fee
            if not (ev.get("Date Purchased") or "").strip() and matched["date"]:
                ev["Date Purchased"] = matched["date"]
            if not (ev.get("Date Started") or "").strip() and matched["date"]:
                ev["Date Started"] = matched["date"]
            filled_count += 1

self.root.after(0, lambda c=filled_count:
    self.log(f"■ Matched billing to {c} evaluation(s)"))

payload = {
    "email": email,
    "firm_billing": self._firm_billing_summary,
}
if all_evals and filled_count > 0:
    payload["evaluations"] = all_evals

response = requests.post(
    f"{dashboard_url}/api/client/push",
    json=payload,

```

```

        headers={"Content-Type": "application/json"},
        timeout=30
    )

    if response.status_code == 200:
        data = response.json()
        if data.get("status") == "success":
            firms = list(self._firm_billing_summary.keys())
            total_fees = sum(f["total_fees"] for f in self._firm_billing_summary.values())
            total_payouts = sum(f["total_payouts"] for f in self._firm_billing_summary.values())
            self.root.after(0, lambda: self.log(
                f"■ Billing pushed – {len(firms)} firm(s): "
                f"Fees ${total_fees:,.2f} | Payouts ${total_payouts:,.2f}"))
        else:
            self.root.after(0, lambda: self.log(
                f"■ Billing push failed: {data.get('message', 'Unknown error')}", "ERROR"))
    else:
        self.root.after(0, lambda: self.log(
            f"■ Billing push HTTP {response.status_code}", "ERROR"))
except Exception as e:
    self.root.after(0, lambda err=str(e): self.log(f"■ Billing push error: {err}", "ERROR"))
finally:
    if self.push_billing_btn:
        self.root.after(0, lambda: self.push_billing_btn.configure(state="normal"))

threading.Thread(target=_do_push, daemon=True).start()

```

Method: TradeOpssAIApp.push_data

```
def push_data(self)
```

Push data to dashboard - NO API KEY REQUIRED.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.

Step by step (top-level body)

1. Assigns `dashboard_url = self.url_entry.get().strip().rstrip('/')`.
2. Assigns `email = self.client_email_entry.get().strip()`.
3. **if** `not self.client_info`: branches conditionally.
4. Assigns `client_name = self.client_info.get('client', '')`.

5. **if** not `client_name`: branches conditionally.
6. **with** `self._push_lock`: enters a context manager.
7. **if** `hasattr(self, 'push_btn_live')` and `self.push_btn_live`: branches conditionally.
8. Calls `self.log(...)` for its side effect.
9. Calls `self.status_var.set(...)` for its side effect.
10. Defines a nested function `_do_push(...)`.
11. Calls `threading.Thread(target=_do_push, daemon=True).start(...)` for its side effect.

Code (copy-paste safe)

```
def push_data(self):
    """Push data to dashboard - NO API KEY REQUIRED."""
    dashboard_url = self.url_entry.get().strip().rstrip('/')
    email = self.client_email_entry.get().strip()

    # Use looked-up hierarchy info
    if not self.client_info:
        messagebox.showerror("Error",
                             "Please lookup the client first by entering email and clicking 'Lookup'")
        return

    client_name = self.client_info.get('client', '')

    if not client_name:
        messagebox.showerror("Error", "Client lookup failed - no client name found")
        return

    # Prevent overlapping push workers. If a push is already running,
    # queue exactly one follow-up run so the latest state still gets sent.
    with self._push_lock:
        if self._push_in_progress:
            if not self._push_pending:
                self._push_pending = True
                self.log("■ Push already running – queued one follow-up push")
            return
        self._push_in_progress = True

    # Guard checks passed – disable the button and run everything in a background thread
    # so the UI stays responsive during MT5 calls and the HTTP round-trip.
    if hasattr(self, 'push_btn_live') and self.push_btn_live:
        try:
            self.push_btn_live.configure(state="disabled")
        except Exception:
            pass
    self.log(f"■ Pushing {client_name}...")
    self.status_var.set("Pushing data...")

    def _do_push():
        should_run_follow_up = False
        try:
            self._push_data_worker(dashboard_url, email, client_name)
        finally:
            with self._push_lock:
                self._push_in_progress = False
                should_run_follow_up = self._push_pending
                self._push_pending = False
```

```

        if should_run_follow_up:
            try:
                self.root.after(0, lambda: self.log("■ Running queued push"))
                self.root.after(0, self.push_data)
            except Exception:
                pass
        elif hasattr(self, 'push_btn_live') and self.push_btn_live:
            try:
                self.root.after(0, lambda: self.push_btn_live.configure(state="normal"))
            except Exception:
                pass

    threading.Thread(target=_do_push, daemon=True).start()

```

Method: TradeOpssAIApp._push_data_worker

```
def _push_data_worker(self, dashboard_url, email, client_name)
```

Heavy push work — runs on a background thread.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **dict comprehension** — Builds a dict in one expression (`{k: v for k, v in items}`) — same intent as a list comprehension but for mappings.
- **set comprehension** — Builds a set in one expression — usually to deduplicate the result of a transform.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.
- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Defines a nested function `_log(...)`.
2. Defines a nested function `_status(...)`.

3. Assigns `account = self.pusher.get_account_info(include_balance_history=False)`.
4. `if not account`: branches conditionally.
5. `if account`: branches conditionally.
6. Assigns `positions = self.pusher.get_positions()`.
7. `if positions is None`: branches conditionally.
8. Assigns `_today_cutoff = time.time() - 86400`.
9. Assigns `raw_deals = self.pusher.get_deals(days=1)` or `[]`.
10. Assigns `_fa_history_days = 365`.
11. Assigns `_login_key = str(account.get('login', 'unknown'))`.
12. `if not hasattr(self, '_fa_history_cache')`: branches conditionally.
13. Assigns `_cached = self._fa_history_cache.get(_login_key)`.
14. Assigns `_now = time.time()`.
15. ... and 40 more statement(s) in the body.

Code (copy-paste safe)

```
def _push_data_worker(self, dashboard_url, email, client_name):
    """Heavy push work – runs on a background thread."""
    def _log(msg, level="INFO"):
        self.root.after(0, lambda m=msg, lv=level: self.log(m, lv))
    def _status(msg):
        self.root.after(0, lambda m=msg: self.status_var.set(m))

    account = self.pusher.get_account_info(include_balance_history=False)
    if not account:
        _log("■■■ MT5 account info returned empty – pushing with no account data", "ERROR")
        account = {}

    # Log detailed MT5 account information
    if account:
        login = account.get('login', 'N/A')
        company = account.get('company', 'Unknown')
        server = account.get('server', 'Unknown')
        balance = account.get('balance', 0)
        equity = account.get('equity', 0)
        deposits = account.get('total_deposits', 0)
        withdrawals = account.get('total_withdrawals', 0)
        _log(f"■■■ MT5 ACCOUNT INFO:")
        _log(f"  Login: {login} | Company: {company} | Server: {server}")
        _log(f"  Balance: ${balance:,.2f} | Equity: ${equity:,.2f}")
        _log(f"  Deposits: ${deposits:,.2f} | Withdrawals: ${withdrawals:,.2f}")

    positions = self.pusher.get_positions()
    if positions is None:
        _log("■■■ MT5 positions returned None – sending empty list")
        positions = []

    # Always fetch the last 24 h fresh (fast – small result set).
    _today_cutoff = time.time() - 86400
    raw_deals = self.pusher.get_deals(days=1) or []

    # For farming aggregation we need long-range history for accurate Hedge Day slots.
```



```

# Cache it per MT5 login with a 5-minute TTL so repeated auto-push calls are fast.
_fa_history_days = 365
_login_key = str(account.get('login', 'unknown'))
if not hasattr(self, '_fa_history_cache'):
    self._fa_history_cache = {}

_cached = self._fa_history_cache.get(_login_key)
_now = time.time()

if _cached and (_now - _cached[0]) < self._FA_HISTORY_CACHE_TTL:
    _log(f"■ Using cached FA history ({int(_now - _cached[0]))s old)")
    _cached_deals = _cached[1]
else:
    _log(f"■ Fetching {_fa_history_days}-day FA history (cache miss or expired)")
    _full = self.pusher.get_deals(days=_fa_history_days) or []
    # Store only the deals older than today's cutoff to keep cache lean.
    # Strip internal transfers up-front so they can NEVER resurface from
    # the cache – guarantees the latest live MT5 state always wins and
    # no stale balance op leaks into hedge aggregates on subsequent pushes.
    _cached_deals = []
    for _d in _full:
        if _d.get('time_raw', 0) >= _today_cutoff:
            continue
        _dt = str(_d.get('type', '')).upper()
        if _dt in ('BALANCE', 'CREDIT', '2', '3', 'CHARGE', 'CORRECTION', 'BONUS'):
            _cl = str(_d.get('comment', '') or '').strip().lower()
            if ('internal transfer' in _cl) or (not _cl):
                continue
            _cached_deals.append(_d)
    self._fa_history_cache[_login_key] = (_now, _cached_deals)

# Merge: fresh last-24h + cached history gives the full aggregation input.
_raw_deal_ids = {d.get('ticket') for d in raw_deals}
all_history_deals = list(raw_deals) + [d for d in _cached_deals if d.get(
    'ticket') not in _raw_deal_ids]

# Derive last-trading-day deals – already fetched above as raw_deals.
# Fall back to the most-recent day if there were no deals in the last 24 h.
if not raw_deals and all_history_deals:
    _max_ts = max(d.get('time_raw', 0) for d in all_history_deals)
    _day_start = _max_ts - 86400
    raw_deals = [d for d in all_history_deals if d.get('time_raw', 0) >= _day_start]

# Mark history-only FA deals so the filter can skip re-parsing them.
# Non-FA deals (CH, FD, DD) should only use last-trading-day data.
# FA deals need the full 90-day history to correctly compute hedge day slots.
_last_trading_day_ids = {d.get('ticket') for d in raw_deals}
aggregation_raw_deals = []
for _d in all_history_deals:
    c_up = str(_d.get('comment', '')).upper()
    is_fa = '_FA' in c_up
    is_history_only = _d.get('ticket') not in _raw_deal_ids
    d_type = str(_d.get('type', '')).upper()
    is_balance_op = d_type in ['BALANCE', 'CREDIT', '2', '3', 'CHARGE', 'CORRECTION', 'BONUS']
    _comment_lower = str(_d.get('comment', '') or '').strip().lower()
    is_internal_transfer = (
        'internal transfer' in _comment_lower
        or (is_balance_op and not _comment_lower)
    )

    if is_internal_transfer:

```

```

        # Internal transfers / unattributed balance ops never contribute to
        # hedge results; drop them so they cannot inflate eval values.
        continue

    if is_balance_op:
        # Keep other balance ops (deposits with comments, charges, bonuses, etc.)
        aggregation_raw_deals.append(_d)
    elif is_fa:
        # FA: only push last-trading-day deals (what to push).
        # Full history is used separately for day COUNT only (fa_day_keys_by_account below).
        if _d.get('ticket') in _last_trading_day_ids:
            aggregation_raw_deals.append(_d)
    else:
        # CH / FD / DD: only last trading day – avoids stale/replaced deals from old resets
        if _d.get('ticket') in _last_trading_day_ids:
            aggregation_raw_deals.append(_d)

_fa_count = sum(1 for d in aggregation_raw_deals if '_FA' in str(d.get('comment', '')).upper())
_other_count = len(aggregation_raw_deals) - _fa_count
_log(
    f"■ Deal scope: {_other_count} CH/FD/DD (last trading day) + {_fa_count} FA (last trading day)"

# Pre-compute full FA trading-day counts per account from full history,
# then use this count as the authoritative Hedge Day slot index.
fa_day_keys_by_account = {}
for _d in all_history_deals:
    _comment = str(_d.get('comment', '') or '')
    if '_FA' not in _comment.upper():
        continue

    _parsed = self.pusher.parse_deal_comment_v2(_comment)
    if not _parsed:
        continue
    # parse_deal_comment_v2 may return either a ParsedComment-like object
    # or a plain dict (parse_mt5_comment convenience path).
    _parsed_is_dict = isinstance(_parsed, dict)
    _is_valid = (_parsed.get('is_valid') if _parsed_is_dict else getattr(_parsed, 'is_valid', True))
    if _is_valid is False:
        continue

    _account_key = str(
        (_parsed.get('account_number') if _parsed_is_dict else getattr(_parsed, 'account_number',
        '')) or ''
    ).strip()
    if not _account_key:
        continue

    _day_key = ''
    _parsed_date = (_parsed.get('farming_date') if _parsed_is_dict else getattr(_parsed,
    'farming_date', None))
    if _parsed_date:
        try:
            if hasattr(_parsed_date, 'strftime'):
                _day_key = _parsed_date.strftime('%Y-%m-%d')
            else:
                _day_key = str(_parsed_date)[:10]
        except Exception:
            _day_key = str(_parsed_date)[:10]

    if not _day_key:

```

```

        _ts = _d.get('time_raw', 0)
        if isinstance(_ts, (int, float)) and _ts > 0:
            try:
                _day_key = datetime.fromtimestamp(_ts).strftime('%Y-%m-%d')
            except Exception:
                _day_key = ''

        if _day_key:
            fa_day_keys_by_account.setdefault(_account_key, set()).add(_day_key)

fa_day_count_by_account = {
    acc: len(days)
    for acc, days in fa_day_keys_by_account.items()
    if days
}

# FNFT challenge filter: challenge accounts reuse the same account numbers
# across resets, so only keep last 24h of deals with _CH in the comment.
# Funded (_FU, _FD, _DD, _FA) deals keep the full date range.
is_fnft = False
try:
    srv = str(self.pusher.server or '').upper()
    cmp = str(getattr(self.pusher, 'company', '') or '').upper()
    if 'FUNDEDNEXT' in srv or 'FNFT' in srv or 'FUNDEDNEXT' in cmp or 'FNFT' in cmp:
        is_fnft = True
except Exception:
    pass

if is_fnft:
    _24h_ago = time.time() - 86400
    if raw_deals:
        before_count = len(raw_deals)
        raw_deals = [
            d for d in raw_deals
            if '_CH' not in str(d.get('comment', '')).upper()
            or d.get('time_raw', 0) >= _24h_ago
        ]
        dropped = before_count - len(raw_deals)
        if dropped:
            _log(f"■ Filtered {dropped} old FNFT challenge deal(s) (>24h)")
    if aggregation_raw_deals:
        aggregation_raw_deals = [
            d for d in aggregation_raw_deals
            if '_CH' not in str(d.get('comment', '')).upper()
            or d.get('time_raw', 0) >= _24h_ago
        ]

# Filter deals: keep balance ops and trades with valid comments.
# FA-history-only deals are pre-validated (added because comment has _FA) - skip re-parse.
def _filter_valid_push_deals(source_deals, skip_fa_reparse=False):
    filtered = []
    for deal in source_deals or []:
        d_type = str(deal.get('type', '')).upper()
        if d_type in ['BALANCE', 'CREDIT', '2', '3', 'CHARGE', 'CORRECTION', 'BONUS']:
            # Drop internal transfers – they are NOT real P&L and must
            # never reach statistics or the dashboard. Catches both:
            # - explicit "internal transfer" comment (any sign)
            # - empty-comment balance ops (legacy untagged transfers)
            _bal_comment_l = str(deal.get('comment', '') or '').strip().lower()
            if ('internal transfer' in _bal_comment_l) or (not _bal_comment_l):

```

```

        continue
    filtered.append(deal)
    continue

    if skip_fa_reparse and deal.get('_fa_history_only'):
        filtered.append(deal)
        continue

    # Also skip re-parsing for any FA-tagged deal when skip_fa_reparse=True.
    # All FA deals were pre-validated by the _FA comment check in the build loop.
    if skip_fa_reparse and '_FA' in str(deal.get('comment', '')).upper():
        filtered.append(deal)
        continue

    comment = deal.get('comment', '')
    parsed = self.pusher.parse_deal_comment_v2(comment)

    is_valid = False
    if parsed:
        if hasattr(parsed, 'is_valid'):
            is_valid = parsed.is_valid
        else:
            is_valid = True

    if is_valid:
        filtered.append(deal)
    return filtered
deals = _filter_valid_push_deals(raw_deals)
aggregation_deals = _filter_valid_push_deals(aggregation_raw_deals, skip_fa_reparse=True)

if len(deals) < len(raw_deals):
    _log(f"Filtered {len(raw_deals) - len(deals)} deals with invalid comments")

statistics = self.pusher.calculate_statistics(deals)

# Aggregate hedge results locally, including farming history for correct FA slotting.
aggregated_by_comment = []
comment_summary = {}
latest_fa_aggregates = []
if COMMENT_PARSER_AVAILABLE and aggregation_deals:
    aggregated_by_comment, _unmatched, _agg_log = aggregate_deals_by_position(aggregation_deals)

    # Keep open-position aggregates strictly on the current trading day.
    # This prevents stale multi-day open entries from re-triggering FA pushes.
    _today_local = datetime.now().date()

    def _is_current_trading_day_open(agg):
        ts = agg.get('timestamp')
        if isinstance(ts, (int, float)) and ts > 0:
            try:
                return datetime.fromtimestamp(ts).date() == _today_local
            except Exception:
                pass

    farming_date = str(agg.get('farming_date') or '').strip()
    if len(farming_date) >= 10:
        try:
            return datetime.fromisoformat(farming_date.replace('Z', '+00:00')).date()
        except Exception:
            pass

```

```

        try:
            return datetime.strptime(farming_date[:10], '%Y-%m-%d').date() == _today_local
        except Exception:
            pass
    return False

if aggregated_by_comment:
    _fresh_open_aggregates = []
    _dropped_stale_open = 0
    for agg in aggregated_by_comment:
        if bool(agg.get('has_open_position')) and not _is_current_trading_day_open(agg):
            _dropped_stale_open += 1
            continue
        _fresh_open_aggregates.append(agg)

    if _dropped_stale_open:
        _log(
            f"■ Suppressed {_dropped_stale_open} stale open trade group(s) from prior trading"
        )
    aggregated_by_comment = _fresh_open_aggregates

def _normalize_fa_day(agg):
    timestamp = agg.get('timestamp')
    if isinstance(timestamp, (int, float)) and timestamp > 0:
        try:
            return datetime.fromtimestamp(timestamp).strftime('%Y-%m-%d')
        except Exception:
            pass
    farming_date = str(agg.get('farming_date') or '').strip()
    if len(farming_date) >= 10:
        return farming_date[:10]
    return farming_date

fa_by_account_day = {}
non_fa_aggregates = []
for agg in aggregated_by_comment:
    if agg.get('phase_code') != 'FA':
        non_fa_aggregates.append(agg)
        continue

    day_key = _normalize_fa_day(agg)
    account_key = str(agg.get('account_number') or '')
    merge_key = (account_key, day_key)
    existing = fa_by_account_day.get(merge_key)
    if not existing:
        fa_by_account_day[merge_key] = dict(agg)
        continue

    existing['net_profit'] = round(existing.get('net_profit', 0) + agg.get('net_profit', 0), 2)
    existing['deal_count'] = existing.get('deal_count', 0) + agg.get('deal_count', 0)
    if agg.get('timestamp', 0) > existing.get('timestamp', 0):
        existing['timestamp'] = agg.get('timestamp', 0)
        existing['farming_date'] = agg.get('farming_date')

latest_fa_aggregates = []
fa_accounts = {}
for (account_key, day_key), agg in fa_by_account_day.items():
    fa_accounts.setdefault(account_key, []).append((day_key, agg))

for account_key, entries in fa_accounts.items():
    entries.sort(key=lambda item: item[0])

```

```

        total_slots = fa_day_count_by_account.get(account_key, len(entries))
        # Push only the LATEST farming day, labelled as Hedge Day <total_slots>.
        # total_slots is derived from the full 90-day history so the slot number
        # is always correct even though we only send one entry per account.
        latest_day_key, latest_agg = entries[-1]
        tagged = dict(latest_agg)
        tagged['trade_number'] = total_slots
        tagged['_fa_slot'] = total_slots
        tagged['field_name'] = f"Hedge Day {total_slots}"
        latest_fa_aggregates.append(tagged)
        _log(f"■ {account_key}: {total_slots} FA day(s) → pushing as Hedge Day {total_slots}")

    aggregated_by_comment = non_fa_aggregates + latest_fa_aggregates

    # Guard against false FA zero pushes when there are no active positions.
    # We still keep zero FA rows when positions are active (user-visible signal that a trade is r
    has_active_positions = bool(positions)
    if not has_active_positions and aggregated_by_comment:
        filtered_aggregates = []
        dropped_fa_zeros = 0
        for agg in aggregated_by_comment:
            is_fa = agg.get('phase_code') == 'FA'
            net_profit = float(agg.get('net_profit', 0) or 0)
            deal_count = int(agg.get('deal_count', 0) or 0)
            has_open_position = bool(agg.get('has_open_position'))
            if is_fa and abs(net_profit) < 1e-9 and deal_count <= 1 and not has_open_position:
                dropped_fa_zeros += 1
                continue
            filtered_aggregates.append(agg)
        if dropped_fa_zeros:
            _log(f"■ Suppressed {dropped_fa_zeros} stale FA zero row(s) (no open FA trade)")
            aggregated_by_comment = filtered_aggregates

    by_phase = {}
    for agg in aggregated_by_comment:
        phase_name = agg.get('phase_name', 'UNKNOWN')
        if phase_name not in by_phase:
            by_phase[phase_name] = {'count': 0, 'total_net_profit': 0.0}
        by_phase[phase_name]['count'] += 1
        by_phase[phase_name]['total_net_profit'] += agg.get('net_profit', 0)
    comment_summary = {'by_phase': by_phase}

    # Collect Tradovate MNQ daily P&L for Prop Day values.
    # ONLY runs if at least one Tradovate account is actively connected.
    # Cache-miss fetches are done in a background thread so they never add
    # to push wall-time – the result is available for the NEXT push.
    tradovate_farming_days = []
    if not hasattr(self, '_tradovate_farming_cache'):
        self._tradovate_farming_cache = {}
    _tv_now = time.time()

    # Find all connected Tradovate accounts (account object must be present).
    _connected_tv = [
        (firm_name, conn.get("account"))
        for firm_name, conn in self._broker_connections.items()
        if conn.get("account") and hasattr(conn.get("account"), 'get_mnq_daily_pnl')
    ]

    if not _connected_tv:
        if latest_fa_aggregates:

```

```

        _log("■ Tradovate not connected – Prop Day values will not be updated", "WARN")
    else:
        for firm_name, tv_account in _connected_tv:
            _tv_cached = self._tradovate_farming_cache.get(firm_name)
            if _tv_cached and (_tv_now - _tv_cached[0]) < self._TRADOVATE_FARMING_CACHE_TTL:
                # Cache hit – instant, no API call.
                mnq_data = _tv_cached[1]
                if mnq_data:
                    total_days = sum(len(a.get('mnq_daily_pnl', [])) for a in mnq_data)
                    _log(f"■ {firm_name}: {total_days} Prop Day(s) ready (cached)")
                    tradovate_farming_days.extend(mnq_data)
            else:
                # Cache miss – use whatever we have now (empty on first push),
                # and refresh asynchronously so the NEXT push benefits.
                existing = (_tv_cached[1] if _tv_cached else []) or []
                if existing:
                    total_days = sum(len(a.get('mnq_daily_pnl', [])) for a in existing)
                    _log(
                        f"■ {firm_name}: {total_days} Prop Day(s) from stale cache (refreshing in background)"
                    )
                    tradovate_farming_days.extend(existing)
                else:
                    _log(f"■ {firm_name}: fetching Tradovate history in background (available next push)")

        def _refresh_tv_cache(fn=firm_name, acc=tv_account):
            try:
                data = acc.get_mnq_daily_pnl() or []
                self._tradovate_farming_cache[fn] = (time.time(), data)
            except Exception as _e:
                pass # silently swallow; no-op on next push miss
            threading.Thread(target=_refresh_tv_cache, daemon=True).start()

        # Cross-check: log Prop Day coverage vs Hedge Day coverage.
        if latest_fa_aggregates and tradovate_farming_days:
            for fa_agg in latest_fa_aggregates:
                acc_key = str(fa_agg.get('account_number') or '')
                hedge_slot = fa_agg.get('_fa_slot', '?')
                acc_digits = [d for d in re.findall(r'\d+', acc_key.upper()) if len(d) >= 4]
                for tv_data in tradovate_farming_days:
                    tv_name = (tv_data.get('account_name') or '').upper()
                    if acc_key.upper() in tv_name or any(d in tv_name for d in acc_digits):
                        prop_count = len(tv_data.get('mnq_daily_pnl', []))
                        _log(f"■ {acc_key}: Hedge Day {hedge_slot} ↔ {prop_count} Prop Day(s)")
                        break

        # Pre-push diagnostic: log what we're about to send
        pos_count = len(positions) if positions else 0
        deal_count = len(deals) if deals else 0
        agg_count = len(aggregated_by_comment) if aggregated_by_comment else 0
        bal = account.get('balance', 0)
        _log(f"■ Payload: Bal=${bal:,.0f} | {deal_count} deals | {pos_count} pos | {agg_count} hedge groups")

        # Detailed per-row logging of what's being pushed
        if aggregated_by_comment:
            _log(f"\n■ INDIVIDUAL ROWS BEING PUSHED:")
            _log(f"{'='*80}")

            # Group by account for summary
            by_account = {}
            by_phase = {}

            for i, agg in enumerate(aggregated_by_comment):

```

```

account_num = agg.get('account_number', 'Unknown')
phase_code = agg.get('phase_code', 'UNKNOWN')
phase_name = agg.get('phase_name', 'Unknown')
trade_num = agg.get('trade_number', 0)
net_profit = agg.get('net_profit', 0)
deal_cnt = agg.get('deal_count', 1)
field_name = agg.get('field_name', '?')

# Track per account
if account_num not in by_account:
    by_account[account_num] = {'rows': [], 'total_profit': 0, 'total_deals': 0}
by_account[account_num]['rows'].append({
    'phase': f"{phase_code}{trade_num}",
    'profit': net_profit,
    'deals': deal_cnt,
    'field': field_name
})
by_account[account_num]['total_profit'] += net_profit
by_account[account_num]['total_deals'] += deal_cnt

# Track per phase
if phase_code not in by_phase:
    by_phase[phase_code] = {'count': 0, 'total_profit': 0}
by_phase[phase_code]['count'] += 1
by_phase[phase_code]['total_profit'] += net_profit

# Log individual row
farming_date = agg.get('farming_date', '')
date_str = f" ({farming_date})" if farming_date else ""
_log(
    f"    [{i+1:2d}] {account_num}_{phase_code}{trade_num} → {field_name:20s} = ${net_pro

_log(f"'='*80}")
_log(f"\n■ SUMMARY BY ACCOUNT:")
for account_num in sorted(by_account.keys()):
    data = by_account[account_num]
    row_count = len(data['rows'])
    total_p = data['total_profit']
    total_d = data['total_deals']
    phases = ', '.join(r['phase'] for r in data['rows'])
    _log(
        f"    {account_num:20s} | {row_count:2d} row(s) | Phases: {phases:30s} | Total: ${total_p:10s}

_log(f"\n■ SUMMARY BY PHASE:")
for phase_code in sorted(by_phase.keys()):
    data = by_phase[phase_code]
    phase_names = {'CH': 'Challenge', 'FD': 'Funded', 'DD': 'DoubleDip', 'FA': 'Farming'}
    phase_name = phase_names.get(phase_code, phase_code)
    _log(
        f"    {phase_name:15s} ({phase_code}) | {data['count']:2d} group(s) | Total: ${data['total_profit']:10s}

payload = {
    "email": email,
    "account": account,
    "positions": positions,
    "deals": deals,
    "statistics": statistics,
    "evaluations": [],
    "aggregated_by_comment": aggregated_by_comment,
    "prefer_client_aggregation": True,

```



```

        "comment_summary": comment_summary,
        "dropdown_options": {},
        "firm_billing": self._firm_billing_summary if self._firm_billing_summary else None,
    }

# Only include Tradovate prop day data if we actually have it.
# Omitting the key entirely means the server will never touch existing Prop Day values.
if tradovate_farming_days:
    payload["tradovate_farming_days"] = tradovate_farming_days

try:
    # Use public endpoint - no API key needed
    response = _gzip_post(
        f"{dashboard_url}/api/client/push",
        payload,
        timeout=120
    )

    if response.status_code == 200:
        try:
            data = response.json()
        except ValueError:
            _log("■ Server returned invalid JSON (not JSON response)", "ERROR")
            _status("Push failed - bad response")
            return
        if data.get("status") == "success":
            bal = account.get('balance', 0)
            dep = account.get('total_deposits', 0)
            hedge_log = data.get("hedge_match_log", [])
            hedge_updates = data.get("hedge_updates", 0)
            _log(
                f"\n■ PUSH SUCCESSFUL → Bal: ${bal:,.0f} | Dep: ${dep:,.0f} | {len(deals)} deals"

            )

            # Log detailed response from server showing what was processed
            _log(f"\n■ SERVER PROCESSING LOG ({len(hedge_log)} entries):")
            _log(f"{'='*80}")
            for i, entry in enumerate(hedge_log, 1):
                # Highlight different types of entries
                if entry.startswith("■"):
                    _log(f"    {entry}", "INFO")
                elif entry.startswith("■■") or entry.startswith("■"):
                    _log(f"    {entry}", "WARN")
                elif entry.startswith("■") or entry.startswith("■") or entry.startswith("■"):
                    _log(f"    {entry}", "INFO")
                else:
                    _log(f"    {entry}", "INFO")
            _log(f"{'='*80}")

            if hedge_updates:
                _log(f"\n■ CONFIRMED: {hedge_updates} hedge cell(s) updated and saved on dashboa")
            elif agg_count:
                _log(
                    "\n■■ Server acknowledged push but returned 0 hedge updates. Check account-n")
            else:
                _log("\n■ No hedge aggregates in this push (MT5 data only)")

            _status("Ready - Data pushed!")
            try:
                self.root.after(0, lambda hu=hedge_updates: self._stat_push_var.set(f"Push: ✓ {h
            except Exception:

```

```

        pass
    else:
        _log(f"■ {data.get('message', 'Push failed')}", "ERROR")
        _status("Push failed")
    else:
        error_msg = f"HTTP {response.status_code}"
        try:
            error_msg = response.json().get("message", error_msg)
        except Exception:
            pass
        _log(f"■ Push failed: {error_msg}", "ERROR")
        _status("Push failed")

except requests.exceptions.Timeout:
    _log("■ Push timeout – server did not respond within 120s", "ERROR")
    _status("Push failed - timeout")
except requests.exceptions.ConnectionError:
    _log("■ Push failed – cannot connect to server", "ERROR")
    _status("Push failed - no connection")
except Exception as e:
    _log(f"■ Push error: {e}", "ERROR")
    _status("Push failed")

# Run hedging review in the background – uses account data already in hand, no second MT5 call.
try:
    threading.Thread(
        target=self._push_hedging_review_worker,
        args=(dashboard_url, email, account),
        daemon=True
    ).start()
except Exception as hr_err:
    _log(f"■ Hedging review thread failed to start: {hr_err}", "ERROR")

```

Method: TradeOpssAIApp.push_hedging_review

```
def push_hedging_review(self)
```

Push hedging review data — called from toolbar button (fetches own account info).

Step by step (top-level body)

1. Assigns `dashboard_url = self.url_entry.get().strip().rstrip('/')`.
2. Assigns `email = self.client_email_entry.get().strip()`.
3. `if not self.client_info`: branches conditionally.
4. `if not self.pusher.connected`: branches conditionally.
5. Calls `self.status_var.set(...)` for its side effect.
6. Assigns `account = self.pusher.get_account_info(include_balance_history=True)`.
7. `if not account`: branches conditionally.
8. Calls `threading.Thread(target=self._push_hedging_review_worker, args=(dashboard_url, email, account), daemon=True).start(...)` for its side effect.

Code (copy-paste safe)

```
def push_hedging_review(self):
    """Push hedging review data — called from toolbar button (fetches own account info)."""
```

```

dashboard_url = self.url_entry.get().strip().rstrip('/')
email = self.client_email_entry.get().strip()

if not self.client_info:
    messagebox.showerror("Error",
        "Please lookup the client first by entering email and clicking 'Lookup'")
    return

if not self.pusher.connected:
    messagebox.showerror("Error", "Please connect to MT5 first")
    return

self.status_var.set("Pushing hedging review...")
account = self.pusher.get_account_info(include_balance_history=True)
if not account:
    self.log("■■ Hedging review skipped – MT5 account info is empty", "ERROR")
    self.status_var.set("Hedging review skipped")
    return

threading.Thread(
    target=self._push_hedging_review_worker,
    args=(dashboard_url, email, account),
    daemon=True
).start()

```

Method: TradeOpssAIApp._push_hedging_review_worker

```
def _push_hedging_review_worker(self, dashboard_url, email, account)
```

Send hedging review payload — refreshes account totals in the background if needed.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.

Step by step (top-level body)

1. Defines a nested function `_log(...)`.
2. Defines a nested function `_status(...)`.
3. `if self.pusher.connected:` branches conditionally.
4. Assigns `deposits = float(account.get('total_deposits', 0) or 0)`.
5. Assigns `withdrawals = float(account.get('total_withdrawals', 0) or 0)`.
6. Assigns `balance = float(account.get('balance', 0) or 0)`.
7. Calls `_log(...)` for its side effect.

8. Assigns payload = {'email': email, 'total_deposits': deposits, 'total_withd....

9. try block with 3 except clauses.

Code (copy-paste safe)

```
def _push_hedging_review_worker(self, dashboard_url, email, account):
    """Send hedging review payload – refreshes account totals in the background if needed."""
    def _log(msg, level="INFO"):
        self.root.after(0, lambda m=msg, lv=level: self.log(m, lv))
    def _status(msg):
        self.root.after(0, lambda m=msg: self.status_var.set(m))

    if self.pusher.connected:
        try:
            account = self.pusher.get_account_info(include_balance_history=True) or account
        except Exception:
            pass

    deposits = float(account.get('total_deposits', 0) or 0)
    withdrawals = float(account.get('total_withdrawals', 0) or 0)
    balance = float(account.get('balance', 0) or 0)

    _log(f"■ HR payload: Dep=${deposits:,.0f} | Wth=${withdrawals:,.0f} | Bal=${balance:,.0f}")

    payload = {
        "email": email,
        "total_deposits": deposits,
        "total_withdrawals": withdrawals,
        "current_balance": balance
    }

    try:
        response = requests.post(
            f"{dashboard_url}/api/client/push_hedging_review",
            json=payload,
            headers={"Content-Type": "application/json"},
            timeout=30
        )

    if response.status_code == 200:
        try:
            data = response.json()
        except ValueError:
            _log("■ Hedging review: server returned invalid JSON", "ERROR")
            _status("HR failed - bad response")
            return
        if data.get("status") == "success":
            hr = data.get("hedging_review") or {}
            actual = hr.get('actual_hedging_results', 0)
            disc = hr.get('discrepancy', 0)
            _log(
                f"■ Hedging Review → Dep: ${deposits:,.0f} | Wth: ${withdrawals:,.0f} | Actual: ${actual:,.0f} | Disc: ${disc:,.0f}"
            )
            _status("Ready - Hedging review pushed!")
        else:
            _log(f"■ {data.get('message', 'Push failed')}", "ERROR")
            _status("Push failed")
    else:
        error_msg = f"HTTP {response.status_code}"
        try:
            error_msg = response.json().get("message", error_msg)
```

```

        except Exception:
            pass
        _log(f"■ Push failed: {error_msg}", "ERROR")
        _status("Push failed")

    except requests.exceptions.Timeout:
        _log("■ Hedging review timeout", "ERROR")
        _status("HR timeout")
    except requests.exceptions.ConnectionError:
        _log("■ Hedging review – cannot connect", "ERROR")
        _status("HR connection failed")
    except Exception as e:
        _log(f"■ Hedging review error: {e}", "ERROR")
        _status("Push failed")

```

Method: TradeOpssAIApp.push_mt5_only

```
def push_mt5_only(self)
```

Push ONLY MT5 data (deals, positions, account) to recalculate hedging review.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `dashboard_url = self.url_entry.get().strip().rstrip('/')`.
2. Assigns `email = self.client_email_entry.get().strip()`.
3. **if not self.client_info**: branches conditionally.
4. **if not self.pusher.connected**: branches conditionally.
5. Assigns `client_name = self.client_info.get('client', '')`.
6. Calls `self.log(...)` for its side effect.
7. Calls `self.status_var.set(...)` for its side effect.
8. Assigns `account = self.pusher.get_account_info()` or `{}`.
9. Assigns `deals = self.pusher.get_deals(days=365)`.
10. **if not account**: branches conditionally.
11. Assigns `balance = account.get('balance', 0)`.
12. Assigns `deposits = account.get('total_deposits', 0)`.
13. Assigns `withdrawals = account.get('total_withdrawals', 0)`.
14. Assigns `actual_hedging = 0.0`.
15. ... and 5 more statement(s) in the body.

Code (copy-paste safe)

```

def push_mt5_only(self):
    """Push ONLY MT5 data (deals, positions, account) to recalculate hedging review."""
    dashboard_url = self.url_entry.get().strip().rstrip('/')
    email = self.client_email_entry.get().strip()

    if not self.client_info:
        messagebox.showerror("Error",
            "Please lookup the client first by entering email and clicking 'Lookup'")
        return

    if not self.pusher.connected:
        messagebox.showerror("Error", "Please connect to MT5 first to push MT5 data")
        return

    client_name = self.client_info.get('client', '')

    self.log(f"■ MT5 rebalance push for {client_name}...")
    self.status_var.set("Pushing MT5 data...")

    # Get MT5 data
    account = self.pusher.get_account_info() or {}
    deals = self.pusher.get_deals(days=365) # Get 1 year of deals for hedging calculations

    if not account:
        self.log("■■ No account info available", "ERROR")
        messagebox.showerror("Error", "Could not retrieve account information from MT5.")
        return

    # Extract rebalance data directly from account info (MT5 provides cumulative totals)
    balance = account.get('balance', 0)
    deposits = account.get('total_deposits', 0)
    withdrawals = account.get('total_withdrawals', 0)

    # Calculate actual hedging results from deals (non-BALANCE trades)
    actual_hedging = 0.0
    trade_count = 0
    for deal in (deals or []):
        d_type = str(deal.get('type', '')).upper()
        if d_type not in ['BALANCE', 'CREDIT', '2', '3']: # Skip balance and credit operations
            profit = deal.get('profit', 0) or 0
            swap = deal.get('swap', 0) or 0
            commission = deal.get('commission', 0) or 0
            actual_hedging += (profit + swap + commission)
            if deal.get('entry') == 'OUT': # Count closed trades
                trade_count += 1

    self.log(
        f"■ Bal: ${balance:,.0f} | Dep: ${deposits:,.0f} | Wth: ${withdrawals:,.0f} | Hedge: ${actual_hedging:,.0f}"
    )

    payload = {
        "email": email,
        "account": account,
        "positions": [],
        "deals": deals or [], # Include deals for actual hedging calculation
        "statistics": {}, # Let server recalculate with MT5 data
        # NOTE: Do NOT include "evaluations" key - server will preserve existing data
        "dropdown_options": {}
    }

    try:

```

```

response = _gzip_post(
    f"{dashboard_url}/api/client/push",
    payload,
    timeout=120
)

if response.status_code == 200:
    data = response.json()

    if data.get("status") == "success":
        self.log(f"■ Rebalance pushed — Bal: ${balance:,.0f} | Dep: ${deposits:,.0f}")
        self.status_var.set("Rebalance data pushed!")
    else:
        self.log(f"■ Push failed: {data.get('message', 'Unknown error')}", "ERROR")
        self.status_var.set("Push failed")
else:
    error_msg = f"HTTP {response.status_code}"
    try:
        error_msg = response.json().get("message", error_msg)
    except:
        pass
    self.log(f"■ Rebalance push failed: {error_msg}", "ERROR")
    self.status_var.set("Push failed")

except Exception as e:
    self.log(f"■ Push error: {e}", "ERROR")
    self.status_var.set("Push failed")

```

Method: TradeOpssAIApp.show_deal_comments

```
def show_deal_comments(self)
```

Debug function to show all deal comments from MT5.

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **if** not self.pusher.connected: branches conditionally.
2. Calls self.log(...) for its side effect.
3. Calls self.log(...) for its side effect.
4. Calls self.log(...) for its side effect.
5. Assigns deals = self.pusher.get_deals(days=365).
6. **if** not deals: branches conditionally.
7. Calls self.log(...) for its side effect.
8. Assigns comment_counts = {}.
9. **for** deal in deals: iterates.
10. Calls self.log(...) for its side effect.
11. Calls self.log(...) for its side effect.

12. `for (comment, info) in sorted(comment_counts.items())`: iterates.

13. Calls `self.log(...)` for its side effect.

14. Calls `self.log(...)` for its side effect.

15. ... and 4 more statement(s) in the body.

Code (copy-paste safe)

```
def show_deal_comments(self):
    """Debug function to show all deal comments from MT5."""
    if not self.pusher.connected:
        messagebox.showerror("Error", "Please connect to MT5 first")
        return

    self.log("=*60)
    self.log("■ MT5 DEAL COMMENTS DEBUG")
    self.log("=*60)

    deals = self.pusher.get_deals(days=365)

    if not deals:
        self.log("No deals found")
        return

    self.log(f"Total deals: {len(deals)}\n")

    # Group by unique comments
    comment_counts = {}
    for deal in deals:
        comment = deal.get('comment', '') or '(empty)'
        d_type = deal.get('type', '')
        if d_type not in ['BALANCE', 'CREDIT', '2', '3']: # Skip balance ops
            if comment not in comment_counts:
                comment_counts[comment] = {'count': 0, 'total_profit': 0, 'sample_deal': deal}
            comment_counts[comment]['count'] += 1
            comment_counts[comment]['total_profit'] += (deal.get('profit', 0) or 0)

    self.log(f"Unique comments: {len(comment_counts)}\n")
    self.log("=-*60)

    for comment, info in sorted(comment_counts.items()):
        parsed = self.pusher.parse_deal_comment(comment)

        self.log(f"\n■ Comment: '{comment}'")
        self.log(f"    Deals: {info['count']}, Total P/L: ${info['total_profit']:.2f}")

        if parsed:
            self.log(f"    ✓ Parsed -> Account: {parsed['account_suffix']}")
            if parsed.get('stage'):
                self.log(f"    ✓ Stage: {parsed['stage']}{parsed['stage_num']}")
            else:
                self.log(f"    ■■ No stage pattern found (CH/FU/FA)")
        else:
            self.log(f"    ■ Could not parse comment")

    self.log("\n" + "=*60)
    self.log("■ Comment format expected:")
    self.log("    Challenge: ..._CH{n} or ...CH{n}")
    self.log("    Funded:    ..._FU{n} or ...FU{n}")
```



```
self.log("    Farming:    ..._FA{n}_DD/MM or ...FA{n}")
self.log("="*60)
```

Method: TradeOpssAIApp.sync_hedge_results

```
def sync_hedge_results(self)
```

Sync hedge results from MT5 deal comments to evaluation records. Parses deal comments to extract account number and stage: - Challenge: {account}_CH{n} - Funded: {account}_FU{n} - Farming: {account}_FA{n}_{DD/MM} Then updates the appropriate Hedge Result fields in evaluations.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **list comprehension** — Builds a list in a single expression ([f(x) for x in xs]). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns dashboard_url = self.url_entry.get().strip().rstrip('/').
2. Assigns email = self.client_email_entry.get().strip().
3. **if** not self.client_info: branches conditionally.
4. **if** not self.pusher.connected: branches conditionally.
5. Assigns client_name = self.client_info.get('client', '').
6. Calls self.log(...) for its side effect.
7. Calls self.log(...) for its side effect.
8. Calls self.log(...) for its side effect.
9. Calls self.status_var.set(...) for its side effect.
10. Calls self.log(...) for its side effect.
11. **try** block with 1 **except** clause.
12. Calls self.log(...) for its side effect.
13. Assigns deals = self.pusher.get_deals(days=365).
14. **if** not deals: branches conditionally.
15. ... and 11 more statement(s) in the body.

Code (copy-paste safe)

```
def sync_hedge_results(self):
    """
    Sync hedge results from MT5 deal comments to evaluation records.
```

```

Parses deal comments to extract account number and stage:
- Challenge: {account}_CH{n}
- Funded: {account}_FU{n}
- Farming: {account}_FA{n}_{DD/MM}

Then updates the appropriate Hedge Result fields in evaluations.
"""
dashboard_url = self.url_entry.get().strip().rstrip('/')
email = self.client_email_entry.get().strip()

if not self.client_info:
    messagebox.showerror("Error",
        "Please lookup the client first by entering email and clicking 'Lookup'")
    return

if not self.pusher.connected:
    messagebox.showerror("Error", "Please connect to MT5 first")
    return

client_name = self.client_info.get('client', '')

self.log("=*60)
self.log("■ SYNC HEDGE RESULTS FROM MT5 COMMENTS")
self.log("=*60)
self.status_var.set("Syncing hedge results...")

# Step 1: Get current evaluations from dashboard
self.log("\n■ Step 1: Fetching current evaluations from dashboard...")
try:
    response = requests.get(
        f"{dashboard_url}/api/data?client_id={client_name}",
        cookies=self.session_cookies if hasattr(self, 'session_cookies') else {},
        timeout=60
    )

    if response.status_code != 200:
        self.log(f"■ Failed to fetch data: HTTP {response.status_code}", "ERROR")
        messagebox.showerror("Error",
            "Could not fetch current data from dashboard. Try logging in via browser first.")
        return

    data = response.json()
    evaluations = data.get('evaluations', [])

    if not evaluations:
        self.log("■■ No evaluations found in dashboard", "WARNING")
        messagebox.showwarning("Warning",
            "No evaluations found. Please import from Google Sheets first.")
        return

    self.log(f" ✓ Found {len(evaluations)} evaluation records")

except Exception as e:
    self.log(f"■ Error fetching data: {e}", "ERROR")
    messagebox.showerror("Error", f"Failed to fetch data: {e}")
    return

# Step 2: Get deals from MT5
self.log("\n■ Step 2: Fetching deals from MT5...")
deals = self.pusher.get_deals(days=365) # Get 1 year of deals

```

```

if not deals:
    self.log("■■■ No deals found in MT5", "WARNING")
    messagebox.showwarning("Warning", "No deals found in MT5 history.")
    return

self.log(f"    ✓ Found {len(deals)} deals")

# Show sample comments for debugging
comments_with_data = [d.get('comment', '') for d in deals if d.get('comment')]
unique_comments = list(set(comments_with_data))[:10]
self.log(f"\n■ Sample deal comments found:")
for c in unique_comments:
    parsed = self.pusher.parse_deal_comment(c)
    if parsed and parsed.get('stage'):
        self.log(
            f"    ✓ '{c}' -> {parsed['stage']}{parsed['stage_num']}, account: {parsed['account_suffi"
        elif parsed and parsed.get('account_suffix'):
            self.log(f"    . '{c}' -> account: {parsed['account_suffix']} (no stage found)")
        else:
            self.log(f"    . '{c}' (not matching pattern)")

# Step 3: Process deals and update evaluations
self.log("\n■ Step 3: Processing deals and matching to evaluations...")
updated_evals, match_log = self.pusher.process_deals_for_evaluations(deals, evaluations)

for log_line in match_log:
    self.log(f"    {log_line}")

# Step 4: Push updated evaluations back to dashboard
self.log("\n■ Step 4: Pushing updated evaluations to dashboard...")

payload = {
    "email": email,
    "evaluations": updated_evals,
    "statistics": {}, # Let server recalculate
    "dropdown_options": {}
}

try:
    response = _gzip_post(
        f"{dashboard_url}/api/client/push",
        payload,
        timeout=120
    )

    if response.status_code == 200:
        data = response.json()
        if data.get("status") == "success":
            self.log(f"\n■ HEDGE RESULTS SYNCED SUCCESSFULLY!")
            self.log(f"    Updated {len(updated_evals)} evaluation records")
            self.log(f"="*60)
            self.status_var.set("Hedge results synced!")
            messagebox.showinfo("Success", "Hedge results synced from MT5 comments!")
        else:
            self.log(f"■ Sync failed: {data.get('message', 'Unknown error')}", "ERROR")
            self.status_var.set("Sync failed")
    else:
        self.log(f"■ HTTP {response.status_code}", "ERROR")
        self.status_var.set("Sync failed")

except Exception as e:

```

```
self.log(f"■ Sync error: {e}", "ERROR")
self.status_var.set("Sync failed")
```

Method: TradeOpssAIApp.analyze_comments_v2

```
def analyze_comments_v2(self)
```

Analyze MT5 deal comments using the new TradeAccountConnector format. Shows detailed breakdown of:

- Comment format: {TradovateAccountNumber}{PhaseSuffix} - Phases: CH (Challenge), FD (Funded), DD (DoubleDip), FA (Farming)

Why these Python concepts were used

- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **if** not self.pusher.connected: branches conditionally.
2. Calls self.log(...) for its side effect.
3. Calls self.log(...) for its side effect.
4. Calls self.log(...) for its side effect.
5. **if** not COMMENT_PARSER_AVAILABLE: branches conditionally.
6. Assigns deals = self.pusher.get_deals(days=365).
7. **if** not deals: branches conditionally.
8. Calls self.log(...) for its side effect.
9. Assigns parser = MT5CommentParser().
10. Assigns comment_analysis = {}.
11. **for** deal in deals: iterates.
12. Calls self.log(...) for its side effect.
13. Calls self.log(...) for its side effect.
14. Assigns valid_comments = [].
15. ... and 13 more statement(s) in the body.

Code (copy-paste safe)

```
def analyze_comments_v2(self):
    """
    Analyze MT5 deal comments using the new TradeAccountConnector format.

    Shows detailed breakdown of:
    - Comment format: {TradovateAccountNumber}{PhaseSuffix}
    - Phases: CH (Challenge), FD (Funded), DD (DoubleDip), FA (Farming)
    """
    if not self.pusher.connected:
        messagebox.showerror("Error", "Please connect to MT5 first")
        return
```

```

self.log("="*70)
self.log("■ MT5 COMMENT ANALYSIS (TradeAccountConnector Format)")
self.log("="*70)

if not COMMENT_PARSER_AVAILABLE:
    self.log("■■■ Comment Parser module not available!", "ERROR")
    self.log("    Please ensure mt5_comment_parser.py is in the TradeOpssAI folder")
    return

deals = self.pusher.get_deals(days=365)

if not deals:
    self.log("No deals found")
    return

self.log(f"Total deals fetched: {len(deals)}\n")

# Use the new parser
parser = MT5CommentParser()

# Analyze all unique comments
comment_analysis = {}
for deal in deals:
    comment = deal.get('comment', '') or ''
    d_type = str(deal.get('type', '')).upper()

    if d_type in ['BALANCE', 'CREDIT', '2', '3']: # Skip balance and credit ops
        continue

    if comment not in comment_analysis:
        parsed = parser.parse(comment)
        comment_analysis[comment] = {
            'parsed': parsed,
            'count': 0,
            'total_profit': 0,
            'total_commission': 0,
            'total_swap': 0
        }

    comment_analysis[comment]['count'] += 1
    comment_analysis[comment]['total_profit'] += deal.get('profit', 0) or 0
    comment_analysis[comment]['total_commission'] += deal.get('commission', 0) or 0
    comment_analysis[comment]['total_swap'] += deal.get('swap', 0) or 0

self.log(f"Unique comments found: {len(comment_analysis)}\n")
self.log("-"*70)

# Group by validity
valid_comments = []
invalid_comments = []

for comment, data in comment_analysis.items():
    parsed = data['parsed']
    if parsed.is_valid:
        valid_comments.append((comment, data))
    else:
        invalid_comments.append((comment, data))

# Show valid comments first
self.log(f"\n■ VALID COMMENTS ({len(valid_comments)}):\n")

```

```

for comment, data in sorted(valid_comments, key=lambda x: x[0]):
    parsed = data['parsed']
    net_profit = data['total_profit'] + data['total_commission'] + data['total_swap']

    self.log(f"■ '{comment}'")
    self.log(f"    Account: {parsed.account_number}")
    self.log(f"    Phase: {parsed.phase.name} ({parsed.phase_code})")
    if parsed.trade_number:
        self.log(f"    Trade #: {parsed.trade_number}")
    if parsed.farming_date:
        self.log(f"    Farming Date: {parsed.farming_date.strftime('%Y-%m-%d')}")
    self.log(f"    Deals: {data['count']}, Net P/L: ${net_profit:.2f}")
    self.log("")

# Show invalid comments
if invalid_comments:
    self.log(f"\n■■ UNRECOGNIZED COMMENTS ({len(invalid_comments)}):\n")

    for comment, data in sorted(invalid_comments, key=lambda x: x[0]):
        net_profit = data['total_profit'] + data['total_commission'] + data['total_swap']
        self.log(f"■ '{comment or '(empty)'}'")
        self.log(f"    Deals: {data['count']}, Net P/L: ${net_profit:.2f}")
        self.log("")

self.log("-"*70)
self.log("\n■ EXPECTED COMMENT FORMATS:")
self.log("    Challenge:      {Account}_CH{1-4}      (e.g., MFFUEVSTP326057008_CH1)")
self.log("    Funded:         {Account}_FD{0-4}      (e.g., MFFUEVSTP326057008_FD2)")
self.log("    Double Dip:     {Account}_DD{1-4}      (e.g., MFFUEVSTP326057008_DD1)")
self.log("    Farming:        {Account}_FA           (e.g., MFFUEVSTP326057008_FA)")
self.log("    Farming+Date:   {Account}_FA_DDMMYY   (e.g., MFFUEVSTP326057008_FA_210126)")
self.log("="*70)

```

Method: TradeOpssAIApp.show_aggregated_data

```
def show_aggregated_data(self)
```

Show aggregated deal data by account and phase. Uses the new comment parser to group deals.

Why these Python concepts were used

- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.
- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. if not self.pusher.connected: branches conditionally.
2. Calls self.log(...) for its side effect.
3. Calls self.log(...) for its side effect.
4. Calls self.log(...) for its side effect.

5. Assigns `result = self.pusher.get_deals_grouped_by_phase(days=365)`.
6. Assigns `aggregated = result.get('aggregated', [])`.
7. Assigns `unmatched = result.get('unmatched', [])`.
8. Assigns `summary = result.get('summary', {})`.
9. Calls `self.log(...)` for its side effect.
10. Calls `self.log(...)` for its side effect.
11. **if** `not aggregated`: branches conditionally.
12. Assigns `by_account = {}`.
13. **for** `agg` in `aggregated`: iterates.
14. Calls `self.log(...)` for its side effect.
15. ... and 5 more statement(s) in the body.

Code (copy-paste safe)

```
def show_aggregated_data(self):
    """
    Show aggregated deal data by account and phase.
    Uses the new comment parser to group deals.
    """
    if not self.pusher.connected:
        messagebox.showerror("Error", "Please connect to MT5 first")
        return

    self.log("=*70)
    self.log("■ AGGREGATED DEAL DATA BY ACCOUNT/PHASE")
    self.log("=*70)

    result = self.pusher.get_deals_grouped_by_phase(days=365)

    aggregated = result.get('aggregated', [])
    unmatched = result.get('unmatched', [])
    summary = result.get('summary', {})

    self.log(f"\nTotal aggregation groups: {len(aggregated)}")
    self.log(f"Unmatched deals: {len(unmatched)}\n")

    if not aggregated:
        self.log("■■ No aggregated data available")
        self.log("    Make sure your deals have comments in the correct format")
        return

    # Group by account for display
    by_account = {}
    for agg in aggregated:
        account = agg.get('account_number', 'Unknown')
        if account not in by_account:
            by_account[account] = []
        by_account[account].append(agg)

    self.log("-=*70)

    for account, trades in sorted(by_account.items()):
        account_total = sum(t.get('net_profit', 0) for t in trades)
```

```

self.log(f"\n■ ACCOUNT: {account}")
self.log(f"    Total Net P/L: ${account_total:.2f}")
self.log("")

for trade in sorted(trades, key=lambda x: (x.get('phase_code', ''), x.get('trade_number',
    0) or 0)):
    phase_code = trade.get('phase_code', '?')
    phase_name = trade.get('phase_name', 'Unknown')
    trade_num = trade.get('trade_number', '')
    net_profit = trade.get('net_profit', 0)
    deal_count = trade.get('deal_count', 0)
    farming_date = trade.get('farming_date', '')

    label = f"{phase_code}{trade_num or ''}"
    if farming_date:
        label += f" ({farming_date})"

    self.log(f"    [{label}] {phase_name}: ${net_profit:.2f} ({deal_count} deals)")

# Show summary
self.log("\n" + "-"*70)
self.log("\n■ SUMMARY BY PHASE:")
by_phase = summary.get('by_phase', {})
for phase, data in sorted(by_phase.items()):
    self.log(
        f"    {phase}: {data.get('count', 0)} groups, Total: ${data.get('total_net_profit', 0):.2f}"
    )

```

Method: TradeOpssAIApp.push_by_comment

```
def push_by_comment(self)
```

Push hedge results to dashboard by matching MT5 order comments to evaluation accounts. This uses the TradeAccountConnector comment format: - {TradovateAccountNumber}_CH{1-4}: Challenge Hedge Results 1-5 - {TradovateAccountNumber}_FD{0-4}: Funded Hedge Results - {TradovateAccountNumber}_DD{1-4}: Double Dip (funded hedge results) - {TradovateAccountNumber}_FA or _FA_DDMMYY: Farming Hedge Days The function: 1. Aggregates MT5 deals by account/phase from comments 2. Sends aggregated data to dashboard 3. Dashboard matches account numbers and updates hedge results

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns dashboard_url = self.url_entry.get().strip().rstrip('/').
2. Assigns email = self.client_email_entry.get().strip().
3. **if** not self.client_info: branches conditionally.

4. `if not self.pusher.connected:` branches conditionally.
5. `if not COMMENT_PARSER_AVAILABLE:` branches conditionally.
6. Assigns `client_name = self.client_info.get('client', '')`.
7. Calls `self.log(...)` for its side effect.
8. Calls `self.log(...)` for its side effect.
9. Calls `self.log(...)` for its side effect.
10. Calls `self.log(...)` for its side effect.
11. Calls `self.log(...)` for its side effect.
12. Calls `self.log(...)` for its side effect.
13. Calls `self.log(...)` for its side effect.
14. Calls `self.log(...)` for its side effect.
15. ... and 23 more statement(s) in the body.

Code (copy-paste safe)

```
def push_by_comment(self):
    """
    Push hedge results to dashboard by matching MT5 order comments to evaluation accounts.

    This uses the TradeAccountConnector comment format:
    - {TradovateAccountNumber}_CH{1-4}: Challenge Hedge Results 1-5
    - {TradovateAccountNumber}_FD{0-4}: Funded Hedge Results
    - {TradovateAccountNumber}_DD{1-4}: Double Dip (funded hedge results)
    - {TradovateAccountNumber}_FA or _FA_DDMMYY: Farming Hedge Days

    The function:
    1. Aggregates MT5 deals by account/phase from comments
    2. Sends aggregated data to dashboard
    3. Dashboard matches account numbers and updates hedge results
    """
    dashboard_url = self.url_entry.get().strip().rstrip('/')
    email = self.client_email_entry.get().strip()

    if not self.client_info:
        messagebox.showerror("Error",
                              "Please lookup the client first by entering email and clicking 'Lookup'")
        return

    if not self.pusher.connected:
        messagebox.showerror("Error", "Please connect to MT5 first")
        return

    if not COMMENT_PARSER_AVAILABLE:
        messagebox.showerror("Error", "Comment Parser module not available!")
        return

    client_name = self.client_info.get('client', '')

    self.log("="*70)
    self.log("■ PUSH HEDGE RESULTS BY COMMENT")
    self.log("="*70)
    self.log("Comment Format: {Account}_{Phase}{Number}")
```

```

self.log(" CH1-5 → Hedge Result 1-5 (Challenge)")
self.log(" FD0-4 → Hedge Result 1.1-5.1 (Funded)")
self.log(" DD1-4 → Additional Funded Hedge Results")
self.log(" FA/FA_DMMYY → Hedge Day 1-50 (Farming)")
self.log("Account Matching: First 4 + Last 4 characters")
self.log("=*70)
self.status_var.set("Processing MT5 deals...")

# Step 1: Get and aggregate deals from MT5
self.log("\n■ Step 1: Aggregating deals from MT5 by comment...")

result = self.pusher.get_deals_grouped_by_phase(days=365)

aggregated = result.get('aggregated', [])
unmatched = result.get('unmatched', [])
summary = result.get('summary', {})

if not aggregated:
    self.log("■■ No deals with valid comments found", "WARNING")
    messagebox.showwarning("Warning",
        "No deals found with valid comment format.\nExpected: {Account}_CH{1-5}, {Account}_FD{0-4}"
    )
    return

self.log(f" ✓ Found {len(aggregated)} trade groups")
self.log(f" ■■ {len(unmatched)} deals without valid comments")

# Show sample groups
for agg in aggregated[:5]:
    account = agg.get('account_number', '')
    phase = agg.get('phase_code', '')
    trade_num = agg.get('trade_number', '')
    profit = agg.get('net_profit', 0)
    sig = f"{account[:4]}...{account[-4:]}" if len(account) >= 8 else account
    self.log(f" • {sig}_{phase}{trade_num or ''}: ${profit:.2f}")

if len(aggregated) > 5:
    self.log(f" ... and {len(aggregated) - 5} more groups")

# Step 2: Get account info
self.log("\n■ Step 2: Getting MT5 account info...")
account = self.pusher.get_account_info() or {}
deals = self.pusher.get_deals(days=365)

if account:
    self.log(f" Balance: ${account.get('balance', 0):.2f}")
    self.log(f" Deposits: ${account.get('total_deposits', 0):.2f}")
    self.log(f" Withdrawals: ${account.get('total_withdrawals', 0):.2f}")

# Step 3: Send to dashboard (dashboard will do the matching)
self.log("\n■ Step 3: Sending to dashboard for matching...")
self.status_var.set("Pushing to dashboard...")

# Prepare aggregated data for dashboard
trade_data = []
for agg in aggregated:
    trade_data.append({
        "account_number": agg.get('account_number'),
        "phase_code": agg.get('phase_code'),
        "trade_number": agg.get('trade_number'),
        "farming_date": agg.get('farming_date'),
    })

```

```

        "net_profit": agg.get('net_profit'),
        "deal_count": agg.get('deal_count')
    })

payload = {
    "email": email,
    "account": account,
    "positions": self.pusher.get_positions(),
    "deals": deals,
    "aggregated_by_comment": trade_data, # Dashboard will match and update
    "comment_summary": {
        "total_groups": len(aggregated),
        "unmatched_deals": len(unmatched),
        "by_phase": summary.get('by_phase', {})
    },
    "statistics": {}, # Let server recalculate
    "dropdown_options": {}
}

try:
    response = _gzip_post(
        f"{dashboard_url}/api/client/push",
        payload,
        timeout=120
    )

    if response.status_code == 200:
        data = response.json()
        if data.get("status") == "success":
            # Show match log from dashboard
            hedge_log = data.get("hedge_match_log", [])
            hedge_updates = data.get("hedge_updates", 0)

            self.log(f"\n■ DASHBOARD MATCHING RESULTS:")
            for log_line in hedge_log:
                self.log(f"    {log_line}")

            self.log(f"\n■ HEDGE RESULTS PUSHED SUCCESSFULLY!")
            self.log(f"    {hedge_updates} hedge results updated")
            self.log("="*70)
            self.status_var.set(f"Pushed! {hedge_updates} updates")
            messagebox.showinfo("Success",
                                f"Pushed {len(trade_data)} trade groups.\n{hedge_updates} hedge results updated")
        else:
            self.log(f"■ Push failed: {data.get('message', 'Unknown error')}", "ERROR")
            self.status_var.set("Push failed")
    else:
        error_msg = f"HTTP {response.status_code}"
        try:
            error_msg = response.json().get("message", error_msg)
        except:
            pass
        self.log(f"■ Push failed: {error_msg}", "ERROR")
        self.status_var.set("Push failed")

except Exception as e:
    self.log(f"■ Push error: {e}", "ERROR")
    self.status_var.set("Push failed")

```

Method: TradeOpssAIApp._toggle_import_source

```
def _toggle_import_source(self)
```

Show/hide the correct input row based on selected import source.

Step by step (top-level body)

1. if self.import_source.get() == 'sheet': branches conditionally (with an **else/elif** arm).

Code (copy-paste safe)

```
def _toggle_import_source(self):
    """Show/hide the correct input row based on selected import source."""
    if self.import_source.get() == 'sheet':
        self.csv_input_frame.pack_forget()
        self.sheet_input_frame.pack(fill="x", pady=2)
        self.import_btn.configure(text="Import Sheet Data")
        self.import_hint.configure(text="Sheet must be publicly shared")
    else:
        self.sheet_input_frame.pack_forget()
        self.csv_input_frame.pack(fill="x", pady=2)
        self.import_btn.configure(text="Import CSV File")
        self.import_hint.configure(text="Use a CSV exported from the dashboard")
```

Method: TradeOpssAIApp._browse_csv

```
def _browse_csv(self)
```

Open file dialog to pick a CSV file.

Step by step (top-level body)

1. Assigns path = filedialog.askopenfilename(title='Select CSV File', filetypes=...
2. if path: branches conditionally.

Code (copy-paste safe)

```
def _browse_csv(self):
    """Open file dialog to pick a CSV file."""
    path = filedialog.askopenfilename(
        title="Select CSV File",
        filetypes=[("CSV files", "*.csv"), ("All files", "*.*")]
    )
    if path:
        self.csv_path_var.set(path)
```

Method: TradeOpssAIApp._do_import

```
def _do_import(self)
```

Route to the correct import method.

Step by step (top-level body)

1. if self.import_source.get() == 'sheet': branches conditionally (with an **else/elif** arm).

Code (copy-paste safe)

```
def _do_import(self):
```

```

        """Route to the correct import method."""
        if self.import_source.get() == 'sheet':
            self.migrate_from_sheet()
        else:
            self.import_from_csv()

```

Method: TradeOpssAIApp.import_from_csv

```
def import_from_csv(self)
```

Import evaluation data from a CSV file previously exported from the dashboard.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.

Step by step (top-level body)

1. Assigns email = self.client_email_entry.get().strip().
2. Assigns csv_path = self.csv_path_var.get().strip().
3. Assigns dashboard_url = self.url_entry.get().strip().rstrip('/').
4. **if not email**: branches conditionally.
5. **if not csv_path**: branches conditionally.
6. **if not os.path.isfile(csv_path)**: branches conditionally.
7. **if not self.client_info**: branches conditionally.
8. Assigns client_name = self.client_info.get('client', "").
9. **if not client_name**: branches conditionally.
10. **if not messagebox.askyesno('Confirm CSV Import', f"This will import CS...: branches conditionally.**
11. Calls self.log(...) for its side effect.
12. Calls self.status_var.set(...) for its side effect.
13. Calls self.import_btn.configure(...) for its side effect.
14. Calls self.root.update_idletasks(...) for its side effect.
15. ... and 2 more statement(s) in the body.

Code (copy-paste safe)

```

def import_from_csv(self):
    """Import evaluation data from a CSV file previously exported from the dashboard."""
    email = self.client_email_entry.get().strip()

```

```

csv_path = self.csv_path_var.get().strip()
dashboard_url = self.url_entry.get().strip().rstrip('/')

if not email:
    messagebox.showerror("Error", "Please enter your client email first")
    return

if not csv_path:
    messagebox.showerror("Error", "Please select a CSV file first")
    return

if not os.path.isfile(csv_path):
    messagebox.showerror("Error", f"File not found:\n{csv_path}")
    return

if not self.client_info:
    messagebox.showerror("Error",
        "Please lookup the client first by entering email and clicking 'Lookup'")
    return

client_name = self.client_info.get('client', '')
if not client_name:
    messagebox.showerror("Error", "Client lookup failed - no client name found")
    return

# Confirm before proceeding
if not messagebox.askyesno("Confirm CSV Import",
    f"This will import CSV data into {client_name}'s dashboard.\n\n"
    f"File: {os.path.basename(csv_path)}\n\n"
    "Existing rows matched by Account # will be updated.\n"
    "New rows will be appended.\n\nContinue?"):
    return

self.log(f"■ Importing CSV for {client_name}...")
self.status_var.set("Importing CSV...")
self.import_btn.configure(state="disabled")
self.root.update_idletasks()

def _do_import():
    try:
        with open(csv_path, 'rb') as f:
            response = requests.post(
                f"{dashboard_url}/api/client/import_csv_companion",
                data={"email": email},
                files={"file": (os.path.basename(csv_path), f, "text/csv")},
                timeout=120
            )

        if response.status_code != 200:
            error_msg = f"HTTP {response.status_code}"
            try:
                error_msg = response.json().get("message", error_msg)
            except:
                pass
            def _on_http_err(msg=error_msg):
                self.log(f"■ CSV import failed: {msg}", "ERROR")
                self.status_var.set("Import failed")
                self.import_btn.configure(state="normal")
                messagebox.showerror("Error", msg)
            self.root.after(0, _on_http_err)

```

```

        return

    data = response.json()
    if data.get("status") != "success":
        error_msg = data.get("message", "Import failed")
        def _on_api_err(msg=error_msg):
            self.log(f"■ {msg}", "ERROR")
            self.status_var.set("Import failed")
            self.import_btn.configure(state="normal")
            messagebox.showerror("Error", msg)
        self.root.after(0, _on_api_err)
        return

    updated = data.get('updated', 0)
    added = data.get('added', 0)
    total = data.get('total_rows', 0)
    def _on_success():
        self.log(f" ■ CSV import complete!")
        self.log(f"   {updated} rows updated, {added} rows added ({total} total evaluations)")
        self.status_var.set(f"Imported {updated + added} rows from CSV")
        self.import_btn.configure(state="normal")
        messagebox.showinfo("Success",
            f"CSV import complete!\n\n"
            f"• {updated} rows updated\n"
            f"• {added} rows added\n"
            f"• {total} total evaluations")
        self.lookup_client()
    self.root.after(0, _on_success)

except requests.exceptions.Timeout:
    def _on_timeout():
        self.log("■ Connection timeout during CSV import", "ERROR")
        self.status_var.set("Timeout")
        self.import_btn.configure(state="normal")
        messagebox.showerror("Timeout", "Connection timed out. Please try again.")
    self.root.after(0, _on_timeout)
except requests.exceptions.ConnectionError:
    def _on_conn_err():
        self.log("■ Could not connect to dashboard server", "ERROR")
        self.status_var.set("Connection failed")
        self.import_btn.configure(state="normal")
        messagebox.showerror("Error",
            "Could not connect to dashboard server. Check the URL and try again.")
    self.root.after(0, _on_conn_err)
except Exception as e:
    def _on_exc(err=str(e)):
        self.log(f"■ CSV import error: {err}", "ERROR")
        self.status_var.set("Import failed")
        self.import_btn.configure(state="normal")
        messagebox.showerror("Error", err)
    self.root.after(0, _on_exc)

threading.Thread(target=_do_import, daemon=True).start()

```

Method: TradeOpssAIApp.migrate_from_sheet

```
def migrate_from_sheet(self)
```

Migrate data from Google Sheets to the dashboard with verification.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.

Step by step (top-level body)

1. Assigns `email = self.client_email_entry.get().strip()`.
2. Assigns `sheet_url = self.sheet_url_entry.get().strip()`.
3. Assigns `dashboard_url = self.url_entry.get().strip().rstrip('/')`.
4. **if not email**: branches conditionally.
5. **if not sheet_url**: branches conditionally.
6. **if 'docs.google.com/spreadsheets' not in sheet_url**: branches conditionally.
7. Calls `self.log(...)` for its side effect.
8. Calls `self.status_var.set(...)` for its side effect.
9. Calls `self.root.update_idletasks(...)` for its side effect.
10. Defines a nested function `_do_migrate(...)`.
11. Calls `threading.Thread(target=_do_migrate, daemon=True).start(...)` for its side effect.

Code (copy-paste safe)

```
def migrate_from_sheet(self):
    """Migrate data from Google Sheets to the dashboard with verification."""
    email = self.client_email_entry.get().strip()
    sheet_url = self.sheet_url_entry.get().strip()
    dashboard_url = self.url_entry.get().strip().rstrip('/')

    if not email:
        messagebox.showerror("Error", "Please enter your client email first")
        return

    if not sheet_url:
        messagebox.showerror("Error", "Please enter the Google Sheet URL")
        return

    if 'docs.google.com/spreadsheets' not in sheet_url:
        messagebox.showerror("Error", "Please enter a valid Google Sheets URL")
        return

    self.log(f"Migrating sheet data to dashboard...")
    self.status_var.set("Migrating sheet data...")
    self.root.update_idletasks()

    def _do_migrate():
        try:
            response = requests.post(
```



```

        f"{dashboard_url}/api/client/migrate_sheet",
        json={"email": email, "sheet_url": sheet_url},
        headers={"Content-Type": "application/json"},
        timeout=180
    )

    if response.status_code != 200:
        error_msg = f"HTTP {response.status_code}"
        try:
            error_msg = response.json().get("message", error_msg)
        except:
            pass
        def _on_err(msg=error_msg):
            self.log(f"■ Migration failed: {msg}", "ERROR")
            self.status_var.set("Migration failed")
            messagebox.showerror("Error", msg)
        self.root.after(0, _on_err)
        return

    data = response.json()
    if data.get("status") != "success":
        error_msg = data.get("message", "Migration failed")
        def _on_api_err(msg=error_msg):
            self.log(f"■ {msg}", "ERROR")
            self.status_var.set("Migration failed")
            messagebox.showerror("Error", msg)
        self.root.after(0, _on_api_err)
        return

    records = data.get("records_imported", 0)
    def _on_success():
        self.log(f"    ■ Successfully imported {records} records")
        self.log(f"    Dashboard data fully replaced.")
        self.status_var.set(f"Imported {records} records")
        messagebox.showinfo("Success",
            f"Successfully imported {records} records.\nDashboard data has been updated.")
        self.lookup_client()
    self.root.after(0, _on_success)

except requests.exceptions.Timeout:
    def _on_timeout():
        self.log("■ Connection timeout - server is still processing the sheet", "ERROR")
        self.status_var.set("Timeout")
        messagebox.showerror("Timeout",
            "Connection timed out. The sheet may be too large or the server is busy. Please t
        self.root.after(0, _on_timeout)
except requests.exceptions.ConnectionError:
    def _on_conn_err():
        self.log("■ Could not connect to dashboard server", "ERROR")
        self.status_var.set("Connection failed")
        messagebox.showerror("Error",
            "Could not connect to dashboard server. Check the URL and try again.")
        self.root.after(0, _on_conn_err)
except Exception as e:
    def _on_exc(err=str(e)):
        self.log(f"■ Migration error: {err}", "ERROR")
        self.status_var.set("Migration failed")
        messagebox.showerror("Error", err)
    self.root.after(0, _on_exc)

threading.Thread(target=_do_migrate, daemon=True).start()

```

Method: TradeOpssAIApp.verify_stats

```
def verify_stats(self, local_stats, dashboard_stats)
```

Compare local stats with dashboard stats and return list of discrepancies.

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.

Step by step (top-level body)

1. Assigns discrepancies = [].
2. Assigns tolerance = 0.01.
3. Defines a nested function compare(...).
4. Assigns local_prof = local_stats.get('profitability_completed', {}).
5. Assigns dash_prof = dashboard_stats.get('profitability_completed', {}).
6. Calls compare(...) for its side effect.
7. Calls compare(...) for its side effect.
8. Calls compare(...) for its side effect.
9. Calls compare(...) for its side effect.
10. Calls compare(...) for its side effect.
11. Assigns local_cash = local_stats.get('cashflow_inprogress', {}).
12. Assigns dash_cash = dashboard_stats.get('cashflow_inprogress', {}).
13. Calls compare(...) for its side effect.
14. Calls compare(...) for its side effect.
15. ... and 15 more statement(s) in the body.

Code (copy-paste safe)

```
def verify_stats(self, local_stats, dashboard_stats):
    """Compare local stats with dashboard stats and return list of discrepancies."""
    discrepancies = []
    tolerance = 0.01 # Allow $0.01 difference for rounding

    # Helper to compare values
    def compare(name, local_val, dash_val):
        if isinstance(local_val, (int, float)) and isinstance(dash_val, (int, float)):
            if abs(local_val - dash_val) > tolerance:
                discrepancies.append(f"{name}: Local=${local_val:,.2f} vs Dashboard=${dash_val:,.2f}")
        elif local_val != dash_val:
            discrepancies.append(f"{name}: Local={local_val} vs Dashboard={dash_val}")
```

```

# Compare profitability_completed
local_prof = local_stats.get('profitability_completed', {})
dash_prof = dashboard_stats.get('profitability_completed', {})
compare("Prof.Challenge Fees", local_prof.get('challenge_fees', 0), dash_prof.get('challenge_fees', 0))
compare("Prof.Hedging Results", local_prof.get('hedging_results', 0), dash_prof.get('hedging_results', 0))
compare("Prof.Farming Results", local_prof.get('farming_results', 0), dash_prof.get('farming_results', 0))
compare("Prof.Payouts", local_prof.get('payouts', 0), dash_prof.get('payouts', 0))
compare("Prof.Net Profit", local_prof.get('net_profit', 0), dash_prof.get('net_profit', 0))

# Compare cashflow_inprogress
local_cash = local_stats.get('cashflow_inprogress', {})
dash_cash = dashboard_stats.get('cashflow_inprogress', {})
compare("Cash.Challenge Fees", local_cash.get('challenge_fees', 0), dash_cash.get('challenge_fees', 0))
compare("Cash.Hedging Results", local_cash.get('hedging_results', 0), dash_cash.get('hedging_results', 0))
compare("Cash.Farming Results", local_cash.get('farming_results', 0), dash_cash.get('farming_results', 0))
compare("Cash.Payouts", local_cash.get('payouts', 0), dash_cash.get('payouts', 0))
compare("Cash.Net Profit", local_cash.get('net_profit', 0), dash_cash.get('net_profit', 0))

# Compare eval_totals
local_et = local_stats.get('eval_totals', {})
dash_et = dashboard_stats.get('eval_totals', {})
compare("Eval.Total Running", local_et.get('total_running', 0), dash_et.get('total_running', 0))
compare("Eval.Total Passed", local_et.get('total_passed', 0), dash_et.get('total_passed', 0))
compare("Eval.Total Failed", local_et.get('total_failed', 0), dash_et.get('total_failed', 0))

# Compare funded_totals
local_ft = local_stats.get('funded_totals', {})
dash_ft = dashboard_stats.get('funded_totals', {})
compare("Funded.Not Started", local_ft.get('not_started', 0), dash_ft.get('not_started', 0))
compare("Funded.Ongoing", local_ft.get('ongoing', 0), dash_ft.get('ongoing', 0))
compare("Funded.Failed", local_ft.get('failed', 0), dash_ft.get('failed', 0))
compare("Funded.Completed", local_ft.get('completed', 0), dash_ft.get('completed', 0))

return discrepancies

```

Method: TradeOpssAIApp.toggle_auto_push

```
def toggle_auto_push(self)
    Toggle automatic data pushing.
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

Step by step (top-level body)

1. if self.auto_push_enabled: branches conditionally (with an **else/elif** arm).

Code (copy-paste safe)

```
def toggle_auto_push(self):
```

```

"""Toggle automatic data pushing."""
if self.auto_push_enabled:
    self.auto_push_enabled = False
    try:
        self.auto_btn.configure(text="Auto-Push")
    except Exception:
        pass
    try:
        self.auto_btn_live.configure(text="Auto-Push")
    except Exception:
        pass
    self.log("Auto-push stopped")
else:
    if not self.client_info:
        messagebox.showerror("Error", "Please lookup the client first")
        return

    # Initialize state
    self.last_deal_count = 0
    self.last_deal_ticket = 0

    self.auto_push_enabled = True
    try:
        self.auto_btn.configure(text="Stop Auto-Push")
    except Exception:
        pass
    try:
        self.auto_btn_live.configure(text="Stop Auto-Push")
    except Exception:
        pass

    self.log("Smart Auto-push started (Checking for new trades...)")
    self.auto_push_thread = threading.Thread(target=self.auto_push_loop, daemon=True)
    self.auto_push_thread.start()

```

Method: TradeOpssAIApp.check_and_push_update

```
def check_and_push_update(self)

    Check if new trades exist and push update if so.
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **if not self.auto_push_enabled**: branches conditionally.
2. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def check_and_push_update(self):
    """Check if new trades exist and push update if so."""
```

```

if not self.auto_push_enabled: return

try:
    if not self.pusher.connected:
        self.log("■■■ Auto-push: MT5 not connected – skipping check", "ERROR")
        return

    deals = self.pusher.get_deals(days=30)

    if not deals:
        self.log("■■■ Auto-push: MT5 returned 0 deals – connection issue?")
        return

    current_count = len(deals)
    last_deal = deals[-1]
    current_ticket = last_deal.get('ticket')

    if current_ticket is None:
        self.log("■■■ Auto-push: last deal has no ticket ID")

    # First check – initialize state and push once
    if self.last_deal_count == 0:
        self.last_deal_count = current_count
        self.last_deal_ticket = current_ticket
        self.log(f"■■ Auto-push scan: {current_count} deals, last ticket: {current_ticket}")
        self.push_data()
        return

    if current_count > self.last_deal_count or current_ticket != self.last_deal_ticket:
        self.log(
            f"■■ New trade! Deals: {self.last_deal_count}→{current_count} | Ticket: {current_ticket}"

            self.last_deal_count = current_count
            self.last_deal_ticket = current_ticket

            self.push_data()

except Exception as e:
    self.log(f"■■ Auto-push check error: {e}", "ERROR")

```

Method: TradeOpssAIApp.auto_push_loop

```
def auto_push_loop(self)
```

Background loop for smart auto-pushing.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns cycle = 0.
2. **while** self.auto_push_enabled: loops until the condition is false.

Code (copy-paste safe)

```
def auto_push_loop(self):
    """Background loop for smart auto-pushing."""
    cycle = 0
    while self.auto_push_enabled:
        try:
            self.root.after(0, self.check_and_push_update)
        except Exception as e:
            # Thread-safe: can't call self.log from background thread directly
            try:
                self.root.after(0, lambda err=str(e): self.log(f"■ Auto-push loop error: {err}", "ERR"))
            except Exception:
                pass

        # Check frequently (every 10s)
        for _ in range(10):
            if not self.auto_push_enabled:
                break
            time.sleep(1)

        cycle += 1
        # Heartbeat every ~60s (6 cycles of 10s)
        if cycle % 6 == 0 and self.auto_push_enabled:
            try:
                self.root.after(0, lambda c=cycle: self.log(
                    f"■ Auto-push heartbeat (cycle {c}) - watching for new trades"))
            except Exception:
                pass
```

Method: TradeOpssAIApp._build_trading_engine_ui

```
def _build_trading_engine_ui(self, parent)
```

Build the Trading Engine section with CTK styled cards.

Why these Python concepts were used

- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns mt5_card = self._section_card(parent, 'MT5 CONNECTION', '■').
2. Calls mt5_card.pack(...) for its side effect.
3. Assigns mt5_row = ctk.CTkFrame(mt5_card, fg_color='transparent') if CTK_AVA....
4. Calls mt5_row.pack(...) for its side effect.
5. **if** CTK_AVAILABLE: branches conditionally.
6. Assigns self.mt5_login = self._ctk_entry(mt5_row, width=120).
7. Calls self.mt5_login.pack(...) for its side effect.
8. **if** CTK_AVAILABLE: branches conditionally.

9. Assigns `self.mt5_password = self._ctk_entry(mt5_row, width=120, show='*')`.
10. Calls `self.mt5_password.pack(...)` for its side effect.
11. `if CTK_AVAILABLE:` branches conditionally.
12. Assigns `self.mt5_server = self._ctk_entry(mt5_row, width=160)`.
13. Calls `self.mt5_server.pack(...)` for its side effect.
14. Assigns `mt5_btn_row = ctk.CTkFrame(mt5_card, fg_color='transparent')` if `CTK_AVA...`
15. ... and 35 more statement(s) in the body.

Code (copy-paste safe)

```
def _build_trading_engine_ui(self, parent):
    """Build the Trading Engine section with CTK styled cards."""

    # ■■ MT5 Connection Card ■■
    mt5_card = self._section_card(parent, "MT5 CONNECTION", "■")
    mt5_card.pack(fill="x", padx=4, pady=(4, 2))

    mt5_row = ctk.CTkFrame(mt5_card, fg_color="transparent") if CTK_AVAILABLE else \
        tk.Frame(mt5_card, bg="#161B22")
    mt5_row.pack(fill="x", padx=10, pady=(2, 2))

    if CTK_AVAILABLE:
        ctk.CTkLabel(mt5_row, text="Login:", font=("Segoe UI", 11),
                     text_color=self.C_TEXT_DIM).pack(side="left", padx=(0, 4))
        self.mt5_login = self._ctk_entry(mt5_row, width=120)
        self.mt5_login.pack(side="left", padx=(0, 8))

    if CTK_AVAILABLE:
        ctk.CTkLabel(mt5_row, text="Pass:", font=("Segoe UI", 11),
                     text_color=self.C_TEXT_DIM).pack(side="left", padx=(0, 4))
        self.mt5_password = self._ctk_entry(mt5_row, width=120, show="*")
        self.mt5_password.pack(side="left", padx=(0, 8))

    if CTK_AVAILABLE:
        ctk.CTkLabel(mt5_row, text="Server:", font=("Segoe UI", 11),
                     text_color=self.C_TEXT_DIM).pack(side="left", padx=(0, 4))
        self.mt5_server = self._ctk_entry(mt5_row, width=160)
        self.mt5_server.pack(side="left", padx=(0, 8))

    mt5_btn_row = ctk.CTkFrame(mt5_card, fg_color="transparent") if CTK_AVAILABLE else \
        tk.Frame(mt5_card, bg="#161B22")
    mt5_btn_row.pack(fill="x", padx=10, pady=(0, 4))
    self.mt5_btn = self._ctk_button(mt5_btn_row, text="Connect MT5",
                                    command=self.toggle_mt5_connection,
                                    fg="#24292F", hover="#000000", width=140)
    self.mt5_btn.pack(side="left")

    if not TRADING_ENGINE_AVAILABLE:
        if CTK_AVAILABLE:
            ctk.CTkLabel(mt5_card, text="Trading engine modules not loaded – broker trading unavailable",
                         font=("Segoe UI", 9, "italic"), text_color=self.C_GOLD,
                         wraplength=500).pack(padx=12, pady=(0, 8))

        return

    # ■■ Broker Connections Card ■■
    broker_card = self._section_card(parent, "BROKER CONNECTIONS", "■")
```

```

broker_card.pack(fill="x", padx=4, pady=2)

# Global settings row: Platform + Mode + Connect All
bk_global = ctk.CTkFrame(broker_card, fg_color="transparent") if CTK_AVAILABLE else \
    tk.Frame(broker_card, bg="#161B22")
bk_global.pack(fill="x", padx=10, pady=(2, 2))

if CTK_AVAILABLE:
    ctk.CTkLabel(bk_global, text="Platform:", font=("Segoe UI", 11),
        text_color=self.C_TEXT_DIM).pack(side="left", padx=(0, 4))
    self.broker_var = tk.StringVar(value="Tradovate")
    platforms = ["Tradovate", "TopStepX"]
    if CTK_AVAILABLE:
        ctk.CTkComboBox(bk_global, variable=self.broker_var, values=platforms,
            state="readonly", width=120, height=30,
            fg_color=self.C_BG_THIRD, border_color=self.C_BORDER,
            button_color=self.C_ACCENT, text_color=self.C_TEXT,
            dropdown_fg_color=self.C_BG_SEC).pack(side="left", padx=(0, 12))
    else:
        ttk.Combobox(bk_global, textvariable=self.broker_var, values=platforms,
            state='readonly', width=14).pack(side="left", padx=(0, 8))

if CTK_AVAILABLE:
    ctk.CTkLabel(bk_global, text="Mode:", font=("Segoe UI", 11),
        text_color=self.C_TEXT_DIM).pack(side="left", padx=(0, 4))
    self.trading_mode_var = tk.StringVar(value="Simulation")
    if CTK_AVAILABLE:
        ctk.CTkComboBox(bk_global, variable=self.trading_mode_var,
            values=["Simulation", "Live"], state="readonly",
            width=120, height=30, fg_color=self.C_BG_THIRD,
            border_color=self.C_BORDER, button_color=self.C_ACCENT,
            text_color=self.C_TEXT,
            dropdown_fg_color=self.C_BG_SEC).pack(side="left", padx=(0, 12))
    else:
        ttk.Combobox(bk_global, textvariable=self.trading_mode_var,
            values=["Simulation", "Live"], state='readonly', width=14).pack(side="left", padx=
            10))

self.broker_connect_all_btn = self._ctk_button(bk_global, text="Connect All",
        command=self._connect_all_brokers,
        fg="#24292F", hover="#000000", width=100)
self.broker_connect_all_btn.pack(side="left", padx=(0, 6))

self.broker_status_var = tk.StringVar(value="")
if CTK_AVAILABLE:
    ctk.CTkLabel(bk_global, textvariable=self.broker_status_var,
        font=("Segoe UI", 9), text_color=self.C_TEXT_DIM).pack(side="left")
else:
    ttk.Label(bk_global, textvariable=self.broker_status_var).pack(side="left")

# Dynamic broker rows container (populated when trades load)
self._broker_rows_frame = ctk.CTkFrame(broker_card, fg_color="transparent") if CTK_AVAILABLE else \
    tk.Frame(broker_card, bg="#161B22")
self._broker_rows_frame.pack(fill="x", padx=6, pady=(0, 4))

if CTK_AVAILABLE:
    ctk.CTkLabel(self._broker_rows_frame,
        text="Load trades to populate prop firm connections",
        font=("Segoe UI", 9, "italic"), text_color="#4A5568").pack(pady=4)

# ■■ Hedge Mode / Direction (inline) ■■

```



```

opts_row = ctk.CTkFrame(parent, fg_color="transparent") if CTK_AVAILABLE else \
    tk.Frame(parent)
opts_row.pack(fill="x", padx=10, pady=(2, 2))

self.hedge_mode_var = tk.StringVar(value="Hedging")
if CTK_AVAILABLE:
    ctk.CTkRadioButton(opts_row, text="Hedging (Broker+MT5)", variable=self.hedge_mode_var,
                        value="Hedging", font=("Segoe UI", 11), text_color=self.C_TEXT,
                        fg_color=self.C_ACCENT, border_color=self.C_BORDER).pack(side="left", padx=
                        12))
    ctk.CTkRadioButton(opts_row, text="Broker Only", variable=self.hedge_mode_var,
                        value="BrokerOnly", font=("Segoe UI", 11), text_color=self.C_TEXT,
                        fg_color=self.C_ACCENT, border_color=self.C_BORDER).pack(side="left", padx=
                        20))
else:
    ttk.Radiobutton(opts_row, text="Hedging (Broker+MT5)", variable=self.hedge_mode_var,
                    value="Hedging").pack(side="left", padx=(0, 8))
    ttk.Radiobutton(opts_row, text="Broker Only", variable=self.hedge_mode_var,
                    value="BrokerOnly").pack(side="left", padx=(0, 16))

self.direction_var = tk.StringVar(value="All Trades")
if CTK_AVAILABLE:
    ctk.CTkLabel(opts_row, text="Direction:", font=("Segoe UI", 11),
                 text_color=self.C_TEXT_DIM).pack(side="left", padx=(0, 4))
    ctk.CTkComboBox(opts_row, variable=self.direction_var,
                    values=["All Trades", "Buy Only", "Sell Only"],
                    state="readonly", width=130, height=32,
                    fg_color=self.C_BG_THIRD, border_color=self.C_BORDER,
                    button_color=self.C_ACCENT, text_color=self.C_TEXT,
                    dropdown_fg_color=self.C_BG_SEC).pack(side="left")
else:
    ttk.Label(opts_row, text="Direction:").pack(side="left", padx=(0, 4))
    ttk.Combobox(opts_row, textvariable=self.direction_var,
                 values=["All Trades", "Buy Only", "Sell Only"],
                 state='readonly', width=12).pack(side="left")

# ■■■ Hidden vars for backward compat with save/load config ■■■
self.prop_firm_var = tk.StringVar(value="MFFU_Flex")
self.phase_var = tk.StringVar(value="challenge_trade1")
self.acct_size_var = tk.StringVar(value="$50,000")
self.strategy_var = tk.StringVar(value="Random Entries")

# Dummy combos (hidden, for _on_prop_firm_change / _update_account_sizes compat)
self.prop_firm_combo = ttk.Combobox(parent)
self.phase_combo = ttk.Combobox(parent)
self.acct_size_combo = ttk.Combobox(parent)

```

Method: TradeOpssAIApp._resolve_starting_balance

```
def _resolve_starting_balance(self, ev, current_phase, acct_size)
```

Return the firm-correct starting balance for profit math. For Funded / Payout phases on firms whose funded balance starts at \$0 (MFFU family), this returns 0 instead of the challenge size. All other firms / phases fall back to the challenge size parsed from acct_size.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't

have to handle it.

- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns `start_str = str(acct_size or "").replace('$', "").replace(',', ' ').strip().lower()`
2. `if start_str.endswith('k')`: branches conditionally (with an **else/elif** arm).
3. Assigns `funded_phases = ('Funded', 'Payout 1', 'Payout 2', 'Payout 3', 'Payout 4')`
4. `if current_phase not in funded_phases`: branches conditionally.
5. Assigns `firm_raw = (ev or {}).get('Prop Firm', '')` if `isinstance(ev, dict)` else...
6. Assigns `canonical = self._FIRM_MAP.get(firm_raw, firm_raw)`.
7. Assigns `mode = self._FUNDED_START_BALANCE_MODE.get(canonical, 'ACCOUNT_SIZE')`
8. `if mode == 'ZERO'`: branches conditionally.
9. **return** `default_start`.

Code (copy-paste safe)

```
def _resolve_starting_balance(self, ev, current_phase, acct_size):
    """Return the firm-correct starting balance for profit math.

    For Funded / Payout phases on firms whose funded balance starts at $0
    (MFFU family), this returns 0 instead of the challenge size. All other
    firms / phases fall back to the challenge size parsed from acct_size.
    """
    # Parse acct_size as the default
    start_str = str(acct_size or '').replace("$", "").replace(",", "").strip().lower()
    if start_str.endswith("k"):
        try:
            default_start = float(start_str[:-1]) * 1000
        except ValueError:
            default_start = 50000.0
    elif start_str.replace(".", "").isdigit():
        default_start = float(start_str)
    else:
        default_start = 50000.0

    # Only Funded / Payout / Double Dip phases differ from challenge size
    funded_phases = ("Funded", "Payout 1", "Payout 2", "Payout 3", "Payout 4", "Double Dip")
    if current_phase not in funded_phases:
        return default_start

    firm_raw = (ev or {}).get("Prop Firm", "") if isinstance(ev, dict) else ""
    canonical = self._FIRM_MAP.get(firm_raw, firm_raw)
    mode = self._FUNDED_START_BALANCE_MODE.get(canonical, "ACCOUNT_SIZE")
    if mode == "ZERO":
        return 0.0
    return default_start
```

Method: TradeOpssAIApp._detect_eval_phase

```
def _detect_eval_phase(self, ev)
```

Determine current phase display name and blueprint key for an evaluation.

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `challenge_status = (ev.get('Status P1', '') or '').strip().lower()`.
2. Assigns `funded_status = (ev.get('Status', '') or '').strip().lower()`.
3. Assigns `has_funded_acct = bool((ev.get('Account #.1', '') or '').strip())`.
4. Assigns `has_farming_marker = bool((ev.get('Prop Day 1', '') or '').strip())`.
5. Assigns `has_hedge_day_data = False`.
6. **if** `has_farming_marker`: branches conditionally.
7. **if** `has_farming_marker` and `has_hedge_day_data`: branches conditionally (with an **else/elif** arm).

Code (copy-paste safe)

```
def _detect_eval_phase(self, ev):
    """Determine current phase display name and blueprint key for an evaluation."""
    challenge_status = (ev.get("Status P1", "") or "").strip().lower()
    funded_status = (ev.get("Status", "") or "").strip().lower()
    has_funded_acct = bool((ev.get("Account #.1", "") or "").strip())

    # Check if farming data exists – must have BOTH the farming marker
    # AND actual Hedge Day cell data for THIS account (not just sheet columns)
    has_farming_marker = bool((ev.get("Prop Day 1", "") or "").strip())
    has_hedge_day_data = False
    if has_farming_marker:
        for i in range(1, 35):
            val = (ev.get(f"Hedge Day {i}", "") or "").strip()
            if val and val not in ("-", "-"):
                has_hedge_day_data = True
                break

    if has_farming_marker and has_hedge_day_data:
        return "Farming", "farming"
    elif has_funded_acct and funded_status not in self._FAILED_STATUSES:
        return "Funded", "funded_trade1"
    else:
        return "Challenge", "challenge_trade1"
```

Method: TradeOpssAIApp._resolve_phase_key_from_day

```
def _resolve_phase_key_from_day(self, ev, firm_code, current_phase)
```

Use the day placeholder cell index to determine the correct blueprint key. The day placeholder position (0-based) maps directly to the trade order in `_PHASE_TRADE_ORDER`. E.g. cell index 2 → third trade in the sequence. Scans all field sets if the detected phase has no day placeholder, and corrects the phase accordingly. Returns (resolved_phase_key, day_index, day_name) or (None, None, None) if no day placeholder is found.

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns (day_idx, day_name, is_today, matched_phase) = self._find_tradeable_day_cell(ev, current_phase).
2. **if** day_idx is None: branches conditionally.
3. Assigns effective_phase = matched_phase if matched_phase else current_phase.
4. **if** matched_phase and matched_phase != current_phase: branches conditionally.
5. **if** not self.prop_firm_mgr: branches conditionally.
6. Assigns phase_map = {'Challenge': 'Challenge', 'Funded': 'Funded', 'Farming':....
7. Assigns phase_group = phase_map.get(effective_phase, effective_phase).
8. Assigns firm_orders = self.prop_firm_mgr._PHASE_TRADE_ORDER.get(firm_code, {}).
9. Assigns trade_keys = firm_orders.get(phase_group, []).
10. **if** not trade_keys: branches conditionally.
11. Assigns key_idx = min(day_idx, len(trade_keys) - 1).
12. Assigns resolved_key = trade_keys[key_idx].
13. **return** (resolved_key, day_idx, day_name).

Code (copy-paste safe)

```
def _resolve_phase_key_from_day(self, ev, firm_code, current_phase):
    """Use the day placeholder cell index to determine the correct blueprint key.

    The day placeholder position (0-based) maps directly to the trade order
    in _PHASE_TRADE_ORDER. E.g. cell index 2 → third trade in the sequence.

    Scans all field sets if the detected phase has no day placeholder,
    and corrects the phase accordingly.

    Returns (resolved_phase_key, day_index, day_name) or (None, None, None)
    if no day placeholder is found.
    """
    day_idx, day_name, is_today, matched_phase = self._find_tradeable_day_cell(ev, current_phase)
    if day_idx is None:
        return None, None, None

    # If day was found in a different phase's fields, correct the phase
    effective_phase = matched_phase if matched_phase else current_phase
    if matched_phase and matched_phase != current_phase:
        self.log(f"■ Phase correction: detected '{current_phase}' but day "
                f"placeholder found in '{matched_phase}' fields")

    if not self.prop_firm_mgr:
        return None, day_idx, day_name
```

```

# Normalize phase for _PHASE_TRADE_ORDER lookup
phase_map = {"Challenge": "Challenge", "Funded": "Funded",
             "Farming": "Farming", "Double Dip": "Double Dip",
             "Payout 1": "Funded", "Payout 2": "Funded",
             "Payout 3": "Funded", "Payout 4": "Funded"}
phase_group = phase_map.get(effective_phase, effective_phase)

firm_orders = self.prop_firm_mgr._PHASE_TRADE_ORDER.get(firm_code, {})
trade_keys = firm_orders.get(phase_group, [])

if not trade_keys:
    return None, day_idx, day_name

# Clamp day index to available trade keys
key_idx = min(day_idx, len(trade_keys) - 1)
resolved_key = trade_keys[key_idx]
return resolved_key, day_idx, day_name

```

Method: TradeOpssAIApp._parse_day_token

```

@classmethod
def _parse_day_token(cls, value)

    Extract a weekday number (0=Mon..6=Sun) from a cell value. Lenient parsing that handles surrounding
    punctuation, extra text, and embedded day names (e.g. ``"MONDAY ✓"`, ``"Mon-04/27"`, ``"Mon -
    waiting"``). Returns the weekday number or ``None``.

```

Why these Python concepts were used

- **@classmethod** — Marked **@classmethod** — receives the class itself as the first argument. The standard pattern for alternate constructors that build an instance from a different input shape (e.g., from a dict, a CSV row, or an MT5 order tuple).
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

Step by step (top-level body)

1. **if** value is None: branches conditionally.
2. **try** block with 1 **except** clause.
3. **if not s**: branches conditionally.
4. **if s in cls._DAY_ABBREVS**: branches conditionally.
5. Imports **re** (lazy import inside the function).
6. **for tok in _re.split('[\s\\-/,:;\.]+', s)**: iterates.
7. **return None**.

Code (copy-paste safe)

```

def _parse_day_token(cls, value):
    """Extract a weekday number (0=Mon..6=Sun) from a cell value.

    Lenient parsing that handles surrounding punctuation, extra text,
    and embedded day names (e.g. ``"MONDAY ✓"`, ``"Mon-04/27"`,
    ``"Mon - waiting"``). Returns the weekday number or ``None``.

```

```

"""
if value is None:
    return None
try:
    s = str(value).strip().lower()
except Exception:
    return None
if not s:
    return None
# Whole-string match first
if s in cls._DAY_ABBREVS:
    return cls._DAY_ABBREVS[s]
# Tokenise on common separators (whitespace, hyphen, slash, comma, colon)
import re as _re
for tok in _re.split(r"[\s\-/,:;\.]+", s):
    tok = tok.strip()
    if tok in cls._DAY_ABBREVS:
        return cls._DAY_ABBREVS[tok]
return None

```

Method: TradeOpssAIApp._get_phase_fields

```
def _get_phase_fields(self, current_phase)
```

Return the ordered list of eval field names for a phase.

Why these Python concepts were used

- **list comprehension** — Builds a list in a single expression ([f(x) for x in xs]). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **if** current_phase == 'Challenge': branches conditionally (with an **else/elif** arm).
2. **return []**.

Code (copy-paste safe)

```

def _get_phase_fields(self, current_phase):
    """Return the ordered list of eval field names for a phase."""
    if current_phase == "Challenge":
        return [f"Hedge Result {i}" for i in range(1, 6)]
    elif current_phase in ("Funded", "Payout 1", "Payout 2", "Payout 3", "Payout 4"):
        return [f"Hedge Result {i}.1" for i in range(1, 8)]
    elif current_phase == "Double Dip":
        return [f"Hedge Result {i}.1" for i in range(1, 8)]
    elif current_phase == "Farming":
        return [f"Hedge Day {i}" for i in range(1, 35)]
    return []

```

Method: TradeOpssAIApp._count_completed_trades

```
def _count_completed_trades(self, ev, current_phase)
```

*Count how many trading day cells are filled for the current phase. A filled cell = a trade already taken.
Empty/blank = not yet traded. Day-name placeholders (MON, TUE, etc.) are NOT counted as completed.*

Returns (completed_count, total_fields, next_empty_index). next_empty_index is 0-based: the position of the first empty cell.

Why these Python concepts were used

- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns fields = self._get_phase_fields(current_phase).
2. Assigns completed = 0.
3. Assigns next_empty = None.
4. **for** (i, f) in enumerate(fields): iterates.
5. **if** next_empty is None: branches conditionally.
6. **return** (completed, len(fields), next_empty).

Code (copy-paste safe)

```
def _count_completed_trades(self, ev, current_phase):
    """Count how many trading day cells are filled for the current phase.

    A filled cell = a trade already taken. Empty/blank = not yet traded.
    Day-name placeholders (MON, TUE, etc.) are NOT counted as completed.
    Returns (completed_count, total_fields, next_empty_index).
    next_empty_index is 0-based: the position of the first empty cell.
    """
    fields = self._get_phase_fields(current_phase)

    completed = 0
    next_empty = None
    for i, f in enumerate(fields):
        val = ev.get(f, None)
        val_str = str(val).strip() if val is not None else ""
        if val_str in ("", "-", "-"):
            # Empty cell
            if next_empty is None:
                next_empty = i
        elif self._parse_day_token(val_str) is not None:
            # Day placeholder – not yet traded
            if next_empty is None:
                next_empty = i
        else:
            # Has a real value (dollar amount, etc.) – completed trade
            completed += 1

    if next_empty is None:
        next_empty = len(fields) # All filled

    return completed, len(fields), next_empty
```

Method: TradeOpssAIApp._find_tradeable_day_cell

```
def _find_tradeable_day_cell(self, ev, current_phase)
```

Find the cell that should be traded today based on day placeholders. A cell is tradeable today if it contains today's day name OR a previous weekday that hasn't been filled with a result yet (i.e. the trader missed entering a day and it's still pending). A cell whose day is AFTER today = already been prepared for the next trading day and should NOT be traded now. Scans the detected phase's fields first. If nothing found, falls back to scanning ALL other phase field sets so a misdetected phase doesn't block trading. Returns (stage_index, day_name, is_today, matched_phase) or (None, None, False, None). stage_index is 0-based position in the field list. matched_phase is the phase name whose fields contained the match.

Step by step (top-level body)

1. Imports datetime (lazy import inside the function).
2. Assigns today_weekday = datetime.date.today().weekday().
3. Assigns primary_fields = self._get_phase_fields(current_phase).
4. Assigns search_order = [(current_phase, primary_fields)].
5. **for** (phase_name, field_list) in self._ALL_PHASE_FIELD_SETS: iterates.
6. **for** (phase_name, fields) in search_order: iterates.
7. **return** (None, None, False, None).

Code (copy-paste safe)

```
def _find_tradeable_day_cell(self, ev, current_phase):
    """Find the cell that should be traded today based on day placeholders.

    A cell is tradeable today if it contains today's day name OR a
    previous weekday that hasn't been filled with a result yet (i.e. the
    trader missed entering a day and it's still pending).

    A cell whose day is AFTER today = already been prepared for the next
    trading day and should NOT be traded now.

    Scans the detected phase's fields first. If nothing found, falls back
    to scanning ALL other phase field sets so a misdetected phase doesn't
    block trading.

    Returns (stage_index, day_name, is_today, matched_phase) or
    (None, None, False, None).
    stage_index is 0-based position in the field list.
    matched_phase is the phase name whose fields contained the match.
    """
    import datetime
    today_weekday = datetime.date.today().weekday() # 0=Mon .. 4=Fri

    # Build ordered list: detected phase first, then all others as fallback
    primary_fields = self._get_phase_fields(current_phase)
    search_order = [(current_phase, primary_fields)]
    for phase_name, field_list in self._ALL_PHASE_FIELD_SETS:
        if field_list != primary_fields:
            search_order.append((phase_name, field_list))

    for phase_name, fields in search_order:
        best_idx = None
        best_day_name = None
        best_day_num = -1
        best_is_today = False
```



```

for i, f in enumerate(fields):
    val = ev.get(f, None)
    if val is None:
        continue
    day_num = self._parse_day_token(val)
    if day_num is None:
        continue # Not a day placeholder (result value or empty)

    if day_num == today_weekday:
        # Exact match – this cell is for today
        return i, str(val).strip().upper(), True, phase_name
    elif day_num < today_weekday:
        # Previous day still has a day placeholder (not yet traded).
        # Pick the most-recent missed day.
        if best_idx is None or day_num > best_day_num:
            best_idx = i
            best_day_name = str(val).strip().upper()
            best_day_num = day_num
            best_is_today = False
        # day_num > today_weekday → future day, skip

    if best_idx is not None:
        return best_idx, best_day_name, best_is_today, phase_name

return None, None, False, None

```

Method: TradeOpssAIApp._validate_stage_consistency

```
def _validate_stage_consistency(self, prediction, ev, current_phase, acct_num)
```

Validate whether a trade should proceed using day placeholders as the primary gate, with balance and cell count as advisory warnings. Day placeholder rule (GATE — controls trade/no-trade): - Cell with today's day (or a missed previous day) → TRADE - No matching day placeholder found → NO TRADE - Only future day placeholders → NO TRADE (already prepared) Balance + cell count (ADVISORY — warnings only, never block): - If balance stage or cell count disagree, log warnings - But trade still proceeds if day placeholder confirms Returns (should_trade: bool, message: str).

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Imports datetime (lazy import inside the function).
2. Assigns `fields = self._get_phase_fields(current_phase)`.
3. Assigns `stages = prediction.get('stages', [])` if prediction else `[]`.
4. Assigns `(day_idx, day_name, is_today, matched_phase) = self._find_tradeable_day_cell(ev, current_phase)`.
5. `if day_idx is None:` branches conditionally.
6. Assigns `day_stage_idx = min(day_idx, len(stages) - 1)` if stages else `day_idx`.

7. Assigns `day_label = 'today' if is_today else f'missed {day_name}'`.
8. Assigns `msgs = []`.
9. Calls `msgs.append(...)` for its side effect.
10. `if` prediction and stages: branches conditionally.
11. Assigns `(completed, total, _) = self._count_completed_trades(ev, current_phase)`.
12. `if` stages: branches conditionally.
13. `return (True, '\n'.join(msgs))`.

Code (copy-paste safe)

```
def _validate_stage_consistency(self, prediction, ev, current_phase, acct_num):
    """Validate whether a trade should proceed using day placeholders as
    the primary gate, with balance and cell count as advisory warnings.

    Day placeholder rule (GATE – controls trade/no-trade):
    - Cell with today's day (or a missed previous day) → TRADE
    - No matching day placeholder found → NO TRADE
    - Only future day placeholders → NO TRADE (already prepared)

    Balance + cell count (ADVISORY – warnings only, never block):
    - If balance stage or cell count disagree, log warnings
    - But trade still proceeds if day placeholder confirms

    Returns (should_trade: bool, message: str).
    """
    import datetime
    fields = self._get_phase_fields(current_phase)
    stages = prediction.get("stages", []) if prediction else []

    # ■■■ Primary gate: day placeholder ■■■
    day_idx, day_name, is_today, matched_phase = self._find_tradeable_day_cell(ev, current_phase)

    if day_idx is None:
        # No day placeholder for today or any missed day → don't trade
        # Check if there's a future day placeholder across ALL field sets
        future_days = []
        today_wd = datetime.date.today().weekday()
        all_fields = []
        for _, flist in self._ALL_PHASE_FIELD_SETS:
            all_fields.extend(flist)
        for i, f in enumerate(all_fields):
            val = ev.get(f, None)
            if val is None:
                continue
            dn = self._parse_day_token(val)
            if dn is not None and dn > today_wd:
                future_days.append((i, str(val).strip().upper()))

        if future_days:
            next_day = future_days[0]
            msg = (f"■■■ {acct_num}: No trade today – "
                  f"next day placeholder is {next_day[1]} in cell {next_day[0] + 1}. "
                  f"Already prepared for next trading day.")
        else:
            msg = (f"■ {acct_num}: No day placeholder found for today – "
                  f"trader needs to enter a day name (MON/TUE/etc.) ")
```

```

        f"in the next cell to enable trading.")
    return False, msg

# Day placeholder found – trade is confirmed
day_stage_idx = min(day_idx, len(stages) - 1) if stages else day_idx
day_label = "today" if is_today else f"missed {day_name}"

msgs = []
msgs.append(
    f"■ {acct_num}: Trade confirmed via {day_name} placeholder "
    f"({'today' if is_today else 'pending from earlier'}) "
    f"in cell {day_idx + 1}")

# ■■ Advisory: balance check ■■
if prediction and stages:
    balance_stage_idx = 0
    current_key = prediction.get("current_phase_key", "")
    for i, (_, key, _, _) in enumerate(stages):
        if key == current_key:
            balance_stage_idx = i
            break

    if balance_stage_idx != day_stage_idx:
        msgs.append(
            f"■■ Balance advisory: balance suggests stage "
            f"{balance_stage_idx + 1} ({prediction['current_stage']}), "
            f"but day placeholder is in cell {day_idx + 1} (stage {day_stage_idx + 1})")

# ■■ Advisory: cell count check ■■
completed, total, _ = self._count_completed_trades(ev, current_phase)
if stages:
    expected_completed = day_idx # Cells before the day placeholder should be filled
    if completed != expected_completed:
        msgs.append(
            f"■■ Cell count advisory: {completed} cells filled, "
            f"expected {expected_completed} before cell {day_idx + 1}")

return True, "\n".join(msgs)

```

Method: TradeOpssAIApp._get_current_phase_profit

```
def _get_current_phase_profit(self, ev, current_phase, broker_account=None, acct_size=None)
```

Get the current P/L for the active phase. Priority: 1. Live broker equity (equity - starting balance) — most accurate 2. Eval hedge result fields — fallback when broker not available

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **list comprehension** — Builds a list in a single expression ([f(x) for x in xs]). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. `if broker_account` and `acct_size`: branches conditionally.
2. Assigns `total = 0.0`.
3. Assigns `fields = []`.
4. `if current_phase == 'Challenge'`: branches conditionally (with an **else/elif** arm).
5. `for f in fields`: iterates.
6. **return** `total`.

Code (copy-paste safe)

```
def _get_current_phase_profit(self, ev, current_phase, broker_account=None, acct_size=None):
    """Get the current P/L for the active phase.

    Priority:
    1. Live broker equity (equity - starting balance) – most accurate
    2. Eval hedge result fields – fallback when broker not available
    """
    # ■■ 1. Try live broker equity ■■
    if broker_account and acct_size:
        try:
            stats = broker_account.get_account_stats()
            balance_str = stats.get("Balance", "N/A")
            if balance_str and balance_str != "N/A":
                cleaned = balance_str.replace("$", "").replace(",", "").strip()
                equity = float(cleaned)
                # Firm-aware starting balance. Critical for MFFU funded
                # accounts which start at $0 – using $50K here would make
                # the TP adjuster think we are -$50K down and inflate TP.
                starting = self._resolve_starting_balance(ev, current_phase, acct_size)
                live_profit = equity - starting
                if abs(live_profit) > 0.01:
                    return live_profit
        except Exception:
            pass

    # ■■ 2. Fallback: sum eval hedge result fields ■■
    total = 0.0
    fields = []
    if current_phase == "Challenge":
        fields = [f"Hedge Result {i}" for i in range(1, 6)]
    elif current_phase in ("Funded", "Payout 1", "Payout 2", "Payout 3", "Payout 4"):
        fields = [f"Hedge Result {i}.1" for i in range(1, 8)]
    elif current_phase == "Farming":
        fields = [f"Hedge Day {i}" for i in range(1, 35)]
    elif current_phase == "Double Dip":
        fields = [f"Hedge Result {i}.1" for i in range(1, 8)]

    for f in fields:
        val = ev.get(f, None)
        if val is None or val == "" or val == "-":
            continue
        try:
            cleaned = str(val).replace("$", "").replace(",", "").strip()
            total += float(cleaned)
        except (ValueError, TypeError):
            continue
    return total
```

Method: TradeOpssAIApp._get_next_phase

```
def _get_next_phase(self, firm_code, current_display)
```

Get the next phase display name from trading_phases progression.

Step by step (top-level body)

1. **if** not self.prop_firm_mgr: branches conditionally.
2. Assigns phases = self.prop_firm_mgr.get_prop_firm_trading_phases(firm_code).
3. **if** not phases: branches conditionally.
4. Assigns current_idx = -1.
5. **for** (i, ph) in enumerate(phases): iterates.
6. **if** current_idx < 0: branches conditionally.
7. **if** current_idx < 0 or current_idx >= len(phases) - 1: branches conditionally.
8. **return** phases[current_idx + 1].

Code (copy-paste safe)

```
def _get_next_phase(self, firm_code, current_display):
    """Get the next phase display name from trading_phases progression."""
    if not self.prop_firm_mgr:
        return "-"
    phases = self.prop_firm_mgr.get_prop_firm_trading_phases(firm_code)
    if not phases:
        return "-"

    # Match current_display to a phase in the list
    current_idx = -1
    for i, ph in enumerate(phases):
        if current_display.lower() in ph.lower():
            current_idx = i
            break

    if current_idx < 0:
        # Not found – guess by keyword
        for i, ph in enumerate(phases):
            if "challenge" in ph.lower() and "challenge" in current_display.lower():
                current_idx = i
                break
            if "fund" in ph.lower() and "fund" in current_display.lower():
                current_idx = i
                break
            if "farm" in ph.lower() and "farm" in current_display.lower():
                current_idx = i
                break

    if current_idx < 0 or current_idx >= len(phases) - 1:
        return "Complete ✓"

    return phases[current_idx + 1]
```

Method: TradeOpssAIApp._is_eval_active

```
def _is_eval_active(self, ev)
```

Check if an evaluation is active (not failed/ended/deleted). Uses substring matching so 'Fail', 'Failed', 'Breached' etc. all caught.

Why these Python concepts were used

- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns `p1 = (ev.get('Status P1', '') or '').strip().lower()`.
2. Assigns `funded = (ev.get('Status', '') or '').strip().lower()`.
3. Assigns `has_funded_acct = bool((ev.get('Account #.1', '') or '').strip())`.
4. Assigns `has_challenge_acct = bool((ev.get('Account #', '') or '').strip())`.
5. Assigns `p1_inactive = any((kw in p1 for kw in self._INACTIVE_KEYWORDS)) if p1 else ...`
6. Assigns `funded_inactive = any((kw in funded for kw in (*self._INACTIVE_KEYWORDS, 'c...'`
7. **if** `p1_inactive` and `(not has_funded_acct)`: branches conditionally.
8. **if** `funded_inactive`: branches conditionally.
9. **if** `p1_inactive` and `has_funded_acct` and `funded_inactive`: branches conditionally.
10. **if** `not has_challenge_acct` and `(not has_funded_acct)`: branches conditionally.
11. **return** `True`.

Code (copy-paste safe)

```
def _is_eval_active(self, ev):
    """Check if an evaluation is active (not failed/ended/deleted).

    Uses substring matching so 'Fail', 'Failed', 'Breached' etc. all caught.
    """
    p1 = (ev.get("Status P1", "") or "").strip().lower()
    funded = (ev.get("Status", "") or "").strip().lower()
    has_funded_acct = bool((ev.get("Account #.1", "") or "").strip())
    has_challenge_acct = bool((ev.get("Account #", "") or "").strip())

    p1_inactive = any(kw in p1 for kw in self._INACTIVE_KEYWORDS) if p1 else False
    funded_inactive = any(kw in funded for kw in (*self._INACTIVE_KEYWORDS,
        "complete")) if funded else False

    # If challenge failed (regardless of whether funded acct number exists)
    if p1_inactive and not has_funded_acct:
        return False
    # If funded status is failed/completed
    if funded_inactive:
        return False
    # If challenge failed AND funded also failed – both phases dead
    if p1_inactive and has_funded_acct and funded_inactive:
        return False
    # Must have at least one account number
    if not has_challenge_acct and not has_funded_acct:
```

```
        return False
    return True
```

Method: TradeOpssAIApp._refresh_eval_for_account

```
def _refresh_eval_for_account(self, acct_num)
```

Fetch fresh eval data from dashboard for a specific account. Returns the updated eval dict, or None if fetch fails. When duplicate rows exist for the same account, prefers the active one.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.
2. **return** None.

Code (copy-paste safe)

```
def _refresh_eval_for_account(self, acct_num):
    """Fetch fresh eval data from dashboard for a specific account.

    Returns the updated eval dict, or None if fetch fails.
    When duplicate rows exist for the same account, prefers the active one.
    """
    try:
        email = self.client_email_entry.get().strip()
        dashboard_url = self.url_entry.get().strip().rstrip('/')
        if not email or not dashboard_url:
            return None
        r = requests.post(
            f"{dashboard_url}/api/client/data",
            json={"email": email},
            headers={"Content-Type": "application/json"},
            timeout=10
        )
        if r.status_code != 200:
            return None
        data = r.json()
        best = None
        for ev in data.get("evaluations", []):
            if ev.get("_deleted"):
                continue
            acct1 = (ev.get("Account #.1", "") or ev.get("Account #", "") or "").strip()
            acct0 = (ev.get("Account #", "") or "").strip()
            if acct_num in (acct1, acct0):
                is_active = ev.get("_is_active", self._is_eval_active(ev))
                if is_active:
                    return ev # Active match – return immediately
                if best is None:
                    best = ev # Keep first inactive as fallback
        return best
```

```

except Exception:
    pass
return None

```

Method: TradeOpssAIApp._load_active_trades

```
def _load_active_trades(self)
```

Fetch evaluations from dashboard and populate the active trades list.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.
- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.

Step by step (top-level body)

1. Assigns email = self.client_email_entry.get().strip().
2. Assigns dashboard_url = self.url_entry.get().strip().rstrip('/').
3. **if** not email: branches conditionally.
4. Calls self.log(...) for its side effect.
5. Calls self.load_trades_btn.configure(...) for its side effect.
6. Calls self.trades_count_var.set(...) for its side effect.
7. Defines a nested function _do_load(...).
8. Calls threading.Thread(target=_do_load, daemon=True).start(...) for its side effect.

Code (copy-paste safe)

```

def _load_active_trades(self):
    """Fetch evaluations from dashboard and populate the active trades list."""
    email = self.client_email_entry.get().strip()
    dashboard_url = self.url_entry.get().strip().rstrip('/')

    if not email:
        messagebox.showerror("Error", "Go to Dashboard tab, enter client email and click Lookup first")
        return

    self.log("Loading active trades from dashboard...")
    self.load_trades_btn.configure(state='disabled')
    self.trades_count_var.set("Loading...")

```



```

def _do_load():
    try:
        r = requests.post(
            f"{dashboard_url}/api/client/data",
            json={"email": email},
            headers={"Content-Type": "application/json"},
            timeout=15
        )
        if r.status_code != 200:
            msg = "Unknown error"
            try:
                msg = r.json().get("message", msg)
            except Exception:
                pass
            self.root.after(0, lambda m=msg: self.log(f"Failed to load trades: {m}", "ERROR"))
            self.root.after(0, lambda: self.trades_count_var.set("Load failed"))
            return
        data = r.json()
        evaluations = data.get("evaluations", [])

        # Filter using dashboard's _is_active flag (source of truth)
        # Falls back to local _is_eval_active() if flag missing
        active_evals = []
        skipped_count = 0
        for ev in evaluations:
            if ev.get("_deleted"):
                skipped_count += 1
                continue

            # Use dashboard-computed flag if available, else local fallback
            if "_is_active" in ev:
                is_active = ev["_is_active"]
            else:
                is_active = self._is_eval_active(ev)

            if not is_active:
                skipped_count += 1
                continue

            # Must have at least one account number
            if not (ev.get("Account #", "") or "").strip() and not (ev.get("Account #.1",
                "") or "").strip():
                skipped_count += 1
                continue

            active_evals.append(ev)

        self.root.after(0, lambda t=len(evaluations), a=len(active_evals), s=skipped_count:
            self.log(
                f"■ {t} total evaluations → {a} active, {s} filtered out (failed/completed/deleted)"
            ))

        self.root.after(0, lambda ae=active_evals: self._populate_trade_rows(ae))

        # Populate broker connection rows per prop firm
        prop_accounts = data.get("prop_accounts", [])
        self.root.after(0, lambda ae=active_evals, pa=prop_accounts: self._populate_broker_rows(ae,
            pa))

        # Auto-launch browsers for prop firms that need dashboard monitoring
        active_firms = list(dict.fromkeys(

```

```

        ev.get("Prop Firm", "") for ev in active_evals if ev.get("Prop Firm"))))
if not RELEASE_DISABLE_PROP_DASHBOARD_ACCESS:
    self.root.after(2000, lambda af=active_firms: self._auto_launch_propfirm_browsers(af))

# Trigger a status poll immediately after scan
if not RELEASE_DISABLE_STATUS_POLL:
    try:
        self._poll_tradovate_balances()
    except Exception:
        pass

# Auto-fill MT5 credentials from TradeOps dashboard.
# Prefer Hedge Accounts Configuration rows (what users actually edit in the UI),
# then fall back to legacy mt5_credentials if present.
mt5_login = ""
mt5_pass = ""
mt5_server = ""

hedge_accounts = data.get("hedge_accounts") or []
client_name = ((data.get("identity") or {}).get("client") or "").strip().lower()
chosen_hedge = None
for hedge in hedge_accounts:
    if str(hedge.get("platform") or "").strip().upper() != "MT5":
        continue
    if not ((hedge.get("login") or "").strip() and (hedge.get("password") or "").strip() and
            hedge.get("server") or "").strip()):
        continue
    hedge_name = str(hedge.get("name") or "").strip().lower()
    if client_name and hedge_name == client_name:
        chosen_hedge = hedge
        break
    if chosen_hedge is None:
        chosen_hedge = hedge

if chosen_hedge:
    mt5_login = (chosen_hedge.get("login") or "").strip()
    mt5_pass = (chosen_hedge.get("password") or "").strip()
    mt5_server = (chosen_hedge.get("server") or "").strip()
else:
    mt5_creds = data.get("mt5_credentials") or {}
    mt5_login = (mt5_creds.get("login") or "").strip()
    mt5_pass = (mt5_creds.get("password") or "").strip()
    mt5_server = (mt5_creds.get("server") or "").strip()

if mt5_login and mt5_pass and mt5_server:
    def _fill_mt5(login=mt5_login, pwd=mt5_pass, srv=mt5_server):
        # Only fill if fields are currently empty
        if not self.mt5_login.get().strip():
            self.mt5_login.delete(0, tk.END)
            self.mt5_login.insert(0, login)
        if not self.mt5_password.get().strip():
            self.mt5_password.delete(0, tk.END)
            self.mt5_password.insert(0, pwd)
        if not self.mt5_server.get().strip():
            self.mt5_server.delete(0, tk.END)
            self.mt5_server.insert(0, srv)
        self.log("■ MT5 credentials auto-filled from TradeOps dashboard")
    self.root.after(0, _fill_mt5)

except Exception as e:

```

```

        self.root.after(0, lambda: self.log(f"Load trades failed: {e}", "ERROR"))
        self.root.after(0, lambda: self.trades_count_var.set("Load failed"))
    finally:
        self.root.after(0, lambda: self.load_trades_btn.configure(state='normal'))

threading.Thread(target=_do_load, daemon=True).start()

```

Method: TradeOpssAIApp._populate_trade_rows

```
def _populate_trade_rows(self, evaluations)
```

Clear and rebuild the active trade rows — futuristic terminal style.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **for** child in self._trades_inner.winfo_children(): iterates.
2. Calls self._active_trade_rows.clear(...) for its side effect.
3. **if** not evaluations: branches conditionally.
4. Assigns firms_seen = set().
5. **for** ev in evaluations: iterates.
6. Assigns firm_bias = self._get_daily_bias(firms_seen).
7. Assigns self._auto_trade_firm_sides = firm_bias.
8. Assigns bias_parts = [].
9. **for** (f, s) in sorted(firm_bias.items()): iterates.
10. Calls self.log(...) for its side effect.
11. **for** (idx, ev) in enumerate(evaluations): iterates.
12. Assigns count = len(self._active_trade_rows).
13. Calls self.trades_count_var.set(...) for its side effect.
14. **try** block with 1 **except** clause.
15. ... and 1 more statement(s) in the body.

Code (copy-paste safe)

```

def _populate_trade_rows(self, evaluations):
    """Clear and rebuild the active trade rows – futuristic terminal style."""

```

```

for child in self._trades_inner.winfo_children():
    child.destroy()
self._active_trade_rows.clear()

if not evaluations:
    if CTk_AVAILABLE:
        empty = ctk.CTkFrame(self._trades_inner, fg_color="transparent")
        empty.pack(fill="both", expand=True, pady=40)
        ctk.CTkLabel(empty, text="■", font=("Consolas", 28),
                      text_color="#0F4C75").pack()
        ctk.CTkLabel(empty, text="NO ACTIVE TRADES DETECTED",
                      font=("Consolas", 11, "bold"),
                      text_color="#1B4965").pack(pady=(4, 2))
        ctk.CTkLabel(empty, text="Connect and verify access to populate",
                      font=("Consolas", 9),
                      text_color="#0F3460").pack()
    else:
        tk.Label(self._trades_inner, text="No active trades found",
                 fg='#94a3b8', bg='#020A14', font=('Segoe UI', 10, 'italic')).pack(pady=20)
    self.trades_count_var.set("[ 0 ]")
    return

# Daily bias per prop firm (persisted, resets each day)
firms_seen = set()
for ev in evaluations:
    firms_seen.add(ev.get("Prop Firm", "Unknown"))
firm_bias = self._get_daily_bias(firms_seen)
# Store for auto-trade compatibility
self._auto_trade_firm_sides = firm_bias

# Log bias
bias_parts = []
for f, s in sorted(firm_bias.items()):
    arrow = "▲" if s == "buy" else "▼"
    bias_parts.append(f"{arrow} {f}: {s.upper()}")
self.log(f"Direction bias: {'', '.join(bias_parts)}")

for idx, ev in enumerate(evaluations):
    prop_firm_name = ev.get("Prop Firm", "Unknown")
    firm_code = self._FIRM_MAP.get(prop_firm_name, "MFFU_Flex")
    acct_num = (ev.get("Account #.1", "") or ev.get("Account #", "") or "-").strip()
    acct_size = ev.get("Account Size", "-") or "-"
    current_display, phase_key = self._detect_eval_phase(ev)

    # Resolve phase_key from day placeholder (primary source of truth)
    resolved_key, _di, _dn = self._resolve_phase_key_from_day(ev, firm_code, current_display)
    if resolved_key:
        phase_key = resolved_key

    next_display = self._get_next_phase(firm_code, current_display)

    strip_color = self.PROP_FIRM_COLORS.get(prop_firm_name, "#95A5A6")
    glow_border, glow_bg, glow_fg = self._PHASE_GLOW.get(
        current_display, ("#475569", "#0A0F1A", "#94A3B8"))

    bias = firm_bias.get(prop_firm_name, "buy")

    if CTk_AVAILABLE:
        row_bg = "#050D18" if idx % 2 == 0 else "#071020"
        row_frame = ctk.CTkFrame(self._trades_inner, fg_color=row_bg,

```

```

        corner_radius=4, height=44,
        border_width=1, border_color="#0A1628")
row_frame.pack(fill="x", pady=1, padx=1)
row_frame.pack_propagate(False)

# Left accent bar
ctk.CTkFrame(row_frame, width=3, fg_color=strip_color,
             corner_radius=0).pack(side="left", fill="y")

# Prop firm name – aligned with header col
ctk.CTkLabel(row_frame, text=prop_firm_name[:18], width=110,
             font=("Consolas", 10, "bold"), text_color=strip_color,
             anchor="w").pack(side="left", padx=(8, 0))

# Account number
acct_display = acct_num[-8:] if len(acct_num) > 8 else acct_num
ctk.CTkLabel(row_frame, text=acct_display,
             width=88, font=("Consolas", 10),
             text_color="#6B8DAD", anchor="w").pack(side="left", padx=(8, 0))

# Size
ctk.CTkLabel(row_frame, text=acct_size[:10], width=68,
             font=("Consolas", 10), text_color="#4A7C8F",
             anchor="w").pack(side="left", padx=(8, 0))

# Phase badge – neon pill with border glow
phase_holder = ctk.CTkFrame(row_frame, fg_color="transparent", width=88)
phase_holder.pack(side="left", padx=(8, 0))
phase_holder.pack_propagate(False)
phase_pill = ctk.CTkFrame(phase_holder, fg_color=glow_bg,
                        corner_radius=8, border_width=1,
                        border_color=glow_border)
phase_pill.pack(side="left", pady=6)
ctk.CTkLabel(phase_pill, text=current_display.upper(),
             font=("Consolas", 8, "bold"),
             text_color=glow_fg).pack(padx=8, pady=1)

# Next phase with arrow
ctk.CTkLabel(row_frame, text=f"→ {next_display}", width=100,
             font=("Consolas", 9), text_color="#00D4FF",
             anchor="w").pack(side="left", padx=(8, 0))

# BUY / SELL action buttons – bias-aware
btn_frame = ctk.CTkFrame(row_frame, fg_color="transparent")
btn_frame.pack(side="right", padx=8)

row_data = {
    "frame": row_frame, "eval": ev, "firm_code": firm_code,
    "phase_key": phase_key, "acct_size": acct_size,
    "acct_num": acct_num, "current_phase": current_display,
}

# Active button gets neon glow, opposite gets muted/grayed
if bias == "buy":
    buy_fg, buy_brd, buy_txt = "#052E16", "#16A34A", "#4ADE80"
    sell_fg, sell_brd, sell_txt = "#0A0F1A", "#1A1A2E", "#2A3040"
else:
    buy_fg, buy_brd, buy_txt = "#0A0F1A", "#1A1A2E", "#2A3040"
    sell_fg, sell_brd, sell_txt = "#2D0A0A", "#DC2626", "#F87171"

buy_btn = ctk.CTkButton(btn_frame, text="▲ BUY", width=58, height=26,

```

```

        fg_color=buy_fg, hover_color="#14532D" if bias == "buy" else "#0A0F1A",
        border_width=1, border_color=buy_brd,
        font=("Consolas", 9, "bold"),
        text_color=buy_txt, corner_radius=4,
        command=lambda rd=row_data: self._execute_row_trade("buy", rd))
buy_btn.pack(side="left", padx=(0, 3))

sell_btn = ctk.CTkButton(btn_frame, text="▼ SELL", width=58, height=26,
        fg_color=sell_fg, hover_color="#450A0A" if bias == "sell" else "#0A0F1A",
        border_width=1, border_color=sell_brd,
        font=("Consolas", 9, "bold"),
        text_color=sell_txt, corner_radius=4,
        command=lambda rd=row_data: self._execute_row_trade("sell", rd))
sell_btn.pack(side="left")
else:
    # Fallback plain tk
    row_bg = '#050D18' if idx % 2 == 0 else '#071020'
    row_frame = tk.Frame(self._trades_inner, bg=row_bg)
    row_frame.pack(fill="x", pady=1)

    tk.Label(row_frame, text=prop_firm_name[:16], width=14, anchor='w',
        bg=row_bg, fg='#e2e8f0', font=('Consolas', 9)).pack(side="left", padx=2)
    tk.Label(row_frame, text=acct_num[:12], width=10, anchor='w',
        bg=row_bg, fg='#6B8DAD', font=('Consolas', 9)).pack(side="left", padx=2)
    tk.Label(row_frame, text=acct_size[:10], width=8, anchor='w',
        bg=row_bg, fg='#4A7C8F', font=('Consolas', 9)).pack(side="left", padx=2)
    tk.Label(row_frame, text=current_display, width=14, anchor='w',
        bg=row_bg, fg='#fbbf24', font=('Consolas', 9, 'bold')).pack(side="left", padx=2)
    tk.Label(row_frame, text=f"→ {next_display}", width=14, anchor='w',
        bg=row_bg, fg='#00D4FF', font=('Consolas', 9)).pack(side="left", padx=2)

    btn_frame = tk.Frame(row_frame, bg=row_bg)
    btn_frame.pack(side="left", padx=4)

    row_data = {
        "frame": row_frame, "eval": ev, "firm_code": firm_code,
        "phase_key": phase_key, "acct_size": acct_size,
        "acct_num": acct_num, "current_phase": current_display,
    }

    buy_btn = tk.Button(btn_frame, text="▲ BUY",
        bg='#052E16' if bias == 'buy' else '#0A0F1A',
        fg='#4ADE80' if bias == 'buy' else '#2A3040',
        font=('Consolas', 8, 'bold'), relief='flat', padx=6, pady=1,
        command=lambda rd=row_data: self._execute_row_trade("buy", rd))
    buy_btn.pack(side="left", padx=(0, 4))

    sell_btn = tk.Button(btn_frame, text="▼ SELL",
        bg='#2D0A0A' if bias == 'sell' else '#0A0F1A',
        fg='#F87171' if bias == 'sell' else '#2A3040',
        font=('Consolas', 8, 'bold'), relief='flat', padx=6, pady=1,
        command=lambda rd=row_data: self._execute_row_trade("sell", rd))
    sell_btn.pack(side="left")

    row_data["buy_btn"] = buy_btn
    row_data["sell_btn"] = sell_btn
    self._active_trade_rows.append(row_data)

count = len(self._active_trade_rows)
self.trades_count_var.set(f"[ {count} ]")

```

```
# Update stats strip
try:
    self._stat_queue_var.set(f"Queue: {count}")
except Exception:
    pass
self.log(f"Loaded {count} active trades from dashboard")
```

Method: TradeOpssAIApp._eval_has_payout

```
def _eval_has_payout(self, ev)
```

Check if any hedge result field contains 'payout' text.

Why these Python concepts were used

- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.

Step by step (top-level body)

1. **if not ev:** branches conditionally.
2. **for (key, val) in ev.items():** iterates.
3. **return False.**

Code (copy-paste safe)

```
def _eval_has_payout(self, ev):
    """Check if any hedge result field contains 'payout' text."""
    if not ev:
        return False
    for key, val in ev.items():
        if "hedge result" in key.lower() and isinstance(val, str) and "payout" in val.lower():
            return True
    return False
```

Method: TradeOpssAIApp._execute_row_trade

```
def _execute_row_trade(self, side, row_data)
```

Execute a trade for a specific row, then remove the row.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to `sum`, `any`, `all`, or `max` where the intermediate list would be wasted.
- **lambda** — Uses a single-expression anonymous function — typically as the `key=` for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **walrus operator (:=)** — Assigns and tests in one expression — typically inside a while or if to capture a regex match or a non-None lookup without an extra line.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns `ev = row_data.get('eval', {})`.
2. `if self._eval_has_payout(ev):` branches conditionally.
3. Assigns `direction = self.direction_var.get()`.
4. `if direction == 'Buy Only' and side == 'sell':` branches conditionally.
5. `if direction == 'Sell Only' and side == 'buy':` branches conditionally.
6. Assigns `firm_code = row_data['firm_code']`.
7. Assigns `phase_key = row_data['phase_key']`.
8. Assigns `acct_size = row_data['acct_size']`.
9. Assigns `acct_num = row_data['acct_num']`.
10. Assigns `fresh_ev = self._refresh_eval_for_account(acct_num)`.
11. `if fresh_ev:` branches conditionally.
12. Assigns `(resolved_key, day_idx, day_name) = self._resolve_phase_key_from_day(ev, firm_code, row_data[...]`
13. `if resolved_key is None:` branches conditionally.
14. `if resolved_key != phase_key:` branches conditionally.
15. ... and 38 more statement(s) in the body.

Code (copy-paste safe)

```
def _execute_row_trade(self, side, row_data):
    """Execute a trade for a specific row, then remove the row."""
    # Check for payout – skip account if any hedge result has payout text
    ev = row_data.get("eval", {})
    if self._eval_has_payout(ev):
        acct_num = row_data.get("acct_num", "?")
        messagebox.showwarning("Payout Pending",
                               f"Account {acct_num} has a PAYOUT pending.\n"
                               f"Request payout first before continuing.")
        return

    # Direction filter
    direction = self.direction_var.get()
    if direction == "Buy Only" and side == "sell":
```



```

        messagebox.showwarning("Direction Lock", "Direction is set to Buy Only")
        return
    if direction == "Sell Only" and side == "buy":
        messagebox.showwarning("Direction Lock", "Direction is set to Sell Only")
        return

    firm_code = row_data["firm_code"]
    phase_key = row_data["phase_key"]
    acct_size = row_data["acct_size"]
    acct_num = row_data["acct_num"]

    # ■■ Resolve phase_key from day placeholder (primary source of truth) ■■
    fresh_ev = self._refresh_eval_for_account(acct_num)
    if fresh_ev:
        ev = fresh_ev
        row_data["eval"] = fresh_ev
    resolved_key, day_idx, day_name = self._resolve_phase_key_from_day(
        ev, firm_code, row_data["current_phase"])
    if resolved_key is None:
        messagebox.showwarning("No Day Placeholder",
            f"Account {acct_num}: No day placeholder found.\n"
            f"Enter a day name (MON/TUE/etc.) in the next cell to enable trading.")
        return
    if resolved_key != phase_key:
        self.log(f"■ {acct_num}: Day cell {day_idx + 1} ({day_name}) → "
            f"blueprint {resolved_key} (was {phase_key})")
        phase_key = resolved_key

    # Get trade config from blueprint
    config = None
    if self.prop_firm_mgr:
        config = self.prop_firm_mgr.get_strategy_config(firm_code, phase_key, acct_size)
    if not config:
        messagebox.showerror("Error", f"No blueprint config for {firm_code} / {phase_key} / {acct_size}")
        return

    hedging = self.hedge_mode_var.get() == "Hedging"
    prop_firm_name = row_data["eval"].get("Prop Firm", firm_code) if row_data.get("eval") else firm_code
    # Auto-detect platform: TopStep firms always use TopStepX (Selenium)
    if "topstep" in prop_firm_name.lower():
        platform = "TopStepX"
    else:
        platform = self.broker_var.get()
    broker_account = self._get_broker_for_firm(prop_firm_name)

    if not broker_account:
        messagebox.showerror("Error", f"Connect broker for {prop_firm_name} first")
        return

    mt5_api = None
    if hedging:
        mt5_api = self._get_mt5_trading_api()
        if not mt5_api:
            messagebox.showerror("Error", "Connect MT5 for hedging mode")
            return

    tradovate_sym = config.get("tradovate_symbol", "") or config.get("topstepx_symbol", "")
    tradovate_qty = int(config.get("tradovate_qty", 2) or config.get("topstepx_qty", 2))

    # ■■ Farming: cap MT5 TP based on hard-stop proximity ■■

```

```

ev = row_data.get("eval", {})
_is_farming_sym = "MNQ" in (config.get("tradovate_symbol", "") or config.get("topstepx_symbol",
    "")).upper()
if _is_farming_sym and self.prop_firm_mgr:
    try:
        _bal = None
        if broker_account:
            _stats = broker_account.get_account_stats()
            _bal_str = _stats.get("Balance", "")
            if _bal_str and _bal_str not in ("N/A", "Error", ""):
                _bal = float(_bal_str.replace("$", "").replace(",", ""))
        if _bal is not None:
            orig_mt5_tp = int(config.get("mt5_tp_points", 0))
            config = self.prop_firm_mgr.adjust_farming_tp_sl(config, _bal, firm_code)
            new_mt5_tp = int(config.get("mt5_tp_points", 0))
            if new_mt5_tp != orig_mt5_tp:
                self.log(
                    f"■ Farming TP cap {acct_num}: balance=${_bal:,.2f} → MT5 TP {orig_mt5_tp}→
                else:
                    self.log(f"■ Farming TP OK {acct_num}: MT5 TP {orig_mt5_tp} pts within safe rang
            else:
                self.log(f"■ Farming {acct_num}: could not read balance – using blueprint TP/SL")
        except Exception as _fe:
            self.log(f"■ Farming TP check failed for {acct_num}: {_fe}")
    # TP/SL comes directly from the stage blueprint (selected by day placeholder)

trado_tp = int(config.get("tradovate_tp_ticks", 151) or config.get("topstepx_tp_ticks", 151))
trado_sl = int(config.get("tradovate_sl_ticks", 200) or config.get("topstepx_sl_ticks", 200))
blueprint_tp_orig = trado_tp # save originals for confirm dialog
blueprint_sl_orig = trado_sl
_adj_reasons = [] # collect adjustment explanations for confirm dialog
mt5_sym = config.get("mt5_symbol", "NAS100")
mt5_vol = float(config.get("mt5_volume", 2.8))
mt5_tp = int(config.get("mt5_tp_points", 46))
mt5_sl = int(config.get("mt5_sl_points", 42))

# ■■ Adjust TP based on stage progress ■■
# If we already have profit in this stage, shrink TP so we don't overshoot.
# If we're short of the expected start balance, grow TP to catch up.
if self.prop_firm_mgr and broker_account and not _is_farming_sym:
    try:
        current_profit = self._get_current_phase_profit(
            ev, row_data["current_phase"], broker_account=broker_account, acct_size=acct_size)
        size_key = self.prop_firm_mgr.convert_account_size_to_key(acct_size)
        stage_start = self.prop_firm_mgr.get_stage_start_target(
            firm_code, row_data["current_phase"], phase_key, size_key)
        stage_profit_so_far = current_profit - stage_start
        trado_sym_for_calc = config.get("tradovate_symbol", "") or config.get("topstepx_symbol",
            tick_val = self.prop_firm_mgr.get_tick_value(
                trado_sym_for_calc) if self.prop_firm_mgr else 5.0
        trado_qty_for_calc = int(config.get("tradovate_qty", 1) or config.get("topstepx_qty", 1))
        if tick_val > 0 and trado_qty_for_calc > 0:
            orig_tp = trado_tp
            orig_mt5_sl = mt5_sl
            # Convert stage profit to ticks and subtract from TP
            profit_ticks = stage_profit_so_far / (tick_val * trado_qty_for_calc)
            adjusted_tp = max(5, round(trado_tp - profit_ticks))
            tp_ratio = adjusted_tp / trado_tp if trado_tp > 0 else 1.0
            adjusted_mt5_sl = max(5, round(mt5_sl * tp_ratio))
            if adjusted_tp != trado_tp:

```

```

        trado_tp = adjusted_tp
        mt5_sl = adjusted_mt5_sl
        _adj_reasons.append(
            f"TP {blueprint_tp_orig}→{trado_tp}t: stage P/L ${stage_profit_so_far:+,.0f}, "
            f"(already earned in this stage)")
        self.log(f"■ TP adjust {acct_num}: stage_start=${stage_start:+,.0f}, "
            f"current P/L=${current_profit:+,.2f}, stage P/L=${stage_profit_so_far:+,.2f}, "
            f"TP {orig_tp}→{trado_tp}t, MT5 SL {orig_mt5_sl}→{mt5_sl}pts")
    except Exception as _te:
        self.log(f"■ TP adjust failed for {acct_num}: {_te}")

# ■■ Adjust SL based on midnight balance + drawdown protection ■■
if broker_account and platform == "Tradovate" and hasattr(broker_account, 'get_min_equity'):
    try:
        min_eq_data = broker_account.get_min_equity()
        if min_eq_data:
            live_net_liq = min_eq_data['net_liq']
            net_liq_sod = min_eq_data.get('net_liq_sod', 0)
            live_min_equity = min_eq_data.get('min_equity', 0)
            tmdl = min_eq_data.get('trailing_max_drawdown_limit', 50000)
            trado_qty_for_calc = int(config.get("tradovate_qty", 1) or config.get("topstepx_qty",
            trado_sym_for_calc = config.get("tradovate_symbol", "") or config.get("topstepx_symbol",
            ""))
            tick_val = self.prop_firm_mgr.get_tick_value(
                trado_sym_for_calc) if self.prop_firm_mgr else 5.0

            # Step 1: Midnight balance SL – floor = SOD - blueprint_sl_dollars
            if net_liq_sod > 0 and tick_val > 0 and trado_qty_for_calc > 0:
                blueprint_sl_dollars = trado_sl * tick_val * trado_qty_for_calc
                sl_floor = net_liq_sod - blueprint_sl_dollars
                available = live_net_liq - sl_floor
                if available > 0:
                    midnight_sl = max(10, int(available / (tick_val * trado_qty_for_calc)))
                    if midnight_sl != trado_sl:
                        orig_sl = trado_sl
                        orig_mt5_tp = mt5_tp
                        trado_sl = midnight_sl
                        mt5_tp = max(5, int(trado_sl / 4) - 1)
                        daily_pnl = live_net_liq - net_liq_sod
                        _adj_reasons.append(
                            f"SL {blueprint_sl_orig}→{trado_sl}t: midnight bal ${net_liq_sod:+,.0f}, "
                            f"daily P/L ${daily_pnl:+,.0f}")
                        self.log(f"■ Midnight SL {acct_num}: SOD=${net_liq_sod:+,.2f}, "
                            f"live=${live_net_liq:+,.2f}, daily P/L=${daily_pnl:+,.2f} → "
                            f"SL {orig_sl}→{trado_sl}t, MT5 TP {orig_mt5_tp}→{mt5_tp}pts")
                    else:
                        self.log(
                            f"■ Midnight SL OK {acct_num}: SOD=${net_liq_sod:+,.2f}, SL={trado_sl}t, MT5 TP={mt5_tp}pts")
                else:
                    self.log(f"■ Midnight SL floor breached {acct_num}: "
                        f"live=${live_net_liq:+,.2f} < floor=${sl_floor:+,.2f} – using min SL")
                    _adj_reasons.append(
                        f"SL → 10t: balance below midnight SL floor ${sl_floor:+,.0f}")
                    trado_sl = 10
                    mt5_tp = max(5, int(trado_sl / 4) - 1)

            # Step 2: TMDL drawdown cap – further tighten if near breach
            if live_min_equity > 0 and live_net_liq < tmdl:
                drawdown_remaining = live_net_liq - live_min_equity
                current_sl_risk = trado_sl * tick_val * trado_qty_for_calc

```

```

        if drawdown_remaining > 0 and current_sl_risk > drawdown_remaining:
            orig_sl = trado_sl
            orig_mt5_tp = mt5_tp
            trado_sl = max(10, int(drawdown_remaining / (tick_val * trado_qty_for_calc)))
            mt5_tp = max(5, int(trado_sl / 4) - 1)
            _adj_reasons.append(
                f"SL →{trado_sl}t: near drawdown limit, only ${drawdown_remaining:,.0f}"
            )
            self.log(f"■ TMDL SL cap {acct_num}: remaining=${drawdown_remaining:,.2f} →"
                    f"SL {orig_sl}→{trado_sl}t, MT5 TP {orig_mt5_tp}→{mt5_tp}pts")
        elif drawdown_remaining <= 0:
            self.log(f"■ No drawdown room for {acct_num} (${drawdown_remaining:,.2f})")
        elif live_min_equity > 0:
            self.log(f"■ TMDL OK {acct_num}: ${live_net_liq:,.2f} ≥ TMDL=${tmdl:,.0f}")
    except Exception:
        pass

    hedge_text = f" + MT5 {'SELL' if side == 'buy' else 'BUY'} {mt5_vol} {mt5_sym}" if hedging else ""

    # Stage info from day placeholder
    stage_text = (f"\n\n■ Stage Info ■"
                  f"\nDay Cell: {day_idx + 1} ({day_name}) → Blueprint: {phase_key}")

    # Adjustment explanations
    if _adj_reasons:
        adj_text = "\n\n■ Adjustments ■\n" + "\n".join(f"• {r}" for r in _adj_reasons)
        adj_text += f"\n(Blueprint was TP {blueprint_tp_orig}t / SL {blueprint_sl_orig}t)"
    else:
        adj_text = ""

    confirm = messagebox.askyesno("Confirm Trade",
        f"{side.upper()} {trado_qty} {trado_sym} on {platform}\n"
        f"{hedge_text}\n\n"
        f"Account: {acct_num} | {firm_code}\n"
        f"Phase: {row_data['current_phase']} | Size: {acct_size}\n"
        f"TP: {trado_tp} ticks | SL: {trado_sl} ticks"
        f"{stage_text}{adj_text}\n\nProceed?")
    if not confirm:
        return

    # Disable buttons immediately
    row_data["buy_btn"].configure(state='disabled', text="...")
    row_data["sell_btn"].configure(state='disabled', text="...")

    prop_name = ev.get("Prop Firm", firm_code) if (ev := row_data.get("eval")) else firm_code
    hedge_tag = f" ↔ MT5 {'SELL' if side == 'buy' else 'BUY'} {mt5_vol} {mt5_sym}" if hedging else ""
    self.log(
        f"■ {side.upper()} {trado_qty} {trado_sym} → {prop_name} {acct_num} [{row_data['current_phase']}]"
    )

    def _do_trade():
        try:
            # 1. Broker order
            if platform == "Tradovate":
                if side == "buy":
                    order_result = broker_account.buy_market(trado_sym, trado_qty, tp=trado_tp,
                                                              sl=trado_sl, expected_account=acct_num)
                else:
                    order_result = broker_account.sell_market(trado_sym, trado_qty, tp=trado_tp,
                                                              sl=trado_sl, expected_account=acct_num)
            elif platform == "TopStepX":
                # Switch to the correct account if multiple accounts under same login

```

```

        if hasattr(broker_account, 'switch_account') and acct_num:
            switched = broker_account.switch_account(account_name_contains=acct_num)
            if not switched:
                raise Exception(f"TopStepX could not switch to account {acct_num}")
        # TopStepX uses two-digit year futures codes (NQM26, MNQM26)
        _tsx_sym = _to_topstepx_symbol(trado_sym)
        # Convert ticks to dollars for TopStepX: dollars = ticks * tick_value * quantity
        _tsx_tick_val = self.prop_firm_mgr.get_tick_value(_tsx_sym) if self.prop_firm_mgr else 0
        _tsx_tp_dollars = trado_tp * _tsx_tick_val * trado_qty
        _tsx_sl_dollars = trado_sl * _tsx_tick_val * trado_qty
        if side == "buy":
            order_result = broker_account.place_buy_order(_tsx_sym, trado_qty,
                tp_dollars=_tsx_tp_dollars, sl_dollars=_tsx_sl_dollars)
        else:
            order_result = broker_account.place_sell_order(_tsx_sym, trado_qty,
                tp_dollars=_tsx_tp_dollars, sl_dollars=_tsx_sl_dollars)

        # Some broker implementations return a status dict instead of raising.
        # Normalize failures so UI/logs don't report false fills.
        if isinstance(order_result, dict) and order_result.get("success") is False:
            raise Exception(order_result.get("message") or "Broker reported unsuccessful order")

    self.log(
        f"■ {platform} filled {side.upper()} {trado_qty} {trado_sym} | TP:{trado_tp}t SL:{trado_sl}"

    # 2. MT5 hedge (opposite direction)
    if hedging and mt5_api:
        hedge_side = "sell" if side == "buy" else "buy"
        comment = f"{acct_num}_{phase_key}"
        if hedge_side == "buy":
            mt5_api.buy_market(mt5_sym, mt5_vol, sl=mt5_sl, tp=mt5_tp, comment=comment)
        else:
            mt5_api.sell_market(mt5_sym, mt5_vol, sl=mt5_sl, tp=mt5_tp, comment=comment)
        self.log(
            f"■ MT5 hedge {hedge_side.upper()} {mt5_vol} {mt5_sym} TP:{mt5_tp} SL:{mt5_sl} c

    # ■■ Auto-status: set "In Progress" when trade goes out ■■
    _ev = row_data.get("eval")
    if _ev:
        _has_funded = bool((_ev.get("Account #.1") or "").strip())
        _sf = "Status" if _has_funded else "Status P1"
        _cur = (_ev.get(_sf) or "").strip().lower()
        if not _cur or _cur in ("not started", "in progress", ""):
            _ev[_sf] = "In Progress"
            self.log(f"■ Auto-status: {acct_num} → {_sf}='In Progress'")

    # Remove row from list
    def _remove():
        row_data["frame"].destroy()
        if row_data in self._active_trade_rows:
            self._active_trade_rows.remove(row_data)
        remaining = len(self._active_trade_rows)
        self.trades_count_var.set(
            f"{remaining} active trade{'s' if remaining != 1 else ''}"
            if remaining > 0 else "All trades complete ✓")
        try:
            self._stat_queue_var.set(f"Queue: {remaining}")
            # Increment trade count
            cur = self._stat_trades_var.get()
            n = int(cur.split(":")[1].strip()) + 1 if ":" in cur else 1

```

```

        self._stat_trades_var.set(f"Trades: {n}")
    except Exception:
        pass

    self.root.after(0, _remove)

    except Exception as e:
        self.log(f"■ Trade failed for {acct_num}: {e}", "ERROR")
        self.root.after(0, lambda: messagebox.showerror("Trade Error", str(e)))
        # Re-enable buttons on failure
        self.root.after(0, lambda: row_data["buy_btn"].configure(state='normal', text="▲ BUY"))
        self.root.after(0, lambda: row_data["sell_btn"].configure(state='normal', text="▼ SELL"))

    threading.Thread(target=_do_trade, daemon=True).start()

```

Method: TradeOpssAIApp._toggle_auto_trade

```
def _toggle_auto_trade(self)
```

Toggle the auto-trade scheduler on/off.

Step by step (top-level body)

1. **if** self.auto_trade_enabled: branches conditionally (with an **else/elif** arm).

Code (copy-paste safe)

```

def _toggle_auto_trade(self):
    """Toggle the auto-trade scheduler on/off."""
    if self.auto_trade_enabled:
        self._stop_auto_trade()
    else:
        self._start_auto_trade()

```

Method: TradeOpssAIApp._test_min_equity

```
def _test_min_equity(self, event=None)
```

Test: dump min equity data from cached prop firm mappings and Tradovate API.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Calls self.log(...) for its side effect.
2. **if** self._cached_acct_mappings: branches conditionally (with an **else/elif** arm).
3. Assigns found = False.
4. **for** (firm_name, conn) in self._broker_connections.items(): iterates.
5. **if** not found: branches conditionally.

6. Calls `self.log(...)` for its side effect.

Code (copy-paste safe)

```
def _test_min_equity(self, event=None):
    """Test: dump min equity data from cached prop firm mappings and Tradovate API."""
    self.log("■ Testing min equity sources...")
    # 1. Cached prop firm mappings
    if self._cached_acct_mappings:
        for firm_name, mapping in self._cached_acct_mappings.items():
            self.log(f" --- {firm_name} (cached mapping) ---")
            for key, info in mapping.items():
                me = info.get("min_equity")
                pt = info.get("profit_target")
                bal = info.get("balance")
                start = info.get("starting_balance")
                self.log(f"      {key}: bal=${bal}, start=${start}, min_eq=${me}, target=${pt}")
    else:
        self.log(" ■ No cached account mappings (connect prop firm dashboards first)")
    # 2. Tradovate API fallback
    found = False
    for firm_name, conn in self._broker_connections.items():
        acct = conn.get("account")
        if not acct or not hasattr(acct, 'get_min_equity'):
            continue
        found = True
        self.log(f" --- {firm_name} (Tradovate API) ---")
        try:
            result = acct.get_min_equity()
            if result:
                self.log(f"      Net Liq:          ${result['net_liq']:, .2f}")
                self.log(f"      Min Equity:       ${result['min_equity']:, .2f}")
                self.log(f"      Drawdown Remaining: ${result['drawdown_remaining']:, .2f}")
                self.log(f"      Max Net Liq:       ${result.get('max_net_liq', 0):, .2f}")
                self.log(f"      Trailing Max DD:   ${result['trailing_max_drawdown']:, .2f}")
                self.log(f"      Trailing DD Limit: ${result['trailing_max_drawdown_limit']:, .2f}")
                self.log(f"      Mode:              {result.get('trailing_mode', '?')}")
            else:
                self.log(f"      ■ get_min_equity returned None")
        except Exception as e:
            self.log(f"      ■ Error: {e}", "ERROR")
    if not found:
        self.log(" ■ No connected Tradovate brokers")
    self.log("■ Min equity test complete.")
```

Method: `TradeOpssAIApp._start_hedge_protector`

```
def _start_hedge_protector(self)
```

Start (or restart) the Hedge Protector engine. Called automatically on broker connect.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **lambda** — Uses a single-expression anonymous function — typically as the `key=` for a sort, the filter predicate, or a small callback.

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **if** `RELEASE_DISABLE_HEDGE_GUARD`: branches conditionally.
2. **if** `not HEDGE_PROTECTOR_AVAILABLE`: branches conditionally.
3. **if** `self._hedge_protector` and `self._hedge_protector.is_running`: branches conditionally.
4. Assigns `connected_tv = {}`.
5. **for** `(firm_name, conn)` in `self._broker_connections.items()`: iterates.
6. **if** `not connected_tv`: branches conditionally.
7. Assigns `mt5_api = self._get_mt5_trading_api()` if `hasattr(self, '_get_mt5_tr...`
8. Defines a nested function `on_event(...)`.
9. Defines a nested function `on_status_change(...)`.
10. Assigns `self._hedge_protector = HedgeProtector(mt5_api=mt5_api, tradovate_accounts=connec....`
11. Calls `self._hedge_protector.start(...)` for its side effect.
12. Calls `self.log(...)` for its side effect.

Code (copy-paste safe)

```
def _start_hedge_protector(self):
    """Start (or restart) the Hedge Protector engine. Called automatically on broker connect."""
    if RELEASE_DISABLE_HEDGE_GUARD:
        return

    if not HEDGE_PROTECTOR_AVAILABLE:
        return

    # If already running, stop first so we can pick up newly connected accounts
    if self._hedge_protector and self._hedge_protector.is_running:
        try:
            self._hedge_protector.stop()
        except Exception:
            pass
        self._hedge_protector = None

    # Gather every connected Tradovate account
    connected_tv = {}
    for firm_name, conn in self._broker_connections.items():
        acct = conn.get("account")
        if acct and hasattr(acct, '_api_fetch'):
            connected_tv[firm_name] = acct

    if not connected_tv:
        return # nothing to guard yet

    # Get MT5 API
```



```

mt5_api = self._get_mt5_trading_api() if hasattr(self, '_get_mt5_trading_api') else None

def on_event(event_type, message):
    """Route HedgeProtector events to the UI."""
    try:
        kind = {
            "info": "info", "warn": "queue", "error": "error",
            "sl_detected": "error", "tp_detected": "success",
            "close_sent": "trade",
        }.get(event_type, "info")
        self.root.after(0, lambda: self._add_activity(f"■■■ {message}", kind))
        self.root.after(0, lambda: self.log(f"■■■ [{event_type.upper()}] {message}"))
    except Exception:
        pass

def on_status_change(acct_name, close_reason, phase_key):
    """Immediately update dashboard status when TP/SL detected."""
    self.root.after(0, lambda: self._handle_hedge_status_change(acct_name, close_reason, phase_key))

self._hedge_protector = HedgeProtector(
    mt5_api=mt5_api,
    tradovate_accounts=connected_tv,
    on_event=on_event,
    on_status_change=on_status_change,
)
self._hedge_protector.start()

self.log(f"■■■ Hedge Guard ACTIVE – monitoring {len(connected_tv)} Tradovate account(s)")

```

Method: TradeOpssAIApp._stop_hedge_protector

```
def _stop_hedge_protector(self)
```

Stop the Hedge Protector engine.

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. if self._hedge_protector: branches conditionally.

Code (copy-paste safe)

```

def _stop_hedge_protector(self):
    """Stop the Hedge Protector engine."""
    if self._hedge_protector:
        stats = self._hedge_protector.get_status()
        self._hedge_protector.stop()
        self._hedge_protector = None
        self.log(
            f"■■■ Hedge Guard stopped – SL protected: {stats.get('sl_protected', 0)}, TP passed: {sta

```

Method: TradeOpssAIApp._handle_hedge_status_change

```
def _handle_hedge_status_change(self, acct_name, close_reason, phase_key)
```

Update dashboard status immediately when TP/SL detected by hedge protector. Args: acct_name: Tradovate account name (e.g. "FNFTCHHARRISONOUKA85625") close_reason: "tp_detected" or "sl_detected" phase_key: Blueprint stage key from MT5 comment (e.g. "challenge_trade2", "funded_trade1")

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **list comprehension** — Builds a list in a single expression ([f(x) for x in xs]). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **if** RELEASE_DISABLE_AUTO_STATUS_UPDATES: branches conditionally.
2. Imports re (lazy import inside the function).
3. Assigns matched_ev = None.
4. Assigns matched_rd = None.
5. Assigns acct_lower = acct_name.lower().
6. **for** rd in self._active_trade_rows: iterates.
7. **if** not matched_ev: branches conditionally.
8. Assigns has_funded = bool((matched_ev.get('Account #.1') or "").strip()).
9. Assigns status_field = 'Status' if has_funded else 'Status P1'.
10. **if** close_reason == 'sl_detected': branches conditionally (with an **else/elif** arm).
11. Assigns current_status = (matched_ev.get(status_field) or "").strip().
12. **if** current_status.lower() == new_status.lower(): branches conditionally.
13. Assigns matched_ev[status_field] = new_status.
14. Calls self.log(...) for its side effect.
15. ... and 9 more statement(s) in the body.

Code (copy-paste safe)

```
def _handle_hedge_status_change(self, acct_name, close_reason, phase_key):
    """Update dashboard status immediately when TP/SL detected by hedge protector.

    Args:
```

```

        acct_name: Tradovate account name (e.g. "FNFTCHHARRISONOUKA85625")
        close_reason: "tp_detected" or "sl_detected"
        phase_key: Blueprint stage key from MT5 comment (e.g. "challenge_trade2", "funded_trade1")
    """
    if RELEASE_DISABLE_AUTO_STATUS_UPDATES:
        return

    import re

    # 1. Find the matching eval from active trade rows
    matched_ev = None
    matched_rd = None
    acct_lower = acct_name.lower()
    for rd in self._active_trade_rows:
        ev = rd.get("eval")
        if not ev:
            continue
        acct_ch = (ev.get("Account #") or "").strip().lower()
        acct_fd = (ev.get("Account #.1") or "").strip().lower()
        if acct_lower in acct_ch or acct_ch in acct_lower or \
            acct_lower in acct_fd or acct_fd in acct_lower:
            matched_ev = ev
            matched_rd = rd
            break

    if not matched_ev:
        self.log(f"■ Hedge status: no eval found for {acct_name} – skipping status update")
        return

    # 2. Determine which status field to update
    has_funded = bool((matched_ev.get("Account #.1") or "").strip())
    status_field = "Status" if has_funded else "Status P1"

    # 3. Extract trade number from phase_key → status value
    # phase_key examples: "challenge_trade1", "funded_trade2", "funded_trade_doubledip_3"
    # Dashboard TP statuses: "Hit TP1", "Hit TP2", "Hit TP3", "Hit TP4"
    # Dashboard SL statuses: "Fail"
    if close_reason == "sl_detected":
        new_status = "Fail"
    else:
        # Extract the trade number from the phase_key
        m = re.search(r'(\d+)$', phase_key)
        trade_num = int(m.group(1)) if m else 1
        new_status = f"Hit TP{trade_num}"

    current_status = (matched_ev.get(status_field) or "").strip()
    if current_status.lower() == new_status.lower():
        self.log(f"■ Hedge status: {acct_name} already {new_status}")
        return

    # 4. Update the eval
    matched_ev[status_field] = new_status
    self.log(f"■ Hedge status: {acct_name} [{phase_key}] → {status_field}='{new_status}' "
            f"({'SL hit' if close_reason == 'sl_detected' else 'TP hit'})")
    self._add_activity(
        f"■ {acct_name}: {new_status} ({phase_key})",
        "error" if close_reason == "sl_detected" else "success")

    # 5. Push to dashboard immediately
    email = self.client_email_entry.get().strip()

```

```

dashboard_url = self.url_entry.get().strip().rstrip('/')
if not email or not dashboard_url:
    self.log(f"■ Hedge status: no email/dashboard URL – skipping push")
    return
all_evals = [rd.get("eval") for rd in self._active_trade_rows if rd.get("eval")]
if not all_evals:
    return

def _push():
    try:
        import requests
        payload = {
            "email": email,
            "evaluations": all_evals,
            "statistics": {},
            "dropdown_options": {},
            "force_fields": [status_field],
        }
        resp = requests.post(
            f"{dashboard_url}/api/client/push",
            json=payload,
            headers={"Content-Type": "application/json"},
            timeout=15)
        if resp.status_code == 200 and resp.json().get("status") == "success":
            self.root.after(0, lambda:
                self.log(f"■ Hedge status pushed: {acct_name} → {new_status}"))
        else:
            self.root.after(0, lambda s=resp.status_code:
                self.log(f"■ Hedge status push failed: HTTP {s}"))
    except Exception as e:
        self.root.after(0, lambda err=str(e):
            self.log(f"■ Hedge status push error: {err}"))

import threading
threading.Thread(target=_push, daemon=True, name="HedgeStatusPush").start()

```

Method: TradeOpssAIApp._close_all_trades

```
def _close_all_trades(self)
```

Liquidate all positions across every connected broker and MT5.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **list comprehension** — Builds a list in a single expression ([f(x) for x in xs]). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.

- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns `connected = [f for f, c in self._broker_connections.items() if c.get(...]`
2. `if not connected:` branches conditionally.
3. `if not messagebox.askyesno('Close ALL Trades', f'This will LIQUIDATE a...:` branches conditionally.
4. Calls `self.log(...)` for its side effect.
5. Calls `self._add_activity(...)` for its side effect.
6. Defines a nested function `_do_close_all(...)`.
7. Calls `threading.Thread(target=_do_close_all, daemon=True, name='CloseAll').start(...)` for its side effect.

Code (copy-paste safe)

```
def _close_all_trades(self):
    """Liquidate all positions across every connected broker and MT5."""
    # Confirm first
    connected = [f for f, c in self._broker_connections.items() if c.get("account")]
    if not connected:
        self.log("■ No brokers connected – nothing to close")
        return

    if not messagebox.askyesno(
        "Close ALL Trades",
        f"This will LIQUIDATE all open positions on:\n\n"
        f" • {len(connected)} broker(s): {'', '.join(connected)}\n"
        f" • MT5 (all open positions)\n\n"
        f"Are you sure?",
        icon="warning",
    ):
        return

    self.log("■ CLOSING ALL TRADES across all platforms...")
    self._add_activity("■ Close All triggered – liquidating everything", "error")

    def _do_close_all():
        closed_tv = 0
        closed_mt5 = 0
        errors = []

        # 1. Liquidate every connected Tradovate / TopStepX account
        for firm_name, conn in self._broker_connections.items():
            acct = conn.get("account")
            if not acct:
                continue
            try:
                if hasattr(acct, 'liquidate_position_api'):
                    # Tradovate – liquidate all accounts under this login
```

```

accounts = acct._api_fetch("/account/list")
if accounts and isinstance(accounts, list):
    for a in accounts:
        aid = a['id']
        aname = a.get('name', '?')
        # Check if positions exist first
        positions = acct.get_positions_api(account_id=aid)
        open_pos = [p for p in positions if p.get('netPos', 0) != 0]
        if not open_pos:
            self.root.after(0, lambda fn=firm_name, n=aname:
                self.log(f" ■ {fn} – {n} (already flat)"))
            # Still cancel any orphaned bracket orders
            try:
                cancelled = acct.cancel_all_orders_api(account_id=aid)
                if cancelled > 0:
                    self.root.after(0, lambda fn=firm_name, n=aname, c=cancelled:
                        self.log(f" ■ {fn} – {n} cancelled {c} orphaned order(s)"))
            except Exception:
                pass
            continue
        self.root.after(0, lambda fn=firm_name, n=aname, cnt=len(open_pos):
            self.log(f" ■ {fn} – {n} closing {cnt} position(s)..."))
        result = acct.liquidate_position_api(account_id=aid)
        if result:
            closed_tv += 1
            self.root.after(0, lambda fn=firm_name, n=aname:
                self.log(f" ■ {fn} – {n} liquidated"))
        else:
            errors.append(f"{firm_name}/{aname}: liquidate returned None")
            self.root.after(0, lambda fn=firm_name, n=aname:
                self.log(f" ■ {fn} – {n} liquidation FAILED", "ERROR"))
        # Cancel remaining bracket orders (stop/limit)
        try:
            cancelled = acct.cancel_all_orders_api(account_id=aid)
            if cancelled > 0:
                self.root.after(0, lambda fn=firm_name, n=aname, c=cancelled:
                    self.log(f" ■ {fn} – {n} cancelled {c} pending order(s)"))
        except Exception:
            pass
    else:
        self.root.after(0, lambda fn=firm_name:
            self.log(f" ■ {fn} – could not fetch account list", "ERROR"))
        errors.append(f"{firm_name}: account list fetch failed")
elif hasattr(acct, 'close_all_positions'):
    # TopStepX
    acct.close_all_positions()
    closed_tv += 1
    self.root.after(0, lambda fn=firm_name:
        self.log(f" ■ {fn} positions closed"))
except Exception as e:
    errors.append(f"{firm_name}: {e}")
    self.root.after(0, lambda fn=firm_name, err=str(e):
        self.log(f" ■ {fn} close failed: {err}", "ERROR"))

# 2. Close all MT5 positions
try:
    if not MT5_AVAILABLE:
        raise RuntimeError("MetaTrader5 module is not available")
    positions = mt5.positions_get()
    if positions:

```

```

mt5_api = self._get_mt5_trading_api() if hasattr(self, '_get_mt5_trading_api') else None
for pos in positions:
    try:
        if mt5_api:
            mt5_api.close_trade(pos.ticket)
        else:
            # Direct close via MT5 API
            request = {
                "action": mt5.TRADE_ACTION_DEAL,
                "position": pos.ticket,
                "symbol": pos.symbol,
                "volume": pos.volume,
                "type": mt5.ORDER_TYPE_SELL if pos.type == 0 else mt5.ORDER_TYPE_BUY,
                "type_filling": mt5.ORDER_FILLING_IOC,
            }
            mt5.order_send(request)
        closed_mt5 += 1
        self.root.after(0, lambda t=pos.ticket, s=pos.symbol:
            self.log(f" ■ MT5 #{t} {s} closed"))
    except Exception as e:
        errors.append(f"MT5 #{pos.ticket}: {e}")
        self.root.after(0, lambda t=pos.ticket, err=str(e):
            self.log(f" ■ MT5 #{t} close failed: {err}", "ERROR"))
except Exception as e:
    errors.append(f"MT5: {e}")
    self.root.after(0, lambda err=str(e):
        self.log(f" ■ MT5 close skipped: {err}"))

# Summary
summary = f"■ Close All done – Brokers: {closed_tv}, MT5: {closed_mt5}"
if errors:
    summary += f", Errors: {len(errors)}"
self.root.after(0, lambda s=summary: self.log(s))
self.root.after(0, lambda s=summary: self._add_activity(s, "error"))

threading.Thread(target=_do_close_all, daemon=True, name="CloseAll").start()

```

Method: TradeOpssAIApp._refresh_hedge_status

```
def _refresh_hedge_status(self)
```

Periodically check Hedge Protector health and log stats.

Step by step (top-level body)

1. if not self._hedge_protector or not self._hedge_protector.is_running: branches conditionally.
2. Calls self.root.after(...) for its side effect.

Code (copy-paste safe)

```

def _refresh_hedge_status(self):
    """Periodically check Hedge Protector health and log stats."""
    if not self._hedge_protector or not self._hedge_protector.is_running:
        return
    # Schedule next check
    self.root.after(5000, self._refresh_hedge_status)

```

Method: TradeOpssAIApp._start_auto_trade

```
def _start_auto_trade(self)
```

Activate auto-trade: compute randomized start time, begin countdown.

Why these Python concepts were used

- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with `append`, and clearer about intent: this is a transform, not a side-effecting loop.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Lazy import from `datetime`.
2. `if not self._active_trade_rows`: branches conditionally.
3. Assigns `connected_firms = [f for f, c in self._broker_connections.items() if c.get(...]`
4. `if not connected_firms`: branches conditionally.
5. `if self.hedge_mode_var.get() == 'Hedging'`: branches conditionally.
6. `if self.auto_trade_signal_var.get()`: branches conditionally.
7. Assigns `EAT = timezone(timedelta(hours=3))`.
8. Assigns `now_eat = datetime.now(EAT)`.
9. Assigns `immediate = self.auto_trade_immediate_var.get()`.
10. `if immediate`: branches conditionally (with an **else/elif** arm).
11. Assigns `self._auto_trade_scheduled_dt = scheduled_eat`.
12. Assigns `self.auto_trade_enabled = True`.
13. Calls `self._auto_trade_stop.clear(...)` for its side effect.
14. Assigns `use_signal = self.auto_trade_signal_var.get()` and `SIGNALS_AVAILABLE`.
15. ... and 11 more statement(s) in the body.

Code (copy-paste safe)

```
def _start_auto_trade(self):
    """Activate auto-trade: compute randomized start time, begin countdown."""
    from datetime import datetime, timedelta, timezone

    # Validation: need trades loaded
    if not self._active_trade_rows:
        self.log("■ Load trades first before enabling auto-trade", "WARN")
        return

    # Validation: need at least one broker connected
    connected_firms = [f for f, c in self._broker_connections.items() if c.get("account")]
    if not connected_firms:
        self.log("■ Connect at least one broker before enabling auto-trade", "WARN")
        return

    # Validation: hedging mode needs MT5
```



```

if self.hedge_mode_var.get() == "Hedging":
    mt5_api = self._get_mt5_trading_api()
    if not mt5_api:
        self.log("■ Connect MT5 first for hedging mode auto-trade", "WARN")
        return

# Validation: signal mode needs MT5 for price data
if self.auto_trade_signal_var.get():
    if not SIGNALS_AVAILABLE:
        self.log("■ Signal indicators not available – install required packages", "WARN")
        return
    if not self._ensure_mt5_for_signals():
        self.log("■ Connect MT5 first for Actual Signal mode (indicators need price data)", "WARN")
        return

EAT = timezone(timedelta(hours=3)) # East Africa Time (UTC+3)
now_eat = datetime.now(EAT)

immediate = self.auto_trade_immediate_var.get()

if immediate:
    # Execute immediately (5-second grace period)
    scheduled_eat = now_eat + timedelta(seconds=5)
    offset_minutes = 0
else:
    # Base time: 2:05 AM EAT today (or tomorrow if already past ~5:05 AM)
    base = now_eat.replace(hour=2, minute=5, second=0, microsecond=0)

    # Random offset: 0 to 180 minutes (3 hours)
    offset_minutes = random.randint(0, 180)
    scheduled_eat = base + timedelta(minutes=offset_minutes)

    # If the scheduled time already passed today, schedule for tomorrow
    if scheduled_eat <= now_eat:
        scheduled_eat += timedelta(days=1)

self._auto_trade_scheduled_dt = scheduled_eat
self.auto_trade_enabled = True
self._auto_trade_stop.clear()

use_signal = self.auto_trade_signal_var.get() and SIGNALS_AVAILABLE
self._auto_trade_use_signal = use_signal

if use_signal:
    # Direction will be determined by indicator signals at execution time
    self._auto_trade_firm_sides = {} # filled at execution
    self.auto_trade_firms_var.set(" ■ Directions from indicator signals")
else:
    # Daily bias per prop firm (persisted, resets each day)
    firms_in_rows = set()
    for rd in self._active_trade_rows:
        firm_name = rd["eval"].get("Prop Firm", rd["firm_code"])
        firms_in_rows.add(firm_name)
    self._auto_trade_firm_sides = self._get_daily_bias(firms_in_rows)

    # Build display string
    dir_lines = []
    for firm, s in self._auto_trade_firm_sides.items():
        arrow = "▲" if s == "buy" else "▼"
        dir_lines.append(f" {arrow} {s.upper():4s} {firm}")

```

```

        self.auto_trade_firms_var.set("\n".join(dir_lines))

mode_label = "indicator signals" if use_signal else "random dirs per firm"
time_str = scheduled_eat.strftime("%I:%M %p EAT")
self.auto_trade_btn.configure(text="■ Stop Auto-Trade")
if CTK_AVAILABLE:
    self.auto_trade_btn.configure(fg_color='#dc2626', hover_color='#b91c1c')
if immediate:
    self.auto_trade_status_var.set(f"Executing in 5s – {mode_label}")
    self.log(f"■ Auto-trade starting immediately – {mode_label}")
else:
    self.auto_trade_status_var.set(f"Scheduled at {time_str} – {mode_label}")
    self.log(f"■ Auto-trade scheduled at {time_str} (+{offset_minutes}min random offset)")
if not use_signal:
    for firm, s in self._auto_trade_firm_sides.items():
        self.log(f"    {'▲' if s == 'buy' else '▼'} {firm} → {s.upper()}")
else:
    self.log("    ■ Directions will be generated from indicators at execution time")

# Start background countdown / executor thread
self.auto_trade_thread = threading.Thread(
    target=self._auto_trade_loop, daemon=True)
self.auto_trade_thread.start()

# Start UI countdown ticker
self._tick_auto_trade_countdown()

```

Method: TradeOpssAIApp._stop_auto_trade

```
def _stop_auto_trade(self)
    Cancel auto-trade scheduler.
```

Step by step (top-level body)

1. Assigns self.auto_trade_enabled = False.
2. Calls self._auto_trade_stop.set(...) for its side effect.
3. Assigns self._auto_trade_scheduled_dt = None.
4. Calls self.auto_trade_btn.configure(...) for its side effect.
5. if CTK_AVAILABLE: branches conditionally.
6. Calls self.auto_trade_status_var.set(...) for its side effect.
7. Calls self.auto_trade_countdown_var.set(...) for its side effect.
8. Calls self.auto_trade_firms_var.set(...) for its side effect.
9. Assigns self._auto_trade_firm_sides = {}.
10. Calls self.log(...) for its side effect.

Code (copy-paste safe)

```

def _stop_auto_trade(self):
    """Cancel auto-trade scheduler."""
    self.auto_trade_enabled = False
    self._auto_trade_stop.set()
    self._auto_trade_scheduled_dt = None
    self.auto_trade_btn.configure(text="■ Start Auto-Trade")

```

```

if CTK_AVAILABLE:
    self.auto_trade_btn.configure(fg_color=self.C_ACCENT, hover_color=self.C_ACCENT_HV)
    self.auto_trade_status_var.set("Auto-trade off")
    self.auto_trade_countdown_var.set("")
    self.auto_trade_firms_var.set("")
    self._auto_trade_firm_sides = {}
    self.log("■ Auto-trade cancelled")

```

Method: TradeOpssAIApp._tick_auto_trade_countdown

```
def _tick_auto_trade_countdown(self)
```

Update the countdown label every second.

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **if** not self.auto_trade_enabled or not self._auto_trade_scheduled_dt: branches conditionally.
2. Lazy import from datetime.
3. Assigns EAT = timezone(timedelta(hours=3)).
4. Assigns now = datetime.now(EAT).
5. Assigns remaining = self._auto_trade_scheduled_dt - now.
6. **if** remaining.total_seconds() <= 0: branches conditionally.
7. Assigns (hours, rem) = divmod(int(remaining.total_seconds()), 3600).
8. Assigns (minutes, seconds) = divmod(rem, 60).
9. Calls self.auto_trade_countdown_var.set(...) for its side effect.
10. Calls self.root.after(...) for its side effect.

Code (copy-paste safe)

```

def _tick_auto_trade_countdown(self):
    """Update the countdown label every second."""
    if not self.auto_trade_enabled or not self._auto_trade_scheduled_dt:
        return
    from datetime import datetime, timedelta, timezone
    EAT = timezone(timedelta(hours=3))
    now = datetime.now(EAT)
    remaining = self._auto_trade_scheduled_dt - now
    if remaining.total_seconds() <= 0:
        self.auto_trade_countdown_var.set("Executing now...")
        return
    hours, rem = divmod(int(remaining.total_seconds()), 3600)
    minutes, seconds = divmod(rem, 60)
    self.auto_trade_countdown_var.set(f"Starts in {hours}h {minutes}m {seconds}s")
    self.root.after(1000, self._tick_auto_trade_countdown)

```

Method: TradeOpssAIApp._auto_trade_loop

```
def _auto_trade_loop(self)
```

Background thread: wait until scheduled time, then execute all trades.

Why these Python concepts were used

- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.

Step by step (top-level body)

1. Lazy import from datetime.
2. Assigns EAT = timezone(timedelta(hours=3)).
3. **while** self.auto_trade_enabled and (not self._auto_trade_stop.is_set()): loops until the condition is false.

Code (copy-paste safe)

```
def _auto_trade_loop(self):
    """Background thread: wait until scheduled time, then execute all trades."""
    from datetime import datetime, timedelta, timezone
    EAT = timezone(timedelta(hours=3))

    while self.auto_trade_enabled and not self._auto_trade_stop.is_set():
        now = datetime.now(EAT)
        if now >= self._auto_trade_scheduled_dt:
            # Time to execute
            self.root.after(0, lambda: self.auto_trade_countdown_var.set("Executing now..."))
            self.root.after(0, self._auto_execute_all_trades)
            return
        # Sleep 1 second between checks
        self._auto_trade_stop.wait(timeout=1)
```

Method: TradeOpssAIApp._auto_execute_all_trades

```
def _auto_execute_all_trades(self)
```

Execute trades for ALL loaded rows, parallel across prop firms. Each prop firm has its own Chrome instance opened during initialization. Trades for different firms run in parallel threads (one thread per firm), while trades for the same firm run sequentially within that thread.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **list comprehension** — Builds a list in a single expression ([f(x) for x in xs]). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **dict comprehension** — Builds a dict in one expression ({k: v for k, v in items}) — same intent as a list comprehension but for mappings.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.

- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns `firm_sides = getattr(self, '_auto_trade_firm_sides', {})`.
2. Assigns `use_signal = getattr(self, '_auto_trade_use_signal', False)`.
3. Assigns `rows = list(self._active_trade_rows)`.
4. `if not rows:` branches conditionally.
5. Calls `self.log(...)` for its side effect.
6. Assigns `hedging = self.hedge_mode_var.get() == 'Hedging'`.
7. Assigns `default_platform = self.broker_var.get()`.
8. Assigns `mt5_api = self._get_mt5_trading_api()` if hedging else `None`.
9. `if hedging and (not mt5_api):` branches conditionally.
10. Assigns `rows_by_firm = defaultdict(list)`.
11. `for row_data in rows:` iterates.
12. Assigns `payout_skipped = []`.
13. `for (firm_name, firm_rows) in list(rows_by_firm.items()):` iterates.
14. `if payout_skipped:` branches conditionally.
15. ... and 8 more statement(s) in the body.

Code (copy-paste safe)

```
def _auto_execute_all_trades(self):
    """Execute trades for ALL loaded rows, parallel across prop firms.

    Each prop firm has its own Chrome instance opened during initialization.
    Trades for different firms run in parallel threads (one thread per firm),
    while trades for the same firm run sequentially within that thread.
    """
    firm_sides = getattr(self, '_auto_trade_firm_sides', {})
    use_signal = getattr(self, '_auto_trade_use_signal', False)
    rows = list(self._active_trade_rows) # snapshot

    if not rows:
```

```

        self.log("■ No trades to execute – list is empty")
        self._stop_auto_trade()
        return

self.log(f"■ Auto-executing {len(rows)} accounts (parallel per firm)...")

hedging = self.hedge_mode_var.get() == "Hedging"
default_platform = self.broker_var.get()
mt5_api = self._get_mt5_trading_api() if hedging else None
if hedging and not mt5_api:
    messagebox.showerror(
        "MT5 Not Connected",
        "Connect MT5 first, or switch to 'Broker Only' mode to skip MT5 hedging."
    )
    self.log("■ Auto-trade aborted – MT5 not connected (Hedging mode requires MT5)")
    self._stop_auto_trade()
    return

# Group rows by firm so each firm's Chrome runs in its own thread
rows_by_firm = defaultdict(list)
for row_data in rows:
    firm_name = row_data["eval"].get("Prop Firm", row_data["firm_code"])
    rows_by_firm[firm_name].append(row_data)

# ■■ PAYOUT CHECK: skip accounts with payout pending ■■
payout_skipped = []
for firm_name, firm_rows in list(rows_by_firm.items()):
    clean = []
    for rd in firm_rows:
        if self._eval_has_payout(rd.get("eval", {})):
            payout_skipped.append(rd)
            self.log(
                f"    ■ {rd['acct_num']} ({firm_name} / {rd['current_phase']}) – PAYOUT pending,
            else:
                clean.append(rd)
    rows_by_firm[firm_name] = clean
if payout_skipped:
    self.log(f"■ {len(payout_skipped)} account(s) skipped – payout pending (request payout first)

# ■■ PRE-VALIDATE: check which accounts actually exist on each Tradovate ■■
skipped_rows = []
not_connected_firms = []
if default_platform == "Tradovate":
    self.log("■ Pre-validating accounts against Tradovate dropdowns...")
    validated_by_firm = {}
    for firm_name, firm_rows in list(rows_by_firm.items()):
        broker_account = self._get_broker_for_firm(firm_name)
        if not broker_account:
            # Firm is NOT connected – skip ALL its accounts immediately
            not_connected_firms.append(firm_name)
            self.log(f"■ {firm_name} is NOT connected – skipping {len(firm_rows)} account(s)")
            for rd in firm_rows:
                skipped_rows.append(rd)
                self.log(
                    f"    ■ {rd['acct_num']} ({firm_name} / {rd['current_phase']}) – firm not co
                rows_by_firm[firm_name] = [] # clear all rows for this firm
                continue

    # Read all accounts from this firm's Tradovate dropdown
    import re as _re

```

```

try:
    trado_accounts = broker_account.get_all_accounts()
except Exception as e:
    self.log(f"■ Could not read accounts for {firm_name}: {e}")
    trado_accounts = []

if not trado_accounts:
    self.log(f"■ No accounts listed for {firm_name} – skipping pre-filter")
    continue

trado_lower = [a.lower() for a in trado_accounts]

valid_rows = []
for rd in firm_rows:
    acct = str(rd["acct_num"]).strip()
    acct_lower = acct.lower()
    digit_match = _re.search(r'(\d{5,})$', acct)
    digit_suffix = digit_match.group(1) if digit_match else None

    found = False
    for ta in trado_accounts:
        ta_lower = ta.lower()
        if acct_lower in ta_lower or ta_lower in acct_lower:
            found = True
            break
        if digit_suffix and ta.endswith(digit_suffix):
            found = True
            break

    if found:
        valid_rows.append(rd)
    else:
        skipped_rows.append(rd)

rows_by_firm[firm_name] = valid_rows
validated_by_firm[firm_name] = len(valid_rows)

# Log skipped accounts (those not found in Tradovate dropdown)
missing_only = [rd for rd in skipped_rows if rd["eval"].get("Prop Firm", rd[
    "firm_code"]) not in not_connected_firms]
if missing_only:
    self.log(f"■ {len(missing_only)} account(s) NOT found on Tradovate – skipped:")
    for rd in missing_only:
        fn = rd["eval"].get("Prop Firm", rd["firm_code"])
        self.log(f"    ■ {rd['acct_num']} ({fn} / {rd['current_phase']})")

# Mark ALL skipped rows visually (unconnected + not found)
for rd in skipped_rows:
    def _mark_skipped(rd=rd):
        try:
            rd["buy_btn"].configure(state='disabled', text="N/A")
            rd["sell_btn"].configure(state='disabled', text="N/A")
        except Exception:
            pass
    self.root.after(0, _mark_skipped)

# Remove empty firms
rows_by_firm = {f: r for f, r in rows_by_firm.items() if r}

total_valid = sum(len(r) for r in rows_by_firm.values())

```

```

self.log(f"■ {total_valid} account(s) validated, {len(skipped_rows)} skipped")

if not rows_by_firm:
    self.log("■ No valid accounts remain – aborting auto-trade")
    self._stop_auto_trade()
    return

total_success = threading.Lock()
counters = {"success": 0, "fail": 0, "skipped": len(skipped_rows)}

def _execute_firm_trades(firm_name, firm_rows):
    """Execute all trades for one firm sequentially on its own Chrome."""
    # Auto-detect platform: TopStep firms always use TopStepX (Selenium)
    if "topstep" in firm_name.lower():
        platform = "TopStepX"
    else:
        platform = default_platform
    broker_account = self._get_broker_for_firm(firm_name)
    if not broker_account:
        for rd in firm_rows:
            self.root.after(0, lambda fn=firm_name, an=rd["acct_num"]: self.log(
                f"■ No broker connected for {fn} – {an} skipped", "ERROR"))
            with total_success:
                counters["fail"] += 1
        return

    for row_data in firm_rows:
        if self._auto_trade_stop.is_set():
            break

        firm_code = row_data["firm_code"]
        phase_key = row_data["phase_key"]
        acct_size = row_data["acct_size"]
        acct_num = row_data["acct_num"]

        # ■■ Resolve phase_key from day placeholder (primary source of truth) ■■
        auto_ev = row_data.get("eval", {})
        fresh_auto_ev = self._refresh_eval_for_account(acct_num)
        if fresh_auto_ev:
            auto_ev = fresh_auto_ev
            row_data["eval"] = fresh_auto_ev
        resolved_key, day_idx, day_name = self._resolve_phase_key_from_day(
            auto_ev, firm_code, row_data.get("current_phase", ""))
        if resolved_key is None:
            # Build a diagnostic message: list any day-like values found,
            # so the user can see whether the cell really has a placeholder
            # or only a future-day prep.
            import datetime as _dtmod
            today_wd = _dtmod.date.today().weekday()
            found_days = []
            future_days = []
            for _ph, _flist in self._ALL_PHASE_FIELD_SETS:
                for _i, _f in enumerate(_flist):
                    _v = auto_ev.get(_f, None)
                    _dn2 = self._parse_day_token(_v)
                    if _dn2 is None:
                        continue
                    label = f"_{f}={str(_v).strip()}"
                    if _dn2 > today_wd:
                        future_days.append(label)

```



```

        else:
            found_days.append(label)
    if future_days and not found_days:
        diag = f"only future placeholder(s) [{', '.join(future_days[:3])}]"
    elif found_days:
        diag = (f"placeholder(s) found [{', '.join(found_days[:3])}] "
                f"but firm '{firm_code}' has no trade order for that phase")
    else:
        diag = "no day name in any Hedge Result/Hedge Day cell"
    _an, _diag = acct_num, diag
    self.root.after(0, lambda an=_an, d=_diag:
        self.log(f"■ {an}: No tradeable day placeholder – {d}", "ERROR"))
    with total_success:
        counters["fail"] += 1
    continue

if resolved_key != phase_key:
    _an, _di, _dn, _rk, _pk = acct_num, day_idx, day_name, resolved_key, phase_key
    self.root.after(0, lambda an=_an, di=_di, dn=_dn, rk=_rk, pk=_pk:
        self.log(f"■ {an}: Day cell {di + 1} ({dn}) → blueprint {rk} (was {pk})"))
    phase_key = resolved_key

# Determine direction: signal-based or random bias
if use_signal:
    # Lazy-compute signal once per firm
    if firm_name not in firm_sides:
        config_tmp = None
        if self.prop_firm_mgr:
            config_tmp = self.prop_firm_mgr.get_strategy_config(
                firm_code, phase_key, acct_size)
        mt5_sym = (config_tmp or {}).get("mt5_symbol", "NAS100")
        sig = self._get_signal_direction(mt5_sym)
        firm_sides[firm_name] = sig
        self.root.after(0, lambda fn=firm_name, s=sig, sym=mt5_sym:
            self.log(f" ■ {fn} ({sym}) → signal: {s.upper()}"))
    side = firm_sides.get(firm_name, random.choice(["buy", "sell"]))

config = None
if self.prop_firm_mgr:
    config = self.prop_firm_mgr.get_strategy_config(
        firm_code, phase_key, acct_size)
if not config:
    self.root.after(0, lambda an=acct_num, fc=firm_code, pk=phase_key, sz=acct_size: self
        log(f"■ No blueprint: {an} ({fc}/{pk}/{sz}) – skipped", "ERROR"))
    with total_success:
        counters["fail"] += 1
    continue

trado_sym = config.get("tradovate_symbol", "") or config.get("topstepx_symbol", "")
trado_qty = int(config.get("tradovate_qty", 2) or config.get("topstepx_qty", 2))

# Log resolved blueprint up-front so the user can verify the
# correct trade is being placed (catches farming/challenge mix-ups).
_bp_tp = config.get("tradovate_tp_ticks", config.get("topstepx_tp_ticks", "?"))
_bp_sl = config.get("tradovate_sl_ticks", config.get("topstepx_sl_ticks", "?"))
_an, _fc, _pk, _sz = acct_num, firm_code, phase_key, acct_size
_bs, _bq, _bt, _bsl = trado_sym, trado_qty, _bp_tp, _bp_sl
self.root.after(0, lambda an=_an, fc=_fc, pk=_pk, sz=_sz, bs=_bs, bq=_bq, bt=_bt, bsl=_bsl:
    self.log(f"■ {an}: blueprint {fc}/{pk}/{sz} → "
            f"{bs} qty={bq} TP={bt} SL={bsl}"))

# ■■ Farming: cap MT5 TP based on hard-stop proximity ■■

```

```

_is_farming_sym_auto = "MNQ" in (config.get("tradovate_symbol", "") or config.get(
    "topstepx_symbol", "")).upper()
if _is_farming_sym_auto and self.prop_firm_mgr:
    # Farming: cap MT5 TP based on hard-stop proximity
    try:
        _bal_auto = None
        if broker_account:
            _stats_auto = broker_account.get_account_stats()
            _bal_str_auto = _stats_auto.get("Balance", "")
            if _bal_str_auto and _bal_str_auto not in ("N/A", "Error", ""):
                _bal_auto = float(_bal_str_auto.replace("$", "").replace(",", ""))
        if _bal_auto is not None:
            orig_mt5_tp_a = int(config.get("mt5_tp_points", 0))
            config = self.prop_firm_mgr.adjust_farming_tp_sl(config, _bal_auto, firm_code)
            new_mt5_tp_a = int(config.get("mt5_tp_points", 0))
            if new_mt5_tp_a != orig_mt5_tp_a:
                _an, _bal_v, _omt, _nmt = acct_num, _bal_auto, orig_mt5_tp_a, new_mt5_tp_a
                self.root.after(0, lambda an=_an, bv=_bal_v, omt=_omt, nmt=_nmt:
                    self.log(
                        f"■ Farming TP cap {an}: balance=${bv:.2f} → MT5 TP {omt}→{nmt}"))
            else:
                _an = acct_num
                self.root.after(0, lambda an=_an, omt=orig_mt5_tp_a:
                    self.log(f"■ Farming TP OK {an}: MT5 TP {omt} pts within safe range"))
        else:
            _an = acct_num
            self.root.after(0, lambda an=_an:
                self.log(f"■ Farming {an}: could not read balance – using blueprint TP/SL"))
    except Exception as _fe:
        _an, _err = acct_num, str(_fe)
        self.root.after(0, lambda an=_an, err=_err:
            self.log(f"■ Farming TP check failed for {an}: {err}"))
# TP/SL comes directly from the stage blueprint (selected by day placeholder)

trado_tp = int(config.get("tradovate_tp_ticks", 151) or config.get("topstepx_tp_ticks", 1))
trado_sl = int(config.get("tradovate_sl_ticks", 200) or config.get("topstepx_sl_ticks", 2))
mt5_sym = config.get("mt5_symbol", "NAS100")
mt5_vol = float(config.get("mt5_volume", 2.8))
mt5_tp = int(config.get("mt5_tp_points", 46))
mt5_sl = int(config.get("mt5_sl_points", 42))

# ■■ Adjust TP based on stage progress ■■
if self.prop_firm_mgr and broker_account and not _is_farming_sym_auto:
    try:
        auto_profit = self._get_current_phase_profit(
            auto_ev, row_data.get("current_phase", ""),
            broker_account=broker_account, acct_size=acct_size)
        size_key_a = self.prop_firm_mgr.convert_account_size_to_key(acct_size)
        stage_start = self.prop_firm_mgr.get_stage_start_target(
            firm_code, row_data.get("current_phase", ""), phase_key, size_key_a)
        stage_profit_so_far = auto_profit - stage_start
        trado_sym_for_calc = config.get("tradovate_symbol", "") or config.get(
            "topstepx_symbol", "")
        tick_val = self.prop_firm_mgr.get_tick_value(
            trado_sym_for_calc) if self.prop_firm_mgr else 5.0
        trado_qty_for_calc = int(config.get("tradovate_qty", 1) or config.get("topstepx_qty",
            1))
        if tick_val > 0 and trado_qty_for_calc > 0:
            orig_tp_a = trado_tp
            orig_mt5_sl_a = mt5_sl

```

```

profit_ticks = stage_profit_so_far / (tick_val * trado_qty_for_calc)
adjusted_tp = max(5, round(trado_tp - profit_ticks))
tp_ratio = adjusted_tp / trado_tp if trado_tp > 0 else 1.0
adjusted_mt5_sl = max(5, round(mt5_sl * tp_ratio))
if adjusted_tp != trado_tp:
    trado_tp = adjusted_tp
    mt5_sl = adjusted_mt5_sl
    _an, _ss, _ap, _sp = acct_num, stage_start, auto_profit, stage_profit_so_far
    _otp, _ntp, _oms, _nms = orig_tp_a, trado_tp, orig_mt5_sl_a, mt5_sl
    self.root.after(0, lambda an=_an, ss=_ss, ap=_ap, sp=_sp, otp=_otp, ntp=_ntp, oms=_oms, nms=_nms:
        self.log(f"■ TP adjust {an}: stage_start=${ss:,.0f}, "
            f"P/L=${ap:,.2f}, stage P/L=${sp:+.2f} → "
            f"TP {otp}→{ntp}t, MT5 SL {oms}→{nms}pts"))
except Exception as _te:
    _an, _err = acct_num, str(_te)
    self.root.after(0, lambda an=_an, err=_err:
        self.log(f"■ TP adjust failed for {an}: {err}"))

# ■■ Adjust SL based on midnight balance + drawdown protection ■■
if broker_account and platform == "Tradovate" and hasattr(broker_account, 'get_min_equity'):
    try:
        min_eq_data = broker_account.get_min_equity()
        if min_eq_data:
            live_net_liq = min_eq_data['net_liq']
            net_liq_sod = min_eq_data.get('net_liq_sod', 0)
            live_min_equity = min_eq_data.get('min_equity', 0)
            tmdl = min_eq_data.get('trailing_max_drawdown_limit', 50000)
            trado_qty_for_calc = int(config.get("tradovate_qty", 1) or config.get(
                "topstepx_qty", 1))
            trado_sym_for_calc = config.get("tradovate_symbol", "") or config.get(
                "topstepx_symbol", "")
            tick_val = self.prop_firm_mgr.get_tick_value(
                trado_sym_for_calc) if self.prop_firm_mgr else 5.0

        # Step 1: Midnight balance SL – floor = SOD - blueprint_sl_dollars
        if net_liq_sod > 0 and tick_val > 0 and trado_qty_for_calc > 0:
            blueprint_sl_dollars = trado_sl * tick_val * trado_qty_for_calc
            sl_floor = net_liq_sod - blueprint_sl_dollars
            available = live_net_liq - sl_floor
            if available > 0:
                midnight_sl = max(10, int(available / (tick_val * trado_qty_for_calc)))
                if midnight_sl != trado_sl:
                    orig_sl = trado_sl
                    orig_mt5_tp = mt5_tp
                    trado_sl = midnight_sl
                    mt5_tp = max(5, int(trado_sl / 4) - 1)
                    _an = acct_num
                    _sod, _nl, _dpnl = net_liq_sod, live_net_liq, live_net_liq - net_liq_sod
                    _osl, _nsl, _omt, _nmt = orig_sl, trado_sl, orig_mt5_tp, mt5_tp
                    self.root.after(0, lambda an=_an, sod=_sod, nl=_nl, dp=_dpnl,
                        osl=_osl, nsl=_nsl, omt=_omt, nmt=_nmt:
                            self.log(f"■ Midnight SL {an}: SOD=${sod:,.2f}, "
                                f"live=${nl:,.2f}, daily P/L=${dp:+.2f} → "
                                f"SL {osl}→{nsl}t, MT5 TP {omt}→{nmt}pts"))
                else:
                    _an, _sod = acct_num, net_liq_sod
                    self.root.after(0, lambda an=_an, sod=_sod, sl=trado_sl:
                        self.log(
                            f"■ Midnight SL OK {an}: SOD=${sod:,.2f}, SL={sl}t unchanged"))
            else:
                self.log(f"■ Midnight SL OK {an}: SOD=${sod:,.2f}, SL={sl}t unchanged")
        else:
            self.log(f"■ Midnight SL OK {an}: SOD=${sod:,.2f}, SL={sl}t unchanged")
    except Exception as _te:
        self.log(f"■ Midnight SL OK {an}: SOD=${sod:,.2f}, SL={sl}t unchanged")

```

```

else:
    _an, _nl, _fl = acct_num, live_net_liq, sl_floor
    self.root.after(0, lambda an=_an, nl=_nl, fl=_fl:
        self.log(f"■ Midnight SL floor breached {an}: "
            f"live=${nl:,.2f} < floor=${fl:,.2f} – using min SL"))
    trado_sl = 10
    mt5_tp = max(5, int(trado_sl / 4) - 1)

# Step 2: TMDL drawdown cap – further tighten if near breach
if live_min_equity > 0 and live_net_liq < tmdl:
    drawdown_remaining = live_net_liq - live_min_equity
    current_sl_risk = trado_sl * tick_val * trado_qty_for_calc
    if drawdown_remaining > 0 and current_sl_risk > drawdown_remaining:
        orig_sl = trado_sl
        orig_mt5_tp = mt5_tp
        trado_sl = max(10, int(drawdown_remaining / (
            tick_val * trado_qty_for_calc)))
        mt5_tp = max(5, int(trado_sl / 4) - 1)
        _an, _dr = acct_num, drawdown_remaining
        _osl, _nsl, _omt, _nmt = orig_sl, trado_sl, orig_mt5_tp, mt5_tp
        self.root.after(0, lambda an=_an, dr=_dr, osl=_osl, nsl=_nsl, omt=_omt,
            nmt=_nmt:
            self.log(f"■ TMDL SL cap {an}: remaining=${dr:,.2f} → "
                f"SL {osl}→{nsl}t, MT5 TP {omt}→{nmt}pts"))
    elif drawdown_remaining <= 0:
        _an, _dr = acct_num, drawdown_remaining
        self.root.after(0, lambda an=_an, dr=_dr:
            self.log(f"■ No drawdown room for {an} (${dr:,.2f})"))
    elif live_min_equity > 0:
        _an, _nl, _tmdl = acct_num, live_net_liq, tmdl
        self.root.after(0, lambda an=_an, nl=_nl, t=_tmdl:
            self.log(f"■ TMDL OK {an}: ${nl:,.2f} ≥ TMDL=${t:,.0f}"))
except Exception:
    pass

try:
    # 1. Broker order – uses this firm's own Chrome instance
    if platform == "Tradovate":
        if side == "buy":
            order_result = broker_account.buy_market(trado_sym, trado_qty, tp=trado_tp,
                sl=trado_sl, expected_account=acct_num)
        else:
            order_result = broker_account.sell_market(trado_sym, trado_qty, tp=trado_tp,
                sl=trado_sl, expected_account=acct_num)
    elif platform == "TopStepX":
        # Ensure we are on the intended sub-account before placing the order.
        if hasattr(broker_account, 'switch_account') and acct_num:
            switched = broker_account.switch_account(account_name_contains=acct_num)
            if not switched:
                raise Exception(f"TopStepX could not switch to account {acct_num}")

        # TopStepX uses two-digit year futures codes (NQ26, MNQM26)
        _tsx_sym = _to_topstepx_symbol(trado_sym)
        _tsx_tick_val = self.prop_firm_mgr.get_tick_value(
            _tsx_sym) if self.prop_firm_mgr else 0.5
        _tsx_tp_dollars = trado_tp * _tsx_tick_val * trado_qty
        _tsx_sl_dollars = trado_sl * _tsx_tick_val * trado_qty
        if side == "buy":
            order_result = broker_account.place_buy_order(
                _tsx_sym,

```

```

        trado_qty,
        tp_dollars=_tsx_tp_dollars,
        sl_dollars=_tsx_sl_dollars,
    )
else:
    order_result = broker_account.place_sell_order(
        _tsx_sym,
        trado_qty,
        tp_dollars=_tsx_tp_dollars,
        sl_dollars=_tsx_sl_dollars,
    )

# TopStepX returns status dicts on both success and failure.
# Convert broker-declared failures to exceptions so counters/logs are accurate.
if isinstance(order_result, dict) and order_result.get("success") is False:
    raise Exception(order_result.get("message") or "Broker reported unsuccessful order")

self.root.after(0, lambda an=acct_num, fc=firm_code, sd=side, sym=trado_sym,
               qty=trado_qty:
    self.log(f"■ {platform} {sd.upper()} {qty} {sym} → {an} ({fc}"))

# 2. MT5 hedge (opposite direction)
if hedging and mt5_api:
    hedge_side = "sell" if side == "buy" else "buy"
    comment = f"{acct_num}_{phase_key}"
    if hedge_side == "buy":
        mt5_api.buy_market(mt5_sym, mt5_vol, sl=mt5_sl, tp=mt5_tp, comment=comment)
    else:
        mt5_api.sell_market(mt5_sym, mt5_vol, sl=mt5_sl, tp=mt5_tp, comment=comment)
    self.root.after(0, lambda an=acct_num, hs=hedge_side, vol=mt5_vol, sym=mt5_sym,
                   cmt=comment:
        self.log(f"■ MT5 hedge {hs.upper()} {vol} {sym} comment:{cmt} → {an}"))

with total_success:
    counters["success"] += 1

# ■■ Auto-status: set "In Progress" when trades go out ■■
if not RELEASE_DISABLE_AUTO_STATUS_UPDATES:
    _ev = row_data.get("eval")
    if _ev:
        _has_funded = bool((row_data.get("eval", {}).get("Account #.1") or "").strip())
        _sf = "Status" if _has_funded else "Status P1"
        _cur = (_ev.get(_sf) or "").strip().lower()
        # Only set In Progress if status is empty, Not Started, or already In Progress
        if not _cur or _cur in ("not started", "in progress", ""):
            _ev[_sf] = "In Progress"
            self.root.after(0, lambda an=acct_num, sf=_sf:
                self.log(f"■ Auto-status: {an} → {_sf}='In Progress'"))

# Remove row from UI
def _remove(rd=row_data):
    rd["frame"].destroy()
    if rd in self._active_trade_rows:
        self._active_trade_rows.remove(rd)
    remaining = len(self._active_trade_rows)
    self.trades_count_var.set(
        f"{remaining} active trade{'s' if remaining != 1 else ''}"
        if remaining > 0 else "All trades complete ✓")
self.root.after(0, _remove)

# Small delay between trades on the same account

```

```

        time.sleep(2)

    except Exception as e:
        with total_success:
            counters["fail"] += 1
        self.root.after(0, lambda an=acct_num, err=str(e):
            self.log(f"■ Auto-trade failed for {an}: {err}", "ERROR"))

def _dispatch_parallel():
    num_firms = len(rows_by_firm)
    self.root.after(0, lambda n=num_firms: self.log(
        f"■ Dispatching trades across {n} firm(s) in parallel..."))
    with ThreadPoolExecutor(max_workers=num_firms) as executor:
        futures = {
            executor.submit(_execute_firm_trades, firm, firm_rows): firm
            for firm, firm_rows in rows_by_firm.items()
        }
        for future in as_completed(futures):
            firm = futures[future]
            try:
                future.result()
            except Exception as e:
                self.root.after(0, lambda fn=firm, err=str(e):
                    self.log(f"■ Firm thread {fn} crashed: {err}", "ERROR"))

    # Final summary
    self.root.after(0, lambda s=counters["success"], f=counters["fail"], sk=counters["skipped"]:
        self.log(
            f"■ Auto-trade complete: {s} succeeded, {f} failed, {sk} skipped (not on Tradovate)"
        ))
    self.root.after(0, self._stop_auto_trade)

threading.Thread(target=_dispatch_parallel, daemon=True).start()

```

Method: TradeOpssAIApp._on_prop_firm_change

```
def _on_prop_firm_change(self, event=None)

    Update phase and size options when prop firm changes (compat stub).
```

Step by step (top-level body)

1. `pass` (placeholder).

Code (copy-paste safe)

```
def _on_prop_firm_change(self, event=None):
    """Update phase and size options when prop firm changes (compat stub)."""
    pass

```

Method: TradeOpssAIApp._update_account_sizes

```
def _update_account_sizes(self)

    Update account size (compat stub).
```

Step by step (top-level body)

1. `pass` (placeholder).

Code (copy-paste safe)

```
def _update_account_sizes(self):
    """Update account size (compat stub)."""
    pass
```

Method: TradeOpssAIApp._populate_broker_rows

```
def _populate_broker_rows(self, evaluations, prop_accounts)

    Build one connection row per unique prop firm from active evaluations.
```

Why these Python concepts were used

- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **for** child in self._broker_rows_frame.wininfo_children(): iterates.
2. Assigns old_connections = dict(self._broker_connections).
3. Calls self._broker_connections.clear(...) for its side effect.
4. Assigns firms = [].
5. Assigns seen = set().
6. **for** ev in evaluations: iterates.
7. **if** not firms: branches conditionally.
8. **if** CTK_AVAILABLE: branches conditionally.
9. Assigns pa_lookup = {}.
10. Assigns pa_unmatched = [].
11. **for** pa in prop_accounts or []: iterates.
12. **if** prop_accounts: branches conditionally.
13. Assigns auto_count = 0.
14. **for** firm in firms: iterates.
15. ... and 1 more statement(s) in the body.

Code (copy-paste safe)

```
def _populate_broker_rows(self, evaluations, prop_accounts):
    """Build one connection row per unique prop firm from active evaluations."""
    for child in self._broker_rows_frame.wininfo_children():
        child.destroy()
    # Preserve any already-connected accounts
    old_connections = dict(self._broker_connections)
    self._broker_connections.clear()

    # Get unique prop firms in order
```

```

firms = []
seen = set()
for ev in evaluations:
    firm = ev.get("Prop Firm", "Unknown")
    if firm not in seen:
        seen.add(firm)
        firms.append(firm)

if not firms:
    if CTK_AVAILABLE:
        ctk.CTkLabel(self._broker_rows_frame,
                      text="No active prop firms – load trades first",
                      font=("Segoe UI", 9, "italic"), text_color="#4A5568").pack(pady=4)
    return

# Header row
if CTK_AVAILABLE:
    hdr = ctk.CTkFrame(self._broker_rows_frame, fg_color="transparent")
    hdr.pack(fill="x", padx=4, pady=(2, 0))
    ctk.CTkLabel(hdr, text="PROP FIRM", width=110, font=("Consolas", 8, "bold"),
                  text_color="#4A5568", anchor="w").pack(side="left", padx=(12, 0))
    ctk.CTkLabel(hdr, text="USERNAME", width=140, font=("Consolas", 8, "bold"),
                  text_color="#4A5568", anchor="w").pack(side="left", padx=(8, 0))
    ctk.CTkLabel(hdr, text="PASSWORD", width=110, font=("Consolas", 8, "bold"),
                  text_color="#4A5568", anchor="w").pack(side="left", padx=(8, 0))

# Build lookup from prop_accounts by prop_firm name
pa_lookup = {}
pa_unmatched = [] # accounts with creds but no firm match yet
for pa in (prop_accounts or []):
    pf = (pa.get("prop_firm") or "").strip()
    has_creds = bool((pa.get("tradovate_username") or "").strip() and
                     (pa.get("tradovate_password") or "").strip()))
    if pf and pf not in pa_lookup:
        pa_lookup[pf] = pa
    elif has_creds and not pf:
        pa_unmatched.append(pa)

# Log what we received for debugging
if prop_accounts:
    self.log(f"Dashboard prop_accounts: {len(prop_accounts)} entries, "
            f"matched firms: {list(pa_lookup.keys())}")

auto_count = 0
for firm in firms:
    strip_color = self.PROP_FIRM_COLORS.get(firm, "#95A5A6")
    # Try exact match first, then case-insensitive, then alias match, then unmatched pool
    pa = pa_lookup.get(firm, {})
    if not pa:
        for pf_key, pf_val in pa_lookup.items():
            if pf_key.lower() == firm.lower():
                pa = pf_val
                break
    if not pa:
        # Alias matching: firm name variants that refer to the same firm
        _FIRM_ALIASES = {
            "funded next": ["fundednext"],
            "fundednext": ["funded next"],
            "my funded futures": ["mffu"],
            "mffu": ["my funded futures"],

```



```

        "lucid": ["lucid trading"],
        "lucid trading": ["lucid"],
    }
    firm_lower = firm.lower()
    aliases = _FIRM_ALIASES.get(firm_lower, [])
    for pf_key, pf_val in pa_lookup.items():
        if pf_key.lower() in aliases:
            pa = pf_val
            break
    if not pa and pa_unmatched:
        pa = pa_unmatched.pop(0)
    pre_user = (pa.get("tradovate_username") or "").strip()
    pre_pass = (pa.get("tradovate_password") or "").strip()

    # Carry over existing connected account if available
    old_conn = old_connections.get(firm, {})
    existing_account = old_conn.get("account")

    if CTK_AVAILABLE:
        row = ctk.CTkFrame(self._broker_rows_frame, fg_color="#0A1220",
                           corner_radius=4, height=36,
                           border_width=1, border_color="#0F1A2A")
        row.pack(fill="x", padx=4, pady=1)
        row.pack_propagate(False)

        # Accent bar
        ctk.CTkFrame(row, width=3, fg_color=strip_color,
                     corner_radius=0).pack(side="left", fill="y")

        # Firm name
        ctk.CTkLabel(row, text=firm[:16], width=110,
                     font=("Consolas", 10, "bold"), text_color=strip_color,
                     anchor="w").pack(side="left", padx=(8, 0))

        # User entry
        user_entry = ctk.CTkEntry(row, width=140, height=26,
                                   fg_color=self.C_BG_THIRD, border_color=self.C_BORDER,
                                   text_color=self.C_TEXT, font=("Consolas", 10))
        user_entry.pack(side="left", padx=(8, 0))
        if pre_user:
            user_entry.insert(0, pre_user)

        # Pass entry
        pass_entry = ctk.CTkEntry(row, width=110, height=26, show="*",
                                   fg_color=self.C_BG_THIRD, border_color=self.C_BORDER,
                                   text_color=self.C_TEXT, font=("Consolas", 10))
        pass_entry.pack(side="left", padx=(8, 0))
        if pre_pass:
            pass_entry.insert(0, pre_pass)

        # Status indicator
        status_var = tk.StringVar(value="■" if existing_account else "■")
        status_lbl = ctk.CTkLabel(row, textvariable=status_var,
                                   font=("Segoe UI", 11), width=24,
                                   text_color="#22C55E" if existing_account else "#4A5568")
        status_lbl.pack(side="right", padx=(0, 8))

        # Connect button
        btn_text = "Connected" if existing_account else "Connect"
        conn_btn = ctk.CTkButton(row, text=btn_text, width=72, height=24,

```

```

        fg_color="#14532D" if existing_account else "#1A2332",
        hover_color="#24292F",
        border_width=1, border_color=self.C_BORDER,
        font=("Consolas", 9), text_color=self.C_TEXT,
        corner_radius=4,
        command=lambda f=firm: self._connect_broker_firm(f))
conn_btn.pack(side="right", padx=(8, 4))

# History button – shows full Tradovate trade history for this account
hist_btn = ctk.CTkButton(row, text="■ History", width=78, height=24,
        fg_color="#1A2332", hover_color="#2A3342",
        border_width=1, border_color="#3B4B5E",
        font=("Consolas", 9), text_color="#60A5FA",
        corner_radius=4,
        command=lambda f=firm: self._show_trade_history(f))
hist_btn.pack(side="right", padx=(4, 0))

# Dashboard button – show for all firms (use _BROWSER_MONITORED_FIRMS with case-insensitive)
_dash_cfg = self._BROWSER_MONITORED_FIRMS.get(firm)
if not _dash_cfg:
    # Case-insensitive lookup
    for _bk, _bv in self._BROWSER_MONITORED_FIRMS.items():
        if _bk.lower() == firm.lower():
            _dash_cfg = _bv
            break
if _dash_cfg and not RELEASE_DISABLE_PROP_DASHBOARD_ACCESS:
    dash_btn = ctk.CTkButton(row, text="■ Dashboard", width=90, height=24,
        fg_color="#1A1A3E", hover_color="#2A2A5E",
        border_width=1, border_color="#3B3B6E",
        font=("Consolas", 9), text_color="#A78BFA",
        corner_radius=4,
        command=lambda f=firm: self._launch_propfirm_dashboard(f))
    dash_btn.pack(side="right", padx=(4, 0))
else:
    row = tk.Frame(self._broker_rows_frame, bg="#0A1220")
    row.pack(fill="x", padx=4, pady=1)
    tk.Label(row, text=firm[:16], width=14, anchor='w',
        bg="#0A1220", fg=strip_color, font=('Consolas', 9)).pack(side="left", padx=2)
    user_entry = ttk.Entry(row, width=16)
    user_entry.pack(side="left", padx=2)
    if pre_user:
        user_entry.insert(0, pre_user)
    pass_entry = ttk.Entry(row, width=12, show="*")
    pass_entry.pack(side="left", padx=2)
    if pre_pass:
        pass_entry.insert(0, pre_pass)
    conn_btn = ttk.Button(row, text="Connect",
        command=lambda f=firm: self._connect_broker_firm(f))
    conn_btn.pack(side="left", padx=2)
    status_var = tk.StringVar(value="■" if existing_account else "■")
    status_lbl = ttk.Label(row, textvariable=status_var)
    status_lbl.pack(side="left", padx=2)

if pre_user and pre_pass:
    auto_count += 1

self._broker_connections[firm] = {
    "user_entry": user_entry,
    "pass_entry": pass_entry,
    "connect_btn": conn_btn,

```

```

        "status_var": status_var,
        "status_lbl": status_lbl,
        "row_frame": row,
        "account": existing_account,
    }

    if auto_count:
        self.log(f"Broker credentials found for {auto_count} prop firm(s) – auto-connecting...")
        # Auto-connect all firms that have credentials from dashboard
        self.root.after(500, self._auto_connect_populated_brokers)

```

Method: TradeOpssAIApp._connect_broker_firm

```
def _connect_broker_firm(self, firm_name)
```

Connect a single prop firm's broker account.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.
- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.

Step by step (top-level body)

1. Assigns conn = self._broker_connections.get(firm_name).
2. **if** not conn: branches conditionally.
3. Assigns firm_lower = firm_name.lower().
4. **if** 'topstep' in firm_lower: branches conditionally (with an **else/elif** arm).
5. Assigns user = conn['user_entry'].get().strip().
6. Assigns pwd = conn['pass_entry'].get().strip().
7. Assigns mode = self.trading_mode_var.get().
8. **if** not user or not pwd: branches conditionally.
9. Calls self.log(...) for its side effect.
10. Calls conn['status_var'].set(...) for its side effect.
11. Calls conn['connect_btn'].configure(...) for its side effect.
12. Defines a nested function _do_connect(...).
13. Calls threading.Thread(target=_do_connect, daemon=True).start(...) for its side effect.

Code (copy-paste safe)

```
def _connect_broker_firm(self, firm_name):
    """Connect a single prop firm's broker account."""
    conn = self._broker_connections.get(firm_name)
    if not conn:
        return

    # Auto-detect platform: TopStep firms always use TopStepX (Selenium)
    firm_lower = firm_name.lower()
    if "topstep" in firm_lower:
        platform = "TopStepX"
    else:
        platform = self.broker_var.get()

    user = conn["user_entry"].get().strip()
    pwd = conn["pass_entry"].get().strip()
    mode = self.trading_mode_var.get()

    if not user or not pwd:
        messagebox.showerror("Error", f"Enter username and password for {firm_name}")
        return

    self.log(f"Connecting {firm_name} to {platform}...")
    conn["status_var"].set("■")
    conn["connect_btn"].configure(text="...")

    def _do_connect():
        try:
            if platform == "Tradovate":
                if not TRADOVATE_AVAILABLE:
                    err = _TRADOVATE_IMPORT_ERROR or 'unknown reason'
                    self.root.after(0, lambda: conn["status_var"].set("■"))
                    self.log(f"Tradovate import failed: {err}", "ERROR")
                    self.root.after(0, lambda: conn["connect_btn"].configure(text="Connect"))
                    return
                account = TradovateAccount(user, pwd, trading_mode=mode)
                account.login()
            elif platform == "TopStepX":
                if not TOPSTEPX_AVAILABLE:
                    err = _TOPSTEPX_IMPORT_ERROR or 'unknown reason'
                    self.root.after(0, lambda: conn["status_var"].set("■"))
                    self.log(f"TopStepX import failed: {err}", "ERROR")
                    self.root.after(0, lambda: conn["connect_btn"].configure(text="Connect"))
                    return
                account = TopStepXAccount(user, pwd)
                account.login()
            else:
                self.root.after(0, lambda: conn["status_var"].set("■"))
                self.log(f"Unknown platform: {platform}", "ERROR")
                self.root.after(0, lambda: conn["connect_btn"].configure(text="Connect"))
                return

            conn["account"] = account
            # Also keep legacy references for backward compatibility
            if platform == "Tradovate":
                self.tradovate_account = account
            else:
                self.topstepx_account = account
```

```

def _update_ui():
    conn["status_var"].set("■")
    conn["connect_btn"].configure(text="Connected", fg_color="#14532D")
    if hasattr(conn.get("status_lbl"), "configure"):
        conn["status_lbl"].configure(text_color="#22C55E")
    # Update global status
    connected = sum(1 for c in self._broker_connections.values() if c.get("account"))
    total = len(self._broker_connections)
    self.broker_status_var.set(f"{connected}/{total} connected")
    self.root.after(0, _update_ui)
    self.log(f"■ {firm_name} connected to {platform} ({mode})")
    # Release build: no status polling / hedge guard side-effects
    if not RELEASE_DISABLE_STATUS_POLL:
        self.root.after(0, self._start_status_polling)
    if not RELEASE_DISABLE_HEDGE_GUARD:
        self.root.after(500, self._start_hedge_protector)

except Exception as e:
    def _fail():
        conn["status_var"].set("■")
        conn["connect_btn"].configure(text="Retry", fg_color="#450A0A")
    self.root.after(0, _fail)
    self.log(f"■ {firm_name} connection failed: {e}", "ERROR")

threading.Thread(target=_do_connect, daemon=True).start()

```

Method: TradeOpssAIApp._connect_all_brokers

```
def _connect_all_brokers(self)
```

Connect all prop firms that have credentials filled in.

Why these Python concepts were used

- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.

Step by step (top-level body)

1. **if** not self._broker_connections: branches conditionally.
2. Assigns to _connect = [].
3. **for** (firm, conn) in self._broker_connections.items(): iterates.
4. **if** not to_connect: branches conditionally.
5. Calls self.log(...) for its side effect.
6. Defines a nested function _do_connect_all(...).
7. Calls threading.Thread(target=_do_connect_all, daemon=True).start(...) for its side effect.

Code (copy-paste safe)

```
def _connect_all_brokers(self):
```

```

"""Connect all prop firms that have credentials filled in."""
if not self._broker_connections:
    self.log("■ Load trades first to see prop firm connections", "WARN")
    return

to_connect = []
for firm, conn in self._broker_connections.items():
    user = conn["user_entry"].get().strip()
    pwd = conn["pass_entry"].get().strip()
    if user and pwd and not conn.get("account"):
        to_connect.append(firm)

if not to_connect:
    self.log("All populated firms are already connected")
    return

self.log(f"Connecting {len(to_connect)} broker(s)...")

def _do_connect_all():
    for firm in to_connect:
        self.root.after(0, lambda f=firm: self._connect_broker_firm(f))
        time.sleep(3) # stagger connections

threading.Thread(target=_do_connect_all, daemon=True).start()

```

Method: TradeOpssAIApp._auto_connect_populated_brokers

```
def _auto_connect_populated_brokers(self)
```

Auto-connect all broker rows that have dashboard credentials pre-filled.

Why these Python concepts were used

- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.

Step by step (top-level body)

1. Assigns to_connect = [].
2. for (firm, conn) in self._broker_connections.items(): iterates.
3. if not to_connect: branches conditionally.
4. Calls self.log(...) for its side effect.
5. Defines a nested function _do_auto(...).
6. Calls threading.Thread(target=_do_auto, daemon=True).start(...) for its side effect.

Code (copy-paste safe)

```

def _auto_connect_populated_brokers(self):
    """Auto-connect all broker rows that have dashboard credentials pre-filled."""
    to_connect = []
    for firm, conn in self._broker_connections.items():

```

```

        user = conn["user_entry"].get().strip()
        pwd = conn["pass_entry"].get().strip()
        if user and pwd and not conn.get("account"):
            to_connect.append(firm)

    if not to_connect:
        return

    self.log(f"■ Auto-connecting {len(to_connect)} broker(s) from dashboard credentials...")

    def _do_auto():
        for firm in to_connect:
            self.root.after(0, lambda f=firm: self._connect_broker_firm(f))
            time.sleep(3) # stagger to avoid overwhelming

    threading.Thread(target=_do_auto, daemon=True).start()

```

Method: TradeOpssAIApp._show_trade_history

```
def _show_trade_history(self, firm_name)
```

Show a popup window with full Tradovate trade history for the given firm.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.

Step by step (top-level body)

1. Assigns `conn = self._broker_connections.get(firm_name)`.
2. `if not conn or not conn.get('account')`: branches conditionally.
3. Assigns `account = conn['account']`.
4. `if not hasattr(account, 'get_trade_history')`: branches conditionally.
5. Calls `self.log(...)` for its side effect.
6. Defines a nested function `_fetch_and_show(...)`.
7. Calls `threading.Thread(target=_fetch_and_show, daemon=True).start(...)` for its side effect.

Code (copy-paste safe)

```

def _show_trade_history(self, firm_name):
    """Show a popup window with full Tradovate trade history for the given firm."""
    conn = self._broker_connections.get(firm_name)
    if not conn or not conn.get("account"):
        self.log(f"■ {firm_name} is not connected – connect first", "WARN")
    return

```

```

account = conn["account"]
if not hasattr(account, 'get_trade_history'):
    self.log(f"■ {firm_name} account does not support trade history", "WARN")
    return

self.log(f"■ Loading trade history for {firm_name}...")

def _fetch_and_show():
    try:
        data = account.get_trade_history()
        if not data:
            self.root.after(0, lambda: self.log(f"■ No history data for {firm_name}", "WARN"))
            return
        self.root.after(0, lambda d=data: self._build_history_window(firm_name, d))
    except Exception as e:
        self.root.after(0, lambda: self.log(f"■ History fetch failed: {e}", "ERROR"))

threading.Thread(target=_fetch_and_show, daemon=True).start()

```

Method: TradeOpssAIApp._build_history_window

```
def _build_history_window(self, firm_name, data)
```

Build and display the trade history popup window. data is a list of account dicts (one per account under this login).

Why these Python concepts were used

- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.
- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. `if isinstance(data, dict):` branches conditionally.
2. Assigns `win = tk.Toplevel(self.root)`.
3. Assigns `acct_count = len(data)`.
4. Calls `win.title(...)` for its side effect.
5. Calls `win.geometry(...)` for its side effect.
6. Calls `win.configure(...)` for its side effect.

7. Calls `win.attributes(...)` for its side effect.
8. Calls `win.after(...)` for its side effect.
9. **if** `acct_count > 1`: branches conditionally.
10. Assigns `content = tk.Frame(win, bg='#0A0E17')`.
11. Calls `content.pack(...)` for its side effect.
12. Defines a nested function `_show_account(...)`.
13. **if** `acct_count > 1`: branches conditionally.
14. Calls `_show_account(...)` for its side effect.
15. ... and 3 more statement(s) in the body.

Code (copy-paste safe)

```
def _build_history_window(self, firm_name, data):
    """Build and display the trade history popup window.
    data is a list of account dicts (one per account under this login)."""
    if isinstance(data, dict):
        data = [data] # backwards compat – single account

    win = tk.Toplevel(self.root)
    acct_count = len(data)
    win.title(f"■ Trade History – {firm_name} ({acct_count} account{'s' if acct_count > 1 else ''})")
    win.geometry("800x660")
    win.configure(bg="#0A0E17")
    win.attributes("-topmost", True)
    win.after(500, lambda: win.attributes("-topmost", False))

    # ■■ Account selector tabs (if multiple accounts) ■■■■■■
    if acct_count > 1:
        tab_frame = tk.Frame(win, bg="#0A0E17")
        tab_frame.pack(fill="x", padx=8, pady=(6, 0))
        tk.Label(tab_frame, text=f"{acct_count} accounts found",
                 bg="#0A0E17", fg="#64748B", font=("Consolas", 9)).pack(side="left", padx=(4, 12))

    # Content container – will be cleared/rebuilt on account switch
    content = tk.Frame(win, bg="#0A0E17")
    content.pack(fill="both", expand=True)

    def _show_account(acct_data):
        # Clear existing content
        for w in content.wininfo_children():
            w.destroy()
        self._render_account_history(content, acct_data)

    # Build tab buttons (if multiple)
    if acct_count > 1:
        for i, acct in enumerate(data):
            aname = acct.get('account_name', f'Account {i+1}')
            bal = acct.get('balance', 0)
            btn_text = f"{aname} ${bal:,.0f}"
            btn = tk.Button(tab_frame, text=btn_text, bg="#1A2332", fg="#60A5FA",
                           activebackground="#2A3342", activeforeground="#93C5FD",
                           font=("Consolas", 9), bd=0, padx=8, pady=2,
                           command=lambda d=acct: _show_account(d))
            btn.pack(side="left", padx=2)
```

```
# Show first account by default
_show_account(data[0])

total_fills = sum(len(a.get('fills', [])) for a in data)
total_days = sum(len(a.get('daily_pnl', [])) for a in data)
self.log(
    f"■ Trade history loaded for {firm_name}: {acct_count} account(s), {total_days} days, {total..."
```

Method: TradeOpssAIApp._render_account_history

```
def _render_account_history(self, parent, data)

    Render a single account's history into the given parent frame.
```

Why these Python concepts were used

- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns acct_name = data.get('account_name', '?').
2. Assigns env = data.get('environment', '?').upper().
3. Assigns balance = data.get('balance', 0).
4. Assigns balance_sod = data.get('balance_sod', 0).
5. Assigns realized = data.get('realized_pnl', 0).
6. Assigns open_pnl = data.get('open_pnl', 0).
7. Assigns dd_max = data.get('trailing_max_drawdown', 0).
8. Assigns dd_limit = data.get('trailing_max_drawdown_limit', 0).
9. Assigns dd_mode = data.get('drawdown_mode', '?').
10. Assigns hdr = tk.Frame(parent, bg='#0F1520', height=80).
11. Calls hdr.pack(...) for its side effect.
12. Calls hdr.pack_propagate(...) for its side effect.
13. Calls tk.Label(hdr, text=f'{acct_name} [{env}]', bg='#0F1520', fg='#E2E8F0', font=('Consolas', 12, 'bold'), anchor='w').place(...) for its side effect.
14. Assigns bal_color = '#22C55E' if realized >= 0 else '#EF4444'.
15. ... and 33 more statement(s) in the body.

Code (copy-paste safe)

```
def _render_account_history(self, parent, data):
```

MT5HedgingEngine — Reading Python Through a Real Codebase

```
hdr_row = tk.Frame(scroll_frame, bg="#151D2B")
hdr_row.pack(fill="x", pady=(0, 1))
for col_name, col_w in cols:
    tk.Label(hdr_row, text=col_name, width=col_w // 8, bg="#151D2B", fg="#64748B",
             font=("Consolas", 9, "bold"), anchor="center").pack(side="left", padx=1)

# Table rows
daily_pnl = data.get('daily_pnl', [])
total_gross = total_fees = total_net = 0
for i, day in enumerate(daily_pnl):
    bg = "#0D1320" if i % 2 == 0 else "#0A0E17"
    r = tk.Frame(scroll_frame, bg=bg)
    r.pack(fill="x")

    date_str = day['date']
    trades = day['trades']
    gross = day['gross_pnl']
    fees = day['fees']
    net = day['net_pnl']
    bal = day['balance_eod']

    total_gross += gross
    total_fees += fees
    total_net += net

    net_color = "#22C55E" if net > 0 else "#EF4444" if net < 0 else "#4A5568"
    gross_color = "#22C55E" if gross > 0 else "#EF4444" if gross < 0 else "#4A5568"

    tk.Label(r, text=date_str, width=11, bg=bg, fg="#CBD5E1",
            font=("Consolas", 9), anchor="center").pack(side="left", padx=1)
    tk.Label(r, text=str(trades) if trades else "--", width=6, bg=bg,
            fg="#CBD5E1" if trades else "#334155",
            font=("Consolas", 9), anchor="center").pack(side="left", padx=1)
    tk.Label(r, text=f"${gross:+,.2f}" if gross else "--", width=11, bg=bg,
            fg=gross_color, font=("Consolas", 9), anchor="center").pack(side="left", padx=1)
    tk.Label(r, text=f"${fees:+,.2f}" if fees else "--", width=10, bg=bg,
            fg="#94A3B8" if fees else "#334155",
            font=("Consolas", 9), anchor="center").pack(side="left", padx=1)
    tk.Label(r, text=f"${net:+,.2f}" if net else "--", width=11, bg=bg,
            fg=net_color, font=("Consolas", 9, "bold"), anchor="center").pack(side="left", padx=1)
    tk.Label(r, text=f"${bal:+,.2f}", width=12, bg=bg, fg="#E2E8F0",
            font=("Consolas", 9), anchor="center").pack(side="left", padx=1)

# Totals row
tot_bg = "#1A2332"
tot_row = tk.Frame(scroll_frame, bg=tot_bg)
tot_row.pack(fill="x", pady=(2, 0))
tk.Label(tot_row, text="TOTAL", width=11, bg=tot_bg, fg="#E2E8F0",
        font=("Consolas", 9, "bold"), anchor="center").pack(side="left", padx=1)
tk.Label(tot_row, text="", width=6, bg=tot_bg).pack(side="left", padx=1)
tk.Label(tot_row, text=f"${total_gross:+,.2f}", width=11, bg=tot_bg,
        fg="#22C55E" if total_gross >= 0 else "#EF4444",
        font=("Consolas", 9, "bold"), anchor="center").pack(side="left", padx=1)
tk.Label(tot_row, text=f"${total_fees:+,.2f}", width=10, bg=tot_bg, fg="#94A3B8",
        font=("Consolas", 9, "bold"), anchor="center").pack(side="left", padx=1)
tk.Label(tot_row, text=f"${total_net:+,.2f}", width=11, bg=tot_bg,
        fg="#22C55E" if total_net >= 0 else "#EF4444",
        font=("Consolas", 9, "bold"), anchor="center").pack(side="left", padx=1)
tk.Label(tot_row, text="", width=12, bg=tot_bg).pack(side="left", padx=1)
```

```

fills = data.get('fills', [])
if fills:
    tk.Label(parent, text=f"Trade Fills ({len(fills)}", bg="#0A0E17", fg="#60A5FA",
              font=("Consolas", 10, "bold"), anchor="w").pack(fill="x", padx=12, pady=(8, 0))

    fills_frame = tk.Frame(parent, bg="#0A0E17")
    fills_frame.pack(fill="both", expand=True, padx=8, pady=(2, 8))

    f_canvas = tk.Canvas(fills_frame, bg="#0A0E17", highlightthickness=0, height=150)
    f_scroll = tk.Scrollbar(fills_frame, orient="vertical", command=f_canvas.yview)
    f_inner = tk.Frame(f_canvas, bg="#0A0E17")
    f_inner.bind("<Configure>", lambda e: f_canvas.configure(scrollregion=f_canvas.bbox("all")))
    f_canvas.create_window((0, 0), window=f_inner, anchor="nw")
    f_canvas.configure(yscrollcommand=f_scroll.set)
    f_canvas.pack(side="left", fill="both", expand=True)
    f_scroll.pack(side="right", fill="y")

    # Fills header
    fhdr = tk.Frame(f_inner, bg="#151D2B")
    fhdr.pack(fill="x", pady=(0, 1))
    for col_name, col_w in [("Date", 80), ("Time", 65), ("Side", 40),
                           ("Qty", 35), ("Contract", 70), ("Price", 80)]:
        tk.Label(fhdr, text=col_name, width=col_w // 7, bg="#151D2B", fg="#64748B",
                  font=("Consolas", 9, "bold"), anchor="center").pack(side="left", padx=1)

    for i, fill in enumerate(fills):
        bg = "#0D1320" if i % 2 == 0 else "#0A0E17"
        fr = tk.Frame(f_inner, bg=bg)
        fr.pack(fill="x")
        action_color = "#22C55E" if fill['action'] == 'Buy' else "#EF4444"
        tk.Label(fr, text=fill['date'], width=11, bg=bg, fg="#CBD5E1",
                  font=("Consolas", 9), anchor="center").pack(side="left", padx=1)
        tk.Label(fr, text=fill['time'], width=9, bg=bg, fg="#94A3B8",
                  font=("Consolas", 9), anchor="center").pack(side="left", padx=1)
        tk.Label(fr, text=fill['action'], width=5, bg=bg, fg=action_color,
                  font=("Consolas", 9, "bold"), anchor="center").pack(side="left", padx=1)
        tk.Label(fr, text=str(fill['qty']), width=5, bg=bg, fg="#CBD5E1",
                  font=("Consolas", 9), anchor="center").pack(side="left", padx=1)
        tk.Label(fr, text=fill['contract'], width=10, bg=bg, fg="#60A5FA",
                  font=("Consolas", 9), anchor="center").pack(side="left", padx=1)
        tk.Label(fr, text=f"{fill['price']:.2f}", width=11, bg=bg, fg="#E2E8F0",
                  font=("Consolas", 9), anchor="center").pack(side="left", padx=1)

```

Method: TradeOpssAIApp._auto_launch_propfirm_browsers

```
def _auto_launch_propfirm_browsers(self, active_firms)
```

Detect which active prop firms have browser-based dashboards and store them so the UI dashboard buttons know what to launch. Does NOT auto-open browsers — the user clicks the "Dashboard" button in the broker row. Tradovate/TopStepX auto-connect with credentials as before.

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. if RELEASE_DISABLE_PROP_DASHBOARD_ACCESS: branches conditionally.

2. **for** `firm` in `active_firms`: iterates.

Code (copy-paste safe)

```
def _auto_launch_propfirm_browsers(self, active_firms):
    """
    Detect which active prop firms have browser-based dashboards and store
    them so the UI dashboard buttons know what to launch. Does NOT auto-open
    browsers – the user clicks the "Dashboard" button in the broker row.
    Tradovate/TopStepX auto-connect with credentials as before.
    """
    if RELEASE_DISABLE_PROP_DASHBOARD_ACCESS:
        return

    for firm in active_firms:
        cfg = self._BROWSER_MONITORED_FIRMS.get(firm)
        if not cfg:
            continue
        class_avail = globals().get(cfg["class_available"], False)
        if class_avail:
            self.log(f"■ {firm} has a dashboard – click the Dashboard button to connect")
```

Method: `TradeOpssAIApp._launch_propfirm_dashboard`

```
def _launch_propfirm_dashboard(self, firm_name)
```

Launch a Chrome browser for the prop firm's dashboard, show login dialog. Called when user clicks the 'Dashboard' button in a broker row. For CDP-based scrapers, attaches to an existing Chrome tab instead of launching new Chrome.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.

Step by step (top-level body)

1. **if** `RELEASE_DISABLE_PROP_DASHBOARD_ACCESS`: branches conditionally.
2. Assigns `cfg = self._BROWSER_MONITORED_FIRMS.get(firm_name)`.
3. **if** `not cfg`: branches conditionally.
4. **if** `not cfg`: branches conditionally.
5. **if** `firm_name` in `self._propfirm_browsers` and `self._propfirm_browsers[fi...]`: branches conditionally.
6. Assigns `class_avail = globals().get(cfg['class_available'], False)`.
7. **if** `not class_avail`: branches conditionally.

8. if `cfg.get('cdp')`: branches conditionally.

Code (copy-paste safe)

```
def _launch_propfirm_dashboard(self, firm_name):
    """
    Launch a Chrome browser for the prop firm's dashboard, show login dialog.
    Called when user clicks the 'Dashboard' button in a broker row.
    For CDP-based scrapers, attaches to an existing Chrome tab instead of launching new Chrome.
    """
    if RELEASE_DISABLE_PROP_DASHBOARD_ACCESS:
        self.log("■ Prop firm dashboard access is disabled in this release", "WARN")
        return

    cfg = self._BROWSER_MONITORED_FIRMS.get(firm_name)
    if not cfg:
        # Case-insensitive fallback
        for _bk, _bv in self._BROWSER_MONITORED_FIRMS.items():
            if _bk.lower() == firm_name.lower():
                cfg = _bv
                break

    if not cfg:
        self.log(f"■ No dashboard config for {firm_name}", "WARN")
        return

    # Skip if already connected
    if firm_name in self._propfirm_browsers and self._propfirm_browsers[firm_name]:
        try:
            if self._propfirm_browsers[firm_name].is_connected():
                self.log(f"■ {firm_name} dashboard already open")
                return
        except Exception:
            pass

    class_avail = globals().get(cfg["class_available"], False)
    if not class_avail:
        self.log(f"■ {firm_name} browser module not available", "WARN")
        return

    # CDP-based scrapers: launch Chrome (or reuse) → open tab → show login dialog → attach
    if cfg.get("cdp"):
        self.log(f"■ Attaching to {firm_name} dashboard tab (CDP)...")
        def _do_cdp_launch():
            try:
                account_cls = globals().get(cfg["account_class"])
                if not account_cls:
                    self.root.after(0, lambda: self.log(f"■ {firm_name}: scraper class not found",
                                                         "ERROR"))
                return

                login_url = cfg.get("login_url", "")

                # Ensure Chrome debug is running and the tab is open
                try:
                    ensure_chrome_debug(url=login_url, port=9222)
                except FileNotFoundError as e:
                    self.root.after(0, lambda err=str(e):
                        self.log(f"■ {err}", "ERROR"))
                return

                acct = account_cls(debug_port=9222)
```

```

# Try to attach immediately (tab may already be logged in)
try:
    acct.login(open_url=login_url)
    self._propfirm_browsers[firm_name] = acct
    self.root.after(0, lambda:
        self.log(f"■ {firm_name} dashboard connected via CDP"))
    acct_mapping = self._autofill_challenge_fees(firm_name, acct)
    self._cached_acct_mappings[firm_name] = acct_mapping or {}
    self._auto_detect_breached_accounts(firm_name, acct, acct_mapping=acct_mapping)
    return
except ConnectionError:
    # Tab not ready yet – show login dialog
    pass

# Show dialog asking user to log in, then retry
self.root.after(0, lambda: self._show_cdp_login_dialog(
    firm_name, acct, cfg))

except Exception as e:
    self.root.after(0, lambda err=str(e):
        self.log(f"■ {firm_name} CDP launch failed: {err}", "ERROR"))
threading.Thread(target=_do_cdp_launch, daemon=True).start()
return

```

Method: TradeOpssAIApp._show_propfirm_login_dialog

```
def _show_propfirm_login_dialog(self, firm_name, account, cfg)
```

Show a dialog prompting the user to log in to the prop firm dashboard.

Why these Python concepts were used

- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **if CTK_AVAILABLE:** branches conditionally (with an **else/elif** arm).
2. Calls `dialog.title(...)` for its side effect.
3. Calls `dialog.geometry(...)` for its side effect.
4. Calls `dialog.resizable(...)` for its side effect.
5. Calls `dialog.attributes(...)` for its side effect.
6. Assigns `status_var = tk.StringVar(value=f'Please log in to {firm_name} in the ...`
7. **if CTK_AVAILABLE:** branches conditionally (with an **else/elif** arm).

Code (copy-paste safe)

```

def _show_propfirm_login_dialog(self, firm_name, account, cfg):
    """Show a dialog prompting the user to log in to the prop firm dashboard."""
    if CTK_AVAILABLE:
        dialog = ctk.CTkToplevel(self.root)
    else:
        dialog = tk.Toplevel(self.root)

```



```

dialog.title(f"{firm_name} Dashboard Login")
dialog.geometry("380x140")
dialog.resizable(False, False)
dialog.attributes("-topmost", True)

status_var = tk.StringVar(
    value=f"Please log in to {firm_name} in the browser window,\nthen click the button below.")
if CTk_AVAILABLE:
    status_lbl = ctk.CTkLabel(dialog, textvariable=status_var, font=("Consolas", 11),
                             wraplength=340, justify="center")
    status_lbl.pack(pady=(16, 8), padx=16)
    confirm_btn = ctk.CTkButton(dialog, text="■ I've logged in", width=160, height=30,
                                fg_color="#14532D", hover_color="#166534",
                                font=("Consolas", 11), text_color="#D1FAE5",
                                command=lambda: self._on_login_confirmed(firm_name, account, cfg, dialog, status_var, confirm_btn))
    confirm_btn.pack(pady=(4, 16))
else:
    status_lbl = tk.Label(dialog, textvariable=status_var, font=("Consolas", 10),
                          wraplength=340, justify="center")
    status_lbl.pack(pady=(16, 8), padx=16)
    confirm_btn = tk.Button(dialog, text="■ I've logged in", width=20,
                            command=lambda: self._on_login_confirmed(firm_name, account, cfg, dialog, status_var, confirm_btn))
    confirm_btn.pack(pady=(4, 16))

```

Method: TradeOpssAIApp._on_login_confirmed

```
def _on_login_confirmed(self, firm_name, account, cfg, dialog, status_var, confirm_btn)
```

Called when user clicks 'I've logged in' — disable button and verify in background.

Step by step (top-level body)

1. Calls `confirm_btn.configure(...)` for its side effect.
2. Calls `status_var.set(...)` for its side effect.
3. Calls `threading.Thread(target=self._verify_propfirm_browser, args=(firm_name, account, cfg, dialog, status_var), daemon=True).start(...)` for its side effect.

Code (copy-paste safe)

```

def _on_login_confirmed(self, firm_name, account, cfg, dialog, status_var, confirm_btn):
    """Called when user clicks 'I've logged in' — disable button and verify in background."""
    confirm_btn.configure(state="disabled")
    status_var.set("■ Verifying login...")
    threading.Thread(target=self._verify_propfirm_browser,
                     args=(firm_name, account, cfg, dialog, status_var),
                     daemon=True).start()

```

Method: TradeOpssAIApp._show_cdp_login_dialog

```
def _show_cdp_login_dialog(self, firm_name, acct, cfg)
```

Show dialog for CDP-based scrapers asking user to log in, then attach.

Why these Python concepts were used

- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. if CTK_AVAILABLE: branches conditionally (with an **else/elif** arm).
2. Calls dialog.title(...) for its side effect.
3. Calls dialog.geometry(...) for its side effect.
4. Calls dialog.resizable(...) for its side effect.
5. Calls dialog.attributes(...) for its side effect.
6. Assigns status_var = tk.StringVar(value=f'Please log in to {firm_name} in the ...
7. if CTK_AVAILABLE: branches conditionally (with an **else/elif** arm).

Code (copy-paste safe)

```
def _show_cdp_login_dialog(self, firm_name, acct, cfg):
    """Show dialog for CDP-based scrapers asking user to log in, then attach."""
    if CTK_AVAILABLE:
        dialog = ctk.CTkToplevel(self.root)
    else:
        dialog = tk.Toplevel(self.root)
    dialog.title(f"{firm_name} Dashboard Login")
    dialog.geometry("400x150")
    dialog.resizable(False, False)
    dialog.attributes("-topmost", True)

    status_var = tk.StringVar(
        value=f"Please log in to {firm_name} in the Chrome window,\nthen click the button below.")
    if CTK_AVAILABLE:
        ctk.CTkLabel(dialog, textvariable=status_var, font=("Consolas", 11),
                     wraplength=360, justify="center").pack(pady=(16, 8), padx=16)
        confirm_btn = ctk.CTkButton(
            dialog, text="■ I've logged in", width=160, height=30,
            fg_color="#14532D", hover_color="#166634",
            font=("Consolas", 11), text_color="#D1FAE5",
            command=lambda: self._on_cdp_login_confirmed(
                firm_name, acct, cfg, dialog, status_var, confirm_btn))
        confirm_btn.pack(pady=(4, 16))
    else:
        tk.Label(dialog, textvariable=status_var, font=("Consolas", 10),
                 wraplength=360, justify="center").pack(pady=(16, 8), padx=16)
        confirm_btn = tk.Button(
            dialog, text="■ I've logged in", width=20,
            command=lambda: self._on_cdp_login_confirmed(
                firm_name, acct, cfg, dialog, status_var, confirm_btn))
        confirm_btn.pack(pady=(4, 16))
```

Method: TradeOpssAIApp._on_cdp_login_confirmed

```
def _on_cdp_login_confirmed(self, firm_name, acct, cfg, dialog, status_var, confirm_btn)
```

Attach CDP after user confirms they've logged in.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.

Step by step (top-level body)

1. Calls `confirm_btn.configure(...)` for its side effect.
2. Calls `status_var.set(...)` for its side effect.
3. Defines a nested function `_attach(...)`.
4. Calls `threading.Thread(target=_attach, daemon=True).start(...)` for its side effect.

Code (copy-paste safe)

```
def _on_cdp_login_confirmed(self, firm_name, acct, cfg, dialog, status_var, confirm_btn):
    """Attach CDP after user confirms they've logged in."""
    confirm_btn.configure(state="disabled")
    status_var.set("■ Attaching to dashboard...")

    def _attach():
        try:
            login_url = cfg.get("login_url", "")
            acct.login(open_url=login_url)
            self._propfirm_browsers[firm_name] = acct
            self.root.after(0, lambda: self.log(
                f"■ {firm_name} dashboard connected via CDP"))
            acct_mapping = self._autofill_challenge_fees(firm_name, acct)
            self._cached_acct_mappings[firm_name] = acct_mapping or {}
            self._auto_detect_breached_accounts(firm_name, acct, acct_mapping=acct_mapping)
            self.root.after(0, dialog.destroy)
        except ConnectionError:
            self.root.after(0, lambda: status_var.set(
                f"■ Could not find {firm_name} tab.\n"
                f"Make sure {cfg.get('login_url', '')} is open and loaded.))
            self.root.after(0, lambda: confirm_btn.configure(state="normal"))
        except Exception as e:
            self.root.after(0, lambda err=str(e): status_var.set(f"■ {err}"))
            self.root.after(0, lambda: confirm_btn.configure(state="normal"))

    threading.Thread(target=_attach, daemon=True).start()
```

Method: TradeOpssAIApp._verify_propfirm_browser

```
def _verify_propfirm_browser(self, firm_name, account, cfg, dialog=None, status_var=None)
    Navigate to accounts page, verify login, auto-fill fees, close dialog.
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **list comprehension** — Builds a list in a single expression ([f(x) for x in xs]). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Defines a nested function `_update_status(...)`.
2. Defines a nested function `_close_dialog(...)`.
3. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def _verify_propfirm_browser(self, firm_name, account, cfg, dialog=None, status_var=None):
    """Navigate to accounts page, verify login, auto-fill fees, close dialog."""
    def _update_status(msg):
        if status_var:
            self.root.after(0, lambda m=msg: status_var.set(m))

    def _close_dialog():
        if dialog:
            self.root.after(0, lambda: dialog.destroy())

    try:
        _update_status("■ Navigating to accounts page...")
        account.driver.get(cfg["accounts_url"])
        time.sleep(3)

        current_url = account.driver.current_url
        if "accounts" in current_url or account.is_connected():
            account.logged_in = True
            account._login_timestamp = time.time()
            self.root.after(0, lambda:
                self.log(f"■ {firm_name} browser connected – monitoring active"))

        # Auto-fill challenge fees from billing history
        _update_status("■ Fetching billing history...")
        acct_mapping = self._autofill_challenge_fees(firm_name, account)

        # Auto-detect breached accounts and mark as failed
```

```

        _update_status("■ Checking breach status...")
        self._cached_acct_mappings[firm_name] = acct_mapping or {}
        self._auto_detect_breached_accounts(firm_name, account, acct_mapping=acct_mapping)

        _update_status("■ Connected!")
        time.sleep(1)
        _close_dialog()
    else:
        self.root.after(0, lambda:
            self.log(f"■ {firm_name} may not be logged in (URL: {current_url}). "
                    f"You can reconnect later.", "WARN"))
        _update_status("■ Login not detected – try again")
        # Re-enable the button so user can retry
        if dialog:
            self.root.after(0, lambda: [
                w.configure(state="normal")
                for w in dialog.winfo_children()
                if hasattr(w, 'configure') and isinstance(w, (
                    ctk.CTkButton if CTK_AVAILABLE else tk.Button,))
            ])
    except Exception as e:
        self.root.after(0, lambda err=str(e):
            self.log(f"■ {firm_name} browser verification failed: {err}", "ERROR"))
        _update_status(f"■ Error: {str(e)[:50]}")

```

Method: TradeOpssAIApp._autofill_challenge_fees

```
def _autofill_challenge_fees(self, firm_name, account)
```

Scrape billing history from the prop firm dashboard and auto-fill the 'Fee', 'Date Purchased', and 'Account #' fields of active evaluations. Then push the updated evaluations to the dashboard. For FundedNext Futures accounts, also resolves the billing login ID to the Tradovate account name (e.g. 945576089 -> FNFTCHHARRISONOUKA85625). Returns the acct_mapping dict (or {}) so callers can reuse it for breach detection.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **list comprehension** — Builds a list in a single expression ([f(x) for x in xs]). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns acct_mapping = {}.
2. try block with 1 except clause.

Code (copy-paste safe)

```
def _autofill_challenge_fees(self, firm_name, account):
    """
    Scrape billing history from the prop firm dashboard and auto-fill
    the 'Fee', 'Date Purchased', and 'Account #' fields of active evaluations.
    Then push the updated evaluations to the dashboard.

    For FundedNext Futures accounts, also resolves the billing login ID
    to the Tradovate account name (e.g. 945576089 -> FNFTCHHARRISONOUKA85625).

    Returns the acct_mapping dict (or {}) so callers can reuse it for breach detection.
    """
    acct_mapping = {}
    try:
        if not hasattr(account, 'get_billing_history'):
            self.root.after(0, lambda:
                self.log(f"■ {firm_name}: No get_billing_history method – skipping"))
            return acct_mapping

        self.root.after(0, lambda:
            self.log(f"■ {firm_name}: Scraping billing history..."))

        # Prefer API-based billing if available (richer data with login field)
        if hasattr(account, 'get_billing_via_api'):
            billing = account.get_billing_via_api()
        else:
            billing = account.get_billing_history()
        if not billing:
            self.root.after(0, lambda:
                self.log(f"■ {firm_name}: No billing records found"))
            return acct_mapping

        self.root.after(0, lambda b=len(billing):
            self.log(f"■ {firm_name}: Found {b} billing record(s)"))

        # Get account mapping (billing login -> Tradovate account name)
        if hasattr(account, 'get_account_mapping'):
            try:
                self.root.after(0, lambda:
                    self.log(f"■ {firm_name}: Fetching account mapping..."))
                acct_mapping = account.get_account_mapping()
                if acct_mapping:
                    self.root.after(0, lambda n=len(acct_mapping):
                        self.log(f"■ {firm_name}: Got {n} login→account mapping(s)"))
            except Exception as e:
                self.root.after(0, lambda err=str(e):
                    self.log(f"■ {firm_name}: Account mapping failed: {err}", "WARN"))

        # Log all billing entries for visibility
        for i, entry in enumerate(billing):
            acct = entry.get("account_no", "?")
            amt = entry.get("paid_amount", "?")
            status = entry.get("status", "?")
            date = entry.get("date", "?")
            pkg = entry.get("funding_package", "?")
            self.root.after(0, lambda idx=i, a=acct, m=amt, s=status, d=date, p=pkg:
                self.log(
                    f"    ■ Billing #{idx+1}: Acct={a} | Amount={m} | Status={s} | Date={d} | Pkg={p}"))
```

```

# Build lookup: account_no -> {amount, date, package}
# First entry wins for billing_by_acct (preserves original fee under login key).
# Last entry wins for billing_by_login (latest fee maps to current Tradovate account).
# This handles resets: e.g. FundedNext login 946645337 has $139.04 (new) + $142.13 (reset).
# The $139.04 stays under "946645337", while $142.13 maps to the active Tradovate account.
billing_by_acct = {}
billing_by_size = {}
billing_by_login = {}
for entry in billing:
    acct_no = (entry.get("account_no") or "").strip()
    status = (entry.get("status") or "").strip().upper()
    amount = entry.get("paid_amount_numeric", 0.0)
    bill_date = (entry.get("date") or "").strip()
    pkg = (entry.get("funding_package") or "").strip()
    login_id = str(entry.get("login") or acct_no).strip()
    if acct_no and amount > 0 and status == "APPROVED":
        info = {"amount": amount, "date": bill_date, "package": pkg, "account_no": acct_no,
                "login": login_id}
        # First entry wins – keeps oldest fee under the raw account_no key
        if acct_no not in billing_by_acct:
            billing_by_acct[acct_no] = info
        # Extract numeric size from package string (e.g. "50000" from "Futures Legacy Challenge")
        import re as _re
        size_match = _re.search(r'(\d{4,})', pkg.replace(",", ""))
        if size_match:
            size_key = int(size_match.group(1))
            if size_key not in billing_by_size:
                billing_by_size[size_key] = dict(info)
        # Last entry wins – latest fee maps to current active Tradovate account
        billing_by_login[login_id] = dict(info)

# Resolve billing login IDs to Tradovate account names via account mapping
# This enriches billing_by_acct so matching by FNFT/TDFY/LFE account name works.
# billing_by_login has the LATEST entry per login → maps to current active Tradovate account
for login_id, bill_info in billing_by_login.items():
    if login_id in acct_mapping:
        tv_name = acct_mapping[login_id].get("tradovate_account_name")
        if tv_name and tv_name not in billing_by_acct:
            enriched = dict(bill_info)
            enriched["tradovate_account_name"] = tv_name
            billing_by_acct[tv_name] = enriched
        if tv_name:
            self.root.after(0, lambda lid=login_id, tv=tv_name, amt=bill_info["amount"]:
                self.log(f"    ■ Mapped billing login {lid} → {tv} (${amt:.2f}"))

# Log challenge fees per account
for acct_key, info in billing_by_acct.items():
    self.root.after(0, lambda a=acct_key, t=info["amount"]:
        self.log(f"    ■ {a}: ${t:.2f}"))

if not billing_by_acct and not billing_by_size:
    self.root.after(0, lambda:
        self.log(f"■ {firm_name}: No APPROVED billing entries with amount > 0"))
    return acct_mapping

self.root.after(0, lambda n=len(billing_by_acct), s=len(billing_by_size),
    ak=list(billing_by_acct.keys()), sk=list(billing_by_size.keys()):
    self.log(f"■ {firm_name}: {n} by-acct ({ak}), {s} by-size ({sk}"))

# Log active trade rows for debugging

```

```

row_count = len(self._active_trade_rows)
self.root.after(0, lambda c=row_count:
    self.log(f"■ {firm_name}: Checking {c} active trade row(s) for missing Fee/Date"))

# Canonical firm name for comparison
canonical_firm = self._FIRM_MAP.get(firm_name, firm_name)

# Match billing to active evaluations missing Fee or Date Purchased
updated_evals = []
filled_count = 0
for row_data in self._active_trade_rows:
    ev = row_data.get("eval")
    if not ev:
        continue

    # Check if eval belongs to this prop firm (flexible matching)
    ev_firm = ev.get("Prop Firm", "")
    ev_canonical = self._FIRM_MAP.get(ev_firm, ev_firm)
    if ev_canonical != canonical_firm and ev_firm != firm_name:
        continue

    acct_challenge = (ev.get("Account #") or "").strip()
    acct_funded = (ev.get("Account #.1") or "").strip()
    existing_fee = (ev.get("Fee") or "").strip()
    existing_date = (ev.get("Date Purchased") or "").strip()

    fee_filled = False
    if existing_fee:
        try:
            existing_val = float(existing_fee.replace("$", "").replace(",", ""))
            fee_filled = existing_val > 0
        except ValueError:
            fee_filled = existing_fee not in ("", "$0", "$0.00", "0")
    date_filled = bool(existing_date)

    # Log each eval's current state
    self.root.after(0, lambda f=ev_firm, ac=acct_challenge, af=acct_funded,
        ef=existing_fee, ed=existing_date, ff=fee_filled, df=date_filled:
        self.log(f" ■ Eval: Firm={f} | Acct#={ac} | Acct#.1={af} | "
            f"Fee='{ef}' ({'✓' if ff else 'X'}) | Date='{ed}' ({'✓' if df else 'X'})"))

# Note: we no longer skip early – fee is always updated if it differs from billing

# Try matching by Account # or Account #.1
matched = None
matched_via = ""
for acct_key, label in [(acct_challenge, "Account #"), (acct_funded, "Account #.1")]:
    if not acct_key:
        continue
    # Direct match
    if acct_key in billing_by_acct:
        matched = billing_by_acct[acct_key]
        matched_via = f"{label} direct: {acct_key}"
        break
    # Partial match: billing account_no may be a substring
    for bill_acct, bill_info in billing_by_acct.items():
        if bill_acct in acct_key or acct_key in bill_acct:
            matched = bill_info
            matched_via = f"{label} partial: eval={acct_key} ~ bill={bill_acct}"
            break

```



```

        if matched:
            break

# Fallback: match by Account Size when Account # is empty
if not matched and billing_by_size:
    ev_size_str = (ev.get("Account Size") or "").strip()
    import re as _re
    size_match = _re.search(r'(\d[\d,]*)', ev_size_str.replace(",", ""))
    if size_match:
        ev_size = int(size_match.group(1))
        if ev_size in billing_by_size:
            matched = billing_by_size[ev_size]
            matched_via = f"Account Size fallback: ${ev_size:,} → bill acct {matched.get('acct_challenge')}"

if matched:
    billing_fee = f"${matched['amount']:.2f}"
    changes = []
    # Always overwrite fee with billing value
    ev["Fee"] = billing_fee
    changes.append(f"Fee={billing_fee}")
    if not date_filled and matched["date"]:
        ev["Date Purchased"] = matched["date"]
        changes.append(f"DatePurchased={matched['date']}")
    # Also set Date Started from billing date when empty
    existing_start = (ev.get("Date Started") or "").strip()
    if not existing_start and matched["date"]:
        ev["Date Started"] = matched["date"]
        changes.append(f"DateStarted={matched['date']}")
    # Auto-fill Account # from account mapping when empty
    tv_name = matched.get("tradovate_account_name")
    if not acct_challenge and tv_name:
        ev["Account #"] = tv_name
        changes.append(f"Account#={tv_name}")
    elif not acct_challenge and matched.get("login") and str(matched["login"]) in acct_mapping:
        tv_name = acct_mapping[str(matched["login"])]["tradovate_account_name"]
        if tv_name:
            ev["Account #"] = tv_name
            changes.append(f"Account#={tv_name}")
    filled_count += 1
    updated_evals.append(ev)
    self.root.after(0, lambda v=matched_via, c=", ".join(changes):
        self.log(f"    ■ Matched ({v}): {c}"))
else:
    acct_display = acct_challenge or acct_funded or "no-acct"
    bill_keys = list(billing_by_acct.keys())
    self.root.after(0, lambda a=acct_display, bk=bill_keys:
        self.log(f"    ■ No billing match for {a} (billing accts: {bk}"))

if not filled_count:
    self.root.after(0, lambda:
        self.log(f"    ■ {firm_name}: No accounts needed Fee/Date updates"))
# Still compute per-firm summary even when no evals updated
total_fees, total_payouts, records = self._compute_firm_billing(
    firm_name, account, billing)
self._firm_billing_summary[firm_name] = {
    "total_fees": total_fees, "total_payouts": total_payouts, "records": records}
self.root.after(0, lambda f=firm_name, tf=total_fees, tp=total_payouts:
    self.log(f"    ■ {f} Summary – Total Fees: ${tf:.2f} | Total Payouts: ${tp:.2f}"))
return acct_mapping

```

```

self.root.after(0, lambda c=filled_count:
    self.log(f"■ {firm_name}: Pushing {c} updated eval(s) to dashboard..."))

# Push updated evaluations to dashboard
email = self.client_email_entry.get().strip()
dashboard_url = self.url_entry.get().strip().rstrip('/')
if not email or not dashboard_url:
    self.root.after(0, lambda:
        self.log(f"■ Cannot push fee updates – no email/dashboard URL", "WARN"))
    return acct_mapping

# Collect ALL active evals (server expects the full set)
all_evals = [rd.get("eval") for rd in self._active_trade_rows if rd.get("eval")]

# Debug: log the Fee value we're about to push
for ev_debug in all_evals:
    ev_fee = ev_debug.get("Fee", "N/A")
    ev_acct = ev_debug.get("Account #", "?")
    self.root.after(0, lambda f=ev_fee, a=ev_acct:
        self.log(f"    ■ Pushing eval Acct={a} Fee={f}"))

self.root.after(0, lambda n=len(all_evals):
    self.log(f"■ {firm_name}: Sending {n} total eval(s) in push payload"))

payload = {
    "email": email,
    "evaluations": all_evals,
    "statistics": {},
    "dropdown_options": {},
    "firm_billing": self._firm_billing_summary,
    "force_fields": ["Fee", "Date Purchased", "Date Started"],
}

try:
    response = _gzip_post(
        f"{dashboard_url}/api/client/push",
        payload,
        timeout=30
    )
    if response.status_code == 200:
        data = response.json()
        if data.get("status") == "success":
            self.root.after(0, lambda c=filled_count:
                self.log(
                    f"■ Auto-filled Fee & Date Purchased for {c} account(s) → synced to dash
                ))
        else:
            self.root.after(0, lambda m=data.get('message', 'Unknown'):
                self.log(f"■ Fee sync response: {m}", "WARN"))
    else:
        self.root.after(0, lambda s=response.status_code:
            self.log(f"■ Fee sync failed: HTTP {s}", "WARN"))
except Exception as e:
    self.root.after(0, lambda err=str(e):
        self.log(f"■ Fee sync error: {err}", "WARN"))

# ■■ Per-firm totals: aggregate challenge fees + payouts ■■
total_fees, total_payouts, records = self._compute_firm_billing(
    firm_name, account, billing)
self._firm_billing_summary[firm_name] = {
    "total_fees": total_fees, "total_payouts": total_payouts, "records": records}

```

```

        self.root.after(0, lambda f=firm_name, tf=total_fees, tp=total_payouts:
            self.log(f"■ {f} Summary – Total Fees: ${tf:.2f} | Total Payouts: ${tp:.2f}"))

    return acct_mapping

except Exception as e:
    self.root.after(0, lambda err=str(e):
        self.log(f"■ {firm_name} billing auto-fill failed: {err}", "WARN"))
    return acct_mapping

```

Method: TradeOpssAIApp._compute_firm_billing

```
def _compute_firm_billing(self, firm_name, account, billing)
```

Compute per-firm total fees and payouts from scraped billing + payout data. Returns (total_fees, total_payouts, records).

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.

Step by step (top-level body)

1. Assigns `total_fees = sum((e.get('paid_amount_numeric', 0.0) for e in billing i....`
2. Assigns `records = []`.
3. **for** `e in billing`: iterates.
4. Assigns `total_payouts = 0.0`.
5. **try** block with 1 **except** clause.
6. **return** `(total_fees, total_payouts, records)`.

Code (copy-paste safe)

```

def _compute_firm_billing(self, firm_name, account, billing):
    """Compute per-firm total fees and payouts from scraped billing + payout data.
    Returns (total_fees, total_payouts, records)."""
    total_fees = sum(
        e.get("paid_amount_numeric", 0.0)
        for e in billing
        if (e.get("status") or "").strip().upper() == "APPROVED"
        and e.get("paid_amount_numeric", 0) > 0
    )
    records = []
    for e in billing:
        if (e.get("status") or "").strip().upper() == "APPROVED" and e.get("paid_amount_numeric", 0) > 0:
            records.append({
                "account_no": e.get("account_no", ""),
                "amount": e.get("paid_amount_numeric", 0),
                "date": e.get("date", ""),
                "package": e.get("funding_package", ""),
                "type": e.get("transition_type", "")
            })

```

```

        })
    total_payouts = 0.0
    try:
        if hasattr(account, 'get_payouts'):
            payouts = account.get_payouts()
            for p in (payouts or []):
                for key in ("amount", "payout_amount", "netAmount", "total", "value"):
                    val = p.get(key)
                    if val is not None:
                        try:
                            total_payouts += abs(float(str(val).replace("$", "").replace(",", "")))
                        except (ValueError, TypeError):
                            pass
                    break
    except Exception:
        pass
    return total_fees, total_payouts, records

```

Method: TradeOpssAIApp._start_status_polling

```
def _start_status_polling(self)
```

Start the 10-second real-time status polling loop. Uses Tradovate broker API to fetch live balances for all accounts.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.

Step by step (top-level body)

1. **if RELEASE_DISABLE_STATUS_POLL:** branches conditionally.
2. **if self._status_poll_active:** branches conditionally.
3. Assigns **self._status_poll_active = True**.
4. Calls **self.root.after(...)** for its side effect.
5. Defines a nested function **_first_poll(...)**.
6. Calls **threading.Thread(target=_first_poll, daemon=True).start(...)** for its side effect.
7. Calls **self.root.after(...)** for its side effect.

Code (copy-paste safe)

```

def _start_status_polling(self):
    """Start the 10-second real-time status polling loop.
    Uses Tradovate broker API to fetch live balances for all accounts."""

```

```

if RELEASE_DISABLE_STATUS_POLL:
    self._status_poll_active = False
    return

if self._status_poll_active:
    return # already running
self._status_poll_active = True
self.root.after(0, lambda: self.log("■ Status polling started (every 5 min)"))
# Run FIRST poll immediately, then schedule repeating every 5 min
def _first_poll():
    try:
        self._poll_tradovate_balances()
    except Exception as e:
        self.root.after(0, lambda err=str(e):
            self.log(f"■ First status poll error: {err}", "WARN"))
        threading.Thread(target=_first_poll, daemon=True).start()
    self.root.after(300_000, self._poll_account_status)

```

Method: TradeOpssAIApp._poll_account_status

```
def _poll_account_status(self)
```

Self-rescheduling timer — fetches Tradovate balances and updates P1 status.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.

Step by step (top-level body)

1. **if RELEASE_DISABLE_STATUS_POLL:** branches conditionally.
2. **if not self._status_poll_active:** branches conditionally.
3. Defines a nested function **_poll_all(...)**.
4. Calls **threading.Thread(target=_poll_all, daemon=True).start(...)** for its side effect.
5. Calls **self.root.after(...)** for its side effect.

Code (copy-paste safe)

```

def _poll_account_status(self):
    """Self-rescheduling timer — fetches Tradovate balances and updates P1 status."""
    if RELEASE_DISABLE_STATUS_POLL:
        return

    if not self._status_poll_active:
        return

```

```

def _poll_all():
    try:
        self._poll_tradovate_balances()
    except Exception as e:
        self.root.after(0, lambda err=str(e):
            self.log(f"■ Status poll error: {err}", "WARN"))

threading.Thread(target=_poll_all, daemon=True).start()
self.root.after(300_000, self._poll_account_status)

```

Method: TradeOpssAIApp._poll_tradovate_balances

```
def _poll_tradovate_balances(self)
```

Fetch live balances from Tradovate broker API and update P1/Status on evaluations.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **dict comprehension** — Builds a dict in one expression (`{k: v for k, v in items}`) — same intent as a list comprehension but for mappings.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to `sum`, `any`, `all`, or `max` where the intermediate list would be wasted.
- **lambda** — Uses a single-expression anonymous function — typically as the `key=` for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **if** `RELEASE_DISABLE_STATUS_POLL`: branches conditionally.
2. Assigns `tv_balances = {}`.
3. **for** `(firm_name, conn)` in `list(self._broker_connections.items())`: iterates.
4. **if not** `tv_balances`: branches conditionally.
5. **if not** `hasattr(self, '_poll_logged_tv_accounts')`: branches conditionally.
6. Assigns `tv_lower = {k.lower(): (k, v) for k, v in tv_balances.items()}`.
7. Assigns `email = self.client_email_entry.get().strip()`.
8. Assigns `dashboard_url = self.url_entry.get().strip().rstrip('/')`.
9. **if not** `email` or `not dashboard_url`: branches conditionally.
10. **try** block with 1 **except** clause.

11. Assigns `self._poll_dashboard_err_logged = False`.

12. `if not all_evals:` branches conditionally.

13. Assigns `any_changed = False`.

14. `for ev in all_evals:` iterates.

15. ... and 3 more statement(s) in the body.

Code (copy-paste safe)

```
def _poll_tradovate_balances(self):
    """Fetch live balances from Tradovate broker API and update P1/Status on evaluations."""
    if RELEASE_DISABLE_STATUS_POLL:
        return

    # ■■ 1. Gather all Tradovate account balances from broker connections ■■
    tv_balances = {} # {account_name: netLiq}
    for firm_name, conn in list(self._broker_connections.items()):
        tv_account = conn.get("account")
        if not tv_account:
            continue
        # Support both TradovateAccount (_api_fetch) and DOM-based stats
        if not hasattr(tv_account, '_api_fetch'):
            continue
        try:
            # First check if browser/driver is alive
            try:
                _ = tv_account.driver.title
            except Exception:
                self.root.after(0, lambda fn=firm_name:
                    self.log(f"■ Status poll: {fn} browser not accessible", "WARN"))
                continue

            raw = tv_account._api_fetch("/account/list")
            if not raw or not isinstance(raw, list):
                continue
            for acct in raw:
                aid = acct.get('id')
                aname = acct.get('name', '')
                if not aid or not aname:
                    continue
                snapshot = tv_account._api_fetch(
                    "/cashBalance/getCashBalanceSnapshot", "POST",
                    {"accountId": aid})
                if snapshot and isinstance(snapshot, dict):
                    net_liq = snapshot.get('netLiq')
                    if net_liq is not None:
                        tv_balances[aname] = float(net_liq)
        except Exception as e:
            self.root.after(0, lambda fn=firm_name, err=str(e):
                self.log(f"■ Status poll: API error for {fn}: {err}", "WARN"))

    if not tv_balances:
        self.root.after(0, lambda: self.log("■ Status poll: no Tradovate balances fetched", "WARN"))
        return

    # Log Tradovate accounts on first poll only
    if not hasattr(self, '_poll_logged_tv_accounts'):
        self._poll_logged_tv_accounts = True
        self.root.after(0, lambda n=len(tv_balances), names=list(tv_balances.keys())):
```

```

        self.log(f"■ Status poll: {n} Tradovate account(s): {names}")

# Build case-insensitive lookup
tv_lower = {k.lower(): (k, v) for k, v in tv_balances.items()}

# ■■ 2. Fetch evaluations from dashboard ■■
email = self.client_email_entry.get().strip()
dashboard_url = self.url_entry.get().strip().rstrip('/')
if not email or not dashboard_url:
    return

try:
    r = requests.post(
        f"{dashboard_url}/api/client/data",
        json={"email": email},
        headers={"Content-Type": "application/json"},
        timeout=15)
    if r.status_code != 200:
        self.root.after(0, lambda s=r.status_code:
            self.log(f"■ Status poll: dashboard returned HTTP {s}", "WARN"))
        return
    all_evals = r.json().get("evaluations", [])
except Exception as e:
    # Only log dashboard connection errors once to avoid spam
    if not getattr(self, '_poll_dashboard_err_logged', False):
        self._poll_dashboard_err_logged = True
        self.root.after(0, lambda err=str(e):
            self.log(f"■ Status poll: dashboard not reachable (will retry silently)", "WARN"))
    return

# Dashboard is reachable – reset error flag
self._poll_dashboard_err_logged = False

if not all_evals:
    return

# ■■ 3. Match evaluations to Tradovate balances and compute status ■■
any_changed = False
for ev in all_evals:
    if not ev or ev.get("_deleted"):
        continue

    acct_challenge = (ev.get("Account #") or "").strip()
    acct_funded = (ev.get("Account #.1") or "").strip()
    current_p1 = (ev.get("Status P1") or "").strip().lower()
    current_status = (ev.get("Status") or "").strip().lower()

    # Determine which account and field to check
    has_funded = bool(acct_funded)
    if has_funded:
        acct_key = acct_funded
        status_field = "Status"
        current_check = current_status
    else:
        acct_key = acct_challenge
        status_field = "Status P1"
        current_check = current_p1

    if not acct_key:
        continue

```



```

# Skip accounts already passed or failed
if "pass" in current_check or any(kw in current_check for kw in ("fail", "breach", "delete",
    "closed", "ended", "lost")):
    continue

# ■■ Find matching Tradovate balance ■■
acct_key_lower = acct_key.lower()
match = tv_lower.get(acct_key_lower)
if not match:
    # Try partial match (account name might be substring)
    for tv_name_lower, tv_pair in tv_lower.items():
        if tv_name_lower in acct_key_lower or acct_key_lower in tv_name_lower:
            match = tv_pair
            break
if not match:
    # Account not found on any connected Tradovate – mark as Failed
    miss_key = f"_miss_logged_{acct_key}"
    if not self._last_known_statuses.get(miss_key):
        self._last_known_statuses[miss_key] = True
        self.root.after(0, lambda a=acct_key, tvk=list(tv_balances.keys()):
            self.log(f"    ■ No match for '{a}' in Tradovate accounts {tvk} → marking Fail"))
    if current_check not in ("fail", "failed", "breach", "delete", "deleted", "closed", "ended",
        "lost"):
        ev[status_field] = "Fail"
        any_changed = True
        self.root.after(0, lambda a=acct_key, sf=status_field:
            self.log(f"    ■ {a}: {sf}='Fail' – not found on Tradovate"))
    continue

tv_name_orig, bal = match

# ■■ Get starting balance from Account Size ■■
size_str = (ev.get("Account Size") or "").replace("$", "").replace(",", "").strip().lower()
if size_str.endswith("k"):
    start = float(size_str[:-1]) * 1000
elif size_str.replace(".", "").isdigit():
    start = float(size_str)
else:
    start = 50000.0

# ■■ Get live targets from cached mapping if available ■■
canonical_firm = self._FIRM_MAP.get(ev.get("Prop Firm", ""), ev.get("Prop Firm", ""))
phase = "Funded" if has_funded else "Challenge"
live_target = None
live_min_eq = None
for _fn, mapping in self._cached_acct_mappings.items():
    for _mk, info in mapping.items():
        tv_name = info.get("tradovate_account_name", "")
        if tv_name and (tv_name.lower() in acct_key_lower or acct_key_lower in tv_name.lower()):
            live_target = info.get("profit_target")
            live_min_eq = info.get("min_equity")
            if info.get("starting_balance"):
                try:
                    start = float(info["starting_balance"])
                except (ValueError, TypeError):
                    pass
            break
    if live_target is not None:
        break

# ■■ Compute status ■■

```

```

# Use profit targets and min equity from _PROFIT_TARGETS table.
# Only mark Failed if balance <= minimum equity limit.
# Only mark Pass if balance >= start + profit target.
# Otherwise In Progress (if balance moved) or skip.
computed = None
try:
    # Get profit target and min equity for this firm/phase
    pt_target = None
    pt_min_eq = None

    # First try live targets from cached prop firm mapping
    if live_target is not None:
        pt_target = float(live_target)
    if live_min_eq is not None:
        pt_min_eq = float(live_min_eq)

    # Fallback to hardcoded profit targets table
    if pt_target is None and self.prop_firm_mgr:
        targets = self.prop_firm_mgr._PROFIT_TARGETS.get(canonical_firm, {})
        phase_key = "Challenge" if phase == "Challenge" else "Funded"
        t = targets.get(phase_key)
        if t is not None:
            pt_target = float(t)

    # Determine status
    if pt_min_eq is not None and bal <= pt_min_eq:
        computed = "Fail"
    elif pt_target is not None and bal >= (start + pt_target):
        computed = "Pass"
    elif abs(bal - start) > 0.50:
        computed = "In Progress"
    # else: balance hasn't moved, skip
except (ValueError, TypeError):
    continue

if not computed:
    continue

# Set the status on the eval
if computed.lower() != current_check:
    ev[status_field] = computed
    any_changed = True
    self.root.after(0, lambda a=acct_key, c=computed, b=bal, s=start:
        self.log(f"    ■ {a}: {c} (bal=${b:,.2f}, start=${s:,.0f})"))

# ■■ 4. Always push all evals so dashboard stays in sync ■■
if not any_changed:
    return

payload = {
    "email": email,
    "evaluations": all_evals,
    "statistics": {},
    "dropdown_options": {},
    "force_fields": ["Status P1", "Status"],
}
try:
    resp = requests.post(
        f"{dashboard_url}/api/client/push",
        json=payload,

```

```

        headers={"Content-Type": "application/json"},
        timeout=30)
    if resp.status_code == 200:
        rj = resp.json()
        if rj.get("status") == "success":
            self.root.after(0, lambda: self.log(f"■ Status poll: synced to dashboard"))
        else:
            self.root.after(0, lambda m=rj.get('message', '?'):
                self.log(f"■ Status poll push rejected: {m}", "WARN"))
    else:
        self.root.after(0, lambda s=resp.status_code, t=resp.text[:200]:
            self.log(f"■ Status poll push HTTP {s}: {t}", "WARN"))
except Exception as e:
    self.root.after(0, lambda err=str(e):
        self.log(f"■ Status poll push error: {err}", "WARN"))

```

Method: TradeOpssAIApp._auto_detect_breached_accounts

```
def _auto_detect_breached_accounts(self, firm_name, account, acct_mapping=None)
```

Check if any evaluations have been breached on FundedNext and auto-set their status to 'Failed' with the breach reason. Uses the account mapping (from get_account_mapping) which includes breach status from the React fiber, plus optionally the account-overview API for detailed breach info (breach reason, reset price etc). Fetches ALL evaluations from the dashboard (not just _active_trade_rows) since breached accounts may already be filtered out of the active rows.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **list comprehension** — Builds a list in a single expression ([f(x) for x in xs]). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.
- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def _auto_detect_breached_accounts(self, firm_name, account, acct_mapping=None):
    """
    Check if any evaluations have been breached on FundedNext
    and auto-set their status to 'Failed' with the breach reason.

```

Uses the account mapping (from get_account_mapping) which includes breach status from the React fiber, plus optionally the account-overview API for detailed breach info (breach reason, reset price etc).

Fetches ALL evaluations from the dashboard (not just _active_trade_rows) since breached accounts may already be filtered out of the active rows.

```

"""
try:
    if not acct_mapping:
        if hasattr(account, 'get_account_mapping'):
            acct_mapping = account.get_account_mapping()
        if not acct_mapping:
            return

    # Build reverse lookup: tradovate_account_name -> mapping info
    tv_to_info = {}
    for login_id, info in acct_mapping.items():
        tv_name = info.get("tradovate_account_name")
        if tv_name:
            tv_to_info[tv_name] = info
            tv_to_info[login_id] = info # also index by login

    self.root.after(0, lambda keys=list(tv_to_info.keys()):
        self.log(f"■ Breach check: mapping keys = {keys}"))

    # Canonical firm name for comparison
    canonical_firm = self._FIRM_MAP.get(firm_name, firm_name)

    # Fetch ALL evaluations from dashboard (not just _active_trade_rows,
    # since breached accounts may have been filtered out as inactive)
    email = self.client_email_entry.get().strip()
    dashboard_url = self.url_entry.get().strip().rstrip('/')
    all_evals = []
    if email and dashboard_url:
        try:
            r = requests.get(
                f"{dashboard_url}/api/client/data",
                params={"email": email},
                timeout=15
            )
            if r.status_code == 200:
                all_evals = r.json().get("evaluations", [])
        except Exception as e:
            self.root.after(0, lambda err=str(e):
                self.log(f"■ Breach check: couldn't fetch evals: {err}", "WARN"))

    if not all_evals:
        # Fallback to active trade rows
        all_evals = [rd.get("eval") for rd in self._active_trade_rows if rd.get("eval")]

    breached_count = 0
    updated_evals = []

    self.root.after(0, lambda n=len(all_evals), fn=firm_name, cf=canonical_firm:
        self.log(f"■ Breach check: {n} eval(s), firm_name='{fn}', canonical='{cf}'"))

    for ev in all_evals:
        if not ev:
            continue

        # Skip deleted

```

```

if ev.get("_deleted"):
    continue

# Check if eval belongs to this prop firm (with alias support)
ev_firm = ev.get("Prop Firm", "")
ev_canonical = self._FIRM_MAP.get(ev_firm, ev_firm)
firm_match = (ev_canonical == canonical_firm or ev_firm == firm_name)
if not firm_match:
    # Check aliases: firm name variants that refer to the same firm
    _FIRM_ALIASES = {
        "funded next": ["fundednext"],
        "fundednext": ["funded next"],
        "my funded futures": ["mffu"],
        "mffu": ["my funded futures"],
    }
    ev_aliases = _FIRM_ALIASES.get(ev_firm.lower(), [])
    firm_match = firm_name.lower() in ev_aliases or canonical_firm.lower() in ev_aliases
if not firm_match:
    continue

acct_challenge = (ev.get("Account #") or "").strip()
acct_funded = (ev.get("Account #.1") or "").strip()
current_status_p1 = (ev.get("Status P1") or "").strip().lower()
current_status = (ev.get("Status") or "").strip().lower()

self.root.after(0, lambda ac=acct_challenge, af=acct_funded, sp=current_status_p1,
                s=current_status:
    self.log(f"■ Breach eval: Acct#='{ac}' Acct#.1='{af}' P1='{sp}' Status='{s}'"))

# Skip if already marked as failed/breached
if any(kw in current_status_p1 for kw in self._INACTIVE_KEYWORDS):
    if not acct_funded or any(kw in current_status for kw in self._INACTIVE_KEYWORDS):
        self.root.after(0, lambda: self.log(f"■ Breach check: already inactive, skipping"))
        continue

# Find matching account in mapping
matched_info = None
for acct_key in [acct_challenge, acct_funded]:
    if not acct_key:
        continue
    if acct_key in tv_to_info:
        matched_info = tv_to_info[acct_key]
        break
# Partial match
for tv_name, info in tv_to_info.items():
    if tv_name in acct_key or acct_key in tv_name:
        matched_info = info
        break
if matched_info:
    break

if not matched_info:
    # Fallback: match by Account Size to any mapping entry
    ev_size_str = (ev.get("Account Size") or "").strip()
    import re as _re
    size_match = _re.search(r'(\d[\d,]*)', ev_size_str.replace(",", ""))
    if size_match:
        ev_size = int(size_match.group(1))
        for _lid, info in acct_mapping.items():
            sb = info.get("starting_balance")

```

```

        if sb and int(sb) == ev_size:
            matched_info = info
            self.root.after(0, lambda sz=ev_size:
                self.log(f"■ Breach: matched by Account Size ${sz:,}"))
            break

    if not matched_info:
        self.root.after(0, lambda ac=acct_challenge, af=acct_funded:
            self.log(f"■ Breach check: no mapping match for Acct#='{ac}' Acct#.1='{af}'"))
        continue

    # ■■ Determine account status: Fail / Pass / In Progress ■■
    breached = matched_info.get("breached")
    acct_display = acct_challenge or acct_funded
    changes = []

    # Detect current phase for this eval
    has_funded_acct = bool(acct_funded)
    if has_funded_acct:
        phase_for_status = "Funded"
        status_field = "Status"
        current_status_check = current_status
    else:
        phase_for_status = "Challenge"
        status_field = "Status P1"
        current_status_check = current_status_p1

    # Skip if already marked with a terminal status
    if any(kw in current_status_check for kw in self._INACTIVE_KEYWORDS):
        self.root.after(0, lambda: self.log(f"■ Status check: already inactive, skipping"))
        continue

    if breached and breached != 0:
        # Account is breached – get detailed info from API if available
        breach_reason = matched_info.get("breachedby") or "Breached"
        account_id = matched_info.get("account_id")
        if account_id and hasattr(account, 'get_account_overview'):
            try:
                overview = account.get_account_overview(account_id)
                if overview:
                    details = overview.get("account_details", {})
                    breach_reason = details.get("breached_by") or breach_reason
            except Exception:
                pass

        ev[status_field] = "Fail"
        changes.append(f"{status_field}='Fail'")
        breached_count += 1
        self.root.after(0, lambda a=acct_display, c=", ".join(changes):
            self.log(f" ■ BREACHED: {a} → {c}"))
    elif self.prop_firm_mgr:
        # ■■ Balance-based auto-status: Pass / In Progress / Fail ■■
        # Prefer live profit_target / min_equity from dashboard API
        acct_balance = matched_info.get("balance")
        acct_starting = matched_info.get("starting_balance") or matched_info.get(
            "initial_balance")
        live_target = matched_info.get("profit_target")
        live_min_eq = matched_info.get("min_equity")
        if acct_balance is not None:
            try:

```

```

bal = float(acct_balance)
start = float(acct_starting) if acct_starting else 50000.0

if live_min_eq is not None or live_target is not None:
    # ■■ Use live values from prop firm dashboard ■■
    min_eq = float(live_min_eq) if live_min_eq is not None else None
    target = float(live_target) if live_target is not None else None

    if min_eq is not None and bal < min_eq:
        computed = "Fail"
    elif target is not None and bal >= (start + target):
        computed = "Pass"
    elif abs(bal - start) > 0.50:
        # Balance differs from starting → a trade was placed
        computed = "In Progress"
    elif min_eq is not None and target is not None:
        computed = "In Progress"
    else:
        computed = self.prop_firm_mgr.compute_account_status(
            canonical_firm, phase_for_status, bal, start,
            breached=False)
else:
    # ■■ No live data – fall back to hardcoded targets ■■
    computed = self.prop_firm_mgr.compute_account_status(
        canonical_firm, phase_for_status, bal, start,
        breached=False)

# If balance is exactly the starting balance, keep "Not Started"
if computed == "In Progress" and abs(bal - start) < 0.50:
    if current_status_check in ("not started", ""):
        computed = None # no change – no trade placed yet

# Only update if status actually changed
last_key = f"{acct_display}_{status_field}"
last_known = self._last_known_statuses.get(last_key)
if computed and computed.lower(
    ) != current_status_check and computed != last_known:
    self._last_known_statuses[last_key] = computed
    if computed == "Pass":
        ev[status_field] = "Pass"
        changes.append(f"{status_field}='Pass'")
        self.root.after(0, lambda a=acct_display, b=bal, s=start:
            self.log(f"    ■ PASS: {a} – balance=${b:,.2f} (start=${s:,.0f})")
    elif computed == "In Progress":
        ev[status_field] = "In Progress"
        changes.append(f"{status_field}='In Progress'")
        self.root.after(0, lambda a=acct_display, b=bal, s=start:
            self.log(
                f"    ■ IN PROGRESS: {a} – balance=${b:,.2f} (start=${s:,.0f})")
    elif computed == "Fail":
        ev[status_field] = "Fail"
        changes.append(f"{status_field}='Fail'")
        breached_count += 1
        self.root.after(0, lambda a=acct_display, b=bal, s=start:
            self.log(f"    ■ FAIL: {a} – balance=${b:,.2f} (start=${s:,.0f})")
except (ValueError, TypeError) as _e:
    self.root.after(0, lambda a=acct_display, b=acct_balance, err=str(_e):
        self.log(f"■ {a}: couldn't parse balance '{b}': {err}", "WARN"))

if changes:

```

```

        updated_evals.append(ev)

    status_updates = len(updated_evals)
    if not status_updates:
        self.root.after(0, lambda:
            self.log(f"■ {firm_name}: No status changes detected"))
        return

    self.root.after(0, lambda c=status_updates, b=breached_count:
        self.log(f"■ {firm_name}: {c} status update(s) ({b} breached) – pushing to dashboard...")

    # Push to dashboard
    email = self.client_email_entry.get().strip()
    dashboard_url = self.url_entry.get().strip().rstrip('/')
    if not email or not dashboard_url:
        return

    payload = {
        "email": email,
        "evaluations": all_evals,
        "statistics": {},
        "dropdown_options": {},
        "force_fields": ["Status P1", "Status"],
    }

    try:
        response = _gzip_post(
            f"{dashboard_url}/api/client/push",
            payload,
            timeout=30
        )
        if response.status_code == 200:
            data = response.json()
            if data.get("status") == "success":
                self.root.after(0, lambda c=status_updates, b=breached_count:
                    self.log(f"■ Auto-status: {c} update(s) ({b} failed) → synced to dashboard"))
            else:
                self.root.after(0, lambda m=data.get('message', 'Unknown'):
                    self.log(f"■ Breach sync response: {m}", "WARN"))
        else:
            self.root.after(0, lambda s=response.status_code:
                self.log(f"■ Breach sync failed: HTTP {s}", "WARN"))
    except Exception as e:
        self.root.after(0, lambda err=str(e):
            self.log(f"■ Breach sync error: {err}", "WARN"))

except Exception as e:
    self.root.after(0, lambda err=str(e):
        self.log(f"■ {firm_name} breach detection failed: {err}", "WARN"))

```

Method: TradeOpssAIApp._get_broker_for_firm

```
def _get_broker_for_firm(self, firm_name)
```

Get the connected broker account for a specific prop firm.

Step by step (top-level body)

1. Assigns conn = self._broker_connections.get(firm_name).

2. if conn and conn.get('account'): branches conditionally.
3. if conn is not None: branches conditionally.
4. if 'topstep' in firm_name.lower(): branches conditionally.
5. Assigns platform = self.broker_var.get().
6. if platform == 'Tradovate' and self.tradovate_account: branches conditionally.
7. if platform == 'TopStepX' and self.topstepx_account: branches conditionally.
8. return None.

Code (copy-paste safe)

```
def _get_broker_for_firm(self, firm_name):
    """Get the connected broker account for a specific prop firm."""
    conn = self._broker_connections.get(firm_name)
    if conn and conn.get("account"):
        return conn["account"]
    # If the firm has a row in broker connections but isn't connected, do NOT
    # fall back – it means the user chose not to connect this firm.
    if conn is not None:
        return None
    # Legacy fallback: only for setups without multi-firm broker panel
    # Auto-detect TopStep firms to use TopStepX
    if "topstep" in firm_name.lower():
        if self.topstepx_account:
            return self.topstepx_account
        return None
    platform = self.broker_var.get()
    if platform == "Tradovate" and self.tradovate_account:
        return self.tradovate_account
    if platform == "TopStepX" and self.topstepx_account:
        return self.topstepx_account
    return None
```

Method: TradeOpssAIApp._get_trade_config

```
def _get_trade_config(self)
    Get current trade configuration from blueprint.
```

Step by step (top-level body)

1. if not self.prop_firm_mgr: branches conditionally.
2. Assigns firm = self.prop_firm_var.get().
3. Assigns phase = self.phase_var.get().
4. Assigns size = self.acct_size_var.get().
5. Assigns config = self.prop_firm_mgr.get_strategy_config(firm, phase, size).
6. return config.

Code (copy-paste safe)

```
def _get_trade_config(self):
    """Get current trade configuration from blueprint."""
    if not self.prop_firm_mgr:
        return None
```

```

firm = self.prop_firm_var.get()
phase = self.phase_var.get()
size = self.acct_size_var.get()
config = self.prop_firm_mgr.get_strategy_config(firm, phase, size)
return config

```

Method: TradeOpssAIApp._get_mt5_trading_api

```
def _get_mt5_trading_api(self)
```

Get or create the MT5 trading API from companion's existing MT5 connection.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **if** self.trading_api and hasattr(self.trading_api, 'is_connected') and ...: branches conditionally.
2. Assigns login = self.mt5_login.get().strip().
3. Assigns pwd = self.mt5_password.get().strip().
4. Assigns server = self.mt5_server.get().strip().
5. **if** login and pwd and server: branches conditionally.
6. **return** None.

Code (copy-paste safe)

```

def _get_mt5_trading_api(self):
    """Get or create the MT5 trading API from companion's existing MT5 connection."""
    if self.trading_api and hasattr(self.trading_api, 'is_connected') and self.trading_api.is_connected:
        return self.trading_api
    # Try to create from companion's MT5 credentials
    login = self.mt5_login.get().strip()
    pwd = self.mt5_password.get().strip()
    server = self.mt5_server.get().strip()
    if login and pwd and server:
        try:
            self.trading_api = MT5API(login, pwd, server)
            if self.trading_api.connect():
                return self.trading_api
        except Exception as e:
            self.log(f"MT5 trading API connection failed: {e}", "ERROR")
    return None

```

Method: TradeOpssAIApp._ensure_mt5_for_signals

```
def _ensure_mt5_for_signals(self)
```

Ensure MT5 is initialized so indicators can fetch price data. Works for both hedging and non-hedging clients: - If pusher already connected MT5, it's already initialized → returns True - Otherwise, tries to

initialize from saved credentials Returns: bool: True if MT5 is ready for price data queries.

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **if not MT5_AVAILABLE:** branches conditionally.
2. **if self.pusher.connected:** branches conditionally.
3. **if mt5.terminal_info():** branches conditionally.
4. Assigns `login = self.mt5_login.get().strip()`.
5. Assigns `pwd = self.mt5_password.get().strip()`.
6. Assigns `server = self.mt5_server.get().strip()`.
7. **if login and pwd and server:** branches conditionally.
8. **return False.**

Code (copy-paste safe)

```
def _ensure_mt5_for_signals(self):
    """Ensure MT5 is initialized so indicators can fetch price data.

    Works for both hedging and non-hedging clients:
    - If pusher already connected MT5, it's already initialized → returns True
    - Otherwise, tries to initialize from saved credentials

    Returns:
        bool: True if MT5 is ready for price data queries.
    """
    if not MT5_AVAILABLE:
        return False
    # Already connected via pusher?
    if self.pusher.connected:
        return True
    # Already connected via trading API?
    if mt5.terminal_info():
        return True
    # Try to connect using saved credentials
    login = self.mt5_login.get().strip()
    pwd = self.mt5_password.get().strip()
    server = self.mt5_server.get().strip()
    if login and pwd and server:
        success, msg = self.pusher.connect_mt5(login, pwd, server)
        if success:
            self.log("■ MT5 connected for signal data")
            return True
        self.log(f"■ MT5 auto-connect failed: {msg}", "WARN")
    return False
```

Method: TradeOpssAIApp._get_daily_bias

```
def _get_daily_bias(self, firms)
```

Get or create today's direction bias per prop firm. Persisted to trader_bias.json so it survives app restarts. Resets automatically on a new calendar day (EAT).

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **dict comprehension** — Builds a dict in one expression (`{k: v for k, v in items}`) — same intent as a list comprehension but for mappings.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to `sum`, `any`, `all`, or `max` where the intermediate list would be wasted.

Step by step (top-level body)

1. Lazy import from `datetime`.
2. Assigns `EAT = timezone(timedelta(hours=3))`.
3. Assigns `today_str = datetime.now(EAT).strftime('%Y-%m-%d')`.
4. Assigns `bias_path = os.path.join(os.path.dirname(__file__), 'trader_bias.json')`.
5. Assigns `saved = {}`.
6. **if** `os.path.exists(bias_path)`: branches conditionally.
7. **if** `saved.get('date') != today_str`: branches conditionally.
8. Assigns `firm_bias = saved.get('firms', {})`.
9. Assigns `changed = False`.
10. **for** `firm in sorted(firms)`: iterates.
11. **if** `changed or saved.get('date') != today_str`: branches conditionally.
12. **return** `{f: firm_bias[f] for f in firms}`.

Code (copy-paste safe)

```
def _get_daily_bias(self, firms):
    """Get or create today's direction bias per prop firm.
    Persisted to trader_bias.json so it survives app restarts.
    Resets automatically on a new calendar day (EAT)."""
    from datetime import datetime, timedelta, timezone
    EAT = timezone(timedelta(hours=3))
    today_str = datetime.now(EAT).strftime("%Y-%m-%d")

    bias_path = os.path.join(os.path.dirname(__file__), "trader_bias.json")
    saved = {}
    if os.path.exists(bias_path):
        try:
            with open(bias_path, 'r') as f:
                saved = json.load(f)
        except Exception:
            saved = {}
```

```

# Reset if date changed
if saved.get("date") != today_str:
    saved = {"date": today_str, "firms": {}}

firm_bias = saved.get("firms", {})
changed = False
# Assign in a deterministic order so balancing is reproducible
for firm in sorted(firms):
    if firm not in firm_bias:
        # Diversify: pick the direction held by FEWER existing firms.
        # This prevents every firm from coincidentally landing on BUY.
        buys = sum(1 for d in firm_bias.values() if d == "buy")
        sells = sum(1 for d in firm_bias.values() if d == "sell")
        if buys > sells:
            firm_bias[firm] = "sell"
        elif sells > buys:
            firm_bias[firm] = "buy"
        else:
            # Tie (or first firm) – random
            firm_bias[firm] = random.choice(["buy", "sell"])
        changed = True

if changed or saved.get("date") != today_str:
    saved["date"] = today_str
    saved["firms"] = firm_bias
    try:
        with open(bias_path, 'w') as f:
            json.dump(saved, f, indent=2)
    except Exception:
        pass

return {f: firm_bias[f] for f in firms}

```

Method: TradeOpssAIApp._get_indicator_map

```

@classmethod
def _get_indicator_map(cls)

    Build indicator map lazily (needs imports to be resolved).

```

Why these Python concepts were used

- **@classmethod** — Marked **@classmethod** — receives the class itself as the first argument. The standard pattern for alternate constructors that build an instance from a different input shape (e.g., from a dict, a CSV row, or an MT5 order tuple).
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

Step by step (top-level body)

1. if `cls._SIGNAL_INDICATORS` is not `None`: branches conditionally.
2. Assigns `indicators = {}`.
3. **try** block with 1 **except** clause.
4. **try** block with 1 **except** clause.

5. **try** block with 1 **except** clause.
6. **try** block with 1 **except** clause.
7. **try** block with 1 **except** clause.
8. **try** block with 1 **except** clause.
9. **try** block with 1 **except** clause.
10. Assigns `cls._SIGNAL_INDICATORS = indicators`.
11. **return** indicators.

Code (copy-paste safe)

```
def _get_indicator_map(cls):
    """Build indicator map lazily (needs imports to be resolved)."""
    if cls._SIGNAL_INDICATORS is not None:
        return cls._SIGNAL_INDICATORS
    indicators = {}
    try:
        indicators["RSI"] = (get_rsi_signal, {"buy"}, {"sell"})
    except Exception:
        pass
    try:
        indicators["MACD"] = (get_macd_signal, {"buy"}, {"sell"})
    except Exception:
        pass
    try:
        indicators["Stochastic"] = (get_stochastic_signal, {"buy"}, {"sell"})
    except Exception:
        pass
    try:
        indicators["CCI"] = (get_cci_signal, {"buy"}, {"sell"})
    except Exception:
        pass
    try:
        indicators["Supertrend"] = (get_supertrend_signal, {"bullish"}, {"bearish"})
    except Exception:
        pass
    try:
        indicators["Momentum"] = (get_momentum_signal, {"bullish"}, {"bearish"})
    except Exception:
        pass
    try:
        indicators["BollingerBands"] = (get_bb_signal, {"lower"}, {"upper"})
    except Exception:
        pass
    cls._SIGNAL_INDICATORS = indicators
    return indicators
```

Method: `TradeOpssAIApp._get_signal_direction`

```
def _get_signal_direction(self, mt5_symbol, timeframe=None, num_indicators=3)
```

Generate a trade direction by polling a random subset of indicators. Picks `num\_indicators` random indicators, queries each on the given MT5 symbol, and uses majority vote to decide buy vs sell. Falls back to random if no indicators produce a signal.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **if not MT5_AVAILABLE:** branches conditionally.
2. Assigns `mt5_mod = mt5`.
3. **if timeframe is None:** branches conditionally.
4. **if not mt5_mod.terminal_info():** branches conditionally.
5. Assigns `indicators = self._get_indicator_map()`.
6. **if not indicators:** branches conditionally.
7. Assigns `available = list(indicators.keys())`.
8. Assigns `pick_count = min(num_indicators, len(available))`.
9. Assigns `chosen = random.sample(available, pick_count)`.
10. Assigns `buy_votes = 0`.
11. Assigns `sell_votes = 0`.
12. Assigns `details = []`.
13. **for name in chosen:** iterates.
14. Assigns `detail_str = ', '.join(details)`.
15. ... and 3 more statement(s) in the body.

Code (copy-paste safe)

```
def _get_signal_direction(self, mt5_symbol, timeframe=None, num_indicators=3):
    """Generate a trade direction by polling a random subset of indicators.

    Picks `num_indicators` random indicators, queries each on the given
    MT5 symbol, and uses majority vote to decide buy vs sell.
    Falls back to random if no indicators produce a signal.
    """
    if not MT5_AVAILABLE:
        self.root.after(0, lambda: self.log("■ MetaTrader5 import unavailable – using random direction",
            "WARN"))
        return random.choice(["buy", "sell"])
    mt5_mod = mt5
    if timeframe is None:
        timeframe = mt5_mod.TIMEFRAME_M5
```

```

# Ensure MT5 is connected (non-hedging clients may not have it open)
if not mt5_mod.terminal_info():
    self._ensure_mt5_for_signals()
    if not mt5_mod.terminal_info():
        self.root.after(0, lambda: self.log("■ MT5 not connected – using random direction", "WARN"))
        return random.choice(["buy", "sell"])

indicators = self._get_indicator_map()
if not indicators:
    self.root.after(0, lambda: self.log("■ No signal indicators available – using random", "WARN"))
    return random.choice(["buy", "sell"])

# Pick a random subset
available = list(indicators.keys())
pick_count = min(num_indicators, len(available))
chosen = random.sample(available, pick_count)

buy_votes = 0
sell_votes = 0
details = []

for name in chosen:
    func, buy_vals, sell_vals = indicators[name]
    try:
        result = func(mt5_symbol, timeframe)
        if result is None:
            details.append(f"{name}=neutral")
            continue
        sig = result.lower() if isinstance(result, str) else str(result).lower()
        if sig in buy_vals:
            buy_votes += 1
            details.append(f"{name}=BUY")
        elif sig in sell_vals:
            sell_votes += 1
            details.append(f"{name}=SELL")
        else:
            details.append(f"{name}={sig}")
    except Exception as e:
        details.append(f"{name}=err")

detail_str = ", ".join(details)
if buy_votes > sell_votes:
    direction = "buy"
elif sell_votes > buy_votes:
    direction = "sell"
else:
    direction = random.choice(["buy", "sell"])
detail_str += " (tie→random)"

self.root.after(0, lambda d=detail_str, dir=direction:
    self.log(f" ■ Signal vote: {d} → {dir.upper()}"))
return direction

```

Method: TradeOpssAIApp.save_config

```
def save_config(self)

    Save configuration to file.
```

Why these Python concepts were used

- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.

Step by step (top-level body)

1. Assigns `config = {'dashboard_url': self.url_entry.get(), 'client_email': s....`
2. `if hasattr(self, 'broker_var')`: branches conditionally.
3. Assigns `config_path = os.path.join(os.path.dirname(__file__), 'trader_config.js....`
4. `with open(config_path, 'w')`: enters a context manager.
5. Calls `self.log(...)` for its side effect.
6. Calls `messagebox.showinfo(...)` for its side effect.

Code (copy-paste safe)

```
def save_config(self):
    """Save configuration to file."""
    config = {
        "dashboard_url": self.url_entry.get(), # Save dynamic URL
        "client_email": self.client_email_entry.get(),
        "sheet_url": self.sheet_url_entry.get(),
        "mt5_login": self.mt5_login.get(),
        "mt5_server": self.mt5_server.get()
    }
    # Trading engine settings
    if hasattr(self, 'broker_var'):
        config["broker_platform"] = self.broker_var.get()
        config["trading_mode"] = self.trading_mode_var.get()
        config["prop_firm"] = self.prop_firm_var.get()
        config["phase"] = self.phase_var.get()
        config["account_size"] = self.acct_size_var.get()
        config["hedge_mode"] = self.hedge_mode_var.get()
        config["direction"] = self.direction_var.get()
        config["strategy"] = self.strategy_var.get()

    config_path = os.path.join(os.path.dirname(__file__), "trader_config.json")
    with open(config_path, 'w') as f:
        json.dump(config, f, indent=2)

    self.log("Configuration saved")
    messagebox.showinfo("Saved", "Configuration saved successfully")
```

Method: TradeOpssAIApp.load_config

```
def load_config(self)

    Load configuration from file.
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `config_path = os.path.join(os.path.dirname(__file__), 'trader_config.js....`
2. `if os.path.exists(config_path):` branches conditionally.

Code (copy-paste safe)

```
def load_config(self):
    """Load configuration from file."""
    config_path = os.path.join(os.path.dirname(__file__), "trader_config.json")
    if os.path.exists(config_path):
        try:
            with open(config_path, 'r') as f:
                config = json.load(f)

            # Do NOT load dashboard_url - keep hardcoded Trade0pps
            # saved_url = config.get('dashboard_url')
            # if saved_url:
            #     self.url_entry.delete(0, tk.END)
            #     self.url_entry.insert(0, saved_url)

            if config.get('client_email'):
                self.client_email_entry.delete(0, tk.END)
                self.client_email_entry.insert(0, config.get('client_email', ''))

            if config.get('sheet_url'):
                self.sheet_url_entry.delete(0, tk.END)
                self.sheet_url_entry.insert(0, config.get('sheet_url', ''))

            if config.get('mt5_login'):
                self.mt5_login.delete(0, tk.END)
                self.mt5_login.insert(0, config.get('mt5_login', ''))

            if config.get('mt5_server'):
                self.mt5_server.delete(0, tk.END)
                self.mt5_server.insert(0, config.get('mt5_server', ''))

            # Trading engine settings
            if hasattr(self, 'broker_var'):
                if config.get('broker_platform'):
                    self.broker_var.set(config['broker_platform'])
                if config.get('trading_mode'):
                    self.trading_mode_var.set(config['trading_mode'])
                if config.get('prop_firm'):
                    self.prop_firm_var.set(config['prop_firm'])
                    self._on_prop_firm_change()
                if config.get('phase'):
                    self.phase_var.set(config['phase'])
                if config.get('account_size'):
                    self.acct_size_var.set(config['account_size'])
                if config.get('hedge_mode'):
                    self.hedge_mode_var.set(config['hedge_mode'])
                if config.get('direction'):
                    self.direction_var.set(config['direction'])
                if config.get('strategy'):
                    self.strategy_var.set(config['strategy'])
```

```

        self.log("Configuration loaded")
    except Exception as e:
        self.log(f"Failed to load config: {e}", "ERROR")

```

Method: TradeOpssAIApp.run

```
def run(self)

    Run the application.
```

Step by step (top-level body)

1. Calls `self.root.mainloop(...)` for its side effect.

Code (copy-paste safe)

```
def run(self):
    """Run the application."""
    self.root.mainloop()
```

Functions

```
def _gzip_post(url, payload, timeout=120, **kwargs)

    POST JSON payload compressed with gzip. Sends Content-Encoding: gzip so the server can decompress it. Falls back to normal JSON POST if compression somehow fails.
```

Why these Python concepts were used

- ****kwargs** — Accepts arbitrary keyword arguments. Usually for forwarding to another function (a thin wrapper) or to support optional named parameters without listing them all in the signature.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def _gzip_post(url, payload, timeout=120, **kwargs):
    """
    POST JSON payload compressed with gzip.
    Sends Content-Encoding: gzip so the server can decompress it.
    Falls back to normal JSON POST if compression somehow fails.
    """
    try:
        raw = json.dumps(payload).encode('utf-8')
        compressed = _gzip.compress(raw, compresslevel=6)
        headers = kwargs.pop('headers', {})
        headers['Content-Type'] = 'application/json'
        headers['Content-Encoding'] = 'gzip'
        return requests.post(url, data=compressed, headers=headers, timeout=timeout, **kwargs)
    except Exception:
        # Fallback: plain JSON
        return requests.post(url, json=payload, headers={'Content-Type': 'application/json'}, timeout=timeout, **kwargs)
```

```
def _to_topstepx_symbol(sym)
```

Convert Tradovate-style futures symbol to TopStepX-style. Tradovate uses a single-digit year (e.g. NQM6 = NQ June 2026). TopStepX expects a two-digit year (e.g. NQM26). Recognized month codes: F G H J K M N Q U V X Z Returns the input unchanged if it doesn't look like a futures symbol.

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **if** not sym: branches conditionally.
2. Assigns `s = str(sym).strip().upper()`.
3. Assigns `m = re.match('^[A-Z]+([FGHJKMNQUVXZ])(\d{1,2})$', s)`.
4. **if** not m: branches conditionally.
5. Assigns `(root, month, year) = (m.group(1), m.group(2), m.group(3))`.
6. **if** len(year) == 2: branches conditionally.
7. Lazy import from datetime.
8. Assigns `cur_year = _dt.now().year`.
9. Assigns `decade = cur_year // 10 * 10`.
10. Assigns `candidate = decade + int(year)`.
11. **if** candidate < cur_year - 1: branches conditionally.
12. **return** `f'{root}{month}{str(candidate % 100).zfill(2)}'`.

Code (copy-paste safe)

```
def _to_topstepx_symbol(sym):
    """Convert Tradovate-style futures symbol to TopStepX-style.

    Tradovate uses a single-digit year (e.g. NQM6 = NQ June 2026).
    TopStepX expects a two-digit year (e.g. NQM26).

    Recognized month codes: F G H J K M N Q U V X Z
    Returns the input unchanged if it doesn't look like a futures symbol.
    """
    if not sym:
        return sym
    s = str(sym).strip().upper()
    m = re.match(r'^([A-Z]+)([FGHJKMNQUVXZ])(\d{1,2})$', s)
    if not m:
        return s
    root, month, year = m.group(1), m.group(2), m.group(3)
    if len(year) == 2:
        return s # Already in TSX format
    # Expand single-digit year using current decade, rolling forward if needed
    from datetime import datetime as _dt
    cur_year = _dt.now().year
    decade = (cur_year // 10) * 10
    candidate = decade + int(year)
```

```

if candidate < cur_year - 1:
    candidate += 10
return f"{root}{month}{str(candidate % 100).zfill(2)}"

```

```
def main()
```

Main entry point.

Step by step (top-level body)

1. if GUI_AVAILABLE: branches conditionally (with an **else/elif** arm).

Code (copy-paste safe)

```

def main():
    """Main entry point."""
    if GUI_AVAILABLE:
        app = TradeOpssAIApp()
        app.run()
    else:
        print("=" * 50)
        print("Tradeopss AI - Console Mode")
        print("=" * 50)
        print("\nGUI not available. Install tkinter to use the graphical interface.")
        print("\nUsage:")
        print("  1. Set your API key in the dashboard")
        print("  2. Use the MT5DataPusher class programmatically")
        print("\nExample:")
        print("  pusher = MT5DataPusher('http://localhost:5001', 'your-api-key')")
        print("  pusher.connect_mt5(login, password, server)")
        print("  pusher.push_to_dashboard('ClientName', 'AdminName', 'TraderName')")

```

Checkpoint — what works now. Run `python trader_companion/trader_app.py` and the Tkinter window opens — broker selection on first launch, then the main tabbed view that exposes everything you have built in phases 3 through 7.

Chapter 11 — Phase 9 — Web dashboard

A Flask application that admins, managers, and traders log into through a browser. We build the supporting services bottom-up — email, notes, API keys, sheet synchronisation, payout calculations — and finally the Flask app factory in `app.py` that registers the routes and starts the request lifecycle.

Files we build in this phase, in order:

- `dashboard/email_service.py`
- `dashboard/notes_service.py`
- `dashboard/manage_api_keys.py`
- `dashboard/api_client.py`
- `dashboard/utils/sheet_helper.py`
- `dashboard/utils/trade_matcher.py`
- `dashboard/calc_like_sheet.py`
- `dashboard/phase_manager.py`
- `dashboard/watermark_service.py`
- `dashboard/financial_overview.py`
- `dashboard/scheduler.py`
- `dashboard/app.py`

Python concepts you'll meet in this phase

- f-strings (12) · Exceptions (11) · Comprehensions (7) · Context managers (with-statements) (6) · Lambdas (6) · Classes and objects (5) · Type hints (4) · Dunder methods (3)

Each is explained in Chapter 2 (the primer). The number in parentheses is how many of this phase's files use that concept.

Step 11.1 — Build `dashboard/email_service.py`

What to do. Create the file `dashboard/email_service.py` in your project root and paste in the code shown in this step. The file is **260** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from `requirements.txt`.

What this file is for. SMTP-backed transactional email helpers — password resets, payout notifications, alert digests.

`dashboard/email_service.py`

260 loc · 0 classes · 4 functions · 5 imports

SMTP-backed transactional email helpers — password resets, payout notifications, alert digests. It defines **4** module-level functions, across **260** lines. Module docstring: *Email Service Module for Trading Dashboard*

Module docstring

Email Service Module for Trading Dashboard Sends email notifications for password changes, account creation, etc.

Python concepts demonstrated in this module

• Context managers (with-statements) • Exceptions • f-strings • Type hints

Imports — what each one is for

```
import smtplib
```

Why imported. SMTP client — sending email. Pairs with the email package, which builds the MIME message.

Used in this file. smtplib.SMTP

Example. `smtplib.send_message(msg)` # where msg is an EmailMessage

Other common scenarios.

- `SMTP_SSL(host, 465)` for implicit-TLS providers
- `starttls()` upgrades a plain connection to TLS
- `login(user, password)` authenticates

```
import os
```

Why imported. Operating-system interface. Path manipulation, environment variables, working-directory operations, exit codes, and thin wrappers around POSIX/Windows syscalls.

Used in this file. `os.getenv`

Example. `os.path.join(root, 'subdir', 'file.txt')` # joins with the OS-correct separator

Other common scenarios.

- `os.environ.get('API_KEY')` to read environment variables
- `os.makedirs(path, exist_ok=True)` to create nested directories
- `os.listdir(path)` to list a directory's contents
- `os.remove(path)` / `os.rename(src, dst)` to delete or move a file
- `os.getcwd()` / `os.chdir(path)` to inspect or change the working dir

```
from email.mime.text import MIMEText
```

Why imported. Build and parse email messages — headers, MIME parts, attachments. Pairs with `smtplib` for sending.

Used in this file. MIMEText

Example. `msg = EmailMessage(); msg['To'] = 'a@b.com'; msg.set_content('hi')`

Other common scenarios.

- `msg.add_attachment(data, maintype='application', subtype='pdf')`
- `email.utils.formataddr(('Name', 'a@b.com'))` builds an addr

`from email.mime.multipart import MIMEMultipart`

Why imported. Build and parse email messages — headers, MIME parts, attachments. Pairs with `smtpplib` for sending.

Used in this file. MIMEMultipart

Example. `msg = EmailMessage(); msg['To'] = 'a@b.com'; msg.set_content('hi')`

Other common scenarios.

- `msg.add_attachment(data, maintype='application', subtype='pdf')`
- `email.utils.formataddr(('Name', 'a@b.com'))` builds an addr

`from datetime import datetime`

Why imported. Dates, times, durations. The fundamental temporal types: `date`, `time`, `datetime`, `timedelta`, `timezone`.

Used in this file. `datetime`

Example. `datetime.now() - timedelta(days=7)` # one week ago

Other common scenarios.

- `datetime.strptime(s, '%Y-%m-%d')` parses a string
- `datetime.strftime(fmt)` formats one
- `datetime.fromtimestamp(n)` converts a Unix epoch
- `datetime.utcnow()` for UTC (better: `datetime.now(timezone.utc)`)

Module constants

Each line below is followed by a plain-English note explaining what it does and why it is written that way.

`SMTP_SERVER = os.getenv('SMTP_SERVER', 'smtp.gmail.com')`

Reads `SMTP_SERVER` from the environment, defaulting to `'smtp.gmail.com'` when not set — overridable per environment without a code change.

`SMTP_PORT = int(os.getenv('SMTP_PORT', '587'))`

Reads `SMTP_PORT` from the environment and converts it to `int` (env vars are always strings). Falls back to `'587'` when unset.

`SMTP_USERNAME = os.getenv('SMTP_USERNAME', '')`

Reads `SMTP_USERNAME` from the environment, defaulting to `"` when not set — overridable per environment without a code change.


```
SMTP_PASSWORD = os.getenv('SMTP_PASSWORD', '')
```

Reads SMTP_PASSWORD from the environment, defaulting to "" when not set — overridable per environment without a code change.

```
FROM_EMAIL = os.getenv('FROM_EMAIL', SMTP_USERNAME)
```

Reads FROM_EMAIL from the environment, defaulting to SMTP_USERNAME when not set — overridable per environment without a code change.

```
FROM_NAME = os.getenv('FROM_NAME', 'Trading Dashboard')
```

Reads FROM_NAME from the environment, defaulting to 'Trading Dashboard' when not set — overridable per environment without a code change.

```
EMAIL_ENABLED = os.getenv('EMAIL_ENABLED', 'false').lower() == 'true'
```

Boolean feature flag. Reads EMAIL_ENABLED from the environment, lowercases it, and compares to 'true' — producing a real Python bool. Defaults to 'false'.

Functions

```
def send_email(to_email: str, subject: str, html_body: str, text_body: str=None) -> bool
```

Send an email using SMTP. Returns True if successful, False otherwise.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., to_email: str, subject: str, html_body: str). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type -> bool. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **if** not EMAIL_ENABLED: branches conditionally.
2. **if** not SMTP_USERNAME or not SMTP_PASSWORD: branches conditionally.
3. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def send_email(to_email: str, subject: str, html_body: str, text_body: str = None) -> bool:
    """
    Send an email using SMTP.
```

```

Returns True if successful, False otherwise.
"""
if not EMAIL_ENABLED:
    print(f"[EMAIL DISABLED] Would send to {to_email}: {subject}")
    return True

if not SMTP_USERNAME or not SMTP_PASSWORD:
    print("[EMAIL ERROR] SMTP credentials not configured")
    return False

try:
    msg = MIMEMultipart('alternative')
    msg['Subject'] = subject
    msg['From'] = f"{FROM_NAME} <{FROM_EMAIL}>"
    msg['To'] = to_email

    # Add text version
    if text_body:
        msg.attach(MIMEText(text_body, 'plain'))

    # Add HTML version
    msg.attach(MIMEText(html_body, 'html'))

    # Connect and send
    with smtplib.SMTP(SMTP_SERVER, SMTP_PORT) as server:
        server.starttls()
        server.login(SMTP_USERNAME, SMTP_PASSWORD)
        server.send_message(msg)

    print(f"[EMAIL SENT] To: {to_email}, Subject: {subject}")
    return True

except Exception as e:
    print(f"[EMAIL ERROR] Failed to send to {to_email}: {str(e)}")
    return False

```

```
def send_password_changed_notification(to_email: str, username: str, changed_by: str='self') -> bool
```

Send notification when password is changed.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `to_email: str`, `username: str`, `changed_by: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> bool`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `timestamp = datetime.now().strftime('%B %d, %Y at %l:%M %p')`.
2. **if** `changed_by == 'self'`: branches conditionally (with an **else/elif** arm).

3. Assigns text_body = f'Hi {username}, your password was changed on {timestamp}....

4. return send_email(to_email, subject, html_body, text_body).

Code (copy-paste safe)

```
def send_password_changed_notification(to_email: str, username: str, changed_by: str = 'self') -> bool:
    """Send notification when password is changed."""
    timestamp = datetime.now().strftime("%B %d, %Y at %I:%M %p")

    if changed_by == 'self':
        subject = "Password Changed Successfully - Trading Dashboard"
        html_body = f"""
        <!DOCTYPE html>
        <html>
        <head>
            <style>
                body {{ font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif; }}
                .container {{ max-width: 600px; margin: 0 auto; padding: 20px; }}
                .header {{ background: linear-gradient(135deg,
                    #667eea 0%, #764ba2 100%); color: white; padding: 30px; text-align: center; border-radius: 10px; }}
                .content {{ background: #f8f9fa; padding: 30px; border-radius: 0 0 10px 10px; }}
                .success {{ color: #16a34a; font-size: 48px; }}
                .footer {{ text-align: center; margin-top: 20px; color: #666; font-size: 12px; }}
            </style>
        </head>
        <body>
            <div class="container">
                <div class="header">
                    <div class="success">✓</div>
                    <h1>Password Changed</h1>
                </div>
                <div class="content">
                    <p>Hi <strong>{username}</strong>,</p>
                    <p>Your password was successfully changed on <strong>{timestamp}</strong>.</p>
                    <p>If you did not make this change, please contact your administrator immediately and</p>
                    <hr style="border: none; border-top: 1px solid #ddd; margin: 20px 0;">
                    <p style="color: #666; font-size: 14px;">
                        <strong>Security Tip:</strong> Never share your password with anyone.
                        Our team will never ask for your password.
                    </p>
                </div>
                <div class="footer">
                    <p>Trading Dashboard Security Team</p>
                </div>
            </div>
        </body>
        </html>
        """
    else:
        subject = "Your Password Has Been Reset - Trading Dashboard"
        html_body = f"""
        <!DOCTYPE html>
        <html>
        <head>
            <style>
                body {{ font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif; }}
                .container {{ max-width: 600px; margin: 0 auto; padding: 20px; }}
                .header {{ background: linear-gradient(135deg,
                    #f59e0b 0%, #d97706 100%); color: white; padding: 30px; text-align: center; border-radius: 10px; }}
                .content {{ background: #f8f9fa; padding: 30px; border-radius: 0 0 10px 10px; }}
            </style>
        </head>
        <body>
            <div class="container">
                <div class="header">
                    <div class="success">✓</div>
                    <h1>Password Reset</h1>
                </div>
                <div class="content">
                    <p>Hi <strong>{username}</strong>,</p>
                    <p>Your password has been successfully reset.</p>
                    <p>If you did not make this change, please contact your administrator immediately and</p>
                    <hr style="border: none; border-top: 1px solid #ddd; margin: 20px 0;">
                    <p style="color: #666; font-size: 14px;">
                        <strong>Security Tip:</strong> Never share your password with anyone.
                        Our team will never ask for your password.
                    </p>
                </div>
                <div class="footer">
                    <p>Trading Dashboard Security Team</p>
                </div>
            </div>
        </body>
        </html>
        """
```

```

        .warning {{ color: #f59e0b; font-size: 48px; }}
        .footer {{ text-align: center; margin-top: 20px; color: #666; font-size: 12px; }}
    </style>
</head>
<body>
    <div class="container">
        <div class="header">
            <div class="warning">■</div>
            <h1>Password Reset</h1>
        </div>
        <div class="content">
            <p>Hi <strong>{username}</strong>,</p>
            <p>Your password was reset by an administrator on <strong>{timestamp}</strong>.</p>
            <p>You will be required to change your password when you next log in.</p>
            <p>If you did not request this change, please contact your administrator.</p>
            <hr style="border: none; border-top: 1px solid #ddd; margin: 20px 0;">
            <p style="color: #666; font-size: 14px;">
                <strong>Next Steps:</strong> Log in with your new temporary password and create a
            </p>
        </div>
        <div class="footer">
            <p>Trading Dashboard Security Team</p>
        </div>
    </div>
</body>
</html>
"""

```

```

text_body = f"Hi {username}, your password was changed on {timestamp}. If you did not make this change, please contact your administrator."

return send_email(to_email, subject, html_body, text_body)

```

```

def send_account_created_notification(to_email: str, username: str, temp_password: str,
    user_type: str) -> bool

```

Send notification when a new account is created.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `to_email: str`, `username: str`, `temp_password: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> bool`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `subject = 'Welcome to Trading Dashboard - Your Account is Ready'.`
2. Assigns `role_display = {'admin': 'Administrator', 'trader': 'Trader', 'client': ...}`
3. Assigns `html_body = f"""<!DOCTYPE html>\n <html>\n <head>\n ...`
4. Assigns `text_body = f'Welcome {username}! Your {role_display} account is read....`

5. **return** send_email(to_email, subject, html_body, text_body).

Code (copy-paste safe)

```
def send_account_created_notification(to_email: str, username: str, temp_password: str,
    user_type: str) -> bool:
    """Send notification when a new account is created."""
    subject = "Welcome to Trading Dashboard - Your Account is Ready"

    role_display = {
        'admin': 'Administrator',
        'trader': 'Trader',
        'client': 'Client'
    }.get(user_type, user_type.title())

    html_body = f"""
<!DOCTYPE html>
<html>
<head>
    <style>
        body {{ font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif; }}
        .container {{ max-width: 600px; margin: 0 auto; padding: 20px; }}
        .header {{ background: linear-gradient(135deg,
            #667eea 0%, #764ba2 100%); color: white; padding: 30px; text-align: center; border-radius: 10px; }}
        .content {{ background: #f8f9fa; padding: 30px; border-radius: 0 0 10px 10px; }}
        .credentials {{
            background: white; padding: 20px; border-radius: 8px; border: 2px solid #667eea; margin: 10px 0;
        }}
        .footer {{ text-align: center; margin-top: 20px; color: #666; font-size: 12px; }}
    </style>
</head>
<body>
    <div class="container">
        <div class="header">
            <h1>■ Welcome!</h1>
            <p>Your Trading Dashboard account is ready</p>
        </div>
        <div class="content">
            <p>Hi <strong>{username}</strong>,</p>
            <p>Your {role_display} account has been created. Here are your login credentials:</p>

            <div class="credentials">
                <p><strong>Username/Email:</strong> {username}</p>
                <p><strong>Temporary Password:</strong> {temp_password}</p>
                <p><strong>Role:</strong> {role_display}</p>
            </div>

            <p style="color: #dc2626;"><strong>■■ Important:</strong> You will be required to change your password</p>

            <hr style="border: none; border-top: 1px solid #ddd; margin: 20px 0;">
            <p style="color: #666; font-size: 14px;">
                <strong>Security Tips</strong>
                <ul>
                    <li>Never share your password</li>
                    <li>Use a strong, unique password</li>
                    <li>Log out when using shared devices</li>
                </ul>
            </p>
        </div>
        <div class="footer">
            <p>Trading Dashboard Security Team</p>
        </div>
    </div>
    """
```

```

        </div>
    </div>
</body>
</html>
"""

```

```

text_body = f"Welcome {username}! Your {role_display} account is ready. Username: {username}, Temp Pa

return send_email(to_email, subject, html_body, text_body)

```

```
def send_password_reset_with_temp(to_email: str, username: str, temp_password: str) -> bool
```

Send email with temporary password after admin reset.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `to_email: str`, `username: str`, `temp_password: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> bool`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `subject = 'Your Password Has Been Reset - Trading Dashboard'`.
2. Assigns `timestamp = datetime.now().strftime('%B %d, %Y at %I:%M %p')`.
3. Assigns `html_body = f"""\n <!DOCTYPE html>\n <html>\n <head>\n ....`
4. Assigns `text_body = f'Hi {username}, your password was reset on {timestamp}'. ....`
5. **return** `send_email(to_email, subject, html_body, text_body)`.

Code (copy-paste safe)

```

def send_password_reset_with_temp(to_email: str, username: str, temp_password: str) -> bool:
    """Send email with temporary password after admin reset."""
    subject = "Your Password Has Been Reset - Trading Dashboard"
    timestamp = datetime.now().strftime("%B %d, %Y at %I:%M %p")

    html_body = f"""
    <!DOCTYPE html>
    <html>
    <head>
        <style>
            body {{ font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif; }}
            .container {{ max-width: 600px; margin: 0 auto; padding: 20px; }}
            .header {{ background: linear-gradient(135deg,
                #f59e0b 0%, #d97706 100%); color: white; padding: 30px; text-align: center; border-radius:
            .content {{ background: #f8f9fa; padding: 30px; border-radius: 0 0 10px 10px; }}
            .credentials {{
                background: white; padding: 20px; border-radius: 8px; border: 2px solid #f59e0b; margin:
            .footer {{ text-align: center; margin-top: 20px; color: #666; font-size: 12px; }}
        </style>
    """

```

```

</head>
<body>
  <div class="container">
    <div class="header">
      <h1>■ Password Reset</h1>
    </div>
    <div class="content">
      <p>Hi <strong>{username}</strong>,</p>
      <p>Your password was reset by an administrator on <strong>{timestamp}</strong>.</p>

      <div class="credentials">
        <p><strong>Your Temporary Password:</strong></p>
        <p style="font-size: 24px; font-family: monospace; color: #667eea;">{temp_password}</p>
      </div>

      <p style="color: #dc2626;"><strong>■■ Important:</strong> You must change this password

      <p>If you did not request this reset, please contact your administrator immediately.</p>
    </div>
    <div class="footer">
      <p>Trading Dashboard Security Team</p>
    </div>
  </div>
</body>
</html>
"""

text_body = f"Hi {username}, your password was reset on {timestamp}. Temporary password: {temp_password}"

return send_email(to_email, subject, html_body, text_body)

```

Step 11.2 — Build dashboard/notes_service.py

What to do. Create the file `dashboard/notes_service.py` in your project root and paste in the code shown in this step. The file is **76** lines long; we walk through it piece by piece below.

Depends on. This file imports from internal modules built in earlier steps: `dashboard`. If any of those imports fail, jump back to the step that introduced them.

What this file is for. Per-account notes (free-text annotations admins leave on trader accounts).

dashboard/notes_service.py

76 loc · 0 classes · 3 functions · 2 imports

Per-account notes (free-text annotations admins leave on trader accounts). It defines **3** module-level functions, across **76** lines.

Python concepts demonstrated in this module

- Context managers (with-statements)
- Exceptions
- f-strings

Imports — what each one is for

```
from dashboard.database import get_connection
```

Why imported. Internal package — the Flask dashboard. See chapter on dashboard/.

Used in this file. `get_connection`

```
import logging
```

Why imported. Structured, levelled logging. Replaces `print()` with filterable, formattable output that can route to files, stderr, syslog, or HTTP endpoints.

Used in this file. `logging.error`

Example. `logging.getLogger(__name__).info('connected to %s', host)`

Other common scenarios.

- `.debug()` / `.info()` / `.warning()` / `.error()` / `.exception()`
- `logger.exception()` captures the current traceback automatically
- `logging.basicConfig(level=logging.INFO)` for quick setup
- Handlers and Formatters route logs to files / streams / syslog

Functions

```
def get_client_notes(client_id)
```

Returns a dictionary of notes for a client. Format: { row_index: { column_key: note_content } }

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def get_client_notes(client_id):
    """
    Returns a dictionary of notes for a client.
    Format: { row_index: { column_key: note_content } }
    """
    try:
        with get_connection() as conn:
            cursor = conn.cursor()
            cursor.execute(''
                SELECT row_index, column_key, note_content
                FROM cell_notes
```



```

        WHERE client_id = ?
''' , (client_id,))
rows = cursor.fetchall()

notes = {}
for row in rows:
    # Handle both dict-like (Row) and tuple results
    try:
        rid = row['row_index']
        col = row['column_key']
        content = row['note_content']
    except (TypeError, IndexError):
        rid = row[0]
        col = row[1]
        content = row[2]

    if rid not in notes:
        notes[rid] = {}
    notes[rid][col] = content
return notes
except Exception as e:
    logging.error(f"Error fetching notes for {client_id}: {e}")
return {}

```

```
def save_client_note(client_id, row_index, column_key, content, user)
```

Saves or updates a note.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def save_client_note(client_id, row_index, column_key, content, user):
    """
    Saves or updates a note.
    """
    try:
        with get_connection() as conn:
            cursor = conn.cursor()
            cursor.execute('''
                INSERT INTO cell_notes (client_id, row_index, column_key, note_content, created_by,
                    updated_at)
                VALUES (?, ?, ?, ?, ?, CURRENT_TIMESTAMP)
                ON CONFLICT (client_id, row_index, column_key) DO UPDATE
                SET note_content = EXCLUDED.note_content,
                    created_by = EXCLUDED.created_by,

```

```

        updated_at = CURRENT_TIMESTAMP
        '''', (client_id, row_index, column_key, content, user))
    conn.commit()
    return True
except Exception as e:
    logging.error(f"Error saving note for {client_id}: {e}")
    return False

```

```
def delete_client_note(client_id, row_index, column_key)
```

Deletes a specific note.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def delete_client_note(client_id, row_index, column_key):
    """
    Deletes a specific note.
    """
    try:
        with get_connection() as conn:
            cursor = conn.cursor()
            cursor.execute(''
                DELETE FROM cell_notes
                WHERE client_id = ? AND row_index = ? AND column_key = ?
            ''', (client_id, row_index, column_key))
            conn.commit()
            return True
    except Exception as e:
        logging.error(f"Error deleting note for {client_id}: {e}")
        return False

```

Step 11.3 — Build dashboard/manage_api_keys.py

What to do. Create the file `dashboard/manage_api_keys.py` in your project root and paste in the code shown in this step. The file is **231** lines long; we walk through it piece by piece below.

Depends on. This file imports from internal modules built in earlier steps: `dashboard`. If any of those imports fail, jump back to the step that introduced them.

What this file is for. Creates and revokes API keys used by the desktop companion to authenticate to the dashboard's ingestion endpoints.

dashboard/manage_api_keys.py

231 loc · 0 classes · 9 functions · 3 imports

Creates and revokes API keys used by the desktop companion to authenticate to the dashboard's ingestion endpoints. It defines **9** module-level functions, across **231** lines. Module docstring: *Secure API Key Management Tool*

Module docstring

Secure API Key Management Tool Manages API keys stored with SHA-256 hashing in SQLite database

Python concepts demonstrated in this module

- f-strings

Imports — what each one is for

```
import os
```

Why imported. Operating-system interface. Path manipulation, environment variables, working-directory operations, exit codes, and thin wrappers around POSIX/Windows syscalls.

Used in this file. `os.path`, `os.path.abspath`, `os.path.dirname`

Example. `os.path.join(root, 'subdir', 'file.txt')` # joins with the OS-correct separator

Other common scenarios.

- `os.environ.get('API_KEY')` to read environment variables
- `os.makedirs(path, exist_ok=True)` to create nested directories
- `os.listdir(path)` to list a directory's contents
- `os.remove(path)` / `os.rename(src, dst)` to delete or move a file
- `os.getcwd()` / `os.chdir(path)` to inspect or change the working dir

```
import sys
```

Why imported. Access to the running interpreter — `argv`, `stdin/stdout`, exit codes, the import search path, and the platform name.

Used in this file. `sys.path`, `sys.path.insert`

Example. `sys.exit(1)` # terminate the process with a non-zero status

Other common scenarios.

- `sys.argv[1:]` reads the command-line arguments
- `sys.path.insert(0, 'extra')` prepends to the import search path
- `sys.stderr.write(msg)` writes to standard error
- `sys.platform` tells you 'win32', 'linux', or 'darwin'
- `sys.modules` is the global cache of already-imported modules

```
from dashboard.database import generate_api_key
from dashboard.database import list_api_keys
from dashboard.database import revoke_api_key
from dashboard.database import delete_api_key
from dashboard.database import verify_admin_password
from dashboard.database import set_admin_password
from dashboard.database import get_audit_log
```

Why imported. Internal package — the Flask dashboard. See chapter on dashboard/.

Used in this file. `delete_api_key`, `generate_api_key`, `get_audit_log`, `list_api_keys`, `revoke_api_key`, `set_admin_password`, `verify_admin_password`

Functions

```
def print_header()
```

Step by step (top-level body)

1. Calls `print(...)` for its side effect.
2. Calls `print(...)` for its side effect.
3. Calls `print(...)` for its side effect.
4. Calls `print(...)` for its side effect.

Code (copy-paste safe)

```
def print_header():
    print("\n" + "=" * 60)
    print("    SECURE API KEY MANAGEMENT")
    print("    (Keys are hashed - never stored in plain text)")
    print("=" * 60)
```

```
def authenticate()
```

Require admin password to access management functions.

Step by step (top-level body)

1. Calls `print(...)` for its side effect.
2. Assigns `password = input('Enter admin password: ').strip()`.
3. `if not verify_admin_password('super_admin', password):` branches conditionally.
4. Calls `print(...)` for its side effect.
5. `return True`.

Code (copy-paste safe)

```
def authenticate():
    """Require admin password to access management functions."""
    print("\nAdmin authentication required.")
    password = input("Enter admin password: ").strip()

    if not verify_admin_password('super_admin', password):
        print("■ Invalid password!")
        return False

    print("✓ Authentication successful\n")
    return True
```

```
def list_keys()
```

Display all API keys (showing prefix only).

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns keys = list_api_keys().
2. Calls print(...) for its side effect.
3. Calls print(...) for its side effect.
4. Calls print(...) for its side effect.
5. if not keys: branches conditionally.
6. Calls print(...) for its side effect.
7. Calls print(...) for its side effect.
8. for key in keys: iterates.
9. Calls print(...) for its side effect.
10. Calls print(...) for its side effect.

Code (copy-paste safe)

```
def list_keys():
    """Display all API keys (showing prefix only)."""
    keys = list_api_keys()

    print("\n" + "=" * 60)
    print("API KEYS (Prefix Only - Full keys are never stored)")
    print("=" * 60)

    if not keys:
        print("No API keys found.")
        return

    print(f"{'Prefix':<15} {'Trader':<15} {'Admin':<15} {'Active':<8} {'Last Used'}")
```

```

print("-" * 75)

for key in keys:
    active = "✓" if key['is_active'] else "✗"
    last_used = key['last_used'] or "Never"
    if len(last_used) > 19:
        last_used = last_used[:19]
    print(f"{key['key_prefix']:<15} {key['trader']:<15} {key['admin']:<15} {active:<8} {last_used}")

print("-" * 75)
print(f"Total: {len(keys)} keys")

```

```
def generate_new_key()
```

Generate a new API key for a trader.

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Calls `print(...)` for its side effect.
2. Assigns `trader = input('Trader name: ').strip()`.
3. **if not trader**: branches conditionally.
4. Assigns `admin = input('Admin name (default: same as trader): ').strip()` or...
5. Assigns `client = input('Client name (optional): ').strip()`.
6. Calls `print(...)` for its side effect.
7. Assigns `api_key = generate_api_key(admin, trader, client)`.
8. **if api_key**: branches conditionally (with an **else/elif** arm).

Code (copy-paste safe)

```

def generate_new_key():
    """Generate a new API key for a trader."""
    print("\n--- Generate New API Key ---")

    trader = input("Trader name: ").strip()
    if not trader:
        print("■ Trader name required!")
        return

    admin = input("Admin name (default: same as trader): ").strip() or trader
    client = input("Client name (optional): ").strip()

    print("\nGenerating API key...")
    api_key = generate_api_key(admin, trader, client)

    if api_key:
        print("\n" + "=" * 60)
        print("✓ API KEY GENERATED SUCCESSFULLY")
        print("=" * 60)
        print(f"\nTrader: {trader}")
        print(f"Admin: {admin}")

```

```

if client:
    print(f"Client: {client}")
    print(f"\n■ API KEY: {api_key}")
    print("\n■■■ IMPORTANT: Save this key now!")
    print("    It will NEVER be shown again.")
    print("    The key is stored as a hash and cannot be recovered.")
    print("=" * 60)
else:
    print("■ Failed to generate API key!")

```

```
def revoke_key()
```

Revoke an existing API key.

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Calls `print(...)` for its side effect.
2. Calls `list_keys(...)` for its side effect.
3. Assigns `key_prefix = input('\nEnter key prefix to revoke (e.g., tk_abc123...):....`
4. **if not** `key_prefix`: branches conditionally.
5. Assigns `confirm = input(f'Are you sure you want to revoke key '{key_prefix}....`
6. **if** `confirm != 'yes'`: branches conditionally.
7. **if** `revoke_api_key(key_prefix)`: branches conditionally (with an **else/elif** arm).

Code (copy-paste safe)

```

def revoke_key():
    """Revoke an existing API key."""
    print("\n--- Revoke API Key ---")

    # Show current keys first
    list_keys()

    key_prefix = input("\nEnter key prefix to revoke (e.g., tk_abc123...): ").strip()
    if not key_prefix:
        print("■ Key prefix required!")
        return

    confirm = input(f"Are you sure you want to revoke key '{key_prefix}'? (yes/no): ").strip().lower()
    if confirm != 'yes':
        print("Cancelled.")
        return

    if revoke_api_key(key_prefix):
        print(f"✓ API key '{key_prefix}' has been revoked!")
    else:
        print(f"■ API key '{key_prefix}' not found!")

def permanently_delete_key()

```

Permanently delete an API key.

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Calls `print(...)` for its side effect.
2. Calls `list_keys(...)` for its side effect.
3. Assigns `key_prefix = input('\nEnter key prefix to DELETE (e.g., tk_abc123...):....`
4. **if** `not key_prefix`: branches conditionally.
5. Calls `print(...)` for its side effect.
6. Assigns `confirm = input(f"Type 'DELETE' to permanently remove key '{key_pre....`
7. **if** `confirm != 'DELETE'`: branches conditionally.
8. **if** `delete_api_key(key_prefix)`: branches conditionally (with an **else/elif** arm).

Code (copy-paste safe)

```
def permanently_delete_key():
    """Permanently delete an API key."""
    print("\n--- Permanently Delete API Key ---")

    # Show current keys first
    list_keys()

    key_prefix = input("\nEnter key prefix to DELETE (e.g., tk_abc123...): ").strip()
    if not key_prefix:
        print("■ Key prefix required!")
        return

    print("\n■■■ WARNING: This action cannot be undone!")
    confirm = input(f"Type 'DELETE' to permanently remove key '{key_prefix}': ").strip()
    if confirm != 'DELETE':
        print("Cancelled.")
        return

    if delete_api_key(key_prefix):
        print(f"✓ API key '{key_prefix}' has been permanently deleted!")
    else:
        print(f"■ API key '{key_prefix}' not found!")

def view_audit_log()
```

View recent audit log entries.

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Calls `print(...)` for its side effect.
2. Assigns `limit = input('Number of entries to show (default: 20): ').strip()`.
3. Assigns `limit = int(limit)` if `limit.isdigit()` else 20.
4. Assigns `action_filter = input('Filter by action (leave empty for all): ').strip()....`
5. Assigns `logs = get_audit_log(limit, action_filter)`.
6. Calls `print(...)` for its side effect.
7. Calls `print(...)` for its side effect.
8. Calls `print(...)` for its side effect.
9. `if not logs`: branches conditionally.
10. `for log in logs`: iterates.
11. Calls `print(...)` for its side effect.
12. Calls `print(...)` for its side effect.

Code (copy-paste safe)

```
def view_audit_log():
    """View recent audit log entries."""
    print("\n--- Audit Log ---")

    limit = input("Number of entries to show (default: 20): ").strip()
    limit = int(limit) if limit.isdigit() else 20

    action_filter = input("Filter by action (leave empty for all): ").strip() or None

    logs = get_audit_log(limit, action_filter)

    print("\n" + "=" * 80)
    print("AUDIT LOG")
    print("=" * 80)

    if not logs:
        print("No log entries found.")
        return

    for log in logs:
        timestamp = log['timestamp'][:19] if len(log['timestamp']) > 19 else log['timestamp']
        success = "✓" if log['success'] else "✗"
        print(f"[{timestamp}] {success} {log['action'][:25]} | {log['user_type']}: {log['user_identifier']}")
        if log['details']:
            print(f"    ■■■ {log['details']}")

    print("-" * 80)
    print(f"Showing {len(logs)} entries")

def change_admin_password()
    Change the admin password.
```

Step by step (top-level body)

1. Calls `print(...)` for its side effect.
2. Assigns `current = input('Enter current password: ').strip()`.
3. **if** `not verify_admin_password('super_admin', current)`: branches conditionally.
4. Assigns `new_password = input('Enter new password (min 8 characters): ').strip()`.
5. **if** `len(new_password) < 8`: branches conditionally.
6. Assigns `confirm = input('Confirm new password: ').strip()`.
7. **if** `new_password != confirm`: branches conditionally.
8. **if** `set_admin_password('super_admin', new_password)`: branches conditionally (with an **else/elif** arm).

Code (copy-paste safe)

```
def change_admin_password():
    """Change the admin password."""
    print("\n--- Change Admin Password ---")

    current = input("Enter current password: ").strip()
    if not verify_admin_password('super_admin', current):
        print("■ Invalid current password!")
        return

    new_password = input("Enter new password (min 8 characters): ").strip()
    if len(new_password) < 8:
        print("■ Password must be at least 8 characters!")
        return

    confirm = input("Confirm new password: ").strip()
    if new_password != confirm:
        print("■ Passwords don't match!")
        return

    if set_admin_password('super_admin', new_password):
        print("✓ Password changed successfully!")
    else:
        print("■ Failed to change password!")

def main()
```

Step by step (top-level body)

1. Calls `print_header(...)` for its side effect.
2. **if** `not authenticate()`: branches conditionally.
3. **while** `True`: loops until the condition is false.

Code (copy-paste safe)

```
def main():
    print_header()

    # Authenticate first
    if not authenticate():
        return
```

```

while True:
    print("\n" + "-" * 40)
    print("OPTIONS:")
    print("1. Generate new API key")
    print("2. List all API keys")
    print("3. Revoke API key")
    print("4. Permanently delete API key")
    print("5. View audit log")
    print("6. Change admin password")
    print("7. Exit")
    print("-" * 40)

    choice = input("\nSelect option (1-7): ").strip()

    if choice == '1':
        generate_new_key()
    elif choice == '2':
        list_keys()
    elif choice == '3':
        revoke_key()
    elif choice == '4':
        permanently_delete_key()
    elif choice == '5':
        view_audit_log()
    elif choice == '6':
        change_admin_password()
    elif choice == '7':
        print("\nGoodbye!")
        break
    else:
        print("Invalid option. Please select 1-7.")

```

Step 11.4 — Build dashboard/api_client.py

What to do. Create the file `dashboard/api_client.py` in your project root and paste in the code shown in this step. The file is **201** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from `requirements.txt`.

What this file is for. Outbound HTTP client used by the dashboard to talk to external APIs (Google Sheets, prop-firm endpoints).

dashboard/api_client.py

201 loc · 1 classes · 0 functions · 4 imports

Outbound HTTP client used by the dashboard to talk to external APIs (Google Sheets, prop-firm endpoints). It defines **1** class, across **201** lines. Module docstring: *Dashboard API Client for MT5 Traders*

Module docstring

Dashboard API Client for MT5 Traders This module allows traders to send data from their local MT5 software to the hosted dashboard.

Python concepts demonstrated in this module

• Classes and objects • Dunder methods • Exceptions • f-strings • Type hints

Imports — what each one is for

```
import requests
```

Why imported. The de-facto Python HTTP client. Synchronous; trades raw speed for an extremely friendly API.

Used in this file. `requests.exceptions`, `requests.exceptions.RequestException`, `requests.get`, `requests.post`

Example. `requests.get(url, timeout=10).json()`

Other common scenarios.

- `Session()` reuses TCP connections across calls
- `params=`, `headers=`, `json=`, `data=` for query/body shapes
- `raise_for_status()` turns 4xx/5xx into an exception
- `stream=True` for large downloads

```
import json
```

Why imported. JSON encoding and decoding — the standard wire format for config files, API payloads, and inter-process messages.

Used in this file. *No direct attribute access detected — likely imported for side effects, re-export, or string references.*

Example. `json.loads(response.text)` # parse a JSON string to dict

Other common scenarios.

- `json.dumps(obj, indent=2)` for pretty-printing
- `json.dump(obj, file)` / `json.load(file)` for file I/O
- `default=str` handles non-serialisable types like `datetime`
- `object_hook=` customises decoding (e.g., to `dataclass` instances)

```
from datetime import datetime
```

Why imported. Dates, times, durations. The fundamental temporal types: `date`, `time`, `datetime`, `timedelta`, `timezone`.

Used in this file. *No direct attribute access detected — likely imported for side effects, re-export, or string references.*

Example. `datetime.now() - timedelta(days=7)` # one week ago

Other common scenarios.

- `datetime.strptime(s, '%Y-%m-%d')` parses a string
- `datetime.strftime(fmt)` formats one
- `datetime.fromtimestamp(n)` converts a Unix epoch
- `datetime.utcnow()` for UTC (better: `datetime.now(timezone.utc)`)

```
from typing import Dict
from typing import List
from typing import Optional
```

Why imported. Type-hint primitives — generics, unions, Optional, Callable, Protocol, TypeVar, TypedDict.

Used in this file. List, Optional

Example. `def lookup(k: str) -> Optional[int]: ...`

Other common scenarios.

- `List[int]` / `Dict[str, Any]` / `Tuple[int, str]` (or bare `list[int]`+ on 3.9+)
- `Callable[[int, int], int]` for function signatures
- Protocol for structural typing (duck typing with checks)
- `TypeVar('T')` for generic functions and classes

Classes

```
class DashboardAPIClient
```

Client to communicate with the hosted dashboard API.

Code (copy-paste safe)

```
class DashboardAPIClient:
    """Client to communicate with the hosted dashboard API."""

    def __init__(self, api_url: str, api_key: str, client_id: Optional[str] = None):
        """
        Initialize the Dashboard API client.

        Args:
            api_url: Base URL of the dashboard API (e.g., 'https://yourusername.pythonanywhere.com')
            api_key: API key for authentication
            client_id: Optional client ID (if not set in API key)
        """
        self.api_url = api_url.rstrip('/')
        self.api_key = api_key
        self.client_id = client_id
        self.headers = {
            'Content-Type': 'application/json',
            'X-API-Key': self.api_key
        }

    def _post(self, endpoint: str, data: dict) -> dict:
        """Make a POST request to the API."""
        try:
            response = requests.post(
                f"{self.api_url}{endpoint}",
```

```

        headers=self.headers,
        json=data,
        timeout=10
    )
    response.raise_for_status()
    return response.json()
except requests.exceptions.RequestException as e:
    print(f"API Error: {e}")
    return {"status": "error", "message": str(e)}

def _get(self, endpoint: str) -> dict:
    """Make a GET request to the API."""
    try:
        response = requests.get(
            f"{self.api_url}{endpoint}",
            headers=self.headers,
            timeout=10
        )
        response.raise_for_status()
        return response.json()
    except requests.exceptions.RequestException as e:
        print(f"API Error: {e}")
        return {"status": "error", "message": str(e)}

def push_account_data(self, account: dict, client_id: Optional[str] = None) -> dict:
    """
    Push account information to the dashboard.

    Args:
        account: Dictionary containing account info (balance, equity, margin, etc.)
        client_id: Optional client ID override

    Returns:
        API response dictionary
    """
    data = {
        "account": account,
        "client_id": client_id or self.client_id
    }
    return self._post("/api/trader/push_account", data)

def push_positions(self, positions: List[dict], client_id: Optional[str] = None) -> dict:
    """
    Push current positions to the dashboard.

    Args:
        positions: List of position dictionaries
        client_id: Optional client ID override

    Returns:
        API response dictionary
    """
    data = {
        "positions": positions,
        "client_id": client_id or self.client_id
    }
    return self._post("/api/trader/push_positions", data)

def push_deals(self, deals: List[dict], client_id: Optional[str] = None) -> dict:
    """

```

```

    Push deal history to the dashboard.

    Args:
        deals: List of deal dictionaries
        client_id: Optional client ID override

    Returns:
        API response dictionary
    """
    data = {
        "deals": deals,
        "client_id": client_id or self.client_id
    }
    return self._post("/api/trader/push_deals", data)

def push_evaluations(self, evaluations: List[dict], client_id: Optional[str] = None) -> dict:
    """
    Push evaluation data to the dashboard.

    Args:
        evaluations: List of evaluation dictionaries
        client_id: Optional client ID override

    Returns:
        API response dictionary
    """
    data = {
        "evaluations": evaluations,
        "client_id": client_id or self.client_id
    }
    return self._post("/api/trader/push_evaluations", data)

def push_all_data(self, data: dict) -> dict:
    """
    Push all data at once (account, positions, deals, evaluations).

    Args:
        data: Dictionary containing all data types with identity info

    Returns:
        API response dictionary
    """
    return self._post("/api/update_data", data)

def health_check(self) -> dict:
    """Check if the dashboard API is accessible."""
    return self._get("/api/health")

def get_client_data(self, client_id: Optional[str] = None) -> dict:
    """
    Retrieve client data from the dashboard.

    Args:
        client_id: Optional client ID override

    Returns:
        Client data dictionary
    """
    cid = client_id or self.client_id
    return self._get(f"/api/data?client_id={cid}")

```

Method: DashboardAPIClient.__init__

```
def __init__(self, api_url: str, api_key: str, client_id: Optional[str]=None)
```

Initialize the Dashboard API client. Args: api_url: Base URL of the dashboard API (e.g., 'https://yourusername.pythonanywhere.com') api_key: API key for authentication client_id: Optional client ID (if not set in API key)

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `api_url: str`, `api_key: str`, `client_id: Optional[str]`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **__init__** — The constructor. Runs after Python has allocated the instance; assigns starting attribute values to `self`.

Step by step (top-level body)

1. Assigns `self.api_url = api_url.rstrip('/')`.
2. Assigns `self.api_key = api_key`.
3. Assigns `self.client_id = client_id`.
4. Assigns `self.headers = {'Content-Type': 'application/json', 'X-API-Key': self.ap...`

Code (copy-paste safe)

```
def __init__(self, api_url: str, api_key: str, client_id: Optional[str] = None):
    """
    Initialize the Dashboard API client.

    Args:
        api_url: Base URL of the dashboard API (e.g., 'https://yourusername.pythonanywhere.com')
        api_key: API key for authentication
        client_id: Optional client ID (if not set in API key)
    """
    self.api_url = api_url.rstrip('/')
    self.api_key = api_key
    self.client_id = client_id
    self.headers = {
        'Content-Type': 'application/json',
        'X-API-Key': self.api_key
    }
```

Method: DashboardAPIClient._post

```
def _post(self, endpoint: str, data: dict) -> dict
```

Make a POST request to the API.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `endpoint: str`, `data: dict`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.

- **return type annotation** — Annotated return type -> dict. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def _post(self, endpoint: str, data: dict) -> dict:
    """Make a POST request to the API."""
    try:
        response = requests.post(
            f"{self.api_url}{endpoint}",
            headers=self.headers,
            json=data,
            timeout=10
        )
        response.raise_for_status()
        return response.json()
    except requests.exceptions.RequestException as e:
        print(f"API Error: {e}")
        return {"status": "error", "message": str(e)}
```

Method: DashboardAPIClient._get

```
def _get(self, endpoint: str) -> dict:
    Make a GET request to the API.
```

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., endpoint: str). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type -> dict. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def _get(self, endpoint: str) -> dict:
```

```

"""Make a GET request to the API."""
try:
    response = requests.get(
        f"{self.api_url}{endpoint}",
        headers=self.headers,
        timeout=10
    )
    response.raise_for_status()
    return response.json()
except requests.exceptions.RequestException as e:
    print(f"API Error: {e}")
    return {"status": "error", "message": str(e)}

```

Method: DashboardAPIClient.push_account_data

```
def push_account_data(self, account: dict, client_id: Optional[str]=None) -> dict
```

Push account information to the dashboard. Args: account: Dictionary containing account info (balance, equity, margin, etc.) client_id: Optional client ID override Returns: API response dictionary

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `account: dict`, `client_id: Optional[str]`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> dict`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.

Step by step (top-level body)

1. Assigns `data = {'account': account, 'client_id': client_id or self.client_id}`
2. `return self._post('/api/trader/push_account', data).`

Code (copy-paste safe)

```

def push_account_data(self, account: dict, client_id: Optional[str] = None) -> dict:
    """
    Push account information to the dashboard.

    Args:
        account: Dictionary containing account info (balance, equity, margin, etc.)
        client_id: Optional client ID override

    Returns:
        API response dictionary
    """
    data = {
        "account": account,
        "client_id": client_id or self.client_id
    }
    return self._post("/api/trader/push_account", data)

```

Method: DashboardAPIClient.push_positions

```
def push_positions(self, positions: List[dict], client_id: Optional[str]=None) -> dict
```

Push current positions to the dashboard. Args: positions: List of position dictionaries client_id: Optional client ID override Returns: API response dictionary

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., positions: List[dict], client_id: Optional[str]). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type -> dict. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.

Step by step (top-level body)

1. Assigns data = {'positions': positions, 'client_id': client_id or self.c....
2. return self._post('/api/trader/push_positions', data).

Code (copy-paste safe)

```
def push_positions(self, positions: List[dict], client_id: Optional[str] = None) -> dict:
    """
    Push current positions to the dashboard.

    Args:
        positions: List of position dictionaries
        client_id: Optional client ID override

    Returns:
        API response dictionary
    """
    data = {
        "positions": positions,
        "client_id": client_id or self.client_id
    }
    return self._post("/api/trader/push_positions", data)
```

Method: DashboardAPIClient.push_deals

```
def push_deals(self, deals: List[dict], client_id: Optional[str]=None) -> dict

Push deal history to the dashboard. Args: deals: List of deal dictionaries client_id: Optional client ID
override Returns: API response dictionary
```

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., deals: List[dict], client_id: Optional[str]). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type -> dict. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.

Step by step (top-level body)

1. Assigns data = {'deals': deals, 'client_id': client_id or self.client_id}.

2. **return** self._post('/api/trader/push_deals', data).

Code (copy-paste safe)

```
def push_deals(self, deals: List[dict], client_id: Optional[str] = None) -> dict:
    """
    Push deal history to the dashboard.

    Args:
        deals: List of deal dictionaries
        client_id: Optional client ID override

    Returns:
        API response dictionary
    """
    data = {
        "deals": deals,
        "client_id": client_id or self.client_id
    }
    return self._post("/api/trader/push_deals", data)
```

Method: DashboardAPIClient.push_evaluations

```
def push_evaluations(self, evaluations: List[dict], client_id: Optional[str]=None) -> dict

    Push evaluation data to the dashboard. Args: evaluations: List of evaluation dictionaries client_id:
    Optional client ID override Returns: API response dictionary
```

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., evaluations: List[dict], client_id: Optional[str]). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type -> dict. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.

Step by step (top-level body)

1. Assigns data = {'evaluations': evaluations, 'client_id': client_id or se....
2. **return** self._post('/api/trader/push_evaluations', data).

Code (copy-paste safe)

```
def push_evaluations(self, evaluations: List[dict], client_id: Optional[str] = None) -> dict:
    """
    Push evaluation data to the dashboard.

    Args:
        evaluations: List of evaluation dictionaries
        client_id: Optional client ID override

    Returns:
        API response dictionary
    """
    data = {
        "evaluations": evaluations,
        "client_id": client_id or self.client_id
    }
```

```

    }
    return self._post("/api/trader/push_evaluations", data)

```

Method: DashboardAPIClient.push_all_data

```
def push_all_data(self, data: dict) -> dict
```

Push all data at once (account, positions, deals, evaluations). Args: data: Dictionary containing all data types with identity info Returns: API response dictionary

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., data: dict). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type -> dict. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.

Step by step (top-level body)

1. **return** self._post('/api/update_data', data).

Code (copy-paste safe)

```

def push_all_data(self, data: dict) -> dict:
    """
        Push all data at once (account, positions, deals, evaluations).

        Args:
            data: Dictionary containing all data types with identity info

        Returns:
            API response dictionary
    """
    return self._post("/api/update_data", data)

```

Method: DashboardAPIClient.health_check

```
def health_check(self) -> dict
```

Check if the dashboard API is accessible.

Why these Python concepts were used

- **return type annotation** — Annotated return type -> dict. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.

Step by step (top-level body)

1. **return** self._get('/api/health').

Code (copy-paste safe)

```

def health_check(self) -> dict:
    """Check if the dashboard API is accessible."""
    return self._get("/api/health")

```

Method: DashboardAPIClient.get_client_data

```
def get_client_data(self, client_id: Optional[str]=None) -> dict
```

Retrieve client data from the dashboard. Args: client_id: Optional client ID override Returns: Client data dictionary

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `client_id: Optional[str]`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> dict`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `cid = client_id` or `self.client_id`.
2. **return** `self._get(f'/api/data?client_id={cid}')`.

Code (copy-paste safe)

```
def get_client_data(self, client_id: Optional[str] = None) -> dict:
    """
    Retrieve client data from the dashboard.

    Args:
        client_id: Optional client ID override

    Returns:
        Client data dictionary
    """
    cid = client_id or self.client_id
    return self._get(f"/api/data?client_id={cid}")
```

Step 11.5 — Build dashboard/utils/sheet_helper.py

What to do. Create the file `dashboard/utils/sheet_helper.py` in your project root and paste in the code shown in this step. The file is **253** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from `requirements.txt`.

What this file is for. Thin wrapper over the Google Sheets API.

dashboard/utils/sheet_helper.py

253 loc · 0 classes · 6 functions · 5 imports

Thin wrapper over the Google Sheets API. It defines **6** module-level functions, across **253** lines.

Python concepts demonstrated in this module

• Comprehensions • Exceptions • f-strings • Lambdas

Imports — what each one is for

```
import pandas as pd
```

Why imported. DataFrame library — the workhorse for tabular data: loading CSV/Excel/SQL, joins, aggregations, time-series.

Used in this file. `pd.isna`, `pd.read_csv`, `pd.to_datetime`

Example. `df = pd.read_csv(path); df.groupby('symbol')['pnl'].sum()`

Other common scenarios.

- `df.merge(other, on='key')` joins like SQL
- `df.resample('1D').agg(...)` for time-series rebucketing
- `df.to_sql` / `df.to_excel` / `df.to_parquet` for output
- `df.loc[mask, 'col']` for label-based selection/assignment

```
import requests
```

Why imported. The de-facto Python HTTP client. Synchronous; trades raw speed for an extremely friendly API.

Used in this file. `requests.get`

Example. `requests.get(url, timeout=10).json()`

Other common scenarios.

- `Session()` reuses TCP connections across calls
- `params=`, `headers=`, `json=`, `data=` for query/body shapes
- `raise_for_status()` turns 4xx/5xx into an exception
- `stream=True` for large downloads

```
import io
```

Why imported. In-memory streams and the abstract base classes for file objects. Useful when an API expects a file but you only have bytes/text in memory.

Used in this file. `io.StringIO`

Example. `io.BytesIO(b'data')` # behaves like an open binary file

Other common scenarios.

- `io.StringIO('text')` for an in-memory text stream
- `io.TextIOWrapper` wraps a binary stream with an encoding

```
import logging
```

Why imported. Structured, levelled logging. Replaces print() with filterable, formattable output that can route to files, stderr, syslog, or HTTP endpoints.

Used in this file. logging.debug, logging.error

Example. logging.getLogger(__name__).info('connected to %s', host)

Other common scenarios.

- .debug() / .info() / .warning() / .error() / .exception()
- logger.exception() captures the current traceback automatically
- logging.basicConfig(level=logging.INFO) for quick setup
- Handlers and Formatters route logs to files / streams / syslog

```
from datetime import datetime
```

Why imported. Dates, times, durations. The fundamental temporal types: date, time, datetime, timedelta, timezone.

Used in this file. *No direct attribute access detected — likely imported for side effects, re-export, or string references.*

Example. datetime.now() - timedelta(days=7) # one week ago

Other common scenarios.

- datetime.strptime(s, '%Y-%m-%d') parses a string
- datetime.strftime(fmt) formats one
- datetime.fromtimestamp(n) converts a Unix epoch
- datetime.utcnow() for UTC (better: datetime.now(timezone.utc))

Module constants

Each line below is followed by a plain-English note explaining what it does and why it is written that way.

```
SHEET_ID = '10eGsivGm5G0aH0orB2AAbjpXzDwDkYY1gIZgux9nIjI'
```

String literal — fixed at code-load time.

```
STATS_GID = '839895136'
```

String literal — fixed at code-load time.

```
WATERLOG_GID = '520289647'
```

String literal — fixed at code-load time.

```
WATERLOG_TAB_NAME = 'Profitability Waterlog'
```

String literal — fixed at code-load time.

Functions

```
def get_sheet_csv(gid, sheet_id=None)
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `sid = sheet_id` or `SHEET_ID`.
2. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def get_sheet_csv(gid, sheet_id=None):
    sid = sheet_id or SHEET_ID
    try:
        url = f"https://docs.google.com/spreadsheets/d/{sid}/export?format=csv&gid={gid}"
        response = requests.get(url, timeout=10)
        response.raise_for_status()
        return response.content.decode('utf-8')
    except Exception as e:
        logging.error(f"Error fetching sheet CSV (gid={gid}): {e}")
        return None
```

```
def _extract_sheet_id(sheet_url)
```

Extract the spreadsheet key from a Google Sheets URL.

Why these Python concepts were used

- **ternary expression** — Uses `x` if `cond` else `y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Imports `re` (lazy import inside the function).
2. **if not** `sheet_url`: branches conditionally.
3. Assigns `m = re.search('/spreadsheets/d/([a-zA-Z0-9-_-]+)', sheet_url)`.
4. **return** `m.group(1)` if `m` else `None`.

Code (copy-paste safe)

```
def _extract_sheet_id(sheet_url):
    """Extract the spreadsheet key from a Google Sheets URL."""
    import re
    if not sheet_url:
        return None
    m = re.search(r'/spreadsheets/d/([a-zA-Z0-9-_-]+)', sheet_url)
    return m.group(1) if m else None
```

```
def _discover_waterlog_gid(sheet_id)
```

Try to discover the Profitability Waterlog GID from the sheet's HTML.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Imports `re` (lazy import inside the function).
2. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def _discover_waterlog_gid(sheet_id):
    """Try to discover the Profitability Waterlog GID from the sheet's HTML."""
    import re
    try:
        url = f"https://docs.google.com/spreadsheets/d/{sheet_id}/edit"
        resp = requests.get(url, timeout=10)
        if resp.status_code != 200:
            return None
        content = resp.text
        safe_name = re.escape(WATERLOG_TAB_NAME)
        for m in re.finditer(safe_name, content):
            window = content[max(0, m.start() - 300):m.start()]
            candidates = re.findall(r'\\?"(\\d+)\\?"(?!\\s*:)', window)
            valid = [c for c in candidates if len(c) > 5]
            if valid:
                return valid[-1]
        return None
    except Exception as e:
        logging.error(f"Error discovering waterlog GID: {e}")
        return None
```

```
def fetch_waterlog_data(sheet_url=None, client_id=None)
```

Fetches the Profitability Waterlog table directly from the sheet. If `sheet_url` is provided the sheet ID (and GID) are resolved dynamically from that URL so each client sees data from their own sheet. Falls back to the hardcoded `SHEET_ID` / `WATERLOG_GID` when not supplied. Sheet structure (Profitability Waterlog tab): Col D/E : Bi-weekly period table — From, To Col F : Low (computed by Google Sheets from live trading data) Col G : High (computed by Google Sheets from live trading data) Low and High are read directly from columns F/G. Profit Split is recomputed using the `HWM/4` formula that matches the reference sheet. If `client_id` is provided, loads and applies any `split_pct_overrides` from the database, preventing the ratio-based detection from overwriting admin-set split percentages.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.
- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns `spct_overrides = {}`.
2. `if client_id:` branches conditionally.
3. Assigns `sid = _extract_sheet_id(sheet_url)` if `sheet_url` else `None`.
4. `if sid:` branches conditionally (with an `else/elif` arm).
5. Assigns `csv_content = get_sheet_csv(gid, sheet_id=sid)`.
6. `if not csv_content:` branches conditionally.
7. `try` block with 1 `except` clause.

Code (copy-paste safe)

```
def fetch_waterlog_data(sheet_url=None, client_id=None):
    """
    Fetches the Profitability Waterlog table directly from the sheet.

    If sheet_url is provided the sheet ID (and GID) are resolved dynamically
    from that URL so each client sees data from their own sheet.
    Falls back to the hardcoded SHEET_ID / WATERLOG_GID when not supplied.

    Sheet structure (Profitability Waterlog tab):
    Col D/E : Bi-weekly period table – From, To
    Col F    : Low (computed by Google Sheets from live trading data)
    Col G    : High (computed by Google Sheets from live trading data)

    Low and High are read directly from columns F/G. Profit Split is
    recomputed using the HWM/4 formula that matches the reference sheet.

    If client_id is provided, loads and applies any split_pct_overrides
    from the database, preventing the ratio-based detection from overwriting
    admin-set split percentages.
    """
    # Load split percentage overrides if client_id provided
    spct_overrides = {}
    if client_id:
        try:
            from dashboard.watermark_service import get_split_pct_overrides
            spct_overrides = get_split_pct_overrides(client_id)
        except Exception as e:
            logging.debug(f"Could not load split_pct_overrides for {client_id}: {e}")
```

```

# Resolve sheet-ID and GID
sid = _extract_sheet_id(sheet_url) if sheet_url else None
if sid:
    gid = _discover_waterlog_gid(sid) or WATERLOG_GID
else:
    sid = None          # use default SHEET_ID inside get_sheet_csv
    gid = WATERLOG_GID

csv_content = get_sheet_csv(gid, sheet_id=sid)
if not csv_content:
    return None

try:
    df = pd.read_csv(io.StringIO(csv_content))

    def _parse_currency(val):
        try:
            s = str(val).replace(',', '').replace('$', '').strip()
            return float(s) if s not in ('', 'nan') else 0.0
        except Exception:
            return 0.0

    def _parse_date(val):
        if pd.isna(val) or str(val).strip() in ('', 'nan'):
            return None
        try:
            d = pd.to_datetime(str(val).strip(), errors='coerce').date()
            if d is None or d.year < 2000 or d.year > 2100:
                return None
            return d
        except Exception:
            return None

    def _fmt_date(d):
        return f"{d.month}/{d.day}/{d.year}"

    def _fmt_currency(v):
        if v < 0:
            return f"-${abs(v):,.2f}"
        return f"${v:,.2f}"

    # Column names – 'From ' sometimes has a trailing space in CSV exports
    from_col = 'From ' if 'From ' in df.columns else 'From'
    # 'Profit Split' column may carry the sheet's actual formula result
    ps_col = next((c for c in df.columns if c.strip() == 'Profit Split'), None)

    # Parse all valid rows first, then sort oldest→newest before HWM calc
    parsed_rows = []
    for _, row in df.iterrows():
        from_date = _parse_date(row.get(from_col, ''))
        to_date = _parse_date(row.get('To', ''))
        if from_date is None or to_date is None:
            continue
        period_low = _parse_currency(row.get('Low', 0))
        period_high = _parse_currency(row.get('High', 0))
        # Read the sheet's own Profit Split value if available
        sheet_ps = _parse_currency(row[ps_col]) if ps_col else 0.0
        parsed_rows.append((from_date, to_date, period_low, period_high, sheet_ps))

    # CRITICAL: HWM must be computed oldest-first regardless of CSV order

```

```

parsed_rows.sort(key=lambda x: x[0])

result = []
hwm_low = 0.0 # running high-water mark on the Low column

for (from_date, to_date, period_low, period_high, sheet_ps) in parsed_rows:
    gain = period_low - hwm_low if period_low > hwm_low else 0.0

    # Determine split percentage:
    # 1. If there's an explicit override for this period, use it (admin-set)
    # 2. Otherwise, detect from the sheet's profit split value using ratio logic
    # 3. Default to 50% for all new/unspecified periods (only legacy 25% for detected historical)
    fmt_from_date = _fmt_date(from_date)
    if fmt_from_date in spct_overrides:
        # Admin-set override takes priority
        split_pct = spct_overrides[fmt_from_date]
    elif gain > 0 and sheet_ps > 0:
        # Detect split % by comparing sheet value to the period gain.
        # Ratio  $\approx 0.25 \rightarrow 25\%$  ( $/4$ ), ratio  $\approx 0.50 \rightarrow 50\%$  ( $/2$ ).
        ratio = sheet_ps / gain
        if 0.40 <= ratio <= 0.60:
            split_pct = 50
        elif 0.15 <= ratio <= 0.35:
            split_pct = 25
        else:
            split_pct = 50 # default to 50% if ratio is ambiguous
    else:
        split_pct = 50 # no gain or no sheet data – default to 50%

    if gain > 0:
        profit_split = gain * split_pct / 100
        hwm_low = period_low
    else:
        profit_split = 0.0

    result.append({
        'from_date':    fmt_from_date,
        'to_date':      _fmt_date(to_date),
        'low':          _fmt_currency(period_low),
        'high':         _fmt_currency(period_high),
        'profit_split': f"${profit_split:,.0f}" if profit_split > 0 else '$0',
        'split_pct':    split_pct,
    })

# Return newest-first for display
result.reverse()
return result

except Exception as e:
    logging.error(f"Error computing Waterlog data: {e}")
    return None

```

```
def fetch_waterlog_periods_from_sheet(sheet_url=None)
```

Reads the bi-weekly period schedule (columns D/E: From, To) from the sheet. Returns list of (from_date_str, to_date_str) in 'YYYY-MM-DD' format, filtered to valid dates (year 2000-2100). Used once during import to seed the waterlog_periods DB table.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns `sid = _extract_sheet_id(sheet_url)` if `sheet_url` else `None`.
2. **if** `sid`: branches conditionally (with an **else/elif** arm).
3. Assigns `csv_content = get_sheet_csv(gid, sheet_id=sid)`.
4. **if not** `csv_content`: branches conditionally.
5. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def fetch_waterlog_periods_from_sheet(sheet_url=None):
    """
    Reads the bi-weekly period schedule (columns D/E: From, To) from the sheet.
    Returns list of (from_date_str, to_date_str) in 'YYYY-MM-DD' format,
    filtered to valid dates (year 2000-2100).
    Used once during import to seed the waterlog_periods DB table.
    """
    sid = _extract_sheet_id(sheet_url) if sheet_url else None
    if sid:
        gid = _discover_waterlog_gid(sid) or WATERLOG_GID
    else:
        sid = None
        gid = WATERLOG_GID

    csv_content = get_sheet_csv(gid, sheet_id=sid)
    if not csv_content:
        return []

    try:
        df = pd.read_csv(io.StringIO(csv_content))

        def _parse_date(val):
            if pd.isna(val) or str(val).strip() in ('', 'nan'):
                return None
            try:
                d = pd.to_datetime(str(val).strip(), errors='coerce').date()
                if d is None or d.year < 2000 or d.year > 2100:
                    return None
                return d
            except Exception:
                return None

        from_col = 'From ' if 'From ' in df.columns else 'From'
        periods = []
```

```

    for _, row in df.iterrows():
        fd = _parse_date(row.get('from_col', ''))
        td = _parse_date(row.get('To', ''))
        if fd and td:
            periods.append((fd.strftime('%Y-%m-%d'), td.strftime('%Y-%m-%d')))
    return periods
except Exception as e:
    logging.error(f"Error fetching waterlog periods from sheet: {e}")
    return []

```

```
def fetch_stats_data()
```

Fetches and parses the Stats tab.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `csv_content = get_sheet_csv(STATS_GID)`.
2. `if not csv_content:` branches conditionally.
3. `try` block with 1 **except** clause.

Code (copy-paste safe)

```

def fetch_stats_data():
    """Fetches and parses the Stats tab."""
    csv_content = get_sheet_csv(STATS_GID)
    if not csv_content:
        return None

    try:
        # Return raw rows for flexible rendering
        df = pd.read_csv(io.StringIO(csv_content), header=None)
        df = df.fillna('')
        return df.values.tolist()
    except Exception as e:
        logging.error(f"Error parsing Stats data: {e}")
        return None

```

Step 11.6 — Build dashboard/utils/trade_matcher.py

What to do. Create the file `dashboard/utils/trade_matcher.py` in your project root and paste in the code shown in this step. The file is **361** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from `requirements.txt`.

What this file is for. Reconciles MT5 deal history against the sheet's trade list to detect missing or duplicated entries.

dashboard/utils/trade_matcher.py

361 loc · 1 classes · 0 functions · 3 imports

Reconciles MT5 deal history against the sheet's trade list to detect missing or duplicated entries. It defines **1** class, across **361** lines.

Python concepts demonstrated in this module

• Classes and objects • Comprehensions • Dunder methods • Exceptions • f-strings • Lambdas

Imports — what each one is for

```
from datetime import datetime
```

Why imported. Dates, times, durations. The fundamental temporal types: `date`, `time`, `datetime`, `timedelta`, `timezone`.

Used in this file. `datetime`

Example. `datetime.now() - timedelta(days=7)` # one week ago

Other common scenarios.

- `datetime.strptime(s, '%Y-%m-%d')` parses a string
- `datetime.strftime(fmt)` formats one
- `datetime.fromtimestamp(n)` converts a Unix epoch
- `datetime.utcnow()` for UTC (better: `datetime.now(timezone.utc)`)

```
import re
```

Why imported. Regular expressions. Pattern matching, search, replace, and text extraction.

Used in this file. `re.search`, `re.sub`

Example. `re.search(r'\d{3}-\d{4}', text)` # returns a `Match` or `None`

Other common scenarios.

- `re.findall(pattern, text)` for every non-overlapping match
- `re.sub(pattern, repl, text)` to replace occurrences
- `re.compile(...)` to reuse a pattern (faster in a loop)

- Named groups: `r'(?P<year>\d{4})'` lets you do `m.group('year')`

`import logging`

Why imported. Structured, levelled logging. Replaces `print()` with filterable, formattable output that can route to files, stderr, syslog, or HTTP endpoints.

Used in this file. `logging.getLogger`

Example. `logging.getLogger(__name__).info('connected to %s', host)`

Other common scenarios.

- `.debug()` / `.info()` / `.warning()` / `.error()` / `.exception()`
- `logger.exception()` captures the current traceback automatically
- `logging.basicConfig(level=logging.INFO)` for quick setup
- Handlers and Formatters route logs to files / streams / syslog

Classes

`class UnifiedTradeMatcher`

Server-side logic to match MT5 deals to Dashboard Evaluation Rows. Solves the 'FundedNext Recycled Account' problem using Time-Based Session Grouping.

Code [\(copy-paste safe\)](#)

```
class UnifiedTradeMatcher:
    """
    Server-side logic to match MT5 deals to Dashboard Evaluation Rows.
    Solves the 'FundedNext Recycled Account' problem using Time-Based Session Grouping.
    """

    def __init__(self, evaluations):
        """
        Args:
            evaluations (list): List of evaluation dicts from dashboard_data.json
        """
        self.evaluations = evaluations
        self.match_log = []

        # Build optimized lookup maps
        self.account_map = self._build_account_map()

    def _build_account_map(self):
        """
        Map Account Numbers -> List of (RowIndex, DatePurchasedObj, Type)
        Handles fuzzy matching (last 8-10 digits).
        """
        lookup = {}

        for idx, row in enumerate(self.evaluations):
            # Parse Dates
            date_purchased = self._parse_date(row.get('Date Purchased'))
            date_started = self._parse_date(row.get('Date Started'))

            # Use Started date if available, else Purchased
            ref_date = date_started if date_started else date_purchased
```

```

# 1. Challenge Account Check
acct_ch = str(row.get('Account #', '')).strip()
if acct_ch and acct_ch.lower() != 'nan':
    self._add_to_lookup(lookup, acct_ch, idx, ref_date, 'challenge')

# 2. Funded Account Check
acct_fu = str(row.get('Account #.1', '')).strip()
if acct_fu and acct_fu.lower() != 'nan':
    self._add_to_lookup(lookup, acct_fu, idx, ref_date, 'funded')

return lookup

def _add_to_lookup(self, lookup, account_str, idx, date, acct_type):
    """Add account and its suffixes to lookup."""
    # Clean account string (remove non-alphanumeric if needed, but usually keep as is)
    # Add full match
    if account_str not in lookup:
        lookup[account_str] = []
    lookup[account_str].append({'idx': idx, 'date': date, 'type': acct_type})

    # Add Suffix Matches (Last 8, 10, 12 digits)
    # Useful for 'FN-123456' vs '123456'
    clean_num = re.sub(r'^[0-9]', '', account_str)
    if len(clean_num) > 5:
        for suffix_len in [8, 10, 12]:
            if len(clean_num) >= suffix_len:
                suffix = clean_num[-suffix_len:]
                # Only add if suffix distinct from full string
                if suffix != account_str:
                    if suffix not in lookup:
                        lookup[suffix] = []
                    # Avoid duplicates
                    if not any(x['idx'] == idx for x in lookup[suffix]):
                        lookup[suffix].append({'idx': idx, 'date': date, 'type': acct_type})

def _parse_date(self, date_str):
    """Parse 'MM/DD/YY' or ISO format to datetime object."""
    if not date_str or str(date_str).lower() in ['none', 'nan', '']:
        return None
    try:
        # Try MM/DD/YY (Dashboard format)
        return datetime.strptime(str(date_str).split('T')[0], "%m/%d/%y")
    except:
        try:
            # Try ISO
            return datetime.fromisoformat(str(date_str))
        except:
            try:
                # Try YYYY-MM-DD
                return datetime.strptime(str(date_str).split('T')[0], "%Y-%m-%d")
            except:
                return None

def process_deals(self, deals):
    """
    Main entry point.
    1. Parse Comments -> Get Account + Phase
    2. Group by Session (Time-based)
    3. Match Session to Best Dashboard Row
    4. Update Rows

```

```

Returns:
    updated_evaluations (list)
    log (list of strings)
"""
if not deals:
    return self.evaluations, ["No deals provided"]

# Step A: Parse & Sort Deals
parsed_deals = []
for d in deals:
    parsed = self._parse_deal_comment(d)
    if parsed:
        parsed['timestamp'] = self._parse_iso(d['time'])
        parsed['profit'] = float(d.get('profit', 0))
        # Store full deal including 'symbol' for potential filtering
        parsed['deal'] = d
        parsed_deals.append(parsed)

if not parsed_deals:
    return self.evaluations, ["No valid parsed deals found"]

# Sort by time
parsed_deals.sort(key=lambda x: x['timestamp'])

# Step B: Group by Account Number (The strict MT5 Account Number from comment)
# Note: 'account_number' here comes from the comment like '12345_CH1' -> '12345'
deals_by_account = {}
for pd in parsed_deals:
    acc = pd['account_number']
    if acc not in deals_by_account:
        deals_by_account[acc] = []
    deals_by_account[acc].append(pd)

# Step C: Process Each Account's Timeline
updates_count = 0

for acc_num, acc_deals in deals_by_account.items():
    sessions = self._segment_into_sessions(acc_deals)

    for session_idx, session in enumerate(sessions):
        session_start_date = session[0]['timestamp']

        # Find Matching Dashboard Row
        target_row_idx = self._find_best_row_match(acc_num, session_start_date)

        if target_row_idx is not None:
            # Apply updates to this row
            updates, new_logs = self._apply_session_to_row(target_row_idx, session)
            if updates > 0:
                updates_count += 1
                self.match_log.extend(new_logs)
            # Update Start Date if missing
            row = self.evaluations[target_row_idx]
            if not row.get('Date Started') and session_start_date:
                row['Date Started'] = session_start_date.strftime("%m/%d/%y")
                self.match_log.append(f"    ■■ Auto-set 'Date Started' to {row['Date Started']}")
        else:
            self.match_log.append(
                f"    ■■ Unmatched Session: Account {acc_num} starting {session_start_date.date()} ("

self.match_log.append(f"    ■ Processed {len(deals)} deals. Updated {updates_count} sessions.")

```

```

        return self.evaluations, self.match_log

def _segment_into_sessions(self, deals):
    """
    Group a list of sorted deals into logical sessions.
    New Session Trigger:
    1. 'CH1' trade appearing (Reset)
    2. Large time gap (> 14 days) between trades
    """
    sessions = []
    current_session = []

    for i, deal in enumerate(deals):
        is_start_of_new_session = False

        if i == 0:
            is_start_of_new_session = True
        else:
            prev_deal = deals[i-1]

            # Rule 1: Phase Reset (CH1 always starts new session unless prev was also CH1 very close)
            # CH1 means Challenge Phase 1. If we see CH1 after say CH2, it's a reset.
            if deal['phase'] == 'CH' and deal['number'] == 1:
                # If prev was NOT CH1, it's definitely a reset
                if not (prev_deal['phase'] == 'CH' and prev_deal['number'] == 1):
                    is_start_of_new_session = True
                # Even if prev was CH1, if > 24 hours gap, assume new attempt
                elif (deal['timestamp'] - prev_deal['timestamp']).total_seconds() > 86400:
                    is_start_of_new_session = True

            # Rule 2: Time Gap (14 Days)
            # If 2 weeks partial silence, assume new attempt if no other indicators
            gap = (deal['timestamp'] - prev_deal['timestamp']).days
            if gap > 14:
                is_start_of_new_session = True

        # Start new session buffer if triggered
        if is_start_of_new_session:
            if current_session:
                sessions.append(current_session)
            current_session = [deal]
        else:
            current_session.append(deal)

    if current_session:
        sessions.append(current_session)

    return sessions

def _find_best_row_match(self, account_num, session_date):
    """
    Find the dashboard row index where:
    1. Account Number matches (fuzzy)
    2. Row Date is closest to Session Date (and not in future)
    """
    # Get candidates from map
    candidates = []

    # Exact match
    if account_num in self.account_map:

```

```

        candidates.extend(self.account_map[account_num])

# Suffix match (Try last 8 chars)
if len(account_num) > 8:
    suffix = account_num[-8:]
    if suffix in self.account_map:
        candidates.extend(self.account_map[suffix])

if not candidates:
    return None

# Deduplicate candidates (idx is unique key)
# We use a dict to deduplicate by idx
unique_candidates_dict = {}
for c in candidates:
    unique_candidates_dict[c['idx']] = c
unique_candidates = list(unique_candidates_dict.values())

best_idx = None
min_diff = None

for cand in unique_candidates:
    row_date = cand['date']

    # If row has no date, treat it as "Old/Default" -> Low priority unless it's the only one
    if not row_date:
        if best_idx is None:
            best_idx = cand['idx']
            min_diff = 99999
        continue

    # Calculate gap: Session Date - Row Date
    # Ideally Session Date >= Row Date (Purchased before trade)
    diff = (session_date - row_date).days

    # Allow trade to be up to 7 days BEFORE purchase date (admin lag)
    # Allow trade to be anytime AFTER purchase date
    if diff >= -7:
        # We want the SMALLEST difference (Closest purchase date)
        # Prefer positive differences (trade after purchase) over negative ones
        # But mostly just shortest absolute distance from purchase date

        if min_diff is None:
            min_diff = diff
            best_idx = cand['idx']
        else:
            # Update if this row is closer in time or (equal and newer index)
            if abs(diff) < abs(min_diff):
                min_diff = diff
                best_idx = cand['idx']
            # Prefer positive (trade after purchase) over negative if equal magnitude
            elif abs(diff) == abs(min_diff) and diff >= 0 and min_diff < 0:
                min_diff = diff
                best_idx = cand['idx']

    return best_idx

def _apply_session_to_row(self, row_idx, session_deals):
    """ aggregate profits in session and update evaluation row """
    row = self.evaluations[row_idx]

```

```

updates = 0
local_logs = []

# Calculate Nettings per field
# Map: "Hedge Result 1" -> 500.00
field_sums = {}

for d in session_deals:
    field = self._get_field_name(d['phase'], d['number'])
    if field:
        if field not in field_sums:
            field_sums[field] = 0.0
        field_sums[field] += d['profit']

# Apply to row - Replaces value (or adds? usually replaces total for that phase)
# BUT wait: The input 'deals' here are ALL deals for that phase in that session.
# So we should sum them up and overwrite the field, assuming the client sent full history.
# If client sends incremental, we might double count.
# Requirement: "Client sends full history for relevant period".
# We will overwrite based on the sum of provided deals. This is safer than adding to existing which

for field, profit in field_sums.items():
    # Only update if changed significantly
    old_val = row.get(field)
    # Handle string conversions if necessary (dashboard usually uses strings or floats)
    row[field] = profit
    local_logs.append(f"    ■ Row {row_idx} {row.get('Account #')} [{field}] <- {profit:.2f}")
    updates += 1

return updates, local_logs

def _get_field_name(self, phase, number):
    """Map Phase Code to Dashboard Column"""
    if phase == 'CH':
        if 1 <= number <= 5: return f"Hedge Result {number}"
    elif phase == 'FD':
        if number == 0: return "Hedge Result 1.1" # FD0
        if 1 <= number <= 4: return f"Hedge Result {number + 1}.1" # FD1->2.1
        if number >= 5: return f"Hedge Result {number + 1}" # FD5->6
    elif phase == 'FA':
        return "Hedge Day 1" # Simplification for Farming
    return None

def _parse_deal_comment(self, deal):
    """
    Extract {Account}_{Phase}{Num}
    Regex: ([A-Za-z0-9]+)_([CFD]+)([0-9]+)
    Ex: 123456_CH1 -> Account=123456, Phase=CH, Num=1
    """
    comment = deal.get('comment', '')
    if not comment:
        return None

    # Try standard pattern
    # <Account>_<Phase><Num>
    # 123456_CH1
    # MFF123456_FD2
    match = re.search(r'([A-Za-z0-9\.-]+)_([A-Z]{2})([0-9]+)', comment)
    if match:
        return {

```

```

        'account_number': match.group(1),
        'phase': match.group(2),
        'number': int(match.group(3))
    }

    # Try without underscore if failed (e.g. 123456CH1)
    match = re.search(r'([0-9]{5,})([A-Z]{2})([0-9]+)', comment)
    if match:
        return {
            'account_number': match.group(1),
            'phase': match.group(2),
            'number': int(match.group(3))
        }

    return None

def _parse_iso(self, date_str):
    try:
        return datetime.fromisoformat(str(date_str).replace('Z', '+00:00'))
    except:
        return datetime.now()

```

Method: UnifiedTradeMatcher.__init__

```
def __init__(self, evaluations)
```

Args: evaluations (list): List of evaluation dicts from dashboard_data.json

Why these Python concepts were used

- **__init__** — The constructor. Runs after Python has allocated the instance; assigns starting attribute values to self.

Step by step (top-level body)

1. Assigns self.evaluations = evaluations.
2. Assigns self.match_log = [].
3. Assigns self.account_map = self._build_account_map().

Code (copy-paste safe)

```

def __init__(self, evaluations):
    """
    Args:
        evaluations (list): List of evaluation dicts from dashboard_data.json
    """
    self.evaluations = evaluations
    self.match_log = []

    # Build optimized lookup maps
    self.account_map = self._build_account_map()

```

Method: UnifiedTradeMatcher._build_account_map

```
def _build_account_map(self)
```

Map Account Numbers -> List of (RowIndex, DatePurchasedObj, Type) Handles fuzzy matching (last 8-10 digits).

Why these Python concepts were used

- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns `lookup = {}`.
2. `for (idx, row) in enumerate(self.evaluations):` iterates.
3. `return lookup`.

Code (copy-paste safe)

```
def _build_account_map(self):
    """
    Map Account Numbers -> List of (RowIndex, DatePurchasedObj, Type)
    Handles fuzzy matching (last 8-10 digits).
    """
    lookup = {}

    for idx, row in enumerate(self.evaluations):
        # Parse Dates
        date_purchased = self._parse_date(row.get('Date Purchased'))
        date_started = self._parse_date(row.get('Date Started'))

        # Use Started date if available, else Purchased
        ref_date = date_started if date_started else date_purchased

        # 1. Challenge Account Check
        acct_ch = str(row.get('Account #', '')).strip()
        if acct_ch and acct_ch.lower() != 'nan':
            self._add_to_lookup(lookup, acct_ch, idx, ref_date, 'challenge')

        # 2. Funded Account Check
        acct_fu = str(row.get('Account #.1', '')).strip()
        if acct_fu and acct_fu.lower() != 'nan':
            self._add_to_lookup(lookup, acct_fu, idx, ref_date, 'funded')

    return lookup
```

Method: UnifiedTradeMatcher._add_to_lookup

```
def _add_to_lookup(self, lookup, account_str, idx, date, acct_type)
    Add account and its suffixes to lookup.
```

Why these Python concepts were used

- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to `sum`, `any`, `all`, or `max` where the intermediate list would be wasted.

Step by step (top-level body)

1. `if account_str not in lookup:` branches conditionally.
2. Calls `lookup[account_str].append(...)` for its side effect.
3. Assigns `clean_num = re.sub('[^0-9]', '', account_str)`.

4. if len(clean_num) > 5: branches conditionally.

Code (copy-paste safe)

```
def _add_to_lookup(self, lookup, account_str, idx, date, acct_type):
    """Add account and its suffixes to lookup."""
    # Clean account string (remove non-alphanumeric if needed, but usually keep as is)
    # Add full match
    if account_str not in lookup:
        lookup[account_str] = []
        lookup[account_str].append({'idx': idx, 'date': date, 'type': acct_type})

    # Add Suffix Matches (Last 8, 10, 12 digits)
    # Useful for 'FN-123456' vs '123456'
    clean_num = re.sub(r'^\0-9', '', account_str)
    if len(clean_num) > 5:
        for suffix_len in [8, 10, 12]:
            if len(clean_num) >= suffix_len:
                suffix = clean_num[-suffix_len:]
                # Only add if suffix distinct from full string
                if suffix != account_str:
                    if suffix not in lookup:
                        lookup[suffix] = []
                    # Avoid duplicates
                    if not any(x['idx'] == idx for x in lookup[suffix]):
                        lookup[suffix].append({'idx': idx, 'date': date, 'type': acct_type})
```

Method: UnifiedTradeMatcher._parse_date

```
def _parse_date(self, date_str)
    Parse 'MM/DD/YY' or ISO format to datetime object.
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

Step by step (top-level body)

1. if not date_str or str(date_str).lower() in ['none', 'nan', '']: branches conditionally.
2. try block with 1 **except** clause.

Code (copy-paste safe)

```
def _parse_date(self, date_str):
    """Parse 'MM/DD/YY' or ISO format to datetime object."""
    if not date_str or str(date_str).lower() in ['none', 'nan', '']:
        return None
    try:
        # Try MM/DD/YY (Dashboard format)
        return datetime.strptime(str(date_str).split('T')[0], "%m/%d/%y")
    except:
        try:
            # Try ISO
            return datetime.fromisoformat(str(date_str))
        except:
            try:
```

```

        # Try YYYY-MM-DD
        return datetime.strptime(str(date_str).split('T')[0], "%Y-%m-%d")
    except:
        return None

```

Method: UnifiedTradeMatcher.process_deals

```
def process_deals(self, deals)
```

Main entry point. 1. Parse Comments -> Get Account + Phase 2. Group by Session (Time-based) 3. Match Session to Best Dashboard Row 4. Update Rows Returns: updated_evaluations (list) log (list of strings)

Why these Python concepts were used

- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **if** not deals: branches conditionally.
2. Assigns parsed_deals = [].
3. **for** d in deals: iterates.
4. **if** not parsed_deals: branches conditionally.
5. Calls parsed_deals.sort(...) for its side effect.
6. Assigns deals_by_account = {}.
7. **for** pd in parsed_deals: iterates.
8. Assigns updates_count = 0.
9. **for** (acc_num, acc_deals) in deals_by_account.items(): iterates.
10. Calls self.match_log.append(...) for its side effect.
11. **return** (self.evaluations, self.match_log).

Code (copy-paste safe)

```

def process_deals(self, deals):
    """
    Main entry point.
    1. Parse Comments -> Get Account + Phase
    2. Group by Session (Time-based)
    3. Match Session to Best Dashboard Row
    4. Update Rows

    Returns:
        updated_evaluations (list)
        log (list of strings)
    """
    if not deals:
        return self.evaluations, ["No deals provided"]

    # Step A: Parse & Sort Deals

```

```

parsed_deals = []
for d in deals:
    parsed = self._parse_deal_comment(d)
    if parsed:
        parsed['timestamp'] = self._parse_iso(d['time'])
        parsed['profit'] = float(d.get('profit', 0))
        # Store full deal including 'symbol' for potential filtering
        parsed['deal'] = d
        parsed_deals.append(parsed)

if not parsed_deals:
    return self.evaluations, ["No valid parsed deals found"]

# Sort by time
parsed_deals.sort(key=lambda x: x['timestamp'])

# Step B: Group by Account Number (The strict MT5 Account Number from comment)
# Note: 'account_number' here comes from the comment like '12345_CH1' -> '12345'
deals_by_account = {}
for pd in parsed_deals:
    acc = pd['account_number']
    if acc not in deals_by_account:
        deals_by_account[acc] = []
    deals_by_account[acc].append(pd)

# Step C: Process Each Account's Timeline
updates_count = 0

for acc_num, acc_deals in deals_by_account.items():
    sessions = self._segment_into_sessions(acc_deals)

    for session_idx, session in enumerate(sessions):
        session_start_date = session[0]['timestamp']

        # Find Matching Dashboard Row
        target_row_idx = self._find_best_row_match(acc_num, session_start_date)

        if target_row_idx is not None:
            # Apply updates to this row
            updates, new_logs = self._apply_session_to_row(target_row_idx, session)
            if updates > 0:
                updates_count += 1
                self.match_log.extend(new_logs)
                # Update Start Date if missing
                row = self.evaluations[target_row_idx]
                if not row.get('Date Started') and session_start_date:
                    row['Date Started'] = session_start_date.strftime("%m/%d/%y")
                    self.match_log.append(f"    ■■ Auto-set 'Date Started' to {row['Date Started']}")
            else:
                self.match_log.append(
                    f"    ■■ Unmatched Session: Account {acc_num} starting {session_start_date.date()} (")

        self.match_log.append(f"    ■ Processed {len(deals)} deals. Updated {updates_count} sessions.")
    return self.evaluations, self.match_log

```

Method: UnifiedTradeMatcher._segment_into_sessions

```

def _segment_into_sessions(self, deals)

```

Group a list of sorted deals into logical sessions. New Session Trigger: 1. 'CH1' trade appearing (Reset)
2. Large time gap (> 14 days) between trades

Step by step (top-level body)

1. Assigns sessions = [].
2. Assigns current_session = [].
3. **for** (i, deal) in enumerate(deals): iterates.
4. **if** current_session: branches conditionally.
5. **return** sessions.

Code (copy-paste safe)

```
def _segment_into_sessions(self, deals):
    """
    Group a list of sorted deals into logical sessions.
    New Session Trigger:
    1. 'CH1' trade appearing (Reset)
    2. Large time gap (> 14 days) between trades
    """
    sessions = []
    current_session = []

    for i, deal in enumerate(deals):
        is_start_of_new_session = False

        if i == 0:
            is_start_of_new_session = True
        else:
            prev_deal = deals[i-1]

            # Rule 1: Phase Reset (CH1 always starts new session unless prev was also CH1 very close)
            # CH1 means Challenge Phase 1. If we see CH1 after say CH2, it's a reset.
            if deal['phase'] == 'CH' and deal['number'] == 1:
                # If prev was NOT CH1, it's definitely a reset
                if not (prev_deal['phase'] == 'CH' and prev_deal['number'] == 1):
                    is_start_of_new_session = True
                # Even if prev was CH1, if > 24 hours gap, assume new attempt
                elif (deal['timestamp'] - prev_deal['timestamp']).total_seconds() > 86400:
                    is_start_of_new_session = True

            # Rule 2: Time Gap (14 Days)
            # If 2 weeks partial silence, assume new attempt if no other indicators
            gap = (deal['timestamp'] - prev_deal['timestamp']).days
            if gap > 14:
                is_start_of_new_session = True

            # Start new session buffer if triggered
            if is_start_of_new_session:
                if current_session:
                    sessions.append(current_session)
                    current_session = [deal]
            else:
                current_session.append(deal)

    if current_session:
        sessions.append(current_session)
```

```
return sessions
```

Method: UnifiedTradeMatcher._find_best_row_match

```
def _find_best_row_match(self, account_num, session_date)
```

Find the dashboard row index where: 1. Account Number matches (fuzzy) 2. Row Date is closest to Session Date (and not in future)

Step by step (top-level body)

1. Assigns candidates = [].
2. if account_num in self.account_map: branches conditionally.
3. if len(account_num) > 8: branches conditionally.
4. if not candidates: branches conditionally.
5. Assigns unique_candidates_dict = {}.
6. for c in candidates: iterates.
7. Assigns unique_candidates = list(unique_candidates_dict.values()).
8. Assigns best_idx = None.
9. Assigns min_diff = None.
10. for cand in unique_candidates: iterates.
11. return best_idx.

Code (copy-paste safe)

```
def _find_best_row_match(self, account_num, session_date):
    """
    Find the dashboard row index where:
    1. Account Number matches (fuzzy)
    2. Row Date is closest to Session Date (and not in future)
    """
    # Get candidates from map
    candidates = []

    # Exact match
    if account_num in self.account_map:
        candidates.extend(self.account_map[account_num])

    # Suffix match (Try last 8 chars)
    if len(account_num) > 8:
        suffix = account_num[-8:]
        if suffix in self.account_map:
            candidates.extend(self.account_map[suffix])

    if not candidates:
        return None

    # Deduplicate candidates (idx is unique key)
    # We use a dict to deduplicate by idx
    unique_candidates_dict = {}
    for c in candidates:
        unique_candidates_dict[c['idx']] = c
    unique_candidates = list(unique_candidates_dict.values())
```

```

best_idx = None
min_diff = None

for cand in unique_candidates:
    row_date = cand['date']

    # If row has no date, treat it as "Old/Default" -> Low priority unless it's the only one
    if not row_date:
        if best_idx is None:
            best_idx = cand['idx']
            min_diff = 99999
        continue

    # Calculate gap: Session Date - Row Date
    # Ideally Session Date >= Row Date (Purchased before trade)
    diff = (session_date - row_date).days

    # Allow trade to be up to 7 days BEFORE purchase date (admin lag)
    # Allow trade to be anytime AFTER purchase date
    if diff >= -7:
        # We want the SMALLEST difference (Closest purchase date)
        # Prefer positive differences (trade after purchase) over negative ones
        # But mostly just shortest absolute distance from purchase date

        if min_diff is None:
            min_diff = diff
            best_idx = cand['idx']
        else:
            # Update if this row is closer in time or (equal and newer index)
            if abs(diff) < abs(min_diff):
                min_diff = diff
                best_idx = cand['idx']
            # Prefer positive (trade after purchase) over negative if equal magnitude
            elif abs(diff) == abs(min_diff) and diff >= 0 and min_diff < 0:
                min_diff = diff
                best_idx = cand['idx']

    return best_idx

```

Method: UnifiedTradeMatcher._apply_session_to_row

```
def _apply_session_to_row(self, row_idx, session_deals)
```

aggregate profits in session and update evaluation row

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns row = self.evaluations[row_idx].
2. Assigns updates = 0.
3. Assigns local_logs = [].
4. Assigns field_sums = {}.
5. **for** d in session_deals: iterates.

6. **for** (field, profit) in field_sums.items(): iterates.

7. **return** (updates, local_logs).

Code (copy-paste safe)

```
def _apply_session_to_row(self, row_idx, session_deals):
    """ aggregate profits in session and update evaluation row """
    row = self.evaluations[row_idx]
    updates = 0
    local_logs = []

    # Calculate Nettings per field
    # Map: "Hedge Result 1" -> 500.00
    field_sums = {}

    for d in session_deals:
        field = self._get_field_name(d['phase'], d['number'])
        if field:
            if field not in field_sums:
                field_sums[field] = 0.0
            field_sums[field] += d['profit']

    # Apply to row - Replaces value (or adds? usually replaces total for that phase)
    # BUT wait: The input 'deals' here are ALL deals for that phase in that session.
    # So we should sum them up and overwrite the field, assuming the client sent full history.
    # If client sends incremental, we might double count.
    # Requirement: "Client sends full history for relevant period".
    # We will overwrite based on the sum of provided deals. This is safer than adding to existing which

    for field, profit in field_sums.items():
        # Only update if changed significantly
        old_val = row.get(field)
        # Handle string conversions if necessary (dashboard usually uses strings or floats)
        row[field] = profit
        local_logs.append(f"    ■ Row {row_idx} {row.get('Account #')} [{field}] <- {profit:.2f}")
        updates += 1

    return updates, local_logs
```

Method: UnifiedTradeMatcher._get_field_name

```
def _get_field_name(self, phase, number)
```

Map Phase Code to Dashboard Column

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **if** phase == 'CH': branches conditionally (with an **else/elif** arm).
2. **return** None.

Code (copy-paste safe)

```
def _get_field_name(self, phase, number):
    """Map Phase Code to Dashboard Column"""
```

```

if phase == 'CH':
    if 1 <= number <= 5: return f"Hedge Result {number}"
elif phase == 'FD':
    if number == 0: return "Hedge Result 1.1" # FD0
    if 1 <= number <= 4: return f"Hedge Result {number + 1}.1" # FD1->2.1
    if number >= 5: return f"Hedge Result {number + 1}" # FD5->6
elif phase == 'FA':
    return "Hedge Day 1" # Simplification for Farming
return None

```

Method: UnifiedTradeMatcher._parse_deal_comment

```
def _parse_deal_comment(self, deal)
```

Extract {Account}_{Phase}{Num} Regex: ([A-Za-z0-9]+)_([CFD]+)([0-9]+) Ex: 123456_CH1 -> Account=123456, Phase=CH, Num=1

Step by step (top-level body)

1. Assigns comment = deal.get('comment', "").
2. if not comment: branches conditionally.
3. Assigns match = re.search('([A-Za-z0-9\-_]+)_([A-Z]{2})([0-9]+)', comment).
4. if match: branches conditionally.
5. Assigns match = re.search('([0-9]{5,})([A-Z]{2})([0-9]+)', comment).
6. if match: branches conditionally.
7. return None.

Code (copy-paste safe)

```

def _parse_deal_comment(self, deal):
    """
    Extract {Account}_{Phase}{Num}
    Regex: ([A-Za-z0-9]+)_([CFD]+)([0-9]+)
    Ex: 123456_CH1 -> Account=123456, Phase=CH, Num=1
    """
    comment = deal.get('comment', '')
    if not comment:
        return None

    # Try standard pattern
    # <Account>_<Phase><Num>
    # 123456_CH1
    # MFF123456_FD2
    match = re.search(r'([A-Za-z0-9\-\_]+)_([A-Z]{2})([0-9]+)', comment)
    if match:
        return {
            'account_number': match.group(1),
            'phase': match.group(2),
            'number': int(match.group(3))
        }

    # Try without underscore if failed (e.g. 123456CH1)
    match = re.search(r'([0-9]{5,})([A-Z]{2})([0-9]+)', comment)
    if match:
        return {
            'account_number': match.group(1),

```



```

        'phase': match.group(2),
        'number': int(match.group(3))
    }

    return None

```

Method: UnifiedTradeMatcher._parse_iso

```
def _parse_iso(self, date_str)
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def _parse_iso(self, date_str):
    try:
        return datetime.fromisoformat(str(date_str).replace('Z', '+00:00'))
    except:
        return datetime.now()

```

Step 11.7 — Build dashboard/calc_like_sheet.py

What to do. Create the file dashboard/calc_like_sheet.py in your project root and paste in the code shown in this step. The file is **111** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from requirements.txt.

What this file is for. Replicates the formulas in the source-of-truth Google Sheet so the dashboard's numbers match what the trader sees in their sheet.

dashboard/calc_like_sheet.py

111 loc · 0 classes · 3 functions · 3 imports

Replicates the formulas in the source-of-truth Google Sheet so the dashboard's numbers match what the trader sees in their sheet. It defines **3** module-level functions, across **111** lines. Module docstring: *Calculate like the sheet: Completed vs In Progress*

Module docstring

Calculate like the sheet: Completed vs In Progress

Python concepts demonstrated in this module

- Comprehensions
- Exceptions
- f-strings

Imports — what each one is for

```
import pandas as pd
```

Why imported. DataFrame library — the workhorse for tabular data: loading CSV/Excel/SQL, joins, aggregations, time-series.

Used in this file. `pd.isna`, `pd.read_csv`

Example. `df = pd.read_csv(path); df.groupby('symbol')['pnl'].sum()`

Other common scenarios.

- `df.merge(other, on='key')` joins like SQL
- `df.resample('1D').agg(...)` for time-series rebucketing
- `df.to_sql` / `df.to_excel` / `df.to_parquet` for output
- `df.loc[mask, 'col']` for label-based selection/assignment

```
import requests
```

Why imported. The de-facto Python HTTP client. Synchronous; trades raw speed for an extremely friendly API.

Used in this file. `requests.get`

Example. `requests.get(url, timeout=10).json()`

Other common scenarios.

- `Session()` reuses TCP connections across calls
- `params=`, `headers=`, `json=`, `data=` for query/body shapes
- `raise_for_status()` turns 4xx/5xx into an exception
- `stream=True` for large downloads

```
from io import StringIO
```

Why imported. In-memory streams and the abstract base classes for file objects. Useful when an API expects a file but you only have bytes/text in memory.

Used in this file. `StringIO`

Example. `io.BytesIO(b'data')` # behaves like an open binary file

Other common scenarios.

- `io.StringIO('text')` for an in-memory text stream
- `io.TextIOWrapper` wraps a binary stream with an encoding

Module constants

Each line below is followed by a plain-English note explaining what it does and why it is written that way.

```
SHEET_URL = 'https://docs.google.com/spreadsheets/d/1rXdWErZD5C0pTWcAu8jCQSFbV2Mm1088cPoJHFauH2E/export?1
```

URL constant pointing to an external resource. Hard-coded at load time; change here, not in callers.

Functions

```
def parse_val(val)
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

Step by step (top-level body)

1. **if** `pd.isna(val)` or `val == ''` or `val == '-'`: branches conditionally.
2. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def parse_val(val):
    if pd.isna(val) or val == '' or val == '-':
        return 0.0
    try:
        clean = str(val).replace('$', '').replace(',', '').replace(' ', '')
        if clean == '' or clean == '-':
            return 0.0
        return float(clean)
    except:
        return 0.0
```

```
def is_completed(row)
```

Step by step (top-level body)

1. Assigns `status_p1 = str(row['Status P1']).strip().lower()`.
2. Assigns `status = str(row['Status']).strip().lower()`.
3. **return** `status_p1 == 'fail'` or `status` in `['fail', 'completed']`.

Code (copy-paste safe)

```
def is_completed(row):
    status_p1 = str(row['Status P1']).strip().lower()
    status = str(row['Status']).strip().lower()

    # Completed if:
    # - Status P1 is Fail (Phase 1 failed)
    # - OR Status is Fail/Completed (Funded phase ended)
    return status_p1 == 'fail' or status in ['fail', 'completed']

def calc_row_values(row)
```

Why these Python concepts were used

- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `fee = parse_val(row.get('Fee', 0)) + parse_val(row.get('Activat...`
2. Assigns `hedge_p1 = sum((parse_val(row.get(f'Hedge Result {i}', 0)) for i in ....`
3. Assigns `hedge_funded = sum((parse_val(row.get(col, 0)) for col in ['Hedge Result....`
4. Assigns `hedge_days = sum((parse_val(row.get(f'Hedge Day {i}', 0)) for i in ran....`
5. Assigns `total_hedge = hedge_p1 + hedge_funded + hedge_days.`
6. Assigns `payouts = sum((parse_val(row.get(f'Payout {i}', 0)) for i in range(...`
7. Assigns `farm = parse_val(row.get('Farming Net', 0)).`
8. **return** (`fee, total_hedge, payouts, farm`).

Code (copy-paste safe)

```
def calc_row_values(row):
    fee = parse_val(row.get('Fee', 0)) + parse_val(row.get('Activation Fee', 0))

    # Individual hedge results (NOT Hedge Net which is a calculated field)
    hedge_p1 = sum(parse_val(row.get(f'Hedge Result {i}', 0)) for i in range(1, 6))
    hedge_funded = sum(parse_val(row.get(col, 0)) for col in
        ['Hedge Result 1.1', 'Hedge Result 2.1', 'Hedge Result 3.1',
         'Hedge Result 4.1', 'Hedge Result 5.1', 'Hedge Result 6', 'Hedge Result 7'])
    hedge_days = sum(parse_val(row.get(f'Hedge Day {i}', 0)) for i in range(1, 51))

    total_hedge = hedge_p1 + hedge_funded + hedge_days

    payouts = sum(parse_val(row.get(f'Payout {i}', 0)) for i in range(1, 5))
    farm = parse_val(row.get('Farming Net', 0))

    return fee, total_hedge, payouts, farm
```

Step 11.8 — Build dashboard/phase_manager.py

What to do. Create the file `dashboard/phase_manager.py` in your project root and paste in the code shown in this step. The file is **242** lines long; we walk through it piece by piece below.

Depends on. This file imports from internal modules built in earlier steps: `database`. If any of those imports fail, jump back to the step that introduced them.

What this file is for. State machine that tracks each account's evaluation phase: `challenge`, `verification`, `funded`. Drives payout eligibility.

dashboard/phase_manager.py

242 loc · 1 classes · 0 functions · 4 imports

State machine that tracks each account's evaluation phase: challenge, verification, funded. Drives payout eligibility. It defines **1** class, across **242** lines.

Python concepts demonstrated in this module

• Classes and objects • @classmethod and @staticmethod • Context managers (with-statements) • Decorators • Exceptions • f-strings • Type hints

Imports — what each one is for

```
import json
```

Why imported. JSON encoding and decoding — the standard wire format for config files, API payloads, and inter-process messages.

Used in this file. `json.dumps`

Example. `json.loads(response.text)` # parse a JSON string to dict

Other common scenarios.

- `json.dumps(obj, indent=2)` for pretty-printing
- `json.dump(obj, file)` / `json.load(file)` for file I/O
- `default=str` handles non-serialisable types like `datetime`
- `object_hook=` customises decoding (e.g., to `dataclass` instances)

```
from datetime import datetime
from datetime import timedelta
```

Why imported. Dates, times, durations. The fundamental temporal types: `date`, `time`, `datetime`, `timedelta`, `timezone`.

Used in this file. `datetime`, `timedelta`

Example. `datetime.now() - timedelta(days=7)` # one week ago

Other common scenarios.

- `datetime.strptime(s, '%Y-%m-%d')` parses a string
- `datetime.strftime(fmt)` formats one
- `datetime.fromtimestamp(n)` converts a Unix epoch
- `datetime.utcnow()` for UTC (better: `datetime.now(timezone.utc)`)

```
from typing import Optional
from typing import Dict
from typing import List
from typing import Tuple
```

Why imported. Type-hint primitives — generics, unions, `Optional`, `Callable`, `Protocol`, `TypeVar`, `TypedDict`.

Used in this file. Dict, List, Optional

Example. `def lookup(k: str) -> Optional[int]: ...`

Other common scenarios.

- `List[int]` / `Dict[str, Any]` / `Tuple[int, str]` (or bare `list[int]`+ on 3.9+)
- `Callable[[int, int], int]` for function signatures
- Protocol for structural typing (duck typing with checks)
- `TypeVar('T')` for generic functions and classes

```
from .database import get_connection
```

Why imported. Sibling module — engine and session factory.

Used in this file. `get_connection`

Classes

```
class PhaseManager
```

Manages the lifecycle of trading phases (Challenge -> Funded -> Farming) using a deterministic, database-driven approach.

Code (copy-paste safe)

```
class PhaseManager:
    """
    Manages the lifecycle of trading phases (Challenge -> Funded -> Farming)
    using a deterministic, database-driven approach.
    """

    @staticmethod
    def initialize_default_phases():
        """Idempotently populates default phase definitions if they don't exist."""
        defaults = [
            # Challenge Phase 1
            {
                "phase_name": "Challenge Phase 1",
                "phase_code": "CH",
                "sequence_order": 1,
                "ruleset": json.dumps({"type": "CHALLENGE", "target_percent": 0.08, "daily_loss": 0.05,
                                      "max_loss": 0.10}),
                "next_phase_code": "CH2" # Assuming 2-step challenge for now, or straight to FD
            },
            # Challenge Phase 2 (Optional, many firms have 2 steps)
            {
                "phase_name": "Challenge Phase 2",
                "phase_code": "CH2",
                "sequence_order": 2,
                "ruleset": json.dumps({"type": "CHALLENGE", "target_percent": 0.05}),
                "next_phase_code": "FD"
            },
            # Funded Phase
            {
                "phase_name": "Funded Account",
                "phase_code": "FD",
                "sequence_order": 3,
                "ruleset": json.dumps({"type": "FUNDED", "payout_split": 0.80}),
            }
        ]
```

```

        "next_phase_code": "FA"
    },
    # Farming Phase
    {
        "phase_name": "Farming Phase",
        "phase_code": "FA",
        "sequence_order": 4,
        "ruleset": json.dumps({"type": "ACCUMULATION"}),
        "next_phase_code": "FA" # Loops or stays in Farming
    }
]

with get_connection() as conn:
    cursor = conn.cursor()
    for phase in defaults:
        try:
            cursor.execute('''
                INSERT OR IGNORE INTO phase_definitions
                (phase_name, phase_code, sequence_order, ruleset, next_phase_code)
                VALUES (?, ?, ?, ?, ?)
            ''', (
                phase["phase_name"],
                phase["phase_code"],
                phase["sequence_order"],
                phase["ruleset"],
                phase["next_phase_code"]
            ))
        except Exception as e:
            print(f"Error seeding phase {phase['phase_code']}: {e}")
    conn.commit()

@staticmethod
def create_evaluation(
    account_signature: str,
    phase_code: str,
    start_date: str, # ISO Format YYYY-MM-DD
    parent_id: Optional[int] = None,
    reset_id: Optional[str] = None
) -> int:
    """
    Creates a new evaluation record.
    """
    with get_connection() as conn:
        cursor = conn.cursor()

        # Get validation rules from definition
        cursor.execute("SELECT sequence_order, ruleset FROM phase_definitions WHERE phase_code=?", (
            phase_code,))
        row = cursor.fetchone()
        if not row:
            raise ValueError(f"Unknown phase code: {phase_code}")

        phase_number = row[
            'sequence_order'] # Mapping sequence to phase_number for simplicity, or we can use auto-increment

        # Use provided reset_id or inherit from parent
        final_reset_id = reset_id
        if not final_reset_id and parent_id:
            cursor.execute("SELECT reset_id FROM evaluations WHERE id=?", (parent_id,))
            parent = cursor.fetchone()

```

```

        if parent:
            final_reset_id = parent['reset_id']

    cursor.execute('''
        INSERT INTO evaluations
        (account_signature, phase_number, phase_type, status, start_date, parent_id, reset_id)
        VALUES (?, ?, ?, ?, ?, ?, ?)
    ''', (
        account_signature,
        phase_number,
        phase_code,
        'active',
        start_date,
        parent_id,
        final_reset_id
    ))
    conn.commit()
    return cursor.lastrowid

@staticmethod
def complete_phase(evaluation_id: int, status: str, end_date: str) -> Optional[int]:
    """
    Marks a phase as completed/failed and triggers the next phase creation if successful.
    Returns the ID of the newly created next phase, or None.
    """
    with get_connection() as conn:
        cursor = conn.cursor()

        # 1. Update current phase
        cursor.execute('''
            UPDATE evaluations
            SET status = ?, end_date = ?
            WHERE id = ?
        ''', (status, end_date, evaluation_id))

        if status != 'passed':
            conn.commit()
            return None

        # 2. Fetch current evaluation details to determine next step
        cursor.execute("SELECT * FROM evaluations WHERE id=?", (evaluation_id,))
        current_eval = cursor.fetchone()

        if not current_eval:
            conn.commit()
            return None

        current_phase_code = current_eval['phase_type']
        account_sig = current_eval['account_signature']
        reset_id = current_eval['reset_id']

        # 3. Find next phase definition
        cursor.execute("SELECT next_phase_code FROM phase_definitions WHERE phase_code=?", (
            current_phase_code,))
        def_row = cursor.fetchone()

        if not def_row or not def_row['next_phase_code']:
            conn.commit()
            return None # End of chain

        next_code = def_row['next_phase_code']

```



```

# 4. Calculate next start date (Next Day)
# Ensure dates are date objects
try:
    end_dt = datetime.strptime(end_date, "%Y-%m-%d")
    next_start_dt = end_dt + timedelta(days=1)
    next_start_str = next_start_dt.strftime("%Y-%m-%d")
except ValueError:
    # Fallback if date is not simple YYYY-MM-DD
    next_start_str = end_date

# 5. Create next phase
# Call create_evaluation (need to pass connection or do it in same transaction?
# Doing here manually to keep transaction atomic)

cursor.execute("SELECT sequence_order FROM phase_definitions WHERE phase_code=?", (next_code,
next_def = cursor.fetchone()
next_seq = next_def['sequence_order'] if next_def else current_eval['phase_number'] + 1

cursor.execute('''
    INSERT INTO evaluations
    (account_signature, phase_number, phase_type, status, start_date, parent_id, reset_id)
    VALUES (?, ?, ?, ?, ?, ?, ?)
''', (
    account_sig,
    next_seq,
    next_code,
    'active',
    next_start_str,
    evaluation_id,
    reset_id
))

new_id = cursor.lastrowid
conn.commit()
return new_id

@staticmethod
def find_evaluation_for_trade(account_signature: str, trade_date_str: str) -> Optional[Dict]:
    """
    Deterministic O(1) matching of trade to evaluation using Date Windows.
    trade_date_str: YYYY-MM-DD
    """
    # We need to handle datetime strings potentially including time
    # The prompt says: "start_date <= trade.close_date <= end_date"
    # We assume start/end dates in DB are YYYY-MM-DD.
    # Trade date might be YYYY-MM-DD HH:MM:SS.

    trade_date_only = trade_date_str.split(' ')[0] if ' ' in trade_date_str else trade_date_str

    with get_connection() as conn:
        cursor = conn.cursor()

        # Find evaluation where trade_date is >= start_date AND (end_date IS NULL OR trade_date <= end_date)
        # We treat NULL end_date as "Open/Active"
        cursor.execute('''
            SELECT * FROM evaluations
            WHERE account_signature = ?
            AND start_date <= ?
            AND (end_date IS NULL OR end_date >= ?)
            ORDER BY start_date DESC
        ''')

```

```

        LIMIT 1
    ''' (account_signature, trade_date_only, trade_date_only))

    row = cursor.fetchone()
    if row:
        return dict(row)
    return None

    @staticmethod
    def get_phase_chain(latest_evaluation_id: int) -> List[Dict]:
        """Traverses up the parent_id chain to get full history."""
        chain = []
        curr_id = latest_evaluation_id

        with get_connection() as conn:
            cursor = conn.cursor()
            while curr_id:
                cursor.execute("SELECT * FROM evaluations WHERE id=?", (curr_id,))
                row = cursor.fetchone()
                if not row:
                    break
                chain.append(dict(row))
                curr_id = row['parent_id']

        return list(reversed(chain))

```

Method: PhaseManager.initialize_default_phases

```

@staticmethod
def initialize_default_phases()

    Idempotently populates default phase definitions if they don't exist.

```

Why these Python concepts were used

- **@staticmethod** — Declared **@staticmethod** because the function logically belongs in this class's namespace but does not depend on either the instance or the class — it is a pure helper that happens to be co-located with related code.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns defaults = [{'phase_name': 'Challenge Phase 1', 'phase_code': 'CH', ...
2. **with** get_connection(): enters a context manager.

Code (copy-paste safe)

```

def initialize_default_phases():
    """Idempotently populates default phase definitions if they don't exist."""
    defaults = [

```

```

# Challenge Phase 1
{
    "phase_name": "Challenge Phase 1",
    "phase_code": "CH",
    "sequence_order": 1,
    "ruleset": json.dumps({"type": "CHALLENGE", "target_percent": 0.08, "daily_loss": 0.05,
        "max_loss": 0.10}),
    "next_phase_code": "CH2" # Assuming 2-step challenge for now, or straight to FD
},
# Challenge Phase 2 (Optional, many firms have 2 steps)
{
    "phase_name": "Challenge Phase 2",
    "phase_code": "CH2",
    "sequence_order": 2,
    "ruleset": json.dumps({"type": "CHALLENGE", "target_percent": 0.05}),
    "next_phase_code": "FD"
},
# Funded Phase
{
    "phase_name": "Funded Account",
    "phase_code": "FD",
    "sequence_order": 3,
    "ruleset": json.dumps({"type": "FUNDED", "payout_split": 0.80}),
    "next_phase_code": "FA"
},
# Farming Phase
{
    "phase_name": "Farming Phase",
    "phase_code": "FA",
    "sequence_order": 4,
    "ruleset": json.dumps({"type": "ACCUMULATION"}),
    "next_phase_code": "FA" # Loops or stays in Farming
}
]

with get_connection() as conn:
    cursor = conn.cursor()
    for phase in defaults:
        try:
            cursor.execute('''
                INSERT OR IGNORE INTO phase_definitions
                (phase_name, phase_code, sequence_order, ruleset, next_phase_code)
                VALUES (?, ?, ?, ?, ?)
            ''', (
                phase["phase_name"],
                phase["phase_code"],
                phase["sequence_order"],
                phase["ruleset"],
                phase["next_phase_code"]
            ))
        except Exception as e:
            print(f"Error seeding phase {phase['phase_code']}: {e}")
    conn.commit()

```

Method: PhaseManager.create_evaluation

```

@staticmethod
def create_evaluation(account_signature: str, phase_code: str, start_date: str, parent_id: Optional[int]=
    reset_id: Optional[str]=None) -> int

```

Creates a new evaluation record.

Why these Python concepts were used

- **@staticmethod** — Declared **@staticmethod** because the function logically belongs in this class's namespace but does not depend on either the instance or the class — it is a pure helper that happens to be co-located with related code.
- **type-annotated parameters** — Parameters carry type hints (e.g., `account_signature: str`, `phase_code: str`, `start_date: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> int`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.

Step by step (top-level body)

1. **with get_connection():** enters a context manager.

Code (copy-paste safe)

```
def create_evaluation(
    account_signature: str,
    phase_code: str,
    start_date: str, # ISO Format YYYY-MM-DD
    parent_id: Optional[int] = None,
    reset_id: Optional[str] = None
) -> int:
    """
    Creates a new evaluation record.
    """
    with get_connection() as conn:
        cursor = conn.cursor()

        # Get validation rules from definition
        cursor.execute("SELECT sequence_order, ruleset FROM phase_definitions WHERE phase_code=?", (
            phase_code,))
        row = cursor.fetchone()
        if not row:
            raise ValueError(f"Unknown phase code: {phase_code}")

        phase_number = row[
            'sequence_order'] # Mapping sequence to phase_number for simplicity, or we can use auto-i

        # Use provided reset_id or inherit from parent
        final_reset_id = reset_id
        if not final_reset_id and parent_id:
            cursor.execute("SELECT reset_id FROM evaluations WHERE id=?", (parent_id,))
            parent = cursor.fetchone()
            if parent:
```

```

        final_reset_id = parent['reset_id']

    cursor.execute('''
        INSERT INTO evaluations
        (account_signature, phase_number, phase_type, status, start_date, parent_id, reset_id)
        VALUES (?, ?, ?, ?, ?, ?, ?)
    ''', (
        account_signature,
        phase_number,
        phase_code,
        'active',
        start_date,
        parent_id,
        final_reset_id
    ))
    conn.commit()
    return cursor.lastrowid

```

Method: PhaseManager.complete_phase

```

@staticmethod
def complete_phase(evaluation_id: int, status: str, end_date: str) -> Optional[int]

```

Marks a phase as completed/failed and triggers the next phase creation if successful. Returns the ID of the newly created next phase, or None.

Why these Python concepts were used

- **@staticmethod** — Declared **@staticmethod** because the function logically belongs in this class's namespace but does not depend on either the instance or the class — it is a pure helper that happens to be co-located with related code.
- **type-annotated parameters** — Parameters carry type hints (e.g., `evaluation_id: int`, `status: str`, `end_date: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> Optional[int]`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **with** `get_connection()`: enters a context manager.

Code (copy-paste safe)

```

def complete_phase(evaluation_id: int, status: str, end_date: str) -> Optional[int]:
    """
    Marks a phase as completed/failed and triggers the next phase creation if successful.

```

Returns the ID of the newly created next phase, or None.
"""

```
with get_connection() as conn:
    cursor = conn.cursor()

    # 1. Update current phase
    cursor.execute('''
        UPDATE evaluations
        SET status = ?, end_date = ?
        WHERE id = ?
    ''', (status, end_date, evaluation_id))

    if status != 'passed':
        conn.commit()
        return None

    # 2. Fetch current evaluation details to determine next step
    cursor.execute("SELECT * FROM evaluations WHERE id=?", (evaluation_id,))
    current_eval = cursor.fetchone()

    if not current_eval:
        conn.commit()
        return None

    current_phase_code = current_eval['phase_type']
    account_sig = current_eval['account_signature']
    reset_id = current_eval['reset_id']

    # 3. Find next phase definition
    cursor.execute("SELECT next_phase_code FROM phase_definitions WHERE phase_code=?", (
        current_phase_code,))
    def_row = cursor.fetchone()

    if not def_row or not def_row['next_phase_code']:
        conn.commit()
        return None # End of chain

    next_code = def_row['next_phase_code']

    # 4. Calculate next start date (Next Day)
    # Ensure dates are date objects
    try:
        end_dt = datetime.strptime(end_date, "%Y-%m-%d")
        next_start_dt = end_dt + timedelta(days=1)
        next_start_str = next_start_dt.strftime("%Y-%m-%d")
    except ValueError:
        # Fallback if date is not simple YYYY-MM-DD
        next_start_str = end_date

    # 5. Create next phase
    # Call create_evaluation (need to pass connection or do it in same transaction?
    # Doing here manually to keep transaction atomic)

    cursor.execute("SELECT sequence_order FROM phase_definitions WHERE phase_code=?", (next_code,))
    next_def = cursor.fetchone()
    next_seq = next_def['sequence_order'] if next_def else current_eval['phase_number'] + 1

    cursor.execute('''
        INSERT INTO evaluations
        (account_signature, phase_number, phase_type, status, start_date, parent_id, reset_id)
```

```

        VALUES (?, ?, ?, ?, ?, ?, ?)
    '''
    (
        account_sig,
        next_seq,
        next_code,
        'active',
        next_start_str,
        evaluation_id,
        reset_id
    ))

    new_id = cursor.lastrowid
    conn.commit()
    return new_id

```

Method: PhaseManager.find_evaluation_for_trade

```

@staticmethod
def find_evaluation_for_trade(account_signature: str, trade_date_str: str) -> Optional[Dict]

```

Deterministic O(1) matching of trade to evaluation using Date Windows. trade_date_str: YYYY-MM-DD

Why these Python concepts were used

- **@staticmethod** — Declared **@staticmethod** because the function logically belongs in this class's namespace but does not depend on either the instance or the class — it is a pure helper that happens to be co-located with related code.
- **type-annotated parameters** — Parameters carry type hints (e.g., `account_signature: str`, `trade_date_str: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> Optional[Dict]`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns `trade_date_only = trade_date_str.split(' ')[0]` if `' '` in `trade_date_str` else....
2. **with** `get_connection()`: enters a context manager.

Code (copy-paste safe)

```

def find_evaluation_for_trade(account_signature: str, trade_date_str: str) -> Optional[Dict]:
    """
    Deterministic O(1) matching of trade to evaluation using Date Windows.
    trade_date_str: YYYY-MM-DD
    """
    # We need to handle datetime strings potentially including time
    # The prompt says: "start_date <= trade.close_date <= end_date"
    # We assume start/end dates in DB are YYYY-MM-DD.
    # Trade date might be YYYY-MM-DD HH:MM:SS.

```

```

trade_date_only = trade_date_str.split(' ')[0] if ' ' in trade_date_str else trade_date_str

with get_connection() as conn:
    cursor = conn.cursor()

    # Find evaluation where trade_date is >= start_date AND (end_date IS NULL OR trade_date <= end_date)
    # We treat NULL end_date as "Open/Active"
    cursor.execute('''
        SELECT * FROM evaluations
        WHERE account_signature = ?
        AND start_date <= ?
        AND (end_date IS NULL OR end_date >= ?)
        ORDER BY start_date DESC
        LIMIT 1
    ''', (account_signature, trade_date_only, trade_date_only))

    row = cursor.fetchone()
    if row:
        return dict(row)
    return None

```

Method: PhaseManager.get_phase_chain

```

@staticmethod
def get_phase_chain(latest_evaluation_id: int) -> List[Dict]

```

Traverses up the parent_id chain to get full history.

Why these Python concepts were used

- **@staticmethod** — Declared **@staticmethod** because the function logically belongs in this class's namespace but does not depend on either the instance or the class — it is a pure helper that happens to be co-located with related code.
- **type-annotated parameters** — Parameters carry type hints (e.g., latest_evaluation_id: int). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type -> List[Dict]. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.

Step by step (top-level body)

1. Assigns chain = [].
2. Assigns curr_id = latest_evaluation_id.
3. **with** get_connection(): enters a context manager.
4. **return** list(reversed(chain)).

Code (copy-paste safe)

```

def get_phase_chain(latest_evaluation_id: int) -> List[Dict]:
    """Traverses up the parent_id chain to get full history."""
    chain = []
    curr_id = latest_evaluation_id

```



```

with get_connection() as conn:
    cursor = conn.cursor()
    while curr_id:
        cursor.execute("SELECT * FROM evaluations WHERE id=?", (curr_id,))
        row = cursor.fetchone()
        if not row:
            break
        chain.append(dict(row))
        curr_id = row['parent_id']

return list(reversed(chain))

```

Step 11.9 — Build dashboard/watermark_service.py

What to do. Create the file dashboard/watermark_service.py in your project root and paste in the code shown in this step. The file is **688** lines long; we walk through it piece by piece below.

Depends on. This file imports from internal modules built in earlier steps: dashboard. If any of those imports fail, jump back to the step that introduced them.

What this file is for. Tracks high-water marks for accounts so that payout calculations only credit profit above the previous peak.

dashboard/watermark_service.py

688 loc · 0 classes · 22 functions · 3 imports

Tracks high-water marks for accounts so that payout calculations only credit profit above the previous peak. It defines **22** module-level functions, across **688** lines.

Python concepts demonstrated in this module

• Comprehensions • Context managers (with-statements) • Exceptions • f-strings • Lambdas

Imports — what each one is for

from dashboard.database import get_connection

Why imported. Internal package — the Flask dashboard. See chapter on dashboard/.

Used in this file. get_connection

from datetime import datetime
from datetime import timedelta

Why imported. Dates, times, durations. The fundamental temporal types: date, time, datetime, timedelta, timezone.

Used in this file. datetime, timedelta

Example. datetime.now() - timedelta(days=7) # one week ago

Other common scenarios.

- `datetime.strptime(s, '%Y-%m-%d')` parses a string
- `datetime.strftime(fmt)` formats one
- `datetime.fromtimestamp(n)` converts a Unix epoch
- `datetime.utcnow()` for UTC (better: `datetime.now(timezone.utc)`)

`import logging`

Why imported. Structured, levelled logging. Replaces `print()` with filterable, formattable output that can route to files, stderr, syslog, or HTTP endpoints.

Used in this file. `logging.error`, `logging.info`

Example. `logging.getLogger(__name__).info('connected to %s', host)`

Other common scenarios.

- `.debug()` / `.info()` / `.warning()` / `.error()` / `.exception()`
- `logger.exception()` captures the current traceback automatically
- `logging.basicConfig(level=logging.INFO)` for quick setup
- Handlers and Formatters route logs to files / streams / syslog

Functions

```
def save_daily_profit(client_id, net_profit, date_str=None, source='auto')
```

Saves or updates the daily net profit for a client. `date_str`: 'YYYY-MM-DD', defaults to today.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **if** not `date_str`: branches conditionally.
2. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def save_daily_profit(client_id, net_profit, date_str=None, source='auto'):
    """
    Saves or updates the daily net profit for a client.
    date_str: 'YYYY-MM-DD', defaults to today.
    """
    if not date_str:
        date_str = datetime.now().strftime('%Y-%m-%d')
```

```

try:
    with get_connection() as conn:
        cursor = conn.cursor()
        # Skip write if value hasn't changed for this date
        cursor.execute(
            'SELECT net_profit_complete FROM daily_watermarks WHERE client_id = ? AND date = ?',
            (client_id, date_str)
        )
        existing = cursor.fetchone()
        if existing and abs(float(existing['net_profit_complete']) - float(net_profit)) < 0.005:
            return True # Unchanged – skip write
        cursor.execute('''
            INSERT INTO daily_watermarks (client_id, date, net_profit_complete, source, created_at)
            VALUES (?, ?, ?, ?, CURRENT_TIMESTAMP)
            ON CONFLICT (client_id, date) DO UPDATE
            SET net_profit_complete = EXCLUDED.net_profit_complete,
                source = EXCLUDED.source,
                created_at = CURRENT_TIMESTAMP
            ''', (client_id, date_str, float(net_profit), source))
        conn.commit()
        logging.info(f"Saved daily watermark for {client_id} on {date_str}: ${net_profit} ({source}")
        return True
except Exception as e:
    logging.error(f"Error saving daily profit: {e}")
    return False

```

```
def get_watermark_history(client_id, days=30)
```

Returns list of dicts: [{'date': 'YYYY-MM-DD', 'profit': 123.45}, ...] Sorted by date.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **list comprehension** — Builds a list in a single expression ([f(x) for x in xs]). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def get_watermark_history(client_id, days=30):
    """
    Returns list of dicts: [{'date': 'YYYY-MM-DD', 'profit': 123.45}, ...]
    Sorted by date.
    """
    try:
        cutoff_date = (datetime.now() - timedelta(days=days)).strftime('%Y-%m-%d')
        with get_connection() as conn:
            cursor = conn.cursor()

```

```

        cursor.execute('''
            SELECT date, net_profit_complete
            FROM daily_watermarks
            WHERE client_id = ? AND date >= ?
            ORDER BY date ASC
        ''', (client_id, cutoff_date))
        rows = cursor.fetchall()
        return [{'date': row['date'], 'profit': row['net_profit_complete']} for row in rows]
    except Exception as e:
        logging.error(f"Error getting watermark history: {e}")
        return []

```

```
def get_lower_watermark(client_id, days=14)
```

Returns the minimum profit recorded in the last X days. Returns: {'date': 'YYYY-MM-DD', 'profit': 123.45} or None

Why these Python concepts were used

- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.

Step by step (top-level body)

1. Assigns history = get_watermark_history(client_id, days).
2. **if** not history: branches conditionally.
3. Assigns min_record = min(history, key=lambda x: x['profit']).
4. **return** min_record.

Code (copy-paste safe)

```

def get_lower_watermark(client_id, days=14):
    """
    Returns the minimum profit recorded in the last X days.
    Returns: {'date': 'YYYY-MM-DD', 'profit': 123.45} or None
    """
    history = get_watermark_history(client_id, days)
    if not history:
        return None

    # Calculate min
    min_record = min(history, key=lambda x: x['profit'])
    return min_record

```

```
def get_high_watermark(client_id, days=14)
```

Returns the maximum profit recorded in the last X days. Returns: {'date': 'YYYY-MM-DD', 'profit': 123.45} or None

Why these Python concepts were used

- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.

Step by step (top-level body)

1. Assigns `history = get_watermark_history(client_id, days)`.
2. **if** not `history`: branches conditionally.
3. Assigns `max_record = max(history, key=lambda x: x['profit'])`.
4. **return** `max_record`.

Code (copy-paste safe)

```
def get_high_watermark(client_id, days=14):
    """
    Returns the maximum profit recorded in the last X days.
    Returns: {'date': 'YYYY-MM-DD', 'profit': 123.45} or None
    """
    history = get_watermark_history(client_id, days)
    if not history:
        return None

    # Calculate max
    max_record = max(history, key=lambda x: x['profit'])
    return max_record
```

```
def get_bulk_watermarks(days=14)
```

Returns a dictionary of watermarks for all clients over the last X days. Returns: {client_id: {'high': float, 'low': float}}

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def get_bulk_watermarks(days=14):
    """
    Returns a dictionary of watermarks for all clients over the last X days.
    Returns: {client_id: {'high': float, 'low': float}}
    """
    try:
        cutoff_date = (datetime.now() - timedelta(days=days)).strftime('%Y-%m-%d')
        with get_connection() as conn:
            cursor = conn.cursor()
            cursor.execute('''
                SELECT client_id, MAX(net_profit_complete) as high, MIN(net_profit_complete) as low
                FROM daily_watermarks
                WHERE date >= ?
                GROUP BY client_id
            ''')
```

```

'''', (cutoff_date,))
rows = cursor.fetchall()
# rows are usually tuples or dict-like depending on row_factory.
# Assuming sqlite3.Row or tuple.
result = {}
for row in rows:
    # row accessible by index if tuple, or key if Row.
    # standard sqlite3 without row_factory returns tuples.
    # But dashboard/database.py might set row_factory.
    # Let's assume keys work if Row, or index if tuple?
    # Safer: check type or try/except.
    # Actually, check get_connection implementation.
    try: # dict-like
        c_id = row['client_id']
        high = row['high']
        low = row['low']
    except: # tuple
        c_id = row[0]
        high = row[1]
        low = row[2]

    result[c_id] = {'high': high, 'low': low}
return result
except Exception as e:
    logging.error(f"Error getting bulk watermarks: {e}")
return {}

```

```
def get_aggregate_watermarks(days=14)
```

Returns the SUM of High and Low Watermarks for all clients (where individual values > 0) over the last X days. Returns: {'hwm': float, 'lwm': float}

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def get_aggregate_watermarks(days=14):
    """
    Returns the SUM of High and Low Watermarks for all clients (where individual values > 0)
    over the last X days.
    Returns: {'hwm': float, 'lwm': float}
    """
    try:
        # Re-using get_bulk_watermarks to get individual HWM/LWM per client
        cutoff_date = (datetime.now() - timedelta(days=days)).strftime('%Y-%m-%d')

```

```

bulk_data = get_bulk_watermarks(days)

sum_hwm = 0.0
sum_lwm = 0.0

for client_id, metrics in bulk_data.items():
    h = metrics.get('high')
    l = metrics.get('low')

    # Convert to float and filter > 0
    try:
        h_val = float(h) if h is not None else 0.0
    except: h_val = 0.0

    try:
        l_val = float(l) if l is not None else 0.0
    except: l_val = 0.0

    if h_val > 0:
        sum_hwm += h_val

    if l_val > 0:
        sum_lwm += l_val

    return {
        'hwm': sum_hwm,
        'lwm': sum_lwm
    }
except Exception as e:
    logging.error(f"Error getting aggregate watermarks: {e}")
    return {'hwm': 0.0, 'lwm': 0.0}

```

```
def bulk_save_history(client_id, history_data)
```

Bulk saves history from external source (e.g., sheet). Overwrites all existing daily_watermarks for this client on each import. history_data: list of {'date': 'YYYY-MM-DD', 'profit': 123.45}

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def bulk_save_history(client_id, history_data):
    """
    Bulk saves history from external source (e.g., sheet).
    Overwrites all existing daily_watermarks for this client on each import.

```

```

history_data: list of {'date': 'YYYY-MM-DD', 'profit': 123.45}
"""
try:
    with get_connection() as conn:
        cursor = conn.cursor()
        # Clear existing daily watermarks for this client then insert fresh
        cursor.execute('DELETE FROM daily_watermarks WHERE client_id = ?', (client_id,))
        for record in history_data:
            cursor.execute('''
                INSERT INTO daily_watermarks (client_id, date, net_profit_complete, source, created_at)
                VALUES (?, ?, ?, 'sheet_import', CURRENT_TIMESTAMP)
            ''', (client_id, record['date'], float(record['profit'])))
        conn.commit()
        logging.info(f"Overwrote daily_watermarks for {client_id}: {len(history_data)} records")
        return True
except Exception as e:
    logging.error(f"Error bulk saving history: {e}")
    return False

```

```
def save_waterlog_periods(client_id, periods, period_values=None)
```

Stores the bi-weekly period schedule for a client. periods: list of (from_date_str, to_date_str) in 'YYYY-MM-DD' format. period_values: optional dict keyed by from_date_str -> {'low': float, 'high': float, 'split_pct': int} If provided, stores the sheet's actual Low/High/split_pct per period so compute_waterlog_from_db() uses them directly instead of recomputing. Existing periods for this client are replaced.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def save_waterlog_periods(client_id, periods, period_values=None):
    """
    Stores the bi-weekly period schedule for a client.
    periods: list of (from_date_str, to_date_str) in 'YYYY-MM-DD' format.
    period_values: optional dict keyed by from_date_str ->
        {'low': float, 'high': float, 'split_pct': int}
        If provided, stores the sheet's actual Low/High/split_pct per period so
        compute_waterlog_from_db() uses them directly instead of recomputing.
    Existing periods for this client are replaced.
    """
    try:
        with get_connection() as conn:
            cursor = conn.cursor()
            # Clear existing schedule for this client then insert fresh

```



```

        cursor.execute('DELETE FROM waterlog_periods WHERE client_id = ?', (client_id,))
    for (from_d, to_d) in periods:
        vals = (period_values or {}).get(from_d, {})
        cursor.execute(
            '''INSERT INTO waterlog_periods
               (client_id, from_date, to_date, period_low, period_high, split_pct)
               VALUES (?, ?, ?, ?, ?, ?)
               ON CONFLICT (client_id, from_date) DO UPDATE
               SET to_date = EXCLUDED.to_date,
                   period_low = EXCLUDED.period_low,
                   period_high = EXCLUDED.period_high,
                   split_pct = EXCLUDED.split_pct'''
            , (client_id, from_d, to_d,
              vals.get('low'),
              vals.get('high'),
              vals.get('split_pct'))
        )
    conn.commit()
    logging.info(f"Saved {len(periods)} waterlog periods for {client_id}")
    return True
except Exception as e:
    logging.error(f"Error saving waterlog periods: {e}")
    return False

```

```
def get_waterlog_periods(client_id)
```

Returns the stored bi-weekly period schedule for a client. Returns: list of (from_date_str, to_date_str) sorted ascending.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **list comprehension** — Builds a list in a single expression ([f(x) for x in xs]). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def get_waterlog_periods(client_id):
    """
    Returns the stored bi-weekly period schedule for a client.
    Returns: list of (from_date_str, to_date_str) sorted ascending.
    """
    try:
        with get_connection() as conn:
            cursor = conn.cursor()
            cursor.execute(

```

```

        'SELECT from_date, to_date FROM waterlog_periods WHERE client_id = ? ORDER BY from_date ASC'
        (client_id,)
    )
    return [(row['from_date'], row['to_date']) for row in cursor.fetchall()]
except Exception as e:
    logging.error(f"Error getting waterlog periods: {e}")
    return []

```

```
def get_waterlog_periods_with_values(client_id)
```

Returns the stored bi-weekly periods including sheet-imported Low/High/split_pct. Returns: list of dicts with from_date, to_date, period_low, period_high, split_pct

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def get_waterlog_periods_with_values(client_id):
    """
    Returns the stored bi-weekly periods including sheet-imported Low/High/split_pct.
    Returns: list of dicts with from_date, to_date, period_low, period_high, split_pct
    """
    try:
        with get_connection() as conn:
            cursor = conn.cursor()
            cursor.execute(
                '''SELECT from_date, to_date, period_low, period_high, split_pct
                   FROM waterlog_periods WHERE client_id = ? ORDER BY from_date ASC''',
                (client_id,)
            )
            return [dict(row) for row in cursor.fetchall()]
    except Exception as e:
        logging.error(f"Error getting waterlog periods with values: {e}")
        return []

```

```
def get_all_daily_watermarks(client_id)
```

Returns ALL daily watermark rows for a client (no date cutoff). Returns: list of (date_obj, float)

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Lazy import from datetime.
2. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def get_all_daily_watermarks(client_id):
    """
    Returns ALL daily watermark rows for a client (no date cutoff).
    Returns: list of (date_obj, float)
    """
    from datetime import datetime as _dt
    try:
        with get_connection() as conn:
            cursor = conn.cursor()
            cursor.execute(
                'SELECT date, net_profit_complete FROM daily_watermarks WHERE client_id = ? ORDER BY date'
                (client_id,)
            )
            result = []
            for row in cursor.fetchall():
                try:
                    d = _dt.strptime(row['date'], '%Y-%m-%d').date()
                    result.append((d, float(row['net_profit_complete'])))
                except Exception:
                    pass
            return result
    except Exception as e:
        logging.error(f"Error getting all daily watermarks: {e}")
        return []
```

```
def get_client_split_pct(client_id)
```

Get the client's default profit split percentage. Policy: always 50% unless a per-period override exists in split_pct_overrides. Legacy values that may sit in identity.split_pct or waterlog_periods.split_pct (imported from sheets) are intentionally ignored — only explicit admin edits via /api/client/split_pct_override may deviate from the default. Returns int (50).

Step by step (top-level body)

1. **return** 50.

Code (copy-paste safe)

```
def get_client_split_pct(client_id):
    """Get the client's default profit split percentage.
    Policy: always 50% unless a per-period override exists in split_pct_overrides.
```

```

Legacy values that may sit in identity.split_pct or waterlog_periods.split_pct
(imported from sheets) are intentionally ignored – only explicit admin edits
via /api/client/split_pct_override may deviate from the default.
Returns int (50)."""
return 50

```

```
def compute_waterlog_from_db(client_id)
```

Computes the Profit Share History table entirely from DB data. For each period: - If period_low/period_high were stored at import time (from the sheet), those exact values are used — this keeps historical data identical to the Google Sheet. - For periods without stored values (new periods after import), Low/High are computed from daily_watermarks as min/max. Returns dict with 'periods' list and 'last_split_net_profit' float, or None if no periods are stored yet (triggers fall-back to sheet). Periods before 2/24/2026 use old HWM logic. Periods overlapping 2/24-3/20/2026 are condensed into one transition row. Periods after 3/20/2026 are generated monthly with new split logic: split = 50% of (current_net_profit - last_split_net_profit) if positive.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **list comprehension** — Builds a list in a single expression ([f(x) for x in xs]). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Lazy import from datetime.
2. Assigns periods_with_vals = get_waterlog_periods_with_values(client_id).
3. if not periods_with_vals: branches conditionally.
4. Assigns daily = get_all_daily_watermarks(client_id).
5. Assigns TRANSITION_START = _date(2026, 2, 24).
6. Assigns TRANSITION_END = _date(2026, 3, 20).
7. Defines a nested function _fmt_date(...).
8. Defines a nested function _fmt_currency(...).
9. Calls periods_with_vals.sort(...) for its side effect.
10. Assigns np_overrides = get_net_profit_overrides(client_id).

11. Assigns `spct_overrides = get_split_pct_overrides(client_id)`.

12. Assigns `result = []`.

13. Assigns `hwm_low = 0.0`.

14. Assigns `last_pre_transition_net = 0.0`.

15. ... and 19 more statement(s) in the body.

Code (copy-paste safe)

```
def compute_waterlog_from_db(client_id):
    """
    Computes the Profit Share History table entirely from DB data.

    For each period:
    - If period_low/period_high were stored at import time (from the sheet),
      those exact values are used – this keeps historical data identical to
      the Google Sheet.
    - For periods without stored values (new periods after import), Low/High
      are computed from daily_watermarks as min/max.

    Returns dict with 'periods' list and 'last_split_net_profit' float,
    or None if no periods are stored yet (triggers fall-back to sheet).

    Periods before 2/24/2026 use old HWM logic.
    Periods overlapping 2/24-3/20/2026 are condensed into one transition row.
    Periods after 3/20/2026 are generated monthly with new split logic:
      split = 50% of (current_net_profit - last_split_net_profit) if positive.
    """
    from datetime import datetime as _dt, date as _date

    periods_with_vals = get_waterlog_periods_with_values(client_id)
    if not periods_with_vals:
        return None # No schedule stored – caller falls back to sheet

    daily = get_all_daily_watermarks(client_id) # [(date_obj, float)]

    TRANSITION_START = _date(2026, 2, 24)
    TRANSITION_END   = _date(2026, 3, 20)

    def _fmt_date(d):
        return f"{d.month}/{d.day}/{d.year}"

    def _fmt_currency(v):
        if v < 0:
            return f"-${abs(v):,.2f}"
        return f"${v:,.2f}"

    # CRITICAL: periods must be sorted oldest-first for correct HWM calculation
    periods_with_vals.sort(key=lambda p: p['from_date'])

    # Load net profit overrides upfront so they feed into HWM / profit split calculation
    np_overrides = get_net_profit_overrides(client_id)

    # Load per-period split percentage overrides
    spct_overrides = get_split_pct_overrides(client_id)

    result = []
    hwm_low = 0.0
    last_pre_transition_net = 0.0 # net profit of the period immediately before transition
```



```

if net_profit > effective_base and net_profit > 0:
    profit_split = (net_profit - effective_base) * monthly_split_pct / 100
else:
    profit_split = 0.0

result.append({
    'from_date': _fmt_date(month_start),
    'to_date': _fmt_date(month_end), # Always show full period end
    'low': _fmt_currency(net_profit),
    'profit_split': f"${profit_split:,.0f}" if profit_split > 0 else '$0',
    'split_pct': monthly_split_pct,
})

# For completed months, update the reference point and HWM
if effective_end >= month_end:
    prev_period_net = net_profit
    if net_profit > hwm_net:
        hwm_net = net_profit

# Advance to next month
month_start = month_end + timedelta(days=1)
if month_start > today:
    break

# Net profit overrides are now applied during HWM calculation above
# (no post-processing needed)

# Apply any manual profit_split overrides (legacy – admin edits from the UI)
overrides = get_profit_split_overrides(client_id)
if overrides:
    for period in result:
        key = period['from_date']
        if key in overrides:
            val = overrides[key]
            period['profit_split'] = f"${val:,.0f}" if val > 0 else '$0'
            period['profit_split_override'] = True

return {
    'periods': result,
    'last_split_net_profit': hwm_net,
}

```

```
def _ensure_profit_split_overrides_table()
```

Create the profit_split_overrides table if it doesn't exist.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def _ensure_profit_split_overrides_table():
    """Create the profit_split_overrides table if it doesn't exist."""
    try:
        with get_connection() as conn:
            conn.execute('''
                CREATE TABLE IF NOT EXISTS profit_split_overrides (
                    client_id    TEXT NOT NULL,
                    from_date    TEXT NOT NULL,
                    amount        REAL NOT NULL,
                    updated_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                    PRIMARY KEY (client_id, from_date)
                )
            ''')
            conn.commit()
    except Exception as e:
        logging.error(f"Error creating profit_split_overrides table: {e}")
```

```
def get_profit_split_overrides(client_id)
```

Return dict of from_date -> amount for all overrides for this client.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **dict comprehension** — Builds a dict in one expression (`{k: v for k, v in items}`) — same intent as a list comprehension but for mappings.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Calls `_ensure_profit_split_overrides_table(...)` for its side effect.
2. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def get_profit_split_overrides(client_id):
    """Return dict of from_date -> amount for all overrides for this client."""
    _ensure_profit_split_overrides_table()
    try:
        with get_connection() as conn:
            cursor = conn.cursor()
            cursor.execute(
                'SELECT from_date, amount FROM profit_split_overrides WHERE client_id = ?',
                (client_id,)
            )
            return {row['from_date']: float(row['amount']) for row in cursor.fetchall()}
    except Exception as e:
```

```
logging.error(f"Error loading profit split overrides: {e}")
return {}
```

```
def save_profit_split_override(client_id, from_date, amount)
```

Save or update a profit split override for a specific period.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Calls `_ensure_profit_split_overrides_table(...)` for its side effect.
2. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def save_profit_split_override(client_id, from_date, amount):
    """Save or update a profit split override for a specific period."""
    _ensure_profit_split_overrides_table()
    try:
        with get_connection() as conn:
            conn.execute('''
                INSERT INTO profit_split_overrides (client_id, from_date, amount, updated_at)
                VALUES (?, ?, ?, CURRENT_TIMESTAMP)
                ON CONFLICT (client_id, from_date) DO UPDATE
                SET amount = EXCLUDED.amount,
                    updated_at = CURRENT_TIMESTAMP
            ''', (client_id, from_date, float(amount)))
            conn.commit()
            logging.info(f"Saved profit split override for {client_id} period {from_date}: ${amount}")
            return True
    except Exception as e:
        logging.error(f"Error saving profit split override: {e}")
        return False
```

```
def _ensure_net_profit_overrides_table()
```

Create the net_profit_overrides table if it doesn't exist.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def _ensure_net_profit_overrides_table():
    """Create the net_profit_overrides table if it doesn't exist."""
    try:
        with get_connection() as conn:
            conn.execute('''
                CREATE TABLE IF NOT EXISTS net_profit_overrides (
                    client_id    TEXT NOT NULL,
                    from_date    TEXT NOT NULL,
                    amount        REAL NOT NULL,
                    updated_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                    PRIMARY KEY (client_id, from_date)
                )
            ''')
            conn.commit()
    except Exception as e:
        logging.error(f"Error creating net_profit_overrides table: {e}")
```

```
def get_net_profit_overrides(client_id)
```

Return dict of from_date -> amount for all net profit overrides for this client.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **dict comprehension** — Builds a dict in one expression (`{k: v for k, v in items}`) — same intent as a list comprehension but for mappings.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Calls `_ensure_net_profit_overrides_table(...)` for its side effect.
2. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def get_net_profit_overrides(client_id):
    """Return dict of from_date -> amount for all net profit overrides for this client."""
    _ensure_net_profit_overrides_table()
    try:
        with get_connection() as conn:
            cursor = conn.cursor()
            cursor.execute(
```

```

        'SELECT from_date, amount FROM net_profit_overrides WHERE client_id = ?',
        (client_id,)
    )
    return {row['from_date']: float(row['amount']) for row in cursor.fetchall()}
except Exception as e:
    logging.error(f"Error loading net profit overrides: {e}")
    return {}

```

```
def save_net_profit_override(client_id, from_date, amount)
```

Save or update a net profit override for a specific period.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Calls `_ensure_net_profit_overrides_table(...)` for its side effect.
2. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def save_net_profit_override(client_id, from_date, amount):
    """Save or update a net profit override for a specific period."""
    _ensure_net_profit_overrides_table()
    try:
        with get_connection() as conn:
            conn.execute('''
                INSERT INTO net_profit_overrides (client_id, from_date, amount, updated_at)
                VALUES (?, ?, ?, CURRENT_TIMESTAMP)
                ON CONFLICT (client_id, from_date) DO UPDATE
                SET amount = EXCLUDED.amount,
                    updated_at = CURRENT_TIMESTAMP
            ''', (client_id, from_date, float(amount)))
            conn.commit()
            logging.info(f"Saved net profit override for {client_id} period {from_date}: ${amount}")
            return True
    except Exception as e:
        logging.error(f"Error saving net profit override: {e}")
        return False

```

```
def _ensure_split_pct_overrides_table()
```

Create the split_pct_overrides table if it doesn't exist.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't

have to handle it.

- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def _ensure_split_pct_overrides_table():
    """Create the split_pct_overrides table if it doesn't exist."""
    try:
        with get_connection() as conn:
            conn.execute('''
                CREATE TABLE IF NOT EXISTS split_pct_overrides (
                    client_id    TEXT NOT NULL,
                    from_date    TEXT NOT NULL,
                    pct          REAL NOT NULL,
                    updated_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                    PRIMARY KEY (client_id, from_date)
                )
            ''')
            conn.commit()
    except Exception as e:
        logging.error(f"Error creating split_pct_overrides table: {e}")

def get_split_pct_overrides(client_id)
    """Return dict of from_date -> pct for all split percentage overrides for this client.
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **dict comprehension** — Builds a dict in one expression (`{k: v for k, v in items}`) — same intent as a list comprehension but for mappings.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Calls `_ensure_split_pct_overrides_table(...)` for its side effect.
2. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def get_split_pct_overrides(client_id):
    """Return dict of from_date -> pct for all split percentage overrides for this client."""
```

```

_ensure_split_pct_overrides_table()
try:
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute(
            'SELECT from_date, pct FROM split_pct_overrides WHERE client_id = ?',
            (client_id,)
        )
        return {row['from_date']: float(row['pct']) for row in cursor.fetchall()}
except Exception as e:
    logging.error(f"Error loading split pct overrides: {e}")
    return {}

```

```
def save_split_pct_override(client_id, from_date, pct)
```

Save or update a split percentage override for a specific period.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Calls `_ensure_split_pct_overrides_table(...)` for its side effect.
2. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def save_split_pct_override(client_id, from_date, pct):
    """Save or update a split percentage override for a specific period."""
    _ensure_split_pct_overrides_table()
    try:
        pct = float(pct)
        if pct < 0 or pct > 100:
            logging.error(f"Invalid split_pct value {pct} – must be 0-100")
            return False
        with get_connection() as conn:
            conn.execute('''
                INSERT INTO split_pct_overrides (client_id, from_date, pct, updated_at)
                VALUES (?, ?, ?, CURRENT_TIMESTAMP)
                ON CONFLICT (client_id, from_date) DO UPDATE
                SET pct = EXCLUDED.pct,
                    updated_at = CURRENT_TIMESTAMP
            ''', (client_id, from_date, pct))
            conn.commit()
            logging.info(f"Saved split pct override for {client_id} period {from_date}: {pct}%")
            return True
    except Exception as e:
        logging.error(f"Error saving split pct override: {e}")
        return False

```

Step 11.10 — Build dashboard/financial_overview.py

What to do. Create the file dashboard/financial_overview.py in your project root and paste in the code shown in this step. The file is **1718** lines long; we walk through it piece by piece below.

Depends on. This file imports from internal modules built in earlier steps: dashboard. If any of those imports fail, jump back to the step that introduced them.

What this file is for. Aggregates deposits, withdrawals, fees, and payouts into the Financial Overview tab the admin dashboard renders.

dashboard/financial_overview.py

1718 loc · 1 classes · 24 functions · 6 imports

Aggregates deposits, withdrawals, fees, and payouts into the Financial Overview tab the admin dashboard renders. It defines **1** class and **24** module-level functions, across **1,718** lines.

Python concepts demonstrated in this module

• Classes and objects • Comprehensions • Decorators • Dunder methods • Exceptions • f-strings • Module-level state • Lambdas

Imports — what each one is for

```
from dashboard.database import get_all_clients
```

Why imported. Internal package — the Flask dashboard. See chapter on dashboard/.

Used in this file. get_all_clients

```
import re
```

Why imported. Regular expressions. Pattern matching, search, replace, and text extraction.

Used in this file. *No direct attribute access detected — likely imported for side effects, re-export, or string references.*

Example. re.search(r'\d{3}-\d{4}', text) # returns a Match or None

Other common scenarios.

- re.findall(pattern, text) for every non-overlapping match
- re.sub(pattern, repl, text) to replace occurrences
- re.compile(...) to reuse a pattern (faster in a loop)
- Named groups: r'(?P<year>\d{4})' lets you do m.group('year')

```
from datetime import datetime
from datetime import timedelta
```

Why imported. Dates, times, durations. The fundamental temporal types: date, time, datetime, timedelta, timezone.

Used in this file. datetime

Example. datetime.now() - timedelta(days=7) # one week ago

Other common scenarios.

- datetime.strptime(s, '%Y-%m-%d') parses a string
- datetime.strftime(fmt) formats one
- datetime.fromtimestamp(n) converts a Unix epoch
- datetime.utcnow() for UTC (better: datetime.now(timezone.utc))

```
import json
```

Why imported. JSON encoding and decoding — the standard wire format for config files, API payloads, and inter-process messages.

Used in this file. json.loads

Example. json.loads(response.text) # parse a JSON string to dict

Other common scenarios.

- json.dumps(obj, indent=2) for pretty-printing
- json.dump(obj, file) / json.load(file) for file I/O
- default=str handles non-serialisable types like datetime
- object_hook= customises decoding (e.g., to dataclass instances)

```
import functools
```

Why imported. Higher-order helpers — caching, partial application, decorator authoring tools, reducers.

Used in this file. functools.wraps

Example. @lru_cache(maxsize=128) # memoise this pure function

Other common scenarios.

- partial(f, x=1) pre-binds arguments
- reduce(f, iterable) folds an iterable to a single value
- @wraps(fn) preserves __name__ / __doc__ when writing decorators
- @cached_property memoises a property per-instance

```
import time
```

Why imported. Timestamps and sleep. Lower-level than datetime — works in Unix-epoch seconds.

Used in this file. time.time

Example. time.sleep(1.5) # pause this thread for 1.5 seconds

Other common scenarios.

- time.time() returns seconds since the epoch

- `time.monotonic()` for measuring elapsed time (never goes backward)
- `time.perf_counter()` for high-precision benchmarking
- `time.strftime` / `time.strptime` mirror the `datetime` versions

Classes

```
class SimpleCache
```

Code (copy-paste safe)

```
class SimpleCache:
    def __init__(self, max_keys=50):
        self._cache = {}
        self._ttl = 300 # 5 minutes default
        self._max_keys = max_keys

    def get(self, key):
        if key in self._cache:
            data, expiry = self._cache[key]
            if time.time() < expiry:
                return data
            else:
                del self._cache[key]
        return None

    def set(self, key, value, ttl=None):
        # Evict expired entries first, then oldest if still over limit
        if len(self._cache) >= self._max_keys:
            self._evict()
        expiration = time.time() + (ttl or self._ttl)
        self._cache[key] = (value, expiration)

    def _evict(self):
        now = time.time()
        # Remove expired entries
        expired = [k for k, (_, exp) in self._cache.items() if now >= exp]
        for k in expired:
            del self._cache[k]
        # If still over limit, remove oldest entries
        while len(self._cache) >= self._max_keys:
            oldest_key = min(self._cache, key=lambda k: self._cache[k][1])
            del self._cache[oldest_key]

    def clear(self):
        self._cache = {}
```

Method: `SimpleCache.__init__`

```
def __init__(self, max_keys=50)
```

Why these Python concepts were used

- `__init__` — The constructor. Runs after Python has allocated the instance; assigns starting attribute values to `self`.

Step by step (top-level body)

1. Assigns `self._cache = {}`.

2. Assigns `self._ttl = 300`.
3. Assigns `self._max_keys = max_keys`.

Code (copy-paste safe)

```
def __init__(self, max_keys=50):
    self._cache = {}
    self._ttl = 300 # 5 minutes default
    self._max_keys = max_keys
```

Method: SimpleCache.get

```
def get(self, key)
```

Step by step (top-level body)

1. if `key in self._cache`: branches conditionally.
2. `return None`.

Code (copy-paste safe)

```
def get(self, key):
    if key in self._cache:
        data, expiry = self._cache[key]
        if time.time() < expiry:
            return data
        else:
            del self._cache[key]
    return None
```

Method: SimpleCache.set

```
def set(self, key, value, ttl=None)
```

Step by step (top-level body)

1. if `len(self._cache) >= self._max_keys`: branches conditionally.
2. Assigns `expiration = time.time() + (ttl or self._ttl)`.
3. Assigns `self._cache[key] = (value, expiration)`.

Code (copy-paste safe)

```
def set(self, key, value, ttl=None):
    # Evict expired entries first, then oldest if still over limit
    if len(self._cache) >= self._max_keys:
        self._evict()
    expiration = time.time() + (ttl or self._ttl)
    self._cache[key] = (value, expiration)
```

Method: SimpleCache._evict

```
def _evict(self)
```

Why these Python concepts were used

- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with `append`, and clearer about intent: this is a transform, not a side-effecting loop.
- **lambda** — Uses a single-expression anonymous function — typically as the `key=` for a sort, the filter predicate, or a small callback.

Step by step (top-level body)

1. Assigns `now = time.time()`.
2. Assigns `expired = [k for k, (_, exp) in self._cache.items() if now >= exp]`.
3. **for** `k` in `expired`: iterates.
4. **while** `len(self._cache) >= self._max_keys`: loops until the condition is false.

Code (copy-paste safe)

```
def _evict(self):
    now = time.time()
    # Remove expired entries
    expired = [k for k, (_, exp) in self._cache.items() if now >= exp]
    for k in expired:
        del self._cache[k]
    # If still over limit, remove oldest entries
    while len(self._cache) >= self._max_keys:
        oldest_key = min(self._cache, key=lambda k: self._cache[k][1])
        del self._cache[oldest_key]
```

Method: SimpleCache.clear

```
def clear(self)
```

Step by step (top-level body)

1. Assigns `self._cache = {}`.

Code (copy-paste safe)

```
def clear(self):
    self._cache = {}
```

Functions

```
def col_idx_to_letter(n)
```

Converts 0-based column index to Excel-style column letters. 0 -> A, 1 -> B, 25 -> Z, 26 -> AA, 27 -> AB

Step by step (top-level body)

1. Assigns `res = ""`.
2. **while** `n >= 0`: loops until the condition is false.
3. **return** `res`.

Code (copy-paste safe)

```
def col_idx_to_letter(n):
    """
    Converts 0-based column index to Excel-style column letters.
```

```

0 -> A, 1 -> B, 25 -> Z, 26 -> AA, 27 -> AB
"""
res = ""
while n >= 0:
    res = chr(ord('A') + (n % 26)) + res
    n = (n // 26) - 1
return res

```

```
def cache_result(ttl=300)
```

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.

Step by step (top-level body)

1. Defines a nested function decorator(...).
2. **return** decorator.

Code (copy-paste safe)

```

def cache_result(ttl=300):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            # Create a cache key based on function name and arguments
            key = f"{func.__name__}:{str(args)}:{str(kwargs)}"
            cached = _overview_cache.get(key)
            if cached is not None:
                return cached

            result = func(*args, **kwargs)
            _overview_cache.set(key, result, ttl)
            return result
        return wrapper
    return decorator

```

```

@cache_result(ttl=10)
def _get_cached_clients()

    Cached wrapper for database.get_all_clients

```

Why these Python concepts were used

- **@cache_result** — Memoises return values keyed by the arguments. Trades memory for speed when the function is pure and called many times with the same inputs.

Step by step (top-level body)

1. **return** get_all_clients().

Code (copy-paste safe)

```
def _get_cached_clients():
```

```

    """Cached wrapper for database.get_all_clients"""
    return get_all_clients()

```

```
def _get_hierarchy_profile_map()
```

Build a {client_name: category} map from the hierarchy JSON (cached 60s).

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **global** — Rebinds a module-level name from inside the function. A smell in larger codebases — usually means a singleton or cache that could be passed in instead.

Step by step (top-level body)

1. Declares globals: `_hierarchy_profile_cache`, `_hierarchy_profile_cache_time`.
2. `if _hierarchy_profile_cache and time.time() - _hierarchy_profile_cache_time < 60:` branches conditionally.
3. `try` block with 1 **except** clause.
4. Assigns `profile_map = {}`.
5. `if SYSTEM_HIERARCHY and 'admins' in SYSTEM_HIERARCHY:` branches conditionally.
6. Assigns `_hierarchy_profile_cache = profile_map`.
7. Assigns `_hierarchy_profile_cache_time = time.time()`.
8. `return profile_map`.

Code (copy-paste safe)

```

def _get_hierarchy_profile_map():
    """Build a {client_name: category} map from the hierarchy JSON (cached 60s)."""
    global _hierarchy_profile_cache, _hierarchy_profile_cache_time
    if _hierarchy_profile_cache and (time.time() - _hierarchy_profile_cache_time) < 60:
        return _hierarchy_profile_cache
    try:
        from config.hierarchy import SYSTEM_HIERARCHY
    except ImportError:
        return {}
    profile_map = {}
    if SYSTEM_HIERARCHY and 'admins' in SYSTEM_HIERARCHY:
        for admin_data in SYSTEM_HIERARCHY['admins'].values():
            for trader_data in admin_data.get('traders', {}).values():
                for client in trader_data.get('clients', []):
                    c_name = client.get('name')
                    cat = (client.get('category') or '').upper()
                    if c_name and cat:
                        profile_map[c_name] = cat
    _hierarchy_profile_cache = profile_map
    _hierarchy_profile_cache_time = time.time()
    return profile_map

```

```
def get_client_profile(client_id, identity=None)
```

Resolve client profile: check identity fields first, then hierarchy category, default PRIVATE.

Why these Python concepts were used

- **ternary expression** — Uses `x` if `cond` else `y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **if** `identity`: branches conditionally.
2. Assigns `h_map = _get_hierarchy_profile_map()`.
3. Assigns `real_name = identity.get('name') if identity else None`.
4. **if** `real_name` and `real_name` in `h_map`: branches conditionally.
5. **if** `client_id` in `h_map`: branches conditionally.
6. **return** `'PRIVATE'`.

Code (copy-paste safe)

```
def get_client_profile(client_id, identity=None):
    """Resolve client profile: check identity fields first, then hierarchy category, default PRIVATE."""
    if identity:
        p = (identity.get('profile') or identity.get('category') or identity.get('source') or '').upper()
        if p:
            return p
    # Fallback: check hierarchy
    h_map = _get_hierarchy_profile_map()
    real_name = identity.get('name') if identity else None
    if real_name and real_name in h_map:
        return h_map[real_name]
    if client_id in h_map:
        return h_map[client_id]
    return 'PRIVATE'
```

```
def parse_currency(value_str)
```

Parses a currency string like "\$120.65", "1,200.00", "-", "\$ -" into a float. Returns 0.0 if the value is missing or represents zero.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.

Step by step (top-level body)

1. **if** `value_str` is `None`: branches conditionally.
2. **if** `isinstance(value_str, (int, float))`: branches conditionally.
3. **if not** `isinstance(value_str, str)`: branches conditionally.
4. Assigns `clean_str = value_str.replace('$', '').replace(',', '').strip()`.
5. **if not** `clean_str` or `clean_str` in `['-', 'n/a', 'null']`: branches conditionally.

6. try block with 1 except clause.

Code (copy-paste safe)

```
def parse_currency(value_str):
    """
    Parses a currency string like "$120.65", "1,200.00", "-", "$ -" into a float.
    Returns 0.0 if the value is missing or represents zero.
    """
    if value_str is None:
        return 0.0
    if isinstance(value_str, (int, float)):
        return float(value_str)

    if not isinstance(value_str, str):
        return 0.0

    # Remove $ and , and spaces
    clean_str = value_str.replace('$', '').replace(',', '').strip()

    if not clean_str or clean_str in ['- ', '\n/a', 'null']:
        return 0.0

    try:
        # Handle parentheses for negative numbers, e.g. (100) -> -100
        if clean_str.startswith('(') and clean_str.endswith(')'):
            return -float(clean_str[1:-1])
        return float(clean_str)
    except ValueError:
        return 0.0

def clear_financial_cache()
```

Invalidate the financial overview cache.

Step by step (top-level body)

1. Calls `_overview_cache.clear(...)` for its side effect.

Code (copy-paste safe)

```
def clear_financial_cache():
    """Invalidate the financial overview cache."""
    _overview_cache.clear()

@cache_result(ttl=30)
def calculate_all_financials(profile_filter=None, start_date=None, end_date=None)
```

Optimized aggregator that computes all financial metrics in a single pass. Returns a dictionary containing all necessary datasets for the dashboard. Optionally filters by start_date and end_date (datetime objects).

Why these Python concepts were used

- **@cache_result** — Memoises return values keyed by the arguments. Trades memory for speed when the function is pure and called many times with the same inputs.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't

have to handle it.

- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to `sum`, `any`, `all`, or `max` where the intermediate list would be wasted.
- **lambda** — Uses a single-expression anonymous function — typically as the `key=` for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns `clients_data = _get_cached_clients()`.
2. Assigns `overview = {}`.
3. Assigns `ts_payouts = []`.
4. Assigns `ts_net_profit = []`.
5. Assigns `ts_fees = []`.
6. Assigns `ts_hedge = []`.
7. Assigns `ts_farming = []`.
8. Lazy import from `collections`.
9. Assigns `deposits_daily = defaultdict(float)`.
10. Assigns `P1_HEDGE_COLS = ['Hedge Result 1', 'Hedge Result 2', 'Hedge Result 3', 'H....`
11. Assigns `FUNDED_HEDGE_COLS = ['Hedge Result 1.1', 'Hedge Result 2.1', 'Hedge Result 3.....`
12. **for** `(client_id, data) in clients_data.items()`: iterates.
13. Assigns `global_stats = {'net': 0.0, 'ended': 0, 'earliest': None, 'expected_valu....`
14. **for** `firm in overview`: iterates.
15. ... and 16 more statement(s) in the body.

Code (copy-paste safe)

```
def calculate_all_financials(profile_filter=None, start_date=None, end_date=None):
    """
    Optimized aggregator that computes all financial metrics in a single pass.
    Returns a dictionary containing all necessary datasets for the dashboard.
    Optionally filters by start_date and end_date (datetime objects).
    """
    clients_data = _get_cached_clients()

    # Initialize containers
```



```

overview = {}

# Time-series containers (list of (date, amount))
ts_payouts = []
ts_net_profit = [] # Events for net profit (payouts, hedges, farming, -fees)
ts_fees = []
ts_hedge = []
ts_farming = []

# Deposits need special handling via deals
# We will do deposits separately or integrate if feasible.
# For now, let's keep deposits separate or integrate if we process deals here too.
# To maximize speed, let's process deals here too if possible.

from collections import defaultdict
deposits_daily = defaultdict(float)

# Constants
P1_HEDGE_COLS = ['Hedge Result 1', 'Hedge Result 2', 'Hedge Result 3', 'Hedge Result 4', 'Hedge Result 5', 'Hedge Result 6', 'Hedge Result 7']
FUNDED_HEDGE_COLS = ['Hedge Result 1.1', 'Hedge Result 2.1', 'Hedge Result 3.1', 'Hedge Result 4.1', 'Hedge Result 5.1', 'Hedge Result 6.1', 'Hedge Result 7.1']

for client_id, data in clients_data.items():
    if not data: continue

    # --- Profile Filtering ---
    if profile_filter and profile_filter.upper() != "ALL":
        identity = data.get('identity', {})
        client_profile = get_client_profile(client_id, identity)
        if client_profile != profile_filter.upper():
            continue

    # --- 1. Process Evaluations (Sheet Data) ---
    evaluations = data.get('evaluations', [])
    for ev in evaluations:
        if not isinstance(ev, dict):
            continue
        # Date filtering
        if start_date or end_date:
            eval_date = parse_date(ev.get('Date Started')) or parse_date(ev.get('Date Purchased')) or parse_date(ev.get('Date'))
            if eval_date:
                if start_date and eval_date.date() < start_date.date():
                    continue
                if end_date and eval_date.date() > end_date.date():
                    continue
        # Prop Firm Overview Logic
        raw_prop_firm = ev.get('Prop Firm')
        if raw_prop_firm and raw_prop_firm != "-" and str(raw_prop_firm).lower() != "prop firm":
            prop_firm = normalize_prop_firm_name(raw_prop_firm)
            if not prop_firm:
                continue

        if prop_firm not in overview:
            overview[prop_firm] = {
                "total_fees": 0.0,
                "total_activation_fees": 0.0,
                "total_payouts": 0.0,
                "net": 0.0,
                "account_count": 0,
            }

```

```

        "hedge_results": 0.0,
        "farming_results": 0.0,
        "active_accounts": 0,
        "passed_accounts": 0,
        "failed_accounts": 0,
        "ended_count": 0,
        "total_duration_days": 0,
        "earliest_date": None,
        "clients": set()
    }

# Financials for Overview
fee = parse_currency(ev.get('Fee'))
activation_fee = parse_currency(ev.get('Activation Fee'))

payouts = 0.0
for i in range(1, 10):
    payouts += parse_currency(ev.get(f'Payout {i}'))

p1_hedges = sum(parse_currency(ev.get(col)) for col in P1_HEDGE_COLS)
funded_hedges = sum(parse_currency(ev.get(col)) for col in FUNDED_HEDGE_COLS)

farming_results = sum(parse_currency(ev.get(f'Hedge Day {i}')) for i in range(1, 51))

# Status Logic
status_p1 = str(ev.get('Status P1', '')).strip()
status_funded = str(ev.get('Status', '')).strip()

# Only count hedge/farming for rows with a populated status
# so accounts without hedging activity don't contribute phantom values.
hedge_results = 0.0
if status_p1:
    hedge_results += p1_hedges
if status_funded:
    hedge_results += funded_hedges
if not status_funded:
    farming_results = 0.0

is_p1_fail = status_p1 == 'Fail'
is_funded_fail = status_funded == 'Fail'
is_funded_completed = status_funded == 'Completed'
is_funded_ended = is_funded_fail or is_funded_completed
is_in_progress = not is_p1_fail and not is_funded_ended
is_passed_p1 = status_p1 == 'Pass' or status_p1.lower() == 'pass'

target = overview[prop_firm]
target["total_fees"] += fee
target["total_activation_fees"] += activation_fee
target["total_payouts"] += payouts
target["hedge_results"] += hedge_results
target["farming_results"] += farming_results
target["account_count"] += 1
target["clients"].add(client_id)

if is_in_progress: target["active_accounts"] += 1
if is_passed_p1: target["passed_accounts"] += 1
if is_p1_fail or is_funded_fail: target["failed_accounts"] += 1
if is_p1_fail or is_funded_ended: target["ended_count"] += 1

# Duration Logic for EV/Day

```

```

duration = 0
if is_p1_fail:
    s_d = parse_date(ev.get('Date Started'))
    e_d = parse_date(ev.get('Date Ended'))
    if s_d and e_d: duration = (e_d - s_d).days
elif is_funded_ended:
    # Duration is total time from start of eval to end of funded account
    s_d = parse_date(ev.get('Date Started'))
    e_d = parse_date(ev.get('Date Ended.1'))
    if s_d and e_d: duration = (e_d - s_d).days
    # Fallback if funded dates are missing but it ended
    elif s_d:
        # Try P1 end date if Funded end date missing? Unlikely for "Ended" status
        pass

if duration > 0:
    target["total_duration_days"] += duration

# Overview Earliest Date
d_str = ev.get('Date Started') or ev.get('Date')
if d_str:
    d_obj = parse_date(d_str)
    if d_obj:
        if target["earliest_date"] is None or d_obj < target["earliest_date"]:
            target["earliest_date"] = d_obj

# Time Series Logic
# Dates
date_purchased = parse_date(ev.get('Date Purchased') or ev.get('Date'))
date_started = parse_date(ev.get('Date Started'))
date_ended = parse_date(ev.get('Date Ended'))
date_started_funded = parse_date(ev.get('Date Started.1'))
date_ended_funded = parse_date(ev.get('Date Ended.1'))
base_date = date_purchased or date_started or datetime.now()

# 1. Fees
fee = parse_currency(ev.get('Fee'))
act_fee = parse_currency(ev.get('Activation Fee'))
total_fee = fee + act_fee
if total_fee > 0:
    fee_date = date_purchased or base_date
    ts_fees.append((fee_date, total_fee))
    ts_net_profit.append((fee_date, -total_fee)) # Cost

# 2. Payouts
for i in range(1, 10):
    d_str = ev.get(f'Date {i}')
    amt = parse_currency(ev.get(f'Payout {i}'))
    if amt > 0:
        d = parse_date(d_str)
        final_d = d or base_date
        ts_payouts.append((final_d, amt))
        ts_net_profit.append((final_d, amt))

# 3. Hedge Results
p1_profit = sum(parse_currency(ev.get(c)) for c in P1_HEDGE_COLS)
if p1_profit != 0:
    d = date_ended or date_started or base_date
    ts_hedge.append((d, p1_profit))
    ts_net_profit.append((d, p1_profit))

```

```

fd_profit = sum(parse_currency(ev.get(c)) for c in FUNDED_HEDGE_COLS)
if fd_profit != 0:
    d = date_ended_funded or date_started_funded or base_date
    ts_hedge.append((d, fd_profit))
    ts_net_profit.append((d, fd_profit))

# 4. Farming Results
farming_calc = sum(parse_currency(ev.get(f'Hedge Day {i}')) for i in range(1, 51))
if farming_calc != 0:
    d = date_ended_funded or date_ended or base_date
    ts_farming.append((d, farming_calc))
    ts_net_profit.append((d, farming_calc))

# --- 2. Process Deals (Deposits) ---
deals_json = data.get('deals', '[]')
try:
    deals = json.loads(deals_json) if isinstance(deals_json, str) else deals_json
except:
    deals = []

if deals:
    for deal in deals:
        d_time = deal.get('time_raw') or deal.get('time')
        if not d_time: continue

        try:
            try:
                dt = datetime.fromtimestamp(int(d_time))
            except (ValueError, TypeError):
                dt = datetime.fromisoformat(str(d_time))
            date_str = dt.strftime("%Y-%m-%d")
        except:
            continue

        # Date filtering for deals
        if start_date and dt.date() < start_date.date():
            continue
        if end_date and dt.date() > end_date.date():
            continue

        d_type = deal.get('type')
        def _f(val):
            try: return float(val)
            except: return 0.0
        profit = _f(deal.get('profit', 0))

        # Check for deposit
        is_balance = str(d_type) == '2' or str(d_type).upper() == 'BALANCE'
        if is_balance and profit > 0:
            deposits_daily[date_str] += profit

# --- Finalize Overview Data ---
global_stats = {
    "net": 0.0,
    "ended": 0,
    "earliest": None,
    "expected_value": 0.0,
    "ev_per_day": 0.0,
    "total_duration": 0
}

```

```

for firm in overview:
    data = overview[firm]
    data["net"] = data["total_payouts"] + data["hedge_results"] + data["farming_results"] - (data[
        "total_fees"] + data["total_activation_fees"])

    # Accumulate global
    global_stats["net"] += data["net"]
    global_stats["ended"] += data["ended_count"]
    global_stats["total_duration"] += data.get("total_duration_days", 0)

    if data.get("earliest_date"):
        if global_stats["earliest"] is None or data["earliest_date"] < global_stats["earliest"]:
            global_stats["earliest"] = data["earliest_date"]

    ended = data.get("ended_count", 0)
    data["expected_value"] = data["net"] / ended if ended > 0 else 0.0

    duration = data.get("total_duration_days", 0)
    data["ev_per_day"] = data["net"] / duration if duration > 0 else 0.0

    if "earliest_date" in data: del data["earliest_date"]
    if "total_duration_days" in data: del data["total_duration_days"]
    data["total_clients"] = len(data["clients"])
    del data["clients"]

# Finalize Global Stats
if global_stats["ended"] > 0:
    global_stats["expected_value"] = global_stats["net"] / global_stats["ended"]

duration = global_stats.get("total_duration", 0)
if duration > 0:
    global_stats["ev_per_day"] = global_stats["net"] / duration

# Remove datetime object before return
if "earliest" in global_stats: del global_stats["earliest"]
if "total_duration" in global_stats: del global_stats["total_duration"]

# --- Finalize Time Series ---
from datetime import date as _date_type
today_str = _date_type.today().strftime("%Y-%m-%d")

def process_ts(events):
    if not events: return [], []
    events.sort(key=lambda x: x[0])
    from collections import defaultdict
    daily = defaultdict(float)
    for dt, val in events:
        day_str = dt.strftime("%Y-%m-%d")
        if day_str <= today_str: # Exclude future dates
            daily[day_str] += val

    dates = []
    vals = []
    cum = 0.0
    for day in sorted(daily.keys()):
        cum += daily[day]
        dates.append(day)
        vals.append(cum)
    # Extend to current date so chart always ends at today
    if dates and dates[-1] < today_str:

```

```

        dates.append(today_str)
        vals.append(cum)
    return dates, vals

# Process Deposits from simple dict
def process_deposits_dict(d_dict):
    if not d_dict: return [], []
    dates = []
    vals = []
    cum = 0.0
    for day in sorted(d_dict.keys()):
        if day > today_str: # Exclude future dates
            break
        cum += d_dict[day]
        dates.append(day)
        vals.append(cum)
    # Extend to current date so chart always ends at today
    if dates and dates[-1] < today_str:
        dates.append(today_str)
        vals.append(cum)
    return dates, vals

payouts_dates, payouts_values = process_ts(ts_payouts)
fees_dates, fees_values = process_ts(ts_fees)
hedge_dates, hedge_values = process_ts(ts_hedge)
farming_dates, farming_values = process_ts(ts_farming)
net_dates, net_values = process_ts(ts_net_profit)
dep_dates, dep_values = process_deposits_dict(deposits_daily)

# Growth data usually refers to Net Profit (or Equity?)
# In original: get_portfolio_growth_data was (Payouts - Fees) basically
# But get_cumulative_trading_profit was the full net profit.
# The chart allows selecting metric.
# We will pass 'net_dates' as 'growth_dates' for default view.

return {
    "overview": overview,
    "global_stats": global_stats,
    "payouts": (payouts_dates, payouts_values),
    "fees": (fees_dates, fees_values),
    "hedge": (hedge_dates, hedge_values),
    "farming": (farming_dates, farming_values),
    "net_profit": (net_dates, net_values),
    "deposits": (dep_dates, dep_values),
    "growth": (net_dates, net_values) # Default growth
}

```

```
def normalize_prop_firm_name(name)
```

Normalizes prop firm names to merge duplicates. Example: "My Funded Futures" and "MyFundedFutures" become "My Funded Futures". Returns None for invalid/junk entries.

Step by step (top-level body)

1. **if** not name: branches conditionally.
2. Assigns original = name.strip().
3. Assigns normalized = original.lower().replace(' ', '').replace('_', '').

4. Assigns JUNK_NAMES = {'other', 'f', 'n/a', 'na', 'none', 'test', '-', ''}.
5. if normalized in JUNK_NAMES or len(normalized) <= 1: branches conditionally.
6. Assigns MAPPING = {'myfundedfutures': 'My Funded Futures', 'myfundedfx': 'M....
7. if normalized in MAPPING: branches conditionally.
8. if 'myfundedfutures' in normalized: branches conditionally.
9. if 'fundednext' in normalized: branches conditionally.
10. return original.

Code (copy-paste safe)

```
def normalize_prop_firm_name(name):
    """
    Normalizes prop firm names to merge duplicates.
    Example: "My Funded Futures" and "MyFundedFutures" become "My Funded Futures".
    Returns None for invalid/junk entries.
    """
    if not name:
        return None

    original = name.strip()
    normalized = original.lower().replace(" ", "").replace("_", "")

    # Skip junk/invalid entries (typos, single chars, generic labels)
    JUNK_NAMES = {'other', 'f', 'n/a', 'na', 'none', 'test', '-', ''}
    if normalized in JUNK_NAMES or len(normalized) <= 1:
        return None

    # Map normalized keys to display names
    MAPPING = {
        "myfundedfutures": "My Funded Futures",
        "myfundedfx": "My Funded Futures",
        "fundednext": "FundedNext",
        "topstep": "Topstep",
        "fundingticks": "Funding Ticks",
        "fundingtick": "Funding Ticks",
        "tradeday": "TradeDay",
        "tradeify": "Tradeify",
        "tradify": "Tradeify",
        "ftmo": "FTMO",
        "alphafutures": "Alpha Futures",
        "blueguardian": "Blue Guardian",
        "fundedtradingplus": "Funded Trading Plus",
        "the5ers": "The 5%ers",
        "apextraderfunding": "Apex Trader Funding",
        "apextrader": "Apex Trader Funding",
        "uprofittrader": "UProfit",
        "uprofit": "UProfit",
        "bulenox": "Bulenox",
        "tickticktrader": "TickTick Trader",
        "elitetraderfunding": "Elite Trader Funding",
        "take profit trader": "Take Profit Trader",
        "takeprofittrader": "Take Profit Trader",
        "toponefutures": "Top One Futures",
        "topone": "Top One Futures",
        "mff": "My Funded Futures",
        "mffu": "My Funded Futures",
    }
```

```

    "mffuflex": "My Funded Futures",
    "fundednextlegacyaccount": "FundedNext (Legacy)", # Keep distinct if wanted, or merge to FundedNext
}

# Direct match in mapping
if normalized in MAPPING:
    return MAPPING[normalized]

# Check if key starts with... (optional, logic for variations)
if "myfundedfutures" in normalized:
    return "My Funded Futures"
if "fundednext" in normalized:
    return "FundedNext"

# Fallback: Just return original if no mapping found, but title cased
return original

```

```
def parse_date(date_str)
```

Parses date string to datetime object.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.

Step by step (top-level body)

1. **if** not `date_str` or not `isinstance(date_str, str)`: branches conditionally.
2. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def parse_date(date_str):
    """Parses date string to datetime object."""
    if not date_str or not isinstance(date_str, str):
        return None
    try:
        # Try MM/DD/YY
        return datetime.strptime(date_str.strip(), "%m/%d/%y")
    except ValueError:
        try:
            # Try YYYY-MM-DD
            return datetime.strptime(date_str.strip(), "%Y-%m-%d")
        except ValueError:
            return None

```

```
def get_payouts_history(start_date=None, end_date=None, prop_firm_filter=None, profile_filter=None)
```

Returns a list of all payouts with details.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **try** block with 1 **except** clause.
2. Assigns `client_map = {}`.
3. **if** `SYSTEM_HIERARCHY` and `'admins'` in `SYSTEM_HIERARCHY`: branches conditionally.
4. Assigns `clients_data = _get_cached_clients()`.
5. Assigns `payouts_list = []`.
6. **for** `(client_id, data)` in `clients_data.items()`: iterates.
7. Calls `payouts_list.sort(...)` for its side effect.
8. **return** `payouts_list`.

Code (copy-paste safe)

```
def get_payouts_history(start_date=None, end_date=None, prop_firm_filter=None, profile_filter=None):
    """
    Returns a list of all payouts with details.
    """
    # Import hierarchy to map clients to traders/admins
    try:
        from config.hierarchy import SYSTEM_HIERARCHY
    except ImportError:
        import sys, os
        sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
        from config.hierarchy import SYSTEM_HIERARCHY

    # Build client map: {client_name: {'trader': T, 'admin': A}}
    client_map = {}
    if SYSTEM_HIERARCHY and 'admins' in SYSTEM_HIERARCHY:
        for admin_name, admin_data in SYSTEM_HIERARCHY['admins'].items():
            for trader_name, trader_data in admin_data.get('traders', {}).items():
                for client in trader_data.get('clients', []):
                    c_name = client.get('name')
                    if c_name:
                        client_map[c_name] = {
                            'admin': admin_name,
                            'trader': trader_name
                        }

    clients_data = _get_cached_clients()
```

```

payouts_list = []

for client_id, data in clients_data.items():
    if not data:
        continue

    # Get Client Metadata
    identity = data.get('identity', {})
    real_client_name = identity.get('name') or client_id

    # Get hierarchy info
    h_info = client_map.get(real_client_name) or client_map.get(client_id)
    admin_name = h_info['admin'] if h_info else "-"
    trader_name = h_info['trader'] if h_info else "-"

    # Apply Profile Filter
    if profile_filter and profile_filter.upper() != "ALL":
        c_prof = get_client_profile(client_id, identity)
        if c_prof != profile_filter.upper():
            continue

    evaluations = data.get('evaluations', [])
    for eval_data in evaluations:
        if not isinstance(eval_data, dict):
            continue
        prop_firm = eval_data.get('Prop Firm')
        if not prop_firm or prop_firm == "-": continue

        prop_firm = normalize_prop_firm_name(prop_firm)
        if not prop_firm: continue

        # Apply prop firm filter if provided
        if prop_firm_filter and prop_firm != prop_firm_filter:
            continue

        account_num = eval_data.get('Account #') or eval_data.get('Account #.1') or '-'

        # Check Payout 1..10
        for i in range(1, 10):
            p_key = f'Payout {i}'
            d_key = f'Date {i}' # Assuming 'Date 1', 'Date 2' etc matches Payout

            amount = parse_currency(eval_data.get(p_key))
            if amount > 0:
                date_str = eval_data.get(d_key)
                date_obj = parse_date(date_str)

                if date_obj:
                    # Filter check
                    if start_date and date_obj < start_date:
                        continue
                    if end_date and date_obj > end_date:
                        continue

                payouts_list.append({
                    "date": date_obj,
                    "date_str": date_str,
                    "prop_firm": prop_firm,
                    "amount": amount,
                    "client_name": real_client_name,

```

```

        "admin_name": admin_name,
        "trader_name": trader_name,
        "account_id": account_num,
        # Keep old keys for safety if used elsewhere
        "client": client_id,
        "account": account_num
    })

    # Sort by date desc
    payouts_list.sort(key=lambda x: x['date'], reverse=True)
    return payouts_list

@cache_result(ttl=300)
def get_payouts_growth_data(profile_filter=None)

    Calculates cumulative payouts over time (ignoring fees). Returns lists of labels (dates) and data points (cumulative payouts).

```

Why these Python concepts were used

- **@cache_result** — Memoises return values keyed by the arguments. Trades memory for speed when the function is pure and called many times with the same inputs.
- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.

Step by step (top-level body)

1. Assigns `clients_data = _get_cached_clients()`.
2. Assigns `events = []`.
3. **for** `(client_id, data)` in `clients_data.items()`: iterates.
4. Calls `events.sort(...)` for its side effect.
5. **if not events**: branches conditionally.
6. Assigns `dates = []`.
7. Assigns `values = []`.
8. Assigns `cumulative = 0.0`.
9. Lazy import from `collections`.
10. Assigns `daily_changes = defaultdict(float)`.
11. **for** `(date_obj, amount)` in `events`: iterates.
12. Assigns `sorted_days = sorted(daily_changes.keys())`.
13. **for** `day` in `sorted_days`: iterates.
14. **return** `(dates, values)`.

Code [\(copy-paste safe\)](#)

```

def get_payouts_growth_data(profile_filter=None):
    """
    Calculates cumulative payouts over time (ignoring fees).
    Returns lists of labels (dates) and data points (cumulative payouts).
    """
    clients_data = _get_cached_clients()
    events = []

    for client_id, data in clients_data.items():
        if not data: continue

        # Apply Profile Filter
        if profile_filter and profile_filter.upper() != "ALL":
            identity = data.get('identity', {})
            client_profile = get_client_profile(client_id, identity)
            if client_profile != profile_filter.upper():
                continue

        evaluations = data.get('evaluations', [])
        for ev in evaluations:
            if not isinstance(ev, dict):
                continue
            # Payouts Only
            for i in range(1, 10):
                date_str = ev.get(f'Date {i}')
                amount = parse_currency(ev.get(f'Payout {i}'))
                if amount > 0 and date_str:
                    date_obj = parse_date(date_str)
                    if date_obj:
                        events.append((date_obj, amount))

    # Sort events by date
    events.sort(key=lambda x: x[0])

    if not events:
        return [], []

    dates = []
    values = []
    cumulative = 0.0

    from collections import defaultdict
    daily_changes = defaultdict(float)

    for date_obj, amount in events:
        date_str = date_obj.strftime("%Y-%m-%d")
        daily_changes[date_str] += amount

    sorted_days = sorted(daily_changes.keys())

    for day in sorted_days:
        cumulative += daily_changes[day]
        dates.append(day)
        values.append(cumulative)

    return dates, values

def get_mt5_deals_data(profile_filter=None)

```

Helper to get processed daily changes for deposits and trading profit. Returns (deposits_daily, profit_daily) dicts: {date_str: amount}

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns `clients_data = _get_cached_clients()`.
2. Lazy import from `collections`.
3. Assigns `deposits_daily = defaultdict(float)`.
4. Assigns `profit_daily = defaultdict(float)`.
5. **for** (`client_id, data`) in `clients_data.items()`: iterates.
6. **return** (`deposits_daily, profit_daily`).

Code (copy-paste safe)

```
def get_mt5_deals_data(profile_filter=None):
    """
    Helper to get processed daily changes for deposits and trading profit.
    Returns (deposits_daily, profit_daily) dicts: {date_str: amount}
    """
    clients_data = _get_cached_clients()

    from collections import defaultdict
    deposits_daily = defaultdict(float)
    profit_daily = defaultdict(float)

    for client_id, data in clients_data.items():
        if not data: continue

        # Apply Profile Filter
        if profile_filter and profile_filter.upper() != "ALL":
            identity = data.get('identity', {})
            client_profile = get_client_profile(client_id, identity)
            if client_profile != profile_filter.upper(): continue

        deals_json = data.get('deals', '[]')
        try:
            deals = json.loads(deals_json) if isinstance(deals_json, str) else deals_json
        except:
            deals = []

        if not deals: continue
```

```

for deal in deals:
    # MT5 deal structure: {'time': epoch, 'type': int, 'profit': float, 'swap': float, 'commission': float}
    d_time = deal.get('time_raw') or deal.get('time')
    if not d_time: continue

    try:
        try:
            dt = datetime.fromtimestamp(int(d_time))
        except (ValueError, TypeError):
            dt = datetime.fromisoformat(str(d_time))
        date_str = dt.strftime("%Y-%m-%d")
    except:
        continue

    def _f(val):
        try: return float(val)
        except: return 0.0

    d_type = deal.get('type')
    profit = _f(deal.get('profit', 0))
    swap = _f(deal.get('swap', 0))
    comm = _f(deal.get('commission', 0))

    # Type 2 is usually BALANCE (Deposits/Withdrawals)
    is_balance = str(d_type) == '2' or str(d_type).upper() == 'BALANCE'

    if is_balance:
        # If profit > 0, it's a deposit. If < 0, it's a withdrawal.
        # User asked to track "Deposits".
        if profit > 0:
            deposits_daily[date_str] += profit
    else:
        # Trading Profit
        trading_profit = profit + swap + comm
        profit_daily[date_str] += trading_profit

return deposits_daily, profit_daily

@cache_result(ttl=300)
def get_cumulative_deposits(profile_filter=None)

```

Calculates cumulative deposits over time.

Why these Python concepts were used

- **@cache_result** — Memoises return values keyed by the arguments. Trades memory for speed when the function is pure and called many times with the same inputs.

Step by step (top-level body)

1. Assigns (deposits_daily, _) = get_mt5_deals_data(profile_filter).
2. if not deposits_daily: branches conditionally.
3. Assigns dates = [].
4. Assigns values = [].
5. Assigns cumulative = 0.0.
6. Assigns sorted_days = sorted(deposits_daily.keys()).

7. **for** day in sorted_days: iterates.

8. **return** (dates, values).

Code (copy-paste safe)

```
def get_cumulative_deposits(profile_filter=None):
    """Calculates cumulative deposits over time."""
    deposits_daily, _ = get_mt5_deals_data(profile_filter)
    if not deposits_daily: return [], []

    dates = []
    values = []
    cumulative = 0.0
    sorted_days = sorted(deposits_daily.keys())

    for day in sorted_days:
        cumulative += deposits_daily[day]
        dates.append(day)
        values.append(cumulative)

    return dates, values
```

```
def parse_date(date_str)
```

Parses date string to datetime object.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.

Step by step (top-level body)

1. **if** not date_str or not isinstance(date_str, str): branches conditionally.
2. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def parse_date(date_str):
    """Parses date string to datetime object."""
    if not date_str or not isinstance(date_str, str):
        return None
    try:
        # Try common formats
        for fmt in ["%m/%d/%y", "%m/%d/%Y", "%Y-%m-%d", "%d-%m-%Y"]:
            try:
                return datetime.strptime(date_str.strip(), fmt)
            except ValueError:
                continue
        return None
    except:
        return None
```

```
@cache_result(ttl=300)
def get_cumulative_trading_profit(profile_filter=None)
```

Calculates cumulative Net Profit over time based on Payouts, Hedge Results, Farming, and Fees. Uses Evaluation data (Sheet) to match the Summary Card 'Net Profit'.

Why these Python concepts were used

- **@cache_result** — Memoises return values keyed by the arguments. Trades memory for speed when the function is pure and called many times with the same inputs.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.
- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.

Step by step (top-level body)

1. Assigns `clients_data = _get_cached_clients()`.
2. Assigns `events = []`.
3. Assigns `P1_HEDGE_COLS = ['Hedge Result 1', 'Hedge Result 2', 'Hedge Result 3', 'H....`
4. Assigns `FUNDED_HEDGE_COLS = ['Hedge Result 1.1', 'Hedge Result 2.1', 'Hedge Result 3.....`
5. **for** `(client_id, data)` in `clients_data.items()`: iterates.
6. **if** not `events`: branches conditionally.
7. Calls `events.sort(...)` for its side effect.
8. Lazy import from `collections`.
9. Assigns `daily_changes = defaultdict(float)`.
10. **for** `(dt, val)` in `events`: iterates.
11. Assigns `dates = []`.
12. Assigns `values = []`.
13. Assigns `cumulative = 0.0`.
14. Assigns `sorted_days = sorted(daily_changes.keys())`.
15. ... and 2 more statement(s) in the body.

Code (copy-paste safe)

```
def get_cumulative_trading_profit(profile_filter=None):
    """
    Calculates cumulative Net Profit over time based on Payouts, Hedge Results, Farming, and Fees.
    Uses Evaluation data (Sheet) to match the Summary Card 'Net Profit'.
    """
    clients_data = _get_cached_clients()
```



```

events = [] # (datetime, amount)

# Columns definition matching calculate_propfirm_overview
P1_HEDGE_COLS = ['Hedge Result 1', 'Hedge Result 2', 'Hedge Result 3', 'Hedge Result 4', 'Hedge Result 5', 'Hedge Result 6', 'Hedge Result 7']
FUNDED_HEDGE_COLS = ['Hedge Result 1.1', 'Hedge Result 2.1', 'Hedge Result 3.1', 'Hedge Result 4.1', 'Hedge Result 5.1', 'Hedge Result 6', 'Hedge Result 7']

for client_id, data in clients_data.items():
    if not data: continue

    # Profile Filter
    if profile_filter and profile_filter.upper() != "ALL":
        identity = data.get('identity', {})
        client_profile = get_client_profile(client_id, identity)
        if client_profile != profile_filter.upper():
            continue

    evaluations = data.get('evaluations', [])
    for ev in evaluations:
        if not isinstance(ev, dict):
            continue

        # Match filtering logic from calculate_propfirm_overview
        raw_prop_firm = ev.get('Prop Firm')
        if not raw_prop_firm or raw_prop_firm == "-" or str(raw_prop_firm).lower() == "prop firm":
            continue

        # Extract Dates
        date_purchased = parse_date(ev.get('Date Purchased') or ev.get('Date'))
        date_started = parse_date(ev.get('Date Started'))
        date_ended = parse_date(ev.get('Date Ended'))
        date_started_funded = parse_date(ev.get('Date Started.1'))
        date_ended_funded = parse_date(ev.get('Date Ended.1'))

        # Default date fallback logic
        # If we have a cost/revenue but no specific date, place it at the closest known date
        base_date = date_purchased or date_started or datetime.now()

        # 1. Fees (Negative)
        fee = parse_currency(ev.get('Fee'))
        act_fee = parse_currency(ev.get('Activation Fee'))
        total_fee = fee + act_fee
        if total_fee > 0:
            events.append((date_purchased or base_date, -total_fee))

        # 2. Payouts (Positive)
        for i in range(1, 10):
            d_str = ev.get(f'Date {i}')
            amt = parse_currency(ev.get(f'Payout {i}'))
            if amt != 0:
                d = parse_date(d_str)
                events.append((d or base_date, amt))

        # 3. Hedge Results P1
        p1_profit = sum(parse_currency(ev.get(c)) for c in P1_HEDGE_COLS)
        if p1_profit != 0:
            # Assign to Date Ended or Date Started
            events.append((date_ended or date_started or base_date, p1_profit))

        # 4. Funded Hedge Results
        fd_profit = sum(parse_currency(ev.get(c)) for c in FUNDED_HEDGE_COLS)

```

```

    if fd_profit != 0:
        # Assign to Date Ended Funded or Date Started Funded
        events.append((date_ended_funded or date_started_funded or base_date, fd_profit))

    # 5. Farming Results
    # Match calculate_propfirm_overview logic: Sum of Hedge Day 1-50
    farming_calc = sum(parse_currency(ev.get(f'Hedge Day {i}')) for i in range(1, 51))

    if farming_calc != 0:
        # Assign to later dates
        events.append((date_ended_funded or date_ended or base_date, farming_calc))

if not events:
    return [], []

# Sort events by date
events.sort(key=lambda x: x[0])

# Aggregate by day
from collections import defaultdict
daily_changes = defaultdict(float)

for dt, val in events:
    d_str = dt.strftime("%Y-%m-%d")
    daily_changes[d_str] += val

dates = []
values = []
cumulative = 0.0
sorted_days = sorted(daily_changes.keys())

for day in sorted_days:
    cumulative += daily_changes[day]
    dates.append(day)
    values.append(cumulative)

return dates, values

@cache_result(ttl=300)
def get_portfolio_growth_data(profile_filter=None)

```

Calculates cumulative portfolio growth over time. Returns lists of labels (dates) and data points (net profit).

Why these Python concepts were used

- **@cache_result** — Memoises return values keyed by the arguments. Trades memory for speed when the function is pure and called many times with the same inputs.
- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.

Step by step (top-level body)

1. Assigns `clients_data = _get_cached_clients()`.
2. Assigns `events = []`.
3. **for** `(client_id, data)` in `clients_data.items()`: iterates.
4. Calls `events.sort(...)` for its side effect.
5. **if not** `events`: branches conditionally.
6. Assigns `dates = []`.
7. Assigns `values = []`.
8. Assigns `cumulative = 0.0`.
9. Lazy import from `collections`.
10. Assigns `daily_changes = defaultdict(float)`.
11. **for** `(date_obj, amount)` in `events`: iterates.
12. Assigns `sorted_days = sorted(daily_changes.keys())`.
13. **for** `day` in `sorted_days`: iterates.
14. **return** `(dates, values)`.

Code (copy-paste safe)

```
def get_portfolio_growth_data(profile_filter=None):
    """
    Calculates cumulative portfolio growth over time.
    Returns lists of labels (dates) and data points (net profit).
    """
    clients_data = _get_cached_clients()

    # Store all financial events: (date, amount)
    events = []

    for client_id, data in clients_data.items():
        if not data: continue

        # Apply Profile Filter
        if profile_filter and profile_filter.upper() != "ALL":
            identity = data.get('identity', {})
            client_profile = get_client_profile(client_id, identity)
            if client_profile != profile_filter.upper():
                continue

        evaluations = data.get('evaluations', [])
        for ev in evaluations:
            if not isinstance(ev, dict):
                continue
            # 1. Payouts (Positive)
            for i in range(1, 10):
                date_str = ev.get(f'Date {i}')
                amount = parse_currency(ev.get(f'Payout {i}'))
                if amount > 0 and date_str:
                    date_obj = parse_date(date_str)
                    if date_obj:
                        events.append((date_obj, amount))

            # 2. Fees (Negative) - Use "Date Paid" or similar if available, else approximate?
```

```

# Many sheets don't have fee dates. We might need to omit fees from the *timeline*
# if we don't have dates, or assume they happened at account start?
# For this request, let's focus on Payouts for growth, or Net Profit if we can find dates.
# Without dates for Fees/Hedges, a true "Net Profit Over Time" is hard.
# Let's try to find dates for Hedges/Farming.

purchase_date_str = ev.get('Date')
purchase_date = parse_date(purchase_date_str)

if purchase_date:
    # Add Fees at purchase date
    fee = parse_currency(ev.get('Fee'))
    act_fee = parse_currency(ev.get('Activation Fee'))
    total_fee = fee + act_fee
    if total_fee > 0:
        events.append((purchase_date, -total_fee))

    # Add Hedge Results? We don't have dates for each hedge result usually...
    # We can assume they happen "after" purchase.
    # For now, let's stick to (Payouts - Fees) which has dates.

# Sort events by date
events.sort(key=lambda x: x[0])

if not events:
    return [], []

dates = []
values = []
cumulative = 0.0

# Aggregate by day
from collections import defaultdict
daily_changes = defaultdict(float)

for date_obj, amount in events:
    date_str = date_obj.strftime("%Y-%m-%d")
    daily_changes[date_str] += amount

sorted_days = sorted(daily_changes.keys())

for day in sorted_days:
    cumulative += daily_changes[day]
    dates.append(day)
    values.append(cumulative)

return dates, values

@cache_result(ttl=300)
def calculate_propfirm_overview(profile_filter=None)
    Aggregates financial data by Prop Firm. Returns a dictionary.

```

Why these Python concepts were used

- **@cache_result** — Memoises return values keyed by the arguments. Trades memory for speed when the function is pure and called many times with the same inputs.

- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns `clients_data = _get_cached_clients()`.
2. Assigns `overview = {}`.
3. Assigns `P1_HEDGE_COLS = ['Hedge Result 1', 'Hedge Result 2', 'Hedge Result 3', 'H....`
4. Assigns `FUNDED_HEDGE_COLS = ['Hedge Result 1.1', 'Hedge Result 2.1', 'Hedge Result 3.....`
5. **for** `(client_id, data) in clients_data.items()`: iterates.
6. **for** `firm in overview`: iterates.
7. **return** `overview`.

Code (copy-paste safe)

```
def calculate_propfirm_overview(profile_filter=None):
    """
    Aggregates financial data by Prop Firm.
    Returns a dictionary.
    """
    clients_data = _get_cached_clients() # Returns {client_id: full_data_dict}

    overview = {}

    # Define columns for calculations
    P1_HEDGE_COLS = ['Hedge Result 1', 'Hedge Result 2', 'Hedge Result 3', 'Hedge Result 4', 'Hedge Result 5', 'Hedge Result 6', 'Hedge Result 7']
    FUNDED_HEDGE_COLS = ['Hedge Result 1.1', 'Hedge Result 2.1', 'Hedge Result 3.1', 'Hedge Result 4.1', 'Hedge Result 5.1', 'Hedge Result 6.1', 'Hedge Result 7.1']

    # Hedge Day 1 to 34

    for client_id, data in clients_data.items():
        if not data:
            continue

        # Filter by Profile if profile_filter is provided
        if profile_filter and profile_filter.upper() != "ALL":
            identity = data.get('identity', {})
            client_profile = get_client_profile(client_id, identity)
            if client_profile != profile_filter.upper():
                continue

        evaluations = data.get('evaluations', [])
        if not evaluations:
            continue

        for eval_data in evaluations:
```

```

if not isinstance(eval_data, dict):
    continue
raw_prop_firm = eval_data.get('Prop Firm')

# Skip if no prop firm name or if it's header/invalid
if not raw_prop_firm or raw_prop_firm == "-" or str(raw_prop_firm).lower() == "prop firm":
    continue

# Normalize Name
prop_firm = normalize_prop_firm_name(raw_prop_firm)
if not prop_firm:
    continue

if prop_firm not in overview:
    overview[prop_firm] = {
        "total_fees": 0.0,
        "total_activation_fees": 0.0,
        "total_payouts": 0.0,
        "net": 0.0,
        "account_count": 0,
        "hedge_results": 0.0,
        "farming_results": 0.0,
        "active_accounts": 0,
        "passed_accounts": 0,
        "failed_accounts": 0,
        "ended_count": 0,
        "earliest_date": None,
        "clients": set()
    }

# === Financials ===
# Fee is OUTFLOW, so we treat it as positive cost.
# When calculating Net Profit, we subtract it.
fee = parse_currency(eval_data.get('Fee'))
activation_fee = parse_currency(eval_data.get('Activation Fee'))

# Payouts (INFLOW)
payouts = 0.0
for i in range(1, 10):
    key = f'Payout {i}'
    if key in eval_data:
        payouts += parse_currency(eval_data.get(key))

# Hedge Results (PROFIT/LOSS)
p1_hedges = sum(parse_currency(eval_data.get(col)) for col in P1_HEDGE_COLS)
funded_hedges = sum(parse_currency(eval_data.get(col)) for col in FUNDED_HEDGE_COLS)

# Farming Results (PROFIT)
farming_results = 0.0
for i in range(1, 51):
    key = f'Hedge Day {i}'
    farming_results += parse_currency(eval_data.get(key))

# === Status ===
status_p1 = str(eval_data.get('Status P1', '')).strip()
status_funded = str(eval_data.get('Status', '')).strip()
status_p1_lower = status_p1.lower()
status_funded_lower = status_funded.lower()

# Only count hedge/farming for rows with a populated status

```

```

hedge_results = 0.0
if status_p1:
    hedge_results += p1_hedges
if status_funded:
    hedge_results += funded_hedges
if not status_funded:
    farming_results = 0.0

# Logic from data_processor.py
is_p1_fail = status_p1 == 'Fail'
is_funded_fail = status_funded == 'Fail'
is_funded_completed = status_funded == 'Completed'
is_funded_ended = is_funded_fail or is_funded_completed
is_in_progress = not is_p1_fail and not is_funded_ended

is_passed_p1 = status_p1 == 'Pass' or status_p1_lower == 'pass'

# Update counts
if is_in_progress:
    overview[prop_firm]["active_accounts"] += 1

if is_passed_p1:
    overview[prop_firm]["passed_accounts"] += 1

if is_p1_fail or is_funded_fail:
    overview[prop_firm]["failed_accounts"] += 1

if is_p1_fail or is_funded_ended:
    overview[prop_firm]["ended_count"] += 1

# Date tracking
d_str = eval_data.get('Date Started') or eval_data.get('Date')
if d_str:
    d_obj = parse_date(d_str)
    if d_obj:
        cur_earliest = overview[prop_firm]["earliest_date"]
        if cur_earliest is None or d_obj < cur_earliest:
            overview[prop_firm]["earliest_date"] = d_obj

# Update totals (accumulate)
overview[prop_firm]["total_fees"] += fee
overview[prop_firm]["total_activation_fees"] += activation_fee
overview[prop_firm]["total_payouts"] += payouts
overview[prop_firm]["hedge_results"] += hedge_results
overview[prop_firm]["farming_results"] += farming_results
overview[prop_firm]["account_count"] += 1
overview[prop_firm]["clients"].add(client_id)

# Finalize calculations
for firm in overview:
    data = overview[firm]
    # Net Profit = Payouts + Hedge Results + Farming Results - (Fees + Activation Fees)
    data["net"] = data["total_payouts"] + data["hedge_results"] + data["farming_results"] - (data[
        "total_fees"] + data["total_activation_fees"])

# EV
ended = data.get("ended_count", 0)
data["expected_value"] = data["net"] / ended if ended > 0 else 0.0

# EV Per Day

```

```

data["ev_per_day"] = 0.0
if data.get("earliest_date"):
    days = (datetime.now() - data["earliest_date"]).days
    if days > 0:
        data["ev_per_day"] = data["net"] / days

# Clean up objects not serializable
if "earliest_date" in data:
    del data["earliest_date"]

# Convert set to count
data["total_clients"] = len(data["clients"])
del data["clients"]

return overview

```

```
def get_cumulative_fees_data(profile_filter=None)
```

Calculates cumulative fees (Fees + Activation) over time.

Why these Python concepts were used

- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.

Step by step (top-level body)

1. Assigns `clients_data = _get_cached_clients()`.
2. Assigns `events = []`.
3. **for** `(client_id, data) in clients_data.items()`: iterates.
4. **return** `_aggregate_events_cumulative(events)`.

Code (copy-paste safe)

```

def get_cumulative_fees_data(profile_filter=None):
    """Calculates cumulative fees (Fees + Activation) over time."""
    clients_data = _get_cached_clients()
    events = [] # (datetime, amount)

    for client_id, data in clients_data.items():
        if not data: continue

        # Profile Filter
        if profile_filter and profile_filter.upper() != "ALL":
            identity = data.get('identity', {})
            client_profile = get_client_profile(client_id, identity)
            if client_profile != profile_filter.upper(): continue

        evaluations = data.get('evaluations', [])
        for ev in evaluations:
            if not isinstance(ev, dict):
                continue
            # 1. Fees (Negative, but usually shown as positive 'Spent' on card.
            # Graph should likely show cumulative SPEND (positive slope) or cumulative CASHFLOW (negative slope)
            # The card says "Total Fees Spent: $1.1M". Correct graph would probably be strictly increasing
            # But "Net Profit" graph subtracts it.
            # Let's show it as Positive Cumulative Cost for the "Total Fees Spent" graph.

```



```

    fee = parse_currency(ev.get('Fee'))
    act_fee = parse_currency(ev.get('Activation Fee'))
    total_fee = fee + act_fee

    if total_fee > 0:
        date_purchased = parse_date(ev.get('Date Purchased') or ev.get('Date'))
        date_started = parse_date(ev.get('Date Started'))
        base_date = date_purchased or date_started or datetime.now()
        events.append((base_date, total_fee)) # Positive value = Total Spent

return _aggregate_events_cumulative(events)

def get_cumulative_hedge_data(profile_filter=None)

```

Calculates cumulative hedge results over time.

Why these Python concepts were used

- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.

Step by step (top-level body)

1. Assigns `clients_data = _get_cached_clients()`.
2. Assigns `events = []`.
3. Assigns `P1_HEDGE_COLS = ['Hedge Result 1', 'Hedge Result 2', 'Hedge Result 3', 'H....`
4. Assigns `FUNDED_HEDGE_COLS = ['Hedge Result 1.1', 'Hedge Result 2.1', 'Hedge Result 3....`
5. `for (client_id, data) in clients_data.items():` iterates.
6. `return _aggregate_events_cumulative(events)`.

Code (copy-paste safe)

```

def get_cumulative_hedge_data(profile_filter=None):
    """Calculates cumulative hedge results over time."""
    clients_data = _get_cached_clients()
    events = []

    P1_HEDGE_COLS = ['Hedge Result 1', 'Hedge Result 2', 'Hedge Result 3', 'Hedge Result 4', 'Hedge Result 5']
    FUNDED_HEDGE_COLS = ['Hedge Result 1.1', 'Hedge Result 2.1', 'Hedge Result 3.1', 'Hedge Result 4.1', 'Hedge Result 5.1', 'Hedge Result 6', 'Hedge Result 7']

    for client_id, data in clients_data.items():
        if not data: continue
        if profile_filter and profile_filter.upper() != "ALL":
            identity = data.get('identity', {})
            cp = get_client_profile(client_id, identity)
            if cp != profile_filter.upper(): continue

        evaluations = data.get('evaluations', [])
        for ev in evaluations:
            if not isinstance(ev, dict):
                continue

```

```

# Dates
date_started = parse_date(ev.get('Date Started'))
date_ended = parse_date(ev.get('Date Ended'))
date_started_funded = parse_date(ev.get('Date Started.1'))
date_ended_funded = parse_date(ev.get('Date Ended.1'))
base_date = date_started or datetime.now()

# P1 Hedges
p1_profit = sum(parse_currency(ev.get(c)) for c in P1_HEDGE_COLS)
if p1_profit != 0:
    events.append((date_ended or date_started or base_date, p1_profit))

# Funded Hedges
fd_profit = sum(parse_currency(ev.get(c)) for c in FUNDED_HEDGE_COLS)
if fd_profit != 0:
    events.append((date_ended_funded or date_started_funded or base_date, fd_profit))

return _aggregate_events_cumulative(events)

```

```
def get_cumulative_farming_data(profile_filter=None)
```

Calculates cumulative farming results over time.

Why these Python concepts were used

- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.

Step by step (top-level body)

1. Assigns `clients_data = _get_cached_clients()`.
2. Assigns `events = []`.
3. **for** `(client_id, data) in clients_data.items()`: iterates.
4. **return** `_aggregate_events_cumulative(events)`.

Code (copy-paste safe)

```

def get_cumulative_farming_data(profile_filter=None):
    """Calculates cumulative farming results over time."""
    clients_data = _get_cached_clients()
    events = []

    for client_id, data in clients_data.items():
        if not data: continue
        if profile_filter and profile_filter.upper() != "ALL":
            identity = data.get('identity', {})
            cp = get_client_profile(client_id, identity)
            if cp != profile_filter.upper(): continue

    evaluations = data.get('evaluations', [])

```

```

for ev in evaluations:
    if not isinstance(ev, dict):
        continue
    # Dates
    date_started = parse_date(ev.get('Date Started'))
    date_ended = parse_date(ev.get('Date Ended'))
    date_ended_funded = parse_date(ev.get('Date Ended.1'))
    base_date = date_started or datetime.now()

    # Farming Results
    farming_calc = sum(parse_currency(ev.get(f'Hedge Day {i}')) for i in range(1, 51))
    if farming_calc != 0:
        events.append((date_ended_funded or date_ended or base_date, farming_calc))

return _aggregate_events_cumulative(events)

```

```
def _aggregate_events_cumulative(events)
```

Why these Python concepts were used

- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.

Step by step (top-level body)

1. **if** not events: branches conditionally.
2. Calls events.sort(...) for its side effect.
3. Lazy import from collections.
4. Assigns daily_changes = defaultdict(float).
5. **for** (dt, val) in events: iterates.
6. Assigns dates = [].
7. Assigns values = [].
8. Assigns cumulative = 0.0.
9. Assigns sorted_days = sorted(daily_changes.keys()).
10. **for** day in sorted_days: iterates.
11. **return** (dates, values).

Code (copy-paste safe)

```

def _aggregate_events_cumulative(events):
    if not events:
        return [], []

    events.sort(key=lambda x: x[0])

    from collections import defaultdict
    daily_changes = defaultdict(float)

    for dt, val in events:
        d_str = dt.strftime("%Y-%m-%d")
        daily_changes[d_str] += val

    dates = []

```

```

values = []
cumulative = 0.0
sorted_days = sorted(daily_changes.keys())

for day in sorted_days:
    cumulative += daily_changes[day]
    dates.append(day)
    values.append(cumulative)

return dates, values

```

```
def calculate_trader_stats(profile_filter=None)
```

Calculates aggregated statistics per trader.

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns `clients_data = _get_cached_clients()`.
2. Assigns `traders_stats = {}`.
3. **for** `(client_id, data) in clients_data.items()`: iterates.
4. **return** `list(traders_stats.values())`.

Code (copy-paste safe)

```

def calculate_trader_stats(profile_filter=None):
    """Calculates aggregated statistics per trader."""
    clients_data = _get_cached_clients()
    traders_stats = {}

    for client_id, data in clients_data.items():
        if not data: continue

        identity = data.get('identity', {})

        # Apply Profile Filter
        if profile_filter and profile_filter.upper() != "ALL":
            c_prof = get_client_profile(client_id, identity)
            if c_prof != profile_filter.upper():
                continue

        trader_name = identity.get('trader_name', 'Unassigned')
        trader_name = identity.get('trader')

        if not trader_name or str(trader_name).strip().lower() in ['none', 'null', '', '-']:

```

```

trader_name = "Unassigned"

if trader_name not in traders_stats:
    traders_stats[trader_name] = {
        "name": trader_name,
        "sheets_reviewed": 0,
        "client_count": 0,
        "total_payouts": 0.0,
        "total_negative_hedge": 0.0,
        "negative_hedge_details": [],
        "farming_days_count": 0,
        "farming_missing_notes": 0,
        "farming_warnings": []
    }

stats = traders_stats[trader_name]
stats['client_count'] += 1
stats['sheets_reviewed'] += 1

evaluations = data.get('evaluations', [])
for idx, ev in enumerate(evaluations):
    if not isinstance(ev, dict):
        continue
    row_num = idx + 3 # Matches frontend assumption (Row 3 start)
    acc_num = ev.get('Account #') or ev.get('Account #.1') or 'Unknown'
    # Payouts 1-10
    for i in range(1, 11):
        val = ev.get(f'Payout {i}')
        amt = parse_currency(val) if val else 0.0
        if amt > 0: stats['total_payouts'] += amt

    # Negative Hedge Logic

    # Helper to check if any hedging occurred
    def has_hedging_activity(ev_data, prefix="Hedge Result", count=5):
        for k in range(1, count + 1):
            val = parse_currency(ev_data.get(f"{prefix} {k}"))
            if val != 0: return True
        return False

    # 1. Phase 1 Net (Column N)
    p1_net = parse_currency(ev.get('Hedge Net'))

    # Only count negative hedge net if actual hedging results exist (not just fees)
    if p1_net < -1.0 and has_hedging_activity(ev, "Hedge Result", 5):
        stats['total_negative_hedge'] += p1_net

    date_str = ev.get('Date Ended') or ev.get('Date')
    date_obj = parse_date(date_str)
    date_iso = date_obj.strftime("%Y-%m-%d") if date_obj else ""

    stats['negative_hedge_details'].append({
        "client": client_id,
        "account": acc_num,
        "amount": p1_net,
        "link": f"/dashboard/{client_id}?range=N{row_num}",
        "date": date_iso
    })

    # 2. Funded Net (Column AA)

```

```

fd_net = parse_currency(ev.get('Hedge Net.1'))

# Check Funded Hedge Results (1.1, 2.1, etc)
funded_hedged = False
# Check 1.1 explicitly
if parse_currency(ev.get("Hedge Result 1.1")) != 0: funded_hedged = True
# Check 2.1 - 5.1
if not funded_hedged:
    for k in range(2, 6):
        if parse_currency(ev.get(f"Hedge Result {k}.1")) != 0:
            funded_hedged = True
            break

if fd_net < -1.0 and funded_hedged:
    stats['total_negative_hedge'] += fd_net

date_str = ev.get('Date Ended.1')
date_obj = parse_date(date_str)
date_iso = date_obj.strftime("%Y-%m-%d") if date_obj else ""

stats['negative_hedge_details'].append({
    "client": client_id,
    "account": acc_num,
    "amount": fd_net,
    "link": f"/dashboard/{client_id}?range=AA{row_num}",
    "date": date_iso
})

# Farming Logic
for d in range(1, 60):
    h_val = parse_currency(ev.get(f'Hedge Day {d}'))
    if h_val != 0:
        stats['farming_days_count'] += 1

    p_val_raw = ev.get(f'Day {d} Profit')
    if not p_val_raw or str(p_val_raw).strip() in ['', '-']:
        stats['farming_missing_notes'] += 1

    # Calculate Prop Day Col
    # Prop Day 1 = AK (Index 36)
    # Prop Day 2 = AM (Index 38)
    col_idx = 36 + (d - 1) * 2
    col_let = col_idx_to_letter(col_idx)

    stats['farming_warnings'].append({
        "client": client_id,
        "day": d,
        "link": f"/dashboard/{client_id}?range={col_let}{row_num}"
    })

return list(traders_stats.values())

@cache_result(ttl=300)
def get_client_performance_stats(profile_filter=None, start_date=None, end_date=None)

    Returns a list of per-client performance statistics. Used for the Client Performance Table. Optionally
    filters by start_date and end_date (datetime objects).

```

Why these Python concepts were used

- **@cache_result** — Memoises return values keyed by the arguments. Trades memory for speed when the function is pure and called many times with the same inputs.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **set comprehension** — Builds a set in one expression — usually to deduplicate the result of a transform.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns BEF_HIDDEN_FIRMS = {'lucid', 'apex', 'tradeday', 'toponefutures'}.
2. Assigns is_bef = profile_filter and profile_filter.upper() == 'BEF'.
3. Defines a nested function _is_firm_hidden(...).
4. **try** block with 1 **except** clause.
5. Assigns client_map = {}.
6. **if** SYSTEM_HIERARCHY and 'admins' in SYSTEM_HIERARCHY: branches conditionally.
7. Assigns clients_data = _get_cached_clients().
8. Assigns clients_list = [].
9. **for** (client_id, data) in clients_data.items(): iterates.
10. Assigns seen_clients = {c['client_id'] for c in clients_list}.
11. **try** block with 1 **except** clause.
12. **if** h and 'admins' in h: branches conditionally.
13. **return** clients_list.

Code (copy-paste safe)

```
def get_client_performance_stats(profile_filter=None, start_date=None, end_date=None):
    """
    Returns a list of per-client performance statistics.
    Used for the Client Performance Table.
    Optionally filters by start_date and end_date (datetime objects).
```

```

"""
# BEF hidden firms – evaluations from these firms are excluded for BEF view
BEF_HIDDEN_FIRMS = {'lucid', 'apex', 'tradeday', 'toponefutures'}
is_bef = profile_filter and profile_filter.upper() == 'BEF'

def _is_firm_hidden(firm_name):
    if not is_bef:
        return False
    return (firm_name or '').strip().lower().replace(' ', '') in BEF_HIDDEN_FIRMS
# Import hierarchy to map clients to traders/admins
try:
    from config.hierarchy import SYSTEM_HIERARCHY
except ImportError:
    import sys, os
    sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
    from config.hierarchy import SYSTEM_HIERARCHY

# Build client map
client_map = {}
if SYSTEM_HIERARCHY and 'admins' in SYSTEM_HIERARCHY:
    for admin_name, admin_data in SYSTEM_HIERARCHY['admins'].items():
        for trader_name, trader_data in admin_data.get('traders', {}).items():
            for client in trader_data.get('clients', []):
                c_name = client.get('name')
                if c_name:
                    client_map[c_name] = {
                        'admin': admin_name,
                        'trader': trader_name,
                        'category': (client.get('category') or '').upper()
                    }

clients_data = _get_cached_clients()
clients_list = []

for client_id, data in clients_data.items():
    if not data: continue

    identity = data.get('identity', {})
    real_client_name = identity.get('name') or client_id

    # Profile Filter – check identity first, then hierarchy category as fallback
    source = get_client_profile(client_id, identity)

    if profile_filter and profile_filter.upper() != "ALL":
        if source != profile_filter.upper():
            continue

    # Hierarchy Info
    h_info = client_map.get(real_client_name) or client_map.get(client_id)
    admin_name = h_info['admin'] if h_info else "-"
    trader_name = h_info['trader'] if h_info else "-"

    # Init Stats
    c_stats = {
        "client_id": real_client_name,
        "trader": trader_name,
        "admin": admin_name,
        "source": source,
        "payouts": 0.0,
        "deposits": 0.0,

```



```

    "fees": 0.0,
    "net_profit": 0.0,
    "active": 0,
    "passed": 0,
    "failed": 0,
    "hedge_profit": 0.0,
    "farming_profit": 0.0,
    "ended": 0,
    "total_duration_days": 0
}

# 1. Evaluations Payouts/Fees/Status
evaluations = data.get('evaluations', [])
for ev in evaluations:
    if not isinstance(ev, dict):
        continue
    # Date filtering
    if start_date or end_date:
        eval_date = parse_date(ev.get('Date Started')) or parse_date(ev.get(
            'Date Purchased')) or parse_date(ev.get('Date'))
        if eval_date:
            if start_date and eval_date.date() < start_date.date():
                continue
            if end_date and eval_date.date() > end_date.date():
                continue
    # Skip hidden firms for BEF view
    if _is_firm_hidden(ev.get('Prop Firm')):
        continue
    # Status logic expanded
    status_p1_raw = str(ev.get('Status P1') or '').strip()
    status_funded_raw = str(ev.get('Status') or '').strip()
    status = status_funded_raw.lower()
    if 'passed' in status or 'funded' in status:
        c_stats['passed'] += 1
    elif 'failed' in status or 'breached' in status or 'blown' in status or 'fail' in status:
        c_stats['failed'] += 1
    elif 'active' in status or 'phase' in status or 'running' in status or 'ongoing' in status or 'ended' in status:
        c_stats['active'] += 1

    # Ended count & duration for EV calculation
    is_p1_fail = status_p1_raw == 'Fail'
    is_funded_fail = status_funded_raw == 'Fail'
    is_funded_completed = status_funded_raw == 'Completed'
    if is_p1_fail or is_funded_fail or is_funded_completed:
        c_stats['ended'] += 1
        s_d = parse_date(ev.get('Date Started'))
        e_d = parse_date(ev.get('Date Ended.1')) if (
            is_funded_fail or is_funded_completed) else ev.get('Date Ended')
        if s_d and e_d:
            c_stats['total_duration_days'] += max(0, (e_d - s_d).days)

    # Fees – collected here as fallback; overridden below by cashflow_inprogress if available
    fee = parse_currency(ev.get('Fee'))
    act_fee = parse_currency(ev.get('Activation Fee'))
    c_stats['fees'] += (fee + act_fee)

    # Payouts
    for i in range(1, 10):
        c_stats['payouts'] += parse_currency(ev.get(f'Payout {i}'))

# Use stored cashflow_inprogress for hedge/farming/fees (matches Net Profit In Progress display)

```

```

# For BEF view, skip stored totals (they include all firms) – use per-eval sums instead
# When date filtering is active, skip stored totals as they represent all-time data
stored_cf = data.get('statistics', {}).get('cashflow_inprogress', {})
if not is_bef and not start_date and not end_date and stored_cf and (stored_cf.get('hedging_results', 0) != 0 or stored_cf.get('farming_results', 0) != 0):
    sheet_hedge = stored_cf.get('hedging_results', 0.0) + stored_cf.get('farming_results', 0.0)
# Compute live actual hedging from MT5 (same formula as client dashboard)
acct = data.get('account', {})
stats_data = data.get('statistics', {})
hr = stats_data.get('hedging_review', {})
try:
    mt5_dep = float(acct.get('total_deposits') or 0)
except (ValueError, TypeError):
    mt5_dep = 0.0
try:
    mt5_with = float(acct.get('total_withdrawals') or 0)
except (ValueError, TypeError):
    mt5_with = 0.0
try:
    mt5_bal = float(acct.get('balance') or 0)
except (ValueError, TypeError):
    mt5_bal = 0.0
hist_dep_h = 0.0; hist_with_h = 0.0; hist_bal_h = 0.0
prior_activity = 0.0
try:
    prior_activity = float(hr.get('current_mt5_prior_activity') or 0)
except (ValueError, TypeError):
    pass
for ha in (hr.get('historical_accounts') or []):
    try: hist_dep_h += float(ha.get('deposits', 0))
    except (ValueError, TypeError): pass
    try: hist_with_h += float(ha.get('withdrawals', 0))
    except (ValueError, TypeError): pass
    try: hist_bal_h += float(ha.get('final_balance', 0))
    except (ValueError, TypeError): pass
    try: prior_activity += float(ha.get('prior_activity_profit', 0))
    except (ValueError, TypeError): pass
combined_dep = mt5_dep + hist_dep_h
combined_with = mt5_with + hist_with_h
combined_bal = mt5_bal + hist_bal_h
# Only apply discrepancy if there's MT5 data
if mt5_dep != 0 or mt5_bal != 0:
    live_actual_hedging = combined_bal - (combined_dep + combined_with) - prior_activity
    c_stats['hedge_profit'] = live_actual_hedging
else:
    c_stats['hedge_profit'] = sheet_hedge
c_stats['farming_profit'] = 0.0 # farming already included in hedge_profit
c_stats['fees'] = stored_cf.get('challenge_fees', 0.0) + stored_cf.get('activation_fee', 0.0)
# Use payouts from cashflow_inprogress so it matches Net Profit In Progress
if stored_cf.get('payouts') is not None:
    c_stats['payouts'] = stored_cf.get('payouts', 0.0)
else:
    # Fallback: recalculate from evaluation columns
    for ev in evaluations:
        if not isinstance(ev, dict):
            continue
        if _is_firm_hidden(ev.get('Prop Firm')):
            continue
        status_p1 = str(ev.get('Status P1') or '').strip()
        status_funded = str(ev.get('Status') or '').strip()

```

```

        if status_p1:
            for col in ['Hedge Result 1', 'Hedge Result 2', 'Hedge Result 3', 'Hedge Result 4',
                        'Hedge Result 5']:
                c_stats['hedge_profit'] += parse_currency(ev.get(col))
        if status_funded:
            for col in ['Hedge Result 1.1', 'Hedge Result 2.1', 'Hedge Result 3.1',
                        'Hedge Result 4.1',
                        'Hedge Result 5.1', 'Hedge Result 6', 'Hedge Result 7']:
                c_stats['hedge_profit'] += parse_currency(ev.get(col))
            c_stats['farming_profit'] += sum(parse_currency(ev.get(f'Hedge Day {i}')) for i in range(
                1, 51))

# 2. Deposits – use MT5 account deposits (current + historical), matching MT5 Accounts Overview
acct = data.get('account', {})
stats_data = data.get('statistics', {})
hr = stats_data.get('hedging_review', {})
# Current MT5 deposits: prefer account field, fall back to hedging_review
try:
    current_dep = float(acct.get('total_deposits') or hr.get('total_deposits') or 0)
except (ValueError, TypeError):
    current_dep = 0.0
# Historical MT5 accounts deposits
hist_dep = 0.0
for hist_acc in (hr.get('historical_accounts') or []):
    try:
        hist_dep += float(hist_acc.get('deposits', 0))
    except (ValueError, TypeError):
        pass
c_stats['deposits'] = abs(current_dep) + abs(hist_dep)

# Net Profit – use stored value from cashflow_inprogress (matches Net Profit In Progress display)
# For BEF view, always recalculate from filtered eval sums
# When date filtering is active, always recalculate
if not is_bef and not start_date and not end_date and stored_cf and stored_cf.get(
    'net_profit') is not None:
    c_stats['net_profit'] = stored_cf.get('net_profit', 0.0)
else:
    c_stats['net_profit'] = c_stats['payouts'] + c_stats['hedge_profit'] + c_stats[
        'farming_profit'] - c_stats['fees']

clients_list.append(c_stats)

# Include hierarchy clients that have no DB records yet
seen_clients = {c['client_id'] for c in clients_list}

try:
    from config.hierarchy import SYSTEM_HIERARCHY
    h = SYSTEM_HIERARCHY
except ImportError:
    h = {}

if h and 'admins' in h:
    for admin_name, admin_data in h['admins'].items():
        for trader_name, trader_data in admin_data.get('traders', {}).items():
            for client in trader_data.get('clients', []):
                c_name = client.get('name')
                if not c_name or c_name in seen_clients:
                    continue
                c_cat = (client.get('category') or '').upper()
                if profile_filter and profile_filter.upper() != "ALL":

```

```

        if (c_cat or 'PRIVATE') != profile_filter.upper():
            continue
        clients_list.append({
            "client_id": c_name,
            "trader": trader_name,
            "admin": admin_name,
            "source": c_cat or 'PRIVATE',
            "payouts": 0.0, "deposits": 0.0, "fees": 0.0,
            "net_profit": 0.0, "active": 0, "passed": 0, "failed": 0,
            "hedge_profit": 0.0, "farming_profit": 0.0
        })

    return clients_list

```

Step 11.11 — Build dashboard/scheduler.py

What to do. Create the file dashboard/scheduler.py in your project root and paste in the code shown in this step. The file is **605** lines long; we walk through it piece by piece below.

Depends on. This file imports from internal modules built in earlier steps: dashboard. If any of those imports fail, jump back to the step that introduced them.

What this file is for. APScheduler setup. Cron-style jobs that resync from Google Sheets, recompute statistics, and send notifications.

dashboard/scheduler.py

605 loc · 0 classes · 12 functions · 10 imports

APScheduler setup. Cron-style jobs that resync from Google Sheets, recompute statistics, and send notifications. It defines **12** module-level functions, across **605** lines.

Python concepts demonstrated in this module

- Comprehensions
- Context managers (with-statements)
- Exceptions
- f-strings
- Module-level state
- Lambdas

Imports — what each one is for

```
import threading
```

Why imported. Threads and synchronisation primitives. Used when work is I/O-bound and the GIL is not a bottleneck (the GIL prevents speed-up on CPU-bound work).

Used in this file. threading.Event, threading.Lock, threading.Thread

Example. Thread(target=worker, args=(q,), daemon=True).start()

Other common scenarios.

- Lock / RLock to guard shared state

- Event for one-shot signals between threads
- Queue (from queue) for producer/consumer patterns
- threading.local() for thread-local storage

`import time`

Why imported. Timestamps and sleep. Lower-level than datetime — works in Unix-epoch seconds.

Used in this file. `time.sleep`

Example. `time.sleep(1.5)` # pause this thread for 1.5 seconds

Other common scenarios.

- `time.time()` returns seconds since the epoch
- `time.monotonic()` for measuring elapsed time (never goes backward)
- `time.perf_counter()` for high-precision benchmarking
- `time.strftime` / `time.strptime` mirror the datetime versions

`import json`

Why imported. JSON encoding and decoding — the standard wire format for config files, API payloads, and inter-process messages.

Used in this file. `json.JSONDecodeError`, `json.dump`, `json.dumps`, `json.load`, `json.loads`

Example. `json.loads(response.text)` # parse a JSON string to dict

Other common scenarios.

- `json.dumps(obj, indent=2)` for pretty-printing
- `json.dump(obj, file)` / `json.load(file)` for file I/O
- `default=str` handles non-serialisable types like datetime
- `object_hook=` customises decoding (e.g., to dataclass instances)

`import os`

Why imported. Operating-system interface. Path manipulation, environment variables, working-directory operations, exit codes, and thin wrappers around POSIX/Windows syscalls.

Used in this file. `os.environ`, `os.environ.get`, `os.path`, `os.path.abspath`, `os.path.dirname`, `os.path.isfile`, `os.path.join`

Example. `os.path.join(root, 'subdir', 'file.txt')` # joins with the OS-correct separator

Other common scenarios.

- `os.environ.get('API_KEY')` to read environment variables
- `os.makedirs(path, exist_ok=True)` to create nested directories
- `os.listdir(path)` to list a directory's contents
- `os.remove(path)` / `os.rename(src, dst)` to delete or move a file
- `os.getcwd()` / `os.chdir(path)` to inspect or change the working dir

```
from datetime import datetime
```

Why imported. Dates, times, durations. The fundamental temporal types: date, time, datetime, timedelta, timezone.

Used in this file. datetime

Example. datetime.now() - timedelta(days=7) # one week ago

Other common scenarios.

- datetime.strptime(s, '%Y-%m-%d') parses a string
- datetime.strftime(fmt) formats one
- datetime.fromtimestamp(n) converts a Unix epoch
- datetime.utcnow() for UTC (better: datetime.now(timezone.utc))

```
from urllib.request import Request
from urllib.request import urlopen
```

Why imported. Stdlib URL utilities — parsing, encoding, and a basic HTTP client. Mostly useful for the parsing helpers; for outbound HTTP, requests is more pleasant.

Used in this file. Request, urlopen

Example. urllib.parse.urlencode({'q': 'hello world'})

Other common scenarios.

- urllib.parse.urlparse(url) splits scheme/host/path/query
- urllib.parse.quote(s) percent-encodes a path segment
- urllib.request.urlopen(url) is the basic HTTP client

```
from urllib.error import URLError
```

Why imported. Stdlib URL utilities — parsing, encoding, and a basic HTTP client. Mostly useful for the parsing helpers; for outbound HTTP, requests is more pleasant.

Used in this file. URLError

Example. urllib.parse.urlencode({'q': 'hello world'})

Other common scenarios.

- urllib.parse.urlparse(url) splits scheme/host/path/query
- urllib.parse.quote(s) percent-encodes a path segment
- urllib.request.urlopen(url) is the basic HTTP client

```
import logging
```

Why imported. Structured, levelled logging. Replaces print() with filterable, formattable output that can route to files, stderr, syslog, or HTTP endpoints.

Used in this file. logging.error, logging.info, logging.warning

Example. logging.getLogger(__name__).info('connected to %s', host)

Other common scenarios.

- `.debug()` / `.info()` / `.warning()` / `.error()` / `.exception()`
- `logger.exception()` captures the current traceback automatically
- `logging.basicConfig(level=logging.INFO)` for quick setup
- Handlers and Formatters route logs to files / streams / syslog

```
from dashboard.database import get_all_clients
```

Why imported. Internal package — the Flask dashboard. See chapter on dashboard/.

Used in this file. `get_all_clients`

```
from dashboard.watermark_service import save_daily_profit
```

Why imported. Internal package — the Flask dashboard. See chapter on dashboard/.

Used in this file. `save_daily_profit`

Module constants

Each line below is followed by a plain-English note explaining what it does and why it is written that way.

```
_SCHEDULER_STATE_FILE = os.path.join(os.path.dirname(os.path.abspath(__file__)), '.scheduler_state.json')
```

Filesystem path resolved relative to the source file at import time — portable across machines.

Functions

```
def _load_ran_today()
```

Load the ran-today state from disk (survives module reloads).

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def _load_ran_today():
    """Load the ran-today state from disk (survives module reloads)."""
    try:
        with open(_SCHEDULER_STATE_FILE, 'r') as f:
            return json.load(f)
    except (FileNotFoundError, json.JSONDecodeError, OSError):
        return {}
```

```
def _mark_ran(job_name, date_str)
```

Mark a job as completed for a given date and persist to disk.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `state = _load_ran_today()`.
2. Assigns `state[job_name] = date_str`.
3. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def _mark_ran(job_name, date_str):
    """Mark a job as completed for a given date and persist to disk."""
    state = _load_ran_today()
    state[job_name] = date_str
    try:
        with open(_SCHEDULER_STATE_FILE, 'w') as f:
            json.dump(state, f)
    except OSError as e:
        logging.warning(f"Could not persist scheduler state: {e}")

def run_scheduler()
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Calls `logging.info(...)` for its side effect.
2. **while** `not stop_event.is_set()`: loops until the condition is false.

Code (copy-paste safe)

```
def run_scheduler():
    logging.info("Starting Daily Scheduler (watermark + quality scan + Slack)...")
    while not stop_event.is_set():
        try:
            now = datetime.utcnow()
            today = now.strftime('%Y-%m-%d')
            ran = _load_ran_today()

            # All times in UTC (datetime.utcnow)
```



```

# PythonAnywhere runs UTC natively.
# Local dev machines may be in any timezone – utcnow() ensures consistency.

# 21:00 UTC (00:00 EAT) – Midnight watermark snapshot
if now.hour == 21 and now.minute == 0 and ran.get('watermark') != today:
    logging.info("Running midnight watermark update (00:00 EAT)...")
    update_all_clients_watermarks()
    _mark_ran('watermark', today)
    time.sleep(60)

# 23:27 UTC (02:27 EAT) – Automated quality scan [TEMP TEST]
if now.hour == 23 and now.minute == 27 and ran.get('quality_scan') != today:
    logging.info("Running scheduled quality scan (TEST 02:27 EAT)...")
    run_scheduled_quality_scan()
    _mark_ran('quality_scan', today)
    time.sleep(60)

# 23:30 UTC (02:30 EAT) – Post daily summary to Slack [TEMP TEST]
if now.hour == 23 and now.minute == 30 and ran.get('slack_summary') != today:
    logging.info("Posting daily quality summary to Slack (TEST 02:30 EAT)...")
    post_slack_summary()
    _mark_ran('slack_summary', today)
    time.sleep(60)

# 23:15 UTC (02:15 EAT) – Daily database cleanup
if now.hour == 23 and now.minute == 15 and ran.get('db_cleanup') != today:
    logging.info("Running daily database cleanup (02:15 EAT)...")
    run_database_cleanup()
    _mark_ran('db_cleanup', today)
    time.sleep(60)

time.sleep(30) # Check every 30s

except Exception as e:
    logging.error(f"Scheduler error: {e}")
    time.sleep(60)

def run_scheduled_quality_scan()

```

Run the quality scan and save results — same as the API but without Flask context.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def run_scheduled_quality_scan():
    """Run the quality scan and save results – same as the API but without Flask context."""
    try:
        from dashboard.app import run_quality_scan
        from dashboard.database import save_quality_scan_results, log_action

        results = run_quality_scan()
        scan_date = datetime.now().strftime('%Y-%m-%d')
        save_quality_scan_results(scan_date, results)

        total_issues = sum(r['total_issues'] for r in results)
        log_action('QUALITY_SCAN', 'system', 'scheduler',
                  '127.0.0.1', f"Scheduled scan: {len(results)} clients, {total_issues} issues")
        logging.info(f"Quality scan complete: {len(results)} clients, {total_issues} issues")
    except Exception as e:
        logging.error(f"Scheduled quality scan failed: {e}")

def _get_slack_webhook_url()
```

Read Slack webhook URL from database, then environment, then .env file.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.

Step by step (top-level body)

1. **try** block with 1 **except** clause.
2. Assigns `url = os.environ.get('SLACK_WEBHOOK_URL', '').strip()`.
3. **if** `url`: branches conditionally.
4. Assigns `env_path = os.path.join(os.path.dirname(os.path.dirname(os.path.abspath(...`
5. **if** `os.path.isfile(env_path)`: branches conditionally.
6. **return** `"`.

Code (copy-paste safe)

```
def _get_slack_webhook_url():
    """Read Slack webhook URL from database, then environment, then .env file."""
    # 1. Check database (set via super admin UI)
    try:
        from dashboard.database import get_setting
        db_url = get_setting('slack_webhook_url')
        if db_url:
            return db_url
    except Exception:
        pass
    # 2. Check environment variable
    url = os.environ.get('SLACK_WEBHOOK_URL', '').strip()
    if url:
        return url
    # 3. Fallback: read from .env file in project root
```

```

env_path = os.path.join(os.path.dirname(os.path.dirname(os.path.abspath(__file__))), '.env')
if os.path.isfile(env_path):
    with open(env_path, 'r') as f:
        for line in f:
            line = line.strip()
            if line.startswith('SLACK_WEBHOOK_URL=') and not line.startswith('#'):
                return line.split('=', 1)[1].strip().strip('').strip('')
return ''

```

```
def send_slack_message(text)
```

Post a message to the configured Slack webhook.

Step by step (top-level body)

1. Assigns `webhook_url = _get_slack_webhook_url()`.
2. **if** not `webhook_url`: branches conditionally.
3. **return** `send_slack_to_webhook(webhook_url, text)`.

Code (copy-paste safe)

```

def send_slack_message(text):
    """Post a message to the configured Slack webhook."""
    webhook_url = _get_slack_webhook_url()
    if not webhook_url:
        logging.warning("Slack webhook URL not configured – skipping Slack post.")
        return False
    return send_slack_to_webhook(webhook_url, text)

```

```
def send_slack_to_webhook(webhook_url, text)
```

Post a message to a specific Slack webhook URL.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 2 **except** clauses.

Code (copy-paste safe)

```

def send_slack_to_webhook(webhook_url, text):
    """Post a message to a specific Slack webhook URL."""
    try:
        payload = json.dumps({'text': text}).encode('utf-8')
        req = Request(webhook_url, data=payload, headers={'Content-Type': 'application/json'})
        resp = urlopen(req, timeout=15)
        if resp.status == 200:
            logging.info("Slack message posted successfully.")
            return True
        else:

```

```

        logging.warning(f"Slack returned status {resp.status}")
        return False
    except URLError as e:
        logging.error(f"Slack post failed: {e}")
        return False
    except Exception as e:
        logging.error(f"Slack post error: {e}")
        return False

```

```
def _build_daily_summary_text()
```

Build the daily quality summary text (same logic as the API endpoint).

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **dict comprehension** — Builds a dict in one expression (`{k: v for k, v in items}`) — same intent as a list comprehension but for mappings.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.
- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Lazy import from dashboard.database.
2. Lazy import from config.hierarchy.
3. Lazy import from config.hierarchy.
4. Assigns `now = datetime.now()`.
5. Assigns `date = now.strftime('%Y-%m-%d')`.
6. Assigns `weekday = now.strftime('%A')`.
7. Assigns `scan_results = get_quality_scan_results(date)`.
8. Assigns `checklists = get_daily_checklists(date)`.
9. **try** block with 1 **except** clause.

10. Assigns `all_clients = hierarchy_get_all_clients()`.
11. Assigns `filtered_clients = []`.
12. `for client_name in all_clients:` iterates.
13. Assigns `total_clients = len(filtered_clients)`.
14. `if excluded_traders or excluded_clients:` branches conditionally.
15. ... and 23 more statement(s) in the body.

Code (copy-paste safe)

```
def _build_daily_summary_text():
    """Build the daily quality summary text (same logic as the API endpoint)."""
    from dashboard.database import get_quality_scan_results, get_daily_checklists
    from config.hierarchy import get_all_clients as hierarchy_get_all_clients, get_client_profile
    from config.hierarchy import SYSTEM_HIERARCHY

    # UTC date – server runs UTC so at 23:05 UTC (2:05 AM Kenyan) the date
    # is still the day we want. It flips at midnight UTC = 3 AM Kenyan.
    now = datetime.now()
    date = now.strftime('%Y-%m-%d')
    weekday = now.strftime('%A')

    scan_results = get_quality_scan_results(date)
    checklists = get_daily_checklists(date)
    # Apply the Daily Summary Tracker exclusions to the Slack report as well.
    # This keeps the bot report consistent with the tracker UI exclusions.
    try:
        from dashboard.database import get_setting
        excluded_traders = set(json.loads(get_setting('summary_tracker_excluded_traders') or '[]'))
        excluded_clients = set(json.loads(get_setting('summary_tracker_excluded_clients') or '[]'))
    except Exception:
        excluded_traders = set()
        excluded_clients = set()

    # Filter portfolio client list using the same exclusions
    all_clients = hierarchy_get_all_clients()
    filtered_clients = []
    for client_name in all_clients:
        if client_name in excluded_clients:
            continue
        prof = get_client_profile(client_name) or {}
        trader = (prof.get('trader') or '') or 'Unassigned'
        if trader in excluded_traders:
            continue
        filtered_clients.append(client_name)

    total_clients = len(filtered_clients)

    # Filter scan results to excluded traders/clients so leaderboard + top issues match the tracker.
    if excluded_traders or excluded_clients:
        scan_results = [
            r for r in (scan_results or [])
            if (r.get('client_id') not in excluded_clients)
            and ((r.get('trader') or 'Unassigned') not in excluded_traders)
        ]

    # Filter out infrastructure scan errors and recalculate scores
    severity_weight = {'critical': 20, 'high': 10, 'medium': 5, 'low': 2, 'warning': 3, 'info': 0}
```

```

for r in scan_results:
    r['issues'] = [i for i in r.get('issues', []) if i.get('check') != 'Scan error']
    r['total_issues'] = len(r['issues'])
    deduction = sum(severity_weight.get(i.get('severity', 'low'), 2) for i in r['issues'])
    r['health_score'] = max(0.0, round(100.0 - deduction, 1))

clients_healthy = sum(1 for r in scan_results if r['health_score'] >= 90)
clients_warning = sum(1 for r in scan_results if 70 <= r['health_score'] < 90)
clients_critical = sum(1 for r in scan_results if r['health_score'] < 70)
total_issues = sum(r['total_issues'] for r in scan_results)
avg_health = round(sum(r['health_score'] for r in scan_results) / len(scan_results),
    1) if scan_results else 0

# Top issues by frequency
issue_counts = {}
for r in scan_results:
    for iss in r['issues']:
        issue_counts[iss['check']] = issue_counts.get(iss['check'], 0) + 1
top_issues = sorted(issue_counts.items(), key=lambda x: -x[1])[0:5]

# Trader breakdown
trader_stats = {}
for r in scan_results:
    t = r.get('trader', 'Unknown')
    if t not in trader_stats:
        trader_stats[t] = {'clients': 0, 'issues': 0, 'health_sum': 0}
    trader_stats[t]['clients'] += 1
    trader_stats[t]['issues'] += r['total_issues']
    trader_stats[t]['health_sum'] += r['health_score']

lines = [f"■ *Daily Quality Summary – {weekday}, {date}*", ""]
lines.append(f"■ *Portfolio:* {total_clients} total clients")
if scan_results:
    lines.append(
        f"■ Healthy (90%+): {clients_healthy} | ■ Warning: {clients_warning} | ■ Critical: {clients_critical}"
    )
    lines.append(f"■ Avg Health Score: {avg_health}% | Total Issues: {total_issues}")
else:
    lines.append("■ No quality scan results for today.")
lines.append("")

if top_issues:
    lines.append("■ *Top Issues:*")
    for check, count in top_issues:
        lines.append(f"    • {check}: {count} occurrences")
    lines.append("")

if trader_stats:
    # Gamified Trader Health Leaderboard – ranked best to worst
    ranked = sorted(trader_stats.items(), key=lambda x: x[1]['health_sum'] / max(x[1]['clients'], 1),
        reverse=True)
    lines.append("■ *TRADER HEALTH LEADERBOARD*")
    lines.append(
        "    _Ranked by average client health score (highest first). Health score is based on: data fresh_"
    )
    lines.append("")
    total_traders = len(ranked)
    for rank, (t, s) in enumerate(ranked, 1):
        avg = round(s['health_sum'] / s['clients'], 1)
        if rank == 1:
            medal = '■'
        elif rank == 2:

```

```

        medal = '■'
    elif rank == 3:
        medal = '■'
    else:
        medal = f'#{rank}'
    if avg >= 95:
        title = '■ Legendary'
    elif avg >= 90:
        title = '■ Elite'
    elif avg >= 80:
        title = '■ Solid'
    elif avg >= 70:
        title = '■ Warming Up'
    elif avg >= 50:
        title = '■ Needs Work'
    else:
        title = '■ SOS'
    bar_filled = round(avg / 10)
    bar_empty = 10 - bar_filled
    bar = '■' * bar_filled + '■' * bar_empty
    lines.append(f"{medal} *{t}* - {title}")
    lines.append(f"    {bar} *{avg}%* · {s['clients']} clients · {s['issues']} issues")
if total_traders > 0:
    best_name = ranked[0][0]
    worst_name = ranked[-1][0]
    best_avg = round(ranked[0][1]['health_sum'] / max(ranked[0][1]['clients'], 1), 1)
    worst_avg = round(ranked[-1][1]['health_sum'] / max(ranked[-1][1]['clients'], 1), 1)
    lines.append("")
    if best_avg >= 90:
        lines.append(f"■ *{best_name}* is on fire! Leading the pack at {best_avg}%")
    if worst_avg < 50 and total_traders > 1:
        lines.append(f"■ *{worst_name}* - time to level up! Let's get those numbers moving ■")
    lines.append("")

lines.append(f"■ Checklists submitted today: *{len(checklists)}*")
lines.append("")

# ■■ Daily Summary Submission Tracker ■■
try:
    from dashboard.database import get_summary_status_for_date, get_setting, get_client_data
    from config.hierarchy import get_client_profile as _gcp
    from datetime import timezone, timedelta as _tz_td
    import json as _json_mod

    _kenyan_tz = timezone(_tz_td(hours=3))
    submissions = get_summary_status_for_date(date)
    for s in submissions:
        ts = s.get('submitted_at', '')
        if ts:
            try:
                dt = datetime.fromisoformat(ts.replace('Z', '+00:00'))
                if dt.tzinfo is None:
                    dt = dt.replace(tzinfo=timezone.utc)
                s['submitted_at'] = dt.astimezone(_kenyan_tz).isoformat()
            except Exception:
                pass

    sent_map = {s['client_id']: s for s in submissions}
    excluded_traders = set(_json_mod.loads(get_setting('summary_tracker_excluded_traders')) or '[]')
    excluded_clients = set(_json_mod.loads(get_setting('summary_tracker_excluded_clients')) or '[]')

```

```

all_hierarchy_clients = hierarchy_get_all_clients()
tracker_traders = {}
tracked_total = 0
for client_name in all_hierarchy_clients:
    profile = _gcp(client_name)
    trader = (profile.get('trader', '') if profile else '') or 'Unassigned'
    if trader in excluded_traders or client_name in excluded_clients:
        continue
    try:
        cdata = get_client_data(client_name)
        if cdata and isinstance(cdata.get('identity'), dict):
            if cdata['identity'].get('active_status') == 'inactive':
                continue
    except Exception:
        pass
    tracked_total += 1
    if trader not in tracker_traders:
        tracker_traders[trader] = {'sent': [], 'not_sent': [], 'total': 0}
    tracker_traders[trader]['total'] += 1
    if client_name in sent_map:
        ts = sent_map[client_name].get('submitted_at', '')
        tracker_traders[trader]['sent'].append(ts)
    else:
        tracker_traders[trader]['not_sent'].append(client_name)

tracker_complete = [] # 100% – will be ranked
tracker_incomplete = [] # partial or zero sends
for t, d in tracker_traders.items():
    sent_count = len(d['sent'])
    if sent_count == d['total']:
        minutes_list = []
        for ts in d['sent']:
            try:
                dt = datetime.fromisoformat(ts)
                minutes_list.append(dt.hour * 60 + dt.minute)
            except Exception:
                pass
        avg_minutes = round(sum(minutes_list) / len(minutes_list)) if minutes_list else 1440
        avg_hh = avg_minutes // 60
        avg_mm = avg_minutes % 60
        avg_time_str = f"{avg_hh:02d}:{avg_mm:02d}"
        tracker_complete.append((t, sent_count, d['total'], avg_minutes, avg_time_str))
    else:
        tracker_incomplete.append((t, sent_count, d['total'], d['not_sent']))

tracker_complete.sort(key=lambda x: x[3])
tracker_incomplete.sort(key=lambda x: x[0])
total_sent_summary = sum(x[1] for x in tracker_complete) + sum(x[1] for x in tracker_incomplete)

lines.append("■ *DAILY SUMMARY SUBMISSION BY MIDNIGHT (KENYAN TIME)*")
# Skip submission tracking on weekends (no trading Sat/Sun)
from datetime import timezone as _tz2, timedelta as _td2
_eat_now = datetime.now(_tz2(_td2(hours=3)))
_is_weekend = _eat_now.weekday() in (5, 6) # Saturday=5, Sunday=6
if _is_weekend:
    lines.append("■ _Weekend – no trading today. Submission tracking resumes on Monday._")
    lines.append("")
else:
    pct = round(total_sent_summary / tracked_total * 100) if tracked_total else 0
    lines.append(f"■ {total_sent_summary}/{tracked_total} sent ({pct}%)")

```



```

lines.append("")
if tracker_complete:
    lines.append("■ *Complete – ranked by earliest avg submission time:")
    lines.append(
        "_All your clients' summaries must be submitted to qualify. The earlier you submit, t
    for rank, (t, sent, total, _avg_m, avg_t) in enumerate(tracker_complete, 1):
        if rank == 1:
            medal = '■'
        elif rank == 2:
            medal = '■'
        elif rank == 3:
            medal = '■'
        else:
            medal = f'#{rank}'
        lines.append(f"{medal} *{t}* – {sent}/{total} ■ · avg {avg_t}")
    lines.append("")
if tracker_incomplete:
    lines.append("■ *Incomplete – missing clients:")
    for t, sent, total, missing in tracker_incomplete:
        lines.append(f"■ *{t}* – {sent}/{total} sent")
        lines.append(f"    ■ {' '.join(missing)}")
    lines.append("")
    lines.append("■■ _We track everything – every submission, every miss, every second._")
    lines.append("")
except Exception as e:
    import traceback
    traceback.print_exc()

# ■■ Admin Tracker Summary (issues + sign-offs) ■■
try:
    from dashboard.app import compute_admin_tracker_payload

    admins_map = SYSTEM_HIERARCHY.get('admins', {}) if isinstance(SYSTEM_HIERARCHY, dict) else {}
    admin_names = sorted([a for a in admins_map.keys() if str(a).strip()])

    if admin_names:
        lines.append("-")
        lines.append("")
        lines.append("■■ *ADMIN HEALTH LEADERBOARD*")
        lines.append(
            "_Ranked by admin health score (highest first). Admin score is derived from admin-owned i
        lines.append("")

        severity_weight = {'critical': 20, 'high': 10, 'medium': 5, 'low': 2, 'warning': 3, 'info': 0}
        admin_rows = []
        total_admin_issues = 0
        total_required_signoffs = 0
        total_signed_signoffs = 0

        for a in admin_names:
            payload = compute_admin_tracker_payload(a, date) or {}
            issues = payload.get('admin_issues') or []
            # Add implicit "signoff pending" as an admin issue driver (already captured as pending_t
            sign = payload.get('summary_signoff') or {}
            required = int(sign.get('required_total') or 0)
            signed = int(sign.get('signed_total') or 0)
            pending = int(sign.get('pending_total') or 0)

            deduction = 0
            for iss in issues:

```

```

        deduction += severity_weight.get((iss.get('severity') or 'low'), 2)
    # Mild penalty for missing sign-offs so leaderboard reflects it even if no other issues.
    deduction += pending * 5
    health = max(0.0, round(100.0 - deduction, 1))

    total_admin_issues += len(issues)
    total_required_signoffs += required
    total_signed_signoffs += signed

    # Determine admin client count (active only, excluded already applied inside payload)
    active_clients = payload.get('total_clients')
    if active_clients is None:
        active_clients = len([c for c in (payload.get('clients') or []) if (c.get(
            'active_status') == 'active')])

    admin_rows.append({
        'admin': a,
        'health': health,
        'clients': int(active_clients or 0),
        'issues': len(issues),
        'pending_signoffs': pending,
        'pending_clients': (sign.get('pending_clients') or []),
    })

admin_rows.sort(key=lambda r: (r['health'], -r['clients']), reverse=True)

# Quick portfolio-level admin stats
avg_admin_health = round(sum(r['health'] for r in admin_rows) / len(admin_rows),
    1) if admin_rows else 0
lines.append(f"■ *Admins tracked:* {len(admin_rows)}")
lines.append(
    f"■ Avg Admin Health: {avg_admin_health}% | Total Admin Issues: {total_admin_issues}"
)
if total_required_signoffs:
    pct = round((total_signed_signoffs / total_required_signoffs) * 100)
    lines.append(
        f"■ Admin sign-offs: {total_signed_signoffs}/{total_required_signoffs} ({pct}%)"
    )
else:
    lines.append("■ Admin sign-offs: *0/0* (no trader submissions yet)")
lines.append("")

# Leaderboard
for rank, r in enumerate(admin_rows, 1):
    if rank == 1:
        medal = '■'
    elif rank == 2:
        medal = '■'
    elif rank == 3:
        medal = '■'
    else:
        medal = f'#{rank}'
    avg = r['health']
    if avg >= 95:
        title = '■ Legendary'
    elif avg >= 90:
        title = '■ Elite'
    elif avg >= 80:
        title = '■ Solid'
    elif avg >= 70:
        title = '■ Warming Up'
    elif avg >= 50:

```

```

        title = '■ Needs Work'
    else:
        title = '■ SOS'
    bar_filled = round(avg / 10)
    bar_empty = 10 - bar_filled
    bar = '■' * bar_filled + '■' * bar_empty
    extra = f" · {r['pending_signoffs']} pending sign-offs" if r['pending_signoffs'] else ""
    lines.append(f"{medal} *{r['admin']}* — {title}")
    lines.append(f"    {bar} *{avg}% · {r['clients']} clients · {r['issues']} issues{extra}")
lines.append("")

# Admin sign-off missing list (only show admins with pending sign-offs)
incomplete = [r for r in admin_rows if r['pending_signoffs'] > 0]
if incomplete:
    lines.append("■ *ADMIN DAILY SUMMARY SIGN-OFF (after trader submits)*")
    lines.append(f"■ *Incomplete — pending client sign-offs:*")
    for r in sorted(incomplete, key=lambda x: (-x['pending_signoffs'], x['admin'])):
        lines.append(f"■ *{r['admin']}* — {r['pending_signoffs']} pending")
        # Keep Slack message readable; cap long client lists.
        missing = r.get('pending_clients') or []
        if len(missing) > 25:
            shown = ", ".join(missing[:25]) + f", +{len(missing) - 25} more"
        else:
            shown = ", ".join(missing)
        lines.append(f"    ■ {shown}")
    lines.append("")
except Exception:
    import traceback
    traceback.print_exc()

return "\n".join(lines)

```

```
def post_slack_summary()
```

Build and post the daily quality summary to Slack.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def post_slack_summary():
    """Build and post the daily quality summary to Slack."""
    try:
        text = _build_daily_summary_text()
        send_slack_message(text)
    except Exception as e:
        logging.error(f"Failed to build/post Slack summary: {e}")

```

```
def run_database_cleanup()
```

Run daily database cleanup: prune data_history, audit_log, expired sessions.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def run_database_cleanup():
    """Run daily database cleanup: prune data_history, audit_log, expired sessions."""
    try:
        from dashboard.database import cleanup_database
        results = cleanup_database()
        logging.info(f"Database cleanup results: {results}")
    except Exception as e:
        logging.error(f"Database cleanup failed: {e}")
```

```
def update_all_clients_watermarks()
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def update_all_clients_watermarks():
    try:
        clients = get_all_clients() # Returns a dict: {client_id: client_data}
        today_str = datetime.now().strftime('%Y-%m-%d')

        for client_id, client in clients.items():
            try:
                # Pull net profit directly from stored statistics
                # (already includes discrepancy from the last data push)
                stored_stats = client.get('statistics', {})
                if not isinstance(stored_stats, dict):
                    logging.info(f"Skipping {client_id}: no statistics data")
```

```

        continue

    net_profit = stored_stats.get('cashflow_inprogress', {}).get('net_profit')
    if net_profit is None:
        net_profit = stored_stats.get('profitability_completed', {}).get('net_profit')
    if net_profit is None:
        logging.info(f"Skipping {client_id}: no net_profit in statistics")
        continue

    # Save daily snapshot
    save_daily_profit(client_id, net_profit, today_str, source='auto')
    logging.info(f"Updated watermark for {client_id}: {net_profit}")

except Exception as e:
    logging.error(f"Error updating client {client_id}: {e}")

except Exception as e:
    logging.error(f"Error in update_all_clients_watermarks: {e}")

def start_scheduler()

```

Why these Python concepts were used

- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **global** — Rebinds a module-level name from inside the function. A smell in larger codebases — usually means a singleton or cache that could be passed in instead.

Step by step (top-level body)

1. Declares globals: `_scheduler_started`.
2. `if os.environ.get('WERKZEUG_RUN_MAIN') == 'false':` branches conditionally.
3. `with _scheduler_lock:` enters a context manager.
4. Calls `stop_event.clear(...)` for its side effect.
5. Assigns `t = threading.Thread(target=run_scheduler, daemon=True)`.
6. Calls `t.start(...)` for its side effect.
7. `return t`.

Code (copy-paste safe)

```

def start_scheduler():
    global _scheduler_started
    # Flask's Werkzeug reloader spawns TWO processes: a parent watcher and a
    # child that actually serves. Only start the scheduler in the child
    # (WERKZEUG_RUN_MAIN='true') to avoid duplicate threads. When the
    # reloader is disabled (e.g. production), WERKZEUG_RUN_MAIN is unset and
    # there is only one process – that's fine, start normally.
    if os.environ.get('WERKZEUG_RUN_MAIN') == 'false':
        return None # Parent watcher process – skip
    with _scheduler_lock:
        if _scheduler_started:
            logging.info("Scheduler already running – skipping duplicate start.")
            return None
        _scheduler_started = True

```

```
# Ensure stop_event is clear so the new thread's loop runs
stop_event.clear()
t = threading.Thread(target=run_scheduler, daemon=True)
t.start()
return t
```

Step 11.12 — Build dashboard/app.py

What to do. Create the file dashboard/app.py in your project root and paste in the code shown in this step. The file is **10470** lines long; we walk through it piece by piece below.

Depends on. This file imports from internal modules built in earlier steps: dashboard, config. If any of those imports fail, jump back to the step that introduced them.

What this file is for. The Flask application factory. Registers blueprints, sets up session config, attaches scheduled jobs, and exposes the request-time hooks (login required, role checks).

dashboard/app.py

10470 loc · 2 classes · 172 functions · 25 imports

The Flask application factory. Registers blueprints, sets up session config, attaches scheduled jobs, and exposes the request-time hooks (login required, role checks). It defines **2** classes and **172** module-level functions, across **10,470** lines.

Python concepts demonstrated in this module

- Classes and objects
- Closures
- Comprehensions
- Context managers (with-statements)
- Decorators
- Exceptions
- Flask routes
- f-strings
- Inheritance
- Lambdas
- Type hints

Imports — what each one is for

```
from flask import Flask
from flask import render_template
from flask import jsonify
from flask import request
from flask import redirect
from flask import url_for
from flask import g
```

Why imported. Lightweight WSGI web framework. Powers the dashboard.

Used in this file. Flask, g, jsonify, redirect, render_template, request, url_for

Example. @app.route('/login', methods=['POST'])\ndef login(): ...

Other common scenarios.

- request.form / request.args / request.json read input
- render_template('x.html', **ctx) renders a Jinja2 template

- `redirect(url_for('view'))` builds URLs from view names
- Blueprints split a large app into pluggable sub-apps

```
from flask_compress import Compress
```

Why imported. *flask_compress* — module specific to this codebase. See the chapter that covers its source for the full definition.

Used in this file. *Compress*

```
from flask_limiter import Limiter
```

Why imported. Per-route rate limiting for Flask — protects login pages and ingestion endpoints from brute force / abuse.

Used in this file. *Limiter*

Example. `@limiter.limit('5/minute')\ndef login(): ...`

Other common scenarios.

- Default keying is by remote IP
- Override `key_func` to limit by user id or API key

```
from flask_limiter.util import get_remote_address
```

Why imported. Per-route rate limiting for Flask — protects login pages and ingestion endpoints from brute force / abuse.

Used in this file. *get_remote_address*

Example. `@limiter.limit('5/minute')\ndef login(): ...`

Other common scenarios.

- Default keying is by remote IP
- Override `key_func` to limit by user id or API key

```
import threading
```

Why imported. Threads and synchronisation primitives. Used when work is I/O-bound and the GIL is not a bottleneck (the GIL prevents speed-up on CPU-bound work).

Used in this file. *threading.Thread*

Example. `Thread(target=worker, args=(q,), daemon=True).start()`

Other common scenarios.

- `Lock` / `RLock` to guard shared state
- `Event` for one-shot signals between threads
- `Queue` (from `queue`) for producer/consumer patterns
- `threading.local()` for thread-local storage

```
import json
```

Why imported. JSON encoding and decoding — the standard wire format for config files, API payloads, and inter-process messages.

Used in this file. `json.dumps`, `json.loads`

Example. `json.loads(response.text)` # parse a JSON string to dict

Other common scenarios.

- `json.dumps(obj, indent=2)` for pretty-printing
- `json.dump(obj, file)` / `json.load(file)` for file I/O
- `default=str` handles non-serialisable types like `datetime`
- `object_hook=` customises decoding (e.g., to `dataclass` instances)

`import os`

Why imported. Operating-system interface. Path manipulation, environment variables, working-directory operations, exit codes, and thin wrappers around POSIX/Windows syscalls.

Used in this file. `os.getenv`, `os.path`, `os.path.abspath`, `os.path.dirname`

Example. `os.path.join(root, 'subdir', 'file.txt')` # joins with the OS-correct separator

Other common scenarios.

- `os.environ.get('API_KEY')` to read environment variables
- `os.makedirs(path, exist_ok=True)` to create nested directories
- `os.listdir(path)` to list a directory's contents
- `os.remove(path)` / `os.rename(src, dst)` to delete or move a file
- `os.getcwd()` / `os.chdir(path)` to inspect or change the working dir

`import sys`

Why imported. Access to the running interpreter — `argv`, `stdin/stdout`, exit codes, the import search path, and the platform name.

Used in this file. `sys.path`, `sys.path.append`, `sys.stderr`

Example. `sys.exit(1)` # terminate the process with a non-zero status

Other common scenarios.

- `sys.argv[1:]` reads the command-line arguments
- `sys.path.insert(0, 'extra')` prepends to the import search path
- `sys.stderr.write(msg)` writes to standard error
- `sys.platform` tells you 'win32', 'linux', or 'darwin'
- `sys.modules` is the global cache of already-imported modules

`import logging`

Why imported. Structured, levelled logging. Replaces `print()` with filterable, formattable output that can route to files, `stderr`, `syslog`, or HTTP endpoints.

Used in this file. logging.DEBUG, logging.FileHandler, logging.Formatter, logging.INFO, logging.StreamHandler, logging.WARNING, logging.basicConfig, logging.debug

Example. logging.getLogger(__name__).info('connected to %s', host)

Other common scenarios.

- .debug() / .info() / .warning() / .error() / .exception()
- logger.exception() captures the current traceback automatically
- logging.basicConfig(level=logging.INFO) for quick setup
- Handlers and Formatters route logs to files / streams / syslog

```
import time
```

Why imported. Timestamps and sleep. Lower-level than datetime — works in Unix-epoch seconds.

Used in this file. time.monotonic, time.sleep, time.time

Example. time.sleep(1.5) # pause this thread for 1.5 seconds

Other common scenarios.

- time.time() returns seconds since the epoch
- time.monotonic() for measuring elapsed time (never goes backward)
- time.perf_counter() for high-precision benchmarking
- time.strftime / time.strptime mirror the datetime versions

```
import errno
```

Why imported. *errno* — module specific to this codebase. See the chapter that covers its source for the full definition.

Used in this file. errno.EPIPE

```
import signal
```

Why imported. Receive POSIX/Windows signals — SIGINT (Ctrl-C), SIGTERM, etc. Used to install graceful-shutdown handlers.

Used in this file. signal.SIGPIPE, signal.SIG_DFL, signal.signal

Example. signal.signal(signal.SIGTERM, on_term)

Other common scenarios.

- Only the main thread can install handlers
- On Windows, SIGTERM is mostly equivalent to a force-kill

```
from functools import wraps
```

Why imported. Higher-order helpers — caching, partial application, decorator authoring tools, reducers.

Used in this file. wraps

Example. @lru_cache(maxsize=128) # memoise this pure function

Other common scenarios.

- `partial(f, x=1)` pre-binds arguments
- `reduce(f, iterable)` folds an iterable to a single value
- `@wraps(fn)` preserves `__name__`/`__doc__` when writing decorators
- `@cached_property` memoises a property per-instance

```
import secrets
```

Why imported. Cryptographically strong randomness — tokens, session IDs, API keys. Never use `random` for anything security-sensitive.

Used in this file. `secrets.token_hex`

Example. `secrets.token_urlsafe(32)` # 32-byte url-safe random string

Other common scenarios.

- `secrets.token_hex(n)` for a hex string
- `secrets.choice(seq)` is the cryptographic equivalent of `random.choice`
- `secrets.compare_digest(a, b)` for timing-safe comparison

```
import hashlib
```

Why imported. Cryptographic hash functions — SHA-256, MD5, BLAKE2.

Used in this file. *No direct attribute access detected — likely imported for side effects, re-export, or string references.*

Example. `hashlib.sha256(b'hello').hexdigest()`

Other common scenarios.

- Pass data in chunks via `.update()` to hash large files
- Use `sha256` / `sha512` for integrity; never MD5 or SHA-1 for security
- For password hashing use `bcrypt`/`argon2`, NOT raw `hashlib`

```
import re
```

Why imported. Regular expressions. Pattern matching, search, replace, and text extraction.

Used in this file. `re.IGNORECASE`, `re.compile`, `re.findall`, `re.match`, `re.search`, `re.sub`

Example. `re.search(r'\d{3}-\d{4}', text)` # returns a `Match` or `None`

Other common scenarios.

- `re.findall(pattern, text)` for every non-overlapping match
- `re.sub(pattern, repl, text)` to replace occurrences
- `re.compile(...)` to reuse a pattern (faster in a loop)
- Named groups: `r'(?P<year>\d{4})'` lets you do `m.group('year')`

```
from datetime import datetime
from datetime import timedelta
```

Why imported. Dates, times, durations. The fundamental temporal types: date, time, datetime, timedelta, timezone.

Used in this file. datetime, timedelta

Example. datetime.now() - timedelta(days=7) # one week ago

Other common scenarios.

- datetime.strptime(s, '%Y-%m-%d') parses a string
- datetime.strftime(fmt) formats one
- datetime.fromtimestamp(n) converts a Unix epoch
- datetime.utcnow() for UTC (better: datetime.now(timezone.utc))

```
from dashboard.financial_overview import calculate_propfirm_overview
from dashboard.financial_overview import get_payouts_history
from dashboard.financial_overview import get_portfolio_growth_data
from dashboard.financial_overview import get_payouts_growth_data
from dashboard.financial_overview import get_cumulative_deposits
from dashboard.financial_overview import get_cumulative_trading_profit
from dashboard.financial_overview import get_cumulative_fees_data
from dashboard.financial_overview import get_cumulative_hedge_data
from dashboard.financial_overview import get_cumulative_farming_data
from dashboard.financial_overview import calculate_trader_stats
from dashboard.financial_overview import parse_date
from dashboard.financial_overview import get_cached_clients_dataset
from dashboard.financial_overview import calculate_all_financials
from dashboard.financial_overview import get_client_performance_stats
```

Why imported. Internal package — the Flask dashboard. See chapter on dashboard/.

Used in this file. calculate_all_financials, calculate_propfirm_overview, calculate_trader_stats, get_client_performance_stats, get_payouts_history, parse_date

```
from config.hierarchy import SYSTEM_HIERARCHY
from config.hierarchy import add_admin
from config.hierarchy import add_trader
from config.hierarchy import add_client
from config.hierarchy import update_admin_details
from config.hierarchy import update_trader_details
from config.hierarchy import update_client_details
from config.hierarchy import update_client_category
from config.hierarchy import get_client_by_email
from config.hierarchy import get_user_by_email
from config.hierarchy import get_client_profile
from config.hierarchy import remove_admin
from config.hierarchy import remove_trader
from config.hierarchy import remove_client
from config.hierarchy import move_client
from config.hierarchy import move_trader
from config.hierarchy import rename_admin
from config.hierarchy import rename_trader
from config.hierarchy import rename_client
from config.hierarchy import reassign_client_trader
```

Why imported. Internal package — application settings and the firm hierarchy.

Used in this file. SYSTEM_HIERARCHY, add_admin, add_client, add_trader, get_client_by_email, get_client_profile, get_user_by_email, move_client, move_trader, reassign_client_trader, remove_admin, remove_client ... (+8 more)

```
from dashboard.database import init_database
from dashboard.database import check_and_repair_database
from dashboard.database import validate_api_key
from dashboard.database import generate_api_key
from dashboard.database import list_api_keys
from dashboard.database import revoke_api_key
from dashboard.database import verify_admin_password
from dashboard.database import set_admin_password
from dashboard.database import admin_password_exists
from dashboard.database import copy_admin_password_row
from dashboard.database import save_client_data
from dashboard.database import get_client_data
from dashboard.database import get_all_clients
from dashboard.database import get_clients_count
from dashboard.database import update_client_field
from dashboard.database import delete_client_data
from dashboard.database import log_action
from dashboard.database import get_audit_log
from dashboard.database import create_session
from dashboard.database import validate_session
from dashboard.database import delete_session
from dashboard.database import create_user
from dashboard.database import verify_user_password
from dashboard.database import verify_client_login
from dashboard.database import update_user_password
from dashboard.database import delete_user_credential
from dashboard.database import update_user_email
from dashboard.database import rename_user_credential
from dashboard.database import rename_client_in_db
from dashboard.database import get_user
from dashboard.database import list_users
from dashboard.database import deactivate_user
from dashboard.database import reset_user_password
from dashboard.database import user_exists
from dashboard.database import record_login_attempt
from dashboard.database import is_account_locked
from dashboard.database import find_user_by_identifier
from dashboard.database import verify_user_by_identifier
from dashboard.database import save_client_data_with_history
from dashboard.database import get_data_history
from dashboard.database import get_data_version
from dashboard.database import rollback_to_version
from dashboard.database import compare_versions
from dashboard.database import get_latest_version
from dashboard.database import add_kyc_link
from dashboard.database import remove_kyc_link
from dashboard.database import get_kyc_linked_clients
from dashboard.database import get_kyc_primary_for
from dashboard.database import get_all_kyc_accounts
from dashboard.database import get_all_kyc_links
from dashboard.database import is_kyc_primary
```

Why imported. Internal package — the Flask dashboard. See chapter on dashboard/.

Used in this file. `add_kyc_link`, `admin_password_exists`, `check_and_repair_database`, `compare_versions`, `copy_admin_password_row`, `create_session`, `create_user`, `deactivate_user`, `delete_client_data`, `delete_session`, `delete_user_credential`, `find_user_by_identifier` ... (+34 more)

```
from dashboard.notes_service import get_client_notes
from dashboard.notes_service import save_client_note
from dashboard.notes_service import delete_client_note
```

Why imported. Internal package — the Flask dashboard. See chapter on dashboard/.

Used in this file. `delete_client_note`, `get_client_notes`, `save_client_note`

```
from dashboard.utils.trade_matcher import UnifiedTradeMatcher
```

Why imported. Internal package — the Flask dashboard. See chapter on dashboard/.

Used in this file. *No direct attribute access detected — likely imported for side effects, re-export, or string references.*

```
from logging.handlers import RotatingFileHandler
```

Why imported. Structured, levelled logging. Replaces `print()` with filterable, formattable output that can route to files, stderr, syslog, or HTTP endpoints.

Used in this file. `RotatingFileHandler`

Example. `logging.getLogger(__name__).info('connected to %s', host)`

Other common scenarios.

- `.debug()` / `.info()` / `.warning()` / `.error()` / `.exception()`
- `logger.exception()` captures the current traceback automatically
- `logging.basicConfig(level=logging.INFO)` for quick setup
- Handlers and Formatters route logs to files / streams / syslog

```
import signal
```

Why imported. Receive POSIX/Windows signals — `SIGINT` (Ctrl-C), `SIGTERM`, etc. Used to install graceful-shutdown handlers.

Used in this file. `signal.SIGPIPE`, `signal.SIG_DFL`, `signal.signal`

Example. `signal.signal(signal.SIGTERM, on_term)`

Other common scenarios.

- Only the main thread can install handlers
- On Windows, `SIGTERM` is mostly equivalent to a force-kill

```
from dashboard.financial_overview import calculate_all_financials
```

Why imported. Internal package — the Flask dashboard. See chapter on dashboard/.

Used in this file. `calculate_all_financials`

Module constants

Each line below is followed by a plain-English note explaining what it does and why it is written that way.

```
BEF_HIDDEN_FIRMS = {'lucid', 'apex', 'tradeday', 'toponefutures'}
```

Mapping or set literal — a lookup table referenced elsewhere in the module.

```
_RECENT_LOG_PATH = 'dashboard/server_recent.log'
```

String literal — fixed at code-load time.

```
_RECENT_LOG_RE = re.compile('^\\[((?:\\d{4}-\\d{2}-\\d{2}) ?)\\d{2}:\\d{2}:\\d{2}(?:,\\d+)?\\]\\')
```

Pre-compiled regular expression — compiled once at import time and reused, which is faster than compiling on every call.

```
_LOG_LEVEL_NAME = str(os.getenv('DASHBOARD_LOG_LEVEL', 'INFO')).upper()
```

Module-level constant — bound once at import time and referenced from the functions and classes below.

```
_LOG_LEVEL = getattr(logging, _LOG_LEVEL_NAME, logging.INFO)
```

Module-level constant — bound once at import time and referenced from the functions and classes below.

```
_STATS_TAB_TTL = 300
```

Numeric literal — a tunable parameter compiled into the source.

```
DASHBOARD_OWNED_EVAL_KEYS = frozenset({'Payout 1', 'Date 1', 'Payout 2', 'Date 2', 'Payout 3', 'Date 3',  
    'Payout 4', 'Date 4', 'Payout 5', 'Date 5', 'Payout 6', 'Dat...
```

Module-level constant — bound once at import time and referenced from the functions and classes below.

```
MAINTENANCE_MODE = False
```

Module-level constant — bound once at import time and referenced from the functions and classes below.

```
MAINTENANCE_EXEMPT = {'super_admin', 'bef_admin', 'kwok_admin', 'admin', 'trader'}
```

Mapping or set literal — a lookup table referenced elsewhere in the module.

```
_INTERNAL_DEAL_TYPES_INT = {2, 3}
```

Mapping or set literal — a lookup table referenced elsewhere in the module.

```
_INTERNAL_DEAL_TYPES_STR = {'BALANCE', '2', '2.0', 'CREDIT', '3', '3.0', 'DEAL_TYPE_BALANCE',  
    'DEAL_TYPE_CREDIT'}
```

Mapping or set literal — a lookup table referenced elsewhere in the module.

Classes

```
class UnbufferedFileHandler(logging.FileHandler)
```

Code (copy-paste safe)

```
class UnbufferedFileHandler(logging.FileHandler):
    def emit(self, record):
        super().emit(record)
        self.stream.flush()
```

Method: UnbufferedFileHandler.emit

```
def emit(self, record)
```

Step by step (top-level body)

1. Calls `super().emit(...)` for its side effect.
2. Calls `self.stream.flush(...)` for its side effect.

Code (copy-paste safe)

```
def emit(self, record):
    super().emit(record)
    self.stream.flush()
```

```
class TraderStyleFormatter(logging.Formatter)
```

Compact log format to match Trader Companion readability.

Code (copy-paste safe)

```
class TraderStyleFormatter(logging.Formatter):
    """Compact log format to match Trader Companion readability."""

    def format(self, record):
        stamp = datetime.fromtimestamp(record.created).strftime('%H:%M:%S')
        msg = record.getMessage()
        if record.levelno >= logging.WARNING:
            msg = f"[{record.levelname}] {msg}"
        return f"[{stamp}] {msg}"
```

Method: TraderStyleFormatter.format

```
def format(self, record)
```

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `stamp = datetime.fromtimestamp(record.created).strftime('%H:%M:%S')`.
2. Assigns `msg = record.getMessage()`.
3. **if** `record.levelno >= logging.WARNING`: branches conditionally.
4. **return** `f"[{stamp}] {msg}"`.

Code (copy-paste safe)

```
def format(self, record):
    stamp = datetime.fromtimestamp(record.created).strftime('%H:%M:%S')
    msg = record.getMessage()
```

```

if record.levelno >= logging.WARNING:
    msg = f"[{record.levelname}] {msg}"
    return f"[{stamp}] {msg}"

```

Functions

```
def _parse_recent_log_timestamp(ts_text)
```

Parse either full datetime or time-only log stamps.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

Step by step (top-level body)

1. **for** `fmt` in `('%Y-%m-%d %H:%M:%S,%f', '%Y-%m-%d %H:%M:%S')`: iterates.
2. **for** `fmt` in `('%H:%M:%S,%f', '%H:%M:%S')`: iterates.
3. **return** `None`.

Code (copy-paste safe)

```

def _parse_recent_log_timestamp(ts_text):
    """Parse either full datetime or time-only log stamps."""
    # Full timestamp variants.
    for fmt in ('%Y-%m-%d %H:%M:%S,%f', '%Y-%m-%d %H:%M:%S'):
        try:
            return datetime.strptime(ts_text, fmt)
        except ValueError:
            pass

    # Time-only variants (new concise format): assume today, with a safe
    # midnight rollover fallback when a future time appears.
    for fmt in ('%H:%M:%S,%f', '%H:%M:%S'):
        try:
            t = datetime.strptime(ts_text, fmt).time()
            now = datetime.now()
            parsed = datetime.combine(now.date(), t)
            if parsed - now > timedelta(minutes=5):
                parsed -= timedelta(days=1)
            return parsed
        except ValueError:
            pass
    return None

```

```
def _purge_recent_log(log_path=_RECENT_LOG_PATH, hours=1)
```

Trim log_path in-place, keeping only entries from the last `hours` hours.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def _purge_recent_log(log_path=_RECENT_LOG_PATH, hours=1):
    """Trim log_path in-place, keeping only entries from the last `hours` hours."""
    try:
        cutoff = datetime.now() - timedelta(hours=hours)
        with open(log_path, 'r', encoding='utf-8', errors='replace') as fh:
            lines = fh.readlines()
            kept = []
            for line in lines:
                m = _RECENT_LOG_RE.match(line)
                if m:
                    ts = _parse_recent_log_timestamp(m.group(1))
                    if ts is None:
                        kept.append(line) # unparseable timestamp → keep
                    elif ts >= cutoff:
                        kept.append(line)
                    # else: drop – older than the window
                else:
                    # Continuation / blank lines: keep only if we have recent content
                    if kept:
                        kept.append(line)
            with open(log_path, 'w', encoding='utf-8') as fh:
                fh.writelines(kept)
    except (FileNotFoundError, IOError):
        pass
```

```
def _start_recent_log_purger(log_path=_RECENT_LOG_PATH, interval_minutes=5)
```

Daemon thread: prune entries older than 1 hour from the recent log every 5 minutes.

Why these Python concepts were used

- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.

Step by step (top-level body)

1. Defines a nested function `_worker(...)`.
2. Assigns `t = threading.Thread(target=_worker, daemon=True, name='recen...`
3. Calls `t.start(...)` for its side effect.

Code (copy-paste safe)

```
def _start_recent_log_purger(log_path=_RECENT_LOG_PATH, interval_minutes=5):
    """Daemon thread: prune entries older than 1 hour from the recent log every 5 minutes."""
    def _worker():
        while True:
            time.sleep(interval_minutes * 60)
            _purge_recent_log(log_path)
    t = threading.Thread(target=_worker, daemon=True, name='recent-log-purger')
    t.start()
```

```
@app.before_request
def _log_request_start()
```

Why these Python concepts were used

- **@app.before_request** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.

Step by step (top-level body)

1. Assigns `g._request_start = time.monotonic()`.

Code (copy-paste safe)

```
def _log_request_start():
    g._request_start = time.monotonic()
```

```
@app.before_request
def _decompress_request_body()
```

Why these Python concepts were used

- **@app.before_request** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. `if request.headers.get('Content-Encoding') == 'gzip':` branches conditionally.

Code (copy-paste safe)

```
def _decompress_request_body():
    if request.headers.get('Content-Encoding') == 'gzip':
        import gzip as _gzip
        import io as _io
        try:
            # Read directly from wsgi.input BEFORE Werkzeug wraps it into a
            # LimitedStream or caches it – calling request.get_data() here would
            # cache the compressed bytes and the JSON parser would see garbage.
            raw_stream = request.environ.get('wsgi.input')
            content_length = request.environ.get('CONTENT_LENGTH')
            try:
                compressed_size = int(content_length) if content_length else 0
            except (TypeError, ValueError):
                compressed_size = 0

            if raw_stream and compressed_size > 0:
                compressed = raw_stream.read(compressed_size)
```

```

    else:
        compressed = raw_stream.read() if raw_stream else b''
        decompressed = _gzip.decompress(compressed)
        request.environ['wsgi.input'] = _io.BytesIO(decompressed)
        request.environ['CONTENT_LENGTH'] = str(len(decompressed))
        request.environ.pop('HTTP_CONTENT_ENCODING', None)
except Exception as e:
    logging.warning(f'[DECOMPRESS] Failed to decompress gzip request: {e}')

```

```

@app.after_request
def _log_request_end(response)

```

Why these Python concepts were used

- **@app.after_request** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.

Step by step (top-level body)

1. **try** block with 1 **except** clause.
2. **return** response.

Code (copy-paste safe)

```

def _log_request_end(response):
    try:
        duration_ms = (time.monotonic() - getattr(g, '_request_start', time.monotonic())) * 1000
        logging.info('[REQ] %s %s -> %s (%.1fms)', request.method, request.path, response.status_code,
                     duration_ms)
    except OSError as e:
        # Client disconnected during response write (SIGPIPE / broken pipe)
        # This is benign and not a server error – suppress as debug log
        if e.errno == errno.EPIPE:
            logging.debug('[REQ_BROKEN_PIPE] Client disconnect on %s %s', request.method, request.path)
        else:
            raise
    return response

```

```

def _init_database_pool()

```

Initialize database connection pool on app startup.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

- **raise** — Raises an exception explicitly. Either signals an invalid argument the caller should fix, or re-raises after logging.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def _init_database_pool():
    """Initialize database connection pool on app startup."""
    try:
        from dashboard.database import _init_pool
        _init_pool()
        logging.info("[STARTUP] Database connection pool initialized")
    except Exception as e:
        logging.error(f"[STARTUP ERROR] Failed to initialize DB pool: {e}")
        raise
```

```
def get_account_signature(account_number)
```

*Extract account signature for matching: first 4 + last 4/5 digits. This handles truncated account numbers in MT5 comments. Handles truncated format with '...' like: FNFT...59574 Examples:
MFFUEVSTP326057008 -> mffu7008 (full account) FNFT...59574 -> fnft59574 (truncated - keep last 5)
12345678 -> 12345678*

Why these Python concepts were used

- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **if not account_number**: branches conditionally.
2. Assigns `account_str = str(account_number).strip()`.
3. **if '...' in account_str**: branches conditionally.
4. **if len(account_str) < 8**: branches conditionally.
5. **return** (`account_str[:4] + account_str[-4:].lower()`).

Code (copy-paste safe)

```
def get_account_signature(account_number):
    """
    Extract account signature for matching: first 4 + last 4/5 digits.
    This handles truncated account numbers in MT5 comments.

    Handles truncated format with '...' like: FNFT...59574

    Examples:
    MFFUEVSTP326057008 -> mffu7008 (full account)
    FNFT...59574 -> fnft59574 (truncated - keep last 5)
    12345678 -> 12345678
    """
    if not account_number:
        return None

    account_str = str(account_number).strip()
```

```

# Handle truncated format: PREFIX...SUFFIX
if '...' in account_str:
    parts = account_str.split('...')
    if len(parts) == 2:
        prefix = parts[0][:4] if len(parts[0]) >= 4 else parts[0]
        suffix = parts[1] # Keep full suffix (usually 5 digits)
        return (prefix + suffix).lower()

# Standard format: first 4 + last 4
if len(account_str) < 8:
    return account_str.lower()

# First 4 + last 4
return (account_str[:4] + account_str[-4:]).lower()

```

```
def get_last_n_digits(account: str, n: int=5) -> str
```

Extract last N digits from account number. Prioritizes digits at the end of the string to avoid prefix contamination.

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `account: str, n: int`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **return type annotation** — Annotated return type `-> str`. Declares the function's contract: callers can rely on this shape, and tools can flag a caller that misuses the result.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to `sum`, `any`, `all`, or `max` where the intermediate list would be wasted.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Imports `re` (lazy import inside the function).
2. `if not account:` branches conditionally.
3. Assigns `match = re.search('(\d+)$', str(account).strip())`.
4. `if match:` branches conditionally.
5. Assigns `digits = "".join((c for c in str(account) if c.isdigit()))`.
6. `return digits[-n:] if len(digits) >= n else digits`.

Code (copy-paste safe)

```

def get_last_n_digits(account: str, n: int = 5) -> str:
    """
    Extract last N digits from account number.
    Prioritizes digits at the end of the string to avoid prefix contamination.
    """
    import re
    if not account:
        return ""

```

```

# Try to extract the final sequence of digits
match = re.search(r'(\d+)$', str(account).strip())
if match:
    digits = match.group(1)
    # For Topstep (V2-) or short accounts, ensure we don't demand more digits than exist
    if len(digits) < n:
        return digits
    return digits[-n:]

# Fallback: extract all digits
digits = ''.join(c for c in str(account) if c.isdigit())
return digits[-n:] if len(digits) >= n else digits

def match_account_to_evaluation(account_number, evaluations, phase_code)

```

Find ALL matching evaluations for an account number based on phase. For Challenge (CH): Match against 'Account #' column For Funded/DoubleDip (FD, DD): Match against 'Account #.1' column For Farming (FA): Match against 'Account #.1' (funded); if blank, fall back to 'Account #' (challenge) Returns: List of (eval_index, matched_account)

Why these Python concepts were used

- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.

Step by step (top-level body)

1. Imports logging (lazy import inside the function).
2. Assigns matches = [].
3. **if** not account_number: branches conditionally.
4. Assigns target_sig = get_account_signature(account_number).
5. Assigns target_last5 = get_last_n_digits(account_number, 5).
6. Calls logging.debug(...) for its side effect.
7. **if** not target_sig and (not target_last5): branches conditionally.
8. Defines a nested function get_prefix(...).
9. Assigns target_prefix = get_prefix(account_number).
10. Assigns _PLACEHOLDER_ACCOUNTS = {'', 'none', '-', '—', '—', 'n/a', 'na', 'tbd', 'pending'}.
11. Defines a nested function _is_real_account(...).
12. Assigns combined_accounts = [].
13. **for** (idx, ev) in enumerate(evaluations): iterates.
14. Assigns is_topstep = str(account_number).upper().startswith('V2').

15. ... and 4 more statement(s) in the body.

Code (copy-paste safe)

```
def match_account_to_evaluation(account_number, evaluations, phase_code):
    """
    Find ALL matching evaluations for an account number based on phase.

    For Challenge (CH): Match against 'Account #' column
    For Funded/DoubleDip (FD, DD): Match against 'Account #.1' column
    For Farming (FA): Match against 'Account #.1' (funded); if blank, fall back to 'Account #' (challenge)

    Returns: List of (eval_index, matched_account)
    """
    import logging
    matches = []
    if not account_number:
        logging.debug(f"[MATCH] No account_number provided.")
        return matches

    target_sig = get_account_signature(account_number)
    target_last5 = get_last_n_digits(account_number, 5)
    logging.debug(
        f"[MATCH] Target account: {account_number} | Signature: {target_sig} | Last5: {target_last5}")
    if not target_sig and not target_last5:
        logging.debug(f"[MATCH] No signature or last5 for target account.")
        return matches

def get_prefix(acc_str):
    import re
    s = str(acc_str).strip().upper()
    if '...' in s:
        s = s.split('...')[0]
    if '-' in s:
        return s.split('-')[0]
    m = re.match(r'^([A-Z]+)', s)
    if m:
        return m.group(1)
    return None

target_prefix = get_prefix(account_number)

# Combine funded and evaluation accounts for matching.
# For FA (farming) phase: prefer Account #.1 (funded account number), but if that
# column is blank for a row, fall back to Account # (challenge account number) so
# that accounts with no separate funded account number can still be matched.
#
# Treat dash/placeholder strings as EMPTY so the matcher cannot write hedge
# results into rows whose user blanked out the account column to "-" / "-." / "-".
_PLACEHOLDER_ACCOUNTS = {'', 'none', '-', '-.', '-.', 'n/a', 'na', 'tbd', 'pending'}

def _is_real_account(s):
    return bool(s) and s.strip().lower() not in _PLACEHOLDER_ACCOUNTS

combined_accounts = []
for idx, ev in enumerate(evaluations):
    funded_account = str(ev.get('account') or '').strip()
    if _is_real_account(funded_account):
        combined_accounts.append({'idx': idx, 'account': funded_account, 'source': 'funded'})

    if phase_code == 'FA':
```

```

        # Prefer funded account number; fall back to challenge account number if blank
        funded_acct = str(ev.get('Account #.1') or '').strip()
        challenge_acct = str(ev.get('Account #') or '').strip()
        if _is_real_account(funded_acct):
            combined_accounts.append({'idx': idx, 'account': funded_acct, 'source': 'Account #.1'})
        elif _is_real_account(challenge_acct):
            combined_accounts.append({'idx': idx, 'account': challenge_acct,
                                     'source': 'Account # (FA fallback)'})
    else:
        for col_name in ['Account #.1', 'Account #']:
            eval_account = str(ev.get(col_name) or '').strip()
            if _is_real_account(eval_account):
                combined_accounts.append({'idx': idx, 'account': eval_account, 'source': col_name})

is_topstep = str(account_number).upper().startswith('V2')
seen_row_indices = set()
for entry in combined_accounts:
    idx = entry['idx']
    eval_account = entry['account']
    source = entry['source']
    if idx in seen_row_indices:
        continue
    eval_sig = get_account_signature(eval_account)
    eval_prefix = get_prefix(eval_account)
    has_letters = any(c.isalpha() for c in str(eval_account).upper())

    # Check for strict prefix mismatch
    prefix_mismatch = False
    if target_prefix and has_letters:
        if eval_prefix and target_prefix != eval_prefix:
            # If they are totally different prefixes, mismatch
            # But allow partials like MFFU vs MFFUEV
            if not (target_prefix.startswith(str(eval_prefix)) or str(eval_prefix).startswith(
                target_prefix)):
                # Also check if target_prefix is inside eval_account (e.g. V2 inside EXPRESS-V2)
                if target_prefix not in eval_account.upper():
                    prefix_mismatch = True

    # eval_last5 = get_last_n_digits(eval_account, 5)
    # logging.debug(f"[MATCH] Comparing to DB account: {eval_account} (source: {source}) | Signature: {eval_sig}")
    if eval_sig == target_sig:
        matches.append((idx, eval_account))
        seen_row_indices.add(idx)
        logging.debug(f"[MATCH] Signature match: {account_number} == {eval_account}")
        continue

    # Strict prefix check applies to partial matches below
    if prefix_mismatch:
        logging.debug(
            f"[MATCH] Rejected partial match due to prefix mismatch: {target_prefix} vs {eval_account}")
        continue

    if target_last5 and len(target_last5) >= 4:
        eval_last5 = get_last_n_digits(eval_account, 5)
        if eval_last5 == target_last5:
            matches.append((idx, eval_account))
            seen_row_indices.add(idx)
            logging.debug(f"[MATCH] Last5 exact match: {target_last5} == {eval_last5}")
            continue
        if len(eval_last5) != len(target_last5) and eval_last5 and target_last5:

```



```

        if eval_last5.endswith(target_last5) or target_last5.endswith(eval_last5):
            matches.append((idx, eval_account))
            seen_row_indices.add(idx)
            logging.debug(f"[MATCH] Last5 suffix match: {target_last5} <-> {eval_last5}")
            continue
    if len(target_last5) >= 4 and len(eval_last5) >= 4:
        if target_last5[-4:] == eval_last5[-4:]:
            matches.append((idx, eval_account))
            seen_row_indices.add(idx)
            logging.debug(f"[MATCH] Last4 match: {target_last5[-4:]} == {eval_last5[-4:]}")
            continue
    if is_topstep:
        target_last4 = get_last_n_digits(account_number, 4)
        eval_last4 = get_last_n_digits(eval_account, 4)
        if target_last4 and len(target_last4) == 4 and target_last4 == eval_last4:
            matches.append((idx, eval_account))
            seen_row_indices.add(idx)
            logging.debug(f"[MATCH] Topstep relaxed 4-digit match: {target_last4} == {eval_last4}")
            continue
    if not matches:
        logging.debug(f"[MATCH] No match found for {account_number} in any DB account.")
    return matches

```

```
def parse_sheet_date(date_str)
```

Parse date string from Google Sheet into datetime object. Supports formats: DD/MM/YYYY, YYYY-MM-DD, MM/DD/YYYY, etc.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

Step by step (top-level body)

1. **if not date_str**: branches conditionally.
2. Assigns `date_str = str(date_str).strip()`.
3. **if not date_str**: branches conditionally.
4. Assigns `formats = ['%m/%d/%Y', '%d/%m/%Y', '%Y-%m-%d', '%m/%d/%y', '%d/%m/%....`
5. **for** `fmt` in `formats`: iterates.
6. **return** `None`.

Code (copy-paste safe)

```

def parse_sheet_date(date_str):
    """
    Parse date string from Google Sheet into datetime object.
    Supports formats: DD/MM/YYYY, YYYY-MM-DD, MM/DD/YYYY, etc.
    """
    if not date_str:
        return None

    date_str = str(date_str).strip()
    if not date_str:
        return None

```

```

formats = [
    '%m/%d/%Y', '%d/%m/%Y', '%Y-%m-%d',
    '%m/%d/%y', '%d/%m/%y',
    '%d-%m-%Y', '%Y/%m/%d', '%d.%m.%Y'
]

for fmt in formats:
    try:
        val = datetime.strptime(date_str, fmt)
        # FORCE TO UTC or STRIP TIMEZONE?
        # MT5 timestamps are usually UTC or server time (which we convert to timestamps).
        # If sheet dates are parsed as naive 00:00:00, and we compare to
        # trade times which might be later in the day, that's fine.
        # But if a trade happens at 23:00 on Feb 18 (UTC), and sheet says Feb 18...
        # The trade timestamp (TS) > Date Purchased TS.
        # But if timezone is involved, eg. Sheet date is parsed as local time?
        # datetime.strptime creates NAIVE datetime.

        # If there's a 1-day difference being observed, it's likely a timezone shift display issue?
        # Or the dashboard is displaying dates differently?
        # Or maybe pandas parsing?

        # User says: "one day difference between the dates on the sheets and the dates on the dashboard"
        # If Sheet says Feb 18, Dashboard says Feb 17? Or Feb 19?

        # If date_str is "2/18/26", datetime is 2026-02-18 00:00:00

        # FIX: Use simple parsing - keep naive dates as imported
        # If date_str is "2/18/26", datetime is 2026-02-18 00:00:00

        return val
    except ValueError:
        continue

return None

```

```
def filter_matches_by_date(matches, evaluations, trade_timestamp, phase_code=None, trade_number=None)
```

Filter matches to find the best matching evaluation. Universal logic for ALL prop firms: - When multiple rows match the same account, always prefer the LATEST row (highest eval_index = most recently added to the dashboard). - Date filtering is used as a secondary signal but never overrides the "latest row wins" rule. Args: matches: List of (eval_index, matched_account) evaluations: List of evaluation dicts trade_timestamp: Timestamp of the trade (float or int) phase_code: (Optional) Phase code to help determine target field for placeholder check trade_number: (Optional) Trade number for target field check

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **list comprehension** — Builds a list in a single expression ([f(x) for x in xs]). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.

Step by step (top-level body)

1. **if** not matches: branches conditionally.
2. **if** not trade_timestamp: branches conditionally.
3. **try** block with 1 **except** clause.
4. Assigns valid_matches = [].
5. Assigns BUFFER_SECONDS = 2 * 24 * 3600.
6. **for** (eval_idx, matched_acc) in matches: iterates.
7. **if** not valid_matches: branches conditionally.
8. Assigns dated_valid = [m for m in valid_matches if m['valid_date']].
9. **if** dated_valid: branches conditionally (with an **else/elif** arm).
10. **return** match_result.

Code (copy-paste safe)

```
def filter_matches_by_date(matches, evaluations, trade_timestamp, phase_code=None, trade_number=None):
    """
    Filter matches to find the best matching evaluation.

    Universal logic for ALL prop firms:
    - When multiple rows match the same account, always prefer the LATEST row
      (highest eval_index = most recently added to the dashboard).
    - Date filtering is used as a secondary signal but never overrides the
      "latest row wins" rule.

    Args:
    matches: List of (eval_index, matched_account)
    evaluations: List of evaluation dicts
    trade_timestamp: Timestamp of the trade (float or int)
    phase_code: (Optional) Phase code to help determine target field for placeholder check
    trade_number: (Optional) Trade number for target field check
    """
    if not matches:
        return None

    # Universal: always prefer the latest row (highest eval_index)
    if not trade_timestamp:
        return max(matches, key=lambda m: m[0])

    # Convert trade timestamp to datetime
    try:
        val = float(trade_timestamp)
        trade_date = datetime.fromtimestamp(val)
    except (ValueError, TypeError):
        # If not a float, try isoformat string if present
        try:
            trade_date = datetime.fromisoformat(str(trade_timestamp).replace('Z', '+00:00'))
        except ValueError:
            return max(matches, key=lambda m: m[0])

    valid_matches = []

    # "give it a 2 day window from the date started"
    # Allow trades to be slightly BEFORE date started (e.g. timezone diffs)
    BUFFER_SECONDS = 2 * 24 * 3600 # 2 days
```

```

for eval_idx, matched_acc in matches:
    ev = evaluations[eval_idx]
    # "Date Purchased" is typically at index 2, but we address by name
    date_purchased_str = ev.get('Date Started', '') or ev.get('Date Purchased', '')
    date_purchased = parse_sheet_date(date_purchased_str)

    if not date_purchased:
        # If no date purchased, keep as fallback
        valid_matches.append({
            'match': (eval_idx, matched_acc),
            'delta': float('inf'),
            'valid_date': False,
            'start_date': 'None'
        })
        continue

    # Reset time to midnight for comparison
    dp_date = date_purchased.replace(hour=0, minute=0, second=0, microsecond=0)
    td_date = trade_date.replace(hour=0, minute=0, second=0, microsecond=0)

    raw_delta_seconds = (td_date - dp_date).total_seconds()

    # Check if Trade Date is within valid window relative to Start Date
    # Valid: Trade >= Start - 2 days
    if raw_delta_seconds >= -BUFFER_SECONDS:
        valid_matches.append({
            'match': (eval_idx, matched_acc),
            'delta': raw_delta_seconds,
            'valid_date': True,
            'start_date': dp_date.strftime('%Y-%m-%d')
        })

    if not valid_matches:
        # Fallback if no valid dates found – prefer latest row (highest eval_index)
        # Prefer the latest row (highest eval_index = most recently added account)
        return max(matches, key=lambda m: m[0])

    # When multiple valid matches exist, prefer the latest row (highest eval_index).
    # This ensures data always goes to the most recently added account on the dashboard.
    dated_valid = [m for m in valid_matches if m['valid_date']]
    if dated_valid:
        match_result = max(dated_valid, key=lambda m: m['match'][0])['match']
    else:
        match_result = max(valid_matches, key=lambda m: m['match'][0])['match']
    return match_result

```

```
def normalize_account_size(value)
```

Normalize account size values to standard format: \$X,XXX Handles: - "50k", "50K" → "\$50,000" - "100000", "100,000" → "\$100,000" - "\$50,000" → "\$50,000" (already correct) - "5000" → "\$5,000"

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Imports re (lazy import inside the function).
2. if not value: branches conditionally.
3. Assigns val = str(value).strip().upper().
4. if re.match('^\\\$[\\d,]+\$ ', val): branches conditionally.
5. Assigns match = re.match('^\\\$[\\d,]?(\\d+\\.?\\d*)K\$', val, re.IGNORECASE).
6. if match: branches conditionally.
7. Assigns val_clean = re.sub('[\\\$,\\s]', '', val).
8. try block with 1 except clause.
9. return value.

Code (copy-paste safe)

```
def normalize_account_size(value):
    """
    Normalize account size values to standard format: $X,XXX

    Handles:
    - "50k", "50K" → "$50,000"
    - "100000", "100,000" → "$100,000"
    - "$50,000" → "$50,000" (already correct)
    - "5000" → "$5,000"
    """
    import re
    if not value:
        return value

    val = str(value).strip().upper()

    # Already in correct format
    if re.match(r'^\\$[\\d,]+$ ', val):
        return value

    # Handle "50k" or "50K" format
    match = re.match(r'^\\$[\\d,]?(\\d+\\.?\\d*)K$', val, re.IGNORECASE)
    if match:
        num = float(match.group(1)) * 1000
        return f"${num:,.0f}"

    # Handle plain numbers like "50000" or "50,000"
    val_clean = re.sub(r'[\\$,\\s]', '', val)
    try:
        num = float(val_clean)
        return f"${num:,.0f}"
    except:
        pass

    # Return original if can't parse
    return value


def normalize_evaluations(evaluations)

    Normalize field values in evaluations list.
```

Step by step (top-level body)

1. **if** not evaluations: branches conditionally.
2. **for** ev in evaluations: iterates.
3. **return** evaluations.

Code (copy-paste safe)

```
def normalize_evaluations(evaluations):
    """Normalize field values in evaluations list."""
    if not evaluations:
        return evaluations

    for ev in evaluations:
        if 'Account Size' in ev and ev['Account Size']:
            ev['Account Size'] = normalize_account_size(ev['Account Size'])

    return evaluations
```

```
def _evaluation_row_merge_key(ev)
```

Stable identity for matching a pushed row to the same row in the DB. Index-based merge breaks when row order changes, when only a subset of rows is pushed (active trades / billing), or when new rows are inserted — which caused Activation Fee and other dashboard-owned fields to attach to the wrong evaluation or appear to "not persist".

Why these Python concepts were used

- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.

Step by step (top-level body)

1. **if** not isinstance(ev, dict): branches conditionally.
2. Assigns firm = (ev.get('Prop Firm') or "").strip().lower().replace(' ', ...)
3. Assigns a0 = (ev.get('Account #') or "").strip().lower().
4. Assigns a1 = (ev.get('Account #.1') or "").strip().lower().
5. Assigns sz = (ev.get('Account Size') or "").strip().lower().
6. Assigns accts = tuple(sorted((a for a in (a0, a1) if a))).
7. **if** not firm and (not accts): branches conditionally.
8. **return** (firm, accts, sz).

Code (copy-paste safe)

```
def _evaluation_row_merge_key(ev):
    """Stable identity for matching a pushed row to the same row in the DB.

    Index-based merge breaks when row order changes, when only a subset of rows
```

```

is pushed (active trades / billing), or when new rows are inserted – which
caused Activation Fee and other dashboard-owned fields to attach to the
wrong evaluation or appear to "not persist".
"""
if not isinstance(ev, dict):
    return None
firm = (ev.get('Prop Firm') or '').strip().lower().replace(' ', '').replace('_', '')
a0 = (ev.get('Account #') or '').strip().lower()
a1 = (ev.get('Account #.1') or '').strip().lower()
sz = (ev.get('Account Size') or '').strip().lower()
accts = tuple(sorted(a for a in (a0, a1) if a))
if not firm and not accts:
    return None
return (firm, accts, sz)

```

```
def _apply_dashboard_owned_merge(merged, base_row, incoming_row, force_fields)
```

Preserve dashboard-owned fields from base_row onto merged (mutates merged).

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

Step by step (top-level body)

1. **for** key in DASHBOARD_OWNED_EVAL_KEYS: iterates.

Code (copy-paste safe)

```

def _apply_dashboard_owned_merge(merged, base_row, incoming_row, force_fields):
    """Preserve dashboard-owned fields from base_row onto merged (mutates merged)."""
    for key in DASHBOARD_OWNED_EVAL_KEYS:
        if key in force_fields:
            if key in incoming_row:
                merged[key] = incoming_row[key]
            continue
        existing_val = base_row.get(key)
        if existing_val and str(existing_val).strip() not in ('', '-'):
            if key in ('Fee', 'Activation Fee'):
                try:
                    numeric = float(str(existing_val).replace('$', '').replace(',', '').strip())
                    if numeric == 0:
                        continue
                except (ValueError, TypeError):
                    pass
            merged[key] = existing_val

```

```
def merge_evaluation_push_with_existing(existing_evals, incoming_evals, force_fields=None)
```

Merge incoming evaluation rows into the full stored list by row identity. - Patches rows that match (firm + account numbers + size); never drops existing rows when the push only contains a subset (e.g. active trades). - Appends rows that have no match (new accounts from the sheet/trader).

Why these Python concepts were used

- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **if** not incoming_evals: branches conditionally.
2. Assigns force_fields = set(force_fields or []).
3. Assigns existing_evals = existing_evals or [].
4. Assigns incoming_evals = incoming_evals or [].
5. Assigns key_to_idx = {}.
6. **for** (i, ex) in enumerate(existing_evals): iterates.
7. Assigns out = [].
8. **for** ex in existing_evals: iterates.
9. Assigns appended = [].
10. **for** (i, ev_in) in enumerate(incoming_evals): iterates.
11. Calls out.extend(...) for its side effect.
12. **return** out.

Code (copy-paste safe)

```
def merge_evaluation_push_with_existing(existing_evals, incoming_evals, force_fields=None):
    """Merge incoming evaluation rows into the full stored list by row identity.

    - Patches rows that match (firm + account numbers + size); never drops
      existing rows when the push only contains a subset (e.g. active trades).
    - Appends rows that have no match (new accounts from the sheet/trader).
    """
    if not incoming_evals:
        return list(existing_evals) if existing_evals else []

    force_fields = set(force_fields or [])
    existing_evals = existing_evals or []
    incoming_evals = incoming_evals or []

    key_to_idx = {}
    for i, ex in enumerate(existing_evals):
        if not isinstance(ex, dict):
            continue
        mk = _evaluation_row_merge_key(ex)
        if mk and mk not in key_to_idx:
            key_to_idx[mk] = i

    out = []
    for ex in existing_evals:
        out.append(dict(ex) if isinstance(ex, dict) else ex)

    appended = []

    for i, ev_in in enumerate(incoming_evals):
```



```

    if not isinstance(ev_in, dict):
        continue
    mk = _evaluation_row_merge_key(ev_in)
    idx = key_to_idx.get(mk) if mk else None
    if idx is None and mk is None and i < len(out):
        idx = i
    if idx is not None and idx < len(out):
        base_snapshot = existing_evals[idx] if isinstance(existing_evals[idx], dict) else {}
        merged = dict(out[idx])
        merged.update(ev_in)
        _apply_dashboard_owned_merge(merged, base_snapshot, ev_in, force_fields)
        out[idx] = merged
    else:
        merged = dict(ev_in)
        _apply_dashboard_owned_merge(merged, {}, ev_in, force_fields)
        appended.append(merged)

out.extend(appended)
return out

```

```
def recalculate_hedge_nets(evaluations)
```

Recalculate Hedge Net and Hedge Net.1 for every evaluation row. Uses the same formulas as the Google Sheet import in data_processor.py so that values stay correct whenever hedge results or statuses change.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.

Step by step (top-level body)

1. Defines a nested function `_num(...)`.
2. Defines a nested function `_is_blank(...)`.
3. **for** `ev` in `evaluations` or `[]`: iterates.
4. **return** `evaluations`.

Code (copy-paste safe)

```

def recalculate_hedge_nets(evaluations):
    """Recalculate Hedge Net and Hedge Net.1 for every evaluation row.

    Uses the same formulas as the Google Sheet import in data_processor.py
    so that values stay correct whenever hedge results or statuses change.
    """

```

```

def _num(val):
    if val is None or str(val).strip() in ('', '-'):
        return 0.0
    try:
        return float(str(val).replace('$', '').replace(',', '').strip())
    except (ValueError, TypeError):
        return 0.0

def _is_blank(val):
    return val is None or str(val).strip() in ('', '-')

for ev in (evaluations or []):
    # --- Hedge Net (Phase 1) ---
    # =IF(OR(ISBLANK(HR1), Status P1<>"Fail"), "", -Fee + HR1+HR2+HR3+HR4+HR5)
    status_p1 = str(ev.get('Status P1', '')).strip()
    if _is_blank(ev.get('Hedge Result 1')) or status_p1 != 'Fail':
        ev['Hedge Net'] = ''
    else:
        fee = _num(ev.get('Fee'))
        hr_sum = sum(_num(ev.get(f'Hedge Result {i}')) for i in range(1, 6))
        ev['Hedge Net'] = -fee + hr_sum

    # --- Hedge Net.1 (Funded) ---
    status = str(ev.get('Status') or ev.get('Status Funded', '')).strip()
    sum_phase1 = sum(_num(ev.get(f'Hedge Result {i}')) for i in range(1, 6))
    sum_funded = sum(_num(ev.get(c)) for c in [
        'Hedge Result 1.1', 'Hedge Result 2.1', 'Hedge Result 3.1',
        'Hedge Result 4.1', 'Hedge Result 5.1', 'Hedge Result 6', 'Hedge Result 7',
    ])
    fee = _num(ev.get('Fee'))
    activation_fee = _num(ev.get('Activation Fee'))

    if status == 'Completed':
        sum_payouts = sum(_num(ev.get(f'Payout {i}')) for i in range(1, 7))
        sum_days = sum(_num(ev.get(f'Hedge Day {i}')) for i in range(1, 51))
        ev['Hedge Net.1'] = sum_payouts + sum_funded + sum_phase1 - fee - activation_fee + sum_days
    elif status == 'Fail':
        ev['Hedge Net.1'] = sum_funded + sum_phase1 - fee - activation_fee
    else:
        ev['Hedge Net.1'] = ''

return evaluations

```

```
def _match_tradovate_farming(tradovate_farming_days, fa_account_key)
```

Find Tradovate MNQ daily P&L matching a farming account. Args: tradovate_farming_days: List of {account_name, account_id, mnq_daily_pnl: [{date, net_pnl}]} fa_account_key: Account key from MT5 comment, e.g. 'FNFT-85625' Returns: Sorted list of {date, net_pnl} dicts, or None if no match.

Why these Python concepts were used

- **list comprehension** — Builds a list in a single expression ([f(x) for x in xs]). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.

Step by step (top-level body)

1. if not tradovate_farming_days or not fa_account_key: branches conditionally.
2. Assigns fa_key_upper = fa_account_key.upper().

3. Assigns `digits = [d for d in re.findall("\d+", fa_key_upper) if len(d) >= 4]`.

4. `for tv_data in tradovate_farming_days:` iterates.

5. `return None`.

Code (copy-paste safe)

```
def _match_tradovate_farming(tradovate_farming_days, fa_account_key):
    """Find Tradovate MNQ daily P&L matching a farming account.

    Args:
        tradovate_farming_days: List of {account_name, account_id, mnq_daily_pnl: [{date, net_pnl}]}
        fa_account_key: Account key from MT5 comment, e.g. 'FNFT-85625'

    Returns:
        Sorted list of {date, net_pnl} dicts, or None if no match.
    """
    if not tradovate_farming_days or not fa_account_key:
        return None

    fa_key_upper = fa_account_key.upper()
    # Extract digits (4+ chars) from the FA key for fallback matching
    digits = [d for d in re.findall(r'\d+', fa_key_upper) if len(d) >= 4]

    for tv_data in tradovate_farming_days:
        tv_name = (tv_data.get('account_name') or '').upper()
        if not tv_name:
            continue

        # Direct substring match (either direction)
        if fa_key_upper in tv_name or tv_name in fa_key_upper:
            return tv_data.get('mnq_daily_pnl', [])

        # Digit-based match: significant digit sequences from the FA key
        for d in digits:
            if d in tv_name:
                return tv_data.get('mnq_daily_pnl', [])

    return None


def get_field_name_for_phase(phase_code, trade_number, farming_date, evaluations, eval_idx,
                             account_number=None)
```

Determine the correct field name to update based on phase. Phase mappings: - CH1-5: Hedge Result 1-5 (Challenge) - FD (MFFU with FD0): FD0→Hedge Result 1, FD1→Hedge Result 2, etc. - FD (Other starting FD1): FD1→Hedge Result 1, FD2→Hedge Result 2, etc. - DD1-4: Additional funded hedge results - FA: Hedge Day N (based on date or next available slot)

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. if phase_code == 'CH': branches conditionally (with an **else/elif** arm).

2. **return** None.

Code (copy-paste safe)

```
def get_field_name_for_phase(phase_code, trade_number, farming_date, evaluations, eval_idx,
                             account_number=None):
    """
    Determine the correct field name to update based on phase.

    Phase mappings:
    - CH1-5: Hedge Result 1-5 (Challenge)
    - FD (MFFU with FD0): FD0→Hedge Result 1, FD1→Hedge Result 2, etc.
    - FD (Other starting FD1): FD1→Hedge Result 1, FD2→Hedge Result 2, etc.
    - DD1-4: Additional funded hedge results
    - FA: Hedge Day N (based on date or next available slot)
    """
    if phase_code == 'CH':
        # Challenge: CH1 → Hedge Result 1, CH2 → Hedge Result 2, etc.
        if trade_number is not None and 1 <= trade_number <= 5:
            return f"Hedge Result {trade_number}"

    elif phase_code == 'FD':
        # Funded: FD1→HR1.1, FD2→HR2.1, etc.
        # Some sources send FD0; treat FD0 the same as FD1 (map to HR1.1).
        if trade_number is not None:
            if trade_number == 0:
                return "Hedge Result 1.1"
            return f"Hedge Result {trade_number}.1"

    elif phase_code == 'DD':
        # Double Dip: DD1 maps to Hedge Result 1.1, DD2→2.1 etc
        if trade_number is not None:
            return f"Hedge Result {trade_number}.1"

    elif phase_code == 'FA':
        ev = evaluations[eval_idx] if (eval_idx is not None and evaluations and eval_idx < len(
            evaluations)) else {}
        incoming_date = str(farming_date or '').strip()

        logging.info(f"[FA SELECT] eval_idx={eval_idx} incoming_date={incoming_date}")

        # Reuse same slot if same date already exists
        for day_num in range(1, 51):
            date_key = f"_Hedge Day {day_num} Date"
            saved_date = str(ev.get(date_key, '')).strip()
            if saved_date and incoming_date and saved_date == incoming_date:
                logging.info(f"[FA SELECT] Reusing Hedge Day {day_num} for date {incoming_date}")
                return f"Hedge Day {day_num}"

        # Otherwise use first empty slot
        for day_num in range(1, 51):
            value_key = f"Hedge Day {day_num}"
            date_key = f"_Hedge Day {day_num} Date"

            existing_value = ev.get(value_key)
            existing_date = ev.get(date_key)

            is_empty_value = existing_value in (None, '', 0, '0', '$0', '$0.00')
            is_empty_date = existing_date in (None, '')
```

```

        if is_empty_value and is_empty_date:
            logging.info(f"[FA SELECT] Using empty slot Hedge Day {day_num} for date {incoming_date}")
            return value_key

        logging.warning(f"[FA SELECT] No empty slot found, defaulting to Hedge Day 1")
        return "Hedge Day 1"

    return None

def update_evaluations_from_aggregated_data(evaluations, aggregated_data=None, raw_deals=None,
    tradovate_farming_days=None)

```

Update evaluation hedge result fields from aggregated MT5 comment data OR raw deals. If 'raw_deals' is provided, performs server-side aggregation (Session Matching). Otherwise, uses client-provided 'aggregated_data'. Args: evaluations: List of evaluation records aggregated_data: List of aggregated trade data (from client) raw_deals: List of raw MT5 deal objects (from client) tradovate_farming_days: List of Tradovate MNQ daily P&L per account (for Prop Day values) Returns: Tuple of (updated_evaluations, match_log)

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **list comprehension** — Builds a list in a single expression ([f(x) for x in xs]). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **set comprehension** — Builds a set in one expression — usually to deduplicate the result of a transform.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.
- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns aggregated_data = aggregated_data or [].
2. Assigns tradovate_farming_days = tradovate_farming_days or [].
3. **if** raw_deals: branches conditionally.

4. Module/function docstring.
5. if not evaluations or not aggregated_data: branches conditionally.
6. Assigns match_log = [].
7. Assigns updates_made = 0.
8. Calls match_log.append(...) for its side effect.
9. Calls match_log.append(...) for its side effect.
10. Defines a nested function sort_key(...).
11. try block with 1 except clause.
12. Assigns fnft_latest_phase_map = {}.
13. for agg in aggregated_data: iterates.
14. Assigns filtered_data = [].
15. ... and 8 more statement(s) in the body.

Code (copy-paste safe)

```
def update_evaluations_from_aggregated_data(evaluations, aggregated_data=None, raw_deals=None,
tradovate_farming_days=None):
    """
    Update evaluation hedge result fields from aggregated MT5 comment data OR raw deals.

    If 'raw_deals' is provided, performs server-side aggregation (Session Matching).
    Otherwise, uses client-provided 'aggregated_data'.

    Args:
        evaluations: List of evaluation records
        aggregated_data: List of aggregated trade data (from client)
        raw_deals: List of raw MT5 deal objects (from client)
        tradovate_farming_days: List of Tradovate MNQ daily P&L per account (for Prop Day values)

    Returns:
        Tuple of (updated_evaluations, match_log)
    """
    aggregated_data = aggregated_data or []
    tradovate_farming_days = tradovate_farming_days or []

    # -----
    # SERVER-SIDE SESSION MATCHING LOGIC
    # -----
    if raw_deals:
        import datetime

        match_log = ["■ Using SERVER-SIDE Session Matching (ignoring client aggregation)"]

        # Helper: Parse simple comment patterns
        def parse_comment(c):
            if not c: return None, None
            # Standard Pattern: PRE...12345_CH1
            m = re.search(r'_(CH|FD|DD|FA)(\d+)?', c, re.IGNORECASE)
            if m:
                return m.group(1).upper(), int(m.group(2)) if m.group(2) else None
            return None, None

        def parse_full_comment_structure(c):
```

```

# Returns (PropPrefix, AccountNum, PhaseStr, PhaseNum)
# Example: MFFU...60076_FD1 -> ('MFFU', '60076', 'FD', 1)
# Example: V2-...4610_CH2 -> ('V2-', '4610', 'CH', 2)
# Example: FNFT...G8326_CH1 -> ('FNFT', 'G8326', 'CH', 1)
if not c: return None, None, None, None

# Regex for "PREFIX...ALPHANUM_PHASE"
# Support prefixes with digits and dashes (e.g. V2-)
# 1. Prefix: Letters, Digits, Dashes, min length 2
# 2. Filler: non-alphanumeric chars (dots, spaces, etc)
# 3. Account: Alphanumeric (Letters + Digits)
# 4. Underscore + Phase Code + Num
m = re.search(r'^([A-Z0-9\-\-]+)[^A-Z0-9]+([A-Z0-9]+)_(CH|FD|DD|FA)(\d+)?$', c.strip(),
re.IGNORECASE)
if m:
    return m.group(1).upper(), m.group(2).upper(), m.group(3).upper(), int(m.group(4)) if m.group(4) else 1
return None, None, None, None

def get_ts(d):
    t = d.get('time')
    if isinstance(t, (int, float)): return t
    if isinstance(t, str):
        try:
            return datetime.datetime.fromisoformat(t).timestamp()
        except:
            pass
    return 0

# Helper: Parse simple comment patterns to extract identifying account number
def extract_account_from_comment(c):
    if not c: return None

    # Known prefixes logic - moved to TOP priority
    known_prefixes = ['MFFU', 'AFAD', 'V2', 'FNFT', 'TDFY', 'ELTD', 'TDF']
    c_upper = c.upper()
    found_prefix = None
    for kp in known_prefixes:
        if kp in c_upper:
            found_prefix = kp
            break

    # If we see a known prefix, we want to capture that + the number (or alphanumeric code)
    if found_prefix:
        # Try to find the number/alphanum pattern specific to the prefix logic
        # For FNFT, account numbers can start with G?
        # Generally look for the part after dots and before _PHASE

        # Check for Structure: PREFIX...ACCOUNT_PHASE
        # Grab whatever is between ... and _
        # Use regex to find alphanumeric chars immediately preceding _PHASE
        m_phase = re.search(r'([A-Z0-9]+)_(CH|FD|DD|FA)', c, re.IGNORECASE)
        if m_phase:
            # This captures G8326 from ...G8326_CH1
            # But we want to prepend prefix if not present?
            extracted = m_phase.group(1)
            # If extracted matches digits, or starts with G, etc.
            # If found_prefix is already in extracted (e.g. FNFT12345), return extracted
            if found_prefix in extracted.upper():
                return extracted

```

```

        # Else return PREFIX-ACCOUNT
        return f"{found_prefix}-{extracted}"

    # Fallback: Just first number sequence if phase not found (shouldn't happen for valid com
    num_match = re.search(r'(\d+)', c)
    if num_match:
        return f"{found_prefix}-{num_match.group(1)}"

    # Look for alphanumeric string of length 4+ (including letters) before phase
    # This covers alphanumeric accounts like "MFFU12345"
    m = re.search(r'([A-Za-z0-9]{3,})_(CH|FD|DD|FA)', c, re.IGNORECASE)
    if m:
        val = m.group(1)
        # If it's just digits, fine
        if val.isdigit():
            return val

        # If it has letters, it might contain the prefix already in the group
        # e.g. MFFU12345
        return val

    # Fallback: Just look for digits before phase
    m = re.search(r'(\d+)_ (CH|FD|DD|FA)', c, re.IGNORECASE)
    if m:
        return m.group(1)
    return None

# 1. Group by Account Number (from login or comment)
# Note: raw_deals usually come from a single login, but might contain history for that login.

# --- NEW: Group Deals by Position ID FIRST ---
# The user requested "extract data from positions not deals".
# We synthesize "Position Objects" from raw deals by grouping on 'position_id'.
# This ensures that Exit deals (often comment-less) are linked to Entry deals (with comments).

position_map = {}
non_position_deals = []

# Initial pass to group
for d in raw_deals:
    pos_id = d.get('position_id')
    # Check if it's a trade deal (not balance/credit) and has valid position_id
    # Balance ops usually have position_id=0 or are distinct types
    d_type = str(d.get('type', '')).upper()
    is_balance = d_type in ['BALANCE', 'CREDIT', '2', '3', 'CHARGE', 'CORRECTION', 'BONUS']

    if not is_balance and pos_id and pos_id > 0:
        if pos_id not in position_map:
            position_map[pos_id] = []
        position_map[pos_id].append(d)
    else:
        non_position_deals.append(d)

# Synthesize Positions
synthesized_deals = []

# Add non-position deals – but skip internal transfers.
# Internal transfers are BALANCE-type deals with NO comment (no
# Tradovate account number). Positive balance resets (also no comment
# but profit > 0) are kept because they drive session splitting.

```



```

for _npd in non_position_deals:
    _npd_type = str(_npd.get('type', '')).upper()
    _npd_comment = str(_npd.get('comment', '')).strip()
    _npd_comment_l = _npd_comment.lower()
    _npd_profit = float(_npd.get('profit', 0))
    # Internal transfers can be tagged either way:
    # - no comment + non-positive profit (legacy heuristic), OR
    # - explicit 'internal transfer' comment regardless of sign.
    if _npd_type in ('BALANCE', '2'):
        if 'internal transfer' in _npd_comment_l:
            continue
        if not _npd_comment and _npd_profit <= 0:
            continue
    synthesized_deals.append(_npd)

for pos_id, p_deals in position_map.items():
    # Create a single 'deal' representing the whole position

    # 1. Find best comment (from entry usually)
    # Sort by time to find entry
    p_deals.sort(key=get_ts)

    common_comment = ""
    for pd in p_deals:
        c = pd.get('comment', '')
        if c and not common_comment:
            common_comment = c
        # Prefer comments that match our parser pattern
        if parse_comment(c)[0]:
            common_comment = c
            break

    # 2. Sum Profits
    total_profit = sum(float(pd.get('profit', 0)) + float(pd.get('commission', 0)) + float(pd.get(
        'swap', 0)) for pd in p_deals)

    # 3. Create Synthesized Object
    # Use Entry Time as the time for this position
    entry_time = get_ts(p_deals[0])

    syn_deal = {
        'time': entry_time,
        'profit': total_profit, # Net result of position
        'comment': common_comment,
        'type': 'POSITION',
        'position_id': pos_id,
        'deal_count': len(p_deals)
    }

    # --- FILTER UNKNOWN FORMATS ---
    # If we cannot extract an account number OR a valid phase from the comment,
    # this position is likely "Unknown" and should be ignored as per user request.
    # We use likelihood check: if extraction returns None, it's unknown format.
    ac_check = extract_account_from_comment(common_comment)
    ph_check, _ = parse_comment(common_comment)

    if not ac_check and not ph_check:
        # logging.debug(f"[FILTER] Skipping position {pos_id} due to unrecognized comment format")
        continue

    synthesized_deals.append(syn_deal)

```

```

# Replace raw_deals with our improved list
raw_deals = synthesized_deals
match_log.append(f"■ Grouped {len(position_map)} positions from raw deals for accurate P&L track")

# --- NEW DATE FILTERING LOGIC ---
# The user requested to only process trades from "today" (active day)
# OR if no trades today, process trades from the "last active day".
if raw_deals:
    # Helper to extract YYYY-MM-DD from timestamp (assuming timestamp is seconds)
    # syn_deal['time'] is already a timestamp float/int
    def get_date_str(ts):
        try:
            if isinstance(ts, str):
                # Try parsing ISO string
                dt = datetime.datetime.fromisoformat(ts)
                return dt.strftime('%Y-%m-%d')
            return datetime.datetime.fromtimestamp(float(ts)).strftime('%Y-%m-%d')
        except Exception as e:
            logging.warning(f"Failed to parse timestamp {ts}: {e}")
            return "1970-01-01"

    # Count FA deals for logging
    fa_count = sum(1 for d in raw_deals if parse_comment(d.get('comment', ''))[0] == 'FA')
    if fa_count:
        match_log.append(f"■ Found {fa_count} farming deals in data")

# --- PRE-COMPUTE FARMING DAILY PROFITS PER ACCOUNT (before date filter) ---
# Scan ALL FA deals to build: {account_num: {date_str: total_profit}}
# This lets us know how many farming days exist for each account
# and which Hedge Day slot each date maps to.
fa_account_days = {}
for _d in raw_deals:
    _phase, _num = parse_comment(_d.get('comment', ''))
    if _phase != 'FA':
        continue
    _acc = extract_account_from_comment(_d.get('comment', ''))
    if not _acc:
        continue
    _ts = get_ts(_d)
    _date = datetime.datetime.fromtimestamp(_ts).strftime('%Y-%m-%d')
    _profit = float(_d.get('profit', 0)) + float(_d.get('commission', 0)) + float(_d.get('swap', 0))
    if _acc not in fa_account_days:
        fa_account_days[_acc] = {}
    fa_account_days[_acc][_date] = fa_account_days[_acc].get(_date, 0.0) + _profit

if fa_account_days:
    for _acc, _days in fa_account_days.items():
        logging.info(
            f"[FA PRE-COMPUTE] account={_acc} farming_days={len(_days)} dates={sorted(_days.keys())}"
        )
    match_log.append(
        f"■ Pre-computed farming: {len(fa_account_days)} account(s), {sum(len(d) for d in fa_account_days.values())} days"
    )

# Track which evals have already had their Hedge Days written by FA pre-compute
fa_evals_written = set()

# Identify all unique dates in the data for logging
unique_dates = sorted(list({get_date_str(d['time']) for d in raw_deals})) if raw_deals else []

# Date filtering is now handled CLIENT-SIDE (push from 23rd of last month,

```

```

# FNFT challenge 24h filter). Server processes ALL deals received.
if unique_dates:
    match_log.append(
        f"■ Processing deals across {len(unique_dates)} date(s): {unique_dates[0]} to {unique_dates[-1]}"
    )
    logging.info(
        f"    [DATES] Processing all {len(raw_deals)} deals across {len(unique_dates)} date(s)"
    )

# -----

# -----

# Let's assume passed 'aggregated_data' logic was doing groupings.
# We will iterate raw_deals, extracting (Phase, TradeNum) from comment.
# And Group by Time Gaps (> 7 days) -> New Session.

raw_deals.sort(key=get_ts)

# Group into sessions by Time Gap (24h) OR Phase Change
sessions = []

# Helper to init session
def new_session_dict(start_t, acc_guess=None):
    return {
        'deals': [],
        'start': start_t,
        'end': start_t,
        'account_guess': acc_guess,
        'phase_guess': None,
        'phase_num': None
    }

# Initialize with first deal info
first_deal = raw_deals[0]
start_ts = get_ts(first_deal)

if len(raw_deals) > 0:
    logging.info(f"[DEBUG] First 20 Raw Comments: {[d.get('comment') for d in raw_deals[:20]]}")

# Extract initial account guess
first_acc_guess = None
for d in raw_deals:
    guess = extract_account_from_comment(d.get('comment', ''))
    if guess:
        first_acc_guess = guess
        break

current_session = new_session_dict(start_ts, first_acc_guess)
last_ts = start_ts

# Initial phase guess
p_init, n_init = parse_comment(first_deal.get('comment', ''))
if p_init:
    current_session['phase_guess'] = p_init
    current_session['phase_num'] = n_init

for d in raw_deals:
    ts = get_ts(d)
    p, n = parse_comment(d.get('comment', ''))

    # Check for Phase Change

```

```

is_phase_change = False
if p and current_session['phase_guess']:
    if p != current_session['phase_guess'] or n != current_session['phase_num']:
        is_phase_change = True

# Check for Account Change
is_account_change = False
current_account_guess = extract_account_from_comment(d.get('comment', ''))

# Only trigger change if we had a guess and the new guess is DEFINITELY different
if current_account_guess and current_session['account_guess']:
    if current_account_guess != current_session['account_guess']:
        is_account_change = True
        logging.debug(
            f"[SPLIT] Account changed from {current_session['account_guess']} to {current_acc_guess}"
        )

# Check Time Gap (36 hours) - lowered from 7 days
time_gap = (ts - last_ts) > (36 * 3600)

# Balance Reset
is_balance_reset = str(d.get('type', '')).upper() == 'BALANCE' and float(d.get('profit', 0)) > 0

# Split Session Logic (Phase, Time, Balance, Account)
should_split = (time_gap or is_balance_reset or is_phase_change or is_account_change)

if should_split and current_session['deals']:
    sessions.append(current_session)

    # Determine next account guess for the new session
    next_acc_guess = current_account_guess if current_account_guess else current_session['account_guess']

    # Create new session
    current_session = new_session_dict(ts, next_acc_guess)

    if p:
        current_session['phase_guess'] = p
        current_session['phase_num'] = n

current_session['deals'].append(d)
current_session['end'] = max(current_session['end'], ts)
last_ts = ts

# Update tracking
if p and not current_session['phase_guess']:
    current_session['phase_guess'] = p
    current_session['phase_num'] = n

# If we don't have an account guess yet (or it's None), see if this comment has one
if current_account_guess and not current_session['account_guess']:
    current_session['account_guess'] = current_account_guess

if current_session['deals']:
    sessions.append(current_session)

match_log.append(f" Found {len(sessions)} distinct sessions based on Phase/Time gaps")

# --- NEW: Summary Aggregation Structure ---
# { PropFirm: { AccountNumber: { PhaseKey: { profit: 0.0, trades: 0 } } } }
trade_summary = {}

```

```

# -----

# Now match each session to an Evaluation
updates_made = 0

for session in sessions:
    if not session['account_guess']:
        logging.warning(f"Session without account guess skipped. Deals: {len(session['deals'])}")
        continue # Can't match without account number

    logging.info(
        f"[DEBUG] Processing Session: AccountGuess={session['account_guess']}, Deals={len(session['deals'])}"
    )

    # Aggregate stats for session – exclude internal transfers.
    # Internal transfers are BALANCE-type deals with no comment (no
    # Tradovate account number). Real trades always have comments.
    def _is_internal_transfer(d):
        dt = str(d.get('type', '')).upper()
        comment = str(d.get('comment', '')).strip()
        comment_l = comment.lower()
        if dt not in ('BALANCE', 'CREDIT', '2', '3', 'CHARGE', 'CORRECTION', 'BONUS'):
            return False
        # Either no comment (legacy) or an explicit "internal transfer" tag.
        return (not comment) or ('internal transfer' in comment_l)
    trade_deals_only = [d for d in session['deals'] if not _is_internal_transfer(d)]
    session_profit = sum(float(d.get('profit', 0)) + float(d.get('commission', 0)) + float(d.get('swap', 0)) for d in trade_deals_only)

    # Determine Phase/TradeNum from MOST frequent in session
    # (To handle noise)
    phases = {}
    full_comment_info = None

    for d in session['deals']:
        # Try full structure parse first
        prefix, acc_part, p, n = parse_full_comment_structure(d.get('comment', ''))
        if prefix and acc_part:
            full_comment_info = (prefix, acc_part, p, n)

        # Fallback to simple parse for counting
        p_simple, n_simple = parse_comment(d.get('comment', ''))
        if p_simple:
            key = (p_simple, n_simple)
            phases[key] = phases.get(key, 0) + 1

    if not phases:
        continue

    best_phase, best_num = max(phases.items(), key=lambda x: x[1])[0]
    logging.info(
        f"[SESSION] account_guess={session['account_guess']} "
        f"best_phase={best_phase} best_num={best_num} "
        f"start={datetime.datetime.fromtimestamp(session['start'])} "
        f"end={datetime.datetime.fromtimestamp(session['end'])} "
        f"profit={session_profit}"
    )

    # If we found a full comment structure, prefer that for matching

```

```

if full_comment_info:
    target_prefix, target_acc_part, target_phase, target_num = full_comment_info
    # Use the derived phase/num from the full comment if available, or fallback to frequency
    if target_phase:
        best_phase = target_phase
    if target_num:
        best_num = target_num

    acc_num = target_acc_part # Use extracted regex digits as the account number to match

    # NEW: If acc_num is purely digits, but we found a PREFIX, try to append it?
    # Or better: Check if appending prefix helps match against verbose DB accounts?
    # e.g. TDFY-72031 might be better search key than 72031 if DB is verbose?
    # But our matching logic 's_acc in ac1' works better with SHORTER s_acc.
    # So keeps '72031'.
else:
    acc_num = session['account_guess']

start_date_ts = session['start']

# Find candidate evaluations
# Check match against BOTH Account # and Account #.1

def normalize_acc(a): return str(a).strip().upper()

# Strip prefix from s_acc if we are relying on substring matching?
# If s_acc is "TDFY-72031", and DB has "TDFYSL...72031", "TDFY-72031" is NOT in it.
# But "72031" IS in it.
# So we should probably strip known prefixes from s_acc before matching loop if we want flexi

s_acc_raw = normalize_acc(acc_num)

# Preserve the comment prefix (e.g. 'V2-', 'MFFU', 'FNFT') for firm validation
# even when acc_num itself has no hyphen (full_comment_info strips prefix from acc_part)
comment_prefix = None
if full_comment_info and full_comment_info[0]:
    comment_prefix = full_comment_info[0].rstrip('-').upper()

# If s_acc contains a hyphen, try splitting
if '-' in s_acc_raw:
    # Keep full for strict check, but try short version for loose check?
    s_acc_short = s_acc_raw.split('-')[-1]
else:
    s_acc_short = s_acc_raw

# Use the SHORTER version for candidate finding to maximize hits
s_search = s_acc_short

candidates = []

# Determines if we have a strict structural match up front
matches_full_strict = (full_comment_info is not None)

for e in evaluations:
    is_match = False
    ac1 = normalize_acc(e.get('Account #', ''))
    ac2 = normalize_acc(e.get('Account #.1', ''))

    # Strip Top Step prefixes (50KTC- for challenge, EXPRESS- for funded) before matching
    for _ts_prefix in ['50KTC-', 'EXPRESS-']:

```

```

        if ac1.startswith(_ts_prefix): ac1 = ac1[len(_ts_prefix):]
        if ac2.startswith(_ts_prefix): ac2 = ac2[len(_ts_prefix):]

# Use ENDING digits matching only – never substring containment
s = s_search

# Extract trailing digits for comparison
import re as _re
s_digits = _re.search(r'(\d+)$', s)
s_trail = s_digits.group(1) if s_digits else s

# Check ac1
if ac1:
    ac1_digits = _re.search(r'(\d+)$', ac1)
    ac1_trail = ac1_digits.group(1) if ac1_digits else ac1
    if s == ac1:
        is_match = True
    elif ac1_trail.endswith(s_trail) or s_trail.endswith(ac1_trail):
        is_match = True

# Check ac2
if not is_match and ac2:
    ac2_digits = _re.search(r'(\d+)$', ac2)
    ac2_trail = ac2_digits.group(1) if ac2_digits else ac2
    if s == ac2:
        is_match = True
    elif ac2_trail.endswith(s_trail) or s_trail.endswith(ac2_trail):
        is_match = True

# STRICT PREFIX CHECK – use comment_prefix (from full_comment_info) OR s_acc_raw hyphen p
prefix_part = comment_prefix # e.g. 'V2' from V2-1128_CH2
if not prefix_part and '-' in s_acc_raw:
    prefix_part = s_acc_raw.split('-')[0]

try:
    if is_match and prefix_part:
        pf_val = str(e.get('Prop Firm', '')).upper()

        # Only proceed if we have a valid Prop Firm string to check against
        if pf_val and prefix_part not in pf_val and pf_val not in prefix_part:
            mapping = {
                'MFFU': ['MYFUNDED', 'MFFU', 'MY FUNDED'],
                'AFAD': ['ALPHA', 'AFAD'],
                'V2': ['TOPSTEP', 'TOP STEP', 'V2'],
                'FNFT': ['FUNDEDNEXT', 'FUNDED NEXT', 'FNFT'],
                'TDFY': ['TRADEIFY', 'TDFY'],
                'ELTD': ['TRADEDAY', 'ELTD'],
                'TDF': ['TRADEDAY', 'TDF', 'TRADEIFY'],
                'FTDF': ['TRADEDAY', 'TDF', 'TRADEIFY'],
                'TPOF': ['TOPONEFUTURES', 'TOP ONE', 'TPOF']
            }
            if prefix_part in mapping:
                valid_keywords = mapping[prefix_part]
                if not any(k in pf_val for k in valid_keywords):
                    is_match = False
except Exception as ex:
    logging.error(f"Error in Strict Prefix Check for {s_acc_raw} prefix={prefix_part}: {ex}")

if matches_full_strict and is_match:
    pass

```

```

        if is_match:
            candidates.append(e)

    if not candidates:
        match_log.append(
            f"■■■ No evaluation found for session {acc_num} (Start: {str(datetime.datetime.fromti
        continue

    # Filter by Date Purchased
    # We want Evaluation.DatePurchased <= Session.Start
    # And closest to it
    valid_candidates = []

    # STRICT MATCH OVERRIDE: If we have a perfectly parsed comment (Structure: PREFIX...ACC_PHASE
    # and we found matching accounts, we TRUST the account number match primarily.
    # Skip date filtering for ALL firms with a structural match – latest row always wins.
    skip_date_filter = matches_full_strict

    for e in candidates:
        if skip_date_filter:
            # Trust the account match implicitly (all matches are suffix-based)
            valid_candidates.append((e, 0))
            continue

        dp_str = str(e.get('Date Started', '')) or str(e.get('Date Purchased', ''))
        try:
            # Try common formats
            dp_dt = None
            for fmt in ["%m/%d/%y", "%Y-%m-%d", "%m/%d/%Y"]:
                try:
                    dp_dt = datetime.datetime.strptime(dp_str, fmt)
                    break
                except: pass

            if dp_dt:
                diff = start_date_ts - dp_dt.timestamp()
                # Allow session to start slightly before purchase? (Maybe same day timezone diff?
                # Allow -24h slack.
                # BUFFER: If strict comment match, assume it's correct even if dates are weird?
                # But typically we still want the LATEST one if duplications exist.
                # Let's keep date logic but maybe relax it for strict matches?
                if diff > -86400 * 2: # 48 hours buffer
                    valid_candidates.append((e, diff))
            else:
                # If no date found, but we have a STRICT digit match?
                # Keep it as a candidate with high drift (unless only one candidate)
                valid_candidates.append((e, float('inf')))
        except:
            pass

    # Decision Time
    if not valid_candidates:
        # If we had a strict comment match, and no valid dates, maybe just pick the best text mat
        if matches_full_strict and candidates:
            # Always prefer the latest row (highest eval_index)
            best_eval = max(candidates, key=lambda e: evaluations.index(e))
            match_log.append(
                f"■■■ Date mismatch but strict ID match for {acc_num}, using latest row.")
        else:
            match_log.append(f"■■■ No valid date match for {acc_num}")

```



```

        continue
    else:
        # Always prefer the latest row (highest eval_index = most recently added)
        # regardless of date proximity – the bigger the row number, the more recent the account
        best_eval = max(
            [vc[0] for vc in valid_candidates],
            key=lambda e: evaluations.index(e)
        )
        drift = next(vc[1] for vc in valid_candidates if vc[0] is best_eval)

        logging.info(
            f"[MATCHED EVAL] eval_idx={evaluations.index(best_eval)} "
            f"account={acc_num} phase={best_phase} num={best_num} drift={drift}"
        )

    # Determined Field Name
    field_name = "Hedge Result" # Default

    # Use parsed phase/num from strict match if available
    if matches_full_strict:
        _, _, best_phase, best_num = full_comment_info

    if best_phase == 'CH':
        # CH1 -> Hedge Result 1
        # CH2 -> Hedge Result 2
        if best_num and best_num > 1:
            field_name = f"Hedge Result {best_num}"
        else:
            field_name = "Hedge Result 1" # Default to 'Hedge Result 1' to match sheet headers

    elif best_phase == 'FD':
        # FD1 -> Hedge Result 1.1? Or just Hedge Result?
        # User provided logic: "last part is the cell to put the data in" (CH2 -> HR2)
        # Wait, for FD, existing logic says "Hedge Result 1.1" usually.
        # Let's check get_field_name_for_phase implementation again if possible, or replicate:
        if best_num is not None:
            # MFFU logic: FD0->1.1, FD1->2.1? Or FD1->1.1?
            # Standard implementation elsewhere:
            # FD -> 1.1 usually means "Funded Account 1"
            # Let's assume matches 1:1 if possible?
            # Logic in get_field_name_for_phase (read earlier):
            # "MFFU accounts: FD0->HR1.1, FD1->HR2.1"
            # "Other: FD0/1 -> HR1.1"

            # Re-implement simplified version here:
            normalized_prefix = (target_prefix or '').upper()
            if 'MFFU' in normalized_prefix:
                # MFFU Logic
                # FD0 -> 1.1 ?? Wait, previous code said FD0->1.1, FD1->2.1
                # Let's safer assume user wants mapped to *.1
                # Map FD1 -> Hedge Result 1.1
                # FD2 -> Hedge Result 2.1
                if best_num == 0: field_name = "Hedge Result 1.1" # FD0 match
                else: field_name = f"Hedge Result {best_num}.1"
            else:
                # Standard FD
                field_name = f"Hedge Result {best_num}.1"

        elif best_phase == 'DD':
            field_name = f"Hedge Result {best_num}.1"

```

```

elif best_phase == 'FA':
    # --- FARMING: Count farming days from MT5 history, only update the LAST one ---
    # fa_account_days has ALL farming dates per account from full MT5 history.
    # Count distinct dates to know which Hedge Day slot the latest trade belongs to.
    # e.g. 5 farming days total → only write Hedge Day 5 with the latest day's profit.
    best_eval_idx = evaluations.index(best_eval)
    row_num = best_eval_idx + 2

    # --- Active account check: skip inactive evals for farming ---
    _s_p1 = str(best_eval.get('Status P1', '')).lower()
    _s_funded = str(best_eval.get('Status', '') or best_eval.get('Status Funded', '')).lower()
    _inactive_kw = ('fail', 'breach', 'sl', 'closed', 'delete')
    _is_inactive = (
        any(kw in _s_p1 for kw in _inactive_kw) or
        any(kw in _s_funded for kw in (*_inactive_kw, 'complete'))
    )
    if _is_inactive:
        match_log.append(
            f"    ■ Skipping FA for inactive eval row {row_num} (P1='{_s_p1}', Funded='{_s_funded}')"
        )
        logging.info(
            f"[FA SKIP] eval_idx={best_eval_idx} inactive – P1='{_s_p1}' Funded='{_s_funded}'"
        )
        continue

    # Only write once per eval
    if best_eval_idx in fa_evals_written:
        continue
    fa_evals_written.add(best_eval_idx)

    # Get this account's farming data from the pre-computed map
    # Pre-compute keys have prefix (e.g. "FNFT-76770"), but acc_num may be just digits ("76770")
    # Try exact match first, then substring fallback
    account_days = fa_account_days.get(acc_num, {})
    if not account_days:
        # Substring match: find key where acc_num appears in it or it appears in acc_num
        for _fa_key, _fa_days in fa_account_days.items():
            if acc_num in _fa_key or _fa_key in acc_num:
                account_days = _fa_days
                logging.info(
                    f"[FA WRITE] Matched acc_num={acc_num} to pre-computed key={_fa_key}"
                )
                break
    if not account_days:
        logging.warning(
            f"[FA WRITE] No pre-computed farming data for account {acc_num} (eval_idx={best_eval_idx})"
        )
        continue

    # Sort dates chronologically – position = Hedge Day number
    sorted_dates = sorted(account_days.keys())
    total_farming_days = len(sorted_dates)
    last_date = sorted_dates[-1]
    last_profit = account_days[last_date]
    slot = total_farming_days # e.g. 5 dates → Hedge Day 5

    if slot > 50:
        logging.warning(
            f"[FA WRITE] eval_idx={best_eval_idx} has {slot} farming days, capping at 50"
        )
        slot = 50

    field_name = f'Hedge Day {slot}'

    best_eval[field_name] = f'{last_profit:.2f}'

```

```

best_eval[f'_{field_name} Date'] = last_date
updates_made += 1

match_log.append(
    f"■ ■ Row {row_num} | {field_name}: ${last_profit:.2f} ({last_date}) [day {slot} of
logging.info(
    f"[FA WRITE] row={row_num} account={acc_num} "
    f"total_farming_days={total_farming_days} → {field_name}=${last_profit:.2f} date={la
)

# --- Write Prop Day values from Tradovate MNQ daily P&L ---
if tradovate_farming_days:
    tv_mnq_days = _match_tradovate_farming(tradovate_farming_days, acc_num)
    if tv_mnq_days:
        prop_days_written = 0
        for day_idx, tv_day in enumerate(tv_mnq_days):
            prop_slot = day_idx + 1
            if prop_slot > 50:
                break
            best_eval[f'Prop Day {prop_slot}'] = f"{tv_day['net_pnl']:.2f}"
            prop_days_written += 1
        if prop_days_written:
            match_log.append(
                f"■ ■ Row {row_num} | Prop Days 1-{prop_days_written}: "
                f"Tradovate MNQ P&L written"
            )
            logging.info(
                f"[FA PROP DAY] row={row_num} account={acc_num} "
                f"wrote {prop_days_written} Prop Day value(s) from Tradovate MNQ"
            )
        else:
            logging.info(f"[FA PROP DAY] No matching Tradovate MNQ data for account {acc_num}")

# --- Also add to trade_summary for logging ---
try:
    p_firm = best_eval.get('Prop Firm') or 'Unknown Firm'
    report_acc = best_eval.get('Account #') or best_eval.get('Account #.1') or acc_num
    phase_label = f"FA{best_num or ''}"
    if p_firm not in trade_summary: trade_summary[p_firm] = {}
    if report_acc not in trade_summary[p_firm]: trade_summary[p_firm][report_acc] = {}
    if phase_label not in trade_summary[p_firm][report_acc]:
        trade_summary[p_firm][report_acc][phase_label] = {
            'profit': 0.0, 'trades': 0, 'source_accounts': set(),
            'comments': set(), 'target_field': field_name,
            'detailed_trades': [], 'row_index': row_num
        }
    sn = trade_summary[p_firm][report_acc][phase_label]
    # Use the pre-computed last-date profit (what was actually written),
    # NOT the session profit (which may come from the earliest session)
    sn['profit'] = float(last_profit)
    sn['trades'] = total_farming_days
    sn['source_accounts'].add(str(acc_num))
    sn['target_field'] = field_name
    # Find the actual trade timestamp for the last farming date
    _last_ts_str = last_date
    for _fd in reversed(raw_deals):
        _fp, _ = parse_comment(_fd.get('comment', ''))
        if _fp == 'FA':
            _fa_acc = extract_account_from_comment(_fd.get('comment', ''))
            if _fa_acc and (acc_num in _fa_acc or _fa_acc in acc_num):

```

```

        _fts = get_ts(_fd)
        _fdate = datetime.datetime.fromtimestamp(_fts).strftime(
            '%Y-%m-%d') if _fts > 0 else ''
        if _fdate == last_date:
            _last_ts_str = datetime.datetime.fromtimestamp(_fts).strftime(
                '%Y-%m-%d %H:%M:%S')
            break
        sn['detailed_trades'] = [
            f"[{_last_ts_str}] [{session['deals']}[0].get('comment', '')}] {acc_num} -> ${last_val}"
    except Exception as e:
        logging.error(f"Error accumulating FA stats: {e}")

    continue # Skip normal write logic – FA is handled above

# Update Logic: ACCUMULATE profit for this push
# Since we are processing all history in this push, we should sum up sessions for the same eval
# But be careful not to sum with existing OLD values from DB if we are replacing them?
# Actually, update_evaluations... is called on the existing 'evaluations' list from DB.
# If we just add, we might double count if we run this multiple times?
# No, 'evaluations' object is transient for this request (loaded from DB/Sheet).
# The 'session_profit' is from the NEW deals being pushed.
# Users usually push ALL history.
# So we should probably clear the field first if it's the first time we touch it IN THIS REQUEST
# Or just assume we are recalculating from scratch for these deals.

# For robustness: valid numeric check (handle $ formatting)
try:
    raw_val = str(best_eval.get(field_name, 0) or 0)
    clean_val = raw_val.replace('$', '').replace(',', '').strip()
    current_val = float(clean_val) if clean_val else 0.0
except:
    current_val = 0.0

# SPECIAL CASE: FundedNext (or others) where multiple evals share account?
# The user said: "one account number belongs to several prop firm evaluations, this only happens once"
# "The rest should just push directly"
# If we touch the same field multiple times in this loop (multiple sessions for same eval), we should sum them
# If the eval field already has value from previous pushes (DB), and we are pushing partial calculation, we should add
# Usually pushes contain full history.
# Let's assume we accumulate.

# However, if this is the FIRST time we are updating this specific field IN THIS BATCH,
# and we want to overwrite old DB data with new calculation?
# The 'evaluations' list comes from DB.
# If we want to replace the old 'Hedge Result' with the new value from this push,
# We should probably track which fields we've updated.

# Key to track uniqueness: (Eval Index, Field Name)
update_key = (candidates.index(best_eval) if best_eval in candidates else -1, field_name)

# This is tricky because we don't have unique IDs easily accessible here without strict indexing
# Let's rely on the object identity `id(best_eval)`

eval_id = id(best_eval)

# Simple aggregation logic: If it's the first time we see this field for this eval in THIS REQUEST,
# we overwrite it (assuming full push). Subsequent sessions add to it.
if '_updated_fields' not in best_eval:
    best_eval['_updated_fields'] = set()

```

```

if field_name not in best_eval['_updated_fields']:
    new_val = session_profit
    best_eval['_updated_fields'].add(field_name)
else:
    new_val = current_val + session_profit

# Store clean numeric value (dashboard JS adds $ at display time)
best_eval[field_name] = f"{new_val:.2f}"

updates_made += 1
if 'Match Log' not in best_eval:
    best_eval['Match Log'] = []
best_eval['Match Log'].append(
    f"Matched matched session (start {datetime.datetime.fromtimestamp(start_date_ts)}) -> {f}"

# Add explicit cell confirmation log
current_row_idx = evaluations.index(best_eval) + 2
match_log.append(
    f"■ Matched session (Start {datetime.datetime.fromtimestamp(start_date_ts)}) -> Column:"

# --- AGGREGATE SUMMARY STATS ---
try:
    # 1. Prop Firm
    # Uses col "Prop Firm" if exists, or guess from prefix
    p_firm = best_eval.get('Prop Firm')
    if not p_firm:
        # Try to guess from account number prefix or comments
        guesser = normalize_acc(acc_num)
        # Also check comments in session for clues if account number is ambiguous
        session_comments = " ".join([d.get('comment', '') for d in session['deals'][:5]])

        if 'MFF' in guesser or 'MFF' in session_comments.upper(): p_firm = 'My Funded Future'
        elif 'V2' in guesser or 'V2' in session_comments.upper(): p_firm = 'Topstep'
        elif 'FN' in guesser or 'FNFT' in session_comments.upper(): p_firm = 'FundedNext'
        elif 'AF' in guesser or 'AFAD' in session_comments.upper(): p_firm = 'Alpha Futures'
        else: p_firm = 'Unknown Firm'

    # 2. Account Number (Use the one from the evaluation record if possible for consistency)
    # "Account #" for Challenge or "Account #.1" for Funded
    # Or just use the Evaluation Index/Name to be clearer?
    # User asked for "Account Number"
    report_acc = best_eval.get('Account #') or best_eval.get('Account #.1') or acc_num

    # 3. Phase Key (e.g. CH1, FD1)
    phase_label = f"{best_phase}{best_num or ''}"

    # InitDicts
    if p_firm not in trade_summary: trade_summary[p_firm] = {}
    if report_acc not in trade_summary[p_firm]: trade_summary[p_firm][report_acc] = {}
    if phase_label not in trade_summary[p_firm][report_acc]:
        trade_summary[p_firm][report_acc][phase_label] = {
            'profit': 0.0,
            'trades': 0,
            'source_accounts': set(), # The raw account number from mt5/comment
            'comments': set(),       # Sample comments used for classification
            'target_field': field_name,
            'detailed_trades': [],   # List of specific trade details
            'row_index': -1          # Will be set below
        }

    # Add

```

```

summary_node = trade_summary[p_firm][report_acc][phase_label]

# Find row index (Add 2 because Sheet usually has headers, Python list is 0-indexed)
try:
    row_idx = evaluations.index(best_eval) + 2
    summary_node['row_index'] = row_idx
except:
    summary_node['row_index'] = '??'

summary_node['profit'] += float(session_profit)

# Count only actual trade deals (exclude internal transfers)
_session_trade_deals = [d for d in session['deals'] if not _is_internal_transfer(d)]
summary_node['trades'] += len(_session_trade_deals)
summary_node['source_accounts'].add(str(acc_num))

# Add detailed trade info (only real trades)
for d in _session_trade_deals:
    t_profit = float(d.get('profit', 0))
    t_comment = d.get('comment', '')
    t_acc = extract_account_from_comment(t_comment) or "N/A"

    # Format timestamp
    t_ts = get_ts(d)
    t_time_str = "Unknown Time"
    if t_ts > 0:
        t_time_str = datetime.datetime.fromtimestamp(t_ts).strftime('%Y-%m-%d %H:%M:%S')

    # Add to list (limit size if needed, but user asked for detail)
    summary_node['detailed_trades'].append(
        f"[{t_time_str}] [{t_comment}] {t_acc} -> ${t_profit:.2f}")

# Add sample comments (limit to 3 unique ones per phase to avoid spam)
current_comments = [d.get('comment', '') for d in session['deals'] if d.get('comment')]
for c in current_comments:
    if len(summary_node['comments']) < 3:
        summary_node['comments'].add(c)

except Exception as e:
    logging.error(f"Error accumulating stats: {e}")
# -----

# CLEANUP: Remove temporary tracking fields before returning
for ev in evaluations:
    if '_updated_fields' in ev:
        del ev['_updated_fields']
    if 'Match Log' in ev:
        del ev['Match Log']

# --- LOG SUMMARY ---
# "Group for me all trades under a specific propfirm, like topstep 2 trades etc, break it even fu
logging.info("=*30)
logging.info(" TRADE PROCESSING SUMMARY")
logging.info("=*30)

# Sort firms
sorted_firms = sorted(trade_summary.keys())
for firm in sorted_firms:

```

```

    firm_data = trade_summary[firm]
    # Total trades for firm
    total_firm_trades = 0
    for acc_data in firm_data.values():
        for stats in acc_data.values():
            total_firm_trades += stats['trades']

    logging.info(f"■ {firm} ({total_firm_trades} trades)")

    sorted_accs = sorted(firm_data.keys(), key=lambda x: str(x))
    for acc in sorted_accs:
        acc_data = firm_data[acc]
        # Total trades for account
        total_acc_trades = sum(item['trades'] for item in acc_data.values())
        logging.info(f" ■■■ ■ Dashboard Account: {acc} ({total_acc_trades} trades)")

        sorted_phases = sorted(acc_data.keys())
        for phase in sorted_phases:
            stats = acc_data[phase]
            profit_str = f"${stats['profit']:, .2f}"
            trades_count = stats['trades']
            target_field = stats.get('target_field', 'Unknown')

            # Format comments and source accounts
            sources_str = ", ".join(sorted(list(stats['source_accounts'])))

            row_idx = stats.get('row_index', '??')

            logging.info(f" ■■■ ■ Phase {phase} -> [{target_field}] (Row #{row_idx})")
            logging.info(f" - Profit: {profit_str}")
            logging.info(f" - Trades: {trades_count}")

            # Detailed Trade Listing
            count = 0
            for t_detail in stats.get('detailed_trades', []):
                logging.info(f" - {t_detail}")
                count += 1
            if count > 50: # Safety limit
                logging.info(f" ... and {trades_count - 50} more")
                break

            logging.info(f" - Source ID(s): {sources_str}")
        logging.info("="*30)
    # -----

    # Return the computed sessions so they can be saved as 'aggregated_by_comment'
    return evaluations, match_log, sessions

# -----
# LEGACY CLIENT-SIDE AGGREGATION LOGIC (Fallback)
# -----

"""
Update evaluation hedge result fields from aggregated MT5 comment data.

Args:
    evaluations: List of evaluation records
    aggregated_data: List of aggregated trade data with account_number, phase_code, etc.

Returns:

```

```

        Tuple of (updated_evaluations, match_log)
"""
if not evaluations or not aggregated_data:
    # If no server-side matching occurred (raw_deals was None), just return None for sessions
    return evaluations, ["No evaluations or aggregated data to process"], None

match_log = []
updates_made = 0

match_log.append(f"■ Processing {len(aggregated_data)} aggregated trade groups")
match_log.append(f"    Against {len(evaluations)} evaluation records")

# Sort aggregated data to ensure farming days are processed chronologically
# Sort by account, then by timestamp (or farming_date)
def sort_key(x):
    # Ensure consistent string type for comparison to avoid TypeError between float/str
    ts = x.get('timestamp') or x.get('farming_date') or ''
    return (
        str(x.get('account_number', '')),
        str(ts)
    )

try:
    aggregated_data.sort(key=sort_key)
except Exception as e:
    match_log.append(f"■ Warning: Sorting failed ({str(e)}), processing unsorted.")

# --- FNFT FILTERING LOGIC ---
# For FundedNext accounts:
# 1. Only process the active (latest) phase.
# 2. Within that phase, only process the LATEST DAY (ignore prior history for same phase).

# Map: account_sig -> (max_timestamp, (phase_code, trade_number))
fnft_latest_phase_map = {}

# 1. Identify latest phase AND latest timestamp for that phase
for agg in aggregated_data:
    acc = str(agg.get('account_number', '')).upper()
    if 'FNFT' in acc or 'FUNDEDNEXT' in acc:
        sig = get_account_signature(acc)
        ts = agg.get('timestamp') or 0

        # Update global max timestamp for account (determines active phase)
        curr_max_ts, curr_combo = fnft_latest_phase_map.get(sig, (0, None))

        if ts >= curr_max_ts:
            p_code = agg.get('phase_code')
            t_num = agg.get('trade_number')
            fnft_latest_phase_map[sig] = (ts, (p_code, t_num))

# 2. Filter loop
filtered_data = []
for agg in aggregated_data:
    acc = str(agg.get('account_number', '')).upper()
    if 'FNFT' in acc or 'FUNDEDNEXT' in acc:
        sig = get_account_signature(acc)
        max_ts_global, latest_combo = fnft_latest_phase_map.get(sig, (0, None))

        this_combo = (agg.get('phase_code'), agg.get('trade_number'))
        this_ts = agg.get('timestamp') or 0

```



```

this_phase_code = agg.get('phase_code', '')

# Check 1: Is this the active phase?
if latest_combo and this_combo != latest_combo:
    match_log.append(f"■ FNFT: Skipping {acc} old phase {this_combo} (Active: {latest_combo})")
    continue

# Check 2: Age Check applied ONLY to Challenge (CH) phase
# "only checking the current days trades will check this"
if this_phase_code == 'CH':
    # Strictly filter to the "Current Day" (represented by max_ts_global).
    # We discard any data older than 18 hours from the latest timestamp.
    # This ensures we don't sum up "Old Reset" results (e.g. from 2 days ago)
    # with "New Reset" results (Today).
    AGE_THRESHOLD = 18 * 3600 # 18 hours (Current session only)
    time_diff = max_ts_global - this_ts

    if time_diff > AGE_THRESHOLD:
        match_log.append(
            f"■ FNFT: Skipping {acc} old Challenge history {this_combo} (Age: {time_diff/3600})")
        continue
    else:
        # Pass through FA/FD/DD to allow standard logic as requested
        pass

filtered_data.append(agg)

aggregated_data = filtered_data
# -----

# Track next available slot for Farming (FA) phase per evaluation
# This ensures we overwrite from Day 1 sequentially instead of appending
fa_slot_tracker = {}

# Track accumulated values for standard phases (CH, FD, DD) because client now sends daily chunks
# Key: (eval_idx, field_name) -> value
accumulation_tracker = {}

for agg in aggregated_data:
    account_number = agg.get('account_number', '')
    phase_code = agg.get('phase_code', '')
    trade_number = agg.get('trade_number')
    farming_date = agg.get('farming_date')
    net_profit = agg.get('net_profit', 0)
    deal_count = agg.get('deal_count', 0)

    if account_number and str(account_number).lower().startswith("unknown"):
        continue

    # Find matching evaluation
    matches = match_account_to_evaluation(account_number, evaluations, phase_code)

    # Use date filtering to pick the best match
    # Try to get timestamp from agg if available, otherwise it will fallback to first match
    trade_timestamp = agg.get('timestamp')
    match = filter_matches_by_date(
        matches,
        evaluations,
        trade_timestamp,
        phase_code=phase_code,

```

```

        trade_number=trade_number
    )

    if not match:
        sig = get_account_signature(account_number)
        match_log.append(
            f"■■■ No match: {account_number} ({sig}) _{phase_code}{trade_number or ''} = ${net_profit:.2f}"
        )
        continue

    eval_idx, matched_account = match
    logging.info(f"MATCHED: MT5 Account {account_number} -> Dashboard Account {matched_account}")

    # Defense-in-depth: refuse to write into a row whose target account
    # column is blank or a placeholder ("-", "-", "n/a", etc.). Catches
    # cases where a stale lookup or fuzzy match returns a row the user
    # has since wiped clean.
    _placeholders = {'', 'none', '-', '-', '-', 'n/a', 'na', 'tbd', 'pending'}
    _matched_norm = str(matched_account or '').strip().lower()
    if _matched_norm in _placeholders:
        match_log.append(
            f"■ Skipped write to row #{eval_idx}: target account is placeholder "
            f"'{matched_account}' (deal {account_number} _{phase_code}{trade_number or ''} = ${net_profit:.2f})"
        )
        continue

    # Also verify the actual eval row still has a real value in BOTH the
    # challenge and funded account columns being placeholders would mean
    # a wiped row that should never receive writes.
    _ev_check = evaluations[eval_idx] if 0 <= eval_idx < len(evaluations) else None
    if _ev_check is not None:
        _ch = str(_ev_check.get('Account #') or '').strip().lower()
        _fd = str(_ev_check.get('Account #.1') or '').strip().lower()
        if (_ch in _placeholders) and (_fd in _placeholders):
            match_log.append(
                f"■ Skipped wiped row #{eval_idx}: both Account # cols are blank/placeholder "
                f"'(deal {account_number} _{phase_code}{trade_number or ''} = ${net_profit:.2f})'"
            )
            continue

    # Determine field to update
    field_name = None

    if phase_code == 'FA':
        client_field_name = str(agg.get('field_name') or '').strip()
        client_slot = agg.get('_fa_slot')

        if client_field_name.startswith('Hedge Day '):
            field_name = client_field_name
        elif isinstance(client_slot, (int, float)) and client_slot > 0:
            field_name = f"Hedge Day {int(client_slot)}"
        else:
            # Fallback for older clients that do not send a pre-computed Hedge Day.
            current_slot = fa_slot_tracker.get(eval_idx, 1)
            field_name = f"Hedge Day {current_slot}"
            fa_slot_tracker[eval_idx] = current_slot + 1
    else:
        field_name = get_field_name_for_phase(phase_code, trade_number, farming_date, evaluations,
                                              eval_idx, account_number)

    if not field_name:
        match_log.append(f"■■■ Unknown field for {phase_code}{trade_number or ''}")

```

```

        continue

    # Determine whether to Accumulate (CH, FD, DD) or Overwrite (FA)
    # UPDATE: FNFT Challenge (CH) should NOT accumulate if we are strictly enforcing "Current Day" in
    # However, checking 'is_fnft' here is tricky without context.
    # But wait - if we are now only sending today's data, accumulation of *just today's* trades is co
    # But we do NOT want to read the OLD value from DB and add to it.
    # The logic below `accumulation_tracker[key] = current_val + float(net_profit)` only sums up what
    # It does NOT pull from evaluations[eval_idx][field_name] first.
    # So it is effectively a "Fresh Sum of Payload".
    # This is correct behavior for "Current Day Only" payload.

    should_accumulate = phase_code in ['CH', 'FD', 'DD']

    # ■■■ Honor manual user clears ■■■
    # If the user explicitly blanked this field on the dashboard, refuse to
    # repopulate it from re-aggregated MT5 history. The clear is lifted
    # automatically when the user types a new value back into the cell
    # (see /api/update_data handler).
    _ev_for_clear = evaluations[eval_idx] if 0 <= eval_idx < len(evaluations) else None
    if _ev_for_clear is not None:
        _cleared_set = set(_ev_for_clear.get('_cleared_fields') or [])
        if field_name in _cleared_set:
            match_log.append(
                f"■ Skipped manually-cleared cell: Eval #{eval_idx} [{field_name}] "
                f"(would have been ${net_profit:.2f} from {account_number})"
            )
            continue

    if should_accumulate:
        # Add to accumulator (sums up multiple entries in SAME payload)
        key = (eval_idx, field_name)
        current_val = accumulation_tracker.get(key, 0.0)
        accumulation_tracker[key] = current_val + float(net_profit)

        # Log individual contribution
        # sig = get_account_signature(account_number)
        # match_log.append(f"    + {account_number} ({sig}) → [{field_name}] += ${net_profit:.2f}")
    else:
        # Direct overwrite (Farming)
        evaluations[eval_idx][field_name] = net_profit
        updates_made += 1
        sig = get_account_signature(account_number)
        match_log.append(
            f"■ {account_number} ({sig}) {_phase_code}{trade_number or ''} → [{field_name}] = ${net_profit:.2f}"
        )
        match_log.append(f"    Matched to: {matched_account}")

    # Apply accumulated updates
    for (eval_idx, field_name), total_profit in accumulation_tracker.items():
        evaluations[eval_idx][field_name] = total_profit
        updates_made += 1
    # Log the final total update
    # We can't easily show which account number triggered it if multiple did, but usually it's one.
    match_log.append(f"■ [Accumulated] → Eval #{eval_idx} [{field_name}] = ${total_profit:.2f}")

    match_log.append(f"\n■ Total updates: {updates_made}/{len(aggregated_data)}")
    return evaluations, match_log, None

def init_admin_password()

```

Initialize default admin password only if not already set.

Step by step (top-level body)

1. if not admin_password_exists('super_admin'): branches conditionally.
2. if not admin_password_exists('bef_admin'): branches conditionally.
3. if not admin_password_exists('kwok_admin'): branches conditionally.

Code (copy-paste safe)

```
def init_admin_password():
    """Initialize default admin password only if not already set."""
    if not admin_password_exists('super_admin'):
        admin_password = os.getenv('ADMIN_PASSWORD', 'BallerAdmin@123')
        set_admin_password('super_admin', admin_password)
        print("super_admin password initialized")
    # BEF Admin - separate credentials, restricted to BEF clients only
    if not admin_password_exists('bef_admin'):
        bef_password = os.getenv('BEF_ADMIN_PASSWORD', 'BEFAdmin@123')
        set_admin_password('bef_admin', bef_password)
        print("bef_admin password initialized")
    if not admin_password_exists('kwok_admin'):
        # Legacy DB username was showcase_admin; copy hash so existing installs keep the same password.
        if not copy_admin_password_row('showcase_admin', 'kwok_admin'):
            kwok_password = os.getenv(
                'KWOK_ADMIN_PASSWORD',
                os.getenv('SHOWCASE_ADMIN_PASSWORD', 'KwokAdmin@123'),
            )
            set_admin_password('kwok_admin', kwok_password)
        print("kwok_admin password initialized or migrated from legacy showcase_admin row")

def provision_hierarchy_passwords()
```

Auto-create or reset user_credentials with default password for hierarchy users.

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.
- **nonlocal** — Declares a name as belonging to an enclosing function's scope. The flag that says *this nested function is rebinding the outer variable, not creating its own*.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns default_pw = 'Test@123'.
2. Assigns created = 0.
3. Assigns reset = 0.

4. Defines a nested function `ensure_password(...)`.
5. `for (admin_name, admin_data) in hierarchy.get('admins', {}).items():` iterates.
6. `if created or reset:` branches conditionally.

Code (copy-paste safe)

```
def provision_hierarchy_passwords():
    """Auto-create or reset user_credentials with default password for hierarchy users."""
    default_pw = 'Test@123'
    created = 0
    reset = 0

    def ensure_password(name, utype, email=None, parent_admin=None, parent_trader=None):
        nonlocal created, reset
        if not user_exists(name, utype):
            if create_user(name, default_pw, utype, email, parent_admin, parent_trader):
                created += 1
        else:
            # Reset password for users who have never logged in (still have old random pw)
            existing = get_user(name, utype)
            if existing and not existing.get('last_login'):
                if update_user_password(name, utype, default_pw):
                    reset += 1

    for admin_name, admin_data in hierarchy.get('admins', {}).items():
        ensure_password(admin_name, 'admin', admin_data.get('email'))
        for trader_name, trader_data in admin_data.get('traders', {}).items():
            ensure_password(trader_name, 'trader', trader_data.get('email'), admin_name)
            for client in trader_data.get('clients', []):
                c_name = client.get('name') if isinstance(client, dict) else client
                c_email = client.get('email', '') if isinstance(client, dict) else ''
                ensure_password(c_name, 'client', c_email, admin_name, trader_name)
    if created or reset:
        print(f"[AUTH] Provisioned {created} new, reset {reset} existing users to default password")

def _bg_provision()
```

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def _bg_provision():
    try:
        provision_hierarchy_passwords()
    except Exception as _prov_exc:
        print(f"[STARTUP] WARNING: provision_hierarchy_passwords failed: {_prov_exc}")
```

```
def require_api_key(f)
```

Decorator to require valid API key for endpoint access.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.

Step by step (top-level body)

1. Defines a nested function `decorated_function(...)`.
2. **return** `decorated_function`.

Code (copy-paste safe)

```
def require_api_key(f):
    """Decorator to require valid API key for endpoint access."""
    @wraps(f)
    def decorated_function(*args, **kwargs):
        api_key = request.headers.get('X-API-Key')
        client_ip = get_remote_address()

        if not api_key:
            log_action('API_ACCESS_DENIED', 'unknown', 'no_key', client_ip, 'Missing API key', False)
            return jsonify({"status": "error", "message": "API key required"}), 401

        try:
            user_info = validate_api_key(api_key)
        except Exception:
            app.logger.exception('[AUTH] validate_api_key raised – DB may be corrupt')
            return jsonify({"status": "error", "message": "Service temporarily unavailable"}), 503
        if not user_info:
            log_action('API_ACCESS_DENIED', 'unknown', api_key[:12], client_ip, 'Invalid API key', False)
            return jsonify({"status": "error", "message": "Invalid API key"}), 403

        # Reject read-only keys from full-access endpoints
        if user_info.get('scope') == 'readonly':
            log_action('API_ACCESS_DENIED', 'unknown', api_key[:12], client_ip,
                'Readonly key on full-access endpoint', False)
            return jsonify({"status": "error",
                "message": "This API key is read-only and cannot access this endpoint"}), 403

        # Add user info to request context
        request.api_user = user_info
        log_action('API_ACCESS', 'trader', user_info.get('trader', 'unknown'), client_ip,
            f"Endpoint: {request.endpoint}")
        return f(*args, **kwargs)
    return decorated_function


def require_admin_password(f)
```

Decorator to require admin password for endpoint access.

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.

Step by step (top-level body)

1. Defines a nested function `decorated_function(...)`.
2. **return** `decorated_function`.

Code (copy-paste safe)

```
def require_admin_password(f):
    """Decorator to require admin password for endpoint access."""
    @wraps(f)
    def decorated_function(*args, **kwargs):
        client_ip = get_remote_address()

        # Check for password in request body or headers
        admin_password = None
        if request.is_json:
            admin_password = request.json.get('admin_password')
        if not admin_password:
            admin_password = request.headers.get('X-Admin-Password')

        if not admin_password:
            log_action('ADMIN_ACCESS_DENIED', 'admin', 'unknown', client_ip, 'Missing password', False)
            return jsonify({"status": "error", "message": "Admin password required"}), 401

        if not verify_admin_password('super_admin', admin_password):
            log_action('ADMIN_ACCESS_DENIED', 'admin', 'super_admin', client_ip, 'Invalid password', False)
            return jsonify({"status": "error", "message": "Invalid admin password"}), 403

        log_action('ADMIN_ACCESS', 'admin', 'super_admin', client_ip, f"Endpoint: {request.endpoint}")
        return f(*args, **kwargs)
    return decorated_function
```

```
def require_role(*allowed_roles)
```

Decorator to require specific roles via session authentication.

Why these Python concepts were used

- ***allowed_roles** — Accepts a variable number of positional arguments. Most common reason: the function forwards them to another callable, or it operates on a list whose length is not fixed up front.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.

Step by step (top-level body)

1. Defines a nested function decorator(...).

2. **return** decorator.

Code (copy-paste safe)

```
def require_role(*allowed_roles):
    """Decorator to require specific roles via session authentication."""
    def decorator(f):
        @wraps(f)
        def decorated_function(*args, **kwargs):
            session_token = request.cookies.get('session_token')

            if not session_token:
                return jsonify({"status": "error", "message": "Authentication required"}), 401

            session_info = validate_session(session_token)
            if not session_info:
                return jsonify({"status": "error", "message": "Invalid or expired session"}), 401

            user_type = session_info.get('user_type')
            if user_type not in allowed_roles:
                log_action('ACCESS_DENIED', user_type, session_info.get('user_identifier'),
                           get_remote_address(), f"Required roles: {allowed_roles}", False)
                return jsonify({"status": "error",
                               "message": "Access denied. Insufficient permissions."}), 403

            request.session_user = session_info
            return f(*args, **kwargs)
        return decorated_function
    return decorator
```

```
def require_session(f)
```

Decorator to require valid session for web access.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.

Step by step (top-level body)

1. Defines a nested function decorated_function(...).

2. **return** decorated_function.

Code (copy-paste safe)

```
def require_session(f):
    """Decorator to require valid session for web access."""
    @wraps(f)
    def decorated_function(*args, **kwargs):
        session_token = request.cookies.get('session_token')

        if not session_token:
```



```

        return redirect(url_for('index'))

    try:
        session_info = validate_session(session_token)
    except Exception:
        app.logger.exception('[AUTH] validate_session raised – DB may be corrupt, redirecting to login')
        return redirect(url_for('index'))

    if not session_info:
        return redirect(url_for('index'))

    request.session_user = session_info
    return f(*args, **kwargs)
return decorated_function

```

```

@app.context_processor
def inject_home_url()

```

Inject home_url into every template so the logo can link to the user's home page.

Why these Python concepts were used

- **@app.context_processor** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.
2. **return** {'home_url': '/' }.

Code (copy-paste safe)

```

def inject_home_url():
    """Inject home_url into every template so the logo can link to the user's home page."""
    try:
        session_token = request.cookies.get('session_token')
        if session_token:
            session_info = validate_session(session_token)
            if session_info:
                user_type = session_info.get('user_type', '')
                user_id = session_info.get('user_identifier', '')
                if user_type == 'super_admin':
                    return {'home_url': '/super_admin'}
                elif user_type == 'bef_admin':
                    return {'home_url': '/bef_admin'}
                elif user_type == 'kwok_admin':
                    return {'home_url': '/kwok_admin'}
                elif user_type == 'admin':
                    return {'home_url': f'/admin/{user_id}'}
                elif user_type == 'trader':
                    return {'home_url': f'/trader/{user_id}'}
                elif user_type == 'client':

```

```

        return {'home_url': f'/dashboard/{user_id}'}
    except Exception:
        pass
    return {'home_url': '/'}

```

```

@app.route('/maintenance')
def maintenance_page()

```

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `session_token = request.cookies.get('session_token')`.
2. **if** `session_token`: branches conditionally.
3. **return** `render_template('maintenance.html')`.

Code (copy-paste safe)

```

def maintenance_page():
    session_token = request.cookies.get('session_token')
    if session_token:
        try:
            session_info = validate_session(session_token)
            if session_info and session_info.get('user_type') in MAINTENANCE_EXEMPT:
                user_type = session_info.get('user_type')
                user_id = session_info.get('user_identifier')
                if user_type == 'super_admin':
                    return redirect('/super_admin')
                elif user_type == 'bef_admin':
                    return redirect('/bef_admin')
                elif user_type == 'kwok_admin':
                    return redirect('/kwok_admin')
                elif user_type == 'admin':
                    return redirect(f'/admin/{user_id}')
                elif user_type == 'trader':
                    return redirect(f'/trader/{user_id}')
            except Exception:
                pass
        return render_template('maintenance.html')

    @app.route('/')
    def index()

```

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `session_token = request.cookies.get('session_token')`.
2. **if** `session_token`: branches conditionally.
3. **return** `render_template('login.html')`.

Code (copy-paste safe)

```
def index():
    # Check if already logged in
    session_token = request.cookies.get('session_token')
    if session_token:
        session_info = validate_session(session_token)
        if session_info:
            # Redirect to appropriate dashboard
            user_type = session_info.get('user_type')
            user_id = session_info.get('user_identifier')
            if user_type == 'super_admin':
                return redirect('/super_admin')
            elif user_type == 'bef_admin':
                return redirect('/bef_admin')
            elif user_type == 'kwok_admin':
                return redirect('/kwok_admin')
            elif user_type == 'admin':
                return redirect(f'/admin/{user_id}')
            elif user_type == 'trader':
                return redirect(f'/trader/{user_id}')
            elif user_type == 'client':
                if MAINTENANCE_MODE:
                    return redirect('/maintenance')
                return redirect(f'/dashboard/{user_id}')
        return render_template('login.html')

@app.route('/super_admin')
@require_session
def super_admin():
```

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_session** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.

Step by step (top-level body)

1. **if** `request.session_user.get('user_type') != 'super_admin'`: branches conditionally.
2. **return** `render_template('super_admin.html', is_bef_admin=False, is_kwok_admin=...)`.

Code (copy-paste safe)

```
def super_admin():
    if request.session_user.get('user_type') != 'super_admin':
        return redirect('/')
```

```
return render_template('super_admin.html', is_bef_admin=False, is_kwok_admin=False)
```

```
@app.route('/bef_admin')
@require_session
def bef_admin()
```

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_session** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.

Step by step (top-level body)

1. **if** request.session_user.get('user_type') != 'bef_admin': branches conditionally.
2. **return** render_template('super_admin.html', user_role='bef_admin', is_bef_a....

Code (copy-paste safe)

```
def bef_admin():
    if request.session_user.get('user_type') != 'bef_admin':
        return redirect('/')
    return render_template(
        'super_admin.html', user_role='bef_admin', is_bef_admin=True, is_kwok_admin=False)
```

```
@app.route('/showcase_admin')
def showcase_admin_legacy_redirect()
```

Old URL/bookmarks from the previous role name.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.

Step by step (top-level body)

1. **return** redirect('/kwok_admin', code=308).

Code (copy-paste safe)

```
def showcase_admin_legacy_redirect():
    """Old URL/bookmarks from the previous role name."""
    return redirect('/kwok_admin', code=308)
```

```
@app.route('/kwok_admin')
@require_session
def kwok_admin()
```

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.

- **@require_session** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.

Step by step (top-level body)

1. **if** request.session_user.get('user_type') != 'kwok_admin': branches conditionally.
2. **return** render_template('super_admin.html', user_role='kwok_admin', is_bef_....

Code (copy-paste safe)

```
def kwok_admin():
    if request.session_user.get('user_type') != 'kwok_admin':
        return redirect('/')
    return render_template(
        'super_admin.html', user_role='kwok_admin', is_bef_admin=False, is_kwok_admin=True)

@app.route('/quality_dashboard')
@require_session
def quality_dashboard()
```

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_session** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.

Step by step (top-level body)

1. **if** request.session_user.get('user_type') not in ('super_admin',): branches conditionally.
2. **return** render_template('quality_dashboard.html').

Code (copy-paste safe)

```
def quality_dashboard():
    if request.session_user.get('user_type') not in ('super_admin',):
        return redirect('/')
    return render_template('quality_dashboard.html')

@app.route('/admin/<admin_name>')
@require_session
def admin_dashboard(admin_name)
```

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_session** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.

Step by step (top-level body)

1. Assigns session_user = request.session_user.
2. **if** session_user.get('user_type') in ('super_admin', 'bef_admin', 'kwok...: branches conditionally.

3. **if** `session_user.get('user_type') != 'admin'` or `session_user.get('user_...':` branches conditionally.
4. **return** `render_template('admin_dashboard.html', admin_name=admin_name)`.

Code (copy-paste safe)

```
def admin_dashboard(admin_name):
    session_user = request.session_user
    # Allow super_admin, bef_admin, and kwok_admin to access admin dashboards
    if session_user.get('user_type') in ('super_admin', 'bef_admin', 'kwok_admin'):
        ut = session_user.get('user_type')
        return render_template('admin_dashboard.html', admin_name=admin_name,
                               is_bef_admin=(ut == 'bef_admin'),
                               is_kwok_admin=(ut == 'kwok_admin'))
    # Check if user is the correct admin
    if session_user.get('user_type') != 'admin' or session_user.get('user_identifier') != admin_name:
        return redirect('/')
    return render_template('admin_dashboard.html', admin_name=admin_name)

@app.route('/financial_overview')
@require_session
def financial_overview()
```

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_session** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **dict comprehension** — Builds a dict in one expression (`{k: v for k, v in items}`) — same intent as a list comprehension but for mappings.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to `sum`, `any`, `all`, or `max` where the intermediate list would be wasted.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns `session_user = request.session_user`.
2. **if** `session_user.get('user_type') not in ('super_admin', 'bef_admin', '...':` branches conditionally.
3. Assigns `profile_filter = request.args.get('profile', 'ALL')`.
4. **if** `session_user.get('user_type') == 'bef_admin':` branches conditionally.
5. **if** `session_user.get('user_type') == 'kwok_admin':` branches conditionally.
6. Assigns `start_date_str = request.args.get('start_date')`.
7. Assigns `end_date_str = request.args.get('end_date')`.

8. Assigns `start_date = None`.
9. `if start_date_str`: branches conditionally.
10. Assigns `end_date = None`.
11. `if end_date_str`: branches conditionally.
12. Assigns `all_data = calculate_all_financials(profile_filter=profile_filter, s....`
13. Assigns `overview_data = all_data['overview']`.
14. Assigns `global_stats = all_data.get('global_stats', {})`.
15. ... and 15 more statement(s) in the body.

Code (copy-paste safe)

```
def financial_overview():
    session_user = request.session_user
    if session_user.get('user_type') not in ('super_admin', 'bef_admin', 'kwok_admin'):
        return redirect('/')

    profile_filter = request.args.get('profile', 'ALL')
    # BEF admin always sees only BEF profile
    if session_user.get('user_type') == 'bef_admin':
        profile_filter = 'BEF'
    # Kwok admin: consolidated view only (no BEF / Private slice in UI or via ?profile=)
    if session_user.get('user_type') == 'kwok_admin':
        profile_filter = 'ALL'

    # Date filtering
    start_date_str = request.args.get('start_date')
    end_date_str = request.args.get('end_date')

    start_date = None
    if start_date_str:
        try:
            start_date = datetime.strptime(start_date_str, "%Y-%m-%d")
        except:
            pass

    end_date = None
    if end_date_str:
        try:
            end_date = datetime.strptime(end_date_str, "%Y-%m-%d")
        except:
            pass

    # NEW: Use optimized single-pass aggregator
    all_data = calculate_all_financials(profile_filter=profile_filter, start_date=start_date,
                                         end_date=end_date)

    overview_data = all_data['overview']
    global_stats = all_data.get('global_stats', {})

    # Card totals from cashflow_inprogress (same source as BEF admin dashboard)
    perf_clients = get_client_performance_stats(profile_filter, start_date=start_date, end_date=end_date)
    card_totals = {
        'payouts': round(sum(c.get('payouts', 0) for c in perf_clients), 2),
        'deposits': round(sum(c.get('deposits', 0) for c in perf_clients), 2),
        'fees': round(sum(c.get('fees', 0) for c in perf_clients), 2),
```

```

        'net_profit': round(sum(c.get('net_profit', 0) for c in perf_clients), 2),
        'hedge': round(sum(c.get('hedge_profit', 0) for c in perf_clients), 2),
        'active': sum(c.get('active', 0) for c in perf_clients),
        'passed': sum(c.get('passed', 0) for c in perf_clients),
        'failed': sum(c.get('failed', 0) for c in perf_clients),
    }
    t_ended = sum(c.get('ended', 0) for c in perf_clients)
    t_duration = sum(c.get('total_duration_days', 0) for c in perf_clients)
    card_totals['ev'] = round(card_totals['net_profit'] / t_ended, 2) if t_ended > 0 else 0.0
    card_totals['ev_day'] = round(card_totals['net_profit'] / t_duration, 2) if t_duration > 0 else 0.0

    # Hide restricted firms from BEF admin only (Kwok admin sees full firm set for demos)
    if session_user.get('user_type') == 'bef_admin':
        overview_data = {k: v for k, v in overview_data.items()
                        if k.lower().replace(' ', '') not in BEF_HIDDEN_FIRMS}

    growth_dates, growth_values = all_data['growth']
    payouts_dates, payouts_values = all_data['payouts']
    net_profit_dates, net_profit_values = all_data['net_profit']
    deposits_dates, deposits_values = all_data['deposits']
    fees_dates, fees_values = all_data['fees']
    hedge_dates, hedge_values = all_data['hedge']
    farming_dates, farming_values = all_data['farming']

    return render_template('financial_overview.html',
                          overview=overview_data,
                          global_stats=global_stats,
                          card_totals=card_totals,
                          selected_profile=profile_filter,
                          start_date=start_date_str,
                          end_date=end_date_str,
                          is_bef_admin=(session_user.get('user_type') == 'bef_admin'),
                          is_kwok_admin=(session_user.get('user_type') == 'kwok_admin'),
                          growth_dates=growth_dates,
                          growth_values=growth_values,
                          payouts_dates=payouts_dates,
                          payouts_values=payouts_values,
                          net_profit_dates=net_profit_dates,
                          net_profit_values=net_profit_values,
                          deposits_dates=deposits_dates,
                          deposits_values=deposits_values,
                          fees_dates=fees_dates,
                          fees_values=fees_values,
                          hedge_dates=hedge_dates,
                          hedge_values=hedge_values,
                          farming_dates=farming_dates,
                          farming_values=farming_values)

@app.route('/payout_history')
@require_session
def payout_history()

```

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_session** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with `append`, and clearer about intent: this is a transform, not a side-effecting loop.

Step by step (top-level body)

1. Assigns `session_user = request.session_user`.
2. **if** `session_user.get('user_type') not in ('super_admin', 'bef_admin', '...: branches conditionally.`
3. Assigns `start_date_str = request.args.get('start_date')`.
4. Assigns `end_date_str = request.args.get('end_date')`.
5. Assigns `start_date = None`.
6. **if** `start_date_str`: branches conditionally.
7. Assigns `end_date = None`.
8. **if** `end_date_str`: branches conditionally.
9. Assigns `prop_firm_filter = request.args.get('prop_firm')`.
10. Assigns `profile_filter = request.args.get('profile', 'ALL')`.
11. **if** `session_user.get('user_type') == 'bef_admin'`: branches conditionally.
12. **if** `session_user.get('user_type') == 'kwok_admin'`: branches conditionally.
13. Assigns `overview_data = calculate_propfirm_overview()`.
14. Assigns `sorted_prop_firms = sorted(overview_data.keys())`.
15. ... and 4 more statement(s) in the body.

Code (copy-paste safe)

```
def payout_history():
    session_user = request.session_user
    if session_user.get('user_type') not in ('super_admin', 'bef_admin', 'kwok_admin'):
        return redirect('/')

    # Filter dates
    start_date_str = request.args.get('start_date')
    end_date_str = request.args.get('end_date')

    start_date = None
    if start_date_str:
        try:
            start_date = datetime.strptime(start_date_str, "%Y-%m-%d")
        except:
            pass

    end_date = None
    if end_date_str:
        try:
            end_date = datetime.strptime(end_date_str, "%Y-%m-%d")
        except:
            pass
```

```

prop_firm_filter = request.args.get('prop_firm')
profile_filter = request.args.get('profile', 'ALL')

# BEF admin always sees only BEF profile
if session_user.get('user_type') == 'bef_admin':
    profile_filter = 'BEF'
if session_user.get('user_type') == 'kwok_admin':
    profile_filter = 'ALL'
# We need overview data just to get the list of prop firms for the dropdown
overview_data = calculate_propfirm_overview()
sorted_prop_firms = sorted(overview_data.keys())

# Hide restricted firms from BEF admin
if session_user.get('user_type') == 'bef_admin':
    sorted_prop_firms = [f for f in sorted_prop_firms if f.lower().replace(' ',
        '') not in BEF_HIDDEN_FIRMS]

payouts_list = get_payouts_history(start_date, end_date, prop_firm_filter, profile_filter=profile_filter)

# Filter payout entries for BEF admin
if session_user.get('user_type') == 'bef_admin':
    payouts_list = [p for p in payouts_list if p.get('prop_firm', '').lower().replace(' ',
        '') not in BEF_HIDDEN_FIRMS]

return render_template('payout_history.html',
    payouts=payouts_list,
    start_date=start_date_str,
    end_date=end_date_str,
    selected_prop_firm=prop_firm_filter,
    selected_profile=profile_filter,
    is_bef_admin=(session_user.get('user_type') == 'bef_admin'),
    is_kwok_admin=(session_user.get('user_type') == 'kwok_admin'),
    prop_firms=sorted_prop_firms)

@app.route('/client_performance')
@require_session
def client_performance():

```

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_session** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.

Step by step (top-level body)

1. Assigns `session_user = request.session_user`.
2. **if** `session_user.get('user_type') == 'kwok_admin'`: branches conditionally.
3. **if** `session_user.get('user_type') not in ('super_admin', 'bef_admin')`: branches conditionally.
4. Assigns `ut = session_user.get('user_type')`.
5. **return** `render_template('client_performance.html', is_bef_admin=ut == 'bef_....`

Code (copy-paste safe)

```
def client_performance():
```

```

session_user = request.session_user
if session_user.get('user_type') == 'kwok_admin':
    return redirect('/kwok_admin')
if session_user.get('user_type') not in ('super_admin', 'bef_admin'):
    return redirect('/')
ut = session_user.get('user_type')
return render_template('client_performance.html', is_bef_admin=(ut == 'bef_admin'),
                      is_kwok_admin=False)

```

```

@app.route('/trader_performance')
@require_session
def trader_performance()

```

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_session** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.

Step by step (top-level body)

1. Assigns `session_user = request.session_user`.
2. **if** `session_user.get('user_type') == 'kwok_admin':` branches conditionally.
3. **if** `session_user.get('user_type') not in ('super_admin', 'bef_admin'):` branches conditionally.
4. Assigns `profile_filter = request.args.get('profile', 'ALL')`.
5. **if** `session_user.get('user_type') == 'bef_admin':` branches conditionally.
6. Assigns `traders_data = calculate_trader_stats(profile_filter=profile_filter)`.
7. Assigns `ut = session_user.get('user_type')`.
8. **return** `render_template('trader_performance.html', traders=traders_data, se...`

Code (copy-paste safe)

```

def trader_performance():
    session_user = request.session_user
    if session_user.get('user_type') == 'kwok_admin':
        return redirect('/kwok_admin')
    if session_user.get('user_type') not in ('super_admin', 'bef_admin'):
        return redirect('/')

    profile_filter = request.args.get('profile', 'ALL')
    # BEF admin always sees only BEF profile
    if session_user.get('user_type') == 'bef_admin':
        profile_filter = 'BEF'
    traders_data = calculate_trader_stats(profile_filter=profile_filter)
    ut = session_user.get('user_type')
    return render_template('trader_performance.html', traders=traders_data, selected_profile=profile_filter,
                          is_bef_admin=(ut == 'bef_admin'), is_kwok_admin=(ut == 'kwok_admin'))

@app.route('/trader/<trader_name>')
@require_session
def trader_dashboard(trader_name)

```

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_session** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

Step by step (top-level body)

1. Assigns `session_user = request.session_user`.
2. **try** block with 1 **except** clause.
3. Assigns `trader_email = ''`.
4. Assigns `trader_admin = ''`.
5. **try** block with 1 **except** clause.
6. **if** `session_user.get('user_type') in ('super_admin', 'bef_admin', 'kwok...)` branches conditionally.
7. **if** `session_user.get('user_type') == 'admin':` branches conditionally.
8. **if** `session_user.get('user_type') != 'trader' or session_user.get('user...)` branches conditionally.
9. **return** `render_template('trader_dashboard.html', trader_name=trader_name, t...`

Code (copy-paste safe)

```
def trader_dashboard(trader_name):
    session_user = request.session_user
    # Resolve trader email + parent admin for admin actions (reset/delete)
    try:
        reload_hierarchy()
    except Exception:
        pass
    trader_email = ''
    trader_admin = ''
    try:
        trader_email = (SYSTEM_HIERARCHY.get('traders', {}) or {}).get(trader_name, {}).get('email', '')
        for admin_name, admin_data in (SYSTEM_HIERARCHY.get('admins', {}) or {}).items():
            traders = (admin_data or {}).get('traders', {}) or {}
            if trader_name in traders:
                trader_admin = admin_name
                trader_email = trader_email or (traders.get(trader_name, {}) or {}).get('email', '') or ''
                break
    except Exception:
        trader_email = trader_email or ''
        trader_admin = trader_admin or ''
    # Allow super_admin, bef_admin, and kwok_admin to access trader dashboards
    if session_user.get('user_type') in ('super_admin', 'bef_admin', 'kwok_admin'):
        ut = session_user.get('user_type')
        return render_template('trader_dashboard.html', trader_name=trader_name,
                               trader_email=trader_email, trader_admin=trader_admin,
                               is_super_admin=(ut == 'super_admin'),
                               is_bef_admin=(ut == 'bef_admin'),
                               is_kwok_admin=(ut == 'kwok_admin'))
```

```

# Allow admin to access traders under them
if session_user.get('user_type') == 'admin':
    return render_template('trader_dashboard.html', trader_name=trader_name,
                           trader_email=trader_email, trader_admin=trader_admin,
                           is_super_admin=False)
# Check if user is the correct trader
if session_user.get('user_type') != 'trader' or session_user.get('user_identifier') != trader_name:
    return redirect('/')
return render_template('trader_dashboard.html', trader_name=trader_name,
                       trader_email=trader_email, trader_admin=trader_admin,
                       is_super_admin=False)

@app.route('/dashboard/<client_id>')
@require_session
def client_dashboard(client_id)

```

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_session** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.

Step by step (top-level body)

1. Assigns `session_user = request.session_user`.
2. Assigns `user_type = session_user.get('user_type')`.
3. Assigns `client_email = ''`.
4. Assigns `is_active = True`.
5. Assigns `client_data = get_client_data(client_id)`.
6. **if** `client_data` and `client_data.get('identity')`: branches conditionally.
7. Assigns `_profile = get_client_profile(client_id)` or `{}`.
8. Assigns `client_trader_name = _profile.get('trader', '')`.
9. Assigns `client_admin_name = _profile.get('admin', '')`.
10. Assigns `_admin_data = SYSTEM_HIERARCHY.get('admins', {}).get(client_admin_name, ...)`.
11. Assigns `client_admin_slack_id = _admin_data.get('slack_user_id', '')`.
12. **if** `user_type` in `['super_admin', 'bef_admin', 'kwok_admin', 'admin', 't...]`: branches conditionally.
13. **if** `user_type == 'client'`: branches conditionally.
14. **return** `redirect('/')`.

Code (copy-paste safe)

```

def client_dashboard(client_id):
    session_user = request.session_user
    user_type = session_user.get('user_type')

    # Get client email and active status for dashboard
    client_email = ''
    is_active = True

```

```

client_data = get_client_data(client_id)
if client_data and client_data.get('identity'):
    client_email = client_data['identity'].get('email', '')
    is_active = client_data['identity'].get('active_status', 'active') != 'inactive'

# Look up the client's parent trader and admin for display in reports
_profile = get_client_profile(client_id) or {}
client_trader_name = _profile.get('trader', '')
client_admin_name = _profile.get('admin', '')
# Look up admin's Slack member ID for direct tagging
_admin_data = SYSTEM_HIERARCHY.get('admins', {}).get(client_admin_name, {})
client_admin_slack_id = _admin_data.get('slack_user_id', '')

# Allow super_admin, bef_admin, kwok_admin, admin, and trader to access client dashboards
if user_type in ['super_admin', 'bef_admin', 'kwok_admin', 'admin', 'trader']:
    # BEF admin can only access BEF-category clients
    if user_type == 'bef_admin' and not can_access_client('bef_admin', None, client_id):
        return redirect('/bef_admin')
    can_edit = user_type != 'kwok_admin'
    return render_template('index.html', client_id=client_id, user_type=user_type,
                           can_edit_hedging=can_edit, client_email=client_email, is_active=is_active,
                           client_trader_name=client_trader_name, client_admin_name=client_admin_name,
                           client_admin_slack_id=client_admin_slack_id)

# Client access: allow own dashboard OR primary KYC can view linked accounts
if user_type == 'client':
    own_name = session_user.get('user_identifier')
    if client_id == own_name:
        return render_template('index.html', client_id=client_id, user_type=user_type,
                               can_edit_hedging=True, client_email=client_email, is_active=is_active,
                               client_trader_name=client_trader_name, client_admin_name=client_admin_name,
                               client_admin_slack_id=client_admin_slack_id)
    # Only primary KYC clients can view linked accounts
    if is_kyc_primary(own_name) and client_id in get_all_kyc_accounts(own_name):
        return render_template('index.html', client_id=client_id, user_type=user_type,
                               can_edit_hedging=True, client_email=client_email, is_active=is_active,
                               client_trader_name=client_trader_name, client_admin_name=client_admin_name,
                               client_admin_slack_id=client_admin_slack_id)

return redirect('/')

def get_filtered_hierarchy(user_type, user_identifier)

```

Returns hierarchy filtered based on user role: - super_admin: sees everything - admin: sees only their traders and clients - trader: sees only their clients - client: sees only themselves

Why these Python concepts were used

- **list comprehension** — Builds a list in a single expression ([f(x) for x in xs]). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns full_hierarchy = hierarchy.
2. if user_type == 'super_admin': branches conditionally.
3. if user_type == 'kwok_admin': branches conditionally.

4. if user_type == 'bef_admin': branches conditionally.
5. if user_type == 'admin': branches conditionally.
6. if user_type == 'trader': branches conditionally.
7. if user_type == 'client': branches conditionally.
8. return {'admins': {}}

Code (copy-paste safe)

```
def get_filtered_hierarchy(user_type, user_identifier):
    """
    Returns hierarchy filtered based on user role:
    - super_admin: sees everything
    - admin: sees only their traders and clients
    - trader: sees only their clients
    - client: sees only themselves
    """
    full_hierarchy = hierarchy

    if user_type == 'super_admin':
        return full_hierarchy

    if user_type == 'kwok_admin':
        # Full client list for demos, without exposing super-admin hierarchy editing UI
        return full_hierarchy

    if user_type == 'bef_admin':
        # BEF admin sees full hierarchy but filtered to only BEF-category clients
        filtered_admins = {}
        for admin_name, admin_data in full_hierarchy.get('admins', {}).items():
            filtered_traders = {}
            for trader_name, trader_data in admin_data.get('traders', {}).items():
                bef_clients = [
                    c for c in trader_data.get('clients', [])
                    if (c.get('category') or '').upper() == 'BEF'
                ]
                if bef_clients:
                    filtered_traders[trader_name] = {
                        'email': trader_data.get('email', ''),
                        'clients': bef_clients
                    }
            if filtered_traders:
                filtered_admins[admin_name] = {
                    'email': admin_data.get('email', ''),
                    'traders': filtered_traders
                }
        return {'admins': filtered_admins}

    if user_type == 'admin':
        # Admin sees only their own data (case-insensitive match)
        admin_name = user_identifier
        admin_name_lower = admin_name.strip().lower()
        for key in full_hierarchy.get('admins', {}):
            if key.strip().lower() == admin_name_lower:
                return {
                    'admins': {
                        key: full_hierarchy['admins'][key]
                    }
                }
    }
```

```

        return {'admins': {}}

    if user_type == 'trader':
        # Trader sees only their clients – collect from ALL admins
        trader_name = user_identifier.strip()
        trader_name_lower = trader_name.lower()
        merged_admins = {}
        for admin_name, admin_data in full_hierarchy.get('admins', {}).items():
            traders = admin_data.get('traders', {})
            for t_key in traders:
                if t_key.strip().lower() == trader_name_lower:
                    if admin_name not in merged_admins:
                        merged_admins[admin_name] = {
                            'email': '',
                            'traders': {}
                        }
                    merged_admins[admin_name]['traders'][t_key] = traders[t_key]
        return {'admins': merged_admins} if merged_admins else {'admins': {}}

    if user_type == 'client':
        # Client sees only themselves (case-insensitive match)
        client_name = user_identifier
        client_name_lower = client_name.strip().lower()
        for admin_name, admin_data in full_hierarchy.get('admins', {}).items():
            for trader_name, trader_data in admin_data.get('traders', {}).items():
                for client in trader_data.get('clients', []):
                    cname = (client.get('name') or '').strip().lower()
                    cemail = (client.get('email') or '').strip().lower()
                    if cname == client_name_lower or cemail == client_name_lower:
                        return {
                            'admins': {
                                admin_name: {
                                    'email': '',
                                    'traders': {
                                        trader_name: {
                                            'email': '',
                                            'clients': [client]
                                        }
                                    }
                                }
                            }
                        }
        return {'admins': {}}

    return {'admins': {}}

```

```

@app.route('/api/hierarchy')
def get_hierarchy()

```

Returns hierarchy filtered by user's role.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **list comprehension** — Builds a list in a single expression ([f(x) for x in xs]). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.

- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `session_token = request.cookies.get('session_token')`.
2. `if not session_token:` branches conditionally.
3. Assigns `user_type = 'super_admin'`.
4. Assigns `user_identifier = 'baller'`.
5. `if session_token:` branches conditionally.
6. Lazy import from `config.hierarchy`.
7. Calls `reload_hierarchy(...)` for its side effect.
8. Assigns `filtered = get_filtered_hierarchy(user_type, user_identifier)`.
9. Lazy import from `dashboard.database`.
10. Imports `copy` (lazy import inside the function).
11. Assigns `all_identities = get_all_client_identities()`.
12. Assigns `enriched = copy.deepcopy(filtered)`.
13. `for admin_data in enriched.get('admins', {}).values():` iterates.
14. `if not enriched.get('admins') or all((not admin_data.get('traders', {}):` branches conditionally.
15. ... and 5 more statement(s) in the body.

Code (copy-paste safe)

```
def get_hierarchy():
    """Returns hierarchy filtered by user's role."""
    session_token = request.cookies.get('session_token')

    if not session_token:
        # Check for simple API key header for scripts
        api_key = request.headers.get('X-API-Key')
        if not api_key:
            return jsonify({"status": "error", "message": "Authentication required"}), 401

    user_type = 'super_admin' # Fallback for trusted scripts
    user_identifier = 'baller'

    if session_token:
        session_info = validate_session(session_token)
        if not session_info:
            return jsonify({"status": "error", "message": "Invalid session"}), 401

        user_type = session_info.get('user_type')
        user_identifier = session_info.get('user_identifier')

    # Reload hierarchy to get latest changes from file
    from config.hierarchy import reload_hierarchy
```

```

reload_hierarchy()

filtered = get_filtered_hierarchy(user_type, user_identifier)

# Enrich client objects with active_status – single bulk query instead of N+1
from dashboard.database import get_all_client_identities
import copy
all_identities = get_all_client_identities()
enriched = copy.deepcopy(filtered)
for admin_data in enriched.get('admins', {}).values():
    for trader_data in admin_data.get('traders', {}).values():
        for client in trader_data.get('clients', []):
            cname = client.get('name', '')
            identity = all_identities.get(cname, {})
            client['active_status'] = identity.get('active_status', 'active')
            # Default split % is always 50; per-period overrides live in split_pct_overrides.
            # Ignore any legacy identity.split_pct that may have been populated from sheet imports.
            client['split_pct'] = 50
            # Hierarchy email is the source of truth (preserves original case).
            # Only fall back to DB identity email if hierarchy has none.
            if not client.get('email'):
                client['email'] = identity.get('email', '')
# Debug logging for empty hierarchy results
if not enriched.get('admins') or all(
    not admin_data.get('traders', {}) for admin_data in enriched.get('admins', {}).values()
):
    logging.warning(
        f"[HIERARCHY] Empty result for user_type={user_type} user_identifier='{user_identifier}' – av

# Shuffle client order daily so traders start with different clients each day
import random
from datetime import date as _date_cls
today_seed = _date_cls.today().toordinal()
for admin_data in enriched.get('admins', {}).values():
    for trader_name, trader_data in admin_data.get('traders', {}).items():
        clients = trader_data.get('clients', [])
        if len(clients) > 1:
            # Seed with today's date + trader name for per-trader unique order
            rng = random.Random(today_seed + hash(trader_name))
            rng.shuffle(clients)

return jsonify(enriched)

@app.route('/api/super_admin/totals')
def get_super_admin_totals()

```

Get aggregated totals across all clients for super admin dashboard.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns `session_token = request.cookies.get('session_token')`.
2. `if not session_token`: branches conditionally.
3. Assigns `session_info = validate_session(session_token)`.
4. `if not session_info or session_info.get('user_type') not in ('super_ad...'`: branches conditionally.
5. Assigns `profile_filter = request.args.get('profile', 'ALL').upper()`.
6. `if session_info.get('user_type') == 'bef_admin'`: branches conditionally.
7. `if session_info.get('user_type') == 'kwok_admin'`: branches conditionally.
8. Assigns `clients = get_client_performance_stats(profile_filter)`.
9. Assigns `t_pay = sum((c.get('payouts', 0) for c in clients))`.
10. Assigns `t_dep = sum((c.get('deposits', 0) for c in clients))`.
11. Assigns `t_fees = sum((c.get('fees', 0) for c in clients))`.
12. Assigns `t_net = sum((c.get('net_profit', 0) for c in clients))`.
13. Assigns `t_hedge = sum((c.get('hedge_profit', 0) for c in clients))`.
14. Assigns `t_farming = sum((c.get('farming_profit', 0) for c in clients))`.
15. ... and 9 more statement(s) in the body.

Code (copy-paste safe)

```
def get_super_admin_totals():
    """Get aggregated totals across all clients for super admin dashboard."""
    session_token = request.cookies.get('session_token')

    if not session_token:
        return jsonify({"status": "error", "message": "Authentication required"}), 401

    # Check session
    # ... (auth check logic is fine, keeping it implicitly via context if needed or re-implementing if I
    # The snippet below replaces the body.

    session_info = validate_session(session_token)
    if not session_info or session_info.get('user_type') not in ('super_admin', 'bef_admin', 'kwok_admin'):
        return jsonify({"status": "error", "message": "Admin access required"}), 403

    profile_filter = request.args.get('profile', 'ALL').upper()

    # BEF admin always sees only BEF profile data
    if session_info.get('user_type') == 'bef_admin':
        profile_filter = 'BEF'
    if session_info.get('user_type') == 'kwok_admin':
        profile_filter = 'ALL'

    # Derive ALL totals from per-client stats (fast – uses precomputed cashflow_inprogress)
    clients = get_client_performance_stats(profile_filter)

    t_pay = sum(c.get('payouts', 0) for c in clients)
    t_dep = sum(c.get('deposits', 0) for c in clients)
    t_fees = sum(c.get('fees', 0) for c in clients)
    t_net = sum(c.get('net_profit', 0) for c in clients)
```

```

t_hedge = sum(c.get('hedge_profit', 0) for c in clients)
t_farming = sum(c.get('farming_profit', 0) for c in clients)
t_active = sum(c.get('active', 0) for c in clients)
t_passed = sum(c.get('passed', 0) for c in clients)
t_failed = sum(c.get('failed', 0) for c in clients)
t_ended = sum(c.get('ended', 0) for c in clients)
t_duration = sum(c.get('total_duration_days', 0) for c in clients)

ev = t_net / t_ended if t_ended > 0 else 0.0
ev_day = t_net / t_duration if t_duration > 0 else 0.0

response_data = {
    "status": "success",
    "totals": {
        "total_payouts": round(t_pay, 2),
        "total_deposits": round(t_dep, 2),
        "total_fees": round(t_fees, 2),
        "total_net_profit": round(t_net, 2),
        "active_accounts": t_active,
        "completed_accounts": t_passed,
        "failed_accounts": t_failed,
        "total_hedge": round(t_hedge, 2),
        "total_farming": round(t_farming, 2),
        "expected_value": round(ev, 2),
        "ev_per_day": round(ev_day, 2)
    },
    "clients": clients
}

return jsonify(response_data)

```

```

@app.route('/api/super_admin/profit_splits')
@require_session
def get_profit_splits()

```

Return per-client current profit split (in progress) for the super admin dashboard.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_session** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.

- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns session_user = request.session_user.
2. if session_user.get('user_type') not in ('super_admin', 'bef_admin', '...': branches conditionally.
3. Assigns profile_filter = request.args.get('profile', 'ALL').upper().
4. if session_user.get('user_type') == 'bef_admin': branches conditionally.
5. if session_user.get('user_type') == 'kwok_admin': branches conditionally.
6. Lazy import from dashboard.watermark_service.
7. Lazy import from dashboard.financial_overview.
8. Lazy import from utils.data_processor.
9. Lazy import from concurrent.futures.
10. Assigns clients_data = _get_cached_clients().
11. Assigns results = [].
12. Assigns _P1_COLS = ['Hedge Result 1', 'Hedge Result 2', 'Hedge Result 3', 'H....
13. Assigns _FUNDED_COLS = ['Hedge Result 1.1', 'Hedge Result 2.1', 'Hedge Result 3.....
14. Assigns _HEDGE_DAY_COLS = [f'Hedge Day {i}' for i in range(1, 51)].
15. ... and 6 more statement(s) in the body.

Code (copy-paste safe)

```
def get_profit_splits():
    """Return per-client current profit split (in progress) for the super admin dashboard."""
    session_user = request.session_user
    if session_user.get('user_type') not in ('super_admin', 'bef_admin', 'kwok_admin'):
        return jsonify({"status": "error", "message": "Admin access required"}), 403

    profile_filter = request.args.get('profile', 'ALL').upper()
    if session_user.get('user_type') == 'bef_admin':
        profile_filter = 'BEF'
    if session_user.get('user_type') == 'kwok_admin':
        profile_filter = 'ALL'

    from dashboard.watermark_service import compute_waterlog_from_db, get_client_split_pct
    from dashboard.financial_overview import _get_cached_clients, get_client_profile
    from utils.data_processor import parse_currency
    from concurrent.futures import ThreadPoolExecutor

    clients_data = _get_cached_clients()
```

```

results = []

# Mirror the client dashboard's `aggregateCashflowFromEvals` (index.html
# ~line 4672) exactly, so the super admin breakdown never disagrees with the
# per-client Profit Split card (#split-profit-amount). The stored
# `statistics.cashflow_inprogress` on disk is sometimes stale, so we
# recompute from raw evaluations here.
# Mirror the sheet (Evaluations tab) formulas exactly:
# Hedging Results = SUM(J:N) + SUM(U:AA)
#                 = P1 hedges (HR 1..5) + ALL funded hedges (HR 1.1..5.1 + HR 6..7)
# Farming Results = SUM(AM:AM, AO:AO, AQ:AQ, ... DA:DA)
#                 = Hedge Day 1..N only (no HR 6/7, no Farming Net override)
_P1_COLS = ['Hedge Result 1', 'Hedge Result 2', 'Hedge Result 3', 'Hedge Result 4', 'Hedge Result 5']
_FUNDED_COLS = [
    'Hedge Result 1.1', 'Hedge Result 2.1', 'Hedge Result 3.1', 'Hedge Result 4.1', 'Hedge Result 5.1',
    'Hedge Result 6', 'Hedge Result 7',
]
_HEDGE_DAY_COLS = [f'Hedge Day {i}' for i in range(1, 51)]

def _live_in_progress_net(evaluations):
    cf_pay = cf_fees = cf_hedge = cf_farm = 0.0
    for ev in (evaluations or []):
        if not ev or ev.get('_deleted'):
            continue
        sp1 = str(ev.get('Status P1', '') or '').lower()
        sf = str(ev.get('Status') or ev.get('Status Funded', '') or '').lower()
        if 'deleted' in sp1 or 'deleted' in sf:
            continue
        p1 = round(sum(parse_currency(ev.get(c)) for c in _P1_COLS), 2)
        fd = round(sum(parse_currency(ev.get(c)) for c in _FUNDED_COLS), 2)
        h_days = round(sum(parse_currency(ev.get(c)) for c in _HEDGE_DAY_COLS), 2)
        fee = parse_currency(ev.get('Fee'))
        act = parse_currency(ev.get('Activation Fee'))
        payouts = round(sum(parse_currency(ev.get(f'Payout {i}')) for i in range(1, 7)), 2)
        row_hedge = round(p1 + fd, 2)
        row_farm = h_days # pure sum of Hedge Day cols - matches sheet
        cf_pay = round(cf_pay + payouts, 2)
        cf_fees = round(cf_fees + fee + act, 2)
        cf_hedge = round(cf_hedge + row_hedge, 2)
        cf_farm = round(cf_farm + row_farm, 2)
    return round(cf_pay + cf_hedge + cf_farm - cf_fees, 2)

def _compute_one(client_id, data):
    try:
        identity = data.get('identity', {})
        real_name = (identity.get('name') or client_id).strip()
        # Profile filter
        source = get_client_profile(client_id, identity)
        if profile_filter != 'ALL' and source != profile_filter:
            return None

        # Mirror the client dashboard's Profit Split card EXACTLY so the super
        # admin "Client Breakdown" matches #split-profit-amount per client.
        #
        # Client dashboard formula (index.html):
        # latestNet = live aggregate from evaluations (aggregateCashflowFromEvals):
        #               payouts + hedging + farming - (fees + activation_fee)
        # prevLow    = periods[-2].low from compute_waterlog_from_db (0 if none)
        # split_pct  = periods[-1].split_pct (default 50)
        # split_amt  = max(0, (latestNet - prevLow) * split_pct / 100)

```

```

latest_net = _live_in_progress_net(data.get('evaluations') or [])

wl = compute_waterlog_from_db(real_name)
prev_low = 0.0
split_pct = 50
if wl and wl.get('periods'):
    periods = wl['periods']
    try:
        split_pct = int(periods[-1].get('split_pct', 50) or 50)
    except (TypeError, ValueError):
        split_pct = 50
    if len(periods) >= 2:
        prev_raw = str(periods[-2].get('low', '$0')).replace('$', '').replace(',', '').strip()
        try:
            prev_low = float(prev_raw)
        except ValueError:
            prev_low = 0.0

split_amt = (
    (latest_net - prev_low) * split_pct / 100.0
    if latest_net > prev_low else 0.0
)

return {
    'client_id': real_name,
    'net_profit': round(latest_net, 2),
    'profit_split_inprogress': round(split_amt, 2),
    'split_pct': split_pct,
}
except Exception as _exc:
    import traceback
    print(f"[profit_splits] error for {client_id}: {_exc}\n{traceback.format_exc()}")
    return None

with ThreadPoolExecutor(max_workers=6) as pool:
    futures = [pool.submit(_compute_one, cid, d) for cid, d in clients_data.items()]
    for f in futures:
        r = f.result()
        if r is not None:
            results.append(r)

total = round(sum(r['profit_split_inprogress'] for r in results), 2)
results.sort(key=lambda x: x['profit_split_inprogress'], reverse=True)
return jsonify({'status': 'success', 'clients': results, 'total': total})

@app.route('/api/client_payouts/<client_id>')
@require_session
def get_client_payouts_detail(client_id)

```

Return individual payout records for a client (used by breakdown modal expand).

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_session** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.

- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns session_user = request.session_user.
2. if session_user.get('user_type') not in ('super_admin', 'bef_admin', '...': branches conditionally.
3. Lazy import from dashboard.financial_overview.
4. Assigns data = get_client_data(client_id).
5. if not data: branches conditionally.
6. Assigns evaluations = data.get('evaluations', []).
7. Assigns records = [].
8. for ev in evaluations: iterates.
9. Calls records.sort(...) for its side effect.
10. Assigns running = 0.0.
11. for r in records: iterates.
12. return jsonify({'payouts': records}).

Code (copy-paste safe)

```
def get_client_payouts_detail(client_id):
    """Return individual payout records for a client (used by breakdown modal expand)."""
    session_user = request.session_user
    if session_user.get('user_type') not in ('super_admin', 'bef_admin', 'kwok_admin'):
        return jsonify({"status": "error"}), 403

    from dashboard.financial_overview import parse_currency, parse_date
    data = get_client_data(client_id)
    if not data:
        return jsonify({"payouts": []})

    evaluations = data.get('evaluations', [])
    records = []
    for ev in evaluations:
        if not isinstance(ev, dict):
            continue
        prop_firm = str(ev.get('Prop Firm') or 'Unknown').strip() or 'Unknown'
        account = str(ev.get('Account #') or ev.get('Account #.1') or '-').strip()
        for i in range(1, 10):
            amt = parse_currency(ev.get(f'Payout {i}'))
            if amt != 0:
                pdate = str(ev.get(f'Date {i}') or '-').strip()
                d_obj = parse_date(pdate)
```



```

        records.append({
            "prop_firm": prop_firm,
            "account": account,
            "amount": round(amt, 2),
            "date": pdate,
            "_sort": d_obj.isoformat() if d_obj else "0000-00-00"
        })

records.sort(key=lambda r: r['_sort'])
# Add running total
running = 0.0
for r in records:
    running += r['amount']
    r['running_total'] = round(running, 2)
    del r['_sort']

return jsonify({"payouts": records})

@app.route('/api/super_admin/recalculate_stats', methods=['POST'])
@require_session
def recalculate_all_stats()

```

Recalculate and save statistics for all clients from stored evaluations.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_session** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

Step by step (top-level body)

1. **if** request.session_user.get('user_type') != 'super_admin': branches conditionally.
2. Lazy import from utils.data_processor.
3. Assigns all_clients = get_all_clients().
4. Assigns results = [].
5. **for** (client_id, client_data) in all_clients.items(): iterates.
6. **return** jsonify({'status': 'success', 'recalculated': len(results), 'result....

Code (copy-paste safe)

```

def recalculate_all_stats():
    """Recalculate and save statistics for all clients from stored evaluations."""
    if request.session_user.get('user_type') != 'super_admin':
        return jsonify({"status": "error", "message": "Unauthorized"}), 403
    from utils.data_processor import calculate_statistics
    all_clients = get_all_clients()
    results = []
    for client_id, client_data in all_clients.items():
        try:

```

```

evals = client_data.get('evaluations', [])
evals = recalculate_hedge_nets(evals)
existing_mt5 = client_data.get('account')
existing_hr = client_data.get('statistics', {}).get('hedging_review', {})
existing_hist = existing_hr.get('historical_accounts')
old_fees = client_data.get('statistics', {}).get('profitability_completed', {}).get(
    'challenge_fees', 0)
new_stats = calculate_statistics(evals, mt5_account=existing_mt5,
    historical_accounts=existing_hist)
# ALWAYS preserve hedging_review MT5-derived fields
new_hr = new_stats.setdefault('hedging_review', {})
new_hr['total_deposits'] = existing_hr.get('total_deposits', 0)
new_hr['total_withdrawals'] = existing_hr.get('total_withdrawals', 0)
new_hr['current_balance'] = existing_hr.get('current_balance', 0)
new_hr['actual_hedging_results'] = existing_hr.get('actual_hedging_results', 0)
# Preserve historical account fields
if existing_hist:
    new_hr['historical_accounts'] = existing_hist
    new_hr['historical_deposits'] = existing_hr.get('historical_deposits', 0)
    new_hr['historical_withdrawals'] = existing_hr.get('historical_withdrawals', 0)
    new_hr['historical_balance'] = existing_hr.get('historical_balance', 0)
# Recalculate discrepancy + net_profit with preserved MT5 values
new_hr['discrepancy'] = round(new_hr['actual_hedging_results'] - new_hr.get(
    'sheet_hedging_results', 0), 2)
# Recalculate net_profit with discrepancy (match frontend formula)
disc = new_hr['discrepancy']
for sk in ["profitability_completed", "cashflow_inprogress"]:
    sec = new_stats[sk]
    sec["net_profit"] = round(sec["payouts"] + sec["hedging_results"] + sec[
        "farming_results"] + disc - sec["challenge_fees"], 2)
    new_fees = new_stats.get('profitability_completed', {}).get('challenge_fees', 0)
    save_client_data(client_id, {'evaluations': evals, 'statistics': new_stats})
    results.append({"client_id": client_id, "old_fees": old_fees, "new_fees": new_fees,
        "changed": abs(float(new_fees) - float(old_fees)) > 0.01})
except Exception as e:
    results.append({"client_id": client_id, "error": str(e)})
return jsonify({"status": "success", "recalculated": len(results), "results": results})

@app.route('/api/super_admin/cleanup_database', methods=['POST'])
@require_session
def api_cleanup_database()

```

Run database cleanup: prune old history, audit log, expired sessions.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_session** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.

Step by step (top-level body)

1. `if request.session_user.get('user_type') != 'super_admin':` branches conditionally.
2. Lazy import from `dashboard.database`.
3. Assigns `results = cleanup_database()`.

4. **return** jsonify({'status': 'success', 'cleaned': results}).

Code (copy-paste safe)

```
def api_cleanup_database():
    """Run database cleanup: prune old history, audit log, expired sessions."""
    if request.session_user.get('user_type') != 'super_admin':
        return jsonify({"status": "error", "message": "Unauthorized"}), 403
    from dashboard.database import cleanup_database
    results = cleanup_database()
    return jsonify({"status": "success", "cleaned": results})

@app.route('/api/client/update_source', methods=['POST'])
@require_session
def update_client_source():

    Update client source (BEF/Private).
```

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_session** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `session_user = request.session_user`.
2. **if** `session_user.get('user_type') != 'super_admin'`: branches conditionally.
3. Assigns `data = request.get_json(silent=True)` or `{}`.
4. Assigns `client_id = data.get('client_id')`.
5. Assigns `source = data.get('source')`.
6. Assigns `allowed = ['BEF', 'Private']`.
7. **if** `not client_id` or `source not in allowed`: branches conditionally.
8. Assigns `client_data = get_client_data(client_id)`.
9. **if** `not client_data`: branches conditionally.
10. Assigns `identity = client_data.get('identity', {})` or `{}`.
11. Assigns `identity['profile'] = source`.
12. Assigns `identity['category'] = source`.
13. Assigns `identity['source'] = source`.
14. Calls `update_client_field(...)` for its side effect.

15. ... and 5 more statement(s) in the body.

Code (copy-paste safe)

```
def update_client_source():
    """Update client source (BEF/Private)."""
    session_user = request.session_user
    if session_user.get('user_type') != 'super_admin':
        return jsonify({"status": "error", "message": "Unauthorized"}), 403

    data = request.get_json(silent=True) or {}
    client_id = data.get('client_id')
    source = data.get('source')
    allowed = ['BEF', 'Private']

    if not client_id or source not in allowed:
        return jsonify({"status": "error", "message": "Invalid data"}), 400

    # Update Database
    client_data = get_client_data(client_id)
    if not client_data:
        return jsonify({"status": "error", "message": "Client not found"}), 404

    identity = client_data.get('identity', {}) or {}
    # Update all relevant fields for consistency
    identity['profile'] = source
    identity['category'] = source
    identity['source'] = source
    update_client_field(client_id, 'identity', identity)

    # clear cache
    from dashboard.financial_overview import clear_financial_cache
    clear_financial_cache()

    # Also update Hierarchy.json if possible
    try:
        from config.hierarchy import get_client_profile, update_client_category
        profile = get_client_profile(client_id)
        if profile:
            update_client_category(profile['admin'], profile['trader'], client_id, source)
    except Exception as e:
        log_action('UPDATE_CLIENT_SOURCE', 'super_admin', client_id, get_remote_address(), str(e),
                  success=False)

    log_action('UPDATE_CLIENT_SOURCE', 'super_admin', client_id, get_remote_address(), f"To: {source}")
    return jsonify({"status": "success"})

@app.route('/api/client/lookup', methods=['POST'])
@require_api_key
def api_client_lookup()
```

Look up client hierarchy info by email.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.

- **@require_api_key** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.

Step by step (top-level body)

1. Assigns `email = request.json.get('email', '').strip()`.
2. **if not email:** branches conditionally.
3. Assigns `client = get_client_by_email(email)`.
4. **if client:** branches conditionally.
5. **return** `(jsonify({'status': 'error', 'message': 'Email not found in system'....`

Code (copy-paste safe)

```
def api_client_lookup():
    """Look up client hierarchy info by email."""
    email = request.json.get('email', '').strip()

    if not email:
        return jsonify({"status": "error", "message": "Email required"}), 400

    client = get_client_by_email(email)
    if client:
        return jsonify({
            "status": "success",
            "client": client['client'],
            "trader": client['trader'],
            "admin": client['admin'],
            "email": client['email']
        })

    return jsonify({"status": "error", "message": "Email not found in system"}), 404

@app.route('/api/check_email', methods=['GET', 'POST', 'OPTIONS'], strict_slashes=False)
def api_check_email():

    Check whether an email (or username) exists in the system. Accepts both GET (?email=...) and POST (JSON body: {"email": "..."}). Requires X-API-Key header (accepts both 'full' and 'readonly' scoped keys). Response (200): {"exists": true, "user_type": "client", "username": "John Doe"} {"exists": false}
```

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.

Step by step (top-level body)

1. **if request.method == 'OPTIONS':** branches conditionally.
2. Assigns `api_key = request.headers.get('X-API-Key')`.
3. Assigns `client_ip = get_remote_address()`.
4. **if not api_key:** branches conditionally.

5. Assigns `user_info = validate_api_key(api_key)`.
6. **if** not `user_info`: branches conditionally.
7. **if** `request.method == 'POST'`: branches conditionally (with an **else**/**elif** arm).
8. **if** not `email`: branches conditionally.
9. Lazy import from `dashboard.database`.
10. Lazy import from `config.hierarchy`.
11. Defines a nested function `_find_client_email(...)`.
12. Assigns `found = _find_client_email(email)`.
13. **if** `found`: branches conditionally.
14. **return** `jsonify({'exists': False})`.

Code (copy-paste safe)

```
def api_check_email():
    """
    Check whether an email (or username) exists in the system.

    Accepts both GET (?email=...) and POST (JSON body: {"email": "..."}).
    Requires X-API-Key header (accepts both 'full' and 'readonly' scoped keys).

    Response (200):
        {"exists": true, "user_type": "client", "username": "John Doe"}
        {"exists": false}
    """
    # Handle CORS preflight
    if request.method == 'OPTIONS':
        response = jsonify({"status": "ok"})
        response.headers['Access-Control-Allow-Origin'] = '*'
        response.headers['Access-Control-Allow-Methods'] = 'GET, POST, OPTIONS'
        response.headers['Access-Control-Allow-Headers'] = 'Content-Type, X-API-Key'
        return response, 200

    # Manual key validation – accepts both 'full' and 'readonly' scope
    api_key = request.headers.get('X-API-Key')
    client_ip = get_remote_address()
    if not api_key:
        return jsonify({"status": "error", "message": "API key required"}), 401
    user_info = validate_api_key(api_key)
    if not user_info:
        log_action('API_ACCESS_DENIED', 'unknown', api_key[:12], client_ip, 'Invalid API key on check_email')
        return jsonify({"status": "error", "message": "Invalid API key"}), 403
    if request.method == 'POST':
        data = request.get_json(silent=True) or {}
        email = data.get('email', '').strip()
    else:
        email = request.args.get('email', '').strip()

    if not email:
        return jsonify({"status": "error", "message": "email parameter required"}), 400

    from dashboard.database import find_user_by_identifier
    from config.hierarchy import reload_hierarchy
```

```

def _find_client_email(search_email):
    """Check both user_credentials (DB) and hierarchy.json for a client with this email."""
    search_email_lower = search_email.lower()
    # 1. Check DB
    user = find_user_by_identifier(search_email)
    if user and user.get('user_type') == 'client':
        return {'username': user.get('username'), 'source': 'db'}
    # 2. Check hierarchy.json
    h = reload_hierarchy()
    for admin_data in h.get('admins', {}).values():
        for trader_data in admin_data.get('traders', {}).values():
            for client in trader_data.get('clients', []):
                if (client.get('email') or '').lower() == search_email_lower:
                    return {'username': client.get('name', ''), 'source': 'hierarchy'}
    return None

found = _find_client_email(email)
if found:
    return jsonify({
        "exists": True,
        "user_type": "client",
        "username": found['username']
    })

return jsonify({"exists": False})

```

```

@app.route('/api/list_emails', methods=['GET', 'POST', 'OPTIONS'], strict_slashes=False)
def api_list_emails()

```

GET — Return all emails registered in the system (each entry includes exists: true). POST — Bulk check: pass {"emails": ["a@b.com", ...]} and get back exists true/false per email. Requires X-API-Key header (accepts both 'full' and 'readonly' scoped keys). GET query params: ?user_type=client|trader|admin — filter by role (default: all) ?active_only=true — only include active accounts (default: true) GET response: {"count": 2, "emails": [{"email": "john@example.com", "username": "John", "user_type": "client", "exists": true}, ...]} POST response: {"results": [{"email": "john@example.com", "exists": true, "user_type": "client", "username": "John"}, {"email": "unknown@x.com", "exists": false}]}

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.

Step by step (top-level body)

1. `if request.method == 'OPTIONS':` branches conditionally.
2. Assigns `api_key = request.headers.get('X-API-Key')`.

3. Assigns `client_ip = get_remote_address()`.
4. `if not api_key`: branches conditionally.
5. Assigns `user_info = validate_api_key(api_key)`.
6. `if not user_info`: branches conditionally.
7. Lazy import from `dashboard.database`.
8. Lazy import from `config.hierarchy`.
9. Defines a nested function `_all_hierarchy_clients(...)`.
10. Defines a nested function `_find_client_email(...)`.
11. `if request.method == 'POST'`: branches conditionally.
12. Assigns `active_only = request.args.get('active_only', 'true').lower() != 'false'`.
13. Assigns `seen_emails = set()`.
14. Assigns `results = []`.
15. ... and 4 more statement(s) in the body.

Code (copy-paste safe)

```
def api_list_emails():
    """
    GET – Return all emails registered in the system (each entry includes exists: true).
    POST – Bulk check: pass {"emails": ["a@b.com", ...]} and get back exists true/false per email.

    Requires X-API-Key header (accepts both 'full' and 'readonly' scoped keys).

    GET query params:
        ?user_type=client|trader|admin – filter by role (default: all)
        ?active_only=true – only include active accounts (default: true)

    GET response:
        {"count": 2, "emails": [
            {"email": "john@example.com", "username": "John", "user_type": "client", "exists": true},
            ...
        ]}

    POST response:
        {"results": [
            {"email": "john@example.com", "exists": true, "user_type": "client", "username": "John"},
            {"email": "unknown@x.com", "exists": false}
        ]}
    """
    if request.method == 'OPTIONS':
        response = jsonify({"status": "ok"})
        response.headers['Access-Control-Allow-Origin'] = '*'
        response.headers['Access-Control-Allow-Methods'] = 'GET, POST, OPTIONS'
        response.headers['Access-Control-Allow-Headers'] = 'Content-Type, X-API-Key'
        return response, 200

    # Validate key – accepts readonly and full scopes
    api_key = request.headers.get('X-API-Key')
    client_ip = get_remote_address()
    if not api_key:
        return jsonify({"status": "error", "message": "API key required"}), 401
    user_info = validate_api_key(api_key)
```


[illegible]

```

active_only = request.args.get('active_only', 'true').lower() != 'false'

# Collect from DB
seen_emails = set()
results = []
for u in list_users(user_type='client'):
    if active_only and not u.get('is_active'):
        continue
    email = (u.get('email') or '').strip()
    if email:
        seen_emails.add(email.lower())
        results.append({'email': email, 'username': u.get('username'), 'user_type': 'client',
                        'exists': True})

# Also collect from hierarchy.json (clients not in DB)
for c in _all_hierarchy_clients():
    if c['email'].lower() not in seen_emails:
        seen_emails.add(c['email'].lower())
        results.append({'email': c['email'], 'username': c['username'], 'user_type': 'client',
                        'exists': True})

log_action('API_ACCESS', 'reader', user_info.get('trader', 'unknown'), client_ip,
          f"list_emails: {len(results)} returned")
return jsonify({"count": len(results), "emails": results})

@app.route('/api/client/auth', methods=['POST'])
@limiter.limit('30 per minute')
def api_client_auth()

```

*Public endpoint - authenticate client by email only. Returns client hierarchy info if email exists in system.
No API key required - just the client email.*

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@limiter.limit** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def api_client_auth():
    """
    Public endpoint - authenticate client by email only.
    Returns client hierarchy info if email exists in system.
    No API key required - just the client email.
    """

```

```

try:
    data = request.get_json(silent=True)
    if not data:
        return jsonify({"status": "error", "message": "Invalid JSON or Content-Type"}), 400

    email = data.get('email', '').strip().lower()

    if not email:
        return jsonify({"status": "error", "message": "Email required"}), 400

    client = get_client_by_email(email)

    # Safe logging
    try:
        remote_addr = get_remote_address()
    except:
        remote_addr = "0.0.0.0"

    if client:
        try:
            log_action('CLIENT_AUTH', 'client', email, remote_addr, 'Email verified')
        except Exception as e:
            print(f"Log action error: {e}", file=sys.stderr)

        return jsonify({
            "status": "success",
            "identity": {
                "admin": client['admin'],
                "trader": client['trader'],
                "client": client['client'],
                "email": client['email'],
                "category": client.get('category', '')
            }
        })

    try:
        log_action('CLIENT_AUTH_FAILED', 'client', email, remote_addr, 'Email not found', False)
    except:
        pass

    return jsonify({"status": "error", "message": "Email not registered in the system"}), 404

except Exception as e:
    import traceback
    traceback.print_exc()
    return jsonify({"status": "error", "message": str(e)}), 500

@app.route('/api/client/data', methods=['POST'])
@limiter.limit('30 per minute')
def api_client_data()

```

Public endpoint - fetch client evaluations by email. Used by Trader Companion to load active trades list.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.

- **@limiter.limiter** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

Step by step (top-level body)

1. Lazy import from dashboard.database.
2. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def api_client_data():
    """
    Public endpoint - fetch client evaluations by email.
    Used by Trader Companion to load active trades list.
    """
    from dashboard.database import get_client_data

    try:
        data = request.get_json(silent=True)
        if not data:
            return jsonify({"status": "error", "message": "Invalid JSON"}), 400

        email = (data.get('email') or '').strip().lower()
        if not email:
            return jsonify({"status": "error", "message": "Email required"}), 400

        client_info = get_client_by_email(email)
        if not client_info:
            return jsonify({"status": "error", "message": "Email not registered"}), 404

        client_id = client_info['client']
        client_data = get_client_data(client_id)
        if not client_data:
            return jsonify({"status": "error", "message": "No data found for client"}), 404

        # Mark each evaluation with is_active using the same logic as data_processor
        evaluations = client_data.get("evaluations", [])
        for ev in evaluations:
            status_p1 = (ev.get("Status P1") or "").strip()
            status_funded = (ev.get("Status") or "").strip()
            is_p1_fail = status_p1 == "Fail"
            is_funded_fail = status_funded == "Fail"
            is_funded_completed = status_funded == "Completed"
            is_funded_ended = is_funded_fail or is_funded_completed
            ev["_is_active"] = not is_p1_fail and not is_funded_ended

        return jsonify({
            "status": "success",
            "evaluations": evaluations,
            "prop_accounts": client_data.get("prop_accounts", []),
            "hedge_accounts": client_data.get("hedge_accounts", []),
            "mt5_credentials": client_data.get("mt5_credentials", {}),
            "identity": {
                "client": client_info['client'],
                "trader": client_info['trader'],
            }
        })
```

```

        "admin": client_info['admin'],
    }
})

except Exception as e:
    import traceback
    traceback.print_exc()
    return jsonify({"status": "error", "message": str(e)}), 500

def _drop_balance_deals(deals)

```

Remove internal-transfer style MT5 deals (BALANCE/CREDIT) from a pushed deal list. Requirement: these internal transfers should "not exist" for the dashboard: they should not be stored, displayed, or used in server-side calculations.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with `append`, and clearer about intent: this is a transform, not a side-effecting loop.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.

Step by step (top-level body)

1. **if** not deals: branches conditionally.
2. **if** not isinstance(deals, list): branches conditionally.
3. Defines a nested function `_is_internal(...)`.
4. Assigns `filtered = [d for d in deals if not _is_internal(d)]`.
5. Assigns `dropped = len(deals) - len(filtered)`.
6. **return** (filtered, dropped).

Code (copy-paste safe)

```

def _drop_balance_deals(deals):
    """
    Remove internal-transfer style MT5 deals (BALANCE/CREDIT) from a pushed deal list.

    Requirement: these internal transfers should "not exist" for the dashboard:
    they should not be stored, displayed, or used in server-side calculations.
    """
    if not deals:
        return [], 0
    if not isinstance(deals, list):
        return deals, 0

    def _is_internal(d):
        if not isinstance(d, dict):

```

```

        return False

    raw_type = d.get('type', '')
    raw_entry = d.get('entry', '')

    # Numeric check (raw MT5 / DataFrame): catches 2, 2.0, 3, 3.0, etc.
    try:
        if int(float(raw_type)) in _INTERNAL_DEAL_TYPES_INT:
            return True
    except (ValueError, TypeError):
        pass

    # String check (JSON payloads / serialized exports)
    str_type = str(raw_type).strip().upper()
    str_entry = str(raw_entry).strip().upper()

    return (
        str_type in _INTERNAL_DEAL_TYPES_STR
        or str_entry in _INTERNAL_DEAL_TYPES_STR
        or ('BALANCE' in str_type)
        or ('CREDIT' in str_type)
    )

    filtered = [d for d in deals if not _is_internal(d)]
    dropped = len(deals) - len(filtered)
    return filtered, dropped

@app.route('/api/client/push', methods=['POST'])
@limiter.limit('60 per minute')
def api_client_push()

```

Public endpoint - push data using client email only (no API key). Automatically looks up hierarchy from email. Recalculates statistics using MT5 deals/account data if provided.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@limiter.limit** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.

- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns data = request.json.
2. Assigns email = data.get('email', '').strip().lower().
3. if not email: branches conditionally.
4. Assigns client_info = get_client_by_email(email).
5. if not client_info: branches conditionally.
6. Assigns admin_id = client_info['admin'].
7. Assigns trader_id = client_info['trader'].
8. Assigns client_id = client_info['client'].
9. Assigns mt5_deals_raw = data.get('deals', []).
10. Assigns (mt5_deals, dropped_internal) = _drop_balance_deals(mt5_deals_raw).
11. if dropped_internal: branches conditionally.
12. Assigns mt5_account = data.get('account', {}).
13. Assigns existing_data = get_client_data(client_id) or {}.
14. if 'evaluations' in data and data['evaluations']: branches conditionally (with an **else/elif** arm).
15. ... and 60 more statement(s) in the body.

Code (copy-paste safe)

```
def api_client_push():
    """
    Public endpoint - push data using client email only (no API key).
    Automatically looks up hierarchy from email.
    Recalculates statistics using MT5 deals/account data if provided.
    """
    data = request.json
    email = data.get('email', '').strip().lower()

    if not email:
        return jsonify({"status": "error", "message": "Email required"}), 400

    # Look up client by email
    client_info = get_client_by_email(email)
    if not client_info:
        return jsonify({"status": "error", "message": "Email not registered in the system"}), 404

    admin_id = client_info['admin']
    trader_id = client_info['trader']
    client_id = client_info['client']

    # Get MT5 data from push
    mt5_deals_raw = data.get("deals", [])
    mt5_deals, dropped_internal = _drop_balance_deals(mt5_deals_raw)
```

```

if dropped_internal:
    app.logger.info(
        f"■ Dropped {dropped_internal} internal transfer deal(s) (BALANCE/CREDIT) from push payload"
    )
    mt5_account = data.get("account", {})

# Get existing data to merge evaluations if needed
existing_data = get_client_data(client_id) or {}

# Only use new evaluations if explicitly provided and not empty
# If "evaluations" key is missing or None, preserve existing data
if "evaluations" in data and data["evaluations"]:
    incoming_evals = data["evaluations"]
    existing_evals_push = existing_data.get('evaluations', [])
    force_fields = set(data.get('force_fields', []))
    evaluations = merge_evaluation_push_with_existing(
        existing_evals_push, incoming_evals, force_fields)
    app.logger.info(
        f"    Merged {len(incoming_evals)} incoming evaluation row(s) "
        f"into {len(evaluations)} total (was {len(existing_evals_push)} in DB)")
else:
    evaluations = existing_data.get("evaluations", [])
    app.logger.info(f"    Preserving {len(evaluations)} EXISTING evaluations")

# Normalize Account Size values to standard format
evaluations = normalize_evaluations(evaluations)

# Check for aggregated comment data (from Push by Comment feature) OR raw deals
aggregated_by_comment = data.get("aggregated_by_comment", [])
prefer_client_aggregation = bool(data.get("prefer_client_aggregation"))
comment_summary = data.get("comment_summary", {})
tradovate_farming_days = data.get("tradovate_farming_days", [])
hedge_match_log = []

if aggregated_by_comment or mt5_deals:
    app.logger.info(
        f"■ Received {len(aggregated_by_comment)} aggregated groups, {len(mt5_deals)} raw deals")
    if prefer_client_aggregation:
        app.logger.info("■ Preferring client-side aggregation for hedge matching")
    if tradovate_farming_days:
        app.logger.info(f"■ Received Tradovate farming data for {len(tradovate_farming_days)} accounts")

# Update evaluations with hedge results from aggregated data OR raw deals
if evaluations:
    app.logger.info(f"■ Matching hedge results to evaluations...")
    evaluations, hedge_match_log, generated_sessions = update_evaluations_from_aggregated_data(
        evaluations, aggregated_data=aggregated_by_comment,
        raw_deals=None if prefer_client_aggregation else mt5_deals,
        tradovate_farming_days=tradovate_farming_days)

# If server-side aggregation occurred, use THAT instead of the client's.
if generated_sessions:
    aggregated_by_comment = generated_sessions
    app.logger.info(
        f"■ Replaced client aggregation with {len(generated_sessions)} server-side sessions")

if hedge_match_log:
    app.logger.info(f"■ Hedge matching produced {len(hedge_match_log)} log entries")
    if app.logger.isEnabledFor(logging.DEBUG):
        for log_line in hedge_match_log:
            app.logger.debug(f"    {log_line}")

```



```

# Debug logging
acct_balance = mt5_account.get('balance', 0) if mt5_account else 0
app.logger.info(
    f"■ Push for {client_id}: {len(mt5_deals)} deals, balance={acct_balance}, {len(evaluations)} evals"

# Log detailed MT5 account info
if mt5_account:
    app.logger.info(f"■ MT5 Account Details:")
    app.logger.info(f"    - balance: ${mt5_account.get('balance', 0):.2f}")
    app.logger.info(f"    - total_deposits: ${mt5_account.get('total_deposits', 0):.2f}")
    app.logger.info(f"    - total_withdrawals: ${mt5_account.get('total_withdrawals', 0):.2f}")
    app.logger.info(f"    - equity: ${mt5_account.get('equity', 0):.2f}")
    app.logger.info(f"    - account_number: {mt5_account.get('account_number', 'N/A')}")

# Log deal types and counts to debug
if mt5_deals:
    deal_types = {}
    for d in mt5_deals:
        dtype = str(d.get('type', 'unknown'))
        deal_types[dtype] = deal_types.get(dtype, 0) + 1
    app.logger.info(f"■ Deal types: {deal_types}")

# Log evaluation hedging/payout data summary
app.logger.info(f"■ Evaluation Rows Summary:")
for i, ev in enumerate(evaluations):
    prop_firm = ev.get('Prop Firm', 'Unknown')
    status_p1 = ev.get('Status P1', '')
    status_fd = ev.get('Status') or ev.get('Status Funded', '')

    # Collect hedge results
    hedge_results = []
    for j in range(1, 8):
        key = f'Hedge Result {j}' if j < 6 else f'Hedge Result {j-5}.1' if j == 6 else f'Hedge Result {j-5}.2'
        val = ev.get(key, '')
        if val and str(val).strip() not in ('', '-', '$0'):
            hedge_results.append(f"{key}={val}")

    # Collect payouts
    payouts = []
    for j in range(1, 10):
        key = f'Payout {j}'
        val = ev.get(key, '')
        if val and str(val).strip() not in ('', '-', '$0'):
            payouts.append(f"{key}={val}")

    hedge_net = ev.get('Hedge Net', '')
    hedge_net_1 = ev.get('Hedge Net.1', '')
    fee = ev.get('Fee', '')
    act_fee = ev.get('Activation Fee', '')

    row_info = f"    [{i}] {prop_firm} | P1:{status_p1} FD:{status_fd}"
    if hedge_results:
        row_info += f" | Hedges: {' '.join(hedge_results)}"
    if payouts:
        row_info += f" | Payouts: {' '.join(payouts)}"
    if hedge_net and str(hedge_net).strip() not in ('', '-'):
        row_info += f" | Hedge Net={hedge_net}"
    if hedge_net_1 and str(hedge_net_1).strip() not in ('', '-'):
        row_info += f" | Hedge Net.1={hedge_net_1}"
    if fee and str(fee).strip() not in ('', '-', '$0'):
        row_info += f" | Fee={fee}"

```

```

        row_info += f" | Fee={fee}"
    if act_fee and str(act_fee).strip() not in ('', '-', '$0'):
        row_info += f" | Activation={act_fee}"

    app.logger.info(row_info)

# Log deal types to debug
if mt5_deals:
    deal_types = [str(d.get('type', 'unknown')) for d in mt5_deals[:5]]
    app.logger.info(f"    Sample deal types: {deal_types}")

# ALWAYS recalculate statistics when we have evaluations or MT5 data
# This ensures discrepancy is only calculated when we have actual MT5 data
statistics = data.get("statistics", {})
push_sheet_url = existing_data.get('sheet_url') or (existing_data.get('identity') or {}).get('sheet_url')

# Recalculate Hedge Net / Hedge Net.1 before stats so formulas stay current
evaluations = recalculate_hedge_nets(evaluations)

if evaluations or mt5_deals or mt5_account:
    try:
        from utils.data_processor import calculate_statistics

        # Log what we're passing to calculate_statistics
        app.logger.info(f"■ Calling calculate_statistics with:")
        app.logger.info(f"    - mt5_account type: {type(mt5_account)}, has data: {bool(mt5_account)}")
        if mt5_account:
            app.logger.info(f"    - mt5_account.balance: {mt5_account.get('balance', 'N/A')}")
            app.logger.info(
                f"    - mt5_account.total_deposits: {mt5_account.get('total_deposits', 'N/A')}")
            app.logger.info(
                f"    - mt5_account.total_withdrawals: {mt5_account.get('total_withdrawals', 'N/A')}")

        # Pass MT5 data - if empty, discrepancy will be 0
        mt5_acc_param = mt5_account if mt5_account else None
        mt5_deals_param = mt5_deals if mt5_deals else None

        # Fetch Stats tab values from Google Sheet so the SUMIF stats
        # use formula-precision values instead of CSV-rounded values.
        # Cached per sheet_url with a 5-minute TTL to avoid blocking on every push.
        push_xlsx_notes = None
        if push_sheet_url:
            import time as _time
            _now = _time.time()
            _cached = _stats_tab_cache.get(push_sheet_url)
            if _cached and (_now - _cached[0]) < _STATS_TAB_TTL:
                push_xlsx_notes = _cached[1]
                app.logger.info(f"■ Stats tab served from cache (age {int(_now - _cached[0]))s}")
            else:
                # Never block the push on a live sheet fetch. Use cache if present,
                # and refresh in the background for the next request.
                if _cached:
                    push_xlsx_notes = _cached[1]
                    app.logger.info("■ Stats tab using stale cache while refreshing in background")
                else:
                    app.logger.info("■ Stats tab cache miss - skipping live fetch for this push")

            if push_sheet_url not in _stats_tab_cache_refreshing:
                _stats_tab_cache_refreshing.add(push_sheet_url)

            def _refresh_stats_tab_cache(sheet_url=push_sheet_url):

```

```

try:
    from utils.data_processor import fetch_evaluations as _fe
    _result = _fe(sheet_url)
    if isinstance(_result, tuple) and len(_result) == 2:
        _, _notes = _result
        _stats_tab_cache[sheet_url] = (_time.time(), _notes)
        if _notes and '__stats_tab__' in _notes:
            app.logger.info("■ Stats tab fetched in background and cached")
except Exception as _e:
    app.logger.warning(f"Stats tab background fetch failed (non-critical): {_e}")
finally:
    _stats_tab_cache_refreshing.discard(sheet_url)

threading.Thread(target=_refresh_stats_tab_cache, daemon=True).start()

statistics = calculate_statistics(evaluations, mt5_deals_param, mt5_acc_param,
                                xlsx_notes=push_xlsx_notes,
                                historical_accounts=existing_data.get('statistics', {}).get(
                                    'hedging_review', {}).get('historical_accounts'))

# Preserve historical MT5 accounts from existing data
existing_hr = existing_data.get('statistics', {}).get('hedging_review', {})
if 'historical_accounts' in existing_hr:
    statistics.setdefault('hedging_review', {})[ 'historical_accounts' ] = existing_hr[
        'historical_accounts' ]
    statistics[ 'hedging_review' ][ 'historical_deposits' ] = existing_hr.get('historical_deposits',
    0)
    statistics[ 'hedging_review' ][ 'historical_withdrawals' ] = existing_hr.get(
        'historical_withdrawals', 0)
    statistics[ 'hedging_review' ][ 'historical_balance' ] = existing_hr.get('historical_balance', 0)

# ALWAYS preserve hedging_review MT5-derived fields from existing data.
# Only manual edits via /api/hedging_review or /api/client/push_hedging_review can set MT5 fields
# Push Data should never overwrite live hedging review values.
app.logger.info(f"■ Preserving hedging review MT5 values – push data never overwrites")
new_hr = statistics.get('hedging_review', {})
# Preserve MT5-derived fields from existing manual/push edits
new_hr[ 'total_deposits' ] = existing_hr.get('total_deposits', 0)
new_hr[ 'total_withdrawals' ] = existing_hr.get('total_withdrawals', 0)
new_hr[ 'current_balance' ] = existing_hr.get('current_balance', 0)
new_hr[ 'actual_hedging_results' ] = existing_hr.get('actual_hedging_results', 0)
# Preserve historical account fields
if existing_hr.get('historical_accounts'):
    new_hr[ 'historical_accounts' ] = existing_hr[ 'historical_accounts' ]
    new_hr[ 'historical_deposits' ] = existing_hr.get('historical_deposits', 0)
    new_hr[ 'historical_withdrawals' ] = existing_hr.get('historical_withdrawals', 0)
    new_hr[ 'historical_balance' ] = existing_hr.get('historical_balance', 0)
# Recalculate discrepancy with preserved MT5 values + fresh sheet values
new_hr[ 'discrepancy' ] = round(new_hr[ 'actual_hedging_results' ] - new_hr.get(
    'sheet_hedging_results', 0), 2)
statistics[ 'hedging_review' ] = new_hr
# Recalculate net_profit with the corrected discrepancy
disc = new_hr[ 'discrepancy' ]
for section_key in [ "profitability_completed", "cashflow_inprogress" ]:
    sec = statistics[ section_key ]
    sec[ "net_profit" ] = round(sec[ "payouts" ] + sec[ "hedging_results" ] + sec[
        "farming_results" ] - sec[ "challenge_fees" ] + disc, 2)

# Log the hedging review results
hr = statistics.get('hedging_review', {})

```

```

app.logger.info(f"Stats calculated:")
app.logger.info(f" - Current balance: ${hr.get('current_balance', 0):.2f}")
app.logger.info(f" - Total deposits: ${hr.get('total_deposits', 0):.2f}")
app.logger.info(f" - Total withdrawals: ${hr.get('total_withdrawals', 0):.2f}")
app.logger.info(f" - Actual hedging: ${hr.get('actual_hedging_results', 0):.2f}")

# Debug info
if '_debug_deal_count' in hr:
    app.logger.info(
        f" - Debug: {hr.get('_debug_deal_count')} deals processed, types seen: {hr.get('_de
except Exception as e:
    app.logger.error(f"Error recalculating stats: {e}")
    import traceback
    app.logger.error(traceback.format_exc())
    # Keep the provided statistics if recalc fails

# Merge incoming account onto existing instead of replacing.
# The lightweight push sends total_deposits/total_withdrawals as 0 (deal-history
# walk is skipped for speed); the follow-up /api/client/push_hedging_review call
# fills them in. If we let the zeros overwrite, any push where the HR follow-up
# fails (MT5 disconnect, history_deals_get timeout, network error) leaves the
# panel showing $0 until the next successful HR push.
incoming_acct = mt5_account or {}
existing_acct = existing_data.get("account") or {}
merged_acct = dict(existing_acct)
merged_acct.update(incoming_acct)
for _preserve_key in ("total_deposits", "total_withdrawals"):
    if not incoming_acct.get(_preserve_key):
        merged_acct[_preserve_key] = existing_acct.get(_preserve_key, 0)

# Prepare client data
client_data = {
    "deals": mt5_deals,
    "positions": data.get("positions", []),
    "account": merged_acct,
    "evaluations": evaluations,
    "statistics": statistics,
    "dropdown_options": data.get("dropdown_options", {}),
    "identity": {
        "admin": admin_id,
        "trader": trader_id,
        "client": client_id,
        "email": client_info.get('email', email),
        "sheet_url": push_sheet_url
    },
    # Store aggregated comment data if provided (from Push by Comment feature).
    # IMPORTANT: a fresh MT5 push (signaled by mt5_deals OR mt5_account being
    # present in the payload, OR an explicit aggregated_by_comment key in the
    # request body) ALWAYS overwrites the persisted aggregates – even if the
    # new value is an empty list. This prevents stale rows (e.g. old internal
    # transfers, deleted positions) from resurrecting from cache when the
    # latest live MT5 state no longer contains them.
    "aggregated_by_comment": (
        aggregated_by_comment
        if (aggregated_by_comment or mt5_deals or mt5_account or ("aggregated_by_comment" in data))
        else existing_data.get("aggregated_by_comment", [])
    ),
    "comment_summary": (
        comment_summary
        if (comment_summary or mt5_deals or mt5_account or ("comment_summary" in data))

```

```

        else existing_data.get("comment_summary", {})
    ),
}

# Merge firm_billing: new push data wins per-firm, but preserve firms not in this push
existing_firm_billing = existing_data.get("firm_billing") or {}
pushed_firm_billing = data.get("firm_billing")
if pushed_firm_billing:
    merged_billing = dict(existing_firm_billing)
    merged_billing.update(pushes_firm_billing)
    client_data["firm_billing"] = merged_billing
    app.logger.info(
        f"    - firm_billing: {list(merged_billing.keys())} (pushed: {list(pushes_firm_billing.keys())})"
    )
elif existing_firm_billing:
    client_data["firm_billing"] = existing_firm_billing

# Final verification before save
hr_final = statistics.get('hedging_review', {})
app.logger.info(f"■ SAVING DATA for {client_id}:")
app.logger.info(f"■ Statistics Section:")
app.logger.info(f"    - hedging_review.total_deposits: ${hr_final.get('total_deposits', 0):.2f}")
app.logger.info(f"    - hedging_review.total_withdrawals: ${hr_final.get('total_withdrawals', 0):.2f}")
app.logger.info(f"    - hedging_review.current_balance: ${hr_final.get('current_balance', 0):.2f}")
app.logger.info(
    f"    - hedging_review.actual_hedging_results: ${hr_final.get('actual_hedging_results', 0):.2f}")
app.logger.info(
    f"    - hedging_review.sheet_hedging_results: ${hr_final.get('sheet_hedging_results', 0):.2f}")
app.logger.info(f"    - hedging_review.discrepancy: ${hr_final.get('discrepancy', 0):.2f}")
app.logger.info(
    f"    - account.total_deposits: ${mt5_account.get('total_deposits', 0) if mt5_account else 0:.2f}")
app.logger.info(f"    - account.balance: ${mt5_account.get('balance', 0) if mt5_account else 0:.2f}")

# Log final evaluation state
app.logger.info(f"■ Final Evaluation Rows Being Saved:")
for i, ev in enumerate(evaluations[:5]): # Show first 5
    prop_firm = ev.get('Prop Firm', 'Unknown')
    hedges = sum(1 for j in range(1, 8) if ev.get(f'Hedge Result {j}', '') and str(ev.get(
        f'Hedge Result {j}', '')).strip() not in ('', '-'))
    payouts = sum(1 for j in range(1, 10) if ev.get(f'Payout {j}', '') and str(ev.get(f'Payout {j}',
        '')).strip() not in ('', '-', '$0'))
    app.logger.info(f"    [{i}] {prop_firm}: {hedges} hedge cells, {payouts} payout cells")

if len(evaluations) > 5:
    app.logger.info(f"    ... and {len(evaluations) - 5} more rows")

if aggregated_by_comment:
    app.logger.info(f"    - aggregated_by_comment: {len(aggregated_by_comment)} groups")

app.logger.info(f"■ Saving to database...")

# Determine change source for history tracking
change_source = 'trader_app'
if aggregated_by_comment:
    change_source = 'mt5_push_with_comments'
elif mt5_account or mt5_deals:
    change_source = 'mt5_push'

# Save to database WITH history tracking
success, version = save_client_data_with_history(
    client_id,
```

```

        client_data,
        action='DATA_PUSH',
        changed_by=email,
        changed_by_type='client',
        ip_address=get_remote_address(),
        change_source=change_source,
        change_description=f"Data push from trader app with {len(evaluations)} evaluations"
    )

    app.logger.info(f"■ SAVED to database successfully (v{version})")
    app.logger.info(f"■ Updated Statistics:")
    stats_complete = statistics.get('profitability_completed', {})
    stats_inprogress = statistics.get('cashflow_inprogress', {})
    app.logger.info(f"    - Profitability Completed: P/L=${stats_complete.get('net_profit', 0):.2f}")
    app.logger.info(f"    - Cashflow In Progress: P/L=${stats_inprogress.get('net_profit', 0):.2f}")
    app.logger.info(f"    - Evaluations saved: {len(evaluations)} rows")

    # Update Hierarchy (in case new)
    add_admin(admin_id)
    add_trader(admin_id, trader_id)
    add_client(admin_id, trader_id, client_id)

    log_action('CLIENT_DATA_PUSH', 'client', email, get_remote_address(),
              f"Data pushed for {client_id} (v{version})")

    response_data = {
        "status": "success",
        "message": f"Data updated for {client_id}",
        "version": version,
        "identity": {
            "admin": admin_id,
            "trader": trader_id,
            "client": client_id
        }
    }

    # Include hedge match log if we processed aggregated data
    if hedge_match_log:
        response_data["hedge_match_log"] = hedge_match_log
        response_data["hedge_updates"] = len([l for l in hedge_match_log if l.startswith("■")])

    return jsonify(response_data)

@app.route('/api/client/migrate_sheet', methods=['POST'])
@limiter.limit('10 per minute')
def api_migrate_sheet()

```

Public endpoint - migrate data from Google Sheets using client email. Fetches data from Google Sheet and pushes it to the dashboard.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@limiter.limit** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns data = request.json.
2. Assigns email = data.get('email', '').strip().lower().
3. Assigns sheet_url = data.get('sheet_url', '').strip().
4. **if** not email: branches conditionally.
5. **if** not sheet_url: branches conditionally.
6. Assigns client_info = get_client_by_email(email).
7. **if** not client_info: branches conditionally.
8. Assigns admin_id = client_info['admin'].
9. Assigns trader_id = client_info['trader'].
10. Assigns client_id = client_info['client'].
11. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def api_migrate_sheet():
    """
    Public endpoint - migrate data from Google Sheets using client email.
    Fetches data from Google Sheet and pushes it to the dashboard.
    """
    data = request.json
    email = data.get('email', '').strip().lower()
    sheet_url = data.get('sheet_url', '').strip()

    if not email:
        return jsonify({"status": "error", "message": "Email required"}), 400

    if not sheet_url:
        return jsonify({"status": "error", "message": "Google Sheet URL required"}), 400

    # Look up client by email
    client_info = get_client_by_email(email)
    if not client_info:
```

```

        return jsonify({"status": "error", "message": "Email not registered in the system"}), 404

admin_id = client_info['admin']
trader_id = client_info['trader']
client_id = client_info['client']

# Fetch data from Google Sheets
try:
    # Import the data processor
    from utils.data_processor import fetch_evaluations, calculate_statistics, fetch_waterlog_history
    from dashboard.watermark_service import bulk_save_history
    import concurrent.futures

    try:
        from utils.sheet_helper import fetch_waterlog_data, fetch_waterlog_periods_from_sheet
    except ImportError:
        try:
            from dashboard.utils.sheet_helper import fetch_waterlog_data, fetch_waterlog_periods_from_sheet
        except ImportError:
            fetch_waterlog_data = None
            fetch_waterlog_periods_from_sheet = None

    try:
        from dashboard.watermark_service import save_waterlog_periods
    except ImportError:
        from watermark_service import save_waterlog_periods

    # Fetch evaluations, waterlog history and waterlog data in parallel
    with concurrent.futures.ThreadPoolExecutor(max_workers=3) as executor:
        future_eval = executor.submit(fetch_evaluations, sheet_url)
        future_wl_hist = executor.submit(fetch_waterlog_history, sheet_url)
        future_wl_data = executor.submit(fetch_waterlog_data, sheet_url,
                                         client_id) if fetch_waterlog_data else None

        eval_result = future_eval.result()
        waterlog_history = future_wl_hist.result()
        wl_full = future_wl_data.result() if future_wl_data else None

    if isinstance(eval_result, tuple):
        evaluations, xlsx_notes = eval_result
    else:
        evaluations = eval_result
        xlsx_notes = {}
    if not evaluations:
        return jsonify({"status": "error",
                        "message": "Could not fetch data from sheet. Make sure it's shared as 'Anyone with the link'."})

    # Save daily waterlog history
    waterlog_count = 0
    if waterlog_history:
        bulk_save_history(client_id, waterlog_history)
        waterlog_count = len(waterlog_history)

    # Save the bi-weekly period schedule WITH Low/High values
    if wl_full:
        from datetime import datetime as _wldt
        import re as _re

        def _parse_currency_str(s):
            try:
                return float(_re.sub(r'^\d-\d-\d', '', str(s))) if s else None

```



```

        except Exception:
            return None

def _to_iso(date_str):
    """Convert M/D/YYYY to YYYY-MM-DD."""
    try:
        return _wldt.strptime(date_str.strip(), '%m/%d/%Y').strftime('%Y-%m-%d')
    except Exception:
        return None

wl_periods = []
wl_values = {}
for row in wl_full:
    fd = _to_iso(row.get('from_date', ''))
    td = _to_iso(row.get('to_date', ''))
    if fd and td:
        wl_periods.append((fd, td))
        low = _parse_currency_str(row.get('low'))
        high = _parse_currency_str(row.get('high'))
        if low is not None or high is not None:
            wl_values[fd] = {
                'low': low,
                'high': high,
                'split_pct': row.get('split_pct', 50),
            }

    if wl_periods:
        save_waterlog_periods(client_id, wl_periods, period_values=wl_values)
elif fetch_waterlog_periods_from_sheet:
    # Fallback: save dates only (no Low/High)
    wl_periods = fetch_waterlog_periods_from_sheet(sheet_url)
    if wl_periods:
        save_waterlog_periods(client_id, wl_periods)

# Get existing data to preserve MT5 account, historical accounts, and deletions
existing_import_data = get_client_data(client_id) or {}
existing_mt5 = existing_import_data.get('account') or None
existing_hist = existing_import_data.get('statistics', {}).get('hedging_review', {}).get(
    'historical_accounts')

# Preserve _deleted flags: build fingerprints of previously deleted rows
existing_evals = existing_import_data.get('evaluations', [])
deleted_fingerprints = set()
for ev in existing_evals:
    if isinstance(ev, dict) and ev.get('_deleted'):
        acct = str(ev.get('Account #') or ev.get('Account #.1') or '').strip()
        firm = str(ev.get('Prop Firm') or '').strip()
        size = str(ev.get('Account Size') or '').strip()
        if acct:
            deleted_fingerprints.add((acct, firm, size))

# Re-apply _deleted to matching incoming rows
if deleted_fingerprints:
    for ev in evaluations:
        if isinstance(ev, dict):
            acct = str(ev.get('Account #') or ev.get('Account #.1') or '').strip()
            firm = str(ev.get('Prop Firm') or '').strip()
            size = str(ev.get('Account Size') or '').strip()
            if acct and (acct, firm, size) in deleted_fingerprints:
                ev['_deleted'] = True

```

```

# Calculate statistics including existing MT5 data and historical accounts
statistics = calculate_statistics(evaluations, None, existing_mt5 if existing_mt5 else None,
                                xlsx_notes=xlsx_notes,
                                historical_accounts=existing_hist)

# Preserve historical accounts in the new statistics
if existing_hist:
    existing_hr = existing_import_data.get('statistics', {}).get('hedging_review', {})
    statistics.setdefault('hedging_review', {})[ 'historical_accounts' ] = existing_hist
    statistics['hedging_review'][ 'historical_deposits' ] = existing_hr.get('historical_deposits',
    statistics['hedging_review'][ 'historical_withdrawals' ] = existing_hr.get('historical_withdraw
    0)
    statistics['hedging_review'][ 'historical_balance' ] = existing_hr.get('historical_balance', 0)

# Prepare client data
client_data = {
    "deals": existing_import_data.get('deals', []),
    "positions": existing_import_data.get('positions', []),
    "account": existing_mt5 or {},
    "evaluations": evaluations,
    "statistics": statistics,
    "dropdown_options": {},
    "identity": {
        "admin": admin_id,
        "trader": trader_id,
        "client": client_id,
        "email": client_info.get('email', email),
        "sheet_url": sheet_url
    },
    "sheet_url": sheet_url,
    "migrated_at": datetime.now().isoformat(),
    # Preserve manually-entered fields that are not sourced from the sheet
    "prop_accounts": existing_import_data.get('prop_accounts', []),
    "vps_accounts": existing_import_data.get('vps_accounts', []),
    "hedge_accounts": existing_import_data.get('hedge_accounts', []),
    "payment_info": existing_import_data.get('payment_info', []),
    "payment_address": existing_import_data.get('payment_address', {}),
    "mt5_credentials": existing_import_data.get('mt5_credentials', {}),
}

# Save to database WITH history tracking - full overwrite (import replaces all existing data)
success, version = save_client_data_with_history(
    client_id,
    client_data,
    action='SHEET_IMPORT',
    changed_by=email,
    changed_by_type='client',
    ip_address=get_remote_address(),
    change_source='sheet_migration',
    change_description=f"Imported {len(evaluations)} records from Google Sheets",
    overwrite=True
)

# Save cell comments as notes (for Prop Progress display)
notes_saved = 0
if xlsx_notes:
    for row_idx, col_notes in xlsx_notes.items():
        if isinstance(row_idx, int) and isinstance(col_notes, dict):
            for col_key, content in col_notes.items():
                if content and str(content).strip():

```

```

        save_client_note(client_id, row_idx, col_key, str(content).strip(),
                        'sheet_import')
        notes_saved += 1

    # Update Hierarchy
    add_admin(admin_id)
    add_trader(admin_id, trader_id)
    add_client(admin_id, trader_id, client_id)

    log_action('SHEET_MIGRATION', 'client', email, get_remote_address(),
              f"Migrated {len(evaluations)} records + {waterlog_count} waterlog entries + {notes_saved} notes")

    # Return statistics for verification
    return jsonify({
        "status": "success",
        "message": f"Successfully migrated {len(evaluations)} records and {waterlog_count} waterlog entries",
        "records_imported": len(evaluations),
        "version": version,
        "statistics": statistics, # Include stats for client-side verification
        "identity": {
            "admin": admin_id,
            "trader": trader_id,
            "client": client_id
        }
    })

except Exception as e:
    log_action('SHEET_MIGRATION_FAILED', 'client', email, get_remote_address(), str(e), False)
    return jsonify({"status": "error", "message": f"Migration failed: {str(e)}"}), 500

@app.route('/api/client/watermark_history/<client_id>')
@require_session
def api_get_watermark_history(client_id)

```

Get daily watermark history for a client. Restricted: Clients can only see their own waterlog history.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_session** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.
- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns `session_user = request.session_user`.
2. Assigns `user_type = session_user.get('user_type')`.
3. Assigns `user_id = session_user.get('user_identifier')`.
4. Assigns `is_authorized = False`.
5. **if** `user_type` in `['super_admin', 'admin', 'trader', 'kwok_admin']`: branches conditionally (with an **else/elif** arm).
6. **if** `not is_authorized`: branches conditionally.
7. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def api_get_watermark_history(client_id):
    """
    Get daily watermark history for a client.
    Restricted: Clients can only see their own waterlog history.
    """
    session_user = request.session_user
    user_type = session_user.get('user_type')
    user_id = session_user.get('user_identifier')

    # Handle URL encoding spaces if necessary (Flask usually decodes)
    # Check authorization
    is_authorized = False
    if user_type in ['super_admin', 'admin', 'trader', 'kwok_admin']:
        is_authorized = True
    elif user_type == 'bef_admin':
        is_authorized = can_access_client('bef_admin', None, client_id)
    elif user_type == 'client':
        # Check specific client ownership
        if user_id == client_id:
            is_authorized = True
        else:
            # Check if email matches (some systems use email as identifier)
            # Or if user_id is the email and client_id is the name?
            # 'client_id' in URL is the NAME (e.g. Jiang Quang Huang)
            # 'user_identifier' in session for client is usually the EMAIL.
            # We need to resolve email -> client_id
            client_by_email = get_client_by_email(user_id)
            if client_by_email and client_by_email['client'] == client_id:
                is_authorized = True

    if not is_authorized:
        return jsonify({"status": "error", "message": "Unauthorized access to client waterlog"}), 403

    try:
```

```

from dashboard.watermark_service import get_watermark_history, get_lower_watermark, save_daily_profit
from dashboard.database import get_client_data

# --- Always snapshot today's live net profit so the Low Watermark is current ---
today_str = __import__('datetime').datetime.now().strftime('%Y-%m-%d')
try:
    client_data = get_client_data(client_id)
    if client_data:
        # Pull net profit directly from stored statistics
        # (already includes discrepancy from the last data push)
        stored_stats = client_data.get('statistics', {})
        live_net = None
        if isinstance(stored_stats, dict):
            live_net = stored_stats.get('cashflow_inprogress', {}).get('net_profit')
            if live_net is None:
                live_net = stored_stats.get('profitability_completed', {}).get('net_profit')
        if live_net is not None:
            save_daily_profit(client_id, live_net, today_str, source='live')
except Exception as snap_err:
    logging.warning(f"Live watermark snapshot failed for {client_id}: {snap_err}")

# Get history for the requested period (14 days)
history = get_watermark_history(client_id, days=14)

# Low watermark among previous days only (exclude today so live value is shown separately)
prev_history = [h for h in history if h['date'] < today_str]
low_watermark = min(prev_history, key=lambda x: x['profit']) if prev_history else None

# Today's live entry
today_entry = next((h for h in history if h['date'] == today_str), None)

return jsonify({
    "status": "success",
    "history": history,
    "low_watermark": low_watermark,
    "today": today_entry
})
except Exception as e:
    return jsonify({"status": "error", "message": str(e)}), 500

@app.route('/api/login', methods=['POST'])
@limiter.limit('10 per minute')
def api_login()

```

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@limiter.limit** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns email = request.json.get('email').

2. Assigns `client_ip = get_remote_address()`.
3. **if** not email: branches conditionally.
4. Assigns `client = get_client_by_email(email)`.
5. **if** client: branches conditionally.
6. Calls `log_action(...)` for its side effect.
7. **return** (`jsonify({'status': 'error', 'message': 'Email not found'})`, 404).

Code (copy-paste safe)

```
def api_login():
    email = request.json.get('email')
    client_ip = get_remote_address()

    if not email:
        return jsonify({"status": "error", "message": "Email required"}), 400

    client = get_client_by_email(email)
    if client:
        log_action('CLIENT_LOGIN', 'client', email, client_ip, 'Successful login')
        return jsonify({"status": "success", "redirect": f"/dashboard/{client['client']}"})

    log_action('CLIENT_LOGIN_FAILED', 'client', email, client_ip, 'Email not found', False)
    return jsonify({"status": "error", "message": "Email not found"}), 404

@app.route('/api/admin_login', methods=['POST'])
@limiter.limit('5 per minute')
def api_admin_login():
```

Secure admin login with session creation.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@limiter.limit** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.

Step by step (top-level body)

1. Assigns `password = request.json.get('password')`.
2. Assigns `client_ip = get_remote_address()`.
3. **if** not password: branches conditionally.
4. **if** `verify_admin_password('super_admin', password)`: branches conditionally.

Code (copy-paste safe)

```
def api_admin_login():
    """Secure admin login with session creation."""
    password = request.json.get('password')
    client_ip = get_remote_address()

    if not password:
        return jsonify({"status": "error", "message": "Password required"}), 400
```

```

    if verify_admin_password('super_admin', password):
        session_token = create_session('admin', 'super_admin', client_ip)
        log_action('ADMIN_LOGIN', 'admin', 'super_admin', client_ip, 'Successful login')

        response = jsonify({"status": "success", "redirect": "/super_admin"})
        response.set_cookie('session_token', session_token, httponly=True, secure=not app.debug,
                           samesite='Strict')
        return response

@app.route('/logout')
def logout()

```

Logout via GET request - clears session and redirects to login.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.

Step by step (top-level body)

1. Assigns `session_token = request.cookies.get('session_token')`.
2. **if** `session_token`: branches conditionally.
3. Assigns `response = redirect('/')`.
4. Calls `response.delete_cookie(...)` for its side effect.
5. **return** `response`.

Code (copy-paste safe)

```

def logout():
    """Logout via GET request - clears session and redirects to login."""
    session_token = request.cookies.get('session_token')
    if session_token:
        delete_session(session_token)
        log_action('LOGOUT', 'user', 'session', get_remote_address())

    response = redirect('/')
    response.delete_cookie('session_token')
    return response

@app.route('/api/logout', methods=['POST'])
def api_logout()

```

Logout and invalidate session (API endpoint).

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.

Step by step (top-level body)

1. Assigns `session_token = request.cookies.get('session_token')`.
2. **if** `session_token`: branches conditionally.
3. Assigns `response = jsonify({'status': 'success'})`.

4. Calls `response.delete_cookie(...)` for its side effect.

5. **return** response.

Code (copy-paste safe)

```
def api_logout():
    """Logout and invalidate session (API endpoint)."""
    session_token = request.cookies.get('session_token')
    if session_token:
        delete_session(session_token)

    response = jsonify({"status": "success"})
    response.delete_cookie('session_token')
    return response
```

```
@app.route('/api/auth/check-admin', methods=['POST'])
@limiter.limit('20 per minute')
def check_admin_identifier():
```

Returns whether the given identifier requires a password (all users do).

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@limiter.limit** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.

Step by step (top-level body)

1. Assigns `data = request.json or {}`.
2. Assigns `identifier = data.get('identifier', '').strip()`.
3. **if not identifier**: branches conditionally.
4. Assigns `user = find_user_by_identifier(identifier)`.
5. **if not user and '@' in identifier**: branches conditionally.
6. Assigns `requires_password = bool(user)`.
7. **return** `jsonify({'requires_password': requires_password})`.

Code (copy-paste safe)

```
def check_admin_identifier():
    """Returns whether the given identifier requires a password (all users do)."""
    data = request.json or {}
    identifier = data.get('identifier', '').strip()
    if not identifier:
        return jsonify({"requires_password": False})
    user = find_user_by_identifier(identifier)
    if not user and '@' in identifier:
        user = get_user_by_email(identifier)
    # All recognised users require a password
    requires_password = bool(user)
    return jsonify({"requires_password": requires_password})
```



```
@app.route('/api/auth/login', methods=['POST'])
@limiter.limit('10 per minute')
def unified_login():
```

Unified login endpoint - auto-detects user type from email/username. All user types require email + password. Default password: Test@123

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@limiter.limit** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns data = request.json.
2. Assigns identifier = data.get('identifier', '').strip().
3. Assigns password = data.get('password', '').
4. Assigns remember = data.get('remember', False).
5. Assigns client_ip = get_remote_address().
6. Assigns DEFAULT_SESSION_HOURS = 24.
7. Assigns REMEMBER_SESSION_DAYS = 7.
8. Assigns session_hours_valid = REMEMBER_SESSION_DAYS * 24 if remember else DEFAULT_SESSI....
9. Assigns cookie_max_age = REMEMBER_SESSION_DAYS * 24 * 60 * 60 if remember else DEF....
10. **if** not identifier: branches conditionally.
11. Assigns user = find_user_by_identifier(identifier).
12. **if** not user and '@' in identifier: branches conditionally.
13. **if** not user: branches conditionally.
14. Assigns user_type = user.get('user_type').
15. ... and 18 more statement(s) in the body.

Code (copy-paste safe)

```
def unified_login():
    """
    Unified login endpoint - auto-detects user type from email/username.
    All user types require email + password. Default password: Test@123
    """
    data = request.json
    identifier = data.get('identifier', '').strip()
    password = data.get('password', '')
```

```

remember = data.get('remember', False)
client_ip = get_remote_address()

# Session durations:
# - Default: 24h
# - Remember: 7 days (requested)
DEFAULT_SESSION_HOURS = 24
REMEMBER_SESSION_DAYS = 7
session_hours_valid = (REMEMBER_SESSION_DAYS * 24) if remember else DEFAULT_SESSION_HOURS
cookie_max_age = (REMEMBER_SESSION_DAYS * 24 * 60 * 60) if remember else (DEFAULT_SESSION_HOURS * 60

if not identifier:
    return jsonify({"status": "error", "message": "Email is required"}), 400

# Find user by identifier (email or username)
user = find_user_by_identifier(identifier)

# If not found in DB, check hierarchy (for email-only logins)
# This allows users defined in hierarchy.json but not yet in user_credentials to login
if not user and '@' in identifier:
    hierarchy_user = get_user_by_email(identifier)
    if hierarchy_user:
        user = hierarchy_user

if not user:
    log_action('LOGIN_FAILED', 'unknown', identifier, client_ip, 'User not found', False)
    return jsonify({"status": "error", "message": "Email not found in system"}), 403

user_type = user.get('user_type')
username = user.get('username', identifier).strip()

# Check account lockout
if is_account_locked(username, user_type):
    log_action('LOGIN_LOCKED', user_type, username, client_ip, 'Account locked', False)
    return jsonify({"status": "error",
        "message": "Account locked. Too many failed attempts. Try again in 15 minutes."}), 429

# Handle Super Admin login - REQUIRES PASSWORD
if user_type == 'super_admin':
    if not password:
        return jsonify({"status": "error", "message": "Password is required for Super Admin"}), 400

    if verify_admin_password('super_admin', password):
        session_token = create_session('super_admin', 'super_admin', client_ip,
            hours_valid=session_hours_valid)
        record_login_attempt('super_admin', 'super_admin', client_ip, True)
        log_action('LOGIN_SUCCESS', 'super_admin', 'super_admin', client_ip)

        response = jsonify({
            "status": "success",
            "user_type": "super_admin",
            "redirect": "/super_admin",
            "must_change_password": False
        })
        response.set_cookie('session_token', session_token, httponly=True, secure=not app.debug,
            samesite='Lax', max_age=cookie_max_age)
        return response

    record_login_attempt('super_admin', 'super_admin', client_ip, False)
    log_action('LOGIN_FAILED', 'super_admin', 'super_admin', client_ip, 'Invalid password', False)

```

```

        return jsonify({"status": "error", "message": "Invalid password"}), 403

# Handle BEF Admin login - REQUIRES PASSWORD (same flow as super_admin)
if user_type == 'bef_admin':
    if not password:
        return jsonify({"status": "error", "message": "Password is required for BEF Admin"}), 400

    if verify_admin_password('bef_admin', password):
        session_token = create_session('bef_admin', 'bef_admin', client_ip,
                                         hours_valid=session_hours_valid)
        record_login_attempt('bef_admin', 'bef_admin', client_ip, True)
        log_action('LOGIN_SUCCESS', 'bef_admin', 'bef_admin', client_ip)

        response = jsonify({
            "status": "success",
            "user_type": "bef_admin",
            "redirect": "/bef_admin",
            "must_change_password": False
        })
        response.set_cookie('session_token', session_token, httponly=True, secure=not app.debug,
                           samesite='Lax', max_age=cookie_max_age)
        return response

    record_login_attempt('bef_admin', 'bef_admin', client_ip, False)
    log_action('LOGIN_FAILED', 'bef_admin', 'bef_admin', client_ip, 'Invalid password', False)
    return jsonify({"status": "error", "message": "Invalid password"}), 403

# Kwok admin – separate password, full read-only client access
if user_type == 'kwok_admin':
    if not password:
        return jsonify({"status": "error", "message": "Password is required for Kwok Admin"}), 400

    if verify_admin_password('kwok_admin', password):
        session_token = create_session('kwok_admin', 'kwok_admin', client_ip,
                                         hours_valid=session_hours_valid)
        record_login_attempt('kwok_admin', 'kwok_admin', client_ip, True)
        log_action('LOGIN_SUCCESS', 'kwok_admin', 'kwok_admin', client_ip)
        response = jsonify({
            "status": "success",
            "user_type": "kwok_admin",
            "redirect": "/kwok_admin",
            "must_change_password": False
        })
        response.set_cookie('session_token', session_token, httponly=True, secure=not app.debug,
                           samesite='Lax', max_age=cookie_max_age)
        return response

    record_login_attempt('kwok_admin', 'kwok_admin', client_ip, False)
    log_action('LOGIN_FAILED', 'kwok_admin', 'kwok_admin', client_ip, 'Invalid password', False)
    return jsonify({"status": "error", "message": "Invalid password"}), 403

# Handle Admin/Trader/Client login - PASSWORD REQUIRED
if not password:
    return jsonify({"status": "error", "message": "Password is required"}), 400

# Auto-provision credentials if user exists in hierarchy but not in user_credentials DB
if not find_user_by_identifier(username) and not find_user_by_identifier(user.get('email', '')):
    default_pw = 'Test@123'
    create_user(username, default_pw, user_type,
                user.get('email'), user.get('parent_admin'), user.get('parent_trader'))

```

```

# Verify password
verified = verify_user_password(username, user_type, password)
if not verified:
    record_login_attempt(username, user_type, client_ip, False)
    log_action('LOGIN_FAILED', user_type, username, client_ip, 'Invalid password', False)
    return jsonify({"status": "error", "message": "Invalid password"}), 403

record_login_attempt(username, user_type, client_ip, True)
log_action('LOGIN_SUCCESS', user_type, username, client_ip)

# Determine redirect URL based on user type
redirect_map = {
    'admin': f'/admin/{username}',
    'trader': f'/trader/{username}',
    'client': '/maintenance' if MAINTENANCE_MODE else f'/dashboard/{username}'
}
redirect_url = redirect_map.get(user_type, '/')

must_change = verified.get('must_change_password', False)

session_token = create_session(user_type, username, client_ip, hours_valid=session_hours_valid)
response = jsonify({
    "status": "success",
    "user_type": user_type,
    "redirect": redirect_url,
    "must_change_password": must_change
})
response.set_cookie('session_token', session_token, httponly=True, secure=not app.debug, samesite='L
    max_age=cookie_max_age)
return response

@app.route('/api/admin/create_user', methods=['POST'])
@require_role('super_admin')
@limiter.limit('20 per hour')
def api_create_user()

```

Create a new user account.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_role** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **@limiter.limit** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.

• **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns data = request.json.
2. Assigns username = data.get('username').
3. Assigns user_type = data.get('user_type').
4. Assigns email = data.get('email').
5. Assigns password = data.get('password').
6. if not password: branches conditionally.
7. Assigns parent_admin = data.get('parent_admin').
8. Assigns parent_trader = data.get('parent_trader').
9. if not username or not user_type: branches conditionally.
10. if user_type not in ['admin', 'trader', 'client']: branches conditionally.
11. Assigns user_db_exists = user_exists(username, user_type).
12. Assigns hierarchy_exists = False.
13. if user_type == 'admin': branches conditionally (with an **else/elif** arm).
14. if user_db_exists and hierarchy_exists: branches conditionally.
15. ... and 4 more statement(s) in the body.

Code (copy-paste safe)

```
def api_create_user():
    """Create a new user account."""
    data = request.json
    username = data.get('username')
    user_type = data.get('user_type') # admin, trader, or client
    email = data.get('email')

    # Auto-generate password if not provided
    password = data.get('password')
    if not password:
        password = 'Test@123'

    parent_admin = data.get('parent_admin')
    parent_trader = data.get('parent_trader')

    if not username or not user_type:
        return jsonify({"status": "error", "message": "Username and user_type required"}), 400

    if user_type not in ['admin', 'trader', 'client']:
        return jsonify({"status": "error", "message": "Invalid user type"}), 400

    # Check if user already exists in DB
    user_db_exists = user_exists(username, user_type)

    # Check if user exists in Hierarchy
    hierarchy_exists = False
    if user_type == 'admin':
```

```

        if username in hierarchy.get('admins', {}):
            hierarchy_exists = True
    elif user_type == 'trader':
        for adm in hierarchy.get('admins', {}).values():
            if username in adm.get('traders', {}):
                hierarchy_exists = True
                break
    elif user_type == 'client':
        for adm in hierarchy.get('admins', {}).values():
            for tr in adm.get('traders', {}).values():
                clients = tr.get('clients', [])
                for cl in clients:
                    c_name = cl.get('name') if isinstance(cl, dict) else cl
                    if c_name == username:
                        hierarchy_exists = True
                        break
                if hierarchy_exists: break
        if hierarchy_exists: break

    if user_db_exists and hierarchy_exists:
        return jsonify({"status": "error",
            "message": f"{user_type.title()} '{username}' already exists"}), 400

    # Create user in database (auth) if not exists
    if not user_db_exists:
        if not create_user(username, password, user_type, email, parent_admin, parent_trader):
            return jsonify({"status": "error", "message": "Failed to create user"}), 500

    # Update Hierarchy (for display) - even if they existed in DB but not hierarchy
    try:
        if user_type == 'admin':
            if not hierarchy_exists:
                add_admin(username, email)
        elif user_type == 'trader':
            if not hierarchy_exists:
                # Map parent_user to parent_admin for simple logic if needed,
                # but request.json usually sends 'parent_user' which we need to grab
                p_admin = data.get('parent_user') or parent_admin
                if p_admin:
                    add_trader(p_admin, username, email)
        elif user_type == 'client':
            if not hierarchy_exists:
                p_trader = data.get('parent_user') or parent_trader
                # We need to find the admin for this trader to call add_client(admin, trader, client)
                # Search hierarchy for the trader
                found_admin = None
                if hierarchy.get('admins'):
                    for adm, a_data in hierarchy['admins'].items():
                        if p_trader in a_data.get('traders', {}):
                            found_admin = adm
                            break
                if found_admin and p_trader:
                    add_client(found_admin, p_trader, username, email)
    except Exception as e:
        print(f"Hierarchy update failed: {e}")
        # Continue, as user was created in DB

    log_action('CREATE_USER', 'admin', username, get_remote_address(), f"Type: {user_type}")
    return jsonify({"status": "success", "message": f"{user_type.title()} '{username}' created successfully"})

```

```
def can_manage_user(manager_type, manager_identifier, target_username, target_user_type)

    Check if a user can manage (reset password, deactivate) another user.
```

Step by step (top-level body)

1. **if** `manager_type == 'super_admin'`: branches conditionally.
2. **if** `manager_type == 'admin'`: branches conditionally.
3. **if** `manager_type == 'trader'`: branches conditionally.
4. **return** `False`.

Code (copy-paste safe)

```
def can_manage_user(manager_type, manager_identifier, target_username, target_user_type):
    """Check if a user can manage (reset password, deactivate) another user."""
    if manager_type == 'super_admin':
        return True # Super admin can manage everyone

    if manager_type == 'admin':
        # Admin can manage traders and clients under them
        admin_data = hierarchy.get('admins', {}).get(manager_identifier, {})

        if target_user_type == 'trader':
            # Check if trader is under this admin
            return target_username in admin_data.get('traders', {})

        if target_user_type == 'client':
            # Check if client is under any of this admin's traders
            for trader_data in admin_data.get('traders', {}).values():
                for client in trader_data.get('clients', []):
                    if client.get('name') == target_username or client.get('email') == target_username:
                        return True
            return False

        return False # Admin cannot manage other admins

    if manager_type == 'trader':
        # Trader can only manage their clients
        if target_user_type != 'client':
            return False

        for admin_data in hierarchy.get('admins', {}).values():
            trader_data = admin_data.get('traders', {}).get(manager_identifier, {})
            for client in trader_data.get('clients', []):
                if client.get('name') == target_username or client.get('email') == target_username:
                    return True
        return False

    return False

@app.route('/api/admin/list_users', methods=['GET'])
@require_admin_password
def api_list_users()

    List all users.
```

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_admin_password** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.

Step by step (top-level body)

1. Assigns `user_type = request.args.get('type')`.
2. Assigns `users = list_users(user_type)`.
3. **return** `jsonify({'status': 'success', 'users': users})`.

Code (copy-paste safe)

```
def api_list_users():
    """List all users."""
    user_type = request.args.get('type')
    users = list_users(user_type)
    return jsonify({"status": "success", "users": users})

@app.route('/api/user/reset_password', methods=['POST'])
@require_role('super_admin', 'admin', 'trader')
def api_reset_password_rbac():
```

Reset a user's password with role-based access control.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_role** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `session_user = request.session_user`.
2. Assigns `manager_type = session_user.get('user_type')`.
3. Assigns `manager_id = session_user.get('user_identifier')`.
4. Assigns `data = request.json`.
5. Assigns `username = data.get('username')`.
6. Assigns `user_type = data.get('user_type')`.
7. Assigns `email = data.get('email')`.
8. **if** not `username` or not `user_type`: branches conditionally.
9. **if** not `can_manage_user(manager_type, manager_id, username, user_type)`: branches conditionally.

10. Assigns `temp_password = reset_user_password(username, user_type)`.

11. `if temp_password`: branches conditionally.

12. `return` (`jsonify({'status': 'error', 'message': 'User not found'})`, 404).

Code (copy-paste safe)

```
def api_reset_password_rbac():
    """Reset a user's password with role-based access control."""
    session_user = request.session_user
    manager_type = session_user.get('user_type')
    manager_id = session_user.get('user_identifier')

    data = request.json
    username = data.get('username')
    user_type = data.get('user_type')
    email = data.get('email')

    if not username or not user_type:
        return jsonify({"status": "error", "message": "Username and user_type required"}), 400

    # Check if user has permission to reset this user's password
    if not can_manage_user(manager_type, manager_id, username, user_type):
        log_action('RESET_PASSWORD_DENIED', manager_type, manager_id, get_remote_address(),
                  f"Attempted to reset: {username} ({user_type})", False)
        return jsonify({"status": "error",
                        "message": "Access denied. You can only manage users under your hierarchy."}), 403

    temp_password = reset_user_password(username, user_type)
    if temp_password:
        log_action('RESET_PASSWORD', manager_type, username, get_remote_address(), f"By: {manager_id}")

        email_sent = False
        if email:
            # Import email sender lazily so we don't touch stdlib email modules
            # unless we actually need to send an email (avoids Windows watchdog reload loops).
            try:
                from dashboard.email_service import send_password_reset_with_temp
                email_sent = send_password_reset_with_temp(email, username, temp_password)
            except Exception as e:
                app.logger.warning(f"[RESET_PASSWORD] Email send failed for {username}: {e}")

        return jsonify({
            "status": "success",
            "message": f"Password reset for {username}",
            "temporary_password": temp_password,
            "email_sent": email_sent
        })

    return jsonify({"status": "error", "message": "User not found"}), 404

@app.route('/api/admin/reset_password', methods=['POST'])
@require_admin_password
def api_reset_password():
```

Reset a user's password (legacy - uses password header).

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_admin_password** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns data = request.json.
2. Assigns username = data.get('username').
3. Assigns user_type = data.get('user_type').
4. Assigns email = data.get('email').
5. **if not username or not user_type**: branches conditionally.
6. Assigns temp_password = reset_user_password(username, user_type).
7. **if temp_password**: branches conditionally.
8. **return** (jsonify({'status': 'error', 'message': 'User not found'}), 404).

Code (copy-paste safe)

```
def api_reset_password():
    """Reset a user's password (legacy - uses password header)."""
    data = request.json
    username = data.get('username')
    user_type = data.get('user_type')
    email = data.get('email') # Optional - for email notification

    if not username or not user_type:
        return jsonify({'status': "error", "message": "Username and user_type required"}), 400

    temp_password = reset_user_password(username, user_type)
    if temp_password:
        log_action('RESET_PASSWORD', 'admin', username, get_remote_address(), f"Type: {user_type}")

        # Send email notification if email provided
        email_sent = False
        if email:
            try:
                from dashboard.email_service import send_password_reset_with_temp
                email_sent = send_password_reset_with_temp(email, username, temp_password)
            except Exception as e:
                app.logger.warning(f"[RESET_PASSWORD] Email send failed for {username}: {e}")

    return jsonify({
        "status": "success",
        "message": f"Password reset for {username}",
        "temporary_password": temp_password,
        "email_sent": email_sent
    })
```

```
return jsonify({"status": "error", "message": "User not found"}), 404
```

```
@app.route('/api/admin/set_password', methods=['POST'])
@require_role('super_admin')
def api_set_password():
```

Super admin sets a specific password for any user.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_role** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `data = request.json`.
2. Assigns `username = data.get('username')`.
3. Assigns `user_type = data.get('user_type')`.
4. Assigns `new_password = data.get('new_password')`.
5. **if** not `username` or not `user_type` or (not `new_password`): branches conditionally.
6. **if** `len(new_password) < 6`: branches conditionally.
7. **if** not `user_exists(username, user_type)`: branches conditionally.
8. **if** `update_user_password(username, user_type, new_password)`: branches conditionally.
9. **return** (`jsonify({'status': 'error', 'message': 'Failed to set password'})`),....

Code (copy-paste safe)

```
def api_set_password():
    """Super admin sets a specific password for any user."""
    data = request.json
    username = data.get('username')
    user_type = data.get('user_type')
    new_password = data.get('new_password')

    if not username or not user_type or not new_password:
        return jsonify({"status": "error", "message": "username, user_type and new_password required"}),
    if len(new_password) < 6:
        return jsonify({"status": "error", "message": "Password must be at least 6 characters"}), 400

    # Auto-provision if the user has no credentials row yet
    if not user_exists(username, user_type):
        create_user(username, new_password, user_type)
        log_action('SET_PASSWORD', 'super_admin', username, get_remote_address(),
                  f"Created + set ({user_type})")
        return jsonify({"status": "success",
                        "message": f"Credentials created for {username} with provided password"})

    if update_user_password(username, user_type, new_password):
```

```

        log_action('SET_PASSWORD', 'super_admin', username, get_remote_address(), f"Type: {user_type}")
        return jsonify({"status": "success", "message": f"Password set for {username}"})

    return jsonify({"status": "error", "message": "Failed to set password"}), 500

@app.route('/api/admin/deactivate_user', methods=['POST'])
@require_admin_password
def api_deactivate_user():

    Deactivate a user account.

```

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_admin_password** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `data = request.json`.
2. Assigns `username = data.get('username')`.
3. Assigns `user_type = data.get('user_type')`.
4. **if** `not username or not user_type`: branches conditionally.
5. **if** `deactivate_user(username, user_type)`: branches conditionally.
6. **return** (`jsonify({'status': 'error', 'message': 'User not found'})`, 404).

Code (copy-paste safe)

```

def api_deactivate_user():
    """Deactivate a user account."""
    data = request.json
    username = data.get('username')
    user_type = data.get('user_type')

    if not username or not user_type:
        return jsonify({"status": "error", "message": "Username and user_type required"}), 400

    if deactivate_user(username, user_type):
        log_action('DEACTIVATE_USER', 'admin', username, get_remote_address(), f"Type: {user_type}")
        return jsonify({"status": "success", "message": f"User '{username}' deactivated"})

    return jsonify({"status": "error", "message": "User not found"}), 404

@app.route('/change-password')
@require_session
def change_password_page():

    Page to change password.

```

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_session** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.

Step by step (top-level body)

1. Assigns `session_user = request.session_user`.
2. Assigns `user_type = session_user.get('user_type')`.
3. Assigns `username = session_user.get('user_identifier')`.
4. Assigns `must_change = False`.
5. **if** `user_type` not in ('super_admin', 'bef_admin', 'kwok_admin'): branches conditionally.
6. **return** `render_template('change_password.html', must_change_password=must_c....`

Code (copy-paste safe)

```
def change_password_page():
    """Page to change password."""
    session_user = request.session_user
    user_type = session_user.get('user_type')
    username = session_user.get('user_identifier')
    must_change = False
    if user_type not in ('super_admin', 'bef_admin', 'kwok_admin'):
        user_record = find_user_by_identifier(username)
        must_change = bool(user_record and user_record.get('must_change_password'))
    return render_template('change_password.html', must_change_password=must_change)
```

```
@app.route('/api/auth/change_password', methods=['POST'])
@require_session
@limiter.limit('5 per hour')
def api_change_password()
```

Change user's own password.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_session** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **@limiter.limit** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.

Step by step (top-level body)

1. Assigns `data = request.json`.
2. Assigns `current_password = data.get('current_password')`.
3. Assigns `new_password = data.get('new_password')`.
4. Assigns `skip_current = data.get('skip_current', False)`.
5. **if** not `new_password`: branches conditionally.

6. if not skip_current and (not current_password): branches conditionally.
7. if len(new_password) < 8: branches conditionally.
8. Assigns session_user = request.session_user.
9. Assigns user_type = session_user.get('user_type').
10. Assigns username = session_user.get('user_identifier').
11. if skip_current and user_type not in ('super_admin', 'bef_admin', 'kwo...: branches conditionally.
12. if user_type == 'super_admin': branches conditionally (with an **else/elif** arm).
13. return jsonify({'status': 'error', 'message': 'Failed to change password'....

Code (copy-paste safe)

```
def api_change_password():
    """Change user's own password."""
    data = request.json
    current_password = data.get('current_password')
    new_password = data.get('new_password')
    skip_current = data.get('skip_current', False)

    if not new_password:
        return jsonify({"status": "error", "message": "New password required"}), 400

    if not skip_current and not current_password:
        return jsonify({"status": "error", "message": "Current and new password required"}), 400

    if len(new_password) < 8:
        return jsonify({"status": "error", "message": "Password must be at least 8 characters"}), 400

    session_user = request.session_user
    user_type = session_user.get('user_type')
    username = session_user.get('user_identifier')

    # If skip_current requested, verify user actually has must_change_password set
    if skip_current and user_type not in ('super_admin', 'bef_admin', 'kwok_admin'):
        user_record = find_user_by_identifier(username)
        if not user_record or not user_record.get('must_change_password'):
            return jsonify({"status": "error", "message": "Current password is required"}), 400

    # Verify current password (unless must_change_password skip)
    if user_type == 'super_admin':
        if not skip_current and not verify_admin_password('super_admin', current_password):
            return jsonify({"status": "error", "message": "Current password is incorrect"}), 403
        if set_admin_password('super_admin', new_password):
            log_action('CHANGE_PASSWORD', 'super_admin', 'super_admin', get_remote_address())
            return jsonify({"status": "success", "message": "Password changed successfully"})
    elif user_type == 'bef_admin':
        if not skip_current and not verify_admin_password('bef_admin', current_password):
            return jsonify({"status": "error", "message": "Current password is incorrect"}), 403
        if set_admin_password('bef_admin', new_password):
            log_action('CHANGE_PASSWORD', 'bef_admin', 'bef_admin', get_remote_address())
            return jsonify({"status": "success", "message": "Password changed successfully"})
    elif user_type == 'kwok_admin':
        if not skip_current and not verify_admin_password('kwok_admin', current_password):
            return jsonify({"status": "error", "message": "Current password is incorrect"}), 403
        if set_admin_password('kwok_admin', new_password):
            log_action('CHANGE_PASSWORD', 'kwok_admin', 'kwok_admin', get_remote_address())
```

```

        return jsonify({"status": "success", "message": "Password changed successfully"})
    else:
        if not skip_current:
            user_info = verify_user_password(username, user_type, current_password)
            if not user_info:
                return jsonify({"status": "error", "message": "Current password is incorrect"}), 403
        if update_user_password(username, user_type, new_password):
            log_action('CHANGE_PASSWORD', user_type, username, get_remote_address())
            return jsonify({"status": "success", "message": "Password changed successfully"})

    return jsonify({"status": "error", "message": "Failed to change password"}), 500

@app.route('/api/kyc/link', methods=['POST'])
@require_role('super_admin')
def api_kyc_link()

```

Link a secondary client to a primary client as a KYC account.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_role** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns data = request.json.
2. Assigns primary = data.get('primary_client', '').strip().
3. Assigns linked = data.get('linked_client', '').strip().
4. **if** not primary or not linked: branches conditionally.
5. **if** primary == linked: branches conditionally.
6. Assigns existing_links = get_kyc_linked_clients(linked).
7. **if** existing_links: branches conditionally.
8. **if** add_kyc_link(primary, linked, request.session_user.get('user_identi...: branches conditionally.
9. **return** (jsonify({'status': 'error', 'message': 'Link already exists or fai...

Code (copy-paste safe)

```

def api_kyc_link():
    """Link a secondary client to a primary client as a KYC account."""
    data = request.json
    primary = data.get('primary_client', '').strip()
    linked = data.get('linked_client', '').strip()
    if not primary or not linked:
        return jsonify({"status": "error", "message": "primary_client and linked_client required"}), 400
    if primary == linked:
        return jsonify({"status": "error", "message": "Cannot link a client to themselves"}), 400
    # Prevent linking a client that is already a primary of other links
    existing_links = get_kyc_linked_clients(linked)

```

```

    if existing_links:
        return jsonify({"status": "error",
            "message": f"'{linked}' is already a primary account with linked KYCs. Unlink those first."})
    if add_kyc_link(primary, linked, request.session_user.get('user_identifier', 'super_admin')):
        log_action('KYC_LINK', 'super_admin', primary, get_remote_address(), f"Linked: {linked}")
        return jsonify({"status": "success", "message": f"'{linked}' linked to '{primary}' as KYC"})
    return jsonify({"status": "error", "message": "Link already exists or failed"}), 400

@app.route('/api/kyc/unlink', methods=['POST'])
@require_role('super_admin')
def api_kyc_unlink()

```

Remove a KYC link.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_role** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns data = request.json.
2. Assigns primary = data.get('primary_client', '').strip().
3. Assigns linked = data.get('linked_client', '').strip().
4. **if** not primary or not linked: branches conditionally.
5. **if** remove_kyc_link(primary, linked): branches conditionally.
6. **return** (jsonify({'status': 'error', 'message': 'Link not found'}), 404).

Code (copy-paste safe)

```

def api_kyc_unlink():
    """Remove a KYC link."""
    data = request.json
    primary = data.get('primary_client', '').strip()
    linked = data.get('linked_client', '').strip()
    if not primary or not linked:
        return jsonify({"status": "error", "message": "primary_client and linked_client required"}), 400
    if remove_kyc_link(primary, linked):
        log_action('KYC_UNLINK', 'super_admin', primary, get_remote_address(), f"Unlinked: {linked}")
        return jsonify({"status": "success", "message": f"'{linked}' unlinked from '{primary}'"})
    return jsonify({"status": "error", "message": "Link not found"}), 404

@app.route('/api/kyc/links', methods=['GET'])
@require_role('super_admin', 'bef_admin', 'kwok_admin')
def api_kyc_list_all()

```

List all KYC links (super admin view).

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_role** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.

Step by step (top-level body)

1. **return** jsonify({'status': 'success', 'links': get_all_kyc_links()}).

Code (copy-paste safe)

```
def api_kyc_list_all():
    """List all KYC links (super admin view)."""
    return jsonify({"status": "success", "links": get_all_kyc_links()})

@app.route('/api/kyc/accounts', methods=['GET'])
@require_session
def api_kyc_accounts():
```

Get all KYC-linked accounts for the current user or a specified client.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_session** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **list comprehension** — Builds a list in a single expression ([f(x) for x in xs]). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.

Step by step (top-level body)

1. Assigns session_user = request.session_user.
2. Assigns user_type = session_user.get('user_type').
3. Assigns client_id = request.args.get('client_id', session_user.get('user_iden....
4. **if** user_type == 'client' and client_id != session_user.get('user_iden...: branches conditionally.
5. **if** is_kyc_primary(client_id): branches conditionally (with an **else/elif** arm).
6. Assigns is_bef = user_type == 'bef_admin'.
7. Lazy import from dashboard.financial_overview.
8. Assigns result = [].
9. **for** name in accounts: iterates.
10. **return** jsonify({'status': 'success', 'accounts': result, 'has_kyc_links':

Code (copy-paste safe)

```
def api_kyc_accounts():
    """Get all KYC-linked accounts for the current user or a specified client."""
```

```

session_user = request.session_user
user_type = session_user.get('user_type')
client_id = request.args.get('client_id', session_user.get('user_identifier', ''))

# Admins/super_admins can query any client; clients can only query themselves
if user_type == 'client' and client_id != session_user.get('user_identifier'):
    return jsonify({"status": "error", "message": "Access denied"}), 403

# Only primary KYC clients see linked accounts; linked clients see only themselves
if is_kyc_primary(client_id):
    accounts = get_all_kyc_accounts(client_id)
else:
    accounts = [client_id]
# Enrich with basic client info
is_bef = user_type == 'bef_admin'
from dashboard.financial_overview import get_client_profile as _gcp
result = []
for name in accounts:
    cdata = get_client_data(name)
    if not cdata:
        if is_bef:
            identity = {}
            if _gcp(name, identity) != 'BEF':
                continue
        result.append({"name": name, "eval_count": 0, "is_current": name == client_id})
        continue
    if is_bef:
        # BEF admin: only show clients whose profile/category is BEF (with hierarchy fallback)
        identity = cdata.get('identity') or {}
        if _gcp(name, identity) != 'BEF':
            continue
    evals = [ev for ev in cdata.get('evaluations', []) if isinstance(ev, dict)]
    if is_bef:
        # Exclude hidden prop firms from eval count
        evals = [ev for ev in evals
                  if str(ev.get('Prop Firm') or '').strip().lower().replace(' ',
                  '') not in BEF_HIDDEN_FIRMS]
    result.append({"name": name, "eval_count": len(evals), "is_current": name == client_id})
return jsonify({"status": "success", "accounts": result, "has_kyc_links": len(result) > 1,
               "is_primary": is_kyc_primary(client_id)})

```

```

@app.route('/api/kyc/portfolio', methods=['GET'])
@require_session
def api_kyc_portfolio()

```

Get combined portfolio stats across all KYC-linked accounts. Reads directly from stored statistics (same as client Stats tab) for consistency.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_session** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.
- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns `session_user = request.session_user`.
2. Assigns `user_type = session_user.get('user_type')`.
3. Assigns `user_id = session_user.get('user_identifier', '')`.
4. Assigns `client_id = request.args.get('client_id', '')`.
5. Assigns `is_bef = user_type == 'bef_admin'`.
6. **if** `user_type` in ('super_admin', 'bef_admin', 'kwok_admin', 'admin', 't...': branches conditionally (with an **else/elif** arm).
7. **if** `not is_kyc_primary(client_id)`: branches conditionally.
8. Assigns `from_date = request.args.get('from', '')`.
9. Assigns `to_date = request.args.get('to', '')`.
10. Assigns `accounts = get_all_kyc_accounts(client_id)`.
11. Lazy import from `dashboard.financial_overview`.
12. Defines a nested function `parse_date_safe(...)`.
13. Assigns `filter_from = parse_date_safe(from_date)` if `from_date` else `None`.
14. Assigns `filter_to = parse_date_safe(to_date)` if `to_date` else `None`.
15. ... and 16 more statement(s) in the body.

Code (copy-paste safe)

```
def api_kyc_portfolio():
    """Get combined portfolio stats across all KYC-linked accounts.
    Reads directly from stored statistics (same as client Stats tab) for consistency.
    """
    session_user = request.session_user
    user_type = session_user.get('user_type')
    user_id = session_user.get('user_identifier', '')
```

```

client_id = request.args.get('client_id', '')

is_bef = user_type == 'bef_admin'

# Determine which client to query
if user_type in ('super_admin', 'bef_admin', 'kwok_admin', 'admin', 'trader'):
    if not client_id:
        return jsonify({"status": "error", "message": "client_id required"}), 400
elif user_type == 'client':
    client_id = client_id or user_id
    if client_id != user_id:
        return jsonify({"status": "error", "message": "Access denied"}), 403
else:
    return jsonify({"status": "error", "message": "Access denied"}), 403

if not is_kyc_primary(client_id):
    return jsonify({"status": "error", "message": "Not a primary KYC account"}), 403

from_date = request.args.get('from', '')
to_date = request.args.get('to', '')

accounts = get_all_kyc_accounts(client_id)
from dashboard.financial_overview import parse_currency, get_client_profile as _gcp

def parse_date_safe(val):
    if not val or not isinstance(val, str):
        return None
    clean = val.strip()
    if not clean or clean in ('-', 'n/a', 'null', ''):
        return None
    for fmt in ('%m/%d/%y', '%m/%d/%Y', '%Y-%m-%d', '%d/%m/%Y', '%Y/%m/%d',
                '%m-%d-%Y', '%m-%d-%y', '%d-%m-%Y', '%d-%m-%y',
                '%b %d, %Y', '%B %d, %Y', '%d %b %Y', '%d %B %Y'):
        try:
            return datetime.strptime(clean, fmt).date()
        except ValueError:
            continue
    try:
        return datetime.fromisoformat(clean.replace('Z', '+00:00')).date()
    except Exception:
        return None

filter_from = parse_date_safe(from_date) if from_date else None
filter_to = parse_date_safe(to_date) if to_date else None
has_date_filter = bool(filter_from or filter_to)

def date_in_period(date_str):
    if not has_date_filter:
        return True
    d = parse_date_safe(date_str)
    if not d:
        return True
    if filter_from and d < filter_from:
        return False
    if filter_to and d > filter_to:
        return False
    return True

def eval_in_period(ev):
    if not has_date_filter:

```

```

        return True
    d = parse_date_safe(ev.get('Date Purchased')) or parse_date_safe(ev.get('Date Started'))
    if not d:
        return False
    if filter_from and d < filter_from:
        return False
    if filter_to and d > filter_to:
        return False
    return True

def format_display_date(date_str):
    if not date_str or not isinstance(date_str, str):
        return date_str or '-'
    d = parse_date_safe(date_str)
    if not d:
        return date_str
    return f"{d.strftime('%b')} {d.day}, {d.year}"

totals = {
    "total_payouts": 0.0, "total_deposits": 0.0, "total_fees": 0.0,
    "total_net_profit": 0.0, "total_hedge": 0.0, "total_farming": 0.0,
    "active_accounts": 0, "passed_accounts": 0, "failed_accounts": 0,
    "total_evaluations": 0
}
per_account = []
all_payouts = []
by_prop_firm = {}

for name in accounts:
    cdata = get_client_data(name)
    if not cdata:
        if is_bef:
            if _gcp(name, {}) != 'BEF':
                continue
            per_account.append({"name": name, "eval_count": 0, "payouts": 0, "fees": 0, "hedge": 0,
                                "farming": 0, "net": 0, "active": 0, "passed": 0, "failed": 0})
            continue
    # BEF admin: skip clients whose profile is not BEF (with hierarchy fallback)
    if is_bef:
        identity = cdata.get('identity') or {}
        if _gcp(name, identity) != 'BEF':
            continue

    all_evals = [ev for ev in cdata.get('evaluations', []) if isinstance(ev, dict)]

    # BEF admin: exclude hidden prop firms (Lucid, Apex, TradeDay, TopOneFutures)
    if is_bef:
        all_evals = [ev for ev in all_evals
                      if str(ev.get('Prop Firm') or '').strip().lower().replace(' ',
                                          ' ') not in BEF_HIDDEN_FIRMS]

    if not has_date_filter and not is_bef:
        # ■■■ No date filter: use stored statistics (consistent with Stats tab) ■■■
        stats = cdata.get('statistics', {}) or {}
        cashflow = stats.get('cashflow_inprogress', {})
        et = stats.get('eval_totals', {})

        s_payouts = cashflow.get('payouts', 0.0) or 0.0
        s_fees = cashflow.get('challenge_fees', 0.0) or 0.0
        s_hedge = cashflow.get('hedging_results', 0.0) or 0.0

```

```

s_farming = cashflow.get('farming_results', 0.0) or 0.0
s_net = s_payouts + s_hedge + s_farming - s_fees
s_active = et.get('total_running', 0) or 0
s_passed = et.get('total_passed', 0) or 0
s_failed = et.get('total_failed', 0) or 0
s_evals = len(all_evals)

# If stored eval_totals seem incomplete, recount from raw evaluations
if s_evals > 0 and (s_active + s_passed + s_failed) == 0:
    for ev in all_evals:
        sp1 = str(ev.get('Status P1') or '').strip().lower()
        sf = str(ev.get('Status') or '').strip().lower()
        if sp1 == 'fail' or sf in ('fail', 'failed', 'breached', 'blown'):
            s_failed += 1
        elif sf in ('completed', 'passed', 'funded'):
            s_passed += 1
        else:
            s_active += 1

acc_stats = {
    "name": name, "eval_count": s_evals,
    "payouts": round(s_payouts, 2), "fees": round(s_fees, 2),
    "hedge": round(s_hedge, 2), "farming": round(s_farming, 2),
    "net": round(s_net, 2),
    "active": s_active, "passed": s_passed, "failed": s_failed
}
else:
    # ■■■ Date filter active: recalculate from evaluations in period ■■■
    period_evals = [ev for ev in all_evals if eval_in_period(ev)]
    acc_stats = {"name": name, "eval_count": len(period_evals),
        "payouts": 0.0, "fees": 0.0, "hedge": 0.0, "farming": 0.0,
        "net": 0.0, "active": 0, "passed": 0, "failed": 0}

    for ev in period_evals:
        status = str(ev.get('Status') or '').lower()
        if any(s in status for s in ['passed', 'funded']):
            acc_stats["passed"] += 1
        elif any(s in status for s in ['failed', 'breached', 'blown', 'fail']):
            acc_stats["failed"] += 1
        elif any(s in status for s in ['active', 'phase', 'running', 'ongoing', 'trading',
            'challenge']):
            acc_stats["active"] += 1

    acc_stats["fees"] += parse_currency(ev.get('Fee')) + parse_currency(ev.get('Activation Fee'))

    # Only count hedge/farming for rows with a populated status
    ev_status_p1 = str(ev.get('Status P1') or '').strip()
    ev_status_funded = str(ev.get('Status') or '').strip()
    if ev_status_p1:
        for col in ['Hedge Result 1', 'Hedge Result 2', 'Hedge Result 3', 'Hedge Result 4',
            'Hedge Result 5']:
            acc_stats["hedge"] += parse_currency(ev.get(col))
    if ev_status_funded:
        for col in ['Hedge Result 1.1', 'Hedge Result 2.1', 'Hedge Result 3.1',
            'Hedge Result 4.1',
            'Hedge Result 5.1', 'Hedge Result 6', 'Hedge Result 7']:
            acc_stats["hedge"] += parse_currency(ev.get(col))
        for di in range(1, 51):
            acc_stats["farming"] += parse_currency(ev.get(f'Hedge Day {di}'))

# Payouts from ALL evals filtered by individual payout date

```

```

for ev in all_evals:
    for i in range(1, 10):
        pval = parse_currency(ev.get(f'Payout {i}'))
        if pval != 0:
            pdate = str(ev.get(f'Date {i}') or '-').strip()
            if date_in_period(pdate):
                acc_stats["payouts"] += pval

acc_stats["net"] = round(acc_stats["payouts"] - acc_stats["fees"] + acc_stats[
    "hedge"] + acc_stats["farming"], 2)
acc_stats["payouts"] = round(acc_stats["payouts"], 2)
acc_stats["fees"] = round(acc_stats["fees"], 2)
acc_stats["hedge"] = round(acc_stats["hedge"], 2)
acc_stats["farming"] = round(acc_stats["farming"], 2)

# Accumulate totals
totals["total_payouts"] += acc_stats["payouts"]
totals["total_fees"] += acc_stats["fees"]
totals["total_hedge"] += acc_stats["hedge"]
totals["total_farming"] += acc_stats["farming"]
totals["total_net_profit"] += acc_stats["net"]
totals["active_accounts"] += acc_stats["active"]
totals["passed_accounts"] += acc_stats["passed"]
totals["failed_accounts"] += acc_stats["failed"]
totals["total_evaluations"] += acc_stats["eval_count"]

per_account.append(acc_stats)

# Collect individual payout records and prop firm breakdown from evals
for ev in all_evals:
    prop_firm = str(ev.get('Prop Firm') or 'Unknown').strip() or 'Unknown'
    account_num = str(ev.get('Account #') or ev.get('Account #.1') or '-').strip()

    for i in range(1, 10):
        pval = parse_currency(ev.get(f'Payout {i}'))
        if pval != 0:
            pdate = str(ev.get(f'Date {i}') or '-').strip()
            if date_in_period(pdate):
                raw_date = parse_date_safe(pdate)
                all_payouts.append({
                    "client": name, "prop_firm": prop_firm, "account": account_num,
                    "payout_num": i, "amount": round(pval, 2), "date": format_display_date(pdate),
                    "_sort_date": raw_date.isoformat() if raw_date else "0000-00-00"
                })

    if prop_firm not in by_prop_firm:
        by_prop_firm[prop_firm] = {"evals": 0, "payouts": 0.0, "fees": 0.0, "hedge": 0.0,
            "farming": 0.0, "net": 0.0, "active": 0, "passed": 0, "failed": 0}
    pf = by_prop_firm[prop_firm]
    pf["evals"] += 1

    status = str(ev.get('Status') or '').lower()
    status_p1_raw = str(ev.get('Status P1') or '').strip()
    status_funded_raw = str(ev.get('Status') or '').strip()
    fee = parse_currency(ev.get('Fee'))
    act_fee = parse_currency(ev.get('Activation Fee'))
    pf["fees"] += (fee + act_fee)

# Only count hedge/farming for rows with a populated status
if status_p1_raw:

```

```

        for col in ['Hedge Result 1', 'Hedge Result 2', 'Hedge Result 3', 'Hedge Result 4',
                    'Hedge Result 5']:
            pf["hedge"] += parse_currency(ev.get(col))
    if status_funded_raw:
        for col in ['Hedge Result 1.1', 'Hedge Result 2.1', 'Hedge Result 3.1', 'Hedge Result 4.1',
                    'Hedge Result 5.1', 'Hedge Result 6', 'Hedge Result 7']:
            pf["hedge"] += parse_currency(ev.get(col))
        for di in range(1, 51):
            pf["farming"] += parse_currency(ev.get(f'Hedge Day {di}'))

    for j in range(1, 10):
        pf["payouts"] += parse_currency(ev.get(f'Payout {j}'))

    if any(s in status for s in ['passed', 'funded']):
        pf["passed"] += 1
    elif any(s in status for s in ['failed', 'breached', 'blown', 'fail']):
        pf["failed"] += 1
    elif any(s in status for s in ['active', 'phase', 'running', 'ongoing', 'trading', 'challenge']):
        pf["active"] += 1

# Round totals
for k in ["total_payouts", "total_fees", "total_hedge", "total_farming", "total_net_profit"]:
    totals[k] = round(totals[k], 2)

# Round prop firm breakdown
prop_firm_list = []
for pf_name, pf in by_prop_firm.items():
    pf["net"] = round(pf["payouts"] - pf["fees"] + pf["hedge"] + pf["farming"], 2)
    pf["payouts"] = round(pf["payouts"], 2)
    pf["fees"] = round(pf["fees"], 2)
    pf["hedge"] = round(pf["hedge"], 2)
    pf["farming"] = round(pf["farming"], 2)
    prop_firm_list.append({"name": pf_name, **pf})
prop_firm_list.sort(key=lambda x: x["payouts"], reverse=True)

all_payouts.sort(key=lambda x: x.get("_sort_date", "0000-00-00"), reverse=True)
for p in all_payouts:
    p.pop("_sort_date", None)

return jsonify({
    "status": "success",
    "primary": client_id,
    "totals": totals,
    "accounts": per_account,
    "payouts": all_payouts,
    "by_prop_firm": prop_firm_list,
    "period": {"from": from_date, "to": to_date}
})

@app.route('/api/add_admin', methods=['POST'])
def api_add_admin()

```

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.

Step by step (top-level body)

1. Assigns `name = request.json.get('name')`.
2. Assigns `email = request.json.get('email', '')`.
3. **if** `not name`: branches conditionally.
4. **if** `add_admin(name, email)`: branches conditionally.
5. **return** `(jsonify({'status': 'error', 'message': 'Admin exists'}), 400)`.

Code (copy-paste safe)

```
def api_add_admin():
    name = request.json.get('name')
    email = request.json.get('email', '')
    if not name: return jsonify({"status": "error", "message": "Name required"}), 400

    if add_admin(name, email):
        log_action('ADD_ADMIN', 'system', name, get_remote_address())
        return jsonify({"status": "success"})
    return jsonify({"status": "error", "message": "Admin exists"}), 400

@app.route('/api/delete_user', methods=['POST'])
@require_role('super_admin')
def api_delete_user():
```

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_role** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `data = request.json`.
2. Assigns `user_type = data.get('type')`.
3. Assigns `name = data.get('name')`.
4. Assigns `admin = data.get('admin')`.
5. Assigns `trader = data.get('trader')`.
6. **if** `not user_type` or `not name`: branches conditionally.
7. Assigns `result = False`.
8. **if** `user_type == 'admin'`: branches conditionally (with an **else/elif** arm).
9. **if** `result`: branches conditionally (with an **else/elif** arm).

Code (copy-paste safe)

```
def api_delete_user():
    data = request.json
    user_type = data.get('type')
    name = data.get('name')
```

```

admin = data.get('admin')
trader = data.get('trader')

if not user_type or not name:
    return jsonify({"status": "error", "message": "Missing arguments"}), 400

result = False
if user_type == 'admin':
    result = remove_admin(name)
    delete_user_credential(name, 'admin')

elif user_type == 'trader':
    if not admin: return jsonify({"status": "error", "message": "Admin parent required"}), 400
    result = remove_trader(admin, name)
    delete_user_credential(name, 'trader')

elif user_type == 'client':
    result = remove_client(admin or '', trader or '', name)
    delete_user_credential(name, 'client')
    # Always delete client data from database (even if hierarchy removal failed,
    # e.g. name mismatch between identity display name and hierarchy name)
    delete_client_data(name)
    if not result:
        # Try to find and remove from hierarchy by searching all admins/traders
        from config.hierarchy import SYSTEM_HIERARCHY, save_hierarchy
        for a_name, a_data in SYSTEM_HIERARCHY.get("admins", {}).items():
            for t_name, t_data in a_data.get("traders", {}).items():
                for i, client in enumerate(t_data.get("clients", [])):
                    if client.get("name") == name:
                        del t_data["clients"][i]
                        save_hierarchy(SYSTEM_HIERARCHY)
                        result = True
                        break
                if result:
                    break
            if result:
                break
        if not result:
            # Client data was still deleted from DB, consider it a success
            result = True

if result:
    log_action(f'DELETE_{user_type.upper()}', user_type, name, get_remote_address())
    return jsonify({"status": "success"})
else:
    return jsonify({"status": "error", "message": "Delete failed (not found)"}), 400

@app.route('/api/update_admin', methods=['POST'])
def api_update_admin()

```

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `name = request.json.get('name')`.
2. Assigns `email = request.json.get('email')`.
3. Assigns `slack_user_id = request.json.get('slack_user_id', None)`.
4. Assigns `new_name = request.json.get('new_name', '').strip()`.
5. **if** `not name`: branches conditionally.
6. **if** `new_name` and `new_name != name`: branches conditionally.
7. **if** `update_admin_details(name, email, slack_user_id=slack_user_id)`: branches conditionally.
8. **return** `(jsonify({'status': 'error', 'message': 'Admin not found'}), 400)`.

Code (copy-paste safe)

```
def api_update_admin():
    name = request.json.get('name')
    email = request.json.get('email')
    slack_user_id = request.json.get('slack_user_id', None)
    new_name = request.json.get('new_name', '').strip()
    if not name: return jsonify({"status": "error", "message": "Name required"}), 400

    # Rename if new_name provided and different
    if new_name and new_name != name:
        if not rename_admin(name, new_name, email):
            return jsonify({"status": "error", "message": "Rename failed (name taken or not found)"})
        rename_user_credential(name, new_name, 'admin')
        log_action('RENAME_ADMIN', 'admin', f'{name} -> {new_name}', get_remote_address())
        return jsonify({"status": "success"})

    if update_admin_details(name, email, slack_user_id=slack_user_id):
        update_user_email(name, 'admin', email)
        log_action('UPDATE_ADMIN', 'admin', name, get_remote_address())
        return jsonify({"status": "success"})
    return jsonify({"status": "error", "message": "Admin not found"}), 400

@app.route('/api/update_trader', methods=['POST'])
def api_update_trader():
```

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `admin = request.json.get('admin')`.
2. Assigns `name = request.json.get('name')`.
3. Assigns `email = request.json.get('email')`.
4. Assigns `new_name = request.json.get('new_name', '').strip()`.
5. **if** `not admin` or `not name`: branches conditionally.
6. **if** `new_name` and `new_name != name`: branches conditionally.

7. **if** `update_trader_details(admin, name, email)`: branches conditionally.
8. **return** `(jsonify({'status': 'error', 'message': 'Trader not found'}), 400)`.

Code (copy-paste safe)

```
def api_update_trader():
    admin = request.json.get('admin')
    name = request.json.get('name')
    email = request.json.get('email')
    new_name = request.json.get('new_name', '').strip()
    if not admin or not name: return jsonify({"status": "error", "message": "Missing fields"}), 400

    # Rename if new_name provided and different
    if new_name and new_name != name:
        if not rename_trader(admin, name, new_name, email):
            return jsonify({"status": "error", "message": "Rename failed (name taken or not found)"}), 400
        rename_user_credential(name, new_name, 'trader')
        log_action('RENAME_TRADER', 'trader', f'{name} -> {new_name}', get_remote_address(),
                  f"Admin: {admin}")
        return jsonify({"status": "success"})

    if update_trader_details(admin, name, email):
        update_user_email(name, 'trader', email)
        log_action('UPDATE_TRADER', 'admin', name, get_remote_address(), f"Admin: {admin}")
        return jsonify({"status": "success"})
    return jsonify({"status": "error", "message": "Trader not found"}), 400

@app.route('/api/update_client', methods=['POST'])
def api_update_client():
```

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `admin = request.json.get('admin')`.
2. Assigns `trader = request.json.get('trader')`.
3. Assigns `name = request.json.get('name')`.
4. Assigns `email = request.json.get('email', '')`.
5. Assigns `category = request.json.get('category', '')`.
6. Assigns `new_name = request.json.get('new_name', '').strip()`.
7. **if** `not admin or not trader or (not name)`: branches conditionally.
8. **if** `new_name and new_name != name`: branches conditionally.
9. **if** `category`: branches conditionally.

10. Assigns `active_status = request.json.get('active_status')`.
11. `if active_status:` branches conditionally.
12. Assigns `split_pct = request.json.get('split_pct')`.
13. `if split_pct is not None:` branches conditionally.
14. `if update_client_details(admin, trader, name, email):` branches conditionally.
15. ... and 1 more statement(s) in the body.

Code (copy-paste safe)

```
def api_update_client():
    admin = request.json.get('admin')
    trader = request.json.get('trader')
    name = request.json.get('name')
    email = request.json.get('email', '')
    category = request.json.get('category', '')
    new_name = request.json.get('new_name', '').strip()

    if not admin or not trader or not name: return jsonify({"status": "error",
        "message": "Missing fields"}), 400

    # Rename if new_name provided and different
    if new_name and new_name != name:
        if not rename_client(admin, trader, name, new_name, email, category or None):
            return jsonify({"status": "error", "message": "Rename failed (not found)"}), 400
        rename_client_in_db(name, new_name)
        rename_user_credential(name, new_name, 'client')
        log_action('RENAME_CLIENT', 'client', f'{name} -> {new_name}', get_remote_address(),
            f"Trader: {trader}")
        return jsonify({"status": "success"})

    if category:
        update_client_category(admin, trader, name, category)
        # Also sync category into the DB identity blob so financial overview reads it
        try:
            client_data = get_client_data(name)
            if client_data:
                identity = client_data.get('identity', {})
                identity['profile'] = category
                identity['category'] = category
                update_client_field(name, 'identity', identity)
        except Exception as e:
            print(f"Error syncing category to DB identity: {e}")

    # Save active_status if provided
    active_status = request.json.get('active_status')
    if active_status:
        try:
            client_data = get_client_data(name)
            if client_data:
                identity = client_data.get('identity', {})
                identity['active_status'] = active_status
                update_client_field(name, 'identity', identity)
        except Exception as e:
            print(f"Error updating active_status: {e}")

    # Save split_pct (profit split percentage) if provided
    split_pct = request.json.get('split_pct')
```

```

if split_pct is not None:
    try:
        split_pct_val = int(split_pct)
        if 0 <= split_pct_val <= 100:
            cd = get_client_data(name)
            if cd:
                identity = cd.get('identity', {})
                identity['split_pct'] = split_pct_val
                update_client_field(name, 'identity', identity)
    except (ValueError, TypeError) as e:
        print(f"Error updating split_pct: {e}")

if update_client_details(admin, trader, name, email):
    update_user_email(name, 'client', email)
    log_action('UPDATE_CLIENT', 'trader', name, get_remote_address(), f"Trader: {trader}")
    return jsonify({"status": "success"})
return jsonify({"status": "error", "message": "Client not found"}), 400

@app.route('/api/add_trader', methods=['POST'])
def api_add_trader():

```

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns admin = request.json.get('admin').
2. Assigns name = request.json.get('name').
3. Assigns email = request.json.get('email', '').
4. **if** not admin or not name: branches conditionally.
5. **if** add_trader(admin, name, email): branches conditionally.
6. **return** (jsonify({'status': 'error', 'message': 'Invalid request or Trader

Code (copy-paste safe)

```

def api_add_trader():
    admin = request.json.get('admin')
    name = request.json.get('name')
    email = request.json.get('email', '')
    if not admin or not name: return jsonify({"status": "error", "message": "Missing fields"}), 400

    if add_trader(admin, name, email):
        # Also ensure record exists in database
        try:
            # We don't have a "trader" table in database, user_credentials holds trader logins.
            if not user_exists(name, 'trader'):
                temp_pass = "trader123"

```

```

        create_user(name, temp_pass, 'trader', email)
    except Exception as e:
        print(f"Error creating trader DB user: {e}")

    log_action('ADD_TRADER', 'admin', name, get_remote_address(), f"Admin: {admin}")
    return jsonify({"status": "success"})
return jsonify({"status": "error", "message": "Invalid request or Trader exists"}), 400

@app.route('/api/add_client', methods=['POST'])
def api_add_client():

```

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `admin = request.json.get('admin')`.
2. Assigns `trader = request.json.get('trader')`.
3. Assigns `name = request.json.get('name')`.
4. Assigns `email = request.json.get('email', '')`.
5. Assigns `category = request.json.get('category', '')`.
6. **if** not admin or not trader or (not name): branches conditionally.
7. **if** `add_client(admin, trader, name, email, category)`: branches conditionally.
8. **return** (`jsonify({'status': 'error', 'message': 'Invalid request or Client ....`

Code (copy-paste safe)

```

def api_add_client():
    admin = request.json.get('admin')
    trader = request.json.get('trader')
    name = request.json.get('name')
    email = request.json.get('email', '')
    category = request.json.get('category', '')

    if not admin or not trader or not name: return jsonify({"status": "error",
        "message": "Missing fields"}), 400

    if add_client(admin, trader, name, email, category):
        # IMPORTANT: Initialize database record for new client
        try:
            if not get_client_data(name):
                initial_data = {
                    "identity": {
                        "name": name,
                        "email": email,

```

```

        "category": category,
        "profile": category,
        "source": category or "Private",
        "admin": admin,
        "trader": trader,
        "client": name
    }
}
save_client_data(name, initial_data)

# Create user credential for client portal access
if email and not user_exists(name, 'client') and not find_user_by_identifier(email):
    temp_pass = "client123"
    create_user(name, temp_pass, 'client', email)

except Exception as e:
    print(f"Error creating client DB record: {e}")

log_action('ADD_CLIENT', 'trader', name, get_remote_address(), f"Trader: {trader}")
return jsonify({"status": "success"})
return jsonify({"status": "error", "message": "Invalid request or Client exists"}), 400

@app.route('/api/move_user', methods=['POST'])
def api_move_user()

```

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns data = request.json.
2. Assigns user_type = data.get('type').
3. Assigns name = data.get('name').
4. **if** user_type == 'trader': branches conditionally (with an **else/elif** arm).
5. **return** (jsonify({'status': 'error', 'message': 'Invalid type'}), 400).

Code (copy-paste safe)

```

def api_move_user():
    data = request.json
    user_type = data.get('type')
    name = data.get('name')

    if user_type == 'trader':

```



```

old_admin = data.get('old_admin')
new_admin = data.get('new_admin')
if not all([name, old_admin, new_admin]):
    return jsonify({"status": "error", "message": "Missing fields"}), 400

if move_trader(name, old_admin, new_admin):
    try:
        # Update DB using raw SQL since no helper exposes raw update
        # We need to manually get a connection from the pool/manager
        # get_connection is imported from dashboard.database
        with get_connection() as conn:
            cursor = conn.cursor()
            cursor.execute(
                "UPDATE user_credentials SET parent_admin = ? WHERE username = ? AND user_type = ?"
            )
            # Also update all clients of this trader to point to new admin
            cursor.execute(
                "UPDATE user_credentials SET parent_admin = ? WHERE parent_trader = ? AND user_type = ?"
            )
            conn.commit()
        except Exception as e:
            print(f"DB update failed: {e}")

        log_action('MOVE_TRADER', 'super_admin', name, get_remote_address(),
            f"{old_admin} -> {new_admin}")
        return jsonify({"status": "success"})
    else:
        return jsonify({"status": "error",
            "message": "Move failed (invalid target or user not found)"}), 400

elif user_type == 'client':
    old_trader = data.get('old_trader')
    old_admin = data.get('old_admin')
    new_trader = data.get('new_trader') # The target trader
    new_admin = data.get('new_admin') # The admin of the target trader (required for consistency)

    if not all([name, old_trader, old_admin, new_trader, new_admin]):
        return jsonify({"status": "error", "message": "Missing fields"}), 400

    # Signature: move_client(client_name, old_admin, old_trader, new_admin, new_trader)
    if move_client(name, old_admin, old_trader, new_admin, new_trader):
        try:
            with get_connection() as conn:
                cursor = conn.cursor()
                cursor.execute(
                    "UPDATE user_credentials SET parent_trader = ?, parent_admin = ? WHERE username = ?"
                    (new_trader, new_admin, name)
                )
                conn.commit()
            except Exception as e:
                print(f"DB update failed: {e}")

            log_action('MOVE_CLIENT', 'super_admin', name, get_remote_address(),
                f"{old_trader} -> {new_trader}")
            return jsonify({"status": "success"})
        else:
            return jsonify({"status": "error", "message": "Move failed (duplicate name?)"}), 400

    return jsonify({"status": "error", "message": "Invalid type"}), 400

@app.route('/api/update_client_profile', methods=['POST'])

```

```
@require_session
def api_update_client_profile()
```

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_session** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `session_user = request.session_user`.
2. **if** `session_user.get('user_type') not in ['admin', 'super_admin']`: branches conditionally.
3. Assigns `data = request.json`.
4. Assigns `admin = data.get('admin')`.
5. Assigns `trader = data.get('trader')`.
6. Assigns `name = data.get('name')`.
7. Assigns `category = data.get('category')`.
8. Assigns `active_status = data.get('active_status', 'active')`.
9. **if** not `admin` or not `trader` or (not `name`) or (`category` is `None`): branches conditionally.
10. Lazy import from `config.hierarchy`.
11. **if** `update_client_category(admin, trader, name, category)`: branches conditionally.
12. **return** (`jsonify({'status': 'error', 'message': 'Client not found'})`, 404).

Code (copy-paste safe)

```
def api_update_client_profile():
    session_user = request.session_user
    if session_user.get('user_type') not in ['admin', 'super_admin']:
        return jsonify({"status": "error", "message": "Access denied"}), 403

    data = request.json
    admin = data.get('admin')
    trader = data.get('trader')
    name = data.get('name')
    category = data.get('category')
    active_status = data.get('active_status', 'active')

    if not admin or not trader or not name or category is None:
        return jsonify({"status": "error", "message": "Missing fields"}), 400

    from config.hierarchy import update_client_category

    if update_client_category(admin, trader, name, category):
```

```

client_id = name

try:
    client_data = get_client_data(client_id)
    if client_data:
        identity = client_data.get('identity', {})
        identity['profile'] = category
        identity['category'] = category
        identity['active_status'] = active_status
        update_client_field(client_id, 'identity', identity)
except Exception as e:
    print(f"Error updating DB identity profile: {e}")

log_action('UPDATE_CLIENT_PROFILE', session_user.get('user_type'), name, get_remote_address(),
    f"To: {category}, Status: {active_status}")
return jsonify({"status": "success"})

return jsonify({"status": "error", "message": "Client not found"}), 404

@app.route('/api/assign_client_trader', methods=['POST'])
@require_session
def api_assign_client_trader()

```

Reassign a client to a new trader and persist to: - hierarchy JSON (reassign_client_trader — auto-creates lane if needed) - user_credentials (parent_trader/parent_admin) - clients_data.identity (admin/trader fields, if record exists)

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_session** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns session_user = request.session_user.
2. if session_user.get('user_type') not in ('super_admin', 'bef_admin'): branches conditionally.
3. Assigns data = request.json or {}.

4. Assigns `client_name = (data.get('client_name') or '').strip()`.
5. Assigns `target_admin = (data.get('admin') or '').strip()`.
6. Assigns `new_trader = (data.get('new_trader') or '').strip()`.
7. `if not client_name or not target_admin or (not new_trader):` branches conditionally.
8. Lazy import from `config.hierarchy`.
9. Calls `reload_hierarchy(...)` for its side effect.
10. Assigns `current = get_client_profile(client_name)`.
11. `if not current:` branches conditionally.
12. Assigns `old_admin = (current.get('admin') or '').strip()`.
13. Assigns `old_trader = (current.get('trader') or '').strip()`.
14. `if not reassign_client_trader(client_name, target_admin, new_trader):` branches conditionally.
15. ... *and 5 more statement(s) in the body.*

Code (copy-paste safe)

```
def api_assign_client_trader():
    """
    Reassign a client to a new trader and persist to:
    - hierarchy JSON (reassign_client_trader – auto-creates lane if needed)
    - user_credentials (parent_trader/parent_admin)
    - clients_data.identity (admin/trader fields, if record exists)
    """
    session_user = request.session_user
    if session_user.get('user_type') not in ('super_admin', 'bef_admin'):
        return jsonify({"status": "error", "message": "Access denied"}), 403

    data = request.json or {}
    client_name = (data.get('client_name') or '').strip()
    target_admin = (data.get('admin') or '').strip()
    new_trader = (data.get('new_trader') or '').strip()

    if not client_name or not target_admin or not new_trader:
        return jsonify({"status": "error", "message": "Missing fields"}), 400

    # Get current location before reassigning (for logging)
    from config.hierarchy import reload_hierarchy, get_client_profile
    reload_hierarchy()
    current = get_client_profile(client_name)
    if not current:
        return jsonify({"status": "error", "message": "Client not found in hierarchy"}), 404

    old_admin = (current.get('admin') or '').strip()
    old_trader = (current.get('trader') or '').strip()

    # Persist in hierarchy (auto-creates trader lane, handles idempotent same-assignment)
    if not reassign_client_trader(client_name, target_admin, new_trader):
        return jsonify({"status": "error", "message": "Reassignment failed"}), 400

    # Skip DB/identity updates if nothing actually changed
    if old_admin == target_admin and old_trader == new_trader:
        return jsonify({"status": "success", "message": "No change"})

    # Persist in DB
```

```

try:
    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute(
            "UPDATE user_credentials SET parent_trader = ?, parent_admin = ? WHERE username = ? AND u
            (new_trader, target_admin, client_name)
        )
        conn.commit()
except Exception as e:
    print(f"[assign_client_trader] DB update failed (user_credentials): {e}")

# Update client identity record (if it exists)
try:
    client_data = get_client_data(client_name)
    if client_data:
        identity = client_data.get('identity', {}) if isinstance(client_data, dict) else {}
        if not isinstance(identity, dict):
            identity = {}
        identity['admin'] = target_admin
        identity['trader'] = new_trader
        update_client_field(client_name, 'identity', identity)
except Exception as e:
    print(f"[assign_client_trader] identity update failed: {e}")

log_action(
    'ASSIGN_CLIENT_TRADER',
    session_user.get('user_type'),
    client_name,
    get_remote_address(),
    f"{old_admin}/{old_trader} -> {target_admin}/{new_trader}"
)
return jsonify({"status": "success"})

@app.route('/api/remove_admin', methods=['POST'])
def api_remove_admin()

```

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.

Step by step (top-level body)

1. Assigns name = request.json.get('name').
2. **if not name:** branches conditionally.
3. **if remove_admin(name):** branches conditionally.
4. **return** (jsonify({'status': 'error', 'message': 'Admin not found'}), 400).

Code (copy-paste safe)

```

def api_remove_admin():
    name = request.json.get('name')
    if not name: return jsonify({"status": "error", "message": "Name required"}), 400

    if remove_admin(name):
        log_action('REMOVE_ADMIN', 'system', name, get_remote_address())
        return jsonify({"status": "success"})

```

```
return jsonify({"status": "error", "message": "Admin not found"}), 400
```

```
@app.route('/api/remove_trader', methods=['POST'])
def api_remove_trader()
```

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `admin = request.json.get('admin')`.
2. Assigns `name = request.json.get('name')`.
3. **if** not admin or not name: branches conditionally.
4. **if** `remove_trader(admin, name)`: branches conditionally.
5. **return** (`jsonify({'status': 'error', 'message': 'Trader not found'})`, 400).

Code (copy-paste safe)

```
def api_remove_trader():
    admin = request.json.get('admin')
    name = request.json.get('name')
    if not admin or not name: return jsonify({"status": "error", "message": "Missing fields"}), 400

    if remove_trader(admin, name):
        log_action('REMOVE_TRADER', 'admin', name, get_remote_address(), f"Admin: {admin}")
        return jsonify({"status": "success"})
    return jsonify({"status": "error", "message": "Trader not found"}), 400

@app.route('/api/remove_client', methods=['POST'])
def api_remove_client()
```

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `admin = request.json.get('admin')`.
2. Assigns `trader = request.json.get('trader')`.
3. Assigns `name = request.json.get('name')`.
4. **if** not admin or not trader or (not name): branches conditionally.
5. Assigns `client_data = get_client_data(name)`.
6. **if** `client_data`: branches conditionally.

7. **if** `remove_client(admin, trader, name)`: branches conditionally.
8. **return** `(jsonify({'status': 'error', 'message': 'Client not found'}), 400)`.

Code (copy-paste safe)

```
def api_remove_client():
    admin = request.json.get('admin')
    trader = request.json.get('trader')
    name = request.json.get('name')
    if not admin or not trader or not name: return jsonify({"status": "error",
        "message": "Missing fields"}), 400

    # Save a final snapshot before deletion so data can be recovered
    client_data = get_client_data(name)
    if client_data:
        from dashboard.database import save_data_snapshot
        save_data_snapshot(
            name, client_data,
            action='CLIENT_DELETED',
            changed_by='system',
            changed_by_type='admin',
            ip_address=get_remote_address(),
            change_source='client_removal',
            change_description=f"Final snapshot before client removal (Trader: {trader}, Admin: {admin})"
        )

    if remove_client(admin, trader, name):
        log_action('REMOVE_CLIENT', 'trader', name, get_remote_address(), f"Trader: {trader}")
        return jsonify({"status": "success"})
    return jsonify({"status": "error", "message": "Client not found"}), 400

@app.route('/api/client/delete_evaluation', methods=['POST'])
@limiter.limit('30 per minute')
def api_delete_evaluation()
```

Delete an evaluation row with history tracking. Only super_admin users can delete evaluation rows. The data is removed from current view but can be recovered from version history.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@limiter.limit** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses `x` if `cond` else `y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns `session_token = request.cookies.get('session_token')`.
2. **if** `not session_token`: branches conditionally.
3. Assigns `session_info = validate_session(session_token)`.

4. **if** not session_info: branches conditionally.
5. **if** session_info.get('user_type') != 'super_admin': branches conditionally.
6. Assigns data = request.json.
7. Assigns email = data.get('email', "").strip().lower().
8. Assigns evaluation_index = data.get('index').
9. **if** not email: branches conditionally.
10. **if** evaluation_index is None: branches conditionally.
11. Lazy import from config.hierarchy.
12. Assigns client_info = get_client_by_email(email).
13. **if** not client_info: branches conditionally.
14. Assigns client_id = client_info['client'].
15. ... and 27 more statement(s) in the body.

Code (copy-paste safe)

```
def api_delete_evaluation():
    """
    Delete an evaluation row with history tracking.
    Only super_admin users can delete evaluation rows.
    The data is removed from current view but can be recovered from version history.
    """
    # Require super_admin for all deletes
    session_token = request.cookies.get('session_token')
    if not session_token:
        return jsonify({"status": "error", "message": "Authentication required"}), 401
    session_info = validate_session(session_token)
    if not session_info:
        return jsonify({"status": "error", "message": "Invalid or expired session"}), 401
    if session_info.get('user_type') != 'super_admin':
        log_action('DELETE_DENIED', session_info.get('user_type'), session_info.get('user_identifier'),
                  get_remote_address(), 'Attempted evaluation delete without super_admin role', False)
        return jsonify({"status": "error", "message": "Only super admins can delete evaluations"}), 403

    data = request.json
    email = data.get('email', '').strip().lower()
    evaluation_index = data.get('index')

    if not email:
        return jsonify({"status": "error", "message": "Email required"}), 400

    if evaluation_index is None:
        return jsonify({"status": "error", "message": "Evaluation index required"}), 400

    # Look up client by email
    from config.hierarchy import get_client_by_email
    client_info = get_client_by_email(email)
    if not client_info:
        return jsonify({"status": "error", "message": "Email not registered"}), 404

    client_id = client_info['client']

    # Get current client data
```



```

client_data = get_client_data(client_id)
if not client_data:
    return jsonify({"status": "error", "message": "Client data not found"}), 404

evaluations = client_data.get('evaluations', [])

if evaluation_index < 0 or evaluation_index >= len(evaluations):
    return jsonify({"status": "error", "message": "Invalid evaluation index"}), 400

# Get details of what we're deleting for the log
deleted_eval = evaluations[evaluation_index]
deleted_info = f"Row {evaluation_index + 1}: {deleted_eval.get('Prop Firm', 'Unknown')} - {deleted_eval.get('Description', '')}"

# Remove the evaluation
evaluations.pop(evaluation_index)
evaluations = recalculate_hedge_nets(evaluations)
client_data['evaluations'] = evaluations

# Recalculate statistics with existing MT5 + historical accounts
from utils.data_processor import calculate_statistics
existing_mt5 = client_data.get('account') or None
existing_hr_del = client_data.get('statistics', {}).get('hedging_review', {})
existing_hist = existing_hr_del.get('historical_accounts')
new_stats = calculate_statistics(evaluations, None, existing_mt5 if existing_mt5 else None,
                                historical_accounts=existing_hist)

# ALWAYS preserve hedging_review MT5-derived fields
new_hr = new_stats.setdefault('hedging_review', {})
new_hr['total_deposits'] = existing_hr_del.get('total_deposits', 0)
new_hr['total_withdrawals'] = existing_hr_del.get('total_withdrawals', 0)
new_hr['current_balance'] = existing_hr_del.get('current_balance', 0)
new_hr['actual_hedging_results'] = existing_hr_del.get('actual_hedging_results', 0)
if existing_hist:
    new_hr['historical_accounts'] = existing_hist
    new_hr['historical_deposits'] = existing_hr_del.get('historical_deposits', 0)
    new_hr['historical_withdrawals'] = existing_hr_del.get('historical_withdrawals', 0)
    new_hr['historical_balance'] = existing_hr_del.get('historical_balance', 0)
new_hr['discrepancy'] = round(new_hr['actual_hedging_results'] - new_hr.get('sheet_hedging_results', 0), 2)

# Recalculate net_profit with discrepancy (match frontend formula)
disc = new_hr['discrepancy']
for sk in ["profitability_completed", "cashflow_inprogress"]:
    sec = new_stats[sk]
    sec["net_profit"] = round(sec["payouts"] + sec["hedging_results"] + sec["farming_results"] + disc - sec["challenge_fees"], 2)
client_data['statistics'] = new_stats

# Save with history tracking - the previous version contains the deleted row
success, version = save_client_data_with_history(
    client_id,
    client_data,
    action='DELETE_EVALUATION',
    changed_by=email,
    changed_by_type='client',
    ip_address=get_remote_address(),
    change_source='dashboard_delete',
    change_description=f"Deleted evaluation: {deleted_info}"
)

log_action('DELETE_EVALUATION', 'client', email, get_remote_address(),
          f"Deleted {deleted_info} from {client_id} (v{version})")

```

```

    return jsonify({
        "status": "success",
        "message": f"Evaluation deleted. Use Version History to recover if needed.",
        "version": version,
        "deleted": deleted_info
    })

```

```

@app.route('/api/move_client', methods=['POST'])
def api_move_client()

```

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.

Step by step (top-level body)

1. Assigns data = request.json.
2. **if** move_client(data['client_name'], data['old_admin'], data['old_trade...: branches conditionally.
3. **return** (jsonify({'status': 'error', 'message': 'Move failed'}), 400).

Code (copy-paste safe)

```

def api_move_client():
    data = request.json
    if move_client(data['client_name'], data['old_admin'], data['old_trader'], data['new_admin'], data[
        'new_trader']):
        log_action('MOVE_CLIENT', 'admin', data['client_name'], get_remote_address())
        return jsonify({"status": "success"})
    return jsonify({"status": "error", "message": "Move failed"}), 400

@app.route('/api/move_trader', methods=['POST'])
def api_move_trader()

```

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.

Step by step (top-level body)

1. Assigns data = request.json.
2. **if** move_trader(data['trader_name'], data['old_admin'], data['new_admin']): branches conditionally.
3. **return** (jsonify({'status': 'error', 'message': 'Move failed'}), 400).

Code (copy-paste safe)

```

def api_move_trader():
    data = request.json
    if move_trader(data['trader_name'], data['old_admin'], data['new_admin']):
        log_action('MOVE_TRADER', 'admin', data['trader_name'], get_remote_address())
        return jsonify({"status": "success"})
    return jsonify({"status": "error", "message": "Move failed"}), 400

@app.route('/super_admin/clients')

```

```
@require_session
def client_management()
```

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_session** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.

Step by step (top-level body)

1. **if** request.session_user.get('user_type') not in ('super_admin', 'bef_a...: branches conditionally.
2. **return** render_template('client_management.html').

Code (copy-paste safe)

```
def client_management():
    if request.session_user.get('user_type') not in ('super_admin', 'bef_admin'):
        return redirect('/')
    return render_template('client_management.html')
```

```
def can_access_client(user_type, user_identifier, target_client)
```

Check if user has permission to access a client's data.

Step by step (top-level body)

1. **if** user_type == 'super_admin': branches conditionally.
2. **if** user_type == 'kwok_admin': branches conditionally.
3. **if** user_type == 'bef_admin': branches conditionally.
4. **if** user_type == 'client': branches conditionally.
5. **for** (admin_name, admin_data) in hierarchy.get('admins', {}).items(): iterates.
6. **return** False.

Code (copy-paste safe)

```
def can_access_client(user_type, user_identifier, target_client):
    """Check if user has permission to access a client's data."""
    if user_type == 'super_admin':
        return True

    if user_type == 'kwok_admin':
        return True

    if user_type == 'bef_admin':
        # BEF admin can only access clients with category == 'BEF'
        for admin_data in hierarchy.get('admins', {}).values():
            for trader_data in admin_data.get('traders', {}).values():
                for client in trader_data.get('clients', []):
                    if (client.get('name') == target_client or client.get('email') == target_client):
                        return (client.get('category') or '').upper() == 'BEF'
        return False

    if user_type == 'client':
```

```

# Client can always access own data
if user_identifier == target_client:
    return True
# Only primary KYC clients can access linked accounts' data
if is_kyc_primary(user_identifier):
    kyc_group = get_all_kyc_accounts(user_identifier)
    return target_client in kyc_group
return False

# For admins and traders, check hierarchy
for admin_name, admin_data in hierarchy.get('admins', {}).items():
    for trader_name, trader_data in admin_data.get('traders', {}).items():
        for client in trader_data.get('clients', []):
            client_name = client.get('name', '')
            client_email = client.get('email', '')

            if client_name == target_client or client_email == target_client:
                if user_type == 'admin' and user_identifier == admin_name:
                    return True
                if user_type == 'trader' and user_identifier == trader_name:
                    return True

return False

def get_accessible_clients(user_type, user_identifier)
    Get list of client names this user can access.

```

Step by step (top-level body)

1. Assigns clients = [].
2. if user_type == 'super_admin': branches conditionally.
3. if user_type == 'kwok_admin': branches conditionally.
4. if user_type == 'bef_admin': branches conditionally.
5. if user_type == 'admin': branches conditionally.
6. if user_type == 'trader': branches conditionally.
7. if user_type == 'client': branches conditionally.
8. return [].

Code (copy-paste safe)

```

def get_accessible_clients(user_type, user_identifier):
    """Get list of client names this user can access."""
    clients = []

    if user_type == 'super_admin':
        # Super admin can access all clients
        for admin_data in hierarchy.get('admins', {}).values():
            for trader_data in admin_data.get('traders', {}).values():
                for client in trader_data.get('clients', []):
                    clients.append(client.get('name'))
        return clients

    if user_type == 'kwok_admin':
        return get_accessible_clients('super_admin', user_identifier)

```

```

if user_type == 'bef_admin':
    # BEF admin can access only BEF-category clients
    for admin_data in hierarchy.get('admins', {}).values():
        for trader_data in admin_data.get('traders', {}).values():
            for client in trader_data.get('clients', []):
                if (client.get('category') or '').upper() == 'BEF':
                    clients.append(client.get('name'))
    return clients

if user_type == 'admin':
    # Admin can access all clients under their traders
    admin_data = hierarchy.get('admins', {}).get(user_identifier, {})
    for trader_data in admin_data.get('traders', {}).values():
        for client in trader_data.get('clients', []):
            clients.append(client.get('name'))
    return clients

if user_type == 'trader':
    # Trader can access only their clients
    for admin_data in hierarchy.get('admins', {}).values():
        trader_data = admin_data.get('traders', {}).get(user_identifier, {})
        for client in trader_data.get('clients', []):
            clients.append(client.get('name'))
    return clients

if user_type == 'client':
    # Client can only access themselves
    return [user_identifier]

return []

@app.route('/api/hedging_review/<client_id>', methods=['POST'])
@require_session
def update_hedging_review(client_id)

```

Update hedging review values manually - only for traders, admins, and super_admins.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_session** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `session_user = request.session_user`.
2. Assigns `user_type = session_user.get('user_type')`.
3. Assigns `user_identifier = session_user.get('user_identifier')`.
4. **if not can_access_client(user_type, user_identifier, client_id)**: branches conditionally.
5. Assigns `data = request.json`.
6. Assigns `client_data = get_client_data(client_id)`.

7. if not client_data: branches conditionally.
8. if 'statistics' not in client_data: branches conditionally.
9. if 'hedging_review' not in client_data['statistics']: branches conditionally.
10. Assigns hr = client_data['statistics']['hedging_review'].
11. Assigns hr['total_deposits'] = float(data.get('total_deposits', hr.get('total_deposits',....
12. Assigns hr['total_withdrawals'] = float(data.get('total_withdrawals', hr.get('total_withdra....
13. Assigns hr['current_balance'] = float(data.get('current_balance', hr.get('current_balance....
14. if hr['total_withdrawals'] > 0: branches conditionally.
15. ... and 11 more statement(s) in the body.

Code (copy-paste safe)

```
def update_hedging_review(client_id):
    """Update hedging review values manually - only for traders, admins, and super_admins."""
    session_user = request.session_user
    user_type = session_user.get('user_type')
    user_identifier = session_user.get('user_identifier')

    # Allow all authenticated users to edit their own hedging review
    # Traders/admins/super_admins can edit any client they have access to

    # Check if user can access this client
    if not can_access_client(user_type, user_identifier, client_id):
        log_action('HEDGING_EDIT_DENIED', user_type, user_identifier, get_remote_address(),
                  f"No access to client: {client_id}", False)
        return jsonify({"status": "error", "message": "Access denied to this client"}), 403

    data = request.json

    # Get existing client data
    client_data = get_client_data(client_id)
    if not client_data:
        return jsonify({"status": "error", "message": "Client data not found"}), 404

    # Update hedging review in statistics
    if 'statistics' not in client_data:
        client_data['statistics'] = {}
    if 'hedging_review' not in client_data['statistics']:
        client_data['statistics']['hedging_review'] = {}

    hr = client_data['statistics']['hedging_review']
    hr['total_deposits'] = float(data.get('total_deposits', hr.get('total_deposits', 0)))
    hr['total_withdrawals'] = float(data.get('total_withdrawals', hr.get('total_withdrawals', 0)))
    hr['current_balance'] = float(data.get('current_balance', hr.get('current_balance', 0)))

    # Auto-negate withdrawals if entered as positive
    if hr['total_withdrawals'] > 0:
        hr['total_withdrawals'] = -hr['total_withdrawals']

    # Recalculate: actual = balance - (deposits + withdrawals), withdrawals are negative
    net_deposits = hr['total_deposits'] + hr['total_withdrawals']
    hr['actual_hedging_results'] = round(hr['current_balance'] - net_deposits, 2)
    sheet_hr = hr.get('sheet_hedging_results', 0)
    hr['discrepancy'] = round(hr['actual_hedging_results'] - sheet_hr, 2)
```

```

# Also store in account for consistency with MT5 push
if 'account' not in client_data:
    client_data['account'] = {}
client_data['account']['balance'] = hr['current_balance']
client_data['account']['total_deposits'] = hr['total_deposits']
client_data['account']['total_withdrawals'] = hr['total_withdrawals']

# Save updated data
save_client_data(client_id, client_data)

log_action('HEDGING_EDIT', user_type, user_identifier, get_remote_address(),
          f"Updated hedging review for {client_id}: deposits={hr['total_deposits']}, withdrawals={hr['total_withdrawals']}")

return jsonify({
    "status": "success",
    "message": "Hedging review updated",
    "hedging_review": hr
})

@app.route('/api/client/push_hedging_review', methods=['POST'])
@limiter.limit('30 per minute')
def api_push_hedging_review():

```

Public endpoint - push Live Hedging Review data using client email. Updates deposits, withdrawals, balance and recalculates actual hedging results. Used by the Trader Companion app.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@limiter.limit** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns data = request.json.
2. Assigns email = data.get('email', "").strip().lower().
3. **if** not email: branches conditionally.
4. Assigns client_info = get_client_by_email(email).
5. **if** not client_info: branches conditionally.
6. Assigns client_id = client_info['client'].
7. Assigns client_data = get_client_data(client_id).
8. **if** not client_data: branches conditionally.
9. **if** 'statistics' not in client_data: branches conditionally.
10. **if** 'hedging_review' not in client_data['statistics']: branches conditionally.
11. Assigns hr = client_data['statistics']['hedging_review'].
12. Assigns hr['total_deposits'] = float(data.get('total_deposits', hr.get('total_deposits',....

13. Assigns `hr['total_withdrawals'] = float(data.get('total_withdrawals', hr.get('total_withdra...`
14. Assigns `hr['current_balance'] = float(data.get('current_balance', hr.get('current_balance....`
15. ... and 19 more statement(s) in the body.

Code (copy-paste safe)

```
def api_push_hedging_review():
    """
    Public endpoint - push Live Hedging Review data using client email.
    Updates deposits, withdrawals, balance and recalculates actual hedging results.
    Used by the Trader Companion app.
    """
    data = request.json
    email = data.get('email', '').strip().lower()

    if not email:
        return jsonify({"status": "error", "message": "Email required"}), 400

    client_info = get_client_by_email(email)
    if not client_info:
        return jsonify({"status": "error", "message": "Email not registered in the system"}), 404

    client_id = client_info['client']
    client_data = get_client_data(client_id)
    if not client_data:
        return jsonify({"status": "error", "message": "Client data not found"}), 404

    if 'statistics' not in client_data:
        client_data['statistics'] = {}
    if 'hedging_review' not in client_data['statistics']:
        client_data['statistics']['hedging_review'] = {}

    hr = client_data['statistics']['hedging_review']
    hr['total_deposits'] = float(data.get('total_deposits', hr.get('total_deposits', 0)))
    hr['total_withdrawals'] = float(data.get('total_withdrawals', hr.get('total_withdrawals', 0)))
    hr['current_balance'] = float(data.get('current_balance', hr.get('current_balance', 0)))

    # Auto-negate withdrawals if entered as positive
    if hr['total_withdrawals'] > 0:
        hr['total_withdrawals'] = -hr['total_withdrawals']

    # Recalculate actual hedging results: balance - (deposits + withdrawals)
    # Withdrawals are negative, so deposits + withdrawals = net deposits
    net_deposits = hr['total_deposits'] + hr['total_withdrawals']
    hr['actual_hedging_results'] = round(hr['current_balance'] - net_deposits, 2)

    # Recalculate discrepancy
    sheet_hr = hr.get('sheet_hedging_results', 0)
    hr['discrepancy'] = round(hr['actual_hedging_results'] - sheet_hr, 2)

    # Also store in account for consistency
    if 'account' not in client_data:
        client_data['account'] = {}
    client_data['account']['balance'] = hr['current_balance']
    client_data['account']['total_deposits'] = hr['total_deposits']
    client_data['account']['total_withdrawals'] = hr['total_withdrawals']

    save_client_data(client_id, client_data)
```



```

app.logger.info(f"■ SAVED Hedging Review for {client_id}:")
app.logger.info(f"    - total_deposits: ${hr['total_deposits']:.2f}")
app.logger.info(f"    - total_withdrawals: ${hr['total_withdrawals']:.2f}")
app.logger.info(f"    - current_balance: ${hr['current_balance']:.2f}")
app.logger.info(f"    - actual_hedging_results: ${hr['actual_hedging_results']:.2f}")
app.logger.info(f"    - sheet_hedging_results: ${hr.get('sheet_hedging_results', 0):.2f}")
app.logger.info(f"    - discrepancy: ${hr['discrepancy']:.2f}")

log_action('PUSH_HEDGING_REVIEW', 'companion', email, get_remote_address(),
          f"Hedging review for {client_id}: deposits={hr['total_deposits']}, withdrawals={hr['total_

return jsonify({
    "status": "success",
    "message": f"Hedging review updated for {client_id}",
    "hedging_review": hr
})

```

```

@app.route('/api/client/check_mt5_auto_populate/<client_id>', methods=['GET'])
@require_session
def check_mt5_auto_populate(client_id)

```

On dashboard load, check if MT5 values need auto-population. Returns status and suggestion for initialization.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_session** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.

Step by step (top-level body)

1. Assigns session_user = request.session_user.
2. Assigns user_type = session_user.get('user_type').
3. Assigns user_identifier = session_user.get('user_identifier').
4. **if** user_type == 'client' and client_id != user_identifier: branches conditionally.
5. **if** user_type in ['trader', 'admin', 'bef_admin', 'super_admin', 'kwok_...': branches conditionally.
6. Assigns client_data = get_client_data(client_id).
7. **if** not client_data: branches conditionally.
8. Assigns hr = client_data.get('statistics', {}).get('hedging_review', {}).
9. Assigns account = client_data.get('account', {}).

10. Assigns `hedge_accounts = client_data.get('hedge_accounts') or []`.
11. Assigns `prop_accounts = client_data.get('prop_accounts') or []`.
12. Assigns `deposits = float(hr.get('total_deposits', 0))`.
13. Assigns `withdrawals = float(hr.get('total_withdrawals', 0))`.
14. Assigns `balance = float(hr.get('current_balance', 0))`.
15. ... and 6 more statement(s) in the body.

Code (copy-paste safe)

```
def check_mt5_auto_populate(client_id):
    """
    On dashboard load, check if MT5 values need auto-population.
    Returns status and suggestion for initialization.
    """
    session_user = request.session_user
    user_type = session_user.get('user_type')
    user_identifier = session_user.get('user_identifier')

    # Only allow client to check their own data or authorized users
    if user_type == 'client' and client_id != user_identifier:
        return jsonify({"status": "error", "message": "Access denied"}), 403

    # Check if user can access this client
    if user_type in ['trader', 'admin', 'bef_admin', 'super_admin', 'kwok_admin']:
        if not can_access_client(user_type, user_identifier, client_id):
            return jsonify({"status": "error", "message": "Access denied"}), 403

    client_data = get_client_data(client_id)
    if not client_data:
        return jsonify({
            "status": "needs_init",
            "message": "Client data not initialized",
            "needs_mt5": True
        })

    # Check if hedging_review MT5 values are empty/zero
    hr = client_data.get('statistics', {}).get('hedging_review', {})
    account = client_data.get('account', {})
    hedge_accounts = client_data.get('hedge_accounts') or []
    prop_accounts = client_data.get('prop_accounts') or []

    deposits = float(hr.get('total_deposits', 0))
    withdrawals = float(hr.get('total_withdrawals', 0))
    balance = float(hr.get('current_balance', 0))
    equity = float(account.get('equity', 0))

    # Check if values are truly empty (all zero or missing)
    has_mt5_values = (deposits != 0) or (withdrawals != 0) or (balance != 0) or (equity != 0)

    # Setup credentials:
    # - The legacy banner logic was: if hedge creds missing, check prop creds;
    #   if prop creds exist, do NOT prompt; otherwise prompt to set up MT5.
    has_hedge_creds = any(
        isinstance(h, dict)
        and (str(h.get('login', '')) or '').strip() or str(h.get('password', '')) or '').strip()
        for h in hedge_accounts
    )
```

```

has_prop_creds = any(
    isinstance(p, dict)
    and (str(p.get('login', '')) or '').strip() or str(p.get('password', '')) or '').strip()
    for p in prop_accounts
)

# Need MT5 setup only when MT5 values are missing AND there are no usable
# hedge creds AND no usable prop creds. If prop creds exist, bypass prompt.
needs_mt5 = (not has_mt5_values) and (not has_hedge_creds) and (not has_prop_creds)

if needs_mt5:
    if not has_mt5_values:
        app.logger.info(f"■■ MT5 auto-populate: {client_id} has zero MT5 values")
        app.logger.info(
            f"■■ MT5 auto-populate: {client_id} missing hedge + prop account credentials (setup required)"
        )
        return jsonify({
            "status": "needs_init",
            "message": "MT5 values need initialization",
            "needs_mt5": True,
            "current_values": {
                "deposits": deposits,
                "withdrawals": withdrawals,
                "balance": balance
            },
            "needs_hedge_creds": (not has_hedge_creds),
            "needs_prop_creds": (not has_prop_creds),
        })
    else:
        app.logger.info(f"■ MT5 auto-populate: {client_id} already has MT5 values")
        return jsonify({
            "status": "ok",
            "message": "MT5 values already initialized",
            "needs_mt5": False,
            "current_values": {
                "deposits": deposits,
                "withdrawals": withdrawals,
                "balance": balance,
                "equity": equity
            }
        })

```

```

@app.route('/api/historical_mt5/<client_id>', methods=['POST'])
@require_session
def manage_historical_mt5(client_id)

```

Manage historical MT5 accounts - add/delete. Only for traders, admins, and super_admins.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_session** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.

Step by step (top-level body)

1. Assigns `session_user = request.session_user`.
2. Assigns `user_type = session_user.get('user_type')`.
3. Assigns `user_identifier = session_user.get('user_identifier')`.
4. **if** `user_type` not in `['trader', 'admin', 'super_admin']`: branches conditionally.
5. **if** not `can_access_client(user_type, user_identifier, client_id)`: branches conditionally.
6. Assigns `data = request.json`.
7. Assigns `action = data.get('action')`.
8. Assigns `client_data = get_client_data(client_id)`.
9. **if** not `client_data`: branches conditionally.
10. Lazy import from `dashboard.database`.
11. Calls `save_data_snapshot(...)` for its side effect.
12. **if** `'statistics'` not in `client_data`: branches conditionally.
13. **if** `'hedging_review'` not in `client_data['statistics']`: branches conditionally.
14. Assigns `hr = client_data['statistics']['hedging_review']`.
15. ... and 11 more statement(s) in the body.

Code (copy-paste safe)

```
def manage_historical_mt5(client_id):
    """Manage historical MT5 accounts - add/delete. Only for traders, admins, and super_admins."""
    session_user = request.session_user
    user_type = session_user.get('user_type')
    user_identifier = session_user.get('user_identifier')

    # Only allow traders, admins, and super_admins to edit
    if user_type not in ['trader', 'admin', 'super_admin']:
        log_action('HISTORICAL_MT5_DENIED', user_type, user_identifier, get_remote_address(),
                  f"Client tried to edit historical MT5 for: {client_id}", False)
        return jsonify({"status": "error",
                        "message": "Only traders, admins, and super admins can manage historical MT5 accounts"}), 403

    # Check if user can access this client
    if not can_access_client(user_type, user_identifier, client_id):
        log_action('HISTORICAL_MT5_DENIED', user_type, user_identifier, get_remote_address(),
                  f"No access to client: {client_id}", False)
        return jsonify({"status": "error", "message": "Access denied to this client"}), 403

    data = request.json
    action = data.get('action') # 'add' or 'delete'

    # Get existing client data
    client_data = get_client_data(client_id)
```

```

if not client_data:
    return jsonify({"status": "error", "message": "Client data not found"}), 404

# Save snapshot BEFORE making changes (for recovery)
from dashboard.database import save_data_snapshot
save_data_snapshot(
    client_id, client_data,
    action='BEFORE_HISTORICAL_MT5_' + action.upper(),
    changed_by=user_idenfier,
    changed_by_type=user_type,
    ip_address=get_remote_address(),
    change_source='historical_mt5_management',
    change_description=f"Snapshot before {action} historical MT5 account"
)

# Ensure hedging_review structure exists
if 'statistics' not in client_data:
    client_data['statistics'] = {}
if 'hedging_review' not in client_data['statistics']:
    client_data['statistics']['hedging_review'] = {}

hr = client_data['statistics']['hedging_review']

# Initialize historical_accounts array if not exists
if 'historical_accounts' not in hr:
    hr['historical_accounts'] = []

if action == 'add':
    account = data.get('account', {})
    hr['historical_accounts'].append({
        'name': account.get('name', 'MT5 Account'),
        'deposits': float(account.get('deposits', 0)),
        'withdrawals': float(account.get('withdrawals', 0)),
        'final_balance': float(account.get('final_balance', 0)),
        'date_added': account.get('date_added', '')
    })

    log_action('HISTORICAL_MT5_ADD', user_type, user_idenfier, get_remote_address(),
        f"Added historical MT5 for {client_id}: {account.get('name')}")

elif action == 'delete':
    index = data.get('index')
    if index is not None and 0 <= index < len(hr['historical_accounts']):
        deleted = hr['historical_accounts'].pop(index)
        log_action('HISTORICAL_MT5_DELETE', user_type, user_idenfier, get_remote_address(),
            f"Deleted historical MT5 for {client_id}: {deleted.get('name')}")
    else:
        return jsonify({"status": "error", "message": "Invalid index"}), 400

elif action == 'set_prior_activity':
    target = data.get('target') # 'current' or integer index for historical
    prior_profit = float(data.get('prior_activity_profit', 0))
    if target == 'current':
        hr['current_mt5_prior_activity'] = prior_profit
        log_action('PRIOR_ACTIVITY_SET', user_type, user_idenfier, get_remote_address(),
            f"Set current MT5 prior activity for {client_id}: {prior_profit}")
    elif isinstance(target, int) and 0 <= target < len(hr['historical_accounts']):
        hr['historical_accounts'][target]['prior_activity_profit'] = prior_profit
        log_action('PRIOR_ACTIVITY_SET', user_type, user_idenfier, get_remote_address(),
            f"Set historical MT5 #{target} prior activity for {client_id}: {prior_profit}")

```

```

    else:
        return jsonify({"status": "error", "message": "Invalid target"}), 400

else:
    return jsonify({"status": "error", "message": "Invalid action"}), 400

# Recalculate totals including historical accounts
hist_deposits = sum(acc.get('deposits', 0) for acc in hr['historical_accounts'])
hist_withdrawals = sum(acc.get('withdrawals', 0) for acc in hr['historical_accounts'])
hist_balance = sum(acc.get('final_balance', 0) for acc in hr['historical_accounts'])

# Store historical totals for data_processor to use
hr['historical_deposits'] = hist_deposits
hr['historical_withdrawals'] = hist_withdrawals
hr['historical_balance'] = hist_balance

# Save updated data WITH history tracking
email = client_data.get('identity', {}).get('email', '')
success, version = save_client_data_with_history(
    client_id, client_data,
    action='HISTORICAL_MT5_' + action.upper(),
    changed_by=user_identifier,
    changed_by_type=user_type,
    ip_address=get_remote_address(),
    change_source='historical_mt5_management',
    change_description=f"{action.capitalize()} historical MT5 account"
)

return jsonify({
    "status": "success",
    "message": f"Historical MT5 {action}ed successfully",
    "historical_accounts": hr['historical_accounts'],
    "historical_totals": {
        "deposits": hist_deposits,
        "withdrawals": hist_withdrawals,
        "balance": hist_balance
    }
})

```

```

@app.route('/api/data')
@limiter.limit('180 per minute')
def get_data()

```

Get client data - requires authentication and role-based access.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@limiter.limit** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `client_id = request.args.get('client_id')`.
2. Assigns `session_token = request.cookies.get('session_token')`.
3. **if** `not session_token`: branches conditionally (with an **else/elif** arm).
4. **if** `client_id`: branches conditionally.
5. **return** `jsonify({'status': 'success', 'deals': [], 'positions': [], 'accoun....`

Code (copy-paste safe)

```
def get_data():
    """Get client data - requires authentication and role-based access."""
    client_id = request.args.get('client_id')

    # Check authentication
    session_token = request.cookies.get('session_token')
    if not session_token:
        # Also check API Key for trader apps
        api_key = request.headers.get('X-API-Key')
        if not api_key:
            return jsonify({"status": "error", "message": "Authentication required"}), 401

        # Validate API Key
        key_info = validate_api_key(api_key)
        if not key_info:
            return jsonify({"status": "error", "message": "Invalid API Key"}), 401

        user_type = 'api'
        user_identifier = key_info.get('owner')
    else:
        session_info = validate_session(session_token)
        if not session_info:
            return jsonify({"status": "error", "message": "Invalid session"}), 401

        user_type = session_info.get('user_type')
        user_identifier = session_info.get('user_identifier')

    if client_id:
        # Check if user can access this client's data
        if not can_access_client(user_type, user_identifier, client_id):
            log_action('ACCESS_DENIED', user_type, user_identifier, get_remote_address(),
                      f"Tried to access: {client_id}", False)
            return jsonify({"status": "error", "message": "Access denied"}), 403

    data = get_client_data(client_id)
    if data is None:
        return jsonify({"status": "error",
                        "message": f"No data found for client '{client_id}'. The client may not exist or has no s

    if data:
        # Inject Visual Notes
        if 'evaluations' in data:
            try:
                notes = get_client_notes(client_id)
                # notes is { row_index: { col: text } }
                for i, ev in enumerate(data['evaluations']):
                    if i in notes:
```

```

        ev['_notes'] = notes[i]
    except Exception as e:
        logging.error(f"Error injecting notes: {e}")

    # Historical MT5 values are shown separately in the MT5 Accounts Overview table.
    # The hedging_review values (deposits, withdrawals, balance, discrepancy) are
    # served as-is from data_processor.py – single source of truth.

    data['status'] = 'success'
    # Include current version so frontend can detect stale refreshes
    try:
        from dashboard.database import get_next_version
        data['_version'] = get_next_version(client_id) - 1
    except Exception:
        pass
    # Include last activity info for dashboard display
    try:
        from dashboard.database import get_client_activity
        data['_activity'] = get_client_activity(client_id)
    except Exception:
        pass
    return jsonify(data)

# If no client specified, return empty
return jsonify({
    "status": "success",
    "deals": [], "positions": [], "account": {},
    "evaluations": [], "statistics": {}, "dropdown_options": {},
    "last_updated": "Never"
})

```

```

@app.route('/api/dashboard/recalculate_statistics', methods=['POST'])
@limiter.limit('30 per minute')
def api_dashboard_recalculate_statistics()

```

Recalculate Hedge Net / Hedge Net.1 and statistics from stored evaluations and persist. Used after a full dashboard page load so DB matches formulas (separate from trader /api/client/push). Uses save_client_data (no extra history snapshot) like super_admin batch recalc.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@limiter.limit** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

Step by step (top-level body)

1. Assigns session_token = request.cookies.get('session_token').
2. if not session_token: branches conditionally.
3. Assigns session_info = validate_session(session_token).

4. **if** not session_info: branches conditionally.
5. Assigns user_type = session_info.get('user_type').
6. Assigns user_idenfier = session_info.get('user_idenfier').
7. Assigns payload = request.get_json() or {}.
8. Assigns client_id = payload.get('client_id').
9. **if** not client_id: branches conditionally.
10. **if** not can_access_client(user_type, user_idenfier, client_id): branches conditionally.
11. Assigns client_data = get_client_data(client_id).
12. **if** not client_data: branches conditionally.
13. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def api_dashboard_recalculate_statistics():
    """
    Recalculate Hedge Net / Hedge Net.1 and statistics from stored evaluations and persist.
    Used after a full dashboard page load so DB matches formulas (separate from trader /api/client/push).
    Uses save_client_data (no extra history snapshot) like super_admin batch recalc.
    """
    session_token = request.cookies.get('session_token')
    if not session_token:
        return jsonify({"status": "error", "message": "Authentication required"}), 401
    session_info = validate_session(session_token)
    if not session_info:
        return jsonify({"status": "error", "message": "Authentication required"}), 401
    user_type = session_info.get('user_type')
    user_idenfier = session_info.get('user_idenfier')
    payload = request.get_json() or {}
    client_id = payload.get('client_id')
    if not client_id:
        return jsonify({"status": "error", "message": "client_id required"}), 400
    if not can_access_client(user_type, user_idenfier, client_id):
        return jsonify({"status": "error", "message": "Access denied"}), 403

    client_data = get_client_data(client_id)
    if not client_data:
        return jsonify({"status": "error", "message": "No data found"}), 404

    try:
        from utils.data_processor import calculate_statistics
        evals = recalculate_hedge_nets(list(client_data.get('evaluations') or []))
        existing_mt5 = client_data.get('account')
        existing_hr = client_data.get('statistics', {}).get('hedging_review', {}) or {}
        existing_hist = existing_hr.get('historical_accounts')
        new_stats = calculate_statistics(evals, mt5_account=existing_mt5, historical_accounts=existing_hist)
        new_hr = new_stats.setdefault('hedging_review', {})
        new_hr['total_deposits'] = existing_hr.get('total_deposits', 0)
        new_hr['total_withdrawals'] = existing_hr.get('total_withdrawals', 0)
        new_hr['current_balance'] = existing_hr.get('current_balance', 0)
        new_hr['actual_hedging_results'] = existing_hr.get('actual_hedging_results', 0)
        if existing_hist:
            new_hr['historical_accounts'] = existing_hist
            new_hr['historical_deposits'] = existing_hr.get('historical_deposits', 0)
            new_hr['historical_withdrawals'] = existing_hr.get('historical_withdrawals', 0)
```

```

        new_hr['historical_balance'] = existing_hr.get('historical_balance', 0)
    new_hr['discrepancy'] = round(
        float(new_hr.get('actual_hedging_results', 0) or 0) - float(new_hr.get('sheet_hedging_results',
        0) or 0), 2
    )
    disc = new_hr['discrepancy']
    for sk in ["profitability_completed", "cashflow_inprogress"]:
        sec = new_stats[sk]
        sec["net_profit"] = round(
            float(sec.get("payouts", 0) or 0)
            + float(sec.get("hedging_results", 0) or 0)
            + float(sec.get("farming_results", 0) or 0)
            + disc
            - float(sec.get("challenge_fees", 0) or 0),
            2,
        )
    # NOTE: Only persist statistics here – not evaluations.
    # Writing evaluations back would race with any in-flight /api/update_data
    # call from the user (page-load recalc reads DB state S0, user's save
    # then commits S1, this recalc then overwrites evaluations back to S0
    # and silently drops the user's edit). Hedge Net / Hedge Net.1 will be
    # recomputed again on the next save (update_data also calls
    # recalculate_hedge_nets), so they don't need to be written here.
    save_client_data(client_id, {'statistics': new_stats})
    return jsonify({"status": "success"})
except Exception as e:
    app.logger.exception("recalculate_statistics on dashboard load failed")
    return jsonify({"status": "error", "message": str(e)}), 500

```

```

@app.route('/api/client/export_csv')
def export_client_csv()

```

Export client evaluation data as CSV download.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **list comprehension** — Builds a list in a single expression ([f(x) for x in xs]). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.

Step by step (top-level body)

1. Imports csv (lazy import inside the function).
2. Imports io (lazy import inside the function).
3. Assigns client_id = request.args.get('client_id').

4. **if** not client_id: branches conditionally.
5. Assigns session_token = request.cookies.get('session_token').
6. Assigns api_key = request.headers.get('X-API-Key').
7. **if** session_token: branches conditionally (with an **else/elif** arm).
8. **if** not can_access_client(user_type, user_identifier, client_id): branches conditionally.
9. Assigns data = get_client_data(client_id).
10. **if** not data: branches conditionally.
11. Assigns evaluations = data.get('evaluations', []).
12. **if** not evaluations: branches conditionally.
13. Assigns DASHBOARD_COLUMN_ORDER = ['Prop Firm', 'Account Size', 'Date Purchased', 'Fee', 'D....
14. Assigns all_keys = set().
15. ... and 19 more statement(s) in the body.

Code (copy-paste safe)

```
def export_client_csv():
    """Export client evaluation data as CSV download."""
    import csv
    import io

    client_id = request.args.get('client_id')
    if not client_id:
        return jsonify({"status": "error", "message": "client_id required"}), 400

    # Auth check
    session_token = request.cookies.get('session_token')
    api_key = request.headers.get('X-API-Key')
    if session_token:
        session_info = validate_session(session_token)
        if not session_info:
            return jsonify({"status": "error", "message": "Invalid session"}), 401
        user_type = session_info.get('user_type')
        user_identifier = session_info.get('user_identifier')
    elif api_key:
        key_info = validate_api_key(api_key)
        if not key_info:
            return jsonify({"status": "error", "message": "Invalid API key"}), 401
        user_type = 'api'
        user_identifier = key_info.get('owner')
    else:
        return jsonify({"status": "error", "message": "Authentication required"}), 401

    if not can_access_client(user_type, user_identifier, client_id):
        return jsonify({"status": "error", "message": "Access denied"}), 403

    data = get_client_data(client_id)
    if not data:
        return jsonify({"status": "error", "message": "No data found"}), 404

    evaluations = data.get('evaluations', [])
    if not evaluations:
```

```

        return jsonify({"status": "error", "message": "No evaluation data"}), 404

# Dashboard column order – matches the HTML template
DASHBOARD_COLUMN_ORDER = [
    'Prop Firm', 'Account Size', 'Date Purchased', 'Fee',
    'Date Started', 'Date Ended', 'Status P1', 'Account #',
    'Hedge Result 1', 'Hedge Result 2', 'Hedge Result 3',
    'Hedge Result 4', 'Hedge Result 5', 'Hedge Net',
    'Account #.1', 'Activation Fee', 'Date Started.1', 'Date Ended.1', 'Status',
    'Hedge Result 1.1', 'Hedge Result 2.1', 'Hedge Result 3.1',
    'Hedge Result 4.1', 'Hedge Result 5.1',
    'Hedge Result 6', 'Hedge Result 7', 'Hedge Net.1',
    'Payout 1', 'Date 1', 'Payout 2', 'Date 2',
    'Payout 3', 'Date 3', 'Payout 4', 'Date 4',
] + [f'Prop Day {i}' for i in range(1, 35)] \
    + [f'Prop Progress {i}' for i in range(1, 35)] \
    + [f'Hedge Day {i}' for i in range(1, 35)]

# Build column list: dashboard order first, then extras (skip Account Number)
all_keys = set()
for ev in evaluations:
    all_keys.update(k for k in ev if not k.startswith('_') and k != 'Account Number')
seen = set()
columns = []
for col in DASHBOARD_COLUMN_ORDER:
    if col in all_keys and col not in seen:
        seen.add(col)
        columns.append(col)
for ev in evaluations:
    for key in ev:
        if key not in seen and not key.startswith('_') and key != 'Account Number':
            seen.add(key)
            columns.append(key)

output = io.StringIO()
writer = csv.writer(output)
writer.writerow(columns)
for ev in evaluations:
    writer.writerow([ev.get(col, '') for col in columns])

# Add statistics summary rows
stats = data.get('statistics', {})
if stats:
    writer.writerow([]) # blank separator
    writer.writerow(['--- Statistics ---'])
    for key, val in stats.items():
        if isinstance(val, dict):
            for k2, v2 in val.items():
                writer.writerow([f'{key}.{k2}', v2])
        else:
            writer.writerow([key, val])

csv_content = output.getvalue()
output.close()

from flask import Response
safe_name = re.sub(r'^[a-zA-Z0-9_-]', '_', client_id)
resp = Response(csv_content, mimetype='text/csv')
resp.headers['Content-Disposition'] = f'attachment; filename={safe_name}_evaluations.csv'
log_action('CSV_EXPORT', user_type, user_identifier, get_remote_address(),

```

```

        f"Exported CSV for {client_id}")
    return resp

```

```
def _parse_date_str(val)
```

Try to parse a date string from evaluation data. Returns YYYY-MM-DD or None.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

Step by step (top-level body)

1. **if** not val: branches conditionally.
2. Assigns val = str(val).strip().rstrip('.').
3. **if** not val or len(val) < 3: branches conditionally.
4. **if** val[0].isalpha() and '/' not in val and '-' not in val: branches conditionally.
5. Assigns normalized = val.replace('.', '/').replace('//', '/').
6. **while** normalized and (not normalized[0].isdigit()): loops until the condition is false.
7. **if** not normalized: branches conditionally.
8. **for** fmt in ('%Y-%m-%d', '%m/%d/%Y', '%m/%d/%y', '%m-%d-%Y', '%m-%d-%y...: iterates.
9. Assigns parts = normalized.split('/').
10. **if** len(parts) == 2: branches conditionally.
11. **return** None.

Code (copy-paste safe)

```

def _parse_date_str(val):
    """Try to parse a date string from evaluation data. Returns YYYY-MM-DD or None."""
    if not val:
        return None
    val = str(val).strip().rstrip('.')
    if not val or len(val) < 3:
        return None
    # Skip obvious non-dates
    if val[0].isalpha() and '/' not in val and '-' not in val:
        return None

    # Normalize: replace dots with slashes, collapse double slashes
    normalized = val.replace('.', '/').replace('//', '/')
    # Strip leading non-digit chars (e.g. "V10/17/25")
    while normalized and not normalized[0].isdigit():
        normalized = normalized[1:]
    if not normalized:
        return None

    # Try standard formats
    for fmt in ('%Y-%m-%d', '%m/%d/%Y', '%m/%d/%y', '%m-%d-%Y', '%m-%d-%y',
               '%b %d, %Y', '%B %d, %Y', '%Y/%m/%d'):
        try:

```

```

        return datetime.strptime(normalized, fmt).strftime('%Y-%m-%d')
    except ValueError:
        continue

# Handle M/D (no year) – infer year
parts = normalized.split('/')
if len(parts) == 2:
    try:
        month = int(parts[0])
        day = int(parts[1])
        if 1 <= month <= 12 and 1 <= day <= 31:
            now = datetime.now()
            candidate = datetime(now.year, month, day)
            # If the date is in the future, assume previous year
            if candidate > now:
                candidate = datetime(now.year - 1, month, day)
            return candidate.strftime('%Y-%m-%d')
    except (ValueError, TypeError):
        pass

return None

def _get_row_dates(ev)

```

Extract all parseable dates from an evaluation row.

Step by step (top-level body)

1. Assigns fields = ['Date Purchased', 'Date Started', 'Date Ended', 'Date St....
2. Assigns dates = [].
3. for f in fields: iterates.
4. for (key, val) in ev.items(): iterates.
5. return sorted(set(dates)).

Code (copy-paste safe)

```

def _get_row_dates(ev):
    """Extract all parseable dates from an evaluation row."""
    fields = ['Date Purchased', 'Date Started', 'Date Ended',
              'Date Started.1', 'Date Ended.1',
              'Date 1', 'Date 2', 'Date 3', 'Date 4']
    dates = []
    for f in fields:
        d = _parse_date_str(ev.get(f, ''))
        if d:
            dates.append(d)
    # Also check farming progress columns (Prop Day N, Hedge Day N)
    for key, val in ev.items():
        k = str(key)
        if ('Prop Day' in k or 'Hedge Day' in k) and not k.startswith('_'):
            d = _parse_date_str(val)
            if d:
                dates.append(d)
    return sorted(set(dates))

def _estimate_issue_date(ev, issue_check, fallback)

```

Estimate when an issue occurred based on row dates and issue type.

Step by step (top-level body)

1. Assigns `dates = _get_row_dates(ev)`.
2. `if not dates`: branches conditionally.
3. Assigns `dp = _parse_date_str(ev.get('Date Purchased', ''))`.
4. Assigns `ds = _parse_date_str(ev.get('Date Started', ''))`.
5. `if issue_check in ('Status blank', 'Empty Fee', 'Empty Account Size', ...)`: branches conditionally.
6. `if issue_check == 'Missing Date Ended'`: branches conditionally.
7. `if issue_check == 'Empty Account #'`: branches conditionally.
8. `if issue_check == 'Empty Activation Fee'`: branches conditionally.
9. `if issue_check in ('No current day value', 'Negative Hedge Net, no not...', ...)`: branches conditionally.
10. `return dates[-1]`.

Code (copy-paste safe)

```
def _estimate_issue_date(ev, issue_check, fallback):
    """Estimate when an issue occurred based on row dates and issue type."""
    dates = _get_row_dates(ev)
    if not dates:
        return fallback
    dp = _parse_date_str(ev.get('Date Purchased', ''))
    ds = _parse_date_str(ev.get('Date Started', ''))
    # Setup/entry issues → earliest date (purchase/start)
    if issue_check in ('Status blank', 'Empty Fee', 'Empty Account Size', 'Missing Date Started',
                      'Missing Date Purchased'):
        return dp or ds or dates[0]
    # End-of-life issues → latest known date
    if issue_check == 'Missing Date Ended':
        return ds or dates[-1]
    # Active account issues → start date
    if issue_check == 'Empty Account #':
        return ds or dp or dates[0]
    # Funded issues → latest date
    if issue_check == 'Empty Activation Fee':
        return dates[-1]
    # Current/ongoing issues → latest date
    if issue_check in ('No current day value', 'Negative Hedge Net, no note', 'Negative Hedge Net-QA'):
        return dates[-1]
    return dates[-1]
```

```
def run_quality_scan(target_client=None)
```

Automated quality scan: checks every client's data for SOP violations. Returns list of per-client scan results with issues and health scores. If target_client is given, only scan that one client.

Why these Python concepts were used

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Lazy import from `config.hierarchy`.
2. Lazy import from `dashboard.database`.
3. Assigns `all_clients = [target_client] if target_client else hierarchy_get_all_c....`
4. Assigns `results = []`.
5. Assigns `now = datetime.now()`.
6. Assigns `today_weekday = now.weekday()`.
7. Assigns `scan_date_str = now.strftime('%Y-%m-%d')`.
8. **for** `client_name` in `all_clients`: iterates.
9. **return** results.

Code (copy-paste safe)

```
def run_quality_scan(target_client=None):
    """
    Automated quality scan: checks every client's data for SOP violations.
    Returns list of per-client scan results with issues and health scores.
    If target_client is given, only scan that one client.
    """
    from config.hierarchy import get_all_clients as hierarchy_get_all_clients, get_client_profile
    from dashboard.database import get_client_data, get_client_activity

    all_clients = [target_client] if target_client else hierarchy_get_all_clients()
    results = []
    now = datetime.now()
    today_weekday = now.weekday() # 0=Mon, 6=Sun
    scan_date_str = now.strftime('%Y-%m-%d')

    for client_name in all_clients:
        profile = get_client_profile(client_name)
        trader = profile.get('trader', '') if profile else ''
        admin = profile.get('admin', '') if profile else ''
        try:
            data = get_client_data(client_name)
```



```

issues = []

if not data:
    issues.append({'check': 'No data', 'severity': 'critical',
                  'detail': 'Client has no saved data in database',
                  'estimated_date': scan_date_str})
    results.append({
        'client_id': client_name, 'trader': trader, 'admin': admin,
        'total_issues': len(issues), 'issues': issues, 'health_score': 0.0
    })
    continue

# Skip inactive clients – their stats are still calculated but no quality checks
identity = data.get('identity', {})
if isinstance(identity, dict) and identity.get('active_status') == 'inactive':
    results.append({
        'client_id': client_name, 'trader': trader, 'admin': admin,
        'total_issues': 0, 'issues': [], 'health_score': 100.0,
        'skipped': True, 'skip_reason': 'inactive'
    })
    continue

evaluations = data.get('evaluations', [])

# Inject cell notes so checks like "Negative Hedge Net, no note" can see them
try:
    notes = get_client_notes(client_name)
    for i, ev in enumerate(evaluations):
        if i in notes:
            ev['_notes'] = notes[i]
except Exception:
    pass

if not evaluations:
    issues.append({'check': 'No evaluations', 'severity': 'warning',
                  'detail': 'Client has zero evaluation rows',
                  'estimated_date': scan_date_str})

# Activity tracking
activity = get_client_activity(client_name) or {}
last_push = activity.get('last_push_at')
if last_push:
    try:
        push_dt = datetime.fromisoformat(last_push)
        hours_since_push = (now - push_dt).total_seconds() / 3600
        # Flag if >24h since last push on a weekday (Mon-Fri)
        if hours_since_push > 24 and today_weekday < 5:
            issues.append({'check': 'No recent MT5 push', 'severity': 'high',
                          'detail': f'Last push {hours_since_push:.0f}h ago',
                          'estimated_date': push_dt.strftime('%Y-%m-%d')})
    except (ValueError, TypeError):
        pass

# MT5 Profit vs Hedging Results (as displayed in the Stats UI)
#
# IMPORTANT: this check must compare the same numbers users see on the dashboard:
# - MT5 table "Profit" uses current+historical MT5 combined, minus prior activity.
# - Stats "Hedging Results" uses (hedging_results + farming_results + discrepancy).
try:
    stats = data.get('statistics', {}) if isinstance(data, dict) else {}

```

```

hr = stats.get('hedging_review', {}) if isinstance(stats, dict) else {}
cf = stats.get('cashflow_inprogress', {}) if isinstance(stats, dict) else {}
acct = data.get('account', {}) if isinstance(data, dict) else {}
if isinstance(hr, dict) and isinstance(cf, dict) and isinstance(acct, dict):
    def _to_float(v):
        try:
            if v is None:
                return 0.0
            s = str(v).replace('$', '').replace(',', '').strip()
            if s == '':
                return 0.0
            return float(s)
        except (ValueError, TypeError):
            return 0.0

    # Compute the SAME MT5 Profit value shown in the MT5 Accounts Overview table.
    # UI source of truth for "Current MT5" is data.account.*
    mt5_dep = _to_float(acct.get('total_deposits', hr.get('total_deposits')))
    mt5_wd = _to_float(acct.get('total_withdrawals', hr.get('total_withdrawals')))
    mt5_bal = _to_float(acct.get('balance', hr.get('current_balance')))
    hist = hr.get('historical_accounts') or []
    hist_dep = hist_wd = hist_bal = 0.0
    prior_activity = _to_float(hr.get('current_mt5_prior_activity'))
    if isinstance(hist, list):
        for a in hist:
            if not isinstance(a, dict):
                continue
            hist_dep += _to_float(a.get('deposits'))
            hist_wd += _to_float(a.get('withdrawals'))
            hist_bal += _to_float(a.get('final_balance'))
            prior_activity += _to_float(a.get('prior_activity_profit'))
    combined_dep = mt5_dep + hist_dep
    combined_wd = mt5_wd + hist_wd
    combined_bal = mt5_bal + hist_bal
    mt5_profit_display = round(combined_bal - (combined_dep + combined_wd) - prior_activity,
                               2)

    # Compute the SAME Hedging Results number shown in the Stats panel.
    hedge_total_display = round(
        _to_float(cf.get('hedging_results')) + _to_float(cf.get(
            'farming_results')) + _to_float(hr.get('discrepancy')),
        2
    )

    # Only evaluate this check when there's meaningful MT5 context or a recent push.
    has_mt5_context = bool(last_push) or (abs(mt5_dep) > 1e-9) or (abs(mt5_wd) > 1e-9) or
        abs(mt5_bal) > 1e-9) or (abs(hist_dep) > 1e-9) or (abs(hist_wd) > 1e-9) or (abs(hist_bal) > 1e-9)
    if has_mt5_context:
        diff = round(mt5_profit_display - hedge_total_display, 2)
        # Tolerance to avoid noise from rounding / tiny differences.
        if abs(diff) >= 1.0:
            issues.append({
                'check': 'Hedging Results mismatch',
                'severity': 'high',
                'detail': f"Mismatch between sheet hedging results and actual hedging results",
                'estimated_date': scan_date_str
            })
    except Exception:
        pass

    # Check each evaluation row

```

```

total_checks = 0
for idx, ev in enumerate(evaluations):
    row_label = f'Row {idx + 1}'
    # Skip internal/deleted rows
    if ev.get('_deleted'):
        continue

    status_p1 = str(ev.get('Status P1', '') or '').strip().lower()
    status_p2 = str(ev.get('Status', '') or '').strip().lower()

    # Skip rows marked as deleted via status text – superadmin review rows, not real issues
    if 'delete' in status_p1 or 'delete' in status_p2:
        continue
    # Match the dashboard's "Active Only" filter semantics:
    # treat rows as inactive if either phase status indicates a terminal/closed state.
    _inactive_tokens_p1 = ('fail', 'breach', 'closed', 'sl')
    _inactive_tokens_p2 = ('fail', 'breach', 'closed', 'sl', 'complete', 'completed')
    is_active = (not any(t in status_p1 for t in _inactive_tokens_p1)) and (not any(
        t in status_p2 for t in _inactive_tokens_p2))

    prop_firm = str(ev.get('Prop Firm', '') or '').strip()
    acct_size = str(ev.get('Account Size', '') or '').strip()
    has_data = bool(prop_firm or acct_size)
    total_checks += 1

    if not has_data:
        continue

    # Skip defunct prop firms – no point flagging issues on closed firms
    if prop_firm.lower() in ('funding ticks', 'fundingticks'):
        continue

    # Quality scan gating for newly added rows:
    # - If a new row has no hedge values yet, we suppress "early" flags
    #   (ex: empty account number / missing weekday) so it isn't flagged
    #   while the user is still populating fees + initial status.
    # - We still flag missing `Fee` and any Status P1 value that is not
    #   exactly "not started".
    # - If a hedge value exists, we allow the normal flagging flow.
    # Only treat "new row" gating/flags as relevant for active rows.
    # Inactive rows (failed/closed/completed) should never be flagged as "new".
    is_new_row = bool(ev.get('_row_added_at')) and is_active
    fee_raw = str(ev.get('Fee', '') or '').strip()
    acct_num_local = str(ev.get('Account #', '') or '').strip()
    acct_num2_local = str(ev.get('Account #.1', '') or '').strip()
    has_account_num_local = bool(acct_num_local or acct_num2_local)

    def _parse_nonzero(v):
        try:
            s = str(v).replace('$', '').replace(',', '').strip()
            if s in ('', '-', None):
                return 0.0
            return float(s)
        except (ValueError, TypeError):
            return 0.0

    _hedge_result_cols = [
        k for k in ev.keys()
        if isinstance(k, str) and k.startswith('Hedge Result') and not k.startswith('_')
    ]

```

```

# Funded-phase hedge result columns use a ".1" suffix in the sheet export
# (e.g. "Hedge Result 1.1"). Keep these separate so we can reason about
# "Funded = not started" without being confused by eval-phase hedge values.
_funded_hedge_result_cols = [k for k in _hedge_result_cols if k.endswith('.1')]

fee_num = _parse_nonzero(ev.get('Fee', ''))
fee_present = fee_num > 0.0

has_hedge_value_local = any(
    abs(_parse_nonzero(ev.get(c))) > 1e-9 for c in _hedge_result_cols
)
has_funded_hedge_value_local = any(
    abs(_parse_nonzero(ev.get(c))) > 1e-9 for c in _funded_hedge_result_cols
)

new_row_strict_mode = is_new_row and not has_hedge_value_local

# If the row is explicitly "not started" but hedge values already exist, flag.
#
# Important nuance:
# - If Funded status is "not started", only flag when *funded-phase* hedge
#   cells contain numeric values. If funded never began, those cells remain
#   blank or text-only and should not be flagged.
is_not_started_p1 = ('not started' in status_p1)
is_not_started_p2 = ('not started' in status_p2)
if (is_not_started_p1 and has_hedge_value_local) or (
    is_not_started_p2 and has_funded_hedge_value_local):
    issues.append({
        'check': 'Not Started but hedge values present',
        'severity': 'high',
        'row': idx,
        'detail': f'{row_label}: Status is \"not started\" but hedge results contain non-
        'estimated_date': _estimate_issue_date(ev, 'Not Started but hedge values present
        scan_date_str),
    })

# Detect "double dip" – MFF/TopStep accounts with an activation fee
# These are reset at funded stage so eval-phase fields are intentionally blank
_dd_firms = ('my funded futures', 'mff', 'topstep', 'top step', 'topstepx')
activation_fee = str(ev.get('Activation Fee', '') or '').strip()
is_double_dip = (
    prop_firm.lower() in _dd_firms
    and bool(activation_fee)
)

# Status blank on non-empty row
if not status_p1 and has_data and not is_double_dip:
    issues.append({'check': 'Status blank', 'severity': 'medium', 'row': idx,
        'detail': f'{row_label}: Has data but no Status P1',
        'estimated_date': _estimate_issue_date(ev, 'Status blank', scan_date_s

# Empty Fee
fee_raw = str(ev.get('Fee', '') or '').strip()
fee_num = _parse_nonzero(ev.get('Fee', ''))
# Treat "0", "0.00", etc. as missing fee – challenge fees must be > 0.00
_notes = ev.get('_notes') or {}
_fee_note = ''
if isinstance(_notes, dict):
    for _k, _v in _notes.items():
        if str(_k or '').strip().lower() == 'fee' and str(_v or '').strip():

```

```

        _fee_note = str(_v).strip()
        break
# If the Fee cell has a note, treat it as an explicit override/explanation
# and do not flag "Empty Fee" even when Fee is 0.00.
if (not fee_raw or fee_num <= 0.0) and has_data and not is_double_dip and not _fee_note:
    issues.append({'check': 'Empty Fee', 'severity': 'low', 'row': idx,
                  'detail': f'{row_label}: Fee not filled in',
                  'estimated_date': _estimate_issue_date(ev, 'Empty Fee', scan_date_str)})

# New-row strict rule:
# If it's a brand new row and there's NO hedge value yet, we only
# allow the scan to flag:
# 1) missing/zero Fee (handled above),
# 2) missing Date Purchased, or
# 3) Status P1 not being exactly "not started".
# Everything else (empty account #, missing weekday, etc.) is suppressed
# until hedge values arrive.
if new_row_strict_mode:
    dp_raw = str(ev.get('Date Purchased', '') or '').strip()
    if not dp_raw:
        issues.append({
            'check': 'Missing Date Purchased',
            'severity': 'medium',
            'row': idx,
            'detail': f'{row_label}: New row missing Date Purchased',
            'estimated_date': _estimate_issue_date(ev, 'Missing Date Purchased',
                                                  scan_date_str),
        })
    # Special case: some clients do not hedge.
    # They may put a weekday into Hedge Result 1 to indicate the prop account should be t
    # while Status P1 is "hit tp1/2/3". Treat this as valid and suppress the "not started
    _hr1 = str(ev.get('Hedge Result 1', '') or '').strip().lower()
    _weekday_tokens = ('mon', 'monday', 'tue', 'tues', 'tuesday', 'wed', 'weds', 'wednesd
                      'thu', 'thurs', 'thursday', 'fri', 'friday')
    _has_weekday_marker = any(tok in _hr1 for tok in _weekday_tokens)
    _is_hit_tp = status_p1.startswith('hit tp') or status_p1.replace(' ', '').startswith(
        'http')
    _nonhedge_ok = _is_hit_tp and _has_weekday_marker

    if fee_present and status_p1 and status_p1 != 'not started' and not _nonhedge_ok:
        issues.append({
            'check': 'New row: Status P1 not started',
            'severity': 'medium',
            'row': idx,
            'detail': f'{row_label}: New row fee present but Status P1 is \"{status_p1}\"',
            'estimated_date': _estimate_issue_date(ev, 'New row: Status P1 not started',
                                                  scan_date_str),
        })

# Empty Account Size
if not new_row_strict_mode and not acct_size and prop_firm and not is_double_dip:
    issues.append({'check': 'Empty Account Size', 'severity': 'low', 'row': idx,
                  'detail': f'{row_label}: Account Size blank',
                  'estimated_date': _estimate_issue_date(ev, 'Empty Account Size',
                                                          scan_date_str)})

# Empty Account #
acct_num = str(ev.get('Account #', '') or '').strip()
acct_num2 = str(ev.get('Account #.1', '') or '').strip()
if not new_row_strict_mode and is_active and not acct_num and not acct_num2:

```

```

# Treat as "new/uninitialized" and suppress the flag when:
# - Status is "not started", and
# - there are no numeric hedge results yet (blank or text markers like weekdays).
if status_p1 == 'not started' and not has_hedge_value_local:
    pass
# For double dips, only the funded-phase account # matters
elif not is_double_dip or not acct_num2:
    issues.append({'check': 'Empty Account #', 'severity': 'medium', 'row': idx,
                  'detail': f'{row_label}: Active but no account number',
                  'estimated_date': _estimate_issue_date(ev, 'Empty Account #',
                  scan_date_str)})

# Empty Activation Fee on funded rows
activation = str(ev.get('Activation Fee', '') or '').strip()
if not new_row_strict_mode and status_p2 in ('funded', 'live', 'payout') and not activation:
    issues.append({'check': 'Empty Activation Fee', 'severity': 'medium', 'row': idx,
                  'detail': f'{row_label}: Funded but no activation fee',
                  'estimated_date': _estimate_issue_date(ev, 'Empty Activation Fee',
                  scan_date_str)})

# Alpha Futures always charges an activation fee when an account
# actually starts trading the funded account – so only flag a
# missing Activation Fee when the row has BOTH reached funded
# stage AND logged at least one non-zero hedge result in the
# funded phase. Accounts that got a funded account # but never
# traded (or were abandoned before any HR) are excluded so we
# don't drown the dashboard in false positives.
if (not new_row_strict_mode) and prop_firm.lower().replace(' ', '') in ('alphafutures',
    ) and not activation:
    _funded_hr_cols = (
        'Hedge Result 1.1', 'Hedge Result 2.1', 'Hedge Result 3.1',
        'Hedge Result 4.1', 'Hedge Result 5.1',
        'Hedge Result 6', 'Hedge Result 7',
    )
    _has_funded_hr = any(
        re.search(r'[1-9]', str(ev.get(_c, '') or ''))
        for _c in _funded_hr_cols
    )
    _funded_marker = bool(
        status_p1 == 'pass'
        or acct_num2
        or str(ev.get('Date Started.1', '') or '').strip()
        or str(ev.get('Date Ended.1', '') or '').strip()
        or status_p2
    )
    if _funded_marker and _has_funded_hr:
        issues.append({'check': 'Alpha Futures: missing Activation Fee', 'severity': 'high',
                      'row': idx,
                      'detail': f'{row_label}: Alpha Futures funded account has no Activation Fee',
                      'estimated_date': _estimate_issue_date(ev,
                      'Alpha Futures: missing Activation Fee', scan_date_str)})

# Active account: at least one hedge result/day column must contain a weekday name (Mon-Fri)
_inactive_p1 = any(k in status_p1 for k in ('fail', 'breach', 'delete', 'closed', 'sl'))
_inactive_p2 = any(k in status_p2 for k in ('fail', 'breach', 'delete', 'closed', 'sl',
    'complete'))
# Only inspect hedge-specific columns for weekday detection (avoid false positives from other columns)
_day_columns = [k for k in ev.keys() if k.startswith('Hedge Result') or k.startswith('Hedge Day') or k.startswith('Prop Day') or k.startswith('Prop Progress')]
if (

```

```

not new_row_strict_mode or has_account_num_local) and not _inactive_p1 and not _inactive_p2
weekdays = {'monday', 'tuesday', 'wednesday', 'thursday', 'friday',
             'mon', 'tue', 'wed', 'thu', 'fri',
             'tues', 'weds', 'thurs'}
# A day cell counts as valid if it either contains a weekday
# name OR has a note attached (note is treated as an explicit
# explanation / override for that cell).
_cell_notes = ev.get('_notes') or {}
if not isinstance(_cell_notes, dict):
    _cell_notes = {}
has_weekday = False
for col in _day_columns:
    s = str(ev.get(col) or '').strip().lower()
    if any(wd in s for wd in weekdays):
        has_weekday = True
        break
    note_val = _cell_notes.get(col)
    if note_val and str(note_val).strip():
        has_weekday = True
        break
if not has_weekday:
    issues.append({'check': 'No current day value', 'severity': 'medium', 'row': idx,
                  'detail': f'{row_label}: Active account has no cell with a weekday',
                  'estimated_date': _estimate_issue_date(ev, 'No current day value',
                  scan_date_str)})

# Negative Hedge Net without note
def _parse_num(v):
    try: return float(str(v).replace('$', '').replace(',', '').strip())
    except (ValueError, TypeError): return None

hedge_net = _parse_num(ev.get('Hedge Net', ''))
if not new_row_strict_mode and hedge_net is not None and hedge_net < 0:
    dp_str = _parse_date_str(ev.get('Date Purchased', '') or '')
    is_post_cutoff = bool(dp_str and dp_str >= '2026-04-29')
    if is_post_cutoff:
        # QA-gated negative hedge net: notes do NOT clear this. Only super_admin can resolve
        try:
            from dashboard.database import is_qa_resolved
            if not is_qa_resolved('Negative Hedge Net-QA', client_name, idx):
                issues.append({
                    'check': 'Negative Hedge Net-QA',
                    'severity': 'high',
                    'row': idx,
                    'detail': f'{row_label}: Hedge Net=${hedge_net:.2f} (requires QA resolution)',
                    'estimated_date': _estimate_issue_date(ev, 'Negative Hedge Net-QA',
                    scan_date_str)
                })
        except Exception:
            issues.append({
                'check': 'Negative Hedge Net-QA',
                'severity': 'high',
                'row': idx,
                'detail': f'{row_label}: Hedge Net=${hedge_net:.2f} (requires QA resolution)',
                'estimated_date': _estimate_issue_date(ev, 'Negative Hedge Net-QA',
                scan_date_str)
            })
    else:
        # Legacy behavior (pre-cutoff): notes clear the issue.
        cell_notes = ev.get('_notes', {}) or {}

```

```

        has_any_note = isinstance(cell_notes, dict) and any(v for v in cell_notes.values()
        ) if v and str(v).strip()
        notes_col = str(ev.get('Notes', '') or '').strip()
        has_note = has_any_note or bool(notes_col)
        if not has_note:
            issues.append({'check': 'Negative Hedge Net, no note', 'severity': 'high',
                'row': idx,
                'detail': f'{row_label}: Hedge Net=${hedge_net:.2f} with no ex
                'estimated_date': _estimate_issue_date(ev,
                'Negative Hedge Net, no note', scan_date_str)})

# Comma used as a decimal separator in Hedge Result / Hedge Day cells.
# We only care about European-style "1000,67" or "1,000,00" where
# a comma stands in for the decimal point – those silently break
# currency parsing (" ,67" gets stripped as a thousands separator
# and the .67 is lost). Normal US thousand separators like
# "1,000.67" are LEGIT and must not be flagged, so a cell that
# contains a '.' is always accepted.
_comma_cols = []
for _col in ev.keys():
    if not isinstance(_col, str):
        continue
    if not (_col.startswith('Hedge Result') or _col.startswith('Hedge Day')):
        continue
    _val = str(ev.get(_col) or '').strip()
    if not _val or ',' not in _val:
        continue
    _probe = _val.replace('$', '').strip()
    if '.' in _probe:
        # '.' present → commas are thousand separators (US style). Legit.
        continue
    # Flag only when the cell ends with exactly ",NN" (two digits) –
    # i.e. the comma is acting as the decimal point.
    if re.search(r'\d{2}\s*$', _probe):
        _comma_cols.append(f'{_col}="{_val}"')
if _comma_cols:
    _preview = '; '.join(_comma_cols[:3])
    _suffix = ' if len(_comma_cols) <= 3 else f' (+{len(_comma_cols) - 3} more)'
    if not new_row_strict_mode:
        issues.append({'check': 'Comma in hedge value', 'severity': 'low', 'row': idx,
            'detail': f'{row_label}: {_preview}{_suffix}',
            'estimated_date': _estimate_issue_date(ev, 'Comma in hedge value',
            scan_date_str)})

# ■■ Hedge account or Prop Firm account: at least one must be filled ■■
# If the client has evaluations with data, they need either hedge account
# credentials OR prop firm account credentials entered.
hedge_accounts = data.get('hedge_accounts') or []
prop_accounts = data.get('prop_accounts') or []
_hedge_filled = any(
    str(hacc.get('login', '') or '').strip() or str(hacc.get('password', '') or '').strip()
    for hacc in hedge_accounts
    if isinstance(hacc, dict)
)
_prop_filled = any(
    str(pa.get('login', '') or '').strip() or str(pa.get('password', '') or '').strip()
    for pa in prop_accounts
    if isinstance(pa, dict)
)
if total_checks > 0 and not _hedge_filled and not _prop_filled:

```



```

        issues.append({
            'check': 'Hedge account or Prop Firm missing',
            'severity': 'high',
            'tab': 'hedge',
            'detail': 'No hedge account credentials found – fill in Hedge Accounts tab',
            'estimated_date': scan_date_str,
        })
        issues.append({
            'check': 'Hedge account or Prop Firm missing',
            'severity': 'high',
            'tab': 'prop',
            'detail': 'No prop firm credentials found – fill in Prop Firm Accounts tab',
            'estimated_date': scan_date_str,
        })

# Calculate health score (100 - deductions)
severity_weight = {'critical': 20, 'high': 10, 'medium': 5, 'low': 2, 'warning': 3, 'info': 0}
deduction = sum(severity_weight.get(i.get('severity', 'low'), 2) for i in issues)
health_score = max(0.0, 100.0 - deduction)

results.append({
    'client_id': client_name,
    'trader': trader,
    'admin': admin,
    'total_issues': len(issues),
    'issues': issues,
    'health_score': round(health_score, 1)
})
except Exception as exc:
    import traceback
    traceback.print_exc()
    results.append({
        'client_id': client_name, 'trader': trader, 'admin': admin,
        'total_issues': 1, 'issues': [{'check': 'Scan error', 'severity': 'critical',
            'detail': f'Error scanning client: {str(exc)}', 'estimated_date': scan_date_str}],
        'health_score': 0.0
    })

return results

@app.route('/api/quality/discrepancies', methods=['GET'])
@require_role('super_admin')
def api_quality_discrepancies()

```

Return all clients sorted by absolute discrepancy (highest first).

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_role** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.
- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Lazy import from dashboard.database.
2. **with** get_connection(): enters a context manager.
3. Assigns results = [].
4. **for** row in rows: iterates.
5. Calls results.sort(...) for its side effect.
6. **return** jsonify({'status': 'success', 'clients': results}).

Code (copy-paste safe)

```
def api_quality_discrepancies():
    """Return all clients sorted by absolute discrepancy (highest first)."""
    from dashboard.database import get_connection
    with get_connection() as conn:
        cursor = conn.cursor()
        # Quote "identity" – it is a reserved keyword in PostgreSQL 10+
        cursor.execute('SELECT client_id, "identity", account, statistics FROM clients_data')
        rows = cursor.fetchall()

    results = []
    for row in rows:
        cid = row['client_id']
        try:
            raw_id = row['identity']
            identity = (json.loads(raw_id) if isinstance(raw_id, str) else raw_id) or {}
        except Exception:
            identity = {}
        try:
            raw_acct = row['account']
            acct = (json.loads(raw_acct) if isinstance(raw_acct, str) else raw_acct) or {}
        except Exception:
            acct = {}
        try:
            raw_st = row['statistics']
            stats = (json.loads(raw_st) if isinstance(raw_st, str) else raw_st) or {}
        except Exception:
            stats = {}
        hr = stats.get('hedging_review', {}) or {}
        cashflow = stats.get('cashflow_inprogress', {}) or {}
```

```

# Dashboard JS reads MT5 data from data.account (the account column):
# const mt5Dep = parseFloat(mt5Acc.total_deposits) || 0;
# const mt5With = parseFloat(mt5Acc.total_withdrawals) || 0;
# const mt5Bal = parseFloat(mt5Acc.balance) || 0;
mt5_dep = float(acct.get('total_deposits') or 0)
mt5_with = float(acct.get('total_withdrawals') or 0)
mt5_bal = float(acct.get('balance') or 0)

# Historical accounts from hedging_review
hist_accts = hr.get('historical_accounts') or []
hist_dep = sum(float(a.get('deposits') or 0) for a in hist_accts)
hist_with = sum(float(a.get('withdrawals') or 0) for a in hist_accts)
hist_bal = sum(float(a.get('final_balance') or 0) for a in hist_accts)

combined_dep = mt5_dep + hist_dep
combined_with = mt5_with + hist_with
combined_bal = mt5_bal + hist_bal

# Prior activity
total_prior = float(hr.get('current_mt5_prior_activity') or 0)
for a in hist_accts:
    total_prior += float(a.get('prior_activity_profit') or 0)

# Skip clients with no MT5 data (same guard as JS: mt5Dep !== 0 || mt5Bal !== 0)
if mt5_dep == 0 and mt5_bal == 0:
    continue

# liveActualHedging = combinedBal - (combinedDep + combinedWith) - totalPriorActivity
actual = combined_bal - (combined_dep + combined_with) - total_prior

# sheet = cashflow_inprogress.hedging_results + farming_results
sheet = float(cashflow.get('hedging_results') or 0) + float(cashflow.get('farming_results') or 0)
disc = actual - sheet

name = identity.get('name') or identity.get('display_name') or identity.get('client') or cid
if disc == 0 and actual == 0 and sheet == 0:
    continue
try:
    results.append({
        'client_id': cid,
        'name': name,
        'discrepancy': round(float(disc), 2),
        'actual': round(float(actual), 2),
        'sheet': round(float(sheet), 2),
    })
except (TypeError, ValueError):
    continue

results.sort(key=lambda r: abs(r['discrepancy']), reverse=True)
return jsonify({'status': 'success', 'clients': results})

@app.route('/api/quality/deleted_rows', methods=['GET'])
@require_role('super_admin')
def api_quality_deleted_rows()

```

Return all evaluation rows across all clients where Status P1 == 'Deleted'.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_role** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Lazy import from config.hierarchy.
2. Lazy import from dashboard.database.
3. Assigns `all_clients = _get_all_clients()`.
4. Assigns `rows = []`.
5. **for** `client_name` in `all_clients`: iterates.
6. **return** `jsonify({'status': 'success', 'rows': rows, 'total': len(rows)})`.

Code (copy-paste safe)

```
def api_quality_deleted_rows():
    """Return all evaluation rows across all clients where Status P1 == 'Deleted'."""
    from config.hierarchy import get_all_clients as _get_all_clients, get_client_profile
    from dashboard.database import get_client_data

    all_clients = _get_all_clients()
    rows = []
    for client_name in all_clients:
        try:
            data = get_client_data(client_name)
            if not data:
                continue
            profile = get_client_profile(client_name)
            trader = profile.get('trader', '') if profile else ''
            evaluations = data.get('evaluations', [])
            for idx, ev in enumerate(evaluations):
                status_p1 = str(ev.get('Status P1', '') or '').strip()
                if status_p1.lower() == 'deleted':
                    rows.append({
                        'client': client_name,
                        'trader': trader,
                        'row': idx,
                        'account': ev.get('Account #') or ev.get('Account #.1') or '',
                        'prop_firm': ev.get('Prop Firm', ''),
                        'date_started': ev.get('Date Started') or ev.get('Date Purchased') or ''
                    })
        except Exception as e:
            logging.warning(f'deleted_rows scan error for {client_name}: {e}')
            continue
```

```

        return jsonify({'status': 'success', 'rows': rows, 'total': len(rows)})

@app.route('/api/quality/negative_hedge_net_qa', methods=['GET'])
@require_role('super_admin', 'bef_admin')
def api_quality_negative_hedge_net_qa()

```

List unresolved Negative Hedge Net-QA issues across the portfolio.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_role** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.

Step by step (top-level body)

1. Lazy import from dashboard.database.
2. Assigns date = request.args.get('date') or datetime.now().strftime('%Y-%m-...')
3. Assigns scan = get_quality_scan_results(date) or [].
4. Assigns resolved = get_qa_resolved_set('Negative Hedge Net-QA').
5. Assigns items = [].
6. **for** r in scan: iterates.
7. Calls items.sort(...) for its side effect.
8. **return** jsonify({'status': 'success', 'date': date, 'items': items, 'total'...

Code (copy-paste safe)

```

def api_quality_negative_hedge_net_qa():
    """List unresolved Negative Hedge Net-QA issues across the portfolio."""
    from dashboard.database import get_quality_scan_results, get_qa_resolved_set
    date = request.args.get('date') or datetime.now().strftime('%Y-%m-%d')
    scan = get_quality_scan_results(date) or []
    resolved = get_qa_resolved_set('Negative Hedge Net-QA')

    items = []
    for r in scan:
        cid = r.get('client_id')
        trader = r.get('trader', '')
        admin = r.get('admin', '')
        for iss in (r.get('issues') or []):
            if iss.get('check') != 'Negative Hedge Net-QA':
                continue
            row = iss.get('row')
            if row is None:
                continue
            try:

```

```

        row_i = int(row)
    except Exception:
        continue
    if (cid, row_i) in resolved:
        continue
    items.append({
        'client_id': cid,
        'trader': trader,
        'admin': admin,
        'row': row_i,
        'detail': iss.get('detail', ''),
        'estimated_date': iss.get('estimated_date', ''),
        'severity': iss.get('severity', 'high'),
    })

items.sort(key=lambda x: (str(x.get('estimated_date') or ''), str(x.get('client_id') or '')),
           reverse=True)
return jsonify({'status': 'success', 'date': date, 'items': items, 'total': len(items)})

@app.route('/api/quality/qa_resolve', methods=['POST'])
@require_role('super_admin', 'bef_admin')
def api_quality_qa_resolve()

```

Resolve a QA-gated issue (super admin only).

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_role** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Lazy import from dashboard.database.
2. Assigns data = request.get_json(force=True) or {}.
3. Assigns check_name = (data.get('check') or "").strip().
4. Assigns client_id = (data.get('client_id') or "").strip().
5. Assigns row = data.get('row').
6. Assigns notes = (data.get('notes') or "").strip().
7. **if** not check_name or not client_id or row is None: branches conditionally.
8. **try** block with 1 **except** clause.
9. Assigns user = request.session_user.get('user_identifier', "").
10. Calls mark_qa_resolved(...) for its side effect.

11. **try** block with 1 **except** clause.

12. **return** jsonify({'status': 'success'}).

Code (copy-paste safe)

```
def api_quality_qa_resolve():
    """Resolve a QA-gated issue (super admin only)."""
    from dashboard.database import mark_qa_resolved
    data = request.get_json(force=True) or {}
    check_name = (data.get('check') or '').strip()
    client_id = (data.get('client_id') or '').strip()
    row = data.get('row')
    notes = (data.get('notes') or '').strip()
    if not check_name or not client_id or row is None:
        return jsonify({'status': 'error', 'message': 'check, client_id, row required'}), 400
    try:
        row_i = int(row)
    except Exception:
        return jsonify({'status': 'error', 'message': 'row must be an integer'}), 400
    user = request.session_user.get('user_identifier', '')
    mark_qa_resolved(check_name, client_id, row_i, user, notes=notes)
    try:
        log_action('QA_RESOLVE', request.session_user.get('user_type'), user, get_remote_address(),
                  f'{check_name} resolved for {client_id} row {row_i}')
    except Exception:
        pass
    return jsonify({'status': 'success'})

@app.route('/api/quality/scan', methods=['POST'])
@require_role('super_admin')
def api_run_quality_scan()
```

Run quality scan on all clients. Super admin only.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_role** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with `append`, and clearer about intent: this is a transform, not a side-effecting loop.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to `sum`, `any`, `all`, or `max` where the intermediate list would be wasted.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **try** block with 1 **except** clause.
2. Assigns `scan_date = datetime.now().strftime('%Y-%m-%d')`.
3. Assigns `save_warning = None`.
4. **try** block with 1 **except** clause.
5. Assigns `severity_weight = {'critical': 20, 'high': 10, 'medium': 5, 'low': 2, 'warn....`
6. **for** `r` in `results`: iterates.
7. Assigns `total_issues = sum((r['total_issues'] for r in results))`.
8. Assigns `clients_with_issues = sum((1 for r in results if r['total_issues'] > 0))`.
9. Assigns `avg_health = sum((r['health_score'] for r in results)) / len(results) ....`
10. **try** block with 1 **except** clause.
11. Assigns `resp = {'status': 'success', 'scan_date': scan_date, 'total_clie....`
12. **if** `save_warning`: branches conditionally.
13. **return** `jsonify(resp)`.

Code (copy-paste safe)

```
def api_run_quality_scan():
    """Run quality scan on all clients. Super admin only."""
    # Step 1: run the scan (reads client data only)
    try:
        results = run_quality_scan()
    except Exception as e:
        import traceback
        traceback.print_exc()
        return jsonify({'status': 'error', 'message': f'Scan failed: {str(e)}'}), 500

    scan_date = datetime.now().strftime('%Y-%m-%d')

    # Step 2: persist results – non-fatal if the DB is in a bad state
    save_warning = None
    try:
        from dashboard.database import save_quality_scan_results
        save_quality_scan_results(scan_date, results)
    except Exception as e:
        save_warning = str(e)
        logging.warning(f'Quality scan results could not be saved: {e}')

    # Compute display stats after filtering out infrastructure scan errors
    severity_weight = {'critical': 20, 'high': 10, 'medium': 5, 'low': 2, 'warning': 3, 'info': 0}
    for r in results:
        r['issues'] = [i for i in r.get('issues', []) if i.get('check') != 'Scan error']
        r['total_issues'] = len(r['issues'])
        deduction = sum(severity_weight.get(i.get('severity', 'low'), 2) for i in r['issues'])
        r['health_score'] = max(0.0, round(100.0 - deduction, 1))

    total_issues = sum(r['total_issues'] for r in results)
    clients_with_issues = sum(1 for r in results if r['total_issues'] > 0)
    avg_health = sum(r['health_score'] for r in results) / len(results) if results else 0

    try:
```



```

        log_action('QUALITY_SCAN', 'super_admin', request.session_user.get('user_identifier'),
                   get_remote_address(), f'Scanned {len(results)} clients, {total_issues} total issues')
    except Exception:
        pass

    resp = {
        'status': 'success',
        'scan_date': scan_date,
        'total_clients': len(results),
        'clients_with_issues': clients_with_issues,
        'total_issues': total_issues,
        'avg_health_score': round(avg_health, 1),
        'results': results
    }
    if save_warning:
        resp['save_warning'] = f'Results not persisted ({save_warning}). Run DB Repair to fix.'
    return jsonify(resp)

```

```

@app.route('/api/quality/client/<client_id>', methods=['GET'])
@require_role('admin', 'trader', 'super_admin')
def api_quality_client(client_id)

```

Get quality issues for a single client. Loads saved results; ?rescan=1 triggers a live re-scan.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_role** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `user_type = request.session_user.get('user_type')`.
2. Assigns `user_identifier = request.session_user.get('user_identifier')`.
3. **if** `not can_access_client(user_type, user_identifier, client_id)`: branches conditionally.
4. Assigns `empty = {'client_id': client_id, 'issues': [], 'health_score': 10....}`
5. Assigns `_hidden = {'Scan error'}`.
6. **if** `request.args.get('rescan') == '1'`: branches conditionally.
7. **try** block with 1 **except** clause.

8. if user_type == 'super_admin': branches conditionally.

9. return jsonify({'status': 'success', 'data': empty}).

Code (copy-paste safe)

```
def api_quality_client(client_id):
    """Get quality issues for a single client. Loads saved results; ?rescan=1 triggers a live re-scan."""
    user_type = request.session_user.get('user_type')
    user_identifier = request.session_user.get('user_identifier')
    if not can_access_client(user_type, user_identifier, client_id):
        return jsonify({"status": "error", "message": "Access denied"}), 403
    empty = {"client_id": client_id, "issues": [], "health_score": 100.0}
    _hidden = {'Scan error'}

    # If rescan=1, run a live scan for this client and update the stored results
    if request.args.get('rescan') == '1':
        try:
            results = run_quality_scan(target_client=client_id)
            if results:
                r = results[0]
                # Persist the updated result into today's scan row for this client
                try:
                    from dashboard.database import get_connection
                    scan_date = datetime.now().strftime('%Y-%m-%d')
                    with get_connection() as conn:
                        cursor = conn.cursor()
                        cursor.execute(
                            'DELETE FROM quality_scan_results WHERE scan_date = ? AND client_id = ?',
                            (scan_date, client_id))
                        cursor.execute(
                            '''INSERT INTO quality_scan_results
                                (scan_date, client_id, trader, admin, total_issues, issues,
                                health_score)
                                VALUES (?, ?, ?, ?, ?, ?, ?)''',
                            (scan_date, r['client_id'], r.get('trader'), r.get('admin'),
                             r['total_issues'], json.dumps(r['issues']), r['health_score']))
                        conn.commit()
                except Exception:
                    pass # Non-fatal – still return live results
                issues = r.get('issues', [])
                filtered = [i for i in issues if i.get('check') not in _hidden]
                r = dict(r, issues=filtered, total_issues=len(filtered))
                if not filtered:
                    r['health_score'] = 100.0
                return jsonify({"status": "success", "data": r})
            except Exception as e:
                return jsonify({"status": "error", "message": f"Scan failed: {str(e)}"}), 500

        # Try loading from saved scan results first (faster, no DB corruption risk)
        try:
            from dashboard.database import get_quality_scan_results
            saved = get_quality_scan_results() # latest scan
            if saved:
                for r in saved:
                    if r.get('client_id') == client_id:
                        issues = r.get('issues', [])
                        filtered = [i for i in issues if i.get('check') not in _hidden]
                        r = dict(r, issues=filtered, total_issues=len(filtered))
                        if not filtered:
                            r['health_score'] = 100.0
                        return jsonify({"status": "success", "data": r})
```

```

except Exception:
    pass
# No saved results — only super admins should trigger a live scan
if user_type == 'super_admin':
    try:
        results = run_quality_scan(target_client=client_id)
        if results:
            return jsonify({"status": "success", "data": results[0]})
    except Exception as e:
        return jsonify({"status": "error", "message": f"Scan failed: {str(e)}"}), 500
return jsonify({"status": "success", "data": empty})

@app.route('/api/quality/results')
@require_role('super_admin', 'bef_admin')
def api_quality_results():

```

Get quality scan results. Supports ?date=, ?start=&end= for ranges. Super admin only.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_role** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def api_quality_results():
    """Get quality scan results. Supports ?date=, ?start=&end= for ranges. Super admin only."""
    try:
        from dashboard.database import get_quality_scan_results, get_weekly_scan_results
        scan_date = request.args.get('date')
        start_date = request.args.get('start')
        end_date = request.args.get('end')

        if start_date and end_date:
            # Date range query

```

```

try:
    s = datetime.strptime(start_date, '%Y-%m-%d')
    e = datetime.strptime(end_date, '%Y-%m-%d')
    days = (e - s).days + 1
    results = get_weekly_scan_results(end_date, days)
except ValueError:
    return jsonify({'status': 'error', 'message': 'Invalid date format. Use YYYY-MM-DD'}), 400
else:
    results = get_quality_scan_results(scan_date)
if not results:
    return jsonify({'status': 'success', 'results': [], 'total_clients': 0,
                    'clients_with_issues': 0, 'total_issues': 0, 'avg_health_score': 0,
                    'scan_dates': [],
                    'message': 'No scan results for this date range.'})

# Collect unique scan dates
scan_dates = sorted(set(r['scan_date'] for r in results))

# Filter out scan errors and recalculate health scores BEFORE deduplication
severity_weight = {'critical': 20, 'high': 10, 'medium': 5, 'low': 2, 'warning': 3, 'info': 0}
for r in results:
    r['issues'] = [i for i in r.get('issues', []) if i.get('check') != 'Scan error']
    r['total_issues'] = len(r['issues'])
    deduction = sum(severity_weight.get(i.get('severity', 'low'), 2) for i in r['issues'])
    r['health_score'] = max(0.0, round(100.0 - deduction, 1))

# For multi-day ranges, deduplicate: keep only the LATEST scan per client
client_latest = {}
for r in results:
    cid = r['client_id']
    if cid not in client_latest or r['scan_date'] > client_latest[cid]['scan_date']:
        client_latest[cid] = r
deduped = list(client_latest.values())

total_issues = sum(r['total_issues'] for r in deduped)
clients_with_issues = sum(1 for r in deduped if r['total_issues'] > 0)
avg_health = sum(r['health_score'] for r in deduped) / len(deduped) if deduped else 0

return jsonify({
    'status': 'success',
    'scan_date': scan_dates[-1] if scan_dates else None,
    'scan_dates': scan_dates,
    'total_clients': len(deduped),
    'clients_with_issues': clients_with_issues,
    'total_issues': total_issues,
    'avg_health_score': round(avg_health, 1),
    'results': results # Full results (all days) for the issues table
})
except Exception as e:
    import traceback
    traceback.print_exc()
    return jsonify({'status': 'error', 'message': f'Failed to load results: {str(e)}'}), 500

```

```
def compute_admin_tracker_payload(admin_name: str, date: str)
```

Compute admin-tracker payload (shared by UI + Slack summaries).

Why these Python concepts were used

- **type-annotated parameters** — Parameters carry type hints (e.g., `admin_name: str`, `date: str`). Hints are not enforced at runtime — they document intent for static checkers (mypy/pyright), IDE auto-complete, and human readers. They are the fastest way to make a public function self-explanatory.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with `append`, and clearer about intent: this is a transform, not a side-effecting loop.
- **dict comprehension** — Builds a dict in one expression (`{k: v for k, v in items}`) — same intent as a list comprehension but for mappings.
- **set comprehension** — Builds a set in one expression — usually to deduplicate the result of a transform.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to `sum`, `any`, `all`, or `max` where the intermediate list would be wasted.
- **lambda** — Uses a single-expression anonymous function — typically as the `key=` for a `sort`, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Lazy import from `config.hierarchy`.
2. Lazy import from `dashboard.database`.
3. Lazy import from `dashboard.notes_service`.
4. Imports `json` (lazy import inside the function).
5. Assigns `excluded_traders = set(_json.loads(get_setting('summary_tracker_excluded_tra...`
6. Assigns `excluded_clients = set(_json.loads(get_setting('summary_tracker_excluded_cli...`
7. Assigns `clients = []`.
8. **for** `client_id` in `hierarchy_get_all_clients()`: iterates.
9. Calls `clients.sort(...)` for its side effect.
10. Assigns `scan_results = get_quality_scan_results()` or `[]`.
11. Assigns `scan_by_client = {r.get('client_id'): r for r in scan_results if r.get('cl...`
12. Assigns `admin_issues = []`.

13. Defines a nested function `_add_issue(...)`.

14. Defines a nested function `_norm_pf(...)`.

15. ... and 18 more statement(s) in the body.

Code (copy-paste safe)

```
def compute_admin_tracker_payload(admin_name: str, date: str):
    """Compute admin-tracker payload (shared by UI + Slack summaries)."""
    from config.hierarchy import get_all_clients as hierarchy_get_all_clients, get_client_profile
    from dashboard.database import (
        get_quality_scan_results,
        get_client_data,
        get_daily_checklists,
        get_summary_status_for_date,
        get_setting,
    )
    from dashboard.notes_service import get_client_notes
    import json as _json

    # Exclusions mirror the Daily Summary Tracker rules
    excluded_traders = set(_json.loads(get_setting('summary_tracker_excluded_traders') or '[]'))
    excluded_clients = set(_json.loads(get_setting('summary_tracker_excluded_clients') or '[]'))

    # Build admin client roster from hierarchy profiles
    clients = []
    for client_id in hierarchy_get_all_clients():
        p = get_client_profile(client_id) or {}
        if str(p.get('admin') or '').strip().lower() != str(admin_name or '').strip().lower():
            continue
        trader = str(p.get('trader') or '').strip()
        category = str(p.get('category') or p.get('profile') or '').strip() or 'PRIVATE'
        active_status = str(p.get('active_status') or 'active').strip().lower()
        clients.append({
            'client_id': client_id,
            'trader': trader,
            'category': category,
            'active_status': active_status,
        })
    clients.sort(key=lambda c: (c.get('active_status') != 'active', (c.get('trader') or ''), c['client_id']))

    # Load latest quality scan issues (used for fee/downtime derivations)
    scan_results = get_quality_scan_results() or []
    scan_by_client = {r.get('client_id'): r for r in scan_results if r.get('client_id')}

    admin_issues = []

    def _add_issue(issue_type, client_id, trader, severity, detail, extra=None):
        rec = {
            'type': issue_type,
            'client_id': client_id,
            'trader': trader or '',
            'severity': severity or 'medium',
            'detail': detail or '',
        }
        if extra and isinstance(extra, dict):
            rec.update(extra)
        admin_issues.append(rec)

    # Helper: normalize prop firm names to a stable key
```

```

def _norm_pf(raw):
    s = str(raw or '').strip().lower().replace(' ', '').replace('_', '')
    if not s:
        return ''
    if 'myfunded' in s or s.startswith('mffu') or 'mffu' in s:
        return 'mffu'
    if s in ('toponefutures', 'topone', 'tpo'):
        return 'toponefutures'
    if s in ('tradeday', 'trade-day'):
        return 'tradeday'
    return s

_inactive_tokens_p1 = ('fail', 'breach', 'closed', 'sl')
_inactive_tokens_p2 = ('fail', 'breach', 'closed', 'sl', 'complete', 'completed')
def _is_row_active(ev):
    sp1 = str(ev.get('Status P1', '')) or '').strip().lower()
    sp2 = str(ev.get('Status', '')) or ev.get('Status Funded', '') or '').strip().lower()
    if 'delete' in sp1 or 'delete' in sp2:
        return False
    return (not any(t in sp1 for t in _inactive_tokens_p1)) and (not any(
        t in sp2 for t in _inactive_tokens_p2))

fee_issue_checks = {'Empty Fee', 'Empty Activation Fee', 'Alpha Futures: missing Activation Fee'}
for c in clients:
    cid = c['client_id']
    trader = c.get('trader') or ''
    active_status = c.get('active_status', 'active')
    if active_status == 'inactive':
        continue
    if cid in excluded_clients or (trader and trader in excluded_traders):
        continue

    r = scan_by_client.get(cid) or {}
    for iss in (r.get('issues') or []):
        if iss.get('check') in fee_issue_checks:
            _add_issue(
                'challenge_fees',
                cid,
                trader,
                iss.get('severity') or 'medium',
                f"{iss.get('check')}: {iss.get('detail') or ''}",
                extra={'row': iss.get('row'), 'estimated_date': iss.get('estimated_date')}
            )
    for iss in (r.get('issues') or []):
        if iss.get('check') == 'Downtime detected':
            _add_issue(
                'downtime',
                cid,
                trader,
                iss.get('severity') or 'high',
                iss.get('detail') or 'Downtime detected',
                extra={'row': iss.get('row'), 'estimated_date': iss.get('estimated_date')}
            )

# Prop firm max-out counts
try:
    cdata = get_client_data(cid) or {}
    identity = cdata.get('identity', {}) if isinstance(cdata, dict) else {}
    if isinstance(identity, dict) and str(identity.get('active_status') or '').lower() == 'inactive':
        continue

```

```

evals = [ev for ev in (cdata.get('evaluations') or []) if isinstance(ev, dict) and not ev.get(
    '_deleted')]

# Inject cell notes so we can suppress "excess accounts" when the extra rows are explained v
try:
    notes_by_row = get_client_notes(cid) or {}
    for _i, _ev in enumerate(evals):
        if _i in notes_by_row:
            _ev['_notes'] = notes_by_row[_i]
except Exception:
    pass

pf_rows = {} # pf_key -> list[{idx, has_note}]
for idx, ev in enumerate(evals):
    if not _is_row_active(ev):
        continue
    pf_key = _norm_pf(ev.get('Prop Firm'))
    if not pf_key:
        continue
    # Excess-account suppression requires a note STRICTLY on the Status P1 cell.
    has_note = False
    cell_notes = ev.get('_notes') or {}
    if isinstance(cell_notes, dict):
        sp1_note = cell_notes.get('Status P1')
        if sp1_note is not None and str(sp1_note).strip():
            has_note = True
    pf_rows.setdefault(pf_key, []).append({'idx': idx, 'has_note': has_note})

for pf_key, rows in sorted(pf_rows.items()):
    count = len(rows)
    expected = 3 if pf_key == 'mffu' else 5
    if count == 0:
        continue
    if count != expected:
        human = 'My Funded Futures' if pf_key == 'mffu' else pf_key
        if count < expected:
            _add_issue(
                'max_out',
                cid,
                trader,
                'medium',
                f"{human}: {expected - count} needed (expected {expected}, has {count})",
                extra={'prop_firm': human, 'expected': expected, 'count': count}
            )
        else:
            excess = count - expected
            noted = sum(1 for r in rows if r.get('has_note'))
            if noted < excess:
                missing_notes = excess - noted
                _add_issue(
                    'max_out',
                    cid,
                    trader,
                    'medium',
                    f"{human}: too many accounts (expected {expected}, has {count}) - {missin
                    extra={'prop_firm': human, 'expected': expected, 'count': count,
                        'excess': excess, 'excess_missing_notes': missing_notes}
                )
except Exception:
    pass

```



```

# Admin summary sign-off (only required after trader submitted)
trader_submissions = get_summary_status_for_date(date) or []
trader_sent_map = {}
for s in trader_submissions:
    cid = s.get('client_id')
    if not cid:
        continue
    if cid not in trader_sent_map:
        trader_sent_map[cid] = {
            'submitted_at': s.get('submitted_at'),
            'user_identifier': s.get('user_identifier'),
        }
trader_submitted_clients = set(trader_sent_map.keys())

admin_checklists = get_daily_checklists(date, admin_name) or []
def _is_admin_signed(checklist_row):
    try:
        if checklist_row.get('checklist_type') != 'admin_daily_summary':
            return False
        items = checklist_row.get('items') or []
        if not isinstance(items, list):
            return False
        for it in items:
            if isinstance(it, dict) and it.get('id') == 'sent_to_client' and bool(it.get('checked')):
                return True
        return False
    except Exception:
        return False

admin_signed_clients = {
    c.get('client_id')
    for c in admin_checklists
    if c.get('client_id') and _is_admin_signed(c)
}

required_clients = sorted([
    c['client_id']
    for c in clients
    if c.get('active_status') == 'active'
    and c['client_id'] in trader_submitted_clients
    and (c['client_id'] not in excluded_clients)
    and ((c.get('trader') or '') not in excluded_traders)
])
pending_clients = sorted([cid for cid in required_clients if cid not in admin_signed_clients])

severity_weight = {'critical': 20, 'high': 10, 'medium': 5, 'low': 2, 'warning': 3, 'info': 0}
deduction = sum(severity_weight.get(i.get('severity', 'low'), 2) for i in admin_issues)
health_score = max(0.0, round(100.0 - deduction, 1))

# Backward/forward compatibility: different UIs may read `issues` or `admin_issues`.
# Keep `issues` as canonical, but include both keys.
return {
    'admin': admin_name,
    'date': date,
    'clients': clients,
    'issues': admin_issues,
    'admin_issues': admin_issues,
    'total_clients': len([c for c in clients if c.get('active_status') == 'active']),
    'total_issues': len(admin_issues),
    'health_score': health_score,
}

```

```

        'summary_signoff': {
            'required_total': len(required_clients),
            'signed_total': sum(1 for cid in required_clients if cid in admin_signed_clients),
            'pending_total': len(pending_clients),
            'pending_clients': pending_clients,
            'signed_clients': sorted([cid for cid in required_clients if cid in admin_signed_clients]),
            'trader_sent': trader_sent_map,
        }
    }

@app.route('/api/admin/tracker')
@require_role('admin', 'super_admin', 'bep_admin', 'kwok_admin')
def api_admin_tracker()

```

Admin tracking dashboard: aggregate admin-owned issues derived from client state and quality scan.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_role** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **dict comprehension** — Builds a dict in one expression (`{k: v for k, v in items}`) — same intent as a list comprehension but for mappings.
- **set comprehension** — Builds a set in one expression — usually to deduplicate the result of a transform.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.
- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. try block with 1 except clause.

Code (copy-paste safe)

```
def api_admin_tracker():
    """Admin tracking dashboard: aggregate admin-owned issues derived from client state and quality scan.
    try:
        from config.hierarchy import get_all_clients as hierarchy_get_all_clients, get_client_profile
        from dashboard.database import (
            get_quality_scan_results,
            get_client_data,
            get_daily_checklists,
            get_summary_status_for_date,
            get_setting,
        )
        import json as _json

    def _compute_admin_tracker_payload(admin_name: str, date: str):
        # Exclusions mirror the Daily Summary Tracker rules
        excluded_traders = set(_json.loads(get_setting('summary_tracker_excluded_traders') or '[]'))
        excluded_clients = set(_json.loads(get_setting('summary_tracker_excluded_clients') or '[]'))

        # Build admin client roster from hierarchy profiles
        clients = []
        for client_id in hierarchy_get_all_clients():
            p = get_client_profile(client_id) or {}
            if str(p.get('admin') or '').strip().lower() != admin_name.lower():
                continue
            trader = str(p.get('trader') or '').strip()
            category = str(p.get('category') or p.get('profile') or '').strip() or 'PRIVATE'
            active_status = str(p.get('active_status') or 'active').strip().lower()
            clients.append({
                'client_id': client_id,
                'trader': trader,
                'category': category,
                'active_status': active_status,
            })
        clients.sort(key=lambda c: (c.get('active_status') != 'active', (c.get('trader') or ''), c[
            'client_id']))

        # Load latest quality scan issues (used for fee/downtime derivations)
        scan_results = get_quality_scan_results() or []
        scan_by_client = {r.get('client_id'): r for r in scan_results if r.get('client_id')}

        admin_issues = []

    def _add_issue(issue_type, client_id, trader, severity, detail, extra=None):
        rec = {
            'type': issue_type,
            'client_id': client_id,
            'trader': trader or '',
            'severity': severity or 'medium',
            'detail': detail or '',
        }
        if extra and isinstance(extra, dict):
            rec.update(extra)
        admin_issues.append(rec)

    # Helper: normalize prop firm names to a stable key
    def _norm_pf(raw):
        s = str(raw or '').strip().lower().replace(' ', '').replace('_', '')
        if not s:
```

```

        return ''
    if 'myfunded' in s or s.startswith('mffu') or 'mffu' in s:
        return 'mffu'
    if s in ('toponefutures', 'topone', 'tpo'):
        return 'toponefutures'
    if s in ('tradeday', 'trade-day'):
        return 'tradeday'
    return s

_inactive_tokens_p1 = ('fail', 'breach', 'closed', 'sl')
_inactive_tokens_p2 = ('fail', 'breach', 'closed', 'sl', 'complete', 'completed')
def _is_row_active(ev):
    sp1 = str(ev.get('Status P1', '') or '').strip().lower()
    sp2 = str(ev.get('Status', '') or ev.get('Status Funded', '') or '').strip().lower()
    if 'delete' in sp1 or 'delete' in sp2:
        return False
    return (not any(t in sp1 for t in _inactive_tokens_p1)) and (not any(
        t in sp2 for t in _inactive_tokens_p2))

fee_issue_checks = {'Empty Fee', 'Empty Activation Fee', 'Alpha Futures: missing Activation Fee'}
for c in clients:
    cid = c['client_id']
    trader = c.get('trader') or ''
    active_status = c.get('active_status', 'active')
    if active_status == 'inactive':
        continue
    if cid in excluded_clients or (trader and trader in excluded_traders):
        continue

    r = scan_by_client.get(cid) or {}
    for iss in (r.get('issues') or []):
        if iss.get('check') in fee_issue_checks:
            _add_issue(
                'challenge_fees',
                cid,
                trader,
                iss.get('severity') or 'medium',
                f"{iss.get('check')}: {iss.get('detail') or ''}",
                extra={'row': iss.get('row'), 'estimated_date': iss.get('estimated_date')}
            )
    for iss in (r.get('issues') or []):
        if iss.get('check') == 'Downtime detected':
            _add_issue(
                'downtime',
                cid,
                trader,
                iss.get('severity') or 'high',
                iss.get('detail') or 'Downtime detected',
                extra={'row': iss.get('row'), 'estimated_date': iss.get('estimated_date')}
            )

# Prop firm max-out counts
try:
    cdata = get_client_data(cid) or {}
    identity = cdata.get('identity', {}) if isinstance(cdata, dict) else {}
    if isinstance(identity, dict) and str(identity.get('active_status') or '').lower() == 'inactive':
        continue
    evals = [ev for ev in (cdata.get('evaluations') or []) if isinstance(ev, dict) and not ev.get('_deleted')]

```

```

# Inject cell notes so we can suppress "excess accounts" when the extra rows are expected
# (Matches how run_quality_scan injects notes.)
try:
    notes_by_row = get_client_notes(cid) or {}
    for _i, _ev in enumerate(evals):
        if _i in notes_by_row:
            _ev['_notes'] = notes_by_row[_i]
except Exception:
    pass

pf_rows = {} # pf_key -> list[{idx, has_note}]
for idx, ev in enumerate(evals):
    if not _is_row_active(ev):
        continue
    pf_key = _norm_pf(ev.get('Prop Firm'))
    if not pf_key:
        continue
    # Excess-account suppression requires a note STRICTLY on the Status P1 cell.
    # Notes on other cells are not accepted for this purpose.
    has_note = False
    cell_notes = ev.get('_notes') or {}
    if isinstance(cell_notes, dict):
        sp1_note = cell_notes.get('Status P1')
        if sp1_note is not None and str(sp1_note).strip():
            has_note = True
    pf_rows.setdefault(pf_key, []).append({'idx': idx, 'has_note': has_note})

for pf_key, rows in sorted(pf_rows.items()):
    count = len(rows)
    expected = 3 if pf_key == 'mffu' else 5
    if count == 0:
        continue
    if count != expected:
        human = 'My Funded Futures' if pf_key == 'mffu' else pf_key
        if count < expected:
            _add_issue(
                'max_out',
                cid,
                trader,
                'medium',
                f"{human}: {expected - count} needed (expected {expected}, has {count}) -",
                extra={'prop_firm': human, 'expected': expected, 'count': count}
            )
        else:
            # Excess accounts are allowed ONLY when the extra rows have notes.
            # If N accounts exceed the required count, we require at least N rows
            # to have a note (cell note or Notes column). Otherwise, flag.
            excess = count - expected
            noted = sum(1 for r in rows if r.get('has_note'))
            if noted < excess:
                missing_notes = excess - noted
                _add_issue(
                    'max_out',
                    cid,
                    trader,
                    'medium',
                    f"{human}: too many accounts (expected {expected}, has {count}) -",
                    extra={'prop_firm': human, 'expected': expected, 'count': count,
                        'excess': excess, 'excess_missing_notes': missing_notes}
                )

```

```

        except Exception:
            pass

# Admin summary sign-off (only required after a summary has been sent "upstream")
# In your workflow: trader sends summary → admin reviews → admin confirms accurate & sent to
trader_submissions = get_summary_status_for_date(date) or []
trader_sent_map = {}
for s in trader_submissions:
    cid = s.get('client_id')
    if not cid:
        continue
    # Keep the most recent submission per client (get_summary_status_for_date is newest-first)
    if cid not in trader_sent_map:
        trader_sent_map[cid] = {
            'submitted_at': s.get('submitted_at'),
            'user_identifier': s.get('user_identifier'),
        }
trader_submitted_clients = set(trader_sent_map.keys())

admin_checklists = get_daily_checklists(date, admin_name) or []
def _is_admin_signed(checklist_row):
    try:
        if checklist_row.get('checklist_type') != 'admin_daily_summary':
            return False
        items = checklist_row.get('items') or []
        if not isinstance(items, list):
            return False
        for it in items:
            if isinstance(it, dict) and it.get('id') == 'sent_to_client' and bool(it.get(
                'checked')):
                return True
        return False
    except Exception:
        return False

admin_signed_clients = {
    c.get('client_id')
    for c in admin_checklists
    if c.get('client_id') and _is_admin_signed(c)
}

required_clients = []
pending_clients = []
for c in clients:
    cid = c['client_id']
    trader = c.get('trader') or ''
    if c.get('active_status') == 'inactive':
        continue
    if cid in excluded_clients or (trader and trader in excluded_traders):
        continue
    if cid not in trader_submitted_clients:
        continue
    required_clients.append(cid)
    if cid not in admin_signed_clients:
        pending_clients.append(cid)
    _add_issue(
        'summary_signoff',
        cid,
        trader,
        'medium',
    )

```

```

        'Daily summary not signed off (confirm sent to client)',
        extra={'date': date}
    )

    sev_rank = {'critical': 0, 'high': 1, 'medium': 2, 'warning': 3, 'low': 4, 'info': 5}
    admin_issues.sort(key=lambda i: (sev_rank.get(i.get('severity', 'medium'), 9), i.get('type',
        i.get('client_id', '')))

    return {
        'admin': admin_name,
        'date': date,
        'clients': clients,
        'admin_issues': admin_issues,
        'summary_signoff': {
            'required_total': len(required_clients),
            'signed_total': sum(1 for cid in required_clients if cid in admin_signed_clients),
            'pending_total': len(pending_clients),
            'pending_clients': pending_clients,
            'signed_clients': sorted([
                cid for cid in required_clients if cid in admin_signed_clients]),
            'trader_sent': trader_sent_map,
        }
    }

    session_user = request.session_user
    if session_user.get('user_type') in ('super_admin', 'bef_admin', 'kwok_admin'):
        admin_name = (request.args.get('admin') or '').strip()
    else:
        admin_name = (session_user.get('user_identifier') or '').strip()
    if not admin_name:
        return jsonify({'status': 'error', 'message': 'Admin name required'}), 400
    date = request.args.get('date', datetime.now().strftime('%Y-%m-%d'))

    payload = compute_admin_tracker_payload(admin_name, date)
    return jsonify({'status': 'success', **payload})
except Exception as e:
    import traceback
    traceback.print_exc()
    return jsonify({'status': 'error', 'message': f'Failed to load admin tracker: {str(e)}'}), 500

@app.route('/api/quality/admin_tracker')
@require_role('super_admin', 'bef_admin')
def api_quality_admin_tracker()

```

Super-admin view of admin tracker issues across admins (quality dashboard source of truth).

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_role** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def api_quality_admin_tracker():
    """Super-admin view of admin tracker issues across admins (quality dashboard source of truth)."""
    try:
        from config.hierarchy import get_all_clients as hierarchy_get_all_clients, get_client_profile
        admins = set()
        for cid in hierarchy_get_all_clients():
            p = get_client_profile(cid) or {}
            a = str(p.get('admin') or '').strip()
            if a:
                admins.add(a)
        admin = (request.args.get('admin') or '').strip()
        date = request.args.get('date', datetime.now().strftime('%Y-%m-%d'))

        # Reuse the existing admin tracker endpoint logic by calling the local view function's helper th
        # We simply invoke /api/admin/tracker-style computation by issuing internal HTTP is overkill; ins
        # is not possible cleanly. So we request per-admin via the public function by recreating minimal
        # Pragmatic: call /api/admin/tracker endpoint from frontend for a single admin; for all admins, l
        # by delegating to the endpoint itself is avoided. Instead, we reuse the DB-backed quality + cli
        # calling the /api/admin/tracker route through WSGI is out of scope.

        # Implementation: make N calls by recomputing via querystring by temporarily setting request args
        # Therefore, we implement a small loop by calling the public endpoint over HTTP is also not allow
        # So: return the list of admins + let the quality dashboard fetch each admin in parallel.
        # This keeps server logic correct and avoids duplicating the computation again.
        result_admins = sorted(admins)
        if admin:
            if admin not in admins:
                return jsonify({'status': 'error', 'message': 'Unknown admin'}), 404
            return jsonify({'status': 'success', 'admins': result_admins, 'admin': admin, 'date': date})
        return jsonify({'status': 'success', 'admins': result_admins, 'date': date})
    except Exception as e:
        import traceback
        traceback.print_exc()
        return jsonify({'status': 'error', 'message': f'Failed to load admin tracker index: {str(e)}'})

@app.route('/api/quality/admin_issues')
@require_role('admin', 'super_admin')
def api_admin_issues():
```

Return latest quality scan results filtered for a specific admin's traders/clients. Admin role only.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_role** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.

- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. try block with 1 **except** clause.

Code (copy-paste safe)

```
def api_admin_issues():
    """Return latest quality scan results filtered for a specific admin's traders/clients. Admin role on"""
    try:
        from dashboard.database import get_quality_scan_results
        session_user = request.session_user
        if session_user.get('user_type') == 'super_admin':
            admin_name = request.args.get('admin', '')
        else:
            admin_name = session_user.get('user_identifier', '')
        if not admin_name:
            return jsonify({'status': 'error', 'message': 'Admin name required'}), 400
        results = get_quality_scan_results() # latest scan
        # Filter for this admin only
        filtered = [r for r in results if (r.get('admin') or '').lower() == admin_name.lower()]
        # Strip scan errors, recalculate health
        severity_weight = {'critical': 20, 'high': 10, 'medium': 5, 'low': 2, 'warning': 3, 'info': 0}
        for r in filtered:
            r['issues'] = [i for i in r.get('issues', []) if i.get('check') != 'Scan error']
            r['total_issues'] = len(r['issues'])
            deduction = sum(severity_weight.get(i.get('severity', 'low'), 2) for i in r['issues'])
            r['health_score'] = max(0.0, round(100.0 - deduction, 1))
        total_issues = sum(r['total_issues'] for r in filtered)
        return jsonify({
            'status': 'success',
            'admin': admin_name,
            'total_clients': len(filtered),
            'clients_with_issues': sum(1 for r in filtered if r['total_issues'] > 0),
            'total_issues': total_issues,
            'results': filtered
        })
    except Exception as e:
        import traceback
        traceback.print_exc()
        return jsonify({'status': 'error', 'message': f'Failed to load admin issues: {str(e)}'}), 500

@app.route('/api/quality/scan_dates')
@require_role('super_admin', 'bef_admin')
def api_quality_scan_dates():
```

Get list of all dates that have scan results.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_role** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def api_quality_scan_dates():
    """Get list of all dates that have scan results."""
    try:
        from dashboard.database import get_connection
        with get_connection() as conn:
            cursor = conn.cursor()
            cursor.execute('SELECT DISTINCT scan_date FROM quality_scan_results ORDER BY scan_date DESC')
            dates = [row['scan_date'] for row in cursor.fetchall()]
            return jsonify({'status': 'success', 'dates': dates})
    except Exception as e:
        return jsonify({'status': 'success', 'dates': []})

@app.route('/api/quality/summary_status')
@require_role('super_admin', 'bef_admin')
def api_summary_status()
```

Get daily summary submission status for all clients, grouped by trader.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_role** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **dict comprehension** — Builds a dict in one expression (`{k: v for k, v in items}`) — same intent as a list comprehension but for mappings.

- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Lazy import from config.hierarchy.
2. Lazy import from dashboard.database.
3. Imports json (lazy import inside the function).
4. Lazy import from datetime.
5. Assigns `_kenyan_tz = timezone(_td(hours=3))`.
6. Assigns `date = request.args.get('date', datetime.now().strftime('%Y-%m-%...'`
7. Assigns `submissions = get_summary_status_for_date(date)`.
8. **for** s in submissions: iterates.
9. Assigns `sent_map = {s['client_id']: s for s in submissions}`.
10. Assigns `all_clients = hierarchy_get_all_clients()`.
11. Assigns `excluded_traders = set(_json.loads(get_setting('summary_tracker_excluded_tra...`
12. Assigns `excluded_clients = set(_json.loads(get_setting('summary_tracker_excluded_cli...`
13. Assigns `traders = {}`.
14. Assigns `tracked_count = 0`.
15. ... and 5 more statement(s) in the body.

Code (copy-paste safe)

```
def api_summary_status():
    """Get daily summary submission status for all clients, grouped by trader."""
    from config.hierarchy import get_all_clients as hierarchy_get_all_clients, get_client_profile
    from dashboard.database import get_summary_status_for_date, get_setting, get_client_data
    import json as _json

    # Use UTC date as default – server runs UTC, so midnight UTC = 3 AM Kenyan.
    # This means the tracker naturally flips ~55 min after the 2:05 AM Slack send.
    from datetime import timezone, timedelta as _td
    _kenyan_tz = timezone(_td(hours=3))
    date = request.args.get('date', datetime.now().strftime('%Y-%m-%d'))
    submissions = get_summary_status_for_date(date)

    # Convert timestamps from UTC to Kenyan time (UTC+3)
    for s in submissions:
        ts = s.get('submitted_at', '')
        if ts:
            try:
                dt = datetime.fromisoformat(ts.replace('Z', '+00:00'))
                if dt.tzinfo is None:
```

```

        dt = dt.replace(tzinfo=timezone.utc)
        s['submitted_at'] = dt.astimezone(_kenyan_tz).isoformat()
    except Exception:
        pass

sent_map = {s['client_id']: s for s in submissions}
all_clients = hierarchy_get_all_clients()

# Load exclusion settings
excluded_traders = set(_json.loads(get_setting('summary_tracker_excluded_traders') or '[]'))
excluded_clients = set(_json.loads(get_setting('summary_tracker_excluded_clients') or '[]'))

traders = {} # trader -> {sent: [...], not_sent: [...]}
tracked_count = 0
for client_name in all_clients:
    profile = get_client_profile(client_name)
    trader = (profile.get('trader', '') if profile else '') or 'Unassigned'

    # Skip excluded traders entirely
    if trader in excluded_traders:
        continue

    # Skip excluded clients
    if client_name in excluded_clients:
        continue

    # Skip inactive clients by default
    try:
        cdata = get_client_data(client_name)
        if cdata and isinstance(cdata.get('identity'), dict):
            if cdata['identity'].get('active_status') == 'inactive':
                continue
    except Exception:
        pass

    tracked_count += 1
    if trader not in traders:
        traders[trader] = {'sent': [], 'not_sent': []}
    if client_name in sent_map:
        s = sent_map[client_name]
        traders[trader]['sent'].append({
            'client_id': client_name,
            'submitted_by': s['submitted_by'],
            'submitted_at': s['submitted_at']
        })
    else:
        traders[trader]['not_sent'].append(client_name)

# Build response
total_sent = sum(len(d['sent']) for d in traders.values())
result = []
for trader, data in sorted(traders.items()):
    result.append({
        'trader': trader,
        'total': len(data['sent']) + len(data['not_sent']),
        'sent_count': len(data['sent']),
        'sent': data['sent'],
        'not_sent': data['not_sent']
    })
return jsonify({
    'status': 'success', 'date': date,

```

```

        'total_clients': tracked_count, 'total_sent': total_sent,
        'total_not_sent': tracked_count - total_sent,
        'excluded_traders': sorted(excluded_traders),
        'excluded_clients': sorted(excluded_clients),
        'traders': result
    })

```

```
def _resolve_trader_for_session(session_user, requested)
```

Resolve which trader a request is asking about and whether the session is allowed. Returns (trader_name, error_response_or_None). If the second element is not None, the caller should return it directly.

Step by step (top-level body)

1. Assigns `user_type = session_user.get('user_type')`.
2. Assigns `user_id = (session_user.get('user_identifier') or '').strip()`.
3. Assigns `requested = (requested or '').strip()`.
4. **if** `user_type == 'trader'`: branches conditionally.
5. Assigns `trader_name = requested or user_id`.
6. **if not** `trader_name`: branches conditionally.
7. **if** `user_type == 'admin'`: branches conditionally.
8. **return** `(trader_name, None)`.

Code (copy-paste safe)

```

def _resolve_trader_for_session(session_user, requested):
    """Resolve which trader a request is asking about and whether the session is allowed.

    Returns (trader_name, error_response_or_None). If the second element is not None,
    the caller should return it directly.
    """
    user_type = session_user.get('user_type')
    user_id = (session_user.get('user_identifier') or '').strip()
    requested = (requested or '').strip()

    if user_type == 'trader':
        # Traders can only query their own data
        if requested and requested != user_id:
            return None, (jsonify({'status': 'error', 'message': 'Access denied'}), 403)
        return user_id, None

    trader_name = requested or user_id
    if not trader_name:
        return None, (jsonify({'status': 'error', 'message': 'Trader required'}), 400)

    if user_type == 'admin':
        admin_data = hierarchy.get('admins', {}).get(user_id, {}) or {}
        if trader_name not in (admin_data.get('traders') or {}):
            return None, (jsonify({'status': 'error', 'message': 'Access denied'}), 403)

    # super_admin / bef_admin: allow any trader
    return trader_name, None

```

```
def _clients_for_trader(trader_name)
```

Return the list of client names managed by the given trader across all admins.

Why these Python concepts were used

- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns names = [].
2. Assigns seen = set().
3. for admin_data in hierarchy.get('admins', {}).values(): iterates.
4. return names.

Code (copy-paste safe)

```
def _clients_for_trader(trader_name):
    """Return the list of client names managed by the given trader across all admins."""
    names = []
    seen = set()
    for admin_data in hierarchy.get('admins', {}).values():
        tdata = (admin_data.get('traders') or {}).get(trader_name)
        if not tdata:
            continue
        for c in tdata.get('clients', []) or []:
            nm = c.get('name') if isinstance(c, dict) else c
            if nm and nm not in seen:
                seen.add(nm)
                names.append(nm)
    return names

@app.route('/api/quality/trader_issues')
@require_role('trader', 'admin', 'super_admin', 'bef_admin', 'kwok_admin')
def api_trader_issues()
```

Latest quality scan issues filtered to a single trader's clients. Traders see only themselves; admins see their own traders; super_admin / bef_admin can pass ?trader=<name>. Clients with no scan record are returned with total_issues=0 so the UI can show "all clear" rows too.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_role** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **set comprehension** — Builds a set in one expression — usually to deduplicate the result of a transform.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.
- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. try block with 1 **except** clause.

Code (copy-paste safe)

```
def api_trader_issues():
    """Latest quality scan issues filtered to a single trader's clients.

    Traders see only themselves; admins see their own traders; super_admin /
    bef_admin can pass ?trader=<name>. Clients with no scan record are
    returned with total_issues=0 so the UI can show "all clear" rows too.
    """
    try:
        from dashboard.database import get_quality_scan_results
        trader_name, err = _resolve_trader_for_session(request.session_user, request.args.get('trader'))
        if err is not None:
            return err

        trader_clients = set(_clients_for_trader(trader_name))
        results = get_quality_scan_results() or []
        filtered = [r for r in results if r.get('client_id') in trader_clients]

        severity_weight = {'critical': 20, 'high': 10, 'medium': 5, 'low': 2, 'warning': 3, 'info': 0}
        for r in filtered:
            r['issues'] = [i for i in r.get('issues', []) if i.get('check') != 'Scan error']
            r['total_issues'] = len(r['issues'])
            deduction = sum(severity_weight.get(i.get('severity', 'low'), 2) for i in r['issues'])
            r['health_score'] = max(0.0, round(100.0 - deduction, 1))

        scanned_ids = {r['client_id'] for r in filtered}
        for nm in sorted(trader_clients - scanned_ids):
            filtered.append({
                'client_id': nm, 'trader': trader_name,
                'issues': [], 'total_issues': 0, 'health_score': 100.0,
                'scan_date': None,
            })

        filtered.sort(key=lambda r: (-r.get('total_issues', 0), r.get('client_id') or ''))
        total_issues = sum(r.get('total_issues', 0) for r in filtered)
        clients_with_issues = sum(1 for r in filtered if r.get('total_issues', 0) > 0)

        return jsonify({
            'status': 'success',
```

```

        'trader': trader_name,
        'total_clients': len(filtered),
        'clients_with_issues': clients_with_issues,
        'total_issues': total_issues,
        'results': filtered,
    })
except Exception as e:
    import traceback
    traceback.print_exc()
    return jsonify({'status': 'error', 'message': f'Failed to load trader issues: {str(e)}'}), 500

@app.route('/api/quality/trader_summary_status')
@require_role('trader', 'admin', 'super_admin', 'bef_admin', 'kwok_admin')
def api_trader_summary_status():

```

Daily summary submission status filtered to a single trader's clients.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_role** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **dict comprehension** — Builds a dict in one expression (`{k: v for k, v in items}`) — same intent as a list comprehension but for mappings.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to `sum`, `any`, `all`, or `max` where the intermediate list would be wasted.
- **lambda** — Uses a single-expression anonymous function — typically as the `key=` for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```

def api_trader_summary_status():
    """Daily summary submission status filtered to a single trader's clients."""
    try:
        from dashboard.database import get_summary_status_for_date, get_setting, get_client_data
        import json as _json
        from datetime import timezone, timedelta as _td

        trader_name, err = _resolve_trader_for_session(request.session_user, request.args.get('trader'))

```



```

if err is not None:
    return err

date = request.args.get('date', datetime.now().strftime('%Y-%m-%d'))
submissions = get_summary_status_for_date(date) or []

_kenyan_tz = timezone(_td(hours=3))
for s in submissions:
    ts = s.get('submitted_at', '')
    if ts:
        try:
            dt = datetime.fromisoformat(str(ts).replace('Z', '+00:00'))
            if dt.tzinfo is None:
                dt = dt.replace(tzinfo=timezone.utc)
            s['submitted_at'] = dt.astimezone(_kenyan_tz).isoformat()
        except Exception:
            pass

sent_map = {s['client_id']: s for s in submissions}

excluded_traders = set(_json.loads(get_setting('summary_tracker_excluded_traders') or '[]'))
excluded_clients = set(_json.loads(get_setting('summary_tracker_excluded_clients') or '[]'))

clients_payload = []
for nm in _clients_for_trader(trader_name):
    if nm in excluded_clients:
        continue
    try:
        cdata = get_client_data(nm)
        if cdata and isinstance(cdata.get('identity'), dict):
            if cdata['identity'].get('active_status') == 'inactive':
                continue
    except Exception:
        pass
    if nm in sent_map:
        s = sent_map[nm]
        clients_payload.append({
            'client_id': nm,
            'sent': True,
            'submitted_by': s.get('submitted_by'),
            'submitted_at': s.get('submitted_at'),
        })
    else:
        clients_payload.append({'client_id': nm, 'sent': False})

clients_payload.sort(key=lambda c: (c['sent'], c['client_id'].lower()))

total_sent = sum(1 for c in clients_payload if c['sent'])
return jsonify({
    'status': 'success',
    'trader': trader_name,
    'date': date,
    'total_clients': len(clients_payload),
    'total_sent': total_sent,
    'total_not_sent': len(clients_payload) - total_sent,
    'trader_excluded': trader_name in excluded_traders,
    'clients': clients_payload,
})
except Exception as e:
    import traceback

```

```

        traceback.print_exc()
        return jsonify({'status': 'error', 'message': f'Failed to load trader summaries: {str(e)}'}, 500)

@app.route('/api/quality/summary_tracker_exclude', methods=['POST'])
@require_role('super_admin')
def api_summary_tracker_exclude()

```

Toggle a trader or client exclusion from the Daily Summary Tracker.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_role** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Lazy import from dashboard.database.
2. Imports json (lazy import inside the function).
3. Assigns data = request.get_json(force=True).
4. Assigns exclude_type = data.get('type').
5. Assigns name = (data.get('name') or '').strip().
6. Assigns action = data.get('action', 'toggle').
7. **if** exclude_type not in ('trader', 'client') or not name: branches conditionally.
8. Assigns key = f"summary_tracker_excluded_{('traders' if exclude_type ==
9. Assigns current = set(_json.loads(get_setting(key) or '[])).
10. **if** action == 'add': branches conditionally (with an **else/elif** arm).
11. Assigns user_id = request.session_user.get('user_identifier', '').
12. Calls set_setting(...) for its side effect.
13. Calls log_action(...) for its side effect.
14. **return** jsonify({'status': 'success', 'excluded': sorted(current)}).

Code (copy-paste safe)

```

def api_summary_tracker_exclude():
    """Toggle a trader or client exclusion from the Daily Summary Tracker."""
    from dashboard.database import get_setting, set_setting
    import json as _json

    data = request.get_json(force=True)
    exclude_type = data.get('type') # 'trader' or 'client'
    name = (data.get('name') or '').strip()

```

```

    action = data.get('action', 'toggle') # 'add', 'remove', or 'toggle'

    if exclude_type not in ('trader', 'client') or not name:
        return jsonify({'status': 'error', 'message': 'Invalid type or name'}), 400

    key = f'summary_tracker_excluded_{("traders" if exclude_type == "trader" else "clients")}'
    current = set(_json.loads(get_setting(key) or '[]'))

    if action == 'add':
        current.add(name)
    elif action == 'remove':
        current.discard(name)
    else: # toggle
        if name in current:
            current.discard(name)
        else:
            current.add(name)

    user_id = request.session_user.get('user_identifier', '')
    set_setting(key, _json.dumps(sorted(current)), updated_by=user_id)

    log_action('SUMMARY_TRACKER_EXCLUDE', 'super_admin', user_id,
              get_remote_address(), f'{action} {exclude_type}: {name}')

    return jsonify({'status': 'success', 'excluded': sorted(current)})

@app.route('/api/quality/trade_count_toggle', methods=['POST'])
@require_session
def api_trade_count_toggle()

```

Toggle a client for trade-count tracking in the daily summary (off by default).

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_session** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Lazy import from dashboard.database.
2. Imports json (lazy import inside the function).
3. Assigns data = request.get_json(force=True).
4. Assigns client_name = (data.get('client') or '').strip().
5. Assigns action = data.get('action', 'toggle').
6. **if** not client_name: branches conditionally.
7. Assigns key = 'trade_count_enabled_clients'.
8. Assigns current = set(_json.loads(get_setting(key) or '[]')).
9. **if** action == 'add': branches conditionally (with an **else/elif** arm).

10. Assigns `user_id = request.session_user.get('user_identifier', '')`.
11. Assigns `user_type = request.session_user.get('user_type', '')`.
12. Calls `set_setting(...)` for its side effect.
13. Calls `log_action(...)` for its side effect.
14. **return** `jsonify({'status': 'success', 'enabled_clients': sorted(current)})`.

Code (copy-paste safe)

```
def api_trade_count_toggle():
    """Toggle a client for trade-count tracking in the daily summary (off by default)."""
    from dashboard.database import get_setting, set_setting
    import json as _json

    data = request.get_json(force=True)
    client_name = (data.get('client') or '').strip()
    action = data.get('action', 'toggle') # 'add', 'remove', or 'toggle'

    if not client_name:
        return jsonify({'status': 'error', 'message': 'Client name required'}), 400

    key = 'trade_count_enabled_clients'
    current = set(_json.loads(get_setting(key) or '[]'))

    if action == 'add':
        current.add(client_name)
    elif action == 'remove':
        current.discard(client_name)
    else:
        if client_name in current:
            current.discard(client_name)
        else:
            current.add(client_name)

    user_id = request.session_user.get('user_identifier', '')
    user_type = request.session_user.get('user_type', '')
    set_setting(key, _json.dumps(sorted(current)), updated_by=user_id)

    log_action('TRADE_COUNT_TOGGLE', user_type, user_id,
              get_remote_address(), f'{action} client: {client_name}')

    return jsonify({'status': 'success', 'enabled_clients': sorted(current)})

@app.route('/api/quality/trade_count_clients')
@require_session
def api_trade_count_clients():
```

Return list of clients with trade-count tracking enabled.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_session** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.

Step by step (top-level body)

1. Lazy import from dashboard.database.
2. Imports json (lazy import inside the function).
3. Assigns enabled = sorted(set(_json.loads(get_setting('trade_count_enabled_c....
4. **return** jsonify({'status': 'success', 'enabled_clients': enabled}).

Code (copy-paste safe)

```
def api_trade_count_clients():
    """Return list of clients with trade-count tracking enabled."""
    from dashboard.database import get_setting
    import json as _json
    enabled = sorted(set(_json.loads(get_setting('trade_count_enabled_clients') or '[]')))
    return jsonify({'status': 'success', 'enabled_clients': enabled})

@app.route('/api/admin/repair_db', methods=['POST'])
@app.route('/api/admin/db_repair', methods=['POST'])
@require_role('super_admin')
def api_repair_database():
```

Attempt to repair a corrupted SQLite database. Super admin only.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_role** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.

Step by step (top-level body)

1. Lazy import from dashboard.database.
2. Assigns (ok, message) = check_and_repair_database().
3. Calls log_action(...) for its side effect.
4. **if** ok: branches conditionally.
5. **return** (jsonify({'status': 'error', 'message': message}), 500).

Code (copy-paste safe)

```
def api_repair_database():
    """Attempt to repair a corrupted SQLite database. Super admin only."""
    from dashboard.database import check_and_repair_database
    ok, message = check_and_repair_database()
    log_action('DB_REPAIR', 'super_admin', request.session_user.get('user_identifier'),
              get_remote_address(), message)
    if ok:
        return jsonify({'status': 'success', 'message': message})
    return jsonify({'status': 'error', 'message': message}), 500

@app.route('/api/checklist/submit', methods=['POST'])
@require_session
```

```
def api_submit_checklist()
```

Submit a daily checklist (per client).

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_session** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Lazy import from dashboard.database.
2. Assigns session_user = request.session_user.
3. Assigns user_type = session_user.get('user_type').
4. Assigns user_identifier = session_user.get('user_identifier').
5. Assigns data = request.json.
6. Assigns checklist_type = data.get('checklist_type', 'daily_summary').
7. Assigns client_id = data.get('client_id', '').
8. Assigns items = data.get('items', []).
9. **if not items**: branches conditionally.
10. Assigns today = datetime.now().strftime('%Y-%m-%d').
11. Calls save_daily_checklist(...) for its side effect.
12. Calls log_action(...) for its side effect.
13. **return** jsonify({'status': 'success', 'message': 'Daily summary saved'}).

Code (copy-paste safe)

```
def api_submit_checklist():
    """Submit a daily checklist (per client)."""
    from dashboard.database import save_daily_checklist
    session_user = request.session_user
    user_type = session_user.get('user_type')
    user_identifier = session_user.get('user_identifier')

    data = request.json
    checklist_type = data.get('checklist_type', 'daily_summary')
    client_id = data.get('client_id', '')
    items = data.get('items', [])

    if not items:
        return jsonify({'status': 'error', 'message': 'No items provided'}), 400

    today = datetime.now().strftime('%Y-%m-%d')
    save_daily_checklist(today, user_identifier, user_type, checklist_type, items,
                        get_remote_address(), client_id=client_id)
```

```

log_action('CHECKLIST_SUBMIT', user_type, user_identifier, get_remote_address(),
          f"{checklist_type} for {client_id}: {len(items)} sections")

return jsonify({'status': 'success', 'message': 'Daily summary saved'})

@app.route('/api/checklist/status')
@require_session
def api_checklist_status():
    Get checklist status for today (or specified date).

```

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_session** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.

Step by step (top-level body)

1. Lazy import from `dashboard.database`.
2. Assigns `session_user = request.session_user`.
3. Assigns `user_identifier = session_user.get('user_identifier')`.
4. Assigns `user_type = session_user.get('user_type')`.
5. Assigns `date = request.args.get('date', datetime.now().strftime('%Y-%m-%...'))`.
6. Assigns `client_id = request.args.get('client_id', '')`.
7. **if** `user_type` in `('super_admin', 'bef_admin')`: branches conditionally (with an **else/elif** arm).
8. **if** `client_id`: branches conditionally.
9. **return** `jsonify({'status': 'success', 'date': date, 'checklists': checklists})`.

Code (copy-paste safe)

```

def api_checklist_status():
    """Get checklist status for today (or specified date)."""
    from dashboard.database import get_daily_checklists
    session_user = request.session_user
    user_identifier = session_user.get('user_identifier')
    user_type = session_user.get('user_type')

    date = request.args.get('date', datetime.now().strftime('%Y-%m-%d'))
    client_id = request.args.get('client_id', '')

    if user_type in ('super_admin', 'bef_admin'):
        checklists = get_daily_checklists(date)
    else:
        checklists = get_daily_checklists(date, user_identifier)

    # Filter by client_id if requested
    if client_id:
        checklists = [c for c in checklists if c.get('client_id', '') == client_id]

```

```

        return jsonify({'status': 'success', 'date': date, 'checklists': checklists})

@app.route('/api/admin/summary_signoff', methods=['POST'])
@require_role('admin', 'super_admin', 'bef_admin', 'kwok_admin')
def api_admin_summary_signoff():

```

Admin sign-off: confirm trader summary reviewed + sent to client (persisted like trader tracking).

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_role** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Lazy import from dashboard.database.
2. Assigns session_user = request.session_user.
3. Assigns user_type = session_user.get('user_type').
4. Assigns session_id = (session_user.get('user_identifier') or "").strip().
5. Assigns data = request.get_json(silent=True) or {}.
6. Assigns client_id = (data.get('client_id') or "").strip().
7. Assigns admin_name = (data.get('admin') or "").strip().
8. Assigns checked = bool(data.get('checked', False)).
9. **if** not client_id: branches conditionally.
10. **if** user_type == 'admin': branches conditionally (with an **else/elif** arm).
11. **if** not effective_admin: branches conditionally.
12. Assigns today = datetime.now().strftime('%Y-%m-%d').
13. Assigns items = [{'id': 'sent_to_client', 'title': 'Sent summary to clien....
14. Calls save_daily_checklist(...) for its side effect.
15. ... and 2 more statement(s) in the body.

Code (copy-paste safe)

```

def api_admin_summary_signoff():
    """Admin sign-off: confirm trader summary reviewed + sent to client (persisted like trader tracking)
    from dashboard.database import save_daily_checklist

```



```

session_user = request.session_user
user_type = session_user.get('user_type')
session_id = (session_user.get('user_identifier') or '').strip()

data = request.get_json(silent=True) or {}
client_id = (data.get('client_id') or '').strip()
admin_name = (data.get('admin') or '').strip()
checked = bool(data.get('checked', False))

if not client_id:
    return jsonify({'status': 'error', 'message': 'client_id required'}), 400

# Admin users can only sign off as themselves.
# Super admin / BEF / kwok can sign off on behalf of a specific admin dashboard.
if user_type == 'admin':
    effective_admin = session_id
else:
    effective_admin = admin_name
if not effective_admin:
    return jsonify({'status': 'error', 'message': 'Admin name required'}), 400

today = datetime.now().strftime('%Y-%m-%d')
items = [{
    'id': 'sent_to_client',
    'title': 'Sent summary to client (approved)',
    'checked': checked,
    'status': 'ok' if checked else 'pending',
    'notes': ''
}]

# Persist under the effective admin identity so the tracker reads it back correctly.
save_daily_checklist(today, effective_admin, 'admin', 'admin_daily_summary', items,
    get_remote_address(), client_id=client_id)

try:
    log_action('ADMIN_SUMMARY_SIGNOFF', 'admin', effective_admin, get_remote_address(),
        f'{"checked" if checked else "unchecked"} {client_id}')
except Exception:
    pass

return jsonify({'status': 'success'})

@app.route('/api/quality/scorecard')
@require_role('super_admin', 'bef_admin')
def api_weekly_scorecard()

```

Generate weekly scorecard aggregating quality scan data per trader.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_role** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **list comprehension** — Builds a list in a single expression ([f(x) for x in xs]). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.

- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.

- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Lazy import from dashboard.database.
2. Assigns end_date = request.args.get('end_date', datetime.now().strftime('%Y-....
3. Assigns days = int(request.args.get('days', 7)).
4. Assigns results = get_weekly_scan_results(end_date, days).
5. if not results: branches conditionally.
6. Assigns severity_weight = {'critical': 20, 'high': 10, 'medium': 5, 'low': 2, 'warn....
7. for r in results: iterates.
8. Assigns start_date = (datetime.strptime(end_date, '%Y-%m-%d') - timedelta(days....
9. Assigns traders = {}.
10. Assigns scan_dates = set().
11. for r in results: iterates.
12. Assigns scorecard = {}.
13. for (t, data) in traders.items(): iterates.
14. Assigns checklist_summary = {}.
15. ... and 5 more statement(s) in the body.

Code (copy-paste safe)

```
def api_weekly_scorecard():
    """Generate weekly scorecard aggregating quality scan data per trader."""
    from dashboard.database import get_weekly_scan_results, get_daily_checklists
    end_date = request.args.get('end_date', datetime.now().strftime('%Y-%m-%d'))
    days = int(request.args.get('days', 7))
    results = get_weekly_scan_results(end_date, days)
    if not results:
        return jsonify({'status': 'success', 'scorecard': {}, 'message': 'No scan data for this period.'})

    # Filter out infrastructure scan errors and recalculate scores
    severity_weight = {'critical': 20, 'high': 10, 'medium': 5, 'low': 2, 'warning': 3, 'info': 0}
    for r in results:
        r['issues'] = [i for i in r.get('issues', []) if i.get('check') != 'Scan error']
        r['total_issues'] = len(r['issues'])
        deduction = sum(severity_weight.get(i.get('severity', 'low'), 2) for i in r['issues'])
        r['health_score'] = max(0.0, round(100.0 - deduction, 1))

    start_date = (datetime.strptime(end_date, '%Y-%m-%d') - timedelta(days=days - 1)).strftime('%Y-%m-%d')

    # Aggregate by trader
    traders = {}
    scan_dates = set()
    for r in results:
```

```

t = r.get('trader', 'Unknown')
sd = r['scan_date']
scan_dates.add(sd)
if t not in traders:
    traders[t] = {'clients': set(), 'daily': {}, 'total_issues': 0, 'total_health': 0,
                  'scan_count': 0}
traders[t]['clients'].add(r['client_id'])
traders[t]['total_issues'] += r['total_issues']
traders[t]['total_health'] += r['health_score']
traders[t]['scan_count'] += 1
if sd not in traders[t]['daily']:
    traders[t]['daily'][sd] = {'issues': 0, 'health_sum': 0, 'count': 0}
traders[t]['daily'][sd]['issues'] += r['total_issues']
traders[t]['daily'][sd]['health_sum'] += r['health_score']
traders[t]['daily'][sd]['count'] += 1

# Build scorecard
scorecard = {}
for t, data in traders.items():
    avg_health = round(data['total_health'] / data['scan_count'], 1) if data['scan_count'] else 0
    # Health trend: compare first half vs second half
    sorted_dates = sorted(data['daily'].keys())
    mid = len(sorted_dates) // 2
    first_half = sorted_dates[:mid] if mid > 0 else sorted_dates
    second_half = sorted_dates[mid:] if mid > 0 else sorted_dates
    fh_health = sum(data['daily'][d]['health_sum'] / data['daily'][d]['count'] for d in first_half) / len(first_half) if first_half else 0
    sh_health = sum(data['daily'][d]['health_sum'] / data['daily'][d]['count'] for d in second_half) / len(second_half) if second_half else 0
    trend = 'improving' if sh_health > fh_health + 2 else (
        'declining' if sh_health < fh_health - 2 else 'stable')

    daily_breakdown = {}
    for d in sorted_dates:
        dd = data['daily'][d]
        daily_breakdown[d] = {
            'issues': dd['issues'],
            'avg_health': round(dd['health_sum'] / dd['count'], 1),
            'clients_scanned': dd['count']
        }

    # Grade based on avg health
    grade = 'A' if avg_health >= 90 else 'B' if avg_health >= 75 else 'C' if avg_health >= 60 else 'D'

    scorecard[t] = {
        'total_clients': len(data['clients']),
        'total_issues': data['total_issues'],
        'avg_health': avg_health,
        'grade': grade,
        'trend': trend,
        'scans_in_period': len(sorted_dates),
        'daily': daily_breakdown
    }

# Checklist completion for the period
checklist_summary = {}
current = datetime.strptime(start_date, '%Y-%m-%d')
end_dt = datetime.strptime(end_date, '%Y-%m-%d')
while current <= end_dt:
    d = current.strftime('%Y-%m-%d')

```

```

cls = get_daily_checklists(d)
for cl in cls:
    uid = cl['user_identifier']
    if uid not in checklist_summary:
        checklist_summary[uid] = {'completed': 0, 'total_days': 0}
    checklist_summary[uid]['completed'] += 1
    current += timedelta(days=1)
for uid in checklist_summary:
    checklist_summary[uid]['total_days'] = days

return jsonify({
    'status': 'success',
    'period': {'start': start_date, 'end': end_date, 'days': days},
    'scan_dates': sorted(scan_dates),
    'scorecard': scorecard,
    'checklist_completion': checklist_summary
})

```

```

@app.route('/api/quality/daily_summary')
@require_role('super_admin', 'bef_admin')
def api_daily_summary()

```

Generate a text summary of today's dashboard state for Discord/team sharing.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_role** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.
- **dict comprehension** — Builds a dict in one expression (`{k: v for k, v in items}`) — same intent as a list comprehension but for mappings.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to sum, any, all, or max where the intermediate list would be wasted.
- **lambda** — Uses a single-expression anonymous function — typically as the key= for a sort, the filter predicate, or a small callback.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Lazy import from dashboard.database.
2. Lazy import from config.hierarchy.
3. Lazy import from config.hierarchy.
4. Assigns date = request.args.get('date', datetime.now().strftime('%Y-%m-%...'))
5. Assigns scan_results = get_quality_scan_results(date).
6. Assigns checklists = get_daily_checklists(date).
7. try block with 1 except clause.
8. Assigns severity_weight = {'critical': 20, 'high': 10, 'medium': 5, 'low': 2, 'warn....'}
9. for r in scan_results: iterates.
10. Assigns all_clients = get_all_clients().
11. Assigns filtered_clients = [].
12. for client_name in all_clients: iterates.
13. Assigns total_clients = len(filtered_clients).
14. if excluded_traders or excluded_clients: branches conditionally.
15. ... and 32 more statement(s) in the body.

Code (copy-paste safe)

```
def api_daily_summary():
    """Generate a text summary of today's dashboard state for Discord/team sharing."""
    from dashboard.database import get_quality_scan_results, get_daily_checklists
    from config.hierarchy import get_all_clients, get_client_profile
    from config.hierarchy import SYSTEM_HIERARCHY

    # UTC date – server runs UTC; midnight UTC = 3 AM Kenyan, so the day
    # flips after the automated 2:05 AM Kenyan Slack send.
    date = request.args.get('date', datetime.now().strftime('%Y-%m-%d'))
    scan_results = get_quality_scan_results(date)
    checklists = get_daily_checklists(date)

    # Apply Daily Summary Tracker exclusions to the generated report as well
    # (so preview matches what the Slack bot posts).
    try:
        from dashboard.database import get_setting
        import json as _json_ex
        excluded_traders = set(_json_ex.loads(get_setting('summary_tracker_excluded_traders') or '[]'))
        excluded_clients = set(_json_ex.loads(get_setting('summary_tracker_excluded_clients') or '[]'))
    except Exception:
        excluded_traders = set()
        excluded_clients = set()

    # Filter out infrastructure scan errors and recalculate scores
    severity_weight = {'critical': 20, 'high': 10, 'medium': 5, 'low': 2, 'warning': 3, 'info': 0}
    for r in scan_results:
        r['issues'] = [i for i in r.get('issues', []) if i.get('check') != 'Scan error']
        r['total_issues'] = len(r['issues'])
        deduction = sum(severity_weight.get(i.get('severity', 'low'), 2) for i in r['issues'])
        r['health_score'] = max(0.0, round(100.0 - deduction, 1))

    all_clients = get_all_clients()
```

```

# Filter portfolio list to excluded traders/clients for the report header.
filtered_clients = []
for client_name in all_clients:
    if client_name in excluded_clients:
        continue
    prof = get_client_profile(client_name) or {}
    trader = (prof.get('trader') or '') or 'Unassigned'
    if trader in excluded_traders:
        continue
    filtered_clients.append(client_name)
total_clients = len(filtered_clients)

# Filter scan_results so top issues + leaderboard align with exclusions
if excluded_traders or excluded_clients:
    scan_results = [
        r for r in (scan_results or [])
        if (r.get('client_id') not in excluded_clients)
        and ((r.get('trader') or 'Unassigned') not in excluded_traders)
    ]

# Count active vs issues from scan
clients_healthy = sum(1 for r in scan_results if r['health_score'] >= 90)
clients_warning = sum(1 for r in scan_results if 70 <= r['health_score'] < 90)
clients_critical = sum(1 for r in scan_results if r['health_score'] < 70)
total_issues = sum(r['total_issues'] for r in scan_results)
avg_health = round(sum(r['health_score'] for r in scan_results) / len(scan_results),
    1) if scan_results else 0

# Top issues by frequency
issue_counts = {}
for r in scan_results:
    for iss in r['issues']:
        key = iss['check']
        issue_counts[key] = issue_counts.get(key, 0) + 1
top_issues = sorted(issue_counts.items(), key=lambda x: -x[1])[:5]

# Trader breakdown
trader_stats = {}
for r in scan_results:
    t = r.get('trader', 'Unknown')
    if t not in trader_stats:
        trader_stats[t] = {'clients': 0, 'issues': 0, 'health_sum': 0}
    trader_stats[t]['clients'] += 1
    trader_stats[t]['issues'] += r['total_issues']
    trader_stats[t]['health_sum'] += r['health_score']

# Checklist status
checklist_count = len(checklists)

# Build the text summary
weekday = datetime.strptime(date, '%Y-%m-%d').strftime('%A')
lines = []
lines.append(f"■ **Daily Quality Summary – {weekday}, {date}**")
lines.append("")
lines.append(f"■ **Portfolio:** {total_clients} total clients")
if scan_results:
    lines.append(
        f"■ Healthy (90%+): {clients_healthy} | ■ Warning: {clients_warning} | ■ Critical: {clients_critical}"
    )
    lines.append(f"■ Avg Health Score: **{avg_health}%** | Total Issues: **{total_issues}**")
else:

```

```

        lines.append("■ No quality scan run today yet.")
    lines.append("")

    if top_issues:
        lines.append("■ **Top Issues:**")
        for check, count in top_issues:
            lines.append(f"    • {check}: {count} occurrences")
        lines.append("")

    # Collect downtime data (will be rendered at the bottom)
    downtime_clients = []
    for r in scan_results:
        for iss in r['issues']:
            if iss['check'] == 'Downtime detected':
                downtime_clients.append((r.get('trader', 'Unknown'), r['client_id'], iss['detail']))

    if trader_stats:
        # Gamified Trader Health Leaderboard – ranked best to worst
        ranked = sorted(trader_stats.items(), key=lambda x: x[1]['health_sum'] / max(x[1]['clients'], 1),
                        reverse=True)
        lines.append("■ **TRADER HEALTH LEADERBOARD**")
        lines.append(
            "_Ranked by average client health score (highest first). Health score is based on: data fresh")
        lines.append("")
        total_traders = len(ranked)
        for rank, (t, s) in enumerate(ranked, 1):
            avg = round(s['health_sum'] / s['clients'], 1)
            # Rank medals
            if rank == 1:
                medal = '■'
            elif rank == 2:
                medal = '■'
            elif rank == 3:
                medal = '■'
            else:
                medal = f'`#{rank}`'
            # Fun titles based on health
            if avg >= 95:
                title = '■ Legendary'
            elif avg >= 90:
                title = '■ Elite'
            elif avg >= 80:
                title = '■ Solid'
            elif avg >= 70:
                title = '■ Warming Up'
            elif avg >= 50:
                title = '■ Needs Work'
            else:
                title = '■ SOS'
            bar_filled = round(avg / 10)
            bar_empty = 10 - bar_filled
            bar = '■' * bar_filled + '■' * bar_empty
            lines.append(f"{medal} **{t}** – {title}")
            lines.append(f"    {bar} **{avg}%** · {s['clients']} clients · {s['issues']} issues")
        # Fun footer
        if total_traders > 0:
            best_name = ranked[0][0]
            worst_name = ranked[-1][0]
            best_avg = round(ranked[0][1]['health_sum'] / max(ranked[0][1]['clients'], 1), 1)
            worst_avg = round(ranked[-1][1]['health_sum'] / max(ranked[-1][1]['clients'], 1), 1)

```

```

        lines.append("")
        if best_avg >= 90:
            lines.append(f"■ **{best_name}** is on fire! Leading the pack at {best_avg}%")
        if worst_avg < 50 and total_traders > 1:
            lines.append(f"■ **{worst_name}** – time to level up! Let's get those numbers moving ■")
    lines.append("")

lines.append(f"■ Checklists submitted today: **{checklist_count}**")
lines.append("")

# ■■ Daily Summary Submission Tracker ■■
try:
    from dashboard.database import get_summary_status_for_date, get_setting, get_client_data
    from config.hierarchy import get_client_profile as _gcp
    from datetime import timezone, timedelta as _td
    import json as _json_mod

    _kenyan_tz = timezone(_td(hours=3))
    submissions = get_summary_status_for_date(date)
    # Convert timestamps to Kenyan time
    for s in submissions:
        ts = s.get('submitted_at', '')
        if ts:
            try:
                dt = datetime.fromisoformat(ts.replace('Z', '+00:00'))
                if dt.tzinfo is None:
                    dt = dt.replace(tzinfo=timezone.utc)
                s['submitted_at'] = dt.astimezone(_kenyan_tz).isoformat()
            except Exception:
                pass

    sent_map = {s['client_id']: s for s in submissions}
    excluded_traders = set(_json_mod.loads(get_setting('summary_tracker_excluded_traders') or '[]'))
    excluded_clients = set(_json_mod.loads(get_setting('summary_tracker_excluded_clients') or '[]'))

    # Build per-trader summary submission data
    tracker_traders = {} # trader -> {sent: [{client_id, time}], total: int}
    tracked_total = 0
    for client_name in all_clients:
        profile = _gcp(client_name)
        trader = (profile.get('trader', '') if profile else '') or 'Unassigned'
        if trader in excluded_traders or client_name in excluded_clients:
            continue
        try:
            cdata = get_client_data(client_name)
            if cdata and isinstance(cdata.get('identity'), dict):
                if cdata['identity'].get('active_status') == 'inactive':
                    continue
        except Exception:
            pass
        tracked_total += 1
        if trader not in tracker_traders:
            tracker_traders[trader] = {'sent': [], 'not_sent': [], 'total': 0}
        tracker_traders[trader]['total'] += 1
        if client_name in sent_map:
            ts = sent_map[client_name].get('submitted_at', '')
            tracker_traders[trader]['sent'].append(ts)
        else:
            tracker_traders[trader]['not_sent'].append(client_name)

    # Split: 100% complete → ranked; everyone else → incomplete

```



```

tracker_complete = []
tracker_incomplete = []
for t, d in tracker_traders.items():
    sent_count = len(d['sent'])
    if sent_count == d['total']:
        minutes_list = []
        for ts in d['sent']:
            try:
                dt = datetime.fromisoformat(ts)
                minutes_list.append(dt.hour * 60 + dt.minute)
            except Exception:
                pass
        avg_minutes = round(sum(minutes_list) / len(minutes_list)) if minutes_list else 1440
        avg_hh = avg_minutes // 60
        avg_mm = avg_minutes % 60
        avg_time_str = f"{avg_hh:02d}:{avg_mm:02d}"
        tracker_complete.append((t, sent_count, d['total'], avg_minutes, avg_time_str))
    else:
        tracker_incomplete.append((t, sent_count, d['total'], d['not_sent']))

tracker_complete.sort(key=lambda x: x[3])
tracker_incomplete.sort(key=lambda x: x[0])
total_sent_summary = sum(x[1] for x in tracker_complete) + sum(x[1] for x in tracker_incomplete)

lines.append("■ **DAILY SUMMARY SUBMISSION BY MIDNIGHT (KENYAN TIME)**")
# Skip submission tracking on weekends (no trading Sat/Sun)
from datetime import timezone as _tz2, timedelta as _td2
_eat_now = datetime.now(_tz2(_td2(hours=3)))
_is_weekend = _eat_now.weekday() in (5, 6) # Saturday=5, Sunday=6
if _is_weekend:
    lines.append("■ _Weekend – no trading today. Submission tracking resumes on Monday._")
    lines.append("")
else:
    pct = round(total_sent_summary / tracked_total * 100) if tracked_total else 0
    lines.append(f"■ {total_sent_summary}/{tracked_total} sent ({pct}%)")
    lines.append("")
    if tracker_complete:
        lines.append("■ **Complete – ranked by earliest avg submission time:**")
        lines.append(
            "_All your clients' summaries must be submitted to qualify. The earlier you submit, t
        )
        for rank, (t, sent, total, _avg_m, avg_t) in enumerate(tracker_complete, 1):
            if rank == 1:
                medal = '■'
            elif rank == 2:
                medal = '■'
            elif rank == 3:
                medal = '■'
            else:
                medal = f'`#{rank}`'
            lines.append(f"{medal} **{t}** – {sent}/{total} ■ · avg {avg_t}")
        lines.append("")
    if tracker_incomplete:
        lines.append("■ **Incomplete – missing clients:**")
        for t, sent, total, missing in tracker_incomplete:
            lines.append(f"■ **{t}** – {sent}/{total} sent")
            lines.append(f"    ■ {' '.join(missing)}")
        lines.append("")
    lines.append("■ _We track everything – every submission, every miss, every second._")
    lines.append("")
except Exception as e:

```

```

import traceback
traceback.print_exc()

# ■■ Downtime Alert (bottom of message for maximum visibility) ■■
if downtime_clients:
    lines.append("■" * 30)
    lines.append("")
    lines.append("■■■ **DOWNTIME ALERT – ZERO TOLERANCE** ■■■")
    lines.append(
        f"■■■ **{len(downtime_clients)} account(s) have stale trading days. This means the account wa"
    )
    lines.append("")
    for trader, client, detail in sorted(downtime_clients):
        acct = ''
        if '[' in detail and ']' in detail:
            acct = detail.split('[')[1].split(']')[0]
            stale_part = detail.split('Stale day(s) found: ')[-1].split(' -')[
                0] if 'Stale day(s) found: ' in detail else detail
            acct_tag = f" . {acct}" if acct and acct != 'no acct#' else ''
            lines.append(f" ■ **{client}** ({trader}{acct_tag}) – {stale_part}")
    lines.append("")
    lines.append(
        "■■■ **Downtime is unacceptable. Every trading day must be accounted for. Traders responsible"
    )
    lines.append("")

lines.append("-")

# ■■ Admin Tracker Summary (issues + sign-offs) ■■
try:
    admins_map = SYSTEM_HIERARCHY.get('admins', {}) if isinstance(SYSTEM_HIERARCHY, dict) else {}
    admin_names = sorted([a for a in admins_map.keys() if str(a).strip()])
    if admin_names:
        lines.append("")
        lines.append("■ **ADMIN TRACKER (issues + sign-offs)**")
        lines.append(
            "_Based on admin-owned checks: fees, prop-firm max-out, downtime, and missing client sign"
        )
        lines.append("")

        admin_rows = []
        total_admin_issues = 0
        total_required_signoffs = 0
        total_signed_signoffs = 0

        for a in admin_names:
            payload = compute_admin_tracker_payload(a, date) or {}
            issues = payload.get('issues') or payload.get('admin_issues') or []
            sign = payload.get('summary_signoff') or {}
            required = int(sign.get('required_total') or 0)
            signed = int(sign.get('signed_total') or 0)
            pending = int(sign.get('pending_total') or 0)

            total_admin_issues += len(issues)
            total_required_signoffs += required
            total_signed_signoffs += signed

            admin_rows.append({
                'admin': a,
                'health': float(payload.get('health_score') or 0.0),
                'clients': int(payload.get('total_clients') or 0),
                'issues': len(issues),
                'pending_signoffs': pending,
            })

```

```

        'pending_clients': (sign.get('pending_clients') or []),
    })

admin_rows.sort(key=lambda r: (r['health'], r['clients']), reverse=True)
avg_admin_health = round(sum(r['health'] for r in admin_rows) / len(admin_rows),
    1) if admin_rows else 0.0

lines.append(f"■ **Admins tracked:** {len(admin_rows)}")
lines.append(
    f"■ Avg Admin Health Score: **{avg_admin_health}%** | Total Admin Issues: **{total_admin_issues}%**")
if total_required_signoffs:
    pct = round((total_signed_signoffs / total_required_signoffs) * 100)
    lines.append(
        f"■ Admin sign-offs: **{total_signed_signoffs}/{total_required_signoffs}%** ({pct}%)"
    )
else:
    lines.append("■ Admin sign-offs: **0/0** (no trader submissions yet)")
lines.append("")

lines.append("■ **ADMIN HEALTH LEADERBOARD**")
lines.append("_Ranked by admin health score (highest first).")
lines.append("")
for rank, r in enumerate(admin_rows, 1):
    if rank == 1:
        medal = '■'
    elif rank == 2:
        medal = '■'
    elif rank == 3:
        medal = '■'
    else:
        medal = f'#{rank}'
    bar_filled = round(r['health'] / 10)
    bar_empty = 10 - bar_filled
    bar = '■' * bar_filled + '■' * bar_empty
    extra = f" · {r['pending_signoffs']} pending sign-offs" if r['pending_signoffs'] else ""
    lines.append(f"{medal} **{r['admin']}**")
    lines.append(
        f"    {bar} **{r['health']}%** · {r['clients']} clients · {r['issues']} issues{extra}"
    )

incomplete = [r for r in admin_rows if r['pending_signoffs'] > 0]
if incomplete:
    lines.append("")
    lines.append("■ **ADMIN DAILY SUMMARY SIGN-OFF (after trader submits)**")
    lines.append("■ **Incomplete – pending client sign-offs:**")
    for r in sorted(incomplete, key=lambda x: (-x['pending_signoffs'], x['admin'])):
        lines.append(f"■ **{r['admin']}** – {r['pending_signoffs']} pending")
        missing = r.get('pending_clients') or []
        if len(missing) > 25:
            shown = ", ".join(missing[:25]) + f", +{len(missing) - 25} more"
        else:
            shown = ", ".join(missing)
        lines.append(f"    ■ {shown}")
    lines.append("")
except Exception:
    import traceback
    traceback.print_exc()

summary_text = "\n".join(lines)

fmt = request.args.get('format', 'json')
if fmt == 'text':

```

```

        return summary_text, 200, {'Content-Type': 'text/plain; charset=utf-8'}

    return jsonify({
        'status': 'success',
        'date': date,
        'summary': summary_text,
        'stats': {
            'total_clients': total_clients,
            'scanned': len(scan_results),
            'healthy': clients_healthy,
            'warning': clients_warning,
            'critical': clients_critical,
            'total_issues': total_issues,
            'avg_health': avg_health,
            'checklists_submitted': checklist_count
        }
    })

```

```

@app.route('/api/settings/slack_webhook', methods=['GET'])
@require_role('super_admin')
def api_get_slack_webhook()

```

Get the current Slack webhook URL (masked). Super admin only.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_role** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Lazy import from dashboard.database.
2. Assigns url = get_setting('slack_webhook_url').
3. if url: branches conditionally.
4. return jsonify({'status': 'success', 'configured': False, 'masked_url': ''}).

Code (copy-paste safe)

```

def api_get_slack_webhook():
    """Get the current Slack webhook URL (masked). Super admin only."""
    from dashboard.database import get_setting
    url = get_setting('slack_webhook_url')
    if url:
        # Mask the URL for display – show first 40 chars + last 6
        masked = url[:40] + '...' + url[-6:] if len(url) > 50 else url
        return jsonify({'status': 'success', 'configured': True, 'masked_url': masked})
    return jsonify({'status': 'success', 'configured': False, 'masked_url': ''})

@app.route('/api/settings/slack_webhook', methods=['POST'])
@require_role('super_admin')
def api_set_slack_webhook()

```

Set the Slack webhook URL. Super admin only.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_role** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Lazy import from dashboard.database.
2. Assigns data = request.get_json(force=True).
3. Assigns url = (data.get('url') or '').strip().
4. Assigns user = request.session_user.get('user_identifier', '').
5. **if** url and (not url.startswith('https://hooks.slack.com/')): branches conditionally.
6. Calls set_setting(...) for its side effect.
7. Assigns action = 'configured' if url else 'removed'.
8. Calls log_action(...) for its side effect.
9. **return** jsonify({'status': 'success', 'message': f'Slack webhook {action} s....

Code (copy-paste safe)

```
def api_set_slack_webhook():
    """Set the Slack webhook URL. Super admin only."""
    from dashboard.database import set_setting
    data = request.get_json(force=True)
    url = (data.get('url') or '').strip()
    user = request.session_user.get('user_identifier', '')

    if url and not url.startswith('https://hooks.slack.com/'):
        return jsonify({'status': 'error',
                        'message': 'Invalid Slack webhook URL. Must start with https://hooks.slack.com/'}), 400

    set_setting('slack_webhook_url', url, updated_by=user)
    action = 'configured' if url else 'removed'
    log_action('SLACK_WEBHOOK', 'super_admin', user, get_remote_address(), f'Slack webhook {action}')
    return jsonify({'status': 'success', 'message': f'Slack webhook {action} successfully.'})

@app.route('/api/quality/send_slack', methods=['POST'])
@require_role('super_admin')
def api_send_slack_summary():
```

Manually post the daily quality summary to Slack. Super admin only. Accepts optional JSON body: { "test_webhook": "https://hooks.slack.com/..." } to override the target (e.g. send to a DM or test channel instead of main).

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_role** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Lazy import from dashboard.scheduler.
2. Assigns data = request.get_json(silent=True) or {}.
3. Assigns test_webhook = (data.get('test_webhook') or '').strip().
4. **if** test_webhook and (not test_webhook.startswith('https://hooks.slack....': branches conditionally.
5. **if** not test_webhook and (not _get_slack_webhook_url()): branches conditionally.
6. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def api_send_slack_summary():
    """Manually post the daily quality summary to Slack. Super admin only.
    Accepts optional JSON body: { "test_webhook": "https://hooks.slack.com/..." }
    to override the target (e.g. send to a DM or test channel instead of main).
    """
    from dashboard.scheduler import send_slack_message, send_slack_to_webhook, _build_daily_summary_text

    data = request.get_json(silent=True) or {}
    test_webhook = (data.get('test_webhook') or '').strip()

    # Validate test webhook if provided
    if test_webhook and not test_webhook.startswith('https://hooks.slack.com/'):
        return jsonify({'status': 'error',
                        'message': 'Invalid webhook URL – must start with https://hooks.slack.com/'}), 400

    if not test_webhook and not _get_slack_webhook_url():
        return jsonify({'status': 'error',
                        'message': 'Slack webhook not configured. Paste your webhook URL in the Settings section below'})

    try:
        text = _build_daily_summary_text()
        if test_webhook:
            text = "■ *[TEST MODE – DM only]*\n\n" + text
            ok = send_slack_to_webhook(test_webhook, text)
        else:
            ok = send_slack_message(text)
        if ok:
            dest = 'test webhook (DM)' if test_webhook else 'main channel'
```

```

        log_action('SLACK_SUMMARY', 'super_admin', request.session_user.get('user_identifier'),
                    get_remote_address(), f'Manual Slack summary posted to {dest}')
        return jsonify({'status': 'success', 'message': f'Summary posted to {dest}.'})
    else:
        return jsonify({'status': 'error',
                        'message': 'Slack post failed – check webhook URL and logs.'}), 502
except Exception as e:
    logging.error(f'Manual Slack post error: {e}')
    return jsonify({'status': 'error', 'message': str(e)}), 500

```

```

@app.route('/api/settings/slack_daily_webhook', methods=['GET'])
@require_role('super_admin')
def api_get_slack_daily_webhook()

```

Get the Daily Summaries Slack webhook URL (masked).

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_role** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Lazy import from dashboard.database.
2. Assigns url = get_setting('slack_daily_summaries_webhook_url').
3. if url: branches conditionally.
4. return jsonify({'status': 'success', 'configured': False, 'masked_url': ''}).

Code (copy-paste safe)

```

def api_get_slack_daily_webhook():
    """Get the Daily Summaries Slack webhook URL (masked)."""
    from dashboard.database import get_setting
    url = get_setting('slack_daily_summaries_webhook_url')
    if url:
        masked = url[:40] + '...' + url[-6:] if len(url) > 50 else url
        return jsonify({'status': 'success', 'configured': True, 'masked_url': masked})
    return jsonify({'status': 'success', 'configured': False, 'masked_url': ''})

@app.route('/api/settings/slack_daily_webhook', methods=['POST'])
@require_role('super_admin')
def api_set_slack_daily_webhook()

```

Set the Daily Summaries Slack webhook URL. Super admin only.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.

- **@require_role** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Lazy import from dashboard.database.
2. Assigns data = request.get_json(force=True).
3. Assigns url = (data.get('url') or '').strip().
4. Assigns user = request.session_user.get('user_identifier', '').
5. **if** url and (not url.startswith('https://hooks.slack.com/')): branches conditionally.
6. Calls set_setting(...) for its side effect.
7. Assigns action = 'configured' if url else 'removed'.
8. Calls log_action(...) for its side effect.
9. **return** jsonify({'status': 'success', 'message': f'Daily summaries Slack we....

Code (copy-paste safe)

```
def api_set_slack_daily_webhook():
    """Set the Daily Summaries Slack webhook URL. Super admin only."""
    from dashboard.database import set_setting
    data = request.get_json(force=True)
    url = (data.get('url') or '').strip()
    user = request.session_user.get('user_identifier', '')
    if url and not url.startswith('https://hooks.slack.com/'):
        return jsonify({'status': 'error', 'message': 'Invalid Slack webhook URL.'}), 400
    set_setting('slack_daily_summaries_webhook_url', url, updated_by=user)
    action = 'configured' if url else 'removed'
    log_action('SLACK_DAILY_WEBHOOK', 'super_admin', user, get_remote_address(),
              f'Daily summaries Slack webhook {action}')
    return jsonify({'status': 'success', 'message': f'Daily summaries Slack webhook {action}.'})

@app.route('/api/checklist/send_slack', methods=['POST'])
@require_session
def api_send_checklist_slack():

    Send a daily summary to Slack.
```

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_session** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't

have to handle it.

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Lazy import from dashboard.database.
2. Lazy import from dashboard.scheduler.
3. Assigns session_user = request.session_user.
4. Assigns user_type = session_user.get('user_type').
5. Assigns user_idenfier = session_user.get('user_idenfier', '').
6. **if** user_type == 'client': branches conditionally.
7. Assigns data = request.get_json(force=True).
8. Assigns summary_text = (data.get('text') or '').strip().
9. Assigns client_id = (data.get('client_id') or '').strip().
10. **if** not summary_text: branches conditionally.
11. Assigns webhook_url = get_setting('slack_daily_summaries_webhook_url').
12. **if** not webhook_url: branches conditionally.
13. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def api_send_checklist_slack():
    """Send a daily summary to Slack."""
    from dashboard.database import get_setting, save_daily_checklist
    from dashboard.scheduler import send_slack_to_webhook
    session_user = request.session_user
    user_type = session_user.get('user_type')
    user_idenfier = session_user.get('user_idenfier', '')

    if user_type == 'client':
        return jsonify({'status': 'error', 'message': 'Not authorized'}), 403

    data = request.get_json(force=True)
    summary_text = (data.get('text') or '').strip()
    client_id = (data.get('client_id') or '').strip()

    if not summary_text:
        return jsonify({'status': 'error', 'message': 'No summary text provided.'}), 400

    webhook_url = get_setting('slack_daily_summaries_webhook_url')
    if not webhook_url:
        return jsonify({'status': 'error',
                        'message': 'No Slack webhook configured. Ask a super admin to set one up in the Quality Dashb

    try:
        ok = send_slack_to_webhook(webhook_url, summary_text)
        if ok:
            if client_id:
                today = datetime.now().strftime('%Y-%m-%d')
                save_daily_checklist(today, user_idenfier, user_type, 'daily_summary',
```

```

        [{'id': 'slack_sent', 'title': 'Sent to Slack', 'status': 'ok',
          'notes': ''}],
        get_remote_address(), client_id=client_id)
    log_action('SLACK_DAILY_SUMMARY', user_type, user_identifier,
              get_remote_address(), f'Daily summary sent to Slack for {client_id}')
    return jsonify({'status': 'success', 'message': 'Summary sent to Slack!'})
    else:
        return jsonify({'status': 'error', 'message': 'Slack post failed – check webhook URL.'}), 500
except Exception as e:
    logging.error(f'Daily summary Slack post error: {e}')
    return jsonify({'status': 'error', 'message': str(e)}), 500

@app.route('/api/client/import_csv', methods=['POST'])
@require_role('super_admin')
def import_client_csv()

```

Import CSV data back into a client's evaluations. Super admin only.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_role** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **dict comprehension** — Builds a dict in one expression (`{k: v for k, v in items}`) — same intent as a list comprehension but for mappings.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to `sum`, `any`, `all`, or `max` where the intermediate list would be wasted.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Imports `csv` (lazy import inside the function).
2. Imports `io` (lazy import inside the function).
3. Lazy import from `dashboard.database`.
4. Assigns `session_user = request.session_user`.
5. Assigns `user_identifier = session_user.get('user_identifier')`.
6. Assigns `client_id = request.form.get('client_id')`.
7. **if not client_id**: branches conditionally.
8. Assigns `file = request.files.get('file')`.

9. **if** not file or not file.filename: branches conditionally.
10. **if** not file.filename.lower().endswith('.csv'): branches conditionally.
11. **try** block with 1 **except** clause.
12. **if** not rows: branches conditionally.
13. Assigns existing_data = get_client_data(client_id).
14. **if** not existing_data: branches conditionally.
15. ... and 11 more statement(s) in the body.

Code (copy-paste safe)

```
def import_client_csv():
    """Import CSV data back into a client's evaluations. Super admin only."""
    import csv
    import io
    from dashboard.database import get_client_data, save_client_data_with_history

    session_user = request.session_user
    user_identifier = session_user.get('user_identifier')

    client_id = request.form.get('client_id')
    if not client_id:
        return jsonify({"status": "error", "message": "client_id required"}), 400

    file = request.files.get('file')
    if not file or not file.filename:
        return jsonify({"status": "error", "message": "CSV file required"}), 400

    if not file.filename.lower().endswith('.csv'):
        return jsonify({"status": "error", "message": "File must be a .csv"}), 400

    # Read and parse CSV
    try:
        content = file.read().decode('utf-8-sig')
        reader = csv.DictReader(io.StringIO(content))
        rows = []
        for row in reader:
            # Stop at statistics separator
            first_val = list(row.values())[0] if row else ''
            if first_val and first_val.strip().startswith('--- '):
                break
            # Skip empty rows
            if all(not v.strip() for v in row.values()):
                continue
            rows.append(dict(row))
    except Exception as e:
        return jsonify({"status": "error", "message": f"Failed to parse CSV: {str(e)}"}), 400

    if not rows:
        return jsonify({"status": "error", "message": "CSV contains no data rows"}), 400

    # Load existing data
    existing_data = get_client_data(client_id)
    if not existing_data:
        return jsonify({"status": "error", "message": f"No existing data for client {client_id}"}), 404

    existing_evals = existing_data.get('evaluations', [])
```

```

# Build index of existing evaluations by Account # for matching
existing_by_account = {}
for idx, ev in enumerate(existing_evals):
    acct = (ev.get('Account #') or '').strip()
    if acct:
        existing_by_account[acct] = idx

# Merge: update matched rows, append new ones
updated_count = 0
added_count = 0
merged_evals = list(existing_evals) # copy

for csv_row in rows:
    acct = (csv_row.get('Account #') or '').strip()
    # Clean out internal keys
    clean_row = {k: v for k, v in csv_row.items() if not k.startswith('_')}

    if acct and acct in existing_by_account:
        # Update existing evaluation
        idx = existing_by_account[acct]
        for key, val in clean_row.items():
            if val.strip(): # only overwrite non-empty values
                merged_evals[idx][key] = val
        updated_count += 1
    else:
        # New row - append
        merged_evals.append(clean_row)
        added_count += 1

# Save with history
existing_data['evaluations'] = merged_evals
save_client_data_with_history(
    client_id, existing_data,
    changed_by=user_identifer,
    change_source='csv_import',
    change_description=f"CSV import: {updated_count} updated, {added_count} added"
)

log_action('CSV_IMPORT', 'super_admin', user_identifer, get_remote_address(),
    f"Imported CSV for {client_id}: {updated_count} updated, {added_count} added from {len(rows)} rows")

return jsonify({
    'status': 'success',
    'message': f'Import complete: {updated_count} rows updated, {added_count} rows added',
    'updated': updated_count,
    'added': added_count,
    'total_rows': len(merged_evals)
})

@app.route('/api/client/import_csv_companion', methods=['POST'])
@limiter.limit('10 per minute')
def import_csv_companion()

```

Import CSV data via companion app. Auth via email (same as sheet migration).

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.

- **@limiter.limit** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **dict comprehension** — Builds a dict in one expression (`{k: v for k, v in items}`) — same intent as a list comprehension but for mappings.
- **generator expression** — Uses a generator expression (parentheses, not square brackets) — the items are produced lazily. Common as the argument to `sum`, `any`, `all`, or `max` where the intermediate list would be wasted.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **ternary expression** — Uses `x if cond else y` — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Imports `csv` (lazy import inside the function).
2. Imports `io` (lazy import inside the function).
3. Lazy import from `dashboard.database`.
4. Assigns `email = (request.form.get('email') or '').strip().lower()`.
5. **if** `not email`: branches conditionally.
6. Assigns `client_info = get_client_by_email(email)`.
7. **if** `not client_info`: branches conditionally.
8. Assigns `client_id = client_info['client']`.
9. Assigns `file = request.files.get('file')`.
10. **if** `not file or not file.filename`: branches conditionally.
11. **if** `not file.filename.lower().endswith('.csv')`: branches conditionally.
12. **try** block with 1 **except** clause.
13. **if** `not rows`: branches conditionally.
14. Assigns `existing_data = get_client_data(client_id)`.
15. ... and 8 more statement(s) in the body.

Code (copy-paste safe)

```
def import_csv_companion():
    """Import CSV data via companion app. Auth via email (same as sheet migration)."""
    import csv
    import io
    from dashboard.database import get_client_data, save_client_data_with_history

    email = (request.form.get('email') or '').strip().lower()
    if not email:
        return jsonify({"status": "error", "message": "Email required"}), 400
```

```

client_info = get_client_by_email(email)
if not client_info:
    return jsonify({"status": "error", "message": "Email not registered in the system"}), 404

client_id = client_info['client']

file = request.files.get('file')
if not file or not file.filename:
    return jsonify({"status": "error", "message": "CSV file required"}), 400

if not file.filename.lower().endswith('.csv'):
    return jsonify({"status": "error", "message": "File must be a .csv"}), 400

try:
    content = file.read().decode('utf-8-sig')
    reader = csv.DictReader(io.StringIO(content))
    rows = []
    for row in reader:
        first_val = list(row.values())[0] if row else ''
        if first_val and first_val.strip().startswith('--- '):
            break
        if all(not v.strip() for v in row.values()):
            continue
        rows.append(dict(row))
except Exception as e:
    return jsonify({"status": "error", "message": f"Failed to parse CSV: {str(e)}"}), 400

if not rows:
    return jsonify({"status": "error", "message": "CSV contains no data rows"}), 400

existing_data = get_client_data(client_id)
if not existing_data:
    return jsonify({"status": "error", "message": f"No existing data for client {client_id}"}), 404

old_count = len(existing_data.get('evaluations', []))

# Full replacement: CSV becomes the new evaluations list
new_evals = []
for csv_row in rows:
    clean_row = {k: v for k, v in csv_row.items() if not k.startswith('_')}
    new_evals.append(clean_row)

existing_data['evaluations'] = new_evals
save_client_data_with_history(
    client_id, existing_data,
    changed_by=email,
    change_source='csv_import',
    change_description=f"CSV import via companion: replaced {old_count} evals with {len(new_evals)}"
)

log_action('CSV_IMPORT', 'companion', email, get_remote_address(),
    f"Imported CSV for {client_id}: replaced {old_count} with {len(new_evals)} evals")

return jsonify({
    'status': 'success',
    'message': f'Import complete: {len(new_evals)} rows imported (replaced {old_count})',
    'imported': len(new_evals),
    'previous': old_count,
    'total_rows': len(new_evals)
})

```

```
@app.route('/api/notes', methods=['POST'])
@require_session
def update_note():
```

Update or Delete a cell note.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_session** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def update_note():
    """Update or Delete a cell note."""
    try:
        session_user = request.session_user
        user_type = session_user.get('user_type')
        user_identifier = session_user.get('user_identifier')

        data = request.json
        client_id = data.get('client_id')
        row_index = data.get('row_index')
        column_key = data.get('column_key')
        content = data.get('content')

        # Clients have full notes access

        if not client_id or row_index is None or not column_key:
            return jsonify({"status": "error", "message": "Missing required fields"}), 400

        # Ensure user has access
        if not can_access_client(user_type, user_identifier, client_id):
            log_action('ACCESS_DENIED', user_type, user_identifier, get_remote_address(),
                f"Note access denied: {client_id}", False)
            return jsonify({"status": "error", "message": "Access denied"}), 403

        if user_type == 'kwok_admin':
            return jsonify({"status": "error", "message": "View-only account"}), 403

        if content:
            save_client_note(client_id, row_index, column_key, content, user_identifier)
            action = 'UPDATE_NOTE'
        else:
            # Empty content means delete
```

```

        delete_client_note(client_id, row_index, column_key)
        action = 'DELETE_NOTE'

    log_action(action, user_type, user_identifier, get_remote_address(),
               f"Note on {client_id} row {row_index} col {column_key}", True)
    return jsonify({"status": "success"})
except Exception as e:
    logging.error(f"Error updating note: {e}")
    return jsonify({"status": "error", "message": str(e)}), 500

@app.route('/api/notes/delete', methods=['POST'])
@require_session
def delete_note():

```

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_session** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.

Step by step (top-level body)

1. Assigns `data = request.json`.
2. Assigns `client_id = data.get('client_id')`.
3. Assigns `row_index = data.get('row_index')`.
4. Assigns `column_key = data.get('column_key')`.
5. Assigns `session_user = request.session_user`.
6. **if** `delete_client_note(client_id, row_index, column_key)`: branches conditionally.
7. **return** (`jsonify({'status': 'error', 'message': 'Database error'})`, 500).

Code (copy-paste safe)

```

def delete_note():
    data = request.json
    client_id = data.get('client_id')
    row_index = data.get('row_index')
    column_key = data.get('column_key')

    session_user = request.session_user

    if delete_client_note(client_id, row_index, column_key):
        return jsonify({"status": "success"})
    return jsonify({"status": "error", "message": "Database error"}), 500

@app.route('/api/update_data', methods=['POST'])
@limiter.limit('60 per minute')
def update_data():

```

Update client data - supports both session and API key authentication.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@limiter.limit** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **nested function** — Defines a function inside this function. Usually a closure that captures local variables, or a callback passed to another function.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def update_data():
    """Update client data - supports both session and API key authentication."""
    try:
        data = request.json
        identity = data.get('identity', {})

        # Try session authentication first (for dashboard UI)
        session_token = request.cookies.get('session_token')
        api_key = request.headers.get('X-API-Key')

        if session_token:
            session_info = validate_session(session_token)
            if session_info:
                user_type = session_info.get('user_type')
                user_identifier = session_info.get('user_identifier')

                # Get client_id from request data
                client_id = identity.get('client') or data.get('client_id')

                if not client_id:
                    return jsonify({"status": "error", "message": "Client ID required"}), 400

                # Check if user can access this client's data
                if not can_access_client(user_type, user_identifier, client_id):
                    log_action('UPDATE_DENIED', user_type, user_identifier, get_remote_address(),
                               f"Tried to update: {client_id}", False)
                    return jsonify({"status": "error", "message": "Access denied"}), 403

                if user_type == 'kwok_admin':
                    return jsonify({"status": "error", "message": "View-only account"}), 403

                # Get existing data to preserve fields not being updated
```

```

existing_data = get_client_data(client_id) or {}

# Get evaluations and normalize Account Size values
evaluations = data.get("evaluations", existing_data.get("evaluations", []))

# Debug: Log incoming evaluations
if evaluations and len(evaluations) > 0:
    sample_incoming = evaluations[0]
    app.logger.info(
        f"[DEBUG] Incoming Hedge Result 1: {sample_incoming.get('Hedge Result 1')}")

evaluations = normalize_evaluations(evaluations)

# Deep-merge evaluations: preserve push-sourced fields (Hedge Results, deals, etc.)
# that the stale frontend may not have received yet.
existing_evals = existing_data.get("evaluations", [])

# --- SAFETY: Prevent stale frontend from wiping evaluations ---
action_type = data.get('action_type', 'UPDATE')

if action_type == 'CREATE':
    # For "Add Account", ALWAYS use DB evaluations as the base
    # and only append genuinely new rows. This prevents a stale
    # frontend copy from overwriting push-sourced data.
    if len(evaluations) > len(existing_evals):
        now_iso = datetime.utcnow().isoformat()
        new_rows = evaluations[len(existing_evals):]
        # Mark appended rows so the quality dashboard can treat
        # them as "new / not populated yet" for initial validations.
        for _r in new_rows:
            if isinstance(_r, dict) and '_row_added_at' not in _r:
                _r['_row_added_at'] = now_iso
        evaluations = normalize_evaluations(existing_evals) + new_rows
    else:
        # Frontend has fewer or equal evals – it's stale.
        # Append a default new evaluation to the DB version.
        evaluations = normalize_evaluations(existing_evals)
        new_row = {
            "Prop Firm": "My Funded Futures",
            "Account Size": "$100,000",
            "Date Purchased": "",
            "Fee": "0"
        }
        new_row['_row_added_at'] = datetime.utcnow().isoformat()
        evaluations.append(new_row)
    existing_evals = existing_data.get("evaluations", [])
elif action_type not in ('DELETE', 'ROLLBACK'):
    # General safety check: block saves that would drop eval count
    # by more than 50% (accidental wipe protection)
    if (len(existing_evals) >= 10
        and len(evaluations) < len(existing_evals) * 0.5):
        log_action('WIPE_BLOCKED', user_type, user_identifier,
            get_remote_address(),
            f'{client_id}: incoming {len(evaluations)} evals vs '
            f'existing {len(existing_evals)} – blocked to prevent data loss',
            False)
    return jsonify({
        "status": "error",
        "message": f"Safety check: your page has {len(evaluations)} evaluations "
            f"but the database has {len(existing_evals)}."
    })

```

```

        f"Please refresh the page and try again."
    }, 409
    PUSH_SOURCED_KEYS = {
        'Hedge Result 1', 'Hedge Result 2', 'Hedge Result 3',
        'Hedge Result 4', 'Hedge Result 5',
        'Hedge Result 1.1', 'Hedge Result 2.1', 'Hedge Result 3.1',
        'Hedge Result 4.1', 'Hedge Result 5.1',
        'Hedge Result 6', 'Hedge Result 7',
    }
    # Include farming Hedge Day / Prop Day fields (push-sourced)
    for _i in range(1, 51):
        PUSH_SOURCED_KEYS.add(f'Hedge Day {_i}')
        PUSH_SOURCED_KEYS.add(f'Prop Day {_i}')
    # Payout/date fields that should only be overwritten by explicit user edits
    # (prevents a stale browser tab from reverting dashboard-entered payouts)
    PAYOUT_KEYS = {
        'Payout 1', 'Date 1', 'Payout 2', 'Date 2', 'Payout 3', 'Date 3',
        'Payout 4', 'Date 4', 'Payout 5', 'Date 5', 'Payout 6', 'Date 6',
    }
    PROTECTED_KEYS = PUSH_SOURCED_KEYS | PAYOUT_KEYS

    # Fields the user explicitly touched in this edit session
    # (sent by frontend so we can distinguish intentional clears from stale data)
    raw_changed = data.get('_changedFields', {})
    user_changed = {} # { int(eval_index): set(field_names) }
    for idx_str, fields in raw_changed.items():
        try:
            user_changed[int(idx_str)] = set(fields) if isinstance(fields, list) else set()
        except (ValueError, TypeError):
            pass

    def _has_non_blank_value(v):
        """Treat numeric 0 as a real value so protected fields are not blanked."""
        if v is None:
            return False
        if isinstance(v, str):
            return v.strip() not in ('', '-')
        return True

    for i, ev in enumerate(evaluations):
        explicitly_changed = user_changed.get(i, set())

        if i < len(existing_evals):
            existing_ev = existing_evals[i]

            # Preserve DB-only internal keys the frontend doesn't send
            for k, v in existing_ev.items():
                if k.startswith('_') and k not in ev:
                    ev[k] = v

            for key in PROTECTED_KEYS:
                # If the user explicitly cleared this field, respect the clear
                if key in explicitly_changed:
                    continue
                existing_val = existing_ev.get(key)
                incoming_val = ev.get(key)
                # Keep the existing (push-sourced) value when the frontend sends empty/missing
                if _has_non_blank_value(existing_val) and not _has_non_blank_value(incoming_val):
                    ev[key] = existing_val

    # ■■■ Track manual clears so MT5 push aggregator cannot resurrect them ■■■

```

```

# When the user explicitly blanks a push-sourced field, record it in
# `_cleared_fields`. When they later type a real value back in,
# remove the entry so push writes resume.
cleared = set(ev.get('_cleared_fields') or [])
for key in explicitly_changed:
    if key not in PUSH_SOURCED_KEYS:
        continue
    if _has_non_blank_value(ev.get(key)):
        cleared.discard(key) # user typed a value back → resume push writes
    else:
        cleared.add(key) # user blanked the cell → freeze it
if cleared:
    ev['_cleared_fields'] = sorted(cleared)
elif '_cleared_fields' in ev:
    # remove empty list to keep payload lean
    ev.pop('_cleared_fields', None)

# Recalculate Hedge Net / Hedge Net.1 from current hedge results & statuses
evaluations = recalculate_hedge_nets(evaluations)

# Recalculate statistics so they reflect latest evaluation changes
from utils.data_processor import calculate_statistics
existing_mt5 = existing_data.get('account') or data.get('account')
existing_hr_stats = existing_data.get('statistics', {}).get('hedging_review', {})
existing_hist = existing_hr_stats.get('historical_accounts')

# Debug: Log hedge values before calculation
app.logger.info(f"[DEBUG] Recalculating stats for {client_id}")
if evaluations and len(evaluations) > 0:
    sample_ev = evaluations[0]
    app.logger.info(f"[DEBUG] Sample eval Hedge Result 1: {sample_ev.get('Hedge Result 1')}")
    app.logger.info(f"[DEBUG] Sample eval Hedge Net: {sample_ev.get('Hedge Net')}")

merged_statistics = calculate_statistics(
    evaluations, mt5_account=existing_mt5,
    historical_accounts=existing_hist
)

# Debug: Log calculated stats
app.logger.info(
    f"[DEBUG] Fresh sheet_hedging_results: {merged_statistics.get('hedging_review', {})}")
app.logger.info(
    f"[DEBUG] Fresh cashflow hedging_results: {merged_statistics.get('cashflow_inprogress')}")

# Merge hedging_review: keep MT5-derived values from DB, but update sheet-derived
# values from fresh calculation (so manual hedge edits in dashboard are reflected)
existing_hr = existing_data.get('statistics', {}).get('hedging_review', {})
fresh_hr = merged_statistics.get('hedging_review', {})
if existing_hr:
    # Preserve MT5-derived fields (actual results, deposits, withdrawals, balance, historical)
    merged_hr = fresh_hr.copy()
    for key in ['actual_hedging_results', 'total_deposits', 'total_withdrawals',
                'current_balance', 'historical_accounts', 'historical_deposits',
                'historical_withdrawals', 'historical_balance']:
        # Preserve DB values even when numeric fields are 0 (truthiness bug skipped those)
        if key in existing_hr and existing_hr.get(key) is not None:
            merged_hr[key] = existing_hr[key]

# Recalculate discrepancy with fresh sheet values + preserved MT5 values
sheet_hedge = merged_hr.get('sheet_hedging_results', 0)

```

```

actual_hedge = merged_hr.get('actual_hedging_results', 0)
merged_hr['discrepancy'] = round(actual_hedge - sheet_hedge, 2)

# Debug: Log merged values
app.logger.info(
    f"[DEBUG] Merged sheet_hedge: {sheet_hedge}, actual_hedge: {actual_hedge}, discre

# Recalculate net_profit in both sections (match frontend formula)
# Frontend: payouts + hedging_results + farming_results + discrepancy - challenge_fee
disc = merged_hr['discrepancy']
for section_key in ["profitability_completed", "cashflow_inprogress"]:
    sec = merged_statistics.get(section_key, {})
    sec["net_profit"] = round(sec.get("payouts", 0) + sec.get("hedging_results", 0)
                            + sec.get("farming_results", 0) + disc - sec.get(
                                "challenge_fees", 0), 2)
    app.logger.info(f"[DEBUG] {section_key} net_profit: {sec.get('net_profit')}")

merged_statistics['hedging_review'] = merged_hr

client_data = {
    "deals": _drop_balance_deals(data.get("deals", existing_data.get("deals", [])))[0],
    "positions": data.get("positions", existing_data.get("positions", [])),
    "account": data.get("account", existing_data.get("account", {})),
    "hedge_accounts": data.get("hedge_accounts", existing_data.get("hedge_accounts", [])),
    "prop_accounts": data.get("prop_accounts", existing_data.get("prop_accounts", [])),
    "vps_accounts": data.get("vps_accounts", existing_data.get("vps_accounts", [])),
    "payment_info": data.get("payment_info", existing_data.get("payment_info", [])),
    "payment_address": data.get("payment_address", existing_data.get("payment_address", {})),
    "mt5_credentials": data.get("mt5_credentials", existing_data.get("mt5_credentials", {})),
    "firm_billing": existing_data.get("firm_billing", {}),
    "evaluations": evaluations,
    "statistics": merged_statistics,
    "dropdown_options": data.get("dropdown_options", existing_data.get("dropdown_options", {})),
    # Persist match log when updating from dashboard
    "aggregated_by_comment": existing_data.get("aggregated_by_comment", []),
    "comment_summary": existing_data.get("comment_summary", {}),
    "identity": data.get("identity", existing_data.get("identity", {}))
}

# Ensure client ID is in identity
if 'client' not in client_data['identity']:
    client_data['identity']['client'] = client_id

# Allow custom action/description from frontend
action_type = data.get('action_type', 'UPDATE')
description = data.get('change_description', f'Manual edit from dashboard by {user_type}')

# Clients are view+edit only – block add/delete of evaluations
if user_type == 'client' and action_type in ('CREATE', 'DELETE'):
    return jsonify({"status": "error",
                    "message": "Clients cannot add or delete evaluations"}), 403

# Only super_admin can delete evaluations
if action_type == 'DELETE' and user_type != 'super_admin':
    log_action('DELETE_DENIED', user_type, user_idenfier, get_remote_address(),
              f'Attempted DELETE on {client_id} without super_admin role', False)
    return jsonify({"status": "error",
                    "message": "Only super admins can delete evaluations"}), 403

# Debug: Log final stats before saving

```

```

app.logger.info(
    f"[DEBUG] Final cashflow_inprogress: {client_data.get('statistics', {}).get('cashflow_inprogress', {})}")
app.logger.info(
    f"[DEBUG] Final hedging_review discrepancy: {client_data.get('statistics', {}).get('hedging_review_discrepancy', {})}")

# Save with history tracking
success, version = save_client_data_with_history(
    client_id,
    client_data,
    action=action_type,
    changed_by=user_identifier,
    changed_by_type=user_type,
    ip_address=get_remote_address(),
    change_source='dashboard_edit',
    change_description=description
)

if success:
    log_action('DATA_UPDATE', user_type, user_identifier, get_remote_address(),
        f"Client: {client_id} (v{version})")
    return jsonify({"status": "success", "message": "Data updated", "version": version})
else:
    return jsonify({"status": "error", "message": "Failed to save data"}), 500

# Fall back to API key authentication
if api_key:
    user_info = validate_api_key(api_key)
    if user_info:
        return update_data_with_api_key(data, identity, user_info)
    else:
        return jsonify({"status": "error", "message": "Invalid API key"}), 403

return jsonify({"status": "error", "message": "Authentication required"}), 401
except Exception as e:
    print(f"Error in update_data: {e}")
    return jsonify({"status": "error", "message": f"Server error: {str(e)}"}), 500

```

```
def update_data_with_api_key(data, identity, user_info)
```

Handle update with API key authentication (for external API calls).

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. **if not identity:** branches conditionally.
2. Assigns `admin_id = identity.get('admin', 'Admin1')`.
3. Assigns `trader_id = identity.get('trader', 'Trader1')`.
4. Assigns `client_id = identity.get('client', 'Client1')`.
5. Assigns `email = identity.get('email', '')`.
6. Assigns `existing_data = get_client_data(client_id)` or `{}`.
7. Assigns `incoming_evals = data.get('evaluations')`.

8. if incoming_evals and len(incoming_evals) > 0: branches conditionally (with an **else/elif** arm).
9. Assigns evaluations = normalize_evaluations(evaluations).
10. Assigns incoming_stats = data.get('statistics', {}).
11. Assigns existing_stats = existing_data.get('statistics', {}).
12. Assigns statistics = existing_stats.copy().
13. Calls statistics.update(...) for its side effect.
14. Assigns existing_hr = existing_stats.get('hedging_review').
15. ... and 15 more statement(s) in the body.

Code (copy-paste safe)

```
def update_data_with_api_key(data, identity, user_info):
    """Handle update with API key authentication (for external API calls)."""
    # Use authenticated user info if no identity provided
    if not identity:
        identity = {
            'admin': user_info.get('admin'),
            'trader': user_info.get('trader'),
            'client': user_info.get('client')
        }

    admin_id = identity.get('admin', 'Admin1')
    trader_id = identity.get('trader', 'Trader1')
    client_id = identity.get('client', 'Client1')
    email = identity.get('email', '')

    # Get existing data to prevent overwriting evaluations with empty list
    existing_data = get_client_data(client_id) or {}

    # Smart merge for evaluations:
    # Only overwrite if incoming evaluations list is NOT empty
    incoming_evals = data.get("evaluations")
    if incoming_evals and len(incoming_evals) > 0:
        evaluations = incoming_evals
    else:
        # If incoming is empty or missing, preserve existing
        evaluations = existing_data.get("evaluations", [])

    evaluations = normalize_evaluations(evaluations)

    # Smart merge for statistics (preserve hedging_review if present in existing)
    incoming_stats = data.get("statistics", {})
    existing_stats = existing_data.get("statistics", {})

    # Start with existing stats, update with incoming
    # This preserves keys like 'hedging_review' that the trader app doesn't send
    statistics = existing_stats.copy()
    statistics.update(incoming_stats)

    # Always preserve hedging_review from DB – only /api/client/push and
    # /api/hedging_review are authoritative sources for this data.
    existing_hr = existing_stats.get('hedging_review')
    if existing_hr:
        statistics['hedging_review'] = existing_hr

    # Smart merge for dropdown_options (preserve if incoming is empty)
```

```

incoming_options = data.get("dropdown_options", {})
if not incoming_options:
    dropdown_options = existing_data.get("dropdown_options", {})
else:
    dropdown_options = incoming_options

# Merge account onto existing – never wipe MT5 totals when the API caller
# omits the field or sends zeros. Same rationale as /api/client/push.
incoming_acct = data.get("account") or {}
existing_acct = existing_data.get("account") or {}
merged_acct = dict(existing_acct)
merged_acct.update(incoming_acct)
for _preserve_key in ("total_deposits", "total_withdrawals"):
    if not incoming_acct.get(_preserve_key):
        merged_acct[_preserve_key] = existing_acct.get(_preserve_key, 0)

# Prepare client data
client_data = {
    "deals": _drop_balance_deals(data.get("deals", []))[0],
    "positions": data.get("positions", []),
    "account": merged_acct,
    "evaluations": evaluations,
    "statistics": statistics,
    "dropdown_options": dropdown_options,
    "aggregated_by_comment": existing_data.get("aggregated_by_comment", []),
    "comment_summary": existing_data.get("comment_summary", {}),
    "identity": identity
}

# Save to database WITH history tracking
success, version = save_client_data_with_history(
    client_id,
    client_data,
    action='UPDATE',
    changed_by=email or trader_id,
    changed_by_type='api',
    ip_address=get_remote_address(),
    change_source='api_push',
    change_description='Update via API'
)

# Update Hierarchy
add_admin(admin_id)
add_trader(admin_id, trader_id)
add_client(admin_id, trader_id, client_id)

log_action('DATA_UPDATE', 'trader', trader_id, get_remote_address(), f"Client: {client_id} (v{version})")
return jsonify({"status": "success", "message": "Data updated", "version": version})

@app.route('/api/admin/generate_key', methods=['POST'])
@require_admin_password
@limiter.limit('10 per hour')
def api_generate_key()

```

Generate a new API key for a trader.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_admin_password** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **@limiter.limit** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.

Step by step (top-level body)

1. Assigns `trader_info = request.json.get('trader_info', {})`.
2. Assigns `admin = trader_info.get('admin')`.
3. Assigns `trader = trader_info.get('trader')`.
4. Assigns `client = trader_info.get('client', '')`.
5. **if** not admin or not trader: branches conditionally.
6. Assigns `api_key = generate_api_key(admin, trader, client)`.
7. **if** api_key: branches conditionally.
8. **return** `(jsonify({'status': 'error', 'message': 'Failed to generate key'}),....`

Code (copy-paste safe)

```
def api_generate_key():
    """Generate a new API key for a trader."""
    trader_info = request.json.get('trader_info', {})
    admin = trader_info.get('admin')
    trader = trader_info.get('trader')
    client = trader_info.get('client', '')

    if not admin or not trader:
        return jsonify({"status": "error", "message": "Admin and trader required"}), 400

    # Generate hashed API key
    api_key = generate_api_key(admin, trader, client)

    if api_key:
        log_action('GENERATE_API_KEY', 'admin', trader, get_remote_address())
        return jsonify({
            "status": "success",
            "api_key": api_key, # Only time the full key is visible
            "trader_info": {"admin": admin, "trader": trader, "client": client}
        })

    return jsonify({"status": "error", "message": "Failed to generate key"}), 500

@app.route('/api/admin/list_keys', methods=['GET'])
@require_admin_password
def api_list_keys():
```

List all API keys (showing only prefix).

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_admin_password** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.

Step by step (top-level body)

1. Assigns `keys = list_api_keys()`.
2. Calls `log_action(...)` for its side effect.
3. **return** `jsonify({'status': 'success', 'keys': keys})`.

Code (copy-paste safe)

```
def api_list_keys():
    """List all API keys (showing only prefix)."""
    keys = list_api_keys()
    log_action('LIST_API_KEYS', 'admin', 'super_admin', get_remote_address())
    return jsonify({"status": "success", "keys": keys})

@app.route('/api/admin/revoke_key', methods=['POST'])
@require_admin_password
def api_revoke_key():
    Revoke an API key.
```

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_admin_password** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.

Step by step (top-level body)

1. Assigns `key_prefix = request.json.get('key_prefix')`.
2. **if not** `key_prefix`: branches conditionally.
3. **if** `revoke_api_key(key_prefix)`: branches conditionally.
4. **return** `(jsonify({'status': 'error', 'message': 'API key not found'}), 404)`.

Code (copy-paste safe)

```
def api_revoke_key():
    """Revoke an API key."""
    key_prefix = request.json.get('key_prefix')

    if not key_prefix:
        return jsonify({"status": "error", "message": "Key prefix required"}), 400

    if revoke_api_key(key_prefix):
        log_action('REVOKE_API_KEY', 'admin', key_prefix, get_remote_address())
        return jsonify({"status": "success", "message": "API key revoked"})

    return jsonify({"status": "error", "message": "API key not found"}), 404
```

```
@app.route('/api/admin/audit_log', methods=['GET'])
@require_admin_password
def api_audit_log()
```

Get audit log entries.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_admin_password** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.

Step by step (top-level body)

1. Assigns `limit = request.args.get('limit', 100, type=int)`.
2. Assigns `action_filter = request.args.get('action', None)`.
3. Assigns `logs = get_audit_log(limit, action_filter)`.
4. **return** `jsonify({'status': 'success', 'logs': logs})`.

Code (copy-paste safe)

```
def api_audit_log():
    """Get audit log entries."""
    limit = request.args.get('limit', 100, type=int)
    action_filter = request.args.get('action', None)

    logs = get_audit_log(limit, action_filter)
    return jsonify({"status": "success", "logs": logs})

@app.route('/api/trader/push_account', methods=['POST'])
@require_api_key
@limiter.limit('30 per minute')
def push_account_data()
```

Endpoint for traders to push account information.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_api_key** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **@limiter.limit** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `data = request.json`.
2. Assigns `client_id = data.get('client_id') or request.api_user.get('client', '....`

3. Calls `update_client_field(...)` for its side effect.
4. Calls `log_action(...)` for its side effect.
5. **return** `jsonify({'status': 'success', 'message': 'Account data updated'})`.

Code (copy-paste safe)

```
def push_account_data():
    """Endpoint for traders to push account information."""
    data = request.json
    client_id = data.get('client_id') or request.api_user.get('client', 'Client1')

    update_client_field(client_id, 'account', data.get('account', {}))
    log_action('PUSH_ACCOUNT', 'trader', request.api_user.get('trader'), get_remote_address(),
              f"Client: {client_id}")

    return jsonify({"status": "success", "message": "Account data updated"})

@app.route('/api/trader/push_positions', methods=['POST'])
@require_api_key
@limiter.limit('30 per minute')
def push_positions():
```

Endpoint for traders to push current positions.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_api_key** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **@limiter.limit** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `data = request.json`.
2. Assigns `client_id = data.get('client_id') or request.api_user.get('client', '....`
3. Calls `update_client_field(...)` for its side effect.
4. Calls `log_action(...)` for its side effect.
5. **return** `jsonify({'status': 'success', 'message': 'Positions updated'})`.

Code (copy-paste safe)

```
def push_positions():
    """Endpoint for traders to push current positions."""
    data = request.json
    client_id = data.get('client_id') or request.api_user.get('client', 'Client1')

    update_client_field(client_id, 'positions', data.get('positions', []))
    log_action('PUSH_POSITIONS', 'trader', request.api_user.get('trader'), get_remote_address(),
              f"Client: {client_id}")
```

```

        return jsonify({"status": "success", "message": "Positions updated"})

@app.route('/api/trader/push_deals', methods=['POST'])
@require_api_key
@limiter.limit('30 per minute')
def push_deals():

```

Endpoint for traders to push deal history.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_api_key** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **@limiter.limit** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `data = request.json`.
2. Assigns `client_id = data.get('client_id')` or `request.api_user.get('client', '....`
3. Assigns `deals_raw = data.get('deals', [])`.
4. Assigns `(deals, dropped_internal) = _drop_balance_deals(deals_raw)`.
5. **if** `dropped_internal`: branches conditionally.
6. Calls `update_client_field(...)` for its side effect.
7. Calls `log_action(...)` for its side effect.
8. **return** `jsonify({'status': 'success', 'message': 'Deals updated'})`.

Code (copy-paste safe)

```

def push_deals():
    """Endpoint for traders to push deal history."""
    data = request.json
    client_id = data.get('client_id') or request.api_user.get('client', 'Client1')

    deals_raw = data.get('deals', [])
    deals, dropped_internal = _drop_balance_deals(deals_raw)
    if dropped_internal:
        app.logger.info(
            f"■ Dropped {dropped_internal} internal transfer deal(s) (BALANCE/CREDIT) from /api/trader/p
        update_client_field(client_id, 'deals', deals)
        log_action('PUSH_DEALS', 'trader', request.api_user.get('trader'), get_remote_address(),
            f"Client: {client_id}")

    return jsonify({"status": "success", "message": "Deals updated"})

@app.route('/api/trader/push_evaluations', methods=['POST'])
@require_api_key

```

```
@limiter.limit('30 per minute')
def push_evaluations()
```

Endpoint for traders to push evaluation data.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_api_key** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **@limiter.limit** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.
- **ternary expression** — Uses x if cond else y — a conditional expression, not a statement. Right tool when both branches produce a value the surrounding expression consumes.

Step by step (top-level body)

1. Assigns data = request.json.
2. Assigns client_id = data.get('client_id') or request.api_user.get('client', '....
3. Assigns new_evals = data.get('evaluations', []).
4. Assigns existing_data = get_client_data(client_id) or {}.
5. Assigns existing_evals = existing_data.get('evaluations', []).
6. Assigns deleted_fingerprints = set().
7. **for** ev in existing_evals: iterates.
8. **if** deleted_fingerprints: branches conditionally.
9. Assigns force_fields = set(data.get('force_fields') or []) if isinstance(data, di....
10. Assigns merged_evals = merge_evaluation_push_with_existing(existing_evals, new_e....
11. Calls update_client_field(...) for its side effect.
12. Calls log_action(...) for its side effect.
13. **return** jsonify({'status': 'success', 'message': 'Evaluations updated'}).

Code (copy-paste safe)

```
def push_evaluations():
    """Endpoint for traders to push evaluation data."""
    data = request.json
    client_id = data.get('client_id') or request.api_user.get('client', 'Client1')

    new_evals = data.get('evaluations', [])

    # Preserve _deleted flags from existing data
    existing_data = get_client_data(client_id) or {}
```

```

existing_evals = existing_data.get('evaluations', [])
deleted_fingerprints = set()
for ev in existing_evals:
    if isinstance(ev, dict) and ev.get('_deleted'):
        acct = str(ev.get('Account #') or ev.get('Account #.1') or '').strip()
        firm = str(ev.get('Prop Firm') or '').strip()
        size = str(ev.get('Account Size') or '').strip()
        if acct:
            deleted_fingerprints.add((acct, firm, size))

if deleted_fingerprints:
    for ev in new_evals:
        if isinstance(ev, dict):
            acct = str(ev.get('Account #') or ev.get('Account #.1') or '').strip()
            firm = str(ev.get('Prop Firm') or '').strip()
            size = str(ev.get('Account Size') or '').strip()
            if acct and (acct, firm, size) in deleted_fingerprints:
                ev['_deleted'] = True

force_fields = set((data.get('force_fields') or []) if isinstance(data, dict) else [])
merged_evals = merge_evaluation_push_with_existing(
    existing_evals, new_evals, force_fields)

update_client_field(client_id, 'evaluations', merged_evals)
log_action('PUSH_EVALUATIONS', 'trader', request.api_user.get('trader'), get_remote_address(),
    f"Client: {client_id}")

return jsonify({"status": "success", "message": "Evaluations updated"})

@app.route('/api/client/history', methods=['POST'])
@limiter.limit('30 per minute')
def api_get_client_history()

```

Get the change history for a client's data. Requires client email for authentication.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@limiter.limit** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.

Step by step (top-level body)

1. Assigns data = request.json.
2. Assigns email = data.get('email', "").strip().lower().
3. Assigns limit = data.get('limit', 50).
4. **if not email**: branches conditionally.
5. Lazy import from config.hierarchy.
6. Assigns client_info = get_client_by_email(email).
7. **if not client_info**: branches conditionally.
8. Assigns client_id = client_info['client'].
9. Assigns history = get_data_history(client_id, limit).

10. `return jsonify({'status': 'success', 'client_id': client_id, 'history': hi....`

Code (copy-paste safe)

```
def api_get_client_history():
    """
    Get the change history for a client's data.
    Requires client email for authentication.
    """
    data = request.json
    email = data.get('email', '').strip().lower()
    limit = data.get('limit', 50)

    if not email:
        return jsonify({"status": "error", "message": "Email required"}), 400

    # Look up client by email
    from config.hierarchy import get_client_by_email
    client_info = get_client_by_email(email)
    if not client_info:
        return jsonify({"status": "error", "message": "Email not registered"}), 404

    client_id = client_info['client']
    history = get_data_history(client_id, limit)

    return jsonify({
        "status": "success",
        "client_id": client_id,
        "history": history,
        "total_versions": len(history)
    })

@app.route('/api/client/version', methods=['POST'])
@limiter.limit('30 per minute')
def api_get_client_version():
```

Get a specific version of client data. Useful for viewing what the data looked like at a previous point.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@limiter.limit** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `data = request.json`.
2. Assigns `email = data.get('email', "").strip().lower()`.
3. Assigns `version = data.get('version')`.
4. **if not email**: branches conditionally.
5. **if version is None**: branches conditionally.

6. Lazy import from config.hierarchy.
7. Assigns client_info = get_client_by_email(email).
8. if not client_info: branches conditionally.
9. Assigns client_id = client_info['client'].
10. Assigns version_data = get_data_version(client_id, int(version)).
11. if not version_data: branches conditionally.
12. return jsonify({'status': 'success', 'client_id': client_id, 'version_info....

Code (copy-paste safe)

```
def api_get_client_version():
    """
    Get a specific version of client data.
    Useful for viewing what the data looked like at a previous point.
    """
    data = request.json
    email = data.get('email', '').strip().lower()
    version = data.get('version')

    if not email:
        return jsonify({"status": "error", "message": "Email required"}), 400

    if version is None:
        return jsonify({"status": "error", "message": "Version number required"}), 400

    # Look up client by email
    from config.hierarchy import get_client_by_email
    client_info = get_client_by_email(email)
    if not client_info:
        return jsonify({"status": "error", "message": "Email not registered"}), 404

    client_id = client_info['client']
    version_data = get_data_version(client_id, int(version))

    if not version_data:
        return jsonify({"status": "error", "message": f"Version {version} not found"}), 404

    return jsonify({
        "status": "success",
        "client_id": client_id,
        "version_info": {
            "version": version_data['version'],
            "action": version_data['action'],
            "changed_by": version_data['changed_by'],
            "change_source": version_data['change_source'],
            "change_description": version_data['change_description'],
            "created_at": version_data['created_at']
        },
        "data": version_data['data']
    })

@app.route('/api/client/rollback', methods=['POST'])
@limiter.limit('5 per hour')
def api_rollback_client_data()
```

Rollback client data to a specific previous version. Creates a new version marking this as a rollback.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@limiter.limit** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Assigns `data = request.json`.
2. Assigns `email = data.get('email', '').strip().lower()`.
3. Assigns `version = data.get('version') or data.get('rollback_version')`.
4. **if not email**: branches conditionally.
5. **if version is None**: branches conditionally.
6. Lazy import from `config.hierarchy`.
7. Assigns `client_info = get_client_by_email(email)`.
8. **if not client_info**: branches conditionally.
9. Assigns `client_id = client_info['client']`.
10. Assigns `version_data = get_data_version(client_id, int(version))`.
11. **if not version_data**: branches conditionally.
12. Assigns `(success, new_version) = rollback_to_version(client_id, int(version), rolled_back_....`
13. **if success**: branches conditionally (with an **else/elif** arm).

Code (copy-paste safe)

```
def api_rollback_client_data():
    """
    Rollback client data to a specific previous version.
    Creates a new version marking this as a rollback.
    """
    data = request.json
    email = data.get('email', '').strip().lower()
    version = data.get('version') or data.get('rollback_version') # support both field names

    if not email:
        return jsonify({"status": "error", "message": "Email required"}), 400

    if version is None:
        return jsonify({"status": "error", "message": "Version number required"}), 400

    # Look up client by email
    from config.hierarchy import get_client_by_email
    client_info = get_client_by_email(email)
    if not client_info:
        return jsonify({"status": "error", "message": "Email not registered"}), 404
```

```

client_id = client_info['client']

# Check if version exists
version_data = get_data_version(client_id, int(version))
if not version_data:
    return jsonify({"status": "error", "message": f"Version {version} not found"}), 404

# Perform rollback
success, new_version = rollback_to_version(
    client_id,
    int(version),
    rolled_back_by=email,
    rolled_back_by_type='client',
    ip_address=get_remote_address()
)

if success:
    log_action('DATA_ROLLBACK', 'client', email, get_remote_address(),
              f"Rolled back {client_id} to version {version} (new version: {new_version})")

    return jsonify({
        "status": "success",
        "message": f"Data rolled back to version {version}",
        "client_id": client_id,
        "rolled_back_to_version": version,
        "new_version": new_version,
        "rolled_back_from_date": version_data['created_at']
    })
else:
    return jsonify({"status": "error", "message": "Failed to rollback data"}), 500

@app.route('/api/client/compare_versions', methods=['POST'])
@limiter.limit('30 per minute')
def api_compare_versions()

```

Compare two versions of client data to see what changed.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@limiter.limit** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.

Step by step (top-level body)

1. Assigns data = request.json.
2. Assigns email = data.get('email', "").strip().lower().
3. Assigns version1 = data.get('version1').
4. Assigns version2 = data.get('version2').
5. **if not email**: branches conditionally.
6. **if version1 is None or version2 is None**: branches conditionally.
7. Lazy import from config.hierarchy.

8. Assigns `client_info = get_client_by_email(email)`.
9. **if** not `client_info`: branches conditionally.
10. Assigns `client_id = client_info['client']`.
11. Assigns `comparison = compare_versions(client_id, int(version1), int(version2))`.
12. **if** not `comparison`: branches conditionally.
13. **return** `jsonify({'status': 'success', 'client_id': client_id, 'comparison':....`

Code (copy-paste safe)

```
def api_compare_versions():
    """
    Compare two versions of client data to see what changed.
    """
    data = request.json
    email = data.get('email', '').strip().lower()
    version1 = data.get('version1')
    version2 = data.get('version2')

    if not email:
        return jsonify({"status": "error", "message": "Email required"}), 400

    if version1 is None or version2 is None:
        return jsonify({"status": "error", "message": "Both version1 and version2 required"}), 400

    # Look up client by email
    from config.hierarchy import get_client_by_email
    client_info = get_client_by_email(email)
    if not client_info:
        return jsonify({"status": "error", "message": "Email not registered"}), 404

    client_id = client_info['client']

    comparison = compare_versions(client_id, int(version1), int(version2))

    if not comparison:
        return jsonify({"status": "error", "message": "One or both versions not found"}), 404

    return jsonify({
        "status": "success",
        "client_id": client_id,
        "comparison": comparison
    })

@app.route('/api/admin/all_history', methods=['GET'])
@require_admin_password
def api_get_all_history():
```

Admin endpoint to get all history across all clients.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_admin_password** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.

- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.
- **list comprehension** — Builds a list in a single expression (`[f(x) for x in xs]`). Faster than the equivalent for-loop with append, and clearer about intent: this is a transform, not a side-effecting loop.

Step by step (top-level body)

1. Lazy import from `dashboard.database`.
2. Assigns `limit = request.args.get('limit', 100, type=int)`.
3. **with** `get_connection()`: enters a context manager.
4. **return** `jsonify({'status': 'success', 'history': history, 'total_entries': ....`

Code (copy-paste safe)

```
def api_get_all_history():
    """
    Admin endpoint to get all history across all clients.
    """
    from dashboard.database import get_connection

    limit = request.args.get('limit', 100, type=int)

    with get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute('''
            SELECT id, client_id, version, action, changed_by, changed_by_type,
                   change_source, change_description, created_at
            FROM data_history
            ORDER BY created_at DESC
            LIMIT ?
        ''', (limit,))
        history = [dict(row) for row in cursor.fetchall()]

    return jsonify({
        "status": "success",
        "history": history,
        "total_entries": len(history)
    })

@app.route('/health', methods=['GET'])
@app.route('/api/health', methods=['GET'])
def health_check():
```

Simple health check endpoint.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.

Step by step (top-level body)

1. **return** `jsonify({'status': 'ok', 'timestamp': datetime.now().isoformat(), '....`

Code (copy-paste safe)

```
def health_check():
    """Simple health check endpoint."""
    return jsonify({
        "status": "ok",
        "timestamp": datetime.now().isoformat(),
        "clients_count": get_clients_count()
    })

@app.route('/api/admin/change_password', methods=['POST'])
@require_admin_password
@limiter.limit('3 per hour')
def change_admin_password():
    Change admin password.
```

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_admin_password** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **@limiter.limit** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.

Step by step (top-level body)

1. Assigns `new_password = request.json.get('new_password')`.
2. **if** not `new_password` or `len(new_password) < 8`: branches conditionally.
3. **if** `set_admin_password('super_admin', new_password)`: branches conditionally.
4. **return** `(jsonify({'status': 'error', 'message': 'Failed to change password'....`

Code (copy-paste safe)

```
def change_admin_password():
    """Change admin password."""
    new_password = request.json.get('new_password')

    if not new_password or len(new_password) < 8:
        return jsonify({"status": "error", "message": "Password must be at least 8 characters"}), 400

    if set_admin_password('super_admin', new_password):
        log_action('CHANGE_PASSWORD', 'admin', 'super_admin', get_remote_address())
        return jsonify({"status": "success", "message": "Password changed successfully"})

    return jsonify({"status": "error", "message": "Failed to change password"}), 500

@app.route('/api/sheet/stats')
@require_session
def get_stats_sheet_data():
    Fetches stats data directly from the Google Sheet.
```

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_session** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

Step by step (top-level body)

1. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def get_stats_sheet_data():
    """Fetches stats data directly from the Google Sheet."""
    try:
        data = fetch_stats_data()
        if data:
            return jsonify({"status": "success", "data": data})
        return jsonify({"status": "error", "message": "Failed to fetch stats data"}), 500
    except Exception as e:
        return jsonify({"status": "error", "message": str(e)}), 500
```

```
@app.route('/api/sheet/waterlog')
@require_session
def get_waterlog_sheet_data()
```

Returns the Profit Share History table. Priority: 1. Compute from DB (daily_watermarks + waterlog_periods) — fully offline, updates automatically as daily data is snapshotted. 2. Fall back to live Google Sheet fetch only if no period schedule is stored yet (i.e. before the first import). Query params: ?client_id=<id> — required to identify whose schedule/daily data to use ?sheet_url=<url> — fallback sheet URL for pre-import clients

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_session** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

Step by step (top-level body)

1. **try** block with 1 **except** clause.
2. **try** block with 1 **except** clause.

Code (copy-paste safe)

```
def get_waterlog_sheet_data():
    """Returns the Profit Share History table.
```

1. Compute from DB (daily_watermarks + waterlog_periods) – fully offline, updates automatically as daily data is snapshotted.
2. Fall back to live Google Sheet fetch only if no period schedule is stored yet (i.e. before the first import).

```
?client_id=<id>    - required to identify whose schedule/daily data to use
?sheet_url=<url>   - fallback sheet URL for pre-import clients
"""
```

```
from dashboard.watermark_service import compute_waterlog_from_db
except ImportError:
```

[illegible]

Save a manual profit split override for a specific period. Admin only.

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_session** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

1. Assigns `session_user = request.session_user`.

2. **if** `session_user.get('user_type')` not in ('admin', 'super_admin'): branches conditionally.
3. Assigns `data = request.get_json(silent=True)` or `{}`.
4. Assigns `client_id = data.get('client_id')`.
5. Assigns `from_date = data.get('from_date')`.
6. Assigns `amount = data.get('amount')`.
7. **if** not `client_id` or not `from_date` or `amount` is `None`: branches conditionally.
8. **try** block with 1 **except** clause.
9. **try** block with 1 **except** clause.
10. **if** `save_profit_split_override(client_id, from_date, amount)`: branches conditionally.
11. **return** (`jsonify({'status': 'error', 'message': 'Failed to save override'})`)....

Code (copy-paste safe)

```
def api_save_profit_split_override():
    """Save a manual profit split override for a specific period. Admin only."""
    session_user = request.session_user
    if session_user.get('user_type') not in ('admin', 'super_admin'):
        return jsonify({"status": "error", "message": "Admin access required"}), 403

    data = request.get_json(silent=True) or {}
    client_id = data.get('client_id')
    from_date = data.get('from_date')
    amount = data.get('amount')

    if not client_id or not from_date or amount is None:
        return jsonify({"status": "error", "message": "client_id, from_date, and amount are required"}),

    try:
        amount = float(str(amount).replace('$', '').replace(',', ''))
    except (ValueError, TypeError):
        return jsonify({"status": "error", "message": "Invalid amount"}), 400

    try:
        from dashboard.watermark_service import save_profit_split_override
    except ImportError:
        from watermark_service import save_profit_split_override

    if save_profit_split_override(client_id, from_date, amount):
        return jsonify({"status": "success", "message": "Profit split override saved"})
    return jsonify({"status": "error", "message": "Failed to save override"}), 500

@app.route('/api/client/split_pct_override', methods=['POST'])
@require_session
def api_save_split_pct_override():
```

Save a per-period split percentage override. Admin only.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.

- **@require_session** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

Step by step (top-level body)

1. Assigns `session_user = request.session_user`.
2. **if** `session_user.get('user_type') not in ('admin', 'super_admin')`: branches conditionally.
3. Assigns `data = request.get_json(silent=True) or {}`.
4. Assigns `client_id = data.get('client_id')`.
5. Assigns `from_date = data.get('from_date')`.
6. Assigns `pct = data.get('pct')`.
7. **if** `not client_id or not from_date or pct is None`: branches conditionally.
8. **try** block with 1 **except** clause.
9. **if** `pct < 0 or pct > 100`: branches conditionally.
10. **try** block with 1 **except** clause.
11. **if** `save_split_pct_override(client_id, from_date, pct)`: branches conditionally.
12. **return** (`jsonify({'status': 'error', 'message': 'Failed to save override'})`)....

Code (copy-paste safe)

```
def api_save_split_pct_override():
    """Save a per-period split percentage override. Admin only."""
    session_user = request.session_user
    if session_user.get('user_type') not in ('admin', 'super_admin'):
        return jsonify({"status": "error", "message": "Admin access required"}), 403

    data = request.get_json(silent=True) or {}
    client_id = data.get('client_id')
    from_date = data.get('from_date')
    pct = data.get('pct')

    if not client_id or not from_date or pct is None:
        return jsonify({"status": "error", "message": "client_id, from_date, and pct are required"}), 400

    try:
        pct = float(str(pct).replace('%', '').strip())
    except (ValueError, TypeError):
        return jsonify({"status": "error", "message": "Invalid percentage"}), 400

    if pct < 0 or pct > 100:
        return jsonify({"status": "error", "message": "Percentage must be between 0 and 100"}), 400

    try:
        from dashboard.watermark_service import save_split_pct_override
    except ImportError:
        from watermark_service import save_split_pct_override

    if save_split_pct_override(client_id, from_date, pct):
```

```

        return jsonify({"status": "success", "message": "Split percentage override saved"})
    return jsonify({"status": "error", "message": "Failed to save override"}), 500

```

```

@app.route('/api/client/net_profit_override', methods=['POST'])
@require_session
def api_save_net_profit_override():

```

Save a manual net profit override for a specific period. Admin only.

Why these Python concepts were used

- **@app.route** — Registers this function as a Flask URL handler. The decorator binds the URL pattern (and optional HTTP methods) to the function so Flask can dispatch incoming requests here.
- **@require_session** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **try/except** — Wraps part of the body in **try/except**. The function expects a particular failure mode (network timeout, missing file, parse error) and either recovers or returns a fallback so the caller doesn't have to handle it.

Step by step (top-level body)

1. Assigns `session_user = request.session_user`.
2. **if** `session_user.get('user_type') not in ('admin', 'super_admin')`: branches conditionally.
3. Assigns `data = request.get_json(silent=True)` or `{}`.
4. Assigns `client_id = data.get('client_id')`.
5. Assigns `from_date = data.get('from_date')`.
6. Assigns `amount = data.get('amount')`.
7. **if** not `client_id` or not `from_date` or `amount` is `None`: branches conditionally.
8. **try** block with 1 **except** clause.
9. **try** block with 1 **except** clause.
10. **if** `save_net_profit_override(client_id, from_date, amount)`: branches conditionally.
11. **return** (`jsonify({'status': 'error', 'message': 'Failed to save override'})`)....

Code (copy-paste safe)

```

def api_save_net_profit_override():
    """Save a manual net profit override for a specific period. Admin only."""
    session_user = request.session_user
    if session_user.get('user_type') not in ('admin', 'super_admin'):
        return jsonify({"status": "error", "message": "Admin access required"}), 403

    data = request.get_json(silent=True) or {}
    client_id = data.get('client_id')
    from_date = data.get('from_date')
    amount = data.get('amount')

    if not client_id or not from_date or amount is None:
        return jsonify({"status": "error", "message": "client_id, from_date, and amount are required"}),

    try:

```

```

        amount = float(str(amount).replace('$', '').replace(',', ''))
    except (ValueError, TypeError):
        return jsonify({"status": "error", "message": "Invalid amount"}), 400

    try:
        from dashboard.watermark_service import save_net_profit_override
    except ImportError:
        from watermark_service import save_net_profit_override

    if save_net_profit_override(client_id, from_date, amount):
        return jsonify({"status": "success", "message": "Net profit override saved"})
    return jsonify({"status": "error", "message": "Failed to save override"}), 500

@app.errorhandler(404)
def not_found(e)

```

Why these Python concepts were used

- **@app.errorhandler** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.

Step by step (top-level body)

1. **if** `request.accept_mimetypes.accept_json` and (not `request.accept_mimety...`: branches conditionally.
2. **return** ('Page not found', 404).

Code (copy-paste safe)

```

def not_found(e):
    if request.accept_mimetypes.accept_json and not request.accept_mimetypes.accept_html:
        return jsonify({"status": "error", "message": "Not found"}), 404
    return "Page not found", 404

@app.errorhandler(500)
def internal_server_error(e)

```

Show maintenance page on any 500 Internal Server Error.

Why these Python concepts were used

- **@app.errorhandler** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Imports logging (lazy import inside the function).
2. Calls `logging.error(...)` for its side effect.
3. **if** `request.accept_mimetypes.accept_json` and (not `request.accept_mimety...`: branches conditionally.
4. **return** (`render_template('500.html')`, 500).

Code (copy-paste safe)

```

def internal_server_error(e):

```

```

"""Show maintenance page on any 500 Internal Server Error."""
import logging
logging.error(f"500 error: {e}")
if request.accept_mimetypes.accept_json and not request.accept_mimetypes.accept_html:
    return jsonify({"status": "error", "message": "Internal server error"}), 500
return render_template('500.html'), 500

```

```

@app.errorhandler(Exception)
def unhandled_exception(e)

```

Catch-all for any unhandled exception — skip HTTP exceptions (404, 403, etc.).

Why these Python concepts were used

- **@app.errorhandler** — Decorator applied to the function — wraps it so that extra behaviour runs around each call without changing the call site.
- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.
- **isinstance check** — Branches on the runtime type of a value. Used when a function accepts more than one input shape (e.g., a string or a list of strings) and needs to handle each.

Step by step (top-level body)

1. Lazy import from `werkzeug.exceptions`.
2. `if isinstance(e, HTTPException)`: branches conditionally.
3. Imports `logging` (lazy import inside the function).
4. Calls `logging.exception(...)` for its side effect.
5. `if request.accept_mimetypes.accept_json and (not request.accept_mimety...:` branches conditionally.
6. `return (render_template('500.html'), 500)`.

Code (copy-paste safe)

```

def unhandled_exception(e):
    """Catch-all for any unhandled exception – skip HTTP exceptions (404, 403, etc.)."""
    from werkzeug.exceptions import HTTPException
    if isinstance(e, HTTPException):
        return e
    import logging
    logging.exception(f"Unhandled exception: {e}")
    if request.accept_mimetypes.accept_json and not request.accept_mimetypes.accept_html:
        return jsonify({"status": "error", "message": "An unexpected error occurred"}), 500
    return render_template('500.html'), 500

def run_dashboard()

```

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Calls `print(...)` for its side effect.

2. Calls `print(...)` for its side effect.
3. Calls `print(...)` for its side effect.
4. Calls `print(...)` for its side effect.
5. Calls `print(...)` for its side effect.
6. Calls `print(...)` for its side effect.
7. Calls `print(...)` for its side effect.
8. Calls `print(...)` for its side effect.
9. Calls `print(...)` for its side effect.
10. Calls `print(...)` for its side effect.
11. Calls `app.run(...)` for its side effect.

Code (copy-paste safe)

```
def run_dashboard():
    print(f"\n{' '*60}")
    print("SECURE DASHBOARD API SERVER STARTING")
    print(f"{' '*60}")
    print(f"Database: PostgreSQL (Alembic-managed schema)")
    print(f"Rate Limiting: Enabled")
    print(f>Password Hashing: PBKDF2-SHA256 (100,000 iterations)")
    print(f"API Keys: Hashed with SHA-256")
    print(f"Audit Logging: Enabled")
    print(f"\nClients in database: {get_clients_count()}")
    print(f"{' '*60}\n")
    app.run(host='0.0.0.0', port=5001, debug=True)
```

Checkpoint — what works now. Run the Flask app (`flask --app dashboard.app run`), open <http://localhost:5000>, and you have a working web dashboard. Log in, browse accounts, view payouts and the financial overview. The scheduler is running periodic jobs in the background.

Chapter 12 — Phase 10 — Deployment

Last phase. We add the entry points the production server (Gunicorn) calls, a CLI for managing users, a wrapper around Alembic migrations, the script that bundles the dashboard for deployment to PythonAnywhere, and the PyInstaller build script that packages the desktop companion into a Windows executable.

Files we build in this phase, in order:

- wsgi.py
- gunicorn.conf.py
- build.py

Python concepts you'll meet in this phase

- f-strings (3) · Exceptions (2) · Context managers (with-statements) (2) · Decorators (1) · Flask routes (1) · Comprehensions (1)

Each is explained in Chapter 2 (the primer). The number in parentheses is how many of this phase's files use that concept.

Step 12.1 — Build wsgi.py

What to do. Create the file wsgi.py in your project root and paste in the code shown in this step. The file is **109** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from requirements.txt.

What this file is for. The WSGI entry point. Gunicorn imports the application object from this module in production. Keep it small — its only job is to construct the app and expose it under a stable name.

wsgi.py

109 loc · 0 classes · 2 functions · 2 imports

The WSGI entry point. Gunicorn imports the application object from this module in production. Keep it small — its only job is to construct the app and expose it under a stable name. It defines **2** module-level functions, across **109** lines. Module docstring: *WSGI Entry Point for MT5 Hedging Dashboard*

Module docstring

WSGI Entry Point for MT5 Hedging Dashboard =====
This is the entry point for production WSGI servers like Gunicorn.

Usage: gunicorn wsgi:app gunicorn -c gunicorn.conf.py wsgi:app

For development: `python wsgi.py`

Python concepts demonstrated in this module

• Comprehensions • Context managers (with-statements) • Decorators • Exceptions • Flask routes • f-strings

Imports — what each one is for

```
import os
```

Why imported. Operating-system interface. Path manipulation, environment variables, working-directory operations, exit codes, and thin wrappers around POSIX/Windows syscalls.

Used in this file. `os.environ`, `os.getenv`, `os.path`, `os.path.abspath`, `os.path.dirname`, `os.path.join`

Example. `os.path.join(root, 'subdir', 'file.txt')` # joins with the OS-correct separator

Other common scenarios.

- `os.environ.get('API_KEY')` to read environment variables
- `os.makedirs(path, exist_ok=True)` to create nested directories
- `os.listdir(path)` to list a directory's contents
- `os.remove(path)` / `os.rename(src, dst)` to delete or move a file
- `os.getcwd()` / `os.chdir(path)` to inspect or change the working dir

```
import sys
```

Why imported. Access to the running interpreter — `argv`, `stdin/stdout`, exit codes, the import search path, and the platform name.

Used in this file. `sys.path`, `sys.path.insert`

Example. `sys.exit(1)` # terminate the process with a non-zero status

Other common scenarios.

- `sys.argv[1:]` reads the command-line arguments
- `sys.path.insert(0, 'extra')` prepends to the import search path
- `sys.stderr.write(msg)` writes to standard error
- `sys.platform` tells you 'win32', 'linux', or 'darwin'
- `sys.modules` is the global cache of already-imported modules

Functions

```
def _maintenance_fallback(envron, start_response)
```

Static WSGI app returned when Flask itself fails to load.

Step by step (top-level body)

1. Assigns `_html = open(os.path.join(os.path.dirname(os.path.abspath(__file__)), ...`

2. Assigns status = '500 Internal Server Error'.
3. Assigns headers = [('Content-Type', 'text/html; charset=utf-8'), ('Content-....
4. Calls start_response(...) for its side effect.
5. **return** [_html].

Code (copy-paste safe)

```
def _maintenance_fallback(envron, start_response):
    """Static WSGI app returned when Flask itself fails to load."""
    _html = open(
        os.path.join(os.path.dirname(os.path.abspath(__file__)),
                     'dashboard', 'templates', '500.html'),
        'rb'
    ).read()
    status = '500 Internal Server Error'
    headers = [('Content-Type', 'text/html; charset=utf-8'),
               ('Content-Length', str(len(_html)))]
    start_response(status, headers)
    return [_html]
```

```
def configure_app(app)
```

Apply production-specific configurations.

Step by step (top-level body)

1. Lazy import from config.production.
2. Assigns config = get_config(os.getenv('FLASK_ENV', 'production')).
3. Assigns app.config['SECRET_KEY'] = config.SECRET_KEY.
4. Assigns app.config['DEBUG'] = config.DEBUG.
5. Assigns app.config['TESTING'] = config.TESTING.
6. Assigns app.config['SESSION_COOKIE_SECURE'] = config.SESSION_COOKIE_SECURE.
7. Assigns app.config['SESSION_COOKIE_HTTPONLY'] = config.SESSION_COOKIE_HTTPONLY.
8. Assigns app.config['SESSION_COOKIE_SAMESITE'] = config.SESSION_COOKIE_SAMESITE.
9. Assigns app.config['PERMANENT_SESSION_LIFETIME'] = config.PERMANENT_SESSION_LIFETIME.
10. Assigns app.config['MAX_CONTENT_LENGTH'] = config.MAX_CONTENT_LENGTH.
11. **return** app.

Code (copy-paste safe)

```
def configure_app(app):
    """Apply production-specific configurations."""
    from config.production import get_config

    config = get_config(os.getenv('FLASK_ENV', 'production'))

    # Apply configuration to app
    app.config['SECRET_KEY'] = config.SECRET_KEY
    app.config['DEBUG'] = config.DEBUG
```

```

app.config['TESTING'] = config.TESTING

# Session configuration
app.config['SESSION_COOKIE_SECURE'] = config.SESSION_COOKIE_SECURE
app.config['SESSION_COOKIE_HTTPONLY'] = config.SESSION_COOKIE_HTTPONLY
app.config['SESSION_COOKIE_SAMESITE'] = config.SESSION_COOKIE_SAMESITE
app.config['PERMANENT_SESSION_LIFETIME'] = config.PERMANENT_SESSION_LIFETIME

# File size limits
app.config['MAX_CONTENT_LENGTH'] = config.MAX_CONTENT_LENGTH

return app

```

Step 12.2 — Build gunicorn.conf.py

What to do. Create the file `gunicorn.conf.py` in your project root and paste in the code shown in this step. The file is **173** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from `requirements.txt`.

What this file is for. Gunicorn configuration. Worker counts, bind address, timeouts, and logging are read from variables at module scope.

`gunicorn.conf.py`

173 loc · 0 classes · 11 functions · 2 imports

Gunicorn configuration. Worker counts, bind address, timeouts, and logging are read from variables at module scope. It defines **11** module-level functions, across **173** lines. Module docstring: *Gunicorn Configuration for MT5 Hedging Dashboard*

Module docstring

```

Gunicorn Configuration for MT5 Hedging Dashboard
===== Production WSGI server configuration.

Usage: gunicorn -c gunicorn.conf.py wsgi:app

Or with custom settings: gunicorn --bind 0.0.0.0:8000 --workers 4 wsgi:app

```

Python concepts demonstrated in this module

- f-strings

Imports — what each one is for

```
import os
```

Why imported. Operating-system interface. Path manipulation, environment variables, working-directory operations, exit codes, and thin wrappers around POSIX/Windows syscalls.

Used in this file. `os.getenv`, `os.path`, `os.path.abspath`, `os.path.dirname`

Example. `os.path.join(root, 'subdir', 'file.txt')` # joins with the OS-correct separator

Other common scenarios.

- `os.environ.get('API_KEY')` to read environment variables
- `os.makedirs(path, exist_ok=True)` to create nested directories
- `os.listdir(path)` to list a directory's contents
- `os.remove(path)` / `os.rename(src, dst)` to delete or move a file
- `os.getcwd()` / `os.chdir(path)` to inspect or change the working dir

```
import multiprocessing
```

Why imported. *multiprocessing* — module specific to this codebase. See the chapter that covers its source for the full definition.

Used in this file. `multiprocessing.cpu_count`

Functions

```
def on_starting(server)
```

Called just before the master process is initialized.

Step by step (top-level body)

1. Calls `print(...)` for its side effect.

Code (copy-paste safe)

```
def on_starting(server):
    """Called just before the master process is initialized."""
    print("■ Starting MT5 Dashboard Gunicorn server...")
```

```
def on_reload(server)
```

Called before reloading the configuration.

Step by step (top-level body)

1. Calls `print(...)` for its side effect.

Code (copy-paste safe)

```
def on_reload(server):
    """Called before reloading the configuration."""
    print("■ Reloading MT5 Dashboard server configuration...")
```

```
def when_ready(server)
```

Called just after the server is started.

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Calls `print(...)` for its side effect.
2. Calls `print(...)` for its side effect.
3. Calls `print(...)` for its side effect.

Code (copy-paste safe)

```
def when_ready(server):
    """Called just after the server is started."""
    print(f"■ MT5 Dashboard server ready at {bind}")
    print(f"  Workers: {workers}")
    print(f"  Worker class: {worker_class}")
```

```
def worker_int(worker)
```

Called when a worker receives SIGINT or SIGQUIT.

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Calls `print(...)` for its side effect.

Code (copy-paste safe)

```
def worker_int(worker):
    """Called when a worker receives SIGINT or SIGQUIT."""
    print(f"■■ Worker {worker.pid} received interrupt signal")
```

```
def worker_abort(worker)
```

Called when a worker receives SIGABRT.

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Calls `print(...)` for its side effect.

Code (copy-paste safe)

```
def worker_abort(worker):
    """Called when a worker receives SIGABRT."""
    print(f"■ Worker {worker.pid} was aborted")
```

```
def pre_fork(server, worker)
```

Called just before a worker is forked.

Step by step (top-level body)

1. `pass` (placeholder).

Code (copy-paste safe)

```
def pre_fork(server, worker):  
    """Called just before a worker is forked."""  
    pass
```

```
def post_fork(server, worker)
```

Called just after a worker has been forked.

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Calls `print(...)` for its side effect.

Code (copy-paste safe)

```
def post_fork(server, worker):  
    """Called just after a worker has been forked."""  
    print(f"■ Worker {worker.pid} spawned")
```

```
def post_worker_init(worker)
```

Called just after a worker has initialized the application.

Step by step (top-level body)

1. `pass` (placeholder).

Code (copy-paste safe)

```
def post_worker_init(worker):  
    """Called just after a worker has initialized the application."""  
    pass
```

```
def worker_exit(server, worker)
```

Called just after a worker has been exited.

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Calls `print(...)` for its side effect.

Code (copy-paste safe)

```
def worker_exit(server, worker):  
    """Called just after a worker has been exited."""  
    print(f"■ Worker {worker.pid} exited")
```

```
def nworkers_changed(server, new_value, old_value)
```

Called when the number of workers has been changed.

Why these Python concepts were used

- **f-string** — Builds strings with **f-strings**. Compiled at parse time — the fastest formatting option, and the format spec after the colon controls width, precision, and alignment.

Step by step (top-level body)

1. Calls `print(...)` for its side effect.

Code (copy-paste safe)

```
def nworkers_changed(server, new_value, old_value):
    """Called when the number of workers has been changed."""
    print(f"■ Workers changed: {old_value} → {new_value}")
```

```
def on_exit(server)
```

Called just before exiting Gunicorn.

Step by step (top-level body)

1. Calls `print(...)` for its side effect.

Code (copy-paste safe)

```
def on_exit(server):
    """Called just before exiting Gunicorn."""
    print("■ MT5 Dashboard server shutting down...")
```

Step 12.3 — Build build.py

What to do. Create the file `build.py` in your project root and paste in the code shown in this step. The file is **87** lines long; we walk through it piece by piece below.

Depends on. No internal imports — this file is self-contained and only relies on the standard library plus packages from `requirements.txt`.

What this file is for. PyInstaller-driven build script. Packages the desktop trader companion into a Windows executable using one of the `.spec` files in the repo root.

build.py

87 loc · 0 classes · 1 functions · 4 imports

PyInstaller-driven build script. Packages the desktop trader companion into a Windows executable using one of the `.spec` files in the repo root. It defines **1** module-level function, across **87** lines.

Python concepts demonstrated in this module

- Context managers (with-statements)
- Exceptions
- f-strings

Imports — what each one is for

```
import os
```

Why imported. Operating-system interface. Path manipulation, environment variables, working-directory operations, exit codes, and thin wrappers around POSIX/Windows syscalls.

Used in this file. `os.path`, `os.path.abspath`, `os.path.dirname`, `os.path.exists`, `os.path.isdir`, `os.path.join`, `os.pathsep`

Example. `os.path.join(root, 'subdir', 'file.txt')` # joins with the OS-correct separator

Other common scenarios.

- `os.environ.get('API_KEY')` to read environment variables
- `os.makedirs(path, exist_ok=True)` to create nested directories
- `os.listdir(path)` to list a directory's contents
- `os.remove(path)` / `os.rename(src, dst)` to delete or move a file
- `os.getcwd()` / `os.chdir(path)` to inspect or change the working dir

```
import sys
```

Why imported. Access to the running interpreter — `argv`, `stdin/stdout`, exit codes, the import search path, and the platform name.

Used in this file. `sys.executable`, `sys.exit`

Example. `sys.exit(1)` # terminate the process with a non-zero status

Other common scenarios.

- `sys.argv[1:]` reads the command-line arguments
- `sys.path.insert(0, 'extra')` prepends to the import search path
- `sys.stderr.write(msg)` writes to standard error
- `sys.platform` tells you 'win32', 'linux', or 'darwin'
- `sys.modules` is the global cache of already-imported modules

```
import shutil
```

Why imported. High-level filesystem operations: copying trees, moving files across filesystems, deleting directories, locating executables on `PATH`.

Used in this file. `shutil.rmtree`

Example. `shutil.copytree(src, dst)` # recursively copies a tree

Other common scenarios.

- `shutil.move(src, dst)` moves or renames

- `shutil.rmtree(path)` deletes a directory and all its contents
- `shutil.which('git')` finds an executable on PATH
- `shutil.disk_usage(path)` returns total / used / free bytes

`import subprocess`

Why imported. Run external programs. The standard way to spawn a child process, capture its output, and wait for its exit code.

Used in this file. `subprocess.CalledProcessError`, `subprocess.run`

Example. `subprocess.run(['git', 'rev-parse', 'HEAD'], capture_output=True, text=True, check=True)`

Other common scenarios.

- `Popen(...)` for streaming I/O while the process is alive
- `check_output(cmd)` returns stdout as text and raises on failure
- `DEVNULL` / `PIPE` redirect stdin/stdout/stderr
- `shell=True` interprets the string with the system shell (security risk)

Module constants

Each line below is followed by a plain-English note explaining what it does and why it is written that way.

`PROJECT_ROOT = os.path.dirname(os.path.abspath(__file__))`

Filesystem path resolved relative to the source file at import time — portable across machines.

`TRADER_APP = os.path.join(PROJECT_ROOT, 'trader_companion', 'trader_app.py')`

Module-level constant — bound once at import time and referenced from the functions and classes below.

`LOGO_SRC = os.path.join(PROJECT_ROOT, 'trader_companion', 'logo.png')`

Module-level constant — bound once at import time and referenced from the functions and classes below.

`UTILS_SRC = os.path.join(PROJECT_ROOT, 'utils')`

Module-level constant — bound once at import time and referenced from the functions and classes below.

`TRADER_COMPANION_DIR = os.path.join(PROJECT_ROOT, 'trader_companion')`

Module-level constant — bound once at import time and referenced from the functions and classes below.

`DIST_DIR = os.path.join(PROJECT_ROOT, 'dist')`

Module-level constant — bound once at import time and referenced from the functions and classes below.

`BUILD_DIR = os.path.join(PROJECT_ROOT, 'build')`

Module-level constant — bound once at import time and referenced from the functions and classes below.

```
VERSION = get_version()
```

Module-level constant — bound once at import time and referenced from the functions and classes below.

```
BUILD_NAME = f'TradeopssAI_v{VERSION}'
```

Module-level constant — bound once at import time and referenced from the functions and classes below.

Functions

```
def get_version()
```

Extract version from trader_app.py

Why these Python concepts were used

- **with-statement** — Uses a **with** block, so the resource (file, lock, transaction, MT5 session) is released even if the body raises. Avoids the easy bug of a missed close on the error path.

Step by step (top-level body)

1. **with** open(TRADER_APP, 'r', encoding='utf-8'): enters a context manager.

2. **return** '1.0.0'.

Code (copy-paste safe)

```
def get_version():
    """Extract version from trader_app.py"""
    with open(TRADER_APP, 'r', encoding='utf-8') as f:
        for line in f:
            if line.strip().startswith('APP_VERSION'):
                return line.split('=')[1].strip().replace("'", '').replace('"', '')
    return '1.0.0'
```

Checkpoint — what works now. The project ships. `gunicorn -c gunicorn.conf.py wsgi:app` runs the dashboard in production-mode workers. `python build.py` packages the desktop companion into a Windows .exe. `python prepare_deployment.py` bundles the dashboard for upload to PythonAnywhere. The book ends here — you have built the platform.

Chapter 13 — The frontend: templates, CSS, and the JavaScript glue

Up to this point the book has been about Python — the request router, the database layer, the prop-firm integrations, the trader desktop app. This chapter is about the layer the user actually sees: Flask templates, the CSS that styles them, and the JavaScript embedded inside them. None of it is in a separate single-page-app: every page is server-rendered Jinja2, with progressive enhancement bolted on via inline `<script>` blocks.

That is a deliberate, slightly old-school choice. It means the dashboard works without a build step, a developer can read a page top to bottom without bouncing between framework files, and the same template that renders the page can also be the file you `grep` when something looks wrong. It costs us the SPA niceties (instant route changes, shared client state), but for a back-office dashboard the trade is fair.

How Flask templating works

Flask uses **Jinja2** as its templating engine. Three mustache-flavoured constructs do the heavy lifting:

- `{{ expression }}` — an *expression*: evaluate the Python expression inside the double braces and substitute the (HTML-escaped) result. This is how you print a variable.
- `{% statement %}` — a *statement*: control flow. `{% if user.is_admin %} ... {% endif %}`, `{% for client in clients %} ... {% endfor %}`, `{% extends 'base.html' %}`, `{% block title %}{% endblock %}`, `{% include 'sidebar.html' %}`.
- `{# comment #}` — a Jinja comment that is stripped before the HTML reaches the browser (unlike `<!-- ... -->`, which is sent and visible to anyone using View Source).

Inheritance is the trick that keeps things DRY. A parent template lays out the shell (header, nav, footer) and declares named `{% block %}` regions. Child templates do `{% extends 'parent.html' %}` and override only the blocks they care about. Read every *Inherits from* note below as: this child page reuses the parent's shell and only customises the named blocks.

Two helpers you will see referenced over and over:

- `url_for('endpoint_name')` — resolve a named Flask route into an actual URL at render time. Using this instead of hard-coding `/login` means moving a route is a one-line change in `app.py`, not a project-wide find-and-replace.
- `csrf_token()` — emits the CSRF token for forms protected by Flask-WTF. The server rejects POSTs where the token is missing or wrong; this is what blocks third-party sites from forging requests on behalf of a logged-in user.

All templates at a glance

Sorted by size. Use this as the chapter's table of contents — the bigger the template, the more of the user's actual experience lives inside it.

Template	LOC	Bytes	Forms	Scripts	Routes
index.html	8,748	477,009	0	1	1
super_admin.html	2,721	159,768	0	1	1
quality_dashboard.html	1,731	111,915	0	1	1
financial_overview.html	1,177	64,296	1	13	2
trader_dashboard.html	973	49,503	0	1	1
admin_dashboard.html	745	44,155	0	1	1
hierarchy.html	729	26,903	0	1	0
login.html	610	22,728	0	1	1
payout_history.html	554	27,968	1	4	2
client_performance.html	551	27,279	0	1	1
trader_performance.html	418	22,962	0	1	1
change_password.html	397	14,304	0	1	0
client_management.html	348	15,286	0	1	1
maintenance.html	259	8,388	0	1	0
500.html	250	8,312	0	0	0

Public & auth pages — what an unauthenticated visitor sees

These four templates are the only ones an unauthenticated or error-state user can reach. They are intentionally small and self-contained: when the rest of the app is broken, these are still expected to render.

dashboard/templates/login.html

610 loc · 22,728 bytes · 2 {{ ... }} expressions · 0 for loops · 0 if blocks

The sign-in screen. Reads email + password, posts to the login route, then redirects based on the user's role (admin / trader / client). Also renders password-reset prompts when applicable.

Standalone template — does not extend a parent. The full HTML document lives inside this single file.

Routes referenced via `url_for(...)`. static. Each entry is a Flask endpoint name — search for the matching `@app.route` or `@bp.route` decorator in `dashboard/app.py`.

UI libraries detected. Google Fonts. Loaded via either a `<script>` tag or a local static asset.

AJAX patterns detected. `fetch()`. The page is not purely server-rendered: parts of it talk back to Flask routes from the browser after the initial load.

Notes. 1 inline `<script>` block · 1 inline `<style>` block.

Opening of the template:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Login - TradeOpss</title>
  <style>
    @import url(
      'https://fonts.googleapis.com/css2?family=Plus+Jakarta+Sans:wght@400;500;600;700;800;900&family=Space+Grotesk'
    );

    * { margin: 0; padding: 0; box-sizing: border-box; }

    body {
      font-family: 'Plus Jakarta Sans', 'Space Grotesk', -apple-system, sans-serif;
      height: 100vh;
      display: flex;
      flex-direction: column;
      align-items: center;
      justify-content: center;
      background-color: #01030a;
      background-image:
        radial-gradient(circle at 15% 50%, rgba(251,191,36,0.07) 0%, transparent 30%),
        radial-gradient(circle at 85% 20%, rgba(30,58,138,0.22) 0%, transparent 40%),
        radial-gradient(circle at 50% 100%, rgba(10,15,40,0.4) 0%, transparent 55%);
      background-attachment: fixed;
      color: #e8eeff;
      overflow: hidden;
      padding: 20px;
      position: relative;
    }
  </style>
</head>
```

dashboard/templates/change_password.html

397 loc · 14,304 bytes · 1 {{ ... }} expressions · 0 for loops · 0 if blocks

Forced password change after first login or admin reset. POSTs the new password and runs strength validation client-side before the server enforces it again.

Standalone template — does not extend a parent. The full HTML document lives inside this single file.

AJAX patterns detected. `fetch()`. The page is not purely server-rendered: parts of it talk back to Flask routes from the browser after the initial load.

Notes. 1 inline `<script>` block · 1 inline `<style>` block.

Opening of the template:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Change Password - Trading Dashboard</title>
  <style>
    * { margin: 0; padding: 0; box-sizing: border-box; }

    body {
      font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, Oxygen, Ubuntu, sans-serif;
      background: linear-gradient(135deg, #1a1a2e 0%, #16213e 50%, #0f3460 100%);
      min-height: 100vh;
      display: flex;
      justify-content: center;
      align-items: center;
      padding: 20px;
    }

    .container {
      background: rgba(255, 255, 255, 0.95);
      border-radius: 20px;
      box-shadow: 0 25px 50px rgba(0, 0, 0, 0.3);
      width: 100%;
      max-width: 420px;
      overflow: hidden;
    }

    .header {
      background: linear-gradient(135deg, #f59e0b 0%, #d97706 100%);
```

dashboard/templates/maintenance.html

259 loc · 8,388 bytes · 0 {{ ... }} expressions · 0 for loops · 0 if blocks

Maintenance-mode shield page. Served when the app is intentionally taken offline (during deploys, schema changes, or incident response).

Standalone template — does not extend a parent. The full HTML document lives inside this single file.

UI libraries detected. Google Fonts. Loaded via either a CDN `<script>` tag or a local static asset.

AJAX patterns detected. `fetch()`. The page is not purely server-rendered: parts of it talk back to Flask routes from the browser after the initial load.

Notes. 1 inline `<script>` block · 1 inline `<style>` block.

Opening of the template:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Under Maintenance - TradeOps</title>
  <style>
    @import url(
      'https://fonts.googleapis.com/css2?family=Plus+Jakarta+Sans:wght@400;500;600;700;800;900&fam

    * { margin: 0; padding: 0; box-sizing: border-box; }

    body {
      font-family: 'Plus Jakarta Sans', 'Space Grotesk', -apple-system, sans-serif;
      height: 100vh;
      overflow: hidden;
      display: flex;
      align-items: center;
      justify-content: center;
      background-color: #01030a;
      background-image:
        radial-gradient(circle at 15% 50%, rgba(251,191,36,0.07) 0%, transparent 30%),
        radial-gradient(circle at 85% 20%, rgba(30,58,138,0.22) 0%, transparent 40%),
        radial-gradient(circle at 50% 100%, rgba(10,15,40,0.4) 0%, transparent 55%);
      background-attachment: fixed;
      color: #e8eeff;
      position: relative;
      padding: 20px;
    }

    body::before {
```

dashboard/templates/500.html

250 loc · 8,312 bytes · 0 {{ ... }} expressions · 0 for loops · 0 if blocks

Error page rendered for unhandled server errors. Deliberately minimal: when the rest of the app is on fire, this template must not require anything of it.

Standalone template — does not extend a parent. The full HTML document lives inside this single file.

UI libraries detected. Google Fonts. Loaded via either a CDN `<script>` tag or a local static asset.

Notes. 1 inline `<style>` block.

Opening of the template:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Under Maintenance - TradeOps</title>
```

```

<style>
  @import url(
    'https://fonts.googleapis.com/css2?family=Plus+Jakarta+Sans:wght@400;500;600;700;800;900&fam

  * { margin: 0; padding: 0; box-sizing: border-box; }

  body {
    font-family: 'Plus Jakarta Sans', 'Space Grotesk', -apple-system, sans-serif;
    height: 100vh;
    overflow: hidden;
    display: flex;
    align-items: center;
    justify-content: center;
    background-color: #01030a;
    background-image:
      radial-gradient(circle at 15% 50%, rgba(251,191,36,0.07) 0%, transparent 30%),
      radial-gradient(circle at 85% 20%, rgba(30,58,138,0.22) 0%, transparent 40%),
      radial-gradient(circle at 50% 100%, rgba(10,15,40,0.4) 0%, transparent 55%);
    background-attachment: fixed;
    color: #e8eeff;
    position: relative;
    padding: 20px;
  }

  body::before {

```

Admin & super-admin views — the operations layer

These are the templates that the people running the desk spend their day inside. Heavier on tables, modals, and click-to-edit forms than on visualisations.

dashboard/templates/admin_dashboard.html

745 loc · 44,155 bytes · 5 {{ ... }} expressions · 0 for loops · 0 if blocks

Top-level admin landing page. Shows system-wide KPIs, the user hierarchy, and shortcuts to management tools. Lighter than super_admin.html — it is for day-to-day operators.

Standalone template — does not extend a parent. The full HTML document lives inside this single file.

Routes referenced via url_for(...). static. Each entry is a Flask endpoint name — search for the matching @app.route or @bp.route decorator in dashboard/app.py.

UI libraries detected. Font Awesome, Google Fonts. Loaded via either a CDN <script> tag or a local static asset.

AJAX patterns detected. fetch(). The page is not purely server-rendered: parts of it talk back to Flask routes from the browser after the initial load.

Notes. 1 inline <script> block · 1 inline <style> block.

Opening of the template:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Admin Dashboard - TradeOpss</title>
  <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
  <link href="https://cdn.jsdelivr.net/npm/font-awesome@6.0.0/css/all.min.css" rel="stylesheet">
  <link href="https://fonts.googleapis.com/css2?family=Space+Grotesk:wght@400;500;600;700&family=IBM+Plex+Sans:ital,wght@0,400;0,500;0,600;0,700;1,400;1,500;1,600;1,700" rel="stylesheet">
  <style>
    /* ■■■ NAVBAR ■■■ */
    .modern-nav {
      background: #07091a;
      border: 1px solid #162040;
      border-top: 1px solid #1e3060;
      border-radius: 14px;
      padding: 0 24px;
      height: 68px;
      display: flex;
      justify-content: center;
      align-items: center;
      margin-bottom: 20px;
      box-shadow: 0 8px 28px rgba(1,3,10,0.8), 0 1px 0 rgba(30,58,138,0.22);
      position: sticky;
      top: 12px;
      z-index: 500;
    }
    .nav-links { display: flex; gap: 4px; align-items: center; }
    .tab-btn {
      color: #7898be; text-decoration: none; font-weight: 600; font-size: 0.78rem;
    }
  </style>
</head>
```


dashboard/templates/super_admin.html

2,721 loc · 159,768 bytes · 4 {{ ... }} expressions · 0 for loops · 11 if blocks

Super-admin console. Adds operations (user CRUD, system-level toggles, deeper audit views) that ordinary admins cannot run. This is the second-largest template in the project.

Standalone template — does not extend a parent. The full HTML document lives inside this single file.

Routes referenced via `url_for(...)`. static. Each entry is a Flask endpoint name — search for the matching `@app.route` or `@bp.route` decorator in `dashboard/app.py`.

UI libraries detected. Font Awesome, Google Fonts. Loaded via either a CDN `<script>` tag or a local static asset.

AJAX patterns detected. `fetch()`. The page is not purely server-rendered: parts of it talk back to Flask routes from the browser after the initial load.

Notes. 1 inline `<script>` block · 1 inline `<style>` block.

Opening of the template:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>{% if user_role|default('super_admin') == 'bef_admin' %}BEF Admin{% elif user_role|default(
    'super_admin') == 'kwok_admin' %}Kwok Admin{% else %}Super Admin{% endif %} - TradeOpss</title>
  <link rel="icon" href="{{ url_for('static', filename='images/logo.png') }}" type="image/png">
  <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
  <link href="https://cdn.jsdelivr.net/npm/font-awesome/6.0.0/css/all.min.css" rel="stylesheet">
  <link href="https://fonts.googleapis.com/css2?family=Space+Grotesk:wght@400;500;600;700&family=IBM+P
  <style>
    /* ■■■ COMPONENT STYLES ■■■ */
    .admin-card {
      background: var(--card-bg);
      border: 1px solid var(--border-color);
      border-top: 1px solid #1e3060;
      border-radius: var(--radius);
      margin-bottom: 24px;
      padding: 2px;
      position: relative;
      overflow: hidden;
      box-shadow: var(--shadow-navy);
    }

    .admin-header {
      background: var(--gradient-card);
      padding: 16px 20px;
      border-radius: var(--radius) var(--radius) 0 0;
      display: flex;
      justify-content: space-between;
```

dashboard/templates/hierarchy.html

729 loc · 26,903 bytes · 0 {{ ... }} expressions · 0 for loops · 0 if blocks

The visual Admin → Trader → Client tree, with edit handles to move users between branches. Edits POST back to the hierarchy endpoints in dashboard/app.py.

Standalone template — does not extend a parent. The full HTML document lives inside this single file.

UI libraries detected. Google Fonts. Loaded via either a CDN <script> tag or a local static asset.

AJAX patterns detected. fetch(). The page is not purely server-rendered: parts of it talk back to Flask routes from the browser after the initial load.

Notes. 1 inline <script> block · 1 inline <style> block.

Opening of the template:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>System Overview - TradeOps</title>
  <style>
    @import url('https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700&display=swap');

    :root {
      --bg: #040816;
      --bg-soft: #0b1328;
      --panel: rgba(10, 20, 44, 0.9);
      --panel-strong: rgba(17, 31, 64, 0.96);
      --border: rgba(115, 144, 255, 0.22);
      --border-strong: rgba(115, 144, 255, 0.4);
      --text-primary: #f8fbff;
      --text-secondary: #96a6cf;
      --accent: #6ea8ff;
      --accent-strong: #8b5cf6;
      --success: #10b981;
      --warning: #f59e0b;
      --danger: #ef4444;
      --shadow: 0 22px 60px rgba(0, 0, 0, 0.4);
      --font-family: 'Inter', sans-serif;
    }

    * {
      box-sizing: border-box;
    }
```

dashboard/templates/client_management.html

348 loc · 15,286 bytes · 2 {{ ... }} expressions · 0 for loops · 3 if blocks

Per-trader client roster and CRUD operations: add a client, deactivate, transfer, edit the metadata that follows them around.

Standalone template — does not extend a parent. The full HTML document lives inside this single file.

Routes referenced via url_for(...). static. Each entry is a Flask endpoint name — search for the matching @app.route or @bp.route decorator in dashboard/app.py.

AJAX patterns detected. `fetch()`. The page is not purely server-rendered: parts of it talk back to Flask routes from the browser after the initial load.

Notes. 1 inline `<script>` block · 1 inline `<style>` block.

Opening of the template:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Client Management - Super Admin</title>
  <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">

  <style>
    /* --- Dark Theme Overrides (TradeOpss) --- */
    :root {
      --bg-dark: #0f172a;
      --card-bg: #1e293b;
      --text-primary: #f8fafc;
      --text-secondary: #94a3b8;
      --accent-color: #fbbf24;
      --border-color: rgba(255,255,255,0.1);
    }

    body {
      background-color: var(--bg-dark);
      color: var(--text-primary);
      font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif;
    }

    .container { max-width: 1200px; margin: 0 auto; padding: 20px; }

    a { color: var(--accent-color); text-decoration: none; }

    table { width: 100%; border-collapse: collapse; margin-top: 20px; background: var(
      --card-bg); border-radius: 8px; overflow: hidden; }
```

Trader-facing views

What a trader sees when they sign in. The trader-facing equivalent of the admin dashboard, scoped to their own clients and accounts.

dashboard/templates/trader_dashboard.html

973 loc · 49,503 bytes · 11 {{ ... }} expressions · 0 for loops · 3 if blocks

Trader's own dashboard view — their clients, their accounts, their performance summary. The trader-facing equivalent of admin_dashboard.html.

Standalone template — does not extend a parent. The full HTML document lives inside this single file.

Routes referenced via url_for(...). static. Each entry is a Flask endpoint name — search for the matching @app.route or @bp.route decorator in dashboard/app.py.

UI libraries detected. Font Awesome, Google Fonts. Loaded via either a CDN <script> tag or a local static asset.

AJAX patterns detected. fetch(). The page is not purely server-rendered: parts of it talk back to Flask routes from the browser after the initial load.

Notes. 1 inline <script> block · 1 inline <style> block.

Opening of the template:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Trader Portal - TradeOpss</title>
  <link href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.0.0/css/all.min.css" rel="stylesheet">
  <link href="https://fonts.googleapis.com/css2?family=Space+Grotesk:wght@400;500;600;700&family=IBM+Plex+Sans" rel="stylesheet">
  <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
  <style>
    /* ■■■ NAVBAR ■■■ */
    .modern-nav {
      background: #07091a;
      border: 1px solid #162040;
      border-top: 1px solid #1e3060;
      border-radius: 14px;
      padding: 0 24px;
      height: 68px;
      display: flex;
      justify-content: center;
      align-items: center;
      margin-bottom: 24px;
      box-shadow: 0 8px 28px rgba(1,3,10,0.8), 0 1px 0 rgba(30,58,138,0.22);
      position: sticky;
      top: 12px;
      z-index: 500;
    }
    .tab-btn {
      color: #7898be; text-decoration: none; font-weight: 600; font-size: 0.78rem;
      font-family: 'Plus Jakarta Sans', sans-serif;
```

dashboard/templates/trader_performance.html

418 loc · 22,962 bytes · 35 {{ ... }} expressions · 3 for loops · 9 if blocks

Per-trader performance numbers, comparable to client_performance but rolled up to the trader aggregation.

Standalone template — does not extend a parent. The full HTML document lives inside this single file.

Routes referenced via url_for(...). static. Each entry is a Flask endpoint name — search for the matching @app.route or @bp.route decorator in dashboard/app.py.

UI libraries detected. Font Awesome, Google Fonts. Loaded via either a CDN <script> tag or a local static asset.

Notes. 1 inline <script> block · 1 inline <style> block.

Opening of the template:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Trader Performance - TradeOpss</title>
  <!-- Use inline styles or copy from other templates if CSS file is generic -->
  <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
  <link href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.0.0/css/all.min.css" rel="stylesheet">
  <link href="https://fonts.googleapis.com/css2?family=Space+Grotesk:wght@400;500;600;700&family=IBM+Plex+Sans:ital,wght@0,400;0,500;0,600;0,700;1,400;1,500;1,600;1,700" rel="stylesheet">
  <style>
    /* ■■ TRADER PERFORMANCE — Style Guide Aligned ■■ */

    .positive, .success { color: #2dd4bf; font-weight: 600; }
    .negative, .danger { color: #f87171; font-weight: 600; }
    .neutral { color: #7898be; }
    .warnings { color: #f59e0b; font-weight: 500; }

    body { padding: 0 16px; }
    body::after {
      content: ''; position: fixed; bottom: 0; left: 0; right: 0; height: 72px; background: linear-gradient(to top, #f8d7da, #fff);
    }

    /* ■■ NAVBAR ■■ */
    .modern-nav {
      background: #07091a;
      border: 1px solid #162040;
      border-top: 1px solid #1e3060;
      border-radius: 14px;
      padding: 0 24px;
      height: 64px;
      display: flex;
    }
  </style>
</head>
<body>
```

Client-facing & financial views

The money pages: per-client breakdowns, the cross-org financial overview, and the payout audit log. The numbers shown here are what conversations with the trader and the prop firm hinge on.

dashboard/templates/client_performance.html

551 loc · 27,279 bytes · 5 {{ ... }} expressions · 0 for loops · 4 if blocks

Performance breakdown for a single client: accounts, payouts, hedging results, fees. Charts rendered client-side from JSON fed by the backend.

Standalone template — does not extend a parent. The full HTML document lives inside this single file.

Routes referenced via `url_for(...)`. static. Each entry is a Flask endpoint name — search for the matching `@app.route` or `@bp.route` decorator in `dashboard/app.py`.

UI libraries detected. Font Awesome, Google Fonts. Loaded via either a CDN `<script>` tag or a local static asset.

AJAX patterns detected. `fetch()`. The page is not purely server-rendered: parts of it talk back to Flask routes from the browser after the initial load.

Notes. 1 inline `<script>` block · 1 inline `<style>` block.

Opening of the template:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Client Performance - TradeOpss</title>
  <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
  <link href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.0.0/css/all.min.css" rel="stylesheet">
  <link href="https://fonts.googleapis.com/css2?family=SpaceGrotesk:wght@400;500;600;700&family=IBM+Plex+Sans:ital,wght@0,400;0,500;0,600;0,700;1,400;1,500;1,600;1,700" rel="stylesheet">
  <style>
    /* ■■■ CLIENT PERFORMANCE — Style Guide Aligned ■■■ */

    .positive { color: #2dd4bf; font-weight: 600; }
    .negative { color: #f87171; font-weight: 600; }
    .neutral { color: #7898be; }

    body { padding: 0 16px; }
    body::after {
      content: ''; position: fixed; bottom: 0; left: 0; right: 0; height: 72px; background: linear-gradient(to top, #f87171 49%, #f2f2f2 49% 51%, #f2f2f2 51% 53%, #2dd4bf 53%);
    }

    /* ■■■ NAVBAR ■■■ */
    .modern-nav {
      background: #07091a;
      border: 1px solid #162040;
      border-top: 1px solid #1e3060;
      border-radius: 14px;
      padding: 0 24px;
      height: 64px;
      display: flex;
      justify-content: center;
      align-items: center;
    }
  </style>
</head>
<body>
  <div class="modern-nav">
    <div class="nav-item">
      <a href="#accounts">Accounts</a>
    </div>
    <div class="nav-item">
      <a href="#payouts">Payouts</a>
    </div>
    <div class="nav-item">
      <a href="#hedging">Hedging</a>
    </div>
    <div class="nav-item">
      <a href="#fees">Fees</a>
    </div>
  </div>
  <div class="content">
    <div class="header">
      <h2>Client Performance</h2>
    </div>
    <div class="main">
      <div class="summary">
        <div class="summary-item">
          <div>Accounts</div>
          <div>12</div>
        </div>
        <div class="summary-item">
          <div>Payouts</div>
          <div>12</div>
        </div>
        <div class="summary-item">
          <div>Hedging</div>
          <div>12</div>
        </div>
        <div class="summary-item">
          <div>Fees</div>
          <div>12</div>
        </div>
      </div>
      <div class="charts">
        <div class="chart">
          <div>Accounts</div>
          <div>12</div>
        </div>
        <div class="chart">
          <div>Payouts</div>
          <div>12</div>
        </div>
        <div class="chart">
          <div>Hedging</div>
          <div>12</div>
        </div>
        <div class="chart">
          <div>Fees</div>
          <div>12</div>
        </div>
      </div>
    </div>
  </div>
</body>
</html>
```

dashboard/templates/financial_overview.html

1,177 loc · 64,296 bytes · 52 {{ ... }} expressions · 3 for loops · 14 if blocks

Cross-organisation money view: revenue, fees, payouts, P&L. The financial source of truth for the dashboard. Matches the calculations done in dashboard/financial_overview.py.

Standalone template — does not extend a parent. The full HTML document lives inside this single file.

Routes referenced via `url_for(...)`. financial_overview, static. Each entry is a Flask endpoint name — search for the matching `@app.route` or `@bp.route` decorator in dashboard/app.py.

Forms. GET → (self). The server route at the receiving end validates inputs and redirects on success.

UI libraries detected. Chart.js, Font Awesome, Google Fonts. Loaded via either a CDN `<script>` tag or a local static asset.

Notes. 13 inline `<script>` blocks · 1 inline `<style>` block · 1 external script.

External scripts loaded.

<https://cdn.jsdelivr.net/npm/chart.js>

Opening of the template:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Financial Overview - TradeOpss</title>
  <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
  <link href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.0.0/css/all.min.css" rel="stylesheet">
  <link href="https://fonts.googleapis.com/css2?family=Space+Grotesk:wght@400;500;600;700&family=IBM+Plex+Sans:ital,wght@0,400;0,500;0,600;0,700;1,400;1,500;1,600;1,700&display=block" rel="stylesheet">
  <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
  <style>
    /* ■■■ FINANCIAL OVERVIEW – Style Guide Aligned ■■■ */

    /* Value colors */
    .positive { color: #2dd4bf; font-weight: 600; }
    .negative { color: #f87171; font-weight: 600; }
    .neutral { color: #7898be; }
    .text-green-bright { color: #4ade80 !important; }
    .text-red-bright { color: #f87171 !important; }

    body { padding: 0 16px; }
    body::after {
      content: ''; position: fixed; bottom: 0; left: 0; right: 0; height: 72px; background: linear-gradient(to top, #f87171 49%, #f2f2f2 49% 51%, #f2f2f2 51% 53%, #2d3748 53%);
    }

    /* ■■■ NAVBAR ■■■ */
    .modern-nav {
      background: #07091a;
      border: 1px solid #162040;
      border-top: 1px solid #1e3060;
      border-radius: 14px;
      padding: 0 24px;
```

dashboard/templates/payout_history.html

554 loc · 27,968 bytes · 25 {{ ... }} expressions · 3 for loops · 12 if blocks

Auditable list of past payouts, filterable by client, account, and date. Drives the data shown alongside payouts.csv-style exports.

Standalone template — does not extend a parent. The full HTML document lives inside this single file.

Routes referenced via url_for(...). payout_history, static. Each entry is a Flask endpoint name — search for the matching @app.route or @bp.route decorator in dashboard/app.py.

Forms. GET → (self). The server route at the receiving end validates inputs and redirects on success.

UI libraries detected. Chart.js, Font Awesome, Google Fonts. Loaded via either a CDN <script> tag or a local static asset.

Notes. 4 inline <script> blocks · 1 inline <style> block · 1 external script.

External scripts loaded.

<https://cdn.jsdelivr.net/npm/chart.js>

Opening of the template:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Payout History - TradeOpss</title>
  <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
  <link href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.0.0/css/all.min.css" rel="stylesheet">
  <link href="https://fonts.googleapis.com/css2?family=Space+Grotesk:wght@400;500;600;700&family=IBM+Plex+Sans:ital,wght@0,400;0,500;0,600;0,700;1,400;1,500;1,600;1,700" rel="stylesheet">
  <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
  <style>
    /* ■■■ PAYOUT HISTORY – Style Guide Aligned ■■■ */

    /* Value colors */
    .positive { color: #2dd4bf; font-weight: 700 !important; }
    .negative { color: #f87171; font-weight: 500; }
    .neutral { color: #7898be; }
    .val-glow { text-shadow: 0 0 10px rgba(52,211,153,0.4); }
    .text-green-bright { color: #4ade80 !important; }
    .text-red-bright { color: #f87171 !important; }

    body { padding: 0 16px; }
    body::after {
      content: ''; position: fixed; bottom: 0; left: 0; right: 0; height: 72px; background: linear-gradient(to top, #f87171 49%, #f2f2f2 49% 51%, #f2f2f2 51% 53%, #f87171 53%);
    }

    /* ■■■ NAVBAR ■■■ */
    .modern-nav {
      background: #07091a;
      border: 1px solid #162040;
      border-top: 1px solid #1e3060;
      border-radius: 14px;
    }
  </style>
</head>
```


Quality & tooling

The data-quality dashboard. This is where the team finds out whether the ingestion pipelines actually loaded what they were supposed to load.

dashboard/templates/quality_dashboard.html

1,731 loc · 111,915 bytes · 3 {{ ... }} expressions · 0 for loops · 0 if blocks

Data-quality dashboard. Surfaces ingestion health, mismatches between sources, missing fields, and sync gaps so the team spots problems before clients do.

Standalone template — does not extend a parent. The full HTML document lives inside this single file.

Routes referenced via `url_for(...)`. static. Each entry is a Flask endpoint name — search for the matching `@app.route` or `@bp.route` decorator in `dashboard/app.py`.

UI libraries detected. Font Awesome, Google Fonts. Loaded via either a CDN `<script>` tag or a local static asset.

AJAX patterns detected. `fetch()`. The page is not purely server-rendered: parts of it talk back to Flask routes from the browser after the initial load.

Notes. 1 inline `<script>` block · 1 inline `<style>` block.

Opening of the template:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Data Quality Dashboard - VPF Futures</title>
  <link rel="icon" type="image/png" href="{{ url_for('static', filename='images/logo.png') }}">
  <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.cs
  <link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700;800&family=Space+Grot
  <style>
    * { margin: 0; padding: 0; box-sizing: border-box; }
    body {
      background: #01030a; color: #e2e8f0; font-family: 'Inter', 'Plus Jakarta Sans', -apple-system
    body::after {
      content: ''; position: fixed; bottom: 0; left: 0; right: 0; height: 72px; background: linear
  /* ■■■ NAVBAR ■■■ */
  .modern-nav {
    background: #07091a;
    border: 1px solid #162040;
    border-top: 1px solid #1e3060;
    border-radius: 14px;
    padding: 0 24px;
    height: 64px;
    display: flex;
    justify-content: center;
    align-items: center;
    margin-bottom: 20px;
    box-shadow: 0 8px 28px rgba(1,3,10,0.8), 0 1px 0 rgba(30,58,138,0.22);
    position: sticky;
    top: 0;
    z-index: 1000;
```


Other

The landing template. Despite the name, this file does not render the literal home URL — Flask routes pick which template to render based on the user's role. This is the fallback that holds the dashboard widgets shared across roles.

dashboard/templates/index.html

8,748 loc · 477,009 bytes · 22 {{ ... }} expressions · 0 for loops · 18 if blocks

The home/landing page after login. The largest template in the project — embeds the live dashboard widgets, charts, and most of the client-facing data tables. If you only read one template, read this one: it is where the 'shape' of the dashboard lives.

Standalone template — does not extend a parent. The full HTML document lives inside this single file.

Routes referenced via `url_for(...)`. static. Each entry is a Flask endpoint name — search for the matching `@app.route` or `@bp.route` decorator in `dashboard/app.py`.

UI libraries detected. Font Awesome, Google Fonts. Loaded via either a CDN `<script>` tag or a local static asset.

AJAX patterns detected. `fetch()`. The page is not purely server-rendered: parts of it talk back to Flask routes from the browser after the initial load.

Notes. 1 inline `<script>` block · 2 inline `<style>` blocks.

Opening of the template:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Client Dashboard - TradeOpss</title>
  <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
  <link rel="icon" href="{{ url_for('static', filename='images/logo.png') }}" type="image/png">
  <link href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.0.0/css/all.min.css" rel="stylesheet">
  <link href="https://fonts.googleapis.com/css2?family=Space+Grotesk:wght@400;500;600;700&family=IBM+Plex+Sans:ital,wght@0,normal;0,400;0,500;0,600;0,700;1,normal;1,400;1,500;1,600;1,700&display=block" rel="stylesheet">
  <link href="https://cdnjs.cloudflare.com/ajax/libs/flag-icon-css/3.5.0/css/flag-icon.min.css" rel="stylesheet">
  <style>
    /* Version History Sidebar Styles - Google Sheets Style */

    html {
      scrollbar-gutter: stable;
    }
    .history-panel {
      position: fixed;
      top: 0;
      right: -400px;
      width: 400px;
      height: 100vh;
      background: var(--card-bg, #1e293b);
      color: var(--text-primary, #f8fafc);
      box-shadow: -4px 0 20px rgba(0,0,0,0.5);
      z-index: 20000;
      transition: right 0.3s ease;
      display: flex;
```

```
flex-direction: column;
```

The CSS layer

All custom styling lives in `dashboard/static/css/`. There is no preprocessor (Sass, Less) and no PostCSS pipeline — this is plain CSS, served as-is by Flask's static handler. That keeps the deploy story simple at the cost of a slightly longer single file.

`dashboard/static/css/style.css`

1,655 loc · 46,661 bytes · ~207 rules · 0 @media queries · 3 @keyframes animations · 32 CSS custom properties

A single concatenated stylesheet shared across every template. The inline `<style>` blocks in individual templates layer on top for page-specific tweaks. The 32 CSS custom properties (the `--name: value` declarations) are how the dashboard themes itself: change one variable at the `:root` level and every rule that references it updates.

Static HTML (no Flask, no Jinja)

A few HTML files live under `dashboard/static/` directly. These are served as plain files — Flask never templates them. Useful for pages that must render even when the Flask app itself is unreachable.

- `dashboard/static/maintenance.html`

How a request actually flows

Worth tracing in your head once. A user clicks a link to `/clients/42`. Unicorn (production) or Flask's dev server (local) accepts the TCP connection and hands it to `wsgi.py`, which exposes the Flask app object built in `dashboard/app.py`. Flask matches the URL against its registered routes, finds the handler, and calls it. The handler queries the database via SQLAlchemy, builds a context dictionary, and returns `render_template('client_performance.html', **context)`. Jinja loads the template, walks the `{% extends %}` chain, runs every `{{ expression }}` against the context, and produces the final HTML string. Flask wraps it in a 200 response and ships it back. The browser loads the inline JavaScript, which fires its own `fetch()` calls back to other Flask routes for the live charts. That last step is the only place where the two halves of the app meet asynchronously.

Chapter 14 — Glossary & concept index

Two reference tools. The glossary defines terms specific to this codebase — the trading domain it lives in, plus a few Python and framework names worth pinning down. The concept index maps every Python feature detected by the AST scanner to the modules that actually use it, so you can jump straight from a concept to a concrete working example.

Glossary

MT5 / MetaTrader 5

The retail-trader desktop platform from MetaQuotes. Exposes a Python API (the `MetaTrader5` package) that this codebase uses for order placement and history retrieval.

Prop firm

A proprietary trading firm that funds traders against an evaluation. Topstep, FundedNext, Tradovate, and TradeOpss are the four this codebase integrates with.

Funded account

An account a prop firm has cleared the trader to trade real capital on, after passing the evaluation challenge.

Hedging

Holding offsetting positions across linked accounts so that drawdown on one is bounded by gains on the other. The `hedge_protector` module implements this.

Watermark / high-water mark

The peak equity an account has reached. Payouts are calculated on profit above this watermark to prevent paying twice on the same gain.

Phase

An account's evaluation stage: challenge, verification, or funded. `phase_manager.py` owns the state machine.

Flask

Python micro-web-framework powering the dashboard. Routes are registered with the `@app.route` decorator.

SQLAlchemy

Python ORM. Database tables are declared as Python classes under `dashboard/models.py`.

Alembic

Migration framework that pairs with SQLAlchemy. Each file in `alembic/versions/` is a forward + rollback step.

APScheduler

Cron-style job scheduler used by dashboard/scheduler.py to run periodic resyncs.

Selenium / CDP

Browser-automation tooling. The prop-firm scrapers either drive a Chrome instance via Selenium or attach via the Chrome DevTools Protocol to read network traffic.

Tkinter

The standard-library GUI toolkit Python ships with. The trader companion's desktop window is built with it.

Gunicorn

Production WSGI server. Loads wsgi.py and runs the Flask app behind a worker pool.

Concept index

Each Python language feature, with the modules that use it. Concepts that appear in many modules are listed first; rarer ones at the end.

Exceptions (48 modules)

build.py, wsgi.py, trader_companion/fundednext.py, trader_companion/hedge_protector.py, trader_companion/mt5_comment_parser.py, trader_companion/mt5_dashboard_sync.py, trader_companion/mt5_trading.py, trader_companion/prop_firm_scrapers.py, trader_companion/push_data.py, trader_companion/signals/adx.py, trader_companion/signals/atr.py, trader_companion/signals/bb.py, trader_companion/signals/ccl.py, trader_companion/signals/dmi.py, trader_companion/signals/ema.py, trader_companion/signals/macd.py, trader_companion/signals/mfi.py, trader_companion/signals/momentum.py ... and 30 more

Comprehensions (40 modules)

wsgi.py, trader_companion/broker_selection.py, trader_companion/fundednext.py, trader_companion/mt5_dashboard_sync.py, trader_companion/mt5_trading.py, trader_companion/prop_firm_manager.py, trader_companion/prop_firm_scrapers.py, trader_companion/signals/adx.py, trader_companion/signals/atr.py, trader_companion/signals/bb.py, trader_companion/signals/ccl.py, trader_companion/signals/dmi.py, trader_companion/signals/ema.py, trader_companion/signals/macd.py, trader_companion/signals/mfi.py, trader_companion/signals/momentum.py, trader_companion/signals/obv.py, trader_companion/signals/roc.py ... and 22 more

f-strings (34 modules)

build.py, gunicorn.conf.py, wsgi.py, trader_companion/fundednext.py, trader_companion/hedge_protector.py, trader_companion/mt5_comment_parser.py, trader_companion/mt5_dashboard_sync.py, trader_companion/mt5_trading.py, trader_companion/prop_firm_manager.py, trader_companion/prop_firm_scrapers.py, trader_companion/push_data.py, trader_companion/signals/rsi.py, trader_companion/topstepx.py, trader_companion/trade_limit_manager.py, trader_companion/trader_app.py, trader_companion/tradovate.py, dashboard/api_client.py, dashboard/app.py ... and 16 more

Type hints (24 modules)

trader_companion/broker_selection.py, trader_companion/mt5_comment_parser.py,
trader_companion/prop_firm_manager.py, trader_companion/signals/cmo.py,
trader_companion/signals/coppock_curve.py, trader_companion/signals/donchian_channel.py,
trader_companion/signals/elder_ray.py, trader_companion/signals/fractal.py,
trader_companion/signals/gator_oscillator.py, trader_companion/signals/keltner_channel.py,
trader_companion/signals/price_channel.py, trader_companion/signals/rsi.py,
trader_companion/signals/ultimate_oscillator.py, trader_companion/signals/vortex.py,
trader_companion/trade_limit_manager.py, trader_companion/trader_app.py, dashboard/api_client.py,
dashboard/app.py ... and 6 more

Classes and objects (22 modules)

trader_companion/broker_selection.py, trader_companion/fundednext.py,
trader_companion/hedge_protector.py, trader_companion/mt5_comment_parser.py,
trader_companion/mt5_dashboard_sync.py, trader_companion/mt5_trading.py,
trader_companion/prop_firm_manager.py, trader_companion/prop_firm_scrapers.py,
trader_companion/topstepx.py, trader_companion/trade_limit_manager.py,
trader_companion/trader_app.py, trader_companion/tradovate.py, dashboard/api_client.py,
dashboard/app.py, dashboard/database.py, dashboard/financial_overview.py, dashboard/models.py,
dashboard/phase_manager.py ... and 4 more

Context managers (with-statements) (20 modules)

build.py, wsgi.py, trader_companion/fundednext.py, trader_companion/mt5_trading.py,
trader_companion/prop_firm_scrapers.py, trader_companion/topstepx.py,
trader_companion/trade_limit_manager.py, trader_companion/trader_app.py,
trader_companion/tradovate.py, dashboard/app.py, dashboard/database.py,
dashboard/email_service.py, dashboard/notes_service.py, dashboard/phase_manager.py,
dashboard/scheduler.py, dashboard/watermark_service.py, connectors/mt5_automator.py,
config/hierarchy.py ... and 2 more

Dunder methods (18 modules)

trader_companion/broker_selection.py, trader_companion/fundednext.py,
trader_companion/hedge_protector.py, trader_companion/mt5_comment_parser.py,
trader_companion/mt5_dashboard_sync.py, trader_companion/mt5_trading.py,
trader_companion/prop_firm_manager.py, trader_companion/prop_firm_scrapers.py,
trader_companion/topstepx.py, trader_companion/trade_limit_manager.py,
trader_companion/trader_app.py, trader_companion/tradovate.py, dashboard/api_client.py,
dashboard/database.py, dashboard/financial_overview.py, dashboard/utils/trade_matcher.py,
connectors/mt5_automator.py, connectors/mt5_connector.py

Lambdas (13 modules)

trader_companion/hedge_protector.py, trader_companion/mt5_trading.py,
trader_companion/trade_limit_manager.py, trader_companion/trader_app.py,
trader_companion/tradovate.py, dashboard/app.py, dashboard/financial_overview.py,
dashboard/scheduler.py, dashboard/watermark_service.py, dashboard/utils/sheet_helper.py,
dashboard/utils/trade_matcher.py, connectors/mt5_automator.py, utils/data_processor.py

Decorators (12 modules)

wsgi.py, trader_companion/broker_selection.py, trader_companion/fundednext.py,
trader_companion/hedge_protector.py, trader_companion/mt5_comment_parser.py,
trader_companion/topstepx.py, trader_companion/trader_app.py, dashboard/app.py,
dashboard/database.py, dashboard/financial_overview.py, dashboard/phase_manager.py,
config/production.py

Module-level state (7 modules)

trader_companion/mt5_dashboard_sync.py, trader_companion/prop_firm_scrapers.py,
dashboard/database.py, dashboard/financial_overview.py, dashboard/scheduler.py, config/hierarchy.py,
utils/data_processor.py

Inheritance (5 modules)

trader_companion/mt5_comment_parser.py, trader_companion/prop_firm_scrapers.py,
dashboard/app.py, dashboard/models.py, config/production.py

Properties (5 modules)

trader_companion/hedge_protector.py, trader_companion/mt5_comment_parser.py,
trader_companion/topstepx.py, dashboard/database.py, config/production.py

@classmethod and @staticmethod (3 modules)

trader_companion/broker_selection.py, trader_companion/trader_app.py, dashboard/phase_manager.py

Flask routes (2 modules)

wsgi.py, dashboard/app.py

Generators and yield (2 modules)

dashboard/database.py, dashboard/db.py

Closures (1 module)

dashboard/app.py

Dataclasses (1 module)

trader_companion/mt5_comment_parser.py

Enums (1 module)

trader_companion/mt5_comment_parser.py

Walrus operator (:=) (1 module)

trader_companion/trader_app.py